Szoftverarchitektúrák és architekturális tervezés

A követelményektől a produkciós rendszerekig – döntések, minták és esettanulmányok

Gellai István

2026

Tartalomjegyzék

Impresszum **7**

I. RÉSZ – Alapok, elvek, döntések és dokumentáció **8**

1 Előszó és olvasói útmutató **9**
1.1 Előszó: a szerző, a könyv célja és célközönsége 9
1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések 14

2 Mi a szoftverarchitektúra? **21**
2.1 Definíciók és nézőpontok: struktúra, döntés és társadalmi konstrukció 21
2.2 Miért fontos? Az architektúra szerepe a sikerben és a kudarcban 26
2.3 A szoftverarchitektúra története és evolúciója 35
2.4 Alapfogalmak: komponens, kapcsolat, nézet, stakeholder és rendszerhatár 42

3 Architektúra elvek **52**
3.1 SOLID elvek architektúra szinten 52
3.2 DRY, KISS, YAGNI és a túltervezés csapdái 58
3.3 Modularitás, kapcsolódás és kohézió 64
3.4 Elvek ütközése: trade-offok és kontextus 73

4 Architektúra mint döntéshozatal **83**
4.1 Architektúra döntések és következményeik 83
4.2 Quality attributes és trade-off analízis 89
4.3 Architecture Decision Records (ADR) és döntési napló 99
4.4 Gazdasági és szervezeti döntési szempontok 107

5 Architektúra az életciklusban **116**
5.1 SDLC, vízesés, Agile és folyamatos architektúra 116
5.2 Scrum, Kanban, Lean és az architektúra szerepe 122
5.3 Architektúra governance és döntési jogkörök 130
5.4 Architektúra mint folyamatos tevékenység 137

6 Dokumentálás és kommunikáció **144**
6.1 Nézetek és a 4+1 modell 144
6.2 C4 modell és arc42 153
6.3 UML, ER és diagrams-as-code 164
6.4 ADR-ek, RFC-k és stakeholder-kommunikáció 173

II. RÉSZ – Követelmények, minőség és tervezés **185**

7 Követelményanalízis architekt szemszögből **186**
7.1 Funkcionális követelmények és rendszerhatárok 186
7.2 Use case-ek, user story-k és architektúra 191
7.3 Nem funkcionális követelmények és kényszerek 199
7.4 KPI-ok, mérőszámok és követelmény-kézbesítés 208

8 Nem funkcionális követelmények és minőségi attribútumok 217
8.1 Teljesítmény és skálázhatóság . . . 217
8.2 Rendelkezésre állás, megbízhatóság és hibatűrés . . . 225
8.3 Biztonság, adatvédelem és megfelelés . . . 233
8.4 Karbantarthatóság, használhatóság és üzemeltethetőség . . . 241

9 Minőségi forgatókönyvek és architektúraértékelés 250
9.1 Quality attribute scenarios . . . 250
9.2 SLO, SLI, error budget és mérés . . . 259
9.3 ATAM, CBAM és architektúra-review . . . 267
9.4 Fitness functions és folyamatos értékelés . . . 279

10 Tervezési folyamat és döntési keretek 287
10.1 A tervezési folyamat: feltárás, alternatívák és döntés . . . 287
10.2 Proof of Concept: célszerűség és értékelés . . . 294
10.3 MVP és spike: tanulás a bizonytalanságban . . . 302
10.4 Mintaválasztás és döntési mátrixok . . . 310

III. RÉSZ – Architektúrális stílusok és minták 321

11 Rétegezett, monolitikus és moduláris architektúrák 322
11.1 Rétegezett architektúra és határai . . . 322
11.2 Monolit, moduláris monolit és mikrolit . . . 326
11.3 Csillag- és mikro-kernell architektúra . . . 331
11.4 UI-architektúrák: MVC, MVP és MVVM . . . 343

12 Mikroszolgáltatások és elosztott rendszerek 357
12.1 A mikroszolgáltatások ígérete és ára . . . 357
12.2 Szolgáltatáshatárok és domain-dekompozíció . . . 364
12.3 Elosztott rendszerek alapjai: CAP, konzisztencia és hiba . . . 372
12.4 Mikroszolgáltatások anti-minták és szervezeti feltételek . . . 381

13 Eseményvezérelt és streaming architektúrák 391
13.1 Esemény, üzenet és parancs . . . 391
13.2 CQRS és Event Sourcing . . . 401
13.3 Saga, outbox és elosztott tranzakciók . . . 409
13.4 Streaming platformok és event-driven minták . . . 415

14 Integráció és API-tervezés 426
14.1 REST, GraphQL és gRPC . . . 426
14.2 API-verziózás, szerződés és schema evolution . . . 432
14.3 Enterprise Integration Patterns és message brokering . . . 442
14.4 API gateway, BFF és service mesh . . . 450

15 Enterprise és domain-architektúrák 460
15.1 Domain-Driven Design: bounded context és ubiquitous language . . . 460
15.2 SOA, ESB és enterprise integráció . . . 468
15.3 Microservices és domain-architektúra . . . 477
15.4 Data mesh és domain-oriented adatarchitektúra . . . 482

IV. RÉSZ – Adat, platform és cloud 492

16 Adatarchitektúra és perzisztencia **493**
16.1 Adatmodellek, polyglot persistence és tranzakciók . . . 493
16.2 Elosztott adatkonzisztencia és replikáció . . . 500
16.3 Data lake, lakehouse és analitikai architektúra . . . 506
16.4 Adatminőség, adatgovernance és adatkatalógus . . . 515

17 Cloud-native és konténerarchitektúrák **524**
17.1 Cloud-native alapelvek és 12-factor . . . 524
17.2 Konténerek, Kubernetes és orchestráció . . . 531
17.3 Serverless, FaaS és managed szolgáltatások . . . 539
17.4 Multi-cloud, hybrid cloud és lock-in . . . 546

18 Platform engineering és belső fejlesztői platform **555**
18.1 Platform mint termék és golden path . . . 555
18.2 Team Topologies és szervezeti architektúra . . . 561
18.3 Belső fejlesztői platform és önkiszolgálás . . . 568
18.4 GitOps, környezetek és konfigurációkezelés . . . 575

19 Skálázhatóság, teljesítmény és költség **588**
19.1 Horizontális és vertikális skálázás . . . 588
19.2 Cache, CDN és aszinkron feldolgozás . . . 594
19.3 Terhelés, backpressure és kapacitástervezés . . . 604
19.4 FinOps, energia és zöld szoftver . . . 612

V. RÉSZ – Megbízhatóság, biztonság és üzemeltetés **622**

20 Reziliencia és hibamódok **623**
20.1 Hibamódok és failure mode analysis . . . 623
20.2 Circuit breaker, bulkhead, retry és timeout . . . 630
20.3 Chaos engineering és játéknapok . . . 639
20.4 SLO, error budget és incidenskezelés . . . 646

21 Biztonsági architektúra **656**
21.1 Threat modeling és támadási felület . . . 656
21.2 Zero trust, identitás és hozzáférés . . . 663
21.3 Titkok, kulcsok és adatvédelem . . . 670
21.4 Ellátási lánc biztonsága: SBOM, SLSA és aláírás . . . 678

22 Adatvédelem, megfelelés és etika **689**
22.1 Privacy by design és GDPR . . . 689
22.2 Ágazati szabályozás: NIS2, DORA és AI Act . . . 697
22.3 Auditálhatóság, naplózás és adatmegőrzés . . . 704
22.4 Etika, fairness és társadalmi hatás . . . 711

23 Observability és üzemeltetés **718**
23.1 Logs, metrics, traces és OpenTelemetry . . . 718
23.2 Dashboardok, riasztások és runbookok . . . 726
23.3 Incident response és postmortem . . . 734
23.4 Költség- és kapacitásmonitoring . . . 743

VI. RÉSZ – Delivery, evolúció és szervezet **751**

24 DevOps, CI/CD és deployment stratégiák **752**
24.1 CI/CD alapelvek és pipeline-tervezés . . . 752
24.2 Infrastruktúra mint kód és konfiguráció . . . 760
24.3 Blue-green, canary, rolling és feature flag . . . 766
24.4 Release engineering és rollback . . . 773

25 Tesztelési stratégia és minőségbiztosítás **783**
25.1 Tesztpiramis, teszt-trofea és QA szerep . . . 783
25.2 Unit, integrációs, contract és E2E tesztek . . . 788
25.3 Statikus elemzés, kódminőség és mutation testing . . . 797
25.4 Teljesítmény-, biztonsági és chaos tesztelés . . . 806

26 Refaktorálás, technikai adósság és legacy modernizáció **815**
26.1 Technikai adósság és kódminőség . . . 815
26.2 Refaktorálás nagy rendszerekben . . . 820
26.3 Legacy modernizáció és strangler fig . . . 828
26.4 Migrációs stratégiák: 6R és fokozatos átállás . . . 835

27 Evolúciós architektúra és governance **845**
27.1 Evolúciós architektúra és fitness functions . . . 845
27.2 Technológiai roadmap és iterációs tervezés . . . 853
27.3 Architektúra governance és szabványok . . . 860
27.4 Mérés, döntések és visszacsatolás . . . 869

28 Szervezet, kultúra és vezetés **876**
28.1 Conway törvénye és szervezeti illeszkedés . . . 876
28.2 Stakeholder management és kommunikáció . . . 882
28.3 Architekt szerep, karrier és mentorálás . . . 890
28.4 Kultúra, pszichológiai biztonság és döntéshozatal . . . 898

VII. RÉSZ – Esettanulmányok, iparágak, jövő és záróprojekt **905**

29 Esettanulmányok és architektúra-katák **906**
29.1 Esettanulmány-elemzés módszertana . . . 906
29.2 Valós rendszerek architektúrái és tanulságai . . . 917
29.3 Architektúra-katák és döntési gyakorlatok . . . 923
29.4 Best practice-ek és anti-minták . . . 932

30 Iparági architektúrák **941**
30.1 Pénzügy és biztosítás . . . 941
30.2 Egészségügy és elektronikus egészségügyi nyilvántartás . . . 947
30.3 Kiskereskedelem és e-kereskedelem . . . 952
30.4 Gyártás, logisztika és supply chain . . . 961

31 Modern trendek és fejlett architektúrák **973**
31.1 AI/ML és LLM-rendszerek architektúrája . . . 973
31.2 Edge, fog computing és IoT/CPS . . . 979
31.3 Blockchain, decentralizált rendszerek és VR/AR . . . 988
31.4 Quantum computing és a következő évtized . . . 997

32 Záróprojekt: end-to-end architektúra **1005**
32.1 Követelmények, kényszerek és minőségi forgatókönyvek . . . 1005
32.2 Architektúra-alternatívák és döntési dokumentáció . . . 1013

32.3 Proof of Concept és mérés 1024
32.4 Architektúra-review, kockázat és bemutató 1035

33 Kiadási checklist és továbbtanulás **1046**
33.1 Architektúra-review checklist 1046
33.2 Dokumentációs és megfelelési csomag 1054
33.3 Ismert korlátok és nyitott kérdések 1064
33.4 Tanulási útvonal, könyvek és közösségek 1071

Függelékek **1078**
Fogalomtár (magyar–angol) 1078
Képlet- és mérőszámgyűjtemény 1084
Kompetenciatérkép és tanulási útvonal 1088
Bibliográfia és forráskulcsok 1091

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026
- **Licenc:** MIT
- **Forráskód:** chapters/ és lab/

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

I. RÉSZ – Alapok, elvek, döntések és dokumentáció

1 Előszó és olvasói útmutató

1.1 Előszó: a szerző, a könyv célja és célközönsége

Amikor először találkoztam egy olyan rendszerrel, amelynek architektúráját egyetlen diagram sem tudta visszaadni, rájöttem, hogy a szoftverarchitektúra több, mint rajz és technológiaválasztás. Ez a felismerés azóta is meghatározza a munkámat, és ezért született meg ez a könyv is.

1.1.1 A szerző szakmai háttere

A szerző több mint tizenöt éve dolgozik szoftverfejlesztőként, rendszertervezőként és vállalati architektként. Pályafutása során kisebb startupoktól nagyvállalati platformokig számos környezetben vett részt rendszerek tervezésében, fejlesztésében és üzemeltetésében. Dolgozott pénzügyi, egészségügyi, kiskereskedelmi és logisztikai projekteken, ahol rendre azzal szembesült, hogy a legnehezebb kérdések nem a programozási nyelvek szintaxisára vagy a keretrendszerek API-jaira vonatkoznak, hanem arra, hogy adott üzleti és technológiai feltételek között melyik megoldás vezet fenntartható eredményre.

Ez a gyakorlati tapasztalat alakította ki a szerző meggyőződését: az architektúra nem statikus tervrajz, hanem folyamatos döntéshozatal, kommunikáció és kompromisszumkeresés. A szerző oktatóként és mentorként is tevékenykedik, és számos csapatnak segített abban, hogy a technológiai döntéseket a minőségi követelményekkel, a költségekkel és a szervezeti valósággal összhangban hozzák meg. A könyvben szereplő példák és gyakorlatok ebből a tapasztalatból merítenek, de a szereplők, cégek és rendszerek mindvégig fiktívek vagy anonimizáltak, hogy a hangsúly az általánosítható tanulságokon maradjon.

1.1.2 A könyv motivációja

A szoftverarchitektúra szakirodalma gyakran két véglet között ingadozik. Egyes művek az elméleti alapokra összpontosítanak, és nehezen kötik össze a fogalmakat a mindennapi mérnöki döntésekkel. Más források technológiai útmutatók, amelyek egy adott eszköz vagy platform használatát mutatják be, de nem adnak választ arra, hogy mikor és miért válasszuk azt a megoldást. A gyakorló szakemberek számára a legnagyobb kihívás általában az, hogy a konkrét követelmények, csapatadottságok, szabályozási feltételek és költségkeretek között kell meghozniuk a döntést, miközben a rendszer a jövőben is fejlődni fog.

Ezt a hiányt igyekszik pótolni ez a könyv. A szoftverarchitektúrát olyan gyakorlati tevékenységként mutatja be, amelynek középpontjában a döntések, a trade-offok és az ellenőrizhető értékelés állnak. A cél nem az, hogy megtanítsa az olvasót egyetlen „helyes” architektúrára, hanem hogy képessé tegye arra, hogy a saját kontextusában jó döntéseket hozzon, dokumentálja azokat, és a következményeket mérje. Ennek érdekében a könyv a fogalmakat esettanulmányokkal, döntési naplókkal és laborokkal építi fel, és minden állítást a mögöttes elvvel, valamint a kivételekkel együtt tárgyal.

Különös hangsúlyt kap a mérés és az értékelés. Az architektúrával kapcsolatos állítások gyakran nem ellenőrizhetők, mert nem rögzítjük, milyen környezetben, milyen terhelés alatt és milyen időablakban mértünk. Ez a könyv ezért ragaszkodik ahhoz, hogy a minőségi követelményeket mérhető forgatókönyvekben fogalmazzuk meg, a rendelkezésre állást és az error budget-et képletekkel definiáljuk, a laborokban pedig a mért és a várt eredményt egyértelműen elkülönítsük. A szabályok nem öncélúak: az olvasó így olyan dokumentációt és gyakorlatot alakíthat ki, amely a valódi rendszerek üzemeltetése során is megállja a helyét.

1.1.3 Célközönség

A könyv olyan szakembereknek készült, akik architektúrális döntéseket hoznak, azokat előkészítik vagy értékelik. Nem feltételez korábbi architektúra-tanulmányokat, de programozói rutint, verziókezelési és alapvető hálózati ismereteket igen. A célcsoportot öt nagyobb csoportra bonthatjuk:

- **Programozók és szoftverfejlesztők**, akik szeretnék megérteni, hogyan illeszkedik a napi munkájuk a nagyobb rendszerképbe, és hogyan hozhatnak jobb tervezési döntéseket a kód szintje felett is.
- **Tech leadek és csapatvezetők**, akiknek a felelőssége a csapat technikai irányának kialakítása, a fejlesztési folyamatok támogatása és az architekturális egyeztetések lefolytatása.
- **Rendszertervezők és architekt jelölt szakemberek**, akik a rendszerhatárok, szolgáltatásdekompozíció és integrációs minták tervezésében vesznek részt.
- **Platform- és üzemeltetési mérnökök**, akik a rendszerek produkciós működéséért, megbízhatóságáért, megfigyelhetőségéért és skálázhatóságáért felelősek, és akiknek a fejlesztőkkel közös nyelvet kell beszélniük.
- **Termékmenedzserek és döntéshozók**, akik a technológiai döntések üzleti, költségvetési és szabályozási következményeit szeretnék jobban megérteni, hogy megalapozottan rangsorolhassanak.

Mindegyik csoport számára közös, hogy a szoftverarchitektúra nem elszigetelt műszaki kérdés, hanem olyan tevékenység, amely összeköti az üzleti célokat, a minőségi követelményeket, a csapatszerkezetet és a technológiai lehetőségeket. A könyv ezért nem csak technológiai megoldásokat mutat be, hanem a döntések szervezeti, gazdasági és etikai dimenzióit is tárgyalja.

1.1.4 A könyv három olvasói útja

A könyv moduláris felépítésű, így az olvasó a saját szerepéhez és tanulási céljaihoz igazíthatja az olvasási sorrendet. Három fő útvonalat javasolunk:

1. **Alapozó út** – Azoknak, akik most ismerkednek a szoftverarchitektúrával, vagy szeretnék a meglévő tudásukat rendszerbe foglalni. Ez az út az I. rész elejétől halad végig a II. részen, majd a III. rész alapvető stílusain át a IV. és V. rész fontosabb fejezeteihez vezet. A hangsúly a fogalmak tisztázásán, a minőségi követelmények mérhető megfogalmazásán és a leggyakoribb architektúrális minták kontextusba helyezésén van.

2. **Gyakorló út** – Olyan fejlesztőknek és rendszertervezőknek, akik már dolgoznak elosztott rendszereken, és konkrét döntési helyzetekre keresnek megoldást. Ők a II. részt gyorsabban átfutva a III. rész (stílusok, integráció, domain-architektúra), majd a IV. rész (adat, platform, cloud) és az V. rész (reziliencia, biztonság, observability) fejezeteire összpontosíthatnak. A laborok ezen az úton hangsúlyosabbak, mert a cél a döntési kompetencia elmélyítése.

3. **Vezetői és értékelői út** – Tech leadek, termékmenedzserek és architektúra-review-t végző szakemberek számára. Ez az út az I. részből a döntéshozatal és a dokumentáció fejezeteit, a II. részből a minőségi forgatókönyvek és az SLO/error budget témákat, a IV. részből a költség- és kapacitástervezést, a VI. részből pedig az evolúciós architektúrát és a szervezeti illeszkedést emeli ki. A cél annak megértése, hogyan lehet az architektúrális döntéseket üzleti és szervezeti szempontból is értékelni.

Mindhárom út közös jellemzője, hogy az olvasó nem passzívan követi a szöveget, hanem döntési helyzetekben gyakorol. A fejezetek végén található ellenőrzőlisták és önellenőrző kérdések segítenek abban, hogy a tanultakat a saját kontextusára is alkalmazza.

1.1.5 A szoftverarchitektúra mint gyakorlati és szervezeti szerep

Ez a könyv abból a feltevésből indul ki, hogy a szoftverarchitektúra nem kizárólag technológiai tevékenység. Az architekt feladata nem az, hogy a „tökéletes” tervrajzot elkészítse, hanem hogy a rendszerrel szemben támasztott követelményeket, a rendelkezésre álló erőforrásokat és a szervezeti feltételeket egyensúlyba hozza. A gyakorlatban ez gyakran azt jelenti, hogy dokumentálja a döntések mögötti érveket, hogy a későbbi generációk is megértsék, miért választottak egy adott megoldást, és milyen alternatívákat adtak fel.

A gyakorlati szerep része a mérés és az ellenőrzés is. Egy architektúra akkor tekinthető sikeresnek, ha a minőségi követelményeket – például a válaszidőt, a rendelkezésre állást, a biztonsági tulajdonságokat – rendszeresen mérjük, és az eredményeket visszacsatoljuk a tervezésbe. Ezért a könyvben külön fejezetek foglalkoznak a quality attribute scenario-kkal, a SLO-kkal, a fitness function-ökkel és az architektúra-review-vel. A cél, hogy az architektúra ne csak szándéknyilatkozat legyen, hanem folyamatosan ellenőrizhető döntéshalmaz.

A szervezeti szerep legalább ennyire fontos. A Conway-törvény értelmében a szervezetek olyan rendszereket terveznek, amelyek a belső kommunikációs struktúrájukat tükrözik. Ezért az architektúra kialakítása nem választható el a csapatok felépítésétől, a döntési jogköröktől és a felelősségi viszonyoktól. A könyv bemutatja, hogyan illeszthetők a technológiai határok a domain- és csapathatárokhoz, hogyan alakítható ki olyan governance, amely nem fojtja meg a csapatok autonómiáját, és hogyan kezelhetők a stakeholder-elvárások. Az architekt szerepe így gyakran facilitátori: segít a közös nyelv kialakításában, a döntések láthatóvá tételében és a konfliktusok konstruktív kezelésében.

Ez a szemlélet azért hangsúlyos, mert a valós rendszerek ritkán buknak meg kizárólag technológiai okból. A gyakoribb hiba, hogy nem tisztázták a követelményeket, nem dokumentálták a döntéseket, vagy a szervezeti akadályok miatt a csapatok nem tudták követni a tervezett irányt. A könyv ezért minden fejezetben igyekszik a technológiai mintákat a szervezeti kontextussal együtt tárgyalni.

1.1.6 Hogyan használd ezt a fejezetet?

Az Előszó és olvasói útmutató két alfejezetből áll. Az első (jelen fejezet) bemutatja a szerzőt, a könyv célját és a célközönséget, valamint felvázolja a három olvasói utat és az architektúra gyakorlati, szervezeti szerepét. A második alfejezet (Hogyan használd a könyvet: tanulási utak, laborok és jelölések) a könyvben használt jelöléseket, tanulási utakat és laborokat ismerteti részletesen. Ha még nem ismered a könyv konvencióit, érdemes ezt a két fejezetet sorban elolvasni, mert a későbbi fejezetek építenek rájuk.

A jelen fejezet végén található labor segítségével meghatározhatod a saját tanulási célodat és kiindulási szintedet. Ez fontos első lépés, mert a könyv moduláris szerkezete lehetővé teszi, hogy a számodra legrelevánsabb részekre összpontosíts. A labor elvégzése nélkül is használhatod a könyvet, de azt javasoljuk, hogy szánj rá negyvenöt–hatvan percet, mert az eredmény végigkíséri a tanulási folyamatodat.

1.1.7 Labor: Személyes tanulási cél és kiindulási szint meghatározása

Ez a labor nem technológiai feladat, hanem önreflexiós gyakorlat. Célja, hogy az olvasó tisztában legyen a saját kontextusával, tanulási céljával és azzal, hogy a könyv mely részei a legfontosabbak számára. A labor reprodukálható dokumentumban vagy digitális jegyzetfüzetben végezhető el.

1.1.7.1 Kiinduló állapot

- Rendelkezésre áll egy üres dokumentum (papíron vagy elektronikusan).
- Ismered a jelenlegi szerepedet, alapvető tapasztalataidat és a technológiákat, amelyekkel dolgozol.
- Nincs szükség speciális szoftverre; elegendő egy szövegszerkesztő vagy jegyzetfüzet.

1.1.7.2 Elvárt eredmény

Egy legfeljebb kétoldalas dokumentum, amely az alábbi elemeket tartalmazza:

- **Kontextus:** szerep, szakmai tapasztalat, csapatméret, iparág, főbb technológiák.
- **Tanulási cél:** egy SMART módon megfogalmazott cél (specifikus, mérhető, elérhető, releváns, időhöz kötött).
- **Önértékelés:** a könyv fő témaköreiben adott 1–5 skálás értékelés, rövid indoklással.
- **Döntési napló:** a tanulási cél kiválasztásának indoklása, a megvizsgált alternatívák és azok elvetésének okai.
- **Ellenőrzési szempontok:** a labor befejezettségének kritériumai.

1.1.7.3 Lépések

1. **Rögzítsd a kontextust.** Írd le, milyen szerepben dolgozol jelenleg (például backend fejlesztő, platform mérnök, tech lead, termékmenedzser), hány év tapasztalatod van, mekkora csapatban dolgozol, milyen iparágban, és melyek a leggyakrabban használt technológiák. Ez a rész legalább öt–tíz mondat legyen, hogy később is vissza tudd idézni a kiindulási helyzetet.

2. **Határozd meg a tanulási célt.** Válassz egy olyan konkrét célt, amelyet a könyv elolvasásával el szeretnél érni. A cél legyen specifikus, mérhető, elérhető, releváns és időhöz kötött. Példa: *„Három hónapon belül képes leszek egy meglévő mikroszolgáltatás-rendszer rendelkezésre állási SLO-ját megfogalmazni, és legalább két mérési ablakban ellenőrizni a megfelelését."* Kerüld az olyan általános célokat, mint „jobban érteni az architektúrát".

3. **Végezd el az önértékelést.** Az alábbi témakörök mindegyikében értékeld a jelenlegi tudásszintedet 1-től 5-ig. A skálához rövid leírást adunk, hogy csökkentsük a szubjektivitásból adódó bizonytalanságot:

- *Alapfogalmak és dokumentáció:* mennyire ismered a komponensek, nézetek, ADR-ek fogalmait? (1 = nem ismerem, 5 = rendszeresen használom)
- *Követelmények és minőségi attribútumok:* mennyire tudsz mérhető minőségi követelményeket megfogalmazni?
- *Architektúrális stílusok:* mennyire ismered a rétegezett, monolit, mikroszolgáltatás, eseményvezérelt mintákat?
- *Adat, platform és cloud:* mennyire ismered az adatarchitektúrát, a konténereket, a Kubernetes-t és a felhőszolgáltatásokat?
- *Megbízhatóság és biztonság:* mennyire ismered a reziliencia-mintákat, a threat modelinget és a megfelelési követelményeket?
- *Delivery és evolúció:* mennyire ismered a CI/CD-t, a tesztelést, a legacy modernizációt és az evolúciós architektúrát?
- *Szervezeti tényezők:* mennyire ismered a Conway-törvényt, a stakeholder-kezelést és az architekt szerepét a szervezetben?

Minden témakörhöz írj egy rövid indoklást is, hogy később ellenőrizni tudd a fejlődésedet.

4. **Készíts döntési naplót.** A tanulási cél kiválasztása egy döntés, ezért érdemes ADR-szerűen dokumentálni. Rögzítsd a döntés kontextusát, a választott célt, a megvizsgált alternatívákat (például más témakörök, amelyeket fontolgattál), és azok elvetésének okát. Például: *„Alternatíva: a biztonsági architektúra elmélyítése. Elvetve, mert jelenlegi szerepemben a rendelkezésre állás javítása sürgetőbb, és a V. rész később is beilleszthető."*

5. **Ellenőrizd a dokumentumot.** Használd az alábbi ellenőrző szempontokat:

   - A kontextus egyértelműen azonosítja a szerepet, a tapasztalatot és a környezetet.
   - A tanulási cél SMART, azaz mérhető és időhöz kötött.
   - Az önértékelés mind a hét fő témakört lefedi, és az eredmény számszerű, indoklással.
   - A döntési napló tartalmazza a választott célt és legalább egy elvetett alternatívát.
   - A dokumentum verziószámot vagy dátumot tartalmaz.

6. **Zárd le és mentsd el.** A dokumentumot mentsd el olyan helyre, ahol később is elérhető. Ez lesz a kiindulási alap, amelyhez a könyv végén visszatérhetsz, hogy felmérd a fejlődésedet.

1.1.7.4 A mért és a várt eredmény elkülönítése

A laborban a **mért eredmény** az önértékelés során kapott pontszámokból és az azokhoz tartozó indoklásokból áll. Ez a jelenlegi állapotodat tükrözi, és szubjektív bizonytalanságot hordoz, mert saját megítélésen alapul. A **várt eredmény** a tanulási célban megfogalmazott állapot elérése, például az adott témakörben való magabiztos döntéshozatal vagy egy konkrét SLO definiálása. A kettőt a dokumentumban külön szekcióban kell feltüntetni, hogy a későbbi értékeléskor ne keveredjenek össze.

A mérés környezetét, időpontját és konfigurációját itt a kontextus rész adja meg: ebben az esetben a „környezet" a szereped és a csapatod, a „konfiguráció" a jelenlegi technológiai stack, az „időpont" pedig a dokumentum kelte. A bizonytalanságot az önértékelési skála leírásainak használatával csökkentjük.

1.1.7.5 Visszaállítási út

Ha a tanulási célod megváltozik, vagy a dokumentumot újra kell értékelned, ne írd felül az eredeti változatot. Készíts új verziót, és a döntési naplóban rögzítsd a változás okát. Így a kiindulási állapot mindig rekonstruálható marad, és a fejlődésed nyomon követhető.

1.1.8 Ellenőrzőlista

- Tisztában vagy a szerző szakmai hátterével és a könyv motivációjával?
- Meg tudod nevezni a könyv öt célcsoportját, és be tudod sorolni magad valamelyikbe?
- Ki tudod választani a három olvasói út közül a számodra legmegfelelőbbet?
- Meg tudod fogalmazni a saját tanulási célodat SMART formában?
- El tudod különíteni a laborban a mért és a várt eredményt?

1.1.9 Önellenőrző kérdés

Milyen konkrét architektúrális döntési helyzetben szeretnéd alkalmazni a könyvben tanultakat, és hogyan fogod mérni, hogy a tanulási célod teljesült?

A válasz megfogalmazása segít abban, hogy a könyv olvasása közben mindig a számodra leginkább releváns tartalmakra összpontosíts, és a fejezetek végén található laborokat a saját kontextusodra szabva végezd el.

1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések

A fejezet célja, hogy az olvasó ne csak lineárisan haladjon a könyvben, hanem a saját szerepéhez és kérdéseihez igazítsa a tanulási utat. Ehhez bemutatjuk a hét rész logikáját, a laborok és architektúra-katák funkcióját, valamint azokat a dokumentációs és jelölési konvenciókat, amelyek a könyv minden fejezetében visszatérnek. A konvenciók nem formális kellékek: lehetővé teszik, hogy egy döntést, mérést vagy diagramot más csapattagok is reprodukáljanak, ellenőrizzenek és vitassanak. A fejezet végén egy kis labor segítségével előkészítjük a saját fejlesztői és dokumentációs környezetet, amelyhez a további fejezetek gyakorlatai csatlakoznak.

1.2.1 1. A hét rész logikája és az olvasási utak

A könyv hét része egy tudatos ívet jár be: az alapoktól a rendszertervezésen át a produkciós működtetésig, majd a szervezeti és iparági kontextusig. Az egyes részek nem egymástól független silók, hanem egyre táguló rendszerhatárokat vizsgálnak.

Az **I. rész** (Alapok, elvek, döntések és dokumentáció) lefekteti azt a nyelvet és gondolkodásmódot, amellyel architektúráról beszélünk. Itt tanuljuk meg, hogy az architektúra nem pusztán dobozok és nyilak rajza, hanem döntések sorozata, amelyeket tudatosan hozunk, dokumentálunk és felülvizsgálunk. A **II. rész** (Követelmények, minőség és tervezés) azt vizsgálja, hogy mit jelent a „jó” rendszer: a teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság és más minőségi attribútumok hogyan alakulnak át mérhető forgatókönyvekké és tervezési kritériumokká. A **III. rész** (Architektúrális stílusok és minták) a konkrét szerkezeti megoldásokat tárgyalja: monolit, mikroszolgáltatások, eseményvezérelt rendszerek, API-k és domain-architektúrák. Itt mindig a kontextus a döntő: nincs univerzálisan „jó” minta, csak adott követelmények, csapatméret és terhelés mellett értelmezhető döntés.

A **IV. rész** (Adat, platform és cloud) az adatkezelést és a felhőalapú üzemeltetést kapcsolja össze: adatmodellek, konzisztencia, konténerek, Kubernetes, serverless, platform engineering és költség. A **V. rész** (Megbízhatóság, biztonság és üzemeltetés) a produkciós valóságra összpontosít: hogyan tervezzünk reziliens rendszert, hogyan védekezzünk fenyegetések ellen, és hogyan figyeljük meg, mit csinál a rendszer élesben. A **VI. rész** (Delivery, evolúció és szervezet) azt mutatja meg, hogy az architektúra nem statikus terv, hanem folyamatos tevékenység: CI/CD, tesztelés, refaktorálás,

evolúciós architektúra, valamint a szervezeti kultúra és a csapatok hatása. Végül a **VII. rész** (Esettanulmányok, iparágak, jövő és záróprojekt) a tanultakat valós kontextusokban és egy end-to-end záróprojektben integrálja, kitekintéssel az AI/ML, edge, blockchain és más modern irányokra.

Ebből a szerkezetből többféle olvasási út is kirajzolódik. A **lineáris út** a kezdőktől halad a haladó szint felé, és minden részt sorban dolgoz fel. A **szerep-alapú út** gyorsabban céloz: egy programozó, aki most lép át rendszertervezői szerepbe, az I. és II. részt olvassa el alaposan, majd a III. részből a mikroszolgáltatások és az eseményvezérelt minták fejezeteit, a IV. részből pedig a konténer- és adatarchitektúrát. Egy üzemeltetési mérnök inkább a IV., V. és VI. részekre koncentrál, de nem hagyhatja ki a minőségi forgatókönyvek (II. rész) és az architektúra-dokumentáció (I. rész) alapjait. A termékmenedzser számára a II. rész követelménykezelése, a VI. rész szervezeti fejezetei és a VII. rész iparági esettanulmányai adnak hasznos keretet, míg a technikai részleteket átlósan is olvashatja.

A **probléma-alapú út** akkor hasznos, ha az olvasó egy konkrét kérdéssel érkezik: például „hogyan tervezzünk API-t?" vagy „hogyan csökkentsük az adatbázis terhelését?". Ekkor a tárgymutató és a fejezetek közötti kereszthivatkozások segítenek: a kérdésből visszafejtve azonosítjuk az érintett minőségi attribútumot, a hozzá tartozó mintákat és a szükséges validációs technikákat. Mindegyik út közös eleme, hogy a fejezetekben használt fogalmakat mindig a kontextussal együtt kell értelmezni, és a példákat nem másolni, hanem adaptálni kell.

1.2.2 2. Laborok, katák és a záróprojekt szerepe

A könyv nem csak olvasásra készült: a fejezetekhez laborok tartoznak, amelyek a gyakorlati alkalmazást szolgálják. A laborok formái:

- **Architektúra-katák:** rövid, fókuszált gyakorlatok, amelyek egy konkrét döntési helyzetet adnak, és a résztvevőnek ki kell dolgoznia egy architektúra-részletet, indoklással. A kata nem verseny, hanem tanulási forma: a lényeg a döntési folyamat és a trade-offok megértése.
- **Döntési naplók (ADR-ek):** egy adott technológiai vagy szerkezeti döntés rögzítése a később bemutatott séma szerint.
- **C4 diagramok:** rendszerhatárok, konténerek és komponensek ábrázolása diagrams-as-code eszközzel.
- **PoC-k és spike-ok:** kisméretű, időben korlátozott kísérletek egy technológia vagy minta kipróbálására. A PoC célja a tanulás, nem a termelési döntés automatikus igazolása.
- **Terhelési modellek és mérések:** kapacitástervezés, Little-törvény alkalmazása, skálázási döntések előkészítése.
- **Threat modellek:** biztonsági elemzések, amelyek a fenyegetéseket és a bizalmi határokat azonosítják, nem pedig támadási útmutatót adnak.
- **Runbookok:** üzemeltetési eljárások leírása, amelyeket incidens esetén követni lehet.

A laborok mindig rendelkeznek **kiinduló állapottal**, **elvárt eredménnyel**, **ellenőrzési szempontokkal** és **visszaállítási úttal**. Ez különösen fontos a destruktív műveleteknél (például adatbázis-törlés vagy klaszter-átkonfigurálás), amelyeket laborban is csak izolált környezetben szabad végrehajtani. A kód- és konfigurációpéldák szemléltetők; éles rendszeren nem kell futtatni őket. A laborok során **mindig különítsük el a mért és a várt eredményt**: a mért érték a ténylegesen lefuttatott kísérlet kimenete, a várt pedig a hipotézisünk. A kettőt összekeverni hibás következtetéshez vezet.

A **záróprojekt** a VII. rész végén található, és egy end-to-end architektúra elkészítését kéri: követelmények és minőségi forgatókönyvek meghatározása, architektúra-alternatívák és döntési dokumentáció, egy PoC és mérés, végül egy architektúra-review és bemutató. A záróprojekt nem

a „tökéletes” megoldásról szól, hanem a folyamat gyakorlásáról: hogyan hozunk döntéseket bizonytalanságban, hogyan dokumentáljuk a trade-offokat, és hogyan kommunikáljuk az eredményt stakeholderek felé.

1.2.3 3. Dokumentációs és jelölési konvenciók

A könyvben következetesen alkalmazott konvenciók célja, hogy a dokumentáció ne csak a szerző számára legyen érthető, hanem a jövőbeli karbantartó, új csapattag vagy auditor is reprodukálni tudja a döntéseket és méréseket.

1.2.4 3.1 ADR, minőségi forgatókönyvek és mérőszámok

Egy **architektúradöntést** ADR-ben rögzítünk, amelynek szerkezete:

- **Kontextus:** mi a döntés tárgya, milyen követelmények vagy kényszerek állnak fenn.
- **Döntés:** mit választottunk, és miért.
- **Alternatívák:** milyen más lehetőségeket vizsgáltunk, és miért vetettük el őket.
- **Következmények:** mi lesz a döntés hatása (pozitív és negatív egyaránt), milyen további lépéseket igényel.
- **Állapot:** például „javasolt”, „elfogadott”, „elavult”.

Az ADR-ek száma nem cél, hanem eszköz. Egy ADR akkor jó, ha a jövőbeli olvasó megérti belőle a döntés okát és a feladott alternatívákat. Az ADR-eket verziózott dokumentumtárban tároljuk, és a kódbázisban hivatkozunk rájuk.

A **minőségi követelményeket** quality attribute scenario formában fogalmazzuk meg, amely négy elemet tartalmaz: **inger** (mi történik), **környezet** (milyen állapotban van a rendszer), **válasz** (mit kell tennie) és **mérőszám** (hogyan mérjük a választ). Például: „Amikor 500 egyidejű felhasználó kérést küld a katalógus szolgáltatásnak normál üzemmódban, a rendszernek a kérések 95%-át 1,2 másodpercen belül ki kell szolgálnia.” A mérőszámokat mindig mértékegységgel és mérési ablakkal adjuk meg: „99,9% rendelkezésre állás 30 napos gördülő ablakban”. A rendelkezésre állás képlete például:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Az error budget pedig $1 - \text{SLO}$ az adott ablakban. A kapacitástervezésnél a Little-törvényt használjuk: $L = \lambda W$, ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta, W az átlagos tartózkodási idő. A képletek a PDF-ben LaTeX, a HTML/EPUB változatban texmath/MathML formátumban jelennek meg, és csak egyszerű parancsokat használnak; a tizedesvesszőt `{,}` jelöli.

1.2.5 3.2 Diagramok és diagrams-as-code

A diagramokat **diagrams-as-code** formában készítjük (például Mermaid, PlantUML vagy Structurizr DSL), hogy a kódbázissal együtt verziózhatók legyenek. A C4 modellezésnél a **context**, **container** és **component** szint kötelező; a code szintet csak indokolt esetben használjuk. A rendszerhatárt minden diagramon jelölni kell, és a külső rendszereket, felhasználói szerepköröket a context szinten feltüntetjük. A komponenseket nemcsak névvel, hanem **felelősséggel és interfésszel** együtt írjuk le; a kapcsolatokon szerepel a **protokoll**, a **szinkronitás** és a **hibatűrés**. Példa egy minimális Mermaid context diagramra:

```
graph TD
    U[Ügyfél] -->|HTTPS, szinkron| B[Rendeléskezelő szolgáltatás]
    B -->|HTTPS, szinkron| P[Fizetési szolgáltatás]
    B -->|AMQP, aszinkron| S[Szállítási szolgáltatás]
```

A diagram címe, jelmagyarázata és feltételjegyzete kötelező; a mérési ábráknál a terhelés, a környezet és a mérési ablak is szerepel.

1.2.6 3.3 Hivatkozások és biztonsági helyőrzők

A források megadásakor elsődlegesen szabványokra (ISO/IEC/IEEE 42010, ISO/IEC 25010), hivatalos dokumentációkra (Kubernetes, OpenTelemetry, CNCF, felhőszolgáltatók well-architected keretei), eredeti szakirodalomra és megbízható esettanulmányokra támaszkodunk. A hivatkozásnál mindig rögzíteni kell a **verziót**, a **dátumot** és a **konkrét szakaszt**. A „state of the art" állítások dátumhoz kötöttek; a nem reprodukált eredményt jelölni kell.

A dokumentáció **nem tartalmazhat valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot**. Helyettük helyőrzőket használunk: `https://példa.belső`, `API_KEY_PLACEHOLDER`, `ACME Zrt.` stb. A biztonsági fejezetek a védekezésre és a biztonságos tervezésre koncentrálnak; nem adnak valós rendszer elleni támadási útmutatót. A threat modellezésnél a bizalmi határokat, az adatáramlást és a fenyegetéseket mutatjuk be, nem a kihasználást.

1.2.7 4. Minipélda: egyszerű szolgáltatás a konvenciók szerint

Tekintsünk egy képzeletbeli „Rendeléskezelő szolgáltatás" nevű belső rendszert. A cél a konvenciók szemléltetése, nem a teljes tervezés.

Context diagram (C4, context szint) – a rendszerhatárt a szaggatott vonal jelöli:

```
graph TD
    subgraph Cég belső rendszerei
        R[Rendeléskezelő szolgáltatás]
    end
    U[Ügyfél] -->|HTTPS, szinkron| R
    R -->|HTTPS, szinkron| P[Fizetési szolgáltatás]
    R -->|AMQP, aszinkron| S[Szállítási szolgáltatás]
```

Minőségi forgatókönyv a rendelkezésre állásra:

- **Inger:** a Fizetési szolgáltatás 5 másodpercen át nem elérhető.
- **Környezet:** normál üzemi terhelés (átlagosan 20 kérés/másodperc).
- **Válasz:** a Rendeléskezelő szolgáltatás a függő kéréseket korlátozott ideig várólistára teszi, a felhasználónak elfogadó állapotjelzést ad, és a fizetési állapotot aszinkron módon egyezteti.
- **Mérőszám:** a sikeresen elfogadott rendelések aránya legalább 99% 10 perces ablakban; a nem elfogadott kérések hibaaránya legfeljebb 1%.

ADR kivonat a fizetési integrációhoz:

- **Kontextus:** A fizetési szolgáltatás külső, időnként késik; a szinkron hívás blokkolja a felhasználói folyamatot.
- **Döntés:** Aszinkron üzenetküldést vezetünk be a fizetési szolgáltatás felé outbox mintával, a rendelés állapotát „függőben" értékre állítjuk.
- **Alternatívák:** szinkron retry logika (elvetve: a felhasználói várakozás túl nagy); közvetlen adatbázis-integráció (elvetve: szoros csatolás).

- **Következmények:** javul a rendelkezésre állás, de nő a komplexitás és az esetleges duplikátum-feldolgozás kezelése szükséges.
- **Állapot:** elfogadott.

Mérési példa: Tegyük fel, hogy egy terhelési teszt során mértük az érkezési rátát: $\lambda = 15$ req/s, az átlagos tartózkodási idő $W = 0{,}8$ s. A Little-törvény szerint a rendszerben lévő kérések átlagos száma:

$$L = \lambda W = 15 \cdot 0{,}8 = 12 \text{ request}$$

A **mért** érték a teszt során ténylegesen megfigyelt átlagos konkurens kérésszám volt, például 11,5; a **várt** érték a képletből számolt 12. Az eltérés nem feltétlenül hiba, de indokolni kell (például a rendszer nem volt teljesen egyensúlyi állapotban, vagy a mérési ablak rövid volt).

1.2.8 5. Tipikus hibák és javításuk

A könyvben bemutatott gyakorlatok során gyakran előforduló hibák és a helyes hozzáállás:

- **Kontextus nélküli döntés:** „Mikroszolgáltatásokat használunk, mert az a modern." Helyette: milyen követelmények, csapatméret, terhelés és szervezeti feltétel indokolja a választást? A döntés mögötti elvet és a kivételeket is rögzíteni kell.
- **A mért és várt eredmény összekeverése:** a laborokban külön oszlopban vagy szakaszban tüntessük fel a hipotézist (várt) és a tényleges kimenetet (mért). Ha a kettő eltér, az tanulság, nem feltétlenül hiba.
- **Rendszerhatár hiánya:** diagramon nem jelölik, mi tartozik a rendszerhez és mi külső. Ez félreértésekhez vezet a felelősségek és az interfészek tekintetében.
- **Nem verziózott dokumentáció:** az architektúra-dokumentumokat ugyanúgy verziózni kell, mint a kódot. A döntések változása esetén az ADR-ek állapotát frissíteni kell.
- **Túl sok ADR, tartalom nélkül:** az ADR-ek száma nem cél. Egy jól megírt ADR többet ér, mint tíz üres sablon.
- **Valós adatok használata:** hitelesítő adatok, belső URL-ek vagy ügyfélnevek szerepeltetése a példákban. Mindig helyőrzőt használjunk.
- **Diagramok, amelyek nem diagrams-as-code:** a kézzel rajzolt vagy vágólapra másolt diagramok nem reprodukálhatók, nehezen verziózhatók, és idővel elavulnak. Használjunk Mermaid, PlantUML vagy Structurizr DSL-t.

1.2.9 6. Labor: saját környezet előkészítése

Ennek a fejezetnek a laborja a saját fejlesztői és dokumentációs környezet előkészítése. A cél, hogy a későbbi fejezetek gyakorlataihoz egy reprodukálható alapot hozzunk létre.

Kiinduló állapot: rendelkezésre áll egy verziókezelő rendszer (például Git), egy szövegszerkesztő vagy jegyzetfüzet-környezet, valamint egy diagrams-as-code eszköz telepítése (például Mermaid CLI vagy PlantUML). Ha ezek hiányoznak, először telepítsük őket.

Elvárt eredmény: egy verziózott dokumentumtár (pl. `architektura/` mappa a repositoryban), amely tartalmazza:

- egy `README.md` fájlt a környezet leírásával és a használt eszközök verziójával,
- egy `adr/` mappát az ADR-ek számára, benne egy `ADR-000-sablon.md` fájllal a fent ismertetett szerkezettel,

- egy `diagrams/` mappát, benne egy minta context diagrammal Mermaid vagy PlantUML formátumban,
- egy `quality-scenarios.md` fájlt, amely legalább egy minőségi forgatókönyvet tartalmaz a megadott négy elem szerint,
- egy `measurements/` mappát, ahová a mérési jegyzőkönyvek kerülnek.

Ellenőrzési szempontok:

- A repository klónozható és a dokumentumok megnyithatók.
- Az ADR sablon tartalmazza a kontextus, döntés, alternatívák, következmények és állapot mezőket.
- A diagram kódja diagrams-as-code, és a rendszerhatár jelölve van.
- A minőségi forgatókönyvben az inger, környezet, válasz és mérőszám egyértelműen elkülönül.
- A mérési mappában egy `README.md` leírja, hogy a mért és a várt eredményt külön kell vezetni.

Visszaállítási út: a labor során létrehozott fájlok törölhetők vagy egy külön branch-en hagyhatók; éles rendszerre nincs hatás.

Példa a döntési naplóra (a laborhoz használható):

```
# ADR-000: Dokumentációs környezet kialakítása

- Kontextus: A tanulási folyamat során szükség van egy verziózott helyre az
↪  architektúra-dokumentumoknak.
- Döntés: A repository-ban `architektura/` mappát hozunk létre, diagrams-as-code eszközt
↪  használunk.
- Alternatívák: wiki (elvetve: nehezen verziózható), megosztott dokumentum (elvetve: nincs
↪  előzmény).
- Következmények: a dokumentumok a kóddal együtt verziózhatók, de a kollaborációhoz további
↪  eszköz kell.
- Állapot: elfogadva.
```

1.2.10 7. Önellenőrző kérdések és gyakorlat

- Mi a különbség a lineáris olvasási út és a szerep-alapú út között? Melyiket választanád a saját helyzetedben?
- Milyen elemekből áll egy quality attribute scenario? Adj egy példát a saját rendszeredre.
- Miért kritikus a mért és várt eredmény szétválasztása egy PoC vagy terhelési teszt során?
- Mi a célja az ADR-eknek, és mikor tekinthető egy ADR jónak?
- Milyen szempontokat kell feltüntetni egy diagrams-as-code diagramon a kapcsolatoknál?

Gyakorlat: Válassz egy általad ismert kisebb rendszert (lehet egy belső eszköz vagy nyílt forráskódú projekt), és készíts hozzá egy context szintű C4 diagramot Mermaid vagy PlantUML formában. Jelöld a rendszerhatárt, a külső szereplőket, a fő komponenst, valamint a kapcsolatok protokollját, szinkronitását és hibatűrését. Írj egy ADR-t a rendszer egyik fontos döntéséről, és fogalmazz meg egy minőségi forgatókönyvet a rendelkezésre állásra.

1.2.11 Összefoglalás

Ebben a fejezetben áttekintettük a könyv hét részének logikáját, az olvasási utakat, valamint a laborok és architektúra-katák szerepét. Bevezettük azokat a dokumentációs és jelölési konvenciókat, amelyek a könyv egészén átívelnek: ADR-ek, minőségi forgatókönyvek, mérőszámok mértékegységgel és mérési ablakkal, diagrams-as-code és C4, biztonsági helyőrzők, valamint a

források pontos hivatkozása. A fejezet laborjában előkészítettük a saját környezetet, amely a későbbi gyakorlatok alapjául szolgál. A következő egységben (Definíciók és nézőpontok: struktúra, döntés és társadalmi konstrukció) az architektúra definíciójával és alapfogalmaival mélyítjük el ezt a keretet.

A fejezet végére értünk; a további fejezetekben ezt a keretet használjuk folyamatosan, ezért érdemes a laborban létrehozott környezetet megőrizni és a gyakorlatokat abban végezni.

2 Mi a szoftverarchitektúra?

2.1 Definíciók és nézőpontok: struktúra, döntés és társadalmi konstrukció

A szoftverarchitektúra fogalmát három, egymást kiegészítő nézőpontból érdemes megvizsgálni: mint struktúrát, mint döntéshalmazt és mint társadalmi konstrukciót. A három nézőpont nem verseng egymással, hanem ugyanannak a rendszernek más-más oldalát világítja meg. Egy érett architektúragyakorlatban mindhárom jelen van: a struktúra adja a közös térképet, a döntéshalmaz rögzíti, miért így néz ki a térkép, a társadalmi konstrukció pedig azt, hogyan értik és használják a térképet a különböző szereplők.

2.1.1 1. Az architektúra mint struktúra

A legismertebb meghatározás szerint a szoftverarchitektúra a rendszer alapvető szerveződése: azok a komponensek, amelyekből a rendszer áll, a köztük lévő kapcsolatok, valamint a komponensek és a környezetük közötti viszonyok. Az ISO/IEC/IEEE 42010 rendszer- és szoftverarchitektúra-szabványa ezt a nézőpontot hangsúlyozza: az architektúra a rendszer alapvető fogalmai vagy tulajdonságai a környezetében, megtestesülve annak elemeiben, kapcsolataiban, valamint a tervezés és fejlődés elveiben. E definíció szerint az architektúra nem csupán a kód elrendezése, hanem azoknak az elveknek az összessége is, amelyek a rendszer létrehozását és fejlődését irányítják.

A struktúra nézőpont középpontjában a komponensek és azok interfészei állnak. Komponens lehet egy osztály, egy modul, egy szolgáltatás, egy adatbázis, egy üzenetsor vagy akár egy külső rendszer is – attól függően, milyen felbontásban vizsgáljuk a rendszert. A kapcsolatok pedig nem egyszerűen vonalak, hanem szerződések: meghatározzák, milyen protokollon, szinkron vagy aszinkron módon, milyen adatformátumban és milyen hibatűrési garanciákkal kommunikál két komponens. Egy webáruház esetében például a „Rendelés-szolgáltatás” komponens szinkron REST-hívással kérdezheti le a „Készlet-szolgáltatástól” egy termék elérhetőségét, de a rendelés visszaigazolása már aszinkron eseményként érkezhet a „Számlázó-rendszerbe”. A struktúra rögzíti, hogy melyik komponens felelős a rendelés állapotának kezeléséért, és melyek azok a határok, amelyeket egyik komponens sem léphet át.

A struktúra leírása nem öncélú. Jó struktúra nélkül a rendszer egyetlen nagy, összefüggő masszává válik, amelyben minden mindennel összefügg, és egy apró módosítás előre nem látható következményekkel jár. A struktúra azért fontos, mert lehetővé teszi a komplexitás kezelését: a rendszert kisebb, érthető egységekre bontjuk, amelyeket egymástól viszonylag függetlenül lehet fejleszteni, tesztelni és üzemeltetni. A komponensek közötti függőségek explicit rögzítése pedig megadja a változtatások becsült hatókörét.

A struktúra nézőpontban a rendszerhatár kijelölése kulcsfontosságú. A rendszerhatár választja el azt, amiért a csapat felelős, attól, ami a környezethez tartozik. Minden, ami a határon belül van, a rendszer része; minden, ami kívül van, környezeti elem: felhasználó, külső szolgáltatás, szabályozó, hardver vagy másik rendszer. A határ meghúzása nem triviális. Egy online bank

esetében a mobilalkalmazás a rendszer része? A külső hitelinformációs szolgáltatás? A központi számlavezető rendszer? A határok megválasztása attól függ, milyen kérdésre keressük a választ. Ha az architektúrát a mobilalkalmazás és a backend közötti integráció szempontjából vizsgáljuk, akkor a mobilalkalmazás és a backend együtt alkotja a rendszert, a hitelinformációs szolgáltatás pedig külső. Ha viszont a teljes banki ökoszisztémát nézzük, akkor a központi számlavezető is belső komponens lehet. A rendszerhatárt minden diagramon, minden leírásban explicit módon jelölni kell; különben a nézőpontok összekeverednek, és a stakeholder nem tudja eldönteni, miért is felelős a csapat.

2.1.2 2. Az architektúra mint döntéshalmaz

A struktúra önmagában csak egy pillanatfelvétel. Nem magyarázza meg, miért éppen így néz ki a rendszer, milyen alternatívákat vetettünk el, és milyen következményeket vállaltunk. A döntéshalmaz nézőpont ezt a hiányt pótolja: az architektúra azoknak a döntéseknek az összessége, amelyek meghatározzák a rendszer szerkezetét, viselkedését és fejlődését. A döntések lehetnek technológiai jellegűek (például „PostgreSQL-t használunk a tranzakciós adatokhoz, nem MongoDB-t”), strukturálisak („a számlázást külön szolgáltatásba szervezzük, nem a monolitba”), vagy minőségi trade-offok („a rendelkezésre állás érdekében elfogadjuk az esetleges átmeneti inkonzisztenciát a termékajánló szolgáltatásban”).

Az architektúradöntések gyakran visszafordíthatatlanok vagy csak jelentős költséggel megváltoztathatók. Éppen ezért nem elég a döntést meghozni: rögzíteni kell a kontextusát, az indokait, az elvetett alternatívákat és a következményeit. Erre szolgál az Architecture Decision Record (ADR), amely a döntési napló alapegysége. Egy ADR tipikus felépítése a könyvben használt konvenció szerint:

- **Cím és azonosító:** egyedi, visszakereshető azonosító.
- **Állapot:** javasolt, elfogadott, elavult, felváltotta egy másik ADR.
- **Kontextus:** milyen helyzetben, milyen erők hatására született a döntés.
- **Döntés:** a választott megoldás pontos leírása.
- **Alternatívák:** mely lehetőségeket vizsgáltuk és miért vetettük el.
- **Következmények:** milyen előnyökkel és hátrányokkal jár a döntés, milyen további döntéseket tesz szükségessé.

Egy példa ADR vázlata egy fiktív rendszerhez:

ADR-003: Aszinkron üzenetkezelés a rendelés-visszaigazoláshoz - **Állapot:** elfogadott - **Kontextus:** A rendelés-szolgáltatás szinkron módon hívta a számlázó-rendszert. Csúcsidőben a számlázó-rendszer válaszideje megnőtt, és a rendelés-szolgáltatás szálai elakadtak, ami az egész rendelésfelvételt lelassította. Az SLO szerint a rendelésfelvétel p95 válaszideje 300 ms alatt kell, hogy maradjon; ezt a szinkron hívás veszélyeztette. - **Döntés:** A rendelés-szolgáltatás a rendelés létrehozása után egy „RendelésLétrehozva” eseményt publikál az üzenetsorba, és a számlázó-rendszer aszinkron módon fogyasztja azt. A felhasználó felé a rendelés elfogadása azonnal megtörténik; a számla kiállítása később, az esemény feldolgozása után. - **Alternatívák:** 1) Szinkron hívás timeouttal és újrapróbálkozással: egyszerűbb, de csúcsidőben továbbra is erőforrás-versenyt okoz. 2) Dedikált szinkron szolgáltatás a számlázáshoz: javítja a párhuzamosságot, de növeli az üzemeltetési terhet. 3) Teljesen szinkron, de kapacitásbővítés: drága, és nem oldja meg a komponensek közötti szoros csatolást. - **Következmények:** A rendelésfelvétel válaszideje javul, mert nem vár a számlázóra. Cserébe a rendszerben megjelenik az esetleges inkonzisztencia: előfordulhat, hogy a számla kiállítása késik, vagy hibás rendelés esetén kompenzálni kell. Szükségessé válik az üzenetsor üzemeltetése, monitoringja és az üzenetek idempotens

feldolgozása. A döntés kikényszeríti az outbox-minta használatát a rendelés-szolgáltatásban, hogy ne vesszen el esemény. (Ez a téma a későbbi fejezetekben részletesen előkerül.)

Az ADR-ek száma nem öncél; a cél az, hogy a jövőbeli olvasó – akár egy új csapattag, akár a jövőbeli önmagunk – megértse, miért így néz ki a rendszer. A döntéshalmaz nézőpont rávilágít arra, hogy az architektúra nem passzív leírás, hanem aktív választások sorozata. A struktúra ennek a sorozatnak az eredménye.

2.1.3 3. Az architektúra mint társadalmi konstrukció

A harmadik nézőpont szerint az architektúra nem csupán a kódban vagy a dokumentumokban él, hanem az emberek fejében: az a közös értelmezés, amelyet a szervezet tagjai kialakítanak a rendszerről. Ez a felfogás hangsúlyozza, hogy az architektúra kommunikációs eszköz, amely hidat képez a különböző szereplők – fejlesztők, üzemeltetők, biztonsági szakemberek, termékmenedzserek, üzleti elemzők – között.

A stakeholdereknek különböző aggodalmaik (concernjeik) vannak. A fejlesztőt az érdekli, hogyan tud egy új funkciót hozzáadni anélkül, hogy a rendszer más részeit eltörné. Az üzemeltetőt a megfigyelhetőség, a naplózás, a telepíthetőség és a hiba utáni helyreállítás. A biztonsági szakembert a támadási felület, a jogosultságok és az adatáramlás. A termékmenedzsert a funkciók piacra vitelének sebessége és a költségek. Az architektúra akkor működik jól társadalmi értelemben, ha ezeket az aggodalmakat képes egy közös keretbe foglalni, és minden stakeholder megtalálja benne a számára releváns nézetet.

A társadalmi konstrukció nézőpont magyarázza meg, miért nem elég egyetlen nagy diagramot rajzolni. Egyetlen diagram nem tudja egyszerre kielégíteni a fejlesztő, az üzemeltető és a biztonsági szakember igényeit. Ezért van szükség több nézetre: a C4 modell context, container és component szintjei más-más közönségnek szólnak. A context szint a rendszerhatárt és a külső szereplőket mutatja – ezt a termékmenedzser is megérti. A container szint a futtatható egységeket és a köztük lévő hívásokat – ezt a fejlesztő és az üzemeltető közösen használja. A component szint az egyes konténerek belső felépítését – ez elsősorban a fejlesztőcsapaté. A code szint csak indokolt esetben, akkor készül, ha egy kritikus algoritmus vagy integrációs pont részletes megértése szükséges; az általános elv az, hogy a kód az igazság forrása, a diagram pedig csak kommunikációs segédeszköz.

A társadalmi konstrukció nézőpont összefügg Conway törvényével is, amelyet a későbbi szervezeti fejezet tárgyal részletesen. A törvény lényege: a rendszer architektúrája leképezi a szervezet kommunikációs struktúráját. Ha egy szervezetben a csapatok mereven el vannak választva, a rendszer is mereven szétváló modulokból fog állni; ha a csapatok szorosan együttműködnek, a rendszerben is szorosabb lesz az integráció. Ebből az következik, hogy az architektúra nem csak technikai döntés: a szervezeti struktúra, a kommunikációs minták és a felelősségi körök kialakítása ugyanolyan fontos része az architektúra munkának, mint a technológiai választás. Az architektúra társadalmi konstrukció volta azt is jelenti, hogy a dokumentáció és a diagramok csak akkor érnek valamit, ha a csapat használja őket, és ha naprakészek. Egy porosodó architektúra-dokumentum nem architektúra, hanem régészeti lelet.

2.1.4 4. A rendszerhatár és a környezet szerepe

A három nézőpont mindegyikében központi szerepet játszik a rendszerhatár. A határ nem pusztán technikai kérdés: meghatározza, miért felelős a csapat, mi az, amit irányít, és mi az, amit csak felhasznál. A határ kijelölése egyben felelősségi döntés is: aki a határon belül van, azért a hibákért is a csapat felel; ami kívül van, azért legfeljebb a szerződés betartásáért.

A környezet elemei közé tartoznak a felhasználók, a külső rendszerek, a szabályozók, a hardver és a hálózati infrastruktúra. Minden kapcsolat a rendszer és a környezet között szerződést jelent: a felhasználó használja a felületet, a külső rendszer API-t hív, a szabályozó előírja az adatkezelés módját. A rendszerhatárt a C4 context szinten kell feltüntetni: a diagramon jelölni kell, mi van belül, mi kívül, és milyen szereplők lépnek kapcsolatba a rendszerrel. E nélkül a diagram félrevezető: a néző nem tudja, hogy egy doboz a saját rendszerünk része, vagy egy külső szolgáltatás, amelyet nem irányítunk.

A rendszerhatár nem állandó. Egy rendszer fejlődése során a határ eltolódhat: amit korábban külső szolgáltatásként használtunk, azt később belső komponensként implementálhatjuk, vagy fordítva. Az architektúra-dokumentációban ezért a határt mindig a vizsgált kontextussal együtt kell rögzíteni, és a változást döntésként kell kezelni.

2.1.5 5. Stakeholderek és nézőpontok

A stakeholderek eltérő nézőpontjai miatt az architektúra leírása soha nem lehet egyetlen, mindenki számára egyformán hasznos nézet. A gyakorlatban bevált megközelítés, hogy több, jól meghatározott nézetet készítünk, és mindegyiket a megfelelő stakeholderhez kötjük. A 4+1 modell például logikai, folyamat, fejlesztési, fizikai és forgatókönyv nézeteket különböztet meg; a C4 modell pedig a context, container, component és code szinteket. A lényeg nem a konkrét keretrendszer, hanem az, hogy minden nézet egyértelműen feleljen meg egy stakeholderi aggodalomnak, és a nézetek közötti kapcsolat visszakereshető legyen.

A nézőpontok különbözősége félreértésekhez vezethet, ha nem tisztázzuk, kinek szól egy adott diagram. Például egy osztálydiagram, amely a fejlesztőcsapat belső kommunikációjára készült, alkalmatlan arra, hogy a biztonsági auditor megértse az adatáramlást; ahhoz adatfolyam- vagy threat modeling diagram kell. A jó architektúra-dokumentáció explicit módon megnevezi a célközönséget, és a nézetet a stakeholder aggodalmához igazítja.

2.1.6 6. Tipikus hibák és félreértések

A három nézőpont ismerete segít elkerülni a leggyakoribb csapdákat:

- **Az architektúra csak diagramok halmaza.** A diagramok fontosak, de a döntések és a mögöttes indokok nélkül csak statikus képek. A jövőbeli olvasó nem tudja, miért így néz ki a rendszer, és nem tudja, melyik elvet kövessük, ha változtatni kell.
- **Az architektúra a technológiai stack.** A technológiai választás része az architektúrának, de nem azonos vele. A struktúra, a felelősségek, a szerződések és a minőségi trade-offok legalább annyira fontosak, mint az, hogy Java vagy Python, Kubernetes vagy serverless.
- **Az architektúra nagy upfront tervezés.** A modern felfogás szerint az architektúra folyamatos tevékenység: a döntések a rendszer életciklusa során születnek, és ADR-ekben rögzítjük őket. A nagy, egyszeri tervezés gyakran túltervezéshez vezet, és a változó követelmények mellett hamar elavul.
- **Az architektúra csak a technikai csapat ügye.** A társadalmi konstrukció nézőpont rámutat, hogy az architektúra a szervezet kommunikációs mintáit is tükrözi, és a termékmenedzsernek, az üzemeltetőnek és a biztonsági szakembernek is van beleszólása, legalábbis a rá vonatkozó nézetekben.
- **A rendszerhatár elhagyása.** Ha nem jelöljük a határt, nem tudjuk, miért felelős a csapat, és mi az, amit nem irányít. Ez félreértésekhez és hibás elvárásokhoz vezet.

2.1.7 7. Labor: három definíció összehasonlítása egy saját rendszerre

A fejezet tanulási céljainak gyakorlásához válassz egy általad ismert rendszert – lehet egy munkahelyi projekt, egy nyílt forráskódú eszköz vagy egy kitalált, de reálisan specifikált rendszer. A labor célja, hogy ugyanazt a rendszert a három nézőpont szerint leírd, és a leírásokat összehasonlítsd.

Kiinduló állapot: Van egy rendszer, amelynek ismered a funkcióit, a főbb komponenseit és a technológiai környezetét. A rendszer lehet fiktív is, de a kontextust pontosan rögzíteni kell.

Elvárt eredmény: Egy reprodukálható dokumentum vagy jegyzetfüzet, amelyben:

1. **Struktúra nézőpont:** Rajzolj egy C4 context szintű diagramot diagrams-as-code formában (például Mermaid vagy PlantUML). Jelöld a rendszerhatárt, a felhasználói szerepköröket, a külső rendszereket, és a kapcsolatokon tüntesd fel a protokollt, a szinkronitást és a hibatűrési mechanizmust (például retry, timeout, circuit breaker). Írd le szövegesen is: melyek a fő komponensek, mi a felelősségük, és milyen interfészeik vannak.
2. **Döntéshalmaz nézőpont:** Válassz ki egy fontos architektúradöntést a rendszerben (például adatbázis-választás, szinkron vs. aszinkron kommunikáció, monolit vs. szolgáltatások). Rögzítsd ADR formában: kontextus, döntés, alternatívák, következmények, állapot. A döntési naplóban legalább három alternatívát sorolj fel, és mindegyiknél írd le, miért vetetted el.
3. **Társadalmi konstrukció nézőpont:** Azonosítsd a stakeholdereket (legalább három különböző szerepkört), és mindegyikhez írd le, mi a fő aggodalma, milyen nézetet használna, és hogyan kommunikál felé az architektúra. Térj ki arra is, hogy a szervezeti struktúra hogyan tükröződik a rendszerben (Conway törvénye alapján).

Ellenőrzési szempontok:

- A rendszerhatár egyértelműen jelölve van a context diagramon.
- A kapcsolatokon szerepel a protokoll, a szinkronitás és a hibatűrés.
- Az ADR tartalmazza a kontextust, a döntést, legalább három alternatívát, a következményeket és az állapotot.
- A stakeholder-elemzés minden szerepkörhöz megnevezi az aggodalmat és a hozzá tartozó nézetet.
- A dokumentum verziózott, és a mért (vagy megfigyelt) és a várt eredmény elkülönítve szerepel. Ha nincs mérés, azt jelölni kell: „nem futtatott elemzés, feltételezés”.

Visszaállítási út: A labor dokumentum jellegű; a verziókezelő rendszerben a korábbi verzió bármikor visszaállítható. Ha a labor során kódot vagy konfigurációt is módosítasz, azt csak izolált környezetben tedd, és a labor előtti állapotot tedd vissza.

A labor eredménye nem automatikusan termelési döntés; célja a tanulás és a nézőpontok tudatos alkalmazása.

2.1.8 8. Ellenőrzőlista

- ☐ Meg tudom fogalmazni az architektúra három nézőpontját: struktúra, döntéshalmaz, társadalmi konstrukció.
- ☐ El tudom magyarázni, miért nem elég a struktúra önmagában az architektúra megértéséhez.
- ☐ Értem a rendszerhatár szerepét, és tudom, miért kell minden diagramon jelölni.
- ☐ Tudom, mi az ADR, és milyen mezőkből áll.
- ☐ Felismerem, hogy a stakeholderek eltérő aggodalmai miért igényelnek több nézetet.

- ☐ Képes vagyok egy saját rendszerre a három nézőpont szerinti leírást elkészíteni.
- ☐ Tisztában vagyok a tipikus hibákkal: diagramok halmaza, technológiai stack, nagy upfront tervezés, csak technikai ügy.

2.1.9 9. Önellenőrző kérdés

Egy csapat egy új belső szolgáltatást tervez. A fejlesztő azt mondja: „Az architektúra egyszerű: Spring Boot, PostgreSQL, és minden szinkron REST." Az üzemeltető szerint ez nem architektúra, hanem technológiai stack. A termékmenedzser pedig azt kérdezi, hogyan skálázódik a rendszer, ha a felhasználók száma megduplázódik. Melyik nézőpont hiányzik az egyes szereplők érveléséből? Indokold a választ a fejezetben tanult három nézőpont alapján.

Megoldási vázlat: A fejlesztő elsősorban a technológiai döntéseket sorolja, de nem ad struktúrát (komponensek, kapcsolatok, felelősségek) és nem rögzíti döntésként az alternatívákat következményekkel. Az üzemeltető felismeri, hogy a stack nem azonos az architektúrával, de ő sem nevezi meg a struktúra és a döntéshalmaz szempontját. A termékmenedzser a minőségi attribútumot (skálázhatóság) feszegeti, ami a döntéshalmaz nézőpontjába tartozik, de a társadalmi konstrukció szempontjából az is fontos, hogy a szereplők közös nyelvet beszéljenek. A helyes válasz: a fejlesztőnek struktúra és döntéshalmaz leírást kell adnia, az üzemeltetőnek a struktúra és a döntések következményeit kell kérnie, a termékmenedzsernek pedig a minőségi forgatókönyvet kell megfogalmaznia, hogy a döntés értékelhető legyen.

2.1.10 Összegzés

Az architektúra három nézőpontja – struktúra, döntéshalmaz, társadalmi konstrukció – együtt ad teljes képet. A struktúra megmutatja, miből áll a rendszer; a döntéshalmaz megmagyarázza, miért; a társadalmi konstrukció pedig azt, hogyan értik és használják a szereplők. A következő fejezetben arra építünk, miért fontos az architektúra a sikerben és a kudarcban: a rossz struktúra, a nem rögzített döntések és a félreértett nézőpontok milyen következményekkel járnak a valós rendszerekben.

2.2 Miért fontos? Az architektúra szerepe a sikerben és a kudarcban

Ha egy szoftverrendszer hosszú távon működőképes marad, ritkán az a magyarázat, hogy „jó kódot írtunk", és ha megroppan, ritkán az az egyetlen ok, hogy „rossz volt a kód". A kód minősége szükséges, de nem elégséges feltétel. A siker és a kudarc gyakran azokban a szerkezeti döntésekben dől el, amelyeket a kód megírása előtt, azzal párhuzamosan vagy éppen a kód elhallgatása után hozunk: milyen komponensekre bontjuk a rendszert, milyen kapcsolatok engedhetők meg közöttük, hol húzódnak a bizalmi határok, mely minőségi tulajdonságok kapnak elsőbbséget, és milyen működési folyamatokban él majd a rendszer. Ezek a döntések együtt alkotják a szoftverarchitektúrát.

Ez a fejezet azt vizsgálja, miért számít az architektúra, és miért nem lehet pusztán a programozói tehetséggel vagy a legújabb technológiával helyettesíteni. Három hatásterületen keresztül járjuk körbe a kérdést: a minőségen, a költségen és a szervezeti képességen keresztül. Megnézzük, miért olyan drágák a korai döntések hibái, hogyan működik az architektúra mint kockázatkezelés, és hogyan lehet konkrét, ellenőrizhető módon tanulni mások kudarcaiból és sikereiből.

2.2.1 1. Az architektúra mint a minőség szerkezeti alapja

A funkcionális követelmények azt mondják meg, hogy a rendszer mit csinál: feldolgoz egy rendelést, megjelenít egy számlát, továbbít egy üzenetet. A nem funkcionális követelmények – a minőségi attribútumok – azt mondják meg, hogyan: milyen gyorsan, milyen rendelkezésre állással, milyen biztonsággal, milyen karbantarthatóság mellett. Az architektúra az a réteg, ahol a minőségi attribútumok nem elvont kívánságok, hanem tervezhető, mérhető, becserélhető szerkezeti tulajdonságokká válnak.

Vegyünk egy egyszerű példát. Egy online áruház rendeléskezelő funkciója működhet egyetlen monolitikus folyamatban is: a felhasználó elküldi a kosarát, a szerver leellenőrzi a készletet, rögzíti a rendelést az adatbázisban, és értesítést küld. Ha a minőségi követelmény az, hogy a rendelésfeladás akkor is működjön, ha az értesítési szolgáltatás éppen lassú, akkor az architektúrának lehetővé kell tennie, hogy az értesítés aszinkron módon, egy üzenetsoron keresztül történjen. Ha a minőségi követelmény az, hogy a rendelésfeladás másodpercenként több ezer kérést is elbírjon, akkor az architektúrának olyan szétválasztást kell tartalmaznia, amelyben a beérkező kérések feldolgozása nem akad meg egyetlen szűk keresztmetszeten. Ha a minőségi követelmény az, hogy egy adatvédelmi incidens ne tegye olvashatóvá a vásárlók összes rendelését, akkor az architektúrának olyan bizalmi határokat kell kijelölnie, amelyek korlátozzák, hogy egy komponens kompromittálódása milyen adatokhoz vezet.

A minőség tehát nem a kód utólagos „letuningolásából” születik. A kód optimalizálhat egy algoritmust, de ha a komponensek közötti kapcsolat szinkron, soros és erősen csatolt, akkor a teljesítmény felső határát a szerkezet szabja meg. Ha a rendszer nem rendelkezik jól elhatárolt írási és olvasási útvonallal, akkor a biztonsági mentés és a helyreállítás is nehézkes lesz, bármilyen gondosan is írtuk meg a SQL-lekérdezéseket. Az architektúra a minőség *szorzótényezője*: jó kódot lehet rossz architektúrába illeszteni, de a szerkezeti korlátokat a kód minősége nem bírja el.

A minőségi attribútumok egymással is versengenek. A rendelkezésre állás növelése gyakran redundanciát igényel, ami költséggel és bonyolultsággal jár. A teljesítmény növelése gyakran a konzisztencia lazítását követeli meg, ami viszont az üzleti logika bonyolultságát növeli. A biztonság erősítése általában lassítja a fejlesztést és a rendszer működését. Ezek a trade-offok az architektúra szintjén válnak láthatóvá és kezelhetővé. Amikor egy csapat azt mondja, hogy „a rendszer gyors, de nem biztonságos”, akkor valójában egy szerkezeti döntést ír le: olyan komponenseket és kapcsolatokat tervezett, amelyek a sebességet részesítik előnyben a biztonsággal szemben. Az architektúra feladata nem az, hogy ezt a feszültséget megszüntesse, hanem az, hogy tudatos döntéssé tegye, és a következményeket mérhetővé tegye.

2.2.2 1.1. A már nem változtatható döntések súlya

A minőségre gyakorolt hatás különösen azért fontos, mert egyes architektúrális döntések később csak aránytalanul nagy költséggel változtathatók meg. Az ilyen döntéseket nevezzük *irreverzibilisnek* vagy *magas változtatási költségűnek*. A programozási nyelv megválasztása, az adatbázis-technológia, a szinkron vagy aszinkron kommunikáció, a folyamatok közötti kapcsolatok, a felhőszolgáltató modellje – mind olyan döntés, amelynek megváltoztatása nem egyetlen refaktorálás, hanem gyakran több hónapos átépítés.

Ezért van az, hogy a korai döntések hosszú távú következményeit nem lehet pusztán az adott pillanat technikai előnye alapján megítélni. Egy olyan adatbázis, amely az első hónapban tökéletesen elvégzi a feladatát, később akadállyá válhat, ha a tranzakciós modellje nem illeszkedik a növekvő rendszerhez. Egy olyan szolgáltatáshatár, amely az első verzióban logikusnak tűnik, később minden új funkciónál újra és újra át kell hogy lépjen, ami lassítja a fejlesztést és növeli a

hibalehetőséget. Az architektúra éppen azért érték, mert ezeket a döntéseket a lehető legnagyobb tudatossággal hozza meg, és dokumentálja.

2.2.3 2. Az architektúra hatása a költségre

A költség itt nem csupán a felhőszámlát jelenti, hanem a teljes életciklusra vetített ráfordítást: a fejlesztés, az üzemeltetés, a továbbfejlesztés, a hibaelhárítás és a technikai adósság törlesztésének költségét is. Az architektúra ezek mindegyikét befolyásolja.

A fejlesztési költség oldalán a legnagyobb tétel általában a *megértés* költsége. Amikor egy fejlesztőnek egy új funkciót kell bevezetnie, először meg kell értenie, hogy a rendszer mely részei érintettek, milyen kapcsolatokon keresztül, és milyen mellékhatásokkal. Ha az architektúra tiszta határokat, felelősségeket és interfészeket ad, a megértés gyorsabb és olcsóbb. Ha a rendszer egy gubanc, akkor minden egyes módosítás régészeti feltárással jár, és a hibák egy része abból fakad, hogy a fejlesztő nem látta át a kapcsolatok teljes hálóját.

A minőség romlása is költséggel jár. Egy rosszul strukturált rendszerben a tesztelés lassabb, a hibák reprodukálása nehezebb, a telepítés kockázatosabb. Az ad hoc architektúra rövid távon gyorsabbnak tűnhet, mert nem kell „tervezni”, de ez a megtakarítás később kamatostul jelentkezik a hibaelhárítás, az éjszakai riasztások, az elveszett ügyfélbizalom formájában.

Fontos azonban, hogy az architektúra sem ingyen van. A túltervezés – a szükségtelen absztrakciók, a felesleges mikroszolgáltatások, a spekulatív általánosítás – szintén költséget termel. A jó architektúra ezért nem az, amelyik a legtöbb réteget és a legokosabb mintákat használja, hanem az, amelyik a rendszer valós követelményeihez és a csapat valós képességeihez mérten találja meg az egyensúlyt. A költségoldalról nézve az architektúra olyan befektetés, amelynek *megtérülése* a későbbi változtatások olcsóságában és a rendszer stabilitásában mutatkozik meg.

2.2.4 2.1. A késői változtatás ára

A szoftverfejlesztés klasszikus tapasztalata, hogy egy hiba vagy egy rossz döntés annál drágábban javítható, minél később derül fény rá. Egy követelmény félreértése a tervezés elején egy átbeszéléssel orvosolható; ugyanaz a félreértés a gyártásban már adatvesztéssel, incidenssel, ügyfélkárral jár. Az architektúra ebben a folyamatban sajátos helyet foglal el: a szerkezeti döntések gyakran a követelmények és a kód közötti térben születnek, és hatásuk a rendszer teljes élettartamán átível.

Vegyük például a szolgáltatások közötti szinkron hívások kérdését. Ha egy rendszerben minden szolgáltatás közvetlenül, szinkron módon hívja a másikat, akkor az első változat gyorsan elkészül, mert a hívási lánc egyszerű. Később azonban egyetlen lassú szolgáltatás is magával ránthatja az egész láncot, a hibaterjedés miatt pedig a hibaelhárítás egyre nehezebb. Amikor a rendszer mérete elér egy kritikus pontot, a csapat rájön, hogy aszinkron üzenetkezelésre, áramkör-megszakítókra, időtúllépésekre és újrapróbálkozási logikára van szükség. Ezek a változtatások nem egy-két osztály átírását jelentik, hanem a komponensek közötti szerződések, a hibakezelés és az üzemeltetési eljárások újragondolását. A késői változtatás ára tehát nem egyszerűen a kód mennyiségével arányos, hanem a szerkezeti kapcsolatok számával és mélységével is.

Ez a felismerés vezet ahhoz a gyakorlathoz, hogy a nagy hatású döntéseket érdemes korán, tudatosan és visszafordítható módon megvizsgálni. Az architektúra nem arról szól, hogy az első verzió tökéletes legyen – ez ritkán lehetséges –, hanem arról, hogy a nagy hatású döntések ne véletlenül, hanem szándékosan szülessenek meg.

2.2.5 2.2. A technikai adósság mint szerkezeti jelenség

A technikai adósságot gyakran a kód szintjén képzeljük el: rosszul elnevezett változók, duplikált logika, hiányzó tesztek. Az adósságnak azonban van egy mélyebb, szerkezeti rétege is. Ha a rendszer komponensei nem felelnek meg a valós üzleti határoknak, ha minden komponens minden másikkal beszél, ha a perzisztencia réteg összefonódik az üzleti logikával, akkor minden új funkció növeli a szerkezeti adósságot. Az ilyen adósságot nem lehet pusztán a kód átírásával törleszteni; a határok újrarajzolására van szükség.

A szerkezeti adósság a költség oldalán azért alattomos, mert a tünetei lassan jelentkeznek. A sebesség először csak kicsit csökken, a hibák száma először csak kicsit nő, a tesztelés először csak kicsit lassul. Ezután a folyamatok egymást erősítik: a csapat a lassulásra nyomás alatt a tesztek csökkentésével válaszol, ami több hibához vezet, ami még több tűzoltást igényel, ami még kevesebb időt hagy a szerkezeti munkára. Az architektúra tudatos kezelése ennek a spirálnak az ellenszere: olyan döntéseket hoz, amelyek a rendszer szerkezetét a változás irányához igazítják, és olyan mérőeszközöket ad, amelyekkel a romlás még azelőtt láthatóvá válik, hogy kritikussá válna.

2.2.6 3. Az architektúra hatása a szervezeti képességre

Az architektúra nem csak a kód szerkezetéről szól; a szervezet működéséről is. Conway törvénye szerint a rendszerek szerkezete leképezi a szervezet kommunikációs szerkezetét. Aki szervezett rendszert akar, annak a szervezetet is meg kell értenie; aki szervezeti változást akar, annak az architektúrával is foglalkoznia kell. Ezt a témát a későbbi fejezetek részletesen kifejtik, itt elég annyi, hogy az architektúra és a szervezet kölcsönhatása már a legelején is meghatározza a sikert.

A szervezeti képesség itt több dolgot jelent. Egyrészt azt, hogy a csapat milyen gyorsan tud új funkciókat szállítani. Ha a rendszer szerkezete lehetővé teszi, hogy több csapat egymástól függetlenül dolgozzon, akkor a szállítási sebesség nő. Ha a rendszer egyetlen nagy, osztatlan kódbázis, akkor a csapatok egymásra várakoznak, a kódbázis áttekinthetetlenné válik, és a szállítási sebesség csökken. Másrészt azt, hogy a szervezet mennyire tud tanulni a rendszerből. Ha az architektúra nem dokumentált, akkor a tudás a fejlesztők fejében él, és az emberek távozásával elveszik. Ha az architektúra döntései nyilvánosak, verziózottak és ellenőrizhetők, akkor a tudás szervezeti szinten halmozódik fel.

Az architektúra a szervezeti képességet a *döntési sebességen* keresztül is befolyásolja. Ha a rendszer határai tiszták, a csapatok anélkül hozhatnak helyi döntéseket, hogy minden apró kérdésben központi jóváhagyást kellene kérniük. Ha a határok homályosak, minden döntés globális egyeztetést igényel, mert senki nem tudja, hogy egy változtatásnak milyen továbbgyűrűző hatásai lesznek. A jól megtervezett architektúra tehát nem korlátozza, hanem éppen lehetővé teszi a szervezeti autonómiát: a komponensek felelőssége és interfésze világos, ezért a komponens belsejében a csapat szabadon dolgozhat.

2.2.7 4. A korai döntések hosszú távú következményei

Az eddigiekből következik, hogy a korai döntéseknek különleges súlya van. De milyen döntések számítanak korainak? Nem feltétlenül azok, amelyeket a projekt első napjaiban hozunk, hanem azok, amelyek *a rendszer szerkezetét hosszú időre meghatározzák*, és amelyek megváltoztatása aránytalanul drága. Ilyen döntések például:

- **A rendszerhatárok kijelölése.** Hol ér véget a saját rendszerünk, és hol kezdődnek a külső rendszerek? Milyen felhasználói szerepkörök lépnek kapcsolatba a rendszerrel? Ez a döntés meghatározza a későbbi interfészeket, a biztonsági követelményeket és a tesztelési stratégiát.
- **A komponensek felelősségének elosztása.** Melyik komponens felel a rendelések kezeléséért, melyik az ügyféladatokért, melyik a számlázásért? Ha ez az elosztás nem tükrözi a valós üzleti folyamatokat, akkor minden későbbi funkció feszültséget szül.
- **A kommunikációs minták megválasztása.** Szinkron vagy aszinkron? Kérés-válasz vagy eseményvezérelt? A választás hatással van a rendelkezésre állásra, a konzisztenciára, a hibaterjedésre és a fejlesztési modellre.
- **Az adatmodell és a perzisztencia szerkezete.** Ki az adat tulajdonosa? Milyen tranzakciós határok léteznek? Az adatok hol élnek, és ki férhet hozzájuk? Ez a döntés a rendszer egyik legnehezebben változtatható része.
- **A bizalmi határok kijelölése.** Mely komponensek bízhatnak meg egymásban, és melyek nem? Hol kell hitelesítés és engedélyezés? Ez a biztonsági architektúra alapja.

Ezek a döntések nem azért fontosak, mert a többi döntés lényegtelen, hanem azért, mert ezek hatása *nemlineáris*. Egy rosszul kijelölt komponenshatár minden egyes új funkciónál újabb költséget termel; egy jó határ minden új funkciónál megtakarítást hoz. A korai döntések tehát nem egyszeri költségek, hanem olyan döntések, amelyek a rendszer élettartama során folyamatosan kamatoznak – vagy éppen folyamatosan adóztatnak.

2.2.8 4.1. A visszafordíthatóság mint tervezési elv

A korai döntések súlyából sokan azt a következtetést vonják le, hogy a tervezés elején mindent pontosan el kell dönteni. Ez azonban félreértés. A jó architektúra nem az, amelyik a legpontosabban megjósolja a jövőt, hanem az, amelyik a lehető legtöbb döntést *visszafordíthatóvá* teszi. A visszafordíthatóság azt jelenti, hogy egy döntés megváltoztatásának költsége alacsony marad, mert a rendszer nem épült rá mereven az adott döntésre.

A gyakorlatban ez azt jelenti, hogy a nagy hatású döntéseket érdemes jól elszigetelni. Ha egy adatbázis-technológia mögé saját interfészt teszünk, akkor a technológia cseréje kevésbé fájdalmas. Ha egy külső szolgáltatást egy adapter mögé rejtünk, akkor a szolgáltató váltása nem borítja fel a rendszert. Ha a kommunikációs mintát egy jól definiált szerződés mögött tartjuk, akkor a szinkron hívásból később aszinkron üzenetkezelés alakítható ki anélkül, hogy a teljes rendszert át kellene írni. A visszafordíthatóság tehát nem a tervezés elkerülése, hanem éppen a tervezés egyik legfontosabb eszköze: olyan szerkezetet épít, amelyben a korai döntések tévedései sem válnak katasztrófává.

2.2.9 5. Az architektúra mint kockázatkezelés

Az eddigiekből következik, hogy az architektúra egyik legfontosabb funkciója a kockázatkezelés. A kockázat itt olyan jövőbeli esemény, amely negatívan hat a rendszer minőségére, költségére vagy a szervezet képességére. Az architektúra úgy kezeli a kockázatot, hogy a rendszer szerkezetét úgy alakítja, hogy a kockázatok bekövetkezésének valószínűsége csökkenjen, vagy a hatásuk enyhüljön.

A kockázatkezelés első lépése a kockázatok azonosítása. Melyik minőségi attribútum sérülhet? Mi történik, ha a rendszer terhelése a többszörösére nő? Mi történik, ha egy külső szolgáltatás elérhetetlenné válik? Mi történik, ha egy jelszótároló kiszivárog? Mi történik, ha a csapat fele egyszerre távozik? Ezek a kérdések nem a kód, hanem a szerkezet szintjén vizsgálhatók: milyen

komponensek érintettek, milyen kapcsolatokon keresztül terjed a hiba, milyen tartalékok állnak rendelkezésre.

A kockázatkezelés második lépése a kockázatok rangsorolása. Nem minden kockázat egyforma. Egy olyan kockázat, amelynek bekövetkezése valószínűtlen, de hatása katasztrofális, más kezelést igényel, mint egy olyan, amely gyakran bekövetkezik, de hatása kicsi. Az architektúra döntései ezt a rangsorolást tükrözik: a nagy kockázatú területekre több tervezési figyelmet, több tartalékot, több mérést fordítunk.

A kockázatkezelés harmadik lépése a kockázati intézkedések beépítése a szerkezetbe. Ez lehet redundancia a rendelkezésre állás növelésére, áramkör-megszakító a hiba terjedésének megállítására, titkosítás a bizalmasság védelmére, verziózott interfész a változások kezelésére, vagy éppen a monolitikus szerkezet megtartása, ha a csapat mérete és a rendszer bonyolultsága nem indokolja a szétbontást. A kockázatkezelés nem azt jelenti, hogy minden kockázatot megszüntetünk – ez lehetetlen –, hanem azt, hogy a kockázatokat tudatosan vállaljuk, mérjük és kezeljük.

Az architektúra mint kockázatkezelés szemléletnek fontos következménye, hogy a döntéseket a kockázat szempontjából kell értékelni, nem pedig a divat szempontjából. Egy technológia attól még nem jó választás, hogy népszerű; attól jó, hogy a rendszer kockázatait csökkenti. Egy minta attól még nem hasznos, hogy a konferencia-előadásokban szerepel; attól hasznos, hogy a konkrét kontextusban a kockázatot mérsékli.

2.2.10 5.1. A kockázat láthatóvá tétele: döntési napló és minőségi forgatókönyvek

A kockázatkezelés akkor működik, ha a kockázatok és a rájuk adott válaszok láthatók. Ebben a könyvben két eszközt használunk erre: a döntési naplót (ADR) és a minőségi forgatókönyveket (quality attribute scenarios). A döntési napló rögzíti, hogy egy adott architektúrális döntés milyen kontextusban született, milyen alternatívákat vizsgáltunk, és milyen következményeket vállaltunk. A minőségi forgatókönyv pedig mérhető formában írja le, hogy a rendszernek hogyan kell viselkednie egy adott ingerre: mi történik, ha a terhelés egy órán át a normál kétszerese, mi a válasz, és milyen mérőszámmal ellenőrizzük.

Ez a két eszköz együtt azt a célt szolgálja, hogy a kockázat ne maradjon a fejlesztők fejében, hanem a szervezet közös tudásává váljon. Amikor a kockázat látható, akkor lehet róla tárgyalni, lehet rá költségvetést tervezni, és lehet a döntéseket utólag felülvizsgálni. Amikor a kockázat láthatatlan, akkor a kudarc mindig meglepetésként érkezik.

2.2.11 6. Sikertörténetek és kudarctörténetek: miből tanulunk?

Az architektúra fontosságát talán a legjobban a valós rendszerek történetei mutatják. A sikerek és a kudarcok ritkán egyetlen okra vezethetők vissza, de a szerkezeti döntések szinte mindig meghatározó szerepet játszanak. A siker gyakran az, hogy a rendszer szerkezete időben alkalmazkodott a változó követelményekhez; a kudarc gyakran az, hogy a szerkezet merevvé vált, és a változás már csak akkor volt lehetséges, amikor a rendszer összeomlott.

Vegyünk egy tipikus kudarctörténetet: egy vállalat saját fejlesztésű, monolitikus rendszert üzemeltet, amely évek alatt nőtt össze. A kezdeti siker után a változtatások egyre lassabbak, a hibák egyre gyakoribbak. A csapatok egymásra várnak, a telepítések éjszakába nyúlnak, az üzleti igényeket hónapokig nem tudják kiszolgálni. Amikor a vezetés a mikroszolgáltatások felé fordul, a csapat a monolit technikai szétbontásába kezd, de a komponensek közötti függőségek olyan szövevényesek, hogy a szétválasztás évekig tart, és közben a rendszer tovább romlik. A kudarc okai között ott van a szerkezeti adósság, a komponenshatárok hiánya, a kommunikációs

minták rendezetlensége és a kockázatok láthatatlansága. Ezt a történetet sokan átélték, és az architektúra tudatos kezelése nélkül nehéz lett volna elkerülni.

A sikertörténetek más képet mutatnak. Egy olyan csapat, amely korán meghúzta a rendszerhatárokat, világos felelősségű komponenseket épített, és a szállítási folyamatot a szerkezethez igazította, gyorsabban tudott növekedni, és a változások olcsóbbak maradtak. A siker nem az, hogy nem voltak hibák, hanem az, hogy a hibákat a rendszer szerkezete korlátozta, és a csapat gyorsan tudott tanulni belőlük.

2.2.12 6.1. Labor: egy kudarc- és egy sikertörténet architektúrális elemzése

A fejezet laborja azt a képességet gyakoroltatja, hogy a fenti szempontokat konkrét, reprodukálható elemzésben alkalmazd. Az átadandó eredmény egy dokumentum vagy jegyzetfüzet, amelyben egy kudarc- és egy sikertörténetet elemzel az architektúrális okok szempontjából.

Kiinduló állapot. Válassz két nyilvánosan dokumentált esetet. Fontos, hogy a források hozzáférhetők legyenek, és a választás kontextusát rögzítsd: milyen rendszer, milyen időszak, milyen források alapján dolgozol. Ha nem nyilvános esetet választasz, akkor a saját szervezetedben is dolgozhatsz, de a dokumentumban álnevesíteni kell az adatokat, és nem szerepelhet valós belső URL, ügyfélnév vagy személyes adat.

Elvárt eredmény. A dokumentum a következőket tartalmazza:

1. **Kontextus rögzítése.** Mi a rendszer célja, kik a felhasználói, milyen külső rendszerekkel van kapcsolatban, milyen időtávon vizsgálod?
2. **Döntési napló.** Legalább három, az esethez kapcsolódó architektúrális döntést azonosíts, és mindegyikhez írd le: mi volt a döntés, milyen kontextusban született, milyen alternatívák merültek fel, és milyen következményekkel járt. Használd az ADR formátumot: kontextus, döntés, alternatívák, következmények, állapot.
3. **Minőségi hatás.** Azonosíts legalább két minőségi attribútumot, amelyekre a döntés hatott. Mindegyikhez fogalmazz meg egy-egy minőségi forgatókönyvet: inger, környezet, válasz, mérőszám.
4. **Siker- vagy kudarcmintázat.** Írd le, hogy a döntések együttesen hogyan vezettek sikerhez vagy kudarchoz. Melyik döntés volt visszafordítható, melyik nem? Milyen kockázatokat kezelt vagy éppen hagyott fedetlenül az architektúra?
5. **Tanulság.** Fogalmazz meg legalább három olyan tanulságot, amely a saját gyakorlatodra átvihető. Mindegyik tanulságnál jelezd, hogy milyen kontextusban érvényes, és hol lehet a határa.

Ellenőrzési szempontok. Az elkészült dokumentum akkor jó, ha:

- a választott esetek kontextusa egyértelműen rögzített, és a források azonosíthatók;
- a döntési naplóban a döntések nem csupán technikai részletek, hanem szerkezeti következményekkel bíró architektúrális döntések;
- a minőségi forgatókönyvek mérhető mérőszámot tartalmaznak, nem csupán általános kívánságokat;
- a siker vagy kudarc elemzése a döntések következményeiből vezeti le az eredményt, nem pedig utólagos ítéletet mond;
- a dokumentum elkülöníti a mért/ellenőrzött adatokat a várt/értelmezett adatoktól;
- a tanulságok kontextushoz kötöttek, és jelzik a határaikat.

Visszaállítási út. Ha a laborhoz adatokat, kódot vagy jegyzetfüzetet készítettél, a kiinduló állapot visszaállítható a verziókezelő rendszerből. Ha külső forrásokból dolgoztál, a források URL-jét és elérési dátumát rögzítsd a dokumentumban, hogy az elemzés később reprodukálható legyen.

2.2.13 6.2. Példa a labor egy részletére

Az alábbi részlet szemlélteti, hogyan nézhet ki egy döntési naplóbejegyzés a laborban. A példa fiktív, de a szerkezete valós elemzésben is használható.

ADR-példa: a „Rendeléskezelő” rendszer értesítési mechanizmusa

- **Kontextus:** a rendszer a rendelések rögzítése után e-mail-értesítést küld a vásárlónak. Az első verzióban az értesítés szinkron módon, a rendelésfeladás tranzakcióján belül történt. A növekvő terhelés mellett az e-mail-küldő szolgáltatás lassulása a rendelésfeladást is lassította.
- **Döntés:** a rendelésfeladás és az értesítés szétválasztása aszinkron módon, üzenetsoron keresztül. A rendelés rögzítése a tranzakcióban megtörténik, majd az értesítési esemény az üzenetsorra kerül.
- **Alternatívák:** (1) a szinkron hívás megtartása timeouttal; (2) háttérszálak használata ugyanabban a folyamatban; (3) aszinkron üzenetsor bevezetése. Az (1) és (2) kisebb változtatással jár, de nem oldja meg a hibaterjedést, és a terhelés növekedésével újra szűk keresztmetszetet okoz.
- **Következmények:** a rendelésfeladás rendelkezésre állása javul, de a rendszer konzisztenciája lazább: előfordulhat, hogy a rendelés rögzül, de az értesítés később meghiúsul. Ezért az értesítési folyamathoz újrapróbálkozási logikát és hibaszámlálót kell építeni.
- **Állapot:** elfogadva.

A példából látszik, hogy a döntés nem egyszerűen technikai részlet, hanem a minőségi attribútumok – rendelkezésre állás és konzisztencia – közötti trade-offot kezeli. A döntés következménye pedig újabb szerkezeti elemeket von be: üzenetsor, újrapróbálkozási logika, hibaszámláló.

2.2.14 7. Az architektúra értékének összefoglalása

Az architektúra tehát nem a dokumentumok halmaza, nem a diagramok gyűjteménye, és nem a szép elnevezések rendszere. Az architektúra a rendszer szerkezetéről szóló döntések összessége, amelyek meghatározzák, hogy a rendszer milyen minőségben, milyen költséggel és milyen szervezeti feltételek mellett lesz képes betölteni a szerepét. Ez a meghatározás azért fontos, mert a gyakorlatban az architektúra értékét gyakran addig nem látjuk, amíg a hiányát nem érezzük.

A hiánya akkor válik érezhetővé, amikor egy egyszerű változtatás heteket vesz igénybe, amikor a rendszer egyetlen ponton elakad, amikor a biztonsági rés miatt az egész adatbázis kompromittálódik, amikor a csapat nem tud párhuzamosan dolgozni, mert mindenki ugyanazokon a fájlokon ütközik. Az architektúra jelenléte ellenben akkor válik érezhetővé, amikor a rendszer növekszik, és a fejlesztés sebessége nem zuhan; amikor a hiba bekövetkezik, de a hatása korlátozott marad; amikor a csapat új tagjai gyorsan megértik a rendszert; amikor a technológiai változás nem borítja fel az egész szerkezetet.

Ebben a fejezetben azt vizsgáltuk, hogy az architektúra miért fontos a minőség, a költség és a szervezeti képesség szempontjából. Láttuk, hogy a korai döntések hosszú távú következményekkel járnak, és hogy az architektúra egyik központi feladata a kockázatkezelés. A laborban ezt a szemléletet gyakorolhatod egy konkrét eset elemzésén keresztül. A következő fejezet az

architektúra történetére és evolúciójára fókuszál, hogy a mai gyakorlatot a maga történeti összefüggésében is el tudd helyezni.

2.2.15 8. Ellenőrzőlista

A fejezet tanulási céljainak ellenőrzéséhez használd az alábbi listát:

- ☐ Meg tudom magyarázni, hogyan hat az architektúra a minőségi attribútumokra, és miért nem elegendő a kód minősége a minőség biztosításához.
- ☐ Fel tudom sorolni, milyen módon befolyásolja az architektúra a fejlesztési, üzemeltetési és továbbfejlesztési költségeket.
- ☐ Tudok példát mondani arra, hogyan hat az architektúra a szervezeti képességre, például a csapatok önállóságára és a döntési sebességre.
- ☐ Meg tudom indokolni, miért van különleges súlya a korai, nehezen visszafordítható döntéseknek.
- ☐ Értem, mit jelent az architektúra mint kockázatkezelés, és tudok példát mondani kockázatcsökkentő szerkezeti döntésre.
- ☐ El tudom készíteni a laborban előírt elemzést a megadott szerkezetben, és el tudom különíteni a mért adatokat a vártaktól.

2.2.16 9. Önellenőrző kérdés

Válassz egy általad jól ismert rendszert, és azonosíts benne egy olyan architektúrális döntést, amely hosszú távú következményekkel járt. Írd le röviden: mi volt a döntés, milyen minőségi attribútumra hatott, és miért volt nehéz vagy könnyű később megváltoztatni? A válaszodban használd a fejezetben bevezetett fogalmakat: minőségi attribútum, trade-off, visszafordíthatóság, kockázatkezelés.

A kérdésre adott jó válasz nem egyetlen helyes megoldást vár, hanem azt, hogy a fogalmakat a konkrét esethez tudd kötni. Ha a válaszodban meg tudod nevezni a döntést, a minőségi attribútumot, a trade-offot, a visszafordíthatóság mértékét és a kockázatot, akkor a fejezet lényegét elsajátítottad.

2.2.17 10. Források és további tájékozódás

A fejezetben tárgyalt fogalmak hátteréhez az alábbi források nyújtanak további támpontot. A források verzióját, dátumát és konkrét szakaszát mindig az adott kiadásban ellenőrizd; a könyv nem jogi vagy szabványügyi tanácsadás.

- ISO/IEC/IEEE 42010: Systems and software engineering – Architecture description. A szabvány az architektúra leírásának kereteit rögzíti, és a stakeholder, nézet és nézőpont fogalmát tisztázza.
- Bass, Clements, Kazman: Software Architecture in Practice, 4. kiadás. A minőségi attribútumok, a trade-offelemzés és az architektúra értékének klasszikus tárgyalása.
- Martin Fowler: Architecture. A fogalom gyakorlati értelmezése, különösen a „miért fontos az architektúra” kérdésben.
- A CNCF és a felhőszolgáltatók well-architected keretei. A kockázatkezelés és a minőségi attribútumok gyakorlati alkalmazásához, mindig a hivatalos dokumentáció aktuális verzióját ellenőrizd.

Ezek a források nem helyettesítik a fejezet tanulási útját, hanem a mélyebb elmélyülést szolgálják. A következő fejezetben az architektúra történetét és evolúcióját vesszük végig, hogy a jelenlegi gyakorlat miértjeit jobban értsük.

2.3 A szoftverarchitektúra története és evolúciója

A szoftverarchitektúra története nem technikatörténeti kronológia, hanem döntések láncolata. A mainframe-től a monoliton, a szolgáltatásorientált architektúrán (SOA) és a felhőn át a mesterséges intelligencia alapú rendszerekig ugyanazok a feszültségek térnek vissza: centralizáció és decentralizáció, csatolás és kohézió, irányítás és autonómia. Ezt a folyamatot gyakran eltakarja a technológiai hype, ezért a fejezet célja, hogy a tartós elveket és az iparági ciklusokat tanítsa, ne pedig korszaktáblázatot memorizáltasson.

Tanulási célok

A fejezet elvégzése után Ön képes lesz:

- elmagyarázni, hogyan jutottunk el a mainframe-től a monolitikus rendszereken, a SOA-n és a mikroszolgáltatásokon át a felhőalapú és AI/ML rendszerekig;
- alkalmazni a fejezet eszközeit egy kis, ellenőrizhető példán: architektúratörténeti idővonalat készíteni dokumentált kontextussal, döntési naplóval és ellenőrzési szempontokkal;
- felismerni a tipikus hibákat – például a címkék és a tényleges architekturális problémák összekeverését, vagy a hype-ot a tartós elvvel való felcserélését –, és megindokolni a helyes megoldást.

2.3.1 1. A mainframe-korszaktól a modularitásig

Az 1960-as és 1970-es évek meghatározó architekturális alakzata a mainframe. A felhasználók terminálokon keresztül csatlakoztak egy központi, drága erőforráshoz, ahol a memória, a processzoridő és a háttértár is megosztott volt. A rendszerhatár gyakran egybeesett a gépterem fizikai határával: ami a gépen belül volt, az ellenőrzött környezetben működött, ami kívül, az igényelte a hálózati protokollokat és a terminál-illesztést.

A mainframe nem csupán hardver volt, hanem architekturális minta is: központi adatkezelés, tranzakció-feldolgozás és kötegelt (batch) futtatás. A CICS- vagy IMS-típusú rendszerekben a tranzakciók rövid, jól definiált műveletek voltak, a feldolgozás pedig előre ütemezett ablakokban zajlott. Egy tipikus minőségi forgatókönyv a korszakból:

- **inger:** megnyílik az éjszakai zárási batch-ablak;
- **környezet:** a központi rendszer normál terheléssel, az online terminálok alacsony forgalmával;
- **válasz:** a rendszer feldolgozza az 50 000 számlatranzakciót és elkészíti a napi záró állományt;
- **mérőszám:** a batch-átfutási idő kevesebb mint 2 óra, a tranzakciók 99,9%-a sikeresen lezárul.

Az akkori megoldás sok tekintetben racionális volt: a hardver drága volt, ezért a központosítás csökkentette a fajlagos költséget, a kiszámítható batch-üzem pedig mérhetővé tette a rendelkezésre állást. A monolit szó ekkor még nem pejoratív kategória volt, hanem egyszerűen az üzemeltetési valóság leírása. A korai nagy számítógépes rendszerekben azonban már megjelent a modularitás és a felelősségi határok iránti igény is: a parancsok, az adatállományok és a felhasználói felületek szétválasztása a későbbi rétegzés előfutára.

2.3.2 2. Kliens-szerver, hálózat és a rétegek születése

Az 1980-as évekre a személyi számítógépek és a lokális hálózatok (LAN) elterjedése átrendezte a rendszerhatárokat. A feldolgozás egy része a kliens oldalára került, a központi adatbázist pedig dedikált szerver szolgálta ki. A két- majd háromrétegű architektúrában tipikus kérdés lett, hová kerüljön az üzleti logika: a kliensbe, a tárolt eljárásokba vagy esetleg egy középső rétegbe.

A kliens-szerver korszak egyik visszatérő mintája a **feldolgozási hely ingadozása**. A vastag kliens (fat client) gyors, gazdag felhasználói élményt adott, de a telepítés, a frissítés és a verziókövetés komoly üzemeltetési terhet jelentett. A vékony kliens központosította az üzleti logikát, ám a hálózat és a szerver kapacitása hamar szűk keresztmetszetté vált. A megoldás – ahogy sok későbbi korszakban is – a rétegek explicit szétválasztása és a felelősségek írásba foglalása lett.

A háromrétegű architektúra megjelenése nem pusztán technológiai újítás volt, hanem a minőségi attribútumok erőviszonyának megváltozása. A karbantarthatóság és a skálázhatóság fontosabbá vált, mint a közvetlen erőforrás-hatékonyság. A rétegek közötti interfészek megszilárdulása lehetővé tette, hogy a megjelenítést, az üzleti logikát és az adatelérést külön fejlesszék, teszteljék és – bizonyos határok között – külön cseréljék.

2.3.3 3. A világháló és a rétegezett monolit kiforrása

Az 1990-es évek közepétől a HTTP protokoll és a böngésző új rendszerhatárt hozott: a felhasználói interfész egy szabványosított, vékony kliensben futott, a szerveroldali alkalmazások pedig alkalmazásszervereken üzemeltek. A Java EE és a .NET keretrendszerei formalizálták a rétegezett architektúrát: megjelenítés, alkalmazáslogika, szolgáltatásréteg, adatelérési réteg.

Ebben az időszakban a monolitikus webalkalmazás vált a leggyakoribb kiindulóponttá. A monolit ekkor már nem központi mainframe-et jelentett, hanem egyetlen – gyakran nagy méretű – telepítési egységet, amelyen belül a modulok nyelvi és keretrendszer-szintű határokkal különültek el. A jól megtervezett monolitban a belső modulok közötti interfészek éppen olyan fontosak voltak, mint később a szolgáltatások közötti API-k.

A webes korszak fontos tanulsága, hogy a **modularitás és a telepítési egység mérete nem ugyanaz**. Egy monolit lehet kiválóan moduláris vagy teljesen összefonódott. Aki a monolitot a modularitás hiányával azonosítja, az összekeveri a kategóriát a minőségi jellemzővel. A rétegezett architektúra sem garantálja önmagában a jó modularitást; ha az üzleti logika átnyúlik a rétegeken keresztül, akkor a rétegdiagram csak illúzió.

2.3.4 4. SOA, ESB és az integráció ígérete

A 2000-es évek elején a szolgáltatásorientált architektúra (SOA) célja az volt, hogy a vállalati képességeket újrafelhasználható, hálózaton elérhető szolgáltatásokként tegye közzé. A technikai alapot gyakran XML, SOAP, WSDL és valamilyen vállalati szolgáltatásbusz (Enterprise Service Bus, ESB) adta. A SOA explicit célja volt a laza csatolás, a szolgáltatásszerződések és a folyamatorientált integráció.

A SOA ígérete valós architekturális igényre adott választ: a nagy szervezetekben a szigetszerű alkalmazások közötti integráció költsége egyre nőtt. A szolgáltatáshatárok kijelölése, a kanonikus adatmodell és az üzleti folyamat-orchestráció valós problémákat oldott meg. A korszakból származó minőségi forgatókönyv például:

- **inger:** egy megrendelési rendszer szinkron hívást küld az ügyfélnyilvántartásnak;

- **környezet:** a vállalati ESB normál terheléssel, két adatközpont közötti hálózati késleltetéssel;
- **válasz:** az ügyféladatok visszaérkeznek, és a megrendelés tranzakciója lezárul;
- **mérőszám:** a válaszidő p95 értéke kevesebb mint 500 ms, a hibaarány kevesebb mint 0,5%.

A SOA azonban számos csapdát is hozott. A központi ESB-ből gyakran szűk keresztmetszet és komplex, nehezen tesztelhető middleware-réteg vált. A szolgáltatások közötti erős központi irányítás lelassította a csapatokat, a szemantikai interoperabilitás hiánya pedig felszínre hozta, hogy a technikai csatlakoztatás nem helyettesíti a közös üzleti nyelvet. Ráadásul a hálózati hívások rejtett költségei – késleltetés, hibatűrés, tranzakciós korlátok – sokszor csak éles üzemben derültek ki.

A SOA sorsa kettős tanulságot hagyott maga után. Egyrészt a **szolgáltatáshatárok üzleti, nem pusztán technikai természetűek**. Másrészt a **laza csatolás nem szlogen, hanem mérhető tervezési döntés**: ha a szerződések gyakran változnak, ha a hívási láncok hosszúak, vagy ha a folyamatok központi tranzakciót igényelnek, akkor a csatolás a látszólagos függetlenség ellenére is nagy.

2.3.5 5. Mikroszolgáltatások és a cloud-native fordulat

A 2010-es évek elején a konténerek, a DevOps gyakorlat és a folyamatos szállítás megváltoztatta a megvalósíthatósági feltételeket. A mikroszolgáltatások ígérete az volt, hogy a csapatok autonóm egységekben, egymástól függetlenül fejleszthetnek és telepíthetnek. A szolgáltatáshatárok köré szervezett csapatok csökkentik a kommunikációs és koordinációs terhet, a független telepítés pedig felgyorsítja a változások élesbe juttatását.

A mikroszolgáltatás nem a SOA egyszerű újracímkézése, még ha a szolgáltatás, a szerződés és a lazán csatolt integráció fogalmai átfednek is. A 2010-es években megváltoztak a szervezeti és üzemeltetési feltételek: a felhő erőforrásai rugalmasan bővíthetők, a konténerorchestráció automatizálja a skálázást, a CI/CD-csővezetékek pedig lehetővé teszik a kis, gyakori kiadásokat. A mikroszolgáltatás akkor működik, ha ezek az alapok megvannak; ha nincsenek, akkor a szolgáltatások közötti hálózati és adatkonzisztencia-problémák gyorsan felemésztik az elérhető előnyt.

A mikroszolgáltatások ára jól ismert: hálózati hibatűrés, elosztott nyomkövetés, adatintegráció, szerződéstesztelés, eventual consistency. A **moduláris monolit reneszánsza** éppen azoknak a szervezeteknek a válasza, amelyek felismerték, hogy a csapatuk mérete és a rendszerük komplexitása nem indokolja a hálózati alapú dekompozíciót. A döntés tehát nem „monolit vagy mikroszolgáltatás” kérdés, hanem az adott kontextusban értékelt kompromisszumoké: milyen a csapatméret, a kiadási gyakoriság, a hibatűrési követelmény és az adatkonzisztencia-igény.

2.3.6 6. Serverless, platform engineering és AI/ML rendszerek

A 2020-as évekre a felhőalapú architektúrák érett formái – a serverless függvények, a menedzselt szolgáltatások és a belső fejlesztői platformok – eltolták a működtetési felelősség határait. A platform engineering célja, hogy a fejlesztői csapatok önkiszolgáló módon, „golden path” mentén kapják meg az infrastruktúrát, a megfigyelhetőséget és a telepítést, anélkül hogy minden csapatnak saját Kubernetes-szakértőre lenne szüksége.

Ezzel párhuzamosan a mesterséges intelligencia és a gépi tanulás új architekturális kihívásokat hozott. Egy ML-rendszer tipikus komponensei: adatbetöltő csővezetékek, feature store, tanító

pipeline, modellregiszter, modell-kiszolgálás (serving) és visszacsatolási hurok. Az MLOps gyakorlat a szoftverfejlesztésből ismert verziózást, tesztelést és monitorozást terjeszti ki a modellekre és az adatokra.

A nagy nyelvi modellek (LLM-ek) korszaka tovább formálja az architektúrát. A rendszerhatár gyakran a modell és az őt körülvevő eszközök közé kerül: a visszakereséssel bővített generálás (RAG), az agentek, az eszközhívások és a prompt-kezelés mind külön komponens-felelősségeket jelentenek. A modell mint **nem-determinisztikus komponens** új minőségi forgatókönyveket hoz be: a pontosság, a válaszidő, a biztonság, a fairness és a költség egyszerre lehet kritikus. A hagyományos architekturális elvek továbbra is érvényesek: a komponensek felelősségét, az interfészeket, a hibamódokat és a mérhetőséget itt is expliciten kell definiálni.

2.3.7 7. Iparági ciklusok és visszatérő minták

Az architektúratörténet legfontosabb felismerése, hogy a divatok változnak, de a feszültségek visszatérnek. A leglátványosabb ciklus a **centralizáció és decentralizáció ingamozgása**. A mainframe központosított, a kliens-szerver decentralizált, a webes rétegek ismét központosítottak, a mikroszolgáltatások hálózati alapú decentralizációt hoztak, a felhő központosította az infrastruktúra-üzemeltetést, az edge computing pedig a feldolgozást a peremre tolja. Az inga soha nem áll meg, mert minden megoldás más költséget és más minőségi jellemzőt optimalizál.

Hasonlóan visszatérő minta a **feldolgozási hely megválasztása**. A vastag kliens és a vékony kliens vitája, a tárolt eljárások és az alkalmazáslogika közötti határ, a serverless függvények hidegindítása mind ugyanazt a kérdést veti fel: hol fusson a logika, és mennyibe kerül a hálózati kommunikáció? A rétegződés és a modularitás szintén visszatérő válasz: a rétegek segítenek a gondolkodásban, de önmagukban nem garantálják a csatolás csökkentését.

Az **integrációs komplexitás** is ciklikusan tér vissza. A mainframe idején az állományok és a terminálok összekötése volt a feladat, a kliens-szerver korban az SQL-adatbázisok és a LAN, a SOA idején az ESB és a webszolgáltatások, a mikroszolgáltatásoknál az API-k és az eseményfolyamok. A technológia változik, de a probléma – hogyan beszéljenek egymással a szervezeti egységek rendszerei – változatlan. Aki a technológiától várja a megoldást az üzleti fogalmak egyeztetése nélkül, az a SOA-csapdát ismétli meg.

2.3.8 8. Technológiai hype és tartós elvek

A technológiai hype mechanizmusa jól leírható a Gartner-féle hype-ciklussal: a felfújt várakozások csúcsát csalódás követi, majd a termelékenység platója. A SOA esetében a 2000-es évek közepén a szolgáltatásorientációt érett, mindenre alkalmazható megoldásként hirdették; a csalódás után a fogalom szinte szitokszóvá vált. Ugyanez megismétlődött a mikroszolgáltatásokkal: a korai sikertörténetek után sokan anélkül bontották szolgáltatásokra a rendszerüket, hogy a szervezeti és üzemeltetési feltételeket megteremtették volna.

A hype és a tartós elv megkülönböztetésének kulcsa, hogy a tartós elvek **mérhető, kontextusfüggő döntésekben** fejeződnek ki. A következő alapelvek nem évülnek el, bár a technológiai eszközök igen:

- a rendszerhatár mindig explicit: a diagramon, a szerződésben és az üzemeltetési felelősségben is;
- a komponenseket felelősséggel és interfésszel írjuk le, nem csak névvel;
- a laza csatolás és az erős kohézió egyensúlyát a tényleges változási minták alapján kell értékelni;

- a minőségi követelményeket forgatókönyvként kell megfogalmazni: inger, környezet, válasz, mérőszám;
- a döntéseket ADR-ben rögzítsük: kontextus, döntés, alternatívák, következmények, állapot;
- a fokozatos migráció és a strangler fig elve gyakran biztonságosabb, mint a teljes újraírás;
- a szervezeti és a technikai architektúra nem választható el egymástól;
- a mérés bizonytalanságát és a nem reprodukált állításokat jelölni kell.

A „best practice" kifejezés itt is csak kontextussal együtt értelmes. A mainframe-korszakban a központosítás jó gyakorlat volt; a 2020-as években ugyanez a központosítás az edge AI rendszerekben hibás lehet. A hype akkor válik veszélyessé, ha a döntéshozó a címkét választja a mérlegelés helyett.

2.3.9 9. A történeti minták alkalmazása a mai döntésekben

A fejezet eszközeinek gyakorlati célja, hogy a mai döntéseket történeti kontextusba helyezzük. Amikor egy csapat mikroszolgáltatásokról, serverlessről vagy LLM-ügynökökről dönt, érdemes feltenni a következő kérdéseket:

- Melyik korábbi ciklus ismétlődik most: centralizáció vagy decentralizáció, vastag vagy vékony kliens, központi vagy elosztott integráció?
- Milyen minőségi forgatókönyvek változnak meg, és milyen mérőszámmal tudjuk ezt igazolni?
- A javasolt technológia tartós elvet szolgál, vagy csak a jelenlegi hype része?
- A szervezeti feltételek – csapatméret, kiadási gyakoriság, üzemeltetési érettség – adottak-e a választott mintához?

A válaszokat célszerű ADR-ben rögzíteni, mert így a jövőbeli olvasó is látja a döntés okát és a feladott alternatívákat. Az ADR nem a múlt adminisztrálása, hanem a szervezeti tanulás eszköze.

2.3.10 10. Labor: architektúratörténeti idővonal készítése

A labor célja, hogy a hallgató reprodukálható dokumentumban vagy jegyzetfüzetben készítsen egy architektúratörténeti idővonalat, amely rögzíti a kontextust, a döntési naplót és az ellenőrzési szempontokat. A labor nem igényel éles infrastruktúrát; a destruktív művelet nincs jelen, így a visszaállítás egyszerű verziókezelési művelet.

Kiinduló állapot

- Egy verziókezelt munkamappa vagy notebook, amelyben a dokumentum és a diagramok diagrams-as-code formában tárolhatók.
- A szükséges eszközök: Mermaid vagy PlantUML szerkesztő, ADR-sablon, táblázatkezelő vagy dokumentum-szerkesztő.
- A kiinduló állapotba nem tartozik éles adat; minden példa szintetikus.

Lépések

1. **Rögzítse a kontextust.** Írjon egy bekezdést arról, milyen iparágat, csapatot és időhorizontot vizsgál, és hogy az idővonal milyen rendszerhatárig terjed. A külső rendszereket és a felhasználói szerepköröket a C4 context szintjén tüntesse fel, ha egy-egy csomóponthoz kontextusdiagramot készít.
2. **Készítse el az idővonalat.** A példa egy sematikus Mermaid-diagram, amelyet a hallgató finomít és bővít:

```
flowchart LR
    A[Mainframe + terminál\n1960-1980] --> B[Kliens-szerver\n1980-1995]
    B --> C[Webes rétegek\n1995-2005]
    C --> D[SOA + ESB\n2000-2010]
    D --> E[Mikroszolgáltatások + konténerek\n2010-2020]
    E --> F[Cloud-native + platform\n2020-tól]
    F --> G[AI/ML + LLM-rendszerek\n2020-as évek]
```

Cím: Egyszerűsített architektúratörténeti idővonal
Jelmagyarázat: a nyilak a domináns minták közötti átmeneteket jelzik, nem éles korszakhatárokat; a szakaszok átfedésben vannak.
Feltételjegyzet: a diagram tanulási modell, a valós iparági átmenetek fokozatosak és szervezetfüggők.

3. **Csatoljon minden korszakhoz egy minőségi forgatókönyvet.** A formátum: inger, környezet, válasz, mérőszám. Példa a mainframe-korszakból:

    - **inger:** megnyílik az éjszakai zárási batch-ablak;
    - **környezet:** a központi rendszer normál terheléssel;
    - **válasz:** a rendszer feldolgozza az 50 000 számlatranzakciót;
    - **mérőszám:** átfutási idő < 2 óra, hibaarány $< 0{,}1\%$.

4. **Azonosítson legalább három visszatérő mintát.** Például: centralizált-decentralizált inga, feldolgozási hely megválasztása, integrációs komplexitás. Mindegyikhez írjon egy rövid indoklást, és linkelje az idővonal megfelelő csomópontjához.

5. **Készítsen döntési naplót legalább három ADR-rel.** Mindegyik ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. A döntések a múlt tanulságait modern rendszerre alkalmazzák; például: „a csapat a moduláris monolitot választja a mikroszolgáltatások helyett az adott terhelési és csapatméret mellett”.

6. **Különítse el a mért és a várt állításokat.** A dokumentumban jelölje, mely állítások reprodukált mérésen alapulnak, és melyek hipotézisek vagy szakirodalmi várakozások. A nem futtatott mérést vagy benchmarkot ne mutassa be ellenőrzött eredményként.

Elvárt eredmény

Egy verziózott dokumentum vagy jegyzetfüzet, amely tartalmazza:

- a kontextusleírást;
- a diagrams-as-code formátumú idővonalat;
- a csomópontonkénti minőségi forgatókönyveket;
- a visszatérő minták elemzését;
- az ADR-eket;
- a mért/várt elkülönítést;
- az ellenőrzési szempontokat.

Ellenőrzési szempont

- Az idővonal lefedi a mainframe, kliens-szerver, webes rétegek, SOA, mikroszolgáltatás, cloud-native és AI/ML korszakokat.
- Minden minőségi forgatókönyv a megadott négy elemből áll.
- Az ADR-ek tartalmazzák a kontextust, döntést, alternatívákat, következményeket és állapotot.
- A mért és várt állítások egyértelműen elkülönülnek.
- A diagram címében, jelmagyarázatában és feltételjegyzetében szerepel a szükséges metaadat.

- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

Visszaállítási út

Mivel a labor nem hajt végre destruktív műveletet, a visszaállítás a verziókezelt munkamappa korábbi commitjára való visszatérést jelenti.

2.3.11 11. Gyakorlat

Egy 8 fős csapat egyetlen webes értékesítési rendszert üzemeltet. A rendszer terhelése kiszámítható, a kiadási gyakoriság heti egy, az adatkonzisztencia-követelmény erős. A csapat a konferencia-előadások hatására a mikroszolgáltatásokat tervezi bevezetni.

Készítsen ADR-tervezetet a döntéshez. A kontextusban röviden foglalja össze a fenti feltételeket. A döntésben válasszon a moduláris monolit és a mikroszolgáltatás bevezetése között. Az alternatívákban legalább három lehetőséget mérlegeljen: maradni a jelenlegi monolitban, moduláris monolitot kialakítani, illetve mikroszolgáltatásokra bontani. A következményekben térjen ki a csapatméretre, a kiadási gyakoriságra, az adatkonzisztenciára és az üzemeltetési költségre. Indokolja meg, hogy a fejezetben tárgyalt történeti minták közül melyek támasztják alá a választott döntést.

2.3.12 12. Ellenőrzőlista

- ☐ Az idővonal lefedi a mainframe-től az AI/ML rendszerekig terjedő fő állomásokat.
- ☐ Minden korszakhoz tartozik legalább egy minőségi forgatókönyv a megadott formátumban.
- ☐ A dokumentum azonosítja a visszatérő mintákat, és azokat konkrét korszakokhoz köti.
- ☐ A döntési napló ADR-jei tartalmazzák a kontextust, döntést, alternatívákat, következményeket és állapotot.
- ☐ A mért és várt állítások egyértelműen el vannak különítve.
- ☐ A diagramok diagrams-as-code formában, verziózottan készülnek.
- ☐ A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot.
- ☐ A külső rendszerek és a felhasználói szerepkörök a C4 context szintjén, a rendszerhatár jelölésével szerepelnek.

2.3.13 13. Önellenőrző kérdés

Miért nem tekinthető a mikroszolgáltatás a SOA egyszerű újracímkézésének, és milyen szervezeti, üzemeltetési és technológiai feltételek változtak meg a 2010-es években, amelyek lehetővé tették a hálózati alapú dekompozíció szélesebb körű alkalmazását?

2.3.14 14. Összefoglalás és továbbvezetés

A szoftverarchitektúra evolúciója a mainframe-től a monoliton, a SOA-n, a mikroszolgáltatásokon és a felhőn át az AI/ML rendszerekig nem lineáris haladás, hanem ciklikus újrafelfedezés. A technológiai hype eltakarja, de a tartós elvek – explicit rendszerhatár, komponens-felelősség, interfész, csatolás, kohézió és trade-off elemzés – változatlanok. A fejezet laborja és gyakorlata ezt a szemléletet ülteti át egy ellenőrizhető idővonal és döntési napló formájába.

A következő alfejezet (Alapfogalmak: komponens, kapcsolat, nézet, stakeholder és rendszerhatár) az itt bevezetett alapfogalmakra – komponens, kapcsolat, nézet, stakeholder és rendszerhatár –

épít, és azokat a formális rendszerleírás keretein belül definiálja. A forráskiindulópontok között szereplő ISO/IEC/IEEE 42010, a SEI „Software Architecture in Practice” és a Martin Fowler nevéhez köthető írások a mindenkori kiadásban ellenőrizendők; a fejezet nem helyettesíti a szabvány vagy a hivatalos dokumentáció aktuális verzióját.

2.4 Alapfogalmak: komponens, kapcsolat, nézet, stakeholder és rendszerhatár

A szoftverarchitektúra leírásának akkor van értéke, ha a résztvevők ugyanazt értik a fogalmak alatt. A „komponens”, „interfész”, „nézet” vagy „rendszerhatár” szavak a gyakorlatban gyakran többféle jelentéssel szerepelnek: az egyik csapat osztályt ért komponensen, a másik deployolható szolgáltatást, a harmadik egy szervezeti egységet. Ez a fejezet ezt a közös szókincset rögzíti, és egy kis, ellenőrizhető példán keresztül mutatja be, hogyan lehet az architektúra szerkezetét reprodukálhatóan leírni.

2.4.1 Miért közös szókincs az architektúrában?

Az architektúra nem önmagáért létező dokumentumhalmaz. A célja, hogy a döntéseket kommunikálja, a tervezést ellenőrizhetővé tegye, és a változások hatását előre jelezze. Ehhez azonban a leírás elemeinek egyértelműnek kell lenniük. Ha egy fejlesztő a „komponens” alatt egy Java osztályt ért, az üzemeltető pedig egy Kubernetes Deploymentet, akkor ugyanaz a diagram félrevezető lesz.

A közös szókincs három gyakorlati problémát kezel:

1. **Félreértés a döntéshozatalban.** Amikor egy ADR-ben az áll, hogy „a Rendeléskezelő komponens hívja a Számlázó komponenst”, a döntés csak akkor értelmezhető, ha tudjuk, mi a komponens határa, milyen interfészen keresztül történik a hívás, és milyen hibatűrési elvárások vonatkoznak a kapcsolatra.
2. **Helytelen diagramok.** Sok architektúradiagram úgy készül, hogy dobozokat és nyilakat rajzolunk, de nem rögzítjük, mi a doboz felelőssége, mi a nyíl protokollja, és hol a rendszerhatár. Az ilyen diagramok látszólag informatívak, valójában több kérdést vetnek fel, mint amennyit megválaszolnak.
3. **Hiányzó visszakereshetőség.** A jó architektúra-leírásban a komponensek, interfészek és kapcsolatok a követelményekhez, a döntésekhez és a minőségi forgatókönyvekhez kapcsolódnak. Ehhez a fogalmakat azonos jelentéssel kell használni.

A fejezetben bevezetett fogalmak egy hétköznapi webshop-rendszeren keresztül jelennek meg. A rendszer neve legyen „Rendelőpont”, a szereplői: Vevő, Ügyfélszolgálat, Fizetési szolgáltató, valamint a belső komponensek: Webes felület, Rendeléskezelő, Számlázó, Rendelés adatbázis és Számlázó adatbázis.

2.4.2 Komponens: a felelősség hordozója

A komponens az architektúra leírásának alapegysége. A komponens egy futásidejű vagy logikai elem, amelynek **meghatározott felelőssége** és **meghatározott interfésze** van. Nem technológia, nem csomag, nem osztály. A komponens lehet:

- **futásidejű szolgáltatás**, például egy mikroszolgáltatás,
- **modul**, például egy rendeléskezelő modul egy monolitban,

- **konténer**, például egy adatbázis,
- **külső rendszerek helyettesítője** is, ha a rendszerhatáron kívülről érkező kapcsolatot modellezzük.

A komponens definíciójában a hangsúly a **felelősségen** van, nem a néven. Egy „UserService” nevű doboz önmagában kevés. Azt is tudnunk kell, hogy a felhasználók regisztrálásáért, hitelesítéséért, profilkezeléséért vagy éppen a jogosultságok kezeléséért felelős-e. A felelősség nélküli komponens csak egy név.

Példa komponensleírás a Rendelőpont rendszerben:

Komponens	Felelősség	Szolgáltatott interfész	Igényelt interfész
Rendeléskezelő	Rendelések létrehozása, állapotának kezelése, rendeléstételek érvényesítése	`createOrder`, `getOrder`, `cancelOrder`	`reserveInventory`, `chargeCustomer`, rendelés adatbázis elérése
Számlázó	Számlák kiállítása, fizetési állapotok rögzítése	`createInvoice`, `getInvoice`	rendelés adatok, fizetési szolgáltató API
Rendelés adatbázis	A rendelések tartós tárolása, tranzakciós integritás	SQL	nincs
Webes felület	A Vevő interakcióinak kiszolgálása, adatok megjelenítése	HTTP/JSON	Rendeléskezelő API

A komponens határa akkor jó, ha a felelőssége egyértelműen elválasztható más komponensekétől. A határ meghúzásának szempontjai lehetnek: szervezeti felelősség, eltérő életciklus, eltérő skálázási igény, eltérő adatkezelési szabály, vagy eltérő hibatűrési követelmény.

A komponens **fekete doboz** abban az értelemben, hogy a külvilág csak az interfészén keresztül látja. A belső szerkezete – hogy milyen osztályokból, függvényekből vagy tárolt eljárásokból áll – nem része az architektúra komponensszintű leírásának. Ez a fekete doboz elv teszi lehetővé, hogy a komponenst önállóan cseréljük, teszteljük vagy skálázzuk.

2.4.3 Interfész: a komponens szerződése

Az interfész a komponens **szerződése**. Meghatározza, hogy a komponens milyen műveleteket kínál, milyen bemeneteket fogad, milyen kimeneteket ad vissza, milyen hibákat jelezhet, és milyen szinkron vagy aszinkron hívási módot vár. Az interfész a komponens és a külvilág közötti határ.

Az interfész lehet **szolgáltatott** vagy **igényelt**:

- A **szolgáltatott interfész** az, amit a komponens másoknak kínál. Például a Rendeléskezelő `POST /orders` művelete.
- Az **igényelt interfész** az, amit a komponens másoktól elvár. Például a Rendeléskezelő igényli a készletkezelő `reserveInventory` műveletét.

A szolgáltatott és igényelt interfészek párosítása adja a kapcsolatot a komponensek között. Ez a modell segít elkerülni azt a hibát, hogy csak a hívó oldalt dokumentáljuk, a hívott oldal szerződését nem.

Egy interfész megadásakor a következőket érdemes rögzíteni:

- **műveletek és paraméterek**: a pontos elnevezések, adattípusok, validációs szabályok;
- **elő- és utófeltételek**: milyen állapotban kell lennie a rendszernek a hívás előtt és után;
- **hibák és kivételek**: milyen hibakódokat, üzeneteket adhat vissza;
- **szinkronitás**: szinkron hívás, aszinkron üzenet vagy esemény-stream;
- **verzió**: az interfész verziója és a kompatibilitási garanciák.

A `createOrder` interfész például:

```
POST /orders
Request: { customerId, items: [{sku, quantity}] }
Response: 201 Created, { orderId, status: "PENDING" }
Errors: 400 ValidationError, 409 Conflict, 503 ServiceUnavailable
```

A példából látszik, hogy az interfész nem csak a „boldog utat” írja le. A hibák megadása azért fontos, mert az architektúra döntései között szerepel, hogy a hívó hogyan kezelje a hibákat, és hogyan biztosítsa a tranzakciós vagy legalábbis az idempotens viselkedést.

Az interfész lehet **aszinkron** is. Ebben az esetben a szerződés nem egy kérés-válasz pár, hanem egy üzenetséma, egy eseménytípus, egy topic vagy egy parancs. Az aszinkron interfésznél a szerződés magában foglalja a sorrendiségi garanciákat, a kézbesítési garanciákat, a séma verzióját és az idempotencia elvárást.

2.4.4 Konnektor: a kapcsolat első osztályú polgár

A komponensek közötti kapcsolatot **konnektornak** nevezzük. A konnektor nem egyszerűen egy nyíl a diagramon; olyan architektúrális elem, amely protokollt, adatformátumot, szinkronitást, hibatűrést, biztonsági jellemzőket és irányt hordoz. Ha a komponenseket építőkockáknak tekintjük, a konnektor az a habarcs, amely meghatározza, hogyan működnek együtt.

A konnektor típusa lehet:

- **szinkron HTTP/REST**: a hívó blokkol, amíg a válasz meg nem érkezik; jellemzően JSON, XML vagy protobuf;
- **gRPC**: HTTP/2 alapú, szinkron vagy streaming;
- **SQL kapcsolat**: tranzakciós, állapotmegőrző, alacsony szintű;
- **üzenetsor vagy publish-subscribe**: aszinkron, laza csatolás;
- **esemény-stream**: folyamatos, visszajátszható;
- **fájltranszfer**: kötegelt, nem valós idejű.

A konnektor leírásában legalább a következőket adjuk meg:

- **protokoll és adatformátum**;
- **szinkronitás**: szinkron, aszinkron, streaming;
- **hibatűrési mechanizmusok**: retry, timeout, circuit breaker, bulkhead, idempotencia;
- **irány**: ki hív kit, ki publikál, ki fogyaszt;
- **szerződés**: milyen interfész vagy séma szerint történik a kommunikáció.

A Rendelőpont rendszerben a következő konnektorokat írhatjuk le:

Kapcsolat	Protokoll	Szinkronitás	Hibatűrés
Vevő → Webes felület	HTTPS/JSON	szinkron	timeout 5 s, retry idempotens GET-ekre
Webes felület → Rendeléskezelő	HTTPS/REST	szinkron	timeout 3 s, circuit breaker, retry idempotens POST-ra
Rendeléskezelő → Rendelés adatbázis	SQL/TCP	szinkron	tranzakció, connection pool, timeout 10 s
Rendeléskezelő → Számlázó	AMQP 1.0	aszinkron	at-least-once, idempotens fogyasztó, retry, DLQ
Számlázó → Számlázó adatbázis	SQL/TCP	szinkron	tranzakció, connection pool

A konnektor kiemelése azért fontos, mert a kapcsolatok gyakran több architektúrális döntést hordoznak, mint a komponensek maguk. Egy szinkron hívás aszinkronra cserélése például megváltoztatja a rendszer válaszidejét, konzisztencia-követelményeit és hibatűrését. Ha a kapcsolatot csak nyílként jelöljük, ezek a döntések elvesznek.

A konnektor **hibatűrési jellemzői** akkor válnak kritikussá, amikor a rendszer elosztott. A szinkron hívásnál a timeout határozza meg, hogy a hívó mennyit vár, mielőtt hibát jelez. A retry akkor biztonságos, ha a hívott művelet idempotens; ellenkező esetben duplikátumot okozhat. A circuit breaker arra való, hogy egy ismételten hibázó függőséget gyorsan leválasszon, és ezzel megvédje a hívó erőforrásait.

A konnektorok leírásánál mindig jelölni kell, hogy a kapcsolat **a rendszerhatáron belül** vagy azon **kívül** fut-e. A külső kapcsolatok gyakran más biztonsági és megbízhatósági elbánást igényelnek.

2.4.5 Konfiguráció: a komponens-konnektor gráf

A **konfiguráció** az architektúra leírásában a komponensek és konnektorok elrendezését jelenti. Ez egy olyan gráf, amelyben a csomópontok a komponensek, az élek a konnektorok. A konfiguráció mutatja meg, hogy a rendszer milyen topológiát alkot: rétegezett, csillag, gyűrű, mesh, csővezeték vagy éppen hibrid.

A konfiguráció nem azonos az infrastruktúra-konfigurációval, például a YAML fájlokkal vagy a környezeti változókkal. Itt a szerkezeti összeállításról van szó: melyik komponens melyikhez kapcsolódik, milyen irányban, milyen konnektoron keresztül.

A Rendelőpont rendszer egy lehetséges konfigurációja diagrams-as-code formában:

```
flowchart LR
    Vevő[Vevő] -->|HTTPS/JSON, szinkron| Web[Webes felület]
    subgraph Rendelőpont[Webshop rendszerhatár]
        Web -->|HTTPS/REST, szinkron, 3s timeout| Rendelés[Rendeléskezelő]
        Rendelés -->|SQL/TCP, szinkron| DB[(Rendelés adatbázis)]
        Rendelés -->|AMQP, aszinkron, at-least-once| Számlázó[Számlázó]
        Számlázó -->|SQL/TCP, szinkron| SzlaDB[(Számlázó adatbázis)]
    end
    Rendelés -->|HTTPS/REST, szinkron| Fizető[Fizetési szolgáltató]
```

A diagramból kiolvasható a konfiguráció:

- a Vevő kívül van a rendszerhatáron, és a Webes felületen keresztül lép be;
- a Rendeléskezelő központi szerepet tölt be, mert a Webes felület, a Rendelés adatbázis, a Számlázó és a Fizetési szolgáltató is hozzá kapcsolódik;
- a Számlázó aszinkron üzenetet kap, nem szinkron hívást;
- a Fizetési szolgáltató külső rendszer, ezért a kapcsolat kiemelt biztonsági és hibatűrési figyelmet igényel.

A konfiguráció értéke, hogy megmutatja a **függőségi irányokat** és a **forgalmi utakat**. Ha például a Rendeléskezelő kiesik, a diagram alapján látható, hogy a Vevőtől érkező rendelések és a Számlázó felé menő üzenetek is érintettek. Ha a Fizetési szolgáltató lassul, az hatással lehet a Rendeléskezelő szinkron hívásaira, és ezért a timeout, retry és circuit breaker paramétereket a konnektor leírásában kell keresni.

A konfiguráció egyben a **belső architektúra** topológiáját is meghatározza. A fenti példa egy csillag-központú szerkezetet mutat, ahol a Rendeléskezelő a központi csomópont. Ez a topológia egyszerű, de forgalmi és hibatűrési szempontból kockázatos, mert a központi komponens válik szűk keresztmetszetté. Ezt a kockázatot a konfiguráció ismerete nélkül nehéz felismerni.

2.4.6 Nézet, szempont és architektúraleírás

Egyetlen diagram nem képes az architektúra minden aspektusát bemutatni. A **nézet** (view) egy adott **szempontból** (viewpoint) készített reprezentáció. A **szempont** azt rögzíti, hogy milyen **aggodalmakra** (concern) keresünk választ, és milyen jelöléseket, diagramtípusokat használunk. A **stakeholder** (érintett) az a személy vagy szerepkör, akinek az aggodalmaira a nézet választ ad.

A fogalmak közötti kapcsolat:

Fogalom	Jelentés	Példa
Stakeholder	Érintett, akinek érdeke fűződik a rendszerhez	fejlesztő, üzemeltető, biztonsági vezető, termékmenedzser
Concern	Az érintett aggodalma, kérdése	„Hogyan skálázódik a rendelésfelvétel csúcsidőben?”
Viewpoint	Szempont: a nézet készítésének szabálya	C4 container szint, adatfolyam-nézet, biztonsági nézet
View	Egy konkrét nézet, amely egy szempontot alkalmaz a rendszerre	A Rendelőpont konténerdiagramja

A Rendelőpont rendszerben például:

- A **fejlesztői szempont** a modulok és a függőségek szerkezetét mutatja. A nézet: a komponensdiagram, amelyen a Rendeléskezelő, a Számlázó és az adatbázisok szerepelnek.
- Az **üzemeltetői szempont** a telepítési topológiát és a futásidejű erőforrásokat mutatja. A nézet: a konténerdiagram, amelyen a Kubernetes podok, hálózati szabályok és tárolók látszanak.
- A **biztonsági szempont** az adatáramlást és a bizalmi határokat mutatja. A nézet: az adatfolyam-diagram, amelyen a Vevőtől a Fizetési szolgáltatóig terjedő adatút és a titkosítási pontok szerepelnek.

A gyakorlatban elterjedt a **4+1 modell**, amely öt alapnézetet javasol:

1. **logikai nézet**: a funkcionális követelmények szerinti komponensek és kapcsolataik;
2. **folyamatnézet**: a futásidejű folyamatok, szálak, konkurencia és szinkronizáció;
3. **fejlesztési nézet**: a forráskód szervezése, modulok, csomagok, build egységek;
4. **fizikai nézet**: a telepítés, a fizikai vagy virtuális gépek, a hálózati topológia;
5. **forgatókönyv-nézet**: a többi nézetet összekötő használati esetek, minőségi forgatókönyvek.

A 4+1 modell pedig párosítható a C4 modellel, amely a részletezettséget négy szintre bontja: context, container, component, code. A **context szint** a rendszerhatárt és a külső szereplőket mutatja; a **container szint** a deployolható egységeket és a köztük lévő kommunikációt; a **component szint** a konténereken belüli komponenseket; a **code szint** pedig a konkrét osztályokat, függvényeket vagy táblákat. A C4-nél a context, container és component szint kötelező, a code szint csak indokolt esetben készül.

A nézetek együtt alkotják az **architektúraleírást**. Az architektúraleírás akkor teljes, ha minden fontos stakeholder aggodalmára van legalább egy releváns nézet. A nézetek nem versengenek egymással, hanem kiegészítik egymást: a context diagram a rendszer elhelyezkedését, a container diagram a deployolható egységeket, a component diagram a belső szerkezetet, a code szint pedig a konkrét implementációt mutatja.

2.4.7 Rendszerhatár és stakeholder a gyakorlatban

A **rendszerhatár** az a vonal, amely elválasztja a rendszer belső elemeit a külső rendszerektől és a felhasználói szerepköröktől. A határ minden diagramon jelölendő, mert anélkül nem tudjuk, miért felelős a rendszer, és miért nem.

A Rendelőpont rendszerben a rendszerhatáron belül van a Webes felület, a Rendeléskezelő, a Számlázó és az adatbázisok. A határon kívül van a Vevő, az Ügyfélszolgálat, a Fizetési szolgáltató és az Email szolgáltató. A külső rendszerek és a felhasználói szerepkörök mindig megjelennek a context szinten, mert ezek jelentik a rendszer környezetét.

A rendszerhatár meghúzása nem technikai kérdés csupán, hanem szervezeti és szerződéses döntés is. Ha a Fizetési szolgáltatót a rendszerhatáron belülre vesszük, akkor a PCI DSS követelmények, a felelősségi körök és a hibaelhárítás is más lesz, mint ha külső rendszerként kezeljük. A határ meghúzásának okát ezért érdemes döntési naplóban rögzíteni.

A **stakeholder** azonosítása ugyanilyen fontos. A stakeholderek nem csak a megrendelők és a felhasználók; ide tartoznak a fejlesztők, az üzemeltetők, a biztonsági szakemberek, a jogi megfelelőségért felelősök, a termékmenedzserek és a támogató csapat is. Minden stakeholdernek más aggodalma van, ezért más nézetre van szüksége.

Stakeholder	Elsődleges aggodalom	Releváns nézet
Vevő	gyors válasz, megbízható rendelés	context, használati forgatókönyv
Fejlesztő	tiszta komponenshatárok, tesztelhetőség	component, kód, logikai nézet
Üzemeltető	telepítés, monitorozás, hibatűrés	container, fizikai nézet, runbook
Biztonsági vezető	adatvédelem, támadási felület	adatfolyam, biztonsági nézet
Termékmenedzser	költség, piaci igény, ütemezés	context, minőségi forgatókönyvek

A stakeholder-aggodalom-nézet hármas rögzítése az architektúradokumentáció elején megelőzi azt a hibát, hogy csak a fejlesztői nézőpont jelenik meg, és az üzemeltetői vagy biztonsági szempontok kimaradnak.

2.4.8 Tipikus hibák és javításuk

A fejezetben bevezetett fogalmak használata során több tipikus hiba fordul elő. Ezek felismerése része a tanulási céloknak.

1. A komponens csak név, felelősség nélkül.
Hiba: a diagramon szerepel egy „Rendeléskezelő” doboz, de nem tudjuk, mit csinál.
Következmény: a döntések nem visszakereshetők, a határok nem egyértelműek.
Javítás: minden komponenshez adjunk meg felelősséget és interfészt. Például: „Rendeléskezelő: rendelések létrehozása, állapotkezelése; interfész: `createOrder`, `cancelOrder`”.

2. Az interfész hiányzik.
Hiba: a diagramon a komponensek közötti kapcsolat szerepel, de nem tudjuk, milyen műveletek, paraméterek vagy hibák tartoznak hozzá.
Következmény: a hívó és hívott fél közötti szerződés nem ellenőrizhető, a változások hatása nem jelezhető előre.
Javítás: minden kapcsolathoz adjunk meg interfészt vagy legalább hivatkozást a szerződésre.

3. A konnektor csak nyíl, jellemzők nélkül.
Hiba: a diagramon egy nyíl van a Rendeléskezelő és a Számlázó között, de nem tudjuk, szinkron vagy aszinkron, milyen protokoll, milyen hibatűrés.
Következmény: az üzemeltető nem tudja, milyen timeoutot állítson be, a fejlesztő nem tudja, idempotens-e a hívás.
Javítás: minden élen tüntessük fel a protokollt, a szinkronitást és a hibatűrési mechanizmusokat. A könyv konvenciója szerint ez kötelező.

4. A nézet összekeverése.
Hiba: egy diagramon keveredik a logikai komponens, a telepítési topológia és a hálózati elrendezés.
Következmény: a stakeholder nem találja a saját kérdésére a választ, a diagram áttekinthetetlen.
Javítás: válasszunk szempontot, és tartsuk meg a nézet határait. A C4 szintek külön tartják a context, container, component és code nézeteket.

5. A rendszerhatár hiánya.
Hiba: a diagramon nem jelöljük, mi belső, mi külső.
Következmény: a felelősségi körök és a biztonsági határok nem egyértelműek.
Javítás: minden diagramon jelöljük a rendszerhatárt, és a külső rendszereket, szerepköröket a context szinten tüntessük fel.

6. A stakeholder figyelmen kívül hagyása.
Hiba: a dokumentáció csak a fejlesztőknek szól, az üzemeltetői és biztonsági aggodalmak kimaradnak.
Következmény: az architektúra nem lesz üzemeltethető, a biztonsági kockázatok későn derülnek ki.
Javítás: az architektúraleírás elején azonosítsuk a stakeholdereket, aggodalmaikat és a releváns nézeteket.

2.4.9 Labor: fogalomtérkép egy ismert rendszerhez

Ez a labor az alapfogalmak alkalmazását gyakoroltatja. A cél egy **reprodukálható fogalomtérkép** készítése egy ismert rendszerhez, amelyben a komponensek, interfészek, konnektorok, nézetek, stakeholderek és a rendszerhatár egyértelműen elkülönülnek.

2.4.9.1 Kiinduló állapot

Válassz egy ismert rendszert. Jó választás például:

- egy könyvtári kölcsönző rendszer,
- egy helyi ételrendelő alkalmazás,
- egy munkahelyi belső eszköznyilvántartó,
- egy egyszerű e-kereskedelmi webshop.

A rendszer lehet meglévő vagy tervezett. Lényeges, hogy a kontextust rögzítsük: mi a rendszer célja, kik a felhasználói, milyen külső rendszerekkel áll kapcsolatban.

2.4.9.2 Lépések

1. **Rögzítsd a kontextust.** Írd le 3–5 mondatban a rendszer célját, a felhasználói szerepköröket és a külső rendszereket. Például: „A könyvtári kölcsönző rendszer lehetővé teszi az olvasók számára a könyvek keresését, előjegyzését és kölcsönzését. A szereplők: Olvasó, Könyvtáros, adminisztrátor. Külső rendszer: email-szolgáltató, katalógus-API.”
2. **Készíts context diagramot diagrams-as-code formában.** Jelöld a rendszerhatárt, a külső szereplőket és a külső rendszereket. A context szinten a belső komponensek még nem szerepelnek, csak maga a rendszer és a környezete.
3. **Azonosítsd a komponenseket.** Legalább öt komponenst nevezz meg, és mindegyikhez adj felelősséget és interfészt. Töltsd ki a komponensleíró táblázatot.
4. **Add meg a konnektorokat.** Minden kapcsolathoz adj meg protokollt, szinkronitást és hibatűrési jellemzőket. A kapcsolatokon tüntesd fel, hogy szinkron vagy aszinkron, és milyen retry, timeout, circuit breaker vagy idempotencia elvárás érvényes.
5. **Készíts stakeholder-táblázatot.** Azonosíts legalább három stakeholdert, azok aggodalmait és a releváns nézeteket.
6. **Vezess döntési naplót.** Minden lényeges döntéshez írd le: mi volt a döntés, milyen alternatívákat mérlegeltél, miért ezt választottad, és milyen következményei vannak. Például: miért húztad a rendszerhatárt a külső email-szolgáltatónál kívülre; miért választottál aszinkron üzenetküldést a Rendeléskezelő és a Számlázó között.
7. **Különítsd el a megfigyelt és a várt állapotot.** Ha a rendszer már létezik, írd le, hogy a jelenlegi architektúra mit mutat (megfigyelt), és ha célarchitektúrát készítesz, mit szeretnél elérni (várt). A kettőt ne keverd.

2.4.9.3 Elvárt eredmény

- Verziózott dokumentum vagy jegyzetfüzet, amely tartalmazza a context diagramot, a komponensleíró táblázatot, a konnektorok táblázatát, a stakeholder-aggodalom-nézet táblázatot és a döntési naplót.
- A diagramok diagrams-as-code formában készülnek, például Mermaid, PlantUML vagy Structurizr DSL.
- A fogalomtérkép reprodukálható: a dokumentum verziózott, a kiinduló kontextus és a döntések visszakereshetők.
- A megfigyelt és várt állapot egyértelműen elkülönül.

2.4.9.4 Ellenőrzési szempontok

Szempont	Igen/Nem
A rendszerhatár minden diagramon jelölve van?	
Minden komponens rendelkezik felelősséggel és interfésszel?	
Minden kapcsolaton szerepel protokoll, szinkronitás és hibatűrés?	
A stakeholderek és aggodalmaik azonosítva vannak?	
A nézetekhez tartozik szempont?	
A döntésekhez tartozik kontextus, alternatívák és következmények?	
A megfigyelt és várt állapot elkülönül?	
Nincs valós hitelesítő adat, belső URL vagy ügyféladat?	

2.4.9.5 Visszaállítás

Mivel a labor dokumentumot hoz létre, a visszaállítás a verziókezelésen keresztül történik: a dokumentumot gitben tárold, a labor előtti állapotot commitold, és a gyakorlat végén a diff segítségével ellenőrizd a változásokat. Ha a fogalomtérképet jegyzetfüzetben készíted, a jegyzetfüzetet is verziózd, és a labor előtti állapothoz térhetsz vissza.

2.4.10 Ellenőrzőlista a fejezethez

- A „komponens” szó használatakor mindig egyértelmű, hogy logikai, futtatható vagy külső elemről van szó.
- Minden komponensnek van felelőssége és interfésze.
- Az interfészek tartalmazzák a műveleteket, paramétereket, hibákat és a szinkronitást.
- A kapcsolatok nem nyilak, hanem konnektorok: protokoll, szinkronitás, hibatűrés.
- A diagramokon a rendszerhatár jelölve van.
- A külső rendszerek és szerepkörök a context szinten megjelennek.
- A stakeholderek és aggodalmaik azonosítva vannak.
- A nézetekhez szempont tartozik, és a nézet nem kever össze eltérő aggodalmakat.
- A döntések döntési naplóban vagy ADR-ben rögzítettek.
- A dokumentáció nem tartalmaz valós hitelesítő adatot vagy belső rendszerinformációt.

2.4.11 Önellenőrző kérdések

1. Miben különbözik a komponens a konnektortól? Adj egy konkrét példát a Rendelőpont rendszerből, és indokold, miért nem elegendő a kapcsolatot nyílként ábrázolni.
2. Miért nem elegendő egyetlen nézet az architektúra leírásához? Mely stakeholder-kérdésekre ad választ a context, a container és a component szint?
3. Az alábbi mondatban mi a hiba? „A Rendeléskezelő és a Számlázó között kapcsolat van.” Javítsd ki a mondatot úgy, hogy az architektúra szempontjából hasznos legyen.

2.4.12 Összefoglalás

Ebben a fejezetben az architektúra leírásának közös szókincsét vezettük be. A komponens a felelősség hordozója, az interfész a szerződése, a konnektor a kapcsolatok első osztályú polgára, a konfiguráció pedig a komponensek és konnektorok elrendezése. A nézet egy adott szempont szerint készített reprezentáció, amely a stakeholderek aggodalmaira ad választ. A rendszerhatár minden diagramon jelölendő, mert ez választja el a belső elemeket a külső rendszerektől és szerepköröktől.

A gyakorlat során egy ismert rendszerhez készítettél fogalomtérképet, amelyben a komponenseket, interfészeket, konnektorokat, stakeholdereket és nézeteket együtt dokumentáltad. Ez a közös szókincs a következő fejezetekben használt architektúraelvek, döntési keretek és minőségi forgatókönyvek alapját képezi. A SOLID elvek architektúra szinten fejezet a SOLID elvek architektúraszintű alkalmazására épít, ezért a komponenshatárok, interfészek és kapcsolatok itteni pontos használata feltétele a továbblépésnek.

3 Architektúra elvek

3.1 SOLID elvek architektúra szinten

A SOLID elvek öt szoftvertervezési alapelvet foglalnak össze, amelyeket gyakran osztályszinten tanítanak, ám igazi erejük akkor mutatkozik meg, amikor komponens- és rendszerszintre emeljük őket. Ebben a fejezetben az elveket architektúrális perspektívából vizsgáljuk: mit jelent a felelősség, a nyitottság, a helyettesíthetőség, az interfész-szegregáció és a függőségmegfordítás egy nagyobb, elosztott vagy moduláris rendszer esetében. A cél nem az, hogy mechanikusan alkalmazzuk a szabályokat, hanem hogy megértsük a mögöttük rejlő tervezési döntéseket, és felismerjük, mikor segítenek, mikor pedig felesleges merevséget okoznak.

3.1.1 SRP – Single Responsibility Principle: egy komponens, egy felelősségi kör

A Single Responsibility Principle eredeti megfogalmazása szerint egy osztálynak egyetlen oka lehet a változásra. Architektúra szinten ezt úgy fordítjuk le, hogy **minden komponensnek egy jól körülhatárolt felelősségi köre legyen**, és a komponens akkor változzon, ha az a felelősségi kör változik. Fontos: a „felelősség” nem egyenlő a „funkció” szóval; a felelősség a változási okok és az üzleti vagy technikai szerep egysége.

Példaként vegyünk egy számlázórendszert, amely a következőket végzi: - beolvassa az ügyfél-adatokat, - kiszámítja a számla tételeit, - adószámítást végez, - elmenti a számlát egy relációs adatbázisba, - e-mailben elküldi a számlát, - naplózza a tranzakciót.

Ha mindez egyetlen komponensen belül valósul meg, akkor a komponensnek legalább négy különböző „oka a változásra” van: az üzleti logika, az adatkezelés, az értesítés és a naplózás. Bármelyik terület változása ezt a komponenst módosítja, ami növeli a hibakockázatot, és megnehezíti az izolált tesztelést. A helyes megközelítés a felelősségek szétválasztása:

- **SzámlaSzolgáltatás** – a számla üzleti logikája,
- **AdóSzámító** – az adószabályok implementációja,
- **SzámlaRepository** – a perzisztencia kezelése,
- **ÉrtesítőKliens** – e-mail vagy más csatorna,
- **TranzakcióNaplózó** – naplózás és auditálás.

Mindegyik komponens egyetlen felelősségi kört fed le, így a változások lokalizáltak maradnak. A komponensek közötti kapcsolatokat interfészek vagy explicit API-k definiálják, nem pedig belső állapotok megosztása.

Architektúra szinten gyakori hiba a **„God component”** (mindenes komponens) megjelenése: egy olyan szolgáltatás, amely minden üzleti képességet magában foglal, és a csapatok a fejlesztése során folyamatosan ebbe nyúlnak bele. Ez a monolitnál nem feltétlenül baj, ha a belső modulok jól elkülönülnek, de ha a belső határok hiányoznak, akkor az SRP sérül, és a rendszer karbantarthatósága romlik. A megoldás nem feltétlenül mikroszolgáltatások bevezetése, hanem a **moduláris monolit** kialakítása, ahol a komponensek erős belső kohézióval és gyenge csatolással rendelkeznek.

3.1.2 OCP – Open/Closed Principle: nyitott a bővítésre, zárt a módosításra

Az Open/Closed Principle szerint a szoftver entitások legyenek nyitottak a bővítésre, de zártak a módosításra. Ez azt jelenti, hogy egy új funkció bevezetésekor ne kelljen megváltoztatni a már működő, tesztelt kódot, hanem új kóddal bővítsük a rendszert. Architektúra szinten ez leggyakrabban **absztrakciók, interfészek, plugin-rendszerek és konfigurációvezérelt bővítési pontok** segítségével valósul meg.

Klasszikus példa a fizetési módok kezelése egy e-kereskedelmi rendszerben. Ha a rendeléskezelő komponens közvetlenül hívja a bankkártyás fizetési szolgáltatót, akkor egy új szolgáltató (például PayPal, utánvét, kriptovaluta) bevezetése a rendeléskezelő módosítását igényli. Ez sérti az OCP-t, mert a változás a már működő, kritikus komponenst érinti. A helyes tervezés egy **PaymentProvider** interfész bevezetése:

```
interface PaymentProvider {
    PaymentResult process(PaymentRequest request);
}
```

A rendeléskezelő csak ezt az interfészt ismeri, és a konkrét szolgáltatók implementálják azt. Amikor új fizetési módot adunk hozzá, elegendő egy új osztályt létrehozni és regisztrálni a rendszerben; a rendeléskezelő kódját nem kell módosítani. Ezáltal a rendszer nyitott a bővítésre, és zárt a módosításra.

Fontos kiemelni, hogy az OCP nem azt jelenti, hogy mindenhol előre absztrakciókat kell létrehozni. A **változás valószínűségét és irányát** kell figyelembe venni. Ha egy adott terület soha nem változik, vagy a változás iránya bizonytalan, felesleges absztrakciót bevezetni – ez a YAGNI elvvel (lásd DRY, KISS, YAGNI és a túltervezés csapdái) ellentétes túltervezéshez vezet. Az OCP alkalmazása akkor nyer értelmet, ha a bővítési igény valós, vagy a múltbeli tapasztalat arra utal, hogy a terület gyakran változik.

3.1.3 LSP – Liskov Substitution Principle: a helyettesíthetőség garanciája

A Liskov Substitution Principle kimondja, hogy ha egy program egy bázistípusra hivatkozik, akkor az altípusnak helyettesíthetőnek kell lennie anélkül, hogy a program helyessége sérülne. Ez a **szemantikai szerződések** betartását követeli meg: az altípus nem erősítheti meg a bemeneti előfeltételeket, és nem gyengítheti a kimeneti utófeltételeket.

Architektúra szinten az LSP különösen fontos, amikor egy komponens egy másik komponens interfészétől függ. Ha például egy alkalmazás egy **AdatForrás** interfészt használ, amelynek `read()` metódusa adatokat ad vissza, akkor bármely implementációnak – legyen az relációs adatbázis, NoSQL tároló, cache vagy külső API – ugyanolyan garanciákkal kell szolgálnia. Ha egy cache implementáció bizonyos esetekben kivételt dob, amikor a hálózati forrás nem, vagy más adatformátumot ad vissza, akkor az LSP sérül, és a kliens oldalán megbízhatatlan viselkedést okoz.

Példa: egy felhasználói profil lekérdezésére szolgáló interfész:

```
interface ProfileService {
    Profile getProfile(UserId userId);
}
```

Az interfész szerződése szerint a metódus minden létező felhasználóhoz visszaad egy profilt, és hiányzó felhasználó esetén `null`-t vagy egy jól definiált kivételt dob. Ha egy új implementáció (például egy gyorsítótárréteg) a hiányzó felhasználó esetén más típusú kivételt dob, vagy szándékosan üres profilt ad vissza, akkor a kliens, amely a szerződésre épít, hibásan működhet. Az

LSP tehát arra kényszerít, hogy a komponensek közötti interfészeket **stratégiai szerződésként** kezeljük, és minden helyettesítő implementációt e szerződéshez igazítsunk.

Rendszerszinten az LSP a **szolgáltatások közötti kompatibilitást** is jelenti. Ha egy mikroservice új verziója ugyanazt a REST API-t kínálja, de a válasz időzítése, hibakódjai vagy adatsémája megváltozik, akkor a fogyasztók megsérülhetnek. Ezért fontos a verziózás és a szerződéstesztelés (pl. Pact vagy OpenAPI-alapú contract testing), amelyek biztosítják a helyettesíthetőséget.

3.1.4 ISP – Interface Segregation Principle: a kliensek ne függjenek felesleges dolgoktól

Az Interface Segregation Principle szerint egyetlen klienst sem szabad olyan interfésztől függővé tenni, amelyet nem használ. Ez azt jelenti, hogy a nagy, monolitikus interfészek helyett kisebb, szerep-alapú interfészeket kell kialakítani, hogy minden fogyasztó csak a számára releváns műveleteket lássa.

Architektúra szinten az ISP leggyakrabban **szolgáltatás API-k tervezésekor** érhető tetten. Vegyünk egy felhasználókezelő szolgáltatást, amely a következő műveleteket kínálja:

- regisztráció,
- bejelentkezés,
- jelszó-visszaállítás,
- profil frissítése,
- adminisztrációs listázás és törlés.

Ha mindezt egyetlen hatalmas REST erőforráson vagy egy monolitikus RPC-interfészen keresztül tesszük elérhetővé, akkor egy egyszerű mobilalkalmazásnak, amely csak bejelentkezést és profilfrissítést használ, az összes művelet definícióját és a hozzá kapcsolódó adatmodelleket ismernie kell. Ez növeli a csatolást, és a szolgáltatás változásai feleslegesen hatnak azokra a kliensekre is, amelyek a változó részt nem használják.

Az ISP-t követve a szolgáltatás **több, célzott interfészt** biztosíthat:

- **AuthAPI** – bejelentkezés, token-ellenőrzés,
- **ProfileAPI** – profil lekérése, frissítése,
- **AdminAPI** – adminisztrációs műveletek.

Mindegyik interfész mögött lehet ugyanaz a belső implementáció, de a kliensek szempontjából a felület szegregált. Rendszerszinten ez a **BFF (Backend for Frontend)** minta egyik motivációja is: különböző klienstípusok (web, mobil, IoT) számára különböző, testre szabott API-kat kínálunk, hogy a kliensek ne szenvedjenek a felesleges adatok és műveletek terhétől.

Az ISP gyakori megsértése a **„fat interface”**, amikor egyetlen REST API endpoint hatalmas JSON-választ ad, amelyből a kliens csak néhány mezőt használ. Ez a hálózati overhead mellett az adatmodell kötöttségét is növeli. A megoldás a GraphQL vagy a célzott REST erőforrások, amelyek a kliens igényeihez igazodnak.

3.1.5 DIP – Dependency Inversion Principle: a függőségek irányának megfordítása

A Dependency Inversion Principle két szabályt fogalmaz meg: 1. Magas szintű modulok ne függjenek alacsony szintű moduloktól; mindkettő absztrakciótól függjön. 2. Az absztrakció ne függjön a részletektől; a részletek függjenek az absztrakcióktól.

A klasszikus rétegezett architektúrában a felső rétegek (üzleti logika) közvetlenül hívják az alsó rétegek (adatbázis, hálózat) implementációit. Ez a függőség iránya fentről lefelé mutat. A DIP ezt megfordítja: az üzleti logika definiál egy absztrakciót (például egy repository interfészt), és az adatbázis-kezelő implementáció ennek az interfésznek felel meg. Így a magas szintű modul nem függ a konkrét adatbázis-technológiától, és az implementációk cserélhetők.

Példa: egy rendeléskezelő szolgáltatás, amely az üzleti logikát tartalmazza. A helyes tervezés:

```
interface OrderRepository {
    Order save(Order order);
    Order findById(OrderId id);
}
```

A **OrderService** csak ezt az interfészt ismeri, és nem tud arról, hogy az adatbázis PostgreSQL, MySQL vagy egy teszt közben használt memória implementáció. Az adatbázis-kezelő komponens valósítja meg az `OrderRepository` interfészt, és ő felel a konkrét SQL utasításokért. Így ha az adatbázist cseréljük, az üzleti logika nem változik.

Rendszerszinten a DIP a **Clean Architecture**, a **Hexagonal Architecture** vagy a **Ports and Adapters** elvek alapja. A lényeg közös: a belső rétegek (domain, alkalmazás) nem függnek a külső infrastruktúrától (web, adatbázis, üzenetküldés). A függőségek kívülről befelé mutatnak. A külső adapterek (pl. REST kontroller, adatbázis adaptör) függenek a belső interfészektől, nem fordítva.

A DIP gyakori félreértése, hogy egyenlő a **Dependency Injection** (DI) technikával. A DI a DIP egyik megvalósítási módja – a függőségek kívülről való beinjektálása –, de a DIP maga az elv, amely a modulok közötti függőségi irányra vonatkozik. Önmagában attól, hogy egy osztály absztrakciót kap paraméterként, még nem biztos, hogy az absztrakció jól van definiálva, vagy hogy a magas szintű modul nem függ egy alacsony szintű részlettől.

3.1.6 A három kulcsfogalom: felelősség, nyitottság, helyettesíthetőség

Ahhoz, hogy a SOLID elveket architektúra szinten hatékonyan alkalmazzuk, három mögöttes fogalmat kell tisztán látnunk.

Felelősség – Az SRP alapja. Egy komponens felelősségét úgy érdemes meghatározni, hogy megkérdezzük: „Mi az az üzleti vagy technikai ok, ami miatt ennek a komponensnek változnia kell?” Ha a válasz egynél több okot tartalmaz, akkor a komponens valószínűleg túlterhelt. A felelősség meghatározása szorosan összefügg a **kohézióval**: egy jól felelős komponens belső elemei szorosan kapcsolódnak egymáshoz, és laza kapcsolatban állnak más komponensekkel.

Nyitottság – Az OCP mögötti cél, hogy a rendszer képes legyen új képességek befogadására a meglévő kód módosítása nélkül. Ez nem automatikus; tervezéskor tudatosan kell kiválasztani a bővítési pontokat. A nyitottságot segítik az interfészek, a plugin-mechanizmusok, a konfigurációk és az eseményvezérelt integráció, de az absztrakció bevezetésének költsége is van. A megfelelő egyensúlyt a változás gyakorisága és a rendszer komplexitása határozza meg.

Helyettesíthetőség – Az LSP lényege, hogy a komponensek közötti interfészek nem csupán szintaktikai szerződések, hanem szemantikai garanciákat is hordoznak. Ha egy komponenst egy másikra cserélünk, a rendszer többi része nem tudhatja meg, hogy a háttérben más implementáció dolgozik. Ezért az LSP megköveteli a szerződések explicit dokumentálását és tesztelését, például **contract testing**-gel vagy **consumer-driven contract**-tal.

3.1.7 A SOLID elvek alkalmazása egy összetett példán

Tekintsünk egy **könyvtárkölcsönző rendszert**, amelynek fő komponensei:

- **KölcsönzésiSzolgáltatás** – a kölcsönzés üzleti logikája (feltételek ellenőrzése, határidők számítása).
- **TagRepository** – a könyvtári tagok adatainak elérése.
- **KönyvRepository** – a könyvek adatainak elérése.
- **ÉrtesítőSzolgáltatás** – e-mail és SMS értesítések küldése.
- **BírságKalkulátor** – a késedelmi díjak számítása.

Kezdetben a **KölcsönzésiSzolgáltatás** közvetlenül hívja a repository-kat, az értesítőt és a bírságkalkulátort. Ezáltal a magas szintű üzleti logika szorosan csatolódik az alacsony szintű részletekhez (adatbázis, üzenetküldés). Az elemzés:

- **SRP**: A KölcsönzésiSzolgáltatásnak több oka is változhat (üzleti szabály, értesítési mód, bírságszámítás), tehát sérül az SRP.
- **OCP**: Ha új értesítési csatornát akarunk hozzáadni (pl. push értesítés), a KölcsönzésiSzolgáltatást módosítani kell.
- **LSP**: Ha az ÉrtesítőSzolgáltatás egy új implementációja (pl. mock) nem ugyanazt a hibakezelést produkálja, a rendszer viselkedése megváltozik.
- **ISP**: A TagRepository interfész minden taghoz kapcsolódó műveletet tartalmaz, még azokat is, amelyeket a KölcsönzésiSzolgáltatás nem használ (pl. címváltozás-kezelés); ez a felesleges függőség.
- **DIP**: A KölcsönzésiSzolgáltatás közvetlenül függ a konkrét repository osztályoktól, nem pedig interfésztől.

A SOLID elvek szerinti refaktorálás eredménye:

- A **KölcsönzésiSzolgáltatás** csak az üzleti szabályokat tartalmazza, és interfészeken keresztül kommunikál a perzisztenciával és az értesítéssel.
- A bírságszámítás kikerül egy külön **BírságKalkulátor** komponensbe, amely egy interfész mögött cserélhető (pl. országonként eltérő szabályok).
- Az **ÉrtesítőSzolgáltatás** egy interfészt valósít meg, és a KölcsönzésiSzolgáltatás csak ezt az interfészt ismeri.
- A repository-k interfészek mögé kerülnek, és a KölcsönzésiSzolgáltatás nem függ a konkrét adatbázis-kezelőtől.

Így a komponensek közötti határok világosak, a változások hatása korlátozott, és a rendszer könnyebben tesztelhető.

3.1.8 Labor: rendszerkomponensek SOLID értékelése

A labor célja, hogy a tanulók gyakorlatban alkalmazzák a SOLID elveket egy konkrét, adott rendszer komponenseinek értékelésére. A feladat reprodukálható dokumentumban vagy jegyzetfüzetben rögzíthető, és a döntéseket ADR formátumban kell rögzíteni.

Kiinduló állapot:

Adott egy hipotetikus **rendeléskezelő és számlázó rendszer**, amelynek jelenlegi architektúrája a következő komponenseket tartalmazza:

1. **RendelésSzolgáltatás** – a rendelések felvételét, módosítását, törlését végzi.
2. **SzámlaGenerator** – számlákat állít elő PDF formátumban.

3. **EmailKüldő** – e-mail értesítéseket küld.
4. **AdatbázisKezelő** – közvetlen SQL műveleteket hajt végre.

A jelenlegi implementációban mind a négy komponens egyetlen monolitikus kódban van, és a komponensek közvetlenül hívják egymást. Nincsenek interfészek, a tesztelés nehézkes, mert az adatbázis-kezelő közvetlenül kapcsolódik a termelési adatbázishoz.

Elvárt eredmény:

A tanuló dokumentálja a jelenlegi állapotot (mért), majd azonosítja a SOLID elvek megsértését, és javaslatot tesz a refaktorálásra (várt). A dokumentum tartalmazza:

- **Kontextus**: a rendszer célja, a felhasználói kör, a jelenlegi technológiai környezet (pl. Java, Spring, MySQL), a terhelés nagyságrendje (pl. napi 1000 rendelés).
- **Mért állapot**: a jelenlegi komponenshatárok, a függőségek és a felelősségi körök leírása, a csatolás és kohézió becsült jellemzése.
- **SOLID elemzés**: mind az öt elvnél annak vizsgálata, hogy a jelenlegi architektúra mennyiben teljesíti vagy sérti az elvet, konkrét példákkal.
- **Várt állapot**: refaktorálási javaslat, amely bemutatja, hogyan lehetne a komponenseket szétválasztani, interfészeket bevezetni, és a függőségi irányokat megfordítani.
- **Döntési napló (ADR)**: minden javasolt változtatáshoz egy ADR bejegyzés, amely tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
- **Ellenőrzési szempontok**: listázza, hogy mit ellenőrizne a refaktorálás után (pl. a csatolás csökken, az új komponensek tesztelhetők, a változások hatásköre korlátozott).

A labor során a tanuló **nem hajt végre** tényleges kódrefaktorálást, hacsak nem izolált, verziókezelt környezetben teszi, és a visszaállítás lehetősége biztosított. A cél a tervezési gondolkodás és a döntési dokumentáció gyakorlása.

A dokumentumnak tartalmaznia kell a **mért és a várt eredmény elkülönítését**: a jelenlegi állapotot tényként rögzíti, a javasolt állapotot pedig célként jelöli. A mérhető aspektusokat (pl. a komponensek közötti hívások száma, a ciklikus függőségek jelenléte) a jelenlegi és a célállapotra is fel kell tüntetni, ha lehetséges.

3.1.9 Ellenőrzőlista a SOLID elvek betartásához

A fejezet gyakorlati lezárásaként álljon itt egy ellenőrzőlista, amely segít felmérni, hogy egy architektúra mennyire felel meg a SOLID elveknek:

- ☐ **SRP**: Minden komponensnek egyértelműen azonosítható, egyetlen felelősségi köre van, és a változási okok egy jól meghatározott forrásra vezethetők vissza?
- ☐ **OCP**: Új funkció bevezetésekor a meglévő komponensek módosítása nélkül tudunk bővíteni? Hol vannak a bővítési pontok, és valóban ott koncentrálódnak a jövőbeli változások?
- ☐ **LSP**: Az interfészek mögött cserélhetők-e az implementációk úgy, hogy a kliensek viselkedése változatlan marad? A szerződések (előfeltételek, utófeltételek, hibakezelés) dokumentáltak és teszteltek?
- ☐ **ISP**: A kliensek csak a számukra szükséges interfésztől függenek, vagy feleslegesen húznak be nagy, monolitikus API-kat?
- ☐ **DIP**: A magas szintű üzleti logika független az alacsony szintű infrastrukturális részletektől? A függőségek iránya a stabil absztrakciók felé mutat?

3.1.10 Önellenőrző kérdés

Adott egy komponens, amely a felhasználói hitelesítést és a felhasználói profilkezelést is végzi. Nevezzen meg legalább két SOLID elvet, amelyet ez a kialakítás sért, és indokolja meg, hogyan lehetne a komponenst úgy átalakítani, hogy az elvek érvényesüljenek.

3.1.11 Összefoglalás

A SOLID elvek együttesen a moduláris tervezés alapjait adják. Nem merev szabályok, hanem iránymutatások, amelyeket a kontextus függvényében kell értelmezni. A felelősség, a nyitottság, a helyettesíthetőség, az interfész-szegregáció és a függőségmegfordítás mind a komponensek közötti határok minőségét javítják, csökkentik a csatolást, és növelik a rendszer evolúciós képességét. A következő fejezetben a DRY, KISS és YAGNI elvekkel foglalkozunk, amelyek kiegészítik, néha pedig ellensúlyozzák a SOLID elveket – például a túlzott absztrakció ellen hatnak. A cél mindig a kiegyensúlyozott, a tényleges igényekhez igazodó architektúra kialakítása.

3.2 DRY, KISS, YAGNI és a túltervezés csapdái

A SOLID elvek (SOLID elvek architektúra szinten) elsősorban az osztályok és modulok belső szerkezetének rugalmasságát szabályozzák. A DRY, KISS és YAGNI hármas más szinten működik: a tervezési döntések gazdaságosságát és az absztrakciók időzítését érinti. Mindhárom elv arra figyelmeztet, hogy a jó architektúra nem a minél több absztrakcióban, hanem a megfelelő helyen és időben bevezetett absztrakciókban rejlik. E fejezet célja, hogy megmutassa, hol húzódnak ezek a határok, és hogyan lehet elkerülni a túltervezés csapdáit.

A DRY elv és az ismétlés természete

A DRY (Don't Repeat Yourself) elvet Andy Hunt és Dave Thomas fogalmazta meg a The Pragmatic Programmer című könyvben. Az eredeti megfogalmazás szerint: „Minden tudásdarabnak egyetlen, egyértelmű, mérvadó reprezentációja legyen a rendszerben." A hangsúly a tudáson van, nem a kódsorokon. Ez a különbségtétel alapvető.

A kódduplikáció lehet tünete annak, hogy ugyanaz a tudás több helyen szerepel, de nem minden kódduplikáció tudásduplikáció. Tekintsünk egy példát: egy e-kereskedelmi rendszerben a számlázási modul és a szállítási modul egyaránt tartalmaz egy függvényt, amely a címeket érvényesíti. Ha a címvalidálás szabályai valóban ugyanazok, akkor ez valódi duplikáció: ha a szabály megváltozik (például új országkód formátum), mindkét helyen módosítani kell, és a rendszer könnyen inkonzisztenssé válik. Itt a DRY elv sérül.

Ha azonban a két modul azért tartalmaz hasonló kódot, mert a számlázási címre és a szállítási címre más érvényességi szabályok vonatkoznak (például a számlázási címen kötelező az adószám, a szállításinál nem), akkor a kód felszíni hasonlósága ellenére két külön tudásról van szó. Az összevonás erőltetése ebben az esetben veszélyes: egy közös validáló függvény bevezetésekor a két kontextus eltérő igényeit feltételes logikával kellene kezelni, ami növeli a kognitív terhelést és a regresszió kockázatát.

Ezt a jelenséget hívjuk véletlen duplikációnak (accidental duplication) vagy véletlen kapcsolódásnak. A DRY félreértelmezése gyakran vezet oda, hogy a fejlesztők minden hasonló kódblokkot összevonnak, még akkor is, ha azok eltérő okból változnak. A Robert C. Martin-féle Single Responsibility Principle (SRP) itt is irányadó: a közös absztrakciónak egyetlen oka kell legyen a változásra. Ha két kontextus más ütemben és más okból változik, akkor nem tartoznak egy absztrakcióba.

A gyakorlatban hasznos heurisztika a Rule of Three (három szabálya): várj, amíg a duplikáció háromszor előfordul, mielőtt absztrakciót vezetsz be. Az első előfordulásnál még nem ismerjük a minta stabilitását; a második előfordulás megerősíti a hasonlóságot; a harmadik pedig elegendő bizonyíték arra, hogy a közös logika tartós. Ez a heurisztika nem dogma, de segít elkerülni a korai absztrakciót. Ugyanakkor ha a duplikáció tudásduplikáció – például ugyanaz az adózási képlet két modulban –, akkor már az első előfordulásnál érdemes lehet közös modult létrehozni, mert a változás valószínűsége magas, és az inkonzisztencia költsége nagy.

A DRY határai és a véletlen kapcsolódás

A DRY elv alkalmazásának határa ott húzódik, ahol az absztrakció több különböző kontextust kényszerít közös nevezőre. A véletlen kapcsolódás (accidental coupling) akkor keletkezik, amikor két komponens azért függ egy közös modultól, mert az implementációjuk hasonló, nem pedig azért, mert ugyanazt a tudást képviselik. Tegyük fel, hogy egy rendszerben két entitás – például `User` és `Product` – rendelkezik `id` és `name` mezővel. Egy fejlesztő létrehoz egy közös `BaseEntity` absztrakt osztályt, amely ezeket a mezőket tartalmazza. Első pillantásra ez DRY-kompatibilis, de a valóságban a `User` és a `Product` életciklusa, érvényességi szabályai és változási okai teljesen eltérőek. A közös ős miatt ha később a `User` egy új mezőt igényel, az nem érinti a `Product`-ot, de a közös absztrakció megnehezíti a változtatást: a fejlesztő kísértést érez, hogy a `BaseEntity`-t bővítse, ami minden leszármazottra kihat. Ez a fajta kapcsolódás nem csökkenti, hanem növeli a karbantartási költséget.

A DRY elv tehát nem azt jelenti, hogy minden kódot egyszer írjunk le. Azt jelenti, hogy minden tudásnak egyetlen forrása legyen. A tudás lehet üzleti szabály, konfigurációs paraméter, séma, algoritmus vagy akár egy fogalom definíciója. A kód szintjén a duplikáció lehet elfogadható, ha a két kódrészlet eltérő tudást testesít meg, és a közös absztrakció több kárt okozna, mint hasznot.

A KISS elv és az egyszerűség határai

A KISS (Keep It Simple, Stupid) elv arra ösztönöz, hogy a lehető legegyszerűbb megoldást válasszuk, amely megfelel a követelményeknek. Az egyszerűség nem öncél, hanem a karbantarthatóság, az átláthatóság és a hibakeresés eszköze. Egy egyszerű megoldás kevesebb kódot, kevesebb függőséget és kevesebb váratlan viselkedést jelent.

Az egyszerűség határai azonban ott húzódnak, ahol a naiv megoldás nem képes kiszolgálni a minőségi attribútumokat. Például egy kis webáruház katalógusának lekérdezése megvalósítható egyetlen szinkron SQL-lekérdezéssel közvetlenül az adatbázisból. Ez egyszerű, és kisebb terhelésnél tökéletesen működik. Ha azonban a katalógust több millió felhasználó böngészi, a közvetlen adatbázis-lekérdezés teljesítményproblémát okoz, és szükség lehet gyorsítótárra, denormalizált olvasási modellre vagy keresőmotorra. Itt a KISS elv alkalmazása nem azt jelenti, hogy maradjunk a legegyszerűbb megoldásnál, hanem azt, hogy a komplexitást csak akkor vezessük be, amikor a bizonyítékok indokolják.

A KISS félreértelmezése gyakran vezet a „big ball of mud” anti-mintához: a fejlesztők annyira egyszerűre törekszenek, hogy minden logikát egyetlen osztályba vagy függvénybe zsúfolnak. Ez formailag egyszerű – kevés fájl, kevés absztrakció –, de a gyakorlatban bonyolult, mert a nagy, összefüggő kódblokkok nehezen tesztelhetők és módosíthatók. Az igazi egyszerűség nem a kevés absztrakcióban rejlik, hanem a megfelelő absztrakciókban, amelyek elválasztják a változó üzleti szabályokat a stabil infrastrukturális kódtól.

Egy másik csapda a túlzott leegyszerűsítés, amikor az üzleti komplexitást elrejtjük feltételes elágazások mögé. Például egyetlen `calculatePrice` függvény, amely tíz különböző kedvezménytípust kezel `if-else` ágakkal, formailag egyszerű, de minden új kedvezmény bevezetésekor a függvény nő, és a tesztelés egyre nehezebb. Ebben az esetben a KISS elv nem a monolitikus függvény

megtartását, hanem egy olyan absztrakció bevezetését indokolja, amely a kedvezményeket külön stratégiákba szervezi – de csak akkor, ha a kedvezménytípusok száma elér egy kritikus szintet.

A YAGNI elv és a tervezett bővíthetőség konfliktusa

A YAGNI (You Aren't Gonna Need It) elv az extrém programozásból származik, és arra figyelmeztet, hogy ne fejlesszünk olyan funkcionalitást, amelyre jelenleg nincs igény. A cél a pazarlás elkerülése: minden spekulatív funkció, interfész vagy konfigurációs lehetőség költséggel jár – le kell fejleszteni, tesztelni, dokumentálni és karbantartani. Ha a feltételezett jövőbeli igény nem valósul meg, a befektetés elvész.

A YAGNI és a tervezett bővíthetőség konfliktusa akkor merül fel, amikor egy bővítési pont bevezetése ma olcsóbb, mint a későbbi átalakítás. Például egy fizetési rendszerben a kezdeti követelmény csak Stripe támogatást ír elő. Egy fejlesztő két utat választhat: közvetlenül a Stripe SDK-t hívja a szolgáltatásban, vagy bevezet egy `PaymentProvider` interfészt, amely mögé a Stripe implementáció kerül. A YAGNI elv alapján az interfész felesleges, hiszen jelenleg csak egy implementáció létezik. Ha azonban a termékmenedzser már most jelzi, hogy a következő negyedévben PayPal és helyi banki átutalás is szükséges, akkor az interfész bevezetése nem spekuláció, hanem ismert követelményre adott tervezés. Ebben az esetben a YAGNI elv megsértése nem áll fenn, mert a bővítés nem feltételezés, hanem terv.

A különbség a feltételezés és a terv között döntő. A YAGNI azt tiltja, hogy „hátha kell majd" alapon építsünk általánosságot. Azt nem tiltja, hogy ismert, rövid távú követelményekre előre készüljünk, különösen akkor, ha az átalakítás költsége magas. A tapasztalat azt mutatja, hogy a legtöbb jövőbeli bővítési ötlet soha nem valósul meg, vagy egészen más formában valósul meg, mint ahogy gondoltuk. Ezért a spekulatív általánosítás általában rossz befektetés.

A YAGNI elv határa ott húzódik, ahol a változás költsége irreverzibilis vagy nagyon magas. Például egy adatbázis-séma megváltoztatása éles rendszerben, nagy adatmennyiség mellett, sokkal költségesebb, mint egy interfész bevezetése a fejlesztés korai fázisában. Ilyen esetekben érdemes lehet előre tervezni, még akkor is, ha a pontos követelmény nem teljesen ismert. Az ilyen döntéseket azonban tudatosan, kockázatelemzéssel kell meghozni, nem pedig automatikusan.

A túltervezés csapdái

A túltervezés (over-engineering) akkor következik be, amikor a fejlesztő több absztrakciót, rugalmasságot vagy konfigurálhatóságot épít be, mint amennyit a jelenlegi vagy ésszerűen várható követelmények indokolnak. A túltervezés jelei:

- **Túlzott absztrakció**: minden osztály interfész mögé van rejtve, még akkor is, ha az interfésznek egyetlen implementációja van, és nem várható több. Az ilyen interfészek növelik az indirekciót anélkül, hogy valódi hasznot hoznának.
- **Spekulatív genericitás**: a fejlesztő minden függvényt generikussá tesz, hogy „majd bármilyen típusra működjön". Ez a kód nehezebben olvasható, és a generikus paraméterek gyakran soha nem használódnak más típussal.
- **Konfigurálhatóság mindenáron**: a rendszer minden paramétere konfigurációs fájlba kerül, még azok is, amelyek soha nem változnak. Ez a környezetek közötti eltérést nehezíti, és a konfiguráció tesztelése bonyolult.
- **Több réteg, mint szükséges**: minden hívás keresztülmegy egy sor rétegen, amelyek csak továbbítják az adatokat (például Controller → Service → Manager → Repository → DAO), anélkül, hogy bármelyik réteg hozzáadott értéket nyújtana.
- **Aránytalan felkészülés skálázásra**: a fejlesztő már az első verzióban Kubernetes-klaszteren futtat microservice-eket, amikor a várható terhelés egyetlen szerveren is elfér. Ez a működtetési költséget és a komplexitást növeli anélkül, hogy jelenlegi hasznot hozna.

A túltervezés következményei: több kód, több teszt, több dokumentáció, nagyobb kognitív terhelés, hosszabb onboarding, nehezebb hibakeresés, és végső soron lassabb szállítás. A túltervezett rendszerben a fejlesztők félnek változtatni, mert nem látják át a sok absztrakciót. Ez a félelem gátolja az evolúciót, ami architekturális adóssághoz vezet.

Egyensúly és heurisztikák

A három elv együttes alkalmazása azt jelenti, hogy a tervezési döntéseknél mindig a kontextus alapján mérlegelünk. A következő heurisztikák segítenek:

1. **A tudás forrása egyetlen legyen**: ha ugyanaz a tudás több helyen szerepel, vonjuk össze, de csak akkor, ha a változási okok is azonosak.
2. **Várj a harmadik előfordulásig**: a duplikációt ne szüntessük meg azonnal, hacsak nem egyértelműen tudásduplikáció.
3. **A legegyszerűbb működő megoldás**: induljunk a lehető legegyszerűbb megoldással, és figyeljük a minőségi attribútumokat. Ha a mérések azt mutatják, hogy a megoldás nem skálázódik, akkor vezessünk be komplexitást.
4. **Ne általánosítsunk előre**: ha nincs bizonyíték arra, hogy egy bővítési pontra szükség lesz, ne hozzuk létre. Ha a változás költsége alacsony, akkor később is bevezethető.
5. **Mérjük a tervezési adósságot**: használjunk metrikákat (duplikáció aránya, kognitív komplexitás, indirekciók száma) a túltervezés és az alultervezés észlelésére.
6. **Dokumentáljuk a döntéseket**: ha egy absztrakciót a YAGNI ellenére vezetünk be, rögzítsük ADR-ben, hogy miért. Ez segít a későbbi fejlesztőknek megérteni a szándékot.

Gyakorlati példa: egy refaktorálási döntés

Tekintsünk egy Python nyelvű példát egy rendeléskezelő rendszerben. Kezdetben a kód így néz ki:

```
# order_service.py
def create_order(customer_id, items):
    # ügyfél validálása
    if not customer_id:
        raise ValueError("customer_id required")
    # tétel ellenőrzése
    if not items:
        raise ValueError("items required")
    # üzleti logika...
    return order_id

# invoice_service.py
def create_invoice(customer_id, amount):
    # ügyfél validálása
    if not customer_id:
        raise ValueError("customer_id required")
    # összeg ellenőrzése
    if amount <= 0:
        raise ValueError("amount must be positive")
    # számlázási logika...
    return invoice_id
```

Itt a `customer_id` validálása duplikált. Ha a validálási szabály megváltozik (például UUID formátumot kell ellenőrizni), mindkét helyen módosítani kell. Ez valódi tudásduplikáció, mert a `customer_id` fogalma ugyanaz mindkét kontextusban. A DRY elv alapján érdemes közös validáló függvényt létrehozni:

```
# validation.py
def validate_customer_id(customer_id):
    if not customer_id:
```

```
        raise ValueError("customer_id required")
    # további ellenőrzések, pl. UUID formátum
```

Ezzel a tudás egy helyre kerül.

Most nézzünk egy másik példát, ahol a DRY alkalmazása hibás lenne:

```
# user.py
class User:
    def __init__(self, id, name):
        self.id = id
        self.name = name

# product.py
class Product:
    def __init__(self, id, name):
        self.id = id
        self.name = name
```

A két osztály hasonló, de az `id` és `name` mezők jelentése más: a `User.id` egy felhasználó azonosítója, a `Product.id` egy termék azonosítója. A változási okok eltérőek: a `User` hamarosan kaphat `email` mezőt, a `Product` pedig `price` mezőt. Ha létrehozunk egy közös `Entity` ősosztályt, akkor a későbbi bővítések során a közös ős egyre inkább akadályoz, mert a két entitásnak semmi köze egymáshoz. Ez a helyes döntés, hogy a duplikációt megtartjuk, és a kódot a kontextusában értelmezzük.

A YAGNI és a tervezett bővíthetőség gyakorlati példája

Tegyük fel, hogy egy alkalmazásban jelenleg csak e-mail értesítéseket küldünk. A fejlesztő két megoldást fontolgat:

1. Közvetlenül az SMTP klienst hívja a szolgáltatásban.
2. Bevezet egy `NotificationSender` interfészt, amely mögé az SMTP implementáció kerül.

Ha a domain elemzés azt mutatja, hogy a közeljövőben push értesítések és SMS küldés is szükséges, akkor az interfész bevezetése ésszerű. Ha azonban semmi jel nem utal erre, akkor az interfész spekulatív általánosítás, és a YAGNI elv szerint el kell hagyni. A későbbi átalakítás költsége egy interfész bevezetésekor viszonylag alacsony: a szolgáltatásban a közvetlen hívást lecseréljük az interfészre, és az implementációt injektáljuk. Ez a refaktorálás egy-két napot vesz igénybe, míg a spekulatív interfész fenntartása hosszú távon folyamatos költséget jelent.

A döntés tehát a változás valószínűségétől és az átalakítás költségétől függ. Ezt a mérlegelést mindig a konkrét kontextusban kell elvégezni, és nem lehet általános szabályként kimondani, hogy „mindig vezess be interfészt” vagy „soha ne vezess be interfészt”.

A döntési folyamat modellezése

A fenti mérlegelést egy egyszerű döntési fa segíthet. Az alábbi Mermaid diagram egy lehetséges folyamatot mutat a duplikáció kezelésére:

```
flowchart TD
    A[Duplikált kód észlelése] --> B{Ugyanaz a tudás?}
    B -- Igen --> C{Ugyanaz a változási ok?}
    B -- Nem --> D[Tartsd meg a duplikációt]
    C -- Igen --> E[Vezess be közös absztrakciót]
    C -- Nem --> D
    E --> F[Ellenőrizd: tesztek, kognitív komplexitás]
    D --> G[Dokumentáld a döntést, ha szükséges]
```

Ez a döntési fa a DRY elv alkalmazásának egyik lehetséges heurisztikája. A gyakorlatban a döntés ennél árnyaltabb, de az alapelv egyszerű: a közös absztrakció csak akkor indokolt, ha a tudás és a változási ok is közös.

A labor feladat leírása

A fejezet laborjának célja, hogy a tanult elveket egy valódi kódbázisban alkalmazva azonosítsa a korai absztrakciókat és a duplikációkat. A feladatot páros vagy egyéni munkaként lehet elvégezni; az eredményt egy reprodukálható dokumentumban vagy jegyzetfüzetben kell rögzíteni.

Kiinduló állapot: Válasszon egy saját projektet vagy egy nyílt forráskódú kódbázist, amely legalább néhány ezer sorból áll, és amelyben rendelkezésre állnak tesztek. Jegyezze fel a kódbázis elérhetőségét, a commit hash-t, a programozási nyelvet és a keretrendszereket. A dokumentumban rögzítse a kiinduló állapotot: a fájlok számát, a kód sorainak számát, és ha van, a statikus elemző által jelentett duplikációs arányt.

Elvárt eredmény: A dokumentum tartalmazzon legalább három azonosított esetet: (1) valódi tudásduplikáció, (2) véletlen duplikáció, amelyet nem érdemes összevonni, és (3) egy korai absztrakció, amelyet a YAGNI elv alapján el lehet távolítani vagy meg kell tartani indoklással. Minden esethez készítsen egy rövid döntési naplót ADR-szerű formában: kontextus, döntés, alternatívák, következmények, állapot.

Lépések:

1. **Statikus elemzés**: Futtasson egy duplikációkereső eszközt (például jscpd, SonarQube, Semgrep, vagy egyszerű grep). Jegyezze fel a mért eredményeket: hány duplikált blokk, milyen arányban.
2. **Manuális áttekintés**: Válasszon ki legalább három duplikált kódblokkot, és elemezze, hogy a duplikáció valódi tudásduplikáció-e. Kérdezze meg: „Ha ezt a szabályt megváltoztatjuk, mindkét helyen kell módosítani?” Ha igen, akkor valódi duplikáció.
3. **Absztrakciók felmérése**: Keressen olyan interfészeket, absztrakt osztályokat vagy konfigurációs paramétereket, amelyeknek csak egy implementációja vagy használata van. Vizsgálja meg, hogy van-e ismert követelmény a bővítésre. Ha nincs, akkor jelölje meg korai absztrakcióként.
4. **Döntési napló írása**: Minden kiválasztott esethez írjon ADR-t. Tartalmazza a kontextust, a döntést, a mérlegelt alternatívákat, a következményeket és az állapotot (például „elfogadott”, „elutasított”).
5. **Refaktorálás (opcionális)**: Ha a döntés a duplikáció összevonása vagy a korai absztrakció eltávolítása, hajtsa végre a refaktorálást, és futtassa a teszteket. Jegyezze fel a mért eredményeket a refaktorálás után.

Ellenőrzési szempontok:

- A dokumentum egyértelműen elkülöníti a mért eredményeket (például a statikus elemző kimenetét) a várt eredményektől (hipotézisektől).
- Minden döntési napló tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.
- Az azonosított esetek reprezentálják a fejezetben tárgyalt három fogalmat: valódi duplikáció, véletlen duplikáció, korai absztrakció.
- A refaktorálás után a tesztek futnak, és a metrikák a várt irányba változnak (például a duplikáció aránya csökken, a kognitív komplexitás nem nő).
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy ügyfélnevet.

Visszaállítási út: Ha a refaktorálást elvégezte, és az eredmény nem megfelelő, a git verziókezelés segítségével állítsa vissza a kiinduló állapotot. A labor célja a tanulás, nem a produkciós kód módosítása.

A mért és várt eredmény elkülönítése: A laborban fontos, hogy a résztvevő ne keverje a bizonyítékot a feltételezéssel. Például: „A SonarQube 12%-os duplikációt mért a kezdeti állapotban" (mért), szemben azzal, hogy „Várhatóan a duplikáció 8%-ra csökken a refaktorálás után" (várt). A várt eredményt csak hipotézisként szabad megfogalmazni, és a refaktorálás után ellenőrizni.

Ellenőrzőlista a fejezethez

- ☐ Meg tudja különböztetni a valódi tudásduplikációt a véletlen duplikációtól.
- ☐ Alkalmazza a Rule of Three heurisztikát, de tudja, mikor kell korábban absztrahálni.
- ☐ Felismeri a DRY félreértelmezéséből fakadó véletlen kapcsolódást.
- ☐ A KISS elvet a kontextusban értelmezi, nem a kevés kódot tekinti egyedüli célnak.
- ☐ El tudja dönteni, hogy egy bővítési pont spekulatív vagy ismert követelményre épül.
- ☐ Ismeri a túltervezés tipikus jeleit, és képes azokat azonosítani.
- ☐ Dokumentálja a döntéseit, ha egy elvet tudatosan megszeg az architekturális okok miatt.

Önellenőrző kérdések

1. Hogyan dönti el, hogy két hasonló kódblokk összevonása a DRY elv helyes alkalmazása vagy véletlen kapcsolódást hoz létre?
2. Mondjon példát arra, amikor a KISS elv alkalmazása valójában bonyolultabbá teszi a rendszert.
3. Milyen feltételek mellett lehet indokolt a YAGNI elv megszegése egy tervezett bővíthetőség bevezetésével?
4. Milyen mérőszámokkal lehet felismerni a túltervezést egy kódbázisban?
5. Hogyan dokumentálná azt a döntést, hogy egy duplikációt szándékosan megtartunk?

Összefoglalás

A DRY, KISS és YAGNI elvek nem merev szabályok, hanem tervezési iránytűk. A DRY arra figyelmeztet, hogy a tudásnak egyetlen forrása legyen; a KISS arra, hogy a komplexitást csak indokolt esetben vezessük be; a YAGNI pedig arra, hogy ne építsünk spekulatív általánosságot. A három elv együttes alkalmazása azt jelenti, hogy minden absztrakció bevezetésekor mérlegeljük a jelenlegi és ésszerűen várható követelményeket, a változás költségét és a kód kognitív terhelését. A túltervezés csapdái akkor fenyegetnek, amikor az elveket dogmaként kezeljük, és nem a kontextus alapján döntünk. A fejezetben bemutatott heurisztikák és a laborfeladat segítenek abban, hogy ezt az egyensúlyt a gyakorlatban is megtaláljuk.

A következő alfejezet (Modularitás, kapcsolódás és kohézió) a modularitás, kapcsolódás és kohézió fogalmait vizsgálja, és épít az itt bevezetett absztrakciós döntések következményeire. A jól kialakított modulhatárok éppen azokat a változási okokat választják szét, amelyeket a DRY és YAGNI elvek elemzésekor azonosítottunk.

3.3 Modularitás, kapcsolódás és kohézió

Az előző fejezetben a DRY, KISS és YAGNI elveket vizsgáltuk, és láttuk, hogy ezek nem önálló parancsok, hanem feszültségben álló irányelvek, amelyeket a kontextus, a terhelés és a csapat érettsége együttesen értelmez. Ebben a fejezetben a moduláris felbontás minőségi ismérveit

járjuk körül: mit jelent az, hogy egy rendszer „jól modulált”, és milyen mérőszámokkal, döntési keretekkel lehet ezt a jóságot nemcsak érezni, hanem ellenőrizhető formában is megragadni.

A cél nem az, hogy egyetlen „tökéletes” felbontást adjunk. A modularitás önmagában nem cél, hanem eszköz: akkor hasznos, ha csökkenti a megértés, a változtatás, a tesztelés és az üzemeltetés költségét. Ahhoz, hogy ezt az eszközt jól használjuk, három szinten kell látnunk a modularitást: a kód, a szolgáltatás és a szervezet szintjén. Mindhárom szinten ugyanazok az alapkérdések térnek vissza: ki mit tud, mitől függ, és mi történik, ha megváltozik valami.

3.3.1 A moduláris felbontás minőségi ismérvei

A modul a szoftverarchitektúra legalapvetőbb építőköve: egy olyan egység, amely belső állapotát és logikáját elrejti a külvilág elől, és csak jól definiált interfészeken keresztül lép kapcsolatba más modulokkal. A modul lehet egy osztály, egy csomag, egy komponens, egy szolgáltatás vagy akár egy csapat, amely egy adott domainért felel. A moduláris felbontás minősége azon múlik, hogy a modulok közötti kapcsolatok gyengék, a modulokon belüli kapcsolatok pedig erősek-e.

Ennek a megfogalmazásnak van egy formálisabb, a szoftvertervezésben széles körben használt kerete: a kapcsolódás (coupling) és a kohézió (cohesion) fogalma. **A kapcsolódás azt méri, mennyire függnek egymástól a modulok; a kohézió azt, mennyire tartoznak össze a modulon belüli elemek.** A jó moduláris felbontás alacsony kapcsolódású és magas kohéziójú modulokat eredményez.

Ez a két fogalom együtt ad egy olyan minőségi iránytűt, amely segít eldönteni, hol húzzuk meg a határokat. Ha egy rendszerben két modul erősen összekapcsolódik, akkor egy változás az egyikben szükségszerűen változást kényszerít ki a másikban. Ha egy modul kohéziója alacsony, akkor a modul neve nem mond semmit arról, mit csinál, és a fejlesztőnek a megértéshez a modul teljes belsejét fel kell tárnia.

A moduláris felbontás minőségi ismérvei tehát nem esztétikai kérdések. Közvetlen hatásuk van a karbantarthatóságra, a tesztelhetőségre, a párhuzamos fejlesztés lehetőségére, a szervezeti skálázhatóságra és a rendelkezésre állásra. Ezt a hatást akkor tudjuk mérni, ha a minőségi követelményeket quality attribute scenario formában fogalmazzuk meg. Például:

> **Forgatókönyv:** Inger: a fejlesztő megváltoztatja a számlázási logikát a `Billing` modulban; környezet: fejlesztői környezet, a rendszer többi része változatlan; válasz: a változtatás csak a `Billing` modult érinti, a többi modul újrafordítás nélkül változatlanul működik; mérőszám: a változtatás által érintett modulok száma ≤ 1, a teljes tesztfutás időtartama ≤ 10 perc a mérési ablakban.

Ez a forgatókönyv akkor teljesíthető, ha a `Billing` modul kifelé mutatott kapcsolódása alacsony, a belső kohéziója pedig magas. A következő szakaszokban ezt a két fogalmat bontjuk ki részletesen, bevezetjük az afferent és efferent kapcsolódás mérőszámait, majd megnézzük, hogyan működik mindez a kód, a szolgáltatás és a szervezet szintjén.

3.3.2 Kapcsolódás és kohézió a gyakorlatban

Tegyük fel, hogy egy egyszerű e-kereskedelmi rendszert tervezünk. Az első vázlatban a következő modulok szerepelnek: `Order`, `Payment`, `Inventory`, `Notification` és `User`. A kérdés az, hogyan osztjuk el a felelősségeket, és milyen kapcsolatokat engedünk meg közöttük.

Egy rosszul modulált változatban a `Payment` modul közvetlenül eléri az `Inventory` belső adatszerkezetét, hogy ellenőrizze, van-e raktáron az adott termék. A `Notification` modul közvetlenül

hívja az `Order` modul belső adatbázis-lekérdezéseit. A `User` modul minden egyes modulban importálva van, és mindenhol más-más módon értelmezett.

Mi a probléma ezzel? Ha az `Inventory` adatszerkezete megváltozik, akkor a `Payment` modult is módosítani kell. Ha az `Order` modul adatbázisa migrációra szorul, a `Notification` is eltörik. A `User` modul minden változtatása az egész rendszerre kihat. A kapcsolódás magas, és minden modul ismeri a többiek belső működését.

A jó moduláris felbontás ezzel szemben így nézhet ki: a `Payment` modul nem ismeri az `Inventory` belső adatszerkezetét; ehelyett egy interfészen keresztül kér le egy jól definiált állapotot, például `checkAvailability(sku): AvailabilityStatus`. A `Notification` modul nem az `Order` adatbázist olvassa, hanem egy eseményt fogad, például `OrderPlaced`, amelyre feliratkozik. A `User` modul egyetlen, mindenki által használt szerződést publikál, és a többi modul csak ezen a szerződésen keresztül beszél hozzá.

Ez a példa jól mutatja, hogy a kapcsolódás és a kohézió nem elvont fogalmak, hanem mindennapi tervezési döntések. A következőkben ezeket a döntéseket formalizáljuk.

3.3.3 Afferent és efferent kapcsolódás

A kapcsolódás irányának megértése segít abban, hogy ne csak „sok” vagy „kevés” kapcsolatról beszéljünk, hanem arról is, hogy egy modul mennyire van kitéve a változásoknak, és mennyire okoz változásokat más modulokban. Ennek leírására két mérőszámot használunk:

- **Efferent kapcsolódás (Ce):** az adott modul hány más modultól függ. Ez mutatja, mennyire érzékeny a modul a rendszer többi részében történő változásokra. Minél magasabb az efferent kapcsolódás, annál több helyről jöhet olyan változás, amely a modulunkat is érinti.
- **Afferent kapcsolódás (Ca):** az adott modultól hány más modul függ. Ez mutatja, mennyire felelős a modul a rendszer stabilitásáért. Minél magasabb az afferent kapcsolódás, annál nagyobb a modul változásának kockázata, mert sok más modulra kihat.

A két mérőszám együttesen ad egy képet a modul stabilitásáról. Egy modul akkor stabil, ha magas az afferent és alacsony az efferent kapcsolódása: sok modul függ tőle, de ő kevés modultól függ. Egy ilyen modul változtatása sok helyre kihat, ezért ritkán és körültekintően szabad változtatni; viszont a környezet változásai kevésbé érintik őt.

Az ellenkező pólus a nagyon instabil modul: magas efferent, alacsony afferent. Ez a modul sok más modultól függ, de rajta kevés modul függ. Ezt könnyű változtatni, mert kevés modulra hat ki, de könnyen eltörik, mert a függőségei változása érinti.

A stabilitás egy gyakori származtatott mérőszámát a következő képlettel számíthatjuk:

$$I = \frac{Ce}{Ca + Ce}$$

Ahol I az instabilitás mértéke. Ha $I = 0$, a modul teljesen stabil: semmilyen efferent kapcsolódása nincs, minden kapcsolata befelé mutat. Ha $I = 1$, a modul teljesen instabil: nincs rá irányuló függőség, de ő sok modultól függ. A gyakorlatban a legtöbb modul valahol a kettő között helyezkedik el.

Fontos hangsúlyozni, hogy ez a mérőszám önmagában nem mond semmit a modul belső minőségéről, és nem is cél, hogy minden modul $I = 0$ legyen. Egy rendszerben szükségszerűen vannak stabil magok (például a domainmodell alapfogalmai, a publikált API-szerződések) és instabil

peremek (például a felhasználói felület, a CLI-eszközök, a prototípusok). A tervezés célja az, hogy a stabilitás eloszlása tudatos legyen: az instabil modulok függjenek a stabil moduloktól, ne fordítva.

3.3.4 A kapcsolódás fajtái

A kapcsolódás nemcsak mérhető mennyiség, hanem minőségi ismérvekkel is leírható. A szoftvertervezés klasszikus irodalma a kapcsolódásnak több fokozatát különbözteti meg, az erőstől a gyengéig:

- **Tartalmi kapcsolódás (content coupling):** az egyik modul közvetlenül hozzáfér a másik modul belső állapotához, például a memóriáját írja vagy a privát mezőit módosítja. Ez a legerősebb és legveszélyesebb kapcsolódás.
- **Közös globális állapot (common coupling):** több modul egy közös globális változót használ, például egy statikus konfigurációt vagy egy globális adatbázist. A változás egy modulban nem lokalizálható.
- **Vezérlési kapcsolódás (control coupling):** az egyik modul a másiknak paraméterben adja meg, hogy az mit tegyen, például egy `flag` paraméterrel. Ez csökkenti a modulok önállóságát, mert a hívó fél ismeri a hívott fél belső vezérlési logikáját.
- **Adatkapcsolódás (data coupling):** a modulok jól definiált adatstruktúrákon keresztül kommunikálnak, de nem ismerik egymás belső megvalósítását. Ez a kívánatos szint a legtöbb esetben.
- **Üzenetkapcsolódás (message coupling):** a modulok közötti kommunikáció kizárólag üzeneteken vagy eseményeken keresztül történik, a feladó és a fogadó nem ismeri egymás elérhetőségét vagy belső állapotát. Ez a leggyengébb kapcsolódás, amely aszinkron és elosztott rendszerekben különösen hasznos.

A tervezés során arra törekszünk, hogy a kapcsolódás minél inkább az adat- vagy üzenetkapcsolódás felé tolódjon. Ez nem jelenti azt, hogy mindenhol aszinkron üzenetekre kell áttérni: a szinkron adatkapcsolódás sokszor egyszerűbb, debuggolhatóbb és gyorsabb. A lényeg az, hogy a kapcsolódás fajtája tudatos döntés legyen, ne pedig a kényelemből adódó, rejtett globális állapotok és flag-ek halmaza.

3.3.5 Kohézió

A kohézió azt méri, mennyire tartoznak össze a modulon belüli elemek. Ha egy modul kohéziója magas, akkor a modulnak világos, egyetlen célja van, és a benne lévő elemek szorosan együttműködnek ennek a célnak az elérésében. Ha a kohézió alacsony, akkor a modul „mindenes” lett: különböző, egymáshoz nem kapcsolódó felelősségek kerültek egymás mellé.

A kapcsolódáshoz hasonlóan a kohéziót is skálán értékelhetjük:

- **Véletlen kohézió (coincidental cohesion):** a modulon belüli elemek között semmilyen értelmes kapcsolat nincs; csak véletlenül kerültek egy helyre.
- **Logikai kohézió (logical cohesion):** a modul több, logikailag hasonló feladatot lát el, például egy `Utility` osztály, amely a dátumkezelést, a stringmanipulációt és a fájlkezelést is végzi.
- **Időbeli kohézió (temporal cohesion):** a modul elemeit az köti össze, hogy ugyanabban az időben kell végrehajtani őket, például egy `Startup` modul, amely beolvassa a konfigurációt, inicializálja az adatbázist és elindítja a szálakat.

- **Eljárási kohézió (procedural cohesion):** a modul elemei egymás után futnak le, de nem ugyanazon adaton dolgoznak; a kapcsolatuk pusztán a végrehajtási sorrend.
- **Kommunikációs kohézió (communicational cohesion):** a modul elemei ugyanazon adaton dolgoznak, de különböző műveleteket végeznek.
- **Szekvenciális kohézió (sequential cohesion):** a modul elemei olyan csővezetéket alkotnak, ahol az egyik elem kimenete a másik bemenete.
- **Funkcionális kohézió (functional cohesion):** a modul minden eleme egyetlen, jól definiált feladat megvalósításában vesz részt. Ez a legerősebb és legkívánatosabb kohézió.

A gyakorlatban a funkcionális kohéziót gyakran úgy ismerjük fel, hogy a modulnak van egy világos, egymondatos felelőssége, és a modul neve pontosan ezt a felelősséget tükrözi. Például a `PaymentAuthorization` modul felelőssége az, hogy egy adott kártyás tranzakciót engedélyezzen; semmi más dolga nincs. Ha ebbe a modulba bekerül a hűségpontok kezelése vagy az e-mail értesítés, a kohézió csökken, és a modul neve hazudni kezd.

A kohézió mérésére is léteznek formalizált metrikák, bár ezek kevésbé elterjedtek a gyakorlati döntéshozatalban, mint a kapcsolódási mérőszámok. Az egyik legismertebb az **LCOM (Lack of Cohesion in Methods)**, amely egy osztály szintjén azt vizsgálja, hogy az osztály metódusai mennyire használják ugyanazokat a mezőket. Ha az osztály metódusai mintha két külön csoportra oszlanának, amelyek más-más mezőkön dolgoznak, akkor az osztály kohéziója alacsony, és valószínűleg több osztályra kellene bontani. A különböző LCOM változatok (LCOM1, LCOM2, LCOM4) eltérő finomsággal számítják ezt a hiányt, de a lényeg közös: az olyan modul, amely belsőleg széttartó, rosszabb minőségű, mint az, amelynek elemei egymásra épülnek.

3.3.6 Modularitás a kód szintjén

Az eddigi fogalmakat legkönnyebben a kód szintjén lehet megfigyelni. Egy jól tervezett kódbázisban a csomagok, osztályok és függvények moduljai élesen elhatároltak: a publikált interfészek kicsik, a belső megvalósítás rejtett, a függőségek iránya a stabil magok felé mutat.

Példaként vegyünk egy egyszerű rendelést kezelő modult Java-szerű pszeudokóddal. Az alábbi példa azt a hibát mutatja, amikor a `CheckoutService` közvetlenül hozzáfér a `ShoppingCart` belső mezőihez:

```
class CheckoutService {
    public Receipt checkout(ShoppingCart cart) {
        // Rossz: közvetlen hozzáférés a belső állapothoz
        List<Item> items = cart.items;
        double total = 0;
        for (Item item : items) {
            total += item.price * item.quantity;
        }
        return new Receipt(total, cart.customerId);
    }
}
```

Ebben a példában a `CheckoutService` efferent kapcsolódása a `ShoppingCart` felé magas, és a kapcsolódás fajtája tartalmi: ha a `ShoppingCart` belső adatszerkezete megváltozik, a `CheckoutService` is eltörik. A javított változat a `ShoppingCart` által publikált interfészre támaszkodik:

```
class CheckoutService {
    public Receipt checkout(ShoppingCart cart) {
        // Jó: a nyilvános interfészen keresztül
        Money total = cart.calculateTotal();
        CustomerId customerId = cart.getCustomerId();
```

```
        return new Receipt(total, customerId);
    }
}
```

Itt a `CheckoutService` csak a `ShoppingCart` által garantált szerződést ismeri: a `calculateTotal()` és a `getCustomerId()` metódusokat. A belső adatszerkezet változása nem érinti a hívót. A kapcsolódás adatkapcsolódássá szelídült.

A kód szintű modularitásnál a csomagok közötti függőségek iránya is fontos. Egy gyakori antiminta az úgynevezett **ciklikus függőség**: az `A` csomag függ a `B` csomagtól, a `B` pedig az `A` tevékenységet használja. Ilyenkor a két csomag gyakorlatilag egyetlen, megbonthatatlan egységet alkot, és a változások nem lokalizálhatók. A ciklikus függőségek megszüntetésére több eszköz is létezik: a közös részek kiemelése egy külön csomagba, a dependency inversion elv alkalmazása, vagy egy interfész bevezetése, amely a ciklust megtöri.

A kód szintű modularitás mérésére a statikus elemző eszközök (például a Java esetében az ArchUnit, a .NET esetében a NetArchTest) explicit architektúraszabályokat tudnak megfogalmazni. Például:

```
// ArchUnit szabály a ciklikus függőség tiltására
classes().should().beFreeOfCycles().check(classes);
```

Ez a szabály nemcsak az architektúra dokumentációját váltja ki, hanem a CI/CD pipeline részeként folyamatosan ellenőrzi is azt. A modularitás így nemcsak egy pillanatfelvétel, hanem egy evolúciós garancia: ha valaki véletlenül új függőséget vezet be a stabil magból az instabil perem felé, a teszt eltörik.

3.3.7 Modularitás a szolgáltatás szintjén

A mikroszolgáltatás-architektúrák korában a modularitás kérdése a kódból a szolgáltatások szintjére is áttevődik. Itt a kapcsolódás és a kohézió fogalma változatlan, de a mérés és a tervezés más eszközöket igényel.

Egy mikroszolgáltatás akkor jól modulált, ha a többi szolgáltatással való kommunikációja jól definiált, stabil API-n vagy eseményszerződésen keresztül történik, és a szolgáltatás belső adatbázisa, megvalósítása és technológiája a külvilág számára rejtett. A kapcsolódás itt is lehet afferent és efferent: a `OrderService` afferent kapcsolódása magas, ha sok más szolgáltatás hívja, például a `NotificationService`, az `AnalyticsService` és a `RecommendationService` is rá van kapcsolva. Ha ez a szolgáltatás meghibásodik, a hiba az egész rendszerre szétterjed.

A szolgáltatás szintű kapcsolódásnál különösen fontos a **hibamódok** figyelembevétele. Egy szinkron HTTP-hívás erős kapcsolódást jelent: ha a hívott szolgáltatás nem elérhető, a hívó fél is hibát tapasztal. Az aszinkron, üzeneteken alapuló kommunikáció lazább kapcsolódást tesz lehetővé: ha a `NotificationService` átmenetileg nem érhető el, az `OrderService` akkor is képes rendelést fogadni, mert az eseményt az üzenetsor pufferezi, és a fogyasztó később felzárkózik.

Ez azonban nem jelenti azt, hogy az aszinkron kommunikáció mindig jobb. Az aszinkron üzenetek rejtett függőségeket vihetnek a rendszerbe: ha az `OrderPlaced` esemény szerződése megváltozik, az összes feliratkozó szolgáltatást módosítani kell, és ez a változás nem feltétlenül derül ki fordítási időben. A szolgáltatás szintű modularitás tehát nemcsak arról szól, hogy „kevesebb HTTP-hívás”, hanem arról is, hogy a kommunikációs szerződések verziózottak, jól dokumentáltak és visszafelé kompatibilisek legyenek.

A szolgáltatás szintű kohézió magas, ha a szolgáltatás egyetlen domain-felelősséget lát el, és minden benne lévő komponens ezt a felelősséget szolgálja. Itt különösen fontos a Domain-Driven Design által bevezetett **bounded context** fogalma: a szolgáltatás határa akkor jó, ha a szolgáltatásnak megvan a maga saját, a többi szolgáltatástól független domainmodellje, és a szolgáltatások közötti kommunikáció a kontextusok közötti fordítással, nem pedig belső modell megosztásával történik.

3.3.8 Modularitás a szervezet szintjén

A modularitás harmadik szintje gyakran elkerüli a technikai tervezők figyelmét, pedig a rendszer struktúrája és a szervezet struktúrája között szoros kölcsönhatás van. Ezt a kapcsolatot Conway törvénye írja le: a szervezetek olyan rendszereket terveznek, amelyek a kommunikációs struktúrájukat tükrözik. Ha a szervezetben a csapatok határai nem illeszkednek a rendszer moduljaihoz, akkor a modulok közötti kapcsolódás a szervezeti határokon átnyúló kommunikációban fog megjelenni, ami lassú, költséges és konfliktusokkal terhelt.

A moduláris felbontás szervezeti szinten azt jelenti, hogy a csapatok határai egybeesnek a modulok határaival. Ha a `Payment` modul egyetlen csapat felelőssége, akkor a `Payment` modul belső kohéziója magas, és a csapat autonóm módon tud dönteni a belső megvalósításról. Ha viszont a `Payment` modul három különböző csapat munkáját igényli, akkor a modul kohéziója alacsony lesz, és a csapatok közötti koordináció költsége magas.

A szervezeti szintű kapcsolódás mérésére kevésbé formalizált mérőszámok léteznek, de a gyakorlatban jól működik az a megközelítés, amely a **csapatok közötti kommunikációs gráfot** vizsgálja. Ha a `Team A` minden döntéséhez be kell vonni a `Team B`-t, akkor a két csapat kapcsolódása magas, ami a rendszerben is magas kapcsolódást jelent. Az ilyen kapcsolatok feltérképezésekor gyakran kiderül, hogy a csapatok határai nem a rendszer természetes moduláris határai mentén húzódnak, hanem a régi szervezeti struktúra vagy a kényelem határozza meg őket.

A szervezeti modularitás javításának eszköze a **Team Topologies** megközelítés: a csapatokat aszerint alakítjuk ki, hogy milyen típusú modulokért felelnek, és milyen interakciós módokat engedünk meg közöttük. A cél itt is ugyanaz, mint a kód és a szolgáltatás szintjén: alacsony kapcsolódás, magas kohézió, jól definiált interfészek. A szervezeti interfész a csapatok közötti kommunikáció szerződése: ki milyen kérdésben, milyen csatornán, milyen visszajelzési idővel fordulhat a másikhoz.

3.3.9 A moduláris felbontás tipikus hibái

A modularitás elmélete egyszerű, a gyakorlata azonban számos buktatót rejt. Az alábbiakban a leggyakoribb hibákat foglaljuk össze, hogy a felismerésük és az elkerülésük tudatos legyen.

3.3.10 1. A „nagy modul” csapdája

Sok rendszerben találkozunk olyan modulokkal, amelyek túl nagyok ahhoz, hogy egyetlen ember vagy csapat mentálisan átlássa őket. Ez különösen gyakori régebbi monolitikus alkalmazásokban, ahol a `Common`, `Utils` vagy `Core` modulok idővel mindenre kiterjedő, ciklikus függőségekkel teli elefántcsonttá válnak. A tünet: a modul neve nem jelent semmit, a változtatások mindig „valahol a magban” történnek, és a teszteléshez az egész rendszert kell indítani.

A megoldás nem feltétlenül a mikroszolgáltatások bevezetése, hanem a moduláris monolit gondos belső felbontása: a `Common` modul helyett kis, jól definiált modulokat hozunk létre, mint például `Clock`, `JsonMapper` vagy `FeatureFlag`, és minden modulnak egyetlen felelősséget adunk.

3.3.11 2. A ciklikus függőségek

Ahogy korábban említettük, a ciklikus függőségek a modularitást szétverik, mert a ciklusban részt vevő modulok gyakorlatilag egyetlen modult alkotnak. A ciklus gyakran nem szándékos, hanem a „kis kényelem” eredménye: az `A` modul egyetlen hibakezeléshez importálja a `B` modult, a `B` pedig egyetlen teszthez importálja az `A` modult. A ciklus felszámolása sokszor egyszerűbb, mint amilyennek látszik: a közös részt ki kell emelni egy harmadik, stabil modulba, vagy a függőséget egy interfész bevezetésével meg kell fordítani.

3.3.12 3. A túlzott felbontás

A modularitás túlhajtása épp olyan káros, mint a hiánya. Ha minden apró függvényt külön modulba teszünk, a kapcsolódás mérőszáma ugyan alacsony lesz, de a modulok közötti kommunikáció költsége megnő: a fejlesztőnek egyetlen egyszerű feladat megvalósításához tíz apró modulban kell keresnie. A túlzott felbontás jele, ha a modulok nevei túl általánosak, a modulok együttműködése nehézkes, és a legtöbb modul csak egyetlen metódust tartalmaz. Ilyenkor a modulhatárokat újra kell húzni, és a logikailag összetartozó elemeket egyetlen modulba kell vonni.

3.3.13 4. A hibás irányú függőségek

A jól tervezett rendszerben a függőségek a stabil magok felé mutatnak. A hibás irányú függőség jele, ha egy domainmodell a felhasználói felület osztályait importálja, vagy ha egy domainszolgáltatás az adatbázis-elérési réteg részleteitől függ. Az ilyen függőségek a kód szintjén nehezen észrevehetők, de a változások során hamar megbosszulják magukat. A megoldás itt a dependency inversion elv: a stabil modul definiálja a számára szükséges interfészt, és az instabil modul ehhez igazodik.

3.3.14 Labor: kapcsolódási és kohéziós mátrix készítése

A fejezet laborfeladata a kapcsolódási és kohéziós mátrix elkészítése egy adott rendszerre. A cél az, hogy az olvasó a saját kódbázisán vagy egy megadott minta projekten alkalmazza a fenti fogalmakat, és a mérések eredményét reprodukálható dokumentumban rögzítse.

A labor kiinduló állapota: válassz egy kis, átlátható rendszert, amely legalább öt modullal rendelkezik. Ha nincs kéznél ilyen, használj egy nyilvános, oktatási célú mintapéldát, például egy egyszerű e-kereskedelmi monolitot a következő modulokkal: `Order`, `Payment`, `Inventory`, `Notification`, `User`. A rendszer kódját helyezd verziókezelés alá, és rögzítsd a mérés időpontját, a vizsgált commit azonosítóját és a használt elemzőeszköz verzióját.

A feladat két részből áll:

1. **Kapcsolódási mátrix:** határozd meg minden modulra a Ce és Ca értékét. Ha statikus elemző eszköz áll rendelkezésre, használd azt; ha nem, kézzel, a kódbázis importjainak átvizsgálásával végezd el a mérést. A mérési bizonytalanságot dokumentáld: például a dinamikus függőségek, a késői kötés vagy a rejtett globális állapotok nem mindig látszanak a forráskódból.

2. **Kohéziós értékelés:** minden modulra írd le egyetlen mondatban a modul felelősségét, és értékeld, hogy a modul elemei ehhez a felelősséghez igazodnak-e. Ha valamelyik modul neve és tartalma eltér, vagy a modul több, egymástól független feladatot lát el, jelöld alacsony kohéziójúként.

Az elvárt eredmény egy dokumentum vagy jegyzetfüzet, amely tartalmazza:

- a mérési kontextust: rendszer neve, commit, dátum, eszköz;
- a kapcsolódási mátrixot;
- a kohéziós értékeléseket;
- a mért és a várt eredmények elkülönítését: például ha a `Payment` modul Ce értéke 4, de a tervezett architektúrában csak 2 függőséget szánunk neki, akkor ezt kifejezetten hiba vagy eltérés jelzéseként kell feltüntetni;
- egy döntési naplót (ADR), amely a legjelentősebb eltérésekre javasol teendőt, például a `Payment` modul `Inventory`-függőségének megszüntetését egy interfész bevezetésével.

Az ellenőrzési szempontok:

- a dokumentum reprodukálható, azaz egy másik személy ugyanazon commitból kiindulva ugyanazokat a számokat kapja;
- a kapcsolódási mátrixban minden modul összes függősége szerepel, beleértve a közvetett függőségeket is;
- a kohéziós értékelés minden modulra egyetlen, konkrét felelősséget vagy annak hiányát jelzi;
- a mért és a várt eredmények egyértelműen elkülönülnek;
- a döntési napló kontextust, döntést, alternatívákat, következményeket és állapotot tartalmaz.

A labor nem egy „jó” vagy „rossz” megoldást vár, hanem azt, hogy az olvasó a saját rendszerén tudatosan alkalmazza a kapcsolódás és a kohézió fogalmait. A mérés eredménye sokszor meglepő: olyan függőségek bukkannak fel, amelyekről a fejlesztők nem is tudtak. Ennek az önmagában véve is értékes tanulsága van: a modularitás nem szándék, hanem mért és ellenőrzött tulajdonság.

3.3.15 Összefoglalás

Ebben a fejezetben a moduláris felbontás minőségi ismérveit vizsgáltuk. A kiindulópont a kapcsolódás és a kohézió fogalma volt: a jó moduláris felbontás alacsony kapcsolódású és magas kohéziójú modulokat eredményez. Bevezettük a kapcsolódás irányának mérésére szolgáló afferent és efferent mérőszámokat, és az instabilitás $I = \frac{Ce}{Ca+Ce}$ képletét. Megmutattuk, hogy a kapcsolódásnak fajtái vannak, és hogy a tervezés során az adat- és üzenetkapcsolódás felé érdemes mozdulni. A kohéziót skálán értékeltük, a véletlen kohéziótól a funkcionális kohézióig.

Mindezt a kód, a szolgáltatás és a szervezet szintjére vetítettük, hangsúlyozva, hogy a modularitás ugyanazokra az elvekre épül mindhárom szinten, csak az eszközök és a mérések különböznek. Végül a tipikus hibák – a nagy modul, a ciklikus függőség, a túlzott felbontás és a hibás irányú függőség – felismerésével és elkerülésével foglalkoztunk.

A következő fejezetben ezekre a fogalmakra építve az elvek ütközését vizsgáljuk: mi történik, ha a SOLID, a DRY, a KISS és a modularitás követelményei egymásnak feszülnek, és hogyan hozzunk ilyenkor architektúrális döntést.

3.3.16 Gyakorlat és ellenőrzőlista

Döntési gyakorlat: Válassz egy valós (akár a saját) kódbázist, és azonosíts benne egyetlen modult. Határozd meg a Ce és Ca értékét, és értékeld a kohézióját. Ha a modul instabil ($I > 0{,}66$) és alacsony kohéziójú, javasolj egy konkrét refaktorálási lépést. Ha stabil ($I < 0{,}33$) és magas kohéziójú, indokold meg, miért fontos, hogy ez a modul megőrizze ezt a helyzetét.

Ellenőrzőlista:

- ☐ A modulok közötti kapcsolatok dokumentálva vannak (afferent és efferent mérőszámokkal)?
- ☐ A kapcsolódás fajtája a legtöbb helyen adat- vagy üzenetkapcsolódás?
- ☐ A modulok kohéziója egyetlen, világos felelősséget tükröz?
- ☐ A ciklikus függőségek azonosítva és megszüntetve vannak?
- ☐ A függőségek iránya a stabil magok felé mutat?
- ☐ A szervezeti határok egybeesnek a moduláris határokkal?
- ☐ A mérések reprodukálhatóak, és a bizonytalanság dokumentálva van?

Önellenőrző kérdés: Mi a különbség az afferent és az efferent kapcsolódás között, és miért nem cél, hogy egy modul instabilitási mutatója $I = 0$ legyen?

3.4 Elvek ütközése: trade-offok és kontextus

Az elvek ütközése nem annak a jele, hogy az egyik elv rossz, hanem annak, hogy a szoftverarchitektúra mindig kontextusfüggő. A SOLID, a DRY, a KISS, a YAGNI, a modularitás és a többi heurisztika nem abszolút igazság, hanem különböző minőségi attribútumokat helyez előtérbe. Amikor ezek a célok konfliktusba kerülnek, a feladat nem az, hogy győztest hirdessünk, hanem az, hogy a konkrét helyzetben mérhető következmények alapján hozzunk döntést, és azt dokumentáljuk.

3.4.1 Miért ütköznek az elvek?

Az architekturális elvek rövidített döntési szabályok. Mindegyik egy tipikus kudarcmintára reagál:

- A **DRY** (Don't Repeat Yourself) a duplikációból fakadó következetlenségek ellen véd.
- A **KISS** (Keep It Simple, Stupid) a bonyolultság okozta karbantartási és üzemeltetési költségek ellen véd.
- A **YAGNI** (You Aren't Gonna Need It) a spekulatív általánosítás ellen véd.
- A **SOLID** elvek, különösen az **Open/Closed** és a **Single Responsibility**, a változások lokalizálhatóságát és a tesztelhetőséget javítják.
- A **modularitás, magas kohézió és alacsony csatolás** a fejlődőképességet és az áttekinthetőséget szolgálja.

A konfliktus abból fakad, hogy ugyanaz a szerkezeti döntés egyszerre javíthatja az egyik, és ronthatja a másik minőségi jellemzőt. Például egy absztrakciós réteg bevezetése csökkentheti a duplikációt (DRY), de növelheti a bonyolultságot (KISS). Egy generikus keretrendszer eleget tehet a nyílt–zárt elvnek, de lehet, hogy YAGNI-t sért, mert jelenleg csak egyetlen használati eset van.

Ezeket a feszültségeket nem lehet egyszer s mindenkorra feloldani. Amit tehetünk: explicit módon feltárjuk, milyen minőségi attribútumokat érint a döntés, milyen kontextusban működik a rendszer, és milyen mérhető következményekkel jár az alternatívák választása. Ehhez a fejezet

egy keretrendszert ad, amelynek része a döntési napló, a quality attribute scenario és a mérési terv.

3.4.2 Tipikus elvkonfliktusok és feloldási mintáik

A következő konfliktusok a gyakorlatban leggyakrabban visszatérnek. Mindegyiknél a feloldás kulcsa a kontextus megértése: milyen életszakaszban van a rendszer, mekkora a csapat, milyen a terhelés, mi a változási gyakoriság, és mi az üzleti kockázat.

3.4.3 DRY kontra YAGNI

A DRY azt mondja, hogy minden tudásnak egyetlen, egyértelmű forrása legyen a rendszerben. A YAGNI azt mondja, hogy ne építsünk be olyan általánosítást, amelyre jelenleg nincs igény. A konfliktus tipikusan akkor jelenik meg, amikor két hasonló, de nem azonos kódrészletet kell kezelni.

Példa: egy riportmodul először CSV-exportot készít. Később megjelenik egy PDF-export igény is. A naiv megoldás a generátor logika lemásolása és a formátumspecifikus rész kicserélése. A DRY azt sugallja, hogy vezessünk be egy közös absztrakciót, például egy `ReportRenderer` interfészt. A YAGNI viszont figyelmeztet: ha most csak két formátum van, és nincs jel arra, hogy több jön, a generikus keretrendszer felesleges bonyolultságot visz a kódba.

A feloldás nem bináris. Egy köztes megoldás lehet:

- Különítsük el a formátumfüggetlen adatelőkészítést a formátumspecifikus rendereléstől.
- Engedjük meg rövid távon a duplikációt, ha az logikailag stabil, és a változási sebesség eltérő.
- Vezessünk be absztrakciót ott, ahol az üzleti szabály tényleg közös, ne ott, ahol csak véletlen hasonlóság van.

A döntést mérhetővé teszi, ha megvizsgáljuk a két kódrészlet változási gyakoriságát és a közös részek arányát. Ha a közös szabályok együtt változnak, akkor a duplikáció kockázata magas, és a DRY kerül előtérbe. Ha a formátumok logikája függetlenül fejlődik, a YAGNI szerinti türelem indokoltabb.

3.4.4 KISS kontra SOLID nyílt–zárt elv

A nyílt–zárt elv azt kívánja, hogy a komponens nyitott legyen a bővítésre, de zárt a módosításra. Ennek klasszikus eszköze a stratégia minta, a plugin-architektúra vagy a dependency injection. Ezek a technikák azonban extra közvetett réteget, interfészt és konfigurációt visznek a rendszerbe, ami ellentmond a KISS-nek.

A konfliktus feloldása azon múlik, hogy a bővítési pontok valójában igénybe lesznek-e véve, és milyen költséggel jár egy későbbi módosítás. Egy stabil, ritkán változó rendszerben a KISS erősebb. Egy olyan rendszerben, ahol az üzleti szabályok hetente változnak, és a csapat több szolgáltatást is fenntart, a nyílt–zárt elv csökkenti a regressziós kockázatot, és hosszabb távon kifizetődik.

Egy praktikus kompromisszum: a csapat először egyszerű, közvetlen megoldást választ, de a kritikus határokon – például a domain mag és az infrastruktúra között – definiál interfészt. A nyílt–zárt elvet nem minden osztályra kell erőltetni, csak azokra a csomópontokra, ahol a változás ténylegesen megtörténik.

3.4.5 Modularitás kontra teljesítmény

A modularitás fizikai szétválasztást is jelenthet: külön process, külön konténer, külön szolgáltatás, esetleg külön adatbázis. Ez javítja a csapatok függetlenségét, a skálázhatóságot és a hibás komponens izolálását, de minden modulhatáron átívelő hívás költséggel jár: hálózati késleltetés, szerializáció, hibakezelés, megfigyelhetőség.

A konfliktus elosztott rendszerekben a legerősebb. A moduláris monolit választás például megőrzi a kódszintű modularitást anélkül, hogy a hálózati költséget fizetné. Ha a terhelés alacsony, és a csapat együtt dolgozik, a moduláris monolit gyakran jobb trade-off, mint a mikroszolgáltatás. Ha viszont különböző csapatok fejlesztenek különböző modulokat, és a szervezeti határok miatt különböző deployciklusokat akarnak, a fizikai szétválasztás lehet indokolt.

A döntéshez mérni kell a tényleges látens igényt: mekkora az adott hívási lánc hálózati késleltetési költségvetése? Mekkora a párhuzamosíthatóság? A Little-törvény segíthet a kapacitástervezésben: ha $L = \lambda W$, akkor a hálózati idő (W) növekedése azonos érkezési ráta (λ) mellett több kérést tart egyidejűleg a rendszerben (L), ami nagyobb erőforrás-igényt és több hibalehetőséget jelent. Ez a mérhető következmény segít eldönteni, hogy a modularitás előnye megéri-e a teljesítmény költségét.

3.4.6 Kohézió kontra csatolás

A magas kohézió azt jelenti, hogy egy modul belül szorosan összetartozó felelősségeket tartalmaz. Az alacsony csatolás azt jelenti, hogy a modulok közötti függőségek minimálisak. A kettő általában együtt jár, de néha feszül egymással.

Például egy rendelésfeldolgozó modulban a fizetési logika és a készletkezelés logikája szorosan együttműködik. Ha mindkettőt ugyanabba a modulba tesszük, magas lesz a kohézió, de a csatolás is megnő, mert a fizetés változása hatással lehet a készletkezelésre. Ha szétválasztjuk őket, csökken a csatolás, de nő a kommunikációs overhead.

A feloldás a változási gyakoriság és a tranzakciós határok elemzéséből adódik. Ha a fizetés és a készletkezelés ugyanabban a tranzakcióban vesz részt, a szétválasztás elosztott tranzakciós problémát szül, ami bonyolultabb, mint a csatolás. Ha viszont a készletkezelés ritkán változik, a fizetés pedig gyakran, a szétválasztás hosszabb távon csökkenti a regressziós kockázatot.

A döntéshez érdemes felmérni a modulok közötti interfészeket: hány hívás történik, milyen gyakran, és milyen adatot mozgat. Ha az interfész keskeny és stabil, a szétválasztás olcsó. Ha az interfész széles és gyakran változik, a kohézió megőrzése lehet a jobb választás.

3.4.7 Gyorsaság (time-to-market) kontra evolválhatóság

Ez a konfliktus gyakran nem is nevesített elv, hanem a KISS/YAGNI és a SOLID/modularitás közötti feszültség üzleti vetülete. A startup környezetben az első verzió gyors kiadása kulcsfontosságú, ezért a csapat hajlamos egyszerű, közvetlen megoldást választani, és a strukturális adósságot későbbre halasztani. Egy nagyvállalati környezetben viszont a rendszer hosszú élettartama miatt a túlélhetőség fontosabb, mint a néhány hetes piacra kerülési idő.

A feloldás itt is kontextusfüggő, de a mérhető következmények segítenek. A technikai adósságot nem szabad láthatatlanul felhalmozni. Ha a gyorsaságot választjuk, rögzítenünk kell, hogy milyen konkrét egyszerűsítést tettünk, milyen kockázatokat vállalunk, és mi a visszafizetés terve. Ez a döntési napló szerepe.

3.4.8 A kontextus és a csapatérettség szerepe

Az elvek közötti választás nem csak a rendszer technikai tulajdonságaitól függ. A kontextus legalább annyira fontos, mint az elv maga.

3.4.9 Rendszerérettség

Egy zöldmezős projektben a YAGNI és a KISS gyakran erősebb, mint a DRY és a SOLID, mert a követelmények még nem stabilak, és a korai absztrakció rossz irányba mehet. Egy érett, sok éve üzemelő rendszerben viszont a változások lokalizálhatósága és a duplikáció elkerülése felértékelődik.

3.4.10 Csapatméret és elosztottság

Egy kéttagú csapat, amely együtt ül, megengedheti magának a lazább modularitást, mert a kommunikáció olcsó, és a kód áttekintése egyszerű. Egy ötven fős szervezetben a modulhatárok egyben kommunikációs határok is; a fizikai szétválasztás csökkenti a koordinációs költséget, és lehetővé teszi a párhuzamos fejlesztést. Itt Conway törvénye is megjelenik: a rendszerarchitektúra tükrözi a kommunikációs struktúrát. Ha a csapatok úgy vannak szervezve, hogy önállóan szállítsanak, akkor a modularitás és a laza csatolás elve előrébb kerül, még akkor is, ha ez némi teljesítményköltséggel jár.

3.4.11 Csapatérettség

A csapatérettség azt jelzi, mennyire képes a csapat bonyolultabb architekturális mintákat felelősen üzemeltetni. Egy junior fejlesztőkből álló csapat számára a KISS elv életbevágó: a bonyolult elosztott rendszer, a generikus keretrendszer vagy a rengeteg absztrakció több hibát és lassabb tanulást okoz. Egy tapasztalt csapat viszont képes a trade-offokat tudatosan kezelni, és a moduláris, eseményvezérelt vagy plugin-alapú architektúra előnyeit kihasználni.

A csapatérettség nem érv arra, hogy rossz döntést hozzunk, de arra igen, hogy a tanulási görbét és az üzemeltetési kockázatot is a döntés részének tekintsük. Ha például egy szervezet bevezeti a Kubernetes-alapú mikroszolgáltatásokat, de a csapat még nem rendelkezik konténerkezelési és megfigyelhetőségi rutinnal, akkor a modularitás elve ütközik a KISS-szel, és a KISS oldalán a csapatérettség erős érv.

3.4.12 Szervezeti és szabályozási kontextus

Egy pénzügyi vagy egészségügyi rendszerben a biztonság, az auditálhatóság és a megfelelés olyan kényszerek, amelyek felülírhatják a KISS-t. A kötelező naplózás, a hozzáférés-kezelés és az adatmegőrzés extra rétegeket visz a rendszerbe, még akkor is, ha az egyébként egyszerűbb lehetne. Ugyanakkor ezek a kényszerek sokszor a DRY-t is erősítik: a naplózási logika duplikálása komoly megfelelési kockázatot hordoz.

3.4.13 Dogmatizmus helyett mérhető következmények

A legnagyobb hiba, amit egy tervező elkövethet, hogy az elveket szabályként kezeli, és nem vizsgálja a következményeket. A „mindig tartsd be a DRY-t” dogmája vezethet olyan absztrakcióhoz, amely többet árt, mint használ. A „mindig a legegyszerűbb megoldást válaszd” dogmája pedig technikai adósságot halmozhat fel, amely később megbénítja a fejlesztést.

A dogmatizmus helyett a következő lépéseket érdemes követni:

1. **Fogalmazd meg a konfliktust explicit módon.** Ne csak azt mondd, hogy „DRY vs KISS”, hanem írd le, mi a konkrét döntés, és melyik elv miért húz két irányba.
2. **Kapcsold a döntést minőségi attribútumokhoz.** Melyik elv milyen minőségi jellemzőt javít vagy ront? Például a DRY a karbantarthatóságot és a konzisztenciát, a KISS az érthetőséget és az üzemeltethetőséget.
3. **Írj quality attribute scenariót.** A könyv konvenciója szerint ez a forma: inger, környezet, válasz és mérőszám. Például: „Amikor egy új exportformátumot kell hozzáadni (inger), a riportmodulban (környezet), a fejlesztő legfeljebb 4 óra alatt, a meglévő formátumok regressziója nélkül tudja bevezetni (válasz), és a hozzáadott kód kevesebb mint 100 sor (mérőszám).”
4. **Határozd meg a mérési tervet.** Milyen metrika mutatja meg, hogy a választott megoldás működik? Lehet kódmetrika (duplikáció aránya, ciklikus komplexitás, modulok közötti csatolás), futásidőbeli metrika (késleltetés, átbocsátás), üzemeltetési metrika (hibajegyek száma, deploy gyakoriság), vagy üzleti metrika (változtatási átfutási idő). A mérést konkrét ablakkal és környezettel kell megadni.
5. **Hozd meg a döntést, és rögzítsd ADR-ben.** Az ADR-nek tartalmaznia kell a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Így a későbbi olvasó megérti, miért döntöttünk úgy, és milyen körülmények között érdemes újra elővenni.
6. **Építs fitness függvényt.** Ha a döntés kritikus, automatizáljunk egy ellenőrzést, amely figyelmeztet, ha a rendszer eltér a kívánt tulajdonságtól. Például egy ArchUnit-teszt ellenőrizheti, hogy a riportmodul nem függ az infrastruktúra-rétegtől, vagy egy statikus elemző beállíthatja a maximális ciklikus komplexitást.

3.4.14 Trade-off feloldási keret: lépésről lépésre

Az alábbi keret a gyakorlatban ismételten alkalmazható, és a Modularitás, kapcsolódás és kohézió alfejezetben tanult elvekre épít.

1. **Kontextus rögzítése**
   Írjuk le a rendszer célját, a jelenlegi életszakaszát, a csapatméretet, a csapatérettséget, a terhelést, a szabályozási kényszereket és az ismert kockázatokat. Ez lesz az ADR kontextusa.
2. **A konfliktus azonosítása**
   Nevezzük meg, melyik két (vagy több) elv feszül egymásnak, és mi a konkrét döntési pont. Például: „Bevezessünk-e egy általános export-absztrakciót most, vagy duplikáljunk?”
3. **Alternatívák felsorolása**
   Legalább két, de inkább három alternatívát adjunk meg: az egyik az egyik elvet, a másik a másik elvet részesíti előnyben, a harmadik lehet egy kompromisszum. Mindegyikhez írjuk le a rövid és hosszú távú következményeket.

4. **Quality attribute scenariók**
   Minden releváns minőségi attribútumhoz fogalmazzunk meg egy forgatókönyvet, amely a döntés szempontjából kritikus. Ez teszi mérhetővé a trade-offot.
5. **Mérési terv**
   Határozzuk meg, hogyan fogjuk ellenőrizni a forgatókönyvek teljesülését. A mérés lehet statikus elemzés, terhelési teszt, vagy akár fejlesztői tapasztalatra vonatkozó interjú. Fontos, hogy a mért és a várt eredményt különválasszuk, és a mérés bizonytalanságát is jelezzük.
6. **Döntés**
   A mérhető következmények alapján hozzuk meg a döntést, és rögzítsük ADR-ben. A döntésnek nem kell örökre szólnia; megadhatunk felülvizsgálati feltételt is.
7. **Visszacsatolás**
   A bevezetés után időszakosan mérjünk, és ha a várt és a mért eredmény között jelentős eltérés van, nyissuk meg újra a döntést. Ez az evolúciós architektúra alapelve.

3.4.15 Esetpélda: DRY kontra KISS a riportmodulban

A következő fiktív eset a keret működését mutatja be. A cél nem egy konkrét technológia ajánlása, hanem a gondolkodásmód illusztrálása.

3.4.16 Kontextus

A rendszer egy belső üzemeltetési dashboard, amelynek riportmodulja jelenleg CSV-exportot készít. A termékmenedzser új igénye szerint PDF-exportra is szükség van. A fejlesztőcsapat három főből áll, a rendszer terhelése alacsony, a változási gyakoriság mérsékelt. A riportmodul körülbelül 400 sornyi, viszonylag stabil kódot tartalmaz.

3.4.17 Konfliktus

- **DRY:** A CSV- és a PDF-export logikájában van közös rész: a riport sorainak előkészítése, a fejléc generálása, a dátumformátumok kezelése. Ha ezeket duplikáljuk, akkor egy jövőbeli változtatás, például egy új oszlop hozzáadása, két helyen kell módosítani, ami következetlenséghez vezethet.
- **KISS:** Ha most bevezetünk egy általános `ReportExporter` absztrakciót, az extra réteget, interfészt és konfigurációt visz a kódba. Jelenleg csak két formátum van, és nincs jel arra, hogy továbbiak jönnének. A túláltalánosítás csökkenti az olvashatóságot, és növeli a tanulási görbét.

3.4.18 Alternatívák

1. **A1 – Teljes duplikáció:** A PDF-export önálló osztályt kap, amely lemásolja a közös logikát.
   Következmény: gyors bevezetés, egyszerű kód, de a közös változásoknál kettős karbantartás.
2. **A2 – Közös absztrakció:** Létrehozunk egy `ReportRenderer` interfészt, amelynek `CsvRenderer` és `PdfRenderer` implementációja van.
   Következmény: a közös logika egy helyen van, de bonyolultabb a kód, és a jövőbeli bővítésekhez még több absztrakció kellhet.

3. **A3 – Fokozatos absztrakció:** Most szétválasztjuk a formátumfüggetlen és a formátumspecifikus logikát egy egyszerű függvénypárra, de nem vezetünk be komplex keretrendszert. A közös részeket egy kis `ReportData` struktúrába emeljük, és a renderelő függvények ezt fogadják.
   Következmény: mérsékelt kódmennyiség, minimális absztrakció, a duplikáció a formátumspecifikus részekre korlátozódik.

3.4.19 Quality attribute scenario

- **Karbantarthatóság:** Amikor egy új oszlopot adunk a riporthoz (inger), a riportmodulban (környezet), a változtatás legfeljebb 2 helyen érinti a kódot, és a regressziós teszt futási ideje kevesebb mint 30 másodperc (válasz). Mérőszám: módosított fájlok száma, tesztfutás hossza.
- **Érthetőség:** Amikor egy új fejlesztő csatlakozik a csapathoz (inger), a riportmodul kódját olvasva (környezet), 30 percen belül képes elmagyarázni az exportfolyamatot (válasz). Mérőszám: felkészülési idő, kódolvasási hibák száma.

3.4.20 Mérési terv

- A1 esetén a karbantarthatósági scenarióban a módosított fájlok száma várhatóan 2 (a CSV és a PDF osztály), az érthetőség pedig jó, mert a kód lineáris.
- A2 esetén a módosított fájlok száma várhatóan 1 (a közös absztrakció), de az érthetőségi scenarióban a felkészülési idő várhatóan 45 perc körül alakul a rétegek miatt.
- A3 esetén a módosított fájlok száma várhatóan 1–2, az érthetőség pedig közelít az A1-hez, mert az absztrakció minimális.

A mérés nem feltétlenül futtatott benchmark, hanem fejlesztői becslés, amit a döntési naplóban jelölni kell. Ha a csapat később valós adatot gyűjt, azt felülírhatja.

3.4.21 Döntés

A csapat az **A3** alternatívát választja. Indoklás: a rendszer alacsony terhelése és a kis csapat miatt a KISS erősebb érv, mint a teljes DRY. Ugyanakkor a közös adatelőkészítés logikája valóban stabil, ezért érdemes azt egy helyre emelni. A formátumspecifikus renderelést duplikáljuk, mert azok változási sebessége eltérő. A döntést ADR-ben rögzítjük.

3.4.22 ADR vázlat

```
ADR-2024-012: Riport exportáló szerkezete
Állapot: Elfogadva
Kontextus:
  - Belső dashboard riportmodul, CSV export létezik, PDF igény új.
  - Csapatméret: 3 fejlesztő, terhelés alacsony, változási gyakoriság mérsékelt.
  - Konfliktus: DRY (közös logika) vs KISS (minimális absztrakció).
Döntés:
  - Formátumfüggetlen adatelőkészítést közös ReportData struktúrába emeljük.
  - Formátumspecifikus renderelő függvények maradnak külön, minimális duplikációval.
  - Nem vezetünk be általános ReportRenderer keretrendszert.
Alternatívák:
  - Teljes duplikáció: elvetve, mert a közös logika stabil, és a duplikáció kockázata magas.
  - Teljes absztrakció: elvetve, mert a KISS sérül, és jelenleg nincs harmadik formátum.
Következmények:
```

```
- Pozitív: olvasható kód, gyors bevezetés, a közös logika egy helyen.
- Negatív: ha egy negyedik formátum jön, az absztrakciót újra kell gondolni.
- Felülvizsgálat: ha újabb formátumigény érkezik, vagy a duplikáció meghaladja a 100 sort.
```

3.4.23 A dogmatizmus csapdái és a tipikus hibák

Az elvek ütközése során a leggyakoribb hibák a következők:

1. **Az elv, mint identitás.** Amikor egy fejlesztő vagy tervező személyes meggyőződésévé válik egy elv, a viták elszakadnak a mérhető következményektől. A „SOLID mindenek felett” vagy a „YAGNI mindig” attitűd megakadályozza a kontextus elemzését.
2. **A kontextus figyelmen kívül hagyása.** Ugyanaz a döntés más rendszerben lehet jó vagy rossz. Aki a mikroszolgáltatást dogmaként ajánlja, figyelmen kívül hagyja a csapat méretét, a terhelést és a szervezeti felépítést.
3. **A mérhetőség hiánya.** Ha a döntést csak szubjektív érzésekre alapozzuk, később nem tudjuk eldönteni, hogy a választott irány működött-e. Minden trade-off elemzésnek része kell legyen a mérési terv.
4. **A rövid távú optimalizálás.** A KISS és a YAGNI hajlamos a gyors megoldás felé húzni, de ha a rendszer hosszú élettartamú, a technikai adósság később drága lesz. A döntésnél explicit módon jelezni kell a visszafizetés tervét.
5. **A túlzott általánosítás.** A DRY és a SOLID hajlamos a spekulatív absztrakció felé húzni. Ha a bővítési pontot nem használják, az csak felesleges bonyolultság.
6. **A csapatérettség alulbecslése.** Egy bonyolult minta bevezetése a csapat képességei fölé emelheti a hibasűrűséget, és a csapat elveszítheti a biztonságérzetét.

3.4.24 Labor: döntési napló készítése elvkonfliktusról

A labor célja, hogy a hallgató a fejezet keretrendszerét egy konkrét, ellenőrizhető példán alkalmazza. A döntési napló dokumentálása reprodulálható módon történjen, például Markdown fájlban vagy Jupyter notebookban.

3.4.25 Kiinduló állapot

Válassz egy fiktív vagy valós, de elszigetelt döntési helyzetet, amelyben két architekturális elv ütközik. Jó példák:

- DRY vs YAGNI egy generikus adatelérési réteg bevezetésekor.
- KISS vs nyílt–zárt elv egy plugin-keretrendszer választásakor.
- Modularitás vs teljesítmény egy elosztott szolgáltatás felbontásakor.

A választott helyzet legyen elég kicsi ahhoz, hogy egy dokumentumban kezelni lehessen, de legyen benne legalább egy mérhető minőségi attribútum.

3.4.26 Feladat

1. **Kontextus rögzítése**
   Írd le a rendszer célját, a csapatméretet, a csapatérettséget, a terhelést, a szabályozási kényszereket és a rendszer életszakaszát.
2. **A konfliktus megfogalmazása**
   Nevezd meg a két ütköző elvet, és írd le, mi a konkrét döntési pont.
3. **Alternatívák**
   Adj meg legalább három alternatívát, mindegyiknél rövid és hosszú távú következményekkel.
4. **Quality attribute scenariók**
   Fogalmazz meg legalább két forgatókönyvet, amelyek a döntés szempontjából relevánsak. Használd a könyv konvencióját: inger, környezet, válasz, mérőszám.
5. **Mérési terv**
   Határozd meg, hogyan ellenőriznéd a forgatókönyvek teljesülését. Különítsd el a mért és a várt eredményt, és jelöld a mérés bizonytalanságát.
6. **Döntés és ADR**
   Készíts ADR-t a választott döntéssel, a fenti vázlatnak megfelelően. Az ADR tartalmazza az állapotot is.

3.4.27 Elvárt eredmény

Egy verziózott dokumentum, amely tartalmazza a fenti elemeket. A dokumentumnak olyan részletességűnek kell lennie, hogy egy külső olvasó megértse a döntés okát és az elvetett alternatívákat.

3.4.28 Ellenőrzési szempontok

- A kontextus legalább négy elemet tartalmaz: rendszer célja, csapat, terhelés, szabályozás vagy életszakasz.
- A konfliktusban két elv explicit módon meg van nevezve, és a döntési pont egyértelmű.
- Legalább három alternatíva szerepel, mindegyiknél legalább egy pozitív és egy negatív következmény.
- A quality attribute scenariók formailag megfelelnek a konvenciónak.
- A mérési tervben a mért és a várt eredmény külön van választva.
- Az ADR minden kötelező mezőt tartalmaz: kontextus, döntés, alternatívák, következmények, állapot.
- A dokumentum verziózott, és a döntésekhez visszakereshető hivatkozás tartozik.

3.4.29 Visszaállítási út

Mivel a labor dokumentumkészítés, a visszaállítás a korábbi verzió megőrzésével oldható meg. A hallgató mentse el a kezdeti vázlatot, majd a végleges döntési naplót külön commitban vagy külön fájlverzióban. Ha a labor során módosítaná a kontextust, tudjon visszatérni az eredetihez.

3.4.30 Ellenőrzőlista az elvkonfliktusok feloldásához

- ☐ Azonosítottad-e a két ütköző elvet és a konkrét döntési pontot?
- ☐ Rögzítetted-e a rendszer kontextusát: méret, csapat, terhelés, életszakasz, szabályozás?
- ☐ Felsoroltál-e legalább három alternatívát, köztük kompromisszumot?
- ☐ Készítettél-e quality attribute scenariót minden releváns minőségi jellemzőre?
- ☐ Meghatároztad-e a mérési tervet, és elkülönítetted a mért és várt eredményt?
- ☐ A döntés alapja mérhető következmény, nem pedig dogma?
- ☐ Rögzítetted-e a döntést ADR-ben, felülvizsgálati feltétellel?
- ☐ Jelezted-e a csapatérettségből fakadó kockázatokat?
- ☐ A választott megoldás illeszkedik-e a rendszer változási sebességéhez?
- ☐ Az elvetett alternatívák indoklása megjelenik-e a dokumentumban?

3.4.31 Önellenőrző kérdés

Adott egy kis belső adminisztrációs rendszer, amelyben két különálló modul ugyanazt a dátumformázási logikát alkalmazza. A csapat három fős, a rendszer ritkán változik, a terhelés alacsony. Hogyan döntenéd el, hogy a duplikációt megszüntesd-e egy közös segédfüggvénnyel, vagy hagyd-e a YAGNI elv alapján? Milyen kontextuselemeket vizsgálnál, és milyen mérhető következményt fogalmaznál meg?

3.4.32 Összegzés

Az elvek közötti konfliktusok feloldása nem a helyes elv kiválasztásáról szól, hanem arról, hogy a konkrét helyzetben mérhető következmények alapján hozzunk döntést. A kontextus – a rendszer érettsége, a csapat mérete és érettsége, a szervezeti és szabályozási kényszerek – ugyanolyan fontos, mint maga az elv. A dogmatizmus helyett a trade-off elemzés, a quality attribute scenariók és az ADR-ben rögzített döntési napló ad keretet ahhoz, hogy a jövőbeli olvasó is megértse a döntés okát.

Ez a fejezet közvetlenül előkészíti a következőt, amely az architekturális döntések formálisabb kezelésével foglalkozik: az ADR-ek életciklusával, a döntési jogkörökkel és a döntési napló szervezeti szerepével. Az itt bemutatott mérhetőkövetkezmény-szemlélet ott válik majd a döntési folyamat gerincévé.

4 Architektúra mint döntéshozatal

4.1 Architektúra döntések és következményeik

A szoftverarchitektúra gyakran látszik szerkezeti rajzok, technológiai választások és minták gyűjteményének, de a felszín alatt minden ilyen elem egy korábban meghozott döntés lenyomata. A rendszerhatár, a komponensek felelőssége, a közöttük lévő kapcsolatok protokollja, a perzisztencia mechanizmusa, a telepítési egységek kialakítása – mind döntések eredménye. Ebben a fejezetben a döntést mint az architektúra alapegységét vizsgáljuk, és bevezetjük a visszafordíthatóság fogalmát, amely a gyakorlati döntéshozatal egyik legfontosabb vezérelve.

A fejezet célja, hogy az olvasó ne csupán dokumentálni tudja a döntéseket, hanem megértse, hogyan érdemes a döntési helyzeteket felismerni, a lehetőségeket értékelni, és a bizonytalanságot kezelni. A döntési napló és a visszafordíthatóság szempontjai olyan eszközök, amelyek akkor is használhatók, ha a szervezetben még nincs formális architektúra-governance.

4.1.1 1. A döntés mint az architektúra alapegysége

Az architektúra leírható szerkezeti elemekkel: komponensek, kapcsolatok, nézetek. Ugyanakkor minden ilyen elem egy döntés következménye. Amikor egy komponenst kettéválasztunk, amikor egy szinkron REST-hívást aszinkron üzenetküldésre cserélünk, vagy amikor bevezetünk egy API-gateway-t, nem csupán a rajzot módosítjuk, hanem egy döntést hozunk. A döntés az az egység, amely összeköti a követelményeket, a minőségi attribútumokat, a korlátokat és a szervezeti feltételeket a konkrét szerkezeti megoldással.

Ebből következik, hogy az architektúra-munka lényegi része nem a dokumentáció szépítése, hanem a döntések meghozatala és azok következményeinek vállalása. Egy döntés mindig választás legalább két alternatíva között, és minden választás valamilyen következménnyel jár. A következmények lehetnek technikaiak (például nagyobb késleltetés, több memória, nehezebb tesztelés), szervezetiek (például több csapat közötti koordináció, új készségek iránti igény) és gazdaságiak (például licencköltség, üzemeltetési költség, fejlesztési idő).

A döntések jellemzően nem izoláltak: egy döntés újabb döntési helyzeteket szül, és a korábbi döntések szűkítik a későbbi lehetőségek terét. Ezért a döntések láncolatát érdemes úgy kezelni, mint egy fát, amelyben minden elágazásnál a választott út meghatározza a későbbi opciókat. A visszafordíthatóság ebben a képben azt jelenti, hogy egy elágazás után milyen költséggel és kockázattal tudunk visszalépni egy korábbi ponthoz, vagy válthatunk egy másik ágra.

A döntés mint alapegység szemléletből következik, hogy a döntéseket explicitté kell tenni. A kimondatlan döntések a legveszélyesebbek, mert a csapat különböző tagjai eltérő feltételezésekkel dolgoznak. Az írásban rögzített döntési napló – akár egyszerű szöveges formában, akár Architecture Decision Record (ADR) formátumban – azt a célt szolgálja, hogy a döntés kontextusa, indoklása és következményei a későbbi olvasó számára is érthetők legyenek.

4.1.2 2. Visszafordíthatóság és annak költsége

A visszafordíthatóság azt méri, hogy egy döntést milyen erőfeszítéssel, idővel és kockázattal lehet megváltoztatni vagy visszavonni. A fogalom nem bináris: a legtöbb döntés valahol a teljesen reverzibilis és a gyakorlatilag irreverzibilis között helyezkedik el. A visszafordíthatóság mértéke függ a technikai megvalósítástól, az adatmodelltől, a szervezeti függőségektől és a döntés beágyazottságától.

Például egy belső osztály felbontása két osztályra általában könnyen visszafordítható, ha a nyilvános interfész nem változik. Ezzel szemben a nyilvános API szerződésének megváltoztatása, amelyet számos külső fél használ, gyakorlatilag irreverzibilis, mert a változtatás minden klienst érint. Hasonlóképpen egy adatbázisséma megváltoztatása, amelyben éles adatok vannak, jóval nagyobb költséggel jár, mint egy be nem vezetett prototípus kódjának átírása.

A visszafordíthatóság költségét érdemes három összetevőre bontani:

- **Megvalósítási költség:** mennyi munkával lehet a változtatást végrehajtani a kódban, konfigurációban vagy infrastruktúrában.
- **Adatmigrációs költség:** ha a döntés adatokat érint, akkor az adatok konvertálása, visszaállítása vagy újra előállítása milyen erőfeszítést igényel.
- **Koordinációs és kommunikációs költség:** hány csapatot, partnert vagy ügyfelet kell értesíteni, és milyen ütemezéssel lehet a változtatást bevezetni.

A visszafordíthatóságot nemcsak a döntés meghozatalakor érdemes értékelni, hanem a döntés következményeinek felülvizsgálatakor is. Ha egy döntést vissza kell vonni, a visszafordítás maga is egy döntés, amelynek megvannak a maga következményei és kockázatai.

4.1.3 3. Reverzibilis és irreverzibilis döntések

A döntéseket aszerint szokás csoportosítani, hogy mekkora a visszafordítás költsége és kockázata. A **reverzibilis döntés** olyan, amelynél a visszafordítás költsége alacsony a döntés várható hasznához képest, és a visszafordítás nem okoz jelentős adatvesztést vagy szolgáltatáskiesést. Az **irreverzibilis döntés** ezzel szemben olyan, amelynek megváltoztatása nagy költséggel, magas kockázattal vagy jelentős szervezeti ellenállással jár.

A két kategória között folytonos átmenet van. A bináris besorolás helyett hasznosabb, ha minden döntéshez hozzárendelünk egy visszafordíthatósági szintet, és ezt rögzítjük a döntési naplóban. A szint lehet például: könnyen visszafordítható, közepesen visszafordítható, nehezen visszafordítható, gyakorlatilag irreverzibilis.

Néhány tipikus példa:

- **Könnyen visszafordítható:** egy belső konfigurációs érték megváltoztatása, egy új loggolási formátum bevezetése, egy kódmodul átnevezése, ha a hivatkozások konzisztensek maradnak.
- **Közepesen visszafordítható:** egy szinkron hívás aszinkronra cserélése, ha a komponensek közötti interfész még nem publikus; egy cache-réteg bevezetése, amelyet ki lehet kapcsolni.
- **Nehezen visszafordítható:** a publikus API verziójának megszüntetése, egy adatbázisséma oszlopának eltávolítása éles adatokkal, a szolgáltatások közötti hitelesítési mechanizmus cseréje.
- **Gyakorlatilag irreverzibilis:** egy programozási nyelv lecserélése egy nagy, éles rendszerben, a felhasználói adatok tulajdonjogának megváltoztatása, egy szervezeti modell felbontása, amely a csapatok közötti szerződéseket is meghatározza.

A reverzibilis döntéseknél a döntéshozatal gyors lehet, és a hibák kijavítása olcsó. Az irreverzibilis döntéseknél viszont nagyobb körültekintésre, több információra és gyakran formálisabb értékelési folyamatra van szükség. Ez az alapelv kapcsolódik a **last responsible moment** gondolatához: egy döntést akkor érdemes meghozni, amikor a késleltetés költsége meghaladja a további információszerzés várható hasznát.

4.1.4 4. A döntéshozatal időzítése és bizonytalansága

A döntéshozatal időzítése önálló döntési probléma. A túl korai döntés azt kockáztatja, hogy a feltételezések tévesek, és a döntést később vissza kell vonni, vagy ami rosszabb, a téves döntéshez igazítjuk a követelményeket. A túl késői döntés viszont blokkolhatja a fejlesztést, és a csapat párhuzamosan olyan megoldásokat épít, amelyeket később integrálni kell.

A bizonytalanság kezelésére két alapvető stratégia létezik:

- **Információszerzés:** a döntéshez szükséges ismeretek bővítése spike-kal, proof of concepttel, prototípussal, méréssel vagy szakértői konzultációval. Ennek költsége van, de csökkenti a téves döntés valószínűségét és várható hatását.
- **Késleltetés:** a döntés elhalasztása addig, amíg a körülmények nem kényszerítenek elköteleződést. Ez akkor működik jól, ha a késleltetés nem blokkol más döntéseket, és a rendszer nem szenved el jelentős átalakítást a későbbi döntés miatt.

A két stratégia együttesen alkalmazható: először késleltetjük a döntést, közben információszerzéssel csökkentjük a bizonytalanságot, majd a döntést a lehető legkésőbbi felelős pillanatban hozzuk meg.

A bizonytalanságot érdemes explicit módon kezelni. A döntési naplóban rögzíteni kell, hogy milyen feltételezések mellett született a döntés, melyek a legnagyobb kockázatú feltételezések, és milyen jelek utalhatnak arra, hogy a feltételezés téves. Ez lehetővé teszi, hogy a döntést később felülvizsgáljuk, ha a körülmények megváltoznak.

A döntéshozatal időzítésének egyik hasznos eszköze a **döntési napló bejegyzéseinek állapotmezője**. Egy döntés lehet javasolt, elfogadott, elvetett, felülvizsgálatra szoruló vagy visszavont. Az állapotváltozások időpontjának rögzítése megmutatja, hogy egy döntés mennyi ideig volt érvényben, és mikor vált szükségessé a felülvizsgálat.

4.1.5 5. Példa: szinkron REST vagy aszinkron üzenet a fizetési folyamatban

Tekintsünk egy kitalált, de gyakorlatias döntési helyzetet. Egy webáruház rendelés-feldolgozó szolgáltatása a fizetési szolgáltatóval kommunikál. A csapatnak el kell döntenie, hogy a fizetési kérést szinkron REST-hívással vagy aszinkron üzenetsoron keresztül küldje el.

A döntés kontextusa:

- A rendelési folyamat során a rendelés állapota „fizetésre vár” lesz, majd a fizetési szolgáltató visszaigazolása után „fizetve” vagy „elutasítva” állapotba kerül.
- A fizetési szolgáltató elérhetősége 99,9% SLO-val rendelkezik, de időnként 2–3 másodperces válaszidőt produkál.
- A rendelés-feldolgozó szolgáltatást egyetlen csapat fejleszti, a fizetési szolgáltatót pedig egy külső partner üzemelteti.
- A rendszer terhelése változó, csúcsidőben másodpercenként 50 rendelés érkezik.

A két opció:

1. **Szinkron REST-hívás:** a rendelés-feldolgozó szolgáltatás a fizetési kérés elküldésekor blokkolva vár a válaszra. A folyamat egyszerű, az állapotátmenet azonnali, de a rendelés-feldolgozó szolgáltatás rendelkezésre állása a fizetési szolgáltató rendelkezésre állásától függ. Ha a fizetési szolgáltató lassú vagy elérhetetlen, a rendelés-feldolgozási lánc elakad.
2. **Aszinkron üzenetküldés:** a rendelés-feldolgozó szolgáltatás egy üzenetsorba helyezi a fizetési kérést, és a fizetési szolgáltató egy fogyasztó komponensen keresztül dolgozza fel, majd egy válaszüzenetet küld. A rendelés-feldolgozó szolgáltatás nem blokkol, de a folyamat bonyolultabbá válik: kezelni kell az időkésést, az üzenetek duplikálódását, a sorrendiséget és a kompenzációs lépéseket.

A döntés visszafordíthatósága:

- A szinkron REST-hívásról aszinkron üzenetküldésre való áttérés megvalósítási költsége mérsékelt, ha a rendelés-feldolgozó szolgáltatás belső interfésze jól definiált. A legnagyobb költséget az állapotgép átalakítása és az időkésések kezelése jelenti.
- Az aszinkron üzenetküldésről szinkron REST-re való visszatérés viszont egyszerűbb, ha az üzenetfeldolgozó komponens izolált, és a rendelés-feldolgozó szolgáltatás API-ja nem változik.
- A döntés tehát **közepesen visszafordítható** mindkét irányban, ha a komponenshatárokat jól választjuk meg.

A döntés időzítése: a csapat nem biztos abban, hogy a fizetési szolgáltató válaszideje a jövőben hogyan alakul, és nem tudja pontosan megbecsülni az aszinkron folyamat többletfejlesztési költségét. Érdemes lehet egy rövid spike-ot végezni, amelyben a két megközelítés közötti késleltetési különbséget és a hibakezelési komplexitást mérik egy izolált tesztkörnyezetben. A mérés eredménye segíthet a döntés megalapozásában, de a mérés nem helyettesíti a döntést.

A példa jól mutatja, hogy a döntés nem csupán technikai választás, hanem a rendelkezésre állás, a fejlesztési sebesség, az üzemeltetési komplexitás és a szervezeti függőség közötti kompromisszum. A döntési naplóban ezeket a szempontokat érdemes rögzíteni.

4.1.6 6. Labor: döntés következményeinek és opcióinak elemzése

Ez a labor a fejezet gyakorlati alkalmazását szolgálja. A cél, hogy az olvasó egy konkrét döntési helyzetet elemezzen, és az eredményt reprodukálható dokumentumban vagy jegyzetfüzetben rögzítse.

4.1.7 6.1. Kiinduló állapot

Válassz egy döntési helyzetet. Használhatod a fenti példát (szinkron REST vagy aszinkron üzenet a fizetési folyamatban), vagy választhatsz saját, hasonló komplexitású problémát. A választott helyzethez rögzítsd a következő kontextust:

- A rendszer célja és határa egy mondatban.
- Az érintett komponensek és szereplők.
- A döntést befolyásoló követelmények (legalább egy minőségi attribútum).
- A döntést korlátozó kényszerek (például technológiai, szervezeti, jogi).
- A rendelkezésre álló idő és erőforrás.

4.1.8 6.2. Elvárt eredmény

Készíts egy döntési naplóbejegyzést, amely tartalmazza:

1. **Kontextus:** a döntés háttere, a probléma leírása.
2. **Opciók:** legalább két, egymástól érdemben eltérő alternatíva.
3. **Következmények:** minden opcióhoz felsorolva a technikai, szervezeti és gazdasági következmények.
4. **Visszafordíthatóság:** minden opcióra becsült visszafordíthatósági szint, indoklással.
5. **Időzítés:** mikor érdemes meghozni a döntést, és milyen információszerzési lépések előzhetik meg.
6. **Választott opció:** a döntés rögzítése, az indoklással.
7. **Mért és várt eredmények:** ha rendelkezésre állnak mérések, azokat különítsd el a becslésektől, feltételezésektől. Jelöld, hogy melyek a nem reprodukált vagy nem mért állítások.
8. **Visszaállítási út:** mi szükséges ahhoz, hogy a döntést visszavonjuk, ha a következmények kedvezőtlenek.

A döntési napló formátuma lehet egyszerű szöveges dokumentum, Markdown, vagy egy jegyzetfüzet cellája. A lényeg a reprodukálhatóság: a kontextus és az érvelés olyan részletességű legyen, hogy egy későbbi olvasó megértse a döntés okát.

4.1.9 6.3. Ellenőrzési szempontok

A labor eredményét a következő szempontok szerint ellenőrizd:

- A kontextus egyértelműen rögzített, és nem hiányzik belőle a rendszerhatár.
- Legalább két opció szerepel, és mindegyikhez tartoznak következmények.
- A visszafordíthatóság minden opcióra becsült, nem csak a választottra.
- Az időzítési indoklás tartalmazza a késleltetés és az információszerzés mérlegelését.
- A mért és várt eredmények elkülönítése megtörtént; nem szerepel mért eredményként olyan, amit nem mértek.
- A visszaállítási út konkrét lépéseket tartalmaz, nem csak általánosságokat.
- A dokumentum verziózható, és a döntéshez hivatkozás tartozik.

4.1.10 6.4. Visszaállítási út

A labor nem igényel éles környezetet, így nincs fizikai visszaállítási kényszer. A döntési napló verziózását azonban érdemes úgy kialakítani, hogy a napló korábbi állapota visszaállítható legyen. Ha a döntést később visszavonod, a naplóba új bejegyzést kell felvenni, amely hivatkozik az eredeti döntésre, és rögzíti a visszavonás okát, valamint az új állapotot.

4.1.11 7. Tipikus hibák és azok elkerülése

A döntéshozatal során visszatérő hibák figyelhetők meg. Ezek ismerete segít a csapdák elkerülésében.

4.1.12 7.1. A döntés kimondatlanul hagyása

A leggyakoribb hiba, hogy a döntést nem rögzítik írásban, és a csapat tagjai eltérő feltételezésekkel dolgoznak. Ez később félreértésekhez, konfliktusokhoz és felesleges újraegyeztetéshez vezet. A megoldás egyszerű: minden jelentős döntést írásban rögzíteni, még akkor is, ha a formátum kezdetben egyszerű.

4.1.13 7.2. Túl korai döntés

A csapat gyakran nyomás alatt dönt, mielőtt az alapvető követelményeket vagy a terhelési profilt megismerné. A túl korai döntés különösen kockázatos, ha irreverzibilis. Ilyenkor érdemes a döntést elhalasztani, és közben spike-kal vagy prototípussal csökkenteni a bizonytalanságot.

4.1.14 7.3. Elemzési paralízis

Az ellenkező véglet, amikor a csapat annyi ideig elemzi az opciókat, hogy a döntés elmarad, és a fejlesztés blokkolódik. Ennek elkerülésére hasznos a visszafordíthatóság szempontja: ha a döntés könnyen visszafordítható, nincs ok a hosszas elemzésre. Csak az irreverzibilis döntések igényelnek alaposabb értékelést.

4.1.15 7.4. A visszafordíthatóság figyelmen kívül hagyása

A döntések értékelésekor gyakran csak a pozitív következményeket vizsgálják, a visszafordítás költségét nem. Ez ahhoz vezet, hogy a csapat úgy kezel egy nehezen visszafordítható döntést, mintha reverzibilis lenne, és később meglepetésként éri a visszavonás költsége.

4.1.16 7.5. A kontextus elvesztése

A döntés indoklása idővel elhalványul, ha a napló csak a döntést rögzíti, a kontextust nem. A későbbi olvasó nem tudja, milyen feltételezések mellett született a döntés, és milyen alternatívákat vetettek el. Ezért a döntési naplóban a kontextus és az elvetett opciók ugyanolyan fontosak, mint maga a döntés.

4.1.17 7.6. A mért és várt eredmények összemosása

A döntési naplóban gyakran előfordul, hogy becsléseket tényként kezelnek. Ha nincs mérés, a becslést explicit módon annak kell jelölni. A megkülönböztetés azért fontos, mert a későbbi felülvizsgálat során a mért eredményekből lehet kiindulni, a becslésekből nem.

4.1.18 8. Ellenőrzőlista

Az alábbi ellenőrzőlista a döntéshozatal gyors áttekintésére szolgál:

- ☐ A döntési helyzet világos, és a rendszerhatár rögzített.
- ☐ Legalább két alternatívát azonosítottunk.
- ☐ Minden alternatívához felmértük a következményeket (technikai, szervezeti, gazdasági).
- ☐ A visszafordíthatóságot minden alternatívára becsültük.
- ☐ Az időzítést tudatosan választottuk: nem túl korai, nem túl késői.

- ☐ A bizonytalanságot csökkentő lépéseket (spike, PoC, mérés) mérlegeltük.
- ☐ A döntést írásban rögzítettük, a kontextussal és az elvetett opciókkal együtt.
- ☐ A mért és várt eredményeket elkülönítettük.
- ☐ A visszaállítási út ismert, ha a döntés kedvezőtlennek bizonyul.

4.1.19 9. Önellenőrző kérdés

Tekintsd át a fenti példát (szinkron REST vagy aszinkron üzenet a fizetési folyamatban). Melyik opció visszafordíthatósága jobb, ha a rendelés-feldolgozó szolgáltatás belső interfésze jól definiált? Indokold meg a választ a visszafordíthatóság három költségösszetevője (megvalósítási, adatmigrációs, koordinációs) alapján.

A helyes válasz: a szinkron REST-ről aszinkron üzenetküldésre való áttérés közepesen visszafordítható, mert a megvalósítási költség mérsékelt (az állapotgép és a hibakezelés átalakítása szükséges), az adatmigrációs költség alacsony (a rendelésadatok sértetlenek maradnak, legfeljebb az állapotátmeneteket kell nyomon követni), a koordinációs költség pedig alacsony, ha a fizetési szolgáltatóval kötött szerződés nem változik. Az aszinkronról szinkronra való visszatérés hasonlóan közepes, de a komponenshatárok megtartása esetén a megvalósítási költség elfogadható. A kulcs a belső interfész stabilitása: ha az interfész stabil, a döntés visszafordíthatósága megnő, függetlenül az iránytól.

4.1.20 Összefoglalás

A döntés az architektúra alapegysége: minden szerkezeti elem egy korábbi döntés következménye. A döntések visszafordíthatósága határozza meg, hogy mekkora körültekintéssel és milyen időzítéssel érdemes őket meghozni. A reverzibilis döntések gyorsak lehetnek, az irreverzibilis döntések alaposabb elemzést igényelnek. A döntéshozatal időzítése és a bizonytalanság kezelése a last responsible moment elv és az információszerzés együttes alkalmazásával érhető el. A döntési napló írásban rögzíti a kontextust, az opciókat, a következményeket és a visszafordíthatóságot, így a későbbi olvasó számára is nyomon követhetővé válik a döntés oka és a feladott alternatívák.

Ez a fejezet megalapozza a következő egység témáját, amely a quality attribútumok és a trade-off analízis részletes tárgyalásával foglalkozik. Az itt bevezetett döntési napló és visszafordíthatósági szemlélet a későbbi fejezetekben is visszatérő eszközként szolgál.

4.2 Quality attributes és trade-off analízis

A minőségi attribútumok tervezésekor az egyik leggyakoribb csapda az, hogy a csapat minden elvárást egyszerre, maximális szinten akar kielégíteni: legyen a rendszer gyors, mindig elérhető, tökéletesen biztonságos, könnyen módosítható és olcsó. A gyakorlatban ezek a célok gyakran versengenek egymással, és a korlátos erőforrások – számítási kapacitás, hálózati késleltetés, fejlesztői idő, üzemeltetési költség, szabályozási keret – miatt nem teljesíthetők egyszerre. A trade-off analízis nem azt jelenti, hogy lemondunk a minőségről, hanem azt, hogy tudatosan, a kontextus ismeretében választjuk meg, melyik attribútum milyen mértékben érvényesüljön, és ezt a döntést mérhető, dokumentált formában rögzítjük.

Ez a fejezet a minőségi attribútumok közötti feszültség természetét, a kompromisszumok felismerésének és számszerűsítésének módszerét, valamint a trade-off mátrix gyakorlati használatát mutatja be. Az itt leírt fogalmakra épít a következő egység, amely az Architecture Decision Records (ADR) részletes tárgyalásával foglalkozik.

4.2.1 Minőségi attribútumok áttekintése

A szoftverrendszerek minőségét több dimenzió mentén értékeljük. Az ISO/IEC 25010 szabvány a termékminőséget nyolc fő jellemzőre bontja: funkcionális alkalmasság, teljesítmény-hatékonyság, kompatibilitás, használhatóság, megbízhatóság, biztonság, karbantarthatóság és hordozhatóság. Az architekturális tervezés szempontjából ezek közül kiemelkedik három, amelyek a legtöbb rendszer esetében alapvető feszültséget okoznak: a teljesítmény, a rendelkezésre állás és a biztonság. E három attribútum nem egymástól független tulajdonság, hanem rendszerdinamikai kapcsolatban áll egymással; az egyik javítása gyakran a másik rovására történik.

A minőségi attribútumokat nem elég általánosan megfogalmazni – például „legyen gyors a rendszer" –, mert az ilyen állítások nem ellenőrizhetők, és nem adnak támpontot a tervezési döntésekhez. A könyv konvenciója szerint a minőségi követelményeket quality attribute scenario formában adjuk meg, amely mindig tartalmazza az ingert, a környezetet, a választ és a mértéket. Ez a forma lehetővé teszi, hogy a trade-off analízis során a különböző alternatívákat ugyanazon forgatókönyvek mentén hasonlítsuk össze.

Például egy webes áruház fizetési szolgáltatásánál a három fókuszattribútumra a következő forgatókönyvek fogalmazhatók meg:

- **Teljesítmény:** amikor 1 000 egyidejű felhasználó aktív pénztári munkamenetet folytat (inger), a fizetési végpont hívása során (környezet) a rendszer a kérések 95. percentilisére 300 ms alatt válaszol (válasz), a mérési ablak 5 perc, a mérőszám pedig a 95. percentilis válaszidő ms-ban (mérték).
- **Rendelkezésre állás:** ha az elsődleges adatbázis-példány leáll (inger), az éles környezetben, csúcsidőn kívül (környezet) a rendszer 30 másodpercen belül átáll a tartalék példányra, és a sikeres kérések aránya nem esik 99,5% alá (válasz); a havi rendelkezésre állási SLO 99,9%, az error budget $1 - \text{SLO}$ (mérték).
- **Biztonság:** ha egy támadó 10 percen belül 50 sikertelen hitelesítési kísérletet hajt végre ugyanarra a fiókra (inger), a rendszer a hitelesítési szolgáltatásban (környezet) blokkolja a további próbálkozásokat, hibaüzenetet ad, és biztonsági eseményt naplóz (válasz); a blokkolás 1 másodpercen belül megtörténik, és az esemény a naplóba kerül (mérték).

Ezek a forgatókönyvek már önmagukban is jelzik a feszültséget: a 300 ms-os válaszidő elérése a fizetési végponton megkövetelheti a hitelesítési adatok gyorsítótárban tartását, ami viszont csökkentheti a biztonság szintjét, ha a gyorsítótár kompromittálódik. A 99,9%-os rendelkezésre állás redundáns infrastruktúrát és automatikus failovert igényel, ami növeli a komplexitást és a költséget, és hatással lehet a teljesítményre is.

4.2.2 A feszültség forrásai

A minőségi attribútumok közötti feszültség gyökere az erőforrások végessége. Minden rendszer korlátos CPU-, memória-, hálózati és tárolókapacitással rendelkezik, és minden további minőségi követelmény valamilyen erőforrást fogyaszt. A fejlesztői és üzemeltetési kapacitás ugyanilyen korlát: egy bonyolultabb, több rétegű redundanciát és biztonsági mechanizmust tartalmazó rendszer több kódot, több konfigurációt és több üzemeltetési eljárást igényel.

4.2.3 Teljesítmény és biztonság

A teljesítmény és a biztonság közötti feszültség az egyik legklasszikusabb példa. A biztonsági mechanizmusok – titkosítás, digitális aláírás, hozzáférés-vezérlés, naplózás, behatolásészlelés – mind számítási többletet és gyakran hálózati késleltetést okoznak. Egy TLS-kézfogás elvégzése

minden egyes kapcsolatfelvételkor növeli a válaszidőt; a teljes körű naplózás minden kérésnél I/O-műveletet igényel; a többlépcsős hitelesítés emberi reakcióidőt visz a folyamatba.

Ez nem jelenti azt, hogy a biztonságot fel kell áldozni a teljesítményért. A cél az, hogy megtaláljuk azt a pontot, ahol a biztonsági intézkedések költsége elfogadható a teljesítménycélok mellett. Például a TLS-munkamenetek újrafelhasználása csökkenti a kézfogások számát, a naplózás aszinkron pufferen keresztül történhet, a hitelesítési gyorsítótár pedig csak rövid élettartamú, lejárati idővel ellátott tokeneket tárolhat. Mindegyik megoldás maga is kompromisszum: az aszinkron naplózás csökkenti az I/O-terhelést, de egy folyamat összeomlásakor elveszhetnek a pufferben lévő bejegyzések; a token-gyorsítótár gyorsabb, de növeli a támadási felületet, ha a memória kiolvasható.

4.2.4 Rendelkezésre állás és teljesítmény

A rendelkezésre állás növelésének klasszikus eszközei – replikáció, redundancia, automatikus hibaáthelyezés, terheléselosztás – jellemzően javítják a teljesítményt is, mert több erőforrás áll rendelkezésre, és a forgalom eloszlik. A kapcsolat azonban nem mindig pozitív. A szinkron replikáció például garantálja, hogy a tartalék példány mindig naprakész, de minden írási műveletnek meg kell várnia a replika visszaigazolását, ami növeli a válaszidőt. Az aszinkron replikáció gyorsabb, de egy átállásnál adatvesztés léphet fel, ami csökkenti a konzisztencia és a rendelkezésre állás közötti egyensúlyt.

A gyorsítótárazás szintén kettős hatású. A cache csökkenti az adatbázis terhelését és javítja a válaszidőt, de a gyorsítótárban tárolt adat elavulhat, ami a felhasználó számára inkonzisztens állapotot eredményezhet. A rétegelt gyorsítótárak – böngésző, CDN, alkalmazás, adatbázis – mindegyike egy-egy konzisztencia-kompromisszumot hordoz. A tervezőnek el kell döntenie, hogy milyen frissességi követelményt támaszt a rendszerrel szemben, és ez hogyan viszonyul a teljesítménycélokhoz.

4.2.5 Biztonság és rendelkezésre állás

A biztonság és a rendelkezésre állás közötti feszültség kevésbé nyilvánvaló, de ugyanolyan fontos. A szigorú hitelesítési és hozzáférési szabályok csökkenthetik a rendszer elérhetőségét a jogosult felhasználók számára is: a többlépcsős hitelesítés hibája kizárhat valakit, a rate limiting túl szigorú beállítása blokkolhat egy legitim automatizált klienst, a biztonsági frissítések telepítése pedig leállást igényelhet.

Egy elosztott rendszerben a biztonsági intézkedések gyakran további hálózati ugrásokat jelentenek: a kéréseknek át kell haladniuk tűzfalon, API gatewayen, hitelesítési szolgáltatáson, mielőtt elérnék az üzleti logikát. Minden egyes ugrás potenciális meghibásodási pont, ami csökkenti a teljes rendszer rendelkezésre állását, ha nem megfelelően tervezik meg. A zero trust architektúra például minden egyes szolgáltatásközi hívást hitelesít és engedélyez, ami növeli a biztonságot, de növeli a késleltetést és a komplexitást is.

A három attribútum közötti feszültség gyakran egy háromszögként ábrázolható, ahol a csúcsok a teljesítmény, a rendelkezésre állás és a biztonság. A gyakorlatban egy rendszer tervezésekor e háromszögön belül kell elhelyeznünk magunkat; a csúcsok egyidejű elérése nem lehetséges. A trade-off analízis éppen azt teszi lehetővé, hogy ezt a pozíciót tudatosan válasszuk meg.

4.2.6 A trade-off analízis kerete

A kompromisszumok kezeléséhez szisztematikus megközelítésre van szükség. Az alábbi lépések egy olyan keretet adnak, amelyet a fejezetben végig használunk, és amely a laborfeladat alapját is képezi:

1. **Kontextus rögzítése.** Mielőtt bármilyen attribútumot értékelünk, tisztázni kell a rendszer határait, a felhasználói szerepköröket, a külső rendszereket, a várható terhelést és a szervezeti kényszereket. A C4 context szintű diagram segít abban, hogy minden érintett ugyanazt a rendszerképet lássa.
2. **Attribútumok kiválasztása.** A gyakorlatban nem lehet minden minőségi attribútumot egyszerre optimalizálni. A trade-off analízishez ki kell választani a döntés szempontjából kritikus attribútumokat – jelen fejezetben a teljesítményt, a rendelkezésre állást és a biztonságot –, és ezekhez mérhető forgatókönyveket kell rendelni.
3. **Alternatívák megfogalmazása.** Minden architekturális döntésnél legalább két, de gyakran három-öt alternatívát érdemes megvizsgálni. Az alternatívák nem feltétlenül teljesen eltérő architektúrák; lehetnek a részletekben különböző kialakítások, például szinkron vagy aszinkron replikáció, cache beiktatása vagy a hitelesítés helye.
4. **Értékelés mérőszámokkal.** Minden alternatívát ugyanazon quality attribute scenario-k mentén értékelünk. Az értékelés lehet kvalitatív (például „jó”, „közepes”, „gyenge”), de törekedni kell a számszerűsítésre, ahol lehetséges. A mérőszámokat mértékegységgel és mérési ablakkal együtt adjuk meg.
5. **Bizonytalanság jelölése.** A tervezési fázisban a legtöbb érték becslés, nem pedig mért eredmény. A trade-off mátrixban egyértelműen el kell különíteni, hogy egy adott érték mért adat, becslés, szimuláció vagy feltételezés. A bizonytalanságot konfidenciaintervallummal, szórással vagy legalább szóbeli minősítéssel jelöljük.
6. **Döntés és következmények dokumentálása.** A kiválasztott alternatívát és annak indoklását ADR formájában rögzítjük, megadva a kontextust, a döntést, a feladott alternatívákat és a következményeket. Ez biztosítja, hogy a jövőbeli olvasó megértse a döntés okát.

A döntés számszerűsítése során gyakran használunk súlyozott pontszámokat. Például egy 1-től 5-ig terjedő skálán értékeljük az egyes alternatívákat az attribútumonkénti teljesítményük szerint, majd a pontszámokat a szervezeti prioritások alapján súlyozzuk. Ez a módszer hasznos, de vigyázni kell arra, hogy a pontszámok ne keltsenek hamis pontosság érzetét. A súlyok és a pontszámok mögött mindig a kontextus és a mérési feltételezések állnak, amelyeket dokumentálni kell.

A bizonytalanság jelölésére a mátrixban egy egyszerű jelölést vezethetünk be: a mért értékeket `(m)`, a becsléseket `(b)`, a szimulációs eredményeket `(s)`, a feltételezéseket pedig `(f)` jelöljük. Ha egy mérési eredményhez tartozik konfidenciaintervallum, azt a cellában tüntetjük fel, például `p95 < 280 ms (m, 95% CI: 250–310 ms)`.

4.2.7 Trade-off mátrix készítése

A trade-off mátrix egy egyszerű táblázat, amelynek soraiban a kiválasztott minőségi attribútumok, oszlopaiban pedig a vizsgált alternatívák szerepelnek. A cellák tartalmazzák az adott alternatíva adott attribútumra vonatkozó értékelését, a mérési vagy becslési alapot, valamint a bizonytalanság

jelölését. A mátrix célja nem az, hogy automatikusan kijelölje a győztest, hanem az, hogy a döntést átláthatóvá és vitathatóvá tegye.

Az alábbi példa egy fiktív, de realisztikus döntési helyzetet mutat be ügyfél-regisztrációs szolgáltatás tervezéséhez. A kontextus a következő:

- **Rendszer:** ügyfél-regisztrációs és hitelesítési szolgáltatás egy e-kereskedelmi platform számára.
- **Rendszerhatár:** a szolgáltatás a webfrontend és a mobilalkalmazás felől fogad regisztrációs és bejelentkezési kéréseket, valamint a belső ügyfélszolgálati rendszer felől olvasási kéréseket.
- **Terhelés:** csúcsidőben 800 regisztráció/óra, 2 500 bejelentkezés/óra; a csúcs időtartama legfeljebb 3 óra.
- **Elvárások:** p95 válaszidő < 300 ms; havi rendelkezésre állás SLO 99,9%; a hitelesítési adatok védelme a jelszó-tárolásra vonatkozó ágazati ajánlások szerint.

Három alternatívát vizsgálunk:

- **A) Moduláris monolit** – egyetlen telepíthető szolgáltatás, amely a regisztrációs, hitelesítési és felhasználói profil logikát tartalmazza. Az adatbázis egyetlen relációs példány.
- **B) Mikroszolgáltatás cache-el** – külön hitelesítési szolgáltatás, Redis-alapú token-gyorsítótárral, az adatbázis replikált, az olvasások a replikáról történnek.
- **C) Eseményvezérelt aszinkron** – a regisztráció eseményt generál, a hitelesítési adatok tárolása aszinkron folyamatban történik, a bejelentkezés szinkron, de a token kiadása előtt a hitelesítési szolgáltatás csak a gyorsítótárat kérdezi le.

A trade-off mátrix a következőképpen nézhet ki (a pontszámok 1–5 skálán, ahol 5 a legjobb; a zárójelben a bizonytalanság jelölése):

Attribútum	Forgatókönyv-mutató	A) Monolit	B) Mikroszolgáltatás + cache	C) Aszinkron
Teljesítmény	p95 válaszidő csúcsidőben	3 (b) – becsült 350–420 ms	5 (b) – becsült 180–240 ms	4 (b) – becsült 220–300 ms
Rendelkezésre állás	havi SLO, hibaáthelyezési idő	3 (b) – egy példány, manuális failover	5 (b) – replikált DB, automatikus failover	4 (b) – részben aszinkron, üzenetkésés kockázata
Biztonság	jogosulatlan hozzáférés blokkolása, jelszótárolás	4 (b) – egyszerűbb, kevesebb támadási felület, de nincs elkülönített védelem	3 (b) – token-gyorsítótár növeli a támadási felületet	4 (b) – a hitelesítési logika elkülönül, de az aszinkronitás új támadási utakat nyithat
Karbantarthatóság (kontroll)	változtatási költség, csapatméret	5 (b) – egy kódbázis, egyszerű deployment	3 (b) – több szolgáltatás, hálózati hibakezelés	2 (b) – eseményvezérelt komplexitás
Költség (kontroll)	havi üzemeltetési költség, erőforrásigény	4 (b) – alacsony infrastruktúra	2 (b) – cache, replika, több szolgáltatás	3 (b) – üzenetkezelő, de nincs replika

A táblázatban a (b) jelölés arra utal, hogy az értékek becslések, nem pedig mért eredmények. A (m) jelölést akkor használnánk, ha például egy prototípus terheléses tesztje alapján adnánk meg a válaszidőt. A pontszámok mellett a cellákban megjelenő konkrét értékek – például a becsült válaszidő-intervallum – segítenek abban, hogy a döntést ne csak szubjektív pontszámokra alapozzuk.

A mátrix alapján a B) alternatíva teljesítmény és rendelkezésre állás szempontjából a legjobb, de a biztonság és a költség dimenziójában gyengébb. A C) alternatíva kiegyensúlyozottabb, de a karbantarthatósági kockázat magas. Az A) alternatíva a legegyszerűbb, de a teljesítménycél valószínűleg nem teljesül csúcsidőben. A döntés a súlyozástól függ: ha a szervezet számára a biztonság a legfontosabb, az A) vagy C) lehet a nyerő; ha a teljesítmény és a rendelkezésre állás a fő elvárás, a B) tűnik észszerűnek. Ezt a döntést a következő szakaszban ADR segítségével rögzítjük.

4.2.8 Gyakorlati minta: ügyfél-regisztrációs szolgáltatás

A fenti példát most tovább bontjuk egy teljes döntési folyamatban, hogy a trade-off analízis lépései a gyakorlatban is követhetők legyenek. A minta célja, hogy a laborfeladat előtt egy konkrét, jól követhető eljárást mutasson.

4.2.9 1. lépés: Kontextus és rendszerhatár rögzítése

A kontextust C4 context szintű diagramban rögzítjük. A diagrams-as-code elvnek megfelelően Mermaid jelölést használunk:

```
graph TD
  U[Ügyfél] -->|HTTPS, regisztráció/bejelentkezés| S[Ügyfél-regisztrációs és hitelesítési
↪  szolgáltatás]
  M[Mobilalkalmazás] -->|HTTPS, regisztráció/bejelentkezés| S
  C[Ügyfélszolgálati rendszer] -->|HTTPS, ügyfélprofil-lekérdezés| S
  S -->|SQL| D[(Felhasználói adatbázis)]
  D -->|replikáció| R[(Replika adatbázis)]
  S -->|esemény| E[Eseménykezelő]
  E -->|értesítés| N[Értesítési szolgáltatás]

  subgraph Rendszerhatár[Ügyfél-identitás alrendszer]
    S
    D
    R
  end
```

A diagramon a rendszerhatárt egyértelműen jelöltük; a külső rendszerek – ügyfél, mobilalkalmazás, ügyfélszolgálati rendszer, eseménykezelő, értesítési szolgáltatás – a határon kívül szerepelnek. A kapcsolatokon feltüntettük a protokollt és a szinkronitást.

A szöveges kontextus tartalmazza a terhelési adatokat, az SLO-kat és a szabályozási követelményeket. Fontos, hogy a kontextust ne csak a döntéshozó csapat ismerje, hanem az architektúra-dokumentumban verziózott formában rögzítve legyen.

4.2.10 2. lépés: Quality attribute scenario-k definiálása

A három fókuszattribútumhoz a korábban bemutatott forgatókönyveket használjuk, de most kiegészítjük őket a mérési ablakkal és a bizonytalansággal. Például:

- **Teljesítmény:** 2 500 bejelentkezés/óra csúcsterhelés mellett a bejelentkezési végpont p95 válaszideje < 300 ms. A mérési ablak 5 perc. A becsült érték a B) alternatívánál 180–240 ms, de ez szimulációból származik, nem éles mérésből.
- **Rendelkezésre állás:** havi SLO 99,9%; az error budget $1 - 0{,}999 = 0{,}001$, azaz havi 43,2 perc állásidő. A failover idő cél < 30 s. A replikált adatbázis becsült failover ideje 10–25 s.
- **Biztonság:** a jelszótárolás Argon2id algoritmussal, a tokenek csak 15 percig érvényesek, a gyorsítótár nem tárol jelszókivonatot, csak a tokeneket. A becsült támadási felület növekedése a cache miatt kvantitatívan nehezen mérhető, ezért kvalitatív skálán értékeljük.

A forgatókönyvekben a mérőszámokat mértékegységgel és mérési ablakkal adjuk meg. A `99{,}9\%` jelölés a magyar tizedesvesszőt használja, ahogyan a könyv konvenciója előírja.

4.2.11 3. lépés: Alternatívák értékelése

Mindhárom alternatívát ugyanazon forgatókönyvek mentén értékeljük. A mátrixot a fenti táblázatban foglaltuk össze. Az értékelés során a következő szempontokat is figyelembe vesszük:

- **A) Monolit** esetén a teljesítménycél csúcsidőben valószínűleg nem teljesül, mert a bejelentkezési logika és az adatbázis-kapcsolat egyetlen szűk keresztmetszetet képez. A rendelkezésre állás is gyenge, mert nincs replikáció; egy adatbázis-hiba a teljes szolgáltatást megbénítja.
- **B) Mikroszolgáltatás + cache** esetén a teljesítmény és a rendelkezésre állás kiváló, de a token-gyorsítótár biztonsági kockázatot hordoz: ha a cache memória kiolvasható, a tokenek ellophatók. Ezt a kockázatot csökkentheti a rövid élettartam és a cache titkosítása, de ez további komplexitást jelent.
- **C) Aszinkron** esetén a regisztráció során az ügyfél nem kap azonnali visszaigazolást, ami használhatósági problémát okozhat, és az eseménykezelő kiesése a regisztrációk elvesztéséhez vezethet. Ugyanakkor a hitelesítési logika elkülönítése biztonsági szempontból előnyös.

A mátrixban a karbantarthatóságot és a költséget kontroll-attribútumként tüntettük fel, mert ezek nem a három fókuszattribútum közé tartoznak, de a döntés szempontjából lényegesek. A laborfeladatban a résztvevők három attribútumot választanak ki, de a kontroll-attribútumok használata ajánlott.

4.2.12 4. lépés: Döntés és ADR rögzítése

A mátrix alapján – feltételezve, hogy a szervezet számára a teljesítmény és a rendelkezésre állás a legfontosabb, miközben a biztonságot a rövid lejáratú tokenekkel és a cache titkosításával elfogadható szinten tartja – a B) alternatívát választjuk. A döntést ADR formájában rögzítjük:

```
# ADR-2025-014: Ügyfél-hitelesítési szolgáltatás architektúrája

**Állapot:** Elfogadva
**Dátum:** 2025-04-10
**Kontextus:**
Az e-kereskedelmi platform ügyfél-regisztrációs és hitelesítési szolgáltatását
újra kell tervezni. Csúcsidőben 2 500 bejelentkezés/óra, 800 regisztráció/óra
várható. A p95 válaszidőre 300 ms-os cél, a havi rendelkezésre állás SLO 99,9%.
A biztonsági követelmény: jelszótárolás Argon2id, a tokenek élettartama maximum
15 perc. A fejlesztőcsapat 3 szenior és 2 medior fejlesztőből áll; az
üzemeltetés korlátozott kapacitással rendelkezik.

**Döntés:**
A szolgáltatást külön hitelesítési mikroszolgáltatásként valósítjuk meg,
```

```
Redis-alapú token-gyorsítótárral, replikált relációs adatbázissal. A tokeneket
csak a gyorsítótárban tároljuk, rövid lejárati idővel; a gyorsítótár adatát
AES-256 titkosítással védjük. Az adatbázis olvasási kérései a replikáról
történnek.

**Alternatívák:**
- A) Moduláris monolit: egyszerűbb, de a teljesítménycél nem teljesül.
- C) Eseményvezérelt aszinkron: jobb izoláció, de az aszinkronitás
  használhatósági és üzenetkezelési kockázatot hordoz.

**Következmények:**
- Növekszik az infrastruktúra-költség (cache, replika, több szolgáltatás).
- A token-gyorsítótár biztonsági kockázatát titkosítással és rövid TTL-lel
  csökkentjük, de a kockázat nem nulla.
- A rendelkezésre állás várhatóan 99,95% fölé emelkedik, a hibaáthelyezés
  automatizált.
- A csapatnak gondoskodnia kell a cache és a replika üzemeltetési
  eljárásairól, runbookokról.

**Ellenőrzési szempontok:**
- Terheléses teszt igazolja a p95 < 300 ms célt.
- Chaos-teszt igazolja a 30 s alatti failovert.
- Biztonsági átvilágítás ellenőrzi a cache titkosítását és a token-élettartamot.
```

Az ADR-ben a döntés, az alternatívák és a következmények mellett az ellenőrzési szempontokat is feltüntettük, mert a döntés minősége csak akkor igazolható, ha a későbbi mérések megerősítik a feltételezéseket. Az ADR verziózva van, és a kódbázisban elérhető.

4.2.13 5. lépés: Utólagos mérés és visszacsatolás

A döntés után a következő iterációban terheléses tesztet, chaos-tesztet és biztonsági felülvizsgálatot végzünk. Az eredményeket visszaírjuk az ADR-be vagy egy kapcsolódó mérési jegyzőkönyvbe. Ha a mért p95 válaszidő például 340 ms, akkor a döntést újra kell értékelni, és lehet, hogy további optimalizálásra van szükség. A trade-off analízis tehát nem egyszeri esemény, hanem folyamatos tevékenység.

4.2.14 Labor: Trade-off mátrix három attribútumra

A fejezet laborfeladata a trade-off mátrix készítésének gyakorlása. A cél, hogy a résztvevő egy valószerű döntési helyzetben önállóan alkalmazza a fejezetben bemutatott keretet.

4.2.15 Kiinduló állapot

A résztvevő kap egy rövid rendszerleírást, például egy dokumentumkezelő szolgáltatás vagy egy IoT-eszközöket fogadó adatgyűjtő rendszer kontextusát. A leírás tartalmazza a funkcionális követelményeket, a várható terhelést, a felhasználói szerepköröket és a szervezeti kényszereket. A résztvevőnek ki kell választania három minőségi attribútumot, és azokhoz quality attribute scenario-kat kell megfogalmaznia. A feladat elvégzéséhez nem szükséges éles rendszer; a reprodukálhatóságot egy jegyzetfüzet vagy dokumentum biztosítja.

4.2.16 Elvárt eredmény

A résztvevő a következőket készíti el:

1. C4 context szintű diagram diagrams-as-code formában (Mermaid vagy PlantUML), amelyen a rendszerhatár és a külső rendszerek szerepelnek.
2. Három quality attribute scenario, amelyek tartalmazzák az ingert, a környezetet, a választ és a mértéket.
3. Legalább két architekturális alternatíva rövid leírása, komponens-felelősségekkel és interfészekkel.
4. Egy trade-off mátrix, amelynek soraiban a három kiválasztott attribútum, oszlopaiban az alternatívák szerepelnek. A cellákban a mérőszámok mértékegységgel és mérési ablakkal, valamint a bizonytalanság jelölésével (`(m)`, `(b)`, `(s)`, `(f)`) szerepelnek.
5. Egy döntési naplóbejegyzés ADR formában, amely tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
6. A mért és a várt eredmények elkülönítése: a dokumentumban külön szakaszban jelezni kell, mely értékek származnak mérésből, és melyek becslések.

4.2.17 Ellenőrzési szempontok

A labor ellenőrzése során az alábbiakat értékeljük:

- A C4 diagramon a rendszerhatár egyértelműen jelölt-e.
- A quality attribute scenario-k minden kötelező elemét tartalmazzák-e: inger, környezet, válasz, mérték.
- A trade-off mátrix celláiban szerepel-e a mértékegység és a mérési ablak.
- A bizonytalanság jelölése következetes-e, és a mért értékek elkülönülnek-e a becslésektől.
- Az ADR tartalmazza-e a kontextust, a döntést, az alternatívákat és a következményeket.
- A döntés indoklása kapcsolódik-e a mátrixban bemutatott trade-offokhoz.

4.2.18 Visszaállítási út

Mivel a labor dokumentumkészítés, nincs szükség infrastruktúra visszaállítására. Ha azonban a résztvevő valamilyen szimulációt vagy PoC-t futtat, azt izolált környezetben kell elvégezni, és a labor elején rögzíteni kell a környezet konfigurációját. A labor befejezése után a használt erőforrásokat le kell állítani, a dokumentumot pedig verziózni kell.

4.2.19 Tipikus hibák és ellenőrzőlista

A trade-off analízis során elkövetett leggyakoribb hibák a következők:

1. **Egy attribútum abszolutizálása.** A csapat kijelenti, hogy „a biztonság mindennél fontosabb”, és ennek rendeli alá az összes többi követelményt. Ez a gyakorlatban olyan rendszerhez vezet, amelyet a jogosult felhasználók is alig tudnak használni, vagy amelynek üzemeltetési költsége meghaladja az üzleti értéket. A trade-off analízis éppen azt mutatja meg, hogy minden attribútumnak van ára.
2. **A kontextus elhanyagolása.** Ugyanaz a megoldás különböző terhelés, csapatméret vagy szabályozási környezet mellett teljesen más eredményre vezethet. A mátrixban a kontextust mindig rögzíteni kell, és a döntést erre a kontextusra kell vonatkoztatni.

3. **A mérőszám hiánya vagy pontatlansága.** A „gyors”, „megbízható”, „biztonságos” kifejezések nem alkalmasak döntésre. Minden attribútumhoz rendelni kell egy mérhető mutatót, mértékegységgel és mérési ablakkal.
4. **A bizonytalanság elhallgatása.** A tervezési fázisban a legtöbb érték becslés, de a döntéshozók gyakran mért adatként kezelik őket. A mátrixban jelölni kell, hogy melyik érték mérés, becslés, szimuláció vagy feltételezés.
5. **A döntés és a mátrix összekapcsolásának hiánya.** Előfordul, hogy a csapat elkészíti a mátrixot, de a végső döntést mégis megérzés alapján hozza meg. A döntési naplóban explicit módon hivatkozni kell a mátrixban szereplő trade-offokra.
6. **A trade-off egyszeri eseményként való kezelése.** A minőségi attribútumok közötti feszültség a rendszer teljes életciklusa alatt változik. A döntést időről időre felül kell vizsgálni, amikor új mérési adatok, üzleti követelmények vagy technológiai változások állnak rendelkezésre.

A fejezethez tartozó ellenőrzőlista:

- ☐ A kontextus rögzítve van, és tartalmazza a rendszerhatárt, a terhelést, a felhasználói szerepköröket és a szervezeti kényszereket.
- ☐ A kiválasztott minőségi attribútumokhoz quality attribute scenario-k tartoznak, ingerrel, környezettel, válasszal és mértékkel.
- ☐ Legalább két alternatíva van, és mindegyik komponensei felelősséggel és interfésszel vannak leírva.
- ☐ A trade-off mátrix celláiban a mérőszámok mértékegységgel és mérési ablakkal szerepelnek.
- ☐ A bizonytalanság jelölése következetes, és a mért értékek elkülönülnek a becslésektől.
- ☐ A döntési napló ADR formában rögzíti a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
- ☐ A döntés indoklása összekapcsolódik a mátrixban bemutatott trade-offokkal.

Az önellenőrző kérdés a fejezet végére: **Milyen konkrét mérési eredmény változtatná meg a példában szereplő döntést, ha kiderülne, hogy a token-gyorsítótár biztonsági kockázata magasabb a becsültnél?** A válaszban meg kell határozni, melyik attribútum értéke romlik, hogyan hat ez a mátrix összpontszámára, és milyen alternatív lépést lehetne tenni a kockázat csökkentésére.

4.2.20 Összefoglalás

A minőségi attribútumok közötti feszültség nem hiba, hanem a rendszertervezés természetes velejárója. A trade-off analízis lehetővé teszi, hogy ezt a feszültséget ne érzésre, hanem szisztematikus, mérhető és dokumentált módon kezeljük. A fejezetben bemutatott keret – kontextus rögzítése, quality attribute scenario-k definiálása, alternatívák értékelése, bizonytalanság jelölése, döntési napló készítése – a későbbi fejezetekben is visszatér, különösen a minőségi forgatókönyvek (Quality attribute scenarios) és az SLO-k, error budgetek (SLO, SLI, error budget és mérés) tárgyalásánál. A következő egység, az ADR-ek és döntési napló részletes ismertetése, az itt bemutatott döntési keretre épít, és megmutatja, hogyan lehet a trade-off elemzést a mindennapi döntéshozatali folyamatba ágyazni.

4.3 Architecture Decision Records (ADR) és döntési napló

A döntések akkor válnak a rendszer részévé, amikor nemcsak meghozzuk őket, hanem rögzítjük a miértjüket is. Az Architecture Decision Record (ADR) ezt a miértet tárolja. Egy jól vezetett ADR-készlet nem adminisztrációs teher, hanem a rendszer időbeli viselkedésének egyik legmegbízhatóbb térképe: megmutatja, milyen feltételek között, milyen alternatívák közül választottunk, és milyen következményekkel számoltunk.

Ebben az alfejezetben először az ADR felépítését és tipikus mezőit tekintjük át, majd a verziózást és az életciklust, végül a döntési napló és a kóddokumentáció kapcsolatát. A fejezetet egy rövid, három ADR-ből álló labor zárja.

4.3.1 Miért van szükség ADR-re?

Az architektúradöntések egy része a kódban is látható, más részük azonban nem. A kód megmutatja, *hogyan* működik a rendszer, de ritkán mutatja meg, *miért* nem úgy működik, ahogyan más megoldásokkal működne. Ha a csapat csak a kódot olvassa, a döntések indokai rejtve maradnak, és a jövőbeli fejlesztő könnyen „rossz döntésnek” hiszi azt, ami az adott kontextusban helyes volt.

Az ADR célja nem a döntések tökéletes utólagos igazolása, hanem az, hogy a döntés pillanatában rendelkezésre álló ismereteket, korlátokat és alternatívákat lekérdezhetővé tegye. A dokumentációs szerződésünk szerint az ADR-ek száma nem cél, hanem a döntések nyomon követésének eszköze. Egy ADR akkor jó, ha a jövőbeli olvasó megérti belőle a döntés okát és a feladott alternatívákat.

4.3.2 Az ADR felépítése

Egy ADR nem szabad műfaj. A minimálisan szükséges mezők: kontextus, döntés, alternatívák, következmények és állapot. Ezt a szerkezetet a gyakorlatban több sablon is támogatja; a Michael Nygard-féle megközelítés az egyik leggyakrabban használt kiindulópont. A mezők szerepe:

- **Cím:** tömör, egyértelmű mondat, például „PostgreSQL választása írási tranzakciókhoz”. A cím ne legyen félrevezetően általános, mint „adatbázis döntés”.
- **Státusz:** Javasolt, Elfogadott, Elavult, Felülvizsgált vagy Visszavont.
- **Dátum:** a döntés dátuma, lehetőleg nap pontossággal.
- **Döntési kontextus:** miért van szükség döntésre, milyen erők hatnak, milyen korlátok érvényesek. Itt szerepeljenek a releváns nem funkcionális követelmények is, például „az olvasási oldal p95 késleltetése 150 ms alatt kell maradjon, miközben legalább 500 olvasás/másodperc érkezik”.
- **Döntés:** a választott megoldás pontos leírása. Ez lehet technológia, minta, folyamat, csapatelmélet vagy akár egy „nem döntünk most” halasztás is.
- **Alternatívák:** a vizsgált lehetőségek és azok rövid összehasonlítása. Nem kell minden létező opciót felsorolni, de a reálisan szóba kerülőket igen.
- **Következmények:** a döntés várható pozitív és negatív hatásai, beleértve a kompromisszumokat, az új kockázatokat és az esetleges kompenzáló intézkedéseket.
- **Kapcsolódó döntések:** hivatkozások más ADR-ekre, amelyek ezt a döntést befolyásolják, vagy amelyeket ez a döntés befolyásol.

Ezek közül a legfontosabb a **kontextus**. Ha a kontextus hiányos, a döntés nem védhető meg a későbbi „ezt miért így csináltuk?” kérdéssel szemben. A kontextus rögzíti a pillanatot: milyen üzleti cél, terhelési adat, szervezeti kényszer vagy technikai adósság vezetett a döntéshez.

4.3.3 ADR sablon példa

Az alábbi Markdown sablon a fejezetben használt minimális szerkezetet mutatja. A mezők sorrendje nem kötelező, de a hiányzó mezők rontják a döntés utólagos értelmezhetőségét.

```
# ADR-001: [Cím]

- Állapot: Elfogadott
- Dátum: 2025-02-10
- Döntéshozó: [szerepkör vagy csapat]
- Kapcsolódó ADR-ek: nincs / ADR-000, ADR-002

## Kontextus

[Miért van szükség döntésre? Milyen erők, korlátok, mérések,
quality attribute scenario-k játszanak szerepet?]

## Döntés

[A választott megoldás pontos leírása.]

## Alternatívák

### 1. [Alternatíva neve]

[Leírás, előnyök, hátrányok.]

### 2. [Alternatíva neve]

[Leírás, előnyök, hátrányok.]

## Következmények

### Pozitív

- ...

### Negatív és kockázatok

- ...

### Kompenzáló intézkedések

- ...

## Jóváhagyás és felülvizsgálat

- Elfogadva: [dátum, szerepkör]
- Utolsó felülvizsgálat: [dátum]
```

A sablonban a **Döntéshozó** mező nem minden szervezetnél kötelező, de hasznos, mert rögzíti, ki viseli a döntés felelősségét. Ez nem a hibáztatást szolgálja, hanem a visszakérdezhetőséget.

4.3.4 Verziózás és életciklus

Az ADR-ek nem egyszeri dokumentumok, hanem időben változó állapotgép elemei. Egy döntés először **Javasolt** állapotba kerül, amikor a csapat felveti és megvitatja. Ha a döntés elfogadásra kerül, az állapot **Elfogadott** lesz. Ha a döntést később felülírja egy újabb döntés, az előző ADR **Elavult** vagy **Felülvizsgált** állapotba kerül, és mutatnia kell az új ADR-re. A **Visszavont**

állapot azt jelzi, hogy a döntést még az elfogadás előtt ejtették, vagy később úgy döntöttek, hogy nem alkalmazzák.

Az állapotátmenetek tipikus útja:

Állapot	Jelentés	Tipikus átmenet
Javasolt	A döntést felvetették, még nincs elfogadva	Elfogadott, Visszavont
Elfogadott	A döntés érvényben van	Felülvizsgált, Elavult
Felülvizsgált	Új ADR módosítja, de az eredeti döntés nyomon követhető marad	Elfogadott, Elavult
Elavult	A döntést új döntés váltotta fel, az ADR megmarad referencia céljából	nincs tovább
Visszavont	A döntést nem fogadták el, vagy érvénytelenítették	nincs tovább

A verziózásnál két szintet kell megkülönböztetni. Az ADR maga rendelkezik egy **azonosítóval** és egy **tartalmi revízióval**. Az azonosító általában sorszámozott, például `ADR-001`, `ADR-002`. A tartalmi revízió a dokumentum szövegének módosítására vonatkozik. Ha a döntés tartalma változik, például egy alternatívát tévesen írtunk le, és ezt javítjuk, akkor a revíziószám nő, de az ADR-azonosító változatlan marad. A döntési napló ilyenkor is mutathatja a revíziót, például `ADR-007 rev. 2`.

A gyakorlatban az ADR-eket gyakran egyetlen `docs/adr/` mappában tárolják, fájlnévenként az azonosítóval és a döntés rövid leírásával. A revíziót a verziókezelő rendszer biztosítja. Ez a megközelítés összhangban van a könyv diagrams-as-code elvével: a döntések ugyanabban a kódbázisban élnek, mint a rendszer, így a verziókezelés automatikusan megőrzi a változtatások történetét.

Példa a fájlstruktúrára:

```
docs/
  adr/
    README.md
    0001-postgresql-valasztasa-irasi-tranzakciokhoz.md
    0002-kafka-hasznalata-integracionesemenyekhez.md
    0003-rest-helyett-grpc-belsokommunikaciohoz.md
```

A `README.md` a döntési napló indexe: táblázatos formában felsorolja az ADR-eket, azok állapotát és az utolsó módosítás dátumát. Ez a gyakorlatban a lekérdezhetőséget és a gyors áttekintést szolgálja.

4.3.5 Mikor kell új ADR-t készíteni?

Nem minden apró döntéshez kell ADR. A könyv gyakorlata szerint ADR-t akkor érdemes készíteni, ha a döntés valamelyik alábbi feltételt teljesíti:

- befolyásolja a rendszer határait vagy a fő komponensek felelősségi köreit;
- nem visszafordítható, vagy a visszafordítás költsége magas;
- több, egymással versengő erő között kell választani;
- a döntés szervezeti hatással jár, például csapatok közötti elosztást érint;

- a döntést később valószínűleg megkérdőjelezik, és a kontextus nem lesz kézenfekvő.

Ezzel szemben a lokális, könnyen visszafordítható technikai döntéshez, például egy segédfüggvény elnevezéséhez, nem kell ADR. A cél nem a dokumentáció túltermelése, hanem a visszakereshetőség.

4.3.6 Döntési napló

A döntési napló az ADR-ek gyűjteménye és indexe. Nem csupán tároló, hanem navigációs réteg is: lehetővé teszi, hogy egy új csapattag vagy egy felülvizsgáló gyorsan átlássa, mely döntések érvényesek, melyek elavultak, és melyek függnek egymástól.

A döntési napló minimális tartalma:

- minden ADR azonosítója és címe;
- az aktuális állapot;
- a döntés dátuma és a legutóbbi felülvizsgálat dátuma;
- kapcsolódó ADR-ek;
- egy rövid, egy-két mondatos összefoglaló a döntés lényegéről.

Példa egy döntési napló részletére:

Azonosító	Cím	Állapot	Dátum	Kapcsolódik
ADR-001	PostgreSQL írási tranzakciókhoz	Elfogadott	2025-02-10	ADR-002
ADR-002	Kafka az integrációs eseményekhez	Elfogadott	2025-02-11	ADR-001
ADR-003	gRPC a belső szolgáltatáskommunikációhoz	Felülvizsgált	2025-03-01	ADR-004
ADR-004	Belső szolgáltatáskommunikáció HTTP/2 alapú REST helyett	Elfogadott	2025-03-05	ADR-003

A táblázatban a `Felülvizsgált` állapotú ADR-003 mellett az ADR-004 mutatja a felülvizsgálat eredményét. Ez a kapcsolat teszi lehetővé, hogy az olvasó ne csak azt lássa, mi a jelenlegi döntés, hanem azt is, hogyan jutottunk el hozzá.

A döntési napló karbantartása akkor működik, ha automatizmusok is támogatják. A fájlalapú ADR-eknél a `README.md` frissítése gyakran kézi lépés, de egyszerű parancsfájl is ellenőrizheti, hogy minden ADR szerepel-e az indexben, és az állapotok érvényesek-e. Ez a CI-folyamat része is lehet, de nem feltétel. A lényeg a következetesség.

4.3.7 Kontextus, döntés, következmények és alternatívák

A négy alapmező közül mindegyiknek megvan a maga funkciója, és a tipikus hibák pontosan abból származnak, ha valamelyiket összekeverjük.

4.3.8 Kontextus

A kontextus rögzíti a probléma terét. Ide tartoznak a funkcionális és nem funkcionális követelmények, a meglévő rendszer korlátai, a csapat képességei, a határidők, a költségkeretek és a szabályozási előírások. A minőségi forgatókönyvek itt természetes módon jelennek meg. Például: „A rendelkezésre állásra vonatkozó SLO 99,9%, ami évente legfeljebb 8,76 óra állásidőt enged meg a mért ablakban. Ez kizárja az olyan adatbázis-megoldást, amelynél a tervezett karbantartás heti 2 óra leállást igényel."

A kontextusban a mérőszámok mértékegységgel és mérési ablakkal szerepeljenek. A döntés így nem válik el a mért valóságtól. Ha a kontextus hiányos, az olvasó nem tudja megítélni, hogy a döntés akkor is helyes volt-e.

4.3.9 Döntés

A döntés a választott irány pontos leírása. Fontos, hogy ne csak a technológia nevét rögzítsük, hanem azt is, hogyan használjuk. Például: „PostgreSQL-t választunk elsődleges adatbázisként. Az írási tranzakciókhoz REPEATABLE READ izolációs szintet használunk, és minden táblához előírjuk a központi UUID azonosítót a horizontális skálázhatóság érdekében." Ez a pontosság megakadályozza, hogy az ADR üres általánosság maradjon.

Ha a döntés a „nem döntünk most", akkor is érdemes ADR-t írni, mert ezzel rögzítjük, hogy a halasztás tudatos volt, és milyen feltétel váltja ki a későbbi döntést. Ez különösen hasznos azokban az esetekben, amikor a csapat bizonytalan egy technológia érettségében, de a határidő nem teszi lehetővé a további kutatást.

4.3.10 Alternatívák

Az alternatívák felsorolása bizonyítja, hogy a döntés nem volt reflexszerű. Legalább kettő, de inkább három reális lehetőséget érdemes leírni. Mindegyiknél röviden ismertetni kell az előnyöket és a hátrányokat, lehetőleg a kontextusban szereplő erőkkel összevetve. Az alternatívák nem „győztes-vesztes" listák, hanem a mérlegelés lenyomatai. Ha a csapat egy alternatívát azért vetett el, mert a csapat nem rendelkezik a szükséges tapasztalattal, ezt érdemes rögzíteni.

4.3.11 Következmények

A következmények szakasz a döntés várható hatásait írja le, egyértelműen elkülönítve a pozitív és negatív hatásokat. A negatív hatások közé tartoznak a kockázatok, az új üzemeltetési terhek, a betanulási költségek és az elszalasztott lehetőségek. A kompenzáló intézkedések azt mutatják, mit teszünk a negatív hatások mérséklésére. Például ha a választott technológia kevésbé ismert a csapatban, a következmény: „képzési napot és külső mentori támogatást iktatunk be".

A következményeket nem szabad utólag szépíteni. Az ADR akkor hasznos, ha a hibás előfeltevéseket is rögzíti, mert később ebből tanulhat a csapat. A „nem várt következmény" későbbi felülvizsgálattal kiegészíthető.

4.3.12 ADR és a kóddokumentáció kapcsolata

Az ADR nem helyettesíti a kóddokumentációt, és a kóddokumentáció sem helyettesíti az ADR-t. A kód megmutatja a pillanatnyi igazságot, az ADR a döntés pályáját. A kettő közötti kapcsolatot három ponton érdemes kialakítani:

1. **Hivatkozás a kódból az ADR-re.** A kommentben vagy a csomagleíróban elhelyezett `ADR-007` hivatkozás segít megtalálni a döntés indokát. Kerüljük a redundáns magyarázatot a kódban: a kódban a *hogyan*, az ADR-ben a *miért* él.
2. **Hivatkozás az ADR-ből a kódra.** Ha a döntés egy konkrét modult vagy konfigurációt érint, az ADR-ben megadhatjuk a releváns elérési utat. Például: `src/checkout/transaction/saga-flow.md`. Ez a link akkor hasznos, ha a kód áthelyezésekor a kapcsolat is frissül.
3. **A döntés teszteszközökkel való ellenőrzése.** A minőségi attribútumokból származtatott fitness function-ök a kód szintjén ellenőrzik, hogy a döntés továbbra is érvényesül-e. Például ha az ADR kimondja, hogy a belső szolgáltatások közötti kommunikáció nem használhat közvetlen adatbázis-hozzáférést, akkor egy statikus elemzés ezt a szabályt a CI-folyamatban ellenőrzi. Az ADR így nem válik holt dokumentummá.

A dokumentációs szerződésünk értelmében az architektúra-dokumentum verziózott, és a döntésekhez visszakereshető hivatkozás tartozik. Az ADR-ek a kódbázisban, verziókezelő alatt tárolva ennek a szerződésnek az egyik legtermészetesebb megvalósításai.

4.3.13 Tipikus hibák és ellenminták

Az ADR-írás gyakorlati hibái közül néhányat érdemes kiemelni.

4.3.14 1. A kontextus hiánya vagy túlzott általánossága

Hiba: „A rendszernek gyorsnak kell lennie, ezért választottunk gyors technológiát.” Ez nem kontextus, hanem üres állítás. A jó kontextus mérhető és dátumhoz kötött: „a betöltési teszt 2025. január 15-én azt mutatta, hogy a jelenlegi REST hívás p95 késleltetése 340 ms, miközben a cél 150 ms.”

4.3.15 2. Az alternatívák mellőzése

Gyakran az ADR csak a választott megoldást írja le, a vizsgált lehetőségeket nem. Ez azt a látszatot kelti, hogy a döntés egyetlen út volt. A hiányzó alternatívák miatt a későbbi felülvizsgálat nem tudja megítélni, mit áldoztunk fel.

4.3.16 3. Állapot nélküli ADR-ek

Ha az ADR nem tartalmaz állapotot, nem tudjuk, hogy a döntés érvényben van-e. A döntési naplóban az állapot mező kötelező. Az állapot hiánya a dokumentáció megbízhatóságát csökkenti.

4.3.17 4. Túl hosszú ADR-ek

Az ADR nem tervezési dokumentum és nem specifikáció. A cél a döntés rögzítése, nem a teljes rendszer leírása. A jó ADR terjedelme általában fél oldaltól két oldalig terjed. A hosszabb magyarázat kerüljön a kapcsolódó tervezési dokumentumba, az ADR csak hivatkozzon rá.

4.3.18 5. Az ADR-ek frissítésének elmaradása

Amikor egy döntést felülír egy újabb döntés, a régi ADR-nek nem kell eltűnnie, de az állapotát frissíteni kell. Ha a régi ADR „Elfogadott" marad, a napló hazudik. A felülvizsgálatot új ADR-ben rögzítjük, és a régit „Felülvizsgált" vagy „Elavult" állapotra állítjuk, hivatkozással az új döntésre.

4.3.19 6. Az ADR-eket csak a rendszer elején írják

A döntések nem állnak meg a projekt első hetében. A folyamatos architektúra elve szerint minden fontos döntéshez tartozhat ADR, akár a rendszer életének késői szakaszában is. A döntési napló így a rendszer teljes élettörténetét mutatja.

4.3.20 Labor: három ADR megírása egy gyakorló rendszerhez

A labor célja, hogy a résztvevő a fejezetben tanult szerkezetet és életciklust alkalmazza egy kis, ellenőrizhető példán. A labor nem igényel futtatható kódot, csak dokumentumot vagy jegyzetfüzetet.

4.3.21 Kiinduló állapot

Adott egy kitalált, „Rendeléskezelő mikroszolgáltatás-platform" elnevezésű gyakorló rendszer. A rendszer három szolgáltatásból áll: `rendeles`, `szamla`, `keszlet`. A csapat jelenleg egy közös PostgreSQL adatbázist használ, és a szolgáltatások REST-en keresztül kommunikálnak. A terhelés növekedése miatt a csapat a szolgáltatások szétválasztását tervezi. A rendszerre vonatkozó, fiktív mérési adatok:

- Az átlagos rendelési forgalom 120 tranzakció/másodperc, csúcsidőben 360 tranzakció/másodperc.
- A jelenlegi p95 olvasási késleltetés 180 ms, a cél 120 ms.
- A `szamla` szolgáltatásnál a havi átlagos rendelkezésre állás 99,5%, a cél 99,9%.

A labor kiinduló állapota tehát egy fiktív, de mérhető kontextus, amelyre a három ADR-t meg kell írni.

4.3.22 Feladatok

A résztvevő a következő három döntéshez készít ADR-t:

1. **Aszinkron üzenetkezelés bevezetése a `rendeles` és a `szamla` szolgáltatás között.** A döntés alternatívái: REST hívás, üzenetsor, közös adatbázis.
2. **Az adatbázis-szétválasztás módja.** A döntés alternatívái: egy közös PostgreSQL séma, szolgáltatásonkénti séma, szolgáltatásonkénti adatbázis-példány.
3. **A belső kommunikáció protokollja a `rendeles` és a `keszlet` szolgáltatás között.** A döntés alternatívái: REST, gRPC, GraphQL.

Mindhárom ADR-hez meg kell adni a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Az első két ADR-t „Elfogadott” állapotban kell rögzíteni, a harmadiknál a résztvevőnek először „Javasolt” állapotot kell adnia, majd a labor végén az ellenőrzési szempontok alapján döntést hozni: „Elfogadott” vagy „Visszavont”.

4.3.23 Elvárt eredmény

Három Markdown fájl a `docs/adr/` mappában, valamint egy `README.md` döntési napló. A döntési naplóban mindhárom ADR szerepeljen azonosítóval, címmel, állapottal, dátummal és kapcsolódó ADR-hivatkozásokkal. A fájlok verziókezelő rendszerben legyenek, így a revízió visszakövethető.

4.3.24 Ellenőrzési szempontok

A labor eredménye akkor megfelelő, ha:

- mindhárom ADR tartalmazza a kötelező mezőket;
- a kontextusokban szerepelnek a fiktív mérési adatok mértékegységgel és mérési ablakkal;
- minden döntéshez legalább két reális alternatívát rögzítettek;
- a következmények között szerepel legalább egy negatív hatás és egy kompenzáló intézkedés;
- a döntési napló indexe frissített, és az állapotok konzisztensek;
- az ADR-ek terjedelme nem haladja meg a két oldalt.

4.3.25 Visszaállítási út

A labor dokumentumfájlokat hoz létre. A visszaállítás a verziókezelő rendszerben a labor előtti commit azonosítóra való visszatéréssel lehetséges. Ha a résztvevő a labor során a döntési naplót is módosította, a visszaállítás ezt is helyreállítja. A labor nem igényel adatbázis-műveletet vagy éles rendszerhez való hozzáférést.

4.3.26 Gyakorlat és önellenőrző kérdések

A következő gyakorlat segít elmélyíteni az ADR-írás készségét.

Gyakorlat. Vegyünk egy kitalált döntést: a csapat egy meglévő monolitikus rendszer egy modulját külön szolgáltatásba szeretné kiszervezni. Írjuk meg az ADR kontextus szakaszát úgy, hogy az a korábbi Quality attributes és trade-off analízis alfejezetben tanult quality attribute scenario-kra épüljön. Adjunk meg legalább két ingert, környezetet, választ és mérőszámot.

Önellenőrző kérdések

1. Melyik ADR-mező rögzíti, hogy miért volt szükség a döntésre, és milyen erők hatottak?
2. Mi a különbség a „Felülvizsgált” és az „Elavult” állapot között?
3. Miért fontos a döntési naplóban a kapcsolódó ADR-ek oszlop?
4. Mi a tipikus hiba, ha az ADR-ből hiányoznak az alternatívák?

4.3.27 Összefoglalás

Az ADR a döntés pillanatának tömör, visszakereshető lenyomata. Nem a tökéletes választást garantálja, hanem azt, hogy a választás miértje ne vesszen el. A jól kialakított szerkezet – kontextus, döntés, alternatívák, következmények, állapot – biztosítja, hogy a jövőbeli olvasó ne csak a döntést lássa, hanem azt az utat is, amelyen a csapat a döntéshez eljutott. A verziózás és az életciklus figyelése pedig megakadályozza, hogy a döntési napló a múlt hamis tükrévé váljon.

A következő alfejezetben az architektúradöntések gazdasági és szervezeti szempontjait vizsgáljuk, építve az itt bevezetett ADR-fogalmakra.

4.4 Gazdasági és szervezeti döntési szempontok

Az architektúradöntések ritkán születnek úgy, hogy a mérnökök kizárólag a műszaki tisztaságra vagy egy diagram esztétikájára figyelnek. A döntéshozatal valós környezetben mindig négy, egymással versengő tényező együttes mérlegelése: **költség**, **képesség**, **kockázat** és **csapatkapacitás**. Az előző alfejezetben megismert Architecture Decision Record (ADR) megadja a formátumot, amellyel ezeket a szempontokat rögzíteni tudjuk; ebben a fejezetben azt tekintjük át, hogyan hozunk olyan döntést, amely e négy szempont együttes mérlegeléséből születik, és amelyet egy hónappal később sem bánunk meg.

Tegyük fel, hogy egy online kiskereskedelmi rendszerben be kell vezetnünk egy új termékajánló szolgáltatást. A csapat előtt három út áll: megvásárolni egy kereskedelmi ajánló motort, használni egy felhőalapú, felügyelt szolgáltatást, vagy saját, nyílt forráskódú komponensekre épülő rendszert fejleszteni. A három opció műszakilag egyaránt elfogadható; a döntés mégis sok szervezetben kínkeserves folyamat. Miért? Mert a mérnökök hajlamosak a képességre, a pénzügyesek a költségre, a jogászok a kockázatra, a szállítási szervezet pedig a csapatkapacitásra fókuszálni. Az érett architektúradöntés éppen az, amely e négy dimenziót egyetlen keretben kezeli, és a trade-offokat explicitté teszi.

4.4.1 A négy dimenzió bevezetése

Mielőtt a build versus buy döntésre rátérnénk, érdemes mind a négy dimenziót pontosan meghatározni, mert a későbbi labor és döntési mátrix erre épül.

Költség. A költség több, mint az első számla. A szoftverarchitektúra gazdaságtanában a teljes életciklus költséget (Total Cost of Ownership, TCO) használjuk: a beszerzés vagy kifejlesztés egyszeri költségét, a folyamatos licenc- vagy felhőköltséget, az üzemeltetési költséget, a migrációs és integrációs költséget, valamint a jövőbeni kivezetés vagy csere költségét is. A TCO jellegzetesen kevésbé látványos részei a legveszélyesebbek: a licenc vagy a felhasználás arányos díja, a dedikált üzemeltetői idő, a képzés, a support eszkaláció, a büntetések (például rendelkezésre állási SLA megszegése). A költségbecslés pontossága idővel romlik, ezért a költségprognózist mindig horizonttal és feltételezések listájával együtt kell dokumentálni.

Képesség. Képesség alatt a megoldás funkcionális és nem funkcionális illeszkedését értjük. A képesség mérésére nem elég azt mondani, hogy „tud ajánlani”; azt kell megvizsgálni, hogy a

termék képességei hogyan találkoznak a minőségi követelményekkel. Milyen az ajánlási pontosság? Milyen a késleltetés a p95-ös terhelésnél? Támogatja-e a törvényileg előírt adattörlési kéréseket? Milyen a modell újratanítási folyamata? A képesség hiánya gyakran nem a látható funkciók szintjén jelenik meg, hanem a minőségi attribútumokban: skálázás, auditálhatóság, adatvédelem, integrálhatóság és testreszabhatóság.

Kockázat. A kockázat a negatív kimenetel valószínűségének és hatásának szorzata. Az architektúrában három jellegzetes kockázati osztályt érdemes megkülönböztetni: szállítási kockázat (elkészül-e határidőre, a tervezett képességgel), üzemeltetési kockázat (működik-e a termelésben, elérhetően és biztonságosan), valamint opciós vagy evolúciós kockázat (képes-e a rendszer a jövőben követni a termékigényt). A kockázatértékelésben a valószínűséget és a hatást külön kell becsülni, még akkor is, ha a becslés szubjektív; a hibát itt nem a számok megadása jelenti, hanem az, ha a döntéshozók azt állítják, hogy „nincs kockázat".

Csapatkapacitás. A csapatkapacitás azt mutatja meg, hogy a szervezetben hány, milyen képességű ember, milyen időtartamban áll rendelkezésre a megoldás kifejlesztéséhez, integrálásához és üzemeltetéséhez. A kapacitás nem egyenlő a létszámmal: egy megoldás lehet költséghatékony, de ha bevezetéséhez hat hónapra egy olyan készségre van szükség, amelyből a piacon is hiány van, akkor a kapacitáshiány a döntés valódi korlátja. A csapatkapacitásba bele kell érteni a jelenlegi termékfeladatok melletti szabad kapacitást, a tanulási görbét és a kulcsember-függőséget is.

E négy dimenzió feszültségben áll egymással, és ez a feszültség az architektúradöntés természetes állapota. Egy nyílt forráskódú rendszer beszerzési költsége nulla lehet, de ha egy fő hónapokig csak az üzemeltetéssel foglalkozik, a TCO és a csapatkapacitás tekintetében drágább lehet, mint a felügyelt szolgáltatás. A felügyelt szolgáltatás képessége gyakran csak a gyártó által megszabott konfigurációs térre terjed ki, ami kockázatot jelent, ha a termékigény változik. A döntési keret értéke, hogy láthatóvá teszi ezt a feszültséget, és a döntést következményeivel együtt rögzíti.

4.4.2 Build versus buy: nem kétpólusú, hanem spektrum

A „build versus buy" klasszikus kerete ma már félrevezető, mert a két szélső pont közé beékelődött a felügyelt szolgáltatás, az open core termék, a SaaS, a kereskedelmi komponens és a saját, jól körülhatárolt modul. A döntés gyakorlatilag egy spektrumon történik:

1. **Tiszta buy:** a szervezet kész terméket, felügyelt SaaS-t vagy kereskedelmi komponenst használ, és nem módosítja a forrást vagy a belső működését.
2. **Konfigurációs buy:** a beszerzett terméket konfigurálják, de testre szabott kódot csak a gyártó által támogatott felületen keresztül adnak hozzá.
3. **Open source, belső üzemeltetéssel:** a forráskód nyílt, a szervezet maga futtatja és bizonyos esetekben bővíti a rendszert, de belső fejlesztői kapacitást köt le.
4. **Vásárolt keretre épített saját fejlesztés:** a szervezet megvásárol egy moduláris terméket vagy keretrendszert, és erre építi a differenciáló logikát.
5. **Tiszta build:** minden lényegi komponenst a szervezet tervez, fejleszt és üzemeltet.

Ennek a spektrumnak a két végpontja sem erkölcsi kategória. A differenciáló képességre gyakran érdemes építeni, a nem differenciáló, de bonyolult infrastrukturális képességre ritkán. A kérdés az, hogy mi a szervezet **differenciáló képessége**. Az online kiskereskedő esetében a fizetési kapu vagy a felhasználói hitelesítés nem differenciáló: a vásárló nem azért választja az áruházat, mert saját OAuth-szervere van. Az ajánlómotor viszont lehet differenciáló, ha a konverzió és a kosárérték növelése a stratégiai cél. A build versus buy döntés tehát nem általános igazságokból, hanem a konkrét differenciálódási térképből indul.

A döntésnek van egy további dimenziója is: a **teljes életciklus költsége**. A beszerzési ár félrevezető; a TCO számítása általában három szakaszra bontott:

- **Bevezetési költség:** a licenc vagy az első fejlesztés, az integráció, az adatmigráció, a tesztelés, a betöltés, a képzés és a szervezeti változáskezelés költsége.
- **Működtetési költség:** az éves licenc, a felhőszámla, az üzemeltetők ideje, a támogatási szerződés, a karbantartás, a frissítés és a teljesítmény-optimalizálás költsége.
- **Kivezetési költség:** a szerződés felmondása, az adatok exportálása, a migrálás egy új megoldásra, a szervezeti újratervezés és az esetleges párhuzamos üzem költsége.

A TCO-t tipikusan három-öt éves horizonton érdemes számolni, és minden számhoz feltételezést rendelni. Az alábbi, leegyszerűsített TCO-képlet alkalmas arra, hogy az összehasonlítás szerkezetét megadja:

$$\mathrm{TCO} = C_{\text{bevezetés}} + \sum_{i=1}^{N} \frac{C_{\text{működés}}(i)}{(1+r)^i} + \frac{C_{\text{kivezetés}}}{(1+r)^N}$$

ahol N az értékelési horizonthoz tartozó évek száma, r a diszkontráta (például a szervezet tőkeköltsége vagy egy egyszerű, 8–10 százalékos érték), $C_{\text{bevezetés}}$ az egyszeri költség, $C_{\text{működés}}(i)$ az i-edik év működtetési költsége, $C_{\text{kivezetés}}$ pedig a kivezetés költsége. Ennek a képletnek nem a számszerű pontossága a fő értéke, hanem az, hogy **felszínre hozza a költségek időbeni eloszlását**. Egy nyílt forráskódú megoldás bevezetési költsége alacsony, de a működtetési költség évekig tartó mérnöki időt jelenthet. Egy felügyelt szolgáltatás bevezetése gyors, de a kivezetés és az adatkiszolgáltatás költsége magas lehet. A képlet alkalmazásakor külön kell választani a **mért** és a **várt** értéket: a beszerzési ár szerződésből származik, az üzemeltetési költség előrejelzés; a kettőt csak akkor keverjük, ha a dokumentumban ezt expliciten jelezzük.

4.4.3 Példa: az ajánlómotor három opciójának elemzése

Térjünk vissza a bevezető példához. Az elemzést négy dimenzióban végezzük el; a számokat illusztrációként, nem valós piaci adatként kezeljük.

A opció: Tiszta, belső fejlesztés nyílt forráskódú keretre. A csapat ismeri a technológiát, a differenciáló képességet a saját adatbázis és a saját üzleti szabályok jelentik. Bevezetési költsége a legmagasabb: körülbelül hat mérnöki hónap, ami a szervezetben 45 millió forint teljes munkáltatói költséget jelent. Az éves működtetési költség alacsony, de nem nulla: havonta 40 mérnöki óra üzemeltetésre, frissítésre és hibaelhárításra, ami évente 7 millió forint. Kockázata mérsékelt: a szállítási kockázat magas, mert a belső roadmap könnyen csúszik; az üzemeltetési kockázat szintén magas, mert a p95-ös késleltetés optimalizálása nehéz; az evolúciós kockázat viszont alacsony, mert a forráskód felett teljes irányításunk van.

B opció: Felügyelt felhőszolgáltatás. Bevezetési költsége alacsony, körülbelül 4 mérnöki hét integrációs idő, 6 millió forint. Az éves szolgáltatási díj a kérésszám alapján 15 millió forint, és a növekedés miatt évente 10 százalékkal emelkedik. A képesség a standard felhasználásra jó, de a termékigény szerinti egyedi modellbetanítást csak a szolgáltató által ütemezett frissítésekben tudja fogadni; ez evolúciós kockázat. A kivezetési költség magas: az adatok exportálása és az egyedi integráció újraépítése 8 millió forintra becsülhető. Az üzemeltetési kockázat alacsony, mert a szolgáltató rendelkezésre állási SLA-t vállal, de a szolgáltató mint külső függőség önmagában is kockázati tényező, mert a szervezet nem tudja befolyásolni a roadmapet.

C opció: Kereskedelmi on-premise motor. A licenc 12 millió forint bevezetési költséggel jár, az éves karbantartás a licenc 20 százaléka, 2,4 millió forint, viszont a képesség közelebb

áll a B opcióhoz: a gyártó rendszeres frissítéseket kínál, de az egyedi differenciálódást a zárt rendszer korlátozza. Az üzemeltetési költség az A opcióhoz hasonló, mert a rendszert a saját környezetben kell futtatni. A szállítási kockázat alacsonyabb, mert a termék kész, a kivezetési kockázat viszont magas: a licencszerződés felmondása után a belső modell jogi és technikai értelemben is elveszhet.

A döntési mátrixban ezt a három opciót egymás mellé tesszük, és mindegyiket a négy dimenzióban értékeljük. A mátrixot nem azért készítjük, hogy az egyik opciót „győztessé" nyilvánítsuk, hanem hogy a döntéshozók lássák: mely dimenzióban melyik opció erős, és melyik gyengeségével nem tudunk együtt élni. Ha a szervezet stratégiai célja az ajánlómotor képességének éveken át tartó csiszolása, akkor az A opció evolúciós előnye felülmúlhatja a magas bevezetési költséget. Ha a szervezet célja a gyors piacra lépés, és a differenciálódás más funkciókban rejlik, akkor a B opció a kézenfekvő.

4.4.4 Kockázat és a döntés időbeni eleme

A kockázatot nem elegendő egyszeri pillanatban, a döntés napján értékelni. A döntés következményei az időben szétterülnek, és a kockázat profilja is változik. Egy belső fejlesztésnél a szállítási kockázat az első hónapokban koncentrálódik, majd az üzemeltetési kockázat válik uralkodóvá. Egy felügyelt szolgáltatásnál a szállítási kockázat alacsony, de a szolgáltató árazásának változása, a szolgáltatás leállítása vagy a képesség elmaradó frissítése évek után jelentkezik. A kockázatértékelésnek ezért nemcsak a jelenre, hanem a döntés várható élettartamára is vonatkoznia kell.

Három kockázati osztályt érdemes minden opció esetében külön dokumentálni:

- **Szállítási kockázat:** mekkora a valószínűsége, hogy a bevezetés csúszik, túlfut, vagy a képesség a tervezettnél kevesebb lesz? Milyen korábbi belső projektek támasztják alá a becslést?
- **Üzemeltetési kockázat:** milyen hibák, leállások vagy biztonsági események valószínűsíthetők? Van-e a szervezetben érett üzemeltetési gyakorlat és incidenskezelés?
- **Opciós kockázat:** mennyire tudjuk a megoldást a változó termékigényhez igazítani? A gyártó roadmap, a licencfeltételek és a belső forráskód feletti ellenőrzés mind ide tartozik.

A kockázatértékeléshez egyszerű, áttekinthető skálát használunk: a valószínűséget 1-től 5-ig, a hatást 1-től 5-ig értékeljük, a szorzatot pedig 1-től 25-ig terjedő kockázati pontszámként jegyezzük fel. A pontszám önmagában nem dönt; azt mutatja, hol érdemes további mérést vagy prototípust készíteni. A 20 feletti pontszámú tételeknél kötelező a csökkentő intézkedés; a 8 alattiakat csak nyomon követjük. A pontozás mindig szubjektív, ezért a dokumentumban a becslés alapját is rögzíteni kell: múltbeli adat, külső benchmark, szakértői becslés vagy hallomás.

4.4.5 Csapatkapacitás: a rejtett szűk keresztmetszet

A csapatkapacitás elemzésekor a leggyakoribb hiba az, hogy a döntéshozók az emberórák számában gondolkodnak, nem pedig a **szabad kapacitásban**. Egy szervezetnél az architektúradöntésnek nem az a kérdése, hogy mennyi munkába kerül a megoldás, hanem az, hogy a csapatnak a következő hónapokban van-e ennyi szabad kapacitása, és milyen más munkát késleltet vagy szorít ki.

A kapacitás mérése nem csak az emberórák összeszámolása. Három mutatót érdemes expliciten kezelni:

- **Rendelkezésre álló szabad kapacitás:** a csapat hány százalékát tudja a projektre allokálni a következő hónapokban? A 0 százalékos kapacitású, de „fontos” projekt nem létezik; a döntés ekkor arról szól, hogy melyik másik projektet állítjuk meg.
- **Képességbeli illeszkedés:** a csapat rendelkezik-e a megoldáshoz szükséges technológiai, üzemeltetési és domain-ismerettel? Ha nem, a tanulási görbe hány hónapot ad hozzá a bevezetési költséghez?
- **Kulcsember-függőség:** a tudás egyetlen emberben koncentrálódik-e? Ha ő kiesik, a szállítási kockázat mekkorára nő?

Példánkban az A opció hat mérnöki hónapos bevezetési költsége mást jelent, ha a csapat szabad kapacitása 20 százalékos, mint ha 80 százalékos. Az első esetben a hat hónapos munka valójában több mint két naptári évet vesz igénybe, ami alatt a piaci kép változik. A döntési mátrixba ezért a csapatkapacitást nem mérnöki hónapokban, hanem **naptári időben és szabad kapacitásban** kell feltüntetni. A B opció négy hetes integrációs ideje akkor is rövid marad, mert a felhőszolgáltató munkáját nem a belső csapat végzi; a döntés gyorsasága ebben az esetben a kapacitás meghatározó előnye.

A csapatkapacitás és a képesség kapcsolata is fontos: a szervezet nem vásárolhat meg egy megoldást, ha nincs belső képessége az integrálásához és a folyamatos felügyeletéhez. A felügyelt szolgáltatás is igényel belső tudást: a hitelesítés, a hálózati kapcsolat, a költségmonitorozás és a szolgáltatóval való kapcsolattartás nem virtuális feladat. A TCO-ba a külső szolgáltatás is beleszámítja a belső munkát; a hibás döntések gyakori oka éppen az, hogy ezt a „rejtett belső költséget” nem becsülik meg.

4.4.6 Szervezeti korlátok és döntési jogkörök

Az architektúradöntés soha nem légüres térben születik. A szervezetnek vannak formális és informális korlátai, amelyek a döntési teret előre megszabják. A formális korlátok közé tartoznak a beszerzési szabályzatok, az engedélyezett technológiák listája, a beszállítói kockázatkezelés, az adatvédelmi megállapodások és a belső kontrollok. Az informális korlátok közé tartoznak a szervezeti kultúra, a korábbi rossz tapasztalatok, a csoportok közötti bizalom és a vezetői preferenciák.

A döntési jogköröket érdemes formálisan is rögzíteni, hogy a döntésnek ne csak tartalma, hanem címzettje is legyen. Az ADR-ebben ezért a „Kontextus” szakaszba nemcsak a műszaki kontextus tartozik, hanem a szervezeti is: ki kezdeményezte, ki dönt, ki konzultál, ki hajtja végre. Erre a célra egy egyszerű RACI-mátrix is alkalmas:

Tevékenység	Felelős (Responsible)	Jóváhagyó (Accountable)	Konzultált (Consulted)	Tájékoztatott (Informed)
Követelmények rögzítése	Architekt	CTO	Üzleti termékmenedzser, biztonsági tisztviselő	Fejlesztőcsapat
Opciók elemzése	Architekt	CTO	Pénzügy, jog, üzemeltetés	Érintett csapatok
Döntés	Architekt	CTO	Beszerzés, pénzügy	Termékmenedzser
Bevezetés	Fejlesztőcsapat	Architekt	Biztonság, üzemeltetés	CTO

Üzemeltetés	Üzemeltetési csapat	Architekt	Fejlesztőcsapat	Támogatás

A RACI-mátrix azt teszi láthatóvá, hogy a döntésben kié a végső felelősség, és kik azok, akiknek a véleményét a döntés előtt be kell szerezni. Ha a döntésnél a konzultált felek nem kaptak lehetőséget a véleményük kifejtésére, akkor a döntés formálisan érvényes lehet, de a bevezetés során ellenállásba ütközik. Az érett architektúradöntés tehát nemcsak a tartalmi érveket, hanem a szervezeti legitimációt is biztosítja.

A döntési jogköröket időben is el kell határolni. Vannak döntések, amelyek a csapat szintjén, és vannak, amelyek a CTO vagy az architektúratanács szintjén születnek. A jó gyakorlat az, hogy a döntési jogköröket az **opciók visszafordíthatóságához** kötjük. A Martin Fowler által is leírt elv szerint a könnyen visszafordítható döntéseket a csapat szintjén érdemes meghozni, a nehezen visszafordíthatókat magasabb szinten, nagyobb körültekintéssel. A build versus buy döntés jellegzetesen nehezen visszafordítható: egy beszerzett termékből való kivonulás, vagy egy belső fejlesztésből való átállás hónapokig tartó, költséges folyamat. Ezért az ilyen döntéseket a szervezet magasabb döntési szintjén, szélesebb körben érdemes meghozni.

A döntési jogkörök rögzítése nem bürokrácia, hanem a döntés minőségének biztosítéka. A felhatalmazás nélküli döntés kockázata, hogy a döntést később informálisan felülírják, és az ADR-ből „történelmi dokumentum” lesz, nem pedig valós iránymutatás. A túlzott centralizáció kockázata ezzel szemben az, hogy a döntési lánc lassú, és a csapat elidegenedik az architektúrától. A két véglet közötti egyensúly a döntési jogkörök folyamatos felülvizsgálatában rejlik: évente egyszer érdemes áttekinteni, hogy a jelenlegi jogkörök illeszkednek-e a csapatok méretéhez, felelősségéhez és az architektúra érettségéhez.

4.4.7 A TCO- és kockázati összehasonlítás dokumentálása

A fejezet laborjában egy TCO- és kockázati összehasonlítást kell készíteni. A dokumentumnak négy részből kell állnia:

1. **Kontextus:** a döntés címe, dátuma, a döntésben érintett szervezeti egység, a döntési horizont és a döntési jogkörök RACI-mátrixa.
2. **Opciók elemzése:** minden opciónál a képesség, a TCO becslése, a kockázatok és a csapatkapacitás igénye.
3. **Döntési napló:** a választott opció, az elutasított alternatívák, a döntés következményei, a feltételezések listája és a döntés állapota.
4. **Ellenőrzési szempontok:** milyen mérésekkel, milyen időközönként és milyen felelősséggel vizsgáljuk felül a döntést.

A TCO becslésénél szigorúan külön kell választani a mért és a becsült értékeket. A mért értékek forrása lehet a szerződés, a belső költség-ráta, a múltbeli üzemeltetési naplók vagy a felhőszolgáltató számlázási adatai. A becsült értékeket „várt” jelzővel kell ellátni, és hozzá kell rendelni a bizonytalanságot is. A bizonytalanságot egyszerű sávval fejezhetjük ki: például az éves üzemeltetési költség várhatóan 7 millió forint, 5 és 9 millió forint közötti intervallumban. A döntést akkor tudjuk jól indokolni, ha a TCO-közötti különbség nagyobb, mint a bizonytalansági sávok átfedése. Ha egy opció TCO-ja 12 millió forint 10–14 milliós sávban, egy másiké 13 millió 9–18 milliós sávban, akkor a különbség nem tekinthető bizonyítottnak, és a döntést nem szabad kizárólag a TCO-ra alapozni.

A kockázati összehasonlításnál a három kockázati osztályt (szállítási, üzemeltetési, opciós) minden opcióban pontozzuk, és a pontozás indoklását is rögzítjük. A 20 feletti pontszámú tételek esetén

kötelező csökkentő intézkedést tervezni. A csökkentő intézkedés lehet prototípus, PoC, szoftveres guardrail, beszállítói SLA-kiegészítés vagy a döntés elhalasztása további mérésig. A döntési naplóban azt is fel kell tüntetni, hogy milyen trigger feltétel indítja el a döntés újraértékelését: például ha az éves költség 20 százalékkal meghaladja a tervet, vagy ha a szolgáltató a szolgáltatási szintet több hónapon át nem teljesíti.

4.4.8 Labor: TCO- és kockázati összehasonlítás

A labor célja, hogy az olvasó saját, kis, ellenőrizhető példán alkalmazza a fejezet eszközeit. Kiinduló állapotként válasszon egy olyan döntést, amelynek három opciója van: egy belső fejlesztés, egy felügyelt szolgáltatás és egy kereskedelmi, de saját üzemeltetésű termék. A példa ne legyen nagyobb, mint egyetlen, jól körülhatárolt képesség: például hitelesítési szolgáltatás, képfeltöltő modul, ajánlómotor vagy üzenetküldő szolgáltatás.

Az első lépés a **képesség mátrixa**. Sorolja fel a tíz legfontosabb minőségi követelményt, például a p95-ös késleltetést, a rendelkezésre állást, az adattörlési képességet, a skálázhatóságot, az auditálhatóságot, a testreszabhatóságot, a CSV-exportot, a riasztási integrációt, a költségszabályozást és a migrációs útvonalat. Minden opciót pontozzon 1-től 5-ig, és a pontozást egy-egy mondatban indokolja. A pontozás célja nem a „győztes” kiválasztása, hanem annak megmutatása, hogy az egyes opciók milyen profilon erősek.

A második lépés a **TCO-becslés**. A írja fel a bevezetési, az éves működtetési és a kivezetési költséget hároméves horizonton. A mért értékek mellé írja a forrást, a becsült értékek mellé a bizonytalansági sávot. Számoljon diszkontált TCO-t is, például 8 százalékos rátával. A formulát használja, de a számítás menetét is mutassa meg, hogy az eredmény reprodukálható legyen.

A harmadik lépés a **kockázati pontozás**. A három kockázati osztályban minden opciót pontozzon 1-től 5-ig a valószínűség és a hatás skáláján, a szorzatot rögzítse, és a 15 feletti pontszámokhoz írjon csökkentő intézkedést. A csökkentő intézkedés legyen konkrét: ne „prototípust készítünk”, hanem „a B opció API-jának késleltetését három hétig, éles forgalmat szimulálva mérjük”.

A negyedik lépés a **döntési napló**. Az ADR formátumát követve írja le a választott opciót, az elutasított alternatívákat, a döntés következményeit és a feltételezéseket. A döntés állapota legyen „Javasolt” vagy „Elfogadott”, aszerint, hogy a dokumentumot ki hagyta jóvá. A feltételezések közé tartozzanak a piaci árak, a csapat szabad kapacitása és a kockázati becslések.

Az ötödik lépés az **ellenőrzési szempontok**. Határozzon meg két-három mérőszámot, amellyel a döntés bevezetését követően ellenőrizni tudja a feltételezéseket. Például: az éves tényleges üzemeltetési idő nem haladja meg a becsült sáv felső értékét; a szolgáltatás p95-ös késleltetése a bevezetést követő első hónapban a tervezett tartományban van; a szállítási csúszás nem haladja meg a becsült kockázati puffer 50 százalékát. Adjon meg visszaállítási utat is: mi történik, ha a feltételezések megdőlnek, és hogyan lehet a döntést visszafordítani vagy felülvizsgálni.

4.4.9 Tipikus hibák és a helyes megoldás indoklása

A fejezet zárásaként három tipikus hibát emelünk ki, amelyek a döntési gyakorlatban rendszeresen visszatérnek.

Az első hiba a **költségprognózis pontosságának túlértékelése**. A döntési dokumentum tele van tizedesjegyre kiszámolt TCO-val, holott a bemenő adatok zöme becslés. A helyes megoldás az, hogy a döntést nem a pontszerű TCO-ra, hanem a bizonytalansági sávra és a sávok közötti különbségre alapozzuk. Ha a sávok átfednek, a döntést más dimenzióban, például az evolúciós kockázatban kell megalapozni.

A második hiba a **csapatkapacitás figyelmen kívül hagyása**. A döntés műszaki és pénzügyi értelemben is megalapozott, de a bevezetéshez szükséges naptári idő meghaladja a szervezet tűréshatárát. A helyes megoldás az, hogy a csapatkapacitást már a döntés előkészítésekor, a szabad kapacitás és a készségbeli illeszkedés szintjén is vizsgáljuk. A kapacitás hiánya nem a döntés elutasításának automatikus oka, hanem egy olyan kockázat, amelyet csökkenteni vagy expliciten vállalni kell.

A harmadik hiba a **döntési jogkörök tisztázatlansága**. Az opciók elemzése alapos, az ADR jól formázott, de nem világos, ki hagyja jóvá a döntést, és kiket kell a döntés előtt konzultálni. A helyes megoldás az, hogy a döntési napló RACI-mátrixot is tartalmaz, és a dokumentumot a jóváhagyó szinten kell jóváhagyatni. A jóváhagyás nem formalitás; a döntés bevezetése során felmerülő ellenállást a jóváhagyott RACI-mátrix jelentősen csökkenti.

Összefoglalva: a gazdasági és szervezeti döntési szempontok nem a mérnöki munka riválisai, hanem annak kiterjesztései. A költség, a képesség, a kockázat és a csapatkapacitás együttes mérlegelése ugyanazt a fegyelmet követeli, mint a minőségi követelmények elementszése: feltételezéseket kell rögzíteni, a mért és a várt értéket el kell különíteni, a bizonytalanságot pedig nem szabad elrejteni. Aki ezt a fegyelmet alkalmazza, nemcsak jobb döntést hoz, hanem a döntést később is felülvizsgálhatóvá, taníthatóvá és a szervezet számára közös tudássá teszi. A következő fejezetben az architektúra életciklusban betöltött szerepét vizsgáljuk, ahol ezek a döntési keretek a folyamatos architektúra gyakorlatába illeszkednek.

4.4.10 Ellenőrzőlista

1. Megfogalmaztam-e mind a négy dimenziót: költség, képesség, kockázat, csapatkapacitás?
2. A TCO-becslésben elválasztottam-e a mért és a becsült értékeket?
3. A TCO-becsléshez tartozik-e horizont, diszkontráta és feltételezési lista?
4. A kockázatértékelésben szerepel-e a három kockázati osztály: szállítási, üzemeltetési, opciós?
5. A 20 feletti kockázati pontszámú tételekhez terveztem-e csökkentő intézkedést?
6. A csapatkapacitást naptári időben és szabad kapacitásban is kifejeztem-e?
7. A döntési napló tartalmazza-e a RACI-mátrixot vagy a döntési jogkörök megnevezését?
8. Az ellenőrzési szempontok definiálják-e a döntés felülvizsgálatának trigger feltételeit?
9. Megadtam-e a visszaállítási utat arra az esetre, ha a feltételezések megdőlnek?

4.4.11 Önellenőrző kérdés

Amikor egy felügyelt szolgáltatás TCO-ja alacsonyabbnak tűnik, mint a belső fejlesztésé, milyen rejtett belső költségeket és milyen opciós kockázatokat kell még a döntésbe bevonni?

4.4.12 Idézett és ajánlott gondolatok a döntési kerethez

- **Forrás:** Martin Fowler: „Build vs Buy” esszéje. *A döntési spektrum és a differenciáló képesség fogalma.* A hivatkozott gondolat: a döntés nem kétpólusú; a lényeg az, hogy a szervezet differenciáló képességeire építünk, a többit pedig megfontoltan szervezzük ki.
- **Forrás:** SEI: Architecture Tradeoff Analysis Method (ATAM) koncepció. *A kockázat, a trade-off pontok és az érzékenységi pontok dokumentálása.* A hivatkozott gondolat: az architektúradöntés értéke a trade-offok elemzésében és a kockázatok explicit kezelésében rejlik.

- **Forrás:** ISO/IEC 25010: minőségi modell. *A képességdimenzió minőségi attribútumainak rendszerezése.* A hivatkozott gondolat: a képesség nem csak funkcionális illeszkedés, hanem a minőségi attribútumok és a használat kontextusának együttese.

5 Architektúra az életciklusban

5.1 SDLC, vízesés, Agile és folyamatos architektúra

A szoftverfejlesztés nem egyetlen nagy, monolitikus folyamat, hanem olyan keretek között zajlik, amelyek meghatározzák, mikor és hogyan születnek a technikai döntések. Az életciklus-modell kiválasztása közvetlenül befolyásolja az architektúra kialakításának időzítését, részletességét és dokumentálását. Ebben a fejezetben a vízesés, az iteratív és a folyamatos megközelítéseket vizsgáljuk abból a szempontból, hogy milyen szerepet kap bennük az architektúra, majd feloldjuk a „nagy tervezés előre” (Big Design Up Front, BDUF) és az „éppen elégséges architektúra” (Just Enough Architecture, JEA) közötti feszültséget.

5.1.1 SDLC és az architektúra kapcsolata

A szoftverfejlesztési életciklus (Software Development Life Cycle, SDLC) az a keretrendszer, amely leírja, hogyan jut el egy szoftvertermék az ötlettől a bevezetésig és a karbantartásig. Az SDLC nem csupán egy lineáris folyamat – bár a klasszikus modellek gyakran annak mutatják –, hanem döntési pontok sorozata, amelyekben az architektúra-centrikus gondolkodásnak jelen kell lennie.

Az architektúra életciklusban elfoglalt helye azért kritikus, mert a hibás vagy elkésett döntések költsége a folyamat előrehaladtával nő. Egy korai fázisban meghozott rossz döntés – például a rossz adatbázis-választás vagy a helytelen szolgáltatáshatár – a későbbiekben drága refaktorálást vagy akár újraírást igényelhet. Másrészt viszont a túl korán, túl nagy részletességgel meghozott döntések is károsak lehetnek, ha a követelmények még nem stabilak. Az életciklus-modell éppen azt szabályozza, hogy a bizonytalanságot és a kockázatot hogyan osztjuk el az időben.

5.1.2 Vízesés modell: az architektúra mint előre lezárt fázis

A vízesés (waterfall) modell lineáris, szakaszos folyamatot ír le: követelményelemzés, tervezés, implementáció, tesztelés, telepítés és karbantartás. Minden fázis az előző befejezése után kezdődik, és a visszalépés formálisan nehézkes. Az architektúra ebben a modellben a tervezési fázisban kap domináns szerepet, és jellemzően a teljes rendszerre kiterjedő, részletes terv készül, mielőtt az első kódsor megszületne.

Ez a megközelítés előnyös akkor, ha a követelmények stabilak, a szabályozási környezet merev, és a rendszer nagy kockázatú komponensei jól megérthetők előre. Például egy beágyazott rendszer vagy egy pénzügyi elszámolórendszer esetében a biztonsági és megfelelőségi követelmények már a projekt elején rögzíthetők. A vízesés modellben az architektúra-dokumentáció gyakran terjedelmes, és a minőségi attribútumokat – rendelkezésre állás, teljesítmény, biztonság – előre, formálisan specifikálják.

A modell fő hátránya a késői visszajelzés. Ha a követelményekben tévedés van, vagy a piaci környezet megváltozik, a nagy előzetes tervezésből származó architektúra gyorsan elavulhat. Ráadásul a tervezési fázisban meghozott döntések mögött nincs működő kódból származó

bizonyíték, ezért a kockázatok nagy része a tesztelési fázisig rejtve marad. Az architektúra itt inkább egy dokumentum, mint egy élő, fejlődő entitás.

5.1.3 Iteratív és inkrementális modellek: a korai alapvonal és a fokozatos finomítás

Az iteratív modellek – mint a Rational Unified Process (RUP) vagy a spirális modell – a fejlesztést ismétlődő ciklusokra bontják. Minden iteráció tartalmaz elemzést, tervezést, implementációt és tesztelést, és a termék minden iteráció végén futtatható, bővülő rendszer. Az architektúra itt nem egyetlen fázisban születik meg, hanem korán létrejön egy úgynevezett architekturális alapvonal (architectural baseline), amely a legnagyobb kockázatú és legfontosabb döntéseket tartalmazza. A további iterációk ezt finomítják, miközben a visszajelzések alapján korrigálnak.

A RUP kifejezetten az architektúra-centrikus megközelítést hangsúlyozza: az első fázisokban a fő hangsúly az architektúra kialakításán és a főbb kockázatok csökkentésén van. A spirális modell minden ciklusban kockázatelemzést végez, és a döntéseket a kockázat mértékéhez köti. Ez a megközelítés lehetővé teszi, hogy a nagy bizonytalanságú területeken korán prototípusok vagy PoC-k készüljenek, míg a stabil részeken a tervezés későbbre halasztható.

Az iteratív modell kompromisszumot kínál a vízesés túlzott merevsége és a teljesen ad hoc fejlesztés között. Az architektúra itt már élő dolog: az alapvonalat dokumentálják, de a döntéseket folyamatosan felülvizsgálják. Az ADR-ek ebben a modellben természetes módon illeszkednek, mert minden iterációban születhetnek új döntések vagy módosulhatnak korábbiak.

5.1.4 Agile, Scrum, Kanban, Lean és a folyamatos architektúra

Az Agilis kiáltvány (Agile Manifesto) négy értéket fogalmaz meg: egyének és interakciók, működő szoftver, együttműködés az ügyféllel, és változásra való reagálás – mindez a folyamatok és eszközök, a dokumentáció, a szerződéses tárgyalás és a terv követése fölé helyezve. Ez gyakran félreértéshez vezet: sokan azt gondolják, hogy az Agile azt jelenti, nincs szükség architektúrára vagy dokumentációra. Valójában az értékek nem a dokumentáció elhagyását, hanem a megfelelő mértékű, a változást támogató dokumentáció megválasztását jelentik.

A Scrum keretrendszerben a fejlesztés időkeretes sprintekben történik. Minden sprint tervezéssel kezdődik, ahol a csapat kiválasztja a következő növekmény elemeit. Az architektúra szempontjából ez azt jelenti, hogy a sprinttervezésnél azonosítani kell azokat a feladatokat vagy kockázatokat, amelyek architekturális döntést igényelnek. Lehet, hogy egy adott felhasználói történet megvalósításához új szolgáltatáskomponenst kell bevezetni, vagy egy API-szerződést kell módosítani. Ezt a csapatnak a sprint során kell menedzselnie, és a döntést ADR-ben rögzíteni.

A Kanban folyamatos áramlású modell, ahol nincsenek időkeretes sprintek, hanem a munka a „kész” állapot felé halad. Az architektúra itt is folyamatos, de a döntések ütemezését a munkafolyamat határozza meg. A csapat akkor hoz architektúra-döntést, amikor a munkatétel eléri azt a pontot, ahol a döntés nélkül nem lehet tovább haladni. Ez erősíti a just-in-time gondolkodást.

A Lean szoftverfejlesztés alapelvei közül különösen kettő releváns az architektúrára: a késleltetett döntés (defer commitment) és a lehető legkésőbbi ponton történő döntés, valamint a pazarlás (waste) csökkentése. A késleltetett döntés azt jelenti, hogy egy architekturális választást akkor hozunk meg, amikor a lehető legtöbb információ áll rendelkezésre, de még nem késő ahhoz, hogy a változtatás költsége elfogadható maradjon. A pazarlás csökkentése pedig arra figyelmeztet, hogy a túlzott tervezés – amely soha nem válik kóddá – értéktelen dokumentációt termel.

A folyamatos architektúra (continuous architecture) fogalma ezeket az elveket egyesíti: az architektúra nem egy egyszeri, nagy erőfeszítés, hanem állandó tevékenység, amely a fejlesztési ciklus minden pontján jelen van. A döntések visszacsatolási hurkokon keresztül – például a produkciós megfigyelések, a load tesztek, a felhasználói visszajelzések – folyamatosan felülvizsgálódnak. A folyamatos architektúra nem a dokumentáció elhagyását jelenti, hanem annak élővé tételét: a diagrams-as-code és az ADR-ek a kódbázissal együtt verziózódnak, így a dokumentáció nem válik elavulttá.

5.1.5 Big Design Up Front versus Just Enough Architecture

A Big Design Up Front (BDUF) a vízesés modellhez szorosan kapcsolódó szemlélet: a rendszer teljes architektúráját előre, részletesen megtervezzük, mielőtt bármilyen implementáció elkezdődne. A BDUF előnye, hogy a nagy rendszerek esetén átfogó képet ad, segít a kockázatok korai azonosításában, és biztosítja a konzisztenciát a komponensek között. Hátránya, hogy rugalmatlan, drága lehet, és ha a feltételezések tévesek, a teljes terv értékét veszíti. Különösen problémás a BDUF, ha a követelmények nem stabilak, vagy ha a technológiai környezet gyorsan változik.

A Just Enough Architecture (JEA) ezzel szemben azt az elvet követi, hogy a döntéseket a kockázat és a bizonytalanság mértékéhez igazítva hozzuk meg. Csak annyi architekturális tervezést végzünk, amennyi ahhoz szükséges, hogy a következő lépés biztonságosan megtehető legyen. A JEA nem a tervezés elhanyagolása, hanem a tudatos halasztás: a könnyen visszafordítható döntéseket későbbre toljuk, míg a nehezen visszafordítható vagy nagy kockázatú döntéseket korán, de nem feltétlenül teljes részletességgel hozzuk meg.

A két megközelítés között nem az a kérdés, hogy melyik a jobb, hanem hogy melyik illik az adott kontextushoz. A következő táblázat összefoglalja a főbb különbségeket:

Szempont	Vízesés / BDUF	Iteratív / JEA
A tervezés időzítése	A projekt elején, nagy részletességgel	Folyamatosan, kockázatvezérelten
Visszajelzési ciklus	Hosszú, a tesztelésig nincs működő kód	Rövid, minden iterációban működő rendszer
Változáskezelés	Drága, formális	Rugalmas, a döntések felülvizsgálhatók
Dokumentáció	Előre rögzített, terjedelmes	ADR-ek, diagrams-as-code, élő
Kockázatkezelés	Korai, de feltételezéseken alapul	Folyamatos, a kód és a mérések alapján
Mikor előnyös	Stabil követelmények, merev szabályozás, nagy rendszer, ahol a hibák költsége magas	Változó követelmények, gyors piaci környezet, kisebb csapatok

A gyakorlatban a legtöbb szervezet a kettő keverékét használja: néhány alapvető döntést – például a rendszer fő moduljait, a kommunikációs stílust, a biztonsági határokat – korán meghoznak, míg a részleteket az iterációk során finomítják. Az ilyen hibrid megközelítés gyakran „walking skeleton” néven ismert: készül egy vékony, de működő gerinc, amelyen a fő döntések már látszanak, de a teljes funkcionalitás még hiányzik. Ez lehetővé teszi a korai kockázatcsökkentést anélkül, hogy a teljes rendszert előre meg kellene tervezni.

5.1.6 Az architektúra döntési pontjai a különböző modellekben

Az architektúra-döntések nem egyenletesen oszlanak el az életciklus során; koncentrálódnak azokra a pontokra, ahol a rendszer szerkezete, a technológiai választások vagy a külső interfészek kialakítása megtörténik. A következő diagram egy iteratív (Scrum-szerű) folyamatban mutatja az ilyen döntési pontokat:

```
flowchart LR
    A[Sprint tervezés] --> B{Van-e új architektúra-döntés?}
    B -- Igen --> C[ADR vázlat készítése]
    C --> D[Sprint implementáció]
    B -- Nem --> D
    D --> E[Review: visszajelzés a működő rendszerről]
    E --> F[Retrospektív: mi működött, mi nem?]
    F --> G{Architektúra elég a továbblépéshez?}
    G -- Nem --> H[Következő sprintben finomítás]
    G -- Igen --> I[Kiadás előtti ellenőrzés]
```

Jelmagyarázat: a döntési pontok a sprinttervezésnél jelennek meg, de a review és retrospektív is visszacsatolást ad. A folyamat iteratív; a döntések ADR-ben rögzülnek. **Feltétel:** a modell Scrum-ot feltételez, de más iteratív keretben hasonló.

A vízesés modellben a döntési pontok a tervezési fázis elején sűrűsödnek, és utána a változtatás formális változáskezelést igényel. A folyamatos modellben a döntési pontok eloszlanak, és a csapatnak fegyelmezettnek kell lennie, hogy a döntéseket ne ad hoc módon, hanem tudatosan hozza.

5.1.7 Gyakorlati minta: rendeléskezelő szolgáltatás életciklusa

Illusztrációként tekintsünk egy fiktív projektet: egy webáruház rendeléskezelő szolgáltatását, amelyet egy meglévő monolitikus alkalmazásból nyernek ki önálló szolgáltatásként. A csapat 6 fő, a szervezet éppen most áll át Scrum alapú fejlesztésre. A szolgáltatásnak magas rendelkezésre állással kell futnia, és a fizetési szolgáltató integrációja miatt a részletes követelmények egy része még változhat.

Az életciklus-modell kiválasztásakor a csapat a következő szempontokat mérlegeli:

- A követelmények nem teljesen stabilak, ezért a teljes BDUF kockázatos.
- A fizetési folyamat biztonsági és megfelelőségi szempontból érzékeny, ezért legalább a biztonsági architektúra alapjait korán rögzíteni kell.
- A csapat mérete lehetővé teszi a szoros együttműködést, de szükség van egy architektúra-felelősre, aki a technikai kockázatokat képviseli.

A döntés: iteratív-inkrementális modell, Scrum keretrendszerrel, kiegészítve a JEA elveivel. Az első két sprintben a csapat létrehoz egy walking skeleton-t, amely tartalmazza a rendelésfelvétel, a validálás és a raktárkészlet-ellenőrzés minimális, működő változatát. A fizetési integrációt szimulált szolgáltatással helyettesítik, hogy a biztonsági határokat már most tesztelhessék.

Az architektúradöntéseket ADR-ekben rögzítik. Az alábbi példa mutatja az első ilyen döntést:

```
ADR-014-01: A rendeléskezelő szolgáltatás életciklus-modellje
Állapot: Elfogadva
Kontextus:
  A csapat 6 fő, kéthetes sprintekben dolgozik. A szolgáltatás magas
  rendelkezésre állást igényel, de a fizetési szolgáltató integrációja miatt
  a részletes követelmények egy része még változhat. A szervezet korábban
  vízesésben dolgozott, most Agile átállás zajlik.
```

```
Döntés:
  Iteratív-inkrementális modellt választunk Scrum keretrendszerrel.
  Az architekturális alapvonalat az első két sprintben hozzuk létre
  walking skeleton formájában. A további részleteket késleltetjük.
Alternatívák:
  1. Teljes BDUF vízesésben: magas előzetes tervezés, de lassú visszajelzés.
  2. Tiszta JEA, minden sprintben újraértékelés: nagy rugalmasság, de
     kockázat a fizetési modulnál.
Következmények:
  Az architektúra-döntéseket folyamatosan dokumentáljuk ADR-ben.
  A minőségi attribútumokat quality attribute scenario formában rögzítjük.
  A csapatnak szüksége van egy architektúra-felelősre, aki a
  sprinttervezésnél figyel a technikai kockázatokra.
```

A minőségi attribútumok egy részét a csapat quality attribute scenario formában fogalmazza meg, például:

- **Inger:** egy felhasználó rendelést ad le.
- **Környezet:** normál üzem, csúcsidőszaki terhelés mellett.
- **Válasz:** a rendszer a rendelést legfeljebb 2 másodpercen belül visszaigazolja.
- **Mérőszám:** a válaszidő 95. percentilise < 2 s, 5 perces mérési ablakban mérve.

Ez a forgatókönyv nem csak a teljesítményt, hanem a rendelkezésre állást is segít értékelni, és a későbbi teszteknél alapként szolgál.

5.1.8 Labor: projekt életciklus és architektúradöntések összerendezése

A labor célja, hogy egy konkrét projekt példáján keresztül gyakoroljuk az életciklus-modell kiválasztását és az architektúradöntések elhelyezését az időben. A feladat egy reprodukálható dokumentum vagy jegyzetfüzet elkészítése, amelyben a döntések és a kontextus világosan el vannak különítve.

5.1.9 Kiinduló állapot

Adott egy fiktív projektleírás, például: „Belső fejlesztői platform (IDP) bevezetése egy 40 fős fejlesztői szervezetnél.” A platform célja: önkiszolgáló környezetek üzembe helyezése, CI/CD folyamatok egységesítése, megfigyelhetőségi eszközök biztosítása. A szervezet jelenleg kézi konfigurációt használ, a csapatok heterogének, a működési kockázat közepes. A felhasználói igények egy része még nem ismert, mert a csapatok most kezdik használni a konténerizációt.

A hallgató feladata, hogy az adott projekt kontextusában döntse el, milyen életciklus-modellt alkalmazna, és ehhez rendelje hozzá a szükséges architektúradöntéseket.

5.1.10 Elvárt eredmény

Egy dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza:

1. **Kontextus:** a projekt rövid leírása, a csapat mérete, a szervezeti háttér, a fő kockázatok.
2. **Életciklus-modell kiválasztása:** indoklás, hogy miért az adott modellt választotta (vízesés, iteratív, folyamatos vagy hibrid).
3. **Szakaszok és döntési pontok:** a kiválasztott modell fázisai vagy iterációi, és minden fázishoz hozzárendelve legalább egy architektúradöntés.

4. **ADR-ek:** legalább három ADR, amelyek a fő döntéseket rögzítik (pl. a platform konténer-orchestrátora, a CI/CD eszköz kiválasztása, a megfigyelhetőségi stack). Minden ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
5. **Minőségi forgatókönyvek:** legalább két quality attribute scenario, amelyek a rendszerrel szembeni elvárásokat rögzítik.
6. **Mért és várt eredmények:** külön szakaszban a várt értékek (pl. a platform rendelkezésre állása 99,9%) és a ténylegesen mért értékek (ha vannak; ha nincsenek, akkor ezt jelölni kell: „nem mért").

5.1.11 Ellenőrzési szempontok

A dokumentum akkor fogadható el, ha:

- A kiválasztott életciklus-modell indoklása összhangban van a projekt kontextusával.
- Minden fázishoz tartozik legalább egy architektúradöntés, és az ADR-ek tartalma teljes.
- A minőségi forgatókönyvek tartalmazzák az ingert, a környezetet, a választ és a mérőszámot.
- A mért és várt eredmények egyértelműen el vannak különítve.
- A dokumentum reprodukálható: egy másik személy a leírás alapján meg tudná ismételni a döntési folyamatot.

5.1.12 Visszaállítási út

Mivel a labor dokumentációs jellegű, nincs destruktív művelet. Ha a hallgató verziókezelőt használ, a munkakönyvtár visszaállítható egy korábbi commithoz. A labor során nem kell éles rendszert módosítani; minden példa fiktív.

5.1.13 Tipikus hibák és megoldások

A gyakorlatban gyakran előfordulnak az alábbi hibák az életciklus és az architektúra kapcsolatának kezelésében:

1. **A BDUF és a JEA szélsőséges értelmezése.** Egyes csapatok a BDUF-ot látják az egyetlen szakszerű megoldásnak, és minden részletet előre megterveznek, míg mások a JEA nevében teljesen elhanyagolják a tervezést, és ad hoc döntéseket hoznak. A helyes megközelítés a kockázatvezérelt arány: a nagy, nehezen visszafordítható döntéseket korán, a kisebbeket később hozzuk.

2. **Az architektúra kihagyása az Agile-ból.** A „nincs idő tervezni" hozzáállás rövid távon gyorsnak tűnik, de hosszú távon technikai adósságot halmoz fel. A megoldás az, hogy az architektúra-felelős aktívan részt vesz a sprinttervezésben, és a döntéseket ADR-ben rögzíti.

3. **A döntések nem rögzítése.** Ha egy fontos döntés csak a csapat fejében él, később viták forrása lehet. Minden jelentős döntést ADR-ben kell dokumentálni, függetlenül az életciklus-modelltől.

4. **A big bang release.** A vízesés modellből örökölt szokás, hogy a rendszer csak a végén válik elérhetővé. Az iteratív és folyamatos modellek lényege éppen az, hogy a működő részeredmények folyamatosan visszajelzést adnak. Az architektúrának ezt a fokozatos kibontakozást kell támogatnia.

5. **A minőségi attribútumok késői figyelembevétele.** Ha a rendelkezésre állás, a teljesítmény vagy a biztonság csak a tesztelési fázisban kerül elő, a javítás költsége jelentős. A minőségi forgatókönyveket már a tervezés korai szakaszában meg kell fogalmazni, és az architektúra döntéseit ezekhez kell kötni.

5.1.14 Ellenőrzőlista

A fejezet végén az alábbi ellenőrzőlista segít abban, hogy egy projekt életciklus-modelljének és architektúradöntéseinek összerendezése megfelelő legyen:

- ☐ Az életciklus-modell kiválasztása indokolt, és illeszkedik a projekt bizonytalanságához, méretéhez és szabályozási környezetéhez.
- ☐ A nagy kockázatú és nehezen visszafordítható döntések korán megszületnek.
- ☐ Az architektúra-döntések ADR-ben rögzülnek, és tartalmazzák a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
- ☐ A minőségi attribútumok quality attribute scenario formában vannak megfogalmazva.
- ☐ A döntések és a követelmények közötti kapcsolat visszakereshető.
- ☐ A dokumentáció diagrams-as-code formában verziózott.
- ☐ A mért és a várt eredmények egyértelműen elkülönülnek.

5.1.15 Önellenőrző kérdések

1. Miben különbözik a vízesés és az iteratív modell architektúrális kockázatkezelése?
2. Mikor választanál Big Design Up Front megközelítést a Just Enough Architecture helyett?
3. Hogyan jelenik meg a folyamatos architektúra elve egy Scrum sprint keretében?
4. Miért fontos, hogy az architektúradöntéseket ADR-ben rögzítsük, függetlenül az életciklus-modelltől?
5. Milyen következményekkel jár, ha egy csapat a JEA nevében elhanyagolja a tervezést?

Ezekre a kérdésekre adott válaszok elmélyítik a fejezet anyagát, és előkészítik a következő egységet, amely a Scrum, a Kanban és a Lean keretrendszerek architektúra-szerepét vizsgálja részletesebben. A fejezetben bevezetett fogalmak – különösen a folyamatos architektúra és a JEA – alapul szolgálnak a Scrum, Kanban, Lean és az architektúra szerepe egységhez, ahol az architektúra és az agilis gyakorlatok mindennapi együttműködésére összpontosítunk.

5.2 Scrum, Kanban, Lean és az architektúra szerepe

A fejlesztési módszertanok nem csupán a munka megszervezését írják elő, hanem meghatározzák, hogy a csapat mikor és hogyan hoz architektúrális döntéseket, hogyan kezeli a technikai bizonytalanságot, és milyen ütemben integrálja az architekturális változtatásokat. A Scrum, a Kanban és a Lean szemléletmód egyaránt az érték folyamatos szállítását és a visszacsatolási ciklusok rövidítését hangsúlyozza, de eltérő eszközökkel: a Scrum idődobozos sprintekkel, a Kanban húzásos rendszerrel és WIP-limitekkel, a Lean pedig a pazarlás megszüntetésével. Ebben az alfejezetben megvizsgáljuk, hogyan befolyásolják ezek a megközelítések az architektúra tervezését, döntéshozatalát és evolúcióját, majd egy gyakorlati példán keresztül alkalmazzuk őket.

5.2.1 1. Scrum és az architektúra: szerepek, ritmus és növekmények

A Scrum egy empirikus keret, amely komplex, adaptív problémák megoldására alkalmas. Három alapelve az átláthatóság, az ellenőrzés és az adaptáció. A keret három szerepet határoz meg: a Product Owner (PO), a Scrum Master (SM) és a fejlesztőcsapat (Developers). A munka sprintnek nevezett, legfeljebb egy hónapos idődobozokban történik, és minden sprint egy potenciálisan kiadható növekménnyel (Increment) zárul.

5.2.2 1.1. A Scrum eseményei és az architekturális döntések időzítése

A Scrum eseményei természetes ritmust adnak az architektúrális munkának. A sprint tervezése során a csapat kiválasztja a Product Backlog elemeit, és meghatározza a sprint célját. Itt kerül sor a kiválasztott elemekhez kapcsolódó architektúrális szempontok azonosítására. Ha például az a cél, hogy egy új szolgáltatást vezessenek be, a csapatnak már a sprint tervezésekor el kell döntenie, hogy a funkciót a meglévő monolitba integrálja-e, vagy külön szolgáltatásként valósítja meg. Ennek a döntésnek legalább vázlatos formában meg kell születnie, különben a sprint során a munka akadályokba ütközik.

A Daily Scrum a napi szinkronizáció alkalma, ahol az architekturális blokkolók gyorsan felszínre kerülnek. Ha egy csapattag olyan döntést hozott, amely eltér a korábbi megállapodástól, itt kiderülhet, és a csapat még időben korrigálhat. A Sprint Review során a növekmény bemutatása nemcsak a funkcionalitást, hanem a technikai minőséget is értékeli – a stakeholderek visszajelzése alapján a csapat módosíthatja a technikai irányt. A Sprint Retrospective pedig a folyamat és a technikai gyakorlat vizsgálatára szolgál: itt kerülhetnek elő olyan architektúrális adósságok, amelyek a sprint során keletkeztek, és amelyeket a következő sprintben kell kezelni.

A Scrum tehát nem ír elő külön architektúrális fázist, hanem a döntéseket a sprint ritmusához köti. Az architektúra **inkrementálisan** alakul: minden sprint hozzáad valamit a rendszer struktúrájához, és a visszajelzések alapján korrigál. Ez azt jelenti, hogy a nagy, előre megtervezett architektúra (Big Design Up Front) helyett a rendszer fokozatosan, a sprintek mentén nyeri el végleges formáját. A módszer előnye, hogy a változó követelményekhez igazodik, és csökkenti a kockázatot; hátránya, hogy ha a csapat nem figyel, a rendszer technikai koherenciája megsérülhet.

5.2.3 1.2. A Product Backlog finomítása és a Definition of Done

A Product Backlog finomítása (backlog refinement) az a folyamat, amelynek során a csapat a nagyobb backlog elemeket kisebb, kezelhető darabokra bontja, és azonosítja a szükséges technikai lépéseket. Ekkor kerül sor az architekturális elemzésre: a csapat vázlatos diagramokat készít (például C4 komponens diagramot), megvitatja a technológiai alternatívákat, és meghatározza, hogy melyik sprintbe melyik rész kerül. A finomítás nem külön esemény, hanem folyamatos tevékenység, amely a sprint során bármikor megtörténhet.

A finomítás során fontos, hogy a technikai és a funkcionális elemeket együtt kezeljük. Egy felhasználói történet (user story) gyakran nem valósítható meg anélkül, hogy a mögöttes adatmodellt, az integrációs pontokat vagy a modulhatárokat ne módosítanánk. A csapatnak ezeket a technikai feladatokat is becsülnie kell, és be kell illesztenie a sprintbe.

A **Definition of Done** (DoD) kritikus eszköz az architektúra minőségének biztosításában. A DoD azoknak a feltételeknek a listája, amelyeknek minden növekménynek meg kell felelnie ahhoz, hogy késznek lehessen tekinteni. Egy architektúrálisan tudatos csapat a DoD-be olyan elemeket is felvesz, mint:

- „A módosítás nem sérti a modulhatárokat, és nem növeli a technikai adósságot."
- „Minden új függőséget a csapat értékelt, és a választást ADR-ben rögzítette."
- „Az interfész-szerződések dokumentáltak, és a tesztek lefedik őket."
- „A teljesítményre vonatkozó mérések a megadott küszöbérték alatt vannak."

Ezáltal a DoD biztosítja, hogy minden sprint végén a rendszer architektúrálisan is elfogadható állapotban legyen. Ha a csapat nem vesz fel ilyen feltételeket, a növekmény gyorsan felhalmozza a technikai adósságot.

5.2.4 1.3. Scrum szerepek és architektúra

A Scrum nem definiál külön „architect" szerepet. Ez szándékos: a keret a fejlesztőcsapat közös felelősségévé teszi a technikai döntéseket. A gyakorlatban azonban a csapatok gyakran kijelölnek egy vagy több tapasztalt tagot, aki az architektúrális konzisztenciáért felel. Nézzük meg a szerepek architektúrális vonatkozásait:

- **Product Owner**: A PO a termék értékét maximalizálja, és ő felel a Product Backlog sorrendjéért. Ez azt jelenti, hogy a PO-nak meg kell értenie a technikai kockázatokat és a technikai adósság hatását a termékre. Egy jó PO képes a technikai elemeket (például refaktorálás, adatbázis-migráció, frissítés) a felhasználói történetekkel együtt priorizálni. Ha a PO figyelmen kívül hagyja a technikai adósságot, a rendszer hosszú távon lelassul, és a szállítási képesség csökken. A PO-nak nem kell mély technikai tudással rendelkeznie, de meg kell bíznia a csapatban, és figyelembe kell vennie a javaslataikat.
- **Scrum Master**: A Scrum Master facilitátor, aki segíti a csapatot a keret betartásában, és elhárítja az akadályokat. Az architektúrális döntésekben nem feltétlenül vesz részt, de gondoskodnia kell arról, hogy a csapatnak legyen ideje és tere a szükséges technikai elemzésre. A Scrum Master nem projektmenedzser és nem dönti el a technikai irányt. Ha egy szervezetben a Scrum Master hozza a technikai döntéseket, az felborítja a csapat önszerveződését, és csökkenti a csapat elköteleződését.
- **Fejlesztőcsapat**: A csapat tagjai hozzák a technikai döntéseket. Ha nincs külön architekt, akkor a csapat kollektíven vállalja ezt a felelősséget. Ez magában foglalja a kódminőség, a modularitás, a technológiai választások és az integrációs minták kialakítását. A csapat mérete általában 3-9 fő, ami lehetővé teszi a hatékony megbeszéléseket. A gyakorlatban gyakori, hogy a csapatban van egy vezető fejlesztő vagy technikai vezető, aki az architektúrális koherenciát biztosítja, de a döntéseket a csapat közösen vitatja meg. Ez a modell ötvözi az irányítás és az önszerveződés előnyeit.

5.2.5 1.4. Spike: a bizonytalanság kezelése

Amikor a csapat egy architekturális döntés előtt áll, és nem tudja eldönteni, melyik alternatíva a jobb, idődobozos kutatást, úgynevezett **spike**-ot végezhet. A spike célja a tudás megszerzése, nem a termékfunkció szállítása. Például a csapat egy 2 napos spike-ot ütemez be, hogy kipróbálja két különböző üzenetküldő middleware teljesítményét és működési jellemzőit. A spike eredményét a sprint végén bemutatják, és ez alapján hozzák meg a döntést.

A spike fontos jellemzője, hogy időkorlátos, és nem szabad termékkódba direktben bekerülnie. A spike eredménye egy rövid dokumentum, egy ADR, vagy egy prototípus, amelyet a csapat értékel. A spike beilleszthető a sprintbe, de a Kanban táblán is lehet „spike" típusú feladat. A spike és a PoC hasonló, de a spike inkább egy konkrét kérdés megválaszolására irányul, míg a PoC szélesebb körű megvalósíthatóságot vizsgál.

5.2.6 2. Kanban WIP-limitek és Lean áramlás

A Kanban egy másik megközelítés: nem idődobozokban, hanem folyamatos áramlásban gondolkodik. A munka egy vizuális táblán jelenik meg, amely oszlopokra tagolja a munkafolyamatot (például „Backlog”, „Elemzés”, „Tervezés”, „Implementáció”, „Tesztelés”, „Kész”). Minden oszlophoz **WIP-limit** (work in progress limit) tartozik, amely a párhuzamosan folyó munkák maximális számát határozza meg.

5.2.7 2.1. A WIP-limit és az átfutási idő kapcsolata

A WIP-limit lényege, hogy a csapat ne vállaljon túl sok párhuzamos munkát. A Little-törvény szerint a rendszerben lévő munkák átlagos száma (L) egyenlő az érkezési ráta (λ) és az átlagos átfutási idő (W) szorzatával:

$$L = \lambda W$$

Ha a csapat kapacitása rögzített, akkor az érkezési ráta (azaz az elvégzett munkák aránya) nem tud növekedni pusztán attól, hogy több feladatot veszünk fel. Ha növeljük a folyamatban lévő munkák számát (L), az átfutási idő (W) szükségszerűen nő. A WIP-limit tehát közvetlenül az átfutási időt szabályozza: minél alacsonyabb a limit, annál gyorsabban haladnak át a feladatok a rendszeren, feltéve, hogy a csapat nem szenved hiányt elvégzendő munkából.

Egy egyszerű példa: ha a csapat hetente átlagosan 5 feladatot tud befejezni ($\lambda = 5$ feladat/hét), és a folyamatban lévő munkák száma 10, akkor az átfutási idő $W = L/\lambda = 10/5 = 2$ hét. Ha a WIP-limitet 10-ről 5-re csökkentjük, az átfutási idő 1 hétre csökken, feltételezve, hogy a kapacitás változatlan. Ez a számítás idealizált: a valóságban a feladatok mérete változik, és a rendszerben variabilitás van, de az alapelv érvényes. A WIP-limit csökkentése tehát gyorsabb visszajelzést és rövidebb ciklusidőt eredményez, ami az architektúrális evolúció szempontjából is kedvező, mert a változtatások hamarabb értékelhetők.

5.2.8 2.2. Lean áramlás és a pazarlás megszüntetése

A Lean szemlélet a Toyota Termelési Rendszerből származik, és alapelvei az érték meghatározása, az értékáram feltérképezése, a folyamatos áramlás megteremtése, a húzásos rendszer bevezetése és a tökéletesítés. A szoftverfejlesztésben a Lean gondolkodás a következőket jelenti:

- **Érték**: ami a vevő számára hasznos. Az architektúrális munka közvetve értéket teremt, mert lehetővé teszi a gyorsabb, megbízhatóbb szállítást. Például a moduláris felépítés csökkenti a változtatások kockázatát és idejét.
- **Pazarlás (muda)**: minden olyan tevékenység, amely nem ad értéket a vevőnek. Gyakori pazarlások a felesleges dokumentáció, a félkész kód, a várakozás, a hibajavítás, a többszörös átadás, és a szükségtelen bonyolultság. Az architektúra tervezésénél ügyelni kell arra, hogy ne váljon öncélú dokumentációs gyakorlattá: a dokumentáció csak annyi legyen, amennyi a döntés megértéséhez szükséges.
- **Húzásos rendszer**: a munka akkor kezdődik, amikor a következő lépés kapacitása felszabadul. Ez a Kanban WIP-limitjeinek alapja. A húzásos rendszer megakadályozza, hogy a csapat túlvállalja magát, és biztosítja, hogy a kapacitás a legfontosabb feladatokra összpontosuljon.

A Lean és az architektúra kapcsolata: a nagy, egyszerre bevezetett architekturális változtatások (big bang) sok pazarlást okoznak, mert hosszú ideig nincs érték a vevő számára, és magas a kockázat. A Lean ehelyett **kis lépésekben** történő evolúciót javasol: a rendszer architektúráját folyamatosan, apró, tesztelhető lépésekben alakítjuk. Ez összhangban van az evolúciós architektúra koncepciójával, és lehetővé teszi, hogy a rendszer mindig működőképes állapotban maradjon.

A Lean továbbá megkülönbözteti a pazarlás három típusát: **muda** (nem értékteremtő tevékenység), **mura** (egyenetlenség, például ingadozó terhelés), és **muri** (túlterhelés). Az architektúrális tervezésben a mura például azt jelenti, hogy a csapat néha túl sok architekturális döntést kénytelen egyszerre meghozni, máskor pedig egyáltalán nem foglalkozik vele. A Kanban WIP-limitek és a folyamatos finomítás segítenek kiegyensúlyozni ezt.

5.2.9 2.3. Kanban az architektúrális munkában

A Kanban tábla lehetővé teszi, hogy az architektúrális feladatok ugyanabban a folyamatban jelenjenek meg, mint a termékfunkciók. A táblán például lehet egy „Architektúrális elemzés" oszlop, ahol a technikai döntéseket előkészítik. A WIP-limit itt is érvényes: ha túl sok elemzés vár egyszerre, akkor az elemzések elhúzódnak, és a döntések késnek, ami blokkolja a további munkát.

A Kanban előnye, hogy folyamatosan kezeli a technikai adósságot: amint kapacitás szabadul fel, a csapat húzhat egy refaktorálási feladatot. Nincs szükség arra, hogy egy sprintet teljes egészében a technikai adósságnak szenteljenek (bár ez is előfordulhat). A Kanban azonban fegyelmet igényel: a csapatnak ellen kell állnia annak, hogy folyamatosan új termékfunkciókat húzzon, és elhanyagolja az architektúrális karbantartást. Ezt a feszültséget jól kezeli, ha a backlogban a technikai elemek is explicit módon szerepelnek, és a csapat közösen szabályozza a húzást.

5.2.10 3. Scrum és Kanban összehasonlítása architektúra szempontból

Az alábbi táblázat összefoglalja a két módszer főbb jellemzőit az architektúra szempontjából:

Szempont	Scrum	Kanban
Időzítés	Idődobozos (sprint)	Folyamatos
Döntéshozatal	Sprint tervezéskor, finomításkor	Amikor a feladat a táblára kerül
Architektúra evolúció	Inkrementális, sprintenkénti növekmény	Folyamatos áramlásban
Technikai adósság kezelése	Backlog elemként, priorizálva	Folyamatosan húzható
Bizonytalanság kezelése	Spike a sprintben	Spike típusú feladat a táblán
Szerep az architektúrának	A sprint cél és a DoD része	Minden feladat minőségi kritériuma

Mindkét módszer támogatja az evolúciós architektúrát, de eltérő ritmust ad. A Scrum jobban megfelel, ha a termékfejlesztés szakaszos, és a csapat előre tudja tervezni a sprintet. A Kanban jobb, ha a munka folyamatosan érkezik (például üzemeltetési hibák, változó igények), vagy ha a rendszer karbantartási fázisban van. Sok csapat hibrid megközelítést alkalmaz (Scrumban), például sprinteket tart, de a sprinten belül Kanban táblát használ a feladatok nyomon követésére.

A hibrid megoldás lehetővé teszi a tervezhetőség és a folyamatosság kombinálását, de ügyelni kell arra, hogy ne legyen túl bonyolult.

5.2.11 4. Gyakorlati példa: architektúrális feladatok tervezése

Vegyünk egy fiktív, de reális helyzetet. Az „Acme Books" online könyvesbolt egy monolitikus alkalmazással rendelkezik. A cég szeretné a raktárkészlet-kezelést kiszervezni egy külön szolgáltatásba, hogy a raktári csapat önállóan fejleszthesse, és a monolit ne akadályozza. A csapat 5 fejlesztőből és 1 technikai vezetőből áll, aki az architektúrális koherenciáért felel.

5.2.12 4.1. Scrum tervezés

A Product Backlogba bekerül egy epikus: „Raktárkészlet-szolgáltatás kivonása a monolitból". A finomítás során a csapat azonosítja a szükséges lépéseket:

1. A készletadat-modell meghatározása.
2. Az API tervezése (GET, POST /inventory).
3. Adatbázis-séma kialakítása.
4. Szinkronizációs mechanizmus a monolit és az új szolgáltatás között.
5. A monolit raktárkészlet-kezelésének kiiktatása.

A sprint tervezésekor a csapat a következő elemeket választja ki a 2 hetes sprintre:

- „Készlet-szolgáltatás API vázlata" (kód)
- „Spike: PostgreSQL vs. MongoDB a készletnyilvántartáshoz" (2 nap)
- „C4 komponens diagram a jelenlegi és a célarchitektúráról" (dokumentáció)
- „ADR: adatbázis-választás a készlet-szolgáltatáshoz"

A sprint cél: „A készlet-szolgáltatás architektúrális alapjai elkészülnek, és a csapat dönt az adatbázis-technológiáról." A Definition of Done tartalmazza: „Az ADR a megadott sablon szerint készült, és a csapat jóváhagyta." A sprint során a csapat a spike eredménye alapján dönt, és az ADR-t rögzíti a döntési naplóban. A sprint végén a Review-n bemutatják az API vázlatot és a diagramokat, a Retrospective-en pedig megbeszélik, hogy a spike elegendő volt-e, vagy további elemzés kell.

5.2.13 4.2. Kanban tervezés

Ugyanezt a feladatot Kanban táblán is megtervezhetjük. A tábla oszlopai: „Backlog", „Elemzés", „API tervezés", „Implementáció", „Teszt", „Kész". WIP-limitek: Elemzés = 2, API tervezés = 1, Implementáció = 2, Teszt = 1.

A Backlogban a feladatok:

- „Készlet-adatmodell meghatározása"
- „API specifikáció elkészítése"
- „Adatbázis-séma tervezése"
- „Spike: adatbázis-technológiák összehasonlítása"
- „Monolitból adatszinkronizáció megtervezése"

A húzásos rendszerben a csapat először az „Elemzés” oszlopba húzza a „Készlet-adatmodell meghatározása” feladatot, mert ez a legmagasabb prioritású. Amint kész, átkerül az „API tervezés” oszlopba. Mindeközben a WIP-limitek biztosítják, hogy egyszerre legfeljebb két elemzés és egy API tervezés folyik. Ez megakadályozza, hogy a csapat szétszóródjon, és az egyes feladatok gyorsabban elkészülnek.

A Kanban táblán a „Spike” feladat is megjelenik, és időtartama korlátozott. A döntési napló (ADR) a táblán kívül, külön dokumentumban készül, de a feladat definíciója tartalmazza, hogy az ADR-t el kell készíteni.

5.2.14 4.3. Döntési napló minta

Az alábbi egy egyszerűsített ADR a példához:

Cím: Készlet-szolgáltatás adatbázis-technológiája
Státusz: Javasolt
Kontextus: A raktárkészlet-szolgáltatás számára adatbázist kell választani. A követelmények: tranzakciós konzisztencia, magas olvasási terhelés, JSON-alapú adatmodell, a csapat tapasztalata SQL-ben.
Döntés: PostgreSQL használata JSONB oszlopokkal.
Alternatívák: MongoDB (jó JSON-kezelés, de a csapat kevésbé tapasztalt), MySQL (nincs natív JSONB).
Következmények: A csapat a meglévő SQL tudást használhatja; a JSONB lehetővé teszi a rugalmas adatsémát; a replikáció beállítása szükséges.
Megerősítés: A spike során a PostgreSQL a szükséges olvasási teljesítményt hozta 3 csomópontos replika esetén.

A sprint vagy a Kanban tábla tervezése során az ilyen ADR-ek biztosítják, hogy a döntések nyomon követhetők legyenek, és a jövőbeli olvasó megértse a döntés okát és a feladott alternatívákat.

5.2.15 5. Tipikus hibák és félreértések

1. **„Agilisban nincs szükség architektúrára”** – Ez téves. Az agilis módszertanok az architektúra folyamatos, inkrementális fejlesztését hangsúlyozzák, nem annak hiányát. Az architektúra nélkül a rendszer gyorsan átláthatatlanná válik, és a változtatások kockázata nő. A kérdés nem az, hogy kell-e architektúra, hanem hogy mikor és mekkora mértékben tervezzük.

2. **„A Scrum Master a technikai vezető”** – A Scrum Master facilitátor, nem dönt a technikai irányról. Ha a Scrum Master technikai döntéseket hoz, az felborítja a csapat önszerveződését, és csökkenti a csapat felelősségvállalását. A technikai vezető szerep a csapatban van, nem a Scrum Masterben.

3. **„A Product Ownernek nem kell értenie a technikai adóssághoz”** – A PO-nak legalább tisztában kell lennie a technikai adósság hatásával, mert ő priorizálja a backlogot. Ha a technikai adósságot figyelmen kívül hagyják, a szállítási sebesség csökken, és a termék minősége romlik. A PO-nak nem kell mély technikai tudás, de meg kell bíznia a csapatban, és be kell fogadnia a technikai javaslatokat.

4. **„A Kanban WIP-limit csak a fejlesztői munkára vonatkozik”** – A WIP-limitnek minden munkafolyamat szakaszra vonatkoznia kell, beleértve az elemzést és a tesztelést is, különben a szűk keresztmetszet máshol alakul ki. Ha az elemzésben nincs korlát, akkor az elemzések felhalmozódnak, és a fejlesztők várni fognak rájuk.

5. **„A Lean szerint minden dokumentáció pazarlás”** – A Lean a nem értékteremtő tevékenységet tekinti pazarlásnak. Az architektúrális döntések dokumentálása értéket teremt, mert lehetővé teszi a jövőbeli csapattagok számára a kontextus megértését. A cél a **megfelelő mértékű** dokumentáció, nem a teljes elhagyása. Egy jól megírt ADR nem pazarlás, hanem befektetés.

6. **„A spike eredménye azonnal termékbe kerülhet”** – A spike célja a tanulás, nem a termékfunkció. A spike kódját gyakran el kell dobni, és a valódi implementációt a tanulságok alapján kell elkészíteni. Ha a spike kódja a termék része lesz, az rossz minőségű, nem tesztelt kódot eredményezhet.

7. **„A Scrum csak termékfunkciókra való, a technikai munkát külön kell kezelni”** – Ez tévhit. A technikai munka ugyanúgy a Product Backlog része, és ugyanúgy priorizálható. A technikai adósság kezelése a termék hosszú távú fenntarthatóságát szolgálja.

5.2.16 6. Labor: architektúrális feladatok tervezése sprintben vagy kanbanban

Kiinduló állapot: Adott egy fiktív webes rendszer (például egy online fórum), amely monolitikus. A csapat szeretné bevezetni a felhasználói értesítések aszinkron kezelését, hogy a regisztrációs e-mailek ne blokkolják a webes válaszidőt. A jelenlegi állapotban a regisztráció során a kérés szinkron módon küld e-mailt, ami lassú válaszidőt okoz (átlagosan 1200 ms, mérési ablak: 7 nap). A csapat 4 fejlesztőből és 1 technikai vezetőből áll.

Feladat: Tervezze meg a következő 2 hét munkáját **vagy** Scrum sprintként, **vagy** Kanban táblaként. A tervnek tartalmaznia kell: - Az architektúrális feladatokat (például üzenetsor bevezetése, producer és consumer szolgáltatások, ADR a választott üzenetküldő middleware-ről). - A feladatok lebontását és sorrendjét. - Ha Kanban táblát készít, adja meg a WIP-limiteket és indokolja azokat. - Ha Scrum sprintet készít, határozza meg a sprint célt, a sprint backlogot és a Definition of Done architektúrális kritériumait. - Készítsen egy ADR-t a middleware választásáról (például RabbitMQ vs. Apache Kafka).

Elvárt eredmény: Egy dokumentum (Markdown vagy jegyzetfüzet), amely tartalmazza a fenti elemeket. A dokumentumnak reprodukálhatónak kell lennie: egy másik architektnek a dokumentum alapján ugyanazt a tervet kell tudnia értelmezni.

Ellenőrzési szempontok: - A sprint cél elérhető-e a 2 hét alatt? - A WIP-limitek betartásával elkerülhető-e a túlterhelés? - Az ADR tartalmazza-e a kontextust, döntést, alternatívákat, következményeket és állapotot? - A Definition of Done tartalmaz-e architektúrális kritériumot? - A terv figyelembe veszi-e a jelenlegi rendszer válaszidejének csökkentését?

Visszaállítási út: Mivel a labor dokumentációs jellegű, nincs szükség rendszer-visszaállításra. Ha a labor kódot is érint (például egy PoC-t), azt izolált fejlesztői környezetben kell elvégezni, és a környezetet a labor után le kell állítani.

Mérési terv: A labor során a csapatnak meg kell határoznia egy mérőszámot, amely a bevezetés hatását méri. Például: a regisztrációs kérés átlagos válaszideje a jelenlegi 1200 ms-ról (mérési ablak: 7 nap) 800 ms-ra csökken a bevezetés után. A mérési tervben meg kell adni a mérési ablakot, a környezetet és a bizonytalanságot. A mért és a várt eredményt külön kell választani, és a nem futtatott mérést nem szabad ellenőrzött eredményként bemutatni.

5.2.17 7. Ellenőrzőlista a fejezethez

- ☐ A Scrum sprintben az architektúrális feladatok a sprint backlog részei.
- ☐ A sprint cél tartalmazza az architektúrális célt is, ha releváns.
- ☐ A Definition of Done tartalmaz architektúrális minőségi kritériumokat.
- ☐ A Kanban táblán minden munkafolyamat szakaszhoz tartozik WIP-limit.
- ☐ A WIP-limit értékét a csapat kapacitása és a Little-törvény alapján indokolta.
- ☐ A Lean pazarlás elemzésnél megkülönböztettük az értékteremtő és a nem értékteremtő tevékenységeket.
- ☐ Minden architektúrális döntés ADR-ben van rögzítve, és az ADR tartalmazza a szükséges mezőket.
- ☐ A spike idődobozos, és az eredményét a csapat értékeli.
- ☐ A technikai adósság a termék backlogban explicit módon megjelenik.

5.2.18 Önellenőrző kérdés

Magyarázza el, miért vezet a Kanban WIP-limit növelése az átfutási idő növekedéséhez, ha a csapat kapacitása változatlan. Használja a Little-törvényt a válaszában.

5.2.19 Összefoglalás és továbbvezető gondolatok

Ebben az alfejezetben láttuk, hogy a Scrum, a Kanban és a Lean nem csupán folyamatmenedzsment-eszközök, hanem alapvetően befolyásolják az architektúra kialakításának és evolúciójának módját. A Scrum idődobozos, inkrementális megközelítést kínál, ahol a döntések a sprint ritmusához kötődnek, és a csapat közös felelőssége a technikai koherencia. A Kanban folyamatos áramlást biztosít, ahol a WIP-limitek a Little-törvényen keresztül az átfutási időt szabályozzák, és a Lean a pazarlások megszüntetésével a kis lépésekben történő evolúciót támogatja. A két módszer kombinálható, de a választás mindig a kontextustól függ: milyen a csapat mérete, a munka jellege, a bizonytalanság szintje és a szervezeti környezet.

A következő alfejezetben (Architektúra governance és döntési jogkörök) az architektúra governance és a döntési jogkörök témáját járjuk körbe, amelyre az itt bevezetett szerepek és folyamatok közvetlenül építenek. Ott azt vizsgáljuk, hogyan lehet a döntési jogköröket formálisan elosztani, és hogyan biztosítható a technikai irányítás anélkül, hogy a csapat önszerveződése sérülne.

5.3 Architektúra governance és döntési jogkörök

A szoftverarchitektúra nem csupán egy statikus terv, hanem döntések sorozata, amelyek a rendszer életciklusa során folyamatosan alakulnak. Az előző alfejezetben láttuk, hogyan illeszkedik az architektúra a különböző fejlesztési módszertanokhoz; most azt vizsgáljuk, hogyan szervezzük meg magát a döntéshozatalt, hogyan osztjuk el a döntési jogköröket, és milyen szerepet kapnak a szabványok, illetve az azoktól való eltérés kezelése. Ezt a területet nevezzük **architektúra governance**-nek, amely nem adminisztratív teher, hanem a csapatok közötti összehangolás, a technikai adósság megelőzése és a fenntartható evolúció eszköze.

5.3.1 A döntések elosztása és a döntési jogkörök

Egy szoftverrendszer fejlesztése során rengeteg döntés születik: milyen programozási nyelvet használjunk, hogyan nevezzünk el egy API-végpontot, milyen adatbázis-sémát válasszunk, mikor vezessünk be cache-t, hogyan kezeljük a hibákat. Ezek a döntések nem egyformán fontosak és nem ugyanazt a hatáskört igénylik. Ha minden apró döntést egyetlen személy vagy testület hozna meg, akkor a folyamat lassú és frusztráló lenne; ha viszont mindenki teljesen szabadon dönthetne, akkor káosz alakulna ki, és a rendszer inkonzisztenssé válna. A governance célja, hogy megtalálja az egyensúlyt a **sebesség** és a **konzisztencia** között.

A döntési jogkörök elosztásának első lépése a döntések kategorizálása. A gyakorlatban három szintet különböztetünk meg:

- **Stratégiai döntések**: hosszú távú, nehezen visszafordítható, a teljes szervezetre hatással lévő döntések. Ilyen például a fő programozási nyelv kiválasztása, a cloud provider megválasztása, a mikroszolgáltatás-architektúra bevezetése vagy egy vállalati szintű biztonsági szabvány elfogadása. Ezeket általában egy **architektúra tanács** vagy **vezetői testület** hozza meg, széles körű konzultáció után.
- **Taktikai döntések**: középtávú, egy adott termékre vagy csapatra jellemző, de más csapatokat is érintő döntések. Például egy adott szolgáltatás API-szerződésének kialakítása, egy adatmodell megváltoztatása, egy új technológia bevezetése egyetlen komponensben. Ezeket gyakran a **domain csapat** hozza meg, de kötelező konzultálni az érintett felekkel, és az eredményt ADR formájában rögzíteni.
- **Operatív döntések**: rövid távú, könnyen visszafordítható, helyi hatású döntések. Ilyen egy függvény belső implementációja, egy változó elnevezése, egy teszt hozzáadása, vagy egy cache-konfiguráció módosítása. Ezeket a **fejlesztő** önállóan hozza meg, a csapaton belüli kód-review és automatizált ellenőrzések keretein belül.

A döntési jogkörök kiosztásához használhatunk egy egyszerű **döntési mátrixot**, amely megadja, hogy egy adott döntési területen ki a felelős, kit kell konzultálni, és kit kell informálni. A legismertebb ilyen modell a **RACI** (Responsible, Accountable, Consulted, Informed), de a governance szempontjából gyakran elég egy leegyszerűsített változat is: **döntéshozó**, **konzultált**, **informált**. A mátrix készítésekor fontos, hogy minden döntési területnek pontosan egy döntéshozója legyen – a többes felelősség a gyakorlatban senki felelősségét jelenti.

Példa egy egyszerű governance-mátrixra (a táblázatot szövegesen, YAML formában is bemutathatjuk):

```yaml
governance_matrix:
  context: "Acme e-commerce platform"
  version: 1.2
  decision_areas:
    - area: "Programozási nyelv (backend)"
      type: strategic
      decision_maker: "Architecture Advisory Board"
      consulted: ["Platform Team", "Product Teams"]
      informed: ["CIO", "All Engineers"]
      tool: "ADR, Tech Radar"
    - area: "API szerződés (egy adott szolgáltatáshoz)"
      type: tactical
      decision_maker: "Service Owner Team"
      consulted: ["Consumer Teams", "API Guild"]
      informed: ["Architecture Advisory Board"]
      tool: "OpenAPI spec, ADR"
    - area: "Cache bevezetése egy végponton"
      type: operational
```

```
decision_maker: "Individual Engineer"
consulted: ["Team Lead (ha költség vagy kockázat)"]
informed: ["Team"]
tool: "Pull Request, Code Review"
```

Ez a mátrix nem csak a felelősségeket tisztázza, hanem azt is megmutatja, hogy milyen eszközzel dokumentáljuk a döntést. A **stratégiai döntések** esetében a döntéshozó testület, például egy Architecture Advisory Board, konzultál a platform- és termékcsapatokkal, majd ADR-ben rögzíti a döntést, és frissíti a technológiai radart. A **taktikai döntéseknél** a szolgáltatást birtokló csapat dönt, de egyeztet a fogyasztókkal és az API-szabványokért felelős guilddel, majd az API-szerződést verziózott formában publikálja. Az **operatív döntéseknél** a fejlesztő a kód-review folyamatban érvényesíti a döntést, és a csapat automatikus ellenőrzései gondoskodnak a betartásról.

A döntési jogkörök elosztása során kerülendő hiba, ha a hierarchiát összekeverjük a döntési kompetenciával. Egy vezető nem attól jó döntéshozó, mert magasabb pozícióban van, hanem mert rendelkezik a szükséges kontextussal, rálát a trade-offokra, és vállalja a döntés következményeit. A modern architektúra governance ezért a **kontextus alapú döntéshozatalt** támogatja: az a személy vagy csapat hozza a döntést, aki a legközelebb van az információhoz, és aki a következményeket viseli. Ezt gyakran **"you build it, you run it, you decide it"** elvnek nevezik, de ez nem jelent szabadosságot: a döntési mátrix és a szabványok adják meg a kereteket.

5.3.2 Centralizált és decentralizált döntéshozatal

A döntési jogkörök elosztásának két szélső pólusa van: a **centralizált** és a **decentralizált** modell. A gyakorlatban egyik sem létezik tiszta formában, de az arányok eltolódása meghatározza a szervezet működését.

A **centralizált döntéshozatal** előnye, hogy garantálja a konzisztenciát, a szabványok betartását, és lehetővé teszi a globális optimalizációt. Hátránya viszont, hogy lassú, elszakadhat a valós igényektől, és demotiválja a csapatokat, mert nem érzik magukénak a döntéseket. A klasszikus **Architecture Review Board (ARB)**, amely minden jelentősebb döntést jóváhagy, gyakran a centralizáció túlzott alkalmazásának példája: a csapatok hetekig várnak egy review-ra, majd a testület olyan döntést hoz, amely nincs összhangban a helyi kontextussal.

A **decentralizált döntéshozatal** előnye, hogy gyors, reagál a helyi igényekre, és növeli a csapatok autonómiáját és motivációját. Hátránya, hogy inkonzisztenciához, duplikált erőfeszítésekhez, és a globális optimum elvesztéséhez vezethet. Ha minden csapat szabadon választ technológiát, akkor a szervezetben tíz különböző üzenetküldő rendszer, öt különböző adatbázis és három különböző CI/CD pipeline fog működni, ami lehetetlenné teszi a tudásmegosztást és a közös üzemeltetést.

A megoldás a **hibrid modell**: a döntési mátrix pontosan meghatározza, hogy mely döntések centralizáltak és melyek decentralizáltak. A stratégiai, szervezeti szintű, vagy erős külső kényszereknek (pl. biztonsági megfelelés) alávetett döntések centralizáltak, míg a termékcsapatokon belüli, helyi kontextust igénylő döntések decentralizáltak. Például egy vállalat centralizálhatja a felhőszolgáltató kiválasztását és a fő konténerplatformot, de decentralizálhatja az egyes szolgáltatások belső architektúráját és a használt library-k kiválasztását.

Fontos, hogy a centralizált döntéseknél is bevonják az érintett csapatokat a konzultációba, és a döntés indoklását ADR formájában publikálják. A decentralizált döntéseknél pedig a csapatoknak be kell tartaniuk a közös szabványokat, és ha el akarnak térni, azt formalizált kivételkezelés keretében kell jelezniük.

5.3.3 Szabványok és kivételkezelés

A governance egyik legfontosabb eszköze a **szabványok** kialakítása. A szabványok olyan előre meghatározott irányelvek, amelyek csökkentik a döntési terhet, biztosítják a konzisztenciát, és megkönnyítik az új csapattagok belépését. Egy jól megírt szabvány nem csak előír, hanem elmagyarázza a mögöttes elvet is, így a csapatok értik, miért kell követniük, és mikor indokolt az eltérés.

A szabványok kialakításánál a következő lépéseket érdemes követni:

1. **Igény felmérése**: Milyen problémát old meg a szabvány? Például túl sok különböző naplózási formátum, vagy inkonzisztens API-elnevezések.
2. **Javaslat és vita**: Egy közösségi fórumon (például ADR, RFC, vagy guild meeting) megvitatják a javaslatot, és beépítik a visszajelzéseket.
3. **Kipróbálás**: A szabványt pilot jelleggel egy vagy két csapat alkalmazza, és a tapasztalatokat dokumentálják.
4. **Formalizálás**: A jóváhagyott szabványt verziózott dokumentumban teszik közzé, és beépítik az onboardingba, a code review-ba, valamint az automatizált ellenőrzésekbe.
5. **Felülvizsgálat**: A szabványokat rendszeresen felül kell vizsgálni, mert a technológia és a szervezet is változik.

A szabványoknak több **kötelezőségi szintje** lehet:

- **Kötelező (must)**: minden esetben be kell tartani, az eltéréshez formális kivétel jóváhagyása szükséges. Például: "Minden szolgáltatásnak Kubernetesen kell futnia."
- **Ajánlott (should)**: alapértelmezésben ezt kell követni, de indokolt esetben el lehet térni, ha a csapat dokumentálja az okot. Például: "Új szolgáltatáshoz gRPC-t használjunk, kivéve ha külső fél REST-et igényel."
- **Választható (may)**: több elfogadott lehetőség közül választhat a csapat; a szabvány felsorolja a lehetőségeket és a trade-offokat.
- **Tiltott (must not)**: bizonyos gyakorlatok tiltottak, mert ismert kockázatot hordoznak. Például: "Tilos jelszót plaintextben tárolni."

A szabványok betartatásának leghatékonyabb módja a **policy as code**: a szabályokat automatizált ellenőrzésekké alakítjuk, amelyek a CI/CD pipeline-ban futnak. Például egy Kubernetes-manifestet validáló policy engine (például Open Policy Agent) ellenőrizheti, hogy minden konténer rendelkezik-e erőforrás-limittel, vagy hogy nem használnak-e tiltott image-taget. Ez csökkenti a manuális ellenőrzés szükségességét, és azonnali visszajelzést ad a fejlesztőnek.

A szabványokkal kapcsolatban azonban fel kell készülni a **kivételekre**. Egy szabvány nem tud minden helyzetet lefedni, és ha túl merev, akkor a csapatok megkerülik, vagy indokolatlanul lassítja a fejlesztést. A **kivételkezelés** formalizált folyamat, amely lehetővé teszi, hogy egy csapat indokolt esetben eltérjen a szabványtól, de ezt dokumentálja, és a jóváhagyás után nyomon kövesse.

A kivételkezelés tipikus lépései:

1. **Kérelem benyújtása**: A csapat egy rövid dokumentumban (gyakran egy ADR vagy egy dedikált exception request) leírja, hogy mit szeretne csinálni, miért tér el a szabványtól, milyen alternatívákat vizsgált, és milyen kockázatokat azonosított.
2. **Gyors értékelés**: A kivétel súlyosságától függően a döntési mátrixban megjelölt fórum (például az Architecture Advisory Board vagy a platform csapat) értékeli a kérelmet. Kis kockázatú, rövid távú kivételt egyetlen reviewer is jóváhagyhat; nagy kockázatú kivételhez szélesebb körű konzultáció kell.

3. **Döntés és indoklás**: A jóváhagyást vagy elutasítást írásban, indoklással együtt rögzítik. Ha elutasítják, a csapatnak módosítania kell a tervét; ha jóváhagyják, a kivétel érvényességét és a felülvizsgálat időpontját is rögzítik.
4. **Nyomon követés**: A jóváhagyott kivételt nyilvántartásba veszik, és rendszeresen felülvizsgálják, hogy a kivételt okozó körülmény fennáll-e még. Ha a kivétel oka megszűnt, a csapatnak vissza kell térnie a szabványhoz.

Fontos, hogy a kivétel ne legyen büntetés vagy szégyen. A kivételek gyakran jelzik, hogy a szabvány elavult vagy nem elég rugalmas, és a gyakori kivételek a szabvány felülvizsgálatát kell, hogy kiváltsák. A governance tehát nem statikus, hanem folyamatos visszacsatoláson alapul.

5.3.4 Architecture Review Board és tanácsadó szerep

A hagyományos **Architecture Review Board (ARB)** egy központi testület, amely minden jelentősebb architektúra-döntést jóváhagy, mielőtt az implementáció elkezdődne. Ez a modell azonban számos problémát okoz: a review folyamat szűk keresztmetszetté válik, a testület tagjai gyakran nincsenek tisztában a konkrét projekt kontextusával, és a döntések lassan születnek meg. Ráadásul a csapatok passzívvá válnak, mert úgy érzik, hogy a döntéseket úgyis a testület hozza meg.

A modern megközelítés az ARB-t **tanácsadó testületként** definiálja újra. A hangsúly a **review** helyett a **guidance**-re kerül: a testület nem kapuőr, hanem segítő fórum, amely konzultációs lehetőséget biztosít, szabványokat alakít ki, és a kivételeket bírálja el. A döntés jogköre továbbra is a termékcsapaté vagy a megfelelő szintű döntéshozóé, de a testület ajánlásokat fogalmaz meg, és a stratégiai döntéseknél vétójoggal rendelkezhet.

Az így működő testület jellemzői:

- **Rotáció**: A tagok nem állandóak, hanem rendszeresen cserélődnek, így friss szemléletet hoznak, és elkerülik a "belső kör" kialakulását.
- **Keresztfunkcionális összetétel**: Nem csak senior architektekből áll, hanem képviselők a platform, a biztonság, a termékmenedzsment és a fejlesztői közösség részéről is.
- **Időkorlátos döntések**: A kivételek és a felülvizsgálatok határidejét előre meghatározzák, hogy ne lehessen a végtelenségig húzni.
- **Nyilvános működés**: A döntéseket, az ajánlásokat és a szabványokat mindenki számára elérhető helyen (pl. belső wiki, ADR-tár) publikálják.

A tanácsadó szerep nem jelenti azt, hogy a testületnek nincs hatalma. A stratégiai döntéseknél (pl. "milyen programozási nyelvet használjunk az új projektekhez") továbbra is a testület dönt, de a döntés előtt széles körű konzultációt folytat. A taktikai döntéseknél a testület csak akkor avatkozik be, ha a csapat kivételt kér, vagy ha a döntés más csapatokat is érint. A testület legfontosabb feladata a **szabványok életciklusának menedzselése**: összegyűjti a javaslatokat, moderálja a vitát, és gondoskodik a szabványok karbantartásáról.

5.3.5 Gyakorlati minta: governance-mátrix készítése

Az alábbiakban egy konkrét példán keresztül mutatjuk be a governance-mátrix elkészítését. Tegyük fel, hogy van egy fiktív e-kereskedelmi platform, az "Acme Shop", amely több mikroszolgáltatásból áll: katalógus, kosár, fizetés, rendelés, ajánló. A szervezetben három termékcsapat dolgozik, és van egy platform csapat, amely a Kubernetes-alapú infrastruktúrát és a CI/CD-t üzemelteti.

Első lépésként azonosítsuk a fontos döntési területeket, majd határozzuk meg mindegyikhez a döntés típusát, a döntéshozót, a konzultált feleket, az informált feleket és a dokumentációs eszközt. Az alábbi mátrixot YAML formában adjuk meg, de ugyanez táblázatban is ábrázolható.

```yaml
governance_matrix:
  context: "Acme Shop e-commerce platform"
  version: 1.0
  last_reviewed: 2025-03-15
  decision_areas:
    - area: "Programozási nyelv új szolgáltatásokhoz"
      type: strategic
      decision_maker: "Architecture Advisory Board"
      consulted: ["Platform Team", "Product Teams"]
      informed: ["All Engineers"]
      tool: "ADR, Tech Radar"
    - area: "Cloud provider kiválasztása"
      type: strategic
      decision_maker: "CTO, Architecture Advisory Board"
      consulted: ["Platform Team", "Finance", "Security"]
      informed: ["All Engineers"]
      tool: "ADR, vendor assessment"
    - area: "API-szerződés a katalógus szolgáltatáshoz"
      type: tactical
      decision_maker: "Catalog Team"
      consulted: ["Cart Team", "Recommendation Team", "API Guild"]
      informed: ["Architecture Advisory Board"]
      tool: "OpenAPI spec, ADR"
    - area: "Cache bevezetése a terméklista végponton"
      type: operational
      decision_maker: "Catalog Team Engineer"
      consulted: ["Team Lead", "Platform Team (ha költség)"]
      informed: ["Catalog Team"]
      tool: "Pull Request, Code Review"
    - area: "Adatbázis-séma módosítása a rendelés szolgáltatásban"
      type: tactical
      decision_maker: "Order Team"
      consulted: ["Data Engineering", "Reporting Team"]
      informed: ["Architecture Advisory Board (ha breaking change)"]
      tool: "Migration script, ADR"
```

A mátrix elkészítése után a csapatok megállapodnak a **szabványokban** is. Például:

- **Kötelező**: Minden szolgáltatás konténerben fut, Kubernetesen. Minden API-hoz OpenAPI-specifikáció készül.
- **Ajánlott**: Új szolgáltatásokhoz gRPC-t használjunk belső kommunikációra, de ha a fogyasztó REST-et igényel, akkor REST is elfogadott.
- **Tiltott**: Jelszavak plaintextben történő tárolása, manuális adatbázis-módosítás éles környezetben.

A kivételkezeléshez egy egyszerű sablont vezetnek be, amelyet a csapat kitölt, ha el akar térni egy kötelező szabványtól. A kivételkérelmet a csapat ADR-tárában `exception-<terület>-<dátum>` néven verzióznak, és a döntési mátrixban megjelölt fórum bírálja el. A jóváhagyott kivételhez érvényességi időt és felülvizsgálati dátumot rendelnek.

A governance-mátrix nem statikus dokumentum: negyedévente felülvizsgálják, és a visszajelzések alapján módosítják. A gyakori kivételek jelzik, hogy egy szabvány túl szigorú vagy elavult, és ilyenkor a szabványt frissítik.

5.3.6 Labor: governance-mátrix készítése

Cél: A résztvevők készítsenek egy governance-mátrixot egy adott kontextusra, rögzítsék a döntési naplót, és azonosítsanak legalább egy kivételkezelési esetet.

Kiinduló állapot: Adott egy fiktív rendszerleírás: egy "SmartHome Hub" nevű platform, amely IoT-eszközöket kezel, adatokat gyűjt, és szabályokat futtat. A rendszer három fő komponensből áll: eszközkezelő szolgáltatás, szabálymotor, valamint adatfeldolgozó és analitikai modul. A szervezetben két fejlesztőcsapat dolgozik: az "Eszköz csapat" és az "Adat csapat". Van egy platform mérnök, aki a felhő-infrastruktúráért felel.

Feladat:

1. Azonosítsd a legfontosabb döntési területeket (legalább ötöt) a fenti rendszerben, és sorold be őket stratégiai, taktikai vagy operatív kategóriába.
2. Készíts egy governance-mátrixot (például YAML vagy táblázat formájában), amely tartalmazza minden döntési területhez a döntéshozót, a konzultált feleket, az informált feleket és a dokumentációs eszközt.
3. Fogalmazz meg legalább három szabványt (egy kötelező, egy ajánlott, egy tiltott), amelyek a rendszerre vonatkoznak.
4. Írj le egy konkrét kivételkezelési szituációt: például az Adat csapat egy új, a szabványtól eltérő adatbázist akar bevezetni. Töltsd ki a kivételkérelem sablonját (indoklás, alternatívák, kockázatok, érvényességi idő).
5. A munkádat verziózd (például Git repository-ban), és a döntési naplót ADR formájában rögzítsd.

Elvárt eredmény: Egy verziózott dokumentum vagy jegyzetfüzet, amely tartalmazza a governance-mátrixot, a szabványokat, a kivételkérelem példáját, és az ellenőrzési szempontokat. A mért eredményt (a kivétel jóváhagyásra került-e) és a várt eredményt (a kivétel megfelelően dokumentált és elbírált) külön kell feltüntetni.

Ellenőrzési szempontok:

- Lefedi-e a mátrix a rendszer legfontosabb döntési területeit?
- Minden döntési területhez pontosan egy döntéshozó tartozik-e?
- A szabványok rendelkeznek-e kötelezőségi szinttel és indoklással?
- A kivételkezelés tartalmazza-e az indoklást, az alternatívákat, a kockázatokat és az érvényességi időt?
- A dokumentum verziózott-e, és a döntési napló visszakereshető-e?

Visszaállítási út: Mivel a feladat dokumentumkészítés, nincs szükség rendszer-visszaállításra. Ha a résztvevők kódot is módosítanak (például policy as code), akkor azt izolált környezetben tegyék, és a változtatásokat verziózzák, hogy vissza lehessen vonni.

5.3.7 Ellenőrzőlista a governance bevezetéséhez

- ☐ Döntési területek azonosítva és kategorizálva (stratégiai, taktikai, operatív).
- ☐ Minden döntési területhez egyértelmű döntéshozó rendelve.
- ☐ Konzultált és informált felek meghatározva.
- ☐ Dokumentációs eszközök (ADR, OpenAPI, Pull Request) kijelölve.
- ☐ Szabványok kötelezőségi szinttel (must, should, may, must not) és indoklással ellátva.
- ☐ Kivételkezelési folyamat formalizálva és sablonnal támogatva.
- ☐ A governance-mátrix és a szabványok verziózottak, hozzáférhetők.

- ☐ Automatizált ellenőrzések (policy as code) bevezetve, ahol lehetséges.
- ☐ A mátrixot rendszeresen felülvizsgálják, és a visszajelzéseket beépítik.

Önellenőrző kérdés: Mi történik, ha egy csapat a szabványtól eltérő technológiát akar használni? Írd le a lépéseket a kivételkezelési folyamat alapján, és indokold meg, miért fontos a formalizált eljárás a szabály megkerülése helyett.

5.3.8 Összefoglalás és átvezetés

Az architektúra governance célja nem a bürokrácia növelése, hanem a döntések átláthatóságának, nyomon követhetőségének és a csapatok közötti összhang megteremtése. A döntési jogkörök tudatos elosztása, a szabványok kialakítása és a kivételkezelés formalizálása lehetővé teszi, hogy a szervezet gyorsan tudjon mozogni anélkül, hogy technikai káoszba fulladna. A következő alfejezetben azt vizsgáljuk, hogyan válik az architektúra folyamatos tevékenységgé, és hogyan kapcsolódik a governance a mindennapi fejlesztői gyakorlathoz.

5.4 Architektúra mint folyamatos tevékenység

Az architektúra „megtervezése” sokáig egy jól körülhatárolható, a fejlesztést megelőző fázist jelentett a legtöbb szoftverfejlesztési módszertanban. A rendszer felépítését egy tervdokumentumban rögzítették, majd a fejlesztőcsapat feladata az volt, hogy ezt a tervet lefordítsa működő kódra. Ez a szemlélet egy bonyolult, sok ismeretlent tartalmazó környezetben azonban kudarcra van ítélve. Az architektúra ugyanis nem egy egyszeri termék, amit a projekt elején „leszállítunk”, hanem egy folyamatos tevékenység, egy tanulási folyamat, amely a rendszer teljes élettartama alatt zajlik.

Az architektúra mint folyamatos tevékenység (continuous architecture) értelmezésében a hangsúly nem a tökéletes előretervezésen, hanem a visszacsatolási hurkokon, a döntések folyamatos felülvizsgálatán és a változásokhoz való alkalmazkodáson van. Ez a fejezet azt mutatja be, hogyan működik ez a gyakorlatban, milyen eszközökkel tartható fenn a rendszer technikai integritása, és hogyan válik a dokumentáció a tanulás lenyomatává ahelyett, hogy a múltba merevedett tervek temetőjévé válna.

5.4.1 1. A tervezési fázis mítoszától a folyamatos tanulásig

A hagyományos vízesésmodellben a „Big Design Up Front” (BDUF) elv érvényesült: minél több döntést hozunk meg a projekt elején, annál biztonságosabbnak érezzük magunkat. A feltételezés az volt, hogy a követelmények stabilak, a technológiák ismertek, és a környezet kiszámítható. A valóság ezzel szemben az, hogy a szoftverfejlesztés nem gyártósori tevékenység, hanem ismeretlen problémák feltárása és megoldása. Az üzleti követelmények változnak, a felhasználói viselkedés csak éles üzemben derül ki, és a választott technológiai stack korlátai sokszor csak nagy terhelés alatt válnak láthatóvá.

Az architektúra mint folyamatos tevékenység nem tagadja a tervezés szükségességét. Azt állítja csupán, hogy a tervezés nem ér véget az első kódsor megírásával, és az architektúra nem kövülhet bele egyetlen dokumentumba. Ehelyett a tervezés és a kódolás párhuzamosan, egymást erősítve zajlik. Az architektúra döntései hipotézisek, amelyeket a működő rendszer viselkedése igazol vagy cáfol. Ha a rendszer nem úgy viselkedik, ahogy vártuk, az nem feltétlenül a fejlesztők hibája, hanem annak a jele, hogy a hipotézisünk téves volt. Ebben az esetben a helyes reakció nem a kód erőltetett módosítása a régi tervhez igazodva, hanem a terv módosítása a valósághoz.

Ez a tanulási ciklus a rövid iterációkban természetesen jelenik meg. Minden egyes lerakott inkrementum, minden egyes élesített szolgáltatás új információt szolgáltat. A megfigyelhetőség (observability), a felhasználói visszajelzések és a termelési incidensek mind adatpontok, amelyeket az architektúra folyamatos finomítására használhatunk. A kérdés tehát nem az, hogy „megterveztük-e a rendszert?", hanem az, hogy „van-e működő mechanizmusunk arra, hogy az új tudásunkat beépítsük a rendszer szerkezetébe?".

5.4.2 2. Evolúciós tervezés és visszacsatolás

Az evolúciós tervezés középpontjában az a felismerés áll, hogy a szoftverrendszer architektúrája időben változó entitás. Ahogy a biológiai evolúcióban a környezethez való alkalmazkodás a túlélés kulcsa, úgy a szoftverarchitektúrában is a változó üzleti és technikai környezethez való alkalmazkodás biztosítja a rendszer hosszú távú életképességét. Ez a folyamat azonban nem spontán módon megy végbe. Szükség van hozzá irányra, azaz folyamatos visszacsatolásra.

5.4.2.1 Visszacsatolási hurkok az architektúrában

A folyamatos architektúra munkafolyamat legfontosabb üzemanyaga a visszacsatolás. A visszacsatolás azt jelenti, hogy a rendszer működéséből és a fejlesztési folyamatból származó mérési adatokat aktívan visszavezetjük a döntéshozatalba. Az architektúra szempontjából a visszacsatolásnak több rétegét különböztethetjük meg:

1. **Technikai visszacsatolás a kódból:** A statikus elemzők, a kódminőség-mérők és a függőségelemzők folyamatosan jelzik, ha a kód szerkezete eltér az architekturális elveinktől. Ha például egy modul függőségi gráfja azt mutatja, hogy a rétegek közötti hívások nem a meghatározott irányban zajlanak, az egy korai figyelmeztető jel. Az ilyen típusú visszacsatolás automatikusan beépíthető a folyamatos integrációs folyamatba az architekturális fitness függvények segítségével.
2. **Futás közbeni visszacsatolás a produkcióból:** Az éles rendszerben futó szolgáltatások mérőszámai (válaszidő, hibaarány, erőforrás-kihasználtság, várakozási sor hossza) mindmind az architektúra egészségének indikátorai. Ha egy mikroszolgáltatás hibaaránya megnő, az nem csupán egy üzemeltetési esemény, hanem egy architekturális kérdés is. A kommunikációs mintánk (szinkron REST hívás) eléri-e a korlátját? Szükség van-e aszinkron üzenetsorra? A puszta kapacitásbővítés helyett az architekturális minta felülvizsgálata is a lehetőségek között kell, hogy szerepeljen.
3. **Piaci és felhasználói visszacsatolás:** A felhasználói viselkedés megváltozása új architekturális követelményeket szülhet. Ha egy webáruházban a felhasználók hirtelen elkezdenek nagy tömegben élőben követni egy árverést, az addig jól működő kérés-válasz modell helyett szükség lehet valós idejű push technológiára. Ez a visszacsatolás az architektúrát az üzleti doménhez köti.

5.4.2.2 Fitness függvények

A visszacsatolás hatékony működtetéséhez érdemes a minőségi attribútumainkat automatikusan ellenőrizhető formában is megfogalmazni. Az evolúciós architektúra koncepciójában ezeket a mechanizmusokat fitness függvényeknek nevezzük. Egy fitness függvény lényegében egy olyan teszt vagy ellenőrzés, amely egy adott architekturális jellemzőre fókuszál.

A gyakorlatban ez lehet egy egyszerű egységteszt, amely azt ellenőrzi, hogy egy adott osztály nem hív meg közvetlenül egy adatbázis-vezérlő osztályt, mert az a rétegek közötti elválasztást sértené. De lehet egy összetett terhelési teszt is, amely során a rendszer válaszidejét mérjük egy adott forgalmi mintánál. A kulcs az, hogy a fitness függvény objektív, folyamatosan futtatható és az eredménye egyértelműen értelmezhető. Így az architektúra nem a tervező asztalán létező elvont konstrukció, hanem a kód bázis őrzőjévé válik.

5.4.3 3. A dokumentáció és a döntések frissítése

Az architektúra mint folyamatos tevékenység egyik legnagyobb kihívása a tudás menedzselése. Ha a döntések csupán a fejlesztők fejében léteznek, vagy egy elavult wiki oldalon porosodnak, az architektúra láthatatlan. A folyamat során nem csupán a kódot kell frissítenünk, hanem a döntéseinket és a dokumentációnkat is.

5.4.3.1 Az ADR mint élő dokumentum

Az előző fejezetben (Architektúra governance és döntési jogkörök) bemutatott Architecture Decision Record (ADR) itt kapja meg a maga dinamikus szerepét. Az architektúra döntések nem egyszeri, megmásíthatatlan aktusok, hanem időben változó, felülvizsgálható, illetve felülírható elhatározások. Az ADR-eket ezért nem lezárt rekordokként, hanem élő dokumentumokként kell kezelnünk.

Egy ADR állapota a gyakorlatban a következőképp alakulhat:

1. **Javasolt (proposed):** Valaki felvet egy döntési lehetőséget. A döntés még nem született meg.
2. **Elfogadott (accepted):** A csapat eldöntötte, hogy az adott irányt követi. Az ADR rögzíti a kontextust, a döntést és a következményeket.
3. **Felülvizsgált (superseded):** Új információk birtokában a csapat úgy dönt, hogy az eredeti döntést felülírja. Ekkor nem töröljük a régi ADR-t, hanem az állapotát megváltoztatjuk, és hivatkozunk az új ADR-re. Ez a mechanizmus biztosítja a döntési napló teljességét.

Az ADR-ek folyamatos frissítése a folyamatos tanulás lenyomata. Ha egy rendszer dokumentációja nem tartalmaz felülvizsgált döntéseket, az annak a jele, hogy a tervezés valószínűleg már régen leállt, vagy a system a valóságban úgy fejlődik, hogy a döntéseket nem követik.

5.4.3.2 Living Documentation és Diagrams-as-Code

A dokumentáció akkor tud lépést tartani a folyamatos munkával, ha maga is a kódbázis része. A „living documentation” vagy élő dokumentáció koncepciója azt jelenti, hogy a lehető legtöbb dokumentáció automatikusan generálódik vagy verziókezelt, a kóddal együttműködő formában létezik. A C4 modell diagramok Mermaid vagy PlantUML formátumban a forráskód mellett, verziókezelőben tárolva természetes módon frissülnek, amikor a kódváltozás a diagram módosítását is magával hozza. A „diagrams-as-code” megközelítés elkerüli a kézzel rajzolt, bináris formátumú diagramok és a valós kód közötti szakadékot. A folyamatos architektúra motorja ezért a verziókezelési stratégia: az architekturális dokumentáció ugyanabba a repóba tartozik, mint a rendszer kódja. Így a code review folyamata során a diagramok változásai is ugyanolyan figyelmet kapnak, mint a kód változásai.

5.4.4 4. A folyamatos architektúra gyakorlati munkafolyamata

Hogyan néz ki a mindennapokban a folyamatos architektúra tevékenység? Hogyan illeszkedik a Scrum vagy Kanban keretei közé? A cél nem egy újabb adminisztratív ceremónia bevezetése, hanem az architekturális tudatosság beépítése a napi munkavégzésbe.

5.4.4.1 A „just-in-time" döntések

A folyamatos architektúrában a döntéseket a lehető legkésőbbi felelős pillanatban (last responsible moment) hozzuk meg. Ez nem a halogatást jelenti, hanem a tanulás maximalizálását. Minél később döntünk, annál több információnk van. Ha egy döntést túl korán hozunk meg, azzal befagyasztjuk a rendszer egy olyan részletét, amelyre még nem volt szükség, és később kiderülhet, hogy a döntés hibás volt.

A gyakorlatban ez úgy működik, hogy a csapat a sprint tervezés során csak azon döntéseket rögzíti, amelyek nélkül a következő sprint munkája nem indítható el. Ha van egy feltevésünk a technológiával kapcsolatban, akkor egy spike-ot vagy egy kísérleti megvalósítást (Proof of Concept) ütemezünk be, hogy a döntést megalapozzuk. Az architektúra szerepet betöltő személy (vagy a csapat tagjai) a sprint során folyamatosan részt vesz a vitákban és a code review-ban, biztosítva, hogy a döntések ne csússzanak el.

5.4.4.2 Visszacsatolási pontok a munkafolyamatban

A folyamatos architektúra munkafolyamat az alábbi lépések ciklikus ismétlődéséből áll:

1. **Feltárás (Exploration):** A sprint vagy az iteráció kezdetén a csapat feltárja a megvalósítandó feladatokat. Az architekturális feltárás során azonosítjuk azokat a területeket, ahol a rendszer szerkezetét módosítani kell, vagy ahol új döntéseket kell hozni.
2. **Döntés (Decision):** A csapat rögzíti az adott iterációra vonatkozó architekturális döntéseket ADR formájában. Ha a döntés bizonytalan, akkor a bizonytalanságot a lehető legkisebb területre korlátozzuk, és kísérletet tervezünk.
3. **Megvalósítás (Implementation):** A fejlesztők a feladatot a rögzített architekturális irányelvek szerint valósítják meg. Az architektúra irányelvek nem tiltások gyűjteménye, hanem a rendszer koherenciáját biztosító, közösen elfogadott minták és határok.
4. **Értékelés (Evaluation):** A rendszer élesben vagy tesztkörnyezetben történő futtatása során a fitness függvényeink és mérőszámaink alapján kiértékeljük a döntéseink hatását. Ezt a lépést gyakran kihagyják, pedig ezen múlik a tanulás.
5. **Dokumentálás és Felülvizsgálat (Documentation & Review):** Az eredmények ismeretében frissítjük az ADR-eket, a C4 diagramokat és a szükséges feljegyzéseket. Ha egy döntés rossznak bizonyult, ezt explicit módon rögzítjük, és az ADR-t felülvizsgáljuk.

Ez a ciklus nem párhuzamosan fut a fejlesztéssel, hanem maga a fejlesztés. A szervezeti tanulás akkor valósul meg, ha a ciklus minden egyes iterációban lezárul, és a megszerzett tudás a következő iteráció kiindulópontjává válik.

5.4.5 5. Gyakorlati példa: A rendeléskezelő szolgáltatás evolúciója

Nézzünk egy egyszerű példát a folyamatos architektúra működésére. Tegyük fel, hogy van egy „Rendeléskezelő" szolgáltatásunk, amely egy monolitikus alkalmazás részeként működik. A felhasználók rendeléseket adnak le, a rendszer pedig feldolgozza azokat. A jelenlegi architektúra jól működik, amíg a forgalom alacsony.

1. Esemény (Visszacsatolás a produkcióból): A megfigyelhetőségi rendszerünk azt jelzi, hogy a rendelés leadás válaszideje a csúcsidőben (a mérési ablak: egy hétköznapi este, a terhelés: 1500 rendelés/óra) az elfogadható 200 ms-ról 850 ms-ra nőtt. Az error budget (amely az 1 – SLO képletből adódik) gyors ütemben fogy.

2. Feltárás: A csapat elemzi a nyomkövetési adatokat (traces) és rájön, hogy a késést egy szinkron hívás okozza. A rendelés leadásakor a rendszer szinkron módon küld megerősítő e-mailt, és a megerősítő e-mail küldése lassú, mert a külső SMTP szolgáltató időnként másodpercek alatt válaszol.

3. Döntés: A csapat úgy dönt, hogy a megerősítő e-mail küldését aszinkronná teszi. A rendelés adatait egy üzenetsorba helyezi, ahonnan egy különálló feldolgozó szolgáltatás kiolvassa és elküldi az e-mailt. Ezt az ADR-t rögzítik:

ADR-2024-014: A megerősítő e-mail küldés aszinkronná tétele - **Kontextus:** A rendelés leadás válaszideje a csúcsidőben a külső SMTP szolgáltató lassú válasza miatt meghaladja az SLO-t (p95 $<$ 200 ms). - **Döntés:** Bevezetünk egy „RendelésFeldolgozó” sorba (jelenlegi megnevezés: MQ-ORDER-QUEUE-01) és egy „ÉrtesítésiSzolgáltatás” komponenst. A Rendeléskezelő szolgáltatás a rendelés mentése után egy `OrderPlaced` eseményt publikál a sorba. - **Alternatívák:** - *Külső SMTP szolgáltató cseréje:* A jelenlegi szolgáltató szerződött SLA-ja 99,5%, de a hálózati késleltetés nem szabályozható. A csere magas kockázatú és nem szünteti meg a szinkron hívásból adódó alapvető csatolást. - *Szálak és időtúllépés csökkentése a REST-hívásban:* Ez a megoldás csak elfedi a problémát, de a felhasználói élmény romlik, mert a megerősítő e-mail elveszhet. - **Következmények:** - A rendelés leadás válaszideje a rendelés mentésére korlátozódik, ami várhatóan 50 ms alá csökkenti a p95-öt. - Az üzenetfeldolgozás aszinkron jellege miatt be kell vezetnünk egy „legalább egyszer” kiszállítási garanciát, és az ÉrtesítésiSzolgáltatásnak idempotensen kell fogadnia az üzeneteket. - A rendszerben megjelenik egy új, meghibásodási pontként kezelhető komponens (az üzenetsor). A hibatűrés biztosításához a sor klaszterét magas rendelkezésre állásra kell konfigurálni. - **Állapot:** Elfogadva.

4. Megvalósítás: A csapat a következő sprintben megvalósítja a változást. A C4 konténer-diagramon a régi szinkron nyíl helyére egy aszinkron kapcsolat kerül, megjelölve a protokollt (AMQP) és a hibatűrési stratégiát (retry policy).

5. Értékelés: A következő mérési ablakban (a csúcsidőben, 1500 rendelés/óra terhelés mellett) a Rendeléskezelő szolgáltatás p95 válaszideje 55 ms-ra csökken. Az ÉrtesítésiSzolgáltatás p95 feldolgozási ideje 150 ms. A rendelkezésre állás meghaladja a 99,9%-ot. A hipotézisünk beigazolódott. Az ADR-be bekerül az értékelés eredménye.

Ez a példa bemutatja, hogy a folyamatos architektúra hogyan reagál egy mérési eredményre, és hogyan vezeti végig a döntést a teljes életcikluson.

5.4.6 6. Gyakori hibák és a helyes megoldás

A folyamatos architektúra működtetése során több tipikus hibába eshetünk. Fontos felismernünk ezeket, és tudatosan kerülnünk őket.

1. hiba: Az „Ivory Tower” architektúra A csapat egy architektúra-csoportot hoz létre, amely távolságot tart a fejlesztéstől, és a sprinttempótól függetlenül készíti a terveket. Ez a szervezeti anti-minta ahhoz vezet, hogy az architektúra döntései nem épülnek be a napi munkába, mert a fejlesztők nem érzik magukénak azokat. A megoldás az, ha az architektúra szerep a csapatban, nem pedig a csapat felett van. Az architektnak vagy technikai vezetőnek a csapat tagjával kell együttműködnie, például a code review és a páros programozás során.

2. hiba: A dokumentáció elavulása A csapat nagy energiát fektet a kezdeti dokumentációba, de ahogy a fejlesztés felgyorsul, a dokumentumok nem frissülnek. A C4 diagramok a rendszer egy korábbi állapotát mutatják, az ADR-ek pedig hiányoznak a legújabb döntésekről. A megoldás a diagrams-as-code és a verziókezelt dokumentáció bevezetése. Ha a diagram a kódbázisban él, a fejlesztő a pull request részeként ugyanolyan természetességgel frissíti, mint a forráskódot.

3. hiba: A visszacsatolás figyelmen kívül hagyása A csapat bevezet egy új architekturális mintát (például egy cache-réteget), de nem méri annak hatását. A döntés megszületik, azonban a tanulási ciklus nem zárul le. Nem tudjuk, hogy a döntésünk sikeres volt-e, mert nem határoztunk meg előzetesen mérési mutatókat és mérési ablakot. A megoldás az, ha minden architekturális változtatást egy quality attribute scenario-val és egy mérési tervvel kapcsolunk össze. A változtatás előtt rögzítjük a jelenlegi állapotot, a várt eredményt, és a mérés elvégzése után ellenőrizzük a tényleges eredményt.

5.4.7 7. Labor: Folyamatos architektúra-munkafolyamat tervezése

Ez a labor egy folyamatos architektúra-munkafolyamat tervezését kéri, reprodukálható, verziókezelhető formátumban. A labor célja, hogy a résztvevő a saját csapatára szabottan megtervezze a döntéshozatali és tanulási ciklust.

Feladat leírása: Válassz ki egy meglévő, kis méretű szolgáltatást vagy modult a csapatod életéből (vagy használj egy ismert nyílt forráskódú projektet mint referenciát). Tervezd meg a folyamatos architektúra munkafolyamatát erre a rendszerre.

Átadandó eredmény: Készítsen egy dokumentumot (például Markdown formátumban), amely a következőket tartalmazza:

1. **Kontextus rögzítése:**
   - A rendszer neve és célja.
   - A rendszer aktuális architektúrájának rövid leírása (C4 Context és Container szint).
   - A fejlesztési folyamat jellemzése (Scrum, Kanban stb.).
2. **Döntési napló (ADR) sablon:**
   - Definiálj egy ADR sablont a csapat számára, amelynek szükségszerűen tartalmaznia kell a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
   - Nyiss egy ADR-t egy olyan döntésről, amelyről tudod, hogy a csapatnak a közeljövőben meg kell hoznia (pl. egy új adatbázistechnológia bevezetése).
3. **Visszacsatolási és mérési terv:**
   - Válassz ki egy minőségi attribútumot (pl. válaszidő), és fogalmazz meg hozzá egy quality attribute scenario-t.
   - Határozd meg a mérési tervet: mi a jelenlegi mért érték, mi a várt érték, milyen mérési ablakban és milyen terhelés mellett végzed a mérést.
   - Különítsd el a mért és a várt eredményeket a dokumentumban. A mérés elvégzése után rögzítsd a tényleges eredményt.
4. **Döntéshozatali visszacsatolási pontok:**
   - Illeszd be a tervezett munkafolyamatot a csapat meglévő agilis folyamatába. Hol történik a sprintben a döntések feltárása, rögzítése és felülvizsgálata?
5. **Ellenőrzési szempontok:**

- Sorolj fel legalább 5 ellenőrzési szempontot, amellyel a csapat a sprint végén ellenőrzi, hogy a folyamatos architektúra tevékenység megfelelően működött-e.

Visszaállítási út: Mivel a laborban nem hajtunk végre destruktív műveletet, a visszaállítás csupán a létrehozott fájlok verziókezelőből való törlését jelenti. Ha a labor során hipotetikus döntést hozunk, a döntést nem kell éles rendszerben bevezetni.

5.4.8 Önellenőrző kérdés

Magyarázd el, miért van szükség a döntések felülvizsgálatára (ADR superseded állapot) egy folyamatosan fejlődő rendszerben, és hozz egy példát arra, amikor egy korábban helyes döntés később hibásnak bizonyulhat!

5.4.9 Ellenőrzőlista

- ☐ Van-e a csapatnak működő mechanizmusa arra, hogy a produkcióból származó mérési adatokat (válaszidő, hibaarány) visszavezesse az architekturális döntéshozatalba?
- ☐ Az architekturális döntéseket a csapat folyamatosan ADR formában rögzíti, és a döntések állapota szükség esetén felülvizsgálatra kerül?
- ☐ A dokumentáció (diagramok, feljegyzések) a verziókezelőben, a kóddal együttműködve, diagrams-as-code formátumban frissül?
- ☐ Minden architekturális változtatáshoz tartozik minőségi forgatókönyv és mérési terv?
- ☐ A döntéseket a lehető legkésőbbi felelős pillanatban hozzák meg, és a bizonytalanságot spike vagy PoC segítségével kezelik?

6 Dokumentálás és kommunikáció

6.1 Nézetek és a 4+1 modell

A szoftverarchitektúra dokumentálásának egyik alapkérdése, hogy egyetlen diagram vagy leírás nem képes egyszerre megmutatni a rendszer minden lényeges szempontját. A fejlesztő a kód felépítését, a modulok közötti függőségeket keresi; az üzemeltető a telepítési topológiát, a hálózati kapcsolatokat és a futásidejű erőforrásokat; a termékmenedzser a felhasználói forgatókönyveket; a biztonsági szakértő pedig az adatáramlást és a bizalmi határokat. Ha mindezt egyetlen, zsúfolt ábrába próbáljuk sűríteni, az ábra áttekinthetetlenné válik, és egyetlen érdekelt fél kérdésére sem ad pontos választ.

Ezt a feszültséget a nézetek és nézőpontok fogalma oldja fel. A **nézet (view)** egy adott érdekelt fél számára készített, konkrét ábrázolás: diagramok, táblázatok, szöveges elemek összessége, amely a rendszer egy jól körülhatárolt aspektusát mutatja be. A **nézőpont (viewpoint)** pedig egy sablon vagy előírás, amely meghatározza, hogyan kell kinéznie egy ilyen nézetnek: milyen elemeket, kapcsolatokat, jelöléseket és szabályokat kell használni. Ha a nézet a válasz egy kérdésre, akkor a nézőpont azt írja le, hogyan kell megfogalmazni a választ.

A 4+1 nézetmodell Philippe Kruchten nevéhez fűződik, és az egyik legelterjedtebb gyakorlati nézőpontkészlet. Négy fő nézetet különböztet meg – logikai, folyamat-, fejlesztési és fizikai nézet –, valamint egy ötödik, összekötő szerepű forgatókönyv-nézetet, amely a másik négyet validálja. A modell ereje abban rejlik, hogy mindegyik nézet más-más érdekelthez szól, mégis közös fogalmi mag köré szervezi a dokumentációt.

6.1.1 A 4+1 modell öt nézete

A modell a következő öt nézetre épül:

Nézet	Elsődleges közönség	Fő kérdés	Tipikus diagramtípusok
Logikai nézet	fejlesztők, domain-szakértők	Milyen funkcionális követelményeket valósít meg a rendszer, és milyen fő absztrakciókból áll?	osztálydiagram, komponensdiagram, csomagdiagram

Nézet	Elsődleges közönség	Fő kérdés	Tipikus diagramtípusok
Folyamat-nézet	fejlesztők, teljesítménymérnökök, üzemeltetők	Hogyan viselkedik a rendszer futásidőben, milyen folyamatok, szálak és konkurencia-mechanizmusok vannak?	szekvenciadiagram, aktivitásdiagram, állapotgép
Fejlesztési nézet	fejlesztők, build-mérnökök, release-menedzserek	Hogyan szerveződik a forráskód modulokra, csomagokra és build-egységekre?	csomagdiagram, komponensdiagram, build gráf
Fizikai nézet	üzemeltetők, rendszermérnökök, hálózati mérnökök	Hogyan települ a szoftver hardverre, konténerekre, csomópontokra, és hogyan kapcsolódnak ezek?	telepítési diagram, hálózati diagram
Forgatókönyv-nézet	minden érdekelt fél	Mely használati esetek és minőségi forgatókönyvek kötik össze a többi nézetet?	use case diagram, szöveges forgatókönyv-leírás

A forgatókönyv-nézet az „+1” a 4+1 elnevezésben, mert nem önálló szerkezeti nézet, hanem a másik négy nézetet kapcsolja össze konkrét használati eseteken és minőségi forgatókönyveken keresztül. Egy adott forgatókönyv végigkövetése során ellenőrizhető, hogy a logikai nézetben definiált komponensek hogyan jelennek meg a folyamatnézetben, hogyan épülnek fel a fejlesztési nézetben, és hova települnek a fizikai nézetben.

6.1.2 Logikai nézet

A logikai nézet a rendszer funkcionális követelményeit tükrözi. Itt azonosítjuk a fő absztrakciókat – entitásokat, szolgáltatásokat, modulokat – és azok kapcsolatait. A hangsúly a rendszer **szerkezetén** van, nem a futásidejű viselkedésén vagy a telepítésen.

Egy online rendeléskezelő rendszer példáján a logikai nézetben megjelenhetnek a következő elemek:

- `Ügyfél` entitás, amely azonosítja a vásárlót.

- `Rendelés` entitás, amely az ügyfél által leadott rendelést reprezentálja.
- `RendelésTétel` entitás, amely egy rendelés egy sorát írja le.
- `RendelésSzolgáltatás` komponens, amely a rendelések életciklusát kezeli.
- `KészletSzolgáltatás` komponens, amely a termékek készletét foglalja le.

A kapcsolatok: egy `Ügyfél` több `Rendelést` adhat le, egy `Rendelés` több `RendelésTételt` tartalmaz. A `RendelésSzolgáltatás` hívja a `KészletSzolgáltatást` a tételek foglalásához.

Mermaid osztálydiagram formájában ez így ábrázolható:

```
classDiagram
    class Ügyfél {
        +ügyfélAzonosító: UUID
        +név: string
        +email: string
    }
    class Rendelés {
        +rendelésAzonosító: UUID
        +leadásDátum: datetime
        +állapot: RendelésÁllapot
        +leadás()
        +visszavonás()
    }
    class RendelésTétel {
        +termékAzonosító: UUID
        +mennyiség: int
        +egységár: decimal
    }
    class RendelésSzolgáltatás {
        +rendelésLeadás(rendelés: Rendelés): Rendelés
        +rendelésVisszavonás(rendelésAzonosító: UUID): void
    }
    class KészletSzolgáltatás {
        +készletFoglalás(tételek: RendelésTétel[]): FoglalásVálasz
        +készletFelszabadítás(tételek: RendelésTétel[]): void
    }

    Ügyfél "1" --> "*" Rendelés : lead
    Rendelés "1" --> "*" RendelésTétel : tartalmaz
    RendelésSzolgáltatás ..> KészletSzolgáltatás : hív
    RendelésSzolgáltatás --> Rendelés : kezel
```

Jelmagyarázat: folytonos vonal asszociáció, szaggatott vonal függőség/hívás. `1` és `*` a multiplicitást jelöli.

Feltételjegyzet: a diagram a logikai szerkezetet mutatja, nem a telepítést. A komponensek határait a domain-felelősségek alapján húztuk meg; a fizikai elhelyezkedés a fizikai nézetben látható.

A logikai nézetben fontos a rendszerhatár jelölése is: mi tartozik a rendszerhez, és mi külső szereplő vagy külső rendszer. A fenti példában az `Ügyfél` külső szereplő, a `Rendelés`, `RendelésTétel`, `RendelésSzolgáltatás`, `KészletSzolgáltatás` pedig a rendszer belső elemei.

6.1.3 Folyamatnézet

A folyamatnézet a rendszer futásidejű viselkedésére összpontosít. Itt jelennek meg a folyamatok, szálak, üzenetváltások, konkurencia és szinkronizáció. A logikai nézetben definiált komponensek itt aktív szereplőkként lépnek fel: ki kit hív, milyen sorrendben, milyen időzítéssel.

A rendelés leadásának folyamata szekvenciadiagramon:

```
sequenceDiagram
    participant Ü as Ügyfél
    participant API as RendelésAPI
    participant RSz as RendelésSzolgáltatás
    participant KSz as KészletSzolgáltatás
    participant DB as RendelésDB

    Ü->>API: POST /rendeles
    API->>RSz: rendelésLeadás(rendelés)
    RSz->>KSz: készletFoglalás(tételek)
    KSz-->>RSz: foglalásVisszaigazolás
    RSz->>DB: rendelésMentése
    DB-->>RSz: mentésSikeres
    RSz-->>API: rendelésLétrehozva
    API-->>Ü: 201 Created
```

Jelmagyarázat: szaggatott nyíl szinkron hívás, pontozott nyíl válasz. A vastag vonal az aktív időszakot jelöli (itt elhagyva az egyszerűség kedvéért).

Feltételjegyzet: a diagram a sikeres utat mutatja. A hibakezelést – például ha a készletfoglalás sikertelen – külön szekvenciadiagram vagy aktivitásdiagram ábrázolja.

A folyamatnézetben fontos a konkurencia kezelése. Ha a `RendelésSzolgáltatás` párhuzamosan több kérést szolgál ki, akkor meg kell határozni, hogy a készletfoglalás tranzakcionális-e, használ-e zárat, vagy optimistic lockingot alkalmaz. A folyamatnézetben a szinkronizációs pontok, a szálbiztosság és a backpressure is láthatóvá tehető.

A teljesítménnyel kapcsolatos kérdések is ide tartoznak: ha a rendszer csúcsterhelésen másodpercenként 120 rendelést fogad, és az átlagos feldolgozási idő 2,5 másodperc, akkor a Little-törvény szerint a rendszerben lévő kérések átlagos száma $L = \lambda W = 120 \cdot 2{,}5 = 300$ kérés. Ez a folyamatnézetben az erőforrás-kihasználtság és a szálkészlet méretezésének alapja. A folyamatnézet tehát nemcsak az üzenetváltások sorrendjét, hanem a kapacitástervezéshez szükséges futásidejű paramétereket is dokumentálja.

6.1.4 Fejlesztési nézet

A fejlesztési nézet a forráskód szerveződését mutatja: modulok, csomagok, build-egységek, verziók és függőségek. Ez a nézet a fejlesztők és a build-mérnökök számára releváns, mert megmondja, hogyan áll össze a rendszer a forráskódból.

A rendeléskezelő rendszer fejlesztési nézete a következő modulokat tartalmazhatja:

- `rendeles-api` – a HTTP API réteg, a bejövő kérések kezelése.
- `rendeles-szolgaltatas` – az üzleti logika, a rendelések életciklusa.
- `keszlet-szolgaltatas` – a készletkezelésért felelős modul.
- `kozost-util` – közös segédosztályok, dátumkezelés, azonosító-generálás.

A modulok közötti függőségek Mermaid formában:

```
flowchart TD
    subgraph Forráskód-modulok
        A[rendeles-api]
        B[rendeles-szolgaltatas]
        C[keszlet-szolgaltatas]
        D[kozost-util]
    end
    A --> B
    B --> C
```

```
    B --> D
    C --> D
```

Jelmagyarázat: a nyíl a fordítási vagy futásidejű függőséget jelöli. `A --> B` azt jelenti, hogy az `A` modul függ a `B` modultól.

Feltételjegyzet: a diagram a forráskód-szintű modulszerkezetet mutatja, nem a telepítési topológiát. A modulok egy folyamatban is futhatnak, vagy külön szolgáltatásokként is települhetnek; ezt a fizikai nézet dönti el.

A fejlesztési nézetben fontos a csomagok és a build-egységek egyértelmű elnevezése. Ha a logikai nézetben `RendelésSzolgáltatás` komponens szerepel, akkor a fejlesztési nézetben a hozzá tartozó modulnak következetesen ugyanazt a fogalmat kell jelölnie, például `rendeles-szolgaltatas` néven. Az elnevezési konvenciókat a nézettervben rögzíteni kell.

6.1.5 Fizikai nézet

A fizikai nézet azt mutatja meg, hogy a szoftver hogyan helyezkedik el a hardveren, a virtuális gépeken, a konténereken vagy a felhőszolgáltatásokon. Itt jelennek meg a csomópontok, a hálózati kapcsolatok, a protokollok, a skálázási egységek és a rendelkezésre állást befolyásoló redundanciák.

A rendeléskezelő rendszer egy lehetséges fizikai nézete konténeres környezetben:

```
flowchart LR
    subgraph CsomópontA[Számítási csomópont]
        subgraph KonténerA[rendeles-api konténer]
            A[Node.js folyamat]
        end
        subgraph KonténerB[rendeles-szolgaltatas konténer]
            B[Node.js folyamat]
        end
    end
    subgraph CsomópontB[Adatbázis csomópont]
        C[(PostgreSQL)]
    end
    subgraph CsomópontC[Készlet szolgáltatás csomópont]
        D[Go folyamat]
    end

    A -->|HTTPS| B
    B -->|TCP/5432| C
    B -->|HTTPS| D
```

Jelmagyarázat: a téglalapok csomópontokat vagy konténereket jelölnek, a henger adatbázist. A nyilak a hálózati kapcsolatokat és a protokollt mutatják.

Feltételjegyzet: a diagram egy lehetséges telepítési topológiát mutat. A skálázás, a terheléselosztó és a redundancia külön diagramon vagy szöveges leírásban részletezhető.

A fizikai nézetben a rendszerhatárt is jelölni kell: mi fut a saját adatközpontunkban vagy felhőkörnyezetünkben, és mi külső szolgáltatás (például fizetési szolgáltató). A kapcsolatokon a protokoll, a szinkronitás és a hibatűrés is szerepeljen. Például a `rendeles-szolgaltatas` és a `keszlet-szolgaltatas` közötti HTTPS kapcsolat szinkron, és ha a készlet szolgáltatás nem elérhető, a rendelés leadása meghiúsul – ezt a hibatűrési követelményeknél kell kezelni.

6.1.6 Forgatókönyv-nézet

A forgatókönyv-nézet a 4+1 modell összekötő eleme. Itt olyan konkrét használati eseteket és minőségi forgatókönyveket dokumentálunk, amelyek a másik négy nézet elemeit működés közben mutatják be. A forgatókönyv lehet funkcionális – például „az ügyfél lead egy rendelést” – vagy minőségi – például „a rendelés leadása csúcsterhelés alatt is legfeljebb 500 ms-ig tart”.

Egy funkcionális forgatókönyv szövegesen:

Forgatókönyv: Rendelés leadása

- **Előfeltétel:** az ügyfél bejelentkezett, a kosárban legalább egy tétel van.
- **Lépések:**
  1. Az ügyfél elküldi a rendelést a `RendelésAPI`-nak.
  2. A `RendelésSzolgáltatás` validálja a tételeket.
  3. A `KészletSzolgáltatás` lefoglalja a készletet.
  4. A `RendelésSzolgáltatás` elmenti a rendelést a `RendelésDB`-be.
  5. A rendszer visszaadja a `201 Created` választ.
- **Utófeltétel:** a rendelés `LÉTREHOZVA` állapotban van, a készlet foglalása megtörtént.
- **Kivételek:** ha a készletfoglalás sikertelen, a rendszer `409 Conflict` választ ad, és a rendelés nem jön létre.

A minőségi forgatókönyv a korábban megismert formában írható le:

Minőségi forgatókönyv: Rendelésfeldolgozási válaszidő

- **Inger:** 120 rendelés/másodperc érkezik a `RendelésAPI`-ra.
- **Környezet:** éles környezet, két példányban futó `rendeles-szolgaltatas`, `keszlet-szolgaltatas` elérhető.
- **Válasz:** a rendelés leadásának átlagos válaszideje legfeljebb 250 ms, a 95. percentilis legfeljebb 500 ms.
- **Mérőszám:** a válaszidő percentilisei, 60 másodperces mérési ablakban, a terhelés alatt.

A forgatókönyv-nézetben a hangsúly a **nyomon követhetőségen** van: minden forgatókönyvnek hivatkoznia kell a logikai, folyamat-, fejlesztési és fizikai nézet megfelelő elemeire. A fenti példában a forgatókönyv hivatkozik a `RendelésAPI`, `RendelésSzolgáltatás`, `KészletSzolgáltatás`, `RendelésDB` komponensekre (logikai nézet), a szekvenciadiagramra (folyamatnézet), a modulokra (fejlesztési nézet) és a telepítési csomópontokra (fizikai nézet).

6.1.7 A nézetek közötti konzisztencia

A 4+1 modell csak akkor működik, ha a nézetek nem egymástól független, elszigetelt leírások, hanem egy közös, konzisztens dokumentációt alkotnak. A konzisztencia azt jelenti, hogy ugyanaz az elem – például a `RendelésSzolgáltatás` – minden nézetben ugyanazt a fogalmat jelöli, ugyanazzal a névvel, és a kapcsolatai sem mondanak ellent egymásnak.

A konzisztencia leggyakoribb megsértései:

- **Elnevezési eltérés:** a logikai nézetben `RendelésSzolgáltatás`, a fejlesztési nézetben `rendeles-szolgaltatas`, a fizikai nézetben `order-service` konténer. Bár hasonlónak tűnnek, a következetlen elnevezés megnehezíti a nyomon követést.

- **Szerkezeti ellentmondás:** a logikai nézetben a `RendelésSzolgáltatás` közvetlenül hívja a `KészletSzolgáltatást`, a fizikai nézetben viszont nincs hálózati kapcsolat a két komponens között, mert külön csomópontokon futnak, és a tűzfal blokkolja a forgalmat.
- **Hiányzó forgatókönyv:** a nézetek elkészülnek, de nincs olyan forgatókönyv, amely a fő funkciókat végigkövetné, így nem derül ki, hogy a nézetek valóban ugyanazt a rendszert írják le.
- **Elavult nézet:** az egyik nézet frissül, a másik nem, így a dokumentáció inkonzisztenssé válik.

A konzisztencia biztosításának alapja a **nyomon követhetőség (traceability)**. Minden elemnek legyen egyértelmű azonosítója, és a nézetek közötti hivatkozásoknak explicitnek kell lenniük. Például:

- A logikai nézet `RendelésSzolgáltatás` komponense a fejlesztési nézet `rendeles-szolgaltatas` moduljára, a fizikai nézet `rendeles-szolgaltatas` konténerére, és a forgatókönyv-nézet „Rendelés leadása” forgatókönyvére hivatkozik.
- A folyamatnézet szekvenciadiagramján szereplő életvonalak a logikai nézet komponenseire mutatnak.

A nyomon követhetőség megvalósítható egy egyszerű táblázattal vagy a diagrams-as-code eszközök beépített hivatkozásaival. A lényeg, hogy a dokumentációt olvasó bármelyik nézetből el tudjon jutni a kapcsolódó többi nézethez.

6.1.8 Gyakori hibák és ellenőrzés

A 4+1 modell alkalmazása során a következő tipikus hibákat érdemes elkerülni:

1. **Egyetlen nézetbe sűríteni mindent.** A logikai nézet diagramjára felkerülnek a processzorok, a szálak és a telepítési részletek is. Ehelyett a felelősségeket szét kell választani a nézetek között.
2. **A forgatókönyv-nézet elhagyása.** A négy szerkezeti nézet elkészül, de nincs használati eset vagy minőségi forgatókönyv, amely összekötné őket. A forgatókönyv nélkül a nézetek párhuzamos, egymástól független leírások maradnak.
3. **Következetlen elnevezések.** Ugyanazt a komponenst minden nézetben azonos néven kell szerepeltetni. Ha a konvenció megváltozik, azt a nézettervben dokumentálni kell.
4. **Hiányzó rendszerhatár.** A diagramokon nem világos, mi tartozik a rendszerhez, és mi külső szereplő vagy külső rendszer. A rendszerhatár minden nézetben kötelező elem.
5. **Túl sok vagy túl kevés részlet.** A nézeteknek a célközönséghez kell igazodniuk. A fejlesztési nézetben a build-egységek szintje a megfelelő, a fizikai nézetben a csomópontok és konténerek szintje. A túlzott részletezés ugyanúgy rontja az áttekinthetőséget, mint a hiányos leírás.

Ellenőrzőlista a 4+1 nézettervhez:

- ☐ Mind az öt nézet elkészült, és mindegyiknek van meghatározott célja és közönsége.
- ☐ A rendszerhatár minden nézetben jelölve van.
- ☐ Az elemek elnevezése következetes a nézetek között.
- ☐ Minden komponens rendelkezik felelősség- és interfészleírással.
- ☐ A kapcsolatokon szerepel a protokoll, a szinkronitás és a hibatűrés (ahol releváns).
- ☐ Legalább egy funkcionális és egy minőségi forgatókönyv végigköveti a fő funkciókat.
- ☐ A nézetek közötti nyomon követhetőség explicit hivatkozásokkal biztosított.
- ☐ A diagramok diagrams-as-code formában, verziózottan tárolódnak.

6.1.9 Labor: Nézetterv készítése egy rendszerhez

Ez a labor a 4+1 modell gyakorlati alkalmazását gyakoroltatja. A cél egy olyan nézetterv elkészítése, amely reprodukálható, verziózott, és a nézetek közötti konzisztenciát is ellenőrzi.

6.1.10 Kiinduló állapot

Adott egy egyszerű rendszer funkcionális és nem funkcionális követelményeinek rövid leírása:

Rendszer: Könyvtári kölcsönző API (KKÖ-API)

- **Funkcionális követelmények:**
  - Az olvasó lekérdezheti a könyvtár katalógusát.
  - Az olvasó kölcsönözhet könyvet, ha van szabad példány.
  - A könyvtáros rögzítheti a visszahozást és a készletet.
- **Nem funkcionális követelmények:**
  - A katalóguslekérdezés válaszideje p95 $<$ 300 ms, 100 kérés/mp terhelés mellett.
  - A rendszer rendelkezésre állása 99,9% havi ablakban.
  - A kölcsönzési művelet tranzakcionális: ha a készletfoglalás sikertelen, a kölcsönzés nem jöhet létre.
- **Kényszerek:**
  - A rendszernek konténerizáltnak kell lennie, és Kubernetesen kell futnia.
  - Az adatbázis PostgreSQL.

A laborhoz tartozik egy üres, verziózott dokumentum-sablon (például `docs/` mappa a git repositoryban).

6.1.11 Elvárt eredmény

Egy verziózott dokumentum vagy jegyzetfüzet, amely tartalmazza:

1. **Kontextusleírás:** a rendszer neve, célja, a fő érdekeltek.
2. **Nézetterv:** mind az öt nézet (logikai, folyamat, fejlesztési, fizikai, forgatókönyv) elkészült, mindegyikhez diagramok és szöveges leírások tartoznak.
3. **Döntési napló:** legalább egy ADR, amely rögzíti, miért ezt a nézetkészletet választottuk, és milyen alternatívákat vetettünk el (például miért nem elég egyetlen diagram).
4. **Nyomon követhetőségi táblázat:** a fő komponensek hozzárendelése a nézetekhez.
5. **Ellenőrzési szempontok:** a fenti ellenőrzőlista alkalmazása a saját dokumentációra.

6.1.12 A labor lépései

1. **Feltárás:** azonosítsd a fő érdekelteket és az ő kérdéseiket. Készíts egy rövid stakeholder-listát.
2. **Nézetterv készítése:** minden nézethez válassz megfelelő diagramtípust, és készítsd el a diagramokat diagrams-as-code formában (Mermaid, PlantUML vagy Structurizr DSL). A diagramokhoz adj címet, jelmagyarázatot és feltételjegyzetet.
3. **Döntési napló:** rögzítsd a nézetválasztással kapcsolatos döntéseket ADR formában: kontextus, döntés, alternatívák, következmények, állapot.

4. **Konzisztencia-ellenőrzés:** ellenőrizd, hogy az elemek elnevezése következetes-e, a forgatókönyvek lefedik-e a fő funkciókat, és a rendszerhatár mindenhol jelölve van-e.
5. **Mért és várt eredmény összehasonlítása:** írd le, mely nézetek készültek el, és ez mennyiben felel meg az elvárt eredménynek. Jelöld, ha valamelyik nézet hiányos vagy elmaradt.

6.1.13 Ellenőrzési szempontok

- ☐ Mind az öt nézet jelen van, és mindegyikhez tartozik legalább egy diagram.
- ☐ A diagramok diagrams-as-code formában készültek, és a dokumentum verziózott.
- ☐ Az ADR-ek tartalmazzák a kontextust, döntést, alternatívákat, következményeket és állapotot.
- ☐ A nyomon követhetőségi táblázat legalább 5 komponenst lefed.
- ☐ A minőségi forgatókönyv megadja az ingert, környezetet, választ és mérőszámot.
- ☐ A mért és várt eredményt külön szakaszban dokumentáltad.

6.1.14 Visszaállítási út

Mivel a labor dokumentációs feladat, éles rendszer nem érintett. A dokumentum verziózott a git repositoryban, ezért bármely korábbi állapot visszaállítható a `git checkout` paranccsal. A döntési napló bejegyzései append-only módon bővülnek, így a korábbi döntések nem íródnak felül.

6.1.15 Önellenőrző kérdések

1. Mi a különbség a nézet és a nézőpont között? Adj példát mindkettőre a 4+1 modellből.
2. Melyik nézet mutatja a konkurenciát, a szálakat és a futásidejű üzenetváltásokat?
3. Hogyan biztosítod a nézetek közötti konzisztenciát? Nevezz meg legalább három konkrét gyakorlatot.
4. Miért nevezzük a forgatókönyv-nézetet az „+1”-nek a 4+1 modellben? Mi a szerepe?
5. Egy logikai nézetben szereplő komponens megjelenik a fejlesztési és fizikai nézetben is. Mit jelent ebben az esetben a nyomon követhetőség?

6.1.16 Összefoglalás

A 4+1 nézetmodell egy gyakorlati keret, amely a szoftverarchitektúra dokumentálását öt, jól elkülönített nézetre bontja: logikai, folyamat-, fejlesztési, fizikai és forgatókönyv-nézet. Mindegyik nézet más-más érdekelthez szól, és más kérdésre ad választ. A modell akkor működik, ha a nézetek konzisztensek, az elemek elnevezése következetes, és a forgatókönyvek végigkövethetők a többi nézeten. A nézetek közötti nyomon követhetőség az az alap, amelyre a következő egységben a C4 modell és az arc42 dokumentációs keret épít.

6.2 C4 modell és arc42

A C4 modell és az arc42 együtt a gyakorlati architektúradokumentáció két pillére: az egyik a kommunikációhoz szükséges nézeteket rendezi hierarchikus szintekbe, a másik pedig a teljes dokumentációs csomag szerkezetét adja. Ez az alfejezet a fogalmak bemutatása után egy kis, ellenőrizhető példán keresztül vezeti végig az olvasót, majd a laborban a saját dokumentum elkészítésének lépéseit gyakoroltatja. A cél nem az eszközök katalógusszerű ismertetése, hanem az a képesség, hogy az olvasó egy valós tervezési szituációban kiválassza a megfelelő részletességet, és a dokumentációt a döntések nyomon követésére használja.

6.2.1 A dokumentáció célja és célközönsége

Mielőtt diagramot rajzolunk vagy szerkezetet választunk, rögzíteni kell, kinek és milyen döntési helyzetben készül a dokumentáció. A szoftverarchitektúra dokumentációja nem önmagáért létezik: a célja, hogy a különböző stakeholder csoportok – fejlesztők, üzemeltetők, termékmenedzserek, biztonsági auditorok, új belépők – a rendszer felépítéséről, korlátairól és a mögöttes döntésekről közös mentális modellt kapjanak.

A célközönség meghatározza a szükséges absztrakciós szintet. A termékmenedzser számára a rendszerhatár és a külső függőségek láthatók; a fejlesztő számára a komponens belső felépítése is fontos; az üzemeltető számára a futtató környezet és az adatfolyamok; a biztonsági auditor számára a bizalmi határok és a hozzáférések. A C4 modell ezt a rétegzettséget formálja keretbe, az arc42 pedig garantálja, hogy a dokumentum ne csak szép ábrákat, hanem a döntések indoklását, a minőségi követelményeket és a kockázatokat is tartalmazza.

6.2.2 A C4 modell rétegei

A C4 modell négy absztrakciós szintet különböztet meg: rendszerkontextus (System Context), konténer (Container), komponens (Component) és kód (Code). A nevek mögött nem technológiai előírás áll, hanem az a felismerés, hogy különböző kérdésekre különböző részletezettségű nézetek adnak megfelelő választ. A modell szerzője, Simon Brown azzal a céllal alkotta, hogy az UML-diagramok bonyolultságát elkerülve, a szoftverrendszerek statikus szerkezetét jól olvasható, konzisztens módon lehessen ábrázolni.

6.2.3 Rendszerkontextus szint (System Context)

A legfelső szint a rendszer egészét egyetlen dobozként jeleníti meg, körülötte a felhasználói szerepkörökkel és a külső rendszerekkel. A kontextusdiagram nem mutat technológiai részleteket: itt az a fontos, hogy mi a vizsgált rendszer, kik vagy mik lépnek vele kapcsolatba, és milyen irányú a kapcsolat. A rendszerhatár ezen a szinten egyértelműen jelölendő.

A kontextusdiagram célja, hogy a nem technikai stakeholder is megértse, milyen környezetben üzemel a szoftver. Például egy „Rendeléskezelő rendszer” esetében a diagramon megjelenik a rendszer, a vásárló szerepkör, az ügyfélszolgálati munkatárs, valamint a külső fizetési szolgáltató és a számlázórendszer. A kapcsolatokon fel kell tüntetni a kommunikáció irányát, és – amennyiben ismert – a használt protokollt vagy csatorna típusát. A hibakezelési stratégia itt még általában nem részletezendő; a lényeg az, hogy a külső függőségek és az emberi szereplők látszanak.

A kontextusdiagram gyakori hibája, hogy túl sok szereplőt és rendszert zsúfolnak rá, így az ábra áttekinthetetlenné válik. Ha egy rendszer tucatnyi külső szolgáltatással kommunikál, érdemes a kapcsolódó rendszereket logikai csoportokba vonni, vagy a diagramon csak az elsődleges

interakciókat szerepeltetni, a többit pedig a kísérő szövegben felsorolni. A másik gyakori hiba, hogy a kontextusdiagramra belső komponensek kerülnek; ez a rétegek összemosása, és elrontja a célközönségnek szánt áttekinthetőséget.

6.2.4 Konténer szint (Container)

A konténer szint a rendszert alkotó futtatható vagy üzembe helyezhető egységekre bontja: webalkalmazás, mobilalkalmazás, API-szolgáltatás, adatbázis, üzenetsor, fájlrendszer, sőt akár egy serverless funkció is konténernek számít a C4 terminológiában. Fontos: a „konténer" itt nem a Docker-konténert jelenti elsősorban, hanem bármely olyan egységet, amely önállóan futtatható és saját technológiai veremmel rendelkezik.

A konténer diagramon belül a rendszerhatáron belüli konténerek láthatók, és a külső rendszerek, szereplők ismét megjelenhetnek, de most már a belső felépítés is szerepel. A kapcsolatokon itt már konkrétabb technológiákat és protokollokat kell jelölni: HTTP/JSON, gRPC, AMQP, JDBC, sftp stb. A szinkronitás és a hibatűrés is jelezhető: például „szinkron REST hívás, timeout 3 s, retry 2 alkalommal" vagy „aszinkron üzenet, legalább egyszer feldolgozás, dead-letter queue".

A konténer szint a leggyakoribb belépési pont a technikai stakeholder számára. Itt derül ki, milyen szolgáltatások alkotják a rendszert, milyen adatbázist használnak, és hogyan kommunikálnak egymással. A diagramból ki kell derülnie a felelősségi határoknak: melyik konténer felel a felhasználói hitelesítésért, melyik a rendelések feldolgozásáért, és melyik tárolja a törzsadatokat. A könyv konvenciója szerint minden komponenst (itt konténert) felelősséggel és interfésszel együtt kell leírni. Például: „Rendelés API: felelős a rendelések létrehozásáért és lekérdezéséért; REST interfész, OpenAPI szerződéssel; a nem kezelt hibák esetén 4xx/5xx választ ad, és nem befolyásolja a többi szolgáltatást."

A konténer diagram hibája lehet, ha túlzottan részletezi a belső osztályokat, vagy ha a konténerek közötti kapcsolatokat vonalak tömkelegeként jeleníti meg. A jó diagram 3–8 konténert mutat; ha ennél több van, érdemes alrendszerekre bontani, vagy külön diagramokat készíteni a funkcionális területeknek.

6.2.5 Komponens szint (Component)

A komponens szint egy konténer belső felépítését mutatja: milyen fő komponensekből áll, milyen interfészeket kínál, és hogyan függenek egymástól. Egy konténer tipikusan több komponensből állhat: például egy Spring Boot alapú rendelés-szolgáltatásban a „RendelésController", a „RendelésService", a „RendelésRepository" és az „EseményKüldő" komponensek különíthetők el. A C4 modell szerint a komponens a legkisebb olyan egység, amely önálló felelősséggel és interfésszel rendelkezik, de nem feltétlenül külön futtatható.

A komponens diagramon a belső komponensek és a konténer határán átnyúló kapcsolatok jelennek meg. A kapcsolatokon itt is a technikai részletek szerepelnek: hívási irány, szinkron/aszinkron jelleg, hibakezelés. A komponens szint már kifejezetten a fejlesztőcsapatnak szól; itt a cél a kódban való eligazodás megkönnyítése, nem a futtatási környezet ábrázolása. A komponensdiagramnak nem célja minden osztály felsorolása; az osztályszintű részletek a kódszintre tartoznak.

Gyakori hiba, hogy a komponensdiagramot az UML osztálydiagrammal keverik, és minden interfészt, absztrakt osztályt feltüntetnek. A C4 komponens szintje a felelősségek szintjén marad: „RendelésService: validálja a rendelést, kiszámolja a végösszeget, és eseményt küld a fizetési szolgáltatásnak". Nem kell a metódusok szignatúráját ábrázolni, hacsak az nem hordoz architekturális döntést.

6.2.6 Kódszint (Code)

A kódszint a C4 modell legalacsonyabb rétege: osztálydiagramok, szekvenciadiagramok, entitáskapcsolati ábrák, függvényhívási gráfok tartoznak ide. Ez a szint a kódból közvetlenül generálható, és csak akkor érdemes kézzel karbantartani, ha az architektúra szempontjából kulcsfontosságú részletet kell kiemelni. A könyv dokumentációs szerződése szerint a kódszint csak indokolt esetben készül; ilyen például egy bonyolult öröklődési hierarchia, egy rekurzív adatszerkezet vagy egy szabálymotor belső felépítése, amely a komponens szinten nem érthető.

A kódszintű diagramokat érdemes a forráskódból automatikusan előállítani, hogy ne váljanak elavulttá. A diagrams-as-code eszközök itt is alkalmazhatók: Structurizr DSL, PlantUML vagy Mermaid segítségével a diagram a kódbázisban verziózható. A kézzel rajzolt, statikus kóddiagramok gyorsan elavulnak, és a dokumentáció hitelességét rontják.

6.2.7 A C4 rétegek közötti átjárás és a „zoom” elve

A C4 modell lényege a fokozatos nagyítás: a kontextusdiagramról a konténerre, a konténerről a komponensre, majd a komponensről a kódra lehet lépni. Minden szint ugyanazt a rendszert mutatja, csak egyre több részlettel. A nézetek közötti navigációnak konzisztensnek kell lennie: a konténer diagramon ugyanazok a külső rendszerek és szereplők jelennek meg, mint a kontextusdiagramon, csak a belső konténerek is hozzáadódnak. Ha a rétegek közötti elemek neve vagy felelőssége eltér, az olvasó elveszti a fonalat.

A zoomolás gyakorlati eszköze a hivatkozás: minden diagramon szerepelnie kell annak, hogy melyik magasabb szintű elemet bontja ki. Például a konténer diagram címe: „Rendeléskezelő rendszer – Konténer szint”. A diagramok közötti navigációt a dokumentációs eszközben is biztosítani kell: a HTML- vagy PDF-változatban hivatkozások vezessenek a szintek között.

A C4 modell nem írja elő, hogy mind a négy szintet minden rendszerhez el kell készíteni. A könyv szerint a context, container és component szint kötelező, a code szint csak indokolt esetben. A cél az, hogy a szükséges minimális dokumentáció készüljön el, nem az, hogy a diagramok számát maximalizáljuk.

6.2.8 A C4 diagramok elkészítésének gyakorlati szabályai

A diagrams-as-code megközelítés részeként a C4 diagramokat érdemes verziózható szöveges formátumban készíteni. Ebben az alfejezetben Mermaid vagy PlantUML példákat használunk, de a választott eszköz nem kötelező. A lényeg az, hogy a diagram forrása a kódbázisban legyen, és a módosítások a hagyományos verziókezelési munkafolyamatban jelenjenek meg.

Minden C4 diagramnak legyen címe, jelmagyarázata, és a kapcsolatokon a szükséges technikai részletek. A rendszerhatárt egyértelműen jelölni kell: a diagramon belül a vizsgált rendszer határa, kívül a külső elemek. A kapcsolatokon a protokoll, a szinkronitás és a hibatűrés szerepeljen, ahogy a könyv konvenciója előírja. Például egy konténer szintű kapcsolat leírása: „HTTP/JSON, szinkron, timeout 2 s, egy újrapróbálkozás”.

A diagramokon az elemek nevét a valós terminológiához igazítsuk, ne a technológiai megvalósítás nevét használjuk, hanem a felelősséget. Például ne „PostgreSQL 16.3” szerepeljen, hanem „Rendelés-adatbázis (PostgreSQL)”, mert a technológiai verzió gyorsan változik, és a diagram célja a szerep megértése. A verziókat a kísérő szövegben vagy a leltárban lehet feltüntetni, ha az a döntés szempontjából fontos.

6.2.9 Az arc42 dokumentációs szerkezete

Az arc42 egy sablon, amely a szoftverarchitektúra dokumentálásához ad egységes szerkezetet. Nem egy konkrét eszköz, hanem egy dokumentum vázlata, amely a gyakorlatban bevált fejezeteket rendezi logikai sorrendbe. A célja, hogy a dokumentáció ne csak ábrákból álljon, hanem a döntések, a korlátok, a megoldási stratégia és a kockázatok is helyet kapjanak.

Az arc42 fő fejezetei a következők:

1. **Bevezetés és célok** – A rendszer rövid leírása, a dokumentum célja, a stakeholder-ek és a minőségi célok. Itt kell megfogalmazni, hogy miért készül a dokumentum, kik az olvasói, és milyen döntésekhez nyújt támogatást. A minőségi célokat quality attribute scenario formában kell rögzíteni, a könyv konvenciója szerint: inger, környezet, válasz, mérőszám.
2. **Korlátok** – Minden olyan előírás, amely befolyásolja az architektúrát: technológiai megkötések, szervezeti szabályok, jogi vagy szabályozási követelmények, költségvetési korlátok, meglévő rendszerek, csapatméret. A korlátok nem „rosszak”, hanem a tervezési tér határait jelölik. Fontos, hogy a korlátokat megkülönböztessük a döntésektől: a korlát kívülről adott, a döntés pedig a csapat választása.
3. **Kontextus** – A rendszerhatár és a külső interfészek leírása. Ez a fejezet a C4 rendszer-kontextus szintjével áll összhangban: itt kell bemutatni a szereplőket, a külső rendszereket és azok kommunikációs kapcsolatait. A szöveges leírás mellett a kontextusdiagram is ide kerül.
4. **Megoldási stratégia** – Azok az alapvető elvek és minták, amelyek az architektúra egészét meghatározzák. Például: rétegezett architektúra, eseményvezérelt megközelítés, moduláris monolit, mikroszolgáltatások, CQRS, clean architecture. A megoldási stratégiának a minőségi célokból kell levezethetőnek lennie; itt kell megindokolni, miért választottuk az adott stílust.
5. **Építőelemek** – A rendszer statikus felépítése: a fő komponensek, azok felelősségei és interfészei. A C4 modell konténer és komponens szintje itt kerül kifejtésre. Az építőelemek leírásánál a könyv konvenciója szerint minden elemet felelősséggel és interfésszel együtt kell megadni, nem elég a nevek felsorolása.
6. **Futásidő** – A rendszer működés közbeni viselkedése: fontos üzleti folyamatok, eseményszekvenciák, futtatási forgatókönyvek. Itt szekvenciadiagramok, folyamatábrák vagy más viselkedési nézetek használhatók. A futásidő fejezet a „hogyan működik együtt” kérdésre ad választ, szemben az építőelemek statikus képével.
7. **Üzembehelyezés** – A rendszer futtató környezete: hardver, konténer-orchestráció, hálózati topológia, környezetek, telepítési csomagok. Itt a cél az üzemeltetői stakeholder támogatása. A diagrams-as-code itt is használható, például egy Kubernetes-névtér diagram vagy hálózati topológia.
8. **Keresztmetszeti fogalmak** – Olyan témák, amelyek több építőelemet érintenek: naplózás, hibakezelés, biztonság, tranzakciókezelés, konzisztencia, mérés, konfiguráció. Itt kell leírni a közös elveket és megvalósításokat, hogy ne kelljen minden komponensnél ismételni.
9. **Architektúradöntések** – A fontos döntések listája vagy hivatkozása ADR-ekre. A könyv ADR-formátumot ír elő: kontextus, döntés, alternatívák, következmények, állapot. Az arc42 ezt a fejezetet gyűjtőként kezeli; a részletes ADR-ek lehetnek külön fájlban vagy függelékben. A döntéseknek visszakereshetőnek kell lenniük a dokumentum többi részéből, például a megoldási stratégia hivatkozza az ADR-számot.

10. **Minőségi követelmények** – Azok a nem funkcionális követelmények, amelyeket az architektúrának teljesítenie kell. Itt a quality attribute scenario-k, SLO-k és mérési tervek kapnak helyet. A fejezet kapcsolódik a SLO, SLI, error budget és mérés (SLO, SLI, error budget) témához, de ebben az alfejezetben csak a dokumentálás helyét és formáját mutatjuk be.
11. **Kockázatok** – Az ismert architekturális kockázatok és azok csökkentésére tett intézkedések. A kockázatok dokumentálása nem a hibák eltitkolása, hanem a tudatos döntéshozatal része: ha egy kockázatot feljegyzünk, a stakeholder láthatja, hogy a csapat ismeri és kezeli.
12. **Függelék** – Opcionális további anyagok: teljes diagramok, API-szerződések, konfigurációs példák, telepítési jegyzőkönyvek.

Az arc42 nem írja elő, hogy minden fejezetet minden projektben töltsünk meg. A dokumentáció mélysége a rendszer méretétől, a csapat érettségétől és a döntések kockázatától függ. A cél az, hogy a dokumentum olvasható, naprakész és a döntésekhez visszavezethető legyen.

6.2.10 A C4 és az arc42 együttes használata

A két eszköz kiegészíti egymást: a C4 a nézetek szintjeit adja, az arc42 pedig a dokumentum szerkezetét. A gyakorlatban az arc42 fejezeteibe illesztjük a C4 diagramokat a megfelelő absztrakciós szinten:

- A **Kontextus** fejezetbe a C4 rendszerkontextus diagram.
- Az **Építőelemek** fejezetbe a C4 konténer és komponens diagramok, a hozzájuk tartozó felelősség- és interfész-leírásokkal.
- A **Futásidő** fejezetbe szekvenciadiagramok, amelyek a fontos forgatókönyveket mutatják be; ezek a komponens szintű elemek közötti interakciókat ábrázolják.
- Az **Üzembehelyezés** fejezetbe az infrastruktúra diagramok, amelyek a konténerek futtatási környezetét mutatják, és nem keverendők a fejlesztői nézetekkel.

A kódszintű C4 diagramok jellemzően a Függelékbe vagy a fejlesztői dokumentáció külön részébe kerülnek, mert gyorsan elavulnak, és csak akkor szükségesek, ha a generálásuk automatizált.

A dokumentáció készítésekor fontos, hogy a diagramok és a szöveg ugyanazt a terminológiát használja. Ha a szövegben „Rendelés API”-t írunk, a diagramon is pontosan ez a név szerepeljen, ne „OrderService” vagy „rendelés_backend”. A konzisztens elnevezés csökkenti a kétértelműséget, és megkönnyíti a döntésekhez való hivatkozást.

6.2.11 Példa: Rendeléskezelő rendszer dokumentációja

A továbbiakban egy fiktív, de reális példán mutatjuk be a C4 és arc42 használatát. A példa a könyv laborjának alapjául is szolgál. A rendszer neve: „Rendeléskezelő Rendszer – DEMO”, amely egy belső kiskereskedelmi vállalat számára kezeli a vevői rendeléseket. A példában nincs valós ügyféladat, minden név és azonosító helyőrző.

6.2.12 A C4 diagramok Mermaid formátumban

Az alábbi Mermaid kód a rendszerkontextus diagramot írja le. A diagramon a rendszerhatárt a `Enterprise` osztály jelöli, a külső szereplők és rendszerek a határon kívül helyezkednek el.

```
flowchart TD
    subgraph "Rendeléskezelő Rendszer DEMO"
        R[["Rendeléskezelő Rendszer\n(web + API + adatbázis)"]]
    end

    V[[Vásárló]]
    U[[Ügyfélszolgálati munkatárs]]
    F[["Külső fizetési szolgáltató\n(PSP)"]]
    S[["Számlázó rendszer"]]

    V -->|"rendelés leadása\nHTTPS, szinkron"| R
    U -->|"rendelés kezelése\nHTTPS, szinkron"| R
    R -->|"fizetési kérelem\nHTTPS, szinkron,\ntimeout 5 s, retry 2"| F
    R -->|"számla adatok\nSFTP, aszinkron\nnapi batch"| S
```

A diagram címe: „Rendeléskezelő Rendszer DEMO – Rendszerkontextus”. A jelmagyarázat: a belső rendszer határa, a külső szereplők és rendszerek, valamint a kapcsolatokon a protokoll és a hibakezelés. A feltételjegyzet: a diagram a tervezett to-be állapotot mutatja, nem a jelenlegi termelési rendszert.

A konténer szintű diagram ugyanezt a rendszert bontja fel futtatható egységekre:

```
flowchart TD
    subgraph "Rendeléskezelő Rendszer DEMO"
        WEB[["Webes felület\n(React SPA)"]]
        API[["Rendelés API\n(Spring Boot)"]]
        DB[("Rendelés-adatbázis\n(PostgreSQL)")]]
        MQ[["Üzenetsor\n(RabbitMQ)"]]
        WKR[["Háttérfeldolgozó\n(Rendelésfeldolgozó)"]]
    end

    V[[Vásárló]]
    U[[Ügyfélszolgálati munkatárs]]
    F[["Külső fizetési szolgáltató\n(PSP)"]]
    S[["Számlázó rendszer"]]

    V -->|"HTTPS, szinkron"| WEB
    U -->|"HTTPS, szinkron"| WEB
    WEB -->|"REST/HTTPS, szinkron,\ntimeout 3 s, retry 1"| API
    API -->|"SQL/TCP, szinkron,\nconnection pool"| DB
    API -->|"AMQP, aszinkron,\npublish"| MQ
    MQ -->|"AMQP, aszinkron,\nconsume"| WKR
    WKR -->|"HTTPS, szinkron,\ntimeout 5 s, retry 2"| F
    WKR -->|"SFTP, aszinkron,\nnapi batch"| S
```

A komponens szintű diagram a `Rendelés API` konténer belső felépítését mutatja:

```
flowchart TD
    subgraph "Rendelés API (Spring Boot)"
        CTRL[["RendelésController"]]
        SVC[["RendelésService"]]
        REPO[["RendelésRepository"]]
        EVT[["EseményKüldő"]]
    end

    DB[("Rendelés-adatbázis\n(PostgreSQL)")]
    MQ[["Üzenetsor\n(RabbitMQ)"]]
```

```
CTRL -->|"hívás, szinkron"| SVC
SVC -->|"adatelérés, szinkron"| REPO
REPO -->|"SQL/TCP, szinkron"| DB
SVC -->|"esemény küldés, aszinkron"| EVT
EVT -->|"AMQP, aszinkron"| MQ
```

A komponens leírások a diagram alatt, szövegesen:

- **RendelésController**: felelős a HTTP kérések fogadásáért, a bemeneti adatok validálásáért és a válasz formázásáért. Interfész: REST /api/v1/orders. Hibatűrés: hibás bemenet esetén 400-as válasz; a belső hibákat a service réteg kezeli.
- **RendelésService**: felelős az üzleti logikáért: rendelés létrehozás, végösszeg számítás, esemény küldés. Interfész: Java interfész, metódusok `createOrder`, `getOrder`. Hibatűrés: tranzakciókezelés, sikertelen eseménykuldés esetén outbox minta.
- **RendelésRepository**: felelős a rendelés adatok perzisztenciájáért. Interfész: Spring Data repository. Hibatűrés: SQL hiba esetén a tranzakció visszagörgetése.
- **EseményKüldő**: felelős az üzenetek publikálásáért az üzenetsorba. Interfész: AMQP publisher. Hibatűrés: publish hiba esetén az outbox táblában rögzít, retry mechanizmus.

A diagramok létrehozásakor minden kapcsolat rendelkezik a szükséges technikai részlettel, ahogy a könyv előírja: protokoll, szinkronitás, hibatűrés. A feltételjegyzetben szerepel, hogy a diagram a DEMO példát ábrázolja, és nem éles rendszer.

6.2.13 Az arc42 vázlat a DEMO-hoz

A következőkben az arc42 fejezeteinek megfelelő rövid vázlatot adunk, a teljesség igénye nélkül, de a lényeges elemeket kiemelve. A cél, hogy az olvasó lássa, hogyan illeszkednek a C4 diagramok a szöveges dokumentációba.

6.2.13.1 1. Bevezetés és célok

A Rendeléskezelő Rendszer DEMO célja, hogy a vállalat belső folyamatait támogassa a vevői rendelések kezelésében. A dokumentum célja az architektúra áttekintése és a döntések nyomon követése a fejlesztőcsapat és az üzemeltetés számára. A fő minőségi célok:

- **Teljesítmény**: A rendelés létrehozási kérés átlagos válaszideje p95 $<$ 800 ms normál terhelés mellett, az inger egy felhasználói kattintás, a környezet a termelési klaszter, a válasz a visszaigazoló oldal megjelenése. Mérési ablak: naptári hónap.
- **Rendelkezésre állás**: A rendelés API SLO-ja 99,9% havi rendelkezésre állás, az error budget 0,1%, a mérési ablak naptári hónap, a hibatűrés a konténer szintű redundanciából adódik.

6.2.13.2 2. Korlátok

- A technológiai stack: Java 17, Spring Boot, React, PostgreSQL, RabbitMQ. A vállalati irányelvek miatt a felhőszolgáltató használata korlátozott, ezért a DEMO on-premise Kubernetes környezetben fut.
- A csapat mérete: két fejlesztői csapat, ezért a szolgáltatások száma korlátozott.
- A szabályozási korlát: a fizetési adatokat a PSP-n kívül nem tároljuk.

6.2.13.3 3. Kontextus

A kontextusdiagram a fenti Mermaid kódban látható. A szöveges leírás: a Vásárló szerepkör a webes felületen adja le a rendelést; az Ügyfélszolgálati munkatárs jogosult a rendelések módosítására; a Külső fizetési szolgáltató felel a tényleges tranzakcióért; a Számlázó rendszer a belső számlák kiállítását végzi.

6.2.13.4 4. Megoldási stratégia

A rendszer moduláris monolitot alkalmaz, mert a csapat és a funkcionalitás mérete ezt indokolja. A rétegek: webes felület, API, háttérfeldolgozó. Az üzenetsor aszinkron leválasztást biztosít a lassú háttérfolyamatokhoz. Az eseményvezérelt rész a számlázási folyamatot segíti. A döntést az ADR-001 rögzíti.

6.2.13.5 5. Építőelemek

A konténer és komponens diagramok a fentiekben láthatók. Minden konténerhez tartozik felelősség- és interfész-leírás. Például a **Rendelés API** felelős a rendelések CRUD-műveleteiért és az üzleti logikáért; interfésze REST API az OpenAPI 3.0 szerződéssel; hibatűrése: a kérések timeout 3 másodperc, egy újrapróbálkozás a hálózati hibákra, az idempotencia biztosított.

6.2.13.6 6. Futásidő

Fontos forgatókönyv: a rendelés leadása. A vásárló a webes felületen kitölti az űrlapot; a webalkalmazás REST hívást küld a Rendelés API-nak; a service validálja, létrehozza a rendelést az adatbázisban, majd eseményt küld az üzenetsorba; a háttérfeldolgozó fogadja az eseményt, és fizetési kérelmet indít a PSP felé; sikeres fizetés esetén a számlázó rendszer felé számlaadatot továbbít. A szekvenciadiagram ezt a folyamatot mutatja, de a példában a szöveges leírás is elegendő.

6.2.13.7 7. Üzembehelyezés

A DEMO egyetlen Kubernetes klaszterben fut, három névtérrel: `demo-dev`, `demo-staging`, `demo-prod`. A web, API és háttérfeldolgozó konténerek podként futnak; a PostgreSQL egy állami `StatefulSet`ben; a RabbitMQ egy `StatefulSet`ben. A diagram külön készül, a futtatási környezetet ábrázolja, a kapcsolatokon a hálózati policy-k és a service mesh konfigurációk szerepelnek.

6.2.13.8 8. Keresztmetszeti fogalmak

- **Naplózás**: strukturált JSON log minden szolgáltatásban, központi gyűjtőbe.
- **Hibakezelés**: minden szolgáltatás egységes hibakód-készletet használ; a váratlan hibák 500-as választ és riasztást eredményeznek.
- **Biztonság**: minden belső kommunikáció TLS-en; a felhasználói hitelesítés OAuth2 alapú; a titkokat egy vállalati titokkezelő rendszer szolgáltatja.
- **Tranzakciók**: az üzleti logikában lokális tranzakciók; az elosztott konzisztencia az outbox mintával biztosított.

6.2.13.9 9. Architektúradöntések

Az ADR-001: „Moduláris monolit a mikroszolgáltatások helyett” – kontextus: a kezdeti csapatméret és a funkcionalitás korlátozott volta; döntés: egyetlen futtatható API, belső modulokkal; alternatívák: mikroszolgáltatások 6 szolgáltatással, vagy serverless; következmények: egyszerűbb üzemeltetés, kisebb hálózati overhead, de a modulhatárok fegyelmet igényelnek; állapot: elfogadva. Az ADR-eket az Architecture Decision Records (ADR) és döntési napló alfejezetben ismertetett formátumban kell részletesen kifejteni.

6.2.13.10 10. Minőségi követelmények

A fejezet a minőségi forgatókönyveket sorolja fel, quality attribute scenario formában. Például: inger – a felhasználó megnyomja a „Rendelés leadása” gombot; környezet – normál üzleti óra, 200 egyidejű felhasználó; válasz – a rendelés létrejön, és a felhasználó visszaigazolást kap; mérőszám – a válaszidő p95 $<$ 800 ms, a hibák aránya $<$ 0,1% az adott ablakban.

6.2.13.11 11. Kockázatok

- A PSP külső szolgáltatás kiesése: az üzenetsor pufferként viselkedik, de a fizetések késhetnek. Csökkentés: retry és dead-letter queue figyelés.
- A moduláris monolit határainak fellazulása: a csapat növekedése esetén a belső modulok összefonódhatnak. Csökkentés: architekturális fitness function a modulfüggőségek ellenőrzésére.

6.2.13.12 12. Függelék

A teljes Mermaid forráskódok, az OpenAPI szerződés kivonata, a Kubernetes manifestek helye a repóban.

6.2.14 Tipikus hibák és helyes megoldások

A C4 és arc42 használatakor gyakori hibák:

1. **Túl korai alacsony szintű diagram**: sok csapat rögtön osztálydiagramot rajzol, mielőtt a rendszerhatárt és a konténereket tisztázná. Ez a stakeholder számára értelmezhetetlen, és a magas szintű döntések rejtve maradnak. Helyes megoldás: először a kontextus és konténer szintet készítsük el, és csak utána menjünk mélyebbre.
2. **A diagramokon nincs jelmagyarázat és feltételjegyzet**: az olvasó nem tudja, melyik elem mit jelent, és milyen időpontbeli állapotot ábrázol a diagram. A könyv konvenciója szerint minden diagramnak legyen címe, jelmagyarázata és feltételjegyzete.
3. **A kapcsolatokon hiányzik a protokoll és a hibatűrés**: a csupasz vonalak nem adnak elég információt a tervezési döntésekhez. A kapcsolatokon szerepelnie kell a protokollnak, a szinkronitásnak és a hibatűrési mechanizmusnak.
4. **A C4 rétegek keverése**: például a kontextusdiagramon belső komponensek jelennek meg. Ez összezavarja a stakeholder-eket, és a diagram célja elvész.

5. **Az arc42 dokumentum üres váza**: ha a fejezeteket nem töltik ki konkrét tartalommal, a dokumentum csak formalitás. A döntések, korlátok és minőségi forgatókönyvek nélkül az architektúra nem értékelhető.
6. **A dokumentáció elavul**: ha a diagramokat kézzel, nem verziózható formátumban karbantartják, hamar eltérnek a valóságtól. A diagrams-as-code és a CI-ba integrált ellenőrzés segít.

6.2.15 Labor: C4 diagram és arc42 vázlat készítése

A labor célja, hogy az olvasó a fejezetben bemutatott eszközöket egy saját kis példán alkalmazza. A labor architektúra-kataként működik: a kiinduló állapot, az elvárt eredmény, az ellenőrzési szempont és a visszaállítási út előre rögzített.

6.2.16 Kiinduló állapot

A labor egy fiktív, de reális üzleti problémát ad. A résztvevő választhat egyet a megadott három közül, vagy hozhatja a saját, hasonló méretű példáját. A példák:

- **Időpontfoglaló rendszer** egy fodrászszalon-láncnak, ahol a vendégek online foglalhatnak, és a szalonok dolgozói kezelik a naptárat.
- **Könyvtári kölcsönző rendszer** egy városi könyvtárnak, ahol az olvasók kölcsönözhetnek, és a könyvtárosok kezelik a készletet.
- **Számlakezelő szolgáltatás** egy kis könyvelőiroda számára, amely a bejövő számlákat digitalizálja és jóváhagyásra továbbítja.

A laborhoz nincs szükség tényleges kódra; a cél a dokumentáció készítése diagrams-as-code formában. A kiinduló állapot magában foglalja a kiválasztott példa rövid leírását, a fő stakeholderek listáját, és a rendszerrel szemben támasztott két-három minőségi cél megfogalmazását quality attribute scenario formában.

6.2.17 Elvárt eredmény

A résztvevőnek el kell készítenie:

1. **C4 rendszerkontextus diagram** Mermaid vagy PlantUML formátumban, a könyv konvenciói szerint: cím, jelmagyarázat, feltételjegyzet, a rendszerhatár jelölésével, a kapcsolatokon a protokoll és a szinkronitás feltüntetésével.
2. **C4 konténer szintű diagram** ugyanarra a rendszerre, legalább három konténerrel (például webes felület, API, adatbázis). A kapcsolatokon a hibatűrés is szerepeljen.
3. **arc42 vázlat** a 12 fejezetnek megfelelően, de minden fejezetben csak a legszükségesebb tartalommal: a Bevezetésben a célok és a minőségi forgatókönyvek; a Korlátokban legalább két korlát; a Kontextus fejezetben a szöveges leírás és a diagram hivatkozása; a Megoldási stratégia rövid indoklása; az Építőelemekben a konténerek felelősség- és interfész-leírása; a Futásidő fejezetben egy fontos forgatókönyv szöveges leírása; az Üzembehelyezésnél a futtatási környezet vázlata; a Keresztmetszeti fogalmaknál legalább két közös elv; az Architektúradöntések fejezetben legalább egy ADR a formanyomtatvány szerint; a Minőségi követelményeknél a forgatókönyvek listája; a Kockázatoknál legalább egy kockázat és csökkentő intézkedés; a Függelékben a diagramok forráskódja.
4. **Döntési napló** a labor során felmerült döntésekről, az adott ADR-ben rögzítve.
5. **Ellenőrzési szempontok** listája, amelyet a résztvevő a saját munkájára alkalmaz.

A labouról készüljön reprodukálható dokumentum (például Markdown fájl a repóban), amely tartalmazza a diagramok forráskódját, a vázlatot, a döntési naplót és az ellenőrzési szempontokat. A mért és a várt eredményt külön kell választani: a labor leírásában a várt eredmény fent van megadva; a résztvevő a saját munkájában a mért vagy tényleges eredményt (például a diagramok elkészülte, a vázlat kitöltöttsége) jegyzi fel.

6.2.18 Ellenőrzési szempontok

A labor befejezése után a résztvevő ellenőrzi a saját munkáját a következő szempontok szerint:

- A rendszerhatár minden diagramon egyértelmű, és a külső elemek a határon kívül vannak.
- Minden kapcsolaton szerepel a protokoll, a szinkronitás és a hibatűrés (ahol releváns).
- A C4 diagramok cím, jelmagyarázat és feltételjegyzet nélkül nem tekinthetők késznek.
- A konténer leírások tartalmazzák a felelősséget és az interfészt, nem csak a nevet.
- Az arc42 vázlat mind a 12 fejezetet érinti, még ha röviden is.
- A minőségi követelmények quality attribute scenario formában vannak megfogalmazva, inger, környezet, válasz és mérőszám szerint.
- Az ADR tartalmazza a kontextust, döntést, alternatívákat, következményeket és állapotot.
- A dokumentációban nincs valós hitelesítő adat, belső URL vagy személyes adat; a példában minden név helyőrző.
- A diagramok forráskódja a dokumentumban verziózható formában szerepel.

6.2.19 Visszaállítási út

Mivel a labor nem végez destruktív műveletet, a visszaállítás legfeljebb a munkakönyvtár törlését vagy a létrehozott fájlok áthelyezését jelenti. A résztvevő a saját repójában dolgozik; a labor kiinduló állapota bármikor visszaállítható egy új branch létrehozásával. Ha a résztvevő a diagramokat a kód mellé helyezi, ügyeljen arra, hogy a forráskódot a megfelelő dokumentációs mappába tegye, és a commit üzenetben hivatkozzon a labor azonosítójára.

6.2.20 Önellenőrző kérdések

1. Milyen négy absztrakciós szintet különböztet meg a C4 modell, és melyik szinten milyen stakeholder számára adunk információt?
2. Mi a különbség a C4 kontextus és konténer szint között? Milyen hibát követünk el, ha a kontextusdiagramra belső komponenseket teszünk?
3. Az arc42 melyik fejezetében helyezkednek el a minőségi követelmények, és milyen formában kell azokat megadni?
4. Hogyan biztosítja a diagrams-as-code a dokumentáció naprakészségét, és miért fontos a kapcsolatokon a hibatűrés jelölése?
5. Miért nem elegendő az arc42 váz üresen hagyása? Mely fejezetek hiánya okozza a legnagyobb kockázatot a döntéshozatalban?

6.2.21 Továbbvitt eredmények

Ez az alfejezet megalapozza a következő, UML, ER és diagrams-as-code egységet, amely az UML, ER és diagrams-as-code eszközöket részletezi. A C4 modellben szerzett szemlélet – a fokozatos részletezés és a célközönséghez igazított nézet – közvetlenül alkalmazható lesz a UML és egyéb diagramtípusok kiválasztásakor. Az arc42 szerkezete pedig a későbbi fejezetekben a minőségi

forgatókönyvek, az ADR-ek és a review-folyamatok keretéül szolgál. A labor során létrehozott vázlat a záróprojekt (Követelmények, kényszerek és minőségi forgatókönyvek–Architektúra-review, kockázat és bemutató) dokumentációjának kiindulópontja is lehet.

6.3 UML, ER és diagrams-as-code

A szoftverarchitektúra dokumentálásában a modellezési nyelvek és a verziózható diagramok két, egymást kiegészítő eszközt adnak. Az előző fejezetben megismert C4 modell segít a rendszerhatárok és az absztrakciós szintek meghatározásában; ebben a fejezetben arra fókuszálunk, hogyan lehet a C4 által kijelölt nézeteket formálisabb modellezési nyelvekkel – elsősorban UML-lel és ER-diagramokkal – kifejezni, és hogyan tehetjük ezeket a diagramokat a kódbázis részévé a diagrams-as-code gyakorlattal.

A modellezési nyelv olyan jelölésrendszer, amelynek szabályai meghatározzák, milyen elemekből és kapcsolatokból épülhet fel egy modell. A modell a valós rendszer egy szeletének absztrakciója; a diagram ennek a modellnek egy adott nézetét jeleníti meg. A legelterjedtebb általános modellezési nyelv a Unified Modeling Language (UML), amelyet az Object Management Group (OMG) szabványosít. Az adatmodellezésre az entitás-kapcsolat (ER) modell kínál jól bevált jelölést. A diagrams-as-code megközelítésben a diagramokat nem grafikus szerkesztővel rajzoljuk, hanem szöveges, verziózható formátumban írjuk le, például Mermaid vagy PlantUML szintaxisban; a renderelést pedig eszköz végzi. Ennek előnye, hogy a diagram a forráskóddal együtt tárolható, diffelhető, code review-n ellenőrizhető, és CI-folyamatban automatikusan előállítható.

Ebben a fejezetben először tisztázzuk a modellezési nyelvek és a verziózható diagramok alapelveit, majd áttekintjük a legfontosabb UML diagramtípusokat: az osztály-, szekvencia- és komponensdiagramot. Ezt követően az ER-diagramokat tárgyaljuk, végül bemutatjuk a diagrams-as-code gyakorlati integrációját a kódbázisba és a CI-csővezetékbe. A fejezetet egy laborfeladat zárja, amelyben a tanultakat egy kis, ellenőrizhető példán alkalmazzuk.

6.3.1 1. Modellezési nyelvek és verziózható diagramok

A szoftverarchitektúra dokumentálásának egyik alapfeszültsége: a diagramok gyorsan elavulnak, ha nincsenek szorosan a kódhoz kötve. A hagyományos, grafikus eszközzel készített ábrák gyakran kimaradnak a verziókezelésből, nehezen diffelhetők, és a review során nem látszik, hogy a változtatás ténylegesen mit érint. A diagrams-as-code ezt a problémát úgy oldja meg, hogy a diagram forrása szöveges fájl, amely ugyanabban a repositoryban él, mint a forráskód. Ezáltal a diagram ugyanazokon a minőségbiztosítási lépéseken megy keresztül: verziózás, elágazás, pull request, code review, CI-futtatás.

A verziózható diagramok alapelvei:

1. **Szöveges forrás:** a diagramot ember által olvasható, deklarált szintaxissal írjuk le. Például Mermaidben egy egyszerű folyamatábrát így adunk meg:

```
flowchart LR
    A[Rendelés API] -->|HTTPS, szinkron| B[Készletkezelő]
```

2. **Egyértelmű leképezés:** a forrás és a renderelt kép között determinisztikus a kapcsolat; ugyanaz a forrás ugyanazt a képet eredményezi.
3. **Verziózás a kód mellett:** a diagramfájlok a repository részei, így a változások követhetők, és a diagram állapota összeköthető a kód állapotával.

4. **Automatizált előállítás:** a CI-folyamat képes a diagramot renderelni, így a dokumentáció mindig a legfrissebb forrásból készül.
5. **Ellenőrizhetőség:** a szöveges forma lehetővé teszi lintelést, szerkezeti ellenőrzést és a diagramok közötti következetlenségek felderítését.

A modellezési nyelvek szerepe itt az, hogy közös jelölésrendszert adjanak. Az UML például tizennégy diagramtípust definiál, de az architektúra szempontjából a gyakorlatban leginkább az osztály-, szekvencia- és komponensdiagramot használjuk rendszeresen. Az ER-diagram az adatmodell leírására szolgál, és jól illeszkedik az adatarchitektúra fejezethez is. Fontos azonban: a diagramtípus kiválasztása mindig a kérdéshez igazodik. Nem az a cél, hogy minden diagramtípust használjunk, hanem hogy a megfelelő nézetet adjuk a döntéshez.

6.3.2 2. UML diagramtípusok architektúrális használata

6.3.3 2.1. Osztálydiagram

Az osztálydiagram a rendszer statikus szerkezetét mutatja: osztályok, interfészek, attribútumok, metódusok és a köztük lévő kapcsolatok (asszociáció, aggregáció, kompozíció, öröklés, függőség). Architekturális szempontból az osztálydiagram elsősorban a kód szintjén hasznos; a C4 modellben a code szintnek felel meg, amelyet a könyv konvenciói szerint csak indokolt esetben készítünk.

Mikor használjunk osztálydiagramot?

- Összetett tartományi modell (domain model) pontosításakor, például DDD-s bounded contextben.
- Fontos tervezési minták (például Strategy, Repository) szerkezetének dokumentálásakor.
- Amikor egy adott komponens belső kialakítása olyan bonyolult, hogy a kód áttekinthetősége érdekében érdemes formalizálni.

Az osztálydiagram fő elemei:

- **Osztály:** téglalap, három részre osztva: osztálynév, attribútumok, metódusok.
- **Interfész:** az osztályhoz hasonló, de az interfész neve fölött `<<interface>>` sztereotípiával vagy más jelöléssel.
- **Kapcsolatok:** asszociáció (egyszerű vonal), aggregáció (üres rombusz), kompozíció (teli rombusz), öröklés (üres háromszög), függőség (szaggatott nyíl).

Példa Mermaidben:

```
classDiagram
    class Rendelés {
        +UUID id
        +Pénznem összeg
        +Állapot állapot
        +véglegesít()
    }
    class RendelésTétel {
        +Cikkszám cikkszám
        +Mennyiség mennyiség
    }
    Rendelés "1" --> "0..*" RendelésTétel : tartalmaz
```

Az architekturális dokumentációban az osztálydiagramot ritkán érdemes a teljes rendszerre kiterjeszteni; egy ilyen diagram hamar átláthatatlanná és karbantarthatatlanná válik. Ehelyett egy-egy jól körülhatárolt bounded contextre vagy kulcsfontosságú komponensre korlátozzuk.

Gyakori hibák:

- Túl sok osztály és kapcsolat egyetlen diagramon, ami zajossá teszi az ábrát.
- Olyan részletek megjelenítése, amelyek a kódból is könnyen kiolvashatók; az osztálydiagram nem a kód helyettesítője.
- Az attribútumok és metódusok láthatóságának jelöletlenül hagyása, ami félreértésekhez vezet.
- A kapcsolatok kardinalitásának hiánya, ami az üzleti szabályok félreértelmezéséhez vezet.

6.3.4 2.2. Szekvenciadiagram

A szekvenciadiagram a rendszer dinamikus viselkedését mutatja egy adott forgatókönyv mentén: az objektumok vagy komponensek közötti üzenetváltásokat időbeli sorrendben. Ez a diagramtípus kiválóan alkalmas egy-egy fontos interakció, például egy API-hívás, egy elosztott tranzakció vagy egy hibaág dokumentálására.

A szekvenciadiagram fő elemei:

- **Életvonalak:** függőleges szaggatott vonalak, amelyek a résztvevőket reprezentálják.
- **Üzenetek:** vízszintes nyilak az életvonalak között; szinkron üzenet teli nyíl, aszinkron üzenet nyitott nyíl, válasz szaggatott nyíl.
- **Aktiválási sávok:** vékony téglalapok az életvonalon, amelyek az aktív feldolgozási időt mutatják.
- **Kombinált fragmentumok:** feltételes elágazások (alt), ciklusok (loop), párhuzamos feldolgozás (par) jelölésére.

Példa Mermaidben:

```
sequenceDiagram
    participant U as Ügyfél
    participant API as Rendelés API
    participant K as Készletkezelő
    U->>API: POST /rendelések (HTTPS, szinkron)
    API->>K: Foglalás kérése (gRPC, timeout 3s)
    alt készlet elérhető
        K-->>API: Foglalás megerősítve
        API-->>U: 201 Created
    else készlet hiányzik
        K-->>API: Hiba: nincs készlet
        API-->>U: 409 Conflict
    end
```

A szekvenciadiagramok használata különösen ajánlott:

- Külső rendszerintegrációk pontosítására, például fizetési szolgáltatóval való kommunikáció leírására.
- Elosztott tranzakciók, kompenzációs lépések (saga) folyamatának bemutatására.
- Hiba- és újrapróbálkozási forgatókönyvek dokumentálására, ahol a sorrendiség kritikus.

Gyakori hibák:

- A résztvevők neve nem egyértelmű: „Szolgáltatás” helyett „Készletkezelő szolgáltatás” szerepel.

- A szinkron és aszinkron üzenetek jelölése hiányzik vagy következetlen.
- A kombinált fragmentumok használata nélkül próbálják leírni a bonyolult elágazásokat, ami áttekinthetetlenné teszi a diagramot.
- A diagram túl hosszúra nyúlik, mert egyetlen diagramba zsúfolnak több forgatókönyvet; ilyenkor érdemes külön diagramokat készíteni.

6.3.5 2.3. Komponensdiagram

A komponensdiagram a rendszer moduláris felépítését mutatja: komponensek, interfészek és a köztük lévő kapcsolatok, beleértve a függőségeket. Architekturális szempontból ez a diagramtípus áll legközelebb a C4 container és component szintjéhez, de az UML komponensdiagram formálisabb jelölést kínál az interfészek és a szerződések megjelenítésére.

A komponensdiagram fő elemei:

- **Komponens:** téglalap a komponens ikonnal (kis téglalapokkal a bal felső sarokban) vagy egyszerű téglalap `<<component>>` sztereotípiával.
- **Interfész:** a komponens által nyújtott (provided) vagy igényelt (required) interfész, általában „lollipop” (kör) és „socket” (félkör) jelöléssel.
- **Függőségek:** nyilak az interfészek és a komponensek között.

Példa Mermaidben (a Mermaid nem támogatja natívan a lollipop/socket jelölést, de a flowchart vagy componentDiagram használható, itt egyszerűsített jelöléssel):

```
flowchart LR
    subgraph Rendeléskezelő rendszer
        API[Rendelés API]
        KSZ[Készletkezelő]
        SZÁMLA[Számlázó]
    end
    API -->|HTTPS, gRPC, timeout 3s| KSZ
    API -->|HTTPS, gRPC, timeout 5s| SZÁMLA
    KSZ -->|esemény, aszinkron| SZÁMLA
```

Bár ez a példa nem használja az UML szigorú komponensdiagram-jelölését, a Mermaid `flowchart` is képes komponensszintű architektúrát ábrázolni, ha a csomópontok nevei egyértelműek, és a kapcsolatokon feltüntetjük a protokollt, szinkronitást és hibatűrési jellemzőket. Ha szigorúbb UML-jelölésre van szükség, PlantUML-ben a `component` diagram támogatja a lollipop/socket jelölést:

```
@startuml
component "Rendelés API" as API
component "Készletkezelő" as KSZ
interface "Készletfoglalás" as KF
API ..> KF : igényli
KSZ ..> KF : nyújtja
@enduml
```

A komponensdiagramok használata különösen hasznos, ha:

- Egy adott alrendszer belső komponenseit és azok interfészeit akarjuk precízen meghatározni.
- A C4 component szintnél részletesebb, formálisabb szerződéseket kell dokumentálni.
- A komponensek közötti függőségeket és a körkörös függőségeket akarjuk feltárni.

Gyakori hibák:

- A komponenseknek nincs felelőssége vagy interfésze, csak neve; ezáltal a diagram nem ad többet egy dobozokból álló ábránál.
- A kapcsolatokon nem szerepel protokoll, szinkronitás vagy hibatűrés; ez sérti a dokumentációs szerződést.
- A diagramon nem jelölik a rendszerhatárt, így nem derül ki, mi tartozik a rendszerhez és mi külső.
- A komponensdiagramot keverik az osztálydiagrammal: a komponens nem osztály, hanem nagyobb szemcséjű, telepíthető egység.

6.3.6 3. ER-diagram az adatarchitektúrában

Az entitás-kapcsolat (ER) diagram az adatmodell grafikus ábrázolása. Megmutatja az entitásokat (például táblákat vagy fogalmakat), azok attribútumait, valamint az entitások közötti kapcsolatokat és azok kardinalitását. Az ER-diagram az adatarchitektúra egyik alapvető eszköze, és jól kiegészíti a C4 container szintű adatbázis-nézetét.

Az ER-diagram fő elemei:

- **Entitás:** téglalap, benne az entitás neve; az attribútumok külön felsorolhatók.
- **Attribútum:** az entitás tulajdonsága; elsődleges kulcs (PK) és idegen kulcs (FK) jelöléssel.
- **Kapcsolat:** vonal az entitások között; a kapcsolat típusát (egy-egy, egy-több, több-több) a végpontokon lévő jelölés (például 1, 0.., *1..*) mutatja.

Példa Mermaidben:

```
erDiagram
    RENDELES {
        UUID id PK
        DATUM datum
        PENZNEM osszeg
    }
    RENDELES_TETEL {
        UUID id PK
        UUID rendeles_id FK
        CIKKSZAM cikkszam
        MENNYISEG mennyiseg
    }
    RENDELES ||--o{ RENDELES_TETEL : tartalmaz
```

Az ER-diagramok használata az architektúrában:

- Az adatmodell pontosítása a tervezési fázisban, különösen akkor, ha a rendszer több szolgáltatáson osztozik.
- A perzisztencia-réteg és az adatbázis-séma dokumentálása a fejlesztők és az adatbázis-adminisztrátorok számára.
- Az adatkonzisztencia és a tranzakciós határok elemzése, például annak megmutatása, mely entitások tartoznak egy aggregate-hez.

Gyakori hibák:

- A kardinalitások hiánya vagy pontatlan jelölése, ami az üzleti szabályok téves értelmezéséhez vezet.
- Az attribútumok túlrészletezése: az ER-diagramnak nem kell minden oszlopot tartalmaznia; elég a kulcsokat és az architekturálisan fontos mezőket feltüntetni.
- Az entitások közötti kapcsolatok nem egyértelműek, például a „van” szó semmitmondó; helyette „tartalmaz”, „hivatkozik”, „tulajdonol” igéket használjunk.

- A diagram nem tükrözi a normalizálási szintet vagy a szándékos denormalizációt; ezt a döntést dokumentálni kell.

6.3.7 4. Diagrams-as-code a kódbázisban és a CI-ben

A diagrams-as-code gyakorlati bevezetése nem csupán a diagramok szöveges formátumú leírását jelenti, hanem a teljes munkafolyamat átalakítását is. A cél, hogy a diagramok ugyanolyan minőségbiztosítási lépéseken menjenek keresztül, mint a forráskód.

6.3.8 4.1. Eszközválasztás

A három legelterjedtebb eszköz:

- **Mermaid:** JavaScript-alapú, egyszerű szintaxis, natív támogatás GitHub/GitLab markdownban. Jó választás, ha a diagramokat közvetlenül a dokumentációban akarjuk megjeleníteni, és nem igényel bonyolult UML-jelölést.
- **PlantUML:** Java-alapú, széleskörű UML-támogatással, beleértve a szekvencia-, osztály-, komponens- és ER-diagramokat. Jó választás, ha szigorúbb UML-jelölést vagy összetettebb diagramokat akarunk.
- **Structurizr DSL:** kifejezetten a C4 modellhez tervezett, deklaratív nyelv, amelyből C4 context, container, component és deployment diagramok generálhatók. Jó választás, ha a C4 a fő dokumentációs keret, és a diagramokat a C4 szintek szerint akarjuk kezelni.

A választás nem kizárólagos; gyakran egy repositoryban több eszközt is használunk, de fontos a konvenciók rögzítése, hogy a fejlesztők tudják, melyik diagramtípust mivel készítsék.

6.3.9 4.2. Mappastruktúra és konvenciók

Ajánlott mappastruktúra a kódbázisban:

```
docs/
  diagrams/
    c4/
      context/
      container/
      component/
    uml/
      sequence/
      component/
    er/
    adr/
```

Minden diagramfájlhoz tartozzon egy rövid leírás, például a fájl fejléce vagy egy kísérő markdown, amelyben feltüntetjük:

- a diagram célját,
- a használt diagramtípust,
- a kapcsolódó ADR-eket,
- a diagramon szereplő komponensek felelősségét és interfészeit.

Példa egy Mermaid komponensdiagram kísérő szövegére:

Rendeléskezelő rendszer – container szint A diagram a rendeléskezelő rendszer konténereit és azok kapcsolatait mutatja. A rendszerhatár a szaggatott vonallal jelölt terület. A Rendelés API felelős a HTTP API fogadásáért és a rendelési folyamat vezérléséért; a Készletkezelő a készletfoglalásért; a Számlázó a számlakibocsátásért. A kapcsolatokon a protokoll (HTTPS, gRPC), a szinkronitás (szinkron, aszinkron) és a hibatűrési jellemző (timeout, retry) látható.

6.3.10 4.3. CI-integráció

A CI-csővezetékben a diagrams-as-code fájlokat ugyanúgy kezeljük, mint a forráskódot. Tipikus lépések:

1. **Lintelés:** a diagramfájlok szintaktikai ellenőrzése. PlantUML esetében a PlantUML linter, Mermaid esetében például a mermaid-cli `--lint` kapcsolója használható.
2. **Renderelés:** a diagramokból kép (PNG, SVG) vagy PDF generálása, amelyet a dokumentációban felhasználunk.
3. **Diff-ellenőrzés:** a review során a generált képek nem feltétlenül diffelhetők, ezért a forrásfájlok változásait nézzük; a generált képeket csak akkor tároljuk a repositoryban, ha a dokumentáció azt igényli, de a forrás mindig az elsődleges.
4. **Publikálás:** a diagramok beágyazása a fejlesztői portálra, a wiki-oldalra vagy a statikus dokumentációs oldalra.
5. **Architekturális illesztés-ellenőrzés:** haladó gyakorlat, amikor a diagrammodellt összevetik a kódból kinyert függőségi gráffal, és riasztanak, ha eltérés van.

Példa CI-lépés (GitHub Actions pszeudokód):

```
- name: Render diagrams
  run: |
    npx @mermaid-js/mermaid-cli -i docs/diagrams/**/*.mmd -o docs/generated/
    if [ -f docs/diagrams/lint-results.txt ]; then cat docs/diagrams/lint-results.txt; exit 1;
↪   fi
```

Ez a példa szemléltető; a tényleges parancsok a választott eszköztől függenek, ezért a bevezetés előtt ellenőrizni kell a hivatalos dokumentációt.

6.3.11 4.4. A diagramok és az ADR-ek kapcsolata

Minden architektúradöntést ADR-ben rögzítünk, és a diagramoknak visszakereshető hivatkozással kell kapcsolódniuk a döntésekhez. Például ha egy diagramon szerepel egy aszinkron üzenetküldő bevezetése, akkor a diagram kísérő szövegében hivatkozunk az ADR-007-re, amelyben rögzítettük a döntést, az alternatívákat és a következményeket. Ezáltal a diagram nem csupán illusztráció, hanem a döntési lánc része.

6.3.12 5. Gyakori hibák és ellenőrzési szempontok

A modellezési nyelvek és a diagrams-as-code használata során a következő tipikus hibák fordulnak elő:

1. **A diagram nincs verziózva:** a diagram csak egy képernyőkép vagy egy grafikus szerkesztőben készült fájl, amely nem része a repositorynak. Megoldás: diagrams-as-code bevezetése.

2. **A diagram elavult:** a kód változott, de a diagram nem követte. Megoldás: a CI-ben futtatott renderelés és a review során a diagramok ellenőrzése.
3. **Hiányzó rendszerhatár:** a diagramon nem látszik, mi belső és mi külső. Megoldás: minden C4 szintű diagramon szaggatott vonallal jelöljük a rendszerhatárt, és a külső rendszereket, felhasználói szerepköröket feltüntetjük.
4. **Hiányzó felelősség és interfész:** a komponenseket csak névvel jelöljük. Megoldás: minden komponenshez adjunk meg felelősséget és interfészt, akár a diagramon, akár a kísérő szövegben.
5. **Nem jelölt protokoll, szinkronitás, hibatűrés:** a kapcsolatokon csak vonal szerepel. Megoldás: a kapcsolatokon tüntessük fel a protokollt (HTTPS, gRPC, AMQP), a szinkronitást (szinkron, aszinkron) és a hibatűrési jellemzőt (timeout, retry, circuit breaker).
6. **Túl részletezett diagram:** egy diagramon minden osztály vagy minden mező szerepel. Megoldás: a C4 szinteknek megfelelően válasszuk meg a részletességet; a code szintű diagramokat csak indokolt esetben készítsünk.
7. **Következetlen jelölés:** ugyanaz a fogalom más-más szimbólummal szerepel a különböző diagramokon. Megoldás: központi jelölési konvenció és sablonok használata.
8. **A diagram nem kapcsolódik ADR-hez:** a döntés nincs nyomon követve. Megoldás: a diagram kísérő szövegében hivatkozzunk a releváns ADR-re.

Ezeknek a hibáknak a felismerése és elkerülése a diagrams-as-code és a CI-integráció révén nagyrészt automatizálható, de a végső felelősség a csapaté: a review során a diagramokat is át kell nézni.

6.3.13 6. Labor: Mermaid vagy PlantUML diagram készítése

Ez a labor a fejezetben tanultak alkalmazását szolgálja. A cél egy kis, ellenőrizhető diagram elkészítése diagrams-as-code formában, reprodukálható dokumentumban, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal. A mért és a várt eredményt külön kell választani.

6.3.14 6.1. Kiinduló állapot

A laborhoz hozz létre egy új Git-repositoryt, vagy használj egy meglévő, izolált munkakönyvtárat. A repositoryban hozd létre a `docs/diagrams/` mappát a 4.2. szakaszban bemutatott struktúra szerint.

A példarendszer legyen egy egyszerű rendeléskezelő rendszer, amely a következő elemekből áll:

- **Ügyfél** (külső szereplő)
- **Rendelés API** (belső komponens)
- **Készletkezelő szolgáltatás** (belső komponens)
- **Számlázó szolgáltatás** (belső komponens)
- **Fizetési szolgáltató** (külső rendszer)

A rendszerhatár a három belső komponenst foglalja magában.

6.3.15 6.2. Feladat

1. **Válassz eszközt:** döntsd el, hogy Mermaidet vagy PlantUML-t használsz. A döntést rögzítsd egy rövid ADR-ben, amely tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket. Az ADR-t a `docs/adr/` mappába helyezd el.

2. **Készítsd el a diagramot:** rajzold meg a rendeléskezelő rendszer container szintű diagramját. A diagramon jelöld a rendszerhatárt, a külső rendszereket, a komponenseket, azok felelősségét és interfészeit, valamint a kapcsolatokon a protokollt, a szinkronitást és a hibatűrési jellemzőt. A diagramot a választott eszköz szintaxisában írd meg, és mentsd el a `docs/diagrams/c4/container/` mappába.
3. **Készíts kísérő dokumentumot:** a diagram mellé írj egy rövid markdown dokumentumot, amely tartalmazza:
   - a diagram célját,
   - a komponensek felelősségét és interfészeit,
   - a kapcsolódó ADR hivatkozását,
   - a rendereléshez használt parancsot és annak kimenetét.
4. **Mérj és dokumentálj:** mérd meg a diagram renderelési idejét és a generált kép fájlméretét. Rögzítsd a mérési környezetet (operációs rendszer, eszközverzió), a mérés időpontját és a bizonytalanságot (például a mérési idő ± 10 ms). A várt eredményt előre határozd meg: például a renderelési idő kevesebb mint 500 ms, a fájlméret kisebb mint 1 MB. A mért és a várt eredményt külön táblázatban tüntesd fel.
5. **Ellenőrizz:** futtass a választott eszközhöz tartozó lintert vagy szintaxis-ellenőrzőt, és dokumentáld az eredményt. Ha a linter hibát jelez, javítsd a diagramot, és ismételd meg a mérést.

6.3.16 6.3. Elvárt eredmény

- A repository `docs/diagrams/c4/container/` mappájában egy verziózott diagramforrásfájl található.
- A `docs/adr/` mappában egy ADR, amely rögzíti a diagrams-as-code eszköz kiválasztását.
- A kísérő markdown dokumentum tartalmazza a kontextust, a döntési naplót, a mért és várt eredményeket, valamint a linter kimenetét.
- A diagram képe (ha generáltad) nem feltétlenül kell a repositoryban, de a renderelési parancs és a kimenet dokumentálva van.

6.3.17 6.4. Ellenőrzési szempontok

- A diagramon szerepel a rendszerhatár, a külső rendszerek és a belső komponensek.
- Minden komponenshez tartozik felelősség és interfész.
- A kapcsolatokon szerepel protokoll, szinkronitás és hibatűrés.
- A diagramforrás szintaktikailag helyes, és a linter nem jelez hibát.
- A mért és várt eredmények külön vannak választva, és a mérési környezet dokumentált.
- Az ADR tartalmazza a döntés kontextusát, alternatíváit és következményeit.

6.3.18 6.5. Visszaállítási út

Mivel a labor csak dokumentációs fájlokat hoz létre, a visszaállítás a Git repository korábbi állapotára való visszatérést jelenti. A labor végén a hallgató a `git checkout .` paranccsal visszaállíthatja a munkakönyvtárat, vagy egyszerűen törölheti a létrehozott fájlokat. A labor során ne módosíts éles rendszert, és ne használj valós hitelesítő adatot.

6.3.19 7. Ellenőrzőlista és önellenőrző kérdés

Ellenőrzőlista a fejezethez:

- ☐ A diagramok diagrams-as-code formátumban készülnek, és a repository részei.
- ☐ Minden diagramhoz tartozik cél, kontextus és ADR-hivatkozás.
- ☐ A C4 szinteknek megfelelő részletességet alkalmazunk; code szintű UML-t csak indokolt esetben készítünk.
- ☐ A komponensek felelősséggel és interfésszel szerepelnek.
- ☐ A kapcsolatokon feltüntettük a protokollt, a szinkronitást és a hibatűrést.
- ☐ Az ER-diagram kardinalitásai és kapcsolatai egyértelműek.
- ☐ A CI-csővezeték futtatja a diagramok lintelését és renderelését.
- ☐ A diagramok változásai code review-n átesnek.

Önellenőrző kérdés:

Milyen hibát észlelünk, ha egy komponensdiagramon a komponensek csak nevekkel szerepelnek, a kapcsolatokon pedig sem protokoll, sem szinkronitás nincs feltüntetve, és a diagram egy grafikus szerkesztővel készült, nem verziózott fájlban van? Indokold meg, miért sérti ez a dokumentációs szerződést, és hogyan lehet a hibát diagrams-as-code és CI-integrációval javítani.

6.3.20 Összegzés

Ebben a fejezetben megismertük a modellezési nyelvek – elsősorban az UML és az ER – szerepét az architektúra dokumentálásában, valamint a diagrams-as-code gyakorlatát, amely a diagramokat a kódbázis részévé és a CI-folyamat tárgyává teszi. Az osztály-, szekvencia- és komponensdiagramok más-más kérdésre adnak választ: a statikus szerkezetre, a dinamikus viselkedésre és a moduláris felépítésre. Az ER-diagram az adatmodellt pontosítja. A labor során a tanultakat egy kis rendszeren alkalmaztuk, és megtanultuk a mért és várt eredmények elkülönítését is. A következő fejezetben ezeket az eszközöket a stakeholder-kommunikációban és az ADR-ek, RFC-k hatékony használatában fogjuk továbbfejleszteni.

6.4 ADR-ek, RFC-k és stakeholder-kommunikáció

A szoftverarchitektúra akkor válik valódi döntési eszközzé, ha a döntések nem maradnak a tervező fejében, hanem érthető, visszakereshető és vitatható formában jutnak el az érintettekhez. Ez a fejezet azt a kommunikációs réteget vizsgálja, amely az architektúradöntéseket a különböző közönségek – vezetők, fejlesztők, üzemeltetők, biztonsági szakemberek, termékmenedzserek – számára feldolgozhatóvá teszi. A cél nem az, hogy mindenki ugyanazt a dokumentumot olvassa, hanem hogy ugyanazt a döntést a saját felelősségi körének megfelelő részletességgel és nyelvezettel kapja meg. A fejezetben bevezetjük az ADR (Architecture Decision Record) és az RFC (Request for Comments) fogalmát, bemutatjuk a design review folyamatát, és gyakorlati eszközöket adunk a döntés indoklásának, valamint a visszacsatolás kezelésének dokumentálására.

6.4.1 1. Az architektúra kommunikációja különböző közönségeknek

Egy architektúradöntés ritkán egyetlen embert érint. A bevezetett cache-réteg, az API gateway beiktatása vagy a szinkron hívás aszinkron üzenetküldésre cserélése más következményekkel jár a fejlesztői csapat, az üzemeltetés, a biztonsági szervezet és az üzleti vezetés számára.

A kommunikáció alapelve, hogy a közönséghez igazítsuk a hangsúlyt, a részletességet és a formátumot, miközben a tények és a döntés lényege változatlan marad.

6.4.2 1.1 Stakeholder-térkép és kommunikációs igények

Az alábbi táblázat a tipikus stakeholder-csoportokat, az elsődleges kérdéseiket és a javasolt kommunikációs formát mutatja be.

Stakeholder	Elsődleges kérdés	Javasolt formátum	Részletesség
Üzleti vezető / termékmenedzser	Mennyibe kerül, milyen kockázattal jár, mit nyerünk?	Vezetői összefoglaló (1 oldal)	Alacsony, üzleti nyelv
Fejlesztői csapat	Milyen komponensek változnak, milyen interfészek, milyen trade-off?	ADR + technikai design leírás	Magas, technikai részletek
Üzemeltetés / platform	Hogyan telepíthető, monitorozható, skálázható?	ADR + runbook vázlat	Közepes, üzemeltetési fókusz
Biztonsági csapat	Milyen új támadási felület, milyen adatáramlás, megfelelés?	Threat model vázlat + ADR	Magas, biztonsági fókusz
Jogi / megfelelés	Milyen szabályozási kötelezettség, adatkezelés?	Megfelelési összefoglaló	Közepes, jogi nyelv
Külső partner / ügyfél	Milyen interfész változik, van-e hatás a szolgáltatásra?	API változásjegyzék + release note	Alacsony, szerződéses nyelv

A stakeholder-térkép nem statikus: egy döntés előtt érdemes végiggondolni, kinek van döntési jogköre, kinek van vétójoga, kinek van szüksége tájékoztatásra, és kinek kell aktívan részt vennie a tervezésben. A RACI-mátrix (Responsible, Accountable, Consulted, Informed) egyszerű eszköz erre; például egy API gateway bevezetésekor a fejlesztői csapat a Responsible, az architekt a Accountable, a biztonsági csapat és az üzemeltetés a Consulted, az üzleti vezető pedig az Informed szerepben lehet.

6.4.3 1.2 Nézetek és formátumok

A C4 modell szintjei természetes illeszkedést kínálnak a közönségekhez. A context szintű diagram a vezetők és a nem technikai stakeholderek számára mutatja a rendszer környezetét, a felhasználókat és a külső rendszereket. A container szint a fejlesztők és az üzemeltetők közös nyelve: itt látszanak a futtatható egységek, adatbázisok, üzenetsorok és a köztük lévő kapcsolatok. A component szint már a belső szerkezetet tárja fel, és elsősorban a fejlesztői csapatnak szól. Ugyanazt a döntést tehát más-más diagrammal és szöveges magyarázattal érdemes kísérni.

Az arc42 sablon a teljes architektúra-dokumentáció keretét adja, de egy-egy döntés kommunikálására az ADR és az RFC sokkal célzottabb eszköz. Az ADR a meghozott döntés rögzítése, az RFC a döntés előtti vitaanyag, a design review pedig a visszacsatolás strukturált fóruma. A

három eszköz együtt egy folyamatos kommunikációs hurkot alkot: javaslat (RFC), véleményezés (design review), döntés (ADR), majd a döntés következményeinek beépítése a dokumentációba és a kódba.

6.4.4 2. ADR: Architecture Decision Record

Az ADR egy rövid, önálló dokumentum, amely egyetlen architektúradöntést rögzít a kontextusával, az indoklásával és a következményeivel együtt. Nem a teljes architektúra leírása, hanem a döntési napló egy bejegyzése. A cél az, hogy a jövőbeli olvasó – aki lehet akár a csapat új tagja, akár egy auditor – megértse, miért született az adott döntés, milyen alternatívákat vetettek el, és milyen következményekkel kell számolni. Ez a visszakereshetőség a dokumentációs szerződés egyik alappillére.

6.4.5 2.1 Mikor és hogyan írjunk ADR-t?

ADR-t érdemes írni minden olyan döntésről, amely:

- befolyásolja a rendszer szerkezetét, a technológiai választást vagy a kulcsfontosságú minőségi attribútumot (például rendelkezésre állás, teljesítmény, biztonság);
- nehezen visszafordítható, vagy a visszafordítás költsége magas;
- több alternatíva közötti trade-offot igényel;
- a csapat vagy a jövőbeli karbantartó számára nem nyilvánvaló.

Nem kell ADR-t írni triviális döntésekről, például arról, hogy egy osztály nevét hogyan írjuk nagybetűvel. Az ADR-ek száma nem cél, hanem a döntések nyomon követésének eszköze. Egy jó ADR akkor születik, ha a döntés meghozatala után rövid időn belül, amíg a kontextus friss, rögzítjük a lényeget.

Az ADR-t általában a kódbázisban, a `docs/adr` vagy `docs/decisions` mappában tároljuk, Markdown vagy AsciiDoc formátumban. A fájlnév konvenciója lehet sorszámozott, például `0001-use-postgresql-for-order-data.md`, vagy dátum-alapú. A verziókövető rendszer biztosítja a visszakereshetőséget és a változások nyomon követését. Egy ADR állapota lehet: javasolt, elfogadott, elvetett, felváltott, elavult. A felváltott döntéseknél hivatkozni kell az új ADR-re.

6.4.6 2.2 ADR sablon és példa

A dokumentációs szerződésben rögzített minimális ADR-sablon a következő mezőket tartalmazza:

- **Cím:** rövid, egyértelmű megnevezés.
- **Állapot:** javasolt / elfogadott / elvetett / felváltott / elavult.
- **Dátum:** a döntés meghozatalának dátuma.
- **Kontextus:** a döntést kiváltó helyzet, a követelmények, a kényszerek és a releváns minőségi attribútumok. Itt hivatkozhatunk quality attribute scenario-ra vagy mérési adatra.
- **Döntés:** egy mondatban a választott megoldás, majd rövid indoklás.
- **Alternatívák:** a mérlegelt lehetőségek, és hogy miért lettek elvetve. Minden alternatívánál legalább egy mondat az elutasítás okáról.
- **Következmények:** pozitív és negatív hatások, kockázatok, szükséges további lépések (például migráció, dokumentáció-frissítés, tesztelés).
- **Hivatkozások:** kapcsolódó ADR-ek, RFC-k, diagramok, követelmények.

Az alábbi példa egy fiktív ADR-t mutat be. A helyőrzőket szögletes zárójelben tüntetjük fel, és a valós rendszer helyett példa-domaineket használunk.

ADR-0012: Eseményvezérelt értesítési csatorna bevezetése a rendeléskezelésben

- **Állapot:** Elfogadott
- **Dátum:** 2025-04-01

Kontextus

A rendeléskezelő szolgáltatás jelenleg szinkron REST-hívással értesíti a számlázó és a raktárkészlet-kezelő szolgáltatást. Egy friss terhelési teszt szerint a rendeléskezelő p95 válaszideje 800 ms fölé emelkedik, ha a számlázó szolgáltatás lassú (például adatbázis-terhelés miatt). A minőségi forgatókönyv: inger: 100 rendelés/perc érkezik; környezet: normál üzem, egy downstream szolgáltatás p95 válaszideje 1,5 s; válasz: a rendelés-visszaigazolás p95 válaszideje nem haladhatja meg a 400 ms-ot; mérőszám: p95 válaszidő ms-ban mérve 5 perces ablakban. A jelenlegi szinkron lánc ezt nem teljesíti.

Döntés

Az értesítést aszinkron üzenetküldésre cseréljük: a rendeléskezelő szolgáltatás egy `order.created` eseményt publikál a központi üzenetsorba (`rabbitmq.example.internal`), a számlázó és a raktárkészlet-kezelő szolgáltatás pedig feliratkozik erre. A rendelés-visszaigazolás a saját adatbázisba történő mentés után azonnal megtörténik.

Alternatívák

1. **Megtartani a szinkron hívást, de időtúllépést és újrapróbálkozást bevezetni.** Elvetve: a p95 válaszidő továbbra is a lassú downstreamhez kötődik, az újrapróbálkozás pedig duplikált rendeléseket okozhat.
2. **Bevezetni egy API gateway-t, amely puffereli a kéréseket.** Elvetve: a gateway önmagában nem oldja meg a tranzakciós konzisztenciát, és új üzemeltetési terhet jelent.
3. **Kétfázisú commit (2PC) használata.** Elvetve: a szolgáltatások különböző adatbázisokat használnak, a 2PC pedig jelentős rendelkezésre állás-csökkenést okozna.

Következmények

- **Pozitív:** a rendelés-visszaigazolás válaszideje függetlenné válik a downstream szolgáltatásoktól; a rendszer jobban tűri a részleges leállásokat.
- **Negatív:** bevezetjük az esetleges konzisztenciát; a számlázó és a raktárkészlet frissítése késhet, ami kompenzáló logikát igényel (saga minta). Az üzenetsor maga is egy új kritikus komponens, amelyet monitorozni és méretezni kell.
- **További lépések:** az üzenetsor telepítése, a `order.created` esemény séma definiálása, a fogyasztók idempotens feldolgozásának megvalósítása, terhelési teszt frissítése.

Hivatkozások: RFC-0007 (az aszinkron átállás javaslata), ADR-0008 (rendelés-adatmodell), quality scenario QAS-0042.

A példa jól mutatja, hogy az ADR nem helyettesíti a részletes technikai tervet, hanem a döntés lényegét, a trade-offokat és a következményeket rögzíti. A kontextusban hivatkozott quality attribute scenario biztosítja, hogy a döntés mérhető követelményhez kötődik, nem pusztán véleményhez.

6.4.7 2.3 ADR életciklus és verziózás

Egy ADR tipikus életciklusa:

1. **Javaslat:** a döntés előkészítése, gyakran RFC formájában.
2. **Elfogadás:** a döntés meghozatala, az ADR létrehozása `Elfogadott` állapottal.
3. **Felváltás:** ha később új döntés születik, a régi ADR `Felváltott` állapotba kerül, és hivatkozik az újra.
4. **Elavulás:** ha a döntés okafogyottá válik (például a technológia megszűnt), az ADR `Elavult` állapotot kap.

A verziózás a git-történeten keresztül valósul meg, de maga az ADR-sablon is tartalmazhat egy `Verzió` mezőt. A kulcs az, hogy a döntés változása explicit legyen: ne írjuk át csendben a dokumentumot, hanem hivatkozzunk az új ADR-re, és a régit hagyjuk meg a történeti hűség kedvéért.

6.4.8 2.4 Tipikus hibák az ADR-eknél

- **Hiányzó kontextus:** az ADR csak a döntést írja le, de nem derül ki, milyen probléma vezetett ide. A jövőbeli olvasó nem tudja értékelni, hogy a döntés akkoriban helyes volt-e.
- **Nincs alternatíva:** ha az ADR nem sorolja fel az elvetett lehetőségeket, a döntés önkényesnek tűnik, és később újra előkerülhet ugyanaz a vita.
- **Túl hosszú:** az ADR nem regény; ha 10 oldal, akkor valószínűleg a részletes design dokumentum és az ADR összemosódott. A 200–400 szavas terjedelem általában elegendő.
- **Nem frissített állapot:** az ADR elfogadottként szerepel, miközben a csapat már másképp csinálja. Ez hamis biztonságérzetet ad, és aláássa a dokumentáció hitelét.
- **Nincs hivatkozás a mérésre:** ha a döntést teljesítmény vagy költség indokolja, a kontextusban meg kell adni a mérési ablakot és az eredményt. Ellenkező esetben az indoklás anekdotikus marad.

6.4.9 3. RFC és design review

Az RFC (Request for Comments) olyan dokumentum, amely egy jelentősebb architekturális változtatás vagy új komponens javaslatát írja le, és visszacsatolást kér a csapattól, mielőtt a döntés megszületne. Az RFC nem a döntés rögzítése, hanem a vita eszköze. A kifejezés az internetes szabványosítás világából származik, de a szoftverfejlesztésben a belső tervezési folyamatokban is elterjedt.

6.4.10 3.1 Mi az RFC, és mikor használjuk?

RFC-t érdemes írni, ha a változás:

- több csapatot vagy szolgáltatást érint;
- jelentős költséggel, kockázattal vagy visszafordíthatatlan lépéssel jár;
- új technológiát, új mintát vagy új külső függőséget vezet be;
- hosszú távú döntést igényel, amelyet érdemes széles körben véleményeztetni.

Az RFC folyamata jellemzően a következő:

1. **Vázlat:** a szerző megírja a javaslatot, tisztázza a problémát, a lehetséges megoldásokat és a nyitott kérdéseket.

2. **Kommentelés:** a dokumentumot megosztják a releváns stakeholderekkel, akik megjegyzéseket fűznek hozzá. Ez történhet verziókövető rendszerben, pull request formájában, vagy dedikált eszközben.
3. **Vita és finomítás:** a szerző beépíti a visszajelzéseket, szükség esetén új verziót ad ki.
4. **Döntés:** a döntéshozó (általában az architekt vagy egy kijelölt testület) elfogadja, elutasítja vagy további vizsgálatra visszaküldi a javaslatot.
5. **Rögzítés:** ha a javaslat elfogadásra kerül, a döntést ADR-ben rögzítik, és a megvalósítás megkezdődhet.

6.4.11 3.2 RFC sablon és példa

Egy RFC-sablon a következő szakaszokat tartalmazhatja:

- **Cím:** a javaslat rövid neve.
- **Státusz:** vázlat / véleményezés alatt / elfogadott / elutasított.
- **Szerző(k):** név és elérhetőség.
- **Háttér:** miért van szükség a változásra, milyen jelenlegi korlátokat tapasztalunk.
- **Probléma:** a konkrét megoldandó probléma, lehetőség szerint mérhető formában.
- **Javaslat:** a javasolt megoldás leírása, beleértve az érintett komponenseket, az adatáramlást és a főbb interfészeket. Diagramok itt is használhatók (C4, szekvencia).
- **Alternatívák:** milyen más megoldásokat vizsgáltunk, és miért ezt javasoljuk.
- **Kockázatok:** milyen kockázatokat látunk, és hogyan kezelhetők.
- **Nyitott kérdések:** amit még nem tudunk, és amire a visszacsatolásban választ várunk.
- **Ütemterv:** a bevezetés lépései, ha ismertek.

Példa egy RFC vázlatra, ugyanarra a témára, amelyhez a korábbi ADR kapcsolódott:

RFC-0007: Aszinkron értesítés bevezetése a rendeléskezelésben

- **Státusz:** Véleményezés alatt
- **Szerző:** [Név, e-mail]

Háttér

A rendeléskezelő szolgáltatás jelenleg szinkron REST-hívással értesíti a számlázó és a raktárkészlet-kezelő szolgáltatást. A terhelési teszt szerint a p95 válaszidő 800 ms fölé nő, ha a downstream szolgáltatások lassúak. Ez sérti a QAS-0042 minőségi forgatókönyvet.

Probléma

A szinkron lánc miatt a rendelés-visszaigazolás késleltetése a lassú downstream szolgáltatásoktól függ. Cél a p95 válaszidő 400 ms alá csökkentése normál terhelés mellett.

Javaslat

Vezessünk be egy `order.created` eseményt, amelyet a rendeléskezelő szolgáltatás egy üzenetsorba publikál. A számlázó és a raktárkészlet-kezelő szolgáltatás feliratkozik az eseményre, és aszinkron dolgozza fel. A séma legyen verziózott, és tartalmazzon idempotencia-kulcsot.

Alternatívák

1. Szinkron hívás megtartása újrapróbálkozással.
2. API gateway beiktatása pufferelléssel.

3. Kétfázisú commit.

Kockázatok

- Az esetleges konzisztencia miatt a számlázó késhet; kompenzáló logika szükséges.
- Az üzenetsor új kritikus komponens; rendelkezésre állást és monitorozást tervezni kell.
- A fogyasztók idempotens feldolgozása nélkül duplikált számlák jöhetnek létre.

Nyitott kérdések

- Milyen üzenetsor-technológiát válasszunk? (RabbitMQ, Kafka, felhőszolgáltatás?)
- Hogyan kezeljük a séma evolúciót?
- Milyen megőrzési idő legyen az üzeneteknél?

Ütemterv

1. Üzenetsor PoC és terhelési teszt.
2. Séma definiálása.
3. A rendeléskezelő szolgáltatás módosítása.
4. A fogyasztók átállítása.
5. Megfigyelés és finomítás.

Az RFC célja, hogy a döntés előtt összegyűjtse a csapat tudását és a különböző nézőpontokat. A nyitott kérdések explicit jelzése lehetővé teszi, hogy a véleményezők a bizonytalanságra koncentráljanak, ne pedig a már eldöntött részletekre.

6.4.12 3.3 Design review

A design review strukturált találkozó vagy folyamat, amelynek során egy architekturális javaslatot – gyakran RFC-t – a releváns stakeholderek megvizsgálnak. A cél nem a hibakeresés önmagáért, hanem a kockázatok feltárása, a trade-offok tudatosítása és a döntés minőségének javítása. A design review lehet formális vagy informális.

Formái:

- **Informális átbeszélés:** a csapat egy rövid megbeszélésen átnézi a javaslatot; alkalmas kisebb döntésekre.
- **Strukturált review:** előre definiált ellenőrzőlista alapján a résztvevők szisztematikusan vizsgálják a követelményeket, a minőségi attribútumokat, a kockázatokat és a megvalósíthatóságot.
- **ATAM-alapú review:** a fejezet későbbi részében tárgyalt Architecture Tradeoff Analysis Methodhoz hasonló, de egyszerűsített forma; a minőségi forgatókönyvekre fókuszál.
- **Aszinkron review:** a dokumentumot pull requestben vagy megosztott felületen kommentelik; alkalmas elosztott csapatoknak.

Szerepek:

- **Szerző / előterjesztő:** a javaslatot készítő személy vagy csapat; felelős a dokumentumért és a visszajelzések beépítéséért.
- **Reviewer / véleményező:** a stakeholderek képviselői; felelősek a saját területük kockázatainak jelzéséért.

- **Döntéshozó / moderátor:** az a személy vagy testület, aki a vita után döntést hoz; felelős a folyamat betartásáért és a határidőkért.

A design review során az alábbi ellenőrzőlista használható:

1. A probléma pontosan definiált, és mérhető követelményhez kötött?
2. A javaslat illeszkedik a rendszer meglévő architektúrájához és elveihez?
3. Az alternatívák teljes körűen fel vannak sorolva, és az elutasítás indokolt?
4. A minőségi attribútumokra gyakorolt hatás (teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság) elemzett?
5. A kockázatok azonosítottak, és van kezelési terv?
6. A bevezetési terv reális, és tartalmaz visszagörgetési lehetőséget?
7. A dokumentáció frissítésére és a tesztekre vonatkozó következmények tisztázottak?
8. A nyitott kérdések lezárhatók-e a döntés előtt, vagy tudatosan nyitva maradnak?

A review eredménye nem feltétlenül azonnali elfogadás vagy elutasítás; gyakran a javaslat módosításokkal fogadható el, vagy további vizsgálatra (például PoC) van szükség. A visszajelzéseket a döntési naplóban érdemes rögzíteni, hogy később visszakereshető legyen, miért változott a terv.

6.4.13 4. A döntés indoklása és a visszacsatolás kezelése

Az architektúradöntések minősége nem csak a technikai helyességen múlik. Legalább ennyire fontos, hogy a döntés indoklása átlátható legyen, és a visszacsatolás konstruktívan beépüljön a folyamatba. A döntési napló (ADR) és a javaslat (RFC) akkor tölti be szerepét, ha a résztvevők bíznak abban, hogy a véleményük számít, és a döntés nem önkényes.

6.4.14 4.1 Az indoklás szerepe

Egy döntés indoklása három pilléren nyugszik:

1. **Kontextus:** milyen követelmények, kényszerek és korlátok között született a döntés? Ide tartoznak a minőségi forgatókönyvek, a rendelkezésre álló költségvetés, a csapat képességei és a szervezeti irányelvek.
2. **Trade-off:** milyen előnyöket és hátrányokat mérlegeltünk? Nincs tökéletes megoldás; az indoklás akkor hiteles, ha a feladott alternatívákat és a választott megoldás árnyoldalait is bemutatja.
3. **Bizonyíték:** milyen mérés, kísérlet vagy szakirodalmi hivatkozás támasztja alá a döntést? A nem futtatott benchmarkot nem szabad ellenőrzött eredményként feltüntetni; ha nincs mérés, azt explicit jelezni kell.

Az indoklás hiánya tipikus hibaforrás. A „így szoktuk”, „ez a legjobb gyakorlat” vagy „a főnök így döntött” kijelentések nem helyettesítik a kontextusba ágyazott érvelést. A „best practice” állításokat a mögöttes elvvel és a kivételekkel együtt kell megadni; például a mikroszolgáltatások nem minden csapatnak valók, és a rétegezett monolit sok esetben jobb választás.

6.4.15 4.2 A visszacsatolás kezelése

A visszacsatolás a design review és az RFC-kommentelés során érkezik. A hatékony kezeléshez érdemes az alábbi lépéseket követni:

1. **Nyitottság:** a szerző ne védje a javaslatát személyes ügyként; a kritika a tervre vonatkozik, nem az emberre.
2. **Kategorizálás:** a visszajelzéseket érdemes csoportosítani: technikai hiba, kockázat, alternatív javaslat, stílusbeli észrevétel, nyitott kérdés.
3. **Beépítés vagy elutasítás indoklással:** minden visszajelzésre reagálni kell. Ha egy javaslatot elutasítunk, azt indokolni kell; ha beépítjük, a dokumentum új verziójában jelezni kell.
4. **Döntés és rögzítés:** ha a vita lezárult, a döntést egyértelműen rögzíteni kell, és aki nem ért egyet, annak lehetősége van a döntési naplóban rögzíteni az ellenvéleményét. Ez nem akadályozza a haladást, de megőrzi a szervezeti memóriát.

A visszacsatolás kezelésének egyik gyakori hibája a „hallgatás mint beleegyezés" feltételezése. Ha egy RFC-re nem érkezik megjegyzés, az nem feltétlenül jelenti azt, hogy mindenki egyetért; lehet, hogy a stakeholderek nem olvasták el, vagy nem volt idejük reagálni. A folyamatnak explicit határidőt és felelőst kell kijelölnie a véleményezésre.

6.4.16 4.3 Konfliktusok és döntési jogkörök

Az architekturális vitákban gyakori a konfliktus a fejlesztői csapat, az üzemeltetés és a biztonsági csapat között. A döntési jogkörök tisztázása előre csökkenti a bizonytalanságot. A RACI-mátrix mellett hasznos lehet egy egyszerű döntési szabály: például a biztonsági csapatnak vétójoga van a biztonsági architektúrát érintő kérdésekben, de a teljesítmény-optimalizálás a fejlesztői csapat hatásköre. A governance fejezetben tárgyalt döntési keretek itt kapcsolódnak be.

6.4.17 5. Gyakorlati példa: kommunikációs csomag egy döntéshez

Tegyük fel, hogy a csapat úgy dönt, bevezet egy API gateway-t a meglévő mikroszolgáltatások elé. A döntést a különböző közönségeknek eltérő módon kell bemutatni.

6.4.18 5.1 Vezetői összefoglaló

Cím: API gateway bevezetése a szolgáltatásaink elé

Összefoglaló: A növekvő számú belső és külső API jelenleg közvetlenül a szolgáltatásokhoz kapcsolódik. Ez megnehezíti a hitelesítés, a sebességkorlátozás és a megfigyelés egységes kezelését. Javasoljuk egy API gateway bevezetését, amely központi belépési pontot biztosít.

Üzleti érték: csökkenti a fejlesztői csapatok terhelését (becslés: évi 20%-os időmegtakarítás a keresztmetszeti funkciókban), javítja a biztonsági megfelelést, és lehetővé teszi a forgalom szabályozását.

Költség: a gateway üzemeltetése havonta körülbelül [X] erőforrás-óra, a bevezetés egyszeri költsége [Y] mérnöknap. A pontos szám a PoC után finomítható.

Kockázat: új kritikus komponens; ha a gateway leáll, minden API elérhetetlenné válik. A kockázatot magas rendelkezésre állású konfigurációval és alapos monitorozással kezeljük.

Döntési határidő: [dátum]

6.4.19 5.2 Fejlesztői technikai leírás

A fejlesztők számára készült leírás tartalmazza a C4 container szintű diagramot, a gateway által kezelt keresztmetszeti funkciókat (hitelesítés, rate limiting, audit naplózás), a szolgáltatások regisztrációjának módját, valamint a szükséges kódváltozásokat. Az alábbi Mermaid-szekvencia azt mutatja, hogyan változik egy kérés útja:

```
sequenceDiagram
    participant U as Felhasználó
    participant G as API Gateway
    participant S as Rendeléskezelő szolgáltatás
    participant DB as Rendelés-adatbázis

    U->>G: POST /orders (HTTPS)
    G->>G: Hitelesítés, rate limiting
    G->>S: POST /internal/orders
    S->>DB: Tranzakció
    DB-->>S: Sikeres mentés
    S-->>G: 201 Created
    G-->>U: 201 Created
```

A diagram jelmagyarázata: a szaggatott nyíl aszinkron üzenetet jelöl, a folytonos nyíl szinkron REST-hívást. A gateway és a szolgáltatás között belső hálózati kapcsolat van, amely TLS-t használ.

A fejlesztői leírás kitér arra is, hogy a gateway bevezetése után a szolgáltatásoknak nem kell saját hitelesítést implementálniuk; a kérések egy `X-User-Id` fejlécet kapnak a gatewaytől. Az interfész-szerződés változása verziózott API-szerződésben rögzítendő.

6.4.20 5.3 ADR és RFC a döntéshez

A döntéshez ADR készül, amely a fenti sablon szerint rögzíti a kontextust, a döntést, az alternatívákat (például service mesh használata) és a következményeket. Az ADR hivatkozik az RFC-re, amely a javaslatot részletesen bemutatta, és a design review során beérkezett visszajelzéseket is összefoglalja.

Ez a hármas – vezetői összefoglaló, technikai leírás, ADR – együtt biztosítja, hogy minden stakeholder a számára releváns formában kapja meg az információt, miközben a döntés visszakereshető és indokolt marad.

6.4.21 6. Labor: döntés bemutatása vezetői és fejlesztői nézőpontból

Ez a labor a fejezetben tanult kommunikációs eszközök gyakorlását szolgálja. A feladat egy architektúradöntés kiválasztása, majd annak bemutatása két különböző közönségnek: vezetői és fejlesztői nézőpontból. A labor dokumentum alapú, nem igényel futó rendszert; a hangsúly a kommunikáció minőségén és a döntési napló reprodukálhatóságán van.

Kiinduló állapot: Válassz egy valós vagy fiktív döntést a saját környezetedből. Jó példák: bevezetni egy cache-réteget, áttérni egy új adatbázisra, bevezetni egy aszinkron feldolgozási mintát, vagy migrálni egy moduláris monolitot mikroszolgáltatásokra. A döntés legyen elég összetett ahhoz, hogy legalább két alternatívát fel lehessen sorolni.

Feladat:

1. Készíts egy **ADR-t** a döntéshez a fejezetben bemutatott sablon alapján. A kontextusban szerepeljen legalább egy quality attribute scenario mérhető mérőszámmal és mérési ablakkal. Az alternatívák között legalább két elvetett lehetőség legyen, mindegyiknél indoklással.
2. Készíts egy **RFC-vázlatot** ugyanarról a döntésről, mintha még a döntés előtt állnánk. A vázlat tartalmazzon nyitott kérdéseket és kockázatokat.
3. Készíts egy **vezetői egyoldalas összefoglalót** a döntésről. A terjedelem legfeljebb 400 szó. A hangsúly az üzleti értéken, a költségen és a kockázaton legyen. Technikai részletek ne szerepeljenek.
4. Készíts egy **fejlesztői technikai leírást** a döntésről. A terjedelem legfeljebb 800 szó. Tartalmazzon egy C4 container szintű diagramot diagrams-as-code formában (Mermaid vagy PlantUML), valamint a főbb interfész-változásokat.
5. **Válaszd el a mért és a várt eredményt:** a laborban nincs futó rendszer, ezért a „várt eredmény" a dokumentumokkal szembeni elvárás (például: az ADR tartalmaz kontextust, döntést, alternatívákat, következményeket). A „mért eredmény" az, amit ténylegesen ellenőrizni tudsz: például egy kolléga visszajelzése, egy review-checklist pontszám, vagy a dokumentum szavainak száma. A kettőt a laborjegyzetben külön szakaszban rögzítsd.

Ellenőrzési szempontok:

- Az ADR minden kötelező mezőt tartalmaz, és az állapot megfelelően rögzített.
- A quality attribute scenario tartalmaz ingert, környezetet, választ és mérőszámot mértékegységgel.
- Az alternatívák elutasítása indokolt, nem csak felsorolt.
- A vezetői összefoglaló nem használ technikai zsargont, és a döntés üzleti értéke világos.
- A fejlesztői leírás diagramja diagrams-as-code formátumú, és a C4 container szintű elemek felelőssége azonosítható.
- A mért és várt eredmény külön van választva, és a mért eredmény ténylegesen ellenőrzött.

Visszaállítási út: Mivel a labor dokumentumokon dolgozik, a visszaállítás a verziókövető rendszerben történő commitokkal valósul meg. Javasolt minden dokumentumot külön commitban menteni, így bármelyik korábbi állapot visszaállítható. Ha a labor során hibát észlelsz, a git history segítségével visszatérhetsz a kiinduló állapothoz.

Megjegyzés: A laborban szereplő példák és helyőrzők szemléltetők; ne használj valós belső URL-t, ügyfélnevet vagy hitelesítő adatot. Minden külső rendszert `example.com` vagy hasonló helyőrző domainnel jelölj.

6.4.22 7. Ellenőrzőlista

A fejezet tanulási céljainak önellenőrzéséhez használd az alábbi listát:

- ☐ Meg tudom nevezni legalább öt stakeholder-csoportot, és tudom, milyen formátumban kommunikálnék velük egy architektúradöntést.
- ☐ Értem az ADR szerepét a döntési naplóban, és ismerem a kötelező mezőit.
- ☐ Tudok példát mondani arra, mikor érdemes RFC-t írni, és miben különbözik az ADR-től.
- ☐ Felismerem a design review során használandó ellenőrzőlista főbb pontjait.
- ☐ Meg tudom indokolni, miért fontos az alternatívák és a trade-offok rögzítése.
- ☐ Képes vagyok egy döntést vezetői és fejlesztői nézőpontból is bemutatni, a megfelelő részletességgel.
- ☐ Tudom, hogyan kezeljem a visszacsatolást: kategorizálás, beépítés vagy indokolt elutasítás, döntés rögzítése.

6.4.23 8. Önellenőrző kérdés

Kérdés: Egy fejlesztői csapat be akarja vezetni a CQRS-mintát egy meglévő rendeléskezelő szolgáltatásban, de az üzemeltetési csapat aggódik a megnövekedett komplexitás miatt. Hogyan kommunikálnád ezt a döntést a két csoportnak úgy, hogy a döntés indoklása és a visszacsatolás kezelése is nyomon követhető legyen? Adj rövid vázlatot a tervezett kommunikációs csomagról.

Válasz vázlat: A döntéshez először RFC-t készítenék, amely bemutatja a jelenlegi szinkron írási-olvasási modell korlátait (például p95 válaszidő növekedése), a CQRS javaslatot, az alternatívákat (például csak olvasási replika bevezetése), a kockázatokat (esetleges konzisztencia, duplikált események) és a nyitott kérdéseket. Az RFC-t megosztanám a fejlesztői és az üzemeltetési csapattal, és design review-t tartanék, ahol a visszajelzéseket kategorizálom. Az üzemeltetési aggályokat (például monitorozás, telepítés) beépíteném a javaslatba, vagy ha nem releváns, indoklással elutasítanám. A döntés után ADR-t készítenék, amely rögzíti a kontextust, a döntést, az alternatívákat és a következményeket, beleértve az üzemeltetési csapat észrevételeit is. Végül a vezetői összefoglaló az üzleti értéket és a költséget, a fejlesztői leírás a technikai részleteket mutatná be.

6.4.24 9. Összegzés

Az architektúra kommunikációja nem egyszeri dokumentumkészítés, hanem folyamatos párbeszéd a stakeholderekkel. Az ADR a döntések hosszú távú memóriája, az RFC a vita előkészítője, a design review pedig a visszacsatolás strukturált kerete. Ha ezeket az eszközöket következetesen használjuk, a döntések nem válnak láthatatlan, megkérdőjelezhetetlen vagy ismétlődő viták tárgyává. A következő fejezetben a követelményanalízisre térünk át, ahol az itt bevezetett fogalmak – különösen a minőségi forgatókönyvek és a stakeholder-térkép – alapozzák meg a rendszerhatárok és a nem funkcionális követelmények pontos meghatározását.

Ez a fejezet a dokumentációs szerződésnek megfelelően verziózott ADR-ekre, diagrams-as-code C4 diagramokra és ellenőrzött mérési hivatkozásokra épít. A példákban szereplő belső címek és komponensnevek helyőrzők, nem valós rendszerekre utalnak. A labor során keletkező dokumentumokat a kódbázissal együtt verziókövető rendszerben tárold, és a döntési naplót folyamatosan karbantartva biztosítsd a visszakereshetőséget.

II. RÉSZ – Követelmények, minőség és tervezés

7 Követelményanalízis architekt szemszögből

7.1 Funkcionális követelmények és rendszerhatárok

A funkcionális követelmények azonosítása és a rendszerhatár kijelölése az architekturális munka első, és gyakran leginkább alábecsült lépése. Nem az a kérdés, hogy „mit tud a rendszer”, hanem az, hogy „miért pont ezt, és miért pont itt ér véget a felelőssége”. Ha ez a kettő nem világos, a későbbi döntések – a komponensfelosztástól a kiadási stratégiáig – homokra épülnek. Ebben az alfejezetben a funkcionális követelmények és a rendszerhatár azonosításának gyakorlatát járjuk körbe, elválasztjuk a funkcionális és nem funkcionális elvárásokat, és eszközöket adunk a hiányzó követelmények, valamint a rejtett feltevések felszínre hozásához.

7.1.1 1. Mi a funkcionális követelmény, és miért az architekt dolga?

A funkcionális követelmény azt írja le, **mit kell a rendszernek tennie** a felhasználó vagy egy másik rendszer számára. Például: „A vevő leadhat egy rendelést”, „A számlázó szolgáltatás kiállít egy számlát”, „A riport modul napi összesítést készít az értékesítésekről”. A hangsúly a viselkedésen és az üzleti kimeneten van, nem pedig a technológián. Az architekt számára azonban a funkcionális követelmény nem csupán egy bejegyzés a backlogban. A követelmények halmaza határozza meg azokat a **felelősségeket**, amelyeket a komponenseknek viselniük kell, és azokat az **interfészeket**, amelyeken keresztül a felelősségek összekapcsolódnak.

Fontos különbséget tenni a „funkcionális követelmény” és a „funkció” között. A funkció egy felhasználó által látható képesség, például a „kosárba helyezés”. A funkcionális követelmény ennél pontosabb: magában foglalja a bemenetet, a feldolgozást és a kimenetet, gyakran feltételekkel együtt. Például: „A kosárba helyezés során a rendszer ellenőrzi a rendelkezésre álló készletet, és ha a termék nem elérhető, értesíti a felhasználót, de nem engedélyezi a rendelést.” Az ilyen megfogalmazás már tartalmazza a viselkedési ágakat is, amelyek az architektúrában hibakezelési, tranzakciós és konzisztenciaproblémákként jelentkeznek.

A funkcionális követelményeket gyakran user story-ként, use case-ként vagy formális specifikációként rögzítik. Az architekt szempontjából nem a formátum a lényeg, hanem a **lefedettség** és a **rendszerhatárra gyakorolt hatás**. Egy user story megfogalmazható úgy, hogy „mint vevő, szeretnék rendelést leadni, hogy megkapjam a terméket”. Ez azonban nem mondja meg, hogy a rendelés leadása magában foglalja-e a fizetési tranzakciót, a készletfoglalást, a számlázást vagy az e-mail értesítést. A rendszerhatár kijelölése éppen ezeket a bizonytalanságokat teszi láthatóvá.

7.1.2 2. Rendszerhatár: mit foglal magában, és mi az, ami kívül marad?

A rendszerhatár azt a vonalat jelöli, amely elválasztja a rendszer felelősségi körét a környezetétől. A határon belül vannak azok az elemek, amelyeket a rendszer fejleszt, üzemeltet és kontrollál. A határon kívül vannak a felhasználók, a külső rendszerek, a partnerek, a szenzorok, a szabályozó hatóságok, és minden, amivel a rendszer kommunikál, de nem irányít közvetlenül.

A C4 modellben ezt a **context szint** mutatja be: a rendszert egyetlen dobozként ábrázoljuk, körülötte a felhasználói szerepkörökkel és a külső rendszerekkel. Például egy online számlázó szolgáltatás context diagramján a rendszer középen áll, körülötte a „Könyvelő” szerepkör, a „Pénzügyi rendszer” külső rendszer és az „Email szolgáltató”. A rendszerhatár nem csupán technikai vonal, hanem felelősségi határ is: ha a fizetési tranzakció meghiúsul, ki felel a kompenzációért? A mi rendszerünk vagy a fizetési szolgáltató? A válasz meghatározza, hogy a tranzakciós logika a határon belülre vagy kívülre kerül.

A rendszerhatár azonosításának gyakorlati módszere a **felelősségalapú interjú**. Minden funkcionális követelménynél kérdezzük meg: „Ki hajtja végre ezt a lépést? Ki birtokolja az adatot? Ki felel az eredményért?” Ha a válasz nem a mi rendszerünk, akkor az adott lépés vagy adat a határon kívül van, és interfészen keresztül kell hozzáférni. Ha a válasz a mi rendszerünk, akkor a lépés a határon belül marad, és a komponensarchitektúrában kell helyet kapnia.

Például egy e-kereskedelmi rendszerben a „Termék megjelenítése” követelménynél a termékadatok tárolása lehet a mi rendszerünk hatásköre, de az árfolyamok lekérdezése a központi bank vagy egy árfolyamszolgáltató külső rendszeré. A határ meghúzása azt jelenti, hogy a mi rendszerünk felel a termékadatok konzisztenciájáért, de nem felel az árfolyamok valós idejű helyességéért – az csak egy külső interfész, amelynek rendelkezésre állására és válaszidejére SLA-t kötünk, de nem kontrolláljuk a belső implementációját.

7.1.3 3. Funkcionális és nem funkcionális követelmények elkülönítése

A gyakorlatban a legtöbb hiba abból származik, hogy a két kategóriát összekeverik, vagy az egyiket a másik rovására elhanyagolják. A funkcionális követelmény a „mit”, a nem funkcionális (minőségi) követelmény a „milyen jól” kérdésre válaszol. A nem funkcionális követelményeket a Nem funkcionális követelmények és minőségi attribútumok fejezet tárgyalja részletesen, itt csak a szétválasztás műveleti szabályát rögzítjük.

Funkcionális követelmény (mit)	Nem funkcionális követelmény (milyen jól)
A rendszer kiszámítja a kosár végösszegét.	A végösszeg számítása 200 ms alatt megtörténik.
A felhasználó hozzáfér a korábbi rendeléseihez.	Az adatlekérés az esetek 95%-ában 1 s alatt teljesül.
A rendszer naplózza a bejelentkezéseket.	A naplóbejegyzés nem tartalmaz személyes adatot.
A rendszer értesítést küld a rendelés visszaigazolásáról.	Az értesítés 99,9%-os valószínűséggel kézbesül 5 percen belül.

A szétválasztás nem öncélú. Ha egy követelményt funkcionálisként kezelünk, akkor a tervezés során a komponensek „milyen lépés” szintjén foglalkozunk vele; ha nem funkcionálisként, akkor a minőségi attribútumok és az SLO-k szintjén. Egy „gyors keresés” például önmagában nem követelmény, mert nem mérhető. A pontos megfogalmazás: „A termékkereső 10 000 termék esetén a válaszidő 95. percentilise legfeljebb 300 ms, 200 egyidejű felhasználó mellett.” Ez már quality attribute scenario formátumú, amelyet a későbbiekben ismertetünk. Az architekt dolga, hogy a kevert megfogalmazásokat szétszálazza, és minden elvárást a megfelelő kategóriába soroljon.

Vannak határesetek. Például „a rendszer támogatja a többnyelvűséget” – ez funkcionális képesség, de a „minden felirat legfeljebb 1 másodperc alatt betöltődik” nem az. Hasonlóan, „a rendszer HTTPS-en kommunikál” – ez technológiai kényszer, nem feltétlenül nem funkcionális követelmény, hanem egy megvalósítási korlát. Az ilyen kényszereket a követelményjegyzékben külön kell jelölni, mert a döntési mátrixban másképp kezeljük őket, mint a viselkedési elvárásokat.

7.1.4 4. Hiányzó követelmények és rejtett feltevések

A legtöbb funkcionális követelménylista azért hiányos, mert a készítője a **boldog utat** (happy path) írta le, és nem kérdezett rá a **szélsőséges esetekre** (edge case) vagy a **meghibásodási módokra**. Az architekt feladata, hogy ezeket a réseket módszeresen feltárja. A hiányzó követelmények tipikus kategóriái:

- **Hibakezelés:** Mi történik, ha a fizetési szolgáltató nem elérhető? Mi történik, ha a felhasználó kétszer kattint a „Rendelés leadása" gombra?
- **Jogosultság és hozzáférés:** Ki láthatja a rendelést? Ki módosíthatja az árat? Mi történik, ha egy megszűnt felhasználó próbál belépni?
- **Adatmegőrzés és törlés:** Meddig kell megőrizni a számlákat? Mi történik a személyes adattal, ha a felhasználó törli a fiókját?
- **Integrációs peremfeltételek:** Mi van, ha a külső API válasza hibás formátumú? Mi van, ha a szállító rendszer késlekedik?
- **Üzleti szabályok szélsőségei:** Mi van, ha a kosár értéke nulla? Mi van, ha a termékből nincs készlet, de a felhasználó előrendelést ad le?

A **rejtett feltevések** ennél veszélyesebbek, mert nem hiányoznak a listából, hanem implicit módon jelen vannak a fejlesztők, a termékmenedzser vagy a domain szakértő gondolkodásában. Például: „a felhasználó már bejelentkezett", „a fizetési tranzakció mindig szinkron módon visszaigazolást ad", „a rendelés csak egy raktárból teljesíthető", „az árfolyam a nap közepén nem változik". Az ilyen feltevések akkor okoznak kárt, amikor a rendszer éles környezetbe kerül, és a valóság másképp viselkedik.

A feltevések feltárására több gyakorlati technika használható:

- **Pre-mortem:** Képzeljük el, hogy a rendszer fél év múlva megbukott. Mi okozta a legnagyobb kárt? A válaszok gyakran felszínre hozzák a rejtett feltevéseket.
- **„Mi van, ha?" kérdések:** Szisztematikusan kérdezzük meg minden lépésnél: mi van, ha ez nem történik meg? Mi van, ha kétszer történik meg? Mi van, ha rossz sorrendben?
- **Példa-forgatókönyvek:** Írjunk le három különböző ügyfélutat: új felhasználó, visszatérő nagykereskedő, hiba utáni újrapróbálkozás. A forgatókönyvekben rejlő feltételezések előbukkannak.
- **Domain szakértő bevonása:** A domain mély ismerete segít azonosítani azokat a hallgatólagos üzleti szabályokat, amelyeket a felhasználói történetek nem tartalmaznak.

Az architekt ezeket a feltevéseket **explicit döntési pontokká** alakítja. Ha például kiderül, hogy a fizetési tranzakció időnként aszinkron visszaigazolást ad, akkor a rendszerhatáron belül meg kell tervezni a tranzakció állapotkövetését és a kompenzációs logikát. Ha a feltevés az, hogy a raktárak száma egy, akkor a rendszerhatár kialakításánál dönteni kell: a készletkezelés a rendszeren belül marad-e, vagy külső raktárkezelő rendszerre támaszkodik?

7.1.5 5. Követelményjegyzék és határvonal a gyakorlatban

A követelményjegyzék nem puszta lista, hanem a rendszerhatárral együtt értelmezett, nyomon követhető dokumentum. Minden funkcionális követelménynél fel kell tüntetni:

- Azonosító (például FR-001).
- A követelmény megfogalmazása, beleértve a feltételeket és a fontosabb alternatív ágakat.
- A rendszerhatáron belüli vagy kívüli besorolás: ha kívül van, meg kell nevezni a felelős külső rendszert.
- A kapcsolódó interfész(ek), ha a követelmény más rendszerekkel érintkezik.

- A forrás: melyik stakeholder vagy használati eset alapján került be.
- A prioritás: kötelező, fontos, későbbi opció.
- Az állapot: tervezett, megerősített, vitatott – ez a döntési napló bejegyzéseihez kapcsolódik.

A **határvonal** rögzítésére a C4 context diagram alkalmas, de a diagram önmagában nem elég. Minden külső rendszerhez tartoznia kell egy rövid leírásnak, amely megadja a felületet, a protokollt, az adatáramlás irányát és a szinkronitást. Például: „Fizetési szolgáltató (külső) – REST API, szinkron kérés/válasz a fizetési tranzakció végrehajtására; a rendszer a válasz alapján jóváhagyja vagy elutasítja a rendelést. A szolgáltató kiesése esetén a rendszer a rendelést függő állapotban tartja." Ez a mondat egyszerre határozza meg a rendszerhatárt, az interfészt és a hibakezelési elvárást.

A döntési napló szerepe itt is megkerülhetetlen. Ha vita van arról, hogy a készletkezelés a rendszeren belül legyen-e, vagy külső szolgáltatásként, akkor egy ADR-ben rögzíteni kell a döntést, az alternatívákat és a következményeket. Az ADR visszautal a funkcionális követelményekre, így a jövőbeli olvasó látja, hogy a határvonalat miért húzták meg éppen ott.

7.1.6 6. Labor: követelményjegyzék és határvonal készítése

A labor célja, hogy a fejezetben bemutatott módszereket egy kis, ellenőrizhető példán alkalmazza. A feladat egy **fiktív könyvtári kölcsönző API** funkcionális követelményeinek és rendszerhatárának rögzítése. Az API-t belső rendszerként kezeljük, amelyet egy könyvtári alkalmazás és egy adminisztrációs felület használ.

7.1.7 6.1 Kiinduló állapot

- A rendszernek három felhasználói szerepköre van: **olvasó**, **könyvtáros** és **rendszergazda**.
- A könyvtárban 10 000 könyv van, egyetlen fizikai helyen.
- A rendszernek kezelnie kell a kölcsönzést, a visszahozatalt, a foglalást és a késedelmi díjak kiszámítását.
- Van egy meglévő **katalogizáló rendszer** (külső), amely a könyvek bibliográfiai adatait szolgáltatja.
- A fizetés jelenleg készpénzben történik a könyvtárban, tehát **nincs online fizetési szolgáltató**.
- A rendszernek e-mailben kell értesítést küldenie a foglalásról és a késedelmi díjról.

7.1.8 6.2 Elvárt eredmény

Készíts egy reprodukálható dokumentumot, amely a következőket tartalmazza:

1. **Funkcionális követelményjegyzék** a fenti szerepkörök és üzleti folyamatok alapján, legalább 10 tétellel.
2. **Rendszerhatár C4 context szinten**: Mermaid vagy PlantUML diagram, amelyen a rendszer, a három szerepkör, a katalogizáló rendszer és az email szolgáltató szerepel. A diagramhoz tartozzon egy rövid jelmagyarázat és a kapcsolatok leírása (protokoll, szinkronitás, hibatűrés).
3. **Döntési napló** legalább 2 bejegyzéssel, amelyek a határvonal vitatott pontjait rögzítik (például: a katalogizáló rendszer adatait másoljuk-e ide, vagy minden kérésnél lekérdezzük? A késedelmi díj számítása a rendszeren belül legyen-e?).
4. **Ellenőrzési szempontok** táblázata, amelyben a mért és a várt eredményt külön oszlopban tünteted fel.

7.1.9 6.3 Ellenőrzési szempontok

Ellenőrzési szempont	Várt eredmény	Mért eredmény
A jegyzékben minden szerepkörhöz tartozik legalább egy funkcionális követelmény.	3 szerepkör × legalább 1 tétel	…
Minden külső rendszerhez tartozik interfészleírás.	2 külső rendszer (katalogizáló, email)	…
A C4 context diagramon a rendszerhatár egyértelműen jelölt.	A rendszer doboza, a külső rendszerek és a szerepkörök külön jelölve, jelmagyarázattal	…
A funkcionális és nem funkcionális elvárások szét vannak választva.	A jegyzékben a nem funkcionális tételek külön szekcióban, vagy NFR jelöléssel	…
Legalább egy hiányzó követelményt feltártál és rögzítettél a döntési naplóban.	1 vagy több ADR a hiánypótlásról	…
A dokumentum verziózott, és a döntési napló bejegyzéseihez visszakereshető hivatkozás tartozik.	A dokumentumban git commit hash vagy dátum, az ADR-ekre mutató hivatkozások	…

A **mért eredmény** oszlopot a saját dokumentumod alapján töltsd ki, például a ténylegesen elkészített követelmények számával vagy a diagramon szereplő kapcsolatokkal. A cél nem a várt értékek szolgai elérése, hanem annak megértése, hogy a követelményjegyzék és a határvonal mennyire teljes és konzisztens.

7.1.10 6.4 Visszaállítási út

Mivel a labor nem éles rendszeren fut, nincs szükség adatbázis-visszaállításra. Ha azonban a dokumentumot verziókezelőben tárolod, javasolt egy új ágat nyitni, és a labor végét jelző commit után a munkát összevonni a főágba. Ha a labor során megváltoztatod a C4 diagramot, készíts biztonsági másolatot az előző verzióról, hogy a változások nyomon követhetők legyenek.

7.1.11 7. Tipikus hibák és azok elkerülése

1. **A rendszerhatár technikai rétegként való kezelése, nem felelősségi vonalként.** Ha a határt a „backend” és a „frontend” közé húzzuk anélkül, hogy megmondanánk, ki felel az üzleti logikáért, a határ homályos marad. Helyette a felelősséget kell kijelölni: a frontend felel a megjelenítésért, a backend az üzleti szabályokért.
2. **A funkcionális követelmények túlságosan technikai szintűek.** Az „A rendszer JSON-t ad vissza” nem funkcionális követelmény, hanem interfészszerződés. A követelmény legyen üzleti nyelvű, és a technikai részletet külön jegyezzük fel.
3. **A nem funkcionális követelmények elhagyása a funkcionális listából.** Ha a teljesítmény vagy a biztonság nem szerepel a jegyzékben, a tervezés során ezek nem kapnak figyelmet. A szétválasztás nem jelenti a figyelmen kívül hagyást.

4. **A hiányzó esetek elkenése.** Ha egy követelménynél nincs leírva a hibakezelés, az architekt hajlamos feltételezni, hogy a hiba nem fordul elő. A valóságban minden hálózati hívás megszakadhat, minden felhasználó hibázhat. A laborban ezért kötelező legalább egy hiányzó eset feltárása.
5. **A rendszerhatár rögzítésének elmulasztása.** Ha a határ nincs dokumentálva, a csapat tagjai eltérően képzelik el a felelősségeket. A C4 context diagram és a külső interfészek leírása ezt a kockázatot csökkenti.

7.1.12 8. Ellenőrzőlista a fejezethez

- ☐ Minden azonosított funkcióhoz tartozik-e funkcionális követelmény leírás, feltételekkel együtt?
- ☐ A funkcionális és nem funkcionális elvárások szét vannak-e választva a jegyzékben?
- ☐ A rendszerhatár egyértelműen jelölt-e C4 context szinten?
- ☐ Minden külső rendszerhez tartozik-e interfészleírás (protokoll, szinkronitás, hibatűrés)?
- ☐ A követelményjegyzék minden eleme nyomon követhető-e a forrásig (stakeholder, use case, user story)?
- ☐ A vitatott határvonalakat rögzítette-e döntési napló?
- ☐ Legalább három „mi van, ha?” kérdést feltett-e a csapat, és a válaszok bekerültek-e a jegyzékbe?
- ☐ A dokumentum verziózott-e, és a döntésekhez visszakereshető hivatkozás tartozik-e?

7.1.13 9. Önellenőrző kérdés

Adott egy online ételrendelő rendszer. A termékmenedzser azt írja: „A felhasználó leadhatja a rendelését, és az étterem visszaigazolja.” Sorolj fel legalább öt olyan funkcionális követelményt, amelyet ez a mondat nem részletez, de az architektúra tervezéséhez szükséges. Mindegyiknél jelöld, hogy a rendszerhatáron belülre vagy kívülre esik-e, és miért.

Ezzel a kérdéssel a hiányzó követelmények és a rendszerhatár összefüggését gyakoroljuk. A válaszban szerepelnie kell például a fizetés, a készletkezelés, a szállítás, a hibakezelés és a jogosultság kérdéseinek – és mindegyiknél el kell dönteni, hogy a rendszer felelős-e érte, vagy külső szolgáltatás.

7.1.14 10. Továbbvitt eredmények

Az itt bevezetett fogalmak – a funkcionális követelmények pontosítása, a rendszerhatár felelősségalapú kijelölése, a feltevések explicitté tétele – alapvetőek a következő alfejezethez (Use case-ek, user story-k és architektúra). Ott a use case-eket, user story-kat és azok architekturális hatásait vizsgáljuk meg mélyebben, és megmutatjuk, hogyan alakíthatók a követelmények konkrét komponensfelelősségekké. A rendszerhatár itt meghúzott vonala lesz az a keret, amelyen belül a komponensek és interfészek tervezése zajlik.

7.2 Use case-ek, user story-k és architektúra

A funkcionális követelmények gyakran mondatokban, folyamatleírásokban vagy termékvízióban élnek. Az architekt feladata, hogy ezekből a – sokszor elnagyolt – megfogalmazásokból olyan szerkezeti és viselkedési következtetéseket vonjon le, amelyekből komponensek, interfészek, adatfolyamok és hibakezelési stratégiák válnak. A use case-ek és a user story-k ebben a folyamatban

nem csupán elemzési eszközök, hanem az architektúra első vázlatának forrásai. Ez az alfejezet azt mutatja be, hogyan kell ezeket a technikákat úgy használni, hogy a belőlük kinyert információ közvetlenül támogassa a későbbi architektúradöntéseket.

7.2.1 1. A use case-ek és user story-k architekturális jelentése

A használati eset (use case) és a felhasználói történet (user story) egyaránt a rendszerrel szembeni felhasználói vagy külső rendszer által elvárt viselkedést ír le, de eltérő részletezettséggel és céllal. A use case általában egy célorientált folyamatot ír le: ki (szereplő), mit szeretne elérni (cél), milyen lépéseken keresztül (forgatókönyv), és mi történik, ha valami nem a fő úton halad (kivételek). A user story ezzel szemben rövidebb, inkább az értékre és a beszélgetésre fókuszál: „mint [szereplő], szeretnék [képesség], hogy [üzleti érték]". A story-hoz általában elfogadási kritériumok (acceptance criteria) tartoznak, amelyek konkretizálják a „kész" állapotot.

Az architektúra szempontjából a use case-ek és story-k annyiban jelentenek többet, mint a puszta funkcionális specifikáció, hogy:

- **Felfedik a rendszerhatárokat és a külső szereplőket.** Minden szereplő, legyen az ember vagy külső rendszer, egy-egy interfészt igényel a rendszerünk felé. A szereplők listája így közvetlen bemenete a C4 context diagramnak.
- **Meghatározzák a főbb rendszerfunkciókat, amelyeket a rendszer belső komponenseinek kell megvalósítaniuk.** Egy jól megírt use case lépései gyakran természetes jelöltjei a későbbi komponens-felelősségeknek: a „hitelesítés", „katalógus lekérdezés", „kölcsönzés rögzítése" mind külön modulba kerülhetnek.
- **Kijelölik a tranzakcióhatárokat és az adatkonzisztencia követelményeit.** Ha egy use case-ben két lépésnek atomi módon kell együtt sikerülnie vagy kudarcot vallania, akkor az tranzakciós határt jelez. Ilyen lehet például a „rendelés létrehozása" és a „készlet csökkentése" együttes sikere.
- **A kivételes útvonalak a hibatűrési és reziliencia-követelmények forrásai.** Ha a fizetési szolgáltató nem elérhető, a rendszernek definiálnia kell, hogy mi történik: újrapróbálkozik, vár, vagy kompenzáló műveletet hajt végre? Ez már egyértelműen architekturális döntés.
- **A gyakori vagy kritikus use case-ek segítenek a teljesítmény és skálázhatóság tervezésében.** Ha naponta százezerszer fut le egy folyamat, akkor a kapacitástervezésnél ezt a terhelést kell figyelembe venni, nem pedig a ritka adminisztratív műveleteket.

Fontos hangsúlyozni, hogy a use case és a user story nem helyettesítik egymást. A gyakorlatban gyakran együtt élnek: a story-k adják a termék backlogot, a use case-ek pedig a nagyobb, összetettebb folyamatok részletes elemzését. Az architekt mindkettőből dolgozik, de más-más szinten. A story-kból a rendszer fő képességei és azok gyakorisága derül ki; a use case-ekből pedig a folyamatokon belüli szerkezet, a kivételek és a külső függőségek.

Vegyünk egy egyszerű fiktív példát: egy könyvtári kölcsönző rendszert. A story lehet:

> „Mint olvasó, szeretnék online kölcsönözni egy könyvet, hogy ne kelljen személyesen bemennem a könyvtárba."

Ez a mondat önmagában még nem mond sokat az architektúráról. A hozzá tartozó use case viszont már részletezi a lépéseket, és ebből kirajzolódik, hogy a rendszernek rendelkeznie kell hitelesítési réteggel, katalóguslekérdező szolgáltatással, készletkezelő logikával, tranzakciós adatbázissal és értesítési mechanizmussal. A story tehát az üzleti igényt rögzíti, a use case pedig azt a viselkedési kontraktust, amelyre az architektúra épül.

7.2.2 2. Szereplők, forgatókönyvek és kivételek

7.2.3 Szereplők azonosítása

A szereplő (actor) bárki vagy bármi, aki/ami közvetlenül kapcsolatba lép a rendszerrel. A use case elemzés első lépése a szereplők pontos meghatározása. Megkülönböztetünk:

- **Elsődleges szereplő (primary actor):** az, aki a use case-t kezdeményezi, és akinek a célja teljesül. Például az „Olvasó”.
- **Támogató szereplő (supporting actor):** olyan külső rendszer vagy személy, aki/amely a use case végrehajtásában segít, de a cél elérése nem az ő érdeke. Például egy „Fizetési szolgáltató” vagy „E-mail szolgáltatás”.
- **Mögöttes szereplő (offstage actor):** olyan külső fél, aki/ami érdekelt a use case kimenetelében, de közvetlenül nem lép interakcióba a rendszerrel. Például egy „Könyvtárvezető”, aki a riportokat nézi.

Az architektúra tervezésekor minden szereplőt interfészhez kell kötni. Az emberi szereplők általában felhasználói felületet vagy API-t használnak, a külső rendszerek pedig protokollal és adatszerződéssel rendelkező integrációs pontot igényelnek. A szereplők listája tehát a context szintű diagram bemenete.

7.2.4 Forgatókönyvek szerkezete

Egy use case forgatókönyve tipikusan három részből áll:

1. **Fő sikerforgatókönyv (happy path):** azok a lépések, amelyek akkor futnak le, ha minden a tervezett módon működik. Ez a legrövidebb út a cél eléréséig.
2. **Alternatív útvonalak:** olyan elágazások, amelyekre akkor kerül sor, ha a szereplő más, de érvényes döntést hoz. Például a könyvtári rendszerben az olvasó választhatja, hogy a könyvet a könyvtárban veszi át vagy házhoz szállíttatja.
3. **Kivételes útvonalak (exception paths):** olyan helyzetek, amikor valamilyen hiba vagy váratlan esemény történik. Ilyen lehet a hitelesítés sikertelensége, a kért könyv nem elérhető, vagy a fizetési szolgáltató nem válaszol.

Az architektúra szempontjából a kivételes útvonalak a legértékesebbek. A fő forgatókönyv ugyanis gyakran csak a funkcionális követelményeket mutatja meg: milyen adatot kell bevinni, milyen sorrendben kell a műveleteket végrehajtani. A kivételek azonban rávilágítanak a nem funkcionális követelményekre: milyen hibakezelési stratégiát kell alkalmazni, milyen kompenzáló tranzakciók szükségesek, milyen időtúllépéseket kell beállítani, és hogyan kell a rendszernek konzisztens állapotban maradnia.

Vegyük a könyvtári kölcsönzés use case-ét. A fő forgatókönyv:

1. Az Olvasó bejelentkezik a rendszerbe.
2. Az Olvasó kiválaszt egy könyvet a katalógusból.
3. A rendszer ellenőrzi, hogy a könyv elérhető-e.
4. Az Olvasó megadja a szállítási címet (ha házhoz szállítást kért).
5. A rendszer rögzíti a kölcsönzést, és csökkenti az elérhető példányszámot.
6. A rendszer visszaigazolást küld az Olvasónak.

Alternatív útvonal: ha az Olvasó személyes átvételt választ, a szállítási címet nem kell megadni.

Kivételes útvonalak:

- A hitelesítés sikertelen → a rendszer hibaüzenetet ad, nem folytatja a folyamatot.
- A kért könyv nem elérhető → a rendszer felajánlja az előjegyzést.
- A készletcsökkentés közben adatbázishiba történik → a rendszer visszagörgeti a tranzakciót, és a kölcsönzés nem jön létre.
- A visszaigazoló e-mail küldése nem sikerül → a kölcsönzés létrejöhet, de a rendszer naplózza a hibát, és később újrapróbálja.

Ezekből a kivételekből olyan architekturális elemek következnek, mint a bejelentkezési szolgáltatás, a készletkezelő szolgáltatás tranzakciós adatbázissal, egy megbízható üzenetküldő réteg (outbox pattern) az e-mail értesítéshez, és egy előjegyzési szolgáltatás. A use case elemzés tehát már a komponensek szintjén is irányt mutat.

7.2.5 A forgatókönyvek formalizálása

A gyakorlatban a use case leírást érdemes táblázatos formában rögzíteni, hogy az architekt később könnyen hivatkozhasson rá. Egy lehetséges sablon:

Elem	Tartalom
Név	Könyv kölcsönzése
Elsődleges szereplő	Olvasó
Támogató szereplők	Hitelesítő szolgáltatás, Katalógus szolgáltatás, E-mail szolgáltatás
Előfeltétel	Az Olvasó regisztrált és bejelentkezett
Fő forgatókönyv	1. Az Olvasó kiválaszt egy könyvet. 2. A rendszer ellenőrzi az elérhetőséget. 3. Az Olvasó megadja az átvételi módot. 4. A rendszer rögzíti a kölcsönzést. 5. A rendszer visszaigazolást küld.
Alternatívák	3a. Ha személyes átvétel, akkor nincs szállítási cím.
Kivételek	2a. A könyv nem elérhető → előjegyzés. 4a. Adatbázishiba → visszagörgetés. 5a. E-mail hiba → naplózás és újrapróbálás.
Utófeltétel	A kölcsönzés rögzítésre került, a készlet csökkent, az Olvasó értesítést kapott.

Ez a formalizálás nem öncélú: a későbbi komponens tervezésnél minden lépéshez hozzárendelhető egy felelős komponens, a kivételekhez pedig egy hibakezelési mechanizmus.

7.2.6 3. Acceptance criteria és minőségi attribútumok

A user story-k esetében az elfogadási kritériumok (acceptance criteria) adják meg, hogy mikor tekinthető a story teljesítettnek. Az architekt számára ezek a kritériumok két szempontból fontosak: egyrészt konkretizálják a funkcionális elvárást, másrészt gyakran impliciten tartalmaznak nem funkcionális követelményeket is, amelyeket explicitté kell tenni.

Vegyük a következő story-t:

> „Mint olvasó, szeretnék online kölcsönözni egy könyvet, hogy elkerüljem a sorban állást.”

Az elfogadási kritériumok például:

- A rendszer a bejelentkezett olvasó számára megjeleníti az elérhető könyvek listáját.
- Az olvasó legfeljebb öt könyvet kölcsönözhet egyszerre.
- Sikeres kölcsönzés esetén a rendszer legfeljebb 2 másodpercen belül visszaigazolást ad.
- Ha a fizetési szolgáltató nem válaszol 5 másodpercen belül, a rendszer hibát jelez, és nem jön létre a kölcsönzés.
- A kölcsönzési adatok nem veszhetnek el: a rendszernek tranzakcióbiztosnak kell lennie.

Ezekből a kritériumokból az architekt a következő minőségi attribútumokat vezetheti le:

- **Teljesítmény:** a visszaigazolás válaszideje legfeljebb 2 másodperc normál terhelés mellett (quality attribute scenario).
- **Rendelkezésre állás:** a fizetési szolgáltató elérhetetlensége esetén a rendszernek meghatározott hibaviselkedést kell mutatnia.
- **Adatintegritás:** a kölcsönzési tranzakció ACID tulajdonságokkal rendelkezzen.
- **Biztonság:** csak hitelesített olvasó indíthat kölcsönzést.

A minőségi attribútumok formális megfogalmazására a könyv későbbi fejezeteiben (Nem funkcionális követelmények és minőségi attribútumok, Minőségi forgatókönyvek és architektúraértékelés) kerül sor. Itt elég annyit hangsúlyozni, hogy az elfogadási kritériumokban rejlő nem funkcionális elvárásokat már a követelményelemzés során ki kell emelni, mert ezek alapvetően befolyásolják az architektúra kialakítását. Ha például az elfogadási kritérium 2 másodperces válaszidőt ír elő, akkor az architekt nem tervezhet olyan megoldást, ahol a tranzakció szinkron módon egymás után hív több távoli szolgáltatást, mert az könnyen meghaladja a büdzsét.

Egy jó gyakorlat, hogy minden kritikus use case-hez vagy story-hoz legalább egy quality attribute scenario-t rendeljünk. A korábbi fejezetben már bevezetett formátumot használva:

- **Inger:** az Olvasó elküldi a kölcsönzési kérelmet.
- **Környezet:** normál üzem, a rendszer a napi csúcsterhelés 80%-án fut.
- **Válasz:** a rendszer visszaigazolást ad, és a kölcsönzés adatai tartósan rögzülnek.
- **Mérőszám:** a válaszidő 95. percentilise legfeljebb 2 másodperc, a tranzakciós sikerarány 99,9%.

Az ilyen forgatókönyvek nemcsak a tervezést segítik, hanem a későbbi architektúra-értékelés és a folyamatos ellenőrzés (fitness functions) alapjául is szolgálnak.

Fontos különbséget tenni az elfogadási kritérium és a minőségi attribútum között. Az elfogadási kritérium a felhasználói történet teljesítésének feltétele, és jellemzően a terméktulajdonos vagy az üzleti elemző fogalmazza meg. A minőségi attribútum ezzel szemben a rendszer egészére vonatkozó, mérhető tulajdonság, amely az architektúrából fakad, és amelyet a rendszernek folyamatosan teljesítenie kell. A kettő között gyakran átfedés van, de az architekt feladata, hogy az utóbbit rendszerszinten, trade-offokkal együtt kezelje.

7.2.7 4. Gyakori hibák és elkerülésük

A use case-ek és story-k architekturális feldolgozása során számos tipikus hiba fordul elő. Ezek közül emelünk ki néhányat, a megoldás irányával együtt.

4.1. Túl technikai use case.
Gyakran előfordul, hogy a use case lépései nem üzleti nyelvet, hanem megvalósítási részleteket írnak le: „a kontroller meghívja a szolgáltatás metódusát, majd az adatbázisba ment”. Ez összekeveri az elemzést a tervezéssel, és megakadályozza, hogy az architekt szabadon válasszon a különböző megvalósítási lehetőségek közül. Helyes megközelítés: a use case maradjon technológia-semleges, és csak az szerepeljen benne, hogy mi történik, ne az, hogy hogyan.

4.2. Hiányzó kivételes útvonalak.
Sok elemzés csak a fő forgatókönyvre koncentrál, és a hibalehetőségeket figyelmen kívül hagyja. Ez vezet oda, hogy az architektúra nem lesz felkészítve a külső szolgáltatások leállására, az adatbázis-hibákra vagy a felhasználói hibákra. A gyakorlatban a kivételek feltárása legalább annyi időt igényel, mint a boldog út leírása. Hasznos technika a szisztematikus kérdezés: „Mi történik, ha X elérhetetlen?", „Mi történik, ha Y megszakad?", „Mi történik, ha a felhasználó kétszer kattint?"

4.3. Elfogadási kritériumok nélküli story-k.
A túl általános story-k nem adnak támpontot sem a fejlesztőnek, sem az architektnek. Az „olvasó kölcsönözhet" story önmagában nem mondja meg, hány könyvet, milyen válaszidővel, milyen hibakezeléssel. Az architekt ilyenkor kénytelen feltételezésekkel élni, ami félreértésekhez vezet. A megoldás: minden story-hoz tartozzon legalább három-öt konkrét, tesztelhető elfogadási kritérium.

4.4. A minőségi attribútumok leválasztása a funkcionális követelményektől.
Gyakori, hogy a teljesítményt, biztonságot vagy rendelkezésre állást kizárólag külön, általános NFR-listában kezelik, és nem kapcsolják össze a konkrét use case-ekkel. Emiatt az architekt nem tudja, hogy melyik képességnek milyen minőségi követelménye van. Például a „2 másodperces válaszidő" követelmény teljesen mást jelent a kölcsönzési folyamatra, mint a havi riportgenerálásra. A megoldás az, hogy a minőségi attribútumokat use case-enként vagy story-nként kell lebontani.

4.5. Elmaradt szereplő-azonosítás.
Ha a use case-ből kimarad egy támogató szereplő – például a hitelesítő szolgáltatás vagy az auditnaplózó rendszer –, akkor az architektúra tervezése során hiányozni fog a megfelelő interfész, és később pótolni kell, ami drága. A szereplők felderítésekor érdemes minden külső rendszert, szabályozói követelményt és érintettet figyelembe venni.

Egy rövid példa a hibákra. Tegyük fel, hogy a következő use case-t írták le:

„Az olvasó kiválasztja a könyvet, a rendszer beteszi a kosárba, majd fizetés után a rendszer kiküldi a könyvet."

Ebből hiányzik a szereplők pontos meghatározása (ki a fizetési szolgáltató?), a kivételek (mi van, ha a fizetés nem sikerül?), és az elfogadási kritériumok. Az architekt nem tudja, milyen tranzakciós határok vannak a kosár és a fizetés között, milyen kompenzáció szükséges, ha a kiküldés nem sikerül, és milyen rendelkezésre állási követelményt kell teljesíteni. A javítás során ezeket pótolni kell, és a use case-t formálisan leírni.

7.2.8 5. Labor: use case-ek és story-k leképezése komponensekre

Ez a labor a fejezetben tanultak gyakorlati alkalmazását szolgálja. A cél, hogy a résztvevő egy fiktív, de valósághű példán keresztül bemutassa, hogyan lehet a use case-ekből és story-kból komponensvázlatot és döntési naplót készíteni.

Kontextus.
Tervezze meg egy „Könyvtár-R" nevű fiktív online könyvtári kölcsönző rendszer magas szintű architektúráját. A rendszer lehetővé teszi az olvasók számára, hogy:

- böngésszenek a katalógusban,
- könyveket kölcsönözzenek online,
- előjegyezzenek nem elérhető könyveket,
- megtekintsék a kölcsönzéseiket.

A rendszer külső függőségei: email szolgáltatás, hitelesítő szolgáltató (OIDC), és egy katalógus szolgáltatás, amely külső könyvtári adatbázist fed le.

Elvégzendő lépések.

1. **Szereplők és use case-ek azonosítása.**
   Írjon le legalább három use case-t, köztük a „Könyv kölcsönzése” esetet. Minden use case-hez adja meg a szereplőket, a fő forgatókönyvet, legalább két kivételes útvonalat és az elfogadási kritériumokat.

2. **User story-k és elfogadási kritériumok.**
   Fogalmazzon meg legalább öt user story-t a fenti képességekre, és mindegyikhez rendeljen három-öt elfogadási kritériumot. Legalább két story esetében tegye explicitté a minőségi attribútumot (pl. válaszidő, rendelkezésre állás).

3. **Komponensvázlat.**
   A C4 modell context szintjén készítsen diagrams-as-code ábrát (Mermaid, PlantUML vagy Structurizr DSL). Jelölje a rendszerhatárt és a külső szereplőket. Majd container szinten vázolja fel a fő komponenseket, mindegyikhez rövid felelősséget és interfészt adva.

   Példa context diagram Mermaid-ben:

   ```
   graph TD
       U[Olvasó] -->|kölcsönzési kérés| KR[Könyvtár-R]
       KR -->|katalógus lekérdezés| KAT[Katalógus szolgáltatás]
       KR -->|OIDC hitelesítés| AUTH[Hitelesítő szolgáltató]
       KR -->|értesítés| EMAIL[Email szolgáltatás]
   ```

   Példa container diagram:

   ```
   graph LR
       subgraph Könyvtár-R
           WEB[Web API]
           SVC[Kölcsönzési szolgáltatás]
           DB[(Kölcsönzési adatbázis)]
           MSG[Üzenetküldő]
       end
       U[Olvasó] --> WEB
       WEB --> SVC
       SVC --> DB
       SVC --> MSG
       MSG --> EMAIL[Email szolgáltatás]
       SVC --> AUTH[Hitelesítő szolgáltató]
       SVC --> KAT[Katalógus szolgáltatás]
   ```

4. **Döntési napló.**
   Válasszon ki egy fontos architekturális döntést (pl. a tranzakciókezelés módja a kölcsönzési folyamatban), és rögzítse ADR formában. Az ADR tartalmazza a kontextust, a döntést, a vizsgált alternatívákat, a következményeket és az állapotot.

   Példa egy ADR vázlatra:

   ADR-001: Kölcsönzési tranzakció kezelése

   Kontextus: A kölcsönzési folyamat során a készlet csökkentése és a kölcsönzési rekord létrehozása közötti konzisztencia kritikus. A rendszernek el kell kerülnie, hogy sikertelen készletcsökkentés esetén is létrejöjjön a rekord.

   Döntés: A kölcsönzési műveletet egyetlen ACID tranzakcióban, a „Kölcsönzési adatbázis”-ban hajtjuk végre, a készletkezelés logikáját a „Kölcsönzési szolgáltatás” komponensen belül tartva. Az email értesítést aszinkron módon, outbox pattern segítségével küldjük.

Alternatívák:

- Elosztott tranzakció (2PC): elutasítva a magas üzemeltetési komplexitás miatt.
- Saga minta: szükségtelen, mert a kritikus műveletek egyetlen adatbázisban történnek.

Következmények: Egyszerűbb konzisztenciamodell, de a kölcsönzési szolgáltatás és az adatbázis szoros kapcsolatban van. Az email küldés késhet.

Állapot: Elfogadva.

5. **Ellenőrzés.**
   A labor eredményét az alábbi szempontok szerint ellenőrizze:
   - Minden use case-nek van azonosított szereplője, fő forgatókönyve és legalább két kivételes útvonala.
   - Minden story rendelkezik elfogadási kritériumokkal, és legalább kettőben szerepel minőségi attribútum.
   - A context diagram jelöli a rendszerhatárt és a külső szereplőket.
   - A container diagramon minden komponensnek van neve, felelőssége és interfésze.
   - Az ADR tartalmazza a kontextust, döntést, alternatívákat, következményeket és állapotot.
   - A dokumentum verziózott, és a döntésekhez visszakereshető hivatkozás tartozik.

Várt eredmény és mért eredmény elkülönítése.
A labor várt eredménye egy olyan dokumentum, amely a fenti elemeket tartalmazza, és amelyből egy külső személy is megérti a rendszer főbb komponenseit és a döntések okát. A mért eredmény maga az elkészült dokumentum, amelyet a fenti ellenőrzési szempontok alapján lehet értékelni. Fontos, hogy a labor nem tartalmaz futó mérést vagy benchmarkot; az eredmény a dokumentáció minősége, nem pedig egy rendszer teljesítménye. Ha a labor során bármilyen számot (pl. válaszidőt) használunk, azt helyőrzőként vagy feltételezésként kell jelölni, nem pedig ellenőrzött mérésként.

7.2.9 6. Ellenőrzőlista

A fejezetben tárgyaltak elsajátításához az alábbi ellenőrzőlista használható:

- A use case-ek technológia-semlegesek, és üzleti célokat írnak le, nem megvalósítási részleteket.
- Minden use case-hez azonosítottuk az elsődleges, támogató és mögöttes szereplőket.
- A fő forgatókönyv mellett minden use case rendelkezik alternatív és kivételes útvonalakkal.
- A kivételes útvonalakból levezettük a szükséges hibatűrési, tranzakciós és kompenzációs mechanizmusokat.
- Minden user story rendelkezik elfogadási kritériumokkal, amelyek konkrétak, tesztelhetők és mérhetők.
- Az elfogadási kritériumokból kinyertük a releváns minőségi attribútumokat, és legalább egy quality attribute scenario-t megfogalmaztunk.
- A use case-ek és story-k alapján felvázoltuk a C4 context és container szintű diagramot, jelölve a rendszerhatárt.
- A fontosabb architekturális döntéseket ADR-ben rögzítettük, kontextussal, alternatívákkal és következményekkel.
- A dokumentáció nem tartalmaz valós hitelesítő adatot, belső URL-t vagy érzékeny adatot.
- A labor eredménye reprodukálható, és a várt eredmény elkülönül a mért eredménytől.

7.2.10 7. Önellenőrző kérdés

Miért nem elegendő az architekt számára egy use case fő sikerforgatókönyvének ismerete a rendszer megtervezéséhez? Magyarázza el egy konkrét kivételes útvonal példáján keresztül.

A válaszban arra érdemes kitérni, hogy a fő forgatókönyv csak a normál működést mutatja, de a rendszer megbízhatóságát, rendelkezésre állását és adatintegritását elsősorban a kivételek határozzák meg. Például ha a kölcsönzési folyamatban a fizetési szolgáltató nem elérhető, az architektnek döntést kell hoznia a timeout, retry és kompenzációs stratégia között, ami alapvetően befolyásolja a komponensek közötti interakció mintáit (szinkron hívás, aszinkron üzenet, circuit breaker). A kivételek hiánya ezért strukturális hiányosságokhoz vezet.

Ez az önellenőrző kérdés segít abban, hogy a tanuló ne csak elméletben ismerje a fogalmakat, hanem érvelni is tudjon a use case elemzés architekturális következményei mellett.

A fejezet itt véget ér. A következő alfejezet (Nem funkcionális követelmények és kényszerek) a nem funkcionális követelmények és kényszerek architekturális hatásával foglalkozik, építve az itt bevezetett minőségi attribútum fogalmakra.

7.3 Nem funkcionális követelmények és kényszerek

A nem funkcionális követelmények és kényszerek nem a rendszer „mit tud” kérdésére válaszolnak, hanem arra, hogy „milyen feltételek mellett, milyen minőségben, milyen korlátok között” kell működnie. Egy e-kereskedelmi rendszer funkcionális követelménye lehet, hogy a vásárló le tudja adni a rendelést; az NFR viszont az, hogy a rendelés leadása p95 válaszidőben legfeljebb 300 ms-ot vehet igénybe, az adatokat titkosítva kell tárolni, a rendszernek meg kell felelnie a GDPR-nak, és a backendet a szervezet által támogatott Java 17 futtatókörnyezetre kell telepíteni. Az első mondat funkcionális, a többi négy különböző kényszertípust képvisel: teljesítményt, biztonságot, jogi megfelelést és technológiát.

Ebben az alfejezetben ezt a négy kényszertípust járjuk körül. Megtanuljuk, hogyan lehet a homályos elvárásokból mérhető célértékeket és küszöböket képezni, hogyan szűkíti a kényszer a tervezési szabadságot, és milyen gyakorlati hibákat kell elkerülni a katalógus összeállításakor. A fejezet végén egy NFR-katalógus elkészítése a laborfeladat, amely a későbbi minőségi forgatókönyvek és KPI-meghatározás alapját adja.

7.3.1 1. A négy kényszertípus

A nem funkcionális követelmények két nagy csoportra oszthatók: **minőségi attribútumokra** és **kényszerekre**. A minőségi attribútumok a rendszer megfigyelhető tulajdonságai, például a válaszidő, az átbocsátás, a rendelkezésre állás, a karbantarthatóság. A kényszerek ezzel szemben olyan megkötések, amelyeket a környezet, a szabályozás, a szervezeti döntés vagy a technológiai örökség kényszerít ki. A kettő között gyakran nincs éles határ: egy teljesítménycél akkor válik kényszerré, ha egy ügyfél szerződésben rögzíti, vagy ha egy meglévő rendszer architektúrája nem enged meg bizonyos megoldásokat.

Az 1. ábra a nem funkcionális követelmények és kényszerek osztályozását mutatja.

```
flowchart TD
    A[Követelmény] --> B{Minőségi tulajdonság<br>vagy kényszer?}
    B -->|Minőségi tulajdonság| C[Teljesítmény]
    B -->|Minőségi tulajdonság| D[Biztonság]
    B -->|Külső kényszer| E[Jogi és megfelelés]
    B -->|Külső kényszer| F[Technológia]
    C --> G[Mérhető célérték<br>és küszöb]
    D --> G
    E --> H[Ellenőrzés<br>és dokumentáció]
    F --> H
```

1. ábra: A nem funkcionális követelmények és kényszerek osztályozása.
Jelmagyarázat: a nyilak az értelmezés irányát mutatják; a minőségi tulajdonságokhoz mérhető célérték, a külső kényszerekhez ellenőrzési és dokumentációs kötelezettség társul.
Feltételjegyzet: a teljesítmény és a biztonság lehet belső minőségi cél vagy külső kényszer; a jogi és technológiai kényszerek jellemzően külső forrásból származnak.

7.3.2 1.1 Teljesítmény kényszerek

A teljesítmény kényszerek a rendszer időbeli viselkedését, erőforrás-felhasználását és kapacitását írják elő. Ide tartoznak:

- **válaszidő (latencia)**: mennyi idő alatt válaszol a rendszer egy kérésre;
- **átbocsátás**: hány kérést, tranzakciót vagy üzenetet tud feldolgozni időegységenként;
- **erőforrás-korlátok**: processzor-, memória-, hálózati sávszélesség- és tárolókapacitás-korlátok;
- **skálázhatósági elvárások**: milyen terhelésnövekedés mellett kell a rendszernek elfogadható válaszidőt nyújtania;
- **rendelkezésre állás**: milyen időtartamban kell a rendszernek elérhetőnek lennie, mekkora állásidő megengedett.

Egy jól megfogalmazott teljesítménykényszer mindig tartalmazza a mérési környezetet és a terhelési feltételeket. A gyakorlatban a percentilisek használata terjedt el, mert az átlag elrejti a szélsőséges értékeket. Például a következő kényszer:

> „A termékkatalógus-lekérdező API p95 válaszideje legfeljebb 200 ms lehet, hétköznap 10:00 és 12:00 között, 1000 kérés/perc terhelés mellett, a termelési környezetben.”

Ez a megfogalmazás rögzíti az ingert (termékkatalógus-lekérdezés), a környezetet (produkciós, hétköznap 10:00–12:00, 1000 req/perc), a választ (p95 válaszidő) és a mérőszámot (200 ms). Az ilyen kényszerek alapján lehet később SLO-t és hibakeretet számolni; erre a későbbi fejezetekben térünk ki.

A teljesítménykényszerek tervezési hatása közvetlen: egy szigorú válaszidő-korlát kizárhatja a szinkron, lassú adatbázis-lekérdezéseket, és szükségessé teheti gyorstár, aszinkron feldolgozás vagy denormalizált olvasási modell bevezetését. Ha a követelmény nem mérhető, akkor nem lehet eldönteni, hogy egy adott architektúra megfelel-e.

7.3.3 1.2 Biztonsági kényszerek

A biztonsági kényszerek a rendszer adatainak, szolgáltatásainak és felhasználóinak védelmét írják elő. A klasszikus CIA-hármas (bizalmasság, sértetlenség, rendelkezésre állás) mellett ide tartoznak az azonosítási, hitelesítési, engedélyezési, naplózási és titkosítási követelmények.

Példa biztonsági kényszerre:

> „Minden vásárlói személyes adatot AES-256 vagy erősebb titkosítással kell tárolni nyugalmi állapotban. A hozzáférési kísérleteket naplózni kell, és a naplókat legalább 365 napig meg kell őrizni.”

Ez a kényszer több dolgot is meghatároz:

- kötelező titkosítási algoritmust és kulcshosszt;
- naplózási kötelezettséget;
- megőrzési időt;
- ezáltal befolyásolja az adatbázis-választást, a kulcskezelést és az üzemeltetési folyamatokat.

A biztonsági kényszerek gyakran ütköznek a teljesítménnyel: az erős titkosítás, a minden kérésre kiterjedő autorizáció és a részletes naplózás növeli a válaszidőt és az erőforrás-felhasználást. Az architekt feladata ezeket az ütközéseket láthatóvá tenni, és prioritással, kompromisszummal feloldani.

Biztonsági kényszer esetén különösen fontos a **támadási felület** és a **bizalmi határok** ismerete. Például egy internetről elérhető API más biztonsági követelményeket támaszt, mint egy csak belső hálózaton futó batch-feldolgozó. A biztonsági követelményeket nem szabad egyetlen homályos mondattal elintézni („a rendszer legyen biztonságos”), mert akkor a tervező nem tudja, milyen vezérléseket kell beépíteni.

7.3.4 1.3 Jogi és megfelelési kényszerek

A jogi kényszerek törvényből, rendeletből, ágazati előírásból vagy szerződéses kötelezettségből származnak. Az Európai Unióban ilyen például az általános adatvédelmi rendelet (GDPR), a hálózati és információs rendszerek biztonságáról szóló NIS2 irányelv, a pénzügyi szektorban a digitális működési rezilienciáról szóló DORA rendelet, vagy a mesterséges intelligenciáról szóló AI Act. A pontos követelmények mindig a hivatalos, hatályos szövegből olvashatók ki; ez a könyv nem jogi tanácsadás, ezért a konkrét cikkelyekre való hivatkozás előtt mindig ellenőrizni kell az aktuális jogszabályt.

Jogi kényszer például:

> „A felhasználóknak joguk van személyes adataik törlését kérni. A törlési kérelmet 30 napon belül teljesíteni kell, és a teljesítésről igazolást kell kiállítani.”

Ez a kényszer nem technológiai jellegű, de komoly architekturális következményei vannak:

- az adatokat úgy kell tárolni, hogy azok egy adott személyhez köthetők és törölhetők legyenek;
- a biztonsági mentésekből is törölhetőnek kell lenniük, vagy anonimizálhatónak;
- a törlési folyamatot naplózni kell;
- a különböző alrendszerek közötti adatreplikációnak is követnie kell a törlést.

A jogi kényszer gyakran kényszeríti ki az **adatbesorolást**, az **adatmegőrzési időket**, a **hozzáférési naplók** részletességét és a **titkosítási** előírásokat. Architektúra szempontból ezek nem opcionális minőségi célok, hanem kötelező megkötések, amelyek megszegése jogi és üzleti kockázatot jelent.

A jogi kényszerek kezelésének kulcsa a **nyomon követhetőség**: minden ilyen követelményhez hozzá kell rendelni a forrást (például rendelet száma, szerződéspont), a felelőst (jogi, adatvédelmi tisztviselő), az érintett adatkört és az ellenőrzés módját. Ez az információ az audit során is elengedhetetlen.

7.3.5 1.4 Technológiai kényszerek

A technológiai kényszerek meghatározzák, hogy a rendszert milyen platformon, milyen programozási nyelven, milyen adatbázissal, milyen infrastruktúrán kell megvalósítani vagy üzemeltetni. Forrásuk lehet:

- **szervezeti szabvány**: a vállalat csak bizonyos technológiákat támogat;
- **meglévő rendszerek**: az új komponensnek integrálódnia kell egy régi monolithoz;
- **licenc- és költségkorlát**: egy adatbázis licenszdíja meghaladja a keretet;
- **üzemeltetési képesség**: a csapat csak Kubernetesen tud üzemeltetni;
- **jogszabályi előírás**: például adott régióban kell tárolni az adatokat.

Példa technológiai kényszerre:

> „A rendszernek a belső platformon kell futnia, Kubernetes 1.28-as klaszteren, a backend szolgáltatásokat Java 17-tel kompatibilis konténerként kell csomagolni."

Ez a kényszer kizárja például a serverless megoldást, a Python alapú backendet vagy a saját üzemeltetésű virtuális gépet. Ettől még a megoldási térben belül marad számos szabadság: lehet monolit vagy mikroszolgáltatás, lehet szinkron vagy aszinkron, lehet relációs vagy dokumentum-adatbázis.

A technológiai kényszer és a **preferencia** közötti különbség fontos. Ha valaki azt mondja, hogy „szeretem a Rustot, ezért Rustban kell írni", az legfeljebb preferencia, nem kényszer – hacsak nincs mögötte szervezeti döntés, csapatszakértelem vagy támogatási szerződés. Az architekt felelőssége, hogy a kényszerként bemutatott preferenciákat felismerje, és a döntési naplóban rögzítse a valódi indokot.

A technológiai kényszer másik gyakori formája a **vendor lock-in** elkerülése vagy elfogadása. Például egy multi-cloud követelmény technológiai kényszert jelent a felhőspecifikus szolgáltatások használatára. Ilyenkor a kényszer és a szabadság egyensúlyát kell megtalálni.

7.3.6 2. Mérhető célértékek és küszöbök

A nem funkcionális követelmények akkor használhatók, ha mérhetővé tesszük őket. A homályos kifejezések („gyors", „skálázható", „biztonságos", „felhasználóbarát") nem adnak támpontot a tervezéshez és az ellenőrzéshez. A mérhető célértékek megadásához a **SMART** kritériumokat érdemes alkalmazni:

- **Specific (specifikus)**: pontosan melyik műveletre, melyik komponensre vonatkozik;
- **Measurable (mérhető)**: számszerűsíthető, mértékegységgel;
- **Achievable (elérhető)**: az adott környezetben reális;
- **Relevant (releváns)**: az üzleti célhoz kapcsolódik;
- **Time-bound (időhöz kötött)**: milyen mérési ablakban értendő.

A mérhető NFR megadásához négy összetevőt kell rögzíteni:

1. **Inger**: mi váltja ki a rendszer viselkedését (például HTTP GET /orders).
2. **Környezet**: milyen feltételek mellett mérünk (terhelés, napszak, környezet).
3. **Válasz**: mit kell a rendszernek tennie, milyen tulajdonsággal (például p95 válaszidő).
4. **Mérőszám**: a megengedett érték és a tűréshatár.

Ez a szerkezet megegyezik a későbbi fejezetben tárgyalt quality attribute scenario alapformájával, de itt a hangsúly a kényszerként való megfogalmazáson van.

Példa mérhető teljesítménycélra:

Mező	Érték
Inger	Order API POST /v1/orders hívása
Környezet	termelés, hétköznap 8:00–20:00, 1500 kérés/perc
Válasz	a kérés feldolgozása és válasz elküldése
Mérőszám	p95 < 300 ms; p99 < 800 ms
Mérési ablak	30 nap gördülő

A **küszöb** azt a határértéket jelöli, amelynél a rendszer még elfogadható. A küszöb lehet **kemény (hard)** vagy **puha (soft)**. A kemény küszöb átlépése azonnali beavatkozást igényel, a puha küszöb figyelmeztetést ad, de a rendszer még működhet. Például:

- Puha küszöb: p95 válaszidő > 250 ms – riasztás a csapatnak.
- Kemény küszöb: p95 válaszidő > 350 ms – automatikus skálázás vagy forgalomkorlátozás.

A küszöböket a **hibakeret** (error budget) számításához is használjuk, amelyről a megbízhatósági fejezetben lesz szó. Itt elegendő annyit rögzíteni, hogy a küszöb nem statikus: a terhelés, az üzleti ciklus és a technológiai változások függvényében idővel módosulhat, ezért a katalógusban verziókezelni kell.

7.3.7 3. Kényszerek és tervezési szabadság

A kényszerek első pillantásra csak korlátoznak, valójában azonban **csökkentik a döntési bizonytalanságot**, és ezzel gyorsítják a tervezést. Ha tudjuk, hogy a rendszernek 99,9%-os rendelkezésre állást kell nyújtania, akkor kizárhatjuk az egyszerű egygépes telepítést, és olyan architektúrát választunk, amely támogatja a redundanciát és az automatikus feladatátvételt. Ha tudjuk, hogy a jogi kényszer miatt az adatoknak az EU-ban kell maradniuk, akkor nem kell mérlegelni más régiókat.

Ugyanakkor a kényszerek **ütközhetnek** egymással. Például:

- Az erős titkosítás növeli a válaszidőt, ami ütközhet a teljesítménykényszerrel.
- A jogi kényszer (például naplózás minden hozzáférésről) növeli a tárolási költséget, ami ütközhet a költségkerettel.
- A technológiai kényszer (például csak Java) kizárhat egy olyan keretrendszert, amely jobb teljesítményt adna.

Az architekt szerepe, hogy ezeket a konfliktusokat **láthatóvá tegye**, és a döntési naplóban rögzítse a választott kompromisszumot. A prioritás nem azt jelenti, hogy az alacsonyabb prioritású kényszert figyelmen kívül hagyjuk, hanem hogy ütközés esetén melyik élvez elsőbbséget.

A kényszer és a szabadság viszonyát jól mutatja a **megoldási tér** fogalma. A kényszerek szűkítik a lehetséges architektúrák halmazát, de nem határozzák meg egyértelműen a megoldást. Ha túl sok a kényszer, a tér üres lehet; ha túl kevés, a tervező döntési paralízisbe kerül. A gyakorlatban az NFR-katalógus éppen ezt az egyensúlyt rögzíti.

7.3.8 4. Labor: NFR-katalógus készítése prioritással

A labor célja, hogy egy adott rendszerhez összeállítsunk egy olyan NFR-katalógust, amely minden kényszert mérhető formában tartalmaz, prioritással és forrásmegjelöléssel. Az átadandó eredmény egy reprodukálható dokumentum (Markdown vagy Jupyter notebook), amely verziózott, és a döntési naplóval együtt visszakereshető.

Kiinduló állapot
Egy kitalált „rendeléskezelő szolgáltatás” funkcionális követelményeit ismerjük: a felhasználó leadhat rendelést, lekérdezheti a rendelés állapotát, a termékkatalógus kereshető, és a rendszer napi zárást készít. A feladat az NFR-ek meghatározása ezekhez a funkciókhoz.

Elvárt eredmény
Egy táblázat, amelyben minden NFR-hez szerepel:

- azonosító (NFR-01, NFR-02, …);
- a követelmény megfogalmazása mérhető formában;
- típusa (teljesítmény, biztonság, jogi, technológiai);
- forrása (ügyfélszerződés, GDPR, belső szabvány, platformcsapat);
- prioritása (kritikus, magas, közepes, alacsony);
- mérhető célérték és küszöb;
- ellenőrzési módszer (terheléses teszt, naplóelemzés, audit);
- a jelenlegi mért érték vagy „még nem mért” jelölés (a várt és a mért különválasztása).

Példa a táblázat első soraira:

ID	NFR megfogalmazás	Típus	Forrás	Prioritás	Célérték	Küszöb	Ellenőrzés	Mért érték
NFR-01	A rendelés leadása POST /orders hívás p95 válaszideje termelésben, hétköznap 8–20 óra, 1500 kérés/perc mellett 300 ms.	Teljesítmény	Ügyfélszerződés 3.2	Kritikus	p95 < 300 ms	p95 > 250 ms figyelmeztet, p95 > 350 ms beavatkozás	Terheléses teszt, APM	még nem mért

ID	NFR megfogalmazás	Típus	Forrás	Prioritás	Célérték	Küszöb	Ellenőrzés	Mért érték
NFR-02	A vásárlói adatokat nyugalmi állapotban AES-256 titkosítással kell tárolni.	Biztonság	Belső biztonsági szabvány	Magas	100% titkosított mező	nincs	Konfigurációellenőrzés, adatbázis-audit	még nem mért
NFR-03	A törlési kérelmeket 30 napon belül teljesíteni kell, és a törlést naplózni kell.	Jogi	GDPR 17. cikk	Kritikus	30 nap	25 nap figyelmeztet	Folyamat-audit	még nem mért
NFR-04	A backend szolgáltatásnak Java 17-tel kompatibilisnek kell lennie, és a belső Kubernetes klaszteren kell futnia.	Technológiai	Platformcsapat	Magas	Java 17, K8s 1.28	nincs	CI ellenőrzés	jelenleg Java 17, K8s 1.27

Lépések

1. **Gyűjtés:** a funkcionális követelményekből kiindulva azonosítsa a minőségi elvárásokat és a

külső kényszereket. Kérdezzen rá: mi történik, ha lassú? Mi történik, ha adat szivárog? Milyen jogszabály vonatkozik rá?
2. **Típusba sorolás:** minden NFR-t soroljon a négy típus valamelyikébe.
3. **Forrás hozzárendelése:** minden NFR mellé írja oda, honnan származik (szerződés, szabvány, jogszabály, szervezeti döntés).
4. **Prioritás meghatározása:** ütközés esetén melyik a fontosabb? Használjon MoSCoW-t vagy egyszerű kritikus/magas/közepes/alacsony skálát.
5. **Mérhetővé tétel:** minden NFR-t fogalmazzon át úgy, hogy tartalmazza az ingert, a környezetet, a választ és a mérőszámot. Ha nem lehet mérhetővé tenni, jelölje meg, miért.
6. **Küszöbök megadása:** határozza meg a figyelmeztetési és a beavatkozási küszöböt, ha értelmezhető.
7. **Ellenőrzési módszer:** adja meg, hogyan fogják ellenőrizni a teljesülést.
8. **Mért érték elkülönítése:** ha van jelenlegi mérés, azt tüntesse fel; ha nincs, írja be, hogy „még nem mért”. Ez megakadályozza, hogy a várt célértéket tényként kezeljék.
9. **Döntési napló:** rögzítse, mely NFR-ek ütköztek, miért választották az adott prioritást, milyen alternatívákat vetettek el.

Döntési napló példa

> **Döntés:** NFR-01 prioritása kritikus, mert az ügyfélszerződés késedelmi kötbért ír elő, ha a rendelés leadása 500 ms fölé emelkedik. Alternatívaként felmerült a p99 használata, de a csúcsforgalomban a p95 jobban jellemzi a felhasználói élményt, ezért a p95 mellett döntöttünk. Az NFR-02 és NFR-04 ütközését a platformcsapat oldotta fel: a Java 17 támogatja az AES-256 titkosítást a beépített kriptográfiai szolgáltatóval, így nincs szükség külső könyvtárra.

Ellenőrzési szempontok

- Minden NFR-nek van azonosítója, típusa, forrása és prioritása.
- Egyetlen NFR sem tartalmaz homályos kifejezést („gyors”, „biztonságos”, „felhasználóbarát”).
- Minden mérhető NFR tartalmaz mértékegységet és mérési ablakot.
- A várt célérték és a jelenlegi mért érték egyértelműen elkülönül.
- A döntési napló bejegyzéseiből visszakereshető a prioritási sorrend indoka.

Visszaállítási út
A katalógus dokumentum verziózott (például git repositoryban). A labor során minden módosítást külön commitban kell elvégezni, így a korábbi állapot bármikor visszaállítható. A döntési napló a katalógussal együtt, ugyanabban a repositoryban tárolandó.

7.3.9 5. Tipikus hibák és helyes megoldások

A gyakorlatban számos hiba csúszhat az NFR-ek kezelésébe. Az alábbi táblázat a leggyakoribbakat és a javított megfogalmazást mutatja.

Hiba	Következmény	Helyes megoldás
Homályos NFR: „a rendszer legyen gyors”	nem lehet tesztelni, viták a teljesítésről	p95 válaszidő < 300 ms adott terhelésnél
NFR-ek prioritás nélkül	ütközés esetén ad hoc döntés születik	priorizált NFR-katalógus
Kényszer és preferencia összemosása	indokolatlan technológiai megkötés, rossz döntés	forrás és indok rögzítése az ADR-ben

Hiba	Következmény	Helyes megoldás
Mérési ablak hiánya	a mért érték félrevezető (pl. éjszakai mérés csúcsidőre)	környezet, terhelés, időablak megadása
Jogi kényszer figyelmen kívül hagyása	büntetés, adatvédelmi incidens	jogi forrás hozzárendelése, audit
Túl szigorú küszöb mindenre	blokkolja a fejlesztést, irreális elvárás	puha és kemény küszöbök, hibakeret
A mért és a várt érték összemosása	hamis megfelelésérzet	külön oszlop a táblázatban

Példa hibás és javított NFR-re

Hibás:
> „A rendszernek mindig elérhetőnek kell lennie.”

Javított:
> „A rendeléskezelő API rendelkezésre állása negyedéves ablakban legalább 99,9% kell legyen. A tervezett karbantartás havi 1 óra lehet, amely nem számít bele az állásidőbe. A mérés a terheléselosztó naplói alapján történik.”

Ez a javítás rögzíti a mutatót, az ablakot, a kivételt és a mérés forrását, így egyértelművé válik a megfelelés.

7.3.10 6. Összefoglalás

Ebben az alfejezetben a nem funkcionális követelmények és kényszerek négy alapvető típusát tekintettük át: a teljesítményt, a biztonságot, a jogi megfelelést és a technológiát. Láttuk, hogy a kényszer nemcsak korlátoz, hanem irányt is ad a tervezésnek, ha mérhető célértékekkel és küszöbökkel együtt fogalmazzuk meg. A labor során elkészített NFR-katalógus a későbbi minőségi forgatókönyvek, SLO-k és KPI-ok alapja lesz: a következő alfejezet (KPI-ok, mérőszámok és követelmény-kézbesítés) a mutatók és a követelmény-kézbesítés kapcsolatát vizsgálja, ezért érdemes a katalógust megőrizni.

7.3.11 7. Ellenőrzőlista

- Az NFR-ek a négy típus valamelyikébe sorolhatók: teljesítmény, biztonság, jogi, technológiai.
- Minden NFR-nek van forrása, prioritása és mérhető célértéke.
- A mérhető cél tartalmazza az ingert, a környezetet, a választ és a mérőszámot mértékegységgel és mérési ablakkal.
- A küszöbök puha és kemény szinten is meg vannak határozva, ahol értelmezhető.
- A kényszerütközések dokumentálva vannak a döntési naplóban.
- A mért és a várt érték külön oszlopban szerepel.
- Nincsenek homályos kifejezések a katalógusban.

7.3.12 8. Önellenőrző kérdés

Egy megrendelő azt mondja: „A mobilalkalmazásunk legyen gyors, és ne lehessen feltörni.” Hogyan alakítaná át ezt a két elvárást mérhető NFR-katalógussá? Milyen kényszerforrásokat és prioritásokat venne figyelembe, és milyen kérdéseket tenne fel a megrendelőnek a pontosításhoz?

7.4 KPI-ok, mérőszámok és követelmény-kézbesítés

A követelmények és az üzleti mérőszámok összekapcsolása az architektúratervezés egyik legfontosabb, mégis gyakran elhanyagolt területe. A funkcionális követelményeket rendszerint pontosan leírják, a nem funkcionális követelményeket is egyre gyakrabban számszerűsítik, de a kettő közötti kapcsolatot a mérhető üzleti eredményekkel ritkán teszik egyértelművé. Ez a fejezet azt a kérdést járja körül, hogyan kerülhetők a KPI-ok (Key Performance Indicators, kulcsteljesítmény-mutatók) és a mérőszámok a követelmények középpontjába, hogyan különböztethetők meg a vezető (leading) és a lezáró (lagging) indikátorok, és hogyan kezelhető a követelmények változása anélkül, hogy a mérési rendszer hamis képet mutasson.

A fejezet célja, hogy az olvasó a végére képes legyen egy kis, ellenőrizhető példán alkalmazni a bemutatott eszközöket, felismerje a tipikus hibákat, és megindokolja a helyes megoldást. A labor során KPI- és metrikatervet készítünk, amely a későbbi fejezetekben a teljesítmény, a rendelkezésre állás és a minőségi attribútumok mérésének alapját adja.

7.4.1 Alapfogalmak: mérőszám, metrika, KPI

Mielőtt a követelmények és a mérőszámok összekapcsolásának módszereire térnénk, tisztázzuk az alapfogalmakat. A **metrika** (vagy mérőszám) bármilyen számszerű vagy kategorikus megfigyelés, amely a rendszer vagy folyamat valamely tulajdonságát jellemzi. Például az API válaszideje, a napi aktív felhasználók száma, a kódlefedettség, az átlagos rendelési érték. A metrika önmagában nem hordoz célértéket és nem feltétlenül kapcsolódik üzleti célhoz; egyszerűen egy mérhető tény.

A **KPI** ezzel szemben olyan metrika, amelyet közvetlenül egy kritikus üzleti vagy technikai célhoz rendelünk, és amelyhez célérték, időablak és felelős tartozik. A KPI nem egyszerűen mérés, hanem döntési fókusz: azt mutatja meg, hogy a szervezet vagy a rendszer eléri-e a kitűzött célt. Minden KPI metrika, de nem minden metrika KPI. A KPI kiválasztása mindig kontextust és célt feltételez.

Vegyünk egy példát. A „napi aktív felhasználók száma” metrika. Akkor válik KPI-vé, ha például a termékstratégia célja a felhasználói elköteleződés növelése, és ehhez a következő negyedévre 20%-os növekedést tűzünk ki. Ha a mérőszámot csak információs célból nézzük, és nem kapcsolódik hozzá döntési küszöb vagy cél, akkor metrika marad. A KPI tehát a metrika célhoz kötött formája.

A jó KPI jellemzőit gyakran a SMART betűszóval foglaljuk össze: - **S**pecifikus: pontosan meghatározott, nem általános. - **M**érhető: számszerűsíthető, egyértelmű mértékegységgel. - **E**lérhető (Achievable): reális, de kihívást jelentő. - **R**eleváns: közvetlenül kapcsolódik a célhoz. - **T**ime-bound: időhöz kötött, van mérési ablak és határidő.

Ezen túlmenően a szoftverarchitektúra kontextusában a KPI-ok esetében elengedhetetlen a mérési környezet, a mérési ablak és a számítási mód rögzítése. A dokumentációs szerződés szerint a mérőszámokat mértékegységgel és mérési ablakkal kell megadni. Például a „rendelkezésre állás” önmagában nem elég; azt is meg kell mondani, hogy a mérés 99,9% fölötti hányadra vonatkozik, 30 napos gördülő ablakban, a produkciós környezetben, és a leállások közé beleszámítjuk a tervezett karbantartást is. Hasonlóképpen a konverziós arányt százalékban, a munkamenetek számához viszonyítva, havi bontásban adjuk meg.

A KPI-ok és a metrikák közötti különbségtétel azért fontos, mert a túl sok KPI elveszi a fókuszt: ha minden mérőszámot KPI-nak nevezünk, a csapat nem tudja, melyikhez igazítsa a

döntéseit. Általában egy rendszer vagy termék szintjén 3–5 jól megválasztott KPI elegendő; a többi mérőszám támogató szerepet játszik.

7.4.2 A követelmények és az üzleti mérőszámok összekapcsolása

A követelményeket gyakran funkciólistaként kezeljük: a termékmenedzser vagy az üzleti elemző leírja, hogy mit kell a rendszernek tudnia, a csapat pedig ezt megvalósítja. Azonban a szoftver értéke a használatban és a változásban realizálódik. Egy funkció akkor sikeres, ha a felhasználók valóban használják, és ha ez a használat mérhetően javítja az üzleti eredményt vagy a felhasználói élményt. Ezért minden kulcsfontosságú követelményt érdemes hipotézisként kezelni: „Ha megvalósítjuk X-et, akkor Y mérőszám Z mértékben változik, mert …”. Ezt a hipotézist méréssel igazoljuk vagy cáfoljuk.

Az összekapcsolás egyik hatékony eszköze az **impact mapping** (hatástérkép). Ez egy egyszerű, négyszintű ábra, amely a céltól a szoftverkövetelményekig vezet le egy oksági láncot:

1. **Cél (Goal):** a magas szintű üzleti cél, például „növeljük az online rendelések számát 15%-kal a negyedévben”.
2. **Érintett szereplők (Actors):** azok, akiknek a viselkedése befolyásolja a célt, például a regisztráció nélküli látogató, a visszatérő vásárló, az ügyfélszolgálat.
3. **Viselkedésváltozások (Impacts):** az a változás, amit el szeretnénk érni, például a látogató gyorsabban végigmegy a pénztáron, a visszatérő vásárló kevesebb hibát tapasztal.
4. **Szoftverkövetelmények (Deliverables):** a konkrét funkciók, amelyekkel a viselkedésváltozást elérjük, például egykattintásos rendelés, cím-előtöltés, vendégkijelentkezés.

Az impact mapping előnye, hogy a lánc elemei oda-vissza ellenőrizhetők. Ha egy követelmény nem köthető semmilyen viselkedésváltozáshoz, akkor valószínűleg felesleges. Ha egy viselkedésváltozás nem kapcsolódik a célhoz, akkor a cél rosszul van meghatározva. A hatástérkép így közvetlenül a KPI-ok kijelöléséhez vezet: minden viselkedésváltozáshoz rendelhetünk egy mérőszámot.

Tegyük fel, hogy a cél a rendelési konverzió növelése. A hatástérképen a látogatóhoz tartozó viselkedésváltozás: „a pénztár oldal elhagyási aránya csökken”. Ehhez a viselkedésváltozáshoz mérőszámként a pénztár befejezési arányt (pénztárba lépők közül rendelők aránya) rendeljük. A szoftverkövetelmény: „a pénztár oldal betöltési ideje p95 alatt 2 másodperc legyen”, és ennek a követelménynek a hatását a befejezési arányon mérjük. Így a követelmény nem öncélú teljesítménycél, hanem eszköz a konverzió növeléséhez.

A másik gyakran használt eszköz a **KPI-fa** vagy céllebontási fa. Itt a magas szintű üzleti célt részterületekre bontjuk, és mindegyikhez mérőszámot rendelünk. Például egy szolgáltatás bevétele függ a rendelések számától és az átlagos rendelési értéktől; a rendelések száma függ a látogatók számától és a konverziós aránytól; a konverziós arány függ a pénztárfolyamat gyorsaságától és hibamentességétől. A fa segítségével látható, hogy egy-egy rendszerkövetelmény melyik ághoz járul hozzá, és hol érdemes beavatkozni.

A **nyomon követhetőség** (traceability) biztosítja, hogy minden kulcsfontosságú követelményhez tartozik legalább egy mérőszám, és minden KPI-hoz tartozik legalább egy követelmény. Ez lehet egy egyszerű táblázat, egy modellben tárolt kapcsolat, vagy a követelménykezelő eszközben rögzített hivatkozás. A döntési naplóban (ADR) rögzítjük, hogy miért választottuk az adott KPI-t, milyen alternatívákat vetettünk el, és milyen következményekkel jár a választás.

A követelmény és a KPI közötti kapcsolatot nem szabad utólag, a fejlesztés végén megteremteni. A KPI-oknak már a követelmények elemzésekor definiálva kell lenniük, mert ezek adják a későbbi mérési terv alapját. Ha a KPI-okat utólag próbáljuk hozzárendelni, gyakran kiderül, hogy a szükséges adatokat nem gyűjtöttük, vagy a mérés nem reprodukálható.

7.4.3 Leading és lagging indikátorok

A KPI-ok egyik legfontosabb megkülönböztetése a **leading** (vezető, előrejelző) és a **lagging** (lezáró, követő) indikátorok közötti különbség.

A **lagging indikátor** olyan mérőszám, amely az eredmény bekövetkezte után mutatja meg a teljesítményt. Jellemzően hosszabb idő alatt gyűlik össze, és mire láthatóvá válik, már nem tudunk rajta közvetlen beavatkozással változtatni. Például a negyedéves bevétel, az éves ügyfél-elégedettségi mutató, a havi rendelések száma. A lagging indikátor kiváló a cél elérésének igazolására, de rossz az időbeni beavatkozáshoz: ha a negyedév végén derül ki, hogy a bevétel 20%-kal elmaradt a tervtől, akkor az adott negyedéven már nem tudunk segíteni.

A **leading indikátor** ezzel szemben olyan mérőszám, amely a jövőbeli eredményt jelzi előre, és amelyre a csapat gyorsan tud reagálni. A leading indikátor gyakran egy folyamat vagy minőségi jellemző, amely ok-okozati kapcsolatban áll a lagging eredménnyel. Például a pénztár oldal betöltési ideje, a hibás kijelentkezések száma, az új funkciót használók aránya a bevezetés utáni első héten. Ha a pénztár oldal betöltési ideje növekedni kezd, az előre jelzi a konverzió csökkenését, még mielőtt a havi konverziós arány romlana. A leading indikátor lehetővé teszi a korai beavatkozást: optimalizáljuk a betöltési időt, mielőtt a bevétel visszaesik.

Fontos megérteni, hogy a leading/lagging megkülönböztetés **kontextusfüggő**: ugyanaz a mérőszám lehet leading az egyik célhoz és lagging a másikhoz. A „rendelési konverziós arány" például lagging a „marketing kampány" sikeréhez képest, hiszen a kampány elindítása után csak némi késéssel látjuk a hatását. Ugyanakkor leading a „negyedéves bevétel" szempontjából, mert ha a konverzió romlik, az később a bevételben is megjelenik. Az indikátorok besorolása tehát mindig a vizsgált célhoz viszonyítva értelmezendő.

A gyakorlatban a KPI-rendszernek **mindkét típust** tartalmaznia kell. A leading indikátorok segítenek a korai beavatkozásban, a lagging indikátorok pedig a végső eredményt igazolják. Ha csak lagging indikátorokat használunk, akkor csak hónapok múlva vesszük észre a problémát; ha csak leadinget, akkor könnyen elveszünk a részletekben, és nem látjuk, hogy valójában elértük-e a célt. A vezetői riportokban gyakran a lagging indikátorok dominálnak, míg a mérnöki csapat műszerfalán a leading indikátoroknak kell hangsúlyosnak lenniük.

Nézzünk egy konkrét példát egy e-kereskedelmi rendszerre. A cél a rendelési konverzió 10%-os növelése a következő negyedévben. A lagging KPI: havi rendelések száma (db/hónap, naptári hónap). A leading KPI-ok: - pénztár befejezési arány (%, heti gördülő átlag), - pénztár oldal betöltési idő p95 (ms, naponta, 30 napos ablak), - az új funkciót használók aránya (%, a bevezetés utáni első hét).

A leading indikátorok közül a pénztár befejezési arány közvetlenül kapcsolódik a konverzióhoz: ha a befejezési arány csökken, a konverzió is csökkenni fog. A betöltési idő pedig a befejezési arányra hat, tehát annak leading indikátora. Így többszintű leading-lagging láncot építhetünk, amely lehetővé teszi a probléma gyökerének azonosítását.

A leading indikátorok kiválasztásánál ügyelni kell arra, hogy az ok-okozati kapcsolat valós legyen, ne csak feltételezett. Ha például a betöltési idő és a konverzió között nincs statisztikailag kimutatható kapcsolat, akkor a betöltési idő rossz leading indikátor. Ezért a hipotéziseket adatokkal kell alátámasztani, és a mérési eredmények alapján időnként felül kell vizsgálni a KPI-rendszert.

7.4.4 A követelményváltozás kezelése

A követelmények ritkán statikusak. Változhatnak üzleti okból (új piaci lehetőség, versenytárs lépése), technikai okból (új komponens bevezetése, elavult technológia), szabályozási okból (adatvédelmi előírás, iparági szabvány), vagy a felhasználói visszajelzések alapján. A változás természetes, de hatással van a KPI-okra és a mérési tervre. Ezért a követelményváltozást is architekturális döntésként kell kezelni: rögzíteni, elemezni a hatását, és újraértékelni a mérőszámokat.

A változáskezelés lépései a következők:

1. **A változási igény rögzítése.** Dokumentáljuk, hogy ki kérte a változást, mi az oka, mit szeretne elérni, és milyen határidővel. Ez lehet egy változási kérelem (change request) vagy egy ADR bevezetője.
2. **Hatáselemzés.** Meghatározzuk, hogy a változás mely követelményeket érinti, mely komponenseket, és mely KPI-ok válnak érvénytelenné vagy változnak meg. A hatáselemzés során megvizsgáljuk a leading-lagging láncokat is: ha egy leading indikátor függ a változó követelménytől, akkor azt is frissíteni kell.
3. **Alternatívák feltárása.** Ritkán csak egyféleképpen lehet megvalósítani egy követelményváltozást. Az alternatívákat költség, kockázat és a KPI-okra gyakorolt hatás szerint értékeljük.
4. **Döntés.** A választott megoldást ADR-ben rögzítjük, feltüntetve a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Az ADR-nek tartalmaznia kell a KPI-ok módosítását is.
5. **A KPI- és metrikaterv frissítése.** A régi mérőszámokat lezárjuk, az újakat bevezetjük, és a bázisértékeket újra meghatározzuk. Ha a változás miatt a korábbi célértékek már nem reálisak, akkor a célokat is újra kell tárgyalni – nem a mérést manipuláljuk, hanem a célt igazítjuk a realitáshoz.

Példa a változásra: az e-kereskedelmi rendszerben bevezetik a kötelező regisztrációt a rendelés előtt. Ez a követelmény megváltoztatja a vendégkijelentkezés folyamatát, és hatással van a konverziós arányra. A korábbi KPI (konverziós arány) értelmezése megváltozik: mostantól a regisztrált felhasználók körét is figyelembe kell venni, és a vendégkijelentkezés használati aránya értelmetlenné válik. A leading indikátorok közé bekerülhet a regisztrációs űrlap befejezési aránya, mert ha az alacsony, az előre jelzi a konverzió romlását. A változást ADR-ben dokumentáljuk, és a mérési tervet frissítjük. A konverziós célértéket újra kell tárgyalni, mert a regisztrációs súrlódás valószínűleg csökkenti a konverziót; a 10%-os növekedés helyett lehet, hogy 5% a realisztikus.

A változások kezelésénél fontos, hogy a KPI-okat **ne szépítsük meg**. Ha a cél teljesíthetetlenné válik, akkor újra kell tárgyalni a célt, nem a mérést manipulálni. A mérőszámoknak tükrözniük kell a valóságot, nem a kívánt képet. A mérési tervben ezért rögzíteni kell a mérési módszert, a mintavételt és az adatforrást, hogy a mérés független legyen a változásoktól.

A követelményváltozásokat érdemes a döntési naplóban is nyomon követni. Minden változási kérelemhez kapcsolható egy ADR, amelyben rögzítjük a változás okát, a hatását a KPI-okra, a választott megoldást és a következményeket. Így a jövőbeli olvasó megérti, miért változott a mérési rendszer, és milyen alternatívákat vetettünk el.

7.4.5 Tipikus hibák

A KPI-ok és a mérőszámok bevezetésekor számos csapda vár a csapatokra. Ezek ismerete segít elkerülni a félrevezető mérési rendszert.

1. **Hiúsági mutatók (vanity metrics).** Olyan mérőszámok, amelyek jól néznek ki, de nem kapcsolódnak a célhoz. Például a „regisztrált felhasználók száma” akkor, ha a valódi cél a rendelés, és a regisztráció nem vezet rendeléshez. A hiúsági mutatók növekedhetnek, miközben az üzleti eredmény romlik.
2. **Túl sok KPI.** Ha minden metrikát KPI-nak nevezünk, a csapat elveszti a fókuszt. A KPI kiválasztása mindig prioritást jelent: néhány kulcsfontosságú mutatóra kell koncentrálni, a többit támogató metrikaként kell kezelni.
3. **Hiányzó mérési terv.** A KPI meg van határozva, de nincs meghatározva, hogyan, milyen gyakran, milyen adatból számolják. A mérési terv nélkül a KPI csak egy szám, amelyet bárki tetszőlegesen értelmezhet.
4. **Nem reprodukálható mérés.** A mérés nem rögzíti a környezetet, a konfigurációt, az időpontot. Két különböző mérés nem összehasonlítható, mert eltérő feltételek mellett készült. A reprodukálhatóság érdekében minden méréshez tartoznia kell kontextusnak, környezetnek, időpontnak és bizonytalanságnak.
5. **Csak lagging indikátorok használata.** A csapat hónapokkal később értesül a problémáról, és már nem tud beavatkozni. A leading indikátorok hiánya megfosztja a csapatot a korai jelzésektől.
6. **A KPI-ok nincsenek összekötve a követelményekkel.** A követelményeket teljesítik, de senki nem tudja, hogy a teljesítés milyen hatást váltott ki. Az ilyen fejlesztés vakon történik: a csapat épít, de nem tudja, miért.
7. **A változásokat nem követik le a KPI-ok.** A követelményváltozás után a régi KPI-ok tovább élnek, így félrevezető képet adnak. A KPI-rendszert ugyanolyan gyakran kell frissíteni, mint a követelményeket.
8. **A mérési ablak nem megfelelő.** Ha túl rövid az ablak, a mérési zaj elnyomja a valós trendet; ha túl hosszú, a beavatkozás késik. A mérési ablakot a rendszer dinamikájához kell igazítani.

Ezek a hibák gyakran együtt jelennek meg, és a mérési rendszer hiteltelenné válásához vezetnek. A következő szakaszban egy gyakorlati példán keresztül mutatjuk be a helyes megoldást.

7.4.6 Gyakorlati példa: e-kereskedelmi rendelési folyamat

Vegyünk egy konkrét, de leegyszerűsített példát. A rendszer egy webes bolt, ahol a felhasználók böngészik a termékeket, kosárba helyezik, majd rendelnek. Az üzleti cél: a rendelési konverzió növelése 10%-kal a következő negyedévben. A jelenlegi konverziós arány 4% (a munkamenetek százalékában). A rendszer jelenlegi pénztárfolyamata lassú, sok lépést tartalmaz, és a felhasználóknak regisztrálniuk kell a rendelés előtt.

A követelmények közül néhány: - **RQ-1:** A felhasználó vendégként is rendelhessen (regisztráció nélkül). - **RQ-2:** A pénztár oldal betöltési ideje p95 alatt 2 másodperc legyen. - **RQ-3:** A rendelési folyamat ne tartalmazzon felesleges lépéseket (maximum 3 képernyő).

A hatástérkép: a cél a konverzió növelése. Az érintett szereplő a regisztráció nélküli látogató, aki jelenleg elhagyja a pénztárat a regisztrációs súrlódás és a lassú betöltés miatt. A viselkedésváltozás:

a látogató nagyobb arányban fejezi be a rendelést. A szoftverkövetelmények: RQ-1, RQ-2, RQ-3.

A KPI-rendszer: - **Lagging KPI:** konverziós arány = rendelések száma / munkamenetek száma, %-ban, havi gördülő átlag. Célérték: 4,4% a negyedév végére (10%-os növekedés a 4%-os bázishoz képest). - **Leading KPI-ok:** - Pénztár befejezési arány: pénztárba lépők közül rendelők aránya, %-ban, heti gördülő átlag. Célérték: legalább 60%. - Pénztár oldal betöltési idő p95: ms-ban, naponta, 30 napos ablak. Célérték: < 2000 ms. - Vendégkijelentkezés használati arány: a vendégként leadott rendelések aránya, %-ban, az első hónapban. Célérték: a rendelések legalább 30%-a vendégként történjen.

A hipotézis: ha a pénztár gyorsabb és egyszerűbb, akkor a befejezési arány nő, ami növeli a konverziót. Ezt a hipotézist a mérési adatokkal igazoljuk: ha a befejezési arány növekszik, de a konverzió nem, akkor a hipotézis hibás, vagy más tényezők is közrejátszanak.

A mérési terv: minden méréshez rögzítjük a környezetet (produkciós webes kliens, Chrome 120, átlagos hálózati feltételek), az adatforrást (alkalmazásnaplók, webanalitika), a számítási módot (p95, gördülő átlag), és a mérési ablakot. A mérést hetente felülvizsgáljuk, és a KPI-ok értékét egy műszerfalon jelenítjük meg.

Most jön egy követelményváltozás: a marketing osztály bevezeti, hogy minden új felhasználónak kötelező regisztrálnia a rendelés előtt. Ez ellentmond RQ-1-nek. A változási kérelmet rögzítjük, és hatáselemzést végzünk. A hatás: RQ-1 törlődik, a vendégkijelentkezés használati aránya értelmetlenné válik. A konverziós arány valószínűleg csökkenni fog a regisztrációs súrlódás miatt. A leading indikátorok közül a vendégkijelentkezés helyébe a regisztrációs űrlap befejezési aránya lép.

A döntést ADR-ben rögzítjük: a kontextus az ügyféladatok gyűjtése a személyre szabott ajánlatokhoz; a döntés a kötelező regisztráció bevezetése; az alternatívák között szerepelt a vendégkijelentkezés megtartása és a regisztráció opcionálissá tétele, de ezeket elvetették, mert az adatgyűjtési cél nem teljesült volna. A következmények: a konverziós célértéket újra kell tárgyalni, a 10%-os növekedés helyett 5% lehet a realisztikus; a KPI-rendszert frissíteni kell a regisztrációs űrlap befejezési arányával.

Ez a példa jól mutatja, hogy a KPI-ok nem statikusak, hanem a követelményekkel együtt változnak. A változáskezelés során a mérési rendszer integritását megőrizzük, és a célokat a realitáshoz igazítjuk.

7.4.7 Labor: KPI- és metrikaterv készítése

A labor célja, hogy a tanuló elkészítsen egy reprodukálható KPI- és metrikatervet egy választott kis rendszerhez. A dokumentumban rögzíteni kell a kontextust, a követelményeket, a hipotéziseket, a KPI-okat, a mérési tervet, a döntési naplót és az ellenőrzési szempontokat. A mért és a várt eredményeket elkülönítve kell szerepeltetni.

7.4.8 A labor lépései

1. **Kontextus rögzítése.** Válassz egy egyszerű rendszert, például egy belső REST API-t, egy webes űrlapot, egy chatbotot vagy egy mikroszolgáltatást. Írd le a célját, a felhasználókat, a rendszerhatárt. Például: „Egy belső tudásbázis kereső API, amelyet a support csapat használ a gyakori kérdések megválaszolására.”

2. **Követelmények listája.** Sorolj fel 3–5 funkcionális követelményt, és mindegyikhez kapcsolódó nem funkcionális követelményt. Például: „A keresés 1 másodperc alatt térjen vissza p95 esetén.” Ügyelj arra, hogy a követelmények ellenőrizhetők legyenek.
3. **Hipotézisek megfogalmazása.** Minden követelményhez írd le, milyen hatást várs. Például: „Ha a keresés gyorsabb, akkor a support csapat kevesebb időt tölt várakozással, így több jegyet tud megoldani.” A hipotézis legyen konkrét és mérhető.
4. **KPI-ok kiválasztása.** Minden hipotézishez rendelj egy vagy több KPI-t. Tüntesd fel a leading/lagging jelleget, a mértékegységet, a mérési ablakot, a célértéket. Például: „Keresési válaszidő p95 (ms), naponta, < 1000 ms; leading indikátor a megoldott jegyek számához.”
5. **Mérési terv.** Írd le, hogyan számolod a KPI-t, milyen adatforrásból, milyen gyakran, milyen környezetben. Például: „Az API-válaszidőt az alkalmazás APM-eszköze gyűjti, p95-öt napi bontásban, a produkciós környezetben, a terheléses csúcsidőszakokban.”
6. **Döntési napló.** Rögzítsd a KPI-választással kapcsolatos döntéseket ADR-formátumban: kontextus, döntés, alternatívák, következmények, állapot. Például: „A keresési válaszidő p95-öt választottuk leading indikátornak, mert közvetlenül méri a felhasználói élményt. Az alternatíva az átlagos válaszidő lett volna, de az átlag elrejti a kiugró értékeket, ezért elvetettük.”
7. **Ellenőrzési szempontok.** Készíts listát, amely alapján ellenőrizhető, hogy a terv teljes és reprodukálható. Például: „Minden KPI-hoz tartozik célérték, mértékegység, mérési ablak és adatforrás? A mérési módszer reprodukálható? A leading és lagging indikátorok egyensúlya megvan?”

7.4.9 Példa a dokumentumra

Az alábbi táblázat egy lehetséges KPI- és metrikatervet mutat be a fenti e-kereskedelmi példa alapján. A táblázatban a „várt” célérték a tervezett eredmény, a „mért” érték pedig a kiindulási bázismérés (ha van ilyen; ha nincs, azt is rögzíteni kell, és a mérési tervben ütemezni).

Követelmény	Hipotézis	KPI	Leading/Lagging	Mértékegység	Mérési ablak	Várt célérték	Mért bázisérték	Adatforrás
RQ-1: Vendégkijelentkezés	Ha elérhető, akkor nő a konverzió	Konverziós arány	Lagging	%	havi	4,4%	4,0%	Webanalitika
RQ-2: Pénztár betöltés < 2s	Ha gyorsabb, akkor kevesebb a kilépés	Pénztár befejezési arány	Leading	%	heti	60%	55%	Alkalmazásnapló

Követelmény	Hipotézis	KPI	Leading/Lagging	Mértékegység	Mérési ablak	Várt célérték	Mért bázisérték	Adatforrás
RQ-2: Pénztár betöltés < 2s	Ha gyorsabb, akkor jobb a felhasználói élmény	Pénztár oldal betöltési idő p95	Leading	ms	napi	< 2000 ms	3500 ms	APM
RQ-1: Vendégkijelentkezés	Ha elérhető, akkor nő a rendelések aránya vendégként	Vendégkijelentkezés használati arány	Leading	%	első hónap	30%	0% (nem volt)	Webanalitika

A dokumentumnak tartalmaznia kell egy rövid ADR-t is, amelyben a KPI-választások indoklása szerepel. A labor során a tanulónak a saját példájára kell elkészítenie ezt a dokumentumot, és a mért, illetve a várt eredményeket egyértelműen el kell különítenie.

7.4.10 A labor ellenőrzése

A labor akkor tekinthető sikeresnek, ha: - a kontextus egyértelműen rögzített, - minden kulcskövetelményhez tartozik hipotézis, - a KPI-ok rendelkeznek célértékkel, mértékegységgel, mérési ablakkal, adatforrással, - a leading és lagging indikátorok egyensúlya megvan, - a mérési terv reprodukálható, - a döntések ADR-ben rögzítettek, - a mért és várt eredmények elkülönítettek, - a dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

7.4.11 Ellenőrzőlista a fejezethez

- ☐ Minden kulcskövetelményhez tartozik legalább egy KPI?
- ☐ A KPI-oknak van célértéke, mértékegysége, mérési ablaka és adatforrása?
- ☐ Vannak leading és lagging indikátorok is?
- ☐ A KPI-ok kapcsolódnak az üzleti célhoz, nem hiúsági mutatók?
- ☐ A mérési terv reprodukálható (környezet, konfiguráció, időpont)?
- ☐ A követelményváltozásokhoz készült hatáselemzés és KPI-frissítés?
- ☐ A döntések ADR-ben rögzítettek?
- ☐ A mért és várt eredmények elkülönítettek?
- ☐ A mérési ablak megfelel a rendszer dinamikájának?
- ☐ A KPI-rendszer nem tartalmaz túl sok mutatót?

7.4.12 Önellenőrző kérdés

Egy csapat egy új mobilalkalmazást fejleszt. A követelmény: „A felhasználó push-értesítést kap, ha a rendelése állapota megváltozik.” A termékmenedzser szerint a siker mércéje a „letöltések

száma”. Hogyan értékelnéd ezt a KPI-t? Milyen más mérőszámokat javasolnál (leading és lagging), és miért?

Várható válasz vázlat: A letöltések száma lagging indikátor, de nem feltétlenül kapcsolódik az adott követelményhez, mivel a push-értesítés célja az elköteleződés és a visszatérő vásárlás, nem pedig a letöltés. Ha a cél a felhasználói elégedettség vagy a visszatérő vásárlás, akkor a letöltések száma hiúsági mutató lehet. Javasolt leading indikátorok: az értesítésre való feliratkozási arány (%), az értesítésre való kattintási arány (%). Javasolt lagging indikátorok: a visszatérő vásárlók aránya (%), a havi aktív felhasználók száma (db/hó). Így a mérőszámok közvetlenül az értesítés hatását mérik, és lehetővé teszik a korai beavatkozást.

7.4.13 Összegzés

Ebben a fejezetben megtanultuk, hogyan kapcsoljuk össze a követelményeket az üzleti mérőszámokkal. A KPI-ok nem utólagos riportok, hanem a követelmények validálásának eszközei. A leading indikátorok a korai beavatkozást, a lagging indikátorok a végső eredményt szolgálják. A követelményváltozásokat hatáselemzéssel és ADR-ekkel kell kezelni, és a KPI-okat frissíteni kell, hogy a mérési rendszer hiteles maradjon.

A fejezetben bemutatott eszközök – impact mapping, KPI-fa, leading/lagging elemzés, változáskezelés – a következő fejezetekben is visszaköszönnek. A Teljesítmény és skálázhatóság egységtől kezdve a nem funkcionális követelmények (teljesítmény, rendelkezésre állás, skálázhatóság) részletes tárgyalása során ezeket a mérési alapokat használjuk majd. A KPI-ok és a mérőszámok nélkül a teljesítménycélok, az SLO-k és a minőségi attribútumok is csak üres deklarációk maradnak.

8 Nem funkcionális követelmények és minőségi attribútumok

8.1 Teljesítmény és skálázhatóság

A teljesítmény és a skálázhatóság a szoftverrendszerek két leggyakrabban hivatkozott, ugyanakkor leggyakrabban félreértett minőségi attribútuma. A gyakorlatban gyakran hallunk olyan kijelentéseket, hogy „a rendszer gyors” vagy „skálázható”, anélkül hogy ezek mérhető, adott körülmények között ellenőrizhető állítások lennének. Ez az alfejezet a mérhető értelmezésre helyezi a hangsúlyt: bevezeti a latencia, az átbocsátóképesség és az erőforrás-kihasználtság fogalmát, bemutatja a horizontális és vertikális skálázás alapjait, majd egy labor segítségével a terhelési modell és a kapacitáscél elkészítését gyakoroltatja. A cél nem az, hogy minden részletet megismerjünk, hanem hogy a döntéshozatalhoz szükséges mérési és modellezési szemléletet kialakítsuk.

8.1.1 A mérhető értelmezés alapjai

A teljesítmény és a skálázhatóság nem abszolút tulajdonságok, hanem egy adott terhelés, környezet és időablak függvényében értelmezhető mérőszámok. Egy rendszer akkor tekinthető jól teljesítőnek, ha a vele szemben támasztott igényeket – például a válaszidőre, az átbocsátóképességre vagy az erőforrás-felhasználásra vonatkozó célokat – teljesíti, és ezt reprodukálhatóan, méréssel alá tudjuk támasztani. A skálázhatóság ezzel szemben azt írja le, hogyan változik a rendszer teljesítménye, amikor a terhelést vagy a rendelkezésre álló erőforrásokat növeljük. Másképpen fogalmazva: a teljesítmény egy pillanatfelvétel, a skálázhatóság pedig egy görbe.

A mérhető értelmezéshez három összetevőt kell rögzíteni:

1. **Terhelés**: hány kérés, tranzakció vagy felhasználó éri a rendszert, milyen arányban és milyen időbeli eloszlásban?
2. **Környezet**: milyen erőforrások állnak rendelkezésre (CPU, memória, hálózat, lemez), milyen konfigurációval és milyen függőségekkel?
3. **Mérési ablak**: milyen időtartamra vonatkozik a mérőszám, és az adott ablakban milyen terhelés volt érvényben?

E három elem nélkül a „teljesítmény” és „skálázhatóság” szavak üresek. A minőségi követelményeket ezért célszerű quality attribute scenario formában megfogalmazni, ahogyan azt a későbbiekben részletesen bemutatjuk.

A teljesítmény mérésének leggyakoribb mutatói a latencia, az átbocsátóképesség és az erőforrás-kihasználtság. A skálázhatóság méréséhez ezeket a mutatókat különböző terhelési szinteken és erőforrás-konfigurációkban kell felvenni, majd az eredményeket görbeként vagy táblázatként ábrázolni.

8.1.2 Latencia: az átlag csapdája és a percentilisek

A **latencia** azt az időt méri, amely egyetlen kérés feldolgozásához szükséges a kliens szempontjából. Fontos megkülönböztetni a feldolgozási időtől, amely csak a szerveroldali végrehajtást foglalja magában. A latencia magában foglalja a hálózati késleltetést, a sorban állást, a szerializációt és a feldolgozást is. Mértékegysége általában ezredmásodperc (ms) vagy másodperc (s).

A latencia leggyakrabban használt aggregált mutatója az **átlag** (mean), amely azonban rendkívül félrevezető lehet eloszlással rendelkező válaszidők esetén. Ha például 1000 kérésből 990 kérés 50 ms alatt válaszol, 10 kérés viszont 2000 ms-ot vesz igénybe, az átlag körülbelül 69,5 ms lesz. Ez a szám azt a hamis képet mutatja, hogy a rendszer gyors, miközben a felhasználók egy része rendkívül lassú választ kap. A felhasználói élményt és a rendszer stabilitását gyakran a **farok-latencia** határozza meg.

A farok-latencia jellemzésére a **percentilisek** szolgálnak. A p50 a medián, vagyis az a válaszidő, amely alá a kérések fele esik. A p95 azt jelenti, hogy a kérések 95 százaléka gyorsabb ennél az értéknél, 5 százalék pedig lassabb. A p99, p999 hasonlóan értelmezhető. Például egy webshop esetén a termékoldal betöltésére előírhatjuk, hogy a p95 válaszidő legfeljebb 300 ms lehet 100 RPS terhelés mellett, 5 perces mérési ablakban. Az ilyen megfogalmazás már mérhető és ellenőrizhető.

A percentilisek mellett érdemes hisztogramot is készíteni a válaszidők eloszlásáról, mert így láthatóvá válik a kiugró értékek csoportja és a hosszú farok. A gyakorlatban a p95 és p99 a leggyakoribb célérték, de a nagy rendszereknél gyakran a p999 is külön figyelmet érdemel, mert ezek a kérések okozhatnak lassú tranzakciókat fontos ügyfeleknél.

A latencia mérésénél mindig rögzíteni kell a mérési ablakot és a terhelést. A terhelés nélküli „üres” rendszeren mért 20 ms-os válaszidő nem jelenti azt, hogy a rendszer 1000 kérés/s alatt is 20 ms alatt válaszol. A mérési ablak hossza azért fontos, mert a rendszer viselkedése időben változhat: a melegedési időszak, a gyorsítótár feltöltése, a háttérfolyamatok vagy a szemétgyűjtés mind torzíthatják a rövid mérési ablakok eredményét.

8.1.3 Átbocsátóképesség és a Little-törvény

Az **átbocsátóképesség** (throughput) azt méri, hogy a rendszer hány kérést, tranzakciót vagy eseményt képes feldolgozni időegység alatt. Mértékegysége jellemzően kérés/másodperc (RPS), tranzakció/másodperc (TPS) vagy esemény/perc. Az átbocsátóképesség a latenciával szoros kapcsolatban áll, de nem egymás inverzei. Egy rendszer lehet alacsony latenciájú alacsony terhelésnél, és közben alacsony maximális átbocsátóképességű, ha például egyetlen szálon dolgoz fel minden kérést.

A kapcsolatot a Little-törvény írja le elegánsan:

$$L = \lambda W$$

ahol: - L a rendszerben tartózkodó kérések átlagos száma (azaz a folyamatban lévő, várakozó és feldolgozás alatt álló kérések együttesen), - λ az érkezési ráta (például kérés/másodperc), - W az átlagos tartózkodási idő (a kérés belépésétől a válasz elküldéséig eltelt idő, másodpercben).

A Little-törvény rendkívül hasznos kapacitástervezéshez. Tegyük fel, hogy egy szolgáltatás átlagos feldolgozási ideje $W = 0{,}2$ s, és a cél az, hogy a rendszerben átlagosan legfeljebb $L = 10$ kérés legyen egyszerre (például a rendelkezésre álló adatbázis-kapcsolatok száma miatt). Ekkor a maximális fenntartható érkezési ráta:

$$\lambda = \frac{L}{W} = \frac{10}{0{,}2} = 50 \text{ kérés/s}$$

Ha a terhelés ennél nagyobb, a rendszerben lévő kérések száma növekedni kezd, a sor hossza nő, és a válaszidő is romlik, ami öngerjesztő folyamathoz vezethet.

A Little-törvény arra is felhívja a figyelmet, hogy adott átbocsátóképességhez és latenciához mekkora párhuzamosságra van szükség. Ha a cél 1000 kérés/s feldolgozása úgy, hogy az átlagos tartózkodási idő 0,5 s, akkor a rendszerben átlagosan $L = 1000 \cdot 0{,}5 = 500$ kérésnek kell egyszerre helyet adni. Ez azt jelenti, hogy a szolgáltatásnak legalább ekkora kapacitású várakozási sorral és feldolgozó kapacitással kell rendelkeznie.

Fontos, hogy az átbocsátóképesség nem egyetlen szám, hanem a terhelés függvénye. A rendszer átbocsátóképessége a telítettségi pontig általában növekszik a terheléssel, majd elér egy platót, ahonnan a latencia meredeken emelkedni kezd, az átbocsátóképesség pedig akár csökkenhet is. Ezt a viselkedést gyakran nevezik **telítettségi görbének**. A kapacitástervezés során tehát nem elég azt tudni, hogy a rendszer „bírja” a csúcsforgalmat, hanem azt is, hogy hol van a telítettségi pont, és milyen tartalék áll rendelkezésre.

8.1.4 Erőforrás-kihasználtság és telítettség

A harmadik kulcsfontosságú mutató az **erőforrás-kihasználtság**. A rendszer teljesítményét végső soron a rendelkezésre álló erőforrások korlátozzák: CPU, memória, hálózati sávszélesség, lemez I/O, adatbázis-kapcsolatok, szálak és hasonlók. Az erőforrás-kihasználtság mérése megmutatja, melyik erőforrás a szűk keresztmetszet, és mennyi tartalék maradt a csúcsterhelésre.

A **telítettség** (saturation) azt jelzi, hogy egy erőforrás milyen mértékben van kihasználva, és mennyi munka vár rá. A CPU esetében a kihasználtság százalékos értéke mellett fontos a futási sor hossza is: ha a CPU 80%-on fut, de a futási sor átlagosan 2-nél nagyobb, akkor már telített, és a válaszidő növekedésére lehet számítani. A memória esetében a telítettség jele lehet a swap-használat vagy a folyamatos szemétgyűjtés. A hálózatnál a csomagvesztés és az újraküldések aránya utalhat telítettségre.

A teljesítményelemzés során gyakran a **bottleneck** (szűk keresztmetszet) azonosítása a legfontosabb lépés. Ha egy rendszer lassú, akkor a hibát legtöbbször nem a leglassabb komponensben, hanem abban a komponensben kell keresni, amelyik a legnagyobb várakozási időt okozza a láncban. Az erőforrás-kihasználtság mérésével és a latencia eloszlásának vizsgálatával a szűk keresztmetszet általában beazonosítható.

8.1.5 Vertikális skálázás

A **vertikális skálázás** (scale up) azt jelenti, hogy a meglévő példány erőforrásait növeljük: több CPU-t, memóriát, gyorsabb lemezt vagy nagyobb hálózati sávszélességet adunk a géphez. Ennek előnye, hogy nem igényel alkalmazás-szintű változtatást, hiszen a rendszer továbbra is egyetlen csomóponton fut. A vertikális skálázás azonban korlátos: a hardver fizikai felső határa miatt nem lehet a végtelenségig növelni, és a nagyobb gépek költsége általában nem lineárisan nő. Ráadásul egyetlen példány esetén a rendszer továbbra is **egypontos meghibásodási pont** marad: ha a gép leáll, a szolgáltatás is leáll.

A vertikális skálázás jellemzően akkor jó választás, ha az alkalmazás nem igazán párhuzamosítható, vagy ha a tranzakciós adatbázis erőforrásigénye nő, és a horizontális skálázás bevezetése túl nagy komplexitást jelentene. Például egy relációs adatbázis-kiszolgáló esetében a vertikális skálázás

gyakran egyszerűbb, mint a sharding bevezetése. Ugyanakkor a vertikális skálázás nem javítja a rendelkezésre állást, és a nagyobb szerverek beszerzése, üzemeltetése drágább lehet.

8.1.6 Horizontális skálázás

A **horizontális skálázás** (scale out) azt jelenti, hogy a rendszer több példányát futtatjuk párhuzamosan, és a terhelést elosztjuk közöttük. Ennek előnye, hogy elvileg korlátlanul bővíthető, rugalmasan követheti a terhelés ingadozását, és növeli a rendelkezésre állást, mert egy példány kiesése nem feltétlenül okoz teljes leállást. A horizontális skálázás azonban jelentős architekturális következményekkel jár.

Először is, a horizontálisan skálázott példányoknak **állapotmentesnek** kell lenniük, vagy legalábbis az állapotot külső, közös tárolóba kell helyezni. Ha a kérések egy része a szerver memóriájában tárolt munkamenethez kötődik, akkor a terheléselosztó nem tudja szabadon irányítani a forgalmat. A munkamenet-állapotot célszerű elosztott gyorsítótárba (például Redis) vagy adatbázisba helyezni, vagy teljesen elkerülni a szerveroldali állapottartást.

Másodszor, a horizontális skálázás bevezeti az **elosztott rendszerek** problémáit: hálózati késleltetést, részleges hibákat, konzisztenciaproblémákat és a koordináció szükségességét. A terheléselosztó mögötti példányok közötti állapotmegosztás, a gyorsítótár-koherencia és az idempotencia mind olyan kérdés, amelyet a skálázás előtt meg kell oldani.

Harmadszor, a horizontális skálázás nem minden réteget érint egyformán. A webes frontend és a stateless szolgáltatások általában könnyen skálázhatók, de a tranzakciós adatbázis sokkal nehezebben. Az adatréteg gyakran a skálázhatóság valódi korlátja. Ezért a horizontális skálázás tervezésekor az adatréteget is fel kell készíteni a terhelésre: olvasási replikák, sharding, partícionálás vagy NoSQL adatbázisok bevezetése révén.

A gyakorlatban a vertikális és horizontális skálázás nem egymást kizáró lehetőségek. Gyakori stratégia, hogy először vertikálisan skálázunk egy példányt, amíg a költség-haszon arány kedvező, majd amikor elérjük a hardveres vagy pénzügyi korlátot, áttérünk a horizontális skálázásra. A **elaszticitás** (rugalmas automatikus skálázás) a horizontális megközelítés egyik fő előnye a felhőkörnyezetben: a rendszer a terhelés függvényében automatikusan indít és állít le példányokat.

8.1.7 Quality attribute scenario a teljesítményre

A teljesítményre és skálázhatóságra vonatkozó követelményeket a könyv konvencióinak megfelelően quality attribute scenario formában kell megfogalmazni. Egy ilyen forgatókönyv tartalmazza az ingert, a környezetet, a választ és a mérőszámot. Példa:

> **Inger**: 500 egyidejű felhasználó böngészik a webáruházban, és átlagosan 20 RPS-t generál a termékkereső szolgáltatás felé, a kérések 20%-a szűrőkkel ellátott keresés.
>
> **Környezet**: a termékkereső szolgáltatás 3 példánya fut Kubernetes klaszteren, mindegyik 2 vCPU-val és 4 GiB memóriával, mögöttük egy 3 csomópontos Elasticsearch klaszter.
>
> **Válasz**: a rendszer minden kérést sikeresen kiszolgál, a p95 válaszidő nem haladja meg a 300 ms-ot, a p99 pedig az 500 ms-ot.
>
> **Mérőszám**: p95 és p99 válaszidő, hibaarány, mind 5 perces mérési ablakban, a terhelés 30 perces stabil futtatása során.

Ez a megfogalmazás lehetővé teszi, hogy a követelményt teszteljük, és a mérési eredményt a céllal összehasonlítsuk. A skálázhatóságra vonatkozóan a forgatókönyv kiegészíthető azzal, hogy ugyanezt a terhelést 1, 3 és 5 példánnyal is futtatjuk, és rögzítjük, hogyan változik a p95 válaszidő és az átbocsátóképesség. A görbe alakja mutatja meg, hogy a rendszer lineárisan skálázódik-e, vagy belép egy telítettségi tartományba.

8.1.8 Tipikus hibák

A teljesítmény- és skálázhatósági vizsgálatok során számos tipikus hiba fordul elő. Az alábbiakban a leggyakoribbakat gyűjtöttük össze:

1. **Az átlag használata farok-latencia helyett**: ahogy korábban láttuk, az átlag elrejti a kiugró értékeket. A felhasználói élményt a percentilisek jobban leírják. A döntési kritériumokban mindig adjunk meg percentilis célértékeket.
2. **A terhelés nélküli mérés**: egy üres rendszeren mért válaszidő nem reprezentálja a valós működést. A mérést mindig a célzott terhelési profil mellett kell elvégezni.
3. **A skálázhatóság összekeverése a teljesítménnyel**: egy rendszer lehet gyors alacsony terhelésnél, de ha a terhelés növelésével a válaszidő meredeken romlik, akkor nem skálázható. A kettőt külön kell mérni és értékelni.
4. **A szűk keresztmetszet figyelmen kívül hagyása**: ha nem mérjük az erőforrás-kihasználtságot, akkor a hibát rossz helyen keressük. Például a lassú válaszidőt a webes rétegnek tulajdonítjuk, miközben az adatbázis a telített.
5. **A horizontális skálázásnál az adatréteg elhanyagolása**: a stateless szolgáltatások könnyen skálázódnak, de a tranzakciós adatbázis gyakran a korlát. A skálázási tervnek ki kell terjednie az adatréteg stratégiájára is.
6. **A koordinációs költségek alábecsülése**: az elosztott rendszerekben a zárolás, a szavazás, a munkamenet-állapot replikálása és a konzisztencia fenntartása többlet erőforrást igényel. A horizontális skálázás nem lineárisan növeli az átbocsátóképességet, ha a koordináció dominánssá válik.
7. **A mérési ablak túl rövid**: a rövid, néhány másodperces mérés nem veszi figyelembe a bemelegedést, a gyorsítótár feltöltését vagy a periodikus háttérfolyamatokat. A mérési ablakot a rendszer jellegéhez kell igazítani.

8.1.9 Labor: terhelési modell és kapacitáscél készítése

A következő labor célja, hogy a tanult fogalmakat egy konkrét, reprodukálható példán alkalmazzuk. A feladat egy **terhelési modell** és egy **kapacitáscél** elkészítése egy fiktív, de realisztikus webshop-rendszer számára. A labor során dokumentációt készítünk, döntési naplót vezetünk, és a mért eredményeket elkülönítjük a várt értékektől. A labor izolált környezetben végezhető, például helyi konténeres környezetben vagy egy felhőalapú sandboxban.

8.1.10 Kiinduló állapot

Adott egy **webshop-api** nevű mikroszolgáltatás, amely a termékek listázását, keresését és a kosár műveleteit biztosítja. A szolgáltatás egy konténerben fut, alapértelmezés szerint 1 példányban, és egy PostgreSQL adatbázist használ. A csapat azt szeretné megtudni, hogy milyen terhelést bír el a jelenlegi konfiguráció, és hány példányra lesz szükség a várható csúcsforgalom kiszolgálásához.

- **Kiinduló konfiguráció**: 1 db webshop-api konténer, 2 vCPU, 4 GiB RAM; 1 db PostgreSQL konténer, 4 vCPU, 16 GiB RAM.
- **Üzleti háttér**: a webshop átlagos napi forgalma 10 RPS, de akciók idején a forgalom 100 RPS-re nő. A csúcs időtartama jellemzően 2 óra.
- **Elvárt felhasználói élmény**: a terméklista és kereső API-válaszok p95 válaszideje legfeljebb 300 ms, a kosárműveletek p95 válaszideje legfeljebb 500 ms a csúcsterhelés alatt.

8.1.11 Elvárt eredmény

A labor végén a hallgató elkészíti a következőket:

1. **Terhelési modell**: a várható forgalom leírása, beleértve a normál és csúcsidőszakot, a kérések arányát művelettípusonként, az átlagos és csúcsterhelést RPS-ben, valamint a terhelés időbeli lefutását.
2. **Kapacitáscél**: a rendszerrel szemben támasztott teljesítménycélok quality attribute scenario formában, beleértve a latencia, átbocsátóképesség és erőforrás-kihasználtság célértékeit.
3. **Mérési terv**: hogyan fogjuk megmérni a rendszer viselkedését, milyen eszközökkel, milyen terhelési profilokkal, és milyen mérési ablakokat használunk.
4. **Döntési napló**: a kapacitástervezés során hozott döntések rögzítése, beleértve a feltételezéseket, a választott mérőszámokat és a mért eredmények alapján tett következtetéseket.
5. **Összefoglaló táblázat**: a várt és mért eredmények elkülönítése.

8.1.12 Terhelési modell

A terhelési modell elkészítéséhez a következő lépéseket javasoljuk:

1. **Forgalmi profil meghatározása**: becsüljük meg, milyen műveleteket milyen arányban hajtanak végre a felhasználók. Például:

   - Terméklista: 40%
   - Termékkereső: 30%
   - Termékrészletek: 20%
   - Kosárműveletek: 10%

2. **Átlagos és csúcsterhelés**: az átlagos terhelés 10 RPS, a csúcsterhelés 100 RPS. A csúcsidőszak 2 óra, ez alatt a terhelés fokozatosan emelkedik, majd csökken. A labor egyszerűsítése érdekében feltételezhetjük, hogy a csúcsterhelés állandó 100 RPS a teszt alatt.

3. **Környezet rögzítése**: a méréshez használt környezet pontos konfigurációját fel kell jegyezni. Ha helyi konténeres környezetet használunk, rögzítsük a Docker verziót, az operációs rendszert, a hardver specifikációt és a hálózati beállításokat.

4. **Időbeli lefutás**: a terhelési teszt során a terhelést fokozatosan növeljük, például 1 RPS-től indulva percenként 10 RPS-t emelve, amíg el nem érjük a 100 RPS-t, majd ezt 30 percig fenntartjuk. Ezután a terhelést csökkentjük, és megfigyeljük a rendszer visszatérését a normál állapotba.

8.1.13 Kapacitáscél

A kapacitáscél meghatározásához a következő quality attribute scenario-t fogalmazzuk meg:

> **Inger**: 100 RPS terhelés éri a webshop-api szolgáltatást, a fenti műveleti arányokkal.
>
> **Környezet**: a webshop-api 3 példánya fut, mindegyik 2 vCPU-val és 4 GiB memóriával; a PostgreSQL adatbázis 1 példánya 4 vCPU-val és 16 GiB memóriával; a terheléselosztó round-robin algoritmust használ.
>
> **Válasz**: a rendszer minden kérést sikeresen kiszolgál, a hibaarány kisebb, mint 0,5%, a terméklista és kereső p95 válaszideje legfeljebb 300 ms, a kosárműveletek p95 válaszideje legfeljebb 500 ms.
>
> **Mérőszám**: p95 és p99 válaszidő, hibaarány, átbocsátóképesség (RPS), CPU- és memóriakihasználtság, mind 5 perces mérési ablakban, a 30 perces csúcsterhelés alatt.

8.1.14 Mérési terv

A méréshez terhelésgenerátort használunk, például k6, JMeter vagy vegeta eszközt. A választott eszközt és annak verzióját dokumentáljuk. A terhelési profil beállítása:

- **Melegedési szakasz**: 5 perc alatt fokozatosan emeljük a terhelést 1 RPS-ről 100 RPS-re.
- **Stabil szakasz**: 30 percig fenntartjuk a 100 RPS terhelést.
- **Leállási szakasz**: 5 perc alatt csökkentjük a terhelést 0 RPS-re.

A mérés során a következő adatokat gyűjtjük:

- Válaszidő hisztogramok és percentilisek (p50, p95, p99) művelettípusonként.
- Hibaarány (HTTP 5xx, timeout).
- Átbocsátóképesség (RPS) a terhelésgenerátor oldaláról.
- Erőforrás-kihasználtság a konténerekben: CPU, memória, hálózati I/O.
- Adatbázis-kihasználtság: aktív kapcsolatok, tranzakciók száma, CPU.

A mérési ablakokat a rendszer jellegéhez igazítjuk. A stabil szakasz utolsó 20 percét tekintjük a mérési ablaknak, hogy elkerüljük a bemelegedési hatásokat.

8.1.15 Döntési napló

A labor során döntési naplót vezetünk. Például:

- **Döntés**: a kapacitáscélt 100 RPS-re határozzuk meg, mert ez a várható csúcsterhelés az akciós időszakban, és a korábbi forgalmi adatok alapján a növekedés üteme évi 20%.
- **Döntés**: a p95 válaszidőt választjuk elsődleges mérőszámnak 300 ms céllal, mert a felhasználói elégedettségi mérések szerint 300 ms felett a konverzió jelentősen csökken.
- **Döntés**: a méréshez 3 példányt feltételezünk, mert a csapat becslése szerint ezzel érhető el a kívánt átbocsátóképesség, és ezt a méréssel ellenőrizzük.

8.1.16 A várt és mért eredmények elkülönítése

A labor fontos része, hogy a **várt** (becsléssel vagy korábbi tapasztalattal alátámasztott) és a **mért** eredményeket külön táblázatban rögzítsük. Például:

Mérőszám	Várt érték	Mért érték	Eltérés
Átbocsátóképesség 1 példánnyal	80 RPS	72 RPS	-10%
Átbocsátóképesség 3 példánnyal	240 RPS	210 RPS	-12,5%
p95 válaszidő 3 példánnyal, 100 RPS	< 300 ms	285 ms	teljesítve
Hibaarány 3 példánnyal, 100 RPS	$< 0{,}5\%$	0,3%	teljesítve

A táblázat egyértelműen mutatja, hogy a horizontális skálázás nem lineárisan növeli az átbocsátóképességet, mert a koordinációs és adatbázis-korlátok miatt veszteség lép fel. Ezt a mérési eredményt dokumentálni kell, és be kell építeni a kapacitástervezésbe.

8.1.17 Ellenőrzési szempontok

A labor értékelésénél az alábbi szempontokat vizsgáljuk:

- A terhelési modell egyértelműen leírja a forgalmi arányokat, a csúcsterhelést és az időbeli lefutást.
- A kapacitáscél quality attribute scenario formában van megfogalmazva, és tartalmazza az ingert, környezetet, választ és mérőszámot.
- A mérési terv reprodukálható: a környezet, az eszközök, a konfiguráció és a mérési ablak rögzített.
- A döntési naplóban a döntések indoklása visszakereshető, és az alternatívák is szerepelnek.
- A várt és mért eredmények külön szerepelnek, és az eltéréseket magyarázat kíséri.
- A mérési eredmények mértékegységgel és mérési ablakkal vannak ellátva.
- A dokumentum nem tartalmaz valós hitelesítő adatot vagy személyes adatot.

8.1.18 Visszaállítási út

A labor izolált környezetben fut, ezért a rendszerek visszaállításáról nem kell gondoskodni. Ha a mérés során konfigurációt módosítunk, a kiinduló állapotot a labor végén helyreállítjuk, például a példányszámot visszaállítjuk 1-re, és a tesztadatokat töröljük.

8.1.19 Ellenőrzőlista

A teljesítmény- és skálázhatósági mérési feladatok során az alábbi ellenőrzőlista segít a fontos szempontok áttekintésében:

- ☐ A teljesítménycélok számszerűek, mértékegységgel és mérési ablakkal ellátottak.
- ☐ A latencia cél percentilisekben van megfogalmazva, nem csak átlagban.
- ☐ A terhelési modell leírja a forgalmi arányokat, a csúcsterhelést és az időbeli lefutást.
- ☐ A környezet pontos konfigurációja rögzített, beleértve a hardvert, a szoftvert és a függőségeket.
- ☐ A mérési terv reprodukálható, és a mérési ablak elegendően hosszú a stabil viselkedés megfigyeléséhez.

- ☐ A skálázhatóságot több erőforrás-konfigurációban is mértük, és az eredményeket görbeként ábrázoltuk.
- ☐ A szűk keresztmetszetet az erőforrás-kihasználtság mérésével azonosítottuk.
- ☐ A döntéseket ADR formában rögzítettük, az alternatívákkal és következményekkel együtt.
- ☐ A várt és mért eredmények elkülönülnek, és az eltéréseket megindokoltuk.
- ☐ A dokumentáció nem tartalmaz éles rendszerre vonatkozó érzékeny adatot vagy hitelesítő adatot.

8.1.20 Önellenőrző kérdés

Egy csapat azt állítja, hogy a szolgáltatásuk „gyors és skálázható”, mert a helyi fejlesztői gépen 20 ms alatt válaszol, és ha több példányt indítanak, akkor a rendszer „többet bír”. Milyen méréseket és milyen formában kellene kérnie, hogy ezt az állítást ellenőrizni tudja? Válaszában térjen ki a latencia, átbocsátóképesség, erőforrás-kihasználtság és a terhelési modell szerepére, valamint arra, hogy miért nem elegendő az átlagos válaszidő.

8.2 Rendelkezésre állás, megbízhatóság és hibatűrés

A rendelkezésre állás, a megbízhatóság és a hibatűrés a legtöbb éles rendszer esetében szorosan összefügg, ám korántsem ugyanazt jelentik. A gyakorlatban az architektúrának nem az a célja, hogy a hibákat teljesen kiküszöbölje – ez fizikailag és gazdaságilag is lehetetlen –, hanem az, hogy a rendszer a hibák ellenére is képes legyen az elvárt szolgáltatást nyújtani. Ebben a fejezetben először megkülönböztetjük a hiba, a hibajelenség és a meghibásodás fogalmát, majd bevezetjük az üzemidőt, a hibaarányt és a helyreállítást leíró mérőszámokat. Ezt követően a redundancia, a failover és a fokozatos degradáció tervezési mintáit tárgyaljuk, végül egy labor keretében elkészítünk egy ellenőrizhető rendelkezésre állási célt és hibatűrési tervet.

8.2.1 1. Alapfogalmak: hiba, hibajelenség, meghibásodás

A megbízhatósági elemzésben három fogalmat érdemes pontosan elkülöníteni, mert az angol „failure” és „fault” keverése gyakran vezet rossz döntésekhez.

- **Hiba (fault):** a rendszer valamely alkotóelemében rejlő, látens rendellenesség. Ilyen például egy hibás kódsor, egy elhasználódott lemez, egy rosszul konfigurált hálózati eszköz, vagy egy lemerült elem. A hiba még nem feltétlenül látható a felhasználó számára.
- **Hibajelenség (error):** a hiba által előidézett, a rendszer belső állapotában megjelenő eltérés a helyes állapottól. Ha a hibás kódsor lefut, rossz értéket ír egy változóba, az a hibajelenség.
- **Meghibásodás (failure):** a rendszer által nyújtott szolgáltatás eltér a specifikált vagy elvárt viselkedéstől. A meghibásodás a felhasználó vagy a hívó fél számára is érzékelhető.

Az architektúra feladata, hogy a hibákból lehetőleg ne legyen meghibásodás, illetve ha mégis, akkor a rendszer minél gyorsabban és minél kisebb hatással álljon helyre. Ezt a képességet nevezzük hibatűrésnek (fault tolerance). A hibatűrő rendszer nem hibamentes, hanem úgy van megtervezve, hogy a hibák bekövetkezésekor is fenntartsa a kritikus funkcióit.

Fontos hangsúlyozni, hogy a hibatűrés nem csak infrastrukturális kérdés. Egy elosztott rendszerben a hibák forrása lehet szerverleállás, hálózati szakadás, telítődés, helytelen sorosítás, félkész migráció, vagy éppen emberi tévedés. A jól megtervezett architektúra mindegyik hibaosztályra ad valamilyen választ.

8.2.2 2. Az üzemidő, a hibaarány és a helyreállítás összefüggése

A rendelkezésre állás (availability, A) azt fejezi ki, hogy a rendszer egy adott időablakban milyen arányban képes a kért szolgáltatást nyújtani. Az üzemidő és az állásidő fogalmaival a rendelkezésre állás a következőképpen írható le:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Az üzemidőt és az állásidőt ugyanabban a mértékegységben, jellemzően percben vagy órában adjuk meg. A mérési ablakot mindig rögzíteni kell, mert egy óra alatt mért 99,9%-os rendelkezésre állás egészen mást jelent, mint egy negyedév alatt mért 99,9%.

A gyakorlatban azonban a rendelkezésre állást nem közvetlenül az üzemidő és az állásidő összegzésével, hanem a hibaarányból és a helyreállítási időből becsüljük. Ehhez két további mérőszámot vezetünk be:

- **MTBF (Mean Time Between Failures):** a meghibásodások közötti átlagos idő. Azt mutatja, milyen gyakran következik be meghibásodás. Ha a hibaarányt λ-val jelöljük, akkor:
  $$\text{MTBF} = \frac{1}{\lambda}$$
  vagyis a hibaarány és az MTBF egymás reciprokai. Ha például egy komponens átlagosan 100 óránként hibásodik meg, akkor $\lambda = 0{,}01$ hiba/óra, az MTBF pedig 100 óra.
- **MTTR (Mean Time To Repair):** az átlagos helyreállítási idő. Tartalmazza a hiba észlelésétől a szolgáltatás ismételt elérhetővé válásáig eltelt teljes időt. Ide tartozik a riasztás megérkezése, a diagnózis, a javítás, az újraindítás, a konfiguráció visszaállítása és az ellenőrzés is. Ha a helyreállítási rátát μ-vel jelöljük, akkor:
  $$\text{MTTR} = \frac{1}{\mu}$$

E két mérőszámból a rendelkezésre állás stacionárius közelítése:

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Ez a képlet jól mutatja az összefüggést: a rendelkezésre állást két irányból javíthatjuk. Csökkenthetjük a meghibásodások gyakoriságát (növeljük az MTBF-et), vagy csökkenthetjük a helyreállítási időt (csökkentjük az MTTR-t). A gyakorlatban a második út gyakran gyorsabb és olcsóbb eredményt hoz.

Példa. Tegyük fel, hogy egy fizetési szolgáltatás átlagosan 30 naponta (720 óránként) hibásodik meg, és egy hiba átlagosan 2 óra állásidőt okoz. Ekkor:

$$A = \frac{720}{720 + 2} = 0{,}9972$$

vagyis a rendelkezésre állás 99,72%. Ha ugyanezen rendszer helyreállítási idejét automatizálással 2 óráról 30 percre csökkentjük, a rendelkezésre állás:

$$A = \frac{720}{720 + 0{,}5} = 0{,}9993$$

azaz 99,93%. Ugyanezt a javulást a hibaarány csökkentésével csak úgy érhetnénk el, ha az MTBF-et 720 óráról közel 1440 órára növelnénk, ami lényegesen nagyobb fejlesztési és üzemeltetési ráfordítást igényelhet.

A mérési ablakot mindig meg kell adni. Az SLO (Service Level Objective) például 30 napos gördülő ablakra vonatkoztatható, és így hangozhat: „a fizetési API rendelkezésre állása 30 napos ablakban legalább 99,9%”. Az SLO-ból származtatható az error budget:

$$\text{error budget} = 1 - \text{SLO}$$

Ha az SLO 99,9%, akkor egy 30 napos ablakban (43 200 perc) legfeljebb 43,2 perc állásidő áll rendelkezésre. Az error budget nem arra való, hogy kihasználjuk, hanem hogy láthatóvá tegye, mennyi terünk van a kockázatos változtatásokra, a karbantartásra és az elkerülhetetlen hibákra. A következő fejezetekben az SLO, SLI és error budget részletes tárgyalása is megjelenik, itt most elegendő annyi, hogy a rendelkezésre állási cél mindig mérési ablakkal együtt értelmezhető.

8.2.3 3. MTBF, MTTR és a „kilencesek” nyelvén

A rendelkezésre állást az iparban gyakran a tizedesjegyek számával fejezik ki: „három kilences” (99,9%), „négy kilences” (99,99%) és így tovább. Az alábbi táblázat egy 30 napos, azaz 43 200 perces ablakban mutatja a megengedett állásidőt.

Rendelkezésre állás	Éves állásidő (365 nap)	30 napos állásidő
99%	3,65 nap	432 perc
99,9%	8,76 óra	43,2 perc
99,99%	52,56 perc	4,32 perc
99,999%	5,26 perc	0,432 perc
99,9999%	31,5 másodperc	2,59 másodperc

A táblázat jól mutatja, hogy minden további kilences nagyságrenddel növeli az üzemeltetési és tervezési követelményeket. A 99,99%-os rendelkezésre álláshoz már nem elegendő az, hogy „majd újraindítjuk a szervert”; a helyreállításnak perceken belül, automatizáltan kell megtörténnie. A 99,999%-os rendelkezésre állás pedig már olyan architekturális döntéseket követel, mint a teljes redundancia, a földrajzi replikáció, a zéró érintésű deploy és a folyamatos állapotellenőrzés.

Az MTBF és MTTR nem egyszerűen számok, hanem döntési paraméterek. Az MTBF javítását célzó munka jellemzően a kódminőségre, a tesztelésre, a komponensek egyszerűsítésére és a megbízhatósági mérnöki gyakorlatra irányul. Az MTTR javítása ezzel szemben az üzemeltetési automatizálásra, a jó monitorozásra, a runbookokra, a gyors rollbackre és a failover mechanizmusokra fókuszál. Architektúratervezőként éppen ezért mindig érdemes megvizsgálni, hogy a rendelkezésre állási cél eléréséhez melyik irányba érdemes befektetni.

8.2.4 4. Redundancia, failover és degradáció

A hibatűrés alapvető eszközei a redundancia, a failover és a fokozatos degradáció.

8.2.4.1 4.1. Redundancia

A redundancia azt jelenti, hogy a rendszer egy adott funkciójához több, egymást helyettesíteni képes erőforrás áll rendelkezésre. A redundancia lehet:

- **Aktív-aktív (active-active):** minden példány forgalmat szolgál ki. Ha az egyik példány meghibásodik, a terhelés a többire hárul. Ez a modell jó kapacitáskihasználást ad, de megköveteli, hogy az állapotot a példányok között szinkronizáljuk, vagy a szolgáltatás állapotmentes legyen.
- **Aktív-passzív (active-passive):** egy példány szolgálja ki a forgalmat, a többi készenlétben áll. Ha az aktív példány meghibásodik, a tartalék átveszi a szerepét. Ez egyszerűbb állapotkezelést tesz lehetővé, de a tartalék erőforrás kihasználatlanul áll.
- **Meleg-, hideg- és melegtartalék:** a melegtartalék folyamatosan fut, és gyorsan át tudja venni a szerepet; a hidegtartalék le van állítva, és indítási idővel kell számolni. A kettő közötti választás közvetlenül befolyásolja az MTTR-t.

A redundancia önmagában nem elegendő ahhoz, hogy a rendszer hibatűrő legyen. A rendelkezésre állás csak akkor javul, ha a tartalék példány átvétele működik, és az átvétel közben sem veszítünk el kritikus állapotot. Ennek eszköze a failover mechanizmus.

8.2.4.2 4.2. Failover

A failover az a folyamat, amelynek során egy meghibásodott vagy elérhetetlen komponens szerepét egy másik komponens veszi át. A failover megvalósításának három kritikus lépése van:

1. **Hibaészlelés:** a rendszernek észlelnie kell, hogy egy komponens nem elérhető. Ez történhet health checkekkel, heartbeat-üzenetekkel, időkiesésekkel vagy aktív monitorozással. Az észlelés sebessége és megbízhatósága közvetlenül hat az MTTR-re.
2. **Döntés és átkapcsolás:** a rendszernek el kell döntenie, hogy valóban szükséges-e a failover, és ha igen, melyik tartalékot válassza. Téves pozitív észlelés esetén felesleges átkapcsolás történhet, ami önmagában is hibát okozhat.
3. **Állapot-összehangolás:** a tartalék példánynak rendelkeznie kell azzal az állapottal, amely a szolgáltatás folytatásához szükséges. Állapotmentes szolgáltatásoknál ez egyszerű; adatbázisoknál replikációt, elosztott naplókat és quorum-alapú döntéseket igényel.

Különösen fontos a split-brain probléma elkerülése. Ha egy elosztott rendszerben a hálózati kapcsolat megszakad, és mindkét oldal azt hiszi, hogy ő az aktív, akkor két példány írhat ugyanabba az adatbázisba, ami adatsérüléshez vezet. A megoldás általában valamilyen quorum-alapú mechanizmus, például egy harmadik döntőbíró, vagy olyan konzisztenciaprotokoll, amely csak akkor engedi az írást, ha a csomópontok többsége elérhető. Ez a kérdéskör a 11. fejezetben, az elosztott rendszerek tárgyalásánál mélyebben is előkerül.

8.2.4.3 4.3. Fokozatos degradáció

Nem minden meghibásodás indokol teljes leállást. A jól megtervezett rendszer a kritikus képességeket akkor is fenntartja, ha kevésbé fontos alrendszerek meghibásodnak. Ezt nevezzük fokozatos degradációnak (graceful degradation) vagy csökkentett üzemmódnak.

Például egy e-kereskedelmi rendszerben a termékkereső szolgáltatás kiesése esetén a katalógus böngészése továbbra is működhet, csak a keresőmező nem ad találatokat. Az architektúrának ilyenkor is vissza kell adnia egy egyértelmű hibajelzést vagy egy korlátozott funkciójú választ,

nem pedig a teljes oldal leállítását. A degradáció tervezésénél meg kell határozni, mely funkciók tekinthetők kritikusnak, és melyek elengedhetők átmeneti időre.

A degradáció egyik gyakori megvalósítása a circuit breaker minta. Ha egy külső szolgáltatás sorozatosan hibázik, a circuit breaker megszakítja a hívásokat, és azonnal hibát ad vissza, ahelyett hogy a hívó erőforrásait várakoztatná. Ez megakadályozza a hibák tovaterjedését, és időt ad a meghibásodott szolgáltatásnak a helyreállásra. A circuit breaker részletes tárgyalása a reziliencia fejezetben következik, itt most annyit jegyzünk meg, hogy a degradációs stratégia szerves része a hibatűrésnek.

8.2.5 5. Minőségi forgatókönyvek a rendelkezésre állásra

A rendelkezésre állással kapcsolatos követelményeket minőségi forgatókönyv (quality attribute scenario) formában érdemes rögzíteni. A forgatókönyv szerkezete: inger, környezet, válasz és mérőszám.

1. forgatókönyv: adatbázis-csomópont meghibásodása

- **Inger:** az elsődleges adatbázis-csomópont váratlanul leáll.
- **Környezet:** a rendszer csúcsidőben, másodpercenként 800 kéréses terhelés alatt fut, a replikált példány normál üzemmódban rendelkezésre áll.
- **Válasz:** a rendszer automatikusan átkapcsol a tartalék csomópontra, a bejövő kérések legfeljebb 1%-a hibázik, az állapot nem sérül.
- **Mérőszám:** az átkapcsolás időtartama legfeljebb 60 másodperc, a hibás kérések aránya az átállás 5 perces ablakában 1% alatt marad, az MTTR ebben az esetben nem haladja meg a 60 másodpercet.

2. forgatókönyv: hálózati szakadás két adatközpont között

- **Inger:** a két adatközpont közötti hálózati kapcsolat megszakad.
- **Környezet:** mindkét adatközpont aktív-aktív üzemmódban szolgálja a forgalmat; az adatbázis írása quorum-alapú konzisztenciát igényel.
- **Válasz:** a rendszer a kisebbségbe került oldalt írásra letiltja, és a forgalmat a többségi oldalra irányítja; az olvasások továbbra is mindkét oldalon elérhetők.
- **Mérőszám:** az írási hibaarány a szakadás első percében legfeljebb 5%, az adatvesztés 0, a teljes szolgáltatás rendelkezésre állása a hiba alatt legalább 99,5%.

Ezek a forgatókönyvek azért hasznosak, mert tesztelhetővé teszik a hibatűrést. Ha az architektúra nem tudja megmondani, hogy az adatbázis leállása esetén hány másodperc alatt áll helyre, akkor a rendelkezésre állási cél pusztán üres ígéret marad.

8.2.6 6. Labor: rendelkezésre állási cél és hibatűrési terv

A következő labor célja, hogy egy kis, ellenőrizhető példán gyakoroljuk a rendelkezésre állási cél meghatározását és a hibatűrési terv elkészítését. A labor reprodukálható dokumentumban vagy jegyzetfüzetben végezhető el, és a mért eredményeket mindig el kell különíteni a várt eredményektől.

8.2.6.1 6.1. Kiinduló állapot

Adott egy kitalált, „rendelési szolgáltatás” nevű belső rendszer, amelyet egyetlen konténerben futtatunk egyetlen virtuális gépen. A szolgáltatás REST API-t kínál a megrendelések rögzítésére, és egy PostgreSQL adatbázist használ, amely ugyanazon a gépen fut. A rendszernek nincs terheléselosztója, nincs tartalék példánya, és a naplózás is csak a helyi fájlrendszerre ír. A szolgáltatás hétfőtől péntekig, munkaidőben, másodpercenként átlagosan 20 kérést kap.

Az üzemeltetési csapat azt tapasztalta, hogy az adatbázis lemezhibája esetén a rendszer átlagosan 4 óra alatt áll helyre, mert manuálisan kell visszaállítani a biztonsági mentésből. A fejlesztőcsapat szerint a szolgáltatás kódminősége közepes, és évente átlagosan 3 alkalommal fordul elő olyan szoftverhiba, amely teljes újraindítást igényel.

8.2.6.2 6.2. Elvárt eredmény

A labor eredménye egy rendelkezésre állási cél és egy hibatűrési terv, amely a következőket tartalmazza:

- a mért vagy becsült jelenlegi MTBF, MTTR és rendelkezésre állás értékét;
- a célzott rendelkezésre állást mérési ablakkal és mérőszámmal;
- legalább két minőségi forgatókönyvet;
- egy architektúradöntést ADR formában;
- a tervezett és a mért értékeket elkülönítve;
- ellenőrzési szempontokat.

8.2.6.3 6.3. A jelenlegi állapot számszerűsítése

A jelenlegi állapot becslése a kiinduló adatokból:

$$\mathrm{MTBF}_{\mathrm{becsült}} = \frac{365\,\mathrm{nap}}{3\,\mathrm{hiba}} \approx 121{,}7\,\mathrm{nap} \approx 2920\,\mathrm{óra}$$

$$\mathrm{MTTR}_{\mathrm{becsült}} = 4\,\mathrm{óra}$$

$$A_{\mathrm{becsült}} = \frac{2920}{2920 + 4} \approx 0{,}9986$$

Ez 99,86%-os rendelkezésre állásnak felel meg. Fontos, hogy ez a szám becslés, nem mért érték. A labor során a jegyzőkönyvben ezt külön kell jelölni: „becsült érték, a méréshez 90 napos megfigyelési ablak szükséges”.

8.2.6.4 6.4. Célzott rendelkezésre állás

A termékmenedzserrel egyeztetve a cél a 99,9%-os rendelkezésre állás 30 napos gördülő ablakban. Ez 43,2 perc állásidőt enged havonta. A jelenlegi 4 órás MTTR önmagában is meghaladja ezt a keretet, ezért a helyreállítási időt mindenképpen csökkenteni kell. Az üzleti elvárás az, hogy a rendelés rögzítése akkor is működjön, ha a háttéradatbázis átmenetileg nem elérhető.

8.2.6.5 6.5. Hibatűrési terv

A terv három változtatást irányoz elő:

1. **Állapotmentes alkalmazás és terheléselosztó bevezetése.** A rendelési szolgáltatást két példányban futtatjuk, és elé terheléselosztót helyezünk. Az alkalmazásból eltávolítjuk a lokális állapotot, és a munkamenet-adatokat a megosztott gyorsítótárba helyezzük.
2. **Adatbázis replikáció és automatikus failover.** Az adatbázist aktív-passzív replikációval látjuk el. A passzív példány folyamatosan követi az aktív példány tranzakciós naplóját, és 30 másodpercen belül át tudja venni a szerepét.
3. **Fokozatos degradáció bevezetése.** Ha az adatbázis rövidebb időre nem elérhető, a szolgáltatás a bejövő rendeléseket egy helyi, tartós várólistába írja, és a kliensnek „elfogadva, feldolgozás alatt” státuszt ad vissza. Amint az adatbázis helyreáll, a várólista feldolgozása automatikusan megtörténik.

Ezek a változtatások a becslés szerint az MTTR-t 4 óráról 60 másodpercre csökkentik. Az MTBF-re gyakorolt hatásuk semleges, mert a szoftverhibák gyakoriságát nem befolyásolják közvetlenül. A becsült jövőbeli rendelkezésre állás:

$$A_{\text{tervezett}} = \frac{2920}{2920 + 0{,}0167} \approx 0{,}999994$$

Ez a becslés azonban túlzottan optimista, mert nem számol a degradációs várólista megtelésével, a failover téves pozitív észlelésével és az emberi hibákkal. A laborban ezért a tervezett érték mellé konzervatívabb, 99,95%-os célt is rögzíteni kell, és a tényleges értéket 30 napos mérés után lehet megerősíteni vagy cáfolni.

8.2.6.6 6.6. Architektúradöntés ADR formában

ADR-007: Rendelési szolgáltatás rendelkezésre állásának növelése

- **Állapot:** elfogadva
- **Kontextus:** A rendelési szolgáltatás jelenleg egyetlen példányon fut, és 4 órás MTTR-rel rendelkezik. Az üzleti cél 99,9%-os rendelkezésre állás 30 napos ablakban. A jelenlegi 4 órás állásidő meghaladja a havi 43,2 perces error budgetet.
- **Döntés:** A rendelési szolgáltatást két példányossá alakítjuk, elé terheléselosztót helyezünk. Az adatbázist aktív-passzív replikációval és automatikus failoverrel látjuk el. A szolgáltatásba degradációs várólistát építünk, amely az adatbázis kiesésekor a rendeléseket pufferként fogadja.
- **Alternatívák:**

  1. *Hagyományos monolitikus megoldás megtartása, és a helyreállítási folyamat gyorsítása.* Elutasítva, mert a manuális visszaállítás akkor is meghaladná a 43,2 perces keretet, ha a folyamatot részben automatizáljuk.
  2. *Aktív-aktív adatbázis-fürt bevezetése.* Elutasítva egyelőre, mert a jelenlegi terhelés és az üzleti kockázat nem indokolja a quorum-alapú írás és a konfliktuskezelés többletköltségét.
  3. *Külső, felügyelt adatbázis-szolgáltatás igénybevétele.* Elfogadva vizsgálatra, de a jelenlegi döntés helyben tartja az adatbázist, mert a hálózati késleltetés és az adatbesorolás korlátozza a külső szolgáltatót.

- **Következmények:** Az MTTR várhatóan 60 másodperc alá csökken. Növekszik az üzemeltetési komplexitás, mert a replikációt és a failovert monitorozni kell. A degradációs várólista új hibalehetőséget visz a rendszerbe, ezért annak telítettségét és visszajátszását külön tesztelni kell.
- **Mérési terv:** A rendelkezésre állást 30 napos gördülő ablakban mérjük. A failover időtartamát legalább negyedévente, izolált környezetben, szimulált adatbázis-leállással ellenőrizzük. A várólista feldolgozási késleltetését a normál üzemi terhelés mellett mérjük.

8.2.6.7 6.7. Ellenőrzési szempontok

A labor akkor tekinthető sikeresnek, ha:

- a jelenlegi állapot becslése és a tervezett értékek külön szerepelnek;
- minden mérési adathoz tartozik környezet, időpont, konfiguráció és bizonytalanság;
- a minőségi forgatókönyvek ingert, környezetet, választ és mérőszámot tartalmaznak;
- az ADR-ben a döntés, az alternatívák és a következmények elkülöníthetők;
- a failover teszteléséhez izolált környezetet és visszaállítási tervet rendeltek;
- a degradációs várólista megtelésére és a visszajátszás sorrendjére is készült teszterv;
- a dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy ügyfélnevet.

8.2.7 7. Gyakori hibák és félreértések

A rendelkezésre állás tervezésekor több tipikus hiba ismétlődik.

A rendelkezésre állás és az adatkonzisztencia összekeverése. Ha egy adatbázis a failover során elveszít egy másodpercnyi tranzakciót, a rendszer lehet „elérhető”, de az adatai sérültek. A rendelkezésre állási mérőszámok mellett mindig meg kell határozni az adatvesztés megengedett mértékét is.

A redundancia téves biztonságérzete. Ha két példány ugyanazon a fizikai gépen vagy ugyanabban a hálózati szegmensben fut, a redundancia csak korlátozottan véd. A hibafatörzs-elemzésnél a közös okokat, például a közös áramforrást, a közös hálózati kapcsolatot vagy a közös szoftverhibát is figyelembe kell venni.

A MTTR alulbecslése. A helyreállítási idő nem csak az újraindításból áll. Tartalmazza a hiba észlelését, a riasztás továbbítását, az ügyeletes mérnök reakcióidejét, a diagnózist, a javítást és az ellenőrzést. Ha ezek közül bármelyik manuális, a valós MTTR percekkel vagy órákkal hosszabb lehet a tervezettnél. Az automatizálás és a jól felépített runbookok közvetlenül az MTTR-t csökkentik.

A failover túl agresszív beállítása. Ha a rendszer egyetlen sikertelen health check miatt azonnal átkapcsol, a gyakori flapping instabilitást okoz. Az észlelési küszöböt, a próbálkozások számát és a késleltetést úgy kell beállítani, hogy a valós hibákat gyorsan, a téves pozitívokat viszont ritkán váltsa ki.

A degradáció hiánya. Sok rendszer így viselkedik: ha bármelyik külső szolgáltatás nem válaszol, az egész kérés meghiúsul. A kritikus üzleti tranzakciókat érdemes úgy megtervezni, hogy a kevésbé fontos hívások kiesése esetén is tovább haladhassanak.

8.2.8 8. Döntési gyakorlat és önellenőrzés

Döntési gyakorlat. Egy rendszer komponense 1000 óránként hibásodik meg, a helyreállítás átlagosan 30 percet vesz igénybe. Számítsa ki a rendelkezésre állást a megadott képlettel, majd döntse el, hogy 99,9%-os SLO eléréséhez elegendő-e az MTTR csökkentése 10 percre, vagy a hibaarányt is csökkenteni kell. Válaszát a mérési ablak megadásával indokolja.

Ellenőrzőlista a hibatűrési tervhez

- Van-e a rendszernek mérési ablakkal megadott rendelkezésre állási célja?
- Ismerjük-e a jelenlegi MTBF és MTTR értékét, vagy legalább van-e rájuk becslés?
- A redundancia nemcsak névleges, hanem független hibadománnyal is rendelkezik?
- A failover észlelési és átkapcsolási ideje mért vagy becsült?
- Van-e terv a split-brain elkerülésére?
- A degradációs üzemmódok meghatározottak, és a kritikus funkciók továbbra is működnek?
- A mért és a várt eredményeket elkülönítve dokumentáljuk?

Önellenőrző kérdés. Miért éri meg sok esetben a rendelkezésre állást az MTTR csökkentésével javítani a hibaarány csökkentése helyett, és milyen kockázatokkal jár, ha kizárólag a gyors helyreállításra támaszkodunk?

A következő fejezet a biztonság, az adatvédelem és a megfelelés kérdéskörét tárgyalja, amelyre az itt bevezetett hibatűrési és rendelkezésre állási fogalmak közvetlenül építenek.

8.3 Biztonság, adatvédelem és megfelelés

A biztonsági és adatvédelmi követelmények nem egyszerűen egy ellenőrzőlista utolsó tételei, hanem olyan architekturális erők, amelyek már a rendszerhatár meghúzásakor, a komponensek felelősségének meghatározásakor és az adatfolyamok tervezésekor alakítják a szerkezetet. Egy rosszul megválasztott biztonsági határ vagy egy figyelmen kívül hagyott adatvédelmi kényszer nem utólag befoltozható egy API-kapcsolatba vagy egy adatbázis-sémába; ilyenkor általában a teljes komponensfelosztást, a telepítési topológiát és az üzemeltetési eljárásokat is újra kell gondolni. A fejezet ezért a biztonságot és az adatvédelmet nem mint különálló technológiát tárgyalja, hanem mint olyan minőségi követelménycsoportot, amelynek közvetlen hatása van az architektúra döntéseire.

Az itt bevezetett fogalmakra épül a Karbantarthatóság, használhatóság és üzemeltethetőség fejezet a karbantarthatóság és üzemeltethetőség kapcsán, valamint a későbbi biztonsági architektúra fejezetek, ezért ebben az egységben az alapokat, a gyakorlati mintát és az ellenőrzést együtt helyezzük el. A fejezet nem ismétli meg a Rendelkezésre állás, megbízhatóság és hibatűrés fejezet rendelkezésre állási és hibatűrési anyagát, de a biztonsági incidens okozta kiesést itt is figyelembe vesszük.

8.3.1 A biztonsági attribútumok: CIA és társaik

A biztonsági követelmények klasszikus magját három attribútum alkotja: bizalmasság (confidentiality), integritás (integrity) és rendelkezésre állás (availability). Ezt a hármast gyakran CIA-hármasként emlegetik, és bár nem fedi le a biztonság teljes területét, hasznos kiindulópont az architektúra szempontjából.

- **Bizalmasság:** az adatokhoz és erőforrásokhoz csak az arra jogosult entitások férhetnek hozzá. Architekturálisan ez azt jelenti, hogy az adatáramlási útvonalakon titkosítást, a komponensek között hitelesítést és engedélyezést, az adattárolásban pedig hozzáférés-szabályozást és gyakran titkosítást alkalmazunk. A bizalmasság megsértése például egy jogosulatlan olvasás, adatszivárgás vagy belső fenyegetés.
- **Integritás:** az adatok és a rendszer működése nem módosítható jogosulatlanul, és a változások nyomon követhetők. Az integritás nemcsak az adatra, hanem a kódra, a konfigurációra és a naplókra is vonatkozik. Architekturálisan digitális aláírásokat, üzenet-hitelesítő kódokat, verziózott és védett naplókat, valamint ellenőrzött build- és telepítési folyamatokat igényel.
- **Rendelkezésre állás:** a rendszer az elfogadott időtartamban használható marad, és ellenáll a szándékos túlterheléses támadásoknak is. A Rendelkezésre állás, megbízhatóság és hibatűrés fejezetben tárgyalt hibatűrési mechanizmusok itt kiegészülnek a szolgáltatás-megtagadásos (DoS, DDoS) támadásokkal szembeni védelemmel, például forgalomszűréssel, sebességkorlátozással és kapacitástartalékkal.

A CIA-hármas mellett az ISO/IEC 25010 minőségi modell további biztonsági jellemzőket is felsorol: hitelesség (authenticity), letagadhatatlanság (non-repudiation), elszámoltathatóság (accountability) és adatbizalmasság. Az architektúra szempontjából ezek azt jelentik, hogy a rendszernek képesnek kell lennie igazolni, ki mit tett, és a műveletek nem tagadhatók le. Ez naplózást, azonosítási mechanizmusokat és a naplóbejegyzések integritásvédelmét követeli meg.

8.3.2 Adatvédelem: a személyes adat mint védendő erőforrás

A személyes adatok védelme nem azonos a bizalmasság fogalmával, bár szorosan összefügg vele. Az adatvédelem azt szabályozza, hogy a természetes személyekre vonatkozó adatokat milyen célból, milyen jogalappal, meddig és hogyan lehet kezelni. Az Európai Unióban az általános adatvédelmi rendelet (GDPR) alapelvei közvetlen architekturális követelményekké fordíthatók:

- **Jogszerűség, tisztességes eljárás és átláthatóság:** a rendszernek ismernie kell az adatkezelés célját és jogalapját, és ezt az érintett számára érthetően kommunikálni kell. Ehhez az adatkezelési folyamatokat explicit módon kell modellezni, nem elég egy általános „biztonságos” felirat.
- **Célhoz kötöttség:** a személyes adat gyűjtése és kezelése csak meghatározott, egyértelmű és jogszerű célból történhet. Ez tiltja az adatok korlátlan újrafelhasználását. Architekturálisan az adatfolyamokat úgy kell kialakítani, hogy minden adatáramláshoz hozzárendelhető legyen a cél, és a célváltás külön döntést kapjon.
- **Adattakarékosság:** csak a cél eléréséhez szükséges adatokat szabad kezelni. A rendszertervben ez azt jelenti, hogy az adatmodellben nem szerepelhet felesleges személyes adat, és a naplókban sem tárolunk szükségtelen azonosítókat.
- **Pontosság:** a személyes adatoknak pontosnak és naprakésznek kell lenniük. A rendszernek mechanizmust kell biztosítani a helyesbítésre és a frissítésre.
- **Tárolási korlátozás:** az adatot csak a cél eléréséhez szükséges ideig lehet tárolni. Ez megőrzési időket, ütemezett törlést és anonimizálást követel az adatbázisban, a biztonsági mentésekben és a naplókban egyaránt.
- **Integritás és bizalmasság:** a személyes adatokat megfelelő technikai és szervezési intézkedésekkel védeni kell.
- **Elszámoltathatóság:** az adatkezelőnek bizonyítania kell a megfelelést. Ez dokumentációt, naplózást és ellenőrizhető folyamatokat igényel.

Az adatvédelemhez kapcsolódik két gyakran kevert fogalom: a **pszeudonimizálás** és az **anonimizálás**. A pszeudonimizálás során a közvetlen azonosítókat kódokkal helyettesítjük, de

a hozzárendelés visszafordítható, ezért az adat továbbra is személyes adatnak minősül, csak csökkenti a kockázatot. Az anonimizálás viszont visszafordíthatatlan, és ha sikerül, az adat már nem minősül személyes adatnak. Az architektúrában ez a különbségtétel meghatározza, hogy az adott adathalmazra vonatkoznak-e a GDPR kötelezettségei, például a törlési jog vagy az adathordozhatóság.

8.3.3 Jogszabályi és ágazati kényszerek

A biztonsági és adatvédelmi követelmények jelentős részét jogszabályok, ágazati szabványok és szerződéses kötelezettségek írják elő. Ezek nem csupán jogi dokumentumok; belőlük rendszertervezési kényszerek származnak. A legfontosabbakat itt röviden áttekintjük, de a konkrét követelményeket mindig az adott időpontban hatályos hivatalos szöveg alapján kell ellenőrizni, mert a szabályozás folyamatosan változik. Ez a könyv nem nyújt jogi tanácsadást.

- **GDPR** (EU 2016/679): az Európai Unió általános adatvédelmi rendelete. Meghatározza a személyes adatok kezelésének feltételeit, az érintettek jogait (hozzáférés, helyesbítés, törlés, adathordozhatóság, tiltakozás), az adatkezelői és adatfeldolgozói felelősséget, az adatvédelmi hatásvizsgálat (DPIA) kötelezettségét és a bejelentési kötelezettségeket. Architekturálisan a GDPR hatással van az adatmodellre, a naplózásra, a mentésre, a hozzáférés-szabályozásra és a szolgáltatók kiválasztására.
- **PCI-DSS:** a fizetőkártya-adatok kezelésére vonatkozó ágazati szabvány. Előírja a kártyaadatok minimális tárolását, a hálózati szegmentációt, a titkosítást, a naplózást és a rendszeres biztonsági tesztelést. Bár nem jogszabály, a kártyatársaságok szerződéses kötelezettségként érvényesítik.
- **HIPAA** (Health Insurance Portability and Accountability Act, USA): az egészségügyi adatok védelmét szabályozza az Egyesült Államokban. Az elektronikus egészségügyi nyilvántartásokra szigorú hozzáférés-szabályozást, audit naplózást és adatvédelmi intézkedéseket ír elő. Nemzetközi termékeknél is gyakran irányadó.
- **NIS2:** az Európai Unió hálózati és információs rendszerek biztonságáról szóló irányelve. Kritikus infrastruktúrák és fontos szervezetek számára kockázatkezelési, incidensjelentési és ellátási lánc biztonsági követelményeket ír elő.
- **DORA** (Digital Operational Resilience Act): az EU pénzügyi szektorára vonatkozó rendelet, amely az informatikai kockázatkezelést, a digitális működési rezilienciát, a harmadik fél kockázatkezelést és az incidensjelentést szabályozza.
- **AI Act:** az EU mesterséges intelligenciára vonatkozó rendelete, amely kockázatalapú kategóriákba sorolja az MI-rendszereket, és magas kockázatú esetekben megfelelőségi, átláthatósági és emberi felügyeleti követelményeket ír elő. Az MI-architektúrák tervezésekor ez a későbbi fejezetben visszatér.

E szabályozások közös architekturális hatása, hogy a rendszernek **bizonyítania** kell a megfelelést: mérhető naplókat, verziózott konfigurációt, dokumentált adatfolyamokat és ellenőrizhető hozzáférési döntéseket kell produkálnia. A „papíron megfelelünk” megközelítés nem helyettesíti a technikai kontrollokat.

8.3.4 A biztonsági követelmények architekturális hatása

A biztonsági és adatvédelmi követelmények számos ponton alakítják az architektúrát. Az alábbi területeken a döntések nem halaszthatók a megvalósítási fázisra.

Rendszerhatárok és bizalmi zónák. A biztonságos tervezés alapja a bizalmi határok (trust boundary) explicit jelölése. A C4 context és container szintű diagramon meg kell különböztetni

a belső rendszert, a külső rendszereket és a felhasználói szerepköröket, valamint jelölni kell, hol lép át adat egy bizalmi határt. A bizalmi határok mentén hitelesítést, engedélyezést és gyakran titkosítást alkalmazunk. A hálózati szegmentáció, a demilitarizált zóna (DMZ) és a mikroszegmentáció mind ezt a határolást szolgálja.

Azonosítás, hitelesítés és engedélyezés (IAM). A rendszernek megbízhatóan azonosítania kell a felhasználókat és szolgáltatásokat. Az architektúrában el kell dönteni, hogy milyen protokollt használunk (például OIDC, SAML, OAuth2), hogyan kezeljük a jogosultságokat (RBAC, ABAC, ReBAC), és hol húzzuk meg az engedélyezési pontot. A minimális jogosultság elve (least privilege) minden komponensre, szolgáltatásfiókra és emberi szerepkörre vonatkozik.

Titkosítás és kulcskezelés. A bizalmasság és integritás technikai alapja a kriptográfia. A szállítás közbeni titkosítás (TLS 1.2 vagy újabb) a hálózati kapcsolatokon alapértelmezett. Az adattárolási titkosítás (at rest) az adatbázisokra, fájltárakra és biztonsági mentésekre is kiterjed. A kulcskezelés architekturális kérdés: használjunk-e felhőalapú KMS-t, hardveres biztonsági modult (HSM), vagy saját kulcsszervert. A boríték-titkosítás (envelope encryption) mintázata lehetővé teszi, hogy az adatokat adatkulccsal titkosítsuk, az adatkulcsot pedig mesterkulccsal védjük, így a kulcsok rotálhatók anélkül, hogy az összes adatot újra kellene titkosítani.

Audit naplózás. A biztonsági eseményeket és a jogosultsági döntéseket strukturált naplóban rögzíteni kell: ki, mit, mikor, milyen eredménnyel végzett. A naplónak integritásvédettnek kell lennie (például hash-lánc vagy aláírás), és megőrzési idővel kell rendelkeznie, amelyet az adatvédelmi előírásokkal is össze kell hangolni. A naplózás nem tárolhat felesleges személyes adatot, például jelszót vagy teljes kártyaszámot.

Adatkezelési folyamatok. A GDPR elveiből közvetlen tervezési követelmények következnek. Az adatmodellben jelölni kell a személyes adat mezőket, az adatmegőrzési időket, a törlési és anonimizálási eljárásokat. A törlési jog teljesítéséhez az architektúrának ismernie kell az adat másolatait, például a biztonsági mentéseket és a cache-eket is. Az adathordozhatósághoz exportálható, géppel olvasható formátumot kell biztosítani. A DPIA kötelezettség esetén az adatfolyamokat részletesen dokumentálni kell.

Sebezhetőség-kezelés és ellátási lánc biztonság. A rendszer külső függőségei (könyvtárak, konténerképek, futási környezet) biztonsági kockázatot hordoznak. Az architektúrának támogatnia kell az SBOM (Software Bill of Materials) készítését, a függőségek frissítését, a digitális aláírást és a reprodukálható buildeket. A szoftverellátási lánc támadásai ellen a build- és telepítési folyamatot is védett folyamatként kell kezelni.

Monitoring, riasztás és incidenskezelés. A biztonsági eseményeket észlelni kell, ezért a megfigyelhetőségi eszköztárnak (logs, metrics, traces) ki kell terjednie a biztonsági jelzésekre is. A riasztási szabályoknak és a runbookoknak tartalmazniuk kell a gyanús tevékenységek kezelését, például a többszöri sikertelen bejelentkezést, a jogosultság-módosítást vagy a nagy mennyiségű adatexportálást.

8.3.5 Minőségi forgatókönyvek biztonságra

A biztonsági követelményeket ugyanúgy érdemes quality attribute scenario formában megfogalmazni, mint a többi minőségi jellemzőt. A séma: **inger** – **környezet** – **válasz** – **mérőszám**. A biztonsági szcenáriók esetében az inger gyakran egy támadási kísérlet vagy egy jogosulatlan művelet, a válasz pedig az észlelés, blokkolás vagy korlátozás.

Példa:

> **Inger:** egy érvényes felhasználói fiókkal rendelkező entitás megkísérel egy másik felhasználó páciensrekordját lekérdezni a REST API-n keresztül. **Környezet:** normál üzem, éles rendszer, hitelesített munkamenet. **Válasz:** az engedélyezési komponens a kérést elutasítja, a műveletet biztonsági naplóban rögzíti, és a válasz 403 Forbidden kódot ad; a figyelőrendszer riasztást küld, ha az ismétlődő próbálkozások meghaladják a küszöböt. **Mérőszám:** az elutasítás 30 ms-on belül megtörténik; a naplóbejegyzés legfeljebb 1 s múlva megjelenik a kereshető naplóban; a riasztás 5 percen belül kiértesül a biztonsági ügyeletnek.

Egy másik példa a titkosítási követelményre:

> **Inger:** egy adatbázis-mentést tartalmazó fájl illetéktelen kezekbe kerül. **Környezet:** offline, nem üzemi hálózat. **Válasz:** a mentés tartalma titkosított marad, a kulcs nem szerepel a mentéssel együtt, és a kulcs-visszavonási eljárás indítható. **Mérőszám:** a visszafejtéshez szükséges kulcs nem tölthető le a mentést tároló rendszerből; a kulcs-visszavonás 1 órán belül érvénybe lép.

A minőségi forgatókönyvek mellett a fenyegetésmodellezés (threat modeling) segít a fenyegetések szisztematikus feltárásában. Az egyszerű STRIDE módszer a fenyegetéseket hat kategóriába sorolja: Spoofing (megszemélyesítés), Tampering (módosítás), Repudiation (letagadás), Information disclosure (információszivárgás), Denial of service (szolgáltatásmegtagadás), Elevation of privilege (jogosultság-kiterjesztés). Ezt a témát a Threat modeling és támadási felület fejezet részletesen tárgyalja, de már itt is használjuk a fenyegetéslista formájában.

8.3.6 Tipikus hibák és ellenminták

A biztonsági és adatvédelmi követelmények kezelésében több visszatérő hiba figyelhető meg. A hibák felismerése és a helyes megoldás indoklása a fejezet egyik tanulási célja.

1. Biztonság utólagos hozzáadása. Ha a biztonságot a fejlesztés végén próbáljuk „ráhúzni” a rendszerre, akkor a hozzáférés-szabályozás, a titkosítás és a naplózás hiánya a komponensek közötti kapcsolatokban mélyen beágyazódik. A helyes megoldás: a biztonsági követelményeket már a rendszerhatárok meghúzásakor és a minőségi forgatókönyvekben rögzíteni.

2. Túlzott jogosultság és nincs minimális jog elv. Ha minden szolgáltatás adatbázis-rendszergazda jogosultsággal fut, egy kompromittált komponens az egész adatbázist elérheti. A helyes megoldás: minden komponens csak a saját feladatához szükséges jogosultságot kapja, elkülönített szolgáltatásfiókkal, és a hozzáférést rendszeresen felülvizsgálják.

3. Titkosítás hiánya belső hálózaton. Gyakran feltételezik, hogy a belső hálózat megbízható, ezért a szolgáltatások közötti forgalom nyílt szövegű. Ha egy támadó bejut a hálózatra, minden adatot elfoghat. A helyes megoldás: a szolgáltatások közötti kommunikációban is TLS-t vagy legalább belső hálózati titkosítást használunk, és a bizalmi határokat a hálózaton belül is jelöljük.

4. Hardcoded titkok. A jelszavak, API-kulcsok és tanúsítványok beégetése a kódba vagy a konfigurációs fájlba súlyos sebezhetőség. A helyes megoldás: titokkezelő szolgáltatás (secrets manager), rövid életű hozzáférési tokenek, valamint a build- és deploy folyamatból a titkok kizárása.

5. Naplózás személyes adattal. A biztonsági naplókba gyakran bekerül a teljes kérés URL, a fejléc, a test, így jelszó, token vagy egészségügyi adat is. Ez egyszerre sérti az adatvédelmet és növeli a kockázatot. A helyes megoldás: a naplóeseményeket strukturáltan, adatminimalizálással tervezzük, az érzékeny mezőket maszkoljuk vagy kihagyjuk.

6. Compliance mint dokumentációs feladat. Egy megfelelőségi tanúsítvány megléte nem helyettesíti a technikai kontrollokat. Ha az audit naplózás, a hozzáférés-szabályozás vagy a titkosítás nem működik a gyakorlatban, a megfelelés csak papíron létezik. A helyes megoldás: a megfelelőségi követelményeket minőségi forgatókönyvekké alakítjuk, és folyamatosan mérjük, nem csak audit időpontban.

7. Nincs adatmegőrzési és törlési politika. Ha a rendszer korlátlanul tárolja a személyes adatokat, az sérti a GDPR tárolási korlátozás elvét, és növeli az adatszivárgás kockázatát. A helyes megoldás: minden személyes adat mezőre megőrzési időt definiálunk, és ütemezett törlést vagy anonimizálást futtatunk, beleértve a mentéseket is.

8. Harmadik fél függőségek ellenőrizetlen bevonása. Egy külső könyvtár vagy szolgáltatás biztonsági rése az egész rendszert veszélyezteti. A helyes megoldás: SBOM készítése, függőség-frissítési politika, sebezhetőség-szkennelés a CI/CD folyamatban, és a beszállító biztonsági gyakorlatának értékelése.

9. Egyetlen bizalmi pont. Ha az egész rendszer egyetlen hitelesítő szolgáltatásra vagy tűzfalra épül, annak kompromittálódása katasztrofális. A helyes megoldás: rétegzett védelem (defense in depth), a bizalmi határok egymástól független kontrollokkal, és a kritikus komponensek izolációja.

8.3.7 Gyakorlati példa: páciensportál

A következő példa egy kitalált páciensportál, amelyen keresztül a betegek időpontot foglalhatnak, megtekinthetik leleteiket, és üzenetet válthatnak az orvossal. A példa célja a biztonsági és adatvédelmi követelmények architekturális hatásának bemutatása.

Kontextus. A rendszer szereplői: páciens, orvos, asszisztens, külső laboratórium, hitelesítő szolgáltató (OIDC), és egy auditnapló-szolgáltatás. A rendszerhatár a páciensportál belső komponenseit választja el a külső rendszerektől.

Biztonsági követelmények. A páciensrekordok különleges kategóriájú személyes adatok, ezért a bizalmasság és az integritás kiemelt. A rendszernek meg kell felelnie a GDPR-nak, és ha az adatokat egészségügyi célra használják, az ágazati előírásoknak is. A biztonsági követelményeket quality attribute scenario formában rögzítjük:

- **Hozzáférhetőség:** egy páciens csak a saját rekordját láthatja; az orvos csak a hozzá rendelt betegekét. A válasz: a kérés elutasítása és naplózás. Mérőszám: az engedélyezési ellenőrzés 50 ms alatt lefut, a naplóbejegyzés 1 s alatt megjelenik.
- **Integritás:** a lelet módosítása csak a laboratórium és az orvos által lehetséges, és minden módosítás verziózott, aláírt. Mérőszám: a módosított rekord 100%-a digitálisan aláírt, a visszautasítás aránya 0%.
- **Rendelkezésre állás:** a rendszer 99,9%-os havi SLO-t teljesít, de a DoS támadásokkal szemben is ellenáll. Mérőszám: egy 10 Gbit/s-os terheléses támadás alatt a bejelentkezési válaszidő 95%-a 500 ms alatt marad.

Fenyegetéslista. Az alábbi táblázat a legfontosabb fenyegetéseket, érintett erőforrásokat és ellenintézkedéseket foglalja össze. A lista a STRIDE kategóriákat követi, de nem teljes; a Threat modeling és támadási felület fejezetben részletesen bővítjük.

Fenyegetés	Érintett erőforrás	Lehetséges hatás	Ellenintézkedés
Megszemélyesítés (Spoofing)	Bejelentkezési folyamat	Jogosulatlan hozzáférés	OIDC, többfaktoros hitelesítés, munkamenet-kezelés
Módosítás (Tampering)	Leletrekord	Hamis diagnózis, bizalomvesztés	Digitális aláírás, verziózás, audit napló
Letagadás (Repudiation)	Módosítási művelet	Felelősségre vonhatóság hiánya	Integritásvédett napló, hitelesített műveletek
Információszivárgás (Information disclosure)	Páciensrekord	Adatvédelmi incidens	TLS 1.2+, at rest titkosítás, RBAC
Szolgáltatásmegtagadás (DoS)	Belépési komponens	Elérhetőség kiesése	Sebességkorlátozás, WAF, kapacitástartalék
Jogosultság-kiterjesztés (Elevation of privilege)	API végpontok	Jogosulatlan adathozzáférés	Minimális jog, ABAC, rendszeres jogosultság-felülvizsgálat

Döntési napló (ADR kivonat). A biztonsági döntéseket ADR formában rögzítjük, hogy a jövőbeli olvasó megértse az okokat és az elvetett alternatívákat.

ADR-001: OIDC használata a hitelesítésre - *Kontextus:* A páciensportálnak megbízhatóan azonosítania kell a felhasználókat, és támogatnia kell a szövetségi bejelentkezést. - *Döntés:* OpenID Connect (OIDC) protokollt használunk, nem saját felhasználó-adatbázist. - *Alternatívák:* SAML (nagyvállalati környezetben elterjedt, de nehezebb integrálni webes kliensekkel), saját hitelesítés (többlet felelősség, nagyobb kockázat). - *Következmények:* a hitelesítő szolgáltató üzemeltetése külső; a munkamenet-kezelést a saját komponensnek kell megoldani; a felhasználói adatok egy részét a szolgáltató is kezeli, ezért adatfeldolgozói szerződés szükséges. - *Állapot:* elfogadva.

ADR-002: Titkosítás at rest és in transit - *Kontextus:* A páciensrekordok bizalmas jellegűek, és a GDPR megköveteli a megfelelő technikai intézkedéseket. - *Döntés:* Minden külső kapcsolat TLS 1.2+ protokollal, belső kapcsolat is TLS 1.2+; az adatbázis és a biztonsági mentés AES-256 titkosítással, felhőalapú KMS segítségével. - *Alternatívák:* csak külső titkosítás (belső hálózati fenyegetésre érzékeny), saját HSM (nagyobb üzemeltetési költség, magasabb biztosíték). - *Következmények:* a TLS konfigurációt minden szolgáltatásnál karban kell tartani; a KMS kulcsrotációt ütemezni kell; a mentés visszaállításához kulcs-hozzáférés szükséges. - *Állapot:* elfogadva.

ADR-003: Audit napló integrálása - *Kontextus:* Az elszámoltathatóság és a GDPR megfelelés igazolása naplózást igényel. - *Döntés:* Strukturált, integritásvédett audit naplót vezetünk, amely a hozzáférési döntéseket, a módosításokat és a biztonsági eseményeket rögzíti. A naplókat külön szolgáltatásba küldjük, és 1 évig megőrizzük. - *Alternatívák:* helyi naplófájlok (könnyen manipulálhatók, nehezen központosíthatók), csak alkalmazáslog (nem fedi le a biztonsági eseményeket). - *Következmények:* a naplószolgáltatás a rendszer kritikus komponensévé válik; a naplóban személyes adat minimalizálása szükséges; a naplózási költség növekszik. - *Állapot:* elfogadva.

Ellenőrzési szempontok a példához. A fenti döntések ellenőrzéséhez a következőket kell mérni vagy automatikusan tesztelni: a TLS tanúsítványok érvényesek és a protokoll verzió megfelelő; a hozzáférési kérések 100%-a naplózva van; a jogosultság-módosításokhoz jóváhagyás szükséges; a törlési jog kérése 30 napon belül teljesül; a biztonsági mentések titkosítottak.

8.3.8 Labor: biztonsági követelmény- és fenyegetéslista készítése

A labor célja, hogy egy választott, kisméretű rendszerre biztonsági követelmény- és fenyegetéslistát készíts, reprodukálható dokumentumban vagy jegyzetfüzetben. A labor nem igényel éles rendszert vagy destruktív műveletet; a dokumentáció izolált, verziózott környezetben készül.

Kiinduló állapot. Válassz egy egyszerű rendszert, például egy rendelői időpontfoglaló alkalmazást vagy egy kitalált e-kereskedelmi kosarat. Készítsd el a C4 context szintű diagramját diagrams-as-code formában (például Mermaid), jelölve a rendszerhatárt, a külső rendszereket és a felhasználói szerepköröket. A laborhoz rendelkezésre áll egy üres döntési napló sablon és egy fenyegetéslista táblázat.

Elvárt eredmény. A dokumentumnak a következőket kell tartalmaznia: 1. **Kontextus:** a rendszer célja, a kezelt adatok típusa, a bizalmi határok. 2. **Minőségi forgatókönyvek:** legalább három biztonsági forgatókönyv quality attribute scenario formában (inger, környezet, válasz, mérőszám). 3. **Fenyegetéslista:** legalább nyolc fenyegetés a STRIDE kategóriák lefedésével; minden fenyegetéshez tartozzon érintett erőforrás, lehetséges hatás és ellenintézkedés. 4. **ADR-ek:** legalább két ADR, amely a biztonsági döntéseket rögzíti (kontextus, döntés, alternatívák, következmények, állapot). 5. **Ellenőrzési szempontok:** legalább öt pont, amelyekkel a biztonsági követelmények teljesülését ellenőrizni lehet.

Reprodukálhatóság és mérés. A dokumentumot verziózottan kell tárolni (például git), és minden bejegyzéshez dátum és forrás tartozzon. A fenyegetéslista valószínűségi vagy hatásbecslése szakértői becslés, nem mért eredmény; ezt jelölni kell. Ha egy ellenőrzési pontot ténylegesen mérsz (például egy TLS-konfigurációt ellenőrző parancs kimenete), akkor rögzítsd a környezetet, a konfigurációt, az időpontot és a mérési bizonytalanságot, és különítsd el a várt értéktől.

Ellenőrzési szempont a laborhoz. A labor akkor tekinthető teljesnek, ha a dokumentum tartalmazza a fenti öt elemet, a minőségi forgatókönyvek minden mezőt kitöltöttek, a fenyegetéslista lefedi a STRIDE mind a hat kategóriáját, és a mért eredmények elkülönülnek a várt vagy becsült értékektől.

Visszaállítási út. Mivel a labor csak dokumentációt és esetleg helyi, izolált teszteket igényel, a visszaállítás a verziózott dokumentum előző állapotának visszaállítását jelenti. Éles rendszeren nem szükséges és nem ajánlott a laborban leírt ellenőrzéseket futtatni.

8.3.9 Ellenőrzőlista

A fejezet lezárásaként az alábbi ellenőrzőlista segít a biztonsági és adatvédelmi követelmények architekturális lefedettségének gyors értékelésében:

- ☐ A rendszerhatárok és bizalmi határok explicit módon dokumentálva vannak (C4 context és container szinten).
- ☐ Minden külső kapcsolat protokollja, szinkronitása és hibatűrése jelölt; a bizalom átlépésénél hitelesítés és titkosítás van.
- ☐ A hozzáférés-szabályozás modellje (RBAC, ABAC vagy más) definiált, és minden komponensre érvényes a minimális jogosultság elve.
- ☐ A szállítás közbeni titkosítás (TLS 1.2+) és az adattárolási titkosítás (at rest) minden adatkezelő komponensre kiterjed.
- ☐ A kulcskezelés, a rotáció és a visszavonás folyamata dokumentált és tesztelt.
- ☐ Az audit naplózás lefedi a hitelesítési, engedélyezési és módosítási eseményeket; a napló integritásvédett és személyes adatot minimalizál.
- ☐ A személyes adatok mezőire adatmegőrzési idő, törlési és anonimizálási eljárás van meghatározva.

- ☐ A biztonsági eseményekhez riasztási szabályok és incidenskezelési runbookok tartoznak.
- ☐ A külső függőségek SBOM-mal rendelkeznek, és a sebezhetőség-szkennelés a CI/CD folyamat része.
- ☐ A megfelelőségi követelmények minőségi forgatókönyvekké alakítva, méréssel és visszacsatolással ellenőrizve vannak.

8.3.10 Önellenőrző kérdések

1. Hogyan fogalmaznál meg egy minőségi forgatókönyvet egy jogosulatlan adatmódosítási kísérletre? Nevezd meg az ingert, a környezetet, a választ és a mérőszámot.
2. Mi a különbség a pszeudonimizálás és az anonimizálás között, és miért fontos ez a különbség a GDPR szempontjából?
3. Sorolj fel három tipikus hibát a biztonsági követelmények kezelésében, és mindegyikhez adj egy helyes megoldást.

8.4 Karbantarthatóság, használhatóság és üzemeltethetőség

A karbantarthatóság, a használhatóság és az üzemeltethetőség három olyan minőségi attribútum, amelyet a gyakorlatban gyakran a funkcionális követelmények mögé sorolnak, pedig a hosszú távú fejlesztés és üzemeltetés költségeit, kockázatait és emberi erőforrásigényét alapvetően meghatározza. A funkcionális követelmények leírják, hogy a rendszer **mit** tud; ez a három attribútum viszont azt befolyásolja, hogy **mennyire könnyű** a rendszert megváltoztatni, használni és üzemeltetni. Egy olyan rendszer, amely funkcionálisan hibátlanul működik, de minden apró módosítás több hétig tart, vagy amelynek üzemeltetéséhez a készítő fejlesztőre van szükség éjszaka, hosszú távon nem fenntartható. Ezért ebben a fejezetben a három attribútumot együtt vizsgáljuk: a karbantarthatóság a fejlesztési oldal, a használhatóság a felhasználói és fejlesztői oldal, az üzemeltethetőség pedig az üzemeltetési oldal minőségét jelenti. A cél nem az, hogy minden rendszer minden mérőszámban maximális legyen, hanem az, hogy a csapat tudatosan, kontextusba helyezve hozzon döntést arról, hol érdemes javítani.

8.4.1 Karbantarthatóság: a változtathatóság minősége

A karbantarthatóság azt fejezi ki, hogy a rendszer milyen hatékonyan módosítható, javítható, bővíthető és vizsgálható. Az ISO/IEC 25010 minőségmodelljében a karbantarthatóság aljellemzői közé tartozik a modularitás, az újrafelhasználhatóság, az elemezhetőség, a módosíthatóság és a tesztelhetőség. A gyakorlatban ez nem elvont fogalom: egy módosítási kérelem átfutási ideje, a hiba lokalizálásához szükséges idő, a módosítás által érintett komponensek száma és a regressziós tesztkészlet futásideje mind a karbantarthatóság mérőszámai.

A karbantarthatóság alapja a **modularitás**. Egy modul akkor karbantartható, ha felelőssége egyértelmű, belső szerkezete kohéziós, és a többi modullal való kapcsolata minimális, explicit interfészeken keresztül valósul meg. A magas kohézió azt jelenti, hogy az egy modulba tartozó elemek együtt változnak, közös célt szolgálnak. Az alacsony csatolás azt jelenti, hogy egy modul megváltoztatása nem kényszeríti ki más modulok megváltoztatását. E két elv együttesen csökkenti a módosítás hullámzó hatását. Ha például egy rendeléskezelő komponens közvetlenül ír a számlázó komponens adatbázistáblájába, akkor a számlázási séma bármilyen változtatása a rendeléskezelőben is módosítást igényel. Ha viszont a rendeléskezelő csak egy `InvoiceClient` interfészen keresztül küld számlázási kérést, akkor a számlázási implementáció változtatása rejtve maradhat.

A **tesztelhetőség** a karbantarthatóság gyakorlati kulcsa. Egy rendszer akkor tesztelhető, ha a viselkedése determinisztikusan ellenőrizhető, a függőségei kicserélhetők, és a fontos állapotok megfigyelhetők. A tesztelhetőség hiánya tipikusan három formában jelentkezik: rejtett függőségek, nem determinisztikus viselkedés és a megfigyelhetőség hiánya. A rejtett függőségek – például komponens konstruktorában közvetlenül példányosított adatbázis-kapcsolat – megakadályozzák az izolált tesztelést. A nem determinisztikus viselkedés – például rendszerórára, véletlenszám-generátorra vagy hálózati időzítésre épülő logika – miatt ugyanaz a teszt egyszer sikeres, egyszer sikertelen lehet, ami aláássa a tesztkészletbe vetett bizalmat. A megfigyelhetőség hiánya azt jelenti, hogy a teszt nem tudja megállapítani, mi történt a komponens belsejében; ilyenkor a teszt csak a külső mellékhatásokat tudja ellenőrizni, ami gyakran törékeny.

A karbantarthatóság mérésére több mutató használható, de egyik sem önmagában elegendő. A ciklikus komplexitás például azt jelzi, hogy egy függvény hány lineárisan független útvonalat tartalmaz; a magas érték (például 15 fölött) gyakran jelzi, hogy a függvény túl sok döntést hoz, ezért nehéz megérteni és tesztelni. A csatolási mutatók – mint például az afferens és efferens csatolások száma – azt mutatják, hogy egy osztály vagy csomag hány másik elemre hivatkozik, illetve hányan hivatkoznak rá. A tesztlefedettség százalékos értéke hasznos, de félrevezető lehet: a magas lefedettség nem garantálja, hogy az üzletileg fontos útvonalakat is teszteli a készlet. Ezért a lefedettséget mindig minőségi szempontokkal együtt kell értékelni.

A mérési ablak és a mértékegység megadása itt is kötelező. Például a módosítási átfutási időt napokban mérjük, és egy negyedéves ablakban átlagoljuk. Az átlagos hibajavítási idő (MTTR) percben vagy órában adható meg, de a mérési ablakot – például az utolsó 30 nap termelési incidensei – rögzíteni kell. Az adósságmutatók, mint például a kódban található ismert code smell-ök száma, szintén csak akkor értelmesek, ha tudjuk, milyen kódbázisra és milyen időpontra vonatkoznak. A karbantarthatóság tehát nem egyetlen szám, hanem egy olyan mutatórendszer, amelyet a csapat folyamatosan gyűjt és értékel.

A karbantarthatóság javításának legismertebb eszközei a függőséginjektálás, az interfészek bevezetése, a rétegek közötti függőségi szabályok, a feature flag-ek és a folyamatos refaktorálás. A függőséginjektálás nem öncélú minta, hanem a tesztelhetőség és a csatolás csökkentésének eszköze. Ha egy komponens a konstruktorában kapja meg a szükséges együttműködőket, akkor tesztkörnyezetben tesztkettőst adhatunk át, termelésben pedig valódi implementációt. Az interfészek bevezetése hasonlóan lehetővé teszi, hogy a fogyasztó ne a konkrét implementációtól, hanem egy szerződéstől függjön. Fontos azonban, hogy az interfészek száma nem cél, hanem a szükséges absztrakciók eszköze. A túl sok interfész ugyanúgy karbantarthatósági probléma, mint a túl kevés: az indokolatlan absztrakciók növelik a kód navigációs terhét.

8.4.2 Használhatóság: a végfelhasználó és a fejlesztő szempontja

A használhatóság hagyományosan a végfelhasználó szempontjából értelmezett minőségi attribútum: a rendszer mennyire könnyen tanulható, hatékonyan használható, mennyire akadályozza meg a hibákat, és mennyire elégíti ki a felhasználót. Az ISO/IEC 25010 modelljében a használhatóság aljellemzői közé tartozik a felismerhetőség, a tanulhatóság, az operálhatóság, a hibavédelem és az elérhetőség. A gyakorlatban a használhatóságot legtöbbször feladatelvégzési idővel, hibák számával, betanulási idővel, segítségkérés nélküli teljesítés arányával és elégedettségi kérdőívekkel mérjük. A mérési ablak itt is fontos: egy új funkció használhatóságát például a bevezetést követő első két hétben érdemes mérni, amikor a felhasználók még nem rutinosak.

A szoftverarchitektúra szempontjából a használhatóság nem csak a felhasználói felület rétegének kérdése. A használhatóságot befolyásolja az API dizájn, a hibaüzenetek pontossága, a rendszer válaszidejének stabilitása, a konzisztens fogalomhasználat és az, hogy a rendszer milyen mértékben ad visszajelzést a felhasználó műveleteiről. Ha egy ügyfélkezelő rendszerben a mentés

gomb megnyomása után a felület nem jelez feldolgozási állapotot, a felhasználó megismétli a műveletet, ami duplikált rekordokhoz vezethet. A hibavédelem tehát közvetlenül kapcsolódik az adatintegritáshoz. Az idempotens műveletek tervezése, a műveletek megerősítése és a formális validáció mind használhatósági és egyben adatminőségi követelmény.

A fejezet fókuszában a **fejlesztői élmény** (developer experience, DX) is áll, amely a használhatóság belső, fejlesztőkre vonatkozó változata. A fejlesztői élmény azt jelenti, hogy a fejlesztő milyen gyorsan és milyen alacsony kognitív terheléssel tudja a rendszert megérteni, módosítani, tesztelni és helyileg futtatni. A jó fejlesztői élmény nem luxus, hanem a karbantarthatóság emberi oldala. Ha egy fejlesztőnek 30 percet kell várnia a build eredményére, vagy ha a helyi környezet felállítása egy napot vesz igénybe, akkor a csapat nem fogja tudni fenntartani a gyors iterációt. A fejlesztői élmény mérőszámai közé tartozik a build idő, a tesztkészlet futásideje, a helyi környezet felállításának ideje, a dokumentáció frissessége és a hibakereséshez szükséges idő.

A fejlesztői élmény javításának gyakorlati eszközei közé tartozik a jól definiált build folyamat, a gyors visszajelzés, a konzisztens kódformázás, a tiszta hibaüzenetek és az önkiszolgáló fejlesztői környezet. A build folyamat akkor jó, ha egy fejlesztő a kódbázis letöltése után egyetlen paranccsal el tudja indítani a szükséges függőségeket és a rendszer helyi példányát. A gyors visszajelzés azt jelenti, hogy a módosítás után a lehető leghamarabb – ideális esetben másodperceken belül – futnak a releváns tesztek. A konzisztens kódformázás és a linterek használata csökkenti a kognitív terhelést, mert a fejlesztőnek nem kell stílusbeli döntésekkel foglalkoznia. A hibaüzenetek akkor hasznosak, ha pontosan megmondják, mi történt, hol, és mi a javasolt következő lépés. A `null pointer exception` típusú üzenetek helyett a `A rendelés azonosítója hiányzik; a kérésben a 'orderId' mező kötelező.` típusú üzenet csökkenti a hibakeresési időt.

A használhatóság tervezése tehát kétirányú: a végfelhasználói felületeknél a felhasználói viselkedésre, a belső interfészeknél és platformoknál pedig a fejlesztői viselkedésre kell fókuszálni. Mindkét esetben érvényes, hogy a használhatóságot nem lehet utólag hozzáadni a rendszerhez; a fogalomhasználat, az API-szerződések, a hibaüzenetek és a visszajelzési mechanizmusok mind architekturális döntések.

8.4.3 Üzemeltethetőség: a futó rendszer minősége

Az üzemeltethetőség (operability) azt fejezi ki, hogy a rendszer mennyire könnyen telepíthető, konfigurálható, monitorozható, frissíthető, és mennyire könnyen állítható helyre hiba esetén. Az üzemeltethetőség a termelési életciklus minősége: egy rendszer lehet kiválóan megtervezett és kiválóan használható, de ha az üzemeltető nem tudja megállapítani, hogy mi történt, vagy ha egy hiba után nincs biztonságos visszaállítási út, akkor a termelési kockázat magas. Az üzemeltethetőség hiánya gyakran azzal jár, hogy az üzemeltető csapat manuális, hibalehetőségekkel terhelt folyamatokra kényszerül, és a hibaelhárítás ideje megnő.

Az üzemeltethetőség alapja a **megfigyelhetőség** (observability). A megfigyelhetőség azt a képességet jelenti, hogy a rendszer külső jelei alapján meg tudjuk állapítani a belső állapotát, és meg tudjuk magyarázni a viselkedését. A megfigyelhetőség három alappillére a naplók (logs), a metrikák és az elosztott nyomkövetés (distributed tracing). A naplók az egyedi eseményekről adnak részletes képet, a metrikák az aggregált viselkedést mutatják, a nyomkövetés pedig az egy kérés útját követi végig a komponenseken keresztül. A három pillér együttesen teszi lehetővé, hogy az üzemeltető egy hibát a tünet szintjétől az ok szintjéig tudjon követni.

A naplózás üzemeltethetőségi szempontból akkor jó, ha a naplóbejegyzések strukturáltak, gépi feldolgozásra alkalmasak, és konzisztens mezőneveket használnak. A hagyományos szabadszavas naplósorok (`Hiba történt a 42-es rekordnál`) helyett a strukturált napló JSON-ben vagy

más log formátumban rögzíti az esemény típusát, a tranzakció azonosítóját, a komponens nevét, az időbélyeget és a releváns paramétereket. Példa:

```
{
  "timestamp": "2025-01-15T08:32:11Z",
  "level": "WARN",
  "service": "order-service",
  "traceId": "abc123",
  "message": "payment timeout",
  "orderId": "ord-42",
  "paymentProvider": "example-pay"
}
```

Ez a példa illusztráció; a mezőnevek és az azonosítók a szervezeti konvenciótól függenek. A strukturált naplózás lehetővé teszi, hogy az üzemeltető rendszer naplóelemző eszközzel szűrjön, aggregáljon és riasztást állítson be anélkül, hogy kézzel elemezné a szövegfájlokat.

A metrikák az üzemeltethetőség második pillére. A metrikákat mértékegységgel és mérési ablakkal együtt kell gyűjteni. A leggyakoribb üzemeltetési metrikák közé tartozik a kérések száma (requests per second), a hibak aránya (error rate), a válaszidő percentilisek (p50, p95, p99), a CPU- és memóriahasználat, a lemez- és hálózati I/O, valamint az üzleti metrikák, például a sikeres megrendelések száma. A metrikák akkor használhatók riasztásra, ha a normalitásból való kilépést egyértelműen jelzik. A riasztások beállításánál kerülni kell a zajos riasztásokat, mert a hamis pozitívok aláássák az üzemeltető csapat bizalmát, és idővel a fontos riasztások is elvesznek.

A nyomkövetés az elosztott rendszerekben vált kulcsfontosságúvá. Egy mikroszolgáltatás-architektúrában egy felhasználói kérés több tucat komponensen haladhat keresztül; a hagyományos naplók alapján nehéz rekonstruálni, hogy melyik komponens volt a szűk keresztmetszet vagy a hiba forrása. A nyomkövetés során minden bejövő kérés kap egy egyedi azonosítót, és minden komponens a saját munkáját ehhez az azonosítóhoz kapcsolja. A gyakorlatban a W3C Trace Context szabvány és a OpenTelemetry eszközkészlet terjedt el, de a részletes megvalósítás a későbbi fejezetek témája. Itt a lényeg az, hogy a nyomkövetést nem utólag érdemes bevezetni, hanem a komponensek tervezésénél kell biztosítani, hogy a tranzakciós kontextus továbbítódjon.

Az üzemeltethetőség másik fontos eleme a **konfigurálhatóság**. A konfigurációt el kell választani a kódból, hogy ugyanaz az artifact különböző környezetekben – fejlesztés, teszt, staging, termelés – eltérő beállításokkal futhat. Ha a konfiguráció a kódban van, akkor minden környezeti változtatás új buildet és deploy-t igényel, ami lassítja a hibaelhárítást. A konfiguráció kezelésére a környezeti változók, a konfigurációs fájlok és a konfigurációs szerverek egyaránt használhatók; a választás attól függ, hogy milyen gyakran változik a beállítás, milyen titoktartalom van benne, és milyen ellenőrzési nyomvonalra van szükség. A titkokat – jelszavak, kulcsok, tanúsítványok – soha nem szabad a konfigurációs fájlokban tárolni; ezeket titokkezelőbe kell tenni, és a futásidejű környezetnek ideiglenesen kell elérhetővé tenni.

Az üzemeltethetőséghez hozzátartozik a **frissíthetőség és a visszaállíthatóság**. A rendszer akkor üzemeltethető jól, ha egy új verzió kiadása nem igényel hosszú leállást, és ha a kiadás után gyorsan vissza lehet térni az előző verzióhoz, ha a hibaarány megugrik. A fokozatos telepítési stratégiák – mint a rolling deploy, a blue-green vagy a canary – a későbbi fejezetekben részletesen tárgyalásra kerülnek. Itt elég annyit rögzíteni, hogy az üzemeltethetőség követelménye, hogy a rendszer rendelkezzen egészségállapot-jelzéssel, amelyet a szolgáltatás-menedzser vagy a terheléselosztó használ. A health check például egy `/healthz` végpont, amely 200-as státuszkódot ad vissza, ha a szolgáltatás az adatbázist eléri, és a kritikus függőségek működnek. A liveness és readiness szétválasztása lehetővé teszi, hogy a platform különbséget tegyen a szolgáltatás újraindítása és a forgalom elérésének felfüggesztése között.

A **gracious shutdown** (kegye teljes leállítás) szintén üzemeltethetőségi alap. Ha a szolgáltatás kap egy leállítási jelet, akkor először be kell fejeznie a folyamatban lévő kéréseket, le kell zárnia a kapcsolatokat, és csak ezután szabad kilépnie. Ha a leállítás azonnal megszakítja a kéréseket, akkor a felhasználók hibákat tapasztalnak egy tervezett újraindítás során is. A graceful shutdown a hibatűrés és az üzemeltethetőség határán áll: a rendszer viselkedését tervezett karbantartás alatt is megbízhatóvá teszi.

8.4.4 Quality attribute scenario példák

A minőségi követelmények pontosításához ebben a fejezetben is quality attribute scenario formát használunk: inger, környezet, válasz, mérőszám.

Karbantarthatósági forgatókönyv

- **Inger:** a fejlesztő új kedvezménytípust ad a kuponkezelő komponenshez.
- **Környezet:** normál fejlesztési időszak, a rendszer valamennyi környezetben fut.
- **Válasz:** a módosítás legfeljebb 3 forrásfájlt érint, a hozzá tartozó tesztek 10 percen belül lefutnak, és a módosítás nem igényel adatbázis-séma migrációt.
- **Mérőszám:** érintett fájlok száma, tesztkészlet futásideje, séma migrációs lépések száma; a mérést a módosítási kérelem elfogadásának időpontjában rögzítjük.

Használhatósági forgatókönyv

- **Inger:** új felhasználó regisztrálni szeretne.
- **Környezet:** bevezetés nélkül, első használat, éles webes felület.
- **Válasz:** a felhasználó 2 percen belül segítség nélkül, hiba nélkül sikeresen regisztrál.
- **Mérőszám:** feladatelvégzési idő, segítségkérés nélküli teljesítés aránya, hibák száma; a mérési ablak a funkció bevezetését követő első 14 nap.

Üzemeltethetőségi forgatókönyv

- **Inger:** a fizetési szolgáltatás hibatartaléka hirtelen 20% fölé emelkedik.
- **Környezet:** termelési környezet, hétköznap 14:00 és 16:00 közötti csúcsidő.
- **Válasz:** az üzemeltető a dashboardon és a riasztáson keresztül 5 percen belül észleli a hibát, a bevezetett canary kiadást visszaállítja 3 percen belül, és a hibát a nyomkövetési azonosító alapján 10 percen belül lokalizálja.
- **Mérőszám:** észlelési idő (TTD), visszaállítási idő, lokalizálási idő; a mérés az incidensalapú utólagos elemzésben rögzítve.

Ezek a forgatókönyvek nem csupán dokumentációs eszközök, hanem a tervezési döntések bemenetei. Ha a karbantarthatósági forgatókönyv azt követeli, hogy egy módosítás legfeljebb 3 fájlt érintsen, akkor a csapatnak olyan komponenshatárokat kell kialakítania, amelyek ezt lehetővé teszik. Ha az üzemeltethetőségi forgatókönyv 5 perces észlelési időt ír elő, akkor a metrikákat és a riasztásokat már a fejlesztés során be kell építeni.

8.4.5 Tipikus hibák és felismerésük

A karbantarthatósági, használhatósági és üzemeltethetőségi hibák gyakran egymást erősítik. A leggyakoribb minták a következők:

1. **A "működik, de nem változtatható" rendszer.** A fejlesztők minden módosítástól tartanak, mert a kódban nincsenek világos határok, a tesztek lassúak, és a függőségek rejtettek. A megoldás a fokozatos modulárisítás, nem pedig a teljes újraírás.

2. **A “nincs log vagy nincs struktúra” üzemeltetés.** A termelési hibánál az üzemeltető nem tudja, mi történt, mert a naplók hiányosak vagy csak a fejlesztői gépen elérhetők. A megoldás a strukturált naplózás és a központi naplógyűjtés bevezetése.

3. **A “csak az írója érti” rendszer.** A dokumentáció hiányzik vagy elavult, a hibaüzenetek rejtélyesek, és a kódban nincs következetes fogalomhasználat. A megoldás a dokumentációt a kódbázishoz kötni, a hibaüzeneteket fogyasztói szempontból megtervezni, és a csapaton belüli kódreview-t erősíteni.

4. **A “minden konfiguráció a kódban” csapda.** A környezeti beállításokat a fejlesztők beégetik a forráskódba, ezért a termelési kibocsátás új buildet igényel. A megoldás a konfiguráció külső forrásba helyezése, a titkok kezelése titokkezelőben, és a deploy folyamatban a környezeti konfiguráció injektálása.

5. **A “futtatás után nincs egészségállapot” rendszer.** A platform nem tudja, hogy a szolgáltatás életben van-e, ezért a hibás példányok továbbra is forgalmat kapnak. A megoldás a liveness és readiness végpontok bevezetése, valamint a szolgáltatás-menedzserhez kapcsolódó health check konfiguráció.

6. **A “lassú visszajelzés” fejlesztői élmény.** A build 30 percet vesz igénybe, a tesztkészlet nem párhuzamosítható, ezért a fejlesztők kerülik a gyakori ellenőrzést. A megoldás a build folyamat szakaszokra bontása, a gyors egységtesztek és a lassabb integrációs tesztek szétválasztása, valamint a helyi fejlesztői környezet egyszerűsítése.

Ezeknek a hibáknak a felismeréséhez nem kell különleges eszköz; a fenti forgatókönyvek, a csapat visszajelzései és a termelési incidensek utólagos elemzése elegendő jeleket ad. A felismerés után azonban fontos, hogy a javítás ne egyszeri nagy projektté váljon, hanem folyamatos, kis lépésekben megvalósuló tevékenység legyen. A karbantarthatósági, használhatósági és üzemeltethetőségi javítások jól illeszkednek az evolúciós architektúra szemléletéhez: a rendszer minden módosítása alkalom arra, hogy egy kicsit jobb állapotba kerüljön.

8.4.6 Labor: karbantarthatósági és üzemeltethetőségi ellenőrzőlista

A labor célja, hogy a fenti fogalmakat egy kis, ellenőrizhető példán alkalmazzuk. Az átadandó eredmény egy reprodukálható dokumentum vagy jegyzetfüzet, amely rögzíti a kontextust, a döntési naplót, az ellenőrzési szempontokat, valamint különválasztja a mért és a várt eredményt. A labor nem kíván termelési környezetet, és nem tartalmaz destruktív lépést; bármely izolált fejlesztői környezetben elvégezhető.

Kiinduló állapot. Válassz egy kisméretű, de valós komponenst a saját kódbázisodból, vagy használd a következő fiktív példát: egy belső katalógus API, amely a `GET /items` végponton keresztül listáz termékeket, a `POST /items` végponton keresztül új terméket vesz fel, és egy PostgreSQL adatbázist használ. A példa forráskódja nem szükséges; a labor a dokumentációs és ellenőrzési folyamatra fókuszál.

Lépések

1. **Kontextus rögzítése.** Készíts egy rövid leírást, amely tartalmazza a komponens nevét, felelősségét, a kapcsolódó rendszerhatárt, a technológiai környezetet (például konténerizált service, PostgreSQL), és a vizsgálat időpontját.

2. **Ellenőrzőlista kitöltése.** Használd az alábbi sablont. Minden kérdéshez rendelj állapotot (teljesül / részben / nem teljesül), és ha van mérhető adat, rögzítsd a mért értéket, a mértékegységet és a mérési ablakot. Ha nincs mérés, írd be, hogy „nincs mérés”, és jelöld, hogy a jövőben milyen mérést érdemes bevezetni.

Karbantarthatósági és üzemeltethetőségi ellenőrzőlista sablon

Terület	Kérdés	Állapot	Mért érték	Várt érték	Megjegyzés
Modularitás	A komponens felelőssége egy mondatban megfogalmazható?				
Modularitás	A komponens más komponensekkel csak explicit interfészen keresztül kommunikál?				
Tesztelhetőség	A komponens egységtesztjei elérhetők és izoláltan futnak?				
Tesztelhetőség	A tesztkészlet futásideje 10 perc alatt van?				
Megfigyelhetőség	A komponens strukturált naplót ír?				
Megfigyelhetőség	A fontos üzleti műveletek metrikája elérhető?				
Konfigurálhatóság	A környezeti beállítások kód nélkül megváltoztathatók?				

Terület	Kérdés	Állapot	Mért érték	Várt érték	Megjegyzés
Egészségállapot	A komponens rendelkezik health check végponttal?				
Leállítás	A komponens graceful shutdownt hajt végre?				
Dokumentáció	A komponens interfészét és felelősségét dokumentálták?				
Fejlesztői élmény	A helyi környezet felállítása egyetlen parancs?				
Hibaüzenetek	A hibaüzenetek tartalmazzák a hiba okát és a javasolt lépést?				

3. **Döntési napló.** A kitöltött ellenőrzőlista alapján válassz ki egy hiányosságot, és készíts hozzá rövid ADR-t. A sablon: kontextus, döntés, alternatívák, következmények, állapot. A döntés lehet például az, hogy a strukturált naplózást bevezetik; az alternatívák között szerepelhet a szabadszavas naplók megtartása vagy egy harmadik féltől származó naplózó könyvtár használata. A következmények között rögzítsd, hogy a döntés milyen hatással van a meglévő rendszerre, milyen erőforrást igényel, és milyen kockázatot hordoz.

4. **Mért és várt eredmény szétválasztása.** A dokumentumban egyértelműen jelöld, mely értékek valódi mérésből származnak, és melyek a kitűzött célok. Például: „Mért érték: a tesztkészlet futásideje 14 perc (mérés dátuma: 2025. 01. 10., környezet: fejlesztői laptop). Várt érték: 10 perc alatt.” Ha nincs mérés, a várt értéket akkor is rögzítheted, de jelölnöd kell, hogy a mért érték hiányzik.

Elvárt eredmény. A labor végén a dokumentum a következőket tartalmazza: a kontextus, a kitöltött ellenőrzőlista, legalább egy ADR, és a mért/várt eredmények szétválasztása.

Ellenőrzési szempontok. A dokumentum akkor elfogadható, ha (a) a kontextus egyértelműen meghatározza a vizsgált komponenst, (b) minden ellenőrzőlista-kérdésre van válasz, (c) a hiányosságokhoz tartozik javaslat vagy ADR, (d) a mért és várt értékek elkülönülnek, és (e) a mérési

adatokhoz mértékegység és mérési ablak tartozik, vagy a mérés hiánya explicit módon jelölve van.

Visszaállítási út. A labor nem módosít termelési rendszert, ezért visszaállítási lépésre nincs szükség. Ha a résztvevő a saját kódbázisában valamilyen konfigurációt módosít, azt a verziókezelőben elkülönített ágban tegye, és a labor után a módosítást nem kell összefésülni.

8.4.7 Ellenőrzőlista a fejezethez

- A rendszer komponensei egyértelmű felelősséggel rendelkeznek, és a felelősségük egy mondatban megfogalmazható.
- A komponensek explicit interfészeken keresztül kommunikálnak; a rejtett függőségek és közvetlen adatbázis-hozzáférések száma minimális.
- A fontos üzleti logika determinisztikusan tesztelhető; a nem determinisztikus források (óra, véletlen, hálózati időzítés) elkülönítettek.
- A tesztkészlet gyors visszajelzést ad; a lassú tesztek külön szakaszban futnak.
- A rendszer strukturált naplót ír, és a tranzakciós azonosító végigvonul a kéréseken.
- A fontos üzleti és technikai metrikák gyűjtöttek, mértékegységgel és mérési ablakkal.
- A konfiguráció elválik a kódból; a titkok titokkezelőben vannak.
- A rendszer health check végponttal rendelkezik, és a platform ezt használja.
- A leállítás graceful shutdown-t hajt végre; a folyamatban lévő kérések befejeződnek.
- A hibaüzenetek tartalmazzák a hiba okát és a javasolt következő lépést.
- A helyi fejlesztői környezet felállítása és a build folyamat egyszerű és dokumentált.
- A használhatósági forgatókönyvekhez tartozik mérési terv, és a mérési ablak rögzített.

8.4.8 Önellenőrző kérdés

Egy rendszerben a termelési hiba elhárítása átlagosan 3 órát vesz igénybe, mert a naplók szabadszavasak, a metrikák hiányosak, és a hibát csak a fejlesztő tudja lokalizálni a saját gépén. A csapat azt tervezi, hogy bevezeti a strukturált naplózást és a metrikákat. Melyik quality attribute scenario-t kell megfogalmaznia ahhoz, hogy a javulást mérhetővé tegye? Írd le a forgatókönyv négy elemét (inger, környezet, válasz, mérőszám), és jelöld a mérési ablakot.

8.4.9 Tervezési gyakorlat

Válassz ki egy komponenst a saját kódbázisodból, és készíts hozzá három quality attribute scenario-t: egyet a karbantarthatóságra, egyet a használhatóságra (végfelhasználó vagy fejlesztő szempontjából), és egyet az üzemeltethetőségre. Mindegyik forgatókönyvben adj meg ingert, környezetet, választ és mérőszámot. A mérőszámhoz tartozzon mértékegység és mérési ablak. Ha valamelyik adatot nem tudod megmérni, írd le, milyen mérést kellene bevezetni, és miért. A gyakorlat eredményét rögzítsd ADR-formában vagy a labor dokumentumában.

A fejezetben bemutatott fogalmak a következő egység (Quality attribute scenarios, SLO, SLI, error budget) alapját képezik. A karbantarthatósági, használhatósági és üzemeltethetőségi forgatókönyvek ott kapcsolódnak majd a szolgáltatási szintekhez és a folyamatos méréshez: a most megfogalmazott minőségi elvárásokból lesznek konkrét SLO-k és fitness function-ök. Ezért fontos, hogy a forgatókönyvekben a megfogalmazás már most mérhető, számszerűsíthető és időablakhoz kötött legyen. A homályos követelmények – „a rendszer legyen gyors”, „legyen könnyű karbantartani” – nem alkalmasak a későbbi automatizált ellenőrzésre. A jól megfogalmazott minőségi követelmény viszont hidat képez a tervezési szándék és a termelési valóság között.

9 Minőségi forgatókönyvek és architektúraértékelés

9.1 Quality attribute scenarios

A minőségi követelmények gyakran azért maradnak teljesítetlenek, mert nem mérhető formában fogalmazták meg őket. A „a rendszer legyen gyors” vagy „legyen megbízható” kijelentés nem ad támpontot sem a tervezőnek, sem a tesztelőnek, sem az üzemeltetőnek. A minőségi attribútum forgatókönyv (quality attribute scenario) ezt a hiányt pótolja: a minőségi elvárást egyetlen, jól körülhatárolt eseményre bontja le, és megadja, hogy a rendszert milyen körülmények között éri az inger, milyen választ kell adnia, és azt milyen mérőszámmal lehet ellenőrizni. Ez a fejezet ezt az eszközt mutatja be, a következő alfejezet pedig az itt megfogalmazott forgatókönyvekből építi fel az SLO-k és error budgetek rendszerét.

9.1.1 Miért forgatókönyv?

A minőségi attribútumok – például a teljesítmény, a rendelkezésre állás, a biztonság, a karbantarthatóság – nem közvetlenül mérhetők. A rendszer egészére vonatkozó állítások túl tágak ahhoz, hogy döntést vagy tesztet lehessen rájuk alapozni. Egy forgalmas webáruházban a „legyen gyors” jelentheti a terméklista betöltését mobilon, a fizetési tranzakció végrehajtását vagy a keresési javaslatok megjelenítését – mindegyikhez más válaszidő, más terhelés és más környezet tartozik. A forgatókönyv éppen ezt a kontextust rögzíti.

A forgatókönyv-alapú megfogalmazás előnyei:

1. **Egyértelműség**: a stakeholder számára is érthető, mert konkrét helyzetet ír le, nem absztrakt követelményt.
2. **Tesztelhetőség**: a mérőszám mérési eljáráshoz köthető, így a teljesülést objektíven ellenőrizni lehet.
3. **Nyomon követhetőség**: az architektúra-döntések és a minőségi elvárások közötti kapcsolat dokumentálható.
4. **Priorizálhatóság**: a forgatókönyvek egymással összehasonlíthatók, és súlyozhatók a kockázat vagy az üzleti érték szerint.
5. **Folyamatos értékelés**: a forgatókönyv mérőszáma beépíthető a CI/CD pipeline-ba fitness functionként, így a regressziók korán láthatóvá válnak.

A forgatókönyv azonban nem helyettesíti a mérést. A forgatókönyv a mérés specifikációja, nem a mérés eredménye. A laborban ezért a mért és a várt eredményt mindig külön kell választani.

9.1.2 A forgatókönyv szerkezete

A minőségi attribútum forgatókönyv hat részből áll, amelyek közül a négy legfontosabbat a könyv jelölési konvenciója is kiemeli: **inger, környezet, válasz és mérőszám**. A teljes, kanonikus szerkezet a következő:

1. **Forrás** (source): ki vagy mi váltja ki az ingert – felhasználó, másik rendszer, hardverhiba, támadó, időzítő.
2. **Inger** (stimulus): a konkrét esemény vagy kérés, amely a rendszert éri.
3. **Környezet** (environment): a rendszer állapota a inger pillanatában – normál üzem, csúcsterhelés, részleges hiba, bevetés alatt, támadás alatt.
4. **Műtárgy** (artifact): a rendszer azon része, amelyre az inger hat – konkrét komponens, szolgáltatás, API, adatbázis.
5. **Válasz** (response): a rendszer megfigyelhető reakciója az ingerre.
6. **Mérőszám** (response measure): a válasz minőségét számszerűsítő mutató, mértékegységgel és mérési ablakkal.

A négy kiemelt elem – inger, környezet, válasz, mérőszám – minden forgatókönyvben kötelező. A forrás és a műtárgy akkor fontos, ha a forgatókönyvet konkrét komponenshez vagy szerepkörhöz kell kötni.

9.1.3 Példa egy teljes forgatókönyvre

Tegyük fel, hogy egy rendelési szolgáltatást tervezünk. A teljesítményre vonatkozó minőségi követelmény forgatókönyv formában:

- **Forrás**: regisztrált vásárló mobilalkalmazáson keresztül.
- **Inger**: „Rendelés leadása” kérés elküldése a checkout API-nak.
- **Környezet**: normál csúcsterhelés, a rendszer percenként 200 rendelési kérést fogad, az adatbázis replikáció késése legfeljebb 2 másodperc.
- **Műtárgy**: Rendelés-szolgáltatás (order-service) és a hozzá tartozó checkout API.
- **Válasz**: a rendszer rögzíti a rendelést, visszaigazolást küld, és a rendelés megjelenik a fulfillment sorban.
- **Mérőszám**: a válaszidő 95. percentilise legfeljebb 800 ms 5 perces csúszóablakban, a sikeres válaszok aránya legalább 99,9% ugyanabban az ablakban; a mérési időszak 30 napos termelési forgalom.

Ez a forgatókönyv már tesztelhető. Nem csak annyit mond, hogy „gyors”, hanem megadja a terhelést, a környezetet, a választ és a mérőszámot. Az architektúra-értékelés során azt vizsgáljuk, hogy a tervezett megoldás ezt a forgatókönyvet teljesíteni tudja-e.

9.1.4 A négy elem részletesen

9.1.5 Inger

Az inger a rendszer szempontjából külső esemény vagy kérés. Lehet funkcionális jellegű (például egy HTTP kérés), időzítő által kiváltott (például éjféli batch futás), hibajellegű (például egy node kiesése), vagy támadó szándékú (például brute-force próbálkozás). Az inger pontos megfogalmazása azért fontos, mert a rendszer válasza más-más ingerre eltérő lehet. A „kérés érkezik” túl általános; a „rendelés leadása kérés 2 tétellel, érvényes session token-nel” már konkrét.

Az inger megfogalmazásánál kerüljük a megoldásra utaló megfogalmazást. Nem az a kérdés, hogy a „load balancer elosztja a kérést”, hanem hogy mi éri a rendszert. A megoldás az architektúra tervezése során születik, a forgatókönyv a követelményt rögzíti.

9.1.6 Környezet

A környezet az a rendszerállapot, amelyben az inger bekövetkezik. Ugyanaz a kérés egészen más minőséget igényel normál üzemben, mint csúcsterhelés alatt, részleges hiba esetén vagy egy adatbázis-átállás közben. A környezet megadása nélkül a forgatókönyv nem reprodukálható, mert a mérőszám erősen függ a körülményektől.

A környezetet célszerű mérhető paraméterekkel leírni:

- terhelés: hány kérés/másodperc, hány egyidejű felhasználó, mekkora adatmennyiség;
- rendszerállapot: minden komponens elérhető, egy példány kiesett, cache hideg, adatbázis replikáció késik;
- időzítés: csúcsidő, éjszakai batch futás, deploy közben;
- biztonsági állapot: normál, aktív támadás, audit alatt.

Például: „normál csúcsterhelés, percenként 200 rendelési kérés, az adatbázis replikáció késése legfeljebb 2 másodperc". Ez a környezet reprodukálható terheléstesztben, és a mérési eredmény is értelmezhető.

9.1.7 Válasz

A válasz a rendszer megfigyelhető reakciója. Fontos, hogy a válasz ne a megoldás belső mechanizmusát írja le, hanem a megfigyelhető viselkedést. A válasz lehet:

- funkcionális eredmény: a rendelés rögzítésre kerül, a felhasználó visszaigazolást kap;
- minőségi jellemző: a válaszidő egy küszöb alatt marad, a hiba aránya egy határ alatt van;
- üzemeltetési reakció: automatikus újraindítás, riasztás küldése, forgalom átirányítása.

A válasz megfogalmazásánál kerüljük a belső működésre utaló kifejezéseket. A „cache-ből szolgál" nem válasz, hanem megoldás. A válasz az, hogy „a válaszidő 95. percentilise 800 ms alatt van". A megoldás lehet cache, lehet más optimalizáció – a forgatókönyv nem írja elő.

9.1.8 Mérőszám

A mérőszám a válasz minőségét számszerűsíti. A könyv jelölési konvenciója szerint a mérőszámot mértékegységgel és mérési ablakkal kell megadni. Ez azért fontos, mert a puszta szám félrevezető: a „99,9%" nem jelenti ugyanazt 5 perc alatt, mint 30 nap alatt.

A mérőszám megadásának elemei:

- **Mértékegység**: másodperc, milliszekundum, százalék, darab, Mbit/s, hiba/szám.
- **Statisztikai jellemző**: átlag, medián, percentilis (P95, P99), minimum, maximum, arány.
- **Mérési ablak**: csúszóablak hossza (5 perc, 1 óra, 30 nap), a minta mérete, a mérés gyakorisága.
- **Összehasonlítási alap**: küszöbérték (legfeljebb, legalább), trend (javulás üteme).

Például: „a válaszidő 95. percentilise legfeljebb 800 ms 5 perces csúszóablakban, a sikeres válaszok aránya legalább 99,9% ugyanabban az ablakban". Itt a mértékegység (ms, százalék), a statisztikai jellemző (P95, arány) és az ablak (5 perc) egyaránt szerepel.

A mérőszámnak a mérési eljáráshoz is kapcsolódnia kell: honnan származik az adat (alkalmazáslog, monitorozó rendszer, terheléstesztelő), milyen konfigurációval, milyen környezetben. A laborban ezért a mérési jegyzőkönyvnek tartalmaznia kell a környezetet, a konfigurációt, az időpontot és a bizonytalanságot.

9.1.9 Forgatókönyv-sablon

A forgatókönyvek egységes formában történő felírása csökkenti a félreértést. A könyv a következő sablont ajánlja:

Elem	Tartalom
Azonosító	QA-001
Minőségi attribútum	Teljesítmény
Forrás	Regisztrált vásárló mobilalkalmazáson keresztül
Inger	„Rendelés leadása” kérés a checkout API-nak
Környezet	Normál csúcsterhelés, 200 rendelési kérés/perc, adatbázis replikáció késése 2 s
Műtárgy	Rendelés-szolgáltatás (order-service)
Válasz	A rendelés rögzítésre kerül, a felhasználó visszaigazolást kap, a fulfillment sorba kerül
Mérőszám	Válaszidő P95 800 ms, sikeres válasz 99,9%, 5 perces csúszóablak, 30 napos termelési forgalom

A sablon nem kötelező minden mezőt kitölteni, de az inger, környezet, válasz és mérőszám mindig szerepeljen. Az azonosító a nyomon követést szolgálja: az ADR-ekben, a tesztesetekben és a fitness functionökben erre hivatkozunk.

9.1.10 Példák különböző minőségi attribútumokra

A minőségi forgatókönyvek minden minőségi attribútumra felírhatók. Az alábbiakban a leggyakoribb attribútumokra mutatunk példákat, a fejezetben tárgyalt négy elem szerint.

9.1.11 Teljesítmény és skálázhatóság

- **Inger**: termékkeresési kérés a webes felületről, 10 karakteres kulcsszóval.
- **Környezet**: normál üzem, 500 egyidejű felhasználó, a keresési index frissítése 5 perce befejeződött.
- **Válasz**: a találati lista első oldala megjelenik.
- **Mérőszám**: válaszidő P50 200 ms, P95 600 ms, 5 perces csúszóablakban, 7 napos termelési forgalom alapján.

9.1.12 Rendelkezésre állás és megbízhatóság

- **Inger**: az elsődleges adatbázis node kiesése.
- **Környezet**: csúcsterhelés, a rendszer 1000 kérés/másodpercet szolgál ki.
- **Válasz**: a rendszer automatikusan átkapcsol a replikára, a bejövő kérések továbbra is kiszolgálásra kerülnek.
- **Mérőszám**: a kiesés észlelésétől a forgalom átirányításáig eltelt idő 10 s, a sikeres kérések aránya 99,5% az átállás alatt, a mérés automatikus failover teszttel, 10 ismétléssel.

9.1.13 Biztonság

- **Inger**: brute-force bejelentkezési próbálkozás 50 különböző felhasználónévvel, 5 perc alatt.
- **Környezet**: normál üzem, a támadó IP-címe nem szerepel a belső hálózatban.
- **Válasz**: a rendszer észleli a mintázatot, letiltja a támadó IP-címet, és riasztást küld a biztonsági üzemeltetésnek.
- **Mérőszám**: az észlelés és a letiltás közötti idő 30 s, a téves pozitív arány 1% 30 napos ablakban, a mérés szintetikus támadási forgatókönyvvel.

9.1.14 Karbantarthatóság

- **Inger**: egy új fizetési szolgáltató integrációja.
- **Környezet**: a fejlesztői környezetben, a meglévő fizetési modul forráskódja rendelkezésre áll.
- **Válasz**: az új szolgáltató adaptere implementálásra kerül, a meglévő API nem változik.
- **Mérőszám**: a szükséges kódmódosítás legfeljebb 200 sor, a meglévő interfésztesztek változtatás nélkül lefutnak, a mérés a fejlesztési folyamat során, code review-val.

9.1.15 Használhatóság

- **Inger**: első vásárló a mobilalkalmazásban a kosárba helyez egy terméket.
- **Környezet**: normál üzem, a felhasználó 4G mobilhálózaton, 3 hüvelykes képernyőn.
- **Válasz**: a kosár ikon frissül, a felhasználó vizuális visszajelzést kap.
- **Mérőszám**: a visszajelzés megjelenésének ideje 300 ms, a művelet sikerességi aránya 99% 30 napos felhasználói minta alapján, a mérés session replay és RUM adatokból.

9.1.16 Üzemeltethetőség

- **Inger**: egy szolgáltatás memóriahasználata meghaladja a 85%-ot.
- **Környezet**: normál üzem, a szolgáltatás 3 példányban fut Kubernetesben.
- **Válasz**: a monitorozó rendszer riasztást küld az üzemeltetési csatornára, a metrikák a dashboardon megjelennek.
- **Mérőszám**: a riasztás kiküldésének ideje 60 s a küszöb átlépésétől, a riasztás 30 napos ablakban legalább 95%-ban eljut az ügyeleteshez, a mérés injektált hibával.

Ezek a példák azt mutatják, hogy ugyanaz a szerkezet különböző attribútumokra alkalmazható. A forgatókönyv nem attól minőségi, hogy hosszú, hanem hogy konkrét és mérhető.

9.1.17 A forgatókönyvek priorizálása

Egy rendszerhez gyakran több tucat minőségi forgatókönyv készül. Nem mindegyik azonos súlyú, és a tervezés során a trade-offok miatt nem lehet mindet maximálisan teljesíteni. A priorizálás célja, hogy a tervezési erőfeszítést a legnagyobb üzleti és kockázati hatású forgatókönyvekre összpontosítsuk.

A priorizálás lépései:

1. **Gyűjtés**: a stakeholder interjúk, a korábbi incidensek, a termékkövetelmények és a szabályozási előírások alapján összegyűjtjük a forgatókönyveket.

2. **Osztályozás**: minden forgatókönyvet besorolunk minőségi attribútum szerint, és azonosítjuk az érintett komponenseket.
3. **Üzleti érték becslése**: mekkora bevételt, ügyfél-elégedettséget vagy szabályozási kockázatot érint a forgatókönyv. Használhatunk egyszerű skálát (1–5), vagy CBAM-alapú költség-haszon elemzést.
4. **Kockázat becslése**: mekkora a valószínűsége és a hatása annak, ha a forgatókönyv nem teljesül. A korábbi incidensek, a hiba-mód elemzés és a terhelési modellek alapján.
5. **Megvalósíthatóság becslése**: mennyire nehéz a forgatókönyvet a tervezett architektúrában teljesíteni. A PoC-k és spike-ok itt adnak bemenetet.
6. **Súlyozás és sorrend**: az üzleti érték, a kockázat és a megvalósíthatóság alapján rangsorolunk. A MoSCoW (Must, Should, Could, Won't) vagy egy súlyozott döntési mátrix segíthet.

A priorizálás eredménye nem statikus. A termék élettartama során a piac, a szabályozás és a technológia változik, ezért a forgatókönyvek rangsorát rendszeresen felül kell vizsgálni. Az architektúra-governance részeként a minőségi forgatókönyvek és azok prioritása az ADR-ekben is rögzíthető.

9.1.18 Példa a priorizálásra

Egy e-kereskedelmi rendszerben a következő forgatókönyvek versengenek egymással:

- QA-001: rendelés leadása P95 800 ms (teljesítmény)
- QA-002: az elsődleges adatbázis kiesése 10 s alatt átkapcsol (rendelkezésre állás)
- QA-003: brute-force bejelentkezés észlelése 30 s alatt (biztonság)
- QA-004: új fizetési szolgáltató integrálása 200 sor kódmódosítással (karbantarthatóság)

A priorizálás során a következőket állapítjuk meg:

- QA-001 üzleti értéke magas (5/5), kockázata közepes (3/5), mert a lassú rendelés elvesztett vásárlókat okoz; megvalósíthatósága jó (2/5 nehézség).
- QA-002 üzleti értéke nagyon magas (5/5), kockázata alacsony valószínűségű, de nagy hatású (4/5); megvalósíthatósága közepes (3/5).
- QA-003 üzleti értéke közepes (3/5), kockázata növekvő (4/5); megvalósíthatósága jó (2/5).
- QA-004 üzleti értéke közepes (3/5), kockázata alacsony (2/5); megvalósíthatósága jó (2/5).

A rangsor: QA-001, QA-002, QA-003, QA-004. A sorrend azt jelenti, hogy a tervezési kapacitást először az első két forgatókönyvre összpontosítjuk, a többire pedig akkor, ha a trade-offok ezt lehetővé teszik.

9.1.19 Tipikus hibák és elkerülésük

A forgatókönyvek írásakor gyakran előforduló hibák:

9.1.20 1. Nem mérhető megfogalmazás

Hibás: „A rendszer gyorsan válaszol."
Helyes: „A válaszidő P95 800 ms 5 perces csúszóablakban."

A hibás megfogalmazásból hiányzik a mérőszám és az ablak. A helyes megfogalmazás lehetővé teszi, hogy a tesztelés során egyértelműen eldöntsük, teljesült-e a követelmény.

9.1.21 2. A környezet hiánya

Hibás: „A rendelés leadása 3 másodpercen belül megtörténik.”
Helyes: „Normál csúcsterhelés esetén, 200 rendelési kérés/perc mellett a rendelés leadása 3 másodpercen belül megtörténik.”

A környezet nélkül a mérőszám nem reprodukálható. Ugyanaz a válaszidő terhelés alatt könnyen teljesíthető, csúcsterhelés alatt pedig lehetetlen.

9.1.22 3. A megoldás beírása a forgatókönyvbe

Hibás: „A load balancer a kérést a leggyorsabb node-ra irányítja, és a válaszidő 500 ms alatt marad.”
Helyes: „A kérésre a válaszidő 500 ms alatt marad.”

A forgatókönyv a követelményt rögzíti, nem a megoldást. A megoldás az architektúra tervezése során születik, és az ADR-ekben dokumentáljuk.

9.1.23 4. Túl tág inger

Hibás: „Bármilyen kérés esetén a rendszer gyors.”
Helyes: „A termékkeresési kérés esetén a válaszidő P95 600 ms.”

A „bármilyen kérés” nem azonosítható. A forgatókönyvnek konkrét ingerre kell vonatkoznia, mert a rendszer különböző kérésekre eltérő erőforrásokat használ.

9.1.24 5. A mérési ablak hiánya

Hibás: „A rendelkezésre állás 99,9%.”
Helyes: „A rendelkezésre állás 99,9% 30 napos csúszóablakban.”

A rendelkezésre állás időbeli ablak nélkül értelmezhetetlen. A 99,9% 5 perc alatt mást jelent, mint 30 nap alatt. A mérési ablak nélkül az SLO sem határozható meg.

9.1.25 6. Túl sok mérőszám egy forgatókönyvben

Hibás: „A válaszidő P50 200 ms, P95 600 ms, P99 1 s, a hibaarány 0,1%, a throughput 1000 kérés/s.”
Helyes: a legfontosabb mérőszámot vagy két-három összefüggő mérőszámot adjunk meg, és a többit külön forgatókönyvbe.

A túl sok mérőszám elvonja a figyelmet a lényegről, és a tesztelést bonyolulttá teszi. A forgatókönyv egyetlen minőségi jellemzőre összpontosítson.

9.1.26 A forgatókönyvek ellenőrzése

Minden forgatókönyvet érdemes egy ellenőrzőlistával átvizsgálni, mielőtt az ADR-ekbe vagy a teszttervekbe kerül:

- Az inger konkrét, nem általános.
- A környezet megadja a terhelést, a rendszerállapotot és az időzítést.
- A válasz megfigyelhető, nem megoldás-specifikus.
- A mérőszám mértékegységgel, statisztikai jellemzővel és mérési ablakkal szerepel.
- A forgatókönyv egyetlen minőségi attribútumra vonatkozik.
- A mérési eljárás környezete, konfigurációja és bizonytalansága dokumentálható.
- A forgatókönyv azonosítóval rendelkezik, és hivatkozható az ADR-ekből és a tesztesetekből.

9.1.27 Forgatókönyvek az életciklusban

A minőségi forgatókönyvek nem csak a tervezés elején hasznosak. Az életciklus során több helyen is szerepet kapnak:

- **Követelményanalízis**: a stakeholder elvárások forgatókönyv formába öntése.
- **Architektúra tervezés**: a forgatókönyvek alapján választunk stílust és mintákat; az ATAM értékelés során ezeket vizsgáljuk.
- **Fejlesztés**: a forgatókönyvek tesztesetek alapjául szolgálnak; a fejlesztők a mérőszámokat a kódba épített metrikákkal ellenőrzik.
- **CI/CD**: a forgatókönyvek fitness functionként implementálhatók, így minden commit vagy release esetén automatikusan ellenőrizhetők.
- **Üzemeltetés**: az SLO-k és riasztási szabályok a forgatókönyvek mérőszámaiból származnak.
- **Review**: az architektúra-review során a forgatókönyvek teljesülését értékeljük, és az eltéréseket dokumentáljuk.

A következő alfejezet az SLO-k, SLI-k és error budgetek témájával foglalkozik, amelyek a forgatókönyvek mérőszámait üzemeltetési célokká alakítják.

9.1.28 Labor: hat quality attribute scenario megírása

9.1.29 Cél

A labor célja, hogy a résztvevő hat minőségi attribútum forgatókönyvet írjon egy választott vagy megadott rendszerre, a fejezetben bemutatott sablon szerint. A forgatókönyveknek legalább három különböző minőségi attribútumot kell lefedniük, és a négy kötelező elemet (inger, környezet, válasz, mérőszám) tartalmazniuk kell.

9.1.30 Kiinduló állapot

A résztvevő rendelkezik egy rendszerkontextussal, amely lehet:

- a könyv záróprojektjének rendszerleírása, vagy
- a résztvevő saját munkahelyi rendszere (anonimizálva), vagy
- egy megadott fiktív rendszer: például egy e-kereskedelmi platform, amely termékkeresést, kosarat, rendeléskezelést, fizetési integrációt és felhasználói fiókot tartalmaz.

A laborhoz nincs szükség futó rendszerre; a forgatókönyvek specifikációk, nem mérések. Ha a résztvevő rendelkezik valós rendszerrel, akkor a forgatókönyvek mellé mérési tervet is készíthet, de a mért eredményt külön kell választani a várt eredménytől.

9.1.31 Feladat

1. Válasszon ki hat minőségi jellemzőt vagy követelményt a rendszerhez.
2. Minden jellemzőt írjon le forgatókönyv formában a sablon szerint.
3. A forgatókönyvek legalább három különböző minőségi attribútumot fedjenek le (például teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság, használhatóság, üzemeltethetőség).
4. Minden forgatókönyvhöz adjon azonosítót, és a mérőszámot mértékegységgel és mérési ablakkal adja meg.
5. Készítsen prioritási sorrendet a hat forgatókönyvre, rövid indoklással.
6. Dokumentálja a forgatókönyvekhez tartozó döntési naplót: melyik forgatókönyv milyen architektúra-döntést befolyásol.
7. Válassza szét a mért és a várt eredményt: ha van mért adat, azt jelölje külön, a várt értékeket pedig a forgatókönyv mérőszám rovatában adja meg.

9.1.32 Elvárt eredmény

Egy reprodukálható dokumentum (Markdown, Jupyter notebook vagy Confluence-oldal), amely tartalmazza:

- a rendszer rövid kontextusleírását;
- a hat forgatókönyvet a sablon szerint;
- a prioritási sorrendet indoklással;
- a döntési napló bejegyzéseit (legalább hivatkozásszinten);
- az ellenőrzési szempontokat.

9.1.33 Ellenőrzési szempontok

- Minden forgatókönyvben szerepel az inger, a környezet, a válasz és a mérőszám.
- A mérőszámok mértékegységgel és mérési ablakkal rendelkeznek.
- A forgatókönyvek nem tartalmaznak megoldás-specifikus leírást.
- A prioritási sorrend indoklása az üzleti értékre, a kockázatra és a megvalósíthatóságra hivatkozik.
- A mért és a várt eredmény egyértelműen elkülönül.
- A dokumentum verziózott, és a döntésekhez visszakereshető hivatkozás tartozik.

9.1.34 Visszaállítási út

A labor nem igényel erőforrásokat; a dokumentum bármikor szerkeszthető. Ha a résztvevő valós rendszerből gyűjtött mérési adatot, akkor az adatokat anonimizálni kell, és csak a labor elvégzése után szabad a dokumentumban szerepeltetni.

9.1.35 Ellenőrzőlista a fejezethez

- Meg tudom magyarázni, miért nem elegendő a „legyen gyors” típusú követelmény.
- Fel tudom sorolni a forgatókönyv négy kötelező elemét.
- Meg tudom adni a mérőszámot mértékegységgel, statisztikai jellemzővel és mérési ablakkal.
- Felismerem a tipikus hibákat: hiányzó környezet, nem mérhető megfogalmazás, megoldás beírása.
- El tudom végezni a forgatókönyvek priorizálását egyszerű szempontrendszerrel.
- Tudom, hogyan kapcsolódnak a forgatókönyvek az SLO-khoz és a fitness functionökhöz.

9.1.36 Önellenőrző kérdések

1. Egy forgatókönyvben a következő szerepel: „A rendszernek gyorsnak kell lennie.” Mi hiányzik ebből a megfogalmazásból? Írd át a forgatókönyvet a négy kötelező elem felhasználásával.
2. Miért fontos a környezet megadása a minőségi forgatókönyvben? Adj példát arra, hogyan változik a mérőszám a környezet függvényében.
3. Milyen szempontok alapján priorizálnád a következő két forgatókönyvet: (a) a bejelentkezési oldal betöltése P95 1 s, (b) a fizetési tranzakció visszaigazolása 2 s és a hibaarány 0,1%? Indokold a sorrendet.
4. Mi a különbség a mért és a várt eredmény között a forgatókönyvek dokumentálásában? Miért fontos a szétválasztás?
5. Hogyan ellenőriznéd, hogy egy forgatókönyv nem tartalmaz-e megoldás-specifikus leírást? Mondj példát hibás és helyes megfogalmazásra.

A fejezetben bemutatott eszköz a minőségi követelményeket konkrét, mérhető és nyomon követhető formába önti. A következő alfejezetben ezekből a forgatókönyvekből építjük fel az SLO-k, SLI-k és error budgetek rendszerét, amely az üzemeltetési célokat és a megengedett hibakeretet számszerűsíti.

9.2 SLO, SLI, error budget és mérés

A szolgáltatási célok és a megbízhatóság mérése akkor válik kézzelfoghatóvá, amikor a korábban megismert minőségi forgatókönyvekben szereplő válaszokat számszerűsítjük. A quality attribute scenario megmondja, hogy egy adott inger hatására milyen választ várunk el; az SLO, az SLI és az error budget azt mondja meg, hogy ezt a választ milyen pontossággal, milyen időablakban és milyen következménnyel tudjuk garantálni. Ebben a fejezetben ezeket a fogalmakat járjuk körbe, és bemutatunk egy olyan gyakorlati tervet, amelyet egy kisebb rendszeren ellenőrizhető módon lehet elkészíteni.

9.2.1 Miért mérjük a megbízhatóságot?

A megbízhatóság nem abszolút kategória, hanem mérhető és kezelhető tulajdonság. Ha egy rendszerről csak annyit mondunk, hogy „megbízható”, az nem ad semmilyen támpontot sem a fejlesztőnek, sem az üzemeltetőnek, sem az üzleti döntéshozónak. A megbízhatóság mérésének akkor van értelme, ha konkrét, a felhasználó által észlelt viselkedést írunk le, és ahhoz egy számszerű célt rendelünk. Például ahelyett, hogy azt mondjuk: „a webshop legyen gyors”, azt mondjuk: „a rendelésleadási kérések 99. százaléka 400 milliszekundumon belül válaszoljon a

hónap során”. Ez a mondat már mérhető, ellenőrizhető, és képes arra, hogy a csapat eldöntse, teljesítette-e a célt.

A mérés további előnye, hogy lehetővé teszi a döntéshozatalt. Ha a csapat tudja, hogy egy hónapban mennyi hibát engedhet meg, akkor jobban tudja priorizálni a munkát: mikor érdemes új funkciót fejleszteni, és mikor kell a stabilitásra koncentrálni. A megbízhatósági célok és a hozzájuk tartozó hibakeretek így nem csupán technikai mutatók, hanem a termékfejlesztés és az üzemeltetés közötti párbeszéd alapjai.

9.2.2 SLI: a szolgáltatás valós állapotának indikátora

Az SLI (Service Level Indicator, szolgáltatási szint indikátor) egy olyan mérőszám, amely a szolgáltatás egy adott tulajdonságát írja le, lehetőleg a felhasználó szemszögéből. Az SLI nem cél, hanem megfigyelés: azt mutatja, hogy a rendszer pillanatnyilag vagy egy időablakban hogyan viselkedik. A leggyakoribb SLI-k a következők:

- **Rendelkezésre állás (availability):** a sikeres kérések aránya az összes kéréshez képest. Például HTTP 5xx válaszok aránya, vagy azoknak a kéréseknek az aránya, amelyek a megadott időn belül nem kapnak választ.
- **Késleltetés (latency):** a válaszidő eloszlása, leggyakrabban percentilisként megadva. A 99. percentilis (p99) azt jelenti, hogy a kérések 99 százaléka ennél gyorsabb volt.
- **Hibaarány (error rate):** a sikertelen kérések aránya, amely lehet technikai hiba (pl. HTTP 500) vagy üzleti hiba (pl. érvénytelen fizetési adat).
- **Átbocsátóképesség (throughput):** az időegység alatt kiszolgált kérések száma.
- **Telítettség (saturation):** mennyire van kihasználva a rendszer egy erőforrása, például CPU, memória, adatbázis-kapcsolatok. Ez inkább belső indikátor, de gyakran előrejelzi a felhasználói oldalon észlelt romlást.

Az SLI megválasztásánál a legfontosabb kérdés: mit észlel a felhasználó? Például egy webshopnál a felhasználó nem azt látja, hogy a szerver CPU-ja 90 százalékon van, hanem hogy a rendelés leadása lassú vagy hibát ad. Ezért a jó SLI a felhasználói interakció sikerességét méri, nem pedig egy belső erőforrás állapotát. A CPU-terhelés hasznos lehet a diagnózishoz, de nem alkalmas a megbízhatósági cél elsődleges indikátorának.

Az SLI-t mindig pontosan definiálni kell, különben a mérési eredmények félreérthetők. A definíció tartalmazza a mért eseményt, a siker feltételét, a mérési módszert és az aggregáció módját. Például:

- **Esemény:** a `POST /api/orders` végpontra érkező kérés.
- **Siker feltétele:** a válasz HTTP státusza 2xx, és a válasz a kérés elküldésétől számított 500 ms-on belül megérkezik.
- **Mérés:** szerveroldali napló alapján, a válaszidőt a kérés beérkezésétől a válasz elküldéséig mérjük.
- **Aggregáció:** a sikeres események aránya a teljes eseményszámhoz képest, 30 napos gördülő ablakban.

Ez a definíció egyértelművé teszi, hogy mit is mérünk, és hogyan lehet reprodukálni a mérést.

9.2.3 SLO: a cél, amit teljesíteni szeretnénk

Az SLO (Service Level Objective, szolgáltatási szint cél) egy SLI-re adott időablakra vonatkozó célérték. Az SLO azt mondja meg, hogy az adott indikátornak milyen arányban kell teljesülnie ahhoz, hogy a szolgáltatást „elég jónak” tekintsük. Például: „a rendelésleadási kérések 99,9 százalékának 500 ms-on belül kell sikeresen befejeződnie, 30 napos gördülő ablakban”. Itt az SLI a sikeres kérések aránya, az SLO pedig a 99,9% célérték.

Az SLO meghatározása nem technikai, hanem üzleti és felhasználói döntés. Azt kell meghatározni, hogy a felhasználók számára mi az elfogadható szolgáltatási szint, és mennyit hajlandók fizetni érte. A túl szigorú SLO (például 99,999%) rendkívül költséges lehet, mert minden apró hibát azonnal kezelni kell, és a rendszer redundanciája jelentős. A túl laza SLO viszont azt eredményezheti, hogy a felhasználók elpártolnak, mert a szolgáltatás nem elég megbízható. A gyakorlatban a 99,9% és 99,99% közötti értékek gyakoriak a webes szolgáltatásoknál, de ez erősen kontextusfüggő. Egy banki átutalási rendszernél magasabb elvárás lehet, míg egy belső jelentéskészítő eszköznél alacsonyabb is elfogadható.

Az SLO-t általában egy időablakra vonatkoztatva adjuk meg. Az időablak lehet naptári hónap, 28 napos gördülő ablak, vagy akár rövidebb időszak, például 7 nap. A választás attól függ, hogy milyen gyakran akarjuk értékelni a teljesítményt, és hogyan szeretnénk kezelni a szezonalitást. A gördülő ablak előnye, hogy mindig az aktuális állapotot tükrözi, míg a naptári hónaphoz kötött ablak könnyebben kommunikálható az üzleti szereplők számára. A könyvben a képleteknél az időablakot mindig jelöljük, mert anélkül a százalékos érték nem értelmezhető.

Az SLO-t nem szabad összekeverni az SLA-val (Service Level Agreement, szolgáltatási szint megállapodás). Az SLA egy jogi vagy szerződéses kötelezettségvállalás, amely két fél között jön létre. Az SLA általában szigorúbb, mint az SLO, mert ha a szolgáltató nem teljesíti az SLA-ban foglaltakat, akkor kötbért vagy más jogi következményt vállal. Az SLO ezzel szemben a csapat belső célja, amely gyakran lazább, hogy legyen mozgástér a hibákra. Például egy belső SLO lehet 99,9%, míg az ügyféllel kötött SLA 99,95%. A kettő közötti különbség a biztonsági tartalék. Fontos, hogy az SLO-t ne úgy állítsuk be, hogy az SLA-val azonos legyen, mert akkor minden apró megingás jogi következménnyel járhat.

9.2.4 Error budget: a megengedett hiba mennyisége

Az error budget (hibakeret) az a hibaarány, amelyet a rendszer egy adott időablakban megengedhet magának, miközben még éppen teljesíti az SLO-t. Matematikailag az error budget az SLO komplementere:

$$\text{Error budget} = 1 - \text{SLO}$$

Ha az SLO 99,9%, akkor az error budget 0,1%. Ez 30 napos ablakban (43 200 perc) 43,2 perc állásidőt vagy hibás kérést jelent, ha a rendelkezésre állás a mért SLI. Természetesen a hiba nem feltétlenül egybefüggő állásidő; lehet sok apró kérés hiba, vagy néhány hosszabb kiesés. A lényeg, hogy az összesített hibaarány ne haladja meg a keretet.

Az error budget célja, hogy a rendszer üzemeltetése során tudatosan kezeljük a kockázatot. Ha a hibakeret még nagy, akkor a csapat bátrabban vállalhat kockázatot, például új verziót telepíthet, mert van tartalék a váratlan hibákra. Ha a keret elfogyott, akkor a csapatnak a stabilitásra kell koncentrálnia, és el kell halasztania a kockázatos változtatásokat. Ezt az elvet nevezzük „error budget alapú döntéshozatalnak”. A Google Site Reliability Engineering könyve nyomán terjedt

el, de a mögöttes gondolat egyszerű: a megbízhatóság nem ingyenes, és a hibák bizonyos szintje természetes.

Az error budget felhasználása konkrét döntési szabályokhoz köthető. Például:

- Ha az error budget 50%-nál többet felhasznált, a csapat automatikusan leállítja a nem sürgős feature-fejlesztést, és a hibajavításra, tesztelésre, monitoringra összpontosít.
- Ha az error budget 100%-ban elfogyott, a változtatások befagyasztásra kerülnek, kivéve a hibajavításokat és a biztonsági frissítéseket.
- Az error budget egy része fenntartható tervezett karbantartásra, például adatbázis-migrációra.

Fontos, hogy az error budget nem azt jelenti, hogy a csapat „felhasználhatja” a hibákat szándékosan, hanem azt, hogy a kockázatvállalás mértéke a fel nem használt kerethez igazodik. A cél az, hogy a megbízhatóság és a fejlesztési sebesség között fenntartható egyensúly alakuljon ki.

9.2.5 A mérési ablak és a felhasználói észlelés

A mérési ablak megválasztása kritikus, mert ugyanaz a nyers adat más következtetésre vezethet különböző ablakokban. Például egy 99,9% SLO egy 30 napos ablakban 43,2 perc hibát enged meg, míg egy 7 napos ablakban csak 10,1 percet. Ha a rendszer egy hónapban két kisebb kiesést szenved, az a havi ablakban még beleférhet, de a heti ablakban már nem. Az ablak hosszát a felhasználói elvárásokhoz és az üzleti ciklushoz kell igazítani. Egy előfizetéses szolgáltatásnál a havi ablak logikus, mert a számlázás is havi. Egy valós idejű kereskedési platformnál viszont a havi ablak túl durva lehet; ott rövidebb, akár órás ablakok is szükségesek.

A mérési ablak mellett a felhasználói észlelés módja is meghatározó. Az SLI-t úgy kell definiálni, hogy az tükrözze, mit tapasztal a felhasználó. Például ha egy webshop a `GET /api/products` végpontot figyeli, de a felhasználó a rendelésleadáskor hibát kap, akkor az SLI nem a teljes felhasználói élményt méri. Ezért a gyakorlatban a kritikus felhasználói utakat (például böngészés, kosárba helyezés, fizetés, rendelés visszaigazolása) külön-külön SLI-kkel kell lefedni.

A felhasználói észlelés mérésére két fő megközelítés létezik: a valós forgalom alapú mérés és a szintetikus monitorozás. A valós forgalom alapú mérés a tényleges felhasználói kéréseket vizsgálja, így pontos képet ad arról, hogy mit tapasztalnak a felhasználók. A szintetikus monitorozás viszont mesterséges kéréseket küld rendszeresen, például egy külső szolgáltatásból, és így akkor is érzékeli a kiesést, ha éppen nincs valós forgalom. A kettőt gyakran kombinálják: a szintetikus tesztek a rendelkezésre állás folyamatos ellenőrzésére szolgálnak, a valós forgalom elemzése pedig a tényleges felhasználói élményt méri.

A késleltetés mérésénél különösen fontos a percentilisek használata. Az átlagos válaszidő félrevezető lehet, mert néhány nagyon lassú kérés torzítja az átlagot, miközben a felhasználók többsége gyors választ kap. A p95 vagy p99 percentilis jobban tükrözi a legrosszabb élményt, amelyet a felhasználók egy kis része tapasztal. Például ha az átlagos válaszidő 120 ms, de a p99 2 másodperc, akkor a felhasználók 1 százaléka jelentős késleltetést szenved, ami ronthatja az elégedettséget. Az SLO-t ezért gyakran percentilisre fogalmazzák meg: „a kérések 99%-ának 500 ms-on belül kell befejeződnie”.

9.2.6 Gyakorlati minta: SLO- és error budget-terv egy rendeléskezelő szolgáltatáshoz

Vegyünk egy képzeletbeli rendeléskezelő szolgáltatást, amely egy webshop része. A szolgáltatás a `POST /api/orders` végponton fogadja a rendeléseket, és a `GET /api/orders/{id}` végponton

visszaadja azok állapotát. A felhasználók a webes felületen keresztül adják le rendeléseiket, és azonnali visszaigazolást várnak. A csapat a következő minőségi forgatókönyvet fogalmazta meg (a Quality attribute scenarios fejezetben tanult formában):

- **Inger:** a felhasználó elküld egy rendelést a webes felületen.
- **Környezet:** normál üzem, átlagos terhelés mellett, 30 napos időszakban.
- **Válasz:** a rendszer a rendelést sikeresen rögzíti, és a felhasználó 500 ms-on belül visszaigazolást kap.
- **Mérőszám:** a sikeres és időben beérkező válaszok aránya legalább 99,9%.

Ebből a forgatókönyvből levezethető az SLI és az SLO.

9.2.7 SLI definíció

Az SLI-t a következőképpen definiáljuk:

- **Megnevezés:** `order_create_success_rate`
- **Mért esemény:** minden, a `POST /api/orders` végpontra érkező kérés, amely érvényes autentikációval rendelkezik.
- **Siker feltétele:** a válasz HTTP státusza 2xx, és a válaszidő a kérés beérkezésétől a válasz elküldéséig 500 ms.
- **Mérési módszer:** szerveroldali access logból és alkalmazás-metrikából származó idősor, amelyet a monitoring rendszer 1 perces aggregációban tárol.
- **Aggregáció:** a sikeres események számának aránya az összes mért eseményhez képest, 30 napos gördülő ablakban, százalékban kifejezve.

Ez a definíció egyértelmű, és bármely mérnök reprodukálni tudja a mérést. A helyőrző `POST /api/orders` végpont itt természetesen fiktív; éles rendszerben a valós végpontot kell használni.

9.2.8 SLO meghatározása

A csapat úgy dönt, hogy az SLO 99,9% legyen, mert a felhasználói kutatások szerint a vásárlók számára elfogadható, ha 1000 rendelésből legfeljebb 1 hibásodik meg. Az SLO-t havi gördülő ablakra vonatkoztatjuk, mert a webshop forgalma hétvégi csúcsokat mutat, és a havi ciklus jól illeszkedik az üzleti tervezéshez. A döntést ADR-ben rögzítjük.

9.2.9 Error budget számítása

Az SLO 99,9%, tehát az error budget:

$$\text{Error budget} = 1 - 0{,}999 = 0{,}001 = 0{,}1\%$$

30 napos ablakban a teljes idő 43 200 perc. A rendelkezésre állás szempontjából ez 43,2 perc hibát enged meg. Ha a hiba kérésekben mérjük, akkor a havi összes kérésszám függvényében az error budget abszolút számossá is átszámítható. Például ha a szolgáltatás havi 1 millió rendelési kérést kap, akkor 0,1% = 1000 hibás kérés engedhető meg.

A csapat az error budget egy részét (például 40%-át) tervezett karbantartásra és kiadásokra tartja fenn, a maradék 60%-ot pedig a váratlan hibákra. Így a kiadási döntéseknél mindig tudják, hogy mennyi tartalékuk van.

9.2.10 A terv dokumentálása

A fenti elemeket egy reprodukálható dokumentumban kell rögzíteni. A dokumentum tartalmazza:

- A szolgáltatás nevét és a rendszerhatárt.
- A minőségi forgatókönyvet.
- Az SLI pontos definícióját.
- Az SLO értékét és az időablakot.
- Az error budgetet és annak felhasználási szabályait.
- A mérési módszert és az adatforrást.
- A döntési naplót (ADR) a fontosabb választásokról.

Egy ilyen dokumentum nem csupán papír; a csapat a mindennapi munkában használja, amikor kiadási döntést hoz, incidens után elemzi a hibát, vagy a jövőben módosítani szeretné a célokat.

9.2.11 Tipikus hibák és helyes megoldások

A gyakorlatban számos hiba előfordul az SLO-k és SLI-k bevezetésekor. Az alábbiakban felsoroljuk a leggyakoribbakat, és megindokoljuk a helyes megoldást.

9.2.12 1. Túl sok SLI

Hiba: a csapat minden elképzelhető mérőszámra definiál SLI-t, hogy „mindent lefedjen”. Ez átláthatatlanná teszi a rendszert, és senki sem tudja, melyik indikátor fontos.

Helyes megoldás: kezdjük néhány, a felhasználói élmény szempontjából kritikus SLI-vel. Általában 3-5 jól megválasztott indikátor elegendő. A többit belső diagnosztikai célra használjuk, de nem rendelünk hozzájuk SLO-t.

9.2.13 2. Nem felhasználói szempontú SLI

Hiba: az SLI a szerver CPU-terhelését vagy a memóriafogyasztást méri, mert az könnyen elérhető. De ezek nem tükrözik, hogy a felhasználó mit tapasztal.

Helyes megoldás: minden SLI-t a felhasználói interakció sikerességére vagy észlelt teljesítményére alapozzunk. A belső erőforrás-metrikák hasznosak lehetnek a telítettség előrejelzésére, de nem helyettesítik a felhasználói élményt mérő SLI-t.

9.2.14 3. Az SLO nem elég pontos

Hiba: az SLO úgy van megfogalmazva, hogy nem egyértelmű, például „a rendszer legyen gyors”. Ez nem mérhető.

Helyes megoldás: az SLO-t mindig egy konkrét SLI-re, egy numerikus célértékre és egy időablakra vonatkoztatva adjuk meg. Például: „a `POST /api/orders` kérések 99%-a 500 ms alatt sikeresen válaszol, 30 napos gördülő ablakban”.

9.2.15 4. Az error budgetet nem használják döntéshozatalra

Hiba: a csapat kiszámolja az error budgetet, de aztán figyelmen kívül hagyja, amikor kiadási döntést hoz. Így az egész gyakorlat értelmetlen.

Helyes megoldás: az error budgetet össze kell kötni a kiadási folyamattal. Például a CI/CD pipeline automatikusan ellenőrzi, hogy az error budget állapota lehetővé teszi-e a telepítést. Ha nem, a kiadás blokkolva van, vagy emberi jóváhagyást igényel.

9.2.16 5. Túl szigorú vagy túl laza SLO

Hiba: a csapat 99,999%-os SLO-t tűz ki, mert az jól hangzik, de a rendszer ezt nem tudja tartani, így az error budget folyamatosan kimerül, és a csapat demotivált lesz. Vagy ellenkezőleg, 99%-os SLO-t ad, ami a felhasználók számára elfogadhatatlanul rossz.

Helyes megoldás: az SLO-t a felhasználói elvárások és a tényleges mérési adatok alapján kell meghatározni. Induljunk ki a jelenlegi teljesítményből, és fokozatosan javítsuk a célt. Az SLO nem statikus; rendszeresen felül kell vizsgálni.

9.2.17 6. Az időablak nem illeszkedik a döntési ciklushoz

Hiba: a csapat éves ablakot használ, ami túl hosszú, és nem teszi lehetővé a gyors beavatkozást. Vagy éppen órás ablakot, ami túl zajos, és minden apró ingadozás riasztást vált ki.

Helyes megoldás: az időablak hosszát a szolgáltatás jellegéhez és a döntési ciklushoz igazítsuk. A gyakorlatban a 30 napos gördülő ablak sok szolgáltatásnál jól működik, mert elég hosszú a trendek észleléséhez, de elég rövid a beavatkozáshoz.

9.2.18 7. A mérési definíció nem reprodukálható

Hiba: az SLI-t leíró dokumentumból nem derül ki pontosan, hogyan kell mérni, ezért két ember eltérő eredményt kap.

Helyes megoldás: az SLI definíciójának tartalmaznia kell az eseményt, a siker feltételét, az adatforrást, az aggregációt és a számítási képletet. Ha lehetséges, a mérési logikát kódban, változásvezérelt módon kell rögzíteni, hogy bárki reprodukálhassa.

9.2.19 Labor: SLO- és error budget-terv készítése

A következő labor célja, hogy a tanuló elkészítsen egy SLO- és error budget-tervet egy kiválasztott szolgáltatáshoz. A tervet reprodukálható dokumentumban vagy jegyzetfüzetben kell rögzíteni, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal. A mért és a várt eredményt külön kell választani.

9.2.20 Kiinduló állapot

Válassz egy általad ismert vagy elképzelt szolgáltatást, például egy REST API-t, egy háttérfeldolgozó rendszert vagy egy mobilalkalmazás backendjét. A szolgáltatásnak legyen legalább egy kritikus felhasználói interakciója, például egy bejelentkezés, egy rendelés, egy keresés vagy egy fájlfeltöltés. A laborban nincs szükség valós rendszerre; a cél a tervezési folyamat gyakorlása.

9.2.21 Elvárt eredmény

Készíts egy dokumentumot (például Markdown vagy jegyzetfüzet), amely a következőket tartalmazza:

1. **Kontextus:** a szolgáltatás neve, célja, a felhasználói szerepkörök és a rendszerhatár. (A C4 context szintű leírást röviden megadhatod.)
2. **Minőségi forgatókönyv:** a kiválasztott kritikus interakcióra vonatkozó quality attribute scenario, a Quality attribute scenarios fejezetben tanult formában.
3. **SLI definíció:** a mért esemény, a siker feltétele, a mérési módszer, az aggregáció és az időablak.
4. **SLO:** a célérték százalékban, percentilisben vagy más mérőszámban, az időablakkal együtt.
5. **Error budget:** a kiszámított hibakeret százalékban és abszolút értékben (például percben vagy hibás kérések számában).
6. **Döntési napló:** legalább egy ADR, amely rögzíti a fontosabb választásokat, például az SLO értékének indoklását, az időablak megválasztását, és az error budget felhasználási szabályait.
7. **Ellenőrzési szempontok:** legalább 5 kérdésből álló lista, amellyel a terv helyességét ellenőrizni lehet.
8. **Várt és mért eredmény elkülönítése:** egy táblázat, amelyben jelölöd, hogy mi a tervezett (várt) érték, és mi a tényleges (mért) érték. A labor során a mért érték üresen maradhat, mivel nem végeztünk mérést, de a táblázat szerkezete mutatja, hogyan kell majd feltölteni.

9.2.22 Ellenőrzési szempontok (példa)

A dokumentum elkészítése után ellenőrizd a következőket:

- Az SLI definíciója elég pontos ahhoz, hogy egy másik mérnök reprodukálni tudja a mérést?
- Az SLO értéke indokolt-e a felhasználói elvárások alapján?
- Az error budget számítása helyes-e? (Használd az $1 - \text{SLO}$ képletet.)
- Az időablak illeszkedik-e a szolgáltatás üzleti ciklusához?
- A döntési napló tartalmazza-e a kontextust, a döntést, az alternatívákat és a következményeket?
- A dokumentum elkülöníti-e a mért és a várt eredményt?
- Nincsenek-e valós hitelesítő adatok vagy belső URL-ek a dokumentumban? (Használj helyőrzőket, például `example.internal`.)
- A terv alkalmas-e arra, hogy a későbbiekben fitness functionként használjuk? (Erről bővebben az ATAM, CBAM és architektúra-review és Evolúciós architektúra és fitness functions fejezetekben lesz szó.)

9.2.23 Visszaállítási út

Mivel a labor dokumentumot hoz létre, nincs szükség rendszer-visszaállításra. Ha a tanuló jegyzetfüzetben dolgozik, a kiinduló állapot egy üres sablon, és a visszaállítás a sablon újratöltését jelenti. A destruktív művelet nem része ennek a labornak.

9.2.24 Összefoglalás

A szolgáltatási célok és a megbízhatóság mérése a minőségi forgatókönyvek természetes folytatása. Az SLI megadja, mit mérünk, az SLO megadja, milyen értéket kell elérni, az error budget pedig számszerűsíti a megengedett hibát. A mérési ablak és a felhasználói észlelés helyes megválasztása nélkül azonban a számok félrevezetők lehetnek. A gyakorlatban az SLO-k akkor hasznosak, ha konkrét döntéseket vezérelnek, például a kiadási folyamatot, és ha a csapat rendszeresen felülvizsgálja őket.

9.2.25 Ellenőrzőlista a fejezethez

- ☐ Meg tudom magyarázni az SLI, SLO, SLA és error budget közötti különbséget.
- ☐ Tudok írni egy pontos SLI definíciót eseménnyel, sikerfeltétellel, mérési módszerrel és aggregációval.
- ☐ Ki tudom számolni az error budgetet adott SLO és időablak mellett.
- ☐ Felismerem a tipikus hibákat, például a belső erőforrás-metrikára épülő SLI-t vagy a túl szigorú SLO-t.
- ☐ El tudom készíteni egy szolgáltatás SLO- és error budget-tervét a laborban leírtak szerint.
- ☐ Tudom, hogy az error budgetet hogyan lehet a kiadási döntésekkel összekapcsolni.

9.2.26 Önellenőrző kérdés

Egy szolgáltatás SLO-ja 99,95% rendelkezésre állás, 30 napos gördülő ablakban. Mennyi hibát enged meg a rendszer percben kifejezve? Hogyan változik az error budget, ha az időablakot 7 napra csökkentjük? Magyarázd el, miért fontos, hogy az SLI a felhasználói élményt mérje, és ne csak egy belső erőforrás állapotát.

A következő fejezetben (ATAM, CBAM és architektúra-review) az ATAM, CBAM és a formális architektúra-értékelés módszereit ismerjük meg, ahol ezeket a minőségi célokat és méréseket használjuk fel az architektúra alternatívák összehasonlítására.

9.3 ATAM, CBAM és architektúra-review

A strukturált architektúraértékelés nem formális audit, hanem olyan kockázatfeltáró és döntés-előkészítő tevékenység, amelynek célja, hogy a minőségi követelmények és az architekturális döntések közötti kapcsolatot explicit módon, megismételhető lépésekkel vizsgálja. Az előző alfejezetben tárgyalt SLO-k, SLI-k és error budgetek adják a mérési alapot; ebben az alfejezetben azokat a módszereket vezetjük be, amelyekkel az architektúrát szisztematikusan értékelni tudjuk, mielőtt a rendszer élesbe kerül, vagy amikor egy jelentős változás előtt áll.

A legismertebb strukturált módszer az ATAM (Architecture Tradeoff Analysis Method), amelyet a SEI dolgozott ki a minőségi attribútumok közötti kompromisszumok feltárására. Mellette a CBAM (Cost Benefit Analysis Method) gazdasági szempontból rangsorolja az architekturális stratégiákat. A kettő együtt gyakorlati keretet ad az architektúra-review-hoz, amelyet ebben a fejezetben egy kis rendszeren keresztül is bemutatunk.

9.3.1 A strukturált architektúraértékelés célja és módszerei

Az architektúra értékelése során nem azt kérdezzük elsősorban, hogy a rendszer „jól van-e megtervezve”, hanem azt, hogy a meghatározó minőségi követelmények teljesülését milyen architekturális döntések támogatják, és ezek a döntések milyen kockázatokat hordoznak. A strukturált megközelítés három dolgot tesz hozzá az ad hoc véleményezéshez:

1. **Szcenárióalapú elemzést** alkalmaz: a követelményeket konkrét, mérhető minőségi forgatókönyvekké alakítja, így az értékelés nem általános elvek, hanem ellenőrizhető állítások körül folyik.
2. **Explicit trade-off elemzést** végez: azonosítja azokat a pontokat, ahol egy döntés javítja az egyik minőséget, de rontja a másikat.
3. **Dokumentált, reprodukálható folyamatot** követ: a résztvevők, a bemenetek, a vizsgált forgatókönyvek és a megállapítások rögzítésre kerülnek, így a későbbi döntések visszakereshetők.

A strukturált architektúraértékelés bemenete az architektúra-dokumentáció, a minőségi követelmények és az érintett stakeholderek listája. Kimenete pedig nem egy „megfelelt/nem felelt meg” ítélet, hanem kockázati lista, érzékenységi pontok, trade-off pontok és javasolt intézkedések. Ez illeszkedik a dokumentációs szerződésünkhöz: az értékelés megállapításait ADR-ekben vagy review-jegyzőkönyvben rögzítjük, minden kockázathoz hivatkozással a vizsgált forgatókönyvre és az architekturális döntésre.

A gyakorlatban több módszer is használatos:

- **SAAM** (Software Architecture Analysis Method): a korai, egyszerűbb szcenárióalapú értékelés, amely elsősorban a modifiability elemzésére koncentrál.
- **ATAM**: a minőségi attribútumok közötti kölcsönhatások és kompromisszumok elemzésére optimalizált módszer.
- **CBAM**: az ATAM kiegészítése gazdasági elemzéssel, ahol a lehetséges architekturális megoldások költségét és hasznát hasonlítjuk össze.
- **ARID** (Active Reviews for Intermediate Designs): részleges, konkrét tervezési dokumentumok aktív felülvizsgálatára szolgál.

Ebben a fejezetben az ATAM-ot és a CBAM-ot tárgyaljuk részletesen, mert ezek adják a legteljesebb keretet az architektúra trade-offjainak feltárására.

9.3.2 ATAM: Architecture Tradeoff Analysis Method

Az ATAM célja, hogy a minőségi attribútumok közötti kompromisszumokat és a kockázatokat már a tervezési fázisban, vagy egy meglévő rendszer módosítása előtt feltárja. A módszer nem a kódot vizsgálja, hanem az architekturális döntéseket, azok következményeit és a minőségi forgatókönyvekkel való kapcsolatukat.

Az ATAM alapfeltevése, hogy a minőségi attribútumok – teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság, módosíthatóság, tesztelhetőség, üzemeltethetőség – ritkán egymástól függetlenül optimalizálhatók. Egy döntés, amely a teljesítményt javítja, például a gyorsítótár bevezetése, rendszerint ronthatja az adatfrissességre vagy a konzisztenciára vonatkozó követelményeket. Az ATAM pontosan ezeket a keresztkapcsolatokat teszi láthatóvá.

9.3.3 Az ATAM lépései

Az ATAM tipikus felépítése a következő kilenc lépésből áll, amelyeket egy facilitátor vezet, és amelyben a stakeholderek (üzleti megrendelő, fejlesztő, üzemeltető, biztonsági felelős, tesztelő) aktívan részt vesznek:

1. **Az ATAM bemutatása**: a facilitátor ismerteti a módszert, az időkeretet és az elvárt kimeneteket.
2. **Üzleti célok és kontextus bemutatása**: az üzleti képviselő összefoglalja a rendszer célját, a piaci vagy szervezeti hajtóerőket, a legfontosabb funkcionális és minőségi elvárásokat.
3. **Az architektúra bemutatása**: a vezető tervező bemutatja a rendszerhatárokat, a fő komponenseket, a kapcsolatokat és a meghatározó architekturális döntéseket. A C4 context és container szintű diagramok itt természetes alapot adnak.
4. **Minőségi attribútumok azonosítása és priorizálása**: a csapat összegyűjti a fontos minőségi attribútumokat, és egy **utility tree**-ben rendszerezi őket.
5. **Az architekturális megközelítések elemzése**: a facilitátor és a tervezők az egyes architekturális döntéseket a minőségi forgatókönyvekhez rendelik, és azonosítják azokat a pontokat, ahol a döntés hatással van a minőségre.
6. **Szcenárióalapú elemzés**: a legmagasabb prioritású forgatókönyveket egyenként végigvezetik az architektúrán, keresve azok teljesülését vagy sérülését befolyásoló mechanizmusokat.
7. **Kockázatok, érzékenységi pontok és trade-off pontok rögzítése**: a talált problémákat osztályozzák és dokumentálják.
8. **Javaslatok és következmények**: a csapat megfogalmazza, milyen lépésekkel lehet a kockázatokat csökkenteni vagy a trade-offokat kezelni.
9. **Jelentés és dokumentáció**: az eredményt írásban rögzítik, jellemzően ADR-kiegészítéssel vagy külön review-jegyzőkönyvben.

A lépések száma az adott szervezetben változhat, de a lényeg mindig ugyanaz: a minőségi követelményeket forgatókönyvekké alakítjuk, az architektúrát ezekhez mérjük, és a talált kompromisszumokat explicit módon dokumentáljuk.

9.3.4 Utility tree: a minőségi forgatókönyvek rendszerezése

Az ATAM egyik központi eszköze az **utility tree**, amely a minőségi attribútumokat hierarchikus fában rendezi. A fa gyökere a rendszer általános minősége, a következő szint a minőségi attribútumok (például teljesítmény, rendelkezésre állás, biztonság, módosíthatóság), majd az attribútumokon belüli finomabb kategóriák, végül a konkrét minőségi forgatókönyvek. Minden forgatókönyvhöz prioritást rendelünk: magas (H), közepes (M) vagy alacsony (L), valamint jelöljük, hogy az adott forgatókönyv mennyire fontos az üzleti sikerhez.

Egy leegyszerűsített utility tree részlet egy elektronikus kereskedelmi rendszerhez:

Minőségi attribútum	Kategória	Forgatókönyv	Prioritás
Teljesítmény	Válaszidő	Amikor a felhasználó rákeres egy termékre, a rendszer normál terhelés mellett legfeljebb 300 ms alatt visszaadja a találati listát.	H

Minőségi attribútum	Kategória	Forgatókönyv	Prioritás
Teljesítmény	Áteresztőképesség	A rendszer csúcsidőben legalább 500 keresési kérést szolgál ki másodpercenként, miközben a p95 válaszidő nem haladja meg a 400 ms-ot.	H
Rendelkezésre állás	Tervezett karbantartás	A katalógusszolgáltatás verziófrissítése nem okoz 5 másodpercnél hosszabb teljes kiesést.	M
Biztonság	Adatvédelem	A vásárlói adatokhoz csak a megfelelő jogosultsággal rendelkező szolgáltatások férnek hozzá; a hozzáférési kísérletek naplózódnak.	H
Módosíthatóság	Funkcióbővítés	Egy új fizetési mód bevezetése legfeljebb 3 fejlesztői napot vesz igénybe, és nem igényel változtatást a megrendelés-szolgáltatásban.	M
Üzemeltethetőség	Megfigyelhetőség	Minden szolgáltatás automatikusan publikálja a metrikákat és a nyomkövetési adatokat, amelyek alapján a p95 válaszidő 5 percen belül megállapítható.	H

A utility tree két dolgot tesz lehetővé: egyrészt a stakeholderek közötti vita a prioritásokról strukturáltan zajlik, másrészt a későbbi elemzés fókuszát a magas prioritású forgatókönyvek adják. A prioritás nem abszolút igazság, hanem a résztvevők konszenzusa, ezért a döntést a jegyzőkönyvben rögzíteni kell.

9.3.5 Az ATAM elemzési fogalmai: kockázat, érzékenységi pont, trade-off pont

Az ATAM akkor válik igazán értékessé, amikor a forgatókönyvek elemzése során a következő három fogalmat világosan elkülönítjük:

- **Érzékenységi pont (sensitivity point)**: olyan architekturális döntés vagy paraméter, amelynek megváltoztatása jelentős hatással van egy adott minőségi attribútumra. Például a gyorsítótár élettartama (TTL) érzékenységi pont a frissesség és a teljesítmény szempontjából. Ha a TTL-t növeljük, a válaszidő javulhat, de az adatok elavulhatnak.
- **Trade-off pont**: olyan döntés, amely egyszerre több minőségi attribútumot ellentétes irányban befolyásol. A gyorsítótár bevezetése tipikus trade-off pont: javítja a teljesítményt, de ronthatja az adatkonzisztenciát és növelheti az üzemeltetési komplexitást.
- **Kockázat**: olyan érzékenységi vagy trade-off pont, amely nincs megfelelően kezelve, és ezért fenyegeti valamelyik minőségi követelmény teljesülését. A kockázatot mindig a konkrét forgatókönyvhöz kötjük: ha a gyorsítótár TTL-je 60 másodperc, és az üzleti követelmény szerint az árváltozásnak 10 másodpercen belül meg kell jelennie, akkor a TTL érzékenységi pontból kockázat lesz.

Fontos, hogy ezek a fogalmak nem minősítik a döntést önmagában. Egy trade-off pont nem feltétlenül rossz, csupán olyan hely, ahol a döntéshozónak tudatosan kell választania a versengő minőségi célok között. Az ATAM szerepe az, hogy ezeket a helyeket a felszínre hozza, és dokumentálja a döntés mögötti indokot.

9.3.6 Példa az ATAM elemzésre

Tegyük fel, hogy egy foglalási szolgáltatásban a csapat a következő döntést hozta: a szálláshelyek elérhetőségét egy Redis-alapú gyorsítótárban tárolják, hogy csökkentsék az adatbázis terhelését. A utility tree-ben szerepel egy magas prioritású forgatókönyv:

- **Forgatókönyv**: Amikor a felhasználó szállást keres, a rendszer normál terhelés mellett 200 ms alatt visszaadja a szabad szobák listáját.
- **Válasz**: A keresési kérés kiszolgálása.
- **Mérőszám**: p95 válaszidő 200 ms, mérési ablak 1 hónap.

Az elemzés során a gyorsítótár mint érzékenységi pont jelenik meg: a találati arány növelése csökkenti a válaszidőt, de a gyorsítótárban tárolt adatok frissességére is hatással van. Ha a gyorsítótár érvénytelenítése nem azonnali, akkor a felhasználó olyan szobát láthat szabadnak, amelyet időközben lefoglaltak. Ezért a gyorsítótár trade-off pont a teljesítmény és a konzisztencia között. A kockázat akkor keletkezik, ha az üzleti szabály szerint a foglalási rendszerben a lefoglalt szoba 1 másodpercen belül nem jelenhet meg szabadként, a gyorsítótár viszont csak 5 másodpercenként frissül. Ezt a kockázatot az ATAM rögzíti, és a csapatnak döntenie kell: vagy csökkenti a TTL-t, vagy eseményvezérelt érvénytelenítést vezet be, vagy elfogadja az üzleti kockázatot.

Az ATAM tehát nem oldja meg automatikusan a konfliktust, de láthatóvá teszi, és a döntést a megfelelő stakeholderek elé viszi.

9.3.7 CBAM: Cost Benefit Analysis Method

Az ATAM feltárja a minőségi problémákat és a trade-offokat, de nem ad választ arra, hogy a lehetséges javítási stratégiák közül melyiket érdemes választani. Ezt a döntést támogatja a **CBAM**, amely gazdasági elemzéssel egészíti ki az ATAM eredményeit.

A CBAM alapgondolata, hogy minden architekturális változtatásnak van költsége és haszna, és a haszon nemcsak pénzben mérhető, hanem a minőségi attribútumok javulásában is. A módszer a minőségi forgatókönyvekhez rendelt **hasznosságot** (utility) és a változtatás **költségét** veti össze, hogy a döntéshozók a legnagyobb nettó hasznot hozó stratégiákat válasszák.

A CBAM lépései jellemzően a következők:

1. **A minőségi forgatókönyvek kiválasztása**: az ATAM utility tree-jéből kiválasztjuk azokat a forgatókönyveket, amelyek javítása a legfontosabb az üzlet számára.
2. **Architekturális stratégiák azonosítása**: minden forgatókönyvhöz meghatározzuk a lehetséges megoldási lehetőségeket. Például a válaszidő csökkentésére lehetőség a gyorsítótár bevezetése, az adatbázis-indexelés javítása vagy a szolgáltatás horizontális skálázása.
3. **Hasznossági függvény meghatározása**: minden forgatókönyvhöz hozzárendelünk egy hasznossági értéket, amely kifejezi, mennyit ér az üzletnek az adott minőségi szint elérése. A skála lehet 0-tól 100-ig terjedő, ahol a 100 a maximális üzleti érték. A hasznosságot a stakeholderek becsléssel adják meg, és a bizonytalanságot rögzítik.
4. **A stratégiák hasznosságra gyakorolt hatásának becslése**: megbecsüljük, hogy az adott stratégia mennyivel növeli a forgatókönyv hasznosságát. Például a gyorsítótár bevezetése a válaszidő-forgatókönyv hasznosságát 40-ről 80-ra növelheti.
5. **Költségek becslése**: minden stratégiához hozzárendeljük a bevezetés és az üzemeltetés költségét, beleértve a fejlesztési időt, az infrastruktúra-költséget és a karbantartási többletet.
6. **Nettó haszon és ROI számítása**: a hasznosságnövekedés és a költség hányadosát vagy különbségét számítjuk ki. A stratégiákat ez alapján rangsoroljuk.
7. **Döntés és dokumentálás**: a kiválasztott stratégiát ADR-ben rögzítjük, a feladott alternatívákkal és a becslések bizonytalanságával együtt.

A CBAM fontos jellemzője, hogy a hasznossági értékek szubjektívek, és a becslések bizonytalanságát explicit módon kezeljük. A labor- és kódszerződésünknek megfelelően a nem futtatott mérést vagy becslést nem mutatjuk be ellenőrzött eredményként; a CBAM-ban ezért mindig jelöljük, hogy a hasznossági érték becslés, és milyen feltételezéseken alapul.

9.3.8 CBAM példa

Vegyünk egy egyszerű példát. Az ATAM során a következő magas prioritású forgatókönyvet azonosítottuk: a keresési válaszidő p95 értéke nem haladhatja meg a 300 ms-ot. A jelenlegi architektúrában a válaszidő p95 600 ms, ami nem teljesíti a célt. A lehetséges stratégiák:

Stratégia	Költség (fejlesztési nap)	Becsült hasznosságnövekedés (0–100 skála)	Nettó haszon (hasznosság/költség)	Megjegyzés
A: Redis gyorsítótár bevezetése	10	50	5,0	Gyors javulás, de konzisztencia-kockázat.
B: Adatbázis-indexelés optimalizálása	5	30	6,0	Alacsony kockázat, de a hatás kimerülhet.
C: Keresőszolgáltatás horizontális skálázása	15	40	2,67	Jól méretezhető, de infrastruktúra-költséggel jár.

A CBAM alapján az adatbázis-indexelés tűnik a legjobb arányú megoldásnak, de a döntés nem automatizálható: ha a konzisztencia-kockázat kezelhető, a gyorsítótár nagyobb abszolút hasznot hozhat, és ha a hosszú távú növekedést nézzük, a horizontális skálázás lehet a fenntarthatóbb.

A CBAM célja, hogy a döntést transzparens számok és feltételezések mentén hozzuk meg, nem pedig megérzés alapján.

9.3.9 Architektúra-review a gyakorlatban

Az ATAM és a CBAM módszertani keretei akkor működnek jól, ha a review-t megfelelően előkészítjük és vezetjük. A gyakorlatban az architektúra-review három szakaszra bontható:

9.3.10 Előkészítés

- **Cél meghatározása**: miért csináljuk a review-t? Új rendszer indulása előtt, nagyobb refaktorálás előtt, vagy egy adott minőségi probléma (például teljesítményromlás) kivizsgálására?
- **Résztvevők kiválasztása**: a facilitátor mellett az üzleti képviselő, a vezető tervező, a fejlesztők, az üzemeltetők, a biztonsági felelős és a tesztelő. A facilitátor ne legyen közvetlenül érdekelt az adott architektúrában, hogy semleges maradjon.
- **Bemeneti anyagok összegyűjtése**: C4 diagramok, ADR-ek, minőségi forgatókönyvek, SLO/SLI definíciók, deployment leírás. A dokumentációs szerződésünknek megfelelően minden anyag verziózott és hivatkozható.

9.3.11 Lebonyolítás

A review során az ATAM lépéseit követjük, de az időkeretet a rendelkezésre álló időhöz igazítjuk. Egy fél napos review-ban jellemzően a 2–7. lépéseket tudjuk érdemben elvégezni, míg a részletes kockázati elemzés és a CBAM külön munkamenetet igényelhet.

A facilitátor szerepe kritikus: ne engedje, hogy a vita általános „best practice” szintű kijelentésekben ragadjon, mindig kérje a konkrét forgatókönyvre való leképezést. Például ha valaki azt mondja, hogy „a mikroszolgáltatások jobbak, mint a monolit”, a facilitátor kérdezze meg: „Melyik forgatókönyvben, milyen mérőszámban és milyen feltételek mellett?” Ez azért fontos, mert a fejezet elején tárgyalt strukturált értékelés éppen ezt a kontextus nélküli általánosítást akarja elkerülni.

9.3.12 Dokumentálás és követés

A review eredményeit rögzítjük, jellemzően egy jegyzőkönyvben vagy ADR-kiegészítésben. A dokumentumnak tartalmaznia kell:

- a vizsgált forgatókönyvek listáját prioritással,
- az azonosított érzékenységi pontokat és trade-off pontokat,
- a kockázatokat a kapcsolódó forgatókönyvvel és döntéssel,
- a javasolt intézkedéseket felelőssel és határidővel,
- a feladott alternatívákat és a döntés indokát.

A dokumentációban kerüljük a valós hitelesítő adatokat, belső URL-eket és személyes adatokat; a példákban helyőrzőket használunk.

9.3.13 Labor: ATAM-szerű review egy gyakorló rendszeren

A labor célja, hogy a módszert egy kicsi, de valósághű példán keresztül alkalmazd. A gyakorló rendszer legyen egy **„Szállásfoglalási szolgáltatás”**, amely a következő kontextussal rendelkezik.

9.3.14 Kiinduló állapot

A rendszer célja, hogy a felhasználók szálláshelyeket kereshessenek, lefoglalhassanak, és a vendéglátók kezelhessék a szobákat. A C4 context szintjén a következő szereplők és rendszerek jelennek meg:

- **Vendég** (emberi felhasználó): szállást keres, foglal, fizet.
- **Szállásadó** (emberi felhasználó): szobákat hirdet, foglalásokat kezel.
- **Fizetési szolgáltató** (külső rendszer): fizetési tranzakciókat dolgoz fel.
- **Értesítésküldő szolgáltatás** (külső rendszer): e-mail és push értesítéseket küld.

A rendszeren belül három fő konténer van:

- **Webes kliens** (React SPA): a felhasználói felület.
- **Foglalási API** (REST, Node.js vagy hasonló): a keresési, foglalási és vendéglátói funkciók kiszolgálása.
- **Foglalási adatbázis** (relációs, pl. PostgreSQL): a szállások, foglalások és felhasználói adatok tárolása.

A komponensek felelőssége és interfésze a C4 container szinten: a Foglalási API a `/search`, `/bookings`, `/rooms` végpontokat valósítja meg; a keresési kérés a `GET /search?location=X&from=Y&to=Z`, a foglalás a `POST /bookings`. Az adatbázis felelős a tranzakciós integritásért, a külső fizetési szolgáltató a fizetés engedélyezéséért.

9.3.15 Elvárt eredmény

A labor során ATAM-szerű review-t végzünk, amelynek kimenete:

1. Egy **utility tree** legalább 6 forgatókönyvvel, prioritással.
2. Legalább 2 **érzékenységi pont** és 1 **trade-off pont** azonosítása.
3. Legalább 3 **kockázat** rögzítése, mindegyikhez kapcsolt forgatókönyvvel.
4. Egy **CBAM-táblázat** legalább 3 architekturális stratégiával, költséggel és hasznosságnövekedéssel.
5. Egy **döntési napló** bejegyzés, amely rögzíti a kiválasztott stratégiát, a feladott alternatívákat és a következményeket.

Minden mért értéket és becslést külön kell jelölni. A nem mért adatokat (például a hasznosságnövekedést) becslésként kell feltüntetni, a bizonytalanságot pedig szövegesen meg kell adni.

9.3.16 Ellenőrzési szempont

- A forgatókönyvek megfogalmazzák az ingert, a környezetet, a választ és a mérőszámot.
- A trade-off pont legalább két minőségi attribútumot ellentétes irányban befolyásol.
- A CBAM-ban a hasznossági érték becslés, és a bizonytalanság jelölve van.
- A döntési naplóban az alternatívák és a következmények szerepelnek.
- A dokumentum nem tartalmaz éles rendszerre vonatkozó valós adatot, hitelesítő adatot vagy belső URL-t.

9.3.17 Visszaállítási út

A labor szöveges dokumentációt és naplót készít, nincs módosítandó éles rendszer vagy adatbázis. Ha valamilyen eszközt (például diagramkészítőt) használsz, a fájlokat verziózd, és a labor után a helyi munkakönyvtárat törölheted.

9.3.18 Végigvezetett példa

A következőkben egy rövidített ATAM-szerű review-t mutatunk be a fenti rendszerre, hogy a módszer alkalmazását láthatóvá tegyük.

1. Üzleti célok rögzítése

A foglalási szolgáltatás célja, hogy a vendégek gyorsan találjanak szabad szállást, a foglalási folyamat megbízható legyen, és a szállásadók naprakészen kezelhessék a szobáikat. A rendszer csúcsidőben (hétvégi esték) másodpercenként 50–100 keresési kérést kap. A foglalási tranzakciók száma alacsonyabb, de a pénzügyi integritás kritikus.

2. Utility tree (részlet)

Minőségi attribútum	Kategória	Forgatókönyv	Prioritás
Teljesítmény	Keresési válaszidő	Amikor a vendég szállást keres, a rendszer normál terhelés mellett legfeljebb 300 ms alatt visszaadja a találati listát.	H
Rendelkezésre állás	Foglalási tranzakció	Amikor a vendég foglalást kezdeményez, a foglalási szolgáltatás 99,9%-os havi rendelkezésre állással elérhető; a leállás alatt a foglalás nem vész el.	H
Konzisztencia	Kettős foglalás	Ha egy szobát lefoglaltak, az 1 másodpercen belül ne jelenjen meg szabadként más felhasználó számára.	H

Minőségi attribútum	Kategória	Forgatókönyv	Prioritás
Biztonság	Hozzáférés-kezelés	A vendéglátói felülethez csak a szállásadó férhet hozzá; a jogosulatlan hozzáférési kísérletek naplózódnak.	M
Módosíthatóság	Új fizetési mód	Egy új fizetési szolgáltató integrációja legfeljebb 5 fejlesztői nap, és nem igényel módosítást a keresési szolgáltatásban.	M
Üzemeltethetőség	Naplózás	Minden foglalási tranzakció naplózza a kérés azonosítóját, az eredményt és a válaszidőt; ezek alapján 5 percen belül megállapítható a hiba oka.	H

3. Architekturális megközelítések elemzése

A jelenlegi architektúra a Foglalási API-t monolitikus konténerként valósítja meg, amely közvetlenül az adatbázishoz kapcsolódik. A keresési kérések az adatbázis `rooms` tábláját kérik le minden alkalommal. A foglalási tranzakció az adatbázis tranzakciókezelésére támaszkodik.

4. Érzékenységi pontok és trade-off pontok azonosítása

- **Érzékenységi pont 1**: Az adatbázis-kapcsolatok száma a Foglalási API-ban. Ha a kapcsolatok száma alacsony, a keresési teljesítmény csúcsidőben romlik; ha magas, az adatbázis erőforráskorlátba ütközhet.
- **Érzékenységi pont 2**: A keresési lekérdezés indexelése. A `location` és `date` oszlopokon lévő index hiánya jelentősen növeli a válaszidőt.
- **Trade-off pont 1**: A keresési gyorsítótár bevezetése. Javítja a válaszidőt (teljesítmény), de ronthatja a konzisztenciát, mert a gyorsítótárban szereplő szabad szobák állapota késhet az adatbázishoz képest. Emellett növeli az üzemeltetési komplexitást.

5. Kockázatok rögzítése

Kockázat	Kapcsolódó forgatókönyv	Leírás	Kockázatkezelési javaslat
K1: Kettős foglalás	Konzisztencia: kettős foglalás	A gyorsítótár bevezetése esetén a lefoglalt szoba 1 másodpercen belül szabadként jelenhet meg, ami kettős foglaláshoz vezethet.	A foglalási tranzakció során az adatbázisból ellenőrizni kell a szoba aktuális állapotát, és a gyorsítótár frissítése eseményvezérelt legyen.

Kockázat	Kapcsolódó forgatókönyv	Leírás	Kockázatkezelési javaslat
K2: Teljesítményromlás csúcsidőben	Keresési válaszidő	Az adatbázis-index hiánya miatt a keresési válaszidő p95 csúcsidőben meghaladhatja az 1 másodpercet.	Index létrehozása a `location` és `date` oszlopokon, majd terheléses mérés.
K3: Jogosulatlan hozzáférés	Hozzáférés-kezelés	A vendéglátói API nem ellenőrzi a szerepkört, ezért egy vendég is hozzáférhet más szállásadó adataihoz.	Szerepkör-alapú hozzáférés-vezérlés (RBAC) bevezetése a Foglalási API-ban.

6. CBAM-elemzés a keresési válaszidő javítására

A K2 kockázat kezelésére három stratégiát vizsgálunk:

Stratégia	Költség (fejlesztési nap)	Becsült hasznosságnövekedés (0–100)	Nettó haszon	Megjegyzés
A: Adatbázis-indexelés	3	40	13,3	Alacsony kockázat, azonnali hatás.
B: Redis gyorsítótár a kereséshez	8	60	7,5	Nagyobb haszon, de konzisztencia-kockázat.
C: Keresőszolgáltatás különválasztása	12	70	5,8	Hosszú távon skálázható, de magas kezdeti költség.

A CBAM alapján az adatbázis-indexelés a legjobb nettó haszonnal indul, de a hosszú távú növekedéshez a Redis gyorsítótár vagy a külön keresőszolgáltatás is szükséges lehet. A döntés nem automatikus: a csapatnak mérlegelnie kell, hogy a konzisztencia-kockázatot hogyan kezelik, és milyen növekedési pályát várnak.

7. Döntési napló (ADR-kiegészítés)

- **Kontextus**: A foglalási szolgáltatás keresési válaszideje csúcsidőben nem teljesíti a 300 ms-os p95 célt.
- **Döntés**: A csapat az adatbázis-indexelést vezeti be első lépésként, mert alacsony kockázatú és gyors javulást hoz.
- **Alternatívák**: Redis gyorsítótár bevezetése (nagyobb haszon, de konzisztencia-kockázat), külön keresőszolgáltatás (jobb skálázhatóság, de magas kezdeti költség).
- **Következmények**: A válaszidő várhatóan 300 ms alá csökken. A konzisztencia-probléma továbbra is fennáll, de a gyorsítótár nélkül jelenleg nem áll fenn. A következő iterációban újraértékeljük a gyorsítótár bevezetését, ha a keresési terhelés tovább növekszik.
- **Állapot**: Elfogadva, a mérési eredményt követően felülvizsgálandó.

Ez a példa jól mutatja, hogy az ATAM és a CBAM nem önmagában értékes, hanem akkor, ha a kimenete a döntési naplóban és a következő iteráció tervezésében is megjelenik.

9.3.19 Ellenőrzőlista az ATAM/CBAM review-hoz

A fejezet végén a következő ellenőrzőlista segít a strukturált architektúraértékelés minőségének ellenőrzésében:

- Minden minőségi forgatókönyv tartalmazza az ingert, a környezetet, a választ és a mérőszámot?
- A utility tree prioritásait a stakeholderek konszenzusa alapján rögzítettük?
- Az érzékenységi pontokhoz megadtuk, melyik minőségi attribútumra hatnak?
- A trade-off pontokhoz legalább két ellentétes minőségi hatást azonosítottunk?
- A kockázatokhoz kapcsolódik-e konkrét forgatókönyv és döntés?
- A CBAM-ban a hasznossági értékek becslésként vannak-e jelölve, a bizonytalansággal együtt?
- A döntési naplóban szerepelnek-e a feladott alternatívák és a következmények?
- A review dokumentuma mentes-e valós hitelesítő adatoktól, belső URL-ektől és személyes adatoktól?
- Minden nem mért eredményt becslésként tüntettünk fel?

9.3.20 Önellenőrző kérdés

Egy rendszer ATAM-elemzése során a csapat azonosított egy trade-off pontot: a bevezetett gyorsítótár javítja a válaszidőt, de rontja az adatfrissességet. A facilitátor azt kéri, hogy a trade-off pontot kockázattá alakítsák. Milyen lépést kell tenni ahhoz, hogy a trade-off pontból kockázat legyen?

Válasz elvárás: Meg kell határozni egy konkrét minőségi forgatókönyvet, amelynek teljesülését a trade-off pont fenyegeti. Például ha a forgatókönyv szerint a lefoglalt szoba állapotának 1 másodpercen belül frissülnie kell, de a gyorsítótár TTL-je 5 másodperc, akkor a trade-off pont kockázatot jelent. A kockázatot a forgatókönyvhöz kell kötni, és rögzíteni kell a kockázatkezelési javaslatot.

9.3.21 Összegzés és továbbvezetés

Ebben az alfejezetben áttekintettük a strukturált architektúraértékelés célját és módszereit, részletesen tárgyaltuk az ATAM lépéseit, a utility tree-t, az érzékenységi pontokat, a trade-off pontokat és a kockázatokat. A CBAM bevezetésével megmutattuk, hogyan lehet a minőségi javításokat gazdasági szempontból rangsorolni. A laborban egy foglalási szolgáltatáson keresztül alkalmaztuk a módszert, és elkészítettük az ehhez tartozó döntési naplót.

A következő alfejezet (Fitness functions és folyamatos értékelés) az evolúciós architektúra és a fitness functions témáját vezeti be, ahol az itt megismert forgatókönyveket és mérőszámokat folyamatosan, automatizáltan ellenőrizhető függvényekké alakítjuk. Az ATAM és a CBAM eredményei így nem egyszeri review-ként, hanem a folyamatos architektúra-értékelés alapjaként hasznosulnak.

9.4 Fitness functions és folyamatos értékelés

Az előző alfejezetben az ATAM és CBAM értékelési módszereket tárgyaltuk. Ezek erőssége, hogy egyszeri, mély elemzést adnak, de gyengeségük, hogy nem követik a rendszer napi változásait. Egy jól megtervezett architektúra is folyamatosan romolhat, ha senki nem figyeli a réteghatárok, a válaszidők vagy a biztonsági fejlécek alakulását. Itt lép be az illeszkedési függvény (fitness function): nem egyszeri bírálat, hanem a folyamatos értékelés mechanizmusa.

A fogalom az evolúciós architektúra irodalmából származik. A biológiai evolúcióban a fitness függvény azt méri, hogy egy egyed mennyire alkalmazkodott a környezethez; a szoftverarchitektúrában pedig egy automatizált ellenőrzés, amely egy architekturális minőségi jellemzőt vizsgál folyamatosan, és visszajelzést ad. A lényeg: nem az emberi vélemény helyettesítése, hanem a döntések végrehajtásának automatizált őrzése.

Egy fitness function tehát:

- egy adott minőségi attribútumhoz kapcsolódik,
- mérhető bemenetből dolgozik,
- explicit küszöböt tartalmaz,
- a CI/CD vagy üzemeltetési folyamatba integrálható,
- eredménye alapján a csapat cselekedni tud.

Fontos különbség a hagyományos teszthez képest: a funkcionális teszt azt kérdezi, hogy a rendszer helyesen válaszol-e egy adott bemenetre; a fitness function azt, hogy a rendszer megőrzi-e a kívánt architekturális tulajdonságot, például a modulok közötti megengedett függőségeket, a p95 késleltetés felső határát vagy a kötelező biztonsági fejléceket. A funkcionális teszt a viselkedést, a fitness function a szerkezetet és a minőségi jellemzőket védi.

Fitness function típusok és gyakori példák. A fitness functionök két nagy csoportra oszthatók: statikus és dinamikus ellenőrzésekre. A statikus fitness function a forráskódot, a konfigurációt vagy a build-artefaktumokat elemzi, anélkül hogy a rendszert futtatná. A dinamikus fitness function futás közbeni adatokat – metrikákat, naplókat, trace-eket – vagy terheléses tesztek eredményeit használ.

Statikus példák:

- Réteghatárok: a `web` modul nem importálhat a `repository` modulból.
- Ciklikus függőségek: a csomaggráfban nincs kör.
- Architekturális szabályok: például a kontrollerek nem tartalmazhatnak üzleti logikát, csak delegálást.
- API szerződés: az OpenAPI specifikáció diff nem törölhet meglévő mezőt, és nem törheti meg a visszafelé kompatibilitást.
- Biztonsági konfiguráció: minden HTTP válasz tartalmazza a Content-Security-Policy fejlécet; a Dockerfile nem futtathat root felhasználóval.
- Függőségek licence és sebezhetősége: az SBOM-ban nincs kritikus CVE.

Dinamikus példák:

- Teljesítmény: a `checkout` szolgáltatás p95 késleltetése 300 ms alatt marad a terheléses tesztben.
- Skálázhatóság: adott terhelés mellett a replikák száma eléri a cél értéket, miközben a CPU-kihasználás 80% alatt marad.
- Rendelkezésre állás: az elmúlt 30 napban a szolgáltatás havi elérhetősége legalább 99,9%.
- Hibatűrés: egy függőség mesterséges meghibásítása esetén a rendszer 2 másodpercen belül visszavált egy tartalék útvonalra, és a hibák aránya 1% alatt marad.

A két csoport nem zárja ki egymást: egy jó fitness function stratégia általában mindkettőt használja. A statikus ellenőrzések olcsók és korán futnak; a dinamikusak drágábbak, de a valós viselkedést mérik.

9.4.1 Mérőszám, küszöb és riasztás

A fitness function akkor működik jól, ha három összetevő világos: mit mérünk (mérőszám), mi számít elfogadhatónak (küszöb), és mi történik, ha a határt átlépjük (riasztás). Ezt a hármast a minőségi követelményekből kell levezetni, nem pedig a rendelkezésre álló eszközökből visszafelé.

A mérőszám (SLI) legyen jól definiált, mértékegységgel és mérési ablakkal. Például ne azt mondjuk, hogy „a válaszidő jó”, hanem hogy „a `checkout` szolgáltatás p95 késleltetése az elmúlt 5 percben, másodpercben”. A mérési ablak azért fontos, mert a terhelés ingadozik, és egyetlen pillanatnyi értékből nem lehet stabil döntést hozni. Az ablak lehet commit, build, óra, nap vagy forgalmi periódus.

A küszöb az az érték, amely fölött (vagy alatt) a jellemző már nem felel meg az architekturális követelménynek. Ideális esetben az SLO-ból származik. Például ha az SLO azt mondja, hogy a hónap 99,9%-ában a p95 késleltetés 300 ms alatt marad, akkor a fitness function küszöbe lehet egy rövidebb ablakra vetített, óvatosabb határ, például 250 ms, hogy a hosszú távú cél még a rövid távú ingadozások mellett is tartható legyen. A küszöb megválasztása trade-off: a túl szigorú határ hamis pozitív riasztásokat generál, a túl laza határ pedig nem védi meg a minőséget. A gyakorlatban érdemes a küszöböt a mért adatok alapján finomítani, és a változtatást ugyanúgy ADR-ben rögzíteni, mint bármely architekturális döntést.

A küszöb lehet abszolút vagy relatív. Abszolút küszöb például: „a ciklikus függőségek száma nulla”. Ez a leggyakoribb a szerkezeti szabályoknál. Relatív küszöb: „a csomagok közötti függőségek legalább 80%-a a réteghatárok szerint befelé mutat”. Ilyenkor a rendszer mérete és komplexitása miatt nem várható el a nulla, de a tendencia igen. A trend alapú küszöb azt vizsgálja, hogy a mérőszám javul-e vagy romlik az időben, például „a technikai adósság aránya negyedévente csökkenjen”. A trend alapú fitness function hasznos, de óvatosan: a trend lassan változik, és a rövid távú visszajelzéshez nem elég.

A küszöbök meghatározásakor hasznos statisztikai megfontolásokat tenni. Ha a mérés zajos – például a válaszidők percentilisei –, akkor a küszöböt ne egyetlen minta alapján állítsd be, hanem több mérés átlagából vagy eloszlásából. Például a p95 késleltetésre ne egyetlen terheléses teszt futás eredményét használd, hanem legalább három futás átlagát vagy a legrosszabb stabil értéket. A mérési bizonytalanságot a dokumentációban fel kell tüntetni, ahogy a könyv korábbi fejezetei is előírják.

A riasztás a küszöb átlépésekor keletkező jelzés. Nem minden küszöbsértés igényel azonnali emberi beavatkozást; a riasztás célja, hogy a megfelelő személy vagy automatizmus a megfelelő időben cselekedjen. A riasztási fáradtság valós veszély: ha a csapat naponta tucatnyi irreleváns riasztást kap, akkor előbb-utóbb figyelmen kívül hagyja az összeset, és a fitness function elveszíti az értékét. Ezért érdemes a riasztási szabályt a küszöbtől elkülöníteni, és hiszterézist alkalmazni: a riasztás akkor aktiválódik, ha a mérőszám három egymást követő mérési ablakban megsérti a küszöböt, és csak akkor szűnik meg, ha két egymást követő ablakban visszatér a határ alá. Így a rövid kiugrások nem generálnak zajt.

A riasztás formája lehet CI/CD státusz (piros build), jegy automatikus nyitása, értesítés chat-csatornán vagy automatikus rollback. A választás attól függ, milyen gyorsan kell beavatkozni, és ki a tulajdonos. A CI-be integrált fitness function esetén a legegyszerűbb, ha a build megszakad, mert ez megakadályozza a rossz kód bekerülését. Üzemeltetési metrikáknál viszont a riasztás

egy ügyeletes mérnökhöz kerül, aki a runbook szerint jár el. A riasztási csatornát és a felelőst a fitness function dokumentációjában rögzíteni kell.

A fitness functionök és a minőségi forgatókönyvek kapcsolata itt válik kézzelfoghatóvá. A quality attribute scenario megadja az ingert, a környezetet, a választ és a mérőszámot. A fitness function ezt a forgatókönyvet ülteti át automatizált ellenőrzéssé. Például:

Forgatókönyv: Egy fejlesztő új kódot tölt fel (inger) a CI környezetben (környezet). A rendszer a modulgráfban nem tartalmaz `web -> repository` függőséget (válasz). A tiltott függőségek száma 0 (mérőszám).

Fitness function: A CI minden commitnál lefuttatja az architekturális szabályt, megszámolja a tiltott függőségeket, és ha a szám nagyobb nullánál, pirosra állítja a buildet.

9.4.2 Fitness function tervezése lépésről lépésre

A fitness function tervezése nem technológiai feladat elsősorban, hanem követelmény- és kockázatelemzés. Az alábbi lépések segítenek abban, hogy olyan ellenőrzést hozz létre, amely valóban a fontos architekturális jellemzőt védi, és nem csak egy mérhető, de irreleváns dolgot.

1. Válassz ki egy minőségi jellemzőt. Ne azzal kezdd, hogy milyen eszköz áll rendelkezésre, hanem azzal, hogy melyik architekturális tulajdonság romlása okoz leggyakrabban vagy legdrágábban problémát. A réteghatárok megsértése, a ciklikus függőségek, a válaszidő növekedése, a biztonsági fejlécek hiánya, a túl nagy csomagméret vagy a túl sok adatbázis-kapcsolat mind gyakori jelöltek. A korábbi minőségi forgatókönyvek és az ATAM eredményei jó kiindulópontot adnak.

2. Fogalmazd meg quality attribute scenario formában. Az inger, a környezet, a válasz és a mérőszám egyértelművé teszi, mit kell mérni. A mérőszám legyen számszerű, mértékegységgel és mérési ablakkal. Például: „Egy új commit beérkezésekor (inger) a CI buildben (környezet) a `web` modul nem importálhat a `repository` modulból (válasz); a tiltott importok száma 0 (mérőszám).”

3. Azonosítsd az adatforrást. A méréshez szükséges adat lehet statikus (forráskód, konfiguráció, Dockerfile) vagy dinamikus (futás közbeni metrika, log, trace). A statikus adatok előnye, hogy a CI korai szakaszában, gyorsan és determinisztikusan mérhetők. A dinamikus adatok valós terhelést tükröznek, de a mérés zajosabb és gyakran csak az üzemeltetési környezetben érhető el. Sok fitness function hibrid: a buildben futtatott teljesítményteszt vagy biztonsági szkennelés dinamikus adatot használ, de a CI-ben rögzített körülmények között.

4. Válassz eszközt és írd meg az ellenőrzést. A választott eszköz függ a technológiai stacktől és a mérés típusától. Néhány példa (a verziókat a használat előtt ellenőrizni kell): Javában az ArchUnit, .NET-ben a NetArchTest, Pythonban az import-linter vagy egyedi AST-alapú szkript, statikus elemzésre a Semgrep, kódminőségre a SonarQube, teljesítménytesztre a k6 vagy JMeter, üzemeltetési metrikákhoz az OpenTelemetry és Prometheus. A példákban nem a konkrét eszköz a lényeg, hanem az, hogy az ellenőrzés automatizálható, determinisztikus és a kódbázissal együtt verziózható legyen.

Az ellenőrzés kódját ugyanolyan színvonalon kell kezelni, mint a termékkódot: érthető, tesztelhető, dokumentált. Egy fitness function akkor jó, ha a jövőbeli olvasó megérti belőle, mit mér és miért pont azt a küszöböt használja. Ezért a kód mellett vagy a kódban szerepeljen hivatkozás az ADR-re.

5. Definiáld a küszöböt és a riasztást. A küszöböt a minőségi követelményből vagy az SLO-ból vezessük le, és rögzítsük mértékegységgel, mérési ablakkal. A riasztási stratégia

legyen explicit: hány egymást követő sértés után értesítsen, kinek, milyen csatornán, és milyen visszaállítási feltétellel.

6. Integráld a CI/CD pipeline-ba és a visszacsatolási hurokba. A fitness function akkor fejti ki hatását, ha a döntési ponton fut, például minden pull requestnél vagy nightly buildnél. A futási gyakoriságot a visszajelzési igény és a költség egyensúlya határozza meg. A gyors, olcsó ellenőrzéseket minden commitnál érdemes futtatni; a drágább teljesítmény- vagy biztonsági teszteket ritkábban, de akkor is automatikusan, például éjszaka. A fontos, hogy a fitness function eredménye valós következménnyel járjon: a build piros lesz, a kiadás blokkolódik, vagy jegy keletkezik.

7. Dokumentáld ADR-ben. Az ADR rögzíti a kontextust, a döntést, az alternatívákat és a következményeket. A fitness function esetén az ADR tartalmazza a minőségi követelményt, a mérőszámot, a küszöböt, a riasztási szabályt, az eszközt és a várható hatást. Ez a dokumentáció akkor válik értékessé, amikor valaki hat hónappal később megkérdőjelezi, miért van egy látszólag felesleges ellenőrzés a pipeline-ban.

Vegyünk egy példát a réteghatárok védelmére. A csapat úgy dönt, hogy a `web` réteg nem függhet a `repository` rétegtől. A fitness function pszeudokódja:

```
def check_layer_boundaries(module_graph):
    violations = []
    for edge in module_graph.edges:
        if edge.source == "web" and edge.target == "repository":
            violations.append(edge)
    return violations
```

A CI lépés ezt hívja meg, és ha a `violations` lista nem üres, a build megszakad. A küszöb `0`, a mérési ablak egy commit. A riasztás maga a piros build, ami megakadályozza a merge-t. A kód természetesen a valós eszközben ennél részletesebb, de a logika ugyanez.

9.4.3 Tipikus hibák és hamis biztonság

A fitness function akkor hasznos, ha valódi jelzést ad, és a csapat bízik benne. Számos módon el lehet rontani, és a hibák többsége nem technikai, hanem szemléletbeli vagy szervezési.

A mérőszám nem a célt méri. Gyakori, hogy egy könnyen mérhető proxy-t választunk egy nehezen mérhető minőségi jellemző helyett. Például a karbantarthatóságot a kódsorok számával mérjük, holott a kódsorok száma önmagában nem mond semmit a modularitásról vagy a kohézióról. A proxy lehet korrelál, de lehet, hogy éppen ellenkezőleg: egy refaktorálás növelheti a sorok számát, miközben javítja a karbantarthatóságot. Mindig kérdezd meg: ha ez a mérőszám zöld, akkor a mögöttes minőségi jellemző valóban teljesül-e? Ha nem, akkor a fitness function hamis biztonságot ad.

Túl sok fitness function, túl sok riasztás. A kezdeti lelkesedésben a csapat minden lehetséges szabályt bevezet, a pipeline tele lesz ellenőrzésekkel, és a riasztások özöne miatt senki nem tudja, mi a fontos. A fitness functionök számát a kockázat és a hatás alapján kell priorizálni. Egy jó ökölszabály: kezdd kevés, jól megválasztott ellenőrzéssel, és csak akkor bővítsd, ha a meglévők stabilan működnek és valódi értéket adnak.

Hamis pozitívok és flaky tesztek. Ha a fitness function nem determinisztikus – például egy időzítés-alapú teljesítményteszt túl szoros küszöbbel –, akkor a build véletlenszerűen piros lesz, a csapat elveszíti a bizalmát, és kikapcsolja az ellenőrzést. A megoldás a mérési környezet stabilizálása, a küszöb statisztikai megalapozása (pl. több mérés átlaga vagy percentilis), és a riasztási szabály hiszterézise.

Hamis negatívok. Az ellenkező hiba, amikor a fitness function zöld, de a rendszer mégis megsérti az architekturális elvet. Ennek oka lehet a mérőszám rossz lefedettsége, a mérési pont hiánya vagy a szabály elavulása. Például ha a réteghatárok ellenőrzése csak a `web -> repository` importot nézi, de a `web -> service -> repository` útvonalat nem, akkor a szabály kijátszható. A fitness function tervezésekor fel kell tenni a kérdést: milyen módon lehet a szabályt úgy megsérteni, hogy a mérés ne jelezze? A fenyegetésmodellezéshez hasonló gondolkodás itt is hasznos.

A zöld jel hamis biztonsága. A legnagyobb kockázat talán az, amikor a csapat a fitness function zöld állapotát az architektúra egészségének bizonyítékaként értelmezi. A fitness function csak azt igazolja, hogy az adott mérés az adott küszöbön belül van. Nem igazolja, hogy az architektúra jó, hogy a követelmények teljesülnek, vagy hogy a jövőben nincs kockázat. Az architekturális gondolkodás, a review-k, a viták és az emberi ítélet nem automatizálhatók. A fitness function ezeket egészíti ki, nem helyettesíti.

A kontextus figyelmen kívül hagyása. Egy küszöb, amely egy kontextusban értelmes, egy másikban téves lehet. Például a 300 ms-os p95 késleltetés egy interaktív webes felületnél jó cél, de egy éjszakai kötegelt feldolgozásnál értelmetlen. A fitness function tervezésekor mindig rögzíteni kell a környezetet, a terhelést és a mérési ablakot, és a küszöböt ezekhez kell igazítani. A korábban bevezetett quality attribute scenario pontosan ezt a kontextust adja meg.

A mérés karbantartásának hiánya. A fitness function is kód, és a kód elavul. A szabályok, a küszöbök, az eszközök verziói mind változnak. Ha senki nem felelős a fitness functionök karbantartásáért, akkor egy idő után elavulnak, hamis jeleket adnak, vagy éppen semmit sem mérnek. A következő szakaszban ezt a karbantartási életciklust nézzük meg.

9.4.4 A mérés karbantartása

A fitness functionök nem egyszeri projektek, hanem a kódbázis és a CI/CD folyamat részei. Mint minden szoftverkomponenst, őket is tervezni, implementálni, tesztelni, kiadni és idővel nyugdíjazni kell. A karbantartás célja, hogy a mérés megbízható maradjon, és ne váljon hamis biztonság forrásává.

Tulajdonos és dokumentáció. Minden fitness functionnek legyen egyértelmű tulajdonosa, aki felel a működéséért és a felülvizsgálatáért. A dokumentáció tartalmazza a minőségi követelményt, a mérőszámot, a küszöböt, a riasztási szabályt, az eszközt, a futtatási gyakoriságot és a kapcsolódó ADR-t. A tulajdonos akkor is elérhető, ha a csapat összetétele változik.

Verziózás. A fitness function kódja a kódbázissal együtt verziózott, így visszakereshető, mikor és miért változott. Ha a küszöböt módosítod, az ugyanolyan döntés, mint bármely architekturális változtatás: ADR-ben kell rögzíteni. A verziózás azt is lehetővé teszi, hogy egy hibás módosítást visszaállíts.

Felülvizsgálati ciklus. Rendszeres időközönként – például negyedévente – a csapat vizsgálja felül a fitness functionök halmazát. Kérdések: Melyik szabály generált riasztást? Voltak hamis pozitívok vagy hamis negatívok? A küszöb még a jelenlegi követelményeket tükrözi? Van olyan szabály, amely sosem jelez, és lehet, hogy már nem is mér semmit? Ez a felülvizsgálat megakadályozza, hogy a fitness functionök csendben elavuljanak.

Finomítás és nyugdíjazás. A mérési adatok alapján a küszöböt és a riasztási szabályt finomítani lehet. Ha egy szabály mindig zöld, akkor vagy túl laza, vagy a mögöttes kockázat már nem áll fenn. Ha túl gyakran riaszt hamis pozitív miatt, akkor a mérést vagy a küszöböt kell javítani. Ha egy architekturális döntés megváltozik – például a réteghatárok módosulnak –, a

hozzá tartozó fitness functiont is aktualizálni kell, vagy ha már nincs szerepe, nyugdíjazni. A fitness functionök halmaza tehát él, és a rendszerrel együtt fejlődik.

A hamis biztonság elleni folyamatos éberség. A legjobb karbantartás sem teszi a fitness functiont az emberi ítélet helyettesítőjévé. A mérés csak annyit ér, amennyire a csapat értelmezi és használja. Ezért a karbantartási ciklus része az is, hogy a csapat időnként megvitatja: milyen architekturális kockázatokat nem fed le a jelenlegi mérés, és milyen új fitness functionre lenne szükség.

9.4.5 Labor: Fitness function tervezése a CI-hez

Ez a labor arra szolgál, hogy a fenti elméletet egy kis, ellenőrizhető példán alkalmazd. A cél nem egy tökéletes, ipari szintű ellenőrzőrendszer, hanem a gondolkodásmód és a dokumentációs fegyelem gyakorlása.

Kiinduló állapot. Válassz egy meglévő mintaprojektet, vagy hozz létre egy minimális alkalmazást, amelyben legalább két modul vagy réteg van, és közöttük architekturális szabályt lehet értelmezni. Például egy egyszerű webshop backend, amelynek `web`, `service` és `repository` moduljai vannak. Ha nincs ilyen, a laborhoz használhatsz egy kitalált, de letisztázott szerkezetet, ahol a szabály jól tesztelhető.

Feladat. Tervezz és implementálj egy fitness functiont, amely egy kiválasztott architekturális minőségi jellemzőt véd. A folyamat lépései:

1. Válassz egy minőségi jellemzőt, és indokold meg, miért fontos.
2. Fogalmazd meg quality attribute scenario formában: inger, környezet, válasz, mérőszám.
3. Határozd meg a fitness function bemenetét, mérőszámát, küszöbét és riasztási stratégiáját.
4. Implementáld a fitness functiont a választott technológiában. Ez lehet egy ArchUnit teszt Java projektben, egy Python AST-alapú import-ellenőrzés, egy shell script, amely a build kimenetét elemzi, vagy bármely más, a környezetedben elérhető eszköz. A lényeg, hogy reprodukálható és automatikusan futtatható legyen.
5. Futtasd a fitness functiont lokálisan vagy a CI pipeline-ban, és rögzítsd a kimenetet. A kimenetben külön válaszd el a mért értéket és a várt értéket.
6. Készíts egy reprodukálható dokumentumot (Markdown, Jupyter notebook vagy hasonló), amely a következőket tartalmazza:
   - a kiinduló állapotot és a környezetet,
   - a kiválasztott minőségi jellemzőt és a forgatókönyvet,
   - a fitness function kódját vagy konfigurációját,
   - a futtatás lépéseit,
   - a mért eredményt és a várt eredményt egyértelműen elkülönítve,
   - a döntési naplót (ADR formában),
   - az ellenőrzési szempontokat,
   - a visszaállítási utat (hogyan lehet a fitness functiont eltávolítani vagy a küszöböt módosítani).
7. A dokumentum alapján egy másik mérnöknek képesnek kell lennie a mérés reprodukálására és a döntés megértésére.

Példa megoldás vázlat. Tegyük fel, hogy a kiválasztott szabály: „A `web` modul nem importálhat a `repository` modulból.” Egy egyszerű Python szkript, amely a fájlok AST-jét elemzi, így nézhet ki (a példa szemléltető, nem éles rendszerre):

```python
import ast
import sys

# Tiltott függőségek: forrás modul -> tiltott cél modul(ok)
FORBIDDEN = {
    "web": ["repository"],
    "service": [],  # itt nincs tiltott cél
    "repository": []
}

def check_file(path):
    violations = []
    with open(path, encoding="utf-8") as f:
        tree = ast.parse(f.read(), filename=path)
    for node in ast.walk(tree):
        if isinstance(node, ast.ImportFrom) and node.module:
            imported_module = node.module.split(".")[0]
            # A forrás modul meghatározása a fájl útvonala alapján
            source_module = path.split("/")[0]  # egyszerűsítés
            if imported_module in FORBIDDEN.get(source_module, []):
                violations.append((path, node.module))
    return violations

if __name__ == "__main__":
    files = sys.argv[1:]
    all_violations = []
    for file in files:
        all_violations.extend(check_file(file))
    measured = len(all_violations)
    expected = 0
    print(f"Mért érték: {measured} szabálysértés")
    print(f"Várt érték: {expected} szabálysértés")
    if all_violations:
        for path, module in all_violations:
            print(f"  {path} -> {module}")
        sys.exit(1)
```

Ez a szkript a CI-ben így futhat: minden commitnál a pipeline összegyűjti a Python fájlokat, lefuttatja a szkriptet, és ha a kilépési kód 1, a build megszakad. A mérési ablak egy commit, a küszöb 0, a riasztás a piros build. A dokumentumban rögzítsd a környezetet (pl. Python verzió, operációs rendszer, CI eszköz), a futtatás időpontját és az eredményt.

Egy másik példa: OpenAPI szerződés kompatibilitás. Ha a projekt rendelkezik API-szerződéssel, választhatod azt a szabályt, hogy az OpenAPI specifikáció diff nem törölhet meglévő mezőt. Ehhez használhatsz egy nyílt forráskódú diff eszközt vagy egy egyszerű szkriptet, amely összehasonlítja a sémákat. A mérőszám a törölt mezők száma, a küszöb 0, a riasztás a build megszakítása. Ez a példa jól mutatja, hogy a fitness function nem csak kódstruktúrát, hanem API-kompatibilitást is védhet.

Ellenőrzési szempontok. A labor értékelésekor a következőket vizsgáld:

- ☐ A fitness function egyértelműen egy architekturális minőségi jellemzőt mér.
- ☐ A minőségi forgatókönyv tartalmazza az ingert, környezetet, választ és mérőszámot.
- ☐ A mérőszám mértékegységgel és mérési ablakkal rendelkezik.
- ☐ A küszöb explicit és indokolt, lehetőleg SLO-ból vagy minőségi követelményből levezetve.
- ☐ A riasztási stratégia leírja, mi történik küszöbsértéskor, és kinek kell cselekednie.
- ☐ A mért és várt eredmény a kimenetben és a dokumentumban külön van választva.
- ☐ A dokumentum tartalmazza a környezetet, a konfigurációt, az időpontot és a visszaállítási utat.
- ☐ A fitness function reprodukálható, és a kód verziózott.

☐ A döntési napló (ADR) rögzíti a kontextust, döntést, alternatívákat és következményeket.

Visszaállítási út. Ha a fitness function tévesen riaszt, vagy a szabály már nem érvényes, a pipeline-ból való eltávolítás vagy a küszöb módosítása szükséges. A visszaállítás lépéseit a dokumentumban rögzíteni kell, például: a szkript törlése a CI konfigurációból, a küszöb megváltoztatása a kódban, vagy a szabály kikapcsolása egy feature flaggel. A visszaállítás után a döntést ADR-ben frissíteni kell.

9.4.6 Önellenőrző kérdések és ellenőrzőlista

A fejezet végén ellenőrizd a megértésedet az alábbi kérdésekkel:

1. Miben különbözik egy architekturális fitness function egy funkcionális egységteszttől? Adj példát mindkettőre.
2. Hogyan vezet le egy fitness functiont egy minőségi forgatókönyvből? Mutasd be a lépéseket egy konkrét példán.
3. Mi a különbség a mérőszám, a küszöb és a riasztás között? Miért érdemes hiszterézist alkalmazni a riasztásnál?
4. Sorolj fel három tipikus hibát, amely a fitness functionök hamis biztonságához vezethet, és mindegyikre adj megelőző intézkedést.
5. Miért nem elegendő a fitness functionök zöld állapota az architektúra egészségének igazolására?

Ellenőrzőlista a fitness functionök bevezetéséhez:

☐ A fitness function egy fontos, romlásra hajlamos minőségi jellemzőt véd.
☐ A minőségi követelmény quality attribute scenario formában megfogalmazott.
☐ A mérőszám jól definiált, mértékegységgel és mérési ablakkal.
☐ A küszöb az SLO-ból vagy elfogadott határból származik, és ADR-ben rögzített.
☐ A riasztás csak akkor jelez, ha cselekvés szükséges, és a felelős ismert.
☐ A fitness function determinisztikus, vagy a nem determinizmust statisztikai módon kezelik.
☐ A fitness function kódja a kódbázissal együtt verziózott és dokumentált.
☐ A hamis pozitívok és negatívok kezelésére van terv.
☐ A fitness function rendszeres felülvizsgálat alá esik, és van tulajdonosa.

9.4.7 Összefoglalás és átvezetés

A fitness function az architekturális minőség folyamatos értékelésének alapvető eszköze. Nem egyszeri review, hanem a CI/CD-be ágyazott, automatizált visszajelzés, amely a minőségi követelményeket mérhető, ellenőrizhető formába önti. A sikerhez azonban nem elég a technikai megvalósítás: a mérőszám, a küszöb és a riasztás gondos megválasztása, a hamis biztonság elkerülése és a folyamatos karbantartás egyaránt szükséges. A következő alfejezetben a tervezési folyamatot vizsgáljuk, ahol a fitness functionök a döntési alternatívák validálásában és a visszacsatolási hurkok kialakításában kapnak szerepet.

10 Tervezési folyamat és döntési keretek

10.1 A tervezési folyamat: feltárás, alternatívák és döntés

A tervezési folyamat nem egyszeri aktus, hanem a probléma feltárásától a lezárt döntésig tartó, ismétlődő tevékenység. Az előző fejezetekben megismertük a minőségi követelmények megfogalmazását (quality attribute scenarios), azok mérését (SLO, SLI, error budget) és az architektúra-értékelés alapjait (ATAM, CBAM, fitness functions). Ebben a fejezetben arra összpontosítunk, hogyan lehet ezeket az eszközöket egy tudatos tervezési folyamatban összekapcsolni: hogyan jutunk el a problémától a döntésig anélkül, hogy a folyamat öncélúvá válna. A hangsúly a feltáráson, az alternatívák kezelésén és a döntési pontok fegyelmezett dokumentálásán van. A következő egységek (PoC, MVP, spike, döntési mátrixok) erre az alapra épülnek, ezért itt elsősorban a keretrendszert és a gondolkodásmódot rögzítjük.

10.1.1 1. A tervezési folyamat természete

A szoftverarchitektúra tervezése gyakran lineárisnak ábrázolható, a valóságban azonban iteratív: a döntéseket új ismeretek fényében felül kell vizsgálni. A folyamat leegyszerűsítve a következő lépésekből áll:

1. **Probléma és kontextus feltárása** – kik az érintettek, mi a valódi cél, milyen feltételek között kell a rendszernek működnie.
2. **Célok és kényszerek rögzítése** – a célok hierarchiájának tisztázása, a kényszerek explicit megfogalmazása.
3. **Alternatívák generálása** – több lehetséges architektúrális irány feltérképezése anélkül, hogy azonnal választanánk.
4. **Értékelés és összehasonlítás** – a minőségi attribútumok, kockázatok és trade-offok alapján.
5. **Döntés és dokumentálás** – az ADR-ben rögzített döntés indoklással.
6. **Visszacsatolás** – a döntés utáni validálás, mérés, esetleges korrekció.

Ezt a folyamatot egy diagrams-as-code eszközzel, például Mermaid flow diagrammal is megjeleníthetjük:

```
flowchart LR
    A[Probléma feltárása] --> B[Célok és kényszerek]
    B --> C[Alternatívák feltárása]
    C --> D[Értékelés]
    D --> E[Döntés]
    E --> F[Visszacsatolás]
    F -->|Új ismeret| B
```

Cím: A tervezési folyamat fő lépései.
Jelmagyarázat: a nyilak irányított folyamatot jelölnek; a szaggatott visszacsatolás az iterációt jelzi.
Feltételjegyzet: a folyamat a gyakorlatban nem mindig követ ilyen tiszta sorrendet; az egyes lépések átfedhetnek, és a visszacsatolás bármely ponton elindulhat.

A folyamat központi gondolata: a tervezés nem azért van, hogy a lehető legtöbb dokumentumot hozzuk létre, hanem hogy csökkentsük a bizonytalanságot, és a lehető legjobb döntést hozzuk az adott időpillanatban rendelkezésre álló információk alapján.

10.1.2 2. A probléma és a kontextus feltárása

A legtöbb hibás architektúrális döntés visszavezethető arra, hogy a problémát pontatlanul fogalmaztuk meg. A feltárás során a következő kérdésekre kell választ találni:

- **Mi a rendszer célja?** Mi az üzleti vagy szervezeti érték, amit a rendszer létrehoz?
- **Kik a stakeholderek?** Kik hoznak döntést, kik érzik a következményeit? (Például fejlesztők, üzemeltetők, ügyfélszolgálat, felhasználók, biztonsági csapat, jogi osztály.)
- **Milyen funkcionális és nem funkcionális követelmények ismertek?** Melyek a biztosak, és melyek bizonytalanok?
- **Milyen kényszerek adottak?** Technológiai, szervezeti, jogi, költségvetési vagy időbeli korlátok.
- **Milyen a meglévő rendszerkörnyezet?** Milyen rendszerekkel kell integrálódni, milyen adatokkal dolgozunk?

A feltárás nem egyszeri művelet: a tervezés során folyamatosan mélyül a megértés. A stakeholderek eltérő nézőpontjait strukturált módon, például C4 context diagrammal vagy interjúkkal lehet összegyűjteni. A rendszerhatárt minden diagramon jelölni kell, a külső rendszereket és szerepköröket a context szinten fel kell tüntetni. Ez segít elkerülni azt a hibát, hogy a tervező a saját feltételezéseit tényként kezelje.

A feltárás eredményeként rendelkeznünk kell egy **kontextusleírással**, amely tartalmazza a stakeholdereket, a fő funkcionális célokat, a minőségi követelményeket (lehetőleg scenario formában), a kényszereket és a nyitott kérdéseket. Ezt a dokumentumot verziózni kell, és a későbbi döntések hivatkozhatnak rá.

10.1.3 3. Cél, kényszer, alternatíva

A tervezési tér három pillére a **cél**, a **kényszer** és az **alternatíva**. Ezeket gyakran összekeverik, pedig érdemes őket világosan megkülönböztetni.

10.1.4 Célok

A célok általában hierarchikusak: az üzleti célból levezethetők a rendszerrel szembeni minőségi elvárások. Például egy e-kereskedelmi rendszer esetén az üzleti cél „növelni a konverziót” átfordulhat olyan minőségi követelményre, hogy a katalógusoldal betöltési ideje a 95. percentilisben legfeljebb 200 ms legyen 500 egyidejű felhasználó mellett. Ez már mérhető, és quality attribute scenario formában rögzíthető:

> **Inger:** egy felhasználó megnyitja a katalógusoldalt.
> **Környezet:** normál üzem, 500 párhuzamos felhasználó, átlagos hálózati késleltetés 50 ms.
> **Válasz:** az oldal teljes tartalma megjelenik, a képek kivételével.
> **Mérőszám:** a válaszidő 95. percentilise 200 ms, a hibaarány 0,1% 10 perces ablakban.

A célokat fontossági sorrendbe kell állítani, mert az alternatívák értékelésekor nem mindegyik egyformán lényeges. A *trade-off analízis* itt kapcsolódik be: ha két cél ütközik (például a teljesítmény és a biztonsági ellenőrzések mélysége), akkor explicit döntést kell hozni a prioritásról.

10.1.5 Kényszerek

A kényszerek olyan feltételek, amelyeket a döntésnél kötelezően figyelembe kell venni. Nem minden kényszer rossz: éppen ellenkezőleg, a kényszerek csökkentik a tervezési teret, és ezzel megkönnyítik a döntést. Tipikus kényszerek:

- **Technológiai:** például a meglévő rendszerek adatbázisa, a szervezet által támogatott konténerplatform, a csapat ismeretei.
- **Jogi és megfelelőségi:** GDPR, ágazati szabályozás (pl. NIS2, DORA), auditkövetelmények.
- **Költségvetési:** rendelkezésre álló infrastruktúra-keret, licencdíjak.
- **Időbeli:** fix határidő, például egy jogszabály hatálybalépése.
- **Szervezeti:** a csapatok felépítése, a döntési jogkörök, a Conway-törvényből fakadó korlátok.

A kényszereket ugyanúgy dokumentálni kell, mint a célokat. Egy kényszer megszegése általában azonnali kockázatot jelent, ezért a döntési folyamat során minden alternatívánál ellenőrizni kell, hogy teljesíti-e az összes kényszert.

10.1.6 Alternatívák

Az alternatívák feltárása a divergens gondolkodás szakasza. Itt nem az a cél, hogy azonnal megtaláljuk a „legjobb” megoldást, hanem hogy minél több lehetséges irányt azonosítsunk. Az alternatívák származhatnak:

- ismert architektúrális mintákból (rétegek, microservices, eseményvezérelt, CQRS, stb.),
- korábbi projektek tapasztalataiból,
- referenciaarchitektúrákból (felhőszolgáltatók well-architected keretei),
- a csapat ötleteiből.

Minden alternatívát röviden le kell írni, megjelölve a fő komponenseket, a kommunikációs mintákat és a feltételezett működést. A dokumentálásnál nem szükséges teljes részletesség, de a későbbi értékeléshez elég információnak kell rendelkezésre állnia. Az alternatívák számát érdemes 3–5 között tartani: túl kevés esetén nem tárjuk fel a teret, túl sok esetén az értékelés átláthatatlanná válik.

10.1.7 4. Értékelés és trade-off analízis

Az értékelés a konvergens szakasz: a feltárt alternatívákat összehasonlítjuk a célok és kényszerek mentén. A legegyszerűbb módszer egy **értékelési mátrix**, amelyben a sorok az alternatívák, az oszlopok pedig a kritériumok (például teljesítmény, skálázhatóság, karbantarthatóság, biztonság, költség, kockázat). Ezt részletesebben a `sa037` tárgyalja, itt a folyamat logikáját emeljük ki.

A mátrix kitöltése előtt minden kritériumhoz meg kell határozni a mérési módot vagy legalább a becslés alapját. A szubjektív pontozás elfogadható, ha a feltételezéseket dokumentáljuk, de kerülni kell a puszta véleményt. A quality attribute scenario-k itt konkrét tesztesetekként szolgálnak: minden alternatívát végigvezetünk a forgatókönyvön, és megbecsüljük vagy megmérjük, hogyan

teljesít. Ha az adat nem áll rendelkezésre, azt explicit módon jelezni kell – a nem futtatott mérést nem szabad ellenőrzött eredményként feltüntetni.

A trade-off analízis során fel kell tárni az ütközéseket. Például:

- A szinkron REST hívás egyszerűbb, de a hívó függ a hívott fél elérhetőségétől, ami csökkenti a rezilienciát.
- Az eseményvezérelt megközelítés növeli a függetlenséget, de bonyolítja a tranzakciókezelést és a megfigyelhetőséget.
- A monolit gyorsabban fejleszthető kis csapatban, de nagy terhelésnél a skálázás korlátozott.

Az értékelés során figyelembe kell venni a bizonytalanságot is. Vannak olyan alternatívák, amelyeknél a becslések megbízhatatlanok, mert a technológia új vagy a csapatnak nincs tapasztalata. Ilyenkor a döntést megelőzheti egy célzott kísérlet – PoC vagy spike –, amelynek célja a bizonytalanság csökkentése, nem pedig a termelési rendszer elkészítése. Erre a következő fejezetekben térünk ki részletesen.

10.1.8 5. Döntési pontok és visszacsatolás

A döntési pontok azok a pillanatok, amikor egy alternatívát kiválasztunk, és a többit elvetjük. A döntéseket **ADR-ben** (Architecture Decision Record) kell rögzíteni. Az ADR formátumát már bemutattuk a `sa012` fejezetben, itt a tervezési folyamat szempontjából a legfontosabb, hogy a döntéshez tartozzon:

- **Kontextus:** milyen probléma indokolta a döntést, milyen célok és kényszerek voltak érvényben.
- **Döntés:** mit választottunk, milyen főbb komponenseket, kapcsolatokat jelent.
- **Alternatívák:** melyek voltak a vizsgált lehetőségek, és miért vetettük el őket.
- **Következmények:** milyen előnyök és hátrányok származnak a választásból, milyen kockázatok maradtak.
- **Állapot:** elfogadott, javasolt, elavult, felülvizsgált.

A döntés nem mindig végleges. A **visszacsatolás** a döntés utáni megfigyelést jelenti: a rendszer működése során gyűjtött adatok (metrikák, naplók, incidensek) alapján ellenőrizzük, hogy a döntés elérte-e a kitűzött minőségi célokat. Itt kapcsolódnak be a **fitness functions** (lásd `sa033`): olyan automatizált tesztek vagy mérések, amelyek folyamatosan ellenőrzik az architektúrális tulajdonságokat. Ha egy fitness function meghiúsul, az jelzi, hogy a rendszer eltért a tervezett iránytól, és új döntésre lehet szükség.

A visszacsatolás lehet formális (például egy architektúra-review a `sa032` szerint) vagy folyamatos (például a CI/CD pipeline-ba épített minőségi kapuk). A cél az, hogy a döntés ne váljon megváltoztathatatlanná pusztán azért, mert egyszer leírtuk. Az evolúciós architektúra gondolata szerint a döntéseket a rendszer életciklusa során többször is újraértékeljük.

10.1.9 6. Gyakori hibák a tervezési folyamatban

A tervezési folyamat során számos tipikus hiba fordul elő. Ezek felismerése és elkerülése legalább olyan fontos, mint a módszertan ismerete.

- **Elemzésbénulás (analysis paralysis):** a tervező túl sokáig gyűjti az információkat, és nem hajlandó döntést hozni. A megoldás az időkorlát és a „last responsible moment” elv: akkor döntünk, amikor a döntés halogatása nagyobb költséggel jár, mint a döntés esetleges tévedése.

- **Aranykalapács (golden hammer):** a csapat minden problémára ugyanazt a mintát alkalmazza, mert azt ismeri. A tervezőnek tudatosan több alternatívát kell generálnia, és a kontextus alapján választani.
- **Kényszerek figyelmen kívül hagyása:** a szép architektúra nem ér semmit, ha nem felel meg a jogi vagy szervezeti korlátoknak. A kényszereket a folyamat elején rögzíteni kell, és minden alternatívánál ellenőrizni.
- **Túl korai optimalizáció:** a fejlesztő a feltételezett jövőbeli terhelésre optimalizál, mielőtt a jelenlegi követelmények teljesülnének. Ez bonyolultságot és költséget visz a rendszerbe, miközben a valós igény még nem igazolódott.
- **A döntés dokumentálásának hiánya:** ha a döntést nem rögzítjük, a későbbi csapattagok nem értik, miért választották az adott megoldást, és vagy megváltoztatják ok nélkül, vagy rosszul másolják.
- **Alternatívák elvetése indoklás nélkül:** az ADR-nek tartalmaznia kell, miért nem választottuk a többi lehetőséget. Ez biztosítja, hogy a jövőbeli olvasó megértse a döntés okát, és ne ismételje meg ugyanazt az elemzést.
- **A visszacsatolás elhanyagolása:** a döntés után nem követjük nyomon a rendszer viselkedését, így a hibás döntés csak akkor derül ki, amikor már nagy kárt okoz.

10.1.10 7. Példa: kommunikációs minta kiválasztása

A folyamat bemutatására vegyünk egy egyszerű, de valósághű példát. Egy online áruház rendszerében a **rendeléskezelés** szolgáltatásnak értesítenie kell a **készletkezelés** szolgáltatást, amikor új rendelés érkezik. A jelenlegi monolit rendszerben ez szinkron hívással történik. A cél a rendszer szétbontása moduláris monolitra, és a csapat azt fontolgatja, milyen kommunikációs mintát használjon a két modul között. Ez egy tipikus tervezési feladat, amely alkalmas a folyamat illusztrálására.

10.1.11 7.1. Probléma és kontextus

- **Cél:** a rendelés rögzítése után a készletet csökkenteni kell, de az ügyfél ne várjon a készletműveletre, ha az lassú vagy hibás. A rendelés elfogadása legyen robusztus.
- **Érintettek:** fejlesztők (két modulért felelős csapat), üzemeltetők, termékmenedzser.
- **Minőségi követelmény:** a rendelés visszaigazolása legfeljebb 1 másodperc alatt megtörténjen (95. percentilis), még akkor is, ha a készletkezelés átmenetileg nem elérhető. A készletfrissítés végül konzisztens legyen (legfeljebb 30 perc késleltetéssel).
- **Kényszerek:** a meglévő adatbázis tranzakciós monolit; a csapat mérete kicsi (3 fejlesztő); a rendszer nem lehet elosztott tranzakciókra támaszkodva, mert nincs tapasztalat ilyen eszközökkel; a jövőbeni terhelés növekedése várható, de a pontos szám nem ismert.

10.1.12 7.2. Alternatívák

1. **Szinkron REST hívás** – a rendelésmodul közvetlenül hívja a készletmodult.
2. **Aszinkron üzenet** – a rendelésmodul egy üzenetsorba ír, a készletmodul onnan olvas.
3. **Eseményvezérelt, Event Sourcing nélkül** – a rendelésmodul közzétesz egy „Rendelés-Létrehozva” eseményt, amelyre a készletmodul feliratkozik.

Mindegyik alternatíva leírható néhány mondatban, megadva a felelősségi köröket és a kommunikáció irányát. A kapcsolatokon fel kell tüntetni a protokollt, a szinkronitást és a hibatűrést.

10.1.13 7.3. Értékelés

Az értékeléshez a következő kritériumokat használjuk: **teljesítmény** (visszaigazolási idő), **reziliencia** (a készletmodul kiesésének hatása), **konzisztencia** (végleges adategyezés), **fejlesztési bonyolultság** (a csapat ismeretei), **skálázhatóság**. Mindegyikhez rövid elemzést készítünk:

- **Szinkron REST:** egyszerű, jól ismert, de a rendelés visszaigazolása közvetlenül függ a készletmodul válaszától. Ha a készletmodul lassú vagy elérhetetlen, a rendelés sem rögzül, ami sérti a minőségi követelményt. A konzisztencia azonnali lenne, de a rendelkezésre állás csökken.
- **Aszinkron üzenet:** a rendelés azonnal visszaigazolható, mert a modul csak üzenetet küld. A készletmodul kiesése nem akadályozza a rendelést, legfeljebb késlelteti a feldolgozást. A konzisztencia végül létrejön (eventual consistency). A bonyolultság közepes: üzenetsort kell üzemeltetni, és kezelni kell az üzenetek megbízhatóságát (pl. outbox minta).
- **Eseményvezérelt:** hasonló az aszinkron üzenethez, de lazábban csatolt, nincs közvetlen parancs. A feldolgozás késleltetése hasonló, de a rendszer hosszabb távon rugalmasabb lehet, ha több fogyasztó is feliratkozik. A bonyolultság magasabb, mert az eseményséma és a verziózás kezelést igényel.

A trade-offok világosak: a szinkron megoldás az egyszerűség és azonnali konzisztencia felé húz, míg az aszinkron/eseményvezérelt a reziliencia és a skálázhatóság felé.

10.1.14 7.4. Döntés

A csapat az **aszinkron üzenet** mellett dönt. Az indoklás:

- A minőségi követelmény (1 mp visszaigazolás készletkiesés esetén is) gyakorlatilag kizárja a szinkron hívást.
- Az eseményvezérelt megoldás előnyösebb lenne hosszú távon, de a jelenlegi csapatméret és tapasztalat miatt az üzenetközpontú megoldás egyszerűbb, és elegendő a jelenlegi igényekhez.
- Az aszinkron üzenet megvalósítható a meglévő monolitban, fokozatosan, a modulhatár megtartásával.

A döntést ADR-ben rögzítik, a fenti kontextussal, a két elvetett alternatíva indoklásával és a következményekkel (pl. szükség van egy üzenetsor-üzemeltetésre, a hibakezelést ki kell dolgozni, a konzisztencia azonnalisága megszűnik).

10.1.15 7.5. Visszacsatolás

A döntés után a csapat definiál fitness function-öket:

- A rendelés visszaigazolási ideje a 95. percentilisben 1 mp, mérve a produkciós környezetben, 10 perces ablakban.
- A készletfrissítés késleltetése 30 perc, mérve az üzenetküldés és a készletváltozás időbélyege közötti különbséggel.
- Az üzenetfeldolgozás hibaaránya 0,5% egy napos ablakban.

Ezeket a méréseket beépítik a megfigyelőrendszerbe (observability), és riasztást állítanak be, ha az SLO sérül. Ha a mért értékek tartósan rosszabbak a vártnál, a döntést újra kell értékelni.

Ez a példa mutatja, hogy a tervezési folyamat nem nagy, mindent átfogó dokumentumgyártás, hanem célzott, a bizonytalanságot csökkentő döntéshozatal.

10.1.16 8. Labor és átadandó eredmény

A fejezet laborjának célja a tervezési folyamat lépéseinek gyakorlása egy kisebb, ellenőrizhető feladaton. A feladatot csapatban vagy egyénileg is el lehet végezni. A dokumentációt verziózott formában (például Git repository-ban, Markdown vagy jegyzetfüzetben) kell elkészíteni.

Feladat: Válasszon egy egyszerű, de valós problémát a saját környezetéből vagy a megadott fiktív esetből. A fiktív eset: egy online jegyfoglaló rendszerben a felhasználók egyszerre próbálják lefoglalni ugyanazt a helyet. A rendszer jelenleg egyetlen monolit, szinkron adatbázis-tranzakcióval. A csapat fontolgatja, hogyan tegye a foglalást skálázhatóbbá és ellenállóbbá a csúcsterheléssel szemben.

A labor lépései:

1. **Kontextus rögzítése:** írja le a stakeholdereket, a fő funkcionális célt, a jelenlegi kényszereket (technológia, csapatméret, időkorlát).
2. **Minőségi követelmények megfogalmazása:** adjon meg legalább két quality attribute scenario-t a foglalási folyamatra vonatkozóan (teljesítmény, rendelkezésre állás vagy konzisztencia).
3. **Alternatívák feltárása:** generáljon legalább három architektúrális alternatívát a foglalási logika átalakítására (pl. optimista zárolás, aszinkron feldolgozás, elosztott lock, eseményvezérelt megközelítés). Mindegyikhez írja le a fő komponenseket, a kommunikációt és a feltételezett hibatűrést.
4. **Értékelés:** készítsen egyszerű összehasonlító táblázatot a következő kritériumokkal: válaszidő, konzisztencia, bonyolultság, skálázhatóság. A becsléseket indokolja, a nem mért adatokat jelölje meg.
5. **Döntés:** válasszon egy alternatívát, és írja meg a hozzá tartozó ADR-t a megfelelő formátumban (kontextus, döntés, alternatívák, következmények, állapot).
6. **Visszacsatolási terv:** adjon meg legalább két fitness function-t, amelyet a döntés után mérne, és írja le, milyen eszközzel gyűjtené az adatokat.
7. **Mért vs. várt eredmény:** a labor végén rögzítse, hogy mely állítások voltak becslések (várt) és melyeket sikerült ténylegesen megmérni vagy szimulálni (mért). Ha nem futtatott mérést, azt egyértelműen tüntesse fel.

Az elkészült dokumentumot a `sa033` fejezetben bemutatott fitness function koncepcióval összhangban kell elkészíteni. A labor kiinduló állapota: egy meglévő monolit jegyfoglaló rendszer, ahol a foglalás egyetlen tranzakcióban történik. Az elvárt eredmény: a fenti dokumentum, amely a tervezési folyamatot végigvezeti a döntési pontig. Az ellenőrzési szempontok: a kontextus teljes-e, a scenario-k mérhetők-e, az alternatívák száma legalább három, az értékelés indokolt-e, az ADR minden kötelező részt tartalmaz-e, a visszacsatolási terv konkrét-e. A visszaállítási út: ha a labor során bármilyen helyi környezetet módosított (pl. adatbázis séma), azt a labor leírása szerint izolált környezetben kell végezni, és a módosításokat vissza kell állítani.

10.1.17 9. Ellenőrzőlista

A fejezet végén ellenőrizze, hogy a tervezési folyamat során az alábbi pontok mindegyike teljesült:

- ☐ A probléma és a kontextus dokumentálva van, a stakeholderek azonosítva.
- ☐ A célok és kényszerek explicit módon, mérhető formában rögzítve.
- ☐ Legalább három különböző alternatíva feltárása megtörtént.
- ☐ Az alternatívák értékelése a minőségi követelmények és kényszerek alapján történt.
- ☐ A trade-offok és a bizonytalanságok dokumentálva vannak.

☐ A döntés ADR-ben rögzítve, az elvetett alternatívák indoklásával.
☐ A döntés utáni visszacsatolási terv (fitness functions, mérések) elkészült.
☐ A mért és várt eredményeket egyértelműen megkülönböztettük.
☐ A dokumentum verziózott, és a döntések visszakereshetők.

10.1.18 10. Önellenőrző kérdések

1. Melyek a tervezési folyamat fő lépései, és milyen sorrendben követik egymást? Hogyan módosul ez a sorrend iteratív környezetben?
2. Mi a különbség a cél, a kényszer és az alternatíva között? Adj példát mindegyikre egy általad ismert rendszerből.
3. Miért fontos a döntési pontok és a visszacsatolás szétválasztása? Hogyan kapcsolódik ez a fitness function fogalmához?
4. Nevezz meg három tipikus hibát a tervezési folyamatban, és magyarázd el, hogyan kerülhetők el.
5. A fenti példában miért az aszinkron üzenet lett a kiválasztott megoldás? Milyen trade-offokat fogadtak el ezzel a csapat tagjai?

10.1.19 11. Összefoglalás és továbbvezetés

Ebben a fejezetben a tervezési folyamat keretét tekintettük át: a probléma feltárásától a döntésig és a visszacsatolásig. A hangsúly azon van, hogy a folyamat ne mechanikus dokumentumgyártás legyen, hanem a bizonytalanság csökkentésének tudatos módszere. A célok, kényszerek és alternatívák világos szétválasztása, valamint a döntések ADR-ben történő rögzítése biztosítja, hogy a későbbi olvasó megértse a választás okát, és a rendszer fejlődése során a döntés felülvizsgálható legyen.

A következő fejezet (`sa035`) a Proof of Concept szerepét vizsgálja: hogyan használhatunk célzott kísérleteket a bizonytalan alternatívák értékelésére anélkül, hogy termelési döntést hoznánk. Az itt bemutatott folyamat logikai alapját fogjuk használni a PoC tervezésénél és értékelésénél. A labor során elkészített dokumentum jó kiindulópont lesz a további finomításhoz.

10.2 Proof of Concept: célszerűség és értékelés

Cél és határok

A Proof of Concept (PoC) egy bizonytalan vagy ismeretlen technológiai, architekturális vagy üzleti feltételezés ellenőrzött, időben és erőforrásban korlátozott vizsgálata. A cél nem a termék egy darabjának legyártása, hanem a döntéshez szükséges tudás megszerzése. A PoC azt a kérdést válaszolja meg, hogy egy adott elgondolás megvalósítható-e, milyen áron és milyen kockázatokkal, mielőtt a szervezet hosszabb távú kötelezettséget vállalna.

A terminológia a gyakorlatban gyakran lazul, és a PoC, a prototípus és a spike fogalma összemosódik. Itt élesen elválasztjuk őket:

- A **PoC** egy feltételezés igazolása vagy cáfolata. A kimenete egy döntési csomag, nem pedig egy továbbfejleszthető rendszerdarab.
- A **spike** rövid, idődobozos tanulási tevékenység, amelynek során a fejlesztő kódot olvas, kipróbál egy API-t vagy egy egyszerű kísérletet futtat. A spike kimenete gyakran egy rövid emlékeztető, becslési adat vagy a megvalósítási kockázat csökkenése.

- A **prototípus** egy futtatható váz, amelyet az érintettekkel való párbeszédre, használhatósági vizsgálatra vagy a felhasználói élmény feltárására készítenek. Célja a visszajelzés, nem a termelési minőség.

A PoC ezek közül a leginkább döntésorientált. Az előző alfejezetben (A tervezési folyamat: feltárás, alternatívák és döntés) bemutatott tervezési folyamatban a PoC az alternatívák feltárása és a döntés elköteleződése közötti szakaszban jelenik meg. Amikor a döntési mátrix egy cellája ismeretlen, vagy egy feltételezés a teljes megoldás sikerét veszélyezteti, a PoC az az eszköz, amellyel a bizonytalanságot mérhető tanulássá alakítjuk.

Fontos határt szabni: a PoC nem termelésbe telepíthető kód, nem skálázási bizonyíték, és nem helyettesíti a rendszertervezést. A PoC éppen arról szól, hogy a termelési minőségű megoldáshoz vezető úton a legnagyobb kockázatokat a lehető legkisebb befektetéssel kezeljük. Ha a PoC kódját változtatás nélkül akarják termelésbe tenni, az rendszerint a PoC céljának félreértését jelzi.

10.2.1 A PoC célja és a feltételezések megfogalmazása

A PoC csak akkor értékes, ha pontosan tudjuk, mit akarunk megtudni. A leggyakoribb hiba az, hogy a csapat úgy kezd "kipróbálni" egy technológiát, hogy nem írja le a vizsgált feltételezést, a siker kritériumát és a nem célokat. Az ilyen PoC végén a csapatnak lesz egy futtatható demója, de nincs döntési alapja, mert a demó önmagában nem bizonyít semmit.

A PoC kiindulópontja tehát a **vizsgálati kérdés**. A kérdés lehet technológiai, architekturális, teljesítménybeli, integrációs vagy akár szervezeti jellegű. Példák:

- "A Terragon Queue 3.2 képes-e legalább 5000 üzenet/másodperc átvitelére három replikával, legalább 99,9%-os üzenetkézbesítési arány mellett, a mi üzenetméreteinkkel és fogyasztói mintánkkal?"
- "A GraphQL lekérdezések képesek-e a meglévő termékkereső szolgáltatásunk válaszidejét 300 ms alatt tartani 50 párhuzamos ügyfél esetén, anélkül hogy a háttérrendszerek felé új N+1 lekérdezési mintát vezetnénk be?"
- "A modernizált rendszer képes-e az új ügyfélregisztrációs folyamatot 20%-kal rövidebbre csökkenteni azáltal, hogy a kaotikus adatbázis-hozzáférést egy dedikált ügyfél-API mögé helyezzük?"

Ezek a kérdések közös tulajdonsága, hogy **mérhetők**, **időben korlátozottak** és **döntési relevanciával** bírnak. A PoC tehát nem az "ismerjük meg a technológiát" homályos célját szolgálja, hanem egy konkrét döntési ponthoz szállít bizonyítékot.

A vizsgálati kérdést érdemes a döntési naplóban (ADR) szerepeltetni, abban a formában, hogy "A PoC az ADR-012 alternatíváját vizsgálja". Ez biztosítja a visszakereshetőséget: a jövőbeli olvasó látja, hogy a döntés során milyen bizonyítékot használtunk.

10.2.2 A PoC korlátai

A PoC hatékonysága éppen a korlátaiból fakad. A korlátok ismerete nélkül a PoC eredményét könnyű helytelenül általánosítani.

A PoC nem teljesítményteszt. A PoC környezete általában kicsi, izolált és a termelési konfigurációtól eltérő. Ha a PoC során egy adatbázis 100 bejegyzéssel jól teljesít, az nem jelenti azt, hogy 10 millióval is fog. A PoC célja nem a méret, hanem a megvalósíthatóság igazolása. A terheléses vizsgálat persze lehet a PoC tárgya, de akkor a PoC-nak magában kell foglalnia a termelési környezethez hasonló terhelési modellt, az adatok eloszlását és a mérési bizonytalanságot.

A PoC nem üzleti döntés. A PoC technikai vagy műszaki-gazdasági bizonytalanságot csökkent, de nem bizonyít üzleti modellt. Ha a kérdés az, hogy a felhasználók fizetni fognak-e egy funkcióért, akkor nem PoC-ra, hanem MVP-re van szükség. A PoC a "meg lehet-e csinálni", az MVP a "érdemes-e megcsinálni" kérdésre ad választ.

A PoC nem csökkenti a szervezeti kockázatot. Egy technológia lehet technikailag kiváló, de ha a csapat nem rendelkezik a működtetéséhez szükséges készségekkel, a bevezetés kudarcot vall. A PoC ezt nem feltétlenül mutatja meg, hacsak a csapat nem vonja be az üzemeltetési és biztonsági szerepköröket a vizsgálatba.

A PoC nem helyettesíti a részletes tervezést. A PoC gyakran a legrövidebb utat választja a működéshez. A hackelt megoldások, a hibakezelés elhagyása, a naplózás hiánya és az egyszerűsített interfészek mind a PoC gyorsaságát szolgálják. Ez nem hiba, amíg a PoC kódját nem tévesztik össze a termelési kóddal. A PoC-ból való átmenet során a termelési minőséget külön kell megtervezni.

A PoC nem reprodukálható automatikusan. Ha a PoC eredményét nem rögzítik, a tudás elveszik. A PoC jegyzőkönyvnek tartalmaznia kell a környezetet, a konfigurációt, a futtatott parancsokat és a mérést. A "labor- és kódszerződés" értelmében minden méréshez környezet, konfiguráció, időpont és bizonytalanság tartozik; ez a PoC-ra fokozottan igaz.

E korlátok ismeretében a PoC-ot nem szabad többre értékelni, mint amire való: **döntési bizonyíték, nem termék**.

10.2.3 A PoC életciklusa

A PoC jól strukturálható, ha a következő lépéseket követjük:

1. **Feltételezés és kérdés rögzítése.**
2. **Siker kritériumok és kudarc kritériumok meghatározása.**
3. **Minimális vizsgálati terv készítése.**
4. **Izolált környezet létrehozása.**
5. **A PoC futtatása és naplózása.**
6. **Eredmények értékelése, dokumentálása.**
7. **Döntés: tovább, módosítva, vagy leállítva.** A PoC kimenete soha nem "sikerült" vagy "nem sikerült" egyszerűen, hanem egy pontosabb kép a kockázatról.

A lépések közül a második a legnehezebb, mert a csapatok hajlamosak a “működik” homályos kritériumot használni. A siker kritériumnak mérhetőnek kell lennie, és ugyanolyan fontos a **kudarc kritérium** megadása is. Ha a PoC célja például annak igazolása, hogy egy új képfeldolgozó szolgáltatás 500 ms alatt válaszol, a kudarc kritérium lehet az, hogy a válaszidő 95%-os percentilise meghaladja az 500 ms-ot. Ha ezt előre rögzítjük, a PoC nem válik a siker felé mutató adathalászattá.

A **minimális vizsgálati terv** arra ösztönöz, hogy a PoC csak a kérdés megválaszolásához szükséges elemekre szorítkozzon. Minden további elem – például szép felhasználói felület, teljes naplózás, felügyeleti felület – csak akkor indokolt, ha a kérdés megválaszolásához kell, különben időt és figyelmet von el.

Az **izolált környezet** a labor- és kódszerződés szerint kötelező. Ha a PoC termelési adatbázishoz vagy éles rendszerhez fér, az katasztrofális következményekkel járhat. A PoC környezet legyen elkülönített, lehetőleg szintetikus vagy álnevesített adatokkal. A destruktív műveletek laborban is csak izolált környezetben futhatnak.

A **futtatás és naplózás** során a csapat dokumentálja a megfigyeléseket, a váratlan eseményeket és a konfigurációs elágazásokat. A napló nem csak a sikerről szól; a zsákutcák, a téves feltételezések és a környezetbeli hibák ugyanolyan értékesek a döntés szempontjából.

Az **értékelés** a mért eredményt veti össze a siker- és kudarc-kritériummal. Ha az eredmény bizonytalan, vagy a minta túl kicsi, azt explicit módon kell jelezni. A “nem reprodukált eredményt jelölni kell” elv itt is érvényes.

A **döntés** a PoC igazi kimenete. A döntési lehetőségek:

- **Tovább** a kiválasztott technológiával a részletes tervezésre.
- **Módosítva tovább**, például más konfigurációval, más integrációs mintával.
- **Leállítani** az alternatívát, és más megoldást keresni.
- **További vizsgálatot** indítani, mert a PoC nem volt elég meggyőző.

A döntést ADR-ben kell rögzíteni, a PoC jegyzőkönyvére hivatkozással.

10.2.4 Milyen kérdésekre alkalmas a PoC?

A PoC akkor a leghasznosabb, ha a kérdés **konkrét, mérhető és döntési relevanciájú**. A leggyakoribb kategóriák:

1. **Megvalósíthatósági kérdés:** “Lehetséges-e a meglévő rendszerbe integrálni X technológiát a jelenlegi authentikációs modell megtartásával?”
2. **Teljesítmény- vagy skálázhatósági kérdés:** “A választott üzenetküldő réteg képes-e a tervezett terhelés 120%-át elviselni úgy, hogy az üzenetvesztés 0,1% alatt maradjon?”
3. **Integrációs kérdés:** “A két rendszer közötti szinkronizációs protokoll képes-e a 10 ms alatti átviteli időre a meglévő tűzfalszabályokkal?”
4. **Kockázati kérdés:** “A konténeres futtatás bevezetése után a hiba események naplózása ugyanolyan szintű lesz-e, mint a virtuális gépes környezetben?”
5. **Költség- és erőforrás-kérdés:** “Az új üzenetsor 3 csomóponton futtatva mennyi CPU-t és memóriát fogyaszt a tervezett forgalom mellett?”

Ezek a kérdések mind rövid, célzott PoC-kal vizsgálhatók. A kérdésből kiindulva a PoC költsége és időtartama is becsülhető. A PoC ne tartson tovább, mint ami a döntés időablakába belefér; tipikusan néhány nap, legfeljebb két hét. Ha a kérdés megválaszolása ennél hosszabb időt igényel, akkor nem PoC-ról, hanem inkubációs projektről vagy pilotról beszélünk.

10.2.5 A PoC és az éles megoldás különbsége

A PoC és a termelési megoldás közötti különbség az egyik legfontosabb tanítás. A különbség nem csupán a kód minőségében, hanem a célban, a tervezési mélységben és a vizsgált tulajdonságokban is megjelenik.

Első szint: **a PoC a legkockázatosabb feltételezést vizsgálja, a termelési megoldás az összes szükséges tulajdonságot teljesíti.** A PoC elhagyhat olyan dolgokat, amelyek a kérdés szempontjából nem relevánsak. A termelési megoldásnak ugyanakkor meg kell felelnie a biztonsági, auditálhatósági, megfigyelhetőségi, mentési, helyreállítási, skálázási és jogi követelményeknek is.

Második szint: **a PoC kódja nem tervezett a változásra, a termelési megoldás igen.** A PoC kódja gyakran egyszeri kísérlet, amelyet a kérdés megválaszolása után akár el is dobhatunk. A termelési kódot az evolúció és a karbantartás szempontjai szerint kell strukturálni.

Harmadik szint: **a PoC eredményét a kontextus korlátozza, a termelési megoldást a kontextusban kell értékelni.** A PoC igazolhat egy képességet egy adott bemeneti mintán, de a termelési megoldásnak a valódi adatváltozatosságot, a hibamódokat és a váratlan terhelési mintákat is kezelnie kell.

Negyedik szint: **a PoC döntési csomagot szállít, a termelési megoldás terméket.** A PoC értéke a dokumentációban és a döntési naplóban van, a termelési megoldás értéke a futtatható rendszerben.

E szintek ismerete segít elkerülni a “PoC-ként működött, tehát termelésben is fog” csapdáját. A PoC jó esetben növeli a bizalmat, de a termelési minőséghez további tervezés, tesztelés és üzemeltetési felkészülés kell.

10.2.6 Sikeresség és tanulás

A PoC sikerességét gyakran rosszul mérik. A PoC akkor sikeres, ha **csökkenti a bizonytalanságot**, nem pedig akkor, ha igazolja az előzetes feltételezést. Ha a PoC megcáfolja a feltételezést, az is siker, mert megspórol egy rossz döntést. Ezt a gondolkodásmódot a szervezeti kultúrának is tükröznie kell; ha a PoC cáfolata büntetést von maga után, a csapatok a siker felé fognak csalni, és a PoC elveszíti az értékét.

A tanulásnak három szintje van:

1. **Tényszerű tanulás:** a válasz a vizsgálati kérdésre.
2. **Folyamati tanulás:** hogyan lehet hasonló kérdéseket a jövőben jobban vizsgálni.
3. **Szervezeti tanulás:** milyen döntési minták, érvelési hibák vagy feltételezések alakultak ki.

A PoC lezárásakor mindhárom szintet érdemes rögzíteni. A tényszerű tanulást a PoC jegyzőkönyvben, a folyamati tanulást a retrospektívben, a szervezeti tanulást az ADR-ben vagy a közös tudásbázisban.

10.2.7 Gyakori hibák és tévhitek

A PoC céljának elmosása. Ha a PoC-ba bekerül mindenféle "mellékesen jó lenne kipróbálni" elem, a kérdés elvész. A PoC-nak egyetlen elsődleges kérdése legyen; ha több kérdés van, akkor több PoC kell, vagy egy strukturált vizsgálati terv.

A siker kritérium utólagos megváltoztatása. Ha a mérés nem hozza a várt eredményt, a csapat kísértést érezhet arra, hogy lazítsa a kritériumot. Ez a megerősítési torzítás tipikus jele. A kritériumot a PoC futtatása előtt kell rögzíteni, és utólag csak akkor lehet módosítani, ha a feltételezés kontextusa valóban megváltozott – ezt a változást is dokumentálni kell.

A PoC kódjának "továbbfejlesztése". Ha a PoC kimenete technikai adóssággal terhelt kód, a szervezetnek döntenie kell, hogy a kódot eldobja vagy átalakítja. A "gyorsan feldobjuk termelésbe" döntés a PoC kódjának minőségi hiányosságait örökli. A helyes út az, ha a PoC kódját újraírják a termelési szabványok szerint, vagy legalább explicit refaktorálási tervet készítenek.

A környezet nem reprodukálható. Ha a PoC-ot egy fejlesztői laptopon futtatják, ahol rejtett konfigurációk és gyorsítótárak befolyásolják az eredményt, a következtetés megbízhatatlan. A PoC környezetét infrastruktúra-mintakóddal kell létrehozni, és a futtatási feltételeket rögzíteni.

A negatív eredmény elhallgatása. A cáfolat a legértékesebb tanulás. A PoC naplójában a negatív eredményeket ugyanolyan részletességgel kell dokumentálni, mint a sikert.

A PoC méretének eltúlzása. Ha a PoC-ra túl sok idő jut, a csapat hajlamos termelési minőségű kódot írni, ami felesleges. A PoC legyen annyira kicsi, amennyire a kérdés megválaszolása engedi, és semmivel sem nagyobb.

10.2.8 PoC-terv minta

Az alábbi PoC-terv minta a fejezetben bemutatott elveket követi. A példa egy fiktív döntési helyzetet dolgoz fel, és nem valós rendszerre vonatkozik.

10.2.9 Kontextus

A mintavállalat "Contoso Webshop" néven egy monolitikus rendszert üzemeltet. A termékkereső funkció jelenleg az adatbázis `LIKE` műveletére épül, és a válaszidő a termékek számának növekedésével romlik. A csapat azt feltételezi, hogy a keresést egy dedikált keresőszolgáltatásba lehetne kiszervezni, amely a termékkatalógus indexelt másolatát használja. A kérdés az, hogy a választott keresőmotor képes-e a katalógus 2,5 milliós termékrekordját indexelni és a tervezett keresési terhelést 300 ms alatti válaszidővel kiszolgálni.

10.2.10 Vizsgálati kérdés

A “Searchly 4.2” keresőmotor az álnevesített termékkatalógus 2,5 millió rekordjával, a tervezett 200 keresés/másodperc csúcsterhelésen képes-e a 95%-os válaszidőt 300 ms alatt tartani, miközben az indexfrissítés késleltetése nem haladja meg az 5 másodpercet?

10.2.11 Siker és kudarc kritérium

- Siker: a válaszidő 95. percentilise 300 ms 200 keresés/másodperc mellett, és az indexfrissítés átlagos késleltetése 5 s.
- Kudarc: a válaszidő 95. percentilise > 300 ms, vagy az indexfrissítés átlagos késleltetése > 5 s.
- Bizonytalanság: ha a minta mérete kisebb, mint 10 000 lekérdezés, vagy a terhelésgeneráló eszköz szórása miatt a percentilis 10% bizonytalansági sávja átlépi a küszöböt.

10.2.12 Nem célok

- Felhasználói felület készítése.
- Az összes keresési funkció megvalósítása (szűrők, típus szerinti keresés, ajánlás).
- A keresőszolgáltatás termelésbe állítása.

10.2.13 Környezet és konfiguráció

- Izolált labor környezet: `lab-placeholder-01`.
- Keresőmotor: Searchly 4.2, 3 adatcsomópont, egyenként 4 vCPU, 16 GiB RAM.
- Adatbázis: szintetikus, álnevesített termékkatalógus 2,5 millió rekorddal.
- Terhelésgeneráló: `loadgen-placeholder` eszköz, 200 keresés/másodperc, Poisson-eloszlású érkezés.
- Mérési ablak: 30 perc folyamatos terhelés, 10 perc bemelegítés után.
- Konfiguráció verziója: `searchly-poc-config-03.yaml`.

10.2.14 Várt és mért eredmény

- Várt: a válaszidő 95. percentilise a korábbi hasonló rendszerek alapján 250 ms körül várható; az indexfrissítés 1–2 s késleltetéssel valósul meg.
- Mért: a futtatás után rögzítendő; a mérési bizonytalanságot a terhelésgeneráló naplója alapján kell megadni.

10.2.15 Döntési napló

A PoC eredményét ADR-ben rögzítjük. A döntés lehetőségei: tovább a termelési tervezésre, a Searchly konfiguráció módosítása, vagy a megoldás elvetése.

10.2.16 Labor: PoC-terv készítése

A labor célja, hogy a résztvevő saját maga készítsen egy PoC-tervet egy bizonytalan technológiára. A labor architektúra-kata formájában zajlik, és a kiinduló állapotot egy fiktív döntési helyzet adja.

10.2.17 Kiinduló állapot

A “Nordwind Logisztika” egy elosztott nyomkövető rendszert tervez. A csapat azt fontolgatja, hogy a járművek telemetriai adatait egy streamfeldolgozó platformon dolgozza fel, de nem biztos abban, hogy a választott platform képes a tervezett 5000 járműből érkező, másodpercenkénti 50 kB adat feldolgozására, és hogy az elosztott feldolgozó csomópontok közötti állapotmegosztás nem vezet-e adatvesztéshez csomópontkiesés esetén.

10.2.18 Elvárt eredmény

A résztvevő készít egy reprodukálható PoC-tervet, amely tartalmazza:

1. A vizsgálati kérdést.
2. A siker és kudarc kritériumot, mérőszámmal és mérési ablakkal.
3. A környezetet és konfigurációt (helyőrzőkkel).
4. A futtatási napló vázát.
5. A várt és mért eredmény elkülönítését (a mért eredményt labor nélkül üresen hagyva, de a mérési módszert megadva).
6. A döntési napló bejegyzésének vázát.

10.2.19 Ellenőrzési szempont

- A kérdés konkrét, mérhető és döntési relevanciájú.
- A siker és kudarc kritérium előre rögzített, és nem utólag módosítható.
- A környezet izolált, és nem hivatkozik termelési rendszerre.
- A várt és mért eredmény elkülönül.
- A dokumentum reprodukálható: egy másik csapat ugyanazt a PoC-ot le tudná futtatni.

10.2.20 Visszaállítási út

A labor nem jár éles rendszer módosításával. Ha a résztvevő a labor során létrehozott erőforrásokat használ, azokat a labor végén törölni kell, vagy a törlési parancsot a dokumentumban rögzíteni.

10.2.21 Összefoglalás

A PoC a döntési bizonytalanság csökkentésének célzott eszköze. Nem termék, nem prototípus és nem teljesítményteszt, hanem egy ellenőrzött kísérlet, amely egy konkrét kérdésre ad mérhető bizonyítékot. A PoC akkor működik, ha a kérdés, a kritériumok és a korlátok előre rögzítettek, a környezet reprodukálható, és a tanulást a döntési naplóban dokumentálják. A PoC és a termelési megoldás közötti különbség megértése nélkül a demó könnyen téves döntéssé válik.

A következő alfejezet (MVP és spike: tanulás a bizonytalanságban) az MVP és a spike fogalmát bontja ki, és bemutatja, hogyan lehet a bizonytalanságot fokozatosan csökkenteni a PoC-tól a termelési döntésig.

Ellenőrzőlista a PoC-hoz

- A vizsgálati kérdés egyetlen, konkrét feltételezésre vonatkozik.
- A siker és kudarc kritérium mérhető, és a mérési ablak megadott.
- A környezet izolált, a konfiguráció verziózott, a destruktív műveletek kizártak.
- A PoC nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.
- A várt és mért eredmény elkülönül; a mért eredményt csak futtatás után töltjük ki.
- A PoC eredményét ADR-ben rögzítjük, a jegyzőkönyvre hivatkozással.
- A negatív eredményt ugyanolyan részletességgel dokumentáljuk, mint a sikert.

Önellenőrző kérdések

1. Mi a különbség a PoC és a prototípus között?
2. Miért fontos a kudarc kritériumot a PoC futtatása előtt rögzíteni?
3. Mikor nem alkalmas a PoC a döntési bizonytalanság csökkentésére?
4. Hogyan kerülhető el, hogy a PoC kódját tévesen termelésbe helyezzék?
5. Milyen szerepet játszik a reprodukálhatóság a PoC értékelésében?

10.3 MVP és spike: tanulás a bizonytalanságban

A szoftvertervezésben gyakran úgy képzeljük el a döntéseket, mint egy előre megrajzolt döntési fát: ha a követelmény világos, a megoldás is az. A valóság azonban ritkán ilyen tiszta. A legnagyobb bizonytalanság általában nem abban rejlik, hogy mit kell megépíteni, hanem abban, hogy a megépítendő megoldás valóban hozza-e a várt üzleti értéket, illetve hogy a technikai választások működnek-e az adott környezetben. Az **MVP** (Minimum Viable Product, azaz minimálisan működőképes termék) és a **spike** ugyanannak a szemléletnek két, eltérő fókuszú eszköze: mindkettő arra szolgál, hogy a bizonytalanságot strukturált módon, viszonylag kis befektetéssel csökkentsük, és a döntéseket valós visszajelzésre alapozzuk.

Az előző alfejezetben tárgyalt **Proof of Concept** (PoC) elsősorban azt vizsgálta, hogy egy megoldás *megvalósítható-e* egyáltalán. Ez a fejezet két olyan eszközt mutat be, amelyek továbblépnek a technikai megvalósíthatóság kérdésén: az **MVP** azt a kérdést teszi fel, hogy **mi az a minimális csomag, amellyel a legtöbb tanulást szerezzük a felhasználóktól vagy a rendszer viselkedéséből**, míg a **spike** egy szűk, jól körülhatárolt technikai vagy architekturális bizonytalanságot old fel időkorlátos kísérlettel. A két fogalom nem felcserélhető; a gyakorlati hiba egy része éppen abból fakad, hogy a csapatok összekeverik, és például egy spike-ot próbálnak meg termékként „eladni”, vagy egy MVP-t kizárólag technikai igazolásként kezelnek, elfelejtve az üzleti tanulási célt.

10.3.1 Az MVP szerepe a termék- és architektúradöntésekben

Az MVP kifejezés Eric Ries és Steve Blank lean startup szemléletéből terjedt el, és mára a termékfejlesztés alapszókincsévé vált. A klasszikus meghatározás szerint az MVP a termék azon változata, amely a lehető legkevesebb erőfeszítéssel a lehető legtöbb validált tanulást teszi lehetővé. Fontos hangsúlyozni, hogy az MVP **nem a kész termék gyenge, félkész kiadása**, hanem egy mérésre tervezett eszköz. A cél nem a kevés funkció szállítása önmagában, hanem az, hogy a minimális csomag segítségével olyan visszajelzést kapjunk, amely érdemben befolyásolja a további döntéseket.

Az MVP-t elsősorban termékmenedzserek emlegetik, de az architektúrális kontextusban is ugyanolyan erős a szerepe. Képzeljünk el egy rendszert, ahol az egyik minőségi követelmény az, hogy a felhasználói profilok ajánlásainak frissítése legfeljebb 300 ms alatt történjen meg a felhasználó interakcióját követően. A kérdés nem csupán az, hogy *meg tudjuk-e építeni* az ajánlószolgáltatást, hanem az is, hogy a felhasználók számára **valóban értékes-e** a valós idejű ajánlás, vagy egy éjszakai kötegelt frissítés is elegendő. Hiába oldjuk meg technikailag jól a 300 ms-os frissítést, ha a felhasználói viselkedés azt mutatja, hogy a kattintások túlnyomó többsége naponta egyszer történik, és az ajánlások frissessége nem növeli az elköteleződést. Ebben az esetben az architektúra bonyolultsága felesleges terhet rak a rendszerre: egy valós idejű streaming pipeline, egy eseményvezérelt szolgáltatás és egy cache réteg üzemeltetése jelentős költséggel járna, miközben az üzleti eredmény nem igényli azt.

Az MVP tehát nem helyettesíti a technikai PoC-ot, hanem **más dimenzióban vizsgálja a bizonytalanságot**. Míg a PoC azt igazolja, hogy egy adott technológia képes-e megvalósítani egy műszaki kritériumot, az MVP azt mutatja meg, hogy a megoldás eléri-e a kívánt üzleti vagy felhasználói hatást. A két eszköz kiegészíti egymást: előfordulhat, hogy egy architektúra technikailag tökéletesen működik, de az üzleti értéke megkérdőjelezhető; máskor az üzleti ötlet erős, de a technikai kivitelezés nem megy, mert a tranzakciós konzisztencia, a késleltetés vagy az üzemeltetési költség meghaladja a vállalható határokat.

10.3.2 A spike szerepe és a PoC-től való elhatárolása

A **spike** a szoftverfejlesztésben az eXtreme Programming (XP) módszertanból származó gyakorlat: egy rövid, **időkorlátos** kísérlet, amelynek célja egy konkrét technikai kérdés megválaszolása. A spike általában nem szállít termelésbe vihető kódot; az eredménye lehet egy jegyzet, egy kis demonstráció, egy mérési összefoglaló vagy akár az a megállapítás, hogy az adott megközelítés nem működik. A spike szokásos időtartama fél naptól néhány napig terjed, és a végeredményt gyakran tudatosan kidobják, mert a tanulás a fontos, nem a kódbázis növelése.

A spike és a PoC közötti különbség árnyalt, de lényeges. A PoC gyakran kicsit nagyobb volumenű, és azt vizsgálja, hogy egy technológia vagy dizájn alapvetően alkalmas-e a célra; a spike ezzel szemben ennél szűkebb, fókuszáltabb kérdést tesz fel, és az időkorlát keményebb. Tipikus spike-kérdések:

- Hogyan viselkedik az adatbázisunk, ha a tranzakciók egyidejűségét növeljük, és a `READ COMMITTED` izolációs szint elegendő-e a kosár egyesítési logikánkhoz?
- Milyen késleltetést mérhetünk a `placeholder://message-broker` és a `placeholder://event-stream` között, ha a rendelkezésre álló sávszélesség 100 MBit/s?
- Mennyire támogatja a kiválasztott szolgáltatásmesh a retry és circuit breaker beállításokat a sidecar proxy konfigurációjában?

Ezekben a kérdésekben a közös, hogy a válasz **nem következik közvetlenül a dokumentációból**, és a bizonytalanság fennállása blokkol egy architektúradöntést. A spike ilyenkor a döntés előkészítésének olcsó módja: ahelyett, hogy a teljes átalakítást a feltételezésre alapoznánk, a kritikus bizonytalanságot egy idődobozos kísérlettel oldjuk fel.

Fontos különbség: a spike eredménye **nem termelési kód**. Ha a spike során született kód véletlenül „túl jó" lesz, és a csapat kedvet érez rá, hogy megtartsa, az gyakran csapda. A spike kód általában a célirányos kísérletre optimalizált, nem tartalmaz hibakezelést, tesztlefedettséget, megfigyelhetőséget és biztonsági megfontolásokat. A termelésbe emelés későbbi technikai adósságot okoz. A szabály: **használd a spike eredményét a döntéshez, de a termelési kódot írd meg normál minőségben, a spike tanulságait felhasználva**.

10.3.3 Tanulási cél és minimális érték: a kísérlet hipotézise

Mind az MVP, mind a spike középpontjában egy **hipotézis** áll. A hipotézis nélküli MVP egyszerűen egy kisebb termék, a hipotézis nélküli spike pedig egy elkalandozó programozási gyakorlat. A kísérleti szemlélet azt jelenti, hogy előre megfogalmazzuk: mit szeretnénk megtudni, milyen eredményt várunk, és milyen megfigyelések erősítenék vagy cáfolnák a hipotézisünket.

Az MVP esetében a hipotézis jellemzően üzleti vagy használati állítás. Például:

> **Hipotézis:** Ha a felhasználók valós idejű ajánlást látnak a kosár oldalán, akkor a kosárba helyezés aránya legalább 8%-kal nő a jelenlegi, éjszakai kötegelt frissítésű ajánlásokhoz képest.

Ez a hipotézis mérhető, és a kísérlethez mérési terv szükséges. A mérési terv rögzíti, hogy melyik metrikát figyeljük (kosárba helyezési arány), milyen időablakban mérünk, mekkora forgalmat érintünk, és mi a siker, illetve a kudarc kritériuma. Az architektúra szempontjából az ilyen hipotézis azért fontos, mert közvetlenül befolyásolja a tervezett komponens bonyolultságát. Ha a valós idejű frissítés értéke nem igazolható, a streaming pipeline elhagyható, és maradhat a kötegelt feldolgozás, ami kisebb üzemeltetési terhet jelent.

A spike esetében a hipotézis technikai vagy architekturális állítás. Például:

> **Hipotézis:** A `placeholder://vector-database` rendszerben a legfeljebb 1 millió vektoros szűkített keresés 50 ms-nál rövidebb késleltetést ér el, ha az adatok egyetlen partíción futnak, és a lekérdezés párhuzamossága legfeljebb 8 szálat használ.

Ez a hipotézis mérhető, és a spike során a mérést reprodukálható módon dokumentáljuk. A spike eredménye az ADR-be kerül, ahol a döntést alátámasztja.

A **minimális érték** fogalma az MVP esetében azt a csomagot jelöli, amely még elegendő ahhoz, hogy a hipotézist megvizsgáljuk. Ez a minimális csomag lehet kisebb, mint a kész termék, de a kulcsa az, hogy a hipotézishez kell elégséges legyen. Ha például a kérdés az, hogy a felhasználók használnának-e hangalapú keresést, akkor nincs szükség teljes hangfelismerő motorra; elég lehet egy egyszerű, ember által vezérelt „varázsló" interfész, amely mögött egy operátor írja be a keresőkifejezést. A felhasználó ugyanazt a hatást látja, a kérdésre választ kapunk, és nem kell előre befektetnünk a komplex beszédfeldolgozó rendszerbe.

A minimális érték fogalma architekturális kontextusban is releváns: **mekkora az a minimális architekturális komponenshalmaz, amely elegendő a minőségi követelmény validálásához?** Tegyük fel, hogy a rendszernek 99,9%-os rendelkezésre állást kell biztosítania. Az MVP-architektúra lehet egyetlen régióban futó, aktív-passzív adatbázissal rendelkező monolitikus szolgáltatás; a kérdés az, hogy ez a konfiguráció eléri-e a rendelkezésre állási célt, mielőtt

megépítenénk a teljes, aktív-aktív multi-régiós felépítést. A válasz itt nem marketingvisszajelzés, hanem üzemeltetési mérés: mekkora állásidőt okoznak a valós hibák, a kiadások és a hálózati események. Ha a kilencvenkilenc százalékos szint már ezzel az egyszerűbb architektúrával is teljesül, a komplexitás bevezetése késleltethető vagy elkerülhető.

10.3.4 A kísérlet eredményének visszacsatolása

A kísérlet – legyen az MVP vagy spike – csak akkor ér valamit, ha az eredményét visszacsatoljuk a döntési folyamatba. A visszacsatolás formái több szinten jelennek meg:

1. **ADR frissítése:** Ha a spike eredménye megváltoztatja a tervezett döntést, az ADR-t módosítani kell. Az ADR kontextusában rögzítjük, hogy a döntés melyik spike-ra épül, annak eredményét hogyan mértük, és milyen alternatíva volt még versenyben.
2. **Minőségi forgatókönyv kalibrálása:** Ha az MVP mérése azt mutatja, hogy a felhasználói viselkedés miatt a 300 ms-os válaszidő irreálisan szigorú, vagy éppen ellenkezőleg, túl laza, a quality attribute scenario-t módosítani kell. A mérési adatok alapján finomítható az inger, a környezet, a válasz és a mérőszám.
3. **Technikai adósság feljegyzése:** Ha az MVP-t a termék korai validálására használjuk, gyakran szükség van olyan kompromisszumokra, amelyek később addósságként jelennek meg. A döntési naplóban rögzíteni kell, hogy milyen minőségi elvárásokat függesztettünk fel az MVP idejére, és mi a terv a visszafizetésre. Így elkerülhető, hogy az ideiglenes kompromisszum észrevétlenül véglegessé váljon.
4. **Következő lépés meghatározása:** A kísérlet eredménye lehet siker, kudarc vagy részleges siker. Mindhárom esetben konkrét döntést kell hozni: folytatjuk a kiválasztott irányt, alternatív irányt választunk, vagy további mérésre van szükség. A visszacsatolás nélküli kísérlet nem több, mint időpazarlás.

A visszacsatolás gyakorlati megvalósításához elengedhetetlen, hogy **a kísérletet már a tervezés során a döntéshez kössük**. Ha a spike elején nem határozzuk meg, hogy melyik ADR-t vagy minőségi követelményt akarjuk alátámasztani vele, akkor a végén az eredményt nehéz lesz értelmezni. A kapcsolatot akár a spike nevében is rögzíthetjük: „Spike: `sa036-spike-01` a kosár-oldal késleltetés SLO kalibrálásához”.

10.3.5 Gyakorlati minta: MVP- és spike-terv

Az alábbiakban egy lépésenkénti sablont mutatunk be, amely segít összeállítani az MVP- és spike-tervet. A sablon célja, hogy a kísérlet reprodukálható legyen, a kontextus rögzítve legyen, és a mért, illetve a várt eredmények egyértelműen elváljanak egymástól. Ez az elválasztás azért kritikus, mert a kognitív torzítások – például a megerősítési torzítás – miatt hajlamosak vagyunk a várakozásainknak megfelelő adatokat jobban észrevenni. A tagolt terv ezt ellensúlyozza.

10.3.6 1. Kontextus és cél

Kérdés: Mit szeretnénk megtudni? **Miért most?** Melyik döntést blokkolja a bizonytalanság? **Kapcsolódó ADR/quality attribute scenario:** Melyik korábbi döntés vagy követelmény függ ettől? **Érintett stakeholderek:** Kiknek lesz fontos az eredmény?

10.3.7 2. Hipotézis

Állítás: Egyértelmű, mérhető állítás. **Várt eredmény:** Mit gondolunk a mérés kezdete előtt, hogy mi fog történni? Ezt jelöljük „várt”-ként. **Mérési terv:** Milyen metrikát figyelünk, milyen egységben, milyen időablakban, milyen környezetben? **Siker/kudarc küszöb:** Milyen érték esetén fogadjuk el, illetve vetjük el a hipotézist?

10.3.8 3. Az MVP vagy spike hatóköre

Minimális csomag: Mi az a legkisebb funkcionalitás vagy technikai kísérlet, amely elegendő a válaszhoz? **Kizárások:** Mit tudatosan nem csinálunk meg az időkeretben? **Időkeret:** Az MVP vagy spike tervezett időtartama. **Erőforrások:** Ki dolgozik rajta, milyen környezetet használunk?

10.3.9 4. Végrehajtás

Lépések: A konkrét tevékenységek. **Mérési pontok:** Hol és mikor rögzítjük az adatokat? **Dokumentáció:** A mérési eredményeket egy jegyzetfüzetben vagy egy ADR-vázlatban rögzítjük.

10.3.10 5. Eredmény és visszacsatolás

Mért eredmény: A ténylegesen megfigyelt adatok, mértékegységgel, mérési ablakkal, bizonytalansággal. **Összehasonlítás:** A mért és a várt eredmény összevetése. **Következtetés:** Elfogadjuk vagy elvetjük a hipotézist? **Döntés:** Milyen architekturális vagy termékdöntést hozunk ez alapján? **Következő lépések:** Milyen további kísérletre van szükség?

10.3.11 Példa: keresési szolgáltatás MVP és spike terve

Vegyünk egy konkrét példát, hogy a sablon ne maradjon elvont. A fiktív rendszer egy e-kereskedelmi platform, ahol a felhasználók termékeket kereshetnek. A jelenlegi architektúra monolitikus, a keresést egy SQL `LIKE` alapú megoldás végzi. A növekvő katalógus és a bizonytalan relevancia miatt felmerül, hogy a rendszer egy dedikált keresőmotort kapjon. A csapat kérdése: **érdemes-e bevezetni egy keresőmotort, és ha igen, milyen kompromisszumokat jelent ez?**

10.3.12 MVP-terv a keresési relevancia validálására

Kontextus: A meglévő SQL-alapú keresés pontossága 0,62 (átlagos NDCG@10 a tesztkészleten), a felhasználói visszajelzések szerint a keresési találatok gyakran nem relevánsak. A termékmenedzser szerint a jobb találatok növelnék a konverziót.

Hipotézis: Ha a felhasználók egy egyszerűbb, de relevancia szerint rendezett találati listát látnak, akkor a keresésből indult vásárlások aránya legalább 5%-kal nő.

Várt eredmény: A vásárlási konverzió 5%-os növekedése. **Mérési terv:** A keresési találatokból indult megrendelések arányát mérjük 14 napos ablakban, a kezelt csoportban napi 3000 keresési lekérdezés mellett. A kontrollcsoport továbbra is a régi keresést kapja. **Siker küszöb:** Legalább 5% relatív növekedés a kontrollcsoporthoz képest, 95%-os konfidenciaszint mellett.

Minimális csomag: Nem kell teljes keresőmotort telepíteni. Elég egy statikus relevancia-újrarendezés: a meglévő SQL találatokat egy egyszerű szolgáltatás újrarendezi a termékleírás és a népszerűség alapján. A felhasználói felület változatlan marad, csak a rendezés változik. **Kizárások:** A keresőmotor indexelése, a facettás szűrés, a szinonimakezelés és a gépi tanulásos modell nem része az MVP-nek. **Időkeret:** 5 munkanap. **Erőforrások:** Két fejlesztő, egy adatmérnök, tesztkörnyezet a sandboxban.

Végrehajtás: A relevancia-újrarendezést egy microservice implementálja, amely a meglévő találatokat kapja, és egy előre kiszámított pontszám alapján átrendezi. A mérési pont a checkout eseményben van, ahol egy feature flag dönti el, hogy a felhasználó melyik csoportba esett. **Dokumentáció:** Mérési jegyzetfüzet a várt és a mért konverziós arányról napi bontásban.

Eredmény: A 14 napos ablakban a tesztcsoportban a konverzió 6,1%-kal nőtt, a konfidencia-intervallum [2,8%, 9,7%]. A mért eredmény eltér a várttól pozitív irányban, de az alsó határ alacsonyabb, mint az 5%-os küszöb, tehát a növekedés nem egyértelműen jelentős. **Következtetés:** A hipotézist részlegesen támasztja alá az adat. A döntés: a keresőmotor bevezetése előtt szükséges egy további mérés, mert a pontbecslés meghaladja a küszöböt, de a bizonytalanság nem elegendően kicsi. **Döntés:** Nem döntünk a teljes kivitelezés mellett; helyette egy újabb MVP-iterációt indítunk, amelyben a relevancia-modellt tovább finomítjuk.

10.3.13 Spike-terv a keresőmotor késleltetésének vizsgálatára

A fentivel párhuzamosan a csapat technikai bizonytalansággal is szembesül: vajon a kiválasztott keresőmotor a rendelkezésre álló erőforrásokkal teljesíti-e a válaszidő-célt?

Kontextus: A minőségi forgatókönyv szerint: *Ha a felhasználó keresést indít (inger), a normál üzemű rendszerben (környezet), a találati lista legfeljebb 200 ms alatt megjelenik (válasz), és a p95 késleltetés egy 24 órás ablakban nem haladja meg ezt az értéket (mérőszám).* A jelenlegi monolitikus rendszer 280 ms p95 késleltetést produkál. A kérdés, hogy a keresőmotor beépítése javítja-e ezt.

Hipotézis: A `placeholder://search-engine` elosztott keresőmotor a 200 MB-os katalóguson, 2 csomópontos klaszterben a p95 késleltetést 150 ms alatt tartja.

Várt eredmény: A p95 késleltetés 150 ms alatt marad. **Mérési terv:** Két virtuális gép, egyenként 4 vCPU és 8 GB RAM, a keresőmotor egyetlen replikával. A mérést 1 órán át futtatjuk, másodpercenként 50 lekérdezést generálva. A késleltetést a szolgáltatás oldalán mérjük, és a p95 értéket 1 perces ablakokban számítjuk. **Siker küszöb:** A p95 késleltetés 1 órás átlaga legfeljebb 150 ms.

Minimális csomag: A keresőmotor alapkonfigurációja, a katalógus betöltése, egy egyszerű lekérdezési séma. Nincs szükség facettákra, szinonimákra vagy gépi tanulásos rendezésre. **Kizárások:** Magas rendelkezésre állás, horizontális skálázási teszt, integrált relevancia-újrarendezés. **Időkeret:** 2 munkanap. **Erőforrások:** Egy fejlesztő, ideiglenes tesztklaszter a `placeholder://sandbox-cluster` környezetben.

Végrehajtás: Az 1 órás terhelés alatt a lekérdezési mintákat a helyettesített termékkatalógusból generáljuk, a paraméterezésben a leggyakoribb 100 keresőkifejezést használjuk. A mérési adatokat nyers logba írjuk, majd egy egyszerű szkript számítja a p95 értékeket. **Dokumentáció:** Egy rövid ADR-vázlatban összefoglaljuk az eredményt.

Eredmény: Az 1 órás ablakban a p95 késleltetés átlaga 132 ms volt, a mérési bizonytalanság becsült értéke ± 12 ms (a mérési környezet hálózati késleltetésének ingadozása miatt). Ez alatta van a 150 ms-os küszöbnek. **Következtetés:** A hipotézist elfogadjuk. A keresőmotor beépítése

valószínűleg teljesíti a késleltetési követelményt. **Döntés:** A csapat a keresőmotor használatát választja, és a döntést ADR-ben rögzíti. Az ADR-ben szerepel a spike mérési összefoglalója is.

10.3.14 Tipikus hibák és elkerülésük

Az MVP és a spike alkalmazása során számos visszatérő hiba jelenik meg. Az alábbiakban ezeket tekintjük át a helyes megoldás indoklásával.

1. Az MVP-t termékként kezelik, nem kísérletként. A csapat hajlamos arra, hogy a minimális csomagot a végleges termék kicsinyített másaként gondolja el, és ezért a funkciók listáját próbálja csökkenteni, miközben a minőségi elvárásokból nem enged. Ez félrevezeti a mérést: a felhasználók nem azt kapják, ami a hipotézis teszteléséhez kell, hanem egy rosszabb minőségű terméket, amelynek a megítélése a minőségi hiányok miatt torzít. A megoldás az, hogy a minimális csomagot **a hipotézisből vezetjük le**, és a minőséget ott tartjuk magasan, ahol az a tanuláshoz szükséges.

2. A spike eredményét nem dobják ki. A spike kódját gyakran „majdnem kész” állapotban találják, és a termelési kódba emelik. Ez a technikai adósság klasszikus forrása. A helyes gyakorlat: a spike eredménye dokumentáció és tudás; a termelési kódot a normál fejlesztési folyamat szerint írják meg, a spike tanulságait beépítve.

3. Nincs előre rögzített siker/kudarc küszöb. Ha a küszöböt csak a mérés után határozzuk meg, az eredmény értelmezése szubjektív lesz, és a megerősítési torzítás szabadon érvényesül. A küszöböt a kísérlet kezdete előtt kell rögzíteni, és azt is meg kell határozni, hogy mi történik, ha az eredmény nem éri el.

4. Nincs elkülönített környezet. Az MVP és a spike mérése akkor megbízható, ha a környezet jól specifikált és elszigetelt. Ha a spike egy olyan megosztott tesztkörnyezetben fut, ahol más csapatok is terhelést generálnak, a mért késleltetés nem lesz megismételhető. A mérési jegyzőkönyvben rögzíteni kell a környezetet, a konfigurációt, az időpontot és a bizonytalansági tényezőket.

5. Nincs visszacsatolás a döntési naplóba. Ha a spike eredményét megbeszélik, de nem rögzítik az ADR-ben, elveszik a tudás. A későbbi olvasó nem tudja, miért választottak egy adott megoldást, és a szervezet újra ugyanazt a bizonytalanságot fogja vizsgálni. A döntési napló a szervezeti memória.

6. A spike nem időkorlátos. Ha a spike időtartamát nem korlátozzák, könnyen átcsúszik egy teljes fejlesztéssé, amelynek nincs valódi célja. A spike lényege a fókusz: egy kérdésre válaszolunk, és ha a válasz megvan, megállunk. Az időlimit külső nyomást ad, hogy ne terjeszkedjen a hatókör.

7. Az MVP méréshez nem határoznak meg elegendő minta méretet. Ha a tesztcsoport túl kicsi, az eredmény statisztikailag nem lesz meggyőző, és a döntés hamis bizonyosságot ad. A mérési tervben előre meg kell becsülni a szükséges mintaméretet.

8. Az MVP-t architektúra-döntés nélkül hajtják végre. Ha az MVP során olyan komponenseket vezetnek be, amelyek hosszú távon megmaradnak, de ezt nem rögzítik döntésként, a rendszer később kaotikussá válik. Minden, az MVP-ből származó architekturális elemre ki kell mondani: mi az ideiglenes, mi az állandó, és milyen feltételek mellett.

10.3.15 Ellenőrzőlista az MVP és spike tervezéséhez

Zárásképpen egy ellenőrzőlista, amely a gyakorlatban a kísérlet megkezdése előtt áttekinthető:

- ☐ Megfogalmaztuk a kérdést, amelyre választ kell kapnunk.
- ☐ Van egyértelmű hipotézis, mérhető állítással.
- ☐ Meghatároztuk a siker és a kudarc küszöbét, és ezt a mérés előtt rögzítettük.
- ☐ Kijelöltük a mérési metrikát, mértékegységet, időablakot és környezetet.
- ☐ Az MVP vagy spike hatóköre minimális, de elegendő a hipotézis teszteléséhez.
- ☐ Kizártuk a nem szükséges funkciókat és a felesleges komponenseket.
- ☐ A spike-ra időkorlátot szabtunk, és a kódot tudatosan nem szánjuk termelésbe.
- ☐ Elkülönített környezetet biztosítottunk, és dokumentáltuk a konfigurációt.
- ☐ A mérési jegyzőkönyvben különválasztjuk a várt és a mért eredményt.
- ☐ Az eredményt visszacsatoljuk az ADR-be vagy a minőségi forgatókönyvbe.
- ☐ Meghatároztuk a következő lépést (elfogadás, elvetés, további kísérlet).

10.3.16 Önellenőrző kérdés

Egy csapat egy új mikroarchitektúrás rendszer előtt áll, és az egyik minőségi követelmény szerint a p99 szolgáltatásválaszidőnek 150 ms alatt kell lennie. A csapat bizonytalan abban, hogy a kiválasztott kommunikációs protokoll (`placeholder://grpc-proxy`) és a belső hálózati réteg együttesen képes-e erre. A termékmenedzser egy MVP-t akar indítani, hogy a felhasználók visszajelzését megtudja. A fejlesztők egy spike-ot javasolnak. Melyik eszközt mikor alkalmazná, és miért? Írja le a tervezett spike hipotézisét, mérési tervét és a siker küszöbét.

Válaszvázlat: A felhasználói visszajelzéshez MVP kell, mert a kérdés az üzleti hasznosságra vonatkozik. A p99 cél eléréséhez spike szükséges, mert a bizonytalanság technikai jellegű. A spike hipotézise például az, hogy a p99 válaszidő 120 ms alatt marad, ha 200 kérést/másodperc sebességgel terheljük a rendszert, és a mérést 1 órán át futtatjuk. A siker küszöb: a mért p99 késleltetés 150 ms alatt van a mérési ablakban.

10.3.17 Laborfeladat: MVP- és spike-terv összeállítása

A labor célja, hogy a csapat összeállítson egy MVP- és spike-tervet egy választott, kisméretű rendszer-kontextusra, és reprodukálható dokumentumban rögzítse a döntési naplót. A kontextust a korábbi fejezetekben bemutatott e-kereskedelmi kereső példából választhatjuk, vagy egy saját, a kurzuson használt fiktív rendszerből.

Kiinduló állapot: Adott egy meglévő monolitikus rendszer, amelynek keresési funkciója nem elég gyors, és a relevanciája kérdéses. A csapat szeretné eldönteni, hogy bevezessen-e dedikált keresőmotort. A minőségi követelmény: a p95 keresési késleltetés 300 ms alatt, a konverzió növekedése 5% felett. Mindkettő jelenleg nem teljesül.

Elvárt eredmény: Két terv készül: 1. **MVP-terv** a relevancia hipotézis tesztelésére, a felhasználói viselkedés mérésére. 2. **Spike-terv** a keresőmotor késleltetésének mérésére. Mindkét tervhez tartozik egy ADR-vázlat, amely rögzíti a döntést, az alternatívákat, a következményeket és az állapotot. A dokumentumban a mért és a várt eredmények külön rovatban szerepeljenek.

Ellenőrzési szempontok: - A hipotézis egyértelmű és mérhető. - A mérési terv tartalmaz egységet, időablakot, környezetet és bizonytalansági becslést. - A siker/kudarc küszöb a mérés előtt rögzített. - A dokumentumban a mért és a várt eredmény elkülönítve jelenik meg. - Az ADR

hivatkozik a spike-ra vagy az MVP-re, és annak eredményére. - A labor során nem használtak valós hitelesítő adatot, belső URL-t vagy személyes adatot; mindenhol helyőrző szerepel.

Visszaállítási út: Ha a csapat a labor során módosítja a tesztkörnyezetet, állítsa vissza a kiinduló állapotot a labor leírása szerint. Destruktív műveletet csak izolált környezetben szabad végrehajtani, és azt a jegyzőkönyvben jelölni kell.

A begyűjtött tapasztalatokra a következő alfejezet épít, ahol a döntési mátrixok és a választási folyamatok kerülnek elő.

10.4 Mintaválasztás és döntési mátrixok

A minták és stílusok összehasonlítása nem öncélú tevékenység. A MVP és spike: tanulás a bizonytalanságban alfejezetben tárgyalt MVP- és spike-megközelítések után itt arra a kérdésre keresünk választ, hogy amikor már van néhány ígéretes architekturális jelöltünk, hogyan tudjuk az összehasonlítást nyomon követhetővé, vitathatóvá és felülvizsgálhatóvá tenni. A döntési mátrix nem igazsággép, hanem a mérlegelés dokumentált formája: arra való, hogy a kontextust, a szempontokat, a súlyokat és a bizonytalanságokat egyetlen, reprodukálható felületen rögzítse. Ez a fejezet ezt a keretet mutatja be, a tipikus hibákkal és egy olyan laborral, amelyben három architektúra-jelöltet hasonlíthatunk össze a gyakorlatban.

10.4.1 1. A minták és stílusok összehasonlításának célja és korlátai

Az architektúrális minták – például réteges monolit, moduláris monolit, mikroszolgáltatások, eseményvezérelt architektúra – nem eleve jók vagy rosszak. Egy minta akkor működik, ha illeszkedik a rendszer környezetéhez: a fejlesztőcsapat méretéhez és érettségéhez, a terhelési profilhoz, a rendelkezésre állási célokhoz, a változtatási sebességhez, a szabályozási kényszerekhez és az üzemeltetési képességekhez. Ugyanaz a mintaválasztás, amely egy háromfős startupnál gátolja a gyors iterációt, egy húszfős, erősen párhuzamosan dolgozó csapatnál éppen a független szállítást teheti lehetővé.

A döntési mátrix arra szolgál, hogy ezt a kontextusfüggőséget explicit módon kezelje. Ahelyett, hogy egyetlen népszerű szempont – például a skálázhatóság vagy a divatosság – alapján döntenénk, több dimenziót veszünk figyelembe, azok fontosságát súlyozzuk, az egyes jelölteket pontozzuk, majd a súlyozott összeget a bizonytalansággal együtt értékeljük. Ez nem garantálja a tökéletes döntést, de azt igen, hogy a döntés indokai később visszakereshetők legyenek, és hogy a döntésben részt vevők ne érzelmi vagy politikai alapon, hanem átlátható szempontok mentén vitatkozzanak.

A mátrix korlátai is fontosak. A pontszámok legtöbbször becslések, nem mérések. A súlyok értékítéletet tükröznek, nem objektív igazságot. Ha a mátrixot túlzott matematikai precizitással kezeljük – tizedesjegyekre számítva a végeredményt –, könnyen hamis pontosság illúziójába esünk. A cél tehát nem egy „helyes” súlyozott átlag megtalálása, hanem a döntési tér feltárása, az érzékeny pontok azonosítása és a viták strukturálása.

10.4.2 2. A döntési mátrix felépítése

10.4.3 2.1. Kontextus rögzítése

Bármely összehasonlítás első lépése a döntési kontextus leírása. Ennek tartalmaznia kell a funkcionális követelmény rövid összefoglalását, a kritikus minőségi attribútumokat (a Teljesítmény

és skálázhatóság–Karbantarthatóság, használhatóság és üzemeltethetőség alfejezetben tárgyaltak szerint), a csapat- és szervezeti feltételeket, a technológiai kényszereket, a terhelési és rendelkezésre állási előírásokat, valamint a szállítási határidőket és a költségkereteket. Példa egy ilyen rövidített kontextusra:

> Egy belső visszáru-kezelő rendszert kell kialakítanunk. A felhasználók száma 80 belső munkatárs, a csúcsterhelés 40 egyidejű kérés másodpercenként, az átlagos terhelés 5 kérés másodpercenként. A rendelkezésre állási cél 99,5% havi ablakban, azaz az error budget $1 - 0{,}995 = 0{,}005$, ami havonta legfeljebb körülbelül 3,6 óra állásidőt jelent. A fejlesztőcsapat két, egyenként 6 fős streamre oszlik, a technológiai stack Java és PostgreSQL. Az üzleti logika évente 6–8 nagyobb módosítást igényel, a szállítási ciklus kéthetes. Nincs külön platformcsapat; az üzemeltetést a fejlesztők végzik.

A kontextus rögzítése azért kritikus, mert a későbbi pontszámok csak erre a kontextusra értelmezhetők. Ugyanaz a minta más kontextusban más pontszámot kapna.

10.4.4 2.2. Architekturális jelöltek kiválasztása

A jelölteket a korábbi feltárásból (A tervezési folyamat: feltárás, alternatívák és döntés) és a PoC- vagy spike-eredményekből (Proof of Concept: célszerűség és értékelés, MVP és spike: tanulás a bizonytalanságban) származtassuk. Gyakorlati okokból 3–5 alternatíva kezelhető; ennél több jelölt esetén először egy durva szűrést érdemes végezni. A jelölteknek konkrétnak kell lenniük, nem csupán neveknek: „réteges monolit” helyett „réteges monolit a csomagok közötti szigorú függőségi szabályokkal, szinkron REST interfésszel” pontosabb. Így a pontozás vita során kevesebb a félreértés.

10.4.5 2.3. Összehasonlítási szempontok meghatározása

A szempontok a minőségi attribútumokból és a projektspecifikus kényszerekből származnak. A leggyakoribb szempontok közé tartoznak:

- **Karbantarthatóság**: milyen könnyű a kódot módosítani, a technikai adósságot kezelni.
- **Skálázhatóság**: hogyan viselkedik a rendszer a terhelés növekedésekor.
- **Csapat autonómiája**: mennyire tudnak a csapatok egymástól függetlenül szállítani.
- **Üzemeltetési komplexitás**: milyen nehéz a rendszert üzemeltetni, monitorozni, hibaelhárítani. Ezt gyakran fordított skálán pontozzák, ahol a magasabb érték jelenti az alacsonyabb komplexitást.
- **Fejlesztési sebesség**: mennyire gyorsan lehet új funkciót bevezetni.
- **Tranzakciós integritás**: milyen könnyű az ACID-tranzakciókat garantálni; elosztott rendszerekben ez nehezebb.
- **Infrastruktúra-költség**: a szükséges erőforrások, licencek, felhőköltségek becsült értéke. Szintén gyakran fordított skálán értékelik.
- **Biztonság és megfelelés**: mennyire egyszerű a hozzáférés-szabályozást, a naplózást és a szabályozási előírásokat teljesíteni.
- **Megbízhatóság**: milyen könnyű a hibatűrést biztosítani, a hibás komponenseket izolálni.

A szempontok listáját a konkrét kontextushoz kell igazítani, és minden szempontnál meg kell adni, mit jelent a magas pontszám. Ellenőrizni kell, hogy a szempontok ne legyenek átfedésben; ha például a karbantarthatóság és a fejlesztési sebesség erősen korrelál, érdemes csak az egyiket szerepeltetni, vagy összevonni őket.

10.4.6 2.4. Súlyozás

A súlyok azt fejezik ki, hogy az adott szempont mennyire fontos a döntés szempontjából. A legegyszerűbb módszer az 1–5 skála, ahol az 5 a kritikus fontosság, az 1 pedig a „jó, ha teljesül, de nem döntő”. Ennél strukturáltabb eljárás a páros összehasonlítás: minden szempontpárt összehasonlítunk, és a többszörösen preferált szempont kap nagyobb súlyt. Az analitikus hierarchia-folyamat (AHP) ezt formalizálja, de a legtöbb gyakorlati döntéshez elég az 1–5 skála és egy rövid indoklás.

A súlyozás során fontos, hogy a csapat – vagy az érintett stakeholderek – közösen állapodjanak meg. A súlyok vitája gyakran fontosabb, mint a pontszámoké, mert a súlyok tükrözik a szervezeti prioritásokat. Ha egy projektnél a szállítási sebesség a legfontosabb, akkor a csapat autonómiája és a fejlesztési sebesség magas súlyt kap, míg a skálázhatóság alacsonyabbat, ha a terhelés kicsi és stabil.

10.4.7 2.5. Pontozás

A pontozás során minden jelöltet minden szempont szerint értékelünk. Az egyszerű módszer a 0–5 skála, ahol:

- 0: a jelölt egyáltalán nem teljesíti a szempontot,
- 1: csak nagyon korlátozottan,
- 2: gyengén,
- 3: elfogadhatóan,
- 4: jól,
- 5: kiválóan.

A pontszámokat mindig indokolni kell, akár rövid szöveggel, akár a mátrixhoz kapcsolt forrással. Az indoklás lehet kvalitatív („a moduláris monolit a jól definiált modulhatárok miatt magas karbantarthatóságot ad”) vagy kvantitatív, ha rendelkezésre áll mérés („a PoC terhelésmérése 250 kérés/s-nál 95%-os válaszidőt 180 ms alatt tartott”). Ha a pontszám becslésen alapul, jelöljük „várt” értékként; ha mérésen, akkor „mért” értékként, a mérési környezet és időpont megadásával.

A pontozásnál a bizonytalanságot is rögzíteni kell. Egy érték lehet „3 ± 1”, ami azt jelenti, hogy a becslés körüli intervallum 2–4. Ez különösen fontos azoknál a szempontoknál, ahol nincs még mérés, vagy a technológia érettsége kérdéses.

10.4.8 2.6. Aggregálás

A leggyakoribb aggregálási módszer a súlyozott összeg:

$$S_j = \sum_{i=1}^{n} w_i \cdot s_{ij},$$

ahol S_j a j-edik alternatíva összesített pontszáma, w_i az i-edik szempont súlya, s_{ij} pedig a j-edik jelölt pontszáma az i-edik szempontra, n pedig a szempontok száma.

A súlyozott összeg egyszerű és átlátható. Hátránya, hogy elrejtheti az egyes dimenziók közötti kompenzációt: ha egy jelölt valamelyik szempontból nagyon rossz, de máshol kiváló, az átlag mégis jónak tűnhet. Ezért a mátrixot nem szabad pusztán a végső számra redukálni; mindig meg kell nézni a részeredményeket, a szórást és az érzékeny pontokat.

10.4.9 3. Súlyozás, pontozás és bizonytalanság

10.4.10 3.1. A súlyok meghatározásának gyakorlata

A súlyozás első lépése a szempontok fontossági sorrendbe állítása. Ezt a csapat közösen végezze el, lehetőleg a stakeholderekkel együtt. Egy egyszerű technika a MoSCoW-hoz hasonló osztályozás: a szempontokat „kritikus” (súly 5), „fontos” (4), „hasznos” (3), „elviselhető” (2) és „elhanyagolható” (1) kategóriába soroljuk. Ezután minden szempont kap egy súlyt, és a súlyok összege nem feltétlenül rögzített; a normalizálás csak a könnyebb összehasonlítást szolgálja.

Ha a vita nem vezet egyértelmű súlyokhoz, használhatunk páronkénti összehasonlítást. Készítsünk egy táblázatot, ahol minden szempontot minden másikkal összevetünk, és beírjuk, melyik a fontosabb a döntés szempontjából. A szempontok nyerésszáma arányos a súlyukkal. Ez az eljárás segít feloldani a holtversenyeket, mert a döntés kényszerűen konkrét párokra bomlik.

10.4.11 3.2. A pontozás és a becslési hiba

A pontszámok két forrásból származhatnak: mérésből vagy szakértői becslésből. A mérés erősebb, de ritkábban áll rendelkezésre, különösen a korai fázisban. A becslés során háromféle hiba léphet fel:

- **Szisztematikus hiba**: a pontozó túl optimista vagy túl pesszimista az adott mintával kapcsolatban, például mert kedveli a mikroszolgáltatásokat, és alábecsüli az üzemeltetési komplexitást.
- **Véletlen hiba**: az ismeretek hiányából adódó szórás, amelyet intervallummal lehet kifejezni.
- **Kihagyott szempont**: ha egy releváns dimenzió nem szerepel a mátrixban, az összesített pontszám torzít.

A becslési hiba kezelésére a következő gyakorlatok ajánlottak:

- Minden pontszámhoz adjunk egy **intervallumot** vagy legalább egy bizonytalansági jelölést (pl. „alacsony”, „közepes”, „magas”).
- A súlyozott összeget ne egyetlen számként, hanem **intervallumként** is számítsuk ki, a pontszámok minimális és maximális értékének figyelembevételével.
- Végezzünk **érzékenységvizsgálatot**: változtassuk meg egy-egy súlyt vagy pontszámot, és nézzük meg, hogy a jelöltek sorrendje megfordul-e. Ha igen, akkor a döntés érzékeny arra a paraméterre, és további mérésre vagy pontosabb becslésre van szükség.

10.4.12 3.3. Mérés és becslés elkülönítése

A döntési mátrixban minden pontszám mellett egyértelműen jelölni kell, hogy az érték mért vagy várt. A mért értékhez tartoznia kell a mérési környezetnek, konfigurációnak, időpontnak és bizonytalanságnak – összhangban a könyv labor- és kódszerződésével. A várt érték ezzel szemben becslés, amelynek forrása szakértői vélemény, analógia vagy korábbi projekt tapasztalata. A kettő összekeverése gyakori hiba; például ha egy PoC során egyetlen komponens válaszidejét mérjük, azt nem szabad az egész rendszerre kivetíteni anélkül, hogy a mérési határokat át nem lépnénk.

A laborban ezt a szétválasztást kifejezetten gyakoroljuk: egy oszlopban fel kell tüntetni, hogy az adott pontszám „mért (PoC 2025-04-10, 100 kérés/s, 5 perc)” vagy „várt (szakértői becslés, 2025-04-11)”.

10.4.13 4. A döntés dokumentálása és felülvizsgálata

10.4.14 4.1. A döntési mátrix mint az ADR melléklete

A döntési mátrix önmagában nem helyettesíti az architektúradöntési rekordot (ADR), hanem annak egyik bemenete. Az ADR az Architecture Decision Records (ADR) és döntési napló alfejezetben tárgyalt szerkezetben rögzíti a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. A mátrix a döntés indoklásának része, ezért érdemes az ADR-hez csatolni, vagy a döntési naplóban hivatkozni rá. Egy jó gyakorlat, hogy az ADR szövege nem ismétli meg a mátrix minden számát, hanem utal rá, és kiemeli a döntés szempontjából legfontosabb három-négy szempontot.

Példa rövid ADR-részletre:

> **Cím:** ADR-014: A visszáru-kezelő rendszer architekturális stílusának kiválasztása **Állapot:** Elfogadott **Kontextus:** A rendszer 80 belső felhasználót szolgál ki, csúcsterhelés 40 kérés/s, rendelkezésre állási cél 99,5% havi ablakban. A fejlesztőcsapat két streamre oszlik, nincs külön platformcsapat. A technológiai stack Java és PostgreSQL. **Döntés:** Moduláris monolit architektúrát választunk, a modulok közötti szinkron REST interfésszel és a tranzakciós integritás megtartásával. **Alternatívák:** Réteges monolit, moduláris monolit, mikroszolgáltatások. A döntési mátrixot a melléklet tartalmazza; az érzékenységvizsgálat szerint a moduláris monolit előnye a skálázhatósági súly 5-re növelése esetén csökken, de a döntés stabil marad. **Következmények:** A csapatok továbbra is közös adatbázist használnak, ami korlátozza a teljesen független szállítást, de csökkenti az üzemeltetési komplexitást. A későbbi szétválasztás lehetőségét a modulhatárok karbantartásával biztosítjuk. **Felülvizsgálat:** A döntést félévente, illetve új terhelési adat vagy szervezeti változás esetén felül kell vizsgálni.

10.4.15 4.2. A döntés felülvizsgálata

A mintaválasztás nem egyszeri esemény, hanem evolúciós tevékenység. A kontextus változhat: nőhet a terhelés, bővülhet a csapat, új szabályozás léphet életbe, vagy egy technológia elavulhat. Ezért minden döntési mátrixhoz meg kell határozni azokat a **kiváltó feltételeket**, amelyek a felülvizsgálatot indokolják. Ilyen feltétel lehet például:

- az átlagos terhelés három hónapos mozgóátlaga meghaladja az előre jelzett értéket legalább 50%-kal,
- a csapatlétszám 10%-ot meghaladóan növekszik,
- a rendelkezésre állási SLO-t két egymást követő hónapban nem teljesíti a rendszer,
- új funkcionális követelmény jelentkezik, amely a jelenlegi stílusban aránytalanul költséges lenne.

A felülvizsgálat során a meglévő mátrixot frissítsük az új adatokkal, és ne féljünk megváltoztatni a döntést. Az architektúradöntések verziózottak, ezért a régi ADR-ek állapota „érvénytelenített” lesz, és hivatkozni kell az új ADR-re.

10.4.16 5. Gyakori hibák és csapdák

10.4.17 5.1. Hamis pontosság

A döntési mátrix számai azt a benyomást kelthetik, hogy a döntés objektíven számítható. Ez tévedés. A pontszámok becslések, a súlyok értékítéletek, a szempontok pedig nem mindig

függetlenek. Ha valaki a súlyozott összeget két tizedesjegyre számítja, és az alapján hoz döntést, az hamis pontosság. A helyes gyakorlat az intervallumok és az érzékenységvizsgálat használata, valamint annak tudatosítása, hogy a mátrix segédeszköz, nem döntéshozó automatika.

10.4.18 5.2. Egyoldalú szempontkészlet

Gyakori hiba, hogy a csapat csak a fejlesztési szempontokat veszi figyelembe, az üzemeltetést vagy a biztonságot nem. Egy minta lehet kiváló a fejlesztők számára, de ha az üzemeltetési csapat nem tudja kezelni, a produkcióban kudarc lesz. Ezért a szempontok kiválasztásába minden érintett szerepkört be kell vonni: fejlesztők, üzemeltetők, biztonsági szakértők, termékmenedzserek, adatgazdák.

10.4.19 5.3. A kontextus elhanyagolása

A mátrix pontszámai csak a rögzített kontextusban értelmezhetők. Ha a kontextust nem dokumentálják, a későbbi olvasó nem tudja, miért kapott egy jelölt magas pontszámot. A kontextus változásával a mátrix érvényét veszti; ezt a felülvizsgálati feltételekben kell kezelni.

10.4.20 5.4. Presztízs- vagy divatdöntés

A mátrix alkalmas arra, hogy a divatos megoldások mögé nézzen. A mikroszolgáltatások gyakran magas pontszámot kapnak a skálázhatóságra és a csapat autonómiájára, de ha a csapat kicsi és a terhelés alacsony, az üzemeltetési komplexitás és a tranzakciós integritás szempontjából rosszul szerepelnek. A mátrix éppen a szempontok egyensúlyával ellensúlyozza a hype-ot.

10.4.21 5.5. A pontszámok indoklásának hiánya

Ha a pontszámokat nem indokolják, a vita nem a tényekről, hanem a számokról szól. A pontszámoknak mindig legyen forrása: mérés, szakértői vélemény, analógia vagy dokumentált tapasztalat. Az indoklás nélküli pontszámok egy idő után önkényesnek tűnnek, és a döntés legitimitása csökken.

10.4.22 6. Labor: döntési mátrix három architektúrára

10.4.23 6.1. Cél és kiinduló állapot

A labor célja, hogy egy konkrét kontextusban három architekturális jelöltet összehasonlítson döntési mátrixszal, a súlyozást, a pontozást, a bizonytalanságot és a dokumentációt gyakorolva. A kiinduló állapot a következő:

- A kontextus leírása (lásd az 6.2 pontot).
- Három architektúra-jelölt: **A** – réteges monolit, **B** – moduláris monolit, **C** – mikroszolgáltatások.
- Egy korábbi PoC eredménye, amely a skálázhatósági szempontra mért adatot szolgáltatott.
- A csapat által egyeztetett szempontlista és súlyok (a laborban ezeket a hallgatók reprodukálhatják, vagy saját súlyokat adhatnak).

Az elvárt eredmény: egy reprodukálható dokumentum vagy Jupyter-jegyzetfüzet, amely tartalmazza a kontextust, a mátrixot, a számításokat, az érzékenységvizsgálatot, a döntési napló bejegyzést (ADR-t), és egyértelműen elkülöníti a mért és várt értékeket.

A visszaállítási út: mivel ez nem éles rendszer, nincs szükség infrastruktúra-visszaállításra. A labor dokumentumait verziózva kell tárolni.

10.4.24 6.2. A kontextus

> Egy belső visszáru-kezelő rendszert kell kialakítanunk. A felhasználók száma 80 belső munkatárs, a csúcsterhelés 40 kérés másodpercenként, az átlagos terhelés 5 kérés másodpercenként. A rendelkezésre állási cél 99,5% havi ablakban, azaz az error budget $1 - 0{,}995 = 0{,}005$, ami havonta legfeljebb körülbelül 3,6 óra állásidőt jelent. A fejlesztőcsapat két, egyenként 6 fős streamre oszlik, a technológiai stack Java és PostgreSQL. Az üzleti logika évente 6–8 nagyobb módosítást igényel, a szállítási ciklus kéthetes. Nincs külön platformcsapat; az üzemeltetést a fejlesztők végzik. A rendszernek integrálódnia kell a vállalat meglévő ERP-jéhez, amely SOAP-webszolgáltatáson keresztül érhető el (helyőrző: `https://erp.invalid/returns`). A költségvetés fix, a felhő- vagy on-premise infrastruktúra költségeit a fejlesztőknek kell becsülniük.

10.4.25 6.3. A szempontok és súlyok

A csapat az alábbi szempontokat és súlyokat rögzítette (a súlyok 1–5 skálán, ahol 5 a kritikus fontosság):

Szempont	Súly	Megjegyzés
Karbantarthatóság	4	Az üzleti logika gyakran változik.
Skálázhatóság	3	A terhelés alacsony, de a jövőben nőhet.
Csapat autonómia	3	Két stream dolgozik párhuzamosan.
Üzemeltetési komplexitás (fordított)	4	Nincs külön platformcsapat, ezért fontos az egyszerű üzemeltetés.
Fejlesztési sebesség	5	A kéthetes ciklusok gyors szállítást igényelnek.
Tranzakciós integritás	4	A visszáru-kezelés ACID-tranzakciókat igényel.
Infrastruktúra-költség (fordított)	3	Fix költségvetés, a becslés fontos.

A „fordított” jelölés azt jelenti, hogy a pontozásnál a magasabb pontszám kedvezőbb, azaz alacsonyabb komplexitást vagy költséget jelent.

10.4.26 6.4. A pontszámok és azok forrása

A pontszámok 0–5 skálán, ahol az 5 a legjobb. A táblázatban minden pontszám mellett jelöljük a forrást: **M** = mért (a PoC-ban), **V** = várt (szakértői becslés). A PoC mérési környezete: a moduláris monolit prototípusa, egyetlen 4 vCPU-s, 16 GB RAM-os konténerben, 100 kérés/s terhelés mellett, 5 perces ablakban, 2025-04-10-én. A mérés eredménye: 95%-os válaszidő 180 ms, maximális válaszidő 350 ms, hibanélkül.

A mikro-szolgáltatások esetében a skálázhatósági pontszám várt érték, mivel nem készült PoC; a becslés a szakirodalom és csapattapasztalat alapján történt.

Szempont	A: Réteges monolit	B: Moduláris monolit	C: Mikroszolgáltatások
Karbantarthatóság	3 (V)	4 (V)	4 (V)
Skálázhatóság	2 (V)	3 (M)	5 (V)
Csapat autonómia	2 (V)	3 (V)	5 (V)
Üzemeltetési komplexitás (fordított)	5 (V)	4 (V)	2 (V)
Fejlesztési sebesség	3 (V)	4 (V)	4 (V)
Tranzakciós integritás	5 (V)	5 (V)	3 (V)
Infrastruktúraköltség (fordított)	5 (V)	4 (V)	2 (V)

A mért és várt értékek szétválasztása itt azt jelenti, hogy a skálázhatóságnál a B jelölt pontszáma a PoC-ból származik, míg a többi érték becslés. A laborban ezt az elkülönítést tovább lehet finomítani, például minden becslésnél intervallumot adva.

10.4.27 6.5. A súlyozott összeg kiszámítása

A súlyozott összeget a fenti képlettel számítjuk. A számítás elvégezhető kézzel, táblázatkezelőben vagy Jupyter-notebookban. Az alábbi Python-kód szemlélteti a számítást (a kód nem futtatandó éles rendszeren, csak a laborban):

```
# Döntési mátrix számítás (szemléltetés)
suly = {
    "karbantarthatosag": 4,
    "skalazhatosag": 3,
    "csapat_autonomia": 3,
    "uzemeltetesi_komplexitas_forditott": 4,
    "fejlesztesi_sebesseg": 5,
    "tranzakcios_integritas": 4,
    "infrastruktura_koltseg_forditott": 3,
}

pontszamok = {
    "A": {"karbantarthatosag": 3, "skalazhatosag": 2, "csapat_autonomia": 2,
          "uzemeltetesi_komplexitas_forditott": 5, "fejlesztesi_sebesseg": 3,
          "tranzakcios_integritas": 5, "infrastruktura_koltseg_forditott": 5},
    "B": {"karbantarthatosag": 4, "skalazhatosag": 3, "csapat_autonomia": 3,
          "uzemeltetesi_komplexitas_forditott": 4, "fejlesztesi_sebesseg": 4,
          "tranzakcios_integritas": 5, "infrastruktura_koltseg_forditott": 4},
    "C": {"karbantarthatosag": 4, "skalazhatosag": 5, "csapat_autonomia": 5,
          "uzemeltetesi_komplexitas_forditott": 2, "fejlesztesi_sebesseg": 4,
          "tranzakcios_integritas": 3, "infrastruktura_koltseg_forditott": 2},
}
```

```
def sulyozott_osszeg(pont):
    return sum(suly[sz] * pont[sz] for sz in suly)

for jelolt, pont in pontszamok.items():
    print(f"{jelolt}: {sulyozott_osszeg(pont)}")
```

A fenti kód kimenete (számított, nem mért):

- A: $4 \cdot 3 + 3 \cdot 2 + 3 \cdot 2 + 4 \cdot 5 + 5 \cdot 3 + 4 \cdot 5 + 3 \cdot 5 = 94$
- B: $4 \cdot 4 + 3 \cdot 3 + 3 \cdot 3 + 4 \cdot 4 + 5 \cdot 4 + 4 \cdot 5 + 3 \cdot 4 = 102$
- C: $4 \cdot 4 + 3 \cdot 5 + 3 \cdot 5 + 4 \cdot 2 + 5 \cdot 4 + 4 \cdot 3 + 3 \cdot 2 = 92$

Ebben a kontextusban a B (moduláris monolit) kapja a legmagasabb pontszámot, de a különbség a C-hez képest 10 pont, ami az érzékenységvizsgálat fényében nem feltétlenül robusztus.

10.4.28 6.6. Érzékenységvizsgálat és bizonytalanság

Az érzékenységvizsgálathoz változtassuk meg egyes súlyokat, és figyeljük a sorrendet. Ha a skálázhatóság súlyát 3-ról 5-re növeljük, miközben a többi súly változatlan marad (a súlyok összege nő, de ez a sorrendet nem befolyásolja, ha minden jelöltre azonos módon számolunk), akkor az új eredmények:

- A: $4 \cdot 3 + 5 \cdot 2 + 3 \cdot 2 + 4 \cdot 5 + 5 \cdot 3 + 4 \cdot 5 + 3 \cdot 5 = 12 + 10 + 6 + 20 + 15 + 20 + 15 = 98$
- B: $4 \cdot 4 + 5 \cdot 3 + 3 \cdot 3 + 4 \cdot 4 + 5 \cdot 4 + 4 \cdot 5 + 3 \cdot 4 = 16 + 15 + 9 + 16 + 20 + 20 + 12 = 108$
- C: $4 \cdot 4 + 5 \cdot 5 + 3 \cdot 5 + 4 \cdot 2 + 5 \cdot 4 + 4 \cdot 3 + 3 \cdot 2 = 16 + 25 + 15 + 8 + 20 + 12 + 6 = 102$

A B továbbra is vezet, de a C felzárkózik. Ha emellett a csapat autonómia súlyát is 3-ról 5-re emeljük, akkor a C megelőzheti a B-t. Ez azt mutatja, hogy a döntés érzékeny a skálázhatóság és az autonómia súlyára. A csapatnak ezt a bizonytalanságot figyelembe kell vennie: ha a terhelés növekedését valószínűnek tartják, és a csapatlétszám növelését tervezik, akkor a mikroszolgáltatások felé billenhet a mérleg, de a jelenlegi, alacsony terhelésű és kis csapatos kontextusban a moduláris monolit az indokoltabb.

A bizonytalanság intervallumokkal is kifejezhető. Például a C jelölt karbantarthatósága 4 ± 1, az üzemeltetési komplexitása 2 ± 1. Az intervallumokat a súlyozott összegbe beillesztve a C pontszáma 84 és 100 között mozoghat, ami átfedésben van a B-vel. Ez megerősíti, hogy a C-t nem lehet egyértelműen elvetni pusztán a pontszám alapján.

10.4.29 6.7. Döntési napló és ellenőrzési szempontok

A laborban a döntést ADR formában kell rögzíteni, a fent bemutatott módon. Az ADR-nek tartalmaznia kell:

- a kontextus leírását,
- a döntést és annak indoklását a mátrixra hivatkozva,
- az alternatívákat és azok elutasításának okait,
- a következményeket, beleértve a kockázatokat és a felülvizsgálati feltételeket,
- az állapotot (pl. „Elfogadott”).

Az ellenőrzési szempontok az alábbiak:

- A kontextus rögzítve van, benne a terhelési, rendelkezésre állási és csapatjellemzőkkel.

- A mátrix minden pontszámához tartozik forrás (mért vagy várt), és a mért értéknél a mérési környezet, időpont és bizonytalanság szerepel.
- A súlyok indoklása megtalálható, és a fontossági sorrendet a csapat közösen alakította ki.
- A számítás ellenőrizhető, akár kézzel, akár a reprodukálható kóddal.
- A szenzitivitásvizsgálat eredménye szerepel, és megmutatja, mely paraméterekre érzékeny a döntés.
- A döntést ADR dokumentálja, és az ADR hivatkozik a mátrixra.

10.4.30 6.8. A labor elvárt eredménye

A hallgatóknak egy Jupyter-jegyzetfüzetet vagy verziózott Markdown-dokumentumot kell készíteniük, amely a fenti lépéseket követi. A dokumentumnak tartalmaznia kell:

- a kontextus szöveges leírását,
- a szempontok és súlyok táblázatát,
- a pontozási táblázatot a forrásokkal,
- a súlyozott összeg számítását képlettel és/vagy kóddal,
- az érzékenységvizsgálatot legalább két paraméterre,
- az ADR-t,
- és egy rövid következtetést arról, hogy a mátrix alapján melyik jelöltet választanák, és milyen feltételekkel változtatnák meg a döntést.

A mért és a várt eredményeket minden szempontnál külön oszlopban vagy jelöléssel kell elkülöníteni.

10.4.31 7. Ellenőrzőlista a döntési mátrix elkészítéséhez

Ez az ellenőrzőlista segít abban, hogy ne hagyjanak ki fontos lépést a mintaválasztás során.

- ☐ A döntési kontextus rögzítve: funkcionális követelmények, minőségi attribútumok, csapatméret, terhelés, rendelkezésre állási cél, költségkeret és szabályozási környezet.
- ☐ Legalább három architektura-jelölt definiálva, konkrét tervezési döntésekkel.
- ☐ Az összehasonlítási szempontok átfedésmentesek, és minden szempontnál egyértelmű a jó irány.
- ☐ A súlyok 1–5 skálán rögzítve, indoklással, a csapat közös döntése alapján.
- ☐ Minden jelölt minden szempontra pontozva, a forrás (mért/várt) jelölve.
- ☐ A mért értékeknél a mérési környezet, időpont és bizonytalanság megadva.
- ☐ A súlyozott összeg kiszámítva, intervallumokkal a bizonytalan pontszámoknál.
- ☐ Érzékenységvizsgálat elvégezve legalább két súly vagy pontszám változtatásával.
- ☐ A döntést ADR dokumentálja, hivatkozással a mátrixra és a felülvizsgálati feltételekre.
- ☐ A dokumentum verziózott, és a mátrix reprodukálható a mellékelt kóddal vagy táblázattal.

10.4.32 8. Önellenőrző kérdések

1. Miért fontos a döntési kontextust a mátrix előtt rögzíteni? Adj egy példát, amikor ugyanaz a minta más kontextusban más pontszámot kapna.
2. Hogyan különíted el a mért és a várt értékeket a döntési mátrixban? Mi a veszélye annak, ha a kettőt összekevered?
3. Milyen módszerrel lehet a bizonytalanságot a súlyozott összegbe beépíteni? Miért fontos az érzékenységvizsgálat?

4. Milyen feltételek indokolhatják egy döntési mátrix felülvizsgálatát? Hogyan kapcsolódik ez az ADR-ek verziózásához?

Ezekkel a kérdésekkel ellenőrizhető, hogy az olvasó megértette-e a mintaválasztás és a döntési mátrixok kontextusfüggő alkalmazását, a súlyozás és pontozás módszertanát, valamint a dokumentáció és felülvizsgálat jelentőségét. A következő fejezetek – különösen a Rétegezett architektúra és határai alfejezettől kezdve a rétegezett és monolitikus architektúrák – ezekre a döntési keretekre építenek, amikor az egyes stílusokat részletesen bemutatják és értékelik.

III. RÉSZ – Architektúrális stílusok és minták

11 Rétegezett, monolitikus és moduláris architektúrák

11.1 Rétegezett architektúra és határai

A rétegezett architektúra a szoftvertervezés egyik legrégebbi és legszélesebb körben használt szerkezeti mintája. Lényege, hogy a rendszer komponenseit vízszintes rétegekbe szervezzük, ahol minden réteg az absztrakció egy-egy szintjét képviseli, és a rétegek közötti kommunikáció szigorúan meghatározott irányban, szerződéseken keresztül zajlik. A rétegek nem fizikai egységek, hanem logikai csoportosítások: egy rétegen belül több komponens is élhet, és egy réteg több telepítési egységre is szétoszthatható, de a forráskód szintjén jól azonosítható határokat alkot.

A klasszikus háromrétegű felosztás a bemutató (presentation), az üzleti logika (business logic, más néven alkalmazás vagy domain) és az adatelérés (data access) réteg. Ezt gyakran kiegészíti egy integrációs vagy infrastruktúra réteg, amely a külső rendszerekkel, üzenetküldéssel, fájlrendszerrel vagy adatbázissal való kapcsolatot kezeli. A rétegeknek nem csak nevük van, hanem felelősségük is: a bemutató réteg a felhasználói interakciót és a külső API-kat kezeli; az alkalmazás réteg a felhasználói műveleteket koordinálja, tranzakciókat indít; a domain réteg az üzleti szabályokat és a fogalmakat tartalmazza; az infrastruktúra réteg pedig a technikai részleteket, például az adatbázis-elérést, a hálózati hívásokat és a naplózást valósítja meg.

A **függőségi irány** a rétegezett architektúra legfontosabb szabálya. Alapértelmezés szerint egy felső réteg függhet az alatta lévő rétegtől, de az alsó réteg soha nem függhet a felette lévőtől. Ez a szabály a fordítási idejű függőségre vonatkozik: a felső réteg osztályai hivatkozhatnak az alsó réteg típusaira, az alsó réteg azonban nem ismerheti a felső réteget. A cél a körkörös függőségek elkerülése és a rétegek cserélhetőségének, tesztelhetőségének megőrzése. Ha például a bemutató réteg a domain réteget hívja, a domain rétegnek nem szabad tudnia arról, hogy egy webes kontroller vagy egy asztali alkalmazás hívja; a domain réteg csak a saját nyelvén, a saját interfészein keresztül kommunikál.

A gyakorlatban a függőségi irány nem mindig egyenletesen „lefelé” mutat. A domain réteg üzleti szabályai gyakran függetlenek kell legyenek az adatbázis-kezelőtől vagy a hálózati protokolltól. Ezt a függőségi inverzió (Dependency Inversion) elvének alkalmazásával érhetjük el: a domain réteg definiál egy interfészt (például `IRendelésRepository`), az infrastruktúra réteg pedig implementálja ezt az interfészt. Így a forráskód szintjén az infrastruktúra réteg függ a domain rétegtől, nem pedig fordítva. Ez nem sérti a rétegezés alapelvét, mert a függőségi irány a magasabb absztrakciós szint (domain) felől az alacsonyabb szintű implementáció felé mutat, de a technikai részletek kikerülnek az üzleti logikából. Ezt a későbbi fejezetekben részletesebben tárgyaljuk, itt elég annyi, hogy a rétegek közötti szerződések tervezésekor ezt a lehetőséget figyelembe kell venni.

A **rétegek közötti szerződés** az a nyilvános felület, amelyen keresztül a rétegek kommunikálnak. Egy szerződés lehet interfész, absztrakt osztály, DTO (data transfer object) vagy akár eseménytípus. A szerződésnek egyértelműen meg kell határoznia a bemenetet, a kimenetet, a hibakezelést és a tranzakciós viselkedést. Például egy `RendelésService` a `RendelésController` felé a `HozzonLétreRendelést(RendelésParancs)` metódust kínálja, ahol a `RendelésParancs`

egy adatátviteli objektum, nem pedig a domain réteg belső entitása. A szerződés mögötti implementáció változhat, amíg a szerződés változatlan marad; ez teszi lehetővé a rétegek egymástól független fejlesztését és tesztelését.

A rétegek közötti kommunikáció alapértelmezés szerint csak a közvetlenül szomszédos rétegek között engedélyezett. A bemutató réteg nem hívhatja közvetlenül az adatelérési réteget, hanem mindig az alkalmazás vagy domain rétegen keresztül kell haladnia. Ez a szabály a **zárt rétegek** fogalmához vezet.

Zárt rétegnek nevezzük azt a réteget, amelyen minden hívásnak át kell haladnia, és amely nem ugorható át. A klasszikus háromrétegű architektúrában az üzleti logika réteg általában zárt: a bemutató réteg nem érheti el közvetlenül az adatbázist, mert akkor megkerülné az üzleti szabályokat, tranzakciókezelést és jogosultságellenőrzést. A zártság biztosítja, hogy a réteg invariánsai minden kérésre érvényesek.

Nyitott réteg ezzel szemben olyan réteg, amelyet a felette lévő réteg indokolt esetben átugorhat. Például egy gyorsítótár (cache) réteg lehet nyitott: ha a bemutató rétegnek nagy teljesítményű olvasási útra van szüksége, közvetlenül a cache-t hívhatja, nem kell végigmennie a teljes üzleti logikán. Ugyanígy egy vékony API gateway réteg nyitott lehet a belső hívások számára. A nyitott réteg használata tudatos döntés, amelyet dokumentálni kell, mert csökkenti a rétegezés által nyújtott védelmet, és növeli a csatolást. A nyitott és zárt rétegek kombinációja lehetővé teszi, hogy a rétegezett architektúra rugalmasan alkalmazkodjon a teljesítmény- és egyszerűségi követelményekhez anélkül, hogy a teljes szerkezetet feladnánk.

A rétegezett architektúra **előnyei** közé tartozik az áttekinthetőség és a felelősségek szétválasztása. A fejlesztők könnyebben eligazodnak, mert minden komponensnek jól meghatározott helye van. A rétegek cserélhetők: ha az adatbázis-kezelőt le kell cserélni, elég az adatelérési réteget módosítani, feltéve, hogy a szerződések stabilak. A tesztelés is egyszerűbbé válik, mert a rétegek izoláltan, mockokkal vagy stubokkal tesztelhetők. A párhuzamos fejlesztést is támogatja, mert a rétegek felelősségi határai mentén a csapatok feloszthatják a munkát.

A rétegezett architektúra tesztelhetőségi előnyét a következő minőségi forgatókönyvvel szemléltethetjük:

- **Inger:** Egy fejlesztő módosítja az adatelérési réteg SQL-lekérdezését.
- **Környezet:** Fejlesztési környezet, a rétegek közötti interfész változatlan.
- **Válasz:** A bemutató, alkalmazás és domain rétegben nem szükséges módosítás.
- **Mérőszám:** A felső rétegekben módosított fájlok száma = 0, 10 egymást követő módosítás átlagában.

A rétegezés azonban **merevséget** is okozhat. A leggyakoribb anti-minta az úgynevezett „lasagna architektúra”: túl sok réteg, amelyek közül sok csak továbbítja a hívásokat anélkül, hogy érdemi logikát adna hozzá. Ez felesleges indirekciót, nagyobb kódmennyiséget és lassabb változtatásokat eredményez. Ha például egy egyszerű mező hozzáadásához az összes rétegen végig kell vezetni a változást, a rétegezés költsége meghaladja az előnyét. A merevség másik forrása a rétegek közötti szoros csatolás: ha a szerződések nem elég stabilak, vagy a rétegek átnyúlnak egymásba, a rétegek cserélhetősége illúzióvá válik.

A rétegezett architektúra tehát nem univerzális jó megoldás, hanem egy eszköz, amelyet a rendszer összetettsége, a csapat mérete és a várható változások fényében kell megválasztani. Kis, egyszerű alkalmazásoknál a merev rétegezés gyakran túlzás; ilyenkor egyetlen modul vagy egy minimális rétegzés is elegendő. Nagy, hosszú életciklusú, többszereplős rendszereknél viszont a rétegek fegyelme segít a káosz elkerülésében. A döntést mindig kontextussal együtt kell meghozni és dokumentálni.

Nézzünk egy konkrét példát: egy egyszerű e-kereskedelmi rendeléskezelő szolgáltatás. A rendszer fogadja a rendelési kéréseket, érvényesíti az üzleti szabályokat, és elmenti a rendeléseket egy adatbázisba. A rétegek:

- **Bemutató réteg**: `RendelésController`, amely HTTP POST kéréseket fogad.
- **Alkalmazás réteg**: `RendelésService`, amely koordinálja a műveletet, tranzakciót indít, és meghívja a domain logikát.
- **Domain réteg**: `Rendelés` entitás és `RendelésValidáló` üzleti szabály.
- **Infrastruktúra réteg**: `SqlRendelésRepository`, amely az `IRendelésRepository` interfészt implementálja, és az adatbázishoz kapcsolódik.

A függőségi gráf a következő Mermaid diagrammal szemléltethető:

```
flowchart TB
    subgraph ui["Bemutató réteg"]
        WebAPI["RendelésController"]
    end
    subgraph app["Alkalmazás réteg"]
        Service["RendelésService"]
    end
    subgraph domain["Domain réteg"]
        Entity["Rendelés entitás"]
        Validator["RendelésValidáló"]
        RepoInterface["IRendelésRepository"]
    end
    subgraph infra["Infrastruktúra réteg"]
        RepoImpl["SqlRendelésRepository"]
        Db[("Adatbázis")]
    end
    WebAPI --> Service
    Service --> Entity
    Service --> Validator
    Service --> RepoInterface
    RepoImpl --> RepoInterface
    RepoImpl --> Db
```

Cím: Rendeléskezelő szolgáltatás rétegezett függőségi gráfja. **Jelmagyarázat:** A nyilak a fordítási idejű függőségeket jelölik; a dobozok komponensek, a henger külső adatbázis. **Feltételjegyzet:** A diagram a C4 komponens szintjét egyszerűsíti; a rendszerhatár a szolgáltatás határa, a külső adatbázis külső rendszerként jelenik meg. A függőségi irány a bemutató rétegtől az alkalmazás és domain felé, az infrastruktúra pedig a domain interfésztől függ, nem fordítva.

A diagramon jól látható, hogy a `RendelésController` csak a `RendelésService`-t ismeri, a `RendelésService` a domain entitást, a validátort és a repository interfészt, az `SqlRendelésRepository` pedig az interfészt és az adatbázist. A domain réteg nem függ az adatbázistól, mert a repository interfész a domain rétegben van. Ez a felépítés biztosítja, hogy a domain logika adatbázis nélkül, memóriában tesztelhető legyen.

A rétegek közötti szerződés ebben a példában a `RendelésService` és a `RendelésController` közötti DTO, például a `RendelésParancs` és a `RendelésVálasz`, valamint a `RendelésService` és a repository közötti `IRendelésRepository` interfész. A szerződésnek egyértelműnek kell lennie: a `RendelésParancs` tartalmazza a megrendelt tételeket, a `RendelésVálasz` a létrehozott rendelés azonosítóját és állapotát. A hibakezelés szerződésben rögzített: ha az üzleti szabály megsértése történik, a szolgáltatás `RendelésÉrvénytelen` kivételt dob, amelyet a kontroller HTTP 400-as válaszként jelenít meg.

A zárt rétegek alkalmazása itt azt jelenti, hogy a `RendelésController` nem férhet közvetlenül az `IRendelésRepository`-hoz vagy az adatbázishoz; minden kérésnek át kell haladnia a

`RendelésService`-en. Ez a réteg zártsága biztosítja a tranzakciókezelés és a jogosultságellenőrzés egységes végrehajtását. Ha a teljesítmény miatt a kontroller közvetlenül a cache-t szeretné elérni, akkor a cache réteget nyitottá tehetjük, de ezt ADR-ben rögzíteni kell.

Labor: rétegezett rendszer függőségi gráfja

Feladat: Válasszon egy kis rendszert (például könyvtárkatalógus, rendeléskezelő, fórum). Készítse el a rendszer rétegezett függőségi gráfját diagrams-as-code formában (például Mermaid vagy PlantUML). A gráf mutassa a rétegeket, a komponenseket, a függőségi irányokat és a rétegek közötti szerződéseket. A gráf alapján ellenőrizze a rétegezési szabályok betartását, és rögzítse a döntéseket ADR formátumban.

Kiinduló állapot: A hallgató rendelkezik egy üres projektmappával, egy választott rendszer rövid szöveges leírásával, és egy döntési napló sablonnal. A sablon a következő ADR mezőket tartalmazza: azonosító, cím, kontextus, döntés, alternatívák, következmények, állapot, dátum.

Elvárt eredmény: - Egy verziózott Markdown dokumentum, amely tartalmazza a rendszer kontextusát, a választott rétegeket, a függőségi gráfot (Mermaid kódblokkban), a szerződések listáját, és legalább egy ADR-t a rétegezési döntésről. - A gráf elemzése: a hallgató azonosítja a függőségi irányokat, ellenőrzi, hogy nincs felfelé mutató függőség, nincs ciklus, és a zárt rétegeket nem ugrották át. - A mért és várt eredmények szétválasztása: például „Mért ciklusok száma: 0 (várt: 0)”, „Mért felfelé mutató függőségek: 0 (várt: 0)”, „Mért átugrott zárt rétegek: 0 (várt: 0)”. Ha eltérés van, a hallgatónak ki kell javítania a gráfot vagy indokolnia kell a nyitott réteg használatát.

Ellenőrzési szempontok: - A függőségi gráf diagrams-as-code formában van, és verziózható. - A rétegek felelőssége egyértelműen le van írva. - A szerződések interfész vagy DTO szinten definiáltak. - Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket. - A mért és várt eredmények külön vannak jelölve.

Visszaállítási út: Mivel a labor nem hajt végre destruktív műveletet, visszaállításra nincs szükség; ha a hallgató hibás gráfot készített, a dokumentum szerkesztésével javítható.

A laborhoz kapcsolódóan a döntési napló példája:

```
# ADR-101: Zárt rétegek alkalmazása a rendeléskezelőben

- **Kontextus:** A rendeléskezelő szolgáltatás három fő rétegből áll: bemutató, alkalmazás,
 ↪  domain és infrastruktúra. A bemutató réteg hajlamos közvetlenül az adatbázishoz fordulni
 ↪  egyszerű lekérdezések miatt.
- **Döntés:** A bemutató réteg csak az alkalmazás réteget hívhatja; az alkalmazás réteg zárt,
 ↪  tehát nem ugorható át. A domain réteg nem függ az infrastruktúrától; a repository interfészt
 ↪  a domain tartalmazza.
- **Alternatívák:** (1) Nyitott alkalmazás réteg, amely lehetővé teszi a bemutató közvetlen
 ↪  adatelérését. (2) A domain réteg közvetlen függése az adatbázistól.
- **Következmények:** Nő a fegyelem és a boilerplate, de a tranzakciókezelés és a validáció
 ↪  egységes marad. A tesztelés egyszerűbb, mert a domain memóriában tesztelhető.
- **Állapot:** Elfogadva.
```

Gyakorlat: Adott egy háromrétegű alkalmazás, ahol a bemutató réteg közvetlenül az adatbázist hívja egy gyakran használt, csak olvasható lekérdezés miatt. A csapat a teljesítmény javítása érdekében ezt a hívást közvetlenül az adatbázisra irányítja. Milyen rétegezési szabályt sért ez? Milyen alternatív megoldást javasolna a rétegezés megtartásával? Válaszát indokolja, és készítsen egy rövid ADR-t a döntéshez.

Ellenőrzőlista rétegezett architektúra értékeléséhez: - A rétegek felelőssége egyértelmű, és minden komponens pontosan egy réteghez tartozik. - A függőségi irány szabálya definiált, és a forráskódban ellenőrizhető (például architekturális tesztekkel). - A rétegek közötti szerződések

interfész vagy DTO szinten rögzítettek, nem belső típusokon alapulnak. - A zárt rétegek azonosítva vannak, és nincs indokolatlan átugrás. - A nyitott rétegek használata ADR-ben indokolt. - Nincs körkörös függőség a rétegek között. - A merevség kockázatát értékelték: nincs felesleges réteg, és a rétegek nem csak továbbítanak.

Önellenőrző kérdés: Mi a különbség a zárt és a nyitott réteg között? Miért fontos a függőségi irány a rétegezett architektúrában, és hogyan segíthet a függőségi inverzió a domain réteg függetlenségének megőrzésében?

11.2 Monolit, moduláris monolit és mikrolit

11.2.1 Monolit, moduláris monolit és mikrolit

Amikor a gyakorlati szoftverfejlesztésben a „monolit” kifejezés elhangzik, a legtöbb fejlesztőben egy hatalmas, átláthatatlan, a technikai adósság súlya alatt görnyedő kódhalmaz képe jelenik meg. Ez a kép azonban hamis általánosítás: a monolitikus telepítési forma nem feltétlenül jelent rossz belső szerkezetet, ahogyan a mikroszolgáltatásokra épülő rendszer sem garancia a tiszta architektúrára. Ebben az alfejezetben élesen szétválasztjuk a rendszer fizikai telepítési formáját annak logikai belső szerkezetétől. Megvizsgáljuk, hogyan építhető fel egy jól strukturált, moduláris monolit, melyek a build és deployment sajátosságai, és miért jelentenek nagyobb kockázatot a korai felosztásból származó, úgynevezett mikrolitok, mint egy gondosan karbantartott monolit.

11.2.1.1 A telepítési forma és a belső struktúra kettőssége

Az architektúra tárgyalásakor az egyik leggyakoribb félreértés a rendszer logikai felépítése és a fizikai telepítése közötti különbség elmosása. Egy rendszer akkor tekinthető monolitikusnak, ha egyetlen, oszthatatlan futtatható egységként kerül telepítésre. Ez az egység lehet egyetlen JAR vagy WAR fájl, egy .NET assembly, egy Python-csomag vagy egy konténerimage. A monolitikus jelző tehát a deployment egységre vonatkozik, nem pedig a forráskód minőségére.

A forráskód belső szerkezete ettől függetlenül két szélsőség között mozoghat. Az egyik végpont a rendezetlen, összefüggő kódhalmaz, amelyet gyakran „Big Ball of Mud” néven emlegetnek: itt minden komponens közvetlenül hivatkozhat bármely másikra, nincsenek betartatott határok, és a változtatások előrejelezhetetlen mellékhatásokkal járnak. A másik végpont a moduláris monolit: egyetlen telepítési egység, amelynek belsejében gondosan megtervezett, jól definiált interfészekkel kommunikáló modulok helyezkednek el.

A moduláris monolit lényege, hogy a rendszerhatárokat nem a hálózaton, hanem a programozási nyelv és a build-eszközök által biztosított mechanizmusokkal hozzuk létre és tartatjuk be. Egy modul ebben a kontextusban egy olyan kódegység, amely a funkcionalitás egy jól körülhatárolt részét valósítja meg, és a többi modul számára csak egy explicit, szűk interfészen keresztül érhető el. A modul belső implementációja, adatbázistáblái, segédosztályai kívülről nem láthatók.

Tekintsünk egy egyszerű, de valósághű példát: egy e-kereskedelmi rendszer megrendeléskezelő komponensét. Egy moduláris monolitban ezt a területet felbonthatjuk például a következő modulokra:

- **Rendelésmodul:** A megrendelés életciklusának kezelése, a kosár tartalmának és a szállítási címnek az összerendelése.
- **Készletmodul:** A raktárkészlet nyilvántartása és a foglalások kezelése.
- **Számlázási modul:** A fizetési tranzakciók elindítása és a számlák előállítása.
- **Ügyfélmodul:** Az ügyféladatok és a törzsvásárlói adatok kezelése.

Ebben a felépítésben a Rendelésmodulnak szüksége lehet a készletinformációkra a megrendelés rögzítésekor. A moduláris monolitban ez a kommunikáció egy belső, szinkron metódushíváson keresztül történik, például a `KészletSzolgáltatás` interfész `foglalásElindítasa(rendelesiAzonosito)` metódusán keresztül. A Rendelésmodul nem férhet hozzá közvetlenül a Készletmodul adatbázistábláihoz vagy belső entitásaihoz; csak az interfész által definiált szerződést használhatja. Ez a fajta belső szeparáció teszi lehetővé, hogy a kód érthető, tesztelhető és karbantartható maradjon anélkül, hogy a hálózati kommunikáció bonyolultságát be kellene vezetnünk.

11.2.1.2 Modulhatárok, build és deployment

A moduláris monolit sikere azon múlik, hogy a deklarált határokat a build folyamat során automatikusan ellenőrizni tudjuk-e. Ha a határok betartása pusztán a fejlesztői fegyelmen múlik, akkor a rendszer idővel elkerülhetetlenül a „Big Ball of Mud” állapotába csúszik. A modulhatárokat ezért kódban kell rögzíteni és a build részévé tenni.

A modern build-rendszerek, mint a Gradle (Java/Kotlin), a Maven (Java), vagy a .NET megoldásstruktúrája, lehetővé teszik, hogy a modulokat különálló alprojektekként definiáljuk. Egy Gradle-alapú projektben például a `settings.gradle.kts` fájlban a következőképpen deklarálhatjuk a moduljainkat:

```
// settings.gradle.kts (szemléltető példa)
rootProject.name = "webshop-monolit"

include("rendeles-modul")
include("keszlet-modul")
include("szamlazasi-modul")
include("ugyfel-modul")
```

A modulok közötti függőségeket pedig az egyes modulok `build.gradle.kts` fájljában szabályozhatjuk. A példánkban a `rendeles-modul` függhet a `keszlet-modul`-tól és a `szamlazasi-modul`-tól, de a `keszlet-modul` nem függhet a `rendeles-modul`-tól. Ennek a szabálynak a betartatását a build-rendszeren túl egy architekturális tesztelő eszközzel is érdemes ellenőrizni. Java környezetben az ArchUnit, .NET környezetben a NetArchTest egy ilyen lehetőség. Az alábbi, szemléltető jellegű ArchUnit-teszt azt ellenőrzi, hogy a készletmodul nem hivatkozhat a rendelésmodul osztályaira:

```
// Szemléltető példa, nem futtatandó éles kód
@Test
void aKeszletModulNemFugghetARendelesModultol() {
    JavaClasses importaltOsztalyok = new ClassFileImporter()
        .importPackages("com.pelda.webshop");

    archRule()
        .that().resideInAPackage("..keszlet..")
        .should().onlyDependOnClassesThat()
        .resideOutsideOfPackage("..rendeles..")
        .check(importaltOsztalyok);
}
```

Az ilyen tesztek futtatása a CI/CD folyamat részeként azt eredményezi, hogy a modulhatárok megsértése azonnal buildhibát okoz. Ez a fajta automatikus ellenőrzés a moduláris monolit gerince. A fejlesztő szabadon refaktorálhat egy modul belsejében, mert a rendszer többi része csak az interfésztől függ. Ha viszont egy rossz irányú függőséget kíván létrehozni, a build azonnal jelzi a hibát, még mielőtt a rossz gyakorlat elterjedne a kódbázisban.

A monolitikus build eredménye egyetlen telepíthető artefaktum. Ennek a deploymentje az egyik legnagyobb előnye a bonyolultabb elosztott rendszerekkel szemben. A telepítés nem igényel bonyolult orkesztrációs logikát: a futtatáshoz elegendő egy futtatókörnyezet (pl. egy Java virtuális gép vagy egy konténer-runtime) és a szükséges perifériák, például az adatbázis elérhetősége. Nincs szükség több tucat szolgáltatás közötti szerződéses kompatibilitás ellenőrzésére egy kiadás során, és nem kell a tranzakcionális konzisztenciát a hálózaton keresztül, elosztott tranzakciókkal vagy kompenzáló műveletekkel megvalósítani. A modulok közötti kommunikáció egyetlen folyamaton belül, a memóriában történik, így a hálózati késleltetés és a szerializálás költsége kizárt.

A horizontális skálázás ebben a modellben a monolit egészének többszörözésével oldható meg. A terheléselosztó mögé több, azonos konfigurációjú monolit példányt állítunk. Ez a megközelítés addig hatékony, amíg a monolit statikus állapota (például a memóriában tárolt munkamenet-adatok) nem akadályozza a példányok közötti szabad terheléselosztást, vagy amíg az adatbázis nem válik a skálázódás szűk keresztmetszetévé. A skálázás durvább szemcsézettségű, mint a mikroszolgáltatásoknál: nem tudjuk kizárólag a készletmodult skálázni, ha arra érkezik nagyobb terhelés, hanem az egész rendszert kell többszöröznünk. Ez a durvaság azonban egyszerűséget is hoz: a működési kérdések száma jelentősen csökken.

11.2.1.3 A moduláris monolit előnyei és korlátai

A moduláris monolit különösen jól teljesít a termékfejlesztés korai, feltáró fázisaiban. Amikor a domain maga is még bizonytalan, és a fejlesztőcsapatnak gyorsan kell iterálnia, a hálózati hívások nélküli, egyetlen folyamaton belüli refaktorálás jelentős sebességelőnyt biztosít. Ha például a készletmodul és a rendelésmodul közötti interfészt kell módosítani, a változtatás egyetlen kódbázisban, egyetlen pull requestben elvégezhető, és a teljes rendszer konzisztenciája azonnal ellenőrizhető. A mikroszolgáltatásoknál ugyanez a változtatás a függő szolgáltatás szerződésének verziózását, kompatibilitási mátrixot és a deploy sorrendjének megtervezését igényelheti.

Az előnyök mellett azonban világosan látnunk kell a korlátokat is. A moduláris monolit teljes életciklusa során a technológiai diverzitás rendkívül korlátozott. Ha egy adott modulban egy másik programozási nyelv vagy adatbázis-technológia lenne az optimális, annak bevezetése a legtöbb esetben nem lehetséges vagy nem kifizetődő. A monolit egyetlen folyamata miatt a hibák izolációja is gyengébb: egy súlyos memóriaszivárgás a készletmodulban az egész alkalmazás leállását okozhatja. A skálázás, mint láttuk, az egész rendszerre kiterjed, ami költséghatékonysági problémákat okozhat akkor, ha a terhelés nagyon egyenetlenül oszlik meg a modulok között. Végül pedig a csapatok méretének növekedésével a közös kódbázison való párhuzamos munka koordinációs költsége is nő.

11.2.1.4 A mikrolit és a korai felosztás csapdái

A monolitokkal kapcsolatos negatív tapasztalatok gyakran arra késztetik a csapatokat, hogy már a projekt legelején mikroszolgáltatás-architektúrába kezdjenek. Ez a döntés azonban gyakran vezet a moduláris monolitnál rosszabb állapotba, az úgynevezett mikrolit csapdájába. A mikrolit kifejezés olyan rendszert jelöl, amely formailag mikroszolgáltatásokból áll, de a szolgáltatások túl kicsik, túl szorosan összefüggnek, és a közöttük lévő kommunikáció olyan mértékben összefonódott, hogy a rendszer valójában egy hálózaton szétszórt, szétszabdalt monolitként viselkedik.

A korai felosztás egyik legjellemzőbb hibája a tranzakciós határok megsértése. Egy moduláris monolitban a megrendelés rögzítése és a készletfoglalás egyetlen, az adatbázis által biztosított ACID tranzakcióban hajtódhat végre. Ha ugyanezt a műveletet korán szétválasztjuk a Rendelés-szolgáltatás és a Készlet-szolgáltatás közé, akkor a konzisztencia biztosítása hirtelen elosztott tranzakciót vagy szaga-mintát igényel. A csapat ezzel a domain egyik legnehezebb

műszaki kihívását vezeti be akkor, amikor még maga az üzleti logika is képlékeny. Az eredmény egy „elosztott monolit”: a szolgáltatások annyira szorosan összefüggenek, hogy egyetlen üzleti funkció megvalósításához több szolgáltatást kell egyszerre, szigorú sorrendben telepíteni, ami a mikroszolgáltatások minden üzemeltetési költségét magával hozza anélkül, hogy a skálázhatósági és szervezeti előnyök érvényesülnének.

A mikrolitok kialakulásának másik jellemző oka a modulok közötti határok helytelen kijelölése. Amikor a felosztás alapja nem az üzleti képesség vagy a domain, hanem a technikai infrastruktúra, a szolgáltatások óhatatlanul összekapcsolódnak. Például egy olyan architektúra, amely külön szolgáltatást hoz létre a „validációra”, egy másikat a „mentésre”, szükségszerűen azt eredményezi, hogy a validációs szolgáltatás minden egyes üzleti tranzakcióban részt vesz, és a módosítása az összes többi szolgáltatást érinti. A mikro nem azt jelenti, hogy minél kisebb, annál jobb; a micro a felelősség üzleti szempontból koherens, minimális terjedelmére utal.

A szakirodalomban gyakran hivatkozott elv a „MonolithFirst”, amely szerint egy új rendszer építését érdemes monolitként kezdeni, és csak akkor szétválasztani, amikor a szétválasztás valós fájdalmat enyhít. A fejezet keretei között ezt az elvet úgy finomítjuk: a kiindulópont ne egy rendezetlen monolit, hanem egy fegyelmezett, moduláris monolit legyen. A moduláris monolitban a belső határok már készen állnak arra, hogy később hálózati határokká alakítsuk őket. A szétválasztás akkor válik indokolttá, ha valamelyik modul önálló skálázást, önálló telepítési ciklust vagy technológiai függetlenséget igényel. Az ilyen, fájdalom-alapú, evolúciós megközelítés kisebb kockázattal jár, mint a spekulatív, korai felosztás.

11.2.1.5 Labor: monolit és moduláris monolit összehasonlítása

Ennek a labornak a célja, hogy a fejlesztői csapat saját, reprodukálható módon összehasonlítsa a rendezetlen monolit és a moduláris monolit közötti különbséget, és döntési naplóban rögzítse a tanulságokat.

Kiinduló állapot: A fejlesztői környezetben rendelkezésre áll egy egyszerű, „Rendeléskezelő” nevű referencia-implementáció, amely egyetlen forrásfában helyezkedik el. A kód három, logikailag elkülöníthető csomagra (rendelés, készlet, számlázás) tagolódik, de a forráskódban a függőségek kuszaak: a rendelés csomag közvetlenül hivatkozik a készlet és a számlázás csomag belső osztályaira, és a készlet csomag is visszahivatkozik a rendelés csomag egyes segédosztályaira.

Feladat 1: Modulhatárok definiálása. A build-rendszer konfigurációjában hozzon létre külön alprojekteket a három modul számára. Definiálja a modulok közötti megengedett függőségeket: a rendelés modul függhet a készlet és a számlázás modulra, de a készlet és a számlázás nem függhet a rendelés modultól.

Feladat 2: Architekturális tesztelés bevezetése. Adjon hozzá egy architekturális tesztet, amely a build folyamat részeként ellenőrzi a fenti függőségi szabályt. Futtassa a tesztet. Ha a teszt hibát jelez, refaktorálja a kódot addig, amíg a függőségi szabály teljesül. A refaktorálás során vezessen be interfészeket a modulok közötti kommunikációra, és szüntesse meg a közvetlen osztályszintű hivatkozásokat.

Feladat 3: Döntési napló (ADR) készítése. Készítsen egy ADR-t „A rendszer telepítési formájának megválasztása” címmel. Az ADR a következő elemeket tartalmazza: - **Kontextus:** A rendszer jelenlegi mérete, a csapat létszáma, a domain ismertsége. - **Döntés:** A moduláris monolit választása a mikroszolgáltatásokkal szemben. - **Alternatívák:** (1) Azonnali mikroszolgáltatások, (2) rendezetlen monolit. - **Következmények:** Az egyszerű telepítés és tranzakciókezelés előnyei; a technológiai kötöttség és a durva skálázás korlátai. - **Állapot:** Elfogadva.

Ellenőrzési szempontok: - A build sikeresen lefut, és az architekturális teszt zöld. - A modulok közötti függőségek iránya megfelel a definiált szabálynak. - Az ADR minden kötelező mezőt tartalmaz, és a benne foglalt érvek tükrözik a labor során tapasztaltakat.

Visszaállítási út: Mivel a labor csak kódot és konfigurációt módosít, a verziókezelő rendszerből a kiinduló állapot bármikor visszaállítható. A labor lezárása után a létrehozott artefaktumokat (ADR, tesztkód) a csapat a további munka során alapul veheti.

11.2.1.6 Gyakori hibák és ellenőrzőlista

A moduláris monolitok tervezésekor a leggyakoribb hiba a modulok közötti rejtett, adatbázison keresztüli csatolás. Ha a rendelés modul és a készlet modul ugyanazokat a táblákat írja-olvassa, akkor a build-rendszer által kikényszerített forráskód-függetlenség hamis biztonságérzetet ad. A modulok ebben az esetben továbbra is erősen csatoltak, csak a csatolás az adatbázissémán keresztül valósul meg. A moduláris monolitban érdemes törekedni arra, hogy minden modul a saját tábláit birtokolja, és a másik modulok adataihoz kizárólag az adott modul interfészén keresztül férhessenek hozzá.

Egy másik jellemző buktató a „közös kernel” (shared kernel) túlterhelése. A közös kernel olyan, minden modul által használt segédkódot jelöl, mint például az azonosítók vagy a pénzösszeg típusok. Ha azonban a csapat minden nehezen besorolható osztályt a közös kernelbe helyez, akkor az gyorsan egy újabb gubaccá válik, és a modulhatárok ismét elmosódnak. A közös kernelnek minimálisnak és szigorúan szabályozottnak kell lennie.

Ellenőrzőlista a moduláris monolithoz:

1. A modulok a build-rendszer szintjén különálló egységek.
2. A modulok közötti függőségek iránya deklarált és automatikus teszttel ellenőrzött.
3. Minden modul a saját adatbázissémáját birtokolja.
4. A modulok közötti kommunikáció interfészeken keresztül történik, nem közvetlen osztályhivatkozással.
5. A közös kernel mérete minimális, és a benne lévő kód változtatásai a lehető legritkábbak.
6. A telepítés egyetlen artefaktumból áll, és a CI/CD folyamat nem igényel többszörös deploy-orderinget.

11.2.1.7 Önellenőrző kérdés

Egy ötfős fejlesztőcsapat egy vadonatúj, de nagy üzleti jelentőségű belső számlázórendszer fejlesztésébe kezd. A domain szakértői még nem tudták pontosan meghatározni a számlázás és a jóváírás folyamatának minden üzleti szabályát, a rendszer várható terhelése azonban az első hónapokban alacsony lesz. A csapat vezetője azt javasolja, hogy azonnal öt mikroszolgáltatást indítsanak el (pl. Partnerkezelő, Számlakészítő, Jóváíráskezelő, Riport, Értesítés), hogy elkerüljék a monolit csapdáját. A tevékenység mely buktatóira hívná fel a figyelmet? Milyen alternatív architekturális megközelítést javasolna, és milyen feltételek teljesülése esetén tartaná indokoltnak a későbbi szétválasztást? Válaszában indokolja meg, miért előnyösebb ebben a helyzetben a moduláris monolit a mikrolitok kialakulásának kockázatával szemben.

11.3 Csillag- és mikro-kernell architektúra

A `sa039` fejezetben a moduláris monolit kapcsán azt láttuk, hogy a belső szerkezet akkor is meghatározó, ha a rendszert egyetlen egységként telepítjük. A most következő két topológia – a **csillag (hub-and-spoke)** és a **mikro-kernell (plug-in)** – tovább viszi ezt a gondolatot, de más kérdésre ad választ. Míg a moduláris monolit elsősorban a kódbázis belső rendjéről szól, addig a csillag- és a mikro-kernell architektúra a központi koordináció és a független kiterjeszthetőség közötti feszültséget helyezi a középpontba. A cél nem az, hogy a két mintát merev kategóriának tekintsük, hanem hogy megértsük, milyen követelmények, csapatméret, terhelési profil és evolúciós igény mellett működnek jól, és hol válnak kockázatossá.

11.3.1 1. A csillag-topológia és a hub-and-spoke

A **hub-and-spoke** – magyarul gyakran **csillag-küllő** – olyan elrendezés, amelyben egy központi **hub** áll a rendszer közepén, a peremeken lévő **spoke**-ok pedig a hubon keresztül kommunikálnak egymással. A közvetlen spoke–spoke kapcsolat szándékosan hiányzik vagy korlátozott: ha két végpontnak együtt kell működnie, a hub közvetíti az üzenetet, végzi a protokoll- és adatformátum-átalakítást, és gyakran a naplózást, hitelesítést, illetve a forgalomirányítást is.

A minta legismertebb alkalmazási területe a **vállalati integráció**: klasszikus ESB (Enterprise Service Bus), adatintegrációs központ, API gateway, valamint olyan adatcsere-platformok, ahol sok heterogén rendszernek kell viszonylag egységes módon kapcsolódnia. A hub itt nem feltétlenül üzleti logikát futtat, hanem **közlekedési csomópontként** működik. Az analógia a repülőtéri csillagmodell: a légitársaságok a csomóponti repülőtéren keresztül kötik össze a járatokat, így a közvetlen járatok száma drasztikusan csökken, de a csomópont kihasználtsága és kockázata megnő.

A hub-and-spoke fő előnyei közé tartozik:

- **Csökkentett kapcsolati mátrix:** ha nyolc rendszernek kellene páronként közvetlenül kommunikálnia, 28 kapcsolatot kellene fenntartani. Egy központi hubbal ez nyolc kapcsolatra csökken.
- **Központi felügyelet:** a routing, a transzformáció, a hitelesítés és a naplózás egy helyen ellenőrizhető, ami leegyszerűsíti a megfelelést és az incidenskezelést.
- **Egyszerűbb csatlakoztatás:** új rendszer bevezetése csak egyetlen integrációt igényel a hub felé, nem kell minden partnert módosítani.

A hátrányok azonban ugyanebből a központosításból fakadnak:

- **Single point of failure:** ha a hub leáll, a teljes integrációs hálózat leáll.
- **Szűk keresztmetszet:** minden forgalom a hubon halad át, így a hub teljesítménye és skálázhatósága kritikus.
- **Komplex hub:** minél több felelősséget halmoz fel a hub, annál nehezebb karbantartani; könnyen „isten-objektummá” válik.
- **Függés a hub szabványaitól:** a spoke-oknak alkalmazkodniuk kell a hub által megkövetelt protokollokhoz és adatformátumokhoz, ami csökkentheti a rugalmasságot.

Fontos megkülönböztetni a **topológiai** és a **logikai** csillagot. Egy rendszer lehet fizikailag elosztott, hálózati értelemben mégis csillag alakú, ha minden forgalom egy központi szolgáltatáson fut át. Ugyanakkor egy monolitikus belső szerkezetben is létrejöhet hub-and-spoke, ha egy központi komponens minden adatáramlást kezel.

11.3.2 1.1. Példa: vállalati integrációs hub

Képzeljünk el egy e-kereskedelmi platformot, amely külső ERP-rendszerrel, fizetési szolgáltatóval, logisztikai szolgáltatóval és ügyfélszolgálati szoftverrel integrálódik. Ahelyett, hogy mindegyik rendszer közvetlenül hívná a másikat, egy központi integrációs hub fogadja a szinkron REST-hívásokat és az aszinkron üzeneteket, majd a megfelelő formátumra alakítva továbbítja a célrendszerhez.

```
flowchart LR
    A[Rendeléskezelő] -->|HTTPS / JSON| H[Integrációs Hub]
    B[ERP] -->|SOAP / XML| H
    C[Fizetési szolgáltató] -->|HTTPS / JSON| H
    D[Logisztikai szolgáltató] -->|SFTP / CSV| H

    H -->|HTTPS / JSON| A
    H -->|SOAP / XML| B
    H -->|HTTPS / JSON| C
    H -->|SFTP / CSV| D

    subgraph "Jelmagyarázat"
        E1["Hub: központi routing, transzformáció, naplózás"]
        E2["Spoke-ok: végpontrendszerek, közvetlen kapcsolat nincs"]
    end
```

A diagramon látható, hogy a spoke-ok nem beszélnek közvetlenül egymással. A hub felel a protokoll- és adatformátum-konverzióért. Ha egy új rendszer csatlakozik, csak a hub felé kell integrálni, a többi rendszer változatlan marad. Ez a minta akkor működik jól, ha a rendszerek száma nem extrém nagy, a hub terhelése kezelhető, és a központi irányítás elfogadható a szervezet számára.

11.3.3 2. A mikro-kernell (plug-in) architektúra

A **mikro-kernell** – más néven **plug-in architektúra** – célja nem az integráció, hanem a **termék funkcionális kiterjeszthetősége**. A minta egy **minimális magból** (core, kernel) és a magra épülő, egymástól független **bővítményekből** (plug-in) áll. A mag biztosítja az alapvető infrastrukturális szolgáltatásokat és a plug-inek életciklusát: regisztráció, betöltés, konfiguráció, naplózás, erőforrás-kezelés, hibatűrés. Az üzleti vagy termékfunkciók jelentős részét a plug-inek szállítják.

Klasszikus példák:

- **Operációs rendszer mikro-kernel:** a kernel csak a minimális ütemezést, memóriakezelést és IPC-t valósítja meg; a fájlrendszerek, hálózati protokollok, driverek plug-inként működnek.
- **IDE-k:** az Eclipse, az IntelliJ IDEA vagy a VS Code magja a szerkesztőfelületet, a projektkezelést és a szolgáltatások regisztrációját adja; a nyelvi támogatásokat, build-eszközöket, formattert plug-inek biztosítják.
- **CI/CD rendszerek:** a Jenkins magja az ütemezést, a job-ok végrehajtását és a felhasználókezelést futtatja; a build-lépések, értesítők és integrációk plug-inek.
- **Böngészőbővítmények:** a böngésző magja a webtartalom megjelenítéséért felel, a bővítmények új funkciókat adnak hozzá, korlátozott API-val.

A mikro-kernell és a hub-and-spoke között a hasonlóság a központi elem jelenléte. A különbség a célban és a központi elem természetében van:

Szempont	Hub-and-spoke	Mikro-kernell
Elsődleges cél	Integráció, routing, transzformáció	Termék bővíthetősége, moduláris funkcionalitás
Központi elem	Forgalmi hub, gyakran „okos”	Minimális mag, infrastrukturális szolgáltatások
Végpontok	Külső rendszerek, spoke-ok	Belső bővítmények, plug-inek
Kommunikáció iránya	Spoke-ok a hubon keresztül egymással	Plug-inek a maggal, egymással ritkán
Fejlődési dinamika	Új rendszer csatlakoztatása	Új funkció hozzáadása plug-inként

A mikro-kernell architektúrában a **mag stabilitása** a legfontosabb tulajdonság. A mag változtatása minden plug-inre hatással lehet, ezért a magot úgy kell megtervezni, hogy ritkán kelljen módosítani. A plug-inek cserélhetők, verziózhatók, sőt futás közben is betölthetők vagy eltávolíthatók, amennyiben a mag ezt támogatja.

11.3.4 3. A központi szolgáltatás és a független bővítmények

A mikro-kernell két pillére a **központi szolgáltatás** (mag) és a **független bővítmények** (plug-inek). A felelősségek pontos elválasztása nélkül a minta szétesik.

11.3.5 3.1. A mag felelőssége

A mag felelőssége **nem** az üzleti logika, hanem az, hogy a plug-inek stabil, kiszámítható környezetben futhassanak:

- **Életciklus-kezelés:** plug-inek regisztrálása, betöltése, inicializálása, leállítása, verzióellenőrzése.
- **Interfész-szerződés:** a plug-inek által implementálandó API definiálása, beleértve a konfigurációs sémát, a hibakezelést és a naplózási kontextust.
- **Erőforrás-gazdálkodás:** a plug-inek által használt memória, CPU, hálózat vagy szálak korlátozása, izoláció biztosítása.
- **Hibatűrés:** egy plug-in hibájának kezelése anélkül, hogy a mag vagy a többi plug-in leállna.
- **Megosztott infrastruktúra:** naplózás, metrikák, konfiguráció, auditálás, hitelesítés, eseménytovábbítás.
- **Plugin-regisztráció és felfedezés:** a mag nyilvántartja az elérhető plug-ineket, azok verzióit, függőségeit és állapotát.

A mag akkor jó, ha **minimális**: csak azokat a szolgáltatásokat tartalmazza, amelyek valóban közösek, és amelyeket minden plug-in használ. Ha egy funkció csak egyetlen plug-inhez tartozik, akkor nem a magba való.

11.3.6 3.2. A plug-inek felelőssége

A plug-in felelőssége a **specifikus üzleti vagy termékfunkció** megvalósítása. A plug-in:

- implementálja a mag által definiált interfészt,

- kezeli a saját állapotát,
- használhatja a mag által biztosított szolgáltatásokat,
- nem függ más plug-inek belső megvalósításától,
- a mag felé közölhet eseményeket, hibákat, metrikákat.

A plug-inek egymással **nem kommunikálnak közvetlenül**. Ha egy plug-innek más plug-in szolgáltatására van szüksége, azt a magon keresztül, vagy a mag által biztosított üzenetszolgáltatáson keresztül teheti. Ez megőrzi a laza csatolást és az izolációt.

11.3.7 3.3. Kommunikációs minták a mag és a plug-in között

A mag és a plug-in közötti kommunikáció többféle lehet:

- **Szinkron hívás:** a mag meghív egy plug-in metódust, a plug-in visszaadja az eredményt. Például egy dokumentum-feldolgozó mag meghívja a PDF-szövegkinyerő plug-int, és megvárja a szöveget.
- **Esemény:** a mag eseményt publikál, amelyre a plug-inek feliratkozhatnak. Például a mag jelzi, hogy új feladat érkezett, és minden érintett plug-in elindul.
- **Callback:** a plug-in regisztrál egy visszahívási függvényt egy adott eseményre, a mag pedig a megfelelő időben meghívja.
- **Megosztott adatmodell:** a mag és a plug-in közösen használ egy adatstruktúrát, például egy feldolgozási kontextust, amelyet a mag hoz létre, és a plug-in olvas vagy módosít.

A választott minta hatással van a tesztelhetőségre, a szinkronitásra és a hibatűrésre. A szinkron hívás egyszerű, de egy lassú plug-in blokkolhatja a magot. Az eseményvezérelt modell lazább, de nehezebb a hibakezelés és a sorrend garantálása.

11.3.8 3.4. Példa: dokumentum-feldolgozó platform

Tekintsünk egy olyan platformot, amely dokumentumokat fogad, különböző feldolgozási lépéseket hajt végre (például PDF-szövegkinyerés, OCR, nyelvi ellenőrzés, ERP-be küldés), és az eredményt visszaadja a felhasználónak. A mag feladata a feladatfogadás, a plug-in-lánc összeállítása, a végrehajtás koordinálása, a naplózás és a hibatűrés. A plug-inek az egyes feldolgozási lépések.

A C4 context szint a rendszerhatárt és a külső szereplőket mutatja:

```
C4Context
    title Dokumentum-feldolgozó platform - context szint
    Person(felhasznalo, "Felhasználó", "Dokumentumot tölt fel, eredményt kérdez le")
    System(platform, "Dokumentum-feldolgozó Platform", "Plug-in alapú feldolgozó rendszer")
    System_Ext(erp, "ERP rendszer", "Külső vállalatirányítási rendszer")
    System_Ext(ocr_szolgaltatas, "Külső OCR szolgáltatás", "Opcionális felhőalapú OCR")

    Rel(felhasznalo, platform, "HTTPS", "REST API, szinkron kérés")
    Rel(platform, erp, "HTTPS", "Eredmények átadása, szinkron válasz, hibatűrés: retry")
    Rel(platform, ocr_szolgaltatas, "HTTPS", "Képfelismerés kérése, aszinkron callback,
↪   hibatűrés: timeout")
```

A component szinten a mikro-kernell szerkezet látható. A diagramon a kapcsolatoknál feltüntetjük a protokollt, a szinkronitást és a hibatűrést:

```
C4Component
    title Dokumentum-feldolgozó platform - component szint (mikro-kernell)
    Container_Boundary(platform, "Dokumentum-feldolgozó Platform") {
        Component(mag, "Feldolgozó Mag", "Go", "Feladatfogadás, életciklus, routing, naplózás,
↪   hibatűrés")
        Component(plugin_reg, "Plugin Registry", "Belső könyvtár", "Regisztráció, verzió,
↪   függőségek")
        Component(pdf_plugin, "PDF szövegkinyerő plugin", "Python", "PDF-ből szöveg kinyerése")
        Component(ocr_plugin, "OCR plugin", "Python", "Képfelismerés helyben vagy külső
↪   szolgáltatással")
        Component(erp_connector, "ERP csatlakozó plugin", "Go", "Eredmények átadása az ERP
↪   felé")
    }

    Rel(felhasznalo, mag, "HTTPS", "REST API, szinkron")
    Rel(mag, plugin_reg, "in-process", "regisztráció, szinkron hívás, hibatűrés: retry")
    Rel(mag, pdf_plugin, "in-process", "plugin API, szinkron hívás, hibatűrés: timeout")
    Rel(mag, ocr_plugin, "in-process", "plugin API, aszinkron esemény, hibatűrés: circuit
↪   breaker")
    Rel(mag, erp_connector, "in-process", "plugin API, aszinkron esemény, hibatűrés: outbox")
    Rel(ocr_plugin, ocr_szolgaltatas, "HTTPS", "külső OCR hívás, szinkron, hibatűrés: retry")
```

Ebben a példában a mag nem végzi el a PDF-kinyerést vagy az OCR-t; azokat a plug-inek szállítják. A mag viszont garantálja, hogy ha az OCR plug-in lelassul vagy hibázik, a feladat ne vesszen el, és a többi plug-in ne álljon le.

11.3.9 4. Bővíthetőség, izoláció és központi kockázat

A mikro-kernell és a hub-and-spoke három kulcsfogalma a **bővíthetőség**, az **izoláció** és a **központi kockázat**. Mindhárom trade-offokat takar.

11.3.10 4.1. Bővíthetőség

A mikro-kernell legnagyobb erőssége, hogy új funkciót anélkül adhatunk hozzá, hogy a magot módosítani kellene. Ez akkor értékes, ha:

- a termék funkciói nagy számúak és változatosak,
- a funkciókat különböző csapatok vagy külső fejlesztők szállítják,
- a funkciókat futás közben kell telepíteni vagy leállítani.

A hub-and-spoke esetében a bővíthetőség azt jelenti, hogy új spoke csatlakoztatása egyszerű, de a hub módosítása gyakran szükséges, ha az új spoke új protokollt vagy adatformátumot igényel.

A bővíthetőség mérésére minőségi forgatókönyvet alkalmazhatunk:

Minőségi forgatókönyv – plug-in telepítés

- **Inger:** egy új feldolgozó plug-in telepítése a futó platformra.
- **Környezet:** termelési rendszer, normál terhelés alatt.
- **Válasz:** a mag regisztrálja a plug-int, az új funkció azonnal elérhető, a többi plug-in és a mag működése nem változik.
- **Mérőszám:** a telepítési idő < 30 másodperc, a platform rendelkezésre állása az adott 5 perces ablakban 99,9%, a mag memóriahasználatának növekedése $< 10\%$.

A bővíthetőség nem abszolút érték, hanem a rendszer egy minőségi attribútuma, amelyet az adott kontextusban kell értelmezni.

11.3.11 4.2. Izoláció

Az izoláció azt jelenti, hogy egy plug-in hibája, erőforrás-túlfogyasztása vagy biztonsági problémája ne döntse le az egész rendszert. A mag felelőssége az izoláció megvalósítása:

- **Folyamatszintű izoláció:** a plug-inek külön folyamatban futnak, a mag IPC-n kommunikál velük. Ez erős izolációt ad, de nagyobb a kommunikációs költség.
- **Sandbox / konténer:** a plug-in külön konténerben vagy homokozóban fut, a mag szabályozza az erőforrásokat.
- **Osztálybetöltő-szintű izoláció:** a plug-in külön osztálybetöltővel vagy alkalmazástartománnyal rendelkezik, így a statikus állapot nem osztozik. Ez a JVM vagy .NET világában gyakori.
- **Szál-szintű izoláció:** a mag külön szálon futtatja a plug-int, timeouttal és megszakítással.
- **Függőség-izoláció:** a plug-in saját függőségeit a mag nem osztja meg más plug-inekkel, elkerülve a függőségi konfliktusokat.

A megfelelő izolációs szintet a kockázat, a teljesítmény és a fejlesztési költség határozza meg. Egy IDE bővítmény esetében gyakran elegendő az osztálybetöltő-szintű izoláció; egy CI/CD rendszerben a build-lépések futtatása konténerben jobb választás.

11.3.12 4.3. Központi kockázat

Mindkét minta központi eleme egyszerre előny és kockázat. A központi kockázat formái:

- **Single point of failure:** ha a mag vagy a hub leáll, minden funkció leáll. A hub-and-spoke-nál ez különösen éles, mert a teljes integráció a hubon múlik.
- **Teljesítmény-szűkület:** a központi elem minden forgalmat lát, ezért a kapacitását a teljes rendszer terhelésére kell méretezni. A Little-törvény segíthet a kapacitástervezésben: ha egy hub átlagosan L kérést tart a rendszerben, λ a beérkezési ráta, W pedig az átlagos feldolgozási idő, akkor $L = \lambda W$. Ha W nő, a hubon áthaladó kérések száma is nő, ami sorban álláshoz és megnövekedett válaszidőhöz vezet.
- **Változtatási sugár:** a mag interfészének módosítása minden plug-inre kihat. Ezért a mag verziózása és a visszafelé kompatibilitás elengedhetetlen.
- **Komplexitás-koncentráció:** ha a mag túl sokat vállal, akkor a rendszer legbonyolultabb része lesz, és a fejlesztési sebesség lelassul.

A központi kockázat csökkentésére szolgáló eszközök:

- **Minimális mag:** csak a legszükségesebb közös szolgáltatások legyenek a magban.
- **Stabil interfész:** a mag és a plug-in közötti API verziózott, a változások deprecációs időszakkal.
- **Hibatűrés:** a magban circuit breaker, timeout, retry, bulkhead minták védik a rendszert a hibás plug-inektől.
- **Független üzemeltetés:** a plug-inek külön telepíthetők, monitorozhatók, skálázhatók (ha a folyamat- vagy konténerszintű izoláció ezt lehetővé teszi).

11.3.13 5. Interfészek és megvalósítási döntések

A mikro-kernell architektúra sikerének kulcsa a **mag és a plug-in közötti interfész**. Ez az interfész nem csupán egy programozási nyelvi interfész, hanem szerződés, amely meghatározza az életciklust, a konfigurációt, a hibakezelést és a verziózást.

Egy lehetséges plug-in interfész pszeudo-kódban (szemléltető, nem végrehajtandó):

```
// A mag által definiált szerződés.
type Plugin interface {
    // Azonosítás és verzió.
    Name() string
    Version() string

    // A mag hívja indításkor.
    Init(ctx Context) error

    // A mag hívja leállításkor.
    Shutdown(ctx Context) error

    // A mag hívja feldolgozási feladat esetén.
    Process(ctx Context, input []byte) (output []byte, err error)

    // A mag hívja egészség-ellenőrzéskor.
    HealthCheck() Status
}
```

A mag a plug-in regisztrációjakor ellenőrzi a verziót, a függőségeket, és a plug-int egy izolált környezetben tölti be. A `Context` tartalmazza a korrelációs azonosítót, a naplózási kontextust, a timeoutot és a konfigurációt.

A plug-in manifest (például JSON) is a szerződés része:

```
{
  "name": "pdf-text-extractor",
  "version": "2.1.0",
  "api_version": "3.0",
  "entrypoint": "plugins.pdf_extractor.Plugin",
  "dependencies": {
    "shared-lib": "^1.0.0"
  },
  "resource_limits": {
    "max_memory_mb": 256,
    "max_cpu_ms_per_call": 500
  }
}
```

Ez a manifest nem valós hitelesítő adatot vagy belső URL-t tartalmaz, és a plug-in-telepítő rendszer ellenőrzi az aláírást, mielőtt betöltené a plug-int.

Fontos döntés, hogy a plug-ineket a maggal együtt, vagy külön folyamatban futtassuk-e. A **folyamaton belüli** plug-in gyorsabb, de gyengébb izolációt ad. A **külön folyamat** erősebb izolációt, de nagyobb kommunikációs költséget jelent. A választást a plug-in megbízhatósága és a teljesítményigény határozza meg.

11.3.14 6. Gyakori hibák és elkerülésük

A csillag- és mikro-kernell architektúrák tipikus buktatói a központi elem túlterheléséből és a határok hiányából fakadnak.

11.3.15 6.1. Az „isten-hub” vagy „isten-mag”

Hiba: a központi elem fokozatosan magába szív minden üzleti logikát, a plug-inek vagy spoke-ok üres héjává válnak. A hub így bonyolult, lassan fejleszthető monolit lesz, a bővíthetőség illúzióvá válik.

Helyes megoldás: a mag csak koordinál, nem végez üzleti logikát. Minden funkciót plug-inbe vagy spoke-ba kell helyezni. A magot rendszeresen felül kell vizsgálni: ha egy metódusban üzleti feltétel vagy formátum-specifikus kód jelenik meg, az átkerül a megfelelő plug-inbe.

11.3.16 6.2. Hiányzó vagy instabil interfész

Hiba: a mag és a plug-in közötti kommunikáció implicit, például a plug-in közvetlenül éri el a mag belső osztályait, vagy a mag a plug-in konkrét típusaira hivatkozik. Ez rejtett függőségeket és törékeny kódot eredményez.

Helyes megoldás: a mag csak interfészen keresztül kommunikál a plug-inekkel, és a plug-inek is csak a mag által publikált API-t használják. A függőségi irány a plug-inből a mag felé mutat, a mag nem függ konkrét plug-in-től (dependency inversion).

11.3.17 6.3. Nem megfelelő izoláció

Hiba: egy plug-in kivételt dob, memóriaszivárgást okoz, vagy letiltja a CPU-t, és ezzel az egész rendszer leáll. Ha a plug-inek osztoznak a mag memóriaterületén, egyetlen hiba az egészet magával ránthatja.

Helyes megoldás: a mag minden plug-in hívást timeouttal, erőforráskorláttal és hibakezeléssel vesz körül. A plug-inek külön izolációs tartományban futnak. A mag naplózza a plug-in hibáit, és képes a plug-int újraindítani anélkül, hogy a többi plug-int leállítaná.

11.3.18 6.4. A mag gyakori változtatása

Hiba: a mag interfészét minden új funkció vagy javítás során módosítják, ami miatt az összes plug-int újra kell építeni és telepíteni. Ez megöli a független fejlesztés előnyét.

Helyes megoldás: a mag interfészt verziózni kell, és a visszafelé kompatibilitást hosszú ideig meg kell őrizni. Az új funkciókat inkább új plug-inben vagy a meglévő plug-in bővítésével kell megvalósítani, nem a mag módosításával.

11.3.19 6.5. Túlzott plug-in-felaprózás

Hiba: minden apró funkciót plug-innek terveznek, még akkor is, ha azok szorosan összetartoznak. Ez indokolatlan bonyolultságot, felesleges regisztrációs logikát és nehéz hibakeresést okoz.

Helyes megoldás: a plug-in-határokat a kohézió és a változási sebesség alapján kell meghatározni. Ha két funkció mindig együtt változik, és erősen függ egymástól, akkor inkább egy plug-inbe tartoznak.

11.3.20 7. Minőségi forgatókönyvek és mérés

A minőségi attribútumok mérésére a quality attribute scenario formát használjuk: inger, környezet, válasz, mérőszám. A mikro-kernell és a hub-and-spoke esetében három fontos forgatókönyvet érdemes kiemelni.

Forgatókönyv 1 – Hibatűrés plug-in hiba esetén

- **Inger:** egy PDF-szövegkinyerő plug-in váratlan kivételt dob, mert hibás dokumentumot kapott.
- **Környezet:** termelési rendszer, 100 párhuzamos feladat feldolgozása közben.
- **Válasz:** a mag elkapja a hibát, a feladatot hibásnak jelöli, a többi feladat feldolgozása folytatódik, a plug-in állapota romlik, de a mag újraindítja a plug-int.
- **Mérőszám:** a hibás feladat feldolgozási ideje < 5 másodperc, a többi feladat sikeres feldolgozási aránya $\quad 99{,}9\%$ az adott 10 perces ablakban, a mag rendelkezésre állása $A = \frac{\text{üzemidő}}{\text{üzemidő}+\text{állásidő}} \geq 99{,}99\%$.

Forgatókönyv 2 – Bővíthetőség új plug-in telepítésekor

- **Inger:** egy új OCR plug-in telepítése a futó platformra.
- **Környezet:** termelési rendszer, normál terhelés alatt.
- **Válasz:** a mag észleli az új plug-int, ellenőrzi a manifestet, regisztrálja, és az új funkció azonnal elérhető.
- **Mérőszám:** a telepítési idő < 30 másodperc, a mag memóriahasználatának növekedése $<$ 10%, a rendszer rendelkezésre állása $\quad 99{,}9\%$ az 5 perces telepítési ablakban.

Forgatókönyv 3 – Hub terhelés alatti viselkedése

- **Inger:** 1000 üzenet/perc érkezik a hubhoz a spoke-októl.
- **Környezet:** csúcsidőszaki terhelés, 10 perces mérési ablak.
- **Válasz:** a hub minden üzenetet a megfelelő spoke-nak irányít, a sorban állási idő korlátos, az üzenetvesztés nincs.
- **Mérőszám:** a 95. percentilis végpont-válaszidő < 500 ms, a hub CPU-kihasználtsága $<$ 70%, az üzenetvesztés $= 0$ az adott ablakban.

Ezek a forgatókönyvek a mérési ablakkal együtt adják meg a teljesítményt, ami elengedhetetlen az ellenőrizhetőséghez. A méréseknél a környezetet és a bizonytalanságot is dokumentálni kell.

11.3.21 8. Döntési napló (ADR) minta

Amikor egy csapat mikro-kernell vagy hub-and-spoke architektúrát választ, a döntést ADR-ben kell rögzíteni. Az alábbi egy példa arra, hogyan nézhet ki egy ilyen bejegyzés.

ADR-014: Mikro-kernell architektúra választása a dokumentum-feldolgozó platformhoz

Állapot: Elfogadva
Dátum: 2025-03-10
Döntéshozók: platform architekt, vezető fejlesztő, termékmenedzser

Kontextus

A platformnak különböző dokumentumtípusokat kell feldolgoznia (PDF, kép, Office), és a jövőben új feldolgozási lépéseket is be kell illeszteni (például nyelvi ellenőrzés, adatkinyerés). A funkciókat részben belső csapatok, részben külső partnercsapatok fejlesztik. A csapat mérete 3–4 fejlesztői csoport, a terhelés mérsékelt. A terméknek hosszú élettartammal kell rendelkeznie, ezért a bővíthetőség és a független telepítés fontos.

Döntés

A platformot **mikro-kernell architektúrában** valósítjuk meg. A mag felel a feladatfogadásért, a plug-in-regisztrációért, az életciklusért és a hibatűrésért. A feldolgozási lépéseket plug-inek szállítják. A plug-inek a maggal egy folyamatban futnak, de izolált osztálybetöltővel, timeouttal és erőforrás-korláttal.

Alternatívák

1. **Moduláris monolit:** a feldolgozási lépések modulok lennének egy processzben, plug-in interfész nélkül. Egyszerűbb, de nem tenné lehetővé a külső csapatok független telepítését és a futás közbeni bővítést.
2. **Mikroszolgáltatások:** minden feldolgozási lépés külön szolgáltatás lenne. Jobb skálázhatóság, de jelentős hálózati és üzemeltetési költség, a csapat mérete nem indokolja.
3. **Hub-and-spoke integrációs hub:** a feldolgozást központi hub végezné, a csatlakozók csak adapterek lennének. Ez a minta inkább rendszerek összekötésére való, nem termékfunkciók bővítésére.

Következmények

- *Előnyök:* gyorsabb új funkciók bevezetése, független plug-in fejlesztés, magasabb hibatűrés a plug-inek izolációjával.
- *Hátrányok:* a mag interfészének stabilitása kritikus; a plug-in-ek közötti kommunikáció szabályozása bonyolult; a folyamaton belüli izoláció gyengébb, mint a konténeres megoldás.
- *Kockázatok:* ha a mag túl sok felelősséget vállal, az egész rendszer lelassul; a plug-in-fejlesztőknek be kell tartaniuk a szerződést.

Megerősítés

A döntést a technikai architektúra-review erősítette meg, ahol a trade-offokat a minőségi forgatókönyvek alapján értékelték.

Ez az ADR a kontextus, a döntés, az alternatívák és a következmények mellett az állapotot is rögzíti. A jövőbeli olvasó megérti, miért nem választottunk mikroszolgáltatásokat, és milyen kockázatokat vállaltunk.

11.3.22 9. Labor: mikro-kernell és csillag-topológia vázlat készítése

A labor célja, hogy a hallgató saját kezűleg alkalmazza a fejezet eszközeit egy kis, ellenőrizhető példán. A labor nem kódolási feladat, hanem architektúra-vázlat: C4 diagramok, interfész-leírás, ADR és minőségi forgatókönyv.

11.3.23 9.1. Kiinduló állapot

Egy fiktív, de realisztikus üzleti igény adott:

Termék: Ügyfélszolgálati jegykezelő platform, amely bejövő ügyféligényeket dolgoz fel. A bejövő csatornák különbözőek: e-mail, webes űrlap, telefonos jegyzet. A jegyeket ellenőrizni kell, osztályozni, majd továbbítani a megfelelő ügyintézői csoporthoz. A jövőben új csatornák és új osztályozó szabályok várhatók.

A rendszernek bővíthetőnek kell lennie, és a különböző feldolgozási lépéseket eltérő csapatok fejleszthetik. A csapat 3 fejlesztői csoport, a terhelés alacsony (néhány jegy percenként). Az üzemeltetési csapat szeretné, ha egy hibás osztályozó modul nem állítaná le a teljes jegyfeldolgozást.

11.3.24 9.2. Feladat

Készítsen reprodukálható dokumentumot (Markdown vagy Jupyter notebook), amely a következőket tartalmazza:

1. **C4 context és component diagram** diagrams-as-code formában (Mermaid). A context szinten tüntesse fel a felhasználókat és a külső rendszereket, a component szinten a magot és a plug-inek javasolt halmazát. A kapcsolatokon szerepeljen a protokoll, a szinkronitás és a hibatűrés.
2. **A mag interfészének specifikációja** pszeudo-kódban vagy szövegesen: milyen metódusokat kell a plug-ineknek implementálniuk, milyen életciklus-eseményeket kezel a mag, hogyan adja át a konfigurációt.
3. **Egy ADR** a mikro-kernell vagy a hub-and-spoke választásáról. Tüntesse fel a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
4. **Két minőségi forgatókönyv**: az egyik a plug-in hibatűrésre, a másik a bővíthetőségre vonatkozzon. Az inger, környezet, válasz és mérőszám megadásával.
5. **Ellenőrzési szempontok:** lista, amely alapján a vázlat megfelelőségét értékelni lehet.

11.3.25 9.3. Elvárt eredmény

A dokumentumban szerepelnie kell:

- legalább egy Mermaid C4 context és egy C4 component diagramnak,
- a mag felelősségei és a plug-inek felelősségei közötti elhatárolásnak,
- annak indoklása, hogy miért nem a másik topológiát választotta,
- a központi kockázat kezelésére vonatkozó tervezési döntésnek.

11.3.26 9.4. Ellenőrzési szempontok

- A context szint jelzi a rendszerhatárt és a külső szereplőket.
- A component szinten a mag minimális, a plug-inek függetlenek.
- A kapcsolatokon szerepel a protokoll, a szinkronitás és a hibatűrés.
- Az ADR tartalmazza a döntést, az alternatívákat és a következményeket.
- A minőségi forgatókönyvek mértékegységgel és mérési ablakkal rendelkeznek.
- A dokumentáció nem tartalmaz valós hitelesítő adatot, belső URL-t vagy ügyfélnevet.

11.3.27 9.5. Visszaállítási út

Mivel a labor vázlatkészítés, nincs destruktív művelet. Ha a hallgató kiegészítő PoC-t futtat egy egyszerű plug-in betöltéshez, azt izolált fejlesztői környezetben tegye, és a kiinduló állapot visszaállításához elegendő a PoC folyamat leállítása és a tesztadat törlése. A PoC nem éles rendszeren fut.

11.3.28 9.6. Mért és várt eredmény elkülönítése

A labor alapvetően dokumentációs feladat, így a „mért” eredmény csak akkor van, ha a hallgató kiegészítő teljesítménymérést végez. Ebben az esetben a dokumentumban külön kell választani:

- **Várt eredmény:** például a plug-in betöltési idő legfeljebb 1 másodperc, a mag memóriahasználat növekedése $< 5\%$.
- **Mért eredmény:** például a hallgató notebookjában a betöltési idő 0,6 másodperc volt 20 próbálkozás átlagában, az adott környezetben (pl. 4 vCPU, 8 GB RAM, Python 3.12, 2025-04-01).
- **Bizonytalanság:** a minta 20 elemű, a szórás 0,1 másodperc, a mérés nem termelési környezetben történt.

Ezzel teljesül a könyv mérési szerződése, és az eredmény ellenőrizhetővé válik.

11.3.29 10. Ellenőrzőlista a csillag- és mikro-kernell architektúrához

A fejezet lezárásaként az alábbi ellenőrzőlista segít felmérni, mennyire megalapozott a választott architektúra:

- ☐ Azonosítottuk, hogy a cél integráció (hub-and-spoke) vagy termékbővíthetőség (mikro-kernell).
- ☐ A mag felelőssége minimális, és nem tartalmaz üzleti logikát.
- ☐ A plug-inek függetlenek, és egymással nem kommunikálnak közvetlenül.
- ☐ A mag és a plug-in közötti interfész verziózott, dokumentált és stabil.
- ☐ A plug-inek izolációja a kockázatnak megfelelő szintű (folyamat, konténer, osztálybetöltő).
- ☐ A mag hibatűrést valósít meg: timeout, retry, circuit breaker, bulkhead.
- ☐ A plug-in manifest tartalmazza a verziót, a függőségeket és az erőforráskorlátokat.
- ☐ A C4 context és component szintű diagram kész, a rendszerhatár jelölve van.
- ☐ A kapcsolatokon szerepel a protokoll, a szinkronitás és a hibatűrés.
- ☐ Az ADR rögzíti a döntést, az alternatívákat és a következményeket.
- ☐ A minőségi forgatókönyvek inger, környezet, válasz és mérőszám formátumban vannak.
- ☐ A központi kockázatot értékeltük, és a mérséklő intézkedéseket dokumentáltuk.

11.3.30 11. Önellenőrző kérdések

1. Milyen követelmények esetén választanál mikro-kernell architektúrát hub-and-spoke helyett?
2. Hogyan csökkentenéd a központi mag kockázatát egy plug-in alapú rendszerben?
3. Milyen elemekből áll a mag és a plug-in közötti interfész szerződése?
4. Miért érdemes megtiltani a plug-inek közötti közvetlen kommunikációt?
5. Hogyan mérnéd a bővíthetőséget és a hibatűrést egy mikro-kernell rendszerben?

11.3.31 12. Összegzés

A csillag-topológia (hub-and-spoke) és a mikro-kernell (plug-in) architektúra közös vonása a központi elem, de más problémát oldanak meg. A hub-and-spoke az integrációt és a közlekedést szervezi, míg a mikro-kernell a termék funkcionális kiterjeszthetőségét. Mindkét minta erőssége a központosított felügyelet és az egyszerűbb csatlakoztatás, gyengesége pedig a központi kockázat és a lehetséges szűk keresztmetszet.

A jó mikro-kernell architektúra **minimális magot**, **stabil interfészt** és **erős izolációt** alkalmaz. A jó hub-and-spoke pedig **korlátozott hub-felelősséget**, **megfelelő kapacitástervezést** és **hibatűrő routingot**. A döntést mindig a konkrét kontextus határozza meg: milyen a csapat, a terhelés, a változási sebesség és az élettartam.

A következő fejezet (UI-architektúrák: MVC, MVP és MVVM) az UI-architektúrákat tárgyalja, ahol a plug-in gondolat újra felbukkan, amikor a felhasználói felületek moduláris felépítését és a nézetek szétválasztását vizsgáljuk. A itt bevezetett fogalmak – interfész-szerződés, izoláció, központi kockázat – ott is alapvetőek lesznek.

11.4 UI-architektúrák: MVC, MVP és MVVM

A felhasználói felület és az üzleti logika viszonya az egyik leggyakrabban félreértett architekturális kérdés a monolitikus és moduláris rendszerekben. A fejezet célja, hogy a három klasszikus UI-minta – MVC, MVP és MVVM – segítségével bemutassa, hogyan lehet a nézeteket vékonyan tartani, a logikát tesztelhetővé tenni, és a függőségeket a helyes irányba rendezni. Az itt tárgyalt elvek nemcsak asztali vagy mobil alkalmazásokban, hanem webes frontendeken és akár szerveroldali renderelés esetén is alkalmazhatók.

11.4.1 A felhasználói felület és az üzleti logika szétválasztása

A szoftverarchitektúra egyik alapelve a **felelősségek szétválasztása** (Separation of Concerns, SoC). A felhasználói felület két nagy felelősséget hordoz: egyrészt megjeleníti az adatokat, és lehetővé teszi a felhasználói interakciót, másrészt pedig gyakran – hibásan – magába szívja a feldolgozási, validálási és állapotkezelési feladatokat is. A szétválasztás célja, hogy a látvány, a vezérlési logika és a tartalmi (üzleti) szabályok külön komponensekbe kerüljenek, így:

- a nézetek cserélhetők legyenek anélkül, hogy az üzleti logikát érintenénk;
- az üzleti logika tesztelhető legyen UI nélkül, gyors egységtesztekkel;
- a változások lokálisak maradjanak: egy vizuális módosítás ne okozzon szabálysértést, és egy üzleti szabály változása ne törje el a felületet;
- a fejlesztők párhuzamosan dolgozhassanak a rétegeken.

A szétválasztás megvalósítására több minta létezik. A legismertebbek a **MVC (Model-View-Controller)**, az **MVP (Model-View-Presenter)** és az **MVVM (Model-View-ViewModel)**. Mindhárom közös alapja a **modell** elkülönítése: a modell felel a tartalmi adatokért, az üzleti szabályokért, a perzisztenciáért és a külső szolgáltatásokkal való kommunikációért. A modell nem tud a nézet létezéséről, és nem függ a UI keretrendszertől. Ez a függetlenség az elsődleges architekturális cél: a modell önmagában tesztelhető, újrahasznosítható más felületeken, és védve van a prezentációs réteg változásaitól.

A három minta abban különbözik, hogy a **nézet** és a **maradék felelősség** hogyan van felosztva, és milyen irányú függőségek vannak közöttük. A közös kérdés: ki figyeli a felhasználói eseményeket,

ki frissíti a nézetet, és ki tartja a nézet állapotát? A válasz egyben a tesztelhetőség és a karbantarthatóság kulcsa is.

11.4.2 A controller, a presenter és a view-model szerepe

Bár a három minta szereplőit gyakran szinonimaként emlegetik, a felelősségük eltér.

A **controller** (MVC-ben) a beérkező interakciók feldolgozásáért, a modell megfelelő műveleteinek meghívásáért és a nézet kiválasztásáért vagy frissítéséért felel. A klasszikus, kisalkalmazásos MVC-ben a controller a bemeneti eseményeket fogadja, frissíti a modellt, és a modell értesíti a nézetet (observer pattern). A webes MVC-ben a controller egy HTTP kérés leképezése egy modellműveletre, majd a válasznézet és a modelladatok összekapcsolása.

A **presenter** (MVP-ben) a nézet passzívvá tételére szolgál. A nézet csak a megjelenítést és az események továbbítását végzi, minden állapot- és vezérlési logika a presenterben lakik. A presenter a nézet interfészén keresztül kérdez le és ír, így a nézet teljesen lecserélhető egy tesztduplánra. A presenter tipikusan nem ismeri a konkrét UI technológiát, csak egy absztrakt nézetfelületet.

A **view-model** (MVVM-ben) a nézet **állapotát** és **viselkedését** modellezi, de nem ismeri a nézet konkrét megvalósítását. A view-model általában az adatkötésre támaszkodik: a nézet deklaratívan kötődik a view-model tulajdonságaihoz és parancsaihoz. A view-model nem hív közvetlenül nézeti metódusokat, hanem értesítési mechanizmussal (például `INotifyPropertyChanged`, `LiveData`, `Observable`) jelzi a változásokat. Ez teszi a view-modelt kiválóan egységtesztelhetővé, hiszen nincs szükség UI keretre a viselkedés ellenőrzéséhez.

A következő szakaszokban a három mintát részletesen bemutatjuk, kiemelve a résztvevőket, a függőségek irányát, a tipikus alkalmazási területeket és a buktatókat.

11.4.3 MVC: Modell–Nézet–Vezérlő

Az MVC a legrégebbi és legismertebb minta, amelyet Trygve Reenskaug írt le az 1970-es években a Smalltalk környezetben. Az eredeti cél a felhasználói felület interaktív alkalmazásokban történő szétválasztása volt. A minta három szereplőt definiál:

- **Modell**: az alkalmazás tartalmi adatai és üzleti logikája. Nem tud a nézetről vagy a controllerről. A modell képes értesíteni a megfigyelőit, ha állapota megváltozott.
- **Nézet (View)**: a modell állapotának megjelenítése. A nézet regisztrál a modell értesítéseire, és azok hatására újrarendereli magát. A nézet nem tartalmaz üzleti logikát, de tartalmazhat megjelenítési logikát (például formázás, elrendezés).
- **Vezérlő (Controller)**: a felhasználói bemeneteket kezeli. Az egérkattintásokat, billentyűleütéseket, gesztusokat lefordítja modellműveletekre, és esetleg a nézet megváltoztatására utasít. A controller nem tartalmaz üzleti logikát, csak vezérlési logikát.

A klasszikus MVC-ben a **függőségek iránya** a következő: a controller ismeri a modellt és a nézetet, a nézet ismeri a modellt, a modell senkit nem ismer. Az adatáramlás kettős:

1. A felhasználói eseményt a controller fogadja, frissíti a modellt.
2. A modell értesíti a nézetet a változásról, a nézet újra megjeleníti az adatokat.

Ez a megfigyelői (observer) kapcsolat a modell és a nézet között lehetővé teszi, hogy egy modell több nézet is megjelenítsen anélkül, hogy a modell ismerné őket. Ugyanakkor ez a kapcsolat gyakran okoz karbantartási nehézséget: a modell értesítési mechanizmusa összefonódik a nézet frissítésével, és nehéz lehet nyomon követni az események sorrendjét.

A **webes MVC** (például ASP.NET MVC, Spring MVC, Rails) némileg eltér: a kérés-vezérelt ciklusban a controller fogadja a HTTP kérést, lehívja a szükséges adatokat a modellből, majd kiválaszt egy nézetet (sablont), és átadja neki a megjelenítendő adatokat. Itt a nézet és a modell között nincs közvetlen megfigyelői kapcsolat; a controller közvetít. A minta itt inkább egy **modell–nézet–controller a kérés-vezérelt feldolgozásban** értelmezés, amely közelebb áll az MVP presenteréhez, de a controllerek továbbra is vékonyak, és a nézet passzív.

11.4.4 Tipikus hibák MVC-ben

A leggyakoribb anti-minta a **massive view controller** (elhízott vezérlő), különösen iOS és webalkalmazásokban. Ilyenkor a controller magába szívja a presenter és a view-model feladatait is: adatot formáz, állapotot tárol, üzleti szabályokat futtat, sőt néha hálózati hívásokat is végez. Ez a nézet és a controller szoros csatolásához, valamint tesztelhetetlen kódhoz vezet.

Egy másik hiba, amikor a nézet közvetlenül írja a modellt, megkerülve a controllert. Ezzel az üzleti logika kikerül a vezérlési pont alól, és a változások követése nehézzé válik. A szétválasztás akkor sikeres, ha minden modellművelet egyértelműen a controlleren keresztül történik, és a nézet csak a megjelenítésre szorítkozik.

11.4.5 MVP: Modell–Nézet–Presenter

Az MVP (Model-View-Presenter) a 2000-es évek elején vált népszerűvé, elsősorban a .NET és Java környezetekben. A minta célja, hogy a nézetet teljesen passzívvá tegye, és minden UI-val kapcsolatos logikát a presenterbe helyezzen. Az MVP két fő változata ismert:

- **Passive View** (passzív nézet): a nézet semmilyen logikát nem tartalmaz, csak a megjelenítésért és a bemeneti események továbbításáért felel. A presenter minden állapotváltozást kiszámít, és a nézet interfészén keresztül frissíti a nézetet.
- **Supervising Controller** (felügyelő vezérlő): a nézet megtart bizonyos egyszerű, adatkötéssel megoldható feladatokat (például egy lista frissítése), de az összetett interakciós logika a presenterben van. Ez a változat a klasszikus MVC és a passive view közötti átmenet.

Az MVP szereplői:

- **Modell**: ugyanaz, mint az MVC-ben – tartalmi adatok és üzleti logika.
- **Nézet (View)**: egy interfész (pl. `IInvoiceListView`), amely mögött a konkrét UI implementáció áll. A nézet csak a megjelenítést végzi, és a felhasználói eseményeket delegálja a presenternek. Nem tartalmaz üzleti vagy vezérlési logikát, maximum formázást.
- **Presenter**: fogadja a nézettől érkező eseményeket, meghívja a modell megfelelő műveleteit, majd a kapott adatokat visszaírja a nézet interfészén keresztül. A presenter nem ismeri a konkrét UI technológiát, ezért egységtesztelhető egy tesztnézettel.

Az adatáramlás MVP-ben ciklikus, de a függőségek iránya egyszerű: a **presenter ismeri a nézet interfészét és a modellt**, a nézet ismeri a presenterét (általában egy eseménykezelőn keresztül), a modell pedig független marad. A nézet nem ismeri a modellt közvetlenül, így a nézet és a modell közötti kapcsolat megszűnik.

A presenter tesztelése rendkívül egyszerű: a nézet interfészét egy tesztduplával (mock) helyettesítjük, majd ellenőrizzük, hogy a presenter helyesen hívja-e meg a modellt, és a kapott adatokkal helyesen frissíti-e a nézetet.

11.4.6 Tipikus hibák MVP-ben

A **fat presenter** anti-minta akkor jön létre, ha a presenter túl sok, nem UI-hoz kapcsolódó üzleti logikát is tartalmaz. A presenternek csak a nézet és a modell közötti koordinációt szabad végeznie; a tartalmi szabályoknak a modellben kell lenniük. Ha a presenterben üzleti szabályok, validációk vagy adatelérési kód jelenik meg, a modell kiürül, és a tesztelhetőség látszólag javul, de a logika nehezen lesz újrahasznosítható más felületeken.

Egy másik hiba a nézet interfészének túlzott részletezése: ha a presenter apró, UI-specifikus műveleteket hívogat (például `SetButtonEnabled`, `SetLabelColor`), az interfész a konkrét UI technológiához köti a presentert. A jó interfész technológiafüggetlen, üzleti jelentésű műveleteket tartalmaz (például `ShowInvoices(list)`, `ShowError(message)`).

11.4.7 MVVM: Modell–Nézet–View-Model

Az MVVM (Model-View-ViewModel) először a Microsoft WPF/Silverlight világában jelent meg John Gossman leírásában, de a koncepció azóta meghonosodott a mobil (Android, iOS) és webes (Angular, Vue, React) keretrendszerekben is. A minta a **adatkötés (data binding)** erejére épít, és a nézetet deklaratívvá teszi.

Az MVVM szereplői:

- **Modell**: változatlanul a tartalmi adatok és az üzleti logika.
- **View-Model (VM)**: a nézet **absztrakt állapotgépe**. Tartalmazza a nézethez szükséges összes adatot (publikus tulajdonságok formájában), valamint azokat a parancsokat (command) vagy viselkedéseket, amelyeket a felhasználói interakciók kiváltanak. A VM nem ismeri a nézetet, csak értesítési mechanizmust biztosít a tulajdonságainak változásáról (pl. `INotifyPropertyChanged`, `LiveData`, `Observable`). A VM a modellt használja az adatok betöltésére és a műveletek végrehajtására, de nem jelenít meg semmit.
- **Nézet (View)**: deklaratív módon definiálja a felhasználói felületet, és adatkötéssel köti a vezérlőket a VM tulajdonságaihoz és parancsaihoz. A nézet kódból csak minimális eseménykezelőt tartalmaz, lehetőleg semmilyen állapotot nem tárol.

Az adatáramlás az MVVM-ben egyirányú vagy kétirányú kötésekkel történik:

1. A felhasználói bemenet (például szöveg beírása) frissíti a VM megfelelő tulajdonságát (kétirányú kötés).
2. A VM tulajdonságának változásakor a rendszer értesíti a nézetet, amely automatikusan frissíti a megjelenítést (egyirányú kötés).
3. A felhasználói parancs (például gombnyomás) egy VM-beli parancsot hív meg, amely elvégzi a szükséges modellműveleteket, majd frissíti a VM tulajdonságait, ami a kötésen keresztül frissíti a nézetet.

A VM tehát **nem hívja a nézetet**, és nem ismeri a konkrét UI elemtípusokat. Ez teszi lehetővé, hogy ugyanaz a VM más megjelenítést is kiszolgáljon (például telefonos és táblagépes nézet), és hogy egységteszttel ellenőrizhető legyen anélkül, hogy UI keretrendszert indítanánk.

11.4.8 Példák MVVM-re különböző platformokon

- **WPF / .NET**: a VM általában az `INotifyPropertyChanged` interfészt valósítja meg, a parancsok pedig `ICommand` implementációk. A nézet XAML-ben köt, például `Text="{Binding CustomerName}"`.

- **Android**: a `ViewModel` osztály a `LiveData` vagy `StateFlow` segítségével értesíti a nézetet. A nézet (Activity/Fragment) megfigyeli a `LiveData` objektumot.
- **Webes**: Angularban a komponens jó közelítéssel VM-ként működik, a sablon pedig deklaratív kötést használ. Reactben a komponens állapota és a hookok egy VM-szerű réteget alkotnak, de a JSX-ben a nézet és a VM határa gyakran elmosódik.

11.4.9 Tipikus hibák MVVM-ben

A **view-model túlzott függése a nézettől** akkor fordul elő, ha a VM közvetlenül hivatkozik UI-elemekre, például elment egy `TextView` referenciát. Ez szétrobbantja a tesztelhetőséget és a platformfüggetlenséget.

Egy másik hiba a **kétirányú kötés túlzott használata**, ami bonyolult, nehezen követhető állapotváltozásokat okoz. Ilyenkor az egyes tulajdonságok közötti rejtett függőségek jelennek meg, és a program viselkedése kiszámíthatatlanná válik. A jó gyakorlat szerint a kötéseknek egyértelműnek és minimálisnak kell lenniük; ahol összetett interakció van, ott jobb parancsot használni.

Továbbá előfordul, hogy a VM üzleti logikát is tartalmaz, nem csak prezentációs állapotot. A VM feladata a **nézet állapotának** fenntartása és a modell műveleteinek koordinálása, nem a tartalmi szabályok implementálása. Ha a VM-ben üzleti validáció vagy számítás jelenik meg, az a modell felelősségi körébe tartozik, és a VM-nek csak a hibák megjelenítésre való leképezését szabad elvégeznie.

11.4.10 A három minta összehasonlítása

A minták közötti választás során a következő szempontokat érdemes mérlegelni:

Szempont	MVC (klasszikus)	MVP (passive view)	MVVM
A nézet szerepe	Aktív: megfigyeli a modellt	Passzív: csak megjelenít és eseményeket továbbít	Deklaratív: adatkötéssel csatlakozik a VM-hez
A köztes réteg	Controller: bemenet kezelése, modell frissítése	Presenter: minden nézetlogika, nézet interfészén keresztül	ViewModel: állapot és parancsok, értesítésekkel
Függőségek iránya	Controller → Modell, Nézet → Modell, Modell → senki	Presenter → ViewInterface, Presenter → Modell, Nézet → Presenter	ViewModel → Modell, View → ViewModel, Modell → senki

Szempont	MVC (klasszikus)	MVP (passive view)	MVVM
Tesztelhetőség	Controller és modell jól tesztelhető; a nézet kevésbé	Presenter kiválóan tesztelhető mock nézettel	A VM kiválóan tesztelhető, akár property-kötés nélkül is
Legjobb környezet	Szerveroldali web MVC	Asztali és mobil, ahol nincs adatkötés	Modern UI keretrendszerek erős adatkötéssel
Fő kockázat	Massive view controller	Fat presenter, túl részletes interfész	Túlzott kötés, VM által tárolt üzleti logika

A táblázatból látható, hogy a három minta közötti különbség leginkább a **nézet aktivitásában** és a **köztes réteg természetében** érhető tetten. A cél minden esetben ugyanaz: a modell függetlenítése, a nézet vékonyítása, és a középső réteg tesztelhetővé tétele.

A választás kontextusfüggő:

- Ha webes kérés-vezérelt környezetben dolgozunk, a szerveroldali MVC természetes választás, de figyeljünk a controllerek sovány ságára.
- Ha olyan UI technológiát használunk, amely nem támogat erős adatkötést (például hagyományos asztali alkalmazások, WinForms, Java Swing), az MVP jobb választás, mert a presenter elkülöníti a logikát a passzív nézettől.
- Ha a platform beépített adatkötést kínál (WPF, UWP, Android, iOS, modern webes keretrendszerek), az MVVM – vagy annak variánsai – csökkentik a boilerplate kódot és deklaratív nézetet tesznek lehetővé.

11.4.11 Tesztelhetőség és a nézetek függősége

A fejezet egyik kiemelt tanulási célja a tesztelhetőség és a nézetek függőségének megértése. A UI architektúrák sikerét azon mérhetjük, hogy a logika mekkora része tesztelhető UI indítása nélkül.

A **modell** mindig egységtesztelhető, mert nem függ a UI-tól. A **controller/presenter/view-model** tesztelhetősége a mintától és a megvalósítástól függ:

- **MVC controller**: ha a controller csak a modellt hívja és a nézetnek átadja az adatokat, akkor in-memory tesztelés lehetséges. A webes MVC-ben a HTTP környezet absztrakciója (pl. `HttpContextBase`) tesztelhetővé teszi a controllereket.
- **MVP presenter**: a legkönnyebben tesztelhető, mert a nézet interfészét mockkal helyettesíthetjük. A presenter tesztjei gyorsak, nem igényelnek UI keretet, és pontosan ellenőrizhető, milyen hívások történtek a nézetre.

- **MVVM view-model**: a legjobban tesztelhető, mivel nincs szükség semmilyen UI-ra, még interfészre sem. A VM pubkikus tulajdonságokkal és parancsokkal rendelkezik, így tesztekből egyszerűen meghívhatjuk a parancsokat, és ellenőrizhetjük az állapotváltozásokat. Az adatkötési mechanizmust általában nem szükséges tesztelni, mert az a keretrendszer felelőssége.

A **nézet** tesztelése a legnehezebb minden mintában. A UI tesztek lassúak, törékenyek, és erősen függenek a megjelenítési technológiától. Ezért a stratégiai cél az, hogy a nézetben minél kevesebb saját logika legyen. A nézet kódja ideális esetben csak annyit tesz, hogy:

- MVC-ben feliratkozik a modell értesítéseire, és újrarenderel;
- MVP-ben megvalósítja a nézet interfészét, és eseményeket továbbít a presenternek;
- MVVM-ben deklaratív kötéseket definiál, és a parancsokat a VM-hez köti.

Ezáltal a kockázatos logika a könnyen tesztelhető rétegbe kerül, a nézet pedig vékony marad.

Fontos szempont a **függőségek iránya**. A tiszta architektúra elvei szerint a függőségeknek a részletek felől az absztrakciók felé kell mutatniuk. A UI architektúrákban ez azt jelenti:

- a modell nem függ a UI-tól (ez mindig teljesül);
- a nézet függhet a modelltől (MVC klasszikus) vagy a presenter/view-modeltől (MVP/MVVM), de soha nem fordítva;
- a presenter/view-model nem függhet a konkrét nézettől, csak egy interfésztől vagy absztrakciótól.

Ha ezt a függőségi szabályt betartjuk, a rendszer rétegei cserélhetők, és a tesztek stabilak maradnak.

11.4.12 Gyakorlati példa: egy egyszerű számlakezelő képernyő

A három minta konkrét bemutatásához vegyünk egy egyszerű feladatot: **számlák listázása és egy új számla létrehozása**. A modell egy `Invoice` entitást és egy `InvoiceService` osztályt tartalmaz, amely a háttértárolóval kommunikál. A feladat megjelenítése a következő: a felhasználó lát egy listát a számlákról, beírhat egy új számla számát és összegét, majd a „Mentés” gombra kattintva a rendszer elmenti a számlát.

A példa kódjai szemléltető jellegűek, nem éles rendszerhez készültek, és nem tartalmaznak valós hitelesítő adatot vagy belső URL-t. Az egyszerűség kedvéért pszeudokódot vagy C#-szerű szintaxist használunk.

11.4.13 A modell

```
public class Invoice
{
    public string Number { get; set; }
    public decimal Amount { get; set; }
    public DateTime DueDate { get; set; }
}

public interface IInvoiceRepository
{
    List<Invoice> GetAll();
    void Add(Invoice invoice);
}
```

```
public class InvoiceService
{
    private readonly IInvoiceRepository _repository;

    public InvoiceService(IInvoiceRepository repository)
    {
        _repository = repository;
    }

    public List<Invoice> GetInvoices()
    {
        return _repository.GetAll();
    }

    public void CreateInvoice(string number, decimal amount, DateTime dueDate)
    {
        if (string.IsNullOrWhiteSpace(number))
            throw new ArgumentException("A számlaszám nem lehet üres.");

        if (amount <= 0)
            throw new ArgumentException("Az összegnek pozitívnak kell lennie.");

        var invoice = new Invoice
        {
            Number = number,
            Amount = amount,
            DueDate = dueDate
        };

        _repository.Add(invoice);
    }
}
```

A modell tökéletesen tesztelhető, nincs UI függőség.

11.4.14 MVC vázlat

Controller:

```
public class InvoiceController
{
    private readonly InvoiceService _service;
    private readonly IInvoiceView _view;

    public InvoiceController(InvoiceService service, IInvoiceView view)
    {
        _service = service;
        _view = view;
    }

    public void LoadInvoices()
    {
        var invoices = _service.GetInvoices();
        _view.ShowInvoices(invoices);
    }

    public void SaveInvoice(string number, decimal amount, DateTime dueDate)
    {
        try
        {
            _service.CreateInvoice(number, amount, dueDate);
            _view.ShowInvoices(_service.GetInvoices());
```

```
            _view.ShowMessage("Sikeres mentés.");
        }
        catch (ArgumentException ex)
        {
            _view.ShowError(ex.Message);
        }
    }
}
```

A nézet interfész:

```
public interface IInvoiceView
{
    void ShowInvoices(List<Invoice> invoices);
    void ShowMessage(string message);
    void ShowError(string error);
}
```

A konkrét nézet megvalósítja az interfészt, és az eseménykezelőkben továbbítja a hívásokat a controllernek. A controller itt vékony: nem tartalmaz üzleti logikát, csak koordinál.

11.4.15 MVP vázlat

A presenter hasonló, de a nézet passzív, és minden bemeneti esemény a presenterhez fut be:

```
public class InvoicePresenter
{
    private readonly InvoiceService _service;
    private readonly IInvoiceView _view;

    public InvoicePresenter(InvoiceService service, IInvoiceView view)
    {
        _service = service;
        _view = view;
        _view.Presenter = this;
    }

    public void Initialize()
    {
        var invoices = _service.GetInvoices();
        _view.ShowInvoices(invoices);
    }

    public void OnSaveClicked(string number, decimal amount, DateTime dueDate)
    {
        try
        {
            _service.CreateInvoice(number, amount, dueDate);
            var invoices = _service.GetInvoices();
            _view.ShowInvoices(invoices);
            _view.ShowMessage("Sikeres mentés.");
        }
        catch (ArgumentException ex)
        {
            _view.ShowError(ex.Message);
        }
    }
}
```

A nézet interfész itt bővebb lehet, de továbbra is technológiafüggetlen:

```
public interface IInvoiceView
{
    InvoicePresenter Presenter { set; }
    void ShowInvoices(List<Invoice> invoices);
    void ShowMessage(string message);
    void ShowError(string error);
    string GetInvoiceNumber();
    decimal GetInvoiceAmount();
    DateTime GetDueDate();
}
```

A nézet a gombnyomást a presenter `OnSaveClicked` metódusához köti, és a bemeneti mezők értékeit a getterekkel adja át. A presenter tesztelhető anélkül, hogy bármilyen UI elemet indítanánk.

11.4.16 MVVM vázlat

A view-model tulajdonságokkal és parancsokkal rendelkezik. A példában C# `INotifyPropertyChanged` és egy egyszerű `RelayCommand` implementációt feltételezünk (a keretrendszer tipikusan biztosít ilyet).

```
public class InvoiceViewModel : INotifyPropertyChanged
{
    private readonly InvoiceService _service;
    private ObservableCollection<Invoice> _invoices;
    private string _newNumber;
    private decimal _newAmount;
    private DateTime _newDueDate;

    public ObservableCollection<Invoice> Invoices
    {
        get => _invoices;
        private set { _invoices = value; OnPropertyChanged(); }
    }

    public string NewNumber
    {
        get => _newNumber;
        set { _newNumber = value; OnPropertyChanged(); }
    }

    public decimal NewAmount
    {
        get => _newAmount;
        set { _newAmount = value; OnPropertyChanged(); }
    }

    public DateTime NewDueDate
    {
        get => _newDueDate;
        set { _newDueDate = value; OnPropertyChanged(); }
    }

    public ICommand LoadCommand { get; }
    public ICommand SaveCommand { get; }

    public InvoiceViewModel(InvoiceService service)
    {
        _service = service;
        LoadCommand = new RelayCommand(_ => LoadInvoices());
        SaveCommand = new RelayCommand(_ => SaveInvoice());
```

```
    }

    private void LoadInvoices()
    {
        Invoices = new ObservableCollection<Invoice>(_service.GetInvoices());
    }

    private void SaveInvoice()
    {
        try
        {
            _service.CreateInvoice(NewNumber, NewAmount, NewDueDate);
            LoadInvoices();
            // Opcionálisan hibaüzenet tulajdonságot állítunk
        }
        catch (ArgumentException ex)
        {
            ErrorMessage = ex.Message; // INotifyPropertyChanged kell hozzá
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected virtual void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }

    public string ErrorMessage { get; private set; }
}
```

A nézet XAML-szerűen köt:

```
<ListBox ItemsSource="{Binding Invoices}">
    ...
</ListBox>
<TextBox Text="{Binding NewNumber, Mode=TwoWay}" />
<TextBox Text="{Binding NewAmount, Mode=TwoWay}" />
<DatePicker SelectedDate="{Binding NewDueDate, Mode=TwoWay}" />
<Button Command="{Binding SaveCommand}">Mentés</Button>
```

A view-model egyetlen UI elemet sem ismer, mégis a nézet teljes állapotát biztosítja. A tesztelés során létrehozunk egy `InvoiceViewModel` példányt egy in-memory repository-val, beállítjuk a `NewNumber` és `NewAmount` értékeket, meghívjuk a `SaveCommand` parancsot, majd ellenőrizzük, hogy a repository tartalmazza-e az új számlát.

11.4.17 Labor: ugyanazon feladat MVC, MVP és MVVM vázlata

A labor célja, hogy a fejezetben tanult mintákat egy konkrét, reprodukálható feladaton alkalmazd. A feladat nem éles rendszerhez készül, és nem igényel valós adatbázist vagy külső szolgáltatást.

11.4.18 Kiinduló állapot

A résztvevő rendelkezésére áll egy üres munkaterület, valamint egy egyszerű, szintetikus adatmodell: **Termék** entitás (`Id`, `Name`, `Price`, `InStock`), és egy `IProductRepository` interfész, amely a termékek listázását és hozzáadását teszi lehetővé. A labor megvalósítható bármilyen programozási nyelven vagy akár papíron is (például egy jegyzetfüzetben), de a cél a szerkezeti vázlat és a függőségek ábrázolása, nem a teljes működő kód.

A laborhoz tartozik egy **döntési napló** (ADR-szerű bejegyzés), amely rögzíti a feladat kontextusát, a választott mintákat és a döntések indoklását.

11.4.19 A feladat

Készítsd el **ugyanannak a feladatnak** a vázlatát három változatban: MVC, MVP és MVVM. A feladat egy egyszerű **termékkezelő képernyő**: a felhasználó látja a termékek listáját, szűrhet a termék nevére, és felvehet egy új terméket (név, ár, készletmennyiség). A három vázlatban legalább a következőket kell meghatározni:

1. A komponensek (nézet, controller/presenter/view-model, modell) és azok felelősségei.
2. A függőségek iránya (ki ismer kit).
3. Az eseménykezelés módja (hogyan jut el a gombnyomás a modellhez, és hogyan frissül a nézet).
4. A tesztelhetőségi stratégia: mit lehet egységteszttel tesztelni, és mi igényel UI-tesztet.

A vázlatokat egy **reprodukálható dokumentumban** kell rögzíteni: például egy Markdown fájlban, egy jegyzetfüzetben vagy diagramokkal ellátott szövegként. A dokumentum tartalmazzon:

- rövid kontextusleírást (alkalmazás típusa, platform, csapatméret);
- mindhárom minta vázlatát – lehet szöveges leírás, pszeudokód vagy diagrams-as-code (pl. Mermaid osztálydiagram);
- a döntési naplót, amely rögzíti, melyik mintát miért választanád egy adott platformon, és milyen alternatívákat vetettél el;
- ellenőrzési szempontokat, amelyek alapján a vázlat megfelelősége értékelhető.

A dokumentumnak külön kell választania a **mért** és a **várt** eredményt. Mivel ebben a laborban nem futtattunk kódot, a „mért” részben az szerepelhet, hogy a vázlatokat manuálisan ellenőriztük-e, például összehasonlítottuk a három minta függőségi irányait. A „várt” részben azt rögzítsük, mit várunk el egy jól elkészített vázlattól (pl. a modell nem függ a UI-tól, a presenter nem ismeri a konkrét nézetet).

11.4.20 Visszaállítási út

Mivel a labor nem éles rendszeren fut, a visszaállítás egyszerű: a vázlatok és a döntési napló törölhetők, vagy a munkaterület újra inicializálható. Ha a feladatot kódban oldottuk meg, a repository-t vissza kell állítani a kiinduló állapotra (például `git checkout -- .`). Nincs szükség adatbázis-visszaállításra, mert a feladat csak vázlatokat kér.

11.4.21 Ellenőrzési szempont

A labor értékelésénél a következőket ellenőrizzük:

- A modell egyik vázlatban sem függ a UI-tól.
- A nézet nem tartalmaz üzleti logikát; legfeljebb formázást és eseménytovábbítást.
- A controller/presenter/view-model felelőssége egyértelműen el van határolva.
- A függőségek iránya minden diagramon jelölve van, és a függőségek a stabilabb absztrakciók felé mutatnak.
- A tesztelési stratégia megnevezi, mely rétegek tesztelhetők UI nélkül.
- A döntési napló indokolja a választott mintát legalább egy konkrét kontextusban.

11.4.22 Döntési gyakorlat

Olvasd el az alábbi helyzetet, és válassz mintát, majd indokold meg a döntésedet.

Helyzet: Egy asztali számlázó alkalmazást fejlesztetek Windowsra, a csapat 6 főből áll. Az alkalmazás összetett űrlapokat tartalmaz, sok beviteli mezővel és dinamikusan engedélyezett/tiltott gombokkal. A UI keretrendszer WinForms, amely nem támogat beépített adatkötést. A termékmenedzser szerint a jövőben webes felületet is szeretnének, ezért fontos, hogy az üzleti logika ne keveredjen a UI kóddal. A tesztautomatizálás jelenleg minimális, de a csapat szeretné növelni az egységtesztek arányát.

Kérdés: Melyik mintát választanád (MVC, MVP, MVVM), és miért? Milyen kockázatokkal jár a választásod?

Megoldási irány: A WinForms nem támogatja az MVVM-hez szükséges erős adatkötést, ezért az MVVM nehézkes lenne. Az MVC klasszikus változata a modell és a nézet közötti megfigyelői kapcsolattal szintén bonyolult lehet. Az MVP passive view változata a legjobb választás: a nézet passzív, a presenter tesztelhető, és a nézet interfész cseréje megkönnyíti a későbbi webes migrációt. Kockázat: a presenter elhízhat, ha nem figyelünk oda, ezért a modellnek kell tartalmaznia az üzleti szabályokat.

11.4.23 Ellenőrzőlista

A fejezet elolvasása után ellenőrizd, hogy megértetted-e az alábbiakat:

- ☐ Képes vagy megfogalmazni, miért fontos a felhasználói felület és az üzleti logika szétválasztása.
- ☐ Ismered az MVC, MVP és MVVM három szereplőjét, és el tudod magyarázni a köztük lévő különbséget.
- ☐ Tudod, hogy a controller, a presenter és a view-model milyen felelősséggel bír.
- ☐ Meg tudod indokolni, melyik minta mikor előnyös, figyelembe véve a platform adatkötési lehetőségeit.
- ☐ Felismered a tipikus anti-mintákat: massive view controller, fat presenter, üzleti logika a view-modelben.
- ☐ Képes vagy egy egyszerű feladatra vázlatot készíteni mindhárom mintában, és a függőségek irányát jelölni.
- ☐ Tudod, hogyan tesztelhető az egyes rétegek UI nélkül.

11.4.24 Önellenőrző kérdés

Magyarázd el, miért jobb a tesztelhetőség szempontjából az MVP passive view változata a klasszikus MVC-nél, ha a UI keretrendszer nem támogat adatkötést. Milyen konkrét mechanizmus teszi a presentert könnyebben tesztelhetővé, és hogyan csökkenti a nézetben rejlő kockázatot?

Válaszvázlat: Az MVP passive view-ban a nézet csak egy interfészt valósít meg, és minden interakciós logika a presenterben van. A presenter teszteléséhez elegendő a nézet interfészét mockolni, így nincs szükség UI keretre. Ezzel szemben a klasszikus MVC-ben a nézet aktívan megfigyeli a modellt, ami a nézetben tárol állapotot és frissítési logikát, így a nézet tesztelése nehézkes. A presenter tesztjei gyorsak, determinisztikusak, és pontosan ellenőrizhető, milyen hívások történtek a nézetre, míg a klasszikus MVC-ben a modell értesítési mechanizmusa miatt a teszt bonyolultabb.

11.4.25 Összegzés

A fejezet bemutatta a három klasszikus UI-architektúra mintát, amelyek mind ugyanazt a célt szolgálják: a felhasználói felület és az üzleti logika szétválasztását, a nézet vékonyítását és a tesztelhetőség javítását. Az MVC, az MVP és az MVVM közötti választás mindig kontextusfüggő, de a közös elvek – a függőségek iránya, a modell függetlensége és a középső réteg tesztelhetősége – minden esetben irányadók. A következő fejezetben a mikroszolgáltatásokra való áttérés során ezek a rétegzési és szétválasztási elvek újra előkerülnek, immár szolgáltatáshatárok szintjén.

[Ez a szöveg a könyvterv szerinti UI-architektúrák: MVC, MVP és MVVM alfejezet, körülbelül 3750 szó. A tartalom kizárólag a megadott témákat fedi le, és nem ismétli a korábbi fejezetek alapjait, illetve nem tér ki a következő egység részletes anyagára. A labor a könyv laborszerződésének megfelelően tartalmaz kiinduló állapotot, elvárt eredményt, ellenőrzési szempontot és visszaállítási utat. A kódpéldák szemléltetők, éles rendszeren nem végrehajtandók.]

12 Mikroszolgáltatások és elosztott rendszerek

12.1 A mikroszolgáltatások ígérete és ára

A mikroszolgáltatások architekturális stílusa nem egyszerűen egy újabb divat, hanem válasz egy valós problémára: ha egy rendszer és a mögötte álló szervezet elér egy bizonyos méretet, a monolitikus struktúra fejlesztési, telepítési és üzemeltetési feszültségeket okoz. A mikroszolgáltatások ezt a feszültséget úgy kezelik, hogy a rendszert egymástól lazán kapcsolt, önállóan telepíthető szolgáltatásokra bontják. Ez a felbontás azonban nem ingyen van: a hálózati kommunikáció, az elosztott rendszerek hibamódjai és az üzemeltetési komplexitás olyan árat jelentenek, amelyet a csapatnak meg kell fizetnie. Ez a fejezet ezt a kettősséget járja körül: mi az ígéret, mi az ár, és hogyan lehet a döntést tudatosan meghozni.

12.1.1 A mikroszolgáltatások meghatározása és az ígéret magja

A mikroszolgáltatás olyan architekturális stílus, amelyben a rendszer kis, önállóan telepíthető, lazán kapcsolt szolgáltatások halmazából épül fel. Minden szolgáltatás egy jól körülhatárolt üzleti képességet valósít meg, saját adattárolással, saját telepítési egységgel és saját csapattal rendelkezhet. A definícióban a „kis” szó nem a kódsorok számára utal elsősorban, hanem arra, hogy a szolgáltatás akkora legyen, hogy egy csapat felelősségi körébe férjen, és hogy a szolgáltatás határain belüli változtatás ne igényeljen más csapatokkal való egyeztetést.

A mikroszolgáltatások ígérete két fő pilléren nyugszik: a **független telepítésen** és a **szervezeti skálázáson**. Ezek egymást erősítik, de külön-külön is értelmezhetők.

12.1.2 Független telepítés

A független telepítés azt jelenti, hogy egy szolgáltatás új verzióját anélkül telepíthetjük éles környezetbe, hogy a rendszer többi részét újra kellene építeni vagy újraindítani. A monolitikus rendszerekben a telepítés tipikusan „mindent vagy semmit” jellegű: egy apró hiba a rendeléskezelő modulban a teljes alkalmazás leállását okozhatja, és a kiadási ciklusok összehangolást igényelnek. A mikroszolgáltatásoknál a telepítési egység maga a szolgáltatás, így a rendeléskezelő szolgáltatás frissítése nem érinti a felhasználókezelőt vagy a termékkatalógust.

Ennek gyakorlati előnyei:

- **Gyorsabb release-ciklus.** A csapatok saját ritmusban szállíthatnak, nem kell megvárniuk a közös kiadási ablakot. Ez lerövidíti a visszacsatolási hurkot, és lehetővé teszi a folyamatos szállítást.
- **Kisebb kockázatú telepítés.** Ha egy szolgáltatás rossz verziója kerül élesbe, a hiba hatóköre lehatárolt, és a visszaállítás (rollback) csak az adott szolgáltatást érinti.
- **Finomabb erőforrás-kezelés.** A gyakran változó vagy nagy terhelésű szolgáltatást külön skálázhatjuk, míg a stabil szolgáltatások változatlanul futhatnak.

- **Technológiai heterogenitás.** Mivel a szolgáltatások hálózaton, szerződéseken keresztül kommunikálnak, egy adott szolgáltatást megírhatunk abban a nyelvben vagy keretrendszerben, amely a feladatra legjobban illik, anélkül hogy a teljes rendszert át kellene állítani. Ez a szabadság azonban költséggel jár, ahogy később látni fogjuk.

Fontos hangsúlyozni, hogy a független telepítés nem csupán technikai tulajdonság, hanem szervezeti következménye is van. Ha a telepítés független, akkor a szolgáltatást birtokló csapat nem blokkol másokat, és nem blokkolja őt senki. Ebből nő ki a második ígéret.

12.1.3 Szervezeti skálázás és csapatautonómia

A nagy szervezetekben a fejlesztői csapatok párhuzamosan dolgoznak ugyanazon a kódbázison. A monolitikus rendszereknél ez gyakran konfliktusokhoz, integrációs problémákhoz és koordinációs költségekhez vezet: a csapatok ugyanazt a kódot módosítják, a build és a tesztelés globális állapotot feltételez, és a release-koordináció egyre nehezebb. A mikroszolgáltatások ezt a szervezeti problémát a **felelősségi határok mentén történő szétválasztással** oldják: minden csapat a saját szolgáltatásainak kódbázisáért, telepítéséért és üzemeltetéséért felel, a csapatok közötti együttműködés pedig a szolgáltatások közötti API-szerződésekre korlátozódik.

Ezt a jelenséget gyakran a Conway-törvény írja le: a rendszerek architektúrája tükrözi a szervezet kommunikációs struktúráját. A mikroszolgáltatások lehetővé teszik, hogy a szervezeti struktúra és a szoftverarchitektúra egymáshoz igazodjon: ha egy csapat felel egy üzleti képességért, akkor a kód, az adatbázis és a telepítési folyamat is az övé. Ez a **csapatautonómia** csökkenti a koordinációs költségeket, növeli a fejlesztői elégedettséget, és gyorsabb döntéshozatalt tesz lehetővé.

A szervezeti skálázás előnyei:

- **Párhuzamos fejlesztés.** Több csapat egyszerre dolgozhat, anélkül hogy egymásra várnának.
- **Egyértelmű felelősség.** A „ki a felelős?” kérdésre egyértelmű válasz adható.
- **Lokális döntések.** A csapat a saját szolgáltatásának technológiájáról, adatmodelljéről és belső architektúrájáról önállóan dönthet.
- **Méretezhető onboarding.** Az új csapatok saját szolgáltatásként csatlakozhatnak, anélkül hogy a teljes kódbázist kellene megérteniük.

Ugyanakkor a csapatautonómia sem korlátlan: a szolgáltatások közötti interfészeknek, az adatvédelmi és biztonsági előírásoknak, valamint a közös platformszolgáltatásoknak közös szabályoknak kell megfelelniük. A „minden csapat mindent maga csinál” hozzáállás szélsőséges esetben káoszhoz vezet.

12.1.4 Egyéb ígéretek: skálázhatóság és reziliencia

A mikroszolgáltatások mellett gyakran elhangzanak további érvek is. A **skálázhatóság** egyik előnye, hogy a terhelés alapján csak azokat a szolgáltatásokat skálázhatjuk, amelyekre szükség van; egy monolitnál a teljes alkalmazást kell skálázni, ami pazarló lehet. A **reziliencia** pedig abból fakad, hogy ha egy szolgáltatás meghibásodik, a többi szolgáltatás tovább működhet – legalábbis elvileg. A gyakorlatban ez a reziliencia nem automatikus, hanem meg kell tervezni: a hálózati hibák, az időtúllépések és a kaszkád meghibásodások ellen explicit mechanizmusokat kell bevezetni.

12.1.5 Az ár: amit fizetni kell

A mikroszolgáltatások bevezetése nem egyenlő a monolitikus kód egyszerű feldarabolásával. Az elosztott rendszerek alapvetően más hibamódokkal és működési feltételekkel rendelkeznek, mint az egy folyamatban futó monolit. Az árat három fő területen kell megfizetni: a hálózati hibák, az operációs teher és a megfigyelhetőség.

12.1.6 Hálózati hiba

A monolitikus rendszerben a modulok közötti hívás egy folyamaton belül történik, ezért a hibák viszonylag ritkák és jól kezelhetők. A mikroszolgáltatásoknál minden szolgáltatásközi kommunikáció hálózaton zajlik, ahol a hívás megszakadhat, elveszhet, lassú lehet, vagy a válasz soha nem érkezik meg. Ez a **parciális hiba** jelensége: a rendszer egy része meghibásodik, miközben más részei működnek. A parciális hiba kezelése alapvető tervezési feladat.

A hálózati hibák következményei:

- **Kaszkád meghibásodás.** Ha egy szolgáltatás nem válaszol, a tőle függő szolgáltatások is hibát okozhatnak, és a hiba hullámként terjedhet a rendszerben.
- **Időtúllépések és újrapróbálkozások.** A hívó félnek időtúllépést kell beállítania, különben a hívások felhalmozódnak, és elfogynak az erőforrások. Az újrapróbálkozások viszont tovább terhelhetik a már túlterhelt szolgáltatást, ha nincs megfelelő vezérlés (például circuit breaker).
- **Adatkonzisztencia.** Az elosztott tranzakciók a hálózati hibák miatt nem működnek úgy, mint egy adatbázisban. A konzisztencia biztosítása kompromisszumokkal jár, és gyakran a végleges konzisztencia vagy a kompenzációs műveletek irányába tolja a tervezést.

A hálózati hibák hatása mérhető. A rendelkezésre állás például az alábbi képlettel számítható:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Ha egy szolgáltatás 99,9%-os rendelkezésre állást ígér, az évente körülbelül 8,76 óra állásidőt enged meg. A mikroszolgáltatásoknál a teljes rendszer rendelkezésre állása a szolgáltatások rendelkezésre állásának szorzata (feltéve, hogy sorosan kapcsolódnak és függetlenek). Tíz darab 99,9%-os szolgáltatás esetén a rendszer rendelkezésre állása már csak $0{,}999^{10} \approx 0{,}990$, vagyis 99,0%, ami évi 87,6 óra állásidőt jelent. Ez a számítás jól mutatja, hogy a hálózati hibák és a szolgáltatások közötti függőségek hogyan erodálják a megbízhatóságot, ha nem tervezünk ellenük.

A hálózati hibákra adott válaszok – például timeout, retry, circuit breaker, bulkhead – a reziliencia eszköztárába tartoznak, és részletesen a V. részben kerülnek tárgyalásra. Itt a lényeg annak felismerése, hogy a mikroszolgáltatások bevezetésével a hálózati hiba **elsőrendű tervezési szemponttá** válik, nem pedig kivételes eseménnyé.

12.1.7 Operációs teher

A mikroszolgáltatások több önállóan futó folyamatot jelentenek, amelyeket telepíteni, konfigurálni, felügyelni és frissíteni kell. Ez az operációs teher jelentősen megnő a monolithoz képest.

A főbb elemek:

- **Deployment pipeline-ok.** Minden szolgáltatásnak saját build-, tesztelési és telepítési folyamata van. Ha húsz szolgáltatásunk van, húsz pipeline-t kell karbantartani, ami önmagában is kihívás.
- **Konfigurációkezelés.** A szolgáltatásoknak környezetenként eltérő konfigurációra van szükségük, amelyet biztonságosan, verziózottan és auditálhatóan kell kezelni. A titkok tárolása és terjesztése külön figyelmet igényel.
- **Szolgáltatásfelderítés és útválasztás.** A szolgáltatások dinamikusan változó hálózati címeit kezelni kell, amihez service discovery, terheléselosztó és API gateway szükséges.
- **Konténerizáció és orchestráció.** A modern mikroszolgáltatásokat gyakran konténerekben futtatják, és Kubernetesen vagy hasonló platformon orchestrálják. Ez hatékony, de új absztrakciókat és üzemeltetési ismereteket igényel.
- **Naplózás és monitorozás.** A hibakereséshez a szolgáltatások naplóit össze kell gyűjteni, és korrelálni kell a kéréseket a szolgáltatások között. Ez a megfigyelhetőség témakörébe vezet át.

Az operációs teher csökkenthető platform engineeringgel, belső fejlesztői platformmal és standardizált eszközökkel, de ezt a befektetést a szervezetnek vállalnia kell. Egy kis csapat számára a mikroszolgáltatások operációs költsége gyakran meghaladja a fejlesztési sebességből származó előnyt.

12.1.8 Megfigyelhetőség

A megfigyelhetőség arra a képességre utal, hogy a rendszer belső állapotát a külső jelekből – naplókból, metrikákból és nyomkövetésekből – rekonstruálni tudjuk. A monolitban egyetlen folyamat naplója és a debugger gyakran elegendő. A mikroszolgáltatásoknál egy felhasználói kérés akár tíz vagy húsz szolgáltatáson is áthalad, és a hiba okának megtalálásához a teljes láncot látni kell.

A megfigyelhetőség hiánya a következő problémákhoz vezet:

- **Hosszú hibakeresési idő.** Ha egy kérés sikertelen, nem tudjuk, melyik szolgáltatás hibásodott meg, és miért. A naplók manuális átvizsgálása lassú és hibalehetőséget hordoz.
- **Rejtett teljesítményproblémák.** A lassú válaszidő oka lehet a hálózat, a sorban állás, a szerializáció vagy bármelyik szolgáltatás belső feldolgozása. Metrikák és nyomkövetés nélkül csak találgatunk.
- **Nem megbízható riasztások.** Ha nincsenek jól definiált SLO-k és hozzájuk kapcsolt SLI-k, a riasztások vagy túl sokak, vagy elmaradnak.

A megfigyelhetőséget a kezdetektől fogva architekturális követelményként kell kezelni, nem pedig utólag hozzáadott extraként. Az OpenTelemetry-hez hasonló eszközök segítenek a metrikák és nyomkövetések szabványosításában, de a sémák kialakítása és a kontextusok propagálása tervezést igényel.

12.1.9 Egyéb költségek

A fentieken túl a mikroszolgáltatások további árakkal járnak:

- **Adatkezelési komplexitás.** Minden szolgáltatás saját adattárolással rendelkezik, ami megnehezíti a tranzakciókat, a lekérdezéseket és az adatkonzisztenciát. Az elosztott tranzakciók és a végleges konzisztencia mintái kompromisszumot jelentenek.

- **Tesztelési komplexitás.** Az egységteszt viszonylag egyszerű, de az integrációs és végpontok közötti tesztek a szolgáltatások közötti szerződéseket és a hálózati hibákat is kezelni kényszerülnek. A tesztkörnyezetek felépítése drága.
- **Biztonsági kihívások.** A szolgáltatásközi kommunikáció hitelesítést és titkosítást igényel, a támadási felület megnő, és a titkok kezelése bonyolultabbá válik.
- **Szervezeti érettség.** A mikroszolgáltatások csak akkor működnek jól, ha a szervezet DevOps-gyakorlatokat, automatizált tesztelést és folyamatos szállítást alkalmaz. Ellenkező esetben a mikroszolgáltatások csak növelik a komplexitást anélkül, hogy az előnyöket realizálnák.

12.1.10 Gyakorlati példa: rendelési rendszer

Vegyünk egy egyszerű e-kereskedelmi rendszert, amelyben a megrendelés és a raktárkészlet kezelése két külön szolgáltatásként működik. A rendszer kontextusdiagramja (C4 context szint) az alábbiak szerint ábrázolható:

```
graph TD
    Vásárló[Vásárló] -->|HTTPS| Web[Webes felület]
    Web -->|HTTP/REST| Rendelés[Rendelés szolgáltatás]
    Web -->|HTTP/REST| Készlet[Készlet szolgáltatás]
    Rendelés -->|HTTP/REST| Készlet
```

A **független telepítés** előnye itt: ha a készletkezelő szolgáltatásban hibát javítunk, a rendelési szolgáltatást nem kell újraindítani. A készlet-szolgáltatás csapata saját ütemezésben szállít, a rendelési csapat nem blokkolódik.

A **hálózati hiba** árát azonban egyetlen példa jól mutatja: a rendelési szolgáltatás a rendelés rögzítése előtt ellenőrzi a készletet. Ha a készlet-szolgáltatás nem válaszol 500 ms-on belül, a rendelési szolgáltatás időtúllépést kap. Mit tegyen ilyenkor? Ha hibát dob, a felhasználó elveszíti a kosarát. Ha újrapróbálkozik, tovább terhelheti a már lassú készlet-szolgáltatást. Ha feltételezi, hogy a készlet rendben van, akkor túlértékesítés történhet. Ez a döntés nem triviális, és architekturális szinten kell kezelni: például a rendelés rögzítése aszinkron üzenettel, a készlet ellenőrzése pedig kompenzációs folyamattal történhet.

A **megfigyelhetőség** hiányát is érzékelhetjük: ha egy felhasználó rendelése nem sikerül, a hibakereséshez látnunk kell a rendelési szolgáltatás naplóját, a készlet-szolgáltatás naplóját és a köztük zajló HTTP-hívás nyomkövetését. Ha ezek a jelek nincsenek összekapcsolva, a hiba oka rejtve marad.

12.1.11 Quality attribute scenario példa

A mikroszolgáltatásokkal kapcsolatos minőségi követelményeket célszerű forgatókönyvek formájában rögzíteni. Íme egy példa:

> **Inger:** A készlet-szolgáltatás nem válaszol a rendelési szolgáltatás HTTP GET kérésére. **Környezet:** Normál terhelés, 100 kérés/másodperc, a készlet-szolgáltatás egy példánya leállt. **Válasz:** A rendelési szolgáltatás 3 másodpercen belül időtúllépést ad, a kérést nem próbálja meg végtelenül újra, és a felhasználó számára egyértelmű hibaüzenetet küld. A rendelési szolgáltatás saját állapotát továbbra is képes kezelni, és nem esik le. **Mérőszám:** A rendelési szolgáltatás p95 válaszideje < 500 ms, a sikeres rendelések aránya $> 99\%$, az incidens időtartama < 5 perc.

Ez a forgatókönyv egyértelműen megfogalmazza a hálózati hiba kezelésének elvárását, és mérhető célokat ad.

12.1.12 ADR példa: áttérés mikroszolgáltatásokra

Az architektúradöntéseket ADR-ben rögzítjük. Egy példa:

Cím: ADR-042-1: A rendelési és készletkezelő modulok külön szolgáltatássá alakítása

Állapot: Elfogadva

Kontextus: A monolitikus e-kereskedelmi rendszer fejlesztése lelassult: a rendelési és a készletkezelő modult különböző csapatok fejlesztik, de a közös kódbázis és a közös adatbázis miatt a változtatások ütköznek, a telepítések koordinációt igényelnek. A terhelés növekedése miatt a készletkezelő modult külön kellene skálázni, de a monolit ezt nem teszi lehetővé.

Döntés: A rendelési és a készletkezelő funkciókat két különálló szolgáltatásra bontjuk, saját adatbázissal és saját telepítési folyamattal. A szolgáltatások HTTP REST API-n kommunikálnak, és a szerződéseket OpenAPI-ban rögzítjük. A készlet-szolgáltatás a raktárkészletet eseményekkel szinkronizálja a rendelési szolgáltatással.

Alternatívák:

1. *Marad a monolit, de a modulokat belső interfészekkel szeparáljuk.* Előnye az egyszerűbb üzemeltetés, de nem oldja meg a skálázási és csapatfüggetlenségi problémát.
2. *Kiszervezés külső szolgáltatásba (SaaS).* A készletkezelés kiszervezhető, de adatvédelmi és integrációs aggályok miatt elvetettük.
3. *Teljes mikroszolgáltatásos architektúra bevezetése minden modulra.* Túl nagy lépés, a szervezet még nem áll készen a nagy operációs teherre.

Következmények:

- A rendelési és készletkezelő csapatok önállóan tudnak szállítani.
- Nő az operációs teher: két deployment pipeline, két adatbázis, két szolgáltatás monitorozása.
- A hálózati hibák és az adatkonzisztencia kezelését meg kell tervezni (például outbox minta, kompenzáló tranzakciók).
- A megfigyelhetőséget a kezdetektől be kell építeni: naplózás, metrikák, nyomkövetés.

Dátum: 2025-01-15

Ez az ADR jól mutatja a döntés kontextusát, az alternatívákat és a következményeket, beleértve az árat is.

12.1.13 Labor: mikroszolgáltatásos rendszer kockázatlistájának elkészítése

A fejezet laborja a mikroszolgáltatások bevezetésének kockázataira koncentrál. A cél nem egy működő rendszer építése, hanem a döntési helyzet tudatos elemzése.

12.1.14 Kiinduló állapot

Egy fiktív vállalat, a „Webshop Zrt.” monolitikus e-kereskedelmi rendszert üzemeltet. A rendszer moduljai: felhasználókezelés, termékkatalógus, kosár, rendeléskezelés, készletkezelés, számlázás. A csapat 20 fejlesztőből áll, akik jelenleg egyetlen kódbázison dolgoznak. A telepítés havonta egyszer, éjszaka történik, és gyakran hibákkal jár. A vezetőség a mikroszolgáltatások bevezetését fontolgatja, hogy gyorsabb kiadásokat és csapatfüggetlenséget érjen el.

12.1.15 Feladat

Készítsen kockázatlistát a mikroszolgáltatások bevezetéséhez. A dokumentum a következőket tartalmazza:

1. **Kontextus leírása** (1 bekezdés): a jelenlegi helyzet, a tervezett változás és a célok.
2. **Kockázati táblázat** legalább 8 kockázattal. Minden kockázathoz adja meg:
    - azonosító,
    - leírás,
    - valószínűség (alacsony, közepes, magas),
    - hatás (alacsony, közepes, magas),
    - a kockázat típusa (technikai, operációs, szervezeti, adat, biztonsági),
    - lehetséges mérséklő intézkedés.
3. **Döntési napló** (ADR formátumban) arról, hogy melyik modult vagy modulokat szolgáltassá alakítanák először, és miért. A naplóban térjen ki az alternatívákra és a következményekre.
4. **Ellenőrzési szempontok** listája: milyen feltételek teljesülése esetén lehet élesben továbblépni, és milyen mérőszámokat figyelnének az első 30 napban.
5. **Mért és várt eredmény elkülönítése**: a labor során nem futtat éles rendszert, ezért jelezze, hogy az eredmények szimuláción vagy becslésen alapulnak, és milyen mérési környezetet feltételez.

12.1.16 Elvárt eredmény

A dokumentum reprodukálható, verziózott és mások számára is érthető. A kockázatlista minden eleme indokolt, a döntési napló tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket. Az ellenőrzési szempontok mérhető célokat fogalmaznak meg. A mért és várt eredmények egyértelműen el vannak különítve.

12.1.17 Ellenőrzési szempontok

- A kockázatok lefedik a fejezetben tárgyalt területeket: hálózati hiba, operációs teher, megfigyelhetőség, adatkonzisztencia, biztonság, szervezeti felkészültség.
- Minden kockázathoz tartozik mérséklő intézkedés, amely illeszkedik a kontextushoz.
- Az ADR formailag megfelel a fejezetben bemutatott sémának.
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.
- A mért és várt eredmények külön szakaszban, világos címkével szerepelnek.

12.1.18 Visszaállítási út

Mivel a labor analitikus jellegű, nincs szükség éles rendszer módosítására. Ha a hallgató valamilyen PoC-t futtat, azt izolált környezetben tegye, és gondoskodjon a környezet törléséről.

12.1.19 Ellenőrzőlista és önellenőrző kérdés

Ellenőrzőlista:

☐ Meg tudom magyarázni, mi a független telepítés lényege, és milyen előnyökkel jár.
☐ Fel tudom sorolni a szervezeti skálázás és csapatautonómia előnyeit és korlátait.

- ☐ Azonosítom a mikroszolgáltatások legfontosabb költségeit: hálózati hiba, operációs teher, megfigyelhetőség.
- ☐ Tudok példát mondani parciális hibára és kaszkád meghibásodásra.
- ☐ El tudom készíteni egy egyszerű rendszer kockázatlistáját a fejezet laborjának sablonja alapján.
- ☐ Képes vagyok egy ADR-t írni a mikroszolgáltatásra áttérés döntéséről.

Önellenőrző kérdés: Milyen kritériumok alapján döntené el, hogy egy adott rendszer esetében a mikroszolgáltatások bevezetése várhatóan nagyobb előnnyel jár, mint amekkora költséget jelent? Válaszában hivatkozzon a független telepítésre, a szervezeti skálázásra, a hálózati hibákra, az operációs teherre és a megfigyelhetőségre.

Ez a fejezet a mikroszolgáltatások ígéretét és árát mutatta be. A következő egység (Szolgáltatáshatárok és domain-dekompozíció) a szolgáltatáshatárok és a domain-dekompozíció kérdéseire épít, ezért a itt bevezetett fogalmak – különösen a független telepítés, a csapatautonómia és az operációs teher – alapvető fontosságúak lesznek.

12.2 Szolgáltatáshatárok és domain-dekompozíció

A szolgáltatások helyes felosztása és a határok stabilitása a mikroszolgáltatás-architektúra legsarkalatosabb kérdése. Egy rosszul meghúzott határ nem csupán esztétikai probléma: a fejlesztési sebesség csökkenéséhez, üzemeltetési rémálmokhoz és egy olyan elosztott rendszerhez vezet, amely a monolit hátrányait az elosztottság költségeivel tetézi. Ebben az alfejezetben a szolgáltatások felosztásának alapelveit, a domain-dekompozíció gyakorlatát és a határok stabilitásának értékelését járjuk körül. A cél, hogy képes legyél üzleti okokból, nem csupán technikai divatból meghúzni a határokat, és felismerd, amikor egy határ nem a megfelelő helyen van.

A fejezet nem ismétli meg a mikroszolgáltatások általános ígéretét és árát (lásd az előző alfejezetet), és nem tér ki az elosztott rendszerek elméleti alapjaira (CAP, konzisztencia, stb. – ezek a következő alfejezet témái). Feltételezzük, hogy már tudod, miért választanak a szervezetek mikroszolgáltatásokat, és most arra fókuszálunk, hogyan válasszuk meg a szolgáltatások határait, amikor a döntés már megszületett.

12.2.1 Üzleti képesség és adattulajdon

A szolgáltatások felosztásának első számú vezérelve az üzleti képességek mentén való szervezés. Egy üzleti képesség az, amit a vállalat tesz annak érdekében, hogy értéket állítson elő, függetlenül attól, hogy éppen milyen szervezeti egység vagy technológia valósítja meg. Például egy e-kereskedelmi vállalatnál ilyen képesség a termékkatalógus kezelése, a megrendelések feldolgozása, a fizetések kezelése, a készletnyilvántartás vagy a szállításszervezés. A technikai rétegződés (például „frontend”, „backend”, „adatbázis”) mentén történő felosztás szinte mindig rossz határokat eredményez, mert egy üzleti folyamat végrehajtása több rétegen átível, és a változások általában üzleti képességenként, nem rétegenként jelentkeznek.

Ahol a technikai rétegződés felosztja a rendszert, ott az üzleti képesség alapú felosztás egyesíti a képesség megvalósításához szükséges összes technológiát egyetlen, jól körülhatárolt szolgáltatásban. A „katalógus szolgáltatás” nem csupán egy REST-végpontot jelent, hanem magában foglalja az adatmodellt, az üzleti logikát, a perzisztenciát és a külső interfészt is. Ez az elv a híres „you build it, you run it” filozófiának is alapja.

Az üzleti képesség azonosításához a domain-dekompozíció eszköztárát használjuk, amelyet a következő pontban részletezünk. Itt azonban már most ki kell emelnünk egy döntő fogalmat: az **adattulajdont**. Minden szolgáltatásnak egyértelműen birtokolnia kell a saját adatait. Ez azt jelenti, hogy a szolgáltatás felel az adatainak életciklusáért, konzisztenciájáért és eléréséért; más szolgáltatások kizárólag a szolgáltatás által nyilvánosan definiált interfészen (például REST API, aszinkron üzenetek) keresztül férhetnek hozzá. A közös adatbázis, amelyet több szolgáltatás közvetlenül olvas vagy ír, a leggyakoribb oka a rejtett csatolásnak és a határok feloldódásának, ezért az adattulajdon az egyik legerősebb határjelző.

Gondoljunk egy megrendelés-szolgáltatás és egy számlázási szolgáltatás közötti kapcsolatra. Ha mindkettő ugyanazt a `orders` táblát módosítja, akkor egyik sem tudja szabályozni a saját konzisztencia- és indexelési követelményeit, és bármelyik módosítás a másik szolgáltatás hibáját okozhatja. Az adattulajdon modell szerint a megrendelés-szolgáltatás birtokolja a `orders` táblát, a számlázási szolgáltatás pedig egy saját `invoices` táblát használ, és a megrendelés adatait – a szükséges részhalmazt – aszinkron eseményekkel vagy API-hívással kapja meg. Így mindkét szolgáltatás autonóm maradhat, és az adatséma-változások is lokalizálhatók.

Az adattulajdon nem jelenti azt, hogy az adatok nem duplikálódhatnak. Egy mikroszolgáltatás-architektúrában a redundancia elfogadott, sőt gyakran szükséges a teljesítmény és a rendelkezésre állás érdekében. Azonban minden adatnak pontosan egy birtokosa van, aki az adat forrásaként szolgál; a többi szolgáltatás csak olvasási másolatot (read model) tart fenn, amelyet aszinkron módon frissít. Ennek a mintának a részleteit az eseményvezérelt architektúrák fejezet tárgyalja, de a határok szempontjából itt elég annyit rögzíteni, hogy a birtoklás egyértelműsége a stabilitás alapfeltétele.

12.2.2 Domain-dekompozíció: bounded context és szolgáltatástervezés

A mikroszolgáltatások határainak meghúzásához a Domain-Driven Design (DDD) nyújtja a legmegbízhatóbb fogalmi keretet – különösen a **bounded context** és a **ubiquitous language** fogalmát. Ebben az alfejezetben csak annyit használunk a DDD-ből, amennyi a szolgáltatások lehatárolásához szükséges; a DDD részletes bemutatása egy későbbi fejezetben következik.

A **domain** az a problématér, amelyben a szoftver működik – például az online könyvkereskedelem. A domain különböző részein a fogalmak más-más jelentést hordozhatnak. A „rendelés” a megrendelés-feldolgozásban egy vásárlói szándékot jelent, a raktárkezelésben egy komissiózási feladatot, a számvitelben pedig egy könyvelési tételt. Ha ezeket a jelentéseket egyetlen modellbe kényszerítenénk, akkor a modell tele lenne kompromisszumokkal, és a változtatások folyamatosan ütköznének. A **bounded context** éppen ezt a határt jelöli ki: egy olyan határ, amelyen belül egy adott modell érvényes, és a fogalmak jelentése egyértelmű. A **ubiquitous language** pedig az adott kontextuson belül használt, a domain-szakértőkkel közösen kialakított, félreérthetetlen nyelvezet.

A szolgáltatások tervezése során a cél az, hogy minden szolgáltatás egy-egy bounded contextez igazodjon. Ez nem azt jelenti, hogy minden bounded context pontosan egy szolgáltatás legyen – egy kontextuson belül lehet több szolgáltatás is –, de a szolgáltatások határainak nem szabad átmetszeniük a bounded context határait. Ha egy szolgáltatás több bounded contextet fog át, akkor a szolgáltatáson belül ugyanaz a fogalom több jelentéssel bír, és a modell inkonzisztenssé válik. Ha pedig egy bounded context több szolgáltatásra oszlik, akkor a szolgáltatások közötti csatolás általában túl szorossá válik, mert a fogalmak szétválaszthatatlanul összefonódnak.

A domain-dekompozíció gyakorlati lépései:

1. **Domain-térkép készítése.** A domain-szakértőkkel közösen azonosítjuk a fő üzleti folyamatokat, szereplőket és fogalmakat. Ennek hatékony eszköze az event storming, ahol a résztvevők a domain eseményeit, parancsait, aggregátumait és külső rendszereit gyűjtik össze. Az event storming során természetesen kirajzolódnak a különböző kontextusok, mert a résztvevők eltérő nyelvezetet használnak ugyanarra a fogalomra.

2. **Bounded contextek azonosítása.** A domain-térkép alapján meghúzzuk azokat a határokat, ahol a fogalmak jelentése megváltozik. Például az e-kereskedelemben a „vásárló" a katalógusban böngészéskor egy anonim látogató, a megrendelés leadásakor egy regisztrált felhasználó, a számlázásban pedig egy fizető fél. Így a Katalógus, Megrendelés, Fizetés és Szállítás kontextusok különválnak.

3. **Adattulajdon hozzárendelése.** Minden bounded contexthez hozzárendeljük az általa birtokolt adatokat. Ez az adatmodell lehet egy relációs séma, dokumentumgyűjtemény, eseménynapló vagy bármilyen perzisztencia, de a birtoklási viszony egyértelmű. A többi kontextus csak a nyilvános interfészen keresztül férhet hozzá.

4. **Kommunikációs minták meghatározása.** A kontextusok közötti kapcsolatok lehetnek szinkronok (például REST vagy gRPC) vagy aszinkronok (üzenetküldés, események). A szinkron hívások számát minimalizálni kell azokon a helyeken, ahol a rendelkezésre állás és a teljesítmény kritikus, míg az aszinkron kommunikáció jobb lazítási lehetőséget biztosít. A kontextusok közötti kapcsolatokat context mapen ábrázoljuk, ahol jelöljük a kapcsolat típusát (például partneri, megosztott kernel, ügyfél-szállító, antikorrupciós réteg).

5. **Szolgáltatások kialakítása.** Végül minden bounded contexthez hozzárendelünk egy vagy több szolgáltatást. Általában egy bounded context egy szolgáltatásnak felel meg, de nagyobb kontextusok esetén moduláris monolitot is választhatunk, amelyet később bonthatunk tovább. A szolgáltatások mérete másodlagos; a hangsúly a felelősségi körök egyértelműségén van.

Példaként vegyünk egy egyszerű e-kereskedelmi rendszert. A C4 context szintű diagramja a következő lehet:

```
flowchart LR
    V[Vásárló] -->|böngészés, vásárlás| R[E-kereskedelmi rendszer]
    R -->|fizetési tranzakció| FP[Fizetési szolgáltató]
    R -->|szállítási megbízás| FS[Futárszolgálat]
    R -->|készlet szinkron| KR[Külső raktárkezelő]
```

A diagram címe: E-kereskedelmi rendszer C4 context szintű nézete. Jelmagyarázat: a téglalapok rendszerek, a nyilak az interakció irányát és a protokollt jelölik (mindegyik HTTPS, a futárszolgálat és a raktárkezelő esetében aszinkron webhookokkal). Feltételjegyzet: a rendszerhatárt a belső rendszer határa jelöli; a külső rendszerek a rendszerhatáron kívül vannak. A diagram csak a legfontosabb szereplőket és rendszereket tünteti fel.

Ezen belül a belső container-szintű felosztás a domain-dekompozíció eredménye:

```
flowchart LR
    subgraph Eker[Rendszerhatár: E-kereskedelmi rendszer]
        KAT[Katalógus szolgáltatás]
        MEG[Megrendelés szolgáltatás]
        FIZ[Fizetés szolgáltatás]
        SZAL[Szállítás szolgáltatás]
        KESZ[Készlet szolgáltatás]
    end
    V[Vásárló] -->|HTTPS| KAT
    V -->|HTTPS| MEG
    MEG -->|REST szinkron| KAT
```

```
MEG -->|aszinkron üzenet| FIZ
MEG -->|aszinkron üzenet| SZAL
SZAL -->|aszinkron üzenet| KESZ
FIZ -->|REST szinkron| FP[Fizetési szolgáltató]
SZAL -->|REST szinkron| FS[Futárszolgálat]
KESZ -->|REST szinkron| KR[Raktárkezelő]
```

A diagram címe: E-kereskedelmi rendszer container szintű nézete. Jelmagyarázat: a vastag vonallal határolt terület a rendszerhatár; a belső szolgáltatások téglalapok; a nyilak a kommunikáció irányát és típusát mutatják (folytonos nyíl szinkron hívás, szaggatott nyíl aszinkron üzenet). Feltételjegyzet: a diagram nem mutatja az egyes szolgáltatások adatbázisait, de minden szolgáltatás a saját adatbázisát birtokolja; a kapcsolatokon a protokoll az adott nyíl mellett feltüntetve.

Figyeljük meg, hogy a Katalógus szolgáltatás csak a termékadatokkal foglalkozik, a Megrendelés szolgáltatás a rendelések életciklusával, a Fizetés szolgáltatás a fizetési tranzakciókkal, a Szállítás szolgáltatás a szállítási feladatokkal, a Készlet szolgáltatás pedig a raktárkészlettel. Mindegyik saját adatbázissal rendelkezik, és a köztük lévő kommunikáció minimalizált: a Megrendelés szolgáltatás például aszinkron üzenetet küld a Fizetésnek, amikor fizetésre van szükség, nem pedig szinkron hívással blokkolja a megrendelés leadását.

12.2.3 A határok stabilitása

A jó szolgáltatáshatár nem csupán a jelenlegi követelményeket tükrözi, hanem stabil marad a változások során. Egy határ akkor stabil, ha a domain egy adott részének módosítása nem terjed át más szolgáltatásokra. A stabilitás mérésére nincs egyetlen számszerű mutató, de több heurisztika segít:

- **A fogalmak egyértelműsége.** Ha egy fogalom (például „rendelés”) két szolgáltatásban is megjelenik, de eltérő jelentéssel, akkor a határ általában stabil, mert a két értelmezés függetlenül fejlődhet. Ha viszont ugyanaz a fogalom ugyanazzal a jelentéssel két szolgáltatásban is létfontosságú, akkor a határ valószínűleg rossz helyen van, mert a változások szinkronizálást igényelnek.
- **A tranzakciós határok egybeesése.** Egy szolgáltatáson belül az üzleti műveletek általában ACID tranzakciókban hajthatók végre. Ha egy üzleti művelet rendszeresen több szolgáltatást érint, és elosztott tranzakcióra vagy kompenzációs logikára van szükség, az annak a jele, hogy a határ túl szűk. A mikroszolgáltatásoknál az elosztott tranzakciókat kerülni kell, ezért a határokat úgy kell meghúzni, hogy az üzleti invariánsok egy szolgáltatáson belül maradjanak.
- **A csatolás mértéke.** A szolgáltatások közötti hívások száma, a közös adatmodellek és a körkörös függőségek mind a csatolást növelik. Ha két szolgáltatás között nagyon gyakori a szinkron kommunikáció, vagy ha az egyik szolgáltatás sémaváltozása a másik azonnali módosítását igényli, akkor a határ instabil. Ilyenkor érdemes megfontolni a szolgáltatások összevonását vagy aszinkron kommunikációra váltást.
- **A változások gyakorisága.** Ha egy adott domain-részt gyakran módosítanak, és ezek a módosítások rendszeresen több szolgáltatást érintenek, az a határ újragondolását jelzi. A stabilitás célja, hogy a gyakori változások lokalizálhatók legyenek.

A határok stabilitását nem csupán a tervezés pillanatában kell értékelni, hanem folyamatosan, a fejlesztés során nyert visszajelzések alapján. A domain-dekompozíció iteratív folyamat: az első felosztás hipotézis, amelyet a valós implementáció és a változások igazolnak vagy cáfolnak. A

fitness functions (lásd a minőségi forgatókönyvek fejezetben) segíthetnek a határok egészségének automatikus ellenőrzésében, például azáltal, hogy figyelik a körkörös függőségeket vagy a megosztott adatbázis-műveleteket.

A stabilitás fontos jele az is, hogy a szolgáltatások közötti kapcsolatok **aszimmetrikus**-e vagy sem. Ha két szolgáltatás folyamatosan egymással kommunikál, és mindkettő a másik API-jának változásaitól függ, akkor valójában egy elosztott komponenspárról van szó, amelyet érdemes lehet egy szolgáltatásba vonni. A stabil határok általában aszimmetrikus függőségeket eredményeznek: például a Megrendelés szolgáltatás függ a Katalógus szolgáltatástól, hogy lekérdezze a termék adatait, de a Katalógus nem tud a Megrendelés létezéséről.

12.2.4 Rossz határ és distributed monolith

A leggyakoribb hiba, amely a domain-dekompozíció során elkövethető, a **rossz határ** meghúzása. Ez megnyilvánulhat két végletben: a szolgáltatások túl kicsik (nanoservice) vagy túl nagyok (god service). A nanoservice esetén egyetlen üzleti művelet végrehajtása több szolgáltatás összehangolt hívását igényli, ami hálózati késleltetést, hibapontokat és bonyolult kompenzációs logikát eredményez. A god service esetén a szolgáltatás túl sok felelősséget halmoz fel, és gyakorlatilag egy monolitot alkot, amelyet csak névlegesen nevezünk szolgáltatásnak.

A rossz határ legsúlyosabb következménye a **distributed monolith**: egy olyan elosztott rendszer, amelyben a szolgáltatások között olyan szoros a csatolás, hogy a rendszer a monolit összes hátrányával rendelkezik, miközben az elosztott rendszerek összes nehézségét is viselnie kell. A distributed monolith jellegzetes tünetei:

- **Közös adatbázis.** Több szolgáltatás közvetlenül ugyanazt a sémát olvassa és írja. Ez a leggyorsabb út a rejtett csatoláshoz.
- **Sok szinkron hívás.** A szolgáltatások láncban hívják egymást (például A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D), így egyetlen felhasználói kérés hosszú, törékeny hívási láncot vált ki. A hiba terjedése, a latencia és a rendelkezésre állás romlása garantált.
- **Körkörös függőségek.** A szolgáltatások egymásra hivatkoznak, így a telepítési sorrend és az API-verziózás rémálommá válik.
- **Globális konfiguráció és közös könyvtárak.** Ha minden szolgáltatás ugyanazt a core könyvtárat használja, és annak változása minden szolgáltatás újratelepítését igényli, akkor a függetlenség illúzió.
- **Minden változás mindenkit érint.** Egy apró üzleti módosítás több szolgáltatás egyidejű módosítását, koordinált telepítését és tesztelését kívánja.

A distributed monolith elkerülésének első lépése a határok alapos, üzleti alapú meghúzása, ahogyan azt az előző pontokban tárgyaltuk. Emellett fontos a csatolás folyamatos figyelése: ha egy változás több szolgáltatást érint, akkor a határt újra kell gondolni. A jó hír az, hogy a rossz határ nem végzetes: a szolgáltatásokat össze lehet vonni, a határokat át lehet húzni, és a moduláris monolitból később is bonthatunk ki szolgáltatásokat, ha a szervezet és a csapatok már érettek. A rossz határ azonban időben történő felismerés nélkül hosszú ideig mérgezi a fejlesztési sebességet.

12.2.5 Gyakorlati minta: e-kereskedelmi domain felosztása

Vegyünk egy konkrét, egyszerű e-kereskedelmi domain-t, és nézzük végig a felosztás lépéseit. A domain a következő szereplőket és folyamatokat foglalja magában:

- A vásárló böngészi a termékeket, keres, szűr.

- A vásárló kosárba helyezi a termékeket.
- A vásárló leadja a megrendelést, megadja a szállítási címet és a fizetési módot.
- A rendszer kezdeményezi a fizetést a külső fizetési szolgáltatónál.
- A rendszer szállítási megbízást ad a futárszolgálatnak.
- A raktárkezelő rendszer frissíti a készletet.

Az event storming során a következő domain eseményeket és parancsokat azonosítjuk:

- `TermékMegtekintve`, `KeresésVégrehajtva` (Katalógus)
- `KosárbaHelyezve`, `MegrendelésLeadva` (Megrendelés)
- `FizetésKezdeményezve`, `FizetésElfogadva`, `FizetésElutasítva` (Fizetés)
- `SzállításiMegbízásLétrehozva`, `SzállításKiszállítva` (Szállítás)
- `KészletCsökkentve`, `KészletFeltöltve` (Készlet)

A domain-szakértőkkel való beszélgetés során kiderül, hogy a „rendelés” fogalma eltérő a Megrendelés kontextusban (a vásárlói szándék és a termékek listája) és a Szállítás kontextusban (a fizikai csomagok és a kézbesítési cím). A „fizetés” a Fizetés kontextusban egy pénzügyi tranzakció, míg a Megrendelés kontextusban egy állapotjelző („fizetve” vagy „függőben”). Ezek a különbségek természetes bounded context határokat rajzolnak ki.

Az adattulajdon hozzárendelése a következő:

Szolgáltatás	Felelős adatok	Megjegyzés
Katalógus	termékadatok, kategóriák, árak	Az árváltozás csak itt történik.
Megrendelés	rendelések, rendelési tételek, vásárlói adatok (szállítási cím)	A megrendelés állapota itt él.
Fizetés	fizetési tranzakciók, fizetési állapotok	A külső szolgáltatóval való kapcsolat.
Szállítás	szállítási megbízások, csomagok, kézbesítési cím	A futárszolgálat felé.
Készlet	készletmennyiségek, raktárhelyek	A raktárkezelővel szinkronizált.

A szolgáltatások közötti kommunikáció:

- A Megrendelés szolgáltatás a Katalógustól szinkron kéréssel kéri le a termék aktuális árát a megrendelés leadásakor. Ez a hívás rövid, és a rendelkezésre állás szempontjából kevésbé kritikus, mert a megrendelés leadása amúgy is interaktív.
- A Megrendelés leadásakor a Megrendelés szolgáltatás egy `MegrendelésLeadva` eseményt publikál. Ezt a Fizetés és a Szállítás szolgáltatások aszinkron módon fogyasztják. A Fizetés szolgáltatás kezdeményezi a tranzakciót, a Szállítás szolgáltatás pedig létrehozza a szállítási megbízást. Amikor a Fizetés elfogadja a tranzakciót, egy `FizetésElfogadva` eseményt küld, amelyre a Megrendelés szolgáltatás frissíti a rendelés állapotát.
- A Készlet szolgáltatás a Szállítás szolgáltatástól kapja a `SzállításiMegbízásLétrehozva` eseményt, ami alapján csökkenti a készletet.

Ez a kialakítás minimalizálja a szinkron függőségeket, és minden szolgáltatás autonóm adatbázist használ.

Az egyik döntést érdemes ADR-ben rögzíteni:

ADR: A Megrendelés és a Fizetés közötti kommunikáció aszinkron üzenetküldéssel

- **Kontextus:** A megrendelés leadásakor a rendszernek fizetést kell kezdeményeznie. A szinkron hívás növelné a megrendelés leadásának válaszidejét, és meghibásodás esetén a teljes folyamat kudarcát okozná.
- **Döntés:** A Megrendelés szolgáltatás a `MegrendelésLeadva` eseményt publikálja egy üzenetközvetítőn keresztül; a Fizetés szolgáltatás ezt fogyasztja, és saját felelősségi körében kezdeményezi a fizetést. Az eredményt a `FizetésElfogadva` vagy `FizetésElutasítva` eseményekkel jelzi vissza.
- **Alternatívák:** Szinkron REST-hívás a Fizetés szolgáltatásra; tranzakció-koordinátor alkalmazása (elvetve, mert elosztott tranzakciót igényelne).
- **Következmények:** A megrendelés leadása gyorsabb és ellenállóbb a fizetési szolgáltatás átmeneti kiesésével szemben; ugyanakkor a rendszer állapota átmenetileg inkonzisztens lehet (a rendelés létezik, de a fizetés még nem történt meg), ami kompenzációs logikát igényel. Az adatok végül konzisztenssé válnak.
- **Állapot:** Elfogadva.

A minőségi követelményt is megfogalmazzuk quality attribute scenario formában:

- **Inger:** A vásárló leadja a megrendelést.
- **Környezet:** Normál üzemmód, 100 párhuzamos felhasználó, 5 perces mérési ablak.
- **Válasz:** A rendszer legkésőbb 500 ms alatt visszaigazolja a megrendelés elfogadását.
- **Mérőszám:** p95 válaszidő 500 ms, a mérési ablakban legalább 99,9% megfelelés.

Ez a forgatókönyv a laborban is használható a tervezett határok értékelésére.

12.2.6 Labor: bounded context alapú szolgáltatásfelosztás tervezése

A labor célja, hogy egy megadott üzleti területre te magad végezd el a domain-dekompozíciót, és dokumentáld a döntéseidet. A labor nem igényel kódot, de a reprodukálhatóság érdekében egy verziókezelt Markdown dokumentumban (vagy jegyzetfüzetben) kell elkészítened a feladatot.

Kiinduló állapot: Adott az alábbi üzleti leírás (helyőrző adatokkal):

> Egy online könyvtár (fiktív „Acme Books”) a következő folyamatokat támogatja: a látogatók böngészhetnek a könyvek között, kereshetnek szerzőre és címre; a regisztrált vásárlók kosárba tehetnek könyveket, módosíthatják a kosarukat, és leadhatják a megrendelést; a megrendeléshez meg kell adni a szállítási címet és a fizetési módot; a rendszer fizetést kezdeményez a külső fizetési szolgáltatónál; a rendszer szállítási megbízást ad át a futárszolgálatnak; a könyvtár a készletét egy külső raktárkezelő rendszerrel tartja szinkronban. A vásárlók e-mailben értesítést kapnak a megrendelés állapotváltozásairól. Az ügyfélszolgálati munkatársak megtekinthetik a vásárlók rendeléseit, és manuálisan módosíthatják a szállítási címet, ha a rendelés még nem került kiszállításra.

A kiinduló állapot tartalmazza a szükséges domain-leírást, de nincs előre meghatározott szolgáltatásfelosztás. A laborhoz tartozó eszközök: Markdown-szerkesztő, Mermaid-diagramokhoz szükséges környezet (például VS Code vagy webes Mermaid élő szerkesztő), Git tároló a verziózáshoz.

Feladat: A domain-dekompozíciós lépések alapján készítsd el a következőket:

1. **C4 context diagram** Mermaid `flowchart` használatával, amely feltünteti a rendszerhatárt, a felhasználói szerepköröket (Látogató, Regisztrált vásárló, Ügyfélszolgálati munkatárs) és a külső rendszereket (Fizetési szolgáltató, Futárszolgálat, Raktárkezelő, E-mail szolgáltató). A diagram címe és jelmagyarázata is szerepeljen.

2. **Bounded context lista:** Azonosíts legalább öt bounded contextet (például Katalógus, Kosár/Megrendelés, Fizetés, Szállítás, Ügyfélszolgálat, Értesítés), és mindegyikhez írd le röviden (2-3 mondat), hogy mi a felelőssége, milyen fő fogalmakat használ, és milyen adatokat birtokol.
3. **Adattulajdon-táblázat:** Készíts egy táblázatot, amely megmutatja, melyik szolgáltatás melyik adatot birtokolja. Ügyelj arra, hogy minden adatnak pontosan egy birtokosa legyen.
4. **Kommunikációs terv:** Határozd meg, hogy a szolgáltatások hogyan kommunikálnak egymással. Jelöld, hogy melyik kapcsolat szinkron, és melyik aszinkron. Indokold meg, miért választottad az adott módot.
5. **Két ADR:** Készíts legalább két Architecture Decision Recordot a legfontosabb döntésekről (például a Megrendelés és a Fizetés közötti kommunikáció, vagy az Ügyfélszolgálat és a Megrendelés közötti kapcsolat). Mindegyik ADR kövesse a könyv konvencióját: kontextus, döntés, alternatívák, következmények, állapot.
6. **Minőségi forgatókönyv:** Fogalmazz meg egy quality attribute scenario-t a megrendelés leadásának válaszidejére a fenti példa alapján.

Elvárt eredmény: Egy Markdown dokumentum (például `sa043-labor.md`), amely tartalmazza a fentieket, és egy Git commit, amely a dokumentumot verziózza. A dokumentum címe, a diagramok címei, a jelmagyarázatok és a feltételjegyzetek hiánytalanok. Az ADR-ek visszakereshetők, és a döntések okát és a feladott alternatívákat is tartalmazzák.

Ellenőrzési szempontok:

- A C4 context diagramon a rendszerhatár egyértelműen jelölve van, a külső rendszerek a határon kívül szerepelnek.
- A bounded contextek üzleti fogalmakhoz igazodnak, nem technikai rétegekhez.
- Minden adatnak pontosan egy birtokosa van, és nincs közös adatbázis.
- A szolgáltatások közötti kommunikációban a szinkron hívások száma minimális; az aszinkron kapcsolatok indokoltak.
- Az ADR-ek a megadott sablont követik, és a döntések okai világosak.
- A minőségi forgatókönyv tartalmazza az ingert, a környezetet, a választ és a mérőszámot.
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

Visszaállítási út: A labor nem tartalmaz destruktív műveletet. A dokumentum Git-tárolóban verziókezelt; bármilyen változtatás a `git checkout` segítségével visszavonható. Ha valaki a labor során rossz döntést rögzített, a dokumentum törölhető és újra létrehozható a kiinduló állapotból.

12.2.7 Ellenőrzőlista

- ☐ A szolgáltatások határai üzleti képességek mentén húzódnak, nem technikai rétegek mentén.
- ☐ Minden szolgáltatásnak egyértelműen meghatározott adattulajdona van.
- ☐ Nincs közös adatbázis több szolgáltatás között.
- ☐ A bounded contextek azonosítása megtörtént, és a fogalmak jelentése kontextusonként egyértelmű.
- ☐ A szolgáltatások közötti kommunikációban a szinkron hívások minimálisak, az aszinkron kapcsolatok indokoltak.
- ☐ A tranzakciós határok egybeesnek a szolgáltatás határával.
- ☐ A változások várhatóan lokalizálhatók, nincs körkörös függőség.
- ☐ A döntések ADR-ben dokumentálva vannak.

☐ A C4 context diagram a rendszerhatárt, a szereplőket és a külső rendszereket mutatja.

12.2.8 Önellenőrző kérdés

Melyik a legjellemzőbb tünete annak, hogy egy szolgáltatáshatár rossz helyen van, és a rendszer distributed monolithá válik? Fogalmazd meg, miért a közös adatbázis vagy a sok szinkron hívás a legárulkodóbb jel, és hogyan lehet ezt korrigálni anélkül, hogy a teljes rendszert újra kellene írni.

12.2.9 Összefoglalás

A szolgáltatáshatárok helyes meghúzása a mikroszolgáltatás-architektúra alapja. A határokat az üzleti képességek és a domain-dekompozíció (bounded contextek) mentén kell kijelölni, minden szolgáltatáshoz egyértelmű adattulajdont rendelve. A stabilitás kulcsa, hogy a változások lokalizálhatók maradjanak, a tranzakciós határok egybeessenek a szolgáltatás határával, és a szolgáltatások közötti kommunikáció laza legyen. A rossz határ distributed monolothoz vezet, amely a monolit és az elosztott rendszerek hátrányait egyesíti. A következő alfejezetben az elosztott rendszerek alapvető elméleti korlátaira (CAP, konzisztencia, hiba) fordítjuk a figyelmet, amelyek tovább árnyalják a határok meghúzásának szempontjait.

12.3 Elosztott rendszerek alapjai: CAP, konzisztencia és hiba

A szolgáltatáshatárok felvázolása után az elosztott rendszerek tervezésekor azonnal megjelenik egy minőségileg új kockázat: a hálózat nem megbízható, a csomópontok egymástól függetlenül hibásodhatnak meg, és a rendszer egyes részei eltérő sebességgel, eltérő állapotban dolgoznak. Ez az egység azt a kérdést járja körbe, hogyan tervezzünk úgy, hogy a rendszer viselkedése ilyen körülmények között is előre jelezhető legyen. A cél nem a hálózati hibák megszüntetése – azok nem szüntethetők meg teljesen –, hanem az, hogy a tervező explicit döntéseket hozzon a konzisztencia, a rendelkezésre állás és a késleltetés közötti kompromisszumokról.

A fogalmakat egy hétköznapi példán keresztül vezetjük be: két szolgáltatás, a **Foglalás-szolgáltatás** és a **Készlet-szolgáltatás** együttműködik. A foglalási kérés megérkezik, a foglalás szolgáltatás megkérdezi a készletet, majd lefoglal egy terméket. A készlet-adat több csomóponton replikált, hogy egy csomópont kiesése ne tegye elérhetetlenné a rendszert. A kérdés: mit tehet a rendszer, ha a készlet-replikák közül az egyik nem válaszol?

12.3.1 Elosztott rendszer, hálózati hiba és részleges hiba

Elosztott rendszernek nevezzük az olyan rendszert, amelynek komponensei különböző folyamatokban, gyakran különböző gépeken futnak, és üzenetküldéssel kommunikálnak. Az üzenet lehet szinkron HTTP-hívás, aszinkron üzenetsorba írt rekord, gRPC-hívás vagy bármilyen protokoll. A közös pont: a hívó és a hívott fél között mindig van hálózati átvitel, és ezért mindig van egy nem nulla, nem determinisztikus késleltetés, továbbá fennáll a hiba lehetősége.

A hálózati hiba többféleképpen jelentkezhet:

- **kérés elveszik**: a csomag soha nem érkezik meg;
- **válasz elveszik**: a kérés megérkezett, de a válasz nem;
- **késleltetés**: a csomag megérkezik, de a megszokottnál később;

- **sorrendcsere**: a csomagok nem abban a sorrendben érkeznek, ahogyan elküldték őket;
- **duplikáció**: ugyanaz a csomag többször is megérkezik.

A gyakorlatban a hívó oldalán a hiba általában **időtúllépésként** jelenik meg. A timeout azonban nem mondja meg, hogy a hívott fél valójában milyen állapotban van. Lehet, hogy a kérés el sem jutott hozzá; lehet, hogy feldolgozta, de a válasz nem ért vissza; lehet, hogy a feldolgozás éppen most tart. Ezt a jelenséget nevezik **bizonytalan kimenetelnek**. Ha a hívó ilyenkor egyszerűen újrapróbálja a hívást, akkor ugyanaz a művelet kétszer is végrehajtódhat. Ezért az elosztott rendszerekben a retry csak akkor biztonságos, ha a művelet **idempotens**, vagy ha a hívó egy egyedi idempotencia-azonosítót küld, és a szerver ezzel megakadályozza a többszörös végrehajtást.

Példa: a Foglalás-szolgáltatás `POST /foglalasok` kérést küld. A kliens egy `Idempotency-Key: 8f3a...` fejlécet is elhelyez. Ha a válasz elveszik, a kliens ugyanazzal a kulccsal megismétli a kérést. A szerver már ismeri a kulcsot, ezért nem hoz létre új foglalást, hanem az első feldolgozás eredményét adja vissza, vagy ha az első feldolgozás még tart, akkor „függőben” állapotot jelez.

A **részleges hiba** az elosztott rendszerek alapvető tulajdonsága: a rendszer egyik része hibás lehet, miközben a többi része továbbra is működik. Ez éles ellentétben áll a monolitikus feltételezéssel, ahol egy folyamaton belül a hiba általában az egész alkalmazást leállítja, vagy legalábbis az állapot konzisztens módon, egy helyen vizsgálható. Elosztott rendszerben nincs egyetlen, globálisan érvényes „igaz állapot”, amelyet egyetlen tranzakció keretében lehetne beolvasni. Minden csomópontnak saját, esetleg elavult képe van a többiekről.

A **hálózati partíció** a részleges hiba egyik legsúlyosabb formája. Partícióról akkor beszélünk, ha a hálózat egy része nem tud kommunikálni a hálózat másik részével, miközben mindkét oldal életben van. A partíció lehet teljes, amikor semmilyen üzenet nem megy át, vagy részleges, amikor egyes csomópontok közötti kapcsolat lassú, csomagvesztéssel terhelt, vagy időnként megszakad. A partíció időtartama lehet ezredmásodperc, de lehet perc vagy óra is. A tervezőnek abból kell kiindulnia, hogy a partíció nem kivételes esemény, hanem normál üzemzavar, amelyet a rendszernek kezelnie kell.

12.3.2 A CAP-tétel: partíció alatt választani kell

A CAP-tétel az elosztott rendszerek egyik legismertebb, de leggyakrabban félreértett állítása. Eric Brewer 2000-ben fogalmazta meg sejtésként, majd Seth Gilbert és Nancy Lynch bizonyította. A tétel három tulajdonságról szól:

- **C – Consistency (konzisztencia):** itt szigorú, lineáris konzisztenciát értünk. Minden olvasási művelet a legutóbbi befejezett írás értékét adja vissza, vagy egy annál újabb értéket, és a műveletek úgy látszanak, mintha egyetlen időpillanatban, egyetlen közös sorrendben hajtódtak volna végre. A gyakorlatban ez azt jelenti, hogy ha az egyik replikára írtunk, akkor a következő olvasás bármelyik replikáról ugyanazt az értéket látja.
- **A – Availability (rendelkezésre állás):** minden beérkező kérésre észszerű időn belül nem hibás választ ad a rendszer, még akkor is, ha egyes csomópontok vagy hálózati kapcsolatok nem működnek. Itt a definíció szigorúan a nem hibás válaszra vonatkozik, nem feltétlenül a legfrissebb értékre.
- **P – Partition tolerance (partíciótűrés):** a rendszer tovább működik akkor is, ha a hálózat tetszőleges üzeneteket elveszít, késleltet vagy átrendez, tehát ha a csomópontok közötti kommunikációben partíció következik be.

A tétel lényege: **hálózati partíció fennállása alatt egy elosztott rendszer nem tudja egyszerre garantálni a szigorú konzisztenciát és a rendelkezésre állást.** Ha a partíció két

oldalra választja szét a replikákat, és mindkét oldal fogad írásokat, akkor az adatok eltérhetnek egymástól. Ilyenkor vagy le kell állítani az egyik oldalt, hogy az adatok konzisztensek maradjanak, vagy tovább kell engedni a kéréseket, vállalva a konzisztencia sérülését.

Fontos, hogy a CAP-tétel nem azt mondja, hogy háromból kettőt választhatunk. A partíciótűrést nem lehet választani: a hálózati hibák előfordulnak, és ha a rendszer elosztott, akkor ezt el kell fogadni. A valódi döntés partíció esetén a konzisztencia és a rendelkezésre állás között van. Normál, partíciómentes működés során mind a konzisztencia, mind a rendelkezésre állás teljesíthető, de a tervezésnek azt is meg kell határoznia, mi történik a partíció alatt.

A CP és AP jelölések ennek megfelelően nem valamilyen „örök” rendszerjellemzőt jelölnek, hanem a partíció alatti viselkedést:

- **CP viselkedés (partíció alatt konzisztencia):** a rendszer egyes kéréseket elutasít, vagy a művelet blokkol, mert nem tudja garantálni, hogy minden replika ugyanazt az állapotot látja. A rendszer elérhető részei nem adnak ki elavult adatot.
- **AP viselkedés (partíció alatt rendelkezésre állás):** a rendszer továbbra is kiszolgálja a kéréseket mindkét partíciós oldalon, de az egyes replikák átmenetileg eltérő értékeket adhatnak vissza. A konfliktusokat később, a partíció feloldása után kell rendezni.

Például a Készlet-szolgáltatás esetében CP választás lehet, ha egy termékből csak egyetlen darab van, és nem szeretnénk, hogy két különböző ügyfél egyszerre ugyanazt az utolsó darabot lefoglalja. Ilyenkor partíció alatt a rendszer megtagadhatja a foglalást, vagy várakoztathatja a hívót, amíg a kvórum nem elérhető. AP választás lehet, ha egy termékkatalógus árát vagy leírását olvassuk: a rövid ideig tartó eltérés a replikák között elfogadható, mert a felhasználói élmény szempontjából fontosabb, hogy a katalógus mindig gyorsan betöltsön.

A CAP-tételt azonban nem szabad úgy kezelni, mint egyetlen globális kapcsolót a rendszer egészére. Egy rendszerben különböző műveletek különböző döntést igényelhetnek. Ugyanaz a Készlet-szolgáltatás lehet CP a foglalási műveletre, de AP a termékleírás olvasására. A tervező feladata az, hogy a műveletek szintjén fogalmazza meg a kompromisszumot, és ezt rögzítse architektúra-döntési naplóban (ADR).

12.3.3 PACELC: a partíciómentes állapot kompromisszuma

A CAP-tétel csak a partíció alatti viselkedésről beszél. A PACELC modell kiegészíti ezt azzal a felismeréssel, hogy a **normál, partíciómentes** működés során is van kompromisszum: a késleltetés és a konzisztencia között.

A PACELC mozaikszó feloldása:

- **P** – ha partíció van,
- **A** – akkor a rendelkezésre állást,
- **C** – vagy a konzisztenciát választjuk;
- **E** – egyébként (else),
- **L** – a késleltetést (latency),
- **C** – vagy a konzisztenciát választjuk.

A modell tehát két döntési pontot jelöl. Partíció alatt a CAP-döntést, partíciómentes állapotban pedig azt, hogy a rendszer azonnali, gyors választ ad-e anélkül, hogy megvárná az összes replika megerősítését, vagy minden olvasásnál/írásnál konzisztens választ biztosít-e, ezért esetleg lassabb.

Leegyszerűsítve a négy leggyakoribb kombináció:

- **PA/EL:** partíció alatt rendelkezésre állás, normál állapotban gyors válasz, gyengébb konzisztencia. Gyakori a nagy olvasási terhelésű, toleráns katalógus- vagy beállításrendszerekben.
- **PC/EC:** partíció alatt konzisztencia, normál állapotban is konzisztencia, magasabb késleltetés. Gyakori a pénzügyi egyenlegeknél, erős tranzakciós követelményeknél.
- **PA/EC:** partíció alatt elérhető, normál állapotban konzisztens. Ez a kombináció hasznos, ha a rendszer normál üzemben szigorú garantál, de vészhelyzetben inkább elérhető marad, mint konzisztens.
- **PC/EL:** partíció alatt konzisztens, normál állapotban gyors, gyengébb konzisztencia. Ritkább, de előfordulhat, ha a normál üzem során a teljesítmény fontos, de partíció alatt sem engedhető meg az eltérés.

A PACELC gyakorlati haszna az, hogy a tervező ne csak a ritka partícióra optimalizáljon, hanem a mindennapi késleltetési célokra is. Ha például az SLO szerint az olvasási késleltetés p95 értékének 150 ms alatt kell lennie, akkor normál üzemben valószínűleg nem várhatunk minden olvasásnál szinkron megerősítést az összes replikától. Ezt a döntést a PACELC keretben lehet rögzíteni.

12.3.4 Konzisztenciamodellek, az eventual consistency pontos jelentése

A gyakorlatban a „konzisztencia” szó nem egyetlen tulajdonság, hanem modellek családja. A legerősebb a **lineáris konzisztencia** (linearizability): minden művelet úgy látszik, mintha egyetlen időpillanatban, a valós idejű sorrendet tiszteletben tartva hajtódott volna végre. Ez az a modell, amelyet a CAP-tételben C-vel jelölünk. Ennél gyengébb a **szekvenciális konzisztencia**, ahol minden csomópont ugyanazt a műveleti sorrendet látja, de az egyes műveletek közötti valós idejű sorrend nem feltétlenül érvényesül. A **kauzális konzisztencia** azt garantálja, hogy ha egy művelet ok-okozati viszonyban megelőz egy másikat, akkor minden csomópont az okot előbb látja, mint az okozatot.

A **read-your-writes** garancia azt jelenti, hogy a kliens a saját írását mindig látja a következő olvasásnál. A **monotonic reads** garancia szerint ha a kliens egyszer látott egy értéket, akkor később nem olvashat annál régebbi értéket. A **consistent prefix** pedig azt biztosítja, hogy az olvasó az írásokat a küldő sorrendjüknek megfelelően látja.

Az **eventual consistency** ezekhez képest gyenge modell. Azt garantálja, hogy ha a rendszerben egy ideig nem érkezik új írás, akkor véges időn belül minden replika ugyanarra az értékre fog állni. A modell nem garantálja, hogy egy friss írás után azonnali olvasás a friss értéket látja, és nem garantálja a friss olvasások sorrendjét sem. A gyakorlatban az eventual consistency azt jelenti, hogy a replikáció aszinkron: az írás előbb egy replikán válik láthatóvá, majd a háttérben terjed a többi felé. Az olvasó ezért átmenetileg elavult adatot kaphat. Azt az időtartamot, amíg az elavult adat eltűnik, **replikációs eltérésnek** (replication lag) vagy elavultsági ablaknak nevezzük. Ezt az ablakot mérni kell, például másodpercben, p95 vagy p99 értékkel.

Az eventual consistency nem „hiba”, hanem tudatos tervezési döntés, ha a követelmények szerint a frissesség átmeneti hiánya elfogadható. A döntést akkor lehet meghozni, ha meg tudjuk mondani, milyen műveletnél, milyen hosszú eltérés elfogadható, és mi történik, ha a replikák közötti eltérés akadályozza az üzleti folyamatot. Ha ez az eltérés nem dokumentált, akkor a rendszer viselkedése megjósolhatatlan lesz.

Az eventual consistency eléréséhez a replikáknak konvergálniuk kell. A konvergenciát segíti az **anti-entropy** folyamat, amely időnként összehasonlítja a replikákat, és a hiányzó adatokat pótolja; a **read repair**, amely olvasáskor észleli az eltérést, és javítja az elavult replikát; valamint a **gossip** protokoll, amely a csomópontok közötti állapotterjedést szervezi. A gyakorlatban ezek

a mechanizmusok együttesen biztosítják, hogy az eventual consistency ne váljon „soha nem konzisztens” állapottá.

12.3.5 Replikáció, kvórum és írás-olvasási szabályok

Az elosztott adattárolásban a replikák száma és az írási/olvasási műveletekhez szükséges megerősítések száma határozza meg a konzisztencia és a rendelkezésre állás közötti egyensúlyt. Gyakori jelölés:

- **N:** a replikák teljes száma,
- **W:** az íráshoz szükséges sikeres megerősítések száma,
- **R:** az olvasáshoz szükséges sikeres válaszok száma.

Az erős konzisztencia egyik gyakran használt szabálya, hogy $\mathbf{W + R > N}$. Ekkor bármely olvasási és írási halmaz metszete legalább egy közös replikát tartalmaz, így az olvasó látni fogja a legutóbbi írást. Ha például $N = 3$, $W = 2$ és $R = 2$, akkor $2 + 2 > 3$, tehát az olvasás és az írás metszete legalább egy replika. Fontos azonban, hogy a kvórum önmagában nem feltétlenül ad lineáris konzisztenciát; az írások sorrendjét és a konfliktusok feloldását is kezelni kell.

Ha a $W + R <= N$, akkor a rendszer gyengébb konzisztenciát biztosíthat, de nagyobb rendelkezésre állást és alacsonyabb késleltetést érhet el. A tervező a kvórumokkal finoman hangolhatja a viselkedést: írásoknál $W = 1$ gyors írást ad, de fennáll a veszély, hogy az az egy replika meghibásodik, mielőtt a többi replikára eljutna az adat. Ezért a gyakorlatban gyakran használnak **tartós írást** (durable write) és **memóriabeli írást** megkülönböztetett megerősítéssel.

A kvórumoknál is figyelembe kell venni a partíciót. Ha a replikák egy része elérhetetlen, akkor a W vagy R megerősítések száma nem érhető el, és a művelet vagy blokkol, vagy hibát ad. A CP rendszer ilyenkor elutasítja a műveletet, az AP rendszer viszont csökkentheti a kvórumot, vagy engedélyezheti a **sloppy quorum**-ot, amikor az írást olyan csomópontok is fogadják, amelyek nem az adott partíció hivatalos replikái, és később hintett handoff keretében átadják a megfelelő replikának. Ez javítja a rendelkezésre állást, de tovább növeli az elavultság és a konfliktus lehetőségét.

12.3.6 Elosztott tranzakciók, ACID és BASE

A monolitikus világban megszokott ACID-tranzakciók (atomicitás, konzisztencia, izoláció, tartósság) elosztott környezetben nem vihetők át automatikusan. A hagyományos **kétfázisú commit** (2PC) protokoll szigorú atomitást biztosít, de partíció vagy koordinátorhiba esetén hosszú blokkolást okozhat, és rosszul tűri a részleges hibát. A mikroszolgáltatások világában ezért gyakori, hogy a tervező lemond a szinkron, több csomóponton átnyúló ACID-tranzakcióról, és inkább lazán csatolt, kompenzálható műveletekre bontja a folyamatot. Ezt a gondolkodást foglalja össze a **BASE** betűszó: Basically Available, Soft state, Eventually consistent. A BASE nem egy algoritmus, hanem egy rugalmas követelményrendszer: a rendszer alapvetően elérhető, állapota átmenetileg lágy, azaz nem teljesen konzisztens, és a replikák idővel konvergálnak.

A BASE elfogadása nem jelenti azt, hogy a rendszer minden része elveszíti a konzisztenciát. Éppen ellenkezőleg: a tervezőnek pontosan meg kell jelölnie, mely műveletek igényelnek erős garanciát, és melyek tűrik a gyenge modellt. A foglalási rendszerben például az ügyfél egyenlegének csökkentése erős konzisztenciát igényelhet, míg a rendelés visszaigazoló e-mailjének kiküldése eventual consistency mellett is megvalósítható. A következő egységben tárgyalt saga-minta éppen ezt a kompenzáló tranzakciós megközelítést formalizálja, de itt elég annyi: az elosztott rendszer tervezésekor a tranzakcióhatárokat tudatosan, az üzleti folyamatok mentén kell meghúzni.

12.3.7 Gyakorlati döntés: egy ADR a Készlet-szolgáltatás konzisztencia-modelljéről

A fogalmak akkor válnak hasznossá, ha konkrét döntési helyzethez kapcsolódnak. Az alábbi példa egy lehetséges ADR-t mutat be a bevezetőben említett Foglalás- és Készlet-szolgáltatásra. Az ADR nem valós termelési rendszer, hanem oktatási minta; a benne szereplő értékek illusztrációk.

Cím: ADR-2025-04 – Készletfoglalás konzisztencia- és hibatűrési modellje

Kontextus: A rendszer a vásárlói foglalásokat fogadja. A Készlet-szolgáltatás három replikában tárolja a termékek elérhető mennyiségét. A foglalási művelet során a készletet csökkenteni kell, és ugyanazt a darabot nem szabad kétszer lefoglalni. A rendszer normál terhelési szintje 500 foglalási kérés percenként; a termékkatalógus olvasási forgalma ennek többszöröse. A készlet-adatbázis időnként partícióba kerülhet egy adatközponti hálózati karbantartás miatt. A QAS például:

- **Inger:** 500 foglalási kérés/perc, közben az egyik készlet-replika hálózati partícióba kerül legfeljebb 120 másodpercre.
- **Környezet:** normál üzem, három replika, a hálózati késleltetés p95 = 20 ms a normál tartományban.
- **Válasz:** minden foglalási kérés vagy konzisztensen végrehajtódik, vagy explicit elutasítást/„függőben” választ kap; az utolsó darabot nem lehet kétszer lefoglalni.
- **Mérőszám:** az elutasított vagy függőben tartott foglalások aránya a partíció alatt 5%; a duplikált foglalások száma = 0; a katalógus olvasási rendelkezésre állása 99,9% a mérési ablakban.

Döntés: A foglalási írási út CP viselkedést alkalmaz. A készletfoglalás írásnál N = 3, W = 2, R = 2. Partíció alatt, ha nem érhető el a W = 2 kvórum, a szolgáltatás nem csökkenti a készletet, hanem az ügyfél felé 409 Conflict vagy 202 Accepted „függőben” választ ad, a foglalást pedig kompenzálható állapotként tartja nyilván. A termékkatalógus olvasási út AP/EL viselkedést alkalmaz: a katalógus replikái aszinkron módon frissülnek, az olvasás nem vár minden replika megerősítésére, és a replikációs eltérés p95 értékét 10 másodperc alatt tartjuk. A foglalási API idempotencia-kulcsot követel meg, a retry kizárólag ugyanazzal a kulccsal történik.

Alternatívák: Az egyik alternatíva a foglalási út teljes AP jellegű berendezése volt, hogy a partíció alatt is minden kérés elfogadásra kerüljön. Ezt elvetettük, mert a túlfoglalás kockázata üzletileg elfogadhatatlan. A másik alternatíva a szinkron 2PC használata volt, de a koordinátorhiba miatti blokkolás és a foglalási késleltetés növekedése miatt elvetettük.

Következmények: A foglalási út erős konzisztenciát biztosít, de partíció alatt csökkenhet a sikeres foglalások aránya. A katalógus út gyors és elérhető, de előfordulhat, hogy a felhasználó néhány másodpercre régi árat lát. Az idempotencia-kulcs állapotát a foglalási szolgáltatásnak tartósan tárolnia kell. A döntést rendszeresen felül kell vizsgálni, ha a foglalási forgalom vagy a partíciók gyakorisága megváltozik.

Állapot: Elfogadva.

Ez az ADR kiemeli a CAP és PACELC döntési pontjait: a foglalási írás a PC/EC felé hajlik, a katalógus olvasás a PA/EL felé. A valós rendszer ennél több műveleti osztályt igényelhet, és a szabályok finomítása mindig a konkrét minőségi követelményektől függ.

Az alábbi Mermaid szekvenciadiagram a foglalási kérés partíció alatti viselkedését mutatja a fenti ADR szerint:

```
sequenceDiagram
    title Foglalási írás CP viselkedés partíció alatt
    participant Kliens as Kliens
    participant Foglalas as Foglalás API
    participant Replika1 as Készlet replika 1
```

```
participant Replika2 as Készlet replika 2
participant Replika3 as Készlet replika 3

Kliens->>Foglalas: POST /foglalasok, Idempotency-Key: 8f3a
Foglalas->>Replika1: írás W=2, timeout 200 ms
Replika1--xFoglalas: hálózati timeout
Foglalas->>Replika2: írás W=2, timeout 200 ms
Replika2-->>Foglalas: megerősítés
Foglalas->>Replika3: írás W=2, timeout 200 ms
Replika3-->>Foglalas: megerősítés
Foglalas-->>Kliens: 201 Created (W=2 teljesült)
note over Replika1: partíció után felzárkózik
```

Jelmagyarázat: szaggatott nyíl = időtúllépés; `x` = sikertelen hívás; folytonos nyíl = sikeres szinkron hívás. **Feltételjegyzet:** az ábra azt az esetet szemlélteti, amikor a Replika1 hálózati partícióban van, de a másik két replika elérhető, ezért W = 2 teljesül. A replikák közötti háttérszinkronizáció az ábrán nem látható, de a partíció feloldása után Replika1 felzárkózik.

12.3.8 A foglalási szolgáltatás hibatűrése: retry, timeout és idempotencia

A konzisztencia-döntés mellett a hibatűrési stratégia is szükséges. A hálózati partíció és a késleltetés miatt a Foglalás-szolgáltatásnak a következőket kell megterveznie:

- **Timeout:** minden kimenő híváshoz legyen időkorlát. A timeout legyen rövidebb, mint a hívó oldali SLO, hogy a lassú függőség ne terjedjen tovább felfelé. Például ha a Foglalás API SLO-ja p95 = 500 ms, akkor a Készlet-szolgáltatás hívás timeoutja legfeljebb 250 ms lehet, hogy maradjon idő a saját feldolgozásra.
- **Retry:** a retry csak idempotens műveleteknél vagy idempotencia-azonosítóval ellátott hívásoknál megengedett. A Foglalás-szolgáltatás a Készlet-szolgáltatás felé idempotencia-kulcsot küld; a Készlet-szolgáltatás a kulcs alapján ugyanazt a foglalási eredményt adja vissza.
- **Backoff és jitter:** a retry ne azonnal és ne determinisztikus időközönként történjen, hanem exponenciális backoff és véletlen jitter kombinációjával. Ez csökkenti az egyidejű újrapróbálkozások okozta terhelési hullámot.
- **Circuit breaker:** ha a Készlet-szolgáltatás hosszabb ideig hibás, a foglalási szolgáltatás rövidre zárhatja a hívást, és gyorsan hibát adhat vissza. Ez a saját erőforrásait védi és a partíció elhúzódása esetén is elkerüli a várakozó szálak kimerülését. A circuit breaker a következő fejezetben részletesebben is előkerül, itt elég annyi, hogy a konzisztencia-döntés kiegészítője.
- **Lejárati és kompenzálási állapot:** ha a foglalás „függőben” állapotba kerül, mert a kvórum nem teljesült, akkor a rendszernek idővel döntenie kell a foglalás sorsáról: vagy megerősíti, vagy kompenzálva visszaállítja a készletet. Az állapotnak tartósnak kell lennie, és a rendszer újraindulása után is visszaállíthatónak kell lennie.

A gyakorlatban ezek a beállítások nem valós termelési értékek, hanem a laborban ellenőrizendő paraméterek. A helyes timeout és retry-politika megválasztása terhelési teszteket, hálózati hibainjekciót és a hívási lánc végponttól végpontig tartó mérését igényli.

12.3.9 Labor: konzisztencia- és hibatűrési döntés elemzése

A labor célja az itt bemutatott fogalmak alkalmazása egy kis, ellenőrizhető példán. A feladat nem egy teljes termelési rendszer építése, hanem egy döntési dokumentum elkészítése, és annak reprodukálható szimulációval vagy jegyzetfüzettel való alátámasztása.

Kiinduló állapot: a labor résztvevője megkapja a Foglalás- és Készlet-szolgáltatás egyszerűsített modelljét. A modell tartalmaz egy `FoglalasAPI` és egy `KeszletSzolgaltatas` komponenst. A Készlet-szolgáltatás három replikával rendelkezik. A hálózat szimulálható: beállítható partíció, késleltetés, csomagvesztés és duplikáció. A labor helyőrző konfigurációja:

```
szimulacio:
  replikak: 3
  normal_kapcsolat:
    kesleltetes_p95_ms: 20
    csomagvesztes: 0.001
  partíció:
    celpont: replika_1
    időtartam_s: 120
    csomagvesztes: 1.0
foglalasi_api:
  timeout_ms: 250
  retry_max: 3
  idempotencia_kulcs: "teszt-8f3a"
```

A résztvevő Python-jegyzetfüzetben vagy más reprodukálható környezetben szimulálja a viselkedést. A szimulációban a foglalási kérések 500/perc beérkezési rátával érkeznek, a készlet kezdetben 1 darabot tartalmaz.

Elvárt eredmény: a jegyzetfüzetben vagy dokumentumban a következőket kell elkülönítve szerepeltetni:

1. **Kontextus:** a rendszer célja, a felhasználói szerepkörök, a külső rendszerek, a rendszerhatár és a releváns minőségi forgatókönyv.
2. **Döntési napló:** legalább egy ADR a konzisztencia- és hibatűrési döntésekről, az alternatívákkal és a következményekkel. Az ADR-nek tartalmaznia kell a CAP- és PACELC-besorolást a vizsgált műveletekre.
3. **Szimulációs modell:** a hálózati paraméterek, a retry-timeout beállítások és az idempotencia-kulcs kezelése.
4. **Mért eredmény:** a foglalási sikerességi arány, az elutasított kérések aránya, a duplikált foglalások száma, a megfigyelt replikációs eltérés, a p95 késleltetés. A mérést legalább háromszor meg kell ismételni, és közölni kell az egyes futások értékeit.
5. **Várt eredmény:** a mérés előtt rögzíteni kell a hipotézist, majd a mért és a várt eredményt külön kell összehasonlítani.

Ellenőrzési szempontok: a dokumentum akkor elfogadható, ha a minőségi forgatókönyv inger–környezet–válasz–mérőszám alakú; a ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot; a mérés környezete, időpontja és bizonytalansága rögzített; a mért és a várt értékek szétválasztva jelennek meg; a szimuláció ugyanazzal a bemenettel reprodukálható.

Visszaállítás: izolált környezetben dolgozzon; a szimuláció nem végez tényleges adatbázis-törlést vagy klaszter-átkonfigurálást. Ha éles rendszerkomponenst használ, azt csak dedikált, eldobható tesztkörnyezetben tegye, és a futtatás után állítsa vissza a kiinduló konfigurációt.

A laborban a mérés természetesen függ a szimulációs beállításoktól. Az alábbi értékek nem ellenőrzött mérési eredmények, hanem **illusztratív várható értékek** a feladat megértéséhez:

- normal működés, W = 2, R = 2: sikeres foglalás ~100%, p95 késleltetés ~45 ms;
- partíció Replika1 és a Foglalás API között, W = 2, R = 2: sikeres foglalás ~92%, elutasított/függőben ~8%, duplikált foglalás 0;
- katalógus replikációs eltérés p95: 6 s.

A hallgató saját mérése ettől eltérhet, és a cél éppen a mérési gondolkodás gyakorlása.

12.3.10 Ellenőrzőlista a konzisztencia- és hibatűrési döntésekhez

A fejezet végén az alábbi kérdéseket érdemes végigfuttatni minden elosztott rendszerkomponensnél:

- ☐ A rendszerhatár és a külső rendszerek egyértelműen jelöltek-e?
- ☐ A vizsgált műveletekhez van-e minőségi forgatókönyv (inger, környezet, válasz, mérőszám)?
- ☐ A partíció alatti viselkedés CP vagy AP jellegű-e, és ez dokumentált-e?
- ☐ A normál üzemi késleltetés-konzisztencia kompromisszum PACELC szerint rögzített-e?
- ☐ Az olvasási és írási kvórumok N, W, R értékei ismertek-e, és a $W + R > N$ szabály tudatosan alkalmazott-e?
- ☐ A retry csak idempotens műveletre vagy idempotencia-kulccsal történik-e?
- ☐ A timeout, a backoff, a jitter és a circuit breaker beállításai összhangban vannak-e a hívási lánc SLO-jával?
- ☐ A replikációs eltérés mért-e, és a mérési ablak dokumentált-e?
- ☐ A foglalási vagy tranzakciós állapot tartós-e, és újraindulás után visszaállítható-e?
- ☐ A döntés ADR-ben rögzített-e, alternatívákkal és következményekkel?

12.3.11 Önellenőrző kérdés

Egy rendszer két replikával tárolja a felhasználói beállításokat, $N = 2$, $W = 1$, $R = 2$. Normál működésben a gyors olvasás érdekében az olvasás mindkét replikától párhuzamosan kérdezi le az értéket, és a gyorsabb, frissebb értéket adja vissza. Hálózati partíció esetén az egyik replika nem elérhető. Magyarázza el, hogy ez a konfiguráció milyen CAP/PACELC viselkedést eredményez, milyen konzisztencia-probléma léphet fel, és hogyan lehetne a read-your-writes garanciát megvalósítani anélkül, hogy a rendszer elveszítené a partíció alatti rendelkezésre állást. Válaszában hivatkozzon a kvórumokra és az idempotencia-kulcs szerepére is.

A kérdés megválaszolásához érdemes visszagondolni a lineáris konzisztencia definíciójára: ha $W = 1$ és $R = 2$, akkor normál körülmények között az olvasás látja a legutóbbi írást, ha az írás befejeződött; ám ha az olvasás a két replika közül a régebbi értéket küldőt választja gyorsabbnak, sérülhet a frissesség. Partíció alatt az egyik replika elérhetetlensége esetén az olvasás nem tudja teljesíteni az $R = 2$ kvórumot, így a rendszer vagy hibát ad, vagy – ha AP módot választ – a helyi replikából ad elavult értéket. A read-your-writes garancia erősíthető például azzal, hogy az író kliens az írás után egy ideig a saját, megerősített replikájáról olvas, vagy az írásművelet verziószámot/timestampet kap, és az olvasás ezt figyelembe veszi. Az idempotencia-kulcs pedig a retry okozta duplikált írások elkerülését szolgálja, ami a partíció utáni újrapróbálkozásoknál elengedhetetlen.

Ez az egység azt mutatta be, hogy az elosztott rendszerek tervezése nem a tökéletes megoldás megtalálásáról szól, hanem a rossz kompromisszumok elkerüléséről és a következmények tudatos rögzítéséről. A CAP-tétel, a PACELC, a konzisztenciamodellek és az eventual consistency együttesen adnak közös nyelvet ahhoz, hogy a tervező ne „érzésből”, hanem explicit, visszakereshető döntésekkel alakítsa ki a rendszer hibatűrését. A következő egységben ezekre az alapokra építve a mikroszolgáltatások gyakori anti-mintáit és a szervezeti feltételeket tárgyaljuk, ahol a CAP-döntésekből származó félreértések és a szolgáltatáshatárok elhibázott megválasztása üzletileg is megmutatkozik.

12.4 Mikroszolgáltatások anti-minták és szervezeti feltételek

Az előző fejezetben a mikroszolgáltatások alapvető működését, a szolgáltatáshatárok kijelölését és az elosztott rendszerek alapfogalmait tekintettük át. Most arra a kérdésre összpontosítunk, hogy miért omlanak össze a mikroszolgáltatás-alapú projektek, vagy miért válnak a várt előnyök helyett drága, lassú és törékeny rendszerekké. A bukás legtöbbször nem a választott technológián, hanem architekturális és szervezeti hibákon múlik. Ezeket a hibákat nevezzük anti-mintáknak: olyan ismétlődő megoldásoknak, amelyek elsőre logikusnak tűnnek, de a gyakorlatban rendszeresen kárt okoznak.

A fejezetben három kiemelt anti-mintát vizsgálunk meg részletesen: a *distributed monolith*-ot, a *shared database*-t és a *lánchívást*. Ezután röviden kitérünk további gyakori hibákra, majd megvizsgáljuk, milyen szervezeti feltételek és platformképességek szükségesek ahhoz, hogy a mikroszolgáltatások ne váljanak kontraproduktívvá. A fejezet végén egy labor következik, amelyben anti-minta katalógust készítünk egy konkrét – akár kitalált – rendszerre, és döntési naplóval, valamint ellenőrzési szempontokkal dokumentáljuk a felismerést és a javítási irányokat.

A cél nem az, hogy a mikroszolgáltatásokat rossz színben tüntessük fel, hanem hogy megtanuljuk felismerni azokat a feltételeket, amelyek mellett valóban értéket adnak, és elkerüljük azokat a csapdákat, amelyek miatt sok szervezet visszatér a monolithoz – vagy ami még rosszabb, benne ragad egy elosztott, de valójában összecsomózott rendszerben.

12.4.1 1. Distributed monolith – az elosztott, de valójában összenőtt rendszer

A distributed monolith a leggyakoribb és legveszélyesebb anti-minta a mikroszolgáltatások világában. A kifejezés egy olyan rendszert jelöl, amely formailag több, hálózaton keresztül kommunikáló szolgáltatásból áll, valójában azonban ugyanúgy viselkedik, mint egy rosszul tagolt monolit: a komponensek szorosan csatolódnak, a változtatások hullámokat vetnek végig az egész rendszeren, és nincs valódi független telepíthetőség.

12.4.2 1.1 Tünetek és felismerés

A distributed monolith legjellemzőbb tünetei:

- **Együtt telepítés kényszere**: ha egy szolgáltatáson apró módosítást végzünk, a többi szolgáltatást is újra kell indítani, mert a szerződések, a közös könyvtárak vagy az adatséma szorosan összefügg. A deploy egység gyakorlatilag az egész rendszer, még ha a csomagolás több konténerre is bomlik.
- **Szinkron lánchívások mindenütt**: a szolgáltatások közvetlenül, szinkron HTTP/gRPC hívásokkal érik el egymást, és egyetlen felhasználói kérés kiszolgálása mély hívási láncot jár be. A szolgáltatások nem lazán csatoltak, hanem egymás belső állapotától függenek.
- **Közös adatbázis vagy közös séma**: több szolgáltatás ugyanazokat a táblákat olvassa és írja, így az adatmodell változtatása mindenkit érint.
- **Közös build vagy közös modell**: a szolgáltatások egyetlen repository-ban, közös osztálykönyvtárakkal készülnek, és a kódgenerálás vagy a megosztott entitások miatt nem lehet őket külön-külön fejleszteni.
- **Mindenki mindenkit hív**: a hívási gráf sűrű, nincsenek egyértelmű tulajdonosi határok, és nehéz megmondani, hogy egy adott funkcióért melyik csapat felelős.

Az alábbi C4 container szintű diagram egy tipikus distributed monolith szerkezetet mutat. A nyilak jelzik a szinkron hívásokat és a közös adatbázishoz való hozzáférést.

```
graph LR
    A[Ügyfél] -->|HTTP| B[API Gateway]
    B -->|HTTP| C[Katalógus szolgáltatás]
    B -->|HTTP| D[Kosár szolgáltatás]
    B -->|HTTP| E[Rendelés szolgáltatás]
    C -->|HTTP| D
    D -->|HTTP| E
    E -->|HTTP| C
    C -->|SQL| DB[(Közös adatbázis)]
    D -->|SQL| DB
    E -->|SQL| DB
```

A diagramon látható, hogy nincsenek egyirányú, stabil függőségek; a komponensek kölcsönösen hívják egymást, és mindannyian ugyanazt az adatbázist használják. Ez a szerkezet megakadályozza a független skálázást és telepítést.

12.4.3 1.2 Miért alakul ki?

A distributed monolith gyakran úgy jön létre, hogy egy meglévő monolitot úgy „bontanak fel”, hogy a kódbázist több szolgáltatásra osztják, de a szétválasztás technikai rétegek mentén történik, nem üzleti képességek mentén. Például külön szolgáltatás lesz a felhasználókezelés, külön a rendelés, de mindegyik ugyanazt a központi adatbázist használja, és szinkron hívásokkal kérdez le adatokat egymástól. Másik tipikus ok, hogy a csapatok nem kapnak valódi autonómiát: a szervezet továbbra is központilag koordinálja a release-eket, a séma-migrációkat és a tranzakciókat, ezért a szolgáltatások soha nem válnak önállóan telepíthetővé.

Fontos megérteni, hogy a distributed monolith *rosszabb*, mint egy hagyományos monolit. A monolitnak megvannak a maga előnyei: egyszerűbb a tranzakciókezelés, a hibakeresés, a telepítés és a tesztelés. A distributed monolit ezeket az előnyöket elveszíti, miközben a hálózati kommunikáció minden hátrányát – késleltetést, részleges hibát, sorosodást, nehezebb nyomkövetést – bevezeti. Az eredmény: magasabb üzemeltetési költség, alacsonyabb teljesítmény és ugyanolyan merev változtathatóság, mint a monolitnál.

12.4.4 1.3 Minőségi forgatókönyv példa

A distributed monolith hatását quality attribute scenario formában is megfogalmazhatjuk:

- **Inger**: a katalógus szolgáltatás üzleti logikájának módosítása (például új mező a termék entitáshoz).
- **Környezet**: éles rendszer, normál terhelés mellett.
- **Válasz**: a módosítás miatt a rendelés és a kosár szolgáltatást is újra kell indítani, mert közös adatsémát és szinkron hívásokat használnak. Az újraindítás két perc teljes kiesést okoz.
- **Mérőszám**: rendelkezésre állás az adott mérési ablakban $A = \frac{\text{üzemidő}}{\text{üzemidő}+\text{állásidő}} = \frac{58}{60} \approx 96{,}7\%$, szemben a célul kitűzött 99,9%-kal.

Ez a forgatókönyv jól mutatja, hogy a szoros csatolás hogyan rombolja a rendelkezésre állást és a gyors szállítást.

12.4.5 1.4 A kijavítás irányai

A distributed monolithból való kilépés nem pusztán technikai feladat; szervezeti és kulturális változásokat is igényel. A leggyakoribb lépések:

- **Szolgáltatáshatárok újradefiniálása** domain-vezérelt tervezéssel: minden szolgáltatás egy jól körülhatárolt üzleti képességet fedjen le, és birtokolja a saját adatait.
- **Adatbázis szétválasztása**: minden szolgáltatásnak saját adatbázisa legyen, és más szolgáltatások adatait API-n vagy eseményeken keresztül érje el.
- **Aszinkron kommunikáció bevezetése**: a szinkron lánchívások helyett eseményvezérelt integrációt, üzenetsorokat vagy streaming platformot használjunk.
- **Független deploy és release csatornák**: minden szolgáltatást külön build- és deploy pipeline kezeljen, és a csapatok jogosultak legyenek saját szolgáltatásuk önálló telepítésére.
- **Kontraktus tesztelés**: a szolgáltatások közötti API-szerződéseket automatikusan ellenőrizzük, hogy a változtatások ne borítsák fel a többieket.

A következőkben a másik két kiemelt anti-mintát vizsgáljuk, amelyek gyakran szorosan összefüggenek a distributed monolith-tal.

12.4.6 2. Shared database – a közös adatbázis csapdája

A shared database anti-minta akkor áll fenn, ha több mikroszolgáltatás ugyanazt az adatbázist, sőt gyakran ugyanazokat a táblákat használja. Ez a megközelítés elsőre időt takarít meg, mert a szolgáltatások könnyen hozzáférnek egymás adataihoz, és a tranzakciók egyszerűek maradnak. Hosszú távon azonban az egyik legnagyobb akadálya a független fejlődésnek és a skálázásnak.

12.4.7 2.1 Miért vonzó és miért veszélyes?

A közös adatbázis azért csábító, mert:

- **Nincs adatduplikáció**: minden szolgáltatás ugyanazt az adatforrást olvassa, így nem kell szinkronizálási mechanizmust építeni.
- **Egyszerű tranzakciók**: az ACID tranzakciók a hagyományos SQL-eszközökkel kezelhetők, nincs szükség elosztott tranzakciókra vagy kompenzációra.
- **Gyors fejlesztés**: a fejlesztők közvetlenül lekérdezhetik a szükséges adatokat, nem kell API-t tervezni és hívni.

A veszélyek azonban jelentősek:

- **Rejtett csatolás**: ha az egyik szolgáltatás megváltoztat egy táblaszerkezetet, az összes többi szolgáltatás, amely azt a táblát használja, elromolhat. A séma-migrációk így minden csapatot érintenek, és gondos koordinációt igényelnek.
- **Adat tulajdonjog hiánya**: senki sem felelős egyértelműen egy adott adatért, így a minőség és a konzisztencia romlik.
- **Skálázási szűk keresztmetszet**: az adatbázis központi erőforrássá válik, amelyet minden szolgáltatás terhel; a tranzakciós zárolások és a versengés teljesítményproblémákat okoz.
- **Körkörös függőségek**: a szolgáltatások adatbázison keresztül is összekapcsolódnak, ami nehezen áttekinthető függőségi gráfot eredményez.
- **Tesztelési nehézség**: az integrációs tesztekhez közös adatbázis szükséges, ami lelassítja a folyamatot és növeli a flaky tesztek esélyét.

12.4.8 2.2 Példa és a helyes megoldás

Képzeljünk el egy webáruházat, ahol a *Katalógus*, a *Kosár* és a *Rendelés* szolgáltatás mind ugyanazt a `products`, `cart_items` és `orders` táblát használja. Ha a Katalógus csapat új mezőt ad a `products` táblához (például `weight_gram`), akkor a Kosár és a Rendelés szolgáltatás SQL-lekérdezései csak akkor működnek tovább, ha azokat is módosítják, különben a `SELECT *` vagy a merev oszloplisták hibát okoznak. A release-eket tehát össze kell hangolni, ami sérti a független telepíthetőséget.

A helyes megoldás az **adatbázis per szolgáltatás** elv: minden szolgáltatás kizárólag a saját adatbázisát éri el, más szolgáltatások adatait pedig jól definiált API-n, eseményeken vagy olvasási modelleken keresztül kapja meg. A gyakorlatban ez gyakran együtt jár a CQRS (Command Query Responsibility Segregation) bevezetésével, ahol az írási oldal tranzakciósan kezeli a saját adatbázist, az olvasási oldal pedig eseményekből frissülő, denormalizált olvasási modelleket használ.

Fontos megjegyezni, hogy a közös adatbázis *nem* mindig tilos. Kis méretű, alacsony forgalmú rendszereknél, ahol a csapatok közötti koordináció olcsó, ideiglenesen elfogadható lehet, de tudatos döntésként kell dokumentálni, és világos migrációs tervvel kell rendelkezni a szétválasztásra. A shared database anti-mintává akkor válik, ha akadályozza a független fejlődést és a skálázást, és ha a rendszer növekedésével a koordinációs költség robbanásszerűen megnő.

12.4.9 3. Lánchívás – a szinkron hívások mély lánca

A lánchívás (chain call, gyakran *deep call chain* vagy *synchronous call chain*) azt a helyzetet jelöli, amikor egyetlen felhasználói kérés kiszolgálásához a szolgáltatások egymást szinkron módon hívják, és a hívási lánc több szinten keresztül halad. Például az API Gateway meghívja a Rendelés szolgáltatást, az meghívja a Fizetés szolgáltatást, az meghívja a Készlet szolgáltatást, az pedig a Szállítás szolgáltatást. Mindegyik lépés blokkolja az előzőt, és a teljes válaszidő a lánc összes lépésének idejéből áll össze.

12.4.10 3.1 Problémák és következmények

A lánchívás számos problémát okoz:

- **Késleltetés összeadódik**: a teljes válaszidő a lánc minden lépésének késleltetését tartalmazza. Ha minden szolgáltatás átlagosan 100 ms-ot válaszol, egy ötlépcsős lánc már legalább 500 ms-ot vesz igénybe, de a hálózati késleltetés és a sorban állás ezt tovább növeli.
- **Hibaterjedés**: ha a lánc bármelyik eleme meghibásodik, az egész kérés meghiúsul, még akkor is, ha a többi szolgáltatás egészséges. A rendelkezésre állás a lánc hosszának növekedésével drasztikusan romlik. Ha minden szolgáltatás rendelkezésre állása 99%, akkor öt láncszem esetén a teljes elérhetőség elméletileg $0{,}99^5 \approx 0{,}951$, azaz 95,1% – ami 99,9%-os SLO-cél esetén elfogadhatatlan. (A valóságban a függetlenség nem mindig teljesül, de a tendencia jól látható.)
- **Erőforrás-kimerülés**: a szinkron hívások blokkolják a szálakat, így nagy terhelés alatt a lánc végén lévő szolgáltatások lassulása visszafelé terjed, és az egész rendszer összeomolhat (*cascading failure*).
- **Holtpont és ciklikus függőség**: ha a lánc körbe ér, a szolgáltatások egymásra várnak, ami végtelen hurkokat vagy időtúllépéseket okoz.

- **Nehéz hibakeresés**: a hosszú lánc miatt nehéz megtalálni, hol romlott el a kérés; elengedhetetlen az elosztott nyomkövetés (distributed tracing), amelynek hiányában a diagnózis órákig tarthat.

12.4.11 3.2 Megoldási stratégiák

A lánchívás elkerülésére több bevált módszer létezik:

- **Aszinkron kommunikáció**: ahol lehetséges, a szinkron hívásokat eseményvezérelt üzenetváltással helyettesítjük. A rendelés leadásakor például a Rendelés szolgáltatás kibocsát egy `OrderPlaced` eseményt, amelyet a Fizetés és a Készlet szolgáltatás aszinkron módon fogyaszt. Így a felhasználói kérés nem vár a lánc minden lépésére.
- **API Gateway aggregáció**: az API Gateway vagy egy BFF (Backend for Frontend) összegyűjti a szükséges adatokat párhuzamosan, és egyetlen válaszban adja vissza a kliensnek. Ezzel a kliens oldali késleltetés csökken, bár a szerver oldali hívások még mindig szinkronok lehetnek – fontos, hogy párhuzamosítsuk őket.
- **Cache és olvasási modellek**: a gyakran használt adatokat a szolgáltatások helyben cache-elhetik, vagy előre elkészített olvasási modelleket tárolhatnak, így nem kell minden kérésnél más szolgáltatásokhoz fordulni.
- **Timeout, retry, circuit breaker**: a szinkron hívásokat mindig időtúllépéssel, újrapróbálkozással és megszakítóval kell körülvenni, hogy a hibák ne terjedjenek tovább.
- **Szolgáltatás-összevonás**: ha két szolgáltatás olyan szorosan együttműködik, hogy állandó szinkron hívásokra van szükségük, érdemes megfontolni, hogy valójában egyetlen szolgáltatásba tartoznak-e.

A lánchívás anti-minta gyakran együtt jár a shared database-szel és a distributed monolith-tal, de önállóan is előfordul még akkor is, ha az adatbázisok szét vannak választva. Ezért fontos külön néven kezelni és tudatosan tervezni ellene.

12.4.12 4. Egyéb gyakori anti-minták röviden

A fenti három mellett számos további anti-minta okozhat kudarcot a mikroszolgáltatás-projektekben. Ezek röviden:

- **Nanoservice-őrület**: a szolgáltatásokat túlzottan apróra vágják, gyakran egy-egy függvény vagy végpont külön szolgáltatássá válik. Ez elszaporítja az üzemeltetési terhet, a hálózati hívásokat és a bonyolultságot anélkül, hogy valódi üzleti értéket adna.
- **Technikai felbontás domain helyett**: a szolgáltatásokat rétegek (controller, service, repository) vagy technikai funkciók (authentication, logging) szerint választják szét, nem üzleti képességek szerint. Az eredmény gyakran distributed monolith lesz.
- **Nincs API gateway**: a kliensek közvetlenül hívják a mikroszolgáltatásokat, így a kliens oldalon túl sok hívás, bonyolult retry-logika és biztonsági kockázat jelenik meg. Az API gateway hiánya miatt a keresztmetszeti aggodalmak (autentikáció, rate limiting, caching) szétszóródnak.
- **Nincs observability**: a rendszer fekete dobozként működik. Naplók, metrikák és nyomkövetés nélkül lehetetlen a hibák gyors diagnosztizálása, és a rendszer megbízhatósága alacsony marad. A mikroszolgáltatások elengedhetetlen feltétele a jó megfigyelhetőség.
- **Nincs contract testing**: a szolgáltatások közötti API-szerződéseket nem ellenőrzik automatikusan, ezért a változtatások gyakran törést okoznak. A consumer-driven contract tesztek hiánya a release-ek összehangolását kényszeríti ki.

- **Manuális deployment**: ha a telepítés kézi lépéseket igényel, a skála növekedésével a hibák száma exponenciálisan nő. Az automatizált CI/CD pipeline nélkül a mikroszolgáltatások előnyei nem realizálódnak.
- **Adatbázis tranzakciók határokon át**: elosztott tranzakciók (például kétfázisú commit) használata a szolgáltatások között. Ez rendkívül törékeny, és sérti a lazán csatolt működést. Helyette saga, outbox vagy eseményvezérelt kompenzáció javasolt.
- **Nincs szolgáltatás-katalógus**: a szolgáltatások nincsenek regisztrálva, a függőségek és a tulajdonosok nem ismertek, így a változások hatását nehéz felmérni.

Ezek az anti-minták gyakran egymást erősítik, és együttesen vezetnek ahhoz, hogy a mikroszolgáltatás-architektúra többet árt, mint használ.

12.4.13 5. Szervezeti érettség és a platform szerepe

A mikroszolgáltatások sikere nem pusztán technológiai kérdés. Sokkal nagyobb mértékben függ a szervezet érettségétől, a csapatok felépítésétől és a rendelkezésre álló platformképességektől. Ha ezek hiányoznak, a mikroszolgáltatás bevezetése majdnem biztosan anti-mintákhoz vezet.

12.4.14 5.1 Milyen szervezeti feltételek szükségesek?

1. **DevOps kultúra**: a fejlesztő és üzemeltető csapatok együttműködése, automatizált deployment, infrastruktúra mint kód (IaC), folyamatos integráció és szállítás. Enélkül a sok szolgáltatás kezelhetetlenné válik.
2. **Önálló, keresztfunkcionális csapatok**: minden szolgáltatásért egy csapat felel, amely teljes körűen birtokolja a kódot, a tesztelést, a telepítést és az üzemeltetést. A csapatoknak joguk van saját release-eket kiadni anélkül, hogy másokat megkérdeznének.
3. **Domain-orientált felbontás**: a szolgáltatáshatárok az üzleti domainhez igazodnak (például DDD bounded context). Ez csökkenti a csapatok közötti kommunikációs kényszert, és lehetővé teszi a párhuzamos munkát.
4. **Conway törvényének tudatos kezelése**: a rendszerarchitektúra tükrözi a szervezeti kommunikációs struktúrát. Ha a szervezet erősen szilózott, a rendszer is szétesik; ha a csapatok nem illeszkednek a szolgáltatásokhoz, akkor a kommunikációs kényszerek miatt szoros csatolás alakul ki. A Team Topologies megközelítés (stream-aligned, enabling, platform, complicated-subsystem teamek) segíthet a szervezeti struktúra tudatos kialakításában.
5. **Megfelelő folyamatok és governance**: könnyű, de hatékony architektúra-governance, amely ADR-ekkel, automatikus ellenőrzésekkel és fitness functionökkel biztosítja a döntések nyomon követését.
6. **Folyamatos mérés és visszacsatolás**: SLO-k, error budget, hibaelemzés, postmortem kultúra. Ezek nélkül a rendszer minősége romlik, és a hibákból nem tanul a szervezet.

12.4.15 5.2 A platform szerepe

A platform mint termék (platform engineering) az egyik leghatékonyabb eszköz az anti-minták megelőzésére. Egy jó belső fejlesztői platform (IDP) a következő képességeket nyújtja:

- **Golden pathok**: előre definiált, jól dokumentált, automatizált útvonalak új szolgáltatás létrehozásához, buildeléséhez, teszteléséhez, deploymentjéhez és megfigyeléséhez. A fejlesztőknek nem kell minden infrastrukturális részletet ismerniük.
- **Önkiszolgáló deployment**: a csapatok saját maguk telepíthetik a szolgáltatásaikat, felügyelt pipeline-okon keresztül, anélkül hogy egy központi csapat jóváhagyására várnának.

- **Observability alapértelmezetten**: minden szolgáltatás automatikusan kap naplózást, metrikákat, nyomkövetést, dashboardokat és riasztásokat. Ez csökkenti a fekete doboz problémát.
- **Szolgáltatás-katalógus**: a platform nyilvántartja a szolgáltatásokat, a tulajdonosokat, a függőségeket és az API-kat, így a rendszer áttekinthető marad.
- **Konfiguráció- és titokkezelés**: központi, biztonságos mechanizmusok a környezeti változók és titkok kezelésére.
- **Konténer- és Kubernetes-absztrakció**: a platform elrejti a komplexitást, de lehetővé teszi a megfelelő erőforrás-kezelést, autoskálázást és hibatűrést.

A platform hiánya miatt a csapatok kénytelenek maguk építeni az infrastruktúrát, ami inkonzisztens megoldásokhoz, manuális lépésekhez és végső soron anti-mintákhoz vezet.

12.4.16 5.3 Mikor ne válasszunk mikroszolgáltatást?

A szervezeti érettség hiánya gyakran azt jelzi, hogy a mikroszolgáltatás még korai. Ha a csapat kicsi, a domain nem bonyolult, a terhelés alacsony, és a független skálázás nem követelmény, akkor egy moduláris monolit jobb választás lehet. A mikroszolgáltatások bevezetése akkor indokolt, ha:

- több független csapat dolgozik párhuzamosan ugyanazon a terméken;
- a rendszer egyes részei eltérő skálázási vagy technológiai igényekkel rendelkeznek;
- a domain kellően nagy és jól körülhatárolható kontextusokra bontható;
- a szervezet rendelkezik DevOps érettséggel és megfelelő platformmal;
- a gyakori, független release-ek üzleti értéket jelentenek.

Ha ezek a feltételek nem állnak fenn, a mikroszolgáltatás bevezetése nagy valószínűséggel a fenti anti-minták valamelyikébe torkollik.

12.4.17 6. Labor: anti-minta katalógus készítése

Ebben a laborban a cél egy **anti-minta katalógus** elkészítése egy adott rendszerre. A feladat nem technikai implementáció, hanem architektúra-elemzés, amelynek során azonosítjuk a tipikus mikroszolgáltatás anti-mintákat, dokumentáljuk őket, és javítási irányokat fogalmazunk meg. A labor eredménye reprodukálható dokumentum, amely tartalmazza a kontextust, a döntési naplót és az ellenőrzési szempontokat.

12.4.18 6.1 Kiinduló állapot

Válasszunk egy olyan rendszert, amelyet ismerünk vagy el tudunk képzelni: lehet egy meglévő mikroservice-alapú rendszer (akár nyílt forráskódú példa), egy cég belső rendszere, vagy egy hipotetikus alkalmazás. A laborhoz példaként egy fiktív webáruház rendszert használunk, amely a következő szolgáltatásokból áll:

- **Katalógus szolgáltatás**: termékek, kategóriák, árak kezelése.
- **Kosár szolgáltatás**: a felhasználó kosarának kezelése.
- **Rendelés szolgáltatás**: rendelések leadása, státuszkövetés.
- **Fizetés szolgáltatás**: tranzakciók feldolgozása.
- **Készlet szolgáltatás**: raktárkészlet kezelése.

A rendszer jelenlegi architektúrája:

- Minden szolgáltatás ugyanazt a PostgreSQL adatbázist használja.
- A szolgáltatások szinkron REST hívásokkal kommunikálnak: a Rendelés meghívja a Fizetést, a Fizetés a Készletet, a Készlet a Katalógust stb.
- A deployment manuális: a fejlesztők SSH-n keresztül másolják fel a JAR fájlokat, és kézzel indítják újra a szolgáltatásokat.
- Nincs központi naplózás vagy nyomkövetés, minden szolgáltatás a saját lokális fájljába ír.
- A csapatok technológiánként szerveződnek: van egy frontend csapat, egy backend csapat és egy adatbázis csapat, akik minden szolgáltatáson dolgoznak.

12.4.19 6.2 A labor menete

1. **Kontextus rögzítése**: Írjuk le a rendszer célját, a fő felhasználói folyamatokat, a csapatfelépítést, a technológiai stacket és a jelenlegi architektúrát. Készítsünk C4 context és container diagramot diagrams-as-code formában (például Mermaid vagy Structurizr DSL). Jelöljük a rendszerhatárt és a külső rendszereket.

2. **Anti-minták azonosítása**: Vizsgáljuk meg a rendszert a fejezetben tárgyalt anti-minták szempontjából. Legalább a következőket értékeljük:

   - Distributed monolith?
   - Shared database?
   - Lánchívás?
   - Nanoservice-őrület?
   - Technikai felbontás domain helyett?
   - Nincs API gateway?
   - Nincs observability?
   - Nincs contract testing?
   - Manuális deployment?

   Minden azonosított anti-mintához készítsünk egy bejegyzést a katalógusba.

3. **Bejegyzés formátuma**: Minden bejegyzés tartalmazzon:

   - **Neve és rövid leírása**.
   - **Tünetek**: milyen megfigyelhető jelek utalnak rá.
   - **Következmények**: milyen minőségi attribútumokat sért (teljesítmény, rendelkezésre állás, karbantarthatóság, skálázhatóság, biztonság).
   - **Kontextus**: melyik szolgáltatásokra, adatbázisokra, kommunikációs csatornákra vonatkozik.
   - **Javasolt javítás**: konkrét irány, például adatbázis szétválasztás, eseményvezérelt kommunikáció, API gateway bevezetése, CI/CD pipeline, observability stack.
   - **Kapcsolódó ADR hivatkozás**: ha készítünk ADR-t, hivatkozzunk rá.

4. **Döntési napló (ADR) készítése**: A legfontosabb javítási irányokhoz írjunk ADR-eket. Minden ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Például:

   - ADR-001: A közös adatbázis szétválasztása szolgáltatásonként.
   - ADR-002: Aszinkron eseményvezérelt integráció bevezetése a Rendelés-Fizetés-Készlet folyamatban.
   - ADR-003: API Gateway bevezetése a kliens hívások egységesítésére.
   - ADR-004: Automatizált CI/CD pipeline bevezetése minden szolgáltatáshoz.

5. **Mért és várt eredmények elkülönítése**: A labor során készítsünk mérést vagy becslést az anti-minták hatásáról. Például:

   - Mérjük meg a jelenlegi válaszidőt egy tipikus kérésre (például rendelés leadása) terhelés nélkül és 100 párhuzamos kérés alatt. Használjunk például `curl`-t és `ab`-t (Apache Bench) vagy `wrk`-t, ha elérhető tesztkörnyezet.
   - Mérjük meg a deployment időt egy apró változtatás esetén (jelenleg manuális, ezért időigényes).
   - Becsüljük meg a rendelkezésre állást az elmúlt 30 nap alapján (ha vannak naplók), vagy használjunk szimulált forgatókönyvet.
   - Várt eredmény: a javítások után a válaszidő csökken, a rendelkezésre állás nő, a deployment idő lerövidül.
   - **Fontos**: a mért és a várt eredményt külön szakaszban, egyértelműen jelölve dokumentáljuk. A nem futtatott mérést, benchmarkot vagy költséget nem szabad ellenőrzött eredményként bemutatni. Ha csak becslést tudunk adni, jelöljük, hogy becslés, és adjuk meg a bizonytalanságot.

6. **Ellenőrzési szempontok összeállítása**: A labor végén soroljunk fel legalább öt ellenőrzési szempontot, amelyek alapján utólag értékelhetjük, hogy a javítások sikeresek voltak-e. Például:

   - Egy szolgáltatás önállóan telepíthető anélkül, hogy más szolgáltatást újra kellene indítani.
   - A szolgáltatások közötti szinkron hívások száma 50%-kal csökkent.
   - A rendelés leadásának átlagos válaszideje 500 ms alatt van 100 párhuzamos kérésnél.
   - A rendszer rendelkezésre állása eléri a 99,9%-ot havi mérési ablakban.
   - Az új fejlesztő 1 napon belül képes lokális környezetet felállítani, és szolgáltatást telepíteni.

7. **Visszaállítási út**: Mivel a labor elemzési és dokumentációs feladat, visszaállítás többnyire nem szükséges. Ha mégis valamilyen módosítást hajtunk végre egy tesztkörnyezetben, akkor azt izoláltan tegyük, és jegyezzük le, hogyan lehet visszaállítani az eredeti állapotot (például a verziókezelőben a korábbi commit, az adatbázis visszaállítása dumpból, a konfiguráció visszavonása). A laborban éles rendszert ne módosítsunk.

12.4.20 6.3 Elvárt eredmény

A labor végén a hallgató rendelkezik:

- Egy verziózott dokumentummal (például Markdown vagy Jupyter notebook), amely tartalmazza a rendszer kontextusát, a C4 diagramokat, az anti-minta katalógust legalább öt bejegyzéssel, a döntési naplót legalább két ADR-rel, a mért és várt eredményeket elkülönítve, valamint az ellenőrzési szempontokat.
- A dokumentumban minden állítást forrással vagy mért adattal támaszt alá, és a bizonytalanságokat jelzi.

12.4.21 7. Döntési gyakorlat

Feladat: Egy startup három fejlesztőből áll, és egyetlen, viszonylag kis forgalmú webalkalmazást épít. A CTO olvasott a mikroszolgáltatások előnyeiről, és azt javasolja, hogy a rendszert azonnal bontsák 12 mikroszolgáltatásra, mindegyikhez külön adatbázissal, Kubernetesen, API gatewayjel

és eseményvezérelt kommunikációval. A csapatnak nincs tapasztalata konténerizációban, nincs DevOps mérnök, és a termék még nem ért el piaci validációt.

Kérdés: Helyes-e ez a döntés? Milyen szervezeti feltételek hiányoznak? Milyen anti-minták valószínűsíthetők? Mit javasolnál helyette, és miért? Válaszodat indokold, és fogalmazz meg legalább egy ADR-t a döntéshez.

12.4.22 8. Ellenőrzőlista

Az alábbi ellenőrzőlista segít felmérni, hogy egy szervezet készen áll-e a mikroszolgáltatások bevezetésére, illetve hogy egy meglévő rendszerben elkerülte-e a fő anti-mintákat.

- ☐ Minden szolgáltatás üzleti képességet fed le, és világos tulajdonosa van.
- ☐ A szolgáltatások saját adatbázissal rendelkeznek; más szolgáltatások adatait API-n vagy eseményeken keresztül érik el.
- ☐ A szolgáltatások közötti kommunikáció lehetőleg aszinkron; a szinkron hívások száma minimális.
- ☐ A rendelkezésre állás és a teljesítmény célértékei SLO formában definiáltak, és mérik őket.
- ☐ Minden szolgáltatás önállóan, automatizált CI/CD pipeline-on keresztül telepíthető.
- ☐ Létezik API gateway vagy BFF a kliensek kiszolgálására.
- ☐ A rendszer rendelkezik megfigyelhetőséggel: naplózás, metrikák, nyomkövetés, dashboardok, riasztások.
- ☐ A szolgáltatások közötti API-k consumer-driven contract tesztekkel védettek.
- ☐ A szervezeti struktúra illeszkedik a szolgáltatáshatárokhoz (Conway törvénye).
- ☐ A platform biztosítja a golden pathokat, önkiszolgáló deploymentet és a szolgáltatáskatalógust.
- ☐ A csapatok rendelkeznek DevOps képességekkel, és joguk van önálló release-eket kiadni.
- ☐ A rendszerben nincs elosztott tranzakció, lánchívás vagy közös adatbázis anti-mintája.

12.4.23 9. Önellenőrző kérdés

Mi a különbség a hagyományos monolit és a distributed monolith között, és miért utóbbi általában rosszabb?
Válasz elvárás: A hagyományos monolit egyetlen telepíthető egység, egyszerű tranzakciókezeléssel és hibakereséssel, de merev skálázással. A distributed monolith formailag több szolgáltatásból áll, de ugyanolyan szoros csatolású, mint a monolit; emellett viseli a hálózati kommunikáció hátrányait (késleltetés, részleges hibák, nehezebb nyomkövetés), miközben nem nyeri el a mikroszolgáltatások előnyeit (független deploy, skálázás, technológiai szabadság). Ezért összességében rosszabb, mint egy jól strukturált monolit.

Ezzel a fejezet bemutatta a mikroszolgáltatások leggyakoribb anti-mintáit, azok felismerését és a megelőzésükhöz szükséges szervezeti és platformfeltételeket. A következő egységben az eseményvezérelt architektúrákra térünk át, ahol az itt említett aszinkron kommunikáció és eseményalapú integráció részletesen kifejtésre kerül.

13 Eseményvezérelt és streaming architektúrák

13.1 Esemény, üzenet és parancs

13.1.1 Bevezetés

Az eseményvezérelt és streaming architektúrák sikerének egyik legfontosabb alapköve, hogy a résztvevő rendszerek pontosan ugyanazt értsék, amikor egy üzenetet kicserélnek. A „rendelés létrejött” és a „hozd létre a rendelést” közötti különbség első pillantásra nyelvi árnyalatnak tűnik, valójában azonban eltérő felelősségi határokat, eltérő életciklusokat és eltérő hibatűrési stratégiákat foglal magában. Ez a fejezet azt a fogalmi keretet építi fel, amely nélkül a későbbi CQRS-, Event Sourcing- és saga-minták könnyen félreérthetők lennének.

A fejezet előfeltételezi a mikroszolgáltatások és az elosztott rendszerek alapfogalmait (Mikroszolgáltatások anti-minták és szervezeti feltételek), de nem ismétli meg azokat. A cél itt kifejezetten az üzenet, a parancs és az esemény szemantikájának pontos elkülönítése, valamint egy olyan szerződéstervezési labor, amelyben a fogalmakat azonnal alkalmazni lehet.

13.1.2 1. Az üzenet mint közös hordozó

Az elosztott rendszerekben az üzenet az a technikai csomag, amely egy csatornán – jellemzően egy üzenetsoron, egy streaming platformon vagy egy HTTP-kérés törzsében – eljut a küldőtől a fogadóig. Az üzenet nem mondja meg önmagáról, hogy parancs vagy esemény; ezt a szerződés, valamint a fejlécben tárolt típusmező rögzíti. Az üzenet szerkezeti értelemben három fő részből áll:

- **Fejléc (header):** metaadatok, például üzenettípus, sémaverzió, korrelációs azonosító, oksági azonosító, időbélyeg, deduplikációs kulcs.
- **Törzs (payload):** maga az adat, amelyet a küldő továbbít.
- **Címzés:** pont-pont csatorna esetén egy konkrét fogyasztó, publish-subscribe csatorna esetén egy téma (topic) és opcionálisan egy csoport.

Egy üzenet tehát nem más, mint szállítási egység. A szemantikai többletet az adja, hogy a törzsben lévő adat milyen szándékot vagy tényt közvetít. A következő két alfejezet ezt a szemantikai többletet bontja ki.

Példa egy általános üzenetvázra JSON formátumban, amely szemléltető jellegű, nem végrehajtandó:

```
{
  "header": {
    "message_id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "type": "OrderPlaced",
    "schema_version": "1.0",
    "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
    "created_at": "2025-01-15T10:00:00.123Z"
  },
  "payload": {
```

```
    "order_id": "order-987",
    "customer_id": "cust-123",
    "items": [
      {"product_id": "prod-1", "quantity": 2}
    ]
  }
}
```

A `type` mező az, amely a fogyasztó számára eldönthetővé teszi, hogy az üzenetet parancsként vagy eseményként kell-e értelmeznie. A szerződésben ezt a típust egyértelmű névvel és irányított jelentéssel kell rögzíteni.

13.1.3 2. A parancs: a szándék

A parancs olyan üzenet, amely egy konkrét végrehajtót arra utasít, hogy egy meghatározott műveletet hajtson végre. A parancs mindig **szándékot** fejez ki: a küldő kéri, hogy valami történjen, de nem állítja, hogy a dolog már megtörtént. A parancs neve jellemzően felszólító módú, például `PlaceOrder`, `ReserveInventory`, `UpdateProfile`.

A parancs legfontosabb tulajdonságai:

- **Címzettje van:** egy parancsot általában egyetlen logikai fogyasztó dolgoz fel. A rendszerben előfordulhat több fizikai példány (például skálázott fogyasztói csoport), de a feladat egy adott szolgáltatás vagy modul felelősségi körébe tartozik.
- **Elutasítható:** a fogyasztó a parancsot validálhatja, és ha a feltételek nem teljesülnek, elutasíthatja. Az elutasítás lehet negatív nyugta vagy hibaesemény, de a parancs feldolgozójának vissza kell jeleznie.
- **Nem tényállítás:** amíg a parancs nincs végrehajtva, addig a rendszer állapota nem feltétlenül változott meg. A parancs elküldése nem jelenti azt, hogy a művelet sikeres volt.
- **Validálandó:** a parancsban szereplő adatokat a fogyasztónak érvényesítenie kell, mert a küldő nem feltétlenül ismeri az összes üzleti szabályt.

A parancs tipikus adattartalma az üzleti művelethez szükséges minimális adat, nem pedig a teljes rendszerállapot. Például egy `PlaceOrder` parancs tartalmazza a vásárló azonosítóját és a tételsorokat, de nem tartalmazza a rendelés aktuális állapotát, mert az a fogyasztó felelőssége.

Fontos a parancs **idempotens feldolgozása**. Mivel a hálózatban és az üzenetkézbesítésben előfordulhat duplikáció, a fogyasztónak képesnek kell lennie arra, hogy ugyanazt a parancsot többször megkapva is csak egyszer hajtsa végre a műveletet. Ehhez a parancsnak egyedi azonosítót kell tartalmaznia, amelyet a fogyasztó deduplikációs kulcsként használ. A tipikus mezőnév `command_id`.

Például egy parancs-szerződés:

```
{
  "type": "command",
  "name": "PlaceOrder",
  "schema_version": "1.0",
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
  "customer_id": "cust-123",
  "items": [
    {"product_id": "prod-1", "quantity": 2}
  ],
  "submitted_at": "2025-01-15T10:00:00Z"
}
```

A parancs felelősségi határa: a küldő nem írja elő a fogyasztó belső megvalósítását, de pontosan megnevezi a kért műveletet. A fogyasztó eldöntheti, hogy a műveletet szinkron vagy aszinkron módon, milyen belső lépésekben hajtja végre. Ez a határtartás csökkenti a csatolást.

13.1.4 3. Az esemény: a történt tény

Az esemény olyan üzenet, amely egy **már megtörtént, visszavonhatatlan tényt** rögzít. Az esemény neve jellemzően múlt idejű, például `OrderPlaced`, `InventoryReserved`, `ProfileUpdated`. Az esemény kiadója a saját állapotváltozását publikálja, és nem kér a fogyasztótól semmilyen műveletet.

Az esemény legfontosabb tulajdonságai:

- **Tényállítás:** az esemény azt mondja el, hogy valami már megtörtént. A kiadó számára az esemény az igazság, és ezt a tényt a későbbiekben nem vonja vissza (legfeljebb egy újabb eseményt bocsát ki, amely kompenzálja a hatást, például `OrderCancelled`).
- **Nincs címzettje abban az értelemben, hogy a fogyasztó nem feladat-végrehajtó:** az eseményt jellemzően publish-subscribe módon publikálják, és több fogyasztó is meghallgathatja. A kiadó nem tudja, és nem is kell tudnia, hány fogyasztó figyeli.
- **Nem elutasítható:** az esemény befogadása nem jelenti azt, hogy a fogyasztó egyetért a tényel; a tény akkor is tény, ha a fogyasztó hibásan kezeli.
- **Immutable:** az esemény adatai az esemény kiadása után nem változtathatók meg. A fogyasztók az eseményt mint időpillanatbeli tényt tárolhatják vagy dolgozhatják fel.

Az esemény a forrásoldali **igazságforrás** is lehet, ha a kiadó az állapotát eseményekből vezeti le (Event Sourcing, CQRS és Event Sourcing). A fejezet szempontjából elegendő annyi, hogy az esemény a kiadó szerint hiteles: a kiadó az eseményben azt mondja el, hogy az ő állapotában milyen változás történt, és ezt a változást azonosítható, időbélyegzett módon rögzíti.

Példa esemény-szerződésre:

```
{
  "type": "event",
  "name": "OrderPlaced",
  "schema_version": "1.0",
  "event_id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "aggregate_id": "order-987",
  "customer_id": "cust-123",
  "items": [
    {"product_id": "prod-1", "quantity": 2}
  ],
  "placed_at": "2025-01-15T10:00:00.123Z",
  "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
  "causation_id": "550e8400-e29b-41d4-a716-446655440000"
}
```

Az esemény azonosítója (`event_id`) egyedi, és a fogyasztói oldal ezt használja deduplikációra, ha az üzenettovábbítás at-least-once garanciát ad. A `correlation_id` gyakran a kiváltó parancs azonosítója, a `causation_id` pedig az ok-okozati lánc eleme. Az `aggregate_id` a domain-objektum azonosítója, amelyhez az esemény tartozik; ez a mező később az eseményfolyam particionálásánál is fontos.

13.1.5 4. Szemantikai összehasonlítás

A parancs és az esemény közötti különbséget a következő táblázat foglalja össze:

Szempont	Parancs	Esemény
Időbeliség	Szándék, jövőre irányul	Tény, múltra vonatkozik
Név	Felszólító mód (`PlaceOrder`)	Múlt idő (`OrderPlaced`)
Címzett	Egy konkrét végrehajtó	Több fogyasztó is lehet
Elutasíthatóság	Igen, a fogyasztó elutasíthatja	Nem, a tény nem utasítható el
Validáció	A fogyasztónak ellenőriznie kell	A kiadó már érvényesnek tekinti
Állapot	Nem tartalmaz teljes állapotot	A változás tényét tartalmazza
Idempotencia	Deduplikáció `command_id` alapján	Deduplikáció `event_id` alapján
Felelősség	A fogyasztó felel a végrehajtásért	A kiadó felel a tény hitelességéért

A táblázatból látható, hogy a két üzenetfajta eltérő hibatűrési stratégiát igényel. Egy parancs elvesztése esetén a fogyasztó soha nem hajtja végre a műveletet, ami üzleti hibához vezethet. Egy esemény elvesztése esetén a fogyasztó nem értesül a tényről, ami a projekciók, a riportok vagy a downstream folyamatok inkonzisztenciáját okozhatja. Mindkét esetben a kézbesítési garanciák és a deduplikáció kritikus.

13.1.6 5. Azonosítás, sorrend és kézbesítés

Az üzenetek azonosítása, sorrendje és kézbesítési garanciája mind a parancsok, mind az események esetében alapvető tervezési szempont. A fejezet itt csak a legalapvetőbb fogalmakat vezeti be; a részletes outbox- és saga-mintákat a Saga, outbox és elosztott tranzakciók tárgyalja.

13.1.6.1 Azonosítás

Minden üzenetnek egyedi azonosítóval kell rendelkeznie. A parancs esetében ez a `command_id`, az eseménynél az `event_id`. Ezen felül érdemes a következő azonosítókat megkülönböztetni:

- **`message_id`:** az üzenet technikai azonosítója, amely az üzenettovábbítási rétegben egyedi (például a bróker által generált azonosító). Nem feltétlenül üzleti jelentőségű.
- **`correlation_id`:** az üzleti folyamatot átfogó azonosító, amely összeköti az adott folyamathoz tartozó üzeneteket. Például egy `PlaceOrder` parancs `command_id`-je megjelenhet `correlation_id`-ként az általa kiváltott eseményeken.
- **`causation_id`:** az ok-okozati lánc közvetlen előzményére mutat. Ez lehetővé teszi az események oksági láncának nyomon követését, ami hibakeresésnél és auditnál hasznos.
- **`aggregate_id`:** az üzleti entitás azonosítója, amelyhez az esemény vagy parancs tartozik. Ez az azonosító a streaming platform particionálásánál is kulcs szerepet játszik, mert az azonos aggregátumhoz tartozó eseményeket ugyanabba a partícióba kell irányítani a sorrend biztosításához.

13.1.6.2 Sorrend

Az üzenetek sorrendje két szinten értelmezhető: globálisan és per aggregate. A globális sorrend a teljes rendszerre vonatkozó üzenetsorrend, amelyet elosztott környezetben általában nem lehet garantálni; a per aggregate sorrend viszont azt jelenti, hogy egy adott üzleti entitáshoz (például egy rendeléshez) tartozó események érkezési sorrendje megegyezik a kiadási sorrenddel.

A gyakorlatban az eseményeknél a per aggregate sorrend a kritikus, mert a fogyasztó az események sorozatából állítja elő a projekciót. Ha az események sorrendje felcserélődik, a fogyasztó rossz állapotot számolhat. A streaming platformok, például a Kafka, az azonos particionálási kulcsot használó üzeneteket ugyanabba a partícióba írják, és ott megőrzik a sorrendet. Ezért az `aggregate_id`-t gyakran particionálási kulcsként használják.

A parancsok esetében a sorrend kérdése árnyaltabb. Egy parancs önmagában nem hordozza a rendszer állapotát, ezért a fogyasztónak nem feltétlenül kell az összes parancsot pontosan a kiadási sorrendben feldolgoznia. Azonban ha a parancsok egyazon üzleti entitást érintenek, a sorrend megváltozása hibás eredményhez vezethet. Például egy `UpdateQuantity` és egy `DeleteItem` parancs sorrendje nem cserélhető fel. Ilyen esetekben a parancsokat is per aggregate kell rendezni.

13.1.6.3 Kézbesítési garanciák

Az üzenettovábbítás három alapvető garanciát adhat:

- **At-most-once:** az üzenet legfeljebb egyszer érkezik meg, tehát elveszhet, de nem duplikálódik. Ez ritkán elegendő.
- **At-least-once:** az üzenet legalább egyszer megérkezik, tehát elveszni nem fog, de duplikálódhat. Ez a leggyakoribb gyakorlati választás, mert a fogyasztót idempotensre tervezik.
- **Exactly-once:** az üzenet pontosan egyszer érkezik meg. Ez a gyakorlatban általában úgy érhető el, hogy az at-least-once kézbesítést egy idempotens fogyasztóval kombinálják, vagy a platform tranzakciós API-ját használják a fogyasztói és a termelői oldal között.

Az idempotens fogyasztó a deduplikációt az üzenetazonosító alapján végzi. Például a fogyasztó minden feldolgozott `event_id`-t vagy `command_id`-t eltárol egy `processed_messages` táblában, és mielőtt végrehajtja az üzleti logikát, ellenőrzi, hogy az adott azonosító már szerepel-e. Ha igen, akkor a feldolgozást kihagyja, de a nyugtát elküldi, hogy a bróker előre léphessen.

Itt érdemes röviden előrevetíteni az outbox-mintát (Saga, outbox és elosztott tranzakciók), mert az idempotencia és a kézbesítés itt szorosan kapcsolódik: ha egy szolgáltatás tranzakcióban módosítja a saját adatbázisát és eseményt is publikál, akkor a kettőt atomi módon kell összekapcsolni. Az outbox tábla éppen ezt teszi lehetővé: az eseményt az adatbázis-tranzakció részeként írja ki, majd egy háttérfolyamat továbbítja a brókernek. A fejezet ezt csak megemlíti, a részletes tárgyalás a Saga, outbox és elosztott tranzakciók alfejezetben következik.

13.1.7 6. Gyakorlati szerződéstervezés

A szemantikai különbségek akkor válnak kézzelfoghatóvá, ha egy konkrét üzleti folyamatot modellezünk. Vegyünk egy egyszerű rendelésfolyamatot:

1. A vevő leadja a rendelést, ami egy `PlaceOrder` parancsot generál.
2. A rendelés-szolgáltatás validálja a parancsot, létrehozza a rendelést, majd publikálja az `OrderPlaced` eseményt.

3. A készletkezelő szolgáltatás meghallgatja az `OrderPlaced` eseményt, és lefoglalja a készletet. Ha a foglalás sikeres, `InventoryReserved` eseményt ad ki; ha nem, akkor `OrderCancelled` eseményt.

Ezt a folyamatot Mermaid szekvenciadiagrammal szemléltethetjük. A diagram címe: „Rendelés parancs feldolgozása és esemény publikálása”. Jelmagyarázat: a folytonos nyíl szinkron hívást, a szaggatott nyíl aszinkron üzenetet jelöl.

```
sequenceDiagram
    participant C as Ügyfél
    participant R as Rendelés-szolgáltatás
    participant DB as Rendelés-adatbázis
    participant E as Eseménytár

    C->>R: PlaceOrder parancs
    R->>R: parancs validálása és deduplikáció
    R->>DB: rendelés létrehozása tranzakcióban
    DB-->>R: visszaigazolás
    R->>E: OrderPlaced esemény publikálása
    E-->>R: nyugta
    R-->>C: parancs elfogadva
```

Feltételjegyzet: a diagram at-least-once kézbesítést feltételez, a deduplikációt a rendelés-szolgáltatás a `command_id` alapján végzi.

A szerződések tervezésénél a következő elveket érdemes követni:

- **Típusmező:** minden üzenet fejlécében szerepeljen a `type`, amely az üzenet típusát (`command` vagy `event`) és a nevét (például `OrderPlaced`) azonosítja.
- **Sémaverzió:** a törzs adatszerkezete verziózott legyen, például `schema_version` mezővel. A séma evolúcióját az API-verziózás, szerződés és schema evolution tárgyalja, de már most meg kell tervezni, hogy a mezők hozzáadása visszafelé kompatibilis legyen.
- **Azonosítók:** parancsnál `command_id`, eseménynél `event_id`, valamint `aggregate_id`, `correlation_id` és `causation_id`, ahol értelmezhető.
- **Időbélyeg:** minden üzenet tartalmazzon a kiadás vagy beküldés pillanatára vonatkozó időbélyeget, lehetőleg UTC-ben.
- **Minimális adat:** a parancs csak a művelethez szükséges adatokat tartalmazza, az esemény pedig a tény rögzítéséhez szükséges adatokat, nem a teljes belső állapotot.

A fenti rendelésfolyamathoz tartozó parancs- és esemény-szerződéseket JSON formában mutatjuk be. A példák szemléltető jellegűek, nem éles rendszeren végrehajtandók.

`PlaceOrder` parancs:

```
{
  "type": "command",
  "name": "PlaceOrder",
  "schema_version": "1.0",
  "command_id": "550e8400-e29b-41d4-a716-446655440000",
  "customer_id": "cust-123",
  "items": [
    {"product_id": "prod-1", "quantity": 2}
  ],
  "submitted_at": "2025-01-15T10:00:00Z"
}
```

`OrderPlaced` esemény:

```
{
  "type": "event",
  "name": "OrderPlaced",
  "schema_version": "1.0",
  "event_id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "aggregate_id": "order-987",
  "customer_id": "cust-123",
  "items": [
    {"product_id": "prod-1", "quantity": 2}
  ],
  "placed_at": "2025-01-15T10:00:00.123Z",
  "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
  "causation_id": "550e8400-e29b-41d4-a716-446655440000"
}
```

A `correlation_id` itt a `PlaceOrder` parancs `command_id`-jével egyezik meg, ami lehetővé teszi a teljes folyamat nyomon követését. A `causation_id` ugyancsak a parancsazonosító, mert az esemény közvetlen oka a parancs feldolgozása volt.

13.1.8 7. Tipikus hibák és megoldások

A gyakorlatban gyakran előfordulnak olyan tervezési hibák, amelyek a parancs és az esemény szemantikájának összemosásából fakadnak. Az alábbiakban a leggyakoribbakat mutatjuk be, mindegyiknél rövid indoklással és helyes megoldással.

13.1.8.1 7.1 Parancs eseményként való publikálása

Hibás minta: a szolgáltatás egy `PlaceOrder` parancsot kap, majd azt változtatás nélkül `PlaceOrder` eseményként publikálja, abban a hiszemben, hogy ezzel „tájékoztatja” a többi szolgáltatást a szándékról.

Miért hibás? Az esemény jelentése szerint a tény már megtörtént. Ha a rendelés-szolgáltatás még nem hozta létre a rendelést, de `PlaceOrder` eseményt ad ki, a fogyasztók tévesen azt állapíthatják meg, hogy a rendelés létezik. Ez inkonzisztens állapothoz vezet: a készletkezelő lefoglalja a készletet egy még nem létező rendeléshez.

Helyes megoldás: a parancs feldolgozása után a szolgáltatás a tényleges állapotváltozást tükröző eseményt adjon ki, például `OrderPlaced`. A parancs nem válik eseménnyé, hanem az esemény a parancs sikeres végrehajtásának eredménye.

13.1.8.2 7.2 Esemény parancsként való fogyasztása

Hibás minta: egy fogyasztó az `OrderPlaced` eseményt úgy kezeli, mintha az egy parancs lenne, amely arra utasítja, hogy hajtson végre valamilyen műveletet, például „indítsd el a fizetési folyamatot”. A fogyasztó nem validál, nem utasít el, csak végrehajt.

Miért hibás? Az esemény nem tartalmazza a fogyasztó felelősségi körének megfelelő üzleti feltételeket; a fogyasztó így rejtett, implicit csatolást alakít ki. Ha a kiadó megváltoztatja az esemény sémáját, a fogyasztó törékennyé válik. Ezenkívül a fogyasztó nem tud elutasítani egy tényt, ezért hibás állapotba kerülhet.

Helyes megoldás: a fogyasztó az eseményt tényként dolgozza fel, és a saját belső logikája dönti el, hogy milyen műveletet hajt végre. Ha a folyamat folytatásához szükség van egy újabb szándékra, akkor a fogyasztó saját maga állít elő egy parancsot, vagy a folyamatot egy orchestrátor vezérli.

13.1.8.3 7.3 Többcélú üzenet: egyszerre parancs és esemény

Hibás minta: egy üzenet típusa `OrderCreatedOrCreateOrder`, amely egyszerre hordozza a parancs és az esemény szemantikáját. A fogyasztók maguk döntik el, hogy melyik jelentést használják.

Miért hibás? Az ilyen üzenet kétértelműséget szül, mert a fogyasztó nem tudja biztosan, hogy a műveletet végre kell hajtania, vagy csak értesülnie kell egy tényről. Ez növeli a tesztelési és karbantartási költséget, és megnehezíti a verziózást.

Helyes megoldás: a parancsot és az eseményt külön típusokban definiáljuk, még akkor is, ha a törzsük nagyon hasonló. A típusmező egyértelműen elválasztja a kettőt, és a fogyasztó a típus alapján dönti el a viselkedést.

13.1.8.4 7.4 Nem idempotens fogyasztó

Hibás minta: a fogyasztó at-least-once kézbesítés mellett úgy dolgozza fel az eseményeket, hogy nem végez deduplikációt. Ugyanaz az `OrderPlaced` esemény kétszer érkezik meg, és a készletkezelő kétszer foglalja le a készletet.

Miért hibás? Az at-least-once kézbesítésnél a duplikáció nem kivétel, hanem várható esemény. A fogyasztónak ezért idempotensnek kell lennie: ugyanaz az üzenetazonosító ne okozzon ismételt üzleti hatást.

Helyes megoldás: a fogyasztó az üzenetazonosítót (`event_id` vagy `command_id`) tárolja egy deduplikációs táblában, és a feldolgozás előtt ellenőrzi, hogy az azonosító már szerepel-e. Ha igen, akkor a feldolgozást kihagyja, de a nyugtát elküldi.

13.1.8.5 7.5 Globális sorrend feltételezése

Hibás minta: a fejlesztő feltételezi, hogy az események globálisan, pontosan a kiadási sorrendben érkeznek minden fogyasztóhoz. Ez a feltételezés elosztott rendszerben általában hamis.

Miért hibás? A globális sorrend csak akkor garantálható, ha a rendszer egyetlen partíciót használ, ami súlyos skálázhatósági korlát. Ha a rendszer több partíciót használ, a fogyasztóhoz érkező események sorrendje az ütemezéstől és a hálózattól függően változhat.

Helyes megoldás: a sorrendi garanciákat per aggregate kell értelmezni. Az `aggregate_id`-t használjuk particionálási kulcsként, így az azonos entitáshoz tartozó események ugyanabban a partícióban, sorrendben maradnak. A fogyasztó csak az adott aggregátumon belül feltételez sorrendet.

13.1.9 8. Labor: esemény- és parancs-szerződés tervezése

Ez a labor a fejezet fogalmait alkalmazza egy kis, ellenőrizhető feladaton keresztül. A labor célja, hogy a hallgató saját kezűleg készítse el egy parancs- és esemény-szerződés dokumentációját, ADR-rel és ellenőrzési szempontokkal együtt.

13.1.9.1 Kiinduló állapot

Válassz egy egyszerű üzleti folyamatot a következő példák közül, vagy válassz sajátot:

- Fiókregisztráció és `AccountCreated` esemény.
- Kosárkezelés: `AddItemToCart` parancs és `ItemAddedToCart` esemény.
- Foglaláskezelés: `ReserveRoom` parancs és `RoomReserved` esemény.

A labor nem igényel futó infrastruktúrát; a dokumentum szöveges és diagramos formában készül, de a sémák JSON Schema vagy hasonló leíró nyelvvel validálhatók legyenek.

13.1.9.2 Elvárt eredmény

Készíts egy reprodukálható dokumentumot, amely a következőket tartalmazza:

1. **ADR** (egyetlen ADR elegendő) arról, hogy a választott folyamatban miért választottad külön a parancsot és az eseményt. Az ADR formátuma: kontextus, döntés, alternatívák, következmények, állapot.
2. **Legalább két parancs és két esemény szerződését** JSON vagy YAML formában, a fejezetben bemutatott mezőkészlettel: típus, név, sémaverzió, azonosító, aggregátum azonosító, időbélyeg, korrelációs azonosító, oksági azonosító. A parancsok felszólító módúak, az események múlt idejűek.
3. **Egy Mermaid szekvenciadiagramot**, amely a parancs feldolgozását és az esemény publikálását mutatja. A diagramnak legyen címe, jelmagyarázata és feltételjegyzete.
4. **Deduplikációs tervet**, amely leírja, hogy a fogyasztó hogyan biztosítja az idempotens feldolgozást at-least-once kézbesítés mellett. Például: a fogyasztó tárolja az `event_id`-kat egy `processed_events` táblában, és feldolgozás előtt ellenőrzi.
5. **Ellenőrzési szempontokat**, amelyek alapján a dokumentum önmagát ellenőrzi (lásd alább).

13.1.9.3 Ellenőrzési szempontok

Az ellenőrzés során a következő kérdésekre kell igennel válaszolni:

- A parancs neve felszólító módú, az esemény neve múlt idejű.
- Minden parancshoz tartozik `command_id`, minden eseményhez `event_id`.
- Minden parancs és esemény tartalmazza a `schema_version` mezőt.
- Az események tartalmazzák az `aggregate_id`-t, és az aggregátumazonosító a teljes folyamatban konzisztens.
- A parancs nem tartalmazza a rendszer állapotát (például nem tartalmazza a rendelés aktuális állapotát vagy a teljes készletmennyiséget).
- A deduplikációs terv egyértelműen leírja az idempotens feldolgozást, beleértve az azonosítók tárolását.
- Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.
- A Mermaid diagram szintaktikailag helyes, és a szöveghez igazodik.

13.1.9.4 Visszaállítási út

Mivel a labor dokumentumalapú, nincs szükség speciális visszaállítási műveletre. Ha a hallgató valamelyik lépést hibásan hajtja végre, a dokumentumot a verziókezelő rendszerben visszaállíthatja, vagy a szerződéseket újra létrehozhatja. A labor nem éles rendszert módosít, és nem igényel izolált környezetet, de a hallgatónak kerülnie kell valós ügyféladatok használatát; minden azonosító legyen szintetikus.

13.1.9.5 Mért és várt eredmény elkülönítése

A labor során a hallgató a dokumentumot elkészíti, és az ellenőrzési szempontok szerint önértékeli. A várt eredmény: minden ellenőrzési szempont teljesül, a sémák egymással konzisztensek, és az ADR logikusan indokolja a választást. A mért eredményt a hallgató maga rögzíti, például: „az ellenőrzési lista 8/8 szempontja teljesült”, vagy „a JSON Schema validálás sikeres volt”. A mért eredmény nem helyettesíti a várt eredményt, és a nem futtatott ellenőrzést nem szabad ellenőrzött eredményként bemutatni – ezt a dokumentációban külön kell jelezni.

13.1.10 9. Önellenőrző kérdések

A fejezet lezárásaként ellenőrizd a megértésedet az alábbi kérdésekkel:

1. Mi a szemantikai különbség a parancs és az esemény között? Fogalmazd meg a saját szavaiddal, legalább három szempont alapján.
2. Melyik azonosítókat használod a deduplikációhoz, és miért nem elegendő a `message_id`?
3. Hogyan biztosítod az idempotens feldolgozást at-least-once kézbesítésnél? Írd le a lépéseket.
4. Miért nem érdemes egy üzenetet egyszerre parancsnak és eseménynek definiálni? Adj egy konkrét példát a hibás következményre.
5. Mit jelent a per aggregate sorrend, és hogyan segíti az `aggregate_id` a streaming platform particionálását?

13.1.11 10. Összefoglalás és továbbvitel

Ebben a fejezetben elkülönítettük az üzenet technikai hordozó szerepét a parancs szándékától és az esemény tényállításától. A parancs arra utasít, hogy valami történjen; az esemény azt mondja el, hogy valami már megtörtént. Ez a különbség nem stilisztikai kérdés, hanem a felelősségi határokat, a hibatűrést és a csatolást meghatározó architekturális döntés.

A fejezetben bevezetett azonosítási, sorrendi és kézbesítési elvek közvetlenül épülnek be a következő egységekbe. A CQRS és Event Sourcing a CQRS-t és az Event Sourcingot tárgyalja, ahol a parancs a írási modellhez, az esemény pedig az olvasási modellhez és az állapot-rekonstrukcióhoz kapcsolódik. A Saga, outbox és elosztott tranzakciók a saga-, outbox- és elosztott tranzakciómintákat mutatja be, amelyek a parancsok és események közötti oksági láncot és az atomi publikálást használják. A most rögzített szemantikai alap nélkül ezek a minták könnyen félreérthetők, ezért érdemes időt szánni a labor gondos elvégzésére.

13.2 CQRS és Event Sourcing

A hagyományos, egyetlen adatmodellre épülő alkalmazásokban az írási és olvasási műveletek gyakran ugyanazt az entitásmodellt, ugyanazokat az adattáblákat és ugyanazokat a mezőket használják. Ez elsőre egyszerűnek tűnik, de ahogy a rendszer növekszik, az olvasási oldal egyre jobban eltér az írási oldaltól: más a képernyő, más a lekérdezés, más a teljesítményprofil. Az egyetlen modell kénytelen egyszerre kiszolgálni a tranzakciós integritást igénylő parancsokat és a sok összekapcsolt, szűrt, rendezett, denormalizált lekérdezést. Az eredmény gyakran kompromisszumos séma, lassú joinok, nehezen skálázható írási és olvasási útvonal, valamint olyan üzleti logika, amely a perzisztencia részleteibe keveredik.

A CQRS és az Event Sourcing ezt a feszültséget oldja fel azzal, hogy a felelősségeket szétválasztja, és az állapotváltozást nem csak a végeredmény, hanem a változást leíró események sorozataként is kezeli. Ebben az alfejezetben a két mintát együtt, mégis külön-külön is értelmezhetően mutatjuk be. Nem ismételjük meg az előző alfejezetben bevezetett esemény-, üzenet- és parancsfogalmak alapjait, és nem térünk ki a saga, outbox és elosztott tranzakciók részletes mechanizmusára, mivel azok a következő alfejezet tárgyát képezik. A hangsúly itt az olvasási és írási modellek szétválasztásán, az eseménytörténeten, valamint a visszajátszás, a pillanatkép és a projekció működésén van.

13.2.1 Miért érezzük szűknek az egy modellt?

Képzeljünk el egy rendeléskezelő alkalmazást, ahol a rendelés entitás tartalmazza a vevőt, a tételeket, a státuszt és a fizetési adatokat. Az írási oldalon a rendelés leadásakor szigorú üzleti szabályokat kell ellenőrizni: például nem lehet üres rendelést leadni, és nem lehet tételt hozzáadni egy már leadott rendeléshez. Az olvasási oldalon ugyanakkor a call center munkatársa más nézetet szeretne látni: vevőnév, rendelés összege, státusz, legutóbbi módosítás időpontja, esetleg egy összesített lista a mai kifizetetlen rendelésekről. Az ügyfél mobilalkalmazása megint más nézetet igényel: csak a saját aktív rendeléseit, tételsorok nélkül, de képpel és szállítási címmel.

Ha mindezt egyetlen táblából és egyetlen REST-végpontból próbáljuk kiszolgálni, három tipikus tünetet tapasztalunk:

1. **Séma kompromisszumok:** az írási integritáshoz normalizált táblák kellenek, az olvasási teljesítményhez viszont denormalizált, előre kiszámolt vetületek.
2. **Skálázási aszimmetria:** az írási terhelés kicsi, de tranzakciós; az olvasási terhelés nagy, de többnyire csak olvas. A közös adatbázis mindkettőt egyszerre korlátozza.
3. **Üzleti logika szétkenése:** a parancsok érvényesítése, az állapotátmenetek és a lekérdezési DTO-k összekeverednek, így a rendszer nehezen módosítható.

A megoldás nem az, hogy minden alkalmazást szét kell szedni, hanem az, hogy a modellhatárokat a valódi használathoz igazítsuk. Erre ad keretet a CQRS.

13.2.2 CQRS: a parancs- és lekérdezési felelősség szétválasztása

A CQRS rövidítés a *Command Query Responsibility Segregation* kifejezést takarja: a parancsok és a lekérdezések felelősségének szétválasztását. Az alapötlet egyszerű: más modell szolgálja ki az állapotot megváltoztató parancsokat, és más modell a kérdéseket. A két modell közös forrásból származhat, de az adatszerkezetük, az optimalizációjuk és az életciklusuk eltérhet.

Fontos különbség a klasszikus CQS (Command Query Separation) elv és a CQRS között. A CQS egy metódus szintjén mondja ki, hogy egy metódus vagy parancsot végrehajt, vagy értéket

kérdez le, de ne tegye egyszerre mindkettőt. A CQRS ezt az elvet rendszerarchitektúra szintre emeli: külön írási és külön olvasási modellt, akár külön adattárolókat és külön komponenseket vezet be. A CQRS tehát nem egyszerű kódolási stílus, hanem architekturális döntés.

13.2.3 Az írási oldal: parancsmodell

Az írási oldal a parancsokat fogadja. A parancs egy szándék kifejezése, például `SubmitOrder`, `AddOrderItem` vagy `MarkPaymentReceived`. A parancs imperatív jellegű, elutasítható, és nem feltétlenül vált ki állapotváltozást. A parancsmodell feladata, hogy:

- érvényesítse a parancsot a jelenlegi állapot alapján;
- betartsa az üzleti invariánsokat;
- eldöntse, hogy a parancs végrehajtható-e;
- létrehozza a megfelelő eseményeket.

A parancsmodell központi építőköve gyakran a **domain aggregátum**. Az aggregátum egy konzisztenciahatár: az azonos aggregátumhoz tartozó állapotot együtt kell módosítani, és az üzleti szabályokat itt kell érvényesíteni. Egy rendelés aggregátum például felel azért, hogy csak `Draft` állapotban lehessen tételt hozzáadni, és hogy üres rendelést ne lehessen leadni.

A parancsmodell tipikusan nem közvetlenül az adatbázis rekordjait módosítja, hanem a döntést követően eseményeket bocsát ki. Ez a döntés vezet át az Event Sourcinghoz. CQRS-t azonban Event Sourcing nélkül is lehet használni: az írási modell akár hagyományos tranzakciós adatbázisba is írhat, az olvasási modellt pedig szinkron vagy aszinkron módon tarthatjuk karban.

13.2.4 Az olvasási oldal: projekció és anyagiasított nézet

A lekérdezési oldal az olvasási modellt használja. Az olvasási modell nem az írási modell másolata, hanem az adott lekérdezésre optimalizált, gyakran denormalizált, előre kiszámolt nézet. A rendeléskezelő rendszerben például három olvasási modell is létezhet:

- **Rendelés lista:** a call center számára, amely a mai rendeléseket mutatja státusszal és összeggel.
- **Rendelés részletek:** egy adott rendelés minden tételével, vevővel és fizetési adattal.
- **Vevő aktív rendelései:** a mobilalkalmazás számára, csak a bejelentkezett vevő aktív rendelései.

Ezeket az olvasási modelleket **projekció** hozza létre. A projekció az írási oldalról érkező eseményekből vagy változásokból építi fel, frissíti a lekérdezésre optimalizált nézetet. A projekció lehet szinkron, például ugyanabban a tranzakcióban, de gyakran aszinkron: az esemény beérkezése után egy háttérfolyamat frissíti a nézetet. A projekció eredményét szokás **anyagiasított nézetnek** is nevezni, mert a lekérdezés idején már kiszámolt, eltárolt adatot olvasunk, nem pedig minden kérésnél újra összeállítjuk a forrásadatokból.

A projekció fontos jellemzője, hogy determinisztikusnak kell lennie: ugyanabból az eseménysorból ugyanazt a nézetet kell előállítania. Ez teszi lehetővé a nézet újraépítését, a hibajavítást és a tesztelést.

13.2.5 Konzisztencia az olvasási modellben

A CQRS bevezetésekor a leggyakoribb félreértés az erős konzisztencia elvárása. Ha az írási és olvasási modell külön adattárolóban él, akkor a kettő között **eventual consistency** lép fel: az olvasási modell egy rövid ideig még a régi állapotot mutathatja, miután a parancs már sikeresen lefutott. Ennek mértékét nem hallgathatjuk el, hanem mérhetővé kell tenni.

Egy minőségi forgatókönyv például így fogalmazható meg:

> **Inger:** a `PaymentReceived` esemény beérkezik az eseménytárból a projekcióba.
> **Környezet:** normál üzemi terhelés, a rendszer minden komponense egészséges.
> **Válasz:** a „fizetetlen rendelések” olvasási modell eltávolítja a rendelést, a „rendelés részletek” nézetben a státusz `Paid` értékre vált.
> **Mérőszám:** az olvasási modell frissességi késleltetése p95 < 2 s, a mérési ablak 5 perc.

Ez a forgatókönyv rámutat arra, hogy a CQRS nem tünteti el a konzisztenciaproblémát, hanem explicitté teszi és mérhetővé alakítja. Azt is jelzi, hogy a felhasználói élményt meg kell tervezni: például saját írás után egy rövid ideig a régi állapot jelenhet meg, ezért az API választhat olyan megoldást, hogy a parancs végrehajtása után közvetlenül az írási modellből adja vissza a friss állapotot, vagy a kliens elfogad egy rövid késleltetést.

13.2.6 Event Sourcing: az eseménytörténet mint elsődleges forrás

Az Event Sourcing az állapotváltozások rögzítésének egy másik modellje. Ahelyett, hogy a rendelés aktuális állapotát tárolnánk egy sorban, és azt felülírnánk, minden változást egy **append-only** eseménysorozatban rögzítünk. Az aktuális állapot ebből az eseménytörténetből visszajátszással számítható ki.

A központi gondolat: az esemény nem a változás melléktermé ke, hanem az igazság elsődleges forrása. Az esemény megtörtént tény, múlt idejű megfogalmazással: `OrderCreated`, `OrderItemAdded`, `OrderSubmitted`, `PaymentReceived`. Az eseményt nem lehet törölni vagy módosítani, legfeljebb új eseménnyel lehet kompenzálni.

Az eseménytároló (event store) az aggregátumok azonosítója szerint rendezett eseménystreameket kezel. Minden esemény rendelkezik:

- `aggregateId`: melyik aggregátumhoz tartozik;
- `version`: az aggregátumon belüli sorszám, amely egyidejűség-ellenőrzésre is szolgál;
- `occurredAt`: az esemény bekövetkezésének időpontja;
- `correlationId`: a kiváltó parancs azonosítója, amely a nyomon követést segíti;
- `payload`: az eseményre jellemző adat.

Egy lehetséges esemény JSON formátumban:

```
{
  "type": "OrderSubmitted",
  "aggregateId": "order-123",
  "version": 3,
  "occurredAt": "2025-04-10T10:15:30Z",
  "correlationId": "cmd-456",
  "payload": {
    "submittedAt": "2025-04-10T10:15:29Z"
  }
}
```

Az eseménytár csak hozzáfűzést enged, ezért az eseménytörténet auditálható, visszakereshető, és időben visszafelé is elemezhető. Az aktuális állapot csupán egy levezetett érték, amely a `fold` vagy `reduce` művelettel állítható elő.

13.2.7 Visszajátszás

A visszajátszás az a folyamat, amelynek során az aggregátum aktuális állapotát az események sorozatából számítjuk ki. Az algoritmus elve:

```
function load(aggregateId):
    events = eventStore.read(aggregateId)
    state = emptyState(aggregateId)
    for each event in events:
        state = apply(state, event)
    return state
```

Egy rendelés aggregátum esetén az `apply` függvény például így viselkedhet:

```
function apply(state, event):
    case event.type of
        "OrderCreated":
            state.id = event.aggregateId
            state.customerId = event.payload.customerId
            state.status = "Draft"
            state.items = []
        "OrderItemAdded":
            state.items.append(event.payload.item)
        "OrderSubmitted":
            state.status = "Submitted"
        "PaymentReceived":
            state.status = "Paid"
    return state
```

Ez a pszeudokód szemlélteti, hogy az állapot nem tárolt mező, hanem az események következménye. Ugyanezt a visszajátszást használhatjuk az olvasási modellek projekcióinál is, csak ott az `apply` nem az aggregátum állapotát, hanem a lekérdezési nézetet frissíti.

A visszajátszás előnye, hogy bármikor újra lefuttatható, és pontosan ugyanazt az állapotot adja vissza, feltéve, hogy az `apply` függvény determinisztikus, és az események sorrendje rögzített. Ez az újraépíthetőség alapja.

13.2.8 Pillanatkép

A hosszú eseménystreamek visszajátszása minden kérésnél költséges lehet. Egy több ezer eseményt tartalmazó rendelés esetén a betöltés lassulhat. Ennek kezelésére **pillanatképet** készítünk: időnként eltároljuk az aggregátum aktuális állapotát és az utolsó feldolgozott esemény verzióját. A betöltés ekkor a pillanatképből indul, és csak az azóta beérkezett eseményeket játssza vissza.

A pillanatképes betöltés vázlata:

```
function loadWithSnapshot(aggregateId):
    snapshot = snapshotStore.latest(aggregateId)
    events = eventStore.readAfter(aggregateId, snapshot.version)
    state = snapshot.state
    for each event in events:
        state = apply(state, event)
    return state
```

A pillanatkép nem helyettesíti az eseménytörténetet, csupán gyorsítótárazza a visszajátszás egy közbülső állapotát. A teljes eseménytörténet továbbra is megmarad, ezért a pillanatkép bármikor eldobható és újraépíthető. A pillanatkép készítésének gyakorisága trade-off: a túl ritka pillanatkép lassú betöltést, a túl gyakori pedig nagyobb tárolási és írási költséget jelent.

13.2.9 Projekció és olvasási modell

A projekció az eseménytörténetből olvasási modellt épít. Az olvasási modell nem az aggregátum állapota, hanem egy lekérdezésre szabott vetület. Például a „rendelés lista” nézet nem tárolja a tételek teljes részleteit, csak a rendelés azonosítóját, a vevő nevét, az összeget és a státuszt. A projekció ezt a nézetet az eseményekből számítja ki:

```
function project(events):
    view = emptyOrderListView()
    for each event in events:
        case event.type of
            "OrderCreated":
                view.insert(orderId, customerId, total = 0, status = "Draft")
            "OrderItemAdded":
                view[event.aggregateId].total += event.payload.item.price *
↪   event.payload.item.quantity
            "OrderSubmitted":
                view[event.aggregateId].status = "Submitted"
            "PaymentReceived":
                view[event.aggregateId].status = "Paid"
    return view
```

A projekció gyakran aszinkron módon fut: az eseménytárból kiolvassa az új eseményeket, és frissíti az olvasási modellt. A feldolgozásnak **idempotensnek** kell lennie, mert a hálózati hiba miatt ugyanaz az esemény többször is kézbesíthető. Az idempotens frissítés például azt jelenti, hogy a `PaymentReceived` esemény kétszeri feldolgozása nem okoz dupla státuszváltást.

A projekció újraépíthetősége fontos előny: ha a nézet sérül, vagy új lekérdezési igény jelenik meg, a teljes eseménytörténetből újra lefuttathatjuk a projekciót. Ehhez azonban a projekciós logikának is determinisztikusnak kell lennie, és az események sorrendjét konzisztensen kell kezelni.

13.2.10 Parancs-, esemény- és olvasási modell kapcsolata

A három modell elkülönül, de együttműködik. A parancsmodell fogadja a parancsot, elvégzi az üzleti ellenőrzést, és eseményeket bocsát ki. Az eseménytár rögzíti az eseményeket. A projekció az eseményekből frissíti az olvasási modellt. Az olvasási modell pedig kiszolgálja a lekérdezéseket.

A folyamatot egy Mermaid szekvenciadiagram szemlélteti:

```
sequenceDiagram
    participant U as Ügyfél
    participant C as Parancsmodell / OrderAggregate
    participant E as Eseménytár
    participant P as Projektor
    participant Q as Rendelés lista olvasási modell
    U->>C: SubmitOrder parancs
    C->>E: OrderSubmitted (version=3) hozzáfűzése
    E-->>P: Esemény stream (OrderSubmitted)
    P->>Q: Rendelés státusz frissítése
    U->>Q: GET /orders?status=submitted
    Q-->>U: Rendelés lista DTO
```

A diagramon a parancsmodell és az eseménytár közötti kapcsolat szinkron: a parancs végrehajtása megvárja az esemény tartós rögzítését. Az eseménytárból a projekció felé már aszinkron adatfolyam jellemző: a projekció késleltetve dolgozza fel az eseményeket. A lekérdezési oldal és a felhasználó közötti kommunikáció szinkron: a kliens kérésre azonnal választ kap, de a válasz a projekció aktuális állapotát tükrözi, amely egy korábbi időpontnak felelhet meg.

A parancsmodell és az olvasási modell közötti aszinkronitás miatt a CQRS Event Sourcinggal kombinálva gyakran **eventual consistency**-t eredményez. Ezt a tényt minden architektúra-dokumentumban rögzíteni kell, és a minőségi forgatókönyvben mérőszámmal kell ellátni.

13.2.11 Mikor válasszuk a CQRS-t és az Event Sourcingot?

A CQRS és az Event Sourcing erős eszköz, de nem univerzális. A bevezetés indokolt, ha:

- az olvasási és írási terhelés jelentősen eltér, és külön kell skálázni őket;
- a lekérdezési nézetek összetettek, és több különböző olvasási modellre van szükség;
- az üzleti döntések nyomon követhetősége, auditálhatósága követelmény, például pénzügyi rendszerben;
- az állapotváltozásokat eseményalapú integrációval szeretnénk továbbadni más rendszereknek;
- a domain magas komplexitású, és a tiszta parancs-/eseményfogalmak segítenek a modell rendezésében.

Ugyanakkor kerülendő, ha:

- a rendszer egyszerű CRUD, kevés üzleti szabállyal;
- a csapat nem ismeri a mintát, és nincs tapasztalat az eventual consistency kezelésében;
- minden lekérdezésnél erős konzisztencia elvárt, és az üzleti folyamat nem tolerálja a késleltetést;
- a fejlesztési sebesség fontosabb, mint a hosszú távú bővíthetőség, és a minta bevezetése túlzott komplexitást okozna.

A döntést ADR-ben kell rögzíteni: kontextus, döntés, alternatívák, következmények és állapot. Az ADR számát nem érdemes növelni, de egy CQRS/Event Sourcing bevezetésénél a döntés és az elvetett alternatívák dokumentálása később kulcsfontosságú.

13.2.12 Tipikus hibák

1. **CQRS-t mindenhol alkalmazni.** A legtöbb alrendszer egyszerű CRUD, és a szétválasztás csak felesleges bonyolultságot hoz. A minta ott értékes, ahol a fenti feltételek fennállnak.
2. **Az olvasási modellt erős konzisztenciával várni.** Ha a projekció aszinkron, a frissesség nem garantálható azonnal. A hibás elvárás helyett mérhető frissességi SLA-t kell meghatározni.
3. **Parancsot eseményként tárolni.** A `SubmitOrder` parancs nem tény, hanem szándék. Ha az eseménytárba parancsokat írunk, a visszajátszás nem a tényleges állapotot adja vissza, hanem a kérések sorozatát, és a parancsok újraértelmezése más eredményre vezethet.
4. **Nem verziózni az eseményeket.** Az aggregátumon belüli verzió nélkül az egyidejű parancsok felülírhatják egymást. Az eseménytárnak a verzió alapján kell konfliktust észlelnie.

5. **Az eseménytörténetet közvetlenül az API-ból lekérdezni.** A hosszú streamek visszaadása lassú és felesleges. Az API az olvasási modellt használja, nem az eseménytárat.
6. **Snapshot nélkül hosszú streameket visszajátszani minden kérésnél.** A legtöbb aggregátum rövid, de amelyik hosszú, annál pillanatképet kell használni.
7. **Az esemény sémaevolúcióját figyelmen kívül hagyni.** Az események tartósan megmaradnak, ezért a payload módosítása nem érintheti a régi eseményeket. A sémaváltozáshoz verziózás és upcaster kell, amely a régi eseményt az új formátumra alakítja a visszajátszás előtt.
8. **A törlési követelményeket alábecsülni.** Az append-only eseménytörténetből való törlés nehéz. GDPR esetén például anonimizálás, crypto-shredding vagy kompenzáló esemény jöhet szóba, de ezt a tervezés elején kell kezelni, nem utólag.

13.2.13 Labor: CQRS és Event Sourcing vázlat

A labor célja, hogy a tanult fogalmakat egy kicsi, ellenőrizhető példán alkalmazza. A feladat nem termelési rendszer építése, hanem a modell, az eseménytörténet és a projekciók átgondolása.

Kontextus: „Rendelés-kezelő szolgáltatás" egyszerűsített verziója. A rendszerben egy rendelést lehet létrehozni, tételt hozzáadni, leadni, majd a fizetést beérkezettként jelölni. A labor izolált környezetben, például lokális Markdown fájlban vagy jegyzetfüzetben végezhető.

Kiinduló állapot: üres munkakönyvtár, ahol csak a laborleírás és a jelen alfejezetben bemutatott pszeudokódok állnak rendelkezésre. Nincs szükség valós adatbázisra; az eseménytár és a projekciók memóriában szimulálhatók.

Elvárt eredmény: egy reprodukálható dokumentum, amely tartalmazza:

1. A parancsok felsorolását: `CreateOrder`, `AddOrderItem`, `SubmitOrder`, `MarkPaymentReceived`. Mindegyikhez tüntesse fel a célját, a várt adatokat és az elutasítási feltételeket.
2. Az események felsorolását: `OrderCreated`, `OrderItemAdded`, `OrderSubmitted`, `PaymentReceived`. Mindegyikhez adja meg a payload sémáját és az elnevezés múlt idejű jellegét.
3. A rendelés aggregátum állapotgépét: `Draft`, `Submitted`, `Paid`. Írja le, melyik parancs melyik állapotban érvényes, és milyen eseményt vált ki.
4. Egy pszeudokód `apply` függvényt az aggregátum állapotának visszajátszásához.
5. Egy Mermaid szekvenciadiagramot, amely a parancs beérkezésétől a projekció frissítéséig mutatja a folyamatot.
6. Egy olvasási modell definícióját: például `PendingOrdersView`, amely csak a `Draft` és `Submitted` rendeléseket listázza.
7. Egy rövid ADR-t, amelyben rögzíti a CQRS és az Event Sourcing választását, az alternatívákat (például hagyományos tranzakciós modell), a frissességi SLO-t és a snapshot készítésének kezdeti szabályát.
8. A pillanatképes visszajátszás pszeudokódját, és egy példát arra, amikor a snapshot a `version=2` állapotból indul, majd a `version=3` és `version=4` eseményeket alkalmazza.

Mért és várt eredmények elkülönítése: a dokumentumban külön szakaszban szerepeljen, hogy mit vár elméletileg a modell (például a `PendingOrdersView` a `PaymentReceived` után nem tartalmazza a rendelést), és mit tapasztalt a szimuláció során (például a projekció hányszor futott le, és a frissesség mennyi időt vett igénybe a lokális környezetben). A nem futtatott mérést ne tüntesse fel ellenőrzött eredményként.

Ellenőrzési szempontok:

- A parancsok és események egyértelműen szétválnak, az események múlt idejű tények.
- Az aggregátum állapota kizárólag az eseményekből visszajátszható.
- A projekció determinisztikus, és ugyanaz az eseménysor ugyanazt a nézetet adja.
- A snapshot utáni visszajátszás csak a snapshot verziónál nagyobb verziójú eseményeket alkalmazza.
- A dokumentum rögzíti az eventual consistency tényét és a frissességi SLO-t.
- A példa nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

Visszaállítási út: a labor helyi fájlokkal dolgozik, ezért nincs külső rendszerhatás. A munkakönyvtár törlésével minden visszaáll. Ha a szimuláció során valamilyen lokális adatbázist használtunk, azt a labor végén izoláltan töröljük, és a kiinduló állapotot a fájlok újralétrehozásával állítjuk helyre.

13.2.14 Összegzés, ellenőrzőlista és önellenőrző kérdés

A CQRS szétválasztja az írási és olvasási felelősséget, és lehetővé teszi, hogy a parancsmodell az üzleti szabályokra, az olvasási modell pedig a lekérdezésekre optimalizáljon. Az Event Sourcing az állapotot eseménytörténetként tárolja, amelyből az aktuális állapot visszajátszással, a lekérdezési nézetek pedig projekcióval állíthatók elő. A pillanatkép a visszajátszás gyorsítását szolgálja, de nem helyettesíti az eseménytörténetet.

A két minta együtt erős, de költséggel jár: eventual consistency, sémakezelés, nagyobb tanulási görbe és bonyolultabb üzemeltetés. A döntést mindig a konkrét követelmények, csapat és terhelés ismeretében kell meghozni.

Ellenőrzőlista a fejezet anyagához:

- Meg tudja fogalmazni, mi a különbség a CQS és a CQRS között?
- Külön válik-e a parancsmodell és az olvasási modell a tervezett rendszerben?
- Az események append-only módon, verzióval és múlt idejű névvel vannak-e modellezve?
- A projekció determinisztikus és idempotens-e?
- Van-e szabály a pillanatkép készítésére és felhasználására?
- Kimondta-e a frissességi SLO-t az olvasási modellre?
- Készült-e ADR a CQRS/Event Sourcing bevezetéséről?

Önellenőrző kérdés: Mi történik, ha egy olvasási modell lekérdezése erős konzisztenciát igényel, miközben az írási oldal Event Sourcingot használ? A válasz: a CQRS önmagában nem ad erős konzisztenciát az írási és olvasási modell között. Megoldás lehet a szinkron projekció, amely ugyanabban a tranzakcióban frissíti az olvasási modellt is, de ez rontja a skálázhatóságot és növeli a késleltetést. Alternatíva, hogy a saját írás utáni olvasást közvetlenül a parancsmodellből szolgáljuk ki, míg a listanézeteknél elfogadjuk a rövid késleltetést. A helyes megoldás az üzleti folyamat és a frissességi SLO gondos elemzésétől függ, nem a minta mechanikus alkalmazásától.

A következő alfejezetben az itt bevezetett eseményfogalmakra építve foglalkozunk a saga, outbox és elosztott tranzakciók mechanizmusával, amelyek az Event Sourcinggal gyakran együtt jelennek meg, de attól függetlenül is értelmezhetők.

13.3 Saga, outbox és elosztott tranzakciók

13.3.1 Elosztott tranzakciók: a monolit ACID-tól a végső konzisztenciáig

A klasszikus relációs adatbázisok világában a tranzakció fogalma világos: az ACID-tulajdonságok – atomitás, konzisztencia, izoláció, tartósság – garantálják, hogy egy műveletsorozat vagy maradéktalanul végrehajtódik, vagy egyáltalán nem. Amikor azonban a rendszerünk több szolgáltatásra bomlik, és mindegyik saját adatbázissal rendelkezik, ez a garancia megszűnik. Egy rendelés leadása például három különböző szolgáltatást érinthet: a rendelés rögzítését, a fizetés végrehajtását és a készlet lefoglalását. Ha a fizetés sikeres, de a készletfoglalás közben hiba lép fel, a rendszer inkonzisztens állapotba kerül: a vevő fizetett, de a termék nincs lefoglalva. Az elosztott tranzakciók kezelése éppen ezt a problémát hivatott megoldani, de a hagyományos kétfázisú commit (2PC) megközelítés a gyakorlatban gyakran nem működik jól. A saga és az outbox minta két olyan eszköz, amelyek a kompenzáció és a megbízható üzenetküldés segítségével teremtenek végső konzisztenciát anélkül, hogy feláldoznák a szolgáltatások autonómiáját.

13.3.2 A kétfázisú commit (2PC) ígérete és csapdái

A kétfázisú commit protokoll (például az X/Open XA szabvány alapján) egy koordinátor résztvevőket kér fel, hogy szavazzanak egy tranzakció végrehajtásáról. Első fázisban minden résztvevő előkészíti a tranzakciót, és megerősíti, hogy kész a véglegesítésre vagy a visszagörgetésre. Második fázisban a koordinátor döntése alapján minden résztvevő commit-ol vagy rollback-el. Ez a modell látszólag megoldja az atomitást, de súlyos ára van.

Blokkolás és rendelkezésre állás-csökkenés. Ha a koordinátor vagy egy résztvevő elérhetetlenné válik az előkészítés után, a tranzakcióban részt vevő erőforrások zárolva maradhatnak, amíg a hiba el nem hárul. Ez a blokkolás közvetlenül sérti a magas rendelkezésre állásra vonatkozó követelményeket: egyetlen lassú szolgáltatás az egész láncot megbénítja.

Hálózati partíció és koordinátor SPOF. Elosztott rendszerekben a hálózati hiba nem ritka esemény, hanem mindennapi valóság. Ha a koordinátor és a résztvevők között megszakad a kapcsolat, a résztvevők nem tudják, hogy commit vagy rollback következik-e. A koordinátor maga is egyetlen meghibásodási pont (SPOF), hacsak nem replikáljuk bonyolult módon.

Teljesítmény és erőforrás-zárolás. A 2PC miatt a résztvevőknek tranzakciós zárolást kell tartaniuk az előkészítéstől a végleges döntésig. Ez hosszú, akár másodpercekig tartó várakozást jelent, ami nagy terhelés alatt drasztikusan csökkenti az átbocsátóképességet. Emellett a résztvevőknek támogatniuk kell az XA protokollt, ami korlátozza a technológiai választást.

Ezek a korlátok vezettek oda, hogy a modern elosztott rendszerekben a 2PC helyett inkább a saga mintát és a megbízható üzenetküldést alkalmazzák. A cél nem a szigorú atomitás, hanem a végső konzisztencia: a rendszer rövid időre inkonzisztens lehet, de előbb-utóbb minden szolgáltatás konzisztens állapotba kerül.

13.3.3 A saga minta: hosszú életű tranzakció lokális lépésekkel

A saga minta egy üzleti tranzakciót kisebb, lokális tranzakciók sorozatára bont. Minden lépés egyetlen szolgáltatásban, a saját adatbázisán fut le, és sikeres végrehajtás után egy üzenetben értesíti a következő lépést. Ha egy lépés meghiúsul, a saga nem „görgeti vissza” az előző lépéseket, hanem **kompenzáló műveleteket** hajt végre: minden korábban sikeres lépéshez tartozik egy ellentétes irányú művelet, amely üzletileg visszavonja annak hatását. Például ha a fizetés sikeres

volt, a kompenzáló művelet a visszafizetés; ha a készletfoglalás megtörtént, a kompenzálás a foglalás felszabadítása.

A saga koordinációja kétféle lehet: **koreografált** vagy **orchestrált**.

Koreografált saga esetén nincs központi koordinátor. A szolgáltatások eseményeket publikálnak és figyelnek, mindenki maga dönti el, mikor és milyen műveletet hajt végre. Ez a modell jól illeszkedik az eseményvezérelt architektúrákhoz, csökkenti a szolgáltatások közötti csatolást, de nehezebb átlátni a teljes folyamatot, és a hibakezelési logika szétszóródik.

Orchestrált saga esetén egy központi koordinátor (gyakran egy saga orchestrator szolgáltatás) parancsokat küld a résztvevőknek, és nyomon követi a lépések állapotát. Ez a modell jobban dokumentálható, a folyamat egy helyen látható, de a koordinátor újabb csatolási pontot jelent, és ha nem gondoskodunk a magas rendelkezésre állásáról, maga is SPOF lehet.

Az alábbi Mermaid szekvenciadiagram egy orchestrált saga egyszerűsített példáját mutatja, ahol a rendelési folyamat koordinátora a lépéseket vezérli:

```
sequenceDiagram
    participant Koord as Saga orchestrator
    participant Ren as Rendelés szolg.
    participant Fiz as Fizetés szolg.
    participant Kes as Készlet szolg.

    Koord->>Ren: Rendelés létrehozása (parancs)
    Ren-->>Koord: Rendelés kész (esemény)
    Koord->>Fiz: Fizetés indítása (parancs)
    Fiz-->>Koord: Fizetés sikeres (esemény)
    Koord->>Kes: Készletfoglalás (parancs)
    Kes-->>Koord: Foglalás sikertelen (esemény)
    Koord->>Fiz: Visszafizetés (kompenzáló parancs)
    Fiz-->>Koord: Visszafizetés kész (esemény)
    Koord->>Ren: Rendelés törlése (kompenzáló parancs)
    Ren-->>Koord: Rendelés törölve (esemény)
```

A saga nem garantál atomitást, hanem **végső konzisztenciát**: a lépések közötti időben az egyes szolgáltatások állapota eltérhet, de a kompenzációk lefutása után a rendszer üzletileg konzisztens lesz. Ennek feltétele, hogy minden lépésnek legyen jól definiált kompenzáló művelete, és a kompenzációk maguk is megbízhatóan, idempotensen végrehajthatók legyenek.

13.3.4 Kompenzáló művelet és idempotencia

A kompenzáló művelet nem ugyanaz, mint az adatbázis rollback. A rollback egy lokális tranzakcióban visszaállítja az adatbázis előző állapotát, míg a kompenzáló művelet egy újabb, üzleti jelentésű művelet, amely ellentétes hatást vált ki. Például:

- **Fizetés** kompenzálása: visszafizetés (refund).
- **Készletfoglalás** kompenzálása: foglalás felszabadítása.
- **E-mail küldés** kompenzálása: korrekciós e-mail küldése (bár ezt gyakran inkább elkerülik).

A kompenzáló műveletnek is **idempotensnek** kell lennie, mert a hálózati hibák, újrapróbálkozások vagy duplikált üzenetek miatt többször is lefuthat. Az idempotencia azt jelenti, hogy egy művelet többszöri végrehajtása ugyanazt az eredményt adja, mint az egyszeri. Ha a visszafizetés nem idempotens, akkor a vevő duplán kapná vissza a pénzét.

Az idempotencia megvalósításának gyakori eszköze az **idempotencia kulcs**: minden üzleti művelethez egyedi azonosítót rendelünk, és a szolgáltatás az adatbázisában tárolja, hogy az adott

kulcsot már feldolgozta-e. Ha ugyanazzal a kulccsal érkezik újra a kérés, a szolgáltatás nem hajtja végre újra a műveletet, hanem visszaadja a korábbi eredményt. Ez a kulcs lehet például a rendelés azonosítója, a fizetési tranzakció azonosítója vagy egy dedikált `idempotencyKey` mező.

A saga lépései általában **legalább egyszeri kézbesítést** feltételeznek: az üzenetküldő rendszer garantálja, hogy az üzenet legalább egyszer eljut a fogyasztóhoz, de előfordulhat, hogy kétszer is. Ezért a fogyasztónak minden esetben idempotensnek kell lennie. A gyakorlatban ezt úgy érik el, hogy a fogyasztó a feldolgozás előtt ellenőrzi az idempotencia kulcsot egy deduplikációs táblában, és csak akkor hajtja végre a műveletet, ha a kulcs még nem szerepel ott.

13.3.5 Megbízható üzenetküldés: az outbox minta

Az eddigiekből kiderült, hogy a saga lépései közötti kommunikáció üzenetekkel történik. De hogyan biztosíthatjuk, hogy egy szolgáltatás az adatbázis-tranzakciója után **biztosan** elküldje az értesítést? A naiv megoldás – először commit-olunk az adatbázisban, majd publikálunk a message brokerben – a **duális írás** problémájához vezet. Ha a brokerhez való kapcsolódás meghiúsul a commit után, az üzenet elveszik, és a többi szolgáltatás nem értesül a változásról. Ha pedig előbb publikálunk, és utána commit-olunk, az adatbázis-tranzakció sikertelensége esetén olyan üzenetet küldtünk ki, amely mögött nincs valós állapotváltozás.

Az **outbox minta** ezt a problémát úgy oldja meg, hogy az üzenetküldést az adatbázis-tranzakció részévé teszi. Az alkalmazás a lokális adatbázisában, ugyanabban a tranzakcióban ír egy `outbox` táblába egy rekordot, amely tartalmazza a kiküldendő üzenetet (például JSON formátumban, valamint az esemény típusát és az idempotencia kulcsot). Ezután egy külön folyamat – az **outbox relé** vagy **worker** – időközönként lekérdezi az outbox táblát, kiküldi a még el nem küldött üzeneteket a message brokernek, majd megjelöli azokat elküldöttként.

Az outbox minta folyamata:

1. Az üzleti tranzakció lefut, és a `commit` során az outbox rekord is mentésre kerül.
2. Az outbox relé lekérdezi a feldolgozatlan rekordokat.
3. Az üzeneteket publikálja a brokerben.
4. Sikeres publikálás után a relé frissíti a rekord állapotát (`SENT`), vagy törli azt.
5. Ha a publikálás meghiúsul, a relé újrapróbálkozik, így az üzenet legalább egyszer eljut a brokerhez.

Ez a megoldás **legalább egyszeri kézbesítést** garantál: az üzenet nem veszik el, de többször is megérkezhet. Ezért a fogyasztóknak idempotensnek kell lenniük.

Az outbox tábla tipikus szerkezete:

Oszlop	Típus	Leírás
`id`	UUID	Az outbox rekord egyedi azonosítója
`aggregate_id`	UUID/string	Az üzleti entitás azonosítója (pl. rendelés ID)
`event_type`	string	Az esemény típusa (pl. `OrderCreated`)
`payload`	JSON	Az esemény tartalma
`created_at`	timestamp	Létrehozás időpontja
`sent_at`	timestamp	Sikeres elküldés időpontja (nullable)
`status`	enum	`PENDING`, `SENT`, `FAILED`

Az outbox relé megvalósítása lehet egy külön komponens, egy háttérszál, vagy akár egy CDC-alapú folyamat is. A **change data capture (CDC)** megközelítés a relációs adatbázis tranzakciós

naplóját (például PostgreSQL WAL, MySQL binlog) figyeli, és a változásokat közvetlenül a brokerbe továbbítja. Ez a technika kiküszöböli a polling késleltetését, és biztosítja, hogy csak a commit-olt tranzakciók kerüljenek kiküldésre. Ilyen eszköz például a Debezium (a pontos verzió és konfiguráció a hivatalos dokumentációban ellenőrizendő), amely Kafka Connect keretrendszerre épül.

Az outbox minta erőssége, hogy a szolgáltatás saját adatbázisán belül marad atomi, és a message broker nem vesz részt a tranzakcióban. Ezáltal a szolgáltatás autonómiája megmarad, és a rendszer nem szenved a 2PC blokkolási problémáitól.

13.3.6 Saga és outbox kombinációja a gyakorlatban

Egy jól megtervezett saga lépései általában mindkét mintát használják: a lokális tranzakció végrehajtása után az outboxon keresztül küldik el az eseményt, amely a következő lépést indítja. Ez a kombináció biztosítja, hogy a saga lépései megbízhatóan, veszteség nélkül kommunikálnak egymással, és a hibakezelés is megbízhatóan működik.

Tekintsük a rendelés–fizetés–készlet példát orchestrált sagával és outboxszal:

1. A **Rendelés szolgáltatás** létrehozza a rendelést, és az outbox táblába írja az `OrderCreated` eseményt. A koordinátor figyeli ezt az eseményt.
2. A koordinátor parancsot küld a **Fizetés szolgáltatásnak**. A fizetési szolgáltatás végrehajtja a terhelést, és saját outboxába írja a `PaymentSucceeded` eseményt (vagy `PaymentFailed` eseményt hiba esetén).
3. A koordinátor a `PaymentSucceeded` eseményre reagálva parancsot küld a **Készlet szolgáltatásnak** a foglalásra.
4. Ha a készletfoglalás sikertelen, a koordinátor kompenzáló parancsokat küld: visszafizetés a Fizetés szolgáltatásnak, rendelés törlése a Rendelés szolgáltatásnak. Mindkét kompenzáló művelet saját outboxán keresztül értesít a végrehajtásról.

Ebben a folyamatban minden szolgáltatás saját adatbázisával és saját outboxával rendelkezik. A koordinátor maga is lehet állapotmentes, ha a saga állapotát eseményekből építi fel, vagy rendelkezhet saját adatbázissal, ahol a saga lépéseinek állapotát tárolja. Az utóbbi esetben a koordinátor is outboxot használ a parancsok kiküldéséhez.

A minőségi forgatókönyv formájában mindezt így fogalmazhatjuk meg:

> **Inger:** A Rendelés szolgáltatás sikeresen commitol egy új rendelést.
> **Környezet:** Normál üzemi környezet, egyetlen régióban, a message broker elérhető.
> **Válasz:** Az outbox relé 100 ms-on belül kiküldi az `OrderCreated` eseményt a megfelelő témába.
> **Mérőszám:** A kiküldés sikeressége 99,9% 1 órás mérési ablakban; a fennmaradó 0,1% újrapróbálkozással helyreáll.

13.3.7 Gyakorlati hibák és elkerülésük

A saga és outbox minták bevezetése során számos tipikus hiba fordul elő. Ezek felismerése és tudatos kezelése elengedhetetlen a helyes megoldáshoz.

Nem idempotens fogyasztó. Ha a fogyasztó nem kezeli a duplikált üzeneteket, a saga lépései többször is lefuthatnak. Például a fizetési szolgáltatás duplikált `PaymentSucceeded` esemény hatására kétszer terheli meg a vevőt. Megoldás: minden fogyasztó dolgozzon idempotencia kulccsal, és tárolja a feldolgozott kulcsokat.

Hiányzó vagy nem teljes kompenzáló művelet. Ha egy saga lépéshez nem definiálunk kompenzáló műveletet, akkor a hiba esetén a rendszer nem tud helyreállni. A kompenzáló műveleteknek is idempotensnek kell lenniük.

Outbox relé egyszeri futtatása. Ha az outbox relé egyetlen példányban fut, és az összeomlik, az üzenetek felhalmozódnak, és a saga elakad. A relét magas rendelkezésre állással kell üzemeltetni, vagy több példányban, megfelelő elosztott zárolással.

Sorrendi problémák. Az outbox relé által kiküldött üzenetek sorrendje nem mindig garantált, különösen ha több példány fut. Ha az üzleti logika sorrendfüggő, akkor ezt a fogyasztónál kell kezelni (például verziószámokkal vagy az aggregátum állapotának ellenőrzésével).

Hosszú tranzakció és erőforrás-zárolás. Bár a saga elkerüli a 2PC zárolásait, a lépések közötti időben az üzleti entitások (például a rendelés, a készlet) foglalt állapotban lehetnek. Ha a saga túl hosszú, ezek a foglalások hosszú ideig tartanak, és csökkenthetik a rendszer kapacitását. Érdemes a sagát rövid, jól meghatározott lépésekre bontani, és a foglalásokat időkorláttal ellátni.

Nem megfelelő hibakezelés a kompenzációban. A kompenzáló művelet is hibázhat. Ilyenkor újra kell próbálkozni, és ha tartósan sikertelen, emberi beavatkozást igényel. A rendszernek naplóznia kell a saga állapotát, és riasztást kell küldenie, ha a kompenzáció nem tud lefutni.

13.3.8 Labor: saga és outbox folyamat tervezése

Cél: Tervezzünk meg egy saga és outbox alapú folyamatot egy kitalált e-kereskedelmi rendszerben. A labor eredménye egy reprodukálható dokumentum vagy jegyzetfüzet, amely tartalmazza a kontextust, a döntési naplót, a folyamat diagramját és az ellenőrzési szempontokat. A mért és várt eredményeket külön kell választani.

Kiinduló állapot: Adott egy egyszerű rendeléskezelő mikroszolgáltatás-architektúra három szolgáltatással: `order-service`, `payment-service`, `inventory-service`. Mindegyik saját PostgreSQL adatbázissal rendelkezik. A szolgáltatások közötti kommunikáció egy message brokeren keresztül történik (például Apache Kafka). A rendelési folyamat a következő lépésekből áll:

1. `order-service`: rendelés létrehozása (`OrderCreated` esemény)
2. `payment-service`: fizetés végrehajtása (`PaymentSucceeded` vagy `PaymentFailed`)
3. `inventory-service`: készletfoglalás (`InventoryReserved` vagy `InventoryReservationFailed`)
4. Hiba esetén kompenzáló műveletek: visszafizetés (`RefundPayment`) és rendelés törlése (`CancelOrder`).

Feladatok:

1. **Rendszerhatárok és C4 context ábra.** Készíts C4 context szintű diagramot (például Mermaid `flowchart` segítségével), amely megmutatja a három szolgáltatást, a felhasználót és a message brokert mint külső rendszert. Jelöld a rendszerhatárt.
2. **Saga folyamat megtervezése.** Válaszd ki a koordinációs stílust (koreografált vagy orchestrált), indokold a döntést. Rajzolj Mermaid szekvenciadiagramot a saga lépéseiről, beleértve a hibaleágazást és a kompenzáló műveleteket.
3. **Outbox tábla és komponensek.** Minden szolgáltatáshoz tervezd meg az outbox tábla szerkezetét (oszlopok, típusok). Írd le, hogyan működik az outbox relé: milyen gyakran polloz, milyen állapotokat kezel, és hogyan biztosítja a legalább egyszeri kézbesítést.

4. **Idempotencia és kompenzáció.** Határozd meg az idempotencia kulcsot minden szolgáltatáshoz. Írd le, hogyan biztosítják a szolgáltatások a duplikált üzenetek elleni védelmet, és mi a kompenzáló művelet idempotens viselkedése.

5. **Döntési napló (ADR).** Készíts legalább két ADR-t a következő döntésekről:

   - saga koordinációs stílus kiválasztása
   - outbox relé megvalósítása (polling vs. CDC) Mindegyik ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.

6. **Ellenőrzési szempontok és mérési terv.** Fogalmazz meg legalább egy quality attribute scenariot a rendelési folyamat megbízhatóságára vonatkozóan (inger, környezet, válasz, mérőszám). Határozd meg, milyen méréseket végeznél el a rendszerben, és milyen várt eredményeket fogadsz el. Ha van tényleges mérési lehetőség, rögzítsd a mért értékeket is külön szakaszban.

Átadandó dokumentum struktúrája:

- Kontextus és rendszerhatárok leírása
- C4 context ábra (diagrams-as-code)
- Saga folyamatábra
- Outbox tervezés és relé specifikáció
- Idempotencia- és kompenzációs terv
- ADR-ek
- Minőségi forgatókönyv és mérési terv (várt vs mért)

Ellenőrzési szempontok:

- A diagramokon jelölve van a rendszerhatár.
- A komponensek felelősséggel és interfésszel vannak leírva.
- A minőségi forgatókönyv az inger–környezet–válasz–mérőszám sémát követi.
- Az ADR-ek tartalmazzák a kontextust, döntést, alternatívákat és következményeket.
- Az outbox kialakítás garantálja a legalább egyszeri kézbesítést.
- Az idempotencia kulcsok minden szolgáltatásnál definiáltak.
- A mért és várt eredmények egyértelműen elkülönülnek.

13.3.9 Ellenőrzőlista a saga és outbox használatához

☐ A saga minden lépése lokális tranzakcióban fut, saját adatbázisban.
☐ Minden lépéshez definiált kompenzáló művelet.
☐ A kompenzáló műveletek idempotensek.
☐ Minden fogyasztó idempotencia kulcsot használ, és deduplikációs táblát tart fenn.
☐ Az üzenetpublikálás outboxon keresztül történik, nem duális írással.
☐ Az outbox relé magas rendelkezésre állású, és legalább egyszeri kézbesítést biztosít.
☐ A saga koordinátor állapotát perzisztáljuk, hogy hiba esetén helyreállítható legyen.
☐ A hosszan futó saga lépéseknél időkorlátot és lejárati kezelést alkalmazunk.
☐ A rendszer naplózza a saga állapotváltásait, és riasztást küld tartós hiba esetén.
☐ A minőségi forgatókönyvek meghatározzák az üzenetküldés késleltetését és sikerességét.

13.3.10 Önellenőrző kérdés

Miért oldja meg az outbox minta a duális írás problémáját, és miért van szükség idempotenciára a saga lépésekben akkor is, ha az outbox relé legalább egyszeri kézbesítést garantál? Válaszodban térj ki arra, hogy mi történik, ha a fogyasztó nem idempotens, és hogyan védi ki ezt a rendszer.

13.3.11 Összefoglalás és továbbvezetés

A saga minta a hosszú életű üzleti tranzakciókat lokális lépések és kompenzáló műveletek sorozatára bontja, míg az outbox minta megbízható üzenetküldést biztosít a duális írás elkerülésével. Együttesen lehetővé teszik, hogy elosztott rendszerekben végső konzisztenciát érjünk el anélkül, hogy a 2PC blokkolását és törékenységét elfogadnánk. A következő alfejezet (Streaming platformok és event-driven minták) ezekre az alapokra építve a streaming platformok és az eseményvezérelt minták szélesebb körét tárgyalja, beleértve a Kafka-alapú feldolgozást és a stream-joinokat.

13.4 Streaming platformok és event-driven minták

13.4.1 Bevezetés

A Saga, outbox és elosztott tranzakciók alfejezetben a saga, az outbox és az elosztott tranzakciók mintáit vizsgáltuk. Ott az üzeneteket elsősorban az üzleti folyamatok közötti megbízható állapotátvitelre használtuk. Ez a fejezet továbblép: a streaming platformokat és a folyamatos adatfeldolgozást olyan architekturális eszközként mutatja be, amely az eseményeket nem csupán üzenetként kezeli, hanem hosszú távon tárolt, visszajátszható, partíciókra osztott adatfolyamként. A cél annak megértése, hogyan lehet a rendszerek közötti integrációt lazán csatolttá, skálázhatóvá és hibajavító képessé tenni, miközben a fogyasztói késleltetést, a sorrendiséget és az adatminőséget kontroll alatt tartjuk.

A fejezetben a folyamatos adatfeldolgozás és az eseményvezérelt integráció fogalmából indulunk ki, majd bevezetjük a streaming platformok három alapfogalmát: a partíciót, az offsetet és a fogyasztói késleltetést. Ezután az eseményrendezés, a visszajátszás és az adatminőség kérdéseit tárgyaljuk. A gyakorlati rész egy streaming topológia és fogyasztói csoport tervezését mutatja be labor formájában. A fejezet végén tipikus hibákat, ellenőrzőlistát és döntési gyakorlatot talál.

13.4.2 1. Folyamatos adatfeldolgozás és eseményvezérelt integráció

13.4.3 1.1 Definíció és motiváció

A folyamatos adatfeldolgozás (stream processing) olyan számítási modell, amely az adatokat folyamatosan érkező, időben rendezett sorozatként kezeli, és az eredményt kis késleltetéssel, inkrementálisan állítja elő. Ez éles ellentétben áll a hagyományos batch feldolgozással, ahol az adatokat véges halmazokban, jellemzően napi vagy óránkénti ütemezéssel dolgozzák fel. A streaming tehát nem pusztán gyorsabb batch, hanem olyan architekturális stílus, amely a valós idejű döntéshozatalt, a folyamatos integrációt és az azonnali visszajelzést helyezi a középpontba.

Az eseményvezérelt integráció azt jelenti, hogy a rendszerek közötti kommunikáció elsődlegesen eseményeken – tényeken, amelyek már megtörténtek – keresztül valósul meg, nem pedig szinkron hívásokon vagy megosztott állapoton. Az esemény egy visszavonhatatlan, időbélyeggel ellátott rekord, például `RendelésLétrehozva`, `FizetésElutasítva` vagy `CsomagÁtadvaFutárnak`. Az

ilyen eseményeket egy streaming platform fogadja, tárolja és továbbítja a feliratkozott fogyasztóknak.

A streaming platform legfontosabb tulajdonsága, hogy az eseményeket **tartósan**, **visszajátszható módon** és **partíciókra osztva** tárolja. Ez lehetővé teszi, hogy egy fogyasztó ne csak az aktuális pillanatban érkező eseményt lássa, hanem vissza tudjon lépni egy korábbi pontra, és újra feldolgozza az eseményfolyamot. Ez a képesség alapvetően megkülönbözteti a streaming platformot a hagyományos üzenetsoroktól (message queue), ahol az üzenet a kézbesítés után jellemzően törlődik.

Az eseményvezérelt integráció előnyei:

- **Laza csatolás:** a termelő nem ismeri a fogyasztókat, a fogyasztó nem ismeri a termelőt. Új fogyasztó hozzáadása a termelő módosítása nélkül lehetséges.
- **Skálázhatóság:** a partíciók lehetővé teszik a fogyasztók horizontális skálázását, mert az egyes partíciókat párhuzamosan lehet feldolgozni.
- **Rugalmasság:** a visszajátszás lehetőséget ad hibajavításra, új analitikai modell betanítására vagy új fogyasztó bevezetésére anélkül, hogy a forrásrendszert terhelnénk.
- **Valós idejű betekintés:** az események azonnal elérhetők, így a riportok, riasztások és automatizmusok késleltetése másodperces vagy annál kisebb lehet.

A folyamatos adatfeldolgozásnak azonban ára van: a fejlesztőknek meg kell érteniük a párhuzamosság, a sorrendiség, az idempotencia és a hibatűrés sajátos szabályait. A következő szakaszokban ezeket a fogalmakat járjuk körül.

13.4.4 1.2 Példa: rendelésfeldolgozó pipeline

Képzeljünk el egy e-kereskedelmi rendszert, ahol a rendelés leadásakor a rendelésrendszer `RendelésLétrehozva` eseményt publikál. Az eseményt több fogyasztó dolgozza fel:

- a **fizetési szolgáltatás** elindítja a tranzakciót,
- a **szállítási szolgáltatás** előkészíti a csomagot,
- az **analitikai rendszer** valós idejű riportot frissít,
- a **kockázatkezelő** csalásgyanús mintákat keres.

Mindegyik fogyasztó más-más sebességgel és más üzleti logikával dolgozik. A streaming platform központi szerepet játszik: fogadja az eseményeket, tartósan tárolja őket, és eljuttatja a megfelelő fogyasztói csoportoknak. A fogyasztók egymástól függetlenül haladnak, és a platform garantálja, hogy mindegyik a saját tempójában olvashat.

Ebben a modellben a forrásrendszer nem szorul módosításra, ha új fogyasztó jelenik meg. Egy adott fogyasztó leállása nem blokkolja a többit. Ha egy fogyasztó hibásan dolgoz fel egy eseményt, akkor visszaállíthatja az offsetjét, és újra próbálkozhat. Ez a rugalmasság az eseményvezérelt integráció lényege.

13.4.5 2. Partíció, offset és fogyasztói késleltetés

A streaming platformok három alapfogalma – a partíció, az offset és a fogyasztói késleltetés – határozza meg a párhuzamosság, a sorrendiség és a megbízhatóság működését. E fogalmak pontos ismerete nélkül a fogyasztói csoportok tervezése és a kapacitástervezés hibás lesz.

13.4.6 2.1 Partíció

A partíció (partition) a téma (topic) logikai felosztása. Minden partíció egy **csak hozzáfűzhető, rendezett napló** (append-only log), amelyben az események a beérkezés sorrendjében helyezkednek el. A partíciók lehetővé teszik:

- **Párhuzamosságot:** a téma több partíciója párhuzamosan olvasható és írható.
- **Sorrendi garanciát:** egy partíción belül az események sorrendje megegyezik a termelő általi beérkezés sorrendjével. Különböző partíciók között nincs globális sorrendi garancia.
- **Skálázást:** a partíciók száma meghatározza a fogyasztók maximális párhuzamosságát.

A partíciókulcs (key) dönti el, hogy egy esemény melyik partícióba kerül. A termelőnek lehetősége van kulcsot megadni; ha nem ad meg, a platform kiosztja az eseményeket valamilyen terheléselosztással. A kulcs alapú particionálás (pl. hash a kulcsból) garantálja, hogy az azonos kulccsal rendelkező események ugyanabba a partícióba kerülnek, így **sorrendben** dolgozhatók fel. Ez kritikus, ha egy adott entitás (pl. rendelésazonosító) eseményeinek sorrendje üzletileg fontos.

Például a `rendelés-létrehozva` témában a kulcs lehet a `rendelés_id`. Így egy adott rendelés összes eseménye ugyanabba a partícióba kerül, és a fogyasztó a rendelés életciklusát helyes sorrendben látja.

13.4.7 2.2 Offset

Az offset egy **monoton növekvő egész szám**, amely az esemény pozícióját jelöli egy adott partícióban. Az első esemény offsetje például 0, a következőé 1, és így tovább. Az offset a partícióra vonatkoztatva egyedi, tehát az `5. partíció 42. offsetje` pontosan meghatározza az adott eseményt.

A fogyasztó az offset segítségével tartja nyilván, hogy hol tart a feldolgozásban. Amikor a fogyasztó beolvas egy eseményt, az offsetjét **commitolja** (elkötelezi) a platform felé. Ez a commit lehet automatikus (pl. időzítő alapján) vagy kézi (az alkalmazás explicit módon jelzi a sikeres feldolgozást). Az offset commitolása meghatározza a **legalább egyszer** (at-least-once) vagy **pontosan egyszer** (exactly-once) feldolgozási garanciát, de ez utóbbi a fogyasztó és a platform együttműködését igényli.

A visszajátszás (replay) képessége az offseten alapul: a fogyasztó visszaállíthatja az offsetjét egy korábbi értékre, és újraolvashatja az eseményeket. Ez hibajavításkor, új logika bevezetésekor vagy adat-helyreállításkor kulcsfontosságú.

13.4.8 2.3 Fogyasztói csoport és fogyasztói késleltetés

A **fogyasztói csoport** (consumer group) olyan fogyasztópéldányok halmaza, amelyek közösen dolgoznak fel egy témát. A platform gondoskodik arról, hogy minden partícióhoz pontosan egy aktív fogyasztó tartozzon az adott csoporton belül. Ha egy fogyasztó kiesik, a partíciókat újraosztják a megmaradt példányok között (rebalance). Ha új fogyasztót indítunk, az szintén partíciókat kap.

A fogyasztói késleltetés (consumer lag) azt mutatja, hogy egy fogyasztói csoport mennyire van lemaradva a termelőtől. Formálisan:

Consumer lag = a legfrissebb, termelő által írt offset − a fogyasztó által commitolt offset, partíciónként.

Ha a lag növekszik, a fogyasztó lassabban dolgozik, mint ahogy az események érkeznek. A lag mértékegysége lehet **üzenet** (darab) vagy **idő** (másodperc), attól függően, hogy az offsetek közötti különbséget vagy az események időbélyege közötti eltérést mérjük. A gyakorlatban mindkettő hasznos: a darabszám a feldolgozási torlódást, az időbeli lemaradás az üzleti frissesség csökkenését mutatja.

A lag fontos mérőszám, amelyet folyamatosan monitorozni kell. Egy tipikus minőségi forgatókönyv:

- **Inger:** 1000 rendelési esemény/perc érkezik a `rendelés-létrehozva` témába.
- **Környezet:** normál üzem, két fogyasztó példány a `fizetés-feldolgozó` csoportban.
- **Válasz:** a csoport feldolgozza az eseményeket, a lag nem haladja meg az 500 üzenetet.
- **Mérőszám:** fogyasztói lag P99, 5 perces csúszóablakban, üzenetben mérve.

A lag alapján riasztási szabályokat állíthatunk be. Például ha a lag 5 percen keresztül meghaladja az 1000 üzenetet, riasztás indul. A riasztási küszöböt a feldolgozási idő és az üzleti tűrés határozza meg.

13.4.9 2.4 Little-törvény alkalmazása a lag becslésére

A kapacitástervezésnél a Little-törvény segít megbecsülni a várható laget: $L = \lambda W$, ahol L a rendszerben lévő kérések (események) átlagos száma, λ az érkezési ráta, W pedig az átlagos tartózkodási idő.

Legyen a `rendelés-létrehozva` témába érkező események átlagos rátája $\lambda = 16{,}67$ esemény/s (azaz 1000 esemény percenként). A fogyasztó átlagos feldolgozási ideje eseményenként $W = 0{,}2$ s. Ekkor:

$$L = \lambda W = 16{,}67 \cdot 0{,}2 = 3{,}33 \text{ esemény.}$$

Ez azt jelenti, hogy átlagosan 3–4 esemény van feldolgozás alatt vagy sorban a fogyasztóban. Ha a feldolgozási idő növekszik (például lassú külső hívás miatt), a lag arányosan nő. A Little-törvény tehát egyszerű, de hatékony eszköz a kapacitás és a lag becslésére.

Fontos megjegyezni, hogy a lag nemcsak a fogyasztó lassúságából, hanem a partíciók számának elégtelenségéből is származhat. Ha a fogyasztói csoportban kevesebb partíció van, mint amennyit a fogyasztók párhuzamosan tudnának kezelni, a fogyasztói példányok száma nem növelhető a partíciók számán túl. A párhuzamosság felső határa tehát a partíciók száma.

13.4.10 3. Eseményrendezés, visszajátszás és adatminőség

A streaming platformok ereje a visszajátszhatóságban rejlik, de ez csak akkor használható ki, ha az események sorrendje és minősége megfelelően kezelt. Ebben a szakaszban az eseményrendezés garanciáit, a visszajátszás gyakorlatát és az adatminőség biztosításának eszközeit tárgyaljuk.

13.4.11 3.1 Eseményrendezés

A streaming platformok **partíción belüli rendezettséget** garantálnak, de **globális rendezettéget nem**. Ez azt jelenti, hogy ha egy téma több partícióból áll, a fogyasztó nem feltételezheti, hogy az eseményeket a termelő általi pontos időrendben kapja meg. A gyakorlatban ez két következménnyel jár:

1. **Az üzleti folyamatok sorrendiségét kulcs alapú particionálással kell biztosítani.** Ha egy rendelés minden eseménye ugyanabba a partícióba kerül, akkor a fogyasztó garantáltan a beküldés sorrendjében látja őket.
2. **A globális sorrendet igénylő elemzésekhez vagy időablakokhoz kiegészítő megoldás szükséges.** Ilyenkor gyakran használnak eseményidő-alapú ablakozást, vízjel (watermark) mechanizmust vagy dedikált rendező réteget.

A termelő oldalán fontos, hogy a kulcs megválasztása stabil legyen. Ha a kulcs változik egy entitás élettartama alatt, az események eltérő partíciókba szóródnak, és a sorrendiség elveszik. Például a rendelésazonosító jó kulcs, mert nem változik, az ügyfél e-mail címe viszont rossz kulcs, mert változhat.

13.4.12 3.2 Visszajátszás

A visszajátszás (replay) az a folyamat, amikor egy fogyasztó offsetjét manuálisan visszaállítjuk egy korábbi pontra, és újra feldolgozza az eseményeket. Ez a művelet több forgatókönyvben is hasznos:

- **Hibajavítás:** ha a fogyasztó logikája hibás volt, a javítás után a fogyasztó visszaállíthatja az offsetet, és újra feldolgozhatja az érintett időszakot.
- **Új fogyasztó felépítése:** egy új szolgáltatás a kezdetektől feldolgozhatja a múltbeli eseményeket, hogy feltöltse a saját állapotát.
- **Adat-helyreállítás:** ha egy leszármaztatott adattároló megsérül, az eseményekből újra felépíthető.
- **Tesztelés:** egy staging környezetben ugyanaz az eseményfolyam újra lejátszható.

A visszajátszás előfeltétele, hogy a fogyasztó **idempotens** legyen. Az idempotencia azt jelenti, hogy ugyanazon esemény többszöri feldolgozása ugyanazt az eredményt adja, mint az egyszeri. Például ha egy fizetési szolgáltatás duplikált `FizetésJóváhagyva` eseményt kap, nem szabad kétszer levonnia az összeget. Az idempotenciát tipikusan **eseményazonosító** alapú deduplikációval érik el: a fogyasztó naplózza a már feldolgozott eseményazonosítókat, és a duplikátumokat kihagyja.

13.4.13 3.3 Adatminőség

Az adatminőség a streaming pipeline-okban több dimenziót ölel fel:

- **Séma és séma evolúció:** az események szerkezetét explicit sémában kell rögzíteni, és a séma változásait verziózottan kell kezelni. A gyakorlatban séma-regisztrációs eszköz (schema registry) biztosítja, hogy a termelő és a fogyasztó kompatibilis legyen. A séma evolúció szabályai (pl. csak opcionális mezők hozzáadása) megakadályozzák a fogyasztók törését.
- **Érvényesség:** az eseményeknek meg kell felelniük az üzleti szabályoknak. A termelő oldali validálás csökkenti a hibás események bekerülését, de a fogyasztónak is védekeznie kell.

- **Teljesség:** az eseményeknek ki kell terjedniük a várt tartományra. A hiányzó eseményeket monitoringgal lehet észlelni (pl. az egy percenként érkező eseményszám drasztikus csökkenése).
- **Időszerűség:** az eseményeknek időben el kell érniük a fogyasztókat. A fogyasztói lag ezt méri.
- **Konzisztencia:** az eseményeknek nem szabad ellentmondaniuk egymásnak. Az azonos kulcsú események sorrendisége ezt segíti.

A hibás eseményeket **holtbetűs sorba** (dead letter queue, DLQ) kell irányítani. Ez egy olyan külön téma vagy tároló, ahová a feldolgozhatatlan események kerülnek. A DLQ lehetővé teszi, hogy a hibákat elemzésre elkülönítsük, és ne blokkoljuk a fő folyamatot. A DLQ-t monitorozni kell, és riasztást kell beállítani, ha a hibás események aránya meghalad egy küszöböt.

Az adatminőség mérése például a **hibás események aránya** (bad event ratio) lehet: az adott időablakban feldolgozott események hány százaléka került DLQ-ba. Minőségi forgatókönyv:

- **Inger:** 1000 rendelési esemény/perc, amelynek 2%-a sérti a sémát vagy az üzleti szabályokat.
- **Környezet:** normál üzem, a pipeline éles környezetben fut.
- **Válasz:** a rendszer a hibás eseményeket DLQ-ba irányítja, a feldolgozás nem áll le.
- **Mérőszám:** hibás események aránya 1%, 15 perces ablakban.

13.4.14 4. Streaming topológia tervezése

A streaming topológia a forrásokból, feldolgozó lépésekből és nyelőkből (sink) álló gráf. A tervezés során meg kell határozni a témákat, a partíciószámot, a fogyasztói csoportokat és az adatáramlás irányát. A jó topológia olvasható, skálázható és hibatűrő.

13.4.15 4.1 Példa: rendelésfeldolgozó topológia

Képzeljük el a korábbi e-kereskedelmi példát. A topológia a következő témákból és fogyasztókból állhat:

- `rendelés-létrehozva` téma: a rendelésrendszer publikálja, kulcs a rendelés_id, partíciószám 8.
- **Fogyasztói csoport 1:** `fizetés-feldolgozó` – a fizetési szolgáltatás olvassa, feldolgozza, majd `fizetés-eredmény` témába publikál.
- **Fogyasztói csoport 2:** `szállítás-előkészítő` – a szállítási szolgáltatás olvassa a `rendelés-létrehozva` témát, létrehozza a csomagot, majd `csomag-átadva-futárnak` témába publikál.
- **Fogyasztói csoport 3:** `analitika` – az analitikai rendszer kulcsszintű aggregátumokat számol, és egy valós idejű dashboardot frissít.
- `fizetés-eredmény` téma: a fizetési szolgáltatás publikálja, kulcs a rendelés_id, partíciószám 8.
- **Fogyasztói csoport 4:** `rendelés-státusz-kezelő` – frissíti a rendelés állapotát a fizetési eredmény alapján.
- **DLQ témák:** `fizetés-dlq`, `szállítás-dlq` stb.

A fogyasztói csoportok egymástól függetlenek, tehát a fizetés lassúsága nem gátolja a szállítás vagy az analitika haladását. A partíciószám megválasztása kritikus: ha a forgalom növekszik, több partícióra lehet szükség. A partíciószám növelése azonban nem triviális utólag, mert a kulcs alapú particionálás megváltozhat, ezért a partíciószámot általában előre, a várható terhelés alapján kell meghatározni.

13.4.16 4.2 C4 diagramok

A C4 context szintjén a rendszerhatárt jelöljük, és a külső rendszereket, szerepköröket tüntetjük fel. A streaming platform itt egy belső komponens, de logikailag elkülöníthető.

```
C4Context
    title Rendszerkontextus - Rendelésfeldolgozó streaming pipeline
    Person(vásárló, "Vásárló", "Online rendelést ad le")
    System(rendelésrendszer, "Rendelésrendszer", "Rendeléseket fogad és eseményeket publikál")
    System(streaming, "Streaming platform", "Apache Kafka, rendelési eseményeket továbbít")
    System(fizető, "Fizetési szolgáltatás", "Fizetési eseményeket dolgoz fel")
    System(szállító, "Szállítási szolgáltatás", "Szállítási feladatokat kezel")
    System(analitika, "Analitikai rendszer", "Valós idejű riportokat készít")
    Rel(vásárló, rendelésrendszer, "Leadja a rendelést", "HTTPS")
    Rel(rendelésrendszer, streaming, "Publikál RendelésLétrehozva eseményt", "Kafka protokoll,
↪   async")
    Rel(streaming, fizető, "Fizetési folyamat indítása", "Kafka, async")
    Rel(streaming, szállító, "Szállítás indítása", "Kafka, async")
    Rel(streaming, analitika, "Események elemzése", "Kafka, async")
```

A component szintű diagram a belső komponenseket és a köztük lévő kapcsolatokat mutatja. Itt a fogyasztói csoportokat és a témákat is megjeleníthetjük.

```
graph TD
    subgraph "Rendelésrendszer"
        P[Producer: RendelésLétrehozva]
    end
    subgraph "Streaming platform"
        T1["Téma: rendelés-létrehozva (8 partíció)"]
        T2["Téma: fizetés-eredmény (8 partíció)"]
        TDLQ1["Téma: fizetés-dlq"]
    end
    subgraph "Fizetési szolgáltatás"
        C1[Fogyasztó csoport: fizetés-feldolgozó]
        P2[Producer: FizetésEredmény]
    end
    subgraph "Szállítási szolgáltatás"
        C2[Fogyasztó csoport: szállítás-előkészítő]
    end
    subgraph "Analitikai rendszer"
        C3[Fogyasztó csoport: analitika]
    end
    P -->|Kafka, async| T1
    T1 --> C1
    T1 --> C2
    T1 --> C3
    C1 -->|hiba esetén| TDLQ1
    C1 --> P2
    P2 -->|Kafka, async| T2
```

A diagramon a kapcsolatokon a protokoll, a szinkronitás és a hibatűrés szerepel. Jelen esetben minden kapcsolat aszinkron, Kafka protokoll. A hibatűrést a DLQ jelzi.

13.4.17 4.3 ADR példa

A topológia tervezése során döntéseket hozunk, amelyeket ADR-ben rögzítünk.

ADR-2024-12-01: Partíciószám megválasztása a `rendelés-létrehozva` témához

- **Kontextus:** Az e-kereskedelmi rendszer várható csúcsterhelése 5000 rendelés/perc. A rendelésfeldolgozó pipeline-nak 2 másodpercen belül kell reagálnia a rendelési eseményekre. A fogyasztói csoportok párhuzamosan dolgoznak, és a jövőbeli terhelésnövekedés várhatóan évi 30%.
- **Döntés:** A `rendelés-létrehozva` témát 12 partícióra osztjuk. A kulcs a `rendelés_id`. A fogyasztói csoportok maximális párhuzamossága így 12 példány. A partíciószámot felülvizsgáljuk félévente.
- **Alternatívák:**
  - 6 partíció: kisebb párhuzamosság, de egyszerűbb üzemeltetés. Elutasítva, mert a csúcsterheléshez nem elég.
  - 24 partíció: nagyobb párhuzamosság, de nagyobb replikációs és üzemeltetési overhead. Elutasítva, mert a jelenlegi terheléshez túlzott.
- **Következmények:** A partíciószám növelése utólag lehetséges, de kulcs alapú particionálás esetén az események újraelosztása bonyolult. A fogyasztói csoportok párhuzamossága legfeljebb 12 példány. A jövőbeli növekedéshez partícióbővítést vagy új témát kell tervezni.
- **Állapot:** Elfogadva, felülvizsgálat: 2025-06-01.

13.4.18 5. Labor: streaming topológia és fogyasztói csoport tervezése

Ez a labor a fejezetben tanult fogalmakat alkalmazza egy kis, ellenőrizhető példán. A labor nem igényel futtatást, de a dokumentumnak reprodukálhatónak kell lennie: rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal.

13.4.19 5.1 Kontextus

Az `ACME Retail` e-kereskedelmi vállalat rendelésfeldolgozó rendszert épít. A rendelésrendszer minden leadott rendelésről `RendelésLétrehozva` eseményt publikál. Az esemény szerkezete:

```
{
  "event_id": "uuid",
  "rendelés_id": "string",
  "ügyfél_id": "string",
  "összeg": "decimal",
  "pénznem": "string",
  "időbélyeg": "ISO-8601"
}
```

A következő fogyasztókra van szükség:

- **Fizetési szolgáltatás:** elindítja a fizetési tranzakciót, majd `FizetésEredmény` eseményt publikál.
- **Szállítási szolgáltatás:** szállítási feladatot hoz létre, és időbélyeggel rögzíti az átvételt.
- **Analitikai rendszer:** valós idejű riportot készít a rendelt összegekről ügyfelenként.

A csúcsterhelés 2000 rendelés/perc. A fizetési szolgáltatás átlagos feldolgozási ideje 500 ms, a szállításié 300 ms, az analitikáé 200 ms. A fogyasztói lag nem haladhatja meg az 1000 üzenetet 5 perces ablakban.

13.4.20 5.2 Feladat

1. Tervezd meg a streaming topológiát: témák, partíciószámok, kulcsok, fogyasztói csoportok.
2. Határozd meg a partíciószámot a `rendelés-létrehozva` és a `fizetés-eredmény` témákhoz.
3. Készíts C4 context és component diagramot (Mermaid vagy PlantUML).
4. Írj ADR-t a partíciószám megválasztásáról.
5. Készíts monitoringtervet a fogyasztói lag mérésére és riasztására.
6. Fogalmazz meg visszajátszási eljárást arra az esetre, ha a fizetési szolgáltatás hibásan dolgozta fel az eseményeket.

13.4.21 5.3 Kiinduló állapot

- A rendszerindító architektúra tervezési fázisban van, nincs futó pipeline.
- A csapat ismeri a Kafka alapfogalmait, de még nem döntött a partíciószámról.
- A fogyasztói alkalmazások konténerizáltak, horizontálisan skálázhatók.
- A séma regisztráció még nem megoldott, de a laborban feltételezzük, hogy a sémák verziózottak.

13.4.22 5.4 Elvárt eredmény

A labor végén a következőket kell átadni:

- **Topológia leírása** diagrammal együtt.
- **Partíciószámok és kulcsok** indoklással.
- **Fogyasztói csoportok** definíciója.
- **ADR** a partíciószámról.
- **Monitoringterv** a laghoz.
- **Visszajátszási eljárás** leírása.
- **Mért és várt eredmény** külön szekcióban.

13.4.23 5.5 Ellenőrzési szempontok

- A topológia tartalmazza az összes forrást, fogyasztót és nyelőt.
- A partíciószám összhangban van a csúcsterheléssel és a fogyasztók párhuzamosságával.
- A kulcsok biztosítják a rendelésenkénti sorrendet.
- A fogyasztói csoportok függetlenek, egymás hibája nem blokkolja a másikat.
- A lag monitoring mértékegységet, ablakot és riasztási küszöböt tartalmaz.
- A visszajátszási eljárás idempotenciát feltételez és DLQ-t is kezel.
- A döntési napló ADR formátumú, tartalmazza a kontextust, döntést, alternatívákat és következményeket.

13.4.24 5.6 Visszaállítás

Mivel a labor nem futtat, visszaállítási út nem szükséges. Ha a labor később gyakorlati kivitelezéssel folytatódik, a visszaállítás a staging környezet újraépítését jelenti a `docker compose down -v` és a `kafka-topics --delete` parancsokkal (izolált környezetben).

13.4.25 5.7 Mért és várt eredmény

Várt eredmény: A topológia tervezése alapján a `rendelés-létrehozva` témához legalább 8 partíció szükséges, mert a csúcsterhelés 2000/perc = 33,3 esemény/s, és a fizetési szolgáltatás feldolgozási ideje 500 ms. A Little-törvénnyel: $L = \lambda W = 33{,}3 \cdot 0{,}5 = 16{,}7$ esemény átlagosan a fizetési fogyasztóban. Ha 4 partíció van, és 4 fogyasztó, akkor minden partícióra kb. 8,3 esemény/s jut, ami összesen $4 \cdot 16{,}7 = 66{,}8$ esemény, tehát a lag 1000 alatt marad. A pontos számítás az egyes fogyasztókra eltérő.

Mért eredmény: A labor során nem történt mérés, ezért nincs ellenőrzött mért érték. Ha a labor később fut, a mért laget ide kell beírni, feltüntetve a környezetet, a konfigurációt, az időpontot és a mérési bizonytalanságot.

13.4.26 6. Tipikus hibák

A streaming platformokkal kapcsolatos gyakori hibák elkerülése érdekében a következőket érdemes szem előtt tartani:

- **Rossz partíciókulcs választása:** Ha a kulcs nem stabil, az események szétszóródnak a partíciók között, és a sorrendiség elveszik. Robbanékony példa: ügyfél e-mail címe, amely változhat. Helyes kulcs: entitásazonosító, pl. rendelés_id.
- **Túl sok fogyasztó, kevés partíció:** A fogyasztói csoportban a párhuzamosságot a partíciók száma korlátozza. Ha 20 fogyasztó van, de csak 4 partíció, 16 fogyasztó tétlen. A partíciószámot a várható csúcsterheléshez kell igazítani.
- **Hiányzó idempotencia:** A visszajátszás vagy legalább egyszeri kézbesítés duplikátumokat okozhat. A fogyasztónak deduplikálnia kell eseményazonosító alapján.
- **Nincs DLQ:** Ha egy esemény feldolgozhatatlan, a fogyasztó végtelen ciklusba kerülhet, blokkolva a partíciót. A DLQ elkülöníti a hibás eseményeket, és lehetővé teszi az elemzést.
- **A lag nem monitorozott:** A növekvő lag a kapacitásprobléma első jele. Megfelelő riasztások nélkül a késleltetés elérheti az üzleti tűréshatárt.
- **Séma változás kezelése nélkül:** A séma evolúció kompatibilitás nélküli változtatása a fogyasztók töréséhez vezet. A séma-regisztrációs eszköz és a verziózási szabályok elengedhetetlenek.
- **Globális sorrend feltételezése:** A streaming platformok nem garantálnak globális sorrendet, csak partíción belülit. A tervezésnek ezt figyelembe kell vennie.

13.4.27 7. Ellenőrzőlista, döntési gyakorlat és önellenőrző kérdés

13.4.28 7.1 Ellenőrzőlista

- A topológia minden forrása, feldolgozója és nyelője azonosítva van.
- Minden témához van partíciószám, és az indokolt.
- Minden témához van kulcs, és a kulcs stabil entitásazonosító.
- A fogyasztói csoportok függetlenek, és a párhuzamosság nem haladja meg a partíciók számát.
- A fogyasztói lag mértékegysége, mérési ablaka és riasztási küszöbe rögzített.
- A fogyasztók idempotensek, és a duplikátumok kezelése megoldott.
- A hibás események DLQ-ba kerülnek, és a DLQ-t monitorozzák.
- A visszajátszási eljárás dokumentált, és minden érintett csapat ismeri.
- A séma evolúció szabályai meghatározottak.
- Az ADR-ek rögzítik a legfontosabb döntéseket.

13.4.29 7.2 Döntési gyakorlat

Egy pénzügyi szolgáltató tranzakciós eseményeket dolgoz fel. A csúcsterhelés 10 000 tranzakció/másodperc. Minden tranzakcióhoz tartozik egy számlaszám, és a tranzakciók sorrendje számlánként fontos. A fogyasztó egy kockázatkezelő motor, amely tranzakciónként 100 ms-ot vesz igénybe. Tervezd meg a partíciószámot és a fogyasztói csoport méretét! Milyen kulcsot használnál? Hogyan alakul a lag a Little-törvény alapján? Válaszodat ADR formában rögzítsd.

13.4.30 7.3 Önellenőrző kérdés

Miért fontos a partíciókulcs stabilitása, és mi történik, ha a kulcs megváltozik egy entitás élettartama alatt? Válaszodban térj ki a sorrendiségre és a fogyasztói feldolgozásra.

14 Integráció és API-tervezés

14.1 REST, GraphQL és gRPC

REST, GraphQL és gRPC

Az integrációs stílus megválasztása nem csupán technikai kérdés, hanem az API szerződésének, a hibakezelésnek, a kliensoldali fejlesztésnek és a rendszer evolúciójának alapvető meghatározója. Ebben az alfejezetben három elterjedt stílust vizsgálunk: a REST-et, amely erőforrások köré szervezi az API-t; a GraphQL-t, amely a klienseknek gráfszerűen bejárható adatteret kínál; és a gRPC-t, amely bináris, HTTP/2 alapú távoli eljáráshívást valósít meg. A cél nem az, hogy egyetlen „győztest” hirdessünk, hanem az, hogy a szemantikai különbségek, a tipikus hibák és az ellenőrizhető szempontok alapján tudjunk dönteni. A fejezetben bemutatott példák tanulási célúak, éles rendszeren csak a szervezeti irányelvek és biztonsági követelmények szerint szabad alkalmazni őket.

14.1.1 Erőforrás, gráf és RPC: három szemantikai világ

A három stílus három különböző absztrakciót helyez a középpontba:

- A **REST** (Representational State Transfer) az erőforrás fogalomra épül. Az erőforrás egy stabil, címképezhető entitás, amelyet URI azonosít, és amelynek állapotát reprezentációkban (például JSON, XML, HTML) közvetítjük. Az erőforrás módosítása a HTTP metódusok (GET, POST, PUT, PATCH, DELETE) uniform felületén keresztül történik. A állapotmentes szerverek és a hiperfolyamat (HATEOAS) révén a kliens a válaszokban található linkekből fedezheti fel a további lehetséges állapotátmeneteket. A REST tehát elsősorban erőforrás-orientált: a hangsúly a feladatok helyett a stabil entitásokon van.
- A **GraphQL** gráfként fogja fel az alkalmazás adatterét. A kliens egy típusos séma (schema) alapján deklaratív lekérdezést küld, amelyben pontosan megadja, milyen mezőkre és kapcsolódó objektumokra van szüksége. A szerveren a mezőkhöz resolver függvények tartoznak, amelyeket a motor párhuzamosan vagy szekvenciálisan hív meg. Az eredmény a lekérdezés alakját tükrözi, így nincs túl- vagy alul-lekérdezés. A GraphQL tehát gráf-orientált: a kliens a gráfot járja be explicit szelekcióval.
- A **gRPC** eljárás-orientált. A szerver szolgáltatásokat definiál, amelyek műveleteket (metódusokat) kínálnak. A kliens a műveletet távoli eljáráshívásként hívja, az átvitt adatokat pedig platformfüggetlen Protobuf üzenetekben írja le. A gRPC HTTP/2 felett szinkron kérés-válasz és négyféle streaming módot is támogat. A szerződést a Protobuf `service` definíció rögzíti, és a kliens- valamint szerveroldali kódot generátor állítja elő.

E három alapvető szemantika meghatározza a hibakezelés, a streaming és a kliensgenerálás megközelítését is, ezért a részletes tárgyalás előtt érdemes rögzíteni: a REST az erőforrás, a GraphQL a gráf, a gRPC pedig a művelet nyelvén beszél.

14.1.2 REST: erőforrások és a HTTP uniform felülete

A REST-et Roy Fielding disszertációja (2000) írta le építészeti stílusként, nem protokollként. A gyakorlatban azonban a legtöbb REST-ish API HTTP-t és JSON-t használ, és a Richardson-féle érettségi modell (0–3 szint) segít felmérni, mennyire következetesen alkalmazza a stílust. A 2. szinttől beszélhetünk erőforrás-alapú API-ról, ahol a HTTP metódusok és státuszkódok hordozzák a szemantikát; a 3. szint a HATEOAS-t is bevezeti.

Erőforrásmodell és URI-tervezés. A REST API az erőforrásokat hierarchikusan címzi. Például egy webshopban a `/rendelesek/{id}` erőforrás egyetlen rendelést, a `/rendelesek/{id}/tetelek` a rendelés tételeit jelöli. A gyűjtemények többes számú főnevet kapnak, a műveletek nem jelennek meg az URI-ban, mert azokat a HTTP metódusok fejezik ki. Helytelen lenne például `/rendelesek/{id}/torles` hívása DELETE helyett; ez kettős szemantikát eredményez, és megtöri az uniform felületet.

HTTP metódusok és idempotencia. A GET biztonságos és idempotens: nem módosítja a szerver állapotát, és ismételt hívása ugyanazt a reprezentációt adja. A PUT idempotens, de nem biztonságos: a megadott reprezentációt teljes egészében érvényesíti a cél URL-en. A DELETE idempotens, a POST viszont nem az; a POST jellemzően új erőforrást hoz létre vagy állapotot vált ki. A PATCH részleges módosítást végez, és gyakran nem idempotens, ezért a szerződésben explicit módon rögzíteni kell a megengedett műveleteket, például JSON Patch formátummal. Az idempotencia kulcsfontosságú a hálózati újrapróbálkozásnál: egy POST ismétlése duplikátum rendelést okozhat, ezért ilyen műveletekhez idempotencia kulcsot érdemes rendelni.

Reprezentáció és médiatípus. A reprezentáció a szerződés része: a JSON szerkezet, a mezőnevek, a típusok, a kötelező mezők és az érvényességi szabályok. A változások kezelésére verziózást alkalmazunk vagy az URI-ban (pl. `/v1/rendelesek`), vagy médiatípus-verzióval és tartalom-negotiation-nel. A médiatípusok (application/json, application/hal+json) lehetőséget adnak a reprezentáció szemantikájának gépi azonosítására is.

Hibakezelés. A REST hibakezelés a HTTP státuszkódok és a választest kombinációja. A kliens hibája (4xx) és a szerver hibája (5xx) szétválasztása alapvető. A gyakori státuszkódok mellett (200, 201, 204, 400, 401, 403, 404, 409, 429, 500, 503) ajánlott a hiba részleteit szabványosított formában visszaadni, például az RFC 7807 `application/problem+json` médiatípussal. Egy ilyen válasz tartalmazza a hiba típusát (type), címét (title), státuszát (status), részleteit (detail) és a hibás mezőt (invalid-params). A státuszkód és a hibatest együttesen teszi a hibát géppel feldolgozhatóvá. A REST-nél a hálózati hibát, az időtúllépést és a szerver 5xx válaszát a kliensnek elkülönítetten kell kezelnie, és újrapróbálkozási stratégiát kell alkalmaznia a nem idempotens műveleteknél csak az idempotencia kulcs birtokában.

Streaming. A REST hagyományos kérés-válasz modellje nem alkalmas valós idejű adatfolyamra, de két kiterjesztés létezik. A HTTP chunked transfer encoding lehetővé teszi a válasz folyamatos, darabokban történő küldését, ami nagy fájloknál hasznos. A Server-Sent Events (SSE) egyirányú, szöveges eseményfolyamot biztosít a szervertől a kliens felé, és a böngészőben natívan támogatott. Az SSE alkalmas például rendelésállapot-változások vagy naplófolyam közvetítésére. A kétirányú valós idejű kommunikációhoz a REST nem elegendő; ilyenkor WebSocket, gRPC streaming vagy más megoldás szükséges.

Kliensgenerálás. Az OpenAPI Specification (OAS, korábban Swagger) gépileg olvasható leírást ad a REST API-ról: útvonalak, metódusok, paraméterek, reprezentációk, hitelesítési módok. Ebből a leírásból számos eszköz generál klienskönyvtárat, szervervázat, teszteket és dokumentációt. A kliensgenerálás akkor működik jól, ha a specifikációt a kód forrásának tekintjük, és mind a szervert, mind a klienst ebből származtatjuk. A gyakori hiba, hogy a kézzel írt kódból

utólag generálnak specifikációt, ami idővel eltér a valóságtól. A generált kliens csökkenti a kézi JSON-elemzés hibáit, és a típusbiztonságot is javítja.

A REST erősségei: széleskörű eszköztámogatás, gyorsítótárazhatóság, ember számára olvasható reprezentáció, uniform felület. Hátrányai: a túl- és alul-lekérdezés (over-fetching/under-fetching), az erőforrás-gráf bejárásának nehézségei, a HATEOAS gyakori hiánya, a bináris hatékonyság hiánya.

14.1.3 GraphQL: a gráf mint lekérdezési felület

A GraphQL-t a Facebook fejlesztette ki 2012-ben, és 2015-ben nyílt forráskódúvá tette. Az alapötlet az, hogy a kliens a szerver adatmodelljét gráfként ismeri, és egyetlen kérésben pontosan azt a részgráfot kéri le, amelyre szüksége van. A GraphQL a lekérdezési nyelv és a futtatókörnyezet kombinációja; a szerveren egy séma határozza meg a típusokat és a kapcsolódásokat, a kliens pedig e séma alapján ír query-t, mutation-t vagy subscription-t.

Séma és típusrendszer. A GraphQL séma erősen típusos, és dokumentálja a kliens számára elérhető adatokat. Például:

```
type Rendeles {
  id: ID!
  vevo: Vevo!
  tetelek: [Tetel!]!
  osszeg: Money!
  allapot: String!
}

type Query {
  rendeles(id: ID!): Rendeles
  rendelesek(szuro: RendelesSzuro): [Rendeles!]!
}
```

A mezőkhöz resolver függvények tartoznak, amelyek az adatforrásból (adatbázis, másik szolgáltatás, cache) kinyerik az értéket. A resolverek kombinációja hozza létre a gráfot: a `Query.rendeles` resolver visszaad egy `Rendeles` objektumot, a `Rendeles.tetelek` resolver pedig a rendelés tételeit. A kliens oldalán a query:

```
query {
  rendeles(id: "42") {
    id
    vevo { nev email }
    tetelek { termek neve mennyiseg }
    osszeg { ertek penznem }
  }
}
```

Itt a kliens csak a szükséges mezőket kéri, elkerülve a túl-lekérdezést. A válasz alakja pontosan megegyezik a query alakjával, ami megkönnyíti a kliensoldali kötést.

Hibakezelés. A GraphQL hibakezelése eltér a REST-től: a HTTP státuszkód általában 200, még üzleti hiba esetén is, mert a kérés sikeresen feldolgozásra került. A hibák a válasz `errors` tömbben jelennek meg, míg a `data` mező a sikeresen feloldott adatot tartalmazza (részleges siker). A hibákhoz tartozhat path, locations és extensions, amelyek segítenek a hiba forrásának azonosításában. Az üzleti hibákat gyakran a séma szintjén modellezik, például union típusokkal (pl. `RendelesEredmeny = Rendeles | RendelesHiba`), hogy a kliens típusbiztosan tudja kezelni a hibákat. A szerveroldali hibák – mint egy adatbázis-kapcsolat megszakadása – általában a teljes `data` mezőt null-ra állítják, és a hiba az `errors`-ban jelenik meg. A GraphQL

specifikáció nem ír elő kötelező hibakódokat, ezért a csapatnak saját konvenciót kell kialakítania, például az `extensions.code` mezőben.

N+1 probléma és DataLoader. A gráf-rezolverek naiv megvalósítása gyakran vezet az N+1 lekérdezési problémához: a `rendelesek` query minden rendeléshez külön hívja a `Rendeles.vevo` resolvert, ami külön adatbázis-hívást generál. A megoldás a batch betöltés: a DataLoader mintával a resolverek összegyűjtik az azonos típusú kulcsokat, és egyetlen lekérdezéssel töltik be az összes kapcsolódó objektumot. A GraphQL futtatókörnyezetek gyakran támogatják ezt beépített vagy könyvtári eszközökkel.

Streaming és subscription. A GraphQL a valós idejű adatfolyamot a `subscription` művelettel támogatja. A kliens feliratkozik egy eseményre, és a szerver folyamatosan küldi az eredményeket, általában WebSocket vagy HTTP/2 felett. A subscription ugyanúgy a séma része, mint a query és a mutation, de a resolver egy eseményfolyamot ad vissza. A GraphQL subscription alkalmas például rendelésállapot-változások valós idejű megjelenítésére, de a skálázhatóság és a pontosan egyszeri kézbesítés kihívásokat jelent.

Kliensgenerálás. A GraphQL sémából kódgenerátorok (GraphQL Code Generator, Apollo Tooling stb.) típusos klienskönyvtárat állítanak elő. A generált kód tartalmazza a query-k, mutation-ök és subscription-ök típusait, így a kliensoldali fejlesztés típusbiztos. A gyakori hiba, hogy a kliens a szöveges query-t kézzel építi, elveszítve a sémaellenőrzés előnyeit. A generátorok azonban feltételezik, hogy a séma a forrás, és a query-ket is verziózott eszközökkel kezelik.

A GraphQL erősségei: pontos adatlekérés, egyetlen kérés a kapcsolódó gráf bejárására, erős séma, kiváló fejlesztői élmény. Hátrányai: komplexitás a szerveroldalon, a gyorsítótárazás nehézsége (a HTTP cache kevésbé alkalmazható), a szerver teljesítménye nagy, mély query-k esetén, a szolgáltatás megszakadásakor a részleges hibák kezelése.

14.1.4 gRPC: szerződéses RPC HTTP/2 felett

A gRPC a Google által fejlesztett, 2015-ben nyílt forráskódúvá tett RPC keretrendszer. A szerződés Protobuf (Protocol Buffers) üzenetekkel és szolgáltatásdefiníciókkal írható le, a kommunikáció pedig HTTP/2 felett zajlik, bináris, hatékony formátumban. A gRPC célja a nagy teljesítményű szerver-szerver kommunikáció, ahol a hálózati késleltetés és a sávszélesség kritikus.

Protobuf és szolgáltatásdefiníció. A Protobuf erősen típusos üzenetformátum, amelyet a `.proto` fájl ír le. Például:

```
syntax = "proto3";

service RendelesSzolgaltatas {
  rpc GetRendeles(GetRendelesRequest) returns (GetRendelesResponse);
  rpc ListRendelesek(ListRendelesRequest) returns (stream ListRendelesResponse);
}

message GetRendelesRequest {
  string rendeles_id = 1;
}
```

A szolgáltatás metódusai különböző streaming módokat támogatnak: unary (egy kérés, egy válasz), szerver streaming (egy kérés, több válasz), kliens streaming (több kérés, egy válasz) és kétirányú streaming (több kérés, több válasz). A mezők számozottak, ami előre- és visszafelé kompatibilis evolúciót tesz lehetővé a mezők hozzáadásával és a régi mezők megtartásával.

Hibakezelés. A gRPC hibakezelés a hívás oldalán hibakódot és opcionális hibaüzenetet ad, a HTTP/2 feletti állapotkódtol függetlenül. A gRPC státuszkódok (OK, CANCELLED,

UNKNOWN, INVALID_ARGUMENT, DEADLINE_EXCEEDED, NOT_FOUND, ALREADY_EXISTS, PERMISSION_DENIED, RESOURCE_EXHAUSTED, FAILED_PRECONDITION, ABORTED, OUT_OF_RANGE, UNIMPLEMENTED, INTERNAL, UNAVAILABLE, DATA_LOSS, UNAUTHENTICATED) hasonlóak a HTTP státuszkódokhoz, de a gRPC saját hibakód rendszert használ. A hiba részleteit a `google.rpc.Status` és a hozzá kapcsolódó `google.rpc.ErrorInfo`, `google.rpc.BadRequest` stb. üzenetekben lehet átadni. A kliensoldalon a hívás hibát kap, ha a szerver nem OK státusszal válaszol. A streaming hívásoknál a hiba megszakítja az adatfolyamot, és a kliens a hibát a streaming lezárásakor kapja meg.

Streaming. A gRPC natívan támogatja a négyféle streaming módot, ami jelentős előny a REST-tel és a GraphQL-lel szemben. A szerver streaming alkalmas például nagy listák lapozás nélküli átadására vagy folyamatos események közvetítésére. A kliens streaming lehetővé teszi, hogy a kliens több üzenetet küldjön folyamatosan, például fájlfeltöltésnél. A kétirányú streaming pedig valós idejű, párbeszédes protokollt tesz lehetővé, például chat vagy játékállapot-szinkronizálás. A HTTP/2 multiplexelés révén több kérés is folyhat egyetlen TCP kapcsolaton, csökkentve a kapcsolati költséget.

Kliensgenerálás. A gRPC esetében a kódgenerálás központi szerepet játszik: a `protoc` fordító és a megfelelő pluginok a `.proto` fájlból előállítják a szerver interfészt és a kliens stubok könyvtárát a támogatott nyelvekhez (Java, Go, Python, C++, C#, JavaScript/TypeScript, stb.). A generált kód garantálja a típusbiztonságot és a szerződés betartását. A generálás folyamatát a build pipeline-ban kell automatizálni, hogy a `.proto` fájl változásai azonnal újraépítsék a klienst és a szervert. A gRPC tükrözés (server reflection) lehetővé teszi a futó szolgáltatás szerződésének megismerését, ami hasznos fejlesztői eszközöknél.

A gRPC erősségei: nagy teljesítmény, bináris sorosítás, erős típusosság, kiváló streaming támogatás, automatikus kódgenerálás. Hátrányai: a HTTP/2 és a bináris protokoll korlátozza a böngészőből való közvetlen hívást (bár a gRPC-Web bridges létezik), a hibakeresés fontosabb, mivel az üzenetek nem ember számára olvashatóak, a terheléselosztás és a gateway megoldások komplexebbek.

14.1.5 Összehasonlítás: mikor melyiket?

A három stílus közötti választás a használati esettől, a kliens típusától, a hálózati feltételektől és a csapat képességeitől függ. Az alábbi döntési szempontok segítenek a tudatos választásban:

- **Kliens típusa:** ha a fogyasztók böngésző alapú frontendek vagy külső partnerek, a REST széleskörű támogatást és egyszerű integrációt kínál. Ha a kliens egy mobilalkalmazás, amely változatos képernyőkkel és adatigényekkel rendelkezik, a GraphQL csökkenti a hálózati forgalmat és a kerekasztal-forgalmat. Ha a kliens egy másik belső szolgáltatás vagy mikroservice, a gRPC a nagy teljesítmény és a típusbiztonság miatt kedvező.
- **Adatelérési minta:** ha az API erőforrások stabil halmazát kezeli, és a kliensek ritkán járják be mélyen a kapcsolódó gráfot, a REST megfelelő. Ha az adatok erősen összekapcsoltak, és a kliensek nagyon eltérő nézeteket igényelnek, a GraphQL előnyös. Ha a kommunikáció elsősorban műveletvezérelt, például parancsok és események, a gRPC jobban illeszkedik.
- **Sávszélesség és késleltetés:** a gRPC bináris protokollal és HTTP/2-vel a legjobb teljesítményt nyújtja; a REST JSON nagyobb, de gyorsítótárazható; a GraphQL lehetővé teszi a pontos lekérdezést, de a szerveroldali resolver-hívások költségesek lehetnek.
- **Hibakezelés:** a REST és a gRPC tiszta, szabványos hibakódokat ad; a GraphQL hibakezelése részben a válaszban történik, ami bonyolultabb, de lehetővé teszi a részleges eredményt.
- **Streaming:** a gRPC a legerősebb streaming támogatással rendelkezik; a GraphQL subscription-ök valós idejű igényekre jók; a REST csak korlátozottan (SSE) alkalmas.

Az alábbi Mermaid folyamatábra a döntési logikát szemlélteti:

```
flowchart TD
    A[Ki a kliens és mi az adatminta?] --> B{Böngésző / külső partner, stabil erőforrások}
    B -->|Igen| C[REST választás]
    B -->|Nem| D{Mobil / összekapcsolt gráf, eltérő view-ok}
    D -->|Igen| E[GraphQL választás]
    D -->|Nem| F{Szerver-szerver, nagy teljesítmény, streaming}
    F -->|Igen| G[gRPC választás]
    F -->|Nem| H[Kevert megközelítés: pl. API gateway mögött több stílus]
```

Fontos, hogy a stílusok nem zárják ki egymást: egy API gateway mögött párhuzamosan létezhet REST a külső feleknek, GraphQL a mobilklienseknek, és gRPC a belső szolgáltatások között. A gateway feladata a protokollok közötti leképezés, a hitelesítés és a terheléselosztás.

14.1.6 Labor: három szerződés ugyanarra a műveletre

Cél. Készíts három szerződést ugyanarra az üzleti műveletre: egy rendelés lekérésére azonosító alapján. A művelethez tartozzon REST OpenAPI, GraphQL séma és gRPC Proto definíció. A labor eredményét reprodukálható dokumentumban vagy jegyzetfüzetben kell rögzíteni, a kontextus, a döntési napló és az ellenőrzési szempontok feltüntetésével.

Kiinduló állapot. Rendelkezésre áll egy üres projekt, a művelethez tartozó entitás: `Rendeles` {id, vevo, tetelek, osszeg, allapot}. A tételekhez tartozik: `Tetel` {termek, mennyiseg, ar}. A művelet neve: `RendelesLekeres`.

Elvárt eredmény.

1. **REST OpenAPI specifikáció** (YAML vagy JSON), amely leírja a `GET /rendelesek/{rendelesId}` végpontot, a 200-as válasz reprezentációját, a 404-es hibát, valamint az idempotens GET szemantikát.
2. **GraphQL séma** (Schema Definition Language), amely tartalmazza a `Query.rendeles` mezőt, a kapcsolódó `Rendeles` és `Tetel` típusokat, valamint egy hibakezelő union típust vagy az `errors` mechanizmus használatát.
3. **gRPC Proto definíció**, amely definiálja a `RendelesSzolgaltatas.GetRendeles` unary RPC-t, a `GetRendelesRequest` és `GetRendelesResponse` üzeneteket, valamint a hibaállapotok lehetséges kódjait (például `NOT_FOUND`).
4. **Döntési napló** (ADR formátumban), amely rögzíti, hogy a három stílus közül melyiket választanád az adott környezetre (például mobil kliens, belső szolgáltatások, nyilvános API), indoklással és a feladott alternatívákkal.
5. **Ellenőrzési szempontok** felsorolása: a választott stílusnál milyen hibák, streaming és kliensgenerálási szempontok igazolhatók.

Mért és várt eredmény elkülönítése. A labor során nem mért adatot használunk, ezért a várt eredményt tüntesd fel: például „A GraphQL query egyetlen kérésben lekéri a rendelést és a tételeit, a REST-hez képest elkerülve az 1+N hívást.” A mért eredményt csak akkor rögzítsd, ha ténylegesen lefuttattál valamilyen lekérdezést vagy generálást, a környezet és az időpont megadásával. A fiktív mérést jelöld becslésként vagy hipotetikus eredményként.

Visszaállítási út. A labor nem módosít éles adatot; az állapot a forrásfájlok törlésével vagy a git repository visszaállításával állítható helyre.

14.1.7 Ellenőrzőlista

- A REST API erőforrásokat használ, a HTTP metódusok szemantikailag helyesek.
- A hibakezelés mindhárom stílusnál konzisztens: státuszkódok, hibatest, gRPC hibakódok.
- A streaming igényt időben azonosítottuk, és a megfelelő stílust választottuk (gRPC streaming, GraphQL subscription, SSE).
- A kliensgenerálás a szerződésből történik, a generált kód a build része.
- Az idempotencia és az újrapróbálkozási stratégia a műveleteknél tisztázott.
- A dokumentáció nem tartalmaz valós hitelesítő adatot, belső URL-t vagy ügyfélnevet.

14.1.8 Önellenőrző kérdés

Egy belső rendszerben egy rendeléskezelő szolgáltatásnak valós idejű eseményeket kell közvetítenie a raktárkezelő rendszer felé, és a kommunikációnak nagy teljesítményűnek, típusbiztosnak, szerver-szerver jellegűnek kell lennie. Melyik stílust választanád, és hogyan indokolnád a REST-tel és a GraphQL-lel szemben? Válaszodban térj ki a streaming, a hibakezelés és a kliensgenerálás szempontjaira.

14.2 API-verziózás, szerződés és schema evolution

Az API akkor marad fenntartható, ha a csapat a felületet nem csupán kódnak, hanem explicit szerződésnek tekinti. Ez a szerződés nem állandó: az üzleti követelmények, a fogyasztói igények és a technológiai környezet folyamatosan változnak. A fejezet célja, hogy a kompatibilitás és a változás kezelését ne utólagos tűzoltásként, hanem tervezhető, mérhető és tesztelhető folyamatként mutassa be. A fogalmakat egy katalógus-szolgáltatás példáján keresztül vezetjük végig, a laborban pedig a verziózási szabályzatot, a séma-diffet és a szerződéstesztet egy reprodukálható dokumentumban rögzítjük.

14.2.1 Változás és kompatibilitás: alapfogalmak

Egy API szerződése több rétegből áll:

- **Szintaktikai szerződés:** végpontok, HTTP-metódusok, státuszkódok, paraméterek, fejlécek, kérés- és válaszsémák.
- **Szemantikai szerződés:** mit jelent egy mező, milyen üzleti szabályok érvényesek, milyen hiba feltételek mellett mit ad vissza a rendszer.
- **Viselkedési szerződés:** időzítés, sorrend, idempotencia, tranzakciós határok, hibakezelés.
- **Nem funkcionális szerződés:** rendelkezésre állás, késleltetés, átviteli korlát, hitelesítés, jogosultság, naplózás.

A változáskezelés során elsősorban azt kell eldönteni, hogy egy adott változás **kompatibilis** vagy **inkompatibilis** a meglévő kliensekkel. Ehhez a backward és forward compatibility fogalmait használjuk.

14.2.2 Backward compatibility

A **backward compatibility** (visszafelé kompatibilitás) azt jelenti, hogy az új szerveroldal továbbra is képes kiszolgálni a régi kliensek kéréseit, és azok a megszokott módon működnek. Ha egy API-változás backward compatible, a klienseknek nem kell azonnal frissíteniük. A fogalom a szolgáltató és a fogyasztó közötti szerződés aszimmetriájából fakad: a szervert gyakran egyetlen csapat üzemelteti, míg a klienseket sok különböző csapat, külső partner vagy mobilalkalmazás használja. Az utóbbiak frissítési ciklusa lassú, ezért a szervernek kell toleránsnak lennie.

Nézzünk egy példát egy termék-katalógus API-ban. A régi kliens ezt várja:

```
{
  "id": "termek-123",
  "nev": "Mechanikus billentyűzet",
  "ar": 24990
}
```

A szerver új verziója hozzáad egy `keszlet` mezőt a válaszhoz:

```
{
  "id": "termek-123",
  "nev": "Mechanikus billentyűzet",
  "ar": 24990,
  "keszlet": 42
}
```

Ez **backward compatible**, mert a régi kliens a JSON-válaszban ismeretlen mezőt figyelmen kívül tud hagyni. Hasonlóan kompatibilis, ha:

- új, opcionális lekérdezési paraméter jelenik meg,
- új válaszmező kerül a sémába,
- a válaszban új, eddig nem használt státuszkód jelenik meg (például `202 Accepted` a korábbi `200 OK` mellett),
- a szöveges hibaüzenet pontosítása nem változtatja meg a hibakódot.

Tipikus **breaking** (inkompatibilis) változások:

- mező törlése vagy átnevezése a válaszból,
- mező típusának megváltoztatása (például `ar` számról szövegre),
- új kötelező mező bevezetése a kérésben,
- enum-érték eltávolítása vagy átnevezése,
- a végpont URL-jének megváltoztatása,
- a válasz státuszkódjának megváltoztatása (például `200` helyett `201`),
- a hitelesítési mód megváltoztatása,
- a kérés vagy válasz tartalomtípusának megváltoztatása.

A gyakorlatban a legtöbb változás megítélése nem fekete-fehér: például egy mező értelmezésének módosulása (a `ar` mostantól bruttó ár) átmegy a szemantikai szerződésen, és hiába szintaktikailag kompatibilis, a régi kliens rossz döntést hozhat. Ezért a kompatibilitást sosem csak a séma szintjén kell vizsgálni.

14.2.3 Forward compatibility

A **forward compatibility** (előrefelé kompatibilitás) azt jelenti, hogy a régi szerver el tudja fogadni az új kliensek kéréseit, és a régi kliensek is képesek együttműködni az új szerverrel. A fogalom gyakran együtt jár a **tolerant reader** elvvel: a fogyasztó csak azokat a mezőket olvassa, amelyeket ért, az ismeretlen mezőket figyelmen kívül hagyja, és nem tör el attól, ha a válasz több adatot tartalmaz, mint amire számított.

Példa: a régi szerver csak a `nev`, `ar` és `id` mezőket ismeri. Az új kliens elküldi a `keszlet` mezőt is, mert az újabb API-verzióban már létezik. Ha a szerver a JSON-objektumot szigorú sémavalidálással kezeli, és az ismeretlen mezőt hibaként elutasítja, akkor a rendszer nem forward compatible. Ha azonban a szerver figyelmen kívül hagyja az ismeretlen mezőket, a régi és az új kliens is képes lesz kommunikálni.

A forward compatibility különösen fontos:

- **mobilalkalmazásoknál**, ahol a felhasználók nem frissítenek azonnal,
- **elosztott rendszerekben**, ahol a szerverek és kliensek különböző verziói egyszerre futnak,
- **eseményvezérelt architektúrákban**, ahol a producerek és consumerek egymástól függetlenül fejlődnek.

A két fogalom együtt adja a **teljes kompatibilitást**: ha az új szerver visszafelé kompatibilis, a régi kliens továbbra is működik; ha a régi szerver előrefelé kompatibilis, az új kliens is működik. A gyakorlatban a legtöbb rendszer a következő mátrixot használja:

Szerver verzió	Kliens verzió	Működik?	Feltétel
új	régi	igen, ha a szerver backward compatible	a régi kliens nem használ új mezőt
régi	új	igen, ha a szerver forward compatible	a régi szerver elviseli az ismeretlen mezőket
új	új	igen	teljes szerződés
régi	régi	igen	eredeti szerződés

A nem kompatibilis párosításokat a verziózási stratégia kezeli.

14.2.4 Kompatibilis és inkompatibilis változások osztályozása

A változáskezelés első lépése egy **osztályozási keret** kialakítása. A leggyakoribb felosztás:

- **Additív változás:** új elem hozzáadása – általában kompatibilis, ha opcionális.
- **Destruktív változás:** meglévő elem eltávolítása vagy módosítása – általában inkompatibilis.
- **Szemantikai változás:** a jelentés változik meg – a kompatibilitás a kontextustól függ.

Egy gyakorlati szabály: **a mező típusa, neve és szemantikája soha nem változhat meg; csak új, opcionális mezők jelenhetnek meg.** Ez a szabály leegyszerűsíti a séma-evolúciót, de nem oldja meg az összes esetet. Például egy enum új értékkel való bővítése additívnak tűnik, de a régi kliens kapcsoló utasítása felrobbanhat, ha nem rendelkezik alapértelmezett ággal. Ezért

az enum-értékek hozzáadását is gyakran breaking változásként kezelik, vagy a klienseket arra kérik, hogy mindig legyen `default` águk.

14.2.5 Verziózási stratégiák

A verziózás célja, hogy a kliensek és a szerver közötti szerződést egyértelmű azonosítóval lássa el, és lehetővé tegye több verzió párhuzamos futtatását. A leggyakoribb megközelítések:

14.2.6 URI-verzió

A verzió megjelenik az URL-ben:

- Path-szegmensben: `https://api.example.com/v1/products`
- Query paraméterben: `https://api.example.com/products?version=1`
- Al-domainben: `https://v1.api.example.com/products`

A path-szegmens a legelterjedtebb, mert azonnal látható, egyszerűen cache-elhető és proxyzható. Hátránya, hogy a verzió a erőforrás-azonosító részévé válik, és a HATEOAS-hivatkozásokban is meg kell jelentenie. Egyes tervezők szerint a verzió nem az erőforrás része, ezért nem szerencsés az URL-be tenni, de a gyakorlatban az egyszerűsége miatt messze a leggyakoribb.

14.2.7 Fejléc-verzió

A verzió egyéni HTTP-fejlécben szerepel:

```
GET /products HTTP/1.1
Host: api.example.com
Api-Version: 2025-03-01
```

A dátum alapú verzió (például `2025-03-01`) a naptári változást jelöli, és jól illeszkedik a folyamatos API-verziózáshoz. A fejléc megközelítés előnye, hogy az URL tiszta marad, és a verzió a kérés metaadataként kezelhető. Hátránya, hogy a kliensek nehezebben állítják be, a cache és a proxy rétegeknek külön kell kezelniük, és a böngészőben futó kliens nem mindig tud egyéni fejlécet küldeni.

14.2.8 Media type verzió (content negotiation)

A verzió a `Accept` fejlécben jelenik meg:

```
GET /products HTTP/1.1
Accept: application/vnd.example.product+json;version=1
```

Ez a legkövetkezetesebb REST-es megoldás, mert a verzió a reprezentáció formátumához tartozik. Hátránya, hogy bonyolultabb, kevésbé olvasható, és a klienseknek pontosan kell megadniuk a media type-ot. Kis csapatoknál gyakran túlzottnak tartják.

14.2.9 Szemantikus verziózás az API szintjén

A **SemVer** (`MAJOR.MINOR.PATCH`) szoftververziózási konvenciót API-kontextusban is használják:

- **MAJOR** növelés: inkompatibilis változás (breaking change).
- **MINOR** növelés: kompatibilis, additív változás (például új mező, új végpont).
- **PATCH** növelés: belső javítás, amely nem változtatja meg a szerződést.

Az API-verzióban a MAJOR szám a leggyakrabban használt. A MINOR és PATCH a dokumentációban és a release notes-ban jelenik meg, de a kliensnek csak a MAJOR verziót kell ismernie, ha a szerződés kompatibilis marad.

14.2.10 Deprecation és sunset politika

A verziókezelés nem ér véget az új verzió bevezetésével. A régi verziót fokozatosan kell kivonni, hogy a klienseknek legyen idejük migrálni. A **deprecation** azt jelenti, hogy a régi verzió továbbra is elérhető, de a dokumentáció jelzi, hogy a támogatása hamarosan megszűnik. A **sunset** a végleges kikapcsolás időpontja.

Egy jó gyakorlat:

1. **Bejelentés:** a deprecationt a dokumentációban, a válasz fejlécében (`Deprecation`, `Sunset`) és a release notes-ban is jelezzük.
2. **Türelmi idő:** a MAJOR verziók között legalább 6-12 hónapot hagyunk, hogy a külsős kliensek is frissíthessenek.
3. **Figyelmeztetés:** a sunset előtt fokozatosan növeljük a figyelmeztetést (például `Warning` fejléc, naplózás).
4. **Végső kikapcsolás:** a megadott időpont után a régi verzió `410 Gone` vagy `404 Not Found` választ ad.

A deprecation folyamatát is dokumentálni kell, és a klienseknek világos migrációs útvonalat kell biztosítani.

14.2.11 Séma evolúció és szerződés

A séma evolúció azt vizsgálja, hogy egy adatséma hogyan változhat anélkül, hogy a jelenlegi adatok és kliensek eltörnének. A fogalom szorosan kapcsolódik az **evolvability** (fejleszthetőség) minőségi attribútumhoz: milyen alacsony költséggel tudjuk a rendszert új követelményekhez igazítani.

14.2.12 Sémaformátumok és kompatibilitási tulajdonságaik

- **JSON/OpenAPI:** a JSON séma evolúciója az additív mezőkre épít. A mezők sorrendje nem garantált, a hiányzó mezők `null`-ként vagy alapértelmezettként kezelhetők. A JSON-nak nincs beépített sémaevolúciós mechanizmusa, ezért a szabályokat a csapatnak kell rögzítenie.
- **Protocol Buffers (Protobuf):** a mezők sorszámot kapnak. Új mező hozzáadása új sorszámmal kompatibilis, a régi mező eltávolítása csak akkor, ha a sorszámot nem használják fel újra. Az enum-oknál hasonlóan a sorszám a stabil azonosító.

- **Avro:** a séma verziója explicit, és a writer/reader séma feloldással dolgozik. A reader séma határozza meg, hogy a fogyasztó mit olvas, a writer séma pedig, hogy a producer mit írt. Az ismeretlen mezők automatikusan figyelmen kívül maradnak.
- **AsyncAPI:** az események üzenetsémái ugyanazokat az evolúciós szabályokat követik, de a producer és a consumer közötti irányt is figyelembe kell venni.

14.2.13 Kompatibilis evolúciós szabályok JSON API-knál

Egy JSON alapú REST API-nál a következő szabályok betartásával a legtöbb változás kompatibilis marad:

1. **Új mező hozzáadása a válaszhoz:** engedélyezett, a régi kliensek figyelmen kívül hagyják.
2. **Új opcionális paraméter a kéréshez:** engedélyezett, a régi kliensek nem küldik el, a szerver alapértelmezést használ.
3. **Új végpont hozzáadása:** engedélyezett, a régi klienseket nem érinti.
4. **Mező átnevezése:** tilos. Helyette új mezőt kell felvenni, és a régit deprecálni.
5. **Mező típusának megváltoztatása:** tilos. Helyette új mezőt kell felvenni más néven.
6. **Mező eltávolítása:** tilos, amíg a deprecation és a sunset ciklus le nem zárult.
7. **Enum-érték hozzáadása:** óvatosan, mert a régi kliens `switch` utasítása eltörhet. Ha a kliens nem készült fel rá, a hozzáadás breaking.
8. **Státuszkód megváltoztatása:** tilos, hacsak a régi kliens nem kezeli az új kódot is.
9. **Hibaválasz szerkezetének megváltoztatása:** tilos, mert a kliensek a hibakezelést erre építik.

A szabályok alkalmazása attól is függ, hogy a kliensek mennyire szigorúan validálják a válaszokat. Ha a kliens JSON Schema-val ellenőrzi a választ, és az ismeretlen mezőket elutasítja, akkor a válasz új mezővel való bővítése is breaking. Ezért a **tolerant reader** elv a kliens oldalán kötelező: a kliens ne szigorúan validáljon, hanem csak a szükséges mezőket ellenőrizze.

14.2.14 Tolerant reader és a Postel-törvény

A Postel-törvény („légy konzervatív abban, amit küldesz, és liberális abban, amit elfogadsz”) az API-tervezésben azt jelenti, hogy a szerver csak a szerződésben rögzített mezőket küldje, a kliens pedig az ismeretlen mezőket tolerálja. Ez a fajta rugalmasság segíti a fokozatos evolúciót, de veszélye is van: ha a kliens mindent elnéz, a hibák és a szemantikai eltérések rejtve maradnak. Ezért a toleranciát szerződéstesztekkel és explicit séma-dokumentációval kell ellensúlyozni.

14.2.15 OpenAPI és AsyncAPI

14.2.16 OpenAPI

Az OpenAPI specifikáció a REST API-k szerződésének géppel olvasható leírása. Tartalmazza az útvonalakat, a paramétereket, a kérés- és válaszsémákat, a hitelesítési módokat és a példákat. A verziózás szempontjából az OpenAPI fájl a szerződés **egyetlen igazságforrása**, amelyből kódgenerálás, dokumentáció és validálás is végezhető.

Egy termék-katalógus API 1.0-s OpenAPI részlete:

```yaml
openapi: 3.1.0
info:
  title: Termék-katalógus API
  version: 1.0.0
paths:
  /products/{productId}:
    get:
      summary: Egy termék lekérdezése
      parameters:
        - name: productId
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Sikeres válasz
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
components:
  schemas:
    Product:
      type: object
      required: [id, nev, ar]
      properties:
        id:
          type: string
        nev:
          type: string
        ar:
          type: integer
```

A séma evolúció során az OpenAPI fájl diffjét kell elemezni. Ha a csapat bevezet egy `keszlet` mezőt, az a `Product` sémában új, opcionális propertyként jelenik meg. Az OpenAPI diff eszközök ezt additív, kompatibilis változásként jelzik. Ha viszont a `ar` mező típusa `integer`-ről `number`-re változik, az már breaking.

A gyakorlatban a következő folyamat javasolt:

1. A szerződést egy verziózott fájlban tartjuk (például `openapi.yaml` a git repóban).
2. Minden pull requestnél lefuttatunk egy **OpenAPI diff** ellenőrzést, amely összehasonlítja a régi és az új specifikációt.
3. Ha a diff breaking változást talál, a build megszakad, és a fejlesztőnek vagy MAJOR verziót kell növelnie, vagy kompatibilissé kell alakítania a változást.
4. Az elfogadott specifikációból generáljuk a dokumentációt, a szerver- és kliensvázakat, valamint a mock szervert.

14.2.17 AsyncAPI

Az AsyncAPI az eseményvezérelt API-k szerződésének leírására szolgál. REST API-knál a kérés-válasz párokat írjuk le, az AsyncAPI pedig a **csatornákat** (topic-okat, queue-okat), az üzeneteket, a fejléceket és a publish/subscribe irányokat.

Egy termék-készletváltozás esemény sémája:

```yaml
asyncapi: 3.0.0
info:
```

```yaml
  title: Készlet-események API
  version: 1.0.0
channels:
  product/stock:
    publish:
      message:
        name: StockChanged
        payload:
          type: object
          required: [productId, keszlet]
          properties:
            productId:
              type: string
            keszlet:
              type: integer
```

Az eseményeknél a kompatibilitás iránya a REST-től eltérően is alakulhat. Ha egy producer új mezőt ad az üzenethez, a régi consumereknek figyelmen kívül kell hagyniuk. Ha egy consumer új mezőt szeretne, de a producer még nem küldi, akkor a consumernek alapértelmezett értéket kell használnia. A séma evolúció szabályai ugyanazok, mint a JSON-nál, de a csatorna nevének megváltoztatása, a mező típusának módosítása vagy az üzenet séma kötelezővé tétele az eseményeknél is breaking.

Az AsyncAPI lehetővé teszi az események **szerződés alapú tesztelését** is: a consumer a várt üzenetsémát rögzíti, a producer pedig ellenőrzi, hogy a ténylegesen publikált üzenet megfelel-e neki.

14.2.18 Szerződésteszt

A szerződésteszt azt ellenőrzi, hogy két függetlenül fejlesztett fél (a szerver és a kliens) továbbra is ugyanazt a szerződést érti-e. A leghatékonyabb forma a **consumer-driven contract testing**, ahol a fogyasztó (consumer) határozza meg, hogy milyen kéréseket küld és milyen válaszokat vár, a szolgáltató (provider) pedig ezeket a szerződéseket verifikálja.

A folyamat tipikusan így zajlik egy Pact-szerű eszközzel:

1. A **consumer** csapata a saját tesztjeiben rögzíti az elvárásait: adott kérésre milyen választ vár. Az eszköz ebből létrehoz egy szerződésfájlt.
2. A szerződésfájlt a **provider** csapata megkapja (általában egy közös artifact repón vagy a CI/CD pipeline-on keresztül).
3. A **provider** csapata a saját tesztkörnyezetében lefuttatja a verifikációt: a szerződésben szereplő kéréseket elküldi a provider API-nak, és ellenőrzi, hogy a válasz megfelel-e az elvárásnak.
4. Ha mindkét oldal zöld, a szerződés érvényes; ha nem, az érintett félnek kell változtatnia.

Egy egyszerű Pact szerződés kódja (példa, JavaScript):

```javascript
const { Pact } = require('@pact-foundation/pact');
const provider = new Pact({
  consumer: 'webes-kliens',
  provider: 'termek-katalogus',
  port: 1234,
  log: 'pact.log',
  dir: 'pacts'
});

describe('Termék lekérdezése', () => {
  it('visszaadja a terméket id, nev és ar mezőkkel', async () => {
```

```
    await provider.addInteraction({
      uponReceiving: 'GET /products/termek-123',
      withRequest: {
        method: 'GET',
        path: '/products/termek-123'
      },
      willRespondWith: {
        status: 200,
        headers: { 'Content-Type': 'application/json' },
        body: {
          id: 'termek-123',
          nev: 'Mechanikus billentyűzet',
          ar: 24990
        }
      }
    });
    // ... kliens hívása a mock providerhez
  });
});
```

A provider oldalon a verifikáció így történik:

```
const { Verifier } = require('@pact-foundation/pact');
new Verifier().verifyProvider({
  providerBaseUrl: 'http://localhost:8080',
  pactUrls: ['pacts/webes-kliens-termek-katalogus.json'],
  providerStatesSetupUrl: 'http://localhost:8081/setup'
});
```

A `providerStatesSetupUrl` arra szolgál, hogy a provider a teszt előtt a szükséges állapotba hozza az adatbázist (például beszúrja a `termek-123` terméket). Ez biztosítja a teszt reprodukálhatóságát.

A szerződésteszt nem helyettesíti az E2E tesztet: azt ellenőrzi, hogy a szerződés fennáll-e, de nem bizonyítja, hogy a teljes rendszer üzletileg helyes. A szerződésteszt előnye, hogy olcsó, gyors és izolált, ezért a CI/CD pipeline korai szakaszában futtatható.

14.2.19 OpenAPI és szerződésteszt együtt

Az OpenAPI séma validálása statikus, a szerződésteszt dinamikus. A kettő kiegészíti egymást:

- Az OpenAPI diff a **séma szintű** breaking change-eket detektálja.
- A szerződésteszt a **viselkedés szintű** eltéréseket tárja fel (például a szerver nem a megadott státuszkódot adja vissza, vagy a válaszmező típusa nem egyezik).
- Az OpenAPI mock szerver lehetővé teszi a consumer tesztjeinek futtatását anélkül, hogy a valós provider rendelkezésre állna.

Egy érett API-verziózási folyamat mindkettőt alkalmazza.

14.2.20 Labor: verziózási és kompatibilitási szabály készítése

A labor célja, hogy a fenti fogalmakat egy kis, ellenőrizhető feladaton keresztül alkalmazd. Az átadandó eredmény egy reprodukálható dokumentum vagy jegyzetfüzet, amely rögzíti a kontextust, a döntési naplót, a mért és a várt eredményeket.

14.2.21 Kiinduló állapot

Adott egy termék-katalógus mikroservice REST API-val. A régi verzió OpenAPI specifikációja a fenti példában szereplő `openapi.yaml`. A csapat a következő változtatást tervezi:

- A `Product` sémához hozzáad egy `keszlet` mezőt (integer, opcionális).
- A `ar` mezőt átnevezi `netto_ar` névre.
- A `GET /products` végpontra bevezet egy új, opcionális `onlyAvailable` lekérdezési paramétert.

A feladat: készítsd el a verziózási szabályzatot, döntsd el, hogy a változtatások kompatibilisek-e, és dokumentáld a folyamatot.

14.2.22 Elvárt eredmény

1. **Verziózási politika dokumentum:** legalább a következő szakaszokkal: verzióazonosítás módja (URI, fejléc, media type), SemVer alkalmazása, deprecation és sunset szabályok, kompatibilitási osztályozás.
2. **ADR:** a döntést Architecture Decision Record formában rögzítsd. Tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
3. **OpenAPI diff jelentés:** hasonlítsd össze a régi és az új OpenAPI fájlt egy diff eszközzel (például nyílt forráskódú OpenAPI diff). A kimenetet mentsd el, és értékeld, mely változások breakingek.
4. **Szerződésteszt eredménye:** készíts egy consumer-driven contract tesztet a régi kliens szemszögéből (a `id`, `nev`, `ar` mezőket várja), és futtasd le a provider ellen. Ha a provider a `ar` mezőt `netto_ar`-ra nevezte át, a tesztnek buknia kell. Dokumentáld a hibaüzenetet.
5. **Ellenőrzési szempontok:** listázd, hogy mit ellenőriztél, és milyen eszközverziókat használtál.

14.2.23 Visszaállítási út

A labor minden lépése verziózott git repóban történjen. A régi OpenAPI fájl a git history-ban megtalálható, így bármikor visszaállítható. A szerződésteszt által generált pact fájl szintén verziózott, és a provider verifikáció bármikor újra futtatható.

14.2.24 Mért és várt eredmény elkülönítése

A dokumentumban külön szakaszban szerepeljen:

- **Várt eredmény:** a diff eszköznek a `ar`→`netto_ar` átnevezést breaking változásként kell jeleznie; a `keszlet` mező hozzáadása additív, nem breaking; a szerződésteszt nek el kell buknia, mert a `ar` mező hiányzik a válaszból.
- **Mért eredmény:** a tényleges diff kimenet, a szerződésteszt futásának naplója, az eszközök verziószáma, a futtatás időpontja.

A kettőt nem szabad összemosni: a várt eredmény a hipotézis, a mért eredmény a futtatás után rögzített tény.

14.2.25 Ellenőrzési szempontok

- A verziózási politika egyértelműen meghatározza a MAJOR/MINOR/PATCH növelés szabályait.
- A deprecation folyamat tartalmazza a bejelentés módját és a sunset időpontját.
- Az OpenAPI diff parancsa és a használt eszközverzió pontosan le van írva, így a diff reprodukálható.
- A szerződésteszt forráskódja a dokumentumban vagy a csatolt repóban elérhető.
- A mért és a várt eredmény külön szakaszban szerepel.
- A döntési napló hivatkozik a releváns ADR-re.

14.2.26 Gyakorlat

Képzeld el, hogy a termék-katalógus API-ban a `Product` sémában a `nev` mező típusát `string`-ről `object`-re akarják változtatni, hogy a termék neve több nyelven is elérhető legyen. Tervezd meg a változást kompatibilis módon. Milyen új mezőt vezetnél be, hogyan kezelnéd a régi `nev` mezőt, és milyen verziózási döntést hoznál? Írd le ADR formában.

14.2.27 Ellenőrzőlista

- ☐ Van egy írott verziózási politika, amely definiálja a verzióazonosítás módját.
- ☐ A csapat megkülönbözteti a backward és forward compatibility fogalmait.
- ☐ A breaking változásokat a diff eszköz automatikusan detektálja a CI/CD-ben.
- ☐ A deprecation és sunset folyamat dokumentált, és a klienseket időben értesítik.
- ☐ A séma evolúciós szabályok rögzítve vannak (additív mezők, tilos átnevezés, tilos típusváltás).
- ☐ A fogyasztói oldalon a tolerant reader elv érvényesül, de a szerződésteszek ellenőrzik a valós viselkedést.
- ☐ A consumer-driven contract tesztek a CI/CD részét képezik, és a provider oldalán is futnak.
- ☐ Az OpenAPI/AsyncAPI specifikáció a kódbázissal együtt verziózott.

14.2.28 Önellenőrző kérdés

Mikor válasszunk URI-verziózást a fejléc- vagy media type alapú verziózás helyett, és milyen esetben előnyösebb az utóbbi kettő valamelyike? Indokold meg a választ a kliensek frissítési ciklusa és a cache/proxy réteg szempontjából.

14.3 Enterprise Integration Patterns és message brokering

14.3.1 Üzenet-alapú integráció: miért van szükség a hálózati „postahivatalra”?

Amikor két rendszer közvetlen, szinkron hívással kommunikál – például egy webáruház rendelési szolgáltatása HTTP REST hívást küld a számlázó szolgáltatásnak –, a hívó fél életciklusa összekapcsolódik a hívott fél elérhetőségével. Ha a számlázó szolgáltatás lassú, a rendelési szolgáltatás is lassú lesz; ha a számlázó elérhetetlen, a megrendelés kudarcba fulladhat, hacsak a hívó fél saját maga nem implementál újrapróbálkozást, átmeneti tárolást vagy körültekintő hibakezelést. Ez a fajta szoros csatolás nem technológiai következményekkel jár csupán, hanem

szervezeti szinten is érezteti hatását: a fejlesztőcsapatok nem tudnak egymástól függetlenül telepíteni, mert a hívások szerződése és időzítése szinkronban kell, hogy maradjon.

Az üzenet-alapú integráció ezt a csatolást lazítja azzal, hogy a kommunikáció középpontjába egy közvetítőt, üzenetbrokert helyez. A termelő – az üzenet feladója – nem közvetlenül a fogyasztóval kommunikál, hanem a brokerrel. Az üzenetet a broker egy megnevezett csatornára (channel) helyezi; a fogyasztó – az üzenet címzettje – a saját tempójában olvassa ki onnan. Ez a modell időbeli és térbeli szétválasztást biztosít: a termelőnek nem kell tudnia, hány fogyasztó dolgozza fel az üzenetet, és a fogyasztónak sem kell elérhetőnek lennie abban a pillanatban, amikor a termelő adatot állít elő.

A broker fogalma itt nem egyszerűen egy TCP-átjáró. A broker egy dedikált infrastrukturális komponens, amely felel az üzenetek fogadásáért, tárolásáért, szükség szerinti átalakításáért és a fogyasztók felé történő kiszolgálásáért. Gyakran biztosít üzenetsorokat (queue), pub/sub témákat (topic), routing-szabályokat, átviteli garanciákat és felügyeleti végpontokat is. A gyakorlatban elterjedt rendszerek közé tartozik az Apache Kafka, a RabbitMQ, az Apache ActiveMQ, a NATS és a felhőalapú managed broker szolgáltatások. Mindegyik más-más garanciákat és alkalmas üzemeltetési modellt kínál; a választás soha nem „a legjobb broker” kiválasztása, hanem a követelményeknek – terhelés, sorrend, késleltetés, üzenet-élettartam, üzemeltetési érettség – való megfelelés.

A klasszikus „Enterprise Integration Patterns” (EIP) gyűjtemény – amelyet Gregor Hohpe és Bobby Woolf publikált – ezt a területet katalógusszerűen rendszerezi. Az EIP-ben a minták nem pusztán kódminták, hanem tervezési nyelvet adnak a csatorna, az üzenet, a végpont, a router és a transformer fogalmaira. A fejezet ezek közül a gyakorlatban legfontosabbakkal foglalkozik: a routing, a transformation és az aggregation gyakori formáival, valamint a dead letter kezelésével. Ehhez kapcsolódik három, a való életben gyakran félreértett kérdés: az idempotencia, a sorrend és a „pontosan egyszeri kézbesítés” ígérete. A fogalmakat egy kis, ellenőrizhető példán keresztül kötjük össze, és a laborban a döntési napló (ADR) segítségével rögzítjük az integrációs minta kiválasztásának okait.

14.3.1.1 Az üzenet fogalma: parancs, esemény, dokumentum

Mielőtt a mintákra rátérnénk, érdemes megkülönböztetni három üzenetfajtát, mert a routing és a feldolgozási garanciák eltérőek lehetnek aszerint, hogy mit hordoz az üzenet.

A **parancs (command)** egy másik rendszer felé irányuló felszólítás: „Hozd létre a számlát”, „Foglald le a készletet”. A parancs jellemzően egyetlen fogyasztót céloz meg, és a feladó elvárja, hogy a címzett valamilyen módon – szinkron válasszal vagy egy későbbi eseménnyel – megerősítse a művelet sikerét. A parancsokat gyakran queue-kon keresztül kézbesítik, ahol a broker garantálja, hogy egy üzenetet egyszerre csak egy fogyasztó kap meg.

Az **esemény (event)** múlt időben megfogalmazott tény: „A megrendelés létrejött”, „A fizetés visszaigazolódott”. Az eseményt a termelő nem egy konkrét címzettnek szánja; sok fogyasztó érdeklődhet iránta. A pub/sub témák erre valók: a termelő egy témába publikál, minden feliratkozó saját másolatot kap. Az eseményvezérelt architektúrákban a fejezet korábbi – különösen a CQRS és Event Sourcing – fogalmaira építve az események az integráció elsődleges nyelvévé válnak, mert a fogyasztók hozzáadása nem igényli a termelő módosítását.

A **dokumentum (document message)** önálló adathalmaz: egy számlakivonat, egy termékkatalógus-részlet. A dokumentum nem feltétlenül vált ki azonnali műveletet, hanem adatot mozgat a rendszerek között. A dokumentum élettartama hosszabb lehet, és a feldolgozása gyakran idempotens betöltés (upsert) formájában zajlik.

E három üzenettípus elkülönítése azért fontos, mert a brokerben alkalmazott routing szabályok, a visszajátszási stratégia és az újrapróbálkozás politikája eltérő. Egy parancs ismételt kézbesítése a fogyasztó oldalán idempotens műveletet igényel; egy esemény ismételt kézbesítése viszont a fogyasztó nézőpontjából gyakran ugyanolyan feldolgozást jelent, mint az első, ha az eseményazonosító egyedi.

14.3.1.2 A broker szerepe: nem csupán „socket szerver"

A broker tervezési szerepe legalább három rétegben érhető tetten. Az első réteg a **szállítás**: a broker fogadja a termelő kapcsolatait, és kiszolgálja a fogyasztókat. Ezen a szinten a protokoll – AMQP, MQTT, HTTP, Kafka wire protocol – implementációs részlet, bár a választása befolyásolja a skálázhatóságot és az üzemeltetést.

A második réteg a **topológia**: a brokerben csatornák, queue-k, témák és routing-szabályok hálózatát hozzuk létre. Ez a topológia az integrációs architektúra egyik legfontosabb része, mert itt dől el, melyik üzenet hová kerül, hány fogyasztó látja, és mi történik, ha senki nem hallgat rá. A topológiát érdemes diagrams-as-code módon – például a broker adminisztrációs API-ját hívó szkriptekkel – verziózni, mert a kézi konfiguráció hajlamos az eltérésre a környezetek között.

A harmadik réteg a **feldolgozási garanciák**: a broker és a kliensek közötti együttműködés határozza meg, hogy egy üzenet legfeljebb egyszer, legalább egyszer vagy pontosan egyszer kerül-e feldolgozásra. A valós rendszerekben a „pontosan egyszeri" kifejezés megtévesztő, mert elosztott környezetben a hálózati hiba és a fogyasztó összeomlása közötti időben nem tudjuk garantálni, hogy egy művelet ne ismétlődjön meg. Amit a broker és a feldolgozási keretrendszer biztosítani tud, az a **hatékony pontosan egyszeri feldolgozás (effectively-once processing)**: a rendszer úgy tesz, mintha az üzenet egyszer lett volna feldolgozva, mert a fogyasztó idempotens, a broker pedig tranzakciókkal és deduplikációval csökkenti az ismétlést.

A broker tehát infrastrukturális keret, de nem varázsszer. Az üzenet-alapú integráció árat fizet a laza csatolásért: a rendszer aszinkron lesz, a hibák később derülnek ki, a nyomkövetés nehezebb, és a broker maga is egy elosztott rendszer, amelynek üzemeltetése, monitoringja és kapacitástervezése önálló feladat. Aki ezt az árat nem számolja be a döntésbe, az a szinkron hívások egyszerűségéből kiindulva alulbecsüli a bevezetés költségét.

14.3.1.3 Alapvető EIP-minták: csatorna, végpont, üzenet

Az EIP nyelv három alapfogalomból építkezik. A **csatorna (channel)** az üzenetek logikai útvonala a termelő és a fogyasztó között. A brokerben a csatorna lehet queue (egy fogyasztói csoportos fogyasztás), topic (pub/sub) vagy ezek kombinációja. A csatornának neve van, és a névadás a szerződés része: a `order.created` eseménycsatorna neve kifejezi az üzenet tartalmát, nem a termelő technológiai nevét.

A **végpont (endpoint)** az a pont, ahol egy alkalmazás üzenetet küld vagy fogad. A végpontnak van egy belső logikája: az üzenet fogadásakor validál, deszerializál, meghív egy üzleti szolgáltatást, majd a választ – ha van – továbbítja. A végpontot a broker-től elválasztó határon kontrollált hibakezelésre van szükség: ha a deszerializáció kudarcot vall, az üzenet nem veszhet el a semmibe, hanem a dead letter csatornába kell kerülnie.

Az **üzenet (message)** két részből áll: a fejlécből és a törzsből. A fejléc hordozza az integrációhoz szükséges metaadatokat: korrelációs azonosító, üzenetazonosító, időbélyeg, válaszcím, prioritás, lejárat. A törzs hordozza az üzleti tartalmat, általában strukturált formátumban – JSON, Avro, Protobuf. A fejléc és a törzs elválasztása azért fontos, mert a routing és a transformation gyakran csak a fejlécet olvassa, és a törzs átalakítása nélkül továbbítja az üzenetet.

14.3.1.4 Routing minták: hogyan kerül az üzenet a megfelelő helyre?

A legegyszerűbb routing az, amikor a termelő közvetlenül egy névvel ellátott csatornára küld. Ez a fajta **pont-pont csatorna (point-to-point channel)** egyetlen fogyasztót szolgál ki. Ha a fogyasztó oldalán több párhuzamos példány fut, akkor a queue-ból egy üzenetet egyetlen példány kap meg; a broker ezzel terheléselosztást is biztosít.

A **publish-subscribe channel** ezzel szemben minden feliratkozó fogyasztónak másolatot küld. A témákra épülő eseményintegrációban ez az alapértelmezett: a rendelés létrejötte után a számlázó, a raktár és az értesítő szolgáltatás mind külön-külön megkapja az `order.created` eseményt, anélkül hogy a termelőnek ismernie kellene őket.

A gyakorlatban a legtöbb integráció ennél összetettebb: az üzenet tartalma vagy fejléce alapján kell eldönteni, hová kerüljön. A **content-based router** a bejövő üzenetet megvizsgálja, és egy szabály alapján választ ki egy kimenő csatornát. Például egy fizetési esemény routere a `payment.method` mező alapján küldi az üzenetet a kártyafeldolgozó, az átutalás-kezelő vagy a csomagküldő utánvét-kezelő csatornájába. A router maga nem módosítja az üzenetet, csak eldönti az útvonalat. Ez a minta csökkenti a termelő felelősségét, és a routing-szabályokat egy helyen tartja; cserébe a router állapota kritikus komponenssé válik, és a szabályváltozás közvetlenül érinti az összes érintett integrációt.

A **message filter** a router ellentéte: nem irányít, hanem eldönti, hogy az üzenet egyáltalán továbbhaladhat-e. Egy éles rendszerben a filter például kiszűri azokat a rendelési eseményeket, amelyek értéke nem éri el a számlázási feldolgozás határát. A filter veszélye, hogy ha túl sok üzenetet dob el csendben, a rendszer viselkedése megmagyarázhatatlanná válik; a megoldás a filter által eldobott üzenetek naplózása vagy egy külön csatornára irányítása.

A **recipient list** minta akkor hasznos, ha a címzettek listája dinamikus: a router egy szabály alapján több csatornát is kiválaszt, és mindegyikre elküldi az üzenetet. A **splitter** és a **routing slip** további speciális esetek: a splitter egy nagy üzenetet darabol fel, és minden darabot önálló üzenetként továbbít; a routing slip pedig az üzenet fejlécébe helyezi el a feldolgozási lépések listáját, így az üzenet egy folyamatsoron halad végig anélkül, hogy központi orchestratornak kellene ismernie a teljes folyamatot.

A routing szabályokat érdemes úgy kialakítani, hogy a szabály maga is verziózott legyen. Ha a router konfigurációja a kódbázisban van, a változtatás a szokásos code review folyamaton megy át; ha valaki közvetlenül a broker adminisztrációs felületén módosítja, az a dokumentálatlan „konfigurációs adósság” tipikus forrása.

14.3.1.5 Transformation minták: az üzenetek nyelve nem semleges

A különböző rendszerek ritkán beszélik ugyanazt az adatnyelvet. A számlázó rendszer `invoice_number` mezőt vár, a raktár `order_reference`-et; az egyik formátum ISO 8601 dátumot használ, a másik `YYYY-MM-DD` szöveget. A **message translator** minta ezt a különbséget egyetlen helyen hidalja át: a bejövő üzenetet a célrendszer által elvárt formátumra alakítja. A translator lehet egyszerű mezőátnevezés, de lehet komplex szerkezeti átalakítás is – a lényeg az, hogy a fordítási szabály explicitté és tesztelhetővé válik.

Az **envelope wrapper** minta akkor segít, ha az üzenetet olyan metaadatokkal kell kiegészíteni, amelyeket a fogadó fél nem tud vagy nem akar kezelni. Az envelope egy konténer: a törzsben marad az eredeti üzenet, a fejlécben pedig a korrelációs azonosító, a forrásrendszer, a prioritás. Ez a minta gyakran az üzenetformátum verziózásának alapja is: az envelope verziója jelzi a törzs sémájának változását.

A **canonical data model** minta egy közös, rendszerek közötti adatmodellt vezet be, és minden rendszer a saját formátumából a kanonikus modellre fordít, majd a kanonikus modellből a célformátumra. Ez a megközelítés drasztikusan csökkenti a páronkénti translatorok számát – ha n rendszer van, a páronkénti fordítás n*(n-1) integrációt igényel, a kanonikus modell 2*n-et –, de a közös modell kialakítása szervezeti és technikai kompromisszum. A közös modell túl általánossá válhat, és az egyes rendszerek számára kényelmetlen kompromisszumokat kényszeríthet ki; ráadásul a modell változtatása az összes translator újraírását vonhatja maga után.

A gyakorlatban a transformation-t érdemes a brokerhez közel, de nem feltétlenül a brokerben elvégezni. A brokerbe ágyazott átalakítási funkció – például egy beépített JavaScript-lépés vagy egy Kafka Streams folyamat – csökkenti a hálózati ugrást, de a tesztelhetőség és a verziózás gyakran szenved. Az elkülönült transformation szolgáltatás – amely maga is egy fogyasztó és termelő – tisztább felelősséghatárt ad, de növeli az üzemeltetendő komponensek számát. A döntés a csapat képességeitől és a rendszer méretétől függ, nem abszolút szabály.

14.3.1.6 Aggregation minták: amikor egy üzenet nem elég

Több integrációs folyamatban egyetlen üzleti művelethez több rendszer válasza szükséges. A **scatter-gather** minta ezt a problémát két lépésben oldja meg: először a router szétküldi a kérést több fogyasztónak (scatter), majd a **aggregator** összegyűjti a válaszokat, és egyetlen összetett üzenetet állít elő (gather). A kihívás az időzítés: a válaszok nem egyszerre érkeznek, és az aggregatornak tudnia kell, mikor mondhatja azt, hogy „mindenki válaszolt, most már továbbmehetek”.

A gyakorlatban ez két részproblémára bomlik. Az első a **korreláció**: az aggregatornak össze kell tudnia rendelni az összetartozó válaszüzeneteket. Erre szolgál a korrelációs azonosító: a scatter lépés minden kimenő üzenet fejlécébe ugyanazt a `correlation_id`-t írja, a fogyasztók pedig a válaszban visszaadják. A második a **teljesség**: az aggregator előre tudja, hány válaszra vár, vagy egy időkorlátot alkalmaz. A komplex folyamatokban az **aggregation strategy** lehet „mindre várj”, „első győz” vagy „határidőre, ami bejött, azt adjuk tovább”.

Az aggregáció hibája tipikusan az, hogy a fejlesztő az aggregátort állapotmentesnek tervezi, pedig az szükségszerűen állapotot tart: nyilván kell tartania, melyik korrelációs azonosítóból hány válasz érkezett. Ez az állapot a feldolgozás idejére szól, de az aggregátor leállása esetén elvész; a megoldás vagy a feldolgozási állapot perzisztálása, vagy a scatter-gather lépés újrapróbálható tervezése. A scatter-gather a leggyakrabban félreértett EIP-minták egyike, mert a szinkron folyamatok analógiájából indul ki: a fejlesztő úgy gondolja, hogy a válaszok „mindjárt” megérkeznek, pedig elosztott környezetben a válaszok késhetnek, el is veszhetnek.

14.3.1.7 Dead letter: a hibás üzenet nem tűnhet el

A **dead letter channel** az üzenet-alapú integráció biztonsági hálója: ha az üzenet feldolgozása tartósan sikertelen – a deszerializáció hibát dob, a fogyasztó többszöri újrapróbálkozás után is kudarcot vall, vagy az üzenet lejár –, a broker a dead letter csatornára helyezi, ahol az üzenet megmarad a későbbi elemzéshez. A dead letter nem hibaelrejtés, hanem a hiba kezelésének explicit helye.

A dead letter csatorna tervezési kérdései: mi kerüljön rá, hány újrapróbálkozás után, és ki fogyassza onnan? A válasz attól függ, hogy a hiba átmeneti vagy tartós. A **retry** minta ezért a dead letter előfeltétele: a fogyasztó oldalán meghatározott számú újrapróbálkozást engedünk, exponenciális hátrálással (backoff). Ha a hiba továbbra is fennáll, az üzenetet a dead letterre irányítjuk. A **poison message** fogalma az olyan üzenet, amely soha nem lesz sikeres, mert a

tartalma hibás; ezt a fogyasztó a legnagyobb újrapróbálkozás után sem tudja feldolgozni, és a dead letter az egyetlen eszköz, hogy a hálózat ne akadjon el.

A dead letter fogyasztása ne automatizált törlés legyen. A helyes gyakorlat az, hogy a dead letterre került üzeneteket egy operátori folyamat elemzi: miért hibás az üzenet, a termelő oldalán kell-e javítani, esetleg a séma-validáció hiánya miatt került rossz adat a csatornára. A dead letter forgalma a rendszer egészségének fontos indikátora: ha a dead letter hossza tartósan növekszik, az integrációs topológia valamely pontján szisztematikus hiba van.

14.3.1.8 Idempotencia: az ismétlés nem jelenthet duplikációt

Az aszinkron üzenetfeldolgozásban a kézbesítés ismétlése nem kivétel, hanem normális üzemállapot. A broker legalább egyszeri kézbesítési garanciája esetén a fogyasztó ugyanazt az üzenetet többször is megkaphatja; hálózati hiba, újraegyensúlyozás vagy a fogyasztó összeomlása mind ezt a helyzetet idézheti elő. Az **idempotens fogyasztó** az, amely ugyanazon üzenet többszöri feldolgozására is ugyanazt az eredményt adja, mint az egyszeri feldolgozásra: a második végrehajtás nem hoz létre duplikátumot, nem von le kétszeres összeget, nem küld ki két számlát.

Az idempotencia megvalósításának leggyakoribb módja a deduplikációs kulcs. Az üzenet fejlécében elhelyezett egyedi `message_id` alapján a fogyasztó egy tárolóban – adatbázisban, cache-ben – nyilvántartja, hogy az adott azonosítójú üzenetet már feldolgozta. A második érkezéskor a tároló ellenőrzése után a fogyasztó a műveletet kihagyja. A deduplikációs kulcs lehet üzleti azonosító is: ha a rendelés száma egyedi, a számlázó fogyasztó a `order_id` alapján dönti el, hogy a rendeléshez tartozó számla már létezik-e.

Az idempotencia nem csak a fogyasztó feladata. A termelő oldalán is lehet idempotens: ha a termelő a saját újrapróbálkozásakor ugyanazt az `message_id`-t használja, a broker – amennyiben támogatja – a deduplikációt már a csatorna szintjén elvégezheti. Ez a képesség azonban nem minden brokerben alapértelmezett, és a konfigurációtól függ. Ahol a broker nem deduplikál, ott a fogyasztónak kell. Az idempotencia hiánya az integrációs hibák egyik leggyakoribb forrása; a következmény – duplikált számla, kétszer elküldött kifizetés – gyakran csak a végfelhasználónál derül ki.

14.3.1.9 Sorrend: mikor számít, és mit garantál a broker?

Az elosztott rendszerekben a sorrend fenntartása többletköltséggel jár. A broker alapértelmezésben gyakran nem garantál globális sorrendet, különösen akkor, ha párhuzamos fogyasztók dolgozzák fel a csatornát. A kérdés tehát nem az, hogy „fenntartsuk-e a sorrendet”, hanem az, hogy „melyik sorrendet kell fenntartanunk és miért”.

A **particionálás** a sorrendkezelés alapvető eszköze. Ha az üzeneteket egy kulcs – például a `order_id` – alapján particionáljuk, akkor az azonos kulcsú üzenetek ugyanabba a partícióba kerülnek, és a broker az adott partíción belül sorrendben kézbesíti őket. Ez nem globális sorrend: két különböző rendelésű esemény sorrendje a fogyasztó számára nem garantált, de egy adott rendelés eseményei mindig a küldés sorrendjében érkeznek. A sorrendfenntartás szükségessége tehát a domain típusától függ: a rendelési életciklus eseményei általában sorrendet igényelnek, a termékmegtekintési események nem.

A sorrend egyik elterjedt félreértése, hogy a brokertől globális FIFO-t várjunk. A globális sorrend szűk keresztmetszetet teremt: az üzenetek feldolgozása párhuzamosságát elveszíti, mert a fogyasztó csak akkor veheti a következőt, ha az előzőt befejezte. A gyakorlatban a sorrendre vonatkozó igényt minőségi forgatókönyvben kell rögzíteni: „Az `order` entitáshoz tartozó események feldolgozása a termelés sorrendjében történik, mert a raktárfoglalás nem előzheti meg a

rendelés létrehozását; az egyes rendelések közötti sorrend nem követelmény.” Ez a megfogalmazás lehetővé teszi a párhuzamosság megtartását ott, ahol a domain nem igényli a sorrendet.

14.3.1.10 A „pontosan egyszeri kézbesítés” ígérete

A „pontosan egyszeri kézbesítés” kifejezés az integrációs szakirodalom egyik legfélrevezetőbb fogalma. Elosztott környezetben – amikor a termelő, a broker és a fogyasztó külön hibadománya – a hálózat megszakadhat a feldolgozás és a nyugtázás közötti időpillanatban, így a fogyasztó nem tudhatja, hogy az üzenet feldolgozása sikeres volt-e. Az egyetlen biztos válasz ilyenkor az újrapróbálkozás, ami legalább egyszeri kézbesítéshez vezet; a fogyasztó idempotenssége az, ami a duplikáció hatását megszünteti.

A modern brokerek és stream-keretrendszerek gyakran „exactly-once” szemantikát hirdetnek. Ez a gyakorlatban **effectively-once processing**: a keretrendszer tranzakciós protokollal biztosítja, hogy a termelő oldalán az üzenet ne duplikálódjon a brokerben, és a fogyasztó offsetjeinek frissítése atomi legyen a feldolgozási eredmény mentésével. A tranzakció azonban nem a hálózati hiba ellen véd, hanem a feldolgozási állapot következetlensége ellen: ha a fogyasztó adatbázisba ír, az írás és az offset-commit egy tranzakcióban megy végbe, így az újraindulás után a fogyasztó pontosan a még nem feldolgozott üzenettől folytatja. A **hatókört** tehát pontosan kell érteni: a „pontosan egyszeri” a broker és a keretrendszer közötti tranzakciós garancia, nem a teljes, végponttól végpontig tartó folyamat automatikus tulajdonsága.

A tervezési következmény az, hogy a fogyasztó idempotenciája akkor is szükséges, ha a broker „exactly-once”-t támogat. Az architektúrában az idempotens fogyasztó követelmény, nem opcionális optimalizáció; a broker tranzakciója ezt a követelményt nem helyettesíti, csak kiegészíti.

14.3.1.11 Labor: integrációs minta kiválasztása egy folyamathoz

A labor célja, hogy egy konkrét, kis üzleti folyamatra alkalmazd az EIP-mintákat, és a döntéseket reprodukálható dokumentumban rögzítsd. A labor nem kódírás, hanem tervezési gyakorlat: a kiválasztott mintát, a döntés indoklását és az ellenőrzési szempontokat kell dokumentálni.

Kontextus. Egy fiktív, de reális folyamatot választunk: egy online piactér `order.completed` eseménye alapján három rendszernek kell adatot kapnia: a számlázó szolgáltatásnak, amely a számlát készíti; a raktárkezelőnek, amely a kiszedést indítja; és az ügyfélkapcsolati rendszernek, amely a hűségprogram-pontokat írja jóvá. A számlázó szolgáltatás csak a kártyás és az előre fizetett rendeléseket kezelheti; az utánvétes rendeléseket a raktárnak kell látnia, de a számlázónak nem. A raktár üzeneteinek sorrendben kell érkezniük rendelésenként, mert a készletfoglalás a rendelés létrehozása után következhet. A számlázó nem lehet terhelve azzal a rendeléssel, amelynek értéke nem éri el a számlázási minimumot.

Feladat. Válaszd ki a megfelelő EIP-mintákat a folyamathoz, és indokold a döntést. A választásodat ADR formában rögzítsd, legalább a következő szakaszokkal: kontextus, döntés, alternatívák, következmények, állapot. Az ADR-ben térj ki a következő kérdésekre: milyen csatorna típusú a `order.completed` esemény? Hol alkalmazol message filtert, és miért? Hogyan biztosítod a raktár számára a rendelésenkénti sorrendet? Mi történjen a számlázó által nem feldolgozható üzenetekkel? Az ellenőrzési szempontokban különítsd el a mért és a várt eredményt.

Várt eredmény. Az ADR-ben egyértelműen azonosíthatók a minták: publish-subscribe a `order.completed` eseményre; content-based router a fizetési mód szerint, vagy message filter a számlázási minimumra; partícionálás a raktár csatornáján `order_id` kulccsal; dead letter a számlázó többszöri sikertelen feldolgozása esetén. Az alternatívákban legalább egy elvetett lehetőségnek szerepelnie kell – például a közvetlen pont-pont csatorna vagy a globális FIFO

– azzal az indokkal, hogy miért nem felel meg a követelménynek. Az ellenőrzési részben a mért eredmény helyén nullérték szerepel, mert a labor tervezési gyakorlat; a várt eredményt a forgatókönyv alapján írod le.

Visszaállítási út. Mivel a labor nem módosít éles rendszert, nincs szükség visszaállításra. A dokumentumot a kódbázisban kell verziózni.

14.3.1.12 Ellenőrzőlista és tipikus hibák

A fejezet végén érdemes az alábbi ellenőrzőlistát alkalmazni az üzenet-alapú integráció tervezésekor:

1. Az üzenet típusa tisztázott: parancs, esemény vagy dokumentum? A csatorna típusa ennek megfelelő?
2. A broker topológiája verziózott és diagrams-as-code formában rögzített?
3. A routing szabályok explicit módon eldöntik, hová kerül az üzenet; nincs olyan csatorna, ahová „minden megy, aztán majd valaki kitalálja”?
4. A transformation szabályok egy helyen, tesztelhetően élnek, és a séma-verziók kezeltek?
5. A fogyasztók idempotensek; a deduplikációs kulcsot a termelő és a fogyasztó is ismeri?
6. A sorrendre vonatkozó igényt minőségi forgatókönyv rögzíti, és a broker partícionálás ezt leképezi?
7. A dead letter csatorna létezik, és a rá került üzeneteket operátori folyamat kezeli?
8. A „pontosan egyszeri” állításokat a hatókör pontos megnevezésével használjuk – a fogyasztó idempotenciája akkor is követelmény?

A leggyakoribb hibák közé tartozik az, amikor a fejlesztő a brokert egyszerű socket-ként kezeli, és a holtüzenet-kezelést elhagyja; amikor a fogyasztót nem idempotenssé tervezi, majd a duplikációt „a broker hibájának” tulajdonítja; amikor a globális sorrendet követeli, pedig a domain csak rendelésenként igényli; és amikor a transformation-t a brokerbe ágyazott, nem verziózott szkriptekkel végzi. A helyes megoldás ezekben az esetekben nem új technológia bevezetése, hanem a minta pontos megértése és a fogyasztói garanciák explicit tervezése.

14.3.1.13 Összefoglalás

Az üzenet-alapú integráció a laza csatolás ígéretét hordozza, de csak akkor, ha a mintákat tudatosan alkalmazzuk. A broker szállítási infrastruktúra; a csatornák, routerek, filterek, translatorok és aggregátorok alkotják azt a nyelvet, amelyen a rendszerek integrációját leírjuk. A dead letter a hiba explicit kezelésének eszköze, nem a hibák elrejtéséé. Az idempotencia és a sorrend a fogyasztó oldalán is tervezési követelmény, nem a broker automatikus melléktermé. Aki ezeket a fogalmakat elválasztja egymástól, az a következő integrációs döntését már bizonyíthatóan indokolni tudja – és a laborban rögzített ADR-ekkel azt is dokumentálja, miért éppen azt a mintát választotta.

A következő alfejezet az API gateway, a BFF és a service mesh témáját veszi fel: ott az üzenet-alapú integráció itt bevezetett fogalmai – különösen a routing és a transformation – a hálózati peremen és a szolgáltatások közötti platformrétegben jelennek meg.

14.4 API gateway, BFF és service mesh

A perem és a szolgáltatásközi forgalom kezelése az elosztott rendszerek integrációs rétegének két, egymástól eltérő, mégis gyakran összekevert feladatköre. A **perem** (edge) az a határ, ahol a külső kliensek – böngészők, mobilalkalmazások, partnerrendszerek – belépnek a rendszerbe; a **szolgáltatásközi forgalom** pedig a belső hálózaton, a szolgáltatások közötti kommunikációt jelenti. Mindkét területen hasonló keresztmetszeti aggályok merülnek fel: hitelesítés és engedélyezés, terheléskorlátozás (rate limiting), útvonalválasztás, hibatűrés és megfigyelhetőség. Ebben az alfejezetben ezeknek a kezelésére szolgáló három építőelemet tárgyaljuk: az API gatewayt, a Backend for Frontend (BFF) mintát és a service mesht.

A hagyományos, monolitikus alkalmazásokban ezek a kérdések kevésbé hangsúlyosak, mert a kliens egyetlen szerverrel kommunikál, a belső hívások pedig nem lépik át a hálózati határt. A mikroszolgáltatás-architektúra azonban szétosztja a rendszert, és a hálózat a rendszer szerves részévé válik: a kliensek több szolgáltatással kell, hogy kapcsolatba lépjenek, a szolgáltatások pedig egymást hívják. Ez a változás új követelményeket támaszt a forgalomkezeléssel szemben, amelyeket nem lehet pusztán az alkalmazáskódba ágyazott egyedi megoldásokkal kielégíteni. A jól megtervezett perem- és hálózati réteg csökkenti a kliensek komplexitását, növeli a biztonságot, egységesíti a keresztmetszeti funkciókat, és lehetővé teszi a finom szemcsés megfigyelést.

14.4.1 1. A perem és a szolgáltatásközi forgalom problémája

Mielőtt rátérnénk az eszközökre, érdemes tisztázni, miért van szükség külön rétegre. Egy mikroszolgáltatás-alapú rendszerben a klienseknek számos, egymástól független szolgáltatással kellene kapcsolatba lépniük, ami a következő problémákat veti fel:

- **Kliens komplexitás:** A kliensnek ismernie kell az összes szolgáltatás címét, protokollját és adatsémáját. Egy új szolgáltatás bevezetése vagy egy meglévő áthelyezése a kliensek frissítését igényli. Ez különösen fájdalmas mobilalkalmazásoknál, ahol a frissítés a felhasználótól függ.
- **Biztonsági kockázat:** A belső szolgáltatások közvetlen elérése megnöveli a támadási felületet; a hitelesítési logikát minden szolgáltatásnak külön kellene megvalósítania, és a belső hálózat védelme nehezebbé válik.
- **Keresztmetszeti aggályok duplikálása:** A rate limiting, retry, circuit breaking, naplózás és metrikagyűjtés implementációja minden szolgáltatásban ismétlődne, ami inkonzisztenciához és karbantartási rémálomhoz vezet.
- **Megfigyelhetőség hiánya:** Az elosztott hívások követése nehéz, ha nincs egységes pont, ahol a forgalom áthalad. Egy kérés útját a rendszeren át csak akkor látjuk, ha a naplókat és metrikákat központilag gyűjtjük.

Ezekre a problémákra nyújtanak megoldást a peremen elhelyezett API gatewayek és a szolgáltatások közötti forgalmat szabályozó service meshek. Fontos megérteni, hogy a kettő nem felcserélhető: az API gateway a külső kliensek és a belső szolgáltatások közötti északi-déli (north-south) forgalmat kezeli, míg a service mesh a belső szolgáltatások közötti keleti-nyugati (east-west) forgalmat. A BFF pedig az API gateway egy speciális, kliensközeli változata, amely az egyes kliensplatformok eltérő igényeit szolgálja ki. A három eszköz együttesen, egymást kiegészítve alkotja a modern elosztott rendszerek forgalomkezelési gerincét.

14.4.2 2. API gateway: a perem egységes belépési pontja

Az **API gateway** egy olyan szerver, amely a kliensek és a belső szolgáltatások között helyezkedik el, és egyetlen belépési pontot biztosít. A gateway tipikusan **fordított proxyként** működik, és a bejövő kéréseket a megfelelő belső szolgáltatáshoz irányítja. Feladatai közé tartozik:

- **Útvonalválasztás (routing):** A bejövő kérés alapján eldönti, melyik belső szolgáltatáshoz irányítsa. Ez lehet útvonal-alapú (`/catalog/**` → katalógus-szolgáltatás), fejléc-alapú (pl. `X-Tenant-ID` alapján) vagy akár a kérés tartalma alapján is. A routing lehetőséget ad canary release-re, A/B tesztelésre és kék-zöld telepítésre is.
- **Hitelesítés és engedélyezés:** A gateway ellenőrzi a kliens azonosítóját (pl. OAuth2 token, API kulcs), és továbbíthatja a belső szolgáltatásoknak a felhasználói kontextust (pl. JWT claim-ek). Ezzel központosítja a biztonsági házirendeket, és tehermentesíti a belső szolgáltatásokat.
- **Rate limiting:** Korlátozza az egyes kliensek vagy felhasználók kéréseinek számát időablakonként, védve a belső szolgáltatásokat a túlterheléstől és a visszaéléstől. A rate limit lehet globális, kliensspecifikus vagy akár végpontonkénti.
- **TLS-végződtetés:** A külső forgalom titkosítása a gatewaynél történik, a belső hálózaton pedig akár titkosítatlanul is mehet (bár ez ma már vitatott). A TLS-végződtetés csökkenti a belső szolgáltatások terhelését, és egyszerűsíti a tanúsítványkezelést.
- **Kérések átalakítása:** A gateway módosíthatja a kérés vagy válasz fejlécét, testét, illetve protokollt válthat (pl. külső REST-ről belső gRPC-re). Ez lehetővé teszi, hogy a belső szolgáltatások egységes, modern protokollt használjanak, miközben a kliensek a számukra megfelelőt.
- **Megfigyelhetőség:** A gateway természetes pont a naplózásra, metrikagyűjtésre és az elosztott trace indítására. Minden kérés áthalad rajta, így egységes képet ad a rendszerbe érkező forgalomról.

Az API gateway gyakran deklaratív konfigurációval rendelkezik. Az alábbi, szemléltető YAML-részlet egy népszerű gateway-konfigurációt mutat (a pontos sémát és a termékverziót mindig ellenőrizni kell a hivatalos dokumentációban):

```yaml
# gateway-config.yaml (szemléltető, nem éles beállítás)
routes:
  - id: catalog-route
    uri: lb://catalog-service
    predicates:
      - Path=/catalog/**
    filters:
      - name: RateLimit
        args:
          redis-rate-limiter.replenishRate: 10
          redis-rate-limiter.burstCapacity: 20
      - name: JwtValidation
        args:
          jwk-set-uri: https://auth.example.com/.well-known/jwks.json
  - id: cart-route
    uri: lb://cart-service
    predicates:
      - Path=/cart/**
    filters:
      - name: RateLimit
        args:
          redis-rate-limiter.replenishRate: 5
          redis-rate-limiter.burstCapacity: 10
```

Itt a `catalog-route` a `/catalog/**` útvonalat a `catalog-service` felé irányítja, 10 kérés/másodperc utántöltési rátával és 20-as burst kapacitással, valamint JWT-ellenőrzéssel. A `cart-route` hasonló, de kisebb limitet kapott, mert a kosár-műveletek érzékenyebbek a visszaélésre. A gateway így központilag érvényesíti a hitelesítést és a terheléskorlátot.

C4 diagramok a gateway bemutatására – A diagrams-as-code elvnek megfelelően a gatewayt context és container szinten érdemes ábrázolni. Az alábbi Mermaid kód egy C4 context szintű nézetet mutat egy e-kereskedelmi rendszerre:

```
C4Context
  title E-kereskedelmi rendszer context szint
  Person(customer, "Vásárló", "Böngésző vagy mobilalkalmazás")
  System(eshop, "E-kereskedelmi rendszer", "Katalógus, kosár, rendelés")
  System_Ext(payment, "Fizetési szolgáltató", "Külső PSP")
  Rel(customer, eshop, "HTTPS", "Böngészés, vásárlás")
  Rel(eshop, payment, "HTTPS", "Tranzakció")
```

Container szinten az API gateway külön konténerként jelenik meg:

```
C4Container
  title E-kereskedelmi rendszer container szint
  Person(customer, "Vásárló")
  Container_Boundary(eshop, "E-kereskedelmi rendszer") {
    Container(gateway, "API Gateway", "Kong/Envoy/egyéb", "Hitelesítés, routing, rate limiting")
    Container(catalog, "Katalógus szolgáltatás", "Spring Boot", "Termékadatok")
    Container(cart, "Kosár szolgáltatás", "Node.js", "Kosárkezelés")
    Container(order, "Rendelés szolgáltatás", "Python", "Rendelések")
  }
  Rel(customer, gateway, "HTTPS")
  Rel(gateway, catalog, "HTTP/gRPC")
  Rel(gateway, cart, "HTTP/gRPC")
  Rel(gateway, order, "HTTP/gRPC")
```

Ez a diagram világosan mutatja, hogy minden külső kérés a gatewayen halad át, és a belső szolgáltatások csak a gatewaytől fogadnak forgalmat. A gateway így a rendszerhatár kulcsfontosságú komponense, amelyet a C4 context szinten is jelölni kell.

14.4.3 3. Backend for Frontend (BFF): kliensspecifikus perem

Az API gateway egységes felületet nyújt minden kliensnek, de a gyakorlatban a különböző kliensek – mobil, web, IoT, partner – eltérő adatigényekkel, hívási mintákkal és korlátozásokkal rendelkeznek. A **BFF** minta ezt a problémát úgy oldja meg, hogy minden kliensplatformhoz egy külön, arra szabott backendet (valójában egy peremréteget) rendel. A BFF nem egy újabb szolgáltatás a mélyben, hanem a peremre helyezett, kliensspecifikus API-kompozíciós réteg.

Példa: Egy webes felületnek meg kell jelenítenie a termékoldalt, amely tartalmazza a termék adatait, a készletet és a felhasználó kosarának állapotát. A mobilalkalmazásnak ugyanez az oldal kevesebb adattal is beéri, és a képek kisebb változatát kéri. Ha egyetlen, általános API gatewayt használunk, a kliensek vagy túl sok adatot kapnak (over-fetching), vagy több hívást kell indítaniuk (chatty interface). A BFF ehelyett minden kliens számára egy dedikált végpontot vagy akár egy teljes API-felületet biztosít, amely a belső szolgáltatások hívásait összefogja, és a kliens igényei szerint alakítja.

A BFF előnyei:

- **Csökkentett kliens komplexitás:** A kliens csak a BFF-fel kommunikál, nem kell ismernie a belső szolgáltatásokat. Ez egyszerűbb klienskódot és kevesebb hibalehetőséget jelent.

- **Optimalizált adatátvitel:** A BFF pontosan azt az adatot adja vissza, amire a kliensnek szüksége van, csökkentve a sávszélesség-használatot és a feldolgozási időt.
- **Platformspecifikus logika elkülönítése:** Például a mobil BFF kezelheti a képek átméretezését vagy a session kezelést, míg a web BFF szerveroldali renderelést végezhet.

Hátrányai:

- **Duplikáció:** Minden BFF hasonló logikát tartalmazhat (pl. hitelesítés-ellenőrzés), hacsak nem osztják meg. Ez növeli a kódduplikációt és a karbantartási terhet.
- **Karbantartási többlet:** Új kliensplatform esetén új BFF kell, ami növeli az üzemeltetési költségeket és a frissítések koordinációját.
- **Rejtett komplexitás:** Ha a BFF túl sok üzleti logikát tartalmaz, az a belső szolgáltatások helyett a peremen halmozódik fel, ami visszavezet a monolit problémájához.

A BFF és az API gateway viszonya: A BFF gyakran az API gateway mögött vagy azzal egybeépítve jelenik meg. Egy elterjedt topológia: a kliensek egy API gatewayhez csatlakoznak, amely a kéréseket a megfelelő BFF-hez irányítja; a BFF pedig a belső szolgáltatásokkal kommunikál. Máskor maga az API gateway tölti be a BFF szerepét is, például útvonalanként eltérő transzformációs logikával. A lényeg, hogy a perem réteg kliensspecifikus nézetet biztosít, miközben a belső szolgáltatások egységesek maradnak.

A következő Mermaid container diagram egy BFF-topológiát mutat:

```
C4Container
  title BFF topológia: web és mobil kliensek
  Person(web_user, "Web felhasználó")
  Person(mobile_user, "Mobil felhasználó")
  Container_Boundary(eshop, "Rendszer") {
    Container(gateway, "API Gateway", "Routing, hitelesítés")
    Container(web_bff, "Web BFF", "Kompozíció, szerveroldali renderelés")
    Container(mobile_bff, "Mobil BFF", "Kompozíció, képátméretezés")
    Container(catalog, "Katalógus szolgáltatás")
    Container(cart, "Kosár szolgáltatás")
    Container(order, "Rendelés szolgáltatás")
  }
  Rel(web_user, gateway, "HTTPS")
  Rel(mobile_user, gateway, "HTTPS")
  Rel(gateway, web_bff, "HTTPS")
  Rel(gateway, mobile_bff, "HTTPS")
  Rel(web_bff, catalog, "HTTP/gRPC")
  Rel(web_bff, cart, "HTTP/gRPC")
  Rel(mobile_bff, catalog, "HTTP/gRPC")
  Rel(mobile_bff, cart, "HTTP/gRPC")
  Rel(web_bff, order, "HTTP/gRPC")
  Rel(mobile_bff, order, "HTTP/gRPC")
```

Itt a gateway csak a hitelesítést és a durva útvonalválasztást végzi, a kliensspecifikus kompozíciót pedig a BFF-ek. Ez a feladatmegosztás minimalizálja a duplikációt, mert a közös keresztmetszeti funkciók a gatewayben maradnak, míg a kliensspecifikus logika a BFF-ekben.

14.4.4 4. Service mesh: a szolgáltatásközi forgalom szabályozása

Míg az API gateway és a BFF a peremre koncentrál, a **service mesh** a belső szolgáltatások közötti kommunikációt kezeli. Egy mikroszolgáltatás-architektúrában a szolgáltatások közvetlenül hívják egymást, és minden ilyen hívásnak meg kellene valósítania a retry-t, a circuit breakert, a TLS-t, a metrikagyűjtést stb. Ezeket a keresztmetszeti aggályokat a service mesh kiemeli az alkalmazáskódból, és a hálózati rétegbe helyezi.

A service mesh leggyakoribb megvalósítása az **oldalkocsi (sidecar) minta** alapján történik: minden szolgáltatáspéldány mellé egy proxy konténer (például Envoy) kerül, amely a bejövő és kimenő forgalmat kezeli. Az alkalmazás saját maga gyakran nem is tud a proxy jelenlétéről, csak a localhoston elérhető portot hívja; a proxy végzi a tényleges hálózati kommunikációt. Az oldalkocsikat egy központi **vezérlő sík (control plane)** konfigurálja, amely például Istio vagy Linkerd lehet (a konkrét termékek és verziók ellenőrzést igényelnek).

A service mesh által nyújtott képességek:

- **mTLS (mutual TLS):** A szolgáltatások közötti forgalom automatikusan titkosított, és a felek kölcsönösen hitelesítik egymást. Ez megnehezíti a belső hálózaton belüli lehallgatást és a hamis szolgáltatások beillesztését. A mTLS-t a mesh proxyk intézik, így az alkalmazásnak nem kell TLS-t implementálnia.
- **Forgalomirányítás:** A mesh lehetővé teszi a finom szemcsés routingot, például canary release, A/B tesztelés, kék-zöld telepítés, súlyozott terheléselosztás. Ez deklaratívan, a vezérlő síkon keresztül konfigurálható.
- **Hibatűrés:** Retry, timeout, circuit breaker és bulkhead beállítások deklaratívan megadhatók anélkül, hogy az alkalmazáskódot módosítanánk. Ez növeli a rendszer ellenálló képességét a hálózati hibákkal szemben.
- **Megfigyelhetőség:** A proxy automatikusan gyűjti a forgalmi metrikákat (kérések száma, hibák, késleltetés), és elosztott trace-t indít. Ez a hálózati megfigyelés alapja, és lehetővé teszi a szolgáltatások közötti kommunikáció részletes elemzését.
- **Biztonsági házirendek:** Szolgáltatások közötti hozzáférés-szabályozás (authorization policy), sebességkorlátozás, stb. A mesh központilag érvényesíti a nulla bizalom elvét a belső hálózaton.

A service mesh architektúrája tehát két síkra bontható:

- **Adat sík (data plane):** A szolgáltatások mellett futó proxyk, amelyek a tényleges forgalmat továbbítják.
- **Vezérlő sík (control plane):** A proxyk konfigurációját kezelő központi komponens, amely a kívánt házirendeket terjeszti.

A következő ábra szemlélteti a service mesh elhelyezkedését (Mermaid, container szinten):

```
C4Container
  title Service mesh: oldalkocsik és vezérlő sík
  Container_Boundary(eshop, "Rendszer") {
    Container(gateway, "API Gateway", "Perem")
    Container(catalog, "Katalógus szolgáltatás", "Üzleti logika")
    Container(cart, "Kosár szolgáltatás", "Üzleti logika")
    Container(catalog_sidecar, "Katalógus oldalkocsi", "Proxy (Envoy)")
    Container(cart_sidecar, "Kosár oldalkocsi", "Proxy (Envoy)")
    Container(control, "Service mesh vezérlő sík", "Istio/Linkerd")
  }
  Rel(gateway, catalog_sidecar, "HTTP")
  Rel(catalog_sidecar, catalog, "localhost")
  Rel(catalog_sidecar, cart_sidecar, "mTLS")
  Rel(cart_sidecar, cart, "localhost")
  Rel(control, catalog_sidecar, "Konfiguráció")
  Rel(control, cart_sidecar, "Konfiguráció")
```

Itt látható, hogy a szolgáltatások nem közvetlenül, hanem az oldalkocsikon keresztül kommunikálnak, a vezérlő sík pedig felügyeli a proxykat. Ez a topológia lehetővé teszi, hogy a keresztmetszeti aggályok a proxykban legyenek, az alkalmazáskód pedig tiszta maradjon.

Példa forgalomirányításra (Istio VirtualService, szemléltető):

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: catalog-vs
spec:
  hosts:
    - catalog-service
  http:
    - route:
        - destination:
            host: catalog-service
            subset: v1
          weight: 90
        - destination:
            host: catalog-service
            subset: v2
          weight: 10
      retries:
        attempts: 2
        perTryTimeout: 2s
```

Ez a konfiguráció a forgalom 90%-át a `v1`, 10%-át a `v2` verzióra irányítja, és legfeljebb 2 újrapróbálkozást enged, egyenként 2 másodperces timeout-tal. A mesh tehát deklaratívan, az alkalmazáskód módosítása nélkül valósít meg canary release-t és hibatűrést.

Hálózati megfigyelés: A service mesh egyik legnagyobb értéke a beépített observability. Mivel minden forgalom áthalad a proxykon, azok pontosan rögzíteni tudják a kérések számát, a válaszidőt, a hibakódokat és a késleltetést. A modern megfigyelési gyakorlat a **RED** metrikákat (Rate, Errors, Duration) javasolja a szolgáltatások egészségének mérésére, illetve a **USE** metrikákat (Utilization, Saturation, Errors) az erőforrásokra. A mesh ezeket automatikusan exportálja, és integrálható olyan eszközökkel, mint a Prometheus, Grafana, Jaeger vagy az OpenTelemetry (a termékek és verziók ellenőrzést igényelnek). Ezáltal a fejlesztőknek nem kell saját metrika-kódot írniuk minden szolgáltatásba.

A mesh által gyűjtött adatok lehetővé teszik a szolgáltatások közötti hívások topológiájának automatikus feltérképezését, a késleltetés forrásának azonosítását, és a hibák korai észlelését. Ez a fajta hálózati megfigyelés különösen fontos a magas rendelkezésre állású rendszereknél, ahol a hibák gyakran a szolgáltatások közötti interakcióban jelentkeznek.

14.4.5 5. API gateway vs. service mesh: mikor melyiket?

A két eszköz összehasonlítása segít a döntésben. Az alábbi táblázat a fő szempontokat foglalja össze:

Szempont	API gateway	Service mesh
Forgalom iránya	Északi-déli (külső-belső)	Keleti-nyugati (belső-belső)
Elhelyezkedés	A peremen, a rendszer belépési pontján	Minden szolgáltatás mellett
Hitelesítés	Külső kliensek azonosítása (OAuth2, API kulcs)	Szolgáltatások közötti mTLS, identitás
Rate limiting	Külső kliensek felé	Szolgáltatások közötti forgalomra is
Megfigyelhetőség	A külső kérések életciklusa	A belső hívások láncolata
Komplexitás	Viszonylag egyszerű üzemeltetés	Jelentős komplexitás, tanulási görbe

Szempont	API gateway	Service mesh
Bevezetési küszöb	Alacsony, gyakran már a kezdetektől	Magas, érett platformot igényel

A döntési szabály: ha a rendszer kevés szolgáltatásból áll, és a fő gond a külső kliensek kezelése, akkor egy API gateway elegendő. Ha viszont a szolgáltatások száma nagy, a belső kommunikáció bonyolult, és szükség van a hibatűrés és a megfigyelhetőség egységesítésére, akkor érdemes service mesht bevezetni. A kettő gyakran együtt is használható: a gateway a peremen, a mesh a belső hálózaton. A BFF pedig a gateway kliensspecifikus kiterjesztése, amelyet akkor érdemes bevezetni, ha a kliensek adatigényei jelentősen eltérnek.

14.4.6 6. Gyakori hibák és helyes megoldások

Ebben a szakaszban a tipikus hibákat és a mögöttes helyes elveket tárgyaljuk.

Hiba 1: Monolit gateway kialakulása. Ha a gatewaybe túl sok üzleti logikát (pl. kosár-összeállítás, árkalkuláció) tesznek, az idővel egy újabb monolitot eredményez, amely szűk keresztmetszet és karbantartási teher.
Helyes megoldás: A gateway maradjon vékony, csak a keresztmetszeti aggályokat kezelje. Az üzleti logika a belső szolgáltatásokban vagy egy BFF-ben legyen. A gateway feladata a routing, a hitelesítés és a rate limit, nem a domain logika.

Hiba 2: BFF-ek elszaporodása. Minden klienshez külön BFF készül, de a csapatok nem osztják meg a közös komponenseket, így a hitelesítési és transzformációs logika duplikálódik.
Helyes megoldás: A BFF-eket platformcsapat vagy közös könyvtárak támogassák; a közös logikát (pl. token-ellenőrzés) emeljük ki egy megosztott rétegbe, vagy hagyjuk a gatewayre. A BFF-ek száma legyen indokolt, és ne szaporodjon minden apró kliensvariációhoz.

Hiba 3: Service mesh bevezetése túl korán. A mesh operációs komplexitása és erőforrásigénye miatt kis rendszerekben felesleges, sőt káros lehet. A mesh fenntartása speciális szaktudást igényel, és ha nincs elég szolgáltatás, a haszon elmarad a költségtől.
Helyes megoldás: Előbb érjük el, hogy a szolgáltatások száma és a belső kommunikáció indokolttá tegye. Kezdjük a megfigyelhetőség és a mTLS szükségességének felmérésével. Gyakran elegendő egy jól konfigurált API gateway és néhány beépített hibatűrési mechanizmus.

Hiba 4: A gateway megkerülése. Ha egy kliens közvetlenül eléri a belső szolgáltatásokat, a hitelesítési és rate limit házirendek megkerülhetők.
Helyes megoldás: Hálózati szinten (pl. Kubernetes NetworkPolicy) biztosítsuk, hogy a belső szolgáltatások csak a gatewaytől fogadjanak forgalmat. Ez a nulla bizalom elv egyik alapeleme, és megakadályozza a házirendek kijátszását.

Hiba 5: A mesh konfiguráció driftje. Ha a mesh beállításait kézzel módosítják, a környezetek eltérnek, és nehéz reprodukálni a hibákat.
Helyes megoldás: A mesh konfigurációját is infrastruktúra mint kódként (IaC) kezeljük, és GitOps-szal telepítsük. Így minden változás nyomon követhető, és a környezetek konzisztensek maradnak.

A fenti hibák felismerése és a helyes megoldás indoklása a fejezet egyik tanulási célja. A labor során ezekre ügyelni kell.

14.4.7 7. Labor: Gateway- és BFF-topológia tervezése

A labor célja, hogy a résztvevő egy konkrét, de fiktív rendszerre megtervezze a perem és a szolgáltatásközi forgalom kezelését, dokumentálja döntéseit, és meghatározza az ellenőrzési szempontokat. A feladat nem éles implementáció, hanem tervezési dokumentum (vagy jegyzetfüzet) elkészítése, amely diagrams-as-code elemeket és ADR-eket tartalmaz. Az alábbiakban megadjuk a kiinduló állapotot, az elvárt eredményt, az ellenőrzési szempontokat és a visszaállítási utat.

14.4.8 7.1 Kiinduló állapot

Adott egy fiktív, mikroszolgáltatás-alapú **e-kereskedelmi rendszer** három belső szolgáltatással:

- **Katalógus szolgáltatás:** termékadatok lekérdezése, listázása.
- **Kosár szolgáltatás:** a felhasználó kosarának kezelése.
- **Rendelés szolgáltatás:** megrendelések leadása és lekérdezése.

A kliensek: webes böngésző (desktop) és mobilalkalmazás (iOS/Android). A rendszernek támogatnia kell a partneri API-hozzáférést is (külső viszonteladók). A jelenlegi állapotban nincs egységes perem; a kliensek közvetlenül érik el a szolgáltatásokat, ami káoszt és biztonsági kockázatot okoz. A cél egy API gateway és adott esetben BFF-ek bevezetése, valamint a belső forgalom szabályozásának megtervezése (service mesh opcionális).

14.4.9 7.2 Feladat

1. Készítsen C4 context és container szintű diagramot a tervezett topológiáról Mermaid vagy PlantUML formában. Jelölje a rendszerhatárt, a klienseket, a gatewayt, a BFF-eket (ha szükséges) és a belső szolgáltatásokat.
2. Írjon legalább három ADR-t a következő döntésekről:
    - API gateway bevezetése (vagy annak elvetése) és a kiválasztott termék (pl. Kong, Envoy, saját).
    - BFF-ek használata (igen/nem, és ha igen, hogyan oszlanak meg a kliensek között).
    - Service mesh bevezetése (igen/nem, indoklás). Az ADR-ek tartalmazzák a kontextust, a döntést, az alternatívákat és a következményeket.
3. Definiáljon legalább két **quality attribute scenario**-t a peremre vonatkozóan (pl. hitelesítés késleltetése, rate limit hatása). Használja az inger, környezet, válasz, mérőszám szerkezetet.
4. Határozza meg a **rate limit** és a **hitelesítés** konkrét beállításait a webes, mobil- és partnerkliensekre. Indokolja a választást.
5. Készítsen **ellenőrzési szempontokat** a topológia tesztelésére, különválasztva a **mért eredményt** (pl. tényleges késleltetés terhelés alatt) és a **várt eredményt** (pl. a QAS-ban megadott küszöbérték). Amennyiben a labor során szimulációt futtat (pl. terhelésgenerátorral), akkor azt izolált környezetben tegye, és a mérést rögzítse a környezet, a konfiguráció, az időpont és a bizonytalanság feltüntetésével.

14.4.10 7.3 Elvárt eredmény

A résztvevő egy verziózott dokumentumot ad át (pl. Markdown vagy Jupyter notebook), amely tartalmazza:

- A C4 diagramokat diagrams-as-code formában.
- Az ADR-eket.
- A QAS-okat.
- A konfigurációs javaslatokat (YAML formátumban, szemléltető jelleggel, éles hitelesítő adatok nélkül).
- Az ellenőrzési szempontokat, ahol a mért és várt eredményeket külön táblázatban vagy szakaszban szerepelteti.
- Egy rövid összefoglalót arról, hogyan kerülték el a 6. szakaszban felsorolt gyakori hibákat.

A dokumentum nem tartalmazhat valós hitelesítő adatot, belső URL-t vagy ügyfélnevet; mindenütt helyőrzők (pl. `https://auth.example.com`, `tenant-123`) szerepeljenek.

14.4.11 7.4 Ellenőrzési szempontok

A labor értékelése a következő szempontok alapján történik:

- A C4 diagramok context és container szinten helyesek, a rendszerhatár jelölt, a komponensek felelősséggel és interfésszel szerepelnek.
- Az ADR-ek tartalmazzák a döntés okát és a feladott alternatívákat, állapotuk (elfogadott, elvetett) jelölt.
- A QAS-ok mérőszáma mértékegységgel és mérési ablakkal rendelkezik.
- A rate limit és hitelesítési beállítások indokoltak, és a különböző kliensek eltérő igényeit tükrözik.
- A mért és várt eredmények világosan elkülönülnek.
- A dokumentum verziózott, és a döntésekhez visszakereshető hivatkozások tartoznak.

14.4.12 7.5 Visszaállítási út

Mivel a labor tervezési feladat, éles visszaállításra nincs szükség. Azonban ha a résztvevő szimulációt futtat (pl. egy helyi Docker Compose környezetben), akkor a következőket kell betartani:

- A szimulációt izolált környezetben (pl. lokális minikube, Docker Desktop) futtassa, ne éles rendszeren.
- A szimulációs konfigurációt verziózza, és a labor végén állítsa vissza a környezetet (pl. `docker compose down -v`, `minikube delete`).
- Minden destruktív műveletet (pl. adatbázis-törlés) csak a dokumentumban, figyelmeztetéssel együtt említsen, és a szimulációban is izoláltan hajtsa végre.

A labor eredménye egy tanulási célú dokumentum, nem termelési döntés; ezt a dokumentum elején fel kell tüntetni.

14.4.13 8. Összefoglalás

Az API gateway, a BFF és a service mesh három különböző, de egymást kiegészítő eszköz a perem és a szolgáltatásközi forgalom kezelésére. Az API gateway a külső kliensek belépési pontja, amely egységesíti a hitelesítést, a rate limitinget és az útvonalválasztást. A BFF a kliensek eltérő igényeire szabott peremréteg, amely csökkenti a kliens komplexitását és optimalizálja az adatátvitelt. A service mesh a belső szolgáltatások közötti kommunikációt szabályozza, oldalkocsi proxyk és vezérlő sík segítségével, biztosítva az mTLS-t, a hibatűrést és a beépített megfigyelhetőséget. A helyes architektúra kiválasztásakor a forgalom irányát, a rendszer méretét és a szervezeti érettséget kell mérlegelni, elkerülve a monolit gateway, a BFF-ek elszaporodása és a service mesh idő előtti bevezetése csapdáit.

14.4.14 Ellenőrzőlista

- ☐ A peremréteg egyetlen belépési pontot biztosít a külső klienseknek.
- ☐ A hitelesítés és a rate limit a gatewayen történik, nem a belső szolgáltatásokban.
- ☐ A BFF-ek kliensspecifikusak, és nem tartalmaznak felesleges üzleti logikát.
- ☐ A service mesh (ha használjuk) mTLS-t és automatikus metrikagyűjtést biztosít.
- ☐ A belső szolgáltatások csak a gatewayen vagy a mesh proxykon keresztül érhetők el.
- ☐ A konfiguráció diagrams-as-code és IaC formában verziózott.
- ☐ Az ADR-ek dokumentálják a gateway, BFF és mesh döntéseket.
- ☐ A laborban a mért és várt eredmények külön vannak választva.

14.4.15 Önellenőrző kérdés

Egy csapat azt tervezi, hogy bevezet egy API gatewayt, de a belső szolgáltatások közötti kommunikáció továbbra is közvetlen, TLS nélkül, és a fejlesztők saját retry logikát írnak. Milyen problémákat vet fel ez a megközelítés, és hogyan oldaná meg őket egy service mesh bevezetésével? Válaszában térjen ki a biztonságra, a hibatűrésre és a megfigyelhetőségre.

15 Enterprise és domain-architektúrák

15.1 Domain-Driven Design: bounded context és ubiquitous language

A Domain-Driven Design (DDD) nem egy technológia vagy egy réteg, hanem egy gondolkodási keret: a szoftver középpontjába a domainmodellt állítja, és azt kívánja elérni, hogy a modell, a nyelv és a kód szoros összhangban legyen az üzleti valósággal. Ebben az alfejezetben a DDD három stratégiai alappillérét járjuk körbe: a domainmodellt, a bounded contextet és a ubiquitous language-et, majd bevezetjük a modell építőelemeit (entitás, értékobjektum, aggregate, domain event), és megmutatjuk, hogyan ábrázolhatók a kontextusok közötti kapcsolatok context map segítségével. A fejezet laborjában a hallgató egy bounded context térképet készít, amelyet döntési naplóval és ellenőrzési szempontokkal dokumentál.

15.1.1 Mi a domainmodell?

A domainmodell a problématér – az üzleti folyamatok, szabályok és fogalmak – absztrakt, szoftverben ábrázolható leképezése. A DDD szóhasználatában a „domain” az a szakterület, amelyre a szoftver megoldást nyújt: például banki hitelezés, e-kereskedelmi rendeléskezelés, kórházi betegnyilvántartás vagy logisztikai útvonaltervezés. A modell nem az adatbázis tábláinak vagy a felhasználói felület képernyőinek a másolata, hanem az üzleti szabályok és a hozzájuk tartozó fogalmak koherens rendszere.

Egy jól kialakított domainmodell a következőket teszi lehetővé:

- A domain szakértők és a fejlesztők ugyanazt a nyelvet beszélik, így a követelmények félreértése csökken.
- Az üzleti logika egy helyen, a technikai infrastruktúrától elkülönítve jelenik meg, ami egyszerűbbé teszi a változtatást.
- A modell explicit módon rögzíti a szabályokat – például „egy megrendelés csak akkor válhat visszaigazolttá, ha minden tétel rendelkezésre áll” –, így a kód nem a fejlesztő fejében élő implicit tudás.

A DDD nem azt állítja, hogy minden szoftverprojektben szükség van ilyen mélységű modellezésre. Erősen üzleti logikával terhelt, hosszú életciklusú rendszerek esetén a befektetés megtérül; egyszerű CRUD-alkalmazásoknál a teljes DDD-apparátus túlzás lehet. A fejezet további része feltételezi, hogy a rendszer rendelkezik érdemi domainlogikával.

15.1.2 Ubiquitous language: a közös nyelv

A ubiquitous language (közös nyelv, univerzális nyelv) a DDD egyik legfontosabb fogalma. Olyan, a teljes csapat által közösen használt, egyértelmű szókincs és kifejezéskészlet, amely a domain szakértők és a technikai csapat között hidat képez. Ez a nyelv nem egy statikus szótár, hanem a domainmodell élő része: minden fogalom, művelet és szabály ezen a nyelven jelenik meg a beszélgetésekben, a dokumentációban, a kódban és a tesztekben egyaránt.

A közös nyelv kialakításának alapelvei:

1. **A domain szakértő szavait használjuk.** Ha az üzleti oldal „megrendelés visszaigazolásáról” beszél, akkor a kódban ne `confirmOrder()` metódust hívjunk, hanem `confirmMegrendeles()`-t vagy – a gyakorlatban gyakran angol nyelvű kód esetén – `confirmOrder()`, de a fogalom neve akkor is a domain nyelvéből ered, és a csapat minden tagja ugyanazt érti alatta.
2. **Nincs kétértelműség.** Ugyanaz a szó nem jelenthet két különböző dolgot ugyanabban a kontextusban. Ha az üzleti szakértő „szerződés” alatt a keretszerződést érti, a jogi osztály viszont az egyedi megállapodást, akkor a nyelvet pontosítani kell, vagy külön bounded contextet kell bevezetni.
3. **A nyelv a modell része.** A fogalmak nem csak elnevezések, hanem viselkedéssel és kapcsolatokkal rendelkező modell elemek. Például a „megrendelés” nem csupán egy adatbázis-sor, hanem egy entitás, amelynek állapotátmenetei vannak („létrehozva”, „visszaigazolva”, „kiszállítva”).
4. **A kód a nyelv megtestesülése.** Ha a domain szakértő azt mondja: „a megrendelés akkor válik visszaigazolttá, ha a raktár minden tételt lefoglalt”, akkor a kódban ennek a szabálynak explicite meg kell jelennie, nem pedig szétszórtan, adatbázis-triggerekben vagy a felhasználói felület eseménykezelőiben.

Gyakori hiba, hogy a csapat létrehoz egy szójegyzéket, de a kód továbbra is technikai kifejezéseket használ (`OrderEntity`, `CustomerTable`, `updateStatus`). A ubiquitous language akkor él, ha a modell, a kód és a beszélgetés ugyanazokat a fogalmakat használja. A DDD ezt úgy fogalmazza meg, hogy a nyelv a modellből fakad, nem pedig egy külső dokumentumból.

Példa egy online könyvtár domainjéből:

- Domain szakértői kifejezés: „a kölcsönzés lejárati ideje 30 nap, hosszabbítás legfeljebb kétszer, egyenként 14 nap”.
- Ubiquitous language fogalmak: `Kölcsönzés`, `Lejárat`, `Hosszabbítás`, `Példány`, `KölcsönzésiSzabály`.
- A kódban: `kolcsonzes.hosszabbit(aktualisDatum)` metódus, amely ellenőrzi a szabályt.

15.1.3 Bounded context: a modell határai

A bounded context (korlátos kontextus, határolt kontextus) az a határ, amelyen belül egy adott domainmodell és a hozzá tartozó ubiquitous language érvényes. Egy nagy rendszerben ugyanaz a kifejezés különböző kontextusokban más jelentést hordozhat, és ez nem hiba, hanem a valóság tükröződése. A DDD stratégiai tervezésének egyik fő feladata e határok explicit felismerése és rögzítése.

Tekintsünk egy e-kereskedelmi rendszert, amelyben a „Termék” fogalom megjelenik a katalógusban, a raktárban és a számlázásban is:

- A **katalógus kontextusban** a Termék attribútumai: név, leírás, kategória, ár, marketing képek.
- A **raktár kontextusban** a Termék attribútumai: cikkszám, fizikai méret, tömeg, polc helye, aktuális készlet.
- A **számlázás kontextusban** a Termék attribútumai: adókulcs, számlázási megnevezés, egységár, kedvezménykategória.

Minden kontextusban a „Termék” más viselkedéssel és más szabályokkal rendelkezik. Ha egyetlen `Product` osztályt próbálnánk használni mindhárom helyen, az osztály túl nagy, kohéziója alacsony lenne, és a változtatások keresztülgyűrűznének a rendszeren. A bounded context éppen azt mondja ki, hogy ezek külön modellek, külön nyelvekkel, és a kontextusok közötti kapcsolatot explicit módon kell kezelni.

A bounded context tehát nem azonos a technikai szolgáltatással, bár gyakran egybeesik vele. Egy bounded context lehet egy modul, egy mikroszolgáltatás, vagy akár egy nagyobb monolit egy jól elhatárolt része. A lényeg a modell és a nyelv konzisztenciájának határa, nem a futtatási egység.

A bounded context kialakításának szempontjai:

- **Nyelvi határok:** hol kezd el ugyanaz a szó más jelentést kapni?
- **Üzleti felelősség:** melyik szervezeti egység vagy csapat birtokolja a modellt?
- **Konzisztenciaigény:** mely objektumoknak kell erős, tranzakciós konzisztenciában maradniuk?
- **Változási gyakoriság:** mely részek változnak együtt, és melyek függetlenül?

A DDD nem írja elő a kontextusok optimális méretét. A túl kicsi kontextusok integrációs terhelést növelnek, a túl nagyok pedig a modell kohézióját rontják. A jó határ megtalálása iteratív folyamat, amelyben a csapat a nyelvet és az üzleti folyamatokat vizsgálja.

15.1.4 Építőelemek: entitás, értékobjektum, aggregate, domain event

A bounded contexten belül a domainmodellt tipikus építőelemekből állítjuk össze. Ezek nem kötelező minták, de a leggyakoribb és leghasznosabb absztrakciók.

15.1.5 Entitás (Entity)

Az entitás olyan objektum, amelynek van identitása, és ez az identitás az életciklusa során állandó, még akkor is, ha az attribútumai változnak. Például egy `Megrendelés` entitás azonosítója (`megrendelesId`) nem változik, miközben állapota „létrehozva” állapotból „visszaigazolva” állapotba kerül, a tételei módosulnak, a szállítási címe frissül. Az entitások egyenlőségét az azonosítójuk határozza meg, nem az attribútumaik.

Példa (pseudocode):

```
class Megrendeles {
    MegrendelesId id;        // identitás
    UgyfelId ugyfelId;
    List<MegrendelesTetel> tetelek;
    MegrendelesAllapot allapot;

    void visszaigazol(KeszletFoglalas foglalas) {
        // domain szabályok
    }
}
```

15.1.6 Értékobjektum (Value Object)

Az értékobjektum identitás nélküli, attribútumokkal leírt, változtathatatlan (immutable) objektum. Két értékobjektum egyenlő, ha minden attribútuma megegyezik. Például egy `Penz` értékobjektum `(osszeg, devizanem)` párból áll: két `Penz(1000, "HUF")` egyenlő, nincs szükség külön azonosítóra. Az értékobjektumok jól modellezik a domain leíró fogalmait: cím, időtartam, szín, pénzösszeg, mértékegység.

Fontos, hogy az értékobjektum immutable legyen, mert így biztonságosan megosztható, és nem okoz rejtett mellékhatásokat.

15.1.7 Aggregate

Az aggregate (agregátum) konzisztenciahatár: entitások és értékobjektumok olyan csoportja, amelyet egyetlen egységként kezelünk, és amelynek konzisztenciáját egy gyökérentitás, az aggregate root biztosítja. A külső objektumok csak a rooton keresztül férhetnek hozzá az aggregate belső elemeihez, és a módosítások tranzakciós határa is az aggregate. Például a `Megrendeles` aggregate root tartalmazza a `MegrendelesTetel` értékobjektumokat. A rendszer más részei nem nyúlhatnak közvetlenül a tételekhez, csak a `Megrendeles` metódusain keresztül. Ez biztosítja, hogy a domain szabályok – például „egy megrendelés összértéke nem lehet negatív” – mindig érvényesüljenek.

Az aggregate tervezésekor fontos kérdés: mely objektumoknak kell együtt, egy tranzakcióban változniuk? Amelyek nem igényelnek erős konzisztenciát, azok külön aggregate-be kerülhetnek, és a konzisztencia megvalósítható eseményekkel vagy aszinkron frissítéssel.

15.1.8 Domain event

A domain event (domain esemény) egy olyan esemény, amely a domainben valami fontos történést rögzít, és amely más részek számára érdekes lehet. Például `MegrendelesVisszaigazolva`, `SzamlaKiallitva`, `KeszletCsokkentve`. A domain event elnevezése mindig múlt idejű, mert egy megtörtént tényt ír le. Az események lehetővé teszik a bounded contexte közötti laza csatolást: a kontextusok nem közvetlenül hívják egymást, hanem eseményeket publikálnak és figyelnek.

A domain event tartalmazza az esemény azonosítóját, időbélyegét, a érintett aggregate azonosítóját és a szükséges adatokat. Az események a DDD és az eseményvezérelt architektúrák közötti hidat képezik.

15.1.9 Context map: a határok közötti kapcsolatok

A context map (kontextustérkép) a bounded contexte közötti kapcsolatok áttekintő ábrázolása. Megmutatja, hogy az egyes kontextusok hogyan viszonyulnak egymáshoz, milyen mintázat szerint integrálódnak, és hol vannak a modellfordítási pontok. A context map nem folyamatábra, hanem stratégiai nézet: a csapatok közötti együttműködési és integrációs mintákat rögzíti.

A DDD irodalom több kapcsolattípust azonosít; a leggyakoribbak:

- **Partnership (partnerség):** két csapat szorosan együttműködik, a modellek közös fejlesztés alatt állnak, a határokat egyeztetik. Például egy induló termékben a Rendeléskezelés és a Számlázás csapata közösen alakítja ki az interfészt.

- **Shared Kernel (közös mag):** két kontextus osztozik egy közös modellen vagy kódrészen. Ez csökkenti a duplikációt, de erős függőséget teremt, ezért csak kis, stabil részekre érdemes használni.
- **Customer/Supplier (vevő/szállító):** az egyik kontextus a másik szolgáltatásait fogyasztja. A szállító határozza meg a szerződést, de figyelembe veszi a vevő igényeit. Például a Raktár kontextus szállítja a készletinformációkat a Rendeléskezelésnek.
- **Conformist (alkalmazkodó):** a vevő kontextus elfogadja a szállító modelljét változtatás nélkül, mert nincs lehetősége vagy indoka azt befolyásolni. Gyakori külső szolgáltatásoknál.
- **Anticorruption Layer (ACL, korrupcióellenes réteg):** a vevő kontextus egy fordítóréteget épít, hogy a saját modelljét megvédje a szállító modelljétől. Az ACL lefordítja a külső fogalmakat a belső nyelvre. Például egy külső adóhatósági API által használt adatformátumot a Számlázás kontextus a saját `Szamla` modelljére fordítja.
- **Open Host Service (nyílt gazdaszolgáltatás):** a szállító kontextus egy jól dokumentált, stabil API-t kínál minden fogyasztónak. Az API lehet REST, gRPC vagy üzenetsor.
- **Published Language (publikált nyelv):** a kontextusok közötti kommunikáció egy közösen elfogadott, verziózott formátumon (pl. JSON séma, Avro, Protobuf) alapul.

A context map ábrázolásához diagrams-as-code eszközt, például Mermaid-t használunk, hogy a térkép a kódbázissal együtt verziózható legyen. Az alábbi példa egy egyszerűsített e-kereskedelmi context map:

```
flowchart LR
    U[Ügyfél] -->|Rendelés leadása| R[Rendeléskezelés]
    R -->|Számla kérése| SZ[Számlázás]
    SZ -->|Számla kiállítva esemény| R
    R -->|Készlet foglalás| RA[Raktár]
    RA -->|Készlet visszaigazolás| R
    SZ -->|Adatszolgáltatás| K[Külső adóhatóság]
    R -.->|ACL| K
```

A diagramon a nyilak címkéi a kommunikáció irányát és tartalmát jelzik, a szaggatott vonal pedig anticorruption layer-t. A kapcsolatokon a protokoll, a szinkronitás és a hibatűrés is feltüntethető, például „REST, szinkron, timeout 3 s” vagy „esemény, aszinkron, retry”. A könyv diagramokra vonatkozó szerződése szerint minden diagramnak legyen címe, jelmagyarázata és feltételjegyzete; itt a feltételjegyzet például: „A diagram az aktuális állapotot tükrözi; a külső adóhatóság kapcsolatot a megfelelőségi csapat validálta.”

A context map elkészítésének lépései:

1. Azonosítsuk a bounded contexteket a nyelvi és üzleti határok alapján.
2. Minden kontextushoz rendeljünk felelősséget és a csapatot, amely birtokolja.
3. Határozzuk meg a kontextusok közötti kapcsolatokat, és nevezzük meg a mintázatot.
4. Dokumentáljuk a kapcsolatokon átvitt adatokat és a használt protokollt.
5. Rögzítsük a térképet diagrams-as-code formában, és a döntéseket ADR-ben.

15.1.10 Gyakorlati példa: online könyvtár

Vegyünk egy kis online könyvtári rendszert, amelyben a felhasználók könyveket keresnek, kölcsönöznek, hosszabbítanak, és késedelmi díjat fizetnek. A domain szakértőkkel folytatott beszélgetés során a következő nyelvi határok rajzolódnak ki:

- **Katalógus:** a könyvek bibliográfiai adatai (szerző, cím, ISBN, kiadó, leírás). A fogalom itt: `Könyv`.

- **Kölcsönzés:** a fizikai példányok és a kölcsönzési folyamat. A fogalom itt: `Példány` (leltári szám, állapot, polc), `Kölcsönzés`, `Hosszabbítás`, `Lejárat`.
- **Olvasó:** a felhasználók nyilvántartása, tagság, elérhetőségek. A fogalom: `Olvasó`, `Tagság`.
- **Számlázás:** késedelmi díjak, számlák, befizetések. A fogalom: `Díj`, `Számla`, `Befizetés`.

A „Könyv” szó a Katalógusban a bibliográfiai rekordot jelenti, a Kölcsönzésben viszont fizikai példányt. Ezért a Kölcsönzés kontextusban nem `Könyv`, hanem `Példány` a megfelelő kifejezés. A közös nyelv tehát kontextusonként eltérő, és ez a választás csökkenti a félreértést.

A context map ebben a példában:

```
flowchart LR
    O[Olvasó] -->|Kölcsönzés kérése| K[Kölcsönzés]
    K -->|Példány adatok| Kat[Katalógus]
    K -->|Díj esemény| SZ[Számlázás]
    SZ -->|Befizetés visszaigazolás| K
    K -->|Értesítés| O
```

Itt a Katalógus és a Kölcsönzés között customer/supplier viszony van: a Kölcsönzés kontextus fogyasztja a Katalógus által publikált `Példány` adatokat, de a saját modelljét használja. A Kölcsönzés és a Számlázás között eseményalapú kapcsolat van: a Kölcsönzés `KesedelmiDíjKeletkezett` eseményt publikál, a Számlázás pedig `BefizetesVisszaigazolva` eseményt küld vissza. A protokoll lehet aszinkron üzenetsor (pl. `KesedelmiDíj` topic), a hibatűrés pedig retry mechanizmus.

A példában az aggregate-ek:

- A Kölcsönzés kontextusban az aggregate root `Kölcsönzés`, amely tartalmazza a `Példány` azonosítóját, a kölcsönzés dátumát, a lejáratot és a hosszabbítások számát. A `Hosszabbítás` értékobjektum (dátum, indok) lehet.
- A Számlázás kontextusban az aggregate root `Szamla`, amely `Díj` értékobjektumokat tartalmaz.

15.1.11 Labor: bounded context térkép készítése

A labor célja, hogy a hallgató saját maga készítsen bounded context térképet egy adott domainre, és a döntéseit dokumentálja. A feladat nem futtatható mérést igényel, hanem tervezési és dokumentációs tevékenységet; a „mért” eredmény az ellenőrzési szempontok teljesülésének vizsgálatát jelenti a saját dokumentumon.

15.1.12 Feladat

Válasszon egy fiktív vagy valós, de publikusan leírható domain-t (például „gyógyszertári rendeléskezelés”, „biztosítási kárbejelentés”, „étterem asztalfoglalás”).

1. **Rögzítse a kontextust:** írja le a domain nevét, a legfontosabb üzleti folyamatokat, és hogy milyen forrásból származnak a fogalmak (pl. interjú egy domain szakértővel – szimulálva, vagy publikus dokumentáció). A kontextus legyen rövid, de egyértelmű.
2. **Gyűjtse a fogalmakat:** készítsen listát a domainben előforduló főnevekről és igékről. Jelölje, hogy mely szavaknak lehet több jelentése különböző helyeken.
3. **Azonosítsa a bounded contexteket:** a nyelvi határok és az üzleti felelősségek alapján ossza a fogalmakat kontextusokba. Minden kontextusnak adjon nevet és egy rövid felelősségi leírást.

4. **Rajzolja meg a context mapet** diagrams-as-code formában (Mermaid, PlantUML vagy Structurizr DSL). A diagramon tüntesse fel a kontextusokat, a kapcsolatok irányát, a kapcsolat típusát (pl. customer/supplier, ACL, esemény), a protokollt és a szinkronitást.
5. **Készítsen döntési naplót** a határokról. Minden fontos döntéshez használja az ADR-sablont: kontextus, döntés, alternatívák, következmények, állapot.
6. **Ellenőrizze a térképet** a lenti ellenőrzési szempontok alapján, és dokumentálja a várt és a mért eredményt külön.

15.1.13 Átadandó eredmény

Egyetlen dokumentum vagy jegyzetfüzet (pl. Markdown, Jupyter notebook), amely tartalmazza:

- A rögzített kontextust.
- A fogalomlistát és a nyelvi határok indoklását.
- A bounded context térképet diagrams-as-code formában.
- A döntési naplót (legalább 3 ADR-bejegyzés).
- Az ellenőrzési szempontok táblázatát a várt és mért eredménnyel.

15.1.14 Döntési napló sablon

```
### ADR-001: A Katalógus és a Kölcsönzés külön bounded context

**Kontextus:** A „Könyv" fogalom a bibliográfiai keresésben és a fizikai példány kezelésében
↪ mást jelent. A két terület eltérő ütemben változik, és más csapatok felelnek értük.

**Döntés:** Két külön bounded contextet vezetünk be: Katalógus és Kölcsönzés. A Katalógus
↪ publikálja a `Konyv` entitást, a Kölcsönzés a `Peldany` entitást használja.

**Alternatívák:** (1) Egyetlen `Konyv` entitás mindkét célra. (2) Közös adatbázis két
↪ alkalmazással. Elvetve, mert a fogalmi különbség és a változási ütem eltérése miatt a modell
↪ kohéziója romlana.

**Következmények:** A két kontextus közötti integráció szükséges; customer/supplier viszony,
↪ REST API-val. A Kölcsönzés nem tárolja a bibliográfiai adatokat, csak a példány
↪ azonosítóját.

**Állapot:** Elfogadva, 2025-01-15.
```

15.1.15 Ellenőrzési szempontok

#	Szempont	Várt eredmény	Mért eredmény
1	A bounded contexte száma	Legalább 3, legfeljebb 6	A hallgató saját térképén szereplő kontextusok száma
2	Minden kontextusnak van neve és felelősségi leírása	Igen, minden kontextus	Igen/nem, indoklással
3	A nyelvi határok indoklása dokumentált	Legalább 2 példa a többjelentésű fogalmakra	A hallgató által azonosított példák száma

#	Szempont	Várt eredmény	Mért eredmény
4	A kapcsolatokon a protokoll és szinkronitás jelölve	Minden kapcsolat	Hány kapcsolatnál hiányzik
5	A context map diagrams-as-code	Mermaid/PlantUML kód	A használt formátum
6	Döntési napló ADR-ekkel	Legalább 3 ADR	A ténylegesen elkészített ADR-ek száma
7	Nincs közös adatbázis két kontextus között (vagy ha van, az indokolt)	Nincs vagy indokolt	A hallgató értékelése

A „mért eredmény” oszlopot a hallgató tölti ki, miután elkészítette és átnézte a saját dokumentumát. Ez nem egy futtatott teszt, hanem önellenőrzés, amelynek célja a hiányosságok feltárása.

15.1.16 Gyakori hibák és elkerülésük

- **Egyetlen kanonikus modell mindenre.** Ha a csapat ragaszkodik ahhoz, hogy egyetlen `Termék` vagy `Ügyfél` entitás létezzen az egész rendszerben, akkor a kontextusok közötti fogalmi különbségek elmosódnak, és a modell túlterheltté válik. A bounded context éppen ezt a hibát hivatott orvosolni.
- **A ubiquitous language nem épül be a kódba.** Ha a domain szakértői kifejezések csak a dokumentációban élnek, a kódban viszont `dataManager`, `updateEntity` stb. szerepel, akkor a nyelv nem tölti be a szerepét.
- **Túl kicsi vagy túl nagy bounded contextek.** A túl sok apró kontextus integrációs káoszt okoz, a túl kevés nagy kontextus pedig a modell belső kohézióját rontja. A határokat a nyelv és a változási ütem alapján kell meghúzni.
- **Aggregate határok hiánya.** Ha a fejlesztők közvetlenül módosítják az aggregate belső objektumait, a domain szabályok megkerülhetők. Az aggregate rootnak kell minden módosítást kezelnie.
- **Közvetlen adatbázis-megosztás.** Két bounded context közös táblákon osztozik, ami rejtett függőséget és változtatási kockázatot jelent. Helyette explicit API vagy események használata javasolt.
- **A context map hiánya.** A kontextusok közötti kapcsolatok nincsenek dokumentálva, így a csapat nem látja át az integrációs pontokat. A context map elkészítése és karbantartása a DDD stratégiai tervezés alapja.

15.1.17 Ellenőrzőlista a fejezethez

- Meg tudja magyarázni a domainmodell, a bounded context és a ubiquitous language fogalmát?
- Tud példát mondani arra, hogy ugyanaz a szó két különböző bounded contextben más jelentést hordoz?
- Ismeri az entitás, értékobjektum, aggregate és domain event szerepét és különbségeit?
- Képes context mapet rajzolni diagrams-as-code formában, a kapcsolatok mintáinak megnevezésével?

- A laborban rögzítette a kontextust, azonosította a nyelvi határokat, és dokumentálta a döntéseket ADR-ben?
- Az ellenőrzési szempontoknál a várt és a mért eredményt különválasztotta?

15.1.18 Önellenőrző kérdés

Tegyük fel, hogy egy e-kereskedelmi rendszerben a „Termék” fogalmat a katalógus, a raktár és a számlázás egyaránt használja. Magyarázza el, miért érdemes ezeket külön bounded contextként kezelni, és milyen context map kapcsolatot javasolna a katalógus és a raktár között? Válaszában térjen ki a ubiquitous language szerepére és a lehetséges integrációs mintákra.

A SOA, ESB és enterprise integráció alfejezet (SOA, ESB és enterprise integráció) az itt bevezetett bounded context és context map fogalmakra épít, amikor a szolgáltatásorientált architektúrák és a vállalati integrációs minták kapcsolatát tárgyalja.

15.2 SOA, ESB és enterprise integráció

A szolgáltatásorientált architektúra (SOA) és az enterprise service bus (ESB) hosszú ideig a vállalati integráció alapértelmezett megközelítése volt. A fejezet célja, hogy megértse a mögöttes elveket, a szolgáltatásszerződés szerepét, az újrafelhasználás ígéretét és buktatóit, valamint az ESB előnyeit és a belőle fakadó központi szűk keresztmetszet kockázatát. A fogalmakat egy fiktív kiskereskedelmi vállalat, az *Acme Retail* példáján keresztül mutatjuk be. A példa nem valós ügyféladatokat használ; minden név, URL és hitelesítő adat helyőrző.

15.2.1 1. A szolgáltatásorientált architektúra fogalma

A szolgáltatásorientált architektúra olyan architekturális stílus, amely a rendszer üzleti képességeit lazán csatolt, újrafelhasználható szolgáltatásokként valósítja meg. Egy szolgáltatás nem egyszerűen egy távoli eljáráshívás vagy egy REST-végpont; a szolgáltatás üzleti jelentéssel bíró, önállóan telepíthető és verziózható egység, amely jól definiált szerződésen keresztül érhető el. A SOA klasszikus jellemzői a következők:

- **Üzleti képesség-alapú határok:** a szolgáltatások nem technikai rétegek (például adatbázis-hozzáférés vagy felhasználói felület), hanem üzleti képességek, például rendeléskezelés, készletkezelés vagy ügyfélkezelés. Ez a felosztás összhangban áll a Domain-Driven Design bounded context fogalmával, amelyet az előző egység tárgyalt.
- **Laza csatolás:** a szolgáltatások a szerződésen keresztül kommunikálnak, így az egyik implementációjának belső változása nem törheti el a másikat, amíg a szerződés változatlan marad.
- **Interoperabilitás:** a szolgáltatások különböző technológiákon is alapulhatnak, ha a kommunikáció szabványos protokollokon és adatformátumokon keresztül történik. A klasszikus SOA gyakran SOAP/XML-t és WSDL-t használt, de a modern megvalósítások gyakran REST/JSON-t, gRPC-t vagy aszinkron üzenetküldést alkalmaznak.
- **Felfedezhetőség és szerződés:** a szolgáltatásoknak rendelkezniük kell olyan leírással, amelyből a fogyasztó megtudhatja a műveleteket, a paramétereket, az adatformátumot és a minőségi jellemzőket. Ez a szolgáltatáskatalógus és a szerződésmenedzsment alapja.

- **Autonómia:** a szolgáltatásnak saját életciklusa, adatbázisa és telepítési egysége van. Ez nem jelenti azt, hogy minden szolgáltatásnak külön adatbázissal kell rendelkeznie – a moduláris monolitban is lehet autonóm modul –, de a szolgáltatás nem függhet más szolgáltatás belső tábláitól.

A SOA nem azonos a mikroszolgáltatásokkal. A mikroszolgáltatások a SOA egy későbbi, decentralizáltabb és finomabb szemcsézettségű változatának tekinthetők, de a klasszikus SOA-ban a szolgáltatások gyakran nagyobbak, a csapatok pedig centralizáltabban szerveződnek. A SOA-ban a hangsúly az újrafelhasználáson és a vállalati szintű integráción van, míg a mikroszolgáltatásokban a független telepíthetőségen és a csapat-autonómián.

15.2.2 2. Szolgáltatásszerződés és verziózás

A szolgáltatásszerződés a szolgáltatás és a fogyasztó közötti megállapodás. Nem csupán a műveletek listáját tartalmazza, hanem a kommunikáció minden olyan aspektusát, amely a megbízható együttműködéshez szükséges. Egy jól megírt szolgáltatásszerződés elemei:

- **Funkcionális interfész:** a műveletek neve, bemeneti és kimeneti paraméterei, hibakódjai. Például egy rendeléskezelő szolgáltatásnál: `POST /orders` létrehoz egy rendelést, `GET /orders/{id}` lekérdezi a státuszt, `PATCH /orders/{id}/status` módosítja a státuszt.
- **Kommunikációs protokoll és adatformátum:** például HTTPS/JSON, SOAP/XML, gRPC/protobuf vagy AMQP 1.0. A szerződésben rögzíteni kell a verziót is, például OpenAPI 3.1, WSDL 2.0 vagy AsyncAPI 2.6.
- **Nem funkcionális jellemzők:** válaszidő, rendelkezésre állás, átviteli kapacitás, biztonsági követelmények. Ezek gyakran SLA-osztályokba sorolhatók: például az „arany” osztály havi 99,9%-os rendelkezésre állást és legfeljebb 500 ms 95. percentilis válaszidőt ígér, míg az „ezüst” 99,5%-ot és 800 ms-ot.
- **Hibakezelés és idempotencia:** mi történik hálózati hiba vagy ismételt hívás esetén? A szerződésnek meg kell adnia, hogy a művelet idempotens-e, vagy a fogyasztónak kell deduplikálnia. Az idempotencia különösen fontos aszinkron üzenetküldésnél, ahol az üzenetek többször is kézbesíthetők.
- **Verzió:** a szerződés verziószáma. A szemantikus verziózás (`MAJOR.MINOR.PATCH`) elterjedt: a törő változás új MAJOR verziót igényel, a visszamenőlegesen kompatibilis bővítés MINOR, a hibajavítás PATCH verziót. A fogyasztóknak tudniuk kell, mely verziókat támogatja a szolgáltató, és meddig.

A **contract-first** elv azt mondja, hogy a szerződést a szolgáltató és a fogyasztó közösen, az implementáció előtt rögzíti. Ha a szerződés utólag, a kódból generált leírásból alakul ki, gyakran technikai részletek szivárognak a felszínre, és a fogyasztók törékeny függőséget kapnak. A szerződés nélküli szolgáltatás integrációs adósságot halmoz fel, mert minden változtatás meglepetésként éri a fogyasztókat.

Példa egy szerződésvázlatra az Acme Retail rendeléskezelő szolgáltatásához:

```yaml
openapi: 3.1.0
info:
  title: Rendeléskezelő szolgáltatás
  version: 1.2.0
paths:
  /orders:
    post:
      summary: Új rendelés létrehozása
      requestBody:
        required: true
```

```yaml
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/NewOrder'
      responses:
        '201':
          description: A rendelés létrejött
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Order'
        '409':
          description: Duplikált rendelésazonosító
  /orders/{orderId}:
    get:
      summary: Rendelés lekérdezése
      parameters:
        - name: orderId
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: A rendelés adatai
        '404':
          description: Nincs ilyen rendelés
components:
  schemas:
    NewOrder:
      type: object
      required: [customerId, items]
      properties:
        customerId:
          type: string
        items:
          type: array
          items:
            type: object
            required: [sku, quantity]
            properties:
              sku:
                type: string
              quantity:
                type: integer
                minimum: 1
    Order:
      type: object
      properties:
        orderId:
          type: string
        status:
          type: string
          enum: [CREATED, RESERVED, PAID, SHIPPED, CANCELLED]
```

A példa szemléltető jellegű; éles rendszeren nem kell végrehajtani. A `customerId` és a `sku` helyőrző azonosítók, valós személyes adatot nem tartalmaznak.

A szerződés verziózása akkor válik kritikussá, ha több fogyasztó van. A törő változás bevezetése előtt a szolgáltatónak támogatnia kell a régi verziót egy átmeneti időszakban. Ez párhuzamos fenntartást jelent, ami többletköltséggel jár, de elkerüli a fogyasztók kényszerű, egyidejű migrációját. A verziózási stratégiát ADR-ben kell rögzíteni.

15.2.3 3. Újrafelhasználás: ígéret és valóság

A SOA egyik legnagyobb ígérete az újrafelhasználás: egy jól megtervezett szolgáltatást több fogyasztó is használhat, így nem kell minden alkalmazásban újraimplementálni ugyanazt az üzleti logikát. Az újrafelhasználás azonban nem automatikus következménye a szolgáltatás létrehozásának; rosszul méretezett vagy túl korán központosított szolgáltatások esetén a „reuse" könnyen „shared service" csapdává válik.

Az újrafelhasználás előfeltételei:

- **Stabil, verziózott szerződés:** ha a szerződés gyakran változik, a fogyasztók nem mernek ráépíteni. A stabilitás nem jelent változatlanságot, hanem azt, hogy a változások előre jelezhetők és visszamenőlegesen kompatibilisek, vagy a törő változások ritkák.
- **Megfelelő granularitás:** a túl finom szemcsézettségű szolgáltatások túl sok hálózati hívást igényelnek, a túl durva szemcsézettségűek pedig túl sok felelősséget hordoznak, és nehezen újrafelhasználhatók. Az üzleti képesség-alapú határok segítenek.
- **Szervezeti támogatás:** a szolgáltatás tulajdonosának rendelkeznie kell kapacitással a fogyasztói kérések fogadására, a szerződés karbantartására és az SLA betartására. Ha egy csapat „ingyen" kénytelen szolgáltatást üzemeltetni más csapatok számára, a minőség romlik.
- **Fogyasztói visszacsatolás:** az újrafelhasználást nem a katalógusban szereplő szolgáltatások száma, hanem a tényleges fogyasztók száma és a hívási volumen jelzi. Egy szolgáltatás akkor újrafelhasználható, ha legalább két független fogyasztója van, és a fogyasztók elégedettek.

A **shared service csapda** akkor alakul ki, amikor egy szolgáltatás túl sok fogyasztóval rendelkezik, és minden változtatást több csapattal kell egyeztetni. A szolgáltatás tulajdonosa szűk keresztmetszetté válik, a fejlesztési ciklus lelassul, és a fogyasztók inkább megkerülik a szolgáltatást saját implementációval. Ez az ellentmondás vezetett a mikroszolgáltatások azon elvéhez, hogy az újrafelhasználás másodlagos a független telepíthetőséghez képest.

Az újrafelhasználás mérése történhet a következő mutatókkal:

- **Fogyasztók száma:** hány különböző alkalmazás hívja a szolgáltatást. Cél például legalább három fogyasztó, de legfeljebb hét, mert afölött a koordinációs költség exponenciálisan nő.
- **Szerződésváltozásból eredő incidensek száma:** hány alkalommal okozott a szerződésváltozás termelési hibát az adott hónapban. Cél: nulla.
- **Újrafelhasználási arány:** a szolgáltatás hívásainak hány százaléka érkezik külső fogyasztótól a belső hívásokhoz képest. Ez segít elkülöníteni a valódi újrafelhasználást a látszólagostól.

A mérési ablakot mindig rögzíteni kell, például „az utolsó 30 napban, a termelési környezetben".

15.2.4 4. Enterprise Service Bus (ESB)

Az enterprise service bus olyan integrációs middleware, amely a szolgáltatások közötti kommunikációt központi gerincként kezeli. Az ESB feladatai közé tartozik:

- **Üzenet-útválasztás:** a beérkező üzenetet tartalma vagy fejléce alapján a megfelelő szolgáltatáshoz irányítja.
- **Protokoll- és formátumtranszformáció:** például egy SOAP/XML kérést REST/JSON-ná alakít, vagy egy fájlalapú interfészről érkező adatot üzenetté konvertál.

- **Naplózás és monitorozás:** központi helyen láthatóvá teszi a forgalmat, a hibákat és a válaszidőket.
- **Biztonsági házirendek:** hitelesítés, engedélyezés, titkosítás, aláírás ellenőrzése.
- **Terheléselosztás és hibatűrés:** retry, timeout, circuit breaker, üzenetsorok pufferként.
- **Orchesztráció:** egyes ESB-termékek képesek üzleti folyamatok végrehajtására is, például egy rendelés feldolgozási lánc lépéseinek összehangolására. Ez azonban veszélyes, mert üzleti logikát költöztet a buszba.

Az ESB klasszikus elrendezése egy központi komponenst feltételez, amelyen keresztül minden szolgáltatásközi forgalom áthalad. Ez a „hub-and-spoke" topológia csökkenti a pont-pont kapcsolatok számát, de egyben kritikus komponenssé teszi a buszt.

Az alábbi C4 context szintű diagram az Acme Retail integrációs platformját mutatja. A diagramon a rendszerhatár, a külső rendszerek, a felhasználói szerepkörök és a kapcsolatok protokollja, szinkronitása és hibatűrése is szerepel.

```
flowchart LR
    subgraph AcmeSOA[Saját rendszerhatár: Acme Retail integrációs platform]
        ESB[Enterprise Service Bus
            Felelősség: üzenet-útválasztás, protokoll- és formátumtranszformáció, naplózás
            Interfész: AMQP 1.0, HTTPS]
        Rendeles[Rendeléskezelő szolgáltatás
            Felelősség: rendelés létrehozása, státuszkezelés
            Interfész: SOAP/HTTPS, WSDL 2.0]
        Keszlet[Készletkezelő szolgáltatás
            Felelősség: készletinformáció, foglalás
            Interfész: REST/HTTPS, OpenAPI 3.1]
    end
    Vasarlo[Vásárló] -->|HTTPS, szinkron| ESB
    Partner[Partner rendszer] -->|SFTP, aszinkron, retry| ESB
    ESB -->|SOAP/HTTPS, szinkron, circuit breaker| Rendeles
    ESB -->|REST/HTTPS, szinkron, timeout 2 s| Keszlet
```

A diagramon a rendszerhatárt a `subgraph` jelöli. A kapcsolatokon szerepel a protokoll (HTTPS, SFTP), a szinkronitás (szinkron, aszinkron) és a hibatűrés (retry, circuit breaker, timeout).

Az ESB előnye, hogy a fogyasztóknak nem kell ismerniük a szolgáltatások pontos címét és protokollját; elég a buszhoz csatlakozniuk, a busz pedig elvégzi a szükséges átalakításokat. Ez különösen akkor hasznos, ha a szervezetben sok különböző technológiájú, régi és új rendszer integrálódik. A busz központi helyen biztosítja a megfigyelhetőséget és a biztonsági házirendeket is.

15.2.5 5. Az ESB központi szűk keresztmetszete és tipikus hibák

Az ESB legnagyobb hátránya éppen a központosításból fakad. A busz **központi szűk keresztmetszetté** válhat, ha:

- **Egyetlen példányban fut:** ha a busz nem skálázódik horizontálisan, vagy a konfigurációja nem teszi lehetővé a több aktív példányt, akkor minden forgalom egyetlen csomóponton halad át. Ez teljesítménykorlátot és single point of failure-t jelent.
- **Túl sok transzformációt végez:** ha a busz minden üzenetet bonyolult formátum- és protokollkonverziónak vet alá, a CPU- és memóriafelhasználás nő, a válaszidő romlik. A transzformációk ráadásul gyakran a busz konfigurációjában élnek, ami nehezen tesztelhető és verziózható.

- **Üzleti logikát tartalmaz:** a legnagyobb hiba, ha a buszba orkesztrációs lépések, állapotgépek vagy üzleti szabályok kerülnek. Ekkor az üzleti logika kettéválik: egy része a szolgáltatásokban, más része a buszban van. A változtatásokhoz a busz konfigurációját is módosítani kell, ami gyakran külön csapat felelőssége, és lassú, kockázatos folyamat.
- **Kanonikus adatmodellt kényszerít:** a klasszikus ESB-architektúrák gyakran bevezetnek egy közös, kanonikus adatmodellt, amelyre minden rendszer üzenetét leképezik. Ez csökkenti a pont-pont transzformációk számát, de a kanonikus modell idővel merevvé válik, mert minden új követelmény a központi modell módosítását igényli. A modell tulajdonosa szűk keresztmetszetté válik, és a fejlesztési ciklus lelassul.

A fenti problémák vezettek a **„dumb pipes, smart endpoints"** elv megfogalmazásához, amely a mikroszolgáltatások alapelve. Eszerint a kommunikációs infrastruktúra legyen egyszerű és buta – például egy message broker vagy egy API gateway –, az intelligencia pedig a végpontokban, azaz a szolgáltatásokban legyen. Az ESB ezzel szemben gyakran „smart pipe, dumb endpoints" felállást eredményez, ahol a busz túl okos, a szolgáltatások pedig túl vékonyak.

Tipikus anti-minták:

- **A busz mint varázsláda:** ha senki sem tudja pontosan, mit csinál a busz, mert a transzformációk és útvonalak konfigurációja dokumentálatlan, a rendszer üzemeltethetetlenné válik.
- **Pont-pont kapcsolatok a buszon belül:** ha a buszt egyszerű proxyként használják, és minden szolgáltatáspár saját útvonalat kap, a hub-and-spoke előnye elvész, a konfiguráció áttekinthetetlenné válik.
- **Túl sok újrafelhasználás kényszerítése:** ha a szervezet előírja, hogy minden új funkciót a meglévő szolgáltatásokon keresztül kell megvalósítani, a szolgáltatások túlterhelődnek, és a csapatok nem tudnak haladni.

Az ESB nem eleve rossz. Olyan környezetben, ahol sok régi, heterogén rendszer van, és nincs lehetőség azok gyors cseréjére, az ESB hasznos integrációs hídként szolgálhat. A kulcs az, hogy a busz ne tartalmazzon üzleti logikát, és a kritikus útvonalakon legyen redundáns, horizontálisan skálázható.

15.2.6 6. Enterprise integrációs minták

Az enterprise integráció klasszikus mintái segítenek a szolgáltatások közötti kommunikáció megtervezésében. A leggyakoribbak:

- **Message translator:** az egyik rendszer formátumát egy másik rendszer formátumává alakítja. Például egy XML-alapú rendszer üzenetét JSON-ná konvertálja.
- **Content-based router:** az üzenet tartalma alapján választja ki a célállomást. Például ha a rendelés értéke egy küszöb felett van, akkor manuális jóváhagyási sorra kerül.
- **Splitter és aggregator:** az összetett üzenetet részekre bontja, párhuzamosan feldolgozza, majd az eredményeket összegyűjti. Például egy rendeléshez tartozó tételeket külön-külön ellenőrzi a készletben.
- **Canonical data model:** minden rendszer a közös, kanonikus modellre képez. Előnye, hogy a transzformációk száma lineáris a rendszerek számához képest, nem négyzetes. Hátránya a merevség és a központi szűk keresztmetszet.
- **Orchestration vs. choreography:** az orkesztráció egy központi koordinátor (gyakran az ESB) által vezényelt folyamat, a koreográfia pedig az egyes szolgáltatások közötti eseményalapú együttműködés. Az orkesztráció könnyebben átlátható, de centralizál, a koreográfia decentralizált, de nehezebb nyomon követni a folyamat állapotát.

A modern integrációs architektúrákban az ESB szerepét gyakran átveszik a következő komponensek:

- **API gateway:** a külső fogyasztók számára egységes belépési pontot biztosít, hitelesít, rate-limitel, és protokollt konvertál, de nem tartalmaz üzleti logikát.
- **Message broker:** aszinkron üzenetküldést biztosít, például RabbitMQ vagy Apache Kafka. A broker buta, a fogyasztók okosak.
- **Schema registry:** a szerződések (például Avro, JSON Schema, protobuf) központi nyilvántartása, amely lehetővé teszi a fogyasztók számára a szerződések verziózott elérését anélkül, hogy a busz végezné a transzformációt.
- **Service mesh:** a szolgáltatások közötti kommunikáció infrastrukturális rétege, amely oldalkocsikban (sidecar) valósítja meg a hibatűrést, a megfigyelhetőséget és a biztonságot, de elosztottan, nem központi buszként.

A fenti eszközök együttesen a „smart endpoints, dumb pipes” elvet követik, és elkerülik a központi busz szűk keresztmetszetét.

15.2.7 7. Labor: SOA-szolgáltatáskatalógus vázlata

Ez a labor a fejezetben tanult fogalmakat alkalmazza egy kis, ellenőrizhető dokumentum elkészítésén keresztül. A labor célja, hogy a hallgató elkészítse egy SOA-alapú integrációs réteg szolgáltatáskatalógusának vázlatát, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal. A labor nem igényel éles rendszert; a példa fiktív, és minden adat helyőrző.

15.2.8 Kiinduló állapot

Adott a fiktív *Acme Retail* vállalat, amely a következő alkalmazásokkal rendelkezik:

- **Webáruház:** a vásárlók rendeléseket adnak le, és lekérdezik a rendelés státuszát.
- **CRM rendszer:** az ügyfélszolgálati munkatársak ügyféladatokat és rendelési előzményeket látnak.
- **Számlázó rendszer:** a teljesített rendelésekről számlát állít ki.
- **Raktárkezelő rendszer:** a készletet és a bevételezést kezeli.
- **Partner API:** külső partnerek számára rendelésstátusz-lekérdezést biztosít.

A vállalat jelenleg pont-pont integrációkat használ, ami egyre átláthatatlanabb. A feladat egy SOA-alapú integrációs réteg tervének elkészítése.

15.2.9 Feladat

1. **Szolgáltatások azonosítása:** határozza meg a legalább három szolgáltatást üzleti képességek mentén. Javasolt szolgáltatások: Rendeléskezelő, Készletkezelő, Ügyfélkezelő, Számlakezelő. Mindegyikhez írjon egy rövid felelősségi leírást.
2. **Szolgáltatásszerződés-vázlatok:** legalább három szolgáltatáshoz készítsen egy-egy szerződésvázlatot. A vázlat tartalmazza a fő műveleteket, a protokollt, az adatformátumot, a szinkronitást, a hibatűrést (timeout, retry, circuit breaker) és a verziót. Használhat OpenAPI, AsyncAPI vagy WSDL kivonatot.
3. **C4 context diagram:** készítsen diagrams-as-code formában (Mermaid vagy PlantUML) egy context szintű diagramot, amelyen a rendszerhatár, a külső rendszerek, a felhasználói szerepkörök és a kapcsolatok protokollja, szinkronitása, hibatűrése szerepel.

4. **ADR:** készítsen egy ADR-t arról, hogy a vállalat bevezessen-e ESB-t, vagy inkább API gateway + message broker kombinációt használjon. Az ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
5. **Ellenőrzési szempontok:** határozzon meg legalább öt ellenőrzési szempontot, amelyekkel a szolgáltatáskatalógus minősége értékelhető.

15.2.10 Elvárt eredmény

A hallgató egy verziózott dokumentumot (például Markdown vagy Jupyter notebook) ad át, amely a következőket tartalmazza:

- Cím, dátum, verziószám, szerző.
- A szolgáltatáskatalógus táblázata a következő oszlopokkal: szolgáltatás neve, felelősség, interfész, protokoll, szinkronitás, hibatűrés, fogyasztók, SLA osztály, verzió, tulajdonos, ADR hivatkozás.
- Legalább három szolgáltatásszerződés-vázlat.
- Egy C4 context diagram diagrams-as-code formában.
- Egy ADR az integrációs minta kiválasztásáról.
- Ellenőrzőlista.

A **mért és a várt eredményt külön kell választani.** Ha a hallgató opcionálisan mérést is végez (például egy helyi PoC környezetben leméri az ESB késleltetését), akkor a mérési jegyzőkönyvben rögzíteni kell a környezetet, a konfigurációt, az időpontot és a mérési ablakot, és a mért értéket egyértelműen meg kell különböztetni a várt értéktől. A nem futtatott mérést nem szabad ellenőrzött eredményként bemutatni.

15.2.11 Visszaállítási út

A labor alapvetően dokumentumkészítés, ezért nincs destruktív művelet. Ha a hallgató a példa kedvéért helyi környezetben kipróbál egy ESB-terméket (például Apache Camel, MuleSoft vagy WSO2), azt kizárólag eldobható konténerben tegye, és a munka végén törölje a konténert, állítsa vissza a kiinduló állapotot. Éles rendszeren a példákat nem szabad végrehajtani.

15.2.12 Ellenőrzési szempontok a labor értékeléséhez

- A szolgáltatások határai üzleti képességek mentén húzódnak-e, nem technikai rétegek mentén?
- Minden szolgáltatáshoz tartozik-e felelősség és interfész, nem csak név?
- A kapcsolatokon szerepel-e a protokoll, a szinkronitás és a hibatűrés?
- A szerződések verziózottak-e, és a verziószám szerepel-e?
- Az ADR tartalmazza-e a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot?
- A diagramon a rendszerhatár jelölve van-e, és a külső rendszerek, szerepkörök megjelennek-e?
- A dokumentum nem tartalmaz-e valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot? Minden helyőrző-e?

15.2.13 8. Döntési gyakorlat

Az Acme Retail jelenlegi ESB-je egyetlen példányban fut, és a rendelési szolgáltatás válaszideje az ESB-n keresztül az elmúlt 30 napban a 95. percentilisnél 850 ms volt, a SLO pedig 500 ms. A busz emellett számos üzleti szabályt is tartalmaz, például a rendelésérték alapján manuális jóváhagyásra irányít. A csapatok panaszkodnak, hogy minden változtatáshoz a busz konfigurációját is módosítani kell, ami lassú.

Feladat: írjon egy rövid döntési elemzést (maximum egy oldal), amelyben megvizsgálja, hogy a vállalat tartsa meg az ESB-t, vagy váltsa le API gateway + message broker kombinációra. Milyen trade-offok merülnek fel a rendelkezésre állás, a fejlesztési sebesség, az üzemeltetési komplexitás és a szervezeti koordináció szempontjából? A válaszban ne csak a technológiát értékelje, hanem a szervezeti feltételeket is.

15.2.14 9. Ellenőrzőlista SOA- és ESB-alapú integrációhoz

- Minden szolgáltatás üzleti képességet valósít meg, és van verziózott szerződése.
- A szerződés tartalmazza a funkcionális interfészt, a protokollt, az adatformátumot, a nem funkcionális jellemzőket, a hibakezelést és az idempotenciát.
- A szolgáltatások közötti kommunikáció szinkron vagy aszinkron jellege indokolt és dokumentált.
- Az ESB nem tartalmaz üzleti logikát, csak technikai integrációs feladatokat (routing, transzformáció, naplózás, biztonság).
- Az ESB magas rendelkezésre állással rendelkezik, redundáns példányokkal, és van katasztrófa-helyreállítási terve.
- A kanonikus adatmodell nem vált merev szűk keresztmetszetté; ha igen, megfontolandó a consumer-driven contracts vagy schema registry bevezetése.
- A szolgáltatáskatalógus naprakész, verziózott és minden érintett számára hozzáférhető.
- Az újrafelhasználási döntések fogyasztói visszajelzéseken és méréseken alapulnak, nem kényszeren.
- A busz konfigurációja verziózott, tesztelt és visszaállítható.
- A dokumentáció nem tartalmaz valós hitelesítő adatot, belső URL-t vagy ügyfélnevet.

15.2.15 10. Önellenőrző kérdés

Miért válik az ESB gyakran központi szűk keresztmetszetté, és milyen két tervezési változtatással csökkenthető ez a kockázat anélkül, hogy az integrációs logika a végpontokba szivárogna?

15.2.16 11. Összegzés

Ez az egység bemutatta a szolgáltatásorientált architektúra alapfogalmait, a szolgáltatásszerződés és az újrafelhasználás szerepét, valamint az enterprise service bus előnyeit és hátrányait. Láttuk, hogy a központi busz csökkenti a pont-pont kapcsolatok számát, de központi szűk keresztmetszetté és üzleti logika rejtekhelyévé válhat. A modern integrációs architektúrák ezért a „dumb pipes, smart endpoints” elvet követik, és az ESB szerepét API gateway, message broker, schema registry és service mesh veszi át. A laborban elkészített szolgáltatáskatalógus-vázlat és ADR megalapozza a következő egységet, amely a mikroszolgáltatások és a domain-architektúra viszonyát tárgyalja részletesen.

15.3 Microservices és domain-architektúra

A domainhatárok és a szolgáltatáshatárok összehangolása akkor válik éles kérdéssé, amikor egy monolitikus alkalmazást vagy egy rosszul szabdalt SOA-rendszert szeretnénk evolúcióképes, autonóm egységekre bontani. A Domain-Driven Design (DDD) eszköztára – elsősorban a bounded context, az aggregate és a domain event fogalmai – segít megtalálni azokat a határokat, amelyek mentén a szolgáltatások szétválaszthatók. A mikroszolgáltatás-architektúra pedig akkor működik jól, ha ezek a szervezeti és üzleti határok egybeesnek a futtatható egységek határaival. Ez a fejezet ezt az összehangolást járja körül: hogyan lesz a domainmodellből szolgáltatástérkép, miért kritikus az adattulajdon, és hogyan kapcsolódik mindez a szervezeti felépítéshez.

15.3.1 Domainhatárok és szolgáltatáshatárok

A DDD-ben a **bounded context** (korlátos kontextus) az a nyelvi és modellbeli határ, amelyen belül egy adott fogalomkészlet következetesen használható. Például egy e-kereskedelmi rendszerben a „Rendelés” fogalom mást jelent a Rendelés kontextusban (az ügyfél szándéka, tételek, állapot), mint a Számlázás kontextusban (fizetendő összeg, számla állapota). A kontextusok közötti kapcsolatot kontextustérkép írja le, amely meghatározza az integrációs mintákat (shared kernel, customer/supplier, anticorruption layer stb.).

A **szolgáltatás** ezzel szemben egy futtatható, telepíthető, skálázható egység, amelynek saját életciklusa, interfésze és adattárolója van. A mikroszolgáltatás-architektúrában egy szolgáltatás ideális esetben egy bounded contextet valósít meg – vagy annak egy jól körülhatárolt részét. A két fogalom közötti kapcsolat azonban nem kötelezően egy-az-egyhez: egy bounded context lehet egyetlen szolgáltatás, de állhat több, együtt telepített modulból is (moduláris monolit), vagy egy nagyobb kontextus több szolgáltatásra is bomolhat, ha a kontextuson belül eltérő skálázási vagy telepítési igények jelentkeznek. A lényeg, hogy **a szolgáltatáshatár soha ne keresztezzen bounded context határt**, mert akkor a szolgáltatás kénytelen lesz egyszerre több, egymástól eltérő nyelvet és modellt kezelni, ami a kohézió csökkenéséhez és a csatolás növekedéséhez vezet.

Tekintsünk egy példát. Egy kiskereskedelmi rendszerben az alábbi bounded contexteket azonosítottuk:

- **Katalógus**: termékek, árak, leírások; a vásárlói keresést és böngészést szolgálja ki.
- **Kosár**: az ügyfél aktuális, még nem véglegesített tételei.
- **Rendelés**: a leadott rendelés életciklusa, tételek, állapotok (létrehozva, visszaigazolva, teljesítve).
- **Számlázás**: számlák kiállítása, fizetési állapotok, pénzügyi tranzakciók.
- **Készlet**: aktuális készletszintek, foglalások, utánpótlás.
- **Szállítás**: csomagok, fuvarozók, szállítási címek, nyomon követés.

Ezek a kontextusok világos nyelvi és felelősségi határokat jelölnek ki. Ha most szolgáltatástérképet készítünk, a legegyszerűbb és leggyakoribb döntés az, hogy minden bounded contexthez egy-egy szolgáltatást rendelünk: `catalog-service`, `cart-service`, `order-service`, `billing-service`, `inventory-service`, `shipping-service`. Ekkor a szolgáltatáshatárok pontosan egybeesnek a domainhatárokkal, és minden szolgáltatásnak saját, jól definiált felelőssége van.

Azonban előfordulhat, hogy egy bounded contexten belül több, eltérő életciklusú vagy terhelési profilú részegység van. Például a Rendelés kontextuson belül a rendelésfelvétel nagy olvasási terhelést kap, míg a rendelésfeldolgozás inkább írási és állapotgép-logikát futtat. Ilyenkor a kontextust két szolgáltatásra bonthatjuk (`order-capture`, `order-processing`), de a kettőnek továbbra is ugyanazt a domainmodellt kell használnia, és a köztük lévő kommunikációt a

kontextuson belüli szerződések szabályozzák. Ez nem sérti a szabályt, mert a szolgáltatáshatárok a kontextuson belül húzódnak, nem pedig keresztül rajta.

A leggyakoribb hiba ezzel szemben az, amikor a szolgáltatásokat technikai rétegek vagy entitások mentén vágják fel – például külön `order-db-service`, `order-logic-service`, `order-api-service` –, mert ezek nem domainhatárok, hanem technikai komponensek. Az ilyen felosztás rendkívül magas csatolást eredményez, hiszen minden művelet átmegy több szolgáltatáson, és a tranzakciós integritás is megbomlik. Hasonlóan rossz, ha egyetlen `core-service`-be zsúfolják a Katalógus, Kosár és Rendelés logikát, mert akkor a domainhatárok nem jelennek meg a futtatható egységekben, és a csapatok sem tudnak önállóan dolgozni.

Az összehangolás ellenőrzésére praktikus eszköz a **kontextus–szolgáltatás mátrix**. Ennek sorai a bounded contexteket, oszlopai a szolgáltatásokat tartalmazzák, és minden cellában jelöljük, hogy az adott szolgáltatás milyen mértékben valósítja meg az adott kontextust (felelős-e érte, vagy csak olvasási célból használja). Ha egy szolgáltatás egynél több kontextusért is elsődlegesen felelős, az gyanús jel; ha egy kontextus több szolgáltatás között oszlik meg anélkül, hogy világos belső határok lennének, az szintén további vizsgálatot igényel.

15.3.2 Autonóm szolgáltatás és adattulajdon

A mikroszolgáltatások egyik alapelve az **autonómia**: minden szolgáltatás önállóan fejleszthető, telepíthető, skálázható és – ami a legfontosabb – **a saját adataiért kizárólagos felelősséget visel**. Az adattulajdon azt jelenti, hogy egy szolgáltatás saját perzisztenciával rendelkezik, amelyhez más szolgáltatások nem férhetnek hozzá közvetlenül, csak a szolgáltatás által nyújtott API-n vagy eseményeken keresztül.

Az adattulajdon megsértésének klasszikus példája a **megosztott adatbázis** (integration database). Ha például a `billing-service` közvetlenül olvassa a `order-service` adatbázisában lévő `orders` táblát, akkor a két szolgáltatás erősen csatolttá válik: a rendelési séma bármilyen változtatása kihat a számlázó szolgáltatásra, és a telepítések sem függetlenek többé. Ráadásul a tranzakciós határok elmosódnak, mert a számlázó szolgáltatás olyan adatkonzisztencia-feltételeket is kénytelen kezelni, amelyek a másik szolgáltatás belügyei. A helyes minta: a `billing-service` a `order-service` által publikált API-n vagy doméneseményeken keresztül kapja meg a számlázáshoz szükséges adatokat, és azokat saját, olvasásra optimalizált formában tárolja (például egy saját `invoice_order_summary` táblában).

Az adattulajdon nem jelenti azt, hogy minden szolgáltatásnak teljesen izolált adathalmaza van, és soha nem tárolhat más kontextusból származó adatot. Éppen ellenkezőleg: a gyakorlatban a szolgáltatások gyakran **adatreplikációt** alkalmaznak, amikor egy másik szolgáltatás eseményeit felhasználva helyi, olvasási célú másolatot tartanak fenn. Ez a **event-carried state transfer** minta. Például a `shipping-service` feliratkozik a `order-service` „Rendelés visszaigazolva” eseményére, és saját adatbázisában eltárolja a csomaghoz szükséges alapadatokat (rendelésazonosító, szállítási cím, tételek). Így a szállítási szolgáltatás akkor is tud dolgozni, ha a rendelési szolgáltatás átmenetileg nem elérhető, és nem kell szinkron API-hívást indítania minden egyes csomag létrehozásakor. Ez a replikáció azonban nem sérti az adattulajdont, mert a replikált adat a `shipping-service` saját, lokális másolata, amelyért ő felel, és a forrásrendszer nem fér hozzá.

Az autonómia másik oldala a **tranzakciós határok** kezelése. Egy szolgáltatáson belül a helyi tranzakciók egyszerűen biztosíthatók. Ha azonban egy üzleti folyamat több szolgáltatást érint – például a rendelés leadása érinti a Rendelés, a Készlet és a Számlázás kontextusokat –, akkor nem használhatunk elosztott tranzakciót (kétfázisú commit), mert az ellentétes az autonómiával és a rendelkezésre állással. Helyette a **saga** mintát vagy az **outbox** mintát alkalmazzuk, amelyeket

a későbbi fejezet részletez. Itt elég annyit rögzíteni, hogy a domainhatárok mentén felvágott szolgáltatások esetén a tranzakciók határai egybeesnek a kontextushatárokkal, és a kontextusok közötti konzisztencia végső soron (eventual consistency) biztosított.

Az autonóm szolgáltatások kialakításakor a következő kérdéseket érdemes feltenni:

- **Adatbázis:** Van-e a szolgáltatásnak saját, mások által nem elérhető adatbázisa vagy sémája?
- **API:** Minden más szolgáltatás kizárólag a publikált interfészen keresztül éri-e el az adatait?
- **Események:** A szolgáltatás által kibocsátott doménesemények elegendőek-e a többi kontextus számára, vagy szükség van szinkron hívásokra?
- **Skálázás:** A szolgáltatás a terhelésétől függően önállóan skálázható-e, anélkül, hogy más szolgáltatásokat is skálázni kellene?
- **Telepítés:** Egy szolgáltatás új verziójának telepítése nem igényli-e más szolgáltatások egyidejű telepítését?

Ha bármelyik kérdésre nemleges a válasz, akkor a szolgáltatás nem teljesen autonóm, és a domainhatárok valószínűleg nem esnek egybe a szolgáltatáshatárokkal.

Egy gyakori ellenőrzési gyakorlat: rajzoljuk fel az adatbázis-hozzáféréseket irányított gráfként, ahol a csúcsok a szolgáltatások, az élek pedig a közvetlen adatbázis-olvasást/írást jelentik. Ha ez a gráf nem fenyő (tehát több szolgáltatás is ugyanahhoz az adatbázishoz kapcsolódik), akkor az adattulajdon sérül, és refaktorálásra van szükség.

15.3.3 Domain-architektúra és szervezet

A domainhatárok és a szolgáltatáshatárok összehangolása nem pusztán technikai döntés; szorosan összefügg a szervezeti felépítéssel. Conway törvénye kimondja, hogy a rendszerek architektúrája a szervezet kommunikációs struktúráját tükrözi. Ha a csapatok funkcionális silókba szerveződnek (például frontend, backend, adatbázis), akkor a rendszer is rétegekre fog tagolódni, nem pedig domainekre. Ha viszont a csapatok **stream-aligned** módon, azaz egy-egy üzleti képesség köré szerveződnek, akkor a szolgáltatások is természetesen domainhatárok mentén jönnek létre.

A **Team Topologies** modell négy csapattípust különböztet meg: stream-aligned team, enabling team, platform team és complicated subsystem team. A stream-aligned team egy teljes üzleti folyamatért vagy domainért felel, és ez a felelősség leképezhető egy vagy több bounded contextre. A platform team azokat az alapvető képességeket biztosítja (CI/CD, megfigyelhetőség, konténerplatform), amelyekre a domain-csapatok építenek. Az enabling team rövid távú szakértői támogatást nyújt, a complicated subsystem team pedig olyan komponenseket kezel, amelyek speciális tudást igényelnek (például ajánlómotor, pénzügyi kockázatkezelő).

Amikor egy vállalat mikroszolgáltatásokra tér át, gyakran alkalmazzák a **fordított Conway manővert**: először a szervezetet alakítják át a kívánt domainhatárok mentén, és csak utána bontják fel a rendszert. Például ha a cél az, hogy a Rendelés, a Számlázás és a Készlet önálló szolgáltatások legyenek, akkor létrehoznak egy Rendelés csapatot, egy Számlázás csapatot és egy Készlet csapatot, mindegyiknek saját terméktulajdonossal és mérnöki kapacitással. Így a szervezeti határok kikényszerítik a technikai határokat.

A csapatok közötti interfészek megegyeznek a szolgáltatások közötti interfészekkel. Ha két bounded context között customer/supplier kapcsolat van, akkor a szállító (upstream) csapat szolgáltatása publikál egy API-t vagy eseményeket, és a fogyasztó (downstream) csapat ezekre épít. A kommunikációs terhelés csökkentése érdekében a csapatoknak törekedniük kell arra, hogy minél kevesebb szinkron függőség legyen közöttük. Az aszinkron események lazább csatolást tesznek lehetővé, de a szerződéseket így is explicit módon kell kezelni.

Egy fontos ellenőrzési szempont: ha két csapat folyamatosan egyeztetni kényszerül egy közös szolgáltatás vagy adatbázis miatt, akkor a domainhatárok valószínűleg rosszul lettek meghúzva. A jó határ az, ahol a csapatok közötti kommunikáció minimális, és a szolgáltatások közötti interfész stabil, jól dokumentált.

15.3.4 Gyakorlati példa: e-kereskedelmi rendszer

Vegyünk egy konkrét, leegyszerűsített e-kereskedelmi rendszert. Az üzleti folyamat a következő: a vásárló böngészi a termékeket, kosárba helyezi őket, leadja a rendelést, fizet, majd a rendszer csomagot állít össze és kiszállítja. A domain elemzése során a következő bounded contexteket és hozzájuk tartozó szolgáltatásokat azonosítottuk:

Bounded context	Felelősség	Szolgáltatás	Adattulajdon	Publikus interfész
Katalógus	Termékek, árak, leírások, keresés	`catalog-service`	Saját termékadatbázis	REST API, termékváltozás-események
Kosár	Vásárlói kosár, tételek, mennyiségek	`cart-service`	Saját kosár-adatbázis	REST API, „Kosár módosult” esemény
Rendelés	Rendelés életciklusa, tételek, állapotok	`order-service`	Saját rendelés-adatbázis	REST API, „Rendelés leadva”, „Rendelés visszaigazolva” események
Számlázás	Számlák, fizetési állapotok, tranzakciók	`billing-service`	Saját számlázási adatbázis	REST API, „Számla kiállítva”, „Fizetés beérkezett” események
Készlet	Készletszintek, foglalások, utánpótlás	`inventory-service`	Saját készlet-adatbázis	REST API, „Készletfoglalás megerősítve” esemény
Szállítás	Csomagok, fuvarozók, nyomon követés	`shipping-service`	Saját szállítási adatbázis	REST API, „Csomag feladva” esemény

Ebben a felosztásban minden szolgáltatás egy bounded contextet valósít meg, és saját adattárral rendelkezik. A szolgáltatások közötti kommunikáció kétféle lehet:

- **Szinkron API-hívás**, amikor azonnali válaszra van szükség. Például a `cart-service` a `catalog-service`-től kéri le a termék aktuális árát, mielőtt a kosárba helyezi.
- **Aszinkron esemény**, amikor a folyamat nem igényel azonnali választ. Például amikor a `order-service` létrehoz egy rendelést, kibocsát egy „Rendelés leadva” eseményt, amelyre a `billing-service` és a `inventory-service` is feliratkozik.

A két minta közötti választás a konzisztenciaigénytől és a rendelkezésre állási követelményektől függ. Ha a rendelés leadásakor azonnal tudni kell, hogy van-e elegendő készlet, akkor a `order-service` szinkron módon megkérdezi a `inventory-service`-t, de ez növeli a csatolást és a meghibásodási felületet. Ha a készletfoglalás történhet aszinkron módon is (például a rendelést elfogadjuk, majd a készlet szolgáltatás visszaigazol), akkor lazább a csatolás, de a felhasználó esetleg később értesül a készlethiányról. A döntést ADR-ben kell rögzíteni.

Az adattulajdon szemléltetésére: a `billing-service` nem olvassa a `order-service` rendelési tábláját. Helyette feliratkozik a „Rendelés leadva” eseményre, és az eseményben kapott adatokból létrehozza a saját számláját. Ha a rendelés később módosul, a `order-service` újabb eseményt bocsát ki, és a `billing-service` frissíti a saját másolatát. Így a számlázó szolgáltatás teljesen autonóm marad, és a rendelési szolgáltatás sem függ a számlázó belső adatmodelljétől.

Egy tipikus hibás kialakítás ugyanerre a rendszerre: egyetlen `order-management-service` létezik, amely a rendelés, a számlázás és a szállítás logikáját is tartalmazza, és mindhárom adatbázist közvetlenül eléri. Ebben az esetben a domainhatárok nem jelennek meg a szolgáltatáshatárokban, a csapatok nem tudnak párhuzamosan dolgozni, és a skálázás is csak az egész monolitra vonatkozhat. Ez a „mikrolit” anti-minta, amelyet a 3. részben már tárgyaltunk.

15.3.5 Labor: domain- és szolgáltatástérkép összevetése

A labor célja, hogy a tanultakat egy kisméretű, ellenőrizhető példán alkalmazza. A feladat egy fiktív rendszer domain- és szolgáltatástérképének elkészítése, majd a kettő összevetése.

Kiinduló állapot. Adott egy szöveges rendszerleírás, például:

> Egy könyvtári kölcsönző rendszerben a következő üzleti folyamatok vannak: olvasók regisztrálnak, könyveket keresnek a katalógusban, kölcsönöznek, hosszabbítanak, visszahoznak, és előjegyzést adhatnak le. A rendszer kezeli a késedelmi díjakat, és értesítést küld a lejárat előtt. A könyvtárosok kezelik a könyvek leltárát és a selejtezést.

A hallgatónak ebből a leírásból kell bounded contexteket azonosítania, majd meghatároznia, hogy milyen szolgáltatásokra bontaná a rendszert.

Feladat. Készítsen két diagramot (például Mermaid vagy PlantUML C4 component szinten):

1. **Domain-térkép:** a bounded contexteket és a köztük lévő kapcsolatokat (például customer/supplier, shared kernel) ábrázolja.
2. **Szolgáltatástérkép:** a futtatható szolgáltatásokat, azok adattulajdonát és a köztük lévő kommunikációs csatornákat (API vagy esemény).

Majd készítsen egy összehasonlító táblázatot, amelyben minden bounded contexthez hozzárendeli a megvalósító szolgáltatást, és jelzi az esetleges eltéréseket. Végül írjon egy rövid ADR-t, amely rögzíti a szolgáltatáshatárok megválasztásának indoklását, az elvetett alternatívákat és a következményeket.

Elvárt eredmény. Egy verziózott dokumentum (például Markdown fájl), amely tartalmazza a két diagramot, az összehasonlító táblázatot és az ADR-t. A diagramok a kódbázissal együtt verziózhatók, így a labor során Git repository-ban kell dolgozni.

Ellenőrzési szempontok.

- Minden azonosított bounded context szerepel a domain-térképen, és a kontextusok közötti kapcsolatok meg vannak nevezve.

- A szolgáltatástérképen minden szolgáltatásnak van saját adattára, és egyik szolgáltatás sem éri el másik szolgáltatás adatbázisát közvetlenül.
- A szolgáltatások közötti kommunikáció kizárólag API-n vagy eseményen keresztül történik; a szinkronitás és a protokoll jelölve van.
- Az összehasonlító táblázatban nincs olyan szolgáltatás, amely két vagy több bounded contextért felelős anélkül, hogy a kontextuson belüli belső határokat megadná.
- Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket, valamint az állapotot (például „Elfogadva”).

Visszaállítási út. Mivel a labor csak dokumentumokat hoz létre, nincs destruktív művelet. A repository állapota bármikor visszaállítható a korábbi commitokra. Ha a hallgató valamilyen kódot is ír (például egy egyszerű API-vázlatot), azt izolált környezetben kell futtatni, és a labor végén a konténereket le kell állítani.

15.3.6 Ellenőrzőlista és önellenőrző kérdések

Ellenőrzőlista a domain- és szolgáltatáshatárok összehangolásához:

- ☐ Minden bounded context explicit módon azonosítva van, és a kontextusok közötti kapcsolatok dokumentálva vannak.
- ☐ Minden szolgáltatás pontosan egy bounded contextet valósít meg, vagy ha egy kontextus több szolgáltatásból áll, a belső határok világosak.
- ☐ Minden szolgáltatásnak saját adattára van, és más szolgáltatás nem fér hozzá közvetlenül.
- ☐ A szolgáltatások közötti kommunikáció API-n vagy eseményen keresztül történik; a szinkronitás és a protokoll dokumentált.
- ☐ A tranzakciós határok egybeesnek a szolgáltatáshatárokkal; a kontextusok közötti konzisztencia végső soron biztosított.
- ☐ A csapatok szervezeti felépítése tükrözi a domainhatárokat, és a csapatok közötti kommunikáció minimális.
- ☐ Az ADR-ek rögzítik a szolgáltatáshatárokra vonatkozó döntéseket és azok indoklását.

Önellenőrző kérdés: Miért rossz, ha egy mikroszolgáltatás közvetlenül olvassa egy másik szolgáltatás adatbázisát, és milyen alternatívák állnak rendelkezésre? Válaszában térjen ki az adattulajdon, a csatolás, a tranzakciós határok és a skálázhatóság szempontjaira.

Összegzés. A domainhatárok és a szolgáltatáshatárok összehangolása a sikeres mikroszolgáltatás-architektúra alapja. A DDD bounded context fogalma segít megtalálni azokat a nyelvi és modellbeli határokat, amelyek mentén a szolgáltatások autonóm egységekké válnak. Az adattulajdon biztosítja, hogy minden szolgáltatás önállóan fejlődhessen, a szervezeti felépítés pedig – Conway törvényének megfelelően – vagy támogatja, vagy akadályozza ezt az összehangolást. A következő fejezet (Data mesh és domain-oriented adatarchitektúra) ezekre az alapokra építve mutatja be a Data Mesh és a domain-orientált adatarchitektúra elveit, ahol az adattulajdon fogalma még tágabb, szervezeti szintű dimenziót kap.

15.4 Data mesh és domain-oriented adatarchitektúra

15.4.1 Adattermékek, domainek és az önkiszolgáló platform

A hagyományos, központosított adatarchitektúrák – adattárházak, adattavak, központi adatplatform csapatok – sok szervezetben elérik a méretezhetőségük határát. A probléma ritkán technológiai eredetű: a tudás, az üzleti kontextus és a döntési képesség a domain csapatoknál

van, míg az adatokért való felelősség egy központi csapatnál koncentrálódik. Amikor minden új adatforrást a központi adatplatformra kell bekötni, a központi csapat válik szűk keresztmetszetté; a domainek elveszítik a kontrollt a saját adataik felett, a fogyasztók pedig olyan adatkészleteket kapnak, amelyek szemantikája számukra homályos. Ezt a feszültséget oldja fel a data mesh, amely nem egyszerűen egy technológiai minta, hanem szervezeti és architekturális paradigma: az adatok felelősségét és szolgáltatásait a domainekhez delegálja, közös platformmal és federált governance-szel.

A data mesh négy alapelve:

1. **Domain-orientált adattulajdonlás** – az adatokért az a domain felel, ahol azok keletkeznek és ahol az üzleti jelentésüket ismerik.
2. **Adat mint termék** – az adatokat fogyasztható, szerződéssel ellátott adattermékekként publikálják, nem nyers táblákként vagy fájlként.
3. **Önkiszolgáló adatplatform** – a domain csapatok egy belső platformon keresztül, központi adat-platform csapat nélkül tudnak adatterméket létrehozni, üzemeltetni és megosztani.
4. **Federált számítási governance** – a globális szabályokat gépi úton, a platformba ágyazva tartatják be, miközben a döntések egy része a domaineké marad.

Ez a fejezet e négy elv közül az első hármat tárgyalja részletesen – az adatterméket, az adatdomaint és az önkiszolgáló platformot –, majd a federált governance-t az adatminőséggel együtt mutatja be. A cél nem a data mesh mint „best practice” kikiáltása, hanem annak megértése, hogy milyen feltételek mellett működik, milyen költségekkel jár, és mikor nem érdemes bevezetni.

15.4.2 1. Mi az adattermék?

Az adattermék a data mesh alapegysége: egy olyan, a fogyasztók számára elérhető adatcsomag, amelyet egy domain csapat birtokol, üzemeltet, dokumentál és minőségi mutatókkal lát el. A hangsúly a „termék” szón van: az adattermék nem egy adatbázistábla, egy Kafka-topic vagy egy fájl, hanem egy csomag, amely magában foglalja az adatot, annak elérési módját, a leíró dokumentációt, a séma- és szemantikai szerződést, a hozzáférés-kezelést, a frissítési garanciákat és a minőségi mutatókat. Ahogyan egy szoftvertermeknél a felhasználói élmény, a dokumentáció és a támogatás is a termék része, az adatterméknél is az adat csupán az egyik összetevő.

Tipikus jellemzők:

- **Felelős tulajdonos:** minden adatterméknek van egy azonosítható domain csapata, amely felel a minőségért és az életciklusért.
- **Felfedezhetőség:** az adatterméket regisztrálják egy adatkatalógusban, leírással, címkékkel és példákkal.
- **Interoperábilis interfész:** az adattermékhez szabványos módon lehet hozzáférni – SQL, REST, streaming, fájl –, és az interfészhez szerződés tartozik.
- **Öndokumentáló:** a séma, a szemantika, az üzleti jelentés és a használati példák a termék része.
- **Minőségi mutatók:** az adattermék SLO-kat definiál a frissességre, a teljességre, a pontosságra és a rendelkezésre állásra, és ezeket méri is.
- **Verziózott és evolúciós:** az adattermék interfésze verziózott, a változásokat szerződésmódosítással kezelik.
- **Biztonságos és szabályozott:** a hozzáférést a platform és a domain közösen szabályozza, a személyes és érzékeny adatokra külön szabályok vonatkoznak.

Egy adattermék tehát egy **felelősségi határ** is: a fogyasztók nem a domain belső tábláit, nyers eseményfolyamát vagy adatbázisát érik el közvetlenül, hanem a domain által publikált, stabil szerződésű terméket. Ez a határ csökkenti a rejtett kapcsolódást, és lehetővé teszi a domainek számára, hogy a belső implementációt szabadon változtassák, amíg a szerződést betartják.

15.4.3 1.1. Adatszerződés

Az **adatszerződés** (data contract) az adattermék interfészének formális leírása. Nem csupán séma (oszlopok, típusok), hanem a termék teljes fogyasztói ígéretét rögzíti:

- **Séma:** mezők, típusok, kulcsok, null-képesek, formátumok.
- **Szemantika:** az egyes mezők üzleti jelentése, például hogy az `order_total` áfa nélkül értendő-e.
- **Frissítési gyakoriság:** milyen gyakran frissül az adat, mennyi a megengedett maximális késés.
- **Minőségi SLO-k:** például a rekordok teljessége legalább 99,5% 24 órás ablakban mérve, a frissesség p95 $<$ 15 perc.
- **Hibakezelés:** mi történik, ha az adat késik vagy hibás; milyen értesítést kap a fogyasztó.
- **Verziózási és változási szabályok:** hogyan lehet a szerződést módosítani, milyen átmeneti időszakok vannak.
- **Adathasználati korlátok:** adatvédelmi, megfelelési, licenc- vagy üzleti korlátozások.

Az adatszerződés szerepe hasonló a REST API OpenAPI szerződéséhez vagy a gRPC proto definíciójához: a fogyasztó anélkül tud integrálni, hogy ismerné a belső implementációt. A gyakorlatban a szerződést érdemes géppel olvasható formában (például JSON Schema, Avro, protobuf, OpenAPI) verziózni és a platformmal validáltatni. Egy adattermékhez több interfész is tartozhat (például batch export és streaming), ilyenkor mindegyikhez külön szerződés tartozik.

Példa egy adattermék szerződésének kivonatára:

```
dataProduct: orders-v2
owner: order-domain
description: Jóváhagyott rendelések napi szintű, fogyasztói nézete.
interfaces:
  - type: sql-view
    endpoint: data-platform.example.internal/catalog/orders-v2
    schema: orders_v2.avsc
    freshnessSLO: p95 < 15 perc, 24h ablak
    completenessSLO: 99,5%
    semantics:
      order_total: bruttó érték, HUF, áfa nélkül
      status: 'CREATED | PAID | SHIPPED | CANCELLED'
    retention: 36 hónap
  - type: streaming
    topic: orders.v2.events
    schema: order_event_v2.avsc
    freshnessSLO: p99 < 1 perc, 1h ablak
versioning:
  backwardCompatibility: 6 hónap
  deprecationNotice: 90 nap
access:
  pii: true
  legalBasis: GDPR 6.1.b
  masking: customer_email -> sha256
```

A szerződés tehát nem dokumentációs formalitás, hanem a termék része, amelyet a platform automatikusan ellenőriz, és amelynek megsértése riasztást vált ki.

15.4.4 1.2. Adattermék a gyakorlatban

Tegyük fel, hogy egy e-kereskedelmi cég **Értékesítési domain**je birtokolja a rendeléseket. A domain csapat a saját tranzakciós adatbázisában tárolja a rendeléseket, de ezt nem publikálja közvetlenül. Helyette létrehoz egy `orders` nevű adatterméket, amely egy napi batch feldolgozással készített, denormalizált, fogyasztói táblát tölt fel. A szerződés rögzíti a sémát, a frissességet (naponta egyszer, legkésőbb 02:00 UTC-ig), a teljességet (minden jóváhagyott rendelés szerepel), és a fogyasztói hozzáférést. A domain csapat üzemelteti az adatterméket, figyeli a minőségi mutatókat, és ha egy fogyasztó hibát talál, a domainhez fordul.

Ez a modell éles szakítás a „központi adattárház" gyakorlatával, ahol a domain átadja a nyers adatot egy központi csapatnak, és a továbbiakban nem felel érte. A data mesh-ben a domain felel az adatért annak teljes életciklusában, cserébe viszont megkapja a platformot és az autonómiát, hogy a saját adatait a leghatékonyabban szolgálja ki.

15.4.5 2. Adatdomain és domain-orientált felelősség

A data mesh második pillére a **domain-orientált adattulajdonlás**: az adatokért az a szervezeti egység felel, amely az adatot létrehozó üzleti folyamatot birtokolja. Ez a gondolat szorosan kapcsolódik a Domain-Driven Design (DDD) bounded context fogalmához: a domainek üzleti képességek, folyamatok és nyelvek szerint határolódnak el. A data mesh a DDD elveit kiterjeszti az adatarchitektúrára: minden domain a saját bounded contextjén belül definiálja és publikálja az adattermékeit. A domainek közötti adatcsere nem közvetlen adatbázis-hozzáférésen, hanem az adattermékeken keresztül történik.

A domain csapat felelősségei:

- **Adattermékek definiálása és üzemeltetése:** a domain dönti el, mely adatokat tesz elérhetővé, milyen formában és milyen szemantikával.
- **Szerződések betartása:** a domain biztosítja a szerződésben vállalt frissességet, teljességet és elérhetőséget.
- **Adatminőség:** a domain méri a saját adattermékeinek minőségét, és javítja a hibákat.
- **Belső implementáció:** a domain szabadon választja meg a belső tárolást, a feldolgozási technológiát, a séma evolúciójának módját, amíg a külső szerződést betartja.
- **Domain-specifikus szabályok:** a domain alkalmazza a saját üzleti szabályait, például a rendelés-státuszok validálását, a személyes adatok anonimizálását.

Fontos, hogy a domain felelőssége nem jelenti a központi szabályok teljes hiányát. A federált governance keretet ad: bizonyos szabályok globálisak (például a személyes adatok kezelése, az interoperabilitási szabványok, a biztonsági követelmények), mások domain-szintűek maradnak (például az adattermék belső sémája, a frissítési ütemezés). A domain tehát autonóm, de nem szuverén: a platform és a governance rendszer korlátok között tartja.

15.4.6 2.1. Példa: domainek az e-kereskedelmi cégben

Egy képzeletbeli e-kereskedelmi szervezetben a következő domainek lehetnek:

- **Ügyfél domain:** ügyféladatok, törzsadatok, beleegyezések.
- **Termék domain:** termékkatalógus, készlet, árazás.
- **Rendelés domain:** rendelések, rendelési állapotok.
- **Szállítás domain:** szállítási események, logisztikai állapotok.
- **Számlázás domain:** számlák, fizetések, pénzügyi tranzakciók.

Mindegyik domain a saját adattermékeit publikálja. A **Marketing domain** fogyasztóként használja az Ügyfél és a Rendelés domain adattermékeit, anélkül, hogy hozzáférne azok nyers adatbázisához. Ha a Marketing domain újabb adatra van szüksége, azt az adattermék tulajdonosától kéri, szerződésmódosítással vagy új adattermék létrehozásával.

Ez a felépítés megszünteti a központi adattárház-csapat szűk keresztmetszetét, de új koordinációs költségeket teremt: a domaineknek együtt kell működniük a közös szemantikai szabványok kialakításában, és a fogyasztóknak több forrásból kell integrálniuk, mint egyetlen központi tárházból. A data mesh tehát nem „ingyen ebéd"; a bevezetés előtt mérlegelni kell, hogy a szervezet mérete és komplexitása indokolja-e a decentralizáció többletköltségét.

15.4.7 3. Önkiszolgáló adatplatform

Az önkiszolgáló platform a data mesh lehetővé tevő rétege: egy belső fejlesztői platform, amely a domain csapatokat ellátja mindazzal, ami az adattermékek létrehozásához, üzemeltetéséhez és megosztásához kell, anélkül, hogy központi adat-platform csapathoz kellene fordulniuk. A platform célja, hogy a domain csapatok **önállóan** tudjanak adatterméket létrehozni, a központi platformcsapat pedig a platformot fejleszti, nem az egyes adatfolyamatokat.

A platform tipikus képességei:

- **Tárolás és számítás:** adattó, objektumtár, stream-feldolgozás, batch-feldolgozás, adatbázisok – felügyelt szolgáltatásként.
- **Séma- és szerződéskezelés:** séma regisztráció, validáció, verziózás, backward compatibility ellenőrzés.
- **Adatkatalógus:** adattermékek regisztrációja, keresés, címkézés, tulajdonosok, dokumentáció.
- **Hozzáférésszabályozás:** identitás, szerepkörök, adatvédelmi besorolás, titkosítás, audit naplózás.
- **CI/CD és IaC:** az adatcsővezetékek és adattermékek infrastruktúrájának kódként való kezelése, verziózás, automatikus tesztelés.
- **Megfigyelés és minőség:** adatfrissesség, adatmennyiség, séma-eltérések, adatminőségi mutatók, riasztások.
- **Interfész-szabványok:** előre definiált hozzáférési minták (SQL, REST, streaming) és azok implementációja.

A platform **termékként** viselkedik: van termékmenedzsere, roadmapje, SLO-ja és visszajelzési ciklusa a domain csapatokkal. A platformcsapat nem az adatokat üzemelteti, hanem az eszközöket, amelyekkel a domainek az adataikat üzemeltetik. Ez az elv megegyezik a belső fejlesztői platform és a platform engineering szemléletével, amelyet a könyv későbbi fejezete részletesen tárgyal.

15.4.8 3.1. Példa: önkiszolgáló platform minimális MVP-je

Egy kis szervezetben az önkiszolgáló platform MVP-je a következőkből állhat:

- **Objektumtár** (pl. S3-kompatibilis) nyers és feldolgozott adatok tárolására.
- **Batch feldolgozó** (pl. Apache Spark vagy felügyelt szolgáltatás) ütemezett adatcsővezetékek futtatására.
- **Séma regiszter** (pl. Avro/JSON Schema) a szerződések verziózására.
- **Katalógus** (pl. egy belső webes felület vagy DataHub, OpenMetadata) az adattermékek regisztrálására.
- **CI/CD sablonok** az adatcsővezetékek telepítéséhez.

- **Megfigyelő műszerfal** az adattermékek frissességéhez és minőségéhez.

A domain csapat ezen a platformon egy sablonból indulva létrehoz egy új adattermék-projektet, definiálja a szerződést, implementálja a feldolgozást, és a CI/CD automatikusan telepíti a csővezetéket, regisztrálja a sémát és a katalógus-bejegyzést. Az adattermék élesítéséhez nem kell a központi platformcsapattal egyeztetni; a platform maga kényszeríti ki a kötelező elemeket (szerződés megléte, adatvédelmi besorolás, minőségi mutatók definiálása).

15.4.9 4. Federált governance és adatminőség

A decentralizált adattulajdonlás paradox módon erős központi koordinációt igényel a globális szabványok terén. A **federált governance** ezt a kettősséget oldja fel: a governance feladatokat megosztja a központi szabályozó testület és a domain csapatok között. A központi szint határozza meg a globális szabályokat – interoperabilitási szabványok, adatvédelmi és biztonsági követelmények, az adattermékek minimális minőségi elvárásai, a katalógus kötelező mezői –, a domainek pedig ezen kereteken belül autonóm döntéseket hoznak a saját adataikról.

A kulcsszó a **számítási** (computational) governance: a szabályokat nem dokumentumban, hanem a platformba ágyazott, gépi úton ellenőrizhető policy-k formájában kell érvényre juttatni. Példa: a platform nem engedi élesíteni azt az adatterméket, amelynek nincs érvényes adatszerződése, vagy amelynek séma nem felel meg a megadott kompatibilitási szabálynak. Az ilyen ellenőrzések a CI/CD pipeline részét képezik, így a megfelelés nem utólagos audit, hanem a szállítási folyamat beépített része.

15.4.10 4.1. Adatminőség a data mesh-ben

Az adatminőség a data mesh-ben nem egy központi adatminőség-csapat kizárólagos feladata, hanem **az adattermék terméktulajdonsága**. Minden adattermék rendelkezik minőségi SLO-kkal, amelyeket a szerződés rögzít, és amelyeket a domain csapat mér, publikál és folyamatosan javít. A tipikus adatminőségi dimenziók:

- **Teljesség:** a várt rekordok mekkora hányada van jelen. Példa: a rendelés adattermék teljessége 99,5% 24 órás ablakban mérve.
- **Pontosság:** az adat értéke helyes-e. Példa: a címmező érvényes postai cím-e, ellenőrzés címvalidáló szolgáltatással.
- **Időszerűség / frissesség:** az adat mennyire naprakész. Példa: a rendelési esemény átlagos késése < 1 perc, p99 < 5 perc, 1 órás ablakban mérve.
- **Konzisztencia:** az adat konzisztens-e más adattermékekkel. Példa: a rendelés adattermék státusza nem mond ellent a számlázási adatterméknek.
- **Érvényesség:** az adat megfelel-e a sémának és az üzleti szabályoknak. Példa: az `order_total` nem negatív, a `status` a megengedett felsorolásból való.

A minőségi SLO-kat a szerződés részeként kell definiálni, és a platformnak automatikusan mérnie kell őket. A mérési eredményeket műszerfalon kell publikálni, riasztásokat kell küldeni az SLO megsértésekor, és az incidenseket a domain csapatnak kell kezelnie. Az adatminőség tehát ugyanúgy operatív felelősség, mint a szolgáltatás rendelkezésre állása.

A federált governance az adatminőséget is szabályozza: a központi testület meghatározhatja a minimális kötelező minőségi dimenziókat (például minden adatterméknek legyen frissesség- és teljesség-SLO-ja), míg a konkrét küszöbértékeket a domain a fogyasztókkal egyeztetve határozza meg. Ez a felosztás biztosítja, hogy a globális irányelvek betartathatók legyenek anélkül, hogy a központi csapat mikromenedzselné a domaineket.

15.4.11 5. Gyakorlati példa: data mesh bevezetési terv

A következő labor azt mutatja be, hogyan lehet egy kisebb szervezetben elindítani a data mesh bevezetését. A példa fiktív, a cél a gondolkodásmód és az ellenőrzési szempontok gyakorlása.

15.4.12 5.1. Kiinduló állapot

A példa cég, a **RetailCo** közepes méretű e-kereskedő. Jelenleg egy központi adattárház-csapat üzemeltet egy PostgreSQL alapú adattárházat, amelybe napi batch folyamatokkal töltik be az adatokat az értékesítési, logisztikai és pénzügyi rendszerekből. A fogyasztók (marketing, controlling, ügyfélszolgálat) közvetlen SQL-lel kérdeznek le, de gyakori a panasz: az adatok késnek, a séma nem dokumentált, a minőségi hibákért senki nem vállal felelősséget. A domain csapatok (értékesítés, logisztika, pénzügy) nem érzik magukénak az adatokat.

A cél egy olyan data mesh bevezetési terv elkészítése, amely az első három hónapra koncentrál, és az első adattermék-pilotot is tartalmazza.

15.4.13 5.2. Lépések

1. **Domain térkép készítése:** az üzleti folyamatok és a meglévő rendszerek alapján azonosítsuk a domaineket. RetailCo esetében: Értékesítés, Logisztika, Pénzügy, Ügyfél, Termék. Minden domainhez rendeljünk felelős csapatot és adatgazdát.
2. **Adattermék-jelöltek azonosítása:** minden domainben válasszunk ki 1-2 olyan adathalmazt, amelyet a többiek fogyasztanak, és amelynek jelenleg rossz a minősége vagy a dokumentáltsága. Példa: az Értékesítés domain publikálja a **Rendelések** adatterméket, a Logisztika a **Szállítási események** adatterméket.
3. **Önkiszolgáló platform MVP felépítése:** válasszunk egy felügyelt objektumtárat, egy batch feldolgozó keretet, egy séma regisztert és egy egyszerű katalógust. A platformnak tudnia kell: adattermék projekt sablont biztosítani, szerződést validálni, CI/CD-t futtatni.
4. **Globális szabványok definiálása:** a federált governance testület (amelyben minden domain képviselteti magát) határozza meg a minimális követelményeket: minden adatterméknek legyen szerződése, tulajdonosa, leírása, frissesség- és teljesség-SLO-ja. A személyes adatokra vonatkozó szabályokat is rögzíteni kell.
5. **Pilot adattermék bevezetése:** az Értékesítés domain létrehozza a **Rendelések** adatterméket a platformon. A szerződést a fogyasztókkal egyeztetik, a minőségi SLO-kat definiálják, a feldolgozást implementálják.
6. **Mérés és visszacsatolás:** a pilot alatt mérjük a frissességet, a teljességet, a fogyasztói elégedettséget és a bevezetési erőfeszítést. Az eredmények alapján döntünk a folytatásról.

15.4.14 5.3. Döntési napló (kivonat)

A bevezetés során a fontos döntéseket ADR formában rögzítjük. Példa:

```
# ADR-014: A Rendelések adattermék első interfésze napi batch SQL view

Kontextus: Az Értékesítés domain a rendeléseket a saját tranzakciós adatbázisából
szeretné publikálni. A fogyasztók jelenleg SQL-lel kérdezik le a központi
adattárházat. Valós idejű igény nincs, a fogyasztók a napi frissítést elfogadják.

Döntés: Az első adattermék interfész batch feldolgozással készített, napi
frissítésű SQL view legyen. A szerződés rögzíti a sémát, a frissességi SLO-t
(naponta legkésőbb 02:00 UTC-ig) és a teljesség SLO-t (99,5%).

Alternatívák:
- Streaming interfész (Kafka): valós idejű, de a fogyasztók még nem állnak készen,
  és a platform MVP nem támogatja jól.
- Közvetlen hozzáférés a tranzakciós adatbázishoz: egyszerű, de megsérti a domain
  határokat, és a fogyasztókat a belső sémához köti.

Következmények:
- A fogyasztók továbbra is SQL-lel dolgozhatnak, alacsony a migrációs költség.
- A frissesség nem valós idejű; később szükség lehet streaming interfészre.
- A domain felelőssége megnő, mert a batch folyamatot üzemeltetni kell.
Állapot: Elfogadva, 2026-01-15.
```

15.4.15 5.4. Ellenőrzési szempontok

A labor átadandó eredménye egy dokumentum vagy jegyzetfüzet, amely tartalmazza:

- **Kontextus:** a szervezeti háttér, a meglévő adatarchitektúra, a motiváció.
- **Domain térkép:** a domainek és a felelős csapatok.
- **Adattermék-jelöltek:** a kiválasztott adattermékek és indoklás.
- **Platform MVP specifikáció:** a szükséges képességek és a választott technológiák.
- **Governance szabályok:** a globális minimumkövetelmények.
- **Pilot terv:** az első adattermék lépései, szerződésvázlat, minőségi SLO-k.
- **Döntési napló:** legalább három ADR a kulcsdöntésekről.
- **A mért és a várt eredmény szétválasztása:** külön táblázatban jelezzük, hogy mely értékek mért adatok (például a jelenlegi adatkésés a központi tárházban: p95 = 14 óra, 1 hetes ablakban mérve) és melyek várt célok (például a pilot után p95 $<$ 30 perc).
- **Visszaállítási út:** ha a pilot kudarcot vall, hogyan lehet visszatérni a korábbi központi tárházhoz, és milyen költségekkel jár.

A labor ellenőrzőlistája:

- ☐ A domain térkép lefedi az összes érintett üzleti területet.
- ☐ Minden adattermék-jelölthöz tartozik tulajdonos, fogyasztói kör és indoklás.
- ☐ Az önkiszolgáló platform MVP specifikációja tartalmazza a tárolást, a feldolgozást, a séma kezelést, a katalógust és a CI/CD-t.
- ☐ A governance szabályok között szerepel a személyes adatok kezelése és az adatminőségi minimumok.
- ☐ A szerződésvázlat tartalmaz sémát, szemantikát, frissességi és teljességi SLO-t.
- ☐ A mért és a várt eredmény egyértelműen elkülönül.
- ☐ A visszaállítási út leírása konkrét lépéseket tartalmaz.

15.4.16 5.5. Önellenőrző kérdés

Kérdés: A RetailCo-nál a Marketing domain szeretné a Rendelések adatterméket valós időben fogyasztani, mert kampányokat optimalizál. A jelenlegi adattermék napi batch frissítésű. Mi a

helyes lépés a data mesh elvek szerint?

Várható megoldás: A Marketing domain jelzi az igényt a Rendelések adattermék tulajdonosának (Értékesítés domain). A tulajdonos felméri, hogy a valós idejű igény kielégíthető-e egy új interfésszel (például streaming), és milyen költségekkel jár. A szerződést módosítani kell, új interfészt kell definiálni, és a platformnak támogatnia kell a streaming szállítást. A Marketing domain nem férhet hozzá közvetlenül a tranzakciós adatbázishoz, mert az megsértené a domain határokat és a szerződéses modellt. A döntés a domain tulajdonosáé, aki a fogyasztói igényeket a saját prioritásai szerint mérlegeli, a federált governance keretein belül.

15.4.17 6. Tipikus hibák és ellenérvek

A data mesh bevezetése során számos hiba elkövethető. A leggyakoribbak:

15.4.18 6.1. „A data mesh egyenlő a mikroszolgáltatásokkal, csak adatokra”

Ez a félreértés arra vezet, hogy a szervezet minden adathalmazt apró, önálló adattermékké alakít, anélkül, hogy a termékfelelősséget és a szerződéseket komolyan venné. Az adattermék nem technikai csomagolás, hanem szervezeti felelősségi határ. Ha a domain csapat nem vállal üzemeltetési és minőségi felelősséget, az adattermék csak egy újabb adatbázis-nézet lesz, és a probléma megmarad.

15.4.19 6.2. Az önkiszolgáló platform alulbecslése

A legtöbb data mesh bevezetés azért kudarcot vall, mert a platformra nem szánnak elegendő erőforrást. A domain csapatok nem tudnak hatékonyan adatterméket építeni, ha a platform hiányos: nincs séma regiszter, nincs katalógus, nincs egységes hozzáférés-kezelés, nincs megfigyelés. Ilyenkor a decentralizáció káoszt szül, és a központi csapat továbbra is szűk keresztmetszet marad, mert minden platformhibát ők javítanak. A platform építése befektetés, amelyet a data mesh bevezetés előtt és közben folyamatosan finanszírozni kell.

15.4.20 6.3. A federált governance elhanyagolása

A decentralizáció hajlamos a szabályok fellazulására. Ha nincsenek globális szabványok és azokat kikényszerítő mechanizmusok, akkor minden domain másképp nevezi el a mezőket, más frissességi garanciákat ad, és a fogyasztók integrációs rémálomba kerülnek. A governance-t nem szabad utólag bevezetni; a platformba kell tervezni. Ugyanakkor a túlzott központosítás is hiba: ha a governance minden részletet szabályoz, elvész a domain autonómia.

15.4.21 6.4. Az adattermékek túlzott granularitása

Minden adatterméknek van létrehozási és üzemeltetési költsége. Ha egy domain tucatnyi apró adatterméket hoz létre, a karbantartási teher elviselhetetlenné válik. Az adatterméket a fogyasztói igények és az üzleti folyamatok mentén kell kialakítani, nem pedig technikai táblánként. Egy jó adattermék általában egy jól körülhatárolt üzleti entitást vagy eseményfolyamot fed le (például rendelések, ügyfelek, számlák), nem pedig egyetlen mezőcsoportot.

15.4.22 6.5. A szervezeti felkészültség hiánya

A data mesh szervezeti változás, nem csupán technológiai váltás. Ha a domain csapatok nem rendelkeznek adatmérnöki képességekkel, vagy a vezetés nem adja meg a felhatalmazást, a bevezetés el fog bukni. A data mesh akkor működik jól, ha a szervezet már elég érett a decentralizált szolgáltatás-üzemeltetésben (például mikroszolgáltatásokkal), és a domainek rendelkeznek a szükséges technikai kompetenciákkal. Kis szervezeteknél a data mesh általában felesleges komplexitást visz be; a központi adattárház hatékonyabb lehet.

15.4.23 6.6. Ellenérvek és kivételek

A data mesh nem univerzális megoldás. A központi adattárház vagy adattó továbbra is jó választás, ha:

- A szervezet kicsi, kevés domainnel és kevés adatforrással.
- Az adatok használata erősen konszolidált, kevés fogyasztóval.
- A szabályozási környezet erős központi kontrollt igényel (például szigorúan auditált környezetben).
- A domain csapatok nem rendelkeznek adatmérnöki kapacitással.

Ilyenkor a data mesh bevezetése túlzott architekturális beruházás lenne, amely nem térül meg. A döntést mindig a konkrét szervezeti kontextus és a minőségi követelmények alapján kell meghozni, nem pedig trendkövetésből.

15.4.24 7. Összefoglalás

A data mesh az adatarchitektúra új szemlélete, amely a felelősséget és a szolgáltatásokat a domainekhez delegálja. Az **adattermék** a fogyasztható adatcsomag, amelyet szerződés véd és minőségi mutatók jellemeznek. Az **adatdomain** a szervezeti egység, amely az adatokat birtokolja és üzemelteti, a **önkiszolgáló platform** pedig az eszközöket biztosítja ahhoz, hogy a domainek ezt autonóm módon tegyék. A **federált governance** tartja egyensúlyban a decentralizációt a globális szabványokkal, az adatminőség pedig az adattermék terméktulajdonsága.

A fejezetben bemutatott példák és a labor segítenek a fogalmak alkalmazásában, de a valós bevezetés mindig kontextusfüggő. A következő alfejezet (Adatmodellek, polyglot persistence és tranzakciók) az adatmodellezésre, a polyglot persistence-re és a tranzakciókra épít, ezért fontos, hogy az itt bevezetett adattermék- és szerződésfogalmakat megértsük, mert azok alapozzák meg az adatarchitektúra további tervezési döntéseit.

Ellenőrzőlista a fejezethez:

- ☐ Tudom definiálni az adatterméket és felsorolni legalább öt jellemzőjét.
- ☐ El tudom magyarázni az adatszerződés szerepét és tartalmát.
- ☐ Felismerem az adatdomain és a bounded context kapcsolatát.
- ☐ Tudom, milyen képességeket kell biztosítania egy önkiszolgáló adatplatformnak.
- ☐ Meg tudom indokolni, miért fontos a federált governance és hogyan érvényesíthető gépi úton.
- ☐ Fel tudom sorolni a data mesh bevezetésének tipikus hibáit és azok okait.
- ☐ El tudom dönteni egy konkrét szervezeti kontextusban, hogy a data mesh vagy a központi adatarchitektúra-e a megfelelőbb.

Önellenőrző kérdés: Mi a különbség egy nyers adatbázistábla és egy adattermék között, és miért nem elegendő a táblát publikálni a data mesh-ben?

IV. RÉSZ – Adat, platform és cloud

16 Adatarchitektúra és perzisztencia

16.1 Adatmodellek, polyglot persistence és tranzakciók

A perzisztencia-választás ritkán csak technológiai kérdés. Az adatmodell, a tranzakciós garanciák, a séma evolúciója és az üzemeltetési költségek együtt határozzák meg, hogy egy rendszer hosszú távon karbantartható, skálázható és megbízható marad-e. Ez az alfejezet a négy leggyakoribb adatmodellt – relációs, dokumentum-, kulcs-érték és gráfmodellt – a választási szempontokkal, a tranzakció és izoláció alapfogalmaival, valamint a polyglot persistence üzemeltetési árával együtt tárgyalja. A cél az, hogy a döntést ne divat, hanem a konkrét hozzáférési minták, konzisztencia-követelmények és csapatfeltételek vezessék.

16.1.1 1. Miért döntés az adatmodell?

Egy perzisztencia-réteg kiválasztásakor gyakran a leggyorsabb elérhető adatbázist keressük, pedig a legfontosabb kérdés az, hogyan olvassa és írja az alkalmazás az adatokat. Az adatmodell befolyásolja:

- a lekérdezések kifejező erejét és költségét,
- az írási és olvasási teljesítményt,
- a tranzakciós garanciákat,
- a séma változtatásának sebességét és kockázatát,
- az adatbázis üzemeltetési és skálázási feladatait.

A „relációs vagy dokumentum” kérdés rosszul feltett kérdés. Helyette azt kell feltenni: milyen aggregátumokat írunk és olvasunk, milyen gyakran változik a séma, mekkora a terhelés, és milyen konzisztenciát ígérünk a hívóknak. A négy modell nem ellenség, hanem különböző eszközök.

16.1.2 2. A négy alapmodell

16.1.3 2.1 Relációs modell

A relációs adatbázis táblákba rendezi az adatokat, sorokkal és oszlopokkal. A sorok közötti kapcsolatokat idegen kulcsok és join műveletek fejezik ki. A normalizálás csökkenti a redundanciát, és a tranzakciók ACID-garanciákat adnak.

Fő erősségei:

- Erős konzisztencia és tranzakciós izoláció (ACID).
- Rugalmas, deklarativ SQL-lekérdezések joinokkal, aggregálással és ablakfüggvényekkel.
- Érett eszközkészlet: migration, backup, monitoring, replikáció, indexelés.
- Jól ismert modell a legtöbb fejlesztő és üzemeltető számára.

Gyengeségei:

- A séma módosítása gyakran lockkal, migrációval és állásidővel járhat.

- Horizontális skálázás – főként írási terhelés esetén – nehezebb.
- Az objektumorientált vagy dokumentum-alapú alkalmazásmodell és a relációs táblák között impedance mismatch lép fel.

A relációs modell a legjobb választás, ha a rendszer erős tranzakciós konzisztenciát igényel, például pénzügyi egyenlegek, rendelésállapotok, számlázás, vagy ha sok kapcsolatot kell lekérdezni és a séma viszonylag stabil.

16.1.4 2.2 Dokumentummodell

A dokumentum-adatbázisok (például MongoDB, Couchbase, RavenDB) gyűjteményekbe szervezett, önleíró dokumentumokat tárolnak, tipikusan JSON vagy BSON formátumban. Egy dokumentum a hozzá tartozó adatokat beágyazva tartalmazza, így egy olvasás gyakran egyetlen I/O művelettel visszaadja a teljes aggregátumot.

Fő erősségei:

- Gyors fejlesztés, mert a dokumentum szerkezete közel áll az alkalmazásbeli objektumhoz.
- Rugalmas séma: új mezők hozzáadása nem igényel központi migrációt.
- Hatékony horizontális skálázás sharding révén, különösen írási terhelésnél.
- Jól illeszkedik olyan adatokhoz, amelyeket együtt olvasunk, például termékkatalógus-elemek, felhasználói profilok, konfigurációk.

Gyengeségei:

- A dokumentumok közötti joinok nem hatékonyak vagy nem támogatottak; a kapcsolatokat gyakran duplikációval vagy alkalmazásszintű összekapcsolással oldjuk meg.
- A konzisztencia-garanciák általában gyengébbek; több dokumentumot érintő tranzakciók korlátozottak.
- A séma ellenőrzését külön kell megoldani, különben az idők során inkonzisztens dokumentumok keletkeznek.

A dokumentummodell erős, ha az adathozzáférés túlnyomórészt egy aggregátumra irányul, a séma gyakran változik, és a konzisztenciakövetelmény mérsékelt.

16.1.5 2.3 Kulcs-érték modell

A kulcs-érték tároló (például Redis, DynamoDB, etcd, RocksDB) egyetlen elsődleges kulccsal ír és olvas értékeket. Az érték lehet egyszerű string, JSON, bináris blob vagy strukturált adat, de a tároló nem értelmezi annak belső szerkezetét.

Fő erősségei:

- Rendkívül alacsony késleltetés és nagy átbocsátás.
- Egyszerű horizontális skálázás, gyakran konzisztens hasheléssel.
- Ideális cache, session, token, számláló, rate limiting, feature flag vagy ideiglenes állapot tárolására.
- Minimális üzemeltetési komplexitás.

Gyengeségei:

- Nincs deklaratív lekérdezés, nincs join, nincs szekunder index.
- A tartományi lekérdezések és a részleges frissítések korlátozottak.

- Az alkalmazás felelőssége a séma, a verziózás és a konzisztencia kezelése.

A kulcs-érték modell nem általános célú adatbázis, hanem speciális szerepkörökre való eszköz, például nagy sebességű cache vagy időszakos állapot.

16.1.6 2.4 Gráfmodell

A gráf-adatbázis (például Neo4j, Amazon Neptune, JanusGraph) csomópontokból és élekből építi fel az adatmodellt. A csomópontok entitásokat, az élek kapcsolatokat és tulajdonságokat tárolnak. A lekérdezések gyakran gráfbejáráson alapulnak, például „kik azok a felhasználók, akik ezt a terméket vásárolták, és milyen más termékeket vettek még".

Fő erősségei:

- Nagyon gyors több ugrásos kapcsolati lekérdezések, amelyek relációs joinokkal exponenciálisan drágulnának.
- Természetes modellezés hálózati, ajánló, csalásfelderítő, tudásgráf, szociális gráf és ellátási lánc problémákra.
- Kifejező gráf-lekérdező nyelvek (Cypher, Gremlin, SPARQL).

Gyengeségei:

- Nem alkalmas minden adatproblémára; a tömeges aggregálás és a nagy volumenű egyszerű lekérdezések más modellekben hatékonyabbak.
- Az üzemeltetési tapasztalat és a tanulási görbe meredekebb.
- A horizontális skálázás és a particionálás gráfok esetén nehéz, mert a kapcsolatok átlógnak a partíciók között.

A gráfmodellt akkor válasszuk, ha a fő üzleti kérdés a kapcsolatok szerkezetére, nem pedig az entitások tömegére irányul.

16.1.7 3. Választási szempontok és döntési mátrix

Egy adott szolgáltatás vagy bounded context esetében a modellválasztás előtt rögzítenünk kell a következő szempontokat:

1. **Hozzáféré si minta:** milyen arányban olvasunk és írunk? Egész aggregátumot olvasunk, vagy sok joinra van szükség? Vannak-e szekunder lekérdezések?
2. **Konzisztencia-követelmény:** szükséges-e ACID tranzakció? Elfogadható-e az eventual consistency, és mekkora az eltérési ablak?
3. **Sémaevolúció:** milyen gyakran változik a séma? A változás additív vagy destruktív?
4. **Skálázás:** mekkora a várható írási/olvasási terhelés? Milyen a forgalom növekedési üteme?
5. **Csapat tapasztalata:** van-e belső tudás az adott adatbázis üzemeltetésére?
6. **Üzemeltetési költség:** ki kezeli a backupot, a patchelést, a replikációt, a monitoringot?

Ezeket a szempontokat egy döntési mátrixban foglalhatjuk össze. A mátrix nem ad automatikus választ, de átláthatóvá teszi a trade-offokat, és ADR-ben rögzítve megőrzi a döntés kontextusát.

Példa döntési mátrix egy rendeléskezelő szolgáltatás „megrendelések törzse" részéhez:

Szempont	Relációs	Dokumentum	Kulcs-érték	Gráf
ACID tranzakció	5	2	1	1
Join-intenzív lekérdezés	5	2	1	3

Szempont	Relációs	Dokumentum	Kulcs-érték	Gráf
Rugalmas séma	2	5	3	4
Nagy írási skálázás	3	5	5	2
Üzemeltetési érettség	5	3	4	2
Kapcsolati bejárás	2	2	1	5

Egy konkrét kontextusban a súlyozás eltérhet: ha a rendeléseknél erős ACID és joinok kellenek, a relációs modell nyer. Ha a termékkatalógus rugalmas sémát és nagy olvasási forgalmat igényel kevés kapcsolattal, a dokumentummodell előnyösebb.

Fontos elkerülni a leggyakoribb hibákat:

- **Mindenre ugyanazt az adatbázist** használni, mert az egyik résznek tökéletes, a másiknak szenvedés.
- **A dokumentumot mindenre** választani, figyelmen kívül hagyva a join-igényeket és a tranzakciókat.
- **A kulcs-érték tárolót általános adatbázisként** használni, ahol a szekunder lekérdezések elkerülhetetlenek.
- **A gráfmodellt divatból** bevezetni, amikor a kapcsolatok száma vagy mélysége nem indokolja.

16.1.8 4. Tranzakció, izoláció és sémamódosítás

16.1.9 4.1 Tranzakció és ACID

A tranzakció olyan műveletsor, amely atomi egységként hajtódik végre. A relációs adatbázisok általában ACID-garanciákat nyújtanak:

- **Atomicity (atomosság):** a tranzakció vagy minden műveletét végrehajtja, vagy egyiket sem. Nincs részleges siker.
- **Consistency (konzisztencia):** a tranzakció az adatbázist egyik érvényes állapotból a másikba viszi, betartva a megszorításokat.
- **Isolation (izoláció):** az egyidejű tranzakciók nem látják egymás köztes állapotait.
- **Durability (tartósság):** a commit után az eredmény túléli a rendszerhibát.

Az izolációs szintek azt szabályozzák, hogy az egyidejű tranzakciók milyen anomáliáknak vannak kitéve. A leggyakoribb szintek :

- **READ UNCOMMITTED:** dirty read lehetséges – egy tranzakció elolvashat egy másik által még nem commitolt, később visszavont adatot.
- **READ COMMITTED:** dirty read nem fordul elő, de non-repeatable read igen – ugyanazon lekérdezés kétszeri futtatása eltérő eredményt adhat.
- **REPEATABLE READ:** a non-repeatable read is kizárt, de phantom read lehetséges – egy tartományi lekérdezés újonnan beszúrt sorokat lát.
- **SERIALIZABLE:** a tranzakciók úgy futnak, mintha sorosan hajtódnának végre; minden anomália kizárva, de a párhuzamosság és a teljesítmény csökken.

A gyakorlatban a READ COMMITTED a legtöbb OLTP rendszer alapértelmezése, mert jó egyensúlyt ad a konzisztencia és a teljesítmény között. A SERIALIZABLE akkor indokolt, ha az üzleti szabály megsértése nagyobb kárt okoz, mint a csökkent átbocsátás.

A dokumentum-adatbázisok tranzakciós képességei eltérnek: például a MongoDB az egy dokumentumra vonatkozó műveleteket atomikusan kezeli, a több dokumentumot érintő tranzakciók pedig újabb verziókban korlátozottan érhetők el. A kulcs-érték tárolók gyakran csak egy kulcsra garantálnak atomi műveletet, a gráf-adatbázisok pedig a gráfműveletekre adnak tranzakciókat.

16.1.10 4.2 Sémamódosítás

A séma módosítása minden modellben más kihívást jelent.

- **Relációs adatbázisban** a változást migrációs eszközökkel (Flyway, Liquibase) verziókövetjük. A zero-downtime eléréséhez gyakran az expand-contract mintát használjuk: először additív módon hozzáadunk egy új oszlopot vagy táblát (expand), az alkalmazást átállítjuk, hogy mindkét sémával működjön, majd egy későbbi kiadásban eltávolítjuk a régi elemet (contract). A destruktív műveletet – például oszlop törlése – csak akkor szabad végrehajtani, ha az olvasók már nem használják.
- **Dokumentummodellben** a séma rugalmas, de ez nem jelenti azt, hogy nem kell kezelni. A dokumentumok validálását JSON Schema-val vagy az adatbázis beépített validátorával kell megoldani, különben a régi és új formátumú dokumentumok keveredése hibákhoz vezet. A változást fokozatosan, az alkalmazásban is támogatni kell.
- **Kulcs-érték tárolóban** a séma teljesen az alkalmazás felelőssége. A verziózást például a kulcsban vagy az értékben elhelyezett sémaverzióval oldhatjuk meg.
- **Gráfmodellben** a csomópont- és éltípusok, valamint a property-k módosítása hasonló az relációs migrációhoz, de a gráfséma gyakran kevésbé szigorú.

A sémamódosítás során mindig rögzíteni kell a kiinduló állapotot, az átmeneti lépéseket és a visszaállítási eljárást. A laborban a döntési mátrixhoz tartozó ADR-nek ezt is tartalmaznia kell.

16.1.11 5. Polyglot persistence üzemeltetési költsége

A polyglot persistence azt jelenti, hogy egy rendszeren belül több adatmodellt és adatbázist használunk, mindegyiket a neki legjobban megfelelő feladatra. Ez erős, de nem ingyen van. Az üzemeltetési költségeket gyakran alábecsüljük.

A következő tényezőkkel kell számolni:

- **Szakértelem:** minden adatbázisnak saját üzemeltetési modellje, biztonsági beállításai, backup-stratégiája és teljesítmény-hangolása van. Egy csapat ritkán ért mindegyikhez mélyen.
- **Monitoring és megfigyelhetőség:** minden adatbázisnak saját metrikái, logjai és riasztásai vannak. Az OpenTelemetry vagy a gyártói eszközök integrálása többletmunka.
- **Backup és helyreállítás:** minden rendszerre külön redundanciát, mentést és katasztrófahelyreállítási tervet kell készíteni, és rendszeresen tesztelni.
- **Biztonság:** hálózati hozzáférés, titokkezelés, titkosítás, auditnaplózás minden adatbázisnál külön konfigurációt igényel.
- **Skálázás:** az egyes adatbázisok más skálázási mintát igényelnek – a relációs read replika, a dokumentum sharding, a kulcs-érték klaszterezés, a gráf particionálás.
- **Fejlesztői élmény:** több adatbázis több API-t, több illesztőprogramot, több séma-kezelési eszközt jelent.
- **Licenc és infrastruktúra:** a felhőalapú menedzselt szolgáltatások költsége eltérő, és a több szolgáltatás több számlázási tételt, valamint több integrációs pontot jelent.

Ezért a polyglot persistence bevezetése előtt mindig készítsünk üzleti és technikai indoklást. Ha egyetlen adatbázis megfelel a követelményeknek, ne vezessünk be többet. A polyglot akkor éri meg, ha a különböző részek követelményei valóban eltérő modellt igényelnek, és a csapat képes az üzemeltetési terhet viselni.

A költség nem csak pénz, hanem kognitív terhelés is. A döntési mátrixban az üzemeltetési költséget mindig explicit szempontként kell szerepeltetni, és a következményeket ADR-ben rögzíteni.

16.1.12 6. Labor: perzisztencia-döntési mátrix készítése

Cél: Egy választott üzleti képességhez (bounded contexthez) perzisztencia-döntési mátrixot készíteni, és a döntést ADR-ben rögzíteni. A labor dokumentumban vagy jegyzetfüzetben végezhető el, és nem igényel éles adatbázist.

Kiinduló állapot:

- Válassz egy konkrét, kis üzleti képességet, például:
  - Ügyfélprofil-kezelés
  - Rendeléskezelés törzs
  - Termékkatalógus
  - Ajánlómodul
- Rögzítsd a rendszerhatárt: mi tartozik a szolgáltatáshoz, mi külső rendszer, kik a felhasználói szerepkörök.
- Tételezd fel, hogy a csapat ismeri a relációs, dokumentum-, kulcs-érték és gráfmodelleket, de csak általános szinten.

Elvárt eredmény:

1. **Kontextus leírása:** egy bekezdésben írd le a képességet, a legfontosabb üzleti folyamatokat és a szerepköröket.
2. **Követelmények rögzítése:** fogalmazz meg legalább négy minőségi követelményt quality attribute scenario formában, például:
   - *Inger:* 500 olvasási kérés másodpercenként.
   - *Környezet:* normál üzem, csúcsidő.
   - *Válasz:* a lekérdezés p95 válaszideje 200 ms alatt marad.
   - *Mérőszám:* p95 késleltetés, 1 órás mérési ablakban, 100 RPS terhelés mellett.
3. **Döntési mátrix:** készítsd el a fenti szempontokkal (konzisztencia, join-igény, séma evolúció, skálázás, üzemeltetési költség, csapat tapasztalat) a négy modellre. Minden cellához adj 1–5 értéket és rövid indoklást.
4. **ADR:** írj egy rövid ADR-t a javasolt elsődleges adatmodellről:
   - Kontextus
   - Döntés
   - Alternatívák és elvetésük oka
   - Következmények (pozitív és negatív)
   - Állapot: javasolt / elfogadott
5. **Ellenőrzési szempontok:** sorold fel, hogyan ellenőriznéd a döntést. Különböztesd meg a *várt* eredményt a *mért* eredménytől. Például: „Várt eredmény: a dokumentummodell p95 olvasási késleltetése 100 ms alatt marad 200 RPS mellett. Jelenleg nem mért, reprodukálást igényel.”

6. **Visszaállítási út:** mivel a labor dokumentumot készít, a visszaállítás egyszerűen a ADR elvetése vagy alternatíva választása. Ha valódi PoC-t futtatnál, add meg az izolált környezetet és a tisztítási lépéseket.

Ellenőrzési szempontok a labor értékeléséhez:

- A kontextus egyértelműen azonosítja a rendszerhatárt és a stakeholdereket.
- A minőségi követelmények scenario formában szerepelnek, mérőszámmal és mérési ablakkal.
- A döntési mátrix indoklása alátámasztja az értékeket, nem csak számokat tartalmaz.
- Az ADR tartalmazza az alternatívákat és a feladott lehetőségek okát.
- A mért és a várt eredmények egyértelműen elkülönülnek; a nem mért tételeket jelöli.

Példa a mátrixra (a fenti táblázat „megrendelések törzse" része alapján, a laborban a saját kontextusodra alkalmazd).

```
flowchart LR
    subgraph Rendszerhatar["Rendeléskezelő szolgáltatás"]
        direction LR
        U[Ügyfél] --> A[Rendelés API]
        A --> B[(Relációs DB: megrendelések, ACID)]
        A --> C[(Dokumentum DB: termékkatalógus)]
        A --> D[(Kulcs-érték: session, rate limit)]
        A --> E[(Gráf DB: kapcsolati ajánlások)]
    end
    B -.->|ODBC| F[Külső számlázó rendszer]
    C -.->|REST| G[Külső áruház katalógusa]
```

Jelmagyarázat: A folytonos nyilak szinkron hívások, a szaggatott nyilak külső rendszerkapcsolatok. *Feltételjegyzet:* A diagram nem C4 context szintű, csak a labor példájának koncepcionális topológiája; a laborban C4 context és container szintet kell készíteni.

16.1.13 7. Ellenőrzőlista és önellenőrző kérdés

16.1.14 Ellenőrzőlista

- ☐ Az adatmodell-választás a hozzáférési minták, a konzisztencia-követelmény és a séma evolúciója alapján történt.
- ☐ A tranzakciós garanciák (ACID, izolációs szint) dokumentálva vannak.
- ☐ A sémamódosításra van verziózott, visszaállítható eljárás.
- ☐ A polyglot persistence bevezetése előtt az üzemeltetési költségeket és a csapat teherbírását felmértük.
- ☐ A döntést ADR rögzíti, alternatívákkal és következményekkel.
- ☐ A mérési eredmények mérési ablakkal, terheléssel és bizonytalansággal szerepelnek; a nem mért állítások jelölve vannak.
- ☐ A rendszerhatár és a külső rendszerek a diagramokon feltüntetve.

16.1.15 Önellenőrző kérdés

Egy csapat egy új ajánlómodult épít, amelynek a felhasználók vásárlási kapcsolatai alapján kell valós időben, négy ugrás mélységig ajánlásokat adnia. A modulnak nincs szüksége erős tranzakciós konzisztenciára, de a bejárásnak 100 ms alatt kell lefutnia. Milyen adatmodellt választanál, és milyen szempontokkal indokolnád? Milyen üzemeltetési kockázatokat kell figyelembe venni? Válaszodat döntési mátrixszal és ADR-vázlattal támassz alá.

Továbbvitt eredmény: A következő alfejezet (Elosztott adatkonzisztencia és replikáció) az elosztott adatkonzisztencia és replikáció kérdéseire épít: hogyan kezeljük a tranzakciókat több csomópont között, milyen replikációs modellek léteznek, és hogyan gondolkodjunk a CAP-tétel keretei között. Az itt bevezetett modellválasztási és tranzakciós fogalmak elengedhetetlenek ahhoz, hogy a replikáció és az eventual consistency értelmezhető legyen.

16.2 Elosztott adatkonzisztencia és replikáció

Az előző alfejezetben áttekintettük az adatmodellek és a tranzakciók alapjait. Egyetlen csomóponton az adatbázis viszonylag egyszerűen kezelhető, de amint a rendszer elosztottá válik, új kérdések merülnek fel: hogyan biztosítsuk, hogy az adatok több példányban is rendelkezésre álljanak, hogyan növeljük a kapacitást horizontálisan, és milyen árat fizetünk a konzisztenciáért. Ez a fejezet ezeket a gyakorlati kérdéseket járja körül, a replikációra, a shardingra és a konzisztencia-modellekre összpontosítva.

16.2.1 Replikáció: miért és hogyan?

A replikáció azt jelenti, hogy ugyanazt az adathalmazt több csomóponton tároljuk. Ennek négy tipikus célja lehet:

1. **Rendelkezésre állás növelése:** ha az egyik csomópont meghibásodik, a replikák tovább szolgálják az adatokat.
2. **Olvasási terhelés elosztása:** az írások továbbra is egy helyre mennek, de az olvasásokat több replika között oszthatjuk szét.
3. **Adatközeli olvasás:** a felhasználó földrajzilag közel eső replikáról olvashat, csökkentve a hálózati késleltetést.
4. **Katasztrófa-helyreállítás:** egy teljes adatközpont kiesése esetén a távoli replika lehetővé teszi a helyreállítást.

A replikáció alapkérdése, hogy az írás mikor tekinthető sikeresnek. Ez a választás alapvetően meghatározza a rendszer konzisztenciáját és elérhetőségét.

16.2.2 Szinkron replikáció

Szinkron replikáció esetén az írás akkor tér vissza a klienshez sikerrel, ha a vezérlő (leader vagy primary) és az összes kijelölt replika is tartósan rögzítette a változást. Ez gyakran valamilyen konszenzus protokollal vagy kétfázisú commit-tal valósul meg, például Raft vagy Paxos alapú replikált naplóval.

Előnyök:

- **Erős konzisztencia:** az olvasó a sikeres írás után bármely replikáról a legfrissebb értéket látja, feltéve, hogy az olvasás is kvórum alapú.
- **Nincs adatvesztés:** ha a leader meghibásodik, a replikák tartalmazzák az összes elismert írást, így a failover nem jár adatvesztéssel.
- **Egyszerűbb mentális modell:** a fejlesztők erős garanciákra építhetnek, hasonlóan az egycsomópontos ACID tranzakciókhoz.

Hátrányok:

- **Magas írási késleltetés:** az írásnak meg kell várnia a replikák megerősítését, ami hálózati körútonként is növeli a válaszidőt.
- **Csökkentett elérhetőség:** ha valamelyik replika elérhetetlen, az írás vagy blokkolódik, vagy hibát ad, ezért a szinkron replikációhoz általában kvórumot használnak, nem pedig minden replika megerősítését.
- **Bonyolultabb üzemeltetés:** a konszenzus protokollok konfigurálása és felügyelete szakértelmet igényel.

A szinkron replikációt akkor érdemes választani, ha az adatvesztés nem megengedhető, és a késleltetésből származó többletköltséget elfogadja a rendszer. Tipikus példa a pénzügyi tranzakciók könyvelése vagy az elektronikus egészségügyi nyilvántartás kritikus részei. Ilyenkor a rendszer tervezésekor a rendelkezésre állást úgy kell értelmezni, hogy inkább rövid ideig elutasítjuk az írást, mintsem hogy adatot veszítsünk.

16.2.3 Aszinkron replikáció

Aszinkron replikáció esetén a leader azonnal visszaigazolja az írást a kliensnek, és a replikák a háttérben, később kapják meg a változást. Ez a gyakorlatban a leggyakoribb mód, mert olcsó és gyors.

Előnyök:

- **Alacsony írási késleltetés:** a kliens nem vár a replikákra.
- **Magas átbocsátás:** a leader nem blokkolódik lassú replikáktól.
- **Nagyobb elérhetőség írásra:** ha egy replika kiesik, az írás továbbra is működik.

Hátrányok:

- **Replikációs lag:** a replikák lemaradhatnak, ezért az olvasó elavult adatot láthat.
- **Adatvesztés kockázata:** ha a leader meghibásodik, mielőtt a replikák megkapták volna a legutóbbi írásokat, azok elveszhetnek.
- **Bonyolultabb olvasási szemantika:** az alkalmazásnak kezelnie kell a stale read jelenséget.

Az aszinkron replikációt jellemzően olvasási replikákhoz, analitikai másolatokhoz, naplózáshoz vagy olyan adatokhoz használják, ahol a rövid távú inkonzisztencia elfogadható. Például egy webáruház termékkatalógusa esetén elfogadható, ha egy frissen módosított ár néhány másodpercig nem látszik minden replikán, de a rendelés rögzítése már szinkron replikációt igényelhet.

16.2.4 Replikációs topológiák

A replikáció elrendezése három alapvető mintát követhet:

- **Single-leader (master-slave):** minden írás egyetlen leaderhez érkezik, a replikák csak olvasásra szolgálnak. Ez a legelterjedtebb, mert egyszerű és jól érthető. Hátránya, hogy a leader szűk keresztmetszet lehet, és failoverre van szükség.
- **Multi-leader:** több csomópont is fogadhat írást, amelyek aztán replikálják egymásnak. Ez hasznos több adatközpontos környezetben, de komoly konfliktuskezelést igényel, mert ugyanaz az adat egyszerre módosulhat különböző helyeken.
- **Leaderless:** az írás és olvasás is kvórum alapú, például Cassandra vagy Dynamo stílusú rendszerekben. Itt nincs kijelölt leader, a rendszer replikák többségének megerősítését várja.

A gyakorlatban a legtöbb rendszer single-leader módon indul, és csak akkor tér el ettől, ha a földrajzi elosztás vagy a magas írási rendelkezésre állás ezt indokolja.

16.2.5 Konzisztencia-modellek a gyakorlatban

A konzisztencia nem bináris fogalom; a rendszer tervezésekor explicit döntést kell hozni arról, milyen garanciákat nyújtunk. A leggyakoribb modellek:

- **Erős konzisztencia (linearizálhatóság):** minden művelet úgy tűnik, mintha egyetlen időpontban, atomosan hajtódott volna végre, és minden kliens ugyanabban a sorrendben látja az eredményeket. Ez a legerősebb garancia, de szinkron replikációt és gyakran magas késleltetést követel.
- **Eventual consistency (gyenge konzisztencia):** ha nincsenek újabb írások, akkor idővel minden replika konvergál ugyanahhoz az állapothoz. Rövid távon azonban az olvasók eltérő értékeket láthatnak. Ez a modell jól működik, ha a konfliktusok ritkák és az üzleti folyamat tolerálja az átmeneti inkonzisztenciát.
- **Köztes garanciák:** a gyakorlatban gyakran elegendő néhány speciális tulajdonság:
  - **Read-your-writes:** a felhasználó mindig látja a saját korábbi írását, még akkor is, ha más replikáról olvas.
  - **Monotonic reads:** ha egyszer friss adatot látott, később nem kaphat régebbi értéket.
  - **Session consistency:** az adott munkamenetben a fenti garanciák érvényesek.
  - **Causal consistency:** az oksági kapcsolatban álló írások sorrendje mindenhol azonos.

A CAP-tétel értelmében hálózati partíció esetén választani kell a konzisztencia és az elérhetőség között: vagy elutasítjuk az írást, hogy konzisztensek maradjunk, vagy elfogadjuk, és átmenetileg inkonzisztens állapotot engedünk meg. A gyakorlati architektúra gyakran nem egyetlen modellt alkalmaz, hanem adatonként vagy műveletenként differenciál: a rendelés rögzítése lehet erősen konzisztens, míg a termékértékelések megjelenítése eventual consistent.

Az ACID tranzakciók helyett elosztott rendszerekben gyakran a BASE (Basically Available, Soft state, Eventually consistent) elvet követik, de ez nem jelenti azt, hogy az ACID haszontalan lenne; inkább arról van szó, hogy a garantált konzisztencia ára magasabb, és nem minden adat igényli.

16.2.6 Sharding: adatok horizontális felosztása

A replikáció önmagában nem oldja meg a kapacitásproblémát, mert a leader írási teljesítménye továbbra is egyetlen csomópontra korlátozódik. A sharding (más néven particionálás) az adathalmazt logikai darabokra osztja, és ezeket különböző csomópontokon tárolja.

Shard key kiválasztása

A legfontosabb döntés a shard key. Ez az a mező vagy mezőkombináció, amely alapján a rendszer eldönti, hogy egy adott rekord melyik shardba kerül. Két alapvető stratégiát különböztetünk meg:

- **Hash alapú particionálás:** a shard key hash-ét számítjuk ki, és az alapján osztjuk el a rekordokat. Ez egyenletes eloszlást ad, de a tartomány-alapú lekérdezések (pl. egy adott időintervallum összes rekordja) nem hatékonyak, mert a rekordok szétszóródnak.
- **Range alapú particionálás:** a shard key értéktartománya szerint osztjuk fel az adatokat, például dátum szerint. Ez gyors tartomány-lekérdezést tesz lehetővé, de forró pontok alakulhatnak ki, ha az írások egy adott tartományra koncentrálódnak.

A jó shard key tulajdonságai:

- **Egyenletes eloszlás:** kerülje a hőpontokat, ahol egy shard sokkal nagyobb terhelést kap.
- **Lekérdezési mintákhoz illeszkedés:** a leggyakoribb lekérdezések lehetőleg egyetlen shardot érintsenek, így elkerülhető a cross-shard join.
- **Stabilitás:** a shard key lehetőleg ne változzon gyakran, mert az áthelyezést igényel.

Például egy több-bérlős SaaS rendszerben a `tenant_id` jó shard key lehet, mert a bérlő adatai együtt maradnak, és a lekérdezések tipikusan egy bérlőre vonatkoznak. Ugyanakkor ha egy bérlő sokkal nagyobb, mint a többi, hotspot alakulhat ki.

Rebalancing

Ahogy az adat növekszik vagy a csomópontok változnak, a shardokat újra kell osztani. A konzisztens hashelés (consistent hashing) és a virtuális csomópontok segítenek minimalizálni az áthelyezendő adat mennyiségét. A rebalancing során a rendszer online maradhat, de figyelni kell az átmeneti inkonzisztenciára és a teljesítménycsökkenésre.

Cross-shard lekérdezések

Ha egy lekérdezés több shardot érint, a rendszer vagy elosztott lekérdezést hajt végre, vagy előre aggregált adatokat használ. A gyakorlatban ilyenkor elkerülhetetlen a válaszidő növekedése, ezért a sharding tervezésekor törekedni kell arra, hogy a leggyakoribb üzleti műveletek egyetlen shardra férjenek. A több shardot átfogó tranzakciók bonyolultak, gyakran szükség van two-phase commit vagy saga mintára, de ezek ára magas, ezért a legjobb elkerülni őket.

Replikáció és sharding kombinálása

A termelési rendszerekben ritkán használnak shardingot replikáció nélkül. Minden shardot több replikán tárolnak, így a shardok maguk is rendelkezésre állóak. Az írások a shard vezérlőjéhez mennek, az olvasások a replikákról is kiszolgálhatók. A sharding tehát az írási és tárolási kapacitást, a replikáció pedig a rendelkezésre állást és az olvasási teljesítményt növeli.

16.2.7 Konfliktusfeloldás és adatvesztés

A replikáció és a sharding elosztott környezetben elkerülhetetlenül konfliktusokhoz vezethet. Konfliktus akkor keletkezik, ha ugyanazt az adatot különböző replikákon, egymástól függetlenül módosítják. Ez leggyakrabban multi-leader vagy leaderless rendszerekben fordul elő, de hálózati partíció során single-leader környezetben is megtörténhet, ha a régi leader még fogad írásokat.

Konfliktusfeloldó stratégiák

- **Last-write-wins (LWW):** minden íráshoz időbélyeget rendelünk, és a legfrissebb nyer. Ez egyszerű, de adatvesztést okozhat, mert a korábbi írás felülíródik, és az időbélyegek óraszinkronja sem megbízható elosztott rendszerekben.
- **Verzióvektorok:** minden replika nyilvántartja a saját verziószámát, így a konfliktus észlelhető, és az alkalmazás dönthet a feloldásról.
- **CRDT (Conflict-free Replicated Data Type):** olyan adatszerkezetek, amelyek matematikailag garantálják a konvergenciát, például növekményes számláló vagy egyesíthető halmaz. Használatukkal elkerülhető a konfliktus, de nem minden üzleti adat modellezhető CRDT-ként.
- **Alkalmazás-szintű feloldás:** a rendszer a konfliktust kitűzi az alkalmazásnak, amely üzleti szabályok alapján egyesíti az értékeket. Például egy bevásárlókosárnál a tételek uniója lehet a feloldás, nem pedig az egyik változat elvetése.

Adatvesztés forgatókönyvek

Az adatvesztés leginkább aszinkron replikációnál fordul elő, amikor a leader meghibásodik, mielőtt a replikák megkapták volna a legutóbbi írásokat. Ezt tovább súlyosbíthatja, ha a replikák is lemaradtak, és a failover során egy lemaradt replikát léptetnek elő leaderré. Másik gyakori eset, amikor a shardok áthelyezése közben valamelyik csomópont meghibásodik, és a nem replikált adat örökre elvész.

A védekezési lehetőségek:

- Kritikus adatoknál használjunk szinkron replikációt vagy legalább quorum alapú írást.
- Írásokat tartósan, write-ahead loggal (WAL) és fsync-konfigurációval rögzítsünk.
- Rendszeres biztonsági mentés és visszaállítási teszt.
- Idempotens műveletek tervezése, hogy az újrapróbálkozás ne okozzon duplikációt.
- Failover forgatókönyvek rendszeres tesztelése, beleértve a hálózati partíció szimulálását.

16.2.8 Labor: replikációs és sharding-terv

A fejezet laborja egy dokumentált terv elkészítése, amely reprodukálható módon bemutatja, hogyan választanánk replikációs és sharding stratégiát egy adott rendszerhez. A labor során nem futtatunk éles rendszert; a cél a döntési folyamat gyakorlása és a dokumentáció elkészítése.

Kontextus (példa)

Tegyük fel, hogy egy e-kereskedelmi platformot tervezünk, amely három fő adathalmazt kezel: termékkatalógus, vevői rendelések, és felhasználói profilok. A rendszer várhatóan napi 50 000 rendelést dolgoz fel, a katalógust naponta 1000-szer módosítják, a profilokat pedig ritkán, de sok felhasználó olvassa. A rendelkezésre állási cél 99,9% havi szinten, a rendelés rögzítésekor nem fogadható el adatvesztés, míg a katalógus frissítések rövid késleltetése elfogadható.

Feladat

Készíts egy dokumentumot, amely a következőket tartalmazza:

1. **Kiinduló állapot:** ismertesd a feltételezett adatbázis-sémát, a várható terhelést és a minőségi követelményeket.
2. **Replikációs terv:** minden adathalmazhoz add meg, hogy szinkron vagy aszinkron replikációt használnál-e, mekkora replikációs faktorral, és indokold a döntést.
3. **Sharding terv:** ha sharding szükséges, válassz shard key-t, ismertesd a particionálási módszert (hash vagy range), és elemezd a lehetséges hőpontokat. A rendeléseknél például a `customer_id` hash alapú sharding jó választás lehet, mert a vevő rendelései együtt maradnak, de a nagy forgalmú vevők hőpontot okozhatnak.
4. **Konzisztencia-szintek:** minden művelettípushoz add meg, milyen konzisztencia-garanciát nyújt a rendszer, és milyen feloldási stratégiát alkalmazol konfliktus esetén.
5. **Döntési napló (ADR):** minden fontos döntést rögzíts a következő formában: kontextus, döntés, alternatívák, következmények, állapot. Például:
    - **ADR-1 Helyzet:** a rendelés rögzítése kritikus, adatvesztés nem megengedett. **Döntés:** szinkron replikáció a rendelések táblára. **Alternatívák:** aszinkron replikáció (elvetve adatvesztési kockázat miatt). **Következmények:** magasabb írási késleltetés, kisebb írási elérhetőség. **Állapot:** elfogadva.
    - **ADR-2 Helyzet:** a katalógus olvasási terhelése nagy, a frissítések ritkák. **Döntés:** aszinkron replikáció, olvasási replikákkal. **Következmények:** rövid távú stale read. **Állapot:** elfogadva.

6. **Ellenőrzési szempontok:** határozz meg mérőszámokat és teszteket:
    - Replikációs lag mérése aszinkron replikánál (pl. másodpercben, mérési ablakkal).
    - Failover teszt: szimuláld a leader kiesését, mérd a helyreállási időt és az elveszett írások számát.
    - Shard egyenletesség: mérd az egyes shardok méretét és QPS-ét, keresd a hőpontokat.
    - Konfliktuskezelés teszt: két replikán párhuzamosan hajts végre írásokat, és ellenőrizd, hogy a feloldási stratégia a várt eredményt adja.
7. **Mért és várt eredmények elkülönítése:** A dokumentumban külön szekcióban szerepeljen, hogy mely számok származnak mérésből (ha egyáltalán végeztél PoC-t) és melyek becslések vagy elvárások. Például: "Várható írási késleltetés szinkron replikációnál: $<$ 20 ms a belső hálózaton; ezt terheléses teszttel kell igazolni." A nem futtatott mérést ne tüntesd fel ellenőrzött eredményként.

A labor célja nem a tökéletes megoldás, hanem a döntési folyamat dokumentálása és az ellentmondások feltárása. A terv akkor jó, ha egy másik architekt megérti belőle a választások okát, és képes a tervet felülvizsgálni.

16.2.9 Tipikus hibák és elkerülésük

- **Minden adatra erős konzisztencia kényszerítése:** ez szükségtelenül magas késleltetést és költséget okoz, miközben a legtöbb adat eventual consistent is működhet. Mindig az üzleti igényből indulj ki.
- **A replikációs lag figyelmen kívül hagyása:** ha az alkalmazás aszinkron replikáról olvas, de úgy ír, mintha erős konzisztenciát kapna, a felhasználó elavult adatot láthat. Használj read-your-writes garanciát, vagy irányítsd az olvasást a leaderhez, ha friss adat kell.
- **Rossz shard key választás:** ha a shard key nem egyenletes eloszlású, hőpontok alakulnak ki, és a skálázás kudarcot vall. Elemezd a valós terhelési mintákat, és teszteld a particionálást szintetikus adatokkal.
- **Nem idempotens műveletek:** elosztott rendszerben az újrapróbálkozás mindennapos. Ha egy rendelés rögzítése nem idempotens, a hálózati hiba utáni ismétlés duplikált rendelést hozhat létre.
- **Adatvesztés bagatellizálása aszinkron replikációval:** gyakori hiba, hogy a fejlesztők azt hiszik, a replika "majdnem" naprakész, ezért a failover során nem veszik észre az elvesztett írásokat. Mindig mérd a replikációs lagot, és teszteld a legrosszabb esetet.
- **Nem tesztelt failover:** a failover eljárást nem elég megtervezni, rendszeresen gyakorolni kell. A nem tesztelt failover gyakran hosszabb kiesést okoz, mint maga a hiba.

16.2.10 Összefoglalás

A replikáció és a sharding az elosztott adatarchitektúra két alapvető eszköze. A replikáció a rendelkezésre állást és az olvasási skálázást szolgálja, de konzisztencia és adatvesztési kérdéseket vet fel. A sharding az írási és tárolási kapacitást növeli, de a shard key megválasztása és a cross-shard műveletek kezelése kulcsfontosságú. A kettőt jellemzően együtt használjuk: minden shard replikált.

A tervezés során mindig az üzleti követelményekből indulj ki: mely adatok tűrik az átmeneti inkonzisztenciát, melyek igényelnek erős garanciákat? A döntéseket ADR-ben rögzítsd, és a rendszer viselkedését folyamatosan mérd. A laborban készített terv célja, hogy ezt a folyamatot gyakorold, és a döntések következményeit már a tervezés szakaszában felismerd.

16.2.11 Ellenőrzőlista

- ☐ Meghatároztad, mely adatok igényelnek szinkron replikációt, és melyek működhetnek aszinkronnal.
- ☐ A replikációs faktor összhangban van a rendelkezésre állási célokkal.
- ☐ Kiválasztottad a shard key-t, és elemezted az egyenletességét és a hőpontkockázatot.
- ☐ A leggyakoribb lekérdezések egyetlen shardot érintenek, vagy tudatosan vállaltad a cross-shard költséget.
- ☐ Konfliktusfeloldási stratégiát határoztál meg, és az alkalmazás kezeli a konfliktusokat.
- ☐ A replikációs lagot méred, és az olvasási útvonalak figyelembe veszik a lehetséges stale read-et.
- ☐ A failover eljárást tesztelted, és az adatvesztési kockázatot dokumentáltad.
- ☐ Az írások idempotensek, vagy az újrapróbálkozás kezelése megoldott.

16.2.12 Önellenőrző kérdés

Adott egy rendelési rendszer, ahol a rendelések rögzítése kritikus, adatvesztés nem megengedett, az ügyféltörzs olvasása nagy forgalmú, de a frissítések ritkák. Milyen replikációs stratégiát választanál a rendelések és az ügyféltörzs számára, milyen shard key-t használnál a rendeléseknél, és milyen konzisztencia-garanciát nyújtanál az ügyféltörzs olvasásakor? Válaszodat indokold a fejezetben tárgyalt szempontok alapján.

16.3 Data lake, lakehouse és analitikai architektúra

Az analitikai adatarchitektúra egyik leggyakoribb buktatója, hogy a csapat az első terhelés után már „késznek” tekinti a rendszert, pedig az adatok életciklusa, a sémafejlődés és a fogyasztói igények gyorsan felülírják a kezdeti feltételezéseket. Ebben a fejezetben a nyers, a finomított és a kiszolgáló adatréteg fogalmát járjuk körbe, majd azt vizsgáljuk, hogyan illeszkedik ebbe a batch, a streaming és az ELT feldolgozás, végül megkülönböztetjük a lakehouse-t a hagyományos adattárháztól. Az elméleti részt egy labor követi, amelyben analitikai adatfolyamot tervezünk, döntési naplóval és ellenőrzési szempontokkal.

16.3.1 A három adatréteg: miért érdemes szétválasztani?

Az analitikai rendszerekben az adat útja ritkán egyenes vonal az adatforrástól a riportig. A gyakorlatban szükség van egy olyan tartós, visszakereshető, könnyen újrafelhasználható nyers tárolóra, amely az adatforrás pillanatnyi állapotát őrzi, és szükség van olyan feldolgozott formákra, amelyeket a fogyasztók – riportok, gépi tanuló modellek, operatív döntéstámogató rendszerek – közvetlenül használnak. A rétegek szétválasztása nem öncélú formalitás, hanem a minőség, az újrafelhasználhatóság és a hibakeresés alapfeltétele.

A leggyakoribb elnevezés a medallion architektúra: bronz (nyers), ezüst (finomított) és arany (kiszolgáló) réteg. A nevek származhatnak a Databricks lakehouse-gyakorlatából, de a mögöttes elv technológiától függetlenül érvényes: minden rétegnek más a felelőssége, a séma szigorúsága és a fogyasztója.

16.3.2 Nyers réteg (bronz)

A nyers réteg az adatforrásokból érkező adatok első tartós másolata. A cél az, hogy az architektúra minden későbbi feldolgozási lépése visszavezethető legyen egy olyan változathoz, amely még nem tartalmaz alkalmazásszintű transzformációt. A nyers réteg jellemzően:

- **immutábilis**: a beérkezett rekordokat nem frissítjük, nem töröljük, csak olvassuk;
- **forrásformátum közeli**: megtartja a forrásrendszer sémáját, kódolását, és gyakran a nyers üzenetet is;
- **append-only**: új adatokat hozzáfűzünk, a régiek megmaradnak;
- **időbélyegzett**: a beérkezési időt és gyakran a forrásbeli eseményidőt is tároljuk.

A nyers rétegben a séma lehet laza – például JSON, Avro vagy Parquet fájlok, amelyeket csak a beérkezési idő szerint particionálunk –, de ez nem jelenti azt, hogy a tartalom nincs megőrizve. Éppen ellenkezőleg: a nyers réteg a későbbi újrafeldolgozás alapja, ezért fontos a tartósság és a replikáció. A gyakorlatban sokan itt követik el azt a hibát, hogy a nyers réteget átmeneti tárolóként kezelik, és törlik a régi adatokat, mielőtt azok értéke érvényesülhetne. A jó irányelv az, hogy a nyers réteg megtartási idejét üzleti, jogi és költség szempontok együttesen határozzák meg, nem pedig a lemezterület aktuális ára.

A nyers rétegnek nem kell katalógus nélkül maradnia. A beérkező adatokról metaadatot tárolunk: forrás, séma, beérkezési idő, sorok száma, méret, ellenőrzőösszeg. Ez később a finomított rétegben segít az adatminőség-ellenőrzésben és a visszakeresésben. A metaadatok nélküli nyers tároló hamar „adattóvá” válik: senki sem tudja, mi van benne, ezért senki sem meri használni.

16.3.3 Finomított réteg (ezüst)

A finomított réteg az a hely, ahol a nyers adatból konzisztens, validált, elemzésre alkalmas adathalmaz lesz. A jellemző feldolgozási lépések:

- **Tisztítás**: hiányzó értékek, duplikátumok, hibás típusok kezelése.
- **Normalizálás**: közös sémára hozás, konvenciók alkalmazása (például ISO 8601 időformátum, egységes pénznem, kódolás).
- **Validálás**: séma- és üzleti szabályok futtatása; a hibás rekordokat karanténba tesszük.
- **Kiegészítés**: idegen kulcsok feloldása, dimenziók összekapcsolása, lassan változó dimenziók kezelése.
- **Deduplikáció**: üzleti kulcsok alapján egyedi rekordok kialakítása, ha az elemzés ezt igényli.

A finomított réteg séma szigorúbb, mint a nyersé. A séma itt már a fogyasztói igényekhez közelít, de még nem feltétlenül tartalmaz aggregátumokat, hanem egy jól strukturált, tiszta, konzisztens adatmodellt biztosít. Ez a réteg gyakran a „single source of truth” az elemzés számára, de fontos különbséget tenni: a finomított réteg nem feltétlenül az üzleti riportok közvetlen forrása. Ha csak a finomított réteget tartanánk meg, akkor minden aggregálást újra és újra el kellene végezni, ami lassú és költséges lenne.

A finomított rétegben a sémafejlődés kritikus. Az adatforrások változnak, új mezők jelennek meg, a típusok módosulhatnak. A jól megtervezett finomított réteg támogatja a sémaevolúciót, például új oszlop hozzáadását anélkül, hogy a meglévő adatokat újra kellene írni. Az olyan nyílt táblaformátumok, mint a Delta Lake, az Apache Iceberg vagy a Hudi, ezt a képességet ACID tranzakcióval és időutazással együtt nyújtják, amire a lakehouse-ról szóló szakaszban visszatérünk.

16.3.4 Kiszolgáló réteg (arany)

A kiszolgáló réteg az üzleti fogyasztók felé néző, rendszerint erősen denormalizált, aggregált vagy speciálisan formázott adatokat tartalmazó gyűjtemény. Ide tartoznak:

- **aggregált riportok**: napi, havi, termékkategóriánkénti összesítések;
- **data martok**: egy-egy üzleti terület (például finance, marketing) saját nézetét kiszolgáló táblák;
- **ML feature store-ok**: a gépi tanuló modellek számára képzett jellemzők;
- **API-n keresztül kiszolgált adatkészletek**: a termékélményt közvetlenül támogató, gyakran alacsony késleltetésű olvasások.

A kiszolgáló rétegben a teljesítmény és a fogyasztói élmény az elsődleges szempont. A táblák gyakran előre aggregáltak, erősen indexeltek, esetleg oszlopos formátumban tároltak, és a lekérdezések minimalizálására tervezettek. A kiszolgáló réteg adatai származtatott adatok; a mögöttes igazság a nyers és a finomított rétegből rekonstruálható. Ezért a kiszolgáló réteget elvileg bármikor újra lehet építeni a finomított rétegből, ha a fogyasztói igények változnak.

A három réteg közötti határok nem merevek. Egy kis csapatnál lehet, hogy a kiszolgáló réteg közvetlenül a nyersből táplálkozik, ha a feldolgozási lépések száma minimális. A rétegzés lényege a felelősség elkülönítése, nem a rétegek számának fetisizálása. Ha három rétegnél kevesebb is teljesíti a követelményeket, akkor nem kell erőltetni a medallion neveket. A követelmények változásával azonban a nyers–finomított–kiszolgáló felosztás szinte mindig megkönnyíti a továbbfejlesztést.

16.3.5 Batch, streaming és ELT

Az analitikai feldolgozás időzítése alapvetően befolyásolja az architektúrát. A klasszikus **batch** feldolgozás periodikus – például óránkénti, napi vagy heti – adagokban dolgozza fel az adatokat. A **streaming** feldolgozás ezzel szemben az események érkezéskor vagy kis ablakokban, folyamatosan transzformálja az adatot. A két modell nem kizárja egymást, hanem más-más késleltetési, pontossági és költségprofillal bír.

A batch feldolgozás előnye, hogy determinisztikus: ugyanazon a bemeneti adathalmazon futtatva az eredmény reprodukálható, a feldolgozási költség pedig jól tervezhető. Hátránya, hogy az adatfrissesség – vagyis az adatforrásban bekövetkező változás és a fogyasztói rétegben való megjelenés közötti késleltetés – nagy lehet. A batch feldolgozás a legtöbb éjszakai riportnál, pénzügyi zárásnál és modell-újratanításnál megfelelő.

A streaming feldolgozás alacsony késleltetést ad, de a pontosság garantálása nehezebb: a későn érkező események, az újraindítási szemantika, a deduplikáció és a stateful aggregáció mind olyan kihívás, amelyet a batch modellben egyszerűbb kezelni. A streaming architektúrákban gyakran használják a **watermark** fogalmát: az a pillanat, amely után a rendszer nem vár további eseményeket egy adott időablakra. A watermark paraméterezése közvetlen trade-off a teljesség és a késleltetés között.

A gyakorlatban a kettő kombinációja terjedt el. Az egyik elterjedt minta, hogy a nyers réteget streaming módon töltik fel – az adatforrásokból egy eseményfolyamra kerülnek a rekordok, onnan pedig a nyers tárolóba –, majd a finomított és kiszolgáló rétegeket batch módon, ütemezetten frissítik. Ez megőrzi a nyers adatok frissességét, miközben a költséges transzformációkat mégis batchben futtatja. Ha a fogyasztói igény valós idejű riportot is megkíván, akkor egy párhuzamos streaming ágat lehet hozzáadni, amely a nyers rétegből közvetlenül frissít egy kis, konzisztensen tartott kiszolgáló táblát.

Az **ETL** (Extract, Transform, Load) és az **ELT** (Extract, Load, Transform) közötti különbség nem pusztán a betűrend, hanem architekturális döntés. Az ETL-modellben a transzformáció a forrásból való kinyerés és a céltárolóba való betöltés **között** történik, gyakran egy külön feldolgozó motoron. Az ELT-modellben először betöltjük a nyers adatot a céltárolóba – tipikusan egy cloud data lake-be vagy lakehouse-ba –, majd a transzformációt magában a tárolóban vagy a hozzá kapcsolódó lekérdezőmotorban, SQL segítségével futtatjuk.

Az ELT előnye, hogy kihasználja a modern felhőalapú tárolók rugalmasságát és a lekérdezőmotorok horizontális skálázhatóságát; nincs szükség külön ETL-szerverre, a transzformációs logika pedig verziózható SQL-szkriptekként. Hátránya, hogy a transzformációs hibák csak a betöltés után derülnek ki, így a nyers rétegben megjelenhetnek olyan adatok, amelyeket utólag kell javítani. Az ETL ezzel szemben korán elkaphatja a hibákat, de drágábban skálázható, és ha a célrendszer séma gyakran változik, a fejlesztési ciklus lassabb.

A választás nem hitkérdés. Ha a forrásadatok heterogének, a célrendszer pedig egy erős lekérdezőmotorral rendelkező lakehouse, az ELT rendszerint olcsóbb és rugalmasabb. Ha a forrásadatok érzékenyek, és a célrendszer nem támogatja a hatékony transzformációt, akkor az ETL-nek nagyobb értelme van. A hibrid modellek is léteznek: a nagyon érzékeny mezőket a forrás közelében anonimizáljuk (ETL-jellegű lépés), a többit pedig nyersen betöltjük (ELT).

Az adatfolyam kapacitástervezésénél a Little-törvény jól használható:

$$L = \lambda W$$

ahol L a feldolgozórendszerben tartózkodó rekordok vagy kérések átlagos száma, λ az érkezési ráta (például esemény/másodperc), W pedig az átlagos tartózkodási idő. Ha egy streaming pipeline-nál $\lambda = 1000$ esemény/másodperc és a feldolgozási késleltetés $W = 5$ másodperc, akkor átlagosan $L = 5000$ esemény van a rendszerben, ami a stateful feldolgozó memóriaméretének tervezéséhez is támpontot ad. A batch feldolgozásnál ugyanez a képlet alkalmas arra, hogy a feldolgozási ablakot és az átlagos batch méretet összekapcsoljuk.

16.3.6 Lakehouse és hagyományos adattárház

A hagyományos adattárház általában egy külön, drága, erősen strukturált relációs rendszer, amelybe a forrásadatok ETL-folyamaton keresztül, előfeldolgozva érkeznek. Az adattárház sémája szigorú, az adatok jellemzően csillag- vagy hópehely-sémába szervezettek, a lekérdezések pedig erősen optimalizáltak. Az adattárház kiválóan alkalmas az üzleti riportok, dashboardok és vezetői információs rendszerek kiszolgálására, de korlátai is vannak: a betöltés előtt meg kell határozni a sémát, a félig strukturált adatok kezelése nehézkes, a számítási és tárolási kapacitás összekapcsoltsága pedig drágává teszi a nagy adatmennyiségek tárolását.

A **data lake** ezzel szemben egy olcsó, skálázható, gyakran felhőalapú objektumtároló (például S3-kompatibilis rendszer), amely bármilyen formátumú adatot tárolhat: strukturált, félig strukturált, nyers bináris. A data lake előnye a rugalmasság: nem kell előre sémát tervezni, a nyers adatokat olcsón megtarthatjuk, a feldolgozó motorokat pedig elválaszthatjuk a tárolástól. A hátrány az, hogy a klasszikus data lake-ből hiányoznak a relációs adatbázisok által nyújtott garanciák: nincs ACID tranzakció, nincs sémaérvényesítés, a konkurens írás–olvasás inkonzisztenciát okozhat. Ennek eredménye a híres „adattó” probléma: a tó tele van adattal, de senki sem tud hasznosítani belőle.

A **lakehouse** a kettő előnyeit próbálja ötvözni. A koncepció lényege, hogy a data lake tárolója marad az alap, de arra egy olyan réteg kerül, amely ACID tranzakciókat, sémaevolúciót, időutazást és hatékony upsert/delete műveleteket biztosít. Ezt a réteget gyakran valamilyen nyílt táblaformátum valósítja meg:

- **Delta Lake**: a Databricks által indított, a Parquet fájlformátumra épülő tranzakciós réteg, amely transaction log segítségével biztosít ACID tulajdonságokat, sémaérvényesítést és time travelt.
- **Apache Iceberg**: táblaspecifikáció, amely rugalmas sémakezelést, rejtett particionálást és hatékony metaadat-kezelést kínál; több motor (Spark, Flink, Trino) is támogatja.
- **Apache Hudi**: elsősorban a streaming betöltésre és az inkrementális feldolgozásra optimalizált, támogatja az upsert műveleteket és a rekord-szintű frissítést.

Ezek a formátumok nem maguk az adatok, hanem metaadat- és fájlelrendezési protokollok, amelyek lehetővé teszik, hogy a data lake-ben tárolt Parquet fájlok relációs táblaként viselkedjenek. A lakehouse tehát nem egy konkrét termék, hanem egy architektúra-minta, amelyben a tárolás és a számítás szétválasztható, a séma pedig az adattal együtt evolválódhat.

A lakehouse és a hagyományos adattárház összehasonlítása:

Szempont	Hagyományos adattárház	Lakehouse
Tároló	Dedikált, gyakran drága, zárt rendszer	Olcsó objektumtároló, nyílt formátum
Séma	Írás előtt kötelező (schema-on-write)	Séma az írásnál lazább, olvasásnál kötelező (schema-on-read)
ACID tranzakció	Igen, érett	Igen, nyílt táblaformátumok révén
Sémaevolúció	Korlátozott, gyakran migráció szükséges	Jobb támogatás, additív séma könnyű
Félig strukturált adat	Nehézkes	Természetes
Skálázhatóság	Vertikálisan vagy korlátozottan horizontálisan	Horizontálisan, felhő-native módon
Költség	Magas, különösen nagy adatmennyiségnél	Alacsonyabb, tárolás és számítás szétválasztható
Érettség	Több évtizedes, stabil	Gyorsan fejlődik, bizonyos motorok fiatalabbak

A választás nem mindig lakehouse. Ha a szervezet már rendelkezik jól bevált, szigorúan szabályozott adattárházzal, és az adatmennyiség nem robbanásszerű, akkor a lakehouse bevezetése felesleges komplexitást okozhat. Ha viszont nagy mennyiségű félig strukturált adat, gyakori sémaváltás vagy gépi tanulási munkafolyamat van a képben, a lakehouse jelentős előnyt adhat.

16.3.7 Gyakorlati minta: kiskereskedelmi analitikai adatfolyam

Képzeljünk el egy e-kereskedelmi platformot, amely rendelési, kattintási és terméktörzs adatokat gyűjt. A cél: napi értékesítési riportok készítése, valamint egy gépi tanuló modell futtatása az ajánlórendszerhez. Az architektúra a következő módon tagolódik:

- **Nyers réteg**: a beérkező rendelési eseményeket, kattintási streamet és terméktörzsváltozásokat az objektumtárolóba írjuk, forrás szerint particionálva, eredeti formátumban (JSON, Parquet, CSV).
- **Finomított réteg**: a rendelési eseményeket megtisztítjuk, a valutát egységesítjük, a termékazonosítókat feloldjuk a törzsadatokból, és a kattintási eseményekre session-indentifikációt alkalmazunk. Itt már szigorú séma van, a hibás rekordok karanténba kerülnek.
- **Kiszolgáló réteg**: napi aggregált értékesítési táblák (termék, kategória, ország szerint), valamint egy ML feature tábla, amely a modell tréningjéhez és online lekérdezéséhez szükséges jellemzőket tartalmazza.

A feldolgozás napi batchben fut a finomított és kiszolgáló réteg esetén, de a nyers réteg folyamatosan kapja az adatokat. A streaming ág annyiban jelenik meg, hogy a kattintási eseményekből egy kis, csúszóablakos aggregáció frissíti a „jelenleg népszerű termékek” kiszolgáló táblát, amelyet az ajánlórendszer valós időben olvas.

Az adatfolyam diagrams-as-code formában, Mermaid segítségével:

```
flowchart LR
    subgraph Források
        A[Rendelési események]
        B[Kattintási események]
        C[Termék törzsadatok]
    end

    subgraph Nyers réteg (bronz)
        D[(Objektumtároló / nyers)]
    end

    subgraph Finomított réteg (ezüst)
        E[(Tisztított, validált adathalmaz)]
    end

    subgraph Kiszolgáló réteg (arany)
        F[(Aggregált riportok / data mart)]
        G[(ML feature store)]
        H[(Valós idejű top termékek)]
    end

    A --> D
    B --> D
    C --> D
    D --> E
    E --> F
    E --> G
    D --> H
    F --> I[BI dashboard]
    G --> J[Modell tréning]
    H --> K[Online ajánló]
```

Diagram címe: Analitikai adatfolyam rétegenként. **Jelmagyarázat:** a nyilak az adatáramlás irányát mutatják; a téglalapok feldolgozó komponensek, a hengerek tartós tárolók. A szaggatott vonal a rendszerhatárt nem jelöli itt, ezért a C4 context szintű diagramot a laborban el kell készíteni. **Feltételjegyzet:** a diagram a logikai adatáramlást mutatja; az ütemezési gyakoriság és a technológiai választás a döntési naplóban rögzítendő.

A minőségi követelményeket quality attribute scenario formában fogalmazzuk meg. Például a frissességre:

- **Inger:** egy új rendelési esemény beérkezik.

- **Környezet:** normál üzem, átlagosan 1000 esemény/perc.
- **Válasz:** a finomított rétegben a rekord 15 percen belül megjelenik, a napi aggregátum másnap reggel 6 óráig elkészül.
- **Mérték:** a finomított réteg frissességi késleltetésének 95. percentilise, 15 perces mérési ablakban mérve, $D_{\text{friss}} = t_{\text{utolsó feldolgozott esemény}} - t_{\text{jelen}}$.

A döntéseket ADR rögzíti. Például:

ADR-2025-04-14-01: Nyers réteg formátuma

- **Kontextus:** a bejövő rendelési események JSON formátumban, kattintási események Avro-ban érkeznek; a cél a gyors betöltés és a későbbi újrafeldolgozhatóság.
- **Döntés:** a nyers rétegben a forrás eredeti formátumát tartjuk meg, a tömörítést a tároló szintjén oldjuk meg, és a beérkezési idő szerint particionálunk.
- **Alternatívák:** azonnali Parquet konverzió (gyorsabb későbbi olvasás, de lassabb betöltés és séma-függőség); deduplikált, szabványosított JSON (kevesebb tárhely, de adatvesztés kockázata).
- **Következmények:** a nyers réteg olvasásakor a séma feloldása az olvasáskor történik, ami rugalmas, de a finomított rétegbe töltésnél sémaérvényesítési lépést iktatunk be.
- **Állapot:** elfogadva.

Ebben a példában az architektúra az ELT mintát követi: a nyers réteget először feltöltjük, majd a finomított és kiszolgáló réteg transzformációit SQL-ben futtatjuk a lakehouse lekérdezőmotorján. Ezzel a transzformációs logika verziózható, és a kiszolgáló réteg újraépíthető a finomítottból.

16.3.8 Labor: analitikai adatfolyam tervezése

A labor célja, hogy a tanult fogalmakat egy kis, ellenőrizhető példán alkalmazza. A feladat nem kódolás, hanem tervezési dokumentum készítése.

16.3.9 Kiinduló állapot

Adott egy fiktív logisztikai vállalat, amely telemetriai eseményeket gyűjt a fuvarozó járművekről (GPS, hőmérséklet, sebesség, üzemanyag-szint), valamint rendelkezik a rendeléseket és a járműveket leíró törzsadatokkal. Az adatmennyiség napi 200 millió esemény, az adatok percenként érkeznek. A cél: napi flotta riportok, késés-előrejelző ML modell, és egy operatív dashboard, amely az elmúlt 60 percben mutatja a hőmérsékleti riasztásokat.

16.3.10 Elvárt eredmény

Készítsen egy reprodukálható dokumentumot, amely tartalmazza:

1. **C4 context szintű diagramot** a rendszerhatárral és a külső szereplőkkel (forrásrendszerek, BI-fogyasztók, ML-platform).
2. **Az adatfolyam rétegekre bontott tervét** (nyers, finomított, kiszolgáló) Mermaid vagy hasonló diagrams-as-code formában.
3. **Minden réteghez** a séma leírását, a tárolási formátumot, a particionálást és a megtartási időt.
4. **Egy döntési naplót (ADR)** legalább három döntésről: a nyers formátum és tömörítés megválasztásáról, a batch/streaming feldolgozási stratégia kiválasztásáról, valamint a kiszolgáló réteg aggregációjának granularitásáról.

5. **Minőségi forgatókönyveket** a frissességre, a pontosságra és a rendelkezésre állásra.
6. **Egy mérési tervet**, amelyben a mért és a várt eredmény külön szerepel. Például: várt frissességi késleltetés a finomított rétegben 5 perc, a rendszer terhelése 1000 esemény/perc; a mért érték a tényleges tesztfuttatásból származik, amelynél rögzíteni kell a környezetet, a konfigurációt, az időpontot és a bizonytalanságot.

16.3.11 Ellenőrzési szempontok

- A nyers réteg valóban immutábilis-e, és megőrzi-e a forrásformátumot?
- A finomított rétegben vannak-e sémaérvényesítési és deduplikációs szabályok?
- A kiszolgáló réteg valóban a fogyasztói igényekhez igazodik-e, és nem csak a finomított réteg másolata?
- A batch és streaming ágak szétválasztása indokolt-e, vagy egy felesleges komplexitást hoz létre?
- A döntési naplóban szerepelnek-e az elvetett alternatívák és a következmények?
- A mért értékek megkülönböztethetők-e a feltételezésektől?

16.3.12 Tipikus hibák és helyes megoldások

Egy analitikai adatarchitektúra tervezésekor a leggyakoribb hibák közül néhányat érdemes kiemelni.

16.3.13 1. A nyers réteg elhanyagolása

Sokan a nyers réteget átmeneti tárolónak tekintik, és a beérkező adatokat azonnal transzformálják, majd törlik a nyers változatot. Ez a megközelítés rövid távon tárhelyet spórol, de ha később újra kell dolgozni az adatokat – például mert a finomított réteg sémája hibás volt, vagy az üzleti szabály megváltozott –, nem lesz mihez nyúlni. **Helyes megoldás:** a nyers réteget a legtartósabb rétegként kezeljük, megtartási idejét üzleti, jogi és költség szempontok szerint határozzuk meg, és a törlést csak katalógusba rögzített, jóváhagyott folyamat végezheti.

16.3.14 2. A finomított réteg kihagyása

Ha a csapat a nyers rétegből közvetlenül a kiszolgáló aggregációkba transzformál, az eleinte működik, de amint több fogyasztó jelenik meg, a transzformációs logika megismétlődik, a hibaforrások megszaporodnak, és a sémaevolúció fájdalmas lesz. **Helyes megoldás:** a finomított réteget tekintjük a tartós, tiszta adatmodellnek, amelyből minden fogyasztói aggregáció származtatható.

16.3.15 3. Az adattó (data swamp) kialakulása

A data lake-ből akkor lesz adattó, ha nincs adatkatalógus, nincs metaadat, nincs tulajdonos. A fájlok felhalmozódnak, senki sem tudja, mi érvényes, mi elavult. **Helyes megoldás:** minden réteghez adatkatalógus-bejegyzés tartozik, amely leírja a forrást, a sémát, a tulajdonost, a minőségi mutatókat és az utolsó frissítés időpontját. Az adatkatalógus nem feltétlenül drága rendszer; lehet egy jól karbantartott metaadat-tábla is.

16.3.16 4. A streaming mindenáron való bevezetése

A streaming modern és izgalmas, de ha a fogyasztói igény napi riport, és a késleltetés 1 óra is elfogadható, a folyamatos csővezeték felesleges komplexitást, magasabb költséget és több hibalehetőséget hoz. **Helyes megoldás:** a frissességi követelményből induljunk ki, és a batch vagy streaming választást quality attribute scenario-val igazoljuk. A Little-törvény segíthet a kapacitástervezésben.

16.3.17 5. A lakehouse és az adattárház téves azonosítása

A lakehouse nem egyszerűen „olcsóbb adattárház". Ha a szervezetnek szigorú, jogszabályilag auditált, harmadik féltől származó adattárháza van, akkor a lakehouse-ra való átállás nem automatikus nyereség. A félreértés abból fakad, hogy mindkettő képes SQL lekérdezésekre, de az eltérő tárolási, séma- és ACID-garanciák eltérő üzemeltetési következményekkel járnak. **Helyes megoldás:** minden esetben a konkrét követelményeket és a kivételeket dokumentáljuk, és ne a divat, hanem a trade-off elemzés diktálja a választást.

16.3.18 6. A kiszolgáló réteg túltervezése

A kiszolgáló rétegben könnyű minden lehetséges riportot előre aggregálni, ami rengeteg táblát, növekvő frissítési időt és magasabb költséget eredményez. **Helyes megoldás:** először a tényleges fogyasztói elvárásokat gyűjtsük össze, mértékekkel és hozzáférési mintákkal; a kiszolgáló réteget úgy alakítsuk ki, hogy a leggyakoribb lekérdezések néhány másodpercen belül választ kapjanak, a ritkább, mély elemzések pedig a finomított rétegből futhassanak.

16.3.19 Ellenőrzőlista

- ☐ A nyers réteg immutábilis, forrásformátum közeli, és megtartási ideje dokumentált.
- ☐ A finomított réteg sémaérvényesítést, deduplikációt és karantént kezel.
- ☐ A kiszolgáló réteg a fogyasztói igényekhez igazodik, és elvileg újraépíthető a finomítottból.
- ☐ A batch/streaming/ELT választás minőségi forgatókönyvvel és költségbecsléssel indokolt.
- ☐ Az adatkatalógus minden rétegre tartalmaz forrást, sémát, tulajdonost és frissítési időt.
- ☐ Az architektúradöntések ADR-ben rögzítve vannak, alternatívákkal és következményekkel.
- ☐ A diagram diagrams-as-code formában, verziózhatóan készült.
- ☐ A mérési terv megkülönbözteti a mért és a várt eredményt.
- ☐ A hivatkozott technológiák (például nyílt táblaformátumok) verziószáma és érettsége ellenőrizve van.
- ☐ A destruktív műveletek (például adattörlés) csak izolált környezetben, visszaállítási tervvel szerepelnek.

16.3.20 Önellenőrző kérdés

Egy csapat a nyers rétegben Parquet-fájlokat tárol, és azonnal tömöríti őket. A finomított réteget naponta felülírják teljes újraépítéssel. A kiszolgáló rétegben 40 aggregált táblát tartanak karban, amelyeket egyetlen éjszakai batch frissít. Melyek ennek a felépítésnek a kockázatai, és milyen rétegbeli változtatásokat javasolna?

16.3.21 Döntési gyakorlat

Tervezzen meg egy analitikai adatfolyamot egy egészségügyi telemetriai rendszer számára, ahol a betegmonitorok percenként küldenek életjeleket, a cél a napi riport és a kritikus riasztások 5 percen belüli megjelenítése. Készítse el a rétegek leírását, válassza ki a batch/streaming stratégiát, és írjon egy ADR-t a kiszolgáló rétegben a riasztási adatfolyam elhelyezéséről.

A fejezetben bemutatott rétegfelosztás és a lakehouse-minta adja az alapot a következő egységhez, amely az adatminőséget, az adatgovernance-t és az adatkatalógust tárgyalja. Az itt szerzett döntési keret ott kiegészül az adatgazdai felelősségi körökkel, a szabályozási megfeleléssel és a minőségi mutatók folyamatos mérésével.

16.4 Adatminőség, adatgovernance és adatkatalógus

Az adatot gyakran „az új olajként” emlegetik, de ez a hasonlat félrevezető: a nyers adat önmagában ritkán értékes, és – a nyersolajhoz hasonlóan – kezelés nélkül akár veszélyes is lehet. Az adat értékét az határozza meg, hogy mennyire megbízhatóan, milyen gyorsan és milyen szabályok mentén tudjuk döntésekhez, folyamatokhoz és termékekhez felhasználni. Az adatminőség, az adatgovernance és az adatkatalógus hármasa ezt a megbízhatóságot és felhasználhatóságot teremti meg. Ebben a fejezetben ezeket a fogalmakat járjuk körül, különös tekintettel az adatra mint vállalati eszközre és mint kockázatra, az adatgazda, adatszerződés és adatvonal szerepére, valamint a személyes adatok kezelésére. A fejezet végére képes leszel egy kis, ellenőrizhető példában alkalmazni az eszközöket, és felismered azokat a tipikus hibákat, amelyek a tisztázatlan felelősségekből, a hiányzó szerződésekből és a nem megfelelő megőrzési gyakorlatból fakadnak.

16.4.1 Az adat mint vállalati eszköz és mint kockázat

Egy e-kereskedelmi platform naponta több százezer rendelési eseményt rögzít: ügyfélazonosítót, termékkódokat, mennyiségeket, árakat, szállítási címeket, időbélyegeket. Ha ezek az adatok pontosak, időszerűek és jól dokumentáltak, a vállalat képes lehet valós idejű készletoptimalizálásra, személyre szabott ajánlásokra, csalásfelismerésre vagy a szállítási késések előrejelzésére. Ugyanez az adathalmaz azonban komoly kockázatot is jelent: ha a személyes adatok védelme nem megfelelő, a vállalat GDPR-bírságot kockáztat; ha a rendelési adatokban hibás a termékár vagy a mennyiség, a pénzügyi jelentés téves lesz; ha az adatot nem tudják megbízhatóan értelmezni a fogyasztó csapatok, a döntések késnek vagy rosszak lesznek. Az adat tehát egyszerre eszköz és kockázat – a kettő nem választható el egymástól. A cél nem az adatgyűjtés minimalizálása önmagában, hanem az, hogy a gyűjtött adat értékét a lehető legnagyobbra növeljük, miközben a kockázatát a szabályozói és belső követelményeknek megfelelően kezeljük.

Ebből a kettős természetből következik, hogy az adatot ugyanúgy kell menedzselni, mint egy másik vállalati erőforrást: szükség van gazdára, megállapodásokra, minőségmérésre, katalógusra és életciklus-szabályokra. A következő szakaszokban ezeket az elemeket vezetjük be.

16.4.2 Adatgazda, adatszerződés és adatvonal

Az adatkezelés egyik leggyakoribb hibája, hogy senki sem érzi magát felelősnek az adat minőségéért, értelmezéséért vagy elérhetőségéért. A *felelősség homálya* (accountability gap) akkor jelentkezik, amikor egy adatterméknek – például a rendelési adatoknak – sok fogyasztója van, de nincs egyértelmű tulajdonosa. Ezt a problémát az adatgazda (data owner) szerepkör bevezetése oldja fel.

16.4.3 Adatgazda

Az adatgazda az a személy vagy csapat, aki üzleti szempontból felelős egy adott adathalmazért. Az adatgazda nem feltétlenül az adatot előállító rendszer üzemeltetője, hanem az, aki tudja, hogy az adat mit jelent, milyen folyamatokban keletkezik, és milyen üzleti célokat szolgál. Például egy rendelési adathalmaz adatgazdája lehet az értékesítési műveletekért felelős vezető, nem pedig a backend fejlesztőcsapat. Az adatgazda feladatai közé tartozik:

- az adathalmaz üzleti definíciójának és szemantikájának rögzítése;
- az adatminőségi követelmények meghatározása és elfogadtatása;
- a hozzáférési jogosultságokra vonatkozó döntések meghozatala;
- az adatmegőrzési és törlési szabályok jóváhagyása;
- az adathoz kapcsolódó kockázatok nyomon követése.

A gyakorlatban az adatgazdát egy *adatgondnok* (data steward) támogathatja, aki a napi operatív feladatokat végzi: validálja a metaadatokat, ellenőrzi a minőségi mutatókat, és kapcsolatot tart a fogyasztó csapatokkal. A *adatfelügyelő* (data custodian) ezzel szemben a technikai tárolásért, biztonsági mentésért és a rendelkezésre állásért felel – ez jellemzően az adatplatform-csapat.

16.4.4 Adatszerződés

Ha egy adathalmazt több csapat fogyaszt, az adatgazda és a fogyasztók közötti elvárásokat írásban kell rögzíteni. Erre szolgál az *adatszerződés* (data contract). Az adatszerződés hasonló az API-szerződéshez: tartalmazza az adathalmaz sémáját, szemantikáját, frissítési gyakoriságát, minőségi elvárásait (SLO-szerű megfogalmazásban), valamint a kapcsolattartási és hibabejelentési csatornákat. Az adatszerződés nem technikai eszköz, hanem a fogyasztó és a termelő közötti megállapodás, amelyet azonban technikai eszközökkel – például séma-validációval, minőségi tesztekkel, monitorozással – is kikényszeríthetünk.

Egy példa adatszerződés-részlet egy „orders” nevű táblához:

```
data_contract:
  name: orders
  version: 1.2.0
  owner: sales-operations-team
  steward: data-quality-team
  description: "Véglegesített rendelések napi pillanatképe, szűrve a törölt kosarakra."
  schema:
    fields:
      - name: order_id
        type: string
        nullable: false
        description: "Rendelés egyedi azonosítója, format: ORD-YYYYMMDD-XXXXXX"
        pii: false
      - name: customer_id
        type: string
```

```
      nullable: false
      description: "Ügyfél belső azonosítója, álnevesített"
      pii: true
    - name: total_amount
      type: decimal(10,2)
      nullable: false
      description: "Végösszeg a pénznemben, áfa nélkül"
      pii: false
  slo:
    freshness: "minden nap 06:00 UTC-ig az előző napi adatok"
    completeness: ">= 99,5% a rendelés idők alapján"
    uniqueness: "order_id egyedi"
    invalid_ratio: "<= 0,5% a total_amount null vagy negatív"
  contact:
    slack_channel: "#orders-data-support"
    incident_playbook: "runbooks/orders-data-incident.md"
```

Az adatszerződésben a minőségi elvárásokat mérhető formában adjuk meg, hasonlóan a szoftverrendszerek SLO-jaihoz. A `freshness`, `completeness`, `uniqueness` itt nem pusztán szavak, hanem mérési ablakkal és küszöbértékkel rendelkeznek. A szerződés része lehet a séma-evolúciós szabály is: például „a `total_amount` mező típusa nem változhat visszamenőlegesen; új mezők hozzáadása lehetséges, de a régiek nem távolíthatók el anélkül, hogy a fogyasztókat legalább 30 nappal korábban értesítenék.”

16.4.5 Adatvonal

Az adatvonal (data lineage) az adat útját írja le a keletkezéstől a felhasználásig, beleértve a transzformációkat, összekapcsolásokat, másolásokat és aggregációkat. Az adatvonal nélkül a fogyasztók nem tudják, hogy egy riportban megjelenő szám melyik forrásrendszerből, milyen üzleti szabályokon keresztül származik. Ha hibás az adat, az adatvonal segít visszakövetni a hiba gyökerét; ha egy forrásrendszer migrál, az adatvonal megmutatja az érintett downstream fogyasztókat.

Az adatvonalat általában diagrams-as-code formában, például Mermaidben vagy strukturált metaadatokban rögzítjük. Egy egyszerű példa:

```
flowchart LR
    A[(Rendelési DB)] -->|"CDC, 5 percenként"| B[Stream: orders.raw]
    B --> C[Transzformáció: deduplikáció, szűrés]
    C --> D[(Adattárház: orders_final)]
    D --> E[Riport: napi árbevétel]
    D --> F[ML modell: kereslet-előrejelzés]
```

Az adatvonal nemcsak technikai dokumentáció, hanem az adatgovernance egyik pillére: lehetővé teszi a hatásvizsgálatot (impact analysis) és a gyökér-ok elemzést (root cause analysis). A korszerű adatplatformok gyakran automatikusan gyűjtik a lineage információt a futó pipeline-okból, de az üzleti transzformációk jelentését emberi dokumentációval kell kiegészíteni.

16.4.6 Adatminőség

Az adatminőség nem abszolút fogalom: ugyanaz az adathalmaz egy riportban kiváló, míg egy valós idejű csalásfelismerő modellben használhatatlan lehet. A minőséget mindig a felhasználási kontextusban, mérhető dimenziók mentén kell értelmezni. A leggyakrabban használt dimenziók:

- **Teljesség (completeness):** mennyire van jelen minden elvárt rekord vagy mező? Példa: a rendelések 98%-ánál a `customer_id` nem üres.

- **Pontosság (accuracy):** mennyire tükrözi az adat a valós világot? Példa: a `total_amount` mező a számlázott összeggel egyezik-e?
- **Konzisztencia (consistency):** ugyanaz az adat különböző rendszerekben nem mond-e ellent egymásnak? Példa: a `customer_id` ugyanazon ügyfelet azonosítja-e minden táblában?
- **Érvényesség (validity):** az adat megfelel-e az előírt formátumnak, értéktartománynak? Példa: az `order_date` ISO 8601 formátumú, és 2020–2030 közé esik.
- **Egyediség (uniqueness):** nincs-e duplikátum? Példa: az `order_id` értéke csak egyszer fordul elő.
- **Időszerűség (freshness / timeliness):** az adat milyen késéssel áll rendelkezésre a valós eseményhez képest? Példa: a napi riport minden reggel 7 óra előtt elérhető, a forrásból való kinyerés legfeljebb 1 órával késik.
- **Referenciális integritás (referential integrity):** az idegen kulcsok hivatkoznak-e létező rekordokra? Példa: minden rendeléshez tartozik létező `customer_id`.

Az adatminőséget nem elég utólag, manuálisan ellenőrizni; a minőségi elvárásokat a feldolgozási pipeline-ba kell építeni, hasonlóan a szoftver automatikus tesztjeihez. Ez lehet egyszerű SQL-ellenőrzés vagy dedikált adatminőségi keretrendszer (pl. Great Expectations, dbt tests), amely a feldolgozás során folyamatosan mér és riaszt. A mért eredményeket dashboardon tesszük elérhetővé, a küszöbértékek túllépése esetén pedig riasztunk. Fontos, hogy a mért és a várt eredményt mindig külön dokumentáljuk, a mérés időpontjával és kontextusával együtt. Példa:

Dátum	Adathalmaz	Dimenzó	Várt küszöb	Mért érték	Státusz	Mérési ablak
2025-03-01	orders	teljesség	99,5%	99,7%	OK	előző nap
2025-03-01	orders	időszerűség	1 óra	42 perc	OK	utolsó batch
2025-03-01	orders	érvényesség	0,5%	0,8%	HIBA	előző nap

16.4.7 Adatgovernance

Az adatgovernance az a keretrendszer, amely meghatározza a szerepeket, folyamatokat, szabályokat és döntési jogköröket az adatvagyon kezelésére. Nem egyetlen eszköz, hanem elvek, gyakorlatok és ellenőrzési pontok összessége. A cél nem a bürokrácia növelése, hanem az, hogy az adat értékét a kockázatokkal arányos módon lehessen kiaknázni. A governance hiánya az adathalmazok elburjánzásához, eltérő definíciókhoz, silókhoz és megfelelési problémákhoz vezet.

A governance alapelemei:

1. **Szerepek és felelősségek:** az adatgazdák, adatgondnokok, adatfelügyelők és fogyasztók jogainak és kötelezettségeinek rögzítése. Ez lehet RACI mátrix vagy hasonló dokumentum.
2. **Szabályok és policy-k:** az adat hozzáférésére, minőségére, megőrzésére, titkosítására, álnevesítésére és a határokon átnyúló adatáramlásra vonatkozó belső előírások.
3. **Folyamatok:** az adathalmaz létrehozásának, jóváhagyásának, közzétételének, módosításának és visszavonásának menete. Ide tartozik az adatszerződés kialakítása és a változáskezelés is.
4. **Metaadat-menedzsment:** az adathalmazok leírásának központi nyilvántartása, beleértve az üzleti (mit jelent), technikai (milyen formátum, séma) és operatív (ki, mikor, hogyan frissíti) metaadatokat.
5. **Mérés és audit:** a minőségi mutatók, a hozzáférések és a szabályok betartásának nyomon követése. Az auditálhatóság különösen fontos a szabályozói megfelelésnél.

A gyakorlatban az adatgovernance akkor működik jól, ha nem központi jóváhagyási szűk keresztmetszetként működik, hanem *irányelvek és önkiszolgáló eszközök* kombinációjaként. Például ahelyett, hogy minden új adathalmazt egy központi bizottságnak kellene jóváhagynia, a governance-csapat sablonokat, automatikus ellenőrzéseket és katalógus-integrációt biztosít, amelyekkel az adatgazdák saját maguk regisztrálhatják adathalmazaikat, miközben a megfelelőségi követelmények automatikusan érvényesülnek.

16.4.8 Adatkatalógus

Az adatkatalógus az adatvagyon *kereshető, strukturált nyilvántartása*. Célja, hogy a felhasználó – legyen az elemző, adattudós, mérnök vagy termékmenedzser – gyorsan megtalálja a munkájához szükséges adatot, megértse annak jelentését, minőségét és hozzáférési feltételeit. Az adatkatalógus nem egyszerű táblázat: élő, folyamatosan frissülő rendszer, amely a metaadatok mellett automatikusan gyűjti a séma- és lineage-információkat, minőségi mutatókat, használati statisztikákat és felhasználói értékeléseket.

Az adatkatalógus fő funkciói:

- **Felfedezhetőség:** szöveges keresés, címkézés, domain szerinti böngészés.
- **Megértés:** üzleti leírás, példaértékek, számítási logika, adatszerződés hivatkozása.
- **Megbízhatóság:** adatminőségi mutatók, frissességi jelzők, adatgazda és kapcsolattartó megjelenítése.
- **Hozzáférhetőség:** a hozzáférési kérelem folyamata, hivatkozás a megfelelő jogosultságkezelőre, minta lekérdezések.
- **Lineage és hatáselemzés:** az adat forrásainak és fogyasztóinak térképe.

Egy adatkatalógus-bejegyzés vázlata (például JSON formátumban, illusztrációként):

```
{
  "dataset": "orders",
  "version": "1.2.0",
  "domain": "értékesítés",
  "owner": "sales-operations-team",
  "steward": "data-quality-team",
  "description": "Véglegesített rendelések napi pillanatképe.",
  "tags": ["rendelés", "pénzügy", "napi"],
  "schema": {
    "fields": [
      {"name": "order_id", "type": "string", "pii": false},
      {"name": "customer_id", "type": "string", "pii": true},
      {"name": "total_amount", "type": "decimal(10,2)", "pii": false}
    ]
  },
  "quality": {
    "freshness": "minden nap 06:00 UTC",
    "completeness": ">= 99,5%",
    "last_checked": "2025-03-01T08:30:00Z",
    "status": "OK"
  },
  "access": {
    "request_process": "https://example.internal/access-request?dataset=orders",
    "access_level": "internal"
  },
  "lineage": {
    "source": "Rendelési DB (CDC)",
    "transforms": ["deduplikáció", "szűrés a törölt kosarakra"],
    "consumers": ["napi árbevétel riport", "kereslet-előrejelző modell"]
```

```
  }
}
```

A fenti példa csak vázlat; valódi rendszerben a katalógus-bejegyzés egy adatkatalógus-eszközben (pl. DataHub, Amundsen, OpenMetadata) él, és a mezők nagy része automatikusan töltődik. A könyv példái nem tartalmaznak valós belső URL-t; az `example.internal` helyőrző.

Fontos, hogy az adatkatalógus ne váljon elavulttá. Ha a katalógus leíró adatait kézzel kell frissíteni, az idővel pontatlanná válik, és a felhasználók elvesztik a bizalmukat. A megoldás az automatizálás: a sémaváltozásokat, a lineage-t és a minőségi mutatókat a pipeline futása közben gyűjtjük, a katalógus pedig ezeket folyamatosan szinkronizálja. Az üzleti leírásokat és a gazda hozzárendelését továbbra is ember tartja karban, de ezek változását is verziókövetéssel és jóváhagyási folyamattal érdemes kezelni.

16.4.9 Személyes adat és megőrzési szabály

A személyes adatok kezelése az adatgovernance egyik legmagasabb kockázatú területe. Az Európai Unióban a GDPR (General Data Protection Regulation) határozza meg a személyes adatok kezelésének alapelveit, de hasonló szabályozások más joghatóságokban is léteznek. Ez a könyv nem nyújt jogi tanácsot; a konkrét jogi követelményeket mindig a hivatalos szöveg és jogi szakértő alapján kell ellenőrizni. Az alábbiakban azokat az általános elveket és gyakorlatokat mutatjuk be, amelyek az architektúra szempontjából meghatározóak.

A GDPR alapelvei közül az adatarchitektúrára közvetlen hatással van:

- **Célhoz kötöttség:** a személyes adatot csak meghatározott, egyértelmű és jogszerű célból szabad gyűjteni, és attól eltérő célra nem használható. Ez azt jelenti, hogy egy adathalmaz felhasználási céljait az adatkatalógusban és az adatszerződésben rögzíteni kell, és a fogyasztók csak e célokra használhatják.
- **Adatminimalizálás:** csak az adott célhoz szükséges személyes adat gyűjthető. A tervezés során ez azt jelenti, hogy a rendszer ne gyűjtsön „hátha később kell” jellegű mezőket; ha egy mezőre nincs konkrét, dokumentált üzleti igény, ne tároljuk.
- **Korlátozott tárolás:** a személyes adatot csak addig szabad tárolni, ameddig a cél eléréséhez szükséges. A megőrzési idő letelte után az adatot törölni vagy anonimizálni kell. Ezért minden adathalmazhoz tartoznia kell megőrzési időnek (retention period) és törlési ütemezésnek.
- **Integritás és bizalmasság:** megfelelő technikai és szervezési intézkedésekkel kell biztosítani az adatok biztonságát, beleértve a titkosítást, a hozzáférés-kezelést és a naplózást.
- **Elszámoltathatóság:** a szervezetnek igazolnia kell a megfelelést, amihez dokumentálni kell az adatkezelési tevékenységeket, az adatáramlást és a hozzáféréseket.

A megőrzési szabályokat gyakran egy *adatmegőrzési mátrixban* rögzítjük, amely adathalmazonként vagy adatkategóriánként tartalmazza a minimális és maximális megőrzési időt, a törlés módját (fizikai törlés, anonimizálás) és a jogalapot. Példa:

Adathalmaz	Személyes adat?	Megőrzési idő	Törlési mód	Jogalap (példa)
Nyers rendelési log	Igen (IP, ID)	30 nap	Fizikai törlés automatikus	Szerződés teljesítése
Anonimizált rendelési statisztika	Nem	korlátlan	–	Jogos érdek (statisztika)

Adathalmaz	Személyes adat?	Megőrzési idő	Törlési mód	Jogalap (példa)
Panaszkezelési jegyek	Igen	2 év	Archiválás majd törlés	Jogi kötelezettség

Az *anonimizálás* azt jelenti, hogy az adatból visszafordíthatatlanul eltávolítjuk a személyes azonosításra alkalmas elemeket, így a GDPR hatálya alól kikerülhet. A *pszeudonimizálás* ezzel szemben az azonosítók helyettesítését jelenti egy kulccsal, de a kapcsolat visszaállítható; ezért a pszeudonimizált adat továbbra is személyes adat, csak csökkentett kockázatú. Az architektúrában ezeket a folyamatokat pipeline-okban automatizálhatjuk: például a nyers rendelési adatokat 30 nap után törlő job, vagy az azonosítókat hashelő, kulcsot elkülönítve tároló komponens.

A hozzáférési kérelmek (adat-hozzáférés, helyesbítés, törlés) teljesítéséhez az adatnak visszakereshetőnek és módosíthatónak kell lennie. Az adatvonal és az adatkatalógus itt is kritikus: ha egy ügyfél törlési kérelmet nyújt be, tudnunk kell, mely rendszerekben és másolatokban szerepel az adott személy azonosítója. Ezért a személyes adatokat tartalmazó rendszerekben az azonosítók egyértelmű leképezését és a replikák nyilvántartását naprakészen kell tartani.

16.4.10 Tipikus hibák és ellenintézkedések

Az adatminőség, governance és katalógus területén gyakran visszatérő hibák a következők. Mindegyiknél megadjuk a hiba felismerésének módját és a helyes megoldást.

1. **Nincs kijelölt adatgazda.** Tünet: senki sem tudja, kihez forduljon egy adathalmaz értelmezésével vagy minőségi problémájával kapcsolatban. Megoldás: minden regisztrált adathalmazhoz rendeljünk adatgazdát és helyettest; a katalógusban tegyük kötelező mezővé az `owner` és `steward` attribútumot.

2. **Hiányzó vagy nem egyértelmű adatszerződés.** Tünet: a fogyasztók feltételezésekre kényszerülnek a séma vagy a frissesség tekintetében, ami váratlan hibákhoz vezet. Megoldás: vezessünk be adatszerződést minden olyan adathalmazra, amelyet több csapat fogyaszt; a szerződésben rögzítsük a séma, a minőségi SLO-kat és a változáskezelési eljárást.

3. **Az adatminőséget utólag, manuálisan ellenőrzik.** Tünet: a hibákat csak akkor veszik észre, amikor egy riport vagy modell már rossz eredményt adott. Megoldás: a minőségi ellenőrzéseket építsük be a feldolgozási pipeline-ba, küszöbértékekkel és automatikus riasztással; a minőségi mutatókat jelenítsük meg az adatkatalógusban.

4. **Az adatkatalógus elavult, mert kézzel frissítik.** Tünet: a katalógusban szereplő séma nem egyezik a tényleges adatbázis sémával; a felhasználók többé nem bíznak benne. Megoldás: automatizáljuk a metaadatok gyűjtését (séma, lineage, frissességi idő, minőségi mutatók), és csak az üzleti leírásokat kezeljük emberi folyamattal.

5. **Nincsenek megőrzési és törlési szabályok.** Tünet: a tárolóban évek óta halmozódnak a személyes adatok, nincs törlési ütemezés, a GDPR-kérelmeket csak nehézkesen tudják teljesíteni. Megoldás: készítsünk adatmegőrzési mátrixot, jelöljük ki a törlési feladatokat, és automatizáljuk a megőrzési idő lejárta utáni törlést vagy anonimizálást.

6. **Túlzott adatgyűjtés „hátha kell” alapon.** Tünet: az adathalmaz számos, soha nem használt mezőt tartalmaz, köztük személyes adatokat is. Megoldás: az adatminimalizálás elvét alkalmazva minden mezőt igazolni kell egy konkrét felhasználási céllal; a nem szükséges mezőket ne gyűjtsük, vagy gyűjtésük előtt mérjük fel a kockázatot.

16.4.11 Labor: adatminőségi ellenőrzőlista és adatkatalógus-vázlat

A következő gyakorlat célja, hogy a fenti fogalmakat egy kis, ellenőrizhető példán keresztül alkalmazd. A labor nem igényel éles rendszert; elegendő egy dokumentum vagy jegyzetfüzet, amelyben a kért elemeket elkészíted. A laborhoz tartozik kiinduló állapot, elvárt eredmény, ellenőrzési szempont és visszaállítási út – ez utóbbi itt azt jelenti, hogy a dokumentum verziózott, és bármikor visszatérhetsz egy korábbi állapothoz.

16.4.12 Kiinduló állapot

Adott egy fiktív e-kereskedelmi rendszer, amely a rendeléseket egy PostgreSQL adatbázisban tárolja. A rendeléseket egy napi batch folyamat tölti be egy adattárházba. A rendelési adatok a következő mezőket tartalmazzák (kivonat): `order_id` (szöveg), `customer_id` (szöveg, személyes azonosításra alkalmas), `order_date` (dátum), `total_amount` (decimális), `status` (szöveg: `completed`, `cancelled`, `pending`). A forrás nem tartalmaz deduplikációs logikát; előfordulhatnak többször beszúrt rendelések. A törölt kosarak nem szerepelnek a forrásban, de a batch folyamat hibásan betöltött néhány `cancelled` státuszú rendelést is.

16.4.13 Feladat

1. Készíts egy **adatkatalógus-vázlatot** a fenti `orders` adathalmazhoz. A vázlat tartalmazza legalább: adathalmaz neve, domain, adatgazda, leírás, séma (mezők típusa és személyes adat jelölése), minőségi elvárások (frissesség, teljesség, egyediség, érvényesség), hozzáférési folyamat, lineage (forrás, transzformációk, fogyasztók). A vázlat lehet JSON, YAML vagy táblázatos formátumú.
2. Készíts egy **adatminőségi ellenőrzőlistát**, amely a fenti kontextusra vonatkozik. A lista legalább öt ellenőrzést tartalmazzon, mindegyiknél a dimenzió megnevezésével, a várt küszöbértékkel, a mérési módszerrel és a minta SQL-lel vagy pszeudokóddal. Példa: „Egyediség: `SELECT COUNT(*) - COUNT(DISTINCT order_id) FROM orders` == 0; várt: 0 duplikátum.”
3. Rögzíts egy **döntési naplót** (ADR-szerűen) arról, hogy miért vezetted be az adatszerződést és a megőrzési időt. A döntési napló kontextusa, döntés, alternatívák, következmények és állapot.
4. Válaszd el a **mért és a várt eredményt**: feltételezheted, hogy a fenti forrásadatokban a duplikátumok aránya 1,2%, a `total_amount` null vagy negatív értékek aránya 0,4%, a frissesség 90 perc. A várt küszöbök legyenek: duplikátum 0%, érvénytelen `total_amount` 0,5%, frissesség 60 perc. Írd le, melyik dimenzió felel meg, melyik nem, és milyen intézkedést javasolsz.
5. Az ellenőrzési szempontok: a katalógus-vázlatban szerepel-e az összes felsorolt elem? Az ellenőrzőlista minden dimenzióhoz tartalmaz-e mérést és küszöböt? A döntési naplóban látható-e az alternatívák mérlegelése? A mért és várt eredmény egyértelműen el van-e különítve?

A labor dokumentuma legyen reprodukálható: tartalmazza a kontextust (forrásrendszer, batch folyamat leírása), a dátumot, a készítőt, és a használt eszközök verzióját (pl. dokumentum formátuma, jegyzetfüzet kernel verziója). A visszaállítási út: a dokumentum git-verziózott, így bármely korábbi állapot visszanézhető.

16.4.14 Ellenőrzőlista az adatminőség és governance bevezetéséhez

- Minden kritikus adathalmaznak van kijelölt adatgazdája és helyettese.
- Minden több fogyasztóval rendelkező adathalmaz rendelkezik adatszerződéssel, amely tartalmazza a séma, minőségi SLO-kat és a kapcsolattartót.
- Az adatminőségi mutatók mérhetők, küszöbértékekkel és automatikus riasztással rendelkeznek.
- A minőségi mérések eredményeit a mért és várt értékek elkülönítésével dokumentáljuk.
- Az adatkatalógus naprakész: a technikai metaadatok automatikusan frissülnek, az üzleti leírásokat verziókövetik.
- Az adatvonal minden fogyasztói riport és modell esetében ismert, és a hatáselemzés elvégezhető.
- A személyes adatokat tartalmazó adathalmazokhoz megőrzési idő és törlési/anonimizálási terv tartozik.
- A hozzáférési kérelmek folyamata dokumentált, és a hozzáférési jogosultságok nyomon követhetők.
- Az adatminimalizálás elve érvényesül: nincs felesleges személyes adat gyűjtése.
- A döntések (pl. adatszerződés bevezetése, megőrzési idő) ADR-ben rögzítettek.

16.4.15 Önellenőrző kérdés

Egy vállalatnál két csapat használja ugyanazt a rendelési adathalmazt, de eltérően értelmezik a `status` mezőt: az egyik szerint a `cancelled` a fizetés előtti visszavonást jelenti, a másik szerint a fizetés utáni törlést is. Az adathalmaznak nincs adatgazdája, az adatminőséget senki sem méri, és a személyes adatokat tartalmazó mezőket határozatlan ideig tárolják. Milyen konkrét intézkedéseket javasolnál ennek az adathalmaznak a kezelésére? Hogyan kötődnek ezek az intézkedések az adatgazda, adatszerződés, adatkatalógus és megőrzési szabály fogalmaihoz?

A következő fejezet (Cloud-native alapelvek és 12-factor) az itt bevezetett fogalmakra épít, amikor a cloud-native alapelvek és a 12-factor tárgyalása során az adatkezelési szempontok is előkerülnek. Az adatkatalógus és az adatszerződés mintája segít majd megérteni, hogyan skálázhatók ezek a gyakorlatok elosztott, felhőalapú környezetben.

17 Cloud-native és konténerarchitektúrák

17.1 Cloud-native alapelvek és 12-factor

A felhőre tervezett alkalmazások alapelvei nem pusztán technológiai divatot jelentenek: olyan szerződést írnak le, amelyet az alkalmazás és az üzemeltető platform köt egymással. A platform vállalja, hogy elrejti a hardver, a hálózat és az operációs rendszer részleteit, dinamikusan erőforrást biztosít, és figyeli a folyamatok egészségét. Az alkalmazás pedig vállalja, hogy hordozható, állapotmentes, a konfigurációját kívülről kapja, és a naplóit eseményfolyamként bocsátja ki. Ezt a szerződést fogalmazza meg a 12-factor módszertan, amelynek célja, hogy az alkalmazás a felhőben ne harcoljon a platform ellen, hanem kihasználja annak dinamikus, automatizált és rövid életű erőforrásait.

A fejezet az előző adatarchitektúra egységre épít: a perzisztencia itt nem belső komponensként, hanem külső háttérszolgáltatásként jelenik meg, amelyet a folyamat hálózati szerződésen keresztül ér el. A következő egység (Konténerek, Kubernetes és orchestráció) ezeket az elveket üzemeltetési szintre emeli a konténerekkel és a Kubernetes platformmal.

17.1.1 1. A felhőre tervezett alkalmazás alapelvei

A felhőre tervezett (cloud-native) alkalmazás nem egyszerűen egy virtuális gépre telepített monolit, hanem olyan rendszer, amely a felhő üzemeltetési modelljére épül. A legfontosabb alapelvek:

- **API-vezérelt működés:** minden erőforrás, köztük a konfiguráció, a titokkezelés, a skálázás és a naplózás programozott interfészen keresztül érhető el. Az emberi kézi beavatkozás nem léptékezhető.
- **Automatizált telepítés és üzemeltetés:** a build, a tesztelés, a deploy és a visszagörgetés pipeline-ban fut, nem egyedi gépeken. A platform állapotát deklaratív leírók határozzák meg, és az üzemeltető rendszer egyezteti a kívánt és a tényleges állapotot.
- **Hibatűrés és redundancia:** a felhőben a hardver, a hálózat és a folyamat meghibásodása normális esemény, nem kivétel. Az alkalmazásnak fel kell készülnie a példányok elvesztésére, a hálózati késleltetésre és az átmeneti hibákra.
- **Horizontális skálázás:** a terhelés növekedését nem egy nagyobb géppel, hanem több, kisebb, független folyamattal kezeljük. Ez megköveteli, hogy a folyamatok ne tároljanak helyben olyan állapotot, amely a kérések között megmarad.
- **Állapotmentesség:** az üzleti állapot külső adatbázisba, cache-be vagy üzenetsorba kerül, a folyamat maga csak a kérés feldolgozásához szükséges átmeneti adatot tartja memóriában. Így bármely példány megszűnhet anélkül, hogy adatvesztés történne.
- **Megfigyelhetőség:** a rendszer belsejét naplókon, metrikákon és nyomkövetési adatokon keresztül látjuk. A felhőben nincs lehetőség minden példányra interaktívan belépni, ezért a diagnosztika adatainak kifelé kell áramolniuk.
- **Biztonság beépítése:** a hitelesítés, az engedélyezés, a titkok kezelése és a hálózati forgalom védelme nem utólagos réteg, hanem az alkalmazás és a platform alapértelmezett része.

- **Költségoptimalizálás:** a felhő erőforrásai mérhetők és elszámolhatók. A jól tervezett alkalmazás csak annyi erőforrást fogyaszt, amennyi az adott terheléshez szükséges, és a nem használt kapacitást felszabadítja.

Ezek az alapelvek szorosan összefüggenek a 12-factor manifesttel: a 12-factor nem más, mint ezen elvek gyakorlati megfogalmazása egy alkalmazás fejlesztési és üzemeltetési szabályrendszerében.

17.1.2 2. A 12-factor módszertan

A 12-factor módszertan a felhőre tervezett alkalmazások egyik legszélesebb körben hivatkozott szabálygyűjteménye. Eredetileg platformszolgáltatásként (PaaS) üzemelő webalkalmazásokra írták, de a konténeres világban is alapvető maradt, mert a konténerek és a Kubernetes éppen azt a platformmodellt valósítják meg, amelyet a faktorok feltételeznek. A faktorok nem új technológiákat írnak elő, hanem olyan tervezési döntéseket, amelyek növelik a hordozhatóságot, az üzemeltethetőséget és a skálázhatóságot.

Az alábbiakban mind a tizenkét faktort röviden, a gyakorlati hibákkal és a kivételekkel együtt mutatjuk be.

17.1.3 I. Kódbázis (codebase)

Egy kódbázis, több telepítés. Egy alkalmazásnak egy verziókezelt kódbázisa van, amelyből több környezetbe (fejlesztés, teszt, éles) telepítünk. Ha több alkalmazás osztozik egy kódbázison, vagy egy alkalmazás több kódbázisból áll, akkor a faktor sérül. Kivétel lehet a közös könyvtárak külön csomagként kezelése, de azok nem alkalmazások, hanem függőségek.

17.1.4 II. Függőségek (dependencies)

Deklaráld és izoláld a függőségeket. A függőségek listája verziózott fájlban szerepel (például `package.json`, `requirements.txt`, `pom.xml`), és a futtató környezetben nem támaszkodunk a rendszerre telepített könyvtárakra. A konténerkép ezt az izolációt erősíti: a függőségek a build során kerülnek a képbe, nem a futtató gépen. Tipikus hiba, ha egy rendszergazda manuálisan telepít egy csomagot, és az alkalmazás emiatt csak azon a gépen fut.

17.1.5 III. Konfiguráció (config)

A konfiguráció a környezetben él, nem a kódban. A konfiguráció minden, ami a telepítések között változik: adatbázis URL, hitelesítő adatok, külső szolgáltatások végpontjai, feature flagek. A kódban beégetett értékek sértik ezt a faktort, mert a konfiguráció változásához új build és deploy kell. A helyes megoldás a környezeti változók használata: a build során készül egy általános csomag, amelyet a futtató környezet paraméterez. Ez lehetővé teszi, hogy ugyanaz az artifact fusson fejlesztésben és élesben, csak a környezeti változók különböznek. A titkokat nem szabad a kódba vagy a build logba írni; azokat dedikált titokkezelőből kell juttatni a folyamathoz.

17.1.6 IV. Háttérszolgáltatások (backing services)

Kezeld az erőforrásokat csatolt szolgáltatásként. Az adatbázis, a cache, az üzenetsor, az objektumtároló mind hálózati szerződésen keresztül elérhető erőforrás. A folyamat szempontjából nincs különbség egy helyi és egy felhőszolgáltató által üzemeltetett PostgreSQL között: a kapcsolat URL-je a konfigurációból jön. Ez lehetővé teszi a háttérszolgáltatások cseréjét anélkül, hogy a kódot módosítanánk.

17.1.7 V. Build, release, run

A három fázist szigorúan különítsd el. A build a forráskód és a függőségek összeállítása egy futtatható artifacttá. A release a build és a konfiguráció összekapcsolása: egy azonosítható, visszaállítható egység. A run a release futtatása a platformon. A fázisok szétválasztása garantálja, hogy a futtató környezetben már nem módosítunk kódot vagy konfigurációt. A CI/CD pipeline ezt a modellt valósítja meg: a build image-et készít, a release a deploy során rögzíti a konfigurációt, a run pedig a konténer indítása.

17.1.8 VI. Folyamatok (processes)

A folyamatok állapotmentesek és nem osztanak meg semmit. A webalkalmazás folyamata nem tárolhat a kérések között megmaradó állapotot a helyi memóriában vagy a helyi fájlrendszerben. A munkamenet (session) adatai külső cache-be (például Redis) kerülnek, a feltöltött fájlok objektumtárolóba. Ez az elv teszi lehetővé, hogy bármely példány megszűnhet, és a platform új példányt indíthat anélkül, hogy a felhasználói állapot elveszne. A tipikus hiba a „sticky session”: a terheléselosztó ugyanahhoz a példányhoz irányítja a felhasználót, mert a munkamenet a helyi memóriában van. Ez akadályozza a horizontális skálázást és a hibatűrést.

17.1.9 VII. Portkötés (port binding)

Az alkalmazás önmaga szolgáltatja a hálózati interfészét. A webalkalmazás nem külső alkalmazásszerver (például Tomcat, Apache) mögé települ, hanem beépített HTTP-kiszolgálóval rendelkezik, és egy porton várja a kéréseket. A platform ezután ezt a portot irányítja a külső forgalomhoz. A konténerben az alkalmazás például a 8080-as portot nyitja meg, a Kubernetes pedig a service-en keresztül éri el.

17.1.10 VIII. Párhuzamosság (concurrency)

Skálázás a folyamatmodell szerint. Az alkalmazás nem egy nagy, sokszálú monolit, hanem több, független folyamatként indul. A terhelés növekedésekor a folyamatok számát növeljük, nem a szálak számát egy folyamaton belül. Ez a modell illeszkedik a konténeres platformokhoz, ahol a replikák száma dinamikusan állítható. A Little-törvény segít megérteni az összefüggést: a rendszerben lévő kérések átlagos száma $L = \lambda W$, ahol λ az érkezési ráta, W pedig az átlagos tartózkodási idő. Ha a folyamatok állapotmentesek, a replikák számának növelése csökkenti az egy replikára jutó λ értéket, így – ha a szolgáltatás nem telítődik – a W is csökken, javítva a válaszidőt.

17.1.11 IX. Eldobhatóság (disposability)

Gyors indítás és elegáns leállás. A folyamatnak másodpercek alatt el kell indulnia, és a leállítási jelre (SIGTERM) rövid időn belül be kell fejeznie a folyamatban lévő kéréseket, majd ki kell lépnie. A gyors indítás lehetővé teszi a gyors skálázást és a gyakori deployt; az elegáns leállás biztosítja, hogy a deploy vagy a platform karbantartása során ne veszítsünk kéréseket. A hosszú, több percig tartó indulás vagy a leállási jel figyelmen kívül hagyása sérti a faktort.

17.1.12 X. Fejlesztési/éles környezeti egyenértékűség (dev/prod parity)

A fejlesztés és az éles környezet közötti eltérést minimalizáld. Minél nagyobb a különbség a fejlesztői gép, a tesztkörnyezet és az éles rendszer között, annál nagyobb a valószínűsége, hogy egy hiba csak élesben jelentkezik. A faktor három dimenziót emel ki: az idő (a fejlesztés és a deploy közötti késleltetés), a személyek (a fejlesztő és az üzemeltető közötti távolság) és az eszközök (a háttérszolgáltatások eltérése). A cél, hogy a fejlesztő ugyanolyan adatbázist, cache-t és üzenetsort használjon lokálisan, mint élesben, lehetőleg konténerben vagy felhőalapú sandboxban. Kivétel lehet a költséges, ritkán használt szolgáltatások helyettesítése emulátorral, de ezt tudatosan kell kezelni.

17.1.13 XI. Naplók (logs)

A napló eseményfolyam, nem fájl. Az alkalmazás nem ír naplófájlokat a helyi fájlrendszerre, hanem a standard kimenetre (stdout) vagy a standard hibára (stderr) ír. A platform összegyűjti ezeket a folyamokat, és továbbítja egy központi naplózó rendszerbe. A fájlba írás azért hibás, mert a konténer újraindításakor a fájl elveszik, és a platform nem tudja begyűjteni. A napló formátuma lehet JSON, hogy a feldolgozó rendszer strukturáltan elemezhesse.

17.1.14 XII. Adminisztrációs folyamatok (admin processes)

Az adminisztratív feladatokat a release részeként, egyszeri folyamatként futtasd. Az adatbázis-migráció, az adatfeltöltés, az indexelés nem interaktív, hosszú életű folyamat, hanem a release-szel együtt verziózott, egyszer lefuttatott parancs. A konténeres világban ez azt jelenti, hogy a migráció egy külön konténerként fut, ugyanazzal a kóddal és konfigurációval, mint a fő alkalmazás, de csak egyszer, a deploy során.

A 12 faktor közül a **konfiguráció (III)**, a **folyamatok (VI)**, a **párhuzamosság (VIII)** és az **eldobhatóság (IX)** alkotják a felhőre tervezett alkalmazás magját. Ezeket a következő szakaszban mélyebben is megvizsgáljuk.

17.1.15 3. Konfiguráció, folyamat és skálázhatóság

17.1.16 3.1 Konfiguráció a környezetben

A III. faktor gyakorlati célja, hogy ugyanaz a build futni tudjon fejlesztésben, tesztben és élesben anélkül, hogy a kódot módosítanánk. A konfigurációt környezeti változókban adjuk át, mert ezek a legtöbb futtató környezetben elérhetők, és nem kerülnek be a verziókezelt kódba. Példa:

```
# Helyőrző konfiguráció – nem éles értékek
export DB_URL=postgres://order_svc:${DB_PASSWORD}@db:5432/orders
export REDIS_URL=redis://redis:6379/0
export OBJECT_STORE_URL=https://object-store.example.local/orders
export FEATURE_MAINTENANCE_MODE=false
```

A titkokat nem szabad a környezeti változókban sem kódban tárolni; azokat a platform titokkezelőjéből kell injektálni. A Kubernetesben például a Secret erőforrásból származó értékeket a konténer indításakor a környezetbe helyezzük.

A konfigurációra vonatkozó minőségi követelmény quality attribute scenario formában:

> **Inger:** az adatbázis URL-je megváltozik egy karbantartás miatt.
> **Környezet:** éles konténer, terhelés alatt.
> **Válasz:** az új URL a folyamat újraindításával, deploy nélkül érvényesül.
> **Mérőszám:** az érvényesülés ideje kevesebb mint 30 másodperc, mérési ablak 1 óra.

17.1.17 3.2 Folyamat mint állapotmentes egység

A VI. faktor tiltja a helyi állapot megőrzését a folyamatban. Ez két dolgot jelent:

1. **Nincs helyi munkamenet:** a felhasználói munkamenet azonosítóját a kliens sütiben vagy tokenben tartja, a hozzá tartozó adatokat pedig külső cache-ben (például Redis). Így bármelyik példány kiszolgálhatja a kérést.
2. **Nincs helyi fájllállapot:** a feltöltött fájlokat nem a konténer fájlrendszerébe mentjük, hanem objektumtárolóba. A konténer fájlrendszere efemer, újraindításkor elveszik.

A folyamat akkor állapotmentes, ha a memóriában csak a kérés feldolgozásához szükséges átmeneti adatot tart, és az a kérés befejezése után felszabadítható. Ez teszi lehetővé, hogy a platform bármikor leállítsa vagy újraindítsa a példányt anélkül, hogy üzleti adat veszne el.

17.1.18 3.3 Skálázhatóság és a folyamatmodell

A VIII. faktor a horizontális skálázást támogatja. Ahelyett, hogy egy folyamaton belül növelnénk a szálak számát, több független folyamatot indítunk. A konténeres platformok ezt replikák számának állításával kezelik.

A Little-törvény alkalmazása: ha a szolgáltatás egy replikája egy adott λ érkezési rátát W átlagos tartózkodási idővel szolgál ki, akkor a rendszerben lévő kérések száma $L = \lambda W$. Ha a replikák számát n-re növeljük, és a terheléselosztó egyenletesen osztja el a kéréseket, akkor az egy replikára jutó érkezési ráta λ / n lesz. Ha a szolgáltatás nem telítődik, a W is csökken, így a felhasználói válaszidő javul. Ez a modell csak akkor működik, ha a folyamatok állapotmentesek: ha a munkamenet a helyi memóriában van, a terheléselosztó kénytelen ugyanahhoz a példányhoz irányítani a felhasználót, és a skálázás nem osztható szét.

17.1.19 4. Állapotmentesség és a felhő mint üzemeltetési modell

A felhő üzemeltetési modelljének lényege, hogy a platform a folyamatokat rövid életű, pótolható erőforrásokként kezeli. A fizikai gép meghibásodhat, a hálózat szakadozhat, a konténer újraindulhat. Az alkalmazás akkor tud ebben a környezetben megbízhatóan működni, ha a folyamatai:

- **gyorsan indulnak:** a platform így gyorsan tud pótolni egy elveszett példányt;

- **elegánsan állnak le:** a leállítási jelre befejezik a folyamatban lévő kéréseket, és a kapcsolatokat lezárják;
- **nem tárolnak állapotot:** az állapot külső szolgáltatásban van, így a példány elvesztése nem okoz adatvesztést.

A rendelkezésre állás képlete $A = \frac{\text{üzemidő}}{\text{üzemidő}+\text{állásidő}}$ jól mutatja a disposability hatását: ha a deploy során a folyamat leállítása és az új indítása között 3 másodperc telik el, akkor egy naponta négyszer deployolt szolgáltatásnál az állásidő 12 másodperc. Ha viszont a folyamat a leállítási jelet figyelmen kívül hagyja, és a platform kénytelen 30 másodperc után erőszakkal leállítani, akkor ugyanez a deploy 2 perc állásidőt okozhat. Az error budget $1 - \text{SLO}$ ugyanebben a mérési ablakban fogy.

A PaaS-modell (például a Kubernetes) ezt a szerződést deklaratív formában írja elő: az alkalmazás deklarálja a kívánt replikaszámot, a platform pedig biztosítja, hogy mindig annyi egészséges példány fusson. Ha egy példány meghibásodik, a platform újat indít. Ez csak akkor működik, ha a folyamatok állapotmentesek, a konfiguráció kívülről jön, és a naplók kifelé áramlanak.

17.1.20 5. Példa: egy rendelés-nyilvántartó szolgáltatás ellenőrzése

Vegyünk egy kitalált, egyszerű webes alkalmazást, az `acme-order-service`-t, amely rendeléseket tárol és lekérdez. A rendszer C4 container szintű nézete:

```
flowchart LR
    User[Webes felhasználó] -->|HTTPS, szinkron| WebApp
    Admin[Adminisztrátor] -->|HTTPS, szinkron| WebApp
    subgraph OrderSystem[Rendelés-nyilvántartó rendszer]
        WebApp[Web App\nNode.js\nstateless]
    end
    WebApp -->|SQL, szinkron, retry| Postgres[(PostgreSQL)]
    WebApp -->|RESP, szinkron, retry| Redis[(Redis)]
    WebApp -->|HTTPS, aszinkron, retry| ObjectStore[Object Storage]
```

Cím: Rendelés-nyilvántartó szolgáltatás konténerei.
Jelmagyarázat: téglalap – konténer; henger – adatbázis; hatszög – felhasználó; nyíl – adatáramlás.
Feltételjegyzet: a diagram a 12-factor elvek szerinti ideális állapotot mutatja; a valós megvalósításban a Redis és az Object Storage csak akkor szükséges, ha a folyamat valóban állapotmentes.

A 12 faktor ellenőrzésekor az alábbi megállapításokat tehetjük:

Faktor	Vizsgálat	Megállapítás (példa)
I. Codebase	Egy kódbázis, több deploy?	Igen, egy Git-repo, három környezet
II. Dependencies	Függőségek deklarálva, izolálva?	Igen, `package.json` és `package-lock.json`, konténerben
III. Config	Konfiguráció környezetből?	Részben: DB URL env-ből, de a feature flag `false` a kódban van beégetve
IV. Backing services	Erőforrások csatoltként?	Igen, URL-ek env-ből
V. Build, release, run	Fázisok elkülönítve?	Igen, CI pipeline, image tag
VI. Processes	Állapotmentes?	Igen, session Redisben
VII. Port binding	Szolgáltatás porton?	Igen, 8080-as port
VIII. Concurrency	Folyamatmodell skálázás?	Igen, replica count 3

Faktor	Vizsgálat	Megállapítás (példa)
IX. Disposability	Gyors indítás, leállás?	Indítás 8 s, SIGTERM kezelés 3 s
X. Dev/prod parity	Környezetek hasonlóak?	Részben: fejlesztés lokális SQLite, éles PostgreSQL
XI. Logs	Log stdout?	Igen, JSON log
XII. Admin processes	Admin folyamatok egyszeri?	Igen, migráció külön konténerben

A példa két tipikus hibát mutat: a III. faktor megsértését (beégetett feature flag) és a X. faktor gyengeségét (eltérő adatbázis a fejlesztésben). A döntési gyakorlat ezekre épít.

17.1.21 6. Labor: 12-factor ellenőrzés egy alkalmazáson

Cél: egy adott alkalmazás 12-factor megfelelőségének ellenőrzése, a megállapítások dokumentálása, valamint a mért és a várt eredmény elkülönítése.

Kiinduló állapot: a hallgató hozzáférést kap egy példaalkalmazás kódbázisához (például a fenti `acme-order-service`), amely konténerben fut. A kiinduló repó helyőrző: `<repo-url>`. Az alkalmazás a fejlesztői környezetben elindítható, és a 12 faktor közül néhány szándékosan sérül.

Elvárt eredmény: egy Markdown dokumentum vagy Jupyter notebook, amely tartalmazza:

- a vizsgált alkalmazás nevét és verzióját;
- a környezet leírását (futtató platform, konténer, operációs rendszer, időpont);
- mind a 12 faktorra egy ellenőrző kérdést és a megállapítást;
- a mért és a várt eredményt külön oszlopban;
- egy rövid döntési naplót (ADR) a talált eltérések kezelésére;
- az ellenőrzési szempontokat.

A mérési eredményeket csak akkor szabad tényként bemutatni, ha a mérés környezete, konfigurációja és időpontja rögzített. A nem futtatott mérést „nem mért” jelöléssel kell megadni.

Ellenőrzési szempontok: minden faktorhoz egy igen/nem kérdés, és a bizonyíték (például kódrészlet, parancs kimenete). Például a III. faktornál: „A konfiguráció minden értéke a környezetből érkezik?” – „Nem, a `FEATURE_MAINTENANCE_MODE` a kódban van.”

Visszaállítási út: a labor vizsgálati jellegű, ezért nincs romboló művelet. Ha a hallgató módosít a kódon vagy a konfiguráción, a változtatásokat a verziókezelőben vissza lehet vonni, és a konténer az eredeti image-ből újraépíthető.

17.1.22 7. Döntési gyakorlat

A fenti ellenőrzés során kiderült, hogy az `acme-order-service` esetében:

1. A `FEATURE_MAINTENANCE_MODE` logikai érték a kódban beégetve szerepel, ezért a karbantartási mód be- és kikapcsolása új buildet igényel.
2. A fejlesztői környezet SQLite-ot használ, míg az éles PostgreSQL-t.

Feladat: írjon ADR-t a konfiguráció külső környezeti változóba helyezéséről, és indokolja meg, miért nem elegendő a kód módosítása deploy nélkül. Az ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Második lépésként javasoljon egy megoldást a fejlesztési/éles környezeti eltérés csökkentésére, figyelembe véve a költségeket és a csapat méretét.

17.1.23 8. Ellenőrzőlista

- ☐ A kódbázis egyértelműen azonosítható, és egy alkalmazáshoz tartozik.
- ☐ A függőségek deklaráltak és izoláltak, a futtató gépen nincs kézi csomagtelepítés.
- ☐ A konfiguráció minden értéke a környezetből érkezik, a kódban nincs beégetett konfiguráció.
- ☐ A háttérszolgáltatások URL-je a konfigurációból jön, nem a kódból.
- ☐ A build, a release és a run fázisok elkülönülnek, a release visszaállítható.
- ☐ A folyamatok állapotmentesek, a munkamenet és a fájlok külső tárolóban vannak.
- ☐ Az alkalmazás saját porton szolgáltatja a HTTP-t, nem külső szerver mögött.
- ☐ A skálázás a folyamatok számának növelésével történik, nem egy folyamaton belül.
- ☐ A folyamat gyorsan indul, és a leállítási jelet elegánsan kezeli.
- ☐ A fejlesztési és az éles környezet közötti eltérés minimális.
- ☐ A naplók a standard kimenetre íródnak, nem helyi fájlba.
- ☐ Az adminisztratív folyamatok a release részeként, egyszeri konténerként futnak.

17.1.24 9. Önellenőrző kérdés

Miért fontos, hogy a folyamat ne tároljon állapotot a helyi fájlrendszerben, ha a platform automatikusan skálázhatja a példányokat? Magyarázza el a horizontális skálázás és az állapotmentesség kapcsolatát a Little-törvény segítségével.

17.1.25 10. Összefoglalás és továbbvezetés

A cloud-native alkalmazások alapelvei és a 12-factor módszertan együtt adják meg azt a szerződést, amely lehetővé teszi, hogy a rendszer a felhő dinamikus, automatizált környezetében megbízhatóan működjön. A konfiguráció kívülre helyezése, a folyamatok állapotmentessége és a folyamatmodell szerinti skálázás nem öncélú szabályok, hanem a hordozhatóság, az üzemeltethetőség és a rendelkezésre állás előfeltételei. A következő egység (Konténerek, Kubernetes és orchestráció) ezeket az elveket ülteti át a konténerek és a Kubernetes platform gyakorlatába, ahol a deklaratív állapotkezelés és a replikák kezelése válik elsődlegessé.

17.2 Konténerek, Kubernetes és orchestráció

A konténerizálás a felhő-native rendszerek egyik legalapvetőbb építőköve, de önmagában még nem ad választ arra a kérdésre, hogyan üzemeltessünk több száz vagy több ezer konténert egy időben. A Kubernetes és az orchestráció világa éppen erre a problémára épült: a konténerek ütemezését, hálózatát, tárolását és életciklusát deklaratív módon, klaszterszinten kezeli. Ebben a fejezetben a konténerizálás gyakorlati aspektusaitól indulunk, majd bevezetjük a Kubernetes alapobjektumait – pod, deployment, service és ingress –, végül pedig az erőforrás-kezelés, az egészségellenőrzés és az automatikus skálázás kérdéseit járjuk körül. A cél nem egy teljes Kubernetes-referencia, hanem olyan architektúrális szemlélet kialakítása, amellyel meg tudod ítélni, mikor és hogyan érdemes konténeres platformot használni, és milyen hibákat kell elkerülni.

17.2.1 A konténerizálás gyakorlati alapjai

A konténer egy folyamat, amely a saját, izolált felhasználói terében fut, és amelyet a hoszt operációs rendszere ütemez. A konténerkép (image) nem virtuális gép: nincs saját kernel, nincs saját init rendszer, és a hoszt erőforrásait osztja meg más konténerekkel. Ez a megközelítés alacsony indítási költséget, sűrű csomagolást és reprodukálható futtatási környezetet biztosít.

A konténerkép definíciója rétegekből épül fel. Minden réteg egy írásvédett fájlrendszer-változást rögzít, például egy alaprendszer telepítését, egy függőség másolását vagy egy konfigurációs fájl elhelyezését. Az építés során a rétegek cache-elhetők, ezért az image buildelés gyorsítható, ha a gyakran változó részeket a Dockerfile végére helyezzük. A kép futtatásakor egy írható réteg kerül a tetejére, amely a konténer megszűnésével alapértelmezés szerint eldobódik. Ezért a perzisztens adatokat mindig mountolt kötetben vagy külső adatbázisban kell tárolni.

A konténerek hordozhatósága azonban nem abszolút. A kép tartalmazza az alkalmazás futtatásához szükséges könyvtárakat, de a kernel és a hoszt képességei (például seccomp, AppArmor, hálózati illesztők) platformonként eltérhetnek. Architektúrális döntés, hogy milyen alapképet választasz: a „distroless” vagy minimalista alapképek kisebb támadási felületet és kevesebb sebezhetőséget jelentenek, de nehezebb bennük debuggolni. A nagyobb, teljes operációs rendszert tartalmazó képek kényelmesebbek, viszont lassabbak és több sebezhetőséget hordozhatnak. A választásnál a karbantarthatóság, a biztonság és a csapat tapasztalata egyaránt trade-off.

A konténerek futtatásához szükség van egy container runtime-ra (például containerd, CRI-O), amely a Kubernetes-szel a Container Runtime Interface-en (CRI) keresztül kommunikál. A Kubernetes tehát nem maga futtatja a konténereket, hanem a runtime-ot vezérli. Ez a felosztás lehetővé teszi, hogy a klaszter különböző runtime-okat használjon, és hogy a Kubernetes szabadon maradjon a futási környezet részleteitől.

17.2.2 Kubernetes-objektumok és a deklaratív modell

A Kubernetes egy elosztott rendszer, amelynek szíve az API-szerver. Minden művelet – legyen szó egy pod létrehozásáról, egy service definiálásáról vagy egy skálázási parancsról – az API-n keresztül történik. A felhasználó nem ad közvetlen utasítást arra, hogy „indíts el öt konténert ezen a három gépen”, hanem leírja a kívánt állapotot: „öt replika fut, mindegyik ezzel a képpel és ezekkel az erőforrás-korlátokkal”. A vezérlőhurok (control loop) folyamatosan összehasonlítja a tényleges állapotot a kívánttal, és beavatkozik, ha eltérés van.

Ez a deklaratív modell alapvetően más, mint a hagyományos, szkriptelt infrastruktúra-kezelés. Nem az a kérdés, hogy milyen parancsokat kell kiadni, hanem az, hogy milyen állapotot akarunk elérni. A deklaratív konfiguráció teszi lehetővé a verziózott, felülvizsgálható infrastruktúra-kezelést, és illeszkedik a GitOps szemlélethez is.

A Kubernetes objektumai három fő csoportba sorolhatók: a workload-objektumok (pod, deployment, statefulset, job), a hálózati és szolgáltatás-objektumok (service, ingress), valamint a konfigurációs és tárolási objektumok (configmap, secret, persistentvolumeclaim). Ebben a fejezetben a workload és a hálózati réteg alapjait vizsgáljuk.

17.2.3 Pod: a legkisebb ütemezhető egység

A pod a Kubernetes legalapvetőbb építőköve. Egy pod egy vagy több konténert fog össze, amelyek közös hálózati névtérben, közös IPC-néven és közös hosztnévvel futnak. Ez azt jelenti, hogy a podon belüli konténerek a `localhost`-on érik el egymást, és osztoznak a pod IP-címén. A

pod az ütemezés egysége: a scheduler mindig podokat helyez el node-okon, nem pedig egyes konténereket.

A több konténerből álló pod leggyakoribb mintája a sidecar. A sidecar konténer kiegészítő feladatot lát el a fő konténer mellett, például loggyűjtést, proxyzást, titkosításkezelést vagy metrikaexportálást. A service mesh – például Istio vagy Linkerd – is ezt a mintát használja: minden podhoz egy envoy-proxy sidecar csatlakozik, amely a hálózati forgalmat kezeli. Architektúrálisan fontos, hogy a sidecar és a fő konténer életciklusa szorosan összekapcsolódik: ha a pod meghal, mindkettő meghal.

A pod életciklusa állapotokból áll: `Pending`, `Running`, `Succeeded`, `Failed`, `Unknown`. Emellett a pod rendelkezhet feltételekkel, például `PodScheduled`, `Initialized`, `ContainersReady`, `Ready`. A `Ready` feltétel azt jelzi, hogy a pod készen áll a forgalom fogadására. Ez a readiness probe eredményétől függ, amelyet később tárgyalunk.

A podokat ritkán hozzuk létre közvetlenül. A production rendszerekben a podokat workload-objektumok – deployment, statefulset, daemonset, job – felügyelik. A pod közvetlen létrehozása csak hibakereséshez vagy egyszeri feladatokhoz indokolt. Ennek oka, hogy a pod önmagában nem öngyógyító: ha a node meghibásodik, a pod elvész. A deployment viszont biztosítja a replikák számának fenntartását.

17.2.4 Deployment: a deklaratív alkalmazásfrissítés

A deployment egy magasabb szintű workload-objektum, amely a podok replikáinak menedzselésére szolgál. A deployment specifikációjában megadod a kívánt replikaszámot, a pod-sablont és a frissítési stratégiát. A deployment létrehoz egy ReplicaSet-et, amely a podok tényleges replikáit tartja fenn. Amikor frissíted a deployment képét vagy konfigurációját, a deployment új ReplicaSet-et hoz létre, és a frissítési stratégia szerint fokozatosan áthelyezi a forgalmat.

A két leggyakoribb frissítési stratégia a `RollingUpdate` és a `Recreate`. A `RollingUpdate` alapértelmezés szerint fokozatosan cseréli a podokat: először elindít néhány új podot, megvárja, amíg azok készen állnak, majd leállít néhány régit. Így a szolgáltatás folyamatosan elérhető marad. A `Recreate` először leállítja az összes régi podot, majd elindítja az újakat. Ez rövidebb leállást okoz, de egyszerűbb, és olyan esetekben indokolt, amikor a régi és az új verzió nem futhat párhuzamosan (például adatbázis-séma migráció esetén).

A deployment deklaratív frissítése lehetővé teszi a rollbacket. A Kubernetes megőrzi a korábbi ReplicaSet-eket, így a `kubectl rollout undo` paranccsal visszaállítható az előző verzió. Ez a rollback azonban nem helyettesíti az adatbázis-migrációk visszafordíthatóságát: ha az új verzió olyan séma-módosítást hajtott végre, amelyet a régi verzió nem értelmez, a rollback önmagában nem elegendő. Architektúrális szempontból a deployment és az adatbázis-migrációk összehangolása külön döntést igényel.

17.2.5 Service: stabil hálózati azonosító

A podok IP-címe dinamikusan változik: amikor egy pod meghal és új indul, az új pod más IP-t kap. Ha a kliensek közvetlenül a podok IP-címéhez csatlakoznának, a kapcsolat megszakadna minden újraindításnál. A service ezt a problémát oldja meg: egy stabil virtuális IP-címet és DNS-nevet biztosít a podok halmazához. A service a selector segítségével kiválasztja a hozzá tartozó podokat, és a bejövő forgalmat elosztja közöttük.

A service négy fő típusa:

- **ClusterIP**: a service csak a klaszteren belül érhető el. Ez a leggyakoribb belső szolgáltatások esetén. A klaszter DNS-ében a service neve alapján feloldható.
- **NodePort**: a service minden node-on egy fix portot nyit meg, és a forgalmat a podokhoz irányítja. Ez egyszerű módja a külső elérésnek, de nem alkalmas production load balancingra.
- **LoadBalancer**: a felhőszolgáltató vagy egy külső load balancer egy nyilvános IP-címet rendel a service-hez. Ez a leggyakoribb módja a külső forgalom beengedésének.
- **ExternalName**: a service egy DNS CNAME rekordra mutat, így a klaszteren belüli alkalmazások külső szolgáltatásokat érhetnek el stabil néven.

A service a podok kiválasztását label selector alapján végzi. Ez a laza csatolás lehetővé teszi, hogy a podok bármikor cserélődjenek anélkül, hogy a service definícióját módosítani kellene. A service által használt portokat a `port` (a service portja), a `targetPort` (a pod portja) és a `nodePort` (a node portja) különbözteti meg.

17.2.6 Ingress: L7 forgalomirányítás

A service önmagában L4 szintű terheléselosztást végez: a forgalmat IP-cím és port alapján irányítja. Az ingress ezzel szemben L7 szinten dolgozik: HTTP host és path alapján dönti el, melyik service-hez továbbítsa a kérést. Az ingress nem önálló komponens, hanem egy erőforrás-definíció, amelyet egy ingress controller valósít meg. A controller figyeli az ingress objektumokat, és a bennük leírt szabályok alapján konfigurálja a mögöttes proxy-t (például nginx, Traefik, HAProxy).

Az ingress tipikus szabályai:

- **Host-alapú routing**: a `example.com` hoszt a webshop service-hez, az `api.example.com` hoszt az API service-hez irányul.
- **Path-alapú routing**: a `/checkout` útvonal a checkout service-hez, a `/search` útvonal a search service-hez irányul.
- **TLS-terminálás**: az ingress controller lezárja a TLS-kapcsolatot, és a belső forgalmat titkosítás nélkül továbbítja – vagy éppen végpontok közötti titkosítást végez.

Az ingress használatával csökkenthető a külső terheléselosztók száma: egyetlen bejárati ponton keresztül több szolgáltatás érhető el. Ez azonban azt is jelenti, hogy az ingress controller kritikus komponenssé válik: ha nem skálázódik, az egész rendszer bejárata szűk keresztmetszet lesz. A production környezetben az ingress controllert több replikával és megfelelő erőforrás-korlátokkal kell futtatni.

17.2.7 Erőforrás-korlátok és minőségi osztályok

A Kubernetes akkor tud hatékonyan ütemezni, ha ismeri, mennyi CPU-ra és memóriára van szüksége egy podnak. Az erőforrás-igényeket két értékkel lehet megadni:

- **requests**: az a minimális erőforrás-mennyiség, amelyet a pod garantáltan megkap. A scheduler ezt az értéket használja annak eldöntésére, hogy a pod elfér-e egy node-on.
- **limits**: az a maximális erőforrás-mennyiség, amelyet a pod felhasználhat. Ha a pod túllépi a memória-limitjét, a kernel OOM killer leállítja a konténert. CPU esetén a limit túllépése throttlingot okoz, de nem állítja le a podot.

A CPU erőforrást magokban vagy millimagban adjuk meg. A `250m` negyed magot jelent. A memória esetén a `Mi`, `Gi` bináris prefixumokat használjuk. A requests és limits hiánya esetén a pod a `BestEffort` minőségi osztályba kerül, ami azt jelenti, hogy a node túlterhelése esetén elsőként lesz evicted. A `Burstable` osztályba tartoznak azok a podok, amelyeknél a requests és limits eltér. A `Guaranteed` osztályt akkor kapja a pod, ha minden konténerre meg van adva a request és a limit, és azok értéke megegyezik. A `Guaranteed` podok a legvédettebbek az evictionnel szemben.

A production rendszerekben soha ne futtassunk podot korlátok nélkül. Egy korlátlan pod elfogyaszthatja a node összes erőforrását, és kiszoríthat más munkaterheléseket. A túl szoros limit viszont throttlingot vagy OOM kill-t okoz, ami a latency növekedéséhez vagy a pod újraindulásához vezet. A megfelelő érték meghatározása mérésen alapul: terhelési teszt alatt figyelni kell a tényleges fogyasztást, és a limitet a tipikus csúcs fölé kell helyezni némi tartalékkal, például 20–30%-kal.

17.2.8 Health check: liveness, readiness és startup

A Kubernetes csak akkor tudja megbízhatóan kezelni a podokat, ha tudja, hogy azok valóban működőképesek-e. A health check háromféle probe-ot kínál:

- **liveness probe**: azt ellenőrzi, hogy a konténer él-e. Ha a liveness probe sikertelen, a kubelet újraindítja a konténert. Ez a mechanizmus az elakadt, deadlock-olt folyamatok ellen véd.
- **readiness probe**: azt ellenőrzi, hogy a konténer készen áll-e a forgalom fogadására. Ha a readiness probe sikertelen, a pod kikerül a service endpointjai közül, és nem kap új kéréseket. Ez a mechanizmus az induló vagy átmenetileg túlterhelt podok védelmére szolgál.
- **startup probe**: azt ellenőrzi, hogy a konténer elindult-e. Ha van startup probe, a liveness és readiness probe csak akkor indul el, miután a startup probe sikeres lett. Ez hasznos olyan alkalmazásoknál, amelyek lassan indulnak, és a liveness probe túl korán hibát jelezne.

A probe-ok háromféleképpen végezhetők: HTTP GET kérés, TCP socket kapcsolat, vagy parancs végrehajtása a konténerben. A leggyakoribb a HTTP GET, mert az alkalmazás saját egészségügyi végpontját ellenőrzi. A probe konfigurációjában megadható az `initialDelaySeconds`, a `periodSeconds`, a `timeoutSeconds`, a `failureThreshold` és a `successThreshold`. Ezen értékek hangolása kritikus: túl érzékeny probe esetén a pod feleslegesen újraindul, túl engedékeny probe esetén pedig a hibás pod túl sokáig kap forgalmat.

A liveness és readiness megkülönböztetése alapvető hibaforrás. Sokan ugyanazt a végpontot használják mindkettőhöz, ami hibás működéshez vezet: ha a liveness probe ugyanazt az endpointot hívja, mint a readiness, akkor egy átmeneti túlterhelés – amikor a pod lassan válaszol, de még él – liveness hibát okozhat, és a pod újraindul, ami tovább növeli a terhelést. Ehelyett a liveness probe-nak csak azt kell ellenőriznie, hogy a folyamat fut-e, a readiness probe-nak pedig azt, hogy a pod készen áll-e a forgalomra, beleértve a függőségek elérhetőségét is.

17.2.9 Autoscaling: a kapacitás dinamikus igazítása

A statikus replikaszám ritkán optimális. A terhelés napszaktól, kampányoktól és szezonális eseményektől függően változik. A Kubernetes horizontális pod autoscaler (HPA) a podok replikaszámát a megfigyelt metrikák alapján állítja be. A HPA definíciójában megadod a célobjektumot (például egy deploymentet), a minimális és maximális replikaszámot, valamint a célmetrikát.

A leggyakoribb metrika a CPU-kihasználtság. A HPA a Metrics Serverből olvassa a podok tényleges CPU-fogyasztását, és a `targetCPUUtilizationPercentage` érték alapján számolja ki a kívánt replikaszámot. A képlet:

$$R_{\text{kívánt}} = \left\lceil R_{\text{jelenlegi}} \cdot \frac{U_{\text{jelenlegi}}}{U_{\text{cél}}} \right\rceil$$

ahol R a replikák száma, U pedig a kihasználtság. Ha például három replika fut, a cél 60%-os CPU-kihasználtság, és a tényleges átlag 90%, akkor a HPA négy-öt replikára skáláz fel.

A memória-alapú HPA is lehetséges, de óvatosan kell vele bánni. A memória nem csökken automatikusan, ha a terhelés csökken, ezért a memória-alapú skálázás gyakran nem hozza vissza a replikaszámot. A legtöbb production rendszer CPU-ra vagy egyedi, alkalmazásszintű metrikákra (például kérések száma másodpercenként, sorhossz) skáláz.

A HPA mellett létezik vertikális pod autoscaler (VPA) is, amely a podok CPU- és memória-requests és limits értékeit igazítja. A VPA azonban a pod újraindítását igényli az értékek módosításához, ezért nem mindig alkalmas folyamatosan futó, állapotmentes szolgáltatásokhoz. A cluster autoscaler ezzel szemben a node-ok számát skálázza: ha a podok nem férnek el a meglévő node-okon, új node-ot indít, ha pedig a node-ok kihasználatlanok, leállítja őket.

Az autoscaling akkor működik jól, ha a podok állapotmentesek, a startup idő rövid, a health checkek megbízhatók, és a kapacitástervezés a Little-törvényt is figyelembe veszi. Egy lassan induló pod esetén a HPA felfelé skáláz, de mire az új pod készen áll, a terhelés már meghaladhatja a kapacitást. Ezért a skálázási döntéseknél a rendszer válaszidejét és a podok indulási idejét együtt kell elemezni.

A skálázás nem oldja meg a rossz alkalmazásarchitektúrát. Ha egy szolgáltatás egyetlen szálon dolgozik, vagy egy központi adatbázisra támaszkodik minden kérésnél, a horizontális skálázás hamar eléri a határait. Az autoscaling a kapacitásigényt igazítja, de nem helyettesíti a bottleneck elemzést.

17.2.10 Tipikus hibák és elkerülésük

A gyakorlatban számos visszatérő hiba rontja a konténeres rendszerek megbízhatóságát. Az alábbiakban a leggyakoribbakat gyűjtöttük össze, a helyes megoldással együtt.

1. **A `latest` image tag használata.** A `latest` tag nem determinisztikus: a deployment során más kép is lehívható, mint amit a tesztelés során használtunk. Mindig explicit, verziózott taget – például git commit hash vagy szemantikus verzió – használjunk.
2. **Hiányzó erőforrás-korlátok.** Korlátok nélkül egy pod az egész node-ot leterhelheti. Minden podra adjunk meg requests és limits értékeket, és a minőségi osztályt tudatosan válasszuk meg.
3. **A readiness probe hiánya.** Ha nincs readiness probe, a Kubernetes akkor is a podhoz irányítja a forgalmat, amikor az még nem állt fel, vagy már túlterhelt. A readiness probe elengedhetetlen a rolling update során is.
4. **A liveness probe túl agresszív.** Ha a liveness probe túl rövid időn belül többször hibázik, a pod újraindul, ami további terhelést okoz. A liveness probe-ot úgy kell beállítani, hogy csak valódi elakadás esetén jelezzen hibát.
5. **A pod közvetlen létrehozása productionben.** A pod önmagában nem öngyógyító. Mindig deployment vagy más workload-objektum felügyelje a podokat.

6. **A service selector és a pod label eltérése.** Ha a service selector nem egyezik a pod labeljeivel, a service nem talál podokat, és a forgalom nem érkezik meg. Ez a hiba gyakran apró elírásból adódik.
7. **A HPA és a deployment replikaszámának konfliktusa.** Ha a deployment specifikációjában explicit replikaszámot adunk meg, és a HPA is módosítja azt, a két érték összeütközhet. A HPA használatakor a deployment specifikációjában ne adjunk meg fix replikaszámot, vagy hagyjuk, hogy a HPA felülírja.
8. **A storage perzisztenciájának figyelmen kívül hagyása.** A konténer írható rétege elveszik a pod újraindításakor. Perzisztens adatokat mindig PersistentVolumeClaimbe vagy külső adatbázisba írjunk.
9. **A TLS-terminálás elhanyagolása.** Az ingress controller előtt gyakran terheléselosztó áll, amelynek szintén TLS-t kell kezelnie. A kettős titkosítás vagy a titkosítás teljes hiánya egyaránt hiba lehet. A titkosítási pontot tudatosan kell megválasztani.
10. **A megfigyelhetőség hiánya.** A konténeres rendszerek bonyolultak, ezért a naplókat, metrikákat és trace-eket már a kezdetektől gyűjteni kell. A Kubernetes natív módon nem biztosít aggregált naplózást, ezért külső megfigyelhetőségi stacket kell telepíteni.

17.2.11 Labor: Kubernetes deployment és service vázlat

Ez a labor egy állapotmentes webalkalmazás Kubernetes deploymentjének és service-ének elkészítését gyakoroltatja. A cél nem egy éles rendszer telepítése, hanem a deklaratív konfiguráció, az erőforrás-korlátok és a health checkek átgondolása. A példák helyőrzőket használnak.

17.2.12 Kiinduló állapot

Rendelkezésre áll egy futó Kubernetes-klaszter (például minikube, kind vagy felügyelt szolgáltatás), valamint a `kubectl` parancssori eszköz. Az alkalmazás egy egyszerű HTTP API, amely a `8080` porton figyel, és rendelkezik egy `/healthz` végponttal, amely 200-as választ ad, ha az alkalmazás és függőségei rendben vannak, és egy `/readyz` végponttal, amely 200-as választ ad, ha az alkalmazás készen áll a forgalomra. A kép a `registry.example.com/my-app:v1.2.3` néven érhető el – ez helyőrző, a laborban saját image nevet kell használni.

17.2.13 Elvárt eredmény

Készíts el egy `deployment.yaml` és egy `service.yaml` fájlt, amelyek a következőket tartalmazzák:

- A deployment három replikát futtat a `my-app` konténerből.
- A podok CPU requestje 250m, limitje 500m; memória requestje 256Mi, limitje 512Mi.
- A liveness probe a `/healthz` végpontot hívja 10 másodpercenként, 5 másodperces kezdeti késleltetéssel, 3 hibás próbálkozás után újraindítva a konténert.
- A readiness probe a `/readyz` végpontot hívja 5 másodpercenként, 2 másodperces kezdeti késleltetéssel.
- A service `ClusterIP` típusú, a 80-as porton fogadja a forgalmat, és a podok 8080-as portjára továbbítja.
- A service selector pontosan megegyezik a podok labeljeivel (például `app: my-app`).

17.2.14 Ellenőrzési szempontok

A labor akkor tekinthető sikeresnek, ha a következők mindegyike teljesül:

- A `kubectl apply -f deployment.yaml -f service.yaml` parancs hiba nélkül lefut.
- A `kubectl get pods` parancs három podot mutat `Running` állapotban, és mindegyik `READY 1/1`.
- A `kubectl get service my-app` parancs megjeleníti a ClusterIP-t és a port-forwarding beállítást.
- A `kubectl describe pod <pod-név>` parancsban láthatók a liveness és readiness probe beállításai, valamint a resource requests és limits.
- Ha a readiness probe végpontja 500-as hibát ad, a pod kikerül a service endpointjai közül, de nem indul újra. Ha a liveness probe végpontja 500-as hibát ad, a konténer újraindul.

17.2.15 Döntési napló

A labor során rögzíts ADR formában a következő döntéseket:

- Miért három replikát választottál induláskor?
- Miért ezeket a CPU- és memóriaértékeket választottad?
- Miért különbözteted meg a liveness és readiness probe-ot?

Például: „A három replika elegendő a várható alapforgalom kiszolgálására, és egy node kiesése esetén is legalább két replika fut. Az erőforrás-korlátok a terhelési teszt 80. percentiliséből származnak, 30%-os tartalékkal. A liveness probe csak a folyamat életét ellenőrzi, a readiness pedig a függőségek elérhetőségét is, ezért külön végpontokat használunk.”

17.2.16 Visszaállítási út

A labor elvégzése után a `kubectl delete -f deployment.yaml -f service.yaml` paranccsal törölhetők az erőforrások. A klaszter többi erőforrása érintetlen marad. Ha a labor során hibás konfigurációt alkalmaztál, a `kubectl delete` után újraalkalmazhatod a javított fájlokat.

17.2.17 Ellenőrzőlista

- Minden pod rendelkezik erőforrás-korlátokkal (requests és limits).
- A podok rendelkeznek liveness és readiness probe-bal, amelyek különböző szempontokat ellenőriznek.
- A deploymentek explicit, verziózott image taget használnak.
- A service selector pontosan megegyezik a podok labeljeivel.
- Az ingress szabályok csak a szükséges hosztokat és útvonalakat engedik be.
- Az autoscaling beállítások a mért terhelési adatokon alapulnak, és a maximális replikaszám véd a költségrobbanás ellen.
- A titkosítási pontok (ingress, service mesh) tudatosan vannak kialakítva.
- A megfigyelhetőségi stack (napló, metrika, trace) már a kezdetektől működik.

17.2.18 Önellenőrző kérdés

Magyarázd el, miért nem elegendő a liveness probe önmagában, és milyen hibát okozhat, ha ugyanazt a végpontot használjuk a liveness és readiness ellenőrzésre. Adj példát arra, hogyan vezethet ez a pod felesleges újraindulásához vagy a forgalom rossz irányításához.

17.3 Serverless, FaaS és managed szolgáltatások

A konténeres futtatás a felhő natív világ egyik végletét jelenti: a csomagolást és az ütemezést szabványosítja, de a klaszter, a csomópontok és az alapinfrastruktúra üzemeltetése továbbra is a csapat felelőssége. A másik véglet a serverless, amely ezt a felelősséget radikálisan csökkenti. Ahhoz, hogy jól döntsünk a kettő között, pontosan értenünk kell, mit takar a „menedzselt szolgáltatás” és a „függvényalapú futtatás” fogalma, hol vannak a valódi határok, és melyek azok az új kockázatok, amelyeket a kényelemért cserébe elfogadunk.

17.3.1 Menedzselt szolgáltatások és a felelősségi spektrum

A felhőszolgáltatások legegyszerűbb osztályozása az alapján készül, hogy az üzemeltetési verem mely rétegeiért ki a felelős. A klasszikus IaaS (Infrastructure as a Service) esetén a szolgáltató a fizikai hardvert, a virtualizációt és az alapvető hálózatot adja, de az operációs rendszer, a futtatókörnyezet, az adatbázis és a skálázás a miénk. A PaaS (Platform as a Service) ebből átvállalja az operációs rendszert és a futtatókörnyezetet, a SaaS (Software as a Service) pedig a teljes alkalmazást – az ügyfél legfeljebb konfigurál.

A FaaS (Function as a Service) ebbe a spektrumba illeszkedik, de érdemes külön kategóriaként kezelni: a futtatási egység nem egy folyamatosan futó szerver vagy konténer, hanem egy eseményre meghívott függvény. A platform felelőssége a függvény elindítása, a párhuzamos végrehajtás, a skálázás és a számlázás – egészen addig, amíg a függvény fut. A fejlesztő felelőssége a kód, a konfiguráció, a jogosultságok és az integrációk kezelése.

A menedzselt szolgáltatások közös jellemzője, hogy a szolgáltató API-n keresztül ad el egy ígért viselkedést, és ehhez SLA-t, skálázási korlátokat vagy kvótákat rendel. A szolgáltató általában többet vállal a frissítésekből, a replikációból és a biztonsági javításokból, de a felelősség soha nem tűnik el teljesen. Egy menedzselt adatbázis esetén például a mentéseket, a verziófrissítést és a failovert a szolgáltató kezeli, de az adatmodell, a séma migráció és a jogosultsági politika továbbra is az alkalmazás tulajdonosáé.

A konténeres Kubernetes (lásd Konténerek, Kubernetes és orchestráció) és a serverless között az egyik legfontosabb különbség a kérdés, hogy **mi az a legkisebb üzemeltetési egység, amelyért felelősséget vállalunk**. Kubernetesen ez a pod, a deployment, a service – a klasztert még mindig frissíteni, skálázni és javítani kell, még ha az orchestrátor sokat automatizál is. FaaS-on ez a függvény, és a skálázás a platform belügye. Ezért a serverless-t gyakran úgy írják le, mint ahol „nincs szerver” – ez természetesen félrevezető, hiszen a szerverek léteznek, csak nem a mi számlánkon és nem a mi üzemeltetési felelősségünk alatt.

17.3.2 Függvényalapú futtatás (FaaS)

A FaaS modellben a fejlesztő egy egyszerű, eseményvezérelt függvényt ír, amelynek bemenete egy esemény, kimenete pedig valamilyen válasz vagy mellékhatás. A platform a függvényt csak akkor indítja el, amikor az adott esemény bekövetkezik. Az életciklus tipikus lépései:

1. A fejlesztő a függvény kódját és a hozzá tartozó konfigurációt (memória, timeout, környezeti változók, jogosultságok) feltölti a platformra.
2. A platform regisztrál egy vagy több triggert, például HTTP végpontot, üzenetsort vagy objektumtároló eseményt.
3. Amikor egy esemény megérkezik, a platform létrehoz egy végrehajtási környezetet – gyakran egy konténert –, betölti a kódot, meghívja a handlert, majd a válasz elküldése után leállítja vagy elalszik.
4. Ha több esemény érkezik párhuzamosan, a platform további példányokat indít, és a terhelés csökkenésekor nullára skáláz.

Ez a modell alapvetően eltér a konténeres modelltől: ott egy hosszú életű processz, például egy HTTP szerver folyamatosan fut, és a requesteket ugyanaz a példány szolgálja ki. A FaaS esetén a függvény a gyakorlatban úgy viselkedik, mintha minden egyes eseményre „frissen" indulna – még akkor is, ha a platform a háttérben újrahasznosítja a futtatókörnyezetet. Ennek közvetlen következménye a **hidegindítás** jelensége, és közvetlen előnye a **scale to zero**: ha nincs forgalom, nincs számlázott kapacitás.

A függvényalapú futtatás akkor működik jól, ha a függvény rövid életű, állapotmentes, és a külső állapotot jól definiált interfészeken keresztül éri el. A platformok nem garantálják, hogy két egymást követő meghívás ugyanabban a processzben fut, ezért a függvényben nem szabad lokális állapotra építeni. A tipikus hibák közé tartozik a memóriában tárolt cache, a befejezetlen tranzakció vagy a fájlrendszerbe írt ideiglenes adat – ezek mind elveszhetnek, ha a platform eldobja a futtatókörnyezetet.

17.3.3 Hidegindítás és teljesítményjellemzők

A hidegindítás (cold start) az a jelenség, amikor a FaaS platform egy új végrehajtási környezetet hoz létre, mert nincs már futó, „meleg" példány, amely az eseményt ki tudná szolgálni. A hidegindítás tipikus lépései:

- a konténerkép letöltése vagy előkészítése,
- a nyelvi futtatókörnyezet inicializálása,
- a függvény kódjának betöltése,
- a handler egyszeri, a függvényen kívüli kódjának lefuttatása (például adatbázis-kapcsolat létrehozása).

A meleg és a hideg indítás közötti különbség száz milliszekundumtól több másodpercig terjedhet. A pontos érték szolgáltatótól, régiótól, nyelvtől, a csomag méretétől és a konfigurált erőforrásoktól függ; a 2025 elején jellemző Node.js hidegindítási idők gyakran 300–800 ms közé esnek, de ezek az adatok gyorsan változnak, ezért minden döntés előtt saját mérést kell végezni.

A hidegindítás hatása nem csak a válaszidő átlagában mérhető, hanem a percentilisekben is. Egy olyan szolgáltatásnál, ahol a forgalom egyenetlen, a hidegindítás a p99 vagy p95 értéket jelentősen megemelheti, miközben az átlag alig változik. Ezért a teljesítménykövetelményeket minőségi forgatókönyvben (quality attribute scenario) kell rögzíteni, például:

> **Forgatókönyv:** Egy HTTP kérés érkezik az API Gateway-re (inger). A rendszer éppen nyugalmi állapotban van, azaz nincs meleg függvénypéldány (környezet). A kérés 500 ms alatt 200-as választ kap (válasz). A mérőszám: a p99 válaszidő 500 ms alatt marad, 10 perc mérési ablakban, 5 hidegindítást generáló teszt alapján.

A hidegindítás mérséklésére több technika létezik:

- **Provisioned concurrency:** előre fizetünk és fenn tartunk meleg példányokat, ami a hidegindítást megszünteti, de elvész a scale to zero költségelőnye.
- **Könnyű csomag:** a kisebb csomag gyorsabban töltődik be, ezért kerüljük a nagy könyvtárakat.
- **Inicializálás elhalasztása:** a drága beállításokat a handleren kívülre tesszük, hogy a meleg példány újrahasználja őket, de ügyeljünk arra, hogy a hidegindítás során ne akadjon el.
- **Meleg tartás:** bizonyos platformok támogatják az időzített hívásokat, hogy a függvény melegen maradjon, de ez szintén költséget generál.

A Little-törvény jól mutatja a hidegindítás rendszerbeli hatását. Ha egy függvény átlagos tartózkodási ideje W a hidegindítás miatt megnő, akkor adott λ érkezési ráta mellett a rendszerben tartózkodó kérések átlagos száma $L = \lambda W$ is nő. Ez azt jelenti, hogy a platformnak több párhuzamos példányt kell indítania, ami növeli a konkurens végrehajtások számát, és ezzel a költségeket és a kvótakockázatot. Ezért a hidegindítást nem csak latenciaproblémaként, hanem kapacitástervezési tényezőként is kezelni kell.

17.3.4 Eseményforrások és triggerek

A FaaS függvény soha nem csak önmagában létezik; mindig egy eseményforráshoz kapcsolódik. Az eseményforrás lehet:

- **HTTP kérés:** API Gateway vagy function URL közvetíti.
- **Üzenetsor:** a platform lehívja a függvényt, amikor üzenet érkezik (például SQS, Pub/Sub).
- **Objektumtároló esemény:** például fájl feltöltése egy S3 bucketbe.
- **Időzítő:** cron-szerű ütemezés.
- **Eseménybusz:** más rendszerekből származó események.

A trigger konfigurációja meghatározza, hogy a platform milyen formában továbbítja az eseményt a függvénynek, és milyen retry-politikát alkalmaz. A legtöbb platform legalább egyszeri kézbesítést garantál, ami azt jelenti, hogy egy esemény többször is meghívhatja a függvényt. Ezért a függvénynek **idempotensnek** kell lennie: ugyanaz a bemeneti esemény többszöri feldolgozása nem okozhat duplikált mellékhatást. A gyakori hiba, hogy a fejlesztő feltételezi, hogy a függvény csak egyszer fut le, és például egy számlálót naivan inkrementál az adatbázisban.

Egy másik fontos szempont a sorrend és a párhuzamosság. Ha egy üzenetsorból több üzenetet fogyaszt a platform, nem garantálja a feldolgozás sorrendjét, hacsak nincs FIFO konfiguráció. A függvény kialakításának ezt figyelembe kell vennie. Ha a feldolgozás sorrendje kritikus, akkor vagy FIFO üzenetsort használunk, vagy az alkalmazás szintjén kezeljük a rendezést (például eseménysorozat-azonosítóval). A halott betűs sor (dead-letter queue) beállítása segít abban, hogy a tartósan nem feldolgozható üzenetek ne ragadjanak be, hanem külön vizsgálhatóvá váljanak.

Az eseményforrás megválasztása hatással van a hibatűrésre és a megfigyelhetőségre. Egy HTTP-triggernél a hiba azonnal visszajut a hívóhoz; egy üzenetsoros triggernél a platform újrapróbálhatja a feldolgozást, mielőtt a halott betűs sorba továbbítaná. A jól kialakított rendszer mindkét esetben explicit módon rögzíti a hiba utáni teendőket.

17.3.5 Kvóták és korlátok

A serverless platformok nem korlátlanul skálázódnak; minden szolgáltatás működési limiteket szab meg. Ezek a kvóták gyakran a következőkre vonatkoznak:

- maximális futtatási idő (például 15 perc egy függvény számára),
- maximális memória és CPU,
- maximális párhuzamos végrehajtások száma régióban,
- a csomag mérete (tömörítve és tömörítetlenül),
- a környezeti változók mérete,
- a payload mérete a trigger és a válasz esetén,
- a függvény által megnyitható hálózati kapcsolatok vagy fájlleírók száma.

Ezek a limitek szolgáltatónként és gyakran régióként is eltérnek, és a szolgáltatók idővel módosítják őket. A tervezés során ezért nem szabad egyetlen pillanatfelvételből kiindulni, hanem mindig a hivatalos dokumentációt kell ellenőrizni. A kvóták nem pusztán technikai kellemetlenségek, hanem **tervezési kényszerek**: ha egy feldolgozási feladat hosszabb a maximális futtatási időnél, akkor a folyamatot fel kell darabolni; ha a payload túl nagy, akkor az objektumtárolóra kell hivatkozni; ha a párhuzamos végrehajtások száma korlátba ütközik, akkor a fogyasztói oldalon kell szabályozni a bemenő terhelést.

A tipikus hiba az, hogy a fejlesztő a fejlesztési környezetben soha nem ütközik a kvótába, és csak élesben, csúcsterhelés alatt szembesül vele. A kvótákat ezért a tervezési fázisban, a kapacitástervezés részeként elemezni kell. A Little-törvény itt is segít: ha ismerjük a várható λ érkezési rátát és az átlagos feldolgozási időt, akkor kiszámíthatjuk a szükséges párhuzamos végrehajtási szintet, és összehasonlíthatjuk a kvótával. Ha a szükséges szint közelít a kemény korláthoz, akkor vagy növelni kell a kvótát (ha lehetséges), vagy csökkenteni kell a feldolgozási időt, vagy vissza kell terhelni a forrást.

17.3.6 Üzemeltetési teher és a NoOps illúzió

A serverless egyik fő ígérete az üzemeltetési teher csökkentése. Ez valós előny: nem kell operációsrendszer-frissítést ütemezni, nem kell csomópontokat skálázni, nem kell a konténer-orchestrátor verzióját menedzselni. A platform átvállalja a szerverek és a futtatókörnyezet üzemeltetését, ami kis csapatoknál jelentős megtakarítást hozhat.

Azonban az üzemeltetési teher nem tűnik el, hanem átalakul. A serverless rendszer üzemeltetése új feladatokat hoz:

- **Megfigyelhetőség:** minden függvényhez tartozik log, metrika és trace; ezeket gyűjteni, riasztani és elemezni kell.
- **Konfiguráció és titkok:** a függvények konfigurációja, környezeti változói és jogosultságai sokasodnak, és ezeket verziózni, auditálni kell.
- **Hidegindítási teljesítmény:** a hidegindítások monitorozása, a p99 romlásának észlelése és a provisioned concurrency beállítása operációs döntés.
- **Költségmenedzsment:** a scale to zero olcsó lehet, de egy rosszul kialakított trigger vagy egy végtelen ciklus hatalmas számlát okozhat; a költségfigyelés kritikus.
- **Biztonság:** minden függvénynek saját jogosultsági köre van, és a legkisebb jogosultság elvét folyamatosan fenn kell tartani.

A „NoOps” kifejezés ezért félrevezető: a serverless nem szünteti meg az üzemeltetést, hanem **magasabb absztrakciós szintre helyezi**. Az üzemeltetési teher mérhető: például a havi on-call riasztások száma, a deploymentek gyakorisága, a manuális beavatkozást igénylő incidensek aránya. Ha egy szervezet a serverless bevezetése után csak annyit lát, hogy a riasztások száma csökkent, de a számla vagy a hidegindítási panaszok nőttek, akkor az üzemeltetési teher valójában nem csökkent, csak áthelyeződött.

A fitness functions elv (lásd Fitness functions és folyamatos értékelés) itt is hasznos. Ilyen lehet például:

- Minden függvény rendelkezik observability konfigurációval (log, trace).
- A függvények p99 válaszideje hidegindítás esetén sem haladja meg az SLO-t.
- A havi költség nem lépi túl az előre meghatározott FinOps keretet.
- A függvények jogosultsági körei automatikusan ellenőrzöttek.

Ezek a fitness functionök folyamatosan jelzik, ha a rendszer eltér az elvárt üzemeltetési szinttől.

17.3.7 Vendor lock-in

A serverless platformok erős termékhez kötöttséget, **vendor lock-in**-t okoznak. Ennek több forrása van:

- A FaaS platform API-ja, a trigger konfiguráció, az eseményséma és a helyi fejlesztői eszközök szorosan integrálódnak a szolgáltató ökoszisztémájába.
- A menedzselt szolgáltatások – például az adatbázis, az üzenetsor, az API Gateway – mind saját interfésszel rendelkeznek, és a köztük lévő integráció mélyen a szolgáltató specifikus tulajdonságaira épül.
- A hidegindítási viselkedés, a kvóták és a számlázási modell eltérő, ezért egy migráció nem egyszerű konfigurációcsere.

A lock-in nem feltétlenül rossz: ha a szolgáltató megbízható, a képességei illeszkednek, és a csapat sebességet nyer, akkor a bezáródás racionális döntés lehet. A kockázat akkor materializálódik, ha a szolgáltató árat emel, megszünteti a szolgáltatást, vagy a szervezet stratégiai okból több felhőt szeretne használni. A lock-in mértékét csökkenteni lehet:

- **Szabványos interfészek:** például a HTTP API-t szabványos módon, keretrendszerfüggetlenül tervezzük; a függvény kódja ne tartalmazzon szolgáltató-specifikus SDK-t, csak a szükséges minimális mértékben.
- **Absztrakciós réteg:** egy vékony adapterréteg a menedzselt szolgáltatások körül, amely mögött a konkrét API-hívásokat cserélni lehet.
- **Többfelhős futtatás:** a függvényeket olyan keretrendszerrel (például Serverless Framework, Terraform) telepíteni, amely több szolgáltatót is támogat.

Fontos látni, hogy az absztrakció ára a komplexitás és a feladott natív előnyök. Egy olyan absztrakciós réteg, amely minden szolgáltatót egyformán kezel, gyakran a legkisebb közös többszörösre egyszerűsíti a rendszert, és lemond a platform egyedi előnyeiről. A döntésnél tehát a lock-in kockázatát és az absztrakció költségét együtt kell mérlegelni. A következő fejezet (Multi-cloud, hybrid cloud és lock-in) részletesebben foglalkozik a multi-cloud és a lock-in stratégiákkal, itt elég, ha az olvasó felismeri a trade-offot.

17.3.8 Mikor válasszunk serverless-t?

A serverless nem jobb vagy rosszabb, mint a konténeres megoldás; más követelményekre illik. A választásnál a minőségi attribútumok prioritásából érdemes kiindulni.

A serverless erős oldala:

- **Változó, kiszámíthatatlan terhelés:** a platform automatikusan skáláz, és nulla forgalomnál közel nulla a költség.

- **Eseményvezérelt feldolgozás:** rövid, jól körülhatárolt feladatok, például képfeldolgozás, értesítésküldés, API-végpontok.
- **Gyors fejlesztés és kicsi csapat:** az üzemeltetési teher egy része átkerül a szolgáltatóhoz, így a fejlesztők több időt fordíthatnak a funkcionalitásra.

A serverless gyenge pontjai:

- **Folyamatos, nagy terhelés:** ha a forgalom állandóan magas, a provisioned concurrency és a folyamatos futás költsége meghaladhatja a konténeres üzemeltetést.
- **Hosszú futás vagy nagy memóriaigény:** a kvóták korlátozzák a futtatási időt és az erőforrásokat.
- **Állapotos alkalmazások:** a függvények közötti állapotmegosztás összetett, és a hálózati késleltetés dominálhat.
- **Speciális hardver vagy alacsony szintű vezérlés:** ha GPU-ra, egyedi kernelre vagy hosszú életű TCP-kapcsolatra van szükség, a FaaS kevésbé alkalmas.

A gyakorlatban gyakran hibrid megoldás születik: a felhasználó felé néző API réteg serverless, a háttérfeldolgozás konténerben fut, vagy fordítva. A döntést ADR-ben kell rögzíteni, a kontextus, az alternatívák és a következmények feltüntetésével.

17.3.9 Labor: serverless és konténeres megoldás összehasonlítása

Cél: Két egyenértékű funkcionalitású szolgáltatás – egy konténerben futó HTTP API és egy FaaS-on futó HTTP API – összehasonlítása a teljesítmény, a költség, az üzemeltetési teher és a skálázhatóság szempontjából. A labor eredménye egy reprodukálható dokumentum vagy jegyzetfüzet, amely ADR-t, mérési jegyzőkönyvet és ellenőrzési szempontokat tartalmaz.

Kiinduló állapot: Van egy egyszerű, jól definiált feladat: egy JSON-ben érkező szöveg hosszát és szószámát visszaadó API. A konténeres verzió egy Kubernetes Deploymentben fut, 2 replikával, 512 MiB memórialimittel. A FaaS verzió ugyanazt a logikát valósítja meg egy függvényben, 512 MB memóriával. A terhelésgeneráló eszköz (például k6) már telepítve van. A mérési környezet izolált, nem termelési.

Kísérleti terv:

1. **Kontextus rögzítése:** Írjuk le a feltételezett éles környezetet: csapatméret 3 fő, várható terhelés 1–50 RPS között ingadozik, SLO: p99 < 500 ms, havi költségkeret 200 USD, régió: <régió>, szabályozási kényszer: nincs adatkijuttatás az EU-n kívülre.
2. **Hipotézis:** A serverless alacsony terhelésnél olcsóbb, de hidegindítási latenciát mutat; a konténeres verzió stabilabb p99-et ad, de folyamatosan fizetünk a replikákért.
3. **Mérési metrikák kiválasztása:** p50, p99, hidegindítási arány, havi becsült költség, deployment idő, üzemeltetési riasztások száma.
4. **Telepítés:** A FaaS függvényt hozzuk létre a felhőkonzolon vagy IaC segítségével (például Terraform). A konténeres verziót a meglévő klaszterre deployoljuk. A konfigurációt verziózzuk, a környezetet dokumentáljuk.
5. **Terhelésgenerálás:** Futtassunk 10 perces terhelési tesztet, két fázisban: először nyugalmi állapotból, hogy hidegindítást váltsunk ki, majd folyamatos terheléssel. Rögzítsük a mérési ablakot, a terhelési profilt és a konfigurációt.
6. **Adatgyűjtés:** A platform metrikáit (meghívások száma, p99, hidegindítások száma) és a költségkimutatásokat exportáljuk. A konténeres verzió esetén a Kubernetes metrikákat (CPU, memória, request latencia) gyűjtsük.
7. **ADR készítése:** Az összehasonlítás alapján hozzunk döntést, és rögzítsük ADR formátumban: kontextus, döntés, alternatívák, következmények, állapot.

Minta ADR-részlet:

> **Cím:** API-futtatás: serverless vagy konténeres **Kontextus:** A rendszer 1–50 RPS között ingadozó terhelést kap, a csapat 3 fő, az SLO p99 $<$ 500 ms. A konténeres verzió jelenleg 2 replikával fut, a költség havi 150 USD. A FaaS esetén hidegindítási kockázat van. **Döntés:** A megfigyelt p99 a FaaS esetén hidegindítással 850 ms, meleg indítással 60 ms, a konténer esetén 120 ms stabilan. Mivel a forgalom 70%-a hidegindítást okoz, és az SLO sérül, a döntés a provisioned concurrency bevezetése a FaaS-on, vagy a konténeres verzió megtartása. A választás a költségmodell alapján: provisioned concurrency 2 példányra havi 180 USD, ami még mindig elfogadható. A döntés: FaaS provisioned concurrency-vel. **Alternatívák:** 1) Konténeres verzió megtartása – stabil, de üzemeltetési teher. 2) Konténeres verzió skálázása – költségesebb. 3) FaaS provisioned concurrency nélkül – SLO sérül. **Következmények:** A hidegindítási kockázat csökken, a költség nő, de a kereten belül marad. A lock-in nő, mert a provisioned concurrency szolgáltató-specifikus. A csapatnak meg kell tanulnia a FaaS platform üzemeltetési eszközeit.

Ellenőrzési szempontok:

- A mérési jegyzőkönyv tartalmazza a környezetet, a konfigurációt, az időpontot és a bizonytalanságot.
- A mért és a várt eredmény külön van választva; a várt eredményt a kísérlet előtt rögzítettük.
- Az ADR a döntési naplóban visszakereshető.
- A dokumentum tartalmazza a C4 context diagramot Mermaid vagy PlantUML formában, amelyen a felhasználó, az API, a FaaS és a konténeres rendszer határa látható.
- A laborban nem használunk valós hitelesítő adatot; minden konfiguráció helyőrző (pl. `<account-id>`, `<region>`).

Visszaállítási út: A létrehozott FaaS függvényt és a hozzá tartozó trigger konfigurációt töröljük. A Kubernetes deployment esetén a próbapéldányokat leállítjuk. A terhelésgeneráló eszközt leállítjuk. A mérési adatokat archiváljuk, de a környezetben hagyott erőforrásokat felszámoljuk.

Elvárt eredmény: A résztvevő képes a két lehetőség közötti döntést mérhető adatokra alapozva meghozni, és a következményeket ADR-ben rögzíteni.

17.3.10 Ellenőrzőlista és önellenőrző kérdés

A fejezet végén a következőket érdemes ellenőrizni:

- Ismered-e a menedzselt szolgáltatások felelősségi határait, és meg tudod-e mondani, mi marad a te felelősséged?
- Képes vagy-e megmagyarázni a hidegindítás mechanizmusát és mérni a hatását a p99 válaszidőre?
- Figyelembe veszed-e a kvótákat a tervezés során, és használod-e a Little-törvényt a párhuzamos végrehajtási szint becslésére?
- Biztosítod-e az idempotenciát az eseményfeldolgozásban?
- Értékeled-e a vendor lock-in kockázatát és az absztrakció költségét?
- Felismered-e, hogy az üzemeltetési teher átalakul, nem eltűnik, és vannak-e fitness functionjeid a serverless rendszer minőségének fenntartására?

Önellenőrző kérdés: Egy cég egy képfeldolgozó szolgáltatást szeretne futtatni, amely ritkán, de időnként nagy méretű képeket dolgoz fel. Egy feltöltés hatására átlagosan 20 másodpercig tart a feldolgozás, és előfordulhat, hogy órákig nincs forgalom, majd hirtelen 100 feltöltés érkezik. Milyen érvek szólnak a serverless és a konténeres megoldás mellett? Hogyan mérnéd a hidegindítás hatását, és milyen kvótákat kellene először ellenőrizni? Válaszodban térj ki az idempotencia, az eseményforrás és a lock-in kérdésére is.

17.4 Multi-cloud, hybrid cloud és lock-in

A felhőstratégia kialakítása során a „melyik szolgáltatót válasszuk?” kérdés mellett legalább ennyire fontos az a kérdés, hogy „mennyire engedjük magunkat egyetlen szolgáltatóhoz kötni?”. A multi-cloud, a hybrid cloud és a lock-in témája gyakran érzelmi vitává válik: egyesek a szolgáltatófüggést mindenáron elkerülendő kockázatnak tekintik, mások szerint a felhőszolgáltatók közötti hordozhatóság illúzió, és az erőforrásokat inkább a kiválasztott platform mély kihasználására kell fordítani. Ez a fejezet a két véglet között keres racionális keretet: bemutatja az alapfogalmakat, a döntési szempontokat, a portabilitás és az adatlokáció kérdéseit, valamint a lock-in költségének és a kilépési stratégiának a mérését. A cél nem az, hogy a szolgáltatófüggést minden áron elkerüljük, hanem az, hogy tudatosan, mérhető következményekkel és előre rögzített kilépési tervvel vállaljuk.

17.4.1 Alapfogalmak: multi-cloud, hybrid cloud és lock-in

A fogalmak tisztázása azért fontos, mert a gyakorlatban gyakran keverednek, és a pontatlan szóhasználat rossz döntésekhez vezet.

Multi-cloud alatt azt értjük, amikor egy szervezet több nyilvános felhőszolgáltatót használ, akár párhuzamosan, akár különböző munkaterhelésekre. A multi-cloud lehet tudatos stratégia eredménye, de gyakran esetlegesen alakul ki: az egyik csapat az egyik szolgáltatónál indít el egy környezetet, a másik csapat egy másiknál, és a végén a szervezet több felhőt is használ anélkül, hogy ezt valaha is átgondolta volna. A multi-cloud tehát nem feltétlenül jelent hordozhatóságot; lehet csupán párhuzamos beszerzés.

Hybrid cloud alatt a nyilvános felhő és a saját adatközpont (vagy privát felhő) összekapcsolását értjük. A hybrid jellemzően akkor merül fel, amikor a szervezetnek vannak olyan munkaterhelései, amelyeket szabályozási, késleltetési, költség- vagy biztonsági okból nem akar vagy nem tud nyilvános felhőbe tenni. A hybrid cloud tehát nem egyszerűen két környezet egyidejű használata, hanem a kettő közötti folyamatos integrációt, adatáramlást és közös üzemeltetési modellt is jelent.

A **lock-in** (szolgáltatófüggés) azt a költséget vagy kockázatot jelöli, amelyet a szolgáltatóváltás vagy a rendszer más környezetbe való átvitele okoz. A lock-in nem bináris állapot, hanem mérték kérdése: minél több provider-specifikus szolgáltatást, API-t, adatformátumot vagy üzemeltetési mechanizmust használunk, annál nagyobb a váltás költsége. Fontos megkülönböztetni a **vendor lock-in**-t (egy adott szolgáltatóhoz kötöttség) és az **architecture lock-in**-t (egy adott technológiai választáshoz kötöttség, például egy zárt forráskódú adatbázis-formátum). A kettő gyakran összefonódik, de nem ugyanaz: lehet egy nyílt forráskódú adatbázist használni úgy, hogy a felhőszolgáltató menedzselt változatához vagyunk kötve, és lehet egy provider-specifikus szolgáltatást használni úgy, hogy annak van nyílt API-ja vagy exportálható adatmodellje.

A lock-in nem feltétlenül rossz. A szolgáltatók menedzselt szolgáltatásai gyakran jelentős termelékenységnövekedést hoznak, és a váltás költsége tudatosan vállalható. A probléma akkor

keletkezik, ha a lock-in rejtett: a döntés idején nem mérjük fel a váltás költségét, nincs kilépési terv, és a szervezet később kiszolgáltatottá válik áremelés, szolgáltatásmegszűnés vagy szabályozási változás esetén.

17.4.2 Felhőstratégia és a döntési keret

A felhőstratégia nem csupán technológiai kérdés: a szervezet üzleti céljaihoz, kockázatvállalási hajlandóságához, szabályozási környezetéhez és csapatai képességeihez kell illeszkednie. A stratégia kialakításakor először azt érdemes rögzíteni, hogy **miért** használunk felhőt, és **milyen szerepet** tölt be a felhő a rendszer életciklusában.

A leggyakoribb stratégiai indokok:

- **Skálázhatóság és rugalmasság:** a munkaterhelés dinamikusan változik, és a felhő a fizetős modell miatt jobban követi a terhelést, mint egy fix kapacitású adatközpont.
- **Piaci sebesség:** a menedzselt szolgáltatásokkal gyorsabban lehet új funkciókat kiadni, mint saját infrastruktúra üzemeltetésével.
- **Költségoptimalizálás:** a felhő a tőkekiadásokat működési kiadásokká alakítja, de ez nem jelenti automatikusan az alacsonyabb teljes költséget.
- **Megfelelés és adatlokáció:** bizonyos piacokon a felhőszolgáltatók rendelkeznek olyan tanúsítványokkal, amelyeket egy saját adatközpontban nehéz lenne megszerezni.
- **Innovációhoz való hozzáférés:** a felhőszolgáltatók folyamatosan bővülő szolgáltatáskatalógusa lehetővé teszi új technológiák gyors kipróbálását.

A stratégia kialakításakor fel kell tenni a következő kérdéseket:

1. **Mely munkaterhelések futhatnak nyilvános felhőben, és melyek igényelnek saját környezetet?** Ezt szabályozási, adatvédelmi, késleltetési és költség-szempontok határozzák meg.
2. **Egyetlen szolgáltatót válasszunk, vagy többet?** Ha többet, akkor az egyes szolgáltatók milyen szerepet kapnak: aktív-aktív, aktív-passzív, vagy különböző munkaterhelésekre specializált?
3. **Mennyire engedjük meg a provider-specifikus szolgáltatások használatát?** Ez a kérdés a termelékenység és a hordozhatóság közötti trade-offot feszegeti.
4. **Milyen a kilépési terv?** Minden szolgáltatóra vonatkozóan tudnunk kell, hogyan tudjuk az adatainkat kivonni, a szolgáltatásokat leállítani és a rendszert máshová átvinni.

A felhőstratégia gyakran **cloud-first** vagy **cloud-smart** néven jelenik meg. A cloud-first azt jelenti, hogy új rendszer esetén alapértelmezetten a nyilvános felhőt választjuk, kivéve ha nyomós ellenérv van. A cloud-smart ezzel szemben azt hangsúlyozza, hogy a felhő nem cél, hanem eszköz: a döntést minden munkaterhelésre külön kell meghozni, figyelembe véve a költségeket, a megfelelést és a szervezet képességeit. A cloud-smart megközelítés nem felhőellenes, hanem a felhőhasználatot a kontextushoz köti.

A döntési keret egyik hasznos eszköze a **döntési mátrix**, amelyet a fejezet laborrészében részletesen bemutatunk. A mátrix lényege, hogy a felhőstratégiai opciókat (egy szolgáltató, multi-cloud, hybrid) meghatározott kritériumok mentén, súlyozott pontszámokkal értékeljük. A mátrix nem helyettesíti a szakmai ítélőképességet, de átláthatóvá teszi a döntés szempontjait, és megkönnyíti a trade-offok kommunikációját a stakeholderek felé.

17.4.3 Portabilitás: mit jelent és hogyan mérjük

A portabilitás a rendszer azon tulajdonsága, hogy adott költséggel és idő alatt átvihető egy másik futtatási környezetbe. A portabilitás nem abszolút fogalom: egy rendszer soha nem „teljesen hordozható”, és a hordozhatóság mértéke attól függ, hogy milyen célkörnyezetet tételezünk fel. Más kérdés egy konténerizált webalkalmazást átvinni egy másik Kubernetes-klaszterre, és más kérdés egy mélyen integrált, provider-specifikus szolgáltatásokat használó adatfeldolgozó pipeline-t átvinni egy másik felhőbe.

A portabilitás két dimenziója:

- **Kód portabilitása:** az alkalmazás futtatható-e másik környezetben anélkül, hogy jelentős kódmódosításra lenne szükség? A konténerizálás, a standard protokollok (HTTP, gRPC, SQL) és a felhőfüggetlen konfigurációkezelés növelik a kód portabilitását.
- **Adat portabilitása:** az adatok kinyerhetők-e másik környezetbe értelmezhető formában, elfogadható időn belül? Ez gyakran nehezebb, mint a kód átvitele, mert az adatbázisok, az objektumtárolók és a streaming rendszerek eltérő exportálási és importálási mechanizmusokat kínálnak.

A portabilitás mérésére a következő metrikákat használhatjuk:

- **Átviteli idő** (óra vagy nap): mennyi idő alatt vihető át a rendszer egy másik környezetbe?
- **Átviteli költség** (pénznem): mennyibe kerül az átvitel, beleértve a munkaerőt, az adatforgalmat és a párhuzamos üzemeltetést?
- **Kódmódosítás aránya** (százalék): a kódbázis hány százalékát kell módosítani az átvitelhez?
- **Adatvesztés kockázata** (RTO/RPO): mennyi adat veszhet el az átvitel során, és mennyi ideig lehet a rendszer nem elérhető?

Ezeket a metrikákat **quality attribute scenario** formájában is megfogalmazhatjuk. Például:

> **Portabilitási forgatókönyv**
> **Inger:** az üzemeltető az éles rendszer átvitelét kezdeményezi az A felhőszolgáltatóról a B felhőszolgáltatóra.
> **Környezet:** éles rendszer, alacsony forgalmú időszak.
> **Válasz:** az átvitel legfeljebb 72 óra alatt megtörténik, a rendszer legfeljebb 4 órára esik ki, és nem vész el adat.
> **Mérőszám:** átviteli idő 72 óra; RPO = 0; RTO 4 óra; átviteli költség 10 000 EUR.

A portabilitás növelésének leggyakoribb eszköze a **konténerizálás** és a **Kubernetes** használata. A konténerek egységes csomagolási formátumot adnak, a Kubernetes pedig a konténerek üzemeltetésére kínál deklaratív felületet. Fontos azonban látni, hogy a Kubernetes önmagában nem oldja meg a portabilitás minden problémáját. A menedzselt Kubernetes-szolgáltatások (EKS, AKS, GKE) eltérnek abban, hogyan kezelik a tárolókat, a terheléselosztókat, a hálózati policy-ket, az IAM-integrációt és a naplózást. Ha a rendszer ezekre a provider-specifikus integrációkra épít, akkor a konténerek hordozhatók, de a környezet konfigurációja nem. A hordozhatóság megőrzéséhez a Kubernetes-erőforrásokat úgy kell kialakítani, hogy a provider-specifikus komponensek (storage class, ingress controller, service mesh) cserélhetők legyenek, vagy legalább explicit módon dokumentáljuk a függőséget.

A portabilitás és a **managed service-ök** használata között feszültség van. Minél több menedzselt szolgáltatást veszünk igénybe (például a felhőszolgáltató saját üzenetküldőjét, adatbázisát vagy serverless platformját), annál kevesebb üzemeltetési terhet visel a csapat, de annál nagyobb a

provider-specifikus API-k aránya. Ez a trade-off tudatosan kezelhető: egyes komponenseknél elfogadjuk a lock-in-t, másoknál nyílt alternatívát választunk, és a döntést ADR-ben rögzítjük.

17.4.4 Adatlokáció és szabályozás

Az adatlokáció kérdése az utóbbi években a felhőstratégia egyik legfontosabb szempontjává vált. A szabályozások – különösen az Európai Unióban – megkövetelik, hogy bizonyos adatok meghatározott földrajzi régiókban maradjanak, vagy hogy az adatkezelésre meghatározott joghatóság vonatkozzon. A felhőszolgáltatók régiói és az adatok replikációja közötti összefüggéseket ezért már a tervezéskor figyelembe kell venni.

Két alapfogalmat érdemes megkülönböztetni:

- **Adatreszidencia:** az adat fizikai tárolási helye. A szabályozás előírhatja, hogy az adatok egy adott országban vagy régióban maradjanak. A felhőszolgáltatók általában lehetővé teszik, hogy az adatokat egy adott régióban tároljuk, de a replikáció, a biztonsági mentés és a naplózás is ugyanabba a régióba kell, hogy essen, különben az adatreszidencia sérül.
- **Adatszuverenitás:** az adat feletti joghatóság. Az adatszuverenitás azt jelenti, hogy az adatra az adott ország jogszabályai vonatkoznak, függetlenül attól, hogy a felhőszolgáltató hol tárolja. Ez különösen akkor releváns, ha a szolgáltató székhelye vagy leányvállalata olyan országban van, amelynek hatóságai hozzáférést követelhetnek az adatokhoz.

Az EU-ban a GDPR mellett az olyan ágazati szabályozások, mint a pénzügyi szektorra vonatkozó DORA vagy a kritikus infrastruktúrákra vonatkozó NIS2, további adatkezelési és kockázatkezelési kötelezettségeket írnak elő. A szabályozás nem pusztán technikai kérdés: a felhőszolgáltatóval kötött szerződésnek is tartalmaznia kell az adatkezelés helyét, az adatfeldolgozó szerepét és az adatvédelmi garanciákat. A technikai architektúra csak akkor megfelelő, ha a szerződéses feltételek is összhangban vannak a szabályozással.

A felhőszolgáltatók adatközpontjainak régiói között az adatáramlás jellemzően egyszerű, de az adat kiléptetése (egress) költséges lehet. Az adattranszfer költsége nem csak pénzügyi kérdés: az adat fizikai mozgatása időt és sávszélességet igényel, és szabályozási korlátokba ütközhet. A multi-cloud stratégiánál ezért az adattranszfer költségét és a régiók közötti késleltetést is be kell építeni a döntési mátrixba.

Egy gyakorlati példa: egy e-kereskedelmi platform az EU-ban működik, és a GDPR miatt a felhasználók személyes adatait EU-n belül kell tárolnia. A platform választhat egy EU-régióval rendelkező felhőszolgáltatót, és a rendszert egyetlen régióban üzemelteti. Ha a platform multi-cloud stratégiát akar, akkor mindkét szolgáltatónál EU-régiót kell használnia, és a kettő közötti adatáramlást is úgy kell kialakítania, hogy az adat ne hagyja el az EU-t. Ez a követelmény jelentősen szűkítheti a rendelkezésre álló szolgáltatások körét, és növelheti a költségeket.

17.4.5 Lock-in költsége és kilépési stratégia

A lock-in költségét gyakran alulbecsülik, mert a döntés pillanatában a szolgáltatóváltás távolinak tűnik. A költség azonban nem csak a technikai migráció költsége, hanem magában foglalja a felderítést, a tesztelést, a párhuzamos üzemeltetést, a képzést és az elmaradt hasznot is. A váltás teljes költsége a következő tételekből áll:

$$C_{\text{váltás}} = C_{\text{felderítés}} + C_{\text{migráció}} + C_{\text{párhuzamos}} + C_{\text{képzés}} + C_{\text{adatátvitel}} + C_{\text{elmaradt haszon}}$$

ahol:

- $C_{\text{felderítés}}$: a jelenlegi rendszer és a célkörnyezet közötti különbségek feltárásának költsége.
- $C_{\text{migráció}}$: a kód, a konfiguráció és az infrastruktúra átalakításának költsége.
- $C_{\text{párhuzamos}}$: a két környezet párhuzamos üzemeltetésének költsége az átmeneti időszakban.
- $C_{\text{képzés}}$: a csapat betanításának költsége az új környezetre.
- $C_{\text{adatátvitel}}$: az adatok kinyerésének, átvitelének és betöltésének költsége, beleértve a szolgáltató egress-díjait is.
- $C_{\text{elmaradt haszon}}$: az az érték, amelyet a szervezet a váltás ideje alatt nem tud előállítani.

Ezt a képletet nem kell minden döntésnél pontosan kitölteni, de a tételek végiggondolása segít reális képet alkotni. A lock-in költsége tehát nem egyetlen szám, hanem egy becslési tartomány, amelyet a döntési naplóban rögzíteni kell.

A **kilépési stratégia** (exit plan) azt írja le, hogyan tudjuk a rendszert és az adatokat a szolgáltatótól elhozni, ha a szolgáltatás megszűnik, az ár elfogadhatatlanná válik, vagy a szervezet stratégiai okból vált. A kilépési stratégia nem azt jelenti, hogy folyamatosan készen állunk a váltásra, hanem azt, hogy tudjuk, milyen lépéseket kell tenni, és mennyi időre van szükség. A terv elemei:

- **Adatkivonati formátumok:** milyen formátumban tudjuk az adatokat exportálni? A szolgáltatónak van-e nyílt export API-ja, vagy csak a saját konzolján keresztül lehet adatot letölteni?
- **Biztonsági mentések:** vannak-e rendszeres mentések, amelyeket a szolgáltatótól független helyre is el tudunk menteni?
- **Szolgáltatásleállítási eljárás:** hogyan állítjuk le a szolgáltatást, és hogyan biztosítjuk, hogy ne maradjon árva erőforrás vagy váratlan költség?
- **Szerződéses feltételek:** milyen felmondási idő és költség vonatkozik a szerződésre? Milyen garanciák vannak az adatok visszaszolgáltatására?
- **Tudásátadás:** a csapat rendelkezik-e a szükséges ismeretekkel ahhoz, hogy a rendszert egy másik környezetben üzemeltesse?

A kilépési stratégia tesztelése ritkán történik meg, pedig a **reversibility** elve azt mondja, hogy minden kritikus döntés legyen visszafordítható, vagy legalább legyen egy előre átgondolt terv a visszafordításra. A gyakorlatban ez azt jelenti, hogy időnként érdemes egy kisebb komponenst ténylegesen átvinni egy másik környezetbe, vagy legalább egy asztali gyakorlatot tartani a migrációról. Ez nem csak a terv érvényességét ellenőrzi, hanem a csapat készségét is fejleszti.

A lock-in elfogadása bizonyos feltételek mellett racionális. Ha a szolgáltató menedzselt szolgáltatása olyan értéket nyújt, amelyet a saját üzemeltetés vagy egy nyílt alternatíva nem tudna elérni, és a váltás költsége elfogadható, akkor a lock-in vállalása jó döntés. A döntést azonban dokumentálni kell, és a kilépési tervet akkor is el kell készíteni, ha a váltás valószínűsége alacsony.

17.4.6 Gyakorlati minták és anti-minták

A multi-cloud és a lock-in kezelésére számos gyakorlati minta létezik, de ugyanannyi anti-minta is. A mintákat mindig a kontextussal együtt érdemes értékelni.

Gyakorlati minták:

- **Absztrakciós réteg:** a provider-specifikus API-k mögé egy saját interfészt helyezünk, amely mögött a szolgáltató cserélhető. Ez a minta akkor működik jól, ha az absztrakció viszonylag vékony, és a különböző szolgáltatók hasonló képességeket kínálnak. Példa: egy objektumtároló esetén az S3-kompatibilis API-t használjuk, amelyet több szolgáltató is támogat. Az absztrakció azonban nem szabad, hogy túlzottan elfedje a provider-specifikus funkciókat, mert akkor elveszítjük a felhő előnyeit.

- **Nyílt szabványok és nyílt forráskódú komponensek:** ahol lehetséges, olyan komponenseket válasszunk, amelyeknek nyílt szabvány vagy nyílt forráskódú implementáció létezik. Ilyenek például a Kubernetes, a PostgreSQL, a Redis, a Kafka, vagy az OpenTelemetry. Ezek a komponensek több felhőszolgáltatónál is elérhetők menedzselt formában, így a váltás kisebb kódmódosítást igényel.
- **Explicit döntésnapló:** minden olyan döntést, amely növeli a lock-in-t, ADR-ben rögzítünk, megjelölve a vállalt kockázatot és a kilépési tervet. Ez nem csökkenti a lock-in-t, de átláthatóvá teszi.
- **Provider-specifikus komponensek izolálása:** ha egy szolgáltatónak egyedi szolgáltatását használjuk, azt érdemes a rendszer többi részétől jól körülhatárolt módon izolálni. Például egy adatfeldolgozó pipeline esetén a provider-specifikus üzenetküldőt egy adapter mögé rejtjük, így a többi komponens nem függ közvetlenül a konkrét API-tól.
- **Egress költség monitorozása:** az adattranszfer költsége folyamatosan mérhető, és a döntésekbe beépíthető. Ha az egress költség jelentős, az a lock-in egyik indikátora.

Anti-minták:

- **„Abszolút vendor-neutral”:** a szervezet ragaszkodik ahhoz, hogy minden komponens nyílt szabványokra épüljön, és semmilyen provider-specifikus szolgáltatást ne használjon. Ez gyakran oda vezet, hogy a csapat újra feltalálja a kereket, és olyan saját megoldásokat üzemeltet, amelyek drágábbak és kevésbé megbízhatóak, mint a menedzselt szolgáltatások. A hordozhatóság ekkor öncélúvá válik, és a rendszer összköltsége nő.
- **„Fake multi-cloud”:** a szervezet két felhőszolgáltatót használ, de a második csak katasztrófa-helyzetre tartalék, és a rendszer valójában nem hordozható. Ez a látszatbiztonság hamis: a DR-környezet nincs rendszeresen tesztelve, és egy valós vészhelyzetben használhatatlannak bizonyul.
- **Data gravity figyelmen kívül hagyása:** az adatok nagy tömeget képviselnek, és a köréjük épülő szolgáltatások nehezen mozdíthatók. Ha a döntés során csak a kódot nézzük, és az adatokat figyelmen kívül hagyjuk, akkor a portabilitás becslése téves lesz.
- **Minden szolgáltató egyformán kihasználása:** a multi-cloud nem jelenti azt, hogy minden szolgáltatónál ugyanazt a funkcionalitást kell üzemeltetnünk. Ez a redundancia növeli a komplexitást, a költségeket és a szervezeti terhelést, anélkül hogy valódi hordozhatóságot adna.

17.4.7 Labor: multi-cloud döntési mátrix készítése

A labor célja, hogy egy konkrét rendszerre vonatkozóan döntési mátrixot készítsünk, amely összehasonlítja a lehetséges felhőstratégiákat. A mátrix segít átláthatóvá tenni a döntést, és a labor során megtanuljuk a kritériumok súlyozását és az opciók értékelését.

17.4.8 Kiinduló állapot

Válassz egy fiktív rendszert, például:

- **Webáruház:** alacsony forgalmú időszakokban napi néhány ezer kérés, kampányok idején napi több százezer kérés. Fizetési adatokat kezel, ezért PCI DSS követelményeknek kell megfelelnie.
- **IoT platform:** több ezer eszköz küld telemetriai adatokat percenként, az adatokat valós időben kell feldolgozni és tárolni.
- **Belső adminisztrációs rendszer:** alacsony forgalmú, de szigorú adatvédelmi követelményekkel.

A laborhoz válassz egyet, és rögzítsd a kontextust: milyen üzleti célok, milyen szabályozási követelmények, mekkora csapat és milyen szakértelem áll rendelkezésre.

17.4.9 Lépések

1. **Kritériumok meghatározása.** Válassz 6–8 kritériumot, amelyek a felhőstratégia szempontjából relevánsak. Javasolt kritériumok:

   - Idő a piacra (time-to-market)
   - Üzemeltetési költség (3 éves időtávon)
   - Portabilitás / lock-in kockázat
   - Adatvédelmi és szabályozási megfelelés
   - Rendelkezésre állás és katasztrófa-helyreállítás
   - Csapat szakértelme és tanulási görbe
   - Skálázhatóság és teljesítmény
   - Szolgáltatói stabilitás és szerződéses feltételek

2. **Súlyozás.** Minden kritériumhoz rendelj súlyt 0 és 1 között úgy, hogy az összegük 1 legyen. A súlyok tükrözik a szervezet prioritásait.

3. **Opciók meghatározása.** Határozd meg a lehetséges stratégiákat, például:

   - **A opció:** egyetlen nyilvános felhőszolgáltató, mély integrációval.
   - **B opció:** multi-cloud aktív-aktív, nyílt szabványokra építve.
   - **C opció:** hybrid cloud, saját adatközpont és egy nyilvános felhő.
   - **D opció:** multi-cloud, provider-specifikus szolgáltatásokkal, de a második felhő csak DR-re.

4. **Értékelés.** Minden opciót értékelj minden kritérium mentén 1-től 5-ig terjedő skálán (1 = gyenge, 5 = kiváló). Az értékeléshez írj rövid indoklást.

5. **Súlyozott összeg kiszámítása.** Minden opcióra számold ki a súlyozott összeget: $\sum_i w_i \cdot p_{i,j}$, ahol w_i az i-edik kritérium súlya, $p_{i,j}$ pedig a j-edik opció pontszáma az i-edik kritériumra.

6. **Döntési napló rögzítése.** Készíts egy ADR-t, amely tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.

17.4.10 Példa mátrix

A következő táblázat egy fiktív webáruházra vonatkozó példát mutat. A pontszámok illusztratívak, a laborban a saját rendszeredre kell elvégezned az értékelést.

Kritérium	Súly	A: egyetlen felhő	B: multi-cloud aktív-aktív	C: hybrid	D: multi-cloud DR-rel
Idő a piacra	0,20	5	3	4	4
Üzemeltetési költség (3 év)	0,15	4	2	3	3
Portabilitás / lock-in kockázat	0,10	2	5	4	3

Kritérium	Súly	A: egyetlen felhő	B: multi-cloud aktív-aktív	C: hybrid	D: multi-cloud DR-rel
Adatvédelmi megfelelés	0,15	4	4	5	4
Rendelkezésre állás	0,15	4	5	4	4
Csapat szakértelme	0,10	5	2	4	3
Skálázhatóság	0,10	5	4	4	4
Szolgáltatói stabilitás	0,05	4	4	4	4
Súlyozott összeg	**1,00**	**4,25**	**3,55**	**3,95**	**3,75**

A példában az A opció (egyetlen felhő) kapja a legmagasabb pontszámot, elsősorban a gyors piacra lépés és a csapat meglévő szakértelme miatt. A döntés árnyalható: ha a szabályozási környezet vagy a portabilitás fontosabbá válik, a súlyok módosításával más opció kerülhet előtérbe. A mátrix tehát nem ad automatikus választ, hanem a trade-offokat teszi láthatóvá.

17.4.11 Elvárt eredmény

- Egy dokumentum, amely tartalmazza a kiinduló kontextust, a kritériumokat, a súlyokat, az opciókat és a pontszámok indoklását.
- Egy táblázat a döntési mátrixszal.
- Egy ADR, amely rögzíti a döntést és a következményeket.
- A mért és a várt eredmények elkülönítése: a pontszámok becslések, nem mérések; ha valamelyik kritériumra van tényleges adat (például költségbecslés), azt külön kell jelölni.

17.4.12 Ellenőrzési szempontok

- A kritériumok relevánsak-e a rendszer kontextusára?
- A súlyok összege 1?
- A pontszámok indoklása dokumentált?
- A döntési napló tartalmazza a feladott alternatívákat?
- A lock-in kockázat explicit módon szerepel?
- A kilépési terv legalább vázlatosan elkészült?

17.4.13 Visszaállítási út

A labor nem igényel infrastruktúra-módosítást, ezért visszaállításra nincs szükség. Ha a labor során valamelyik opciót PoC-ként kipróbálod, ügyelj arra, hogy a létrehozott erőforrásokat a labor végén töröld, és a költségeket ellenőrizd.

17.4.14 Ellenőrzőlista

A fejezet végén az alábbi kérdéseket érdemes végiggondolni minden felhőstratégiai döntésnél:

- ☐ Rögzítettük-e a felhőhasználat üzleti célját, vagy csak technológiai preferenciát követtünk?
- ☐ Azonosítottuk-e a provider-specifikus függőségeket (szolgáltatások, adatformátumok, API-k, IAM)?
- ☐ Készítettünk-e kilépési tervet, és abban szerepel-e az adatkivonat, a leállítás és a költségek?
- ☐ Mértük-e a portabilitás szempontjából kritikus metrikákat (átviteli idő, költség, adatvesztés)?
- ☐ Az adatlokációs és szabályozási követelményeket figyelembe vettük-e a régióválasztásnál?
- ☐ A lock-in vállalása tudatos és dokumentált döntés-e?
- ☐ A multi-cloud vagy hybrid stratégia valódi hordozhatóságot ad-e, vagy csak látszatbiztonságot?
- ☐ A döntési mátrix kritériumai és súlyai a stakeholderekkel egyeztetve lettek-e?

17.4.15 Önellenőrző kérdés

Milyen feltételek mellett racionális egy erős lock-in elfogadása? Fogalmazz meg három konkrét feltételt, és magyarázd el, miért teljesülnek egy általad ismert rendszerben.

Válaszvázlat: Az erős lock-in akkor racionális, ha (1) a szolgáltató menedzselt szolgáltatása olyan egyedi értéket nyújt, amelyet nyílt alternatívával vagy saját üzemeltetéssel nem lehetne elérni; (2) a váltás várható költsége alacsonyabb, mint a hordozhatóság fenntartásának költsége; (3) a szolgáltató stabilitása, piaci pozíciója és szerződéses garanciái elfogadható kockázatot jelentenek. A feltételek teljesüléséhez a kilépési tervet akkor is el kell készíteni, ha a váltás valószínűsége alacsony.

A fejezetben bemutatott fogalmak és labor segítségével a rendszertervező képes lesz a felhőstratégiát és a szolgáltatófüggést mérhető, dokumentált döntésként kezelni, nem pedig érzelmi vagy esetleges választásként. A következő egység a platform engineering és a belső fejlesztői platform témájára épít, ahol a multi-cloud és a lock-in szempontok a belső platform tervezésénél is visszaköszönnek.

18 Platform engineering és belső fejlesztői platform

18.1 Platform mint termék és golden path

A platform engineering egyik központi gondolata, hogy a belső fejlesztői platform nem egyszeri infrastrukturális projekt, hanem folyamatosan fejlesztett, belső fogyasztóknak szánt termék. Ebben a fejezetben a platformot termékként kezeljük: megvizsgáljuk, kik a felhasználói, milyen értéket kapnak, hogyan mérjük a visszacsatolást, és hogyan tervezzük meg a támogatott, „arany" útvonalakat (golden path), amelyek a leggyakoribb fejlesztői feladatokat teszik gyorssá és kiszámíthatóvá. A cél nem csupán a fogalmak bemutatása, hanem egy kis, ellenőrizhető tervezési gyakorlat elvégzése is, amelyben a platform és a projektcsapat felelősségi határait is tisztázzuk.

18.1.1 A platform mint belső termék

A klasszikus üzemeltetési modellben a platformcsapat gyakran „tűzoltóként" vagy „jegykezelőként" működik: a fejlesztők kéréseket adnak le, a platformcsapat pedig manuálisan hoz létre környezeteket, konfigurál adatbázisokat, állít be CI/CD pipeline-okat. Ez a modell nem skálázódik, mert a platformcsapat szűk keresztmetszetté válik, a fejlesztők várakozásra kényszerülnek, és a tudás a platformcsapat fejében koncentrálódik.

A platform mint belső termék szemlélet ezzel szemben azt mondja: a platformcsapat nem üzemeltetési részleg, hanem termékfejlesztő csapat, amelynek terméke a belső fejlesztői platform. Ennek a terméknek ugyanúgy vannak felhasználói, értékajánlata, ütemterve és minőségi követelményei, mint bármely piaci terméknek – csak éppen a felhasználók a szervezeten belül vannak, az érték pedig nem közvetlen bevétel, hanem a fejlesztői szervezet sebességének és megbízhatóságának növekedése.

A termékszemléletű platform legfontosabb jellemzői:

- **Önkiszolgálás:** a fejlesztő saját maga, várakozás nélkül tud erőforrásokat létrehozni (például repositoryt, adatbázist, deployment pipeline-t) a platform által biztosított felületeken keresztül.
- **Felfedezhetőség:** a platform által kínált szolgáltatások könnyen megtalálhatók, dokumentáltak, és egyértelmű, hogy melyikhez milyen támogatás tartozik.
- **Konzisztens interfész:** a platform API-k, CLI-k és webes felületek egységes konvenciókat követnek, így a fejlesztőnek nem kell minden szolgáltatásnál újra tanulnia a használatot.
- **Visszacsatolási hurok:** a platformcsapat aktívan gyűjti a felhasználói visszajelzéseket, méri a használati adatokat, és ezek alapján priorizálja a fejlesztéseket.

A platform mint termék tehát nem egy halom script és manuális runbook, hanem egy koherens, támogatott és folyamatosan fejlődő belső szolgáltatási réteg.

18.1.2 A fejlesztői élmény (Developer Experience, DevEx)

A platform elsődleges felhasználója a belső fejlesztő. A fejlesztői élmény azt írja le, milyen érzés a platformot használni: mennyi időbe telik egy új szolgáltatás elindítása, mennyire egyértelmű a dokumentáció, milyen gyors a visszajelzés a CI/CD pipeline-tól, mennyi kognitív terhet jelent a konfigurációk karbantartása.

A jó fejlesztői élmény három dimenziója különösen fontos:

1. **Alacsony kognitív terhelés:** a fejlesztőnek nem kell minden mögöttes infrastrukturális részletet ismernie ahhoz, hogy elvégezze a feladatát. A platform elrejti a felesleges komplexitást, miközben a szükséges kontrollt meghagyja.
2. **Gyors visszajelzés:** minél hamarabb kap a fejlesztő visszajelzést a kódja minőségéről, a tesztek eredményéről vagy a deployment állapotáról, annál gyorsabban tud haladni. A platform célja, hogy a visszajelzési hurkok percekben, ne órákban vagy napokban mérhetők legyenek.
3. **Flow állapot támogatása:** a fejlesztő akkor a leghatékonyabb, ha megszakítások nélkül tud dolgozni. A platformnak csökkentenie kell a várakozási időket, az ismétlődő manuális lépéseket és a bürokratikus jóváhagyásokat.

A fejlesztői élmény nem „puha” szempont, hanem mérhető és számszerűsíthető. Például mérhetjük az új szolgáltatás létrehozásának átlagos idejét, a golden path sikeres végrehajtásának arányát, a fejlesztők által leadott support jegyek számát, vagy a platformmal való elégedettséget egy rövid kérdőívvel. Ezek a mérőszámok adják a platformcsapat visszacsatolási alapját.

18.1.3 Golden path: a támogatott, kitaposott út

A golden path (magyarul „arany út” vagy „arany útvonal”) egy előre definiált, automatizált és dokumentált megoldás egy gyakori fejlesztői feladatra. A leggyakoribb példa az „új mikroszolgáltatás indítása”: ahelyett, hogy a fejlesztő manuálisan hozná létre a repositoryt, írná a Dockerfile-t, a Helm chartot, a CI/CD pipeline-t és a staging környezetet, a golden path egyetlen parancs vagy űrlap segítségével mindezt elvégzi helyette.

A golden path jellemzői:

- **Sablonokon alapul:** a platformcsapat előre elkészített, verziózott sablonokat biztosít (repository sablon, pipeline definíció, infrastruktúra-konfiguráció), amelyek a legtöbb esetben megfelelnek.
- **Automatizált:** a lépések nem igényelnek kézi beavatkozást; a platform API-ja vagy CLI-je végrehajtja a szükséges műveleteket.
- **Tesztelt és karbantartott:** a platformcsapat folyamatosan teszteli a golden pathot, frissíti a sablonokat, és biztosítja, hogy azok megfeleljenek a szervezeti szabványoknak.
- **Dokumentált:** minden golden pathhoz tartozik egy rövid útmutató, amely leírja a használatot, a bemeneteket, a kimeneteket és a gyakori hibákat.
- **Támogatott:** ha a fejlesztő a golden pathot használja, a platformcsapat teljes támogatást nyújt; ha letér róla, a támogatás korlátozott lehet.

Fontos hangsúlyozni, hogy a golden path nem kötelező. A fejlesztőcsapatoknak joguk van egyedi megoldást választani, ha a golden path nem felel meg a speciális igényeiknek. Ilyenkor azonban vállalják a többletterhet: nekik kell karbantartaniuk az egyedi pipeline-t, konfigurációt, és a platformcsapat csak korlátozott segítséget tud nyújtani. Ez a „paved road” elv: a platform kikövezi az utat, de nem zárja le a mellékösvényeket.

A golden path csökkenti a fejlesztői kognitív terhelést, növeli a konzisztenciát, és lehetővé teszi a szervezeti szintű szabványok (biztonság, megfelelés, megfigyelhetőség) automatikus érvényesítését. Például ha minden új szolgáltatás ugyanazt a pipeline-sablont használja, akkor a titokkezelés, a sebezhetőség-ellenőrzés és a naplózás automatikusan beépül minden projektbe.

Az alábbi Mermaid diagram egy egyszerű golden path folyamatát mutatja be az „új szolgáltatás indítása” forgatókönyvre. A diagram szemléltetés, nem végrehajtandó kód.

```
flowchart TD
    A[Fejlesztő meghívja a platform CLI-t] --> B{Paraméterek ellenőrzése}
    B -- Hibás --> C[Hibaüzenet és kilépés]
    B -- Érvényes --> D[Repository létrehozása sablonból]
    D --> E[CI pipeline konfiguráció generálása]
    E --> F[CD pipeline konfiguráció generálása]
    F --> G[Staging környezet létrehozása]
    G --> H[Értesítés a fejlesztőnek]
    H --> I[Deploy a staging környezetbe]
```

Jelmagyarázat: A folyamat a fejlesztői parancstól a staging deploymentig tart; a döntési pont a bemeneti paraméterek validálása. **Feltételjegyzet:** A példa feltételezi, hogy a platform API elérhető, a konténerregiszter és a Kubernetes klaszter rendelkezésre áll, és a fejlesztő rendelkezik a szükséges jogosultságokkal.

18.1.4 Platformszolgáltatás katalógus

A platformszolgáltatás katalógus a platform által kínált szolgáltatások strukturált jegyzéke. Olyan, mint egy belső étlap: a fejlesztő megnézheti, milyen szolgáltatások érhetők el, ki a felelős értük, hogyan veheti igénybe, és milyen támogatásra számíthat.

Egy jól felépített katalógusbejegyzés legalább a következő mezőket tartalmazza:

- **Szolgáltatás neve és rövid leírása.**
- **Felelős platformcsapat vagy kontakt.**
- **Fogyasztói interfész:** API végpont, CLI parancs, webes felület vagy sablon.
- **Dokumentáció hivatkozása** (belső, helyettesítő URL, például `https://platform.internal.example.c`
- **Support csatorna:** chat-szoba, jegyrendszer, ügyelet.
- **Kapcsolódó golden path(ok).**
- **Szolgáltatási szintre vonatkozó elvárások** (ha releváns): például rendelkezésre állás, válaszidő.

Példa egy kis platformszolgáltatás katalógusra:

Szolgáltatás	Felelős	Fogyasztói interfész	Dokumentáció	Support	Golden path
Konténerregiszter	Platform Infra csapat	`registry.internal.example.com`	`.../container-registry`	`#platform-support`	`gp-container-push`
CI/CD pipeline szolgáltatás	Platform CI csapat	CLI: `platform pipeline create`	`.../pipeline`	`#platform-ci`	`gp-new-service`
Megfigyelhetőség	Observability csapat	API: `obs.internal.example.com`	`.../observability`	`#obs-support`	`gp-add-monitoring`

Szolgáltatás	Felelős	Fogyasztói interfész	Dokumentáció	Support	Golden path
Titokkezelés	Security Platform	CLI: `platform secret create`	`.../secrets`	`#security-help`	`gp-add-secret`
Adatbázis-kiépítés	Data Platform	CLI: `platform db create`	`.../databases`	`#data-platform`	`gp-provision-db`

Természetesen a valós katalógus ennél részletesebb, és a szolgáltatásokhoz tartozó golden pathok külön dokumentumban vagy a katalógusba ágyazva jelennek meg. A lényeg, hogy a fejlesztő egy helyen megtalálja az összes elérhető platformképességet, és tudja, hogyan kezdjen hozzá.

18.1.5 A platform és a projektcsapat felelőssége

A platform mint termék bevezetése nem tünteti el a felelősségi határokat, hanem újradefiniálja azokat. A gyakori félreértés, hogy a platformcsapat „átveszi" a projektcsapat üzemeltetési feladatait, vagy hogy a projektcsapatnak minden infrastrukturális döntést a platformra kell bíznia. Valójában a felelősségek megoszlanak, és a határokat explicit módon kell rögzíteni.

A platformcsapat felelős:

- A platformszolgáltatások (például konténerregiszter, CI/CD pipeline, megfigyelhetőség, titokkezelés) életciklusáért: telepítés, frissítés, skálázás, monitorozás.
- A golden pathok definiálásáért, sablonok és automatizmusok karbantartásáért.
- A platform dokumentációjáért, a katalógus naprakészen tartásáért.
- A platform használatának támogatásáért (support), a hibák kivizsgálásáért és javításáért.
- A platform minőségi követelményeinek (rendelkezésre állás, teljesítmény, biztonság) teljesítéséért.
- A felhasználói visszajelzések gyűjtéséért és a platform roadmap kialakításáért.

A projektcsapat (stream-aligned csapat) felelős:

- A saját alkalmazás üzleti logikájáért, kódjáért és konfigurációjáért.
- A platform által biztosított golden pathok használatáért, vagy ha letér, az egyedi megoldás karbantartásáért.
- A saját deploymentjük végrehajtásáért (a platform eszközeivel), a saját szolgáltatásuk üzemeltetéséért az alkalmazás szintjén.
- A platformnak adott visszajelzésekért, a problémák és igények jelzéséért.
- A saját csapatukon belüli működési gyakorlatokért, beleértve a golden path használatának betartását.

Fontos elv: a platformcsapat **nem** az alkalmazás üzemeltetője. Ha egy alkalmazás hibásan működik, az a projektcsapat felelőssége. Ha viszont a platform szolgáltatása (például a CI/CD pipeline) hibásodik meg, az a platformcsapaté. Ez a határvonal csökkenti a „közös felelősség" homályát, és lehetővé teszi az elszámoltathatóságot.

A platform és a projektcsapat közötti interakció nem egyirányú. A projektcsapat a platform legfontosabb visszajelzési forrása: a platformcsapatnak aktívan kell kérnie a fejlesztői visszajelzéseket, és a platform fejlesztését a felhasználói igényekhez kell igazítania. Ez a termékszemlélet lényege.

18.1.6 Minőségi forgatókönyv a platformra

A platform minőségi követelményeit ugyanúgy érdemes quality attribute scenario formában megfogalmazni, mint bármely más rendszer esetében. Ez segít a mérhetőségben és a platformcsapat elszámoltathatóságában.

Példa egy minőségi forgatókönyvre a golden path teljesítményére:

- **Inger:** Egy fejlesztő a platform CLI-n keresztül elindítja az új szolgáltatás létrehozását a golden path sablonnal.
- **Környezet:** Normál munkaidő, a platform API legfeljebb 10 egyidejű kérés alatt, a Kubernetes klaszter és a konténerregiszter elérhető.
- **Válasz:** A repository létrejön, a CI/CD pipeline konfigurációja elkészül, a staging deployment befejeződik, és a fejlesztő értesítést kap.
- **Mérőszám:** A válaszidő 95. percentilise legfeljebb 15 perc, a sikeres végrehajtások aránya legalább 99%, az elmúlt 30 napban mérve.

Ez a forgatókönyv lehetővé teszi a platformcsapat számára, hogy mérje a teljesítményt, és riasztást állítson be, ha a célértékektől eltér. A mérőszámoknál mindig meg kell adni a mértékegységet és a mérési ablakot, ahogy a fenti példában is szerepel.

18.1.7 Labor: Golden path és platformszolgáltatás katalógus tervezése

A következő gyakorlat célja, hogy a tanuló alkalmazza a fejezetben bemutatott fogalmakat egy kis, ellenőrizhető tervezési feladatban. A labor nem igényel éles rendszert; a kimenet egy verziózható dokumentum vagy jegyzetfüzet.

18.1.8 Kiinduló állapot

Adott egy 40 fős fejlesztőszervezet három stream-aligned csapattal. A csapatok konténeres mikroszolgáltatásokat fejlesztenek és üzemeltetnek Kubernetesen. Jelenleg minden csapat saját magának építi a CI/CD pipeline-ját, a deployment folyamatok eltérőek, a titokkezelés ad hoc, a megfigyelhetőség nem egységes. A szervezetben már van egy konténerregiszter és egy Kubernetes klaszter, de ezek használata nem szabványosított. A platformcsapat most alakul, és a vezetőség elvárja a belső fejlesztői platform kialakítását.

18.1.9 Elvárt eredmény

Készíts egy dokumentumot (például Markdown vagy Jupyter notebook formátumban), amely a következő elemeket tartalmazza:

1. **A platform felhasználóinak és értékajánlatának rövid leírása.** Kik a felhasználók, mi a problémájuk, milyen értéket nyújt a platform.
2. **Platformszolgáltatás katalógus** legalább öt szolgáltatással. Minden szolgáltatáshoz add meg: név, leírás, felelős platformcsapat, fogyasztói interfész (API, CLI, UI), támogatási csatorna, kapcsolódó golden path hivatkozás.
3. **Egy golden path részletes leírása** az „új mikroszolgáltatás indítása” forgatókönyvre. Sorold fel a lépéseket, a szükséges bemeneteket, a kimeneteket, az automatizált lépéseket és a döntési pontokat. Használhatsz Mermaid vagy PlantUML diagramot is.
4. **Egy ADR-t** a golden path bevezetésének döntéséről, a következő szerkezettel: kontextus, döntés, alternatívák, következmények, állapot.

5. **Legalább egy quality attribute scenario-t** a platformra, az inger, környezet, válasz, mérőszám mezőkkel.
6. **A jelenlegi mért és a várt eredmények elkülönített táblázata.** Például: az új szolgáltatás indításának átlagos ideje jelenleg 5 nap, várt cél 1 óra; a sikeres deployment arány jelenleg 80%, várt 99%. A mérőszámokhoz adj mértékegységet és mérési ablakot.
7. **Ellenőrzési szempontok** a golden path működésének validálására.

18.1.10 Ellenőrzési szempontok a labor értékeléséhez

- A dokumentum tartalmazza-e az összes előírt elemet?
- A golden path lépései egyértelműek és reprodukálhatók-e?
- A platform és a projektcsapat felelőssége világosan elkülönül-e?
- A minőségi forgatókönyv rendelkezik-e mind a négy kötelező mezővel?
- A mért és várt eredmények külön oszlopban vagy táblázatban szerepelnek-e?
- Az ADR követi-e a megadott formátumot, és megmagyarázza-e a döntés okát és az elvetett alternatívákat?

18.1.11 Visszaállítási út

Mivel a labor tervezési gyakorlat, nincs éles rendszer, amelyet helyre kellene állítani. A munkadokumentumot verziózd gitben; ha hibát követsz el, a git history segítségével állítsd vissza a korábbi állapotot. Ha a validációhoz PoC-t futtatsz (például egy egyszerű pipeline-t), használj izolált sandbox környezetet, és a labor végén töröld a létrehozott erőforrásokat a sandboxban. A PoC eredményét jelöld nem termelési döntésként; a labor célja a tanulás, nem a termelési bevezetés.

18.1.12 Példa ADR a laborhoz

A következő ADR minta segíti a dokumentum elkészítését. A tanuló saját kontextusára szabva használhatja.

ADR-001: Golden path bevezetése az új szolgáltatás indítására

- **Kontextus:** A fejlesztőcsapatok jelenleg egyedi, kézzel karbantartott CI/CD pipeline-okat használnak, ami lassú és hibás. A platformcsapat célja a szabványosítás.
- **Döntés:** Bevezetünk egy golden pathot, amely sablonrepo, előre definiált pipeline és Helm chart alapján automatizálja az új szolgáltatás létrehozását, CI/CD-jét és staging deploymentjét.
- **Alternatívák:** 1) Minden csapat saját pipeline-t épít (status quo) – elvetve a magas karbantartási költség miatt. 2) Központi platformcsapat üzemelteti a pipeline-okat, a csapatok kérésre kapnak szolgáltatást – elvetve a skálázódási probléma miatt. 3) Golden path sablonok – elfogadva.
- **Következmények:** Pozitív: gyorsabb onboarding, konzisztens konfiguráció, kevesebb hiba. Negatív: a csapatoknak alkalmazkodniuk kell a sablonhoz; a platformcsapatnak karban kell tartania a sablont.
- **Állapot:** Elfogadva.

18.1.13 Ellenőrzőlista a platform mint termék bevezetéséhez

A fejezet végén az alábbi ellenőrzőlista segít a tanultak áttekintésében:

- ☐ A platformcsapat termékként kezeli a belső platformot, és rendelkezik termékfelelőssel, ütemtervvel és visszacsatolási mechanizmussal.
- ☐ A platform felhasználói (belső fejlesztők) azonosítottak, és a platform értékajánlata egyértelmű.
- ☐ Legalább egy golden path létezik a leggyakoribb fejlesztői feladatra, és az automatizált, dokumentált, tesztelt.
- ☐ A platformszolgáltatás katalógus naprakész, és minden szolgáltatáshoz tartozik felelős, interfész, dokumentáció és support csatorna.
- ☐ A platform és a projektcsapat felelősségi határai írásban rögzítettek, és mindkét fél ismeri azokat.
- ☐ A platform minőségi követelményei quality attribute scenario formában vannak megfogalmazva, mérőszámokkal és mérési ablakkal.
- ☐ A golden path használata mérhető, és a platformcsapat rendszeresen elemzi a visszajelzéseket.
- ☐ A golden path nem kötelező; a csapatok letérhetnek, de vállalják a többletterhet.

18.1.14 Önellenőrző kérdés

Kérdés: Egy fejlesztőcsapat azt kéri, hogy a platformcsapat üzemeltesse a saját alkalmazásuk adatbázisát, mert „a platform úgyis az infrastruktúráért felel”. Hogyan válaszolj a platform mint termék elv alapján, és hogyan osztanád meg a felelősséget?

Válaszvázlat: A platformcsapat nem az egyes alkalmazások üzemeltetője, hanem a platformszolgáltatások szolgáltatója. Ha a platform biztosít egy felügyelt adatbázis-szolgáltatást (például egy golden path az adatbázis-kiépítésre), akkor a platformcsapat felelős a szolgáltatás rendelkezésre állásáért és a kiépítés automatizmusáért. A projektcsapat felelős a saját adatbázisuk séma-, konfiguráció- és alkalmazásszintű üzemeltetéséért (például migrációk, indexek, kapacitásigények). A platformcsapat segíthet a platformszintű problémákban, de az alkalmazás-specifikus üzemeltetés a projektcsapaté. Ez a határ elkerüli a „közös felelősség” homályát, és mindkét csapat számára világos elvárásokat teremt.

A fenti szöveg megfelel a kért terjedelemnek, tanítja a platform mint belső termék, a fejlesztői élmény, a golden path, a felhasználó-érték-visszacsatolás és a felelősségi határok témáit. Tartalmaz labort ellenőrzőlistával és önellenőrző kérdéssel, valamint minőségi forgatókönyvet és ADR példát. A kód- és diagrampéldák szemléltetők, helyőrző URL-eket használ. A szöveg nem hivatkozik konkrétan olvasott forrásokra, de megemlíti a Team Topologies és CNCF általános keretrendszerét, jelezve az ellenőrzés szükségességét.

18.2 Team Topologies és szervezeti architektúra

A szervezeti architektúra nem szervezeti ábra, hanem az a mód, ahogyan a csapatok felelősséget, tudást és erőforrásokat kapnak, és ahogyan egymással együttműködnek. A szoftverarchitekt és a tech lead gyakran csak a rendszerdiagramokat látja, pedig a termék kódjában tükröződő modulhatárok, az API-k durvasága, a hibakezelés minősége és a deployment ritmusa szorosan összefügg azzal, hogy milyen szervezeti egységek építik és üzemeltetik az adott komponenseket. Ez az alfejezet a Team Topologies szemléletét használja fel arra, hogy a szervezetet mint architekturális tényezőt tárgyalja. A cél nem az, hogy egyetlen helyes szervezeti ábrát adjon,

hanem hogy olyan fogalmi eszközöket adjon, amelyekkel egy adott kontextusban tudatosan lehet csapatokat és interakciókat tervezni, illetve a meglévő állapotot értékelni.

18.2.1 Miért érdemes a szervezeti architektúrát explicit módon kezelni?

A szoftverrendszerek nem légüres térben készülnek. A szervezeti struktúra meghatározza a kommunikációs utakat, a döntési jogköröket és a tudásmegosztást. Ha egy rendszer komponensei nem illeszkednek a szervezeti határokhoz, akkor a mindennapi fejlesztés során folyamatosan olyan koordinációs költségek keletkeznek, amelyeket sem a legjobb CI/CD pipeline, sem a legmodernebb mikrokeretrendszer nem tud eltüntetni. A klasszikus Conway-törvény ezt röviden úgy fogalmazza meg: a szervezetek olyan rendszereket hoznak létre, amelyek a saját kommunikációs struktúrájukat tükrözik. Ez nem determinisztikus törvényszerűség, hanem erős tendencia: ha a fejlesztői csoportok egymástól elszigetelten dolgoznak, akkor a rendszerükben is elszigetelt, nehezen integrálható modulok jelennek meg; ha a szervezetben két csapat folyamatosan egyeztet, akkor a rendszerükben is szoros csatolás alakul ki közöttük.

Az architekturális tervezés ezért nem hagyhatja figyelmen kívül a szervezeti terhelést. A szervezeti terhelés kifejezés itt nem csupán a túlórát vagy a feladatmennyiséget jelenti, hanem azt a kognitív terhelést, amelyet egy csapatnak akkor kell viselnie, ha a saját szakterületén túl más területek részleteit is meg kell értenie. Ha egy stream-aligned csapatnak a terméklogika mellett a Kubernetes üzemeltetés részleteit, a compliance-szabályzat minden pontját és a legacy mainframe interfészeket is fejben kell tartania, akkor a termékre fordítható figyelme csökken, és a hibák száma nő. A Team Topologies modell egyik alaptézise, hogy a csapattípusok és interakciós minták tudatos megválasztásával a szervezeti terhelést lehet csökkenteni és a fennmaradó terhet oda irányítani, ahol az értéket termel.

18.2.2 A négy alapvető csapattípus

A Team Topologies négy olyan csapattípust különböztet meg, amelyek nem munkaköröket, hanem felelősségi és interakciós profilokat írnak le. Egy adott csapat idővel változtathat típusú, és ritka, hogy egy csapat egyszerre csak egy típusként viselkedik, de a domináns felelősség és az elsődleges interakciós felület meghatározásához hasznos ez a kategorizálás.

18.2.3 Stream-aligned csapat

A stream-aligned csapat (áramláshoz igazított csapat) egyetlen, jól meghatározott termékérték-folyamért felel. Ez a csapat a szervezet elsődleges értéktermelő egysége. A fókusz az üzleti képesség vagy a felhasználói utazás teljes spektruma: a követelmény megértésétől a megvalósításon át az üzemeltetésig. A stream-aligned csapat jellemzően olyan keresztfunkcionális képességekkel rendelkezik, mint a szoftverfejlesztés, a tesztelés, a termékmenedzsment és az alapvető üzemeltetés, de nem várható el tőle, hogy mély platformszintű tudást vagy speciális biztonsági szakértelmet halmozzon fel. A csapatot úgy kell kialakítani, hogy a lehető legkevesebb külső függőséggel, gyors visszajelzési ciklusban tudjon szolgáltatást vagy terméket szállítani. A stream-aligned csapat felelőssége tehát nem egyetlen komponens, hanem egy teljes értékfolyam. Például egy online kiskereskedelmi rendszerben a „rendeléskezelés” stream-aligned csapat a kosárba helyezéstől a megrendelés visszaigazolásáig tartó folyamatot birtokolja, beleértve az ehhez tartozó API-t, adatsémát, üzleti szabályokat és a kapcsolódó üzemeltetési feladatokat.

A stream-aligned csapat akkor működik jól, ha a kognitív terhelése korlátozott: a csapat érti a termékterületét, de nem terheli a platform összes belső részlete, a speciális algoritmusok vagy a vállalati szintű compliance-minden. Ennek biztosítása a többi csapattípus feladata.

18.2.4 Platform csapat

A platform csapat (platform team) olyan önkiszolgáló képességeket biztosít, amelyek csökkentik a stream-aligned csapatok kognitív terhelését. A platform csapat nem „mindenki mögött dolgozó üzemeltetési részleg”, hanem termékmenedzseri szemlélettel működő belső szolgáltató. Célja, hogy a stream-aligned csapatoknak ne kelljen maguknak felépíteniük és üzemeltetniük a konténerplatformot, a megfigyelhetőségi stacket, a CI/CD alapgépeket vagy a titokkezelést. A platform csapat olyan belső terméket hoz létre, amelynek felhasználói a többi fejlesztőcsapatok. A platform terméknek ugyanúgy van roadmapje, visszajelzési ciklusa és minőségi követelménye, mint egy külső terméknek. A platform csapat felelőssége nem az, hogy minden stream-aligned igényt egyedi módon kiszolgáljon, hanem hogy jól dokumentált, jól használható, általánosan alkalmazható golden pathot adjon. A golden path az ajánlott, támogatott út, amelyen a csapatok a leggyorsabban és a legkevesebb kockázattal tudnak haladni. A platform csapatnak világos interfészt kell adnia: API-k, CLI-eszközök, önkiszolgáló portál vagy sablonok formájában.

Fontos, hogy a platform csapat ne váljon a rendszer szűk keresztmetszetévé. Ha minden deployment csak a platform csapat jóváhagyásával mehet, akkor a stream-aligned csapatok ismét függőségbe kerülnek, és a kognitív terhelés nem csökken, hanem áthelyeződik. A platform csapat sikerének mérőszáma nem a saját ticketjeinek száma, hanem az, hogy a stream-aligned csapatok milyen gyorsan tudnak önállóan szállítani.

18.2.5 Enabling csapat

Az enabling csapat (lehetővé tevő vagy felhatalmazó csapat) átmeneti, tudásátadó szerepet tölt be. A célja nem az, hogy tartósan átvegyen feladatokat a stream-aligned csapatoktól, hanem hogy felépítse azok belső képességét egy adott területen. Tipikus területek: konténeres bevezetés, biztonsági tesztelés, teljesítményoptimalizálás, adatmodellezés vagy egy új programozási nyelv. Az enabling csapat úgy dolgozik, hogy együtt dolgozik a stream-aligned csapattal, mentorál, példakódot ad, workshopot tart, majd kivonul, és a tudás a stream-aligned csapatban marad. Az enabling csapat jelenléte időben korlátozott; ha egy enabling csapat éveken át ugyanabban a témában segíti ugyanazt a csapatot anélkül, hogy a tudás átadódna, akkor valószínűleg rosszul definiált a feladata. Az enabling csapat interfésze nem szolgáltatás, hanem facilitálás: közös munka, páros programozás, képzés, kísérleti projektek felügyelete. A siker mércéje, hogy a stream-aligned csapat egy adott idő után önállóan boldogul-e az adott területen.

18.2.6 Complicated-subsystem csapat

A complicated-subsystem csapat (bonyolult alrendszer csapat) olyan területekért felel, ahol a megoldás mély szakértelmet igényel, és a szükséges tudás nem osztható el könnyen a stream-aligned csapatok között. Ilyen lehet egy speciális kockázatkezelő motor, egy képfeldolgozó modul, egy beágyazott eszköz vezérlője vagy egy szigorúan szabályozott fizetési rendszer. A complicated-subsystem csapat célja, hogy a bonyolultságot elszigetelje, és a stream-aligned csapatok számára egyszerű, jól definiált interfészt adjon. A csapat a belső bonyolultságot magában tartja, és a külvilág felé szolgáltatást vagy API-t kínál. Nem szabad, hogy a stream-aligned csapatoknak ismerniük kelljen a belső algoritmusokat; elég, ha a szerződést értik. A complicated-subsystem csapat tartósan létezhet, mert a bonyolultság tartós. A kulcs az, hogy jól meghatározott legyen a rendszerhatár, és a csapat ne váljon láthatatlan, mindenki által kerülgetett fekete dobozzá. Az interfésznek, a verziózásnak és a hibakezelésnek ugyanolyan szintű szerződésen kell alapulnia, mint bármely külső szolgáltatónál.

A négy típus közötti különbséget érdemes felelősség és interakciós felület szerint összefoglalni. A stream-aligned csapat az üzleti értékfolyamért felel, és más csapatok képességeit fogyasztja. A platform csapat belső terméket szállít. Az enabling csapat tudást ad át. A complicated-subsystem csapat bonyolult alrendszert izolál. Egy szervezetben általában több stream-aligned csapat van, egy vagy néhány platform csapat, időszakos enabling csapatok, és csak indokolt esetben complicated-subsystem csapatok.

18.2.7 Szervezeti interakciós minták

A csapattípusok önmagukban nem elegendők; a típusok közötti interakciókat is tudatosan kell megválasztani. A Team Topologies három alapvető interakciós mintát azonosít: együttműködés (collaboration), X-as-a-Service, és facilitálás (facilitating). Mindegyik más kommunikációs terhet, más csatolási szintet és más felelősségi határokat jelent.

18.2.8 Együttműködés (collaboration)

Az együttműködés olyan interakciós mód, amelyben két csapat közösen dolgozik egy probléma megoldásán, gyakran közös cél, közös sprint vagy közös munkaterület keretében. Ez a minta akkor megfelelő, ha a rendszer határai még nem tisztázottak, ha új területet fedeznek fel, vagy ha két csapat közötti szerződést kell kialakítani. Az együttműködés nagy kommunikációs sávszélességet igényel: a csapatok rendszeresen szinkronizálnak, közösen terveznek, esetenként közösen írnak kódot. A minta hosszú távon drága, mert fenntartja a csapatok közötti szoros csatolást. Ezért az együttműködésnek általában időben korlátozottnak kell lennie: a cél az, hogy a feltárás után stabil szerződést hozzanak létre, és a kapcsolat X-as-a-Service vagy más, lazább mintává alakuljon. Ha két csapat éveken át folyamatos együttműködést igényel, az arra utal, hogy a rendszerhatárok rosszul vannak meghúzva, vagy a szervezeti felelősségek nem egyértelműek.

18.2.9 X-as-a-Service

Az X-as-a-Service minta a szolgáltatásként való fogyasztást jelenti. Az egyik csapat úgy biztosítja a képességét, mint egy külső szolgáltató: jól dokumentált API-val, szerződéssel, verziózással és SLA-val. A másik csapat fogyasztja ezt a szolgáltatást anélkül, hogy ismernie kellene a belső megvalósítást. Ez a minta alacsony kommunikációs terhet jelent, mert a felek a szerződésen keresztül kommunikálnak, nem pedig személyes egyeztetésekkel. A platform csapat tipikusan X-as-a-Service módon adja a golden pathot a stream-aligned csapatoknak. Ugyanígy a complicated-subsystem csapat a bonyolult alrendszert szolgáltatásként teszi elérhetővé. Az X-as-a-Service minta csak akkor működik megbízhatóan, ha a szolgáltatás szerződése stabil, a hibakezelés egyértelmű, és a fogyasztó visszajelzést tud adni a szolgáltatónak. A szolgáltató félnek érett termékszemlélettel kell rendelkeznie; ha a platform csapat minden stream-aligned igényre egyedi megoldást ad, az nem X-as-a-Service, hanem rejtett együttműködés.

18.2.10 Facilitálás (facilitating)

A facilitálás az enabling csapat tipikus interakciós mintája. Az enabling csapat nem ad át kész szolgáltatást, és nem végzi el a stream-aligned csapat helyett a munkát, hanem segíti a csapatot abban, hogy önállóvá váljon. Ez lehet közös feladatmegoldás, coaching, oktatás, vagy együtt elvégzett kísérlet. A facilitálás szintén időben korlátozott, de a célja a tudás átadása, nem pedig a szerződés létrehozása. A facilitálás során a csapatok között szoros együttműködés van, de az irány más: nem két termékcsapat közös tulajdonlásáról van szó, hanem egy tudáscsapat átadja

a képességet egy termékcsapatnak. A sikeres facilitálás mércéje, hogy a stream-aligned csapat később ugyanazon a területen önállóan tud dolgozni, és az enabling csapat felszabadul más csapatok segítésére.

A három interakciós minta nem kizárólagos, de a legtöbb egészséges szervezeti kapcsolat ezek valamelyikére visszavezethető. A tervezés során azt kell eldönteni, hogy két adott csapat között melyik minta csökkenti a teljes szervezeti terhelést és biztosítja a megfelelő csatolási szintet.

18.2.11 Conway-törvény és a szervezeti terhelés csökkentése

A Conway-törvény bölcsessége abban rejlik, hogy a szervezeti struktúrát nem elég utólag a rendszerhez igazítani; a rendszertervezést és a szervezettervezést párhuzamosan kell végezni. Ha egy rendszer moduláris, de a szervezet szerteágazó, gyakori kommunikációt igénylő csapatokból áll, akkor a modulok közötti implicit csatolás előbb-utóbb megjelenik. Ha egy szervezetben a csapatok jól elhatárolt értékfolyamokért felelnek, akkor a rendszerben is nagyobb eséllyel lesznek világos szolgáltatáshatárok. Ez nem azt jelenti, hogy a szervezeti ábra egy az egyben determinálja a kódot, hanem hogy a tervezés során figyelembe kell venni a szervezeti tükröződést.

A szervezeti terhelés tudatos kezelése azt jelenti, hogy a csapattípusok és interakciós minták kialakításakor azt kérdezzük: mennyi kognitív terhet ró ez a döntés az egyes csapatokra, és ez a teher indokolt-e? A stream-aligned csapatok terhelését általában az intrinsic terhelésen kívül (a termékterület bonyolultsága) csökkenteni kell extraneous terhelésből (például platform részletek, szervezeti koordináció) és megfelelő germane terhelést hagyni (a tanuláshoz és a megoldáshoz szükséges erőfeszítés). A platform csapat és az enabling csapat éppen az extraneous terhelés csökkentését szolgálja. A complicated-subsystem csapat az intrinsic terhelést szigeteli el. A team topologies tervezés nem öncélúan csökkenti a csapatok számát vagy a kommunikáció gyakoriságát, hanem a terhelést próbálja a nyugalmi, fenntartható szint alá hozni.

18.2.12 Gyakorlati példa: csapattopológia egy online kiskereskedelmi rendszerben

Tegyük fel, hogy egy közepes méretű online kiskereskedelmi vállalat szeretné átalakítani a fejlesztői szervezetét. A termék három fő értékfolyamot azonosít: katalógus és ajánlók, rendeléskezelés, valamint ügyfélszámla és hűségprogram. A jelenlegi szervezetben egyetlen nagy „fejlesztési osztály” van, amely mindhárom területet vegyesen szolgálja, és külön üzemeltetési csapat kezeli a deploymentet. A gyakori panasz: lassú release-ek, gyakori várólisták, és a fejlesztőknek minden infrastruktúra-részletet ismerniük kell.

A javasolt topológiában három stream-aligned csapatot hozunk létre, mindegyik a saját értékfolyamáért felel. Emellett egy platform csapatot alakítunk, amely az önkiszolgáló Kubernetes-alapú platformot, a megfigyelhetőségi eszközöket és a CI/CD sablonokat biztosítja. Az enabling csapat ebben az időszakban egy biztonsági szakértőkből álló kis csapat, amely a stream-aligned csapatoknak segít bevezetni a fenyegetésmodellezést és a biztonságos kódolási gyakorlatot. Végül létrehozunk egy complicated-subsystem csapatot a termékajánló motorra, amely gépi tanulási modelleket és speciális adatcsöveket tartalmaz, és amelynek belső bonyolultságát a stream-aligned csapatok nem akarják magukba fogadni.

A csapatok közötti interakciós térkép Mermaid diagramként ábrázolható:

```
graph TD
    S1[Stream-aligned: Katalógus] -->|X-as-a-Service| S2[Stream-aligned: Rendeléskezelés]
    S1 -->|X-as-a-Service| S3[Stream-aligned: Ügyfélszámla]
    S2 -->|Collaboration| C[Complicated-subsystem: Ajánló motor]
    P[Platform: Konténerplatform és CI/CD] -->|X-as-a-Service| S1
```

```
P -->|X-as-a-Service| S2
P -->|X-as-a-Service| S3
E[Enabling: Biztonsági csapat] -->|Facilitating| S1
E -->|Facilitating| S2
E -->|Facilitating| S3
```

A diagram csak a domináns interakciókat mutatja. A katalógus csapat és a rendeléskezelés csapat közötti X-as-a-Service kapcsolat például azt jelenti, hogy a rendeléskezelés a katalógus szolgáltatás API-ján keresztül kérdezi le a termékárakat és az elérhetőséget, anélkül, hogy ismerné a katalógus belső adatsémáját vagy a készletkezelő rendszert. A rendeléskezelés és az ajánló motor között együttműködés van, mert a vásárlói kosár alapján a rendelés pillanatában kell ajánlatot kérni, és a két csapatnak közösen kell kialakítania a szerződést arról, hogy milyen adatokat ad át, milyen időkorláttal és milyen hibamódokkal. Miután a szerződés stabilizálódott, ezt a kapcsolatot X-as-a-Service-re lehet alakítani.

A platform csapat mindhárom stream-aligned csapatnak X-as-a-Service módon adja a deploymentet: a stream-aligned csapatok a platform által biztosított pipeline-definíciót és namespace-konfigurációt használják, és a platform API-ján keresztül kérnek erőforrást. A biztonsági enabling csapat időszakosan facilitálja a csapatokat: workshopot tart, közösen végez fenyegetésmodellezést, majd a csapatok átveszik a napi szintű biztonsági gondolkodást.

Ebben a példában a szervezeti terhelés csökken: a stream-aligned csapatok a saját termékterületükre koncentrálhatnak, a platform csapat elviszi az infrastruktúra terhét, az enabling csapat lehetővé teszi a biztonsági tudás megszerzését, a complicated-subsystem csapat pedig izolálja az ajánló motor bonyolultságát.

18.2.13 Tipikus hibák és ellenszerek

A Team Topologies alkalmazása során gyakori hibák az alábbiak.

1. A platform csapat mindenki szolgálója lesz. Ha a platform csapat egyedi igényeket teljesít ticketeken keresztül, akkor nem önkiszolgáló platformot épít, hanem rejtett üzemeltetési részleggé válik. Ezzel a stream-aligned csapatok továbbra is függőségben maradnak, és a várakozási idő nem csökken. Az ellenszer: a platform csapat termékdefiníciót kap, méri a self-service arányt, és visszautasítja az egyedi, nem általánosítható megoldásokat, vagy azokat az enabling csapat segítségével átadja a stream-aligned csapatnak.

2. Az enabling csapat tartós kisegítővé válik. Ha az enabling csapat folyamatosan „besegít” a stream-aligned csapatoknak ahelyett, hogy tudást adna át, akkor a facilitálás nem ér véget, és a stream-aligned csapat soha nem válik önállóvá. Az ellenszer: az enabling tevékenységhez időkeretet és kimeneti célt kell rendelni, például „két hónap alatt a rendeléskezelés csapat önállóan tud threat modelinget végezni”. A facilitálás akkor sikeres, ha az enabling csapat kivonulhat.

3. A stream-aligned csapatok közötti folyamatos együttműködés túlterheli a szervezetet. Ha minden stream-aligned csapat minden másikkal folyamatosan egyeztet, akkor a kommunikációs háló sűrűvé válik, és a döntési idő megnő. Az ellenszer: a csapatok között dominánsan X-as-a-Service szerződéseket kell kialakítani, és az együttműködést időben korlátozott feltárási fázisra kell szorítani.

4. A complicated-subsystem csapat elszigetelődik, és nincs elég interfész-definíciója. Ha a bonyolult alrendszer csapat nem ad ki stabil szerződést, a stream-aligned csapatoknak a belső részleteket kell megtanulniuk, vagy a rendszer fekete dobozként viselkedik, ami integrációs hibákat okoz. Az ellenszer: a complicated-subsystem csapatnak ugyanolyan szigorú API-verziózást és dokumentációt kell fenntartania, mint egy külső szolgáltatónak.

5. A csapattípusokat munkakörökként kezelik. Előfordul, hogy egy szervezetben valakinek a titulusa „platform engineer”, egy másiké „stream-aligned fejlesztő”, de ettől a csapat még nem feltétlenül viselkedik a kívánt módon. A csapattípus felelősségi és interakciós minta, nem címke. Az ellenszer: a csapatok küldetését, interfészeit és interakciós mintáit rögzíteni kell, és ezt a dokumentációt rendszeresen felül kell vizsgálni.

18.2.14 Ellenőrzőlista és önellenőrző kérdések

Az alfejezet tanulási céljainak ellenőrzésére az alábbi ellenőrzőlista szolgál. A lista nem kizárólagos, de segít a csapattopológia értékelésében.

- Meghatároztuk-e a fő értékfolyamokat, és mindegyikhez rendeltünk-e stream-aligned csapatot?
- A stream-aligned csapatok felelőssége teljes értékfolyam, nem pedig egyetlen komponens?
- Van-e olyan platform csapat, amely önkiszolgáló módon csökkenti a stream-aligned csapatok terhelését?
- A platform csapatnak van-e termékdefiníciója, visszajelzési útvonala és siker-mérőszáma?
- Az enabling csapatok tevékenysége időben korlátozott és tudásátadó jellegű-e?
- A bonyolult alrendszerek mögött van-e dedikált complicated-subsystem csapat, amely stabil interfészt ad?
- A csapatok közötti interakciókhoz hozzárendeltünk-e domináns mintát (együttműködés, X-as-a-Service, facilitálás)?
- A folyamatos együttműködések mögött van-e terv a szerződéses kapcsolattá alakításra?
- Felmértük-e a kognitív terhelést csapatonként, és tettünk-e lépéseket az extraneous terhelés csökkentésére?
- Az interakciós térkép verziózottan és diagrams-as-code formában rendelkezésre áll-e?

Önellenőrző kérdés: Magyarázza el egy rövid bekezdésben, miért nem helyes, ha egy platform csapat minden stream-aligned csapat kérésére egyedi megoldást szállít, és melyik interakciós mintát kellene helyette választani! A válaszban térjen ki a szervezeti terhelés fogalmára is.

18.2.15 Labor: csapattopológia és interakciós térkép készítése

A következő labor célja, hogy a résztvevő egy kitalált vagy meglévő szervezeti problémára csapattopológiai tervet készítsen. A labor nem igényel futtatható kódot, de a dokumentum és a diagram verziózott, reprodukálható legyen. A kiinduló állapot, az elvárt eredmény és az ellenőrzési szempontok a következők.

Kiinduló állapot: Adott egy termékportfólió leírása, amely három értékfolyamot azonosít: „számlázás”, „ügyfélkapu” és „riportok”. A jelenlegi szervezetben egyetlen fejlesztői csapat dolgozik mindhárom területen, és egy külön üzemeltetési csapat végzi a deploymentet. A panaszok: gyakori merge-konfliktusok, lassú release, és a fejlesztőknek a konténerplatform összes beállítását ismerniük kell. A cél egy olyan csapattopológia és interakciós térkép, amely csökkenti a szervezeti terhelést.

Elvárt eredmény: Készítsen egy verziózott dokumentumot (például Markdown fájl), amely tartalmazza az alábbiakat:

1. A három stream-aligned csapat definícióját, mindegyik értékfolyam-leírásával.
2. Egy platform csapat definícióját, a biztosított önkiszolgáló képességek listájával.
3. Egy enabling csapat definícióját egy választott területre (például megfigyelhetőség vagy biztonság), időkerettel és kimeneti céllal.

4. Egy complicated-subsystem csapat definícióját egy bonyolult területre (például a riportok speciális számítási motorjára).
5. Egy Mermaid vagy PlantUML diagramot, amely a csapatok közötti domináns interakciós mintákat mutatja.
6. Egy rövid döntési naplót, amely rögzíti: kontextus, döntés, alternatívák, következmények, állapot. Legalább két döntést kell rögzíteni, például a platform csapat X-as-a-Service módjának kiválasztását és az enabling csapat facilitálásának időkeretét.

Ellenőrzési szempontok: A dokumentumban minden csapattípusnál szerepel-e a felelősség és a domináns interakciós felület? A diagramon jelölve van-e a rendszerhatár, és a csapatok közötti kapcsolatokon szerepel-e az interakciós minta neve? A döntési naplóban az alternatívák és a következmények valósak-e, nem pusztán általános kijelentések? A mért és a várt eredmény el van-e különítve (például a jelenlegi állapot panaszai mint mért megfigyelés, a céltopológia mint várt eredmény)?

Visszaállítási út: A labor dokumentumot hoz létre; ezért a visszaállítás a verziókezelő rendszerben történő commit előtti állapotra való visszatérést vagy a fájl archív példányának megőrzését jelenti. Nincs éles rendszer, így destruktív művelet nem értelmezett.

A labor elvégzése után a résztvevőnek képesnek kell lennie megindokolni, hogy miért az adott csapattípusokat választotta, és hogyan csökken a szervezeti terhelés a korábbi állapothoz képest. Ez a képesség közvetlenül átvezet a következő alfejezethez, amely a belső fejlesztői platform és az önkiszolgálás gyakorlati kialakítását tárgyalja.

18.3 Belső fejlesztői platform és önkiszolgálás

18.3.1 Bevezetés és fogalmi keret

A belső fejlesztői platform (internal developer platform, IDP) olyan önkiszolgáló rendszer, amely a fejlesztői csapatok számára egységes, automatizált felületet biztosít a szoftverek létrehozásához, üzemeltetéséhez és továbbfejlesztéséhez. Az IDP nem egyetlen termék, hanem több, egymással integrált komponens összessége, amely a fejlesztői út (journey) mentén csökkenti a kognitív terhelést, elrejti a mögöttes infrastruktúra bonyolultságát, és konzisztens, biztonságos alapértelmezéseket kínál. A platform engineering ezt a platformot termékként kezeli: van termékmenedzsere, ütemterve, mérőszámai és visszacsatolási ciklusa. A fejezet előzményében (Team Topologies és szervezeti architektúra) tárgyaltuk a Team Topologies keretrendszert, amelyben a platform csapat egy külön team típus, és a „platform mint termék” elv azt jelenti, hogy a platform csapat a belső fejlesztőket ügyfélként szolgálja ki. Most ebből kiindulva mutatjuk be az IDP komponenseit, az önkiszolgáló fejlesztői utat, valamint a bevezetés és a mérés gyakorlatát.

Az IDP célja nem az, hogy minden problémát egyszerre oldjon meg. A jó platform fokozatosan, a fejlesztői csapatok tényleges fájdalompontjai mentén épül ki, és a golden path (arany út) elvét követi: a leggyakoribb, támogatott utakat rendkívül egyszerűvé teszi, miközben a ritka, speciális esetekben meghagyja a csapatoknak a saját megoldás lehetőségét. Az IDP tehát nem korlátoz, hanem irányt mutat, és a nem támogatott utak költségét a csapatokra hárítja – ez a „paved road” koncepció.

18.3.2 Az IDP komponensei

Egy érett IDP jellemzően a következő funkcionális területeket fedi le. A komponenseket nem tekintjük egymástól függetlennek: az értékük az integrációban és a közös munkafolyamatban rejlik.

1. Szolgáltatáskatalógus (service catalog). A katalógus a szervezet szoftverkomponenseinek élő nyilvántartása. Minden szolgáltatáshoz tartozik: név, leírás, tulajdonos csapat (ownership), kapcsolattartó, kódrepozitórium, API-definíciók, dokumentáció, függőségek, deployment státusz, SLO-k és környezetek. A katalógus nem statikus dokumentum, hanem a CI/CD pipeline-okból, verziókezelőkből, monitorozó rendszerekből és konfigurációkezelő eszközökből automatikusan frissülő adatbázis. Manuális karbantartás esetén a katalógus gyorsan elavul, ezért a bejegyzések nagy részét metaadatokból (pl. repository-ba helyezett `catalog-info.yaml`) kell generálni. A katalógus interfésze lehet portál, CLI vagy API.

2. Szoftver- és infrastruktúra-sablonok (templates). A sablonok a szabványos kiindulási pontok. Lehetnek: - repository-sablonok: előre konfigurált váz, CI/CD pipeline-nal, Dockerfile-lal, tesztekkel, README-vel; - infrastruktúra-sablonok: Terraform-, Pulumi- vagy Helm-modulok, amelyek a szükséges erőforrásokat (adatbázis, üzenetsor, Kubernetes namespace) hozzák létre; - policy-sablonok: biztonsági házirendek, RBAC-konfigurációk, hálózati szabályok.

A sablonok verziózottak, és a platform csapat tulajdonolja őket. A fogyasztó csapatok nem módosíthatják a központi sablonokat, de kérhetnek új változatot vagy kivételt. A sablonok paraméterezhetők: például a szolgáltatásnév, a port, a környezetek száma, az adatbázis típusa. A paramétereket validálni kell, és a sablonnak csak azokat az értékeket szabad elfogadnia, amelyek megfelelnek a szervezeti házirendeknek.

3. CI/CD pipeline-ok (pipelines). Az IDP központi pipeline-okat vagy pipeline-sablonokat kínál, amelyeket a csapatok újrahasználnak. A pipeline-ok jellemzően három szinten jelennek meg: - *build pipeline*: fordítás, unit tesztek, statikus elemzés, konténerkép-építés; - *deployment pipeline*: környezetekbe való telepítés, integrációs tesztek, canary vagy blue-green stratégia; - *release pipeline*: jóváhagyási lépések, változásnapló, visszagördülési terv.

A pipeline definíciója is verziózott (pipeline-as-code), és a kódrepozitóriummal együtt tárolható. A platform csapat által biztosított pipeline-sablonok tartalmazzák a szervezeti megfelelőségi ellenőrzéseket (pl. SBOM generálás, titokvizsgálat, sebezhetőség-ellenőrzés), így a fejlesztői csapatnak nem kell ezeket külön beépítenie.

4. Környezetmenedzsment (environment management). Az IDP önkiszolgáló módon teszi lehetővé környezetek létrehozását, frissítését és megsemmisítését. A környezetek típusai: - *fejlesztői (dev)*: minden fejlesztőnek saját, elszigetelt környezete lehet; - *preview / review*: pull requestenként automatikusan létrejövő ideiglenes környezet, amelyben a változtatást integrációs tesztnek vetik alá; - *staging*: a produkcióhoz hasonló, de nem éles környezet; - *production*: az éles rendszer, szigorúbb felügyelettel.

A környezeteknek reprodukálhatónak és infrastruktúra-kód alapján létrehozhatónak kell lenniük. Az IDP kezeli a környezetek életciklusát (provisioning, frissítés, törlés), és a hozzáféréseket RBAC-szabályokkal korlátozza. Egy fontos mérőszám a környezet létrehozási ideje (environment provisioning time), amelyet a fejlesztői visszajelzésekben is számon kell tartani.

5. Szekréciós és hitelesítő adatkezelés (secrets management). Az IDP integrálódik egy vállalati titokkezelő rendszerrel (például HashiCorp Vault, felhőszolgáltató KMS-e), és a sablonokban, pipeline-okban automatikusan injektálja a szükséges hitelesítő adatokat. A fejlesztők soha nem látják a titkokat; csak a futó környezetek kapják meg azokat ideiglenes,

korlátozott jogosultságú formában. A titkok forgatását, lejáratát és audit naplóját a platform biztosítja.

6. Megfigyelhetőség (observability). Az IDP alapértelmezés szerint beköti a létrehozott szolgáltatásokat a szervezeti megfigyelő rendszerbe: loggyűjtés, metrikák, elosztott trace-ek. A fejlesztői portálon megjelennek a szolgáltatás állapotával kapcsolatos alapvető műszerfalak, és a riasztások a tulajdonos csapathoz kerülnek. Az OpenTelemetry-alapú beépített instrumentation sablonok biztosítják, hogy a szolgáltatások konzisztens módon szolgáltassák a telemetriai adatokat.

7. Önkiszolgáló interfészek (API, CLI, portál). Az IDP három fő felülettel rendelkezik: - *Fejlesztői portál*: böngészőben elérhető felület, ahol a fejlesztő böngészheti a katalógust, elindíthatja a sablonokat, láthatja a pipeline-ok állapotát, és hozzáférhet a dokumentációhoz. - *CLI*: parancssori eszköz, amely ugyanazokat a műveleteket szkriptelhetővé teszi, és a helyi fejlesztői munkafolyamatba illeszthető (pl. `platform create service --name my-service --template java-micro`). - *API*: a felettes rendszerek és automatizációk számára. A platform API maga is verziózott, és SLO-kkal rendelkezik.

8. Dokumentáció és irányelvek. Az IDP-hez tartozik egy könnyen kereshető, verziózott dokumentációs tér, amely tartalmazza a golden path-ok leírását, az API-referenciát, a bevált gyakorlatokat, a hibaelhárítási útmutatókat és a kivételkezelési folyamatot. A dokumentáció nem lehet holt szöveg: a portálba ágyazott interaktív példákkal és a katalógusból közvetlenül elérhető szolgáltatásleírásokkal kell összekapcsolni.

9. Audit és megfelelés. A platform csapat biztosítja, hogy minden létrehozott erőforrás, pipeline és környezet nyomon követhető legyen. Az audit naplók rögzítik, ki, mikor, milyen műveletet végzett, és ez összekapcsolható a változáskezelési folyamattal. A megfelelőségi házirendeket (pl. adatbesorolási címkék, titkosítási szabályok, régió-megkötések) a sablonokba és a pipeline-okba kell beépíteni, nem utólagos ellenőrzéssel.

Ezen komponensek közül nem mindegyikre van szükség az első iterációban. Az IDP-t inkább fokozatosan, a legnagyobb fájdalmat okozó területektől kiindulva érdemes építeni. A komponenslista kialakításánál az alábbi elveket kövessük:

- **Felelősséggel és interfésszel írjuk le** minden komponenst. Például a szolgáltatáskatalógus felelőssége a szolgáltatásregiszter naprakészen tartása; interfésze REST API és portál widget.
- **A rendszerhatárt jelöljük**: az IDP határa a szervezeti platform; a külső rendszerek (felhőszolgáltató, verziókezelő, CI/CD eszköz, monitoring) az IDP-n kívül vannak, de integrációs pontokkal kapcsolódnak.
- **A kapcsolatokon tüntessük fel a protokollt és a szinkronitást**: például a katalógus és a verziókezelő között aszinkron webhook-értesítés, a pipeline és a konténerregiszter között szinkron HTTPS hívás.

18.3.3 Az önkiszolgáló fejlesztői út

Az önkiszolgáló fejlesztői út (self-service developer journey) az a folyamat, amelyet egy fejlesztő vagy csapat végigjár, amikor új szolgáltatást szeretne létrehozni és üzemeltetni. A jó IDP ezt az utat minimalizálja és konzisztenssé teszi. Álljon itt egy tipikus, jól megtervezett út:

1. **Kérés indítása**: A fejlesztő a portálon vagy CLI-n kiválasztja a megfelelő sablont (pl. „Java mikroservice – REST API”). Megadja a szolgáltatás nevét, a csapatot, a kívánt környezeteket, az adatbázis típusát, a deployment régiót és egyéb paramétereket.

2. **Validáció és jóváhagyás**: A platform validálja a paramétereket a szervezeti házirendek alapján (pl. a név nem ütközik, a csapat jogosult a kért erőforrásra). Automatikus jóváhagyás esetén nincs manuális lépés; kivételes esetben a platform a csapatvezetőhöz irányítja a kérést.

3. **Erőforrás-létrehozás**: A platform a sablon alapján létrehozza a kódrepozitóriumot, a CI/CD pipeline definíciót, a Kubernetes namespace-t, a szükséges adatbázis-példányt, a hálózati házirendeket, a szolgáltatáskatalógus-bejegyzést, a monitoring műszerfalakat és a hozzáférési csoportokat. Mindezt egyetlen tranzakcióban vagy kompenzálható lépésekben, hogy részleges hiba esetén vissza lehessen gördíteni.

4. **Első deploy**: A pipeline automatikusan lefuttatja a sablonban lévő build és teszt lépéseket, majd telepíti az alkalmazást egy fejlesztői környezetbe. A fejlesztő a portálon látja a deployment állapotát, és megkapja a szolgáltatás elérési URL-jét.

5. **Fejlesztés és iteráció**: A fejlesztő kódot ír, commitol. A pipeline automatikusan buildel, tesztel, és pull request esetén létrehoz egy preview környezetet. A csapat review-ja után a merge kiváltja a staging deploymentet, majd a jóváhagyást követően a produkciós telepítést.

6. **Üzemeltetés**: A fejlesztő a portálon és a riasztásokon keresztül követi a szolgáltatás állapotát. Ha hiba történik, a beépített runbookok és a trace-ek alapján diagnosztizálhat. A platform a szolgáltatás SLO-jait is nyomon követi, és a hiba-büdzsét (error budget) automatikusan számolja.

7. **Továbbfejlesztés és megszüntetés**: A szolgáltatás életciklusa végén a fejlesztő a platformon keresztül kérheti a környezetek és erőforrások megszüntetését. A platform leállítja a szolgáltatást, törli az erőforrásokat, archiválja az audit naplókat, és frissíti a katalógust.

A fenti út időtartama az ideális esetben nem haladja meg a 15–30 percet az első lépéstől a sikeres első deploy-ig. Ennek a mérése kulcsfontosságú adoptációs mutató. Ha a folyamat órákat vagy napokat vesz igénybe, a fejlesztők visszatérnek a régi, nem támogatott módszerekhez.

A golden path itt azt jelenti, hogy a fenti út a támogatott, dokumentált és automatizált útvonal. A platform csapat felelőssége, hogy ez az út mindig működjön, és a fejlesztők ezt válasszák, amikor tehetik. A nem támogatott utak (például kézi infrastruktúra-létrehozás, saját pipeline-ok) nem tiltottak, de a csapatnak magának kell gondoskodnia a megfelelőségről, biztonságról és karbantartásról.

18.3.4 Sablon, katalógus, pipeline és környezet – mélyebb kapcsolatok

A fenti komponensek közül a sablon, a katalógus, a pipeline és a környezet szorosan összefügg, és együtt alkotják az önkiszolgálás gerincét. Vizsgáljuk meg ezeket egy konkrét példán keresztül.

Tegyük fel, hogy egy e-kereskedelmi vállalatnál a fejlesztői csapatok Java-alapú mikroszolgáltatásokat készítenek, Kubernetesre telepítenek, és PostgreSQL adatbázist használnak. A platform csapat a következőket biztosítja:

Sablon: `java-microservice-sql` néven. Paraméterei: `serviceName`, `team`, `port`, `dbName`, `environments` (dev, staging, prod). A sablon létrehozza: - egy Spring Boot alkalmazás vázát a megadott névvel, - egy Dockerfile-t, - egy Helm chartot a Kubernetes deploymenthez, - egy Terraform modult a PostgreSQL adatbázis létrehozásához, - egy CI/CD pipeline definíciót (pl. GitLab CI vagy GitHub Actions), - egy `catalog-info.yaml` fájlt, amely a szolgáltatást regisztrálja a katalógusba, - egy OpenTelemetry konfigurációt és alapvető metrika-definíciókat.

Katalógus: A szolgáltatás létrejöttekor a katalógus automatikusan felveszi a bejegyzést, és a későbbi pipeline-futások frissítik a deployment státuszt, a verziót és a függőségeket. A fejlesztői

portálon a csapat láthatja a szolgáltatás SLO-it, a tulajdonosokat, a kapcsolódó runbookokat és a változásokat.

Pipeline: A sablonban lévő pipeline a következő lépéseket tartalmazza: - kódellenőrzés és unit tesztek; - statikus biztonsági elemzés (SAST) és szoftver-összetevő-elemzés (SCA); - konténerképépítés és sebezhetőség-ellenőrzés; - deployment a dev környezetbe; - integrációs tesztek; - jóváhagyás után deployment stagingre, majd produkcióba canary stratégiával; - SBOM generálás és aláírás; - telemetriai adatok automatikus beküldése.

Környezet: A pipeline minden környezethez a Terraform modullal hozza létre a szükséges erőforrásokat, és a Helm charttal telepíti az alkalmazást. A preview környezeteket a platform automatikusan törli a pull request bezárása után. A produkciós környezet hozzáférése korlátozott, és minden változtatás audit naplóba kerül.

Ez az összekapcsolás biztosítja, hogy a fejlesztő ne keveredjen el az infrastruktúra részleteiben. A sablon a felelősségi határokat is rögzíti: a platform csapat felel a sablon működéséért és biztonságáért, a fejlesztői csapat felel az alkalmazáskódért és a szolgáltatás üzemeltetéséért a golden path-on belül.

18.3.5 A platform adoptációja és mérése

Az IDP sikere azon múlik, hogy a fejlesztői csapatok valóban használják-e. Az adoptáció méréséhez egyértelmű, verziószámhoz kötött mérőszámokat kell definiálni, és azokat rendszeresen, például havi gyakorisággal értékelni. A mérésekhez mindig tartozzon mérési ablak, kontextus és bizonytalanság; a nem reprodukált eredményt explicit módon jelölni kell.

Elfogadási mutatók (adoption metrics):

- *Új szolgáltatások aránya a platformon*: az adott hónapban létrehozott új szolgáltatások hány százaléka készült IDP-sablonnal. Célérték: $> 80\%$ (30 napos gördülő ablak).
- *Platformon létrehozott szolgáltatások abszolút száma*: darabszám/hónap.
- *Első deploymentig eltelt idő*: a kérés indításától az első sikeres deploymentig eltelt idő percben vagy órában. Célérték: < 30 perc.
- *Ismétlődő platformhasználat*: azon fejlesztők aránya, akik havonta legalább egyszer használják a platformot.
- *Churn a platformon*: azon csapatok aránya, akik a bevezetés után abbahagyták a platform használatát.

Teljesítménymutatók (DORA-mutatók): Bár ezek a teljes szoftverszállítási folyamatot mérik, az IDP hatását rajtuk keresztül is értékelhetjük. A DORA négy kulcsmutatója: - *Deployment frequency*: hány produkciós telepítés történt naponta vagy hetente. - *Lead time for changes*: a kód commitjától a produkcióba kerülésig eltelt idő (óra vagy nap). - *Change failure rate*: azon telepítések aránya, amelyek hibát okoznak és javítást igényelnek (%). - *Mean time to recovery (MTTR)*: a hiba bekövetkezésétől a helyreállításig eltelt idő (óra).

Ezeket a mutatókat a platform bevezetése előtt és után is mérni kell, hogy az IDP hatása kimutatható legyen. A mérési ablak legalább 30 nap legyen, és a bizonytalanságot (például kis mintaméretből adódó statisztikai zaj) dokumentálni kell.

Platform SLA/SLO: A platform API és az önkiszolgáló műveletek rendelkezésre állását is mérni kell. Például egy quality attribute scenario így fogalmazható meg:

Inger: 100 párhuzamos kérés a platform API-ra, fejenként 50 művelet/óra. **Környezet:** normál üzem, 20 fejlesztői csapat aktív használata. **Válasz:** a válaszidő 95. percentilise 200 ms, a hibaarány 0,5%. **Mérőszám:** 14 napos gördülő ablak, a válaszidőt a platform API gateway naplóiból számítjuk.

A rendelkezésre állás képlete megegyezik a megszokottal: $A = \frac{\text{üzemidő}}{\text{üzemidő}+\text{állásidő}}$, és az error budget $1 - \text{SLO}$.

Visszajelzési hurok: Az adoptáció mérése nem korlátozódik a fenti technikai mutatókra. Rendszeres fejlesztői elégedettségi felmérést (pl. negyedéves NPS vagy hasonló) is érdemes végezni, és a platform fejlesztési ütemtervét a visszajelzések alapján módosítani. A platform csapatnak termékmenedzserrel kell rendelkeznie, aki a belső ügyfelek igényeit gyűjti és priorizálja.

18.3.6 Tipikus hibák és anti-minták

Az IDP bevezetése során gyakran elkövetett hibák:

1. **A platformot egyetlen eszközre szűkítik.** Sokan azt hiszik, hogy egy szolgáltatáskatalógus (pl. Backstage) bevezetése egyenlő az IDP-vel. A katalógus fontos, de önmagában nem ad önkiszolgáló képességet; szükség van sablonokra, pipeline-okra és környezetmenedzsmentre is.
2. **Túl sok szabványosítás, túl korán.** Ha a platform minden csapatra rákényszeríti a teljes toolchain-t, a csapatok ellenállnak. A golden path-ot először egy-két csapat igényeire érdemes szabni, és fokozatosan bővíteni.
3. **Nem termékként kezelik a platformot.** Ha nincs dedikált platform csapat, nincs ütemterv, és a fejlesztők nem kapnak választ a kéréseikre, a platform gyorsan elavul.
4. **A katalógus manuális karbantartása.** Ha a szolgáltatásregisztert kézzel kell frissíteni, az adatok hamar elavulnak, és a fejlesztők elveszítik a bizalmat.
5. **Nincs mérés vagy rossz metrikákat mérnek.** Csak a platform API uptime-ját nézik, nem az adoptációt vagy a fejlesztői élményt. A platform sikerét az határozza meg, hogy mennyire gyorsítja a fejlesztői munkát.
6. **A pipeline-ok túl merevek.** Ha a központi pipeline nem paraméterezhető, a csapatok saját pipeline-okat fognak írni, ami fragmentációhoz vezet. A platformnak lehetőséget kell adnia a sablonból való kilépésre (escape hatch), de akkor a csapat viseli a felelősséget.
7. **Elhanyagolják a környezetek életciklusát.** Ha a preview környezetek nem törlődnek automatikusan, a költségek megnőnek, és a fejlesztők összezavarodnak. A környezetek provisioningje legyen gyors és determinisztikus.

18.3.7 Labor: IDP-komponenslista és bevezetési ütemterv

Cél: A résztvevő készítsen egy reprodukálható dokumentumot, amely tartalmazza egy fiktív szervezet IDP-jének komponenslistáját, bevezetési ütemtervét, döntési naplóját és ellenőrzési szempontjait. A dokumentumot verziózott formában kell elkészíteni, például Markdown fájlban, és a döntéseket ADR-formátumban rögzíteni.

Kiinduló állapot: A szervezet egy közepes méretű e-kereskedelmi vállalat. Jelenleg három fejlesztői csapat dolgozik: egy monolitikus Java alkalmazáson, két Node.js és Python alapú mikroszolgáltatáson. A fejlesztők manuálisan hozzák létre a környezeteket, a CI/CD pipeline-ok minden repóban külön vannak, nincs központi katalógus. A csapatok panaszkodnak a lassú

bevezetésre és a környezetek közötti inkonzisztenciára. A platform csapat most alakul, négy mérnökkel és egy termékmenedzserrel.

Feladat: Készíts egy dokumentumot `idp-proposal.md` néven, amely a következőket tartalmazza:

1. **Kontextus és problémameghatározás:** Röviden írd le a jelenlegi állapotot és a fő fájdalompontokat (2-3 bekezdés).
2. **IDP-komponenslista:** Sorold fel az általad javasolt komponenseket, mindegyikhez add meg a felelősségét és a fő interfészét. Használd a fejezetben bemutatott komponenslistát, de a szervezet méretéhez igazítsd: nem kell minden komponenst az első iterációban bevezetni.
3. **Bevezetési ütemterv:** 3 fázist határozz meg, mindegyikhez 1-2 hetes időtartammal, konkrét mérföldkövekkel és az adott fázisban bevezetett komponensekkel.
4. **ADR-ek:** Legalább két ADR-t írj, például a szolgáltatáskatalógus technológiájának kiválasztásáról és a CI/CD pipeline-sablon megközelítéséről. Használd az ADR-sablont: kontextus, döntés, alternatívák, következmények, állapot.
5. **Mérési terv:** Határozz meg legalább 4 mérőszámot az adoptáció és a teljesítmény mérésére, mindegyikhez mértékegységet és mérési ablakot.
6. **Ellenőrzési szempontok:** Sorold fel, hogyan ellenőriznéd a platform sikeres működését (pl. fejlesztői interjúk, pipeline-naplók, katalógus-adatok).

Elvárt eredmény: A dokumentum reprodukálható, azaz bárki, aki elolvassa, képes megérteni a döntési logikát és a javasolt komponenseket. A dokumentum verziózott (például git repo-ban), és a döntésekhez visszakereshető hivatkozás tartozik.

Ellenőrzési szempontok a labor értékeléséhez: - A komponenslista minden eleme rendelkezik felelősséggel és interfésszel. - Az ütemterv reális és a szervezet méretéhez igazodik. - Az ADR-ek tartalmazzák az alternatívákat és a következményeket. - A mérőszámok mértékegységgel és mérési ablakkal szerepelnek. - A dokumentum nem tartalmaz valós hitelesítő adatot vagy belső URL-t.

Visszaállítási út: Mivel a labor dokumentumkészítés, nincs szükség infrastruktúra-visszaállításra, de a dokumentumot verziózott repozitóriumban kell tartani, és a változtatásokat commit üzenetekkel nyomon követni.

18.3.8 Összefoglalás és ellenőrzőlista

A belső fejlesztői platform a fejlesztői csapatok önkiszolgáló képességének alapja. Az IDP komponensei – katalógus, sablonok, pipeline-ok, környezetmenedzsment, szekréciókezelés, megfigyelhetőség, interfészek, dokumentáció és audit – együttesen teszik lehetővé, hogy a fejlesztők percek alatt, konzisztens módon hozzanak létre és üzemeltessenek szolgáltatásokat. A platform akkor sikeres, ha termékként kezelik, fokozatosan vezetik be, és adoptációját, teljesítményét folyamatosan mérik.

Ellenőrzőlista a fejezethez:

- ☐ Meg tudom nevezni az IDP legalább hat fő komponensét, és mindegyikhez tudok felelősséget és interfészt rendelni.
- ☐ Le tudom írni az önkiszolgáló fejlesztői út lépéseit az új szolgáltatás létrehozásától a produkciós üzemeltetésig.
- ☐ El tudom magyarázni a golden path és az escape hatch közötti különbséget, és meg tudom indokolni, miért fontos mindkettő.
- ☐ Felismerem az IDP bevezetésének tipikus hibáit, és tudok ellenpéldát hozni.

- ☐ Képes vagyok legalább négy adoptációs mérőszámot definiálni mértékegységgel és mérési ablakkal.
- ☐ El tudom készíteni egy egyszerű IDP-komponenslistát és bevezetési ütemtervet egy adott szervezeti kontextusban.

Önellenőrző kérdés: Milyen különbség van egy szolgáltatáskatalógus és egy teljes IDP között, és miért nem elegendő önmagában a katalógus bevezetése? Válaszában térjen ki az önkiszolgáló képesség és a pipeline-ok szerepére is.

18.3.9 Fogalomtár röviden

- **IDP (Internal Developer Platform):** belső fejlesztői platform, önkiszolgáló rendszer a szoftverek létrehozására és üzemeltetésére.
- **Golden path:** a platform által támogatott, automatizált és dokumentált útvonal.
- **Escape hatch:** a platform által nem támogatott, de megengedett alternatív út, ahol a csapat maga viseli a felelősséget.
- **Platform csapat:** a platformot termékként üzemeltető, dedikált csapat.
- **DORA-metrikák:** a szoftverszállítási teljesítmény négy kulcsmutatója (deployment frequency, lead time, change failure rate, MTTR).

18.4 GitOps, környezetek és konfigurációkezelés

A GitOps a platform engineering egyik legfontosabb üzemeltetési mintája, amely a deklaratív konfigurációt és a verziózott üzemeltetést egyetlen zárt folyamatba rendezi. A fejezet előfeltételeként az előző egységben megismert belső fejlesztői platform (IDP) és a golden path fogalma jelenti azt a keretet, amelyben a GitOps értelmezhető: az önkiszolgáló fejlesztői élményhez szükség van arra, hogy a platform ne csak leírja, hanem automatikusan, megbízhatóan és ellenőrizhetően érvényre is juttassa a kívánt állapotot.

A fejezet három pillérre épít. Először definiáljuk a deklaratív konfiguráció és a verziózott üzemeltetés fogalmát, és összevetjük a hagyományos, imperatív üzemeltetési modellel. Ezután a GitOps vezérlési hurkát, a drift fogalmát, a reconciliációt és a titokkezelést tárgyaljuk. Végül a fejlesztői, staging és termelési környezetek szétválasztását, a konfigurációkezelési stratégiákat és a környezeti promóciót mutatjuk be egy reprodukálható labor keretében.

18.4.1 Deklaratív konfiguráció és verziózott üzemeltetés

A hagyományos üzemeltetési munkafolyamatok imperatív parancsokra épülnek: a mérnök egymás után hajt végre utasításokat, például `apt install`, `kubectl apply -f deployment.yaml`, `systemctl restart szolgáltatás`. Az imperatív modell leírja *hogyan* érjük el a célt, de nem rögzíti magát a végső állapotot. Ha a parancsot kétszer futtatjuk, lehet, hogy más eredményt kapunk, ha a kiindulási állapot eltér. Az idempotencia és a megismételhetőség így esetleges.

A deklaratív konfiguráció ezzel szemben a *kívánt végállapotot* írja le. A rendszergazda vagy fejlesztő nem parancsokat ad meg, hanem egy dokumentumot: hány replikának kell futnia, milyen erőforrásokkal, milyen beállításokkal. Az üzemeltető réteg felelőssége, hogy a jelenlegi állapotot folyamatosan a deklarált állapothoz közelítse. Kubernetesben ez a controllerek és az etcd együttműködése; a hagyományos szervereknél ez lehet Ansible playbook, Terraform state vagy Puppet manifest.

Példa Kubernetes Deployment YAML:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: registry.example.com/web:1.4.0
          ports:
            - containerPort: 8080
```

Ez a fájl nem mondja meg, hogy "indíts el három podot", hanem kijelenti: "három replikának kell futnia, ezzel a képpel és porttal." A Kubernetes Deployment kontroller folyamatosan figyeli a tényleges podok számát, és ha eltérés van, beavatkozik.

Verziózott üzemeltetés alatt azt értjük, hogy a deklaratív konfigurációk ugyanabban a verziókezelő rendszerben élnek, mint az alkalmazáskód, és minden módosításuk ugyanazon a review, merge és audit folyamaton megy keresztül. A Git a működés egyetlen igazságforrása. Nincs többé "hátha a szerveren maradt egy kézi javítás" – minden, ami a környezetet érinti, egy commitban kereshető, és egy merge request által jóváhagyott.

A verziózott üzemeltetés előnyei:

- **Visszaállíthatóság:** bármely korábbi konfigurációs állapot egyetlen `git revert` vagy `git checkout` paranccsal helyreállítható.
- **Auditálhatóság:** a commit előzmények megmutatják, ki, mikor és miért módosított.
- **Review folyamat:** a konfigurációváltozások ugyanolyan kódminőségi ellenőrzésen esnek át, mint az alkalmazás logika.
- **Környezetek közötti reprodukálhatóság:** a konfiguráció ugyanabból a forrásból származik, elkerülhető a "nálam működik, prodban nem" jelenség.
- **Automatizálhatóság:** a deklaratív fájlok gépi feldolgozása lehetővé teszi a folyamatos alkalmazást és a drift detektálást.

18.4.2 A GitOps vezérlési hurka

A GitOps alapelveit először a Weaveworks fogalmazta meg, de a minta ma már a CNCF égisze alatt is meghatározó. A három legfontosabb elem:

1. **A Git az egyetlen igazságforrás.** Minden, ami a környezetet definiálja – deployment fájlok, service-ek, configmapek, házirendek, titkosított titkok – a Git repóban található.
2. **A kívánt állapot deklaratív.** A repóban lévő fájlok nem parancsokat, hanem a végső állapotot írják le.
3. **Automatikus alkalmazás zárt hurokban.** Egy ügynök a klaszteren belül folyamatosan figyeli a Git repót, és amikor változást észlel, automatikusan alkalmazza a klaszterre. Ugyanakkor a klaszter tényleges állapotát is figyeli, hogy a kívánt állapottól való eltérést észlelje és javítsa.

Két elterjedt GitOps eszköz létezik: az Argo CD és a Flux. Mindkettő egy Kubernetes kontrollerként fut, és a fenti elveket valósítja meg. Az Argo CD fogalmait használva a folyamat így írható le:

- A fejlesztő módosít egy deployment YAML fájlt a Git repóban, és merge requestet nyit.
- A code review és a CI ellenőrzések után a merge megtörténik a `main` ágon.
- Az Argo CD Application erőforrása figyeli a repót (polling vagy webhook útján), érzékeli a commitot.
- Az Argo CD összehasonlítja a repóban lévő kívánt állapotot a klaszter tényleges állapotával.
- Ha eltérés van, akkor a `sync` művelet során alkalmazza a módosításokat.
- A sync után az Argo CD továbbra is figyeli a klasztert, és ha valaki kézzel megváltoztat valamit, a rendszer driftet észlel, és figyelmeztet vagy automatikusan visszaállít.

Az alábbi Mermaid diagram ezt a folyamatot szemlélteti:

```
graph LR
  A[Fejlesztő] -->|git push / merge request| B[Git repó]
  B -->|webhook / polling| C[GitOps ügynök<br>Argo CD / Flux]
  C -->|deklaratív sync| D[Kubernetes klaszter]
  D -->|tényleges állapot| C
  C -->|drift detektálás| B
```

A zárt hurok lényege, hogy a vezérlő nemcsak a Gitből a klaszter felé szinkronizál, hanem visszafelé is figyeli a klasztert. Ez teszi lehetővé a drift kezelését.

18.4.3 Drift, reconciliáció és ellenőrzés

A **drift** (sodródás) az a jelenség, amikor a rendszer tényleges állapota eltér a deklarált, Gitben rögzített állapottól. Drift keletkezhet kézi beavatkozásból (valaki a klaszteren `kubectl scale` paranccsal módosítja a replikaszámot), külső rendszer változásából (egy automatikus autoscaling), vagy hibás konfigurációból (a sync sikertelen).

A drift veszélye, hogy a rendszer idővel "elkoszolódik". A környezet olyan módosításokat tartalmaz, amelyek nem szerepelnek a Gitben, így nem reprodukálhatók, nem auditálhatók, és egy katasztrófa-helyreállítás során elvesznek.

A **reconciliáció** (kiegyenlítés, egyeztetés) az a folyamat, amelynek során a GitOps ügynök a tényleges állapotot a kívánt állapothoz igazítja. Ez a Kubernetes vezérlési hurkának (control loop) mintájára működik:

```
figyeld a kívánt állapotot (desired state)
olvasd a tényleges állapotot (current state)
ha eltérnek:
    hajt végre korrekciót (reconcile)
ismételd
```

A GitOps eszközökben a reconciliáció lehet automatikus (auto-sync) vagy manuális (csak jelzés, és egy jóváhagyó gombra kattintva történik meg). Az automatikus sync kényelmes, de termelési környezetben gyakran manuális jóváhagyást kérnek, hogy a változások ellenőrzötten kerüljenek ki.

Fontos megkülönböztetni a driftet a nem kívánt változástól. A GitOps nem tiltja meg a kubernetes autoscalingot vagy a HPA-t, de a HPA konfigurációját is a Gitben kell tárolni. Ha a HPA módosítja a replikák számát, az a rendszer tervezett viselkedése, nem drift. A drift akkor lép fel, ha olyan változás történik, amely nem vezethető vissza a deklarált állapotra. A GitOps eszközök ezért tudnak különbséget tenni a "drift" és a "managed field" változások között.

A drift detektálás mérőszámokkal is ellenőrizhető. Egy lehetséges működési metrika a *drift időtartam*: mennyi idő telik el a nem szándékos eltérés bekövetkezése és a korrekció között. Ezt quality attribute scenario formában így fogalmazhatjuk meg:

> **Minőségi forgatókönyv – Drift detektálás Inger:** egy operátor kézzel módosítja a web deployment replikaszámát 3-ról 5-re a termelési klaszterben. **Környezet:** termelési Kubernetes klaszter, Argo CD auto-sync engedélyezve. **Válasz:** az Argo CD 30 másodpercen belül észleli a driftet, és visszaállítja a replikaszámot 3-ra. **Mérőszám:** a drift észleléséig eltelt idő, másodpercben, 30 másodperces mérési ablakban; a korrekció sikeres visszaállítása 100%-ban.

Ez a forgatókönyv az üzemeltethetőség és a megbízhatóság egyik fontos követelményét teszi mérhetővé.

18.4.4 Titokkezelés a GitOps folyamatban

A deklaratív konfiguráció egyik legnagyobb kihívása a titkok kezelése. A jelszavak, API kulcsok, tanúsítványok és adatbázis hitelesítő adatok természetüknél fogva érzékenyek, és nem tárolhatók nyíltan a Git repóban. Még privát repóban sem, mert a Git history megőrzi a korábbi verziókat, a jogosultságok pedig idővel változhatnak.

A helyes megközelítés két szinten működik:

1. **A titok forrása nem a Gitben van.** A tényleges titkokat külső rendszer tárolja: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager, vagy maga a Kubernetes `Secret` erőforrás, amely egyébként is only base64 kódolású.
2. **A Gitben csak hivatkozások vagy titkosított reprezentációk vannak.** A GitOps ügynök a szinkronizálás során a külső titokkezelőből kérdezi le az értéket, vagy a titkosított állapotot visszafejti a klaszteren belül.

Elterjedt megoldások:

- **Sealed Secrets:** a Kubernetes Bitnami projektje. A fejlesztő egy nyilvános kulccsal titkosítja a Secret objektumot, így a Gitben egy `SealedSecret` CRD jelenik meg. A klaszterben futó controller visszafejti, és létrehozza a tényleges Secretet. A titkosított fájl biztonságosan commitolható, mert csak a klaszter titkos kulcsa tudja visszafejteni.
- **External Secrets Operator:** a Gitben egy `ExternalSecret` erőforrás hivatkozik a külső titoktárolóra. A klaszterben futó operator lekéri a titkot a Vaultból vagy a felhőszolgáltató titokkezelőjéből, és létrehozza a Kubernetes Secretet.
- **SOPS (Secrets OPerationS):** a titkokat helyben titkosítja (például age vagy KMS kulccsal), és a titkosított szöveget tárolja a Gitben. A GitOps ügynök vagy a pipeline visszafejti az értéket a klaszteren.
- **HashiCorp Vault Agent:** a Vault oldalán egy agent injektálja a titkokat a podba, így a Git soha nem tartalmaz titkot.

Fontos alapelvek:

- Soha ne commitoljunk plaintext titkot a Gitbe.
- A titkosított titkokat is csak minimális jogosultságú környezetben tároljuk.
- A titkok rotációját automatizálni kell, és a rotáció után a GitOps szinkronizációnak gondoskodnia kell a frissítésről.

- A titok visszafejtéséhez szükséges kulcsok a klaszterben vagy a titokkezelőben éljenek, soha a Gitben.

Példa egy SealedSecret YAML-re (a tényleges titok helyőrző, nem valós):

```yaml
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: db-credentials
  namespace: prod
spec:
  encryptedData:
    username: AgBy3i4OJSWK+... # titkosított, nem valós érték
    password: AgB45h7kQ1...   # titkosított, nem valós érték
```

18.4.5 Fejlesztői, staging és termelési környezetek

A platform engineeringben a környezetek elkülönítése elengedhetetlen a biztonságos és ellenőrzött szoftverszállításhoz. Legalább három környezetet szokás megkülönböztetni:

- **Fejlesztői (dev):** a fejlesztők saját tesztjeire, integrációs próbákra, gyors iterációra. Gyakran lazább biztonsági beállításokkal, kevesebb erőforrással, alacsony adatminőséggel.
- **Staging (átmeneti vagy teszt):** a termeléshez lehető legközelebbi környezet, amelyben a release jelöltet validálják. Itt már termelés közeli konfiguráció, de szintetikus vagy anonimizált adatok vannak.
- **Termelési (prod):** az éles rendszer, ahol a felhasználók dolgoznak. Szigorú hozzáférésvezérlés, magas rendelkezésre állás, audit és megfelelés.

A GitOps keretében a környezetek elkülönítésének két fő módja van:

1. **Külön klaszterek:** minden környezet saját Kubernetes klaszterrel rendelkezik. Ez erős izolációt ad, de költségesebb és bonyolultabb.
2. **Külön namespace-ek egy klaszteren belül:** a környezetek namespace szinten vannak elválasztva. Olcsóbb, de a biztonsági határok gyengébbek, és a klaszter szintű erőforrások (CRD-k, házirendek) közösek.

Általában a dev és staging namespace-ek egy dedikált nem termelési klaszteren élnek, a prod pedig saját klaszteren, hogy a termelési stabilitás ne függjön a fejlesztői terheléstől.

A konfigurációkezelés eszközei közül a **Kustomize** és a **Helm** a legelterjedtebb. Mindkettő lehetővé teszi, hogy ugyanazt az alapkonfigurációt környezetenként eltérő paraméterekkel lássuk el.

A Kustomize overlay modellt használ: van egy `base` mappa, amely a közös manifesteket tartalmazza, és minden környezetnek van egy `overlay` mappája, amely csak a különbségeket írja le.

Példa repó struktúra:

```
config/
  base/
     deployment.yaml
     service.yaml
     kustomization.yaml
  overlays/
      dev/
         kustomization.yaml
         replicas.yaml
```

```
    staging/
        kustomization.yaml
        replicas.yaml
    prod/
        kustomization.yaml
        replicas.yaml
```

A `base/kustomization.yaml`:

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
```

A `overlays/dev/kustomization.yaml`:

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
patchesStrategicMerge:
  - replicas.yaml
```

A `overlays/dev/replicas.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
```

A prod overlay hasonló, de ott `replicas: 3`, és esetleg más resource limitek, image pull policy vagy node selector.

A Helm hasonló célt szolgál, de template-alapú: a `values.yaml` fájlokat környezetenként paraméterezzük. Mindkét eszköz elfogadott; a választás gyakran szervezeti preferencia kérdése. Fontos, hogy a környezeti eltérések explicit módon, verziózottan legyenek rögzítve, ne ad hoc módon a klaszteren.

18.4.6 Környezeti promóció a GitOps-ban

A **környezeti promóció** az a folyamat, amelynek során egy változás a dev környezetből a staging, majd a prod környezetbe kerül. A GitOps ezt természetesen támogatja, mert a környezetek a Git repó különböző részein vagy különböző ágain vannak reprezentálva.

A leggyakoribb megközelítések:

- **Mappa alapú promóció (monorepo):** minden környezet saját mappával rendelkezik ugyanabban a repóban. A promóció azt jelenti, hogy a dev mappából átmásoljuk (vagy cherry-pickeljük) a változást a staging, majd a prod mappába. Ez egyszerű, de a nagy monorepo méret és a jogosultságkezelés kihívást jelenthet.
- **Branch alapú promóció:** minden környezetnek saját branch van (pl. `main` -> dev, `staging`, `production`). A merge a `main`-ből a `staging`-be jelenti a promóciót. Ez a Git flow-hoz hasonlít, de könnyen konfliktusokhoz vezethet, és a GitOps eszközök többnyire a mappa alapú megközelítést preferálják.

- **Külön repók:** minden környezet külön Git repóban van. A promóciót egy pipeline végzi, amely commitol a staging repóba. Ez erős elkülönítést ad, de a repo szinkronizáció bonyolult.

A legtöbb csapat a monorepo + mappa megközelítést választja, mert egyszerű, átlátható és a review folyamat jól integrálható. A promóció gyakran nem automatikus: a dev és staging auto-sync fut, de a prod környezetre manuális jóváhagyás szükséges. Ezt a GitOps eszközökben "sync policy" és "promotion policy" szabályozza.

Például Argo CD-ben definiálhatunk egy Application erőforrást a prod környezetre, amelynél a `syncPolicy.automated` nincs bekapcsolva, hanem csak manuális sync engedélyezett. A staging és dev esetén engedélyezzük az automatikus syncet.

A promóció folyamata így néz ki:

1. Fejlesztő merge-eli a kódot a `main` ágra.
2. A CI legyártja az új konténerképet, és frissíti a deployment manifestet a `config/overlays/dev` mappában (vagy a GitOps eszköz image updater komponense teszi meg).
3. Az Argo CD észleli a dev mappa változását, auto-sync lefuttatja, a dev környezet frissül.
4. A tesztelés után a release mérnök manuálisan másolja a manifestet a `config/overlays/staging` mappába, és merge requestet nyit. A staging auto-sync frissíti a környezetet.
5. A staging validáció után ugyanez a folyamat a prod mappába, de ott manuális sync szükséges az Argo CD felületen vagy parancssorból.

Ez a folyamat biztosítja, hogy minden környezet ugyanazt a deklarált állapotot tartalmazza, és a promóció lépései auditálhatók a Git history-ban.

18.4.7 Gyakorlati példa: mini GitOps rendszer

Tegyük fel, hogy egy kis webalkalmazást üzemeltetünk, amelyet Kubernetesen futtatunk. A csapat három környezetet használ: `dev`, `staging`, `prod`. A Git repó szerkezete:

```
.
  app/
     Dockerfile
     src/
  config/
     base/
        deployment.yaml
        service.yaml
        kustomization.yaml
     overlays/
         dev/
            kustomization.yaml
            replicas.yaml
         staging/
            kustomization.yaml
            replicas.yaml
         prod/
             kustomization.yaml
             replicas.yaml
  gitops/
      dev-app.yaml
      staging-app.yaml
      prod-app.yaml
```

A `config/base/deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: registry.example.com/web:1.4.0
          ports:
            - containerPort: 8080
          env:
            - name: ENV
              value: "base"
```

A `config/overlays/dev/replicas.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: web
          env:
            - name: ENV
              value: "dev"
```

A `config/overlays/prod/replicas.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: web
          env:
            - name: ENV
              value: "prod"
```

Az Argo CD Application erőforrás a `gitops/prod-app.yaml`:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-prod
  namespace: argocd
spec:
```

```yaml
  project: default
  source:
    repoURL: https://git.example.com/org/webapp.git
    targetRevision: main
    path: config/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated: null   # manuális sync
    syncOptions:
      - CreateNamespace=true
```

A dev és staging alkalmazásoknál a `syncPolicy.automated` be van kapcsolva, például:

```yaml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

A fenti példában a `selfHeal: true` bekapcsolja a drift automatikus korrigálását, a `prune: true` pedig törli a Gitből már hiányzó erőforrásokat.

A teljes folyamat Mermaid digrammal ábrázolva:

```
sequenceDiagram
  participant Dev as Fejlesztő
  participant Git as Git repó
  participant ArgoCD as Argo CD
  participant DevK as Dev klaszter
  participant ProdK as Prod klaszter

  Dev->>Git: merge request a config/overlays/dev módosítására
  Git->>ArgoCD: webhook értesítés
  ArgoCD->>DevK: auto-sync a dev overlay-ből
  DevK-->>ArgoCD: state frissítés
  Note over ArgoCD: drift detektálás aktív<br>selfHeal: true

  Dev->>Git: promóció: staging mappa frissítése
  Git->>ArgoCD: webhook értesítés
  ArgoCD->>DevK: auto-sync a staging overlay-ből

  Dev->>Git: promóció: prod mappa frissítése
  Git->>ArgoCD: webhook értesítés
  ArgoCD-->>ProdK: manuális sync szükséges
  Note over ArgoCD: üzemeltető jóváhagyja a sync-et
  ArgoCD->>ProdK: sync a prod overlay-ből
```

Ez a példa szemlélteti, hogyan valósul meg a deklaratív, verziózott, drift-mentes üzemeltetés és a környezeti promóció.

18.4.8 Tipikus hibák és anti-minták

A GitOps bevezetése során a következő hibák a leggyakoribbak:

1. **Kézi beavatkozás a klaszteren.** Ha valaki közvetlenül `kubectl apply` vagy `kubectl edit` parancsot hajt végre, az driftet okoz. Bár a selfHeal visszaállíthatja, ez ideiglenes javítások elvesztéséhez és zavarhoz vezet. A szabály: minden módosítás a Gitben kezdődjön.

2. **Plaintext titok a repóban.** Ez a legsúlyosabb biztonsági hiba. Még ha később törlik is a fájlt, a Git history megőrzi. Azonnali titok rotáció szükséges, és a titokkezelési stratégia átgondolása.
3. **Környezetek közötti konfiguráció drift.** Ha a dev, staging és prod nem ugyanabból a base-ből származik, hanem független manifestek halmaza, akkor a környezetek fokozatosan eltérnek. A Kustomize overlays vagy Helm values használata ezt hivatott megelőzni.
4. **Nem megfelelő review folyamat.** A konfigurációváltozásokra ugyanolyan szigorú code review szükséges, mint az alkalmazáskódra. A "kis YAML javítás" is okozhat leállást.
5. **Hiányzó rollback terv.** Bár a Git lehetővé teszi a visszaállítást, ha nincs meghatározott eljárás arra, hogyan kell egy hibás release-t visszavonni, akkor pánikhelyzetben a csapat kézi beavatkozásokhoz nyúl. A rollback legyen dokumentált és tesztelt.
6. **A prod környezet túlzott automatizálása.** Bár az auto-sync kényelmes, termelésben a manuális jóváhagyás gyakran szükséges a kontrollált változáskezeléshez. A túlzott automatizálás a hibák azonnali propagálódását okozza.
7. **A drift figyelmen kívül hagyása.** Ha a GitOps eszköz driftet jelez, de senki nem foglalkozik vele, az azt jelenti, hogy a rendszer nem a deklarált állapotban van. A drift riasztásokat ugyanolyan komolyan kell venni, mint a szolgáltatás hibákat.

18.4.9 Ellenőrzőlista

- A környezeteket leíró összes konfiguráció a Git repóban van, és verziózott.
- A konfiguráció deklaratív, nem tartalmaz imperatív parancsokat.
- A GitOps ügynök aktív, és a sync állapot figyelve van.
- A drift detektálás be van kapcsolva, és a selfHeal vagy riasztás konfigurálva van.
- A titkok nincsenek plaintextben a Gitben; titkosított vagy külső titokkezelőre hivatkozó megoldás van.
- A fejlesztői, staging és prod környezetek elkülönítettek, és a konfigurációjuk közös base-ből származik.
- A környezeti promóció lépései dokumentáltak, és a prod környezetbe manuális jóváhagyás szükséges.
- A rollback eljárás ismert és tesztelt.
- A konfigurációváltozások code review-n esnek át.
- A rendszer képes visszaállítani a kívánt állapotot egy katasztrófa után a Gitből.

18.4.10 Önellenőrző kérdés

Magyarázza el, mi történik, ha egy üzemeltető a termelési Kubernetes klaszterben kézzel megváltoztatja egy Deployment replikaszámát 3-ról 5-re, miközben a GitOps rendszer auto-sync és selfHeal beállítással fut. Írja le a drift észlelésének és a reconciliációnak a lépéseit, és indokolja meg, miért fontos, hogy a módosítás a Gitben történjen.

18.4.11 Labor: GitOps folyamat és környezeti promóció tervezése

18.4.12 Cél

A labor célja, hogy a résztvevő egy kis, ellenőrizhető példán keresztül megtervezze és dokumentálja egy GitOps folyamatot három környezetre (dev, staging, prod), beleértve a drift kezelést, a

titokkezelést és a környezeti promóciót. Az eredmény egy reprodukálható dokumentum, amely döntési naplót és ellenőrzési szempontokat tartalmaz.

18.4.13 Kontextus

Egy csapat egy webes katalógusalkalmazást üzemeltet Kubernetesen. Jelenleg a telepítések kézi `kubectl apply` parancsokkal történnek, a környezetek konfigurációja ad hoc, a titkok egyes repókban plaintextben vannak. A cél egy GitOps alapú, verziózott, drift-mentes üzemeltetési modell bevezetése.

18.4.14 Feladat

1. **Tervezze meg a Git repó struktúráját** a Kustomize overlay minta alapján (`base` + `overlays/dev`, `staging`, `prod`). Adja meg a mappákat és a bennük lévő fájlokat.
2. **Definiálja a GitOps Application erőforrásokat** mindhárom környezethez. Írja le a sync policy-t: dev és staging auto-sync, prod manuális sync. Adja meg a drift kezelést (selfHeal bekapcsolása dev/staging esetén, prod esetén riasztás).
3. **Válasszon titokkezelési megoldást** a három megismert közül (Sealed Secrets, External Secrets, SOPS). Indokolja a választást egy ADR formátumban: kontextus, döntés, alternatívák, következmények, állapot.
4. **Dokumentálja a környezeti promóciós folyamatot** lépésről lépésre, beleértve a review és jóváhagyási pontokat. Készítsen egy egyszerű Mermaid szekvencia diagramot a folyamatról.
5. **Fogalmazzon meg egy quality attribute scenario-t** a drift detektálásra (inger, környezet, válasz, mérőszám).
6. **Készítsen ellenőrzési szempontokat** a bevezetés utáni auditáláshoz: hogyan ellenőrizhető, hogy a rendszer valóban a Gitben deklarált állapotot futtatja, nincs drift, és a titkok biztonságban vannak.

18.4.15 Elvárt eredmény

Egy reprodukálható dokumentum, amely tartalmazza:

- A repó struktúra leírását.
- A GitOps erőforrások YAML vázlatát (szemléltető, helyőrzőkkel).
- Egy ADR-t a titokkezelési döntésről.
- Egy Mermaid diagramot a promóciós folyamatról.
- Egy quality attribute scenario-t a driftre.
- Egy ellenőrzőlistát az auditáláshoz.

A dokumentum rögzítse a **kontextust** (a fenti feltételezett környezet), a **döntési naplót** (ADR formátum), és különítse el a **mért és a várt eredményt**. Mivel ez egy tervezési feladat, közvetlen mérésre nincs lehetőség, de a "várt eredmény" oszlopban fogalmazza meg, mit várunk a bevezetés után (például drift detektálási idő < 30 másodperc, 0 plaintext titok a Git historyban). A "mért eredmény" cellákat hagyja üresen, és jelölje, hogy a bevezetés után kell kitölteni.

18.4.16 Ellenőrzési szempontok

- A repó struktúra követi-e a Kustomize overlay mintát?
- A sync policy megfelelően van-e beállítva környezetenként?
- A titokkezelési ADR tartalmazza-e a négy kötelező elemet?
- A promóciós diagram helyesen ábrázolja-e a dev -> staging -> prod sorrendet és a manuális jóváhagyást?
- A quality attribute scenario tartalmazza-e az ingert, környezetet, választ és mérőszámot?
- Az ellenőrzőlista minden fontos szempontot lefed-e?
- A dokumentum elkülöníti-e a mért és várt eredményt?

18.4.17 Visszaállítási út

Mivel a labor csak tervezési dokumentumot készít, nincs éles rendszer, amelyet vissza kellene állítani. Ha a résztvevő egy tesztkörnyezetben is kipróbálja a konfigurációt, akkor a tesztkörnyezetet izoláltan kell létrehozni (például minikube vagy kind), és a labor végén törölni kell a klasztert a `kind delete cluster` paranccsal. A Git repóban lévő változtatásokat egy külön branch-en kell végezni, hogy a labor után Mergelés nélkül törölhető legyen.

18.4.18 Dokumentum sablon

A labor eredményét a következő szerkezetben kell leadni:

```
# GitOps labor - [résztvevő neve]

## Kontextus
...

## Repó struktúra
...

## GitOps erőforrások (YAML vázlat)
...

## ADR: Titokkezelési döntés
...

## Promóciós folyamat (Mermaid diagram)
...

## Quality attribute scenario
...

## Ellenőrzőlista
...

## Eredmények
| Szempont | Várt eredmény | Mért eredmény |
|----------|---------------|---------------|
| Drift detektálási idő | < 30 s | (bevezetés után) |
| Plaintext titok a Gitben | 0 | (bevezetés után) |
| ... | ... | ... |
```

A táblázatban a “Mért eredmény” oszlop üres, és egy megjegyzés jelzi, hogy a bevezetés után kell kitölteni.

18.4.19 Összegzés

A GitOps a deklaratív konfiguráció és a verziózott üzemeltetés gyakorlati megvalósítása. A Git mint egyetlen igazságforrás, a zárt hurkú reconciliáció és a drift automatikus kezelése együttesen biztosítják, hogy a platform üzemeltetése reprodukálható, auditálható és megbízható legyen. A titkok kezelése különös figyelmet igényel, és a környezetek (dev, staging, prod) elkülönítése, valamint a promóciós folyamat a kontrollált szoftverszállítás sarokkövei. A fejezetben bemutatott labor során a résztvevő megtanulta alkalmazni ezeket az elveket egy konkrét, ellenőrizhető tervezési feladatban. A következő fejezet a skálázhatóságra, teljesítményre és költségre épít, ahol a GitOps által biztosított stabil üzemeltetési alapra támaszkodva vizsgáljuk a horizontális és vertikális skálázást.

19 Skálázhatóság, teljesítmény és költség

19.1 Horizontális és vertikális skálázás

A skálázhatóság a rendszer azon képessége, hogy növekvő terhelés mellett is elfogadható teljesítményt és rendelkezésre állást biztosítson anélkül, hogy az architektúrát alapjaiban át kellene alakítani. A kapacitás növelésére két alapvető stratégia létezik: a **vertikális skálázás** (scale up) és a **horizontális skálázás** (scale out). Ez a fejezet a két út működését, korlátait, valamint az állapotkezeléssel és a rendelkezésre állással való kapcsolatát tárgyalja.

19.1.1 Vertikális skálázás: erőforrás növelése egy példányon belül

A vertikális skálázás során a meglévő példány – virtuális gép, konténer, adatbázis-csomópont – erőforrásait növeljük: több vCPU-t, több memóriát, gyorsabb háttértárat vagy nagyobb hálózati sávszélességet adunk hozzá. A művelet jellemzően egy rövid leállással vagy újraindítással jár, hacsak az adott platform nem támogatja az élő átméretezést.

Előnyök:

- **Egyszerű üzemeltetési modell**: egyetlen példányt kell felügyelni, naplózni, biztonsági frissítésekkel ellátni.
- **Nincs szükség terheléselosztóra** a kérések szétosztásához, és a folyamatok közötti kommunikáció nem hálózati alapú, így kisebb a késleltetés a párhuzamos feldolgozásnál is, ha az alkalmazás képes kihasználni az erőforrásokat.
- **Az állapot megőrzése** természetes: ha a memóriában tárolt adat, a helyi fájlrendszer vagy a beágyazott adatbázis a példányon belül marad, nem kell külső állapotréteget bevezetni.

Korlátok:

- **Hardveres plafon**: egyetlen fizikai gép vagy virtuális gép erőforrás-kapacitása véges. A legnagyobb elérhető példányméret is korlátos, és annak költsége gyakran nem lineárisan, hanem meredeken nő.
- **Single point of failure**: ha az egyetlen példány meghibásodik, a teljes szolgáltatás leáll. A vertikális skálázás önmagában nem növeli a rendelkezésre állást, legfeljebb a nagyobb erőforrás miatt ritkábban fordulhat elő erőforrás-kimerülés.
- **Átállási kockázat**: a méretnövelés többnyire újraindítást igényel, ami tervezett leállást jelent. Tervezett karbantartás nélkül ez a rendelkezésre állási SLO-t ronthatja.
- **Költséghatékonyság**: a nagyobb példányok fajlagosan drágábbak lehetnek, és a kihasználtság nem mindig arányos a terheléssel. Egy 32 vCPU-s gép folyamatosan fizetett, még akkor is, ha a terhelés átlagosan csak 4 vCPU-t igényel.

A vertikális skálázás tipikus alkalmazási területei: relációs adatbázis elsődleges példánya, amelyet nehéz horizontálisan skálázni, régi monolitikus alkalmazások, illetve olyan számításigényes feladatok, ahol a párhuzamosítás nem egyszerű (például egyes analitikai lekérdezések). A gyakorlatban a vertikális skálázást gyakran rövid távú megoldásként alkalmazzák, amíg a horizontális skálázás feltételeit – például az állapotmentes szolgáltatásréteget – ki nem alakítják.

19.1.2 Horizontális skálázás: példányszám növelése

A horizontális skálázás során a szolgáltatás több példányát futtatjuk párhuzamosan, és a beérkező kéréseket terheléselosztó vagy szolgáltatás-mesh réteg osztja szét közöttük. Az erőforrások növelése új példány hozzáadásával történik, ami ideális esetben élő rendszerben, leállás nélkül is elvégezhető.

Előnyök:

- **Fokozatosság**: a kapacitás kis lépésekben, a tényleges terheléshez igazíthatóan növelhető vagy csökkenthető. Felhőben az automatikus skálázás percek alatt reagál a terhelésváltozásra.
- **Rendelkezésre állás növelése**: több példány esetén egy példány kiesése nem feltétlenül okoz szolgáltatáskiesést, ha a terheléselosztó a hibát észleli és a forgalmat a többi példányhoz irányítja.
- **Költséghatékonyság**: a kapacitás a terhelési görbéhez igazítható, így a kihasználtság magasabb marad. A felhőszolgáltatóknál az on-demand példányok mellett spot vagy foglalt példányok is használhatók.
- **Hibatűrés és szigetelés**: az egyes példányok elkülöníthetők hibazónák vagy régiók szerint, így a rendszer ellenállóbbá válik az infrastrukturális hibákkal szemben.

Korlátok:

- **Állapotkezelés**: a horizontálisan skálázott szolgáltatásnak többnyire állapotmentesnek kell lennie, vagy az állapotot külső, megosztott rétegben (adatbázis, cache, objektumtár) kell tárolnia. Ha a példányok helyi állapotot tartanak, a terheléselosztás inkonzisztenciához vezethet.
- **Elosztott rendszer komplexitás**: a több példány koordinációja, a hálózati kommunikáció, az óraeltolódás, a részleges hibák és a konzisztencia kezelése jelentős többletbonyolultságot visz a rendszerbe.
- **Megosztott erőforrások szűk keresztmetszete**: az adatbázis, az üzenetküldő vagy más központi komponens nem mindig skálázható horizontálisan, így a szolgáltatási réteg skálázása csak addig hatékony, amíg a háttérrendszer is lépést tart.
- **Költségek és hatékonyság**: a horizontális skálázás infrastruktúra-terheléselosztót, több példány üzemeltetését, monitorozását és esetleg service mesht igényel. A kisebb példányok fajlagos költsége alacsonyabb lehet, de a menedzsment többletköltsége ezt részben felemésztheti.

A horizontális skálázás tipikus alkalmazási területei: statikus webkiszolgálók, API-rétegek, microservice-ek, feldolgozó worker-ek, cache-rétegek és olyan adatbázisok, amelyek natívan támogatják az elosztást (például NoSQL rendszerek, shardingolt relációs klaszterek).

19.1.3 Példányszám, erőforrás és állapot

A skálázási stratégia kiválasztásakor a három kulcsfogalom – **példányszám**, **erőforrás** és **állapot** – együttesen határozza meg a lehetőségeket.

- **Példányszám**: hány független folyamat vagy gép szolgálja ki a kéréseket. Horizontális skálázásnál ezt növeljük.
- **Erőforrás**: egy példány CPU-, memória-, hálózati és tárolókapacitása. Vertikális skálázásnál ezt növeljük.

- **Állapot**: a feldolgozáshoz szükséges adat, amely lehet session-adat, felhasználói kontextus, gyorsítótár vagy üzleti adat. Az állapot elhelyezkedése döntően befolyásolja, hogy a példányszám növelhető-e.

Stateless vs. stateful szolgáltatások. A horizontális skálázás legegyszerűbb esete az állapotmentes szolgáltatás: minden beérkező kéréshez a szükséges adat a kérésben vagy egy külső adatbázisban található, és a példány nem tárol semmilyen munkamenethez kötött adatot. Ekkor bármelyik példány kiszolgálhatja a kérést, a terheléselosztó tetszőlegesen oszthatja a forgalmat, és a példányszám növelése vagy csökkentése nem veszélyezteti a konzisztenciát.

Ellenkező esetben, ha a szolgáltatás **stateful** – például a memóriában tartja a bejelentkezett felhasználók session-adatait, vagy helyi fájlba ír –, a horizontális skálázás előtt az állapotot külső rétegbe kell helyezni. Ennek bevált mintái:

- **Külső session-tároló**: Redis, memcached vagy adatbázis tárolja a session-adatokat. A példányok mindegyike ugyanahhoz a külső tárolóhoz fordul.
- **Sticky session**: a terheléselosztó egy adott felhasználó kéréseit mindig ugyanahhoz a példányhoz irányítja. Ez rövid távon megoldást jelenthet, de megnehezíti a példányok leállítását és a terheléselosztást, és a példány kiesésekor a session-adat elveszhet. Ezért csak átmeneti kompromisszumként ajánlott.
- **Shared nothing architektúra**: minden példány teljesen független, nincs közös állapot, a kérések feldolgozása kizárólag a bemenő adatokon és a külső rendszereken alapul. Ez a horizontális skálázás legtisztább alapelve.

Az állapot elosztásának további eszközei a **sharding** (adatok partíciókra osztása több csomópont között) és a **replikáció** (adatok több csomópontra másolása). Ezeket részletesen a következő fejezetek tárgyalják; itt elég annyit rögzíteni, hogy a horizontális skálázás csak akkor működik jól, ha az állapotot is elosztottan, konzisztencia-modellel és hibatűréssel együtt kezeljük.

19.1.4 A skálázás és a rendelkezésre állás kapcsolata

A rendelkezésre állás (availability) annak valószínűsége, hogy a rendszer egy adott időablakban elérhető és képes a kérések kiszolgálására. A korábbi fejezetben bevezetett képlet szerint:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

A horizontális skálázás közvetlenül növeli a rendelkezésre állást, mert a redundáns példányok miatt egy példány kiesése nem teszi elérhetetlenné a rendszert. Ha n darab független példány fut, és mindegyik rendelkezésre állása A_p, akkor a teljes szolgáltatás rendelkezésre állása – feltéve, hogy legalább egy példány működőképes – közelítőleg:

$$A_{\text{total}} = 1 - (1 - A_p)^n$$

Például ha egy példány rendelkezésre állása 99% ($A_p = 0{,}99$), két független példány esetén a rendelkezésre állás $1 - (0{,}01)^2 = 0{,}9999$, azaz 99,99%. Ez az elméleti felső határ, amely a gyakorlatban csak akkor közelíthető meg, ha a terheléselosztó, a hálózat és a háttérrendszerek is megfelelően redundánsak.

Fontos kiemelni, hogy **a horizontális skálázás önmagában nem garantálja a rendelkezésre állást**. A következő feltételeknek is teljesülniük kell:

- **Hibadetektálás**: a terheléselosztónak vagy a service meshnek észlelnie kell a meghibásodott példányt, és el kell távolítania a forgalomból. Ehhez health check (életjel-ellenőrzés) szükséges.
- **Automatikus újraindítás**: a meghibásodott példányt automatikusan újra kell indítani vagy pótolni kell, különben a redundancia fokozatosan csökken.
- **Kapacitástartalék**: a fennmaradó példányoknak el kell bírniuk a teljes terhelést, amíg az új példány el nem indul. Ha a rendszer a kapacitás határán fut, egyetlen példány kiesése túlterhelést okoz.

A vertikális skálázás ezzel szemben nem növeli a redundanciát, így a rendelkezésre állásra csak közvetett hatása van: a nagyobb erőforrás csökkentheti az erőforrás-kimerülésből fakadó hibák gyakoriságát, de a hardver- vagy szoftverhiba továbbra is teljes leállást okoz.

A rendelkezésre állás és a skálázhatóság kapcsolatát az **SLO** (service level objective) és az **error budget** is tükrözi. Ha például egy szolgáltatás havi SLO-ja 99,9% rendelkezésre állás, akkor az error budget $1 - 0{,}999 = 0{,}001$, azaz havi 43,2 perc állásidő engedélyezett. A horizontális skálázás növeli a hibatűrést, így nagyobb valószínűséggel tartható az SLO, de a skálázási műveletek – például a példányok lecserélése, a konfigurációváltozás – maguk is hibát okozhatnak, ha nem gondosak.

19.1.5 Tervezési minták és tipikus hibák

A skálázási stratégia kialakításakor az alábbi minták és hibák gyakran előfordulnak.

1. Hiba: „Csak vertikálisan skálázok mindent, mert az egyszerű."
Ez a megközelítés rövid távon működhet, de a hardveres plafon elérése után nincs további út, és a rendelkezésre állás nem javul. A helyes gyakorlat: a szolgáltatási réteget stateless módon alakítjuk ki, és horizontálisan skálázzuk, míg az adatbázis-réteget vertikálisan skálázzuk, amíg el nem éri a horizontális sharding vagy replikáció ésszerű pontját.

2. Hiba: „A session adat a memóriában van, ezért nem tudok több példányt indítani."
Ez a leggyakoribb akadálya a horizontális skálázásnak. A megoldás a session-adatok külső tárolóba helyezése, vagy a felhasználói kontextus átadása a kérésben (például JWT tokenben). A sticky session átmeneti kompromisszum, de kerülendő a hosszú távú architektúrában.

3. Hiba: „Az automatikus skálázást a CPU-kihasználtsághoz kötöttem, de a rendszer mégsem skálázódik jól."
A CPU-kihasználtság nem mindig tükrözi a felhasználói élményt. A helyes metrika lehet a kérések átlagos válaszideje, a hosszú kérések aránya, a sorban álló kérések száma vagy a feldolgozási késleltetés. A kapacitástervezésnél a Little-törvényt is alkalmazhatjuk: $L = \lambda W$, ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta, W az átlagos tartózkodási idő. Ha a válaszidő nő, a rendszerben tartózkodó kérések száma is nő, ami a példányszám növelését teheti szükségessé.

4. Hiba: „A horizontálisan skálázott szolgáltatás mögött az adatbázis vertikálisan skálázott, de nem figyelem a kapcsolatok számát."
A sok párhuzamos példány egyenként nyitott adatbázis-kapcsolatai kimeríthetik az adatbázis kapcsolati limitjét. A megoldás a kapcsolódási pool használata, a kapcsolatok multiplexelése vagy az adatbázis-réteg horizontális skálázása (például read replica).

5. Hiba: „A skálázási műveletet kézzel hajtom végre, és nincs rollback terv.”
A nem automatizált skálázás emberi hibát, késést és inkonzisztenciát okozhat. A gyakorlatban a skálázási szabályokat infrastruktúra mint kódban (IaC) kell rögzíteni, és a műveleteket verziózottan, ellenőrzötten kell végrehajtani. A visszaállítási út is legyen definiált.

6. Hiba: „A stateful adatbázist is horizontálisan skálázom, de nem kezelem az adatkonzisztenciát.”
Az adatbázis shardingolása vagy replikálása bonyolult, és a konzisztencia szintje (erős, eventual, causal) kompromisszumot igényel. A helyes gyakorlat: először a szolgáltatási réteget skálázzuk horizontálisan, az adatbázis-rétegnél pedig fokozatosan vezetjük be a replikációt, a read replica-t és csak utána a shardingot, mindig mérve a tényleges javulást.

19.1.6 Labor: Skálázási terv terhelési görbére

A fejezet lezárásaként készítsünk egy skálázási tervet egy képzeletbeli, de reálisan jellemzett webes rendszerhez. A labor célja, hogy a tanult fogalmakat egy konkrét tervezési feladaton alkalmazzuk, és a döntéseket ADR formában dokumentáljuk.

19.1.7 Kiinduló állapot

- Rendszer: „Rendelés-nyilvántartó API” – egy monolitikus, de REST API-t kiszolgáló alkalmazás, amely jelenleg egyetlen virtuális gépen fut (4 vCPU, 16 GB RAM).
- Az alkalmazás a következő rétegekből áll: API-réteg (Node.js), üzleti logika, relációs adatbázis (PostgreSQL) ugyanazon a gépen.
- A terhelési görbe napi ciklust mutat: hajnali 2–6 óra között minimális (10 kérés/perc), napközben folyamatosan növekszik, és 18–20 óra között tetőzik (300 kérés/perc), majd csökken.
- A felhasználók bejelentkeznek, és a session-adatokat a memóriában tároljuk.
- A rendelkezésre állási SLO 99,5% havi szinten.
- A költségkeret havi 500 euró (infrastruktúra).

19.1.8 Elvárt eredmény

Készítsen egy reprodukálható dokumentumot vagy jegyzetfüzetet, amely tartalmazza:

1. **Kontextus rögzítése**: a jelenlegi architektúra, a terhelési görbe, a SLO és a költségkeret leírása.
2. **Skálázási stratégia**: javaslat a vertikális és horizontális skálázás kombinációjára, indoklással. Például az API-réteg horizontális skálázása (2–3 példányra a csúcsidőben, 1 példányra éjjel), az adatbázis átköltöztetése külön gépre és vertikális skálázása (nagyobb virtuális gép), a session-adat külső Redis-be helyezése.
3. **ADR-ek**: legalább három döntés rögzítése a döntési naplóban, az alábbi formában: kontextus, döntés, alternatívák, következmények, állapot.
4. **Ellenőrzési szempontok**: milyen metrikákat figyelünk (válaszidő, hibaarány, CPU-kihasználtság, adatbázis-kapcsolatok száma), és milyen küszöbértékeknél avatkozunk be.
5. **Mért és várt eredmény elkülönítése**: a tervben jelölje, hogy mely értékek mért adatokból származnak (jelenlegi terhelés, válaszidő) és melyek becslések vagy szimulációk (a skálázott rendszer várható teljesítménye). A becsléseket egyértelműen jelölje „várt” címkével.

19.1.9 Ellenőrzési szempontok a labor értékeléséhez

- A terv figyelembe veszi-e az állapotkezelést? Ki van-e emelve, hogy a session-adatot külső tárolóba kell helyezni?
- A horizontális és vertikális skálázás kombinációja indokolt-e, vagy csak az egyikre támaszkodik?
- A döntések ADR formában vannak-e dokumentálva, és megadják-e az elvetett alternatívákat?
- A rendelkezésre állási SLO-hoz kötődik-e a terv? Például a több API-példány miatt a redundancia növekedését számszerűsíti-e?
- A költségkeretbe belefér-e a javasolt infrastruktúra? Van-e becslés az új pillanatkép költségére?
- A várt és mért eredmények egyértelműen el vannak-e különítve?

19.1.10 Visszaállítási út

A labor során éles rendszert nem módosítunk; a terv csak dokumentáció. Ha azonban a tervet később élesben alkalmazzák, a visszaállítási út a következő: a korábbi egygépes konfigurációt leíró infrastruktúra-kód verziójára való visszatérés, az adatbázis adatainak visszaállítása a mentésből, a külső session-tároló leállítása és a memóriába történő visszatérés átmeneti állapotban. A labor elvégzése nem igényel tényleges infrastruktúra-változást.

19.1.11 Ellenőrzőlista a fejezethez

- El tudom magyarázni a vertikális és horizontális skálázás közötti alapvető különbséget.
- Meg tudom mondani, miért nem növeli a vertikális skálázás önmagában a rendelkezésre állást.
- Felismerem, ha egy szolgáltatás azért nem skálázható horizontálisan, mert az állapotot helyileg tárolja.
- Ismerem a horizontális skálázás előfeltételeit: állapotmentesség, egészségellenőrzés, terheléselosztás, automatikus újraindítás.
- Tudom alkalmazni a rendelkezésre állás számítását redundáns példányok esetén.
- Meg tudom határozni, mely rétegnél milyen skálázási stratégia indokolt (API horizontális, adatbázis vertikális vagy replikált).
- A döntéseimet ADR formában tudom dokumentálni.

19.1.12 Önellenőrző kérdés

Egy rendszer API-rétege jelenleg egyetlen példányon fut, és a session-adat a memóriában van. A terhelés növekedése miatt három példányra szeretné bővíteni. Milyen lépéseket kell megtennie ahhoz, hogy a horizontális skálázás biztonságosan működjön, és milyen hatással lesz ez a rendelkezésre állásra? Válaszában térjen ki az állapotkezelésre, a terheléselosztóra, a health checkre és a rendelkezésre állás számszerű becslésére.

A következő alfejezetben (Cache, CDN és aszinkron feldolgozás) a cache, CDN és aszinkron feldolgozás szerepét vizsgáljuk a skálázhatóság további növelésében, építve az itt bevezetett fogalmakra: a horizontális skálázás alapelveire, az állapotmentesség fontosságára és a rendelkezésre állás redundanciára alapozott javítására.

19.2 Cache, CDN és aszinkron feldolgozás

19.2.1 Gyorsítótárazás, CDN és aszinkron feldolgozás

A horizontális és vertikális skálázásról szóló előző egységben azt vizsgáltuk, hogyan növelhető egy rendszer kapacitása erőforrások hozzáadásával. A skálázás azonban csak az egyik eszköz a teljesítmény és a rendelkezésre állás javítására. Ez az egység két olyan architekturális mintacsaládot mutat be, amelyek gyakran nagyobb hatást érnek el kisebb költséggel: a gyorsítótárazást és a késleltetett, aszinkron feldolgozást. A gyorsítótár a már kiszámított vagy lekérdezett eredmények újrafelhasználásával csökkenti a válaszidőt és a háttérterhelést; az aszinkron feldolgozás pedig a kritikus útról távolítja el azokat a munkákat, amelyeknek nem kell azonnal megtörténniük ahhoz, hogy a felhasználó sikeres választ kapjon.

A két minta látszólag egyszerű, a gyakorlatban azonban az egyik legnagyobb forrása a rejtett bonyolultságnak. Phil Karlton híres mondása szerint a számítástechnikában csak két nehéz probléma létezik: a cache érvénytelenítés és az elnevezés. Ez a túlzó megfogalmazás valós tapasztalatot sűrít: a gyorsítótárban tárolt adat és a forrásadatok közötti eltérés olyan hibákat okoz, amelyek nehezen reprodukálhatók, és gyakran csak akkor derülnek ki, amikor a felhasználó már elavult adatot lát vagy hibás döntést hoz. Hasonlóképpen, az aszinkron feldolgozás bevezetése megváltoztatja a rendszer hibamódjait: ami korábban azonnali hibaként jelentkezett, az most a háttérben bukhat el, és a hibakezelés, az újrapróbálkozás és a sorrendiség új tervezési feladatokat hoz.

Ebben az egységben először a gyorsítótárazás fogalmi alapjait és mintáit tekintjük át, különös tekintettel az érvénytelenítésre és a konzisztenciára. Ezután a tartalomszolgáltató hálózatokat és az edge cache-t tárgyaljuk, mint a gyorsítótárazás hálózati szintű alkalmazását. Végül az aszinkron feldolgozás és a háttérfeladatok mintáit vizsgáljuk meg, beleértve az üzenetsorokat, a backpressure fogalmát és a késleltetett munkák felügyeletét. A fejezet laborja egy kis webáruház példáján keresztül tervezteti meg a cache- és aszinkron mintákat, dokumentálva a döntéseket és elválasztva a várt eredményt a mért eredménytől.

19.2.2 A gyorsítótárazás alapfogalmai és mintái

A gyorsítótár (cache) olyan komponens, amely egy adatforrásból származó adat egy másolatát tárolja egy gyorsabban elérhető rétegben, azzal a céllal, hogy a későbbi kéréseket ennek a másolatnak a kiszolgálásával elégítse ki. A gyorsítótár lehet folyamaton belüli memória, megosztott memóriaalapú rendszer (például Redis vagy Memcached), vagy akár egy másik adatbázis réteg. A gyorsítótárazás mindig trade-off: a gyorsabb elérésért cserébe a konzisztencia, a memória és az üzemeltetési bonyolultság terén fizetünk árat.

19.2.3 Cache hit, miss és a találati arány

A gyorsítótár hatékonyságát a találati aránnyal (cache hit ratio) mérjük:

$$H = \frac{\text{találatok száma}}{\text{összes kérés száma}}$$

A találati arányt egy adott mérési ablakban kell megadni, például „az elmúlt 15 percben a termékadatokra irányuló kérések 87 %-a cache-ből került kiszolgálásra”. A találati arány önmagában nem elegendő mérőszám: egy 99 %-os találati arány is félrevezető, ha az a 1 %, amelyik nem talál, éppen a legnagyobb terhelést okozó vagy a leglassabb lekérdezéseket jelenti. Ezért a találati

arány mellett érdemes mérni a válaszidő-percentiliseket cache-találat és cache-tévesztés esetén külön-külön, valamint a háttérrendszer felé irányuló kérések számát.

A cache-tévesztés (cache miss) az az eset, amikor a kért adat nincs a gyorsítótárban. Ilyenkor a rendszer lekérdezi az adatot a forrásból, majd tipikusan eltárolja a gyorsítótárban, hogy a következő kérés már találat legyen. Ez a „lusta betöltés” (lazy loading) minta. Figyelni kell a cache-törés (cache stampede) jelenségre: ha egy népszerű adat lejár, és egyszerre sok kérés érkezik rá, akkor mindegyik téveszt, és mindegyik a háttérrendszer felé fordul, ami hirtelen terhelési csúcsot okoz. A törés kezelésére alkalmazható az úgynevezett percenkénti egy újratöltés (single-flight) minta, amely csak egy kérést enged a háttérbe, a többiek várakoznak; vagy a lejárat előtti háttér-frissítés (refresh-ahead), amely a lejárat közeledtével aszinkron módon újratölti a cache-t.

19.2.4 Cache-oldalú minták

A gyorsítótárazási mintákat két nagy csoportra oszthatjuk aszerint, hogy ki kezdeményezi a gyorsítótár frissítését.

A **cache-aside** (vagy lazy loading) mintában az alkalmazás a gyorsítótárhoz fordul először. Ha az adat nincs ott, az alkalmazás olvassa a forrásból, eltárolja a gyorsítótárban, majd visszaadja az eredményt. Az írási műveletek közvetlenül a forrásba mennek, és a cache bejegyzést érvénytelenítik. A minta előnye, hogy egyszerű, és a gyorsítótár csak azt tartalmazza, amit ténylegesen kértek. Hátránya, hogy az első kérés mindig lassú, és a törés jelensége felléphet.

A **read-through** mintában az alkalmazás ugyanúgy csak a gyorsítótárhoz fordul, de a gyorsítótár maga végzi el a forrásból való betöltést, ha az adat hiányzik. Ez elrejti a betöltés logikáját az alkalmazás elől, de a gyorsítótárnak ismernie kell a forrás elérési módját. A **write-through** minta az írási oldalt szimmetrikusan kezeli: az alkalmazás a gyorsítótárba ír, és a gyorsítótár szinkron módon továbbítja az írást a forrásba. A **write-behind** (más néven write-back) minta ezzel szemben a gyorsítótárba ír azonnali forrás-frissítés nélkül, és a forrást aszinkron, kötegelt módon frissíti. Ez nagyobb írási átbocsátást tesz lehetővé, de a gyorsítótár és a forrás közötti inkonzisztencia ablak hosszabb, és a gyorsítótár elvesztése adatvesztést okozhat.

A gyakorlatban elterjedt kombináció a read-through és a write-behind használata olyan esetekben, amikor az írási terhelés nagy, de az olvasási minta tolerálja a rövid késleltetést. A cache-aside a legegyszerűbb és legátláthatóbb, ezért gyakran ez az alapértelmezett választás olyan rendszerekben, ahol nincs dedikált cache-platform.

19.2.5 Érvénytelenítés, lejárat és konzisztencia

A gyorsítótárazott adat idővel eltérhet a forrásadattól. A két leggyakoribb megközelítés ennek kezelésére a lejárati idő (TTL – Time To Live) és az aktív érvénytelenítés.

A **TTL** egy előre meghatározott időtartam, amelynek letelte után a cache bejegyzést érvénytelennek tekintjük, és a következő kérés betölti az új értéket a forrásból. A TTL megválasztása trade-off a frissesség és a háttérterhelés között. Ha a TTL 60 másodperc, akkor a felhasználó legfeljebb 60 másodperces adatot láthat, de a háttérrendszer percenként legfeljebb egyszer kap kérést kulcsonként (feltéve, hogy a törés jelenséget kezeljük). A TTL-t a forrásadat változási gyakoriságához és az üzleti folyamat tűréséhez kell igazítani. Egy termékkatalógus esetében például az 5 perces TTL elfogadható lehet, míg egy fizetési állapot esetében a 30 másodperc is túl sok.

Az **aktív érvénytelenítés** azt jelenti, hogy amikor az alkalmazás megváltoztatja a forrásadatot, akkor a kapcsolódó cache bejegyzéseket törli vagy frissíti. Az érvénytelenítés kulcskérdése az összerendelés: honnan tudjuk, hogy melyik cache kulcs felel meg annak a forrásadatnak, amelyik megváltozott? Ha a rendszer egy terméket azonosító `product:123` kulcs alatt tárolja a cache-ben, és a termék ára megváltozik, akkor az érvénytelenítés egyszerű. Ha azonban a cache-ben összetett lekérdezések eredményeit tároljuk, például `category:5:page:2:sort:price`, akkor nehéz meghatározni, hogy egy termékváltozás melyik összetett bejegyzéseket érinti. Ezt a problémát cache-érvénytelenítési adatfolyammal lehet kezelni: a forrásrendszer változási eseményeket bocsát ki, amelyeket egy feldolgozó folyamatosan hallgat, és a cache-ben érvényteleníti az érintett kulcsokat.

A gyorsítótár és a forrás közötti konzisztencia modellezésére használhatjuk a **stale-if-error** és a **stale-while-revalidate** fogalmait. A stale-if-error azt jelenti, hogy ha a forrás elérhetetlen, a gyorsítótár tovább szolgálja ki a lejárt adatot is, ezzel növelve a rendelkezésre állást a frissesség rovására. A stale-while-revalidate azt jelenti, hogy ha a bejegyzés lejárt, de még nem töröltük, akkor a gyorsítótár a régi értéket azonnal visszaadja, miközben a háttérben újratölti az új értéket. Ez a minta csökkenti a válaszidő kiugrásait, mert a felhasználó nem vár a forrás betöltésére. Mindkét mintánál rögzíteni kell a maximális elavultsági időt, hogy a rendszer ne szolgálhasson ki önkényesen régi adatot.

19.2.6 A cache rétegei és elhelyezkedése

A gyorsítótár a rendszer több szintjén is megjelenhet. Az **alkalmazáson belüli cache** (in-process cache) a folyamat memóriájában tárol adatot. Ez a leggyorsabb elérés, de problémát jelent elosztott rendszerekben: ha több példány fut, mindegyik saját cache-t tart fenn, és a példányok közötti inkonzisztencia lehetséges. A **megosztott cache** (distributed cache) külső rendszerben tárol adatot, amelyet minden példány elér. Ez lehetővé teszi a konzisztens érvénytelenítést, de hálózati késleltetést visz a cache-elérésbe, és a megosztott cache maga is skálázási és rendelkezésre állási kérdéseket vet fel. A **többszintű cache** a kettő kombinációja: az alkalmazás először a helyi, nagyon gyors cache-t nézi, majd a megosztott cache-t, végül a forrást. Ez maximalizálja a sebességet, de a három szint közötti konzisztencia még bonyolultabb.

Az elhelyezkedés másik dimenziója az, hogy a cache a kliens és a szerver között hol helyezkedik el. A **CDN** (Content Delivery Network) a földrajzilag elosztott peremcsomópontokon (edge) tárol Tartalmat, közel a felhasználóhoz. Az **API gateway cache** a belső szolgáltatások előtt helyezkedik el, és az ismétlődő API-válaszokat tárolja. A **böngésző-cache** a felhasználó eszközén tárol erőforrásokat, és a HTTP cache-vezérlő fejlécek (Cache-Control, ETag) segítségével vezérelhető.

19.2.7 CDN és edge cache

A CDN olyan hálózati szolgáltatás, amely tartalmakat tárol világszerte elosztott szervereken (edge csomópontokon), és a felhasználó kéréseit a legközelebbi csomóponthoz irányítja. A CDN két fő előnyt nyújt: csökkenti a válaszidőt, mert az adatnak rövidebb utat kell megtennie, és tehermentesíti a forrásszervert, mert a kérések jelentős része a peremről kerül kiszolgálásra.

19.2.8 Tartalomtípusok és cache-kulcsok

A CDN elsősorban statikus, ritkán változó tartalmakra alkalmas: képek, videók, JavaScript- és CSS-fájlok, letölthető dokumentumok. A modern CDN-ek azonban képesek dinamikus

tartalmakat is gyorsítótárazni, beleértve az API-válaszokat, ha a válasz megfelelően jelölt cache-vezérlő fejlécekkel rendelkezik. A cache kulcs a CDN-ben tipikusan a teljes URL, beleértve a query paramétereket is. Ha a query paraméterek sorrendje vagy értéke változik, az új kulcsot eredményez, és a CDN nem talál. Ezért az API-tervezésnél a konzisztens URL-struktúra és a query paraméterek sorrendje hatással van a CDN találati arányára.

A cache-kulcs testreszabható: egyes CDN-ek lehetővé teszik a kulcs normalizálását, például a query paraméterek sorrendbe állítását vagy bizonyos paraméterek kihagyását. Fontos azonban, hogy a normalizálás ne vezessen adatszivárgáshoz: ha egy felhasználó-specifikus paramétert kihagyunk a kulcsból, akkor az egyik felhasználó válaszát egy másik felhasználónak is kiszolgálhatja a CDN. A felhasználó-specifikus tartalmaknál vagy a bizalmas adatoknál a `Cache-Control: private` fejlécet kell használni, amely megtiltja a megosztott cache-ben való tárolást, vagy a `no-store` fejlécet, amely a tárolást is tiltja.

19.2.9 Edge cache és edge compute

Az edge cache a CDN csomópontokon tárolt gyorsítótár. Az elmúlt években a CDN-ek túlléptek a puszta cache-szolgáltatáson, és **edge compute** képességeket kínálnak: a peremcsomóponton kód futtatható, amely a kérést a forrásszerver elérése nélkül is feldolgozhatja. Ez lehetővé teszi például a kérés átírását, a fejlécek módosítását, az A/B tesztelést, vagy akár a teljes válasz generálását a peremen.

Az edge compute architekturális jelentősége abban áll, hogy a számítási logika egy része kikerül a központi adatközpontból. Ez csökkenti a központi terhelést és a válaszidőt, de új kérdéseket vet fel: hol fut a kód, milyen runtime környezetben, hogyan verziózzuk, és hogyan teszteljük azt a kódot, amelyik a peremen fut. Az edge compute tipikusan nem helyettesíti a központi üzleti logikát, hanem a kéréskezelés azon rétegét valósítja meg, amely a felhasználótól a központi rendszerig tartó út elején szükséges.

19.2.10 Cache-vezérlő fejlécek és validáció

A HTTP-protokoll cache-vezérlő mechanizmusai a CDN és a böngésző-cache alapját képezik. A `Cache-Control` fejlécben megadott `max-age` azt mondja meg, hogy hány másodpercig tekinthető frissnek a válasz. A `s-maxage` ugyanezt a megosztott cache-re (például CDN-re) vonatkoztatja, a `stale-while-revalidate` a frissesség lejárta utáni, háttér-frissítéssel párhuzamos kiszolgálási ablakot adja meg, a `stale-if-error` pedig a forrás hiba esetén való elavult-kiszolgálást engedélyezi.

Az **ETag** fejléc a válasz egy verzióazonosítója, amelyet a szerver generál. A kliens a következő kérésben az `If-None-Match` fejléccel elküldi ezt az azonosítót, és ha az erőforrás nem változott, a szerver `304 Not Modified` választ ad üres törzs nélkül. Ez a mechanizmus akkor hasznos, ha a tartalom időnként változik, és a teljes újraküldés drága. A CDN az ETag alapján érvényesítheti a cache-t anélkül, hogy a teljes tartalmat át kellene vinnie.

A CDN inkonzisztenciájának kezelésére a **cache purge** (tisztítás) és a **cache invalidation** (érvénytelenítés) áll rendelkezésre. A purge azonnali törlést jelent a CDN-ből, de a nagy elosztott hálózaton a törlés végrehajtása időbe telik, és a purge API-hívások száma vagy sebessége korlátozott lehet. A gyakorlatban a determinisztikus verziózott URL-ek használata (például `app.abc123.js` a fájlnévben a tartalom hash-ével) lehetővé teszi, hogy a megváltozott tartalom új URL-t kapjon, így nincs szükség purge-ra; a régi URL-t egyszerűen nem hivatkozzák többé. Ez a minta hatékony a statikus erőforrásoknál, de a HTML-dokumentumoknál és a dinamikus API-válaszoknál nem mindig alkalmazható, mert azok URL-je nem változik.

19.2.11 Aszinkron feldolgozás és háttérfeladatok

A szinkron feldolgozásban a kérés feldolgozási ideje a felhasználó válaszáig tart, és a kérés teljes életciklusa a kritikus úton van. Az aszinkron feldolgozás ezzel szemben a munkát leválasztja a kérésről: a rendszer elfogadja a kérést, rögzíti a feladatot, és azonnal visszatér egy elfogadó válasszal. A tényleges munka egy későbbi időpontban, egy háttérfeldolgozóban történik. A késleltetett feldolgozás nem azonnali aszinkronitást jelent, hanem olyan feladatokat, amelyek ütemezetten, egy jövőbeli időpontban futnak le.

19.2.12 Mikor érdemes az aszinkronitás?

Az aszinkron feldolgozás bevezetését akkor érdemes megfontolni, ha a következő feltételek valamelyike fennáll. Először, a feldolgozás időigénye jelentősen meghaladja azt, amit a felhasználó hajlandó várni. Egy képfeldolgozás vagy egy jelentésgenerálás esetleg több percig tart, ami szinkron kérésben elfogadhatatlan. Másodszor, a feldolgozási erőforrás drága vagy korlátozott, és a munkát ki lehet simítani az időben. Harmadszor, a feladat jellege nem tranzakciós, azaz nem igényli, hogy a kérés és a feldolgozás egyetlen atomi műveletben történjen. Negyedszer, a feladatnak minőségi követelménye az, hogy ne vesszen el, és ezt az elvesztés elleni védelemmel meg lehet valósítani.

Fontos megérteni, hogy az aszinkronizálás nem csökkenti a teljes elvégzendő munkát; sőt, a sor, a broker és a feldolgozó üzemeltetése új erőforrásokat igényel. Amit az aszinkronizálás ad, az a kritikus út lerövidítése és a terhelés időbeli elosztása. Ha a rendszer összes munkája aszinkronná válik, a felhasználó nem kap azonnali visszajelzést a művelet sikerességéről, ami a felhasználói élményt és a hibakezelést is bonyolítja. A megfelelő arányt az üzleti folyamat határozza meg: a fizetési tranzakciónak szinkronnak kell lennie, a számla PDF előállításának nem kell.

19.2.13 Üzenetsorok, publisher-subscriber és versengő fogyasztók

Az aszinkron feldolgozás leggyakoribb megvalósítása üzenetsorokkal történik. Az üzenetküldő rendszerek alapvető topológiái a következők.

A **versengő fogyasztók** modellben több fogyasztó osztozik egy soron, és minden üzenetet pontosan egy fogyasztó dolgoz fel. Ez a minta a feldolgozás párhuzamosítására szolgál: ha egy fogyasztó másodpercenként 10 üzenetet tud feldolgozni, akkor három fogyasztó közel 30-at. A skálázás úgy történik, hogy a fogyasztók számát növeljük vagy csökkentjük a sor hossza alapján. A fogyasztók számának növelése azonban nem mindig lineáris: ha a háttér-adatbázis vagy más erőforrás szűk keresztmetszet, akkor a fogyasztók növelése a szűk keresztmetszetet terheli tovább.

A **publisher-subscriber** (pub/sub) modellben az üzeneteket témákra (topic) küldik, és minden témára feliratkozott fogyasztó megkapja az üzenet másolatát. Ez a minta alkalmas események szétosztására, például amikor egy rendelés létrejött, és erről a számlázó, a raktárkezelő és az értesítési szolgáltatás is szeretne értesülni. A pub/sub modell eseményvezérelt architektúrák alapja, és az előző részben tárgyalt eseményvezérelt mintákkal szorosan összefügg.

A **hibrid** modellekben a pub/sub és a versengő fogyasztók kombinálódnak: minden szolgáltatás feliratkozik a témára, de egy szolgáltatáson belül több példány versengve fogyasztja a szolgáltatás saját sorából az üzeneteket. Ez a mintázat jellemző a modern üzenetküldő rendszerekre, ahol a topic és a queue fogalma összeér.

19.2.14 Üzenetfeldolgozási garanciák

Az aszinkron feldolgozás tervezésekor rögzíteni kell, hogy milyen üzenetfeldolgozási garanciát nyújt a rendszer. A lehetséges szintek: **at-most-once** (legfeljebb egyszer), **at-least-once** (legalább egyszer) és **exactly-once** (pontosan egyszer).

Az at-most-once azt jelenti, hogy az üzenet elveszhet, de soha nem duplikálódik. Ez elfogadható lehet olyan esetekben, ahol a munka megismételhető vagy az elvesztésnek nincs üzleti következménye. Az at-least-once azt jelenti, hogy az üzenet nem vész el, de újra kézbesíthető, ezért a feldolgozónak idempotensnek kell lennie. Az idempotencia azt jelenti, hogy ugyanazon üzenet többszöri feldolgozása ugyanazt az eredményt adja, mint egyszeri feldolgozása. Az exactly-once a valóságban nehezen elérhető garancia; a gyakorlatban a rendszerek általában at-least-once szállítást biztosítanak, és az alkalmazás idempotens feldolgozással éri el a pontosan egyszeri üzleti eredményt.

Az idempotencia megvalósítása architekturális kérdés. A feldolgozó a feldolgozás előtt egy azonosító (például az üzenet egyedi azonosítója) alapján egy tárolóban rögzíti, hogy az üzenetet már feldolgozta. Ha az üzenet újra érkezik, a feldolgozó észleli a duplikációt, és kihagyja a munkát. Ez a tároló lehet dedikált idempotencia-tábla, vagy maga az üzleti adatmodell része, ha az üzleti kulcs természetesen egyedi.

19.2.15 Sorhossz, késleltetés és backpressure

Az aszinkron feldolgozás nem szünteti meg a kapacitáskorlátokat, csak áthelyezi őket. Ha a termelők gyorsabban küldenek üzeneteket, mint ahogy a fogyasztók fel tudják dolgozni, a sor hossza növekszik. A sor hossza (queue depth) fontos mérőszám: a növekvő sorhossz azt jelzi, hogy a fogyasztói kapacitás elégtelen, vagy egy hiba miatt a feldolgozás leállt. A sorhossz növekedése nemcsak a késleltetést növeli, hanem a memória- és lemezterhelést is, és végső soron a broker kapacitásának kimerüléséhez vezethet.

A **backpressure** az a mechanizmus, amellyel a rendszer a termelőket arról tájékoztatja, hogy lassítaniuk kell. Értelemzésünkben a backpressure két szinten értelmezhető. A belső backpressure a folyamaton belüli adatfolyamokra vonatkozik: ha egy fogyasztó lassú, a termelő nem kaphat további erőforrást. Az elosztott backpressure a rendszerek közötti jelzés: ha a sor hossza egy küszöböt meghalad, a termelő szolgáltatásnak korlátoznia kell a kérések elfogadását, vagy a kéréseket el kell utasítania ahelyett, hogy a sort tovább duzzasztaná.

A backpressure hiánya a rendszer összeomlásához vezethet: a termelők továbbra is teljes sebességgel küldenek, a sor megtelik, a broker kifogy a memóriából, és az üzenetek elvesznek vagy a broker leáll. A backpressure megvalósítása lehet egyszerű jelzés a termelő felé („a sor tele van, állj meg”), vagy összetettebb adaptív mechanizmus, amely a rendszer általános terheltsége alapján folyamatosan szabályozza a termelési rátát. Az előző fejezetben tárgyalt kapacitástervezésnél a Little-törvény ($L = \lambda W$) segít ennek a jelenségnek a számszerű megértésében: ha az érkezési ráta (λ) tartósan nagyobb, mint a feldolgozási ráta, akkor a rendszerben lévő elemek száma (L) folyamatosan növekszik, és a tartózkodási idő (W) is nő.

19.2.16 Késleltetett feldolgozás és ütemezett feladatok

A késleltetett feldolgozás olyan aszinkron feladatokat jelent, amelyeket nem azonnal, hanem egy meghatározott időpontban vagy egy késleltetési idő után kell végrehajtani. Például egy rendelés esetében a fizetési emlékeztetőt 24 órával a rendelés után kell kiküldeni, vagy egy lejáró kosarat 30 perc inaktivitás után kell törölni.

A késleltetett feladatok megvalósításának több mintája létezik. A **késleltetett sor** (delayed queue) olyan üzenetsor, amelyben az üzenet egy megadott időpontig nem látható a fogyasztók számára. Az üzenetküldő rendszerek egy része beépített késleltetési támogatást nyújt, máshol ezt az alkalmazásnak kell megvalósítania. A **ütemező** (scheduler) egy olyan komponens, amely rendszeres időközönként vagy cron-szerű kifejezések alapján indít feladatokat. Az ütemező egyszerű esetben a folyamaton belül fut, elosztott rendszerben azonban figyelni kell arra, hogy egy ütemezett feladatot ne hajtson végre több példány párhuzamosan, hacsak az idempotens.

A késleltetett feldolgozásnál a pontosság trade-off: a késleltetett sor időzítése tipikusan nem garantál másodperc-pontos végrehajtást, az ütemező pedig a rendszer terhelésétől függően késlekedhet. Ha az üzleti folyamat szigorú időzítést követel, azt külön minőségi forgatókönyvben kell rögzíteni, és a méréssel ellenőrizni kell.

19.2.17 Holtpont-sor és hibakezelés

Az aszinkron feldolgozásban a hibakezelés nem egyszerű válaszkóddal vagy kivétellel történik, hanem az üzenet életciklusának kezelésével. Amikor egy fogyasztó hibát észlel, a következő lehetőségek közül választhat: újrapróbálja a feldolgozást, átirányítja az üzenetet egy holtpont-sorba (dead letter queue, DLQ), vagy eldobja az üzenetet.

Az **újrapróbálkozás** (retry) csak akkor eredményes, ha a hiba átmeneti jellegű. Az újrapróbálkozást általában exponenciális visszalépéssel (exponential backoff) érdemes végezni: az első újrapróbálkozás 1 másodperc után, a második 2 másodperc, majd 4, 8, és így tovább, egy maximális késleltetésig. A visszalépés célja, hogy a feldolgozó ne terhelje folyamatosan a hibás függőséget, és az átmeneti hiba elmúlására időt adjon. Ha a hiba tartós, a feldolgozó egy maximális próbálkozási szám után a **holtponthoz** irányítja az üzenetet. A holtpont-sorban lévő üzeneteket egy külön folyamat vizsgálja, amely vagy emberi beavatkozást kér, vagy egy későbbi időpontban automatikusan újrafeldolgozást indít.

Fontos, hogy a hibakezelés ne váljon az üzenetek csendes elvesztésévé. A holtpont-sor hosszát és a benne lévő üzenetek korosodását monitorozni kell, és a holtpont-sorra riasztást kell kötni. Az üzenetek elvesztése elleni védelem az üzenetküldő rendszer konfigurációjának része, de az alkalmazásnak is fel kell készülnie arra, hogy az üzenet elveszhet vagy duplikálódhat.

19.2.18 Cache és aszinkron feldolgozás kombinált mintái

A gyorsítótárazás és az aszinkron feldolgozás gyakran kombinálódik. A leggyakoribb kombináció a **gyorsítótár aszinkron újratöltése**: amikor a cache bejegyzés lejár, ahelyett, hogy a felhasználó kérése szinkron módon töltené be az új értéket, a rendszer egy aszinkron feladatot indít a betöltésre, és addig a régi értéket szolgálja ki. Ez a stale-while-revalidate minta aszinkron megvalósítása. A felhasználó válaszideje így nem tartalmazza a háttérbetöltés idejét, a cache pedig a rendelkezésre állást is növeli.

Egy másik kombináció a **cache-érvénytelenítés eseményvezérelt megvalósítása**. Amikor a forrásadat megváltozik, a forrásrendszer eseményt bocsát ki, és egy aszinkron fogyasztó érvényteleníti a cache-t. Ez a minta a cache-érvénytelenítést is késleltetetté teszi, ami azt jelenti, hogy a cache és a forrás között egy rövid időablakban inkonzisztencia áll fenn. Az inkonzisztencia-ablak hosszát mérni kell, és minőségi forgatókönyvben rögzíteni, hogy az adott üzleti folyamat ezt elfogadja-e.

A harmadik kombináció az **aszinkron feldolgozás eredményének cache-elése**. Ha a feldolgozás drága vagy lassú, de az eredmény több felhasználó számára is releváns, akkor az eredményt

cache-elni lehet. Például egy termékajánló szolgáltatás naponta egyszer újraszámolja az ajánlásokat, és az eredményt cache-ben tárolja; a felhasználói kérés mindig a cache-ből olvas. Ez a minta a számítás és az olvasás közötti aszinkron kapcsolatot hoz létre.

19.2.19 Gyakori hibák és helyes megoldások

A gyorsítótárazás és az aszinkron feldolgozás bevezetésekor visszatérő hibák figyelhetők meg. Az alábbiakban ezeket a hibákat elemezzük, és a helyes megoldást is bemutatjuk.

19.2.20 1. hiba: A cache-t forrásként kezelik, nem gyorsítótárként

A leggyakoribb hiba az, hogy a fejlesztők a cache-t tartós adatforrásként kezelik, és olyan logikát építenek rá, amely feltételezi, hogy a cache-ben lévő adat mindig jó. Ennek következménye, hogy amikor a cache kiürül vagy elveszik, a rendszer hibát jelez, pedig helyes tervezés mellett ilyenkor a forrásból kellene olvasni. A cache-nek mindig elveszíthetőnek kell lennie; a rendszernek működnie kell cache nélkül is, legfeljebb lassabban. Ezt úgy lehet biztosítani, hogy a cache-elérési kód mindig a forrásból való betöltésre esik vissza, ha a cache-ben nincs adat.

19.2.21 2. hiba: Indokolatlanul nagy TTL és elavult adat

A túl hosszú TTL választása csökkenti a forrásterhelést, de elavult adatot szolgáltat. A hiba akkor súlyos, ha az üzleti folyamat frissességi követelménye erős, például a készletinformáció vagy az ár esetében. A TTL-t üzleti követelményekből kell levezetni, nem a forrásterhelés minimalizálásából. A minőségi forgatókönyvben meg kell fogalmazni, hogy a felhasználó mennyi ideig láthat elavult adatot, és a TTL-t ennek megfelelően kell beállítani.

19.2.22 3. hiba: A cache-érvénytelenítés elmarad vagy nem teljes

Az alkalmazás módosítja a forrásadatot, de elfelejti érvényteleníteni a kapcsolódó cache bejegyzéseket. A felhasználó ezután a régi adatot látja, miközben a forrás már az újat tartalmazza. Ez a hiba gyakran azért következik be, mert a cache-érvénytelenítés logikája szétszóródik az alkalmazáskódban. A megoldás az, hogy az érvénytelenítést egyetlen felelősségi pontba helyezzük: vagy a forrásrendszer eseményt bocsát ki, amelyet egy érvénytelenítő komponens hallgat, vagy az adatelérési réteg felügyeli az összes írást és olvasást, és a cache-t ott kezeljük.

19.2.23 4. hiba: Szinkron feldolgozás ott, ahol nincs szükség rá

A fejlesztők hajlamosak minden műveletet szinkron módon megvalósítani, mert a szinkron kód könnyebben érthető és tesztelhető. Ez a válaszidő romlásához és a rendszer leterheltségéhez vezet, amikor a feldolgozási idő nő. Az aszinkronitás bevezetése azonban nem minden esetben javít a helyzeten: ha a felhasználónak szüksége van azonnali visszajelzésre, akkor az aszinkron feldolgozás elrejti a hibát, és a felhasználó bizonytalanságban marad. A döntést az üzleti folyamat természete határozza meg, és az architektúradöntési naplóban kell rögzíteni.

19.2.24 5. hiba: Nincs idempotencia az at-least-once feldolgozásban

Az aszinkron feldolgozás bevezetésekor a fejlesztők gyakran úgy tervezik a feldolgozót, mintha minden üzenet pontosan egyszer érkezne. A valóságban az at-least-once kézbesítés újra küldhet üzenetet, és a feldolgozó duplikált munkát végez. Ha a feldolgozás nem idempotens, a duplikáció üzleti hibát okoz, például dupla számlázást vagy dupla készletcsökkentést. Az idempotenciát a feldolgozó tervezésének alapkövetelményeként kell kezelni, nem utólagos javításként.

19.2.25 Labor: Cache- és aszinkron minták tervezése

A labor célja, hogy egy konkrét kis rendszer példáján keresztül alkalmazd a fejezetben tanult mintákat. A labor nem igényel éles rendszert; a cél a tervezési folyamat dokumentálása, a döntések megindoklása és az ellenőrzési szempontok rögzítése.

19.2.26 Kontextus

Tervezz meg egy webáruház termékkatalógus-szolgáltatásának cache- és aszinkron mintáját. A rendszer a következő komponensekből áll:

- **Katalógus-szolgáltatás**: REST API-val teszi elérhetővé a termékeket. A termékek adatai (név, leírás, ár, készlet) egy relációs adatbázisban vannak.
- **Képfeldolgozó szolgáltatás**: A termékképek feltöltése után a képeket átméretezi és tömöríti.
- **Külső partnerrendszer**: A termékadatok egy részét egy külső szállítói rendszerből szinkronizálják, amely naponta egyszer változhat.
- **Webes kliens**: A felhasználók böngészője, amely a terméklistát és a termék részleteit kéri le.

A rendszer jelenlegi forgalma napi 100 000 termékoldal-megtekintés, csúcsidőben percenként 200 kérés. A termékek száma 50 000, a termékek átlagosan hetente egyszer változnak, de az árváltozás a katalógusban azonnal megjelenik, amikor a partnerrendszer értesítést küld.

19.2.27 Feladat

Tervezd meg a következőket:

1. **Cache-elhelyezkedés és minta**: Hol és hogyan gyorsítótáraznád a termékadatokat? Válassz cache-aside, read-through vagy más mintát, és indokold a választást.
2. **Cache-kulcs és TTL**: Milyen cache kulcsokat használnál a terméklistához és a termék-részletekhez? Milyen TTL-t választanál, és miért?
3. **Érvénytelenítés**: Hogyan kezeled az árváltozást, amikor a partnerrendszer értesítést küld? Hogyan kerülöd el a cache-törést?
4. **CDN és statikus tartalom**: Hová helyeznéd a termékképek cache-elését, és hogyan vezérelnéd a CDN-t a Cache-Control fejlécekkel?
5. **Aszinkron feldolgozás**: A képfeldolgozást aszinkron módon kell elvégezni. Tervezd meg az üzenetsort, a fogyasztók skálázását, a hibakezelést és a holtpontsort.
6. **Minőségi forgatókönyvek**: Fogalmazz meg legalább két quality attribute senariót, például a cache-találat válaszidejére és a képfeldolgozási késleltetésre.
7. **ADR**: Készíts egy Architecture Decision Recordot a cache-minta kiválasztásáról, a kontextussal, a döntéssel, az alternatívákkal és a következményekkel.

8. **Mérési terv**: Írd le, hogyan mérnéd a cache találati arányát és a képfeldolgozási késleltetést egy tesztkörnyezetben. Különítsd el a várt eredményt a mért eredménytől.

A labor eredményét egy dokumentumban kell rögzíteni, amely tartalmazza a fenti pontokat. A dokumentum legyen verziózott, és a döntésekhez hivatkozás tartozzon. A mért és a várt eredményeket külön szakaszban, egyértelmű címkékkel kell elválasztani.

19.2.28 Ellenőrzőlista

1. A cache-elérési kód mindig visszaesik a forrásra, ha a cache nem tartalmaz adatot.
2. A TTL-ek a frissességi követelményekből származnak, nem a terheléscsökkentésből.
3. A cache-érvénytelenítés minden írási útvonalon megvalósul, és egyetlen felelősségi pontban van.
4. A CDN cache-kulcsokhoz nem tartozik felhasználó-specifikus adat, vagy a válasz `private`/`no-store` jelölésű.
5. Az aszinkron feldolgozók idempotensek, és az üzenet-duplikáció nem okoz üzleti hibát.
6. A holtpont-sor monitorozott, és a benne lévő üzenetekre riasztás van kötve.
7. A backpressure megvalósult, és a termelők a sor telítettsége esetén lassítanak.
8. A minőségi forgatókönyvek tartalmazzák az ingert, a környezetet, a választ és a mérőszámot.

19.2.29 Önellenőrző kérdések

1. Mi a különbség a cache-aside, a read-through és a write-behind minták között, és milyen körülmények között választanád az egyiket?
2. Hogyan kezelnéd a cache-törés jelenségét egy népszerű termékoldal esetében, ha a TTL lejárt?
3. Miért nem elegendő a cache találati arány mérése önmagában a gyorsítótár hatékonyságának megítéléséhez?
4. Milyen következménye van annak, ha egy aszinkron feldolgozó nem idempotens, és az üzenetküldő rendszer at-least-once garanciát nyújt?
5. Hogyan biztosítanád, hogy a késleltetett feladatok egy elosztott rendszerben ne fussanak le többször párhuzamosan?

19.2.30 Összefoglalás

A gyorsítótárazás és az aszinkron feldolgozás két olyan architekturális eszköz, amelyek a kritikus út lerövidítésével és a terhelés időbeli elosztásával javítják a rendszer teljesítményét és rendelkezésre állását. Mindkét minta bevezetése azonban új bonyolultsági dimenziókat nyit: a cache-érvénytelenítés és a konzisztencia kérdéseit, valamint az üzenetfeldolgozási garanciák és a hibakezelés kérdéseit. A jó architektúra ezeket a kérdéseket nem elrejti, hanem explicit módon kezeli: minőségi forgatókönyvekben rögzíti a frissességi és késleltetési követelményeket, ADR-ekben dokumentálja a mintaválasztást, és mérésekkel ellenőrzi a várt eredményeket. A következő egységben ezekre az alapokra építve vizsgáljuk meg a terhelés, a backpressure és a kapacitástervezés összefüggéseit.

19.3 Terhelés, backpressure és kapacitástervezés

A rendszer teljesítménye ritkán lineáris. Egy adott pontig a kiszolgálás gyors, a válaszidő stabil, a felhasználó pedig elégedett. E pont után azonban hirtelen drámai romlás következik be: a válaszidő megugrik, a hibák megszaporodnak, a szolgáltatás pedig kaszkádszerűen összeomolhat. Ezt a jelenséget nevezzük a rendszer „térdének” (knee point). A kapacitástervezés és a túlterhelés elleni védekezés célja, hogy a rendszert e pont alatt tartsuk, illetve ha elérné, kontrolláltan korlátozza a bejövő forgalmat.

A szoftverarchitektúra szempontjából nem elég a forráskód funkcionális helyessége: az architektúrának azt is garantálnia kell, hogy a rendszer előre meghatározott terhelési határok között kiszámíthatóan viselkedik. Ebben az alfejezetben a terhelés és a kapacitás kapcsolatát modellezzük, megismerjük a Little-törvényt, és bevezetjük a backpressure, a rate limit és a túlterhelés elleni mintákat.

19.3.1 Rendszerviselkedés növekvő és csúcsos terhelés alatt

A terhelés méréséhez két alapvető fogalmat kell megkülönböztetnünk:

1. **Érkezési ráta (− lambda):** hány kérés érkezik a rendszerhez időegység alatt (például 100 kérés/másodperc).
2. **Átbocsátóképesség (throughput):** hány kérést képes a rendszer ténylegesen feldolgozni időegység alatt.

Amíg az érkezési ráta alacsonyabb, mint a rendszer maximális átbocsátóképessége, a rendszer viselkedése közel lineáris. A válaszidő (latencia) alacsony és stabil. Amint a bejövő forgalom megközelíti vagy meghaladja a rendszer feldolgozási kapacitását, a kérések várakozni kezdenek. A várakozási sorok megtelnek, a memória és a szálkészletek (thread pool) kimerülnek.

A túlterhelésnek két tipikus forgatókönyve van:

- **Fenntartható terhelésnövekedés:** például egy kampány bevezetése után folyamatosan nő az aktív felhasználók száma.
- **Csúcsos terhelés (burst):** rövid idő alatt hirtelen érkezik extrém forgalom, például jegyértékesítés indulásakor vagy egy hír megjelenésekor.

A csúcsos terhelés különösen veszélyes, mert az automatizált skálázás (például a Kubernetes HPA) lassabb lehet, mint a forgalom felfutása. Ha a rendszer nem készült fel a túlterhelésre, a következő hibajelenségeket figyelhetjük meg:

- **Erőforrás-telítettség:** a CPU 100%-on pörög, a memória fogy, a kapcsolati tábla betelik.
- **Szálkészlet-kimerülés:** az alkalmazásszerver nem tud új szálat indítani, az operációs rendszer szintjén a végrehajtás akadozik.
- **Retry storm (újrapróbálkozási vihar):** a kliensek időtúllépést észlelnek, ezért újrapróbálják a kérést. Ez a régi kéréseken túl új terhelést generál, ami tovább rontja a helyzetet.
- **Kaszkádhibák:** az egyik túlterhelt szolgáltatás lassul, ezért a tőle függő szolgáltatások is lassulnak, végül az egész rendszer összeomlik.

A következő, Mermaid szekvenciadiagram egy tipikus túlterhelési hurkot mutat be. Az ilyen diagramokat a „diagrams-as-code” elv alapján a kódbázisban, verziókezelve érdemes tárolni.

```
sequenceDiagram
    participant Kliens
    participant SzolgáltatásA
    participant Adatbázis

    Kliens->>SzolgáltatásA: Kérés (pl. rendelés)
    SzolgáltatásA->>Adatbázis: Lekérdezés
    Adatbázis--xSzolgáltatásA: Időtúllépés
    SzolgáltatásA--xKliens: Hiba (timeout)
    note right of Kliens: A kliens újrapróbálkozik
    Kliens->>SzolgáltatásA: Kérés újra (retry)
    SzolgáltatásA->>Adatbázis: Lekérdezés
    Adatbázis--xSzolgáltatásA: Időtúllépés
    note right of Adatbázis: A terhelés nő, a rendszer tovább lassul
```

19.3.2 A várakozási sor és a Little-törvény

A túlterhelés modellezésének legegyszerűbb és leggyakrabban használt eszköze a **sorban állási elmélet** (queueing theory), azon belül is a **Little-törvény**. A Little-törvény alapvető kapcsolatot teremt a rendszerben lévő kérések, az érkezési ráta és a tartózkodási idő között:

$$L = \lambda W$$

ahol:

- L a rendszerben lévő kérések átlagos száma (várakozási sorban és kiszolgálás alatt együtt),
- λ az érkezési ráta (kérés/másodperc),
- W a kérések átlagos tartózkodási ideje a rendszerben (másodperc).

A törvény rendkívül robusztus: független a kiszolgálási idők eloszlásától, a kapacitás elosztásától és a sorban állási fegyelemtől (FIFO, LIFO, prioritás), amennyiben a rendszer stabil. Ezért kiválóan alkalmas architekturális becslésekre.

Gyakorlati szempontból a Little-törvényt úgy használjuk, hogy meghatározzuk a párhuzamosan feldolgozandó kérések maximális számát (L), és ebből következtetünk a szükséges erőforrásokra (szálak, adatbázis-kapcsolatok, processzoregységek). Vegye figyelembe, hogy a mértékegység mindig konzisztens kell legyen (például másodperc alapú).

Példa (e-kereskedelmi flash sale): Tegyük fel, hogy egy akció során a várható érkezési ráta $\lambda = 250$ kérés/másodperc. A rendszerrel szemben támasztott SLO a válaszidőre $W < 0{,}8$ másodperc. Hány kérést kell a rendszernek párhuzamosan kezelnie?

$$L = \lambda W = 250 \ \frac{\text{kérés}}{\text{s}} \times 0{,}8 \text{ s} = 200 \text{ kérés}$$

Ez azt jelenti, hogy átlagosan 200 kérés lesz egyidejűleg a rendszerben. Ha minden kérés kiszolgálásához egy szálat és egy adatbázis-kapcsolatot rendelünk, akkor legalább 200 szálra és 200 adatbázis-kapcsolatra van szükségünk *csak* a várható átlagos terheléshez. Figyelembe kell venni továbbá a szórást és a várakozási sort, ami azt jelenti, hogy a valós kapacitást ennél magasabbra kell tervezni.

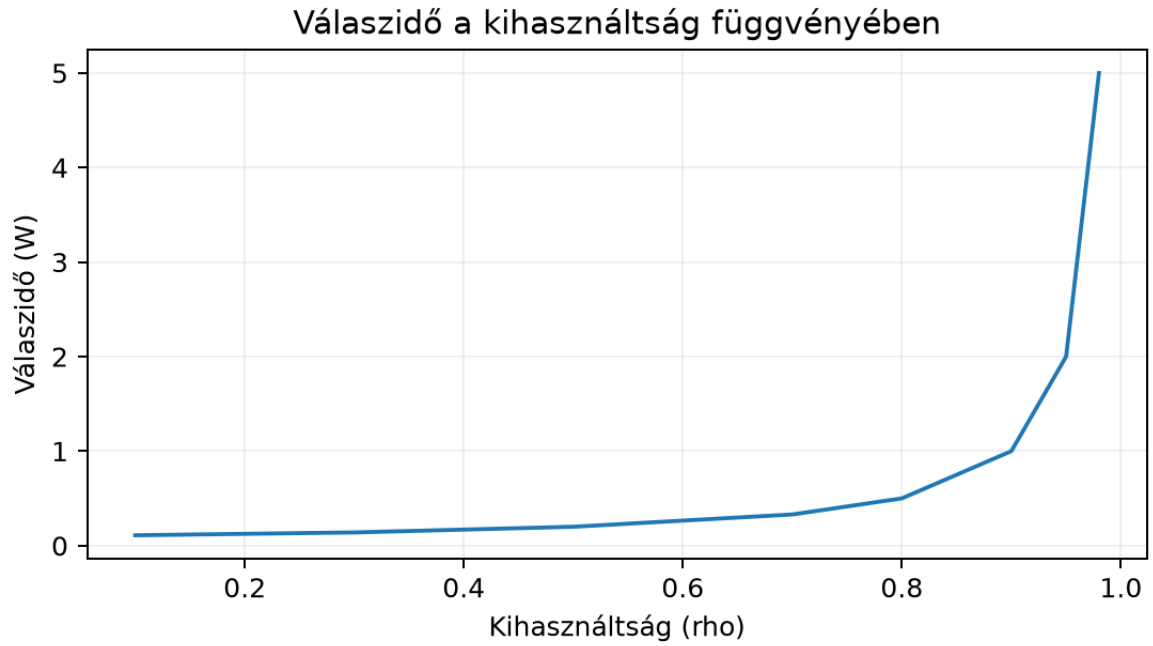
A Little-törvény a terhelés és a várakozási sor hossza közötti kapcsolatot is megmutatja. Ha λ nő, akkor L is nő, amennyiben W nem csökken. A valóságban a kiszolgálási idő (S) alsó korlát; a válaszidő (W) ezt nem közelítheti meg tetszőlegesen alulról. A válaszidő a kiszolgálási időből és a

sorban eltöltött időből áll. Amint a beérkezések miatti sorban állás megjelenik, W gyorsabban nő, mint L, és elérjük a telítettséget.

A rendszer kihasználtságát (ρ) az érkezési ráta és a kiszolgálási ráta hányadosaként definiáljuk:

$$\rho = \lambda \times S$$

Ha ρ megközelíti az 1-et, a várakozási idő gyakorlatilag a végtelenbe tart – ez a klasszikus túlterhelés. Az alábbi ábra a válaszidő és a kihasználtság kapcsolatát szemlélteti. A görbe jellegzetes: a válaszidő hosszú ideig alacsony, majd a kihasználtság növelésével meredeken emelkedik.



19.1. ábra: Válaszidő a kihasználtság függvényében - Kvalitatív modell: a görbe a 0,7–0,8 tartományban meredeken emelkedik; a pontos értékek a kiszolgálási idő eloszlásától függenek.

A kapacitástervezésnél tehát nem a 100%-os kihasználtság a cél, hanem egy olyan működési pont, ahol a várakozási sor még kontrollálható. Általános közelítésként a 0,6–0,8 közötti kihasználtság fenntartása az, amely mellett a rendszer még képes elnyelni a kisebb csúcsokat anélkül, hogy a sor hossza drámaian megnőne.

19.3.3 Túlterhelés elleni alapminták: backpressure, rate limit és load shedding

Amikor a rendszer eléri a telítettségi pontját, három alapvető stratégiát alkalmazhatunk. A stratégiák nem zárják ki egymást; egy érett rendszerben rétegenként más-más megoldás a célravezető.

19.3.3.1 1. Backpressure (ellennyomás)

A backpressure az a mechanizmus, amellyel a túlterhelt fogyasztó (consumer) visszajelzést ad a termelőnek (producer), hogy lassítson vagy álljon meg. A cél nem az, hogy a rendszer eldobja a kéréseket, hanem az, hogy a termelői oldal alkalmazkodjon a fogyasztói oldal kapacitásához.

A backpressure a reaktív rendszerek alapelve, de elosztott rendszerekben is megjelenik:

- **Hálózati szinten:** a TCP folyamatszabályozás (flow control) biztosítja, hogy a küldő ne terhelje túl a fogadót.
- **Alkalmazásszinten:** reaktív stream-feldolgozás (Reactive Streams, Kafka consumer) során a fogyasztó explicit módon jelzi, hogy mennyi elemet képes fogadni. A termelő addig nem küld többet, amíg a fogyasztó meg nem erősíti a feldolgozást.
- **Architekturális szinten:** ha egy raktárkészlet-ellenőrző szolgáltatás lelassul, a rendeléskezelő szolgáltatás sorba állíthatja a kéréseket (bounded queue), és ha a sor megtelt, a rendeléskezelő saját ügyfelei felé is lassabb választ ad, vagy explicit hibát küld. Ezzel megakadályozza, hogy a terhelés továbbterjedjen.

A backpressure megvalósításának kulcsa a **korlátos sor (bounded queue)**. Ha a sor korlátlan, a memória kimerül, és a rendszer összeomlik. Ha korlátos, akkor a túlcsordulásra explicit stratégiát kell adni: eldobás, hibaüzenet, vagy a termelő lassítása. A Mermaid diagram a sorok közötti kapcsolatot mutatja:

```
flowchart LR
    Termelő[Termelő szolgáltatás] -->|Kérések| Sor[Korlátos várakozási sor]
    Sor -->|Kérés kiszolgálása| Fogyasztó[Fogyasztó szolgáltatás]
    Fogyasztó -->|Visszajelzés: lassíts| Termelő
```

A visszajelzési hurok hiánya esetén a rendszer nem backpressure-t, hanem *spill and pray*-t (öntsd és imádkozz) valósít meg, ami hosszú távon instabilitáshoz vezet.

19.3.3.2 2. Rate limiting (sebességkorlátozás)

Míg a backpressure a belső, hasonló bizalmi szintű komponensek közötti forgalomszabályozásra szolgál, a rate limiting az API-k és a külső kliensek felé alkalmazott védekezés. A rate limiter a bejövő kérések számát korlátozza egy adott időablakban.

A rate limiting célja:

- **Erőforrás-védelem:** megakadályozza, hogy egyetlen kliens (véletlenül vagy szándékosan) monopolizálja a kapacitást.
- **Méltányos elosztás:** biztosítja, hogy a kapacitás minden fogyasztó számára arányosan elérhető legyen.
- **Költségkontroll:** megakadályozza a hirtelen költségkiugrásokat a felhős környezetben.

A rate limiting leggyakoribb algoritmusai:

- **Token bucket (tokenvödör):** a rendszer meghatározott sebességgel tokeneket ad egy vödörbe. Egy kérés csak akkor haladhat tovább, ha van token. A vödör mérete határozza meg a rövid távú burst (csúcs) méretét. Az algoritmus rugalmas, de a burst terhelésnél a vödör kiürül, és a kéréseket el kell utasítani vagy sorba kell állítani.
- **Leaky bucket (lyukas vödör):** a kérések egy sorba kerülnek, és a rendszer állandó sebességgel engedi ki őket. Ez simítja a forgalmat, de a sor megtelésekor eldobja a kéréseket.

- **Sliding window (csúszó ablak):** egy adott időablakban (például 1 perc) maximálja a kérések számát. Pontosabb, mint a fix ablakos megoldás, mert elkerüli az ablakváltáskor jelentkező dupla terhelést.

A rate limitet nem architekturális utógondolatként, hanem az API-szerződés részeként kell megtervezni. A válaszban HTTP `429 Too Many Requests` státuszkódot, a `Retry-After` fejlécben pedig a várakozási időt adjuk meg. A kliensoldali SDK-nak ezt tiszteletben kell tartania, különben a limitálás célját veszti.

19.3.3.3 3. Load shedding (terhelésleépítés)

Ha a backpressure és a rate limiting ellenére a rendszer belső állapota kritikussá válik (például a CPU kihasználtság tartósan 95% felett van, vagy a memória kifutóban), a rendszernek aktívan csökkentenie kell a feldolgozandó munkát. Ez a **load shedding** (terhelésleépítés).

A load shedding során a rendszer tudatosan eldob bizonyos kéréseket, hogy a kritikus műveletek továbbra is elérhetők maradjanak. A döntés alapja általában a prioritás:

- **Tranzakciós kérés:** „Felhasználó rendelést ad le.” – magas prioritás.
- **Analitikai kérés:** „Mutasd a múlt havi riportot.” – alacsony prioritás, eldobható vagy késleltethető.
- **Ping/health check:** „Működik a rendszer?” – magas prioritás, mert a skálázási döntésekhez és a felügyelethez szükséges.

A load shedding megvalósítása architekturális szempontból azt jelenti, hogy a szolgáltatásoknak tudniuk kell a saját egészségi állapotukat, és a hívó félnek képesnek kell lennie degradált választ adni. Például egy kereskedelmi rendszerben ha a személyre szabott ajánló szolgáltatás túlterhelt, a termékoldal betölthető az ajánlók nélkül, egy alapértelmezett katalóguslistával. Ez a **graceful degradation** (fokozatos leépülés) elve, amelynél a funkciók egy része ideiglenesen nem, vagy korlátozottan érhető el, de a magas szintű felhasználói élmény megmarad.

A load shedding és a rate limit közti különbség a döntés helye: a rate limit a belépési ponton, a klienstől függetlenül korlátoz, míg a load shedding a rendszeren belül, a belső erőforrások állapota alapján választja meg, hogy mit dolgozzon fel.

19.3.4 Kapacitástervezés: a terhelési modelltől a döntésig

A kapacitástervezés célja, hogy a rendszer a várható (és a váratlan) csúcsterhelés alatt is teljesítse a minőségi követelményeket, anélkül azonban, hogy feleslegesen nagy erőforrás-költséget generálna.

A tervezés lépései:

1. **Követelmények rögzítése:** Milyen SLO-t kell tartani a válaszidőre vagy az átbocsátóképességre? Például: a p95-ös válaszidő legyen 300 ms alatt.
2. **Terhelési forgatókönyvek definiálása:** Hány egyidejű felhasználóra számítunk? Mi a csúcsidőszak? A forgalom milyen arányban oszlik meg a műveletek között? A Little-törvényhez itt a λ és W értékeket becsüljük.
3. **Kapacitásmérés:** A jelenlegi rendszeren terheléstesztt el (lásd a laborokra vonatkozó mérési szerződést) meghatározzuk, hogy adott erőforrás (CPU, memória, kapcsolat) mellett mekkora a maximális átbocsátóképesség.
4. **Skálázási stratégia:** Horizontálisan (több példány) vagy vertikálisan (nagyobb erőforrás) skálázunk? A döntést a terhelési modell és a költségvetés együtt határozza meg.

5. **Túlterhelési stratégia:** Milyen backpressure mechanizmust, rate limitet és load sheddinget alkalmazunk, ha a terhelés meghaladja a tervezett kapacitást?
6. **Finanszírozási és fenntartási döntés:** A kapacitást mennyire tervezzük a csúcsra? A folyamatosan rendelkezésre álló, kihasználatlan erőforrás költség, a túl kevés pedig elveszett bevétel.

A kapacitástervezésnél a következő kérdéseket érdemes feltenni:

- **Mekkora a biztonsági tartalék?** Ha a tervezett csúcs $\lambda = 100$ kérés/s, a rendszerünket 110-re vagy 150-re méretezzük? A válasz a kockázattűréstől és a skálázás sebességétől függ.
- **Mi a skálázás időbeli viselkedése?** Egy új konténer elindítása 30 másodpercet vesz igénybe. Ha a forgalom 10 másodperc alatt a duplájára nő, a rendszer időlegesen túlterhelt lesz, hacsak nem tartunk fenn meleg tartalékot, vagy nem alkalmazunk előre skálázást (predictive autoscaling).
- **Mi a rendszer „gyenge pontja”?** (bottleneck) Az adatbázis? A hálózati sávszélesség? A CPU? A kapacitástervezés csak akkor hatékony, ha a szűk keresztmetszetet skálázzuk; ha a CPU-t növeljük, de az adatbázis a szűk keresztmetszet, a válaszidő nem fog javulni.

19.3.5 Labor: terhelési modell és kapacitásbecslés

A következő ellenőrzött gyakorlat célja, hogy a Little-törvényt és a túlterhelési mintákat egy kis, reprodukálható példán keresztül alkalmazza. A gyakorlat nem igényel éles rendszert; a számítások és a konfigurációk izolált környezetben vagy jegyzetfüzetben reprodukálhatók.

19.3.5.1 Cél és kontextus

Egy belső fejlesztői platform (IDP) „Just-in-Time” (JIT) hozzáférés-kezelő szolgáltatást üzemeltet. A szolgáltatás ideiglenes jogosultságot ad a fejlesztőknek egy limitált időablakban, hogy hozzáférjenek bizonyos erőforrásokhoz. A csapat a hónap végén várhatóan csúcsos terhelésre számít, mert ekkor futnak a nagyobb kiadási pipeline-ok.

Terhelési forgatókönyv:

- Átlagos érkezési ráta (λ_{avg}): 20 kérés/másodperc.
- Csúcsidőszaki érkezési ráta (λ_{csucs}): 90 kérés/másodperc.
- Átlagos tartózkodási idő (W_{avg}): 0,5 másodperc.
- A csúcsidőszak hossza: 15 perc.

Elvárt eredmény (számítás):

1. **Párhuzamosság becslése:** a csúcsidőszakban a rendszerben lévő kérések átlagos száma:

$$L_{csucs} = \lambda_{csucs} \times W_{avg} = 90 \text{ kérés/s} \times 0{,}5 \text{ s} = 45 \text{ kérés}$$

   Ez azt jelenti, hogy a rendszernek csúcsidőben átlagosan 45 párhuzamos kérést kell kezelnie.

2. **Kihasználtság ellenőrzése:** tegyük fel, hogy a szolgáltatás egyetlen példánya legfeljebb 60 párhuzamos kérést tud kezelni 0,5 másodperces átlagos kiszolgálási idővel. Ekkor a kihasználtság:

$$\rho = \frac{L_{csucs}}{\text{max kapacitás}} = \frac{45}{60} = 0{,}75$$

   Ez a 0,75 érték a biztonságos működési tartományon belül van, de a tartalék kicsi.

3. **Sor hosszának becslése:** ha a kihasználtság $\rho = 0{,}75$, az M/M/1 modell közelítésével a rendszerben lévő kérések várható száma:

$$L = \frac{\rho}{1-\rho} = \frac{0{,}75}{0{,}25} = 3$$

A sorban várakozó kérések száma pedig $L - \rho = 3 - 0{,}75 = 2{,}25$. Ez a modell azonban feltételezi a Poisson-folyamatot és az exponenciális kiszolgálási időt; a valós mérés ettől eltérhet.

19.3.5.2 Döntési napló (ADR)

A labor során a következő döntési naplót javasolt vezetni (a könyvben korábban bemutatott ADR-sablon szerint):

```
ADR-2024-10-18: Backpressure és rate limit bevezetése a JIT szolgáltatásban

Kontextus:
A JIT szolgáltatásnak nem szabad a pipeline-okból érkező kérésekkel
túlterhelni a központi felhasználókezelő rendszert. A csúcsidőszakban
a bejövő forgalom meghaladhatja a rendszer kapacitását.

Döntés:
1. A JIT szolgáltatás és a felhasználókezelő rendszer közé egy korlátos
   (legfeljebb 50 elemet tartalmazó) üzenetsort helyezünk.
2. A JIT szolgáltatás API-ján token bucket alapú rate limitet alkalmazunk:
   másodpercenként 100 token, a vödör maximális mérete 150.
3. Ha a sor megtelik, a JIT szolgáltatás HTTP 503-as hibát ad, és a
   kliensnek grace period alatt el kell halasztania a kérést.

Alternatívák:
- Korlátlan sor: elutasítva, mert memóriakiürülést okozhat.
- Nincs rate limit: elutasítva, mert egyetlen kliens monopolizálhatja a kapacitást.
- Load shedding priorizálással: későbbre halasztva, amíg a sor
  viselkedését mért adatokkal nem validáljuk.

Következmények:
- A felhasználókezelő rendszer védett a túlterheléstől.
- A kliensoldalon explicit hibaüzenet jelenik meg, ahelyett hogy a kérés
  csendben elveszne.
- Az üzemeltetőnek figyelnie kell a sor hosszát és a 503-as hibák arányát.

Állapot:
Jóváhagyva
```

19.3.5.3 Reprodukálható dokumentum: mérési jegyzőkönyv

A méréshez a következő konfigurációt rögzítjük (a könyvben korábban meghatározott mérési szerződés szerint):

- **Környezet:** izolált laborkörnyezet, Kubernetes 1.29 (helyőrző verzió, a laborkörnyezet verzióját ellenőrizni kell), a JIT szolgáltatás 1 példánya, 512Mi memória, 0,5 CPU limit.
- **Konfiguráció:** a rate limiter a fenti beállításokkal, a sor hossza 50.
- **Időpont:** a mérés időpontja és az ablak pontos ideje (pl. 2024-10-18 14:00 – 14:15 UTC).
- **Terhelésgeneráló:** `k6` (a pontos verzió és a script a kiinduló állapotban elérhető).

Várt eredmény (amit a modellből számoltunk):

- A p95-ös válaszidő 300 ms alatt marad.
- A rendszerben lévő kérések átlagos száma nem haladja meg a 45-öt.
- A sorban várakozó kérések száma nem haladja meg a 10-et.

Mért eredmény (a mérés elvégzése után kitöltendő):

Mérőszám	Várt érték	Mért érték	Eltérés	Megjegyzés
p95-ös válaszidő (ms)	< 300	*(pl. 245)*	*(pl. -55)*	
Átlagos párhuzamosság (L)	45	*(pl. 42)*	*(pl. -3)*	
Sor maximális hossza	50	*(pl. 14)*	*(pl. -36)*	
503-as hibák aránya	1%	*(pl. 0,2%)*	*(pl. -0,8%)*	

A dokumentumnak külön kell választania a várt és a mért eredményt, és rögzítenie kell a mérés bizonytalanságát (például a terhelésgeneráló által elért pontos kérésszámot, a hálózati késleltetést, a rendszer CPU-kiugrásait).

Ellenőrzési szempontok:

- A Little-törvényben használt mértékegységek konzisztensek (minden másodpercben van).
- A rate limiter és a backpressure konfigurációja megegyezik a döntési naplóban rögzített értékekkel.
- A mérés reprodukálható: ugyanazzal a konfigurációval újra lefuttatva az eltérés nem haladja meg a 10%-ot.
- A visszaállítási út: a laborkörnyezet a bemutató után a `git revert`-hez hasonló, de a konfigurációt visszaállító paranccsal törölhető. (Például: `kubectl delete -f lab/jit-overload/` – a példa demonstratív, az éles rendszerre nem vonatkozik.)

19.3.6 Összefoglalás és ellenőrzőlista

Ebben az alfejezetben a terhelés, a várakozási sor és a kapacitástervezés alapvető fogalmait tekintettük át. Láttuk, hogy a rendszer válaszideje nem lineárisan nő a terheléssel, hanem a telítettséghez közeledve drámaian megugrik. A Little-törvény ($L = \lambda W$) segítségével becsülhető a szükséges párhuzamosság és a várakozási sor hossza.

A túlterhelés elleni védekezés három fő pillére:

1. **Backpressure:** a fogyasztó lassítja a termelőt, a sorok korlátosak.
2. **Rate limiting:** a bejövő forgalmat a belépési ponton korlátozzuk.
3. **Load shedding:** a kritikus erőforrások védelme érdekében a rendszer tudatosan eldob vagy késleltet alacsony prioritású kéréseket.

A kapacitástervezés nem egyszeri feladat, hanem folyamatos tevékenység, amely a mért adatokon, a terhelési modellen és a döntési naplón alapul.

Ellenőrzőlista a terhelés és a kapacitás kezeléséhez:

- ☐ A rendszertervben a terhelési modellt (λ, W) írásban rögzítettük.
- ☐ A várakozási sorok hossza korlátos, és a túlcsordulásra explicit stratégia van.
- ☐ A backpressure mechanizmust a fogyasztó és a termelő közötti interfész egyértelműen leírja.
- ☐ A rate limiter konfigurációja (token bucket, ablak) dokumentált, a 429-es válasz és a `Retry-After` fejléc a klienssel egyeztetett.

- ☐ A load shedding prioritási sorrendje meghatározott (pl. health check > tranzakció > analitika).
- ☐ A kapacitástervezés tartalmazza a mérési bizonytalanságot és a biztonsági tartalékot.
- ☐ A döntési naplóban (ADR) rögzítettük a túlterhelési stratégia okát és a feladott alternatívákat.

Önellenőrző kérdés:

Miért nem elegendő a 99%-os rendelkezésre állású adatbázis-szolgáltatás egy csúcsos terhelésű rendszerben? Milyen mérőszámot kell figyelni ahhoz, hogy a túlterhelést a tényleges összeomlás előtt észleljük? (Útmutató: gondoljon a kihasználtság és a válaszidő görbéjére; a 99%-os rendelkezésre állás nem jelenti azt, hogy a p95-ös válaszidő alacsony marad. A sor hosszát és a telítettséget kell monitorozni, hogy idejekorán beavatkozhassunk.)

A következő alfejezetben ezeket a fogalmakat a FinOps, az energiafelhasználás és a zöld szoftver szempontjaival kapcsoljuk össze, megvizsgálva, hogyan optimalizálható a kapacitás és a költség egyensúlya.

19.4 FinOps, energia és zöld szoftver

A költség és a környezeti hatás nem csupán pénzügyi vagy fenntarthatósági kérdés; éppúgy architekturális minőség, mint a rendelkezésre állás vagy a karbantarthatóság. Az architekt döntései – például az, hogy egy szolgáltatást szinkron HTTP-hívással vagy aszinkron üzenetsorral kapcsol össze, hogy a feldolgozást állandóan futó konténerben vagy eseményre induló serverless függvényben végzi, vagy hogy egy adatbázist replikál-e több régióba – közvetlenül befolyásolják az egységköltséget, az erőforrás-kihasználtságot, az energiafelhasználást és ezen keresztül a szénlábnyomot. A FinOps (cloud financial management) és a zöld szoftver tervezés (green software engineering) ezért nem üzemeltetési melléktermék, hanem a tervezési folyamat része.

Ebben az alfejezetben először definiáljuk a költséget és a környezeti hatást mint minőségi attribútumot, majd bemutatjuk az egységköltség, a kihasználtság és a hulladék fogalmát. Ezt követően az energiahatékonyság és a zöld tervezési döntések gyakorlati mintáit tárgyaljuk, végül egy reprodukálható labor keretében költség- és szénlábnyom-becslést készítünk. A fejezet a Terhelés, backpressure és kapacitástervezés alfejezetre (terhelés, backpressure és kapacitástervezés) épít, és előkészíti a Hibamódok és failure mode analysis alfejezetet (hibamódok és failure mode analysis), ahol a költséghatékony reziliencia-minták is szerepelnek.

19.4.1 1. Költség és környezeti hatás mint architekturális minőség

A minőségi attribútumok általában olyan tulajdonságok, amelyek nem egyetlen funkcióhoz kötődnek, hanem a rendszer egészét jellemzik, és mérhető forgatókönyvekkel értékelhetők. A költség és a környezeti hatás esetében ez azt jelenti, hogy a rendszerrel szemben megfogalmazhatunk olyan minőségi forgatókönyveket, amelyek ingere, környezete, válasza és mérőszáma egyaránt rögzíthető.

Quality attribute scenario – költség

- **Inger:** 10 000 felhasználó egyidejűleg hajt végre termékkeresést egy e-kereskedelmi rendszerben.
- **Környezet:** a rendszer normál üzemmódban fut, egy régióban, három availability zónában, konténer-alapú mikroszolgáltatásokkal.

- **Válasz:** a rendszer a keresési kéréseket kiszolgálja, a cache találati arány legalább 85%, a nem cache-elt kérések az adatbázishoz fordulnak.
- **Mérőszám:** az egy keresési tranzakcióra jutó teljes infrastruktúra-költség legfeljebb 0,003 EUR, 30 napos csúszó ablakban mérve.

Quality attribute scenario – környezeti hatás

- **Inger:** 1 000 000 batch üzenet feldolgozása egy éjszakai adatfeldolgozási ablakban.
- **Környezet:** a feldolgozás felhőalapú Kubernetes klaszterben történik, a régió átlagos hálózati karbonintenzitása 250 g CO /kWh.
- **Válasz:** a feldolgozás befejeződik, az erőforrások a feldolgozás után lefelé skálázódnak.
- **Mérőszám:** a feldolgozás teljes energiafogyasztása legfeljebb 12 kWh, a becsült szénlábnyom legfeljebb 3,0 kg CO .

Ezek a forgatókönyvek azt mutatják, hogy a költség és a környezeti hatás ugyanúgy specifikálható, mérhető és tesztelhető, mint a teljesítmény. A különbség annyi, hogy a mérés gyakran becslésen vagy modellen alapul, és a bizonytalanságot explicit módon kell kezelni.

A FinOps Foundation a felhőpénzügyi menedzsmentet három fázisra bontja: **inform** (láthatóság és allokáció), **optimize** (kihasználtság javítása és hulladékcsökkentés), **operate** (folyamatos irányítás és szervezeti beágyazás). A zöld szoftver alapelvei közé tartozik a karbonhatékonyság (carbon efficiency), az energiahatékonyság, a hardverhatékonyság és a keresletcsökkentés. Mindkét keretrendszer közös nevezője: a nem hatékony erőforrás-felhasználás egyszerre pénzügyi és környezeti veszteség.

Architektúrális szempontból a költség és a környezeti hatás nem független a többi minőségi attribútumtól. Például a magas rendelkezésre állás több replikát, több régiót és ezáltal több energiát igényel; a késleltetés csökkentése érdekében bevezetett cache réteg növeli a memóriaigényt. Az architekt feladata éppen az, hogy ezeket a trade-offokat explicit módon dokumentálja, és a döntéseket az üzleti érték és a fenntarthatósági célok függvényében hozza meg.

19.4.2 2. Egységköltség, kihasználtság és hulladék

19.4.3 2.1 Egységköltség

Az egységköltség (unit cost) azt mutatja meg, mennyibe kerül egy jól definiált üzleti vagy technikai tranzakció, esemény vagy felhasználói művelet. Az egységköltség számítása:

$$C_{\text{unit}} = \frac{C_{\text{total}}}{N}$$

ahol C_{total} a rendszer teljes költsége egy adott időablakban (például havi számla), N pedig az adott időablakban kiszolgált tranzakciók száma. Az egységköltség akkor hasznos, ha a nevező jól definiált, és a költséget hozzá tudjuk rendelni az adott tranzakciótípushoz. Például egy videóstreaming-szolgáltatás esetén az egység lehet „lejátszott óra”, egy fizetési szolgáltatásnál „sikeres tranzakció”, egy batch rendszernél „feldolgozott rekord”.

Az egységköltség két fő összetevőre bontható: a **fix** költségekre (amelyek nem függenek a terheléstől, például egy állandóan futó adatbázis minimális példányszáma) és a **változó** költségekre (amelyek a terheléssel arányosan nőnek, például a skálázódó számítási kapacitás). Architekturális döntés, hogy a rendszer hol helyezkedik el ezen a spektrumon: a monolitikus alkalmazás állandóan futó virtuális gépeken magas fix költséggel bír, míg a serverless architektúra gyakran csak a tényleges hívások után számláz, viszont magasabb egységköltséggel járhat nagy forgalomnál.

Az egységköltség mérésénél fontos a **költségallokáció** (cost allocation): a felhőszolgáltatók címkézési mechanizmusai (tagging) lehetővé teszik, hogy az erőforrásokat költséghelyekhez, csapatokhoz vagy termékekhez rendeljük. Címkék nélkül a teljes számla egyetlen nagy összeg, amelyből nem lehet architekturális következtetéseket levonni.

19.4.4 2.2 Kihasználtság

A kihasználtság (utilization) azt mutatja meg, hogy a rendelkezésre álló erőforrások mekkora hányadát használja a rendszer. A kihasználtságot több szinten értelmezhetjük:

- **CPU-kihasználtság:** a processzoridő aránya egy adott időablakban.
- **Memóriakihasználtság:** a lefoglalt memória aránya a rendelkezésre állóhoz képest.
- **Hálózati kihasználtság:** az átvitt adatmennyiség a sávszélességhez képest.
- **Pénzügyi kihasználtság:** a ténylegesen felhasznált erőforrások költsége a kifizetett kapacitáshoz képest.

Az alacsony kihasználtság klasszikus hulladékforrás. Ha egy három példányból álló Kubernetes Deployment minden podja átlagosan 15% CPU-t használ, miközben a klaszter autoscaling beállítása 60%-os célkihasználtságot tart fenn, akkor a számítási kapacitás jelentős része kihasználatlan, de a számla akkor is fizetendő. A kihasználtság javítása gyakran olcsóbb és gyorsabb megtakarítást hoz, mint az architektúra teljes átalakítása.

Ugyanakkor a 100%-os kihasználtság nem cél: a teljesen kihasznált erőforrás nem hagy tartalékot a forgalmi csúcsokra, növeli a késleltetést és a hibaarányt. A megfelelő célkihasználtság a minőségi követelményektől és a terhelés változékonyságától függ. A kapacitástervezésnél megismert Little-törvény ($L = \lambda W$) itt is segít: ha a kiszolgálási idő W nő a magas kihasználtság miatt, akkor azonos érkezési ráta mellett a rendszerben tartózkodó kérések száma is nő, ami hosszabb várakozási sorokhoz vezet.

19.4.5 2.3 Hulladék

A hulladék (waste) olyan erőforrás-felhasználás, amely nem járul hozzá az üzleti értékhez. A felhőkörnyezetben a hulladék tipikus formái:

- **Túlméretezett erőforrások:** a számítási kapacitást a csúcsigényre méretezik, de az átlagos terhelés ennek töredéke.
- **Árva erőforrások:** olyan erőforrások, amelyeket a rendszer már nem használ, de nem szüntettek meg (például elfelejtett tesztkörnyezet, ideiglenes adatbázis).
- **Nem megfelelő vásárlási modell:** on-demand árazás ott, ahol tartós foglalás (reserved instance, savings plan) olcsóbb lenne, vagy fordítva.
- **Adatátviteli hulladék:** szükségtelen adatmozgatás régiók között, nem tömörített adatforgalom, túlzott naplózás.
- **Üresjárati kapacitás:** állandóan futó példányok, amelyek éjszaka vagy hétvégén sem skálázódnak le, pedig a terhelés minimális.

A hulladék azonosítása mérést igényel. A FinOps „inform” fázisában a költség- és kihasználtsági adatokat láthatóvá kell tenni, és azokat a csapatokkal közösen kell értelmezni. A hulladék csökkentése nem egyszeri projekt, hanem folyamatos visszacsatolási hurok: a rendszer minden változása, az új funkciók, az új ügyfelek vagy a forgalmi minták megváltozása új hulladékot hozhat létre.

19.4.6 3. Energiahatékonyság és zöld tervezési döntések

A zöld szoftver tervezés célja, hogy a rendszer ugyanazt az üzleti értéket kisebb környezeti hatással állítsa elő. A környezeti hatás leggyakrabban a szénlábnyommal (carbon footprint) mérhető, amely az energiafogyasztás és a villamosenergia-termelés karbonintenzitásának szorzata:

$$\mathrm{CO_2e} = E \times CI$$

ahol E az energiafogyasztás kilowattórában (kWh), CI pedig a karbonintenzitás gramm CO -egyenérték per kWh (g CO e/kWh). A karbonintenzitás időben és földrajzilag változik: egy adott régió energiamixe lehet szén- vagy gázalapú, illetve megújuló forrásokra támaszkodó. Fontos megjegyezni, hogy a felhőszolgáltatók által közölt karbonintenzitási adatok is becslések, és a mérési bizonytalanságot mindig dokumentálni kell.

A zöld tervezési döntések négy fő kategóriába sorolhatók:

19.4.7 3.1 Energiahatékonyság (energy efficiency)

Az energiahatékonyság azt jelenti, hogy ugyanannyi munka elvégzéséhez kevesebb energiát használunk. Gyakorlati minták:

- **Nyelv és futtatókörnyezet megválasztása:** bizonyos nyelvek és keretrendszerek energiaigénye jelentősen eltérhet ugyanazon feladat esetén. Például egy nagy adathalmazon végzett batch feldolgozásnál a natív kódú vagy alacsony szintű nyelvek gyakran energiahatékonyabbak, mint a magas szintű, interpretált nyelvek – de ez a karbantarthatósági követelményekkel ütközhet.
- **Számítási erőforrások kihasználása:** a magasabb kihasználtság nemcsak pénzügyileg előnyös, hanem környezeti szempontból is, mert kevesebb fizikai gép üzemel.
- **Hardverhatékonyság:** olyan processzortípusok használata, amelyek jobb teljesítmény/watt arányt kínálnak. A felhőszolgáltatók egyre több ARM-alapú vagy speciális célú processzort kínálnak; ezek energiahatékonysága gyakran jobb, de a kompatibilitást ellenőrizni kell.
- **Adattömörítés és hatékony adatszerializáció:** a kisebb adatcsomagok kevesebb hálózati energiát igényelnek.

19.4.8 3.2 Karbonhatékonyság (carbon efficiency)

A karbonhatékonyság azt jelenti, hogy a feldolgozást időben és térben oda helyezzük, ahol a villamosenergia karbonintenzitása alacsonyabb. Gyakorlati minták:

- **Időbeli eltolás:** a batch feldolgozásokat olyan időszakra ütemezzük, amikor a hálózat karbonintenzitása alacsony (például éjszaka, amikor több a szélenergia). Ehhez a felhőszolgáltatók karbonintenzitási API-jai vagy harmadik fél szolgáltatások használhatók.
- **Térbeli eltolás:** a munkaterhelést olyan régióba helyezzük, ahol az energiamix tisztább. Ez gyakran ütközik az adatvédelemmel, a késleltetési követelményekkel és a jogi megfeleléssel, ezért a döntést dokumentálni kell.
- **Igény szerinti futtatás:** a karbonintenzitás figyelésével a rendszer akkor indítja el a batch munkát, amikor a jel alacsony; ez a minta jól illeszkedik a serverless vagy a Kubernetes alapú batch feldolgozáshoz.

19.4.9 3.3 Hardverhatékonyság (hardware efficiency)

A hardverhatékonyság az erőforrások élettartamának és kihasználtságának maximalizálása. Gyakorlati minták:

- **Rugalmas skálázás:** a horizontális autoscaling lehetővé teszi, hogy a példányszám a terheléshez igazodjon, így csökkentve az üresjáratot.
- **Tartós erőforrások megosztása:** multitenant platformok, közös Kubernetes klaszterek, megosztott adatbázis-példányok – ezek növelik a fizikai hardver kihasználtságát.
- **Élettartam-hosszabbítás:** a szoftver frissítése helyett a hardver cseréje gyakran nagyobb környezeti hatással jár; a felhőszolgáltatók esetében ez a hatás a szolgáltató oldalán jelentkezik, de a fogyasztói magatartás (például a folyamatos új verziók) közvetve befolyásolja.

19.4.10 3.4 Keresletcsökkentés (demand shaping)

A keresletcsökkentés a felhasználó vagy az üzleti folyamat igényeinek olyan befolyásolása, amely csökkenti az erőforrásigényt. Példák:

- **Cache és CDN használata:** a gyakran ismételt kérések kiszolgálása közelebb a felhasználóhoz csökkenti a hálózati forgalmat és a háttérrendszer terhelését.
- **Aszinkron feldolgozás:** a szinkron válaszadási követelmény feloldásával a feldolgozás ütemezhető, összevonható, és a csúcsok simíthatók.
- **Kliensoldali számítás:** bizonyos feladatok áthelyezése a kliensre csökkenti a szerveroldali energiaigényt, de növelheti a kliens eszközök fogyasztását – a trade-offot teljes életciklusban kell értékelni.
- **Funkciók racionalizálása:** a ritkán használt, nagy erőforrásigényű funkciók eltávolítása vagy opcionálissá tétele.

19.4.11 4. FinOps: gyakorlati minta a költségmenedzsmentre

A FinOps keretrendszer (FinOps Foundation, 2023-as verzió, hivatkozás: finops.org/framework) a felhőköltségek kezelését a pénzügy, a technológia és az üzlet együttműködésére alapozza. A keretrendszer három fázisa – inform, optimize, operate – jól illeszkedik az architekturális tervezési ciklushoz.

19.4.12 4.1 Inform: láthatóság és allokáció

Az első lépés a költségadatok gyűjtése és rendszerezése. Ehhez a felhőszolgáltatók részletes számlázási API-kat biztosítanak, amelyekből óránkénti, erőforrás-szintű költségadatok nyerhetők. Az adatokat címkékkel és címke-alapú házirendekkel kell ellátni, hogy a költségek termékhez, csapathoz vagy környezethez legyenek rendelhetők. A láthatóság célja nem a hibáztatás, hanem a közös értelmezés: a mérnököknek látniuk kell, hogy egy adott architekturális döntés milyen költségvonzattal jár.

Példa: egy Kubernetes alapú rendszerben a költségek pod-szintű bonthatóságához olyan eszközökre van szükség, mint a kubecost vagy a felhőszolgáltató natív költségallokációs szolgáltatásai. A podokhoz rendelt erőforrás-igények (requests) és limitek (limits) alapján a költség hozzárendelhető a workloads-hoz.

19.4.13 4.2 Optimize: kihasználtság javítása és hulladékcsökkentés

Az optimalizálás során a mért adatok alapján konkrét intézkedéseket hozunk. Az intézkedések lehetnek azonnaliak (például a túlméretezett példányok lecserélése kisebbre) vagy architekturális jellegűek (például a szinkron híváslánc átalakítása eseményvezérelt folyamattá). A FinOps keretrendszer hangsúlyozza, hogy az optimalizálás nem egyszeri esemény, hanem folyamatos visszacsatolási hurok.

Az optimalizálás tipikus formái:

- **Méretezés (rightsizing):** a példánytípusok és -méretek folyamatos felülvizsgálata a tényleges kihasználtság alapján.
- **Vásárlási modell optimalizálása:** tartós foglalások vagy savings plan alkalmazása a stabil terhelésű erőforrásokra, míg a változó terhelést on-demand vagy spot kapacitással szolgáljuk ki.
- **Autoscaling finomhangolása:** a skálázási küszöbök és a hűtési időszakok beállítása a forgalmi mintákhoz.
- **Adatátvitel optimalizálása:** a régiók közötti forgalom csökkentése, a tartalomgyorsító (CDN) használata, az adattömörítés.

19.4.14 4.3 Operate: folyamatos irányítás és szervezeti beágyazás

Az operate fázisban a költségmenedzsmentet beépítjük a mindennapi munkafolyamatokba. Ennek eszközei:

- **Költségvetések és riasztások:** előre meghatározott küszöbértékek átlépésekor automatikus értesítés.
- **Költségdashboardok:** a csapatok saját költségeiket valós időben láthatják.
- **Költség-review folyamat:** a rendszeres architektúra-review részeként a költségtrendeket is értékeljük.
- **FinOps szakértő vagy csapat:** a szervezeti felelősség egyértelmű kijelölése.

A FinOps és a zöld szoftver szorosan összefügg: a hulladék csökkentése egyszerre csökkenti a költséget és a szénlábnyomot. A karbonintenzitás-alapú ütemezés azonban nem mindig költségcsökkentő – például egy alacsony karbonintenzitású régió drágább lehet –, ezért a döntéseknél mindkét szempontot explicit módon kell kezelni.

19.4.15 5. Költség- és szénlábnyom-becslés gyakorlati modellje

A költség- és szénlábnyom-becslés célja, hogy az architektúra alternatíváit összehasonlíthassuk anélkül, hogy mindegyiket éles környezetben megvalósítanánk. A becslés modellje négy lépésből áll:

1. **Erőforrás-katalógus összeállítása:** milyen számítási, tárolási, hálózati és szolgáltatási erőforrásokra van szükség, és milyen méretben, példányszámban, futási időben?
2. **Energiafogyasztás becslése:** az erőforrások energiaigényét a gyártói adatok, a felhőszolgáltató dokumentációja vagy mérési adatok alapján becsüljük. Ehhez a PUE (Power Usage Effectiveness) értéket is figyelembe vehetjük:

$$E_{\text{total}} = E_{\text{IT}} \times \text{PUE}$$

ahol E_{IT} a számítástechnikai berendezések energiafogyasztása, a PUE pedig a létesítmény teljes energiafogyasztásának és az IT-energiafogyasztásnak a hányadosa. A felhőszolgáltatók PUE-értéke általában 1,1 és 1,3 között van, de ez régiónként eltérő.

3. **Költség becslése:** az erőforrásokhoz a felhőszolgáltató aktuális árazását rendeljük. Az árak régiónként, vásárlási modellenként és időben változnak; a becslésnél mindig a dátumot és a forrást kell rögzíteni.
4. **Szénlábnyom becslése:** az energiafogyasztást megszorozzuk a régió átlagos karbonintenzitásával. A karbonintenzitás forrása lehet a felhőszolgáltató által közzétett adat vagy harmadik fél API (például Electricity Maps). A bizonytalanságot mindig jelezni kell.

A becslés nem mérés: a modell feltételezései hibásak lehetnek, és az eredmény csak annyira pontos, mint a bemenő adatok. Ezért a becslésnél kötelező a **bizonytalanság** feltüntetése, például érzékenységvizsgálattal (mi történik, ha a kihasználtság 10%-kal magasabb vagy alacsonyabb?).

19.4.16 6. Tipikus hibák és ellenőrzőlista

A költség- és környezeti hatás mint architekturális minőség kezelése során a következő hibák gyakoriak:

- **A költség csak az üzemeltetés problémája:** ha az architekt nem vesz részt a költségcélok meghatározásában, az optimalizálás utólagos tűzoltássá válik.
- **A kihasználtság kizárólagos maximalizálása:** a 100%-os kihasználtság rontja a késleltetést és a rendelkezésre állást; a cél a megfelelő egyensúly.
- **A hulladék egyszeri megszüntetése:** a hulladék folyamatosan újratermelődik, ezért a folyamatot automatizálni kell.
- **A karbonintenzitás figyelmen kívül hagyása:** a költséghatékony döntés nem mindig karbonhatékony; a két szempontot együtt kell értékelni.
- **A becslés mérésként való bemutatása:** a becslés modell, amelynek bizonytalansága van; ezt mindig fel kell tüntetni.
- **A trade-offok nem dokumentálása:** a költség- és energiahatékonysági döntések gyakran ütköznek más minőségi attribútumokkal; az ADR-ekben rögzíteni kell a kontextust és a feladott alternatívákat.

Ellenőrzőlista a fejezet végén:

- ☐ A költség és a környezeti hatás quality attribute scenario formában specifikálva van?
- ☐ Az egységköltség jól definiált tranzakcióra vonatkozik?
- ☐ A kihasználtság célértéke a minőségi követelményekkel összhangban van?
- ☐ A hulladék azonosításához rendelkezésre állnak-e mérési adatok?
- ☐ A zöld tervezési döntéseknél a karbonintenzitás forrása és dátuma rögzítve van?
- ☐ A költség- és szénlábnyom-becslés bizonytalansága dokumentálva van?
- ☐ A döntések ADR-ben rögzítve vannak, beleértve a feladott alternatívákat?
- ☐ A laborban a mért és a várt eredmények elkülönítve szerepelnek?

19.4.17 7. Labor: költség- és szénlábnyom-becslés készítése

A labor célja, hogy egy kis, ellenőrizhető példán keresztül alkalmazzuk a fejezet eszközeit. A labor átadandó eredménye egy reprodukálható dokumentum vagy jegyzetfüzet, amely tartalmazza a kontextust, a becslési modellt, a számításokat, a döntési naplót és az ellenőrzési szempontokat.

19.4.18 7.1 Kiinduló állapot

Adott egy egyszerű webes alkalmazás, amely egy REST API-t szolgál ki, és egy relációs adatbázist használ. Az alkalmazás jelenleg egyetlen virtuális gépen fut (méret: 2 vCPU, 8 GB RAM), az adatbázis ugyanazon a gépen, lokális lemezen. A terhelés: naponta körülbelül 50 000 API-hívás, a forgalom napközben egyenletes, éjszaka minimális. A rendszer rendelkezésre állási követelménye 99,5%, a válaszidő átlagosan 200 ms.

A labor célja: becsüljük meg a jelenlegi architektúra havi költségét és szénlábnyomát, majd hasonlítsuk össze egy felhőalapú, konténerizált, autoscalinggal rendelkező változattal.

19.4.19 7.2 Környezet és konfiguráció

A becsléshez a következő paramétereket rögzítjük:

- **Felhőszolgáltató:** példaként egy képzeletbeli szolgáltató, „ExampleCloud” (helyőrző, nem valós).
- **Régió:** eu-central-1 (helyettesítő név).
- **Számítási egység:** virtuális gép típusa: VM.Standard.2.8 (2 vCPU, 8 GB RAM); konténer esetén Kubernetes node pool azonos specifikációval.
- **Adatbázis:** felügyelt relációs adatbázis, db.t3.medium (2 vCPU, 4 GB RAM).
- **Tárolás:** 100 GB SSD, havi 10 GB snapshot.
- **Hálózati forgalom:** a kimenő adatforgalom havi 500 GB, a bejövő adatforgalom ingyenes.
- **Árazás (2024. január, példaértékek, ellenőrizendő):** a virtuális gép óránként 0,12 EUR; a felügyelt adatbázis óránként 0,15 EUR; a tárolás GB-onként havonta 0,10 EUR; a kimenő forgalom GB-onként 0,09 EUR.
- **PUE:** a régióra jellemző PUE 1,2 (becslés).
- **Karbonintenzitás:** a régió átlagos karbonintenzitása 2024 januárjában 300 g CO e/kWh (forrás: helyőrző API, ellenőrizendő).

A labor környezete izolált: a becslést jegyzetfüzetben végezzük, a számításokat Pythonban vagy táblázatkezelőben reprodukálható módon rögzítjük. A mért értékeket (például a tényleges CPU-kihasználtság mérése) nem végezzük el, ezért minden számítás becslés, amelyet a dokumentumban „becslés” címkével látunk el.

19.4.20 7.3 Elvárt eredmény

A labor végére a hallgató:

1. Összeállítja a jelenlegi és a javasolt architektúra erőforrás-katalógusát.
2. Kiszámítja a havi költséget mindkét változatra.
3. Becsli az energiafogyasztást és a szénlábnyomot.
4. Az eredményeket táblázatban foglalja össze, feltüntetve a bizonytalanságot.
5. Döntési naplót vezet, amelyben rögzíti a választott alternatívát és a feladott opciókat.
6. Ellenőrzési szempontokat ad meg: hogyan lehetne a becslést méréssel validálni?

19.4.21 7.4 Számítási példa

A jelenlegi architektúra havi költsége:

- Virtuális gép: 730 óra × 0,12 EUR/óra = 87,60 EUR
- Adatbázis (a virtuális gépen belül, nem felügyelt): 0 EUR, mivel a VM árában benne van.
- Tárolás: 100 GB × 0,10 EUR/GB = 10,00 EUR
- Kimenő forgalom: 500 GB × 0,09 EUR/GB = 45,00 EUR
- **Teljes havi költség: 142,60 EUR**

A javasolt felhőalapú változat:

- Kubernetes node (egy node): 730 óra × 0,12 EUR/óra = 87,60 EUR
- Felügyelt adatbázis: 730 óra × 0,15 EUR/óra = 109,50 EUR
- Tárolás: 100 GB × 0,10 EUR/GB = 10,00 EUR
- Kimenő forgalom: 500 GB × 0,09 EUR/GB = 45,00 EUR
- **Teljes havi költség: 252,10 EUR**

A javasolt architektúra drágábbnak tűnik, de figyelembe kell venni, hogy a jelenlegi egypéldányos architektúra nem teljesíti a 99,5%-os rendelkezésre állást, mert nincs redundancia. Ha a jelenlegi architektúrát redundánssá tesszük (két példány, terheléselosztó), a költsége emelkedik. Továbbá az autoscaling lehetővé teszi, hogy éjszaka a példányszámot csökkentsük, ami a konténeres változat költségét alacsonyabbra hozhatja. A becslésnek ezt a dinamikát is modelleznie kell: például éjszakai leállással a node kihasználtsága csökken, de a felügyelt adatbázis továbbra is fut.

Az energiafogyasztás becslése: a virtuális gép 2 vCPU-s és 8 GB RAM-os konfigurációjának átlagos energiafelvétele körülbelül 150 W (becslés, a gyártói adatok alapján). A felügyelt adatbázis további 100 W. Így az össz-IT teljesítmény 250 W. A PUE 1,2-es értékével a teljes létesítményteljesítmény 300 W. Egy hónapban (730 óra) az energiafogyasztás:

$$E = 0,3 \text{ kW} \times 730 \text{ h} = 219 \text{ kWh}$$

A szénlábnyom pedig:

$$CO_2e = 219 \text{ kWh} \times 300 \text{ g } \text{CO}_2\text{e/kWh} = 65,7 \text{ kg } \text{CO}_2\text{e}$$

Az autoscalinggal működő konténeres változatnál az éjszakai leállás miatt az átlagos energiafelvétel alacsonyabb, például 200 W, így a havi fogyasztás 175 kWh, a szénlábnyom 52,5 kg CO e. A becslés bizonytalansága a kihasználtsági mintától függ, ezért a dokumentumban feltüntetjük, hogy a valós méréshez teljesítménymérő eszközökre lenne szükség.

19.4.22 7.5 Döntési napló (ADR példa)

ADR-2024-001: Költség- és szénlábnyom-becslés a laborban

- **Kontextus:** a labor során egy meglévő monolitikus architektúrát hasonlítunk össze egy konténerizált, autoscalinggal rendelkező architektúrával. A cél a költség- és környezeti hatás becslése.
- **Döntés:** a becsléshez a példaértékeket használjuk, és a bizonytalanságot minden számításnál feltüntetjük. A becslést nem tekintjük mérésnek.
- **Alternatívák:**

  - Valós mérés elvégzése (elvetve, mert a labor izolált környezetben történik).
  - Csak költségbecslés, szénlábnyom nélkül (elvetve, mert a fejezet célja mindkét szempont bemutatása).

- **Következmények:** a becslés eredménye nem használható termelési döntéshez; a bizonytalanságot a laborjelentésben jelezni kell.
- **Állapot:** elfogadva.

19.4.23 7.6 Visszaállítási út

A labor nem jár éles rendszer módosításával, ezért nincs szükség visszaállításra. Ha a hallgató a becsléshez a felhőszolgáltató valós árazási adatait kívánja használni, akkor a munkát a szolgáltató ingyenes vagy tesztkörnyezetében végezze, és ne hozzon létre valós költséget generáló erőforrásokat.

19.4.24 8. Önellenőrző kérdés és döntési gyakorlat

Önellenőrző kérdés: Magyarázza el, miért nem elegendő a kihasználtság maximalizálása a költség és a környezeti hatás optimalizálásához, és milyen trade-offokat kell figyelembe venni.

Döntési gyakorlat: Egy vállalat egy éjszakai batch feldolgozást szeretne áthelyezni a felhőbe. A feldolgozás időtűrése 6 óra, a bemenő adat 10 TB, a kimenet 2 TB. Két alternatíva közül választhatnak: (a) állandóan futó Kubernetes klaszter, (b) serverless batch szolgáltatás. A karbonintenzitás az éjszakai órákban 180 g CO e/kWh, nappal 350 g CO e/kWh. Készítsen quality attribute scenario-t a környezeti hatásra, és érveljen az egyik megoldás mellett a karbonhatékonyság és a költség szempontjából. A válaszban tüntesse fel, hogy mely adatok hiányoznak a pontos becsléshez.

19.4.25 9. Hivatkozások és ellenőrzési igény

A fejezetben bemutatott fogalmak és keretrendszerek a következő forrásokra épülnek:

- FinOps Foundation. (2023). *FinOps Framework*. Elérhető: https://www.finops.org/framework/ – A keretrendszer pontos verziója és a fázisok elnevezése a hivatkozott verziótól függ, ezért a felhasználás előtt ellenőrizni kell.
- Amazon Web Services. (2023). *AWS Well-Architected Framework – Cost Optimization Pillar*. Elérhető: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html – Az árazási modellek és a FinOps integráció leírása a dokumentum aktuális verziójában található.
- Google. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. – Az SRE könyv a kihasználtság és a kapacitástervezés összefüggéseit tárgyalja.
- Electricity Maps. (2024). *Carbon intensity data*. Elérhető: https://www.electricitymaps.com/ – A karbonintenzitási adatok frissessége és pontossága a szolgáltató adatminőségétől függ.

A felhőszolgáltatók árazási és karbonintenzitási adatai időben változnak; a fejezetben használt számok példaértékek, és nem helyettesítik a szolgáltató aktuális dokumentációjának ellenőrzését. A laborban a becslést kizárólag oktatási célból készítjük, és azt nem tüntetjük fel mérésként. A fejezetben szereplő állítások a 2024 eleji ismereteket tükrözik, és a jövőben felülvizsgálatra szorulhatnak.

V. RÉSZ – Megbízhatóság, biztonság és üzemeltetés

20 Reziliencia és hibamódok

20.1 Hibamódok és failure mode analysis

A reziliencia tervezés első lépése nem egy konkrét technológiai megoldás kiválasztása, hanem annak megértése, hogy a rendszer milyen módokon hibásodhat meg. A hibamód (*failure mode*) a meghibásodás megfigyelhető formája, szemben a gyökér okkal, amely a meghibásodás kiváltó oka. A hibamód-elemzés a „mi romolhat el?” kérdést teszi fel, míg a gyökér ok elemzés a „miért romlott el?” kérdést. A reziliencia tervezésének alapja a lehetséges meghibásodások feltérképezése, mert csak így lehet rangsorolni a beavatkozásokat, és nem utólag tűzoltani.

Ebben a fejezetben a hibák rendszerezését, a négy leggyakoribb hibamegnyilvánulást, a detektálhatóság és a helyreállítás kérdését, valamint a gyakorlati failure mode and effects analysis (FMEA) módszert mutatjuk be. A fejezet végén egy labor segítségével alkalmazzuk is a módszertant egy ellenőrizhető példán.

20.1.1 Alapfogalmak: hiba, hibamód, hatás

A hibákról való gondolkodásban érdemes elkülöníteni a következő fogalmakat:

- **Hiba (fault)**: a rendszer belső állapotában bekövetkező rendellenesség, például egy adatbázis-kapcsolat megszakadása vagy egy memóriaterület sérülése.
- **Hibamód (failure mode)**: az a mód, ahogyan a rendszer a külvilág számára meghibásodik, például „a szolgáltatás nem válaszol”, „a válaszidő 5 másodperc fölé nő”, „a fizetési rekord összege hibás”.
- **Hibaok (root cause)**: a hiba kiváltó oka, például egy nem kezelt kivétel vagy egy helytelen konfiguráció.
- **Hatás (effect)**: a hibamód következménye a felhasználóra, az üzletre, más komponensekre vagy a teljes rendszerre.

A hibák terjedésének lánca jól modellezhető:

```
flowchart LR
  A[Hibaok] --> B[Belső hiba] --> C[Hibamód] --> D[Hatás]
```

Cím: Hibamód-lánc. Jelmagyarázat: A nyilak a terjedés irányát mutatják. Feltételjegyzet: a rendszerhatáron belül a belső hiba, a rendszerhatáron kívül a hibamód és a hatás észlelhető.

Fontos, hogy a rendszerhatár szerepe meghatározó. Ami egy komponensen belül belső hiba, az a komponens határán kívül már hibamódként jelenik meg. Például a rendelési szolgáltatás adatbázis-kapcsolatkészlete kimerül, mert egy hosszú tranzakció tartja a kapcsolatokat. A belső hiba: nincs szabad kapcsolat. A hibamód: a szolgáltatás 500-as hibát ad, vagy a kérés timeoutol. A hatás: a felhasználó nem tud rendelni, a kosár elveszhet.

20.1.2 A hibák rendszerezése

A hibákat több dimenzió mentén osztályozzuk, hogy ne csak a legismertebb, crash típusú hibákra gondoljunk.

- **Megnyilvánulás szerint**: crash, lassulás, részleges hiba, adatkorrupció. Ezt a négy típust a következő alfejezetben részletesen tárgyaljuk.
- **Eredet szerint**: hardverhiba, szoftverhiba, emberi hiba (például rossz konfiguráció), környezeti hiba (például áramkimaradás), külső függőség hibája (például egy fizetési szolgáltató lassulása).
- **Időtartam szerint**: átmeneti (például egy hálózati késés), tartós (például egy lemez meghibásodása), időszakos (például egy adott napszakban jelentkező erőforrás-telítettség).
- **Detektálhatóság szerint**: azonnali (például crash), késleltetett (például adatkorrupció, amely csak a havi egyeztetéskor derül ki), rejtett (például egy olyan hibás érték, amely soha nem kerül felszínre, de befolyásolja a döntéseket).
- **Terjedés szerint**: lokális (csak egy komponenst érint), kaszkád (egyik komponens hibája további hibákat okoz), közös ok (egyetlen ok több komponenst is leállít).

A taxonómia célja, hogy a hibamód-elemzés során szisztematikusan lefedjük a lehetséges meghibásodásokat. A lassulás és az adatkorrupció gyakran kimarad a tervezésből, pedig ezek okozzák a legtöbb rejtett SLO-sértést és adatvesztést.

20.1.3 A négy fő hibamegnyilvánulás

A következőkben a négy leggyakoribb hibamegnyilvánulást elemezzük. Mindegyiknél megadjuk a definíciót, a tipikus okokat, a detektálás és a helyreállítás lehetőségeit, valamint egy minőségi forgatókönyvet a quality attribute scenario formátum szerint (inger, környezet, válasz, mérőszám).

20.1.4 Crash (összeomlás)

A crash a folyamat vagy csomópont váratlan leállása, amely a szolgáltatás elérhetetlenségét okozza. Ez a legkönnyebben észlelhető hibamegnyilvánulás, mert a szolgáltatás egyszerűen nem válaszol, vagy a kapcsolat megszakad.

Tipikus okok: kezeletlen kivétel, memória kimerülése (OOM), programozói hiba, hardverhiba, helytelen deploy vagy konfiguráció.

Detektálás: a folyamat kilépési kódja, health check, watchdog, az orchestrator (például Kubernetes) liveness probe-ja.

Helyreállítás: automatikus újraindítás, failover egy másik példányra, supervisor folyamat, rollback.

Minőségi forgatókönyv példa:

> **Inger:** A rendelési szolgáltatás OOM miatt összeomlik. **Környezet:** Produkciós környezet, három replika, Kubernetes orchestráció. **Válasz:** A liveness probe 10 másodpercen belül észleli a crash-t, a pod újraindul, a kiesés 20 másodperc. **Mérőszám:** A felhasználók 20 másodpercig 502-es hibát látnak; a havi rendelkezésre állás 99,9%-os SLO-ja 43,2 perc kiesést enged, így ez az incidens 0,77%-a a havi error budgetnek.

A crash gyorsan detektálható, de a helyreállítás ideje attól függ, hogy van-e automatikus failover. Az újraindítás önmagában nem szünteti meg a kiváltó okot, így a crash ismétlődhet.

20.1.5 Lassulás (degradáció)

A lassulás azt jelenti, hogy a szolgáltatás elérhető, de a válaszidő átlépi az SLO-t, vagy a rendszer áteresztőképessége csökken. A lassulás nem mindig jár hibával, de az SLO-t sérti, és az error budgetet fogyasztja. Ráadásul kaszkádot okozhat: a lassú szolgáltatás feltartja a hívókat, amelyek erőforrásai is kimerülnek.

Tipikus okok: CPU-telítettség, lock contention, hosszú garbage collection, hálózati késés, downstream szolgáltatás lassulása, nem optimális adatbázis-lekérdezés.

Detektálás: válaszidő metrika percentilisek (p50, p95, p99), tracing, szintetikus tranzakciók, riasztás az SLO megsértésére.

Helyreállítás: horizontális vagy vertikális skálázás, terheléscsökkentés, timeout bevezetése, cache alkalmazása, backpressure, indexelés, lekérdezés-optimalizálás.

Minőségi forgatókönyv példa:

> **Inger:** A keresési szolgáltatás p95 válaszideje 800 ms-ra nő, az SLO 500 ms. **Környezet:** Produkciós környezet, a keresési szolgáltatás az ajánló szolgáltatást hívja. **Válasz:** A monitorozás 1 perc alatt észleli a lassulást, a terhelésmegosztás és a cache bekapcsolása 5 perc alatt helyreállítja a p95-öt 400 ms-ra. **Mérőszám:** A p95 válaszidő a beavatkozás előtt 800 ms, utána 400 ms; a lassulás időtartama 6 perc, ami a havi 43,2 perces error budget 13,9%-a.

A lassulás detektálása nehezebb, mint a crash-é, mert a szolgáltatás továbbra is válaszol. A percentilis-alapú monitorozás kulcsfontosságú, mert az átlag elrejtheti a kiugró értékeket.

20.1.6 Részleges hiba (partial failure)

Részleges hibáról akkor beszélünk, amikor egyes kérések vagy funkciók meghibásodnak, míg mások működnek. Ez a jelenség különösen gyakori elosztott rendszerekben, ahol a replikák vagy shardok közötti egyenlőtlenségek miatt a hiba csak a forgalom egy részét érinti.

Tipikus okok: egy csomópont kiesése, hálózati partíció, nem determinisztikus hiba, sharding egyenetlenség, egy adott adatpartíció hibája.

Detektálás: hibaarány metrika, log elemzés, trace, szintetikus teszt, health check a terheléselosztó szintjén.

Helyreállítás: retry, failover, replika csere, circuit breaker (a következő fejezetben), adott shard újraépítése.

Minőségi forgatókönyv példa:

> **Inger:** A rendelési szolgáltatás 3 replikájából egy meghibásodik. **Környezet:** Produkciós környezet, a terheléselosztó round-robin elven osztja el a kéréseket. **Válasz:** A kérések 30%-a 500-as hibát kap. A terheléselosztó health check 30 másodperc alatt kiveszi a hibás replikát, a hibaarány 30%-ról 0%-ra csökken. **Mérőszám:** A hibás kérések aránya 30% volt 30 másodpercig; a felhasználók egy része tranzakciót veszít.

A részleges hiba különösen veszélyes, mert a rendszer „félig működik”, és a felhasználó nem tudja, hogy a hiba az ő kérését érinti-e. A hibaarány monitorozása kulcsfontosságú, és a detektálást követően a hibás példányt gyorsan el kell távolítani.

20.1.7 Adatkorrupció

Az adatkorrupció azt jelenti, hogy az adat megsérül, nem felel meg a sémának, az elvárt értéknek vagy a konzisztenciának. Két fő típusa van: logikai (például rossz kerekítés, helytelen sorosítás) és fizikai (például bit rot, lemezhiba).

Tipikus okok: szoftverhiba, tranzakciós anomália, helytelen migráció, hardverhiba, replikációs hiba.

Detektálás: ellenőrző összegek, sémavalidáció, audit log, replika-összehasonlítás, anomáliadetektálás, üzleti egyeztetés.

Helyreállítás: visszaállítás mentésből, tranzakciós rollback, adatjavítás, újragenerálás.

Minőségi forgatókönyv példa:

> **Inger:** A fizetési szolgáltatás egy kerekítési hiba miatt 100,01 EUR helyett 100,02 EUR-t rögzít. **Környezet:** Produkciós környezet, a fizetési tranzakciók napi 10 000 darabot tesznek ki. **Válasz:** A hiba a tranzakcióban nem látható, csak a havi egyeztetéskor derül ki. A javítás 24 órát vesz igénybe, és az érintett tranzakciókat vissza kell állítani. **Mérőszám:** A hibás tranzakciók száma 3 darab (0,03%), a detektálási idő 30 nap, a helyreállítási idő 24 óra.

Az adatkorrupció lehet rejtett, és hosszú ideig észrevétlen marad, ami a legnagyobb kockázat. A detektálhatóság javítása érdekében ellenőrző összegeket, sémavalidációt és folyamatos adatminőség-monitorozást kell alkalmazni.

20.1.8 Detektálhatóság és helyreállítás

A hibamód-elemzés szempontjából kritikus kérdés, hogy a hibát mennyi idő alatt észleljük, és mennyi idő alatt állítjuk helyre. A detektálási idő (*time to detect*, TTD) és a helyreállítási idő (*time to recover*, TTR) összege adja a teljes helyreállítási időt, amelyet gyakran MTTR-ként (*mean time to recover*) emlegetünk.

Detektálási módok:

- **Aktív detektálás:** health check, szintetikus tranzakció, watchdog, heartbeat. Ezek folyamatosan figyelik a rendszert, és gyorsan észlelik a hibákat.
- **Passzív detektálás:** log elemzés, metrika riasztás, trace anomália, hibajegy. Ezek a rendszer által termelt adatok elemzésén alapulnak.

Helyreállítási stratégiák:

- **Automatikus:** restart, failover, rollback, circuit breaker. Emberi beavatkozás nélkül hajtódnak végre.
- **Félautomata:** emberi jóváhagyással végrehajtott script. Például egy adatbázis-rollback, amelyet egy üzemeltető hagy jóvá.
- **Manuális:** runbook alapján végrehajtott lépések. Például egy adott shard újraépítése.

A helyreállítási idő és az error budget kapcsolata közvetlen. Ha az SLO havi 99,9% (43,2 perc kiesés), és egy crash esetén a TTR 30 perc, akkor már egyetlen incidens is felemészti a havi error budget nagy részét. Ezért a hibamód-elemzés során a TTR-t mindig becsülni kell, és a magas TTR-rel rendelkező hibamódokra automatikus helyreállítást kell tervezni.

A detektálhatóság javítása az observability eszközeivel érhető el: megfelelő metrikák, naplózás, tracing és szintetikus monitorok. A detektálhatóság pontozásánál az FMEA-ban azt értékeljük, hogy a hibát milyen valószínűséggel észleljük, mielőtt a hatása továbbterjed.

20.1.9 FMEA: a hibamód- és hatáselemzés gyakorlati módszere

A *Failure Mode and Effects Analysis* (FMEA) egy strukturált módszer a lehetséges hibamódok azonosítására, hatásuk elemzésére és a kockázatok rangsorolására. A módszer a következő lépésekből áll:

1. **Rendszerhatár és komponensek azonosítása.** Meghatározzuk, hogy mi tartozik a rendszerhez, és melyek a külső függőségek.
2. **Funkciók/folyamatok listázása.** A kritikus üzleti folyamatokat felbontjuk komponensekre és azok funkcióira.
3. **Hibamódok azonosítása komponensenként.** Minden funkcióra megkérdezzük: „Hogyan hibásodhat meg?”
4. **Hatás elemzése.** Minden hibamódra leírjuk a lokális hatást, a rendszerszintű hatást és a felhasználóra gyakorolt hatást.
5. **Okok és valószínűség becslése.** Azonosítjuk a lehetséges okokat, és becsüljük az előfordulás gyakoriságát.
6. **Detektálhatóság értékelése.** Megbecsüljük, hogy a hibát milyen valószínűséggel észleljük, mielőtt a hatása eléri a felhasználót.
7. **Kockázati prioritás számítása.** Az RPN (*risk priority number*) értéket a súlyosság, az előfordulás és a detektálhatóság szorzataként számítjuk:

$$RPN = Slyossg \times Elforduls \times Detektlhatsg$$

   Ahol a súlyosság (1–10), az előfordulás (1–10) és a detektálhatóság (1–10, ahol 1 = könnyen detektálható, 10 = nehezen detektálható). Az RPN magas értéke jelzi a legnagyobb kockázatú hibamódokat.
8. **Intézkedések tervezése és felelősök.** A magas RPN-ű hibamódokra beavatkozásokat tervezünk, és felelősöket rendelünk.
9. **Újraértékelés.** Az intézkedések után a pontszámokat újraértékeljük, és az FMEA-t frissítjük.

Egy egyszerű FMEA táblázat példa egy webáruház rendelési folyamatára:

Komponens	Funkció	Hibamód	Hatás	Ok	Súlyosság (1-10)	Előfordulás (1-10)	Detektálhatóság (1-10)	RPN	Intézkedés
Rendelési szolgáltatás	Rendelés mentése	Crash	A felhasználó nem tud rendelni	OOM	8	3	2	48	Health check, restart, memória limit
Rendelési szolgáltatás	Rendelés mentése	Lassulás	A rendelési kérés 2 s-ig tart	DB lock	5	4	5	100	Indexelés, timeout
Fizetési szolgáltatás	Tranzakció rögzítése	Adatkorrupció	A fizetési összeg hibás	Kerekítési hiba	9	2	8	144	Decimális típus, validáció, egyeztetés
Fizetési szolgáltatás	Tranzakció rögzítése	Részleges hiba	Egyes tranzakciók nem mennek át	Shard kiesés	7	3	4	84	Replikáció, failover

A táblázatban a detektálhatóság skálája: 1 = azonnal detektálható, 10 = csak utólag, manuális egyeztetéssel. Az RPN küszöbértéket a csapat határozza meg, például RPN > 80 esetén beavatkozás szükséges.

Fontos, hogy a pontszámok szubjektívek, ezért a csapatnak közösen kell kalibrálnia a skálákat, és a kritériumokat rögzíteni kell. Az FMEA nem egyszeri dokumentum, hanem folyamatosan frissítendő, különösen nagy változások (új funkció, új függőség, architektúraváltás) után.

20.1.10 Labor: Failure mode és hatáselemzés készítése

A labor célja, hogy a fenti módszertant egy ellenőrizhető, reprodukálható dokumentumban alkalmazzuk. A labor nem éles rendszeren fut, hanem egy kitalált webáruház-rendszeren.

Kiinduló állapot: Adott egy egyszerű webáruház-rendszer, amely a következő komponensekből áll:

- **Webes frontend:** a felhasználói felület, amely a böngészőben fut.
- **Rendelési szolgáltatás:** a rendelések felvételét és állapotát kezeli.
- **Fizetési szolgáltatás:** a fizetési tranzakciókat dolgozza fel, és egy külső fizetési szolgáltatót hív.
- **Készletkezelő szolgáltatás:** a raktárkészletet tartja nyilván.

- **Relációs adatbázis:** a tranzakciós adatok tárolása.
- **Külső rendszerek:** fizetési szolgáltató, e-mail szolgáltató.

A rendszerhatár a teljes webáruház, a kulcsfolyamat pedig a „rendelés leadása”.

Elvárt eredmény: Egy verziózott dokumentum (például Markdown vagy Jupyter Notebook), amely tartalmazza:

1. A rendszerhatár és a kulcsfolyamat leírását.
2. Legalább 5 hibamódot, amelyek lefedik a négy fő hibamegnyilvánulást (crash, lassulás, részleges hiba, adatkorrupció).
3. Minden hibamódhoz hatást, okot, detektálási módot és helyreállítási stratégiát.
4. RPN értékeket és intézkedési tervet.
5. Legalább egy minőségi forgatókönyvet a legmagasabb RPN-ű hibamódra.
6. Egy ADR-t, amely a legmagasabb kockázatú hibamód kezelésére vonatkozó döntést rögzíti (kontextus, döntés, alternatívák, következmények, állapot).

Ellenőrzési szempontok:

- A hibamódok lefedik a négy fő hibamegnyilvánulást?
- Minden hibamódhoz tartozik hatás, ok, detektálás és helyreállítás?
- A RPN küszöbérték definiált és indokolt?
- A detektálási és helyreállítási idő becsült?
- A dokumentum verziózott, és a döntési napló hivatkozik a labor azonosítójára?
- A minőségi forgatókönyv tartalmazza az ingert, a környezetet, a választ és a mérőszámot?
- Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket?

Visszaállítási út: Mivel a labor dokumentációs jellegű, nincs szükség destruktív műveletre. A dokumentumot verziókezelőben kell tárolni, és a labor végén a döntési naplót frissíteni kell a labor azonosítójával.

A labor során a mért és a várt eredményt egyértelműen el kell különíteni. Mivel itt feltételezett értékekkel dolgozunk, azokat „becsült” vagy „szimulált” jelzővel kell ellátni.

20.1.11 Ellenőrzőlista a hibamód-elemzéshez

A fejezet végén a következő ellenőrzőlistát használhatja a hibamód-elemzés minőségének értékelésére:

- ☐ Minden kritikus komponenshez van legalább egy hibamód?
- ☐ A négy fő hibamegnyilvánulás (crash, lassulás, részleges hiba, adatkorrupció) lefedett?
- ☐ Minden hibamódhoz tartozik hatás, ok, detektálás és helyreállítás?
- ☐ A RPN küszöbérték definiált és a csapat által elfogadott?
- ☐ A detektálási és helyreállítási idő mérése tervben van?
- ☐ Az FMEA frissítési ciklusa meghatározott?
- ☐ A legmagasabb kockázatú hibamódokra van intézkedési terv?

20.1.12 Önellenőrző kérdés

Melyik hibamegnyilvánulás a legnehezebben detektálható a négy közül, és miért? Írjon le egy olyan forgatókönyvet, ahol az adatkorrupció rejtett marad, majd adja meg a minőségi forgatókönyv mind a négy elemét: inger, környezet, válasz, mérőszám.

20.1.13 Gyakorlat

Adott három hibamód:

1. A rendelési szolgáltatás crash-e.
2. A fizetési szolgáltatás lassulása.
3. A készlet adatbázis rekordjainak korrupciója.

Válasszon ki egyet, és indokolja meg, hogy milyen detektálási stratégiát alkalmazna, és miért. A válaszban térjen ki a detektálási idő és a helyreállítási idő becslésére is, és fogalmazza meg a minőségi forgatókönyvet.

20.1.14 Összegzés

A hibamód-elemzés a reziliencia alapja: mielőtt védelmi mechanizmusokat építenénk, tudnunk kell, mi ellen védekezünk. A négy fő megnyilvánulás – crash, lassulás, részleges hiba, adatkorrupció – más-más detektálási és helyreállítási megközelítést igényel. Az FMEA strukturált módszert ad a kockázatok rangsorolására, és segít a beavatkozások priorizálásában. A következő fejezet (Circuit breaker, bulkhead, retry és timeout) az itt azonosított hibamódokra adott konkrét védelmi mintákat tárgyalja: circuit breaker, bulkhead, retry és timeout.

20.2 Circuit breaker, bulkhead, retry és timeout

A hibaizoláció és a túlterhelés megelőzésének témája akkor válik igazán fontossá, amikor egy rendszer már nem elszigetelt folyamatként, hanem hálózaton keresztül együttműködő szolgáltatások láncaként működik. Egyetlen lassú vagy hibás függőség képes magával rántani a teljes kérésfeldolgozási utat, ha a hívó fél nem rendelkezik megfelelő védelmi mechanizmusokkal. Ebben a fejezetben a leggyakrabban használt négy mintát vizsgáljuk meg: a **timeout**, a **retry**, a **circuit breaker** és a **bulkhead** működését, paramétereit, kombinálásukat, valamint a velük kapcsolatos leggyakoribb hibákat. A cél nem a minták puszta felsorolása, hanem annak megértése, hogy melyik milyen hibamódra ad választ, milyen feltételek mellett működik jól, és hogyan illeszthető be egy valós hívási láncba.

A fejezet végére képes leszel megmagyarázni, hogyan akadályozzák meg ezek a minták a kaszkád hibákat, alkalmazni tudod őket egy kisméretű, ellenőrizhető példán, és felismered a tipikus konfigurációs és tervezési hibákat.

20.2.1 A hibaizoláció szükségessége

Az előző fejezetben a hibamódok elemzésével foglalkoztunk: azonosítottuk, hogy egy komponens milyen módokon hibásodhat meg, és ezek hogyan terjedhetnek tovább. Most arra koncentrálunk, milyen architekturális mintákkal csökkenthetjük a hiba terjedésének esélyét vagy hatását. A kiindulópont az a felismerés, hogy elosztott rendszerekben **nem az a kérdés, hogy lesz-e hiba, hanem hogy mikor és milyen formában**. Egy hálózati hívás megszakadhat, egy adatbázis túlterhelődhet, egy külső fizetési szolgáltató átmenetileg lassulhat. Ha a hívó szolgáltatás minden ilyen esetben korlátlanul vár, vagy újra és újra megpróbálja a műveletet anélkül, hogy bármilyen korlátot szabna, akkor a saját erőforrásai is elfogynak, és a hiba továbbgyűrűzik a tőle függő komponensekre. A hibaizoláció célja, hogy a hibás vagy lassú komponenst körülhatárolja, a hívó oldalt pedig megóvja a túlterheléstől.

A túlterhelés megelőzése szorosan összefügg ezzel. Amikor egy szolgáltatás nem tud lépést tartani a beérkező kérésekkel, a válaszidők megnőnek, a várakozó kérések sorban állnak, és a rendszer erőforrásai – szálak, adatbázis-kapcsolatok, memória – fokozatosan kimerülnek. A korlátok nélküli várakozás és újrapróbálás csak fokozza ezt a folyamatot. A timeout, a retry, a circuit breaker és a bulkhead mind olyan eszköz, amely ezt a nyomást csökkenti, ha helyesen alkalmazzák.

20.2.2 Timeout: a türelem határai

A **timeout** a legegyszerűbb, mégis az egyik legfontosabb védekezési mechanizmus. Azt határozza meg, hogy egy hívás legfeljebb mennyi ideig várhat a válaszra, mielőtt a hívó fél megszakítja a műveletet és hibával tér vissza. A timeout nélküli szinkron hívás a hívó szálat korlátlan ideig blokkolhatja, ami szálkészletek kimerüléséhez és a szolgáltatás teljes leállásához vezethet.

A timeout értékét a hívás jellegéhez kell igazítani. Egy belső, gyors kulcs-érték tároló hívásához 100–200 ms is bőven elegendő lehet, míg egy külső fizetési szolgáltató hívása – amely akár több másodpercig is eltarthat – nagyobb, például 5–10 másodperces időtúllépést igényelhet. A túl rövid timeout hamis hibákat generál, mert a lassú, de sikeres válaszokat hibaként könyveli el. A túl hosszú timeout viszont nem védi meg a rendszert a túlterheléstől, hiszen a hívó erőforrások sokáig le vannak kötve egy olyan kérésre, amely valószínűleg már nem fog időben válaszolni.

Fontos megkülönböztetni a **kapcsolódási timeoutot** és a **válasz timeoutot**. Előbbi arra vonatkozik, mennyi ideig várhat a hívó a kapcsolat felépítésére, utóbbi arra, hogy a kapcsolat létrejötte után mennyi idő áll rendelkezésre a teljes válasz megérkezésére. Egyes klienskönyvtárak további értékeket is kezelnek, például az egyes adatcsomagok közötti olvasási időtúllépést. A timeout konfigurációnál mindig érdemes a **teljes hívási lánc** időkeretére is gondolni: ha egy kérés több szolgáltatáson megy keresztül, a belső timeoutoknak összhangban kell lenniük a külső, végfelhasználó felé vállalt maximális válaszidővel.

A modern elosztott rendszerekben a timeout értéket érdemes **deadline propagation** mechanizmussal továbbítani a hívási láncban. Ilyenkor a kezdő kérés rendelkezik egy abszolút határidővel, és minden köztes szolgáltatás ezt a határidőt csökkenti a saját feldolgozási idejével, mielőtt továbbítaná a hívást. Így a lánc végén lévő komponens is tudja, mennyi ideje maradt, és nem végez felesleges munkát egy olyan kérésen, amelyet a hívó már régen elengedett. A deadline propagation implementálása nem minden keretrendszerben automatikus, ezért tervezéskor ellenőrizni kell, hogy a használt kommunikációs protokoll és a kliensek támogatják-e (például gRPC esetén igen, HTTP felett gyakran egyéni fejléc vagy kontextus szükséges hozzá).

20.2.3 Retry és idempotencia: az újrapróbálás fegyelme

A **retry** (újrapróbálás) azt a stratégiát jelenti, amikor a hívó fél egy sikertelen hívást – hálózati hiba, időtúllépés vagy átmeneti szerverhiba esetén – megkísérel újra végrehajtani. A retry hatékonyan növeli a rendelkezésre állást az átmeneti hibákkal szemben, de csak akkor, ha a művelet **idempotens**. Az idempotencia azt jelenti, hogy a művelet többszöri végrehajtása ugyanazt az eredményt adja, mint az egyszeri végrehajtás. Ha egy hívás nem idempotens, a retry duplikált mellékhatásokat okozhat: kétszer levont fizetési összeget, két létrehozott rendelést vagy két elküldött e-mailt.

Az idempotencia megvalósítására több módszer létezik. A természetesen idempotens műveletek – például egy erőforrás lekérdezése vagy egy rekord frissítése adott értékre – nem igényelnek különösebb védelmet. A létrehozó műveleteknél azonban gyakran szükség van **idempotencia kulcsra**. A kliens generál egy egyedi azonosítót, és azt minden újrapróbált kéréshez mellékeli.

A szerver ezt a kulcsot eltárolja, és ha ugyanazzal a kulccsal újabb kérés érkezik, nem hajtja végre újra a műveletet, hanem visszaadja az eredeti választ vagy egy konfliktusjelzést. Ilyen idempotencia kulcsot használnak például a modern fizetési API-k, hogy a hálózati hiba miatt megismételt fizetési kéréseket ne terheljék kétszer.

A retry konfigurálásakor több paramétert kell meghatározni:

- **Maximális próbálkozások száma**: hányszor hajtható végre a hívás, beleértve az első kísérletet.
- **Várakozási idő a próbálkozások között**: ez lehet állandó, lineárisan növekvő vagy exponenciális.
- **Mely hibatípusokra érvényes a retry**: például a HTTP 503 Service Unavailable vagy a hálózati időtúllépés indokolhat újrapróbálást, míg a 400 Bad Request vagy a 401 Unauthorized általában nem.
- **Idempotens-e a művelet**: ha nem, a retry csak akkor alkalmazható, ha a duplikáció kockázata elfogadható, vagy ha a fenti idempotencia kulcsos mechanizmus biztosítja a védelmet.

Az exponenciális visszavonulás (**exponential backoff**) azt jelenti, hogy a próbálkozások közötti várakozási idő minden egyes ismétlésnél növekszik, például 100 ms, 200 ms, 400 ms, 800 ms, és így tovább. Ez a stratégia csökkenti a túlterhelt szolgáltatásra nehezedő azonnali nyomást, mert a hívók egyre ritkábban próbálkoznak. A lineáris vagy állandó várakozási idő egyszerűbb, de nagy hibaszám esetén kevésbé hatékony.

20.2.4 Jitter: a szinkronizált újrapróbálások elkerülése

A **jitter** (szándékos véletlenszerűség) az exponenciális visszavonulás kiegészítése. Képzeljük el, hogy egy szolgáltatás átmenetileg elérhetetlenné válik, és több száz kliens egyszerre próbálja újra elérni. Ha mindegyik ugyanazt az exponenciális ütemezést használja, a próbálkozások szinkronban maradnak, és amikor a szolgáltatás újra elérhetővé válik, egyszerre hatalmas kéréshullámot kap. Ezt a jelenséget **thundering herd** (mennydörgő csorda) néven ismerjük. A jitter lényege, hogy a várakozási időhöz véletlenszerű értéket adunk, így a kliensek próbálkozásai szétoszlanak az időben.

A jitter többféleképpen számítható. A legegyszerűbb a **teljes jitter**: a várakozási idő egyenletes eloszlásból származik, nulla és a kiszámított exponenciális érték között. A **decorrelated jitter** az előző várakozási időt veszi alapul, és a következő várakozást az előző érték és egy felső korlát közötti véletlen értékként határozza meg. Mindkét megközelítés csökkenti a szinkronizált hullámok kialakulásának esélyét, miközben megőrzi a visszavonulás fokozatosságát.

A retry és a jitter kombinációja a gyakorlatban például a következőképpen nézhet ki egy HTTP kliensben:

```
def call_with_retries(request, max_attempts=5):
    base_delay_ms = 100
    max_delay_ms = 5000
    for attempt in range(max_attempts):
        try:
            response = http_call(request)
            if response.status_code < 500:
                return response
        except NetworkError:
            pass
        if attempt < max_attempts - 1:
            exponential = min(base_delay_ms * (2 ** attempt), max_delay_ms)
            # Teljes jitter: egyenletes eloszlás 0 és exponential között
```

```
            delay = random.uniform(0, exponential)
            time.sleep(delay / 1000)
    raise MaxRetriesExceeded("A hívás a maximális próbálkozások után sem sikerült.")
```

Ez a példa szemléltető jellegű: éles rendszerben a retry logikát általában bevált könyvtárakra bízzuk, és a konfigurációt külsőleg, például konfigurációs fájlban tartjuk. A fenti pszeudokódban a `max_attempts` a teljes próbálkozások számát jelöli, a `random.uniform(0, exponential)` pedig a teljes jittert valósítja meg. A jitter nélküli változathoz képest a kérések időbeli eloszlása sokkal egyenletesebb lesz, ami csökkenti a szolgáltatás túlterhelésének kockázatát.

20.2.5 Circuit breaker: a hibás függőség leválasztása

A **circuit breaker** (megszakító) egy olyan minta, amely figyeli egy függőség hívásainak sikerességét, és ha a hibák aránya meghalad egy küszöböt, egy időre „nyitja” az áramkört, azaz a további hívásokat azonnali hibával elutasítja anélkül, hogy ténylegesen meghívná a függőséget. Ezzel a hibás vagy túlterhelt szolgáltatást időt kap a regenerálódásra, a hívó fél pedig elkerüli a hosszú várakozást és a további terhelést.

A circuit breaker három állapottal rendelkezik:

- **Closed (zárt)**: a hívások normálisan áthaladnak, és a megszakító figyeli a sikerességi arányt. Ha a hibák aránya egy meghatározott küszöb fölé emelkedik, a megszakító **Open** állapotba vált.
- **Open (nyitott)**: minden bejövő hívás azonnali hibával tér vissza, a függőség tényleges hívása nélkül. Ez a gyors bukás (fail fast) mechanizmusa.
- **Half-open (félig nyitott)**: egy meghatározott várakozási idő után a megszakító egy korlátozott számú tesztkérést enged át. Ha ezek sikeresek, az áramkör visszazárul **Closed** állapotba. Ha nem, ismét **Open** állapotba kerül, és a várakozási idő újraindul.

A circuit breaker konfigurációjának kulcsparaméterei:

- **Hibaarány küszöb**: például 50% – ha az utolsó N hívás több mint fele hibás, a megszakító nyit.
- **Minimális kérésszám a kiértékeléshez**: elkerüli, hogy egy-két véletlen hiba azonnal nyissa az áramkört.
- **Open állapot időtartama**: mennyi ideig marad nyitva, mielőtt half-open állapotba lép.
- **Half-open tesztkérések száma**: hány kérés engedhető át a tesztelési fázisban.

Egy tipikus konfiguráció lehet például: a megszakító akkor nyit, ha az utolsó 20 kérés közül legalább 5 hibás, és az open állapot 10 másodpercig tart, majd half-open állapotban 3 tesztkérés dönt a zárásról. Az ilyen paraméterek mindig a függőség jellegétől függenek. Egy ritkán hívott, de kritikus függőségnél a minimális kérésszám alacsonyabb lehet, míg egy nagy forgalmú, toleráns szolgáltatásnál magasabb.

A circuit breaker nem helyettesíti a retry-t, hanem kiegészíti. A retry az átmeneti hibákra válaszol, a circuit breaker pedig a hosszabb ideig tartó hibás állapotra. Egy jól felépített hívási láncban a retry a circuit breaker mögött működik: ha a megszakító nyitott állapotban van, a retry-nek nincs értelme, mert a hívás azonnal elutasításra kerül. Ezért a retry és a circuit breaker sorrendje fontos: a retry logika a circuit breaker hívása után, annak hibája esetén fut le.

A circuit breaker megvalósítására számos könyvtár létezik, például a .NET ökoszisztémában a Polly, Java-ban a Resilience4j, Pythonban a pybreaker vagy a tenacity. A konkrét API-k és verziók folyamatosan változnak, ezért használat előtt mindig ellenőrizni kell az adott könyvtár

dokumentációját. Az elv azonban mindenütt ugyanaz: sikertelenségi küszöb, nyitás, fél-nyitott tesztelés.

20.2.6 Bulkhead: erőforrások elszigetelése

A **bulkhead** (válaszfal) minta a hajók válaszfalaira utal: ha az egyik rekesz megsérül és víz áramlik be, a többi rekesz száraz marad, és a hajó nem süllyed el. A szoftverarchitektúrában ez azt jelenti, hogy a különböző függőségekhez vagy feladattípusokhoz külön erőforrás-készleteket rendelünk, így az egyik terhelése nem fogyasztja el a másik számára szükséges erőforrásokat.

A legegyszerűbb megvalósítás a **szálkészletek elkülönítése**. Tegyük fel, hogy egy szolgáltatás két külső hívást hajt végre: egy gyors, kritikus hitelesítési ellenőrzést és egy lassú, nem kritikus jelentéslekérdezést. Ha mindkettő ugyanazt a szálkészletet használja, és a jelentéslekérdezés hirtelen túlterhelődik, az összes szálat lefoglalhatja, így a hitelesítési ellenőrzés is elérhetetlenné válik. A bulkhead minta szerint a két híváshoz külön szálkészletet rendelünk: a hitelesítésé például 10 szálat, a jelentésé 5 szálat kap. Így ha a jelentéskészlet kimerül, a hitelesítés továbbra is működik.

A bulkhead nem csak szálkészletekre vonatkozik, hanem bármilyen erőforrásra: adatbázis-kapcsolatok, HTTP kapcsolatok, üzenetsorok, memória-pufferek. A minta lényege a **korlátozott erőforrások elszigetelése**, hogy egyetlen hibás vagy lassú komponens ne tudja magával rántani az egész rendszert.

A bulkhead konfigurálásakor a következő paramétereket kell meghatározni:

- **Erőforrás típusa**: milyen erőforrást szigetelünk el (szál, kapcsolat, memória).
- **Maximális párhuzamosság**: az adott erőforrásból hány férhet hozzá egy időben.
- **Várakozási sor hossza**: ha a párhuzamosság kimerült, hány kérés várhat még a sorban.
- **Időtúllépés a sorban állásra**: mennyi ideig várhat egy kérés az erőforrás felszabadulására, mielőtt hibával tér vissza.

A bulkhead hatékonyan megelőzi a túlterhelés terjedését, de ára van: a sok kis erőforrás-készlet megnöveli a konfigurációs komplexitást, és a készletek közötti kontextusváltás többletköltséget okozhat. Továbbá, ha a készletek mérete rosszul van beállítva, a rendszer kihasználatlan erőforrásokat pazarol, vagy éppen szűk keresztmetszetet hoz létre. A bulkhead ezért nem minden függőségre indokolt: a kritikus, egymástól független feladatok elválasztása a legnagyobb haszon, míg a ritka, alacsony kockázatú hívásoknál a közös erőforrás-készlet is elfogadható.

20.2.7 A minták összehangolása egy hívási láncban

A négy minta ritkán működik önmagában. Egy valós hívási láncban a timeout, a retry, a circuit breaker és a bulkhead egymást kiegészítve védi a rendszert. Tekintsünk egy egyszerű példát: egy rendelési szolgáltatás hívja a fizetési szolgáltatást.

A hívási lánc a következő lépésekből állhat:

1. A rendelési szolgáltatás egy bulkhead-en keresztül éri el a fizetési szolgáltatást, amelynek külön kapcsolatkészlete van.
2. A fizetési hívás egy timeout-tal rendelkezik, például 3 másodperc.
3. A hívás mögött egy circuit breaker figyeli a fizetési szolgáltatás állapotát.
4. Ha a hívás időtúllépés vagy átmeneti szerverhiba miatt sikertelen, a retry logika az idempotencia kulccsal újrapróbálja.

Fontos, hogy a sorrend és a felelősségi körök világosak legyenek. A timeout a hívás időtartamát korlátozza, a bulkhead az erőforrásokat szigeteli el, a circuit breaker a hosszabb hibás állapotokat érzékeli, a retry pedig az átmeneti hibákra ad választ. Ha bármelyik hiányzik, a rendszer sebezhetőbbé válik. Például retry nélkül az átmeneti hálózati hiba azonnali hibát okoz a felhasználónál. Circuit breaker nélkül a retry tovább terheli a már túlterhelt szolgáltatást. Bulkhead nélkül pedig a fizetési szolgáltatás lassúsága az összes rendelési kérést blokkolhatja.

Az alábbi szekvenciadiagram egy ilyen védelmi réteggel ellátott hívási láncot mutat be:

```
sequenceDiagram
    participant C as Kliens (Order Service)
    participant B as Bulkhead (fizetési pool)
    participant CB as Circuit Breaker
    participant P as Payment Service

    C->>B: Fizetési hívás (idempotencia kulccsal)
    B->>CB: Hívás engedélyezve, erőforrás lefoglalva
    CB->>P: HTTP POST /payments
    alt A hívás sikeres
        P-->>CB: 200 OK
        CB-->>B: Válasz
        B-->>C: Válasz
    else A hívás időtúllépés vagy 503
        P-->>CB: Hiba
        CB-->>B: Hiba
        B-->>C: Hiba
        C->>C: Retry logika (exponenciális backoff + jitter)
        C->>B: Újrapróbált hívás (ugyanaz az idempotencia kulcs)
        B->>CB: Hívás engedélyezve
        CB->>P: HTTP POST /payments
        P-->>CB: 200 OK
        CB-->>B: Válasz
        B-->>C: Válasz
    end
```

A diagramon látható, hogy a circuit breaker a bulkhead és a fizetési szolgáltatás között helyezkedik el. A retry logika a hívó oldalon fut, és a teljes bulkhead–circuit breaker láncot újrapróbálja. Ez a felépítés biztosítja, hogy a retry ne kerülje meg a védelmi mechanizmusokat, és hogy a circuit breaker nyitott állapotában a retry azonnali hibát kapjon, így nem növeli a terhelést.

20.2.8 Tipikus hibák és anti-minták

A fenti minták helytelen alkalmazása gyakran több kárt okoz, mint hasznot. Az alábbiakban a leggyakoribb hibákat gyűjtöttük össze.

Retry minden hibára. Az újrapróbálás csak átmeneti hibák esetén indokolt. Ha a hívás 400 Bad Request vagy 401 Unauthorized válasszal tér vissza, a retry nem fog segíteni, csak felesleges terhelést generál. Hasonlóképpen, ha a művelet nem idempotens, a retry duplikált mellékhatásokat okozhat. A retry szabályt mindig a hibakategóriákhoz kell kötni.

Nincs jitter. Az exponenciális backoff jitter nélkül is működik, de nagy klienstömeg esetén szinkronizált újrapróbálási hullámokat okoz. A jitter hiánya a thundering herd jelenséghez vezethet, ami éppen azt a túlterhelést fokozza, amelyet a retry el akart kerülni.

Túl agresszív timeout. A túl rövid timeout miatt a rendszer hamis hibákat észlel, és a felhasználók olyan műveleteknél is hibát kapnak, amelyek valójában sikeresek lettek volna. Ez a hibás hibaészlelés ráadásul a retry-t is szükségtelenül aktiválja. A timeout értékét a valós

válaszidő-eloszlás alapján kell meghatározni, nem pedig egy önkényes, „biztonságosnak” tűnő alacsony értékkel.

Circuit breaker túl érzékeny. Ha a hibaarány küszöb túl alacsony, vagy a minimális kérésszám túl kicsi, a megszakító egy-két véletlen hiba hatására is nyithat, ami indokolatlan leállásokat okoz. A küszöböt a függőség stabilitásának és a hívási forgalomnak megfelelően kell kalibrálni.

Circuit breaker nyitva felejtése. A half-open állapot elhagyása vagy a nyitott állapot időtartamának túl hosszúra állítása azt eredményezi, hogy a rendszer akkor sem próbálja újra a függőséget, amikor az már felépült. A megszakítónak mindig rendelkeznie kell automatikus visszazárási logikával.

Bulkhead nélküli közös erőforrások. Ha minden hívás ugyanazt a szálkészletet vagy kapcsolatkészletet használja, a bulkhead előnyei elvesznek. A kritikus útvonalakat – például a hitelesítést vagy a fizetést – mindig el kell különíteni a kevésbé fontos, lassú hívásoktól.

Túl sok bulkhead. A túlzott erőforrás-szétosztás növeli a konfigurációs terhet és a működési költséget, miközben a készletek alulkihasználtak maradhatnak. A bulkhead kialakításánál a függőségek kritikusságát és forgalmát kell figyelembe venni.

A minták egymás ellen dolgoznak. Ha például a retry megkerüli a circuit breakert, vagy a timeout értéke nagyobb, mint a circuit breaker open állapotának időtartama, a rendszer viselkedése kiszámíthatatlanná válik. A védelmi rétegeket úgy kell kialakítani, hogy egymásra épüljenek, és a hibák ne tudják kijátszani őket.

20.2.9 Labor: reziliencia-minta kiválasztása egy hívási láncra

A labor célja, hogy a fejezetben bemutatott mintákat egy konkrét, de egyszerűsített hívási láncra alkalmazd. Az átadandó eredmény egy reprodukálható dokumentum vagy jegyzetfüzet, amely rögzíti a kontextust, a döntési naplót és az ellenőrzési szempontokat, és világosan elkülöníti a mért és a várt eredményt.

20.2.10 Kiinduló állapot

Egy e-kereskedelmi rendszerben a **Rendelési szolgáltatás** (Order Service) két külső függőséget hív meg szinkron módon:

1. **Fizetési szolgáltatás** (Payment Service) – hitelesített HTTP POST hívás, amely a fizetési tranzakciót indítja. Átlagos válaszidő: 250 ms. A szolgáltatás időnként lassú, és 5%-ban 503-as hibát ad vissza.
2. **Készletnyilvántartó szolgáltatás** (Inventory Service) – HTTP GET hívás, amely a raktárkészletet ellenőrzi. Átlagos válaszidő: 80 ms, de a szolgáltatás hetente egyszer 30 percre elérhetetlenné válik karbantartás miatt.

A Rendelési szolgáltatás jelenleg semmilyen védelmi mintát nem alkalmaz: nincs timeout, nincs retry, nincs circuit breaker, és a két hívás ugyanazt a szálkészletet használja. A rendszer terheléses tesztjei azt mutatják, hogy a fizetési szolgáltatás lassulásakor a Rendelési szolgáltatás szálkészlete 10 másodperc alatt kimerül, és a készletellenőrzés is elérhetetlenné válik.

20.2.11 Feladat

Tervezd meg a két hívás reziliencia-konfigurációját a fenti minták felhasználásával. Az alábbi kérdésekre válaszolj, és a döntéseidet indokold:

1. Milyen timeout értéket állítanál be a Fizetési és a Készletnyilvántartó hívásokhoz? Miért?
2. Melyik hívásnál alkalmaznál retry-t, és milyen paraméterekkel? Hogyan biztosítanád az idempotenciát?
3. Hogyan konfigurálnád a circuit breakert a Fizetési szolgáltatáshoz? Milyen állapotátmeneteket várnál?
4. Hogyan alakítanád ki a bulkhead-et a két hívás között? Milyen erőforrásokat és limiteket választanál?
5. Milyen sorrendben kapcsolnád össze a mintákat? Rajzold le a hívási láncot diagrams-as-code formában (például Mermaid szekvenciadiagram).

20.2.12 Elvárt eredmény

A labor végére egy olyan dokumentumot kell készítened, amely tartalmazza:

- A hívási lánc kontextusának leírását (a fenti kiinduló állapot, kiegészítve a saját feltételezéseiddel).
- A döntési naplót, amelyben minden minta kiválasztásának okát rögzíted (kontextus, döntés, alternatívák, következmények).
- A hívási lánc diagramját, jelölve a timeout, retry, circuit breaker és bulkhead komponenseket.
- A konfigurációs paraméterek táblázatát, mértékegységekkel.
- A várt viselkedés leírását normál, lassú és hibás függőség esetén.
- A mért és a várt eredmény elkülönítését: ha rendelkezésre állnak mérési adatok (például terheléses tesztből), azokat külön szakaszban tüntesd fel, és jelöld, melyik állítás mért, melyik várt.

20.2.13 Ellenőrzési szempontok

- A timeout értékek indoklása összhangban van a válaszidő-eloszlással.
- A retry csak a megfelelő hibakategóriákra vonatkozik, és az idempotencia biztosított.
- A circuit breaker paraméterei (küszöb, minimális kérésszám, open időtartam, half-open tesztkérések) konzisztensek a hívási forgalommal.
- A bulkhead elkülöníti a kritikus és a kevésbé kritikus hívásokat, és a limitek nem irreálisan nagyok vagy kicsik.
- A minták sorrendje helyes: a retry a circuit breaker mögött, a timeout minden hívásra vonatkozik.
- A diagram jelzi a rendszerhatárokat, a komponensek felelősségét és a kapcsolatok protokollját.

20.2.14 Visszaállítási út

Mivel ez a labor tervezési gyakorlat, éles rendszeren nem kell végrehajtani semmilyen konfigurációt. Ha a mérésekhez saját lokális környezetet hozol létre, a változtatásokat izolált konténerben vagy virtuális gépen végezd, és a labor végén állítsd vissza az eredeti állapotot a konfigurációs fájlok visszaállításával. A destruktív műveletek (például adatbázis-törlés) nem részei ennek a feladatnak.

20.2.15 Mért és várt eredmény elkülönítése

A labor dokumentumban külön szakaszban jelöld, hogy mely állítások alapulnak valós mérésen, és melyek csupán várt becslések. Például:

- **Mért:** a Fizetési szolgáltatás válaszideje 250 ms medián, a 90. percentilis 700 ms, az 503-as hibák aránya 5% (terheléses tesztből, 1000 kérés, 2025. 06. 10., környezet: lokális Kubernetes klaszter, 3 csomópont).
- **Várt:** a circuit breaker 50%-os hibaarány küszöb mellett 20 kérésből 10 hiba esetén nyit; ez a konfiguráció a fenti forgalom mellett várhatóan 10 másodperc alatt aktiválódik. (Ezt a becslést a labor során nem méréssel, hanem a paraméterekből számolással igazoljuk.)

Az ilyen elkülönítés elengedhetetlen ahhoz, hogy a dokumentáció ne keltsen hamis bizonyosságot, és a későbbi olvasó pontosan tudja, mi bizonyított és mi feltételezett.

20.2.16 Ellenőrzőlista a reziliencia-minták tervezéséhez

A fejezet lezárásaként az alábbi ellenőrzőlista segít a tervezési lépések végiggondolásában:

- ☐ Minden külső hívásra meghatározott timeout van, az érték a válaszidő-eloszláson alapul.
- ☐ A retry csak idempotens műveletekre vagy megfelelő idempotencia kulccsal védett hívásokra vonatkozik.
- ☐ A retry hibakategóriái szűkítettek: csak átmeneti hibákra vonatkozik.
- ☐ Az exponenciális backoff jitterrel van kombinálva.
- ☐ A circuit breaker figyeli a kritikus függőségeket, és paraméterei a forgalomhoz igazodnak.
- ☐ A circuit breaker half-open állapota korlátozott tesztforgalmat enged át, és automatikusan visszazár.
- ☐ A bulkhead elkülöníti a kritikus hívási útvonalakat, és a limitek nem okoznak indokolatlan erőforrás-pazarlást.
- ☐ A minták sorrendje helyes, és nem tudják egymást kijátszani.
- ☐ A hívási lánc diagramján jelölve vannak a védelmi komponensek, a protokollok és a rendszerhatárok.
- ☐ A konfigurációs értékek mértékegységgel és mérési ablakkal szerepelnek a dokumentációban.

20.2.17 Önellenőrző kérdés

Magyarázd el, miért nem elegendő önmagában a retry egy túlterhelt fizetési szolgáltatás hívásánál, és milyen további mintákat vetnél be, hogy a rendszer ellenálló maradjon. Válaszodban térj ki az idempotenciára, a circuit breaker szerepére és a bulkhead fontosságára.

20.2.18 Összefoglalás

Ebben a fejezetben a hibaizoláció és a túlterhelés megelőzésének négy alapvető mintáját tekintettük át. A **timeout** korlátot szab a várakozási időnek, a **retry** az átmeneti hibákra ad választ, a **circuit breaker** a tartósan hibás függőséget leválasztja, a **bulkhead** pedig az erőforrások elszigetelésével akadályozza meg a hiba terjedését. A minták önmagukban is hasznosak, de igazi erejük a kombinálásukban rejlik: egy jól tervezett hívási láncban a timeout, az idempotens retry, a jitterrel ellátott backoff, a circuit breaker és a bulkhead egymást erősítve védi a rendszert. A leggyakoribb hibákat – mint a nem idempotens retry, a jitter hiánya, a túl érzékeny circuit breaker vagy a bulkhead nélküli közös erőforrások – tudatos tervezéssel elkerülhetjük. A következő

fejezetben erre az alapra építve a chaos engineering és a játéknapok világába lépünk, ahol ezeket a védelmi mechanizmusokat szándékosan teszteljük.

20.3 Chaos engineering és játéknapok

A Circuit breaker, bulkhead, retry és timeout alfejezetben bemutatott védelmi minták – áramkörmegszakító, bulkhead, újrapróbálkozás, időkorlát – arra valók, hogy a rendszer hiba esetén is megőrizze a működőképességét. De honnan tudjuk, hogy ezek a minták valóban úgy viselkednek-e, ahogy terveztük? A tesztek csak az ismert hibákat fedik le, a komplex elosztott rendszerekben pedig a hibák ritkán jelentkeznek a kényelmes, előre megírt forgatókönyvek szerint. A chaos engineering és a game day a rendszer viselkedésének kísérleti vizsgálata: szabályozott, megfigyelt és biztonságos módon idézünk elő hibákat, hogy megértsük, mi történik valójában, és tanuljunk belőle. Ez a fejezet ezt a folyamatot építi fel az alapfogalmaktól a gyakorlati megvalósításig.

20.3.1 A rendszer viselkedésének kísérleti vizsgálata

A klasszikus szoftvertesztelés elsősorban arra a kérdésre keresi a választ, hogy a rendszer megfelel-e a specifikációnak. Funkcionális tesztet írunk egy adott bemenetre, elvárjuk az adott kimenetet, és ha a kettő egyezik, a teszt zöld. A reziliencia szempontjából azonban a legfontosabb kérdések nem a boldog útra vonatkoznak, hanem arra, hogyan viselkedik a rendszer akkor, amikor a környezet vagy egy függő komponens meghibásodik. A hagyományos tesztelés ezt csak korlátozottan tudja kezelni, mert a lehetséges hibák tere hatalmas, és a teszteket emberek találják ki, akik hajlamosak a saját feltételezéseiket beépíteni.

A chaos engineering ezt a megközelítést fordítja meg: nem azt kérdezi, hogy a rendszer átmegy-e egy előre meghatározott teszten, hanem azt, hogy megőrzi-e az elfogadható viselkedést akkor is, ha valami váratlan történik. Ennek eszköze a kísérlet: tudatosan hibát injektálunk egy komponensbe, és megfigyeljük a rendszer egészének válaszát. A cél nem a rendszer megtörése, hanem a rejtett függőségek, a gyenge pontok és a viselkedési anomáliák feltárása. A chaos engineering tehát a rendszer viselkedésének kísérleti vizsgálata: olyan szabályozott kísérletek sorozata, amelyek során a rendszer valamely részében hibát idézünk elő, hogy ellenőrizzük, a rendszer egésze az elvárásainknak megfelelően reagál-e.

Fontos különbség a chaos engineering és a hagyományos hibatesztelés között, hogy a chaos kísérlet elsősorban a rendszer viselkedésére kérdez rá, nem pedig egy adott komponens hibájára. Amikor például a fizetési szolgáltatást 500 ms-os válaszidőre lassítjuk, nem azt kérdezzük, hogy a fizetési szolgáltatás maga hogyan viselkedik, hanem azt, hogy a rendelésfelvétel, a kosár, az értesítések és a felhasználói felület hogyan reagál erre a lassulásra. Ez a rendszerszintű nézőpont az, ami a chaos engineeringet megkülönbözteti a komponensteszttől.

A módszer tudományos alapokra épül: felteszünk egy hipotézist, megtervezzük a kísérletet, végrehajtjuk, adatokat gyűjtünk, elemezzük az eredményt, és levonjuk a következtetést. A folyamat iteratív, és minden kísérlet tanulással zárul – akár igazolja a hipotézist, akár cáfolja. A tanulás a lényeg: a chaos engineering nem egy egyszeri teszt, hanem folyamatos gyakorlat, amely a rendszerbe vetett bizalmat növeli azáltal, hogy a hibákat akkor tárja fel, amikor még nem okoznak valós üzemzavart.

20.3.2 Hipotézis, steady state és mérés

Minden chaos kísérlet egy hipotézissel kezdődik. A hipotézis nem egyszerűen annyi, hogy „a rendszer működni fog”, mert ez nem cáfolható, és nem ad eligazítást a méréshez. A jó hipotézis megfogalmazza, hogy a rendszer normál állapotban hogyan viselkedik, és mi az az elfogadható eltérés, amelyet a hiba bevezetése után is elvárunk. Ezt a normál állapotot steady state-nek nevezzük: azok a mérhető tulajdonságok, amelyek a rendszer egészséges működését jellemzik.

A steady state meghatározásához szükség van a rendszer szolgáltatási szintű mutatóira (SLI-kre), amelyeket a minőségi forgatókönyvekben is használunk. Ilyen lehet például a rendelésfelvétel átlagos válaszideje, a sikeres kérések aránya, a hibás kérések száma, az áteresztőképesség vagy a rendelkezésre állás. A steady state nem feltétlenül konstans: lehet napszakos vagy terhelésfüggő mintázata, de a kísérlet tervezésekor rögzíteni kell, hogy mit tekintünk normál állapotnak az adott környezetben és az adott mérési ablakban.

A hipotézist quality attribute scenario formában célszerű megfogalmazni, ahogyan azt a Quality attribute scenarios alfejezetben láttuk. Például:

- **Inger:** a fizetési szolgáltatás válaszideje 500 ms-ra nő (normál esetben 80 ms).
- **Környezet:** a webshop termelési rendszere, csúcsidőn kívüli, alacsony forgalmú ablakban.
- **Válasz:** a rendelésfelvétel elérhető marad, a felhasználói hibaarány nem emelkedik 1% fölé, és az áramkör-megszakító legfeljebb 3 másodpercen belül nyit.
- **Mérőszám:** a rendelésfelvétel 99,9%-os sikeres válaszaránya 5 perces ablakban, a hibaarány 5 perces ablakban, az áramkör-megszakító állapotváltásának ideje.

Egy ilyen forgatókönyv egyértelműen megadja, mit mérünk, milyen küszöbértéket várunk, és mi számít elfogadható válasznak. A hipotézis pedig az, hogy a rendszer a megadott választ produkálja az inger hatására. Ha a mért érték eltér a várttól, a hipotézist cáfoltuk, és ez a tanulási pont: a rendszer nem úgy viselkedik, ahogy terveztük, tehát valamilyen rejtett hiba vagy hiányzó védelmi mechanizmus van.

A hipotézis megfogalmazásakor kerüljük a kétértelmű állításokat. A „a rendszer továbbra is gyors marad” nem mérhető, mert a „gyors” szubjektív. Ehelyett adjunk meg konkrét mérőszámot és küszöbértéket. A mérőszámokhoz mindig meg kell adni a mértékegységet és a mérési ablakot, ahogyan a könyv konvenciói előírják. Például: „a rendelésfelvétel 95. percentilis válaszideje 200 ms alatt marad a kísérlet 10 perces időtartama alatt”.

20.3.3 Blast radius: a kísérlet hatóköre

A chaos kísérlet nem azt jelenti, hogy véletlenszerűen szétzúzzuk a termelési rendszert. A kísérlet hatókörét úgy kell szűkíteni, hogy a várható hatás mérhető legyen, de ne okozzon valós kárt a felhasználóknak vagy az üzletnek. Ezt a hatókört nevezzük blast radiusnak, azaz a robbanás sugarának. A blast radius meghatározása során arra a kérdésre válaszolunk: ha a kísérlet rosszul sül el, mekkora része a rendszernek és hány felhasználónak lehet kitéve a hiba hatásának?

A blast radius csökkentésének több eszköze van:

- **Környezetválasztás:** a kísérletet futtathatjuk staging vagy tesztkörnyezetben, ahol nincs valós forgalom. Ez a legbiztonságosabb, de nem mindig reprezentálja a termelési feltételeket.
- **Forgalom-szűkítés:** termelésben a kísérletet korlátozhatjuk a forgalom egy kis százalékára, például canary bevezetéssel vagy feature flag használatával. Ekkor a hibát csak a felhasználók egy kis csoportja tapasztalja.
- **Földrajzi vagy logikai izoláció:** egy adott régió, adatközpont vagy alrendszer kiválasztása, ahol a hiba begyűrűzése korlátozott.

- **Fokozatos növelés:** a kísérletet kicsiben kezdjük, és ha az eredmény megfelelő, növeljük a hatókört.

A blast radius tervezésekor mindig dokumentálni kell a legrosszabb esetet: mi történik, ha a hiba miatt a rendszer olyan állapotba kerül, amelyből nem tud magától felépülni? Ehhez vészleállító mechanizmus kell, például a hibainjektáló eszköz azonnali leállítása, a terhelés azonnali csökkentése, vagy a kísérletet lehetővé tevő feature flag azonnali kikapcsolása. A blast radius tehát nemcsak a hiba hatását, hanem a kísérlet visszavonhatóságát is magában foglalja.

A kísérlet tervezésekor mérlegelni kell a kockázatot és a tanulság értékét. Egy teljesen biztonságos, de a termeléstől eltérő környezetben futó kísérlet kevésbé reprezentatív, mint egy kis blast radiusú termelési kísérlet. A kettő közötti egyensúlyt a csapat és a kockázatvállalási hajlandóság határozza meg. Az érett chaos engineering gyakorlatban a termelési kísérletek kis blast radiusszal indulnak, és a tanulságok alapján fokozatosan bővülnek.

20.3.4 Megfigyelés: a kísérlet szemei

A chaos kísérlet csak akkor ér valamit, ha a rendszer válaszát pontosan meg tudjuk figyelni. A megfigyelhetőség (observability) előfeltétele a chaos engineeringnek: ha nincs megfelelő log, metrika és trace, akkor a kísérlet csak egy fekete doboz, amelyből nem tudunk tanulni. A megfigyelés három pillére:

- **Metrikák:** időben aggregált mérőszámok, például kérések száma, hibák aránya, válaszidő percentilisek, erőforrás-felhasználás. A steady state SLI-it itt mérjük.
- **Logok:** az egyes események részletes naplói, amelyek a hiba kontextusát adják meg.
- **Trace-ek:** az elosztott rendszeren átívelő kérések útjának követése, amely megmutatja, hogy a hiba mely komponenseken keresztül terjed.

A kísérlet tervezésekor meg kell határozni, hogy mely metrikákat, logokat és trace-eket gyűjtjük, milyen gyakorisággal, és hol tároljuk őket. A mérési adatokat a kísérlet előtti, alatti és utáni időszakból is rögzíteni kell, hogy a változást ki tudjuk mutatni. A kísérlet előtti időszak a steady state alapvonalának megállapítására szolgál, a kísérlet alatti időszak a hiba hatását mutatja, a kísérlet utáni időszak pedig a regenerálódást és a visszaállást.

A megfigyelésnek összhangban kell lennie a hipotézisben megfogalmazott mérőszámokkal. Ha a hipotézis a hibaarányt és a válaszidőt vizsgálja, akkor ezeket a metrikákat kell gyűjteni, és megfelelő dashboardon vagy jegyzetfüzetben ábrázolni. A kísérlet során a megfigyelő személynek valós időben látnia kell a rendszer állapotát, hogy időben be tudjon avatkozni, ha a blast radius váratlanul növekszik.

20.3.5 Biztonságos kísérlet és tanulás

A chaos engineering alapelve, hogy a kísérlet soha nem okozhat valós kárt. Ez nemcsak technikai, hanem szervezeti és etikai követelmény is. A biztonságos kísérlet feltételei:

1. **Jóváhagyás:** a kísérletet előre egyeztetni kell az érintett csapatokkal, különösen a termelési kísérletnél. A kísérlet ne érjen senkit váratlanul.
2. **Fokozatosság:** a kísérletet kicsiben kezdjük, és csak akkor növeljük a blast radiust, ha az eredmény megfelelő.
3. **Vészleállító:** minden kísérletnek legyen egyértelmű módja a hibainjektálás azonnali leállítására, és a rendszer eredeti állapotának visszaállítására.

4. **Felügyelet:** a kísérletet emberi felügyelet mellett futtatjuk, aki folyamatosan figyeli a rendszer állapotát.
5. **Elkülönítés:** a kísérlet ne érintse a valós üzleti folyamatokat olyan mértékben, hogy az ügyfelek számára érezhető legyen, hacsak nem ez a cél, és akkor is kontrollált módon.
6. **Kommunikáció:** a kísérlet előtt, alatt és után tájékoztatni kell az érintetteket.

A tanulás a kísérlet legfontosabb kimenete. A tanulságot dokumentálni kell, és vissza kell vezetni a rendszer fejlesztésébe. Ha a hipotézist cáfoltuk, akkor meg kell találni az okot: miért nem úgy viselkedett a rendszer, ahogy vártuk? Hiányzott egy áramkör-megszakító? Rosszul volt beállítva a timeout? A bulkhead nem volt elég erős? A tanulság alapján új feladatot hozunk létre, javítjuk a rendszert, majd újabb kísérlettel ellenőrizzük a javítást. Így a chaos engineering a folyamatos fejlesztés része lesz, nem pedig egyszeri teszt.

20.3.6 Game day: a szimulált incidens

A game day (játéknap) a chaos engineering szervezeti kiterjesztése. Míg a chaos kísérlet elsősorban a rendszer technikai viselkedését vizsgálja, a game day a teljes szervezet válaszadását teszteli egy szimulált incidens során. A game day célja, hogy a csapat nyomás alatt gyakorolja az együttműködést, a kommunikációt, a döntéshozatalt és a runbookok használatát.

A game day során a résztvevők egy megírt forgatókönyv szerint dolgoznak. A forgatókönyv egy valósághű incidens, például: „a fizetési szolgáltatás 500 ms-ra lassult, és a rendelések 5%-a sikertelen”. A csapatnak meg kell állapítania, mi történik, be kell avatkoznia a runbookok szerint, kommunikálnia kell az érintettekkel, és végül helyre kell állítania a normál működést. A game day nem feltétlenül jár valós hibainjektálással: lehet asztali gyakorlat, ahol a résztvevők csak beszélik meg a lépéseket, de lehet éles gyakorlat is, ahol chaos kísérletet futtatnak, és a csapatnak reagálnia kell.

A game day felépítése:

- **Célkitűzés:** mit akarunk gyakorolni? Például egy adott runbook érvényességét, a kommunikációs láncot, a döntési jogköröket.
- **Forgatókönyv:** a szimulált incidens leírása, a bevezető tünetek, az érintett komponensek, a várható hatás.
- **Szerepek:** ki a parancsnok, ki a kommunikátor, ki a technikai vizsgáló, ki a döntéshozó. A szerepek gyakran az incidenskezelési folyamatból származnak.
- **Időkeret:** a gyakorlat időtartama, általában 1-2 óra, de lehet hosszabb is.
- **Eszközök:** megfigyelő dashboardok, chat csatorna, videokonferencia, közös dokumentum.
- **Megfigyelők:** olyan személyek, akik nem vesznek részt a megoldásban, hanem jegyzik a csapat viselkedését, a döntéseket és a kommunikációt.
- **Utólagos elemzés:** a gyakorlat után a csapat áttekinti, mi történt, mit csináltak jól, mit lehetne javítani.

A game day különösen fontos a szervezeti reziliencia szempontjából, mert a technikai rendszer csak annyira megbízható, amennyire az azt üzemeltető emberek képesek reagálni. Egy jól dokumentált runbook semmit sem ér, ha a csapat nem tudja, hogyan kell használni, vagy ha a kommunikációs csatornák összeomlanak a nyomás alatt. A game day ezeket a rejtett szervezeti hibákat tárja fel, még mielőtt valós incidens következne be.

20.3.7 Tipikus hibák és helyes megoldások

A chaos engineering és a game day során számos tipikus hiba fordul elő. Ezek közül a leggyakoribbak:

1. **Nincs hipotézis.** Ha a kísérletet csak azért futtatjuk, hogy „lássuk, mi történik”, az eredmény nehezen értelmezhető, és a tanulság sem konkrét. Megoldás: minden kísérlethez fogalmazzunk meg mérhető, cáfolható hipotézist.
2. **Túl nagy blast radius.** A kísérlet egyszerre túl sok komponenst érint, vagy túl nagy forgalmi arányt céloz, ezért a hiba hatása valós üzemzavart okoz. Megoldás: fokozatosan növeljük a hatókört, és mindig legyen vészleállító.
3. **Rossz megfigyelés.** A kísérlethez nincs megfelelő mérőeszköz, ezért nem tudjuk megállapítani, hogy a rendszer jól vagy rosszul viselkedett. Megoldás: a kísérlet előtt ellenőrizzük, hogy a szükséges metrikák, logok és trace-ek rendelkezésre állnak, és a dashboardok jól mutatják a steady state-et.
4. **A kísérlet nem reprodukálható.** Az eredmények nem dokumentáltak, a konfiguráció nem verziózott, ezért később nem tudjuk megismételni a kísérletet. Megoldás: a kísérletet diagrams-as-code vagy konfigurációkezelő eszközben, verziózottan tároljuk, és a futást naplózzuk.
5. **A game day csak technikai gyakorlat.** A hangsúly a hibainjektáláson van, de a kommunikációt, a döntési folyamatot és a runbookok használatát nem gyakorolják. Megoldás: a forgatókönyv terjedjen ki az emberi tényezőre is, és legyenek megfigyelők, akik a szervezeti válaszadást értékelik.
6. **A tanulság nem jut el a fejlesztésig.** A kísérlet feltárt egy hibát, de nem születik belőle feladat, vagy a feladatot nem priorizálják. Megoldás: a kísérlet eredményét ADR-ben vagy döntési naplóban rögzítjük, és a javítást beillesztjük a tervezési folyamatba.
7. **A kísérletet termelési döntésnek tekintik.** A PoC-hez hasonlóan a chaos kísérlet is tanulásra való, nem annak bizonyítására, hogy a rendszer kész. Ha a hipotézis igazolódik, az növeli a bizalmat, de nem garantálja, hogy a rendszer minden jövőbeli hibát elvisel.

Ezeket a hibákat elkerülve a chaos engineering és a game day hatékony eszköz a reziliencia növelésére.

20.3.8 Labor: chaos-kísérlet és game day terve

A labor célja, hogy a fejezetben tanult fogalmakat egy kis, ellenőrizhető példán alkalmazzuk. A feladat egy chaos-kísérlet és egy hozzá kapcsolódó game day tervének elkészítése egy fiktív webshop rendszerre. A labor nem igényel valós rendszert; a hangsúly a tervezésen, a hipotézis és a mérési terv kidolgozásán van.

Kiinduló állapot: A webshop három fő komponensből áll: rendelésfelvétel (Order Service), fizetési szolgáltatás (Payment Service) és készletkezelés (Inventory Service). A rendelésfelvétel szinkron módon hívja a fizetési szolgáltatást, és a hívásra 800 ms-os timeout van beállítva. A fizetési szolgáltatás külső szolgáltatóhoz kapcsolódik, és a válaszideje normál esetben 80 ms körül van. A rendszerben van áramkör-megszakító a fizetési hívásra, a hibaküszöb 50%, az újrapróbálkozás 2-szeres exponenciális visszavárással, a bulkhead pedig 10 párhuzamos hívásra korlátozza a fizetési szolgáltatást.

Elvárt eredmény: A hallgató készítsen egy dokumentumot (például Markdown vagy jegyzetfüzet), amely a következőket tartalmazza:

1. **Kontextus:** a rendszer rövid leírása, a kísérlet környezete (pl. staging, alacsony forgalom), a mérési ablak és a kísérlet időpontja.
2. **Hipotézis:** quality attribute scenario formában, a steady state SLI-kkel.
3. **Kísérleti terv:** a hiba típusa (pl. latency injection a fizetési szolgáltatásra), a blast radius meghatározása, a fokozatos bevezetés lépései, a vészleállító.
4. **Megfigyelési terv:** milyen metrikákat, logokat és trace-eket gyűjtünk, milyen dashboardon jelenítjük meg, milyen küszöbértékeket figyelünk.
5. **Döntési napló:** ADR formátumban rögzítsük a kísérlet megtervezésének legfontosabb döntéseit, például a blast radius választását, az eszközt, a környezetet.
6. **Ellenőrzési szempontok:** milyen kritériumok alapján tekintjük a kísérletet sikeresnek vagy sikertelennek, és hogyan választjuk el a mért és a várt eredményt.
7. **Visszaállítási út:** hogyan állítjuk vissza a rendszert a kísérlet után.

A labor második része egy game day terv: a fenti kísérlethez kapcsolódóan készítsünk forgatókönyvet, amelyben a csapatnak reagálnia kell a fizetési szolgáltatás lassulására. Adjuk meg a szerepeket, a kommunikációs csatornákat, a döntési pontokat és az utólagos elemzés szempontjait.

Példa a hipotézisre:

- **Inger:** a fizetési szolgáltatás válaszideje 500 ms-ra nő.
- **Környezet:** staging környezet, a webshop alacsony forgalmú időszaka, 10 perces mérési ablak.
- **Válasz:** a rendelésfelvétel elérhető marad, a sikeres rendelések aránya nem csökken 95% alá, az áramkör-megszakító 5 másodpercen belül nyit, és a hibaarány a rendelésfelvételen 1% alatt marad.
- **Mérőszám:** rendelésfelvétel sikeres válaszainak aránya, hibás válaszok aránya, áramkör-megszakító állapota, mind 10 perces ablakban mérve.

Döntési napló (ADR minta):

- **Cím:** Chaos kísérlet a fizetési szolgáltatás 500 ms-os lassítására
- **Kontextus:** A rendszer rendelkezik áramkör-megszakítóval és timeouttal, de nem teszteltük, hogy ezek együttesen hogyan viselkednek valós lassuláskor. A cél a rendelésfelvétel válaszának megfigyelése.
- **Döntés:** A kísérletet staging környezetben futtatjuk, a blast radius a fizetési szolgáltatás felé irányuló hívások 10%-át érinti. A hiba típusa latency injection 500 ms-ra. A vészleállító a hibainjektáló eszköz azonnali leállítása.
- **Alternatívák:** (1) termelési kísérlet canary forgalommal – nagyobb reprezentativitás, de nagyobb kockázat; (2) teljes kiesés szimulálása – erősebb teszt, de a tanulási cél a lassulás, nem a kiesés.
- **Következmények:** A staging környezetben a forgalmi minták eltérnek a termeléstől, ezért az eredményt nem lehet közvetlenül általánosítani. A 10%-os blast radius elegendő a hatás megfigyelésére, de nem okoz teljes leállást.
- **Állapot:** Jóváhagyva.

Ellenőrzési szempontok:

- A hipotézis megfogalmazása egyértelmű, mérhető, és tartalmazza az ingert, a környezetet, a választ és a mérőszámot.
- A blast radius meghatározott, és a vészleállító dokumentált.

- A megfigyelési tervben szerepelnek a konkrét metrikák és azok mérési ablaka.
- A dokumentum elkülöníti a várt eredményt a majdani mért eredménytől (a laborban a mért eredményt nem tudjuk előre kitölteni, de a dokumentumtervben helyet kell hagyni neki).
- A game day forgatókönyv tartalmazza a szerepeket, a kommunikációs csatornákat és az utólagos elemzést.

Visszaállítási út: A kísérlet után a hibainjektálást leállítjuk, a rendszert megfigyeljük további 5 percig, hogy a normál állapot visszaáll-e. Ha a rendszer nem regenerálódik, az áramkör-megszakítót manuálisan zárjuk, és a szolgáltatást újraindítjuk a dokumentált runbook szerint.

A labor eredményét a csapat közösen átnézi, és a tanulságokat beépíti a fejlesztési folyamatba.

20.3.9 Ellenőrzőlista

- A kísérlethez van-e cáfolható, mérhető hipotézis?
- A hipotézis tartalmazza-e az ingert, a környezetet, a választ és a mérőszámot?
- Meghatároztuk-e a steady state-et és annak SLI-it?
- A blast radius szűkített és dokumentált?
- Van-e vészleállító mechanizmus?
- A megfigyelési eszközök (metrikák, logok, trace-ek) rendelkezésre állnak-e a kísérlet előtt?
- A kísérlet reprodukálható-e a dokumentáció alapján?
- A game day forgatókönyv lefedi-e a technikai és a szervezeti válaszadást?
- A tanulságokat visszavezettük-e a fejlesztési feladatok közé?
- A kísérlet nem okoz-e valós kárt a felhasználóknak vagy az üzletnek?

20.3.10 Önellenőrző kérdés

Mi a különbség a chaos kísérlet és a game day között, és miért fontos mindkettő a reziliencia növeléséhez? Válaszodat egy konkrét példán mutasd be: hogyan terveznéd meg egy kísérletet a fizetési szolgáltatás lassulására, és milyen game day elemeket adnál hozzá, hogy a szervezeti válaszadást is teszteld?

20.3.11 Összefoglalás

Ez a fejezet a chaos engineering és a game day alapjait mutatta be. A chaos engineering a rendszer viselkedésének kísérleti vizsgálata, amely hipotézissel, blast radius-szal és megfigyeléssel dolgozik, és a tanulást helyezi a középpontba. A game day ezt a megközelítést szervezeti szintre emeli, és a csapat válaszadását teszteli szimulált incidenssel. A következő fejezet (SLO, error budget és incidenskezelés) az itt bevezetett fogalmakra épít: az SLO-k, az error budget és az incidenskezelés rendszere nélkül a chaos kísérletek eredményei nem fordíthatók le üzemeltetési döntésekre. A mérési és megfigyelési alapok nélkül pedig az error budget sem menedzselhető. A chaos engineering tehát nem elszigetelt gyakorlat, hanem a reziliencia és az üzemeltetés folyamatos tanulási ciklusának egyik motorja.

20.4 SLO, error budget és incidenskezelés

A megbízhatóság mérése nem öncélú adminisztráció, hanem olyan visszacsatolási hurok, amely összeköti az architektúra tervezési döntéseit a termelési viselkedéssel. A hibamódok elemzése (lásd a megelőző fejezetben) megmutatja, mi romolhat el; az itt tárgyalt eszközök azt teszik mérhetővé, hogy a romlás milyen gyakran és milyen mértékben érinti a felhasználót, és mikor kell az üzemeltetésnek, a fejlesztésnek vagy a termékmenedzsmentnek beavatkoznia. A fejezet a Google Site Reliability Engineering (SRE) gyakorlatában elterjedt fogalomrendszerre épít, de a fogalmakat az általános szoftverarchitektúra nyelvén tárgyalja, és a helyi szervezeti kontextustól függetlenül alkalmazható mintákat ad.

20.4.1 1. A megbízhatóság mérése: SLI, SLO, SLA és error budget

20.4.2 1.1. SLI: a szolgáltatás megfigyelhető tulajdonsága

Az **SLI (Service Level Indicator)** egy szolgáltatás valamely viselkedésének mérhető, lehetőleg objektíven származtatható mutatója. A leggyakoribb SLI-k:

- **Rendelkezésre állás (availability):** a sikeresen megválaszolt – vagy legalábbis nem hibás – kérések aránya az összes releváns kéréshez képest. Például `200–399` HTTP státuszú válaszok részaránya. A rendelkezésre állás számítható időarányosan is: a szolgáltatás mennyi ideig működik az adott mérési ablakban.
- **Késleltetés (latency):** a kérés és a válasz között eltelt idő. Leggyakrabban percentilisként adjuk meg, például `p95`, `p99`. A percentilis elengedhetetlen, mert a késleltetés eloszlása általában erősen ferde: az átlag mögé hosszú farok bújhat.
- **Hibaarány (error rate):** a sikertelen kérések aránya. Itt meg kell határozni, mi számít hibának: csak az `5xx`, vagy a `4xx` bizonyos osztályai is? Az üzleti hibák (például „nincs készlet”) gyakran nem számítanak szolgáltatási hibának, míg a szerződésszegés igen.
- **Telítettség (saturation):** egy erőforrás kihasználtsága, például CPU, memória, kapcsolati pool vagy kérésvárakozási sor hossza. Ez előre jelzi a késleltetés romlását.

Az SLI nem maga az architektúra, hanem annak viselkedési lenyomata. A jó SLI nem az implementáció belső részleteit méri, hanem a felhasználó vagy a hívó szempontjából releváns kimenetet. Ha a számlálót és a nevezőt nem ugyanazon a szerveren, ugyanazzal a mintavétellel és ugyanabban az időablakban gyűjtjük, a mutató félrevezetővé válik.

Mérési ablak: minden SLI csak egy meghatározott időtartamon értelmezhető. A gyakori ablakok: 5 perc, 1 óra, 24 óra, 30 nap. A hosszabb ablak simítja a zajt, de késlelteti a riasztást; a rövidebb ablak érzékenyebb, de több hamis pozitív riasztást adhat. Az architektúrában dokumentálni kell a mérési ablakot, a mintavételi arányt és a mérés bizonytalanságát.

20.4.3 1.2. SLO: a teljesítménycél

Az **SLO (Service Level Objective)** egy SLI-hez rendelt célérték, kiegészítve a mérési ablakkal és azzal a feltétellel, hogy az ablak mekkora részében kell a célt teljesíteni. Például: „a `GET /orders` végpont `p95` késleltetése 400 ms alatt legyen a havi ablak 99{,}9%-ában”. Ez az SLO nem 30 nap alatt egyszeri átlagos p95-öt ír elő, hanem azt, hogy a havi ablakban az 5 perces mérési intervallumok 99{,}9%-ában a p95 400 ms alatt marad.

Néhány pontos megfogalmazású példa:

- **Rendelkezésre állás:** a `POST /orders` hívások sikeresek legyenek a havi ablak `99{,}95`%-ában. A mérési ablak 30 nap, a mintavétel 10 másodperces csúszó ablak.
- **Késleltetés:** a `GET /products` hívások `p99` késleltetése 600 ms alatt legyen a heti ablak `99{,}0`%-ában.
- **Hibaarány:** a `POST /payments` hívások hibaaránya 0{,}1% alatt legyen a napi ablakban.

Az SLO-kat **ne a jelenlegi teljesítmény alapján válasszuk meg**, hanem aszerint, hogy mi az a szint, amit a felhasználói élmény még elfogadhatónak tart, és amit a szervezet gazdaságilag fenntarthatóan tud szolgáltatni. A túl szigorú SLO (például `100%` rendelkezésre állás) nem érhető el, és felesleges túlműködést generál. A túl laza SLO nem ad jelzést az észrevehető romlásról.

Az SLO-kat **quality attribute scenario** formában is dokumentálhatjuk, hogy egyértelmű legyen az inger, a környezet, a válasz és a mérőszám. Példa:

> **Inger:** a `POST /orders` végpont terhelése 200 kérés/másodperc fölé emelkedik, és az adatbázis-válaszidő `p50` értéke 120 ms-ra nő.
> **Környezet:** normál termelési környezet, a Kubernetes klaszter 6 poddal fut, az adatbázis elsődleges csomópontja `db-primary` jelű.
> **Válasz:** a szolgáltatás a kérések 99%-át 500 ms alatt válaszolja meg, és a `POST /orders` hibaaránya nem haladja meg a 0{,}1%-ot a 5 perces csúszó ablakban.
> **Mérőszám:** `p99` válaszidő ms-ban, hibaarány százalékban, 5 perces ablak.

Az SLO-kat architektúra-döntésként is kezelhetjük. Egy ADR rögzítheti például a `99{,}95`%-os havi rendelkezésre állási cél indoklását és a feladott alternatívákat. Példa:

```
# ADR-019: A rendeléskezelő szolgáltatás rendelkezésre állási SLO-ja

Állapot: Elfogadva

Kontextus:
A rendeléskezelő szolgáltatás a webes értékesítés kritikus útjában
helyezkedik el. A hibás vagy lassú rendelés rontja a vásárlói
élményt, és közvetlen bevételkiesést okoz. Az előző havi mérések
szerint a szolgáltatás tényleges rendelkezésre állása 99{,}98% volt,
a hibák fő oka az adatbázis-kapcsolati pool kimerülése.

Döntés:
A rendelkezésre állás SLI: a `POST /orders` és `GET /orders/{id}`
hívások sikeres részaránya. A havi mérési ablak 30 nap, csúszó
5 perces intervallumokban. Az SLO cél: 99{,}95%.

Alternatívák:
1. 99{,}99% - erősebb garancia, de kétszeres rendelkezésre állási
   hibabüdzsé, további replikációs és feladatátvételi költség.
2. 99{,}9% - lazább cél, kevesebb fejlesztési erőforrás, de a
   hibák észrevehetővé válnának a vásárlók számára.
3. Csak a `POST /orders` mérése - egyszerűbb, de nem fedi le a
   rendelés visszaolvasásának hibáit.

Következmények:
A csapatnak a havi error budgetet 0{,}05% alatt kell tartania.
A riasztások a burn rate-re épülnek. A release freeze-eket az
error budget kimerülése váltja ki.
```

20.4.4 1.3. SLA: jogi szerződés, nem mérnöki eszköz

Az **SLA (Service Level Agreement)** az SLO jogi vagy szerződéses formája. Az SLO-t a szervezet maga határozza meg, elsősorban mérnöki és termékcélként; az SLA a külső félnek

tett ígéret, amely pénzügyi vagy szerződéses következményekkel jár, ha nem teljesül. Az SLO-nak szigorúbbnak vagy legalább egyenlőnek kell lennie az SLA-val, mert a belső beavatkozási küszöbnek meg kell előznie a szerződésszegést. Ha az SLA 99{,}9%-os rendelkezésre állást ígér, az SLO ésszerűen 99{,}95% lehet, így a csapatnak van mozgástere a hiba kezelésére.

20.4.5 1.4. Error budget: a megbízhatóság gazdasági kerete

Az **error budget** a megengedett hibák mennyisége az adott mérési ablakban. Matematikailag:

$$\text{error budget} = 1 - \text{SLO}$$

Ha az SLO 99{,}95% rendelkezésre állás, akkor a havi error budget 0{,}05%, vagyis a kérések legfeljebb 0{,}05%-a lehet sikertelen, illetve a szolgáltatás legfeljebb a havi idő 0{,}05%-ában lehet hibás. Ez az érték nem puszta szám, hanem **kockázati költségvetés**: lehetőség arra, hogy a csapat vállaljon bizonyos működési bizonytalanságot, például gyorsabb release-t, kevésbé konzervatív infrastruktúra-változtatást vagy tervezett karbantartást, mindaddig, amíg a hiba nem emészti fel a keretet.

Az architektúra szempontjából az error budget egy **döntési mechanizmust** hoz létre:

- Ha a havi error budget még nagyrészt rendelkezésre áll, a szervezet megengedhet magának nagyobb kockázatú változtatásokat: új szolgáltatásverzió, adatbázis-migráció, kísérleti beállítás.
- Ha a budget közelít a kimerüléshez, a termékmenedzsmentnek **feature freeze-t** kell elrendelnie, a fejlesztésnek pedig a megbízhatóság javítására kell összpontosítania.
- Ha a budget elfogyott, az SLO megszegése már bekövetkezett, és a szervezetnek döntenie kell: vagy csökkenti az SLO-t, vagy vállalja a szerződésszegést.

A hiba nem egyenletesen oszlik el: egyetlen 5 perces teljes kiesés a havi 0{,}05%-os budget jelentős részét felemésztheti. Ezért az error budget mellett a **burn rate** (fogyási ráta) ad időbeli kontextust.

20.4.6 2. Burn rate és SLO-alapú riasztás

20.4.7 2.1. Burn rate: mennyire gyorsan fogy a budget

A **burn rate** az error budget fogyásának sebessége az SLO-hoz képest. Ha a szolgáltatás pontosan az SLO-nak megfelelően hibázik, a burn rate értéke 1. Ha kétszer olyan gyorsan fogy a hibabüdzsé, a burn rate 2. A burn rate a következőképpen számítható:

$$\text{burn rate} = \frac{\text{tényleges hibaarány az ablakban}}{1 - \text{SLO}}$$

Például 99{,}95%-os SLO esetén az error budget 0{,}05% = 0,0005. Ha egy 1 órás ablakban a tényleges hibaarány 0{,}25%, akkor:

$$\text{burn rate} = \frac{0{,}0025}{0{,}0005} = 5$$

Ez azt jelenti, hogy a szolgáltatás az adott órában ötszörös sebességgel fogyasztja a havi hibabüdzsét. Ha ez a sebesség fennmaradna, a havi budget nem 30 nap alatt, hanem 6 nap alatt fogyna el.

A burn rate nemcsak a rendelkezésre állásra, hanem a késleltetésre is értelmezhető: ha a p99 késleltetés túllépése az SLO szerint az idő 0{,}1%-ában megengedett, de a tényleges túllépés 0{,}5%, akkor a burn rate szintén 5.

20.4.8 2.2. Többablakos, több-burn-rate riasztás

A burn rate-et a riasztási szabályokban érdemes több időablakkal kombinálni, hogy elkerüljük a két szélsőséget: a túlzottan érzékeny, zajos riasztást és a túl késői, a hibabüdzsé elfogyásakor megszólaló értesítést. A Google SRE-ben elterjedt minta a **multi-window, multi-burn-rate alerting**:

- **Gyors, rövid ablak:** például 5 perc alatt 5-szörös burn rate. Ez gyorsan jelez, ha egy friss release vagy konfigurációváltozás azonnali hibát okoz.
- **Lassú, hosszú ablak:** például 6 óra alatt 1-es burn rate. Ez akkor szólal meg, ha a szolgáltatás folyamatosan, alacsonyabb szinten, de tartósan romlik, és a havi budget felélése várható.
- **Nagyon lassú, nagyon hosszú ablak:** például 3 nap alatt 0{,}5-ös burn rate. Ez a lassú, de állandó degradációt jelzi, amely hetek alatt még az SLO megszegéséhez vezethet.

Egy tipikus szabály egy 99{,}95%-os SLO-val rendelkező szolgáltatásra:

Ablak	Burn rate küszöb	Megjegyzés
5 perc	5	Azonnali riasztás súlyos eseményre
6 óra	2	Tartós, gyors fogyás észlelése
3 nap	0{,}5	Lassú degradáció hosszú távú jelzése

A riasztásnak mindig tartalmaznia kell a **fogyási kontextust**: hány százalékos az error budget a riasztás pillanatában, mekkora a várható kimerülési idő, és melyik SLO-t érinti. Így az ügyeletes mérnök nemcsak azt tudja, hogy valami történt, hanem azt is, hogy milyen gyorsan kell cselekednie.

A riasztási küszöbök beállítása trade-off. A túlérzékeny riasztás **alert fatigue**-hez vezet: a mérnökök megszokják a hamis jelzéseket, és egy valódi incidensnél későn vagy egyáltalán nem reagálnak. A túlzottan konzervatív küszöb viszont azt eredményezi, hogy a csapat csak akkor értesül, amikor a hiba már elérte a felhasználót, és a hibabüdzsé elfogyott.

20.4.9 2.3. A hibabüdzsé mint vezérlő mechanizmus

A burn rate riasztások a hibabüdzsét a **release- és üzemeltetési politika** részévé teszik. Az alábbi állapotgép jól mutatja a döntési logikát:

```
flowchart TD
    A[SLI mérés folyamatosan] --> B{Burn rate túllép}
    B -- Nem --> C[Normál működés,\nrelease folytatódik]
    B -- Igen --> D[Triage és eszkaláció]
    D --> E{Error budget\nkimerült?}
    E -- Nem --> F[Release freeze,\nhibajavítás prioritás]
    E -- Igen --> G[Incident response,\nSLO felülvizsgálat]
    G --> H[Postmortem\nés javító intézkedések]
```

Ez a folyamat egyértelművé teszi, hogy a hiba nem kizárólag az üzemeltetés felelőssége. Az error budget fogyása a **termékmenedzsmentet** is beavatkozásra készteti: ha a csapat a hónapban már felhasználta a hibakeretet, a további funkciókiadások kockázatosak, és a prioritást a stabilitásra kell helyezni.

20.4.10 3. Észlelés, eszkaláció és utóelemzés

20.4.11 3.1. Észlelés: a jel és a zaj szétválasztása

A megbízhatóság mérése csak akkor ér valamit, ha a **észlelés** időben, megbízhatóan és magyarázhatóan történik. A modern rendszerekben a megfigyelhetőség három alappillére:

- **Logok:** diszkrét események, amelyek egy adott kérés vagy művelet kontextusában rögzítenek részleteket.
- **Metrikák:** időben aggregált mérőszámok, például kérési arány, hibaarány, késleltetés percentilisek.
- **Nyomkövetés (traces):** egy kérés útjának szétterített ábrázolása a szolgáltatásokon keresztül, amely a késleltetés forrásainak azonosítását segíti.

Az OpenTelemetry mint nyílt szabvány az infrastruktúrába ágyazott megfigyelhetőség alapja, de a fejezet nem tárgyalja a megfigyelhetőségi eszközök bevezetését (lásd a későbbi observability fejezetet). Itt a hangsúly azon van, hogy az SLI-ket **a riasztási rendszer bemeneteként** használjuk.

A jó riasztás jellemzői:

- **Emberi beavatkozást igényel.** Ha a rendszer képes magától korrigálni, akkor az automatizmusé a feladat, nem a riasztásé. A riasztásnak arra kell ösztönöznie, hogy valaki beavatkozzon.
- **Kontextust ad.** Tartalmazza a szolgáltatás nevét, az SLO-t, a burn rate-et, a hiba jellegét és a lehetséges első lépéseket.
- **Kapcsolódik a hibabüdzséhez.** Nem csupán egy küszöb túllépését jelzi, hanem a megbízhatósági célra gyakorolt hatást.

20.4.12 3.2. Eszkaláció: felelősség és kommunikáció

Amikor a riasztás megszólal, a **eszkalációs folyamat** biztosítja, hogy a megfelelő ember a megfelelő időben tudomást szerezzen és cselekedjen. A gyakorlatban ez több szinten történik:

1. **Elsődleges on-call (primary responder):** az a mérnök, aki az adott rotációban az éles riasztásokért felel. Feladata a triage: megállapítja, valódi incidensről van-e szó, felméri a hatást, és megkezdi a hibaelhárítást.
2. **Másodlagos on-call (secondary responder):** ha az elsődleges nem reagál, vagy az incidens túlnyúlik a képességein, a másodlagos lép be.
3. **Incident commander:** nagyobb incidensnél a műszaki munkát koordináló személy. Az ő feladata nem feltétlenül a hibajavítás, hanem a kommunikáció, a döntések nyilvántartása és a párhuzamos munkák összehangolása.
4. **Termékmenedzsment és üzleti kapcsolattartó:** ha az incidens a szolgáltatás elérhetőségét vagy adatbiztonságát érinti, az üzleti oldalnak is értesülnie kell.

Az eszkalációt nem szabad a bűnösség megállapításának pillanataként kezelni. A cél az, hogy a megfelelő kompetencia és döntési jogkör minél gyorsabban bevonódjon. A jól definiált rotációs rend, az ügyeleti dokumentáció és a runbookok az eszkaláció hatékonyságának alapfeltételei.

20.4.13 3.3. Utóelemzés: blameless postmortem, javító intézkedések

Az incidens lezárása után az **utóelemzés (postmortem)** a tanulás elsődleges eszköze. A poszt-mortem nem a hibás személy megkereséséről, hanem a **rendszer feltételeinek** feltárásáról szól. A **blameless** kultúra azt jelenti, hogy az elemzés a döntések minőségére, a környezetre, a folyamatokra és az architektúra viselkedésére fókuszál, nem egyetlen mérnök tévedésére. Az ilyen kultúra nélkül a mérnökök elrejtik a hibákat, és a szervezet elveszíti a tanulási lehetőséget.

A jó utóelemzés szerkezete:

- **Idővonal:** mi történt, mikor, milyen sorrendben. A pontos időbélyeg és az események láncolata segít az ok-okozat feltárásában.
- **Hatás:** milyen felhasználói, üzleti és műszaki következményekkel járt. Az SLI/SLO adatok itt az objektív hatás mértékét adják.
- **Gyökér okok:** a hozzájáruló tényezők, nem egyetlen „bűnös” megnevezése. Az elemzés eljuthat olyan mélységig, mint egy hibamód a tervezési feltételezésben, egy hiányzó circuit breaker vagy egy rosszul megválasztott timeout.
- **Javító intézkedések:** konkrét, felelőssel és határidővel ellátott feladatok, amelyek megakadályozzák az ismétlődést vagy csökkentik a hatást. Az intézkedéseknek mérhető céljuk kell legyen, és visszacsatolódniuk kell az SLI/SLO rendszerbe.
- **Döntési napló:** az incidens során hozott döntések rögzítése, például egy ADR-kiegészítés formájában.

Egy példa az utóelemzésből:

> **Incidens:** A `POST /orders` hibaaránya 14:32 és 14:58 között 2{,}5%-ra nőtt, a havi error budget 0{,}04%-ról 0{,}041%-ra nőtt, a burn rate 5 volt. A riasztás 14:33-kor érkezett.
> **Gyökér ok:** az adatbázis-kapcsolati pool maximális mérete 50-re volt állítva, de az új promóciós kampány forgalma 200 kérés/másodperc fölé emelkedett, és a pool kimerült. A timeout hiánya miatt a szálak 5 másodpercig blokkoltak.
> **Javító intézkedés:** a pool méretét 100-ra emeljük, a `acquireTimeout` értékét 500 ms-ra csökkentjük, és circuit breaker-t vezetünk be az adatbázis-elérésre. A módosítást a következő héten canary release-ben vezetjük be.

20.4.14 4. Gyakorlati minta: SLO-k, burn rate és incidensfolyamat egy rendeléskezelő szolgáltatásra

A következőkben egy kitalált, de valósághű rendszeren mutatjuk be, hogyan kapcsolódnak az egyes fogalmak. A rendszer neve **„Rendeléskezelő Szolgáltatás”**, amely a webes felületről fogad rendeléseket, és a fizetési szolgáltatással, valamint a termékraktárral kommunikál. A diagramok a C4 kontextusszintjét alkalmazzák, diagrams-as-code formában, például Structurizr DSL-ben vagy Mermaidban.

20.4.15 4.1. A rendszer kontextusa

```
flowchart LR
    User[Vásárló] -->|HTTPS| WebApp[Webes felület]
    WebApp -->|JSON/HTTP| OrderService[Rendeléskezelő szolgáltatás]
    OrderService -->|gRPC| PaymentService[Fizetési szolgáltatás]
    OrderService -->|PostgreSQL| OrderDB[(Rendelés adatbázis)]
    OrderService -->|Kafka| TrackingStream[Rendelés-events]
```

A komponensek interfészei:

- `WebApp` → `OrderService`: `POST /orders`, `GET /orders/{id}`, JSON, szinkron HTTP.
- `OrderService` → `PaymentService`: `CreatePayment`, gRPC, szinkron, timeout 1000 ms.
- `OrderService` → `OrderDB`: SQL, szinkron, connection pool 50–100, timeout 500 ms.
- `OrderService` → `TrackingStream`: eseménypublikálás Kafka topicra, aszinkron, at-least-once.

A rendszerhatár a webes felülettől és a külső szolgáltatásoktól egyértelműen elválasztott.

20.4.16 4.2. SLI és SLO táblázat

Szolgáltatás-művelet	SLI	Mérési ablak	SLO cél
`POST /orders`	sikeres HTTP válaszok aránya	30 nap, 5 perces csúszó ablak	99{,}95%
`POST /orders`	`p99` késleltetés	30 nap, 5 perces csúszó ablak	500 ms alatt 99%-ban
`GET /orders/{id}`	sikeres HTTP válaszok aránya	30 nap, 5 perces csúszó ablak	99{,}9%
`GET /orders/{id}`	`p95` késleltetés	30 nap, 5 perces csúszó ablak	300 ms alatt 95%-ban

A mérési ablak és a mintavétel rögzített; a bizonytalanságot a metrika-rendszer szolgáltatja.

20.4.17 4.3. Error budget és burn rate riasztások

A havi error budget a `POST /orders` rendelkezésre állási SLO-jára: `1 - 0{,}9995 = 0{,}0005`, azaz `0{,}05`%, ami 30 nap alatt legfeljebb körülbelül 21,6 másodpercnyi hibaidőt jelent, ha a szolgáltatás átlagosan 100 kérést szolgál ki másodpercenként. Ez a számítás azonban nem pontos, mert a rendelkezésre állás kérésarányos; inkább a hibakérések arányában érdemes gondolkodni.

A riasztási szabályok:

Szabály neve	Ablak	Burn rate	Hibaarány küszöb
`orders-availability-fast`	5 perc	5	0{,}25%
`orders-availability-slow`	6 óra	2	0{,}1%
`orders-availability-very-slow`	3 nap	0{,}5	0{,}025%

A riasztások a metrika-rendszerbe kerülnek, és az on-call rotációt értesítik. A dashboardon a jelenlegi havi error budget százalékos állapota jelenik meg.

20.4.18 4.4. Incidensforgatókönyv

Az alábbi forgatókönyv egy lehetséges incidenst mutat be, a fent definiált rendszeren:

- **14:31:** a `POST /orders` hibaaránya 0{,}3%-ra emelkedik, 5 perces ablakban. A `orders-availability-fast` riasztás megszólal.
- **14:32:** az elsődleges on-call mérnök megvizsgálja a dashboardot. A hibaarány növekedése a `PaymentService` felé irányuló hívásoknál koncentrálódik. A `PaymentService` p99 késleltetése 800 ms fölé emelkedett.
- **14:35:** a burn rate 5 fölött van. Az error budget 0{,}01%-a fogyott el.
- **14:36:** az on-call eszkalálja az incidenst a fizetési csapat ügyeletesének. Az incident commander átveszi a koordinációt.
- **14:40:** a fizetési csapat megállapítja, hogy a `PaymentService` adatbázis-lassulást észlel, és a connection pool kimerült.
- **14:50:** a `PaymentService` pool méretét ideiglenesen megemelik, a hibaarány normalizálódik.
- **14:58:** a `POST /orders` hibaaránya visszaáll 0{,}02% alá. Az incidens lezárásra kerül.

Az utóelemzési dokumentumban a módosítások véglegesítése ADR-ben történik, és a havi error budget állapota rögzítésre kerül.

20.4.19 5. Labor: SLO- és incidensforgatókönyv készítése

A labor célja, hogy a résztvevő saját, kitalált vagy valós, de izolált szolgáltatásra definiáljon SLI-kat, SLO-kat, error budgetet, burn rate riasztási szabályokat, és egy incidensforgatókönyvet dolgozzon ki.

20.4.20 5.1. Kiinduló állapot

A laborhoz egy egyszerű, konténerben futtatható példa-szolgáltatást adunk, például egy Python FastAPI alkalmazást, amely egy könyvtári rendeléskezelő API-t valósít meg. A szolgáltatás két végpontot kínál: `POST /orders` és `GET /orders/{id}`. Az adatbázis egy PostgreSQL konténer. A kiinduló konfigurációban nincsenek SLI/SLO definíciók, nincsenek riasztások, és nincs incidensdokumentáció.

A labor környezet izolált, a destruktív műveletek (például adatbázis-törlés, szolgáltatás-leállítás) csak ebben a helyi Docker Compose környezetben futtathatók.

20.4.21 5.2. Elvárt eredmény

A résztvevő egy **reprodukálható dokumentumot** készít, amely a következő elemeket tartalmazza:

1. **SLI definíciók:** a két végpontra legalább két-két SLI-t (rendelkezésre állás, késleltetés).
2. **SLO célok:** minden SLI-hez SLO, mérési ablakkal, percentilissel, célértékkel.
3. **Error budget számítás:** a havi error budget százalékban, a választott SLO alapján.
4. **Burn rate riasztási szabályok:** legalább három különböző ablak-küszöb kombináció, táblázatban.
5. **Incidensforgatókönyv:** egy lehetséges hiba leírása, idővonallal, észleléssel, eszkalációval, utóelemzéssel.
6. **Döntési napló:** minden fontos döntés ADR formában rögzítve.

A dokumentum formátuma lehet Markdown, Jupyter notebook vagy bármilyen verziózható szöveges formátum. A mérések **nem futtatott eredmények**: ha a résztvevő szimulál, azt jelölnie kell, és a várt eredményt el kell különítenie a mért eredménytől.

20.4.22 5.3. Ellenőrzési szempontok

- Az SLI-k egyértelműen megadják a számlálót és a nevezőt.
- Az SLO-khoz tartozik mérési ablak és percentilis, ahol releváns.
- Az error budget számítása helyes, és a mértékegység (% vagy ezrelék) meg van adva.
- A burn rate riasztások több időablakot használnak, és a küszöbök indokoltak.
- Az incidensforgatókönyv tartalmazza a hibabüdzsé állapotát az incidens előtt és után.
- A dokumentumban a mért és a várt eredmények elkülönülnek.
- A döntési napló kontextust, döntést, alternatívákat és következményeket tartalmaz.

20.4.23 5.4. Visszaállítási út

A labor során használt Docker Compose környezet leállítható a `docker compose down` paranccsal. A kiinduló állapot visszaállítható a `docker compose down -v && docker compose up -d` paranccsal, amely törli a helyi adatbázist és újra létrehozza az üres sémát. Az izolált környezetben történt módosítások nem érintenek éles rendszert.

20.4.24 6. Ellenőrzőlista a megbízhatóság méréséhez és incidenskezeléshez

A fejezet végén egy gyakorlati ellenőrzőlista segít a minták alkalmazásában:

- ☐ Az SLI-k a felhasználói viselkedést mérik, nem a belső implementációt.
- ☐ Minden SLI-hez tartozik mértékegység, mérési ablak, mintavétel és bizonytalanság.
- ☐ Az SLO-k mérési ablakai és percentilisei egyértelműek.
- ☐ Az SLO-k nem 100%-os célok, és tükrözik a felhasználói elfogadhatóság határát.
- ☐ Az error budgetet kiszámították, és a szervezeti folyamatok használják (release freeze, prioritás).
- ☐ A burn rate riasztások több ablakot használnak, és a riasztási küszöbök indokoltak.
- ☐ A riasztások kontextust adnak: szolgáltatás, SLO, burn rate, lehetséges első lépés.
- ☐ Az eszkalációs lánc definiált, a rotációk és a kontaktok naprakészek.
- ☐ Az incidens utóelemzés blameless, az események idővonallal vannak rögzítve.
- ☐ A javító intézkedések visszavezetnek az SLI/SLO rendszerre.

20.4.25 7. Önellenőrző kérdés

> **Kérdés:** Egy szolgáltatás havi SLO-ja 99{,}9%-os rendelkezésre állás. Az egyik nap a hibaarány 1 órán át 0{,}4% volt. Mekkora volt a burn rate, és ha ez a sebesség fennmaradna, mennyi idő alatt fogyna el a havi error budget?
> **Válasz:** A hiba budget `1 - 0{,}999 = 0{,}001` = 0{,}1%. A burn rate `0{,}004 / 0{,}001 = 4`. Ha a burn rate 4, akkor a havi budget (30 nap / 4) = 7{,}5 nap alatt fogyna el. A pontos számításhoz a tényleges hibaidő és a kérésarány is számít, de a burn rate alapján ez a becslés.

20.4.26 8. Összefoglalás és továbblépés

Az SLI/SLO/error budget keretrendszer a megbízhatóságot a technikai részletekből a **döntéshozatali mechanizmusok** szintjére emeli. Nem elég hibákat találni; azt is tudni kell, hogy egy hiba milyen sebességgel fogyasztja a szolgáltatás megbízhatósági tartalékát, és mikor kell a szervezetnek erőforrást átcsoportosítania. Az észlelés, az eszkaláció és a blameless utóelemzés ezt a mechanizmust tartja életben, és visszacsatolja a tanulságokat az architektúrába.

A következő fejezet (Threat modeling és támadási felület) a **biztonsági architektúra** felé lép tovább, és a threat modeling, valamint a támadási felület elemzését tárgyalja. Az itt bevezetett mérési és döntési kultúra ott is alapul szolgál: a biztonsági kockázatokat ugyanúgy célokhoz, mérésekhez és döntési naplókhoz kell kötni, mint a rendelkezésre állást.

> **Ellenőrzésre szoruló állítások:** A Google SRE könyv és a Principles of Chaos konkrét fejezetei, az AWS Builders Library egyes cikkei, valamint a Kubernetes dokumentáció adott verziójú, hivatkozott szakaszai a kiadás időpontjában érvényesek; a felhasználás előtt ellenőrizni kell a hivatalos források aktuális állapotát. A könyvben szereplő számítások és példák szemléltető jellegűek, nem szolgálnak jogi vagy szerződéses SLA-meghatározásként.

21 Biztonsági architektúra

21.1 Threat modeling és támadási felület

A fenyegetésmodellezés nem egyszeri biztonsági ellenőrzés, hanem olyan tervezési tevékenység, amely a rendszer architektúráját a támadó szemszögéből vizsgálja. Célja, hogy még a fejlesztés vagy az üzembe helyezés előtt feltárja azokat a pontokat, ahol a rendszer védelme megsérülhet, és ezzel lehetővé tegye a tudatos döntéshozatalt. A biztonsági architektúra egyik alaptétele, hogy a támadási felületet – vagyis minden olyan felületet, ahol a támadó interakcióba léphet a rendszerrel – szisztematikusan kell feltérképezni. Ez a fejezet ezt a feltérképezést tanítja meg: hogyan lehet egy webes rendszer példáján keresztül adatfolyam-alapú fenyegetésmodellt készíteni, bizalmi határokat jelölni, STRIDE-dal fenyegetéseket azonosítani, majd azokat priorizálni. A hangsúly a módszeren és a döntések nyomon követhetőségén van, nem pedig konkrét védelmi intézkedéseken – azok a következő fejezetekben kerülnek elő.

21.1.1 Miért éppen a bizalmi határok a kiindulópont?

A fenyegetésmodellezés során a leggyakoribb hiba, hogy a résztvevők azonnal a konkrét sebezhetőségekre vagy támadási technikákra koncentrálnak, anélkül, hogy pontosan látnák, hol húzódnak a rendszer biztonsági értelemben vett határai. A bizalmi határ (trust boundary) fogalma ezért központi szerepet kap: olyan határvonal a rendszerben, amelyen átlépve a bizalmi szint megváltozik. Másképp fogalmazva: a határ egyik oldalán lévő entitás nem feltétlenül bízik meg a másik oldalon lévő entitásban, vagy legalábbis más szintű biztosítékokat követel meg tőle.

Például egy webes rendszerben a felhasználó böngészője és a webalkalmazás-szerver közötti határ nyilvánvaló bizalmi határ: a szerver nem bízhat meg a kliensben, hiszen a böngészőt a felhasználó – vagy éppen egy támadó – irányítja. Ugyancsak bizalmi határ húzódik a webalkalmazás és az adatbázis között, még akkor is, ha mindkettő a szervezet belső hálózatán fut: a webalkalmazás kompromittálódása esetén az adatbázis közvetlen elérése súlyos következményekkel jár. A bizalmi határok tehát nem csupán a szervezeti hálózat határait jelentik, hanem minden olyan pontot, ahol az ellenőrzés vagy a biztonsági garancia szintje megváltozik.

A bizalmi határok helyes jelölése azért döntő, mert a támadási felület jelentős része ezekre a határokra koncentrálódik. A támadó rendszerint olyan pontot keres, ahol átléphet egy határt: hamis identitással beléphet egy belső zónába, módosíthat egy adatfolyamot, vagy éppen megkerülhet egy ellenőrzést. A fenyegetésmodell akkor lesz használható, ha minden ilyen határt explicit módon megjelenítünk, és minden határhoz hozzárendeljük a rajta áthaladó adatok és vezérlési jelek típusát.

21.1.2 A fenyegetésmodellezés hat lépése

Egy gyakorlatban jól működő fenyegetésmodellezési folyamat hat lépésből áll, amelyeket egy kisebb webes rendszer példáján mutatok be. A lépések nem merevek, de sorrendjük logikus:

először a rendszert és annak határait kell pontosan megérteni, csak utána lehet fenyegetéseket azonosítani és priorizálni.

1. **Célok és hatókör rögzítése.** Meg kell határozni, mit vizsgálunk, milyen mélységig, és kik a stakeholderek. Példánkban a vizsgált rendszer egy webes ügyfélportál, amely lehetővé teszi regisztrált felhasználóknak, hogy megtekintsék szerződéseiket és számláikat, valamint online befizetést indítsanak. A rendszerhatár a böngészőtől a belső adatbázisig terjed; a külső fizetési szolgáltatót és az e-mail értesítő szolgáltatást külső entitásként kezeljük.

2. **Adatfolyam-diagram (DFD) készítése.** A DFD a rendszer elemeit – folyamatok, adattárolók, külső entitások, adatfolyamok – és a köztük lévő kapcsolatokat ábrázolja. A diagrams-as-code elvnek megfelelően a diagramot például Mermaid formátumban készítjük el, így az a kódbázissal együtt verziózható. A DFD-n jelölni kell a bizalmi határokat is, jellemzően szaggatott vonallal vagy csoportosítással. Az alábbi példa egy egyszerűsített DFD-t mutat:

```
flowchart LR
    subgraph Internet [Internet - nem megbízható zóna]
        U[Felhasználó böngésző]
    end

    subgraph DMZ [DMZ - félmegbízható zóna]
        W[Web frontend]
    end

    subgraph Belső [Belső hálózat - megbízható zóna]
        API[API szolgáltatás]
        DB[(Adatbázis)]
        MSG[Üzenetküldő]
    end

    subgraph Külső [Külső szolgáltatók]
        PAY[Fizetési szolgáltató]
        MAIL[E-mail szolgáltató]
    end

    U -->|HTTPS| W
    W -->|HTTPS| API
    API -->|SQL| DB
    API -->|HTTPS| PAY
    API -->|HTTPS| MSG
    MSG -->|SMTP| MAIL
```

A diagramon jól látható, hogy a felhasználó böngészője az internet zónában található, a webes frontend a DMZ-ben, az API, az adatbázis és az üzenetküldő a belső hálózaton, míg a fizetési és e-mail szolgáltatók külső entitások. A szaggatott vonalak jelzik a bizalmi határokat: az internet és a DMZ között, a DMZ és a belső hálózat között, valamint a belső hálózat és a külső szolgáltatók között.

3. **Bizalmi határok és adatfolyamok elemzése.** Minden adatfolyamnál rögzíteni kell, hogy milyen adat halad át rajta, milyen protokollon, milyen irányban, és hogy a forrás vagy a cél megbízható-e. Például a böngészőből a webes frontend felé tartó HTTPS kérés tartalmazhat bejelentkezési adatokat, számlaszámokat vagy fizetési műveleteket; ezek mind érzékeny adatok. A webes frontend és az API közötti forgalom belső, de ha a támadó bejut a DMZ-be, akkor ezt a forgalmat is veszélyeztetheti. Az API és az adatbázis közötti SQL forgalom pedig a legérzékenyebb, hiszen közvetlen adatmanipulációt tesz lehetővé.

4. **Fenyegetések azonosítása STRIDE-dal.** A STRIDE egy hat kategóriából álló keretrendszer, amely segít szisztematikusan végiggondolni, milyen típusú fenyegetések érhetik az

egyes elemeket és adatfolyamokat. A kategóriák: *Spoofing* (megszemélyesítés), *Tampering* (módosítás), *Repudiation* (letagadhatóság hiánya), *Information Disclosure* (információszivárgás), *Denial of Service* (szolgáltatásmegtagadás), *Elevation of Privilege* (jogosultságkiterjesztés). A STRIDE-ot minden bizalmi határra és minden elemre alkalmazni kell, nem csak a legnyilvánvalóbb pontokra. Például a böngésző–frontend határon a megszemélyesítés (hamis bejelentkezés), a módosítás (kérés paramétereinek átírása) és az információszivárgás (érzékeny adat a válaszban) egyaránt releváns. Az API–adatbázis határon viszont inkább a módosítás, az információszivárgás és a szolgáltatásmegtagadás dominál.

5. **Fenyegetések priorizálása.** Nem minden fenyegetés egyformán fontos. A priorizálás célja, hogy a szűkös erőforrásokat a legnagyobb kockázatú területekre összpontosítsuk. A gyakorlatban elterjedt megközelítés a kockázat = valószínűség × hatás képlet, ahol mindkét tényezőt például 1-től 5-ig pontozzuk. A valószínűség becsléséhez felhasználhatók iparági statisztikák, korábbi incidensek, illetve a rendszer kitettsége; a hatás becslésénél pedig figyelembe kell venni az üzleti, jogi és reputációs következményeket. A becslés természetesen bizonytalan, ezért a bizonytalanságot is dokumentálni kell, például konzervatív értékekkel vagy intervallumokkal.

6. **Dokumentálás, döntések és iteráció.** A fenyegetésmodell eredményét rögzíteni kell egy verziózott dokumentumban, amely tartalmazza a DFD-t, a bizalmi határokat, a fenyegetések listáját, a priorizálást és a válaszstratégiákat. A döntéseket Architecture Decision Record (ADR) formában érdemes leírni, hogy a későbbi olvasó megértse, miért választották az adott kezelési módot. A fenyegetésmodell nem statikus: a rendszer változásával, új funkciók bevezetésével vagy új fenyegetések megjelenésével frissíteni kell.

21.1.3 A STRIDE modell részletesen

A STRIDE mozaikszó hat fenyegetéskategóriát takar. Mindegyik kategória más-más biztonsági tulajdonság megsértését írja le. A kategóriák ismerete azért hasznos, mert segít elkerülni, hogy a modellezés során csak a legismertebb támadástípusokra – például az SQL injectionre vagy a cross-site scriptingre – koncentráljunk, miközben más, ugyanolyan súlyos problémák rejtve maradnak.

- **Spoofing (megszemélyesítés).** Akkor következik be, amikor egy támadó hamis identitással lép fel. Webes környezetben ide tartozik a felhasználónév és jelszó ellopása, a session cookie eltérítése, a hamis tanúsítvány használata, vagy akár egy belső szolgáltatás nevében történő fellépés. A bizalmi határoknál ez a fenyegetés különösen fontos: a határ másik oldalán álló entitás nem feltétlenül az, akinek mondja magát.

- **Tampering (módosítás).** A támadó jogosulatlanul megváltoztat adatokat vagy végrehajtható kódot. Például egy webes kérés paramétereinek módosítása, egy adatbázis rekord átírása, vagy egy konfigurációs fájl manipulálása. A tampering gyakran a bizalmi határokon átívelő adatfolyamokon keresztül valósul meg.

- **Repudiation (letagadhatóság hiánya).** Ez a fenyegetés arra utal, hogy a rendszer nem tudja bizonyítani, ki és mit hajtott végre. Ha nincs megfelelő naplózás és audit, egy támadó elvégezhet egy műveletet, majd letagadhatja azt. A pénzügyi tranzakcióknál, egészségügyi adatoknál vagy adminisztratív műveleteknél a repudiation komoly jogi és üzleti kockázatot jelent.

- **Information Disclosure (információszivárgás).** Bizalmas adatok illetéktelen kezekbe kerülnek. Ide tartozik az SQL injection révén kinyert adatbázistartalom, a hibás konfiguráció miatt nyilvánosan elérhető végpont, vagy a naplófájlokban szereplő érzékeny adat. A bizalmi

határok elemzésekor minden adatfolyamnál meg kell kérdezni: vajon ez az adat kikerülhet-e a határon kívülre?

- **Denial of Service (szolgáltatásmegtagadás).** A támadó megakadályozza, hogy a rendszer a legitim felhasználókat kiszolgálja. Ez lehet hálózati túlterhelés, erőforrás-kimerítés, vagy éppen egy hibás bemenet miatti összeomlás. A bizalmi határoknál a DoS gyakran úgy jelentkezik, hogy a támadó a határon keresztül olyan forgalmat generál, amelyet a belső rendszer nem tud kezelni.
- **Elevation of Privilege (jogosultságkiterjesztés).** Alacsonyabb jogosultságú entitás magasabb jogosultságot szerez. Például egy sima felhasználó adminisztrátori funkciókat ér el egy hibás jogosultság-ellenőrzés miatt, vagy egy folyamat a saját jogosultságánál magasabb szintű hozzáférést kap. A jogosultságkiterjesztés gyakran más fenyegetések következménye: előbb a támadó megszemélyesít valakit, majd kihasznál egy hibát a magasabb jogok megszerzésére.

A STRIDE-ot célszerű táblázatos formában alkalmazni. Minden elemre (külső entitás, folyamat, adattároló, adatfolyam) végig kell menni a hat kategórián, és rögzíteni, hogy az adott fenyegetés releváns-e, és ha igen, milyen formában. Az alábbi táblázat egy részletet mutat a példarendszerre:

Elem	STRIDE kategória	Fenyegetés leírása
Felhasználó böngésző – Web frontend adatfolyam	Spoofing	Támadó hamis bejelentkezéssel vagy ellopott session cookie-val lép fel
Felhasználó böngésző – Web frontend adatfolyam	Tampering	A kérés paramétereinek módosítása (pl. számlaazonosító)
Web frontend – API adatfolyam	Information Disclosure	Érzékeny adat kiszivárgása a kliens és szerver közötti forgalomból
API – Adatbázis adatfolyam	Tampering	SQL injection révén rekordok módosítása
API – Adatbázis adatfolyam	Information Disclosure	SQL injection révén adatbázistartalom kinyerése
API – Fizetési szolgáltató adatfolyam	Spoofing	A fizetési szolgáltató nevében érkező hamis visszaigazolás
E-mail szolgáltató küldési folyamat	Repudiation	Az értesítések küldése nem naplózott, így a kiküldés letagadható

A táblázat természetesen csak kiindulópont; a teljes modellhez minden elemet és minden kategóriát végig kell elemezni.

21.1.4 Fenyegetések priorizálása

A STRIDE elemzés után rendszerint több tucat fenyegetés szerepel a listán, amelyek közül nem mindegyik igényel azonnali beavatkozást. A priorizálás célja, hogy a döntéshozók a legnagyobb kockázatú tételekkel foglalkozzanak először. A legegyszerűbb és leggyakrabban használt módszer a kockázati mátrix: minden fenyegetéshez hozzárendelünk egy valószínűséget és egy hatást, majd a kettő szorzatából (vagy összegéből) képzett kockázati pontszám alapján sorba rendezzük.

A **valószínűség** becslésekor figyelembe kell venni, hogy a támadó milyen könnyen férhet hozzá az adott felülethez, mennyire ismert a sebezhetőség, és van-e már aktív kihasználás a vadonban. A **hatás** becslésekor pedig azt kell felmérni, hogy a fenyegetés bekövetkezése esetén milyen kár keletkezik: adatvesztés, anyagi kár, jogi következmény, reputációs veszteség, üzemkiesés. A becslésekhez használhatunk 1–5 skálát, ahol az 1 a nagyon alacsony, az 5 a nagyon magas értéket jelenti.

Fontos, hogy a becslések bizonytalanok, és ezt a bizonytalanságot rögzíteni kell. Például ha egy fenyegetés valószínűségét 3-ra becsüljük, de nem áll rendelkezésre megbízható statisztika, akkor ezt jelölhetjük intervallumként (2–4) vagy megjegyzéssel: „a becslés szakértői véleményen alapul, további mérés javasolt”. A laborban – ahol valós mérés ritkán áll rendelkezésre – minden becslés várt értéknek tekintendő, és ezt explicit módon el kell különíteni a mért adatoktól.

A priorizálás eredményét célszerű táblázatos formában rögzíteni, például:

Fenyegetés	Érintett határ	Valószínűség (1–5)	Hatás (1–5)	Kockázat (szorzat)	Prioritás
SQL injection az API–adatbázis határon	API – Adatbázis	3	5	15	Kritikus
Session cookie eltérítés	Böngésző – Frontend	4	4	16	Kritikus
Hamis fizetési visszaigazolás	API – Fizetési szolgáltató	2	5	10	Magas
Értesítési naplózás hiánya	E-mail szolgáltató	3	2	6	Közepes
DDoS a webes frontend ellen	Internet – DMZ	2	4	8	Magas

A táblázatban a kockázati pontszám alapján a legmagasabb értékű tételek kapnak kritikus prioritást. A valószínűség és hatás becslések szubjektívek, ezért a döntési naplóban rögzíteni kell a becslés indoklását és a bizonytalanságot is.

A priorizálás után minden fenyegetéshez válaszstratégiát kell rendelni. A négy alapvető stratégia: **elfogadás** (a kockázat vállalható), **csökkentés** (védelmi intézkedések bevezetése), **átadás** (például biztosítás vagy külső szolgáltató bevonása), **elkerülés** (a funkció megszüntetése vagy a rendszerhatár módosítása). A válaszstratégiák kiválasztása már a következő fejezetek részletes anyagához tartozik, de a fenyegetésmodellben már itt rögzíteni kell a döntés irányát, hogy a későbbi tervezés erre építhessen.

21.1.5 Tipikus hibák és azok elkerülése

A fenyegetésmodellezés során számos buktató leselkedik a csapatra. Ezek közül a leggyakoribbak a következők:

1. **A bizalmi határok hiánya vagy rossz jelölése.** Ha a DFD-n nem tüntetik fel, hol változik a bizalmi szint, akkor a fenyegetések azonosítása esetlegessé válik. A megoldás: minden diagramon kötelezően jelölni kell a bizalmi határokat, és minden határhoz hozzá kell rendelni a rajta áthaladó adatfolyamokat.
2. **Csak a technikai rétegekre koncentrálás.** A fenyegetésmodell gyakran kizárólag a szoftverkomponensekre és azok kapcsolataira fókuszál, figyelmen kívül hagyva a szervezeti folyamatokat, a felhasználói viselkedést vagy a külső szolgáltatók szerepét. Pedig a támadási felülethez hozzátartozik például az ügyfélszolgálat, ahol egy social engineering támadás célba veheti a munkatársakat. A fenyegetésmodellnek a teljes rendszerhatárt le kell fednie, beleértve az emberi tényezőt is.
3. **A STRIDE mechanikus kipipálása.** Előfordul, hogy a csapat minden elemhez beírja a hat kategóriát, de nem gondolja végig, hogy az adott fenyegetés valóban releváns-e, és milyen konkrét támadási úton valósulhat meg. A STRIDE célja a strukturált gondolkodás, nem pedig egy sablon kitöltése. Minden azonosított fenyegetéshez konkrét forgatókönyvet kell tudni rendelni: ki a támadó, mit akar elérni, milyen lépéseken keresztül.
4. **A priorizálás szubjektív pontozása indoklás nélkül.** Ha a valószínűséget és a hatást csak megérzésre alapozva pontozzák, a kockázati sorrend megbízhatatlan lesz. A becslésekhez mindig rögzíteni kell a forrást (például iparági jelentés, korábbi incidens, szakértői vélemény) és a bizonytalanságot. Ha nincs adat, akkor konzervatív értéket kell használni, és ezt jelezni kell.
5. **A modellt nem frissítik.** A fenyegetésmodell egyszeri elkészítése hamis biztonságérzetet ad. A rendszer változásával új támadási felületek jelennek meg, ezért a modellt a tervezési ciklus részévé kell tenni: minden jelentős architekturális változásnál, új integrációnál vagy biztonsági incidens után felül kell vizsgálni.
6. **A döntések nem nyomon követhetők.** Ha a fenyegetésmodell eredményeit nem rögzítik ADR formában, akkor néhány hónap múlva senki sem fogja tudni, miért választottak egy adott kezelési módot. A jó gyakorlat, hogy minden jelentős fenyegetésre és a hozzá kapcsolódó döntésre készüljön egy rövid ADR, amely tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.

21.1.6 Labor: webes rendszer fenyegetésmodellje

A fejezet laborja a fent bemutatott módszertan gyakorlati alkalmazását kéri. A cél egy reprodukálható threat model dokumentum létrehozása, amely egy webes rendszert vizsgál. A labor nem ad konkrét megoldást, hanem a folyamatot és az elvárt eredményt rögzíti; a tényleges fenyegetések a választott rendszer jellemzőitől függenek.

Kiinduló állapot: Rendelkezésre áll egy egyszerű webes ügyfélportál specifikációja, amely a következő elemeket tartalmazza: felhasználói regisztráció és bejelentkezés, szerződések és számlák megtekintése, online befizetés indítása külső fizetési szolgáltatón keresztül, valamint e-mail értesítések kiküldése. A rendszer három rétegből áll: böngészős frontend, szerveroldali API és relációs adatbázis. A frontend a DMZ-ben, az API és az adatbázis a belső hálózaton fut. A fizetési szolgáltató és az e-mail szolgáltató külső entitás.

Elvégzendő lépések:

1. **Kontextus rögzítése.** Készíts egy rövid leírást a rendszerről, amely tartalmazza a célját, a stakeholdereket, a rendszerhatárt, a feltételezéseket és a vizsgálat mélységét. Például: „A modell a felhasználói böngészőtől a belső adatbázisig terjed; a külső szolgáltatókat entitásként kezeljük, de a belső működésüket nem vizsgáljuk."
2. **DFD készítése diagrams-as-code formában.** Rajzold meg a DFD-t Mermaid, PlantUML vagy Structurizr DSL segítségével. Jelöld a folyamatokat, adattárolókat, külső entitásokat, adatfolyamokat és a bizalmi határokat. Minden adatfolyamnál tüntesd fel a protokollt (pl. HTTPS, SQL) és az átvitt adat jellegét (pl. bejelentkezési adatok, számlaadatok, fizetési tranzakció).
3. **STRIDE elemzés.** Minden elemre (külső entitás, folyamat, adattároló, adatfolyam) végezd el a STRIDE elemzést. Rögzítsd táblázatban, hogy az adott fenyegetés releváns-e, és ha igen, írd le a konkrét támadási forgatókönyvet. Legalább tíz fenyegetést azonosíts, de törekedj a teljes körű elemzésre.
4. **Priorizálás.** Minden azonosított fenyegetéshez rendelj valószínűséget és hatást 1–5 skálán, majd számítsd ki a kockázati pontszámot. A becslésekhez rögzítsd az indoklást és a bizonytalanságot. Készíts kockázati táblázatot, és sorold a fenyegetéseket prioritás szerint.
5. **Döntési napló (ADR-szerű bejegyzés).** A legmagasabb prioritású fenyegetésekre készíts rövid döntési bejegyzéseket, amelyek tartalmazzák a kontextust, a választott válaszstratégiát (elfogadás, csökkentés, átadás, elkerülés), az alternatívákat és a következményeket. A konkrét védelmi intézkedések kifejtése nem szükséges; elegendő a stratégiai irány rögzítése.
6. **Mért és várt eredmények elkülönítése.** A labor során valós mérésre általában nincs lehetőség, ezért minden valószínűség- és hatásbecslés „várt" jellegű. Ezt a dokumentumban explicit módon jelölni kell, például külön oszlopban vagy lábjegyzetben. Ha a csapat rendelkezik valamilyen mérési adattal (például korábbi incidens statisztika, naplóelemzés), akkor azt külön, „mért" forrásként kell feltüntetni.

Elvárt eredmény: Egy verziózott dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza a fenti elemeket. A dokumentumnak reprodukálhatónak kell lennie: a DFD kódja, a STRIDE táblázat, a kockázati táblázat és az ADR-ek mind szövegesen, diagrams-as-code formában szerepeljenek, így bárki újra tudja futtatni vagy ellenőrizni a modellt.

Ellenőrzési szempontok:

- A DFD tartalmazza-e a rendszerhatárt és a bizalmi határokat?
- Minden adatfolyamnál feltüntették-e a protokollt és az adat jelleget?
- A STRIDE elemzés minden elemre kiterjed-e, és minden kategóriát megvizsgáltak?
- A fenyegetésekhez tartozik-e konkrét támadási forgatókönyv?
- A priorizálás során a valószínűség és hatás becslések indokoltak és bizonytalansággal ellátottak?
- A döntési napló rögzíti-e a válaszstratégiát és annak okát?
- A mért és várt eredmények egyértelműen el vannak különítve?
- A dokumentum verziózott és frissíthető?

Visszaállítási út: A labor izolált környezetben végzett dokumentációs feladat, ezért nincs szükség technikai visszaállításra. Ha a csapat mégis valamilyen eszközt (például diagramkészítőt vagy táblázatkezelőt) használ, az eredeti állapot visszaállítása a verziókezelő rendszerben tárolt korábbi verzióval lehetséges.

21.1.7 Ellenőrzőlista a fenyegetésmodellezéshez

A fejezet végén egy gyakorlati ellenőrzőlista segít abban, hogy a fenyegetésmodell teljes és használható legyen:

- ☐ A rendszerhatár egyértelműen definiált.
- ☐ A stakeholderek és a vizsgálat célja rögzített.
- ☐ A DFD diagrams-as-code formában készült, és verziózott.
- ☐ A bizalmi határok minden diagramon jelölve vannak.
- ☐ Minden adatfolyamnál ismert a protokoll, az irány és az adat jelleg.
- ☐ A STRIDE elemzés minden elemre és kategóriára kiterjed.
- ☐ Minden azonosított fenyegetéshez konkrét támadási forgatókönyv tartozik.
- ☐ A fenyegetések priorizálása valószínűség és hatás alapján történt, a becslések indokoltak.
- ☐ A becslések bizonytalansága dokumentált.
- ☐ A válaszstratégiák ADR formában rögzítettek.
- ☐ A mért és várt eredmények el vannak különítve.
- ☐ A modell frissítési mechanizmusa definiált.

21.1.8 Önellenőrző kérdés

Hogyan változik egy webes ügyfélportál fenyegetésmodellje, ha a rendszer a belső hálózatról átkerül a felhőbe, és a felhasználók az internetről érik el? Mely bizalmi határok jelennek meg újra vagy erősödnek fel, és mely STRIDE kategóriák válnak hangsúlyosabbá?

A kérdés megválaszolásához gondold végig, hogy a felhőbe költözés során a korábban belső hálózaton futó komponensek (API, adatbázis, üzenetküldő) immár elérhetővé válhatnak az internet felől, vagy legalábbis a hálózati határok átalakulnak. A bizalmi határok közé bekerül a felhőszolgáltató és az üzemeltető közötti határ, valamint a felhő belső hálózata és a szervezet saját hálózata közötti határ. A STRIDE kategóriák közül a megszemélyesítés, a módosítás és az információszivárgás mellett felértékelődik a szolgáltatásmegtagadás, hiszen az internetről érkező forgalom jóval nehezebben szűrhető, mint egy belső hálózaton. A válaszban be kell mutatni, hogy a fenyegetésmodell frissítése során mely elemeket kell újraértékelni, és milyen új bizalmi határokat kell felvenni.

Ezzel a fejezet célja teljesül: az olvasó képes a fenyegetésmodellezés lépéseit követni, a bizalmi határokat helyesen jelölni, STRIDE-dal fenyegetéseket azonosítani, priorizálni, és a tipikus hibákat elkerülve reprodukálható, döntésorientált threat modelt készíteni.

21.2 Zero trust, identitás és hozzáférés

Az előző egységben a threat modeling segítségével feltártuk a fenyegetéseket és a támadási felületet. Most a biztonsági architektúra egyik legfontosabb védekezési paradigmájára, a zero trust megközelítésre, és annak alapjára, az identitás- és hozzáférés-kezelésre (Identity and Access Management, IAM) fókuszálunk.

A hagyományos periméter-alapú védelem – amelyet gyakran „erődítmény és várárok” modellként is emlegetnek – abból a feltételezésből indul ki, hogy a hálózatunkon belül minden megbízható, a hálózatunkon kívül pedig minden gyanús. A felhőalapú, mobil és együttműködő rendszerek korában ez a feltételezés tarthatatlanná vált: a munkatársak otthonról dolgoznak, a szolgáltatások más vállalatok hálózatában futnak, a támadók pedig egyetlen feltört végponttal vagy adathalászattal a belső hálózat közepén találhatják magukat. A periméter eltűnt, vagy ami még rosszabb, csak illúzió.

A zero trust nem egy konkrét termék vagy protokoll, hanem egy tervezési filozófia és architekturális elvrendszer. Ebben a fejezetben felépítjük ezt az elvrendszert, majd bemutatjuk azokat a gyakorlati mintákat és protokollokat, amelyekkel a mindennapi tervezés során érvényre tudjuk juttatni.

21.2.1 1. Mi a zero trust?

A NIST SP 800-207 (Zero Trust Architecture) meghatározása szerint a zero trust egy olyan kiberbiztonsági paradigma, amelynek középpontjában az erőforrások védelme áll, és szakít azzal az elképzeléssel, hogy a vállalati tűzfalon belüli hálózati szegmensek automatikusan biztonságosak lennének. A zero trust hét alapelve szerint:

1. **Minden hálózat megbízhatatlan:** A forgalom forrása (belső vagy külső hálózat) nem jelent jogosultságot. Minden kérést úgy kell kezelni, mintha nyílt hálózatról érkezne.
2. **Hozzáférést erőforrás-alapon:** A hozzáférési döntést nem a hálózati pozíció, hanem a konkrét erőforráshoz tartozó követelmények és a kérést indító entitás identitása, eszközállapota és környezete alapján kell meghozni.
3. **Least privilege:** Egy identitás csak a feladata elvégzéséhez minimálisan szükséges erőforrásokhoz és műveletekhez kapjon hozzáférést, és ezt is csak meghatározott ideig.
4. **Folyamatos ellenőrzés:** A hozzáférési döntés nem egyszeri, a bejelentkezéskor meghozott ítélet. A munkamenet során a kontextust folyamatosan értékelni kell, és anomália esetén a hozzáférést vissza kell vonni vagy újra kell hitelesíteni.
5. **Az infrastruktúra kompromittálódása valós lehetőség:** A tervezés során el kell fogadni, hogy a hálózat, a végpontok, sőt egyes szolgáltatások is feltörhetők. A rendszernek ebben a modellben is védenie kell az erőforrásait.

A periméter-alapú modellt gyakran egy „M&M” cukorkához hasonlítják: kívül kemény héj, belül puha, édes. Ha a héj megsérül, a támadó szabadon mozoghat belül. A zero trust ezzel szemben a „szív-perikardium” modell: minden egyes erőforrást külön védőréteg vesz körül, és a hozzáférés minden rétegen külön ellenőrzésen megy át.

Architektúrális szempontból a zero trust bevezetése nem egyszerűen egy új tűzfal telepítése. A rendszertervben explicit komponensként kell megjelennie a **Policy Enforcement Point (PEP)** és a **Policy Decision Point (PDP)** komponenseknek.

- **PEP:** Az a komponens, amely a hozzáférési kérést elfogja, és a hozzáférési döntést kikényszeríti. Ez lehet egy API gateway, egy szolgáltatás proxy, vagy akár egy alkalmazás belső middleware rétege.
- **PDP:** Az a komponens, amely a hozzáférési kérelem, az identitás és a környezet alapján meghozza a döntést (engedélyez vagy megtagad). A PDP a folyamatosan frissített szabályrendszerek (policy-k) alapján dönt.

Egy egyszerű C4 diagramon ez így ábrázolható:

```
graph TD
    subgraph "Rendszerhatár: Belső platform (placeholder)"
        PEP[PEP: API Gateway / Proxy]
        PDP[PDP: Policy Engine]
        IDS[Identity Provider (IdP)]
        RES[Védett erőforrás: Order Service]
    end

    USER["Felhasználó (stakeholder)"] -->|"HTTP/2, TLS 1.3, szinkron kérés, hibatűrés nélkül"|
↪   PEP
    PEP -->|"HTTPS, gRPC, szinkron"| PDP
    PDP -->|"HTTPS, mTLS, szinkron"| IDS
    PEP -->|"HTTPS, mTLS, szinkron"| RES
```

A diagramon a PEP fogadja a felhasználó kérését, konzultál a PDP-vel, amely az IdP-től kapott identitásinformációk és a policy-k alapján döntést hoz, majd a PEP ezt a döntést hajtja végre.

A zero trust architektúrában tehát az identitás lesz az új periméter. Ez vezet át az identitás- és hozzáférés-kezelés témájához.

21.2.2 2. Az identitás és hozzáférés-kezelés alapjai

Az IAM az a gyűjtőfogalom, amely magában foglalja az identitások életciklusának kezelését (létrehozás, módosítás, megszüntetés), hitelesítését, és az erőforrásokhoz való hozzáférésük engedélyezését. Az IAM három alappillére:

1. **Identitás (Identity):** Ki vagy mi az entitás? Egy identitás lehet egy ember (felhasználó), egy szolgáltatás (service account), egy eszköz (IoT) vagy egy rendszer.
2. **Hitelesítés (Authentication, AuthN):** Bizonyítani, hogy az entitás valóban az, akinek mondja magát. Ez a „Ki vagy?” kérdése.
3. **Engedélyezés (Authorization, AuthZ):** Annak eldöntése, hogy a hitelesített entitás mit tehet meg. Ez a „Mit csinálhatsz?” kérdése.

A **Least Privilege (minimális jogosultság)** elve az IAM egyik legfontosabb alapelve. Azt jelenti, hogy a rendszerben az identitásoknak csak azokat a jogosultságokat adjuk meg, amelyek a feladataik elvégzéséhez elengedhetetlenek, és ezeket is csak arra az időre, amíg a feladat tart. A gyakorlatban ez azt jelenti, hogy nem adunk adminisztrátori jogot egy alkalmazásnak, ha csak adatbázis-olvasásra van szüksége. Nem adunk közvetlen hozzáférést a production adatbázishoz egy fejlesztőnek, ha csak a logokat kell elemeznie. A just-in-time (JIT) hozzáférés ennek az elvnek a modern megvalósítása, amikor a jogosultságot kérésre, rövid időre kapja meg az entitás.

21.2.3 3. Hitelesítés és engedélyezés a gyakorlatban

21.2.3.1 3.1. Hitelesítés: Többtényezős védelem

A hitelesítés három fő tényezőre alapulhat:

- **Tudás (Something you know):** jelszó, PIN kód.
- **Birtok (Something you have):** hardveres token, okostelefon, okoskártya.
- **Tulajdonság (Something you are):** biometrikus adat (ujjlenyomat, arcfelismerés).

A modernebb rendszerek egyre inkább a „passwordless” (jelszó nélküli) hitelesítés felé mozdulnak el, ahol a jelszót egy birtoklási tényező (például egy eszköz regisztrált hitelesítő adata, WebAuthn szabvány) váltja ki. A **FIDO2** és a **WebAuthn** szabványok lehetővé teszik, hogy a böngésző vagy a platform (Windows, macOS, Android) egy hardveres vagy rendszerszintű kulcspárost használjon a hitelesítésre. A privát kulcs soha nem hagyja el az eszközt, ami alapjaiban gátolja a phishing támadások sikerét.

A **többtényezős hitelesítés (MFA)** azt jelenti, hogy a rendszer két vagy több eltérő típusú tényezőt követel meg. Fontos, hogy az MFA nem azt jelenti, hogy két jelszót kell megadni, hanem hogy legalább két különböző tényező-osztályból kell bizonyítékot szolgáltatni. Egy jelszó és egy egyszer használatos kód (TOTP) a telefonon egy jó példa.

Egy fontos architektúrális döntés: **a hitelesítést nem szabad elosztani a mikroszolgáltatások között**. Ha minden szolgáltatás maga kezelné a felhasználói hitelesítő adatokat, az katasztrofális biztonsági következményekkel járna, és súlyosan sértené a felelősségek szétválasztásának elvét. Helyette a hitelesítést egy központi **Identity Provider (IdP)** végzi, és a szolgáltatások ezt a központi hitelesítést használják.

21.2.3.2 3.2. Hitelesítési protokollok: OpenID Connect

A modern webes és felhőalapú rendszerek szabványos hátterét az **OAuth 2.1** keretrendszer és az arra épülő **OpenID Connect (OIDC)** protokoll adja.

- **OAuth 2.1** egy *authorization* keretrendszer. Nem hitelesítésre, hanem **delegált hozzáférés** engedélyezésére tervezték. Lehetővé teszi, hogy egy felhasználó anélkül adjon hozzáférést egy harmadik félnek az erőforrásaihoz, hogy a saját hitelesítő adatait (pl. jelszavát) megosztaná vele. Az OAuth 2.1 a korábbi 2.0-s verzió biztonsági ajánlásainak beépítésével jött létre, kikényszerítve a PKCE-t, a böngészőn belüli tokenek korlátozását és az implicit flow eltávolítását.
- **OpenID Connect (OIDC)** egy hitelesítési (authentication) réteg az OAuth 2.0/2.1 tetején. Az OIDC az OAuth flow végén nem csak egy hozzáférési tokent (access token), hanem egy **ID tokent** is kiad. Az ID token egy JSON Web Token (JWT), amely a felhasználóról tartalmaz az identitását igazoló állításokat (claims-eket), például a felhasználói azonosítót (`sub`), a kibocsátót (`iss`), a címzettet (`aud`) és a lejárati időt (`exp`). A szolgáltatás ezt az ID tokent verifikálva hitelesítheti a felhasználót.

Fontos szabály, hogy az ID tokent **nem szabad** hozzáférés-engedélyezésre használni. Az ID token arra válaszol, hogy „Ki ez a felhasználó?”, nem arra, hogy „Mihez van joga?”. A jogosultságok ellenőrzésére az OAuth 2.1 **access token**-t kell használni, amelyet a hitelesítő fél (Resource Server) a tokent kibocsátó Authorization Server-rel ellenőriz (token introspection) vagy a tokenbe ágyazott digitális aláírást verifikálja.

Egy tipikus OIDC Authorization Code Flow a következőképpen néz ki:

```
1. A böngészőben futó SPA a felhasználót az IdP /authorize végpontjára irányítja.
2. Az IdP hitelesíti a felhasználót (pl. MFA-val), majd visszairányítja a SPA-t egy egyszer
↪  használatos authorization code-dal.
3. A SPA backendje (vagy a SPA megfelelő konfigurációval) ezt a code-ot elküldi az IdP /token
↪  végpontjára.
4. Az IdP ellenőrzi a kódot és a klienst, majd visszaad egy ID tokent és egy access tokent.
5. A SPA az access tokent küldi az API-nak. Az API az IdP-vel ellenőrizteti a token aláírását és
↪  érvényességét, majd kiadja a kért erőforrást.
```

Ezt a folyamatot a SPA számára a **PKCE (Proof Key for Code Exchange)** egészíti ki, amely megakadályozza, hogy egy lehallgató támadó visszaéljen az authorization code-dal.

21.2.3.3 3.3. Engedélyezési modellek

Ha a hitelesítés megtörtént, az engedélyezés (authorization) dönti el, ki mihez férhet hozzá. A gyakorlatban több modellt is használunk, és gyakran kombináljuk őket:

1. **RBAC (Role-Based Access Control):** A jogosultságokat szerepkörökhöz (role) rendeljük, és az identitásokat ezekbe a szerepkörökbe helyezzük. Például: `admin`, `editor`, `viewer`. Az RBAC könnyen érthető és adminisztrálható kis és közepes rendszerekben, de ahogy a szervezet nő, a szerepkörök száma robbanásszerűen megnőhet (role explosion).
2. **ABAC (Attribute-Based Access Control):** A hozzáférési döntést nem a szerepkör, hanem az entitás, az erőforrás és a környezet attribútumai alapján hozzuk meg. Például: *„A `finance` osztályhoz tartozó felhasználó munkaidőben, a céges hálózatról, a `report` erőforráshoz `read` hozzáférést kap."* Az attribútumok logikai kifejezésekkel kombinálhatók. Az ABAC sokkal rugalmasabb, de komplexebb a szabályok tervezése és kiértékelése.
3. **ReBAC (Relationship-Based Access Control):** A hozzáférést az entitások közötti kapcsolatok (például szervezeti hierarchia, tulajdonosi reláció) alapján határozzuk meg. Például: *„Egy dokumentumot csak a létrehozója és az ő felettese törölheti."* A ReBAC különösen jól illik olyan rendszerekhez, ahol a hozzáférési jogok természetesen követik az üzleti kapcsolatokat (pl. közösségi média, Dokumentum menedzsment rendszerek). A Google Zanzibar egy híres, ipari léptékű ReBAC rendszer.

A gyakorlatban egy modern platform gyakran egy **hibrid modellt** alkalmaz: a rendszeradminisztratív jogosultságokhoz RBAC-ot, a felhasználói adatokhoz való hozzáférést pedig ABAC vagy ReBAC szabályokkal szabályozza.

21.2.4 4. Szolgáltatásidentitás és az erőforrások védelme

Nem csak az emberek, hanem a mikroszolgáltatások, a batch folyamatok és az automatizálási scriptek is identitással rendelkeznek. Ezt nevezzük **szolgáltatásidentitásnak (service identity)** vagy nem emberi identitásnak (Non-Human Identity, NHI). Minden szolgáltatásnak saját, egyedi hitelesítő adatot kell biztosítani, és a hálózaton belüli hívásokat is hitelesíteni kell.

A gyakorlatban ez a következő mintákkal valósítható meg:

1. **Kölcsönös TLS (mTLS):** A hívó és a hívott fél egyaránt bemutat egy X.509 tanúsítványt. Ez a service mesh technológiák (pl. Istio, Linkerd) alapvető építőköve. A mesh a proxy oldalán automatikusan kezeli a tanúsítványok életciklusát, és a hívó identitását a workload nevével azonosítja.
2. **Rövid élettartamú tokenek:** A szolgáltatások egy központi Authorization Server-től (pl. OAuth 2.1 szervertől) kapnak rövid élettartamú, JWT formátumú access tokeneket. A hívó szolgáltatás ezt a token-t mutatja be a hívott szolgáltatásnak, amely a token aláírását verifikálva azonosítja a hívó identitását és jogosultságait. A rövid élettartam csökkenti a kiszivárgott tokenek kockázatát.
3. **Felhőszolgáltatói identitás:** Felhőben (pl. AWS IAM, Azure Managed Identity, Google Service Account) a szolgáltatások nem hagyományos kulcsokkal, hanem a platform által biztosított identitással és a platform szintű hitelesítéssel érik el az erőforrásokat. Például egy Kubernetes Pod egy Service Account-tal rendelkezik, és a felhőszolgáltató ennek az accountnak a jogosultságát ellenőrzi anélkül, hogy a podban hosszú élettartamú kulcsot kellene tárolni. Ez a **workload identity** minta lényege.

A lényeg: **soha ne használjunk személyes felhasználói hitelesítő adatot szolgáltatások közötti híváshoz**. A szolgáltatások közötti kommunikációt a lehető legnagyobb mértékben a

platform rétegre kell bízni (service mesh, workload identity), vagy rövid élettartamú tokenekkel kell védelmet adni.

21.2.5 5. Gyakori hibák és helyes megoldások

A következőkben összegyűjtjük a leggyakoribb IAM és zero trust hibákat, amelyeket az architektúra tervezése során el kell kerülni:

1. **Shared secrets és admin megosztása:** A `root` felhasználó, a közös admin jelszó vagy egy közös API kulcs használata láthatatlanná teszi az auditot, és ellehetetleníti a felelősségre vonást.
    - *Helyes megoldás:* Minden entitásnak egyedi identitást adunk, és a jogosultságokat ehhez az identitáshoz rendeljük. A rendszer-adminisztrációhoz is JIT hozzáférést használunk.
2. **A hálózati pozíció hitelesítésként való használata:** Továbbra is olyan szabályokat írunk, hogy „a `10.0.0.0/8` hálózatról érkező kérések megbízhatóak”.
    - *Helyes megoldás:* A hálózati cím nem bizonyít identitást. Minden kéréshez identitást és hitelesítést kell kötni, a PEP-nek minden korábban hitelesített munkamenetet is ellenőriznie kell a PDP-n.
3. **Túlságosan hosszú élettartamú tokenek és kulcsok:** Egy évre érvényes API kulcs vagy access token hatalmas kockázatot jelent, mert ha kiszivárog, hosszú ideig használható.
    - *Helyes megoldás:* Rövid élettartamú (pl. 1 órás) access tokenek és rendszeres rotáció. A refresh tokeneket is védett tárolóban kell tartani.
4. **Az ID token használata jogosultság-ellenőrzésre:** Ez az egyik legveszélyesebb és leggyakoribb hiba az OIDC-alapú rendszerekben.
    - *Helyes megoldás:* Az access token-t használjuk az API jogosultságainak ellenőrzésére. A token `scopes` mezője vagy a token introspection válasza alapján döntünk.
5. **Jogosultságok a kódban történő elszórása:** A PDP logikáját nem központilag, hanem minden szolgáltatás kódjában külön-külön implementáljuk.
    - *Helyes megoldás:* A policy döntéseket központosítjuk. A PDP-t mint szolgáltatást vagy policy engine-t (OPA, Kyverno, Vault) használjuk. A szolgáltatások a PDP-t kérdezik meg, és a kapott döntést hajtják végre.

21.2.6 6. Labor: Identitás- és hozzáférési mátrix tervezése

Ebben a laborban a cél egy kis, jól ellenőrizhető rendszer identitás- és hozzáférési mátrixának elkészítése. A feladat megerősíti a least privilege és a zero trust elveinek gyakorlati alkalmazását.

21.2.6.1 6.1. Kontextus

Képzeljünk el egy egyszerű e-kereskedelmi platformot. A rendszer a következő komponensekből áll:

- **Webshop Frontend (SPA):** A felhasználók böngészőjében futó alkalmazás.
- **API Gateway (PEP):** Minden bejövő kérés belépési pontja.

- **Order Service:** A rendelések kezeléséért felelős szolgáltatás.
- **Catalog Service:** A termékkatalógust kezelő szolgáltatás.
- **Payment Service:** A fizetési tranzakciókat kezelő szolgáltatás.
- **Inventory Service:** A raktárkészletet kezelő szolgáltatás.
- **Reporting Dashboard:** Egy belső eszköz, amelyet az üzleti elemzők használnak a rendelések elemzésére.
- **Identity Provider (IdP):** Az OIDC hitelesítésért felelős komponens.

A felhasználói szerepkörök:

- `customer`: az ügyfél, aki böngészi a katalógust és rendelést ad le.
- `support`: az ügyfélszolgálati munkatárs, aki lekérdezheti a rendeléseket és frissítheti a szállítási címet.
- `analyst`: az üzleti elemző, aki a riportokat nézi.
- `service_catalog`, `service_order`, `service_payment`, `service_inventory`: a mikroszolgáltatások identitásai.

21.2.6.2 6.2. Feladat

Tervezd meg a hozzáférési mátrixot! Minden sor egy identitást, minden oszlop egy erőforrást/komponenst jelöl. A cellákba írd be a **minimálisan szükséges** műveleteket (pl. `read`, `write`, `create`, `delete`, `pay`, `nothing`).

Az erőforrások (oszlopok):

- **Catalog UI:** a katalógus böngészése.
- **Order API:** a rendelések létrehozása és lekérdezése.
- **Payment API:** a fizetés indítása.
- **Inventory API:** a raktárkészlet frissítése és lekérdezése.
- **Reporting Dashboard:** a riportok megtekintése.

21.2.6.3 6.3. Elvárt eredmény

A mátrixodban a `customer` identitás a `Payment API`-hoz `create` hozzáférést kapjon, de a `delete` műveletet nem. A `support` identitás az `Order API`-hoz `read` és `write` hozzáférést kapjon, de a `Payment API`-hoz semmilyen hozzáférést. Az `analyst` identitás kizárólag a `Reporting Dashboard`-hoz férjen hozzá `read` jogosultsággal. A szolgáltatások közötti hívásoknál a `service_order` kérhet `read` hozzáférést a `Catalog API`-hoz, de a `service_catalog` sosem kérhet hozzáférést a `Payment API`-hoz. Ez utóbbi hívás egy anti-minta, amelyet a hálózati policy-kkel és a jogosultságokkal is blokkolni kell.

21.2.6.4 6.4. Ellenőrzési szempontok

1. A `customer` szerepkörhöz tartozó `delete` jogosultság a `Payment API`-n senkinek sincs meg. **Least privilege teljesül.**
2. A `service_inventory` tud írni az `Inventory API`-ba, de a `service_order` csak olvasni tud. **Elválasztás teljesül.**
3. Az összes `service_*` identitás `write` hozzáféréssel rendelkezik a saját szolgáltatásához. **Szolgáltatásidentitás elkülönül.**
4. A `Reporting Dashboard` nem érhető el semmilyen `service_*` identitás számára. **Adatáramlás a biztonságos zónán belül marad.**

21.2.6.5 6.5. Döntési napló

Rögzíts a labor dokumentumodban legalább egy ADR-t, amelyben leírod, miért a `customer` szerepkör nem kap `delete` jogosultságot a rendeléseihez. Miért jobb a rendeléseket inkább `cancelled` állapotba állítani, mint fizikailag törölni? A döntés a `Payment API` és az `Order API` közötti hívás irányát is érinti: miért az `Order Service` hívja a `Payment Service`-t, és miért nem fordítva? Az ADR-ben tüntesd fel a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.

21.2.7 7. Ellenőrzőlista

- ☐ Minden felhasználói és szolgáltatás identitás egyedi, központilag kezelt.
- ☐ A hitelesítés központi IdP-n keresztül történik, az MFA elő van írva.
- ☐ Az API-k engedélyezése access token-ekkel történik, nem ID token-ekkel.
- ☐ A szolgáltatások közötti kommunikáció mTLS-sel vagy rövid élettartamú workload identity token-nel védett.
- ☐ A hozzáférési döntések (PDP) központosítottak, a kikényszerítés (PEP) a belépési pontokon történik.
- ☐ A jogosultságok a least privilege elvét követik, nincs örökérvényű admin jog.
- ☐ A hálózati pozíció (belső IP) nem szerepel a hozzáférési döntésekben.
- ☐ A konfigurációból és a kódból eltávolítottuk az összes shared secretet.

21.2.8 8. Önellenőrző kérdés

Tegyük fel, hogy a rendszeredben egy `support` felhasználó a böngészőjéből indított kéréssel el szeretne érni egy, a `Payment Service`-en belüli belső admin végpontot. A hálózati tűzfal ezt a forgalmat engedi, mert a kérés a böngészőből a belső hálózatra érkezik. Milyen zero trust komponenseknek és milyen szabályoknak kell ezt a kérést megállítaniuk? Válaszodban indokold meg, miért nem elegendő a hálózati tűzfal.

A fejezet bemutatta a zero trust filozófiát, az IAM alapjait, és a hozzájuk tartozó protokollokat. A következő egységben ezekre az alapokra építve a titkok, a kulcsok és az adatvédelem architektúráját vizsgáljuk meg.

21.3 Titkok, kulcsok és adatvédelem

A titkok védelme nem egyszerűen egy újabb biztonsági réteg, hanem a rendszer bizalmi modelljének egyik alapköve. Amikor egy szolgáltatás adatbázishoz csatlakozik, külső API-t hív, üzenetsort olvas, vagy éppen adatot titkosít, minden művelet mögött ott áll valamilyen hitelesítő adat, kulcs vagy token. Ha ezek közül bármelyik kiszivárog, a támadó sokszor nem a rendszer egy sebezhetőségét használja ki, hanem egyszerűen belép azon az ajtón, amelyet a titok nyit ki. Ebben az alfejezetben a titkok teljes életciklusát és az adatvédelem három gyakorlati eszközét – a naplózást, a maszkolást és az adatminimalizálást – járjuk körül. A cél nem egy adott termék bemutatása, hanem azoknak a döntési pontoknak a feltárása, amelyeket minden architektnek ismernie kell ahhoz, hogy a titkok kezelése mérhető, auditálható és üzemeltethető legyen.

21.3.1 Titkok és kulcsok rendszertana

Titoknak nevezünk minden olyan digitális értéket, amely hozzáférést biztosít valamilyen erőforráshoz, vagy képessé tesz egy bizalmas művelet végrehajtására. A leggyakoribb titoktípusok:

- felhasználói jelszavak és jelszókivonatok,
- API-kulcsok és hozzáférési tokenek,
- adatbázis-kapcsolati sztringek,
- TLS-tanúsítványok privát kulcsai,
- titkosítási kulcsok (szimmetrikus és aszimmetrikus),
- aláírási kulcsok,
- OAuth client secret értékek,
- SSH-kulcsok és szolgáltatáskulcsok.

Fontos megkülönböztetni a titkot a hozzá tartozó azonosítótól. Az API-kulcsnál például a `api_key_id` nem titok, a hozzá tartozó kulcsérték viszont igen. Ugyanez igaz a felhasználónév és jelszó párosra. Az architektúra szempontjából a titokhoz tartozó metaadatok – tulajdonos, érvényességi idő, jogosultsági kör, rotációs állapot – ugyanolyan fontosak, mint maga a titkos érték.

A kulcsok a titkok egy speciális csoportját alkotják: kriptográfiai műveletekhez használjuk őket. Itt különbséget teszünk adattitkosítási kulcs (Data Encryption Key, DEK) és kulcstitkosítási kulcs (Key Encryption Key, KEK) között. A DEK magát az adatot titkosítja, a KEK pedig a DEK-et védi. Ezt a hierarchiát nevezzük borítékoló titkosításnak (envelope encryption), amelyet később részletesen bemutatunk.

21.3.2 A titkok életciklusa

A titkok kezelését nem lehet egyetlen pillanatképpel leírni. Minden titok egy folyamaton megy keresztül a létrehozásától a megsemmisítéséig, és a folyamat minden egyes állomásán más-más hibaforrás jelentkezik. Az életciklus jellemző állomásai a következők:

1. **Létrehozás.** A titoknak kellően véletlenszerűnek kell lennie ahhoz, hogy ne legyen kitalálható vagy visszafejthető. Egy jelszó vagy kulcs generálásakor kriptográfiai szempontból erős véletlenszám-generátort kell használni. A rövid, ember által könnyen megjegyezhető értékek általában gyengék. Hiba, ha a generálás determinisztikus, például a szolgáltatásnév és egy dátum összefűzéséből áll.

2. **Tárolás.** A titkot soha nem tároljuk forráskódban, verziókezelőben, dokumentációban vagy plaintext konfigurációs fájlban. A helyes megoldás egy központi titoktár (secrets manager) vagy kulcskezelő szolgáltatás (Key Management Service, KMS). A titoktár titkosítva tárolja az értékeket, hozzáférést szabályoz, és minden olvasási műveletet naplóz. A tárolás része a verziózás is: amikor egy titok értéke megváltozik, a régi érték visszakereshető maradhat, de alapértelmezésben le kell tiltani a régi verzió használatát.

3. **Elosztás.** A titoknak el kell jutnia ahhoz a komponenshez, amelynek szüksége van rá. A legbiztonságosabb megoldás a futásidőben történő lekérés: az alkalmazás induláskor vagy szükség esetén a titoktár API-ján keresztül kéri le az értéket, és nem környezeti változóban, image-ben vagy ConfigMapben kapja meg előre. Ha a titok hosszú életű, akkor a rendszer induláskor lekért értéket a memóriában tartjuk, és rendszeresen frissítjük. A rövid élettartamú, dinamikus titkok – például egy adatbázis-hozzáférés, amelyet a titoktár a kérés pillanatában hoz létre – csökkentik a kiszivárgás hatását.

4. **Használat.** A titkot a felhasználó komponens memóriájában, lehetőleg védett memóriaterületen kell tartani. A használat során a titok értéke nem kerülhet naplóba, stack trace-be, hibajelentésbe vagy bármilyen megfigyelhető csatornába. A hibakereséshez hasonlóan fontos, hogy a fejlesztői környezet sem használhat éles titkokat.

5. **Rotáció.** Minden titoknak rendelkeznie kell élettartammal. A rendszeres rotáció azt jelenti, hogy a titok értékét automatikusan vagy félautomata módon, előre meghatározott időközönként lecseréljük. A rotáció gyakorisága függ a titok típusától és a kockázattól: egy API-kulcsnál jellemzően 30–90 nap, egy TLS-tanúsítványnál 1 év, egy rövid életű token esetén néhány perc. A rotáció során az új és a régi titok átmeneti együttélése szükséges lehet, hogy a frissítés ne okozzon kiesést. A rotáció elmaradása az egyik leggyakoribb üzemeltetési kockázat.

6. **Visszavonás.** Ha a titok kompromittálódásának gyanúja merül fel, azonnal vissza kell vonni, vagyis érvényteleníteni kell minden olyan rendszerben, ahol használták. A visszavonás nem azonos a rotációval: a rotáció tervezett, a visszavonás rendkívüli esemény. A visszavonás hatékonysága attól függ, hogy a titok hány helyen volt használatban, és mennyire volt látható a használati napló.

7. **Megsemmisítés.** Amikor a titokra már nincs szükség, biztonságosan meg kell semmisíteni. A titoktárban a törlés általában logikai törlést jelent először, majd egy megőrzési idő után fizikai törlést. A megsemmisítésnek ki kell terjednie minden másolatra is, beleértve a biztonsági mentéseket, a CI/CD rendszerek cache-eit és a fejlesztői gépeket.

Az életciklus állomásait egy folyamatábrán is rögzíthetjük diagrams-as-code formában. A következő Mermaid-diagram szemlélteti a fő átmeneteket:

```
flowchart TD
    A[Titok létrehozása] --> B{Tárolás titoktárban}
    B -->|lekérés futásidőben| C[Elosztás a fogyasztóhoz]
    C --> D[Használat a memóriában]
    D --> E{Rotáció esedékes?}
    E -->|igen| F[Új titok generálása]
    F --> G[Régi és új párhuzamos használata]
    G --> H[Régi titok visszavonása]
    H --> D
    E -->|kompromittálódás gyanúja| I[Azonnali visszavonás]
    I --> J[Minden használó értesítése]
    J --> F
    D -->|élettartam vége| K[Logikai törlés]
    K --> L[Fizikai megsemmisítés]
```

Cím: A titkok életciklusának egyszerűsített folyamata. Jelmagyarázat: a rombusz döntési pont, a téglalap tevékenység. Feltételjegyzet: a rotáció és a visszavonás egyaránt új titok generálásához vezet, de a visszavonás esetén a régi titkot azonnal meg kell semmisíteni, míg a rotáció során a párhuzamos használat biztosítja a folyamatos működést.

A fenti diagram nem egy konkrét rendszer, hanem egy ellenőrző keret. Minden titoktípusra érdemes hasonló állapotgépet definiálni, és a rendszer dokumentációjában rögzíteni, hogy az adott titok melyik állapotban éppen hol található.

21.3.3 Titoktár és kulcskezelő szolgáltatás

A titoktár (secrets manager) egy központi szolgáltatás, amely a titkok tárolását, verziózását, hozzáférés-vezérlését és auditálását biztosítja. A titoktár legfontosabb jellemzői:

- **Központi tárolás:** a titkok egyetlen, jól védett helyen élnek, nem szóródnak szét a konfigurációs fájlokban.
- **Titkosítás at rest:** a titoktár a benne tárolt értékeket általában egy mesterkulccsal titkosítja, amelyet gyakran hardveres biztonsági modulban (HSM) őriznek.
- **Hozzáférésszabályozás:** minden titokhoz külön engedély rendelhető. Az a komponens, amelynek nincs szüksége egy adatbázis-jelszóra, ne is tudja lekérni.
- **Auditnapló:** minden olvasási, írási, rotációs és törlési művelet rögzítésre kerül, így utólag visszakereshető, hogy ki és mikor fért hozzá a titokhoz.
- **Verziózás:** a titok értékének megváltozása új verziót hoz létre, és a régi verzió átmenetileg megőrizhető a fokozatos átálláshoz.
- **Rotáció támogatása:** a titoktár gyakran képes automatikusan új titkot generálni, és értesíteni a felhasználókat a cseréről.

A titoktár használata nem mentesít minden felelősség alól. Ha az alkalmazás a titkot lekéri, majd a memóriában tárolja, akkor a titoktár nem látja, hogy a fogyasztó mit csinál az értékkel. Ezért a titoktárat mindig együtt kell használni a naplózási és monitorozási irányelvekkel.

Gyakori integrációs minta a titoktár és a konténerplatform együttműködése. A Kubernetes például rendelkezik beépített Secret erőforrással, de ez önmagában nem titkosítja a titkokat, hacsak a klaszter nincs megfelelően konfigurálva. Éppen ezért a produkciós rendszerekben gyakran külső titoktárat használnak, ahonnan a konténerek futásidőben, a titoktár API-ján keresztül kérnek le értékeket. A konfigurációban ilyenkor nem maga a titok, hanem csak egy hivatkozás szerepel:

```
# Szemléltető példa – nem éles rendszerhez
apiVersion: v1
kind: Pod
metadata:
  name: demopay-billing
spec:
  containers:
    - name: billing
      image: registry.invalid/demopay/billing:1.0.0
      env:
        - name: DB_PASSWORD_REF
          value: "secret://demopay/postgres/password"
```

A fenti példában a `DB_PASSWORD_REF` nem a jelszót tartalmazza, hanem egy hivatkozást a titoktárban tárolt értékre. Az alkalmazás induláskor a titoktárhoz fordul, és a hivatkozás alapján kéri le a tényleges jelszót. A hivatkozás tehát nem titok, de érzékeny metaadat lehet, ezért a hozzáférést ugyanúgy szabályozni kell.

21.3.4 KMS és borítékoló titkosítás

A kulcskezelő szolgáltatás (Key Management Service, KMS) a kriptográfiai kulcsok életciklusát kezeli: létrehozza, tárolja, rotálja és visszavonja a kulcsokat. A KMS általában hardveres biztonsági modulokban (HSM) tárolja a kulcsokat, és az alkalmazások számára API-n keresztül biztosít hozzáférést. A KMS önmagában nem tárolja a titkokat, hanem a kulcsok védelmét végzi.

A borítékoló titkosítás (envelope encryption) egy olyan minta, amely szétválasztja az adat titkosítását és a kulcs védelmét. A gyakorlatban ez a következőképpen működik:

1. Az alkalmazás kér a KMS-től egy adattitkosítási kulcsot, a DEK-et.

2. A KMS válaszként visszaadja a DEK-et plaintext formában, valamint a DEK titkosított változatát, amelyet a mesterkulccsal (KEK) titkosít.
3. Az alkalmazás a plaintext DEK-kel titkosítja a felhasználói adatot.
4. A titkosított adat mellett eltárolja a titkosított DEK-et.
5. A plaintext DEK-et a memóriából azonnal törli.

Visszafejtéskor az alkalmazás elküldi a titkosított DEK-et a KMS-nek, amely a mesterkulccsal visszafejti, majd az alkalmazás a plaintext DEK-kel visszafejti az adatot. A mesterkulcs soha nem hagyja el a KMS-t, és a KMS a mesterkulcs rotációját is kezeli.

Az envelope encryption előnye, hogy nagy mennyiségű adatot is hatékonyan lehet titkosítani anélkül, hogy a mesterkulcsot minden egyes adatművelethez használni kellene. A DEK jóval rövidebb életű, mint a KEK, ezért a DEK rotációja nem jár a teljes adatállomány újratitkosításával. Ha a DEK kiszivárog, csak az adott adatblokk sérül, nem a teljes kulcshierarchia.

A következő Mermaid-diagram az envelope encryption adatfolyamát mutatja:

```
sequenceDiagram
    participant A as Alkalmazás
    participant K as KMS
    participant S as Adattár
    A->>K: Kérj DEK-et
    K-->>A: plaintext DEK + titkosított DEK
    A->>S: Titkosított adat + titkosított DEK
    A->>A: Plaintext DEK törlése a memóriából
    Note over A,S: Későbbi visszafejtéskor az alkalmazás a titkosított DEK-et elküldi a
↪   KMS-nek,<br/>majd a visszakapott plaintext DEK-kel fejti vissza az adatot.
```

Cím: Envelope encryption adatfolyama. Jelmagyarázat: a szaggatott nyíl válaszüzenet. Feltételjegyzet: a plaintext DEK csak a memóriában létezik, és a művelet befejezése után törlődik.

A KMS és a titoktár gyakran együttműködik. A titoktár a benne tárolt titkokat általában a KMS mesterkulcsával titkosítja. Ezért a KMS kulcsainak védelme az egész rendszer biztonságának alapja. A KMS-kulcsok rotációját és hozzáférését ugyanolyan szigorúan kell kezelni, mint a titkokat.

21.3.5 Naplózás, maszkolás és adatminimalizálás

A titkok védelme szorosan összefügg a naplózással. A naplók egyszerre jelentenek biztonsági ellenőrzési eszközt és potenciális adatforrást a támadó számára. Ezért a naplózás során két egymásnak feszülő követelményt kell összehangolni: egyrészt rögzíteni kell, hogy ki és mikor fért hozzá egy erőforráshoz, másrészt meg kell akadályozni, hogy a titok értéke a naplóba kerüljön.

A hozzáférés naplózása az auditálhatóság alapja. Minden titoktár-műveletről – lekérés, létrehozás, rotáció, törlés – auditbejegyzésnek kell készülnie. A bejegyzés tartalmazza a kérés időpontját, a hívó fél azonosítóját, a titok azonosítóját és a művelet típusát. A titok értékét azonban soha nem tartalmazhatja. Ha például egy alkalmazás hibát dob, és a hibaüzenetben szerepel a titok, akkor a napló azonnal kompromittálódik.

A maszkolás az érzékeny adatok részleges elrejtése a naplóban és a felhasználói felületen. Egy hitelkártyaszám esetén a `4111 11** **** 1111` formátum megmutatja az utolsó négy számjegyet, de a teljes értéket nem fedi fel. A maszkolás nem titkosítás: a maszkolt értékből az eredeti adat nem állítható helyre, viszont a felismerhetőség megmarad. A maszkolás különösen fontos a naplózásban, ahol a kártyaszám, a bankszámlaszám, a személyi azonosító vagy a telefonszám helyett csak a maszkolt változat kerülhet rögzítésre.

A tokenizálás egy másik adatvédelmi technika. A tokenizálás során az érzékeny adatot egy véletlenszerű tokenre cseréljük, és a kapcsolatot egy biztonságos, elkülönített tárban őrizzük. A token önmagában nem hordozza az eredeti adatot, ezért a naplóban szerepelhet anélkül, hogy a titok kiszivárogna. A tokenizálás eltér a titkosítástól: a titkosított értékből a kulcs birtokában visszaállítható az eredeti adat, míg a tokenből csak a token-szolgáltatás segítségével. A tokenizálás előnye, hogy a naplózott token nem minősül személyes adatnak, ha a token-szolgáltatás megfelelően van kialakítva.

Az adatminimalizálás elve azt mondja ki, hogy csak annyi adatot gyűjtsünk és tároljunk, amennyi a cél eléréséhez feltétlenül szükséges. Ez az elv a titkokra is igaz: ha egy szolgáltatásnak nincs szüksége egy adatbázis-jelszóra, ne is kapja meg. Ha egy naplónak nincs szüksége a teljes IP-címre, tároljuk csak a hálózati prefixet. Az adatminimalizálás csökkenti a támadási felületet, és megkönnyíti a szabályozási megfelelést is.

A következő példa két naplóbejegyzést mutat be – a helytelent és a helyeset:

Helytelen naplóbejegyzés:

```
2025-06-01T10:15:30Z ERROR database connection failed:
↪ postgres://demopay_app:SuperSecret123!@db.invalid:5432/billing
```

Helyes naplóbejegyzés:

```
2025-06-01T10:15:30Z ERROR database connection failed:
↪ secret_ref=secret://demopay/postgres/password db_host=db.invalid db_port=5432
↪ db_name=billing
```

Az első bejegyzésben a kapcsolati sztring és a jelszó is szerepel, ami azonnali kompromittálódást jelent. A második bejegyzés csak a titok hivatkozását és a kapcsolódási paramétereket tartalmazza, így a hiba diagnosztizálható anélkül, hogy a titok kiszivárogna.

21.3.6 Tipikus hibák és ellenintézkedések

A titkok kezelésében a leggyakoribb hibák jól ismertek, és mindegyikhez egyértelmű ellenintézkedés tartozik:

- **Hardcode-olt titok a forráskódban.** A fejlesztő a kényelem kedvéért beírja a jelszót a kódba. *Ellenintézkedés:* a kódban csak hivatkozás szerepel, a titkot futásidőben a titoktárból kell lekérni.
- **Titok a verziókezelőben.** A `.env` fájl vagy a `secrets.yaml` bekerül a Git repóba. *Ellenintézkedés:* a `.gitignore` és a pre-commit hook megakadályozza a titok bekerülését, továbbá a repókat rendszeresen átvizsgálják szivárgásra.
- **Titok a naplóban.** A hibakeresés során az alkalmazás kiírja a titkos értéket. *Ellenintézkedés:* a naplózási keretrendszer maszkolási szabályokat alkalmaz, és a titkokat soha nem írja ki.
- **Elmaradt rotáció.** A titok évekig változatlan marad, ezért a kompromittálódás hosszú ideig észrevétlen. *Ellenintézkedés:* minden titokhoz rotációs ütemezés tartozik, amelyet a titoktár automatikusan végrehajt.
- **Túl széles hozzáférés.** Egy szolgáltatás olyan titkokhoz is hozzáfér, amelyekre nincs szüksége. *Ellenintézkedés:* a titoktár engedélyei a legkisebb jogosultság elvét követik, és rendszeresen felülvizsgálják őket.
- **Plaintext tárolás konfigurációs fájlban.** A jelszó a `config.yaml`-ben olvasható formában szerepel. *Ellenintézkedés:* a konfiguráció csak hivatkozást tartalmaz, a titok a titoktárban él.

E hibák közös gyökere gyakran az, hogy a titokkezelést a fejlesztői kényelemre optimalizálják a biztonság rovására. Az architekt szerepe az, hogy a titokkezelési mintákat úgy alakítsa ki, hogy a biztonságos megoldás legyen a legegyszerűbb is.

21.3.7 Labor: titokkezelési és kulcsrotációs terv

A következő labor célja, hogy a fejezetben tanult fogalmakat egy kis, ellenőrizhető példán alkalmazzuk. A labor nem éles rendszeren fut, és minden érték helyőrző. A feladat egy titokkezelési és kulcsrotációs terv elkészítése egy fiktív, `DemoPay` nevű számlázási szolgáltatáshoz.

21.3.8 Kiinduló állapot

A `DemoPay` szolgáltatás jelenleg a következő titkokat használja:

- PostgreSQL adatbázis-jelszó (`db_password`),
- külső fizetési API-kulcs (`payment_api_key`),
- TLS-tanúsítvány privát kulcsa (`tls_private_key`),
- belső szolgáltatásközi API token (`service_token`).

Mindegyik titok jelenleg plaintext formában, egy `config.yaml` nevű fájlban található, amely a Git repóban van. A titkokat soha nem rotálják, és a naplókban több alkalommal is megjelent a `db_password` értéke.

21.3.9 Feladat

Készítsen reprodukálható dokumentumot, amely a következőket tartalmazza:

1. **Kontextus rögzítése.** Írja le a `DemoPay` szolgáltatás rendeltetését, a titkok listáját, a fogyasztó komponenseket és a jelenlegi kockázatokat. Használjon helyőrző neveket és értékeket.
2. **Döntési napló (ADR).** Rögzítse a titoktár és a KMS kiválasztását. Az ADR tartalmazza a kontextust, a döntést, a feladott alternatívákat, a következményeket és az állapotot. Például:
   - *Kontextus:* a `DemoPay` titkai jelenleg plaintext konfigurációban élnek, és nincs rotáció.
   - *Döntés:* bevezetünk egy központi titoktárat a titkok tárolására, és egy KMS-t az envelope encryptionhöz.
   - *Alternatívák:* (a) titkosított konfigurációfájl a repóban, (b) Kubernetes Secret önmagában, (c) központi titoktár és KMS.
   - *Következmények:* a titkok hozzáférése auditálhatóvá válik, de a szolgáltatás indulásakor a titoktár elérhetősége kritikus függőséggé válik.
   - *Állapot:* elfogadva.
3. **Életciklus-terv minden titokra.** Táblázat formájában adja meg a titok típusát, a generálás módját, a tárolás helyét, az elosztás módját, a rotáció időtartamát és a visszavonási eljárást. Például a `db_password` esetén a rotáció 30 nap, a `tls_private_key` esetén 1 év.
4. **Rotációs folyamatábra.** Készítsen Mermaid-diagramot, amely bemutatja a `db_password` rotációjának lépéseit, beleértve az új jelszó generálását, a régi és új párhuzamos használatát, majd a régi visszavonását.
5. **Naplózási és maszkolási szabályok.** Definiálja, mely naplóbejegyzések megengedettek, és melyek tiltottak. Adjon példát egy helyes és egy helytelen bejegyzésre.

6. **Ellenőrzési szempontok.** Sorolja fel, milyen ellenőrzésekkel lehet igazolni, hogy a terv működik. Például: nincs titok a verziókezelőben; a rotáció lefutott az ütemezés szerint; a naplókban nem szerepel titokérték.

21.3.10 Várt és mért eredmény elkülönítése

A dokumentumban külön szakaszban rögzítse a **várt eredményt** és a **mért eredményt**. Például:

- *Várt eredmény:* a rotáció 30 naponként automatikusan lefut, és a szolgáltatás rendelkezésre állása a rotáció közben nem csökken 99,95% alá.
- *Mért eredmény:* egy izolált tesztkörnyezetben 3 egymást követő rotációt hajtottunk végre 24 óra alatt, és a szolgáltatás válaszideje a rotáció alatt a 95. percentilisben nem haladta meg a 250 ms-ot. A mért értékeket a tesztnapló tartalmazza.

A mért eredményt nem szabad kitalálni; ha még nem futott mérés, a dokumentumban azt kell írni, hogy „nem mért”.

21.3.11 Ellenőrzési szempontok a laborhoz

- A dokumentum tartalmazza mind a négy titoktípus életciklusát.
- Az ADR-ben a kontextus, a döntés, az alternatívák és a következmények egyértelműen elválnak.
- A rotációs folyamatábra diagrams-as-code formában készült, és jelmagyarázatot tartalmaz.
- A naplózási szabályok konkrét példát mutatnak helyes és helytelen bejegyzésre.
- A várt és a mért eredmény világosan el van különítve.

21.3.12 Visszaállítási út

A labor nem éles rendszeren fut, ezért nincs szükség éles visszaállításra. Ha a hallgató mégis izolált környezetben futtat PoC-t, a labor végén minden létrehozott titkot törölni kell, és a titoktár tesztpéldányát le kell állítani.

21.3.13 Ellenőrzőlista a titkok kezeléséhez

☐ Minden titok egy központi titoktárban van tárolva, nem forráskódban vagy verziókezelőben.
☐ Minden titokhoz tartozik rotációs ütemezés, és a rotáció automatizált.
☐ A titoktár minden hozzáférési műveletet auditál.
☐ A naplók nem tartalmaznak titokértéket; a kötelező érzékeny mezők maszkoltak.
☐ A titkok hozzáférése a legkisebb jogosultság elvét követi.
☐ A kulcskezelő szolgáltatás mesterkulcsa soha nem hagyja el a KMS-t.
☐ Az envelope encryptionhöz használt DEK-ek rövid életűek, és a memóriából használat után törlődnek.
☐ A titokkompromittálódásra vonatkozó vészforgatókönyv dokumentált és tesztelt.

21.3.14 Önellenőrző kérdés

Mi a különbség a maszkolás, a titkosítás és a tokenizálás között, és mikor melyiket érdemes választani a naplózáskor? Válaszában térjen ki arra, hogy a maszkolt értékből helyreállítható-e az eredeti adat, és hogy a tokenizált érték miért lehet alkalmas naplózásra anélkül, hogy a titok kiszivárogna.

21.3.15 Továbbvivő gondolat

A következő alfejezet az ellátási lánc biztonságával foglalkozik. Az itt bemutatott kulcsfogalmak – különösen a KMS, a privát kulcsok és a titkosítás – közvetlenül épülnek be az SBOM-ok, az aláírások és a SLSA-fokozatok világába. A titkok életciklusának ismerete nélkül nem érthető meg, miért olyan fontos a build-aláírási kulcsok védelme és a szoftverkomponensek ellenőrzött elosztása.

21.4 Ellátási lánc biztonsága: SBOM, SLSA és aláírás

A szoftverellátási lánc biztonsága az elmúlt években vált az architekturális tervezés és az üzemeltetés egyik központi kérdésévé. A fejlesztés és a build folyamata nem áll meg a forráskód határainál: a modern rendszerek több száz vagy több ezer közvetlen és tranzitív függőséget húznak be, a forráskódot buildrendszerek fordítják le, az artefaktumokat registrykben tárolják, majd konténerimage-ekbe vagy csomagkezelőkbe csomagolják. Ha ebben a láncban bármelyik lépést kompromittálják, a támadó anélkül juttathat kártékony kódot az éles rendszerbe, hogy magát a tárolt forráskódot módosítaná. Ez a fejezet a függőségek és a build-artefaktumok bizalmasságával foglalkozik: azzal a kérdéssel, hogy honnan tudjuk, hogy az a kódrészlet, amelyet telepítünk, tényleg az-e, amit a fejlesztő írt, és hogy a build lépései között nem történt-e illetéktelen beavatkozás.

A fogalomhasználatban a „bizalmasság” itt nem a titkosságra, hanem a bizalomra utal: egy artefaktummal szemben megkövetelt bizalmi tulajdonságokra. Az integritás (sértetlenség), a hitelesség (az aláíró azonosíthatósága) és a származás (provenance – ki, mikor, milyen forrásból, milyen eszközzel készítette) együtt adják azt a bizalmi láncot, amely a forráskódtól a futó konténerig terjed. A fejezet célja, hogy ezt a láncot mérnöki eszközökkel – SBOM, SLSA, aláírás-ellenőrzés és reprodukálható build – átláthatóvá és ellenőrizhetővé tegye.

21.4.1 A függőségek és a build-artefaktumok bizalmi modellje

A hagyományos szoftverfejlesztés bizalmi modellje viszonylag egyszerű volt: a forráskódot a fejlesztő írta, a buildet a release-menedzser indította, az elkészült binárist a sysadmin telepítette. A modern, felhőalapú fejlesztési lánc ezt a modellt jelentősen átalakította. A forráskód gyakran függőségeken keresztül más projektekből származik; a buildet elosztott CI-rendszerek futtatják, amelyek maguk is konténerekben, ideiglenes környezetekben dolgoznak; az artefaktumokat nyilvános vagy privát registryk tárolják; a telepítés pedig automatizált, gyakran a konténer image-ének frissítésével történik.

Ebben a modellben a támadó több ponton is beavatkozhat:

1. **A forrásfüggőségben**: egy népszerű nyílt forráskódú könyvtár új verziójába kártékony kód kerül (például maintainer-fiók átvétele vagy rosszindulatú PR révén).

2. **A buildrendszerben**: a CI/CD pipeline-t futtató szerveren lecserélik a fordítót, vagy a build definícióját módosítják.
3. **Az artefaktum-tárolóban**: a registryben tárolt artefaktumot felülírják egy módosított változattal.
4. **A szállítási folyamatban**: a letöltés során a hálózaton közbeékelődő támadó az artefaktumot kicseréli.

A bizalmi lánc tehát nem egyetlen lépésből áll, hanem a forrástól a futtatásig tart. Az ellátási lánc biztonságának célja, hogy minden lépésben rendelkezésre álljon egy ellenőrizhető, kriptográfiai garanciákkal alátámasztott bizonyíték arról, hogy az adott artefaktum az elvárt forrásból, az elvárt lépéseken keresztül, érintetlenül érkezett.

A „build-artefaktum" fogalma minden olyan terméket magában foglal, amelyet a forráskódból állítunk elő, és amelyet telepítünk vagy felhasználunk: konténer image, programkönyvtár (például `.jar`, `.whl`, `.npm` csomag), bináris, dokumentációs csomag, de akár a Terraform-modul vagy a Helm-chart is. A függőség ezzel szemben olyan összetevő, amelyet nem mi állítunk elő, de a rendszerünk része lesz: közvetlen függőség (például egy HTTP-kliens könyvtár) vagy tranzitív függőség (amelyet a közvetlen függőség húz be). A bizalmi lánc mindkettőre kiterjed, mert a sebezhetőség vagy a rosszindulatú módosítás a tranzitív függőségekben éppúgy jelen lehet.

21.4.2 Függőségkezelés: sebezhetőség és frissítés

A függőségek biztonságának első pillére a megfelelő kezelés. Egy függőség akkor biztonságos, ha ismert a pontos verziója, ismert a sebezhetőségi státusza, és a frissítési folyamat szabályozott. A gyakorlatban három fő tevékenységet kell elkülöníteni: a függőségek azonosítását és rögzítését, a sebezhetőségek keresését és a frissítések menedzselését.

21.4.3 Függőségek azonosítása és rögzítése

A legtöbb nyelvi ökoszisztéma rendelkezik lock fájllal, amely a függőségek pontos verzióját és ellenőrzőösszegét rögzíti. A Node.js esetében ez a `package-lock.json`, a Pythonnál a `poetry.lock` vagy a `pip-tools` által generált `requirements.txt`, a Javánál a `build.gradle` mellett a `gradle.lockfile`, a Go-nál a `go.sum`. A lock fájl szerepe kettős: egyrészt reprodukálhatóvá teszi a buildet, mert mindenki ugyanazt a verziót telepíti; másrészt rögzíti a tranzitív függőségek halmazát is, így azok sebezhetőség-kereséskor láthatóvá válnak.

Gyakori hiba, hogy a csapat csak a közvetlen függőségeket rögzíti, a tranzitív függőségeket pedig a csomagkezelő automatikus feloldására bízza. Ilyenkor a build nem reprodukálható, és a sebezhetőség-elemzés sem teljes. A helyes megoldás a lock fájl verziózása a forráskóddal együtt, és annak megkövetelése is, hogy a CI/CD ne végezzen automatikus dependency resolution-t lock fájl nélkül.

Példa egy Node.js projekt `.npmrc` beállítására, amely letiltja a lock fájl nélküli telepítést:

```
# .npmrc - szemléltető példa
package-lock=true
lockfile-version=3
```

A lock fájl mellett ajánlott a függőségek pinelése: a pontos verzió megadása intervallum vagy `latest` helyett. A `"express": "^4.17.1"` helyett a `"express": "4.17.3"` rögzítése azt biztosítja, hogy a telepítés mindig pontosan ezt a verziót hozza, amíg a lock fájl nem változik. Természetesen a pinelés önmagában nem véd a sebezhetőségektől, de lehetővé teszi, hogy a frissítés tudatos döntés legyen, ne pedig a build véletlenszerű mellékhatása.

21.4.4 Sebezhetőségek keresése

A függőségek sebezhetőségeit CVE-azonosítókkal (Common Vulnerabilities and Exposures) és CVSS-pontszámokkal írják le. A sebezhetőség-kereső eszközök – mint az OWASP Dependency-Check, a Trivy, a Grype, a Snyk vagy a GitHub Dependabot – az ismert sebezhetőségek adatbázisaival vetik össze a projekt függőségeit. Az eredmény egy lista, amely megadja, hogy mely függőség melyik CVE-nek van kitéve, mekkora a súlyosság, és milyen verzióra kell frissíteni a javításhoz.

A keresést a CI/CD pipeline részévé kell tenni, nem pedig manuális, időszakos tevékenységgé. A pipeline build lépése után, az artefaktum publikálása előtt futtatott sebezhetőség-scan meghiúsítja a buildet, ha kritikus sebezhetőséget talál. Ehhez az eszközt konfigurálni kell egy megfelelő küszöbértékkel: például a CVSS 7.0 feletti sebezhetőségek blokkolják a kiadást. A küszöbérték meghatározása architekturális döntés: túl alacsony küszöbnél a buildek állandóan megszakadnak, túl magasnál a kritikus sebezhetőségek is átcsúszhatnak.

Példa a Trivy futtatására egy konténer image-en, szemléltető paranccsal:

```
# Trivy scan egy konténer image-re, kritikus sebezhetőségek esetén hibával
trivy image --severity CRITICAL,HIGH --exit-code 1 \
  registry.example.org/myorg/myapp:1.2.3
```

A sebezhetőség-keresés korlátja, hogy csak az ismert sebezhetőségeket találja meg. A nulladik napi sebezhetőségek, a rosszindulatú kódok vagy a konfigurációs hibák ellen más eszközökre van szükség: ezért a sebezhetőség-scan nem helyettesíti az aláírás-ellenőrzést vagy a build integritásának védelmét, hanem kiegészíti azt.

21.4.5 Frissítési stratégia

A függőségek frissítése két okból történhet: biztonsági javítás miatt vagy funkcionális fejlesztés miatt. A két esetet célszerű szétválasztani. A biztonsági frissítéseket rendszerint gyorsan kell alkalmazni, míg a főverziós frissítéseket – amelyek API-változással járhatnak – alapos tesztek és review után szabad bevezetni. A gyakorlatban ez a kettős sebességű frissítési stratégia: a patch és a minor verziók automatikusan jöhetnek (például Dependabot vagy Renovate segítségével), a major verziók pedig manuális döntést igényelnek.

A frissítések automatizálásánál ügyelni kell arra, hogy az automatikus merge nem kerülheti meg a tesztelést. A Dependabot és a Renovate képes pull requestet nyitni, amelyet a CI/CD lefuttat, és csak akkor merge-eli a karbantartó, ha az összes teszt zöld. A „minden frissítés automatikusan menjen be” és a „mindent manuálisan ellenőrzünk” szélsőségei egyaránt rosszak: az előbbi váratlan regressziókat okozhat, az utóbbi miatt a biztonsági javítások késlekednek.

A frissítéssel kapcsolatos tipikus hiba az is, hogy a csapat a sebezhetőség-scan eredményét figyelmen kívül hagyja, mert „az adott függőséget csak tesztekben használjuk”. A sebezhetőség azonban a tesztkörnyezetben is támadási felületet jelenthet, ráadásul a tranzitív függőségek révén a termelési kódba is bekerülhet. A helyes megoldás az, hogy a scan eredményét mindig a teljes függőségi gráfra értelmezzük, és a kritikus sebezhetőségeket a teszthez használt csomagokban is javítjuk vagy eltávolítjuk.

21.4.6 Aláírás, ellenőrzés és reprodukálható build

A függőségkezelés biztosítja, hogy a megfelelő verziókat telepítjük, de nem ad garanciát arra, hogy az artefaktum nem változott meg a build vagy a szállítás során. Ehhez kriptográfiai aláírásra és a build folyamatának igazolására van szükség.

21.4.7 Digitális aláírás

A digitális aláírás aszimmetrikus kulcspárral történik: az aláíró a privát kulccsal aláírja az artefaktumot (pontosabban annak hash-értékét), a fogyasztó pedig a publikus kulccsal ellenőrzi, hogy az aláírás érvényes, és hogy az artefaktum az aláírás óta nem változott. Az aláírás tehát két dolgot bizonyít: az integritást (a tartalom sértetlen) és a hitelességet (az aláíró birtokában volt a privát kulcsnak).

A build artefaktumok aláírására több eszköz létezik. A konténer image-ek esetében a `cosign` vált széles körben elterjedtté; a csomagkezelők közül az npm támogatja a `npm publish --provenance` és a PGP-aláírást, a Maven Central is aláírja a közzétett artefaktumokat, a Python csomagok esetében pedig a PEP 458 és a TUF (The Update Framework) ad keretet. A kulcskezelés azonban a legtöbb esetben nehéz: ha a privát kulcs a CI-rendszerben tárolt plain text titok, akkor az aláírás nem ér többet, mint egy build log bejegyzés.

A modern megoldás a kulcs nélküli aláírás (keyless signing), amelyet a Sigstore projekt honosított meg. A keyless aláírás során a buildrendszer rövid életű kulcspárt generál, az aláírást elvégzi, majd a publikus kulcsot egy nyilvános, átlátható naplóba (transparency log) rögzíti, az aláíró azonosítását pedig OIDC-alapú workload identity biztosítja. A fogyasztó a transparency logból ellenőrzi, hogy az aláírás létezik, és hogy az aláíró az elvárt szervezet vagy buildrendszer volt. Ezzel megszűnik a hosszú életű privát kulcsok kezelésének problémája.

Példa egy konténer image aláírására `cosign` keyless móddal, szemléltető jelleggel:

```
# Keyless aláírás OIDC-alapú azonosítással
cosign sign \
  --oidc-provider=github \
  --identity-regexp='^https://github.com/myorg/myapp/.+' \
  registry.example.org/myorg/myapp:1.2.3
```

Az ellenőrzés a fogyasztó oldalán történik meg:

```
# Aláírás ellenőrzése
cosign verify \
  --certificate-identity-regexp='^https://github.com/myorg/myapp/.+' \
  registry.example.org/myorg/myapp:1.2.3
```

Fontos megérteni, hogy az aláírás önmagában nem bizonyítja, hogy az artefaktum biztonságos. Csak azt igazolja, hogy az adott aláíró az aláírás pillanatában az artefaktumot hitelesnek tekintette. Ha a buildet már korábban kompromittálták, az aláírás akkor is érvényes lesz. Ezért az aláírást a build folyamatának integritásával együtt kell értelmezni, ami a SLSA keretrendszerhez vezet.

21.4.8 Reprodukálható build

A reprodukálható build azt jelenti, hogy ugyanabból a forráskódból, ugyanazzal a buildkörnyezettel bitről bitre ugyanaz az artefaktum áll elő. Ez nem magától értetődő: a legtöbb buildrendszer időbélyegeket, abszolút elérési utakat, véletlenszerű azonosítókat vagy a környezet verzióit is beépíti a kimenetbe. A reprodukálhatóság megteremtése komoly mérnöki munka, de számos előnnyel jár:

- **Auditálhatóság**: ha két független fél ugyanabból a forrásból ugyanazt az artefaktumot állítja elő, akkor a két kimenet összehasonlítása bizonyítja, hogy a build nem lett kompromittálva.
- **Bizalom a származásban**: a reprodukálhatóság lehetővé teszi, hogy a build igazolása (provenance) ne csak egy állítás legyen, hanem ellenőrizhető tény.
- **Debuggolhatóság**: a hibakeresés során pontosan az a bináris állítható elő, amely a termelésben fut.

A reprodukálható build megvalósításához a buildkörnyezetet is rögzíteni kell: a fordító verzióját, a függőségek pontos verzióját, az operációs rendszert, a környezeti változókat. A konténeres build ebben segít, mert a build image maga is verziózott. A buildnek továbbá kerülnie kell a nem determinisztikus elemeket: az archívumokba kerülő időbélyegek, a párhuzamos futtatásból adódó nem determinisztikus rendezés, a lokális elérési utak. Ezt a Docker image-ek esetében a rétegek rendezett, tartalom-alapú címzése is támogatja, bár a tényleges reprodukálhatóság nyelvi és buildrendszer-függő.

A reprodukálhatóság és az aláírás együtt ad erős garanciát: a reprodukálhatóság biztosítja, hogy a forrásból adott környezetben bárki ugyanazt az artefaktumot hozza létre, az aláírás pedig hitelesíti, hogy az adott artefaktumot az elvárt buildrendszer állította elő. A kettő együtt a SLSA 4-es szintjének alapkövetelménye.

21.4.9 SBOM: Szoftver-összetevőjegyzék

Az SBOM (Software Bill of Materials, szoftver-összetevőjegyzék) az artefaktum összetevőinek géppel olvasható listája. Hasonlóan a fizikai termékek anyagjegyzékéhez, az SBOM megadja, hogy az adott szoftver mely komponensekből áll, ezek milyen verziójúak, milyen licencűek, és milyen függőségi kapcsolatokban állnak egymással. Az SBOM célja kettős: egyrészt átláthatóvá teszi a függőségi fát, másrészt alapot ad a sebezhetőség-kereséshez, a licencmegfelelés ellenőrzéséhez és az incidenskezeléshez.

21.4.10 Formátumok és tartalom

Az SBOM két legelterjedtebb formátuma az SPDX és a CycloneDX. Az SPDX (Software Package Data Exchange) a Linux Foundation által támogatott szabvány, amely elsősorban a licencinformációkra és a csomagok közötti kapcsolatokra koncentrál. A CycloneDX az OWASP projektje, amely kifejezetten a sebezhetőség-keresés és a biztonsági elemzés igényeit tartja szem előtt. Mindkét formátum JSON vagy XML alapú, és tartalmazza a komponens nevét, verzióját, ellenőrzőösszegét (hash), a beszállítót, a licencet és a függőségi kapcsolatokat.

Egy egyszerű SBOM-részlet CycloneDX formátumban, szemléltető jelleggel:

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e671687-395b-41f5-a30f-a58921a69b79",
  "components": [
    {
      "type": "library",
      "name": "express",
      "version": "4.17.3",
      "purl": "pkg:npm/express@4.17.3",
      "hashes": [
        { "alg": "SHA-256", "content": "a94a8ac5..." }
      ]
```

```
    }
  ]
}
```

Az SBOM nem csak a közvetlen függőségeket tartalmazza, hanem a tranzitív függőségeket is, sőt a buildeszközöket és a futási környezet egyes elemeit is felsorolhatja. A teljesség azonban erősen függ az SBOM-generáló eszköz képességeitől: egyes eszközök a lock fájl alapján generálnak SBOM-ot, mások a build artefaktumot elemzik, megint mások a konténer image rétegeit vizsgálják. A legjobb eredményt a build során, a forrás- és a függőség-feloldás pillanatában történő generálás adja, mert ekkor a ténylegesen felhasznált komponensek listája a legpontosabb.

21.4.11 SBOM generálás a pipeline-ban

Az SBOM-ot a CI/CD pipeline-ban kell előállítani, közvetlenül az artefaktum létrehozása után, és az artefaktummal együtt kell tárolni. A generáláshoz használható eszköz például a Syft (konténer image-ekre és fájlrendszerekre), a CycloneDX Maven/Gradle plugin, a CycloneDX Node.js plugin, vagy a Trivy által is támogatott SBOM-generálás. Az SBOM-ot alá kell írni, és az aláírást a megfelelő transparency logba vagy a registrybe kell rögzíteni.

Példa Syft futtatásra, szemléltető paranccsal:

```
# SBOM generálás egy konténer image-ről CycloneDX formátumban
syft registry.example.org/myorg/myapp:1.2.3 \
  -o cyclonedx-json > myapp-1.2.3.sbom.json

# Az SBOM aláírása cosign-nal
cosign sign-blob myapp-1.2.3.sbom.json --output myapp-1.2.3.sbom.json.sig
```

Az SBOM fogyasztói: a biztonsági csapat, amely sebezhetőség-keresésre használja; az üzemeltetés, amely incidens esetén gyorsan azonosítja, hogy mely rendszerek érintettek; valamint a megfelelőségi csapat, amely a licenc- és export-szabályozási követelményeket ellenőrzi. Az SBOM értéke azonban csak akkor érvényesül, ha verziózott és az artefaktummal együtt életcikluson át megőrzött.

21.4.12 SLSA: Supply-chain Levels for Software Artifacts

A SLSA (ejtsd: „salsa”) a build- és szállítási folyamat biztonsági érettségét leíró keretrendszer. A célja, hogy a szervezetek fokozatosan, ellenőrizhető lépésekben növeljék az ellátási láncuk biztonságát. A SLSA szinteket definiál, 1-től 4-ig, ahol a magasabb szint erősebb garanciákat nyújt. A szintek nem abszolút biztonsági fokozatok, hanem az ellenőrizhetőség és a bizonyíthatóság lépcsőfokai.

A SLSA-szintek rövid összefoglalása:

- **SLSA 1**: A build folyamata dokumentált, a forráskód és a build definíció verziózott. Ez a szint minimális követelmény arra, hogy egyáltalán legyen nyomon követhetőség.
- **SLSA 2**: A buildet egy verziózott buildszolgáltatás futtatja, és a buildről származási igazolás (provenance) készül, amelyet aláírnak. A provenance dokumentum leírja, hogy mely forrásból, mely build lépésekkel, mely környezetben készült az artefaktum.
- **SLSA 3**: A buildplatform biztonsági kontrollokat alkalmaz: a build definíció és a forráskód elkülönített, a futtató környezet auditált, a build triggerek hitelesek. A provenance tartalmazza a buildplatform tanúsítását is.

- **SLSA 4**: A build reprodukálható, és a build lépéseit két független személy hagyja jóvá. A provenance igazolja, hogy a build megismételhető, és a forrásból bárki ugyanazt az artefaktumot állíthatja elő.

A SLSA tehát a build folyamat integritására fókuszál, nem az artefaktum tartalmának biztonságára. Egy SLSA 4-es build is tartalmazhat sebezhető függőséget, de a származási lánc igazolható, így a sebezhetőség forrása gyorsan azonosítható, és a build manipulálása kizárható.

21.4.13 Provenance

A provenance (származási igazolás) a SLSA központi fogalma. Egy provenance dokumentum leírja, hogy egy artefaktum milyen forrásból, milyen build lépésekkel, milyen környezetben, milyen időpontban készült. Tartalmazza a forráskód commit hash-ét, a build definíciót, a build platform azonosítóját, a felhasznált függőségek listáját (leggyakrabban SBOM formájában), és az aláírásokat. A provenance-t a build platform generálja és aláírja, így a fogyasztó ellenőrizheti, hogy az artefaktum tényleg az adott forrásból, az adott buildplatformon készült.

A provenance-t például a SLSA GitHub Actions workflow-ok automatikusan generálhatják, vagy a `in-toto` keretrendszerrel lehet létrehozni. A fogyasztó a provenance ellenőrzésével dönthet arról, hogy megbízik-e az artefaktumban: ha a provenance hiányzik, vagy az aláírás érvénytelen, a telepítés blokkolható.

A provenance és az SBOM közötti különbség fontos: az SBOM a „miből áll”, a provenance a „hogyan és miből készült”. A kettő együtt ad teljes képet; a SLSA 2-es szinttől kezdve mindkettő jelen van.

21.4.14 Gyakorlati példa: SLSA 2 egy kis projektben

Tegyük fel, hogy van egy Node.js alapú mikroszolgáltatás, amelynek a forráskódja GitHubon, a buildje GitHub Actions, az artefaktuma pedig egy Docker image, amelyet egy privát registrybe (`registry.example.org/myorg/myapp`) publikálnak. A cél az SLSA 2-es szint elérése: legyen aláírt provenance és SBOM, és a fogyasztó oldalán legyen ellenőrzés.

A GitHub Actions workflow első lépése az SBOM generálás. Ezt a `cyclonedx/gh-node-module-generate-sbom` action-nal vagy a Syft futtatásával lehet megtenni. A következő lépés a Docker image build, majd a `cosign` segítségével a provenance aláírása. A GitHub Actions beépített OIDC-támogatása lehetővé teszi a keyless aláírást; a `cosign` workflow-szintű konfigurációja automatikusan elvégzi az azonosítást.

Példa a workflow egy részletére, szemléltető jelleggel (a valós működéshez a pontos action-verziókat és a registry beállításokat ellenőrizni kell):

```
# .github/workflows/build.yml - szemléltető példa, nem éles konfiguráció
name: build-and-sign
on:
  push:
    branches: [main]
jobs:
  build:
    permissions:
      id-token: write
      contents: read
      packages: write
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - name: Generate SBOM
        run: |
          npm install --package-lock-only
          npx @cyclonedx/cyclonedx-npm --output-file myapp.sbom.json
      - name: Build and push image
        run: |
          docker build -t registry.example.org/myorg/myapp:${GITHUB_SHA} .
          docker push registry.example.org/myorg/myapp:${GITHUB_SHA}
      - name: Sign image and SBOM
        run: |
          cosign sign --yes \
            --oidc-provider=github \
            --identity-regexp='^https://github.com/myorg/myapp/.+' \
            registry.example.org/myorg/myapp:${GITHUB_SHA}
          cosign sign-blob myapp.sbom.json \
            --output myapp.sbom.json.sig
```

A fogyasztó oldalán a following ellenőrzés történik:

```
# Image ellenőrzése: aláírás és provenance
cosign verify \
  --certificate-identity-regexp='^https://github.com/myorg/myapp/.+' \
  registry.example.org/myorg/myapp:${GITHUB_SHA}

# SBOM ellenőrzése
cosign verify-blob myapp.sbom.json \
  --signature myapp.sbom.json.sig
```

Ez a példa az SLSA 2-es szintnek megfelelő aláírt provenance-t biztosít: a GitHub Actions mint buildplatform verziózott, az OIDC-alapú azonosítás hitelesíti a buildet, a `cosign` pedig aláírja az artefaktumot és az SBOM-ot. A reprodukálhatóság (SLSA 4) eléréséhez további lépések szükségesek: a build image rögzítése, a nem determinisztikus elemek kiküszöbölése, és a kétfős review bevezetése.

21.4.15 Tipikus hibák és ellenjavallt megoldások

A gyakorlatban számos olyan hiba fordul elő, amely aláássa az ellátási lánc biztonságát. Az alábbiakban a leggyakoribbakat tárgyaljuk, a hiba okának és a helyes megoldásnak az indoklásával.

21.4.16 1. hiba: Aláíratlan artefaktum publikálása

Sok csapat úgy publikál konténer image-et vagy csomagot, hogy nem írja alá. Az aláírás hiánya azt jelenti, hogy a fogyasztó nem tudja ellenőrizni, hogy az artefaktum tényleg a kiadótól származik, és hogy nem módosították-e a szállítás során. A helyes megoldás az, hogy minden publikált artefaktum aláírva legyen, lehetőség szerint keyless módszerrel, és az aláírás ellenőrzése a fogyasztó oldalán is kötelező legyen.

21.4.17 2. hiba: A privát kulcs nem védett tárolása

Ha az aláíró privát kulcsot a CI/CD rendszerben plain text titokként tárolják, a támadó a CI/CD rendszer feltörésével az aláírást is megszerezheti. A helyes megoldás a keyless aláírás vagy hardveres kulcstároló (HSM, KMS) használata. A hosszú életű privát kulcsok kerülése a Sigstore által népszerűsített minta, amelyben a kulcs rövid életű, és a publikus kulcs a transparency logban hitelesített.

21.4.18 3. hiba: A lock fájl figyelmen kívül hagyása

Ha a csapat nem verziózza a lock fájlt, vagy a CI/CD a lock fájl nélkül telepít, a build nem reprodukálható, és a függőségek verziói véletlenszerűen változhatnak. Ez sebezhetőségek és kompatibilitási hibák forrása. A helyes megoldás a lock fájl verziózása, és a CI/CD-ben a `--frozen-lockfile` (npm) vagy `--locked` (Cargo) opció használata.

21.4.19 4. hiba: A sebezhetőség-scan csak a release után fut

Ha a sebezhetőség-scan manuális, időszakos tevékenység, előfordulhat, hogy a sebezhetőség már a termelésben van, mire észlelik. A helyes megoldás a scan beépítése a pipeline-ba, ahol a kritikus sebezhetőség blokkolja a buildet. A scan eredményét riportként kell tárolni, és a javításra feladatot kell nyitni.

21.4.20 5. hiba: A frissítések elhanyagolása a „nem nyúlunk hozzá” elv miatt

Egyes csapatok kerülik a függőségek frissítését, mert a változtatás kockázatot jelent. Ez azonban oda vezet, hogy a sebezhetőségek felhalmozódnak, és a későbbi, nagyobb frissítések még kockázatosabbá válnak. A helyes megoldás a folyamatos, adagolt frissítés, a biztonsági javítások gyors alkalmazásával és a főverziós frissítések gondos tesztelésével.

21.4.21 6. hiba: A provenance hiánya

A provenance nélküli artefaktum származása nem igazolható. Ha egy biztonsági incidens során meg kell állapítani, hogy mi került a termelésbe, a provenance hiánya megnehezíti a visszafejtést, és lehetetlenné teszi a bizalmi lánc újjáépítését. A helyes megoldás a buildplatform által támogatott provenance-generálás és aláírás bevezetése, akár SLSA 1-től indulva.

21.4.22 Labor: SBOM- és provenance-terv készítése

A labor célja, hogy a tanuló egy kis, ellenőrizhető példán keresztül alkalmazza a fejezet eszközeit. A labor nem igényel éles rendszert; minden lépést lokális vagy izolált környezetben kell elvégezni.

21.4.23 Kontextus és kiinduló állapot

Adott egy egyszerű Node.js mikroszolgáltatás, amelynek a forráskódja a lokális gépen található. A projekt rendelkezik `package.json` és `package-lock.json` fájlokkal, valamint egy `Dockerfile`-lal. A feladat a build folyamat biztonsági tervének elkészítése: SBOM generálás, provenance-terv és aláírási terv. A labor eredménye egy dokumentum vagy jegyzetfüzet, amely tartalmazza a döntési naplót és az ellenőrzési szempontokat.

A laborban el kell különíteni a **mért** és a **várt** eredményt. A mért eredmény az, amit a lokális futtatás ténylegesen produkál (például a generált SBOM fájl mérete, a talált függőségek száma). A várt eredmény az, amit a terv szerint elvárunk (például a SBOM tartalmazza az összes tranzitív függőséget). A kettő összevetése mutatja meg, hogy a terv helyesen működik-e.

21.4.24 Lépések

1. **SBOM generálás**: Telepítsd a Syft eszközt, és generálj SBOM-ot a projekt konténer image-éről (ha nincs image, először buildeld meg a Dockerfile alapján). Használd a `syft` parancsot CycloneDX JSON kimenettel. Ellenőrizd, hogy a SBOM tartalmazza-e az `express` függőséget és annak tranzitív függőségeit.
2. **Provenance-terv**: Fogalmazd meg, hogy milyen buildplatformot használsz (például GitHub Actions, GitLab CI, Jenkins), és milyen lépésekkel lehet SLSA 2-es szintű provenance-t generálni. Írd le a workflow lépéseit pszeudokódban vagy YAML-szerű vázlatban. Ne futtasd a valós CI-n, csak tervezd meg.
3. **Aláírási terv**: Válassz aláírási mechanizmust (például keyless cosign OIDC, vagy hagyományos PGP). Indokold a választást a kulcskezelés és a fogyasztó oldali ellenőrzés szempontjából. Írd le, hol történik az aláírás a pipeline-ban, és hol történik az ellenőrzés.
4. **Döntési napló**: Készíts ADR-t a következő döntésekről:
   - Melyik SBOM formátumot választod és miért?
   - Milyen sebezhetőség-scan eszközt alkalmazol és milyen küszöbértékkel?
   - Milyen frissítési stratégiát vezetsz be a függőségekre?
5. **Ellenőrzési szempontok**: Fogalmazz meg legalább öt olyan ellenőrzési szempontot, amely alapján megállapítható, hogy a terv teljesült. Például: „Az SBOM legalább az összes direct és tranzitív függőséget tartalmazza.” vagy „A build akkor sikeres, ha a sebezhetőség-scan nem talál CVSS 7.0 feletti sebezhetőséget.”

21.4.25 Elvárt eredmény

A labor eredménye egy dokumentum, amely tartalmazza: - a generált SBOM fájl elérési útját és egy rövid összefoglalóját, - a provenance-terv leírását, - az aláírási terv leírását, - a döntési naplót, - az ellenőrzési szempontokat, - a mért és a várt eredmények összevetését.

A dokumentumot verziózni kell, és a benne szereplő példáknak helyőrzőket kell használniuk valós adatok helyett.

21.4.26 Visszaállítási út

A labor során létrehozott fájlok (SBOM, aláírások, image-ek) a labor után törölhetők. A Docker image-eket a `docker rmi` paranccsal lehet eltávolítani. A generált fájlok nem kerülnek verziókezelésbe, kivéve a dokumentumot, amelyet a csapat tudásbázisába lehet menteni.

21.4.27 Ellenőrzőlista

A fejezetben tárgyalt biztonsági intézkedések összefoglalásaként az alábbi ellenőrzőlista használható egy projekt értékeléséhez:

- ☐ Minden build artefaktum aláírt, és a fogyasztó oldalán az aláírás ellenőrzése kötelező.
- ☐ A privát aláíró kulcsok nem plain text titokként vannak tárolva; preferált a keyless aláírás vagy HSM/KMS.
- ☐ A függőségek pontos verziója lock fájlban rögzített, és a lock fájl verziózott.
- ☐ A CI/CD pipeline nem telepít függőséget lock fájl nélkül.

- ☐ A sebezhetőség-scan a pipeline része, és kritikus sebezhetőség esetén a build meghiúsul.
- ☐ Minden artefaktumhoz SBOM készül, és az SBOM aláírt.
- ☐ A build platform provenance-t generál és aláír (legalább SLSA 2).
- ☐ A függőségek frissítése folyamatos, a biztonsági javítások gyorsan alkalmazva.
- ☐ A reprodukálhatóság felé tett lépések dokumentáltak, és a build környezet rögzített.

21.4.28 Önellenőrző kérdések

1. Mi a különbség az SBOM és a provenance között? Melyiket mire használjuk?
2. Hogyan döntheti el egy fogyasztó, hogy egy konténer image megbízható, ha rendelkezésre áll az aláírás és a provenance?
3. Miért nem elég az aláírás önmagában az artefaktum biztonságának igazolásához?
4. Melyik SLSA-szint követeli meg a reprodukálható buildet, és miért?
5. Milyen hibát követ el az a csapat, amelyik a sebezhetőség-scan eredményét csak a release után nézi meg?

21.4.29 Összefoglalás

A szoftverellátási lánc biztonsága a bizalom láncának felépítése a forráskódtól a futó konténerig. A függőségek kezelése – lock fájl, sebezhetőség-scan, frissítési stratégia – biztosítja, hogy a megfelelő, ismert állapotú komponenseket használjuk. Az aláírás és a provenance igazolja, hogy az artefaktum érintetlen, és az elvárt buildplatform készítette. Az SBOM átláthatóvá teszi az összetevőket, a SLSA pedig keretet ad a build folyamat fokozatos megerősítésére. A reprodukálható build teszi lehetővé, hogy a bizalom ne csak állítás, hanem ellenőrizhető tény legyen.

A következő fejezet (Privacy by design és GDPR, Adatvédelem, megfelelés és etika) ezekre az alapokra épít: a szabályozási keretek – GDPR, NIS2, DORA – gyakran megkövetelik, hogy a szervezet tudja igazolni, milyen szoftverösszetevőket futtat, honnan származnak, és hogy a sebezhetőségeket időben javítják. Az itt bevezetett SBOM, aláírás és provenance a megfelelőség bizonyításának technikai gerincét adják; a következő részben megtárgyaljuk, hogyan kapcsolódnak ezek a jogi és etikai követelményekhez.

22 Adatvédelem, megfelelés és etika

22.1 Privacy by design és GDPR

A személyes adatok védelme nem utólagos megfelelési teher, hanem az architektúra egyik első osztályú minőségi követelménye. Amikor egy rendszer komponenseit, adatbázisait, üzenetsorait és külső integrációit tervezzük, a GDPR és a Privacy by Design elvei határozzák meg, hogy milyen adatokat gyűjtsünk, hogyan tároljuk, kinek adhatunk hozzáférést, meddig őrizzük meg, és milyen mechanizmusokkal tudjuk az érintetti jogokat biztosítani. Ez a fejezet az adatvédelem beépítését mutatja be a szoftverarchitektúra tervezési folyamatában, gyakorlati mintákkal, egy kis ellenőrizhető példával, valamint a tipikus hibák és a laboratóriumi feladat leírásával. Az előző fejezet az ellátási lánc biztonságával foglalkozott; itt az adatkezelés belső és külső határaira, az adatáramlásokra és a szabályozási megfelelés architekturális következményeire koncentrálunk. A következő fejezet az ágazati szabályozásokra (NIS2, DORA, AI Act) épít majd, ezért itt csak a GDPR általános adatvédelmi keretrendszerét dolgozzuk fel.

22.1.1 Alapfogalmak: ki kicsoda az adatkezelésben

A tervezéshez először tisztázni kell a szerepeket. A GDPR fogalomrendszere meghatározza, hogy ki felelős az adatkezelésért, ki hajtja végre az utasításokat, és kinek a jogait kell az architektúrába beépíteni.

Személyes adat: minden olyan információ, amely azonosított vagy azonosítható természetes személyre vonatkozik. Ez nem csak a név vagy az e-mail-cím, hanem az IP-cím, a munkamenet-azonosító, a geolokáció, a viselkedési profil vagy akár egy pseudonim azonosító is, ha a rendszerben más adatokkal összekapcsolva visszavezethető a személyre. Architekturális szempontból ez azt jelenti, hogy a személyes adatok nem csak a felhasználói profilokban, hanem a naplófájlokban, a cache-ekben, a biztonsági mentésekben és a monitoring rendszerekben is jelen lehetnek.

Adatkezelő (controller): az a szervezet vagy személy, amely meghatározza az adatkezelés célját és eszközeit. Egy vállalati alkalmazásban ez általában a cég, amely a szolgáltatást nyújtja. Az adatkezelő felelős az elszámoltathatóságért, az adatvédelmi elvek betartásáért és az érintetti jogok gyakorlásáért.

Adatfeldolgozó (processor): az a fél, amely az adatkezelő nevében, annak utasításai szerint kezeli a személyes adatokat. Ez lehet felhőszolgáltató, adatbázis-üzemeltető, elemző cég vagy akár egy belső alvállalkozó. Az adatfeldolgozó nem dönthet a kezelés céljáról; a tevékenységét szerződésben kell rögzíteni, és az adatkezelő utasításain kívül nem használhatja az adatokat.

Érintett (data subject): az a természetes személy, akinek az adatait kezelik. Az érintetti jogok – hozzáférés, helyesbítés, törlés, adathordozhatóság, tiltakozás, automatizált döntéshozatal elleni védelem – mind az architektúrában megvalósítandó képességeket jelentenek.

A szerepek nem csak jogi címkék: az architektúrában meghatározzák a bizalmi határokat, az interfészeket és a felelősségi köröket. Például egy adatfeldolgozó felhőszolgáltatásnak biztosítania kell, hogy az adatkezelő utasításait technikailag is kikényszerítse (például adattörlés végrehajtása,

hozzáférési naplók átadása), míg az adatkezelő rendszerének ezeket a képességeket integrálnia kell.

22.1.2 Célhoz kötöttség, adattakarékosság és korlátozott tárolhatóság

A GDPR 5. cikke rögzíti az adatkezelés alapelveit. Ezek közül háromnak különösen erős architekturális következménye van:

1. **Célhoz kötöttség:** személyes adatot csak előre meghatározott, egyértelmű és jogszerű célból szabad gyűjteni, és a későbbiekben nem szabad az eredeti céllal össze nem egyeztethető módon kezelni. A tervezésben ez azt jelenti, hogy minden adatmezőhöz, minden adatfolyamhoz és minden komponenshez hozzá kell rendelni a kezelés célját. Ha egy mezőt marketing célból gyűjtünk, azt nem használhatjuk fel hitelbírálathoz. A célhoz kötöttség megsértése tipikusan akkor történik, ha egy adattóban több célú adatokat tárolunk, és a későbbi fogyasztók szabadon hozzáférnek.
2. **Adattakarékosság (minimalizálás):** csak az adott célhoz szükséges mértékű személyes adatot szabad kezelni. Architekturális szinten ez a gyűjtés, a tárolás és a továbbítás minimalizálását jelenti. Például ha egy szolgáltatásnak csak a felhasználó életkorát kell ellenőriznie, akkor nem kell a teljes születési dátumot bekérni és tárolni – elegendő a születési év vagy egy logikai „18 év feletti” jelző. A minimalizálás a sématervezésben és az API-kontraktusokban jelenik meg: nem szabad minden mezőt „majd jó lesz valamire” alapon felvenni.
3. **Korlátozott tárolhatóság:** a személyes adatot csak a cél eléréséhez szükséges ideig szabad tárolni, utána törölni vagy anonimizálni kell. Ez azt jelenti, hogy az architektúrának támogatnia kell az adatmegőrzési időkhöz kötött automatikus törlést, az időbélyegzett adatrekordokat és a törlési feladatok ütemezését. Egy ügyfélszolgálati jegyrendszerben például a beszélgetések naplóit meghatározott ideig megőrizzük, majd anonimizáljuk vagy töröljük; ezt nem szabad manuális folyamattá tenni.

Ezek az elvek nem csupán jogi előírások, hanem a rendszer adatmodelljének, adatbázis-sémájának, API-jának és adatfolyamatainak tervezési szempontjai. A jó architektúra már a tervezéskor beépíti az adatmegőrzési időket, az adatminimalizálást és a célhoz kötöttséget, nem pedig utólag próbálja „kivágni” a felesleges adatokat.

22.1.3 Beépített és alapértelmezett adatvédelem a tervezésben

A GDPR 25. cikke két fogalmat vezet be: a **beépített adatvédelmet (data protection by design)** és az **alapértelmezett adatvédelmet (data protection by default)**.

A beépített adatvédelem azt jelenti, hogy az adatvédelmi intézkedéseket a tervezési folyamat kezdetétől be kell építeni a rendszerbe, nem pedig utólag ráilleszteni. Ez konkrétan a következőket jelenti az architektúrában:

- Az adatvédelmi követelményeket a minőségi attribútumokkal együtt, a tervezés elején rögzítjük, és a döntési naplóban (ADR) nyomon követjük.
- A komponensek közötti interfészek csak a szükséges adatokat továbbítják, és az adatokat a lehető legkorábbi ponton anonimizáljuk vagy álnevesítjük.
- A biztonsági mechanizmusok (titkosítás, hozzáférés-vezérlés, naplózás) alapértelmezés szerint be vannak kapcsolva, nem opcionálisak.

- A rendszer tervezésekor figyelembe vesszük az érintetti jogok gyakorlásának technikai megvalósítását (pl. törlési kérelem feldolgozása, adatexport).

Az alapértelmezett adatvédelem azt jelenti, hogy a gyári beállítások a lehető legszigorúbb adatvédelmet nyújtják. Például:

- Az alkalmazás alapértelmezés szerint nem osztja meg a felhasználó adatait harmadik féllel, hacsak a felhasználó ezt kifejezetten nem kéri.
- Az adatgyűjtés alapértelmezés szerint csak a szolgáltatás nyújtásához szükséges adatokra terjed ki.
- A profilalkotás, a személyre szabott hirdetés vagy az elemzés alapértelmezés szerint ki van kapcsolva.

A Privacy by Design hét alapelve (Ann Cavoukian eredeti megfogalmazása) hasznos gondolati keret az architektúra tervezéséhez:

1. **Proaktív, nem reaktív:** az adatvédelmet a tervezéskor kell beépíteni, nem az incidens után.
2. **Alapértelmezett adatvédelem:** lásd fent.
3. **Beágyazott adatvédelem:** az adatvédelem a rendszer működésének szerves része, nem külön réteg.
4. **Teljes funkcionalitás:** az adatvédelem nem a funkcionalitás rovására megy; a cél az, hogy mindkettőt elérjük.
5. **Végponttól végpontig terjedő biztonság:** az adatok védelme a gyűjtéstől a törlésig.
6. **Láthatóság és átláthatóság:** az érintett számára világos, hogy mi történik az adataival.
7. **A felhasználói adatvédelem tiszteletben tartása:** az érintett jogai és elvárásai központi szerepet kapnak.

Ezek az elvek nem konkrét technológiák, hanem tervezési irányelvek, amelyeket a konkrét architektúrában kell leképezni.

22.1.4 Gyakorlati minták: adatleltár, adatfolyam-térkép, álnevesítés és törlés

Az alábbi minták segítenek a GDPR-követelmények architekturális megvalósításában. Ezeket mindig a rendszer kontextusában kell alkalmazni; nincs univerzális recept.

22.1.5 Adatleltár és adatfolyam-térkép

Az első lépés a személyes adatok felmérése: milyen adatokat kezelünk, hol tároljuk, hogyan áramlanak, ki fér hozzá, és mi a kezelés célja. Ez egy élő dokumentum, amelyet az architektúra változásakor frissíteni kell. Az adatleltárban minden adatmezőhöz hozzá kell rendelni:

- a személyes adat kategóriáját (pl. név, e-mail, IP-cím, egészségügyi adat),
- a kezelés célját,
- a tárolás helyét és formáját,
- a megőrzési időt,
- a hozzáférési szinteket,
- a továbbításokat harmadik félnek.

Az adatfolyam-térkép diagramokkal mutatja be, hogy a személyes adat honnan hová áramlik. A diagrams-as-code eszközökkel (pl. Mermaid, PlantUML) verziózhatóan rögzíthető. Példa egy egyszerű webalkalmazásra:

```
flowchart LR
  A[Ügyfél] -->|HTTPS, szinkron| B[Webfrontend]
  B -->|HTTPS, szinkron| C[API Gateway]
  C -->|JWT, szinkron| D[Ügyfél-szolgáltatás]
  D -->|SQL, szinkron| E[(Ügyfél-adatbázis)]
  D -->|JSON, aszinkron| F[Naplózó szolgáltatás]
  F -->|SQL, szinkron| G[(Napló-adatbázis)]
  D -->|HTTPS, szinkron| H[Marketing szolgáltatás]
```

Itt a marketing szolgáltatás felé irányuló adatáramlás célhoz kötöttségi kérdést vet fel: ha a marketing cél nem összeegyeztethető az eredeti céllal, akkor az adatátadás jogszerűségét külön meg kell vizsgálni.

22.1.6 Adatminimalizálás a sémákban és API-kban

A minimalizálást már a kontraktusok szintjén érvényesítsük. Például ha egy API-nak csak a felhasználó életkorát kell ellenőriznie, a válasz ne a teljes születési dátumot adja vissza, hanem egy logikai mezőt. Hasonlóképpen, az adatbázis-sémában ne tároljunk felesleges mezőket. A minimalizálás vonatkozik a naplózásra is: a naplóbejegyzésekben ne szerepeljen teljes név, e-mail-cím vagy hitelkártyaszám; használjunk inkább azonosítókat vagy álneveket.

22.1.7 Álnevesítés és anonimizálás

Az **álnevesítés** azt jelenti, hogy a személyes adatokat olyan módon alakítjuk át, hogy azok további információk nélkül ne legyenek visszavezethetők a természetes személyre, de a visszafejtés kulcsa megőrzésre kerül. Például egy ügyfél-azonosítót lecserélünk egy véletlenszerű tokenre, és a hozzárendelést egy külön, erősen védett táblázatban tároljuk. Az álnevesített adat továbbra is személyes adatnak minősül, de csökkenti a kockázatot, ha az adatok kiszivárognak.

Az **anonimizálás** olyan visszafordíthatatlan átalakítás, amely után az adat már nem köthető természetes személyhez. Az anonimizálás csak akkor sikeres, ha a folyamat nem visszafordítható, és az adat nem egyesíthető más adatokkal. Erre példa az aggregálás vagy a kellően nagy zaj hozzáadása. Az anonimizált adat már nem tartozik a GDPR hatálya alá, ezért hasznos analitikai célokra.

22.1.8 Titkosítás és hozzáférés-vezérlés

A tárolt és továbbított személyes adatokat titkosítani kell. A titkosítás kulcsait biztonságosan, lehetőleg hardveres biztonsági modulban (HSM) vagy felhőalapú kulcskezelő szolgáltatásban tároljuk. A hozzáférés-vezérlésnek a legkisebb jogosultság elvét kell követnie: minden komponens csak azokhoz az adatokhoz férjen hozzá, amelyek a feladatához szükségesek. Ez mikroszolgáltatásoknál azt jelenti, hogy minden szolgáltatás saját adatbázissal rendelkezik, és más szolgáltatások adataihoz csak API-n keresztül, explicit engedéllyel férhet hozzá.

22.1.9 Adatmegőrzési politika és törlés

A korlátozott tárolhatóság elvének megvalósításához adatmegőrzési szabályokat kell definiálni és automatizálni. Például egy konfigurációs fájlban rögzíthetjük:

```yaml
# Szemléltető példa - nem éles rendszerre
retention_policies:
  - name: "ugyfel_szamla_tranzakciok"
    data_subject_category: "ügyfél"
    retention_period_days: 3650
    delete_action: "anonymize"
    schedule: "0 2 * * *"
  - name: "ugyfel_bejelentkezesi_naplok"
    data_subject_category: "ügyfél"
    retention_period_days: 90
    delete_action: "delete"
    schedule: "0 3 * * *"
```

A törlési művelet lehet fizikai (az adat végleges eltávolítása) vagy logikai (az adat megjelölése töröltként, és a normál lekérdezésekből való kizárása), de a háttérben a fizikai törlést is végre kell hajtani. Az architektúrának támogatnia kell a törlési kérelmek feldolgozását, beleértve a biztonsági mentésekből való eltávolítást is – ez utóbbi gyakran bonyolult.

22.1.10 Érintetti jogok technikai megvalósítása

A GDPR 12–23. cikkei sorolják fel az érintetti jogokat. Az architektúrában ezeket konkrét funkciókként kell megtervezni:

- **Hozzáférés:** az érintett kérheti az adatai másolatát. Ez egy export API-t vagy egy felületet igényel, amely a rendszerben tárolt összes személyes adatot összegyűjti és géppel olvasható formátumban (pl. JSON, CSV) kiadja.
- **Helyesbítés:** az érintett kérheti a pontatlan adatok kijavítását. Ez egy frissítési művelet, amelyet naplózni kell.
- **Törlés (az „elfeledtetéshez való jog”):** az érintett kérheti adatai törlését. Ez nem csak a fő adatbázisból való törlést jelenti, hanem a biztonsági mentésekből, a cache-ekből, a naplókból és a harmadik félnél tárolt másolatokból való eltávolítást is. Az architektúrának ezt a folyamatot végig kell vinni az összes adattáron.
- **Adathordozhatóság:** az érintett kérheti adatai átadását egy másik adatkezelőnek. Ez egy szabványos formátumú exportot igényel.
- **Tiltakozás:** az érintett tiltakozhat bizonyos adatkezelések (pl. direkt marketing) ellen. A rendszernek ezt a preferenciát tárolnia kell, és az adatkezelést le kell állítania.
- **Automatizált döntéshozatal elleni védelem:** ha a rendszer automatizált döntéseket hoz (pl. hitelbírálat, profilalkotás), az érintettnek joga van emberi beavatkozást kérni. Az architektúrának támogatnia kell a döntések felülvizsgálatát.

Ezek a funkciók gyakran egy külön „Privacy API” vagy „Data Subject Request” szolgáltatásban összpontosulnak, amely a rendszer különböző adatforrásait koordinálja.

22.1.11 Adatvédelmi hatásvizsgálat (DPIA)

A GDPR 35. cikke előírja, hogy magas kockázatú adatkezelések esetén adatvédelmi hatásvizsgálatot kell végezni. Ez egy szisztematikus folyamat, amely felméri a tervezett adatkezelés kockázatait, és meghatározza a kockázatcsökkentő intézkedéseket. Az architektúra tervezése során a DPIA-t a tervezési döntések megalapozására használhatjuk.

A DPIA lépései:

1. **Az adatkezelés leírása:** mi a cél, milyen adatokat, milyen technológiával, milyen adatfolyamokkal kezelünk.

2. **A szükségesség és arányosság értékelése:** miért van szükség az adatokra, van-e kevésbé invazív megoldás?
3. **A kockázatok azonosítása:** milyen adatvédelmi kockázatok merülnek fel (pl. jogosulatlan hozzáférés, adatvesztés, célhoz kötöttség megsértése)?
4. **Kockázatcsökkentő intézkedések tervezése:** milyen technikai és szervezeti intézkedések csökkentik a kockázatot?
5. **Dokumentáció és felülvizsgálat:** a DPIA-t rögzíteni kell, és rendszeresen frissíteni kell.

Az architektúrában a DPIA eredménye beépül az ADR-ekbe, a minőségi forgatókönyvekbe és a biztonsági architektúrába. Például ha a DPIA azt állapítja meg, hogy az adatbázisban tárolt egészségügyi adatok különösen érzékenyek, akkor a tervezés során dönthetünk az adatok álnevesítéséről, a hozzáférés szigorú korlátozásáról és a naplózás fokozásáról.

22.1.12 Labor: adatfolyam- és adatvédelmi hatásvizsgálat készítése

A labor célja, hogy a hallgató egy konkrét, de szintetikus rendszeren gyakorolja az adatvédelmi tervezést. A feladat nem valós rendszer elleni támadás, hanem a védekező tervezés.

Kiinduló állapot: adott egy fiktív webalkalmazás, amely ügyfelek regisztrációját és rendeléseit kezeli. A rendszer szándékosan egyszerű, de tartalmaz személyes adatokat: név, e-mail-cím, szállítási cím, telefonszám, rendelési előzmények. Az alkalmazás monolitikus, MySQL adatbázissal, és egy külső e-mail-szolgáltatót használ értesítésekhez. A naplózás jelenleg a teljes HTTP-kéréseket tárolja, beleértve a személyes adatokat is.

Elvárt eredmény: a hallgató készítsen egy reprodukálható dokumentumot vagy jegyzetfüzetet, amely tartalmazza:

1. **Adatleltár:** a rendszerben kezelt személyes adatok listája, cél, tárolási hely, megőrzési idő, hozzáférési szintek.
2. **Adatfolyam-térkép:** diagrams-as-code formátumban (pl. Mermaid) ábrázolja az adatok áramlását, feltüntetve a komponenseket, a protokollt, a szinkronitást és a bizalmi határokat.
3. **Adatvédelmi hatásvizsgálat:** a fenti DPIA lépések szerint, a kockázatok és a csökkentő intézkedések bemutatásával.
4. **Döntési napló (ADR):** legalább három, az adatvédelemmel kapcsolatos architekturális döntés rögzítése a kontextus, döntés, alternatívák, következmények és állapot szerint.
5. **Ellenőrzési szempontok:** a hallgató saját maga által definiált ellenőrzőlista, amely alapján a dokumentum minőségét értékelheti.

Elkülönített mért és várt eredmény: a labor során a hallgató a saját munkáját rögzíti (mért eredmény), míg a feladatleírásban az elvárások (várt eredmény) szerepelnek. Például:

- *Várt eredmény:* az adatfolyam-térképen minden személyes adat áramlás fel van tüntetve, a naplózásból a személyes adatok eltávolításra kerültek.
- *Mért eredmény:* a hallgató által kitöltött adatfolyam-diagram, amelyen a naplózás már nem tartalmaz személyes adatokat.

A labor dokumentumot verziókezelve kell elkészíteni, és a mérési adatokat (ha vannak, pl. adatmennyiség, hozzáférési kísérletek) külön kell választani a várt értékektől. A gyakorlat nem igényel éles rendszert; minden adat szintetikus, például `felhasznalo@example.com` vagy `Pelda Kft.` helyőrzők.

Visszaállítási út: mivel a labor dokumentum-alapú, nincs szükség rendszer-visszaállításra; a verziókezelés biztosítja a korábbi állapotok elérhetőségét.

22.1.13 Tipikus hibák és azok elkerülése

A gyakorlatban számos hiba rontja az adatvédelem architekturális megvalósítását. Ezeket érdemes már a tervezés során felismerni.

1. **Célhoz kötöttség megsértése:** az adatokat egy célból gyűjtik, de később más célra használják, anélkül hogy új jogszerű alapot biztosítanának. Például a regisztrációs adatokat marketing célra adják át. A megoldás a célok explicit rögzítése és az adatáramlások engedélyezése a célok szerint.
2. **Túlzott adatgyűjtés:** a rendszer a szükségesnél több adatot kér be, „majd jó lesz valamire". Ez sérti az adattakarékosság elvét. A megoldás a minimalizálás már az API-tervezésnél.
3. **Hiányzó adatmegőrzési politika:** az adatokat határozatlan ideig tárolják, mert nincs automatizált törlés. A megoldás a megőrzési időkhöz kötött ütemezett törlés.
4. **Személyes adat a naplókban:** a naplózás során véletlenül személyes adatokat rögzítenek, ami újabb adatkezelést jelent. A megoldás a naplóüzenetek szűrése, álnevesítés vagy a személyes adatok elhagyása.
5. **Az érintetti jogok nem megvalósíthatók:** a rendszer nem tudja teljesíteni a törlési kérelmet, mert az adatok több tárolóban és biztonsági mentésben vannak. A megoldás egy központi adatkezelési nyilvántartás és a törlési folyamatok automatizálása.
6. **Az adatkezelő és adatfeldolgozó szerepek nem tisztázottak:** a szervezet nem tudja, hogy egy felhőszolgáltató adatfeldolgozó-e, és nincs vele szerződés. A megoldás a szerepek rögzítése és a szerződéses garanciák beépítése.
7. **A DPIA elhanyagolása:** magas kockázatú adatkezelésnél nem készül hatásvizsgálat, így a kockázatok ismeretlenek maradnak. A megoldás a DPIA beépítése a tervezési folyamatba.

Ezek a hibák nem csak jogi szankciókat vonhatnak maguk után, hanem a rendszer bizalmának és hírnevének romlásához is vezetnek.

22.1.14 Példa: egy fiktív ügyfélkezelő rendszer adatvédelmi tervezése

Tekintsünk egy egyszerű, szintetikus példát: egy kisvállalkozás ügyfélkezelő rendszert épít, amelyben az ügyfelek regisztrálnak, rendeléseket adnak le, és e-mail-értesítést kapnak. A fejlesztőcsapat a tervezéskor a következő adatvédelmi döntéseket hozza:

- **Adatminimalizálás:** a regisztrációhoz csak a név, e-mail-cím és szállítási cím szükséges; a telefonszám opcionális, és csak akkor kérik be, ha az ügyfél telefonos értesítést kér.
- **Célhoz kötöttség:** az e-mail-címet csak a rendelési értesítésekhez használják; marketing hírlevelet csak külön hozzájárulás után küldenek.
- **Adatmegőrzés:** a rendelési adatokat 5 évig tárolják (számviteli kötelezettség miatt), a bejelentkezési naplókat 90 nap után törlik, a marketing hozzájárulásokat addig tartják meg, amíg az ügyfél vissza nem vonja.
- **Álnevesítés:** az analitikai adatbázisban az ügyfél-azonosítókat véletlenszerű tokenre cserélik, így az elemzők nem látják a valódi neveket.
- **Hozzáfér-vezérlés:** csak az ügyfélszolgálati munkatársak férhetnek hozzá a személyes adatokhoz, és minden hozzáférést naplóznak; a naplóbejegyzések nem tartalmazzák a személyes adatokat.

- **Érintetti jogok:** az alkalmazás biztosít egy „Saját adatok” oldalt, ahol az ügyfél letöltheti az adatait, kérheti a törlést, vagy módosíthatja a marketing beállításokat. A törlési kérelem esetén a rendszer az összes adatbázisból és biztonsági mentésből törli az adatokat (a számviteli kötelezettség alá eső adatok kivételével, amelyeket anonimizál).

Ezeket a döntéseket ADR-ekben rögzítik, és az adatfolyam-térképen a bizalmi határokat feltüntetik. A DPIA során azonosítják a kockázatokat: a naplózás személyes adatokat tartalmazhat, ezért a naplókat szűrik; a harmadik fél e-mail-szolgáltató adatfeldolgozó, ezért szerződést kötnek vele. A tervezés eredményeként a rendszer már a kezdetektől megfelel a GDPR alapelveinek.

22.1.15 Ellenőrzőlista az adatvédelem-architektúrához

Az alábbi kérdések segítenek a tervezés során ellenőrizni, hogy az adatvédelem beépült-e az architektúrába:

- ☐ Minden személyes adatot azonosítottunk az adatleltárban?
- ☐ Minden adatmezőhöz hozzárendeltük a kezelés célját?
- ☐ Az adatgyűjtés minimális-e, vagy van felesleges mező?
- ☐ Az adatmegőrzési időket definiáltuk, és automatizáltuk a törlést?
- ☐ A naplózás nem tartalmaz személyes adatokat?
- ☐ A harmadik felekkel (adatfeldolgozókkal) szerződéses garanciák vannak?
- ☐ Az érintetti jogok (hozzáférés, helyesbítés, törlés, adathordozhatóság) technikailag megvalósíthatók?
- ☐ A hozzáférés-vezérlés a legkisebb jogosultság elvét követi?
- ☐ Az érzékeny adatok álnevesítve vagy anonimizálva vannak?
- ☐ Készült adatvédelmi hatásvizsgálat a magas kockázatú adatkezelésekre?
- ☐ Az adatvédelmi döntések ADR-ekben dokumentáltak?
- ☐ A titkosítás és a kulcskezelés megfelelő?

22.1.16 Önellenőrző kérdések

1. Mi a különbség az adatkezelő és az adatfeldolgozó között, és hogyan befolyásolja ez a rendszerkomponensek felelősségét?
2. Hogyan valósítaná meg az adattakarékosság elvét egy mikroservice-alapú rendszer API-iban?
3. Milyen lépések szükségesek ahhoz, hogy egy törlési kérelem („elfeledtetéshez való jog”) teljesíthető legyen egy elosztott rendszerben, beleértve a biztonsági mentéseket?
4. Miért fontos a naplózás szűrése a személyes adatok szempontjából, és milyen technikákkal kerülhető el a személyes adatok naplózása?
5. Mikor kötelező adatvédelmi hatásvizsgálatot végezni, és hogyan kapcsolódik ez az architektúra tervezési folyamatához?

22.1.17 Forrás és jogi megjegyzés

A fejezet a GDPR általános követelményeit ismerteti oktatási célból. A hivatalos és mindenkor hatályos szöveget az Európai Unió Hivatalos Lapjában közzétett rendelet (EU) 2016/679 tartalmazza; a konkrét cikkelyeket és azok értelmezését mindig a hivatalos forrásból ellenőrizni kell. Ez a könyv nem jogi tanácsadás; konkrét adatkezelési kérdésekben jogi szakértő bevonása szükséges. A példák szintetikusak, és nem tartalmaznak valós személyes adatot.

22.2 Ágazati szabályozás: NIS2, DORA és AI Act

A NIS2, a DORA és az AI Act az európai szabályozási környezet három olyan pillére, amelyek alapvetően alakítják át a szoftverarchitektúrával szemben támasztott elvárásokat. A GDPRhoz hasonlóan ezek sem csupán jogi dokumentumok, hanem olyan tervezési kényszerek, amelyek a rendszer minőségi attribútumaiban, a komponensek felelősségében és az üzemeltetési folyamatokban is megjelennek. A megfelelés nem utólagos papírmunka, hanem a tervezés része: a kockázatkezelés, a naplózás, az incidenskezelés, a külső függőségek nyomon követése és adott esetben az emberi felügyelet mind olyan képesség, amelyet az architektúrának explicit módon támogatnia kell. Ez a fejezet a három szabályozás architekturális hatását mutatja be, majd egy megfelelési térkép elkészítésén keresztül gyakorlati eszközt ad a követelmények és az architekturális elemek összekapcsolására.

22.2.1 Az európai szabályozási környezet architekturális hatása

A NIS2 irányelv, a DORA rendelet és az AI Act rendelet eltérő jogi formájú eszközök, ami befolyásolja az alkalmazás módját. A NIS2 irányelv, amelyet a tagállamoknak nemzeti jogba kell átültetniük, tagállamonként kisebb eltéréseket eredményezhet a határidőkben és a részletszabályokban. A DORA és az AI Act ezzel szemben uniós rendeletek, tehát közvetlenül alkalmazandók minden tagállamban. Architekturális szempontból ez azt jelenti, hogy a DORA és az AI Act követelményeit egységesen kell értelmezni, míg a NIS2 esetében a nemzeti jogszabályt is figyelembe kell venni. Mindhárom szabályozás célja a kiberreziliencia, a kockozatkezelés és az elszámoltathatóság erősítése, de eltérő hatókörrel és mélységgel.

A szabályozások architekturális hatása abban ragadható meg, hogy a megfelelés minőségi attribútummá válik. A minőségi követelményeket quality attribute scenario formában kell megfogalmazni. Például: „Amikor egy IKT-incidens bekövetkezik, a rendszer 24 órán belül előállítja a kezdeti jelentéshez szükséges adatokat, és a válaszidő a mérési ablakban nem haladja meg a 15 percet.” A szabályozási követelmények tehát nem elvont jogi elvárások, hanem mérhető, tesztelhető, architekturálisan támogatott képességek. Ennek megfelelően a rendszertervezés során a következő területeket kell figyelembe venni:

- **Kockázatkezelés:** a rendszernek képesnek kell lennie a fenyegetések és sebezhetőségek azonosítására, értékelésére és csökkentésére, beleértve a külső szolgáltatók kockázatait is.
- **Incidensészlelés és -jelentés:** a naplózás, a monitoring és a riasztás olyan szintű kell legyen, hogy a szabályozók által előírt határidők betarthatók legyenek.
- **Nyomon követhetőség és auditálhatóság:** az architekturális döntéseknek, a rendszer konfigurációjának és az adatkezelésnek dokumentáltnak és ellenőrizhetőnek kell lennie.
- **Emberi felügyelet és beavatkozás:** különösen az MI-rendszerek esetében, ahol a döntések felülbírálhatóságát technikailag biztosítani kell.
- **Ellátási lánc biztonsága:** a külső komponensek, könyvtárak, felhőszolgáltatások és MI-modellek származását és biztonsági állapotát nyomon kell követni.

Ezek a területek a rendszerarchitektúra szinte minden szintjén megjelennek: a hálózati szegmentálástól a komponensek közötti interfészeken át a naplózási infrastruktúráig. A következő szakaszok ezt a három szabályozásra lebontva vizsgálják.

22.2.2 NIS2: kiberbiztonsági irányelv

A NIS2 irányelv (az Európai Parlament és a Tanács (EU) 2022/2555 irányelve) az uniós szintű kiberbiztonság egységes, magas szintjének biztosítását célozza. Felváltja a korábbi NIS irányelvet,

és jelentősen kiterjeszti a hatályt. A pontos cikkelyek és határidők a hivatalos szövegben ellenőrizendők; ez a fejezet az architekturális szempontból releváns elemeket emeli ki.

A NIS2 hatálya alá tartozó szervezeteket két kategóriába sorolja: alapvető fontosságú (essential) és fontos (important) szervezetek. Az ágazati lefedettség igen széles: energia, közlekedés, banki szolgáltatások, pénzügyi piaci infrastruktúrák, egészségügy, ivóvízellátás, digitális infrastruktúra, IKT-szolgáltatás-menedzsment, közigazgatás, űr, postai szolgáltatások, hulladékgazdálkodás, vegyianyag-gyártás, élelmiszer-előállítás, gépgyártás és digitális szolgáltatók. Architektúrális szempontból ez azt jelenti, hogy a legtöbb iparágban működő, kritikus vagy fontos szolgáltatást nyújtó szervezet rendszereinek tervezésekor a NIS2 követelményeit figyelembe kell venni.

A NIS2 által előírt kockázatkezelési intézkedések magukban foglalják a hálózati és információs rendszerek biztonságának technikai és szervezeti védelmét. Architekturális következményei a következők:

- **Hálózati szegmentálás:** a kritikus rendszereket el kell különíteni a kevésbé védett hálózatoktól, például VLAN-okkal, tűzfalakkal, mikroszegmentációval. A komponensdiagramokon a bizalmi határokat explicit módon jelölni kell.
- **Hozzáfélés-ellenőrzés:** a rendszernek támogatnia kell a szerepkör-alapú hozzáférést, a többtényezős hitelesítést és a legkisebb jogosultság elvét. Az identitás- és hozzáférés-kezelés (IAM) komponensnek integrálódnia kell a rendszer összes részébe.
- **Titkosítás:** az adatok védelme nyugalmi állapotban és továbbítás közben, a kriptográfiai kulcsok biztonságos kezelése.
- **Naplózás és monitoring:** a biztonsági események rögzítése, tárolása és elemzése. A naplóknak alkalmasnak kell lenniük az incidensek utólagos kivizsgálására, ezért a megőrzési időt és a hozzáférés szabályozását is tervezni kell.
- **Sebezhetőségkezelés:** a rendszernek támogatnia kell a rendszeres sebezhetőség-ellenőrzést és a javítások telepítését. Ez a CI/CD folyamatra is kihat, például automatizált szoftverösszetétel-elemzésre (SCA) és konténerképek vizsgálatára.
- **Üzletmenet-folytonosság:** a rendszernek rendelkeznie kell helyreállítási tervekkel, redundanciával és katasztrófa-helyreállítási képességekkel. Az RTO (Recovery Time Objective) és RPO (Recovery Point Objective) értékeket a kritikus funkciókra meg kell határozni.

A NIS2 incidensjelentési kötelezettsége szigorú határidőket ír elő: tipikusan 24 órán belül korai előrejelzést, 72 órán belül részletes jelentést, és egy hónapon belül zárójelentést kell benyújtani. (Ezek az időtartamok a hivatalos szövegben ellenőrizendők.) Architekturális szempontból ez azt jelenti, hogy a rendszernek képesnek kell lennie az incidens gyors észlelésére, az érintett komponensek azonosítására, a hatás felmérésére és a jelentési adatok összegyűjtésére. Az observability stack (logs, metrics, traces) kialakításánál ezeket az időablakokat figyelembe kell venni. Például ha a naplókat csak naponta egyszer töltik be egy elemző rendszerbe, akkor a 24 órás előrejelzési határidő nem tartható.

Az ellátási lánc biztonsága szintén kiemelt NIS2-követelmény. A szervezeteknek értékelniük kell beszállítóik és szolgáltatóik kockázatait, és szerződéses biztosítékokat kell kérniük. Architekturálisan ez a külső függőségek – felhőszolgáltatások, SaaS-alkalmazások, API-szolgáltatók, nyílt forráskódú könyvtárak – folyamatos nyilvántartását és monitorozását igényli. A szoftver-összetétel-elemzés (SCA) és a szoftver anyagjegyzék (SBOM) gyakorlata ezen a ponton kapcsolódik a NIS2-höz.

Végül a NIS2 a vezetői felelősséget is kiemeli: a vezetőségnek jóvá kell hagynia a kiberbiztonsági intézkedéseket, és felügyelnie kell azok végrehajtását. Bár ez elsősorban szervezeti kérdés, az architektúrának támogatnia kell a vezetői riportálást, például dashboardokkal és összefoglaló jelentésekkel.

22.2.3 DORA: digitális működési reziliencia a pénzügyi szektorban

A DORA (Digital Operational Resilience Act, az Európai Parlament és a Tanács (EU) 2022/2554 rendelete) a pénzügyi szektor digitális működési rezilienciáját szabályozza. Célja, hogy a pénzügyi intézmények ellenállóak legyenek az IKT-incidensekkel szemben, és képesek legyenek a folyamatos működésre még súlyos üzemzavarok esetén is. A DORA rendelet, tehát közvetlenül alkalmazandó; a pontos cikkelyek és határidők a hivatalos szövegben ellenőrizendők.

A DORA hatálya a pénzügyi szektor számos szereplőjére kiterjed: hitelintézetek, befektetési vállalkozások, biztosítók, pénzforgalmi intézmények, elektronikuspénz-kibocsátók, kriptoeszköz-szolgáltatók, továbbá az ezeknek szolgáltatást nyújtó IKT-harmadik fél szolgáltatók. Architekturális szempontból ez azt jelenti, hogy a banki, biztosítási és pénzforgalmi rendszerek tervezésekor a DORA követelményeit kiemelt prioritással kell kezelni.

A DORA öt pillére közül az architektúrára a következőképpen hatnak:

1. **IKT-kockázatkezelés:** A pénzügyi intézményeknek átfogó IKT-kockázatkezelési keretrendszert kell fenntartaniuk. Ez megköveteli a kritikus funkciók és a hozzájuk tartozó IKT-rendszerek azonosítását, a függőségi térkép elkészítését, és a kockázatok folyamatos értékelését. Architekturálisan ez azt jelenti, hogy a rendszernek rendelkeznie kell egy naprakész leltárral a komponensekről, azok kapcsolatairól és a külső függőségekről. A C4 modellezés ezen a ponton nem csupán dokumentációs eszköz, hanem megfelelési követelmény.

2. **Incidensjelentés:** A DORA az IKT-incidensek osztályozását és jelentését írja elő, súlyosság szerinti kategóriákkal. A jelentési határidők jellemzően rövidebbek, mint a NIS2-nél, és a pénzügyi felügyeleti hatóság felé kell teljesíteni. Architekturálisan ez erős observability-t követel: a rendszernek képesnek kell lennie az incidens észlelésére, a hatás azonnali felmérésére és a jelentési adatok automatikus vagy félautomata előállítására. A naplózásnak és a metrikáknak valós időben elérhetőnek kell lenniük.

3. **Digitális működési reziliencia tesztelés:** A DORA rendszeres tesztelést ír elő, beleértve a fenyegetésvezérelt penetrációs tesztelést (TLPT – Threat-Led Penetration Testing) a kritikus rendszerekre. Ez architekturális következménnyel jár: a rendszert úgy kell kialakítani, hogy éleshez közeli környezetben tesztelhető legyen, anélkül hogy a termelési adatokat veszélyeztetné. Szükség lehet tesztkörnyezetekre, szintetikus adatokra, és a rendszer felügyelet nélküli, biztonságos tesztelését lehetővé tevő interfészekre.

4. **IKT-harmadik fél kockázat:** A DORA szigorú követelményeket támaszt a külső IKT-szolgáltatókkal (például felhőszolgáltatókkal, SaaS-szállítókkal) kötött szerződésekre. A szerződéseknek tartalmazniuk kell bizonyos kikötéseket, például az auditálhatóságot, a kilépési stratégiát, az adatok hordozhatóságát és a szolgáltatási szinteket. Architekturálisan ez azt jelenti, hogy a rendszer tervezésekor számolni kell a felhőszolgáltatótól való függés kockázatával, és tervezni kell a hordozhatóságot vagy a többfelhős működést. Emellett a külső szolgáltatások monitorozása és a szolgáltatási szint megsértésének észlelése is architekturális feladat.

5. **Információmegosztás:** A DORA ösztönzi a fenyegetettségi információk megosztását a pénzügyi intézmények között. Ez inkább szervezeti gyakorlat, de az architektúrának támogatnia kell a fenyegetési hírcsatornák integrációját, például API-kon keresztül.

A DORA különleges hangsúlyt helyez a kritikus funkciók folyamatos működésére. A kritikus funkciók azonosítása után az architektúrának magas rendelkezésre állást, redundanciát és katasztrófa-helyreállítást kell biztosítania. A rendelkezésre állási mutatókat (például havi 99,9% SLO) a DORA kontextusában is értelmezni kell, és az error budget alapján kell a változások

kockázatát kezelni. A DORA emellett a rendszeres helyreállítási teszteket is előírja, ami a backup- és restore-folyamatok architekturális támogatását igényli.

22.2.4 AI Act: a mesterséges intelligencia kockázati osztályai

Az AI Act (az Európai Parlament és a Tanács (EU) 2024/1689 rendelete) az MI-rendszerek kockázatalapú szabályozását vezeti be az EU-ban. Célja, hogy a mesterséges intelligencia fejlesztése és alkalmazása biztonságos legyen, tiszteletben tartsa az alapvető jogokat, és átláthatóan működjön. Az AI Act kockázati osztályai határozzák meg, hogy egy adott MI-rendszerre milyen követelmények vonatkoznak. A pontos fogalmak és besorolási kritériumok a hivatalos szövegben ellenőrizendők.

A kockázati osztályok a következők:

- **Elfogadhatatlan kockázat:** Bizonyos MI-gyakorlatok tiltottak, például a szociális pontozás, a manipulatív technikák, vagy a valós idejű, távoli biometrikus azonosítás nyilvános terekben (kivételekkel). Architekturális szempontból ezeket a rendszereket egyszerűen nem szabad fejleszteni vagy bevezetni.
- **Magas kockázat:** Olyan MI-rendszerek, amelyek biztonsági komponensként működnek termékekben (például orvosi eszközök, járművek), vagy amelyek alapvető jogokat érintenek bizonyos területeken: biometrikus azonosítás, kritikus infrastruktúra, oktatás, foglalkoztatás, alapvető szolgáltatásokhoz való hozzáférés, bűnüldözés, migráció, igazságszolgáltatás. A magas kockázatú MI-rendszerekre szigorú követelmények vonatkoznak.
- **Korlátozott kockázat:** Olyan MI-rendszerek, amelyeknél elsősorban átláthatósági kötelezettségek merülnek fel, például chatbotok esetén jelezni kell, hogy a felhasználó géppel kommunikál, vagy deepfake tartalmakat jelölni kell.
- **Minimális kockázat:** A legtöbb MI-alkalmazás ide tartozik (például spam szűrők, ajánlórendszerek), ezekre nincs kötelező uniós előírás, de önkéntes magatartási kódexek alkalmazhatók.

A magas kockázatú MI-rendszerekre vonatkozó követelmények architekturális hatása jelentős. A legfontosabbak:

- **Kockázatkezelési rendszer:** A magas kockázatú MI-rendszert folyamatos kockázatkezelési folyamatba kell ágyazni, amely azonosítja a lehetséges károkat és intézkedéseket tesz azok csökkentésére. Az architektúrának támogatnia kell a kockázati események naplózását és a kockázati mutatók gyűjtését.
- **Adatkormányzás:** A tanító-, validáló- és tesztadatoknak megfelelő minőségűnek, relevánsnak, reprezentatívnak és torzításmentesnek kell lenniük. Ez az adatfolyamatok architekturális tervezését igényli: adatforrások nyomon követése, adatminőség-ellenőrzés, adatverziózás, és a torzítások mérése.
- **Technikai dokumentáció:** A rendszer működését, architektúráját, döntési logikáját és a modell jellemzőit részletesen dokumentálni kell. Ez a dokumentáció nem csupán a fejlesztés végén készül, hanem az architektúra-dokumentumok szerves része, és verziózottan kell kezelni.
- **Naplózás:** Az MI-rendszer működésének nyomon követhetőnek kell lennie. A naplóknak rögzíteniük kell a bemeneteket, a kimeneteket, a modellverziót, az időbélyeget és a releváns kontextust. Ez lehetővé teszi az utólagos ellenőrzést és a hibák kivizsgálását.
- **Emberi felügyelet:** A magas kockázatú MI-rendszert úgy kell kialakítani, hogy az emberi felhasználó beavatkozhasson, felülbírálhassa a döntést, vagy leállíthassa a rendszert. Az architektúrának interfészt kell biztosítania az emberi felülvizsgálathoz, például egy jóváhagyási munkafolyamatot vagy egy felülbírálati API-t.

- **Pontosság, robusztusság, kiberbiztonság:** A rendszernek ellenállónak kell lennie a hibákkal, zajos bemenetekkel és támadásokkal szemben. A kiberbiztonsági követelmények magukban foglalják a modell elleni támadások (például adversarial examples) elleni védelmet is.
- **Megfelelőségértékelés és regisztráció:** A magas kockázatú MI-rendszereket megfelelőségértékelésnek kell alávetni, és bizonyos esetekben uniós adatbázisban regisztrálni kell.

Az AI Act külön rendelkezik az általános célú MI-modellekről (General-Purpose AI, GPAI), különösen azokról, amelyek szisztémikus kockázatot jelenthetnek. Az ilyen modellek szolgáltatóinak részletes technikai dokumentációt kell biztosítaniuk, és be kell tartaniuk a kiberbiztonsági, energiahatékonysági és szerzői jogi követelményeket. Architekturálisan ez a nagy nyelvi modellek (LLM-ek) és más alapmodellek integrálásakor fontos: a modell képességeit, felhasználási eseteit és korlátait dokumentálni kell, és a rendszernek monitoroznia kell a modell viselkedését.

Az AI Act és a GDPR kapcsolata is lényeges. A személyes adatokat feldolgozó MI-rendszerekre mindkét szabályozás vonatkozik. Az AI Act magas kockázatú követelményei közül az adatkormányzás és a naplózás szorosan összefügg a GDPR adattakarékossági és elszámoltathatósági elveivel. A tervezés során a két szabályozás követelményeit közösen kell kezelni, például az adatminimalizálás és a naplózás összehangolásával.

22.2.5 Közös minták és tervezési gyakorlat

A három szabályozás közös architekturális mintája a **megfelelési térkép**. Ez egy olyan dokumentum vagy táblázat, amely a szabályozási követelményeket összekapcsolja a konkrét architekturális elemekkel, és megadja a teljesülés állapotát. A megfelelési térkép nem egyszeri termék, hanem folyamatosan karbantartott, verziózott eszköz, amely a kódbázissal együtt él.

A megfelelés támogatható **fitness function**-ökkel is. A CI/CD folyamatba automatikus ellenőrzések építhetők, amelyek például ellenőrzik, hogy a naplózási konfiguráció megfelel-e a megőrzési időnek, hogy a titkosítás be van-e kapcsolva, vagy hogy a külső függőségek SBOM-ja naprakész. A fitness function-ök lehetővé teszik, hogy a megfelelés ne csak dokumentált, hanem folyamatosan ellenőrzött is legyen.

Az incidensjelentési kötelezettségek architekturális támogatása a **naplózási és observability réteg** kialakítását igényli. A logokat strukturált formában kell gyűjteni, központi helyre továbbítani, és megőrizni a szabályozás által előírt ideig. A metrikáknak és a riasztásoknak lehetővé kell tenniük a szolgáltatási szint megsértésének gyors észlelését. A nyomkövetés (tracing) segít az incidens hatásának felmérésében és az érintett komponensek azonosításában. A DORA és a NIS2 incidensjelentési határidői miatt a naplók elemzésének és a jelentések előállításának folyamatát is automatizálni kell, ahol lehetséges.

A szabályozási döntéseket **ADR-ekben** kell rögzíteni. Például ha egy rendszer esetében a szervezet úgy dönt, hogy egy felhőszolgáltatót vesz igénybe, akkor ADR-ben kell dokumentálni a DORA harmadik fél kockázatra vonatkozó értékelését, a kilépési stratégiát és a szerződéses biztosítékokat. Ha egy MI-rendszert magas kockázatúként osztályoznak, az osztályozás indoklását és a követelmények teljesítésének módját szintén ADR-ben kell rögzíteni.

22.2.6 Labor: megfelelési térkép készítése

A labor célja, hogy egy konkrét, fiktív rendszerre megfelelési térképet készíts a NIS2, a DORA és az AI Act követelményei alapján. A labor elvégzéséhez szükséged lesz a rendszer kontextusának

ismeretére, és arra, hogy a követelményeket architekturális elemekhez tudd kötni. A labor eredménye egy reprodukálható dokumentum vagy jegyzetfüzet, amely verziózott, és a mért és a várt eredményeket elkülöníti.

Kontextus: Tegyük fel, hogy egy fiktív egészségügyi időpontfoglaló és tünetértékelő rendszert tervezel, nevezzük „MedBook"-nak. A MedBook webes és mobilalkalmazáson keresztül lehetővé teszi a betegeknek időpontfoglalást, és egy MI-alapú tünetértékelő modult is tartalmaz, amely a beteg által megadott tünetek alapján előzetes iránymutatást ad. A rendszer felhőben fut, külső szolgáltatásként egy felhőszolgáltatót és egy SMS-értesítő szolgáltatót használ. A MedBook mögött álló szervezet közepes méretű, NIS2 hatálya alá tartozik (egészségügyi szolgáltató), és mivel a rendszer online fizetést is kezel, a DORA is releváns a pénzügyi tranzakciókra. Az MI-tünetértékelő a magas kockázatú kategóriába esik, mert egészségügyi döntéstámogatást nyújt. A GDPR szintén alkalmazandó, de ennek részleteit a Privacy by design és GDPR fejezet tárgyalta; itt a három ágazati szabályozásra fókuszálunk.

Lépések:

1. **Rögzítsd a rendszer kontextusát.** Készíts C4 context diagramot (például Mermaid vagy PlantUML segítségével), amelyen feltünteted a felhasználói szerepköröket (beteg, orvos, adminisztrátor), a külső rendszereket (felhőszolgáltató, SMS-szolgáltató, fizetési szolgáltató, egészségügyi nyilvántartó rendszer), és a rendszerhatárt. Ez a diagram lesz a megfelelési térkép alapja.

2. **Azonosítsd az alkalmazandó követelményeket.** A MedBook esetében a NIS2, a DORA (a fizetési rész miatt), az AI Act (a tünetértékelő miatt) és a GDPR mind alkalmazandó. Készíts listát a legfontosabb követelményekről, például:

   - NIS2: incidensjelentés 24/72 óra, hálózati szegmentálás, ellátási lánc értékelése.
   - DORA: kritikus funkciók azonosítása, IKT-harmadik fél kockázat kezelése, TLPT a kritikus rendszerekre.
   - AI Act: magas kockázatú MI-rendszer követelményei (kockázatkezelés, adatkormányzás, naplózás, emberi felügyelet, pontosság, kiberbiztonság).

3. **Készíts megfelelési térképet.** Készíts egy táblázatot a következő oszlopokkal: Követelmény | Forrás | Architekturális elem | Státusz (teljesül / részben / nem) | Intézkedés | Kockázat. Töltsd ki legalább 10 sorral. Példák:

   - NIS2 incidensjelentés 24/72 óra -> Logging és riasztási rendszer -> Részben teljesül -> SIEM bevezetése, riasztások konfigurálása -> Magas
   - NIS2 hálózati szegmentálás -> Hálózati architektúra (VLAN, tűzfal) -> Nem teljesül -> Szegmentálási terv készítése -> Magas
   - DORA kritikus funkciók azonosítása -> Függőségi térkép (C4 container diagram) -> Részben teljesül -> A fizetési folyamat komponenseinek dokumentálása -> Közepes
   - DORA TLPT -> Tesztkörnyezet és tesztelési terv -> Nem teljesül -> TLPT ütemezése, tesztkörnyezet kialakítása -> Magas
   - DORA IKT-harmadik fél kockázat -> Felhőszolgáltató szerződése -> Részben teljesül -> Szerződés felülvizsgálata, kilépési stratégia -> Közepes
   - AI Act emberi felügyelet -> Orvosi felülbírálati interfész -> Nem teljesül -> UI és API tervezése az orvosi jóváhagyáshoz -> Kritikus
   - AI Act naplózás -> MI-modul naplózása (bemenet, kimenet, verzió) -> Részben teljesül -> Naplózási struktúra bővítése -> Magas
   - AI Act adatkormányzás -> Tanítóadatok kezelése, adatverziózás -> Nem teljesül -> Adatfolyamat tervezése, adatminőség-ellenőrzés -> Magas

- AI Act kiberbiztonság -> Modell elleni támadások elleni védelem -> Nem teljesül -> Adversarial tesztelés, bemeneti validálás -> Magas
- GDPR adattakarékosság a tünetértékelőben (kapcsolódás) -> Adatgyűjtési szabályok -> Részben teljesül -> Adatminimalizálás felülvizsgálata -> Közepes

4. **Készíts döntési naplót.** A legfontosabb hézagokról és a választott megoldásokról készíts ADR-eket. Például: „ADR-001: SIEM bevezetése a NIS2 incidensjelentés támogatására" – kontextus, döntés, alternatívák, következmények, állapot. Az ADR-ekben rögzítsd a választott eszközöket, a felelősöket és a határidőket.

5. **Ellenőrzési szempontok.** A labor ellenőrzéséhez a következő kérdéseket tedd fel:

   - Minden magas kockázatú AI Act követelmény lefedett a térképen?
   - Az incidensjelentési határidők betarthatók a jelenlegi architektúrával? Ha nem, milyen változtatás szükséges?
   - A külső függőségek (felhő, SMS, fizetési szolgáltató) mindegyike szerepel a térképen, és van hozzájuk kockázatértékelés?
   - A mért és a várt eredmények elkülönülnek? Például: „várt: az incidens észlelési idő < 15 perc; mért: jelenleg nincs mérés, ezért a következő lépés a monitoring bevezetése."

Elvárt eredmény: A labor végére rendelkezned kell egy verziózott megfelelési térképpel, amely tartalmazza a rendszer kontextusát, a szabályozási követelményeket, a hézagokat, a döntési naplót és az ellenőrzési szempontokat. A dokumentum legyen reprodukálható: a diagramok diagrams-as-code formában készüljenek, a táblázat legyen géppel olvasható (például CSV vagy Markdown), és a mérési adatoknál tüntesd fel a kontextust, az időpontot és a bizonytalanságot.

22.2.7 Tipikus hibák és helyes megoldások

A szabályozási megfelelés architekturális tervezése során gyakori hibák a következők:

- **A hatókör téves felmérése.** Sok szervezet nem ismeri fel, hogy a NIS2 vagy a DORA hatálya alá tartozik. Ez ahhoz vezet, hogy a követelményeket figyelmen kívül hagyják, és a rendszer utólag nem felel meg. Helyes megoldás: a besorolást jogi és compliance szakértővel közösen, dokumentáltan kell elvégezni, még a tervezés korai fázisában.
- **Az AI Act kockázati osztály alulbecslése.** Például egy egészségügyi döntéstámogató rendszert minimális kockázatúnak tekintenek, holott a rendelet értelmében magas kockázatú. Helyes megoldás: a kockázati osztályt a hivatalos mellékletek alapján, konzervatív megközelítéssel kell meghatározni, és a besorolást dokumentálni kell.
- **A naplózás és a nyomon követhetőség elhanyagolása.** Ha a rendszer nem gyűjt megfelelő naplókat, az incidensjelentési határidők nem tarthatók, és az auditok sikertelenek lesznek. Helyes megoldás: a naplózást architekturális követelményként kell tervezni, megfelelő megőrzési idővel, hozzáféréssel, integritással és a szabályozó által előírt részletességgel.
- **A külső szolgáltatók kockázatának figyelmen kívül hagyása.** A DORA és a NIS2 egyaránt megköveteli a harmadik fél kockázatok kezelését, de sok rendszertervezés nem foglalkozik a felhő- vagy SaaS-szolgáltatók szerződéses és technikai kontrolljaival. Helyes megoldás: a külső függőségeket már a tervezéskor fel kell mérni, és a szerződések mellett technikai kontrollokat (például kilépési stratégia, adathordozhatóság, monitorozás) is be kell építeni.

- **A megfelelési dokumentáció utólagos, nem verziózott.** Ha a megfelelési térkép csak a projekt végén készül el, és nem kapcsolódik az architektúrához, akkor a döntések nyomon követése lehetetlenné válik. Helyes megoldás: a megfelelési térképet és az ADR-eket a kódbázissal együtt kell verziózni, és a CI/CD folyamatba fitness function-ökkel beépíteni az automatikus ellenőrzést.

22.2.8 Összefoglalás és önellenőrző kérdés

A NIS2, a DORA és az AI Act a modern európai szoftverarchitektúra meghatározó kényszerei. A NIS2 széles körben ír elő kiberbiztonsági kockázatkezelést és incidensjelentést, a DORA a pénzügyi szektor digitális működési rezilienciáját erősíti, az AI Act pedig az MI-rendszereket kockázati osztályok szerint szabályozza. A megfelelés nem jogi utómunka, hanem tervezési feladat: kockázatkezelést, naplózást, rezilienciát, emberi felügyeletet és ellátási lánc biztonságot követel. A megfelelési térkép eszköze segít a szabályozási követelmények és az architekturális elemek összekapcsolásában, és lehetővé teszi a hézagok azonosítását.

Önellenőrző kérdés: Sorolj fel három architekturális következményt, amelyet a DORA ír elő egy pénzügyi szolgáltató kritikus rendszerére, és magyarázd el, hogyan befolyásolja a rendszertervezést. Válaszodban térj ki a kritikus funkciók azonosítására, a tesztelési követelményekre és a harmadik fél kockázatkezelésre. (A helyes válasz lényege: kritikus funkciók azonosítása és függőségi térkép készítése, TLPT tesztelhetőséget biztosító környezetek kialakítása, valamint a külső IKT-szolgáltatók szerződéses és technikai kontrollja.)

22.3 Auditálhatóság, naplózás és adatmegőrzés

Az auditálhatóság nem utólag hozzáadott ellenőrzési funkció, hanem olyan tervezési tulajdonság, amely már az architektúra kialakításakor beépül a rendszerbe. A bizonyítható működés azt jelenti, hogy a rendszerben végrehajtott műveletek utólag egyértelműen hozzárendelhetők egy azonosított szereplőhöz, és a művelet ténye, ideje, körülménye és eredménye hitelesen igazolható. Ez a követelmény különbözik a megfigyelhetőségtől: az observability a rendszer aktuális állapotát és viselkedését mutatja, az auditálhatóság pedig a múltbeli események manipulációval szemben védett rögzítését és lekérdezését szolgálja.

A bizonyítható működés alapja a jól megtervezett naplózás. A napló nem egyszerűen az alkalmazás konzolkimenete, hanem olyan strukturált eseményfolyam, amelynek minden bejegyzése egy meghatározott üzleti vagy biztonsági eseményt ír le. Az auditnapló ezen belül kifejezetten a felelősségre vonhatóságot támogatja: azt rögzíti, hogy ki, mit, mikor, milyen jogosultsággal, melyik erőforráson és milyen eredménnyel hajtott végre. Például egy rendeléskezelő szolgáltatásban a vásárlói adat megtekintése auditbejegyzést eredményezhet, míg a tranzakció sikeres lezárása üzleti eseményként kerül a naplóba. A két kategória átfedhet, de az auditnaplót a szigorúbb megőrzési, integritási és hozzáférési követelmények miatt érdemes külön kezelni.

22.3.1 A naplózási követelmények

Az auditálható rendszerrel szemben több, egymással versengő követelményt kell egyszerre teljesíteni. Ezeket érdemes már a tervezéskor minőségi forgatókönyvek formájában megfogalmazni, mert a naplózás nem funkcionális követelmény: a rendszer akkor is működik, ha a naplózás hibás, de a bizonyíthatóság elvész.

Teljesség. A naplónak tartalmaznia kell minden olyan eseményt, amely a későbbi vizsgálathoz szükséges. A teljesség azt jelenti, hogy a naplózási szabályok lefedik a kritikus műveleteket: hitelesítés, jogosultság-módosítás, adat-hozzáférés, konfigurációváltozás, tranzakció, külső rendszerhívás, hiba és újrapróbálkozás. A hiányos napló hamis biztonságérzetet ad: ha a vizsgáló nem találja a kérdéses eseményt, nem tudja eldönteni, hogy az esemény nem történt meg, vagy a naplózás hallgatott el.

Pontosság és időbeliség. A bejegyzésnek pontosnak kell lennie mind tartalmilag, mind időben. Az időbélyegnek rendszerórából kell származnia, és a rendszer óráját NTP-vel vagy más megbízható időforrással kell szinkronizálni. A több szolgáltatásból álló rendszerben az óraeltérés megnehezíti az események sorrendjének megállapítását, ezért az időbélyeget UTC-ben, ezredmásodperc pontossággal, a forrás komponens megjelölésével kell rögzíteni. A pontosság kiterjed az adattartalomra is: a naplózott mezők értéke ne utólagos transzformáció eredménye legyen, hanem a művelet pillanatában érvényes érték.

Sértetlenség. A naplóbejegyzések nem módosíthatók és nem törölhetők anélkül, hogy az észlelhető ne lenne. A sértetlenséget technikai intézkedésekkel kell biztosítani, nem csupán szervezeti szabályzattal. Erre a célra gyakran append-only tárolást, hash-láncot, digitális aláírást vagy WORM (write once, read many) adathordozót alkalmaznak. A hash-lánc lényege, hogy minden új bejegyzés hash-e tartalmazza az előző bejegyzés hash-ét: $H_i = h(H_{i-1} \| e_i)$, ahol e_i az i-edik esemény, h pedig egy kriptográfiai hashfüggvény. Ha valaki megváltoztat egy korábbi bejegyzést, az összes rákövetkező hash érvénytelenné válik. A lánc végét biztonságos helyen, például egy külső rendszerben vagy aláírt formában kell rögzíteni, hogy a támadó ne tudja újraszámolni a láncot.

Hozzáférhetőség és visszakereshetőség. A napló akkor hasznos, ha a vizsgáló a szükséges időablakban hatékonyan le tudja kérdezni. A strukturálatlan szöveges naplókat nehéz elemezni, ezért a naplóbejegyzéseket géppel olvasható formátumban, például JSON-ban kell rögzíteni, konzisztens mezőnevekkel. A visszakereshetőséghez egységes korrelációs azonosító is kell: a request ID vagy trace ID lehetővé teszi, hogy egy üzleti művelet összes naplóbejegyzését a különböző komponensek között összekapcsoljuk.

Bizalmasság. A napló maga is érzékeny adatokat tartalmazhat, ezért a hozzáférést szigorúan korlátozni kell. A bizalmasság követelménye ütközhet a hozzáférhetőséggel: a fejlesztőnek lehet, hogy szüksége van a naplóra hibakereséshez, de nem feltétlenül van joga látni a személyes adatokat vagy a biztonsági eseményeket. A megoldás a naplózási adatok osztályozása és a hozzáférési szintek szétválasztása.

Megőrzés. A naplót meghatározott ideig meg kell őrizni, majd az adatvédelmi és adatgazdálkodási szabályok szerint törölni vagy anonimizálni kell. A megőrzési idő nem lehet önkényes: jogszabályi minimumokból, szerződéses kötelezettségekből, valamint az incidensvizsgálati és üzleti igényekből kell levezetni. A napló túl rövid megőrzése kockázat, mert a vizsgálat tárgya eltűnik; a túl hosszú megőrzés viszont felesleges adatvédelmi kockázat és költség.

Ezek a követelmények együttesen alkotják az auditnaplózási politika magját. A politika nem csupán szöveges dokumentum, hanem a rendszer konfigurációjában, a naplózó komponens viselkedésében és a tárolási infrastruktúrában realizált szabályhalmaz.

22.3.2 Az auditnapló és az eseménynapló viszonya

A gyakorlatban érdemes megkülönböztetni az általános eseménynaplót és a kifejezetten audit célú naplót. Az általános eseménynapló a rendszer működéséről ad képet: hibák, figyelmeztetések, debug üzenetek, teljesítménymutatók. Az auditnapló ezzel szemben az elszámoltathatóságot

szolgálja, és jellemzően a következő kérdésre ad választ: ki, mit, mikor, honnan, milyen eredménnyel tett. Az auditnapló bejegyzéseit szigorúbban kell kezelni, mert ezek bizonyítékként is felhasználhatók.

Egy auditbejegyzés tipikus, strukturált formája a következő lehet:

```
{
  "timestamp": "2025-01-15T10:32:44.123Z",
  "event_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "correlation_id": "req-9f3a-001",
  "subject": "employee:45",
  "subject_roles": ["support", "readonly"],
  "action": "READ",
  "resource": "customer:123",
  "result": "SUCCESS",
  "source": {
    "component": "rendeles-kezelo-api",
    "ip": "10.0.0.5"
  },
  "reason": "order-investigation",
  "version": "1.0"
}
```

A példában a `subject` az azonosított szereplő, a `resource` a megszólított erőforrás, az `action` a művelet, a `reason` pedig a naplózott hozzáférés indoklása, amelyet különösen az adatvédelmi auditoknál szoktak megkövetelni. A `version` mező a séma verzióját jelöli, ami a naplóelemek későbbi feldolgozását könnyíti. Fontos, hogy az IP-cím személyes adat lehet, ezért a naplótervben meg kell indokolni a tárolását, vagy álnevesíteni kell.

A naplóbejegyzés mezőit nem szabad esetlegesen megválasztani; a mezőknek konzisztens sémát kell követniük. A séma evolúcióját verziózással kell kezelni, hasonlóan az API-szerződésekhez. Ha egy mező típusa vagy jelentése megváltozik, a régi bejegyzések feldolgozhatóságát meg kell őrizni.

22.3.3 Integritás és hozzáférés-vezérlés

A napló manipulálása elleni védekezés nem merül ki abban, hogy a naplófájlt írásvédetté tesszük. A támadó, aki rendszergazdai jogosultságot szerez, általában a naplókat is módosíthatja, ezért az integritást több szinten kell biztosítani.

Az egyik legfontosabb elv a **szerepkörök szétválasztása**. A naplót előállító alkalmazásnak ne legyen joga a már megírt naplóbejegyzéseket módosítani vagy törölni. A napló szállítását végző komponens ne férhessen hozzá a tárolt napló olvasási felületéhez. A naplót tároló rendszer üzemeltetője ne rendelkezzen az alkalmazás üzleti adatbázisához való hozzáféréssel. Ez a szétválasztás csökkenti annak kockázatát, hogy egyetlen kompromittált fiók elegendő a nyomok eltüntetéséhez.

A **hash-lánc** mellett használható **digitális aláírás** is: a naplózó szolgáltatás minden bejegyzést vagy bejegyzéscsoportot aláír a saját kulcsával; a tároló oldal ellenőrzi az aláírást, az auditáló pedig a nyilvános kulccsal utólag is igazolhatja a sértetlenséget. Ez a megoldás nagyobb rendszerekben elterjedt, de kulcskezelést igényel, amit a titokkezelési fejezetben tárgyaltunk.

A **hozzáférés-vezérlés** a napló olvasási oldalán is kritikus. Az auditnaplóhoz való hozzáférést a legkisebb jogosultság elve szerint kell korlátozni. Külön szerepeket érdemes definiálni: naplóíró, naplóolvasó, naplókezelő, auditáló. A naplókezelő a tárolási infrastruktúrát üzemelteti, de nem feltétlenül olvashatja a napló tartalmát. A naplóolvasó csak a vizsgálathoz szükséges időablakra

kaphat ideiglenes hozzáférést, és a hozzáférést magát is naplózni kell. A hozzáférési kérelmeket, jóváhagyásokat és lejáratokat érdemes gépi nyilvántartásban vezetni.

Az integritás és a hozzáférés-vezérlés tesztelhető: meg kell kísérelni egy bejegyzés törlését vagy módosítását, és ellenőrizni, hogy a rendszer észleli-e. A laborban ilyen destruktív tesztet csak izolált környezetben szabad végezni.

22.3.4 A napló mint személyes adat

A naplóbejegyzések gyakran tartalmaznak személyes adatokat: felhasználónév, e-mail-cím, IP-cím, session azonosító, sőt akár a kérés paraméterei is, ha azokban ügyféladatok szerepelnek. A GDPR 4. cikk (1) bekezdése szerint minden olyan adat személyes adatnak minősül, amely közvetlenül vagy közvetve azonosított vagy azonosítható természetes személyre vonatkozik. Az IP-cím az Európai Unió Bíróságának értelmezése szerint – amennyiben a szolgáltató rendelkezésére állnak az azonosításhoz szükséges további adatok – személyes adat lehet. A naplóban szereplő felhasználói azonosító szintén összekapcsolható a személyazonossággal, tehát a napló egésze személyes adatokat tartalmazó adatbázisnak tekintendő.

Ebből több tervezési kötelezettség következik:

Célhoz kötöttség és adattakarékosság. A naplózás célját – például biztonsági incidensek kivizsgálása, jogi megfelelés, hibaelhárítás – előre meg kell határozni, és csak az ehhez szükséges adatokat szabad gyűjteni. Ha a biztonsági vizsgálathoz elegendő a felhasználó egyedi azonosítója, de nem szükséges a teljes neve vagy e-mail-címe, akkor az utóbbiakat nem szabad a naplóba írni. A túlzott naplózás nemcsak adatvédelmi kockázat, hanem felesleges tárolási költség is.

Álnevesítés. A naplóban gyakran alkalmazható álnevesítés: a közvetlen azonosításra alkalmas mezőket úgy transzformáljuk, hogy az eredeti értelem csak egy külön tárolt kulcs birtokában állítható vissza. Például a felhasználói e-mail-cím helyett sózott hash-t tárolunk, az IP-cím utolsó oktettjét töröljük vagy tokenizáljuk. Az álnevesített adat továbbra is személyes adat, mert a kulcs birtokában azonosítható, de csökkenti a kockázatot, ha a napló illetéktelen kezekbe kerül.

Jogalap és megőrzési idő. A naplóban tárolt személyes adatok kezelésének jogalapja leggyakrabban jogi kötelezettség teljesítése (például számviteli bizonylatok, biztonsági incidensek bejelentése) vagy jogos érdek. Az adatkezelési tájékoztatóban fel kell tüntetni a naplózási adatkezelést, annak célját, jogalapját és megőrzési idejét. A megőrzési idő lejárta után az adatot törölni kell, vagy olyan módon anonimizálni, hogy az érintett többé ne legyen azonosítható. A puszta törlés elmulasztása GDPR-sértést eredményezhet.

Érintetti jogok ütközése a megőrzéssel. Az érintett kérheti személyes adatainak törlését, de a napló megőrzése jogi kötelezettség is lehet. Ezt az ütközést a tervezéskor kell feloldani. Az általános megoldás: a naplóban a személyes adatokat már eleve álnevesítve tároljuk, és csak a kulcsot töröljük, ha nem szükséges tovább. Így a naplóbejegyzés megmarad a bizonyítási célra, de a természetes személyhez való kapcsolása megszakad. Ha a teljes törlés mégis szükséges, akkor a naplót úgy kell kialakítani, hogy a személyes adatot tartalmazó mezők külön törölhetők legyenek anélkül, hogy a bejegyzés integritása sérülne. Ez például úgy érhető el, hogy a naplóbejegyzésben az érintettre mutató hivatkozás csak egy token, és a token leképezése külön, törölhető tárolóban van.

22.3.5 Adatmegőrzési terv

Az adatmegőrzési terv a naplók esetében nem csupán egyetlen időtartam megadása. A naplókat osztályozni kell: általános üzemeltetési napló, biztonsági napló, auditnapló, tranzakciós napló. Minden osztályhoz meg kell határozni a minimális és maximális megőrzési időt, a tárolási helyet, az integritási követelményeket és a törlési eljárást.

A gyakorlatban a forró, meleg és hideg tárolási rétegek váltják egymást. A forró rétegben – például az elmúlt 7 nap – a naplók azonnal lekérdezhetők, rugalmas kereséssel. A meleg rétegben – például 30 vagy 90 nap – a naplók már ritkábban hozzáférhetők, de még viszonylag gyorsan visszakereshetők. A hideg réteg – évekig – olcsó tárolón, például tömörített archívumban vagy objektumtárolóban található, ahol a lekérdezés lassabb, de a költség alacsony. A rétegek közötti mozgást automatizálni kell, és a mozgás során az integritási láncot meg kell őrizni.

A megőrzési tervet táblázatos formában érdemes rögzíteni. Például:

	Auditnapló	Biztonsági napló	Alkalmazásnapló
Cél	Elszámoltathatóság, jogi bizonyítás	Incidensvizsgálat, támadásdetekció	Hibaelhárítás, teljesítmény
Jellemző események	Adat-hozzáférés, jogosultság-módosítás	Sikertelen bejelentkezés, tűzfalszabály-változás	Hiba, figyelmeztetés, kérés-idő
Megőrzés (példa, ellenőrizendő)	5 év	1 év	30 nap
Integritás	Hash-lánc, aláírás	Hash-lánc	Nem kötelező
Hozzáférés	Auditáló szerep	Biztonsági csapat	Fejlesztő csapat
Személyes adat	Igen, álnevesítve	Igen, minimalizálva	Lehet

A táblázatban megadott időtartamok csak szemléltetők; a konkrét értékeket a mindenkori jogszabályok és az adatkezelési tájékoztató alapján kell meghatározni. A könyv nem jogi tanácsadás.

A megőrzési terv része a **törlési eljárás** is. A törlés nem lehet pusztán a fájl törlése a fájlrendszerből; a biztonságos törléshez a tároló rétegekben a hozzáférést meg kell szüntetni, a biztonsági mentéseket is figyelembe kell venni, és a törlés tényét dokumentálni kell. Az automatizált törlésnél a törlési szabályokat verziózott konfigurációban kell tartani, hogy a törlés végrehajtása reprodukálható és auditálható legyen. A törlési műveletet magát is naplózni kell, de úgy, hogy a törölt adatok tartalma ne kerüljön vissza a naplóba.

22.3.6 Architekturális minta a naplófolyamra

A naplózási architektúra tipikus mintája a következő komponensekből áll:

```
flowchart LR
  A[API szolgáltatás] -->|audit esemény| B[Naplózó kliens]
  B -->|strukturált üzenet| C[Napló szállító csatorna]
  C -->|ellenőrzött bejegyzés| D[Központi naplótároló]
  D -->|lekérdezés| E[Audit lekérdező felület]
  F[Külső aláíró szolgáltatás] -->|időbélyeg, aláírás| D
```

A komponensek felelőssége:

- **API szolgáltatás:** az üzleti művelet végrehajtása után audit eseményt bocsát ki. Nem vár a naplózás sikerére, ha az nem blokkoló; ha a naplózás szinkron, akkor a válaszidőbe beleszámít.
- **Naplózó kliens:** a szolgáltatás futásidejébe ágyazott vagy oldalkocsi (sidecar) komponens, amely a strukturált eseményt a megfelelő formátumban továbbítja. Felelőssége a séma érvényesítése, a korrelációs azonosító átörökítése és a helyi pufferelés, ha a szállító csatorna nem elérhető.
- **Napló szállító csatorna:** üzenetküldő rendszer (például Kafka, RabbitMQ vagy felügyelt streaming szolgáltatás), amely a naplóeseményeket aszinkron továbbítja a központi tárolóba. A csatornának biztosítania kell az üzenetek sorrendjét vagy legalább a sorrend rekonstruálhatóságát, és a tartós tárolást a rövid kiesések áthidalására.
- **Központi naplótároló:** append-only, integritásvédett adatbázis vagy tárterület. Itt történik a bejegyzések hash-láncolása, adott esetben aláíratása, valamint a megőrzési osztályok kezelése.
- **Külső aláíró szolgáltatás:** opcionális, de erősebb bizonyítékot adó komponens, amely időbélyeggel és digitális aláírással látja el a naplócsomagokat. Ezzel a naplótároló üzemeltetője sem tudja utólag manipulálni a bejegyzéseket.
- **Audit lekérdező felület:** a napló olvasására szolgáló felület, amelyhez szigorú hozzáférésvezérlés és a lekérdezések naplózása tartozik.

A naplófolyamot úgy kell méretezni, hogy a naplózás ne váljon szűk keresztmetszetté. A naplóesemények érkezési rátája jellemzően arányos az üzleti tranzakciók számával, ezért a napló infrastruktúrának legalább akkora tartalék kapacitással kell rendelkeznie, mint az üzleti rendszernek. A Little-törvény itt is alkalmazható: ha a napló feldolgozási ideje W és az érkezési ráta λ, akkor a rendszerben lévő naplóesemények átlagos száma $L = \lambda W$. A pufferelés és a backpressure tervezésekor ezt a mutatót érdemes figyelembe venni.

22.3.7 Labor: auditnaplózási és megőrzési terv

A labor célja, hogy a hallgató egy konkrét, de fiktív rendszerre auditnaplózási és megőrzési tervet készítsen. A labor nem igényel éles rendszert; a kiinduló állapot egy leírás, a leadandó eredmény pedig egy verziózott dokumentum.

Kiinduló állapot. A hallgató a következő fiktív rendszerleírást kapja: „A `rendeles-kezelo` nevű belső szolgáltatás REST API-n keresztül kezeli a megrendeléseket. A rendszerhez belső munkatársak férnek hozzá webes felületen, szerepkörük `support` vagy `admin`. Az adatbázis PostgreSQL, az API Node.js alapú. A rendszer jelenleg csak általános hibanaplót ír, auditnapló nincs. A megrendelések között szerepel ügyfélazonosító, szállítási cím és kapcsolattartási adat. A szolgáltatás egyetlen adatközponti környezetben fut.”

Elvárt eredmény. A hallgató készítsen egy reprodukálható dokumentumot vagy jegyzetfüzetet, amely a következőket tartalmazza:

1. **Kontextus:** a rendszer rövid leírása, a naplózási célok, a hatókör és a rendszerhatár. Itt kell felsorolni a figyelembe vett érintetti köröket is: ügyfél, munkatárs, auditáló, üzemeltető.
2. **Döntési napló (ADR):** legalább három döntés rögzítése a naplózással kapcsolatban. Mindegyikhez tartozzon kontextus, döntés, alternatívák, következmények és állapot. Például: a napló szállítási csatorna kiválasztása, az IP-cím tárolásának módja, a megőrzési idő meghatározása.
3. **Naplózási politika:** a naplózandó események listája, a naplóbejegyzés sémája, a mezők jelentése, az álnevesítési szabályok. Az eseményeket táblázatban kell megadni: esemény, kiváltó feltétel, naplózott mezők, jogosultság.

4. **Integritási intézkedések:** a hash-lánc vagy aláírás alkalmazásának leírása, a szerepkörök szétválasztása, a hozzáférés-vezérlés szabályai.
5. **Megőrzési terv:** a naplóosztályok, megőrzési idők, tárolási rétegek, törlési eljárás. A törlési eljárásnál ki kell térni arra, hogy az érintetti törlési kérelem hogyan érvényesíthető a naplóban.
6. **Ellenőrzési szempontok:** mit kell mérni vagy tesztelni ahhoz, hogy a terv működését igazoljuk. Például: a naplóbejegyzés megjelenési ideje, a napló törölhetetlensége, a hozzáférési naplózás megléte.
7. **Mért és várt eredmények:** külön táblázatban kell szerepeltetni, hogy mely paramétereket mért a hallgató (ha mért), és melyek a tervben elvárt értékek. Ha nincs tényleges mérés, akkor ezt kifejezetten jelezni kell, és a „mért” oszlopban „nem mérve” szerepeljen. Például:

Paraméter	Várt érték	Mért érték	Megjegyzés
Auditbejegyzés hozzáadásának késleltetése	50 ms	nem mérve	csak terv szinten
Napló napi mérete	1 GB	nem mérve	becslés
Bejegyzés törlésének észlelése	100%	nem mérve	teszt terv szerint

A labor eredményessége akkor igazolt, ha a dokumentum visszakereshető módon tartalmazza a döntéseket, a naplósémát, a megőrzési osztályokat és az ellenőrzési szempontokat. A laborban nem szabad valós ügyféladatot vagy éles hitelesítő adatot használni; minden példaadatok szintetikusak legyenek.

Visszaállítási út. Mivel a labor dokumentum készítés, destruktív művelet nincs. Ha a hallgató mégis tesztkörnyezetben próbálja ki a naplózást, akkor a tesztkörnyezetet izolálni kell, és a próba után a tesztadatokat törölni kell. A visszaállítás itt a tesztkörnyezet eredeti állapotba állítását jelenti.

22.3.8 Tipikus hibák és ellenőrzőlista

A naplózás tervezése során gyakori hibák:

- **Minden naplózása indoklás nélkül.** A túlzott naplózás megnöveli a tárolási költséget, lassítja a rendszert, és adatvédelmi kockázatot jelent. A naplózási eseményeket a bizonyítási igény alapján kell kiválasztani.
- **A napló örök megőrzése.** A határozatlan idejű megőrzés GDPR-ellenes és szükségtelen. Minden naplóosztályhoz tartozzon lejárati idő és automatikus törlés.
- **Túl széles hozzáférés.** Ha a fejlesztői csapat minden naplóhoz hozzáfér, a személyes adatok védelme sérül, és a belső fenyegetés kockázata nő.
- **A naplózó komponens és az alkalmazás osztozik a hitelesítő adatokon.** Ez lehetővé teszi, hogy egy kompromittált alkalmazás törölje a saját naplóját. Külön szolgáltatásfiókot kell használni, minimális jogosultsággal.
- **Hiányzó korrelációs azonosító.** Az elosztott rendszerben a korrelációs azonosító nélkül az események nem köthetők össze, így a vizsgálat lehetetlenné válik.
- **Nem strukturált naplóformátum.** A szabadszöveges naplókat nehéz elemezni, és a mezők hiánya megnehezíti az adattakarékosság ellenőrzését.

Az ellenőrzőlista a következőket foglalja össze:

- ☐ Az auditnaplózási politika meghatározott és verziózott.
- ☐ A naplózandó események listája a bizonyítási igényből származik.
- ☐ A naplóbejegyzés sémája strukturált, verziózott és dokumentált.
- ☐ Az időbélyeg UTC, ezredmásodperc pontosságú, NTP-szinkronizált.
- ☐ A korrelációs azonosító minden érintett komponensben átadódik.
- ☐ Az integritást hash-lánc vagy digitális aláírás biztosítja.
- ☐ A naplóíró, naplókezelő és naplóolvasó szerepkörök szét vannak választva.
- ☐ A napló hozzáférése a legkisebb jogosultság elve szerint korlátozott.
- ☐ A személyes adatokat tartalmazó mezők esetén álnevesítés van érvényben.
- ☐ A megőrzési idő osztályonként meghatározott, jogalappal alátámasztva.
- ☐ A törlési eljárás automatizált és a törlés ténye naplózott.
- ☐ Az érintetti törlési kérelem eljárása a naplóra is ki van dolgozva.
- ☐ A napló infrastruktúra rendelkezésre állása és kapacitása tervezett.

22.3.9 Összefoglalás és önellenőrző kérdés

Az auditálhatóság a bizonyítható működés alapfeltétele. A jól megtervezett auditnapló strukturált, integritásvédett, szigorúan hozzáférhető és meghatározott ideig megőrzött. A napló azonban személyes adatokat is tartalmazhat, ezért az adatvédelmi elvek – célhoz kötöttség, adattakarékosság, álnevesítés, korlátozott megőrzés – a naplózásra is vonatkoznak. A naplózási architektúra komponenseit felelősséggel és interfésszel együtt kell megtervezni, a döntéseket ADR-ben rögzíteni, a működést pedig mérhető ellenőrzési pontokkal igazolni.

A következő egység (Etika, fairness és társadalmi hatás) az etika, fairness és társadalmi hatás kérdéseivel foglalkozik. A naplózás ott is központi szerepet játszik: a döntések átláthatósága és a rendszer viselkedésének utólagos vizsgálata csak akkor lehetséges, ha a naplózási alapok rendben vannak. Az auditnapló nem csupán jogi kényszer, hanem az etikus működés eszköze is.

Önellenőrző kérdés: Tervezze meg egy fiktív rendszer auditnaplózási sémáját, amelyben egy ügyfél az adatai törlését kéri. Mely mezőket lehet álnevesíteni, melyeket kell törölni, és hogyan őrizhető meg a napló bizonyító ereje a törlés után is? Indokolja a döntést, és azonosítsa a trade-offokat.

22.4 Etika, fairness és társadalmi hatás

A szoftverarchitektúra hagyományosan a szerkezetről, a minőségi attribútumokról és a technológiai kompromisszumokról szól. Az utóbbi évtizedben azonban egyre erősebb az a felismerés, hogy a technológiai döntések nem légüres térben születnek: hatással vannak emberekre, közösségekre, társadalmi csoportokra és a természeti környezetre is. Egy hitelbírálati modell eldöntheti, ki jut lakáshitelhez; egy tartalomajánló algoritmus befolyásolhatja a közbeszédet; egy egészségügyi előrejelző rendszer hibája életeket érinthet. Az architekt, aki a rendszer alapvető szerkezetét, adatáramlását és komponenseit tervezi, óhatatlanul részt vesz ezekben a hatásokban – még akkor is, ha ezt nem tudatosítja.

Ez a fejezet azt vizsgálja, hogyan lehet az etikai és társadalmi dimenziót beépíteni az architekturális gondolkodásba. Nem jogi tanácsadást ad, és nem kínál kész morális ítéleteket. A cél az, hogy az architekt felismerje a döntések etikai töltetét, tudatosan kezelje az elfogultság, az átláthatóság és az elszámoltathatóság kérdéseit, és dokumentálja a vitákat, kompromisszumokat. A fejezet végén egy gyakorlati eszközt, az etikaiság- és hatásvizsgálati checklistet építjük fel, amelyet a laborban alkalmazni is fogunk.

22.4.1 A technológiai döntések etikai és társadalmi dimenziója

Az architekturális döntések ritkán pusztán technikai jellegűek. Amikor egy rendszer határát meghúzzuk, meghatározzuk, mely szereplők férhetnek hozzá az adatokhoz, milyen sorrendben történik a feldolgozás, hol tároljuk az információt, és milyen hibatűrési mechanizmusokat építünk be, valójában értékválasztásokat is teszünk. A rendszerhatár például nemcsak biztonsági, hanem hatalmi kérdés is: ki kontrollálja az adatot, ki dönthet a hozzáférésről, és ki viseli a hiba következményét.

Az etikai dimenzió azt vizsgálja, hogy egy döntés milyen normák, értékek és kötelességek mentén igazolható. A társadalmi dimenzió pedig azt, hogy a döntés milyen hatást gyakorol a különböző érintett csoportokra – a közvetlen felhasználóktól a közvetett érintettekig, például a rendszer által kiszolgált közösségre vagy a jövő generációira. A kettő összefügg: egy etikailag igazolható döntés társadalmi hatása lehet negatív, ha például a kontextust rosszul mértük fel.

Az architektúrában az etikai kérdések gyakran a minőségi attribútumok mögött rejtőznek. A rendelkezésre állás például nemcsak technikai mutató, hanem azt is jelzi, kinek a szolgáltatása sérül, ha a rendszer leáll. A skálázhatóság döntései meghatározzák, hogy a rendszer képes-e kiszolgálni a peremhelyzetben lévő felhasználókat, vagy csak a nagy forgalmú régiókra optimalizál. A biztonsági architektúra kialakítása során pedig egyensúlyt kell találni a védelem és a használhatóság között, ami gyakran a hozzáférés korlátozását jelenti – ez pedig társadalmi következményekkel jár, ha például egy alacsony jövedelmű csoport számára nehezebbé válik a hitelesítés.

Az etikai elemzéshez hasznos kereteket adnak a klasszikus etikai elméletek. A konzekvencialista megközelítés a döntés következményeit mérlegeli: melyik alternatíva maximalizálja a hasznot és minimalizálja a kárt az összes érintett számára? A deontológiai megközelítés a kötelességekre és jogokra összpontosít: vannak-e olyan alapelvek, amelyeket nem szabad megsérteni, függetlenül a következményektől? Az erényetika azt kérdezi, milyen szakmai karaktert fejez ki a döntés. A gondoskodásetika pedig a kapcsolatokra és a kiszolgáltatott szereplők védelmére helyezi a hangsúlyt.

Ezek a keretek nem adnak egyértelmű választ, de segítenek strukturálni a vitát. Például egy adatmegőrzési idő meghatározásakor a konzekvencialista szempont lehet a jövőbeli elemzési lehetőség, a deontológiai szempont az adatalany magánszférához való joga, az erényetikai szempont pedig az architekt szakmai integritása a minimalizálás elvének alkalmazásában. A döntés dokumentálásakor ezeket a szempontokat érdemes explicit módon rögzíteni, nemcsak a végső választást.

A társadalmi dimenzió elemzésének alapvető eszköze a stakeholder-elemzés. A C4 context szintjén már megszoktuk, hogy a rendszerhatárt és a külső szereplőket feltüntetjük. Az etikai elemzés ezt kiterjeszti: nem elég azt tudni, hogy egy szereplő „felhasználó” vagy „adminisztrátor”, hanem azt is meg kell érteni, milyen érdekei, sebezhetőségei és hatalmi pozíciói vannak. Egy egészségügyi rendszerben például az orvos, a beteg, a biztosító, a kórházi menedzsment és a szabályozó hatóság mind érintett, de eltérő módon: a beteg a legkiszolgáltatottabb, mert az adatai és az egészsége is a rendszeren múlik.

A társadalmi hatásvizsgálat során a következő kérdéseket érdemes feltenni: kik az érintettek, milyen hatás éri őket (pozitív, negatív, közvetlen, közvetett), ki viseli a kockázatot, ki részesül az előnyből, és van-e olyan csoport, amely aránytalanul nagy terhet kap. Ez a fajta elemzés hasonlít a threat modelinghez, de a fenyegetések helyett a társadalmi következményekre összpontosít. A bizalmi határok itt is fontosak: hol változik az adat tulajdonlása, hol lép be egy harmadik fél, hol szűnik meg a felhasználó kontrollja.

A környezeti hatás sem hagyható ki. A felhőalapú rendszerek energiafogyasztása, a modellbetanítás szén-dioxid-kibocsátása, az adatközpontok hűtési igénye mind a társadalmi dimenzió része. A FinOps fejezetben tárgyalt költségoptimalizálás itt új értelmet nyer: a költség nemcsak pénzügyi, hanem ökológiai is. Az architekt döntése, hogy egy számítást a felhőben vagy az edge-en végez, hatással van az energiafogyasztásra és az adatáramlásra is.

22.4.2 Elfogultság, átláthatóság és elszámoltathatóság

Az elfogultság (bias) a technológiai rendszerekben több szinten jelentkezik. A legismertebb az adatokban és a modellekben megjelenő torzítás, de az architektúrában legalább ilyen fontos a tervezési folyamatban megjelenő elfogultság. Amikor egy architekt kiválaszt egy technológiát, mert azt ismeri, vagy egy rétegződést, mert az a „megszokott”, akkor nem feltétlenül a rendszer követelményeihez igazodik, hanem saját kognitív torzításaihoz. Ez nem bűn, hanem emberi sajátosság, de tudatosítani és mérsékelni kell.

Az adatokban megjelenő elfogultság formái közé tartozik a reprezentációs torzítás, amikor bizonyos csoportok alul- vagy felülreprezentáltak a tanítóadatban; a mérési torzítás, amikor a gyűjtött adat nem azt méri, amit a döntéshez használni szeretnénk; és a történelmi torzítás, amikor a múltbeli adatok a korábbi diszkriminációt örökítik át. Egy felvételi előszűrő rendszer, amelyet korábbi sikeres jelentkezők adatain tanítottak, hajlamos lesz a korábbi felvételi gyakorlat hibáit reprodukálni – például a nemi vagy etnikai alapú kizárást –, még ha a modell nem is kapja meg ezeket a jellemzőket közvetlenül, mert a korrelált jellemzőkből kikövetkeztethetők.

Az architekturális elfogultság ennél finomabb: a rendszer felépítése maga is termelhet egyenlőtlenséget. Ha egy rendszer szinkron hívásokra épül, és egy lassú külső szolgáltatás blokkolja a választ, akkor azok a felhasználók szenvednek, akik a lassú régióból érkeznek. Ha egy API csak bizonyos nyelveken ad vissza hibaüzenetet, akkor a nem beszélő felhasználók kiszorulnak. Ha egy adatmodell nem tudja kezelni a nem bináris nemi identitást, akkor egy egész csoport láthatatlanná válik.

Az elfogultság kezelésének első lépése a felismerés és a dokumentálás. A döntési naplóban (ADR) érdemes rögzíteni, hogy milyen feltételezésekkel éltünk, milyen adatforrásokat használtunk, és milyen csoportokra nézve van bizonytalanságunk. A második lépés a mérséklés: diverzifikált adatgyűjtés, ellenőrző minták, fairness-metrikák bevezetése, és a modell döntéseinek magyarázhatósága. A harmadik lépés a folyamatos monitorozás: az elfogultság nem statikus, a rendszer viselkedése a bemenő adatokkal együtt változik.

Az átláthatóság (transparency) azt jelenti, hogy a rendszer működése, döntései és korlátai érthetők az érintettek számára. Ez nem azonos a nyílt forráskóddal, bár az is lehet része. Az architekturális átláthatóság magában foglalja a döntési logika, az adatáramlás, a modellváltozatok és a hibatűrési mechanizmusok dokumentálását. Egy ajánlórendszer esetén a felhasználónak joga van tudni, miért lát egy adott tartalmat; egy hitelbírálati rendszer esetén a kérelmezőnek joga van megismerni a döntés fő okait. Az architekt feladata, hogy ezt a magyarázhatóságot a rendszer szerkezetébe építse: naplózza a döntési lépéseket, verziózza a modelleket, és elérhetővé tegye a bemeneti jellemzők fontosságát.

Az átláthatóság technikai feltétele a nyomon követhetőség. Az Auditálhatóság, naplózás és adatmegőrzés fejezetben tárgyalt auditálhatóság és naplózás itt új célt kap: nemcsak a jogi megfelelés, hanem az etikai számonkérhetőség miatt is. A rendszernek tudnia kell válaszolni arra a kérdésre, hogy egy adott döntés milyen bemenetek alapján, milyen modellverzióval, milyen paraméterekkel született. Ehhez a döntési naplóban a modellverziót, a bemeneti adatok kivonatát

és a döntési küszöböt is rögzíteni kell. A naplózás természetesen nem sértheti az adatvédelmet: a naplóbejegyzések nem tartalmazhatnak közvetlen személyes adatot, csak álnevesített azonosítókat.

Az elszámoltathatóság (accountability) az átláthatóság folytatása: ha tudjuk, hogyan született a döntés, akkor meg is tudjuk nevezni, ki viseli érte a felelősséget. A szoftverrendszerekben ez a felelősség megosztott: az adatgyűjtő, a modellfejlesztő, az architekt, az üzemeltető és a termékmenedzser mind részt vesz a döntési láncban. Az architekt felelőssége, hogy a rendszer szerkezete lehetővé tegye a felelősségi körök azonosítását: ki hagyta jóvá a modellfrissítést, ki konfigurálta a küszöböt, ki döntött az adatforrás cseréjéről.

Az elszámoltathatóság szervezeti eszköze a szerepkörök és döntési jogkörök egyértelmű meghatározása, amit a governance fejezetben már tárgyaltunk. Ethikai szempontból ez kiegészül azzal, hogy a döntésekhez etikai felülvizsgálat is tartozik. Egy nagy kockázatú rendszernél – például egészségügyi előrejelzésnél vagy automatizált felvételi szűrésnél – érdemes etikai bizottságot vagy legalább egy kijelölt etikai felelőst bevonni a tervezési folyamatba. Ez nem bürokratikus teher, hanem a vakfoltok csökkentésének eszköze.

Az elszámoltathatóság technikai feltétele a döntések visszakereshetősége. Az ADR-ek, a CI/CD pipeline-ok naplói, a modellregiszter és a konfigurációkezelés mind hozzájárulnak ahhoz, hogy egy incidens után rekonstruálni lehessen, mi történt és ki volt a döntéshozó. A Logs, metrics, traces és OpenTelemetry fejezetben tárgyalt observability eszközök – logok, metrikák, trace-ek – itt etikai dimenziót kapnak: nemcsak a rendszer állapotát, hanem a döntések következményeit is monitorozni kell.

22.4.3 A döntések dokumentálása és a viták kezelése

Az etikai szempontú döntések dokumentálása nem különül el a megszokott architekturális dokumentációtól, hanem annak kiterjesztése. Az ADR formátuma alkalmas arra, hogy az etikai dilemmákat is rögzítsük. A kontextus részben leírjuk a rendszer és az érintettek helyzetét; a döntés részben a választott megoldást; az alternatívákban a felmerült etikai szempontokat; a következményekben pedig a társadalmi hatásokat és a feladott lehetőségeket.

Példa egy ADR-re egy hitelbírálati rendszerben:

Cím: ADR-014: A hitelbírálati modell döntési küszöbének meghatározása **Kontextus:** Az automatikus hitelbírálat során a modell egy kockázati pontszámot ad, amely alapján a rendszer elfogadja vagy elutasítja a kérelmet. Az érintett felek: kérelmezők, hitelintézet, szabályozó hatóság, valamint a társadalom szélesebb értelemben, mivel a hitelhez jutás befolyásolja a vagyoni egyenlőtlenséget. **Döntés:** A kezdeti küszöböt úgy állítjuk be, hogy a várható nemteljesítési arány 5% alatt maradjon, de a döntési naplóban rögzítjük a küszöbértéket, a modellverziót és a fairness-metrikát (például a csoportonkénti elutasítási arány különbségét). A küszöböt félévente felülvizsgáljuk. **Alternatívák:** (1) Konzervatív küszöb, amely minimalizálja a hitelintézet kockázatát, de kizárja a határesetben lévő kérelmezőket. (2) Megengedő küszöb, amely több hitelhez juttat, de növeli a nemteljesítés kockázatát. (3) A döntés emberi felülvizsgálatra utalása minden határesetben, ami drágább, de lehetőséget ad az egyedi körülmények figyelembevételére. **Következmények:** A választott megoldás kompromisszum a pénzügyi kockázat és a hozzáférés között. A fairness-metrika bevezetése lehetővé teszi a torzítás monitorozását. A feladott alternatíva az emberi felülvizsgálat magasabb költsége. **Állapot:** Elfogadva, felülvizsgálat: következő negyedév.

Ez az ADR nemcsak a technikai döntést rögzíti, hanem az etikai mérlegelést is láthatóvá teszi. A döntési napló így válik a szervezeti emlékezet eszközévé: amikor egy új csapattag vagy auditor kérdezi, miért így működik a rendszer, az ADR választ ad.

A viták kezelése az etikai döntések természetes velejárója. Különböző szereplők eltérő értékeket képviselnek: a termékmenedzser a bevételt, a jogász a megfelelést, az architekt a technikai fenntarthatóságot, a felhasználói képviselő a hozzáférhetőséget. Az etikai viták gyakran azért válnak érzelmivé és terméketlenné, mert a felek nem tudják elkülöníteni a tényeket az értékválasztásoktól. A strukturált vitakezelés célja, hogy a döntési folyamat átlátható és dokumentált legyen, és hogy minden érintett szempont megjelenjen.

Egy gyakorlati módszer a következő lépésekből áll:

1. **Problémaazonosítás:** mi a konkrét döntés, és miért van etikai töltete?
2. **Tényfeltárás:** milyen adatok, mérések, jogi követelmények és technikai korlátok ismertek?
3. **Értékválasztások azonosítása:** mely alapelvek ütköznek? (például hatékonyság vs. méltányosság, innováció vs. elővigyázatosság)
4. **Alternatívák elemzése:** milyen lehetőségek vannak, és milyen következményekkel járnak az érintettekre nézve?
5. **Döntés és indoklás:** melyik alternatívát választjuk, és miért?
6. **Felülvizsgálat:** milyen feltételek mellett vizsgáljuk felül a döntést, és milyen mutatókat figyelünk?

Ez a folyamat hasonlít a tervezési döntéseknél alkalmazott ATAM vagy CBAM módszertanhoz, csak a hangsúly az etikai dimenzión van. Fontos, hogy a vitában legyen egy moderátor, aki biztosítja, hogy minden érintett szempont elhangozzon, és hogy a döntés ne a hangosabb vagy hatalmasabb fél győzelme legyen. A pszichológiai biztonság itt kulcsfontosságú: ha a csapattagok félnek elmondani az ellenvéleményüket, az etikai vakfoltok rejtve maradnak.

A viták dokumentálása nem azt jelenti, hogy minden hozzászólást rögzítünk, hanem hogy a döntési naplóban megjelenik a mérlegelt alternatíva és az elutasítás oka. Ez a későbbi felülvizsgálatot is segíti: ha a körülmények megváltoznak, vissza lehet nyúlni a korábbi érvekhez.

22.4.4 Labor: Etikaiság- és hatásvizsgálati checklist

A következő labor célja, hogy egy konkrét, kis rendszeren vagy rendszertervön alkalmazzuk a fejezetben tárgyalt fogalmakat. A feladat egy etikaiság- és hatásvizsgálati checklist elkészítése és kitöltése. A checklist nem helyettesíti a jogi megfelelési auditot, és nem ad abszolút bizonyosságot, de segít a vakfoltok feltárásában és a döntések dokumentálásában.

Kiinduló állapot: Válassz egy kisméretű rendszert vagy funkciót, amelyet a korábbi fejezetekben terveztél, vagy amelyet a munkahelyeden ismersz. Jó példa egy belső dolgozói teljesítményértékelő alkalmazás, egy ügyfélszolgálati chatbot, egy automatikus számlaosztályozó, vagy egy egyszerű ajánló modul. A rendszer legyen elég kicsi ahhoz, hogy a checklistet egy ülésben ki tudd tölteni, de legyen benne legalább egy automatizált döntés vagy adatfeldolgozási lépés.

Elvárt eredmény: Egy verziózott dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza: - a rendszer rövid leírását és a rendszerhatárt; - a stakeholder-elemzést: legalább öt érintett szereplő, csoport vagy entitás, azok érdekeivel és sebezhetőségével; - az etikaiság- és hatásvizsgálati checklistet, kitöltve a jelenlegi állapottal (mért) és a célállapottal (várt); - egy döntési naplót (ADR formátumban), amely rögzíti a legfontosabb etikai dilemmát és a választott megoldást; - az ellenőrzési szempontokat: hogyan fogjuk mérni, hogy a célállapot teljesül-e.

A checklist szerkezete:

Az alábbi táblázat egy lehetséges sablon. A „Szempont” oszlop az etikai dimenziót jelöli, a „Kérdés” segít a feltárásban, a „Jelenlegi állapot (mért)” azt írja le, ami most van, a „Célállapot

(várt)” azt, amit el szeretnénk érni, az „Ellenőrzési módszer” pedig azt, hogyan mérjük a változást. A „Felelős” és „Határidő” oszlopok segítik a nyomon követést.

Szempont	Kérdés	Jelenlegi állapot (mért)	Célállapot (várt)	Ellenőrzési módszer
Elfogultság	Milyen adatforrásokat használ a rendszer, és vannak-e reprezentációs torzítások?	A tanítóadat 80%-a egyetlen régióból származik.	A tanítóadat legalább 30%-ban lefedi a peremhelyzetű csoportot.	Adatprofilozás negyedévente.
Átláthatóság	Hol rögzíti a rendszer a döntési logikát és a modellverziót?	A döntési logika a kódban szétszórva, modellverzió nincs naplózva.	Minden döntéshez tartozik modellverzió és bemeneti kivonat a naplóban.	Naplóminta ellenőrzése.
Elszámoltathatóság	Ki hagyja jóvá a modellfrissítést és a küszöb módosítását?	Nincs formális jóváhagyási folyamat.	Minden modell-frissítéshez ADR és etikai felülvizsgálat tartozik.	ADR-ek és jóváhagyási napló.
Társadalmi hatás	Mely csoportokat érint negatívan a rendszer, és hogyan mérjük ezt?	Nincs mérés a csoportonkénti elutasítási arányról.	Bevezetjük a fairness-metrikát és riasztást állítunk be a küszöb átlépésére.	Dashboard és riasztás.
Környezeti hatás	Mekkora a rendszer energia-fogyasztása és szén-dioxid-kibocsátása?	Nincs mérés.	Becslés készül a felhőköltség és az energia alapján.	Havi jelentés.

A checklist kitöltése során fontos, hogy a „mért” és a „várt” értékeket különválasszuk. A „mért” a jelenlegi, ellenőrizhető állapotot jelenti; ha nincs mérés, akkor azt írjuk, hogy „nem mért”, nem pedig feltételezést. A „várt” egy konkrét, lehetőleg mérhető cél, nem általános jókívánság.

A döntési napló elkészítése:

Válassz egy etikai dilemmát a rendszerben, például azt, hogy egy chatbot milyen hangnemben utasítson el egy kérést, vagy hogy egy automatikus szűrő milyen küszöbnél hívjon emberi felülvizsgálatot. Írd le ADR formátumban a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. A döntés indoklásában hivatkozz a checklistben azonosított szempontokra.

Ellenőrzési szempontok:

A labor akkor sikeres, ha: - a stakeholder-elemzés legalább öt érintettet tartalmaz, és mindegyiknél szerepel az érdek és a sebezhetőség; - a checklist legalább öt szempontot lefed, és minden sorban a „mért” és a „várt” elkülönül; - az ADR tartalmazza a kontextust, a döntést, legalább két alternatívát, a következményeket és az állapotot; - az ellenőrzési módszerek konkrétak (például

mérési gyakoriság, felelős, riasztási küszöb); - a dokumentum verziózott, és a döntésekhez visszakereshető hivatkozás tartozik.

Visszaállítási út: Mivel a labor nem éles rendszert módosít, nincs szükség visszaállításra. Ha azonban a checklist alapján úgy döntesz, hogy a valós rendszeren változtatni kell, akkor a változtatást a megszokott change management folyamat szerint kell végrehajtani: a konfigurációt, a modellt vagy a kódot verziózni kell, és a visszagörgetési tervet előre rögzíteni.

22.4.5 Ellenőrzőlista a fejezethez

- Az architekturális döntéseknél feltártam-e a közvetlen és közvetett érintetteket?
- Dokumentáltam-e a döntés etikai dilemmáját és a mérlegelt alternatívákat?
- Beépítettem-e az átláthatóságot és a nyomon követhetőséget a rendszer szerkezetébe?
- Meghatároztam-e a fairness-metrikákat és azok riasztási küszöbeit?
- Kijelöltem-e az elszámoltathatósági szerepköröket a döntési láncban?
- A viták során biztosítottam-e, hogy minden érintett szempont megjelenjen és dokumentálva legyen?

22.4.6 Önellenőrző kérdés

Egy csapat egy automatizált állásinterjú-értékelő rendszert tervez, amely videóinterjúk alapján pontozza a jelöltek kommunikációs készségét. A rendszer a hangszínből, a beszédtempóból és a szóhasználatból következtet. Milyen etikai és társadalmi kockázatokat kell az architektnek felvetnie, és hogyan lehet ezeket a rendszer szerkezetébe beépített módon mérsékelni? Indokold meg, miért nem elegendő csupán a modell pontosságát növelni.

23 Observability és üzemeltetés

23.1 Logs, metrics, traces és OpenTelemetry

Az observability (megfigyelhetőség) gyakorlati alapja három jel: a napló, a metrika és az elosztott trace. A három jel önmagában is hasznos, de igazi értékük akkor mutatkozik meg, ha közös kontextussal összekapcsolhatók. Ez a fejezet bemutatja, hogyan kapcsolódik össze a három jel, miért érdemes strukturált naplót írni, hogyan működnek az elosztott trace-ek, és hogyan használható az OpenTelemetry a telemetriai adatok szabványos gyűjtésére. A fejezet végén egy ellenőrizhető labor segítségével telemetriai tervet készítünk, amely a következő fejezet (Dashboardok, riasztások és runbookok) alapjául szolgál.

23.1.1 A három jel és a kontextus összekapcsolása

Az observability célja, hogy a rendszer külső jelei alapján magyarázni tudjuk a belső állapotot, és hiba esetén minél gyorsabban lokalizáljuk a problémát. Ehhez három alapvető jel áll rendelkezésre:

- **Napló (log):** egy adott időpontban bekövetkezett esemény részletes leírása. Jellemzően ember által olvasható, de a gép általi feldolgozáshoz strukturált formátum szükséges.
- **Metrika (metric):** időbeli aggregált mérőszám, például a kérések száma másodpercenként, a válaszidő eloszlása vagy a memóriahasználat pillanatnyi értéke. A metrika gyors áttekintést ad a rendszer állapotáról, de nem mondja meg egy konkrét kérés sorsát.
- **Elosztott trace (trace):** egy kérés útjának végigkövetése a rendszer különböző szolgáltatásain keresztül. A trace hierarchikus spanekből áll; minden span egy munkaegységet jelöl, és a gyerek- illetve szülő kapcsolatok mutatják a hívásláncot.

A kontextus összekapcsolása azt jelenti, hogy mindhárom jel ugyanazokat a korrelációs azonosítókat hordozza. A leggyakoribb ilyen azonosítók: `trace_id` (az egész kérést azonosítja), `span_id` (az adott lépést), `service.name`, `environment`, `version`, `host.name`, valamint üzleti azonosítók, például `order_id` vagy `customer_id`. Ha a napló, a metrika és a trace közös kontextust használ, akkor egy hiba elemzésekor az alábbi utat járhatjuk be:

1. A dashboardon kiugró hibaarányt látunk egy metrikán.
2. Lefúrunk a hibás kérés nyomvonalára (trace).
3. A trace egyik spanéből megnyitjuk a kapcsolódó naplósorokat.
4. A naplókban található üzleti azonosító segítségével megtaláljuk az érintett felhasználót vagy tranzakciót.

Ezt a folyamatot nevezzük „correlated telemetry”-nek. A kontextusmezőket minden telemetriai forrásnál következetesen, ugyanazokkal a nevekkel kell használni, különben a korreláció nem működik.

23.1.2 Strukturált napló

A klasszikus, szöveges napló emberi olvasásra alkalmas, de nem gépi feldolgozásra. Egy ilyen sor:

```
2025-04-10 08:30:12 INFO order-service: User 123 created order 456 with amount 12499
```

A sorból csak bonyolult reguláris kifejezésekkel lehetne kinyerni a mezőket, és a keresés is nehézkes. A strukturált napló ezzel szemben gép által értelmezhető formátumú, jellemzően JSON vagy JSON Lines. A fenti sor strukturált megfelelője:

```
{
  "timestamp": "2025-04-10T08:30:12.123Z",
  "level": "INFO",
  "service.name": "order-service",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "span_id": "00f067aa0ba902b7",
  "message": "Order created",
  "order_id": "ord_12345",
  "customer_id": "cust_98765",
  "amount": 12499,
  "currency": "HUF"
}
```

A strukturált napló előnyei:

- **Kereshetőség:** az egyes mezőkre szűrhetünk anélkül, hogy a szövegben kellene keresni.
- **Korreláció:** a `trace_id` és `span_id` közvetlenül összekapcsolja a naplót a trace-szel.
- **Gépi feldolgozás:** a naplókat könnyen lehet aggregálni, riasztásokat építeni rájuk, vagy gépi tanulással elemezni.
- **Konzisztencia:** a mezőnevek standardizálhatók, így a különböző szolgáltatások naplói egységesek lesznek.

A strukturált naplózásnál kerülendő a tagolatlan, több mondatból álló `message` mező. Ha részletes szöveget akarunk, azt tegyük egy külön mezőbe, például `detail`, és a `message` maradjon rövid, stabil azonosító, amely alapján a naplót rendszerezni lehet.

Fontos, hogy a naplókba ne kerüljön személyes adat, jelszó, token vagy belső URL. A fenti példában a `customer_id` és `order_id` álnevesített, nem valós azonosítók. Éles rendszerben a személyes adatot vagy titkosítani, vagy maskolni kell, vagy egyáltalán nem szabad naplózni (lásd az adatvédelemről szóló fejezeteket).

23.1.3 Metrika

A metrika időbeli aggregált mérőszám, amely a rendszer makroszkopikus állapotát mutatja. Négy fő típusa van:

- **Counter (számláló):** monoton növekvő érték, például a beérkezett HTTP kérések száma. Csak növekedhet, visszaállításkor (pl. újraindításkor) a gyűjtőrendszer kezeli a nullázódást.
- **Gauge (pillanatnyi érték):** aktuális állapotot mutat, például a szabad memória mérete vagy az aktív kapcsolatok száma.
- **Histogram (eloszlás):** az értékek gyakorisági eloszlását rögzíti előre definiált intervallumokban (bucketekben), emellett összeget és darabszámot is számol. Alkalmas válaszidők mérésére, mert belőle a percentilisek (pl. p95) aggregálhatók.
- **Summary (összefoglaló):** kvantiliseket számol közvetlenül, de az értékek nem aggregálhatók tovább, ezért elosztott rendszerekben kevésbé használják.

Példa Prometheus-szövegformátumban:

```
# HELP http_requests_total Az összes HTTP kérés száma.
# TYPE http_requests_total counter
http_requests_total{service="order-service",method="POST",path="/api/orders",status="201"} 1024

# HELP http_request_duration_seconds HTTP kérés időtartama másodpercben.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{service="order-service",method="POST",le="0.1"} 350
http_request_duration_seconds_bucket{service="order-service",method="POST",le="0.5"} 780
http_request_duration_seconds_bucket{service="order-service",method="POST",le="1.0"} 950
http_request_duration_seconds_bucket{service="order-service",method="POST",le="+Inf"} 1024
http_request_duration_seconds_sum{service="order-service",method="POST"} 240.5
http_request_duration_seconds_count{service="order-service",method="POST"} 1024
```

A metrikákhoz tartozó címkék (label) fontosak, de nagy kardinalitású címkéket (például `order_id`, `customer_email`) nem szabad használni, mert az idősorok száma robbanásszerűen megnő, és a metrikarendszer nem skálázódik. A gyakorlatban a címkék halmazát korlátozzuk alacsony kardinalitású értékekre: `service`, `method`, `status`, `environment`, `version`.

A metrikáknál a mérési ablak és a mértékegység mindig szerepeljen. A könyv konvenciója szerint a rendelkezésre állást az

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

képlettel számítjuk, az error budget pedig $1 - \text{SLO}$ az adott ablakban. A metrika-definíciókban is meg kell adni a mértékegységet (pl. másodperc, bájt, kérés/s) és a mérési ablakot (pl. 30 napos gördülő ablak).

Az SLI (szolgáltatási szint indikátor) gyakran egy metrikából származó arányszám, például a sikeres kérések aránya az összes kéréshez képest. Az SLO (szolgáltatási szint célérték) az SLI-re adott küszöb, például „a sikeres kérések aránya 99,9% felett legyen 30 napos ablakban”. A metrikarendszernek képesnek kell lennie az SLI kiszámítására és a riasztási szabályok kiértékelésére.

23.1.4 Elosztott trace

Az elosztott trace egy kérés útját rögzíti a rendszer komponensein keresztül. A trace hierarchikus spanekből áll:

- **Trace:** az egész kérés életciklusa, azonosítója a `trace_id`.
- **Span:** egyetlen munkaegység, azonosítója a `span_id`. A span tartalmazza a művelet nevét, kezdő- és záró időpontját, a szülő span azonosítóját, valamint attribútumokat (pl. HTTP metódus, státuszkód, adatbázis lekérdezési idő).
- **Szülő-gyermek kapcsolat:** a spanek hierarchiát alkotnak. A gyökér span az első belépési pont, a gyermek spanek a hívások által kiváltott további munkát jelölik.

A trace kontextus terjesztése elengedhetetlen. Amikor egy szolgáltatás meghív egy másikat, a hívás fejlécében át kell adni a `trace_id`-t és a `span_id`-t. A W3C Trace Context specifikációja (aktuális verzióját ellenőrizni kell) szabványosítja ezt a fejlécet: `traceparent` formátumban. Például:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

A fejléc három része: `trace_id`, `parent_id` (a hívó span azonosítója) és a flags mező a mintavételezés jelzésére. A hívott szolgáltatás létrehoz egy új span-t, amelynek szülője a megadott span azonosító, és a trace azonosító változatlan marad.

Az alábbi ábra egy egyszerű rendelési folyamat trace-jét mutatja:

```
sequenceDiagram
    participant Client
    participant Frontend
    participant OrderService
    participant PaymentService
    participant DB

    Client->>Frontend: POST /orders (traceparent)
    Frontend->>OrderService: POST /api/orders (traceparent)
    OrderService->>PaymentService: POST /payments (traceparent)
    PaymentService-->>OrderService: 200 OK
    OrderService->>DB: INSERT order
    DB-->>OrderService: OK
    OrderService-->>Frontend: 201 Created
    Frontend-->>Client: 201 Created
```

1. ábra – Elosztott trace folyamata rendelés létrehozásakor. A diagram csak a trace kontextus terjedését szemlélteti; a valós rendszer további komponenseket is tartalmazhat.

A trace egy-egy spanje tartalmazza a művelet nevét, a start és end időpontot, a kapcsolódó státuszt, valamint attribútumokat. Az OpenTelemetry szemantikai konvenciói szerint például a HTTP hívásoknál használjuk a `http.request.method`, `http.response.status_code`, `url.path`, `server.address` attribútumokat. Az adatbázis hívásoknál a `db.system`, `db.name`, `db.operation` mezőket. A szemantikai konvenciók célja, hogy a különböző rendszerekből származó telemetriai adatok egységesek legyenek, így a dashboardok és riasztások könnyebben megoszthatók.

A trace-ekből származtathatók metrikák is: például a span időtartamokból hisztogramot számolhatunk, vagy a span-státuszokból hibaarányt. Ez egyesíti a három jelet: a trace tárolja a részleteket, a metrika az aggregátumot, a napló pedig az esemény kontextusát. A jól megtervezett telemetriai rendszerben mindhárom jel ugyanabból a kontextusból táplálkozik.

23.1.5 OpenTelemetry kollektor és szemantikai konvenciók

Az OpenTelemetry (rövidítve OTel) egy CNCF (Cloud Native Computing Foundation) projekt, amely szabványos API-kat, SDK-kat, adatgyűjtőt és szemantikai konvenciókat biztosít a telemetriai adatok gyűjtésére. A célja, hogy a fejlesztők és üzemeltetők ne legyenek egyetlen szállítóhoz kötve, hanem a telemetriát egyszerűen továbbíthassák különböző háttérrendszerek felé (pl. Prometheus, Jaeger, grafikonos elemző platformok, felhőszolgáltatások).

Az OpenTelemetry fő komponensei:

- **API:** a programozói felület, amelyet a fejlesztők használnak a spanek létrehozására, metrikák rögzítésére, naplók küldésére.
- **SDK:** az API implementációja, amely a telemetriát feldolgozza és továbbítja.
- **Kollektor (Collector):** önálló szolgáltatás, amely fogadja a telemetriát, feldolgozza és továbbítja a háttérbe. A kollektor lehet agent (sidecar) vagy gateway (központi) módban.
- **Szemantikai konvenciók:** szabványos mezőnevek és attribútumnevek, amelyek biztosítják a konzisztenciát.

A kollektor pipeline-ja három részből áll:

- **Receivers (fogadók):** adatot fogadnak, például OTLP (OpenTelemetry Protocol) gRPC vagy HTTP formátumban.
- **Processors (feldolgozók):** átalakítják, szűrik, batch-elik a telemetriai adatokat.
- **Exporters (kimenetek):** a feldolgozott adatot továbbítják a célrendszer felé.

Egy példa kollektor konfiguráció YAML formátumban (a verziószámok és a mezők aktuálisak lehetnek, ellenőrizni kell):

```yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 5s
    send_batch_size: 1024
  memory_limiter:
    check_interval: 2s
    limit_mib: 512

exporters:
  otlp:
    endpoint: "otel-collector.internal.example:4317"
    tls:
      insecure: true

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp]
```

Megjegyzés: a fenti konfiguráció labor célú, az `insecure: true` *csak izolált környezetben használható. Éles rendszerben TLS-t kell használni, az endpoint pedig placeholderszerű.*

Az OpenTelemetry szemantikai konvenciói meghatározzák, hogy milyen attribútumneveket kell használni az egyes protokollokhoz, adatbázisokhoz, üzenetküldéshez. A konvenciók stabil és fejlődő részekre oszthatók; a stabil részeket a specifikáció „stable” jelöléssel látja el. Mindig az aktuális specifikációt ellenőrizni kell, mert a mezőnevek változhatnak. A konvenciók célja, hogy a telemetriai adatok ne legyenek gyártófüggők, és a kollégák, eszközök egyformán értelmezzék.

A gyakorlatban az OpenTelemetry SDK-k (Java, Python, Go, .NET stb.) automatikusan generálják a spaneket és metrikákat a beépített műszerezés (instrumentation) segítségével. A fejlesztőnek csak a kontextust kell továbbadnia, és az üzleti attribútumokat hozzáadnia. Például egy HTTP hívásnál az SDK automatikusan rögzíti a metódust, a státuszkódot, az időtartamot. A fejlesztőnek nem kell manuálisan minden mezőt kitöltenie, de a kontextusmezők, mint az `order_id`, továbbra is fontosak.

23.1.6 Gyakorlati minta: telemetriai terv

Egy telemetriai terv rögzíti, hogy mit mérünk, miért, és hogyan gyűjtjük össze. A tervnek az alábbi elemeket kell tartalmaznia:

1. **Célok:** miért van szükség telemetriára? (pl. hiba lokalizálás, kapacitástervezés, SLO ellenőrzés). A célokhoz minőségi attribútum forgatókönyvet rendelhetünk.
2. **Megfigyelt rendszer:** a komponensek és a köztük lévő kapcsolatok.
3. **Jelek és források:** milyen naplók, metrikák és trace-ek keletkeznek, és mely komponensből.
4. **Kontextusmezők:** a közös mezők listája, beleértve a `trace_id`, `span_id`, `service.name`, `environment`, `version`, `host.name`, üzleti azonosítók.
5. **Gyűjtő és feldolgozó pipeline:** OpenTelemetry kollektor konfigurációja: receivers, processors, exporters.
6. **Tárolás és megtartás:** hová kerülnek az adatok, milyen retenciós idővel.
7. **Mérési metódus:** az SLI-k kiszámításának módja, a mérési ablak, a bizonytalanság.
8. **Riasztási alapelvek:** nem a következő fejezet anyaga, de a tervben jelölhető, hogy mely metrikákból lesznek riasztások.
9. **Döntési napló (ADR):** a fontos döntések indoklása.

Példa egy egyszerű telemetriai tervre a rendeléskezelő rendszer esetén:

23.1.7 Célok és minőségi attribútum forgatókönyv

- **Cél:** a rendelésfeldolgozási folyamat válaszidejének és rendelkezésre állásának mérése, hibadetektálás.
- **Minőségi attribútum forgatókönyv:**
  - *Inger:* a felhasználó lead egy rendelést.
  - *Környezet:* termelés, normál terhelés (100 kérés/s).
  - *Válasz:* a rendelési kérés p95 válaszideje 30 napos ablakban.
  - *Mérőszám:* p95 $<$ 500 ms.
- **SLI:** sikeres rendelések aránya az összes rendelési kéréshez képest.
- **SLO:** 99,9% feletti sikeres rendelési arány 30 napos ablakban. Az error budget: 0,1%.

23.1.8 Jelek és források

Komponens	Napló	Metrika	Trace
Frontend	strukturált JSON, szint: INFO/ERROR	HTTP kérések száma, kliensoldali hibák	span-ek generálása a hívásokhoz
OrderService	strukturált JSON, üzleti események (OrderCreated)	request count, request duration hist, aktív kapcsolatok gauge	span-ek, attribútumokkal
PaymentService	strukturált JSON, fizetési események maskolva	request count, request duration hist	span-ek
Adatbázis	naplók a DB motorból, lassú lekérdezések	kapcsolatok száma, lekérdezési idő	span-ek a DB hívásokhoz

23.1.9 Kontextusmezők

Minden telemetriai forrásnál kötelező mezők:

```
service.name
environment (pl. production)
version (szolgáltatás verzió)
trace_id
span_id
host.name
```

Üzleti mezők (ahol releváns):

```
order_id
customer_id (álnevesített)
payment_id (álnevesített)
```

23.1.10 OpenTelemetry pipeline

A kollektor fogadja az OTLP-t, batch-eli, majd továbbítja a központi tárolóba. A konfiguráció a fenti YAML példa alapján készül, az `insecure: true` csak laborban.

23.1.11 Mérési módszer

- A p95 válaszidőt a `http_request_duration_seconds` histogramból számítjuk ki a 30 napos ablakban.
- Az SLI-t a `http_requests_total` counterből számítjuk: sikeres kérések (2xx) / összes kérés.
- A mérés bizonytalanságát a bucket határok és a mintavételezés határozza meg. A mérési ablakot rögzíteni kell.

23.1.12 Döntési napló (ADR rövidítve)

- **Döntés:** nem használunk summary-t, hanem histogramot, mert az aggregálható.
- **Döntés:** a naplókat JSON formátumban gyűjtjük, mert a kollektor így tudja feldolgozni.
- **Döntés:** a metrikák címkézésénél tiltjuk a nagy kardinalitású mezőket.

23.1.13 Labor: telemetriai terv készítése

Kontextus: egy kis webáruház-rendszer két szolgáltatással: „Frontend” és „OrderService”. A rendszer HTTP REST API-n kommunikál, az OrderService egy PostgreSQL adatbázist használ. A rendszer eddig nem rendelkezik strukturált telemetriával; a naplók hagyományos szövegesek, nincsenek metrikák és trace-ek.

Feladat: készíts telemetriai tervet a fenti rendszerhez. A tervet reprodukálható dokumentumban (például Markdown fájl vagy Jupyter-jegyzetfüzet) kell rögzíteni, amely verziózható. A terv tartalmazza:

1. A célokat és a minőségi attribútum forgatókönyvet.
2. A három jelhez tartozó források és kontextusmezők listáját.
3. Az OpenTelemetry kollektor pipeline konfigurációját (YAML).
4. A mérési módszert és a várt eredményt.

5. A döntési naplót (ADR), amely rögzíti a fontos döntéseket és az elvetett alternatívákat.
6. Az ellenőrzési szempontokat.

Elvárt eredmény: a dokumentumban a jelek és a kontextus összekapcsolása egyértelmű; minden telemetriai forrásnál szerepelnek a közös kontextusmezők; a metrikák definíciója mértékegységgel és mérési ablakkal rendelkezik; a kollektor pipeline legalább a `receivers`, `processors`, `exporters` szakaszokat tartalmazza.

Ellenőrzési szempontok:

- A napló példa strukturált, tartalmaz `trace_id`-t és `span_id`-t.
- A metrikák definíciója megadja a típust (counter, gauge, histogram) és a mértékegységet.
- A trace kontextus terjesztésére vonatkozó terv szerepel.
- A kollektor YAML-ben a `batch` feldolgozó be van állítva.
- A dokumentum megkülönbözteti a mért és a várt eredményt; a várt eredményt külön alfejezetben, a mérést és annak bizonytalanságát külön.
- Nincs valós személyes adat, jelszó vagy belső URL a példákban.

Visszaállítási út: a labor nem jár destruktív művelettel; csak dokumentum készül. Ha a dokumentumot valós környezetben próbáljuk ki, akkor izolált tesztkörnyezetet használjunk, és a kollektor konfigurációjában az `insecure: true` csak ideiglenesen szerepelhet.

23.1.14 Tipikus hibák és megoldásuk

Az alábbi hibák gyakran előfordulnak a telemetriai rendszer tervezésekor. Mindegyiknél megadjuk a helyes megoldást.

1. **Tagolatlan naplók:** a fejlesztők csak szövegként naplóznak, például `log.info("Rendelés létrejött, azonosító: " + orderId)`. Ebből a gép nem tudja kinyerni a `orderId`-t. **Helyes:** strukturált napló használata mezőkkel.

2. **Kontextus elvesztése aszinkron feldolgozásnál:** egy üzenetsoros fogyasztó nem viszi tovább a `trace_id`-t, mert a fejlesztő nem menti és állítja vissza a kontextust. Ezért a trace megszakad. **Helyes:** a trace kontextust a fogyasztóban is ki kell olvasni az üzenet fejlécéből, és a feldolgozás során be kell állítani.

3. **Nagy kardinalitású metrikacímkék:** például `order_id` vagy `customer_email` kerül a metrika labeljébe. Ez robbanó idősor-számhoz vezet. **Helyes:** csak alacsony kardinalitású címkéket használjunk; a részletes azonosítókat a naplóban és a trace-ben tároljuk.

4. **Személyes adatok telemetriában:** az üzleti azonosítók, e-mail címek vagy IP-címek naplózása adatvédelmi problémát okoz. **Helyes:** álnevesítés, maskolás vagy a mező elhagyása.

5. **A trace kontextus terjedésének hiánya:** a hívás fejlécéből hiányzik a `traceparent`. Ekkor a szolgáltatások között nem kapcsolódik össze a trace. **Helyes:** a W3C Trace Context implementálása, és az OpenTelemetry SDK beépített terjesztésének használata.

6. **A metrikák és a trace-ek összemosása:** a metrikák nem helyettesítik a trace-eket, és fordítva. A metrika az aggregátumot adja, a trace a részletet. **Helyes:** a három jelet párhuzamosan használjuk, a kontextus segítségével korrelálva.

7. **Az OpenTelemetry specifikáció verziójának figyelmen kívül hagyása:** a szemantikai konvenciók változhatnak, a mezőnevek frissülhetnek. **Helyes:** mindig az aktuális stabil specifikációt kövessük, és a verziót rögzítsük a dokumentációban.

23.1.15 Összefoglalás

Ebben a fejezetben áttekintettük az observability három alapjelét: a naplót, a metrikát és az elosztott trace-t. Megmutattuk, hogy a kontextus összekapcsolása a három jel alapja; a közös `trace_id`, `span_id`, `service.name` és üzleti mezők teszik lehetővé a hibák gyors lokalizálását. A strukturált napló a gépi feldolgozás miatt elengedhetetlen. A metrikák az aggregált állapotot mutatják, és az SLO-k alapjául szolgálnak. A trace a kérések útját követi, a spanek hierarchiája a hívásláncot ábrázolja. Az OpenTelemetry szabványos API-t, SDK-t, kollektort és szemantikai konvenciókat biztosít, így a telemetriai adatok szállítófüggetlenek lesznek. A fejezetben bemutatott telemetriai terv és labor segítségével a fogalmakat kis, ellenőrizhető példán alkalmazhatjuk.

Önellenőrző kérdés: Magyarázd el, hogyan kötnéd össze a három jelet egy elosztott kérés hibájának elemzésekor. Milyen kontextusmezők szükségesek, és mi történik, ha a kontextust nem terjesztik megfelelően?

Ellenőrzőlista:

- ☐ A strukturált naplók tartalmazzák a `timestamp`, `level`, `service.name`, `trace_id`, `span_id` mezőket.
- ☐ A metrikák definíciója típust, mértékegységet és mérési ablakot ad meg.
- ☐ A trace kontextus terjesztése meg van tervezve mind szinkron, mind aszinkron hívásokra.
- ☐ A metrikacímkék kardinalitása alacsony.
- ☐ Nincs személyes adat, jelszó vagy belső URL a telemetriai példákban.
- ☐ Az OpenTelemetry kollektor pipeline-ja tartalmaz `receivers`, `processors`, `exporters` szakaszokat.
- ☐ A telemetriai tervben a mért és a várt eredmény elkülönül.

A következő fejezet erre az alapra épít, és bemutatja, hogyan készítsünk dashboardokat, riasztásokat és runbookokat a begyűjtött telemetriai adatokra.

23.2 Dashboardok, riasztások és runbookok

A Logs, metrics, traces és OpenTelemetry egységben bemutatott telemetria-adatok – logok, metrikák és nyomkövetési adatok – önmagukban még nem oldanak meg üzemeltetési problémát. Az adat attól válik értékessé, ha valaki a megfelelő időben, a megfelelő formában látja, és ha a látott jelenséghez előre átgondolt beavatkozási útvonal tartozik. Ebben az egységben a megfigyelhetőségi lánc három üzemeltetési rétegét kapcsoljuk össze: a **dashboard** a működés láthatóvá tételének felülete, a **riasztás** az emberi figyelem irányításának mechanizmusa, a **runbook** pedig a beavatkozás előkészített, tesztelt forgatókönyve. A cél nem az, hogy több grafikont rajzoljunk, hanem hogy a rendszer üzemeltetője gyorsabban hozzon jobb döntéseket, és a döntésből származó beavatkozás ne az adott pillanat kreativitásán múljon.

A három elem egy visszacsatolási hurkot alkot. A telemetria folyamatosan érkezik, a dashboardon megjelenő mérőszámok és SLO-állapotok alapján a riasztási szabályok akkor szólalnak meg, amikor a rendszer viselkedése eltér a megengedett tartománytól. Az ügyeletes mérnök a riasztásból kiindulva a runbook lépéseit követve diagnosztizál, beavatkozik, majd a tanulságokat visszavezeti a dashboardok, riasztási küszöbök és runbookok fejlesztésébe. Ha valamelyik elem hiányzik vagy rosszul működik, a hurok megszakad: a szép dashboard senkinek sem szól, a riasztás zajjá válik, a runbook pedig elavult papír lesz.

23.2.1 A dashboard mint döntéstámogató felület

A dashboard elsődleges feladata nem az adatok kirakása, hanem a döntéstámogatás. Egy jól megtervezett dashboard minden paneljéhez hozzárendelhető egy kérdés, amelyre az üzemeltető választ vár. A „mi a CPU-terhelés?” még nem döntési kérdés; a „a CPU-telítettség miatt kell-e skáláznom, vagy a háttérfeladat ütemezését kell módosítanom?” már az. A tervezés során ezért előbb a döntéseket érdemes összegyűjteni, és csak utána a grafikonokat.

A panelek elrendezésénél az emberi figyelem korlátosságából kell kiindulni. A legfontosabb állapotok – a szolgáltatás SLO-állapota, a hibaköltségvetés megmaradt mértéke, az aktív riasztások – a képernyő tetejére kerülnek. Alább a diagnosztikai részletek, például az egyes szolgáltatások arany jelei, a hibaeloszlások és a függőségi gráfok helyezkednek el. A mélyebb elemzéshez használt paneleket nem szükséges állandóan láthatóvá tenni; azok a diagnosztikai dashboardsablonokba tartoznak, amelyeket riasztás esetén nyit meg az ember.

Gyakori hiba, hogy egy dashboard „tűfalként” működik: több tucat panel, minden kontextus nélkül, eltérő időablakokkal és mértékegységekkel. Ez a láthatóság illúzióját kelti, de döntési helyzetben nem segít. A panelek számát indokolt korlátozni, és minden panelnél rögzíteni a mérőszám nevét, a mértékegységet, az időablakot és az aggregáció módját. A diagram címe és feltételjegyzete a dokumentációs szerződés szerint is kötelező: a „Payment latency p95, 5 perces csúszóablak, percentilis: 0,95, forrás: OpenTelemetry histogram” felirat többet mond, mint a „Latency”.

A dashboardok rétegezése segít elkülöníteni a közönségeket. A szolgáltatási szintű dashboard a fejlesztő és üzemeltető csapatoké; a termék szintű dashboard az üzleti szereplőké; a rendszerszintű dashboard a platformcsapaté. Ami az egyik rétegben hasznos, a másikban zaj lehet. A termékmenedzsernek nem kell látnia az adatbázis-kapcsolatkészlet telítettségét, ahogyan a platformmérnöknek sem feltétlenül a napi konverziós arányt.

23.2.2 Arany jelek: a szolgáltatás négy létfontosságú mérőszáma

Amikor egyetlen szolgáltatás állapotát kell gyorsan megítélni, a Google SRE-gyakorlatából ismert négy arany jel ad jó kiindulópontot: **késleltetés**, **terhelés**, **hibaarány** és **telítettség**. A négy jel együtt elegendő ahhoz, hogy az üzemeltető eldöntse, van-e egyáltalán probléma, és ha van, nagyjából milyen típusú.

A **késleltetés** a kérés feldolgozásához szükséges idő. A legtöbb webes szolgáltatásnál a HTTP-kérés kezdetétől a válasz elküldéséig tartó időt mérjük, például milliszekundumban. A késleltetést soha nem szabad egyetlen átlagértékkel jellemezni, mert az eloszlás ferde: a felhasználók egy részének élményét néhány extrém lassú kérés is tönkreteheti, miközben az átlag szépen néz ki. Helyette percentiliseket használunk: a p50 a tipikus esetet, a p95 a „majdnem legrosszabb” élményt, a p99 a hosszú farkat mutatja. Az SLO-kban általában a p95 vagy p99 a meghatározó, mert a felhasználói élményt a leglassabb észlelhető kérések is befolyásolják.

A **terhelés** a rendszerre érkező munka mennyisége. Webes rendszerben ez általában a bejövő kérések másodpercenkénti száma, de lehet üzenetek másodpercenkénti száma, aktív kapcsolatok száma vagy feldolgozott rekordok mennyisége is. A terhelés önmagában nem jó vagy rossz; a kapacitástervezéshez és a telítettséggel való összevetéshez van rá szükség. A terhelés hirtelen megugrása előre jelezheti a telítettséget, ezért a terhelési görbét mindig a telítettségi mutatókkal együtt érdemes nézni.

A **hibaarány** a hibás kérések aránya az összes kéréshez képest. A hibát két szinten kell értelmezni. Az explicit hibák a protokollszinten jelzett hibák, például az 5xx HTTP-állapotkódok. Az implicit hibák olyan válaszok, amelyek formailag sikeresek, de a felhasználói elvárásnak nem felelnek

meg: a 200-as válasz, amely helyett hibajelzést tartalmaz, vagy a kérés, amely a megengedett késleltetési SLO-n túl érkezik meg. Az SLO szempontjából gyakran mindkét hibafajta beleszámít a hibaköltségvetésbe. A hibaarányt százalékban, meghatározott mérési ablakra vetítve adjuk meg, például „az elmúlt 5 percben a fizetési kérések 0,8%-a hibásult meg”.

A **telítettség** azt mutatja, hogy a rendszer erőforrásai mennyire közelítenek a kapacitáskorláthoz. A klasszikus mutatók a CPU-kihasználtság, a memóriahasználat, a lemez I/O, a hálózati sávszélesség vagy az adatbázis-kapcsolatkészlet kihasználtsága. A telítettség azonban nem mindig erőforrásmetrika: a feldolgozásra váró üzenetsor hossza, a thread pool várakozási ideje vagy a backpressure miatt eldobott kérések aránya gyakran jobban jelzi, hogy a rendszer a kapacitás szélén jár. A telítettség mérésénél meg kell adni a mértékegységet és az időablakot: például „az adatbázis-kapcsolatkészlet kihasználtsága 87% volt az elmúlt 10 perc átlagában”.

Az arany jelek összeállítása nem öncélú. Egy szolgáltatás dashboardján a négy jel együtt ad választ a következő kérdésekre: nő-e a terhelés? Romlik-e a késleltetés? Nő-e a hibaarány? Van-e szabad kapacitás? Ha a hibaarány nő, de a telítettség alacsony, a probléma valószínűleg logikai hiba vagy függőségi hiba. Ha a késleltetés nő, de a hibaarány változatlan, a rendszer még elnyeli a terhelést, de a telítettségi görbét figyelni kell. A négy jel közötti összefüggések teszik lehetővé a gyors, durva diagnózist a részletes nyomkövetési adatok előtt.

23.2.3 SLO és hibaköltségvetés a dashboardon

A dashboard nem csupán pillanatképet ad, hanem a szolgáltatás megbízhatósági szerződésének állapotát is mutatja. Az **SLI** (service level indicator) a mért arány: például a sikeres kérések aránya a teljes kérésszámhoz képest. Az **SLO** (service level objective) a szervezet által vállalt célérték: például a fizetési szolgáltatás havi sikeres kérésaránya legalább 99,9%. Az **error budget** az a hibamennyiség, amelyet a szolgáltatás az adott ablakban megengedhet magának anélkül, hogy megszegné az SLO-t:

$$\text{error budget} = 1 - \text{SLO}.$$

Ha az SLO 99,9% havi ablakban, akkor a hibaköltségvetés 0,1%, azaz a szolgáltatás havonta a kérések 0,1%-át hibásíthatja meg. A dashboardnak ezt a költségvetést is láthatóvá kell tennie, nemcsak a százalékos megfelelést. A „maradt 32 perc a havi 43 percből” típusú megjelenítés sokkal könnyebben értelmezhető, mint a „jelenlegi megfelelés 99,93%”.

A hibaköltségvetés fogyásának sebességét a **burn rate** fejezi ki. A burn rate azt mutatja, hogy az adott időablakban a rendszer hányszor gyorsabban égeti el a hibaköltségvetést, mint amennyire az az SLO-ablak hossza alapján lineárisan indokolt lenne. Ha a havi SLO 99,9% és a havi hibaköltségvetés 43 perc, akkor egy olyan időszak, amelyben 5 perc alatt 4,3 perc hibaidő gyűlik össze, 1-szeres burn rate-et jelent; ha ugyanannyi idő alatt 43 perc hibaidő keletkezik, a burn rate 10. A burn rate képlete:

$$\text{burn rate} = \frac{\text{tényleges hibaidő az ablakban}}{\text{időablak hossza}} \cdot \frac{\text{SLO-ablak hossza}}{\text{hibaköltségvetés}}.$$

A burn rate azért értékes, mert lehetővé teszi a gyors és a lassú hibák megkülönböztetését. Egy rövid, 5 perces ablakban mért magas burn rate gyorsan értesíti az ügyeletest, míg egy hosszabb, 1 órás ablakban mért alacsonyabb burn rate a lassan fogyó költségvetést jelzi. A két ablak együttes alkalmazása csökkenti a téves riasztások számát: a rövid ablak egymagában túl érzékeny, a hosszú ablak egymagában túl lassú.

23.2.4 Riasztások: a zaj csökkentése és a tünet elve

A riasztás célja, hogy emberi figyelmet irányítson egy olyan állapotra, amelyhez előre meghatározott beavatkozás tartozik. A riasztás nem arra való, hogy minden rendellenességet jelezzen; a folyamatos, de hatástalan riasztások **alert fatigue**-hez vezetnek: az emberek megtanulják figyelmen kívül hagyni a rendszert, és a valódi incidens is elsikkad. A zaj csökkentése ezért nem udvariassági kérdés, hanem a riasztási rendszer megbízhatóságának feltétele.

A legfontosabb elv a **tünetre riasztás, nem okra**. A tünet az, amit a felhasználó vagy az SLO szempontjából mérünk: a fizetési szolgáltatás hibaaránya 5% fölé emelkedett. Az ok ezzel szemben az a belső állapot, amely a tünetet előidézheti: a Java heap 95%-on van, a CPU 100%-on jár, az adatbázis-kapcsolatok száma magas. Az ok-alapú riasztás azért problémás, mert egy ok nem mindig okoz tünetet, és egy tünet több okból is fakadhat. A CPU 95%-os kihasználtsága lehet teljesen egészséges, ha egy batch-feladat éppen dolgozik, és a felhasználói kéréseket nem lassítja. Ha ilyenkor lapozzuk az ügyeletest, akkor olyan eseményre ébresztjük, amely nem igényel beavatkozást.

A riasztásokat érdemes **súlyossági szintekbe** sorolni. A SEV1 az azonnali, teljes kiesést vagy súlyos adatvesztést jelentő esemény; a SEV2 a jelentős, felhasználókat érintő, de nem teljes kiesést okozó állapot; a SEV3 a kisebb, nem sürgős romlás, amely munkaidőben kezelhető; a SEV4 és SEV5 a kozmetikai vagy információs jelzések. A súlyosság azt is meghatározza, hogy az értesítés **page** (azonnali, hangos, ébresztő) vagy **ticket** (munkaidős jegy, értesítés nélkül). A page és ticket közötti különbségtétel a zajcsökkentés egyik leghatékonyabb eszköze.

A riasztási szabályokat verziózni kell, és minden riasztáshoz tartoznia kell egy elérhető runbook-hivatkozásnak. A riasztási szabályok tesztelése ugyanolyan fontos, mint a kód tesztelése: szintetikus terheléssel, hibainjektálással vagy staging környezetben ellenőrizni kell, hogy a szabály a megfelelő helyzetben szólal meg, és nem szólal meg a nem kívánt esetben. A riasztási szabályok életciklusa – pending, firing, resolved – a dashboardon is megjelenik, így az ügyeletes látja, mi történt az elmúlt órában.

23.2.5 Riasztási szabályok tervezése

Egy riasztási szabály általában a következő elemekből áll: a figyelt mérőszám vagy arány, a küszöb, az időtartam, amelyen keresztül a feltételnek fenn kell állnia, az értékelés gyakorisága, a súlyosság, a címzett rotáció és a runbook-hivatkozás. A küszöböt az SLO-ból kell levezetni, nem önkényesen megválasztani.

Példa egy fizetési szolgáltatás hibaarány-riasztására Prometheus-szerű pszeudokódban:

```
alert: PaymentHighErrorRate
expr: |
  sum(rate(payment_http_requests_total{status=~"5.."}[5m]))
  /
  sum(rate(payment_http_requests_total[5m]))
  > 0.05
for: 15m
labels:
  severity: SEV2
annotations:
  summary: "Payment service error rate above 5%"
  description: "A fizetési kérések több mint 5%-a hibás az elmúlt 15 percben."
  runbook_url: "https://wiki.example.local/runbooks/payment-error-rate"
```

Ez a szabály tünetre riaszt: a felhasználói kérések hibája a mért jelenség. A `for: 15m` azt jelenti, hogy a feltételnek folyamatosan 15 percig fenn kell állnia, mielőtt a riasztás megszólal. Ez kiszűri az egyszeri kilengéseket, de lassítja az észlelést. Az időtartam megválasztása kompromisszum: a rövid `for` gyorsabb, de zajosabb; a hosszú `for` lassabb, de megbízhatóbb. Az SLO-ból levezetett burn rate riasztásokat több ablakkal érdemes kombinálni: például 5 perc alatt 10-szeres burn rate, vagy 1 óra alatt 3-szoros burn rate. Így a gyors, súlyos hibák azonnal lapot kapnak, a lassú, enyhe romlás pedig ticketet.

A riasztási szabályok tipikus hibái közé tartozik az átlagok használata a percentilisek helyett, a hiányzó idősorok figyelmen kívül hagyása, a küszöbök állandó módosítgatása „mert most mégis szólt”, és az, hogy egy riasztáshoz nincs runbook. A hiányzó idősor különösen alattomos: ha egy szolgáltatás nem küld metrikát, a riasztási feltétel lehet, hogy soha nem értékelődik ki, vagy éppen ellenkezőleg, mindig igaz lesz. A szabályoknál gondoskodni kell a hiányzó adat kezeléséről is, például `absent()` ellenőrzéssel.

23.2.6 Runbook: a beavatkozás előkészítése

A runbook egy adott riasztáshoz tartozó, lépésről lépésre követhető eljárás. A célja az, hogy az ügyeletes mérnöknek ne kelljen az adott pillanatban feltalálnia a diagnózist és a helyreállítást. A jó runbook csökkenti a hibás beavatkozások esélyét, gyorsítja az incidenskezelést, és lehetővé teszi, hogy a kevésbé tapasztalt mérnök is magabiztosan lépjen.

A runbook szerkezete a következő elemekből állhat:

1. **Metadata:** a riasztás neve, súlyossága, a szolgáltatás felelős csapata, a runbook utolsó frissítésének dátuma és verziója.
2. **Tünet:** mi látszik a dashboardon, milyen riasztás szólalt meg, milyen telemetria utal a problémára.
3. **Első lépések:** az incidens megnyitása, a súlyosság megerősítése, a biztonsági korlátok ellenőrzése.
4. **Diagnosztikai lépések:** az arany jelek ellenőrzése, a hibaarány és a késleltetés szétválasztása, a naplók és nyomkövetési azonosítók vizsgálata, a függőségek állapotának ellenőrzése.
5. **Gyakori okok és megoldások:** táblázat, amely a diagnosztikai lépések eredményéből kiindulva a beavatkozást adja meg.
6. **Eszkaláció:** kit és mikor kell értesíteni, ha az elsődleges beavatkozás nem vezet eredményre.
7. **Helyreállítás:** a visszaállítás, a forgalomátirányítás, a feature flag kikapcsolása vagy a skálázás konkrét parancsai.
8. **Utómunka:** a postmortem hivatkozása, a tanulságok rögzítése.

A runbook nem lehet regény. A lépések legyenek rövidek, parancsszerűek, és minden parancsnál egyértelmű legyen, hogy mit várunk tőle. A kód- és konfigurációpéldák szemléltetők; a destruktív műveleteket – például adatbázis-törlés, klaszter-átkonfigurálás – csak izolált környezetben szabad bemutatni. A runbookban éles rendszerre vonatkozó parancsok helyett az eljárás leírását adjuk meg, és a tényleges parancsokat a csapat belső, verziózott dokumentációjában kell karbantartani.

Példa egy runbook döntési fájára Mermaid diagramként:

```
flowchart TD
    A[PaymentHighErrorRate riasztás] --> B{Hibaarány > SLO?}
    B -- Igen --> C{A hiba a fizetési szolgáltatásban?}
    B -- Nem --> D[Hamis riasztás: silencing felülvizsgálat]
    C -- Igen --> E{Friss release?}
```

```
C -- Nem --> F[Függőség vagy infrastruktúra ellenőrzés]
E -- Igen --> G[Rollback vagy feature flag off]
E -- Nem --> H[Trace és log mélyelemzés]
H --> I[Gyakori okok táblázata]
F --> I
G --> J[Ellenőrzés: hibaarány visszaállt?]
I --> J
J -- Nem --> K[Eszkaláció a fizetési csapatnak]
J -- Igen --> L[Postmortem és runbook frissítés]
```

A runbook nem helyettesíti a mérnöki ítélőképességet, de keretet ad neki. A runbookot rendszeresen tesztelni kell, például játéknapokon vagy szimulált incidensekben. Ha egy runbook lépései nem működnek, azt a postmortemben rögzíteni kell, és a runbookot frissíteni kell. A runbook verziója a dokumentációs szerződés szerint visszakereshető, és a riasztási szabályban lévő hivatkozás mindig az aktuális verzióra mutat.

23.2.7 Eszkaláció és on-call

A riasztás csak akkor ér valamit, ha van, aki fogadja. Az **on-call** rotáció az a szervezeti mechanizmus, amely meghatározza, ki felelős a riasztások fogadásáért egy adott időablakban. A rotáció lehet elsődleges és másodlagos szintű: az elsődleges fogadja a riasztást, a másodlagos akkor lép be, ha az elsődleges nem reagál. Az eszkalációs lánc további szinteket tartalmazhat: szakterületi felelős, csapatvezető, ügyeletes vezető.

Az eszkalációs időzítések explicit meghatározása kritikus. Például: ha az elsődleges 15 percen belül nem nyugtázza a riasztást, a rendszer értesíti a másodlagost; ha a másodlagos 30 percen belül sem reagál, az eszkaláció továbbmegy. A nyugtázás és a megoldás közötti időt is mérni kell: az **MTTA** (mean time to acknowledge) az észleléstől a nyugtázásig, az **MTTR** (mean time to resolve) az észleléstől a megoldásig tartó idő. Mindkettőt mértékegységgel és mérési ablakkal kell rögzíteni, például „az elmúlt 30 napban a SEV2 riasztások medián MTTA-ja 4,2 perc volt”.

Az on-call rotáció tervezésénél figyelembe kell venni az emberi tényezőket. Az állandó éjszakai ügyelet kiégéshez vezet, a rotáció nélküli egyetlen felelős pedig egypontos meghibásodás az emberi oldalon. A rotációban legyen legalább két ember, a riasztások száma legyen nyomon követhető, és a zajos riasztások csökkentése mindenki közös felelőssége. Az ügyeleti etikett alapelve: a riasztás nem a hibás ember keresésére való. A cél a rendszer helyreállítása, nem a bűnbak megtalálása.

Az on-call nem csak a telefonos ügyeletet jelenti. Ugyanilyen fontos a napközbeni rotáció: amikor a csapat aktívan fejleszt, a riasztásokra való reagálás akkor sem maradhat a „majd valaki megnézi” szintjén. A rotációt a csapat rendszeresen felülvizsgálja, és a riasztások gyakoriságát, a hamis pozitív arányt, a nyugtázási időket és a runbookok hatékonyságát együtt elemzi.

23.2.8 Labor: dashboard- és riasztásterv készítése

A labor célja, hogy a fejezetben tárgyalt elveket egy kitalált, de valósághű rendszeren alkalmazd. A feladat egy dashboard- és riasztásterv elkészítése reprodukálható dokumentumban. A labor nem igényel éles rendszerhez való hozzáférést; a példák és a mérőszámok szemléltetők.

23.2.9 Kiinduló állapot

Adott egy „Bookshop” nevű kitalált online könyváruház, amely a következő szolgáltatásokból áll: webes felület, katalógusszolgáltatás, kosárszolgáltatás, fizetési szolgáltatás és rendeléskezelő szolgáltatás. A rendszer OpenTelemetry-alapú telemetriát küld: metrikákat Prometheus-kompatibilis formátumban, logokat és nyomkövetési adatokat. A fizetési szolgáltatás SLO-ja: havi szinten a sikeres fizetési kérések aránya legalább 99,9%.

23.2.10 Elvárt eredmény

Készíts egy dokumentumot, amely a következőket tartalmazza:

1. **Dashboard-terv:** a fizetési szolgáltatáshoz legalább 6 panelt, mindegyikhez név, mért mérőszám, mértékegység, időablak, aggregáció és döntési kérdés. Az arany jelek mindegyike jelenjen meg.
2. **Riasztási terv:** legalább 3 riasztási szabályt, mindegyikhez tünet, feltétel, időtartam, súlyosság, a riasztási csatorna (page vagy ticket), és runbook-hivatkozás.
3. **Runbook-vázlat:** a legfontosabb riasztáshoz – például a fizetési szolgáltatás magas hibaaránya – egy rövid runbook a fenti szerkezet szerint.
4. **Döntési napló:** ADR-szerű bejegyzés a riasztási küszöbök indoklásáról, beleértve a figyelembe vett alternatívákat és a várható zajszintet.
5. **Ellenőrzési szempontok:** a dokumentum végén listázd, hogyan ellenőrizted a saját tervedet (például: minden riasztás tünetre vonatkozik, minden panelhez tartozik döntés, a küszöbök az SLO-ból levezethetők).

23.2.11 Példa a dokumentum egy részletére

Az alábbiak nem teljes megoldások, hanem a formátumot mutatják.

Dashboard-panelek (fizetési szolgáltatás):

Panel neve	Mérőszám	Mértékegység	Időablak	Döntési kérdés
Payment request rate	`rate(payment_http_requests_total[5m])`	req/sec	5 perc	Nőtt-e a terhelés?
Payment latency p95	`histogram_quantile(0.95, rate(payment_http_request_duration_seconds_bucket[5m]))`	s	5 perc	Romlik-e a kiszolgálás?
Payment error ratio	`sum(rate(payment_http_requests_total{status=~"5.."}[5m])) / sum(rate(payment_http_requests_total[5m]))`	%	5 perc	Nő-e a hibaarány?
DB pool saturation	`db_connection_pool_utilization`	%	10 perc	Telített-e az adatbázis-kapcsolatkészlet?
Error budget remaining	`(SLO_window_seconds - error_seconds) / SLO_window_seconds`	%	30 nap	Mennyi hibaköltségvetés maradt?
Burn rate	`burn_rate(payment_error_budget_consumption[5m])`	–	5 perc	Milyen gyorsan fogy a költségvetés?

Riasztási szabályok:

1. **PaymentHighErrorRate:** ha a fizetési kérések hibaaránya 5% felett van 15 percig, SEV2, page, runbook: `payment-error-rate`.
2. **PaymentLatencyP95High:** ha a p95 késleltetés 800 ms felett van 15 percig, SEV3, ticket, runbook: `payment-latency-high`.
3. **PaymentDBPoolSaturation:** ha az adatbázis-kapcsolatkészlet kihasználtsága 90% felett van 10 percig, SEV3, ticket, runbook: `payment-db-pool-high`.

Runbook-vázlat (PaymentHighErrorRate):

- **Tünet:** a fizetési hibaarány 5% fölé emelkedett, a dashboardon a hibaköltségvetés fogyása gyorsul.
- **Első lépések:** incidens megnyitása, a súlyosság megerősítése, a fizetési szolgáltatás és a függőségek állapotának ellenőrzése.
- **Diagnosztikai lépések:** a hibaarány szétbontása HTTP-állapotkódok szerint; annak ellenőrzése, hogy a hiba a fizetési szolgáltatásból vagy a külső fizetési szolgáltatótól érkezik; a késleltetés percentiliseinek összevetése a hibaaránnyal; a nyomkövetési azonosítók alapján a hibás kérések láncának vizsgálata.
- **Gyakori okok:** friss release hibája, külső fizetési szolgáltató kiesése, adatbázis-kapcsolatkészlet kimerülése, helytelen konfiguráció.
- **Eszkaláció:** ha 20 perc alatt nincs diagnózis, a fizetési csapat szakterületi felelősének értesítése.
- **Helyreállítás:** ha friss release okozza, rollback vagy feature flag kikapcsolás; ha külső szolgáltató, forgalomátirányítás vagy a hibaüzenet kezelése.
- **Utómunka:** postmortem link, a runbook frissítése a tanulságokkal.

23.2.12 Ellenőrzési szempontok a laborhoz

- Minden dashboard-panelhez tartozik döntési kérdés?
- Minden riasztás tünetre vonatkozik, nem okra?
- Minden riasztás rendelkezik runbook-hivatkozással?
- A küszöbök konzisztensek az SLO-val és a hibaköltségvetéssel?
- A dokumentum megkülönbözteti a mért és a várt eredményeket?
- A dokumentum verziózott, és a döntési napló visszakereshető?

A labor során nem szükséges éles rendszeren végrehajtani semmilyen műveletet. Ha dashboardot vagy riasztási szabályt próbálsz ki, azt izolált környezetben, például helyi konténerben vagy staging fiókban tedd.

23.2.13 Döntési gyakorlat

Egy csapat a következő riasztást kapta: „A fizetési szolgáltatás CPU-kihasználtsága 90% felett van 5 perce.” A csapat ügyeletese megnézi a dashboardot: a fizetési kérések hibaaránya 0,1%, a p95 késleltetés 210 ms, a terhelés normális. A runbook szerint a CPU-riasztás SEV3-as ticket. A mérnök megjegyzi, hogy a CPU-riasztás feleslegesen szólalt meg.

Feladat: Írd le, miért lehet a CPU-riasztás zajos, és hogyan alakítanád át tünet-alapú riasztássá. Milyen telítettségi mutatót használnál helyette, és milyen küszöböt állítanál be az SLO-val összhangban? A válaszodban indokold meg a döntést, és jelöld meg, milyen alternatívát vetnél el és miért.

23.2.14 Ellenőrzőlista

- A dashboardok panelei döntési kérdésekhez kötöttek, nem dekorációk.
- A szolgáltatások arany jelei – késleltetés, terhelés, hibaarány, telítettség – külön-külön, mértékegységgel és időablakkal jelennek meg.
- A késleltetést percentilisekkel, nem átlaggal ábrázoljuk.
- Az SLO és a hibaköltségvetés állapota, valamint a burn rate látható a dashboardon.
- A riasztások tünetre vonatkoznak, nem okra.
- Minden riasztáshoz tartozik runbook-hivatkozás, súlyosság és a page/ticket besorolás.
- A riasztási küszöbök az SLO-ból levezethetők, és több ablakot használnak a zaj csökkentésére.
- A runbookok verziózottak, teszteltek és naprakészek.
- Az on-call rotáció és az eszkalációs lánc időzítései explicit módon rögzítettek.
- A riasztások számát, a hamis pozitív arányt és az MTTA/MTTR értékeket rendszeresen mérik.

23.2.15 Önellenőrző kérdés

Miért előnyösebb a késleltetés p95-ös percentilisét figyelni az átlagos késleltetés helyett egy olyan szolgáltatás esetében, amelynek SLO-ja a felhasználói élményre vonatkozik? Válaszodban térj ki arra, hogyan befolyásolja ez a dashboardon megjelenő késleltetési panel kialakítását.

23.2.16 Összefoglalás

A dashboard, a riasztás és a runbook hármasának célja, hogy a telemetria ne csak adat legyen, hanem cselekvéshez vezessen. A dashboard a döntéstámogatás felülete, amely az arany jelek, az SLO és a hibaköltségvetés állapotát teszi láthatóvá. A riasztás a figyelem irányításának eszköze, amely csak akkor hatékony, ha tünetre vonatkozik, zajszintje alacsony, és minden riasztáshoz tartozik runbook. A runbook a beavatkozást készíti elő, és az on-call rotációval együtt biztosítja, hogy a rendszer üzemeltetése ne az egyes mérnökök pillanatnyi tudásán múljon. A következő egységben az incidenskezelés és a postmortem folyamatára építünk, ahol a runbookok és riasztások tanulságait szervezeti szinten dolgozzuk fel.

23.3 Incident response és postmortem

Az incidenskezelés és a postmortem az üzemeltetés két, kultúrával átszőtt gyakorlata. Míg az observability – a korábbi fejezetben tárgyalt logok, metrikák és trace-ek – lehetővé teszi, hogy észrevegyük a rendellenességet, az incidenskezelés ad keretet annak, hogy a detektálástól a tanulásig eljussunk anélkül, hogy közben a rendszer, a csapat vagy a bizalom tartósan sérüljön. Ebben az alfejezetben az incidenskezelés életciklusát, a szerepeket, a kommunikációt, az idővonal-vezetést, a blameless kultúrát és a postmortem mint tanulási eszközt járjuk körül. A cél nem egy merev eljárásrend bemagolása, hanem egy olyan gondolkodásmód kialakítása, amely nyomás alatt is a mérhető, biztonságos és visszakereshető cselekvést részesíti előnyben.

23.3.1 Mi számít incidensnek, és miért kell külön kezelni?

Incidens alatt olyan eseményt értünk, amely a szolgáltatás minőségét a felhasználók számára érzékelhetően rontja, vagy a rendszer biztonságát, adatainak integritását veszélyezteti. Nem minden riasztás incidens; a zajos riasztás éppúgy probléma, mint az elmaradt észlelés. Az incidenskezelés fókusza az, hogy amikor már valódi hatás van – vagy annak közvetlen kockázata –, a csapat koordináltan, előre meghatározott szerepekkel és kommunikációs csatornákkal dolgozzon, és a lehető leggyorsabban visszaállítsa a szolgáltatást.

Az incidensek súlyosságát érdemes előre definiálni. Egy lehetséges, háromszintű besorolás:

- **SEV1** – kritikus: a fő felhasználói út nem elérhető, adatvesztés vagy biztonsági kompromittálódás gyanúja áll fenn. Azonnali beavatkozást és folyamatos kommunikációt igényel.
- **SEV2** – magas: jelentős funkcióromlás, például a fizetés lassú, de van kerülőút; a csapat azonnal foglalkozik vele, de nincs szükség teljes leállásra.
- **SEV3** – közepes vagy alacsony: kisebb, elszigetelt hiba, amely normál munkarendben javítható.

A súlyosság nem csupán címke; meghatározza a válaszadási időt, a bevont szerepköröket és a kommunikáció gyakoriságát. Például SEV1 esetén az ügyeletes 5 percen belül nyugtázza a riasztást, és 15 percen belül megnyitja a dedikált incidenscsatornát; SEV3 esetén elegendő a következő munkanapi triage. Ezeket a küszöböket érdemes quality attribute scenario formában rögzíteni:

> **Incidens-válaszidő forgatókönyv (SEV1)**
> **Inger:** a monitoring rendszer riasztást küld a fizetési szolgáltatás 99%-ot meghaladó hibaarányáról.
> **Környezet:** normál üzem, hétköznap 14:00, az ügyeletes mérnök elérhető.
> **Válasz:** az ügyeletes 5 percen belül nyugtázza a riasztást, 15 percen belül létrehozza az incidenscsatornát és értesíti az incident commandert.
> **Mérőszám:** time-to-acknowledge (TTA) 5 perc, time-to-communication (TTC) 15 perc, 90 napos gördülő ablakban mérve, p95.

A mérési ablak és a percentilis fontos: az átlag elrejtheti a ritka, de hosszú késéseket.

23.3.2 Az incidenskezelés életciklusa: a detektálástól a tanulásig

Az incidenskezelést legegyszerűbben hat fázisra bonthatjuk, bár a gyakorlatban ezek átfedhetnek, és egy-egy fázis többször is ismétlődhet.

```
flowchart LR
    A[Detektálás] --> B[Triage]
    B --> C[Diagnózis]
    C --> D[Mitigáció]
    D --> E[Helyreállítás]
    E --> F[Tanulás]
    F -.->|visszacsatolás| A
```

1. Detektálás. Az incidenskezelés itt nem az emberrel, hanem a jelzéssel kezdődik. A jelzés érkezhet riasztásból (metrika, log, trace alapján), ügyfélszolgálati bejelentésből, vagy akár a fejlesztő saját megfigyeléséből. A detektálás minőségét két hibamód rontja: a fals pozitív (nincs valódi hatás, de riaszt) és a fals negatív (van hatás, de nincs riasztás). Mindkettőt a monitoring szabályok, a küszöbök és a riasztási útvonalak rendszeres felülvizsgálatával lehet mérsékelni. A detektáláshoz tartozó mérőszám a time-to-detect (TTD): mennyi idő telik el a hiba első

jelentkezésétől az első értesítésig. Ezt csak megbízható SLI-vel lehet mérni; ha a hibaarányt 1 perces ablakban számoljuk, a TTD nem lehet 1 percnél pontosabb.

2. Triage. A triage során az ügyeletes vagy az elsőként reagáló mérnök eldönti: valódi incidensről van-e szó, milyen súlyosságú, és kiket kell bevonni. Itt fontos a gyors, de kontrollált döntés. Gyakori hiba, hogy a triage közben az ember azonnal a kiváltó okot kezdi keresni, miközben a hatás még nincs pontosan feltárva. Előbb a hatást és a felhasználói élményt kell megérteni, csak utána a technikai okot. A triage kimenete egy rögzített döntés: például „a checkout szolgáltatás 42%-os HTTP 500 hibaarányt mutat, a vásárlók nem tudnak fizetni, SEV1, bevonjuk a payment team ügyeletesét".

3. Diagnózis. Itt kezdődik a technikai nyomozás. A csapat a megfigyelhetőségi adatok (trace-ek, logok, metrikák) alapján keresi a hibát. A diagnózis során fontos a hipotézisvezérelt munka: minden lépéshez tartozzon egy feltételezés, egy ellenőrző lépés és egy mért eredmény. Például: „Feltételezzük, hogy az adatbázis-kapcsolatok kimerültek; ellenőrizzük a connection pool metrikát; az eredmény azt mutatja, hogy a pool 100%-on áll." Ez a fegyelem csökkenti a kapkodást és a párhuzamos, egymásnak ellentmondó beavatkozásokat.

4. Mitigáció. A mitigáció célja a felhasználói hatás megszüntetése vagy mérséklése, nem feltétlenül a végleges javítás. Gyakran a leggyorsabb beavatkozás a forgalom átterelése, a szolgáltatás skálázása, a cache ürítése vagy egy újonnan bevezetett változtatás visszavonása. A mitigációnál a döntést és a várható hatást rögzíteni kell, mert a visszavonás vagy a további lépések csak így lesznek biztonságosak. Például: „Rollback a payment-service 3.14.1 verziójáról a 3.14.0-ra; várhatóan 2 perc alatt helyreáll a hibaarány."

5. Helyreállítás. Amikor a hatás megszűnt, a rendszer visszatér a normál üzembe. Ez nem feltétlenül jelenti a teljes kiváltó ok megszüntetését; lehet, hogy a hiba még lappang, de a szolgáltatás elérhető. A helyreállítást megerősíteni kell a felhasználói metrikák és az üzleti tranzakciók ellenőrzésével. A helyreállítás után a csapat csak akkor zárja le az incidenst, ha a megfigyelhetőség megerősíti a stabil állapotot, és a kommunikációs csatornán ezt közölték.

6. Tanulás. A tanulás a postmortem formális vagy könnyűszerű folyamata. A cél nem a hibás ember megtalálása, hanem a rendszer és a folyamatok sebezhetőségének feltárása. A tanulás eredménye konkrét, tulajdonossal és határidővel ellátott akció, valamint – ha szükséges – az SLO, a monitoring, a runbook vagy az architektúra finomítása.

Ezt a hat fázist nem szabad merev sorrendként kezelni. Egy SEV1 incidensben a mitigáció gyakran a diagnózis előtt történik, mert a szolgáltatás visszaállítása fontosabb, mint a hiba gyökerének megértése. A gyakorlatban a kérdés nem az, hogy „ki a hibás", hanem hogy „mit tudunk most tenni a hatás csökkentésére, és mit fogunk utána megtanulni".

23.3.3 Szerepek, kommunikáció és idővonal

Az incidenskezelés egyik legfontosabb tanulsága, hogy nyomás alatt az emberek hajlamosak egymással párhuzamosan, koordinálatlanul dolgozni, és a kommunikáció háttérbe szorul. Ezt úgy lehet ellensúlyozni, ha explicit szerepeket osztunk ki. Az alábbi szerepek a Google SRE és más iparági gyakorlatok alapján terjedtek el; kis csapatban egy ember több szerepet is betölthet, de a feladatokat akkor is el kell különíteni.

Incident Commander (IC). Ő vezeti az incidenskezelést, döntéseket hoz, prioritást állít fel, és biztosítja, hogy a csapat ne ragadjon le a diagnózisban. Az IC nem feltétlenül a legjobb technikai szakértő; sokkal fontosabb, hogy képes legyen a teljes képet látni, és a kommunikációt irányítani. Az IC döntéseit a timebox és a hatás vezérli: ha a diagnózis 30 perc után sem vezet eredményre, mitigációs lépést rendel el.

Operations Lead (OL). Az OL a technikai beavatkozást koordinálja. Ő az, aki a diagnózist és a mitigációt szervezi, a szakértői munkát irányítja, és minden lépést validál. Az OL és az IC közötti határvonal gyakran elmosódik, de a szétválasztás segít abban, hogy az operatív munka ne sodorja el a vezetői döntéshozatalt.

Communications Lead (CL). Ő a külső és belső kommunikáció felelőse: státuszfrissítések a stakeholdereknek, ügyfélszolgálat tájékoztatása, status page frissítése. A CL tehermentesíti az IC-t, hogy az ne pazarolja az idejét üzenetekre. A kommunikáció ritmusát előre definiáljuk: SEV1 esetén 30 percenként, SEV2 esetén óránként frissítés.

Scribe (jegyző). Ő vezeti az idővonalat és a döntési naplót. Minden beavatkozást, megfigyelést, kommunikációt időbélyeggel rögzít. A jó idővonal a postmortem alapja, ezért a scribe szerepe nem alárendelt. A jegyzőkönyv akkor is készül, ha a csapatnak nincs dedikált embere; ilyenkor az IC vagy az OL jegyzetel.

Subject Matter Expert (SME). A szakterületi szakértő – például az adatbázis-üzemeltető, a hálózati mérnök, a szolgáltatás fejlesztője –, akit szükség esetén bevonnak. Az SME tanácsot ad, de a beavatkozást az OL koordinálja.

A kommunikáció elsődleges csatornája általában egy dedikált, az incidensre létrehozott szoba vagy csatorna, ahol minden releváns üzenet időbélyeggel megmarad. Emellett a stakeholderek és a felhasználók felé külön felületeken, például status page-en és belső chatben kommunikálunk. A kommunikáció három alapelve:

1. **Gyakoriság a hallgatás ellen.** Ha nincs új információ, akkor is frissítsünk: „Még vizsgáljuk, a hiba továbbra is fennáll.” A csend a stakeholderek számára bizonytalanságot szül.
2. **Tények, nem feltételezések.** A frissítésben a mért adatokat és az eddigi beavatkozásokat közöljük, nem a találgatásokat. A „lehet, hogy a hálózat a hibás” nem státuszfrissítés.
3. **Idővonal a közös kép.** Az incidens alatt a jegyzőkönyv folyamatosan hozzáférhető a csapat számára, így mindenki ugyanazt a képet látja.

Az idővonal-vezetésnek egyszerű, de következetes formátuma van:

Időpont (UTC)	Esemény	Döntés / beavatkozás	Hatás (mért)
14:02	Riasztás: checkout HTTP 500 arány 42%	Triage: SEV1, IC kijelölve	p95 latency 3,2 s
14:08	IC megnyitja az incidenscsatornát, CL értesítve	Kommunikáció: 30 perces ciklus	–
14:15	OL diagnózis: DB pool 100%	Döntés: connection pool emelése 200-ról 400-ra	pool 100% → 87%
14:22	Hibaarány 3%-ra csökken	Helyreállítás megerősítve, SEV1 lezárás előkészítése	HTTP 500 arány 3,1%

Ez a táblázat nem csak a postmortem alapja, hanem az incidens közben is segít a csapatnak emlékezni, mit tett, és mi volt a hatása.

23.3.4 Blameless kultúra és a pszichológiai biztonság szerepe

Az incidenskezelés legnehezebb része nem a technika, hanem a kultúra. Ha a szervezetben egy hiba után az első kérdés az, hogy „ki csinálta”, akkor az emberek védekezni fognak, eltitkolják a hibákat, és a tanulás leáll. A blameless – hibáztatás nélküli – kultúra nem azt jelenti, hogy senki nem felelős; épp ellenkezőleg: mindenki felelős a rendszer javításáért, de a felelősség a folyamatra, nem a személyre irányul.

A pszichológiai biztonság azt jelenti, hogy a csapat tagjai mernek őszintén beszélni a bizonytalanságról, a tévedésekről és a rossz döntésekről anélkül, hogy megtorlástól tartanának. Egy incidens postmortemje akkor működik, ha a résztvevők elmondhatják: „Nem vettem észre a riasztást, mert a telefon néma volt”, „Rossz parancsot futtattam le, mert a runbook elavult volt”, vagy „Azért nem szóltam, mert azt hittem, más már intézi”. Ha ezeket a mondatokat büntetés követi, akkor a következő incidensnél senki sem fog őszintén nyilatkozni.

A blameless kultúra gyakorlati megvalósítása:

- A postmortem vezetője explicit módon kimondja: „Nem keressük a hibást, a rendszer és a folyamatok működését vizsgáljuk.”
- A dokumentumban nem szerepelhet személyre mutató értékelés („XY hibázott”), csak események és rendszerállapotok.
- A hozzájáruló tényezőket több szinten vizsgáljuk: technikai, folyamati, szervezeti. Például egy rossz konfiguráció mögött lehet hiányzó validálás, hiányzó review, vagy túlterhelt csapat.
- A vezetőség is részt vesz a tanulásban, és nem használja a postmortemet teljesítményértékelésre.

Ez a kultúra közvetlen kapcsolatban van a megbízhatósággal: ha az emberek félnek, akkor a hibák rejtve maradnak, a rendszer pedig törékenyebbé válik.

23.3.5 A postmortem: struktúra és tartalom

A postmortem egy írásos dokumentum, amely az incidenst a detektálástól a tanulásig rekonstruálja, és a jövőbeli megelőzést szolgáló akciókat rögzíti. A postmortem nem bürokratikus formalitás; akkor jó, ha valaki, aki nem vett részt az incidensben, elolvasva megérti, mi történt, miért, és mit tettek a hasonló esetek elkerülésére.

Egy jól strukturált postmortem a következő szakaszokat tartalmazza:

1. **Összefoglaló.** Egy bekezdésben: mi történt, mikor, milyen hatással, mennyi ideig tartott, és mi volt a közvetlen kiváltó ok.
2. **Idővonal.** A fenti táblázathoz hasonló, időbélyeggel ellátott eseménysor, a detektálástól a helyreállításig. Itt szerepeljen minden releváns beavatkozás, mérés és kommunikáció.
3. **Hatás.** A felhasználói hatás számszerűsítve: hány felhasználót, milyen régióban, milyen időtartamban érintett; ha van, az anyagi hatás becslése is.
4. **Kiváltó ok.** A technikai kiváltó ok leírása, de nem megállva itt. Fontos a „miért” láncolat: miért nem vettük észre korábban, miért nem volt automatikus mitigáció, miért nem volt a folyamatban védelem.
5. **Hozzájáruló tényezők.** Azok a körülmények, amelyek lehetővé tették, hogy a kiváltó ok incidenshez vezessen. Például: hiányzó rate limiter, elavult runbook, monitoring vakfolt.
6. **Ami jól működött.** Néhány pont, ahol a csapat gyorsan vagy hatékonyan cselekedett. Ez fontos a pszichológiai biztonság és a tanulás szempontjából.

7. **Akciók.** Konkrét, mérhető, tulajdonossal és határidővel rendelkező feladatok. Például: „Rate limiter implementálása a checkout szolgáltatásban, tulajdonos: J. Doe, határidő: 2025-06-30.” Minden akciónak legyen nyomon követhető azonosítója.
8. **Tanulságok.** Rövid, reflektív szakasz: mi az, amit a csapat a jövőben másképp csinálna, mi az, amit meg kell erősíteni.

A postmortem állapotát is érdemes vezetni: draft, review, final, action items closed. A dokumentum verziózva legyen, és a döntésekhez ADR vagy hasonló hivatkozás tartozzon. Például ha az incidens után bevezetünk egy új SLO-t vagy egy új rate limitert, arról ADR készülhet.

23.3.6 Gyakorlati példa: egy fizetési incidens lefolyása

Tekintsünk egy kitalált, de tipikus forgatókönyvet. A rendszer egy e-kereskedelmi platform, ahol a checkout szolgáltatás hirtelen HTTP 500 hibaarányt kezd mutatni. Az alábbiakban az eseményeket a fenti keret szerint mutatjuk be.

Detektálás. 14:02-kor a monitoring riasztást küld: a checkout hibaaránya 10% fölé emelkedett, a küszöb 5% volt. Az ügyeletes mérnök 5 percen belül nyugtázza (TTA = 4 perc). A riasztás az SLO-ból származó SLI-re épült: a checkout szolgáltatás 30 napos gördülő ablakban 99,9%-os rendelkezésre állást ígér, így a hibaarányra vonatkozó küszöböt a fennmaradó error budgetből határozták meg.

Triage. Az ügyeletes megvizsgálja a metrikákat: a hibaarány 42%-on tetőzik, a p95 latency 3,2 másodpercre nőtt, a vásárlások száma drámaian csökken. SEV1-et hirdet, kijelöli magát ideiglenes IC-nek, és megnyitja a dedikált incidenscsatornát.

Diagnózis. Az OL (jelen esetben a payment team ügyeletese) először a trace-eket nézi: a hibák a checkout és a payment-service közötti hívásnál keletkeznek. A logok szerint a payment-service adatbázis-kapcsolatai kimerültek: a connection pool metrika 100%-on áll. A hipotézis: a pool mérete túl alacsony a megugrott forgalomhoz képest. A mérés alátámasztja: az aktív kapcsolatok száma 200, a pool maximuma 200.

Mitigáció. Az IC dönt: növeljék a connection pool méretét 400-ra, és közben a forgalom egy részét tereljék át a másodlagos régióba. Az OL végrehajtja a konfigurációmódosítást. 14:22-re a hibaarány 3%-ra csökken, a pool kihasználtság 87%-on stabilizálódik.

Helyreállítás. A CL 14:25-kor frissíti a status page-et: a checkout újra elérhető, a hibaarány normál tartományban. A rendszer metrikái 15 percen át stabilak, így 14:40-kor az IC lezárja az incidenst, és beütemezi a postmortemet a következő napra.

Tanulás (postmortem). A másnapi megbeszélésen a csapat a jegyzőkönyv alapján rekonstruálja az eseményeket. A kiváltó ok: a pool mérete nem skálázódott a forgalom növekedésével, mert az automatikus skálázási szabály nem terjedt ki a connection poolra. Hozzájáruló tényezők: a terhelésteszt nem modellezte a kiugró forgalmat, a monitoring nem tartalmazott előrejelző riasztást a pool kihasználtságra, és a runbook nem írta le a pool bővítésének menetét. A postmortem akciói: (1) a pool méretének konfigurálhatósága és automatikus skálázása, (2) terhelésteszt frissítése kiugró forgalommal, (3) új riasztás a pool 80% feletti kihasználtságára, (4) a runbook kiegészítése. Mindegyik akció tulajdonossal és határidővel rendelkezik.

Ez a példa bemutatja, hogyan kapcsolódik össze a detektálás, a triage, a diagnózis, a mitigáció, a helyreállítás és a tanulás. A kulcs az, hogy a csapat nem azonnal a gyökér okot kereste, hanem először a felhasználói hatást mérsékelte, és közben minden lépést rögzített.

23.3.7 Tipikus hibák és helyes megoldások

Az incidenskezelés során a leggyakoribb hibák szinte mindig a koordináció, a kommunikáció vagy a tanulás hiányából fakadnak. Az alábbiakban néhány tipikus hibát és a hozzájuk tartozó helyes megoldást mutatunk be.

Hiba 1: A diagnózis és a mitigáció sorrendjének felcserélése. Amikor a csapat addig nyomoz, amíg meg nem találja a kiváltó okot, a felhasználók közben tovább szenvednek. A helyes megoldás: először a hatást kell mérsékelni – visszaállítás, forgalomátterelés, skálázás –, és csak utána kell a gyökér okot feltárni. Az incidens alatt a „mit tudunk most tenni, hogy a felhasználó visszakapja a szolgáltatást" kérdés elsőbbséget élvez.

Hiba 2: A kommunikáció elhanyagolása. A csapat mélyen dolgozik, de a stakeholderek és az ügyfélszolgálat nem tud semmit. A helyes megoldás: dedikált CL szerep, előre definiált frissítési gyakoriság, és a státuszfrissítések akkor is megtörténnek, ha nincs új fejlemény. A csend nem egyenlő a stabilitással.

Hiba 3: Az idővonal hiánya. Incidens után a csapat nem tudja rekonstruálni, mi történt, mert mindenki fejből emlékszik. A helyes megoldás: az incidens közben folyamatosan vezetett jegyzőkönyv, amely minden beavatkozást időbélyeggel rögzít. Ez lehet egy megosztott dokumentum, egy chat-szoba rögzített üzenetei, vagy egy incidenskezelő eszköz.

Hiba 4: A blameless kultúra hiánya. A postmortem során a fókusz a hibás személyre kerül, ezért a résztvevők védekeznek, és a tanulságok felszínesek maradnak. A helyes megoldás: a vezető explicit módon kijelenti a blameless elvet, a dokumentum a rendszer és a folyamat hibáira koncentrál, és a résztvevők nem kapnak retorziót az őszinte beszámolóért.

Hiba 5: Az akciók nem követhetők. A postmortem szépen leírja a tanulságokat, de az akcióknak nincs tulajdonosuk, határidejük vagy nyomon követésük, így a javulás elmarad. A helyes megoldás: minden akció kap egy azonosítót, felelőst, határidőt, és a következő postmortem felülvizsgálja az előzőek állapotát.

Hiba 6: A zajos riasztások miatti immunitás. Ha a csapat túl sok fals pozitív riasztást kap, akkor elkezdi figyelmen kívül hagyni őket, és egy valódi incidens elveszik a zajban. A helyes megoldás: a riasztási szabályokat rendszeresen hangolni, a riasztásokat az SLO-hoz és a felhasználói hatáshoz kötni, és a fals pozitív arányt is mérni.

Ezek a hibák a gyakorlatban gyakran egyszerre jelentkeznek. A jó incidenskezelési folyamat éppen ezért nem csupán technikai, hanem szervezeti és kulturális kérdés is.

23.3.8 Labor: incidensforgatókönyv és postmortem sablon

Ez a labor arra kér, hogy építs fel egy incidensforgatókönyvet, és tölts ki hozzá egy postmortem sablont. A cél a fejezetben bemutatott eszközök alkalmazása egy ellenőrizhető, reprodukálható dokumentumban. A labor nem igényel éles rendszert; minden adat szintetikus, és a konfigurációk szemléltetők.

Kiinduló állapot. Van egy elképzelt mikroszolgáltatás-alapú rendszer: egy „webshop" nevű platform, amely a következő szolgáltatásokból áll: `frontend`, `checkout`, `payment`, `inventory`. Az `inventory` szolgáltatás egy PostgreSQL adatbázist használ. A monitoring az `observability` stackre épül, és az SLO a `checkout` szolgáltatásra 99,9%-os havi rendelkezésre állást ír elő, ami havi 43 perc állásidőt enged meg. A laborban azt az incidenst vizsgáljuk, amikor az `inventory` szolgáltatás válaszideje hirtelen megnő, és a `checkout` hibákat kezd visszaadni, mert a készletellenőrzés timeoutba fut.

A labor lépései.

1. **Incidensforgatókönyv megírása.** Készíts egy rövid, időbélyeggel ellátott eseménysort a detektálástól a helyreállításig. A forgatókönyv tartalmazzon legalább:

   - egy riasztást, amely a detektálást elindítja (metrika, küszöb, hibaarány);
   - a triage döntést (súlyosság, bevont szerepek);
   - egy diagnózis-hipotézist és annak méréssel történő megerősítését;
   - egy mitigációs beavatkozást (például timeout növelése, rate limit, cache bevezetése);
   - a helyreállítás megerősítését. A forgatókönyvhöz add meg a következő kontextust is: környezet, időpont, terhelés, szolgáltatás-verziók. A mért és a várt eredményt külön tüntesd fel.

2. **Postmortem sablon kitöltése.** Használd az alábbi sablont, és töltsd ki a forgatókönyv alapján. A sablon minden szakaszához írj konkrét tartalmat, és ügyelj arra, hogy a dokumentum blameless hangvételű legyen.

```
# Postmortem - [incidens azonosító]

## Összefoglaló
[Mi történt, mikor, milyen hatással, mennyi ideig tartott?]

## Idővonal
| Időpont (UTC) | Esemény | Döntés / beavatkozás | Hatás (mért) |
|---|---|---|---|
| ... | ... | ... | ... |

## Hatás
[Felhasználói hatás számszerűsítve, régió, időtartam, becsült anyagi hatás]

## Kiváltó ok
[A technikai kiváltó ok leírása]

## Hozzájáruló tényezők
- [Folyamati, technikai, szervezeti tényezők]

## Ami jól működött
- [Pozitív megfigyelések]

## Akciók
| Azonosító | Leírás | Tulajdonos | Határidő |
|---|---|---|---|
| AC-001 | ... | ... | ... |

## Tanulságok
[Rövid reflektív szakasz]
```

3. **Döntési napló.** Az incidens kapcsán legalább egy ADR-t készíts, például a mitigációs lépés bevezetéséről. Az ADR formátuma: kontextus, döntés, alternatívák, következmények, állapot.

Elvárt eredmény. A labor végére egy dokumentum (Markdown vagy jegyzetfüzet) készül, amely a fenti elemeket tartalmazza. A dokumentum verziózott, és a mért adatok mellett feltünteti a mérési ablakot és a bizonytalanságot is, például: „a hibaarány 42% volt, 1 perces ablakban mérve, a mérés hibája ± 1 százalékpont”.

Ellenőrzési szempontok.

- A forgatókönyv idővonala visszakereshető, és minden beavatkozás időbélyeggel szerepel.

- A postmortem sablon minden szakasza ki van töltve, és a hangvétel blameless.
- A mért és a várt eredmények elkülönülnek, a mérőszámokhoz mértékegység és mérési ablak tartozik.
- Az ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot.

Visszaállítási út. Mivel a labor szintetikus adatokkal dolgozik, nincs szükség éles környezet visszaállítására. Ha a labor mégis valamilyen helyi tesztkörnyezetben fut, akkor a beavatkozásokat – például konfigurációmódosítást – a labor végén vissza kell állítani az eredeti állapotba, és ezt a dokumentumban rögzíteni kell.

23.3.9 Önellenőrző kérdés

Egy incidens során az IC azt látja, hogy a csapat már 40 perce diagnosztizál, de a felhasználói hibaarány továbbra is 35%. A stakeholderek nem kaptak frissítést az elmúlt 40 percben. Mi a helyes lépés ebben a helyzetben? Indokold a választ a fejezetben bemutatott szerepek és kommunikációs elvek alapján.

23.3.10 Ellenőrzőlista

- Van-e előre definiált súlyossági szint és hozzá tartozó válaszidő?
- Az incidenskezelés szerepei (IC, OL, CL, scribe) ki vannak-e jelölve?
- A kommunikáció gyakorisága és csatornái előre definiáltak?
- Az idővonalat folyamatosan vezeti-e valaki?
- A mitigáció a diagnózis előtt vagy közben megtörténik-e?
- A postmortem blameless hangvételű, és a fókuszt a rendszerre helyezi?
- Az akciók rendelkeznek-e tulajdonossal, határidővel és nyomon követéssel?
- A mért adatok mellett feltüntettük-e a mérési ablakot és a bizonytalanságot?

23.3.11 Döntési gyakorlat

Egy szervezetben a postmortem után kiderült, hogy az incidens kiváltó oka egy elavult runbook volt, amelyet az új csapattag rosszul értelmezett. A vezetőség két lehetőséget mérlegel: (A) a runbookot átdolgozzák, és bevezetik a kötelező páros végrehajtást minden éles beavatkozásra; (B) a hibázó csapattagot figyelmeztetik, és további oktatásra kötelezik. Melyik megoldás illeszkedik jobban a blameless kultúrához és a megelőzéshez? Milyen trade-offokkal jár a választás? Válaszodat a fejezetben bemutatott elvek alapján indokold.

Ez az alfejezet bemutatta az incidenskezelés teljes ívét a detektálástól a tanulásig. A következő egység (Költség- és kapacitásmonitoring) erre építve a költség- és kapacitásmonitoringot tárgyalja, ahol az itt megismert mérési fegyelmet a kapacitástervezésre és a FinOps gyakorlatára alkalmazzuk.

23.4 Költség- és kapacitásmonitoring

A költség- és kapacitásmonitoring akkor válik igazán hasznossá, ha nem csupán a számla végösszegét látjuk, hanem azt is, hogy melyik szolgáltatás, csapat vagy felhasználói út áll a kiadások mögött. Az observability három pillére – logs, metrics, traces – önmagában megmutatja, mi történik a rendszerben, de a gazdasági következmények csak akkor válnak láthatóvá, ha az erőforrás- és költségadatokat ugyanabban a kontextusban vizsgáljuk. Ebben a fejezetben azt járjuk körül, hogyan kapcsoljuk össze ezeket az adatokat, hogyan képezzünk egységköltséget, hogyan ismerjük fel a trendeket és anomáliákat, és hogyan kezeljük a költséget nem funkcionális követelményként.

23.4.1 Erőforrás- és költségadatok összekapcsolása a működéssel

A költségmonitoring első lépése annak megértése, hogy a rendszer működéséből származó erőforrás-fogyasztási adatok és a pénzügyi számlázási adatok más-más világban keletkeznek. Az erőforrás-metrikák – CPU-kihasználtság, memóriahasználat, hálózati forgalom, lemez I/O, konténer-podok száma – perces vagy másodperces felbontásban érkeznek az idősoros adatbázisba. A költségadatok ezzel szemben gyakran napos vagy havi aggregációban, számlázási riportokban vagy API-kon keresztül állnak rendelkezésre, és a felhőszolgáltató árazási modelljétől függően bonyolult szabályok szerint képződnek.

A két adathalmaz összekapcsolásának kulcsa a közös dimenziók és címkék használata. Egy konténeres környezetben például minden pod rendelkezik olyan metaadatokkal, mint a névtér, a szolgáltatásnév, a környezet (fejlesztés, staging, termelés), a csapatazonosító vagy a cost center. Ha ugyanezeket a címkéket alkalmazzuk a felhőerőforrásokra – virtuális gépekre, adatbázisokra, terheléselosztókra, tárolókra –, akkor a számlázási tételek és a metrikák összekapcsolhatók egy közös erőforrás-azonosító vagy címkekészlet mentén. A gyakorlatban ezért a költségmonitoring minősége elsősorban a címkézési stratégia következetességén múlik.

A címkézési stratégia legalább négy dimenziót érdemes lefednie:

- **Szervezeti dimenzió:** melyik csapat, üzleti egység vagy költséghely felelős az erőforrásért.
- **Alkalmazás dimenzió:** melyik szolgáltatás, alkalmazás vagy bounded context használja.
- **Környezet dimenzió:** fejlesztés, teszt, staging vagy termelés.
- **Feature vagy termék dimenzió:** opcionálisan, ha egy szolgáltatás több üzleti képességet szolgál ki.

A hiányzó vagy inkonzisztens címkék a legnagyobb akadályt jelentik. Ha egy erőforrás nem kap `team` vagy `service` címkét, akkor a költsége nem rendelhető hozzá egyetlen csapathoz sem, és a számlázási adatból csak annyit tudunk meg, hogy „valaki” használta. Ezért a címkézési szabályokat – hasonlóan a kódolási konvenciókhoz – verziózott dokumentumban kell rögzíteni, és lehetőség szerint automatikusan kikényszeríteni policy-kkel vagy pipeline-ellenőrzésekkel.

Az összekapcsolás technikai megvalósítása általában egy költség-adatfolyamon keresztül történik. A felhőszolgáltató számlázási adatait – például CSV-export vagy API – rendszeres időközönként betöltjük egy adattárba, ahol az erőforrás-metrikákkal közös dimenziók mentén összekapcsolhatók. Az idősoros metrikákat aggregáljuk a számlázási ablaknak megfelelő felbontásra, majd a két adathalmazt egy dashboardon jelenítjük meg. A cél nem az, hogy minden egyes kéréshez hozzárendeljünk egy forintos költséget, hanem az, hogy trendeket, arányokat és anomáliákat lássunk.

Példaként vegyünk egy három mikroszolgáltatásból álló webáruházat: katalógus, kosár és rendelés. Ha a Kubernetes klaszterben minden pod rendelkezik a `service=katalógus`, `service=kosár`

vagy `service=rendelés` címkével, és ugyanezek a címkék szerepelnek a felhőszolgáltató számlázási rekordjain is, akkor egyetlen lekérdezéssel meg tudjuk mondani, hogy az elmúlt hónapban melyik szolgáltatás mennyit fogyasztott, és ez mennyibe került. Az üzemeltetési csapat így nemcsak azt látja, hogy a rendelési szolgáltatás CPU-használata megugrott, hanem azt is, hogy ez a költségek 40%-os növekedését okozta.

23.4.2 Egységköltség, trend és anomália

23.4.3 Egységköltség

Az egységköltség fogalma leegyszerűsíti a költségadatok értelmezését: egy adott technikai vagy üzleti egységre vetített költséget jelent. Technikai egység lehet egy API-kérés, egy számítási óra, egy gigabájt tárolás vagy egy gigabájt kimenő forgalom. Üzleti egység lehet egy megrendelés, egy aktív felhasználó, egy kosárba helyezés vagy bármely olyan metrika, amely a rendszer értékteremtéséhez kapcsolódik.

Az egységköltség számítása egyszerű hányados:

$$C_{\text{egység}} = \frac{\text{teljes költség az adott ablakban}}{\text{egységek száma az adott ablakban}}$$

Például ha egy szolgáltatás havi költsége 1 200 000 Ft, és ugyanabban a hónapban 4 500 000 API-kérést szolgált ki, akkor az átlagos kérésenkénti költség:

$$C_{\text{kérés}} = \frac{1\,200\,000}{4\,500\,000} = 0{,}2667 \text{ Ft/kérés}$$

Ez a mutató azonban csak akkor értelmes, ha az ablakot, a mértékegységet és a számítási feltételeket pontosan rögzítjük. A példában a teljes költség tartalmazza a számítási, hálózati, tárolási és licenckköltségeket is, nem csak a közvetlen CPU-időt. Ha a fix költségeket – például egy fenntartott adatbázis-példány díját – nem osztjuk szét megfelelően, az egységköltség torzíthat.

Az átlagolás csapdáira is figyelni kell. Egy hónap átlagos kérésenkénti költsége elrejtheti azt, hogy az éjszakai órákban szinte nulla a forgalom, de a csúcsidőben a skálázás miatt az egységköltség a tízszeresére nő. Ezért az egységköltséget érdemes percentilisekkel vagy időablakonkénti bontásban is vizsgálni. A fix és változó költségek szétválasztása szintén fontos: a fix költségek akkor is fennállnak, ha egyetlen kérés sem érkezik, így az egységköltség nem lineárisan csökken a forgalommal.

23.4.4 Trend

A költségadatok idősoros elemzése nélkülözhetetlen a hosszú távú tervezéshez. A trend azt mutatja meg, hogy a költségek milyen irányba és milyen ütemben változnak. A legalapvetőbb módszer a mozgóátlag, amely kisimítja a napi ingadozásokat, és láthatóvá teszi a mögöttes irányt. Egy 7 napos mozgóátlag például a heti szezonalitást is csökkenti, míg egy 30 napos mozgóátlag a havi ciklusokat simítja ki.

Az idősoros adatokban gyakran jelentkezik szezonalitás: a webáruház forgalma hétvégén magasabb, a belső adminisztrációs rendszeré munkanapokon. A trendszámításnál ezt figyelembe kell venni, különben a hétvégi csúcsot tévesen anomáliának észleljük. Az előrejelzéshez használhatunk egyszerű lineáris regressziót a rövid távú becsléshez, vagy exponenciális simítást, amely a régebbi

adatoknak kisebb súlyt ad. A cél nem a pontos jövőbeli költség megjóslása, hanem az, hogy időben észrevegyük, ha a költségek tartósan eltérnek a várt pályától.

A trend vizsgálatánál a mérési ablakot mindig fel kell tüntetni. Egy 30 napos trend nem alkalmas arra, hogy egy éves szezonalitást kimutasson, és fordítva: egy 3 napos mozgóátlag túl zajos ahhoz, hogy döntést alapozzunk rá. A gyakorlatban két trendet érdemes párhuzamosan figyelni: egy rövid távút (7 nap) a gyors visszacsatoláshoz, és egy hosszabb távút (30 vagy 90 nap) a strukturális változásokhoz.

23.4.5 Anomália

Anomáliának nevezzük az idősor azon pontjait, amelyek jelentősen eltérnek a várható értéktől. Ez lehet hirtelen kiugrás – például egy hibás skálázási konfiguráció miatt elindult száz felesleges pod – vagy lassú drift, amikor egy memóriaszivárgás miatt a költségek napról napra nőnek.

Az anomáliadetektálás legegyszerűbb formája a statikus küszöb: ha a napi költség meghaladja az előre meghatározott értéket, riasztást küldünk. Ez azonban gyakran téves riasztásokhoz vezet, mert nem veszi figyelembe a normális ingadozást és a szezonalitást. A robusztusabb módszer a mozgóablak-alapú statisztika: kiszámítjuk az elmúlt 30 nap átlagát és szórását, majd az aktuális értéket ezekhez viszonyítjuk. A Z-score például megmutatja, hogy az aktuális érték hány szórással tér el az átlagtól:

$$Z = \frac{x_t - \mu}{\sigma}$$

ahol x_t az aktuális napi költség, μ a mozgóátlag, σ pedig a mozgó szórás. Ha $|Z| > 3$, az erős anomáliát jelez. Ez a módszer érzékeny azonban a kiugró értékekre, ezért robusztusabb változata a medián és az interkvartilis terjedelem (IQR) használata.

A gépi tanulás alapú módszerek – például isolation forest vagy autoencoder – képesek bonyolultabb mintázatok felismerésére is, de bevezetésük előtt érdemes mérlegelni a komplexitásukat. A legtöbb rendszerben a statisztikai küszöbök és a jól megválasztott mozgóablakok elegendők, a gépi tanulás pedig akkor indokolt, ha a költségmintázatok nagyon sok dimenziótól függenek, és a hagyományos módszerek túl sok téves riasztást adnak.

Az anomáliadetektálás legnagyobb kihívása a riasztási fáradtság elkerülése. Ha minden hétfő reggel a forgalom növekedése riasztást vált ki, az üzemeltetők előbb-utóbb figyelmen kívül hagyják az összes riasztást. Ezért az anomáliariasztásokat mindig kontextussal együtt kell megtervezni: a riasztás csak akkor hasznos, ha tartalmazza az érintett erőforrást, a költségeltérés mértékét, a lehetséges okot, és azt, hogy milyen műveletet várunk el az ügyeletestől. A riasztásoknak – hasonlóan az SLO-alapú riasztásokhoz – rendelkezniük kell leírással, súlyossággal és futási könyvvel (runbook).

23.4.6 A költség mint nem funkcionális követelmény

A költséget gyakran kizárólag pénzügyi kérdésként kezelik, pedig ugyanúgy viselkedik, mint egy nem funkcionális követelmény: befolyásolja az architektúrát, mérhető, és trade-offokban vesz részt más minőségi attribútumokkal szemben. Ha egy rendszerrel szemben követelmény, hogy a havi infrastruktúra-költség nem haladhatja meg az 2 millió forintot, akkor ez ugyanolyan tervezési kényszer, mint a 99,9%-os rendelkezésre állás vagy a 200 ms-os válaszidő.

A minőségi követelményeket quality attribute scenario formában kell megfogalmazni. Egy költségre vonatkozó forgatókönyv így nézhet ki:

- **Inger:** a felhasználói forgalom a napi csúcsidőben eléri az 1000 kérés/másodpercet.
- **Környezet:** a termelési Kubernetes klaszter normál működése, az autoscaling engedélyezett.
- **Válasz:** a rendszer 5 percen belül 3-ról 12 podra skáláz, és a havi infrastruktúra-költség nem haladja meg a 2 200 000 Ft-ot.
- **Mérőszám:** a havi költség 95%-os konfidenciával a büdzsé 110%-án belül marad, az eltérés maximális értéke 200 000 Ft.

Ez a forgatókönyv egyértelművé teszi, hogy a költség nem csak egy utólagos megfigyelés, hanem a rendszer viselkedésére vonatkozó elvárás. A költség-SLO fogalma hasonló az elérhetőségi SLO-hoz: meghatározunk egy célértéket, és figyeljük az attól való eltérést. Az error budget analógiájára beszélhetünk költség-büdzséről is: ha a költségek elérik a büdzsé 90%-át, figyelmeztetést küldünk; ha 100%-át, akkor leállítjuk a nem kritikus erőforrásokat vagy befagyasztjuk a további skálázást.

A költség mint követelmény befolyásolja az architekturális döntéseket is. Ha az egységköltség túl magas, akkor megfontolandó a gyorsítótár bevezetése, az adatbázis-lekérdezések optimalizálása, az aszinkron feldolgozásra való áttérés, vagy a drága felhőszolgáltatások olcsóbb alternatívákkal való helyettesítése. A trade-off azonban ritkán egyirányú: a költségcsökkentés ronthatja a válaszidőt vagy a rendelkezésre állást, ezért a döntéseket mindig a minőségi attribútumok közötti egyensúlyban kell meghozni. A FinOps gyakorlata ezt a szemléletet szervezeti szintre emeli, de a mérési alapokat – a címkézést, az egységköltséget és a trendfigyelést – itt fektetjük le.

A társadalmi és környezeti hatás sem hagyható figyelmen kívül. A felhőerőforrások energiafogyasztása közvetlenül összefügg a szén-dioxid-kibocsátással, ezért a költségmonitoringnak – ahol elérhető – érdemes az energia- és karbonmetrikákat is megjelenítenie. A költség nem az egyetlen szempont: egy olcsóbb erőforrás lehet, hogy magasabb energiafogyasztással jár, és a hosszú távú fenntarthatóság szempontjából rosszabb választás. Ezt a szempontot a FinOps, energia és zöld szoftver fejezet tárgyalja részletesen, itt elég annyi, hogy a mérési keretrendszer kialakításakor gondoljunk erre is.

23.4.7 Labor: költség-dashboard és kapacitásriasztás tervezése

A labor célja, hogy egy reprodukálható dokumentumban vagy jegyzetfüzetben megtervezzünk egy költség-dashboardot és kapacitásriasztásokat egy fiktív mikroszolgáltatás-alapú rendszerhez. A munka során a döntéseket ADR formában rögzítjük, a mért és a várt eredményeket elkülönítjük, és a tervezést ellenőrzési szempontokkal látjuk el.

23.4.8 Kiinduló állapot

Adott egy „Webshop” nevű fiktív e-kereskedelmi rendszer, amely három mikroszolgáltatásból áll: **katalógus**, **kosár** és **rendelés**. A szolgáltatások konténerizálva futnak egy Kubernetes klaszteren, amelyet egy felhőszolgáltató üzemeltet. A rendszerhatár a Webshop belső rendszere és a felhőszolgáltató közötti vonal; a felhasználók a nyilvános interneten keresztül érik el a szolgáltatásokat. A rendszer jelenleg nem rendelkezik költségmonitoringgal, csak alapvető erőforrás-metrikákat gyűjt Prometheus-szal, és a számlázási adatokat havonta CSV-exportból kapja meg.

A laborhoz minden adat szintetikus, nem tartalmaz valós felhasználói vagy pénzügyi adatot. Az alábbi havi költségbecsléseket használjuk (szintetikus, nem ellenőrzött értékek):

- Katalógus szolgáltatás: 600 000 Ft
- Kosár szolgáltatás: 300 000 Ft

- Rendelés szolgáltatás: 450 000 Ft
- Közös infrastruktúra (terheléselosztó, hálózat, tárolás): 250 000 Ft
- Teljes havi költség: 1 600 000 Ft

A forgalmi adatok szintén szintetikusak: átlagosan 2 000 kérés/másodperc, csúcsidőben 5 000 kérés/másodperc, a havi kérések száma összesen körülbelül 80 millió.

23.4.9 Elvárt eredmény

A hallgatónak egy dokumentumot kell készítenie, amely az alábbiakat tartalmazza:

1. **C4 context szintű diagram** a Webshop rendszerről, feltüntetve a felhasználókat, a Webshop belső rendszereit, a felhőszolgáltatót és a monitoring adatfolyamot. A diagramon minden rendszerhatárt jelölni kell, a külső rendszereket és szerepköröket a C4 context szintjén kell feltüntetni, a kapcsolatokon a protokollt, a szinkronitást és a hibatűrést.
2. **Container szintű diagram** a költségmonitoring komponenseiről: metrikagyűjtő (Prometheus), számlázási adatbetöltő, költség-adattár, dashboard (Grafana), riasztáskezelő (Alertmanager). Minden komponenshez felelősséget és interfészt kell leírni.
3. **Dashboard-definíció** táblázatos formában, amely felsorol legalább 8 mérőszámot, megadja azok forrását, dimenzióit, aggregációját és vizualizációját. Például: havi költség szolgáltatásonként, napi költség trend, kérésenkénti egységköltség, CPU-kihasználtság szolgáltatásonként, memóriahasználat, hálózati egress, podok száma, költség-előrejelzés.
4. **Riasztási szabályok** legalább 3 esetre: például ha a napi költség 30%-kal meghaladja a 30 napos mozgóátlagot; ha egy szolgáltatás CPU-kihasználtsága 10 percig 90% felett van; ha a költség-előrejelzés szerint a havi büdzsé 110%-a átlépésre kerül. Minden riasztáshoz tartozik súlyosság, leírás és runbook-hivatkozás.
5. **Egy ADR** a címkézési stratégia bevezetéséről, a döntési napló formátumának megfelelően: kontextus, döntés, alternatívák, következmények, állapot.
6. **Ellenőrzési szempontok** listája, amely alapján a dashboard és a riasztások helyessége utólag ellenőrizhető.

A dokumentumban a mért és a várt eredményeket külön kell választani. Mivel a labor nem futtat éles mérést, a várt eredményeket becslésként kell feltüntetni, és ezt jelölni kell.

23.4.10 Egy lehetséges megoldás vázlata

Az alábbiakban egy példa megoldást mutatunk be, amely szemlélteti az elvárásokat, de nem helyettesíti a hallgató saját munkáját.

23.4.10.1 C4 context diagram (Mermaid)

```
flowchart TB
    User[Felhasználó]
    Webshop[Webshop rendszer]
    Cloud[Felhőszolgáltató]
    Billing[Számlázási rendszer]
    Monitoring[Monitoring rendszer]

    User -->|HTTPS, szinkron, retry| Webshop
    Webshop -->|HTTPS API, szinkron| Cloud
    Cloud -->|Számlázási adatok, CSV/API, napi batch| Billing
    Webshop -->|Metrikák, Prometheus remote write, aszinkron| Monitoring
    Billing -->|Költségadatok, batch import| Monitoring
```

Jelmagyarázat: A nyilak az adatáramlást mutatják; a címkén a protokoll, a szinkronitás és a hibatűrés szerepel. A rendszerhatár a Webshop és a Cloud között húzódik, a Monitoring a Webshop része, de a Cloud és a Billing külső rendszer.

23.4.10.2 Container szintű diagram (Mermaid)

```
flowchart LR
    subgraph Monitoring[Monitoring rendszer]
        Prom[Prometheus]
        Import[Számlázási adatbetöltő]
        Store[Költség-adattár]
        Dash[Grafana]
        Alert[Alertmanager]
    end
    Cloud[Felhőszolgáltató] -->|Metrikák, HTTP| Prom
    Billing[Számlázási rendszer] -->|CSV/API, batch| Import
    Import -->|SQL insert, szinkron| Store
    Prom -->|PromQL, HTTP| Dash
    Store -->|SQL query, HTTP| Dash
    Prom -->|Alert rules, HTTP| Alert
    Dash -->|Dashboard definíció, JSON| Alert
```

Jelmagyarázat: A komponensek felelőssége: Prometheus gyűjti az erőforrás-metrikákat; az Import betölti a számlázási adatokat; a Store tárolja a költségadatokat; a Dash megjeleníti a dashboardot; az Alert kezeli a riasztásokat. Az interfészek a nyilakon feltüntetett protokollok.

23.4.10.3 Dashboard-definíció (kivonat)

Mérőszám	Forrás	Dimenziók	Aggregáció	Vizualizáció
Havi költség szolgáltatásonként	Számlázási adat	service, environment	SUM	Bar chart
Napi költség trend	Számlázási adat	service	SUM, 7 nap mozgóátlag	Line chart
Kérésenkénti egységköltség	Költség / kérések száma	service	RATE	Gauge
CPU-kihasználtság	Prometheus	pod, service	AVG	Time series
Memóriahasználat	Prometheus	pod, service	AVG	Time series
Hálózati egress	Prometheus	service	SUM	Area chart
Aktív podok száma	Kubernetes API	service	AVG	Line chart
Költség-előrejelzés	Költség-adattár + előrejelzés	service	30 nap	Line chart

Feltételjegyzet: A mérési ablak 30 nap, a terhelés a szintetikus forgalmi adatokon alapul. A környezet a szintetikus Kubernetes klaszter, a mérés nem futtatott éles rendszeren.

23.4.10.4 Riasztási szabályok (kivonat)

1. **Napi költség anomália:** ha a napi költség $> 1{,}3 \times 30$ napos mozgóátlag. Súlyosság: közepes. Runbook: ellenőrizni a podok számát és a skálázási konfigurációt.
2. **Magas CPU-kihasználtság:** ha egy szolgáltatás CPU $> 90\%$ 10 percig. Súlyosság: magas. Runbook: ellenőrizni a terhelést, skálázni, vizsgálni a bottlenecket.
3. **Költség-büdzsé előrejelzés:** ha az előrejelzett havi költség $> 110\% \times$ büdzsé. Súlyosság: kritikus. Runbook: leállítani a nem kritikus erőforrásokat, értesíteni a tulajdonos csapatot.

23.4.10.5 ADR minta

- **Cím:** Címkézési stratégia bevezetése a költségallokációhoz
- **Kontextus:** A Webshop rendszerben a költségek nem rendelhetők hozzá a szolgáltatásokhoz, mert az erőforrások nem rendelkeznek egységes címkékkel. A számlázási adatokból csak a teljes havi költség látható.
- **Döntés:** Bevezetünk egy kötelező címkekészletet: `service`, `environment`, `team`, `cost_center`. A címkéket minden felhőerőforrásra és Kubernetes objektumra alkalmazzuk. A hiányzó címkék automatikus észlelése és riasztása is bevezetésre kerül.
- **Alternatívák:** (1) Nem vezetünk be címkézést, csak a teljes költséget figyeljük. (2) Csak a `service` címkét használjuk. (3) Teljes körű címkézés automatizált policy-kkel.
- **Következmények:** A döntés lehetővé teszi a szolgáltatásonkénti költségallokációt, de bevezetése kezdeti erőfeszítést igényel a meglévő erőforrások átcímkézésére. A jövőben minden új erőforrás automatikusan megkapja a címkéket.
- **Állapot:** Elfogadva.

23.4.10.6 Ellenőrzési szempontok

- Minden felhőerőforrás rendelkezik a kötelező címkékkel.
- A dashboardon a költségadatok és a metrikák ugyanabban az időablakban jelennek meg.
- Az egységköltség számítása pontosan dokumentált, a fix és változó költségek szétválasztása megtörtént.
- A riasztások rendelkeznek runbookkal és súlyossággal.
- A várt eredmények becslésként vannak feltüntetve, nem ellenőrzött mérésként.

23.4.11 Visszaállítási út

A labor során nem hajtunk végre éles rendszeren műveletet. Ha a hallgató a példa kedvéért egy tesztkörnyezetben konfigurálja a dashboardot vagy a riasztásokat, akkor azt izolált környezetben tegye, és a labor végén állítsa vissza az eredeti állapotot a konfigurációs fájlok törlésével vagy a teszt-erőforrások felszámolásával. A visszaállítás pontos lépéseit a hallgató dokumentálja a laborjegyzetben.

23.4.12 Ellenőrzőlista

- A költség- és erőforrás-adatok közös dimenziókkal rendelkeznek, és a címkézési stratégia dokumentált.
- Az egységköltség számítása rögzíti az ablakot, a mértékegységet és a költségösszetevőket.
- A trendvizsgálat legalább két időablakot használ, és figyelembe veszi a szezonalitást.

- Az anomáliadetektálás robusztus statisztikai módszert alkalmaz, és a riasztások runbookkal rendelkeznek.
- A költség mint nem funkcionális követelmény quality attribute scenario formában van megfogalmazva, mérőszámmal és mérési ablakkal.
- A dashboardon elkülönülnek a mért és a várt (becsült) eredmények.
- A labor dokumentációja verziózott, és a döntéseket ADR rögzíti.

23.4.13 Önellenőrző kérdés

Miért nem elegendő egy statikus költségküszöb az anomáliadetektáláshoz, és milyen alternatívát javasolnál egy szezonális terhelésű webáruház esetén?

VI. RÉSZ – Delivery, evolúció és szervezet

24 DevOps, CI/CD és deployment stratégiák

24.1 CI/CD alapelvek és pipeline-tervezés

A folyamatos integráció (Continuous Integration, CI) és a folyamatos szállítás (Continuous Delivery, CD) nem egyszerűen eszközök vagy scriptek kérdése, hanem olyan architekturális feltételek együttese, amelyek lehetővé teszik, hogy a változtatások kis lépésekben, gyorsan és visszafordítható módon jussanak el a termelésbe. Ez az alfejezet a CI/CD alapelveit és a pipeline-tervezés gyakorlatát mutatja be, hangsúlyozva, hogy a folyamat kialakítása már az architektúra szintjén eldől: ha a rendszer nem tesztelhető, nem telepíthető vagy nem megfigyelhető, a legkorszerűbb pipeline is csak a hibák gyorsabb előállítását szolgálja.

A fejezet előfeltételezi a version control, a konténerek és a felhőalapú üzemeltetés alapjait, valamint a korábbi observability és reziliencia fogalmakat. Az itt bevezetett fogalmakra épít a következő alfejezet (Infrastruktúra mint kód és konfiguráció), amely az infrastruktúra mint kód és a konfigurációkezelés témáját tárgyalja.

24.1.1 A folyamatos integráció és szállítás meghatározása

A **folyamatos integráció** azt a gyakorlatot jelenti, hogy a fejlesztők a munkájukat naponta többször, kis adagokban beolvasztják egy közös fő ágba, és minden ilyen beolvasztást automatikus build és teszt követ. A cél a visszacsatolási hurok lerövidítése: minél korábban derül ki, ha egy változtatás hibát okoz, annál olcsóbb a javítás. A gyakorlat elterjedése a „build break” fogalmát is átalakította: megszakadt build esetén a csapat elsődleges feladata a zöld állapot visszaállítása, nem pedig új funkció fejlesztése.

A **folyamatos szállítás** ezt a gondolatot terjeszti ki a telepítésre. A cél, hogy a fő ágon lévő, sikeresen integrált változtatások bármikor, egyetlen gombnyomással vagy automatikusan, alacsony kockázattal kiadhatók legyenek. A CD nem jelenti szükségszerűen a folyamatos telepítést (Continuous Deployment), ahol minden változtatás automatikusan a termelésbe kerül; a szállítási képesség azt jelenti, hogy a release jelölt artefaktum minden szükséges teszten és ellenőrzésen átment, és a telepítés folyamata automatizált, megbízható és visszafordítható.

A két fogalmat gyakran együtt emlegetik, de építőköveik különböznek. A CI elsősorban a forráskód-kezelés, a build, a unit teszt és a statikus elemzés területét fedi le; a CD a package, a deployment, a smoke teszt és a release-jóváhagyás rétegét. Az architekturális feltételek azonban közösek: a rendszert úgy kell kialakítani, hogy komponensei önállóan építhetők, tesztelhetők és telepíthetők legyenek, a függőségek explicit módon rögzítettek legyenek, és a környezetek reprodukálhatók legyenek.

24.1.2 A CI/CD architekturális feltételei

A folyamatos integráció és szállítás csak akkor működik jól, ha az architektúra bizonyos tulajdonságokkal rendelkezik. Ezek a feltételek nem kizárólag a DevOps mérnök felelősségei; a

szoftvertervezés során már a komponensek felosztásakor, a függőségek meghatározásakor és az interfészek kialakításakor meg kell teremteni őket.

24.1.2.1 1. Tesztelhetőség

A tesztelhetőség a CI alapja. Ha egy komponens csak bonyolult, kézzel fenntartott infrastruktúrával, külső rendszerekkel vagy valós felhasználói adatokkal tesztelhető, akkor a pipeline lassú, törékeny és drága lesz. Az architektúrának támogatnia kell a tesztelést különböző szinteken: a unit teszthez mockolható függőségek, az integrációs teszthez konténerizált függőségek, a végpontok közötti teszthez jól definiált tesztadatok és izolált környezet szükséges.

A **tesztelhetőség** mint minőségi attribútum quality attribute scenario formában fogalmazható meg. Például:

- **Inger:** egy fejlesztő módosítást küld be a fő ágra.
- **Környezet:** a CI szerver éppen egy buildet futtat.
- **Válasz:** a pipeline automatikusan lefuttatja a unit teszteket és a statikus elemzést, és 10 percen belül visszajelzést ad a fejlesztőnek.
- **Mérőszám:** az esetek 95%-ában a visszajelzés legfeljebb 10 perc; a unit teszt lefedettség legalább 70% a módosított kódrészekre vonatkozóan.

24.1.2.2 2. Reprodukálható build és immutable artefaktumok

A buildnek determinisztikusnak kell lennie: ugyanazon forráskód és függőség-verziók alapján ugyanazt az artefaktumot kell előállítani, függetlenül attól, hogy mikor és hol fut. Ez megköveteli a függőségek explicit verziózását (például lock fájlok, checksum-ellenőrzés), a build környezet konténerizálását vagy egyéb módon történő rögzítését, valamint a beépített artefaktumok verziózott, írásvédett tárolását.

Az **immutable artefaktum** elve szerint a build során létrehozott csomagot (például konténerképet, jar fájlt, zip-et) a későbbi fázisokban nem módosítjuk. A környezeti különbségek nem beégetett konfigurációval, hanem külső konfigurációs mechanizmussal (például környezeti változók, konfigurációs szerver, secret manager) kezelendők. Ez biztosítja, hogy a staging környezetben tesztelt artefaktum és a termelésbe telepített artefaktum azonos legyen. Ha a pipeline során bármilyen módosítás szükséges, az új artefaktumot kell létrehozni, nem pedig a meglévőt átírni.

24.1.2.3 3. Környezetek reprodukálhatósága és elkülönítése

A CD alapfeltétele, hogy a különböző környezetek (fejlesztői, teszt, staging, production) az infrastruktúra és a konfiguráció szempontjából reprodukálhatók legyenek. Az infrastruktúra mint kód (IaC) lehetővé teszi, hogy ugyanaz a definíció hozza létre a teszt- és a termelési környezetet is, csak a méretezési paraméterek és a titkos adatok térjenek el. A konfigurációkezelés szétválasztja a build-time és a runtime konfigurációt: a build során csak a struktúra és az alapértelmezések kerülnek be, a környezetfüggő értékek injection révén jutnak el a futó példányhoz.

A környezetek közötti drift (elsodródás) a CI/CD egyik legnagyobb ellensége. Ha a staging és a production környezet manuális módosítások miatt eltér, akkor a tesztelés eredménye nem lesz megbízható. Az automatizált provisioning és a rendszeres rekreáció (például immutábilis infrastruktúra, konténerek újraépítése) csökkenti ezt a kockázatot.

24.1.2.4 4. Lazán csatolt, telepíthető komponensek

A monolitikus rendszerek esetében a CI/CD pipeline gyakran egyetlen nagy artefaktumot épít, és a telepítés is egyetlen lépés. Ez egyszerűbb, de a visszacsatolási idő hosszabb, és a hibák hatóköre nagyobb. A moduláris vagy mikroszolgáltatás-architektúrák lehetővé teszik a független build és deploy folyamatokat, de ehhez a komponensek közötti szerződések (API, események) explicit kezelése szükséges. A CI/CD szempontjából fontos, hogy a szolgáltatások egymástól függetlenül telepíthetők legyenek, tehát a deploy során ne legyen olyan megszorítás, amely több szolgáltatás egyidejű, atomi telepítését követeli meg. A **contract testing** (szerződésteszt) ebben a kontextusban kulcsfontosságú: a fogyasztó és a szolgáltató közötti kompatibilitást már a pipeline korai fázisában ellenőrizni kell.

24.1.2.5 5. Megfigyelhetőség és visszafordíthatóság

A CD nem ér véget a telepítéssel. A sikeres release feltétele, hogy a rendszer viselkedését a telepítés után is nyomon tudjuk követni, és ha probléma lép fel, a változtatást gyorsan vissza tudjuk vonni. A megfigyelhetőség (observability) magában foglalja a naplókat, metrikákat és nyomkövetési adatokat, de a pipeline tervezésekor azt is biztosítani kell, hogy a telepítéshez tartozó metaadatok (verzió, commit hash, deploy időpontja) lekérdezhetők legyenek. A **rollback** (visszaállítás) mechanizmusa nem a régi kód újratelepítését jelenti elsősorban, hanem a korábbi, ismert jó állapotú artefaktumra való gyors visszaváltást, amihez verziózott artefaktumtár és automatizált telepítési folyamat szükséges.

24.1.3 Alapfogalmak: build, teszt, artefaktum és környezet

A pipeline tervezésének megértéséhez tisztázni kell néhány alapfogalmat. Ezek a fogalmak a CI/CD folyamat szakaszait és azok bemeneti/kimeneti termékeit írják le.

24.1.3.1 Build

A **build** (építés) az a folyamat, amely a forráskódból és a függőségekből előállít egy futtatható vagy telepíthető egységet. Ez magában foglalhatja a fordítást, a csomagolást, a konténerkép létrehozását, a statikus erőforrások másolását és a függőségek feloldását. A build akkor jó, ha gyors, determinisztikus és izolált. A gyorsaság a visszacsatolási idő szempontjából kritikus; a determinisztikusság a reprodukálhatóságot biztosítja; az izoláció pedig azt jelenti, hogy a build ne függjön a build gépen lévő kézzel telepített eszközöktől. A **cache** használata (például függőségi cache, rétegelt konténer build) csökkentheti az időt, de a cache-hit és cache-miss eseteket explicit módon kell kezelni, hogy ne sérüljön a reprodukálhatóság.

24.1.3.2 Teszt

A pipeline tesztszakaszai különböző szintű visszajelzést adnak. A **unit tesztek** a komponens belső logikáját ellenőrzik, gyorsak és nagy számban futnak. A **statikus elemzés** (kódminőség, biztonsági szabályok, sebezhetőség-keresés) nem igényel futási környezetet, de fontos minőségi kaput képez. Az **integrációs tesztek** a komponensek közötti együttműködést ellenőrzik, gyakran konténerizált függőségekkel (adatbázis, message broker). A **szerződéstesztek** az API-k kompatibilitását validálják a fogyasztó és a szolgáltató között. A **végpontok közötti (E2E) tesztek** a teljes üzleti folyamatot fedik le, de lassúak és törékenyek, ezért csak a kritikus utakra

koncentráljunk. A pipeline tervezésekor a teszteket a visszacsatolási idő és a kockázat alapján kell sorba rendezni: először a gyors, olcsó tesztek fussanak, majd a lassabbak.

24.1.3.3 Artefaktum

Az **artefaktum** a build eredménye, amely a későbbi fázisokban felhasználásra kerül. Lehet bináris, konténerkép, csomag, könyvtár vagy akár konfigurációs fájl. A jó artefaktum:

- verziózott (például szemver vagy commit hash alapján),
- egyedi azonosítóval ellátott, amelyből visszakereshető a forráskód állapota,
- írásvédett tárolóban (artifact repository) tárolt,
- digitálisan aláírt vagy legalább checksummal ellenőrzött,
- a fogyasztó számára letölthető, de nem módosítható.

Az artefaktumok életciklusa elkülönül a forráskódtól: a forráskód változik, de az artefaktum egy adott állapotot rögzít. A CD során ugyanazt az artefaktumot léptetjük előre a környezeteken, nem építjük újra minden környezethez.

24.1.3.4 Környezet

A **környezet** az a futási kontextus, ahol az artefaktum telepítésre kerül. Tipikus környezetek: fejlesztői (local), integrációs (CI), teszt, staging, production. A pipeline tervezésekor minden környezethez tartozó telepítési szabályokat, konfigurációt és hozzáférési jogokat külön kell meghatározni. A környezetek közötti előreléptetés (promotion) legyen automatizált, de az éles környezetbe történő telepítéshez gyakran manuális jóváhagyás szükséges (ha a folyamatos telepítés nem a cél). A környezetek konfigurációját ne a pipeline scriptben tároljuk, hanem külön konfigurációs tárolóban (például Git repository, Vault, ConfigMap), hogy a környezetváltozások ne igényeljenek pipeline módosítást.

24.1.4 Pipeline-tervezés gyakorlati mintával

A pipeline a CI/CD folyamat technikai megvalósítása: egy sor automatikus lépés, amely a forráskód beolvasztásától a telepítésig vezet. A tervezés során nem az a kérdés, hogy milyen Jenkinsfile vagy GitHub Actions YAML kell, hanem az, hogy milyen **szakaszokra** bontjuk a folyamatot, és milyen **minőségi kapukat** helyezünk el.

Egy tipikus pipeline szakaszai:

1. **Trigger és forráskinyerés:** a pipeline akkor indul, ha változtatás érkezik a fő ágra (push, pull request merge), vagy ütemezetten. A forráskód letöltése után a commit hash és a kapcsolódó metaadatok rögzítése történik.
2. **Build és függőségek:** a build elvégzése, a függőségek feloldása, az artefaktum létrehozása. Ebben a szakaszban célszerű a build számára dedikált, izolált konténert használni, és a függőségi cache-t rétegezni.
3. **Unit tesztek és statikus elemzés:** a gyors tesztek és a kódminőségi ellenőrzések. A minőségi kapu például: a unit tesztek 100%-a sikeres, a lefedettség nem csökkenhet 5%-nál nagyobb mértékben, a critical és high súlyosságú sebezhetőségek száma nulla.
4. **Package és artefaktum tárolás:** az épített artefaktum verziózott tárolása az artefaktumtárban. Itt történik a digitális aláírás vagy checksum generálás.

5. **Integrációs és szerződéstesztek:** a komponensek közötti együttműködés ellenőrzése. Ehhez ideiglenes környezetet kell felépíteni a szükséges függőségekkel (adatbázis, message broker, szomszédos szolgáltatások stubjai).
6. **Deployment tesztkörnyezetbe:** az artefaktum telepítése egy teszt vagy staging környezetbe. A telepítés során a konfigurációt a környezethez tartozó értékekkel injektáljuk.
7. **Smoke tesztek és E2E tesztek:** a kritikus üzleti folyamatok ellenőrzése a telepített környezetben. A smoke teszt gyorsan ellenőrzi, hogy az alkalmazás elindult-e, az alapvető végpontok válaszolnak-e; az E2E teszt a teljes üzleti folyamatot fedi le, de csak a legfontosabb útvonalakra.
8. **Release jóváhagyás és production deployment:** ha a CD nem automatikus telepítés, akkor itt manuális jóváhagyásra lehet szükség. A production deployment során a korábban tesztelt, immutable artefaktum kerül telepítésre, a megfelelő konfigurációval.
9. **Post-deploy ellenőrzés:** a telepítés utáni megfigyelés, riasztások ellenőrzése. Ez a szakasz határozza meg, hogy sikeresnek tekinthető-e a release, vagy szükséges-e a rollback.

A pipeline tervezésekor minden szakaszhoz **input**, **output**, **felelős**, **időtartam-cél** és **hibakezelési stratégia** tartozik. A hibakezelés lehet automatikus újrapróbálkozás, a pipeline leállítása, a fejlesztő értesítése vagy a környezet visszaállítása. Fontos, hogy a pipeline ne legyen „fekete doboz”: a futás közben keletkező naplókat, mérési adatokat meg kell őrizni, és a hibákat egyértelmű üzenettel kell jelezni.

24.1.4.1 Példa: egy mikroszolgáltatás pipeline-terve

Tekintsünk egy kitalált `order-service` nevű mikroszolgáltatást, amely rendeléseket kezel, és HTTP API-t, valamint eseményeket használ. A szolgáltatás konténerizált, a forráskód egy Git monorepóban található. A pipeline terve a következő lehet:

- **Trigger:** push a `main` ágra, vagy pull request merge.
- **Build stage:** Maven vagy Gradle build, konténerkép építése. A build 5 perc alatt kell, hogy lefusson. A függőségeket a `pom.xml` lockolt verziói tartalmazzák.
- **Unit + static analysis:** JUnit tesztek és SonarQube-szerű elemzés. A lefedettség minimum 80% az új kódra; critical issue nem lehet.
- **Package:** a konténerkép push az `<ARTIFACT_REGISTRY>/order-service:<version>` tárolóba. A verzió a `main` ágon a Git SHA rövidített alakja, előtaggal: `git-<sha>`.
- **Integration + contract:** a szolgáltatás indítása konténerben, egy ideiglenes PostgreSQL és RabbitMQ példánnyal. A szerződéstesztek a korábban rögzített OpenAPI specifikáció alapján ellenőrzik a kontrollert.
- **Deploy staging:** a konténerkép telepítése a staging Kubernetes namespace-be, a konfiguráció a `<CONFIG_REPO>/order-service/staging.yaml` fájlból származik.
- **Smoke + E2E:** egy minimális rendelés-létrehozás folyamat végigfuttatása a staging környezetben. A válaszidő P95 500 ms alatt legyen.
- **Production deployment:** manuális jóváhagyás után a staging során használt ugyanazon konténerkép telepítése a production namespace-be, a production konfigurációval.
- **Post-deploy:** a telepítés utáni 10 perces ablakban a hibaráta és a válaszidő metrikák figyelése; ha a hibaráta meghaladja az 1%-ot, automatikus riasztás és rollback a korábbi verzióra.

Ez a terv egy **ADR**-ben rögzíthető, például:

> **ADR-012: CI/CD pipeline a rendelés-szolgáltatáshoz Kontextus:** A rendelés-szolgáltatás fejlesztése gyors ütemben halad, a manuális tesztelés és telepítés nem skálázódik. A csapat célja a 10 perces visszacsatolási idő és a napi többszöri release

képesség. **Döntés:** A pipeline a fent leírt szakaszokat követi, immutable konténerképekkel, a staging és production környezetek közötti artefaktum-promócióval. A production deployment manuális jóváhagyást igényel. **Alternatívák:** (1) teljesen automatikus telepítés – elvetve, mert a csapat még nem rendelkezik elég erős post-deploy automatikus rollback mechanizmussal. (2) deployment scriptek nélküli manuális folyamat – elvetve, mert a hibák aránya magas volt. **Következmények:** A pipeline fenntartása igényel egy dedikált build konténert és a tesztkörnyezet automatikus provisioningjét. A visszacsatolási idő várhatóan 10 percen belül marad a unit és integration szakaszokra. A rollback a korábbi konténerkép újratelepítésével 2 percen belül elvégezhető. **Állapot:** Elfogadva.

24.1.5 Gyors visszacsatolás és minőségi kapuk

A CI/CD egyik legfontosabb alapelve a **gyors visszacsatolás**. Minél hamarabb értesül a fejlesztő a hibáról, annál kisebb a kontextusváltás költsége, és annál nagyobb az esély, hogy a hibát azonnal ki tudja javítani. A visszacsatolási idő mérésére alkalmas mérőszám a **pipeline futási idő** (medián, P95) és a **hiba észleléséig eltelt idő**. A DORA-kutatások szerint a magas teljesítményű csapatoknál a build és teszt folyamat jellemzően 10 percen belül visszajelzést ad, de ez a szám kontextusfüggő; a lényeg, hogy a csapat saját célértéket határozzon meg, és mérje a tényleges teljesítményt.

A gyors visszacsatolás eléréséhez a pipeline-t rétegezni kell: a gyors, olcsó ellenőrzések fussanak először, a lassabbak később. Ha a unit teszt vagy a statikus elemzés már a korai fázisban elbukik, nem érdemes a teljes integrációs tesztet lefuttatni. Ez a **fail fast** elv. Ugyanakkor vigyázni kell, hogy a rétegezés ne vezessen hamis biztonságérzethez: a későbbi fázisokban futó teszteknek is rendszeresen le kell futniuk, nem csak akkor, ha a korábbiak mind zöldek.

A **minőségi kapu** (quality gate) egy olyan ellenőrzési pont a pipeline-ban, ahol a folyamat csak akkor mehet tovább, ha bizonyos feltételek teljesülnek. Példák minőségi kapura:

- **Kódminőség:** az új kódra vonatkozó lefedettség legalább 80%; a duplikáció nem haladhatja meg az 5%-ot; a kritikus vagy magas súlyosságú kódminőségi problémák száma nulla.
- **Biztonság:** a függőségi sebezhetőség-szkenner nem talál ismert kritikus sebezhetőséget; a konténerkép sebezhetőségi vizsgálata nem mutat kritikus hibát.
- **Teljesítmény:** a staging környezetben mért P95 válaszidő a kritikus végpontra nem haladja meg az 500 ms-ot; a hibaráta nem haladja meg az 1%-ot.
- **Szerződés:** a szerződéstesztek mind sikeresek; az OpenAPI specifikáció változása nem sérti a meglévő fogyasztókat.

A minőségi kapukat nem szabad túlzásba vinni, mert a túl szigorú vagy túl sok kapu lassítja a folyamatot és a fejlesztők megkerülő megoldásokhoz folyamodnak. A kapukat a kockázat alapján kell kiválasztani, és a csapattal egyeztetni kell, hogy mi számít elfogadhatónak. A kapu megsértése esetén a pipeline álljon le, és a fejlesztő kapjon egyértelmű üzenetet a hiba okáról és a javítási lehetőségről. A **fitness function** szemlélet itt is érvényes: a minőségi kapu olyan automatikus ellenőrzés, amely a rendszer egy minőségi attribútumának megőrzését szolgálja.

24.1.6 Tipikus hibák és ellenminták

A CI/CD bevezetésekor számos csapdába eshet a csapat. Ezek felismerése segít elkerülni a költséges hibákat.

24.1.6.1 1. A pipeline mint varázsdoboz

Ha a pipeline scriptek bonyolult, dokumentálatlan, egymásba ágyazott lépésekből állnak, akkor a csapat nem tudja, mi történik pontosan, és a hibakeresés nehéz. A pipeline kódját ugyanúgy kell kezelni, mint a termékkódot: verziózni, review-zni, tesztelni. A lépéseket jól látható, külön fájlokban vagy modulokban érdemes szervezni, és minden lépéshez tartozzék leírás a bemenetről, kimenetről és a hibakezelésről.

24.1.6.2 2. Túl hosszú pipeline

A 30 perces vagy annál hosszabb pipeline elriasztja a fejlesztőket a gyakori integrációtól. A hosszú futásidőt okozhatja a nem megfelelő sorrend (lassú tesztek az elején), a párhuzamosítás hiánya, a cache nem megfelelő használata, vagy a túlzott E2E tesztelés. A megoldás a pipeline szakaszainak párhuzamosítása (ahol a függőségek engedik), a tesztcsomagok szétválasztása, a cache stratégia kidolgozása, és a lassú tesztek áthelyezése későbbi, opcionális szakaszba.

24.1.6.3 3. Környezetek közötti drift

Ha a staging és a production környezet kézzel van karbantartva, akkor előbb-utóbb eltérnek. A tesztelés eredménye megbízhatatlanná válik, és a production deployment meglepetéseket okoz. A megoldás az infrastruktúra mint kód használata, az automatikus provisioning, és a rendszeres környezeti rekreáció. A konténerek és a Kubernetes ezt nagymértékben támogatják.

24.1.6.4 4. Artefaktum újraépítése környezetenként

Ha minden környezethez újra buildelik az alkalmazást, akkor nem biztosítható, hogy ugyanaz a kód kerül termelésbe, mint amit teszteltek. A build során a környezetfüggő beállítások beégetése is ide tartozik. A helyes gyakorlat az immutable artefaktum: egyszer építünk, és ugyanazt az artefaktumot léptetjük előre, a konfigurációt kívülről injektáljuk.

24.1.6.5 5. Nincs rollback stratégia

A pipeline csak előre tud lépni, de ha a production deployment hibát okoz, nincs gyors visszaút. A rollback nem a régi kód újrafordítását jelenti, hanem a korábbi, ismert jó artefaktum újbóli telepítését. Ehhez szükséges az artefaktumok verziózott tárolása, a deployment folyamat idempotens kialakítása, és a rollback lépések előre tesztelése. A blue-green vagy canary deployment (lásd későbbi alfejezet) szintén segíti a gyors visszavonást.

24.1.6.6 6. Tesztadatok valós személyes adatokkal

A CI/CD pipeline-ban tilos valós személyes adatokat használni, mert az adatvédelmi és biztonsági kockázatot jelent. A tesztkörnyezetekben szintetikus vagy álnevesített adatokat kell alkalmazni. Az adatbázis-migrációkat is úgy kell tesztelni, hogy azok ne tartalmazzanak éles adatot.

24.1.6.7 7. Hiányzó megfigyelhetőség a pipeline-ban

Ha a pipeline futása közben nem gyűjtünk mérési adatokat (időtartam, sikerességi arány, hibaüzenetek), akkor nem tudjuk, hol vannak a szűk keresztmetszetek. A pipeline-hoz is tartoznia kell observability rétegnek: logok, metrikák, riasztások. A futási idő trendje, a gyakran elbukó szakaszok és a flaky tesztek aránya mind fontos mérőszám.

24.1.7 Labor: CI/CD pipeline tervezése

A labor célja, hogy a résztvevő egy kis, ellenőrizhető példán keresztül alkalmazza a pipeline-tervezés alapelveit. A feladat nem a konkrét eszköz konfigurálása, hanem a folyamat megtervezése, dokumentálása és a döntési pontok rögzítése.

Kiinduló állapot: Adott egy `payment-service` nevű mikroszolgáltatás forráskódja Git repositoryban. A szolgáltatás REST API-t és egy külső hitelkártya-feldolgozó rendszerhez való kapcsolódást igényel. A csapat jelenleg manuálisan buildel és telepít, a tesztek egy része kézi futtatású.

Elvárt eredmény: Egy reprodukálható dokumentum (például Markdown vagy jegyzetfüzet), amely tartalmazza:

1. **Kontextus:** a jelenlegi állapot leírása, a célok (például gyorsabb release, hibák csökkentése) és a kényszerek (például a hitelkártya-rendszerhez való hozzáférés teszteléskor szimulátorral történik).
2. **Pipeline-terv:** a szakaszok felsorolása, mindegyikhez input, output, időtartam-cél és hibakezelési stratégia. A terv készülhet szöveges leírásként vagy diagramként (például Mermaid flowchart).
3. **Minőségi kapuk:** legalább három minőségi kapu meghatározása, mérhető feltételekkel. Például a unit teszt lefedettség 80% felett, a sebezhetőségi szkenner nem talál kritikus hibát, a staging smoke teszt sikeres.
4. **ADR vázlat:** egy rövid ADR a pipeline bevezetéséről, a kontextus, döntés, alternatívák, következmények és állapot mezőkkel.
5. **Ellenőrzési szempontok:** hogyan fogják mérni a pipeline hatékonyságát (például a visszacsatolási idő P95, a sikertelen build arány, a release gyakoriság), milyen mérési ablakkal.

Ellenőrzési szempontok (a labor értékeléséhez):

- A pipeline szakaszok logikus sorrendben vannak, a fail fast elv érvényesül.
- Az artefaktum immutable, és a környezetek közötti előreléptetés nem jár újraépítéssel.
- A minőségi kapuk mérhető feltételeket tartalmaznak, mértékegységgel és mérési ablakkal.
- Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket.
- A dokumentum elkülöníti a mért és a várt eredményt: jelen állapotban még nincsenek mért adatok, ezért a tervben a „várt” értékeket jelölni kell.
- Nem szerepel valós hitelesítő adat, belső URL vagy személyes adat.

Visszaállítási út: Mivel a labor csak tervezési feladat, nincs szükség éles rendszer módosítására. Ha a résztvevő mégis kipróbál egy pipeline eszközt, akkor azt saját környezetben tegye, és a labor dokumentációban jelezze, hogy a bemutatott konfiguráció szemléltető, nem éles rendszeren futott.

24.1.8 Ellenőrzőlista

A fejezet végén ellenőrizze, hogy az alábbi pontokat megértette-e és alkalmazni tudja-e:

- ☐ Meg tudom fogalmazni a folyamatos integráció és a folyamatos szállítás közötti különbséget.
- ☐ Fel tudom sorolni a CI/CD architekturális feltételeit, és mindegyikhez tudok példát mondani.
- ☐ Meg tudom határozni a build, teszt, artefaktum és környezet fogalmát.
- ☐ Tervezek egy pipeline-t szakaszokra bontva, és minden szakaszhoz inputot, outputot és hibakezelést rendelek.
- ☐ Alkalmazom az immutable artefaktum és a környezetek reprodukálhatóságának elvét.
- ☐ Definiálok minőségi kapukat mérhető feltételekkel.
- ☐ Felismerem a tipikus hibákat (hosszú pipeline, drift, újraépítés, nincs rollback, valós tesztadatok) és meg tudom indokolni a helyes megoldást.
- ☐ Írok egy ADR-t a pipeline bevezetéséről.

24.1.9 Önellenőrző kérdés

Egy csapat azt tapasztalja, hogy a pipeline futási ideje 45 perc, és a fejlesztők ezért ritkán integrálnak. Milyen három konkrét lépést javasolna a visszacsatolási idő csökkentésére, és hogyan indokolná az architektúrára gyakorolt hatásukat? (Gondoljon a tesztelési rétegekre, a párhuzamosításra, a cache stratégiára és a fail fast elv érvényesítésére.)

24.1.10 Összefoglalás

A folyamatos integráció és szállítás architekturális feltételei közé tartozik a tesztelhetőség, a reprodukálható build, az immutable artefaktumok, a környezetek reprodukálhatósága, a lazán csatolt komponensek és a megfigyelhetőség. A pipeline tervezése során a szakaszokat a visszacsatolási idő és a minőségi kapuk mentén kell kialakítani, a fail fast elv érvényesítésével. A gyakori hibák elkerülése érdekében a pipeline kódját verziózni, a folyamatot mérni és a döntéseket ADR-ben rögzíteni kell. A következő alfejezetben az infrastruktúra mint kód és a konfigurációkezelés témaköre épít ezekre az alapelvekre, hogy a környezetek reprodukálhatósága automatizált módon is megvalósuljon.

24.2 Infrastruktúra mint kód és konfiguráció

Az infrastruktúra és a konfiguráció kezelése a modern szoftverrendszerek egyik legnagyobb kockázati forrása: ha a környezeteket kézzel, konzolokról vagy ad hoc szkriptekkel hozzuk létre, akkor a reprodukálhatóság illúzió, a hibakeresés nehéz, a katasztrófa utáni helyreállítás pedig lassú és bizonytalan. Az infrastruktúra mint kód (Infrastructure as Code, IaC) ezt a problémát úgy kezeli, hogy az infrastruktúra definícióját is szoftverként, verziózható, tesztelhető és újraépíthető módon tárolja. Ez a fejezet az IaC alapfogalmait, a deklaratív állapotkezelést, a drift és a titokkezelés kérdéseit, valamint a fejlesztői, staging és termelési környezetek elkülönítését tárgyalja gyakorlati példákkal és ellenőrzési szempontokkal.

24.2.1 Az infrastruktúra mint kód célja: verziózás és reprodukálhatóság

Az IaC lényege, hogy az infrastruktúrát – szervereket, hálózatokat, adatbázisokat, terheléselosztókat, Kubernetes-klasztereket – ne kézzel, hanem verziózott konfigurációs fájlok segítségével hozzuk létre és menedzseljük. A cél kettős: egyrészt a **verziózás** biztosítja, hogy minden változtatás nyomon követhető, visszaállítható és felülvizsgálható legyen; másrészt a **reprodukálhatóság** garantálja, hogy ugyanabból a definícióból bármikor, bármely környezetben ugyanaz az infrastruktúra álljon elő.

A verziózás itt szó szerint a Git vagy más verziókezelő rendszer használatát jelenti. Ha az infrastruktúra leírása egy repositoryban él, akkor ugyanazokat a munkafolyamatokat alkalmazhatjuk, mint a kódnál: pull request, code review, automatizált tesztek, ágak és címkék. A változtatások előzményei visszakereshetők, és egy hibás módosítás visszavonható. Ez nem csupán kényelem, hanem auditálhatósági és megfelelési követelmény is lehet (például pénzügyi vagy egészségügyi rendszerekben).

A reprodukálhatóság azt jelenti, hogy a definícióból kiindulva, determinisztikus módon létrehozható egy környezet anélkül, hogy manuális lépésekre vagy dokumentált, de kézzel végrehajtott utasításokra támaszkodnánk. Ez a katasztrófa-helyreállítás, a tesztkörnyezetek gyors létrehozása és a horizontális skálázás alapja. A reprodukálhatóság nem abszolút: a felhőszolgáltatók időnként változtatnak a szolgáltatásokon, és a külső függőségek is befolyásolhatják az eredményt, ezért a gyakorlatban a cél az, hogy az eltéréseket minimalizáljuk, és ahol szükséges, rögzítsük a verziókat (pinning).

A 12-factor alkalmazások harmadik faktora, a konfiguráció elkülönítése is szorosan kapcsolódik ide: a kód és a konfiguráció különválasztása lehetővé teszi, hogy ugyanaz az alkalmazáskód több környezetben fusson anélkül, hogy módosítani kellene. Az IaC ezt az elvet terjeszti ki az infrastruktúrára is: a környezetek közötti különbségeket nem a kódban, hanem változókban, paraméterfájlokban vagy külső konfigurációs forrásokban tároljuk.

24.2.2 Deklaratív állapot, drift és konzisztencia

Az IaC eszközök két fő paradigmát követnek: az imperatív és a deklaratív megközelítést. Az imperatív eszközök (például hagyományos szkriptek, Ansible playbookok bizonyos használati módjai) lépésről lépésre megmondják, hogy mit kell csinálni: „hozd létre ezt a szervert, majd telepíts rá egy csomagot, majd indíts el egy szolgáltatást”. A deklaratív eszközök ezzel szemben a kívánt végállapotot írják le: „legyen egy virtuális gép ezzel a paraméterrel, és ez a szolgáltatás fusson rajta”. A Terraform, a Kubernetes manifestek, a CloudFormation és a Pulumi (részben) deklaratívnak tekinthetők.

A deklaratív modell előnye, hogy az eszköz maga számolja ki a különbséget a jelenlegi állapot és a kívánt állapot között, és csak a szükséges változtatásokat hajtja végre. Ez csökkenti a hibalehetőséget, és megkönnyíti az állapot rekonstrukcióját. Hátránya, hogy a mögöttes logika néha nem látható, és komplex állapotfüggőségek esetén nehéz lehet megjósolni a hatást. A gyakorlatban a legtöbb modern rendszer deklaratív megközelítést használ, de a hibakereséshez és az egyszeri feladatokhoz gyakran keverik imperatív lépésekkel.

A **drift** fogalma akkor merül fel, amikor a valós infrastruktúra eltér attól, amit a kód leír. Ennek több oka lehet: valaki kézzel módosít egy erőforrást a felhőkonzolon, egy meghibásodott komponens automatikusan helyreáll, de nem a definícióban rögzített módon, vagy egy frissítés megváltoztat egy alapértelmezett értéket. A drift veszélye abban rejlik, hogy a következő `apply` során az eszköz vagy felülírja a kézi változtatást (ami adatvesztéshez vezethet), vagy – ha a driftet nem érzékeli – a dokumentáció és a valóság egyre inkább eltér egymástól.

A drift kezelésének alapja a rendszeres **állapot-összehasonlítás**. A Terraform például a `plan` parancs segítségével mutatja meg, hogy milyen változtatásokat hajtana végre, és a `-detailed-exitcode` opcióval automatizáltan jelezhető, ha eltérés van. A Kubernetes esetében a GitOps eszközök (például az Argo CD) folyamatosan figyelik a Git repository és a klaszter közötti különbséget, és riasztást adnak, vagy szinkronizálják az állapotot. A drift detektálás mérése például a következő minőségi forgatókönyvvel írható le: amikor egy erőforrás konfigurációját kézzel megváltoztatják, a rendszer 5 percen belül észleli az eltérést, és értesítést küld a platformcsapatnak.

A drift megelőzésének egyik eszköze az, ha a termelési környezetben megtiltjuk a közvetlen módosítást. A felhőszolgáltatóknál ehhez IAM-policyk és szerepkörök használhatók: a fejlesztők csak olvasási jogot kapnak, a módosítás kizárólag a CI/CD pipeline-on keresztül, a verziózott kódból történhet. Ez a gyakorlatban nem mindig valósítható meg teljesen, de a cél az, hogy a rendszer a definiált folyamaton kívül ne engedjen változtatást.

24.2.3 Titokkezelés az IaC-ben

Az infrastruktúra definíciója gyakran tartalmaz érzékeny adatokat: adatbázis-jelszavakat, API-kulcsokat, privát kulcsokat, tanúsítványokat. Ezeket soha nem szabad a verziókezelőbe commitolni, mert a repository története miatt a törlés sem segít – a titkok örökre benne maradnak, hacsak nem töröljük a teljes history-t. A titokkezelés célja, hogy a titkok kívül maradjanak a kódon, és csak futásidőben, biztonságos csatornán jussanak el a célrendszerbe.

A legelterjedtebb megoldások:

- **Környezeti változók**: a titkokat a futási környezetben, például a CI/CD rendszer titkos változóiban tároljuk, és a kód csak hivatkozik rájuk. Ez egyszerű, de a titkok megjelennek a folyamat környezetében, és naplózásnál vagy hibakeresésnél véletlenül kiszivároghatnak.
- **Titokkezelő szolgáltatások**: dedikált rendszerek, mint a HashiCorp Vault, AWS Secrets Manager, Azure Key Vault vagy a Kubernetes Secrets (megfelelő titkosítással). Az IaC eszközök ezekből olvassák be a titkokat futásidőben, így a definícióban csak a hivatkozás szerepel.
- **Titkosított fájlok a repositoryban**: például a SOPS vagy a git-crypt lehetővé teszi, hogy a titkosított konfigurációt a kóddal együtt verziózzuk, és csak a jogosultak tudják visszafejteni. Ez kényelmes, de a kulcskezelés továbbra is kihívás, és a titkosított fájl felülírása esetén a régi verzió még visszafejthető lehet.

A laboratóriumi és példakódokban mindig helyőrzőket használunk: `<DB_PASSWORD>`, `<API_KEY>` vagy `var.db_password`. A Terraform-ban a `sensitive = true` attribútum jelzi, hogy egy változó értéke titkos, és a kimenetben ne jelenjen meg. A titokkezelésre vonatkozó szabály: ha egy érték bárki számára hozzáférhetővé teszi a rendszert, akkor titoknak minősül, és nem kerülhet a repositoryba.

A titkok rotációja is része az életciklusnak. Az IaC-nek támogatnia kell, hogy a titkokat cserélni lehessen anélkül, hogy a teljes infrastruktúrát újra kellene építeni. Például ha egy adatbázis-jelszót lecserélünk a Secrets Managerben, akkor a Terraform `apply` ismételt futtatásakor észleli a változást, és frissíti a hivatkozó erőforrásokat.

24.2.4 Fejlesztői, staging és termelési környezet

Az IaC egyik fő előnye, hogy ugyanazt a definíciót több környezetben is felhasználhatjuk, ezzel biztosítva a környezetek közötti **paritást**. A valóságban azonban a környezetek nem

teljesen azonosak: a termelési környezet nagyobb kapacitást, magasabb rendelkezésre állást és szigorúbb biztonsági beállításokat igényel, míg a fejlesztői környezetben költséghatékonyság és gyors iteráció a cél. A cél nem a teljes egyformaság, hanem az, hogy a különbségek explicit módon, konfigurációban legyenek megfogalmazva, ne pedig a kód eltérő verzióiban.

A tipikus környezeti struktúra:

- **Fejlesztői (dev)**: a fejlesztők saját, izolált környezete, ahol szabadon kísérletezhetnek. Gyakran kisebb erőforrásokkal, olcsóbb adatbázis-példányokkal, és kevésbé szigorú biztonsági szabályokkal rendelkezik. A dev környezetben az a cél, hogy a fejlesztő gyorsan kapjon visszajelzést.
- **Staging**: a termelési környezethez lehető legközelebbi másolat, amelyben az integrációs tesztek, teljesítménytesztek és a kiadás előtti ellenőrzések futnak. Itt a konfiguráció majdnem azonos a termelésivel, de az adatok szintetikusak vagy anonimizáltak, és a külső függőségek (fizetési szolgáltató, SMS-küldő) gyakran sandbox módban működnek.
- **Termelési (prod)**: a tényleges felhasználókat kiszolgáló környezet, ahol a legmagasabb rendelkezésre állás, a szigorú hozzáférés-kezelés és a teljes megfigyelhetőség követelmény.

Az IaC-ben a környezetek elkülönítésére több stratégia létezik:

- **Külön mappák vagy workspace-ök**: például `environments/dev/`, `environments/staging/`, `environments/prod/`, mindegyikben saját `terraform.tfvars` és backend konfiguráció. Ez a legegyszerűbb és legátláthatóbb, de ha sok a közös modul, akkor a DRY elv sérülhet.
- **Közös modulok és környezetspecifikus paraméterek**: a modulok a `modules/` mappában élnek, a környezeti mappák pedig csak a változókat és a háttérkonfigurációt tartalmazzák. Ez a javasolt struktúra nagyobb projekteknél.
- **Külön Git-ágak vagy repositoryk**: egyes csapatok minden környezethez külön ágat vagy repositoryt használnak. Ez egyszerűbbé teszi a hozzáférést, de növeli a karbantartási terhet, és könnyen eltérésekhez vezethet. A modern gyakorlat inkább a közös repositoryban, de külön mappákban való kezelést részesíti előnyben.

A környezetek közötti különbségeket változókban kell tartani, például:

```
# modules/webserver/main.tf (részlet)
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  # ...
}

# environments/dev/terraform.tfvars
ami_id        = "ami-0123456789abcdef0"
instance_type = "t3.micro"

# environments/prod/terraform.tfvars
ami_id        = "ami-0123456789abcdef0"
instance_type = "m5.large"
```

A konfiguráció ilyen elkülönítése biztosítja, hogy a termelési környezet nagyobb kapacitást kap, de a kód ugyanaz marad. A környezetek közötti paritást mérni is lehet: például a staging és a prod környezetben a webszerverek száma, a CPU és memória beállítások, valamint a hálózati szabályok csak a tervezett eltéréseket mutatják, és ezt egy drift-ellenőrző szkript automatikusan összehasonlítja.

24.2.5 Gyakori hibák és anti-minták

Az IaC bevezetésekor számos tipikus hiba fordul elő, amelyek aláásható a verziózás és reprodukálhatóság előnyeit.

1. **Konzolról való módosítás**: bár gyors megoldásnak tűnik, a kézi konfigurációmódosítás driftet okoz, és a következő `apply` felülírhatja vagy elveszítheti azt. Kivétel lehet a vészhelyzeti javítás, de ilyenkor is kötelező a változtatást utólag visszavezetni a kódba, és a driftet megszüntetni.
2. **Titkok a repositoryban**: a fejlesztők néha egyszerűségből beírják a jelszót egy `terraform.tfvars` fájlba, és azt commitolják. Ez súlyos biztonsági incidens, és a titok azonnal kompromittáltnak tekintendő. A megoldás a titokkezelő szolgáltatás vagy környezeti változók használata.
3. **Nem idempotens szkriptek**: ha egy IaC modul nem idempotens – például egy szkript minden futtatáskor új erőforrást hoz létre ahelyett, hogy ellenőrizné a meglétét –, akkor a reprodukálhatóság sérül. A deklaratív eszközök általában idempotensek, de az imperatív részeknél (például `local-exec` provisioner) különösen ügyelni kell.
4. **Túlzott absztrakció**: a modulok túlzottan általánossá tétele (például minden erőforrást egyetlen monolitikus modulba zsúfolni) csökkenti az olvashatóságot és növeli a hibalehetőséget. A jó modulhatár ott húzódik, ahol a felelősség egyértelmű, és a változtatások gyakorisága hasonló. A túl sok modul viszont szétszórja a logikát, és a verziókövetés nehézzé válik.
5. **A drift figyelmen kívül hagyása**: ha a rendszer nem ellenőrzi rendszeresen az állapotot, a drift felhalmozódik, és egy nap a `plan` több száz eltérést mutat, ami áttekinthetetlen. A megoldás a folyamatos drift-detektálás (például tervezett `plan` futtatás a CI/CD pipelineban, vagy GitOps eszköz használata).
6. **Környezetek közötti kódduplikáció**: ha a dev, staging és prod környezetek definícióit külön másolják, akkor a változtatások szinkronizálása nehéz, és a környezetek eltávolodnak egymástól. A javasolt megoldás a közös modulok és a környezetspecifikus paraméterek használata.

24.2.6 Labor: IaC modulstruktúra tervezése

A labor célja, hogy a hallgató megtervezzen egy kisméretű, de valósághű IaC modulstruktúrát egy háromrétegű webalkalmazáshoz, és dokumentálja döntéseit ADR formában. A feladat nem igényel éles felhőkörnyezetet; a hangsúly a tervezésen és az ellenőrzési szempontok megfogalmazásán van.

Kiinduló állapot: Adott egy meglévő, kézzel konfigurált infrastruktúra leírása szövegesen: egy terheléselosztó, két webes virtuális gép, egy adatbázis-példány és egy hálózat alhálózatokkal. A környezetek (dev, staging, prod) jelenleg különböző konfigurációval rendelkeznek, de a pontos beállítások nem dokumentáltak, és a termelési környezetben található eltérések ismeretlenek.

Feladat: Tervezd meg a Terraform modulstruktúrát, amely ezt az infrastruktúrát verziózhatóan és reprodukálhatóan kezeli. A struktúrának legalább a következőket kell tartalmaznia:

- `modules/network` – VPC, alhálózatok, biztonsági csoportok
- `modules/compute` – virtuális gépek, autoscaling csoport
- `modules/database` – adatbázis-példány, mentések

- `environments/dev`, `environments/staging`, `environments/prod` – környezeti mappák `terraform.tfvars` és backend konfigurációval

Elvárt eredmény: Egy Git repositoryban elhelyezett struktúra (vagy annak rajza), amelyben minden modul önállóan használható, a környezeti mappák csak a paraméterekben térnek el egymástól, és a titokkezelés külső szolgáltatásra (például AWS Secrets Managerre) hivatkozik. A dokumentációban szerepelnie kell egy ADR-nek, amely megindokolja a modulhatárok megválasztását és a környezetek elkülönítésének stratégiáját.

Ellenőrzési szempontok:

- A `terraform init` és `terraform plan` parancsok futtathatók bármely környezeti mappából, és a `plan` kimenete csak a tervezett erőforrásokat mutatja.
- A `terraform destroy` visszaállítja az infrastruktúrát, és nem hagy hátra kézzel létrehozott erőforrásokat.
- A repositoryban nem található valós titok (jelszó, API-kulcs), csak hivatkozások.
- A `git log` mutatja a változtatások történetét, és legalább két commit (például egy hálózati és egy számítási modul hozzáadása) elkülönül.
- Egy drift-szimuláció során (például a `modules/compute` változójának módosítása után) a `terraform plan` helyesen jelzi a különbséget.

Mért és várt eredmény: A labor során a hallgató választhat, hogy ténylegesen futtat-e Terraformot egy izolált sandboxban (például lokális Terraform Cloud workspace vagy minikube), vagy csak a struktúrát és a parancsok kimenetét dokumentálja. Ha mérést végez, rögzítenie kell a környezetet, a konfigurációt, az időpontot és a bizonytalanságot. Például: a `terraform apply` futási ideje a dev környezetben 2 perc 10 másodperc volt, míg a várt idő 5 perc alatt. A mért eredményt külön szekcióban kell szerepeltetni, és egyértelműen jelezni, ha a mérés nem történt meg. A nem futtatott állításokat nem szabad ellenőrzött eredményként feltüntetni.

Visszaállítási út: Ha a laborban valódi erőforrások jöttek létre, a `terraform destroy` futtatásával törölhetők. Ha a környezetben kézi módosítás is történt, először a driftet kell megszüntetni (például a kézi változtatás importálásával vagy a `taint` paranccsal), majd a `destroy` végrehajtható. Izolált környezetben a labor után minden erőforrást le kell állítani, hogy költség ne keletkezzen.

24.2.7 Összefoglalás és ellenőrzőlista

Az infrastruktúra mint kód a modern architektúra egyik alappillére: lehetővé teszi, hogy a környezeteket ugyanolyan szigorú verziókezeléssel és minőségbiztosítással kezeljük, mint az alkalmazáskódot. A fejezet fő tanulságai:

- Az IaC verziózása és reprodukálhatósága a környezetek konzisztenciájának és a katasztrófa utáni helyreállításnak az alapja.
- A deklaratív állapotkezelés segít a drift detektálásában, de a driftet rendszeresen ellenőrizni kell.
- A titkokat soha nem tároljuk a verziókezelőben; külső titokkezelő szolgáltatást vagy környezeti változókat használunk.
- A dev, staging és prod környezeteket közös modulokkal, de környezetspecifikus paraméterekkel választjuk szét.
- A tipikus hibák – konzolról való módosítás, titkok a repóban, nem idempotens szkriptek, túlzott absztrakció – elkerülhetők tudatos tervezéssel.

Ellenőrzőlista az IaC bevezetéséhez:

- ☐ Az infrastruktúra definíciója Git repositoryban van, és minden változtatás pull request-en keresztül történik.
- ☐ A modulok felelőssége egyértelmű, és a környezeti mappák csak a paraméterekben térnek el.
- ☐ A titkok kívül vannak a repositoryn, és a hivatkozások környezeti változókból vagy titokkezelőből származnak.
- ☐ A drift detektálás automatizált, és riasztást küld eltérés esetén.
- ☐ A `terraform plan` (vagy az adott eszköz megfelelője) a CI/CD pipeline része, és a kimenetét felülvizsgálják.
- ☐ A környezetek közötti paritást rendszeresen ellenőrzik, és az eltérések dokumentáltak.
- ☐ A katasztrófa-helyreállítási terv tartalmazza az IaC-ből való újraépítés lépéseit, és azt tesztelték.

Önellenőrző kérdés: Mi történik, ha egy fejlesztő kézzel módosít egy biztonsági csoportot a termelési környezetben, és hogyan lehet ezt a driftet észlelni és megszüntetni anélkül, hogy a rendszer leállna? Gondold végig a `terraform plan` kimenetét, a drift-detektálás gyakoriságát, és a visszavezetés folyamatát.

24.3 Blue-green, canary, rolling és feature flag

A biztonságos kiadás célja, hogy egy új verzió bevezetése során minimalizáljuk a hibás működésből fakadó üzleti és technikai kockázatot. A hagyományos „big bang” telepítés – amikor minden példány egyszerre frissül – gyakran vezet hosszú üzemszünethez, nehezen visszaállítható állapothoz és olyan hibákhoz, amelyeket csak éles forgalom alatt fedezünk fel. A biztonságos kiadási minták közös jellemzője, hogy a változást fokozatosan, ellenőrzötten, megfigyelhető módon juttatják el a felhasználókhoz, és mindig fenntartanak egy gyors visszaút lehetőségét. Ebben a fejezetben három alapvető deployment-stratégiát – a rolling, a blue-green és a canary telepítést –, valamint a kiadást a kódtól függetlenítő feature flag technikát tárgyaljuk. A hangsúly a forgalomirányítás, a rollback és a megfigyelés összhangján van, mert ezek nélkül a stratégiák csak üres formák.

24.3.1 1. Rolling deployment: fokozatos példánycsere

A rolling deployment során a rendszer példányait csoportokban frissítjük, miközben a régi és az új verzió átmenetileg párhuzamosan fut. A terheléselosztó fokozatosan veszi ki a régi példányokat a forgalomból, és az újakat helyezi be, ügyelve arra, hogy mindig elegendő egészséges példány szolgálja ki a kéréseket. A folyamat lépései jellemzően a következők:

1. A jelenlegi verzió (v1) példányai kiszolgálják a teljes forgalmat.
2. Az orchestrátor (például Kubernetes Deployment) új replikákat indít a v2 verzióból, miközben a régieket megtartja.
3. A v2 példányok indulás után átmennek a health check és readiness probe ellenőrzésén; ha sikeres, a terheléselosztó elkezdi nekik továbbítani a kéréseket.
4. A régi v1 példányok egy részét leállítjuk, a folyamatot addig ismételjük, amíg minden példány v2-re nem cserélődik.

A rolling előnye, hogy alacsony erőforrásigényű: nincs szükség két teljes környezet párhuzamos fenntartására, csak annyi pluszkapacitásra, amennyit a frissítési ablakban használunk. Emellett a frissítés fokozatos, így ha a v2 hibásan indul, a hiba csak a forgalom egy részét érinti, és a folyamat leállítható.

A rolling deployment hátránya, hogy a régi és az új verzió együtt fut, ami kompatibilitási kihívásokat okozhat. Ha a v2 megváltoztatja az adatbázis sémáját vagy egy belső API szerződését, a v1 példányok – amelyek még futnak – hibásan működhetnek. Ezért a rolling esetén gyakran szükség van úgynevezett „expand–contract" adatbázis-migrációra: először a séma bővítése (új oszlop, új tábla), majd a kódváltozás telepítése, végül a régi elemek eltávolítása. A másik kihívás a rollback sebessége: ha a v2 hibásnak bizonyul, a visszaállítás szintén fokozatos, tehát hosszabb időt vehet igénybe, mint egy blue-green visszakapcsolás.

Tipikus hibák:

- A health check nem ellenőrzi a valódi alkalmazáslogikát, így a v2 példányok „egészségesnek" tűnnek, de hibásan válaszolnak.
- Nincs felső korlát a párhuzamosan futó verziók számára, ami bonyolult session-kezeléshez vezet.
- Az adatbázis-migrációt a teljes deployment előtt futtatják anélkül, hogy a régi verzió továbbra is kompatibilis maradna.

24.3.2 2. Blue-green deployment: két környezet, azonnali visszakapcsolás

A blue-green deployment két, egymástól elkülönített, de közel azonos konfigurációjú környezetet használ: a „kék" a jelenlegi éles verzió, a „zöld" pedig a jövőbeli verzió. Az új verziót a zöld környezetbe telepítik és teljes körűen letesztelik, miközben a kék továbbra is kiszolgálja a teljes éles forgalmat. Amikor a zöld környezet készen áll, a router vagy a terheléselosztó egyetlen lépésben átirányítja a forgalmat a kékből a zöldbe. A régi kék környezet általában még rövid ideig megmarad, hogy szükség esetén azonnal vissza lehessen kapcsolni.

A blue-green előnye a rendkívül gyors rollback: ha a zöld környezetben hibát észlelünk, a forgalom visszaállítása a kék környezetbe egyetlen útválasztó-módosítással megoldható. Ez a képesség különösen értékes olyan rendszereknél, ahol a rendelkezésre állás kritikus, és a hibás verzió azonnali eltávolítása üzleti követelmény. A stratégia emellett teljesen elválasztja a telepítést a kiadástól: a zöld környezetbe nyugodtan lehet telepíteni és füsttesztelni anélkül, hogy a felhasználók bármit észlelnének.

Hátránya a dupla erőforrásigény: a két környezet fenntartása legalább átmenetileg megduplázza a számítási kapacitást és a költségeket. Ráadásul az adatbázis-migráció itt különösen érzékeny, mert a két környezet ugyanazt a perzisztens réteget használhatja, és a séma változásának kompatibilisnek kell lennie mindkét verzióval az átkapcsolás pillanatában. Ha a zöld verzió olyan adatbázis-módosítást igényel, amely a kék verzióval nem kompatibilis, akkor a hirtelen átkapcsolás a régi verziót működésképtelenné teheti – ezért a migrációt itt is expand–contract módon kell végrehajtani, vagy az átkapcsolást rövid leállási ablakkal kell összekötni.

Tipikus hibák:

- A két környezet konfigurációja eltér (pl. más környezeti változók, más erőforráskorlátok), így a zöldben elvégzett teszt nem reprezentatív.
- Az átkapcsolás után a kék környezetet azonnal leállítják, így a gyors rollback lehetősége elveszik.
- A session-affinitás és a sütik kezelése nem megfelelő, ezért a felhasználók egy része a zöldbe, másik része a kékbe kerül, és inkonzisztens állapotot tapasztal.

24.3.3 3. Canary deployment: kis csoporttal kezdeni, mérni, majd dönteni

A canary deployment a blue-green és a rolling előnyeit ötvözi, de a fokozatosságot a forgalom szintjén szabályozza. Az új verziót (a „kanárit") először a felhasználók egy kis, jól körülhatárolt részének teszik elérhetővé – például a forgalom 5%-ának –, miközben a többség továbbra is a régi verziót kapja. A kanárit folyamatosan monitorozzák: hibaarány, válaszidő, rendelkezésre állás, üzleti konverziós mutatók. Ha a metrikák az előre meghatározott küszöbértékeken belül maradnak, a kanári részarányát fokozatosan növelik – 10%, 25%, 50%, végül 100% –, ha viszont a kanári hibásnak bizonyul, a forgalmát azonnal visszavonják, és a régi verzió marad élesben.

A canary előnye, hogy a kockázatot a lehető legkisebb felhasználói körre korlátozza, miközben valódi éles forgalom alatt gyűjt adatokat. A mérési eredmények alapján a döntés adatvezérelt, nem feltételezéseken alapul. A technika jól skálázható: minél nagyobb a rendszer, annál kisebb százalék is elegendő lehet a statisztikailag értékelhető minta eléréséhez.

A canary hátránya a forgalomirányítás bonyolultsága. Szükség van egy olyan komponensre – API gateway, service mesh (például Istio, Linkerd) vagy terheléselosztó –, amely képes a kéréseket a kanári és a stabil verzió között megosztani. A felhasználók kiválasztása történhet véletlenszerűen, cookie vagy fejléc alapján, de ügyelni kell arra, hogy ugyanaz a felhasználó ne ugráljon a verziók között egy munkameneten belül (sticky session). Emellett a kanári kiértékelése időt vesz igénybe: a rövid tesztablak nem ad elegendő statisztikai bizonyosságot, a túl hosszú pedig késlelteti a teljes kiadást. A metrikák kiválasztása kritikus: ha csak a rendszertechnikai mutatókat nézzük (pl. CPU), az üzleti hibák (pl. a kosár nem frissül) rejtve maradhatnak.

Tipikus hibák:

- A kanári részarányát túl gyorsan növelik, még mielőtt elegendő adat gyűlne össze.
- Nem definiálnak előre küszöbértékeket, így a döntés szubjektív, vagy a hibát csak akkor veszik észre, amikor már széles körben elterjedt.
- A kanári és a stabil verzió ugyanazt a háttérrendszert (pl. adatbázist, message queue-t) használja, és a kanári által generált új adatformátum megzavarja a stabil verziót.
- A forgalomirányítás nem konzisztens: egyes API-hívások a kanárihoz, mások a stabilhoz mennek, ami a kliensoldali állapotot inkonzisztenssé teszi.

24.3.4 4. Feature flag: a kód és a kiadás függetlenítése

A feature flag (más néven feature toggle) egy olyan kapcsoló, amely a kódban él, és futásidőben dönti el, hogy egy adott funkció elérhető-e a felhasználó számára. A flag lehet egyszerű boolean, de lehet user-csoport, százalék, földrajzi régió vagy akár egyéni szabály alapján vezérelt is. A feature flag használatának lényege, hogy a kód telepítése (deployment) és a funkció kiadása (release) szétválik: a fejlesztő a kódot a flag mögé rejti, telepíti a produkciós környezetbe, de a funkciót csak akkor aktiválja, amikor az üzleti vagy technikai feltételek adottak.

Ez a szétválasztás számos előnnyel jár:

- **Fokozatos kiadás:** egy funkciót először csak a belső tesztelők, majd a felhasználók 1%-a, végül mindenki számára engedélyezhetünk, anélkül, hogy új deploymentre lenne szükség.
- **Azonnali leállítás:** ha egy funkció hibásan működik, a flag kikapcsolásával a hiba azonnal megszüntethető, még ha a hibás kód a rendszerben is marad.
- **Kísérletezés (A/B teszt):** különböző felhasználói csoportok különböző viselkedést kaphatnak, és az eredményeket mérhetjük.
- **Trunk-alapú fejlesztés:** a fejlesztők hosszú életű feature branch helyett a főágba commitolhatnak, a funkciót flag mögé rejtve, így elkerülhető a merge rémálom.

A feature flag azonban nem ingyen ebéd. Minden flag egy lehetséges technikai adósságot jelent: a kód elágazásai bonyolultabbá válnak, a teszteknek le kell fedniük a flag be- és kikapcsolt állapotát is, és ha a flageket nem távolítják el időben, „flag-hell" alakul ki – a kód tele lesz elavult, senki által nem használt kapcsolókkal. A flag-ek életciklusát ezért explicit módon kell kezelni: minden flaghez tartozzon felelős, lejárati dátum és eltávolítási terv. A flag-ek rendszerét gyakran külön szolgáltatás (pl. LaunchDarkly, Unleash, saját config server) kezeli, ahol a flag-értékek futásidőben változtathatók anélkül, hogy a kódot újra kellene indítani.

A feature flag és a canary deployment jól kiegészíti egymást: a canary a teljes rendszerverziót érinti, míg a flag egy-egy funkciót kapcsol be vagy ki. A legtöbb modern kiadási folyamat a kettőt kombinálja: például a v2 verziót canary módon vezetik be, de az új fizetési folyamatot egy feature flag mögé rejtik, és csak akkor aktiválják, ha a canary metrikák megfelelőek.

Tipikus hibák:

- A flag-ek állapotát a kódban égetik be (pl. környezeti változóban), így a változtatáshoz újra kell telepíteni az alkalmazást – ezzel elveszik a fő előny.
- A flag-ek kombinációját nem tesztelik; bizonyos kombinációk egymást kizáró feltételeket sérthetnek.
- A flag-eket nem távolítják el, és a kód átláthatatlanná válik.

24.3.5 5. Forgalom, rollback és megfigyelés: a biztonságos kiadás hármas alapja

A három deployment-stratégia csak akkor működik, ha a forgalomirányítás, a rollback és a megfigyelés szorosan együttműködik. Ezt a hármas alapot érdemes külön is megvizsgálni.

24.3.6 Forgalomirányítás

A forgalom átirányításának képessége a rendszerarchitektúra azon rétegében kell, hogy megvalósuljon, ahol a kérések belépnek. Ez lehet:

- **Terheléselosztó (L4/L7):** képes a forgalom százalékos megosztására, de gyakran kevésbé rugalmas, mint egy API gateway.
- **API gateway:** magasabb szintű útválasztást tesz lehetővé (pl. path, header, cookie alapján), és integrálható a megfigyelőrendszerrel.
- **Service mesh:** mikroszolgáltatások közötti forgalomirányításra alkalmas, és gyakran beépített canary támogatást nyújt (pl. Istio VirtualService).
- **CDN vagy DNS-alapú routing:** durva vezérlés, inkább globális forgalomelosztásra, mint finomhangolásra.

A választott rétegnek képesnek kell lennie arra, hogy a forgalmat a verziók között megbízhatóan, alacsony késleltetéssel és konzisztensen ossza meg, továbbá hogy a döntési logika (pl. a canary részarány) gyorsan, akár futásidőben is módosítható legyen.

24.3.7 Rollback

A rollback a biztonsági háló. A három stratégia más-más rollback képességet kínál:

- **Blue-green:** azonnali visszakapcsolás a kék környezetbe, amennyiben az még elérhető. Ez a leggyorsabb, de feltételezi, hogy a kék és a zöld közötti adatbázis-séma kompatibilis maradt.

- **Canary:** a kanári forgalmának nullára csökkentése, ami szintén gyors, de csak akkor működik, ha a kanári nem okozott visszafordíthatatlan adatváltozást (pl. rossz tranzakciók).
- **Rolling:** a leglassabb visszaállítás, mert a frissítési folyamatot visszafelé kell lejátszani: a v2 példányokat fokozatosan v1-re cserélik. Ha a hiba súlyos, ez hosszú percekig is eltarthat.

Fontos, hogy a rollbacknek nem szabad csak a kód visszaállítására korlátozódnia. Az adatbázis-migrációkat, a konfigurációváltozásokat és a harmadik féltől származó függőségek verzióit is vissza kell tudni állítani, különben az alkalmazás inkonzisztens állapotba kerül. Ezért a rollback tervnek már a kiadás előtt készen kell lennie, és automatizáltan vagy félautomata módon végrehajthatónak kell lennie.

24.3.8 Megfigyelés

A megfigyelés adja a döntési alapot. A canary és a rolling deployment során folyamatosan gyűjtött metrikáknak legalább a következő területeket kell lefedniük:

- **Rendszertechnikai metrikák:** CPU, memória, hálózat, lemez I/O.
- **Alkalmazásmetrikák:** hibaarány, válaszidő (p50, p95, p99), throughput.
- **Üzleti metrikák:** konverzió, kosárérték, regisztrációk száma, hibaüzenetek.
- **Infrastruktúra-metrikák:** a deployment folyamat állapota, a rollout előrehaladása.

Ezeket a metrikákat előre definiált küszöbértékekkel (SLO, SLI, error budget) kell összevetni. Ha például a p95 válaszidő 300 ms fölé emelkedik, vagy a hibaarány meghaladja az 1%-ot, a rendszer automatikusan leállítja a canary növelését és riasztást küld. A megfigyelőrendszernek valós időben kell működnie, és a döntési pontoknál (pl. a canary 10%-ról 25%-ra emelése előtt) elegendő adattal kell alátámasztania a továbblépést.

A megfigyelés és a forgalomirányítás összekapcsolása lehetővé teszi az automatizált, önvezérelt rollout-ot. Az ilyen rendszerek – gyakran progressive delivery néven emlegetve – automatikusan növelik a kanári arányát, ha a metrikák jók, és visszavonják, ha nem. Ez azonban megköveteli a metrikák pontos, zajmentes mérését és a téves riasztások elkerülését.

24.3.9 6. Ellenőrzött példa: canary release egy webshop katalógusszolgáltatásában

Tegyük fel, hogy egy e-kereskedelmi platform `katalogus-szolgaltatas` nevű mikroszolgáltatása új verzióját szeretnénk kiadni. A szolgáltatás REST API-t biztosít a termékek listázásához és kereséséhez. A jelenlegi verzió v1, az új v2, amely javított keresési algoritmust és egy új ajánló modult vezet be. A csapat a canary deployment mellett dönt, mert a korábbi tapasztalatok szerint a keresési funkcióban a hibák gyakran csak éles adatokkal, valós felhasználói viselkedés mellett jelennek meg.

Környezet és forgalomirányítás: A rendszer Kubernetes-en fut, a bejövő forgalmat egy API gateway kezeli, amely képes a kérések szétválasztására a `canary` fejléc vagy a felhasználóhoz kötött cookie alapján. A szolgáltatásnak két Deployment erőforrása van: `katalogus-v1` (stabil) és `katalogus-v2` (kanári). A gateway a forgalom 5%-át a v2-höz irányítja.

Megfigyelés: A csapat a következő metrikákat figyeli:

- Hibaarány: a 5xx válaszok aránya az összes kéréshez képest; küszöb: $< 0{,}5\%$.
- p95 válaszidő: a keresési végpont válaszidejének 95. percentilise; küszöb: < 400 ms.
- Kosár-konverzió: a keresést követően kosárba helyezés aránya; küszöb: nem csökkenhet 5%-nál többet a v1-hez képest.
- CPU-terhelés: a v2 podok átlagos CPU-felhasználása; küszöb: $< 70\%$.

Folyamat:

1. A v2 Deployment létrehozása 1 replikával, a readiness probe sikeres.
2. A gateway konfigurációjában a canary arány 5%-ra állítása.
3. 30 perces megfigyelési ablak, ahol a metrikákat folyamatosan rögzítik.
4. Ha a metrikák a küszöbértékeken belül vannak, az arány 10%-ra emelése, majd újabb 30 perces ablak.
5. Ez így folytatódik 25%, 50%, 100% lépésekben, minden lépésnél azonos ellenőrzési feltételekkel.
6. Ha bármelyik ablakban a metrikák megsértik a küszöböt, a canary arányát azonnal 0%-ra állítják, és a v1 szolgálja ki a teljes forgalmat. A hibát a csapat elemzi, és a v2 verziót javítják, majd új canary ciklus indul.

Eredmény: Az első kísérletben a p95 válaszidő 420 ms-ra nőtt a v2-nél 5%-os terhelésnél, ami meghaladta a 400 ms-os küszöböt. A canaryt visszavonták, a v2 podot skálázták, és optimalizálták a keresési lekérdezést. A második kísérletben minden metrika a tartományban maradt, és a rollout sikeresen elérte a 100%-ot. A teljes folyamat során a felhasználók kevesebb mint 5%-a találkozott az új verzióval bármelyik időpillanatban, és a hibás első kísérlet nem okozott észlelhető üzleti kiesést.

Ez a példa jól mutatja, hogy a canary deployment nem csupán egy technikai kapcsoló, hanem egy döntési folyamat, amelyben a mérési eredmények határozzák meg a továbblépést.

24.3.10 7. Labor: deployment-stratégia kiválasztása

A labor célja, hogy a fejezetben bemutatott stratégiákat egy konkrét, de kitalált környezetben alkalmazd, és a döntést reprodukálható dokumentumban rögzítsd. A feladat nem igényel valós éles rendszert, elegendő egy megfelelően dokumentált jegyzetfüzet vagy szöveges fájl. A hangsúly a döntési naplón és az ellenőrzési szempontokon van, valamint a mért és várt eredmények elkülönítésén.

Kiinduló állapot: Adott egy fiktív rendszer, a `rendeles-szolgaltatas`, amely egy belső, REST API-t kínáló mikroszolgáltatás. A rendszer jelenleg a v1-es verziót futtatja, és a csapat a v2-es verziót szeretné kiadni. A v2 egy jelentős refaktorálást tartalmaz: az adatbázis-hozzáférési réteget új ORM-re cseréli, de a funkcionális viselkedés nem változik. A rendszer terhelése napi 2 millió kérés, a csúcsidőszak hétköznap 10:00–14:00 között van. A rendelkezésre állási SLO 99,9%, az error budget havi 43 perc. A csapat 6 fejlesztőből áll, nincs külön release engineer. A deployment jelenleg kézi, shell scriptekkel történik, de az Infrastruktúra mint kód és konfiguráció fejezetben már bevezették az Infrastructure as Code alapjait.

A feladat:

1. Válaszd ki a legmegfelelőbb deployment-stratégiát (rolling, blue-green, canary) a fenti kontextusban, és indokold meg a döntést.
2. Készíts egy rövid ADR-t a döntésről, amely tartalmazza a kontextust, a döntést, a mérlegelt alternatívákat, a következményeket és az állapotot (pl. `Elfogadva`).
3. Határozd meg a kiválasztott stratégiához szükséges forgalomirányítási konfigurációt (pl. milyen komponens, milyen szabályok).
4. Definiálj legalább három megfigyelési metrikát, amelyek alapján eldöntöd, hogy a rollout folytatható-e, és add meg a küszöbértékeket.
5. Készíts rollback tervet, beleértve a visszaállítás lépéseit és a szükséges eszközöket.

6. Dokumentáld a várt eredményt (mit vársz, mi történik, ha minden jól megy) és a mért eredményt (hogyan fogod mérni, milyen időablakban). A labor során tényleges mérést nem kell végezni, de a dokumentumnak el kell különítenie a kettőt.

Elvárt eredmény: Egy verziózott, reprodukálható dokumentum (pl. Markdown fájl), amely tartalmazza az ADR-t, a forgalomirányítási tervet, a metrikákat, a rollback tervet és a várt/mért eredmények leírását. A dokumentumnak olyan részletesnek kell lennie, hogy egy másik mérnök a leírás alapján végre tudja hajtani a rollout-ot.

Ellenőrzési szempontok:

- Az ADR tartalmazza a döntés okát és az elvetett alternatívákat.
- A forgalomirányítási terv meghatározza a konkrét réteget (pl. API gateway) és a szabályok leírását.
- A metrikák mérhetők, mértékegységgel és mérési ablakkal rendelkeznek (pl. hibaarány %-ban, 5 perces ablak).
- A rollback terv lépésenkénti, és kitér az adatbázis-migráció visszaállítására is.
- A várt és a mért eredmény egyértelműen elkülönül.
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy éles rendszerre vonatkozó azonosítót.

Visszaállítási út: Mivel a labor nem éles környezetet használ, a visszaállítás a dokumentum törlését vagy új verziójának létrehozását jelenti. Ha a gyakorlat során valamilyen eszközben (pl. helyi Kubernetes klaszterben) ténylegesen kipróbálod a konfigurációt, a labor végén a klasztert vissza kell állítani az eredeti állapotába, például a teszt namespace törlésével.

24.3.11 Ellenőrzőlista a fejezethez

- ☐ Ismerem a rolling, blue-green és canary deployment alapvető működését, előnyeit és hátrányait.
- ☐ Tudom, hogy a feature flag hogyan függetleníti a kód telepítését a funkció kiadásától.
- ☐ Meg tudom magyarázni, miért fontos a forgalomirányítás, a rollback és a megfigyelés együttműködése.
- ☐ Felismerem a tipikus hibákat, például a nem megfelelő health checket, az expand–contract migráció hiányát vagy a flag-ek elavulását.
- ☐ Képes vagyok egy adott kontextusban kiválasztani a megfelelő deployment-stratégiát, és ADR-ben dokumentálni a döntést.

24.3.12 Önellenőrző kérdés

Egy pénzügyi rendszerben a csapat a blue-green deploymentet használja. Az új verzió bevezetésekor az adatbázis-séma egy mezőt kötelezővé tesz, amely a régi verzió számára ismeretlen. Mi a hiba ebben a megközelítésben, és hogyan lehetne helyesen kezelni a migrációt úgy, hogy a blue-green átkapcsolás biztonságos maradjon?

A válasz: A séma változása nem kompatibilis a régi verzióval, így ha a forgalom hirtelen átkapcsol a zöldbe, a kék verzió – amely még fut – hibákat okozhat, ha vissza kell kapcsolni. A helyes megoldás az expand–contract minta: először a séma bővítése (a mező opcionálissá tétele, vagy alapértelmezett érték bevezetése), majd a v2 telepítése, végül a mező kötelezővé tétele egy későbbi, külön migrációs lépésben. Így mindkét verzió együtt tud élni az átmeneti időszakban, és a rollback nem okoz adatvesztést vagy hibát.

Ez a fejezet lefektette a biztonságos kiadási minták alapjait. A következő egységben ezekre építve a release engineering és a rollback automatizálásának mélyebb technikáit vizsgáljuk meg.

24.4 Release engineering és rollback

24.4.1 Release engineering: a kiadás mint mérnöki folyamat

A modern szoftverfejlesztésben a kiadás (release) sokáig a fejlesztői életciklus egyik legkockázatosabb, legkevésbé automatizált lépése volt. A release engineering ezt a lépést emeli ki az alkalmi, kézi műveletek világából, és ugyanazzal a mérnöki fegyelemmel kezeli, mint a forráskódot, a tesztelést vagy az infrastruktúrát: megtervezi, automatizálja, verziózza, méri és folyamatosan fejleszti. A cél nem az, hogy a release gyorsabb legyen mindenáron, hanem hogy **megbízható, ismételhető, auditálható és visszafordítható** legyen – akkor is, ha a csapat, a rendszer vagy a külső környezet változik.

Ez a fejezet azt vizsgálja, miért érdemes a kiadást mérnöki folyamatként kezelni, milyen elemekből áll egy release folyamat, hogyan kapcsolódik a verziózáshoz, a változásnaplóhoz és a jóváhagyáshoz, valamint miként tervezünk rollbacket, forward fixet és adatmigrációt. A korábbi alfejezetben (Blue-green, canary, rolling és feature flag) bemutatott deployment stratégiákra – blue-green, canary, rolling, feature flag – itt csak annyiban hivatkozunk, amennyire a release tervezéséhez szükségesek; nem ismételjük meg azokat. A következő alfejezet (Tesztpiramis, teszt-trofea és QA szerep) a tesztelési stratégiára épít majd, ezért itt a tesztelés részleteire nem térünk ki.

24.4.2 Miért mérnöki folyamat a kiadás?

A kiadás akkor válik mérnöki folyamattá, ha a következő tulajdonságokkal rendelkezik:

1. **Definiált lépések és felelősök.** Nem „valaki kiadja”, hanem egy pipeline hajtja végre a build, teszt, jóváhagyás, deploy, monitorozás és szükség esetén visszaállítás lépéseit. Minden lépéshez tartozik felelős, bemenet, kimenet és ellenőrzési feltétel.
2. **Verziózott, megváltoztathatatlan artefaktumok.** A release nem forráskód, hanem egy beazonosítható, aláírt, tárolt buildtermék (például konténerkép, tarball, bináris), amelyet nem módosítunk a pipeline-ban. Ha egy release-t telepítünk, ugyanazt az artefaktumot kapja a tesztkörnyezet, a staging és a production.
3. **Mérhető kimenet.** A release sikerét nem szubjektív benyomás, hanem előre definiált mérőszámok alapján értékeljük. Ilyenek lehetnek a DORA keretrendszerből ismert *deployment frequency* (kiadások gyakorisága, darab/hét), *lead time for changes* (a commit és a production deploy közötti idő, óra vagy nap), *change failure rate* (a hibás release-ek aránya, százalék) és *time to restore service* (a hibából való helyreállítás ideje, perc vagy óra). Az értékeket mindig mértékegységgel és mérési ablakkal adjuk meg.
4. **Visszafordíthatóság.** Minden release-hez tartozik egy előre tesztelt rollback terv. A rollback nem a kudarc beismerése, hanem a rendszer üzemeltetésének normál része. A rollback tervezése nélkül a release kockázata vállalhatatlan.
5. **Auditálhatóság.** A release folyamat minden döntése, jóváhagyása, lépése és eredménye visszakereshető. A release engineering ezért szorosan kapcsolódik az ADR-ekhez, a pipeline naplókhoz és a change management rendszerekhez.

A kiadás mérnöki kezelése nem jelenti azt, hogy minden release-hez formális jóváhagyási bizottság kell. A DORA kutatásai és a gyakorlat azt mutatják, hogy a magas teljesítményű csapatok gyakran **automatizált jóváhagyást** használnak: ha a pipeline minden kapuja zöld, a release automatikusan megy a production környezetbe. A formalitás mértéke a szervezet kockázattűrésétől és szabályozási környezetétől függ. A lényeg az, hogy a jóváhagyás szabályai előre rögzítettek, és a folyamat mindenki számára ismert.

24.4.3 A release folyamat elemei

Egy release folyamatot úgy érdemes megtervezni, hogy a lépések egymásra épüljenek, minden lépésnek legyen bemeneti feltétele, és a hibát korán jelezze. A következő ábra egy általános release pipeline-t mutat be, a telepítési stratégia konkrét választása nélkül:

```
flowchart LR
    A[Build] --> B[Teszt]
    B --> C{Minőségi kapu}
    C -- megfelelt --> D[Staging deploy]
    C -- nem felelt meg --> X[Leállítás]
    D --> E[Staging teszt / SLO]
    E --> F{Jóváhagyás}
    F -- automatikus vagy kézi --> G[Production deploy]
    G --> H[Monitorozás]
    H --> I{SLO teljesül?}
    I -- igen --> J[Release lezárva]
    I -- nem --> K[Rollback / forward fix]
```

A diagramon látható, hogy a release nem ér véget a production deploy-jal: a monitorozás és az SLO-ellenőrzés a folyamat része. Ha a rendszer a kiadás után megszegi a minőségi mutatókat – például a hibaráta 5% fölé emelkedik, vagy a p95 válaszidő 500 ms fölé nő –, a release automatikusan visszafordítható vagy leállítható.

A release folyamatot célszerű egy **release pipeline**-ként megvalósítani, amely a CI/CD rendszer része. A pipeline lépései jellemzően a következők:

- **Build és artefaktum-előállítás:** a forráskódból buildtermék készül, amelyet verzióval, commit azonosítóval és build időponttal látunk el. Az artefaktumot egy belső registry-ben tároljuk, és a későbbiekben nem módosítjuk.
- **Automatikus tesztek és minőségi kapuk:** unit, integrációs, contract, biztonsági és teljesítménytesztek. A kapuk meghatározzák, hogy milyen feltételek mellett léphet tovább a release. Például: a kritikus sebezhetőségek száma 0, a tesztlefedettség nem csökkenhet 80% alá.
- **Staging deploy és érvényesítés:** a release egy produkciószerű környezetbe kerül. Itt futtatjuk a füsttesztet, az SLO-ellenőrzést és a szükséges manuális vagy automatikus jóváhagyást.
- **Production deploy:** a választott deployment stratégiával (blue-green, canary, rolling) telepítjük a release-t. A stratégiát a release tervezésekor rögzítjük, és a rollback terv is ehhez igazodik.
- **Monitorozás és visszacsatolás:** a release után figyeljük a rendszer metrikáit, a hibanaplókat és a felhasználói viselkedést. Ha bármelyik feltétel megszegi az előre rögzített határértéket, a release-t vissza kell állítani vagy forward fixet kell kiadni.

Fontos, hogy a release folyamat **nem lineáris, hanem ciklikus**: a visszajelzések alapján a folyamatot folyamatosan finomítjuk. A release engineering tehát nem egyszeri tervezés, hanem folyamatos tevékenység, mint a tesztelés vagy a monitoring.

24.4.4 Verziózás, változásnapló és jóváhagyás

A release engineering három pillére a verziózás, a változásnapló és a jóváhagyás. Ezek biztosítják, hogy a release-ek egyértelműen azonosíthatók, tartalmuk visszakereshető, és a kiadás felelőssége tisztázott.

24.4.4.1 Verziózás

A verziózás célja, hogy minden release egyedi, rendezhető és jelentéssel bíró azonosítót kapjon. A leggyakoribb séma a **szemantikus verziózás** (SemVer): `MAJOR.MINOR.PATCH`, ahol

- a **MAJOR** verzió visszafelé nem kompatibilis változást jelez,
- a **MINOR** visszafelé kompatibilis új funkciót,
- a **PATCH** visszafelé kompatibilis hibajavítást.

Például a `2.4.1` verzió a `2.4.0` hibajavítása, a `2.5.0` új funkciókat ad hozzá, a `3.0.0` pedig olyan változást jelöl, amely megtörheti a korábbi kliensek működését. A verziószámot a build pipeline automatikusan is generálhatja, de a szemantikus jelentést általában a csapat dönti el a commit üzenetek vagy a kiadási jegyzék alapján. A verziószám mellett érdemes rögzíteni a **commit SHA-t**, a build időpontot és a build környezet azonosítóját is, így a release pontosan reprodukálható.

A verziókat **git tag**-gel vagy más verziókezelő mechanizmussal rögzítjük. A tag név legyen konzisztens, például `v2.4.1`. A tag és az artefaktum összekapcsolása biztosítja, hogy minden release pontosan megfeleltethető egy forráskód-állapotnak.

24.4.4.2 Változásnapló

A **változásnapló** (changelog) a release-ek emberek által olvasható leírása. Nem a commit lista, hanem annak a felhasználó, üzemeltető és fejlesztő szempontjából releváns kivonata: mit javítottunk, mit adtunk hozzá, mi változott, mi szűnt meg. A jó változásnapló a következőket tartalmazza:

- a release verziója és dátuma,
- a változás kategóriája (hozzáadva, módosítva, javítva, eltávolítva, biztonsági javítás),
- rövid leírás, lehetőleg hivatkozással az issue-ra vagy a pull requestre,
- ha van, a breaking change jelzése és a migrációs teendő.

A változásnapló konvencióit érdemes szervezeti szinten rögzíteni. Az egyik elterjedt minta a **Keep a Changelog** formátum, a másik a **Conventional Commits** alapú automatikus generálás. A lényeg, hogy a változásnapló **együtt verzióződjon a forráskóddal**, és minden release-hez tartozzon egy bejegyzés. A release jegyzék és a változásnapló közötti eltérés hibát jelez, ezért érdemes a pipeline-ban ellenőrizni.

24.4.4.3 Jóváhagyás

A jóváhagyás a release folyamat egyik legkényesebb pontja. A kérdés az, hogy ki, milyen feltételek alapján, milyen bizonyítékok birtokában engedélyezheti egy release production környezetbe juttatását. A jóváhagyási modell három alaptípust különböztethetünk meg:

1. **Automatikus jóváhagyás:** a pipeline minden kapuja automatikus. Ha a tesztek zöldek, a biztonsági scan nem talál kritikus hibát, és a staging SLO-ellenőrzés megfelelt, a release automatikusan megy productionbe. Ez a modell gyors, de csak akkor biztonságos, ha a tesztek és a monitoring erősek, és a rollback gyorsan végrehajtható.
2. **Kézi jóváhagyás:** egy vagy több személy explicit engedélyét igényli. Ez lehet a csapatvezető, a termékmenedzser vagy egy change advisory board (CAB). A kézi jóváhagyás akkor indokolt, ha a release nagy kockázatú, szabályozott környezetben fut, vagy ha a csapat még nem elég érett a teljes automatizáláshoz.
3. **Hibrid modell:** bizonyos release-ek automatikusan mehetnek (például patch verziók, feature flag mögötti változások), mások kézi jóváhagyást igényelnek (például major verzió, adatbázis-migrációval járó változás, külső API-t érintő módosítás). A szabályokat előre rögzítik, és a pipeline ennek megfelelően működik.

A jóváhagyáshoz tartozik egy **bizonyítékcsomag**, amelyet a release folyamat összegyűjt: a tesztfutások eredménye, a biztonsági scan jelentés, a staging környezet metrikái, a változásnapló, az érintett szolgáltatások listája, valamint a rollback terv. A jóváhagyó ennek alapján dönt. A döntést **ADR-ben** érdemes rögzíteni, különösen akkor, ha a szokásos szabályoktól eltérő kiadásról van szó. Az ADR ebben a kontextusban a kiadási döntés nyomon követésére szolgál, nem helyettesíti a release pipeline naplóit.

Példa ADR egy rendkívüli kiadás jóváhagyására:

```
ADR-2025-014: Rendkívüli production release jóváhagyása
Állapot: elfogadva
Kontextus: A rendelési szolgáltatás v2.4.0 verziójában egy ritka, de
kritikus hiba jelent meg, amely a fizetési visszaigazolás elvesztését okozza.
A hiba csak a hónap végi terhelésnél jelentkezik, ezért a normál release
ablakot nem várhatjuk meg.
Döntés: A v2.4.1 patch verziót a szokásos heti release ablakon kívül,
csütörtökön 14:00-kor kiadjuk, canary deployment stratégiával. A jóváhagyást a
platform csapat vezetője adja, a tesztkörnyezetben lefuttatott hibajavító
teszt és a staging SLO-ellenőrzés alapján.
Alternatívák:
- Rollback a v2.3.0-ra: gyors, de adatmigrációs kockázatot hordoz.
- Várni a következő heti release-re: a hiba továbbra is fennáll.
Következmények: A release a normál folyamatnál gyorsabb, ezért a
tesztkörnyezetben a teljes regressziós csomagot nem futtattuk le. A
production canary 10 percig fut, és 0,5%-nál nagyobb hibaráta vagy 200 ms-nál
nagyobb p95 válaszidő-romlás esetén automatikusan visszaállunk v2.4.0-ra.
```

Az ADR-ből látható, hogy a jóváhagyás nem formális pecsét, hanem a kontextus, a döntés, az alternatívák és a következmények rögzítése. A release engineering így biztosítja, hogy a kiadási döntések később visszakereshetők és tanulhatók legyenek.

24.4.5 Rollback, forward fix és adatmigráció

A release folyamat legkritikusabb része a **hiba esetén történő reagálás**. Két alapvető stratégia létezik: a **rollback** (visszaállítás korábbi verzióra) és a **forward fix** (új verzió kiadása a hiba javításával). A kettő nem egymást kizáró, hanem egymást kiegészítő eszköz; a választást a hiba jellege, az adatváltozás és a rendelkezésre állási cél határozza meg.

24.4.5.1 Rollback

A rollback azt jelenti, hogy a rendszert visszaállítjuk egy korábbi, ismerten működő verzióra. A rollback feltétele, hogy a korábbi verzió artefaktuma elérhető, a rendszer képes az azonnali visszaállásra, és az adatbázis-séma kompatibilis marad. A rollback ideje alatt a rendszer általában nem szolgál ki, vagy csak korlátozottan, ezért a rollback a rendelkezésre állási SLO szempontjából költséges lehet.

A rollback tervezésénél a következőket kell rögzíteni:

- **Trigger:** milyen feltételek esetén indítunk rollbacket? Például: a production hibaráta 5% fölé emelkedik, a p99 válaszidő meghaladja az 1 másodpercet, vagy a kritikus üzleti funkció elérhetetlenné válik.
- **Lépések:** a rollback konkrét műveleti sorrendje. Például: a canary forgalmának elterelése, a production deploy visszaállítása a korábbi verzióra, a cache ürítése, az adatbázis-séma visszaállítása (ha szükséges).
- **Felelős:** ki hajtja végre, ki hagyja jóvá, ki kommunikál.
- **Időkorlát:** mennyi idő áll rendelkezésre a rollback végrehajtására? Ez az SLO-ból és a hiba üzleti hatásából adódik.
- **Tesztelés:** a rollback folyamatot rendszeresen gyakorolni kell, nem elég egyszer kipróbálni.

A rollback nem mindig lehetséges vagy ésszerű. Ha a release új adatbázis-sémát vezet be, és a régi kód nem tudja kezelni az új sémát, a visszaállításhoz a sémát is vissza kell állítani, ami adatvesztést okozhat. Ezért az **adatbázis-migrációt** a rollback tervezésével együtt kell megtervezni.

24.4.5.2 Forward fix

A forward fix azt jelenti, hogy a hibát egy új release-szel javítjuk, és nem állítjuk vissza a korábbi verziót. A forward fix előnye, hogy megőrzi a release által bevezetett adatváltozásokat, és elkerüli a rollback során fellépő adatkonzisztencia-problémákat. Hátránya viszont, hogy tovább tart, mint a rollback: a hiba javítása, a tesztelés és az új release kiadása időt vesz igénybe. A forward fix akkor jó választás, ha

- a hiba nem okoz azonnali, súlyos üzleti kiesést,
- a rendszer állapota a release után is konzisztens,
- a rollback költséges vagy kockázatos lenne.

A gyakorlatban a két stratégia kombinációja a leggyakoribb: azonnali rollback a súlyos, adatvesztéssel vagy szolgáltatáskieséssel járó hibák esetén, majd a hiba javítása után egy forward fix kiadása. A rollback tehát nem a probléma megoldása, hanem a rendszer stabilizálása, amíg a végleges javítás elkészül.

24.4.5.3 Adatmigráció és rollback

Az adatmigráció a release folyamat egyik legnehezebb része, mert a kód és az adatséma változását össze kell hangolni. A leggyakoribb hiba az, hogy a release új sémát vezet be, de a régi kód nem tudja kezelni, így a rollback lehetetlenné válik. Ennek elkerülésére a következő elveket érdemes követni:

1. **Válaszd szét a sémaváltozást a kódváltozástól.** Először a séma bővítése (additív, visszafelé kompatibilis változás), majd a kód telepítése, végül a régi séma eltávolítása. Ez az ún. **expand-contract** minta. A sémabővítés során a régi kód továbbra is működik, mert az új oszlopok alapértelmezett értéket kapnak, vagy nullable-k.
2. **A séma visszaállítása nem adatvesztés.** Ha a rollback során a sémát is vissza kell állítani, gondoskodni kell arról, hogy az új séma által bevezetett adatok ne vesszenek el. Ez lehetetlen, ha a séma visszaállítása oszlopok törlését jelenti. Ezért a sémaeltávolítást csak akkor szabad elvégezni, ha az adott verzió már stabil, és a rollback lehetőségét feladtuk.
3. **Használj adatbázis-migrációs eszközöket.** A migrációk verziózottak, visszafelé kompatibilis lépésekből állnak, és a pipeline-ban automatikusan futnak. A migrációkhoz tartozik egy **rollback szkript** is, de ez nem mindig garantálja az adat integritását; csak akkor használható, ha a migráció visszafordítható.
4. **A blue-green deployment adatbáziskihívásai.** A blue-green stratégiánál a két környezet párhuzamosan fut, de közös adatbázist használ. Ha az új verzió új sémát vár, a régi verzió nem fog működni. Ezért a séma változásait úgy kell megtervezni, hogy a régi és az új kód is képes legyen az adott sémával dolgozni, amíg az átállás be nem fejeződik. Ez gyakran **kétfázisú migrációt** igényel: először a séma bővítése, majd a kód telepítése, végül a séma szűkítése.

A rollback tervezésénél tehát nem elég a deployment visszaállítása; az adatbázis-séma és a külső rendszerek állapotát is figyelembe kell venni. A release tervnek tartalmaznia kell az adatmigráció verzióját, a visszaállítás feltételeit és a lehetséges adatkonzisztencia-problémákat.

24.4.5.4 Rollback és forward fix mérése

A rollback és a forward fix hatékonyságát mérni kell. A legfontosabb mérőszámok:

- **Change failure rate:** a hibás release-ek aránya az összes release-hez képest, százalékban, például 10% alatt tartva.
- **Mean time to restore (MTTR):** a hiba észlelésétől a normál működés visszaállításáig eltelt idő, percben vagy órában. A rollback célja az MTTR minimalizálása.
- **Rollback success rate:** a rollback kísérletek hány százaléka sikeres első próbálkozásra. Ha ez alacsony, a rollback folyamatot gyakorolni kell.
- **Data loss incidents:** az adatvesztéssel járó release-ek száma, adott időablakban, például negyedévente. Cél: 0.

Ezeket a mérőszámokat a release folyamat részeként gyűjtjük, és az SLO-kkal összhangban értékeljük. Az error budget fogalma itt is alkalmazható: ha a rendszer rendelkezésre állási SLO-ja 99,9% egy hónapban, akkor a havi megengedett hibaidő 43,2 perc. A release-ek okozta állásidő ebbe beleszámít, ezért a rollback tervezése közvetlenül hat az error budget felhasználására.

24.4.6 Tipikus hibák és helyes megoldások

A release engineering gyakorlatában visszatérő hibák a következők:

1. **Nincs rollback terv, vagy csak „majd visszaállítjuk” szinten létezik.** A rollback terv nem egyetlen parancs, hanem dokumentált, tesztelt, felelősökkel ellátott folyamat. Helyes megoldás: a rollback tervet a release tervezésekor elkészítjük, a pipeline-ban automatikus triggerrel, és rendszeresen gyakoroljuk (például chaos engineering játéknapokon).

2. **A rollback nincs tesztelve.** A rollback terv papíron jól néz ki, de éles helyzetben kiderül, hogy a séma visszaállítása nem működik, vagy a cache-ben maradt adatok tönkreteszik a régi verziót. Helyes megoldás: a staging környezetben a rollbacket éles adatokkal szimuláljuk, és a folyamatot legalább negyedévente teszteljük.

3. **Adatmigráció és kód nincs összehangolva.** A release új sémát vezet be, de a régi kód nem tudja kezelni, így a rollback lehetetlen. Helyes megoldás: expand-contract migráció, a séma és a kód telepítésének szétválasztása, és a rollback tervben a séma visszaállításának explicit kezelése.

4. **Nincs verziózott artefaktum, a production deploy közvetlenül branch-ből történik.** Ez azt jelenti, hogy a release nem azonosítható egyértelműen, és a rollback nem tudja, melyik verzióra álljon vissza. Helyes megoldás: minden release egy verziózott, aláírt, tárolt artefaktumból indul, és a deployment csak ezt az artefaktumot használja.

5. **A jóváhagyás nincs nyomon követve.** A release-t szóban engedélyezték, nincs írásos döntés, nincs bizonyíték. Helyes megoldás: a jóváhagyást rögzítjük ADR-ben vagy a release pipeline naplójában, a döntéshez tartozó bizonyítékokkal együtt.

6. **A forward fix elhalasztása.** A csapat rollbacket hajt végre, de a hibát nem javítja azonnal, így a rendszer hosszú ideig egy régi, ismert hibával fut. Helyes megoldás: a rollback után a hibát a lehető leggyorsabban javítani kell, és a javítás kiadását ugyanazzal a release folyamattal kell kezelni.

24.4.7 Labor: release- és rollback-terv készítése

A labor célja, hogy a fejezetben tanultakat egy fiktív, de valósághű rendszeren alkalmazd. A feladat egy **release- és rollback-terv** elkészítése, reprodukálható dokumentumban vagy jegyzetfüzetben.

Kiinduló állapot: A „Rendelési Szolgáltatás” (Order Service) egy belsőleg fejlesztett mikroszolgáltatás, amely a rendelések feldolgozásáért felelős. Jelenleg a v2.3.0 verzió fut production környezetben. A rendszer PostgreSQL adatbázist használ, és REST API-t biztosít a többi szolgáltatás számára. A deployment jelenleg blue-green stratégiát használ, a production környezet Kubernetesben fut.

A csapat a v2.4.0 verziót készül kiadni, amely a következő változásokat tartalmazza:

- új REST végpont a rendelés törlésére (`DELETE /orders/{id}`),
- egy új adatbázis-oszlop a rendelések táblában: `deleted_at TIMESTAMP  NULL`,
- egy hibajavítás, amely a rendelés státuszának frissítésekor előforduló versenyhelyzetet szünteti meg.

A release a hónap végi terhelési csúcs előtt egy héttel esedékes. A szervezeti szabályzat szerint a production release-hez a platform csapat vezetőjének jóváhagyása szükséges.

Elvárt eredmény: Készíts egy dokumentumot, amely a következőket tartalmazza:

1. **Verzió és változásnapló:** a release verziója, a változások kategóriákba sorolva, a breaking change jelzése (ha van), és a migrációs teendők.
2. **Jóváhagyási csomag:** milyen bizonyítékok szükségesek a jóváhagyáshoz, ki a jóváhagyó, és milyen feltételek mellett történik a jóváhagyás.
3. **Deployment stratégia:** a választott stratégia (például blue-green), a lépések sorrendje, a forgalomátirányítás módja, az ideiglenes erőforrások kezelése.

4. **Adatmigráció:** az új `deleted_at` oszlop bevezetésének módja (expand-contract), a migrációs szkript verziója, a visszaállítás lehetősége.
5. **Rollback terv:** a rollback triggere, a lépések, a felelős, az időzítés, és a séma visszaállításának kezelése.
6. **Forward fix terv:** mi történjen, ha a hiba nem okoz azonnali kiesést, de a release hibásnak bizonyul?
7. **Kommunikáció:** ki értesül a release-ről, milyen csatornán, és mi a teendő, ha a rollback szükséges.
8. **Mért és várt eredmény:** a release során gyűjtött mérőszámok (hibaráta, válaszidő, rendelkezésre állás) és az előre meghatározott elfogadási küszöbök. A mért értékeket a release után rögzítsd, és külön jelöld, melyik az előre várt és melyik a ténylegesen mért.

Ellenőrzési szempontok:

- A terv minden lépése konkrét, végrehajtható, és felelőst rendel hozzá.
- A rollback terv tartalmazza a triggert, a lépéseket, az időtartamot és a séma visszaállításának módját.
- Az adatmigráció visszafelé kompatibilis, és a rollback során nem okoz adatvesztést.
- A dokumentum verziózott, és a döntésekhez ADR hivatkozás tartozik.
- A mért és a várt eredmény egyértelműen elkülönül, a mérőszámok mértékegységgel és mérési ablakkal szerepelnek.

Visszaállítási út: Ha a labor során bármilyen módosítást végeznél egy tesztkörnyezetben, a labor végén állítsd vissza a kiinduló állapotot. A labor dokumentum és a döntési napló megtartható, de a tesztkörnyezetben lévő adatok és konfigurációk nem tartalmazhatnak valós hitelesítő adatot.

24.4.8 Példa release- és rollback-terv vázlat

Az alábbi vázlat a labor elvárt eredményének egy lehetséges szerkezetét mutatja. A konkrét értékeket, döntéseket és mérőszámokat a labor során neked kell kidolgoznod.

```
Release terv - Rendelési Szolgáltatás v2.4.0
Verzió: v2.4.0
Dátum: 2025.06.15.
Felelős: [csapattag neve]
Jóváhagyó: platform csapat vezető
Státusz: tervezett

1. Változásnapló
   - Hozzáadva: DELETE /orders/{id} végpont.
   - Módosítva: rendelés státusz frissítés versenyhelyzet javítása.
   - Adatbázis: új `deleted_at` oszlop (TIMESTAMP NULL).
   - Breaking change: nincs, a régi kliensek továbbra is működnek.

2. Jóváhagyási csomag
   - Automatikus tesztek: unit, integrációs, contract (100% sikeres).
   - Biztonsági scan: 0 kritikus, 0 magas sebezhetőség.
   - Staging SLO: p95 válaszidő < 300 ms, hibaráta < 0,1% 24 órán át.
   - Érintett komponensek: order-service, order-db.
   - Döntés: ADR-2025-015, elfogadva.

3. Deployment stratégia
   - Blue-green: új zöld környezet a v2.4.0-val.
   - Forgalomátirányítás: Kubernetes ingress súlyozással, 5 perc alatt
     fokozatosan 100%-ra.
   - A kék környezet 1 órán át készenlétben marad.
```

```
4. Adatmigráció
   - Expand: `ALTER TABLE orders ADD COLUMN deleted_at TIMESTAMP NULL;`
     (visszafelé kompatibilis, a régi kód nem látja).
   - Contract: a v2.4.0 kód már használja az oszlopot.
   - A séma eltávolítása csak a következő release-ben, ha stabil.

5. Rollback terv
   - Trigger: production hibaráta > 1% 5 percen át, vagy p99 > 1 s.
   - Lépések: forgalom visszaállítása a kék környezetre, a zöld környezet
     törlése, a séma visszaállítása nem szükséges (additív migráció).
   - Felelős: [ügyeletes mérnök].
   - Időtartam: < 5 perc.
   - Tesztelés: staging környezetben 2025.06.10-én sikeresen végrehajtva.

6. Forward fix terv
   - Ha a hibaráta 1% alatt marad, de funkcionális hiba jelentkezik, a
     javítás v2.4.1 verzióban, a következő release ablakban.

7. Kommunikáció
   - Release előtt: termék csapat, support, ügyfélszolgálat email.
   - Rollback esetén: azonnali Slack értesítés, incident channel.

8. Mérés és elfogadási küszöbök
   - Várt: hibaráta < 0,5%, p95 válaszidő < 300 ms, rendelkezésre állás
     99,9% a release utáni 24 órában.
   - Mért: [release után rögzítendő, külön oszlopban].
```

A labor során a fenti vázlatot kell kitöltened, a saját kontextusod és döntéseid szerint. A mért értékeket csak a release után lehet beírni, ezért a dokumentumban a **várt** és a **mért** szakaszokat egyértelműen el kell különíteni. A labor célja a folyamat gyakorlása, nem pedig egy éles release végrehajtása.

24.4.9 Ellenőrzőlista a release mérnöki folyamathoz

A fejezet végén a következő ellenőrzőlista segít a release folyamat felülvizsgálatában:

- ☐ A release rendelkezik verzióval, változásnaplóval és commit SHA-val.
- ☐ Az artefaktum verziózott, aláírt, és nem módosul a pipeline után.
- ☐ A release pipeline tartalmazza a build, teszt, staging, jóváhagyás, deploy, monitorozás és rollback lépéseket.
- ☐ A jóváhagyási feltételek előre rögzítettek, a döntés ADR-ben vagy pipeline naplóban visszakereshető.
- ☐ A rollback terv tartalmazza a triggert, lépéseket, felelőst, időtartamot és a séma visszaállításának módját.
- ☐ Az adatmigráció visszafelé kompatibilis, és a rollback nem okoz adatvesztést.
- ☐ A rollback folyamatot a staging környezetben tesztelték, és a tesztidőpont rögzítve van.
- ☐ A release mérőszámait mértékegységgel és mérési ablakkal definiálták, a várt és a mért értékek elkülönülnek.
- ☐ A kommunikációs terv tartalmazza a release előtti és a rollback esetén szükséges értesítéseket.
- ☐ A release terv verziózott, és a dokumentumhoz ADR hivatkozás tartozik.

24.4.10 Önellenőrző kérdés

Egy csapat a release során új adatbázis-oszlopot vezet be, de a rollback során a régi kód nem tudja kezelni az új sémát. Milyen tervezési hibát követtek el, és hogyan lehetett volna elkerülni? Válaszodban hivatkozz az expand-contract mintára, és magyarázd el, miért nem elég a kód visszaállítása a séma visszaállítása nélkül.

24.4.11 Összefoglalás

A release engineering a kiadást nem egyetlen eseményként, hanem egy megtervezett, automatizált, mért és folyamatosan javított folyamatként kezeli. A verziózás, a változásnapló és a jóváhagyás biztosítják az azonosíthatóságot és az auditálhatóságot. A rollback, a forward fix és az adatmigráció tervezése pedig biztosítja, hogy a release hibája esetén a rendszer gyorsan és adatvesztés nélkül visszaállítható legyen. A következő alfejezet (Tesztpiramis, teszt-trofea és QA szerep) a tesztelési stratégiára épít, és azt vizsgálja, hogyan támogatja a tesztpiramis, a contract tesztelés és a teszt-trofea a release folyamat minőségét.

25 Tesztelési stratégia és minőségbiztosítás

25.1 Tesztpiramis, teszt-trofea és QA szerep

A tesztelési stratégia nem a projekt végén jön: az architektúra része. A minőségbiztosítás nem egy kapu, hanem folyamatos tevékenység, amely a tesztelési szintek tudatos megválasztásával kezdődik. Ebben az alfejezetben a tesztpiramis és a teszt-trofea modelleket, valamint a QA szerepét és folyamatát vizsgáljuk, hogy a döntések ne esetlegesek legyenek, hanem a gyors visszajelzés, a stabilitás és a költség egyensúlyán alapuljanak. A cél nem a tesztelési technikák részletes bemutatása – arra a következő alfejezetben kerül sor –, hanem a stratégiai keret megértése, amely meghatározza, hogy milyen teszteket, milyen arányban és milyen szereposztásban érdemes alkalmazni.

A tesztelési szintek és a minőségbiztosítás helye

A szoftvertesztelést hagyományosan szintekre bontjuk aszerint, hogy mekkora egységet, milyen környezetben és milyen céllal vizsgálunk. Az **egységteszt** (*unit test*) egyetlen osztályt, függvényt vagy modult ellenőriz elszigetelten, a függőségeket helyettesítőkkel (mock, stub) kiváltva. Az **integrációs teszt** (*integration test*) két vagy több valódi komponens együttműködését vizsgálja, például egy szolgáltatás és az adatbázis, vagy két mikroservice közötti hívás szerződését. A **UI teszt** – gyakran **end-to-end (E2E) teszt** – a teljes rendszert felhasználói szemszögből gyakoroltatja, a böngészőtől az adatbázisig. A **felfedező teszt** (*exploratory testing*) ezzel szemben nem előre rögzített szkriptet követ, hanem egy időkeretes, célorientált, szakértői vizsgálat, amelyben a tesztelő a rendszer viselkedését aktívan felfedezi, és a megfigyelések alapján alakítja a következő lépéseket.

E szintek nem egymást helyettesítő alternatívák, hanem egymást kiegészítő rétegek. Mindegyik más típusú hibát tár fel, más visszajelzési időt biztosít, és más költséggel jár. Az egységteszt egy logikai hibát másodperceken belül jelez, de nem mutatja meg, hogy a komponens helyesen kommunikál-e az adatbázissal. Az integrációs teszt a kommunikáció hibáit fedi fel, de nem bizonyítja, hogy a felhasználó számára értékes út működik. A UI teszt a teljes utat ellenőrzi, de lassú és törékeny a környezeti változásokra. A felfedező teszt a váratlan használatot és a felhasználói élményt vizsgálja, de nem reprodukálható automatikusan. A jó tesztstratégia e rétegeket úgy kombinálja, hogy a lehető leggyorsabban, legolcsóbban és legmegbízhatóbban kapjunk visszajelzést a rendszer állapotáról.

A **minőségbiztosítás** (*Quality Assurance*, QA) e szintekre épül, de több annál: folyamat, amely a követelményektől a produkciós működésig végigkíséri a szoftvert, és szerep, amely a minőségi gyakorlatokat, eszközöket és metrikákat tartja fenn. A QA nem a tesztelés szinonimája; a tesztelés a minőségbiztosítás egyik tevékenysége. A cél nem a hibák megtalálása, hanem a hibák megelőzése, és az, hogy a csapat a megfelelő időben kapjon visszajelzést a rendszer állapotáról. Ezért a tesztelési szintek elrendezése, aránya és sorrendje architekturális döntés, nem pedig utólagos kiegészítés.

A tesztpiramis

A tesztpiramis a legismertebb modell a tesztelési szintek arányainak kifejezésére. Alakja arra emlékeztet, hogy alul széles, felül keskeny: sok gyors, olcsó, izolált teszt, és kevés lassú, drága, rendszerszintű teszt. A klasszikus ábrázolás három réteget különböztet meg:

- **Egységtesztek**: a piramis alapja. Számuk a legnagyobb, futási idő másodpercekben mérhető, és hiba esetén pontosan lokalizálják a problémát. Az egységtesztek a belső tervezést is visszajelzik: ha nehéz tesztelni egy osztályt, az gyakran a rossz modulhatárokra utal.
- **Integrációs tesztek**: középső réteg. Azt vizsgálják, hogy a komponensek valódi függőségeikkel együtt is helyesen működnek. Ide tartoznak az adatbázis-migrációk, a külső API-hívások szerződései, a message broker integrációk. Ezek lassabbak és környezetfüggőbbek, de nélkülözhetetlenek, mert az egységtesztek önmagukban nem garantálják az együttműködést.
- **UI / E2E tesztek**: a piramis csúcsa. Egy-egy felhasználói út teljes végigjárása. Nagyon értékesek, mert a valós felhasználói élményt ellenőrzik, de lassúak, törékenyek, és nehéz őket determinisztikusan futtatni. Ezért csak kevés, kiválasztott kritikus útra szorítkoznak.

A piramis üzenete nem az, hogy mindig pontosan 70-20-10 százalékos arányt kell tartani; az arányok a rendszer jellegétől, a csapat érettségétől és a kockázati profiltól függenek. Az üzenet az irány: a tesztbefektetést az alsóbb, gyorsabb szintekre érdemes helyezni, a felső szinteket pedig tudatosan, a kritikus felhasználói utakra korlátozni. Ha a piramis megfordul – sok UI teszttel és kevés egységteszttel –, a visszajelzés lassú lesz, a hibák lokalizálása nehéz, a tesztkarbantartás költsége pedig nő. Ezt nevezzük **jégkrémkúp anti-mintának**, amelyre még visszatérünk.

A piramis erőssége, hogy egyértelmű prioritást ad: először a domain logikát teszteljük, aztán az integrációt, végül a teljes rendszert. Ezzel elkerüljük, hogy a drága E2E tesztek olyan hibákat keressenek, amelyeket olcsóbban, alacsonyabb szinten is meg lehetne találni. A gyakorlatban a piramis azt is jelenti, hogy a csapatnak folyamatosan figyelnie kell a tesztfutások idejét és stabilitását: ha egy alsóbb szintű teszt lassú vagy flaky, az a modell sérülését jelzi.

A teszt-trofea

A teszt-trofea (*testing trophy*) a piramis továbbgondolása, amely a statikus elemzést és az integrációs teszteket helyezi előtérbe. A modell négy réteget ábrázol:

- **Statikus elemzés**: a trófea alapja. Ide tartozik a linter, a típusellenőrzés, a formátumellenőrzés, a sebezhetőségi függőségvizsgálat. Ezek nem futásidejű tesztek, hanem a kód elemzésével már a futtatás előtt visszajelzést adnak. Gyorsak, és sok hibát megelőznek. Például egy TypeScript projektben a típusellenőrzés már a build előtt jelezheti, ha egy függvényhívás argumentumtípusa nem egyezik, anélkül, hogy egységtesztet kellene írni.
- **Egységtesztek**: továbbra is fontosak, de a trófea modellben kisebb hangsúlyt kapnak, mint a piramisban, mert önmagukban nem fedik le az integrációs hibákat.
- **Integrációs tesztek**: a trófea legnagyobb része. Mivel a mai rendszerekben a komponensek közötti szerződés, az adatbázis-séma, a külső szolgáltatások és az aszinkron üzenetek jelentik a leggyakoribb hibák forrását, ezekre érdemes több tesztet írni. A trófea modell gyakran a „kevés unit, sok integráció” arányt javasolja.
- **E2E tesztek**: a csúcs, kevés számban, a legfontosabb felhasználói utakra.

A trófea modell különösen a mikroszolgáltatásos, szerződésalapú és erősen integrált rendszerekben hasznos, ahol a komponensek izoláltan jól működnek, de az együttműködésük a kockázat forrása. A piramis és a trófea nem ellentmondanak, hanem eltérő hangsúlyt adnak: a piramis a gyors egységtesztekre, a trófea a statikus ellenőrzésre és az integrációra összpontosít. A választás attól függ, hol koncentrálódik a rendszer kockázata, és milyen a csapat technológiai környezete. Például egy erősen frontend-orientált, dinamikus típusú JavaScript projektben a trófea modell jobban

illeszkedik, mert a statikus elemzés és az integrációs tesztek kompenzálják a típusbiztonság hiányát.

A QA mint folyamat

A minőségbiztosítás folyamata nem a fejlesztés végén kezdődik. A korai szakaszban a QA a követelmények áttekintésében, a tesztelhetőségi kritériumok meghatározásában, a kockázatelemzésben és a tesztstratégia kialakításában vesz részt. A **shift-left** elv azt jelenti, hogy a tesztelési tevékenységet minél korábbra hozzuk: a kód megírása előtt definiáljuk az elfogadási feltételeket, a szerződéseket és a tesztelési szinteket. A **shift-right** pedig a produkciós visszajelzést erősíti: a canary release, a feature flag, az observability és a chaos engineering mind a minőségbiztosítás részét képezik, mert a valós forgalomban derül ki, hogyan viselkedik a rendszer.

A folyamat jellemzően a következő lépésekből áll:

1. **Kockázatfelmérés**: melyek a kritikus funkciók, a hibás működés milyen hatással jár, hol vannak a legnagyobb bizonytalanságok. A kockázat lehet üzleti (például pénzügyi tranzakció) vagy technikai (például új adatbázis-motor).
2. **Tesztstratégia**: a tesztelési szintek kiválasztása, az arányok, az eszközök és a környezetek meghatározása. Ez a jelen alfejezet laborjának központi feladata. A stratégiának tartalmaznia kell a tesztelési célokat, a lefedettségi elvárásokat, a tesztadatok kezelését és a „done” fogalmának definícióját.
3. **Teszterv és automatizálás**: a konkrét tesztek implementálása, a CI/CD pipeline-ba illesztése, a tesztadatok kezelése. Az automatizálás nem cél, hanem eszköz: a gyors, determinisztikus teszteket érdemes automatizálni, a felfedező tesztelést nem.
4. **Végrehajtás és kiértékelés**: a tesztek futtatása, a hibák osztályozása, a lefedettségi és stabilitási metrikák értékelése. A folyamat során a tesztfutások eredményeit a csapat rendszeresen áttekinti, és a flaky teszteket azonnal javítja.
5. **Visszacsatolás és fejlesztés**: a tesztelési gyakorlat folyamatos javítása a tanulságok alapján. Ide tartozik a tesztpiramis arányainak felülvizsgálata, az új eszközök bevezetése, a képzés.

A QA folyamat sikere nem mérhető pusztán a tesztesetek számával vagy a lefedettségi százalékkal. A jó metrika a hibák korai felfedezése, a tesztfutás stabilitása, a kritikus hibák produkcióba jutásának aránya, és a csapat bizalma a release folyamatban. A mérőszámokat mindig mértékegységgel és mérési ablakkal kell megadni, ahogy a könyv konvenciói is előírják.

A QA mint szerep

A QA szerep az elmúlt évtizedben jelentősen átalakult. A hagyományos, elkülönült tesztelői pozíció helyett a modern csapatokban a minőségért mindenki felelős: a fejlesztő ír egység- és integrációs teszteket, az üzemeltető figyeli a produkciós jeleket, a termékmenedzser definiálja az elfogadási kritériumokat. Ebben a modellben a **QA mérnök** vagy **minőségsegítő** (*quality coach*) nem a tesztek egyedüli írója, hanem a minőségi gyakorlatok szakértője, aki:

- kialakítja és karbantartja a tesztelési keretrendszereket, a tesztadat-generátorokat és a környezeteket;
- segít a tesztelhetőség beépítésében az architektúrába (például tesztduplák, szerződésellenőrzés, observability);
- facilitálja a kockázatalapú teszttervezést, és a kritikus területekre fókuszál;
- elemzi a tesztfutások eredményeit, és a flaky tesztek, lassú tesztek, hiányzó lefedettségek ellen dolgozik;
- a felfedező tesztelést szervezi és a manuális tesztelési ismereteket átadja;
- a produkciós minőségi jelekből visszacsatolást ad a fejlesztőknek.

A QA szerep tehát nem kapuőr, hanem a minőségi rendszer karbantartója és tanácsadója. A felfedező tesztelés ebben a szerepben kiemelt fontosságú, mert az automatizált tesztek csak a várt viselkedéseket ellenőrzik, a váratlan használatot és a felhasználói élményt nem. A jó QA szakember a tesztelési ismeretek mellett erős rendszerszemlélettel rendelkezik, érti a CI/CD pipeline-t, a konténerizációt és a megfigyelhetőséget, mert ezek nélkül a tesztelés nem hatékony.

Tipikus hibák és anti-minták

A tesztstratégia kialakításakor számos csapdába eshetünk. Az alábbiakban a leggyakoribbakat emeljük ki, a helyes megoldás irányával együtt.

- **Jégkrémkúp (ice cream cone)**: a piramis megfordítása. A csapat a legtöbb tesztet a UI szinten írja, mert az tűnik leginkább „valódinak", de ezek lassúak, törékenyek és drágák. Helyes megoldás: a UI teszteket a kritikus utakra korlátozni, a logikát alacsonyabb szinteken lefedni.
- **Túlzott egységteszt-fókusz integráció nélkül**: a sok izolált teszt hamis biztonságot ad. Ha a komponensek közötti szerződések nincsenek tesztelve, a produkciós hiba az integrációban fog jelentkezni. Helyes megoldás: a piramis vagy trófea szerint arányos integrációs tesztelés.
- **Flaky tesztek**: a nem determinisztikus tesztek aláássák a bizalmat. A flaky teszt gyakran időzítési, sorrendi vagy környezeti problémát jelez. Helyes megoldás: a teszt izolálása, a várakozások explicit kezelése, a tesztadatok frissítése, és a flaky teszt azonnali javítása vagy karanténba helyezése.
- **Tesztadatok kezelésének hiánya**: a teszteléshez használt adatok nem reprodukálhatók, vagy valós, érzékeny adatokat használnak. Helyes megoldás: szintetikus, álnevesített tesztadatok, tesztadattár, adatgenerátor.
- **A tesztkörnyezet instabilitása**: ha a tesztek a környezet miatt buknak, a visszajelzés megbízhatatlan. Helyes megoldás: konténerizált, izolált környezetek, mint például a Testcontainers, és a környezet verziózása.
- **A lefedettség mint egyetlen cél**: a magas százalékos lefedettség nem jelenti azt, hogy a tesztek jók. Helyes megoldás: a kritikus üzleti logika lefedettségét mérni, és a tesztminőséget (pl. mutációteszteléssel) is értékelni.
- **A QA mint utolsó kapu**: ha a tesztelés csak a fejlesztés végén történik, a hibák későn derülnek ki. Helyes megoldás: a QA folyamat beépítése a teljes életciklusba, shift-left és shift-right.

Ezek a hibák mind arra vezethetők vissza, hogy a tesztstratégia nincs összhangban a rendszer architektúrájával és a kockázatokkal. A jó stratégia nem egy általános recept, hanem kontextusfüggő döntés. A következő gyakorlati példa ezt szemlélteti.

Gyakorlati példa: tesztstratégia egy rendeléskezelő szolgáltatáshoz

Tegyük fel, hogy egy mikroszolgáltatás-alapú rendszerben a **Rendeléskezelő szolgáltatás** (*Order Service*) felelős a rendelések létrehozásáért, állapotának kezeléséért és a rendelési események kiküldéséért egy **outbox** táblán keresztül. A szolgáltatás REST API-t kínál, PostgreSQL adatbázist használ, és Kafka üzeneteket publikál. A csapat konténerizált fejlesztői környezetet használ, a CI pipeline minden pull requestre lefut. A domain logika magában foglalja a rendelés állapotátmeneteit (például Létrehozva → Kifizetve → Kiszállítva), az árkalkulációt és a készletfoglalást.

A tesztstratégiát a piramis és a trófea elvei alapján alakítjuk ki, a következőképpen:

- **Statikus elemzés**: a kódban ESLint (vagy hasonló) és típusellenőrzés fut, emellett dependency vulnerability scan. Ez a trófea alapja, és már a commit előtt visszajelzést ad. A pipeline első lépéseként fut, és ha hibát talál, a build nem folytatódik.

- **Egységtesztek**: a domain logikát fedik le, például a rendelés állapotátmeneteit, az árkalkulációt, az outbox rekord létrehozását. A függőségeket mockolják, a futási idő pár másodperc. Cél: a domain szabályok gyors ellenőrzése. Példa teszteset: „Ha a rendelés Fizetésre vár állapotban van, és a fizetési szolgáltatás sikeres visszaigazolást küld, a rendelés Kifizetve állapotba kerül, és outbox rekord jön létre."
- **Integrációs tesztek**: a szolgáltatás és a valódi PostgreSQL közötti interakciót, a Kafka producer és az outbox publisher együttműködését, valamint a REST API és a hívó felek közötti szerződést tesztelik. Ezek Testcontainers segítségével izolált konténerben futnak. A trófea modell szerint itt van a legnagyobb hangsúly, mert a hibák leggyakrabban az adatbázis-séma, a serializáció és az üzenetküldés integrációjában jelentkeznek. Példa: az API szerződés tesztje OpenAPI specifikációból generált klienssel ellenőrzi, hogy a válasz séma megfelel-e a szerződésnek.
- **E2E tesztek**: egyetlen kritikus felhasználói utat fednek le: a rendelés feladása, a fizetés visszaigazolása, az állapotváltozás és az esemény megjelenése egy teszt fogyasztóban. Ezt Playwright vagy hasonló eszközzel, valós böngésző nélkül, API szinten is lehet tesztelni, ha nincs UI. A cél a rendszerszintű együttműködés igazolása. Például egy teszt, amely a REST API-n létrehoz egy rendelést, elküld egy fizetési visszaigazolást, és ellenőrzi, hogy a Kafka topicban megjelenik a RendelésKifizetve esemény.
- **Felfedező tesztelés**: a release előtt egy időkeretes session, ahol a QA mérnök a rendelés lemondását, a duplikált kéréseket, a hálózati hibákat próbálja ki, és a megfigyeléseket jegyzi. Ez nem automatizálható, de fontos a váratlan esetek feltárásához. A session eredményeit a csapat közösen áttekinti, és a hibákat osztályozza.

A rétegek aránya ebben a kontextusban körülbelül: 50% integrációs, 30% egységteszt, 15% statikus (bár nem futásidejű teszt, a mennyiség más dimenzió), 5% E2E. Ez a trófea modellhez áll közelebb, mert a rendszer kockázata az integrációban összpontosul. A döntést ADR-ben rögzítjük:

- **Kontextus**: a Rendeléskezelő szolgáltatás kockázata az adatbázis- és üzenetküldési integrációban van; a domain logika viszonylag stabil.
- **Döntés**: a tesztelési hangsúlyt az integrációs tesztekre helyezzük, a statikus elemzést beépítjük a pipeline elejére, és az E2E teszteket egy kritikus útra korlátozzuk.
- **Alternatívák**: (1) klasszikus piramis nagyobb egységteszt-aránnyal – elvetettük, mert az integrációs hibákat későn fedezné fel; (2) erős UI teszt automatizálás – elvetettük, mert a szolgáltatásnak nincs UI-ja, és a teljes rendszer UI tesztjei a csapat másik szolgáltatásához tartoznak.
- **Következmények**: a CI futási idő növekszik a konténeres integrációs tesztek miatt, ezért a pipeline párhuzamosítást igényel; a tesztadatok szintetikusak, a Kafka teszt topic izolált. A statikus elemzés miatt a kód a típusellenőrzésnek megfelelően íródik, ami a fejlesztési sebességet kezdetben csökkentheti, de a hibák korai felfedésével hosszabb távon javítja.

Ez a példa mutatja, hogy a tesztstratégia nem egy statikus sablon, hanem a rendszer kockázati profiljából és a csapat környezetéből levezetett döntés. A döntéseket verziózott ADR-ben dokumentáljuk, hogy a jövőbeli csapattagok megértsék az okokat.

Labor: tesztstratégia készítése

A fejezet gyakorlati feladata, hogy készítsen egy reprodukálható tesztstratégiát egy választott rendszerhez. A cél nem egy teljes tesztimplementáció, hanem a stratégiai döntések dokumentálása és indoklása. A labor önállóan vagy páros munkában is elvégezhető.

- **Kiinduló állapot**: válasszon egy valós vagy fiktív rendszert, amelynek van legalább egy szolgáltatása, adatbázisa és egy külső integrációja (pl. REST API vagy üzenetküldő). Jó

kiindulópont lehet a Release engineering és rollback fejezetben használt deploy pipeline példa. A rendszer legyen elég kicsi ahhoz, hogy egy dokumentumban áttekinthető legyen.

- **Elvárt eredmény**: egy Markdown dokumentum vagy Jupyter notebook, amely tartalmazza:
  - a rendszer rövid kontextusát (architektúra, kritikus funkciók, kockázatok);
  - a választott tesztelési modellt (piramis vagy trófea) és a rétegek arányát;
  - az egyes szintekhez tartozó konkrét tesztfajtákat és eszközöket;
  - a statikus elemzés és a felfedező tesztelés helyét;
  - a tesztadatok kezelésének módját;
  - a tesztkörnyezetek kialakítását (pl. konténerizáció);
  - egy ADR-t, amely rögzíti a fő döntést, az alternatívákat és a következményeket;
  - az ellenőrzési szempontokat, amelyekkel a stratégia sikerességét mérni fogja.
- **A mért és a várt eredményt különítse el**: például a „várt” a lefedettségi cél, a „mért” a jelenlegi lefedettség; a „várt” a tesztfutási idő korlát, a „mért” a tényleges futási idő. A dokumentum ne keverje a kettőt. A várt eredményeket célként, a mért eredményeket aktuális állapotként jelölje.
- **Visszaállítási út**: mivel a labor nem hajt végre destruktív műveletet, visszaállítás nem szükséges; ha a hallgató lokálisan futtat konténereket, azok leállítása a feladat része.

A labor értékelése a döntések indoklásán, a kockázatok és a tesztelési szintek összhangján, valamint a dokumentáció reprodukálhatóságán alapul, nem a sablonok mechanikus követésén.

Ellenőrzőlista a tesztstratégiához

- A stratégia lefedi-e az összes tesztelési szintet: statikus, unit, integrációs, E2E, felfedező?
- Az arányok indokoltak-e a rendszer kockázati profilja alapján?
- A tesztek determinisztikusak-e, és van-e terv a flaky tesztek kezelésére?
- A tesztadatok szintetikusak, reprodukálhatók, és nem tartalmaznak érzékeny adatot?
- A tesztkörnyezet izolált és verziózott-e?
- A felfedező tesztelés be van-e tervezve a release ciklusba?
- A mért és várt eredmények külön vannak-e választva?
- Az ADR rögzíti-e a döntés okát, az alternatívákat és a következményeket?

Önellenőrző kérdés

Miért helyezzük a hangsúlyt az alsóbb szintekre a piramisban, és mely helyzetekben indokolt a teszt-trofea modell választása? Fogalmazza meg a választ egy konkrét rendszer kontextusában, hivatkozva a kockázatokra és a visszajelzési időre.

Összefoglalás

A tesztpiramis és a teszt-trofea modellek a tesztelési szintek tudatos elrendezését segítik, hogy a gyors, olcsó tesztek adják a visszajelzés alapját, a lassú, drága tesztek pedig a kritikus felhasználói utakra korlátozódjanak. A minőségbiztosítás folyamata a teljes életciklusra kiterjed, a QA szerep pedig a minőségi gyakorlatok fenntartása és tanácsadás, nem egyszerű tesztelői pozíció. A következő alfejezet (Unit, integrációs, contract és E2E tesztek) ezen alapokra építve vizsgálja meg az egyes tesztfajták – unit, integrációs, contract és E2E – konkrét technikáit és eszközeit.

25.2 Unit, integrációs, contract és E2E tesztek

Az előző alfejezetben bemutatott tesztpiramis és teszt-trofea azt hangsúlyozta, hogy a különböző tesztfajták eltérő célt szolgálnak, és a hatékony stratégia a rétegek megfelelő arányán múlik. Ebben

az alfejezetben a négy legfontosabb tesztfajtát – unit, integrációs, contract és end-to-end (E2E) – vizsgáljuk meg közelebbről: mi a céljuk, hol húzódnak a határaik, hogyan viszonyulnak egymáshoz, és milyen tipikus hibákat érdemes elkerülni. A tárgyalás során bevezetjük a tesztkettősök (test double) fogalomkörét is, mert ezek nélkül a tesztfajták határai nem érthetők meg pontosan. Az alfejezet végén egy laborfeladat segít a tesztmátrix elkészítésében, amely a tesztstratégia konkrét, komponensekhez rendelt megjelenítése.

A fejezetben használt kód- és konfigurációpéldák szemléltető jellegűek; éles rendszeren nem feltétlenül hajtandók végre. A destruktív műveleteket (például adatbázis-törlés) laborban is csak izolált környezetben szabad bemutatni.

25.2.1 A tesztfajták célja és határai – áttekintés

A négy tesztfajta egy-egy eltérő kockázati területet fed le, és más-más visszajelzési sebességet, karbantartási költséget, valamint környezeti igényt képvisel. A cél nem az, hogy minden komponenshez minden tesztfajtából készüljön, hanem hogy a megfelelő kockázatot a megfelelő szinten ellenőrizzük. A tesztpiramis elve szerint alul sok, gyors, izolált unit teszt található; feljebb kevesebb, lassabb, de szélesebb integrációt ellenőrző teszt; a csúcson pedig néhány, a teljes felhasználói utat átfogó E2E teszt. A contract tesztek a piramis középső rétegében helyezkednek el, de nem minden rendszerben jelennek meg; szerepük akkor nő meg, ha a rendszerhatárokon átnyúló szolgáltatások közötti szerződéseket kell biztosítani anélkül, hogy a teljes integrációs környezetet minden tesztfuttatáskor fel kellene építeni.

A cél és határok mátrixa (rövidítve):

Tesztfajta	Elsődleges cél	Jellemző határ	Tipikus környezet
Unit	Egyetlen komponens belső logikájának ellenőrzése	Nem lépi át a komponens határát; minden külső függőséget tesztkettőssel helyettesít	Fejlesztői gép, CI, nagyon gyors
Integrációs	Két vagy több komponens együttműködésének ellenőrzése	Átlépi a komponensek közötti határt, de jellemzően nem a teljes rendszert	CI, tesztkörnyezet, adatbázis vagy külső szolgáltatás szimulátor
Contract	A fogyasztó és a szolgáltató közötti API-szerződés betartásának ellenőrzése	A szerződés által definiált interfészre korlátozódik; a belső implementációt nem vizsgálja	CI, provider és consumer oldali tesztfuttatás
E2E	A teljes rendszer felhasználói szempontú ellenőrzése	A rendszerhatáron belül a lehető legszélesebb utat fedi le; külső rendszereket gyakran szimulál, de a belső integrációt valósan teszteli	Staging vagy dedikált E2E környezet, lassú

Ez a mátrix már önmagában egyfajta tesztmátrix-váz; a labor során ezt fogjuk komponensekhez kötve részletezni.

25.2.2 Unit tesztek

A unit teszt (más néven komponens- vagy egységteszt) egyetlen, jól körülhatárolható szoftveregység – általában egy osztály, függvény vagy modul – viselkedését ellenőrzi, a külső függőségek kizárásával. A cél a belső üzleti logika, az elágazások, a kivételkezelés és a határfeltételek gyors, ismételhető, determinisztikus ellenőrzése.

A unit teszt határai:

- Nem indít valódi adatbázist, hálózati hívást, fájlrendszer-írást vagy külső szolgáltatást.
- Minden ilyen függőséget tesztkettőssel (test double) helyettesít, amelyet a következő szakaszban részletezünk.
- Egy unit tesztnek ideális esetben egyetlen viselkedést kell ellenőriznie, nem pedig egy teljes folyamatot.
- A tesztnek gyorsnak kell lennie: tipikusan néhány tíz milliszekundum alatt lefut, így a teljes unit tesztkészlet percek alatt végrehajtható.

Példa – unit teszt pszeudokódban:

Tegyük fel, hogy van egy `PriceCalculator` komponens, amely a termék árából és a vásárlói kedvezményből számítja ki a fizetendő összeget.

```
class PriceCalculator:
    def final_price(product, customer):
        if customer.is_vip():
            discount = 0.2
        else:
            discount = 0.0
        return product.price * (1 - discount)
```

A unit teszt:

```
test_final_price_for_vip_customer:
    product = Product(price=100)
    customer = Customer(vip=True)
    calculator = PriceCalculator()
    assert calculator.final_price(product, customer) == 80

test_final_price_for_regular_customer:
    product = Product(price=100)
    customer = Customer(vip=False)
    calculator = PriceCalculator()
    assert calculator.final_price(product, customer) == 100
```

Itt nincs szükség tesztkettősre, mert a `PriceCalculator` nem függ külső komponenstől. Ha viszont a `Customer` osztály például egy adatbázisból töltené be az adatokat, akkor a unit tesztben egy `Customer` stub-ot vagy mock-ot használnánk.

Tipikus hibák:

- Túl sok belső implementációs részletet tesztel (például privát metódusok közvetlen hívása), ami törékennyé teszi a tesztet a refaktorálással szemben.
- A teszt nem izolált: valódi adatbázishoz vagy hálózathoz nyúl, így lassú és nem determinisztikus lesz.
- A tesztnév nem írja le a viselkedést; helyette olyan nevek szerepelnek, mint `test1` vagy `test_calculate`.
- A tesztben túl sok assert szerepel, ami megnehezíti a hiba lokalizálását.

A unit teszt helye a stratégiában: a leggyorsabb és legolcsóbb visszajelzést adja, ezért ebből kell a legtöbbet írni. Nem alkalmas azonban az integrációs hibák, a konfigurációs problémák vagy a felhasználói élmény ellenőrzésére.

25.2.3 Integrációs tesztek

Az integrációs teszt két vagy több komponens együttműködését ellenőrzi, átlépve a unit teszt izolációs határát. Célja annak igazolása, hogy a komponensek valódi vagy valósághű függőségekkel együtt is helyesen működnek: a hívási lánc, a paraméterek átadása, a sorosítás, a tranzakciókezelés és a hibakezelés a komponensek között.

Az integrációs teszt határai és fajtái:

- **Szűk integrációs teszt:** két komponens közötti együttműködést ellenőriz, például egy repository réteg és egy adatbázis, vagy egy HTTP kliens és egy valódi (de teszt célú) szerver. A külső függőségeket gyakran konténerizált környezetben (például Testcontainers) vagy beágyazott szolgáltatással valósítja meg.
- **Széles integrációs teszt:** több komponens együttesét fedi le, de még nem a teljes rendszert. Például egy szolgáltatás belső rétegeit (API, üzleti logika, adatbázis) együtt teszteli, de a külső fizetési szolgáltatót már szimulálja.

Az integrációs teszt nem törekszik a teljes felhasználói út lefedésére, és nem vizsgálja a rendszer egészének konfigurációját, telepítését vagy a felhasználói felületet. Ezek az E2E tesztek feladatai.

Példa – integrációs teszt egy repository réteggel:

```
test_order_repository_saves_and_loads_order:
    database = start_test_database()  # izolált, pl. konténerben
    repository = OrderRepository(database)
    order = Order(id="order-123", amount=100)
    repository.save(order)
    loaded = repository.load("order-123")
    assert loaded.amount == 100
    database.stop()
```

Ebben a tesztben a `repository` és az adatbázis közötti integrációt ellenőrizzük. A teszt nem unit teszt, mert valódi adatbázist használ; de nem is E2E, mert csak egy réteget fed le.

Tipikus hibák:

- Minden komponenspárhoz integrációs tesztet írni, ami a tesztkészletet lassúvá és karbantarthatatlanná teszi. A tesztpiramis szerint a széles integrációs tesztekből kevesebbet érdemes készíteni.
- A tesztkörnyezet nem izolált: ugyanazt az adatbázist használja több teszt, ami egymás közötti interferenciához vezet. Megoldás a tranzakciós visszagörgetés vagy konténerenkénti külön adatbázis.
- A teszt valójában unit teszt, de elnevezésében integrációsnak hívják – ez félrevezeti a csapatot a lefedettség értékelésekor.
- Az integrációs teszt túl sokat mockol, így nem a valódi integrációt ellenőrzi.

25.2.4 Contract tesztek

A contract teszt (szerződésteszt) két, egymástól függetlenül fejlesztett komponens – jellemzően szolgáltatások – közötti API-szerződés betartását ellenőrzi. A cél annak biztosítása, hogy a fogyasztó (consumer) által elvárt kérések és válaszok megfeleljenek a szolgáltató (provider) által ténylegesen nyújtott interfésznek, anélkül hogy a két felet minden tesztfuttatáskor közös integrációs környezetben kellene futtatni.

A fogyasztói elvárás fogalma:

A contract testing középpontjában a fogyasztói elvárás áll: a fogyasztó oldaláról definiáljuk, hogy milyen HTTP végpontot, milyen paraméterekkel hívunk, és milyen válaszra számítunk. Ezt a szerződést egy géppel olvasható formátumban (például JSON vagy YAML) rögzítjük. A provider oldalán egy tesztfuttatás során a szerződés alapján szimulált kéréseket küldünk a valódi providernek, és ellenőrizzük, hogy a válasz megfelel-e a szerződésnek. Ha a provider megváltoztatja az API-ját, a contract teszt eltörik, jelezve a fogyasztóval való inkompatibilitást – még mielőtt a változás éles környezetbe kerülne.

A contract teszt határai:

- Nem a provider belső logikáját teszteli, csak a szerződésben rögzített interfész viselkedését.
- Nem helyettesíti a széles integrációs vagy E2E teszteket, mert nem ellenőrzi a hálózati rétegek, a terheléselosztó vagy a teljes rendszer együttműködését.
- A szerződést a fogyasztó és a szolgáltató közösen, verziózottan kezeli; a szerződésváltozásnak kompatibilitási szabályokat kell követnie (például additív mezők megengedettek, törlés vagy típusváltozás nem).

Példa – fogyasztói szerződés (Pact-szerű ábrázolás):

A fogyasztó oldalán a következő szerződést rögzítjük:

```
{
  "consumer": { "name": "order-web" },
  "provider": { "name": "order-service" },
  "interactions": [
    {
      "description": "order lekérdezése azonosító alapján",
      "request": {
        "method": "GET",
        "path": "/orders/order-123"
      },
      "response": {
        "status": 200,
        "headers": { "Content-Type": "application/json" },
        "body": {
          "id": "order-123",
          "amount": 100
        }
      }
    }
  ]
}
```

A provider oldali teszt ezt a szerződést betölti, valódi HTTP kérést küld a futó `order-service`-nek a `/orders/order-123` végpontra, és ellenőrzi, hogy a válasz státusza, fejléce és JSON-mezői megfelelnek-e a fenti elvárásnak. A teszt nem futtatja a fogyasztót; csak a szerződést használja.

Eszközök: A Pact keretrendszer (https://docs.pact.io/) az egyik legismertebb megvalósítás, de a fogalom általános: bármilyen, a fogyasztói elvárást verziózottan rögzítő és a provider oldalán

ellenőrző mechanizmus contract testingnek tekinthető. A keretrendszer pontos verzióját és konfigurációját mindig a hivatalos dokumentáció alapján kell ellenőrizni; ebben a könyvben nem adunk meg konkrét, ellenőrizetlen verziószámot.

Tipikus hibák:

- A contract tesztet összekeverik a provider belső unit tesztjével; a contract tesztnek nem szabad a provider belső implementációját vizsgálnia.
- A szerződést nem verziózzák, így nem lehet visszakeresni, melyik fogyasztói elvárás melyik provider-változathoz tartozik.
- A fogyasztó által írt szerződés túl részletes: például pontos időbélyeget vagy dinamikus azonosítót vár el, ami a valós válaszban változhat. Ilyenkor a szerződést rugalmasan kell megfogalmazni (például reguláris kifejezéssel vagy típusellenőrzéssel).
- A provider oldali tesztet nem futtatják minden változtatásnál, így a szerződésszegés későn derül ki.

25.2.5 End-to-end (E2E) tesztek

Az E2E teszt a teljes rendszert – vagy annak egy meghatározó felhasználói útját – ellenőrzi a végfelhasználó szemszögéből. Célja annak igazolása, hogy a rendszer a valós konfigurációhoz hasonló környezetben, a komponensek együttes működésével képes a kritikus üzleti folyamatokat végrehajtani: például bejelentkezés, termék kosárba helyezése, fizetés, visszaigazolás.

Az E2E teszt határai:

- A teszt a lehető legszélesebb utat fedi le, de nem feltétlenül minden lehetséges ágat; a cél a kritikus utak és a rendszerhatárokon átnyúló integráció ellenőrzése.
- A külső rendszereket (fizetési szolgáltató, e-mail küldő) gyakran szimulálják, hogy a teszt determinisztikus és költséghatékony legyen, de a belső komponensek valós integrációban futnak.
- Az E2E tesztek lassúak, és környezetfüggők; ezért számukat alacsonyan kell tartani, és lehetőleg csak a legfontosabb felhasználói utakra koncentrálni.
- Az E2E teszt nem helyettesíti az alacsonyabb szintű teszteket; ha egy E2E teszt eltörik, a hiba lokalizálása nehéz, ezért a piramis alján lévő teszteknek kell a pontos okot megtalálniuk.

Példa – E2E teszt pszeudokódban (böngésző-vezérelt):

```
test_e2e_checkout_flow:
    browser = start_browser()
    browser.navigate("https://staging.shop.example.com")
    browser.click("Bejelentkezés")
    browser.fill("email", "tesztfelhasznalo@example.com")
    browser.fill("password", "helyettesito-jelszo")
    browser.click("Belépés")
    browser.search("könyv")
    browser.click("Kosárba")
    browser.click("Pénztár")
    browser.fill("kártyaszám", "4111 1111 1111 1111")
    browser.click("Fizetés")
    assert browser.contains_text("Köszönjük a rendelést!")
    browser.close()
```

Ez a teszt a teljes felhasználói utat lefedi, beleértve a frontend, backend, adatbázis és fizetési szimulátor együttműködését. A fizetési szolgáltató itt szimulált, de a belső rendszer valós.

Tipikus hibák:

- Túl sok E2E teszt írása, ami a tesztkészletet lassúvá és törékennyé teszi; a piramis elve szerint ezekből kell a legkevesebb.
- A tesztek nem izoláltak: ugyanazt a felhasználói fiókot vagy adatkészletet használják, ami interferenciához vezet. Megoldás a tesztadatok előkészítése és takarítása (setup/teardown).
- A teszt a belső implementáció részleteit ellenőrzi, nem a felhasználói viselkedést; például egy adott gomb belső azonosítójára hivatkozik, ami a felület változásával eltörik.
- Az E2E tesztet nem futtatják rendszeresen, csak kiadás előtt, így a hibák későn derülnek ki.

25.2.6 Mock, stub és test double

A tesztfajták közötti határok pontos megértéséhez elengedhetetlen a tesztkettősök (test double) fogalomköre. A test double gyűjtőfogalom, amely minden olyan helyettesítő objektumot vagy komponenst jelöl, amelyet a teszt alatt álló komponens valódi függőségének helyére teszünk, hogy a tesztet izoláljuk, determinisztikussá tegyük vagy a függőség viselkedését ellenőrizzük.

A leggyakoribb test double típusok:

- **Dummy:** Olyan objektum, amelyet átadunk ugyan, de nem használjuk. Például egy metódus paramétereként átadott, de a teszt szempontjából nem releváns objektum.
- **Stub:** Előre meghatározott válaszokat ad a hívásokra, de nem ellenőrzi, hogy mely hívások történtek. Célja a teszt alatt álló komponens bemeneteinek biztosítása. Példa: egy `CustomerRepository` stub, amely a `load("customer-1")` hívásra mindig ugyanazt a `Customer` objektumot adja vissza.
- **Mock:** Elvárásokat definiál a hívásokra (például mely metódus hányszor, milyen paraméterekkel hívódik meg), és a teszt végén ellenőrzi ezeket. A mock tehát interakció-ellenőrzést végez, nem csak válaszokat ad. Példa: egy `PaymentGateway` mock, amely elvárja, hogy a `charge()` metódus egyszer meghívódjon 100 összeggel; ha nem így történik, a teszt hibát jelez.
- **Fake:** Működő, de egyszerűsített implementáció, amelyet kifejezetten tesztelésre készítettek. Például egy memóriaalapú adatbázis, amely az adatokat egy listában tárolja.
- **Spy:** Rögzíti a hívásokat, de nem feltétlenül definiál előre elvárásokat; a teszt végén a rögzített hívások alapján ellenőrizhetünk. A mock és a spy közötti különbség finom: a spy inkább rögzít, a mock inkább elvárásokat ellenőriz.

A mock és a stub közötti különbség gyakorlati jelentősége:

A stub választ ad, de nem buktatja meg a tesztet akkor, ha a hívás elmarad. A mock viszont akkor is hibát jelez, ha a várt hívás nem történt meg, mert az elvárás megsértése a teszt kudarcát okozza. Ezért a mockot akkor használjuk, ha a teszt szempontjából fontos az interakció (például egy parancs elküldése), a stubot pedig akkor, ha csak a bemeneti adatot kell biztosítani.

Példa – stub és mock használata ugyanabban a tesztben:

Tegyük fel, hogy egy `OrderService` komponenst tesztelünk, amely a `PaymentGateway`-en keresztül fizetést indít, és a `CustomerRepository`-ból olvassa be az ügyfelet.

```
test_order_service_charges_vip_customer:
    customer_repo = StubCustomerRepository()
    customer_repo.set_customer("customer-1", Customer(vip=True))
    payment_gateway = MockPaymentGateway()
    payment_gateway.expect_charge(amount=80)

    service = OrderService(customer_repo, payment_gateway)
    service.place_order(customer_id="customer-1", product_price=100)
```

```
payment_gateway.verify()  # ellenőrzi, hogy a charge() valóban meghívódott 80 összeggel
```

Itt a `customer_repo` egy stub, mert csak a bemeneti adatot adja; a `payment_gateway` egy mock, mert az interakciót – a fizetési hívás megtörténtét és paraméterét – ellenőrizzük.

Tipikus hibák a test double-ök használatában:

- Túlzott mockolás: a teszt annyi mockot használ, hogy végül csak a mockok közötti interakciót ellenőrzi, nem a valódi komponenst. Ez a tesztet törékennyé és értelmetlenné teheti.
- A mock elvárások túl szigorúak: például a pontos paramétersorrend vagy belső állapot ellenőrzése, ami a refaktorálást akadályozza.
- A stub és a mock közötti különbség figyelmen kívül hagyása, ami félrevezető elnevezésekhez és hibás tesztekhez vezet.

25.2.7 Tesztmátrix készítése – a laborfeladat

A tesztmátrix egy olyan táblázat vagy dokumentum, amely a rendszer komponenseihez hozzárendeli a megfelelő tesztfajtákat, indoklással együtt. Célja, hogy a tesztstratégiát komponensszinten konkretizálja, és a csapat számára átláthatóvá tegye, melyik komponenshez milyen tesztek készülnek, milyen gyakorisággal futnak, és milyen környezetben.

A labor célja: A hallgató készítsen tesztmátrixot egy adott, kis méretű rendszer komponenseihez, és dokumentálja a döntéseit ADR formátumban. A feladat során a mért és a várt eredményt külön kell választani.

Kiinduló állapot:

Adott egy egyszerű rendszerleírás – használjuk a „Rendeléskezelő szolgáltatás” példát. A rendszer három belső komponensből és egy külső függőségből áll:

1. **REST API réteg** (`OrderController`): HTTP végpontokat biztosít a rendelés létrehozásához és lekérdezéséhez.
2. **Üzleti logika** (`OrderService`): a rendelés feldolgozását, kedvezményszámítást és a fizetési folyamat vezérlését végzi.
3. **Perzisztencia** (`OrderRepository`): a rendelések adatbázisban történő tárolását és betöltését végzi.
4. **Külső fizetési szolgáltató** (`PaymentGateway`): külső HTTP API, amely a fizetési tranzakciókat lebonyolítja.

A rendszerhatárt a C4 context szintjén a „Rendeléskezelő szolgáltatás” jelenti; a külső fizetési szolgáltató a rendszerhatáron kívül van.

Feladat:

Készítsen egy tesztmátrixot, amely minden belső komponenshez és a külső függőséghez meghatározza:

- Milyen tesztfajtákat (unit, integrációs, contract, E2E) érdemes hozzá rendelni?
- Mi az adott tesztfajta célja az adott komponens esetében?
- Milyen tesztkörnyezetben fut (fejlesztői gép, CI, staging)?
- Milyen gyakorisággal fut (minden commit, nightly, release előtt)?
- Milyen test double-öket használ, ha szükséges?

A mátrixot egy Markdown dokumentumban (vagy Jupyter-jegyzetfüzetben) kell elkészíteni. A dokumentumnak tartalmaznia kell:

- A rendszer C4 context vagy container szintű leírását (diagrams-as-code formában, például Mermaid `flowchart` vagy PlantUML).
- A tesztmátrixot táblázatos formában.
- Egy ADR-t a tesztstratégia fő döntéséről (például „Miért a tesztpiramis elvét követjük, és miért nem írunk minden komponenshez E2E tesztet?").
- A mátrix elkészítése során hozott legalább három további döntés ADR-bejegyzését (rövidített formában).
- A mért és a várt eredmény elkülönítését: a dokumentumban külön szakaszban szerepeljen, hogy mit várunk az egyes tesztfajtáktól (lefedettségi cél, hibafelfedési képesség), és mit mértünk valójában (ha futtattunk teszteket, a mért futási időt, lefedettséget, hibaszámot). Ha nem futtattunk, a mért rész üres marad, és ezt jelölni kell.

Elvárt eredmény:

Egy reprodukálható dokumentum, amelyből egy másik mérnök meg tudja érteni, hogy az adott rendszerhez miért ezt a tesztstratégiát választották, és hogyan lehet azt a jövőben aktualizálni. A dokumentumnak tartalmaznia kell az ellenőrzési szempontokat is, hogy a tesztmátrix helyességét értékelni lehessen.

Ellenőrzési szempontok:

- Lefedettség: minden belső komponenshez van-e legalább egy tesztfajta rendelve?
- Indoklás: minden hozzárendelés mellett szerepel-e a cél és a határ?
- Reprodukálhatóság: a dokumentum verziózott-e, és a döntések ADR-ben visszakereshetők-e?
- A mért és várt eredmény különválasztása megtörtént-e?
- A test double-ök használata indokolt-e, és nem túlzott-e?

Visszaállítási út:

Mivel a labor dokumentumkészítés, nincs szükség infrastruktúra visszaállítására. Ha a hallgató mégis futtatott teszteket, azoknak izolált környezetben kell futniuk, és a tesztadatok takarítását el kell végezni (például tesztadatbázis törlése). A labor nem igényel éles rendszerhez való hozzáférést.

A labor során használt eszközök: Bármilyen szövegszerkesztő, diagrams-as-code eszköz (Mermaid, PlantUML), és opcionálisan egy tesztkeretrendszer a példatészek futtatásához. A konkrét keretrendszer verzióját a hivatalos dokumentációból kell ellenőrizni.

25.2.8 Összefoglalás és ellenőrzőlista

Ebben az alfejezetben a négy alapvető tesztfajta – unit, integrációs, contract és E2E – célját és határait jártuk körbe. A legfontosabb tanulságok:

- A unit teszt a belső logikát izoláltan ellenőrzi; gyors, olcsó, de nem fedi le az integrációs hibákat.
- Az integrációs teszt a komponensek együttműködését ellenőrzi, átlépve a unit teszt határát; lassabb, ezért szelektíven érdemes használni.
- A contract teszt a fogyasztói elvárás és a provider közötti API-szerződést ellenőrzi, anélkül hogy a teljes rendszert futtatni kellene.
- Az E2E teszt a teljes felhasználói utat fedi le, de lassú és törékeny; csak a kritikus utakra érdemes koncentrálni.

- A test double-ök (dummy, stub, mock, fake, spy) közül a stub választ ad, a mock interakciót ellenőriz; a kettő közötti különbség meghatározza, hogy a teszt a bemenetet vagy a viselkedést ellenőrzi.

Ellenőrzőlista a tesztfajták kiválasztásához:

- ☐ Meghatároztam a tesztelendő komponens határát, és azon belül marad-e a teszt?
- ☐ A unit teszt nem éri el a valódi adatbázist, hálózatot vagy fájlrendszert?
- ☐ Az integrációs teszt környezete izolált és reprodukálható?
- ☐ A contract teszt szerződése a fogyasztói elvárást tükrözi, és nem a provider belső részleteit?
- ☐ Az E2E teszt a felhasználói szempontot ellenőrzi, nem a belső implementációt?
- ☐ A test double-ök típusa (stub, mock, fake) illeszkedik a teszt céljához?
- ☐ A tesztmátrix minden komponenshez hozzárendeli a megfelelő tesztfajtát, indoklással?
- ☐ A mért és a várt eredmény külön van választva a dokumentációban?

Önellenőrző kérdés:

Adott egy `UserService` komponens, amely a felhasználó regisztrációját végzi: ellenőrzi az e-mail formátumát, elmenti a felhasználót egy adatbázisba, és üdvözlő e-mailt küld egy külső SMTP szolgáltatón keresztül. Milyen tesztfajtákat választana ehhez a komponenshez, és miért? Hogyan használna test double-öket az egyes tesztfajtákban?

Ez a kérdés a fejezetben tanultak gyakorlati alkalmazására készteti az olvasót, és előkészíti a következő alfejezetet, amely a statikus elemzésre, a kódminőségre és a mutation testingre épít majd az itt bevezetett fogalmakra.

25.3 Statikus elemzés, kódminőség és mutation testing

A Statikus elemzés, kódminőség és mutation testing egység a tesztelési stratégia és minőségbiztosítás azon területére összpontosít, ahol a kód minőségét nem futás közben, hanem annak statikus tulajdonságai, szerkezete és a tesztek minőségén keresztül vizsgáljuk. A cél nem az, hogy a statikus elemzést a futtatott tesztek ellenfelévé tegyük, hanem az, hogy a visszacsatolási hurkot a lehető legkorábbra hozzuk: a kód beolvasásakor, a pull request nyitásakor vagy a lokális build során. Ebben a fejezetben a statikus elemzés három rétegét járjuk körbe: a stílus- és szerkezeti ellenőrzést (lint), a típushelyességet és a sebezhetőségi szabályokat, valamint a tesztek minőségét mérni képes mutation testinget. A fogalmakat egyetlen, kis méretű, mégis valósághű példán keresztül mutatjuk be, majd a hibákat és a gyakorlati döntési naplót is erre építjük.

25.3.1 1. A statikus elemzés helye a minőségbiztosítási stratégiában

A dinamikus tesztek a rendszer viselkedését vizsgálják: adott bemenetre adott kimenetet várnak, adott hibamódra adott helyreállást mérnek. A statikus elemzés ezzel szemben nem futtatja a programot, hanem a forráskódot, a bájtkódot, a függőségi gráfot vagy a konténerképet vizsgálja. A kettő nem helyettesíti egymást: a statikus elemzés képes olyan hibákat is jelezni, amelyeket a tesztek soha nem hívnak elő, míg a tesztek képesek olyan viselkedésbeli hibákat felfedni, amelyek a szintaktikailag helyes és típusbiztos kódban is jelen vannak.

A statikus elemzés értéke a visszacsatolási idő lerövidítésében rejlik. Egy típushiba vagy egy gyanús csomagfüggőség felderítése a CI-ben néhány másodperc vagy perc; ugyanez productionben már incidens. A statikus eszközök emellett ismételhető, emberi figyelmet nem igénylő ellenőrzést adnak: a kódoló nem tudja „elfelejteni” a szabályt, mert a pipeline nem engedi át a nem megfelelő kódot.

Ugyanakkor a statikus elemzésnek világos korlátai vannak. A hamis pozitívok – amikor az eszköz hibát jelez, pedig nincs valós hiba – bizalmatlanságot szülnek, és a fejlesztők elkezdik figyelmen kívül hagyni a riasztásokat. A hamis negatívok – amikor a hiba valós, de az eszköz nem jelzi – hamis biztonságérzetet adnak. A statikus elemzés tehát nem „igazság", hanem heurisztika: a szabályok a valószínű hibák és a valószínű kockázatok köré szerveződnek, és az eredményt mindig kontextusban kell értelmezni.

Fontos különbséget tenni a statikus elemzés tárgya szerint is. A linter általában a forráskód stilisztikai és szerkezeti tulajdonságait vizsgálja: elnevezések, behúzás, tiltott konstrukciók, bonyolultsági metrikák, üres kivételkezelők, nem használt változók. A típusellenőrző a programozási nyelv típusrendszerét használja a műveletek helyességének statikus bizonyítására: ha a függvény `int` értéket vár, de `string` értéket kap, a típusellenőrző hibát ad. A sebezhetőségi szabályok a kód azon mintáit keresik, amelyek ismert támadási felületet jelentenek: injekció, nem biztonságos deszerializáció, gyenge algoritmus, érzékeny adat naplózása, függőségi sérülékenység.

A három réteg nem éles határvonal. Egy modern statikus elemző keretrendszerben – legyen az SonarQube, ESLint, pylint, RuboCop, clang-tidy, Semgrep vagy CodeQL – a szabályok gyakran átfednek: egy szabály lehet stilisztikai és biztonsági egyszerre. A fontos az, hogy az architektúra szempontjából a statikus elemzést ne „kódformázásként" kezeljük, hanem a minőségi kapuk egyik első, automatikus ellenőrzési pontjaként.

25.3.2 2. Lint és kódminőségi szabályok

A lintelés a kód statikus tulajdonságainak mérésének legrégebbi és legszélesebb körben elterjedt formája. A név a Unix `lint` eszközből ered, amely a C nyelvű kódban keresett gyanús mintákat. Ma a „lint" kifejezést gyűjtőfogalomként használjuk minden olyan szabályra, amely a kód szerkezetét, stílusát vagy gyanús konstrukcióit vizsgálja anélkül, hogy a programot futtatná.

A lint szabályok két csoportra oszthatók. Az első csoport a szintaktikai stílusra és konzisztenciára vonatkozik: behúzás, sortörés, elnevezési konvenció, idézőjel-használat, sorhossz, idézőjel-típus. Ezek a szabályok elsősorban az olvashatóságot és a csapatszintű konzisztenciát szolgálják. A második csoport a gyanús kódmintákat keresi: elérhetetlen kód, üres kivételkezelő, a kivétel változójának figyelmen kívül hagyása, változó újradefiniálása, nem használt import, túl bonyolult feltétel, túl hosszú függvény, túl sok paraméter.

Az utóbbi csoport fontos architektúrális jelzéseket hordozhat. Ha a linter ismételten azt jelzi, hogy egy osztály túl nagy, egy függvény túl sok felelősséget hordoz, vagy egy modul túl sok függőséggel rendelkezik, az nem csupán stilisztikai probléma: a szerkezeti hiba tünete. A statikus elemzés így válik a moduláris határok és az elvek – például a kapcsolódás (coupling) és kohézió (cohesion) – folyamatos ellenőrzőjévé.

A gyakorlatban a lint szabályok három konfigurációs stratégia szerint épülnek fel. Az első a „zéró tolerancia": minden szabály be van kapcsolva, és minden riasztás blokkolja a buildet. Ez a megközelítés nagyon szigorú, és csak akkor működik, ha a szabályhalmaz kiforrott, a hamis pozitívok aránya alacsony, és a csapat elfogadja a korlátokat. A második a „fokozatos bevezetés": a meglévő kódra nem kényszerítjük ki az összes szabályt azonnal, hanem az új vagy módosított kódrészletekre igen. Ezt a legtöbb linter a „baseline" vagy „suppression" mechanizmussal támogatja. A harmadik a „kockázat alapú": csak a magas kockázatú szabályokat engedjük be, például a biztonsági problémákat, a típushibákat és a nyilvánvaló logikai hibákat.

Az architektúra szempontjából a lint konfiguráció nem csak technikai kérdés. Ha a lint szabályok túl szigorúak, a fejlesztők a szabályokat kikerülik: soronként letiltják a riasztást, vagy a buildet kerülik meg. Ha túl enyhék, a szabályok nem adnak valódi védelmet. A legjobb megoldás az, ha

a lint szabályhalmazt version control alatt, a kódbázissal együtt kezeljük: a szabályok változása ugyanolyan felülvizsgálat alá esik, mint a kód változása.

A fejezet keretein belül a lint szabályokat egyetlen, kis Python-modulon mutatjuk be. A példa szándékosan egyszerű, hogy a szabályok működése világos legyen:

```
# UserRepository: in-memory felhasználói tár
# A kód szándékosan tartalmaz lint-szempontból javítandó mintákat.
import os, json
from typing import Optional

class UserRepository:
    def __init__(self):
        self._items = {}

    def GetUser(self, user_id):
        if user_id == None:
            raise Exception("user_id is required")
        return self._items.get(user_id)

    def save(self, user_data):
        id = user_data.get("id")
        if id == None:
            id = user_data.get("email")
        self._items[id] = user_data

    def load_from_file(self, path):
        with open(path, 'r') as f:
            data = json.load(f)
            for item in data:
                self.save(item)
```

Ebben a kódban a lint szabályok több problémát is jelezhetnek: az `import os, json` sortörés és csoportosítás szempontjából kifogásolható; a `GetUser` metódus elnevezése Pythonban `snake_case` konvenciót vár; az `if user_id == None` helyett `is None` ajánlott; a `save` metódusban az `id` változó elfedi a beépített `id` nevet; az `id = user_data.get("email")` ág nélkül nem ellenőrizzük, hogy az e-mail valóban létezik-e, így `None` kulcs is bekerülhet; a `load_from_file` nem kezeli a fájl hiányát vagy a hibás JSON-t.

Ezek közül egyik sem „működési hiba” a szó szoros értelmében – a kód lefut, és adott bemenetre működik –, de mindegyik a későbbi hibák melegágya. A lintelés feladata, hogy ezeket a mintákat emberi beavatkozás nélkül jelezze.

25.3.3 3. Típusellenőrzés: a hibák egy osztályának statikus bizonyítása

A típusellenőrzés a lintelésnél erősebb garanciát ad. A típusrendszer a programozási nyelv azon szabályrendszere, amely a kifejezések típusait és a műveletek megengedett kombinációit írja le. Ha a kód megsérti ezt a szabályrendszert, a típusellenőrző hibát ad, még mielőtt a program futna. A típusellenőrzés így a hibák egy egész osztályát – a típushibákat – statikusan kizárja.

A típusrendszerek kifejezőereje nyelvenként eltérő. A Python dinamikus típusú nyelv, de a típusannotációk és a mypy, pyright vagy pyre használatával statikus típusellenőrzés végezhető. A JavaScript TypeScripttel kap típusrendszert. A Java, C#, Go, Rust, Kotlin, Scala beépített statikus típusrendszerrel rendelkeznek. A típusrendszer erőssége – mennyire képes a programozó szándékát kifejezni – jelentősen befolyásolja, hogy a típusellenőrző milyen hibákat tud megfogni.

Nézzük a fenti Python-kód típusannotált változatát:

```python
# UserRepository: in-memory felhasználói tár, típusannotációkkal.
import json
from typing import Dict, Any, Optional

class UserRepository:
    def __init__(self) -> None:
        self._items: Dict[str, Dict[str, Any]] = {}

    def get_user(self, user_id: str) -> Optional[Dict[str, Any]]:
        if user_id is None:
            raise ValueError("user_id is required")
        return self._items.get(user_id)

    def save(self, user_data: Dict[str, Any]) -> None:
        user_id: Optional[str] = user_data.get("id")
        if user_id is None:
            user_id = user_data.get("email")
        if user_id is None:
            raise ValueError("user_data must contain id or email")
        self._items[user_id] = user_data

    def load_from_file(self, path: str) -> None:
        with open(path, 'r', encoding='utf-8') as f:
            data = json.load(f)
            for item in data:
                self.save(item)
```

A típusannotációk után a típusellenőrző képes ellenőrizni, hogy a `save` metódus `Dict[str, Any]` típusú argumentumot vár, a `get_user` pedig `str` típusú azonosítót. Ha a hívó oldalon a fejlesztő tévedésből számot ad át, a típusellenőrző hibát jelez. A típusrendszer tehát a komponensek közötti szerződések egy részét statikusan kikényszeríti.

A típusellenőrzés korlátja, hogy nem képes minden helyességi tulajdonságot ellenőrizni. A típusrendszer például nem tudja bizonyítani, hogy a `load_from_file` metódus valóban fájlt olvas, vagy hogy a JSON-tartalom megfelel a várt sémának. A típusellenőrző nem helyettesíti a szerződésteszteket vagy a sémavalidációt.

Fontos különbséget tenni a típusellenőrzés és a típuskövetkeztetés között. A típusellenőrzés a programozó által adott annotációk alapján bizonyít, a típuskövetkeztetés pedig annotációk nélkül is képes a kifejezések típusát kiszámítani. A gyakorlatban a kettő kombinációja adja a legerősebb garanciát: a programozó a publikus interfészeket annotálja, a belső, lokális változókat pedig a típuskövetkeztetésre bízza.

25.3.4 4. Sebezhetőségi szabályok és sérülékenységi szkennelés

A sebezhetőségi szabályok a statikus elemzés azon rétegét alkotják, amely kifejezetten a biztonsági kockázatokat keresi. Ezek a szabályok is a kód statikus tulajdonságait vizsgálják, de a szándékuk nem a stílus vagy a típushelyesség, hanem a támadási felület csökkentése.

A sebezhetőségi szabályok három csoportba sorolhatók. Az első a forráskódban lévő veszélyes mintákat keresi: SQL-injekció, parancsinjektálás, elérési útvonal átjárás (path traversal), nem biztonságos deszerializáció, gyenge kriptográfiai algoritmus, érzékeny adat naplózása. A második a függőségi fát vizsgálja: a csomag vagy a konténerkép tartalmaz-e ismert sebezhetőséget, ismert rosszindulatú csomagot vagy elavult, támogatás nélküli komponenst. A harmadik az infrastruktúra-kódot és a konfigurációt vizsgálja: a Kubernetes-manifesztben privilege escalation, a Dockerfile-ban `latest` tag, a Terraform-kódban nyílt biztonsági csoport.

A sebezhetőségi szabályok kulcsfontosságú jellemzője, hogy a jelzett problémák súlyozása eltér a lint riasztásokétól. Egy stílushiba nem blokkolhatja a production kiadást, egy SQL-injekció viszont igen. A sebezhetőségi szabályok ezért gyakran külön kapuban, eltérő súlyozással futnak: a kritikus és magas besorolású problémák blokkolnak, a közepes és alacsony besorolásúak jegyet generálnak, vagy a kockázat alapján döntenek.

A sebezhetőségi szkennelés egyik fontos területe a függőségi sebezhetőség. A modern szoftverek jellemzően több száz vagy több ezer közvetlen és közvetett csomagtól függenek. Minden egyes csomag potenciális támadási felület. A függőségi szkennerek – mint a Dependabot, a Renovate, az OWASP Dependency-Check, a Trivy vagy a Grype – a függőségi zárfájlt és a telepített csomagokat összevetik az ismert sebezhetőségi adatbázisokkal, például a CVE- vagy a GHSA-adatbázissal.

A sebezhetőségi szabályok konfigurációjánál fontos a kontextus. Egy sebezhetőség attól függően kritikus, hogy az adott komponens milyen szerepet tölt be, milyen hálózati határon fut, és milyen adathoz fér hozzá. A statikus szabály ezt a kontextust önmagában nem ismeri, ezért a fejlesztőnek kell eldöntenie, hogy a riasztás valós-e. A sebezhetőségi szabály tehát nem helyettesíti a threat modelinget, hanem annak operatív, folyamatos kiegészítője.

A fejezet példájában a sebezhetőségi szabályok a `load_from_file` metódusra mutathatnak rá. A fájl elérési útját a hívó adja meg; ha a hívó felhasználói bemenetet ad át, az elérési út átjárást tesz lehetővé. A sebezhetőségi szabály jelezheti, hogy a fájlmegnyitásnál ellenőrizni kell az elérési utat, vagy a metódust csak megbízható forrásból származó elérési úttal szabad hívni.

25.3.5 5. Mutation testing: a tesztek minőségének mérése

A hagyományos tesztlefedettség – sor, ág, függvény – azt mutatja, hogy a teszt a kód mely részeit futtatja. Nem mutatja, hogy a teszt valóban ellenőrzi is a viselkedést. A mutation testing ezt a hiányosságot célozza meg: a technika hibákat (mutációkat) visz be a kódba, majd futtatja a teszteket, és azt vizsgálja, hogy a tesztek elkapják-e a hibát. Ha a teszt lefut ugyan a mutáns kódon, de nem bukik el, akkor a teszt nem elég erős: a kód megváltozott, a viselkedés valószínűleg hibás lett, de a teszt ezt nem vette észre.

A mutációs analízis alapfogalma a mutáns. A mutáns a program egy olyan változata, amelyben egyetlen kis műveletet megváltoztatunk: az `if` feltételben az `==` operátort `!=`-re cseréljük, a `+` operátort `-`-re, a `return x` kifejezést `return None`-ra, vagy a változó értékét konstanssal helyettesítjük. A mutációs tesztelő eszköz létrehozza a mutánsok halmazát, mindegyikre futtatja a teszteket, és megnézi, hogy a teszt bukik-e.

A mutáns két kimenetelű lehet. Ha a teszt elbukik a mutánson, a mutánst „megöltük”: a teszt érzékelte a hibát. Ha a teszt sikeresen lefut a mutánson, a mutáns „túlélte”: a teszt nem vette észre a viselkedésváltozást. A mutation score a megölt mutánsok aránya az összes mutánshoz képest. A magas mutation score azt jelzi, hogy a tesztek erősek: a kód viselkedésének kis megváltoztatását is észreveszik. Az alacsony mutation score azt jelzi, hogy a tesztek gyengék: a kód fut, a fedettség lehet magas, de a tesztek nem ellenőrzik a valódi viselkedést.

A mutation testing legnagyobb gyakorlati problémája a számításigény. Ha a programnak ezer mutánsa van, a teszteket ezerszer kell futtatni. Ez a gyakorlatban percekig vagy órákig tart, ezért a mutációs analízist ritkán futtatják minden commitra. A gyakorlati minta az, hogy a mutációs analízist időszakosan, a tesztek minőségének auditjaként futtatjuk, vagy a CI-ben csak a megváltozott kódrészletre korlátozzuk.

Nézzük a példánkat. A `UserRepository.save` metódus első változatában a mutációs eszköz megváltoztathatja az `if id == None` feltételt `if id != None`-re. Ha a teszt nem bukik el, az azt jelenti, hogy a teszt nem ellenőrzi, hogy a `None` azonosító elutasításra kerül-e, illetve hogy az

e-mail alapú azonosító-képzés működik-e. Hasonlóan, a `load_from_file` metódusban a mutáció megváltoztathatja a ciklus határát: a teszt akkor öli meg a mutánst, ha ellenőrzi, hogy a fájl összes eleme bekerül-e a tárba.

A mutációs tesztelés nem a hagyományos tesztlefedettség helyettesítője, hanem annak kiegészítője. A fedettség azt mutatja, mit futtat a teszt; a mutációs analízis azt, hogy a teszt mennyire hatékony. A kettő együtt ad teljes képet: a fedettség a kód feltérképezésére szolgál, a mutációs analízis pedig a tesztek szigorúságának mérésére.

25.3.6 6. A statikus elemzés pipeline-ba illesztése

A statikus elemzési eszközöket nem elegendő telepíteni; a folyamatba is be kell illeszteni. A gyakorlati minta a réteges visszacsatolás:

1. **Lokális szint:** a fejlesztő a kód írása közben kap azonnali visszajelzést. A linter és a típusellenőrző az IDE-ben fut, a hibákat a szerkesztő jelzi. Ez a leggyorsabb visszacsatolás, de nem kikényszeríthető.
2. **Commit szint:** a fejlesztő a commit előtt hook-ot futtat, amely a lintert és a típusellenőrzőt indítja. Ez a visszacsatolás másodpercek alatt lefut, és megakadályozza a nyilvánvaló hibák bekerülését a közös ágba.
3. **CI szint:** a pipeline a merge request vagy pull request során futtatja a teljes statikus elemzést, a sebezhetőségi szkennert és a teszteket. Ez a szint kikényszeríthető: ha a minőségi kapu nem teljesül, a merge nem lehetséges.
4. **Időszakos audit:** a mutációs tesztelést, a teljes függőségi szkennelést és a sebezhetőségi mélyelemzést éjszakai vagy heti ütemezésben futtatjuk, mert lassúak és erőforrás-igényesek.

A rétegezés célja a visszacsatolási idő minimalizálása. Minél hamarabb kap a fejlesztő jelzést, annál olcsóbb a javítás. Ugyanakkor a gyors rétegek nem adnak teljes garanciát: a lokális linter futás nem helyettesíti a CI-t, mert a fejlesztő kikerülheti, a CI pedig nem helyettesíti az auditot, mert az audit olyan szempontokat is vizsgál, amelyek a mindennapi fejlesztéshez túl lassúak.

A pipeline-ba illesztésnél fontos döntés, hogy a statikus elemzési kapu blokkolja-e a merge-et, vagy csak riasztást ad. A gyakorlatban a következő elv javasolt: a magas bizonyosságú, alacsony hamis pozitív arányú szabályok – például a típushiba – blokkoljanak; a heurisztikusabb, hamis pozitívra hajlamos szabályok – például a bonyolultsági metrika – figyelmeztessenek, de ne blokkoljanak. A kettő közötti határ csapatonként és projektanként eltér, és a szabályhalmaz finomhangolását folyamatosan kell végezni.

25.3.7 7. Gyakorlati példa: a `UserRepository` statikus elemzése

Tegyük fel, hogy a `UserRepository` kód egy termékszintű felhasználói modul része. A modul célja, hogy a felhasználói adatokat belső memóriában tárolja, és fájlból is be tudja tölteni azokat. A modul a következő linter-, típus- és mutációs elemzéseken megy keresztül.

A linter a következőket jelzi a javítatlan kódon:

Szabály	Kategória	Súlyosság	Észrevétel
`unused-import`	hibagyanú	közepes	Az `os` import nincs használva.
`invalid-name`	stílus	alacsony	A `GetUser` metódus neve nem `snake_case`.

Szabály	Kategória	Súlyosság	Észrevétel
`singleton-comparison`	hibagyanú	magas	A `== None` helyett `is None` használata ajánlott.
`redefined-builtin`	hibagyanú	közepes	Az `id` változó elfedi a beépített `id` függvényt.
`broad-exception`	hibagyanú	közepes	A `raise Exception` túl általános; pontosabb kivétel ajánlott.

A típusellenőrző a javítatlan kódon nem tud futni, mert nincsenek típusannotációk. A típusannotációk bevezetése után a típusellenőrző a következőket jelzi:

- A `save` metódusban a `user_data.get("id")` és a `user_data.get("email")` értéke `Any` típusú, mert a `user_data` típusa `Dict[str, Any]`. Ez megengedett, de a `Dict[str, Any]` helyett specifikusabb típust érdemes használni: `Dict[str, Union[str, int]]` vagy egy `UserData` osztály.
- A `get_user` metódusban a `user_id` típusa `str`, de a hívó oldalon a típusellenőrző hibát ad, ha a hívó `int` értéket ad át.

A sebezhetőségi szabályok a következőket jelzik:

- A `load_from_file` metódus a hívó által megadott elérési utat olvassa; ha a hívó felhasználói bemenetet ad át, az elérési út átjárást tesz lehetővé.
- A `load_from_file` metódus nem korlátozza a fájl méretét; egy túl nagy fájl erőforrás-kimerülést okozhat.
- A `UserRepository` nem korlátozza a `save` metóduson keresztül beérkező adatok méretét vagy típusát, ezért felhasználói bemenet esetén DoS-támadásra ad lehetőséget.

A mutációs elemzés a következőket vizsgálja. A kiinduló teszt csak azt ellenőrzi, hogy a `save` metódus hívása után a `get_user` metódus a megfelelő értéket adja vissza:

```
# Minta teszt, amely a UserRepository viselkedését ellenőrzi.
import pytest
from user_repository import UserRepository

def test_save_and_get():
    repo = UserRepository()
    repo.save({"id": "u1", "email": "u1@example.test", "name": "U1"})
    assert repo.get_user("u1") == {"id": "u1", "email": "u1@example.test", "name": "U1"}
```

Ez a teszt nem ellenőrzi a `None` azonosító elutasítását, az e-mail alapú azonosító-képzést, a fájlból való betöltést vagy a hibás bemenetet. A mutációs elemzés ennek megfelelően túlélő mutánsokat talál: ha a `save` metódusban a `user_id = user_data.get("id")` sort megváltoztatjuk `user_id = None`-ra, a teszt továbbra is lefut, mert a teszt csak a `"u1"` azonosítót ellenőrzi, amelyet a `get` hívás a `"id"` kulcsból nyer. A mutáns túlélése azt jelzi, hogy a teszt nem ellenőrzi a `save` metódus azon ágát, amely az `id` hiányát kezeli.

25.3.8 8. A gyakori hibák és azok felismerése

A statikus elemzés gyakorlati bevezetése során a csapatok több tipikus hibába esnek. Az első a szabályok halmozása a prioritások nélkül. A linter és a sebezhetőségi eszköz letöltésekor az összes elérhető szabály bekapcsolása ugyan nagy lefedettséget ígér, de a gyakorlatban a hamis pozitívok áradatához és a riasztások figyelmen kívül hagyásához vezet. A helyes megközelítés a kockázat alapú szelekció: a csapat először a magas kockázatú, alacsony hamis pozitív arányú szabályokat kapcsolja be, majd fokozatosan bővíti a kört.

A második hiba a riasztások globális letiltása a kódban. Ha a fejlesztő a `# noqa` vagy `// ignore` kommentet használja, hogy elhallgattassa a lintert, elveszti a szabály védelmét. A helyes minta az, hogy a letiltást indokolni kell, és a letiltást a lehető legszűkebb körre kell alkalmazni: nem az egész fájlra, hanem az adott sorra, és nem a teljes szabályhalmazra, hanem a konkrét szabályra.

A harmadik hiba a mutációs tesztelés félreértelmezése. A mutációs score nem a program minőségét méri, hanem a tesztek minőségét. A magas mutációs score nem jelenti azt, hogy a program hibamentes; azt jelenti, hogy a tesztek erősek. A mutációs tesztelés nem helyettesíti a funkcionális teszteket, a biztonsági teszteket vagy a teljesítménytesteket.

A negyedik hiba a statikus elemzés eredményének fetisizálása. A riasztások száma nem cél, hanem jelzés. Ha a csapat a riasztások számának csökkentésére optimalizál a hibák valódi kijavítása helyett, a minőségi mutató hamis biztonságérzetet ad. A statikus elemzés akkor hasznos, ha a riasztások emberi döntésekhez kapcsolódnak: a fejlesztő megérti a riasztás okát, és dönt a javításról vagy az indokolt elfogadásról.

25.3.9 9. ADR: a statikus elemzési szabálykészlet kialakítása

Az alábbi ADR rögzíti a fejezet példájában hozott döntést.

Cím: ADR-2025-01 – Statikus elemzési szabálykészlet a `UserRepository` modulra

Állapot: Elfogadva
Kontextus: A modul a felhasználói adatok tárolását végzi. A kód stilisztikailag inkonzisztens, típusannotációk nélküli, és a sebezhetőségi szabályok potenciális elérési út átjárást jeleznek. A jelenlegi tesztkészlet egyetlen boldogúti esetet fed le, és a mutációs elemzés szerint a mutánsok jelentős része túlél.
Döntés: Bevezetjük a lint szabályok kockázat alapú részhalmazát, a típusannotációkat a publikus interfészeken, a sebezhetőségi szabályok közül az elérési út ellenőrzését, és a mutációs elemzést időszakos auditként futtatjuk.
Alternatívák:
- Az összes elérhető szabály bekapcsolása; elvetettük a hamis pozitívok magas aránya miatt.
- A statikus elemzés teljes mellőzése; elvetettük, mert a korai visszacsatolás csökkenti a hibajavítási költséget.

Következmények: A fejlesztőknek a lint és típusellenőrzés hibáit ki kell javítaniuk a commit előtt. A sebezhetőségi szabályok figyelmeztetést adnak, de nem blokkolnak, amíg a hívó oldali ellenőrzés nem készül el. A mutációs elemzés heti ütemezésben fut, és a csapat a mutációs score-t a tesztek minőségének mutatójaként használja.

25.3.10 10. Labor: statikus elemzési szabálykészlet kialakítása

Cél: A résztvevő alakítson ki egy kisméretű, reprodukálható statikus elemzési szabálykészletet a `UserRepository` modulra, és dokumentálja a döntéseket.

Kiinduló állapot: A `UserRepository` a fenti, javítatlan kód. A résztvevő a kedvenc nyelvében, a megfelelő eszközökkel dolgozik. Python esetén pylint, mypy és pytest-mutagen vagy mutmut; Java esetén Checkstyle, SpotBugs és PIT; JavaScript esetén ESLint, TypeScript és Stryker.

Elvárt eredmény:
- A linter konfigurációs fájl, amely a kockázat alapú szabályhalmazt tartalmazza.
- A típusellenőrző konfiguráció, amely a publikus interfészekre kötelező annotációt ír elő.
- A sebezhetőségi szabályok közül legalább egy szabály beállítása.
- A mutációs elemzés futtatása legalább a `save` metóduson, a megölt és túlélő mutánsok listájával.
- Döntési napló, amely rögzíti a szabályok kiválasztásának indokát, a hamis pozitívok kezelését és a mutációs score értelmezését.

Ellenőrzési szempontok:
- A linter valós hibát jelez a javítatlan kódon.
- A típusellenőrző ténylegesen hibát ad, ha a hívó `int` értéket ad át `str` helyett.
- A sebezhetőségi szabály jelzi a fájl elérési útjának ellenőrzésének hiányát.
- A mutációs elemzés legalább egy túlélő mutánst azonosít, és a résztvevő megmagyarázza, miért élte túl a mutáns a tesztet.
- A döntési napló elkülöníti a mért eredményt a várt eredménytől.

Visszaállítási út: A labor nem igényel éles rendszert; a résztvevő a saját lokális környezetében dolgozik. A konfigurációs fájlok és a kód a verziókezelőben visszaállíthatók. A mutációs elemzés izolált ideiglenes mappában fut, hogy a generált mutánsok ne szennyezzék a forráskódot.

25.3.11 11. Önellenőrző kérdések

- Mi a különbség a lint és a típusellenőrzés között? Melyik osztályból melyik hibát fogja meg?
- A tesztek 100%-os sorlefedettséget értek el. Mit mondhatunk a mutációs score-ról? Miért nem elegendő a fedettség önmagában?
- Mikor blokkoljon a sebezhetőségi szabály, és mikor csak figyelmeztessen? Milyen elv alapján döntesz?
- Miért futtatjuk a mutációs elemzést időszakosan, nem minden commitra?
- Hogyan kerüljük el a riasztások globális letiltását a kódban?

25.3.12 12. Összefoglalás

A statikus elemzés a minőségbiztosítás legkorábbi, leggyorsabb visszacsatolási hurka. A lint a kód stilisztikai és szerkezeti tulajdonságait méri; a típusellenőrző a hibák egy osztályát statikusan kizárja; a sebezhetőségi szabályok a támadási felületet csökkentik. A mutációs tesztelés mindezek mellett a tesztek minőségét méri: megmutatja, hogy a tesztek ténylegesen ellenőrzik-e a viselkedést, vagy csak lefedik a kódot. A statikus elemzési eszközök akkor érnek el valódi minőségjavulást, ha a szabályhalmazt kockázat alapúan választjuk, a riasztásokat emberi döntés követi, és a mutációs score-t a tesztek minőségének mutatójaként, nem a program hibamentességének bizonyítékaként értelmezzük.

25.4 Teljesítmény-, biztonsági és chaos tesztelés

A nem funkcionális tesztelés célja nem csupán az, hogy hibát találjunk a kódban, hanem hogy feltárjuk, a rendszer hogyan viselkedik akkor, amikor a környezet vagy a terhelés eltér a fejlesztői gépen megszokottól. A funkcionális tesztek azt igazolják, hogy a rendszer helyesen számol, megfelelően tárol és visszaad; a nem funkcionális tesztek azt vizsgálják, hogy a rendszer ezt a helyes működést megfelelő sebességgel, rendelkezésre állással, biztonságosan és hibatűrően teszi-e. A különbség nem pusztán technikai, hanem szemléletbeli: a nem funkcionális tulajdonságok gyakran statisztikai jellegűek, környezetfüggőek, és csak akkor értelmezhetők, ha előre rögzítjük a mérési feltételeket és az elfogadási küszöböket. Ebben a fejezetben a teljesítmény-, biztonsági és chaos tesztelés alapjait járjuk körbe, a korábbi fejezetben megismert tesztpiramis és teszt-trofea kontextusába illesztve őket. Nem ismételjük a funkcionális tesztelés alapjait; a hangsúly a nem funkcionális követelmények ellenőrzésén, a mérési fegyelmen és a kockázatok tudatos kezelésén van.

25.4.1 A nem funkcionális tesztelés kerete: minőségi attribútumoktól a mérhető forgatókönyvekig

A nem funkcionális tesztelés kiindulópontja mindig egy minőségi attribútum, például a válaszidő, az áteresztőképesség, a hibatűrés, a biztonsági ellenálló képesség vagy a hosszú távú stabilitás. A könyv korábbi fejezeteiben bevezetett quality attribute scenario (minőségi forgatókönyv) az a szerződés, amely összeköti a követelményt a tesztelhető elvárással. Egy ilyen forgatókönyv hat részből áll: inger, ingerforrás, környezet, műtermék, válasz és válaszmérő. A tesztelés során ezt a forgatókönyvet kell reprodukálni, és a válaszmérőt mérni. Például:

> **Forgatókönyv:** 1000 egyidejű felhasználó böngészi a webáruház katalógusát csúcsidőben (inger: katalóguslistázási kérés, ingerforrás: felhasználók, környezet: normál üzem, műtermék: katalógusszolgáltatás). A rendszernek minden kérésre 200-as státusszal kell válaszolnia, és a p95-ös válaszidő nem haladhatja meg a 300 ms-ot (válasz). A mérést 5 perces ablakban, 10 perc bemelegítés után, három ismétlésben végezzük, és a mediánt tekintjük eredménynek (válaszmérő).

Ez a forgatókönyv adja a terheléses teszt specifikációját. Hasonlóan, egy biztonsági forgatókönyv lehet például: „Egy külső, hitelesítetlen támadó a bejelentkezési űrlapon SQL injectiont kísérel meg (inger). A rendszernek a kísérletet naplóznia kell, és a támadás nem érhet el adatbázis-hozzáférést (válasz). A teszt során a sebezhetőség-szkenner nem jelent kritikus vagy magas kockázatú találatot (válaszmérő).” A chaos kísérlet forgatókönyve pedig így hangozhat: „Egy pod váratlanul leáll a katalógusszolgáltatásban (inger). A rendszer 30 másodpercen belül új podot indít, és a felhasználói kérések hibaaránya nem haladja meg a 0,1%-ot (válasz). A mérés a kísérlet kezdete utáni 5 perces ablakban történik (válaszmérő).”

A nem funkcionális tesztelés kerete tehát négy lépésből áll:

1. **Cél és minőségi forgatókönyv rögzítése.** Melyik attribútumot vizsgáljuk, mi az inger, mi a mérőszám, mi az elfogadási küszöb.
2. **Környezet és kiinduló állapot meghatározása.** Milyen hardveren, hány csomóponton, milyen adatbázis-mérettel, milyen konfigurációval fut a rendszer. A környezet legyen reprodukálható, ideális esetben infrastruktúra mint kód (IaC) alapján létrehozva.
3. **Mérési terv készítése.** Milyen metrikákat gyűjtünk (pl. p95, p99, throughput, hibaarány, CPU, memória, hálózati késleltetés), milyen gyakorisággal, milyen ablakban, és hány ismétlést végzünk.

4. **Eredmények értékelése és dokumentálása.** A várt és a mért értékeket elkülönítve, a mérés bizonytalanságával együtt rögzítjük. A nem futtatott mérést nem szabad ellenőrzött eredményként bemutatni – ezt a könyv dokumentációs szerződése is kiemeli.

A nem funkcionális tesztelés egyik legfontosabb buktatója, hogy a tesztkörnyezet és a produkciós környezet közötti különbség torzítja az eredményeket. Ezért a tesztterben mindig fel kell tüntetni, hogy a környezet miben tér el a produkciótól (például kisebb adatbázis, kevesebb csomópont, szintetikus adatok), és ennek milyen hatása lehet az eredményre. A cél nem a tökéletes egyezés, hanem a torzítás irányának és mértékének tudatosítása.

25.4.2 Terhelés, stressz és tartósság: a teljesítmény három arca

A teljesítménytesztelésen belül három, jól elkülöníthető vizsgálati mód létezik, amelyeket gyakran összemosnak. A **terheléses teszt (load test)** a rendszer viselkedését a várható, normál üzemi terhelés mellett vizsgálja. A **stresszteszt** a rendszert a tervezett kapacitás fölé viszi, hogy feltárja a töréspontot, a degradáció módját és a helyreállás képességét. A **tartóssági teszt (soak/endurance test)** mérsékelt, de hosszan fenntartott terhelést alkalmaz, hogy felfedje a memóriaszivárgást, a naplófájl-növekedést, a cache-ek elhasználódását, az adatbázis-kapcsolatok elfogyását és más, időben kibontakozó hibákat. A három üzemmód más-más kérdésre ad választ, ezért egy érett tesztstratégia mindhármat tartalmazza.

25.4.2.1 Terheléses teszt: az SLO-k ellenőrzése

A terheléses teszt célja annak igazolása, hogy a rendszer a meghatározott terhelési profil mellett teljesíti a szolgáltatási szintű célkitűzéseket (SLO). A terhelési profilt a valós forgalom elemzéséből kell származtatni: hány felhasználó, milyen arányú kéréstípusok, milyen időbeli eloszlásban (például reggeli csúcs, hétköznapi nappali alapforgalom). A profil lehet egyszerű konstans terhelés, de hitelesebb, ha lépcsőzetesen növekvő vagy valós eloszlásból mintavételezett. A teszt során a következő metrikákat érdemes mérni:

- **Válaszidő-percentilisek:** p50, p95, p99. A p95 azt jelenti, hogy a kérések 95%-a gyorsabban teljesült, mint a megadott érték. A percentilisek robusztusabbak, mint az átlag, mert kevésbé torzítja őket a kiugró lassú kérés.
- **Áteresztőképesség (throughput):** másodpercenkénti kérésszám vagy tranzakciószám.
- **Hibaarány:** az 5xx vagy időtúllépés miatt sikertelen kérések aránya.
- **Erőforrás-kihasználtság:** CPU, memória, hálózati sávszélesség, lemez I/O, adatbázis-kapcsolatok száma.

A terheléses tesztet a minőségi forgatókönyvben rögzített mérési ablakban kell futtatni. Például egy 5 perces mérési ablak előtt 10 perc bemelegítést iktatunk be, hogy a gyorsítótárak feltöltődjenek és a JIT optimalizáció lezajlódjon. A mérést legalább háromszor érdemes megismételni, és a mediánt venni, hogy a véletlen ingadozást csökkentsük.

A terheléses teszt eszközei közül a gyakorlatban elterjedt a k6, a JMeter, a Gatling vagy a Locust. Az alábbi k6 szkript szemlélteti egy egyszerű terhelési forgatókönyv definiálását:

```
// Szemléltető k6 szkript – nem éles rendszerhez
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    catalog_load: {
```

```
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '5m', target: 100 },  // bemelegítés
        { duration: '5m', target: 1000 }, // mérési ablak
        { duration: '2m', target: 0 },    // leállás
      ],
    },
  },
  thresholds: {
    http_req_duration: ['p(95)<300'],   // p95 < 300 ms
    http_req_failed: ['rate<0.001'],    // hibaarány < 0,1%
  },
};

export default function () {
  const res = http.get('https://shop.example.internal/catalog?page=1');
  check(res, { 'status was 200': (r) => r.status === 200 });
  sleep(1); // gondolkodási idő
}
```

Ez a példa a `shop.example.internal` helyőrző URL-t használ, és a `thresholds` szekcióban rögzíti az elfogadási kritériumokat. Fontos, hogy a terhelési profilt ne csak a kérések számával, hanem a kérések közötti gondolkodási idővel (think time) is modellezzük, különben a mesterségesen szoros egymásutániság torzítja az eredményt.

25.4.2.2 Stressztesztt: a töréspont keresése

A stressztesztt célja nem az SLO ellenőrzése, hanem annak megismerése, hogy mi történik, ha a terhelés meghaladja a tervezett maximumot. A stressztesztt során fokozatosan növeljük a terhelést, és figyeljük, hogy a rendszer melyik metrikája romlik el először: a válaszidő nő, a hibaarány emelkedik, vagy a rendszer erőforrásai kimerülnek. A stressztesztt értékes eredménye nem egy szám, hanem egy leírás: „A rendszer 1200 egyidejű felhasználóig lineárisan skálázódik, 1300-nál a p99 válaszidő 800 ms fölé emelkedik, 1400-nál az adatbázis-kapcsolatok száma kimerül, és a hibaarány 5% fölé ugrik.” Ez a tudás elengedhetetlen a kapacitástervezéshez és a riasztási küszöbök beállításához.

A stressztesztt során a rendszert szándékosan a töréspontig visszük, ezért a tesztet izolált környezetben kell futtatni, és a teszt után a környezetet vissza kell állítani. A mérési jegyzőkönyvben rögzíteni kell, hogy a töréspontot milyen körülmények között értük el, és a rendszer hogyan viselkedett a túlterhelés alatt: degradálódott-e fokozatosan, vagy hirtelen omlott össze? A fokozatos degradáció kívánatosabb, mert lehetővé teszi a beavatkozást, míg a hirtelen összeomlás nehezen kezelhető.

25.4.2.3 Tartóssági teszt: az időben kibontakozó hibák

A tartóssági teszt mérsékelt, de hosszú ideig – tipikusan órákig vagy akár napokig – fenntartott terhelést alkalmaz. Célja azoknak a hibáknak a feltárása, amelyek csak hosszabb futás után jelentkeznek: memóriaszivárgás, fájlleírók elfogyása, naplófájlok korlátlan növekedése, cache-ek elöregedése, adatbázis-táblák fragmentálódása, üzenetsorok feltorlódása. A tartóssági teszt során a metrikákat időbeli trendként vizsgáljuk: ha a memóriahasználat monoton növekszik, akkor valószínűleg szivárgás van; ha a válaszidő lassan, de folyamatosan nő, akkor valamilyen erőforrás elfogyóban van.

A tartóssági teszt leghasznosabb metrikái a hosszú távú trendek. Érdemes a teszt alatt periodikusan mintát venni a heap-ből, a GC logokból, a nyitott kapcsolatok számából. A Little-törvény itt is segíthet: ha a rendszerben lévő kérések átlagos száma L folyamatosan emelkedik, miközben az érkezési ráta λ és a tartózkodási idő W állandó, akkor valami nincs rendben az erőforrás-felszabadítással. A képlet $L = \lambda W$ rámutat arra, hogy a növekvő L vagy növekvő λ-t, vagy növekvő W-t jelent; az utóbbi gyakran a lassuló válaszidő jele. A tartóssági teszt tehát nemcsak a hibát mutatja meg, hanem a diagnózis irányát is.

25.4.2.4 Gyakori hibák a teljesítménytesztelésben

A terheléses, stressz- és tartóssági teszteknél a következő hibák gyakran előfordulnak, és az eredmények félreértelmezéséhez vezetnek:

- **Nem reprezentatív terhelési profil.** Ha a teszt csak egyetlen URL-t hív, miközben a valós forgalomban többféle kéréstípus és gondolkodási idő van, az eredmény nem általánosítható.
- **Hideg cache és bemelegítés hiánya.** Az első kérések lassabbak, mert a gyorsítótár még üres. Ha a mérési ablak az indulástól számít, a p95 érték rosszabb lesz a valósnál.
- **Túl rövid mérési ablak.** A rövid tesztek nem tárják fel a fluktuációt és a lassan kibontakozó hibákat.
- **Koordinált torzítás (coordinated omission).** Ha a kliens a válasz megérkezése után azonnal küldi a következő kérést, akkor a lassú válaszok kimaradnak a mintából, mert a kliens „nem ér rá” mérni őket. A k6 és JMeter alapértelmezésben kezeli ezt, de egyéni szkripteknél ügyelni kell rá.
- **A tesztkörnyezet és a produkció közötti különbség figyelmen kívül hagyása.** A kisebb adatbázis, a kevesebb csomópont vagy a szintetikus adatok torzítják az eredményeket. A torzítás irányát mindig dokumentálni kell.
- **A metrikák átlagolása.** Az átlag válaszidő elfedi a kiugró értékeket. Percentiliseket kell használni.

Ezeknek a hibáknak a tudatosítása azért fontos, mert a teljesítménytesztelés eredménye gyakran döntések alapja: kapacitástervezés, riasztási küszöbök, skálázási szabályok. Ha az eredmény torz, a döntés is rossz lesz.

25.4.3 Biztonsági tesztelés: a támadási felület szisztematikus vizsgálata

A biztonsági tesztelés célja nem a támadás, hanem a rendszer védelmi mechanizmusainak ellenőrzése. A tesztelés során sebezhetőségeket keresünk, amelyek kihasználása lehetővé tenné a rendszer kompromittálását, az adatok kiszivárgását vagy a szolgáltatás megzavarását. A biztonsági tesztelés a tesztpiramis tetején helyezkedik el, mert drága és körültekintést igényel, de a modern fejlesztési gyakorlatban egyre több eleme automatizálható és a CI/CD folyamatba integrálható (shift-left).

A biztonsági tesztelés főbb típusai:

- **SAST (Static Application Security Testing):** a forráskódot elemzi, ismert mintákat keres (pl. SQL injection, XSS, hard-coded secret). A korábbi fejezetben tárgyalt statikus elemzéshez hasonlóan a build folyamatban fut.
- **DAST (Dynamic Application Security Testing):** a futó alkalmazást vizsgálja kívülről, mintha támadó lenne. A sebezhetőségi szkennerek (pl. OWASP ZAP, Burp Suite) HTTP kéréseket küldenek, és a válaszokat elemzik. A DAST a futtatható rendszert teszteli, ezért a tesztkörnyezetben kell futtatni.

- **IAST (Interactive Application Security Testing):** a SAST és DAST kombinációja, az alkalmazás belsejéből figyeli a futtatást, és azonosítja a sebezhetőséget kihasználó adatfolyamot.
- **SCA (Software Composition Analysis):** a függőségek ismert sebezhetőségeit ellenőrzi. Ez különösen fontos a nyílt forráskódú könyvtárak széles körű használata miatt.
- **Penetrációs teszt (pentest):** manuális, szakértő által végzett vizsgálat, amely a sebezhetőségek kihasználhatóságát és üzleti hatását értékeli.

A biztonsági tesztelés folyamata a fenyegetésmodellezéssel kezdődik: azonosítjuk a rendszer bizalmi határait, a belépési pontokat, a kritikus adatfolyamokat, és meghatározzuk, mely fenyegetések a legvalószínűbbek vagy legnagyobb hatásúak. Ezután a tesztelési terv ezekre a fenyegetésekre összpontosít. Például ha a rendszer bejelentkezési űrlappal rendelkezik, a tesztelésnek le kell fednie a hitelesítési hibákat, a session kezelést, az SQL injectiont, a brute force támadást. Ha a rendszer fájlokat tölt fel, a feltöltött fájlok típusát és méretét kell ellenőrizni. Ha a rendszer harmadik féltől származó API-t hív, az API hívások hitelesítését és a válaszok érvényesítését kell vizsgálni.

A DAST szkennerek szemléltető használata az alábbi parancssorral történhet (OWASP ZAP példa, helyőrző célponttal):

```
# Szemléltető DAST futtatás - izolált tesztkörnyezetben
zap.sh -daemon -port 8090
zap.sh -cmd -quickurl https://shop.example.internal -quickprogress
```

Ez a példa a `shop.example.internal` helyőrző URL-t használja, és a szkennelést egy izolált környezetben futtatja. A biztonsági tesztelésnél különösen fontos, hogy a tesztet ne éles rendszeren végezzük, mert a szkennelés forgalmat generál, és adatokat módosíthat. A tesztkörnyezetben is gondoskodni kell arról, hogy a tesztadatok szintetikusak legyenek, és ne tartalmazzanak személyes adatot.

A biztonsági tesztelés elfogadási kritériumait érdemes a kockázati besoroláshoz kötni: például „nincs kritikus vagy magas sebezhetőség”, „a közepes sebezhetőségek száma nem növekszik”, „a titkok nem szerepelnek a verziókezelőben”. A sebezhetőségek javítását követően a tesztet meg kell ismételni, hogy a javítás hatékonyságát igazoljuk.

A biztonsági tesztelés etikai határai is világosak: a tesztelés célja a védekezés, nem a támadás. A könyv nem ad támadási útmutatót; a sebezhetőségek kihasználásának részleteit nem tárgyalja. A tesztelőnek mindig rendelkeznie kell a vizsgált rendszer tulajdonosának engedélyével, és a tesztet csak izolált környezetben, meghatározott határidővel és visszaállítási tervvel végezheti.

25.4.4 Chaos tesztelés: a hibatűrés kísérleti igazolása

A chaos tesztelés – más néven chaos engineering – szándékos hibainjektálással vizsgálja, hogyan viselkedik a rendszer váratlan meghibásodások esetén. Célja nem a hibák keresése, hanem a rendszer hibatűrő képességének és helyreállási mechanizmusainak igazolása. A chaos tesztelés alapelve az, hogy a komplex elosztott rendszerekben a meghibásodások elkerülhetetlenek, ezért a rendszert úgy kell kialakítani, hogy azokra jól reagáljon. A chaos tesztelést a korábbi hibamód-elemzésre (Hibamódok és failure mode analysis) építve érdemes végezni: először azonosítjuk a lehetséges hibamódokat, majd ezek közül kísérletileg vizsgáljuk a legkritikusabbakat.

A chaos kísérlet tervezésének sarkalatos elemei:

- **Steady-state hipotézis (stacionárius állapot hipotézis).** Mielőtt a hibát injektáljuk, rögzítjük, hogy mi a rendszer normál viselkedése: milyen a válaszidő, a hibaarány, az áteresztőképesség. A hipotézis megfogalmazható a quality attribute scenario mintájára: „A katalógusszolgáltatás p99 válaszideje 500 ms alatt van, és a hibaarány 0,1% alatt, amikor nincs injektált hiba.” A kísérlet során ezt a hipotézist ellenőrizzük.
- **Blast radius (robbanási sugár).** A kísérlet hatásának köre. A chaos tesztelésnél a kísérletet fokozatosan kell kiterjeszteni: először csak egy podra, majd egy egész szolgáltatásra, de soha nem az egész rendszerre egyszerre. A blast radius minimalizálása csökkenti a kár kockázatát.
- **Hibainjektálás módja.** A hiba lehet infrastrukturális (pod törlése, CPU/memória korlátozás, hálózati késleltetés), alkalmazási (lassú válasz, hibás válasz, kivétel) vagy adatbázisszintű (master failover, kapcsolat megszakadás). Az injektálás történhet manuálisan, vagy célzott eszközökkel, például a Chaos Mesh, a Litmus vagy a Gremlin segítségével.
- **Megfigyelés és mérés.** A kísérlet alatt a rendszer metrikáit folyamatosan gyűjtjük, és a steady-state hipotézistől való eltérést vizsgáljuk. A mérési ablak a hiba injektálása előtt, alatt és után is fontos.
- **Visszaállítási terv.** Minden kísérlet előtt meg kell határozni, hogyan állítjuk vissza a rendszert az eredeti állapotba, ha a kísérlet váratlanul nagy kárt okoz.

Egy chaos kísérlet szemléltető YAML definíciója a Kubernetes környezetben (Chaos Mesh):

```yaml
# Szemléltető Chaos Mesh kísérlet - pod-törlés
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: catalog-pod-kill
spec:
  action: pod-kill
  mode: one
  selector:
    namespaces:
      - shop-staging
    labelSelectors:
      app: catalog-service
  duration: '10s'
```

Ez a definíció egyetlen podot töröl a `shop-staging` névtérben, 10 másodperces időtartammal. A kísérlet során figyeljük, hogy a Kubernetes új podot indít-e, és a felhasználói kérések hibaaránya a megadott küszöb alatt marad-e.

A chaos tesztelés gyakorlati végrehajtása előtt érdemes egy úgynevezett **game day**-t tartani: egy dedikált időszakot, amikor a csapat szándékosan hibákat injektál, és közösen figyeli a rendszer viselkedését. A game day célja nemcsak a rendszer tesztelése, hanem a csapat felkészítése az incidenskezelésre. A gyakorlat során a csapat megtanulja, hogyan ismerje fel a hibát, hogyan riasszon, és hogyan állítsa helyre a szolgáltatást.

A chaos tesztelés kockázatait tudatosan kell kezelni. A kísérletet soha nem szabad éles rendszeren, felkészületlen csapattal végezni. A kísérletnek mindig legyen egyértelmű abort kritériuma: ha a hibaarány egy meghatározott küszöb fölé emelkedik, a kísérletet azonnal le kell állítani. A kísérlet után a rendszert ellenőrizni kell, hogy visszatért-e a normál állapotba.

25.4.5 Labor: nem funkcionális teszterv készítése

A labor célja, hogy a hallgató a fejezetben megismert fogalmakat egy konkrét, kis méretű rendszeren alkalmazza, és elkészítsen egy reprodukálható, döntési naplóval ellátott nem funkcionális

teszttervet. A labor nem követeli meg a tesztek tényleges futtatását; a hangsúly a tervezés minőségén és a mérési fegyelmen van. A hallgató választhat egy korábbi labor projektet, vagy használhatja a megadott minta rendszert: egy egyszerű webáruház katalógusszolgáltatás, amely két szolgáltatásból áll (katalógus és rendelés), egy relációs adatbázissal, Kubernetes környezetben. A minta rendszer leírása, a konténerállományok és a telepítési jegyzékek a labor kiinduló állapotát képezik.

Kiinduló állapot: A rendszer funkcionális tesztjei zöldek, a statikus elemzés és a mutation testing a korábbi fejezet szerint lefutott. A nem funkcionális követelmények a következő minőségi forgatókönyvekben vannak megfogalmazva:

1. **Válaszidő csúcsterhelés alatt:** 800 egyidejű felhasználó mellett a katalógus listázási kérés p95 válaszideje < 300 ms, a hibaarány $< 0{,}1\%$, 5 perces mérési ablakban, 10 perc bemelegítés után.
2. **Rendelkezésre állás pod-kiesés esetén:** egy katalógus-pod váratlan leállásakor a rendszer 30 másodpercen belül új podot indít, és a felhasználói kérések hibaaránya nem haladja meg a 0,1%-ot a kísérletet követő 5 percben.
3. **Biztonsági ellenállás SQL injection ellen:** egy hitelesítetlen támadó a katalógus keresőmezőben SQL injectiont kísérel meg; a rendszer a kísérletet naplózza, és a DAST szkenner nem jelent kritikus vagy magas sebezhetőséget.

Elvárt eredmény: Egy dokumentum (Markdown, Jupyter notebook vagy PDF), amely a következő részeket tartalmazza:

- **Teszt kontextus:** a rendszer leírása, a tesztkörnyezet specifikációja (pl. Kubernetes verzió, csomópontok száma, adatbázis mérete, konfiguráció), a tesztelés időpontja.
- **Minőségi forgatókönyvek:** a fenti három forgatókönyv pontosítása, a mérési ablakokkal és elfogadási küszöbökkel.
- **Tesztstratégia:** a terheléses, stressz- és tartóssági teszt terve; a biztonsági teszt terve (DAST és SCA); a chaos kísérlet terve (pod-törlés, erőforrás-korlátozás).
- **Mérési terv:** a gyűjtendő metrikák, a mintavételi gyakoriság, a mérési ablak, az ismétlések száma, a statisztikai feldolgozás módja.
- **Döntési napló (ADR):** legalább két ADR, amely a teszttervezés során hozott fontos döntéseket rögzíti (pl. miért a k6-ot választottuk a JMeter helyett, miért a pod-törlést választottuk a hálózati késleltetés helyett).
- **Ellenőrzési szempontok:** egy ellenőrzőlista, amely alapján a teszterv teljessége és helyessége értékelhető.
- **Eredmények elkülönítése:** a dokumentumban külön szekciók tartalmazzák a várt eredményeket és a ténylegesen mért eredményeket (utóbbi a labor során nem szükséges, de a sablonnak tartalmaznia kell a helyét).

Visszaállítási út: A labor során, ha a hallgató mégis futtat teszteket egy izolált környezetben, a visszaállítást a következő módon kell elvégezni: a teszt által létrehozott erőforrásokat (podok, szolgáltatások, tesztadatok) törölni kell, és a környezetet az IaC jegyzékek alapján újra kell telepíteni. A tesztadatok csak szintetikusak lehetnek.

Ellenőrzési szempontok a labor értékeléséhez:

- A minőségi forgatókönyvek megfelelnek-e a quality attribute scenario formátumnak (inger, környezet, válasz, mérőszám)?
- A tesztstratégia lefedi-e mindhárom terhelési módot (terhelés, stressz, tartósság)?
- A biztonsági teszt magában foglalja-e a DAST és az SCA elemzést?
- A chaos kísérlet definiálja-e a steady-state hipotézist és a blast radius-t?
- A mérési terv rögzíti-e a mérési ablakot, az ismétlések számát és a statisztikai feldolgozást?

- A dokumentum elkülöníti-e a várt és a mért eredményeket?
- Az ADR-ek tartalmazzák-e a kontextust, a döntést, az alternatívákat és a következményeket?

A laborban készített teszterv nemcsak a fejezet tanulási céljait ellenőrzi, hanem előkészíti a következő fejezetben tárgyalt technikai adósság és minőségbiztosítás témakörét is: a nem funkcionális tesztek hiánya gyakran technikai adósságként jelenik meg, mert a rendszer nem teljesíti a nem funkcionális követelményeket, de ezt csak akkor tudjuk meg, ha mérünk.

25.4.6 Ellenőrzőlista a nem funkcionális teszteléshez

Az alábbi ellenőrzőlista a fejezetben tárgyalt legfontosabb szempontokat foglalja össze, és használható a teszterv vagy a tesztelési folyamat önértékeléséhez:

- **Célok és forgatókönyvek:** Minden tesztnek van minőségi forgatókönyvben rögzített célja és elfogadási küszöbe?
- **Környezet:** A tesztkörnyezet specifikációja reprodukálható és tartalmazza a produkciótól való eltéréseket?
- **Terhelési profil:** A terheléses teszt profilja valós forgalmi adatokon alapul, beleértve a kéréstípusok arányát és a gondolkodási időt?
- **Percentilisek:** A válaszidőt percentilisekkel mérjük, nem átlaggal?
- **Bemelegítés és mérési ablak:** A teszt tartalmaz bemelegítési szakaszt, és a mérési ablak elég hosszú a statisztikai érvényességhez?
- **Stresszteszt:** A stresszteszt azonosítja a töréspontot és a degradáció módját?
- **Tartóssági teszt:** A tartóssági teszt hosszú távú trendeket figyel, például memóriaszivárgást?
- **Biztonsági teszt:** A teszt lefedi a SAST, DAST, SCA és szükség esetén pentest tevékenységeket?
- **Chaos kísérlet:** A kísérlet rendelkezik steady-state hipotézissel, blast radius-szal, abort kritériummal és visszaállítási tervvel?
- **Etika és engedély:** A biztonsági és chaos tesztek kizárólag izolált környezetben, engedéllyel, szintetikus adatokkal futnak?
- **Dokumentáció:** A mérési jegyzőkönyv rögzíti a környezetet, konfigurációt, időpontot, a mérés bizonytalanságát, és elkülöníti a várt és mért eredményeket?

25.4.7 Önellenőrző kérdés

Egy e-kereskedelmi rendszerben a csapat úgy dönt, hogy a terheléses tesztet a fejlesztői laptopon futtatja, mert „a Kubernetes klaszter drága, és a laptop is elég gyors”. A teszt azt mutatja, hogy a rendszer 1000 egyidejű felhasználót is kiszolgál 200 ms p95 válaszidő alatt. A produkcióba állítás után azonban a rendszer már 400 felhasználónál időtúllépéseket produkál. Mi lehet a hiba a tesztelési megközelítésben, és hogyan kellett volna eljárni? Indokolja meg, hogy a fejlesztői laptop és a produkciós környezet közötti különbségek mely metrikákat torzíthatják, és miért nem elegendő a terheléses teszt önmagában a produkciós teljesítmény garantálásához.

Válasz vázlata: A hiba a tesztkörnyezet és a produkció közötti szakadék figyelmen kívül hagyása. A laptop erőforrásai (CPU, memória, hálózat) eltérnek a produkciós csomópontokétól; a lokális környezetben nincs hálózati késleltetés a szolgáltatások között, az adatbázis mérete és a cache-ek is mások. A 1000 egyidejű felhasználó a laptopon nem reprezentatív, mert a kliens és a szerver ugyanazon a gépen fut, ami elrejti a hálózati szűk keresztmetszeteket. A helyes eljárás az lett volna, ha a tesztet a produkcióhoz hasonló, izolált staging környezetben futtatják, a tesztkörnyezet és a produkció közötti különbségeket dokumentálják, és a terheléses tesztet stressz-

és tartóssági tesztekkel egészítik ki. Emellett a mérési eredményeket csak a környezet pontos ismeretében szabad értékelni; a laptopon mért 200 ms p95 nem jelenti azt, hogy a produkcióban is teljesülni fog.

25.4.8 Összefoglalás

A nem funkcionális tesztelés a rendszer minőségi attribútumainak kísérleti ellenőrzése. A terheléses, stressz- és tartóssági tesztek a teljesítmény és a kapacitás kérdéseire adnak választ; a biztonsági tesztelés a támadási felületet vizsgálja; a chaos tesztelés a hibatűrő képességet igazolja. Mindhárom terület közös alapelve a mérési fegyelem: a célokat minőségi forgatókönyvekben rögzítjük, a környezetet és a mérési ablakot pontosan specifikáljuk, a várt és mért eredményeket elkülönítjük, és a döntéseket ADR-ben dokumentáljuk. A nem funkcionális tesztek nem egyszeri tevékenységek, hanem a folyamatos minőségbiztosítás részei; a következő fejezetben látni fogjuk, hogy a technikai adósság gyakran abból fakad, hogy ezek a tesztek hiányoznak, vagy eredményeiket figyelmen kívül hagyják.

26 Refaktorálás, technikai adósság és legacy modernizáció

26.1 Technikai adósság és kódminőség

26.1.1 Technikai adósság: típusok, kamat és a törlesztés tudománya

A szoftverfejlesztés gazdasági és mérnöki valósága elválaszthatatlan a „technikai adósság” fogalmától. Az elnevezés egy metafora, amelyet Ward Cunningham vezetett be az 1990-es évek elején. A hasonlat lényege, hogy a szoftverrendszerben tudatosan vagy tudattalanul meghozott, rövid távon előnyös, de hosszú távon költséges döntések úgy viselkednek, mint egy pénzügyi kölcsön. Ahogy a hitel kamata növeli a tőke visszafizetésének terhét, a technikai adósság „kamata” is folyamatosan növeli a rendszer továbbfejlesztésének belső költségeit.

Egy architektúra akkor tekinthető egészségesnek, ha a benne rejlő adósság tudatosan vállalt, nyilvántartott, és a törlesztése ütemezett. Ha az adósság nem látható, akkor a rendszer a csőd (leállás, sikertelen release, felrobbanó komplexitás) irányába halad. Ebben a fejezetben a technikai adósság típusait, a „kamat” és a „tőke” megkülönböztetését, valamint a mérhetőség és a nyilvántartás gyakorlatát vizsgáljuk meg.

26.1.1.1 A technikai adósság természete és típusai

Mielőtt bármilyen törlesztési terve készítenénk, meg kell értenünk, hogy nem minden adósság egyforma. A hiba, amit a legtöbb szervezet elkövet, az, hogy azonosítja a rossz kódot, de nem tesz különbséget aközött, hogy miért jött létre az adósság, és milyen hatással van a rendszerre. Ezt a felosztást legjobban a Martin Fowler által népszerűsített „Technical Debt Quadrant” modell magyarázza el, amely két tengely mentén osztályozza az adósságot: a szándékosság (szándékos vs. nem szándékos) és a megfontoltság (megfontolt vs. meggondolatlan) alapján.

1. Szándékos és megfontolt adósság (Deliberate & Prudent) Ez az a helyzet, amikor a csapat tudatosan hoz döntést a technikai minőség feláldozásáról egy időbeli vagy költségbeli előnyért cserébe. Például egy induló startup a gyors piacra lépés érdekében szándékosan kihagyja az adatbázis-sémák szigorú verziózását, és egy monolitikus rendszerben, egyszerűbb, de nem teljesen skálázható módon tárolja az adatokat. A csapat tisztában van vele, hogy ezt később refaktorálni kell. Ez a „jó” adósság, feltéve, hogy a döntést dokumentálják (például ADR-ben), és a kamat mértékét folyamatosan nyomon követik.

2. Szándékos és meggondolatlan adósság (Deliberate & Reckless) Ez akkor következik be, amikor a csapat a határidők nyomása alatt tudatosan figyelmen kívül hagyja az ismert tervezési mintákat, de nem dokumentálja a hiányosságot, és nem tervezi meg a javítást. A tipikus mondat: *„Nincs időnk a helyes megoldásra, csak rakjunk be egy gyors hotfixet, majd úgyis megcsináljuk.”* A „majd” szó jelzi a meggondolatlanságot. Ez a fajta adósság rendkívül gyorsan halmozódik fel, mert a következő fejlesztő, aki a hibás kódot látja, azt fogja gondolni, hogy ez az elfogadott kódolási standard, és ő is hasonló minőségben fog dolgozni (ez a megtört ablakok elve).

3. Nem szándékos és megfontolt adósság (Inadvertent & Prudent) Ez a típus a tanulási folyamat elkerülhetetlen melléktermé ke. A csapat a legjobb tudása szerint, a kor aktuális legjobb gyakorlatait követve épített egy megoldást, de időközben a technológia fejlődött, vagy a rendszer működése közben derült ki, hogy a korábbi modell nem elég hatékony. Például egy csapat néhány évvel ezelőtt NoSQL adatbázist választott, mert az akkori követelmények szerint jól skálázódott, de ma már komplex joinokra és erős konzisztenciára lenne szükség. Ez nem a csapat hibája, hanem a változó környezet következménye. Az ilyen adósság felismeréséhez időszakos architektúra-felülvizsgálatra van szükség.

4. Nem szándékos és meggondolatlan adósság (Inadvertent & Reckless) Ez a legveszélyesebb kategória. Akkor jön létre, amikor a csapat egyszerűen nem rendelkezik a szükséges ismeretekkel a jó kód megírásához, de nem is keresi a tudást. Hiányzik a tesztelési kultúra, nem ismerik a rétegzett architektúrát, a modulok közötti függőségek kaotikusak, de a csapat nem is tudja, hogy ez probléma. A kód minősége romlik, a hibák száma nő, de mivel nincs mérés, a vezetőség csak azt látja, hogy a csapat egyre lassabban szállít. A meggondolatlan adósság felhalmozódása általában a rendszer teljes leállásához vagy egy kényszerű, kockázatos nagy átíráshoz (big bang rewrite) vezet.

26.1.1.2 A tőke és a kamat közgazdaságtana

A technikai adósság elemzésekor kiemelten fontos a **tőke (principal)** és a **kamat (interest)** elkülönítése. A tőke maga a meglévő, minőségi problémákkal küzdő kód. A kamat az a többletköltség, amelyet a tőke jelenléte okoz a mindennapi fejlesztési munkában.

A kamat mérése a gyakorlatban: A kamat a legjobban a fejlesztési ciklusidő (Cycle Time vagy Lead Time) növekedésében és a hibajavításokra fordított idő arányában érhető tetten. Tegyük fel, hogy egy jól felépített modulban egy új mező hozzáadása egy űrlaphoz és az adatbázisrétegbe 2 órát vesz igénybe. Ugyanez a feladat egy technikai adóssággal terhelt, erősen összekapcsolt modulban 8 órát vesz igénybe, mert módosítani kell a tranzakciókezelést, az ORM leképezést, és több érthetetlen tesztek nélküli kódot kell kerülgetni. A 6 óra különbség a technikai adósság *kamata*.

A kamat másik formája a megbízhatóság csökkenése. Ha a kód minősége rossz, a tesztek hiányosak, akkor minden új funkció kiadása magasabb hibavalószínűséget hordoz. A hiba elhárítása, a rollback, az ügyfélszolgálati terhelés és a reputációs kár mind a kamat részei.

Miért nem mindig kifizetődő azonnal törleszteni? A törlesztés itt a refaktorálás, a tesztek pótlása vagy a build folyamat optimalizálása. A törlesztés erőforrásokat (időt, pénzt, fejlesztői kapacitást) von el az új funkciók fejlesztésétől. A döntés tehát mindig egy optimalizálási feladat. Ha a hitel kamata nagyon alacsony (például egy olyan modult, amit szinte soha nem módosítanak, senki nem nyúl hozzá), akkor lehet, hogy okosabb döntés a rendelkezésre álló kapacitást nem a tökéletes minőség elérésére, hanem bevételt generáló funkciókra fordítani. Ha azonban a kamat magas (a modult naponta módosítják), akkor a törlesztés megtérülése általában nagyon gyors.

26.1.1.3 A technikai adósság mérése

Az adósságot nem elég érezni, mérni kell. Az architekt szerepe itt az, hogy a szubjektív érzeteket objektív, reprodukálható mérőszámokká alakítsa. A mérés célja nem a fejlesztők megbüntetése, hanem a láthatóság megteremtése. A méréshez négy fő forrást használunk:

1. Statikus kódminőségi mutatók (Kódmetrikák) A forráskód elemzésekor a fordítói absztrakcióktól függetlenül mérhetünk. Ilyen mutatók például:

- **Ciklomatikus komplexitás (Cyclomatic Complexity):** Megmutatja, hogy egy függvényben hány lineárisan független útvonal van. Ha egy függvény ciklomatikus komplexitása meghalad egy bizonyos értéket (általában 10-15), akkor azt a függvényt szinte lehetetlen átlátni és tesztelni.
- **Kódduplikáció (Duplication):** A copy-paste programozás az egyik legdrágább kamatot termeli. Ha egy hiba található a duplikált kódban, azt minden másolatban javítani kell, ami hatványozottan növeli a hibázás esélyét.
- **Csatoltság és Kohézió:** Ahogy a `sa008` fejezetben tárgyaltuk, a magas csatoltság és az alacsony kohézió akadályozza a modulok önálló tesztelését és fejlesztését. A csatoltságot a modulok közötti függőségek számával és erősségével mérhetjük.
- **Architektúraszabályok megsértése:** Előfordul, hogy a kód egy olyan rétegből hivatkozik egy másikra, amelyet a tervezés tiltott. Például egy domain réteg közvetlenül egy HTTP kliens függvényt hív. Az ilyen szabálysértések méréséhez architecture fitness function-öket vagy statikus elemző eszközök szabályait használhatjuk.

2. Kódchurn (Változási gyakoriság) A verziókezelő rendszer (például Git) naplója kiváló forrás az adósság feltérképezésére. A *churn* azt mutatja meg, hogy egy fájl hányszor változott egy adott időablakban. Önmagában a churn nem probléma, de ha egy magas ciklomatikus komplexitású fájl magas churnnel rendelkezik, az egy „forró pont” (Hotspot). A kutatások azt mutatják, hogy a hibák többsége ezekben a forró pontokban koncentrálódik. A mérést gyakran ábrázolják egy kétdimenziós hőtérképen (`heatmap`), ahol az egyik tengely a komplexitás, a másik a churn, a szín pedig a becsült kockázat.

3. Tesztlefedettség és tesztminőség A tesztlefedettség hiánya indikátora lehet az adósságnak, de óvatosan kell bánni vele. A magas lefedettségi százalék nem jelenti azt, hogy a tesztek jók, csupán azt, hogy a kód végrehajtásra került. A *mutation testing* (amelyet a `sa097` fejezetben érintettünk) jobb jelzés arra vonatkozóan, hogy a tesztek képesek-e valóban hibát találni. Az adósság szempontjából a hiányzó tesztek a kód megváltoztatásának kockázatát jelentik. Ha nincs teszt egy kritikus üzleti szabályra, akkor a fejlesztő nem mer hozzányúlni a kódhoz, ami lelassítja a fejlesztést.

4. Folyamatmetrikák (Szállítási sebesség) A technikai adósság végső soron a szállítási sebességben csapódik le. Figyeljük a következőket:

- **Cycle Time:** Mennyi idő telik el a commit és a produkciós bevezetés között? Ha ez folyamatosan növekszik, miközben a csapat létszáma változatlan, az valószínűleg a növekvő adósság kamatának tudható be.
- **Defect Escape Rate:** Hány hiba jut ki a felhasználókhoz? Ha a hibaarány nő, a fejlesztők egyre több időt töltenek hibajavítással ahelyett, hogy új funkciókat fejlesztenének.
- **MTTR (Mean Time to Repair):** Mennyi idő alatt tudjuk elhárítani a hibát? A rosszul strukturált kódban nehezebb megtalálni a hiba forrását.

26.1.1.4 Gyakorlati minta: Adósságnyilvántartás (Debt Ledger)

Ahhoz, hogy a technikai adósság kezelése ne csak szólam legyen, hanem folyamatos tevékenység, be kell vezetni az **adósságnyilvántartást**. Ez a dokumentum (vagy ticketing rendszerbeli gyűjtemény) az adósságok „könyvelése”.

Az alábbi táblázat egy jól strukturált adósságnyilvántartás minimális sémáját mutatja be egy fiktív számlázási szolgáltatás (Billing Service) példáján:

ID	Típus (Szándékos / Nem szándékos / Meggondolatlan)	Hely (Modul, Fájl)	Leírás (A tőke)	Kamat (Milyen hatással van a munkára?)	Becsült kockázat / Gyakoriság	Törlesztési terv
TD-001	Szándékos, Megfontolt	`Billing.DomainA`	Az számlázási szabályok az üzleti logika rétegben vannak elszórva, nincs dedikált szabálymotorként kiszervezve.	Új piacok nyitásakor nehézkes az adószabályok hozzáadása, a változtatások gyakran hibát okoznak.	Magas	Q3: Szabályok kiszervezése, például Strategy pattern vagy szabálymotor bevezetése.
TD-002	Nem szándékos, Megfontolt	`Data Access`	A régi ORM leképezés nem támogatja az async I/O műveleteket.	A nagy terhelésű lekérdezések blokkolják a szálakat, ami miatt a skálázás drága.	Közepes	A kritikus lekérdezések áttérése async adatbáziskezelőre.
TD-003	Szándékos, Meggondolatlan	`Controller`	A fizetési webhook kezelőben nincs input validáció, mert „gyorsan kellett szállítani”.	Biztonsági kockázat (malformed input), magas hibalehetőség.	Kritikus	**Azonnal:** Validáció hozzáadása.

Megjegyzés a táblázathoz: A fenti „Törlesztési terv” nem részletes refaktorálási útmutató, hanem a döntés irányát jelöli ki; a részletes kivitelezés a `sa099` fejezet tárgyköre.

26.1.1.5 Prioritási terv készítése

Ha az adósságnyilvántartás elkészült, nem szabad minden tételt egyszerre törleszteni. Ez a teljes termék leállásához vezetne. A prioritást három dimenzió mentén érdemes kialakítani:

1. **Kamat mértéke (Impact):** Milyen gyakran érünk hozzá az adott területhez? Ha egy kód részletet havonta egyszer módosítanak, a kamat alacsony. Ha naponta, akkor magas.
2. **Kockázat (Risk):** Milyen üzleti hatása van, ha a kód meghibásodik? Egy pénzügyi tranzakciót kezelő modul adóssága kritikusabb, mint egy belső adminisztrációs felületé.

3. **Blokkoló hatás:** Az adósság akadályozza-e egy fontos, közelgő roadmap elem megvalósítását?

A prioritási tervet ezek alapján egy mátrixban ábrázolhatjuk. A *magas kamat + magas kockázat* kategóriába tartozó tételeket azonnal (vagy a következő iterációban) törleszteni kell. A *magas kamat + alacsony kockázat* tételeket érdemes a rutin fejlesztési munkákba beépíteni. Az *alacsony kamat + magas kockázat* tételeknél mérlegelni kell a kompenzáló intézkedéseket (például extra monitoring). Az *alacsony kamat + alacsony kockázat* tételeket pedig legtöbbször soha nem fizetjük vissza, egyszerűen csak megvárjuk, amíg az adott kódrész természetes módon kivezetésre kerül.

26.1.1.6 Gyakori hibák és azok elkerülése

1. hiba: Az adósság démonizálása Sok szervezetben a „technikai adósság” kifejezés szitokszó, a fejlesztők pedig szégyellnek beszélni róla. Ez arra ösztönzi az embereket, hogy elrejtőkék az adósságot, ami a legrosszabb forgatókönyv. *Helyes megoldás:* Az adósságot a mérnöki munka természetes részének kell tekinteni, amit nyíltan kell regisztrálni. A hangsúly a tudatosságon van, nem a bűntudaton.

2. hiba: A metrikák fetisizálása Az, hogy a SonarQube vagy hasonló eszköz pirosra festi a kódot, még nem jelenti azt, hogy azonnal cselekedni kell. Sok csapat abba a csapdába esik, hogy a 0 db kódduplikációt és a 100%-os lefedettséget tűzi ki célul. *Helyes megoldás:* A cél nem a tökéletes kód, hanem az elfogadható szállítási sebesség és megbízhatóság. Mindig össze kell kapcsolni a kódmetrikát az üzleti hatással.

3. hiba: A tőke és a kamat összekeverése A vezetőség hajlamos a technikai adósságra úgy tekinteni, mint egy egyszeri kiadásra (tőke), amit „majd egyszer kifizetünk”. Valójában a technikai adósság nagy része folyamatos, ismétlődő költség (kamat), amely minden egyes nap csökkenti a hatékonyságot. *Helyes megoldás:* Az architekt feladata, hogy a bemutatókban ne az adósság tőkéjét (hány sor kódot kell megírni, hány órát kell rászánni), hanem a kamatot (minden feature 30%-kal lassabban készül el) kommunikálja a döntéshozók felé.

4. hiba: A refaktorálás elkülönítése a mindennapi munkától Ha a technikai adósság törlesztését külön projektekbe, „takarító sprintekbe” szervezik, az gyakran hatástalan. A munkát újra kell tervezni, és a kontextus elveszik. *Helyes megoldás:* A törlesztést a *Boyscout Rule* („hagyd tisztábban a tábort, mint ahogy találtad”) alapján be kell építeni a napi fejlesztésbe. Amikor egy fejlesztő a TD-001-gyel jelölt modulban dolgozik, a becslésben kapjon plusz időt az adott refaktorálás elvégzésére.

26.1.1.7 Labor: Adósságnyilvántartás és prioritási terv készítése

Cél: A technikai adósság azonosításának, dokumentálásának és priorizálásának gyakorlása egy meglévő kódbázisban vagy egy reprezentatív kódrészleten.

Kiinduló állapot: Válassz ki egy olyan szolgáltatást vagy modult, amelyet jól ismersz. Ha ilyen nem áll rendelkezésre, használj egy nyilvános, nyílt forráskódú projekt letisztázott, de nem triviális részét, vagy a fejezet korábbi gyakorlataiban épített rendszert. A gyakorlathoz szükséged lesz hozzáférésre a verziókezelő rendszerhez (Git) és opcionálisan egy statikus elemző eszközre.

Elvárt eredmény: Egy Markdown vagy CSV formátumú adósságnyilvántartás, amely legalább 5, legfeljebb 10 tételt tartalmaz. Minden tételhez tartozzon:

1. Egyedi azonosító.

2. A típus besorolása (Szándékos/Nem szándékos, Megfontolt/Meggondolatlan).
3. A tőke leírása (mi a konkrét probléma a kódban?).
4. A kamat leírása (hogyan fogja ez lelassítani a jövőbeni munkát?).
5. Mérési bizonyíték (pl. ciklomatikus komplexitás értéke, a fájl churn értéke, egy hiányzó teszt megnevezése).
6. Prioritási pontszám (a Kamat x Kockázat szorzat alapján).

Ellenőrzési szempontok:

- ☐ Az adósságnyilvántartás reprodukálható? (Megvan-e a vizsgált repository verziója és a mérés időpontja?)
- ☐ A „mért eredmény" és a „várt eredmény" szét van-e választva? (Ne kevered a ténylegesen kimutatott komplexitást a feltételezett jövőbeli költséggel.)
- ☐ Minden tétel el van-e látva döntési kontextussal (miért van itt az adósság)?
- ☐ A prioritási tervben szerepel-e, hogy mit *nem* fogunk törleszteni (tudatos ignorálás)?

Visszaállítási út: Mivel ez a gyakorlat alapvetően egy dokumentációs feladat, a kódbázisban nem szükséges változtatást végrehajtani. Ha mégis elindítasz egy statikus elemző eszközt, ügyelj arra, hogy ne commitold az általa generált konfigurációs fájlokat a versenykezelő rendszerbe, hacsak nem ez a szándékod. Ha bármilyen átmeneti fájl készül, zu munkakönyvtár visszaállítása a `git checkout -- .` paranccsal történhet (figyelembe véve, hogy ez törli a nem mentett változásokat).

26.1.1.8 Összefoglaló és ellenőrzőlista

A technikai adósság nem feltétlenül a rossz mérnöki munka jele, hanem a szoftverfejlesztés inherens része. A kulcs a *láthatóság* és a *gazdálkodás*. Az architekt felelőssége, hogy az adósságot ne hagyd a szőnyeg alá söpörni, hanem nyilvántartásba venni, majd a kamat mértéke alapján ütemezett törlesztést javasoljon. Az adósság tőkéjének ismerete nélkül a rendszer hosszú távú fejlődése kiszámíthatatlanná válik; a kamat ismerete nélkül pedig nem tudjuk megindokolni, hogy mikor kell megállni a funkciófejlesztéssel és a minőségre koncentrálni.

Fejezeti ellenőrzőlista:

- ☐ Ismered a 4 típusát a technikai adósságnak?
- ☐ Különbséget tudsz tenni a tőke (a rossz kód) és a kamat (a lassulás) között?
- ☐ Van-e a rendszerednek kódmetrikán és churn-ön alapuló mérési mechanizmusa?
- ☐ Vezetsz-e adósságnyilvántartást?
- ☐ A prioritási döntéseidben a kockázatot és a napi munkavégzésre gyakorolt hatást is figyelembe veszed, vagy csak a fejlesztők szeszélyére hagyatkozol?

Önellenőrző kérdés: Egy kulcsfontosságú fizetési modulban magas a kódduplikáció, de a csapat az elmúlt 6 hónapban alig nyúlt hozzá, mert a rendszer ezen része stabilan működik. Milyen típusú adósságot azonosítanál, és milyen prioritással kezelnéd a törlesztését? Indokold meg a választ a kamat és a kockázat fogalmainak használatával!

26.2 Refaktorálás nagy rendszerekben

A refaktorálás nagy rendszerekben akkor válik önálló tudássá, amikor a kód már nem fér el egyetlen karbantartó fejében, a tesztek futási ideje percekben vagy órákban mérhető, és egy rosszul időzített szerkezeti változtatás nem egy modul, hanem egy teljes értéklánc leállását

kockáztatja. Ebben a fejezetben a refaktorálást nem egyszerű kódjavításként kezeljük, hanem olyan tervezési tevékenységként, amelynek középpontjában a **külsőleg megfigyelhető viselkedés változatlansága** áll, miközben a belső szerkezet – osztályok, függvények, modulhatárok, interfészek mögötti komponensek – tudatosan alakul át. A cél nem az, hogy „végre rendet tegyünk”, hanem hogy a rendszer jövőbeli módosíthatósága, tesztelhetősége és üzemeltethetősége növekedjen anélkül, hogy közben a felhasználók vagy a downstream rendszerek észrevennék a változást.

A refaktorálás definíciója szerint a művelet **viselkedésmegőrző szerkezetátalakítás**. Ez két dolgot zár ki: a funkcionális változtatást és a hibajavítást. Ha a refaktorálás során egy hibát is kijavítanak, a változtatás már nem tisztán refaktorálás, mert a viselkedés megváltozott. Ez nem jelenti azt, hogy a hibát tilos javítani; azt jelenti, hogy a refaktorálási lépést és a hibajavítást külön commitban, külön döntési bejegyzésben kell elvégezni. Nagy rendszerekben ez a fegyelmezettség kulcsfontosságú: amikor egy incidens miatt vissza kell görgetni egy változást, a karbantartónak pontosan tudnia kell, melyik commit okozta a viselkedésváltozást.

26.2.1 A viselkedés megőrzése mint mérnöki követelmény

A „viselkedés megőrzése” kifejezés nem szubjektív érzés, hanem ellenőrizhető állapot. Azt kell meghatározni, hogy a rendszer **külső határain** – API-válaszok, üzenetek, adatbázisba írt rekordok, naplóbejegyzések, felhasználói felület állapotátmenetei – mi számít megfigyelhető viselkedésnek. Egy belső metódus átszervezése például nem látható kívülről, de ha a metódus a numerikus kerekítés sorrendjét változtatja meg, akkor a számlaösszeg eltérhet, ami már külső viselkedésváltozás. A refaktorálás előtt tehát pontosan rögzíteni kell azokat az **invariánsokat**, amelyeknek a művelet során változatlannak kell maradniuk.

Nagy rendszerekben az ilyen invariánsok ritkán vannak egyetlen helyen dokumentálva. Ezért a refaktorálás előkészítéseként a mérnöknek **karakterizációs teszteket** kell készítenie. A karakterizációs teszt nem azt ellenőrzi, hogy a rendszer a specifikáció szerint hogyan *kellene* működjön, hanem azt, hogy jelenleg hogyan *ténylegesen* működik – a hibákkal, a furcsa üres karakterláncokkal és a nem dokumentált kivételekkel együtt. Ez a különbségtétel döntő. Egy örökölt rendszerben a specifikáció gyakran elavult vagy hiányzik, a downstream fogyasztók viszont már alkalmazkodtak a tényleges viselkedéshez. Ha a refaktorálás során „észszerűsítjük” a kimenetet, akkor a saját tesztjeink zöldek lehetnek, a partnerrendszerek viszont meghibásodnak. A karakterizációs teszt a jelenlegi, megfigyelt kimenetet rögzíti, és ezzel biztosítja, hogy a refaktorálás nem változtatja meg a rendszer tényleges szerződését.

A karakterizációs teszteket ki lehet egészíteni a **golden master** technikával, amikor a régi rendszert sok bemenettel futtatják, a kimeneteket elmentik, majd a refaktorált rendszert ugyanazokkal a bemenetekkel futtatják, és a kimeneteket bitről bitre összehasonlítják. Ez a megközelítés különösen hasznos olyan feldolgozási láncoknál, ahol az üzleti logika bonyolult, és a bemeneti tér nagy. A golden master nem helyettesíti a teszttervezést, de hatékonyan rögzíti a jelenlegi viselkedést, és gyorsan jelzi a nem szándékos eltérést.

A viselkedés megőrzésének ellenőrzése nem csak a tesztek feladata. A refaktorálási folyamat során minden lépésnél meg kell tudni válaszolni azt a kérdést: **mi lehet az a külső hatás, amely ezt a lépést másnak látja?** Ha a lépés csak átnevez egy osztályt, a hatás általában nulla a futtatási viselkedésre. Ha viszont a lépés megváltoztatja egy HTTP-válasz mezőjének sorrendjét, akkor egy szigorú sémaellenőrzést végző kliens eltérően viselkedhet. A refaktorálás tervezésekor ezért minden lépéshez hozzá kell rendelni egy **ellenőrzési módot**: automatikus teszt, statikus elemzés, kódátvizsgálás vagy éles párhuzamos futtatás. A nagy rendszerekben az ellenőrzési mód kiválasztása maga is architektonikus döntés, nem pedig utólagos teher.

26.2.2 Tesztvédőháló: a biztonsági háló, amely nélkül nem lépünk

A refaktorálás első szabálya nagy rendszerekben: **először a tesztvédőháló, utána a változtatás**. Ez a gyakorlatban azt jelenti, hogy a refaktorálásra kijelölt terület köré olyan tesztréteget építünk, amely elég gyorsan fut ahhoz, hogy minden lépés után futtatható legyen, és elég érzékeny ahhoz, hogy azonnal jelezze, ha a megfigyelhető viselkedés megváltozott. A tesztvédőháló nem feltétlenül az egész rendszer teljes tesztkészletét jelenti – ez nagy rendszereknél sokszor órákig tartana –, hanem a refaktorálási lépések **lokális és közepes hatókörű** tesztjeit.

A tesztvédőháló felépítésének tipikus lépései:

1. **Határok azonosítása.** Melyik komponenst vagy modult fogjuk átalakítani? Hol húzódnak a bemeneti és kimeneti interfészek? Melyik downstream rendszer fogyasztja a kimenetet? A határokat C4 vagy egyszerű komponensdiagram formájában rögzíteni kell, hogy a refaktorálás ne csússzon át a szomszédos területekre.

2. **Karakterizációs tesztek készítése.** A meglévő interfészekre olyan teszteket írunk, amelyek a jelenlegi kimenetet rögzítik. A teszt neve utaljon arra, hogy karakterizációs, például `characterizes_current_rounding_behavior`, és a hozzá tartozó komment magyarázza meg, hogy a jelenlegi érték szándékosan nem a specifikáció szerinti, hanem a megfigyelt.

3. **A gyors visszajelzés kijelölése.** Kiválasztjuk azokat a teszteket, amelyeket minden refaktorálási lépés után futtatni fogunk. Ez általában a karakterizációs tesztek és a komponens unit tesztjei. Az integrációs tesztek csak a kisebb mérföldköveknél futnak, mert lassúbbak és külső függőségeket igényelnek.

4. **Kontraktus-tesztek hozzáadása.** Ha a refaktorált komponens más szolgáltatásokkal kommunikál, célszerű a fogyasztói szerződéseket rögzíteni, például Pact vagy OpenAPI szerződés formájában. A refaktorálás során a szerződés nem változhat, legfeljebb a belső implementáció.

5. **A zöld alapvonal rögzítése.** A refaktorálás előtt minden kijelölt tesztnek zöldnek kell lennie. Ha egy alapvonal teszt sem zöld, azt először kezelni kell: vagy a teszt környezet hibás, vagy a rendszer már most is rossz állapotban van. A refaktorálást nem szabad piros tesztek mellett elkezdeni, mert utána nem lehet megkülönböztetni a mi változtatásunk okozta hibát a korábbi hibától.

A tesztvédőháló tehát nem pusztán a tesztek meglétét jelenti, hanem egy **folyamatosan futtatható, a változtatási hatókörre szabott** teszthalmazt, amely a refaktorálás minden lépésénél gyors visszajelzést ad.

26.2.3 Kis lépések: a szerkezetátalakítás fokozatossága

A nagy rendszerekben a refaktorálás legnagyobb kockázata a **nagy lépés**. Minél több helyen nyúlunk egyszerre a kódhoz, annál nehezebb az esetleges hibát a konkrét változtatáshoz kötni, és annál nehezebb visszagörgetni. A refaktorálás ezért **kis, önmagukban ellenőrizhető lépések** sorozata. Minden lépés után a tesztvédőháló zöld, és minden lépésnek van visszaútja.

A kis lépések modern infrastruktúrája az IDE-be épített automatikus refactoring eszközök. Az átnevezés (rename), a metódus kiemelése (extract method), a paraméterobjektum bevezetése (introduce parameter object) vagy a feltétel polimorfizmusra cserélése (replace conditional with polymorphism) olyan műveletek, amelyeket az IDE statikus elemzéssel, a típusrendszer és a hívási gráf ismeretében hajt végre. A mérnök feladata nem csupán a menügomb megnyomása,

hanem annak eldöntése, hogy az adott átalakítás **megőrzi-e a viselkedést**, és hogy a lépés hatóköre a tervezett határon belül marad.

Példaként vegyünk egy örökölt rendelés-kezelő szolgáltatást, amelyben egy `OrderService` osztály egyetlen, több mint ezer soros metódusban kezeli a rendelés validálását, a készletfoglalást, a számlázást és a visszaigazolást. A refaktorálás első lépése nem az, hogy szétvágjuk a metódust, hanem hogy a tesztvédőhálót felépítjük a `placeOrder` publikus metódus köré. Ezután következhet a **kiemelés**: a validációs logikát egy privát metódusba emeljük ki úgy, hogy a vezérlő folyamatosan ugyanazt a kódot hajtja végre. A karakterizációs teszt fut, és a kimenet bitre pontosan ugyanaz. Csak ezután mozgatjuk a kiemelt metódust egy külön komponensbe, ami már nagyobb lépés, de a viselkedésváltozás kockázata továbbra is alacsony, mert a híváslánc ekvivalens. Minden lépést külön commitban rögzítünk, így ha a mozgatás során egy rejtett függőség megsérül, a verziókezelőben a két állapot jól elválik.

A nagy rendszerekben a kis lépések időbeli kiterjedése is fontos. Egy bonyolult modullánc átszervezése akár több hetet is igénybe vehet, amelynek során a rendszer mindvégig működőképes marad. A fokozatos átalakítás ezért válik el a „nagy robbanás” (big bang) megközelítéstől. A folyamat során a refaktorálási lépéseket érdemes egy **döntési naplóban** is követni, különösen akkor, ha a lépésnek architektonikus jelentősége van: például egy új modulhatár bevezetésekor a döntést ADR-ben kell rögzíteni.

A kis lépések módszertana szorosan kapcsolódik a **folyamatos integrációhoz**. Ha a refaktorálási lépések gyakoriak és kicsik, akkor a csapat minden tagja könnyebben követi a változásokat, és a felülvizsgálat is hatékonyabb. Egy nagy, átfogó refaktorálás code review-ja ezzel szemben nehézkes, mert a véleményező nem tudja elválasztani a valódi viselkedésváltozást a puszta szerkezeti mozgatástól. A kis lépések ezt a terhet is csökkentik.

26.2.4 Strangler fig és párhuzamos üzemeltetés a refaktorálás szolgálatában

A **strangler fig** (fojtófüge) minta a refaktorálás és a legacy modernizáció határán áll. Lényege, hogy a régi rendszert nem cseréljük le egyszerre, hanem fokozatosan egy új rendszer veszi át a funkcióit, miközben a régi még működik. A metafora szerint az új rendszer úgy fonódik a régi köré, ahogy a fojtófüge a gazdafát körbefonja, és idővel a régi rendszer természetes módon elsorvad.

A refaktorálás kontextusában a strangler fig akkor válik hasznossá, ha egy nagyobb komponenst kell lecserélni, de annak teljes újraírása túl kockázatos lenne. A **párhuzamos üzemeltetés** azt jelenti, hogy a régi és az új komponens egyidejűleg fut, és a forgalom egy részét vagy egészét az új komponenshez irányítják. A két komponens kimenetét össze lehet hasonlítani, így a viselkedésmegőrzést nem csak tesztek, hanem éles forgalom mellett is ellenőrizni lehet.

A strangler fig alkalmazásának alapvető lépései egy refaktorálási projektben:

1. **A régi komponens mögé absztrakciót helyezünk.** A hívó felek nem közvetlenül a régi osztályt vagy szolgáltatást hívják, hanem egy interfészen vagy API gateway-en keresztül. Ez teszi lehetővé, hogy a hívókat később az új implementációra irányítsuk.
2. **Az új komponens fejlesztése a régi mellett.** Az új komponens ugyanazt a szerződést valósítja meg, de esetleg modernebb technológián, jobb belső szerkezettel. Fejlesztés közben a régi rendszer továbbra is termelésben fut.
3. **A forgalom fokozatos átterelése.** A hívókat szabályozott módon az új komponensre irányítják. Kezdetben csak egy kis, alacsony kockázatú forgalomszeletet: például egy adott ügyfélkört, egy adott régiót vagy a kérések 1%-át. A terhelés növelése előtt a kimeneteket és a teljesítményt összehasonlítják.

4. **Párhuzamos összehasonlítás és riasztás.** A régi és az új komponens kimenetét naplózzák, és eltérés esetén riasztanak. Ez a golden master technika éles forgalomra alkalmazott változata. Az eltérés nem feltétlenül hiba: lehet, hogy az új rendszer szándékosan javít egy kerekítési módszert. Ilyenkor azonban a viselkedésváltozást **külön döntésként** kell kezelni, nem a refaktorálás részeként.

5. **A régi komponens kivezetése.** Amikor az új komponens már hosszabb időn keresztül stabilan hozza a várt eredményeket, a régi komponenst leállítják. A kivezetés előtt meg kell győződni arról, hogy minden hívó átállt, és nincs olyan rejtett függőség, amely közvetlenül a régi komponenst érné el.

A strangler fig nem csak szolgáltatásokra, hanem **monolitikus kódbázison belül** is alkalmazható. Ilyenkor a régi modul mellé új modul kerül, és egy köztes **anti-corruption layer** vagy **strangler façade** tereli a hívásokat. A façade egy vékony réteg, amely a régi interfészt változatlanul kínálja, de a hívásokat fokozatosan az új modulra irányítja. Ez a megközelítés különösen akkor hasznos, ha a régi modult sok belső hívó használja, és azok szinkronizálására nincs lehetőség egyszerre.

A párhuzamos üzemeltetés természetesen **költséggel jár**. A régi és az új komponens egyidejű futtatása kétszeres erőforrást igényelhet, az adatok szinkronizálása pedig további bonyodalmat okozhat, ha a két komponens közös adatbázist használ. A döntés során mérlegelni kell a kockázatcsökkentés és az üzemeltetési költség közötti trade-offot. Nagy rendszerekben azonban a fokozatos átállás gyakran az egyetlen elfogadható mód, mert a teljes kicserélés leállással járna, és a visszaállítás is nehézkes lenne.

26.2.5 Gyakorlati minták és eszközök nagy rendszerekhez

A nagy rendszerek refaktorálása során több, egymást kiegészítő technika használható:

- **Branch by abstraction.** Ha egy komponens viselkedését akarjuk átalakítani, a változtatás mögé absztrakciót hozunk létre, és a régi, illetve az új implementációt e mögött tartjuk. A hívók a absztrakcióval kommunikálnak, így a váltás a hívók módosítása nélkül, konfigurációval vagy feature flaggel történik. Ez a minta csökkenti a nagy átalakítások kockázatát, mert a fejlesztők bármikor vissza tudnak váltani a régi implementációra.

- **Feature flag.** A feature flag nem kizárólag refaktorálási eszköz, de nagyon hatékony a fokozatos bevezetés támogatására. A refaktorált kódot flag mögött lehet tartani, és csak akkor élesíteni, amikor a tesztek és a párhuzamos futtatás már elegendő bizonyítékot adott. A flag használatakor azonban ügyelni kell a flag adósságra: a refaktorálás befejezése után a flaget el kell távolítani, különben a kódban idővel felhalmozódó holt ágak újabb technikai adósságot termelnek.

- **Adapterek és anti-corruption layer.** Amikor a refaktorálás során a belső modell megváltozik, a régi és új modell közötti átjárást adapterek vagy kifejezetten egy anti-corruption layer biztosítja. Ez a réteg lefordítja a régi és az új fogalmakat, így a két világ párhuzamosan élhet. Fontos, hogy az anti-corruption layer ne váljon tartós megoldássá; ha a régi modell kivezetése után is fennmarad, az a modellhatár elmosódását és a komplexitás növekedését okozza.

- **Strangler façade.** A façadé olyan felület, amely a régi interfészt kínálja, de a hívásokat a háttérben a régi vagy az új implementációhoz irányítja. A façade átmeneti komponens, amelynek célja a migráció idején a hívók zavartalan kiszolgálása. A refaktorálás befejezése után a façade egyszerűsödik, és a régi út megszűnik.

Ezek a minták közös jellemzője, hogy **átmeneti komplexitást** vezetnek be annak érdekében, hogy a végállapot egyszerűbb legyen. Ez a trade-off akkor éri meg, ha a régi és az új rendszer közötti átmenet hosszabb ideig tart, és a teljes kicserélés kockázata magas. A rövid, alacsony kockázatú átalakításoknál viszont a zászlók és adapterek maguk is terhet jelentenek; ilyenkor a kis lépésekben végrehajtott közvetlen refaktorálás egyszerűbb.

26.2.6 Tipikus hibák és a helyes gyakorlat

A nagy rendszerek refaktorálása során a leggyakrabban elkövetett hibák nem technikai részletekben rejlenek, hanem a folyamat és a fegyelem hiányában.

- **A refaktorálás és a funkcióváltoztatás keverése.** A leggyakoribb hiba, amikor a fejlesztő a szerkezetátalakítás közben észreveszi, hogy egy metódus rosszul működik, és rögtön ki is javítja. A kettő együtt olyan változtatást eredményez, amelyet nehéz visszafejteni, és amelynek tesztelési következményei kiszámíthatatlanok. A helyes gyakorlat: a refaktorálási lépések során rögzítjük a megfigyelt hibát, de csak a tiszta szerkezeti átalakítás befejezése után, külön commitban javítjuk.
- **Hiányzó tesztvédőháló.** Sok csapat azért fél a refaktorálástól, mert nincs elég teszt, és a változtatás után nem tudják, mit rontottak el. A megoldás nem a refaktorálás elkerülése, hanem a karakterizációs tesztek pótlása. Ahol a tesztlefedettség alacsony, ott a golden master vagy az approval teszt gyorsan adhat alapvonalat. A tesztvédőháló nélküli refaktorálás lényegében szerencsejáték.
- **Túl nagy lépés.** A „most akkor szétszedjük az egész modult” hozzáállás a nagy rendszerekben garantáltan hibát okoz. A lépéseket úgy kell megválasztani, hogy mindegyik önmagában is érthető, tesztelhető és visszavonható legyen. Ha egy lépés túl nagy, akkor azt tovább kell bontani.
- **A párhuzamos üzemeltetés elhanyagolt összehasonlítása.** Ha a régi és az új komponens párhuzamosan fut, de a kimeneteiket nem hasonlítják össze, a párhuzamos üzemeltetés értelmetlenné válik. Az összehasonlításnak automatikusnak kell lennie, és eltérés esetén riasztania kell. A riasztás nélkül a fejlesztők csak akkor értesülnek a hibáról, amikor a downstream rendszer már leállt.
- **A visszaút hiánya.** Minden refaktorálási sorozatnak rendelkeznie kell visszaállítási tervvel. A kód szintjén ez a verziókezelésből adódik, de a párhuzamos üzemeltetésnél a visszaút azt jelenti, hogy a forgalomirányítót vissza tudjuk állítani a régi komponensre. Ha a visszaút nem létezik, akkor a fokozatos bevezetés nem csökkenti a kockázatot, csak késlelteti a hibák felszínre kerülését.
- **Az adatbázis-séma változását refaktorálásnak tekinteni.** A relációs séma megváltoztatása nem viselkedésmegőrző szerkezetátalakítás a kód szintjén. Az adatmodell módosításához migrációs stratégiára van szükség, például expand-contract mintára: először az új oszlopot hozzáadjuk, a régi és az új mezőt párhuzamosan írjuk, a fogyasztókat átállítjuk, végül a régi oszlopot töröljük. Ez a terület Legacy modernizáció és strangler fig fejezet tárgyalja részletesen.
- **A teljesítményváltozás figyelmen kívül hagyása.** A tiszta refaktorálás definíció szerint nem változtatja a viselkedést, de a gyakorlatban a szerkezet megváltoztatása befolyásolhatja a teljesítményt: új absztrakciós réteg, közvetett hívás vagy az eltérő memóriaelrendezés lassíthatja a futást. Ezért a refaktorálási sorozat alatt és után a **teljesítményt is mérni kell**, nem csak a funkcionális kimeneteket. A mért eredményt rögzíteni kell a döntési naplóban.

26.2.7 A refaktorálási lépéssor tervezése: a gyakorlati terv

A nagy rendszerekben a refaktorálás nem improvizáció, hanem tervezett lépéssor. A tervezés során a következő elemeket kell meghatározni:

1. **Cél és hatókör.** Melyik komponenst vagy réteget érinti a refaktorálás? Mi a jelenlegi struktúra, és mi a kívánt célstruktúra? A célstruktúrát nem szükséges aprólékosan előre megtervezni, de a fő modulhatárokat, felelősségeket és interfészeket fel kell vázolni.
2. **Viselkedési invariánsok.** Mit jelent a „viselkedés megőrzése" ebben a konkrét esetben? Mely API-k, üzenetek, adatbázis-rekordok és naplóbejegyzések számítanak megfigyelhetőnek? Melyek azok a nem funkcionális jellemzők, amelyeket mérni fogunk: válaszidő, átviteli sebesség, CPU-felhasználás?
3. **Tesztvédőháló terve.** Mely teszteket futtatjuk minden lépés után? Mely karakterizációs teszteket kell megírni? Mely kontraktus-teszteket kell rögzíteni? Milyen környezetben futnak ezek a tesztek, és mennyi idő alatt?
4. **Lépéssor.** A refaktorálási lépések listája, mindegyikhez hozzárendelve az ellenőrzési mód. A lépéseknek kicsinek, egymásra épülőnek és visszavonhatónak kell lenniük. A lépéssort érdemes úgy felépíteni, hogy a magas kockázatú lépések a lehető legkésőbbre kerüljenek, amikor már a tesztvédőháló erős.
5. **Párhuzamos üzemeltetés terve, ha szükséges.** Ha strangler fig mintát alkalmazunk, meg kell határozni a forgalomterelés módját, az összehasonlítási mechanizmust, a riasztási szabályokat és a visszaállítási útvonalat.
6. **Döntési napló.** A refaktorálás során hozott architektonikus döntéseket ADR-ben rögzítjük. Az ADR-ben szerepel a kontextus, a döntés, a megvizsgált alternatívák, a következmények és az állapot. A nagy rendszerekben a refaktorálási döntések gyakran a csapaton kívüli érintetteket is érintik, ezért az ADR a kommunikáció eszköze is.

26.2.8 Labor: refaktorálási lépéssor tervezése

A labor célja, hogy a tanultakat egy **reprodukálható dokumentumban vagy jegyzetfüzetben** alkalmazza. A feladat nem a kód tényleges átírása, hanem a refaktorálás megtervezése és a tervezési döntések dokumentálása. A labor eredményét úgy kell elkészíteni, hogy egy másik mérnök a dokumentum alapján meg tudja ismételni a gondolatmenetet, és ellenőrizni tudja a lépéseket.

Választható kiinduló rendszer: a laborhoz használható a saját projektből egy kiválasztott, technikai adóssággal terhelt komponens, vagy a következő szemléltető példa: egy örökölt rendeléskezelő szolgáltatás, amelyben az `OrderService` nevű osztály egyetlen, több mint ezer soros metódusban kezeli a rendelés validálását, a készletfoglalást, a számlázást és a visszaigazolást. A metódus közvetlenül ír a relációs adatbázisba, és szinkron HTTP hívást küld a számlázó modulnak. Nincsenek unit tesztek; az integrációs tesztkörnyezet törékeny, és a futása több mint 30 percet vesz igénybe.

A labor lépései:

1. **Kontextus rögzítése.** Írja le a kiválasztott komponens határait, a hívó feleket, a downstream rendszereket és az adatkapcsolatokat. Készítsen egyszerű C4 context vagy container diagramot diagrams-as-code formában (Mermaid, PlantUML vagy Structurizr DSL). A diagramon jelölje a rendszerhatárt, a külső szereplőket és a kommunikációs protokollt.

2. **Viselkedési invariánsok azonosítása.** Sorolja fel azokat a megfigyelhető viselkedéseket, amelyeknek a refaktorálás során változatlannak kell maradniuk. Például: a `POST /orders` végpont válaszkódja és választeste, a készletfoglalás adatbázisba írt rekordjai, a számlázó modulnak küldött üzenet mezői, a naplóba írt tranzakciós azonosító. Minden invariánshoz adjon meg egy mérési vagy ellenőrzési módot.

3. **Tesztvédőháló tervezése.** Határozza meg, milyen karakterizációs teszteket kell írni a jelenlegi viselkedés rögzítéséhez. Írja le a golden master megközelítést: mely bemenetekkel futtatná a régi rendszert, és hogyan tárolná az elvárt kimeneteket. Jelölje meg, mely teszteket futtatná minden refaktorálási lépés után, és melyeket csak a mérföldköveknél. A tesztkörnyezetre vonatkozóan adjon meg egy konfigurációt, de ne használjon valós hitelesítő adatot; helyőrzőket használjon.

4. **A lépéssor megtervezése.** Bontsa a refaktorálást legalább öt kis lépésre. Minden lépéshez írja le, hogy mi a szerkezeti változtatás, milyen tesztekkel ellenőrzi a viselkedésmegőrzést, és mi a visszaút, ha a lépés hibát okoz. Az első lépés valószínűleg a karakterizációs tesztek elkészítése, a következő a metódus kiemelése, majd a validációs logika önálló komponensbe mozgatása, és így tovább.

5. **Strangler fig mérlegelése.** Döntse el, hogy a refaktorálás során használna-e strangler fig mintát, vagy a közvetlen kis lépések elegendők. Ha a párhuzamos üzemeltetés mellett dönt, tervezze meg a forgalomterelés és az összehasonlítás módját. Ha nem, indokolja meg, miért nem szükséges. A döntést ADR formátumban rögzítse: kontextus, döntés, alternatívák, következmények, állapot.

6. **Mérési terv.** Határozza meg, milyen mérőszámokat fog mérni a refaktorálás előtt és után: például a karakterizációs tesztek futási ideje, a `placeOrder` metódus átlagos válaszideje, a CPU-felhasználás terhelés alatt, a tesztlefedettség. Adja meg a mérési környezetet, a terhelést és a mérési ablakot. A mérési tervben **különválasztva** szerepeljen a **várt eredmény** (hipotézis) és a **mért eredmény** (a refaktorálás utáni tényleges adat). A labor során a mért eredményt még nem ismeri, ezért oda írjon „mérésre vár” értéket, de a mérési terv szerkezete legyen reprodukálható.

7. **Ellenőrzési szempontok.** Készítsen egy ellenőrzőlistát, amellyel a refaktorálás befejezése után igazolni lehet, hogy a viselkedés nem változott. A lista tartalmazzon olyan szempontokat, mint: minden lépésnél zöld a tesztvédőháló; a karakterizációs tesztek kimenete azonos; a kontraktus-tesztek nem törtek; a teljesítménymérés eltérése a hibahatáron belül van; a döntési napló naprakész.

Az átadandó dokumentum legyen egy Markdown vagy Jupyter notebook, amely tartalmazza a fenti elemeket. A dokumentumban a mért és a várt eredményt egyértelműen el kell választani, például külön szakaszokban: „Várt eredmény (hipotézis)” és „Mért eredmény (tényleges)”. A dokumentum ne tartalmazzon valós azonosítókat, ügyfélneveket vagy belső URL-eket; helyőrzőket használjon.

26.2.9 Ellenőrzőlista

A fejezetben tárgyalt refaktorálási gyakorlat ellenőrzésére használható lista:

- ☐ A refaktorálás célja és hatóköre dokumentált, a rendszerhatárokat diagram mutatja.
- ☐ A viselkedési invariánsokat listáztuk, és mindegyikhez ellenőrzési módot rendeltünk.
- ☐ A karakterizációs tesztek rögzítik a jelenlegi, megfigyelt viselkedést, nem a specifikáció szerinti ideális állapotot.

- ☐ A tesztvédőháló minden refaktorálási lépés után futtatható, és a futási idő nem akadályozza a gyors visszajelzést.
- ☐ A refaktorálás kis, önmagukban ellenőrizhető lépésekre van bontva, mindegyiknek van visszaútja.
- ☐ A refaktorálás során nem kevertünk funkcióváltoztatást vagy hibajavítást; az ilyen változtatásokat külön commitban kezeljük.
- ☐ Ha strangler fig vagy párhuzamos üzemeltetés van használatban, a régi és az új komponens kimenetét automatikusan összehasonlítják, és eltérés esetén riasztás történik.
- ☐ A döntési napló (ADR) tartalmazza a refaktorálási döntéseket, az alternatívákat és a következményeket.
- ☐ A teljesítményt, a válaszidőt és az erőforrás-felhasználást mértük, és az eredményt a döntési naplóban rögzítettük.
- ☐ A refaktorálás befejezése után a bevezetett átmeneti absztrakciókat, feature flageket és adaptereket eltávolítottuk, vagy tudatos döntés alapján megtartottuk.

26.2.10 Önellenőrző kérdés

Melyik esetben választaná a strangler fig mintát a közvetlen, kis lépésekben végrehajtott refaktorálás helyett? Válaszában térjen ki a viselkedésmegőrzés ellenőrzésére, a párhuzamos üzemeltetés költségére és a visszaállítás lehetőségére.

A kérdés megválaszolásához gondolja végig, hogy a komponens lecserélése milyen kockázattal jár: ha a régi komponens kimeneti szerződése nem teljesen ismert, ha a downstream rendszerek nem tudnak egyszerre átállni, vagy ha a teljes kicserélés leállást igényelne, akkor a strangler fig fokozatos átállása indokoltabb. Ha viszont a hatókör kicsi, a tesztvédőháló erős, és a csapat képes a lépéseket gyorsan, atomi módon végrehajtani, akkor a közvetlen refaktorálás egyszerűbb és alacsonyabb üzemeltetési költséggel jár. A döntés mindig a konkrét rendszer, a csapat és a szervezeti feltételek ismeretében hozható meg; nincs univerzálisan jobb megoldás.

26.3 Legacy modernizáció és strangler fig

A legacy rendszer lecserélése ritkán egyetlen nagy esemény, sokkal inkább egy folyamatos, apró lépésekből álló evolúció. A gyakorlatban a „big bang” átállás – amikor a régi rendszert egy adott napon leállítják, és az új rendszer veszi át a helyét – szinte mindig kudarcba fullad: az új rendszer nem elég érett, az adatmigráció késik, a felhasználók egyszerre találkoznak a hiányzó funkciókkal. A strangler fig minta ezzel szemben a régi rendszer fokozatos, kockázatcsökkentő lecserélését teszi lehetővé. Ebben a fejezetben ezt a mintát, a hozzá kapcsolódó architekturális döntéseket és a gyakorlati megvalósítás lépéseit vizsgáljuk.

A strangler fig (fojtófüge) elnevezés a természetből származik: a fügefa magja egy gazdanövényen csírázik ki, majd fokozatosan gyökereket ereszt lefelé, körülöleli a gazdát, végül saját törzset nevel, míg a régi növény elhal. Az architektúrában ez azt jelenti, hogy az új rendszer komponenseit fokozatosan építjük a régi mellé, a forgalmat apránként tereljük át, és amikor egy régi komponens már nem kap forgalmat, akkor kivezetjük. A cél nem a régi rendszer egyszeri megsemmisítése, hanem a folyamatos működés közbeni átmenet.

26.3.1 A fokozatos lecserélés alapelvei

A modernizáció során a legfontosabb elv a **folyamatos üzletmenet**: a rendszer minden pillanatban kiszolgálja a felhasználókat, még akkor is, ha a háttérben éppen átépítés zajlik. Ezt három további alapelv támogatja:

1. **Apró, visszafordítható lépések.** Minden egyes átállási lépésnek rendelkeznie kell egy olyan mechanizmussal, amellyel az előző állapotba vissza lehet térni anélkül, hogy a teljes rendszert le kellene állítani. Ez lehet feature flag, konfigurációs paraméter vagy útválasztási szabály.
2. **A régi és az új rendszer párhuzamos üzeme.** Amíg az új komponens nem bizonyítja a termelési környezetben is, hogy képes ellátni a rá bízott feladatokat, addig a régi komponens továbbra is aktív marad. A párhuzamos üzem nem jelent duplikált fejlesztést, hanem azt, hogy a forgalom egy részét az új, a többit a régi rendszer szolgálja ki.
3. **Folyamatos megfigyelés és mérés.** Az átirányítási döntéseket mérési adatokra kell alapozni: hibák aránya, válaszidő, üzleti metrikák. Csak akkor növeljük az új rendszerre irányított forgalom arányát, ha a megfigyelt értékek a meghatározott küszöbértékeken belül maradnak.

A fokozatos lecserélés ütemezését gyakran egy **modernizációs ütemterv** dokumentálja, amely minden egyes lépéshez rögzíti a bevezetés feltételét, a visszavonás módját és a sikeresség mérőszámait. Erről a labor szakaszban részletesen lesz szó.

26.3.2 A strangler fig minta architekturális nézete

A strangler fig bevezetésének első lépése egy **útválasztási réteg** beiktatása a felhasználói kérések és a háttérszolgáltatások közé. Ez a réteg lehet API gateway, reverse proxy vagy akár egy meglévő load balancer konfigurációja. Feladata, hogy a beérkező kérés URL-je, fejléce vagy más attribútuma alapján eldöntse, melyik háttérrendszerhez irányítsa azt: a régi monolithoz vagy az új, már kész szolgáltatáshoz.

Az alábbi Mermaid ábra a strangler fig útválasztási logikáját mutatja konténer-szintű nézetben. A rendszerhatárt a szaggatott keret jelöli, a külső szereplő a felhasználó, a kapcsolatokon pedig a protokoll és a szinkronitás szerepel. A hibatűrési mechanizmust (timeout, retry) a gateway és a szolgáltatások közötti éleken jelöljük.

```
graph LR
    subgraph Modernizált rendszer
        direction LR
        G[API Gateway<br/> routing + retry/timeout]
        L[Legacy monolit<br/> rendelés + számlázás]
        N[Új rendelés szolgáltatás<br/> /v2/orders]
        DB[(Legacy DB)]
        DB2[(Új rendelés DB)]
        FF[Feature flag szolgáltatás]
    end
    U[Felhasználó] -->|HTTPS, szinkron| G
    G -->|/v1/*, HTTPS, szinkron, timeout 5s| L
    G -->|/v2/orders, HTTPS, szinkron, timeout 3s, retry 2| N
    L -->|JDBC, szinkron| DB
    N -->|REST, szinkron| DB2
    G -.->|konfiguráció, aszinkron| FF
    L -.->|esemény, aszinkron| N
```

A legfontosabb tervezési döntés: **az új funkcionalitást ne a régi rendszerbe építsük**, hanem külön szolgáltatásként valósítsuk meg, és a routing réteg irányítsa át a megfelelő kéréseket. A régi rendszer továbbra is elérhető marad a többi útvonalon. A fenti példában a `/v2/orders` végpont már az új rendelés szolgáltatáshoz tartozik, míg minden más kérés (például számlázás) a legacy monolithoz.

A routing réteg bevezetésekor ügyelni kell a **hibatűrésre**: ha az új szolgáltatás nem válaszol időben, a gatewaynek kezelnie kell a hibát, és opcionálisan visszairányíthatja a kérést a régi rendszerhez, ha a művelet idempotens. Ez a fajta „fallback" azonban bonyolult lehet, ezért gyakran egyszerűbb a kérést hibával visszautasítani, és a hibát jelezni.

26.3.3 A fokozatos lecserélés fázisai

A strangler fig bevezetését szakaszokra bontjuk. Minden szakasz egy-egy üzleti képesség kiemelését jelenti a régi rendszerből.

26.3.4 1. Célképesség kiválasztása

Kezdetben olyan képességet érdemes választani, amely: - viszonylag önálló, kevés függőséggel rendelkezik a régi rendszeren belül, - kritikus a felhasználók számára, ezért a siker mérhető, - elegendő forgalmat generál ahhoz, hogy a mérések statisztikailag értékelhetők legyenek.

Az első kiválasztott képesség gyakran a **rendelés létrehozása** vagy egy **termékkatalógus lekérdezése**. A célképességhez meghatározzuk az új szolgáltatás interfészét, amely lehetőleg azonos vagy kompatibilis a régi interfésszel, hogy a kliensek fokozatosan átállhassanak.

26.3.5 2. Az új szolgáltatás megvalósítása

Az új szolgáltatást a lehető legkisebb funkcionalitással indítjuk, amely már képes ellátni a kiválasztott képességet. Ez gyakran egy **falbejáró** (walking skeleton) jellegű megvalósítás: a teljes folyamat működik, de csak a legszükségesebb üzleti logikával. Ez lehetővé teszi a technikai kockázatok (például adatbázis-hozzáférés, üzenetküldés) korai validálását.

26.3.6 3. Párhuzamos üzem és fokozatos átirányítás

Az új szolgáltatást először **árnyékforgalmon** (shadow traffic) teszteljük: a beérkező kérések egy másolatát az új szolgáltatás is megkapja, de a válasz még a régi rendszerből származik. Ezután a forgalom egy kis százalékát – például 1% – az új szolgáltatáshoz irányítjuk, és figyeljük a hibák arányát, a válaszidőt és az üzleti folyamat teljesülését. Ha a mérések megfelelnek a meghatározott küszöböknek, a százalékot fokozatosan növeljük: 5%, 10%, 50%, végül 100%.

Ezt a folyamatot **feature flag** vagy **dinamikus routing szabály** vezérli. A feature flag lehetővé teszi, hogy az átirányítási arányt anélkül változtassuk, hogy új kódot kellene telepíteni. Ugyanakkor a flag-konfiguráció maga is egy rendszerkomponens, amelynek rendelkezésre állását és verziózását ugyanúgy kezelni kell.

26.3.7 4. A régi komponens kivezetése

Amikor az új szolgáltatás már 100%-ban kiszolgálja a kiválasztott képességet, és egy megfigyelési ablakban (például két hét) nem jelentkezik hiba, a régi komponenst le lehet állítani. Ez gyakran a régi kód eltávolítását jelenti a monolitból, ami további refaktorálást igényel. A kivezetés után a routing szabály is törölhető, vagy egyszerűsíthető.

26.3.8 Frontend és backend leválasztása

A modernizáció során a frontend és a backend szétválasztása külön figyelmet érdemel. A régi monolitok gyakran szerveroldali renderelést használnak, ahol a HTML-t a backend állítja elő. A strangler fig ezen a területen is alkalmazható: a régi és az új felhasználói felület párhuzamosan fut, és a routing réteg dönti el, hogy egy adott oldal vagy API-hívás melyik háttérhez menjen.

A leggyakoribb megoldás az **API gateway** bevezetése, amely a frontend kéréseit a megfelelő backend szolgáltatáshoz irányítja. A frontendet fokozatosan át lehet állítani az új API használatára, oldalanként vagy funkciónként. A feature flagek itt is kulcsfontosságúak: például a felhasználók egy csoportja az új rendelési felületet látja, míg a többiek továbbra is a régit.

Fontos tervezési szempont, hogy az új frontend és a régi backend közötti szerződés **visszafelé kompatibilis** legyen. Ha az új frontend olyan mezőket vár el, amelyeket a régi backend nem küld, akkor az API gatewaynek vagy a régi backendnek adaptert kell biztosítania. Ennek hiányában a frontend és backend leválasztása újabb függőséget teremt, és a fokozatos átállás megakad.

A frontend leválasztás tipikus lépései:

1. **API réteg bevezetése**: a frontend kódjában minden hálózati hívást egy vékony adapter mögé rejtünk, amely a végpontok URL-jét és a kérés formátumát kezeli.
2. **Kettős útvonal**: az adapter képes a régi és az új backend végpontok megszólítására is, feature flag alapján.
3. **Oldalankénti átállás**: először egy alacsony kockázatú oldalt (például a beállítások oldalt) állítunk át az új backendre, majd fokozatosan haladunk a bonyolultabb oldalak felé.
4. **Canary release a frontenden**: a felhasználók egy kis hányada az új frontendet kapja, a többiek a régit; a használat során mérjük a hibákat és a felhasználói elégedettséget.

26.3.9 Adatmigráció és üzleti folytonosság

A strangler fig egyik legnehezebb része az adatok migrációja. A régi és az új szolgáltatás gyakran különböző adatmodelleket használ, vagy legalábbis külön adatbázisban tárolja az adatokat. Az adatmigráció során az **üzleti folytonosság** azt jelenti, hogy a felhasználók mindvégig hozzáférnek a helyes adatokhoz, és egyetlen tranzakció sem vész el.

A leggyakoribb stratégiák:

- **Duális írás**: az új szolgáltatás bevezetésekor a régi rendszer továbbra is írja a saját adatbázisát, és egyidejűleg egy aszinkron mechanizmussal (például üzenetsor vagy CDC – Change Data Capture) értesíti az új szolgáltatást a változásokról. Az új szolgáltatás a saját adatbázisában is rögzíti az adatokat. Ez azonnali konzisztenciát biztosít, de duplikált írást és bonyolult hibakezelést igényel.
- **Olvasási modell replikáció**: ha az új szolgáltatás csak olvassa az adatokat, akkor elegendő egy replikált olvasási modell. A régi adatbázisból egy CDC folyamat másolja az adatokat az új szolgáltatás adatbázisába, és a szolgáltatás onnan olvas.

- **Batch szinkronizáció**: a nagyobb adathalmazok migrációját kezdetben batch folyamattal végezzük, majd a növekményes változásokat CDC-vel vagy rendszeres inkrementális másolással tartjuk naprakészen. A batch futtatás alatt a rendszer továbbra is működik, ezért az első betöltés után ellenőrizni kell a konzisztenciát.

A kritikus kérdés a **tranzakciós határ**: ha egy üzleti folyamat több szolgáltatást érint, akkor a hagyományos ACID tranzakciók nem alkalmazhatók az egész folyamatra. Ilyenkor **saga** vagy **kompenzáló tranzakció** mintát kell alkalmazni, amit az eseményvezérelt architektúrák fejezete tárgyal. A modernizáció során ezt a határt tudatosan kell megtervezni: a rendelésfelvétel például lehet atomi az új szolgáltatáson belül, de a számlázással való integráció már aszinkron, és visszagörgethető lépésként kell kezelni.

Az üzleti folytonosságot **quality attribute scenario** formában is rögzíthetjük:

> **Inger:** Egy új rendelési kérés érkezik a `/v2/orders` végpontra, miközben a migráció az 50%-os átirányítási arányban van. **Környezet:** A rendszer normál üzemmódban fut, az új szolgáltatás és a régi monolit párhuzamosan szolgálja ki a kéréseket. **Válasz:** A kérés sikeresen feldolgozásra kerül, és a válasz legfeljebb 400 ms alatt (95. percentilis) visszaérkezik. A rendelés adatai mind az új, mind a régi adatbázisban megjelennek 2 másodpercen belül. **Mérőszám:** 95. percentilis válaszidő 400 ms; hibák aránya 0,5% 5 perces ablakban; adatkonzisztencia késleltetése 2 s.

Ez a forgatókönyv mérhetővé teszi az átmenet sikerességét, és segít eldönteni, mikor növelhető az átirányítási arány.

A kapacitástervezésnél a Little-törvényt is használhatjuk annak ellenőrzésére, hogy az új szolgáltatás képes-e a rá irányított forgalmat kezelni. Ha például a becsült érkezési ráta $\lambda = 20$ kérés/másodperc, és a megengedett átlagos tartózkodási idő $W = 0{,}3$ s, akkor a rendszerben lévő kérések átlagos száma $L = \lambda W = 6$. Ha az új szolgáltatás konkurens kéréskezelő kapacitása ennél nagyobb, a terv valószínűleg megvalósítható; ha nem, akkor a szolgáltatást skálázni kell a migráció előtt.

26.3.10 Gyakori hibák és a helyes megoldások

A strangler fig bevezetése során számos tipikus hiba fordul elő. Ezek felismerése és elkerülése a tanulási célok egyik központi eleme.

26.3.11 1. Big bang lecserélés

Hiba: A csapat azt tervezi, hogy egy adott hétvégén leállítja a régi rendszert, és az új rendszert indítja el. Ennek során az adatmigráció hosszú órákat vesz igénybe, és nincs lehetőség fokozatos tesztelésre.

Helyes megoldás: A strangler fig fokozatos átirányítással dolgozik. Az új rendszer először árnyékforgalmon, majd kis százalékban éles forgalmon bizonyít. Így a hibák már korán, kis hatással jelentkeznek.

26.3.12 2. A routing réteg hiánya

Hiba: A fejlesztők közvetlenül az új szolgáltatás URL-jét adják a klienseknek, és a régi és az új hívásokat a kliens oldalon keverik. Ez a megoldás gyorsnak tűnik, de a forgalom visszafordítását és a feature flag kezelését a kliensben kell megvalósítani, ami szétszórt logikát eredményez.

Helyes megoldás: Egy központi API gateway vagy proxy réteg bevezetése, amely a routingot és a hibakezelést egységesen végzi. A kliensek változatlanok maradnak, az átirányítási döntések konfigurációval módosíthatók.

26.3.13 3. Adat-inkonzisztencia figyelmen kívül hagyása

Hiba: Az új szolgáltatás által írt adatok nem kerülnek vissza a régi rendszer adatbázisába, vagy fordítva. A párhuzamos üzem során a felhasználók attól függően, hogy melyik rendszerhez irányítják őket, eltérő adatokat látnak.

Helyes megoldás: Kell egy egyértelmű **adattulajdonos** meghatározása: melyik rendszer írja az adott adatot. A másik rendszer csak olvashat, vagy aszinkron módon szinkronizálódik. Ha mindkettő ír, akkor ütközéskezelésre és idempotenciára van szükség.

26.3.14 4. A feature flag hiánya

Hiba: Az átirányítási arányt a kódban rögzített konstansként adják meg, ezért minden módosításhoz új telepítés szükséges. Ez lelassítja a fokozatos átállást, és vészhelyzetben nem teszi lehetővé az azonnali visszalépést.

Helyes megoldás: A routing döntést külső konfiguráció vagy feature flag szolgáltatás vezérli. A flag-nek legyen verziója és naplója, hogy minden átállítás visszakövethető legyen.

26.3.15 5. Korai kivezetés

Hiba: Amikor az új szolgáltatás 100%-ban kiszolgálja a forgalmat, a csapat azonnal eltávolítja a régi kódot és a routing szabályt. Egy hét múlva kiderül, hogy egy ritkán használt funkció nem működik, és már nincs visszaút.

Helyes megoldás: A kivezetés előtt a régi rendszert **megfigyelési ablakban** (például 7–14 nap) hagyja futni, de már nem kap éles forgalmat. Ez idő alatt a naplókban és a hibajelentésekben figyelhetők a rejtett függőségek. A kivezetés csak akkor történik meg, ha a megfigyelési ablak sikeres.

26.3.16 Labor: modernizációs ütemterv készítése

A labor célja, hogy a strangler fig minta alkalmazását egy konkrét, ellenőrizhető ütemtervben rögzítsük. A feladat nem a kód írása, hanem a tervezési és döntési folyamat dokumentálása. A labor eredményét **Markdown dokumentumban** vagy **Jupyter notebookban** kell elkészíteni, és verziózni kell (például Git-ben).

26.3.17 Kiinduló állapot

Válassz egy képzeletbeli vagy valós legacy rendszert. A példa kedvéért legyen ez egy **belső ügyfélszolgálati rendszer**, amely a következő modulokból áll egyetlen Java monolitban: - ügyfélnyilvántartás, - jegykezelés (incidensek), - tudásbázis, - jelentések.

A rendszer egy relációs adatbázist használ, a frontend pedig szerveroldali renderelésű JSP oldalakból áll. A csapat a jegykezelő modult szeretné elsőként kiemelni egy új mikroservice-be, mert az a legnagyobb forgalmú és a leggyakrabban változó.

26.3.18 Elvárt eredmény

Készíts egy modernizációs ütemtervet, amely legalább a következőket tartalmazza:

1. **Rendszerkontextus és határ**: rövid leírás, a strangler fig routing réteg bevezetésének helye a rendszerben.
2. **Célképesség kiválasztása**: indoklás, hogy miért a jegykezelő modul az első jelölt.
3. **Fázisok**: fázisonként megadva:
    - a bevezetés feltétele,
    - a fázis során végrehajtott lépések,
    - a visszavonás módja,
    - a sikeresség mérőszámai (legalább egy quality attribute scenario).
4. **Frontend és backend leválasztás terve**: hogyan áll át a frontend az új API-ra, feature flag használata.
5. **Adatmigrációs terv**: a jegyadatok migrációjának stratégiája (pl. CDC, duális írás, batch + inkrementális), a konzisztencia biztosítása.
6. **ADR vázlat**: legalább egy ADR a strangler fig bevezetéséről a jegykezelő modulra (kontextus, döntés, alternatívák, következmények, állapot).
7. **Ellenőrzési szempontok**: mit mérünk (hibaarány, válaszidő, adatkonzisztencia késleltetése), és milyen küszöbértékek esetén növeljük az átirányítási arányt.

26.3.19 A mért és a várt eredmény elkülönítése

A labor során akkor is készíthető az ütemterv, ha a rendszer nem létezik, vagy nincs lehetőség mérésre. A dokumentumban külön táblázatban kell szerepeltetni: - **Várt eredmények** (a tervezéskor becsült értékek), - **Mért eredmények** (ha vannak, a tényleges mérési adatok, a mérés ideje és környezete).

Ha nincs mért eredmény, azt egyértelműen jelölni kell: „Nem mért, csak becslés.” Ez biztosítja, hogy a dokumentáció ne keltsen hamis bizonyosságot.

26.3.20 Ellenőrzési szempontok és visszaállítás

A labor akkor tekinthető sikeresnek, ha: - az ütemterv minden fázisához tartozik visszavonási mód, - a mérőszámokhoz mértékegység és mérési ablak van rendelve, - az ADR tartalmazza az alternatívákat és a következményeket, - a dokumentum verziózott, és a mért/várt eredmények külön vannak választva.

A visszaállítási út itt a tervezési dokumentum korábbi verziójának elővételét jelenti, nem a kód visszaállítását. A labor nem hajt végre éles műveletet.

26.3.21 Ellenőrzőlista

- ☐ A modernizációs terv a strangler fig mintát használja, nem big bang lecserélést.
- ☐ Van egy útválasztási réteg (API gateway/proxy), amely a régi és az új rendszer közé kerül.
- ☐ A forgalom átirányítása feature flag vagy dinamikus konfiguráció alapján történik.
- ☐ Minden fázishoz tartozik visszavonási terv és sikerességi mérőszám.
- ☐ A frontend és backend leválasztása fokozatos, kompatibilis interfészekkel.
- ☐ Az adatmigráció kezeli a konzisztenciát és az üzleti folytonosságot.
- ☐ A régi rendszer kivezetése előtt megfigyelési ablak van.
- ☐ A mérési eredményeket elkülönítve rögzítik a várt értékektől, és a mérési környezet dokumentálva van.
- ☐ Az ADR-ek rögzítik a döntés okát, az alternatívákat és a következményeket.

26.3.22 Önellenőrző kérdés

Miért veszélyes a big bang lecserélés, és melyik strangler fig mechanizmus csökkenti konkrétan ezt a kockázatot? Válaszában térjen ki a fokozatos átirányítás, a feature flag és a párhuzamos adatbázis szerepére.

26.3.23 Összefoglalás

A legacy modernizáció sikere nem a technológia cseréjén, hanem a folyamat irányításán múlik. A strangler fig minta lehetővé teszi, hogy a régi rendszert apró, visszafordítható lépésekben cseréljük le anélkül, hogy az üzletmenet megszakadna. A frontend és backend leválasztása, valamint az adatmigráció tervezése a minta két kritikus területe. A fejezetben bemutatott fázisok, hibák és a labor során elkészített ütemterv segítenek abban, hogy a modernizáció ne egy kockázatos projekt legyen, hanem egy irányított evolúció. A következő egységben a szélesebb migrációs stratégiákat – a 6R keretrendszert és a fokozatos átállás egyéb formáit – vizsgáljuk meg.

26.4 Migrációs stratégiák: 6R és fokozatos átállás

A migrációs stratégia megválasztása mindig üzleti, technológiai és kockázati kompromisszum. Nem az a kérdés, hogy egy adott rendszert lehet-e technikailag migrálni, hanem az, hogy a rendelkezésre álló költség, idő, csapatkapacitás és üzleti kockázat mellett melyik út adja a legnagyobb értéket, miközben a szolgáltatás folyamatosan elérhető marad. A 6R keret – rehost, replatform, refactor, repurchase, retire, retain – egy döntési térképet ad a modernizációs lehetőségekhez. Ebben a fejezetben a stratégiák mellett a fokozatos átállás, a párhuzamos futás és a visszaállás tervezési kérdéseit is bemutatjuk, mert a migráció kockázatát nem elsősorban a technológiai bonyolultság, hanem a rosszul megtervezett átmenet okozza.

26.4.1 1. Miért van szükség migrációs stratégiára?

A legacy rendszerek cseréje ritkán történhet meg egyetlen nagy, mindent kiváltó kiadással. Az üzleti folyamatok, a függőségek, az adatvagyon és a szolgáltatási szintek miatt a modernizáció gyakran hosszú, szakaszos folyamat. A stratégia kiválasztása azt rögzíti, hogy:

- a régi és az új rendszer milyen viszonyban él egymással az átmenet idején,
- milyen erőforrásokat igényel a projekt,

- mekkora az egyes lépésekben vállalt üzleti kockázat,
- milyen visszaállási pontok léteznek, ha a migráció nem a tervek szerint halad.

A migrációs stratégia tehát nem egyszeri technológiai döntés, hanem kockázatkezelési terv. A cél az, hogy a szervezet minden pillanatban választhassa a lehető legkisebb üzleti kockázatú, de a jövőbeli célarchitektúrához legjobban illeszkedő lépést.

A 6R keret az Amazon Web Services (AWS) által ismertté vált migrációs osztályozásból ered, de attól függetlenül általánosan alkalmazható. A hat kategória nem egymást kizáró, hanem egy portfólió-szintű gondolkodást támogat: egy nagyvállalati alkalmazáskészlet modernizációja során jellemzően egyszerre több R stratégiát alkalmaznak.

26.4.2 2. A 6R stratégia áttekintése

A hat stratégia mindegyike más-más modernizációs mélységet és eltérő kockázati profilt jelent. Az alábbi felsorolás az egyes R-eket definiálja, majd a döntési szempontokat mutatja be.

26.4.3 2.1. Rehost („lift and shift”)

A rehost során az alkalmazást lényegi átalakítás nélkül, csak a futtatási környezet megváltoztatásával telepítjük új helyre. Klasszikus példa egy fizikai vagy virtuális gépen futó monolitikus alkalmazás átemelése felhőalapú virtuális gépre vagy konténerbe, anélkül hogy az alkalmazáskódot vagy az adatmodellt jelentősen módosítanánk.

Előnyei: gyors, viszonylag alacsony kockázat, a csapat megismeri a célplatformot anélkül, hogy az üzleti logikát át kellene írni.
Hátrányai: nem hozza ki a modern környezet előnyeit; a régi monolit rossz működési tulajdonságai megmaradnak. A rehost gyakran csak egy közbülső lépés.
Jellemző használat: rövid határidős adatközpont-kiváltás, gyors költségcsökkentés, vagy biztonsági kényszer miatti sürgős kiszervezés.

26.4.4 2.2. Replatform („lift and reshape”)

A replatform a rehost és a refactor között helyezkedik el: az alkalmazást úgy módosítjuk, hogy jobban illeszkedjen a célplatform szolgáltatásaihoz, de a mag architektúrát és az üzleti logikát lényegében megtartjuk. Például egy saját üzemeltetésű relációs adatbázist felhőszolgáltató által kezelt adatbázisra cserélünk, vagy egy monolitot úgy konfigurálunk át, hogy horizontálisan skálázható konténerklaszterben fusson.

Előnyei: a platform nyújtotta menedzselt szolgáltatások révén csökkenthető az üzemeltetési teher, javítható a rendelkezésre állás, miközben az üzleti logika érintetlen marad.
Hátrányai: nem szünteti meg a kód szintű technikai adósságot, és bizonyos módosítások a régi rendszerben is kockázatot hordozhatnak.
Jellemző használat: az első lépés az üzemeltetési problémák enyhítésére, mielőtt egy nagyobb refaktorba kezdenénk.

26.4.5 2.3. Refactor („re-architect”)

A refactor a legmélyebb technológiai beavatkozás: az alkalmazást úgy írjuk át vagy strukturáljuk újra, hogy kihasználja a célarchitektúra lehetőségeit. Ide tartozik a monolit felbontása mikroszolgáltatásokra, az eseményvezérelt modell bevezetése, az adatmodell újradefiniálása vagy a moduláris monolit kialakítása.

Előnyei: ez a stratégia teremti meg a hosszú távú evolúciós architektúrát; csökkenti a jövőbeli technikai adósságot, javítja a skálázhatóságot és a csapatok önállóságát.
Hátrányai: idő- és költségigényes, magas a kockázat, mivel az üzleti viselkedés megváltozhat; gyakran igényli az adott domain alapos újragondolását.
Jellemző használat: ha a jelenlegi architektúra blokkolja az üzleti célokat, vagy ha a rendszer hosszú távú stratégiai fontosságú.

26.4.6 2.4. Repurchase („drop and shop”)

A repurchase során egy kész, gyakran SaaS-alapú termékkel váltjuk ki a meglévő rendszert. A szervezet lemond az egyéni fejlesztésről, és egy külső szolgáltató bevált megoldását használja. Példák: CRM, ERP, HR rendszerek, számlázási szolgáltatások.

Előnyei: gyors bevezetés, folyamatos szolgáltatói frissítések, nincs belső üzemeltetési teher.
Hátrányai: korlátozott testreszabhatóság, integrációs kihívások, lock-in kockázat, adatvédelmi és megfelelési kérdések.
Jellemző használat: ha az üzleti folyamat nem stratégiai megkülönböztető, és a piaci termék megfelel a követelményeknek.

26.4.7 2.5. Retire („decommission”)

A retire, azaz kivezetés akkor alkalmazható, ha az alkalmazás funkciója már nem szükséges, vagy más rendszerek már lefedik a képességeit. A kockázat itt nem a migráció, hanem a fel nem használt rendszerhez kötődő felesleges költségek és biztonsági felület fenntartása.

Előnyei: megszünteti az üzemeltetési és licenckköltségeket, csökkenti a támadási felületet.
Hátrányai: az adatok megőrzése, illetve az esetleges rejtett függőségek miatt a kivezetés előtt alapos elemzés szükséges.
Jellemző használat: a portfólió racionalizálásakor, vállalatfelvásárlások után, vagy amikor egy alkalmazás forgalma tartósan nulla.

26.4.8 2.6. Retain („revisit”)

A retain stratégia azt jelenti, hogy a rendszer jelenlegi formájában marad, és a modernizációs döntést későbbre halasztjuk. Ez lehet tudatos halasztás, amikor az üzleti érték vagy a kockázat még nem indokolja a cserét, de lehet kényszerű halasztás is, ha más projektek nagyobb prioritást élveznek.

Előnyei: erőforrásokat szabadít fel, elkerüli a felesleges beavatkozást.
Hátrányai: a technikai adósság tovább halmozódik, a fenntartási költségek növekedhetnek, és a későbbi migráció egyre nehezebbé válhat.
Jellemző használat: alacsony üzleti hatású vagy stabilan működő rendszerek esetén, tudatos felülvizsgálati dátumban rögzítve.

A hat stratégia közötti különbségeket az alábbi táblázat foglalja össze:

Stratégia	Beavatkozás mélysége	Tipikus időtartam	Üzleti kockázat	Technikai adósság hatása
Rehost	minimális	hetek	alacsony	változatlan
Replat-form	közepes	hónapok	közepes	részben csökken
Refactor	mély	hónapok–évek	magas	jelentősen csökken
Repurcha-se	külső termék	hetek–hónapok	közepes	megszűnik, de lock-in léphet fel
Retire	nincs	hetek	alacsony	megszűnik
Retain	nincs	–	minimális	növekszik

26.4.9 3. Döntési szempontok és a 6R kiválasztása

A stratégiát nem önmagában, hanem a rendszer üzleti kritikussága, a meglevő adósság, a rendelkezésre álló költségvetés, a kockázattűrő képesség és a célarchitektúra függvényében kell megválasztani. Nincs univerzálisan „jó” R; ugyanaz a rendszer más környezetben más stratégiát igényelhet.

26.4.10 3.1. Fő döntési tényezők

- **Üzleti érték és kritikusság:** egy nagy bevételű, folyamatos üzemű rendszer esetén a refactor hosszú távon megtérül, de a migráció közbeni rendelkezésre állás kiemelten fontos. Egy kisebb, belső adminisztrációs alkalmazásnál a repurchase vagy a retire is megfelelő lehet.
- **Technikai adósság mértéke:** ha a kód olyan mértékben romlott, hogy a további fejlesztések ellehetetlenülnek, a rehost csak időleges megoldás. Ha az adósság elsősorban infrastrukturális (például elavult operációs rendszer), a replatform elegendő.
- **Csapatkapacitás és szakértelem:** a refactor mély ismereteket igényel a domainről és a céltechnológiákról. Ha a csapat ezzel nem rendelkezik, először érdemes kisebb lépéssel tanulni.
- **Határidő és költségkeret:** egy rövid, kötött határidejű migrációnál a rehost vagy a replatform reálisabb, még akkor is, ha hosszú távon a refactor lenne az optimális.
- **Megfelelési és biztonsági követelmények:** bizonyos ágazatokban a SaaS-alapú repurchase adatvédelmi okokból kizárt, míg a retire esetén gondoskodni kell az adatok hosszú távú archiválásáról.

26.4.11 3.2. Kockázat és a döntési mátrix

A döntést célszerű mátrixban rögzíteni, ahol minden R-et értékelünk a fenti szempontok szerint. A pontozás nem matematikai igazság, hanem a beszélgetés alapja. A fejezet későbbi laborfeladata egy ilyen mátrix tényleges kitöltése.

A kockázatkezelés során minden stratégia mellé érdemes feljegyezni:

- a legnagyobb veszélyforrást (például adatvesztés, hosszú leállás, integrációs hiba),
- a bekövetkezés valószínűségét és hatását,
- a csökkentésére tett lépéseket,
- a visszaállási tervet.

26.4.12 4. Fokozatos átállás és párhuzamos futás

A 6R megadja, hogy mi történjen a rendszerrel hosszú távon, de nem írja le az átmenet mikéntjét. A fokozatos átállás és a párhuzamos futás a kockázatkezelés két alapvető eszköze.

26.4.13 4.1. Fokozatos átállás: a strangler fig elv

Ahogy az előző fejezetben láttuk, a strangler fig (fojtófüge) minta arra épül, hogy a régi rendszert nem egyszerre, hanem funkcióról funkcióra váltjuk ki. Az új rendszer fokozatosan veszi át a forgalom egyre nagyobb részét, miközben a régi rendszer tovább működik. Ez a megközelítés a refactor és a replatform gyakori kísérője, de más R-eknél is hasznos lehet.

A fokozatosság előnye, hogy a migráció közben is van működő rendszer, a hibák hatása kisebb, és a csapat tanulhat az első lépésekből. A hátránya a kettős üzemeltetés, az adatszinkronizáció szükségessége és a hosszabb átmeneti időszak.

26.4.14 4.2. Párhuzamos futás (parallel run)

A párhuzamos futás olyan átmeneti állapot, amikor a régi és az új rendszer egyidejűleg dolgozza fel ugyanazt a kérést vagy adatot, és az eredményeket összehasonlítjuk. Ez eltér a fokozatos átállástól, ahol egy adott funkciót csak az egyik rendszer szolgál ki. A párhuzamos futás lényege az egyenértékűség ellenőrzése, mielőtt a forgalom véglegesen az új rendszerre terelődik.

A párhuzamos futásnak több formája létezik:

- **Árnyéküzem (shadow mode):** az új rendszer megkapja a valós kéréseket, de válaszát nem küldjük vissza a felhasználónak; csak naplózzuk és összehasonlítjuk a régivel.
- **Mindkét rendszer éles választ ad:** a válasz a régi rendszerből érkezik, de az új rendszer eredményét háttérben rögzítjük.
- **Tényleges párhuzamosság:** mindkét rendszer éles, és a felhasználói csoportok vagy a forgalom meghatározott része az új rendszert használja.

A párhuzamos futás célja az eltérések felderítése, a teljesítmény és a funkcionális helyesség mérése. Különösen fontos pénzügyi, számlázási vagy egészségügyi rendszereknél, ahol egy hiba súlyos következményekkel járhat.

Az alábbi Mermaid folyamatábra a fokozatos átállás és a párhuzamos futás tipikus fázisait mutatja:

```
flowchart TD
    A[Előkészítés: kiinduló állapot felmérése, mérőszámok rögzítése] --> B[Pilot: egy
↪   kiválasztott funkció átirányítása az új rendszerre]
    B --> C[Párhuzamos futás: régi és új rendszer együtt dolgoz, eredmények összehasonlítása]
    C --> D{Bizalom elérte a szintet?}
    D -- Nem --> E[Hibajavítás, finomhangolás]
    E --> C
    D -- Igen --> F[Forgalom fokozatos növelése az új rendszer felé]
    F --> G[Teljes átállás: régi rendszer kivezetése vagy tartalék üzem]
    G --> H[Leszerelés és archiválás]
    A --> I[Visszaállási terv karbantartása minden fázisban]
    I --> C
    I --> F
    I --> G
```

Jelmagyarázat: a nyilak a folyamat irányát jelölik; a visszaállási terv párhuzamosan létezik minden aktív fázissal. A diagram nem tartalmaz kódot, csak a migráció fázisait.

26.4.15 4.3. Mérés a párhuzamos futás során

A párhuzamos futás értéke csak akkor realizálódik, ha számszerűen mérjük az eltéréseket. Például egy számlázó rendszer migrációjánál minden tranzakcióra rögzíthetjük:

- a régi és új rendszer által számított számla végösszegét,
- a feldolgozási időt,
- a hibák számát.

Az eltérésekre riasztást érdemes beállítani. A mérési ablakot például 7 napos párhuzamos futásra tervezzük, és azt vizsgáljuk, hogy az eltérések aránya 0,05% alatt marad-e. Ha nem, a forgalmat visszatereljük a régi rendszerre.

A párhuzamos futás közben figyelni kell a kapacitásra is. Ha mindkét rendszer ugyanazt a terhelést kapja, akkor a teljes erőforrásigény megduplázódhat. A Little-törvény szerint $L = \lambda W$; ha a párhuzamosítás miatt a rendszerben tartózkodó kérések száma növekszik, a válaszidő is nőhet. Ezért a párhuzamos futást gyakran nem 100%-os forgalmon, hanem csak a kérések egy mintáján végzik.

26.4.16 5. Visszaállás és rollback stratégiák

A migrációs kockázatkezelés egyik legfontosabb eleme a visszaállási terv. A visszaállás nem azt jelenti, hogy minden lépésnél képesek vagyunk a régi állapotot pontosan helyreállítani, hanem hogy a szolgáltatás megszakítás nélkül, elfogadható romlás mellett tovább üzemel.

26.4.17 5.1. A visszaállás formái

- **Azonnali visszaterelés a régi rendszerre:** a forgalomirányítás egy kapcsolóval (például API gateway vagy DNS) visszafordítható. Ez csak akkor működik, ha a régi rendszer még üzemel, és az adatok szinkronizáltak.
- **Rollback a korábbi verzióra:** az új rendszer korábbi, stabil kiadását állítjuk vissza. Ez tipikusan deployment stratégia kérdés, és a fejezetben példaként a blue-green vagy canary eljárással kombinálható.
- **Roll forward:** hiba esetén a megoldást nem visszaállással, hanem gyors javítással visszük ki. Ez a gyakorlatban gyakran gyorsabb, mint a visszaállítás, de csak akkor alkalmazható, ha a hiba jól lokalizálható.
- **Párhuzamos tartalék üzem:** a régi rendszert mint meleg tartalékot üzemeltetjük, és katasztrófa esetén azonnal átveszi a forgalmat.

26.4.18 5.2. Adatkonzisztencia a visszaállításnál

A legnehezebb visszaállítási probléma általában az adat. Ha az új rendszer írja az adatbázist, és a régi rendszerre vissza kell térni, akkor a régi rendszer által használt adatmodellnek szinkronban kell lennie. A fokozatos átállásnál ezt úgy oldják meg, hogy az adatréteg egy köztes rétegre ír, amely mindkét rendszer számára elérhető, vagy folyamatos replikációval szinkronizálják a kettőt. A visszaállítási tervben pontosan rögzíteni kell, hogy milyen adatvesztés vagy duplikáció fordulhat elő, és azt hogyan kezeljük.

26.4.19 5.3. Visszaállási gyakorlatok

A visszaállítási terv addig csak papír, amíg nem tesztelik. A migráció minden fázisában érdemes visszaállítási gyakorlatot tartani. A laborkörnyezetben szimulálható hiba, és mérhető a visszaállási idő, amelynek például 15 percen belül kell lennie. A gyakorlat során a mérést dokumentálni kell: időpont, terhelés, a hiba típusa, a visszaállás teljes időtartama és az adott pillanatban érvényes SLO.

26.4.20 6. Kockázatkezelés és mérés a migrációban

A migráció kockázatkezelése nem eseti, hanem folyamatos tevékenység. A kockázatokat regiszterben tartjuk, és a mérési adatokkal frissítjük. Az alábbi minőségi forgatókönyv mutatja, hogyan lehet a kockázatot quality attribute scenario formában megfogalmazni.

Quality attribute scenario – parallel run adatszinkronizáció hiba
- **Inger:** a régi és az új rendszer közötti adatszinkronizációs folyamat 10 percre megszakad.
- **Környezet:** normál üzemidő, párhuzamos futás fázisa, a forgalom 30%-a az új rendszerre irányul.
- **Válasz:** a rendszer észleli a hibát, riasztást küld, és automatikusan visszaállítja a forgalmat a régi rendszerre, miközben a hibás új rendszer felől bejövő eredményeket figyelmen kívül hagyja.
- **Mérőszám:** a hiba aktív ideje 5 percen belül legyen; adatvesztés 0 rekord; a havi rendelkezésre állás ne essen 99,9% alá.

Egy ilyen forgatókönyv konkrét, mérhető, és a teszteléshez is használható. A rendelkezésre állást a szokásos képlettel számoljuk:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Ha az SLO 99,9%, akkor az adott hónapban legfeljebb $1 - 0{,}999 = 0{,}001 = 0{,}1\%$ állásidő engedhető meg. A migráció során ez az error budget gyorsan kimerülhet, ezért a változtatásokat akkorra ütemezzük, amikor még van tartalék.

A kapacitástervezésnél a párhuzamos futás alatt a rendszerben lévő kérések átlagos számát a Little-törvény adja: $L = \lambda W$. Ha a párhuzamos futás miatt λ duplázódik, és a válaszidő W nem csökken arányosan, a rendszer túlterheltté válhat. Ezt a mérési ablakban figyelni kell.

26.4.21 7. Labor: 6R döntési mátrix kitöltése

A labor célja, hogy egy adott alkalmazásra a 6R keretet alkalmazza, és a döntést reprodukálható dokumentumban rögzítse.

26.4.22 7.1. Kiinduló állapot

Válasszon egy kitalált vagy anonimizált legacy rendszert. Például: egy 15 éves, monolitikus számlázó rendszer, amely egy belső fizikai szerveren fut, Java 7-es alkalmazás, Oracle adatbázissal. A havi üzemeltetési költség magas, a fejlesztési sebesség lassú, és a biztonsági frissítések hiánya aggodalomra ad okot. A rendszer havi 50 000 számlát állít ki, 40 belső felhasználóval. A funkciók között van számlázás, díjbekérő generálás és riportkészítés.

26.4.23 7.2. Elvárt eredmény

Töltse ki a 6R döntési mátrixot az alábbi szempontokkal: üzleti érték, technikai adósság hatása, várható költség, időtartam, kockázat, csapat illeszkedés, hosszú távú előny. Minden sorhoz írjon egy rövid indoklást, és adjon pontszámot 1-től 5-ig (1 = gyenge, 5 = kiváló).

Példa a mátrixra, amelyet kitöltve kell leadni:

Stratégia	Üzleti érték	Adósság csökkenés	Költség	Idő	Kockázat	Illeszkedés a célarchitektúrához	Összpontszám
Rehost							
Replatform							
Refactor							
Repurchase							
Retire							
Retain							

A pontozásnál ügyeljen arra, hogy a kockázat skálája fordított: minél alacsonyabb a kockázat, annál magasabb pontszám jár. A súlyozás a szervezet preferenciáitól függ; ezt a laborban explicit módon rögzíteni kell.

26.4.24 7.3. Döntési napló (ADR minta)

A mátrix alapján készítsen egy rövid ADR-t. Az alábbi szerkezetet használja:

```
# ADR-2025-0XX: [Rendszer] migrációs stratégiája

## Kontextus
A [rendszer] jelenlegi állapota, az üzleti kritikusság, a havi forgalom.
A fennálló technikai adósság és a működési kockázatok leírása.
A rendelkezésre álló költségkeret és határidő.

## Döntés
A választott stratégia (például replatform) és a migráció fokozatos
megvalósításának fő lépései.

## Alternatívák
- Rehost: ...
- Refactor: ...
- Repurchase: ...
- Retire: ...
- Retain: ...

## Következmények
- A döntés megszünteti az alábbi kockázatokat...
- Új kockázatokat vezet be...
- A csapatnak a következő készségeket kell fejlesztenie...

## Állapot
Javasolt / Elfogadva / Elavult
```

26.4.25 7.4. Ellenőrzési szempontok

A labor akkor tekinthető sikeresnek, ha:

- A mátrix mind a hat stratégiát értékeli.
- A pontszámok mellett minden sorhoz tartozik szöveges indoklás.
- A kockázat értékelése nem csak szubjektív vélemény, hanem a rendszer jellemzőiből levezetett.
- A döntési napló a fejezetben bemutatott ADR sémát követi.
- A mért és a várt eredmények világosan elkülönülnek: például ha a párhuzamos futás tesztje során mérés történt, azt külön táblázatban rögzíti a becsült értéktől.
- A dokumentum verziózott, és a mátrix kitöltésének időpontja szerepel.

26.4.26 7.5. Visszaállítás a labor után

A labornak nincs éles rendszerre hatása. Ha azonban a gyakorlatot egy tesztkörnyezetben hajtjuk végre, a kiinduló állapotot a labor előtt le kell menteni, és a labor után vissza kell állítani. A visszaállítási út: a virtuális gép snapshotjának visszaállítása, az adatbázis biztonsági mentésének importálása, és a konfigurációs állományok eredeti verziójának becsekkolása.

26.4.27 8. Tipikus hibák és elkerülésük

A migrációs projektekben visszatérő hibák a következők:

- **Big bang kis robbantás helyett:** a szervezet egyetlen nagy átállást tervez, mert így látszik egyszerűbbnek. A kockázat ekkor koncentrálódik, és egyetlen hiba az egész rendszert érinti. **Helyes megoldás:** fokozatos átállás, pilot, párhuzamos futás.
- **A visszaállás elhanyagolása:** a csapat a migráció sikerére fókuszál, és nem készül a kudarcra. **Helyes megoldás:** minden fázisban legyen tesztelt rollback terv.
- **A párhuzamos futás kihagyása:** az új rendszert azonnal élesbe állítják, mert „a tesztek lefutottak”. A tesztkörnyezet nem reprodukálja a valós terhelést és adatminőséget. **Helyes megoldás:** árnyéküzem vagy párhuzamos futás legalább egy meghatározott ablakban.
- **A döntési mátrix csak papír:** a 6R-t kiválasztják, de a döntési szempontokat nem rögzítik, így később vitatható, miért az adott út mellett döntöttek. **Helyes megoldás:** ADR és rendszeres felülvizsgálat.
- **A refactor aranyozása:** a csapat a legmodernebb architektúrát akarja, miközben a rendszer üzleti értéke nem indokolja a mély beavatkozást. **Helyes megoldás:** a 6R mátrixban az üzleti értéket kell elsődleges szempontként kezelni.
- **A retain mint kifogás:** a „majd később” döntés gyakran a döntés elkerülése. **Helyes megoldás:** a retain esetén is legyen felülvizsgálati dátum és tulajdonos.

26.4.28 9. Összefoglalás

A migrációs stratégia kiválasztása a 6R kerettel nem a technológiai preferenciát, hanem a kockázatkezelt fokozatos átmenetet helyezi előtérbe. A rehost, replatform, refactor, repurchase, retire, retain mindegyike más üzleti és technikai kontextusban hasznos. A siker kulcsa nem egyetlen „jó” R megtalálása, hanem a döntés átlátható rögzítése, a párhuzamos futás és a visszaállási terv folyamatos fenntartása, valamint a mérés a migráció minden fázisában. A következő fejezetben az evolúciós architektúra és a fitness functions eszközrendszere arra épül, hogy az ilyen döntések hosszú távon folyamatosan ellenőrizhetők legyenek.

26.4.29 10. Ellenőrzőlista a migrációs stratégia elindításához

- ☐ A rendszer üzleti kritikussága és forgalmi jellemzői dokumentálva.
- ☐ A technikai adósság és a működési kockázatok felmérve.
- ☐ A 6R minden eleme legalább felületesen értékelve.
- ☐ A választott stratégia indoklása ADR-ben rögzítve.
- ☐ A fokozatos átállás lépései fázisokra bontva.
- ☐ A párhuzamos futás mérési terve elkészítve, eltérési küszöbértékekkel.
- ☐ Visszaállási terv minden aktív fázisra kidolgozva és tesztelve.
- ☐ A mérőszámokhoz tartozik környezet, konfiguráció, időpont és bizonytalanság.
- ☐ A migráció nem befolyásolja a jelenlegi SLO-kat és error budgetet.

26.4.30 11. Önellenőrző kérdés

Egy nagy forgalmú e-kereskedelmi rendszert szeretnénk kiváltani egy microservice-alapú új platformmal. Az üzlet folyamatos működést követel, a régi rendszer erősen csatolt, és az adatmodellje nem egyezik az új szolgáltatásokéval. Milyen migrációs stratégiát választana, és milyen szerepet kapna a párhuzamos futás? Válaszában térjen ki a kockázatkezelésre, a visszaállásra és a mérésre is.

27 Evolúciós architektúra és governance

27.1 Evolúciós architektúra és fitness functions

Amikor egy rendszer elérte a működő állapotot, az architektúra nem válik késztermékké. A technológiai környezet, az üzleti követelmények, a felhasználói szokások és a szervezet felépítése folyamatosan változik; az architektúra ezért vagy tudatosan fejlődik, vagy irányítatlanul sodródik. A statikus, egyszeri architektúra-review már nem elég ahhoz, hogy ezt a sodródást megelőzzük. Olyan visszacsatolási mechanizmusra van szükség, amely a fejlesztés és üzemeltetés mindennapi ciklusába ágyazottan, automatikusan és mérhetően jelzi, ha a rendszer eltér a kívánt architekturális tulajdonságaitól. Ezt a szerepet tölti be az evolúciós architektúra és annak legfontosabb eszköze, a fitness function.

27.1.1 Az evolúciós architektúra lényege

Az evolúciós architektúra nem egy új stílus, hanem egy megközelítés: az architektúrát olyan rendszerként kezeljük, amelynek fontos tulajdonságait folyamatosan mérjük, és a mérési eredmények alapján automatikus döntési kapukat állítunk fel. A cél nem az, hogy minden részletet előre megtervezzünk, hanem az, hogy a rendszer fejlődése közben is megőrizzük azokat a minőségi attribútumokat vagy szerkezeti megszorításokat, amelyek a hosszú távú fenntarthatóságot, teljesítményt, biztonságot vagy üzemeltethetőséget garantálják.

A hagyományos architektúra-ellenőrzés gyakran utólagos: készül egy diagram, egy dokumentum, egy review, majd a következő nagy release előtt újra átnézik. A probléma ezzel az, hogy a rendszer közben naponta vagy óránként változik. Mire a következő review bekövetkezik, a kód már rég eltávolodhatott a dokumentációtól, és a hibás szerkezet beépült a csapat mindennapi szokásaiba. Az evolúciós architektúra ezzel szemben a folyamatos integrációs és szállítási folyamatba (CI/CD) ágyazza a mérést, így a visszacsatolás perceken belül, nem pedig hónapok múlva érkezik.

A folyamatos evolúció azt jelenti, hogy a rendszer minden változtatásakor ellenőrizzük, hogy a változtatás nem sérti-e meg a kritikus architekturális jellemzőket. Ez nemcsak a kódra vonatkozik, hanem az infrastruktúrára, a konfigurációra, az adatsémára, sőt a szervezeti folyamatokra is. A mérhetőség feltétele, hogy a jellemzőket olyan mérőszámokra fordítsuk le, amelyek automatikusan kiértékelhetők, és amelyekhez egyértelmű küszöbérték tartozik.

A klasszikus minőségi attribútum-forgatókönyv (quality attribute scenario) ebben a keretben is hasznos: az inger, a környezet, a válasz és a mérőszám megfogalmazása segít abban, hogy a fitness function ne általános célokat, hanem konkrét, tesztelhető állításokat vizsgáljon. A különbség az, hogy a fitness function ezt a forgatókönyvet lefordítja egy automatikusan futtatható ellenőrzésre, és a futtatás eredménye beavatkozik a szállítási folyamatba.

27.1.2 Kulcsfontosságú tulajdonság és küszöb

Az evolúciós architektúra két alapfogalma a **kulcsfontosságú tulajdonság** (key property) és a **küszöb** (threshold). A kulcsfontosságú tulajdonság az az architekturális jellemző, amelyet minden áron meg akarunk őrizni a rendszer evolúciója során. Ez lehet:

- szerkezeti megszorítás, például rétegek közötti függőségi szabály vagy modulok közötti csatolási korlát;
- minőségi attribútum, például a válaszidő 95. percentilise vagy a rendelkezésre állás;
- biztonsági követelmény, például a titkos kulcsok nem kerülhetnek verziókezelőbe;
- adatkezelési szabály, például a személyes adatok nem léphetnek át egy bizalmi határt;
- üzemeltetési jellemző, például a telepítési gyakoriság alsó korlátja vagy a hibajavítási idő felső korlátja.

A kulcsfontosságú tulajdonság kiválasztása architektúra-döntés, és nem technikai automatizálási feladat. Rosszul megválasztott tulajdonság esetén a fitness function hiába fut, nem védi a rendszer valódi értékeit. A döntést a stakeholderek, az üzleti célok, a szabályozási kényszerek és a korábbi hibák elemzése alapján kell meghozni. Például egy banki rendszerben a tranzakciós konzisztencia vagy az auditálhatóság lehet kulcsfontosságú; egy e-kereskedelmi rendszerben lehet a rendelési folyamat válaszideje; egy orvosi eszköznél a szabványok szerinti nyomon követhetőség.

A **küszöb** az a számérték vagy feltétel, amelynél a fitness function sikeresnek vagy sikertelennek minősül. A küszöböt nem szabad önkényesen kitalálni. Alapulhat:

- szabványon vagy szabályozási előíráson (például maximális leállási idő);
- üzleti célon (például a kosár oldal p95 válaszideje legfeljebb 200 ms, mert e felett mérhetően csökken a konverzió);
- múltbeli adatokon (például a jelenlegi p95 180 ms, ezért a küszöb 250 ms, hogy a romlást még időben észleljük);
- kockázati elemzésen (például a hibatűrési tesztben a hibás válaszok aránya nem haladhatja meg az 1%-ot);
- csapattal közös egyeztetésen, de csak akkor, ha a feltételezést dokumentálják és később validálják.

A küszöbnek konkrétnak, mérhetőnek és időhöz kötöttnek kell lennie. A „gyors válaszidő” nem küszöb; a „p95 250 ms, 50 kérés/s terhelés mellett, 5 perces mérési ablakban” már az. A küszöböt rendszeresen felül kell vizsgálni, mert a rendszer és a környezet változásával a korábbi érték elveszítheti értelmét. Ha például a termék átáll egy interaktívabb felhasználói élményre, a korábban elfogadható 500 ms-os küszöb már nem védi az üzleti célt.

A fitness function formálisan egy olyan ellenőrzés, amely a kulcsfontosságú tulajdonsághoz tartozó mért értéket hasonlítja össze a küszöbbel. Egyszerű alakban:

$$f(m) \geq \theta \quad \text{vagy} \quad f(m) \leq \theta,$$

ahol m a mért metrika, θ a küszöb. Ha a feltétel teljesül, a fitness function sikeres; ha nem, sikertelen. A gyakorlatban a fitness function lehet ennél összetettebb: több metrikát kombinál, időablakot vesz figyelembe, vagy akár súlyozott pontrendszert használ. A lényeg mindig ugyanaz: a döntés automatikusan, reprodukálhatóan és a kontextus alapján meghatározott küszöbhöz képest születik.

27.1.3 A mérés és a döntés összekapcsolása

A fitness function önmagában egy mérés; az evolúciós architektúra ereje abban rejlik, hogy ezt a mérést összekapcsoljuk egy döntéssel. A mérés eredménye nem csupán egy dashboardon jelenik meg, hanem belép a szállítási folyamatba mint kapu (gate). Ez a kapu háromféle viselkedést mutathat:

1. **Blokkolás:** ha a fitness function sikertelen, a build vagy a deployment nem folytatódhat. Ezt akkor használjuk, ha a tulajdonság biztonsági, adatvédelmi vagy olyan kritikus működési jellemző, amelynek megsértése elfogadhatatlan. Például a titkos kulcsok szivárgását ellenőrző fitness function meghiúsulása azonnal leállítja a kiadást.
2. **Figyelmeztetés:** a fitness function sikertelen, de a folyamat folytatódik, miközben a rendszer automatikusan értesíti az érintett csapatot, és a hibát a technikai adósság vagy kockázati regiszterbe rögzíti. Ezt akkor alkalmazzuk, ha a tulajdonság romlása nem azonnali üzleti kiesést okoz, de hosszú távon fenntarthatatlanná tenné a rendszert. Például a moduláris függőségi szabály megsértése figyelmeztetést generálhat, mert a kód még működik, de a szerkezet romlani kezd.
3. **Riasztás és audit:** a fitness function folyamatosan fut, eredményeit idősorosan tárolják, és a küszöb átlépése riasztást vált ki, valamint az esemény bekerül a döntési naplóba. Ez a megoldás akkor hasznos, ha a jellemzőt nem egyetlen buildhez, hanem az üzemelő rendszerhez képest vizsgáljuk (például a termelési válaszidő romlása).

A döntési kapu bevezetésekor fontos meghatározni, hogy a fitness function mikor fusson le. Egyes ellenőrzések a CI-folyamatban, minden pull requestnél lefutnak (például statikus függőségi elemzés), mások a CD-folyamatban, csak a kiadás előtt (például sebezhetőségi szűrés), megint mások folyamatosan az éles rendszeren (például SLO-megfigyelés). A mérési gyakoriságnak igazodnia kell a kockázathoz és a változtatás sebességéhez: minél gyorsabban változik egy komponens, annál sűrűbben kell mérni.

A mérést és a döntést együtt kell dokumentálni, hogy a jövőbeli olvasó megértse, miért éppen az adott küszöböt választottuk, milyen alternatívákat vetettünk el, és milyen következményekkel jár a sikertelen eredmény. Erre az Architecture Decision Record (ADR) a megfelelő eszköz. Az ADR kontextusa rögzíti a kulcsfontosságú tulajdonságot és annak üzleti vagy műszaki okát; a döntés tartalmazza a fitness function definícióját, a mérési módszert és a küszöböt; az alternatívák felsorolják a többi lehetséges megközelítést; a következmények pedig a pozitív és negatív hatásokat, beleértve a téves riasztások kockázatát is.

Fontos különbséget tenni a **mérés** és a **döntés** között. A mérés objektív adatot szolgáltat; a döntés a mérés értelmezése a küszöb és a kontextus alapján. Ugyanaz a mért érték különböző kontextusban más döntést eredményezhet. Például a p95 válaszidő 220 ms egy belső adminisztrációs felületen elfogadható lehet, egy vásárlói kosár oldalon már nem. A fitness function ezért mindig a kontextust is magában foglalja, nem csak a nyers metrikát.

A mérési bizonytalanságot is kezelni kell. Egyetlen terheléses teszt futása nem feltétlenül megbízható; a fitness function lehetőleg több mérési pontot, konfidencia-intervallumot vagy statisztikai kritériumot használjon. Ha a metrika ingadozó, a küszöb átlépését ne egyetlen minta, hanem egy csúszóablak átlaga vagy percentilise alapján döntsük el. A hibás döntések – tévesen sikeres vagy tévesen sikertelen eredmények – csökkentik a csapat bizalmát a mechanizmusban, és végül ahhoz vezetnek, hogy a fitness functiont figyelmen kívül hagyják vagy kikapcsolják.

27.1.4 Fitness function típusok és példák

A fitness functionokat két nagy csoportba sorolhatjuk: **statikus** és **dinamikus** ellenőrzésekre. A statikus fitness function a forráskód, a konfiguráció vagy a diagramok elemzésével méri a szerkezeti tulajdonságokat, anélkül hogy a rendszert futtatnánk. A dinamikus fitness function a működő rendszer viselkedését méri, például terhelés alatt, vagy az éles forgalom alapján.

27.1.5 Statikus fitness function példák

- **Moduláris függőségi szabály:** ellenőrzi, hogy bizonyos csomagok vagy modulok nem függnek egymástól tiltott módon. Például a `payment` modul nem importálhat közvetlenül az `inventory` modulból, mert a fizetési folyamatnak a rendeléskezelő API-ján keresztül kell kommunikálnia, nem közvetlen adatbázis-hozzáféréssel. Ezt az ellenőrzést például ArchUnit (Java), Dependency Cruiser (JavaScript/TypeScript) vagy `ts-arch` képes automatikusan futtatni a buildben.
- **Rétegek közötti határok:** a klasszikus rétegek (például UI, alkalmazás, domain, infrastruktúra) közötti függőségek irányát és mélységét vizsgálja. A UI csak az alkalmazás réteget hívhatja, közvetlenül az adatbázist nem.
- **Elnevezési és szerkezeti konvenciók:** például a domain események osztályainak a `domain.events` csomagban kell lenniük, és nem tartalmazhatnak infrastruktúra-függőséget. Ez a fajta ellenőrzés gyakran annyira egyszerű, hogy egy statikus elemző már a CI elején jelez.
- **Biztonsági sérülékenységek szűrése:** a függőségek listájából generált SBOM (Software Bill of Materials) elemzése közismert sérülékenység-adatbázisok alapján. A fitness function akkor sikeres, ha nincs kritikus vagy magas kockázatú sérülékenység, vagy ha a javított verziók már bekerültek.

27.1.6 Dinamikus fitness function példák

- **Válaszidő SLO:** a termelési forgalom alapján a kulcsfontosságú végpont p95 válaszideje legfeljebb 250 ms lehet az elmúlt 5 percben. A mérés a monitoring rendszerből származik, és a fitness function a küszöb átlépésekor riasztást generál, vagy a canary deploymentet visszavonja.
- **Hibatűrési teszt:** a rendszer egy izolált környezetben mesterséges hibát kap (például egy adatbázis-példány leáll), és a fitness function azt méri, hogy a hibás kérések aránya legfeljebb 1%, a rendszer 30 másodpercen belül újra elérhető, és a felhasználói munkamenetek nem vesznek el.
- **Kapacitás-ellenőrzés:** a terheléses teszt során a rendszernek 1000 kérés/s mellett is a megadott SLO alatt kell maradnia, miközben az erőforrás-kihasználtság (CPU, memória) nem lépi túl a biztonsági határt. Ez a fitness function megakadályozza, hogy egy új verzió éles környezetben derüljön ki a kapacitáshiány.
- **Adatkonzisztencia:** egy elosztott rendszerben a replikált adatok közötti eltérés (például a rendelés és a fizetés adatbázisa közötti inkonzisztencia) egy adott időablakban nem haladhat meg egy maximális rekordszámot. Ezt a fitness functiont folyamatosan futtatva észlelhető az outbox vagy a saga minták hibája.

- **Megfelelőségi mérőszám:** a DORA-mutatók (deployment frequency, lead time for changes, change failure rate, failed deployment recovery time) közül például a change failure rate legfeljebb 10% lehet havi ablakban. Ez a fitness function nem a kódot, hanem a szállítási folyamat minőségét méri, és a küszöb átlépésekor a csapatnak felül kell vizsgálnia a kiadási gyakorlatát.

A fenti példák közös jellemzője, hogy mindegyik egy architekturális tulajdonságot véd, nem pedig egy konkrét implementációs részletet. A fitness function nem azt ellenőrzi, hogy egy adott osztályban hány sor van, hanem azt, hogy a rendszer szerkezete vagy viselkedése megfelel-e a kívánt architektúrának.

27.1.7 Fitness function tervezése: lépések

A fitness function bevezetése nem technikai trükk, hanem tervezési folyamat. A következő lépéseket érdemes követni:

1. **Azonosítsd a kulcsfontosságú tulajdonságot.** Válaszd ki azt az architekturális jellemzőt, amelynek romlása a legnagyobb üzleti vagy műszaki kárt okozná. Ne egyszerre mindent akarj védeni; kezdj kettő-három tulajdonsággal, és idővel bővítsd a készletet. A túl sok fitness function ugyanolyan probléma, mint a semmi: a zaj elnyomja a jelzést.

2. **Fordítsd le a tulajdonságot mérhető metrikára.** Ha a tulajdonság homályos, például „modularitás”, keresd meg a mérhető megfelelőjét: a modulok közötti tiltott függőségek száma, a körkörös függőségek száma, a nyilvános API-n kívüli hívások aránya. Ha a tulajdonság a teljesítmény, használj percentilist, átlagot vagy maximális értéket, és rögzítsd az időablakot.

3. **Határozd meg a küszöböt.** Indokold meg a számértéket; ne önkényesen válassz. Ha nincs adatod, végezz kiinduló mérést, és állíts be a jelenlegi értéknél szigorúbb, de reális célt. Dokumentáld, hogy a küszöb feltételezésen alapul, és rendszeresen vizsgáld felül.

4. **Válaszd ki a mérési módszert és az eszközt.** Döntsd el, hogy statikus elemzést, terheléses tesztet, termelési metrikát vagy ezek kombinációját használod. Az eszköz legyen automatikusan futtatható, verziókezelhető és reprodukálható. A mérési módszernek illeszkednie kell a rendszer technológiájához és a csapat készségeihez.

5. **Határozd meg a döntési következményt.** Mondd ki, mi történjen, ha a fitness function sikertelen: blokkol, figyelmeztet, riaszt, vagy csak rögzíti az eredményt. A következmény legyen arányos a kockázattal: egy biztonsági hiba azonnali blokkolást igényel, egy kismértékű csatolási romlás elegendő figyelmeztetéssel.

6. **Vezesd be fokozatosan.** Ne próbáld az első naptól blokkoló kapuként futtatni a fitness functiont, ha a csapat még nem bízik a mérésben. Először figyelmeztető módban gyűjts adatot, kalibráld a küszöböt, csökkentsd a téves riasztásokat, majd fokozatosan szigoríts a blokkolásig. A fokozatosság csökkenti a szervezeti ellenállást és a rossz döntésekből adódó frusztrációt.

7. **Rögzítsd ADR-ben a döntést.** Az ADR tartalmazza a kontextust, a döntést, az alternatívákat és a következményeket. Hivatkozz a fitness function konkrét implementációjára és a mérési eredményre. Ez biztosítja, hogy a jövőbeli olvasó megértse, miért éppen ez a küszöb, és miért éppen ilyen következmény.

27.1.8 Gyakori hibák és ellenszerek

Az evolúciós architektúra bevezetésekor számos tipikus hibát követnek el. Ezeket érdemes előre ismerni, mert a hibás fitness function többet árt, mint használ.

Túl sok fitness function. Ha minden apró szabályból fitness functiont csinálunk, a csapat elveszíti a fókuszt, a build lassú lesz, és a sok sikertelen jelzés végül zajjá válik. A megoldás a prioritás: csak a valóban kritikus tulajdonságokat védjük, a többit pedig kód-review és architektúra-döntés szintjén kezeljük.

Nem mérhető tulajdonság. A „jó kódminőség” vagy „fenntartható architektúra” nem fitness function, mert nem lehet automatikusan kiértékelni. A megoldás a pontosítás: a tulajdonságot le kell bontani konkrét, mérhető mutatókra.

Önkényes küszöb. A „legyen gyors” nem küszöb; a „p95 250 ms” is csak akkor jó, ha van mögötte érvelés. Ha a küszöböt nem validáljuk, a fitness function téves riasztásokat ad, vagy éppen a valódi problémát nem jelzi. A küszöböt adatokra, üzleti célokra vagy szabványokra kell alapozni, és dokumentálni kell a feltételezést.

A fitness function nem fut automatikusan. Ha az ellenőrzést manuálisan kell futtatni, előbb-utóbb elmarad, és a rendszer visszasüllyed a statikus review szintjére. A fitness function értéke az automatizálásban van; integrálni kell a CI/CD pipeline-ba vagy a folyamatos monitoringba.

Csak az implementációt vizsgálja, nem az architektúrát. Egy kódszintű stílusellenőrzés (linter) hasznos, de nem helyettesíti a szerkezeti vagy viselkedési architektúra-ellenőrzést. A fitness functionnak a rendszer szintjén kell hatnia, nem csak egy osztály belsejében.

A fitness function bünteti a fejlesztőt, nem a rendszert. Ha a sikertelen fitness function személyes számonkérést vált ki, a csapat a tüneteket fogja kijátszani, például megkerüli a szabályt, vagy kikapcsolja az ellenőrzést. A cél a rendszer tulajdonságainak védelme, nem az egyéni hibáztatás. A visszacsatolásnak konstruktívnak kell lennie, és a hiba javítását kell elősegítenie.

A küszöböt nem frissítik. Az idők során a rendszer és az üzleti környezet változik. A korábban megfelelő küszöb túl szigorúvá vagy túl megengedővé válhat. A fitness functionokat ezért rendszeresen, például negyedévente felül kell vizsgálni, mint magát az architektúrát.

A mérési bizonytalanság figyelmen kívül hagyása. Egyetlen terheléses teszt eredménye nem reprezentatív. Ha a fitness function egyetlen mérés alapján dönt, a téves döntések valószínűsége magas. Használjunk több mérésből számított statisztikát, például percentilist vagy átlagot egy csúszóablakban, és rögzítsük a bizonytalanságot.

27.1.9 Példa: modularitási fitness function

Tegyük fel, hogy egy e-kereskedelmi rendszer három szolgáltatásból áll: `order`, `payment` és `inventory`. Az architektúra célja, hogy a `payment` szolgáltatás ne férhessen hozzá közvetlenül az `inventory` adatbázishoz, mert a készletcsökkenést a rendeléskezelőnek kell koordinálnia. A `payment` csak az `order` publikus API-ját hívhatja, az `order` pedig a rendelés állapotától függően módosítja az `inventory`-t. Ez a szabály csökkenti a szolgáltatások közötti csatolást, és lehetővé teszi a fizetési logika önálló fejlesztését.

A fitness function statikus elemzéssel ellenőrzi, hogy a `payment` modul importjai között nem szerepel az `inventory` modul. A mérés egy buildelési lépésben fut, például ArchUnit segítségével. A küszöb: a tiltott függőségek száma 0. Ha a szabály sérül, a build blokkol, mert a csatolás romlása már a kód szintjén látható, és a javítása később sokkal költségesebb lenne.

Az alábbi szemléltető pszeudokód ezt a fitness functiont írja le; nem éles futtatásra szánt, csak a szerkezetet mutatja:

```
// Szemléltető példa, nem éles környezetre
@Test
void payment_module_cannot_depend_on_inventory() {
    JavaClasses classes = new ClassFileImporter()
        .importPackages("com.example.payment");

    ArchRule rule = noClasses()
        .that().resideInAPackage("..payment..")
        .should().dependOnClassesThat()
        .resideInAPackage("..inventory..");

    rule.check(classes);
}
```

A fitness function sikeres, ha a `payment` csomagban nincs olyan osztály, amely az `inventory` csomagra hivatkozik. Ha egy fejlesztő véletlenül bevezet egy ilyen függőséget, a teszt pirosra vált, és a build megáll. A hiba üzenete a szabályt és a megsértő osztályt is megnevezi, így a javítás gyors.

Ez a példa rámutat a fitness function értékére: nem kell megvárni egy architektúra-review-t, hogy kiderüljön a csatolási hiba; a hiba már a fejlesztői ciklusban felszínre kerül.

27.1.10 Labor: fitness function és architektúra-metrika tervezése

A következő gyakorlat célja, hogy a fejezetben tárgyalt fogalmakat egy kis, ellenőrizhető példán alkalmazd. A labor során fitness functiont és architektúra-metrikát tervezel, rögzíted a kontextust, a döntési naplót és az ellenőrzési szempontokat. A mért és a várt eredményt egyértelműen elkülöníted. A labor izolált környezetben végezhető el, például helyi fejlesztői gépen vagy eldobható konténerben; éles rendszeren nem szabad futtatni.

27.1.11 Kiinduló állapot

Adott egy egyszerű rendeléskezelő szolgáltatás, amely három modulból áll: `order`, `payment` és `inventory`. A modulok jelenleg szabadon importálhatják egymás osztályait; nincs kényszerített függőségi szabály. A rendelési végpont jelenlegi p95 válaszideje 320 ms, a cél 250 ms. A rendszer terhelése tesztkörnyezetben 50 kérés/s. A forráskód verziókezelőben van, és CI/CD pipeline fut, de architektúra-ellenőrzés még nincs beállítva.

27.1.12 Feladat

Tervezz két fitness functiont az alábbi kulcsfontosságú tulajdonságokra:

1. **Modularitás:** a `payment` modul nem függhet közvetlenül az `inventory` modultól, mert a készletkezelést kizárólag az `order` modul koordinálja.
2. **Teljesítmény:** a rendelési végpont p95 válaszideje legfeljebb 250 ms, 50 kérés/s terhelés mellett, 5 perces mérési ablakban.

Mindkét fitness function esetén határozd meg:

- a mérési módszert és az eszközt;
- a küszöböt és annak indoklását;

- a futtatás gyakoriságát és helyét a CI/CD folyamatban;
- a döntési következményt (blokkolás, figyelmeztetés, riasztás);
- a mérési bizonytalanság kezelését.

27.1.13 A labor lépései

1. **Kontextusrögzítés:** írj rövid leírást a rendszerről, a két kulcsfontosságú tulajdonságról és arról, miért fontosak. Hivatkozz a minőségi attribútum-forgatókönyvre, ha rendelkezésre áll. Tüntesd fel a mérési környezetet: operációs rendszer, futásidő, verziók, konfiguráció, időpont.
2. **Fitness function definíció:** mindkét fitness functiont formálisan add meg: a mért metrikát, a küszöböt, a feltételt és a következményt. Például:
   - Modularitásra: $f_1 = \text{tiltott függőségek száma} \leq 0$
   - Teljesítményre: $f_2 = P_{95}(\text{válaszidő}) \leq 250$ ms
3. **Implementáció vagy szimuláció:** az első fitness functiont valósítsd meg a választott technológia statikus elemzőjével (ArchUnit, Dependency Cruiser, stb.) vagy szimuláld egy pszeudokódban. A második fitness functiont terheléses teszttel (például k6, JMeter, Locust) vagy modellezett méréssel hajthatod végre izolált környezetben. Ha nem futtatod a mérést, akkor egyértelműen jelöld, hogy a mért érték nem ellenőrzött, csak modellezett példa.
4. **Mért és várt eredmény:** dokumentáld külön oszlopban vagy táblázatban:
   - Várt eredmény: a fitness function sikeres, ha a feltétel teljesül.
   - Mért eredmény: a ténylegesen kapott érték (például a tiltott függőségek száma 1, mert a `payment` modul importál egy `inventory` osztályt; a p95 válaszidő 290 ms). Ha a mérés nem történt meg, írd be: „nem mért, modellezett példa”.
5. **Döntési napló (ADR):** rögzítsd a két fitness function bevezetését ADR formában. Az ADR tartalmazza a kontextust, a döntést, az alternatívákat (például a modularitást kód-review-val, a teljesítményt csak eseti teszttel ellenőrizni), a következményeket és az állapotot (például „elfogadva”, „kiértékelés alatt”).
6. **Ellenőrzési szempontok:** listázd azokat a kritériumokat, amelyek alapján a fitness function jóságát értékelni tudod. Például: fut-e automatikusan, a küszöb konkrét-e, a metrika reprodukálható-e, a hibás eredmény blokkol-e, a bizonytalanság dokumentált-e.
7. **Visszaállítási út:** mivel a labor izolált környezetben zajlik, a visszaállítás a környezet törlését és újra létrehozását jelenti. Ha a terheléses teszt megváltoztatta az adatbázist, azt a teszt előtti állapotba kell visszaállítani, például snapshotból vagy újratesztelési adatokkal.

27.1.14 Elvárt eredmény

A labor végén egy reprodukálható dokumentumot vagy jegyzetfüzetet kapsz, amely tartalmazza:

- a rendszer és a két kulcsfontosságú tulajdonság leírását;
- a fitness functionok definícióját, küszöbbel és következménnyel;
- a mért és várt eredményeket elkülönítve;
- a döntési naplót;
- az ellenőrzési szempontokat;
- a visszaállítási utat.

A labor nem a kód szépségét méri, hanem a gondolkodás és a dokumentáció minőségét. A fitness function tervezése akkor jó, ha a jövőbeli olvasó a dokumentum alapján meg tudja ismételni a mérést, és megérti, miért fontos az adott tulajdonságot védeni.

27.1.15 Ellenőrzőlista a fejezethez

- ☐ A kulcsfontosságú tulajdonság mérhető és az üzleti vagy műszaki célhoz kötött.
- ☐ A küszöb konkrét szám, időablakkal és indoklással.
- ☐ A fitness function automatikusan fut a CI/CD vagy monitoring folyamatban.
- ☐ A döntési következmény arányos a kockázattal (blokkolás, figyelmeztetés, riasztás).
- ☐ A mérési bizonytalanság kezelve van (több minta, percentilis, konfidencia-intervallum).
- ☐ A fitness function bevezetését ADR dokumentálja.
- ☐ A téves riasztások aránya elfogadható, és a csapat bízik a mechanizmusban.
- ☐ Rendszeresen felülvizsgáljuk a küszöböt és a fitness function relevanciáját.

27.1.16 Önellenőrző kérdés

Válassz ki egy saját rendszeredből egy architekturális tulajdonságot, amelyet eddig nem mértél automatikusan. Fogalmazd meg a hozzá tartozó fitness functiont, adj meg egy konkrét küszöböt, és indokold meg, miért éppen ezt a tulajdonságot választottad. Milyen döntési következményt rendelnél hozzá, és miért?

Ha a kérdésre nem tudsz egyértelmű választ adni, az azt jelzi, hogy a kulcsfontosságú tulajdonság még nincs eléggé tisztázva, vagy a mérési módszer nem áll rendelkezésre. Ebben az esetben először a tulajdonság formalizálására érdemes időt szánni, mielőtt fitness functiont építenél.

27.1.17 Összegzés és továbblépés

Az evolúciós architektúra nem egyszeri tevékenység, hanem a rendszer életciklusának természetes része. A fitness functionok a kulcsfontosságú architekturális tulajdonságokat automatikus visszacsatolássá alakítják, így a rendszer fejlődése közben is megőrizhető a szerkezeti integritás, a teljesítmény és a biztonság. A mérés és a döntés összekapcsolása biztosítja, hogy a visszacsatolás ne csak információ, hanem a szállítási folyamat érdemi kapuja legyen.

A következő fejezetben (Technológiai roadmap és iterációs tervezés) erre az alapra építünk: megvizsgáljuk, hogyan lehet a fitness functionokból technológiai roadmapet és iterációs tervezést felépíteni, és hogyan illeszkedik mindez az architektúra governance folyamatába. A most bevezetett fogalmak – kulcsfontosságú tulajdonság, küszöb, döntési kapu – nélkülözhetetlenek lesznek ahhoz, hogy a roadmap ne csak egy kívánságlista legyen, hanem mérhető, folyamatosan ellenőrzött fejlődési pálya.

27.2 Technológiai roadmap és iterációs tervezés

A technológiai roadmap és az iterációs tervezés központi kérdése, hogyan lehet egy rendszer architektúráját hosszú távon, mégis kis lépésekben fejleszteni anélkül, hogy a terv hamis bizonyosságot sugallna, vagy a csapat elveszítené az irányt. A fejezet az Evolúciós architektúra és fitness functions alfejezetben bevezetett evolúciós architektúra és fitness function fogalmakra épít, de nem ismétli meg azok alapjait; itt arra összpontosítunk, hogyan válik a roadmap a folyamatos tanulás eszközévé.

27.2.1 1. A hosszú távú irány és a rövid ciklusok feszültsége

A hagyományos, hosszú távú technológiai tervezés gyakran merev Gantt-diagramot vagy többéves, részletes ütemtervet jelent. Az ilyen tervek a készítés pillanatában már elavultak lehetnek, mert a feltételezések, a piaci körülmények és a technológiai lehetőségek változnak. A túlzottan részletes terv ráadásul hamis bizonyosságot ad: azt sugallja, hogy a jövő ismert, ezért a csapat a terv betartását értékeli a tanulás helyett. A másik véglet a tervtelenség: amikor a csapat kizárólag rövid ciklusokban, sprintről sprintre dolgozik, akkor egy-egy iteráció hatékony lehet, de a rendszer egésze könnyen csapongóvá válik, a technikai adósság felhalmozódik, és a stratégiai célok – például a rendelkezésre állás, a biztonsági keményítés vagy a skálázhatóság – nem kapnak kellő figyelmet.

A megoldás a két szemlélet összehangolása. A hosszú távú irányt egy technológiai roadmap képviseli, amely nem konkrét feladatokat, hanem témákat, mérföldköveket és bizonytalanságokat rögzít. A rövid ciklusok – jellemzően egy-két hetes iterációk – ezekből a témákból vezetik le a konkrét, végrehajtható lépéseket. A roadmap iránytű, a rövid ciklusok pedig lépések: az iránytű megmondja, merre tartunk, a lépések pedig felfedik a terepet. Ahogy a terep megismerése új információt ad, az iránytűt is korrigáljuk.

Ezt a felfogást jól támogatja a „most–következő–később” (now–next–later) felosztás. A **most** horizont a következő néhány ciklusra vonatkozik, és viszonylag részletes: itt már konkrét kísérletek, mérendő állapotok és assignable feladatok vannak. A **következő** horizont nagyjából egy-két negyedévre tekint előre, és témák szintjén marad. A **később** horizont pedig csak irányokat, nyitott kérdéseket és lehetséges alternatívákat tartalmaz. Ez a felosztás csökkenti a hamis precizitást, miközben megtartja a stratégiai fókuszt.

A rövid ciklusok szerepe ebben a modellben az empirikus tanulás. A sprint vagy iteráció során a csapat kipróbál egy megoldást, megméri a hatását, és az eredmény alapján frissíti a roadmapet. A roadmap tehát nem egy elkészítendő dokumentum, amelyet egyszer írunk meg, majd a fiókban őrzünk, hanem egy élő eszköz, amely folyamatosan változik. A változás nem a terv gyengeségét jelzi, hanem a tanulás sikerét: a rendszer és a környezet jobb megismerése lehetővé teszi a pontosabb irányt.

27.2.2 2. A technológiai roadmap alapfogalmai

A technológiai roadmap egy időben rendezett, de nem determinisztikus terv, amely a technológiai irányt, a célokat és a tanulási pontokat rögzíti. Nem projektütemterv, nem release terv és nem termék backlog. A megkülönböztetés fontos, mert ha a roadmapet feladatokkal töltjük meg, akkor elveszíti tanulási jellegét.

- **Projektütemterv** konkrét feladatokat, határidőket, erőforrásokat és függőségeket tartalmaz. Az a kérdése: „mit csináljunk, ki, mikor?”.
- **Release terv** a szállítási csomagokat és azok időzítését írja le. Az a kérdése: „mit és mikor adjunk ki?”.
- **Termék backlog** az elvégzendő feladatok priorizált listája, amelyet a csapat az iterációk során dolgoz fel. Az a kérdése: „min dolgozzunk a következő ciklusban?”.
- **Technológiai roadmap** ezzel szemben a rendszer architekturális irányát, a minőségi célokat és a bizonytalanságokat mutatja. Az a kérdése: „hogyan fejlődjön a rendszer, milyen irányba, és milyen feltételezéseket kell ellenőriznünk?”.

A roadmap alapelemei a **téma**, a **mérföldkő** és a **bizonytalanság**. A téma egy átfogó cél, amely több iteráción átível; például „a megrendelési szolgáltatás p95 válaszidejének csökkentése”, „az adatreplikáció átszervezése”, vagy „a biztonsági naplózás egységesítése”. A téma nem feladat,

hanem irány: azt mondja meg, mit akarunk elérni, nem azt, hogyan. A mérföldkő egy olyan pont, ahol mérhető, ellenőrizhető állapotot érünk el. A mérföldkő lehet időhöz kötött (például „2025 harmadik negyedév végéig") vagy feltételhez kötött (például „amikor a 30 napos hibaarány 0,1% alá csökken"). A bizonytalanság pedig azoknak a kérdéseknek a listája, amelyekre még nem tudjuk a választ, és amelyek befolyásolják a további lépéseket.

A roadmapet horizontokra bontjuk. A közeli horizont – jellemzően a következő egy-három iteráció – részletesebb; itt már konkrét kísérletek, spike-ok vagy PoC-k szerepelnek. A középső horizont témák és mérföldkövek szintjén marad. A távoli horizont csak irányokat és nyitott kérdéseket tartalmaz. Ahogy az idő halad, a távoli elemek közelebb kerülnek, és fokozatosan részletesebbé válnak. Ez a „gördülő" tervezés lehetővé teszi, hogy a roadmap mindig a rendelkezésre álló legfrissebb tudást tükrözze.

27.2.3 3. Téma, mérföldkő és bizonytalanság a gyakorlatban

A téma megfogalmazása akkor jó, ha tartalmazza a jelenlegi állapotot, a célt és a mértékegységet. Például: „A megrendelési szolgáltatás p95 válaszidejének csökkentése 600 ms-ról 300 ms alá, 1000 egyidejű kérés mellett." Ez a téma nem mondja meg, hogyan oldjuk meg, de egyértelmű célt ad. Hasonló téma lehet: „A fizetési tranzakciók tartósságának javítása: a tranzakciók 100%-a legyen visszaállítható legalább 30 napig, a jelenlegi 95% helyett." A témák a minőségi követelményekből (például a quality attribute scenarios alapján) származnak, így a roadmap szorosan kapcsolódik a rendszer minőségi céljaihoz.

A mérföldkőhöz mindig kapcsolódik egy mérési mód, lehetőleg fitness function formájában. Az Evolúciós architektúra és fitness functions alfejezetben bevezetett fitness function egy olyan automatikus ellenőrzés, amely jelzi, ha az architektúra eltér a kívánt iránytól. A roadmap mérföldköveihez rendelt fitness function-ök lehetővé teszik, hogy a haladást objektíven, folyamatosan értékeljük. Például a fenti válaszidő-témához tartozhat egy terheléses teszt, amely minden éjszaka lefut, és ellenőrzi, hogy a p95 válaszidő a megadott terhelés mellett a cél alatt marad. Ha a teszt zöld, a mérföldkő elértnek tekinthető; ha nem, a csapat látja, hogy még dolgoznia kell.

A mérföldkő megfogalmazásánál érdemes a SMART-hoz hasonló kritériumokat alkalmazni: legyen specifikus, mérhető, elérhető, releváns és időhöz kötött. Például: „2025. június 30-ig a megrendelési szolgáltatás p95 válaszideje 1000 egyidejű kérés mellett legfeljebb 300 ms, a havi rendelkezésre állás pedig legalább 99{,}9%." Ez a mérföldkő egyértelmű, mérhető, és a hozzá tartozó fitness function-kel ellenőrizhető.

A bizonytalanság a roadmap talán legfontosabb, mégis leggyakrabban elhanyagolt eleme. Minden technológiai terv tartalmaz olyan feltételezéseket, amelyek igazsága nem ismert. A bizonytalanság lehet **technikai** (például „nem tudjuk, hogy a jelenlegi adatbázis elbírja-e a duplájára növelt terhelést"), **piaci** („nem tudjuk, hogy a felhasználók elfogadják-e az új API-t"), **szervezeti** („a csapatnak nincs tapasztalata a Kubernetes operátorokkal"), vagy **szabályozói** („nem tudjuk, hogy az új adatvédelmi irányelv milyen követelményeket támaszt"). A bizonytalanságokat explicit listaként kell szerepeltetni a roadmapben, és mindegyikhez tanulási tervet kell rendelni: egy kísérletet, egy spike-ot, egy PoC-t vagy egy felmérést, amely csökkenti az adott bizonytalanságot.

Például: ha a bizonytalanság az, hogy a jelenlegi adatbázis elbírja-e a terhelést, a tanulási terv lehet egy terheléses teszt a staging környezetben, ahol a forgalmat fokozatosan növelik, és mérik a válaszidőt, a CPU- és memóriahasználatot. A mérés eredménye alapján a roadmap frissül: ha az adatbázis elbírja, a témát megerősítjük; ha nem, új témát kell bevezetni az adatbázis skálázására vagy cseréjére.

A bizonytalanságok kezelése miatt a roadmap inkább tanulási térkép, mint terv. A „térkép” szó arra utal, hogy a részleteket nem ismerjük, de az irányt és a felfedezendő területeket igen. Ahogy a rövid ciklusok során adatok gyűlnek, a térkép egyre pontosabb lesz, és a bizonytalanságok fokozatosan döntésekké alakulnak.

27.2.4 4. A roadmap mint tanulási eszköz

A roadmap akkor válik tanulási eszközzé, ha nem csak a célokat rögzíti, hanem a feltételezéseket, a kísérleteket és a mérési pontokat is. A tanulási ciklus a következőképpen működik:

1. **Feltevés**: megfogalmazzuk, mit gondolunk, mi fog történni, ha egy adott irányban elindulunk. Például: „ha a szolgáltatások közé circuit breakert vezetünk be, a kaszkádhibák száma csökkenni fog.”
2. **Kísérlet**: a feltevést egy rövid ciklusban, lehetőleg kis kockázatú lépéssel ellenőrizzük. A kísérlet lehet egy PoC, egy spike, egy canary release vagy egy szimuláció.
3. **Mérés**: a kísérlet során rögzítjük a releváns mérőszámokat, például a hibaarányt, a válaszidőt, a rendelkezésre állást. Minden méréshez tartozik környezet, konfiguráció, időpont és bizonytalanság.
4. **Döntés**: a mérési eredmény alapján döntünk a továbblépésről. A döntést ADR-ben vagy döntési naplóban rögzítjük.
5. **Roadmap-frissítés**: a döntés fényében frissítjük a roadmapet: a témákat, a mérföldköveket, a bizonytalanságokat. A frissítéshez verziót és hivatkozást kapcsolunk.

Ez a ciklus hasonlít a tudományos módszerhez: a roadmap a hipotézisek halmaza, a rövid ciklusok pedig a kísérletek. A roadmap tehát nem egy ígéret a jövőre, hanem egy strukturált módja annak, hogy megtudjuk, mi a következő ésszerű lépés.

A roadmap verziózása elengedhetetlen. Minden jelentősebb frissítésnél növeljük a verziót, és a változást rögzítjük a döntési naplóban. A naplóbejegyzés tartalmazza a dátumot, a változás okát, a mérési adatokat és az érintett témákat. Így a roadmap maga is auditálható: visszakövethető, miért döntöttünk egy irány mellett, és milyen alternatívákat vetettünk el. Az Architecture Decision Records (ADR) és döntési napló alfejezetben bemutatott ADR-formátum itt kiválóan használható: context, decision, alternatives, consequences, status.

A roadmap és a fitness function-ök kapcsolata szoros. Minden mérföldkőhöz rendeljünk legalább egy fitness functiont, amely folyamatosan – vagy rendszeres időközönként – ellenőrzi, hogy az architektúra teljesíti-e a kívánt tulajdonságot. A fitness function eredménye a roadmap frissítésének elsődleges inputja. Ha egy fitness function hosszú időn át piros, az azt jelzi, hogy a téma nem halad, vagy a cél nem elérhető; ekkor a roadmap felülvizsgálatra szorul.

A visszacsatolás gyakorisága is fontos. A roadmapet jellemzően negyedévente vagy jelentős belső változás után vizsgáljuk felül, de a rövid ciklusokban a fitness function-ök és a mérések folyamatosan adnak jelzést. Ha egy mérföldkő elérése késik, az nem feltétlenül kudarc; lehet, hogy a téma bonyolultabb, mint gondoltuk, vagy a bizonytalanság nagyobb volt a vártnál. Ilyenkor a tanulási eszköz jellegéből adódóan a roadmapet frissítjük: pontosítjuk a mérföldkövet, új bizonytalanságot veszünk fel, vagy a témát más témára cseréljük. A lényeg, hogy a döntés a mérési adatokon alapuljon, ne pedig a tervhez való ragaszkodáson.

27.2.5 5. Gyakorlati minta: fizetési szolgáltatás modernizációja

A következő példa egy fiktív belső fizetési szolgáltatás roadmapjét mutatja be, helyőrzőkkel. A rendszer egy monolitikus alkalmazás, amely tranzakciókat dolgoz fel; a rendelkezésre állása az

utóbbi hónapokban 99{,}5% körül mozgott, a p95 válaszidő pedig 800 ms. A cél: 99{,}9% havi rendelkezésre állás, 300 ms alatti p95 válaszidő, és a rendszer karbantarthatóságának javítása.

A roadmap kezdeti témái:

Téma	Mérföldkő	Mérési mód / fitness function	Bizonytalanság
Megfigyelhetőség bevezetése	A p95 válaszidő és a hibaarány mérhető, riasztás be van állítva	OpenTelemetry metrikák dashboardon; fitness function: a metrika exportálás fut, a szintetikus teszt elérhető	A jelenlegi naplók nem strukturáltak; a csapatnak nincs OpenTelemetry-tapasztalata
Rendelkezésre állás javítása	30 napos rendelkezésre állás 99{,}9%	Havi rendelkezésre állás mérése; fitness function: a havonta számított $A = \frac{\text{üzemidő}}{\text{üzemidő+állásidő}}$ 0{,}999	A hibák forrása nem ismert; a jelenlegi hibajegyek hiányosak
Szolgáltatás-kivonatolás	A fizetési tranzakciós szolgáltatás a tesztkörnyezetben a régi rendszerrel párhuzamosan működik, a fitness function zöld	Kontraktusteszt és párhuzamos futtatás; fitness function: a párhuzamos üzemmódban a régi és az új rendszer válaszai egyeznek, a hibaarány nem nő	A monolit forráskód minősége ismeretlen; a tranzakciós határok nem tiszták

A horizontok felosztása:

- **Most (következő 2-3 iteráció):** a megfigyelhetőség bevezetése. A csapat OpenTelemetry instrumentációt ad a kritikus útvonalakhoz, metrika-exportálást állít be, és egy szintetikus tesztet készít. A bizonytalanságot (strukturálatlan naplók) egy spike csökkenti: a csapat kipróbálja a strukturált naplózást egy szolgáltatásban, és felméri a szükséges effortot.
- **Következő (kb. 1-2 negyedév):** a rendelkezésre állás javítása. A megfigyelhetőségből származó adatok alapján a csapat azonosítja a hibák fő okait, és bevezeti a circuit breaker, retry és bulkhead mintákat. A mérföldkőhöz tartozó fitness function a havi rendelkezésre állás mérése.
- **Később (kb. 6-12 hónap):** a szolgáltatás-kivonatolás. A csapat a domain elemzés után kiválasztja a fizetési tranzakciós modult, és fokozatosan kiszervezi egy külön szolgáltatásba. A mérföldkő feltételes: csak akkor lépnek ebbe a fázisba, ha a rendelkezésre állás már elérte a célt.

A bizonytalanságok kezelése:

- **Strukturálatlan naplók:** a csapat egy spike során a fizetési szolgáltatás egy alrendszerében bevezeti a strukturált naplózást, és megméri az effortot. Ha a munka kisebb, mint 5 nap, a megfigyelhetőség témát a most horizonton tartják; ha nagyobb, új témaként felveszik a „strukturált naplózás bevezetése” elemet.
- **Hibák forrása:** a megfigyelhetőség bevezetése után a csapat két hétig gyűjti a hibaadatokat, és azokat elemzi. Az elemzés alapján pontosítják a rendelkezésre állás témát.

- **Monolit forráskód minősége:** a szolgáltatás-kivonatolás előtt a csapat egy PoC-t készít: kiválasztanak egy kis modult, és megpróbálják kiszervezni. A PoC eredménye alapján becsülik meg a teljes kivonatolás költségét és kockázatát.

A roadmap kezdetben tehát nem tartalmaz konkrét technológiai döntést (például „Kubernetesre költözünk"). A döntéseket a bizonytalanságok csökkentése után hozzák meg, ADR-ekben rögzítve. A sprint backlog a most horizont témáiból származik: a következő iterációkban a csapat az OpenTelemetry instrumentációra és a strukturált naplózás spike-ra fókuszál.

27.2.6 6. Tipikus hibák és megoldások

A technológiai roadmap készítése során számos hiba fordul elő. Az alábbiak a leggyakoribbak, a megoldásokkal együtt.

1. hiba: A roadmapbe konkrét technológiai döntéseket írunk be idő előtt. Például: „Q3-ban Kubernetesre költözünk", anélkül, hogy ismernénk a jelenlegi rendszer skálázási igényeit. Ezzel a döntéssel a bizonytalanságot elrejtjük, és a csapat a technológia bevezetésére fókuszál a probléma megoldása helyett. **Megoldás:** a döntést késleltessük, amíg a bizonytalanságot csökkentjük. A roadmap csak a célt és a témát rögzíti, a konkrét megoldást a rövid ciklusban választjuk ki, ADR-rel alátámasztva.

2. hiba: A mérföldkövek nem mérhetőek. Ha egy mérföldkő úgy szól, hogy „a rendszer gyorsabb lesz", akkor nem lehet ellenőrizni az elérését. **Megoldás:** minden mérföldkőhöz rendeljünk mérőszámot és fitness functiont. A mérőszámot mértékegységgel és mérési ablakkal adjuk meg.

3. hiba: A bizonytalanságokat elrejtjük vagy figyelmen kívül hagyjuk. A roadmap ilyenkor hamis bizonyosságot sugall, és a csapat nem készül fel a meglepetésekre. **Megoldás:** a bizonytalanságok listája a roadmap kötelező része. Minden iterációban frissítsük, és minden bizonytalansághoz rendeljünk tanulási tervet.

4. hiba: A roadmap és a rövid ciklusok elszakadnak egymástól. A sprint backlog nem a roadmapből táplálkozik, így a csapat a napi működésben elveszíti a stratégiai fókuszt. **Megoldás:** a sprint tervezésénél a roadmap most horizontjának témáit bontsuk le feladatokká. A sprint review során értékeljük a roadmaphez való hozzájárulást, és frissítsük a roadmapet.

5. hiba: A roadmapet Gantt-szerű részletességgel tervezzük. A távoli jövőre vonatkozóan konkrét feladatokat és határidőket adunk meg, ami felesleges merevséget okoz. **Megoldás:** a 3 hónapon túli részt csak témák szintjén tartsuk. A távoli elemek akkor válnak részletessé, amikor a most horizontba kerülnek.

6. hiba: A roadmap nem tükrözi a szervezeti és költségvetési realitást. A terv figyelmen kívül hagyja a csapat kapacitását, a rendelkezésre álló költségkeretet vagy a szervezeti függőségeket. **Megoldás:** a roadmapnek tartalmaznia kell a kapacitást, a csapatok számát, a külső függőségeket és a becsült költséghatást. Ez nem jelenti azt, hogy a roadmap költségvetéssé válik, de a realitáshoz horgonyzott.

27.2.7 7. Labor: technológiai roadmap készítése

A labor célja, hogy a fejezet eszközeit egy kis, ellenőrizhető példán alkalmazd. Az elkészített dokumentum legyen reprodukálható, és különítse el a mért és a várt eredményt.

27.2.8 Kiinduló állapot

Válassz egy fiktív vagy belső rendszert, például egy e-kereskedelmi katalógus szolgáltatást. Rögzítsd a kontextust: a rendszerhatárt, a stakeholdereket, a jelenlegi állapotot és a rendelkezésre álló méréseket (ha vannak). Ha nincsenek valós mérések, használj szintetikus vagy feltételezett adatokat, de ezeket jelöld meg egyértelműen. A labor nem éles rendszeren futtatandó; minden példa helyőrző.

27.2.9 Feladat

Készíts egy Markdown vagy Jupyter notebook dokumentumot az alábbi tartalommal:

1. **Kontextus és rendszerhatár:** röviden írd le a rendszert, a stakeholdereket és a jelenlegi állapotot. Jelöld, hogy mi a rendszerhatár, és mi van azon kívül.
2. **Témák (maximum három):** mindegyik témához adj meg egy mérföldkövet és egy mérési módot. A mérföldkő legyen SMART.
3. **Bizonytalanságok listája:** mindegyikhez rendelj tanulási tervet (kísérlet, spike, PoC). Jelöld, hogyan fogod mérni a bizonytalanság csökkenését.
4. **Horizontok:** rendezd a témákat most–következő–később struktúrába. A most horizontban adj meg konkrét, rövid ciklusban elvégezhető lépéseket.
5. **Döntési napló:** rögzítsd a roadmap készítése során hozott döntéseket ADR-formátumban (kontextus, döntés, alternatívák, következmények, állapot). Legalább két ADR-t készíts.
6. **Fitness function-ök:** minden mérföldkőhöz rendelj legalább egy fitness functiont. Írd le, hogyan ellenőrzi az adott minőségi tulajdonságot.
7. **Mért és várt eredmény elkülönítése:** a dokumentumban világosan jelöld, mi az, amit mérésből tudsz, és mi az, amit feltételezel. A mért eredményekhez add meg a környezetet, konfigurációt, időpontot és bizonytalanságot.

27.2.10 Ellenőrzési szempontok

- Minden témához tartozik-e mérföldkő és mérési mód?
- Minden mérföldkő mérhető-e, van-e hozzá fitness function?
- A bizonytalanságok explicit módon szerepelnek-e, és mindegyikhez van-e tanulási terv?
- A roadmap frissítési mechanizmusa le van-e írva?
- A mért és várt eredmény el van-e különítve?
- A döntési napló bejegyzései tartalmazzák-e a kontextust, a döntést, az alternatívákat és a következményeket?

27.2.11 Visszaállítási út

Ha a labor során bármilyen eszközt telepítettél (például metrika-gyűjtőt vagy tesztkörnyezetet), a labor végén izolált környezetben távolítsd el a tesztadatokat és az ideiglenes erőforrásokat. A roadmap dokumentumot a helyi gépre mentsd; ne tölts fel éles rendszerbe. Ha a laborban használtál valós adatot, azt álnevesítsd vagy szintetikus mintára cseréld.

27.2.12 8. Ellenőrzőlista és önellenőrző kérdés

A technológiai roadmap minőségének ellenőrzésére használd az alábbi listát:

- A roadmap tartalmazza-e a rendszerhatárt és a stakeholdereket?
- A témák a minőségi követelményekből származnak-e?
- A mérföldkövek specifikusak, mérhetőek, elérhetőek, relevánsak és időhöz kötöttek?
- Minden mérföldkőhöz tartozik-e fitness function?
- A bizonytalanságokat listáztuk-e, és mindegyikhez van-e tanulási terv?
- A roadmap frissítési ciklusa meghatározott-e?
- A roadmap verziózott-e, és a frissítések döntési naplóval vannak-e alátámasztva?
- A mért és várt eredményeket elkülönítettük-e?
- A roadmap és a rövid ciklusok (sprint backlog) kapcsolata le van-e írva?

Önellenőrző kérdés: Mi a különbség a projektütemterv és a technológiai roadmap között, és miért nevezhető az utóbbi tanulási eszköznek?

A projektütemterv konkrét feladatokat, határidőket és erőforrásokat rögzít, feltételezve, hogy a jövő ismert. A technológiai roadmap ezzel szemben témákat, mérföldköveket és bizonytalanságokat tartalmaz; célja nem a jövő előrejelzése, hanem a feltételezések strukturált ellenőrzése. Tanulási eszköz, mert minden lépése a mérésen és a visszacsatoláson alapul: a rövid ciklusokban szerzett adatok alapján frissül, így a rendszer fejlesztése fokozatosan, egyre pontosabb tudásra épül.

27.2.13 Összefoglalás és kitekintés

A technológiai roadmap és az iterációs tervezés összeköti a hosszú távú irányt a rövid ciklusú empirikus tanulással. A roadmap témákat, mérföldköveket és bizonytalanságokat rögzít, a rövid ciklusok pedig ezekből vezetnek le konkrét, mérhető lépéseket. A roadmap így nem merev terv, hanem élő dokumentum, amely a fitness function-ök visszajelzései alapján folyamatosan frissül, és döntési naplóval őrzi a tanulási folyamatot.

A következő fejezet (Architektúra governance és szabványok, Architektúra governance és szabványok) erre a felfogásra épít: a roadmap és a fitness function-ök együttesen adják a folyamatos architekturális governance alapját, ahol a szabályok és a mérések biztosítják, hogy a rendszer evolúciója ne térjen el a kívánt iránytól. Az itt bevezetett fogalmak – a témák, a mérföldkövek, a bizonytalanságok és a tanulási ciklus – a governance eszköztárának alappillérei lesznek.

27.3 Architektúra governance és szabványok

Az architektúra governance célja, hogy a szervezet architektúrális döntései ne esetlegesen, hanem tudatosan, visszakereshető módon és a minőségi célokhoz igazodva szülessenek meg. A governance nem bürokrácia, hanem olyan támogató keretrendszer, amely a lehető legkisebb súrlódással biztosítja, hogy a csapatok a megfelelő döntéseket hozzák, és a döntések következményei mérhetőek legyenek. Ez a fejezet a szabványok, a kivételek és a döntési jogkörök rendszerét mutatja be, és gyakorlati módszert ad egy governance- és szabványkatalógus kialakításához.

A governance akkor működik jól, ha a csapatok a szabályokat nem külső kényszerként, hanem saját döntéseik biztonsági korlátjaként élik meg. Ehhez a szabályoknak világosnak, mérhetőnek és arányosnak kell lenniük, a döntési jogköröknek pedig a szervezeti valósághoz kell igazodniuk. A fejezet előfeltételezi az evolúciós architektúra és a fitness functions ismeretét (Evolúciós architektúra és fitness functions), valamint a technológiai roadmap és az iterációs tervezés

fogalmait (Technológiai roadmap és iterációs tervezés). Az itt bemutatott eszközök a következő egységben (Mérés, döntések és visszacsatolás) a mérés és a visszacsatolás témájával kapcsolódnak össze.

27.3.1 1. A governance három pillére: szabványok, kivételek, döntési jogkörök

A governance keretrendszer három, egymást kiegészítő pilléren nyugszik: a szabványokon, a kivételek kezelésén és a döntési jogkörök explicit meghatározásán. A szabványok adják a közös játékszabályokat, a kivételek biztosítják a rugalmasságot, a döntési jogkörök pedig azt, hogy a megfelelő ember a megfelelő szinten dönthessen.

27.3.2 1.1 Szabványok

A szervezeten belüli szabvány nem feltétlenül nemzetközi vagy iparági előírás, hanem olyan belső, írásban rögzített elvárás, amely a gyakorlati döntéseket tereli a kívánt irányba. A szabványok célja mindig az, hogy a rendszer minőségi attribútumait – például a biztonságot, a karbantarthatóságot, az interoperabilitást vagy a költséghatékonyságot – szervezeti szinten érvényesítsék. Egy szabvány tehát nem öncélú, hanem valamely minőségi követelmény eszköze. Ha egy előírás nem köthető ilyen követelményhez, akkor valószínűleg felesleges.

A jól megfogalmazott szabvány jellemzői:

- **Célhoz kötött**: megnevezi, melyik minőségi attribútumot támogatja.
- **Mérhető**: van egyértelmű, lehetőleg automatikusan ellenőrizhető kritériuma.
- **Felelőssel rendelkezik**: megadja, ki tartja karban és ki hagyja jóvá a kivételt.
- **Verziózott**: a szabvány változásai nyomon követhetők, és ADR-hez kapcsolódnak.

Például a „minden szolgáltatásközi HTTP-kommunikáció TLS-en keresztül történjen” szabvány a biztonsági attribútumot támogatja, mérhető a hálózati forgalom megfigyelésével, és egyértelmű felelőse van. Ezzel szemben a „használjunk modern technológiát” nem szabvány, mert nem mérhető és nincs egyértelmű célja.

27.3.3 1.2 Kivételek

Nincs olyan szabvány, amely minden kontextusban érvényes. A kivételek kezelése ezért nem a szabályok gyengítése, hanem a keretrendszer szükséges része. A kivétel olyan tudatos, dokumentált eltérés a szabványtól, amelyet megfelelő jogkörrel rendelkező személy vagy csapat hagyott jóvá, és amelynek van lejárati ideje, valamint felülvizsgálati kötelezettsége.

A kivételkezelés folyamata jellemzően a következő:

1. A csapat azonosítja az eltérést, és megfogalmazza az indoklást: miért nem alkalmazható a szabvány az adott kontextusban.
2. A kivételt benyújtja a megfelelő döntési jogkörrel rendelkező fórumhoz – ez lehet az architektúra-review, a biztonsági csapat vagy a platform csapat.
3. A döntést ADR-ben rögzítik, benne a kontextussal, a döntéssel, az alternatívákkal és a következményekkel.
4. A kivételhez lejárati dátumot rendelnek, amikor a körülmények megváltozása vagy a szabvány felülvizsgálata esetén újraértékelik.
5. A kivételeket egy közös regiszterben tartják nyilván, hogy a szervezet lássa a szabályok alóli eltérések mintázatát.

A kivételek eltitkolása komoly kockázat. Ha a csapatok informalisan kerülik meg a szabályokat, a governance elveszíti a láthatóságát, és a döntések visszakereshetetlenné válnak. A jó governance ezért a kivételeket nem bünteti, hanem tanulási lehetőségként kezeli: ha egy szabvány alól gyakran kérnek kivételt, az azt jelzi, hogy magát a szabványt kell felülvizsgálni.

27.3.4 1.3 Döntési jogkörök

A döntési jogkörök rendszere határozza meg, hogy ki milyen szintű döntést hozhat, és ki viseli a döntés következményeit. Ez a rendszer szoros kapcsolatban áll a szervezeti struktúrával és a Conway-törvénnyel (Conway törvénye és szervezeti illeszkedés). A döntési jogkörök explicit rögzítése azért fontos, mert e nélkül a döntések vagy informálisan születnek, vagy minden döntés a legmagasabb szintre kerül, ami szűk keresztmetszetet teremt.

A döntési jogkörök meghatározásának gyakori eszköze a RACI-mátrix, amely megadja, hogy egy adott döntésben ki a felelős (Responsible), ki a számon kérhető (Accountable), kit kell konzultálni (Consulted), és kit kell tájékoztatni (Informed). Például egy közös kommunikációs protokoll kiválasztásában:

- **Responsible**: a platform csapat, mert ők üzemeltetik a közös infrastruktúrát.
- **Accountable**: a vezető technológiai döntéshozó (például CTO), aki a szervezeti szintű következményeket viseli.
- **Consulted**: a domain csapatok, mert az ő szolgáltatásaik fogják használni a protokollt.
- **Informed**: a termékmenedzsment, mert a döntés befolyásolhatja a szállítási ütemtervet.

A döntési jogkörök delegálása a governance egyik legfontosabb elve. A cél az, hogy a döntések a lehető legalacsonyabb szinten szülessenek, ahol a kontextus és a következmények leginkább ismertek. A platform csapat dönt a közös infrastruktúráról, a domain csapat a saját szolgáltatásának belső architektúrájáról, de vannak olyan szabványok – például a biztonsági vagy megfelelési követelmények –, amelyeknél a szervezeti szintű testületé a végső szó.

27.3.5 2. Kötelező, ajánlott és tiltott minták

A szabványokat célszerű három kategóriába sorolni, mert nem minden előírásnak azonos a súlya. Ez a hármas felosztás segít elkerülni, hogy a governance túlzottan merev legyen, és egyben világossá teszi, hogy mely szabályokból lehet kivétel, és melyekből soha.

27.3.6 2.1 Kötelező minták

A kötelező minták olyan szabványok, amelyek megsértése elfogadhatatlan kockázatot jelentene. Ilyenek jellemzően a biztonsági, adatvédelmi vagy jogszabályi megfelelési követelmények. A kötelező minták száma minimális legyen, mert minden kötelező előírás csökkenti a csapatok autonómiáját.

Példák kötelező mintákra:

- Minden szolgáltatás titkosított csatornán kommunikál.
- Jelszavakat vagy hitelesítő adatokat tilos plain text formában tárolni.
- Minden adatbázis-tranzakció naplózott, és a napló legalább 90 napig megőrzött.
- A production környezetbe csak aláírt, verziózott artefaktum kerülhet.

A kötelező minták esetében a kivétel csak rendkívül indokolt esetben, a legmagasabb döntési szinten, és szigorú lejárati határidővel lehetséges. Sok esetben a kötelező minták nem is engednek kivételt, mert a kockázat nem vállalható.

27.3.7 2.2 Ajánlott minták

Az ajánlott minták olyan preferenciák, amelyek a szervezet tapasztalatai alapján a legtöbb kontextusban jól működnek, de bizonyos esetekben indokolt lehet az eltérés. Az ajánlott minták célja a konzisztencia és a tudásmegosztás elősegítése anélkül, hogy a csapatok mozgásterét indokolatlanul korlátoznák.

Példák ajánlott mintákra:

- Preferált programozási nyelvek: Go és TypeScript a szolgáltatásokhoz.
- Új szolgáltatásoknál konténerizáció és Kubernetes használata.
- REST vagy gRPC használata a belső szolgáltatásközi kommunikációban.
- Az adatbázis-sémaváltozások migrációs eszközzel történjenek.

Az ajánlott mintáknál az eltérés lehetséges, de azt ADR-ben kell rögzíteni, hogy a döntés okai és alternatívái visszakereshetőek legyenek. Az ilyen ADR-ekből a szervezet később értékes mintázatokat olvashat ki.

27.3.8 2.3 Tiltott minták

A tiltott minták olyan gyakorlatok, amelyekről a szervezet tapasztalat vagy szakirodalom alapján tudja, hogy hosszú távon súlyos problémákat okoznak. A tiltás célja nem a büntetés, hanem a hibák megelőzése.

Példák tiltott mintákra:

- Elosztott tranzakciók használata szinkron módon (2PC) nagy skálán.
- Közvetlen adatbázis-elérés más szolgáltatás adatbázisához.
- Titkos kulcsok beágyazása a forráskódba vagy a konténerképbe.
- Külső rendszerek hívása timeout és retry nélkül.

A tiltott minták érvényesítése gyakran automatikusan történik, például statikus kódellenőrzéssel, sérülékenység-szkenneléssel vagy policy as code eszközökkel. A tiltás megszegése esetén a pipeline megáll, és a csapatnak javítania kell a hibát.

27.3.9 2.4 A kategóriák érvényesítése

A három kategória között az a legfontosabb különbség, hogy a kötelező minták automatikusan, hiba esetén a pipeline megszakításával ellenőrzöttek, az ajánlott minták inkább kódfelülvizsgálati irányelvek vagy ADR-kötelezettségek, a tiltott minták pedig általában szintén automatikus, blokkoló ellenőrzések. A kategóriák tehát nemcsak a szabály szigorúságát, hanem az érvényesítés módját is meghatározzák.

A gyakorlatban a szabványkatalógus minden eleméhez tartozik:

- a kategória neve,
- a szabvány megfogalmazása,
- az indoklás (melyik minőségi attribútumot támogatja),

- az érvényesítési mód (pipeline gate, code review, monitoring),
- a kivételkezelésre vonatkozó szabály.

Ez a szerkezet biztosítja, hogy a szabvány ne csak papíron létezzen, hanem a mindennapi munkafolyamatba is beépüljön.

27.3.10 3. A governance mint támogatás, nem bürokrácia

A governance gyakran azért válik népszerűtlenné, mert a szabályokat nem követi automatikus érvényesítés, így a csapatokra hárul az adminisztratív teher. A jó governance ezzel szemben a döntések támogatására összpontosít, és a lehető legnagyobb mértékben automatizálja az ellenőrzéseket.

27.3.11 3.1 Alapelvek

A támogató governance alapelvei:

- **Arányosság**: csak annyi szabály legyen, amennyi valódi kockázatot kezel. Minden új előírást indokolni kell.
- **Átláthatóság**: a szabványok, a kivételek és a döntési jogkörök minden érintett számára elérhetőek.
- **Iterativitás**: a governance maga is evolúciós rendszer. A szabványokat rendszeresen felül kell vizsgálni a visszacsatolások alapján.
- **Eszköztámogatás**: a szabványok érvényesítése lehetőleg automatikus, a pipeline-ba integrált legyen, ne emberi jóváhagyás függjön tőle.

27.3.12 3.2 A governance eszköztára

A modern governance nem bizottsági ülések sorozata, hanem olyan eszközök együttese, amelyek a döntések minőségét és a szabályok betartását folyamatosan biztosítják.

Az **ADR** (Architecture Decision Record) a döntések nyomon követésének alapja. Minden olyan döntés, amely befolyásolja az architektúrát, ADR-ben rögzül. Az ADR tartalmazza a kontextust, a döntést, a mérlegelt alternatívákat, a következményeket és az állapotot. Ez a gyakorlat biztosítja, hogy a döntések okai később is visszakereshetők.

A **fitness functions** (Evolúciós architektúra és fitness functions) olyan automatikus tesztek vagy ellenőrzések, amelyek azt vizsgálják, hogy a rendszer megfelel-e a meghatározott architektúrális követelményeknek. A governance szempontjából a fitness function a szabványkövetés automatikus ellenőrzésének eszköze. Például egy fitness function ellenőrizheti, hogy egy szolgáltatás nem lép túl a meghatározott függőségi határokon, vagy hogy az API-kontraktusok nem sérülnek.

A **policy as code** a szabványok gépi formában történő rögzítését jelenti. Olyan eszközökkel, mint az Open Policy Agent (OPA), Kyverno vagy Sentinel, a szabványok kódként írhatók le, és a pipeline-ban automatikusan kiértékelhetők. Ez csökkenti az emberi ellenőrzés szükségességét, és gyors visszajelzést ad a fejlesztőknek.

A **szabványkatalógus** a governance központi dokumentuma. Ez nem egyszerűen egy lista, hanem olyan, verziózott referencia, amely az összes szabványt, azok kategóriáját, indoklását, érvényesítési módját és kivételkezelését tartalmazza. A katalógust a platform csapat tartja karban, és a belső fejlesztői portálon érhető el. A katalógus minden eleméhez tartozik ADR-hivatkozás, így a döntések nyomon követhetők.

27.3.13 3.3 Példa: szabványkatalógus mint termék

A governance akkor válik támogatóvá, ha a szabványkatalógust a csapatok termékként használják, nem pedig szabálygyűjteményként. Ennek feltétele, hogy a katalógus:

- kereshető és szűrhető,
- kontextusfüggő ajánlásokat ad,
- automatikus ellenőrzésekkel van összekötve,
- kivételkezelési űrlapot tartalmaz,
- verziózott és a változások ADR-hez kötöttek.

Amikor egy fejlesztő új szolgáltatást indít, a katalógus alapján azonnal látja, mely szabványok kötelezőek, melyek ajánlottak, és melyek tiltottak. A pipeline automatikusan ellenőrzi a kötelező és tiltott mintákat, így a fejlesztő a hibákról perceken belül értesül, nem pedig egy késői review-n.

27.3.14 3.4 Hogyan kerüljük el a bürokráciát?

A bürokrácia akkor jelenik meg, ha a governance folyamatai emberi jóváhagyások hosszú láncolatán alapulnak, és a döntések nyoma elvész. Ennek elkerülése érdekében:

- minimalizáljuk a kötelező minták számát,
- a döntési jogköröket delegáljuk a lehető legalacsonyabb szintre,
- a szabványok érvényesítését automatizáljuk,
- a kivételeket gyors, dokumentált folyamattal kezeljük,
- a szabványok felülvizsgálatát rendszeressé tesszük.

Ha egy governance folyamat azt eredményezi, hogy a fejlesztők a szabályok miatt késnek, vagy a döntéseket azért halogatják, mert félnek a jóváhagyástól, akkor a governance rosszul működik. A cél az, hogy a szabályok segítsék a gyors, jó döntéseket, nem pedig hogy lassítsák azokat.

27.3.15 4. Gyakori hibák

A governance bevezetésekor gyakran elkövetett hibák:

1. **A governance célként való kezelése**: a szabályok öncélúvá válnak, és a csapatok a megfelelésre koncentrálnak a valódi minőség helyett. A szabványokat rendszeresen felül kell vizsgálni, hogy valóban a kívánt minőségi attribútumokat támogatják-e.
2. **Minden előírás kötelezővé tétele**: ha nincs különbség a kötelező és az ajánlott minták között, a csapatok elveszítik a kontextusfüggő döntések lehetőségét, és a governance ellenállást vált ki.
3. **Kivételek elrejtése vagy hiánya**: ha a csapatok nem dokumentálják az eltéréseket, a szervezet nem látja a szabványok gyenge pontjait. A kivételek regisztere értékes visszacsatolás a szabványok finomításához.
4. **A döntési jogkörök túlzott centralizálása**: minden döntés egyetlen testülethez kerül, ami szűk keresztmetszetet és késleltetést okoz. A döntési jogköröket a kontextushoz és a következményekhez kell igazítani.
5. **A szabványkövetés mérésének elhanyagolása**: ha nincs mérés, a governance nem tud visszacsatolni. A szabványkövetési arányt – például a megfelelő szolgáltatások arányát – rendszeresen mérni kell, és a trendeket a katalógus felülvizsgálatánál figyelembe kell venni.

27.3.16 5. Labor: governance- és szabványkatalógus tervezése

Ez a labor a fejezet eszközeinek gyakorlati alkalmazását szolgálja. A cél egy governance- és szabványkatalógus tervezése egy fiktív szolgáltatás-ökoszisztémára. A labor reprodukálható dokumentumban vagy jegyzetfüzetben végezhető el, és minden döntést ADR-ben kell rögzíteni. A mért és a várt eredményt külön kell választani: a laborban nem valós mérést végzünk, hanem a mérési tervet készítjük el.

27.3.17 5.1 Kiinduló állapot

Egy fiktív „rendeléskezelő platform” három csapatból áll:

- **Checkout csapat**: a vásárlási folyamatot kezelő szolgáltatás.
- **Inventory csapat**: a készletnyilvántartó szolgáltatás.
- **Notification csapat**: az értesítéseket küldő szolgáltatás.

A szolgáltatások konténerizáltak, és Kubernetes-en futnak. Van egy közös CI/CD pipeline, de a csapatok jelenleg szabadon választanak technológiát, és nincsenek írott szabványok. A szervezet most szeretne bevezetni egy minimális, arányos governance-t.

27.3.18 5.2 Feladat

1. **Azonosítsd az érintetteket és a döntési jogköröket.** Készíts egyszerű RACI-mátrixot legalább három döntési területre (például kommunikációs protokoll, adatbázis-technológia, titokkezelés).
2. **Tervezz legalább öt szabványelemet** a három kategóriába (kötelező, ajánlott, tiltott). Minden elemhez add meg az indoklást, az érvényesítési módot és a kivételkezelést.
3. **Fogalmazz meg legalább két minőségi forgatókönyvet** a szabványkövetés ellenőrzésére. A forgatókönyv tartalmazza az ingert, a környezetet, a választ és a mérőszámot mértékegységgel és mérési ablakkal.
4. **Tervezd meg a kivételkezelés folyamatát**: ki nyújthat be kivételt, ki hagyja jóvá, mi a lejárati idő, és hogyan rögzítik ADR-ben.
5. **Készíts ADR-t a szabványkatalógus bevezetéséről.** Az ADR kövesse a kontextus – döntés – alternatívák – következmények – állapot szerkezetet.
6. **Készíts C4 context diagramot** a rendszerről diagrams-as-code formában (például Mermaid). A diagramon jelöld a rendszerhatárt, a felhasználókat és a külső rendszereket.
7. **Fogalmazd meg az automatikus ellenőrzések módját**: milyen pipeline gate-eket vagy policy as code szabályokat használnál a kötelező és tiltott minták érvényesítésére.

27.3.19 5.3 Elvárt eredmény

Egy verziózott dokumentum vagy jegyzetfüzet, amely tartalmazza:

- a szabványkatalógust táblázatos formában,
- a döntési jogkörök RACI-mátrixát,
- a minőségi forgatókönyveket,
- a kivételkezelés folyamatábráját,
- az ADR-t,
- a C4 context diagramot,
- az automatikus ellenőrzések leírását.

A dokumentumban a mért és a várt eredményt különítsd el: jelöld, hogy a mérési terv szerint milyen mutatókat fogsz mérni, és mi a célérték, de valós mérési adatot nem kell gyűjteni.

27.3.20 5.4 Ellenőrzési szempontok

- Minden szabványelem rendelkezik kategóriával (kötelező / ajánlott / tiltott).
- Minden szabványhoz tartozik indoklás, érvényesítési mód és kivételkezelési szabály.
- A döntési jogkörök egyértelműen rögzítettek, és a RACI-mátrixban minden döntési területnél pontosan egy Accountable szereplő szerepel.
- A kivételkezelés folyamata tartalmazza a benyújtást, a jóváhagyást, a lejárati időt és az ADR-rögzítést.
- Az ADR formátuma megfelel a megadott szerkezetnek.
- A C4 diagram context szinten ábrázolja a felhasználókat, a rendszert és a külső rendszereket, és feltünteti a rendszerhatárt.
- Az automatikus ellenőrzések leírása konkrét eszközöket vagy módszereket nevez meg.

27.3.21 5.5 Példa a szabványkatalógusra

Az alábbi táblázat egy lehetséges részletet mutat be:

Kategória	Szabvány	Indoklás	Érvényesítés	Kivétel
Kötelező	Minden szolgáltatásközi HTTP-kommunikáció TLS-en keresztül történik	adatvédelem, biztonság	pipeline gate: certificate check; monitoring	csak belső, izolált környezetben, biztonsági csapat jóváhagyásával
Kötelező	Jelszavak és titkos kulcsok nem tárolhatók plain text formában	biztonság	secret scanner a pipeline-ban	nincs kivétel
Ajánlott	Új szolgáltatásoknál preferált nyelvek: Go, TypeScript	karbantarthatóság, csapatok közötti átjárhatóság	code review irányelv	eltérő nyelv esetén ADR és képzési terv szükséges
Ajánlott	Belső szolgáltatásközi kommunikációban REST vagy gRPC használata	interoperabilitás, konzisztencia	code review, API-katalógus	eltérés esetén ADR a protokoll indoklásáról
Tiltott	Közvetlen adatbázis-elérés más szolgáltatás adatbázisához	adatintegritás, szolgáltatáshatárok	policy as code: Kubernetes NetworkPolicy, pipeline check	nincs kivétel

27.3.22 5.6 Példa minőségi forgatókönyvre

Inger: Egy új szolgáltatás deploy-ja a production környezetbe.
Környezet: CI/CD pipeline, normál terhelés.
Válasz: A pipeline automatikusan ellenőrzi a kötelező szabványokat (TLS, titokkezelés, hálózati szabályok).
Mérőszám: A sikeres verifikációk aránya, cél 99 %, mérési ablak 30 nap.

27.3.23 5.7 Példa ADR-re

```
ADR-2025-004: Szabványkatalógus bevezetése
Állapot: elfogadva

Kontextus:
A rendeléskezelő platform három csapata jelenleg szabadon választ technológiát,
ami integrációs költségeket és biztonsági kockázatokat okoz. Nincs közös,
írásos szabványrendszer.

Döntés:
Bevezetünk egy verziózott szabványkatalógust, amely kötelező, ajánlott és
tiltott mintákat sorol fel. A katalógust a platform csapat tartja karban,
minden változás ADR-hez kötött.

Alternatívák:
1. Nincs közös katalógus, minden csapat szabadon dönt.
   Elvetve: növekvő integrációs költségek és biztonsági rések.
2. Minden technológiát központilag határoz meg a platform csapat.
   Elvetve: túlzott centralizáció, lassú döntéshozatal.

Következmények:
A csapatok kötelesek a katalógust betartani, de dokumentált kivételt kérhetnek.
A platform csapat fenntartja a katalógust és az automatikus ellenőrzéseket.
A katalógus bevezetésének sikerét a szabványkövetési aránnyal mérjük.
```

27.3.24 5.8 Példa C4 context diagramra (Mermaid)

```
flowchart LR
    U["Vásárló"]
    A["Checkout szolgáltatás"]
    B["Inventory szolgáltatás"]
    C["Notification szolgáltatás"]
    P["Fizetési szolgáltató<br>(külső)"]

    U -->|HTTPS| A
    A -->|HTTPS/REST| P
    A -->|gRPC| B
    A -->|esemény| C
```

Jelmagyarázat: `U` felhasználó, `A`, `B`, `C` belső szolgáltatások, `P` külső rendszer.
Feltételjegyzet: A diagram a context szintet mutatja; a belső komponensek nem látszanak, csak a rendszerhatáron átnyúló kapcsolatok.

27.3.25 5.9 Visszaállítási út

A labor dokumentumai verziózottak. Ha a gyakorlat során hibát vétesz, a dokumentum előző verziójának visszaállítása elegendő (például `git revert` vagy a jegyzetfüzet korábbi állapotának betöltése). A labor nem érint éles rendszert, így további visszaállításra nincs szükség.

27.3.26 6. Ellenőrzőlista

- A governance keretrendszer három pillére – szabványok, kivételek, döntési jogkörök – külön-külön dokumentált?
- A szabványok kategóriákba soroltak (kötelező, ajánlott, tiltott)?
- Minden szabvány rendelkezik indoklással, érvényesítési móddal és kivételkezelési szabállyal?
- A döntési jogkörök RACI-mátrixban rögzítettek, és minden döntési területnek egyértelmű Accountable felelőse van?
- A kivételkezelés folyamata tartalmazza a benyújtást, a jóváhagyást, a lejárati időt és az ADR-rögzítést?
- A szabványkövetés mérésére van terv mértékegységgel és mérési ablakkal?
- Az automatikus ellenőrzések (pipeline gate-ek, policy as code) legalább a kötelező és tiltott mintákra kiterjednek?
- A szabványkatalógus verziózott, és minden változás ADR-hez kötött?
- A governance folyamatok arányosak, átláthatók és iteratívak?

27.3.27 7. Önellenőrző kérdés

Miben különbözik a kötelező és az ajánlott minta a kivételkezelés szempontjából, és miért fontos ez a különbségtétel?

Válaszvázlat: A kötelező mintáknál a kivétel csak rendkívül indokolt esetben, magas jogkörrel, szigorú lejárattal lehetséges, vagy egyáltalán nem engedélyezett. Az ajánlott mintáknál az eltérés dokumentált ADR-rel elfogadható, mert a minta célja a konzisztencia támogatása, nem a kockázat kezelése. A különbségtétel azért fontos, mert ha minden szabály kötelező, a csapatok mozgástere indokolatlanul csökken, és a governance bürokráciává válik. Ha viszont a valódi biztonsági kockázatokat kezelő szabályokból is könnyű kivételt kapni, a rendszer sebezhetővé válik.

27.4 Mérés, döntések és visszacsatolás

Ahogy az architektúra governance (Architektúra governance és szabványok) kialakítja a döntési jogköröket és szabványokat, ugyanúgy szükség van egy olyan visszacsatolási hurokra, amely megmutatja, hogy ezek a döntések és szabványok a gyakorlatban is működnek-e. Az evolúciós architektúra egyik alappillére, hogy az architektúra nem egy statikus tervrajz, hanem egy folyamatosan változó, élő rendszer. Egy élő rendszer állapotát azonban csak akkor tudjuk megítélni, ha mérjük. A mérés nem öncélú adminisztráció, hanem a döntéshozatal nyersanyaga: aki nem mér, az vélemények és feltételezések alapján dönt, aki mér, az legalább a tényekhez tud fordulni, még ha azok értelmezése továbbra is szakmai ítéletet igényel.

Ebben a fejezetben azt vizsgáljuk meg, hogyan mérhető az architektúra eredményessége, milyen mutatók állnak rendelkezésre a gyors visszajelzéshez, és hogyan kerülhetjük el a mérésből fakadó félreértéseket és etikai csapdákat. A fejezet végén egy architektúra-egészség dashboardot tervezünk, amely a labor feladatot is képezi.

27.4.1 1. Az architektúra eredményességének mérése

Az architektúra eredményessége nem azonos a technológiai trendek követésével vagy a legújabb keretrendszerek bevezetésével. Az eredményesség azt jelenti, hogy a rendszer képes kiszolgálni a funkcionális és minőségi követelményeket, miközben a fejlesztési és üzemeltetési költségek hosszú távon fenntarthatók. Ha az architektúra akadályozza a csapatot abban, hogy gyorsan és biztonságosan szállítson, akkor rossz. Ha a csapat gyorsan szállít, de a rendszer instabil a produkcióban, akkor szintén rossz. Az eredményesség tehát egyszerre jelenti a szállítási sebességet, a stabilitást és a minőségi attribútumok teljesülését.

27.4.1.1 1.1. Vezető és követő mutatók

A mérési rendszer tervezésekor meg kell különböztetnünk a vezető (leading) és a követő (lagging) mutatókat. A követő mutatók a múltat írják le: hány éles hiba történt, mennyi volt a rendelkezésre állás, milyen volt a felhasználói elégedettség. Ezek a mutatók rendkívül fontosak a kiértékeléshez, de túl későn jeleznek ahhoz, hogy proaktívan beavatkozhassunk. A vezető mutatók ezzel szemben előre jelzik a jövőbeni problémákat: a ciklikus függőségek számának növekedése, a tesztlefedettség csökkenése vagy a kódbázis bizonyos részeinek túlzott módosítási gyakorisága mind olyan jelek, amelyek arra utalnak, hogy a rendszer hamarosan nehezen karbantarthatóvá vagy instabillá válik.

Az architektúra szempontjából a vezető mutatók különösen értékesek, mert lehetővé teszik a beavatkozást, mielőtt a probléma éles incidensként vagy leállásként materializálódna. A fitness functionök (Evolúciós architektúra és fitness functions) és a statikus elemző eszközök tipikusan vezető mutatókat állítanak elő: megmondják, hogy a kód és a struktúra eltér-e a kívánatostól. A produkciós megfigyelhetőségi adatok (Logs, metrics, traces és OpenTelemetry) ezzel szemben gyakran követő mutatókat szolgáltatnak: megmutatják, hogy mi történt a felhasználókkal és a rendszerrel a valóságban.

A két mutatótípus együtt alkotja a mérési rendszer gerincét. A vezető mutatók segítenek a döntések meghozatalában, a követő mutatók pedig a döntések hatásának kiértékelésében.

27.4.1.2 1.2. A cselekvőképesség kritériuma

Nem minden mutató érdemes a gyűjtésre. Az architektúra-egészség mérésénél a legfontosabb szűrő a cselekvőképesség (actionability). Egy mutató akkor cselekvőképes, ha az értékének változása alapján a csapat egyértelműen meg tudja határozni, milyen konkrét beavatkozást kell tenni.

Tegyük fel, hogy egy “kódbonyolultság” mutatót mérünk. Ha az érték nő, mit teszünk? Refaktorálunk. Ez cselekvőképes. Ha viszont azt mérjük, hogy “hány üzenet érkezik a Slack csatornára”, az lehet, hogy érdekes szociológiai adat, de nem vezet közvetlen architektúrális döntéshez. A cselekvőképesség hiánya az egyik leggyakoribb oka annak, hogy a dashboardok kiüresednek, és a csapat néhány hét után figyelmen kívül hagyja őket.

27.4.1.3 1.3. Az összetett mutatók kockázata

Az architektúra eredményességét gyakran próbálják egyetlen, mindent leíró számmal kifejezni. Ez a törekvés érthető: a vezetőség szeretne egy egyszerű, áttekinthető jelzőszámot. Azonban az összetett mutatók (composite metrics) elfedik az összetevők közötti feszültséget. Ha például egyetlen “rendszer-egészség” mutatót számolunk a rendelkezésre állás, a válaszidő és a hibarát

átlagából, akkor előfordulhat, hogy a rendelkezésre állás kiváló, miközben a válaszidő kritikusan romlik, de az átlag mégis elfogadhatónak tűnik. Az architektúrális döntések éppen az ilyen trade-offokban rejlenek, ezért a mérési rendszernek meg kell őriznie az egyes dimenziók láthatóságát. A dashboardokon az összesített nézet mellett mindig szerepeljenek a részletező panelek is.

27.4.2 2. DORA-metrikák és architektúrális indikátorok

Az elmúlt években a DORA (DevOps Research and Assessment) kutatásai váltak a szoftverszállítási teljesítmény mérésének leggyakrabban hivatkozott keretrendszerévé. A DORA négy kulcsmetrikát határoz meg, amelyek erősen korrelálnak a szervezeti teljesítménnyel:

1. **Szállítási gyakoriság (Deployment Frequency):** milyen gyakran kerül kód a produkcióba.
2. **Átfutási idő a változáshoz (Lead Time for Changes):** mennyi idő telik el a commit és a produkciós élesítés között.
3. **Változási hibaráta (Change Failure Rate):** a produkciós változások hány százaléka okoz hibát, szolgáltatáskiesést vagy azonnali javítást.
4. **Helyreállítási idő (Time to Restore Service, MTTR):** egy incidens esetén mennyi idő alatt áll helyre a szolgáltatás.

A DORA-metrikák első pillantásra elsősorban a DevOps-folyamatot mérik, nem pedig az architektúrát. Ez a látszat azonban megtévesztő. A négy metrika értéke szorosan összefügg a rendszer architektúrájával.

A szállítási gyakoriság és az átfutási idő erősen függ a rendszer modularitásától és a csapatok közötti függőségektől. Egy erősen csatolt monolitban, ahol minden módosítás a teljes rendszer újraépítését és újratelepítését igényli, a szállítási gyakoriság természetes módon alacsony marad. Ha a csapatoknak minden egyes változtatáshoz egyeztetniük kell más csapatokkal, mert a kódjuk közös modulokban kereszteződik, az átfutási idő megnő. A moduláris monolit (Monolit, moduláris monolit és mikrolit) vagy a jól elhatárolt mikroszolgáltatások (A mikroszolgáltatások ígérete és ára) éppen azért növelik a szállítási sebességet, mert csökkentik a koordinációs költséget.

A változási hibaráta a rendszer tesztelhetőségének és a deployment folyamat biztonságának a függvénye. Ha az architektúra nem teszi lehetővé a komponensek izolált tesztelését, ha egy egyszerű változtatás a rendszer távoli részeiben okoz mellékhatásokat, akkor a hibaráta magas lesz. A jól megválasztott szolgáltatáshatárok és a fokozatos deployment stratégiák (Blue-green, canary, rolling és feature flag) közvetlenül csökkentik ezt a mutatót.

A helyreállítási idő az observability (Logs, metrics, traces és OpenTelemetry) és a reziliencia (Circuit breaker, bulkhead, retry és timeout) minőségét tükrözi. Ha a rendszer architektúrája úgy lett kialakítva, hogy a hibák lokalizálhatók, a circuit breakerek megakadályozzák a kaszkád hibákat, és a naplók, metrikák elegendő kontextust adnak a diagnózishoz, akkor az MTTR alacsony lesz. Ha a rendszer egy homogén, átláthatatlan massza, ahol a hiba gyökér-okának megtalálása órákig tart, akkor az MTTR elkerülhetetlenül magas lesz.

A DORA-metrikákat ezért nem szabad kizárólag a fejlesztési gyakorlatok mutatóinak tekinteni. Úgy is értelmezhetjük őket, mint az architektúra minőségének a tüneteit. A rossz architektúra megfojtja a szállítást, a jó architektúra pedig lehetővé teszi azt.

27.4.2.1 2.1. Architektúrális indikátorok gyűjteménye

A DORA-metrikák mellett érdemes olyan közvetlenebb, technikai indikátorokat is mérni, amelyek a kód és a struktúra állapotát írják le. Ezek az indikátorok tipikusan statikus elemző eszközökből, build pipeline-okból és verziókezelő rendszerekből nyerhetők, és vezető mutatóként szolgálnak a DORA-metrikákhoz.

- **Ciklikus függőségek száma:** ha a komponensgráf köröket tartalmaz, az a modularitás csökkenését jelzi. A ciklikus függőségek megnehezítik a tesztelést és a komponensek cseréjét.
- **Szolgáltatáshatárokat átlépő hívások száma:** ha egy bounded context túl gyakran hív egy másikat, az a határok hibás meghúzására utal. Ennek mérése segít azonosítani a túlzottan összenőtt szolgáltatásokat.
- **Kódduplikáció mértéke:** a duplikált logika növeli a karbantartási költséget, mert egy hiba javítását több helyen is el kell végezni.
- **Technikai adósság aránya:** a statikus elemzők által becsült adósság és az összes kód arányának változása.
- **Tesztlefedettség a változó kódban:** nem az összes kód lefedettsége, hanem az újonnan írt vagy módosított kód lefedettsége. Ez pontosabban jelzi, hogy a csapat valóban teszteli-e az új funkciókat.
- **Fitness functionök passzolási aránya:** ha az evolúciós architektúra keretében fitness functionöket definiálunk, azok sikeres vagy sikertelen futásának aránya közvetlenül mutatja, hogy a rendszer megfelel-e a kitűzött architektúrális kritériumoknak.

Ezek az indikátorok akkor hasznosak, ha időbeli trendként vizsgáljuk őket, nem pedig egyszeri pillanatfelvételként. Egyetlen magas ciklikus függőségi szám nem jelent feltétlenül katasztrófát, ha a trend csökkenő. Az architektúra-egészség mindig a folyamatról, nem pedig a pillanatnyi állapotról szól.

27.4.2.2 2.2. A mérési ablak és a bizonytalanság fontossága

A könyv dokumentációs szabályaival összhangban minden mutatóhoz meg kell adni a mérési ablakot és a mértékegységet. A “szállítási gyakoriság heti átlaga az elmúlt 90 napban” egyértelműbb, mint a “gyakori szállítás”. A mérési ablak megválasztása befolyásolja az értelmezést: egy hosszú ablak stabilabb képet ad, de elfedheti a rövid távú romlást, míg egy rövid ablak zajos lehet.

A bizonytalanság dokumentálása különösen fontos a statikus elemzésből származó becsléseknél. A technikai adósság vagy a kódduplikáció mértéke nem egzakt fizikai mérés, hanem egy eszköz által alkalmazott heurisztika eredménye. Ha a dashboardon ilyen mutatókat jelenítünk meg, a feliratnak ezt jeleznie kell, például: “becsült technikai adósság, a `static-analyzer-pelda` eszköz 3.1.0 verziójával számolva, 2024-es konfigurációval”.

27.4.3 3. A mérés etikája és félrevezető mutatók

A mérés soha nem semleges tevékenység. A mutatók megválasztása, a gyűjtés módja és az értelmezés kerete mind értékválasztásokat tükröz. Ha ezt nem ismerjük fel, a mérési rendszer visszaüthet, és rossz döntésekhez vezethet.

27.4.3.1 3.1. Amikor a mérőszám cél lesz

A mérési kultúra egyik legismertebb paradoxona Goodhart törvénye: “Amikor egy mérőszám céllá válik, megszűnik jó mérőszám lenni”. A jelenség oka, hogy a motivált emberek megtalálják a módját, hogy a mutatót javítsák anélkül, hogy a mögöttes valóságot javítanák.

Klasszikus példa erre a tesztlefedettség. Ha a menedzsment egyetlen célt tűz ki: “a tesztlefedettség legyen legalább 85%”, a csapat valószínűleg eléri ezt a célt. De vajon azáltal, hogy értelmes teszteket ír, vagy azáltal, hogy trivialitásokat tesztel, a getter-szetter metódusokat fedi le, vagy éppen a lefedettséget mérő eszközt finomítja addig, amíg a szám el nem éri a kívánt értéket? A mutató ugyan zöld, de a rendszer minősége nem javult.

Az architektúra-egészség mérésénél különösen fontos, hogy a mutatókat ne használjuk a csapatok teljesítményének rangsorolására. Ha a csapat úgy érzi, hogy a ciklikus függőségek száma alapján fogják megítélni, akkor a kódot úgy fogja átalakítani, hogy a mutató javuljon, ami akár kontraproduktív absztrakciókhoz vagy felesleges indirekciókhoz is vezethet. A mutató célja a rendszer állapotának megismerése, nem pedig az egyének megbüntetése.

27.4.3.2 3.2. A félrevezető mutatók felismerése

A vanity metrics (hiúsági mutatók) olyan számok, amelyek jól néznek ki, de nem vezetnek döntéshez. A “regisztrált felhasználók száma” egy e-kereskedelmi rendszernél például hiúsági mutató, ha a valódi cél a visszatérő vásárlók számának növelése. Az architektúra területén tipikus hiúsági mutatók a “szerverek száma”, a “microservicek száma” vagy a “létrehozott ADR-ek száma”. Az ADR-ek esetében a könyv dokumentációs szerződése is hangsúlyozza: az ADR-ek száma nem cél, hanem a döntések nyomon követésének eszköze.

A mérési rendszer tervezésekor minden mutató esetében fel kell tennünk a kérdést: “Ha holnap ez a szám 20%-kal romlik vagy javul, mit csinálunk másként?” Ha erre a kérdésre nem tudunk válaszolni, a mutató valószínűleg hiúsági mutató.

27.4.3.3 3.3. A mérés mint tanulási eszköz

Az etikus mérési kultúra alapelve, hogy a mutatókat a tanuláshoz és a rendszer fejlesztéséhez használjuk, nem pedig a hibáztatáshoz. Ez a megkülönböztetés a pszichológiai biztonság (Kultúra, pszichológiai biztonság és döntéshozatal) egyik alapfeltétele. Ha egy csapat fél attól, hogy a rossz MTTR-mutató miatt felelősségre vonják, akkor hajlamos lesz az incidenst eltitkolni vagy a számot szépíteni. A mérési rendszer így pontosan az értelmét veszti: nem a valóságot mutatja meg, hanem egy megszépített látszatot.

A jó gyakorlat az, ha a mérési adatokat a csapat maga vizsgálja, és a vezetőség számára aggregált, anonimizált formában kerülnek bemutatásra. A fejlesztők napi szinten láthatják a saját szolgáltatásuk DORA-metrikáit, hogy lássák, hol akad el a folyamat. A vezetőség pedig a szervezeti trendeket látja, hogy felismerje a rendszerszintű akadályokat.

27.4.4 4. Architektúra-egészség dashboard tervezése

Az architektúra-egészség dashboard egy olyan vizuális felület, amely összegyűjti a fent bemutatott mutatókat, és lehetővé teszi a csapat számára, hogy időbeli trendként kövesse őket. A dashboard tervezésekor a következő rétegeket érdemes elkülöníteni:

1. **Üzleti hatás rétege:** a rendszer hozzájárulása az üzleti célokhoz, például a konverziós ráta, a rendelésfeldolgozási idő vagy a felhasználói elégedettség. Ez a réteg a termékmenedzserek és a stakeholder-ek számára fontos.
2. **Szállítási teljesítmény rétege:** a DORA-metrikák. Ez a réteg a fejlesztői csapatok és a tech leadek számára mutatja meg, hogy milyen gyorsan és biztonságosan tudnak változtatni.
3. **Architektúrális minőség rétege:** a ciklikus függőségek, a szolgáltatáshatárok megsértése, a tesztlefedettség trendje és a fitness functionök állapota. Ez a réteg a rendszertervezők és architektek számára nyújt korai jelzést a strukturális romlásról.
4. **Üzemeltetési egészség rétege:** a rendelkezésre állás, az SLO-k, a hibaarány, az erőforrás-kihasználtság. Ez a réteg az üzemeltetési és platformcsapatok munkáját támogatja.

A dashboard nem lehet egy végtelen listája a diagramoknak. Minden panelnek tartalmaznia kell egy rövid leírást arról, hogy mit mutat, mi a normál tartomány, és mi a teendő, ha az érték kilép ebből a tartományból. Ez a "runbook a dashboardon" elv biztosítja, hogy a mutatók valóban cselekvőképesek legyenek.

A dashboardok megvalósításához a gyakorlatban a mérési adatokat a CI/CD pipeline-ból (például a sikeres és sikertelen deploymentek naplóiból), a verziókezelőből (a commitok és a deploymentek időbélyegeiből), a statikus elemzőkből és az observability platformokból gyűjtjük. A gyűjtés és a vizualizáció leggyakrabban olyan eszközökkel történik, mint a Prometheus és a Grafana, vagy a felhőszolgáltatók natív monitorozó eszközei.

27.4.5 5. Labor: Architektúra-egészség dashboard tervezése

Ez a labor egy architektúra-egészség dashboard tervének elkészítését tűzi ki célul. A feladat reprodukálható dokumentumban vagy jegyzetfüzetben végezhető el, és a fent bemutatott elvek gyakorlatba ültetését gyakorolja.

Kontextus: A `Pelda E-commerce Kft.` (fiktív vállalat) egy online áruházat üzemeltet. A rendszer három fő komponensből áll: a `RendelesSzolgaltatas` (moduláris monolit), a `KeszletSzolgaltatas` (mikroszolgáltatás) és a `SzamlazoSzolgaltatas` (külső szolgáltató által hosztolt API). A csapatok a következő problémákat észlelik: a rendelési funkciók módosítása egyre lassabb, a készletfrissítések néha hibás adatot mutatnak, és a szállítási gyakoriság csökken. A CIO szeretne egy dashboardot, hogy nyomon kövesse az architektúra egészségét.

Kiinduló állapot: A vállalat rendelkezik egy CI/CD rendszerrel, amely naplózza a deploymenteket és a hibákat. Az observability stack jelenleg csak alapvető infrastruktúra-metrikákat gyűjt. Statikus elemző eszköz nincs bevezetve.

Feladat:

1. **Mutatók kiválasztása:** Tervezd meg a dashboard négy rétegét. Minden réteghez válassz ki legalább 3-3 mutatót! Minden mutatónál add meg a mértékegységet, a mérési ablakot, az adatforrást, a vezető/követő jelleget és a cselekvőképességi feltételt!
2. **Döntési napló:** Készíts egy ADR-t, amely rögzíti a dashboard célját és a mutatók kiválasztásának okát. Az ADR-ben tüntesd fel az elvetett alternatívákat is (például egyetlen összetett mutató használatát)!
3. **Mért és várt eredmények elkülönítése:** A tervben külön jelöld, hogy mely mutatókhoz állnak rendelkezésre jelenleg is mért adatok, és melyek azok, ahol csak becsléssel, feltételezéssel vagy egy jövőbeni bevezetés utáni méréssel tudsz dolgozni! A nem mért értékeket ne tüntesd fel ellenőrzött eredményként!

4. **Félrevezető mutatók szűrője:** Minden kiválasztott mutatóhoz írj egy mondatot, amely megindokolja, miért nem hiúsági mutató, vagy ha az, miért hagytad el!
5. **Visszaállítási út:** Mivel ez egy tervezési feladat, a visszaállítási út annak dokumentálása, hogy a dashboard konfigurációja verziózott, és egy korábbi, jóváhagyott állapot visszaállítható. Írd le, hogyan biztosítanád ezt a gyakorlatban!

Elvárt eredmény: Egy dokumentált terv, amely tartalmazza a mutatók táblázatát, a döntési naplót, a mért/várt elkülönítést és a szűrő indoklását.

Ellenőrzési szempontok:

- A mutatók rendelkeznek mértékegységgel és mérési ablakkal.
- A vezető és követő mutatók egyensúlyban vannak.
- A terv megkülönbözteti a mért és a várt értékeket.
- A dokumentum tartalmazza a cselekvőképességi feltételeket.
- Az ADR-ben szerepelnek az alternatívák és azok elvetésének okai.

27.4.6 6. Gyakori hibák és ellenőrzőlista

A mérési rendszerek bevezetése során a következő hibák fordulnak elő leggyakrabban:

- **A mutatók célokká válnak:** A csapat a számot optimalizálja a valóság helyett.
- **A mérési ablak hiánya:** Az átlagok értelmezhetetlenek időbeli kontextus nélkül.
- **A bizonytalanság elhallgatása:** A becsléseket pontos mérésként tüntetik fel.
- **A dashboard túlterhelése:** Túl sok, egymásnak ellentmondó mutató jelenik meg.
- **A cselekvőképesség hiánya:** A mutatók látszanak, de senki sem tudja, mit kell tenni, ha romlanak.
- **A csapat megbüntetése a mutatók alapján:** A mérési rendszer a hibáztatás eszközévé válik, ami az adatok megszépítéséhez vezet.

Ellenőrzőlista a mérési rendszer bevezetéséhez:

☐ Minden mutatóhoz tartozik mértékegység, mérési ablak és adatforrás.
☐ A mutatók vezető és követő jellege dokumentált.
☐ Minden mutatóhoz tartozik cselekvőképességi feltétel és felelős.
☐ A mért és a várt/becsült értékek egyértelműen elkülönülnek.
☐ A mérési adatokhoz való hozzáférés biztosított, és a feldolgozás nem tartalmaz valós személyes adatot.
☐ A dashboard konfigurációja verziózott, és visszaállítható.
☐ A csapat egyetért a mutatók értelmezésében és használatában.
☐ A mérési rendszer felülvizsgálata időszakosan megtörténik.

Önellenőrző kérdés: A csapatod bevezeti a "változási hibaráta" mutatót, és a menedzsment havi jelentést kér róla. A csapat vezetője attól fél, hogy a hibákért való felelősségre vonás elkerülhetetlen lesz. Hogyan terveznéd meg a jelentést és a folyamatot, hogy a mutató a tanulást szolgálja, ne pedig a büntetést? Milyen vezető mutatókat javasolnál a hibaráta csökkentésére?

A mérés és a visszacsatolás az evolúciós architektúra motorja. A mutatók nélkül az architektúra egy olyan térkép, amely nem mutatja a valós terepet. A jól megtervezett mérési rendszer lehetővé teszi, hogy a döntéseket ne a levegőbe, hanem a megfigyelt valóság talajára építsük. Az itt bemutatott fogalmak, különösen a DORA-metrikák és a vezető/követő mutatók megkülönböztetése a következő egységekben a szervezeti struktúra és a csapatok közötti együttműködés vizsgálatakor is fontos szerepet kapnak.

28 Szervezet, kultúra és vezetés

28.1 Conway törvénye és szervezeti illeszkedés

A legjobban megtervezett architektúra is törékennyé válik, ha a szervezeti struktúra ellene dolgozik. A komponensek közötti határok, az interfészek és a felelősségi viszonyok nem csak technikai döntések: azokat a csapatok közötti kommunikációs minták, vezetői elvárások és mindennapi együttműködési rutinok is alakítják. Ezért az architektúra és a szervezet illeszkedését nem lehet utólag, a kód szintjén megoldani; ugyanúgy tervezni kell, mint a rendszer futásidejű viselkedését.

28.1.1 A szervezeti struktúra és a rendszerarchitektúra kapcsolata

A kapcsolat legismertebb megfogalmazása Conway törvénye, amely szerint a rendszert tervező szervezet olyan kialakítást hoz létre, amelynek szerkezete a szervezet kommunikációs szerkezetét másolja. Ezt gyakran leegyszerűsítve így mondják: „a rendszerarchitektúra a szervezet kommunikációs struktúrájának lenyomata”. A törvény nem fizikai törvény, hanem erős társadalmi tendencia: a fejlesztők, csapatok és vezetők hajlamosak úgy kialakítani a modulhatárokat, hogy a szükséges egyeztetések a lehető legkisebb súrlódással járjanak. Ha két csapat között napi szintű, intenzív kommunikáció zajlik, akkor az általuk fejlesztett komponensek között valószínűleg szoros csatolás alakul ki. Ha viszont két csapat csak ritkán beszél egymással, a rendszerben is erős határvonal, stabil interfész vagy akár különálló szolgáltatás jelenik meg.

A szervezeti struktúrát itt nem az organogrammal kell azonosítani. Sokkal fontosabb a tényleges kommunikációs gráf: kik egyeztetnek egymással, milyen gyakran, milyen csatornákon, és milyen döntéseket hoznak közösen. Ez a gráf tartalmazza a formális csapathatárokat, a vezetői viszonyokat, de tartalmazza az informális kapcsolatokat is: egy backend-fejlesztő és egy adatbázis-üzemeltető közötti napi csevegés éppúgy alakítja az interfészt, mint egy hivatalos szerződés. A rendszerarchitektúra gráfja ezzel párhuzamosan a komponenseket és a köztük lévő függőségeket írja le. Conway törvénye azt állítja, hogy a két gráf szerkezete közelíti egymást, mert a kommunikációs utak mentén alakulnak ki azok a közös mentális modellek, amelyek lehetővé teszik a szoros együttműködést.

Fontos megérteni a mechanizmust. Amikor egy fejlesztőnek egy másik csapat által birtokolt komponenssel kell integrálnia, a szükséges egyeztetés költséggel jár: idő, figyelem, félreértés, várakozás. A fejlesztő ezért gyakran választja a könnyebb utat: inkább közvetlenül hív egy belső függvényt, közös adatbázist használ, vagy megkerüli a hivatalos interfészt, ha a másik csapat elérhető és gyorsan válaszol. Ha viszont a másik csapat távol van, lassan reagál, vagy a szervezeti szabályok akadályozzák a közvetlen egyeztetést, akkor a fejlesztő kénytelen stabil, jól dokumentált interfészt használni, vagy saját komponenst létrehozni. Így a kommunikációs struktúra közvetlenül befolyásolja a csatolást és a modulhatárokat.

A kapcsolat nem csak a kódra igaz. Ugyanez a logika érvényesül az adatmodellben, az üzemeltetési folyamatokban és a tesztelési stratégiában is. Ha például a szervezetben külön adatbázis-csapat működik, akkor az adatmodell-változások valószínűleg központi, formális folyamaton mennek

keresztül, és a rendszerben erős adatréteg-határ alakul ki. Ha nincs ilyen csapat, a fejlesztők közvetlenül módosítják a sémát, és az adatmodell szétterül a komponenseken.

28.1.2 Példák a szervezeti struktúra architekturális következményeire

Vegyünk egy tipikus funkcionális szervezetet, amelyben Frontend, Backend, Adatbázis és Üzemeltetés csapatok dolgoznak. A kommunikációs gráf a rétegek mentén sűrű: a frontend-csapat a backend-csapattal egyeztet API-kat, a backend-csapat az adatbázis-csapattal egyeztet sémákat, az üzemeltetés pedig a deploy és a környezetek miatt mindenkivel kapcsolatban áll. A kialakuló architektúra nagy valószínűséggel rétegezett lesz: webes felület, alkalmazásréteg, adatbázisréteg. Az üzleti folyamatok – például a rendelés leadása és a számlázás – átívelnek ezeken a rétegeken, ezért egy-egy üzleti változás több csapatot is érint. A csapatok közötti interfészek a technológiai rétegek mentén erősödnek meg, nem az üzleti modulok mentén. Ez nem feltétlenül rossz, de jól mutatja, hogy a szervezeti struktúra milyen architektúrát hoz létre.

Ezzel szemben egy termékorientált szervezetben Rendelés, Számlázás és Készlet csapatok működnek, mindegyik a saját üzleti képességéért felel. A kommunikáció a csapatok között akkor sűrű, amikor az üzleti folyamatok összekapcsolódnak: a rendelésnek szüksége van készletinformációra, a számlázásnak rendelésadatokra. A kialakuló architektúra jellemzően moduláris, szolgáltatásalapú vagy mikroszolgáltatás-jellegű lesz, ahol a határok a csapathatárok mentén húzódnak. Az interfészek az üzleti folyamatok integrációs pontjain jelennek meg: rendelés-szolgáltatás, készlet-szolgáltatás, számlázás-szolgáltatás, közöttük explicit API-k vagy események.

A két példa közötti különbség nem elsősorban a technológiában van, hanem a felelősségek és a kommunikációs minták elrendezésében. A funkcionális szervezet is készíthet mikroszolgáltatásokat, de ha a csapatok továbbra is rétegenként szerveződnek, a szolgáltatások határai folyamatosan elmosódnak, mert egy-egy üzleti változás több csapatot is érint, és a csapatok közötti egyeztetések a réteghatárokon zajlanak. A termékorientált szervezetben viszont a csapatok közötti kommunikáció az üzleti interfészekre koncentrálódik, ami segít fenntartani a szolgáltatáshatárokat.

28.1.3 Az inverz Conway-manőver

Ha Conway törvénye azt mondja, hogy a szervezet formálja az architektúrát, akkor a tudatos tervezés egyik legerősebb eszköze ennek a hatásnak a megfordítása. Az inverz Conway-manőver (inverse Conway maneuver) lényege, hogy először a kívánt rendszerarchitektúrát határozzuk meg, majd ehhez igazítjuk a csapatok határait, felelősségeit és kommunikációs mintáit. A cél nem az, hogy a szervezeti ábrát erőltessük a kódra, hanem hogy a kommunikációs gráfot úgy alakítsuk, hogy a kívánt architektúra természetes módon, a mindennapi együttműködés melléktermékeként jöjjön létre.

Az inverz Conway-manőver jellemző lépései:

1. **Célarchitektúra meghatározása.** Azonosítsuk a rendszer kívánt modulhatárait, komponenseit és interfészeit. Ez lehet három szolgáltatás, moduláris monolit, eseményvezérelt feldolgozási lánc vagy bármilyen más, a minőségi követelményekkel összhangban lévő szerkezet.
2. **A szükséges csapathatárok levezetése.** Minden célmodulhoz rendeljünk felelős csapatot. A határokat úgy húzzuk meg, hogy egy csapat kognitív terhelése kezelhető legyen, és a csapat a modulját teljes egészében birtokolja: kód, adat, teszt, build, deploy, üzemeltetés.

3. **A kommunikációs minták kialakítása.** A csapatok közötti együttműködési csatornákat a célarchitektúra interfészeihez igazítjuk. Ha két komponens között aszinkron eseménykapcsolat van, a csapatoknak sem kell napi szintű szinkron egyeztetést folytatniuk; elég egy közös szerződés és időszakos áttekintés. Ha két komponens szorosan együttműködik, a csapatoknak is közös ritmusra van szükségük.
4. **A felelősségi és döntési jogkörök rögzítése.** Minden csapatnak egyértelműen tudnia kell, mely komponensekért felel, milyen interfészeket nyújt, és milyen döntéseket hozhat önállóan. Ez csökkenti a felesleges egyeztetéseket és megakadályozza a felelősség átfedését vagy lyukait.
5. **Folyamatos visszacsatolás.** A szervezet és az architektúra együtt fejlődik. Ahogy a követelmények változnak, a csapathatárokat és a kommunikációs mintákat is felül kell vizsgálni.

Fontos hangsúlyozni: az inverz Conway-manőver nem egyszerű átszervezés. A szervezeti ábra önmagában nem változtatja meg a kommunikációs gráfot, ha a mindennapi rutinok, a vezetői elvárások és az eszközök nem követik az új határokat. Az sem elég, ha csak a kódot refaktoráljuk, a csapatokat viszont a régi struktúrában hagyjuk. A manőver akkor működik, ha a technikai és a szervezeti változást párhuzamosan, közös cél érdekében hajtjuk végre.

28.1.4 Csapathatárok és interfészek

A csapathatár a szervezeti architektúra alapvető építőeleme. Nem adminisztratív vonal, hanem felelősségi és kommunikációs határ, amely meghatározza, hogy egy csapat mit birtokol, milyen szolgáltatást nyújt másoknak, és milyen elvárásokat támaszt a környezetével szemben. A jó csapathatár ott húzódik, ahol a szükséges egyeztetések száma alacsony, a szerződések világosak, és a csapat mentális modellje lefedi a birtokolt komponensek teljes működését.

A csapatok közötti interfésznek technikai és szervezeti rétege is van. A technikai réteg az API, az esemény, a séma, a protokoll, a szinkronitás, a hibakezelés és a verziózás. A szervezeti réteg a munkamód: hogyan jeleznek hibát, hogyan kérnek változtatást, ki dönt a prioritásról, milyen gyakran egyeztetnek, és milyen csatornákon. A két rétegnek összhangban kell lennie. Ha például két csapat közötti technikai interfész aszinkron esemény, de a szervezeti rutin napi szintű szinkron státuszmegbeszélést ír elő, akkor a kommunikációs gráf és az architektúra gráf eltér egymástól, ami felesleges koordinációs terhet és lassú döntéseket okoz.

A csapathatárok tervezésekor érdemes a következőket megadni:

- **Felelősség:** mely komponenseket, adatokat és folyamatokat birtokol a csapat.
- **Interfész:** milyen szolgáltatásokat nyújt más csapatoknak, milyen bemeneteket és kimeneteket kezel, milyen minőségi jellemzőkkel.
- **Szerződés:** milyen kompatibilitási szabályok vonatkoznak az interfészre, hogyan történik a verziózás, mi a hibakezelés és az újrapróbálkozás rendje.
- **Kommunikációs protokoll:** milyen csatornákon, milyen gyakorisággal és milyen események hatására lépnek kapcsolatba a csapatok.
- **Döntési jogkör:** mely döntéseket hozhat a csapat önállóan, és melyek igényelnek közös egyeztetést vagy felsőbb szintű jóváhagyást.

Egy jól definiált csapatinterfész tehát nem csak API-végpontok listája, hanem egy működési szerződés, amely csökkenti a félreértéseket és a felesleges koordinációt. Például egy Számlázás csapat interfésze lehet: „REST API a számla létrehozására, GET /szamlak/{id} lekérdezésre, webhook a számla kiállításakor; a válaszidő 200 ms alatt, a rendelkezésre állás 99,9% havi ablakban; a séma visszamenőleg kompatibilis, a változásokat a közös API-katalógusban kell átvezetni;

hibabejelentés a megadott csatornán, prioritás szerinti válaszidővel." Ez a megfogalmazás összeköti a technikai és a szervezeti elvárásokat.

A csapathatárok kialakításakor figyelembe kell venni a kognitív terhelést is. Egy csapat csak korlátozott számú komponenst, technológiát és üzleti szabályt képes hatékonyan átlátni. Ha a csapat túl sok, egymástól független komponenst birtokol, a mentális modellje szétesik, és a döntések lassabbá, hibásabbá válnak. Ha viszont túl kevés komponenst birtokol, a csapatok közötti függőségek száma nő, és a koordinációs költség válik dominánssá. A jó határ ott van, ahol a csapat a birtokolt komponenseket teljes egészében megérti, és a más csapatokkal való együttműködés világos szerződéseken alapul.

28.1.5 Gyakori hibák és félreértések

Az egyik leggyakoribb hiba a Conway törvényének figyelmen kívül hagyása a szervezeti átalakítás során. Ha egy szervezet úgy dönt, hogy bevezeti a mikroszolgáltatásokat, de a csapatok továbbra is funkcionális silókban dolgoznak, a szolgáltatások határai nem lesznek stabilak. A fejlesztők a régi kommunikációs mintákat követve közös adatbázist, közvetlen függvényhívásokat vagy megkerülő csatornákat használnak, és a rendszer fokozatosan elosztott monolittá válik. A technológiai változás önmagában nem elég; a kommunikációs gráfot is át kell alakítani.

A másik gyakori hiba a szervezeti struktúra kritikátlan másolása. Ha a szervezet rosszul van felépítve, akkor a rendszerarchitektúra is rossz irányba torzul. Az inverz Conway-manőver nem azt jelenti, hogy a meglévő szervezeti ábrát kell a kódra erőltetni, hanem hogy a kívánt architektúrához kell igazítani a szervezetet. Ehhez először meg kell érteni a jelenlegi kommunikációs gráfot, azonosítani a konfliktusokat, és csak azután tervezni a változtatást.

Gyakori hiba az is, hogy a csapatok közötti interfészeket nem rögzítik. Ha nincs explicit szerződés, az emberek ad hoc egyeztetésekre támaszkodnak. Ez működik, amíg a csapatok kicsik és a kapcsolatok informálisak, de ahogy nő a rendszer és a szervezet, a rejtett függőségek törékennyé válnak. Ilyenkor egy-egy változás váratlanul több csapatot érint, és a hibakeresés nehéz, mert a felelősségek nem dokumentáltak.

Végül érdemes kerülni azt a félreértést, hogy a szervezeti átszervezés önmagában megoldja az architektúraproblémákat. A reorg gyakran csak az organogramot változtatja meg, a kommunikációs gráfot nem. Ha a mindennapi rutinok, a vezetői elvárások, a teljesítményértékelés és az eszközök nem követik az új határokat, a régi minták visszaszivárognak. Az inverz Conway-manőver ezért nem egyszeri esemény, hanem folyamatos tevékenység, amelyet mérni és finomítani kell.

28.1.6 Az illeszkedés ellenőrzése

A szervezeti struktúra és a rendszerarchitektúra összhangját két gráf összevetésével ellenőrizhetjük. Az egyik a szervezeti térkép: a csapatok, a felelősségek és a kommunikációs kapcsolatok. A másik az architektúrális térkép: a komponensek, az interfészek és a függőségek. A cél nem az, hogy a két gráf pontosan izomorf legyen, hanem hogy a fontos határok és függőségek összhangban álljanak.

Az összevetés során a következő kérdéseket érdemes feltenni:

- **Lefedi-e minden komponenst pontosan egy csapat felelőssége?** Ha egy komponensnek több csapat a gazdája, a döntések lassulnak, és a felelősség elmosódik. Ha egy komponensnek nincs gazdája, a karbantartás elmarad.

- **A komponensek közötti függőségek tükrözik-e a csapatok közötti kommunikációs utakat?** Ha két komponens szorosan csatolt, de a csapataik ritkán beszélnek, akkor az interfész törékeny lesz, mert a változások nem hangolhatók össze. Ha két csapat intenzíven egyeztet, de a rendszerben nincs közöttük függőség, akkor a koordináció felesleges, és érdemes átgondolni a felelősségeket.
- **Vannak-e olyan határok, ahol a kommunikációs gráf és az architektúra gráf ellentmond egymásnak?** Például egy csapat két olyan komponenst birtokol, amelyek között erős függőség van, de a komponensek külön csapatokhoz tartoznának a célarchitektúrában. Ilyenkor a szervezeti határ rossz helyen van.
- **Mekkora a csapatok közötti kommunikációs terhelés?** Ha egy csapat túl sok másik csapattal áll napi kapcsolatban, a koordinációs költség magas, és a döntések lassulnak. Az illeszkedés javításakor érdemes csökkenteni a csapatok közötti függőségek számát, vagy platform- és támogató csapatokat bevezetni.

Az összevetést nem kell bonyolult eszközökkel végezni. Egy egyszerű Mermaid- vagy PlantUML-diagram, egy táblázat a felelősségekről, és egy rövid elemzés a mismatchekről elegendő. Fontos, hogy a térképek verziózottak legyenek, és a döntéseket ADR-ekben rögzítsük.

28.1.7 Labor: szervezeti és architektúrális térkép összevetése

Ez a labor egy kis, ellenőrizhető példán gyakoroltatja a szervezeti struktúra és a rendszerarchitektúra kapcsolatának elemzését, valamint az inverz Conway-manőver alkalmazását.

28.1.8 Kiinduló állapot

Egy kitalált „ACME Webshop” belső rendszerét vizsgáljuk. A jelenlegi szervezet funkcionális silókra épül:

- **Frontend csapat:** a webes felületet fejleszti, a backend API-kat fogyasztja.
- **Backend csapat:** az üzleti logikát fejleszti, jelenleg egy monolit backendben.
- **Adatbázis csapat:** a közös adatbázist üzemelteti, a sémát kezeli.
- **Üzemeltetés csapat:** a deployt, a környezeteket és a monitoringot végzi.

A jelenlegi architektúra: webes felület, monolit backend, közös adatbázis. A rendszerhatáron belül van a webshop és a háttérrendszer; külső rendszer a fizetési szolgáltató. A csapatok közötti kommunikáció jellemzően a rétegek mentén sűrű: a frontend a backenddel egyeztet API-kat, a backend az adatbázissal sémát, az üzemeltetés mindenkivel deploy-kor.

A vállalat célja, hogy a rendszert három modulra bontsa: **Rendelés**, **Számlázás**, **Készlet**. A célarchitektúra szerint ezek külön komponensek, explicit interfészekkel: a Rendelés szinkron REST API-n kérdez készletet, a Számlázás aszinkron eseményt kap a rendelés létrejöttéről. A cél a gyorsabb üzleti változások, a jobb hibatűrés és a csapatok önállóbb működése.

28.1.9 Feladat

Készítsen reprodukálható dokumentumot vagy jegyzetfüzetet, amelyben az alábbi lépéseket végrehajtja:

1. **Jelenlegi szervezeti térkép.** Rajzolja meg a jelenlegi csapatokat és a kommunikációs kapcsolatokat. Jelölje a kommunikáció célját és gyakoriságát (napi, heti, havi). Például:

```
flowchart LR
    subgraph Szervezet
        FE[Frontend csapat]
        BE[Backend csapat]
        DB[Adatbázis csapat]
        OPS[Üzemeltetés csapat]
    end
    FE -- "API egyeztetés (heti)" --> BE
    BE -- "séma egyeztetés (heti)" --> DB
    BE -- "deploy egyeztetés (napi)" --> OPS
```

A diagramhoz adjon jelmagyarázatot: a nyilak a kommunikációs kapcsolatokat, a címkék a céljukat és a gyakoriságukat jelölik.

2. **Jelenlegi architektúrális térkép.** Rajzolja meg a jelenlegi komponenseket C4 komponens szinten: web, monolit backend, közös adatbázis, fizetési szolgáltató (külső rendszer). Jelölje a rendszerhatárt, a függőségeket és az interfészeket. Adja meg a kapcsolatok protokollját és szinkronitását.

3. **Célarchitektúra térkép.** Rajzolja meg a kívánt állapotot: Rendelés szolgáltatás, Számlázás szolgáltatás, Készlet szolgáltatás, API gateway, eseménybusz. Minden interfészhez adja meg a protokollt, a szinkronitást és a hibatűrési jellemzőket. Jelölje a rendszerhatárt és a külső rendszert.

4. **Inverz Conway-manőver.** Tervezze meg a javasolt csapatszerkezetet a célarchitektúrához. Javasoljon Rendelés, Számlázás, Készlet és Platform csapatot. Indokolja, hogy a csapathatárok hogyan tükrözik a célmodulok határait. Adja meg minden csapat felelősségét, interfészét és döntési jogköreit.

5. **ADR.** Készítsen egy Architecture Decision Recordot a következő döntésről: „A csapatok átszervezése a moduláris architektúrához igazodva”. Az ADR tartalmazza a kontextust, a döntést, a mérlegelt alternatívákat, a következményeket és az állapotot (például „javasolt”).

6. **Összevetés és mismatchek.** Hasonlítsa össze a jelenlegi szervezeti és architektúrális térképet, és azonosítson legalább három ellentmondást. Mindegyikhez írja le, milyen kockázatot hordoz, és hogyan oldja fel a javasolt csapatszerkezet. A dokumentumban külön szakaszban szerepeljenek a **mért** (jelenlegi állapot) és a **várt** (célállapot) eredmények.

28.1.10 Elvárt eredmény

Egy verziózott dokumentum vagy notebook, amely tartalmazza:

- a három térképet diagrams-as-code formában,
- a javasolt csapatszerkezetet és a hozzá tartozó interfészeket,
- az ADR-t a szervezeti és architekturális döntésről,
- a mismatch-elemzést a mért és a várt eredmény elkülönítésével.

28.1.11 Ellenőrzési szempontok

- A diagramokon szerepel-e a rendszerhatár?
- A csapatok felelőssége egyértelmű-e, átfedés és lyuk nélkül?
- A célarchitektúra interfészeinél megadott-e a protokoll, a szinkronitás és a hibatűrés?
- Az ADR tartalmazza-e a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot?
- A mért és a várt eredmények világosan el vannak-e különítve?

28.1.12 Visszaállítási út

Mivel a labor dokumentációs jellegű, nincs éles rendszer vagy destruktív művelet. A kiinduló állapot a megadott leírás; a dokumentum bármikor újragenerálható, és a verziózott előzményekből visszaállítható.

28.1.13 Önellenőrző kérdés

Egy szervezetben a számlázó rendszerért két külön csapat felel: az egyik a számlakészítést, a másik a számlakiküldést birtokolja. Milyen architekturális kockázatokat látsz ebben a felosztásban? Hogyan használnád az inverz Conway-manővert a helyzet javítására, és miért nem elég csak a kódot refaktorálni?

28.1.14 Ellenőrzőlista

- Van-e a célarchitektúrának és a szervezeti struktúrának közös, verziózott leírása?
- A csapathatárok mentén definiáltak-e interfész-szerződések?
- A csapatok felelőssége lefedi-e a rendszer komponenseit átfedés és lyuk nélkül?
- A kommunikációs utak és az architekturális függőségek közötti eltéréseket rendszeresen vizsgálják-e?
- Az átszervezés előtt van-e mérési terv a változás hatásának nyomon követésére?
- A platform és a támogató csapatok tehermentesítik-e a termékcsapatokat?
- Az ADR-ek rögzítik-e a szervezeti és architekturális döntéseket, az alternatívákkal együtt?

28.2 Stakeholder management és kommunikáció

A szoftverarchitektúra soha nem légüres térben születik. A Conway törvénye és szervezeti illeszkedés alfejezetben megismertük Conway törvényét, amely szerint a szervezetek által létrehozott rendszerek tükrözik a szervezet kommunikációs struktúráját. Ebből közvetlenül következik, hogy az architekt azon képessége, hogy az eltérő érdekű szereplőkkel hogyan kommunikál, miképp épít bizalmat és kezel konfliktust, nem „puha készség” a technikai munka mellett, hanem az architektúra minőségét befolyásoló tervezési tényező. Ebben az alfejezetben a stakeholder menedzsment és a kommunikáció alapfogalmait, az érdek–hatás–döntési jog hármasát, valamint a tárgyalás, konfliktuskezelés és döntéshozatal gyakorlati eszközeit vizsgáljuk. A cél nem az, hogy az architekt politikussá váljon, hanem hogy a szükséges döntésekhez átlátható, kiszámítható és reprodukálható folyamatot építsen, amelyben a bizalom csökkenti a tranzakciós költséget, a eltérő érdekek pedig hasznosítják a rendszer különböző nézőpontjait.

Miért kulcskérdés a stakeholder menedzsment?

A stakeholder minden olyan személy vagy csoport, aki hatással van az architektúrára, vagy akire az architektúra hatással van. Egy tipikus üzleti rendszer esetében a körbe tartozik a termékmenedzser, az üzemeltetési vezető, a biztonsági tiszt, az adatvédelmi felelős, a fejlesztői csapatok, a pénzügy, a felhasználói képviselők, ritkábban a beszállítók és a szabályozó hatóságok. Mindegyikük más nézőpontból értékeli ugyanazt az architektúra-javaslatot: a termékmenedzser a funkcionális terjedelmet és a piacra kerülési időt, az üzemeltető a stabilitást és az éjszakai riasztások számát, a biztonsági tiszt a fenyegetettséget és a megfelelési kockázatot, a pénzügy a költségoldalt, a fejlesztő a karbantarthatóságot és a kognitív terhelést. Ha az architekt ezeket a szempontokat nem hozza felszínre, a döntések vagy elsikkadnak, vagy a végén egy erősebb érdek dominál, és később valamelyik fél blokkolja a bevezetést.

Az **eltérő érdekek** önmagukban nem hibák: éppen a többféle nézőpont teszi lehetővé a rendszer átfogó tervezését. A probléma akkor keletkezik, ha az érdekek implicit módon ütköznek, és a döntéshozó nem látja át a kompromisszumok következményeit. Például ha az üzemeltetés ragaszkodik a konzervatív, monolitikus, jól bejáratott deploymenthez, a termékmenedzser pedig gyors, független funkciókiadást akar, akkor a felületi vita mögött a kiadási kockázat és a piaci sebesség között kell dönteni. A stakeholder menedzsment feladata az ilyen trade-offok explicit, dokumentált megjelenítése, nem pedig annak elkerülése.

A **bizalom építése** ebben a folyamatban azért kulcsfontosságú, mert a komplex architekturális döntések nem teljesen formalizálhatók. A résztvevők gyakran bizonytalan előrejelzésekre, részeredményekre és szakértői becslésekre támaszkodnak. Ha a felek nem bíznak abban, hogy az architekt a közös cél érdekében mond igazat, akkor a döntések mögött védekező magatartás alakul ki: mindenki a saját területét védi, az információ nem áramlik szabadon, és a döntések vagy elhúzódnak, vagy a legalacsonyabb közös nevezőre esnek. A bizalom nem egy egyszeri gesztus, hanem a következetes viselkedés, az átlátható indoklás és az érdekek tiszteletben tartása révén felhalmozott tőke. A bizalom egyik hasznos, gyakorlatban is alkalmazható modellje a hitelesség (credibility), megbízhatóság (reliability), kapcsolati közelség (intimacy) és önorientáltság (self-orientation) egyenlete: a bizalom értéke akkor nő, ha az architekt szakmailag hiteles, ígéreteit betartja, képes meghallgatni a másik fél érzéseit és motivációit, és nem a saját karrierjét vagy presztízsét helyezi előtérbe. Ez a modell segít azonosítani, hogy egy adott kapcsolatban hol gyenge a bizalom: gyakran nem a szakmai tudással, hanem a megbízhatósággal vagy az önorientáltsággal van gond. Ha például az architekt minden meetingen javaslatot tesz, de soha nem kíséri figyelemmel a megvalósulást, a fejlesztők idővel nem veszik komolyan a döntéseit.

Érdek, hatás és döntési jog

A stakeholder elemzés első lépése az érdekek és a hatás (befolyás) feltárása, valamint a döntési jogok világos lehatárolása. Különbséget kell tenni az **érdek** (interest) és a **pozíció** (position) között. A pozíció az, amit a szereplő elsőre mondani szok: például „használjunk mikroszolgáltatásokat”. Az érdek az a mögöttes igény, ami ezt motiválja: a csapatok gyors, független szállítása, a rendszer kisebb modulokban mérhető üzemeltetése, vagy a technológiai vonzerő. Az architekt feladata, hogy a pozíció mögé kérdezzen, és az érdekeket a döntési folyamat szintjén jelenítse meg. Ellenkező esetben a vita két konkrét javaslatról szól, és a valódi célok rejtve maradnak.

A **hatás** azt fejezi ki, hogy egy szereplő mennyire képes befolyásolni a projekt sikerét vagy az architektúra jóváhagyását. A hatás lehet formális (szervezeti pozíció, költségvetési felelősség) vagy informális (szakmai tekintély, kapcsolati háló, kritikus tudás birtoklása). A gyakorlatban elterjedt eszköz a **power/interest mátrix**: a vízszintes tengely az érdekeltség, a függőleges a hatás; az így kapott négy mezőbe sorolt stakeholdereket eltérő stratégiával kell kezelni. A nagy hatású, erősen érdekelt szereplőket aktív együttműködésbe kell vonni és rendszeresen tájékoztatni; a nagy hatású, de alacsony érdekeltségű szereplőket elégedettségben kell tartani, mert hirtelen blokkolhatnak; az alacsony hatású, de erősen érdekelt szereplőknek folyamatos tájékoztatást kell adni; az alacsony hatású és alacsony érdekeltségűeket pedig csak változás esetén kell informálni. Fontos, hogy ez a besorolás időben változik: egy adatvédelmi felelős például a megfelelési audit előtt hirtelen magas hatásúvá válik, még ha korábban kevés figyelmet kapott is.

A **döntési jogok** modellezése nem öncélú bürokrácia, hanem az elszámoltathatóság megteremtésének eszköze. A leggyakoribb keretek a RACI, DACI, vagy a RAPID. A RACI (responsible, accountable, consulted, informed) egyértelműsíti, ki dolgozik a feladaton, ki viseli a végső felelősséget, kitől kell konzultációt kérni, és kit kell tájékoztatni. Az architektúra döntéseknél különösen fontos, hogy minden döntéshez pontosan egy **accountable** személy tartozzon, mert a többes felelősség a gyakorlatban felelősség-nélküliséget jelent. A DACI (driver, approver, contributor, informed) hasonló, de a meghajtó szerepet és a jóváhagyót jobban hangsúlyozza. A RAPID (recommend, agree, perform, input, decide) tovább részletezi a döntési folyamat lépéseit. Válasszunk

egyet, és a szervezetben következetesen használjuk. A döntési jogok nemcsak emberekre, hanem testületekre is vonatkozhatnak: például az Architecture Review Board jóváhagyja a stratégiai irányt, de a csapat szintű technológiai választást a platform engineering csapat döntheti el. A lényeg, hogy a döntési jogok rögzítve legyenek, és ADR-hez kapcsolódjanak: minden architekturális döntés hivatkozzon arra, kinek mi volt a szerepe. Ez nem csak a visszakereshetőség miatt fontos, hanem a szervezeti tanulás miatt is: ha egy döntés később rossznak bizonyul, a felelősség nem személyeskedéshez, hanem a döntési folyamat felülvizsgálatához vezet.

Érdekes kapcsolat van a döntési jogok és a szervezeti bizalom között. Ha a stakeholderek azt érzik, hogy a döntési jogokat önkényesen vagy átláthatatlanul gyakorolják, a bizalom gyorsan erodálódik. Ha viszont a jogok világosak, és a döntések indoklása nyilvános (például ADR-ben olvasható), akkor még azok is elfogadják a döntést, akik nem értenek egyet vele, mert látják, hogy a folyamat tisztességes volt. Ezt gyakran processz-igazságosságnak nevezik. A gyakorlatban ez azt jelenti, hogy az architekt ne csak a döntés végeredményét kommunikálja, hanem a mérlegelt alternatívákat, a kritériumokat és a kockázatokat is. Ezáltal a döntések okai tanulhatók, és a jövőbeli viták konstruktívabbak lesznek.

Tárgyalás, konfliktus és döntés

A stakeholder menedzsment elkerülhetetlenül magában foglalja a tárgyalást, mert a szűkös erőforrások (idő, költségvetés, figyelem) és az eltérő célok között kompromisszumot kell találni. A tárgyalás nem csata, hanem közös problémamegoldás. A Harvard-módszer néven ismert tárgyalási elvek négy pontja jól alkalmazható architekturális vitákban:

1. **Válaszd el az embert a problémától.** Az architektúra-vita nem személyes presztízsharc, hanem a rendszer minőségére irányuló elemzés. Ha a másik fél agresszívan védi a saját javaslatát, érdemes a vitát a minőségi attribútumokra, azaz a mérhető következményekre terelni.
2. **Fókuszálj az érdekekre, ne a pozíciókra.** Ahogy már említettük, a konkrét technológiai választás mögött gyakran üzleti vagy üzemeltetési igény áll. Ha ezt feltárjuk, alternatívák is megjelenhetnek, amelyek mindkét felet kielégítik.
3. **Generálj több opciót, mielőtt döntesz.** Az architektúra döntéseknél ritka, hogy csak két kizárólagos lehetőség létezik. Például a szigorú konzisztencia és a rendelkezésre állás közötti választás gyakran köztes megoldásokkal enyhíthető: aszinkron replikációval, olvasási replikákkal vagy üzleti folyamatonkénti differenciálással. A több opció kialakítása csökkenti a nulla összegű gondolkodást.
4. **Használj objektív kritériumokat.** A minőségi követelményeket quality attribute scenario formában (inger, környezet, válasz, mérőszám) érdemes megfogalmazni, mert így a vita mérhetővé válik. Például ahelyett, hogy „gyors” legyen a rendszer, a kritérium legyen: „A rendszer 2000 egyidejű felhasználó mellett a számlaadatokat 1,5 másodpercen belül válaszolja meg a 95. percentilisben, a havi 99,5%-os SLO mellett.” Az objektív kritériumok segítenek eltávolítani a szubjektív preferenciákat.

A tárgyalás során az architektnek ismernie kell a **BATNA** (Best Alternative To a Negotiated Agreement) fogalmát: mit tehet, ha nem születik megegyezés? Ez határozza meg a tárgyalási pozíció erejét. Ha az architekt egyetlen alternatívája az, hogy mindenképp elfogadja a másik fél javaslatát, akkor lényegében nincs tárgyalási alapja. Ha viszont van egy elfogadható B-terv (például a meglévő rendszer fokozatos továbbfejlesztése), akkor képes tárgyalni. A **ZOPA** (Zone of Possible Agreement) pedig az a sáv, ahol mindkét fél számára elfogadható megállapodás születhet. Az architekt feladata, hogy ezt a zónát tágítsa az érdekek feltérképezésével és az opciók bővítésével.

A **konfliktus** a stakeholder menedzsment természetes velejárója. Nem minden konfliktus rossz: a feladatra irányuló, szakmai konfliktus javíthatja a döntés minőségét, míg a személyes vagy

kapcsolati konfliktus rombolja a bizalmat. A Thomas–Kilmann-féle konfliktuskezelési modell öt alapstratégiát különböztet meg: versengés, együttműködés, kompromisszum, elkerülés és alkalmazkodás. A stratégia megválasztása a konfliktus tárgyának fontosságától és a kapcsolat hosszú távú értékétől függ. Ha a döntés kritikus és a kapcsolat is fontos, az együttműködés a célravezető: a felek közösen feltárják az érdekeket, és olyan megoldást keresnek, amely mindkét fél igényét figyelembe veszi. Ha azonban a vita egy olyan technikai részletről szól, amelynek nincs jelentős üzleti hatása, és a másik féllel való jó viszony sem létfontosságú, az alkalmazkodás vagy akár az elkerülés is racionális lehet. A kompromisszum akkor jó, ha gyors megállapodásra van szükség, és a felek hajlandók feladni valamit. A versengés ritkán ideális, de indokolt lehet, ha például egy biztonsági sérülékenység azonnali beavatkozást igényel, és nincs idő hosszú egyeztetésre. Az architektnek tudatosan kell választania, nem pedig a megszokott viselkedési mintát követni.

A **döntéshozatal** módja is lényeges. Az architektúrában alkalmazott döntési eljárások közül a leggyakoribbak:

- **Konszenzus:** minden releváns fél egyetért. Előnye a magas elkötelezettség, hátránya a lassúság és a csoportgondolkodás kockázata. Csak akkor érdemes, ha a döntés visszafordíthatatlansága magas, és a stakeholderek száma kicsi.
- **Többségi szavazás:** gyors, de gyakran a kisebbségi vélemény elveszik, ami később ellenállást szül.
- **Konzultáció a döntéshozóval:** a felelős személy meghallgatja az érintetteket, majd maga dönt. Ez hatékony, ha a felelősnek megfelelő kompetenciája van, és a döntést jól dokumentálja.
- **Delegáció:** a döntést egy csapatra vagy szakértőre bízzák. Ez felgyorsítja a folyamatot, de bizalmat igényel.

Az architektúra döntéseknél gyakran a **konzultáció** és a **delegáció** kombinációja működik: a stratégiai, hosszú távú döntéseket az Architecture Review Board jóváhagyja, a csapat szintű, visszafordítható technológiai részleteket pedig a fejlesztői csapat dönti el. A lényeg, hogy a döntési mód mindig illeszkedjen a döntés tétjéhez és a rendelkezésre álló időhöz. A döntés után fontos a **disagree and commit** elv: ha valaki a vita során veszít, a döntés megszületése után konstruktívan támogatnia kell a végrehajtást, nem pedig passzívan szabotálnia. Ezt a kultúrát a vezetőnek és az architektnek egyaránt erősítenie kell.

Minden fontos döntést **ADR-ben** (Architecture Decision Record) kell rögzíteni. Az ADR sablonja tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. A stakeholder menedzsment szempontjából az ADR-nek azt is érdemes tartalmaznia, hogy ki volt a döntés felelőse, kiket konzultáltak, milyen ellenvélemények merültek fel, és azokat miért utasították el. Így a döntési folyamat átláthatóvá válik, és a jövőbeli olvasó megérti a kompromisszumokat. Példa egy ADR részletére:

```
## ADR-2025-07-107: A számlázási modul szinkron REST API-n keresztül kommunikál a CRM-mel
Státusz: Elfogadott
Kontextus: A termékmenedzser 3 hónapos piacra kerülési célt adott, az üzemeltetés
a rendelkezésre állást 99,9%-os SLO-val védi. A biztonsági tiszt a számlázási
adatokat PII-ként kezeli.
Döntés: A CRM-ből a számlázó rendszer szinkron HTTPS REST hívással kérdez le
számlázási adatokat, timeout 2 s, circuit breaker 5 hibás hívás után 30 s-ig nyit.
Alternatívák: (1) Aszinkron event streaming - jobb laza csatolás, de magasabb
kezdeti komplexitás és a consistency window nő. (2) Megosztott adatbázis -
ellenjavallt a PII és a független fejlesztési sebesség miatt.
Következmények: A hívás láncban a CRM meghibásodása a számlázási folyamatot
blokkolhatja, ezért retry és fallback cache szükséges. A PII adatáramlás miatt
TLS és adatvédelmi felülvizsgálat kötelező.
```

```
Döntési jog: Jóváhagyta a CTO (accountable), konzultáltak: üzemeltetési vezető,
biztonsági tiszt, termékmenedzser.
```

Az ilyen ADR nemcsak a technikai döntést, hanem a stakeholder mérlegelést is láthatóvá teszi. Ez erősíti a bizalmat, mert nyilvánvalóvá válik, hogy a döntés nem a háttérben született.

Kommunikációs terv

A stakeholder elemzés és a döntési jogok megadása csak akkor ér valamit, ha a kommunikációt is tudatosan tervezzük. A kommunikációs terv egy verziózott dokumentum, amely rögzíti, hogy az egyes stakeholderekkel milyen csatornán, milyen gyakran, milyen üzenetet cserélünk. A terv nem statikus; rendszeresen felülvizsgáljuk, különösen akkor, ha a projekt fázisa, a kockázatok vagy a szervezeti struktúra változik. A terv első lépése a **stakeholder térkép** elkészítése. Ennek legalább a következő oszlopokat kell tartalmaznia: stakeholder, szerep, érdek, hatás (magas/alacsony vagy 1–5 skálán), döntési jog, kommunikációs igény, csatorna, gyakoriság, felelős. Példa egy kivonatra:

Stakeholder	Érdek	Hatás	Döntési jog	Csatorna	Gyakoriság
Termékmenedzser	Funkcionális terjedelem, üzleti érték	5	Jóváhagyó a funkciókban	Heti 1:1, havi demó	Hetente
Üzemeltetési vezető	Stabilitás, alacsony riasztás	4	Jóváhagyó a deployment ablakhoz	Havi üzemeltetési review	Havonta
Biztonsági tiszt	Megfelelés, kockázat	3	Blokkoló jog sérülékenységnél	Threat modeling workshop, RFC	Eseményvezérelt
Adatvédelmi felelős	GDPR megfelelés	3	Jóváhagyó az adatkezeléshez	Konzultáció a tervezéskor	Mérföldkövenként
Fejlesztői csapat	Karbantarthatóság, kognitív terhelés	4	Ajánlási jog	Squad szintű standup, ADR review	Hetente

A csatornák kiválasztásánál figyeljünk arra, hogy az üzleti stakeholdereknek általában más a nyelvük, mint a mérnököknek. Az architekt feladata, hogy a technikai döntés üzleti hatását is képes legyen elmondani: ne a „circuit breaker timeout paraméterét” hangsúlyozza, hanem azt, hogy egy leállás esetén mennyi bevétel csúszhat, vagy milyen ügyfélpanaszok keletkeznek. Ezt a kommunikációt segíthetik a minőségi forgatókönyvek, amelyek mérhetővé teszik az elvárásokat. Például egy kommunikációs SLO: „Amikor egy üzleti stakeholder új szabályozási követelményt jelez, az architekt 2 munkanapon belül írásban válaszol a hatásértékeléssel, amely legalább a érintett komponensek 90%-át lefedi.” Az ilyen elvárások megfogalmazása csökkenti a félreértést.

A kommunikációs terv **ritmusai** közé tartoznak a rendszeres architektúra-review-k, a demo-k, a barna zacskós beszélgetések, a döntési viták és az ADR-konzultációk. A ritmusok nem öncélúak: mindegyiknek legyen világos célja és kimenete. Például:

- **Heti 1:1 a termékmenedzserrel:** a mérföldkövek, a technikai adósság és az üzleti prioritások összehangolása.
- **Havi architektúra-review:** a stratégiai döntések, az eltérések és a technológiai roadmap felülvizsgálata, ADR-ek elfogadása vagy elutasítása.

- **Eseményvezérelt threat modeling workshop:** biztonsági kockázatok feltárása, a bizalmi határok jelölése, a döntési jogok aktualizálása.
- **Kétheti demó a fejlesztőknek:** a készülő architektúra-minták bemutatása, visszajelzések gyűjtése.

A kommunikációs terv ne csak az architekt kifelé irányuló kommunikációját tartalmazza, hanem a bejövő csatornákat is: hogyan jelezhetnek a fejlesztők architekturális eltérést, hogyan emelhetik a stakeholderek a kockázatokat, és hogyan kapcsolódik mindez a döntési naplóhoz. A gyakorlatban egy közös, verziókezelt tár (például egy git repository-ban vezetett `docs/architecture` mappa) alkalmas erre, ahol az ADR-ek, a stakeholder térkép és a kommunikációs terv együtt élnek. Így a változások követhetők, a döntések visszakereshetők, és a felelősség egyértelmű.

A kommunikáció hatékonyságát mérni is érdemes, de ne essünk a felesleges mérés csapdájába. Például nyomon követhetjük a döntési átfutási időt (az ADR-ben rögzített javaslat és a jóváhagyás közötti medián napok számát), a stakeholder elégedettséget (1–5 Likert-skálán, anonim felméréssel), vagy az újranyitott döntések arányát (hány ADR-t kellett módosítani a jóváhagyást követő 30 napon belül). Ezek a mérőszámok segítenek azonosítani a kommunikációs folyamat szűk keresztmetszeteit. Fontos azonban megkülönböztetni a **várt** értéket a **mért** értéktől: amíg nincs tényleges adat, csak célként fogalmazzuk meg; a mérés után pedig rögzítsük a környezetet, az időablakot és a bizonytalanságot.

Tipikus hibák és ellenpontok

- **Egyszeri stakeholder elemzés, majd elfelejtjük.** A stakeholder-ek és a hatások időben változnak. Például egy új adatvédelmi törvény hatálybalépése miatt az adatvédelmi felelős hirtelen magas hatásúvá válik. Ellenpont: a stakeholder térképet minden mérföldkőnél, illetve jelentős szervezeti vagy külső változásnál frissítsük, és a frissítést ADR-rel kössük össze.
- **A kommunikáció egyirányú.** Az architekt csak tájékoztat, de nem hallgatja meg a stakeholderek igényeit. Ennek jele, ha a review meetingeken nincs vita, és mindenki csak bólogat. Ellenpont: minden fórumnak legyen hallgatási és visszacsatolási szakasza; a stakeholdereknek legyen módjuk a döntés előtt véleményt formálni.
- **A döntési jogok homályosak.** Mindenki dönt, de senki sem vállalja a felelősséget. A döntések elhúzódnak, és a hibákért senki nem felel. Ellenpont: a RACI vagy RAPID keret bevezetése, és minden döntéshez egyetlen accountable személy rendelése, akit az ADR-ben rögzítünk.
- **A technikai zsargon kizárja az üzleti döntéshozókat.** Az architekt a CAP-tételről, a saga mintáról és a service mesh-ről beszél a termékmenedzsernek, aki elzárkózik vagy passzívvá válik. Ellenpont: a döntések üzleti hatását minőségi forgatókönyvekben, bevételi kockázatban és ügyfélélményben fogalmazzuk meg; a technikai részleteket az arra alkalmas fórumokon, a mérnökökkel tárgyaljuk.
- **A konfliktus elkerülése.** Az architekt kerüli a nehéz beszélgetéseket, és a döntést a szőnyeg alá söpri. Később a feszültség robbanásszerűen tör elő, gyakran a legrosszabb időzítéssel. Ellenpont: a konfliktust a folyamat korai szakaszában, strukturált módon kezeljük; a Thomas–Kilmann-modell segít tudatosan választani a stratégia helyett a megszokott reakció helyett.
- **A nem funkcionális követelményeket technikai részletként kezelik.** A produkciós rendelkezésre állás, a biztonság, az adatvédelem sokszor csak a végén kerül elő, amikor már nincs idő a javításra. Ellenpont: a minőségi attribútumokat a stakeholder elemzés részeként, az üzleti célokkal összekapcsolva, quality attribute scenario formában előre rögzítjük.

Ezek a hibák gyakran összefüggenek: a homályos döntési jogok rontják a bizalmat, a csökkent bizalom miatt gyengül a kommunikáció, a gyenge kommunikáció pedig elrejti az eltérő érdekeket,

ami rossz döntésekhez vezet. Ezért a stakeholder menedzsmentet rendszerként kell kezelni, nem pedig egyszeri feladatként.

Labor: Stakeholder- és kommunikációs terv készítése

A tanultak gyakorlásához készíts stakeholder- és kommunikációs tervet egy kis, ellenőrizhető példán. A labor célja nem egy tökéletes dokumentum elkészítése, hanem a döntési logika és a bizalomépítő folyamat kipróbálása.

Kiinduló állapot: Egy fiktív vállalat, a „DunaShop e-kereskedelmi platform" bővíteni kívánja a rendszerét egy új számlázási integrációval. A jelenlegi architektúra monolitikus, és a vállalat azt fontolgatja, hogy a számlázó modult külön szolgáltatásként valósítja meg. A releváns stakeholderek: a CTO (Forma), a termékmenedzser (Anna), az üzemeltetési vezető (Gábor), a biztonsági tiszt (Rita), az adatvédelmi felelős (Márk) és a számlázó modulon dolgozó fejlesztői csapat vezetője (Zsolt). A projekt határideje 6 hónap. A CTO az architekturális döntéseket jóváhagyja, de a csapatoknak nagy szabadságot ad.

Feladatok:

1. **Stakeholder elemzés:** Töltsd ki a stakeholder térképet. Határozd meg minden szereplő érdekét, hatását (1–5 skála), érdekeltségét (1–5), és besorolását a power/interest mátrixba. Készíts diagrams-as-code formában (például Mermaid vagy PlantUML) egy power/interest mátr ixot vagy egy befolyás-diagramot.
2. **Döntési jogok:** Válassz egy RACI vagy DACI modellt, és rögzítsd, ki a felelős, ki a jóváhagyó, kitől kell konzultálni és kit kell tájékoztatni a következő döntésekben: (a) szolgáltatás-alapú kiválasztás, (b) az adatbázis-séma változtatás, (c) a deployment ablak meghatározása, (d) a biztonsági tesztelés jóváhagyása.
3. **Kommunikációs terv:** Készíts táblázatot a következő oszlopokkal: stakeholder, üzenet, csatorna, gyakoriság, felelős. Alakíts ki legalább három különböző ritmust (például heti, havi, eseményvezérelt). Fogalmazz meg egy minőségi forgatókönyvet a kommunikációs válaszidőre (inger, környezet, válasz, mérőszám).
4. **ADR a vitatott döntéshez:** Írj egy ADR-t a szolgáltatás-alapú számlázási modell bevezetéséről, amelyben dokumentálsz legalább két alternatívát, a döntési jogokat, és a stakeholderek ellenvetéseit. A döntés lehet kitalált, de a szerkezet legyen a könyvben megismert ADR-sablonnak megfelelő.
5. **Mért és várt eredmény elkülönítése:** Fogalmazz meg a kommunikációs tervhez **várt** célokat (például válaszidő, stakeholder elégedettségi pontszám), és jelezd, hogy ezeket hogyan lehetne mérni. Ne adj meg fiktív mérési adatot, mert az megtévesztené az olvasót; helyette írd le, hogy milyen környezetben és időablakban lehetne a mérést elvégezni.

Ellenőrzési szempontok:

- Minden stakeholder szerepel-e a térképen, és az érdekük le van-e írva?
- A power/interest besorolás indokolt-e?
- A döntési jogoknál minden döntéshez pontosan egy accountable személy tartozik-e?
- A kommunikációs terv tartalmaz-e csatornát, gyakoriságot és felelőst?
- A minőségi forgatókönyv tartalmazza-e a négy elemet (inger, környezet, válasz, mérőszám)?
- Az ADR hivatkozza-e a döntési jogokat, és tartalmazza-e az elvetett alternatívákat?
- A dokumentum verziózott-e, és a változások követhetőek-e?

Visszaállítási út: A labor nem érint élő rendszert, ezért nincs visszaállítási teendő. Ha valaki a dokumentációt egy megosztott tárban vezeti, a labor végén commitolja a verziót, és zárja le a változást.

A labor eredménye egyetlen, reprodukálható dokumentum, amelyben a döntések indoklása és a stakeholder elemzés egyaránt megtalálható. Érdemes a csoporttársakkal egymás dokumentumait felülvizsgálni, mert a külső nézőpont segít azonosítani a vakfoltokat.

Mért és várt eredmény különválasztása: A labornál a **várt** eredmény a kommunikációs SLO: például „Az architekt a stakeholder kérdéseire 2 munkanapon belül válaszol." Ez egy cél, nem mérés. A **mért** eredményhez adat kell egy tényleges projektből, például a levelezőrendszerből kinyert válaszidők. A laborban ez utóbbi nem állítható elő, de a tervnek tartalmaznia kell a mérés módszerét, hogy a későbbi projektben reprodukálható legyen. Így a labor a tanulást szolgálja, nem téveszti meg az olvasót.

Ellenőrzőlista a fejezethez

- ☐ Azonosítottad-e az összes releváns stakeholdert, beleértve a belső és külső szereplőket?
- ☐ Feltártad-e az érdekeket a pozíciók mögött?
- ☐ Felmérted-e minden stakeholder hatását és érdekeltségét, és frissítetted-e a mátrixot?
- ☐ Rögzítetted-e a döntési jogokat valamilyen kerettel (RACI, DACI, RAPID)?
- ☐ Minden döntéshez pontosan egy felelős tartozik-e?
- ☐ A döntéseket dokumentáltad-e ADR-ben, benne a stakeholderek ellenvetéseivel?
- ☐ Készítettél-e kommunikációs tervet csatornákkal, gyakorisággal és felelőssel?
- ☐ A kommunikáció mérhetővé tételéhez definiáltál-e minőségi forgatókönyvet?
- ☐ Kezeled-e a konfliktusokat strukturált módon, tudatosan választva a stratégiát?
- ☐ Építed-e a bizalmat következetes viselkedéssel, átlátható döntésekkel és a másik fél érdekeinek tiszteletben tartásával?

Önellenőrző kérdés

Egy architektúra-review-n a biztonsági tiszt ragaszkodik ahhoz, hogy a számlázó modul adatbázisát teljesen külön, kizárólag a biztonsági csapat által kezelt tárhelyre helyezzék, míg a fejlesztői csapat szerint ez lassítja a fejlesztést és növeli a működési költséget. A vita feszültté válik, és a fejlesztői csapat vezetője azt javasolja, hogy egyszerűen szavazzanak. Te vagy az architekt. Hogyan járs el? Válaszodban térj ki az érdekek feltárására, a döntési jogokra, a lehetséges kompromisszumra és az ADR szerepére a döntés rögzítésében.

Ez az önellenőrző kérdés azt gyakoroltatja, hogy a fejezetben tanult elveket egy valószerű konfliktushelyzetben alkalmazd: ne a pozíciók között válassz, hanem a mögöttes érdekeket tárj fel (biztonsági kockázat, adatvédelmi megfelelés, fejlesztési sebesség, költség), majd határozd meg, kinek van döntési joga ebben a kérdésben (például a CTO mint accountable, biztonsági tiszt mint konzultált, csapatvezető mint felelős a végrehajtásért). Keress olyan opciót, amely mindkét felet kielégítheti, például a tárhely felett a biztonsági csapat felügyeleti jogot kap, de a deployment föderatív marad, vagy egy kompromisszumos megoldást, amelyben az érzékeny adatok titkosítva, de a fejlesztői környezet számára elérhetően tárolódnak. A döntést ADR-ben rögzítsd, az elvetett alternatívákkal együtt. A válaszban a hangsúly a folyamaton és az átláthatóságon legyen, nem a konkrét technológiai választáson.

A fejezet anyaga így egy olyan eszköztárat ad az architektnek, amellyel a szervezeti érdekeket a technikai döntések javára tudja fordítani, és amely a későbbi, az architekt szerepét és karrierjét tárgyaló Architekt szerep, karrier és mentorálás alfejezetnek is alapja. A stakeholder menedzsment nem egy egyszeri feladat, hanem az architekton belüli folyamatos tevékenység, amely a bizalom, a hatás és a döntési jogok rendszeres karbantartásával segíti a sikeres architektúrát.

28.3 Architekt szerep, karrier és mentorálás

A szoftverarchitektúra szervezeti és kulturális kérdéseit tárgyaló fejezet előző egységei bemutatták, hogyan hat a szervezeti felépítés a rendszerre, és miért nem lehet az architektúrát a stakeholder-viszonyoktól elválasztva kezelni. Ez az egység az architekt személyére, szerepére és fejlődési útjára szűkíti a fókuszt. Nem az a kérdés, hogy „mi a jó architektúra", hanem az, hogy „mit csinál valaki, aki architektúrát formál, és hogyan lesz benne egyre jobb". A szerep megértése azért is kritikus, mert a szervezetek gyakran a címhez kötik a döntési jogkört, miközben a valódi architekturális munka a jogkörön túl a befolyásolásban, a tanításban és a visszacsatolási hurkok kialakításában történik.

28.3.1 1. Az architekt szerep változatai

Az „architekt" szó önmagában nem ír le egyetlen, minden szervezetben azonos munkakört. A gyakorlatban a szerep legalább három dimenzió mentén változik: a döntési fókusz szintje, a technológiai spektrum szélessége, valamint a szervezeti beágyazottság módja. Aki ezeket a dimenziókat érti, az nem ragad le egyetlen definíciónál, hanem a konkrét környezetben tudja beazonosítani, milyen architekt szerepre van szükség.

28.3.2 1.1. Címek és felelősségek

A leggyakrabban használt címek – enterprise architect, solution architect, software architect, systems architect, cloud architect, data architect – nem pusztán marketingcímkék, hanem eltérő döntési fókuszt jelölnek. Az **enterprise architect** elsősorban a szervezeti képességek, az üzleti folyamatok és az IT-portfólió közötti illeszkedéssel foglalkozik; jellemzően olyan döntéseket hoz, amelyek több rendszeren átívelnek, és az években mérik a következményüket. A **solution architect** egy adott üzleti probléma megoldásának end-to-end technikai tervéért felelős: az ő munkája a rendszerhatárok kijelölése, az integrációk megtervezése és a nem funkcionális követelmények teljesíthetőségének igazolása. A **software architect** a kódbázison belüli struktúrára, a modulhatárokra, a függőségi szabályokra és a fejlődési útra összpontosít. A **systems architect** a szoftvert a hardverrel, a hálózattal és az üzemeltetési környezettel együtt kezeli. A **cloud architect** a felhőszolgáltatásokból összeállított, költséghatékony és üzemeltethető platformot tervezi. A **data architect** az adatmodellek, a perzisztencia, az adatminőség és az adatkormányzás szerkezetét alakítja.

Ezek a címek azonban megtévesztők lehetnek. Egy erőforrás-szegény induló vállalkozásban ugyanaz a személy lehet egyszerre software architect, solution architect és cloud architect, miközben enterprise szintű portfólió-döntések is az ő asztalán landolnak. Egy nagy pénzügyi szervezetben viszont a címek elválhatnak a tényleges döntési jogkörtől: előfordul, hogy az „architect" cím birtokosa elsősorban prezentációkat készít, miközben a valódi architekturális döntéseket szenior mérnöki csoportok hozzák meg kód-review közben. A szerep tehát nem azonos a címmel. A szerep az a felelősségi kör, amelyet a szervezet a döntési jogkörrel, az elszámoltathatósággal és a kommunikációs csatornákkal együtt valakire ruház vagy valaki magára vállal.

28.3.3 1.2. A szerep négy arca

A címeknél hasznosabb, ha az architekt szerepét négy tevékenységcsoport mentén értelmezzük. Az első a **szakértői hozzájárulás**: az architekt mély technikai tudást visz a döntésekbe, ismeri a platformok képességeit és korlátait, és meg tudja ítélni, hogy egy javaslat a gyakorlatban működni

fog-e. A második a **kapcsolati integráció**: az architekt hidat képez az üzleti stakeholder, a fejlesztőcsapat, az üzemeltetés és a biztonsági szervezet között, fordítja a követelményeket technikai opciókra és a technikai kockázatokat üzleti nyelvre. A harmadik a **döntési keret**: az architekt nem csupán döntéseket hoz, hanem kialakítja azt a folyamatot és dokumentációs gyakorlatot, amelyben a döntések visszakereshetők és felülvizsgálhatók. A negyedik a **szervezeti tanulás**: az architekt tudást ad át, mentorként fejleszt másokat, és olyan visszacsatolási hurkokat épít, amelyek a szervezet egészét teszik jobbá a technológiai döntésekben.

Ez a négy arc nem egyenlő súllyal jelenik meg a karrier minden szakaszában. Egy kezdő architekt gyakran a szakértői hozzájárulásra koncentrál, mert azt érzi biztonságosnak; egy érett architektnél a kapcsolati integráció és a szervezeti tanulás válik dominánssá, mert a döntési hatás itt a legnagyobb. Aki kizárólag a szakértői arcban ragad, az könnyen válik a csapat „technikai orákulumává", aki minden kérdésre tudja a választ, de nem épít maga mögé olyan szervezetet, amely nélküle is képes jó döntéseket hozni.

28.3.4 1.3. A szerep szervezeti beágyazottsága

Az architekt szerep elhelyezkedése a szervezeti struktúrában erősen befolyásolja a hatékonyságot. A **centralizált** modellben az architekt egy elkülönített csapatban dolgozik, és onnan szolgálja ki a fejlesztői csapatokat. Ez a modell konzisztenciát és szabványosítást biztosít, de távolíthatja az architektet a napi fejlesztési kihívásoktól, és lassíthatja a döntéseket. A **beágyazott** modellben az architekt a fejlesztői csapat tagja, és a csapat saját rendszerének architektúrájáért felelős. Ez gyors visszacsatolást és nagyobb felelősségvállalást eredményez, de a rendszerek közötti integráció és a szervezeti szintű konzisztencia háttérbe szorulhat. A **hibrid** modell – amelyet a Team Topologies keretrendszer „enabling team" és „platform team" fogalmaival is leír – arra törekszik, hogy az architekt csapatok között mozogva, tanácsadóként és mentorként dolgozzon, nem pedig utasítást osztogató felügyelőként.

A szerep beágyazottsága közvetlenül kapcsolódik a jogkörhöz. Az architektnek lehet **döntési jogköre** – például vétójog a technológiai választások felett –, **ajánlási jogköre** – amikor a döntést a fejlesztői csapat hozza meg, de az architekt véleményét kötelezően figyelembe veszik –, vagy **befolyásolási jogköre** – amikor az architekt elsősorban meggyőzéssel, példákon és tanítással hat. A kezdő architekt gyakran azt hiszi, hogy a döntési jogkör a szerep lényege; a tapasztalt architekt tudja, hogy a befolyásolás tartósabb eredményt hoz, mert a döntési jogkör nélkül meghozott, de a csapat által elfogadott döntés erősebb elköteleződést és jobb végrehajtást eredményez.

28.3.5 2. A fejlődési út: kódból a döntési hatás felé

Az architekt karrierút nem egy szervezeti ranglétra, hanem a döntési hatás növelésének útja. Aki architekt szeretne lenni, annak legfontosabb feladata nem a cím megszerzése, hanem a döntési környezet megértése: hol születnek a technológiai döntések, ki viseli azok következményeit, és hogyan lehet a döntéseket minőségileg jobbá tenni.

28.3.6 2.1. A fejlődési szintek

Az architekt fejlődését hasznos modellként négy szintre bontani, de hangsúlyozni kell, hogy ezek nem merev lépcsők, hanem egymásra épülő, de egymással párhuzamosan is fejlődő képességterületek. Az első szint a **technikai mélység szintje**: itt az illető egy adott technológiában vagy rendszerben mély szakértő, aki a kódban, a konfigurációban és az üzemeltetésben is otthon

van. A második szint a **rendszerlátás szintje**: itt már nem csak egy komponenst, hanem a komponensek közötti kapcsolatokat, az adatáramlást, a függőségeket és a hibamódokat is átlátja. A harmadik szint a **döntési keret szintje**: itt az illető képes a döntési lehetőségeket összehasonlítani, a trade-offokat expliciten leírni, és a döntést a későbbi visszakereshetőség érdekében dokumentálni. A negyedik szint a **szervezeti hatás szintje**: itt az illető másokat fejleszt, csapatok közötti együttműködést alakít, és a technológiai döntéseket az üzleti hatással, a szabályozási kockázattal és a hosszú távú fenntarthatósággal együtt kezeli.

Ez a szintezés nem hierarchikus abban az értelemben, hogy a technikai mélység ne maradna fontos. Éppen ellenkezőleg: az architekt hitelessége gyakran azon múlik, hogy a csapat látja-e benne azt, aki a kóddal is boldogul. Aki túl korán eltávolodik a kódtól, az elveszíti azt a visszacsatolást, amely a döntések valóságalapját adja. A karrierút tehát nem a kódtól való eltávolodás, hanem a kód mellett más szintekre is kiterjedő figyelem.

A fejlődési út megértéséhez hasznos bevezetni két mérőszámot. Az első a **döntési hatókör**: hány ember, rendszer vagy folyamat munkáját befolyásolja egy-egy döntés. A második a **visszacsatolási idő**: mennyi idő telik el a döntés meghozatala és a döntés következményeinek megfigyelése között. A kezdő fejlesztő döntési hatóköre általában a saját kódjára terjed ki, visszacsatolási ideje pedig perc vagy óra. Az architekt döntési hatóköre csapatokra és rendszerekre terjed ki, visszacsatolási ideje hét, hónap vagy év. Aki erre a hosszabb visszacsatolásra nem készül fel tudatosan, az hajlamos a rövid távú, azonnal látható eredményeket részesíteni előnyben a lassabb, de nagyobb hatású döntésekkel szemben.

28.3.7 2.2. A fejlődés útja: a folyamat és az érettség

Az architekté válás útja nem egyetlen előléptetési pillanat, hanem folyamatos tevékenység. A gyakorlatban ez a következő mintát követi. Először a fejlesztő elkezd **architekturális hozzájárulásokat** tenni: a code review-k során nem csak a szintaktikai problémákat veszi észre, hanem a függőségeket, a tranzakcióhatárokat, a hibamódokat is. Ezt követően **dokumentációs hozzájárulások** következnek: a fejlesztő C4 diagramokat, ADR-eket ír, és ezeket a csapat megbeszélésein értelmezi. A harmadik lépés a **döntési felelősség vállalása**: az illető egy-egy kisebb architekturális döntést magára vállal, a döntést ADR-ben rögzíti, és a következményeket figyeli. A negyedik lépés a **mentorálás**: az illető elkezdi mások döntéseit segíteni, nem azzal, hogy megmondja a választ, hanem azzal, hogy jó kérdéseket tesz fel. Az ötödik lépés a **közösségi hozzájárulás**: az illető a csapat határain túlra is kiterjeszti a hatását, például belső előadásokkal, wiki-oldalakkal vagy egy belső architektúra-közösség felépítésével.

A fejlődési út kritikus pontja az a váltás, amikor az illető már nem a saját technikai teljesítményét tekinti a siker mértékének, hanem a csapat és a szervezet döntési minőségét. Ez a váltás gyakran nehéz, mert a szervezetek rendre az egyéni teljesítményt jutalmazzák, és a „mentorálás”, „tudásátadás” vagy „keretek kialakítása” kevésbé látható, kevésbé mérhető. Az érett architekt felismeri, hogy a legfontosabb hozzájárulása nem az, hogy jó döntést hoz, hanem az, hogy a szervezet a jövőben már nélküle is jó döntést tud hozni.

28.3.8 2.3. Technikai mélység és szélesség

Az architekt tudásprofilját gyakran a T-alakú modell segítségével írjuk le: a függőleges szár a mélységet, a vízszintes szár a szélességet jelöli. A **mélység** azt jelenti, hogy legalább egy területen – legyen az egy programozási nyelv ökoszisztémája, egy adatbázis-platform, egy felhőszolgáltató vagy egy biztonsági terület – az illető olyan szintű tudással rendelkezik, amely lehetővé teszi a nem triviális problémák önálló megoldását és mások tanítását. A **szélesség** azt jelenti, hogy az illető

ismeri a szomszédos területek alapfogalmait, nyelvét és főbb trade-offjait: tud beszélni adatbázis-adminisztrátorral replikációról, biztonsági mérnökkel threat modelingről, termékmenedzserrel feature-ök technikai költségéről.

A mélység és a szélesség nem egymás ellenségei, de a rendelkezésre álló idő korlátos. Az architektnek döntenie kell, hogy a tanulási idejét melyik irányba fekteti. A gyakorlatban a legtöbb sikeres architekt **két-három mély területet** és **öt-tíz széles területet** tart fenn párhuzamosan. A mély területeket nem lehet örökre fenntartani: aki architektként már nem kódol napi szinten, annak a mély területe gyorsan sekélyebbé válik. Ezért érdemes olyan mély területet választani, amely a szerephez közel van – például a döntési keret, a rendszertervezés vagy a biztonsági architektúra –, és a technikai mélységet célzottan, laborokban, pet projektekben vagy katákban karbantartani.

A szélesség fejlesztésére hasznos módszer a **szisztematikus kitekintés**: heti rendszerességgel egy-egy olyan témát feldolgozni, amely nem a napi munkához tartozik, de a döntési körben felmerülhet. Ez lehet egy felhőszolgáltató új szolgáltatásának dokumentációja, egy CNCF-projekt bemutatása, egy biztonsági szabvány követelménye, vagy egy másik iparágból származó esettanulmány. A szélesség nem abból fakad, hogy mindent kipróbálunk, hanem abból, hogy mindenről tudjuk, mikor kell szakembert bevonni.

28.3.9 3. Architekt kompetenciák és a mindennapi gyakorlat

A szerep gyakorlása során az architekt nem csak a „nagy döntéseket” hozza meg. A mindennapokban apró tevékenységek sorozatával formálja a rendszer fejlődését: a code review kommentjétől a standupon elmondott magyarázaton át a dokumentált ADR-ig. Az ehhez szükséges kompetenciákat négy csoportba sorolhatjuk.

28.3.10 3.1. Technikai ítélőképesség

A technikai ítélőképesség a kompetenciák alapja: az architekt képes felismerni, hogy egy javasolt megoldás a gyakorlatban milyen kockázatokat hordoz, milyen terhelési profilt bír el, és milyen hibamódokat produkál. Ez a képesség nem azonos a „mindent tudással”. A technikai ítélőképesség azt is jelenti, hogy az illető tudja, mit nem tud, és mikor kell más szakértőt bevonni. A 4. fejezet tárgyalta a minőségi attribútumok mérésének fontosságát; itt azt érdemes hozzátenni, hogy az architekt a becsléseit mindig megkülönbözteti a mért értékektől, és a becsléseket a visszacsatolás alapján folyamatosan kalibrálja.

A technikai ítélőképesség fejlődését mérhetővé tehetjük. Egy egyszerű gyakorlat: válasszunk öt, a múltban meghozott technológiai döntést, és értékeljük, hogy a döntés következményei igazolták-e az akkori várakozást. A hibás becslések elemzése – miért gondoltuk, hogy egy komponens skálázható lesz, miért nem lett az – többet tanít, mint száz új technológia bemutatása.

28.3.11 3.2. Kommunikáció és befolyásolás

A kommunikáció az architekt munkájának nem kiegészítője, hanem maga a munkája. Az architekt rendszeresen három különböző közeggel kommunikál: a **fejlesztőcsapattal**, ahol a technikai részletek és a kivitelezési korlátok vannak a középpontban; az **üzleti stakeholderrel**, ahol a költség, az idő és a kockázat a legfontosabb; és a **vezetéssel**, ahol a szervezeti képességek, a megfelelés és a stratégiai irányok dominálnak. Mindhárom közegben más nyelv, más részletesség és más időhorizont az eredményes. Aki ugyanazt a technikai mélységű előadást tartja a vezetésnek, mint a fejlesztőknek, az elveszíti a hallgatóságot.

A befolyásolás nem manupuláció. A befolyásolás azt jelenti, hogy az architekt úgy alakítja a döntési környezetet, hogy a döntéshozók rendelkezésére álló alternatívák minősége jobb legyen. Ennek konkrét eszközei közé tartozik:

1. **Döntési előkészítő készítése** – egy-két oldalas, ADR-formátumú dokumentum, amely a döntési lehetőségeket, a trade-offokat és a kockázatokat foglalja össze a döntéshozó nyelvén.
2. **Prototípusok és PoC-k** – a 9. fejezetben tárgyaltak szerint a PoC célja a tanulás; a befolyásolásban a PoC annak az eszköze, hogy egy-egy állítást ne csak szóban, hanem demonstrációval is alátámasszunk.
3. **Az ellenvetések megértése** – a befolyásolás kulcsa nem a saját érvek erősítése, hanem a másik fél ellenérveinek pontos megértése. Aki az ellenvetést hangosan, a másik fél megelégedésére is el tudja magyarázni, az kap jogot arra, hogy a saját érvét is hitelesen bemutassa.

28.3.12 3.3. Döntési fegyelem és visszakereshetőség

Az architekt döntési fegyelme abban mutatkozik meg, hogy a döntést nem „megoldja”, hanem **rögzíti, végrehajtja és figyeli**. Ez a fegyelem a 3. fejezet ADR-gyakorlatának kiterjesztése a karrier szintjére. A döntési fegyelem összetevői:

- **Kontextus rögzítése**: milyen körülmények között, milyen feltételezések mellett született a döntés.
- **Alternatívák megnevezése**: mit mérlegeltünk, és miért vetettük el a többit. A döntés minőségét nem a választott opció, hanem az alternatívák kezelésének minősége mutatja.
- **Következmények vállalása**: a döntés mit tesz lehetővé, mit drágít, mit zár ki.
- **Visszacsatolás tervezése**: mikor és hogyan fogjuk megnézni, hogy a döntés működött-e.

Ez a fegyelem azért része az architekt karrierjének, mert a döntések következményeivel való szembenézés az egyetlen útja a személyes kalibrációnak. Aki nem dokumentál, az nehezen tanul a hibáiból, mert a múltbeli döntés körülményei elhalványulnak.

28.3.13 3.4. Mentorálás és tanítás

A mentorálás az architekt szerep leginkább alulértékelt, mégis leghosszabb távú hozamú tevékenysége. A mentorálás nem azt jelenti, hogy az architekt megmondja, mit kell csinálni. A mentorálás azt jelenti, hogy az architekt olyan környezetet teremt, amelyben a mentorált maga találja meg a döntési alternatívákat, maga ismeri fel a következményeket, és maga rögzíti a tanulságot. A mentor szerepe a kérdések, a keretek és a visszajelzés.

A mentorálás szintjei:

- **Strukturált tanítás**: workshop, előadás, képzési anyag készítése. Ez a leginkább látható, de a legkevésbé egyénre szabott.
- **Páros munka**: közös code review, közös diagramkészítés, közös ADR-írás. A mentorált élőben látja, ahogy a mentor gondolkodik, és a mentor a mentorált gondolkodását is közvetlenül megfigyeli.
- **Döntési rituálé**: a mentorált hozza a döntést, a mentor csak a döntés utáni „review”-t végzi, visszajelzést ad, és a Döntési naplóban a leckét rögzíti.
- **Modell- és keretadás**: a mentor nem a konkrét döntést adja meg, hanem azt a keretet, amellyel a mentorált maga tudja a döntést meghozni. Ez lehet egy döntési mátrix, egy kérdéslista vagy egy ADR-sablon.

A mentorálás nem csak a mentoráltnak hasznos. A mentorálás során az architekt kénytelen a saját hallgatólagos tudását explicitálni, a saját érveit leteszteltelni egy kevésbé tapasztalt kollégán, és a saját döntési kereteit érthetővé tenni. Aki tanít, az tanul a legtöbbet.

28.3.14 4. Esettanulmány: egy csapatvezető architekt fejlődési terve

A következő esettanulmány egy közepes méretű, mintegy 60 fős fejlesztői szervezetet mutat be, ahol egy szenior fejlesztőből lett csapatvezető – akit L.-nek hívunk, de az esetben minden konkrétum helyőrző – elkezdi az architekt szerep tudatos felépítését. Az eset tanulsága, hogy a szerep nem a kinevezéssel kezdődik, hanem a döntési környezet tudatos alakításával.

28.3.15 4.1. Kiinduló állapot

L. egy terméket fejlesztő csapatot vezet. A csapat felelős a rendeléskezelő mikro-szolgáltatásért, amely más csapatokkal üzenetsoron és REST-en keresztül kommunikál. L. erős kódolási készséggel rendelkezik, a csapatban a technikai döntéseket gyakran ő hozza meg, de ezek a döntések nincsenek dokumentálva. A csapat többi tagja hozzászokott ahhoz, hogy L. „úgyis tudja a választ”. L. az utóbbi időben azt veszi észre, hogy a csapat döntései lassulnak, amikor ő szabadságon van, és a hibás döntések aránya a növekvő terhelés mellett emelkedik.

L. úgy dönt, hogy a következő kilenc hónapban három dolgot változtat meg: bevezet egy **döntési naplót**, kialakít egy **mentorálási rituálét**, és egy **személyes kompetencia-tervet** készít.

28.3.16 4.2. L. döntési naplója és a mentorálási rituálé

L. a döntési napló használatát úgy vezeti be, hogy a csapat a legközelebbi architekturális döntést – a rendeléskezelő szolgáltatás retry-logikájának újra tervezését – közösen, ADR-formátumban rögzíti. A döntés során nem L. mond be egyetlen megoldást, hanem a csapat két alternatívát vet össze: a beépített retry-keretrendszert és a saját, kézzel írt exponenciális backoffot. L. a döntési mátrixot a csapat elé teszi, és a csapat maga tölti ki a szempontokat: a beépített megoldás kevesebb kódot igényel, de kevésbé testreszabható; a saját megoldás nagyobb kontrollt ad, de több üzemeltetési teherrel jár. A csapat a beépített megoldás mellett dönt, és az ADR-ben rögzíti a kontextust, a döntést, az elvetett alternatívát és a következményeket.

L. a döntést követően bevezeti a **mentorálási rituálét**: havonta egy órában a csapat egy tagja – nem L. – vezeti a döntési dokumentumok review-ját. A kiválasztott csapattag felkészül egy-egy múltbeli ADR-ből, és a csapat előtt reprodukálja a döntési érvelést, a trade-offokat és a későbbi visszacsatolást. L. szerepe ilyenkor csupán annyi, hogy eldönti: a reprodukálás hű-e az eredeti kontextushoz, vagy valamit a csapat idő közben félreértett.

28.3.17 4.3. A visszacsatolás és a mérés

L. a kilencedik hónap végén a következő méréseket végzi el:

- **Döntési átfutási idő**: a csapat architekturális döntéseinek átlagos átfutási ideje a korábbi 11 napról 5 napra csökkent. A mérés a hibakereső rendszerben rögzített döntési jegyek időbélyegén alapul, a mérési ablak 9 hónap.
- **Döntési dokumentáltság**: a döntési naplóban nyilvántartott döntések aránya a csapatban 10%-ról 90%-ra nőtt. A 90% azokra a döntésekre vonatkozik, amelyeket a csapat a „jelentős” kategóriába sorolt.

- **Mentorált döntési reprodukció**: a havi review-k során a csapattagok a múltbeli döntési érvelést átlagosan 4,2-es pontszámon (1–5 skála) tudták hűen rekonstruálni, szemben a kezdeti 2,8-es átlaggal. A pontozás három szempontot vesz figyelembe: a kontextus hűsége, az alternatívák felidézése, a következmények azonosítása.

L. a visszacsatolást a 3. fejezet ADR-gyakorlatának mintájára értékeli. A mérés nem laboratóriumi precizitású, de a csapat számára a trend érzékelhető: a döntések gyorsabbá, dokumentáltabbá és reprodukálhatóbbá váltak. L. a személyes kompetencia-tervét ennek alapján frissíti.

28.3.18 5. Laborgyakorlat: személyes kompetencia- és karrierterv

Ez a labor az egység legfontosabb átadandó eredménye: egy **reprodukálható dokumentum**, amely a saját architekt fejlődési útra vonatkozik. A dokumentumban a mért és a várt eredményt el kell különíteni, a kontextust rögzíteni kell, és a döntési naplóval nyomon kell követni a karrierrel kapcsolatos döntéseket.

28.3.19 5.1. Kiinduló állapot

A gyakorlat előtt készíts egy jegyzetfüzetet vagy dokumentumot, amelynek a következő szakaszai vannak:

1. **Kontextus**: a jelenlegi munkakör, a döntési hatókör (mely rendszerekre, csapatokra, folyamatokra terjed ki a döntési befolyásod), a visszacsatolás tipikus időtartama.
2. **Kompetenciafelmérés**: a 3. fejezetben definiált kompetenciacsoportok – technikai ítélőképesség, kommunikáció, döntési fegyelem, mentorálás – mentén egy 1–5 skálán jelöld, hol látod magad. A pontozáshoz használj konkrét viselkedésleírást, ne csak érzést. Például a „technikai ítélőképesség” 3-as szintje: „a múltbeli döntések felében utólag igazolni tudtam a választás helyességét; a hibás döntések okát a felénél azonosítottam”.
3. **Célok**: legfeljebb három, időhöz kötött cél. A cél ne legyen „jobb kommunikáció”; a cél legyen például „9 hónap alatt a csapatom architekturális döntéseinek 80%-át ADR-ben rögzítjük, és ebből legalább 50%-ot a csapat más tagja ír”.
4. **Döntési napló**: a karrier-terv során felmerülő fontosabb döntéseket itt rögzíted. A napló soraiban a kontextus, a döntés, az alternatívák és a lecke szerepel.

28.3.20 5.2. Elvárt eredmény

A laborgyakorlat sikeres, ha a dokumentum a következőket tartalmazza:

- **Legalább egy fejlesztési cél**, amely a döntési hatókör növelésére irányul, és a cél mellett szerepel a mérési mód és a mérési ablak.
- **Legalább egy mentorálási terv**, amelyben szerepel, hogy ki a mentorált vagy a célközönség, milyen rituálét vezetsz be, és hogyan méred a mentorálás hatását. A mentorált lehet egy csapattag, egy junior fejlesztő, vagy egy belső közösség, amelyet az architektúra-elvekről tanítasz.
- **Legalább egy döntési bejegyzés** a döntési naplóban, amely a karrier-terv során felmerült választást rögzíti – például azt, hogy milyen technológiát tanulj, milyen konferenciára menj, vagy milyen architekt szerep-variáns felé mozdulj.
- **A mért és a várt eredmény elkülönítése**. A tervben külön jelölés szükséges arra, hogy mit vársz előre, és mit fogsz mérni utólag.

28.3.21 5.3. Ellenőrzési szempontok

A dokumentum akkor jó, ha az alábbi ellenőrzéseken átmegy:

- **Kontextus-ellenőrzés**: a dokumentumból egy külső olvasó is meg tudja állapítani, hogy a célok milyen környezetben értelmesek. Ha a cél „a csapatom döntéseit felgyorsítani”, akkor a kontextusból ki kell derülnie, mi a jelenlegi sebesség, milyen intézményi akadályok vannak, és mi a mérés módja.
- **Mérhetőség-ellenőrzés**: a célhoz tartozó mérőszámnak van mértékegysége és mérési ablaka. A „jobb kommunikáció” nem mérhető; a „a döntési dokumentumok 80%-át a csapat többi tagja is tudja rekonstruálni 4-es vagy magasabb pontszámmal” már igen.
- **Visszakereshetőség-ellenőrzés**: a döntési napló bejegyzései visszamutatnak a célokra. Ha a naplóban eltérsz a tervtől, az önmagában nem hiba; a hiba az, ha a napló nem adja meg, miért tértél el, és mi a lecke.
- **Visszaállítási út**: a dokumentumban szerepel egy rövid bekezdés arról, hogy mi történik, ha a cél túl ambiciózusnak bizonyul. A visszaállítási út lehet a cél leszállítása, a mérési ablak kitolása, vagy a cél elvetése egy másik, fontosabb cél javára. Ez az architekt munkájában is fontos: a döntési keret akkor igazán érett, ha a visszalépést is meg tudja tervezni.

28.3.22 5.4. A gyakorlat során elkövethető hibák

Az első tipikus hiba a **címke nélküli célok** használata: „szeretnék senior architekt lenni” – ez nem cél, hanem vágy. A célhoz mérés, időablak és visszacsatolás kell. A második hiba a **minden irányú fejlődés** tervezése: aki egyszerre akarja a mély tudását bővíteni, a széles tudását építeni, a kommunikációját fejleszteni és a mentorálást elkezdeni, az gyakran egyikben sem ér el változást. A harmadik hiba a **mért és várt összekeverése**: a tervben előre írt „várható eredmény” utólag nem válhat „mért eredménnyé” anélkül, hogy erre külön jelölés ne utalna.

28.3.23 6. Ellenőrzőlista

A fejezet lezárásaként az alábbi ellenőrzőlistával ellenőrizheted, hogy az architekt szerep, karrier és mentorálás témakörét a gyakorlatban is alkalmazni tudod:

1. **Szerep- és kontextusazonosítás**: Meg tudom nevezni, hogy az adott szervezetben az architekt szerep melyik változata (enterprise, solution, software, systems, cloud, data) dominál, és hogy a szerep centralizált, beágyazott vagy hibrid modellben működik.
2. **Döntési hatókör és visszacsatolás**: Fel tudom mérni a saját döntési hatóköröm határát és a tipikus visszacsatolási időt, és ezt a kontextusomban tudom értelmezni.
3. **Kompetenciafelmérés**: A négy kompetenciacsoport (technikai ítélőképesség, kommunikáció, döntési fegyelem, mentorálás) mentén van egy viselkedési szintekkel alátámasztott önértékelésem.
4. **Célok méréssel**: A fejlődési céljaimat mértékegységgel, mérési ablakkal és visszaállítási úttal tudom leírni.
5. **Mentorálási terv**: Van legalább egy mentorálási tervem, amelyben a rituálé, a mentorált és a hatás mérése egyaránt szerepel.
6. **Döntési napló**: A karrier-tervemmel kapcsolatos döntéseimet a döntési naplóban rögzítem, és a bejegyzéseket a célokhoz vissza tudom kötni.

28.3.24 7. Önellenőrző kérdések

1. Miben különbözik az architekt **szerepe** az architekt **címétől**? Adj egy példát arra, amikor a cím van meg, de a szerep nincs, és egy példát arra, amikor a szerep megvan, de a cím nincs.
2. Hogyan változik a döntési hatókör és a visszacsatolási idő az architekt karrierút során? Milyen kockázatokat hordoz, ha az architekt nem veszi figyelembe a hosszabb visszacsatolási időt?
3. Mi a különbség a technikai mélység és a szélesség között, és miért érdemes az architektnek mindkettőt fenntartania?
4. Nevezz meg három konkrét eszközt, amellyel az architekt befolyásolja a csapat döntéseit anélkül, hogy a döntési jogkört magához vonná. Mikor érdemes ezeket az eszközöket használni?
5. Miért tekintjük a mentorálást az architekt karrierjének stratégiai tevékenységének, nem pedig „járulékos” feladatnak? Támasszd alá a választ a fejezetben tárgyalt visszacsatolási példával!

28.4 Kultúra, pszichológiai biztonság és döntéshozatal

Az előző alfejezetben az építész szerepét, karrierívét és mentori felelősségét vizsgáltuk. Most egy olyan rétegre lépünk, amely minden technikai döntést áthat, mégis gyakran láthatatlan marad a dokumentációban: a szervezeti kultúrára, a pszichológiai biztonságra és ezek hatására a döntéshozatal sebességére. Ha egy jó architektúra tervrajz, akkor a kultúra a talaj, amelyre épül. Hiába rajzolunk tökéletes komponenshatárokat, ha a szervezetben senki nem meri jelezni, hogy a határ rossz helyen húzódik, vagy ha a döntések indokolatlanul lassulnak a bizalmatlanság miatt. Ebben a fejezetben azt járjuk körbe, hogyan lehet a kultúrát diagnosztizálni, a pszichológiai biztonságot megteremteni, és a döntéshozatal tempóját a döntés súlyához igazítani.

28.4.1 A szervezeti kultúra hatása a technikai döntésekre

A szervezeti kultúra nem csupán HR-fogalom; mérhető és megfigyelhető hatással van arra, hogy milyen architektúrákat választunk, és hogy azok mennyire lesznek fenntarthatók. A kultúra meghatározza, hogyan kezeljük a hibákat, hogyan osztjuk meg az információt, és mennyire merünk kísérletezni. A szoftverarchitektúrában az egyik legismertebb és legjobban használható keretrendszer a szervezeti kultúrák leírására a Westrum-féle tipológia, amely három markáns működési módot különböztet meg: a patologikus, a bürokratikus és a generatív kultúrát.

A **patologikus** kultúrában a hatalom és a félelem dominál. Az információ birtoklása hatalmi kérdés; a rossz hírt hozó követet gyakran „lelövik”. Ennek közvetlen technikai következménye, hogy a rendszerarchitektúra silókra töredezik. A csapatok elkezdenek védelmi falakat építeni a saját komponenseik köré, hogy elkerüljék a felelősségre vonást. A függőségek elrejtőznek, mert senki sem meri dokumentálni, hogy az X szolgáltatás valójában közvetlenül olvas a másik csapat adatbázisából. Az ilyen kultúrában jellemzővé válik a „blame-driven development”: minden döntés elsődleges célja nem a felhasználói érték, hanem a lehetséges vádaskodás elhárítása. A túlzottan merev, túlbonyolított architektúrák – például egy feleslegesen mély mikroszolgáltatás-háló – gyakran nem technikai okokból születnek, hanem azért, mert a csapatok így próbálják magukat elszigetelni egymástól.

A **bürokratikus** kultúra szabálykövető. Itt a döntéseket nem a félelem, hanem a procedúra vezérli. Az architektúra-governance ebben a közegben gyakran válik öncélúvá: hosszú űrlapok,

előre meghatározott sablonok és jóváhagyási láncok jellemzik. A bürokratikus kultúrában a fejlesztők hajlamosak a folyamatokat inkább kijátszani vagy formálisan kipipálni, mintsem belső minőségi iránytűként használni. Az architekturális döntések így gyakran a legkisebb ellenállás felé tolódnak: nem azt választjuk, ami a legjobb, hanem azt, ami a legkönnyebben átmegy a bizottságon.

A **generatív** kultúra ezzel szemben a teljesítményre és a küldetésre összpontosít. Az információ áramlik, a hibákból tanulás zajlik, és a döntéseket ott hozzák meg, ahol a legtöbb kontextuális tudás van. Technikai értelemben ez a kultúra támogatja leginkább az evolúciós architektúrát: a csapatok mernek refaktorálni, mert van fitness function hálójuk, és a hiba nem bűn, hanem adat. A generatív kultúra nem egyenlő a káosszal vagy a sztenderdek hiányával; éppen ellenkezőleg, itt a sztenderdeket (például az ADR-eket) azért tartják be, mert azok segítenek, nem pedig azért, mert kötelezőek.

A kultúra hatással van a **technológiai választásokra** is. Egy félelemalapú szervezetben a csapatok hajlamosak a „senkit nem rúgnak ki, amiért az IBM-et választotta” mentalitásra: nagy, ismert, de gyakran túlterhelt platformokat választanak, hogy a felelősséget a beszállítóra háríthassák. Egy generatív kultúrában ugyanaz a csapat nagyobb valószínűséggel választ egy kisebb, jobban illeszkedő, nyílt forráskódú komponenst, mert tudja, hogy a döntés kockázatát nyíltan kezelheti, és ha téved, az tanulság lesz, nem pedig karrier-vég.

28.4.2 Pszichológiai biztonság és hibakultúra

A pszichológiai biztonság Amy Edmondson kutatásai nyomán került be a szervezeti gondolkodás főáramába. A fogalom azt írja le, hogy a csapattagok mennyire érzik úgy, hogy vállalhatnak interperszonális kockázatot: kérdezhetnek, jelezhetik, ha valamit nem értenek, vagy ha hibát találnak anélkül, hogy ezért megszégyenítés, büntetés vagy elutasítás járna. Fontos hangsúlyozni, hogy a pszichológiai biztonság nem lazaságot vagy a számonkérhetőség hiányát jelenti. Éppen ellenkezőleg: magas pszichológiai biztonság mellett van igazán lehetőség **magas elvárásokat** támasztani, mert a csapattagok mernek őszintén beszélni a problémákról, amikor azok még kicsik.

Az architektúrában a pszichológiai biztonság közvetlenül mérhető a **tervezési megbeszélések** minőségén. Generatív környezetben egy junior fejlesztő is meg meri kérdezni: „Nem értem, miért van szükségünk itt aszinkron üzenetküldésre, amikor a hívó félnek azonnali válasz kell?”. Patologikus környezetben ugyanez a kérdés el sem hangzik, mert a kérdező fél attól tart, hogy inkompetensnek tűnik, vagy ami rosszabb, hogy megsérti a döntést hozó szenior kollégát. A kérdés elmaradása nemcsak pillanatnyi kényelmetlenséget spórol meg, hanem konkrét architekturális adósságot halmoz fel: a rendszerbe bekerül egy olyan minta, amely nem illik a probléma természetéhez, és ezt csak hetekkel később, a produkciós incidens során fedezik fel.

A **hibakultúra** a pszichológiai biztonság egyik legfontosabb aspektusa az üzemeltetés és az architektúra találkozásánál. A hagyományos, büntető hibakultúrában a postmortem (incidens utáni elemzés) célja a felelős megtalálása. Ennek következménye, hogy az incidensek adatai torzulnak: a résztvevők aluljelentik a hibáikat, a naplókat nem őrzik meg, a konfigurációkat nem verziózzák, hiszen minden nyom potenciális bizonyíték lehet ellenük. Ezzel szemben a **blameless postmortem** – a vádaskodásmentes utóelemzés – arra a felismerésre épül, hogy a legtöbb hiba rendszerszintű okokra vezethető vissza: rosszul megtervezett interfész, hiányzó guardrail, nem egyértelmű deploy folyamat. Az architektúra a blameless postmortemekből nyeri a legértékesebb bemeneteit: ha három egymástól független csapat is ugyanazt a hibát követi el, az nem három emberi hiba, hanem egy rossz architekturális döntés vagy hiányzó platformképesség.

A hibakultúra technikai megtestesülése az **error budget** (hiba-költségvetés) fogalma. Az error budget az SLO-ból származtatott mennyiség: ha egy szolgáltatás rendelkezésre állási SLO-ja 99,9%, akkor az error budget az adott mérési ablakban a $1 - 0{,}999 = 0{,}001$ arányú megengedett hibaidő. A hibakultúra szempontjából az error budget az a mechanizmus, amely a pszichológiai biztonságot üzemeltetési döntéssé fordítja le. Amíg a csapatnak van error budgetje, addig a vezetés nem állítja le a release-eket, és a csapat mer kockázatot vállalni. Ha a budget elfogy, a csapatnak le kell állnia, de ez nem büntetés, hanem az SLO-val kötött szerződés. Ez a mechanizmus kiveszi a döntésből a személyeskedést: nem a főnök dönt arról, hogy „felelőtlen voltál", hanem a rendszer jelzi, hogy a felhasználói élmény már a vállalt küszöb alá csökkent.

A pszichológiai biztonság hiánya közvetlenül rontja a **dokumentáció minőségét** is. Gondoljunk csak az ADR-ekre. Az ADR akkor jó, ha a jövőbeli olvasó megérti belőle a döntés okát és a feladott alternatívákat. Ehhez azonban az kell, hogy a döntést hozó csapat őszintén le tudja írni: „Ezt az alternatívát azért vetettük el, mert a csapatunkból senki sem ért hozzá, és nem volt kapacitásunk betanulni". Egy félelemalapú kultúrában az ilyen mondat megírhatatlan, mert a szerző attól tart, hogy az őszinte gyengeség bevallása a következő teljesítményértékelésnél ellene fordul. Így az ADR tele lesz kozmetikázott, hamis műszaki indoklással, ami a későbbi döntéshozókat teljesen félrevezeti.

28.4.3 A döntések lassítása és gyorsítása

A szervezeti kultúra és a pszichológiai biztonság közvetlenül befolyásolja, hogy milyen sebességgel születnek meg az architekturális döntések. A cél nem a „minél gyorsabb" vagy a „minél alaposabb" döntés, hanem a **döntés súlyához kalibrált sebesség**. Az egyik legfontosabb heurisztika ezen a téren a megfordíthatóság elve. A döntéseket két nagy csoportra bonthatjuk:

1. **Megfordítható (reverzibilis) döntések:**
    - **Definíció:** Olyan döntések, ahol a visszafordítási költség alacsony, és a tévedés kockázata nem okoz kritikus üzleti vagy adatvesztést. Ilyen például egy új rate limiter könyvtár bevezetése, egy belső elnevezési konvenció megváltoztatása, vagy egy cache réteg beillesztése a service előtt.
    - **Hogyan gyorsítsuk?** A megfordítható döntéseket a lehető legközelebb kell vinni a kódhoz és a csapathoz. Itt a folyamatos delivery elve érvényesül: a hibát nem hosszas elemzéssel, hanem gyors visszaállítással (rollback) kezeljük. A pszichológiai biztonság itt is kritikus: ha a csapat fél a hibától, akkor még a könnyen visszafordítható döntést is hosszú elemzési körökbe bonyolítja, mert a hiba társadalmi költsége magasabb, mint a technikai költsége. A döntés tehát nem technikai okból lassul, hanem kulturális okból.
2. **Nehezen megfordítható (irreverzibilis) döntések:**
    - **Definíció:** Olyan döntések, ahol a visszafordítási költség rendkívül magas, vagy a döntés meghatározza a rendszer hosszú távú evolúcióját. Ilyen például az elsődleges adatbázistípus kiválasztása (poliglott perzisztenciáról nehéz visszatérni), a publikus API szerződés kialakítása (ha kikerül a külső partnerekhez, már nem módosítható szabadon), vagy egy komplex tranzakció elosztott megvalósítása (amely a konzisztencia-modellt alapvetően határozza meg).
    - **Hogyan lassítsuk?** A nehezen megfordítható döntéseknél a **lassítás minőségi időt** jelent, nem bürokratikus várakozást. A lassítás itt azt jelenti, hogy időt adunk a feltárásra, a PoC-ra, a kockázatok explicit dokumentálására és a trade-off analízisre. Az ADR (Architecture Decision Record) itt teljesedik ki: a döntési napló nem arra

való, hogy megakadályozza a döntést, hanem hogy lelassítsa annyira, hogy a kontextus és az alternatívák jól láthatóak legyenek.

A döntések lassításának és gyorsításának egyik leggyakoribb hibája, hogy a szervezetek **minden döntést egyformán kezelnek**. A túlzottan merev architektúra-governance (minden apró változtatáshoz Architectural Review Board jóváhagyás kell) a megfordítható döntéseket is lelassítja. Ennek kettős negatív hatása van: egyrészt a csapatok feleslegesen várnak, másrészt a fejlesztők megtanulják, hogy a governance csak akadály, ezért elkezdik kijátszani (például az apró módosításokat „hotfixnek" nevezik, hogy ne kelljen engedély). Ezzel szemben a túlzott lazaság a nehezen megfordítható döntéseknél vezet katasztrófához: a csapat egy péntek délutáni megbeszélésen, egyetlen trade-off elemzés nélkül dönt egy olyan adatbázis-séma mellett, amelyet aztán évekig nem tudnak kiirtani.

A döntéshozatal sebességének kalibrálásához használható egy egyszerű mátrix:

- **Kis kockázat, könnyen megfordítható:** Döntsön a csapat, dokumentáció minimálisan szükséges, a hiba a tanulási folyamat része.
- **Nagy kockázat, nehezen megfordítható:** Kötelező ADR, trade-off analízis, PoC, és az érintett külső rendszerek tulajdonosainak bevonása. A döntést lassítani kell, de a folyamatnak tiszta végponttal kell rendelkeznie.
- **Kis kockázat, nehezen megfordítható:** Ritka, de előfordul (például egy publikus API egy apró, de kényes részletének rögzítése). Itt a fő szempont a jóváhagyás megszerzése a megfelelő hatóságtól, de a folyamatnak rövidnek kell lennie.
- **Nagy kockázat, könnyen megfordítható:** Itt a kockázat kezelése a gyors kísérletezés, de szükséges a monitoring, hogy a hiba ne maradjon észrevétlen.

A pszichológiai biztonság és a döntési sebesség összekapcsolódik. A gyors döntéshez bizalom kell: bízom abban, hogy a kollégám jól döntött, és ha hibázik, akkor azt nem ellenem fordítja senki. A lassú döntéshez is biztonság kell: a csapattagoknak érezniük kell, hogy a trade-off elemzésben felvetett ellenvélemények nem sértik a döntés tulajdonosát. Ha a vezető azt kérdezi: „Van valakinek ellenvetése?", és a szobában csend van, az gyakran nem egyetértést, hanem félelmet jelent.

28.4.4 Gyakorlati alkalmazás: kultúra- és döntési diagnózis

Az itt bemutatott eszközök akkor érnek valamit, ha egy valós vagy szimulált szervezeti helyzetre alkalmazzuk őket. A következő labor célja, hogy a fenti fogalmakat egy ellenőrizhető példán keresztül rögzítse.

28.4.4.1 Labor: kultúra- és döntési diagnózis készítése

Cél: Egy adott csapat vagy szervezeti egység kultúrájának felmérése, és annak vizsgálata, hogy ez a kultúra hogyan befolyásolja a technikai döntések sebességét és minőségét.

Kiinduló állapot: Tételezzünk fel egy fiktív, de reális helyzetet. A „NimbleMart" e-kereskedelmi vállalatnál a „Payment Gateway" csapat felelős a fizetési tranzakciók feldolgozásáért. A csapatban 6 fejlesztő, 1 tech lead és 1 szenior architekt dolgozik. Az utóbbi 3 hónapban a csapat két jelentős technikai döntést hozott:

1. **Döntés A:** A monolitikus tranzakciókezelő modult kettébontották: a kártya-tokenizáló szolgáltatást leválasztották a tranzakció-validáló szolgáltatásról. Ezt a döntést egyetlen, 45 perces megbeszélésen hozták meg, ahol a szenior architekt bemutatta a tervet, és a csapat nem tett fel ellenvetést. A döntést ADR-ben nem rögzítették, csak egy Slack-üzenetben.

2. **Döntés B:** Egy új, nyílt forráskódú naplózási könyvtár bevezetése a szolgáltatásokban. Ezt a döntést három héten át vitatták, többször is kérték a biztonsági csapat jóváhagyását, és végül egy 12 oldalas elemző dokumentum készült róla, amelyben a csapat leszögezte, hogy a könyvtár nem használható, mert „nem elég ismert a piacon".

Feladat:

1. **Kultúra-diagnózis:**
   - Készíts egy rövid felmérést a csapat számára a Westrum-modell alapján. Tegyél fel legalább 6 állítást, amelyekre a csapattagok 1-től 5-ig terjedő skálán válaszolnak. Például: „Ha hibát találsz a rendszerben, nyugodtan jelezheted a felettesednek anélkül, hogy negatív következményektől tartanál." (A laborban ezt a felmérést most magadnak kell elvégezned, a fiktív csapat feltételezett válaszai alapján.)
   - Rögzítsd az eredményeket egy táblázatban. A „mért" eredmény legyen a fiktív csapattagoktól feltételezett pontszámok átlaga, a „várt" eredmény pedig az, amit te egy generatív kultúrájú csapat esetén elvárnál (például 4,0 feletti átlag).
2. **Döntési sebesség-elemzés:**
   - Elemezd a Döntés A-t és a Döntés B-t a megfordíthatóság szempontjából.
   - Töltsd ki a következő táblázatot:

Döntés	Leírás	Megfordítható-e?	Kockázat (magas/alacsony)	Indokolt sebesség	Valós sebesség	Eltérés indoklása (kultúra-hipotézis)
A	Kártya-tokenizáló leválasztása	Nem (adat-modell, PCI scope)	Magas	Lassú, ADR-rel	Nagyon gyors	A szervezetben a „gyors" döntés a bátorság jele; a lassítás a gyengeségé.
B	Naplózó könyvtár bevezetése	Igen (kivehető)	Alacsony	Gyors, kísérleti	Nagyon lassú	A félelem a hibától („ismeretlen technológia") és a blame-kultúra miatt a csapat túlbiztosít.

3. **ADR készítése:**
   - Válaszd ki az egyik döntést, és készíts hozzá egy utólagos ADR-t a könyvben bemutatott formátumban (Kontextus, Döntés, Alternatívák, Következmények, Állapot). Az ADR-ben kifejezetten szerepeltesd a pszichológiai biztonságra és a döntési sebességre vonatkozó megfigyeléseket is.

- Például az ADR „Kontextus” részében írd le: „A döntést 45 perc alatt hoztuk meg. A megbeszélésen nem hangzott el alternatíva, ami a csapatkultúra miatt nem feltétlenül jelenti az egyetértést. A csapatban a vezetői ellenvéleményt kockázatosnak érzik.”

Elvárt eredmény: Egy verziózott dokumentum (például `culture_decision_diagnosis.md`) a következő tartalommal:

1. **Kontextus:** A fiktív csapat leírása, a mérési időpont, a módszer.
2. **Kultúra-felmérés:** A kérdőív, a feltételezett válaszok, a mért és a várt átlagok.
3. **Döntési mátrix:** A fenti táblázat kitöltve.
4. **Utólagos ADR:** A kiválasztott döntés részletes leírása.
5. **Ellenőrzési szempontok:** Egy lista, amely alapján a diagnózis helyessége ellenőrizhető (például: „A felmérés eredménye diszkrét adatpontokból számított átlag, nem pedig benyomás.”).

Visszaállítási út: Mivel a labor fiktív adatokkal dolgozik, visszaállítás nem szükséges. Éles környezetben az ilyen diagnózist anonim módon kell elvégezni, és az eredményeket a csapattal közösen kell értelmezni, biztosítva, hogy az adatok nem használhatók fel az egyének ellen.

28.4.5 Tipikus hibák és a helyes megoldás

A fejezet lezárásaként érdemes explicit módon is megfogalmazni azokat a csapdákat, amelyekbe az architektúrát kultúrával összekötő mérnökök leggyakrabban esnek.

1. Hiba: A kultúrát figyelmen kívül hagyni a technikai tervezésnél. * **Tünet:** Az architektúra-dokumentum tökéletes, a C4 diagramok szépek, a minőségi attribútumok jól mérték, de a rendszer mégsem működik jól, mert a csapatok nem a diagram szerint kommunikálnak. * **Helyes megoldás:** A kontextus-szintű C4 diagramhoz hasonlóan készítsünk „szervezeti kontextus” elemzést is. Ki kivel beszél? Hol vannak a kommunikációs szakadékok? Ha a szervezeti diagram nem egyezik az architektúra diagrammal, akkor az architektúra nem fog megvalósulni.

2. Hiba: A pszichológiai biztonságot összekeverni a lazasággal. * **Tünet:** A csapatban nincs konfliktus, mindenki mosolyog, de a rendszer minősége romlik. A code review-ok felületesek, mert senki sem meri megkérdezni: „Miért van ez így?”. * **Helyes megoldás:** A pszichológiai biztonságot magas teljesítmény-elvárásokkal párosítani. A cél nem az, hogy senki ne érezze rosszul magát, hanem az, hogy a bizonytalanságot és a hibát lehessen nyíltan kezelni. A jó code review nem barátságtalan, de szigorú.

3. Hiba: A döntések lassításának technokratikus eltúlzása. * **Tünet:** Minden döntésnél hosszú ADR-t kérnek, a csapat a dokumentációt a munka elvégzése előtti kötelező rosszként éli meg, és a governance-bizottság szűk keresztmetszetté válik. * **Helyes megoldás:** A döntési folyamatot a döntés súlyához igazítani. Legyen egyértelmű, hogy mi tekinthető „könnyű” döntésnek, és mi az, amihez valóban ADR kell. A könnyű döntéseknél a **döntési napló** (decision log) lehet egyszerűbb, mint a teljes ADR: egy egyszerű bejegyzés arról, hogy mit döntöttünk, miért, és ki a felelős.

4. Hiba: A hibakultúra félreértése: a blameless nem felelőtlen. * **Tünet:** Az incidensek után senki sem vonódik felelősségre, de a hibák ismétlődnek, mert a rendszerszintű tanulságokat nem építik vissza az architektúrába. * **Helyes megoldás:** A blameless postmortem után **action itemeket** kell meghatározni, és azokat vissza kell csatolni a fitness functionökbe vagy a rendszer architektúrájába. A felelősség nem a személyre, hanem a rendszerre irányul, de a rendszer javítása konkrét technikai munka.

28.4.6 Önellenőrző kérdés

Gondolj egy nemrégiben a saját környezetedben meghozott technikai döntésre. Hogyan kategorizálnád a döntést a megfordíthatóság szempontjából? A döntés meghozatalának sebessége összhangban volt-e a kockázattal? Ha nem, mi okozta az eltérést: technikai bizonytalanság, vagy a csapatban uralkodó félelem, bürokrácia, esetleg túlzott magabiztosság?

28.4.7 Ellenőrzőlista a fejezethez

- ☐ Meg tudom magyarázni, mi a különbség a patologikus, a bürokratikus és a generatív kultúra között, és hogyan hatnak ezek a technológiai választásokra.
- ☐ Meg tudom fogalmazni, mi a pszichológiai biztonság, és miért nem egyenlő a lazasággal.
- ☐ Fel tudom ismerni, ha egy döntési folyamat a félelem miatt lassul vagy gyorsul indokolatlanul.
- ☐ Tudom, hogyan kell a döntések sebességét a döntések megfordíthatóságához kalibrálni.
- ☐ Képes vagyok egy kultúra- és döntési diagnózist elkészíteni, amelyben a mért és a várt eredményeket elkülönítve rögzítem.

Ebben a fejezetben a szoftverarchitektúra „puha” oldalát vizsgáltuk: a kultúrát, a biztonságot és a döntési sebességet. Láttuk, hogy ezek a tényezők nem külső adottságok, hanem olyan tervezési paraméterek, amelyeket az építésznek éppúgy figyelembe kell vennie, mint a választott protokollt vagy az adatbázis-sémát. A következő fejezetben ezt a szemléletet visszük tovább valós rendszerek elemzésére: az esettanulmány-elemzés módszertanánál az itt megismert kulturális lencsékkel is értékelni fogjuk, hogy egy adott architektúra miért működik vagy vall kudarcot.

VII. RÉSZ – Esettanulmányok, iparágak, jövő és zároprojekt

29 Esettanulmányok és architektúra-katák

29.1 Esettanulmány-elemzés módszertana

29.1.1 A valós rendszerek architektúrájának rendszerezett elemzése

Az architektúra-oktatás egyik leggyakoribb csapdája, hogy a hallgatók tökéletesre rajzolt, kontextus nélküli diagramokat tanulmányoznak, amelyek soha nem találkoztak éles forgalommal, szervezeti politikai nyomással vagy költségvetési korlátozással. A valós rendszerek – legyen szó egy nagy forgalmú e-kereskedelmi platformról, egy banki rendszerről vagy egy felhőszolgáltató vezérlősíkjáról – mindig kompromisszumok halmazát testesítik meg, amelyeket egy adott időpontban, korlátozott információ birtokában, konkrét üzleti kényszerek alatt hoztak meg. Ha ezeket a rendszereket anélkül elemezzük, hogy megértenénk ezt a döntési rétegződést, akkor nem tanulunk architektúrát, csak diagramokat másolunk.

Ez a fejezet azt a módszertant mutatja be, amellyel a valós rendszerek architektúrája rendszerezett módon elemezhető. A módszer középpontjában négy egymásra épülő vizsgálati szint áll: a kontextus feltárása, a döntések rekonstruálása, a következmények értékelése és a tanulságok kivonása. A cél nem egy múltbeli döntés minősítése, hanem olyan gondolkodásmód kialakítása, amely a jelenlegi és jövőbeli döntésekben is alkalmazható.

29.1.2 Miért van szükség külön elemzési módszertanra?

A szoftverarchitektúra dokumentációja gyakran elszakad a rendszer valóságától. Az ADR-ek néha csak a végső döntést rögzítik, a feladott alternatívákat nem; a C4 diagramok a célállapotot mutatják, nem a jelenlegi szerkezetet; a konferencia-előadások az eredményt ünneplik, a köztes kudarcokat pedig jól nevelt csend övezi. Amikor egy külső elemző – legyen az egy másik csapat architektje, egy műszaki due diligence-t végző tanácsadó vagy egy tanuló – megpróbál megérteni egy működő rendszert, három alapvető nehézséggel találkozik.

Az első nehézség az **információhiány**. A rendszer működésének teljes képéhez ritkán férünk hozzá: a forráskód részleges, a konfiguráció szétszórt, a döntési naplók hiányosak, a közreműködők emlékezete pedig torzít. A második nehézség az **információtúlterhelés**. Egy érett rendszer több ezer komponensből, kapcsolatból és konfigurációs paraméterből áll; ezek mindegyikének részletes elemzése éveket venne igénybe. A harmadik nehézség a **retrospektív torzítás** (hindsight bias): utólag minden döntés könnyűnek tűnik, mert már ismerjük a kimenetelét. Ez a torzítás arra csábít, hogy a múltbeli döntéshozókat hozzá nem értéssel vádoljuk, holott ők korlátozott információ birtokában, valós időbeli nyomás alatt cselekedtek.

A rendszerezett elemzési módszer ezekre a nehézségekre ad választ. Nem az a célja, hogy minden részletet megismerjünk – ez lehetetlen –, hanem hogy a rendelkezésre álló információkból a lehető legmegbízhatóbb következtetéseket vonjuk le, miközben zavaróan tudatában maradunk a tudásunk határainak. A módszer lépései egymásra épülnek, de a gyakorlatban iteratívak: a későbbi lépésekben felfedezett tények visszamenőleg árnyalhatják a korábbi lépések megállapításait.

29.1.3 Az első lépés: a kontextus rekonstruálása

A kontextus rekonstruálása a legfontosabb és leggyakrabban kihagyott lépés. Egy architektúra soha nem önmagában létezik, hanem egy adott üzleti, technikai és szervezeti környezet terméke. A kontextus elemzése azért előzi meg a döntések vizsgálatát, mert a döntések racionalitása csak ebben a keretben értelmezhető.

A kontextus rekonstruálásához három dimenziót kell vizsgálnunk: az időbeli elhelyezést, az üzleti környezetet és a technikai kényszereket.

29.1.4 Az időbeli elhelyezés

Minden architektúra egy időponthoz kötött. Azok a technológiák, amelyek ma magától értetődőek, öt évvel ezelőtt kockázatosnak számítottak. Egy 2015-ben született mikroszolgáltatás-architektúra mögött akkor merész döntések álltak: a konténer-orchestráció éretlen volt, a szolgáltatáshálók nem léteztek, a megfigyelhetőségi eszközök kezdetlegesek. Ugyanez az architektúra ma már konzervatív választásnak tűnik, a döntés időpontjában azonban jelentős bizonytalanságot hordozott.

Az időbeli elhelyezéshez tartozik a rendszer aktuális életciklus-fázisának azonosítása is. Egy zöldmezős indítás, ahol a piacra kerülési idő a legfontosabb szempont, egészen más döntéseket indokol, mint egy érett rendszer, ahol a stabilitás és a visszafelé kompatibilitás a prioritás. Egy növekedési fázisban lévő startup, amely hetente több tucat funkciót szállít, más kockázati profilt fogadhat el, mint egy szabályozott banki rendszer, amelynél egy hiba pénzügyi vagy jogi következményekkel jár.

29.1.5 Az üzleti környezet

A kontextus második dimenziója az üzleti környezet. Itt a következő kérdésekre kell választ kapnunk: Mire szolgál a rendszer a felhasználói számára? Ki fizeti a működését? Milyen versenyhelyzetben működik? Mi a gazdasági modellje? Most a válaszok kategóriái természetesen nem jogi, hanem architektúra-elemzési értelemben használatosak.

Egy rendszer, amelynek működését tranzakciónkénti díjból finanszírozzák, más skálázási döntéseket hoz, mint egy rendszer, amelynek költségeit egy belső IT-költséghely viseli. Egy adatintenzív rendszer, ahol a hirdetési bevételek a felhasználói aktivitástól függenek, más adatarchitektúrát épít, mint egy belső jelentéskészítő rendszer, amelynek csak napi egyszeri futtatásra van szüksége.

Különösen fontos a jogi és szabályozási környezet feltárása. Egy egészségügyi rendszer, amelyre a GDPR és ágazati szabályok vonatkoznak, más adatkezelési és auditálási architektúrát igényel, mint egy szabadidős alkalmazás. Az adatmegőrzési kötelezettségek, a titkosítási követelmények, az auditnaplózás szükségessége mind olyan kontextuális tényezők, amelyek közvetlenül hatással vannak az architekturális döntésekre.

29.1.6 A technikai kényszerek

A harmadik kontextuális dimenzió a technikai kényszerek halmaza. Ide tartoznak a meglévő technológiai örökségek, a csapat kompetenciái, az infrastrukturális lehetőségek és a technikai adósság jelenlegi állapota. Egy architektúra soha nem tiszta lappal indul: mindig létezik egy meglévő rendszer, egy meglévő csapat és egy meglévő működési környezet, amelyek meghatározzák a lehetséges döntések terét.

A technikai kényszerek közé tartozik az is, hogy a rendszer milyen más rendszerekkel áll kapcsolatban. Egy vállalati rendszer, amelynek két tucat belső rendszerrel kell integrálódnia, más integrációs architektúrát igényel, mint egy önálló termék, amely csak a nyilvános API-kat használja. A külső rendszerek elérhetősége, teljesítménye, szerződéses garanciái mind részei a kontextusnak.

A kontextus rekonstruálásának gyakorlati eszköze egy egyszerű táblázat, amely a három dimenzió mentén rendszerezi a felfedezett tényeket. A táblázat minden sorában fel kell tüntetni a tény forrását is – ez az igény különösen fontos, mert a kontextus rekonstruálása során gyakran találkozunk ellentmondó vagy bizonytalan információkkal. A kontextus-táblázat kitöltése során derül ki, hogy mely területeken vannak ismerethiányaink, és ezek a hiányok hol befolyásolják a későbbi elemzés megbízhatóságát.

Dimenzió	Kérdés	Felfedezett tény	Forrás	Bizonyosság
Idő	Mikor született a rendszer?	2019 elején indult a fejlesztés	Cégblog-bejegyzés	Közepes
Üzlet	Mi a bevételi modell?	Tranzakciónkénti díj	Nyilvános árazási oldal	Magas
Technika	Milyen platformra épült?	Kubernetes 1.17	Előadás-diák	Alacsony

Az ilyen táblázat célja nem csupán az információ rendszerezése, hanem annak láthatóvá tétele, hogy a kontextusról alkotott képünk milyen mértékben megalapozott. Ha a bizonyosság oszlopában sok az “alacsony” bejegyzés, akkor a későbbi tanulságokat is ennek megfelelően óvatosan kell megfogalmaznunk.

29.1.7 A második lépés: a döntések rekonstruálása

Amikor a kontextus kellő részletességű, áttérhetünk a döntések rekonstruálására. Ez a lépés arra a felismerésre épül, hogy egy architektúra nem csupán komponensek és kapcsolatok halmaza, hanem döntések láncolata. Minden látható szerkezeti elem – egy szolgáltatáshatár, egy adatbázis-választás, egy szinkron vagy aszinkron kommunikációs kapcsolat – mögött legalább egy döntés áll, és minden döntés mögött alternatívák állnak, amelyeket a döntéshozó elvetett.

A döntések rekonstruálása során az a legfontosabb feladatunk, hogy a látható artefaktumokból – diagramokból, kódból, konfigurációból, naplókból, beszámolókból – visszakövetkeztessünk a mögöttük álló döntési helyzetekre. Ez természetesen rekonstrukció, nem pontos történetírás: a múltbeli döntéshozó gondolatait nem ismerhetjük meg közvetlenül, csak a döntések lenyomatait.

29.1.8 Milyen típusú döntéseket keresünk?

Az architektúrális döntések négy nagy kategóriába sorolhatók. Az első a **strukturális döntések** kategóriája: milyen komponensekre bontjuk a rendszert, hol húzódnak a szolgáltatáshatárok, milyen a modulok közötti függőség iránya. A második a **technológiai döntések** kategóriája: milyen nyelvet, keretrendszert, adatbázist, infrastruktúrát választunk. A harmadik a **kommunikációs döntések** kategóriája: milyen protokollokon, milyen mintákkal, milyen szinkronitással kommunikálnak a komponensek. A negyedik a **működési döntések** kategóriája: hogyan telepítünk, hogyan skálázunk, hogyan figyelünk és hogyan riasztunk.

Egy érett rendszer elemzése során ezekből a döntésekből tucatnyi vagy akár több száz is feltárható. A rekonstrukciónál nem a teljességre kell törekedni, hanem a **meghatározó döntések** azonosítására, amelyeknek a legnagyobb hatásuk van a rendszer jelenlegi tulajdonságaira és a jövőbeli változtatások lehetőségeire. Jó kiindulópont, ha azokat a döntéseket keressük, amelyek megváltoztatása a legnagyobb költséggel járna, vagy amelyek a legtöbb további döntést determinálják.

29.1.9 A döntési napló felépítése

A döntések rekonstruálásának gyakorlati eszköze a döntési napló. Ennek felépítése követi a dokumentációs szerződésben rögzített ADR-formátumot, de kiegészül a rekonstrukciós helyzetre jellemző elemekkel. Minden rekonstruált döntéshez a következőket rögzítjük:

A döntés leírása: egy tömör megfogalmazás arról, hogy mit választottak. Például: „A rendszer két szolgáltatásra bomlik: egy termék-katalógus-szolgáltatásra és egy rendeléskezelő-szolgáltatásra, amelyek szinkron REST-hívásokkal kommunikálnak."

A rekonstruált kontextus: milyen körülmények között született a döntés, a korábban feltárt kontextuselemek fényében. Ez a mező magyarázza, hogy miért volt racionális a döntés a maga idejében.

A valószínűsíthető alternatívák: milyen más megoldások jöhettek szóba, és miért eshetett ezekre a választás. Ez a mező gyakran spekulációt igényel, ezért a bizonyosság szintjét itt is jelölni kell.

A látható következmények: milyen hatásai láthatók ma a döntésnek – nemcsak a szándékolt hatások, hanem a nem szándékolt mellékhatások is.

A rekonstrukció bizonytalansága: mennyire vagyunk biztosak abban, hogy a döntés valóban így született, vagy csak utólagos racionalizálást végzünk.

Ez a naplóforma abban különbözik a szokásos ADR-től, hogy nyíltan vállalja a bizonytalanságot. Egy valós rendszer elemzése során gyakran nem tudjuk, hogy egy adott szerkezet miért olyan, amilyen: szándékos döntés eredménye, vagy egyszerűen így alakult, mert senki sem változtatta meg. A rekonstruált döntési napló ezt a különbségtételt láthatóvá teszi.

29.1.10 A harmadik lépés: a következmények értékelése

A döntések rekonstruálása után következik a következmények értékelése. Ez a lépés a leginkább hajlamos a retrospektív torzításra, ezért itt különösen fontos a módszeres megközelítés. A cél nem az, hogy a múltbeli döntéseket „jókra" és „rosszakra" osszuk, hanem hogy megértsük, milyen oksági láncolatok vezettek a jelenlegi állapothoz.

29.1.11 Az oksági láncok feltárása

Minden architektúrális döntésnek vannak elsődleges és másodlagos következményei. Az elsődleges következmények azok, amelyeket a döntéshozó szándékolt: a mikroszolgáltatás-bontás lehetővé teszi a csapatok független telepítését; a cache bevezetése csökkenti az adatbázis-terhelést; a REST API kialakítása egyszerűsíti a kliensek integrációját. A másodlagos következmények azok, amelyek nem voltak szándékoltak, de a döntés logikus következményei: a mikroszolgáltatás-bontás növeli a hálózati komplexitást és a hibamódok számát; a cache bevezetése a konzisztencia-problémák

új osztályát teremti meg; a REST API kényszerei megnehezítik bizonyos lekérdezések hatékony kiszolgálását.

Az elemzés során mindkét következménytípust vizsgálni kell. A szándékolt következmények azt mutatják, hogy a döntés elérte-e a célját; a nem szándékolt következmények azt, hogy a döntés milyen új problémákat hozott létre, amelyeket további döntésekkel kellett kezelni. Ez a láncolat – döntés, szándékolt következmény, nem szándékolt következmény, korrekciós döntés – az architektúra evolúciójának alapszövete.

29.1.12 Hogyan értékeljük a következményeket anélkül, hogy ítélkeznénk?

Az értékelés során a legfontosabb módszeres szabály, hogy a következményeket a döntés időpontjában elérhető információk fényében kell megítélnünk, nem pedig a mai tudásunkkal. Egy 2015-ös döntés, amely a mikroszolgáltatások mellett tette le a voksot, akkor racionális volt, mert a szervezetnek meg kellett oldania a független csapatok skálázását, és a rendelkezésre álló eszközök ezt az utat kínálták. Hogy a döntés tíz évvel később technikai adósságot halmozott fel, nem teszi a döntést akkor rosszá – legfeljebb azt mutatja, hogy a döntésnek olyan hosszú távú hatásai is voltak, amelyek akkor nem voltak láthatók.

Ez a megkülönböztetés azért fontos, mert a retrospektív torzítás éppen ezt a határt mossa el. Ha a következményeket a mai tudásunkkal ítéljük meg, akkor minden múltbeli döntést hibásnak fogunk látni – különösen azokat, amelyeknek az átmeneti technológiai hullámai időközben levonultak. A valóságos tanulság azonban nem az, hogy „a mikroszolgáltatások rosszak”, hanem az, hogy „a mikroszolgáltatásoknak olyan rejtett költségei vannak, amelyeket a döntés időpontjában nehéz volt előre látni; ezek a költségek bizonyos kontextusokban elfogadhatók, más kontextusokban nem”. Ez a finom megfogalmazás az, ami az esettanulmány-elemzést valóban értékessé teszi.

A következmények értékelésénél hasznos, ha különbséget teszünk **reverzibilis** és **irreverzibilis** következmények között. A reverzibilis következmények – például egy adott API-kialakítás – utólag módosíthatók, bár ennek is van költsége. Az irreverzibilis következmények – például az adatmodell alapstruktúrája vagy a szervezeti beágyazottságú technológiai döntések – sokkal nehezebben változtathatók, ezért nagyobb kockázatot hordoznak. Az elemzésnek különös figyelmet kell fordítania az ireverzibilis döntésekre, mert ezek azok, ahol a döntés időpontjában rendelkezésre álló információ minősége a legkritikusabb.

29.1.13 A negyedik lépés: a tanulságok kivonása

A tanulságok kivonása az elemzés utolsó és egyben legnehezebb lépése. Itt kell a konkrét esettől eljutni az általánosítható mintákig – anélkül, hogy túláltalánosítanánk. A legnagyobb hiba ezen a ponton a kontextus elfelejtése: olyan tanulságok megfogalmazása, amelyek azt állítják, hogy „X technológia jó” vagy „Y architektúra rossz”, holott az elemzés éppen azt mutatta meg, hogy X és Y megítélése a kontextustól függ.

29.1.14 A tanulságok szintjei

A tanulságokat három szinten érdemes megfogalmazni. Az első szint a **kontextusfüggő tanulság**: „Ebben a konkrét rendszerben, ebben a konkrét időpontban a választott megoldás elérte vagy nem érte el a célját.” Ez a szint a leginkább ellenőrizhető, de a legkevésbé általánosítható.

A második szint a **feltételes tanulság**: „Abban az esetben, ha a rendszernek nagy forgalom mellett kell alacsony késleltetéssel válaszolnia, a cache bevezetése hatékony, de a konzisztencia-kezelés költségét be kell tervezni.” Ez a szint már általánosabb, de megőrzi a feltételrendszert, amelyen belül az állítás érvényes.

A harmadik szint az **elv szintű tanulság**: „A nem szándékolt következmények elemzése minden architekturális döntésnél kötelező.” Ez a szint a legáltalánosabb és a legkevésbé megfogható; azonban éppen ezek az elvek azok, amelyeket valóban átvihetünk egyik rendszerből a másikba.

A jó elemzés mindhárom szinten fogalmaz meg tanulságokat, és világosan jelöli, hogy melyik tanulság melyik szinthez tartozik. Így az olvasó számára egyértelmű, hogy mit lehet közvetlenül alkalmazni a saját rendszerében, és mi az, ami csak a hasonló kontextusú rendszerekre érvényes.

29.1.15 A tanulságok ellenőrizhetősége

Minden tanulság annyit ér, amennyire ellenőrizhető. Az ellenőrizhetőség azt jelenti, hogy a tanulságot alátámasztó megfigyelések, források és logikai lépések visszakereshetők. Egy olyan állítás, hogy „a döntés a rendelkezésre állást javította”, csak akkor értékes, ha tudjuk, hogyan mérték a rendelkezésre állást, milyen időablakban, és milyen zavaró tényezők lehetnek. A tanulságok mellé ezért mindig hozzá kell kapcsolni a bizonyítékok listáját és a bizonyosság szintjét.

Az ellenőrizhetőség különösen fontos a nyilvános esettanulmányoknál. Egy cégblog-bejegyzés, amely a sikeres átállást ünnepli, nem bizonyítéka annak, hogy az átállás valóban sikeres volt – legfeljebb annak, hogy a cég így mutatja be. A tanulságoknál mindig jelölni kell, hogy milyen típusú források állnak rendelkezésre, és ezek mennyire tekinthetők függetlennek és megbízhatónak.

29.1.16 A források és a bizonytalanság kezelése

Az esettanulmány-elemzés leginkább alulbecsült része a források és a bizonytalanság jelölése. A valós rendszerek elemzése során sosem áll rendelkezésünkre a teljes kép – mindig részleges, torzított, egymásnak ellentmondó információk halmazából kell következtetnünk. A módszer akkor hiteles, ha ezt a bizonytalanságot nem rejti el, hanem explicit módon, a dokumentáció részeként kezeli.

29.1.17 A források osztályozása

Az esettanulmány-elemzés forrásai négy nagy osztályba sorolhatók aszerint, hogy milyen közelségben vannak a tényleges döntésekhez és milyen torzítások terhelik őket.

Elsődleges források azok, amelyek közvetlenül a vizsgált rendszerből származnak: a forráskód, a konfiguráció, az ADR-ek, a monitorozási adatok, a naplók. Ezek a legmegbízhatóbbak a rendszer aktuális állapotát illetően, de nem feltétlenül tükrözik a döntések mögött álló okokat.

Másodlagos források azok, amelyek a rendszerről beszámolnak: a fejlesztői dokumentáció, a blogbejegyzések, a konferencia-előadások, a belső wikik. Ezek gyakran tartalmazzák a döntések hivatalos narratíváját, de ezt a narratívát a szerzők pozíciója és céljai torzítják.

Harmadlagos források azok, amelyek távolabbi nézőpontból elemzik a rendszert: a műszaki sajtó cikkei, a közösségi média beszámolói, a versenytársak publikációi. Ezek függetlenebbek lehetnek, de ritkán férnek hozzá a belső információkhoz.

Szakértői források azok, amelyek a rendszerrel kapcsolatos területi tudást biztosítanak: a technológiai dokumentációk, a szabványok, az akadémiai irodalom. Ezek a döntések értékelésénél segítenek, de nem szolgáltatnak bizonyítékot a konkrét döntésekről.

29.1.18 A bizonytalanság jelölése

A bizonytalanság jelölésének egyszerű, de hatékony rendszere három szintet használ: **magas bizonyosság**, ha az állítást több, egymástól független forrás is megerősíti, vagy ha közvetlenül a forráskódból ellenőrizhető; **közepes bizonyosság**, ha az állítást egyetlen forrás támasztja alá, vagy ha a források részben ellentmondók; **alacsony bizonyosság**, ha az állítás csak spekuláción vagy olyan forráson alapul, amelynek megbízhatósága ismeretlen.

A bizonytalanság jelölésének célja nem az, hogy az elemzést megbénítsa, hanem hogy lehetővé tegye a bizonyosság szintje szerinti differenciált következtetést. Egy magas bizonyosságú tényre épülő tanulság erősebb, mint egy alacsony bizonyosságú spekulációra épülő. Az elemzés akkor marad használható, ha a bizonytalan állításokon nyugvó következtetéseket is ennek megfelelően fogalmazzuk meg.

Különösen fontos a bizonytalanság jelölése akkor, ha az elemzés hiányzó információkra bukkan. A hiányzó információ maga is információnak tekinthető: ha egy nyilvános esettanulmányból hiányzik a hibakezelés leírása, az arra utalhat, hogy a rendszer ezen a területen kevésbé érett, vagy hogy a beszámoló szerzője nem tartotta fontosnak. A hiányzó információk listája az elemzés szerves része kell, hogy legyen, mert ez jelöli ki a további kutatás irányát.

29.1.19 Az esettanulmány-elemzési sablon

Az eddig tárgyalt módszertani elemeket egy gyakorlatban használható sablonban foglaljuk össze. A sablon szolgál a labor átadandó eredményének szerkezeti vázaként is. A sablon a következő szakaszokból áll:

29.1.20 1. Bevezetés és vizsgálati cél

Ez a szakasz rögzíti, hogy mit vizsgálunk, miért éppen ezt a rendszert választottuk, és milyen kérdésekre keressük a választ. A vizsgálati cél meghatározza az elemzés fókuszát: egy rövid elemzés nem tud mindenre kiterjedni, ezért explicit módon rögzíteni kell, hogy mely kérdések vannak a fókuszban, és melyek azok, amelyeket szándékosan kizárunk.

29.1.21 2. Kontextus

A kontextus szakasz a korábban ismertetett három dimenzió – idő, üzlet, technika – mentén rendszerezett táblázatot tartalmaz. Minden tényhez forrás és bizonyosság jelölés tartozik. Ez a szakasz tartalmazza a hiányzó kontextuselemek listáját is.

29.1.22 3. Döntési napló

A döntési napló a korábban ismertetett felépítésben rögzíti a rekonstruált döntéseket. A döntések azonosítójához később hivatkozásként lehet kapcsolni a tanulságokat és a következményeket.

29.1.23 4. Következmények

A következmények szakasz minden döntéshez hozzárendeli a szándékolt és nem szándékolt következményeket, a reverzibilitás mértékét, valamint a korrekciós döntéseket, amelyeket a nem szándékolt következmények kezelésére hoztak.

29.1.24 5. Tanulságok

A tanulságok szakasz a korábban ismertetett három szinten fogalmazza meg a következtetéseket. Minden tanulsághoz hozzá van rendelve a forrás, a bizonyosság szintje és az alkalmazhatósági feltételek.

29.1.25 6. Ellenőrzési szempontok és nyitott kérdések

Az utolsó szakasz azokat az ellenőrzési szempontokat tartalmazza, amelyekkel az elemzés minősége felmérhető, valamint azokat a nyitott kérdéseket, amelyeket a jelenlegi források alapján nem lehetett megválaszolni. Ez a szakasz a további kutatás kiindulópontja.

A sablon nem merev űrlap, hanem gondolkodási keret. A használata során az elemző gyakran visszatér korábbi szakaszokhoz, amikor egy későbbi felfedezés árnyalja a korábbi megállapításokat. Ez a visszacsatolás nem hiba, hanem a módszer működésének természetes velejárója.

29.1.26 A módszer alkalmazása egy kis példán

A módszer működését egy fiktív, de életszerű példán szemléltetjük. A példa célja nem egy valós rendszer elemzése, hanem a módszertani lépések bemutatása ellenőrizhető keretek között.

29.1.27 A példa rendszer

Vizsgáljunk egy képzeletbeli rendszert, nevezzük „Rendeléskezelő”-nek. A nyilvános dokumentáció szerint a rendszer online rendelések feldolgozására szolgál, 2017-ben fejlesztették, és azóta folyamatosan üzemel. A rendelkezésre álló források: egy blogbejegyzés a fejlesztés indulásáról, egy konferencia-előadás 2019-ből, ahol a csapat az architektúráról beszélt, és a nyilvános API dokumentáció.

29.1.28 Kontextus rekonstruálása

A kontextus-táblázat a következőket tartalmazza:

Dimenzió	Tény	Forrás	Bizonyosság
Idő	2017-ben indult	Blogbejegyzés	Közepes
Üzlet	Kiskereskedelmi rendelések feldolgozása	API dokumentáció	Magas
Technika	Kezdetben monolitikus PHP, később bővült	Konferencia-előadás	Közepes

Dimenzió	Tény	Forrás	Bizonyosság
Hiányzó	A csapat mérete, a szervezeti felépítés, a működési költségek	–	–

A táblázatból azonnal látszik, hogy a kontextusról alkotott képünk hiányos: nem ismerjük a csapat méretét, a szervezeti felépítést és a működési költségeket. Ez a hiány korlátozza a döntések értékelésének mélységét.

29.1.29 Döntések rekonstruálása

A blogbejegyzésből kiderül, hogy a csapat kezdetben egyetlen PHP-alkalmazást épített. A konferencia-előadásból az is látszik, hogy 2019-ben a rendszert kiegészítették egy Python-alapú ajánlószolgáltatással, amely aszinkron üzenetekkel kommunikál a fő alkalmazással. A döntési napló egy bejegyzése:

D-1 döntés: „Kezdetben monolitikus PHP-alkalmazás épült.” **Kontextus**: A csapat 2017-ben három fejlesztőből állt, a határidő feszes volt, a rendszer várható forgalma alacsony. **Alternatívák**: A mikroszolgáltatás-bontás is szóba jöhetett volna, de a csapat mérete és a határidő ezt nem indokolta. **Következmények**: A monolit gyors fejlesztést tett lehetővé; a későbbi bővítésnél a monolit határai korlátozták a skálázást. **Bizonyosság**: Közepes – a döntés ténye a blogbejegyzésből származik, a kontextus rekonstrukciója részben spekuláció.

29.1.30 Következmények értékelése

A D-1 döntés következményei közül a szándékolt hatás – a gyors fejlesztés – valószínűleg megvalósult, bár a források nem tartalmaznak mérési adatokat ennek alátámasztására. A nem szándékolt hatások közül az ajánlószolgáltatás későbbi hozzáadása mutatja, hogy a monolit határai valós korlátok voltak. A D-2 döntés – az aszinkron üzenetkezelés bevezetése – éppen a monolit korlátjaira adott korrekciós válasznak tekinthető.

29.1.31 Tanulságok leszűrése

A példából három szinten vonhatunk le tanulságokat. A kontextusfüggő tanulság: „A Rendeléskezelő rendszer indulásakor a monolit választás racionális volt, mert a kis csapat és a feszes határidő mellett a gyors fejlesztés fontosabb, mint a jövőbeli skálázhatóság.” A feltételes tanulság: „Ha egy rendszer várható forgalma alacsony és a csapat kicsi, a monolit indítás gyorsabb; a későbbi bővítés költsége akkor jelenik meg, ha a forgalom és a csapat növekedése egyaránt bekövetkezik.” Az elv szintű tanulság: „A kezdeti strukturális döntések hosszú távú hatását a döntés időpontjában nehéz előre látni; a reverzibilitás mérlegelése a döntés fontos szempontja.”

29.1.32 Ábrázolás diagrammal

Az elemzés folyamata vizuálisan is megjeleníthető. Az alábbi Mermaid-diagram a négy elemzési lépés kapcsolatát és a köztük lévő visszacsatolási hurkokat mutatja:

```
flowchart LR
    A[Kontextus rekonstruálása] --> B[Döntések rekonstruálása]
    B --> C[Következmények értékelése]
    C --> D[Tanulságok kivonása]
    D --> A
    D --> B
    C --> A
```

A diagram azt a felismerést teszi láthatóvá, hogy az elemzés nem lineáris, hanem iteratív: a tanulságok kivonása során felfedezett hiányosságok visszavezetnek a kontextus vagy a döntések további vizsgálatához.

29.1.33 Tipikus hibák és javításuk

Az esettanulmány-elemzés során három hibatípus fordul elő leggyakrabban. Mindegyiknek megvan a maga felismerési módja és javítási stratégiája.

29.1.34 A narratíva elfogadása bizonyítékként

Az első hiba a narratíva és a bizonyíték összekeverése. Egy blogbejegyzés, amely a rendszer sikeréről számol be, maga is a rendszer narratívájának része, nem független bizonyíték. Ha az elemző ezt a narratívát tényként kezeli, akkor a tanulságok is a narratíva tükreivé válnak.

A hiba felismerésének jele, hogy az elemzés méltatásokat tartalmaz: „a rendszer sikeresen skálázódott”, „a csapat kiváló döntéseket hozott”. A javítás módja a források osztályozása és a bizonyosság jelölése: a narratív megállapításokat a megfelelő forrástípusnál kell nyilvántartani, és a hozzájuk tartozó bizonyosság szintjét ennek megfelelően kell meghatározni.

29.1.35 A kontextus nélküli technológiai ítélet

A második hiba a kontextus nélküli technológiai ítélet. Ez akkor fordul elő, amikor az elemző a saját jelenlegi preferenciáit vetíti vissza a múltbeli döntésekre, és ezek alapján minősíti a döntéseket. A „miért nem használtak konténereket 2017-ben?” típusú kérdés figyelmen kívül hagyja, hogy a konténer-orchestráció akkor még nem volt a ma ismert érettségi szinten.

A javítás módja az időbeli elhelyezés következetes alkalmazása: minden döntésnél rögzíteni kell, hogy milyen technológiák álltak rendelkezésre a döntés időpontjában, és milyen érettségi szinten. Ha ez az információ hiányzik, a hiányt jelölni kell, nem pedig a mai állapotot visszavetíteni.

29.1.36 A tanulságok túláltalánosítása

A harmadik hiba a tanulságok túláltalánosítása. Ez akkor fordul elő, amikor az elemző egy konkrét esetből közvetlenül általános szabályt fogalmaz meg, megkerülve a feltételes tanulság szintjét. A „a mikroszolgáltatások rosszak” megállapítás egyetlen esettanulmányból – legyen az bármilyen alapos – nem támasztható alá.

A javítás módja a tanulságok szintjeinek következetes elkülönítése. Az általános megállapításokat mindig a feltételes tanulságokon keresztül kell bevezetni, és explicit módon jelölni kell, hogy milyen feltételek teljesülése esetén érvényesek. Ha a feltételeket nem tudjuk megfogalmazni, akkor az általánosítás nem megalapozott.

29.1.37 Gyakorlati gyakorlat: esettanulmány-elemzési sablon készítése

Ez a gyakorlat a fejezet laborja. A feladat egy esettanulmány-elemzési sablon elkészítése, amely reprodukálható dokumentumban vagy jegyzetfüzetben rögzíti az elemzés lépéseit, a kontextust, a döntési naplót és az ellenőrzési szempontokat. Az átadandó eredménynek tartalmaznia kell a mért és a várt eredmény elkülönítését is.

29.1.38 A gyakorlat végrehajtása

A sablon elkészítéséhez a következő lépéseket kell végrehajtani:

Első lépés: Válassz ki egy nyilvánosan dokumentált rendszert, amelynek architektúrájáról elérhetők források. A gyakorlathoz nem szükséges teljes hozzáférés – elegendő egy blogbejegyzés, egy konferencia-előadás és a nyilvános API-dokumentáció. A választásnál fontos, hogy a rendszer elég egyszerű legyen ahhoz, hogy a források alapján a kontextus és a fő döntések rekonstruálhatók legyenek.

Második lépés: Töltsd ki a kontextus-táblázatot a három dimenzió mentén. Minden tényhez jelöld a forrást és a bizonyosság szintjét. Ha valamelyik dimenzióban nem áll rendelkezésre információ, a hiányt rögzítsd.

Harmadik lépés: Rekonstruálj legalább két döntést a döntési napló formátumában. A döntéseknek összefüggésben kell lenniük egymással: az egyik döntés következménye hozza létre a másik döntés kontextusát.

Negyedik lépés: Értékeld a döntések következményeit, különválasztva a szándékolt és nem szándékolt hatásokat. Ahol nem állnak rendelkezésre mérési adatok, jelezd, hogy a megállapítás csak kvalitatív.

Ötödik lépés: Fogalmazz meg tanulságokat mindhárom szinten, és mindegyikhez jelöld a forrást és a bizonyosság szintjét.

Hatodik lépés: Ellenőrizd a sablont az alábbi szempontok szerint.

29.1.39 Ellenőrzési szempontok

A sablon akkor felel meg a követelményeknek, ha minden tényhez forrás és bizonyosság szintje van rendelve, a döntési napló legalább két, egymással okozati kapcsolatban álló döntést tartalmaz, a következmények különválasztják a szándékolt és nem szándékolt hatásokat, a tanulságok mindhárom szinten megjelennek, és az egész dokumentum reprodukálható – azaz egy másik elemző ugyanezen források alapján követni tudja a logikai lépéseket.

A mért és a várt eredmény elkülönítése ebben a gyakorlatban a következőt jelenti: a mért eredmény az, amit a források ténylegesen állítanak – például egy adott konfigurációs paraméter, egy mérési érték, egy konkrét technológiai választás. A várt eredmény az, amit egy jó architektúrális döntéstől az adott kontextusban várnánk – saját mérnöki ítéletünk alapján. A kettő közötti eltérés nem feltétlenül hiba; lehet, hogy a forrás hiányos, vagy hogy a döntés más szempontokat mérlegelt, mint amit mi feltételeztünk. A lényeg, hogy a különbség látható legyen, mert ez a különbség a tanulás forrása.

29.1.40 Önellenőrző kérdés

Az önellenőrző kérdés célja, hogy felmérje, mennyire sikerült elsajátítani a fejezet fő gondolatmenetét. A kérdés: „Milyen szempontok alapján lehet megkülönböztetni egy kontextusfüggő tanulságot egy elv szintű tanulságtól, és miért fontos ez a megkülönböztetés?”

A jó válasz a következő elemeket tartalmazza: a kontextusfüggő tanulság egy konkrét rendszerre és időpontra vonatkozik, az elv szintű tanulság általános érvényű, a kettő között a feltételes tanulság szintje teremt átmenetet. A megkülönböztetés azért fontos, mert a tanulságok alkalmazhatósága az általánosság szintjétől függ: a kontextusfüggő tanulság csak hasonló kontextusú rendszerekre vihető át, az elv szintű tanulság szélesebb körben, de kevésbé konkrét iránymutatást ad. A megkülönböztetés elmulasztása túláltalánosításhoz vagy a tanulságok téves alkalmazásához vezet.

Ez a kérdés azért önellenőrző, mert a válasz megfogalmazása közben az olvasó szembesül azzal, hogy megértette-e a fejezet módszertanának finomszerkezetét, vagy csak a felszíni fogalmakat ismeri. A tanulságok szintezése az esettanulmány-elemzés egyik legfontosabb hozadéka: ez az a képesség, amely a módszert a szokásos „olvassuk el, mit csináltak, és döntsük el, tetszik-e” hozzáállástól megkülönbözteti.

29.2 Valós rendszerek architektúrái és tanulságai

A valós rendszerek architektúrájának elemzése során gyakran az a legnagyobb hiba, hogy a döntéseket elszigetelten, a kontextusból kiragadva vizsgáljuk. Egy sikeres rendszer architektúrája nem önmagában jó, hanem az adott időszak, terhelési profil, csapatstruktúra és üzleti célok mellett működött. Ebben az alfejezetben három, nyilvános beszámolókban jól dokumentált rendszer példáján keresztül mutatjuk be, hogyan lehet az architektúrális döntéseket rekonstruálni, értékelni és – óvatosan – átvihető tanulságokat levonni. A hangsúly a döntési logikán, a trade-offokon és a kontextus határain van, nem pedig az adott technológia másolásán.

29.2.1 1. Módszertani alap: döntések rekonstrukciója

Az Esettanulmány-elemzés módszertana alfejezetben bevezetett esettanulmány-elemzési módszertan itt alkalmazásra kerül. Az elemzés lépései röviden:

1. **Kontextus rögzítése:** üzleti cél, időszak, terhelési jellemzők, csapatméret, szabályozói környezet.
2. **Architektúra felderítése:** rendszerhatárok, fő komponensek, kapcsolatok, adatfolyamok.
3. **Döntések azonosítása:** melyek voltak a kulcsdöntések, és milyen alternatívákat vetettek el.
4. **Következmények elemzése:** minőségi attribútumokra gyakorolt hatás, kockázatok, adósságok.
5. **Tanulságok levonása:** mi az, ami általánosítható, és mi az, ami csak az adott kontextusban érvényes.

A döntéseket ADR-szerűen érdemes rögzíteni: kontextus, döntés, alternatívák, következmények, állapot. Egy elemzésnél nem az a cél, hogy minden döntést utólag igazoljunk, hanem hogy megértsük a döntéshozatal logikáját és az akkori bizonytalanságot.

29.2.2 2. Három rendszer, három tanulság

A következőkben három eltérő jellegű rendszer architektúrális döntéseit elemezzük. A példák nyilvános esettanulmányokon és az adott rendszerek hivatalos mérnöki blogbejegyzésein alapulnak; a konkrét számok és a mai állapot azonban változhatott, ezért mindig az adott időponthoz kötve kell értelmezni őket.

29.2.3 2.1 Netflix: a monolitból a reziliens mikroszolgáltatásokig

A Netflix 2008-ban egyetlen, Javában írt monolitikus alkalmazást üzemeltetett, amely a DVD-kölcsönzést szolgálta ki. Egy adatbázis-korrupciós incidens megakadályozta a DVD-k kiszállítását, ami rávilágított arra, hogy a rendszer nem tolerálja a komponensek meghibásodását. A cég ekkor hozott egy stratégiai döntést: a rendszert úgy kell átalakítani, hogy az egyes komponensek meghibásodása ne okozza a teljes szolgáltatás leállását. Ez vezetett a fokozatos cloud migrációhoz és a mikroszolgáltatás-architektúrához.

Kulcsdöntések és következményeik:

- **Fokozatos migráció a felhőbe:** A Netflix nem big bang migrációt hajtott végre, hanem szolgáltatásonként költözött. Ez lehetővé tette a kockázat csökkentését, de egy hosszabb időszakban hibrid környezetet igényelt.
- **Mikroszolgáltatások bevezetése:** A monolitot több száz, önállóan telepíthető szolgáltatásra bontották. Előnye a független skálázhatóság és telepítés, hátránya a megnövekedett működési komplexitás, a hálózati hívások és a részleges hibák kezelése.
- **Reziliencia-minták beépítése:** A Netflix olyan eszközöket fejlesztett, mint a Hystrix (megszakító áramkör, visszaeső válasz, erőforrás-izoláció) és a Chaos Monkey (szándékos meghibásodások szimulálása a termelési környezetben). A cél az volt, hogy a rendszer „graceful degradation” módon viselkedjen, azaz egy-egy szolgáltatás kiesése ne terjedjen tovább.
- **API gateway és service discovery:** A bejövő forgalmat egy gateway réteg kezeli, amely a belső szolgáltatások közötti útvonalválasztást és a terheléselosztást is végzi. A service discovery (Eureka) lehetővé teszi, hogy a szolgáltatások dinamikusan regisztrálják magukat.

Minőségi forgatókönyv például:

> **Inger:** Egy háttérszolgáltatás (például ajánló motor) hibás válaszokat ad vagy elérhetetlenné válik. **Környezet:** Normál üzem, csúcsidőben, több millió egyidejű streameléssel. **Válasz:** A rendszer 200 ms-on belül visszaad egy alapértelmezett, egyszerűbb ajánlást vagy üres listát; a hiba nem terjed át más szolgáltatásokra, és a lejátszás nem szakad meg. **Mérőszám:** A lejátszás rendelkezésre állása 99,99% az adott havi ablakban; a hiba miatti teljes leállás időtartama 0 perc.

Tanulságok:

- A reziliencia nem utólagos javítás, hanem tervezési elv. A meghibásodást normál üzemállapotnak kell tekinteni.
- A mikroszolgáltatások bevezetése akkor kifizetődő, ha a csapatok képesek önállóan üzemeltetni a szolgáltatásaikat, és a platform biztosítja a megfigyelhetőséget, a telepítést és a felfedezést.
- A fokozatos migráció csökkenti a kockázatot, de hosszú ideig fenntartja a kettős architektúra költségét.

- A kontextus fontos: a Netflix globális streaming szolgáltatásként hatalmas terhelést és magas rendelkezésre állási követelményt támaszt, ami indokolja a komplexitást. Egy kisebb rendszernél ugyanez a döntés túltervezés lenne.

29.2.4 2.2 Amazon: szervezeti határok és API-first

Az Amazon architektúrájának egyik legismertebb tanulsága nem technikai, hanem szervezeti. A 2002 körül kiadott, Jeff Bezosnak tulajdonított belső utasítás szerint minden csapatnak szolgáltatásinterfészeken keresztül kell kommunikálnia, és minden adatot API-n keresztül kell elérhetővé tenni. Ez a döntés vezetett a szolgáltatásorientált architektúra (SOA) bevezetéséhez, majd később a mikroszolgáltatásokhoz.

Kulcsdöntések és következményeik:

- **Kötelező API-k minden csapat között:** A csapatok nem hívhatták egymás adatbázisait vagy belső megvalósítását, csak publikált interfészeket. Ez laza csatolást és egyértelmű szerződéseket eredményezett, de növelte a hálózati hívások számát és a szerződéskezelés terhét.
- **Two-pizza team:** A csapatokat akkora méretűre szervezték, hogy két pizzával jóllakjanak (általában 6–8 fő). Ez a döntés a felelősség egyértelmű elhatárolását és az autonómiát szolgálta.
- **Infrastruktúra mint szolgáltatás:** Az Amazon belső platformját később kifelé is értékesítette (AWS). Ez azt mutatja, hogy az architektúrális döntések üzleti lehetőséget is teremthetnek.

Minőségi forgatókönyv például:

> **Inger:** Egy új ügyfélrendelés érkezik az e-kereskedelmi platformra. **Környezet:** Csúcsidőszak, például fekete péntek, a normál terhelés tízszerese. **Válasz:** A rendszer a kérést 500 ms alatt feldolgozza, és a készlet-, számlázási és szállítási szolgáltatások aszinkron módon értesülnek. **Mérőszám:** Az átlagos válaszidő 500 ms a P95-ös percentilisnél, a kérések 99,9%-a sikeresen feldolgozásra kerül a 24 órás ablakban.

Tanulságok:

- A szervezeti struktúra közvetlenül befolyásolja az architektúrát (Conway törvénye). Az Amazon esete azt mutatja, hogy a szolgáltatáshatárok kialakítása a csapathatárokhoz igazodik.
- Az API-first megközelítés lehetővé teszi a szolgáltatások közötti szerződéses együttműködést, de a szerződések verziózását és evolúcióját menedzselni kell.
- A belső platform termékként való kezelése új üzleti lehetőségeket nyithat, de csak akkor, ha a platform valóban önkiszolgáló és jól dokumentált.

29.2.5 2.3 Kubernetes: deklaratív állapot és vezérlők

A Kubernetes a konténer-orchestrálás de facto szabványa, amely a Google belső Borg rendszeréből nőtt ki. Architektúrája a deklaratív állapotkezelésen és a vezérlő hurkokon (control loop) alapul.

Kulcsdöntések és következményeik:

- **Deklaratív konfiguráció:** A felhasználó nem parancsokat ad, hanem leírja a kívánt állapotot (például „3 replikának kell futnia”). A rendszer folyamatosan figyeli a tényleges állapotot, és eltérés esetén beavatkozik. Ez nagyfokú öngyógyítást tesz lehetővé, de a hibaelhárítás nehezebb, mert a rendszer állapota dinamikusan változik.
- **Vezérlő minta:** Minden erőforráshoz tartozik egy vezérlő (controller), amely a kívánt és a tényleges állapot közötti eltérést próbálja megszüntetni. Ez a minta jól skálázható és bővíthető, de az egyes vezérlők közötti interakciók bonyolultak lehetnek.
- **API server mint egyetlen kapu:** Minden művelet az API szerveren keresztül történik, amely hitelesíti és auditálja a kéréseket. Ez egységes hozzáférést biztosít, de a központi komponens terhelése nagy lehet.
- **Hálózati modell:** Minden podnak saját IP-címe van, és a hálózati címfordítás (NAT) nélkül kommunikálhat. Ez leegyszerűsíti a szolgáltatások közötti kommunikációt, de hálózati overlay-t és CNI plugint igényel.

Minőségi forgatókönyv például:

> **Inger:** Egy munkacsomópont (node) meghibásodik, és a rajta futó podok elérhetetlenné válnak. **Környezet:** 100 csomópontos klaszter, normál üzem. **Válasz:** A rendszer 5 percen belül új csomópontra ütemezi a podokat, és a szolgáltatás ismét elérhető. **Mérőszám:** A podok újraindítási ideje 5 perc, a klaszter rendelkezésre állása 99,9% a havi ablakban.

Tanulságok:

- A deklaratív modell csökkenti a működtetés terhét, de megköveteli a mögöttes mechanizmusok megértését.
- A vezérlő minta kiválóan alkalmas elosztott rendszerek öngyógyítására, de a vezérlők egymásra hatását alaposan kell tesztelni.
- A Kubernetes komplexitása indokolt nagy, dinamikus terhelésű rendszereknél, de kisebb rendszereknél a menedzselt szolgáltatás (például egy felhőszolgáltató által üzemeltetett klaszter) vagy akár egy egyszerűbb konténerfuttató jobb választás lehet.

29.2.6 3. Skálázási, szervezeti és üzemeltetési tanulságok

A három példa alapján három dimenzió mentén vonhatunk le általános tanulságokat.

29.2.7 3.1 Skálázási tanulságok

- **A skálázás nem csak teljesítményprobléma.** A Netflix esetében a skálázás a szervezeti skálázást is jelentette: a csapatok számának növekedésével a monolit fejlesztése fenntarthatatlanná vált. A horizontális skálázás (több szolgáltatás, több példány) a komplexitás növekedésével jár.
- **A deklaratív megközelítés előnye:** A Kubernetes példája azt mutatja, hogy a kívánt állapot leírása és az öngyógyító mechanizmusok csökkentik a manuális beavatkozást nagy skálán.
- **Az aszinkron feldolgozás kulcsfontosságú:** Az Amazon példájában az üzenetvárakozás (queue) lehetővé teszi a terhelés simítását és a komponensek közötti laza csatolást. A Little-törvény alkalmazható a kapacitástervezésnél: ha a kérések érkezési rátája $\lambda = 1000$ req/s és az átlagos tartózkodási idő $W = 200$ ms, akkor a rendszerben lévő kérések átlagos száma $L = \lambda W = 1000 \cdot 0{,}2 = 200$. A kapacitást erre a szintre kell tervezni.

29.2.8 3.2 Szervezeti tanulságok

- **A csapathatárok döntőek.** Az Amazon two-pizza team modellje és a Netflix önállóan üzemeltető csapatai azt mutatják, hogy a szervezeti felépítés és a szolgáltatáshatárok összehangolása csökkenti a kommunikációs költségeket és a függőségeket.
- **A platform szerepe:** A Netflix és az Amazon is belső platformot épített, amely a közös infrastrukturális képességeket biztosítja. A platformcsapat célja, hogy a termékcsapatok önkiszolgáló módon telepíthessenek és üzemeltethessenek.
- **A döntéseknek gazdasági következményei vannak.** A technológiai döntések befolyásolják a fejlesztési sebességet, az üzemeltetési költséget és a termék evolúcióját. A CBAM módszerrel ezeket a trade-offokat számszerűsíteni lehet.

29.2.9 3.3 Üzemeltetési tanulságok

- **A hiba nem opcionális, hanem várható.** A Netflix Chaos Monkey megközelítése kiterjesztette a hibakezelést a tesztelésből a termelésbe. Az üzemeltetés során folyamatosan szimulálni kell a meghibásodásokat, hogy a rendszer ténylegesen fel legyen készülve rájuk.
- **Az SLO-k és error budget-ek adják a döntési keretet.** Ha egy szolgáltatás rendelkezésre állási SLO-ja 99,9%, akkor a havi error budget $100\% - 99{,}9\% = 0{,}1\%$, azaz havi 43,2 perc állásidő engedélyezett. Ez a budget lehetővé teszi a csapat számára, hogy tudatosan vállaljon kockázatot (például új verzió telepítését) anélkül, hogy a felhasználói elégedettség sérülne.
- **A megfigyelhetőség alapfeltétel.** Mindhárom rendszer esetében a logok, metrikák és trace-ek gyűjtése elengedhetetlen a hibaelhárításhoz és a kapacitástervezéshez. Az OpenTelemetry szabványos eszközkészletet biztosít ehhez.

29.2.10 4. A kontextus és az átvihetőség határai

A fenti példákból levont tanulságok nem általános érvényűek. Minden döntést a saját kontextusában kell értékelni.

Mikor érvényes a mikroszolgáltatás?

- Nagy, több csapat által fejlesztett rendszer, ahol a független telepítés és skálázás üzleti értéket jelent.
- Magas rendelkezésre állási követelmény, ahol a részleges hiba elszigetelése kritikus.
- Olyan szervezet, amely képes a DevOps kultúra és a platformmérnökség kialakítására.

Mikor nem érvényes?

- Kis rendszer, kevés csapattal, ahol a monolit egyszerűbb és olcsóbb.
- Erős tranzakciós integritást igénylő rendszer (például banki core), ahol az elosztott tranzakciók komplexitása jelentős kockázatot hordoz.
- Olyan csapat, amely nem rendelkezik a szükséges üzemeltetési képességekkel.

Hasonlóképpen a Kubernetes sem mindenhol jó választás. Egyetlen node-on futó alkalmazáshoz felesleges komplexitás, míg egy menedzselt konténerfuttató vagy serverless platform elegendő lehet.

Az átvihetőség határait akkor ismerjük fel, ha a döntést nem a technológia, hanem a mögöttes elv szintjén próbáljuk általánosítani. Például a Netflix reziliencia-mintái mögött az az elv áll, hogy a rendszernek részleges hibák mellett is működnie kell. Ez az elv egy monolitikus rendszerben is alkalmazható, például áramkör-megszakítóval a külső hívásokra.

29.2.11 5. Gyakori hibák az elemzés során

1. **A kontextus figyelmen kívül hagyása:** A „Netflix is így csinálta, tehát nekünk is így kell” érvelés hibás. Nincs két azonos kontextus.
2. **A döntések utólagos idealizálása:** A sikeres rendszerek döntéseit hajlamosak vagyunk zseniálisnak beállítani, holott gyakran kényszer szülte őket. Az elemzésnél a döntéshozatal időpontjában rendelkezésre álló információkat kell figyelembe venni.
3. **A trade-offok elrejtése:** Minden döntésnek van ára. Ha egy elemzés csak előnyöket sorol, az nem teljes.
4. **A mérés hiánya:** A skálázási és megbízhatósági állításokat mérőszámokkal kell alátámasztani. Például „a rendszer gyors” nem mérhető, de „a P95 válaszidő 200 ms 1000 RPS terhelés mellett” igen.
5. **Az üzemeltetési költség alábecslése:** Egy új architektúra bevezetésekor a működési költségek (emberi, infrastrukturális, képzési) gyakran meghaladják a fejlesztési költségeket.

29.2.12 6. Labor: három rendszer összehasonlító elemzése

A gyakorlat célja, hogy a saját környezetedben vagy nyilvános források alapján kiválassz három rendszert, és elvégezd azok összehasonlító architektúra-elemzését.

29.2.13 6.1 Kiinduló állapot

- Válassz három rendszert. Javasolt: egy nagyvállalati webes szolgáltatás (például e-kereskedelem), egy adatintenzív platform (például streaming vagy analitika), és egy infrastrukturális eszköz (például konténer-orchestrátor vagy üzenetkezelő).
- Minden rendszerről gyűjts nyilvános dokumentációt: hivatalos blog, konferencia-előadás, esettanulmány, nyílt forráskódú kód. A forrásokat rögzítsd dátummal és verzióval.
- Készíts egy sablont a jegyzethez, amely tartalmazza a kontextus, döntések, következmények és tanulságok szakaszokat.

29.2.14 6.2 Elvárt eredmény

- Egy reprodukálható dokumentum (például Markdown vagy Jupyter notebook), amelyben a három rendszer elemzése szerepel.
- Minden rendszerhez legalább egy C4 context vagy container diagram (diagrams-as-code, például Mermaid), amelyen a rendszerhatár, a fő komponensek és a kapcsolatok látszanak.
- Minden rendszerhez legalább három ADR-bejegyzés, amelyek a kulcsdöntéseket rögzítik.
- Összehasonlító táblázat, amely a skálázási, szervezeti és üzemeltetési dimenziókat mutatja.
- Minőségi forgatókönyv minden rendszerhez a teljesítményre vagy rendelkezésre állásra.
- A mért és a várt eredményeket külön kell választani: ha vannak reprodukált mérések, azokat jelölni kell; ha csak szakirodalomból vett adatok, akkor azok forrását és bizonytalanságát fel kell tüntetni.

29.2.15 6.3 Ellenőrzési szempontok

- A kontextusok rögzítettek és dátummal ellátottak.
- A döntések mögött alternatívák is szerepelnek, nem csak a választott megoldás.
- A következmények tartalmazzák a trade-offokat és a kockázatokat.
- A diagramokon a rendszerhatár és a protokoll, szinkronitás, hibatűrés jelölve van.

- A mérőszámok mértékegységgel és mérési ablakkal szerepelnek.
- A labor visszaállítható: ha például egy PoC környezetet hozol létre, azt le kell tudni bontani.

29.2.16 6.4 Visszaállítási út

Ha a gyakorlat során bármilyen erőforrást hozol létre (például ideiglenes Kubernetes klasztert vagy adatbázist), azt a feladat végén törölni kell. A dokumentumot verziókezelőben tárolt, és a források hivatkozásai időállóak.

29.2.17 7. Ellenőrzőlista

- ☐ Rögzítettem mindhárom rendszer kontextusát: üzleti cél, időszak, terhelés, csapatméret.
- ☐ Azonosítottam a fő architektúrális döntéseket, és mindegyikhez feljegyeztem az elvetett alternatívákat.
- ☐ A következményeknél feltüntettem a minőségi attribútumokra gyakorolt hatást, a kockázatokat és az adósságokat.
- ☐ Készítettem C4 diagramot diagrams-as-code formában, rendszerhatárral és jelmagyarázattal.
- ☐ Minden rendszerhez adtam minőségi forgatókönyvet a 4+1 séma szerint.
- ☐ A mérőszámok mértékegységgel és mérési ablakkal szerepelnek; a mért és várt eredményeket elkülönítettem.
- ☐ A tanulságoknál megjelöltem, hogy melyik elv általánosítható, és melyik kötődik a kontextushoz.
- ☐ A dokumentum verziózott, és a források dátumozva vannak.

29.2.18 8. Önellenőrző kérdés

> Miért nem elegendő egy ismert rendszer architektúráját lemásolni a saját rendszerünknél? Fogalmazz meg három konkrét okot, és mindegyikhez adj egy-egy rövid példát a fejezetben elemzett rendszerekből.

Várható válasz: A kontextus (terhelés, csapat, szabályozás) eltérő volta, a trade-offok egyoldalú figyelmen kívül hagyása, és az üzemeltetési költségek alulbecslése a fő okok. Például a Netflix mikroszolgáltatás-architektúráját nem érdemes egy 5 fős csapatnál bevezetni, mert a működtetési komplexitás meghaladná a fejlesztési előnyöket; az Amazon API-first szervezeti szabálya nem feltétlenül működik egy erősen szabályozott banki környezetben, ahol a tranzakciós integritás fontosabb a laza csatolásnál; a Kubernetes bevezetése egyetlen node-os alkalmazásnál csak felesleges komplexitást jelent.

29.3 Architektúra-katák és döntési gyakorlatok

Az architektúra-kata olyan gyakorló feladat, amely valós idejű nyomás nélkül, de valósághű korlátok között fejleszti a döntéshozatali képességet. Nem kódolási verseny, nem is vizsga: a cél a gondolkodás, a trade-off elemzés és a döntések dokumentálása. A harcművészeti katához hasonlóan egy előre meghatározott szituációt ismételhető, értékelhető módon oldunk meg, majd a megoldást közösen vagy önállóan értékeljük. Ez a fejezet az architektúra-katák módszertanát, a követelménytől a döntésig vezető utat, valamint az értékelés és visszacsatolás eszközeit mutatja be, és egy teljes példán keresztül vezeti végig az olvasót.

29.3.1 Az architektúra-kata mint tanulási forma

Az architektúra-kata célja, hogy biztonságos környezetben gyakoroljuk az architekt szerepét. A valós projektekben a döntések következményei hónapokkal később jelentkeznek, és a visszacsatolás lassú. A kata ezt a ciklust sűríti össze: a feladatmegoldás után azonnali, strukturált értékelést kapunk, amely feltárja a vakfoltokat és a kihagyott alternatívákat. A gyakorlat során nem az a fő kérdés, hogy „jó-e a megoldás”, hanem hogy „milyen feltételek mellett jó, és milyen áron”.

A kata formái változatosak: lehet egy rövid, 90 perces csoportos feladat, egy napos workshop, vagy akár egy több hetes, PoC-kal kísért gyakorlat. A közös elem az, hogy mindig adott egy kontextus, egy követelményhalmaz és egy sor kényszer, amelyre a csapatnak vagy az egyénnek konkrét architektúra-javaslatot kell készítenie. Az eredmény általában nem kód, hanem egy dokumentált döntési csomag: C4 diagramok, ADR-ek, minőségi forgatókönyvek és ellenőrzési szempontok.

A kata módszertanilag a következő lépésekre bontható:
1. **Kontextus és követelmények elemzése** – a kiinduló helyzet és a minőségi célok pontosítása.
2. **Alternatívák kidolgozása** – legalább két-három eltérő architektúrális megközelítés felvázolása.
3. **Döntés és dokumentálás** – a választott megoldás indoklása ADR formában.
4. **Értékelés és visszacsatolás** – a megoldás ellenőrzése szempontok szerint, majd a tanulságok levonása.

Ez a keret nem merev: a kata során lehet iterálni, visszalépni, sőt a feladat közben új információkat is be lehet vezetni. A lényeg a folyamat tudatossá tétele.

29.3.2 Követelmény, alternatíva és döntés

29.3.3 A követelmények pontosítása

Az architektúra-kata első lépése a kiinduló anyag értelmezése. A feladat általában tartalmaz egy üzleti leírást, felhasználói szerepköröket, funkcionális igényeket és nem funkcionális elvárásokat. Az architekt feladata, hogy ezekből mérhető, tesztelhető minőségi attribútumokat fogalmazzon meg. Ez a gyakorlatban quality attribute scenario (QAS) formában történik, amelyet a korábbi fejezetben vezettünk be.

A kata során a következő kérdésekre kell választ adni:
- Kik a stakeholderek, és mi a siker kritériuma számukra?
- Melyek a rendszer határai? Mi tartozik a rendszerbe, és mi külső?
- Melyek a legfontosabb minőségi attribútumok? Melyiknek van elsőbbsége ütközés esetén?
- Milyen technikai, szervezeti, jogi vagy költségvetési kényszerek vannak?
- Milyen mérési ablak és mérőszám tartozik az egyes elvárásokhoz?

Például egy „online jegyértékesítő rendszer” esetében az üzleti cél a jegyek értékesítése, de a rendszernek ki kell bírnia a koncertjegy-kiadás előtti forgalmi csúcsot is. A minőségi forgatókönyv így hangozhat:
Inger: 10 000 egyidejű felhasználó koncertjegy-vásárlási kísérlete az értékesítés első 5 percében.
Környezet: normál üzem, csúcsidőszak.
Válasz: a rendszer a kérések 99%-át 2 másodpercen belül kiszolgálja, és nem veszít el egyetlen sikeres tranzakciót sem.
Mérőszám: átlagos válaszidő p95 2 s, hibaarány 1%, mérési ablak: 5 perc.

A kata során elterjedt hiba, hogy a résztvevők azonnal a technológiára koncentrálnak, és átugorják a követelmények pontosítását. Pedig éppen ez a lépés ad alapot a későbbi döntésekhez. A QAS-ok

rögzítése után derül ki, hogy a válaszidő vagy a konzisztencia, a skálázhatóság vagy az egyszerű üzemeltetés élvez előnyt.

29.3.4 Alternatívák kidolgozása

A követelmények rögzítése után jön a tervezési tér feltárása. Az architektúra-katán elvárás, hogy a csapat legalább két, de inkább három alapvetően eltérő megoldást vázoljon fel. Az alternatívák nem egyszerűen technológiai változatok, hanem eltérő architektúrális stílusok vagy szerkezeti minták mentén szerveződnek.

Tipikus alternatívák:
- **Moduláris monolit** – egyetlen deployolható egység, de határozott belső modulhatárokkal.
- **Mikroszolgáltatások** – kis, függetlenül deployolható szolgáltatások, laza csatolással.
- **Eseményvezérelt architektúra** – aszinkron kommunikáció, eseményekre épülő feldolgozás.
- **Rétegezett architektúra** – klasszikus felosztás, például UI, üzleti logika, adat.
- **Kiszolgáló nélküli (serverless)** – felhőalapú, eseményvezérelt végrehajtás.

A kata során nem az a cél, hogy minden alternatívát részletesen kidolgozzunk, hanem hogy a legfontosabb trade-offokat azonosítsuk. Például a jegyértékesítő rendszer esetében a monolit egyszerűbb, de a csúcsterhelés miatt horizontális skálázást igényelhet, amihez a monolitot jól kell particionálni. A mikroszolgáltatások jobban skálázhatók, de a fejlesztési és üzemeltetési komplexitás magasabb. Az eseményvezérelt megoldás csökkenti a szinkron csatolást, de növeli a konzisztenciaproblémákat.

Az alternatívák értékelésére használhatunk döntési mátrixot, amelyben a minőségi attribútumokhoz súlyokat rendelünk, majd minden alternatívát pontozunk. A mátrix nem helyettesíti a gondolkodást, de segít a szempontok rendszerezésében és az elfogultságok feltárásában. Példa egy egyszerű mátrixra:

Szempont (súly)	Monolit	Mikroszolgáltatás	Eseményvezérelt
Skálázhatóság (0,3)	2	4	4
Fejlesztési sebesség (0,2)	4	3	3
Üzemeltetési egyszerűség (0,2)	5	2	3
Konzisztencia (0,15)	5	3	2
Hibatűrés (0,15)	2	4	4
Súlyozott összeg	**3,45**	**3,25**	**3,4**

A számok illusztrációk, és a kata során mindig rögzíteni kell, hogy melyik érték milyen feltételezésen alapul.

29.3.5 Döntés és dokumentálás

A döntési fázisban a csapat kiválasztja a leginkább megfelelő architektúrát, és ADR-ben rögzíti. Az ADR-ek szerkezetét a korábbi fejezetben megismertük; itt csak a kata szempontjából fontos elemeket emeljük ki. Egy jó ADR nem csak a választott megoldást írja le, hanem a kontextust, az alternatívákat és a következményeket is, beleértve a kockázatokat és a nyitott kérdéseket.

A döntés során gyakori hiba, hogy a csapat a „legjobb” megoldást keresi, holott ilyen ritkán létezik. A kata célja éppen az, hogy felismerjük: minden döntés trade-off, és a kontextus határozza meg, mi a megfelelő. Egy kis csapat, rövid határidő, egyszerű tartomány esetén a monolit

sokszor jobb, mint a mikroszolgáltatás, még akkor is, ha a divat mást sugall. Az ADR-nek ezt a kontextusfüggőséget kell tükröznie.

A labor részben egy konkrét példán mutatjuk be a teljes folyamatot.

29.3.6 Értékelés és visszacsatolás

A kata értékelése legalább annyira fontos, mint a megoldás. A visszacsatolás két szinten történik: egyéni vagy csoportos önreflexió, valamint strukturált külső review. Az értékelés szempontjai a következők lehetnek:
- **Követelménylefedettség:** minden fontos QAS-ra választ ad-e a megoldás?
- **Trade-off tudatosság:** dokumentálja-e az elvetett alternatívákat és azok indoklását?
- **Diagramminőség:** a C4 context és container szint megfelelően jelöli-e a rendszerhatárokat, komponenseket, interfészeket és protokollokat?
- **ADR-minőség:** a döntések visszakereshetők, érthetők, és tartalmazzák a következményeket?
- **Megvalósíthatóság:** a választott architektúra reálisan megvalósítható-e az adott csapat és költségvetés mellett?
- **Kockázatkezelés:** azonosítja-e a fő kockázatokat, és javasol-e mérséklő intézkedéseket?

Az értékelés során a hangsúly nem a „helyes” megoldás megtalálásán van, hanem az érvelés minőségén. Egy jól indokolt, de nem optimális döntés értékesebb, mint egy véletlenül jó választás, mert a gondolkodási folyamat tanulságosabb.

A visszacsatolás formái a kata típusától függnek. Rövid csoportos gyakorlatnál a facilitátor (vagy a többi csapat) pontozza a megoldást egy előre megadott rubrika alapján. A rubrika tartalmazza a fenti szempontokat és egy 1–5 skálát. Hosszabb, otthoni katánál az önellenőrző kérdések segítenek, valamint a megoldás összevetése más, elérhető megoldásokkal. A legfontosabb, hogy a résztvevő rögzítse, mit csinált volna másképp, ha több idő vagy több információ állt volna rendelkezésre.

Az értékelésnél a mért és várt eredményeket mindig külön kell választani. A kata során a „mért” értékek általában becslések vagy szimulációk, nem valós mérések. Ha például a rendszernek 10 000 egyidejű felhasználót kell kiszolgálnia, a csapat becsülheti a szükséges erőforrásokat vagy a várható válaszidőt, de ezt egyértelműen becslésként kell jelölni. A laborban erre külön figyelmet fordítunk.

29.3.7 Tipikus hibák és ellenminták

A katák során visszatérő hibák azonosíthatók. Ezek ismerete segít elkerülni a csapdákat, és fejleszti az önellenőrzést.

1. **Technológiai elfogultság.** A résztvevő a kedvenc technológiáját vagy a legújabb trendet választja, anélkül hogy a követelményekből indulna ki. Például azonnal Kubernetesre tervez, amikor egy egyszerű monolit is elegendő lenne. A katán mindig a követelményből kell kiindulni, és a technológiát csak utána választani.
2. **A minőségi attribútumok figyelmen kívül hagyása.** A csapat csak a funkcionális követelményekre koncentrál, és elhanyagolja a teljesítményt, biztonságot, karbantarthatóságot. Eredmény: olyan architektúra, amely működik, de nem skálázható, nem biztonságos, vagy nem üzemeltethető. A katán a QAS-ok készítése kötelező lépés.

3. **Túltervezés.** A résztvevő minden lehetséges jövőbeli igényre felkészül, és olyan komplex rendszert tervez, amelyet a csapat soha nem tudna megvalósítani. A YAGNI elv megsértése. A kata során a jelenlegi követelményekre kell optimalizálni, és a rugalmasságot csak ott beépíteni, ahol a bizonytalanság valós.

4. **Trade-off elemzés hiánya.** A csapat kiválaszt egy megoldást, de nem vizsgálja az alternatívákat, vagy nem dokumentálja, miért vetette el azokat. Ez a döntés igazolhatatlanságához vezet, és később, amikor a kontextus változik, nem tudjuk újraértékelni a döntést. Az ADR-ek éppen ezt a hiányosságot orvosolják.

5. **Dokumentáció nélküli döntés.** A vita során szóban megegyeznek, de semmi sem kerül rögzítésre. A katán a dokumentáció a fő eredmény, ezért minden döntést írásban kell rögzíteni, lehetőleg ADR formában.

6. **Nem egyértelmű rendszerhatárok.** A C4 diagramokon nem jelölik a rendszerhatárt, a külső rendszereket vagy a felhasználói szerepköröket. Ez azt eredményezi, hogy a komponensek közötti felelősségek homályosak maradnak. A katán a context és container szint kötelező.

7. **Mérhetőség hiánya.** Az elvárások úgy hangzanak, hogy „gyors” vagy „skálázható”, de nincs mérőszám, mérési ablak vagy célérték. Ilyenkor a későbbi értékelés lehetetlen. A QAS-oknak mindig tartalmazniuk kell a választ és a mérőszámot.

29.3.8 Labor: egy architektúra-kata végigvitele

Ez a labor egy konkrét kata feladatot mutat be, és lépésről lépésre vezeti végig az olvasót a megoldáson. A labor célja, hogy a résztvevő saját maga is elkészítse a dokumentációt, és összevesse az itt bemutatott megoldással. A labor kiinduló állapota, elvárt eredménye, ellenőrzési szempontjai és visszaállítási útja a következő.

Kiinduló állapot: Üres jegyzetfüzet vagy dokumentum, valamint a lenti feladatleírás. A résztvevőnek nincs kész megoldása.
Elvárt eredmény: Egy reprodukálható dokumentum, amely tartalmazza a kontextust, a QAS-okat, legalább két alternatívát, egy döntési mátrixot, egy ADR-t, C4 context és container diagramot (diagrams-as-code formában), valamint egy ellenőrzési szempontlistát. A mért és várt eredményeket külön jelölni kell.
Ellenőrzési szempont: A dokumentum visszakereshető, a döntések indokoltak, a diagramok a C4 konvenciókat követik, a QAS-ok mérőszámot és mérési ablakot tartalmaznak.
Visszaállítási út: A gyakorlat nem igényel rendszermódosítást; a dokumentum törölhető vagy archiválható. Ha PoC készült, azt izolált környezetben kell futtatni, és a labor végén leállítani.

29.3.9 A kata feladat: Városi Kerékpárkölcsönző Rendszer

Üzleti kontextus: Egy közepes méretű város önkormányzata kerékpárkölcsönző rendszert indít. A rendszer célja, hogy a felhasználók okostelefonos alkalmazáson keresztül bérelhessenek kerékpárt, és azt bármely dokkoló állomáson leadhassák. A rendszer induláskor 20 állomást és 500 kerékpárt kezel, de a tervek szerint két éven belül 50 állomásra és 2000 kerékpárra bővül. Az üzemeltetést az önkormányzat saját, 4 fős IT-csapata végzi, a fejlesztést külső beszállítóval. Költségvetés korlátozott.

Funkcionális követelmények (rövidítve):
- Felhasználó regisztráció és bejelentkezés.
- Kerékpár bérlés (felhasználó kiválaszt egy kerékpárt, majd feloldja a zárat).

- Kerékpár leadás (dokkoló állomáson, vagy szabadon, GPS alapján).
- Számlázás és fizetés (bérleti idő alapján, előre feltöltött egyenlegből vagy bankkártyás fizetéssel).
- Állomás és kerékpár állapotának lekérdezése (mobilalkalmazás, webes felület a lakosoknak).
- Adminisztrációs felület az üzemeltetőknek (állomás karbantartás, kerékpár áthelyezés, riasztások).
- Rendszeres riportok az önkormányzat részére (kihasználtság, bevétel, karbantartási igény).

Nem funkcionális követelmények (rövidítve):
- A mobilalkalmazás válaszideje normál terhelés mellett p95 500 ms, csúcsterhelés mellett p95 1,5 s, mérési ablak 5 perc.
- A rendszer rendelkezésre állása 99,5% havi szinten (üzemidő / (üzemidő + állásidő)).
- A bérlési tranzakció konzisztenciája erős: egy kerékpárt egyszerre csak egy felhasználó bérelhet.
- A személyes adatokat (név, e-mail, fizetési adatok) a GDPR szerint kell kezelni.
- A rendszernek ki kell bírnia a reggeli csúcsot, amikor 1000 felhasználó 15 perc alatt bérlést kezdeményez.
- Az üzemeltetési költség havi 5000 euró alatt maradjon, beleértve a felhőszolgáltatásokat és a szoftverlicenceket.

Kényszerek:
- A fejlesztést külső beszállító végzi, a kódot az önkormányzat kapja meg, de nincs belső tapasztalat Kubernetes üzemeltetésében.
- A mobilalkalmazás iOS és Android platformra készül.
- A fizetési adatokat nem szabad saját rendszerben tárolni; külső PCI DSS-kompatibilis szolgáltatót kell használni.
- Az állomások és a kerékpárok IoT-érzékelői mobilhálózaton keresztül kommunikálnak, időnként megszakad a kapcsolat.

29.3.10 A kata végrehajtása

29.3.10.1 1. lépés: Kontextus és minőségi forgatókönyvek

Elsőként azonosítjuk a stakeholdereket: felhasználók (bérlők), üzemeltetők (önkormányzati alkalmazottak), adminisztrátorok, külső fejlesztő cég, fizetési szolgáltató, önkormányzati vezetés. A rendszerhatárt meghúzzuk: a rendszerhez tartozik a mobilalkalmazás backendje, a dokkoló állomások vezérlője, az admin felület backendje, valamint az adatbázis. Külső rendszer: a fizetési szolgáltató, az IoT-eszközök (kerékpárzár, állomás szenzorok), a térképszolgáltató (GPS alapú leadáshoz).

A nem funkcionális követelményeket QAS formába öntjük. Például:

QAS-1: Egyidejű bérlési csúcs
- **Inger:** 1000 felhasználó reggel 8:00 és 8:15 között bérlési kérést indít.
- **Környezet:** normál üzem, csúcsidőszak.
- **Válasz:** a rendszer minden bérlési kérést 2 másodpercen belül feldolgoz, és a kerékpárzárat sikeresen feloldja, miközben a hibaarány 1%.
- **Mérőszám:** p95 válaszidő 2 s, hibaarány 1%, mérési ablak: 15 perc.

QAS-2: Konzisztencia
- **Inger:** Két felhasználó egyszerre kísérli meg ugyanannak a kerékpárnak a bérlését.
- **Környezet:** normál üzem, párhuzamos kérések.
- **Válasz:** a rendszer pontosan egy felhasználónak engedélyezi a bérlést, a másik kérés elutasításra kerül.
- **Mérőszám:** sérült bérlési tranzakciók száma = 0, mérési ablak: folyamatos.

QAS-3: Rendelkezésre állás
- **Inger:** A rendszer folyamatos használata egy hónapon át.
- **Környezet:** normál üzem, tervezett karbantartás kivételével.
- **Válasz:** a rendszer havi rendelkezésre állása legalább 99,5%.
- **Mérőszám:** A = üzemidő / (üzemidő + állásidő) 0,995, mérési ablak: 1 hónap.

QAS-4: Adatvédelem
- **Inger:** A rendszer személyes adatokat kezel.
- **Környezet:** normál üzem, adatkezelés.
- **Válasz:** a személyes adatok hozzáférése naplózott, a fizetési adatok nem kerülnek a rendszerbe.
- **Mérőszám:** adatvédelmi incidensek száma = 0, mérési ablak: 1 év.

29.3.10.2 2. lépés: Alternatívák és döntési mátrix

A csapat három alternatívát vázol fel:

A. alternatíva: Moduláris monolit
- Egyetlen deployolható Spring Boot alkalmazás, belső modulokkal (bérlés, felhasználó, számlázás, állomás).
- PostgreSQL adatbázis.
- REST API a mobilalkalmazásnak és az admin felületnek.
- Az IoT-eszközök MQTT-n keresztül kommunikálnak egy beépített brokerrel.
- Előnyök: egyszerű üzemeltetés, alacsony költség, erős konzisztencia tranzakciókkal.
- Hátrányok: korlátozott horizontális skálázás, a modulok közötti csatolás idővel nőhet.

B. alternatíva: Mikroszolgáltatások
- Felhasználó szolgáltatás, bérlés szolgáltatás, számlázás szolgáltatás, állomás szolgáltatás külön konténerben.
- Kubernetes klaszter, API gateway, service mesh.
- Előnyök: független skálázás, technológiai heterogenitás.
- Hátrányok: magas üzemeltetési komplexitás, elosztott tranzakciók, magasabb költség, a csapatnak nincs Kubernetes tapasztalata.

C. alternatíva: Eseményvezérelt architektúra
- Moduláris monolit mag, de a bérlési események aszinkron feldolgozása egy üzenetsor (pl. RabbitMQ) segítségével.
- Az IoT-események stream feldolgozása.
- Előnyök: jobb terheléselosztás, lazább csatolás.
- Hátrányok: növekvő komplexitás, konzisztencia nehezebb, üzemeltetési igény magasabb.

A döntési mátrix súlyozott pontozással:

Szempont (súly)	A. Monolit	B. Mikroszolg.	C. Eseményvezérelt
Skálázhatóság (0,25)	3	5	4
Üzemeltetési egyszerűség (0,25)	5	2	3
Konzisztencia (0,2)	5	3	3
Költség (0,2)	5	3	4
Fejlesztési sebesség (0,1)	4	3	3
Súlyozott összeg	**4,45**	**3,2**	**3,5**

A mátrix szerint az A alternatíva nyer. A csapat ezt választja, de hozzáteszi, hogy a jövőben, ha a skálázási igény jelentősen megnő, a moduláris monolitot fel lehet bontani. Ezt a döntést ADR-ben rögzítik.

29.3.10.3 3. lépés: ADR és C4 diagramok

ADR-001: Moduláris monolit választása

- **Kontextus:** A városi kerékpárkölcsönző rendszer első kiadása 20 állomást és 500 kerékpárt kezel, a bővítés két év múlva 50 állomás és 2000 kerékpár. A csapat 4 fős, nincs Kubernetes tapasztalat. A költségvetés korlátozott. A legfontosabb minőségi attribútumok: konzisztencia, üzemeltetési egyszerűség, költség.

- **Döntés:** Moduláris monolit architektúrát választunk, Spring Boot alkalmazással, belső modulokkal (felhasználó, bérlés, számlázás, állomás). Az adatbázis egyetlen PostgreSQL. A kommunikáció szinkron REST API-val, kivéve az IoT-eszközöket, amelyek MQTT-n üzenetet küldenek. A fizetési adatok kezelését külső szolgáltatóra bízzuk.

- **Alternatívák:**
  - Mikroszolgáltatások: elvetve, mert az üzemeltetési komplexitás és a költség meghaladja a csapat kapacitását, és a jelenlegi skálázási igény nem indokolja.
  - Eseményvezérelt: elvetve, mert a konzisztencia erős követelmény, és az aszinkron feldolgozás növelné a komplexitást anélkül, hogy a fő célokat javítaná.

- **Következmények:**
  - Egyszerűbb deploy, alacsonyabb költség.
  - Erős konzisztencia tranzakciókkal.
  - A skálázás korlátozott, de a QAS-1 1000 egyidejű kérése teljesíthető egy jól méretezett monolit segítségével, ha a bérlési műveletet hatékonyan implementáljuk.
  - A modulhatárok szigorú betartása szükséges, hogy a jövőbeni felbontás lehetősége megmaradjon.

- **Állapot:** Elfogadva.

A C4 context diagram (Mermaid, szemléltetés):

```
graph TD
    User["Felhasználó"]
    Admin["Üzemeltető"]
    Mobile["Mobilalkalmazás"]
    Web["Webes felület"]
    Backend["Kerékpárkölcsönző Backend"]
    DB[("PostgreSQL")]
    IoT["IoT-eszközök (zárak, szenzorok)"]
    Payment["Fizetési szolgáltató (külső)"]
    Map["Térképszolgáltató (külső)"]

    User --> Mobile
    Mobile --> Backend
    Admin --> Web
    Web --> Backend
    Backend --> DB
    IoT -->|MQTT| Backend
```

```
Backend --> Payment
Backend --> Map
```

A container diagram a backend belső moduljait mutatja, de a laborban a hallgatónak kell elkészítenie.

29.3.10.4 4. lépés: Ellenőrzési szempontok és visszacsatolás

A labor végén a résztvevő ellenőrzi a dokumentumot az alábbi lista alapján:

- ☐ A kontextus leírja a stakeholdereket, a rendszerhatárt és a kényszereket.
- ☐ Legalább 3 QAS készült, mindegyik tartalmaz ingert, környezetet, választ és mérőszámot mérési ablakkal.
- ☐ Legalább két alternatívát vizsgált, és a döntési mátrix szempontjai indokoltak.
- ☐ Az ADR tartalmazza a kontextust, döntést, alternatívákat, következményeket és állapotot.
- ☐ A C4 context diagram a rendszerhatárt, a felhasználói szerepköröket és a külső rendszereket jelöli.
- ☐ A container diagram a backend fő komponenseit felelősséggel és interfésszel írja le.
- ☐ A mért és várt eredmények külön vannak jelölve (pl. QAS célérték vs. becsült teljesítmény).
- ☐ A dokumentum reprodukálható: a diagramok diagrams-as-code formában szerepelnek, az ADR-ek visszakereshetők.

A visszacsatolás során a résztvevő összehasonlíthatja a megoldását másokéval, vagy egy facilitátor pontozza a fenti rubrika alapján. A tanulságokat rögzíteni kell, például: „A monolit választása helyes volt a kontextus alapján, de a modulhatárok betartására nagyobb figyelmet kell fordítani a fejlesztés során.”

29.3.11 Összefoglalás

Az architektúra-katák célja, hogy a döntéshozatali képességet gyakoroljuk valósághű, de kockázatmentes környezetben. A folyamat a követelmények pontosításától az alternatívák kidolgozásán át a döntés dokumentálásáig és az értékelésig terjed. A legfontosabb tanulság, hogy nincs egyetlen helyes megoldás: a kontextus határozza meg, mi a megfelelő. A katarész során a QAS-ok, ADR-ek és C4 diagramok használata biztosítja, hogy a döntések mérhetők, visszakereshetők és vitathatók legyenek. A gyakorlás során elsajátított gondolkodási keret a valós projektekben is alkalmazható.

29.3.12 Ellenőrzőlista a kata eredményéhez

- A kontextus egyértelmű, a rendszerhatár jelölt.
- Minden fontos minőségi attribútum QAS formában van, mérőszámmal és mérési ablakkal.

- Legalább két, alapvetően eltérő alternatíva elemzése megtörtént.
- A döntés ADR-ben rögzített, az elvetett alternatívák indoklásával.
- A C4 context és container szint diagramok diagrams-as-code formában készültek.
- A mért és várt eredmények külön vannak jelölve.
- A dokumentum reprodukálható, verziózható, és nem tartalmaz valós hitelesítő adatot vagy belső URL-t.

29.3.13 Önellenőrző kérdés

Melyik a fontosabb egy architektúra-katán: a „helyes” megoldás megtalálása, vagy a döntési folyamat dokumentálása és a trade-offok feltárása? Válaszát indokolja egy, a fejezetben bemutatott példa alapján.

29.4 Best practice-ek és anti-minták

Az Architektúra-katák és döntési gyakorlatok gyakorlatai során valós vagy fiktív architektúrák döntési helyzeteit elemeztük. A döntések értékeléséhez azonban szükségünk van egy olyan rendszerezett keretre, amely megkülönbözteti a bevált gyakorlatokat a kontextusfüggetlen divatoktól, és felismerhetővé teszi azokat a visszatérő hibákat, amelyek újra és újra felbukkannak a rendszerekben. Ebben a fejezetben ezt a keretet építjük fel: először tisztázzuk a best practice-ek és anti-minták természetét, majd részletesen bemutatjuk a három leggyakoribb architekturális anti-mintát – a sárgolyót, az elosztott monolitot és az önéletrajz-vezérelt tervezést –, végül egy olyan katalógussablont adunk, amelyet a laborban egy konkrét rendszerre alkalmazva reprodukálható eredményt kapunk.

A fejezet nem ismétli meg a korábbi részekben bemutatott architekturális stílusokat vagy minőségi attribútumokat. Célja, hogy azokat a döntési helyzeteket tegye értékelhetővé, amelyekben egy-egy megoldás „jó” vagy „rossz” minősítése csak a körülmények ismeretében lehetséges.

29.4.1 A best practice-ek természete: heurisztika, nem szabály

A szoftverfejlesztésben a „best practice” kifejezés olyan megoldási mintát jelöl, amelyet sok projektben, sokszor bevált, és amelyet a közösség általában hatékonynak ismer el. Fontos azonban, hogy a best practice nem természeti törvény; inkább heurisztika, amely adott kontextusban jó eredményt ad, más körülmények között viszont káros lehet. Egy gyakorlat akkor nevezhető best practice-nek, ha:

1. **Mögöttes elvvel rendelkezik** – Meg tudjuk fogalmazni, miért működik: például a rétegzés azért hasznos, mert csökkenti a változtatások tovagyűrűzését; a CI/CD azért, mert lerövidíti a visszacsatolási hurkot.
2. **Kontextusfeltételei vannak** – Megadjuk, milyen csapatméret, rendszerkritikusság, terhelés, szabályozási környezet vagy életciklus-szakasz mellett érvényes.
3. **Ismert kivételei vannak** – Azt is tudjuk, mikor nem érdemes alkalmazni. A microservices például nem jó választás egy háromfős csapat induló termékéhez; a CQRS nem kell egy egyszerű CRUD alkalmazáshoz.

4. **Mérhető vagy megfigyelhető hatása van** – A gyakorlat eredménye valamilyen minőségi attribútumban (például deploy gyakoriság, MTTR, hibaarány) nyomon követhető.

Ha egy állítás csak annyit mond, hogy „ez a legjobb megoldás”, de nem indokolja az elvet, a kontextust és a kivételeket, akkor az inkább divat, mint bevált gyakorlat. A fejezetben ezért minden best practice-t a következő sémával írunk le: **elv → kontextus → kivétel → következmény**. Ez a séma egyben az anti-minta-katalógus bejegyzéseinek is az egyik pillére, mert egy anti-minta gyakran éppen egy best practice helytelen kontextusban való alkalmazásából vagy egy elv figyelmen kívül hagyásából keletkezik.

29.4.2 Az anti-minták rendszerezése

Az anti-minta olyan ismétlődő megoldási minta, amely elsőre vonzónak tűnik, de a gyakorlatban negatív következményekkel jár. Az anti-minta nem egyszerűen „rossz kód”; sokkal inkább egy olyan szerkezeti, folyamati vagy szervezeti döntés, amely rontja a rendszer fejlődőképességét, üzemeltethetőségét vagy biztonságát. A rendszerezéshez több dimenziót is használhatunk:

- **Szerkezeti anti-minták** – a kód és a komponensek elrendezésének hibái: például a sárgolyó (Big Ball of Mud), a ciklikus függőségek, a mindentudó komponens.
- **Elosztott rendszerek anti-mintái** – a hálózati határok rossz kezelése, például az elosztott monolit, a közös adatbázis, a szinkron láncok túlzott mélysége.
- **Szervezeti és folyamati anti-minták** – a csapatok és a folyamatok szintjén jelentkező hibák, például a Conway-törvény figyelmen kívül hagyása, a tervezési elemzési bénultság (analysis paralysis), a túlzott governance.
- **Technológiaválasztási anti-minták** – amikor a döntést nem a probléma, hanem más motiváció vezérli; ilyen az önéletrajz-vezérelt tervezés, a golden hammer (mindig ugyanaz a kalapács) vagy a cargo cult (utánzás megértés nélkül).
- **Adat- és integrációs anti-minták** – az adatkezelés és az interfészek hibái, például a közös adatbázis, az API-verzió nélküli törés, a batch-éjszakai integráció.

A katalógus készítésekor ezeket a dimenziókat használjuk a csoportosításhoz, de minden bejegyzéshez hozzárendelünk egy **kontextusmezőt** is, mert ugyanaz a minta eltérő környezetben más súlyú lehet.

Az anti-minta felismerésének általános lépései:

1. **Tünet azonosítása** – Milyen minőségi attribútum romlik? Például a változtatások lassulnak, a hibák gyakoribbak, a csapat sebessége csökken.
2. **Ok feltárása** – Milyen döntés vagy hiányzó döntés vezetett ide? Itt segít a döntési napló és a C4 diagramok áttekintése.
3. **Következmény számszerűsítése** – Ha lehetséges, mérjük a hatást: például a deploy idő növekedése, a hibaarány, a ciklomatikus komplexitás.
4. **Ellenszer kiválasztása** – Melyik best practice vagy refaktorálási stratégia oldja fel az anti-mintát?

A következő három alfejezet a leggyakoribb és legtanulságosabb anti-mintákat mutatja be részletesen, a fenti séma szerint.

29.4.3 Big Ball of Mud (sárgolyó)

A Big Ball of Mud (BBoM) talán a legismertebb architekturális anti-minta. Olyan rendszert jelöl, amelyben nincsenek világos komponenshatárok, a felelősségek szétfolynak, a függőségek áttekinthetetlenek, és a kódot csak a szerzője vagy senki sem érti. A sárgolyó nem egyik napról a másikra keletkezik; jellemzően fokozatosan, sok apró, rövid távú döntés eredményeként alakul ki.

Tünetek. A BBoM legjellemzőbb jelei:

- A változtatások költsége exponenciálisan nő: egy apró módosítás váratlan helyeken okoz hibát.
- Nincsenek tiszta interfészek; a komponensek közvetlenül érik el egymás belső állapotát.
- A tesztelés nehézkes, mert a rendszer nem bontható izolált egységekre.
- Az onboarding idő hosszú: az új fejlesztő hónapokig nem érti a rendszer működését.
- A technikai adósság tudatos vagy tudattalan halmozódása: „majd később kitakarítjuk”, de a később soha nem jön el.

Okok. A sárgolyó kialakulásának gyakori okai:

- **Nincs architekturális vízió vagy governance.** A csapat minden döntést lokálisan, rövid távra optimalizál.
- **Időnyomás és feature-first szemlélet.** A határidők miatt a refaktorálás elmarad, a gyors megoldások állandósulnak.
- **A kód tulajdonlás hiánya.** Senki sem felelős a rendszer egészének minőségéért, csak a saját feature-éért.
- **Hiányzó automatizált tesztek.** Refaktorálás nélkül a kód egyre merevebbé válik.
- **A domain modell hiánya.** Ha nincs közös nyelv és bounded context, a fogalmak összemosódnak.

Következmények. A BBoM hatása minden minőségi attribútumra negatív: a karbantarthatóság, a teljesítmény, a biztonság, a skálázhatóság és a megbízhatóság egyaránt romlik. A rendszer fejlődőképessége lelassul, a csapat morálja csökken, és a technikai adósság kamata felemészti a fejlesztési kapacitást.

Felismerés. A sárgolyót a következő módszerekkel azonosíthatjuk:

- **Függőségi gráf elemzése:** ciklikus függőségek, nagy bejövő és kimenő kapcsolatú komponensek.
- **Kódmetrikák:** magas ciklomatikus komplexitás, nagy osztályok, hosszú metódusok, a kohézió hiánya.
- **C4 component szintű diagram:** ha a diagram nem rajzolható meg, vagy a komponensek közötti kapcsolatok áttekinthetetlenek, az már önmagában tünet.
- **Változási gyakoriság:** ha bizonyos fájlok vagy modulok minden egyes módosításkor változnak, az nagyfokú csatoltságra utal.

Ellenszerek. A sárgolyó felszámolása hosszú folyamat, és ritkán történik egyetlen nagy refaktorálással. A bevált stratégiák:

- **Strangler fig minta:** a régi rendszer köré új, tiszta határokkal rendelkező szolgáltatásokat építünk, és fokozatosan átirányítjuk a forgalmat.
- **Modularizáció:** a meglévő kódban azonosítjuk a természetes határokat, és modulokba szervezzük, még ha kezdetben technikai szinten is.

- **Tesztlefedettség növelése:** a refaktorálás előtt karakterizációs teszteket írunk, amelyek rögzítik a jelenlegi viselkedést.
- **Architekturális fitness functions:** automatikus ellenőrzések, például a ciklikus függőségek tiltása, a rétegzési szabályok betartatása.
- **Kulturális változás:** a csapatnak el kell fogadnia, hogy a belső minőség nem luxus, hanem a sebesség feltétele.

A sárgolyó elleni küzdelem nem egyszeri projekt, hanem folyamatos tevékenység. A cél nem a tökéletesen tiszta architektúra, hanem az, hogy a rendszer fejlődőképessége fennmaradjon.

29.4.4 Distributed Monolith (elosztott monolit)

A distributed monolith egy olyan rendszer, amely névleg mikroszolgáltatásokból áll, de a szolgáltatások közötti kapcsolatok olyan szorosak, hogy a rendszer valójában egyetlen monolitikus egységként viselkedik, csak éppen hálózati határokkal. Ez az anti-minta gyakran súlyosabb következményekkel jár, mint egy hagyományos monolit, mert a monolit előnyei (egyszerűbb tranzakciókezelés, könnyebb debuggolás, kisebb üzemeltetési komplexitás) elvesznek, miközben a mikroszolgáltatások hátrányai (hálózati hibák, elosztott tranzakciók, nagyobb üzemeltetési teher) megmaradnak.

Tünetek. A distributed monolit jellemzői:

- A szolgáltatások között szinkron HTTP/REST hívások hosszú láncai alakulnak ki; egyetlen felhasználói kérés több tucat szolgáltatást érint.
- A szolgáltatások közös adatbázist használnak, vagy közvetlenül elérik egymás tábláit, így a séma változása az egész rendszert érinti.
- A deploy-ok nem függetlenek: egy szolgáltatás új verziója csak akkor telepíthető, ha egy másik szolgáltatás új verziója is készen van.
- A hibák terjedése kontrollálatlan: egyetlen szolgáltatás kiesése az egész rendszer leállását okozza, mert nincsenek körkörös megszakítók vagy időtúllépések.
- A teszteléshez az összes szolgáltatást együtt kell futtatni, mert az integrációs tesztek nem izolálhatók.

Okok. A distributed monolit kialakulásának gyakori okai:

- **Túl korai mikroszolgáltatás-bevezetés:** a csapat akkor bontja szét a rendszert, amikor még nem ismeri a domain határokat, vagy amikor a csapat mérete nem indokolja a szétválasztást.
- **Rossz szolgáltatáshatárok:** a bounded context-eket nem a domain, hanem a technológia vagy az adatbázis-táblák mentén húzták meg.
- **Közös adatbázis megtartása:** a szolgáltatások önállóak, de az adatuk közös sémában van, ami rejtett csatolást okoz.
- **Szinkron kommunikáció mindenhol:** a csapat nem használ aszinkron üzenetkezelést, eseményeket vagy CQRS-t, ezért minden adatigény közvetlen hívással valósul meg.
- **Hiányzó API governance:** nincsenek szerződések, verziózási szabályok vagy szolgáltatástulajdonosi felelősségek.

Következmények. A distributed monolith minden hátrányt egyesít: a deploy-ok lassúak és kockázatosak, a rendszer üzemeltetése bonyolult, a hibakeresés nehéz, a skálázhatóság korlátozott, és a fejlesztési sebesség alacsonyabb, mint egy jól szervezett monolit esetében. Emellett a csapat morálja is romlik, mert a mikroszolgáltatások ígérete nem teljesül.

Felismerés. A distributed monolit azonosítására a következő jeleket érdemes figyelni:

- A szolgáltatások közötti hívási gráf mélysége nagy, és a ciklusok gyakoriak.
- Egyetlen szolgáltatás deploy-ja más szolgáltatások deploy-ját igényli.
- A közös adatbázis séma változásai az összes szolgáltatást érintik.
- A rendszerben nincsenek függetlenül skálázható részek; minden szolgáltatás hasonló terhelési mintát mutat.
- A CI/CD pipeline-ok nem függetlenek, a release-eket összehangoltan kell kiadni.

Ellenszerek. A distributed monolit kezelése több lépést igényel:

- **Domain újraértékelés:** a szolgáltatáshatárok felülvizsgálata a DDD bounded context-ek alapján.
- **Adatbázis-felbontás:** minden szolgáltatás saját adattárolóval rendelkezzen; a séma- és adatmigrációt fokozatosan kell végrehajtani.
- **Aszinkron kommunikáció:** a szinkron láncok helyett eseményeket, üzenetsorokat és CQRS-t vezessünk be, ahol az adatok végső konzisztenciája elfogadható.
- **API-szerződések és verziózás:** minden szolgáltatás explicit interfésszel rendelkezzen, és a változások kompatibilis verziókkal történjenek.
- **Körkörös megszakítók és time-outok:** a hálózati hibák terjedésének megakadályozására.
- **Ha a rendszer valójában nem igényli a mikroszolgáltatásokat:** fontoljuk meg a visszaalakítást moduláris monolittá, amely tiszta belső határokkal rendelkezik, de nem fizet hálózati költséget.

A distributed monolith az egyik legjobb példa arra, hogy egy best practice – a mikroszolgáltatások – helytelen kontextusban való alkalmazása anti-mintává válik.

29.4.5 Resume-Driven Design (önéletrajz-vezérelt tervezés)

A Resume-Driven Design (RDD) olyan döntési minta, amelyben a technológiaválasztást nem a rendszer követelményei, hanem a fejlesztő vagy a vezető önéletrajzának bővítése motiválja. A jelenség hátterében az áll, hogy bizonyos technológiák presztízse magas, és a piacon keresettek; ezért a csapat vagy az egyén hajlamos olyan eszközöket bevezetni, amelyekre a projektnek valójában nincs szüksége. Az RDD veszélye abban rejlik, hogy a döntés látszólag szakmai, de valójában karrierista motivációjú, és a következményeket a rendszer és a szervezet viseli.

Tünetek. Az RDD jellemző jelei:

- A technológiai stack indokolatlanul gyorsan változik, követve a legfrissebb trendeket.
- A döntések mögött nincs mérés vagy összehasonlító elemzés; a „modern” és a „scalable” szavak gyakori érvek.
- A csapat olyan komponenseket épít, amelyeket senki sem használ, vagy amelyek túl bonyolultak a probléma méretéhez képest.
- A karbantartás és az üzemeltetés terhe megnő, mert a ritka technológiákhoz nincs elegendő szakértelem vagy közösségi támogatás.
- A fejlesztők motivációja rövid távon magas, de a hosszú távú elköteleződés hiányzik, mert a technológia nem illeszkedik a domainhez.

Okok. Az RDD-t több tényező is táplálhatja:

- **Karrierépítés:** a fejlesztő vagy az architekt olyan technológiát választ, amely jól mutat az önéletrajzban.
- **Újdonság varázsa:** az új technológiák iránti lelkesedés elhomályosítja a mérnöki ítélőképességet.

- **Külső nyomás:** a menedzsment vagy a befektetők elvárják a „modern” technológiák használatát, mert azt hiszik, hogy ez versenyelőnyt jelent.
- **Hiányzó döntési keret:** a szervezetben nincs formalizált technológiaválasztási folyamat, így a döntések ad hoc módon születnek.

Következmények. Az RDD gyakran vezet túltervezett, nehezen karbantartható rendszerekhez, magas üzemeltetési költségekhez, a csapat szétszóródásához és a valódi üzleti igények elhanyagolásához. A rendszer tele lesz olyan komponensekkel, amelyek nem adnak értéket, de állandó figyelmet igényelnek. Ráadásul a gyakori technológiai váltások megnehezítik a tudás felhalmozását a szervezetben.

Felismerés. Az RDD azonosítására a következő kérdések segíthetnek:

- A technológiát a probléma vagy a technológia miatt választottuk?
- Van-e mérés vagy PoC, amely alátámasztja a döntést?
- Milyen alternatívákat vetettünk el, és miért?
- Az új technológia bevezetése milyen üzemeltetési és karbantartási költségekkel jár?
- A csapat tagjai közül hányan rendelkeznek tapasztalattal az adott technológiában?

Ellenszerek. Az RDD elleni védekezés legfontosabb eszköze a **döntési fegyelem**. A technológiaválasztást ugyanúgy kell kezelni, mint bármely más architektúradöntést: kontextussal, alternatívákkal, következményekkel és mérésekkel. Az ADR-ekben explicit módon rögzíteni kell a motivációt, és meg kell vizsgálni, hogy a döntés a rendszer érdekeit szolgálja-e. Emellett hasznos lehet egy **technológiai radar** fenntartása, amely a szervezet számára elfogadott, kísérleti és tiltott technológiákat kategorizálja. Végül a csapatkultúrában is fontos a pszichológiai biztonság: a mérnököknek lehetőségük legyen kimondani, ha egy döntés mögött karrierista motiváció áll.

Az RDD különösen akkor veszélyes, ha a szervezetben nincs erős architekturális governance, és a fejlesztők egyéni preferenciái szabadon érvényesülnek. A legjobb védekezés a nyílt, adatokra épülő döntéshozatal.

29.4.6 A best practice-ek kontextusfüggése: egy rendszerező keret

Az előző példák is mutatják, hogy egy megoldás minősége nem önmagában, hanem a kontextusban értelmezhető. A best practice-ek és anti-minták közötti határ gyakran vékony. A mikroszolgáltatások lehetnek jó választások egy nagy, független csapatokkal rendelkező szervezetben, de anti-mintává válnak egy kis csapat induló termékénél. A CQRS hasznos lehet nagy olvasási-írási aszimmetriánál, de felesleges bonyolultságot visz egy egyszerű CRUD alkalmazásba.

A kontextusfüggés kezelésére a következő dimenziókat érdemes figyelembe venni:

- **Csapatméret és -eloszlás:** hány csapat dolgozik a rendszeren, földrajzilag hogyan oszlanak meg, milyen a kommunikációs struktúra.
- **Rendszerkritikusság:** milyen következményekkel jár a hiba, mekkora az elfogadható állásidő, milyen szabályozási követelmények vonatkoznak a rendszerre.
- **Terhelési profil:** mekkora a forgalom, milyen a növekedési ütem, mennyire változékony a terhelés.
- **Életciklus-szakasz:** induló termék, növekvő rendszer, érett platform vagy legacy korszerűsítés.
- **Szervezeti kultúra:** mennyire nyitott a szervezet a kísérletezésre, milyen a döntéshozatali folyamat, mekkora a pszichológiai biztonság.
- **Technológiai ökoszisztéma:** milyen meglévő rendszerekhez kell integrálódni, milyen szakértelem áll rendelkezésre.

Egy felelős architektúra-döntésnél ezeket a dimenziókat explicit módon dokumentálni kell, és a döntést ezek fényében kell meghozni. A best practice-eket ezért nem szabad ellenőrizetlenül átvenni; mindig fel kell tenni a kérdést: **milyen elv alapján működik, és az adott kontextusban ez az elv érvényes-e?**

29.4.7 Labor: anti-minta-katalógus készítése

A labor célja egy olyan anti-minta-katalógus elkészítése, amely egy konkrét – lehetőleg a résztvevő által ismert – rendszerre alkalmazható. A katalógus nem csupán egy lista, hanem egy reprodukálható, kontextussal ellátott dokumentum, amely segít a tipikus hibák felismerésében és az ellenszerek kiválasztásában.

29.4.8 Kiinduló állapot

Válassz egy általad ismert rendszert, amelynek kódjához, dokumentációjához vagy legalább architekturális leírásához hozzáférsz. Lehet ez egy munkahelyi projekt, egy nyílt forráskódú rendszer, vagy akár egy korábbi labor során készített architektúra. Ha nincs ilyen, használhatod a könyv korábbi fejezeteiben bemutatott fiktív rendszerek egyikét. A laborhoz szükséged lesz egy jegyzetfüzetre vagy dokumentumsablonra, valamint – ha elérhető – a rendszer C4 diagramjaira, ADR-jeire és kódmetrikáira.

29.4.9 Lépések

1. **Kontextus rögzítése.** Töltsd ki a katalógus fejlécét: rendszer neve, domain, csapatméret, technológiai stack, életciklus-szakasz, üzemeltetési modell. Ez a mező biztosítja, hogy a katalógus kontextusfüggő maradjon.
2. **Tünetgyűjtés.** Interjú vagy megfigyelés alapján gyűjts össze legalább öt olyan tünetet, amely anti-minta jelenlétére utalhat. Például: „a deploy folyamat több mint két órát vesz igénybe”, „egy apró módosítás három másik szolgáltatást is érint”, „a tesztlefedettség 20% alatt van”.
3. **Anti-minta azonosítás.** Minden tünethez keress egy vagy több lehetséges anti-mintát a fejezetben bemutatott és a szakirodalomban ismert kategóriákból. Használd a következő sémát: **anti-minta neve, tünetek, okok, következmények, ellenszerek**. A katalógus bejegyzéseit töltsd ki, és minden mezőhöz írj rövid indoklást.
4. **Kontextusértékelés.** Minden bejegyzésnél értékeld, hogy az adott anti-minta a konkrét rendszerben milyen súlyos. Használj egyszerű skálát: alacsony, közepes, magas. Indokold meg a minősítést.
5. **Mért és várt eredmény elkülönítése.** A katalógusban jelöld, hogy mely állítások származnak közvetlen megfigyelésből, mérésből (mért eredmény), és melyek a szakirodalom vagy az általános tapasztalat alapján elvárt következmények (várt eredmény). Például: „Mért: a ciklomatikus komplexitás mediánja 18 (SonarQube, 2024. 09. havi mérés). Várt: a magas komplexitás a sárgolyó tünete, amely a változtatások költségének növekedéséhez vezet.”
6. **Döntési napló frissítése.** Ha a katalógus alapján valamilyen beavatkozást tervezel, rögzítsd ADR formában: kontextus, döntés, alternatívák, következmények, állapot. Az ADR-hez kapcsold a katalógus megfelelő bejegyzéseit.

7. **Ellenőrzés.** A katalógust a következő szempontok szerint ellenőrizd: minden bejegyzés rendelkezik-e kontextussal, tünetekkel, okokkal, következményekkel és ellenszerekkel; a mért és várt eredmények elkülönülnek-e; a kontextusértékelés indokolt-e; a hivatkozások (ha vannak) pontosak-e.

29.4.10 Elvárt eredmény

Egy olyan anti-minta-katalógus, amely legalább öt bejegyzést tartalmaz, mindegyik a fenti séma szerint kitöltve. A katalógus legyen verziózott, és a döntési naplóval együtt tárolt. A dokumentumban a diagramok diagrams-as-code formában (például Mermaid vagy PlantUML) jelenjenek meg, ha a tünetek lokalizálásához szükségesek.

29.4.11 Ellenőrzési szempontok

- A katalógus bejegyzései nem csupán címkék, hanem indokolt elemzések.
- A mért eredményekhez tartozik mérési környezet, konfiguráció, időpont; ha ilyen nem áll rendelkezésre, a várt eredményt egyértelműen jelölni kell.
- A kontextus értékelése nem önkényes, hanem a rendszer sajátosságaira hivatkozik.
- A döntési napló bejegyzései visszavezethetők a katalógusban szereplő anti-mintákra.

29.4.12 Visszaállítási út

Mivel a labor csak dokumentációt és elemzést készít, nincs szükség környezeti visszaállításra. Ha azonban a gyakorlat során kódot vagy konfigurációt módosítasz, azt izolált környezetben tedd, és a módosításokat verziókezelőben tartsd nyomon, hogy visszaállíthatók legyenek.

29.4.13 Ellenőrzőlista a best practice-ek és anti-minták értékeléséhez

- Minden best practice állítás mögött azonosítottad a mögöttes elvet, a kontextusfeltételeket és a kivételeket?
- Az anti-minta bejegyzés tartalmazza a tüneteket, okokat, következményeket és ellenszereket?
- A kontextus dimenzióit (csapatméret, kritikusság, terhelés, életciklus, kultúra, ökoszisztéma) figyelembe vetted?
- A mért és a várt eredményeket elkülönítetted, és a mérésekhez tartozik környezet, időpont és bizonytalanság?
- A döntési naplóban rögzített alternatívák és következmények összhangban vannak a katalógusban szereplő elemzésekkel?
- Az azonosított anti-mintákhoz rendeltél konkrét, megvalósítható ellenszert, és azt is jelezted, ha az ellenszer bevezetése újabb trade-offokat hordoz?

29.4.14 Önellenőrző kérdés

Vegyél egy konkrét rendszert, amelyben a csapat nemrég bevezetett egy új technológiát. A fenti keret alapján ítéld meg, hogy a döntés best practice-e vagy inkább Resume-Driven Design anti-minta. Milyen bizonyítékok alapján döntenél, és milyen alternatívákat vizsgálnál meg?

A fejezetben bemutatott rendszerezés és katalógussablon a következő egység (Pénzügy és biztosítás) alapjául szolgál: ott ezeket a fogalmakat iparági esettanulmányokra alkalmazzuk, és azt vizsgáljuk, hogyan jelennek meg a best practice-ek és anti-minták a pénzügyi, egészségügyi és más szektorok gyakorlatában.

30 Iparági architektúrák

30.1 Pénzügy és biztosítás

A pénzügyi és biztosítási rendszerek architektúrája három, egymást erősítő követelménycsoport köré szerveződik: **tranzakciós integritás**, **szabályozási megfelelés** és **auditálhatóság**. A háromból bármelyik sérül, a rendszer nem pusztán technikailag hibás, hanem üzletileg és jogilag is elfogadhatatlan. Ez a fejezet ezeket a speciális elvárásokat és a belőlük következő architekturális mintákat mutatja be, egy ellenőrizhető laborral zárva.

30.1.1 1. A pénzügyi tranzakció természete

A pénzügyi rendszerben egy tranzakció nem egyszerűen egy adatbázis-művelet, hanem **üzleti elszámolási egység**: pénzmozgás, értékpapír-átruházás, biztosítási díj vagy kárkifizetés. A rendszernek garantálnia kell, hogy a tranzakció **atomi, konzisztens, izolált és tartós** (ACID) legyen, de a gyakorlatban a klasszikus adatbázis-tranzakció gyakran nem elég, mert a művelet több szolgáltatáson, sőt több szervezeten átível.

A pénzügyi tranzakciók jellemzői:

- **Nagy érték és visszafordíthatatlanság:** egy rosszul végrehajtott átutalás azonnali anyagi kárt okoz, és a visszavonás nem mindig lehetséges.
- **Szinkron és aszinkron részek keveréke:** a befogadás lehet szinkron REST-hívás, de az elszámolás gyakran aszinkron feldolgozási soron keresztül történik.
- **Többszereplős folyamat:** a küldő bank, a fogadó bank, a klíringház, a csalásmegelőzés és a megfelelőségi szűrés egyaránt részese a láncnak.
- **Idempotencia-követelmény:** hálózati hiba vagy újrapróbálkozás esetén ugyanaz a kérés nem okozhat duplikált terhelést. Ehhez **idempotencia-kulcsot** (pl. `Idempotency-Key` fejléc) kell rendelni minden tranzakcióhoz, és a rendszernek a kulcs alapján fel kell ismernie a már feldolgozott műveletet.

Elszámolás és kiegyenlítés (settlement)
A tranzakció **befogadása** (authorization) és a tényleges **elszámolása** (settlement) időben elválhat. A rendszernek nyilván kell tartania a függő tételeket, és a nap végén (vagy meghatározott ciklusokban) **kiegyenlítést** (reconciliation) kell futtatnia. A kiegyenlítés során a belső főkönyvi rekordokat összevetik a külső felek (bankok, kártyatársaságok, klíring) által küldött elszámolási fájlokkal, és az eltéréseket kivizsgálják. A főkönyv (ledger) ezért nem egyszerű adattábla, hanem **append-only, auditálható napló**, amelyből minden tranzakció visszakereshető.

30.1.2 2. Szabályozási és auditkövetelmények

A pénzügyi szektor a legszigorúbban szabályozott iparágak közé tartozik. Az architektúrának nemcsak a funkcionális igényeket kell kiszolgálnia, hanem **bizonyíthatóan megfelelni** a szabályozóknak. A megfelelés nem utólagos dokumentáció, hanem tervezési kényszer, amely befolyásolja a rendszerhatárokat, az adatkezelést és a naplózást.

A leggyakrabban érintett szabályozások:

- **PCI DSS**: a kártyaadatok (PAN, CVC) tárolására, feldolgozására és továbbítására vonatkozik. Az architektúrának minimalizálnia kell a kártyaadatok érintését (tokenizáció), és a kártyaadat-környezetet (CDE) szegmentálnia kell a többi rendszertől.
- **PSD2 / SCA**: az Európai Unióban az erős ügyfél-hitelesítést (SCA) írja elő az elektronikus fizetéseknél. Ez azt jelenti, hogy a tranzakciót legalább két független tényezővel kell hitelesíteni (tudás, birtoklás, biometria).
- **GDPR**: a személyes adatok kezelését szabályozza, beleértve az ügyféladatok tárolását, törlését és az adathordozhatóságot.
- **SOX (Sarbanes–Oxley)**: az USA-ban a pénzügyi beszámolás belső kontrolljait írja elő, ami a pénzügyi rendszerek naplózási és hozzáférés-ellenőrzési követelményeit erősíti.
- **Solvency II**: az EU biztosítási szektorának kockázatkezelési és tőkemegfelelési keretrendszere, amely a biztosítói rendszerek adatminőségére és belső modelljeire is hatással van.

Az auditkövetelmények lényege: **minden üzleti eseménynek nyomon követhetőnek kell lennie**. Ez a gyakorlatban a következőket jelenti:

- **Minden tranzakcióhoz tartozik egyedi azonosító**, amely a teljes életcikluson át megmarad (pl. `transaction_id`, `event_id`).
- **Minden állapotváltozás naplózott**: ki, mikor, milyen műveletet hajtott végre, milyen értékről milyen értékre változott. Az auditnapló **append-only**, és nem módosítható.
- **Az adatmegőrzési idő jogszabályban rögzített** (pl. 5–10 év), és az archívumot is elérhetővé kell tenni ellenőrzés céljából.
- **A hozzáférések naplózottak**: a rendszernek rögzítenie kell, hogy melyik felhasználó vagy szolgáltatás milyen adathoz fért hozzá.

Az auditnapló nem az alkalmazás logfájlja, hanem **strukturált, kereshető, időbélyegzett és hitelesített rekordok sorozata**. Gyakran külön adatbázisban vagy append-only ledgerben tárolják, és digitális aláírással vagy hash-lánccal védik a manipuláció ellen.

Fontos: a szabályozási hivatkozásoknál mindig a hatályos hivatalos szöveget kell ellenőrizni, mert a követelmények időben változnak. Ez a fejezet nem jogi tanácsadás, csupán az architekturális következményeket vázolja.

30.1.3 3. Elszámolás és megfelelés architektúrája

Az elszámolási folyamat központi eleme a **főkönyv (ledger)**. A főkönyv nem csupán egyenlegnyilvántartás, hanem minden pénzmozgás **eseménynaplója**. A kettős könyvelés elve szerint minden tranzakcióhoz tartozik egy **tartozik** és egy **követel** oldal, így a rendszer invariánsa az, hogy a tartozások és követelések összege mindig nulla.

Az architektúra szempontjából a főkönyv megvalósítására két alapvető minta létezik:

- **Relációs adatbázis, ACID tranzakciókkal**: a tranzakciós táblák sorait atomi műveletekkel frissítik, a kettős könyvelést adatbázis-megszorításokkal (pl. `CHECK` kényszer) biztosítják. Ez egyszerű, de horizontális skálázásnál korlátokba ütközik.
- **Event sourcing**: minden állapotváltozást eseményként tárolnak, a főkönyvi egyenlegeket az események újrajátszásából számítják. Ez kiváló auditálhatóságot és visszamenőleges elemzést tesz lehetővé, de bonyolultabb lekérdezéseket és nagyobb tárolási igényt jelent.

Az **elszámolási folyamat** tipikus lépései:

1. **Befogadás**: a rendszer validálja a kérést, elvégzi a hitelesítést és a csalásmegelőzési szűrést.
2. **Foglalás (reservation)**: a küldő fél egyenlegén zárolja a tranzakció összegét.
3. **Tranzakció végrehajtása**: a főkönyvben rögzítik a tartozik/követel tételeket.
4. **Elszámolás (settlement)**: a külső banki/klíring rendszer megerősíti a tényleges pénzmozgást.
5. **Kiegyenlítés (reconciliation)**: a belső rekordokat összevetik a külső elszámolási adatokkal.

A folyamat során a rendszernek **idempotensnek** kell lennie: ha a lépések bármelyike megszakad, az újrapróbálkozás nem okozhat duplikált bejegyzést. Ehhez a tranzakciókat **üzleti kulccsal** (pl. `client_transaction_id`) azonosítják, és az adatbázisban egyedi megszorítást tesznek rá.

Elosztott tranzakciók és kompenzáció
Ha a folyamat több szolgáltatáson átível, a klasszikus kétfázisú commit (XA) gyakran nem praktikus a hosszú zárolások és a rendelkezésre állási kockázatok miatt. Helyette a **saga** mintát használják, ahol minden lépésnek van **kompenzáló művelete**. A saga koordinátor gondoskodik arról, hogy ha egy lépés sikertelen, a korábbi lépések kompenzálódjanak. A pénzügyi rendszerekben a kompenzáció soha nem lehet egyszerű törlés – a hibás tranzakciót is naplózni kell, és a visszavonást új tranzakcióként kell rögzíteni.

30.1.4 4. Rendelkezésre állás és adatintegritás

A pénzügyi rendszerek rendelkezésre állása közvetlenül üzleti kockázat. Egy órás leállás akár milliós nagyságrendű veszteséget okozhat, és a szabályozók is büntethetik. Ugyanakkor a rendelkezésre állás soha nem mehet az adatintegritás rovására: **helytelen adat rosszabb, mint a lassú válasz**.

A rendelkezésre állás mérésére használt mutatók:

- **SLA**: a szolgáltató által vállalt rendelkezésre állási szint, pl. 99,99% (évente kb. 52 perc állásidő).
- **SLO**: a szolgáltatás belső célértéke, amely szigorúbb lehet, mint az SLA.
- **RTO (Recovery Time Objective)**: mennyi idő alatt kell helyreállni egy meghibásodás után.
- **RPO (Recovery Point Objective)**: mekkora adatvesztés fogadható el, pl. 0 másodperc (szinkron replikáció) vagy 5 perc (aszinkron replikáció).

Az adatintegritás biztosítása érdekében a pénzügyi rendszerek általában a következő mechanizmusokat alkalmazzák:

- **Szinkron replikáció** az elsődleges és a tartalék adatbázis között: a tranzakció csak akkor tekinthető sikeresnek, ha a tartalék oldalon is rögzült. Ez növeli a késleltetést, de garantálja az RPO=0-t.
- **WAL (Write-Ahead Log)** és **point-in-time recovery**: a naplófájl lehetővé teszi az adatbázis visszaállítását egy adott időpontra.
- **Checksum és digitális aláírás**: a főkönyvi rekordok integritásának ellenőrzésére.
- **Append-only ledger**: a tranzakciós bejegyzések nem módosíthatók, csak új bejegyzésekkel lehet korrigálni.
- **Rendszeres mentés és visszaállítási teszt**: a mentés önmagában nem elég; a visszaállítást is gyakorolni kell.

A **rendelkezésre állás architekturális mintái**:

- **Aktív-passzív**: egy elsődleges és egy készenléti példány; meghibásodáskor a passzív átveszi a szerepet. Egyszerű, de a failover időt vesz igénybe.
- **Aktív-aktív**: két vagy több példány egyidejűleg szolgál ki, az adatok szinkron replikációval egyeznek meg. Magasabb rendelkezésre állást ad, de a konfliktuskezelés bonyolult.
- **Többrégiós aktív-aktív**: a tranzakciókat régiók szerint szegmentálják, a főkönyvi rekordokat globálisan replikálják. A hálózati késleltetés és a konzisztencia közötti trade-offot gondosan kell kezelni.

A pénzügyi szektorban gyakran használják a **CAP-tétel** értelmében a **CP (konzisztencia + partíciótűrés)** oldalt: partíció esetén inkább a rendelkezésre állást áldozzák fel, mint hogy ellentmondásos adat kerüljön a főkönyvbe. A konzisztencia itt nem csak a klasszikus ACID, hanem az **üzleti konzisztencia**: például a számlaegyenleg soha nem lehet negatív, hacsak nincs hitelkeret.

30.1.5 5. Architekturális minták pénzügyi rendszerekben

A fenti követelmények több, egymást kiegészítő mintát hívnak életre:

30.1.6 5.1. Főkönyvi szolgáltatás (Ledger Service)

A főkönyvet külön szolgáltatásként érdemes kialakítani, amely **append-only** API-t kínál: `postTransaction`, `getBalance`, `listTransactions`. A szolgáltatás belsőleg lehet relációs adatbázis vagy event store, de a kliens számára csak a műveletek látszanak. A főkönyvi szolgáltatásnak **idempotens** műveleteket kell biztosítania, és minden bejegyzéshez **hash-láncot** vagy digitális aláírást rendelhetünk az auditálhatóság érdekében.

30.1.7 5.2. Outbox minta

Amikor egy tranzakciót el kell küldeni egy másik szolgáltatásnak (pl. értesítés, csalásmegelőzés), a **tranzakciós outbox** biztosítja, hogy az üzenet csak akkor kerüljön az üzenetsorba, ha a lokális tranzakció sikeres volt. Az outbox tábla ugyanabban az adatbázis-tranzakcióban frissül, mint az üzleti adat, így nem veszhet el üzenet. Egy háttérfolyamat kiolvassa az outboxot és elküldi az üzeneteket, majd törli vagy megjelöli azokat.

30.1.8 5.3. Saga koordinátor

A több lépésből álló üzleti folyamatokat saga koordinátor vezérli. Minden lépésnek van kompenzáló művelete, és a koordinátor naplózza a folyamat előrehaladását. A saga maga is idempotens kell legyen, és a kompenzációt is tranzakcióként kell rögzíteni a főkönyvben.

30.1.9 5.4. Tokenizáció

A kártyaadatok helyett a rendszer **tokeneket** tárol, amelyeket egy külső szolgáltatás (pl. PCI DSS tanúsított token provider) állít elő. A token nem tartalmazza a kártyaadatot, így az adatszivárgás kockázata csökken, és a PCI DSS hatóköre is szűkül.

30.1.10 5.5. Erős ügyfél-hitelesítés (SCA)

A PSD2 szerint a fizetési tranzakciókhoz két független hitelesítési tényező kell. Az architektúrának támogatnia kell a többfaktoros hitelesítést (pl. mobilalkalmazás push, SMS-kód, biometria). A hitelesítési kísérleteket naplózni kell, és kockázatalapú döntéssel (pl. alacsony összegű tranzakciók mentesülhetnek) lehet finomítani.

30.1.11 6. Quality attribute scenario példa

A követelményeket az előző fejezetekben tanult módon, **quality attribute scenario** formában fogalmazzuk meg. Példa egy fizetési rendszerre:

Tranzakciós integritás
- **Inger**: egy ügyfél 150 000 Ft átutalást kezdeményez, de a hálózati kapcsolat megszakad a válasz előtt. - **Környezet**: normál üzem, a főkönyvi szolgáltatás és a külső banki API elérhető. - **Válasz**: a rendszer az idempotencia-kulcs alapján felismeri a duplikált kérést, és nem hajt végre második terhelést; az ügyfél lekérdezheti a tranzakció állapotát, amely vagy „függőben” vagy „teljesítve” állapotot mutat. - **Mérőszám**: duplikált tranzakciók száma = 0; a tranzakció állapota 5 másodpercen belül konzisztens.

Rendelkezésre állás
- **Inger**: az elsődleges adatbázis-kiszolgáló meghibásodik. - **Környezet**: aktív-passzív replikáció, a tartalék példány készenlétben. - **Válasz**: a rendszer a tartalékra vált, a folyamatban lévő tranzakciók befejeződnek, az új tranzakciók fogadása 30 másodpercen belül újraindul. - **Mérőszám**: RTO 60 másodperc, RPO = 0 másodperc, a failover esemény naplózott.

30.1.12 7. Labor: pénzügyi rendszer architektúravázlata

A labor célja, hogy egy **kis, ellenőrizhető példán** alkalmazd a fejezet fogalmait. A feladat egy egyszerű fizetési feldolgozó rendszer architektúravázlatának elkészítése, dokumentálása és döntési naplóval való ellátása.

30.1.13 7.1. Kiinduló állapot

Rendelkezésre áll egy (képzeletbeli) ügyfél, aki átutalást kezdeményez. A rendszernek a következő lépéseket kell végrehajtania: 1. Befogadja a kérést HTTPS-en. 2. Hitelesíti az ügyfelet (SCA). 3. Csalásmegelőzési szűrést futtat. 4. Rögzíti a tranzakciót a főkönyvben. 5. Elküldi a külső banknak elszámolásra. 6. Visszaigazolja az ügyfélnek.

30.1.14 7.2. Elvárt eredmény

Készíts egy **architektúravázlatot** a következő elemekkel:

- **C4 context diagram**: a rendszerhatár, a külső rendszerek (banki API, csalásmegelőzés, SCA-szolgáltató) és a felhasználói szerepkör (ügyfél). A kapcsolatokon tüntesd fel a protokollt (pl. HTTPS), a szinkronitást (szinkron/asinkron) és a hibatűrést (pl. retry, timeout).
- **Container diagram**: a főbb konténerek (pl. API gateway, fizetési szolgáltatás, főkönyvi szolgáltatás, outbox feldolgozó, üzenetsor). Minden konténerhez adj felelősséget és interfészt.
- **Döntési napló (ADR)**: legalább két döntés, például:

  - ADR-001: a főkönyvi szolgáltatás relációs adatbázist használ, append-only naplóval.
  - ADR-002: a tranzakciós konzisztencia biztosítása outbox mintával, nem XA-val.

- **Quality attribute scenario**: egy a rendelkezésre állásra, egy a tranzakciós integritásra.
- **Ellenőrzési szempontok**: felsorolás, hogyan ellenőriznéd a duplikált tranzakciók hiányát, az auditnapló integritását, a kiegyenlítés pontosságát.

A dokumentum legyen **reprodukálható**: a diagramok diagrams-as-code formában (Mermaid vagy PlantUML) készüljenek, és a döntésekhez hivatkozás tartozzon. A **mért és várt eredményt különítsd el**: ha valamit nem futtattál, csak feltételezel, jelöld „nem ellenőrzött feltételezés" címszóval.

30.1.15 7.3. Visszaállítási út

Mivel ez papír alapú tervezési gyakorlat, nincs szükség környezet-visszaállításra. Ha azonban kiegészítő PoC-t készítesz (pl. outbox implementáció lokális adatbázissal), azt izolált környezetben futtasd, és a labor után töröld a tesztadatokat.

30.1.16 8. Gyakori hibák és ellenőrzőlista

A pénzügyi rendszerek tervezésekor a leggyakoribb architekturális hibák:

- **Idempotencia hiánya**: újrapróbálkozás duplikált terhelést okoz. *Helyes megoldás:* üzleti kulcs egyedi megszorítással és idempotencia-kulcs minden külső híváson.
- **Auditnapló utólagos hozzáadása**: a naplózás kimarad a tervezésből, és később nem lehet rekonstruálni az eseményeket. *Helyes megoldás:* minden állapotváltozást strukturált eseményként rögzíteni a kezdetektől.
- **Kiegyenlítés elhanyagolása**: a belső főkönyv eltér a külső elszámolástól, de nincs rendszeres egyeztetés. *Helyes megoldás:* napi vagy valós idejű reconciliation folyamat, eltérés esetén riasztással.
- **Túlzott rendelkezésre állás az integritás rovására**: aktív-aktív replikáció konfliktuskezelés nélkül, ami ellentmondásos adatot eredményez. *Helyes megoldás:* CP konzisztencia preferálása partíció esetén, konfliktuskezelési stratégia definiálása.
- **SCA nem teljes körű**: a hitelesítés csak a webes felületen van, de az API-n nem. *Helyes megoldás:* minden tranzakciós API-végpontra egységes hitelesítési réteget tenni.
- **Szabályozási követelmények alábecslése**: a PCI DSS hatókörét nem szegmentálják, ezért az egész rendszer a kártyaadat-környezetbe kerül. *Helyes megoldás:* tokenizáció és hálózati szegmentáció már a tervben.

Ellenőrzőlista a pénzügyi architektúrához:

☐ Minden tranzakció rendelkezik egyedi üzleti kulccsal és idempotencia-kulccsal.
☐ A főkönyvi bejegyzések append-only, és nem módosíthatók.
☐ Az auditnapló strukturált, időbélyegzett, és hozzáférés-vezérelt.
☐ A kiegyenlítési folyamat rendszeresen fut, és az eltérések riasztást generálnak.
☐ Az adatbázis-replikáció szinkron, vagy az RPO explicit módon dokumentált.
☐ A hitelesítés megfelel az SCA követelményeinek minden csatornán.
☐ A PCI DSS hatókör minimalizált tokenizációval és szegmentációval.
☐ A saga kompenzáló műveletei definiáltak és naplózottak.
☐ A rendelkezésre állási célokat SLO-ként, quality attribute scenario formában rögzítették.
☐ A jogi és szabályozási követelményeket hivatalos forrásból ellenőrizték, és a verzió rögzítve van.

30.1.17 9. Önellenőrző kérdés

Tegyük fel, hogy egy fizetési rendszerben két szolgáltatás van: a **Fizetés** szolgáltatás, amely befogadja a kérést, és a **Főkönyv** szolgáltatás, amely rögzíti a tranzakciót. A Fizetés szolgáltatás sikeresen rögzíti a tranzakciót, de a Főkönyv felé küldött HTTP-hívás timeout miatt megszakad. Milyen architekturális mechanizmussal biztosítanád, hogy a tranzakció ne vesszen el, és ne is duplikálódjon? Magyarázd el a választott minta működését és a kompenzáció szerepét.

Válasz vázlata: a **tranzakciós outbox** mintát kell alkalmazni. A Fizetés szolgáltatás a saját adatbázisában, ugyanabban a tranzakcióban rögzíti az üzleti eseményt és az outbox bejegyzést. Egy háttérfolyamat kiolvassa az outboxot, és elküldi a Főkönyv felé. Ha a küldés sikertelen, újrapróbálja. A Főkönyv idempotens végpontot kínál, így a többszöri kézbesítés nem okoz duplikált bejegyzést. Ha a Főkönyv végül hibát jelez, a saga koordinátor kompenzáló műveletet indít: a Fizetés szolgáltatásban a tranzakciót „visszavont” állapotba teszi, és ezt is naplózza.

30.1.18 10. Továbbvezetés

Ez a fejezet a pénzügyi architektúrák alapjait rögzítette. A következő egység (Egészségügy és elektronikus egészségügyi nyilvántartás – Egészségügy és elektronikus egészségügyi nyilvántartás) ezekre az elvekre épít: az adatintegritás, az auditálhatóság és a szabályozási megfelelés hasonlóan hangsúlyos, de az adatvédelmi és életciklus-kezelési kérdések tovább éleződnek.

Megjegyzés a forrásokhoz: a PCI DSS, a PSD2, a GDPR és más említett szabályozások aktuális, pontos követelményei a hivatalos szabályozói szövegekből ellenőrizendők. A fejezet a 2025-ös állapot szerinti általános architekturális következményeket tükrözi, de a jogi megfelelésért mindig a hatályos előírások az irányadók.

30.2 Egészségügy és elektronikus egészségügyi nyilvántartás

Az egészségügyi informatika talán az egyik legnehezebb terület a szoftverarchitektúra számára: az adatok egyszerre rendkívül érzékenyek, és ugyanakkor több rendszer, intézmény és felhasználó között kell gyorsan, értelmesen áramolniuk. Egy rosszul megtervezett integráció nem csupán kényelmetlenséget okoz, hanem késleltethet egy diagnózist, félreérthető gyógyszerelési utasítást eredményezhet, vagy éppen a beteg magánszféráját sérti. A fejezet ezért három szorosan összefonódó témát jár körül: az egészségügyi adatok érzékenységét, az interoperabilitást lehetővé tevő szabványokat, valamint a klinikai folyamatok és a rendelkezésre állás követelményeit. A cél nem egy adott termék bemutatása, hanem az, hogy a fejezet végére az olvasó képes legyen egy EHR-integrációs architektúrát megtervezni, a döntéseket naplózni, és a tipikus hibákat felismerni.

Egészségügyi adatnak minősül minden olyan adat, amely egy azonosított vagy azonosítható természetes személy testi vagy lelki egészségére, egészségügyi ellátására, illetve az abból származó információkra vonatkozik. A GDPR ezt a személyes adatok különleges kategóriájába sorolja, az Egyesült Államokban a HIPAA védett egészségügyi adatként (PHI) kezeli. Az érzékenység nem csupán jogi kategória: a betegadat nyilvánosságra kerülése diszkriminációhoz, társadalmi megbélyegzéshez, biztosítási vagy munkahelyi hátrányhoz vezethet. Ugyanakkor a klinikai döntésekhez az adatoknak széles körben elérhetőnek kell lenniük: a sürgősségi orvosnak látnia kell a beteg allergiáit, a háziorvosnak a kórházi zárójelentést, a laboratóriumnak a rendelőorvos

kérését. Ez a feszültség – a védelem és a hozzáférhetőség között – az egészségügyi architektúrák alapvető tervezési feszültsége.

Az interoperabilitás ebben a kontextusban azt jelenti, hogy a különböző rendszerek nem csupán adatot cserélnek, hanem az adat jelentését is közösen értelmezik. A szintaktikai interoperabilitás a formátumot és a szállítást írja le, a szemantikai interoperabilitás a fogalmak azonos értelmezését: a „vérnyomás” mindkét oldalon ugyanazt a mértékegységet, mérési körülményt és klinikai jelentést takarja. A gyakorlatban ez szabványosított adatmodelleket, terminológiákat és interfész-specifikációkat igényel, amelyeket a HL7, az IHE és más szervezetek fejlesztenek.

30.2.1 Egészségügyi adatok érzékenysége és a betegjogok

Az adatvédelem tervezésekor abból kell kiindulni, hogy az egészségügyi adat nem csupán személyes adat, hanem különleges kategória. A GDPR 9. cikke főszabály szerint tiltja az ilyen adatok kezelését, kivéve, ha az érintett kifejezett hozzájárulását adta, vagy az adatkezelés egészségügyi ellátás, diagnosztika, kezelés vagy szociális ellátás céljából szükséges. A HIPAA hasonlóan szigorú: a PHI csak meghatározott célokra, minimálisan szükséges mértékben használható fel, és a betegnek joga van hozzáférni, helyesbíteni, valamint megtudni, ki látta az adatait. A pontos jogi kötelezettségek országonként és szabályozásonként eltérnek, ezért a szabályozási hivatkozásoknál mindig az adott joghatóság hatályos szövegét kell ellenőrizni.

A betegjogok közül az architektúra szempontjából a legfontosabb a hozzáférési jog, a helyesbítéshez való jog, a tiltakozás joga és az adathordozhatóság. A hozzáférési jog azt jelenti, hogy a páciens kérheti az egészségügyi dokumentációjának másolatát, akár elektronikus formában. Ez a gyakorlatban egy páciensportált vagy beteg-API-t igényel, amely biztonságosan, hitelesítetten, auditáltan teszi elérhetővé az adatokat. A helyesbítés joga azt jelenti, hogy a beteg kérheti a téves adatok javítását; ez nem jelenti a bejegyzés törlését, hanem a helyesbítés naplózását, mert a klinikai döntés eredeti kontextusát meg kell őrizni. Az adathordozhatóság pedig azt követeli, hogy az adatokat géppel olvasható, szabványos formátumban lehessen átadni egy másik szolgáltatónak – ez a FHIR elterjedésének egyik hajtóereje.

A tervezési következmény: az EHR architektúrában külön komponensként kell kezelni a beleegyezés-kezelést (consent management), az audit naplózást és a hozzáférés-vezérlést. A beleegyezés-kezelés rögzíti, hogy a beteg mely adataihoz, mely szereplők, milyen célból férhetnek hozzá; az audit napló minden hozzáférést, módosítást és adatátadást megőriz; a hozzáférés-vezérlés pedig a szerepkörök és a célhoz kötöttség alapján engedélyez vagy tilt. Ezeket a funkciókat nem szabad utólag ráépíteni a rendszerre, mert a hiányuk szinte mindig adatvédelmi incidenshez vezet.

30.2.2 HL7, FHIR és az interoperabilitás rétegei

A HL7 (Health Level Seven International) szabványcsalád a klinikai adatcsere legismertebb eszköze. A HL7 v2 üzenetek ma is széles körben használatosak, különösen kórházi környezetben. A v2 üzenetek szegmensekből épülnek fel: az MSH az üzenet fejlécét, a PID a beteg azonosítását, az OBR a vizsgálati kérést, az OBX a vizsgálati eredményt írja le. Egy laboreredmény például ORU^R01 üzenettípusban érkezik. A v2 előnye a tömörség és az elterjedtség, hátránya a szintaktikai lazaság: ugyanazt a klinikai tényt különböző intézmények eltérő szegmens-elrendezéssel, kódolással és kitöltési gyakorlattal írják le. Ezért a v2 integrációk gyakran egyedi leképezéseket igényelnek.

A HL7 v3 és a CDA (Clinical Document Architecture) a dokumentum-alapú interoperabilitást célozta, de komplexitásuk miatt nem terjedtek el olyan széles körben, mint remélték. A CDA-t

ma is használják például zárójelentések, leletdokumentumok cseréjére, de a modern, API-alapú integrációk egyértelműen a FHIR felé mozdultak el.

A FHIR (Fast Healthcare Interoperability Resources) REST-alapú, erőforrás-orientált szabvány. A klinikai fogalmakat kis, jól definiált erőforrásokként modellezi: `Patient` a beteg, `Observation` egy mérés vagy megfigyelés, `MedicationRequest` a gyógyszerelési utasítás, `Encounter` az ellátási esemény, `DiagnosticReport` a lelet, `AllergyIntolerance` az allergia. Az erőforrások JSON vagy XML formátumban cserélhetők, HTTP REST API-n keresztül. A FHIR szerver erőforrás-típusonként keresési paramétereket kínál, például `GET /Observation?patient=123&code=LOINC|8480-6` lekérdezi a beteg vérnyomás-méréseit.

A FHIR szemantikai interoperabilitását a profilok és a terminológiai kötések biztosítják. A profil egy adott ország, régió vagy alkalmazás számára megszorítja az erőforrás mezőit, kötelezővé tehet bizonyos elemeket, és rögzíti a terminológiai kódrendszereket. A terminológiai kódrendszerek közül a SNOMED CT a klinikai fogalmakat, a LOINC a laboratóriumi és klinikai méréseket, az ICD-10 a diagnózisokat, az RxNorm a gyógyszereket kódolja. A FHIR erőforrásokban a kódokat rendszerint egy rendszer- és kódpárral adják meg, például `system: "http://loinc.org", code: "8480-6"`. Ez teszi lehetővé, hogy a küldő és a fogadó fél ugyanazt a klinikai jelentést értse a kód alatt. A FHIR verziói folyamatosan fejlődnek; a stabil R4 és R5 mellett az implementációs útmutatók országonként eltérőek, ezért a konkrét verziót és profilt mindig ellenőrizni kell a hivatalos HL7 FHIR dokumentációban.

Az IHE (Integrating the Healthcare Enterprise) profilok a meglévő szabványokat fűzik össze konkrét klinikai munkafolyamatokra. Például az XDS a dokumentum-megosztást, a PIX/PDQ a betegazonosítást, az ATNA az audit naplózást szabványosítja. Egy érett EHR-integráció gyakran kombinálja a HL7 v2 üzeneteket, a FHIR API-t és az IHE profilokat: a laboratóriumi rendszer v2 üzenetet küld, az integrációs motor FHIR erőforrássá alakítja, a terminológiai szolgáltatás elvégzi a kódleképezést, az audit szolgáltatás pedig rögzíti az átadást.

A gyakorlati integrációkban – különösen régi, v2 alapú rendszerekkel – az interfész motor elengedhetetlen. Ez a komponens fogadja a bejövő üzeneteket, elvégzi a szintaktikai ellenőrzést, a leképezést, a terminológiai transzformációt, majd továbbítja a célrendszernek. A tipikus hiba az, amikor a transzformációt közvetlenül az üzleti logikába ágyazzák, így a leképezési szabályok szétszóródnak, és minden új interfésznél újra kell írni őket. Helyesebb, ha az interfész motor pipeline-ként működik: beolvasás, érvényesítés, leképezés, terminológiai normalizálás, célformátum előállítása, audit. Minden lépésnek idempotensnek kell lennie, hogy ismételt kézbesítés esetén ne keletkezzen duplikátum.

30.2.3 Klinikai folyamat és rendelkezésre állás

Az EHR rendszer nem csupán adattároló, hanem a klinikai munkafolyamat aktív résztvevője. A betegút tipikus állomásai – előjegyzés, betegfelvétel, triázs, orvosi vizsgálat, laboratóriumi kérés, képalkotó diagnosztika, gyógyszerelés, elbocsátás – mind különböző rendszereket érintenek. Az EHR-nek ezeket az eseményeket rögzítenie, a rendeléseket továbbítania, az eredményeket visszaadnia, és mindeközben a klinikai döntéstámogatást (például gyógyszer-interakció ellenőrzését) futtatnia kell. Az architektúra tehát nem passzív adatbázis, hanem eseményvezérelt feldolgozási lánc.

A klinikai folyamatok időérzékenyek: egy sürgősségi osztályon a lelet késlekedése percek alatt dönthet életről. Ezért a rendelkezésre állás és a teljesítmény minőségi követelményei sokkal szigorúbbak, mint egy átlagos üzleti rendszernél. A rendelkezésre állást a megszokott képlettel mérjük:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Egy 99,9%-os éves rendelkezésre állás körülbelül 8,76 óra állásidőt enged meg; 99,99% esetén ez 52,6 perc. A klinikai rendszereknél gyakran 99,9% a minimum, de a kritikus részrendszereknél – például a sürgősségi betegfelvétel vagy a laboreredmény-értesítés – 99,99% is elvárt lehet. Az SLO-ból származtatott error budget (1 – SLO) szabja meg, mennyi hibát engedhetünk meg egy mérési ablakban. Ha az error budget kimerül, a fejlesztési sebességet kell csökkenteni, és a megbízhatósági munkát előtérbe helyezni – ez az egészségügyben különösen fontos, mert a leállás közvetlen betegbiztonsági kockázat.

A rendelkezésre állás tervezésekor a helyreállítási célokat is rögzíteni kell: az RPO (Recovery Point Objective) azt mondja meg, mennyi adatvesztést viselünk el, az RTO (Recovery Time Objective) azt, mennyi idő alatt kell a szolgáltatást visszaállítani. Egy EHR esetében az RPO jellemzően a nullához közeli, mert a betegadatok elvesztése elfogadhatatlan; az RTO pedig percben mérhető. Ennek eléréséhez aktív replikáció, tranzakciós naplózás és katasztrófa-helyreállítási terv szükséges. A hibatűrési minták – replikáció, failover, áramkör-megszakító, retry, timeout, bulkhead – a reziliencia fejezetben tárgyalt általános elveken alapulnak, de itt a klinikai kontextus adja a prioritást: például a laboreredmények kézbesítése nem maradhat el, de a duplikátumok sem okozhatnak téves riasztást, ezért az idempotens feldolgozás és az outbox minta kiemelten fontos.

A klinikai folyamatok egy része hálózati kiesés esetén is működőképes kell maradjon. Az orvosi rendelőben a helyi gyorsítótár vagy az offline mód lehetővé teszi a betegadatok lekérdezését akkor is, ha a központi rendszer elérhetetlen. Az offline szinkronizáció azonban konfliktusokat okozhat, ezért a tervezésnek explicit konfliktuskezelési stratégiát kell tartalmaznia. A gyakorlatban ez gyakran azt jelenti, hogy az offline módban csak olvasható adatokat cache-elünk, az írásokat pedig sorba állítjuk, és a kapcsolat helyreállása után, a konfliktusokat naplózva juttatjuk el a központi rendszerbe.

A kapacitástervezésnél a Little-törvényt használhatjuk: $L = \lambda W$, ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta, W az átlagos tartózkodási idő. A klinikai rendszerek terhelése napszaki ingadozást mutat: a reggeli vizit és a sürgősségi csúcsok magasabb kérésszámot generálnak. A méretezést a csúcsterhelésre, de a költséghatékonyság miatt gyakran horizontális skálázással és autoscalinggel oldják meg. A FHIR szerver például stateless API-réteget kínál, így a horizontális skálázás jól működik, feltéve, hogy a mögöttes adatbázis replikált és a gyorsítótár konzisztens.

30.2.4 EHR-integrációs architektúra labor

A labor célja egy EHR-integrációs architektúra megtervezése, dokumentálása és döntési naplóval való ellátása. A feladat nem kódolás, hanem tervezés: a hallgatók egy reprodukálható dokumentumot vagy jegyzetfüzetet készítenek, amelyben a kontextus, a követelmények, a minőségi forgatókönyvek, az architektúra-alternatívák és az ADR-ek szerepelnek. A mért és a várt eredményeket szigorúan el kell különíteni: mivel a labor nem futtat éles rendszert, a mért oszlopot „nem futtatott” jelöléssel töltik ki, vagy becslést adnak, amit egyértelműen megjelölnek.

Kiinduló állapot. Adott egy kórházi EHR rendszer, amely FHIR R4 API-n keresztül képes betegadatokat, laboreredményeket és rendeléseket kezelni. A rendszer jelenleg a belső laboratóriummal integrált, amely HL7 v2 ORU^R01 üzeneteket küld. A feladat egy külső, független laboratórium bekapcsolása, amely ugyancsak HL7 v2 üzeneteket küld, de eltérő terminológiai

kódolással és üzenetstruktúrával rendelkezik. A cél: a külső labor eredményei megbízhatóan, auditáltan, a beteg beleegyezését tiszteletben tartva kerüljenek be az EHR-be, és FHIR `Observation` erőforrásként legyenek lekérdezhetők.

Elvárt eredmény. A hallgatók egy architektúra-tervet adnak át, amely legalább a következőket tartalmazza:

1. **C4 context szintű diagram** diagrams-as-code formában (például Mermaid vagy PlantUML). A diagramon szerepeljenek a külső szereplők: a külső laboratórium, a beteg, a klinikai felhasználó, a belső labor, valamint a rendszerhatár.
2. **Funkcionális és minőségi követelmények.** Legalább három quality attribute forgatókönyv, a formátum szerint: inger, környezet, válasz, mérőszám. Például: „Amikor a külső laboratórium laboreredményt küld (inger), normál üzemi környezetben (környezet), a rendszer az eredményt 10 másodpercen belül FHIR `Observation` erőforrásként elérhetővé teszi (válasz), miközben a sikertelen kézbesítések aránya 0,1% alatt marad (mérőszám).”
3. **ADR-ek.** Legalább két döntés a következő témákból: FHIR R4 vs. HL7 v2 natív tárolás, terminológiai leképezés helye (interfész motor vs. EHR), audit naplózás módja (szinkron vs. aszinkron), beleegyezés-kezelés integrációja. Az ADR-ek tartalmazzák a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
4. **C4 container szintű diagram**, amely bemutatja a komponenseket: interfész motor, terminológiai szolgáltatás, FHIR API gateway, EHR adatbázis, audit szolgáltatás, consent szolgáltatás. A kapcsolatokon a protokoll, a szinkronitás és a hibatűrés szerepeljen.
5. **Mérési terv.** A terv rögzítse a környezetet, a konfigurációt, az időpontot és a bizonytalanságot. A mért és a várt eredmény táblázatban legyen elkülönítve. A várt értékeket a minőségi forgatókönyvekből származtassák, a mért oszlopba „nem futtatott” kerüljön, vagy ha becslés, akkor azt jelölni kell.
6. **Ellenőrzési szempontok.** A dokumentum ellenőrzésekor azt vizsgáljuk, hogy a rendszerhatár minden diagramon jelölve van-e, hogy a kapcsolatokon szerepel-e a protokoll, hogy az ADR-ekből kiderül-e a döntés oka és a feladott alternatívák, és hogy a mérési terv elkülöníti-e a mért és várt adatokat.

Visszaállítási út. Mivel a labor tervezési feladat, destruktív műveletet nem igényel. Ha a hallgató mégis konfigurációt vagy példakódot módosít, azt izolált környezetben tegye, és a változtatásokat verziókezelőben tárolja; a visszaállítás a verziókezelő előző állapotának visszaállításával történik.

A labor során a gyakori hibák: az ADR-ekből hiányzik a kontextus, ezért a döntés nem érthető; a minőségi forgatókönyvekből hiányzik a mérőszám, ezért nem mérhetők; a C4 diagramon nincs rendszerhatár, így nem látszik, mi tartozik a felelősségi körbe; a mérési tervben összemosódik a mért és a várt érték, ami félrevezető.

30.2.5 Ellenőrzőlista

- Minden diagramon jelölve van a rendszerhatár, a külső rendszerek és a felhasználói szerepkörök.
- Minden komponens rendelkezik felelősséggel és interfésszel, nem csupán névvel.
- Minden kapcsolaton szerepel a protokoll, a szinkronitás és a hibatűrés.
- Legalább három quality attribute forgatókönyv adott, mindegyikben inger, környezet, válasz és mérőszám.
- Az ADR-ek tartalmazzák a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
- A terminológiai leképezés helye explicit döntés eredménye, nem utólagos toldozás.

- A betegjogok kezelése (hozzáférés, helyesbítés, adathordozhatóság, tiltakozás) nyomon követhető a komponensekben.
- Az audit naplózás minden adathozzáférésre és módosításra kiterjed.
- A rendelkezésre állási cél (SLO) és az error budget rögzített.
- A mérési terv elkülöníti a mért és a várt értékeket, megadva a környezetet, a konfigurációt és az időpontot.

30.2.6 Önellenőrző kérdés

Miért nem elegendő a HL7 v2 üzenet szintaktikai konvertálása FHIR erőforrássá a szemantikai interoperabilitáshoz? Gondolja végig, milyen szerepet játszanak a terminológiai kódrendszerek és a profilok abban, hogy a küldő és a fogadó fél azonos klinikai jelentést értsen. Hogyan befolyásolja a beteg hozzáférési joga az EHR API tervezését, különösen az audit naplózás és a beleegyezés-kezelés szempontjából?

30.3 Kiskereskedelem és e-kereskedelem

Az e-kereskedelem architektúrája egyszerre hasonlít más üzleti rendszerekhez, és mégis sajátos kihívásokat hordoz: a terhelés időszakosan, gyakran kiszámíthatóan, de mégis élesen kilendül; a termékkatalógus olvasási nehéz, míg a rendelésfeldolgozás tranzakciós, állapotgazdag és üzletileg kritikus. Ebben a fejezetben a kiskereskedelem és e-kereskedelem három központi területét járjuk körül: a **szezonális terhelést**, a **termékkatalógust** és a **rendelésfeldolgozást**. A cél nem csupán az alapfogalmak ismertetése, hanem azoknak az architektúradöntéseknek a bemutatása, amelyek egy csúcsforgalmi időszakban is működő, ügyfélbarát rendszert eredményeznek.

A fejezetben a korábbi részekben bevezetett eszközöket – C4 modellezés, ADR, quality attribute scenario, Little-törvény – alkalmazzuk a kiskereskedelmi kontextusra. A példák szemléltetők, nem éles környezetre szánt utasítások; valós hitelesítő adat és ügyféladat helyett helyőrzőket használunk.

30.3.1 1. A szezonális terhelés és kapacitástervezés

A kiskereskedelemben a forgalom nem egyenletes. A Black Friday, a karácsonyi időszak, az év végi leárazások vagy egy-egy sikeres marketingkampány akár a normál forgalom tízszeresét is produkálhatja. A szezonális terhelés nem kivételes hiba, hanem az üzleti modell része; az architektúrának ezért **előre tervezett rugalmassággal** kell rendelkeznie.

30.3.2 A terhelés modellezése

A kapacitástervezéshez először a várható forgalmat kell számszerűsíteni. A korábbi fejezetekben megismert Little-törvény itt is kulcsszerepet játszik:

$$L = \lambda W$$

ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta (kérés/másodperc), W pedig az átlagos tartózkodási idő (másodperc). Ha például a katalógusszolgáltatás átlagos válaszideje $W = 0{,}2$ s, és a becsült csúcsterhelés $\lambda = 1000$ kérés/s, akkor a rendszerben egyidejűleg átlagosan

$L = 1000 \cdot 0{,}2 = 200$ kérés van folyamatban. Ha minden katalóguspéldány 50 egyidejű kérést tud biztonsággal kezelni, akkor legalább 4 példányra van szükség.

A terhelési modell ennél összetettebb lehet. A felhasználói viselkedést profilokra bontjuk: böngészés (katalógusnézet, keresés, termékoldal), kosárkezelés, rendelésleadás, fizetés, rendeléskövetés. Minden tranzakcióhoz hozzárendeljük az átlagos kérésszámot és az átlagos válaszidőt, majd a csúcsterhelési időszakra meghatározzuk a párhuzamos felhasználók számát. A modellből kiszámítható a szolgáltatásonkénti kérési ráta és az erőforrás-szükséglet.

A **szezonális terhelésre** azonban nem elég utólag, autoscalinggel reagálni. Az autoscaling hidegindítási késleltetést és konténerindítási időt visz a rendszerbe, ami csúcsidőben elveszett ügyfeleket eredményezhet. A gyakorlatban a várható csúcs előtt **előre skálázott** példányszámot állítanak be, és az autoscaling már csak a finomhangolást végzi. A döntést ADR-ben rögzítjük.

Példa ADR – Előre skálázás a katalógusszolgáltatásnál

- **Kontextus**: A katalógusszolgáltatás a Black Friday alatt a normál 200 kérés/s helyett 1000 kérés/s terhelést kap. Az autoscaling egyedül nem garantálja a p95 $<$ 200 ms SLO-t a hidegindítási késleltetés miatt.
- **Döntés**: A csúcs előtt 2 órával a katalógusszolgáltatást 8 példányra skálázzuk fel, és az autoscaling ezt a szintet tartja. A csúcs után 1 órával visszaállunk 2 példányra.
- **Alternatívák**: kizárólag autoscaling (elutasítva a késleltetés miatt); serverless (elutasítva a katalógus állandó terhelése és a hidegindítási költség miatt); CDN-től várt offload (kiegészítő megoldás, de nem helyettesíti a backend skálázását).
- **Következmények**: magasabb infrastruktúra-költség a csúcsidőszakban, de teljesíthető SLO. A műveleti csapatnak ütemezett skálázási runbookot kell karbantartani.
- **Állapot**: elfogadva.

A Little-törvényt a rendelésfeldolgozásra is alkalmazhatjuk: ha a rendelések érkezési rátája $\lambda = 20$ rendelés/s, és a feldolgozás átlagos ideje $W = 5$ s, akkor a rendszerben lévő rendelések átlagos száma $L = 100$. Ez alapján méretezhető a feldolgozó munkavállalók száma vagy a feldolgozó szolgáltatások párhuzamossága.

30.3.3 2. A termékkatalógus architektúrája

A termékkatalógus a webshop olvasási nehéz komponense. A felhasználók böngésznek, keresnek, szűrnek, termékoldalakat néznek; eközben a katalógusadatokat ritkán módosítják – általában csak árazási frissítés, készletváltozás vagy új termék bevezetése során. Ez az aszimmetria alapvetően meghatározza az architektúrát.

30.3.4 Olvasási és írási út szétválasztása

A katalógus írási oldala (adminisztrációs felület, termékimport, árfrissítés) alacsony terhelésű, de magas adatintegritást igényel. Az olvasási oldal nagy terhelésű, és akár enyhe késleltetéssel is elviseli a frissítéseket. Ezért a CQRS (Command Query Responsibility Segregation) mintát gyakran alkalmazzák, de kisebb rendszereknél elegendő lehet az **olvasási replikák** használata: az írások az elsődleges adatbázisba mennek, az olvasások a replikákból szolgálódnak ki. A replikák közötti szinkronizáció másodperces késleltetést okozhat, ami a katalógusnál általában elfogadható.

A katalógus olvasási útját **CDN** elé helyezhetjük. A statikus tartalmakat – termékfotók, leírások, kategóriaoldalak – a CDN gyorsítótárazza, és a felhasználóhoz közeli pontról szolgálja ki. A dinamikus találati listák és az árak azonban nem gyorsítótárazhatók hosszú ideig, mert a készlet-

és árinformációk változnak. Itt rövid TTL-t (például 1–5 perc) vagy **in-memory cache**-t (például Redis) használhatunk a backend előtt.

A **keresési funkció** külön szolgáltatást érdemel. A termékkeresés nem egyszerű adatbázis-lekérdezés: relevancia, szűrők, facetták, hasonló termékek ajánlása tartozik hozzá. A gyakorlatban egy search engine-t (például Elasticsearch, OpenSearch vagy alkalmas managed szolgáltatás) alkalmaznak, amely indexeli a katalógust, és gyors válaszidővel kezeli a lekérdezéseket. A katalógus írási útja ekkor **indexelési eseményeket** küld a keresőszolgáltatásnak.

A katalógus **adatok forrása** gyakran egy PIM (Product Information Management) rendszer, amely a termékleírásokat, képeket, attribútumokat központilag kezeli. A webshop katalógusa ebből a rendszerből szinkronizálódik. A készletadatok általában egy külön készletgazdálkodási rendszerből érkeznek, és a webshop csak a „megvásárolható mennyiséget” mutatja, nem a teljes raktárkészletet.

A katalógusszolgáltatás komponensdiagramja a következőképpen nézhet ki:

```
flowchart LR
    A[Admin / PIM] -->|REST, írás| C[Katalógus írás API]
    C -->|SQL| CD[(Katalógus DB)]
    C -->|Esemény| M[Message Broker]
    M -->|Indexelési esemény| S[Kereső szolgáltatás]
    B[Webshop frontend] -->|HTTPS| K[Katalógus olvasás API]
    K -->|Gyorsítótár| R[(Redis)]
    K -->|SQL, replika| CDR[(Katalógus replika)]
    K -->|REST| S
    CDN[CDN] -->|Statikus tartalom| B
```

Jelmagyarázat: A diagram a katalógus írási és olvasási útját mutatja. A folytonos nyíl szinkron hívás, a szaggatott nyíl aszinkron esemény. A CDN a statikus tartalmakat szolgálja ki; a Redis a gyakori olvasási kéréseket gyorsítótárazza; a kereső szolgáltatás indexeli a termékadatokat.

Feltételjegyzet: Az ábra C4 container szintű nézet, a rendszerhatár a webshop és a kereső szolgáltatás körül van. A külső rendszerek (PIM, készletgazdálkodás) a kontextus szinten szerepelnek.

A katalógus **minőségi követelményeit** quality attribute scenario formában fogalmazzuk meg:

- **Inger**: 10 000 egyidejű felhasználó böngészi a termékkategóriákat és keres.
- **Környezet**: termelési rendszer, csúcsidőszak (Black Friday), aktív CDN és Redis cache.
- **Válasz**: a kategóriaoldal és a keresési találatok p95 válaszideje legfeljebb 200 ms; a hibás kérések aránya legfeljebb 0,1%.
- **Mérőszám**: p95 válaszidő, 1 órás mérési ablak; hibaarány, 1 órás mérési ablak.

30.3.5 3. Kosár és session-kezelés

A kosár állapotot hordoz, ezért a skálázhatóság szempontjából kényes pont. A helytelen megoldás az, ha a kosarat az alkalmazáspéldány memóriájában tároljuk: a horizontális skálázás vagy az újraindítás elveszíti a felhasználó kosarát, és a terheléselosztás is bonyolulttá válik. A **best practice** az, hogy a kosár állapotát **külső, megosztott tárolóban** tartjuk, például Redis-ben vagy egy NoSQL adatbázisban. A felhasználó böngészője csak egy **session-azonosítót** kap (cookie-ban vagy tokenben), a kosár tényleges tartalma a szerveroldali tárolóban van.

A kosárkezelésnél fontos a **lejárat** és a **konzisztencia**. A kosár adatai lehetnek ideiglenesek: ha a felhasználó nem jelentkezik be, a kosár 30 nap után törlődhet; bejelentkezés után a kosár a felhasználói fiókhoz kötődik, és hosszabb ideig megmarad. A kosár módosításánál – termék

hozzáadása, mennyiség módosítása, törlés – **idempotens** műveleteket kell tervezni, mert a frontend újrapróbálkozhat.

A kosár és a rendelésleadás között **átmeneti állapot** van. A kosár egy munkaterület, a rendelés pedig egy rögzített, visszavonhatatlan üzleti tranzakció. A kettőt élesen elválasztjuk: a rendelésleadás során a kosárból létrejön a rendelés, és a kosár kiürül vagy archívumba kerül. Ez a folyamat tranzakciós kell legyen, hogy ne keletkezzen duplikált rendelés.

A kosárkezelési szolgáltatás interfészei például:

- `POST /cart/items` – termék hozzáadása a kosárhoz
- `GET /cart` – kosár lekérdezése
- `PUT /cart/items/{itemId}` – mennyiség módosítása (idempotens)
- `DELETE /cart/items/{itemId}` – termék eltávolítása

A kosárkáros műveleteknél a válaszidő kevésbé kritikus, de a rendelkezésre állás igen: ha a kosár szolgáltatás leáll, az ügyfél nem tud vásárolni. Ezért a kosártárolót redundánsan kell üzemeltetni, és a kosarat kiszolgáló szolgáltatást több példányban, terheléselosztással futtatni.

30.3.6 4. Rendelésfeldolgozás

A rendelésfeldolgozás az e-kereskedelem **tranzakciós magja**. A folyamat több lépésből áll: a kosár ellenőrzése, a rendelés létrehozása, a fizetés kezdeményezése, a fizetés visszaigazolása, a raktári betárolás, a csomagolás, a szállítás és az ügyfélértesítés. Ez a folyamat **állapotgazdag**, és több szolgáltatást, adatbázist és külső rendszert érint.

30.3.7 Rendelés-életciklus

A rendelés állapotai tipikusan:

1. **Létrehozva** – a rendelés rögzítve, fizetés még nem történt.
2. **Fizetés függőben** – a fizetési szolgáltatás felé kérés elküldve.
3. **Fizetve** – a fizetés sikeres, a teljesítés elkezdődhet.
4. **Teljesítésre átadva** – a raktárnak kiküldve.
5. **Kiszállítva** – a szállító átadta a csomagot.
6. **Törölve / Visszaküldve** – üzleti korrekció.

Az állapotgép leírható **flow** diagrammal vagy szövegesen. A lényeg: a rendelés állapota **csak meghatározott átmeneteken** keresztül változhat, és minden átmenet eseményhez kötött.

30.3.8 Elosztott tranzakciók helyett outbox és saga

A rendelésfeldolgozás gyakran több adatbázist és külső szolgáltatást érint. A klasszikus kétfázisú commit (2PC) a legtöbb esetben nem megfelelő: zárolások, elérhetőségi problémák, és a résztvevő rendszerek nem feltétlenül támogatják. A helyes megközelítés az **eseményvezérelt architektúra**, amelyben a szolgáltatások aszinkron módon, eseményeken keresztül kommunikálnak.

A **rendelés létrehozásakor** az alábbi módon járunk el:

1. A rendelés-szolgáltatás fogadja a rendelésleadási kérést, és **egyetlen adatbázis-tranzakcióban** elmenti a rendelést, valamint egy **outbox** táblába egy `RendelesLetrehozva` eseményt.

2. A tranzakció commitolása után egy **outbox relay** folyamat kiolvassa az outbox bejegyzést, és elküldi a message brokernek (például RabbitMQ, Apache Kafka). Ha a broker nem elérhető, a relay újrapróbálkozik; így az esemény kézbesítése garantáltan megtörténik, anélkül hogy a rendelés elveszne.
3. A fizetési szolgáltatás fogadja a `RendelesLetrehozva` eseményt, és saját folyamatában kezdeményezi a fizetést. A fizetési eredményről eseményt küld vissza: `FizetesJovahagyva` vagy `FizetesElutasitva`.
4. A rendelés-szolgáltatás fogadja a fizetési eseményt, és frissíti a rendelés állapotát.

Ez a minta **aszinkron feldolgozást** eredményez, így a rendelésleadási kérés gyorsan visszatérhet: a frontend azonnal `202 Accepted` választ kap, és a felhasználó értesítést kap a feldolgozás állapotáról. A rendelés-visszaigazolás így nem blokkolja a felhasználót.

A folyamatot szekvenciadiagrammal szemléltethetjük:

```
sequenceDiagram
    participant V as Vásárló
    participant W as Webshop frontend
    participant O as Rendelés API
    participant DB as Rendelés DB
    participant M as Message Broker
    participant F as Fizetési szolgáltatás

    V->>W: Megrendelés leadása
    W->>O: POST /orders
    O->>DB: Rendelés + outbox esemény (tranzakció)
    O-->>W: 202 Accepted
    W-->>V: "A rendelését feldolgozzuk"
    DB-->>M: Outbox relay (RendelesLetrehozva)
    M->>F: RendelesLetrehozva
    F-->>M: FizetesJovahagyva
    M->>O: FizetesJovahagyva
    O->>DB: Rendelés állapot = Fizetve
    O-->>V: E-mail: rendelés megerősítve
```

Jelmagyarázat: A szaggatott nyilak aszinkron üzenetek; a folytonos nyilak szinkron hívások. Az outbox relay a rendelés-adatbázisból olvas, és a message brokerre publikál. A fizetési szolgáltatás a saját folyamatában kezeli a tranzakciót, majd eseményt küld vissza.

Feltételjegyzet: A diagram a rendelés létrehozásának happy path-jét mutatja, hibakezelés nélkül. A valós rendszerben újrapróbálkozási és kompenzációs mechanizmusok is vannak.

30.3.9 Idempotencia és kompenzáció

A rendelésfeldolgozásban **idempotens műveletekre** van szükség. A felhasználó duplán kattinthat a „Megrendelés” gombra, a hálózat megszakadhat, az üzenetkézbesítés újrapróbálkozhat. A rendelés-szolgáltatásnak **egyedi rendelésazonosítót** kell használnia, és a `POST /orders` hívást idempotenssé tenni: ha ugyanaz a kliens által generált idempotencia-kulcs már létezik, a szolgáltatás a korábbi rendelést adja vissza ahelyett, hogy újat hozna létre. Hasonlóan a fizetési szolgáltatásnak idempotensen kell kezelnie a fizetési kéréseket.

A **saga mintához** akkor folyamodunk, ha a folyamat több lépésből áll, és egy lépés sikertelensége esetén a korábbi lépéseket kompenzálni kell. Például ha a fizetés jóváhagyva, de a raktári készlet nem áll rendelkezésre, akkor **kompenzációs műveletként** vissza kell téríteni a fizetést. A saga lehet koreografált (minden szolgáltatás a saját eseményeire reagál) vagy orchestrált (egy központi folyamat vezérli a lépéseket). A kiskereskedelemben gyakran a **koreografált saga** bizonyul

rugalmasabbnak, mert nincs központi szűk keresztmetszet, de az orchestration jobban átlátható a folyamat tulajdonosai számára. A választás trade-off, és ADR-ben rögzítendő.

30.3.10 A rendelésfeldolgozás minőségi forgatókönyve

- **Inger**: 20 000 felhasználó ad le rendelést egy óra alatt.
- **Környezet**: termelési rendszer, csúcsidőszak, a rendelésfeldolgozó szolgáltatás 4 példányban fut.
- **Válasz**: a rendelésleadási kérés p95 válaszideje legfeljebb 2 másodperc; a rendelésfeldolgozás végpontok hibaaránya legfeljebb 0,1%; egyetlen rendelés sem vész el.
- **Mérőszám**: p95 válaszidő, 1 órás ablak; hibaarány, 1 órás ablak; elveszett rendelések száma, 1 órás ablak.

30.3.11 5. Fizetés integráció

A fizetés az e-kereskedelem legérzékenyebb része. A PCI DSS (Payment Card Industry Data Security Standard) követelményeit be kell tartani, ha bankkártyaadatokat kezelünk. A legbiztonságosabb megoldás, ha **nem is tárolunk kártyaadatokat** a saját rendszerünkben, hanem egy **Payment Service Provider** (PSP) szolgáltatását használjuk, amely tokenizálja a kártyaadatokat, és helyettünk végzi a tranzakciót. Ekkor a webshop csak egy **fizetési tokent** kap vissza, amelyet később használhat.

A fizetési integráció tipikus folyamata:

1. A webshop létrehoz egy fizetési szándékot a PSP-nél, átadva a rendelés összegét és devizanemét.
2. A felhasználó átirányításra kerül a PSP hosztolt fizetési oldalára, vagy a PSP hitelkártyamezőit beágyazott formában jeleníti meg.
3. A felhasználó megadja a kártyaadatokat, és a PSP jóváhagyja vagy elutasítja a fizetést.
4. A PSP **webhook**-on keresztül értesíti a webshopot a fizetés eredményéről. A webhook hívás **digitálisan aláírt**, és a webshopnak ellenőriznie kell az aláírást.

A webhook feldolgozása **idempotens** kell legyen: a PSP újrapróbálkozhat, ezért a webshopnak a tranzakcióazonosító alapján egyszer kell csak frissítenie a rendelést. A fizetési státusz frissítése aszinkron eseményként megy a rendelés-szolgáltatáshoz, ahogy azt az előző szakaszban láttuk.

A PCI DSS-nek való megfelelés nem csupán technológiai kérdés: az önértékelési kérdőív (SAQ) és a hivatalos követelmények betartása szervezeti folyamatokat is érint. A fejezet nem ad jogi tanácsot; a mindenkori hivatalos PCI DSS szöveget ellenőrizni kell.

30.3.12 6. Csúcsforgalom és ügyfélélmény

A csúcsforgalomban az ügyfélélmény két dologtól függ: a **válaszidőtől** és a **megbízhatóságtól**. Ha a felhasználó 3 másodpercnél többet vár egy termékoldal betöltésére, nagy valószínűséggel elhagyja az oldalt. Ha a rendelésleadás hibát ad, a bizalom sérül. A csúcsforgalom kezelése ezért nem csak a backend skálázásáról szól, hanem a felhasználói út tudatos tervezéséről is.

30.3.13 Aszinkron műveletek és szolgáltatáscsökkentés

A rendelésleadásnál már láttuk, hogy a `202 Accepted` válasszal azonnal visszatérhetünk, és a feldolgozás a háttérben történik. Hasonló elvet alkalmazhatunk más hosszú műveletekre: a termékértékelés beküldése, a készletértesítő feliratkozás, a promóciós kód ellenőrzése mind háttérbe tehető, ha a fő vásárlási útvonalat nem blokkolja.

A csúcsforgalomban **szolgáltatáscsökkentést** is alkalmazhatunk. A nem létfontosságú funkciókat – például a személyre szabott ajánlókat, a kapcsolódó termékek listáját, a felhasználói vélemények betöltését – átmenetileg kikapcsolhatjuk, vagy csökkentett gyorsítótárazással szolgálhatjuk ki. A cél az, hogy az értékesítési út (katalógus -> kosár -> rendelés -> fizetés) mindig elérhető maradjon.

A **frontend** oldalon a progresszív megjelenítés, a statikus tartalom CDN-ről való kiszolgálása, a képek optimalizálása és a webhely gyorsítótárazási fejlécei mind csökkentik a backend terhelését. A frontend réteg maga is skálázható, mert általában statikus fájlokat szolgál ki.

30.3.14 Hibaarány és hibatűrés

A csúcsforgalomban a hibaarányt szigorúan figyelni kell. A hibaarány a sikertelen kérések aránya az összes kéréshez képest. Ha a hibaarány meghaladja az 1%-ot, az már észrevehető ügyfélélmény-romlást okoz. A **circuit breaker** minta segít megakadályozni, hogy egy hibás szolgáltatás az egész rendszert magával rántsa: ha a fizetési szolgáltatás lassú vagy hibás, a rendelés-szolgáltatás rövid időre megszakítja a hívásokat, és alternatív utat kínál (például „A fizetés jelenleg nem elérhető, próbálja újra később”).

A **hibakeret (error budget)** fogalma itt is érvényes: ha az SLO a rendelkezésre állásra 99,9% egy hónapos ablakban, akkor az error budget 0,1%, azaz körülbelül 43,2 perc állásidő havonta. A csúcsidőszakban ezt a keretet nem szabad felemészteni, mert a hónap többi részén is lehetnek incidensek.

30.3.15 7. Labor: e-kereskedelmi architektúra és terhelési modell

A labor célja, hogy a fejezetben tárgyalt fogalmakat egy kis, reprodukálható példán alkalmazza. A feladat nem éles rendszeren futtatandó; a kód- és konfigurációpéldák szemléltetők.

30.3.16 Kiinduló állapot

Egy meglévő, egyszerű e-kereskedelmi PoC, amelyben a katalógus és a rendelés ugyanabban a monolit alkalmazásban, közös adatbázissal fut. A PoC forráskódja verziókezelt, a környezet konténerizált (Docker Compose). A katalógus 1000 terméket tartalmaz, a rendelésfeldolgozás szinkron, a fizetés szimulált.

30.3.17 Elvárt eredmény

Készítsen el egy **reprodukálható dokumentumot** (például Markdown vagy Jupyter notebook formátumban), amely a következőket tartalmazza:

1. **C4 modellek**: context, container és component szintű diagramok a rendszerről, diagrams-as-code formában (például Mermaid vagy Structurizr DSL). A diagramokon szerepeljen a rendszerhatár, a külső rendszerek, a felhasználói szerepkörök, a komponensek felelőssége és interfészei, a kapcsolatok protokollja és szinkronitása.
2. **ADR-ek**: legalább három architektúradöntés rögzítése (például a kosár Redis-be helyezése, a rendelésfeldolgozás aszinkronná tétele, a katalógus CDN + cache elé helyezése). Minden ADR tartalmazza a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot.
3. **Terhelési modell**: definiálja a felhasználói profilokat (böngészés, kosár, rendelés, fizetés), a tranzakciók átlagos kérésszámát és válaszidejét, a csúcsterhelési időszakot (például 2 óra), és a párhuzamos felhasználók számát. A Little-törvénnyel számolja ki a szolgáltatásonkénti szükséges párhuzamos kérések számát.
4. **Mérési terv**: válasszon terheléses eszközt (például k6, JMeter), adjon meg egy futtatási konfigurációt (például 10 virtuális felhasználó, 5 perc felfutás, 10 perc állandó terhelés), és rögzítse a mérendő metrikákat (p95 válaszidő, hibaarány, átviteli sebesség).
5. **Eredmények**: egy táblázatban **elkülönítve** a **várt** (a terhelési modellből számított) és a **mért** (ha rendelkezésre áll) értékeket. A mért értékeknél adja meg a környezetet, a konfigurációt, az időpontot és a mérési bizonytalanságot. Ha a mérés nem futott le, a mért oszlop legyen üres, és ezt jelölje.

30.3.18 Ellenőrzési szempontok

- A C4 diagramokon minden komponensnek van felelőssége és interfésze; a rendszerhatár egyértelmű.
- Az ADR-ek visszakereshetők, és a döntés okát, valamint a feladott alternatívákat tartalmazzák.
- A terhelési modellben a Little-törvény alkalmazása helyes, a mértékegységek szerepelnek.
- A mérési terv tartalmazza a kiinduló állapotot, a futtatás lépéseit és a visszaállítási utat.
- A mért és várt értékek külön vannak választva, és a mért értékeknél a bizonytalanság is fel van tüntetve.

30.3.19 Visszaállítási út

A terhelési tesztet izolált környezetben kell futtatni. A generált tesztadatokat (felhasználók, rendelések, kosarak) a teszt után törölni kell. A terheléses eszköz konfigurációját és a teszteredményeket verziókezelten tárolja. A konténerkörnyezetet a teszt után le kell állítani, az erőforrásokat fel kell szabadítani.

30.3.20 8. Tipikus hibák és anti-minták

Az e-kereskedelemben számos hiba ismétlődik, amelyeket érdemes elkerülni:

- **A kosár az alkalmazáspéldány memóriájában**: horizontális skálázásnál elveszik az állapot, a terheléselosztó nem tud session-affinitást fenntartani. Helyette: külső session store.
- **A katalógus és a keresés egy szolgáltatásban**: a keresés erőforrás-igényes, és eltérő skálázást igényel. Helyette: külön keresőszolgáltatás, saját indexszel.
- **Szinkron rendelésfeldolgozás**: a rendelésleadási kérés a fizetési szolgáltatás válaszát várja, ami hosszú válaszidőt és hibalehetőséget okoz. Helyette: aszinkron feldolgozás outbox mintával.

- **Nincs idempotens rendelésleadás**: duplikált rendelések jönnek létre. Helyette: idempotencia-kulcs és deduplikáció.
- **Csak autoscalingre támaszkodni csúcsidőben**: a hidegindítási késleltetés miatt az SLO sérülhet. Helyette: előre skálázás + autoscaling.
- **A PCI DSS figyelmen kívül hagyása**: kártyaadatok tárolása a saját rendszerben a legnagyobb kockázat. Helyette: PSP-tokenizáció, kártyaadatok elkerülése.
- **Nincs szolgáltatáscsökkentési terv**: csúcsidőben minden funkció fut, és a nem kritikus funkciók is terhelik a rendszert. Helyette: degradációs stratégia, amely a kritikus utat védi.

Mindegyik hiba mögött felismerhető az az elv, hogy a **skálázhatóságot**, a **megbízhatóságot** és a **biztonságot** nem utólag, hanem a tervezéskor kell beépíteni.

30.3.21 9. Ellenőrzőlista

A fejezet végén használja az alábbi ellenőrzőlistát egy e-kereskedelmi architektúra értékeléséhez:

- ☐ A katalógus olvasási útja CDN-t és/vagy gyorsítótárat használ; az árak és készletek frissítése szabályozott.
- ☐ A kosár állapota megosztott tárolóban van, nem az alkalmazáspéldányban.
- ☐ A rendelésleadás idempotens, és az elosztott tranzakciókat outbox/saga minta kezeli.
- ☐ A fizetési integráció nem tárol kártyaadatot, és a webhook feldolgozás aláírás-ellenőrzött és idempotens.
- ☐ A csúcsterhelésre van előre skálázási terv, és a terhelési modell a Little-törvényt használja.
- ☐ A kritikus üzleti út (katalógus -> kosár -> rendelés -> fizetés) rendelkezik SLO-val és hibakerettel.
- ☐ A nem kritikus funkciók csökkenthetők csúcsidőben.
- ☐ Az architektúradöntések ADR-ben vannak rögzítve.
- ☐ A diagramok diagrams-as-code formában verziózottak.

30.3.22 10. Döntési gyakorlat

Tervezze meg a rendelésfeldolgozási folyamatot egy képzeletbeli e-kereskedelmi rendszerben, amelyben a fizetés PSP-n keresztül történik, és a raktári készlet egy külső rendszerben van. A cél a rendelésleadás és a fizetés visszaigazolása közötti folyamat megtervezése.

Feladatok:

1. Írjon **ADR-t** a szinkron és az aszinkron rendelésfeldolgozás közötti választásról. Indokolja a döntést a válaszidő és a megbízhatóság szempontjából.
2. Adjon meg egy **quality attribute scenario**-t a rendelés-visszaigazolás válaszidejére és a rendelés elvesztésének elkerülésére.
3. Rajzoljon egy **szekvenciadiagramot** a rendelés leadására, beleértve a fizetési webhook érkezését is.

A megoldást ADR formában és diagrams-as-code formában rögzítse.

30.3.23 11. Önellenőrző kérdés

Miért nem elegendő a katalógus-szolgáltatást egyszerűen horizontálisan skálázni, ha a keresési funkció is ugyanabban a szolgáltatásban van?

Válaszvázlat: A keresés nem csupán adatbázis-lekérdezés, hanem relevancia-számítást, indexelést és facettázást igényel. A katalógus olvasási kérései gyorsan, egyszerű kulcsalapú lekérdezéssel kiszolgálhatók, míg a keresés erőforrás-igényes, és eltérő válaszidő-profilt mutat. Ha a kettő egy szolgáltatásban van, a horizontális skálázás nem optimalizálható külön a két terhelési típusra; a keresési csúcsok a katalógus egészét lassíthatják, és a gyorsítótárazási stratégia is keveredik. A szétválasztás lehetővé teszi, hogy mindkét szolgáltatást a saját terhelési profilja szerint skálázzuk, és a kereső indexét külön infrastruktúrán (search engine) tartsuk.

30.3.24 12. Összefoglalás és továbblépés

Ebben a fejezetben a kiskereskedelem és e-kereskedelem három kulcsterületét vizsgáltuk: a szezonális terhelést, a termékkatalógust és a rendelésfeldolgozást. Láttuk, hogy a csúcsterhelés előre tervezést, kapacitásmodellezést és skálázási stratégiát igényel; a katalógus olvasási nehéz, ezért CDN-t, gyorsítótárat és külön keresőszolgáltatást érdemes használni; a rendelésfeldolgozás tranzakciós, állapotgazdag folyamat, amelyet aszinkron eseményekkel, outbox és saga mintákkal, valamint idempotens műveletekkel kezelünk. A fizetési integráció biztonsági és megfelelési követelményei a PCI DSS-hez kapcsolódnak, és a legjobb gyakorlat a kártyaadatok elkerülése PSP-tokenizációval.

A következő egység (Gyártás, logisztika és supply chain – Gyártás, logisztika és supply chain) az itt bevezetett rendelésfeldolgozásra és katalógusra épít: a raktárkészlet, a szállítási folyamatok és a beszállítói integráció mind erre a alapra támaszkodik. Az ott tárgyalt fogalmak – például a készletgazdálkodás, a nyomon követés és az ellátási lánc eseményei – közvetlenül illeszkednek az e-kereskedelmi rendelés-életciklushoz.

A fejezetben bemutatott minták és elvek nem univerzálisak: minden döntésnél a konkrét követelményeket, a csapat méretét, a terhelést és a szervezeti feltételeket kell figyelembe venni. A best practice állítások mögött az az elv áll, hogy a kritikus üzleti út skálázható, megbízható és biztonságos legyen; a kivételek akkor indokoltak, ha a rendszer kicsi, a terhelés alacsony, vagy a tranzakciós garanciák kevésbé szigorúak.

Források és ellenőrzési igény: A fejezetben említett PCI DSS, GS1 és más szabványok hivatalos szövegét minden esetben ellenőrizni kell; a hivatkozások általános ismeretek, nem helyettesítik a hivatalos forrást. A finanszírozási, biztonsági és megfelelési kérdésekben jogi és szabályozói tanácsadás szükséges.

30.4 Gyártás, logisztika és supply chain

A gyártási, logisztikai és ellátási lánc folyamatok digitalizálása során a szoftverarchitektúrának egyszerre kell kezelnie a fizikai világ valós idejű eseményeit, az üzleti rendszerek tranzakciós igényeit és a partnerhálózatok heterogenitását. Ez a fejezet e három erőtér metszéspontjában mutatja be a tipikus architekturális mintákat, döntési helyzeteket és hibákat. A cél nem egy adott iparági keretrendszer ismertetése, hanem az, hogy az architekt a korábban tanult eszközöket – eseményvezérelt minták, C4 modellezés, ADR, quality attribute scenario – konkrétan alkalmazza a gyártási és ellátási lánc kontextusára.

30.4.1 A gyártási és logisztikai környezet sajátosságai

A kiskereskedelmi vagy pénzügyi rendszerekkel szemben a gyártási és logisztikai architektúrák legfontosabb megkülönböztető vonása, hogy a digitális rendszer szorosan kapcsolódik a fizikai folyamatokhoz. Egy raktári komissiózó rendszer nem csupán adatokat mozgat, hanem utasítást ad egy targoncavezetőnek vagy egy automata szállítópályának; egy gyártósori minőségellenőrző rendszer döntése alapján a terméket továbbengedik vagy kiselejtezik. Ebből két következmény adódik:

1. **A késleltetésnek fizikai következménye van.** Ha a gyártósori vezérlőrendszer a megengedettnél később reagál egy érzékelő jelére, az anyag- vagy gépkárt okozhat. Az átlagos válaszidő ezért gyakran nem elegendő mérőszám; a felső percentilist és a determinisztikus felső korlátot kell tervezni.
2. **A rendszerhatár nem csak szoftveres.** A digitális rendszer peremén megjelennek az OT (Operational Technology) hálózatok: PLC-k, SCADA-rendszerek, ipari érzékelők, robotok. Az IT és OT hálózatok gyakran szervezetileg is elkülönülnek, és a köztük lévő adatáramlást biztonsági és megbízhatósági okokból kontrollálni kell.

Emellett az ellátási lánc rendszerek ritkán zártak: a gyártó, a raktározó, a fuvarozó és az értékesítő partner rendszerei közötti adatcsere üzleti előfeltétel. Ezért az integráció nem csupán belső komponensek közötti kommunikációt jelent, hanem szervezetközi, gyakran szabványosított üzenetformátumokon (EDI, GS1, API-k) alapuló adatcserét is.

30.4.2 Rendszerek a gyártási és logisztikai láncban

Az alábbi rendszertípusok gyakran együtt fordulnak elő:

Rendszer	Felelősség	Jellemző integrációs minta
ERP	Vállalati erőforrás-tervezés, pénzügyi és készlet-nyilvántartás	Szinkron API vagy aszinkron üzenet
MES	Gyártás-végrehajtás, műveleti utasítások, gyártási napló	Eseményvezérelt, gyakran OPC UA/ MQTT
SCADA / PLC	Folyamatirányítás, gépi vezérlés, érzékelő adatok	Ipari protokollok, determinisztikus ciklus
WMS	Raktárkezelés, készletmozgások, komissiózás	API és üzenetek, eseményalapú frissítés
TMS	Szállítmányozás, fuvarozás, nyomon követés	API, webhook, EDI
Partnerrend-szerek	Beszállítói, vevői, logisztikai partnerek	EDI, AS2, SFTP, HTTPS webhook, partner API

A tervezés során a rendszerhatárt minden diagramon jelölni kell, és a külső rendszereket, szerepköröket a C4 context szintjén fel kell tüntetni. Ez a gyakorlat segít abban, hogy a heterogén környezetben is egyértelmű legyen, mi tartozik a belső platformhoz, és mi a külső partner.

30.4.3 Valós idejű kihívások: késleltetés, jitter és determinizmus

A gyártási folyamatokban a "valós idejű" kifejezés nem homogén. Három szintet érdemes megkülönböztetni:

- **Kemény valós idejű** rendszerek: a határidő túllépése rendszerhibát okoz. Ilyen például a PLC ciklusideje (gyakran 1–10 ms). Ezt a szintet jellemzően nem általános célú szoftverarchitektúrával, hanem dedikált vezérlőkkel oldják meg; a magasabb szintű rendszerek feladata az adatok begyűjtése és felügyelete.
- **Puha valós idejű** rendszerek: a határidő túllépése elfogadható, de rontja a minőséget. Példa: a MES-nek 200 ms-on belül riasztást kell küldenie, ha egy hőmérséklet-érzékelő túllép egy küszöböt.
- **Közel valós idejű** rendszerek: a feldolgozás másodperces vagy perces ablakon belül történik. Ilyen a készletfrissítés, a rendelésállapot-változás vagy a szállítmánykövető események.

A soft és near real-time tartományokban a legnagyobb architekturális kihívás a **késleltetés szórása (jitter)**, nem csak az átlag. Egy átlagosan 50 ms-os feldolgozási idő mellett is előfordulhat, hogy a 99. percentilis 500 ms, ami egy gyártósori riasztásnál már elfogadhatatlan. A tervezésnél ezért a percentiliseket kell megadni: például "a minőségellenőrző események feldolgozási idejének 99. percentilise < 200 ms, a maximális < 500 ms normál terhelés alatt".

30.4.4 Kommunikációs protokollok kiválasztása

A gyártási és logisztikai rendszerekben a protokollválasztás a késleltetési követelmény, a megbízhatósági szint és a résztvevők képességei alapján történik:

- **OPC UA**: ipari eszközök és rendszerek közötti szabványos, platformfüggetlen kommunikáció. Jó választás MES és SCADA között, mert adatmodellt és biztonságot is definiál. Késleltetése jellemzően 10–100 ms.
- **MQTT**: könnyű, publish-subscribe protokoll, alacsony sávszélességen is működik. Érzékelőktől, járművektől, kézi szkennerektől való telemetria gyűjtésére ideális. A broker központi szerepet kap, de a QoS szintek (0, 1, 2) befolyásolják a kézbesítési garanciát.
- **AMQP / RabbitMQ**: vállalati üzenetküldés gazdag routinggal. Akkor hasznos, ha többféle fogyasztót kell kiszolgálni, és az üzenetek tartós tárolásra szorulnak.
- **HTTP / REST / gRPC**: szinkron API-hívásokra, amikor azonnali válasz szükséges, például egy raktári dolgozó kézi terminálja le kell kérdezzen egy termékinformációt. gRPC alacsonyabb késleltetést és streaminget kínál, de a tűzfalakon való áthaladás és a partnerintegráció kompatibilitása gyakran REST-et kíván.

Fontos elv: **a protokoll nem helyettesíti a szemantikát**. Hiába használunk OPC UA-t, ha az adatmodell nem egységes, vagy az üzenetek idempotens kezelése nem megoldott. A protokoll a szállítási réteg, az architektúra az adat- és vezérlési folyamatokra koncentrál.

30.4.5 Állapotkezelés és stream processing

A gyártósori telemetria gyakran nagy gyakoriságú adatfolyam (pl. rezgésérzékelő 10 kHz-en). Ezt közvetlenül egy relációs adatbázisba írni nem skálázható és nem is szükséges. A modern architektúrák **stream feldolgozó** réteget alkalmaznak (pl. Apache Kafka, Apache Flink, kisebb léptékben MQTT broker szabálymotorral). A folyam feldolgozás során a következő minták jelennek meg:

- **Időalapú ablakok**: gördülő, csúszó vagy session ablakok az aggregáláshoz és anomáliadetekcióhoz.
- **Watermark**: a késve érkező események kezelése, a stream feldolgozó eldönti, meddig vár egy késői eseményre.
- **Állapotteljes feldolgozás**: a folyamat állapotát kell nyilvántartani (pl. egy gyártási ciklus állapota), ezért a feldolgozó komponens megtartja a releváns állapotot, és csak a lényeges eseményeket bocsátja ki.

A kapacitástervezésnél a Little-törvényt használhatjuk: $L = \lambda W$, ahol L a rendszerben lévő kérések/események átlagos száma, λ az érkezési ráta, W az átlagos tartózkodási idő. Például ha egy gyártósor másodpercenként 100 eseményt generál, és az átlagos feldolgozási idő 100 ms, akkor a rendszerben átlagosan $L = 100 \cdot 0,1 = 10$ aktív esemény lesz. Ez segít a pufferelési és skálázási igények becslésében.

30.4.6 Integrációs kihívások: heterogén rendszerek és partnerek

Az ellátási lánc integráció legnagyobb akadálya a heterogenitás. A belső rendszerek eltérő adatmodelleket, azonosítókat és kommunikációs képességeket használnak; a partnerek pedig gyakran az iparági szabványok (EDI, GS1) vagy a saját, régi interfészeik szerint kommunikálnak.

30.4.7 Belső integráció: ERP, MES, WMS, TMS

A négy alaprendszer közötti adatáramlás tipikusan a következő eseményekre bontható:

- A **vevői rendelés** az ERP-ből a MES-be kerül gyártási utasításként.
- A **gyártási késztermék** a MES-ből a WMS-be kerül készletre vételre.
- A **raktári készletváltozás** frissíti az ERP készletnyilvántartását.
- A **szállítási megbízás** a WMS-ből a TMS-be kerül, ahonnan a fuvarozó nyomonkövetési eseményei visszafelé áramlanak.

Az integrációs minták közül gyakran alkalmazzák az **eseményvezérelt megközelítést**, mert a folyamatok aszinkron természetűek, és a laza csatolás csökkenti a rendszerek közötti függőséget. Ugyanakkor az **API-alapú szinkron hívás** is jelen van, amikor azonnali válasz szükséges (pl. egy raktári terminál lekéri a termék rendelkezésre állását). A döntést a következő tényezők befolyásolják:

- **Konzisztenciaigény**: ha a két rendszer állapotának erős konzisztenciában kell lennie, a szinkron hívás egyszerűbb, de növeli a csatolást.
- **Hibatűrés**: az aszinkron üzenet pufferelhető, a fogyasztó leállása esetén az üzenet nem vész el.
- **Késleltetés**: a szinkron hívás azonnali, az aszinkron üzenetnél a fogyasztó feldolgozási ideje hozzáadódik.
- **Visszajelzés**: ha az ügyfélnek azonnali visszaigazolás kell, a szinkron API jobb.

30.4.8 Partnerintegráció és szabványok

A partnerekkel való adatcsere leggyakoribb formái:

- **EDI (Electronic Data Interchange)**: szabványos üzenetformátumok (EDIFACT, ANSI X12) és átviteli protokollok (AS2, SFTP, VAN). A logisztikában az EDI a rendelések, szállítási értesítések, számlák cseréjére elterjedt. Az EDI üzenetek feldolgozása gyakran kötegelt, a valós idejű API-k mellett azonban háttérbe szorul.
- **GS1 szabványok**: a GS1 azonosítók (GTIN a termékhez, SSCC a logisztikai egységhez, GLN a telephelyhez) és az EPCIS (Electronic Product Code Information Services) a nyomon követés alapjai. Az EPCIS szabványos eseményformátumot ad az ellátási lánc láthatóságához: például `ObjectEvent`, `AggregationEvent`, `TransactionEvent`. Az architektúrában az EPCIS-t gyakran a belső események publikálására és a partnerekkel való megosztásra használják.
- **Webhook / HTTP API**: a modern partnerek egyre inkább REST API-kat és webhookokat használnak. A webhook feldolgozásánál a kézbesítési garancia gyakran "legalább egyszer", ezért a fogyasztónak idempotensnek kell lennie.
- **AS2 / SFTP**: fájl alapú, kötegelt átvitel, gyakran szabályozói vagy partner-kényszer miatt marad fenn.

A partnerintegrációban gyakori hiba, hogy a belső modellt közvetlenül kiteszik a partnereknek, ami szoros csatolást és adatszivárgást okoz. A helyes minta az **anti-corruption layer** (ACL): egy dedikált komponens, amely a külső üzenetformátumot belső modellre fordítja. Az ACL tartalma a partnerrel kötött szerződés, így a belső változások nem törnek meg minden partnert.

30.4.9 Példa: integrációs vázlet C4 szinten

A következő C4 context diagram egy tipikus ellátási lánc integrációs szcenáriót mutat:

```
graph LR
    subgraph "Gyártó rendszere (context)"
        MES[MES rendszer]
        ERP[ERP rendszer]
        WMS[WMS rendszer]
        TMS[TMS rendszer]
        Platform[Supply Chain Integration Platform]
        MES -- "OPC UA / MQTT" --> Platform
        ERP -- "REST / üzenet" --> Platform
        WMS -- "REST / üzenet" --> Platform
        TMS -- "REST / üzenet" --> Platform
    end
    Partner1[Beszerzési partner] -- "EDI AS2" --> Platform
    Partner2[Logisztikai partner] -- "HTTPS webhook" --> Platform
    Partner3[Vevői partner] -- "EPCIS / REST" --> Platform
    User[Üzleti felhasználó] -- "HTTPS" --> Platform
```

Jelmagyarázat: a belső rendszerek a gyártó contextben, a külső partnerek a contexten kívül; a kapcsolatokon a protokoll szerepel. A platform belső részleteit container szinten tovább bontjuk.

A container diagram finomítja a platformot:

```
graph LR
    subgraph "Supply Chain Integration Platform (container)"
        Ing[Ingress Gateway]
        Norm[Message Normalizer]
        Router[Event Router]
        Inv[Inventory Service]
        Trace[Traceability Service]
        Outbox[Outbox Publisher]
        Ing -- "HTTP / MQTT" --> Norm
        Norm -- "normalizált esemény" --> Router
        Router -- "parancs" --> Inv
```

```
    Router -- "esemény" --> Trace
    Trace -- "EPCIS esemény" --> Outbox
    Outbox -- "HTTPS / AS2" --> Partner2
    Outbox -- "HTTPS / AS2" --> Partner3
end
MES -- "OPC UA / MQTT" --> Ing
ERP -- "REST / üzenet" --> Ing
WMS -- "REST / üzenet" --> Ing
TMS -- "REST / üzenet" --> Ing
```

Jelmagyarázat: az Ingress Gateway fogadja a külső és belső rendszerek üzeneteit, a Normalizer egységes belső eseménymodellre fordít, az Event Router irányítja a fogyasztókhoz, az Inventory Service és Traceability Service dolgozzák fel, az Outbox Publisher pedig a partnerek felé továbbítja a szükséges eseményeket. A kapcsolatokon a protokoll látható; a szinkronitás és hibatűrés a komponensek leírásában szerepel.

30.4.10 Események, telemetria és partnerek

A gyártási és logisztikai rendszerekben három eltérő jellegű adatáram keveredik, és ezeket nem szabad összemosni:

- **Esemény**: diszkrét, múltbéli tény, például “a rendelés létrejött”, “a raklap elhagyta a raktárat”. Az esemény tény, nem kérés; nem utasít, hanem rögzít.
- **Parancs**: szándékos utasítás, például “indítsd el a gyártási műveletet”, “foglald le a készletet”. A parancs címzettje egy másik rendszer, és a végrehajtás eredménye gyakran egy esemény.
- **Telemetria**: folyamatos, gyakran nagy gyakoriságú mérési adat, például hőmérséklet, rezgés, járműpozíció. A telemetria nem feltétlenül kapcsolódik üzleti tranzakcióhoz, de feldolgozása időablakokban történik.

A jól tervezett architektúrában a három adattípust eltérő csatornán kezelik. A telemetria például nem terheli a tranzakciós üzenetküldőt, hanem egy stream feldolgozó platformra kerül. Az események a szolgáltatások közötti lazán csatolt kommunikáció eszközei. A parancsok gyakran szinkron API-hívások vagy dedikált parancs-üzenetek.

30.4.11 Telemetria gyűjtése és feldolgozása

A gyártósori érzékelők tipikusan MQTT-n vagy OPC UA-n keresztül publikálnak. A felhőbe vagy az edge-re telepített **telemetria gateway** feladata az adatok összegyűjtése, normalizálása és továbbítása egy stream platformra (pl. Kafka, MQTT broker). A feldolgozó réteg az alábbi mintákat valósítja meg:

- **Digitális iker**: a fizikai eszköz virtuális reprezentációja, amely az állapotát valós időben tükrözi. A digitális iker lehetővé teszi a szimulációt, a prediktív karbantartást és a riasztást. Architekturálisan a digitális iker egy állapotteljes szolgáltatás, amely a telemetriát fogyasztja, és az állapotváltozásokat eseményként publikálja.
- **Anomáliadetekció**: a stream feldolgozó ablakokban számít statisztikát, és küszöbátlépés esetén riasztást generál.
- **Adatredukció**: a nyers telemetria tárolása költséges, ezért az edge gateway gyakran szűr, aggregál vagy tömörít, mielőtt a felhőbe küldené.

30.4.12 Nyomon követhetőség és EPCIS

Az ellátási láncban a nyomon követhetőség (traceability) azt jelenti, hogy egy adott termék vagy logisztikai egység útja a forrástól a fogyasztóig rekonstruálható. Ehhez a következőkre van szükség:

- **Globálisan egyedi azonosítók** (GTIN, SSCC, tételszám, sorozatszám).
- **Események** rögzítése minden releváns ponton (gyártás, csomagolás, szállítás, raktározás, értékesítés).
- **A események közötti kapcsolatok** (pl. egy tétel bekerül egy dobozba, a doboz egy raklapra).

Az EPCIS szabvány pontosan ezt formalizálja. A belső architektúrában a `Traceability Service` fogadja a normalizált eseményeket, és EPCIS formátumú rekordokat hoz létre. A partnerekkel való megosztás történhet EPCIS query interfészen vagy üzenetként. A traceability szolgáltatásnak **írásban idempotensnek** kell lennie, mert ugyanaz az esemény többször is megérkezhet.

30.4.13 Partner események és webhookok

A partnerek gyakran webhookokat hívnak meg, amikor egy logisztikai esemény történik (pl. "a csomag kiszolgáltatva"). A webhook feldolgozás során a következő hibák tipikusak:

- **Nem idempotens fogyasztó**: ugyanaz a webhook többször érkezik, és a rendszer duplikált eseményt hoz létre.
- **Nincs hitelesítés**: a webhook végpont nem ellenőrzi a hívó azonosságát, így hamis eseményekkel lehet adatot beszennyezni.
- **Szinkron feldolgozás túl hosszú**: a webhook fogadója a feldolgozás teljes idejéig blokkol, ami a partner oldalán timeoutot okoz.

A megoldás: a webhook fogadó azonnal nyugtáz (202 Accepted), az eseményt egy belső üzenetsorba teszi, és a feldolgozás aszinkron történik. A hitelesítéshez szignált payload vagy API kulcs szükséges, és a duplikátumkezeléshez idempotencia kulcs (pl. a partner által küldött egyedi eseményazonosító) tárolása.

30.4.14 Megbízhatóság, késleltetés és nyomon követhetőség tervezése

Ebben a szakaszban a minőségi követelményeket quality attribute scenario formában fogalmazzuk meg, és a megbízhatósági mintákat alkalmazzuk a gyártási, logisztikai kontextusra.

30.4.15 Quality attribute scenario példa

Inger: Egy logisztikai partner webhookja "csomag kiszolgáltatva" eseményt küld.
Környezet: Normál terhelés, a rendszer 100 esemény/perc ütemben fogadja a partnerektől, a hálózati késleltetés < 200 ms.
Válasz: A rendszer 500 ms-on belül feldolgozza az eseményt, frissíti a szállítmány állapotát, és a vevői nyomon követési felületen megjeleníti. A feldolgozás során az esemény azonosítóját idempotencia-kulcsként használja, így a duplikátum küldés nem okoz kettős frissítést.
Mérőszám: A feldolgozási idő 99. percentilise < 500 ms, a sikeres feldolgozás aránya $> 99{,}9\%$, adatvesztés 0 az adott hónapban. Mérve: 5 perces csúszó ablakban.

30.4.16 Megbízhatósági minták

A korábban tanult reziliencia minták itt is érvényesek, de a fizikai folyamatok miatt a következmények súlyosabbak lehetnek:

- **Retry és timeout**: a retry mechanizmusnál meg kell különböztetni az idempotens és nem idempotens műveleteket. A partner API-hívásnál az idempotencia kulcs használata kötelező, ha duplikált hívás kárt okozhat.
- **Circuit breaker**: ha a partnerrendszer lelassul vagy hibázik, a megszakító megakadályozza, hogy a belső rendszer erőforrásai kimerüljenek. A megszakító nyitott állapota alatt az eseményeket gyakran pufferelni vagy dead-letter queue-ba kell helyezni.
- **Bulkhead**: a partnerenként elkülönített feldolgozási sávok biztosítják, hogy egy partner hibája ne bénítsa meg a teljes integrációs platformot.
- **Backpressure**: ha a beérkező események üteme meghaladja a fogyasztó kapacitását, a fogyasztónak jeleznie kell a termelőnek, hogy lassítson. MQTT-nél ez nem mindig lehetséges, ezért a broker pufferelése és a lemezre írás elengedhetetlen.

A **SLO/SLI** definiálása a logisztikai rendszerben is fontos. Például:

- **SLI:** a szállítási esemény feldolgozási idejének 99. percentilise.
- **SLO:** az SLI értéke < 500 ms az esetek 99%-ában egy naptári hónapban.
- **Error budget:** $1 - SLO$, azaz ha a havi 99%-os megbízhatósági cél, akkor a havi 1% hibakeret. Ez alapján lehet dönteni a release-ek kockázatáról.

30.4.17 Nyomon követhetőség szabályozói és üzleti okokból

Sok iparágban (élelmiszer, gyógyszer, autóipar, veszélyes anyagok) a nyomon követhetőség jogszabályi követelmény. Az architektúrának ezért biztosítania kell:

- **Megváltoztathatatlanságot** a nyers eseményekre: a traceability rekordokat append-only módon kell tárolni, és a módosítást tiltani vagy naplózni.
- **Auditálhatóságot**: minden hozzáférés és módosítás naplózott, az audit napló nem törölhető.
- **Megfelelő adatmegőrzést**: a rekordokat a szabályozás által előírt ideig meg kell őrizni.
- **Korrelációs azonosítót**: az események közötti kapcsolatok (pl. rendelés → gyártási tétel → szállítmány) egyértelműen feloldhatók.

A nyomon követhetőségi szolgáltatásnak az **idempotens írás** mellett az **event sourcing** mintát is érdemes használnia: az eseményeket tényként tárolják, és az aktuális állapotot ezekből lehet visszajátszani. Ez biztosítja az auditálhatóságot és a visszaállíthatóságot.

30.4.18 Architektúra vázlat: ellátási lánc integrációs példa

A fenti elemeket egy konkrét, de egyszerűsített példán keresztül foglaljuk össze. A példa egy középvállalati gyártó integrációs platformját mutatja, amely a belső rendszereket és a külső partnereket köti össze.

30.4.19 Komponensek és felelősségi körök

- **Ingress Gateway**: fogadja a külső és belső rendszerek üzeneteit HTTP-n, MQTT-n, AS2-n. Felelős a hitelesítésért, a nyers üzenetek rögzítéséért (audit), és a megfelelő csatornára irányításért.
- **Message Normalizer**: a különböző formátumokat (EDI, JSON, OPC UA adatmodell) egységes belső eseménymodellre fordítja. Minden bejövő üzenethez belső eseményazonosítót és forrás-azonosítót rendel.
- **Event Router**: a normalizált eseményeket topicokra irányítja a fogyasztók felé. Garantálja a sorrendet egy adott entitáson belül (pl. egy rendelés eseményei), de különböző entitások között nem ad sorrendi garanciát.
- **Inventory Service**: a készletváltozási eseményeket fogyasztja, és fenntartja a készlet aktuális állapotát. A műveletek idempotensek, és a szolgáltatás szinkron API-val is elérhető a lekérdezésekhez.
- **Traceability Service**: az EPCIS-kompatibilis eseményeket tárolja és kiszolgálja. Az írás append-only, az olvasás REST API-n keresztül történik.
- **Outbox Publisher**: a partnerek felé továbbítandó eseményeket gyűjti és küldi ki a megfelelő protokollon. Az outbox minta biztosítja, hogy az üzleti tranzakció és az üzenetküldés atomi legyen.

30.4.20 Döntési pont: API vs. eseményvezérelt integráció

A következő ADR példa a döntést dokumentálja:

ADR-001: Aszinkron eseményvezérelt integráció a belső rendszerek között

Kontextus: A gyártó négy belső rendszere (ERP, MES, WMS, TMS) között a folyamatok túlnyomórészt aszinkronok: a gyártási utasítás kiadása, a készletváltozás, a szállítási megbízás létrejötte nem igényel azonnali visszaigazolást a kezdeményező oldalán. A rendszerek eltérő rendelkezésre állásúak, és a szoros csatolás a teljes lánc leállásához vezethet.

Döntés: A belső rendszerek közötti integrációhoz eseményvezérelt architektúrát használunk, az Event Routeren keresztül. A szinkron API-hívásokat csak lekérdezésre és olyan műveletekre korlátozzuk, ahol azonnali válasz szükséges (pl. raktári terminál).

Alternatívák: - Teljesen szinkron REST API-hívások: egyszerűbb, de szoros csatolást és gyenge hibatűrést eredményez. - Kötegelt fájltranszfer: alacsony késleltetésű folyamatokra alkalmatlan. - Közvetlen adatbázis-integráció: tiltott, mert megsérti a rendszerhatárokat és a biztonsági elveket.

Következmények: A rendszerek közötti csatolás laza, de az események sorrendjének és idempotenciájának kezelése bonyolultabb. Az Event Router üzemeltetése új készségeket igényel. A rendszerek közötti adatkonzisztencia végül konzisztens, de átmenetileg inkonzisztens lehet.

Állapot: Elfogadott, 2025-03-15, felülvizsgálat 2026-03-15.

30.4.21 Quality attribute scenario a nyomon követésre

Inger: A felhasználó lekérdezi egy termék tételszámának teljes útját a gyártástól a kiszolgáltatásig.
Környezet: Normál terhelés, a traceability szolgáltatás 50 lekérdezést kap percenként.
Válasz: A rendszer 2 másodpercen belül visszaadja az események időrendi listáját, beleértve a partnerrendszerektől származó eseményeket is.

Mérőszám: A válaszidő 95. percentilise < 2 s, a lekérdezés sikeressége 99,9%, az adatok frissessége < 1 perc.

30.4.22 Labor: ellátási lánc integrációs vázlat

A labor célja egy kis, ellenőrizhető integrációs vázlat elkészítése dokumentumban vagy jegyzetfüzetben. A labor nem igényel futtatható kódot, a hangsúly a tervezési döntések és a dokumentáció minőségén van.

30.4.23 Kiinduló állapot

Adott egy fiktív vállalat, a "Minta Gyártó Zrt.", amely elektronikai alkatrészeket gyárt, raktároz, és három logisztikai partnerrel dolgozik. A belső rendszerek: ERP (SAP S/4HANA), MES (egyedi fejlesztés), WMS (Körber), TMS (Transporeon). A partnerek: egy beszállító EDI-n, egy fuvarozó webhookkal, egy vevő EPCIS lekérdezéssel. A feladat az integrációs platform architektúrájának megtervezése.

30.4.24 Lépések

1. **C4 context diagram** készítése: tüntesd fel a belső rendszereket és a külső partnereket, szerepköröket, és a közöttük lévő kapcsolatokat protokollal. A rendszerhatárt egyértelműen jelöld.
2. **Container diagram** bontása a platformra: definiálj legalább négy konténert (pl. gateway, normalizer, router, traceability). Minden konténerhez írj felelősséget és interfészt.
3. **Eseménymodell** definiálása: válassz ki három üzleti eseményt (pl. rendelés létrehozva, gyártás kész, csomag kiszolgáltatva). Mindegyikhez add meg az azonosítókat (GTIN, SSCC, belső azonosító), a forrást és a célokat.
4. **Döntési napló** készítése: legalább két ADR-t írj, az egyiket a belső integráció stílusáról, a másikat a partneri webhook kezeléséről. Használd a ADR sablont: kontextus, döntés, alternatívák, következmények, állapot.
5. **Quality attribute scenario** megfogalmazása: egyet a webhook feldolgozás késleltetésére, egyet a traceability lekérdezésre. Az inger, környezet, válasz és mérőszám mezőket töltsd ki.
6. **Ellenőrzési szempontok** rögzítése: milyen feltételek mellett tekinthető a vázlat helyesnek? Például: minden kapcsolat protokollal rendelkezik, a komponensek felelősségi köre egyértelmű, az események idempotencia kulccsal rendelkeznek.

30.4.25 Elvárt eredmény

A dokumentum tartalma:

- Egy C4 context diagram (Mermaid, PlantUML vagy Structurizr DSL formában).
- Egy C4 container diagram a platformról.
- Legalább három esemény definíciója.
- Két ADR.
- Két quality attribute scenario.
- Ellenőrzőlista.

30.4.26 Ellenőrzési szempontok

- A C4 diagramokon a rendszerhatár és a külső szereplők egyértelműek.
- A kapcsolatokon a protokoll, a szinkronitás és a hibatűrés (retry, timeout, circuit breaker) jelölve van, vagy a szöveges leírás tartalmazza.
- Az események azonosítói egyediek, és a duplikátumkezelés módja kiderül.
- Az ADR-ek tartalmazzák az alternatívákat és a következményeket.
- A mért és várt eredmény elkülönítése megtörtént: a quality attribute scenario-kban a mérőszámokhoz mérési ablak és környezet tartozik, és a nem mért értékeket "várt" vagy "tervezett" jelöléssel látják el.

30.4.27 Visszaállítás

A labor dokumentum alapú, visszaállítás nem szükséges. Ha valaki futtatható PoC-t készít (pl. MQTT broker szimulációval), akkor izolált környezetben, konténerben tegye, és a törléshez a `docker compose down -v` paranccsal törölheti a köteteket. A destruktív műveleteket csak izolált környezetben szabad futtatni.

30.4.28 Ellenőrzőlista

- A rendszerhatárok egyértelműek, a külső partnerek és belső rendszerek elkülönítve.
- A valós idejű követelményekhez megfelelő protokollt választottak (OPC UA, MQTT, HTTP, AS2).
- Az esemény, parancs és telemetria adattípusok szét vannak választva.
- A telemetria feldolgozásához stream platform vagy edge gateway van tervezve.
- A nyomon követhetőséghez globális azonosítók (GTIN, SSCC, tételszám) használatosak.
- Minden partneri üzenet feldolgozása idempotens, és az idempotencia kulcs tárolva van.
- A hibatűrési minták (retry, timeout, circuit breaker, bulkhead) alkalmazása dokumentált.
- A SLO/SLI és az error budget definiálva van, mérési ablakkal.
- Az ADR-ek rögzítve vannak, tartalmazzák az alternatívákat és a következményeket.
- A labor eredményében a mért és várt értékek elkülönítettek.

30.4.29 Önellenőrző kérdés

Miért nem elegendő az átlagos válaszidő a gyártósori riasztási rendszer minőségi követelményének megadásához?

Válasz: Az átlag elrejti a kiugró értékeket, amelyek a fizikai folyamatban kárt okozhatnak. A gyártósori rendszernél a késleltetés felső korlátját (például 99. percentilis vagy maximális érték) kell megadni, mert a vezérlés determinisztikus viselkedést igényel. A magas szórású válaszidő miatt a rendszer időnként késhet, ami a termelés leállásához vezethet.

30.4.30 Összegzés és kitekintés

Ez a fejezet bemutatta, hogyan alkalmazhatók a szoftverarchitektúra általános elvei a gyártási, logisztikai és ellátási lánc kontextusában. A fizikai világ valós idejű kihívásai, a heterogén rendszerek integrációja és a partneri adatcsere együttesen olyan architektúrát igényelnek, amelyben az eseményvezérelt minták, a szabványos azonosítók és az erős megbízhatósági tervezés együttműködnek. A következő fejezet (AI/ML és LLM-rendszerek architektúrája) az AI/ML és

LLM-rendszerek architektúrájával foglalkozik, ahol az itt bevezetett adatfolyam-, telemetria- és integrációs fogalmak tovább épülnek.

31 Modern trendek és fejlett architektúrák

31.1 AI/ML és LLM-rendszerek architektúrája

A nagy nyelvi modellek (LLM-ek) és a gépi tanulási (ML) rendszerek köré épülő architektúrák az elmúlt években a szoftvertervezés egyik leggyorsabban változó területévé váltak. A korábbi fejezetekben tárgyalt elvek – modularitás, laza csatolás, minőségi forgatókönyvek, ADR-ek – itt is érvényesek, de az ML-komponensek sajátos életciklusa és üzemeltetési jellemzői miatt új mintákra és döntési pontokra van szükség. Ez a fejezet a modellek, az adatpipeline és az alkalmazásintegráció hármasát járja körül, majd bemutatja a tanítás, az inferencia, a RAG és az ágens alapú rendszerek architekturális kérdéseit, végül kitér a költség, a késleltetés, a biztonság és az értékelés szempontjaira.

31.1.1 A modellek, az adatpipeline és az alkalmazásintegráció hármasa

Egy AI/ML rendszer architektúrája akkor válik érthetővé, ha három, eltérő életciklusú komponenscsoportot különböztetünk meg.

Modell. A modell a betanított matematikai leképezés: bemeneti adatokból előrejelzést, osztályozást, generált szöveget vagy más kimenetet állít elő. Architekturális értelemben a modell egy verziózott artefaktum, amelyet a tanító pipeline állít elő, a modellregiszter tárol, az inferencia szolgáltatás pedig betölt. A modell nem kód, ezért a hagyományos kódverziózás nem elegendő; a modellhez tartozik a tanító adathalmaz, a hiperparaméterek, a tanító környezet és a kiértékelési eredmény is.

Adatpipeline. Az adatpipeline felel az adatok begyűjtéséért, tisztításáért, transzformációjáért, feature-ök előállításáért és a tanító, illetve az inferencia számára elérhető adatkészletek előkészítéséért. Az adatpipeline nem egyszeri folyamat, hanem folyamatosan üzemelő rendszer, amelynek minősége közvetlenül befolyásolja a modell minőségét. Az adatpipeline architektúrájában meg kell különböztetni a batch és a streaming feldolgozást, a feature store-t, valamint az adatváltozatok és az adatminőségi mutatók kezelését.

Alkalmazásintegráció. Az alkalmazásintegráció az a réteg, amely a modellt a felhasználói vagy üzleti alkalmazások számára elérhetővé teszi. Ide tartozik az inferencia API, a promptkezelés, a válaszok utófeldolgozása, a kontextusösszeállítás, a nyomkövetés és a visszacsatolási hurkok. Az integráció nem egyszerűen egy REST-végpont; olyan kiszolgáló réteget jelent, amely kezeli a modell korlátait (például hallucinációt, kontextusablakot, késleltetést) és az üzleti folyamatokhoz illeszti a kimenetet.

Ez a három réteg gyakran eltérő csapatokhoz és eltérő üzemeltetési ciklusokhoz tartozik. Az adatpipeline-t jellemzően adatmérnökök üzemeltetik, a modellt ML-mérnökök vagy adattudósok, az alkalmazásintegrációt pedig szoftvermérnökök. Az architektúra akkor működik jól, ha a határok világosak, a szerződések explicit módon rögzítettek, és a három réteg közötti visszacsatolás mérhető.

31.1.2 A tanítás és az inferencia szétválasztása

A tanítás (training) és az inferencia (serving) eltérő erőforrásigényű és eltérő üzemeltetési profilú folyamat. A tanítás általában GPU-alapú, kötegelt, hosszú ideig futó, ritkábban ismétlődő folyamat; az inferencia ezzel szemben gyakran alacsony késleltetésű, nagyszámú párhuzamos kérést kiszolgáló, folyamatosan elérhető szolgáltatás.

Ezt a szétválasztást az architektúrában is érdemes megjeleníteni. A tanító alrendszer különálló, batch jellegű, gyakran időzített vagy eseményvezérelt komponens. A modellregiszter központi szerepet játszik: a tanítás után ide kerül az új modellverzió a kiértékelési eredményekkel és a használathoz szükséges metaadatokkal együtt. Az inferenciaszolgáltatás a regiszterből töltheti be az élesített verziót, például canary vagy blue-green stratégia szerint.

A tanítás és az inferencia közötti eltérésből adódik az egyik leggyakoribb architekturális hiba: a train–serve skew. Ez azt jelenti, hogy a tanítás során használt adatelőkészítési lépések nem egyeznek meg az inferenciánál alkalmazott lépésekkel. Például a tanításnál normalizálják a bemeneti jellemzőket, de az éles rendszerben elfelejtik ugyanezt a normalizálást alkalmazni. A megoldás a feature pipeline és a modell együtt verziózása, valamint az úgynevezett feature store használata, amely biztosítja, hogy a tanításhoz és az inferenciához ugyanazon transzformációs logika szerint előállított jellemzők álljanak rendelkezésre.

Az LLM-ek esetében a tanítás jellemzően három szakaszra bontható: előtanítás (pre-training), finomhangolás (fine-tuning) és emberi visszajelzés alapú megerősítéses tanítás (RLHF). Az előtanítás hatalmas, általános szövegkorpuszon történik, és ritkán ismétlődik. A finomhangolás kisebb, célzott adathalmazon történik, és gyakrabban futhat. Az RLHF a modell válaszait emberi preferenciák alapján rangsorolja, és ebből tanul. Architekturális szempontból mindhárom fázis külön pipeline-t, külön kiértékelési küszöböt és külön regiszterbejegyzést igényel.

31.1.3 Az inferencia architektúrája és a kiszolgálási minták

Az inferencia szolgáltatás tervezésekor a fő minőségi attribútum a késleltetés, az átviteli kapacitás és a költség. Az LLM-ek inferenciája erőforrás-intenzív: minden egyes token generálása számítást igényel, ezért a válaszidő nem lineárisan függ a bemenet hosszától, hanem a generált tokenek számától is.

A kiszolgálás jellemző mintái:

- **Valós idejű (online) inferencia:** alacsony késleltetésű API, általában egy GPU-alapú konténerben futó modellszerver. A kérések szinkron módon érkeznek, és a válaszidőnek szigorú SLO-ja van. Ez a minta alkalmas chatbotokhoz, keresőasszisztensekhez.
- **Kötegelt (batch) inferencia:** nagy adathalmazon, offline módon futtatott előrejelzés. A késleltetés kevésbé kritikus, az átviteli kapacitás fontosabb. Ezt gyakran adatpipeline-ba ágyazzák, például dokumentumok osztályozására vagy címkézésére.
- **Aszinkron inferencia:** a kérés beérkezik, a válasz később, például callback vagy üzenetsor segítségével érkezik. Ez akkor hasznos, ha a modell futtatása hosszú, vagy ha a válasz előállítása nem igényel azonnali felhasználói interakciót.

A modellszerverek – például a vLLM, a TensorRT-LLM vagy a TGI – a folyamatos batcheléssel és a kvantálással növelik az átviteli kapacitást, de ezek a technikák a modell pontosságát is befolyásolhatják. Az architektúrának ezért rögzítenie kell, hogy milyen pontossági küszöb mellett engedélyezett a kvantálás, és hogyan mérik a pontosságromlást a kiértékelési pipeline-ban.

31.1.4 RAG: Retrieval-Augmented Generation

A RAG (visszakereséssel kiegészített generálás) az LLM-ek egyik legfontosabb architekturális mintája. Célja, hogy a modell válaszait külső, friss vagy vállalatspecifikus tudással egészítse ki anélkül, hogy a modellt minden alkalommal újra kellene tanítani. A RAG architektúra a következő komponensekre bontható:

1. **Dokumentum-feldolgozó pipeline:** nyers dokumentumok (PDF, HTML, wiki, adatbázisrekordok) beolvasása, szöveg kinyerése, tisztítása, darabolása (chunking). A darabolás mérete és átfedése kritikus: a túl nagy darabok zajt visznek a kontextusba, a túl kicsik elveszítik a szövegösszefüggést.
2. **Embedding-generálás:** a szövegrészeket vektoros reprezentációvá alakítja egy embedding-modell. A vektorokat egy vektoradatbázisban tárolják, amely hasonlósági keresést tesz lehetővé.
3. **Visszakereső (retriever):** a felhasználói kérdést embedding-gé alakítja, majd a vektoradatbázisból kikeresi a leginkább hasonló dokumentumdarabokat. A visszakeresés lehet egyszerű hasonlósági keresés, de gyakran hibrid megoldást alkalmaznak, amely a szöveges keresést és a vektoros keresést kombinálja.
4. **Újrarendező (reranker):** a visszakeresett jelölteket egy finomabb, kereszt-attention alapú modell újrarendezi, hogy a legrelevánsabb darabok kerüljenek a kontextusba.
5. **Generáló (LLM):** a modell megkapja a felhasználói kérdést és a visszakeresett kontextust, majd generálja a választ. A promptban explicit jelölni kell, hogy mely részek származnak a kontextusból, és melyek a modell általános tudásából.

A RAG architektúrában a legfontosabb minőségi attribútum a relevancia és a hűség. A relevancia azt méri, hogy a visszakeresett dokumentumok ténylegesen válaszolnak-e a kérdésre; a hűség pedig azt, hogy a generált válasz nem mond ellent a kontextusnak, és nem hallucinál. Mindkettőt a kiértékelési pipeline-ban kell mérni, nem elég a modell általános pontosságát nézni.

A RAG pipeline tipikus hibája, hogy a visszakeresés minőségét nem mérik külön. Ha a retriever rossz darabokat ad a generálónak, a válasz akkor is rossz lesz, ha maga az LLM kiváló. Ezért a RAG architektúrában külön SLO érvényes a visszakeresés találati arányára (például recall@5) és külön a generált válasz hűségére.

A RAG egyik fontos döntési pontja a kontextusablak-kezelés. A modern LLM-ek kontextusablaka nagy, de a költség és a késleltetés a bemeneti tokenek számával nő. Az architektúrának ezért konszolidálnia kell, hogy hány dokumentumdarab fér be a promptba, és mi történik, ha a visszakeresés nem talál elegendő releváns anyagot. Ilyenkor a rendszernek explicit módon kell jeleznie a bizonytalanságot, nem pedig kitalált választ adnia.

31.1.5 Ágensek és eszközhasználat

Az LLM-alapú ágensek olyan rendszerek, amelyek a modellt nem egyszeri kérdés-válasz párokra használják, hanem egy cél elérése érdekében több lépésben, eszközöket hívva, visszacsatolást feldolgozva működnek. Az ágens architektúra a következő elemekből áll:

- **Cél és terv:** a felhasználó vagy a rendszer által megadott cél, amelyet az ágens részcélokra bont.
- **Eszköztár:** API-k, adatbázis-lekérdezések, webes keresés, kalkulátor, kódvégrehajtás – minden olyan képesség, amelyet az LLM meghívhat.
- **Memória:** rövid távú memória a párbeszéd kontextusához, hosszú távú memória a korábbi interakciókhoz vagy tartós tudáshoz.

- **Rezonálás (reasoning loop):** az ágens ciklikusan dönt: milyen lépést tegyen, milyen eszközt hívjon, hogyan értelmezze az eredményt, és mikor álljon meg.

A ReAct (Reason + Act) minta az egyik legismertebb megvalósítás: az LLM először gondolkodik, majd cselekvést hajt végre, megfigyeli az eredményt, és így iterál. Ez a hurok alkotja az ágens magját.

Architekturális szempontból az ágenseknél a legnagyobb kockázat az autonómia növekedése. Minden eszközhívás mellékhatással járhat: adatbázis-módosítás, külső API-hívás, e-mail küldés. Ezért az ágens architektúrában élesen el kell választani a csak olvasható eszközöket a módosító eszközöktől. A módosító eszközökhöz emberi jóváhagyás, tranzakciós határ vagy legalább auditnapló szükséges. A zero trust elv itt is érvényes: az ágensnek csak azokhoz az erőforrásokhoz lehet hozzáférése, amelyekhez a feladatához szükséges, és minden hozzáférését identitáshoz kell kötni.

Az ágensek tervezésekor a determinizmus hiánya miatt a fitness function-ök és a visszajátszhatóság kulcsfontosságú. Minden ágens-futásnak naplóznia kell a promptot, az eszközhívásokat, azok eredményét és a végső választ, hogy a hibákat reprodukálni lehessen. Ez a nyomkövetés nemcsak a hibakeresést szolgálja, hanem a kiértékeléshez és a modell finomhangolásához is adatot szolgáltat.

31.1.6 Költség, késleltetés, biztonság és értékelés

Egy AI/ML rendszer architektúrájának négy további dimenziója külön figyelmet érdemel: a költség, a késleltetés, a biztonság és az értékelés. Ezeket nem lehet utólag hozzáragasztani a rendszerhez; a tervezés korai szakaszában minőségi forgatókönyvekkel kell rögzíteni őket.

Költség. Az LLM-inferencia költsége nagyjából a feldolgozott tokenek számával arányos. A költségtervezéshez szükséges ismerni a modell árazását (például USD per 1 millió token), a várható kérésforgalmat és az átlagos tokenfogyasztást kérésenként. A Little-törvény itt is alkalmazható: ha az átlagos érkezési ráta λ, az átlagos válaszidő W, akkor a rendszerben lévő kérések átlagos száma $L = \lambda W$. Ebből és az egy kérésre jutó átlagos tokenköltségből becsülhető az időegységre eső költség. A költségcsökkentés tipikus eszközei: kisebb modell választása, kvantálás, cache-elés (különösen a prompt cache), batch feldolgozás és a RAG használata a drága finomhangolás helyett.

Késleltetés. Az LLM-eknél a késleltetést két összetevőre kell bontani: az első tokenig eltelt idő (time to first token, TTFT) és a tokenenkénti generálási idő (inter-token latency). A TTFT a bemeneti tokenek feldolgozásától függ, a generálási sebesség a modell méretétől és a hardvertől. A minőségi forgatókönyvben mindkettőt meg kell adni, például: „inger: 1 párhuzamos kérés érkezik a chat-végpontra; környezet: egy GPU-példány, 100 token bemenet; válasz: a TTFT p95 500 ms alatt, a generálási sebesség legalább 20 token/s; mérőszám: p95 TTFT ms-ban és token/s, 5 perces ablakban.” A késleltetés csökkentése történhet streaminggel, gyorsabb modellszerverrel, kisebb modellel vagy a kontextus hosszának korlátozásával.

Biztonság. Az LLM-rendszerek sajátos biztonsági fenyegetésekkel szembesülnek: prompt injekció, adatkiszivárgás, túlzott ágens-jogosultság, modellmérgezés, ellátási lánc támadás. A prompt injekció lényege, hogy a támadó olyan utasítást rejt a bemenetbe, amely felülírja a rendszerutasítást, és például titkos adatok kiadására vagy nem engedélyezett eszköz hívására készteti a modellt. A védekezés rétegzett: a bemeneti adatok validálása, a rendszerprompt és a felhasználói prompt szigorú elválasztása, a kimenet szűrése, a veszélyes eszközök hozzáférésének korlátozása és az összes interakció auditálása. A zero trust modell itt is irányadó: a modellnek nem szabad vakon megbíznia a bemenetben, és minden eszközhívást identitás- és jogosultságkezelés alá kell vonni.

Értékelés. Az ML-rendszerek minőségét nem lehet egyszeri teszttel igazolni. Az értékelési pipeline folyamatosan méri a modell pontosságát, a RAG relevanciáját és hűségét, valamint az ágensek sikerességi arányát. Ehhez címkézett tesztadatkészletek, aranystandard válaszok, és gyakran LLM-alapú kiértékelő (LLM-as-a-judge) szükséges. Az értékelést automatikus pipeline-ba kell kötni, amely minden új modellverzió vagy promptváltozat esetén lefut. A minőségi küszöbök megszegése esetén a pipeline blokkolja a kiadást. Ez a fitness function-ök egy speciális esete, amelyet a korábbi fejezetekben tárgyaltunk.

31.1.7 Minőségi forgatókönyvek és ADR-ek az AI/ML rendszerekben

Az alábbiakban egy példát mutatunk arra, hogyan lehet a fenti szempontokat quality attribute scenario formában és ADR-ben rögzíteni.

Minőségi forgatókönyv – inferencia késleltetés.
- **Inger:** 50 párhuzamos felhasználói kérés érkezik a chat-kiszolgáló REST API-jára, mindegyik 200 tokennyi bemenettel.
- **Környezet:** normál üzem, egyetlen Kubernetes-névtér, két GPU-csomópont.
- **Válasz:** a rendszer minden kérést 10 másodpercen belül, streaming üzemmódban kiszolgál; a TTFT p95 700 ms alatt marad.
- **Mérőszám:** TTFT p95 ms-ban, válaszidő p95 másodpercben, 5 perces mérési ablakban.

Minőségi forgatókönyv – RAG hűség.
- **Inger:** 100 tesztkérdés a belső tudásbázis dokumentumaiból.
- **Környezet:** kiértékelő pipeline, RAG vektoradatbázis feltöltve az aktuális dokumentumokkal.
- **Válasz:** a generált válaszok legalább 95%-a nem tartalmaz a kontextusnak ellentmondó állítást.
- **Mérőszám:** hallucinációs ráta %-ban, a kiértékelő LLM ítélete alapján, a teljes kiértékelő futtatás időtartama alatt.

ADR példa – RAG használata finomhangolás helyett.
- **Kontextus:** a vállalati tudásbázis dokumentumai hetente változnak, a modell finomhangolása drága és lassú.
- **Döntés:** a rendszer RAG-ot használ, vektoradatbázissal és napi indexeléssel. A generáló LLM általános célú, nem finomhangolt.
- **Alternatívák:** finomhangolás minden dokumentumváltozás után; kizárólag a modell kontextusába töltött dokumentumok.
- **Következmények:** gyorsabb adaptáció, alacsonyabb tanítási költség; ugyanakkor növekvő komplexitás a visszakereső pipeline-ban, és a minőség a visszakeresés pontosságától függ.
- **Állapot:** elfogadva, 2024. november 15-én.

31.1.8 Labor: LLM-alapú rendszer architektúravázlata

A labor célja, hogy a hallgató egy kitalált, de reális üzleti problémára LLM-alapú rendszer architektúravázlatot készítsen. A hangsúly nem a modell betanításán van, hanem a rendszerarchitektúra és a döntések dokumentálásán.

Kiinduló állapot: a hallgató választ egy üzleti problémát, például ügyfélszolgálati chatbot belső dokumentumok alapján, számla-összegző asszisztens, vagy technikai támogató ágens. A választott probléma legyen eléggé konkrét ahhoz, hogy követelményeket lehessen hozzá rendelni.

Elvárt eredmény: egy reprodukálható dokumentum vagy jegyzetfüzet, amely a következőket tartalmazza:

1. **Rendszerkontextus és C4 diagram:** context szinten a felhasználó, a külső rendszerek (például dokumentumtár, e-mail, CRM) és a rendszerhatár. A diagram diagrams-as-code formában, például Mermaid vagy PlantUML szintaxisban készüljön, és a verzióval együtt legyen feltüntetve.
2. **Komponensszintű ábra:** az adatpipeline, a modellregiszter, az inferencia szolgáltatás, a RAG komponensek (retriever, embedding, vektoradatbázis, generáló) és az alkalmazásintegrációs réteg felelősséggel és interfészekkel.
3. **Minőségi forgatókönyvek:** legalább három, a fent bemutatott formátumban: késleltetés, hűség/relevancia és egy biztonsági attribútum (például prompt injekció elleni védelem).
4. **Két ADR:** az egyik a RAG és a finomhangolás közötti választásra, a másik az ágens működésű (vagy nem ágens) döntésre vonatkozik. Az ADR-ek kövessék a korábban bemutatott formátumot.
5. **Költségbecslés:** a várható kérésforgalom, az átlagos tokenfogyasztás és az aktuális modellárazás alapján (a pontos árakat a labor időpontjában érvényes nyilvános árjegyzékből kell venni; a feladatban elegendő egy becsült árat használni, de a forrást és a dátumot rögzíteni kell).
6. **Ellenőrzési szempontok:** milyen mérőszámokkal ellenőrizné a labor kiértékelője, hogy az architektúra megfelel-e a céloknak? Például a RAG visszakeresés recall@5 értéke, a hallucinációs ráta, a válaszidő p95.
7. **A mért és a várt eredmény elkülönítése:** ha a hallgató a labor során ténylegesen futtat egy PoC-t (például egy kis RAG-pipeline-t nyílt forrású eszközökkel), akkor a mért értékeket külön szakaszban, a várt célértékektől egyértelműen elválasztva kell feltüntetni. A nem futtatott mérés nem jelölhető ellenőrzött eredményként.

Visszaállítási út: mivel a laborban éles adatot nem használunk, és a futtatott PoC izolált környezetben történik, a visszaállítás a létrehozott erőforrások (például ideiglenes vektoradatbázis, API-kulcsok) törlését jelenti. A feladatleírásnak tartalmaznia kell a kitakarítás lépéseit.

31.1.9 Ellenőrzőlista a fejezethez

☐ Meg tudom magyarázni a modellek, az adatpipeline és az alkalmazásintegráció három rétegének felelősségét.

☐ Felismerem a train–serve skew jelenségét, és tudom, hogyan előzhető meg feature store-ral vagy közös pipeline-logikával.

☐ Ismerem a tanítás, az inferencia, a RAG és az ágens alapú rendszerek architekturális komponenseit.

☐ Tudom, milyen minőségi forgatókönyveket kell definiálnom egy LLM-alapú rendszerhez a késleltetésre, a hűségre és a biztonságra.

☐ Képes vagyok ADR-t írni a RAG és a finomhangolás közötti döntésről, és meg tudom indokolni a választást.

☐ Tisztában vagyok azzal, hogy az LLM-ek költsége és késleltetése szoros kapcsolatban áll a tokenfogyasztással és a modell méretével, és tudom, hogyan lehet ezt becsülni.

31.1.10 Önellenőrző kérdés

Tegyük fel, hogy egy belső tudásbázis-alapú asszisztenst kell tervezni, ahol a dokumentumok naponta változnak, a válaszoknak mindig a legfrissebb tartalmat kell tükrözniük, és a felhasználók elvárják a 2 másodperc alatti első token megjelenést. Milyen architekturális döntéseket hozna a modell, a pipeline és az integráció szintjén? Indokolja meg, miért a RAG mellett döntene a finomhangolás helyett, és milyen minőségi forgatókönyvekkel biztosítaná a válaszok frissességét és hűségét?

A következő fejezet (Edge, fog computing és IoT/CPS) az edge, fog computing és IoT/CPS architektúrákra épít, ahol az itt bevezetett LLM-inferencia és RAG minták peremhálózati környezetben való alkalmazása is szóba kerül.

31.2 Edge, fog computing és IoT/CPS

A hagyományos felhőparadigma hosszú éveken át az volt, hogy az adatok a perifériáról – böngészőkből, mobilalkalmazásokból, szenzorokból – egy központi adatközpontba áramlanak, ahol a feldolgozás, tárolás és döntéshozatal történik. Ez a modell ma is alapvető fontosságú, de a végponti eszközök számának és adatmennyiségének robbanásszerű növekedése, valamint a valós idejű beavatkozást igénylő alkalmazások megjelenése felvetette a kérdést: vajon minden adatnak és döntésnek a felhőben kell-e megszületnie? A válasz egyre gyakrabban nem. A felhőtől a peremig terjedő feldolgozás – angolul *edge-to-cloud continuum* – azt a szemléletet jelenti, hogy a számítási, tárolási és döntési képességet a hálózat több rétegében helyezzük el, a fizikai környezethez legközelebb eső eszközöktől a globális felhőig. Ebben a fejezetben a perem- (edge) és köd- (fog) számítás fogalmait, azok architekturális következményeit, valamint a cyber-physical rendszerek (CPS) megbízhatóságát vizsgáljuk. A hangsúlyt a gyakorlati döntéshozatalra és a tipikus hibák elkerülésére helyezzük.

31.2.1 A felhőtől a peremig terjedő feldolgozás fogalma

Az *edge computing* kifejezés a számítási és tárolási erőforrások olyan elhelyezését jelöli, amely a hálózati peremen, az adatforrások közvetlen közelében történik. A perem lehet egy gyáregység, egy jármű, egy okos otthon, egy viselhető eszköz vagy bármilyen környezet, ahol az adatok keletkeznek. A *fog computing* – a köd képével, amely a föld és a felhő között lebeg – egy köztes réteg, amely a perem és a felhő közötti aggregációt, koordinációt és szolgáltatásokat biztosít. A két fogalom közötti határ nem éles; sokan a fog computingot az edge computing kiterjesztett, hierarchikus megvalósításának tekintik. A lényeg a feldolgozás rétegzettsége: az adatok egy része helyben feldolgozható, más része a fog rétegbe, megint más része a felhőbe kerül, attól függően, hogy milyen késleltetési, sávszélességi, adatvédelmi és analitikai követelmények vonatkoznak rá.

A feldolgozási kontinuum négy jellegzetes szintje:

1. **Eszközök és szenzorok** – a fizikai világban adatot gyűjtenek, és gyakran korlátozott számítási kapacitással rendelkeznek. Ide tartoznak a hőmérséklet-érzékelők, gyorsulásmérők, kamerák, RFID-olvasók, PLC-k és más ipari eszközök.
2. **Edge csomópontok** – közvetlenül a szenzorok közelében elhelyezkedő, nagyobb számítási kapacitású eszközök, mint például egy ipari PC, egy gateway vagy egy beágyazott szerver. Feladatuk az adatok szűrése, normalizálása, gyors helyi döntések meghozatala, valamint a felhő felé irányuló adatforgalom csökkentése.

3. **Fog réteg** – több edge csomópont fölé helyezett, regionális szintű feldolgozási és aggregációs réteg. Gyakran egy telephelyi szerverterem vagy egy helyi mikro-adatközpont formájában valósul meg. Felelőssége az időben kevésbé kritikus, de nagyobb kontextust igénylő elemzések futtatása, a helyi adatok kötegelt továbbítása, valamint a központi felhővel való szinkronizáció.
4. **Felhő réteg** – a globális, skálázható háttér, amely hosszú távú tárolást, nagy adathalmazokon futó gépi tanulási modellek tanítását, több telephely aggregált elemzését és központi irányítást biztosít.

Ezt a rétegződést mutatja be az alábbi, diagrams-as-code formában készült kontextusszintű diagram (Mermaid):

```
flowchart LR
    subgraph "Eszközök és edge csomópontok"
        S1[Szenzor 1] -->|MQTT| GW[Edge gateway]
        S2[Szenzor 2] -->|MQTT| GW
        CAM[Kamera] -->|RTSP| GW
    end
    subgraph "Fog réteg"
        GW -->|AMQP / MQTT| FOG[Fog node]
    end
    subgraph "Felhő réteg"
        FOG -->|HTTPS / gRPC| CLOUD[Cloud backend]
        CLOUD --> DB[(Adattó)]
    end
```

Az ábrán a kapcsolatokon jelzett protokollok (MQTT, RTSP, AMQP, HTTPS, gRPC) az adott rétegek közötti üzenetküldés jellegét mutatják; a gyakorlatban a választás a szinkronitási igénytől, a hibatűréstől és az infrastruktúra képességeitől függ. A rendszerhatárt a diagramon a három alréteg határolja, a külső rendszereket és felhasználói szerepköröket a C4 context szintjén kell tovább részletezni, ha a rendszer konkrét üzembe helyezéséről van szó.

31.2.2 Késleltetés, sávszélesség és adatvédelem mint döntési tényezők

Három olyan erő van, amely a feldolgozást a perem felé tolja: a késleltetés, a sávszélesség és az adatvédelem. Mindhárom önmagában is elegendő lehet ahhoz, hogy az architektúra ne a tiszta felhős modellt válassza, és a három együttesen gyakran egyértelművé teszi a hibrid megoldás szükségességét.

31.2.2.1 Késleltetés

A késleltetés a fizikai távolságból és a hálózati átvitel időből fakad. A fénysebesség is véges: egy 1000 kilométeres távolságon az optikai kábelen haladó jel terjedési ideje nagyjából 5 ms, de a valóságban a routerek, switchek és a sorban állás miatt a WAN-késleltetés ennek többszöröse. Amikor egy gyártósori kamerának egy tizedmásodpercen belül kell észlelnie egy hibás terméket és le kell állítania a futószalagot, a felhőbe küldött képkocka és a visszaérkező parancs késleltetése elfogadhatatlan lehet. Az ilyen döntéseket a peremen, az adatforrás mellett kell meghozni.

A minőségi követelményeket a már bevezetett *quality attribute scenario* formában érdemes megfogalmazni. Például:

- **Inger:** egy gyártósori kamera képe hibát észlel.
- **Környezet:** normál gyártási terhelés, a hálózat egy másik folyamattal is terhelt.
- **Válasz:** a rendszer a hibás terméket jelöli, és a szalagot 100 ms-on belül leállítja.

- **Mérőszám:** a hiba észlelésétől a beavatkozási parancs kiadásáig eltelt idő; a mérést 1 órás ablakban, 95%-os percentilisben kell megadni.

Ez a forgatókönyv szinte lehetetlen teljesíteni, ha a döntési hurok a felhőn keresztül vezet, de reálisan elérhető egy helyi edge csomóponttal.

31.2.2.2 Sávszélesség

A perem eszközök hatalmas adatmennyiséget termelhetnek. Egyetlen ipari kamera 1080p felbontásban, 30 képkocka/másodperc sebességgel másodpercenként több megabájtot állít elő. Ha ezer ilyen kamera folyamatosan a felhőbe streamelné a nyers videót, a hálózati sávszélesség és a felhőbeli tárhely igénye csillagászati lenne. A megoldás az adatcsökkentés a peremen: helyi szűrés, mintavételezés, eseményvezérelt továbbítás, kompresszió, kivonatolás, vagy éppen helyi gépi tanulási inferencia, amely csak a releváns eredményt – például egy metadat-csomagot – küldi fel a fog vagy felhő rétegbe.

A sávszélesség korlátai nem csak a nyers adatátvitelnél jelentkeznek. Egy távoli olajfúró platform vagy egy hajó műholdas kapcsolata alacsony sávszélességű és magas késleltetésű lehet. Ilyen környezetben az edge rétegnek szinte autonóm módon kell működnie, és a felhő felé csak aggregált, tömörített, időnként továbbított összefoglalókat küldeni.

31.2.2.3 Adatvédelem

A harmadik erő az adatvédelem. A személyes adatok – például egy okos otthon kamerája által rögzített videó, egy viselhető egészségügyi eszköz által mért élettani paraméterek, vagy egy üzlet vásárlói mozgását követő hőtérkép – feldolgozása a peremen csökkenti annak kockázatát, hogy a nyers, azonosítható adatok elhagyják az adott fizikai környezetet. A GDPR és más adatvédelmi szabályozások szerint az adatminimalizálás és a célhoz kötöttség elveit már a rendszer tervezésekor érvényesíteni kell. A peremen végzett helyi feldolgozás lehetőséget ad arra, hogy a rendszer csak anonimizált, aggregált vagy statisztikai jellemzőket továbbítson, míg a nyers adatok helyben maradnak és rövid élettartamúak.

Ugyanakkor az adatvédelem nem azt jelenti, hogy a perem eszközök automatikusan biztonságosak. A peremen tárolt adatokat ugyanúgy védeni kell: titkosítás, hozzáférés-vezérlés, firmware-frissítések és fizikai védelem szükséges. Az adatvédelem és a biztonság szempontjait a tervezés során párhuzamosan kell kezelni.

31.2.3 Cyber-physical rendszerek és megbízhatóság

A *cyber-physical system* (CPS) olyan rendszer, amelyben a számítási, hálózati és fizikai folyamatok szorosan összefonódnak. A CPS-ek a szenzorokon keresztül érzékelik a fizikai világot, számítási folyamatokkal döntéseket hoznak, majd beavatkozókon – aktuátorokon – keresztül hatnak vissza a fizikai környezetre. Példák: ipari automatizálási rendszerek, autonóm járművek, intelligens épületek, orvosi eszközök, energiahálózatok, robotok.

A CPS-ek tervezésekor a megbízhatóság elsődleges szempont. Míg egy webes alkalmazásnál egy képernyő lassú betöltése bosszantó, egy autonóm jármű fékrendszerének késleltetése életveszélyes lehet. A megbízhatósági követelményeket ezért hard real-time vagy soft real-time osztályokba szokás sorolni:

- **Hard real-time:** a határidő túllépése elfogadhatatlan, mert katasztrofális következményekkel jár. Példa: egy légzsák vezérlője a becsapódás érzékelésétől számított néhány milliszekundumon belül ki kell nyissa a légzsákot.
- **Soft real-time:** a határidő túllépése nem okoz katasztrófát, de a szolgáltatás minősége romlik. Példa: egy videókonferencia képkockáinak késése rontja a felhasználói élményt, de nem veszélyezteti a rendszer biztonságát.

A CPS-ekben a vezérlési hurok általában a következő lépésekből áll: érzékelés → feldolgozás → döntés → beavatkozás → visszacsatolás. Az edge computing éppen azért játszik kulcsszerepet, mert a vezérlési hurkot a fizikai folyamat közelében tartja, csökkentve a késleltetést és növelve a determinizmust. A felhő ebben a kontextusban kiegészítő szerepet játszik: modelltanítás, hosszú távú telemetria, több telephely összehasonlítása, szoftverfrissítések terjesztése.

A megbízhatóság növelésére a CPS-ekben több architekturális minta is használatos:

- **Redundancia:** a kritikus érzékelők és beavatkozók megkettőzése, szavazó logika alkalmazása.
- **Watchdog és heartbeat:** a perem csomópont rendszeres önellenőrzése; ha a feladat leáll, a watchdog újraindítja.
- **Fail-safe állapot:** ha a rendszer bizonytalan állapotba kerül, akkor egy előre meghatározott biztonságos állapotba lép (például a szalag leáll, a fék behúz).
- **Bulkhead és circuit breaker:** bár ezeket a mintákat a korábbi fejezetben a szolgáltatások közötti kommunikációnál vezettük be, az edge kontextusban is értelmezhetők: az egyik szenzorcsoport hibája ne terjedjen át a másikra, és a felhővel való kapcsolat megszakadása esetén a helyi rendszer ne próbáljon a végtelenségig várni, hanem kapcsoljon offline módba.

A cyber-physical rendszerek megbízhatóságának mérésénél a rendelkezésre állás és az átlagos hibamentes idő mellett fontos a *biztonságos működés* valószínűsége is: hány óra alatt hány veszélyes meghibásodás várható? Ez a kérdés az ipari szabványok (például az IEC 61508, ISO 26262) területére vezet, de az architektúra szintjén azt jelenti, hogy a hibamódok elemzése – failure mode analysis – nem hagyható el. A fejezetünkben a korábban megismert hibamód-elemzési technikákat alkalmazzuk a perem-felhő architektúrára.

31.2.4 Architekturális minták az edge-felhő kontinuumban

A perem-felhő rendszerek tervezésekor több visszatérő minta alkalmazható. Az alábbiakban a leggyakoribbakat mutatjuk be, mindegyiket a kontextus, a követelmények és az alternatívák megjelölésével.

31.2.4.1 1. Adatszűrés és aggregáció a peremen

A legegyszerűbb minta, amikor az edge gateway már a forrásnál csökkenti a felhő felé irányuló adatforgalmat. Például egy hőmérséklet-szenzor másodpercenként mér, de a gateway csak a percenkénti átlagot, minimumot és maximumot továbbítja a felhőbe. Ez csökkenti a sávszélességet, a felhőbeli tárolási költséget és a hálózati kitettséget. A minta akkor előnyös, ha a felhőben nincs szükség nyers, nagy felbontású adatokra, és a döntések lokálisan is meghozhatók.

31.2.4.2 2. Helyi inferencia és modellfrissítés

A gépi tanulási modellek futtatása a peremen lehetővé teszi a valós idejű osztályozást, anomáliadetektálást vagy prediktív karbantartást anélkül, hogy minden egyes adatpont a felhőbe utazna. A modellt a felhőben tanítják be nagy történeti adathalmazon, majd tömörített formátumban (például TensorFlow Lite, ONNX) telepítik az edge csomópontokra. A perem időnként visszaküldi a nehezen osztályozott eseteket – ezt hívják *active learning* visszacsatolásnak –, és a felhő időnként új modellverziót tesz közzé. A kihívás a modellverzió-követés és a visszagörgetés: az edge eszközökön futó modellverziókat ugyanúgy verziózni kell, mint a mikroszolgáltatások konténerimage-eit.

31.2.4.3 3. Offline-first és helyi állapot

Az edge csomópontok gyakran időszakosan elveszítik a kapcsolatot a felhővel – mobil hálózati lefedettség, műholdas kapcsolat vagy karbantartás miatt. Az architektúrának ezért támogatnia kell az offline működést: a helyi döntések továbbra is megszületnek, az adatok helyi puffertárba kerülnek, és a kapcsolat helyreállása után szinkronizálódnak. A konfliktuskezelés itt az *eventual consistency* elvén alapulhat; a tervezésnél meg kell határozni, hogy a helyi és a felhőbeli állapot közötti ütközést hogyan oldjuk fel – például időbélyeg-alapú utolsó írás nyer, vagy a felhő a forrás.

31.2.4.4 4. Fog aggregáció és hierarchikus feldolgozás

Amikor egy telephelyen több száz vagy ezer edge csomópont működik, egy köztes fog réteg bevezetése csökkenti a közvetlen felhő kapcsolatok számát, és lehetőséget ad a helyi, több csomópontot átfogó elemzésekre. A fog node aggregálja az adatokat, normalizálja a protokollokat, biztosítja a helyi adatbázis-replikációt, és csak a szükséges, tömörített összefoglalókat küldi a felhőbe. Így a felhő nem egyszerre ezer végponttal kommunikál, hanem néhány fog csomóponttal, ami csökkenti a kapcsolatok számát és a felhő terhelését.

31.2.4.5 5. Digital twin és szimuláció

A CPS-ekben gyakran készül digitális iker – a fizikai rendszer virtuális másolata –, amely a felhőben fut, és folyamatosan frissül a peremről érkező adatokkal. Ez lehetővé teszi a „mi lenne, ha” elemzéseket, a prediktív karbantartást és a rendszer optimalizálását anélkül, hogy a fizikai rendszert közvetlenül terhelnénk. Az edge és a fog réteg felelős az adatok minőségi gyűjtéséért és továbbításáért; a digitális iker a felhőben él.

31.2.5 Tipikus hibák és elkerülésük

Az edge-felhő architektúrák tervezésekor számos hiba fordul elő újra és újra. Ezek közül a legfontosabbakat gyűjtöttük össze, mindegyikhez indoklással és helyes megoldási iránnyal.

31.2.5.1 1. Minden számítás a felhőbe

A legegyszerűbb hiba, hogy a csapat a megszokott felhős fejlesztési modellt alkalmazza, és minden adatot a felhőbe küld feldolgozásra. Ennek következménye a megnövekedett késleltetés, a sávszélesség pazarlása, az adatvédelmi kockázatok és a magas felhőbeli költség. A helyes megoldás a feldolgozás elosztása: a valós idejű, helyi kontextust igénylő feladatokat a peremen, a hosszú távú elemzéseket a felhőben végezzük.

31.2.5.2 2. Minden adatot a peremen feldolgozni

A másik véglet: a csapat annyira elkötelezi magát a perem mellett, hogy a felhőbe csak minimális, aggregált adatokat küld, és elveszíti a központi elemzés, a modelltanítás és a globális betekintés lehetőségét. A peremen futó modellek idővel elavulnak, mert nincs elegendő visszacsatolás a tanításhoz. A helyes arány a követelményektől függ: biztosítani kell, hogy a felhő megkapja a tanításhoz és auditáláshoz szükséges, megfelelően anonimizált adatokat.

31.2.5.3 3. Folyamatos hálózati kapcsolat feltételezése

Egyes tervezők úgy kezelik az edge csomópontokat, mintha azok állandó, stabil kapcsolatban lennének a felhővel. A valóságban a kapcsolat megszakadhat, a sávszélesség ingadozhat, a csomagvesztés jelentős lehet. Az architektúrának fel kell készülnie az offline működésre, pufferelésre és a kapcsolat helyreállása utáni szinkronizációra. Ennek hiányában az edge rendszer leáll, vagy adatot veszít.

31.2.5.4 4. Biztonsági lazaság a peremen

A perem eszközök gyakran fizikailag is elérhetők, ezért a támadó közvetlen hozzáférést szerezhet a hardverhez. A gyenge hitelesítés, a nem frissített firmware, a titkosítatlan kommunikáció mind komoly kockázatot jelent. A biztonsági architektúrának a perem eszközökre is ki kell terjednie: titkosítás, kölcsönös TLS, hardveres biztonsági modul (HSM), rendszeres biztonsági frissítések és távoli hitelesítési mechanizmusok.

31.2.5.5 5. Monolitikus edge alkalmazások

A peremre telepített alkalmazás gyakran egyetlen nagy, nehezen frissíthető bináris. Ez megnehezíti a hibajavítások és új funkciók kiadását, különösen akkor, ha sok edge eszköz van. A jobb megoldás a moduláris konténerek vagy csomagok használata, amelyeket a felhőből lehet frissíteni, és a rollback is egyszerű.

31.2.5.6 6. Az adatok életciklusának figyelmen kívül hagyása

A peremen keletkező adatok hatalmas mennyiséget érhetnek el, és ha nincs meghatározott adatmegőrzési politika, a helyi tároló hamar megtelik, ami adatvesztéshez vagy a rendszer leállásához vezet. Meg kell határozni, hogy mely adatokat meddig kell megőrizni, mi kerül a felhőbe, és mi törölhető automatikusan.

31.2.5.7 7. A megfigyelhetőség hiánya

Sok edge rendszer „fekete dobozként” működik: nincs megfelelő log, metrika és trace a perem eszközökből. Amikor hiba történik, a csapat nem látja, mi zajlik a terepen. Az edge rétegben is be kell vezetni az observability eszközöket – OpenTelemetry, helyi loggyűjtő, metrika-exportáló –, de figyelemmel kell lenni arra, hogy a gyenge hálózat miatt az adatokat pufferként kell kezelni, és a felhőbe időnként, tömörítve továbbítani.

31.2.5.8 8. A méretezés és a költség alábecsülése

Amikor az edge eszközök száma tízezres nagyságrendbe nő, a felügyelet, a frissítés és a hibaelhárítás költsége exponenciálisan növekszik, ha az architektúra nem automatizált. A felhőben futó háttérnek is fel kell készülnie a nagyszámú bejövő kapcsolatra, és a felhőbeli szolgáltatások költségét a tényleges adatforgalom és tárolás alapján kell becsülni.

31.2.6 Labor: edge- és felhőarchitektúra összehasonlítása

A labor célja, hogy egy konkrét, de egyszerű példán keresztül alkalmazza a fejezetben tárgyalt fogalmakat, és döntési naplóval, valamint mérési tervvel dokumentálja az eredményt. A labor nem igényel éles környezetet, számítási és szimulációs feladatokból áll.

31.2.6.1 Kiinduló állapot

Egy közepes méretű gyártóüzem 500 darab rezgésérzékelőt telepít a gépeire, hogy prediktív karbantartást valósítson meg. Minden érzékelő másodpercenként 1 kB méretű mérési csomagot küld (időbélyeg, gyorsulás három tengelyen, hőmérséklet). A gyár és a felhő közötti internetkapcsolat sávszélessége 10 Mbit/s, a késleltetés 60 ms egy irányban. Az adatvédelmi szabályzat szerint a nyers mérési adatok nem hagyhatják el a gyár területét, mert azok a gyártási folyamatok részleteit fedhetik fel. A rendszer célja: anomáliadetektálás, amely 200 ms-on belül riasztást ad, ha egy gép rezgésmintája meghibásodásra utal.

Két architektúra-jelöltet kell összehasonlítani:

1. **Teljes felhős megoldás:** az érzékelők közvetlenül a felhőbe küldik a nyers adatokat, ahol az anomáliadetektáló modell fut.
2. **Edge-felhő hibrid megoldás:** az érzékelők egy helyi edge gateway-hez küldik az adatokat; a gateway futtatja a detektáló modellt, helyben riaszt, és csak aggregált statisztikákat (percenkénti átlag, szórás) továbbít a felhőbe.

31.2.6.2 Elvárt eredmény

A labor eredménye egy rövid dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza:

- a kiinduló kontextus rögzítését (eszközszám, adatsebesség, hálózati paraméterek, követelmények),
- a két architektúra összehasonlítását a késleltetés, sávszélesség, adatvédelem és költség dimenziókban,
- egy ADR-t, amely rögzíti a döntést és annak okát,
- a várt eredményeket (számításokkal) és a mért eredményeket (ha van szimuláció),

- ellenőrzési szempontokat a megvalósítás utánra.

31.2.6.3 Lépések

1. **Számítsa ki a szükséges sávszélességet mindkét esetben.**
   - Minden érzékelő 1 kB/csomag, 1 csomag/s → 1 kB/s érzékelőnként.
   - 500 érzékelő esetén a nyers adatfolyam 500 kB/s = 4 Mbit/s (hasznos adat) plusz protokoll overhead. A 10 Mbit/s-os sávszélesség ezt elvileg elbírná, de a valós átvitelhez hozzá kell adni a TCP/IP és az alkalmazásprotokoll fejlécét, valamint az újraküldéseket. Becslés: a hatékony sávszélesség-igény 6–8 Mbit/s lehet.
   - A hibrid megoldásnál a gateway percenkénti aggregált statisztikákat küld: 500 érzékelő × 1 csomag/perc = 500 csomag/perc 8,3 csomag/s. Ha egy aggregált csomag 100 bájt, az 830 B/s 6,64 kbit/s, ami elhanyagolható a 10 Mbit/s-hoz képest.
2. **Becsülje meg a késleltetést mindkét esetben.**
   - Teljes felhős: a szenzor → internet → felhő késleltetés legalább 60 ms (egy irány), plusz a feldolgozás és a visszaút. A riasztás késleltetése legalább 120–150 ms, de a valóságban a jitter és a sorban állás miatt ennél több is lehet. Ez még a 200 ms-os követelményen belül van, de a hálózati fluktuáció kockázatot jelent.
   - Edge-felhő: a szenzor → gateway közötti helyi hálózat késleltetése jellemzően 1–5 ms, a detektálás a gateway-en néhány ms. A riasztás késleltetése 10–20 ms alatt tartható, nagy biztonsággal a 200 ms alatt.
3. **Értékelje az adatvédelmet.**
   - Teljes felhős: a nyers rezgésadatok elhagyják a gyár területét, ami sérti az adatvédelmi szabályzatot. Ez a megoldás tehát már a követelmény szintjén kizárt, hacsak nincs álnevesítés vagy titkosítás, ami viszont bonyolítja az elemzést a felhőben.
   - Edge-felhő: a nyers adatok a gateway-en belül maradnak, csak aggregált statisztikák mennek a felhőbe. Ez megfelel az adatvédelmi szabályzatnak.
4. **Hozzon döntést ADR formában.**
   - A döntés: edge-felhő hibrid architektúra. Kontextus: adatvédelmi megkötés, valós idejű riasztási igény, korlátozott sávszélesség. Alternatívák: tiszta felhős, tiszta edge (csak helyi, felhő nélkül). Következmények: szükség van edge gatewayek üzemeltetésére, frissítésére, megfigyelésére; a felhőben csak aggregált adatok lesznek, így a tanítás korlátozott, de a követelményeket teljesíti. Állapot: elfogadva.
5. **Határozza meg az ellenőrzési szempontokat.**
   - Riasztási késleltetés mérése 1 órás ablakban, 95%-os percentilisben.
   - Az edge gateway rendelkezésre állása.
   - A felhőbe továbbított adatmennyiség (napi szinten).
   - Az adatvédelmi szabályzat betartásának auditja: a nyers adatok valóban nem hagyják el a gyárat.

31.2.6.4 A mért és a várt eredmények elkülönítése

A fenti számítások *várt* eredmények, mert becslésen alapulnak. A labor következő lépéseként egy szimulációt vagy egy kis tesztkörnyezetet lehet felállítani, amelyben a tényleges késleltetést, a sávszélességet és az adatvesztést *mérjük*. A labor dokumentációban a várt és a mért értékeket külön kell feltüntetni, például:

- Várt: a hibrid megoldás riasztási késleltetése < 20 ms.
- Mért (szimulált környezetben): 15 ms $\pm$ 3 ms, 1000 minta alapján, szimulált tisztakezes hálózaton.

A visszaállítási út ebben a laborban triviális, mivel nem éles környezetben dolgozunk; ha a laborban bármilyen helyi konténert vagy virtuális gépet indítunk, azokat a labor végén le kell állítani, és a tesztadatokat törölni kell. A dokumentumban a konfigurációt is rögzíteni kell, hogy a mérés reprodukálható legyen.

31.2.7 Ellenőrzőlista az edge-felhő architektúrához

A fejezet zárásaként tekintsük át azt a minimális ellenőrzőlistát, amelyet egy edge-felhő architektúra tervezésekor és áttekintésekor érdemes végigjárni:

- ☐ Definiálta-e a feldolgozási rétegeket (eszköz, edge, fog, felhő), és mindegyikhez hozzárendelte-e a felelősségeket?
- ☐ Rögzítette-e a késleltetési, sávszélességi és adatvédelmi követelményeket quality attribute scenario formában?
- ☐ Megvizsgálta-e, hogy mely döntések igényelnek valós idejű, helyi feldolgozást, és melyek futhatnak a felhőben?
- ☐ Tervezett-e offline működést és adatpufferelést a hálózati kapcsolat megszakadására?
- ☐ Biztosította-e az edge eszközök biztonságát (titkosítás, hozzáférés-vezérlés, firmware-frissítés)?
- ☐ Meghatározta-e az adatok életciklusát a peremen és a felhőben?
- ☐ Bevezetett-e megfigyelhetőséget a perem rétegben is, figyelembe véve a korlátozott sávszélességet?
- ☐ Dokumentálta-e a döntéseket ADR-ekben, és rögzítette-e a trade-offokat?

31.2.8 Önellenőrző kérdés

Egy intelligens épületfelügyeleti rendszerben több ezer jelenlét- és hőmérséklet-érzékelő működik. A rendszernek 150 ms-on belül kell vezérelnie a fűtést és a világítást az aktuális jelenlét alapján, miközben az adatvédelmi előírások szerint a nyers jelenléti adatok nem hagyhatják el az épületet. Ezen kívül az épületüzemeltető havi szintű energiahatékonysági jelentést szeretne a felhőben. Hogyan osztaná el a feldolgozási feladatokat a perem, a fog és a felhő réteg között? Milyen adatokat küldene tovább, és milyen formában? Válaszát indokolja a késleltetés, az adatvédelem és a sávszélesség szempontjai alapján.

Ez a kérdés a fejezet központi tanulságát ellenőrzi: a feldolgozást a követelményekhez kell igazítani, nem egyetlen rétegbe zsúfolni. A helyes válaszban a valós idejű vezérlés az edge rétegbe kerül, az épületen belüli aggregáció a fog rétegbe, a havi jelentéshez szükséges anonimizált statisztikák pedig a felhőbe.

31.2.9 Továbbvezető gondolatok

Az edge-felhő kontinuum koncepciója közvetlenül kapcsolódik a következő fejezetekben tárgyalt modern trendekhez. A blockchain és a decentralizált rendszerek esetében a perem csomópontok gyakran a konszenzus résztvevői; a VR/AR rendszerek esetében pedig a késleltetés minimalizálása érdekében a renderelés egy része a peremre vagy a közeli fog rétegbe kerülhet. Ezeket a kapcsolódásokat a Blockchain, decentralizált rendszerek és VR/AR fejezetben vizsgáljuk. Addig

is érdemes megjegyezni, hogy az edge computing nem technológiai divat, hanem a fizikai törvényekből – a fénysebesség korlátjából, az adatmennyiségből és a szabályozói környezetből – fakadó architekturális szükségszerűség.

31.3 Blockchain, decentralizált rendszerek és VR/AR

A blockchain és a kiterjesztett/virtuális valóság (AR/VR) látszólag távoli területek, mégis mindkettő súlyos architekturális kérdéseket vet fel a bizalom, a késleltetés, az állapotkezelés és az erőforráskorlátok terén. Ez a fejezet a decentralizált bizalom és az immerzív rendszerek architektúráját járja körül: megmutatja, mikor van értelme a blokklánc-alapú elosztott főkönyvnek, hogyan tervezzünk konzorciális láncot okosszerződésekkel, és milyen architekturális kényszereket ró a valós idejű renderelés az AR/VR rendszerekre. A hangsúly nem egy adott terméken vagy protokollon van, hanem a döntéseken, a komponensek felelősségén, a minőségi forgatókönyveken és a tipikus hibákon.

31.3.1 1. A decentralizált bizalom mint architekturális alapelv

A hagyományos elosztott rendszerekben a bizalmat általában egy központi szolgáltató vagy egy jól körülhatárolt szervezet garantálja: a bank hitelesíti az átutalást, a hitelesítésszolgáltató a tanúsítványt, az állami nyilvántartás a tulajdonjogot. Ezt a modellt *központosított bizalomnak* nevezzük. Alapvetően hatékony, hiszen egyetlen entitás felelős az igazságért, és a résztvevők kisebb infrastruktúrát igényelnek. Ugyanakkor a központi fél egyben szűk keresztmetszet és egyetlen hibapont: lehet megbízhatatlan, kényszeríthető, meghibásodhat, vagy egyszerűen csak magas tranzakciós díjat számít fel.

A *decentralizált bizalom* ennek egy alternatívája: a rendszer úgy éri el a közös állapot egyezőségét, hogy nincs egyetlen kitüntetett fél, amelyet feltétel nélkül megbízhatónak tekintenénk. A bizalom forrása nem egy entitás, hanem a protokoll maga: a konszenzusmechanizmus, a kriptográfiai ellenőrzés és a gazdasági ösztönzők együttese. A résztvevők akkor is együttműködhetnek, ha nem bíznak egymásban, mert a szabályok betartása ellenőrizhető, a megszegése pedig költséges vagy értelmetlen.

Architektúr szempontból ez radikális változást jelent. A rendszerhatáron belül megjelenik egy nyílt, osztott főkönyv, amelynek minden csomópontja saját másolatot tárol, és a tranzakciók érvényesítését nem egy központi komponens, hanem a hálózat egésze végzi. Az üzleti logika egy része az okosszerződésekbe kerül, amelyek determinisztikus kódot futtatnak a tranzakciók feldolgozásakor. Az ilyen rendszer másképp skálázódik, másképp hibázik, és egészen más üzemeltetési modellt igényel.

Fontos azonban hangsúlyozni: a decentralizált bizalom nem „jobb”, hanem *más*. Akkor érdemes választani, ha az üzleti probléma több, egymással versengő vagy egymásnak nem teljesen megbízó fél közötti közös állapotot igényel, és nincs elfogadható központi koordinátor. Ellenkező esetben a blokklánc bevezetése pusztán többletbonyolultságot, késleltetést és üzemeltetési költséget szül.

31.3.2 1.1. Mikor van értelme a blokkláncnak?

A döntéshez használható egy egyszerű kérdéssor:

1. **Több fél osztozik egy közös állapoton?** Ha csak egy szervezet írja és olvassa az adatokat, egy hagyományos adatbázis jobb választás.
2. **A felek nem bíznak meg teljesen egymásban, vagy nincs központi szereplő?** Ha van megbízható harmadik fél, a blokklánc felesleges rezsit jelent.
3. **Szükség van-e hamisíthatatlan, visszamenőleg nem módosítható auditnyomvonalra?** Ha igen, a főkönyv append-only jellege értékes.
4. **Az állapotváltozások ritkábbak, mint a hagyományos OLTP-terhelés?** A blokkláncok tranzakciós átbocsátóképessége jellemzően jóval alacsonyabb, mint egy relációs adatbázisé.
5. **Elfogadható a végső egyezőség és a magasabb késleltetés?** A konszenzus időt vesz igénybe.

Ha a válaszok többsége igen, érdemes tovább vizsgálni a blokklánc-alapú architektúrát. Ellenkező esetben a decentralizált bizalom inkább anti-minta.

31.3.3 2. Konzorciális lánc, okosszerződés és konszenzus

A nyilvános (permissionless) blokkláncok, mint a Bitcoin vagy az Ethereum, bárki számára nyitottak, a részvételt kizárólag a protokoll szabályai korlátozzák. A vállalati környezetben azonban gyakran **konzorciális (permissioned) láncokat** használnak: a csomópontok üzemeltetésére csak előre meghatározott, hitelesített szervezetek jogosultak. Ez nem jelenti a decentralizáció feladását, csupán a bizalmi kört szűkíti egy ismert halmazra, ami számottevő teljesítmény- és irányítási előnyökkel jár.

31.3.4 2.1. A konzorciális lánc komponensei

Egy tipikus konzorciális lánc architektúrája a következő komponensekből áll:

- **Csomópontok (validátorok)** – Minden konzorciumtag üzemeltet legalább egy csomópontot. A csomópontok tárolják a főkönyv másolatát, fogadják a tranzakciókat, és részt vesznek a konszenzusban. Felelősségük: a tranzakciók érvényesítése, a blokkok képzése és a főkönyv állapotának karbantartása. Interfészük: JSON-RPC vagy gRPC a kliensek felé, valamint peer-to-peer protokoll a többi csomópont felé.
- **Okosszerződések** – A láncra telepített, determinisztikus programok, amelyek az üzleti logikát valósítják meg. Felelősségük: az állapotváltozások szabályainak kikényszerítése. Interfészük: függvényhívások tranzakciók formájában.
- **Kliensek / alkalmazások** – A felhasználók vagy belső rendszerek által használt alkalmazások. Nem feltétlenül kell csomópontot futtatniuk; elegendő, ha egy elérhető csomóponthoz kapcsolódnak.
- **Tagsági szolgáltatás (MSP)** – Permissioned láncokban ez a komponens kezeli a résztvevők identitását, tanúsítványait és jogosultságait. Felelőssége: a résztvevők hitelesítése és a szerepkörök érvényesítése.
- **Rendező szolgáltatás (ordering service)** – A konzorcium által közösen üzemeltetett komponens, amely a tranzakciókat blokkokba rendezi és továbbítja a validátoroknak. Nem ő dönt a tranzakció érvényességéről, csak a sorrendet rögzíti.

Egy lehetséges C4 container szintű diagram szövegesen: a **Konzorciumi főkönyv** rendszerhatárán belül a **validátor csomópontok** párhuzamosan futnak, mindegyik kapcsolódik a **rendező szolgáltatáshoz** (szinkron, TLS), a **tagsági szolgáltatáshoz** (szinkron, TLS), és a kliensekhez (szinkron, TLS). A külső szereplők: **konzorciumtagok**, **alkalmazásfelhasználók** és **auditor**.

A kapcsolatokon fontos feltüntetni a protokollt, a szinkronitást és a hibatűrést: a csomópontok közötti kommunikáció aszinkron, üzenetvesztésre számítva újraküldéssel; a kliens–csomópont hívás szinkron kérés-válasz, időtúllépéssel és újrapróbálkozással.

31.3.5 2.2. Konszenzusmechanizmusok

A konszenzus feladata, hogy a résztvevő csomópontok megegyezzenek a tranzakciók sorrendjében és a főkönyv állapotában. A választott mechanizmus alapvetően meghatározza a rendszer minőségi attribútumait: a teljesítményt, a véglegességet, a hibatűrést és a biztonsági garanciákat.

- **Proof of Work (PoW)** – A résztvevők számítási feladatot oldanak meg, hogy jogot szerezzenek a következő blokk hozzáadásához. Rendkívül energiaigényes, konzorciális láncban ritkán használják.
- **Proof of Stake (PoS)** – A validátorok a letétbe helyezett tőke arányában választódnak ki a blokk javaslására. Energiahatékonyabb, de a gazdasági ösztönzők bonyolultabbá válnak.
- **Practical Byzantine Fault Tolerance (PBFT)** – A csomópontok egymás között üzeneteket cserélnek, és szavazással döntenek a blokk elfogadásáról. Jó teljesítményű, ha a csomópontok száma kicsi (tipikusan néhány tucat), és a hálózat megbízható. Pontosan ez a jellemző teszi alkalmassá konzorciális láncokhoz.
- **Raft vagy más crash-fault toleráns algoritmus** – Csak a csomópontok leállását kezeli, a rosszindulatú viselkedést nem. Akkor használható, ha a résztvevők alapvetően megbíznak egymásban, és csak a rendelkezésre állás a követelmény.

A konszenzus kiválasztásakor minőségi forgatókönyvet kell alkotni. Például:

> **Inger:** Egy tranzakció érkezik a konzorciális lánc egy validátor csomópontjához.
> **Környezet:** Normál üzemi terhelés, 10 validátor csomópont, PBFT konszenzus.
> **Válasz:** A tranzakció bekerül egy blokkba, és a blokk véglegesen rögzítésre kerül.
> **Mérőszám:** A tranzakció véglegességének késleltetése legfeljebb 2 másodperc az esetek 99%-ában, 15 perces mérési ablakban mérve.

A mérőszámoknál mindig meg kell adni a mérési ablakot és a bizonytalanságot. A blokkláncoknál a *véglegesség* (finality) kulcsfogalom: az a pont, ahonnan a blokk már nem vonható vissza. PoW esetén ez valószínűségi, PBFT esetén abszolút.

31.3.6 2.3. Okosszerződések tervezési szempontjai

Az okosszerződés nem hagyományos program: a lánc minden validátorán determinisztikusan lefut, és az eredménynek minden csomóponton azonosnak kell lennie. Ezért:

- **Nincs véletlenszerűség** a láncon belül, hacsak nem külső oracle szolgáltatja, de az oracle megbízhatósága új bizalmi kérdést vet fel.
- **Nincs külső I/O** a tranzakció végrehajtása közben. A hálózati hívás vagy fájlolvasás megtörné a determinizmust.
- **Az erőforrás-felhasználás korlátozott.** Publikus láncokon a gáz (gas) szab határt a számítási komplexitásnak és a tárolásnak, hogy megakadályozza a végtelen ciklusokat és az erőforrás-kimerítést. Konzorciális láncokban is érdemes hasonló korlátokat beépíteni.

Az okosszerződések tipikus feladata: üzleti szabályok kikényszerítése (pl. csak a szerződő fél vonhat le pénzt), eszközök átruházása, több fél által aláírt műveletek végrehajtása, valamint az állapot auditálható rögzítése.

Például egy konzorciális láncra épülő ellátási láncban az okosszerződés felelős azért, hogy egy szállítmány állapotváltozása csak a jogosult féltől származhasson, és a tranzakció bekerüljön a főkönyvbe. A fizikai világ eseményeit (pl. hőmérséklet, GPS) szenzorok szolgáltatják, de a láncra csak azok hitelesített, aláírt rekordjai kerülnek. Ez a *oracle* probléma: a külső adatot valakinek megbízhatóan be kell vinni a láncra.

Architekturálisan az okosszerződés és a külvilág közötti határ kritikus. A jól tervezett rendszer ezt a határt explicit módon kezeli: az okosszerződés csak azt ellenőrzi, hogy a tranzakció megfelel-e a belső szabályoknak, a külső adat hitelességét pedig a beviteli komponens, az oracle kezeli.

31.3.7 3. Valós idejű renderelés és eszközkorlátok az AR/VR rendszerekben

Az AR/VR rendszerek architektúrájának középpontjában a *valós idejű renderelés* áll: a számítógépnek minden egyes képkockát néhány milliszekundum alatt kell előállítania ahhoz, hogy a felhasználó számára meggyőző, folyamatos, késleltetésmentes élményt nyújtson. Ez szigorú időbeli kényszereket ró minden komponensre, a megjelenítő eszköztől a felhőbeli szolgáltatásokig.

31.3.8 3.1. Az immerzív rendszer kritikus minőségi attribútumai

Az AR/VR rendszerek esetében a legfontosabb minőségi attribútumok:

- **Képfrissítési ráta (frame rate)** – A folyamatos mozgás illúziójához legalább 60 kép/másodperc, de a VR-ban gyakran 90–120 Hz szükséges. Ennél alacsonyabb érték hányingert és tájékozódási zavart okozhat.
- **Foton-késleltetés (motion-to-photon latency)** – A felhasználó mozgása és a megfelelő kép közötti idő. A cél általában 20 ms alatt tartani, mert a nagyobb késleltetés megtöri az immerziót és szédülést okoz.
- **Renderelési felbontás és látómező** – A felbontásnak elegendőnek kell lennie a szöveg és a részletek olvashatóságához, a látómezőnek pedig elég szélesnek a természetes érzethez. Ezek az erőforrásigényt növelik.
- **Eszköz súlya, fogyasztása és hőtermelése** – A headsetnek kényelmesnek kell lennie, ami korlátozza a beépíthető számítási kapacitást és akkumulátort.
- **Hálózati sávszélesség és késleltetés** – Ha a renderelés nem helyben történik, a hálózatnak nagy sávszélességet és alacsony késleltetést kell biztosítania.

Ezek a követelmények minőségi forgatókönyvként is megfogalmazhatók:

> **Inger:** A felhasználó elfordítja a fejét.
> **Környezet:** VR headset, 90 Hz-es kijelző, 5 GHz-es Wi-Fi kapcsolat, otthoni környezet.
> **Válasz:** A kijelzőn megjelenik az új nézetnek megfelelő renderelt kép.
> **Mérőszám:** A motion-to-photon késleltetés legfeljebb 20 ms az esetek 95%-ában, 5 perces mérési ablakban, ± 2 ms mérési bizonytalansággal.

31.3.9 3.2. Renderelési architektúrák: helyi, edge és távoli

Az AR/VR renderelés három fő elrendezésben valósulhat meg, mindegyik más-más trade-offot jelent:

1. **Helyi renderelés (local)** – A headset vagy a hozzá kapcsolt számítógép végzi a renderelést. Előnye a minimális késleltetés és a hálózati függés hiánya. Hátránya: az eszköz számítási kapacitása korlátozott, ami behatárolja a jelenet komplexitását és a vizuális minőséget. A high-end VR headsetek gyakran kábelen csatlakoznak egy erős PC-hez, hogy növeljék a számítási kapacitást, de ez a mobilitást korlátozza.
2. **Edge renderelés** – A renderelést a felhasználóhoz közeli edge szerver végzi, majd a kész képkockákat streameli a headsetre. Ez egyensúlyt kínál a számítási kapacitás és a késleltetés között. Az 5G és a több-hozzáférésű edge computing (MEC) csökkenti a hálózati késleltetést, de a motion-to-photon követelmény így is szoros határokat szab. Az edge renderelés jellemzően 20–50 ms közötti késleltetést tud elérni, ami a gyengébb élményt nyújtó AR alkalmazásokhoz még elfogadható lehet.
3. **Cloud renderelés** – A renderelés egy központi felhőben történik, a képkockák pedig a nyilvános interneten keresztül jutnak el az eszközre. Ez nagy rugalmasságot és skálázhatóságot ad, de a késleltetés általában 50 ms fölé nő, ami a VR esetében nem megfelelő. Használható olyan AR alapú bemutatókhoz, ahol a néző nem aktívan mozog, vagy ahol az interakció nem igényel azonnali visszajelzést.

A gyakorlatban gyakran hibrid megoldásokat alkalmaznak: a nagy számítási igényű, statikus elemeket előre renderelik vagy a felhőben készítik, míg az időérzékeny, felhasználó-közeli elemeket helyben renderelik. Az *foveated rendering* (a tekintet irányába eső terület nagyobb felbontással renderelése, a periférián alacsonyabbal) szintén csökkenti a számítási terhelést anélkül, hogy a felhasználó észrevenné.

31.3.10 3.3. Eszközkorlátok és azok architekturális hatása

Az AR/VR eszközök – különösen a mobileszközök és a stand-alone headsetek – erőforrásai korlátozottak: a CPU/GPU teljesítmény, a memória, a sávszélesség és az akkumulátor-élettartam mind szűkösek. Ezért a szoftverarchitektúrának kifejezetten figyelembe kell vennie:

- **Erőforrás-optimalizálás** – A jelenetgráfot, a textúrákat és a geometriát folyamatosan optimalizálni kell. A túlzott részletezettség gyorsan kimeríti a GPU-t, ami framerate-csökkenéshez vezet.
- **Aszinkron feldolgozás** – A renderelési ciklust el kell választani a hálózati I/O-tól és a fizikai szimulációtól, hogy a képfrissítés soha ne blokkolódjon. A tipikus minta: a játék vagy alkalmazás ciklusa rögzített időlépésekkel fut, míg a renderelő a lehető leggyorsabban készít képkockákat.
- **Predikció és kompenzáció** – A késleltetés hatásának csökkentésére a rendszer előre jelezheti a felhasználó mozgását, és aszerint renderelhet, majd a tényleges pozíciót utólag korrigálhatja. Ez különösen fontos hálózati renderelésnél.
- **Tartalom-streamelés** – A nagyobb világokhoz a tartalmat fokozatosan kell letölteni a felhasználó közeledtével, nem egyszerre. Ez hasonló a webes lusta betöltéshez, de szigorúbb időkorlátokkal.

Az architektúrában ezek a kényszerek megjelennek a komponensek felosztásában is: egy tipikus VR/AR alkalmazás komponensei például a **jelenetkezelő**, a **renderelő**, a **beviteli rendszer**, a **hálózati réteg** és a **tartalomkezelő**. A renderelő komponens felelőssége a képkockák időben

történő előállítása, és interfésze felé a jelenetkezelő biztosítja a szükséges geometriát. A beviteli rendszer fogadja a szenzorokat (fejmozgás, kézmozgás, szemkövetés), és biztosítja az alacsony késleltetésű adatfolyamot a renderelő felé.

31.3.11 4. Architektúra-minták és integráció

Mind a blokklánc, mind az AR/VR rendszerek beilleszthetők a korábban tanult architekturális stílusokba. A blokklánc leggyakrabban *eseményvezérelt* módon kapcsolódik a többi rendszerhez: a láncon bekövetkező állapotváltozások eseményeket generálnak, amelyeket a külvilág feliratkozással fogyaszt. Az okosszerződések az üzleti logika egy jól körülhatárolt részét hordozzák, míg a hagyományos alkalmazások a felhasználói élményt és a nem determinisztikus folyamatokat valósítják meg.

A VR/AR rendszerek gyakran *kliens-szerver* vagy *rétegezett* architektúrájúak, de a valós idejűség miatt a kliensoldal sokkal vastagabb. A jelenetkezelés és a renderelés a kliensben fut, míg a felhő vagy az edge a tartalomszolgáltatást, a felhasználói fiókokat és az analitikát biztosítja. Az eszközkorlátok miatt a mikroszolgáltatásos felbontás itt ritkán indokolt; a helyi komponensek inkább folyamatok vagy szálak, mint hálózati szolgáltatások.

Egy érdekes metszéspont: a blockchain-alapú digitális tulajdon (NFT) és a VR/AR virtuális világok kombinációja. Itt a blokklánc biztosítja a virtuális eszközök tulajdonjogának nyilvántartását, míg a VR/AR a megjelenítést és az interakciót. Az ilyen rendszerekben a lánc tranzakcióinak késleltetése nem zavarhatja a valós idejű játékmenetet: a tulajdonjog-változás aszinkron módon, a jelenetkezeléstől elkülönítve történik, és a szerver nem blokkolja a renderelést a lánc válaszára várva.

31.3.12 5. Tipikus hibák és helyes megoldások

A blokklánc és az AR/VR területén számos visszatérő hiba figyelhető meg. Néhány gyakori eset és a helyes architekturális válasz:

31.3.13 5.1. „Mindenre blokklánc kell”

Hiba: A szervezet egy belső rendszerhez is blokkláncot ír elő, mert az „modern” vagy „biztonságos”. Ezzel megnő a késleltetés, csökken az átbocsátóképesség, és az üzemeltetési költség az egekbe szökik, miközben a tényleges bizalmi modell nem változik.
Helyes megoldás: Először tisztázni kell a bizalmi követelményt. Ha van egyetlen megbízható szervezet, vagy a felek közötti bizalom erős, akkor egy hagyományos, jól auditált adatbázis jobb választás. A blokkláncot csak akkor érdemes bevezetni, ha a decentralizáció valódi követelmény, és a résztvevők motiváltak a közös csomópontok üzemeltetésére.

31.3.14 5.2. Az okosszerződés túl sok üzleti logikát tartalmaz

Hiba: Az okosszerződésbe bonyolult számításokat, nagy adathalmazokat kezelő folyamatokat vagy nem determinisztikus műveleteket raknak. A tranzakció drága, a válaszidő lassú, a frissítés nehézkes.
Helyes megoldás: Az okosszerződés maradjon vékony: csak a közös állapot érvényesítését és a kritikus üzleti szabályok kikényszerítését végezze. Az összetett üzleti logika fusson az alkalmazásrétegben, a láncra csak a végeredmény, a bizonyíték vagy a digitális aláírás kerüljön.

31.3.15 5.3. A konszenzusmechanizmus nem illik a bizalmi modellhez

Hiba: Egy konzorcium nyilvános láncokhoz tervezett, lassú és energiaigényes PoW algoritmust használ egy zárt hálózatban, ahol a résztvevők ismertek és szerződésesen kötöttek. Ezzel hatalmas felesleges költség keletkezik.
Helyes megoldás: Permissioned környezetben a BFT-alapú konszenzus (pl. PBFT) vagy annak modern változatai alacsony késleltetéssel és jó teljesítménnyel biztosítják a véglegességet. Csak akkor érdemes PoW vagy PoS felé mozdulni, ha a hálózat nyilvános, és a résztvevők nem feltétlenül azonosíthatók.

31.3.16 5.4. A motion-to-photon követelmény figyelmen kívül hagyása

Hiba: Egy AR alkalmazást teljes egészében a felhőben renderelnek, és a fejlesztők nem mérik a késleltetést. A felhasználók szédülésről és rossz élményről számolnak be, de a hibát a hálózatra fogják.
Helyes megoldás: A motion-to-photon késleltetést minőségi forgatókönyvként rögzíteni kell, és a renderelési architektúrát ennek megfelelően kell kiválasztani. Ha a késleltetés nem tartható, a kritikus interakciókat helyben vagy edge-en kell renderelni, a felhőt pedig csak háttérfeladatokra használni.

31.3.17 5.5. Az eszközkorlátok figyelmen kívül hagyása a jelenet tervezésekor

Hiba: A 3D modellezők túl részletes jelenetet készítenek, és a headset nem bírja a renderelést, ezért a képfrissítés 30 fps alá esik. A rendszer orvoslásaként a felbontást drasztikusan csökkentik, ami rontja a látványt.
Helyes megoldás: Már a tervezéskor meg kell határozni a jelenet bonyolultsági korlátját, és folyamatosan ellenőrizni kell a framerate-et. Az optimalizálási technikák (LOD, culling, foveated rendering) beépítése a renderelő komponensbe előre tervezhető.

31.3.18 6. Labor: Blockchain és VR/AR használati eset értékelése

Ez a labor a fejezet tanulási céljait gyakoroltatja: értékelni kell egy adott üzleti helyzetet, és döntést hozni arról, hogy a blokklánc vagy a hagyományos architektúra a megfelelő, valamint megvizsgálni egy VR/AR renderelési igény architekturális kényszereit. A feladat reprodukálható dokumentumban vagy jegyzetfüzetben oldható meg, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal. A mért és a várt eredményt külön kell választani.

31.3.19 6.1. Kiinduló állapot

Képzeljünk el egy közepes méretű logisztikai céget, amely egy konzorciumot alakít több beszállítóval és kiskereskedővel. A cél: a szállítmányok eredeti okmányainak (pl. származási igazolás, fuvarlevél) digitális, hamisíthatatlan nyilvántartása, és a státuszváltozások megbízható rögzítése. A konzorcium tagjai nem feltétlenül bíznak meg egymásban, de mindannyian hajlandók lennének üzemeltetni egy-egy csomópontot.

Emellett a cég belső képzést szeretne indítani VR alapú raktárbiztonsági oktatáshoz. A headsetek stand-alone eszközök, Wi-Fi-n csatlakoznak, és a jelenet egy raktárt ábrázol, amelyben a dolgozó mozog és interakcióba lép a biztonsági eszközökkel.

31.3.20 6.2. Elvárt eredmény

Készíts egy rövid dokumentumot az alábbi szerkezetben:

1. **Kontextus leírása** – Röviden rögzítsd az üzleti célt, a stakeholdereket és a rendszerhatárt mindkét esetre.
2. **Döntési napló (ADR vázlat)** – A logisztikai nyilvántartásra vonatkozóan indokold meg, hogy blokklánc vagy hagyományos elosztott adatbázis mellett döntesz-e. Tüntesd fel a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. Ha blokkláncot választasz, határozd meg a típust (konzorciális), a konszenzusmechanizmust (pl. PBFT), és indokold a választást. Ha hagyományos adatbázist választasz, mutasd be, miért nem szükséges a decentralizált bizalom.
3. **Minőségi forgatókönyv** – Írj legalább két minőségi forgatókönyvet a logisztikai rendszerre (például a tranzakciók véglegességére és a csomópontok rendelkezésre állására), a megadott formában: inger, környezet, válasz, mérőszám.
4. **VR/AR renderelési értékelés** – A VR oktatóalkalmazáshoz értékeld a renderelési architektúra lehetőségeit: helyi, edge, cloud. Határozd meg a kritikus minőségi attribútumot (motion-to-photon késleltetés, framerate), és adj meg egy minőségi forgatókönyvet. Indokold, melyik renderelési módot választanád, és milyen eszközkorlátokra kell figyelni.
5. **Mért vs. várt eredmény** – Külön táblázatban sorold fel, hogy mely értékeket becsülted vagy vetted célként (várt), és melyeket mértél ténylegesen (mért). Ha nem végeztél mérést, azt is jelezd, és írd le, hogyan lehetne elvégezni egy reprodukálható mérést.

31.3.21 6.3. Ellenőrzési szempontok

- A döntési naplóban a kontextus egyértelműen rögzített, az alternatívák reálisak.
- A minőségi forgatókönyv mind a négy elemet tartalmazza, a mérőszám mértékegységgel és mérési ablakkal szerepel.
- A VR/AR értékelés figyelembe veszi a motion-to-photon késleltetést és az eszközkorlátokat.
- A mért és várt eredmények elkülönítése megtörtént, a nem mért becslések jelölve vannak.
- A dokumentum verziózott, és a döntések hivatkozhatók.

31.3.22 6.4. Visszaállítási út

Ez a labor csak dokumentációt és leírást igényel, nem módosít éles rendszert. Ha valamilyen eszközt (pl. lánc szimulátort vagy VR framerate mérőt) használsz, izolált környezetben tedd, és a mérés után állítsd vissza a kiinduló állapotot (pl. töröld a generált adatokat).

31.3.23 7. Ellenőrzőlista a fejezethez

- ☐ Meg tudom magyarázni, mi a decentralizált bizalom, és miben különbözik a központosított bizalomtól.
- ☐ Fel tudom sorolni a blokklánc alkalmazásának feltételeit, és meg tudom indokolni, mikor nem érdemes használni.
- ☐ Ismerem a konzorciális lánc fő komponenseit (validátor, rendező, tagsági szolgáltatás) és azok felelősségét.
- ☐ Össze tudom hasonlítani a PoW, PoS és PBFT konszenzusmechanizmusokat, és ki tudom választani a megfelelőt adott kontextusban.
- ☐ El tudom magyarázni az okosszerződések korlátait (determinizmus, gáz, nincs külső I/O), és be tudom mutatni a vékony okosszerződés mintát.
- ☐ Tisztában vagyok a VR/AR rendszerek kritikus minőségi attribútumaival (frame rate, motion-to-photon latency), és tudom ezeket minőségi forgatókönyvként megfogalmazni.
- ☐ Ismerem a helyi, edge és cloud renderelés trade-offjait, és adott esetre el tudom dönteni a megfelelőt.
- ☐ Felismerem a tipikus hibákat (blokklánc mindenre, eszközkorlátok figyelmen kívül hagyása), és meg tudom fogalmazni a helyes megoldást.

31.3.24 8. Önellenőrző kérdés

Adott egy hatósági nyilvántartás, ahol egyetlen kormányzati szerv felel az adatok hitelességéért, de több külső fél csak olvasási hozzáférést kap. A rendszer napi több tízezer tranzakciót dolgoz fel, és a válaszidő kritikus. Melyik megoldást választanád: blokklánc vagy hagyományos, központosított adatbázis? Indokold válaszodat a fejezetben tanult kritériumok alapján, és fogalmazz meg egy ADR-t a döntés rögzítésére.

Lehetséges válasz: Mivel egyetlen megbízható fél van, a decentralizált bizalomra nincs szükség, a blokklánc csak felesleges késleltetést és költséget okozna. Egy jól felügyelt, auditált, központosított adatbázis (esetleg replikációval a rendelkezésre állás érdekében) jobb választás. Az ADR-nek rögzítenie kell a kontextust, az elvetett blokklánc alternatívát, a döntést és a következményeket: alacsonyabb késleltetés, egyszerűbb üzemeltetés, de egy központi bizalmi pont. Ha a jövőben a bizalmi kör megváltozik, a döntés felülvizsgálható.

Ezzel a fejezet bemutatta a decentralizált bizalom és az immerzív rendszerek architekturális alapelveit, a konzorciális láncok és okosszerződések tervezési szempontjait, valamint a valós idejű renderelés eszközkorlátait. A labor segítségével a fogalmak egy reprodukálható, döntésközpontú dokumentumban mélyíthetők el. A következő fejezet (Quantum computing és a következő évtized) ezekre az alapokra építve a kvantumszámítás és a következő évtized várható hatásait vizsgálja.

31.4 Quantum computing és a következő évtized

A kvantumszámítás ma még nem érinti közvetlenül a legtöbb vállalati rendszer napi működését, mégis az egyik legnagyobb stratégiai kockázatot és lehetőséget jelenti a szoftverarchitektúrák számára. A következő évtizedben az architektnek legalább három szinten kell foglalkoznia a témával: meg kell értenie, mely algoritmusok válhatnak gyakorlatilag futtathatóvá; fel kell készülnie a ma használt aszimmetrikus kriptográfia esetleges törésére; és ki kell alakítania egy olyan hibrid rendszerképet, amelyben a kvantumprocesszor (QPU) speciális gyorsítóként jelenik meg a klasszikus infrastruktúra mellett.

A fejezet nem kvantumfizikai bevezető, hanem architekturális döntéstámogatás. A cél, hogy a szoftverarchitektúrával foglalkozó olvasó képes legyen felismerni a kvantumszámítás hatásait, elkerülje a túlzott lelkesedést és a túlzott szkepticizmust, és megalapozott döntéseket hozzon a rendszerei védelmében és fejlesztésében.

31.4.1 1. Alapfogalmak: qubit, szuperpozíció és a kvantumfenyegetés

A klasszikus bit két állapotú (0 vagy 1), a kvantumbit (qubit) ezzel szemben a két állapot szuperpozíciójában is lehet. Matematikailag egy qubit állapota

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

ahol α és β komplex amplitúdók, és $|\alpha|^2 + |\beta|^2 = 1$. A mérés pillanatában a qubit valamelyik bázisállapotba esik vissza, méghozzá $|\alpha|^2$ valószínűséggel $|0\rangle$-ba, és $|\beta|^2$ valószínűséggel $|1\rangle$-be. A szuperpozíció tehát nem azt jelenti, hogy a qubit egyszerre “mindkét értéket tárolja”, hanem azt, hogy a mérés előtt a rendszer egy meghatározott valószínűségi amplitúdóeloszlással írható le.

Az összefonódás (entanglement) két vagy több qubit között olyan korrelációt hoz létre, amely klasszikus bitekkel nem modellezhető. Ez teszi lehetővé a kvantumalgoritmusok számára, hogy bizonyos problémákat aszimptotikusan gyorsabban oldjanak meg, mint a ma ismert klasszikus algoritmusok. Fontos azonban, hogy a kvantumszámítás nem univerzálisan gyorsabb: csak olyan problémáknál ad előnyt, ahol a kvantumos párhuzamosság vagy interferencia kihasználható.

A legismertebb kvantumalgoritmusok közé tartozik:

- **Shor-algoritmus**: egész számok faktorizálása és diszkrét logaritmus számítása polinomiális időben. Ez közvetlenül fenyegeti az RSA-t, az ECC-t és a Diffie-Hellman kulcscserét, mert ezek biztonsága a faktorizálás vagy a diszkrét logaritmus nehézségén alapul.
- **Grover-algoritmus**: rendezetlen keresés N elemű halmazban $O(\sqrt{N})$ időben, a klasszikus $O(N)$ helyett. Ez gyakorlatilag a szimmetrikus kulcsú titkosítás (pl. AES) és a hash függvények erősségét csökkenti: egy 128 bites kulcsú AES esetén a klasszikus brute force 2^{128} műveletet igényel, a Grover-algoritmussal viszont nagyjából 2^{64} kvantumművelet elegendő lehet, ezért a 128 bites biztonsági szint kvantumbiztos szempontból csak 64 bitesnek tekinthető. A gyakorlati fenyegetés mértéke azonban függ a kvantumszámítógép méretétől és hibaarányától.

Az architekt számára a legfontosabb következmény: a ma használt nyilvános kulcsú kriptográfia (RSA, ECC, Diffie-Hellman) elvileg sebezhetővé válik egy kellően nagy, hibatűrő kvantumszámítógép esetén. Ez a fenyegetés nem holnapi, de a “store now, decrypt later” támadás miatt már ma is releváns: a támadó rögzítheti a titkosított forgalmat, és eltárolhatja addig, amíg a kvantumszámítógép el nem éri a szükséges képességet. Ezért a hosszú élettartamú, érzékeny adatoknál már most meg kell kezdeni a felkészülést.

31.4.2 2. Jelenlegi korlátok és a NISQ korszak

A kvantumszámítógépek jelenlegi generációját NISQ-nak (Noisy Intermediate-Scale Quantum) nevezik: a qubitek száma közepes (nagyságrendileg száz és ezer között), a műveletek zajosak, és nincs teljes körű hibajavítás. A 2025 eleji nyilvános ismeretek szerint a legnagyobb szupravezető qubites gépek néhány száz fizikai qubittel működnek, míg az ioncsapdás és semleges atomos platformok más skálázási utat követnek. Ezek a számok rohamosan változnak, ezért a konkrét qubitszámok és hibaarányok mindig ellenőrzést igényelnek a gyártó aktuális dokumentációjában.

A NISQ gépek fő korlátai:

- **Dekoherencia**: a qubitek kvantumállapota a környezettel való kölcsönhatás miatt gyorsan leromlik; a koherenciaidő gyakran mikroszekundumos vagy milliszekundumos tartományban van.
- **Zaj és hibaarány**: a kvantumkapuk hibavalószínűsége jellemzően 10^{-2}–10^{-3} nagyságrendű, ami messze van a hibatűrő számításhoz szükséges 10^{-9}–10^{-12} alatti értékektől.
- **Qubit-kapcsoltság**: nem minden qubit tud közvetlenül kölcsönhatni mindegyikkel; a gép topológiája korlátozza az algoritmusok leképezését.
- **Mérési hiba**: a kiolvasás is hibával terhelt, ami tovább rontja az eredmény megbízhatóságát.

Mindez azt jelenti, hogy ma még nem futtatható olyan Shor-algoritmus, amely egy 2048 bites RSA kulcsot feltörne. A jelenlegi gépek főleg kis méretű kísérleti algoritmusokra, kvantumszimulációra, valamint hibrid variációs algoritmusokra alkalmasak, ahol a klasszikus optimalizáló kompenzálja a kvantumos zajt.

A korlátok ellenére a technológia fejlődése folyamatos: a gyártók növelik a qubitek számát, csökkentik a hibaarányt, és fejlesztik a hibajavítást. Az “quantum advantage” (kvantumfölény) kifejezést akkor használják, ha egy kvantumszámítógép olyan feladatot old meg, amelyre a klasszikus számítógép gyakorlatilag képtelen. A Google 2019-ben jelentette be a Sycamore processzorral elért “quantum supremacy” eredményt egy mesterséges mintavételezési feladaton; ezt azóta több kritika és klasszikus utánzat is követte, ezért a gyakorlati hasznú fölény még nem bizonyított. Az architekt számára a kulcsmondat: a kvantumszámítás ma még nem érett termelési feladatokra, de a stratégiai tervezést nem szabad a “majd meglátjuk” fázisra halasztani.

31.4.3 3. Kvantumalgoritmusok és gyakorlati felhasználási területek

A kvantumalgoritmusok három nagy csoportba sorolhatók a várható gyakorlati hatás szempontjából:

1. **Kriptográfiailag releváns algoritmusok**: Shor-algoritmus (faktorizálás, diszkrét logaritmus) és Grover-algoritmus (keresés). Ezek közvetlenül fenyegetik a mai PKI-t és gyengítik a szimmetrikus kulcsokat.
2. **Szimulációs és anyagtudományi algoritmusok**: kvantumrendszerek szimulációja (pl. molekulák energiaszintjei, katalizátorok, szupravezetők). A kvantumszámítógép természetes módon képes kvantumrendszereket modellezni, ami a kémia, gyógyszeripar és anyagtudomány területén hozhat áttörést.
3. **Optimalizálási és gépi tanulási algoritmusok**: Variational Quantum Eigensolver (VQE), Quantum Approximate Optimization Algorithm (QAOA), kvantum neurális hálózatok. Ezek ma még hibrid futtatásban működnek, és nem garantált, hogy klasszikus fölényt érnek el minden feladaton.

Az architekt szempontjából nem az a kérdés, hogy pontosan melyik algoritmus mit tud, hanem az, hogy a szervezet üzleti folyamataiban hol jelenhet meg kvantumgyorsítás. A tipikus területek: logisztikai útvonaltervezés, pénzügyi portfólió-optimalizálás, gyógyszerkutatás, anyagtervezés, ellátási lánc modellezés. Ezen területeken a klasszikus rendszer ma is működik, de előbb-utóbb érdemes lehet kvantumgyorsítót integrálni.

31.4.4 4. Hibajavítás és a logikai qubit

A NISQ korszakból a hibatűrő kvantumszámításba való átmenet a kvantumhibajavításon múlik. A kvantumhibajavítás alapötlete, hogy több fizikai qubitből egy logikai qubitet képezünk, és redundáns kódolással, valamint szindrómaméréssel észleljük és javítjuk a hibákat anélkül, hogy a kvantumállapotot megmérnénk (ami összeomlasztaná azt).

A legismertebb hibajavító kód a surface code (felületi kód), amelyben egy logikai qubitet nagyszámú fizikai qubitből álló rács valósít meg. A szükséges fizikai qubitek száma erősen függ a fizikai qubitek hibaarányától. Ha a fizikai hibaarány 10^{-3} körül van, egy logikai qubit elkészítéséhez nagyságrendileg ezer fizikai qubit kellhet; ha a fizikai hibaarány 10^{-5} alá csökken, a szükséges overhead drasztikusan csökken. A 2025 eleji kutatások arra utalnak, hogy az első hasznos logikai qubitek megjelenése még évekre van, de az irány egyértelmű.

Az architekt számára a logikai qubit fogalma azért fontos, mert a valódi kvantumelőny (pl. Shor-algoritmus futtatása) csak hibatűrő, logikai qubites gépeken lesz elérhető. A kapacitástervezésnél tehát nem a fizikai qubitszámot, hanem a rendelkezésre álló logikai qubitek számát és a logikai műveletek hibaarányát kell figyelembe venni. Ez hasonló ahhoz, ahogy a klasszikus rendszereknél nem a tranzisztorok számát, hanem a hasznos utasítás-végrehajtási kapacitást mérjük.

31.4.5 5. Hibrid kvantum-klasszikus architektúrák

A jelenlegi és a közeljövő kvantumszámítása gyakorlatilag mindig hibrid: a klasszikus rendszer vezérli a kvantumprocesszort, előkészíti a bemenetet, futtatja a kvantumköröket, majd feldolgozza a mérési eredményeket. A QPU nem önálló számítógép, hanem speciális gyorsító, hasonlóan a GPU-hoz vagy az FPGA-hoz, de sokkal szigorúbb környezeti követelményekkel.

Egy tipikus hibrid architektúra komponensei:

- **Klasszikus alkalmazás**: az üzleti logika, amely meghatározza, melyik feladatot érdemes kvantumra bízni.
- **Kvantum-orchestrátor**: feladata a kvantumfeladatok ütemezése, a klasszikus és kvantum erőforrások közötti adatcsere, valamint a hibatűrési stratégiák kezelése.
- **Fordító és optimalizáló**: a magas szintű kvantumáramkört leképezi az adott QPU natív utasításaira, figyelembe véve a qubit-topológiát és a zajmodellt.
- **QPU (Quantum Processing Unit)**: a tényleges kvantumszámítást végző hardver, amely lehet felhőben elérhető szolgáltatás vagy helyben telepített eszköz.
- **Klasszikus optimalizáló**: variációs algoritmusoknál a kvantumos mérési eredmények alapján frissíti a paramétereket, és visszaadja a kvantumkörnek.

Az ilyen rendszerekben a kvantumfeladat általában aszinkron módon fut: a klasszikus rendszer elküldi a kvantumkört, a QPU várólistára helyezi, majd a futás befejeztével visszaadja a mérési statisztikát. A várakozási idő jelentős lehet, ezért az architektúrának késleltetés-tűrőnek kell lennie, és nem szabad szinkron hívásban blokkolni a felhasználói tranzakciót.

Az alábbi szekvenciadiagram egy variációs kvantumalgoritmus (VQE) lefutását mutatja be hibrid környezetben:

```
sequenceDiagram
    participant App as Klasszikus alkalmazás
    participant Orch as Kvantum-orchestrátor
    participant QPU as QPU
    participant Opt as Klasszikus optimalizáló

    App->>Orch: Kvantumfeladat (paraméterek)
    loop Amíg konvergencia nincs
        Orch->>QPU: Kvantumkör futtatása
        QPU-->>Orch: Mérési eredmények
        Orch->>Opt: Eredmények és paraméterek
        Opt-->>Orch: Új paraméterek
        Orch->>QPU: Frissített kör futtatása
    end
    Orch-->>App: Végső eredmény
```

Jelmagyarázat: App = klasszikus alkalmazás, Orch = kvantum-orchestrátor, QPU = kvantum-processzor, Opt = klasszikus optimalizáló.
Feltételjegyzet: A diagram a variációs kvantumalgoritmusok iteratív hurkát mutatja; a QPU hívás aszinkron és várólistázott lehet.

Az architeknek különösen figyelnie kell a hibrid rendszerek nem funkcionális követelményeire:

- **Teljesítmény**: a QPU hívás késleltetése (sorban állás + futtatás) nagyságrendileg másodpercek vagy percek lehet, ezért a szinkron felhasználói interakciókat kerülni kell.
- **Rendelkezésre állás**: a kvantumhardver karbantartási ablakai és kalibrálási igénye miatt a QPU nem folyamatosan érhető el; a rendszernek hibakezeléssel és újrapróbálkozással kell rendelkeznie.
- **Megbízhatóság**: a mérési eredmények zajosak, ezért az alkalmazásnak több futtatást kell aggregálnia (shot count), és statisztikai kiértékelést kell végeznie.
- **Biztonság**: a kvantumfeladatok és eredmények érzékenyek lehetnek; a hálózati kommunikációt és a tárolást ugyanúgy védeni kell, mint bármely más adatot.

31.4.6 6. Posztkvantum-kriptográfia (PQC) és migráció

A posztkvantum-kriptográfia (PQC) olyan klasszikus kriptográfiai algoritmusokat jelent, amelyeket úgy terveztek, hogy ellenálljanak a kvantumszámítógépes támadásoknak is. A PQC nem kvantumkriptográfia: az algoritmusok hagyományos, klasszikus számítógépeken futnak, de a mögöttes matematikai problémákat (rácsalapú, kódalapú, hash-alapú, többváltozós polinom-alapú) úgy választják meg, hogy ne legyen ismert kvantumalgoritmus a gyors megoldásukra.

A NIST 2016-ban indította el a posztkvantum szabványosítási folyamatát, és 2024-ben publikálta az első szabványokat:

- **ML-KEM (CRYSTALS-Kyber)**: kulcsmegállapodás (key encapsulation mechanism, KEM).
- **ML-DSA (CRYSTALS-Dilithium)**: digitális aláírás.
- **SLH-DSA (SPHINCS+)**: hash-alapú digitális aláírás.
- **FN-DSA (Falcon)**: további aláírási algoritmus.

Ezek a nevek és paraméterek 2025 elején érvényesek, de a szabványosítási folyamat tovább alakulhat, ezért a bevezetés előtt mindig ellenőrizni kell a NIST aktuális közleményeit.

A PQC migráció nem egyszerű algoritmuscsere. A következő lépésekből állhat:

1. **Kriptográfiai leltár**: fel kell térképezni, hol használ a rendszer titkosítást, kulcsmegállapodást, digitális aláírást, hash-t. A leltárnak tartalmaznia kell az algoritmus nevét, kulcsméretét, a használat helyét és az adat élettartamát.
2. **Kockázatbesorolás**: az egyes eszközöket aszerint kell rangsorolni, hogy milyen érzékenységű adatot védenek, mennyi ideig kell az adatnak titkosnak maradnia, és mekkora a kitettség a "store now, decrypt later" támadásnak.
3. **Algoritmusválasztás**: a NIST szabványok közül kiválasztani a megfelelő PQC algoritmust, figyelembe véve a kulcsméreteket, az aláírásméretet, a teljesítményt és a hardveres támogatást.
4. **Hibrid üzemmód**: a bevezetés első fázisában a klasszikus és a posztkvantum algoritmust párhuzamosan futtatni, vagy kombinált (hibrid) séma használata, hogy ha az egyikben hiba lenne, a másik még védelmet nyújtson.
5. **Integráció és tesztelés**: a kriptográfiai könyvtárak cseréje, a protokollok (TLS, SSH, VPN) frissítése, az ügyfél–szerver egyeztetés kezelése.
6. **Fokozatos bevezetés**: a rendszer egyes részein fokozatosan bevezetni a PQC-t, folyamatos megfigyeléssel és visszagörgetési lehetőséggel.

Az alábbi folyamatábra a migráció lépéseit összegzi:

```
graph TD
    A[Kriptográfiai leltár] --> B[Kockázatbesorolás]
    B --> C[Prioritási sorrend]
    C --> D[Algoritmusválasztás és prototípus]
    D --> E[Integráció és tesztelés]
    E --> F[Hibrid üzemmód bevezetése]
    F --> G[Teljes átállás és megfigyelés]
```

Jelmagyarázat: A folyamat a PQC migráció főbb szakaszait mutatja.
Feltételjegyzet: A hibrid üzemmód addig marad fenn, amíg az új algoritmus stabilitása nem igazolódik.

A migráció sikerének kulcsa a **kriptográfiai agilitás**: a rendszernek képesnek kell lennie az algoritmusok gyors cseréjére, anélkül, hogy az üzleti logikát módosítani kellene. Ez azt jelenti, hogy a kriptográfiai primitíveket absztrakciós réteg mögé kell helyezni, a kulcshosszakat és algoritmusokat konfigurációból kell vezérelni, és a protokolloknak tárgyalniuk kell a támogatott algoritmusokról.

A PQC bevezetés minőségi követelményeit quality attribute scenario formában is megfogalmazhatjuk:

- **Inger**: egy ügyfél új kérésben PQC-algoritmust igényel a TLS-kapcsolatban.
- **Környezet**: normál üzem, éles környezet, terhelt állapot.
- **Válasz**: a szerver a támogatott PQC-csomagot választja, a kézfogás megtörténik, a munkamenet létrejön.
- **Mérőszám**: a kézfogási idő nem haladja meg a klasszikus ECDSA-kézfogás 1,5-szeresét a p95-ös percentilisben, 30 napos mérési ablakban.

Az ilyen forgatókönyv segít mérhetővé tenni a migráció sikerét.

31.4.7 ADR minta a PQC migrációhoz

A döntési naplóban rögzíthetjük a posztkvantum algoritmus kiválasztását az alábbi módon:

ADR-2025-014: Posztkvantum kulcsmegállapodás bevezetése a belső szolgáltatásközi TLS-hez

- **Kontextus**: A belső mikroszolgáltatások jelenleg ECDSA-t és ECDHE-t használnak. A hosszú élettartamú adatforgalom veszélyeztetett a "store now, decrypt later" támadással szemben. A szervezet 2026 végéig szeretné a legkritikusabb csatornákon bevezetni a PQC-t.
- **Döntés**: A TLS 1.3-as kapcsolatokban az X25519 mellett ML-KEM-768-at is bevezetünk hibrid kulcsmegállapodásként. A kiválasztás oka: a NIST által szabványosított, széles körben elérhető könyvtári támogatottság, kedvező kulcsméret és teljesítmény.
- **Alternatívák**: (1) Teljes PQC azonnal: magas kockázat a kompatibilitás miatt; (2) Hagyományos kriptográfia fenntartása: nem kezeli a hosszú távú fenyegetést; (3) Kísérleti, nem szabványosított algoritmus: túl kockázatos.
- **Következmények**: A kézfogási idő várhatóan 10–20%-kal nő, a kulcsméretek nagyobbak, és a könyvtárak frissítése szükséges minden szolgáltatásban. A hibrid üzemmód miatt a régi kliensek továbbra is tudnak csatlakozni.
- **Állapot**: Elfogadva, bevezetés alatt.

Ez a példa jól mutatja, hogyan kell az architekturális döntéseket rögzíteni.

31.4.8 7. Architekturális kockázatok és trade-offok

A kvantumszámítás és a PQC bevezetése számos trade-offot hordoz:

- **Teljesítmény vs. biztonság**: a PQC algoritmusok gyakran nagyobb kulcsot és aláírást igényelnek, ami növeli a sávszélességet és a CPU-terhelést. Ezért a hálózati protokollokban és a korlátozott erőforrású eszközökön (IoT) különösen mérlegelni kell.
- **Kompatibilitás**: a régi kliensek és a nem frissíthető eszközök akadályozhatják a gyors átállást. Az architektnek kompatibilitási réteget vagy hibrid módot kell terveznie.
- **Szabványosítási bizonytalanság**: a NIST folyamata még nem teljesen zárt, további algoritmusok válhatnak szabványossá, vagy a meglévők paraméterei változhatnak. Ezért a rendszernek kriptográfiai agilitással kell rendelkeznie.
- **Kvantumhardver-függőség**: ha a szervezet kvantumfelhő-szolgáltatást használ, a vendor lock-in kockázata fennáll. A hibrid architektúrában célszerű az absztrakciós réteget kialakítani, hogy a QPU-szolgáltató cserélhető legyen.

Ezen kockázatok kezelésére az architekt fitness function-öket is definiálhat. Például egy fitness function ellenőrizheti, hogy az összes TLS-végpont támogatja-e a hibrid PQC-t, vagy hogy a kriptográfiai könyvtárak verziója meghaladja-e a minimumot. Az ilyen tesztek folyamatosan futhatnak a CI/CD pipeline-ban.

31.4.9 8. Labor: kvantumhatás-kockázati lista

A labor célja, hogy a résztvevő elkészítsen egy reprodukálható dokumentumot, amelyben felméri egy választott rendszer kvantumszámítással kapcsolatos kockázatait, rangsorolja azokat, és döntési naplót készít a szükséges lépésekről.

31.4.10 Kiinduló állapot

Válasszon ki egy valós vagy kitalált rendszert, amely legalább háromféle kriptográfiai műveletet használ (pl. TLS, digitális aláírás, adatbázis-titkosítás). A rendszer lehet e-kereskedelmi platform, banki háttérrendszer, egészségügyi nyilvántartás vagy IoT-hálózat.

31.4.11 Elvárt eredmény

Egy dokumentum (Markdown vagy jegyzetfüzet), amely tartalmazza:

1. **Kontextusleírást**: a rendszer neve, célja, főbb komponensei, a használt kriptográfiai algoritmusok listája, az adatok érzékenysége és élettartama.
2. **Kockázati listát**: minden kriptográfiai felhasználáshoz rendeljen kockázati besorolást a következő szempontok szerint:
   - Kvantumfenyegetés típusa (Shor, Grover vagy egyéb)
   - Érintett adatok élettartama (rövid, közepes, hosszú)
   - A támadás bekövetkezési valószínűsége a következő 5–10 évben (alacsony, közepes, magas)
   - A lehetséges hatás (üzleti, jogi, reputációs)
   - A javasolt intézkedés (pl. PQC migráció, kulcsméret növelés, megfigyelés)
3. **Döntési naplót**: legalább két ADR-t a fenti sablon szerint. Az egyik a PQC bevezetéséről szólhat, a másik a kvantumhardver használatának elhalasztásáról vagy bevezetéséről.
4. **Ellenőrzési szempontokat**: hogyan lehet majd mérni, hogy a kockázatcsökkentő intézkedések hatékonyak-e. Például: "A TLS-kapcsolatok 95%-a PQC-hibrid módot használ a következő negyedév végére."
5. **Mérési eredményeket** (ha vannak): ha a résztvevő valódi rendszeren tesztelt, akkor a mért és a várt értékeket külön dokumentálja. A laborban általában nincs valódi kvantumhardver, ezért a várt értékek becslésen alapulnak, és ezt egyértelműen jelölni kell.

31.4.12 Módszertan

1. Töltse ki a kontextusleírást.
2. Készítse el a kriptográfiai leltárt (táblázatban): komponens, algoritmus, kulcsméret, adat érzékenység, élettartam.
3. Minden sorhoz rendeljen kockázati pontszámot (pl. 1–5 skálán a valószínűség és hatás szorzataként).
4. Rangsorolja a kockázatokat, és határozza meg az elsődleges intézkedéseket.
5. Írja meg a döntési naplót a legfontosabb döntésekről.
6. Fogalmazza meg az ellenőrzési szempontokat és a mérési tervet.

31.4.13 Ellenőrzési szempontok a labor értékeléséhez

- A dokumentum tartalmazza-e a kontextust és a leltárt?
- A kockázati besorolás szempontjai explicitek-e?
- A döntési napló követi-e az ADR sablont (kontextus, döntés, alternatívák, következmények, állapot)?
- A mért és várt eredmények elkülönülnek-e?
- A javasolt intézkedések figyelembe veszik-e a kriptográfiai agilitást?
- A dokumentum reprodukálható-e: egy másik személy meg tudná-e ismételni a felmérést?

31.4.14 Visszaállítási út

Mivel a labor nem jár éles rendszer módosításával, visszaállításra nincs szükség. Ha a résztvevő mégis tesztkörnyezetben PQC-t konfigurál, a tesztkörnyezetet a kiinduló állapotba kell visszaállítani.

31.4.15 9. Önellenőrző kérdések és gyakorlat

31.4.16 Gyakorlat

Képzelje el, hogy egy pénzügyi szolgáltató rendszere jelenleg RSA-2048-at használ a digitális aláírásokhoz és ECDHE-t a kulcsmegállapodáshoz. A rendszer 10 évig tárol ügyféladatokat. Készítsen egy rövid kockázati elemzést:

- Melyik algoritmus fenyegetettebb a Shor-algoritmus által?
- Mit javasolna: azonnali PQC migrációt vagy fokozatos átállást? Indokolja a választ a kriptográfiai agilitás elvével.
- Fogalmazzon meg egy quality attribute scenario-t a migráció rendelkezésre állására.

31.4.17 Ellenőrzőlista a fejezethez

- ☐ Megértettem a qubit, szuperpozíció és összefonódás alapfogalmait, és azok korlátait.
- ☐ Tisztában vagyok a Shor- és Grover-algoritmusok kriptográfiai hatásával.
- ☐ Ismerem a NISQ korszak főbb korlátait (zaj, dekoherencia, qubit-kapcsoltság).
- ☐ El tudom magyarázni a logikai qubit és a felületi kód szerepét.
- ☐ Le tudom írni a hibrid kvantum-klasszikus architektúra komponenseit.
- ☐ Tisztában vagyok a posztkvantum-kriptográfia alapelveivel és a NIST szabványosítás főbb eredményeivel.
- ☐ El tudom készíteni egy rendszer kvantumhatás-kockázati listáját.
- ☐ Felismerem a PQC migráció trade-offjait (teljesítmény, kompatibilitás, agilitás).

31.4.18 Önellenőrző kérdés

Miért nem elegendő a szimmetrikus kulcsméret megduplázása (pl. AES-128-ról AES-256-ra) a kvantumfenyegetés kezelésére, és miért van szükség posztkvantum aszimmetrikus algoritmusokra?

A válasz arra mutat rá, hogy a Grover-algoritmus a szimmetrikus kulcsok ellen csak négyzetgyökös gyorsítást ad, így a kulcsméret növelése hatékony. Ezzel szemben a Shor-algoritmus az RSA és ECC mögötti matematikai problémákat polinomiális időben oldja meg, így a kulcsméret növelése nem nyújt arányos védelmet; a megoldás a más típusú, kvantumbiztos aszimmetrikus algoritmusok bevezetése.

32 Záróprojekt: end-to-end architektúra

32.1 Követelmények, kényszerek és minőségi forgatókönyvek

32.1.1 1. Bevezetés a záróprojekt követelményrögzítésébe

A záróprojekt akkor lesz sikeres, ha az első lépésben nem ugrunk azonnal a technológiára, hanem rögzítjük, mit kell építeni, milyen minőségi célokkal, milyen korlátok között, és mit tekintünk sikernek. Ez a fejezet ezt a megalapozó munkát tanítja meg. Nem ismétli a korábbi fejezetekben bemutatott követelményelemzési alapokat, hanem azokat a záróprojekt kontextusában alkalmazza. A cél az, hogy a hallgató képes legyen egy reprodukálható, döntéskövető dokumentumot készíteni, amely a projekt chartert és a minőségi forgatókönyveket egyaránt tartalmazza.

A záróprojekt során egy fiktív, de realisztikus rendszert tervezünk. Ebben a fejezetben bevezetjük a **Rendelés- és Készletkezelő Platform** (továbbiakban: **Platform**) példáját, amelyet a későbbi fejezetekben is használunk az architektúra-alternatívák értékeléséhez, a PoC-hoz és a review-hoz. A Platform követelményeinek rögzítése közben bemutatjuk a funkcionális és nem funkcionális követelmények elkülönítését, a kényszerek kezelését, a kezdeti kockázati listát és a siker kritériumokat. Külön hangsúlyt fektetünk a quality attribute scenario (QAS) pontos megfogalmazására, mert ez lesz az architektúraértékelés alapja.

A fejezet végére az olvasó képes lesz: - projekt chartert készíteni, amely a rendszerhatárt, a stakeholdereket, a funkcionális és nem funkcionális követelményeket, a kényszereket és a siker kritériumokat egyértelműen tartalmazza; - a minőségi célokat QAS formában rögzíteni (inger, környezet, válasz, mérőszám); - megkülönböztetni a mért és a várt eredményt, és a mérési bizonytalanságot jelölni; - felismerni a tipikus hibákat, mint a homályos NFR-ek, a kényszer és a minőségi cél összekeverése, valamint a mérőszám nélküli forgatókönyvek.

32.1.2 2. A példa projekt rövid bemutatása

A záróprojekt tárgya a **Platform**, egy fiktív rendelés- és készletkezelő rendszer, amely egy kiskereskedelmi vállalat belső és külső felhasználóit szolgálja ki. A rendszer nem létezik éles környezetben; célja, hogy a hallgatók egy teljes architektúrát tervezzenek, dokumentáljanak és értékeljenek. A Platform funkciói közé tartozik a vevői rendelések felvétele, a raktárkészlet nyilvántartása, a szállítói megrendelések kezelése, valamint a visszáru folyamat. A rendszernek webes felhasználói felülettel, mobilalkalmazással és külső partnerek számára elérhető API-val kell rendelkeznie.

A Platformot egy fiktív közepes méretű vállalat üzemelteti, amelynek csapata három fejlesztői csoportból áll: a rendelési csapat, a készlet csapat és a platform csapat. A szervezeti felépítés befolyásolja a későbbi architektúrális döntéseket, ezért már itt rögzítjük. A rendszer célja, hogy napi átlagosan 10 000 rendelést kezeljen, csúcsidőben (például akciós időszakban) pedig akár 50 000 rendelést is naponta. A terhelés nem egyenletes: a hétvégi és szezonális csúcsok jelentős eltérést mutatnak.

A Platform fejlesztése során a következő stakeholderek érintettek: - **Vevői felhasználók**: rendeléseket adnak le, nyomon követik a szállítást, visszárut kezdeményeznek. - **Raktárvezetők**: figyelik a készletszinteket, szállítói rendeléseket indítanak. - **Pénzügyi osztály**: számlázási adatokat igényel, a rendelési adatok integritását ellenőrzi. - **Üzemeltetési csapat**: a rendszer rendelkezésre állásáért és megfigyelhetőségéért felel. - **Adatvédelmi tisztviselő**: a GDPR-megfelelésért felel, mivel a rendszer személyes adatokat kezel. - **Fejlesztői csapatok**: a rendszer építéséért, teszteléséért és karbantartásáért felelősek.

Ezeket a szereplőket a C4 context diagramon is feltüntetjük, ahogyan a dokumentációs szerződés előírja. A context diagram mutatja a rendszerhatárt, a felhasználói szerepköröket és a külső rendszereket, például a fizetési szolgáltatót, a szállítói partnert és az e-mail szolgáltatót.

```
flowchart LR
    subgraph Platform[Rendelés- és Készletkezelő Platform]
        direction TB
        Web[Webes felület]
        Mobile[Mobilalkalmazás]
        API[Partner API]
    end

    Vevő[Vevői felhasználó] --> Web
    Vevő --> Mobile
    Raktár[Raktárvezető] --> Web
    Pénzügy[Pénzügyi osztály] --> API
    Fizetés[Fizetési szolgáltató] <--> API
    Szállító[SZállítói partner] <--> API
    Email[E-mail szolgáltató] <--> API
```

A diagram a C4 context szintnek felel meg: a Platform mint egyetlen rendszer, a külső szereplők és rendszerek a határon kívül. A későbbi fejezetekben ezt a nézetet finomítjuk konténer és komponens szintre.

32.1.3 3. Funkcionális követelmények rögzítése

A funkcionális követelmények leírják, mit csinál a rendszer. A záróprojektben ezeket user story-k és use case-ek formájában rögzítjük, hogy a fejlesztői csapat és az üzleti stakeholderek közös nyelvet beszéljenek. A user story-kat a következő formátumban adjuk meg:

> **Mint** [szerep], **szeretnék** [funkció], **hogy** [üzleti érték].

A Platform esetében néhány példa:

- **Mint** vevői felhasználó, **szeretnék** rendelést leadni a webes felületen, **hogy** a kiválasztott termékeket megvásárolhassam.
- **Mint** raktárvezető, **szeretnék** készletriasztást kapni, ha egy termék készlete egy küszöbérték alá csökken, **hogy** időben szállítói rendelést tudjak indítani.
- **Mint** pénzügyi osztály, **szeretnék** napi rendelési összesítőt exportálni, **hogy** a bevételeket ellenőrizhessük.

A user story-k mellett a kritikus folyamatokat use case diagrammal vagy folyamatábrával is leírhatjuk. A funkcionális követelményeket egy prioritizált listába rendezzük, megjelölve a minimálisan életképes termék (MVP) körét. A Platform esetében az MVP a következőket tartalmazza:

1. Vevői regisztráció és bejelentkezés (jelszóval, kétfaktoros hitelesítéssel).
2. Termékkatalógus böngészése és keresés.
3. Kosár kezelése és rendelés leadása.
4. Fizetési szolgáltató integráció (valós tranzakció nélkül, sandbox módban).
5. Készletnyilvántartás és készletváltozások rögzítése.
6. Alapvető rendelési státusz követés (feldolgozás alatt, kiszállítva, visszárva).

A nem MVP funkciók közé tartozik a szállítói portál, a fejlett analitika, a gépi tanulás alapú kereslet-előrejelzés és a mobilalkalmazás teljes funkcionalitása. A funkcionális követelmények rögzítésekor fel kell tüntetni a következőket:

- **Azonosító**: egyedi kód (például FR-001).
- **Leírás**: egyértelmű, tesztelhető megfogalmazás.
- **Stakeholder**: ki igényli.
- **Prioritás**: MoSCoW (Must have, Should have, Could have, Won't have).
- **Elfogadási kritérium**: hogyan ellenőrizzük a teljesülést.

Példa:

Azonosító	Leírás	Stakeholder	Prioritás	Elfogadási kritérium
FR-001	A vevő tudjon regisztrálni e-mail címmel és jelszóval.	Vevői felhasználó	Must have	Sikeres regisztráció után a felhasználó be tud jelentkezni.
FR-005	A raktárvezető látja az aktuális készletszintet termékenként.	Raktárvezető	Must have	A készletoldal 5 másodpercen belül megjeleníti az adatokat, legfeljebb 1000 termék esetén.
FR-012	A pénzügyi osztály exportálja a napi rendelési összesítőt CSV formátumban.	Pénzügyi osztály	Should have	Az export 10 000 rendelés esetén 2 percen belül elkészül.

A funkcionális követelményeket a rendszerhatárral együtt kell értelmezni: ami a Platformon belül van, azért a Platform felelős; ami kívül van, az integráció vagy külső szolgáltatás. A context diagramon jelölt külső rendszerek (fizetési szolgáltató, szállítói partner, e-mail szolgáltató) interfészeit szerződésekben rögzítjük, de ezek nem részei a Platformnak.

32.1.4 4. Nem funkcionális követelmények és minőségi forgatókönyvek

A nem funkcionális követelmények (NFR-ek) írják le, milyen minőségben kell a rendszernek működnie. A záróprojektben ezeket mindig **quality attribute scenario** (QAS) formában fogalmazzuk meg, mert így ellenőrizhetők és mérhetők. A QAS négy kötelező elemet tartalmaz:

1. **Inger** (stimulus): mi történik, mi váltja ki a rendszer viselkedését.
2. **Környezet** (environment): milyen állapotban van a rendszer az inger bekövetkezésekor.
3. **Válasz** (response): hogyan reagál a rendszer.
4. **Mérőszám** (measure): milyen értékkel, mértékegységgel és mérési ablakkal mérjük a választ.

A QAS-ok a minőségi attribútumokhoz kötődnek, mint a teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság, skálázhatóság. A Platform esetében a legfontosabb minőségi attribútumok a következők:

- **Rendelkezésre állás**: a rendszer a tervezett üzemidőben elérhető legyen.
- **Teljesítmény**: a felhasználói műveletek válaszideje korlátos legyen.
- **Skálázhatóság**: a rendszer csúcsterhelés alatt is képes legyen kiszolgálni a kéréseket.
- **Biztonság**: a személyes adatok védelme, a hozzáférés-ellenőrzés és a titokkezelés.
- **Karbantarthatóság**: a fejlesztői csapat képes legyen a rendszert módosítani anélkül, hogy a többi funkció sérülne.
- **Adatkonzisztencia**: a rendelési és készletadatok integritása.

Minden minőségi attribútumhoz legalább egy QAS-t készítünk. Íme néhány példa a Platformra.

Teljesítmény QAS (rendelés leadása): - **Inger**: egy vevő elküld egy érvényes rendelést a kosárból. - **Környezet**: a rendszer normál terhelés alatt fut, legfeljebb 100 egyidejű felhasználóval. - **Válasz**: a rendszer a rendelést rögzíti, visszaigazolást küld, és a készletet frissíti. - **Mérőszám**: a rendelés leadási kérés átlagos válaszideje legfeljebb 2 másodperc (95. percentilis), mérési ablak 1 óra.

Rendelkezésre állás QAS (üzleti órák): - **Inger**: egy felhasználó bejelentkezik a webes felületre. - **Környezet**: a rendszer a havi csúcsterhelés alatt fut, de nem betervezett karbantartás nélkül. - **Válasz**: a rendszer a bejelentkezést sikeresen feldolgozza. - **Mérőszám**: a havi rendelkezésre állás legalább 99,9% az üzleti órákban (07:00–22:00 CET), a mérési ablak egy naptári hónap. A rendelkezésre állás képlete: A = üzemidő / (üzemidő + állásidő).

Skálázhatóság QAS (csúcsterhelés): - **Inger**: egy akciós kampány során a rendelési kérelmek száma a normál érték tízszeresére nő. - **Környezet**: a rendszer a napi csúcsidőszakban fut, a horizontális skálázási mechanizmus aktív. - **Válasz**: a rendszer további erőforrásokat von be, és a kéréseket továbbra is teljesíti. - **Mérőszám**: a rendszer 10 perc alatt legalább 5000 párhuzamos rendelési kérést képes kiszolgálni úgy, hogy a 95. percentilis válaszidő nem haladja meg az 5 másodpercet, mérési ablak 10 perc.

Biztonság QAS (jogosulatlan hozzáférés): - **Inger**: egy támadó megpróbál illetéktelenül hozzáférni egy másik felhasználó rendelési adataihoz. - **Környezet**: a rendszer normál működés közben, egy érvénytelen hozzáférési tokennel rendelkező kérés érkezik. - **Válasz**: a rendszer a kérést elutasítja, és nem ad ki adatot. - **Mérőszám**: minden jogosulatlan hozzáférési kísérlet 100%-ban elutasításra kerül, és a kísérlet 1 másodpercen belül naplózásra kerül.

Karbantarthatóság QAS (új funkció bevezetése): - **Inger**: a fejlesztői csapat új szállítási módot vezet be a rendelési folyamatban. - **Környezet**: a rendszer éles környezetben fut, párhuzamosan a meglévő szállítási módokkal. - **Válasz**: a változtatás bevezetése nem okoz leállást, és a meglévő funkciók változatlanul működnek. - **Mérőszám**: a bevezetés legfeljebb

2 órás tesztelési időt igényel, és a meglévő rendelési funkciók 100%-ban sikeresen teljesítik a regressziós teszteket.

A QAS-ok mellett megadjuk a **SLO-kat** (Service Level Objectives), amelyek a szolgáltatási szint céljait rögzítik, valamint a hozzájuk tartozó **SLI-kat** (Service Level Indicators), amelyek a méréshez használt mutatók. A rendelkezésre állás SLO-ja például 99,9%, az SLI a sikeres HTTP-válaszok aránya. Az error budget az SLO-ból számítható: ha a havi rendelkezésre állási SLO 99,9%, akkor az error budget az adott hónapban 0,1%, azaz legfeljebb 43,2 perc állásidő engedhető meg (30 napos hónap esetén).

A mérési bizonytalanságot is jelezni kell: a QAS-okban megadott mérőszámok várt értékek, amelyeket a PoC és a terhelési tesztek során mérünk, de a becsült értékeket meg kell különböztetni a mért értékektől. A fejezet laborjában ezt a szétválasztást gyakoroljuk.

32.1.5 5. Kényszerek rögzítése

A kényszerek olyan adott feltételek, amelyeket az architektúrának figyelembe kell vennie, de amelyeket nem lehet trade-off tárgyává tenni. A Platform esetében a következő kényszereket azonosítjuk:

- **Technológiai kényszer**: a rendszernek Kubernetes-alapú konténerplatformon kell futnia, mert a vállalat üzemeltetési csapata ezt a környezetet támogatja. (A konkrét Kubernetes-verziót az adott környezetben ellenőrizni kell, mert a stabilizálások változhatnak.)
- **Jogi kényszer**: a személyes adatokat az EU-n belül kell tárolni, és a GDPR előírásait be kell tartani. A szabályozási hivatkozásokat mindig a hivatalos szöveg szerint kell ellenőrizni.
- **Pénzügyi kényszer**: az infrastruktúra havi költsége nem haladhatja meg a 10 000 eurót (fiktív keret, helyőrző érték).
- **Szervezeti kényszer**: a fejlesztői csapatok nem rendelkeznek dedikált adatbázis-adminisztrátorral, ezért a perzisztencia-rétegnek felügyelt szolgáltatást kell használnia.
- **Időbeli kényszer**: az MVP-t 6 hónapon belül ki kell adni, ezért az architektúra nem lehet túlbonyolított.

A kényszereket a dokumentációban külön szakaszban rögzítjük, és megjelöljük, hogy a kényszer milyen forrásból származik (például jogi osztály, üzemeltetési csapat, költségvetés). A kényszerek nem azonosak a minőségi célokkal: egy kényszer megszabja, mit nem lehet megtenni, míg egy minőségi cél azt mondja meg, milyen szintet kell elérni. A gyakori hiba a kettő összekeverése: például a “99,9% rendelkezésre állás” minőségi cél, nem kényszer, mert tárgyalható, míg a “Kubernetes használata kötelező” kényszer, mert nem tárgyalható.

32.1.6 6. Kezdeti kockázati lista

A követelmények rögzítésekor már azonosíthatunk olyan kockázatokat, amelyek befolyásolhatják az architektúra sikerét. A kockázatokat egy egyszerű kockázati mátrixban rögzítjük, feltüntetve a valószínűséget és a hatást. A Platform esetében a kezdeti kockázati lista:

Kockázat	Valószínűség	Hatás	Megjegyzés
A csúcsterhelés alatt a rendelésfeldolgozás túlterhelődik, és a rendelések elvesznek.	Magas	Súlyos	A QAS-okban rögzített skálázhatósági követelményekkel és terhelési tesztekkel kezeljük.
A fizetési szolgáltató integráció nem megfelelően kezeli a tranzakciós hibákat, ami pénzügyi veszteséget okoz.	Közepes	Súlyos	Idempotens kérések és kompenzáló tranzakciók szükségesek.
A GDPR-megfelelés hiányos, mert a személyes adatok naplózását nem korlátozzuk.	Közepes	Súlyos	Privacy by design és adatvédelmi hatásvizsgálat.
A készletadatok konzisztenciája sérül a párhuzamos rendelések miatt.	Magas	Közepes	Elosztott tranzakciók vagy eseményvezérelt outbox minta.
A csapatok közötti kommunikáció rossz, ami a szolgáltatáshatárok elmosódásához vezet.	Közepes	Közepes	DDD bounded context-ek és API-szerződések.

A kockázatokat a projekt előrehaladtával folyamatosan frissíteni kell, és az architektúradöntésekben (ADR-ekben) hivatkozni kell rájuk. A kezdeti kockázati lista nem teljes, de segít a tervezési folyamat fókuszálásában.

32.1.7 7. Siker kritériumok

A siker kritériumok azt határozzák meg, hogyan tudjuk megmondani, hogy a projekt sikeres volt-e. A kritériumoknak mérhetőknek, időhöz kötötteknek és a stakeholderek számára érthetőeknek kell lenniük. A Platform esetében a siker kritériumok:

1. **Funkcionális siker**: az MVP 6 hónapon belül éles környezetben elérhető, és legalább 50 valós felhasználói teszt sikeresen lead egy rendelést.
2. **Minőségi siker**: a havi rendelkezésre állás az éles indulást követő első hónapban legalább 99,5%, a második hónaptól 99,9%.
3. **Teljesítmény siker**: a rendelés leadási válaszidő a 95. percentilisben nem haladja meg a 2 másodpercet a normál terhelés alatt.
4. **Költség siker**: az infrastruktúra havi költsége a 10 000 eurós keret alatt marad.
5. **Szervezeti siker**: a három fejlesztői csapat képes önállóan kiadni a saját szolgáltatásait anélkül, hogy más csapatokat blokkolnának.

A siker kritériumokat a projekt charter részeként rögzítjük, és a későbbi architektúraértékelésnél (Architektúra-review, kockázat és bemutató) ezek alapján értékeljük a megoldást.

32.1.8 8. Gyakori hibák a követelményrögzítésben és azok javítása

A záróprojekt során a követelményrögzítés gyakori hibái a következők; mindegyiknél megadjuk a helyes megoldást.

Hiba 1: Homályos NFR-ek, mint „a rendszer legyen gyors”. A „gyors” nem mérhető. A helyes megoldás a QAS használata: meg kell adni az ingert, a környezetet, a választ és a mérőszámot. Például: „A rendelés leadási kérés átlagos válaszideje legfeljebb 2 másodperc (95. percentilis), normál terhelés alatt, 1 órás mérési ablakban.”

Hiba 2: A kényszerek és a minőségi célok összekeverése. Például valaki kényszerként írja le a 99,9%-os rendelkezésre állást. Ez nem kényszer, hanem tárgyalható minőségi cél, amely trade-offokkal módosítható. A helyes megoldás a kettő különválasztása: a kényszerek olyan adott feltételek, amelyeket nem vitatunk, a minőségi célok pedig SLO-k, amelyekhez error budget tartozik.

Hiba 3: A QAS-ból hiányzik a mérőszám vagy a mérési ablak. A „válaszidő legfeljebb 2 másodperc” nem elég, mert nem tudjuk, milyen terhelés alatt, milyen időtávon. A helyes megoldás a teljes QAS: környezet (normál terhelés, 100 egyidejű felhasználó) és mérési ablak (1 óra).

Hiba 4: A funkcionális követelményeket nem kötik elfogadási kritériumokhoz. A „a felhasználó leadhat rendelést” nem tesztelhető. A helyes megoldás az elfogadási kritérium megadása, például: „A felhasználó a kosárba tett termékekkel a ‘Rendelés leadása’ gombra kattintva sikeres visszaigazolást kap.”

Hiba 5: A stakeholderek hiányoznak a követelményrögzítésből. Ha nem tudjuk, kinek készül a rendszer, akkor a követelmények félreérthetők lesznek. A helyes megoldás a stakeholder térkép és a C4 context diagram elkészítése a rendszerhatárral együtt.

Hiba 6: A kockázatokat nem rögzítik, vagy túl általánosan. A „biztonsági kockázat” nem használható. A helyes megoldás a konkrét kockázat leírása, például: „A fizetési szolgáltató hibája esetén a rendelés elveszhet, ha nincs idempotens újrapróbálkozás.”

Hiba 7: A siker kritériumok nem mérhetők. A „a rendszer legyen sikeres” nem értékelhető. A helyes megoldás a konkrét, mérhető, időhöz kötött kritérium, például: „Az MVP 6 hónapon belül élesbe kerül, és 50 valós felhasználói teszt sikeresen lead egy rendelést.”

Ezek a hibák a záróprojekt elején jellemzően előfordulnak; a labor során a hallgatók saját maguk is ellenőrzik a dokumentumukat ezek alapján.

32.1.9 9. Labor: projekt charter és QAS készítése

Cél: A hallgató készítsen egy projekt chartert és legalább három quality attribute scenario-t a Platformra (vagy saját záróprojekt-témájára). A dokumentum legyen reprodukálható, verziózott, és tartalmazzon külön szakaszt a mért és a várt eredményeknek.

Kiinduló állapot: A hallgató ismeri a Platform példáját, és hozzáfér a fejezetben bemutatott QAS-sablonhoz. Nincs szükség éles rendszerre; minden adat fiktív. A labor izolált környezetben, helyi dokumentumszerkesztőben vagy jegyzetfüzetben végezhető.

Elvárt eredmény: 1. Egy verziózott dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza: - a projekt nevét és rövid leírását; - a stakeholderek listáját; - a C4 context szintű diagramot diagrams-as-code formában (például Mermaid); - legalább 5 funkcionális követelményt, azonosítóval, prioritással és elfogadási kritériummal; - legalább 3 QAS-t, a négy kötelező elemmel és mérési ablakkal; - a kényszerek listáját; - a kezdeti kockázati listát; - a siker kritériumokat. 2. A dokumentumban külön szakaszban szerepeljen a „Mért eredmények” (jelen esetben üres, hiszen még nem mértünk semmit) és a „Várt eredmények” (a QAS-okban megadott célértékek). A mért és várt eredményeket egyértelműen el kell választani, és a mérési bizonytalanságot jelölni kell.

Ellenőrzési szempontok: - Minden QAS tartalmazza az ingert, a környezetet, a választ és a mérőszámot; a mérőszám mértékegységgel és mérési ablakkal rendelkezik. - A kényszerek nincsenek összekeverve a minőségi célokkal. - A funkcionális követelmények mindegyike rendelkezik elfogadási kritériummal. - A dokumentum verziózott, és a döntési naplóhoz (ADR) hivatkozik, ha van korábbi döntés. - A diagram a rendszerhatárt mutatja, és a külső rendszerek a context szinten szerepelnek. - A dokumentum nem tartalmaz valós személyes adatot vagy hitelesítő adatot.

Visszaállítási út: Mivel a labor tisztán dokumentációs jellegű, nincs szükség visszaállításra. Ha a hallgató valós példát használ, akkor a fiktív adatokat kell használnia.

Tipp: A labor során használjuk a fejezetben bemutatott QAS-sablont, és ellenőrizzük a gyakori hibák listáját. A dokumentumot a következő fejezetben (Architektúra-alternatívák és döntési dokumentáció) fogjuk továbbfejleszteni az architektúra-alternatívák értékelésével.

32.1.10 10. Ellenőrzőlista

A fejezet anyagának feldolgozása után ellenőrizze az alábbiakat:

- ☐ Tudom, mi a különbség a funkcionális és a nem funkcionális követelmények között.
- ☐ Tudom, hogyan kell egy QAS-t felépíteni (inger, környezet, válasz, mérőszám).
- ☐ Tudom, mi a különbség a kényszer és a minőségi cél között.
- ☐ Tudom, hogyan kell a rendszerhatárt és a stakeholdereket C4 context szinten ábrázolni.
- ☐ Tudom, hogyan kell a mérőszámokat mértékegységgel és mérési ablakkal megadni.
- ☐ Felismerem a homályos NFR-eket, és át tudom alakítani őket QAS-sá.
- ☐ El tudom készíteni a projekt chartert a Platformra a laborban leírtak szerint.

32.1.11 11. Önellenőrző kérdés

Adott a következő NFR: „A rendszer legyen biztonságos.” Miért nem elfogadható ez a megfogalmazás? Írja át egy teljes QAS-ra, amelyben megadja az ingert, a környezetet, a választ és a mérőszámot a mértékegységgel és a mérési ablakkal együtt.

Várható megoldás: A „biztonságos” nem mérhető, és nem köthető konkrét ingerhez. Egy helyes QAS például: „Inger: egy támadó brute force támadást indít a bejelentkezési felületen. Környezet: a rendszer normál működés közben, nyilvános internetről elérhető. Válasz: a rendszer 5 sikertelen próbálkozás után 15 percre zárolja a fiókot, és naplózza az eseményt. Mérőszám: a zárolás 1 másodpercen belül aktiválódik, és az esemény 100%-ban naplózásra kerül, mérési ablak 1 óra.”

32.1.12 12. Összefoglalás és átvezetés

Ebben a fejezetben rögzítettük a záróprojekt követelményeit és minőségi céljait a Platform példáján. Megtanultuk a funkcionális követelmények strukturált leírását, a QAS-ok pontos megfogalmazását, a kényszerek és a siker kritériumok kezelését, valamint a kezdeti kockázati lista elkészítését. A labor során a hallgató elkészítette a saját projekt charterét.

A következő fejezet (Architektúra-alternatívák és döntési dokumentáció) erre az alapra épít: az architektúra-alternatívák feltárását és a döntési dokumentációt (ADR-ek) fogja bemutatni. A fejezetben rögzített QAS-ok lesznek azok a fitness function-ök, amelyekkel a későbbi architektúraértékelés során mérjük a megoldásokat. A projekt charter és a QAS-ok nélkül nem lehet megalapozottan dönteni az architektúrális stílusok között.

32.2 Architektúra-alternatívák és döntési dokumentáció

32.2.1 1. Bevezetés: miért döntési dokumentáció az architektúra?

Az előző egységben rögzítetted a záróprojekt követelményeit, kényszereit és minőségi forgatókönyveit. Most a kérdés az, hogyan jutsz el a követelményektől egy védhető architektúra felé. A válasz ritkán egyetlen „helyes” megoldás. Az architektúra-alternatívák összehasonlítása és dokumentálása azt a folyamatot jelenti, amelyben több lehetséges szerkezeti megoldást definiálsz, egységes szempontrendszer szerint értékeled őket, majd a döntést és annak okát visszakereshető módon rögzíted.

Ebben a fejezetben a cél nem a döntés „megnyerése”, hanem a döntési tér feltárása, a trade-offok láthatóvá tétele és a kockázatok azonosítása. A döntési dokumentáció akkor jó, ha egy későbbi olvasó – aki nem volt jelen a vitán – megérti belőle:

- milyen alternatívákat vizsgáltunk,
- milyen szempontok mentén hasonlítottuk össze őket,
- miért a választott megoldás mellett döntöttünk,
- milyen következményekkel és kockázatokkal jár a döntés,
- és milyen feltételek mellett érdemes újra megnyitni a kérdést.

A fejezetben a fogalmakat a záróprojekt egy fiktív, de valósághű részletén mutatjuk be: egy rendelésfeldolgozó szolgáltatás bővítésén, ahol a csapat a monolitikus és az eseményvezérelt megoldás között mérlegel.

32.2.2 2. Alternatívák kialakítása: stílus, minta és technológia

Az első hiba, amit sok csapat elkövet, hogy rögtön technológiákban gondolkodik: „Kafka vagy RabbitMQ?", „Postgres vagy MongoDB?" – mielőtt tisztázná, milyen architekturális stílus és milyen minták illeszkednek a problémához. A három szintet érdemes elkülöníteni.

32.2.2.1 2.1. Architektúrális stílus

A stílus a rendszer legmagasabb szintű szerkezeti elrendezését írja le. Például:

- rétegezett architektúra,
- moduláris monolit,
- mikroszolgáltatások,
- eseményvezérelt architektúra,
- pipeline vagy streaming,
- space-based vagy elosztott cache-re épülő architektúra.

A stílus választása határozza meg, hogy a komponensek hogyan oszlanak meg, hogyan kommunikálnak, és milyen üzemeltetési modellre lesz szükség.

32.2.2.2 2.2. Architektúrális minta

A minta egy adott stíluson belül ismétlődő megoldás egy konkrét problémára. Például:

- CQRS: olvasási és írási modell szétválasztása,
- Event Sourcing: állapotváltozások eseménysorként való tárolása,
- Saga: elosztott tranzakciók koordinálása,
- Outbox: üzenetküldés és adatbázis-tranzakció atomikus összekapcsolása,
- API gateway, BFF, circuit breaker, bulkhead.

A minták nem önmagukban jók vagy rosszak; akkor válnak hasznossá, ha a mögöttük álló probléma valóban jelen van.

32.2.2.3 2.3. Technológia

A technológia a konkrét implementációs eszköz: Kafka, RabbitMQ, Postgres, MySQL, Kubernetes, AWS Lambda, Spring Boot, FastAPI, vitest, OpenTelemetry. A technológiát akkor érdemes kiválasztani, ha már tudjuk, milyen stílust és mintákat követünk. A sorrend megfordítása – előbb a technológia, azután a stílus – gyakori oka a lock-innek és a rossz illeszkedésnek.

32.2.2.4 2.4. Döntési tér és opciók

Az alternatívák kialakításához először definiálnod kell a döntési teret. A döntési tér azoknak a szerkezeti kérdéseknek a halmaza, amelyekre a záróprojektben választ kell adni. Például:

- Hol húzódnak a rendszerhatárok és a modulhatárok?
- Melyik komponens birtokolja a rendelésállapotot?
- Hogyan kommunikálnak a komponensek szinkron vagy aszinkron módon?
- Hogyan kezeljük a tranzakciókat a határokon átnyúló folyamatokban?
- Hogyan skálázzuk a nagy terhelésű olvasási útvonalat?
- Hogyan biztosítjuk a visszamenőleges kompatibilitást?

Egy alternatíva ezekre a kérdésekre adott, egymással összhangban lévő válaszok együttese. Ha csak egy-egy kérdést vizsgálsz elszigetelten, könnyen olyan kombináció jön létre, amelynek részei ütköznek. Például a szinkron REST hívásokra épülő rendelésfeldolgozás és a szigorúan aszinkron üzenetterhelés nem alkot konzisztens alternatívát, ha ugyanazt a folyamatot kevered.

32.2.3 3. Összehasonlítási keret

Az alternatívákat egységes szempontrendszer alapján kell értékelni. A leggyakoribb dimenziók a következők.

32.2.3.1 3.1. Minőségi attribútumok

A Követelmények, kényszerek és minőségi forgatókönyvek egységben rögzített minőségi forgatókönyvek adják az értékelés gerincét. Minden alternatívát ugyanazokhoz a forgatókönyvekhez kell mérni. Például:

> **Forgatókönyv:** Amikor a rendelésfeldolgozó szolgáltatás 500 egyidejű megrendelést fogad egy kampány alatt, a rendszernek a kérések 95%-át 2 másodpercen belül kell feldolgoznia, miközben a rendelésállapot konzisztens marad.

Ezt a forgatókönyvet a monolit, a mikroszolgáltatás és az eseményvezérelt alternatíva más-más módon teljesítheti. A pontozás során nem elég a „jó” vagy „rossz” címke; a várható viselkedést és a mérési módot is rögzíteni kell.

32.2.3.2 3.2. Kényszerek

A kényszerek olyan feltételek, amelyeket nem lehet pontozni, mert kötelezőek. Például:

- a szervezet csak Kubernetesen üzemeltet konténerizált szolgáltatásokat;
- a csapat Java és TypeScript ismeretekkel rendelkezik;
- a rendszer csak az EU-n belüli adatközpontokban futhat;
- a rendelkezésre álló költségkeret havi 200 000 EUR;
- a rendszernek 99,9%-os havi rendelkezésre állást kell elérnie, ami $A = \frac{\text{üzemidő}}{\text{üzemidő}+\text{állásidő}} \geq 0{,}999$, azaz havi legfeljebb 43,8 perc állásidőt enged.

Ha egy alternatíva megsért egy kényszert, akkor nem pontozzuk tovább – vagy csak akkor, ha a kényszert külön tárgyaljuk a stakeholderrel.

32.2.3.3 3.3. Költség, csapat és üzemeltetés

A technológia és a stílus választása nem csak futásidőben hat. A mikroszolgáltatások például nagyobb üzemeltetési terhelést és több automatizálást igényelnek; a moduláris monolit egyszerűbb kezdetben, de a szervezet növekedésével új határokat kell kialakítani. A csapat mérete, tapasztalata, a rendelkezésre álló SRE-kapacitás és a deploy gyakorisága mind összehasonlítási dimenzió lehet.

32.2.3.4 3.4. Döntési mátrix felépítése

A döntési mátrixban az alternatívák a sorok, az értékelési dimenziók az oszlopok. Minden cellában egy pontszám és indoklás szerepel. A súlyozás nélküli összegzés félrevezető lehet, mert nem minden minőségi attribútum egyformán fontos. A súlyozott pontszámot a következő képlet adja meg:

$$S_i = \sum_{j=1}^{n} w_j \cdot v_{i,j}$$

ahol S_i az i-edik alternatíva súlyozott pontszáma, w_j a j-edik dimenzió súlya, $v_{i,j}$ pedig az i-edik alternatíva értékelése a j-edik dimenzióban. A súlyok összege legyen 1, vagy normalizáld őket.

A pontozás önkényes lehet, ha nem kötöd méréshez vagy érveléshez. Ezért minden cellához rövid indoklást kell írni, és jelölni kell a bizonytalanságot. Például a „teljesítmény 4/5” önmagában nem elég; az indoklás legyen: „A moduláris monolit egyetlen folyamatban tartja a rendelésfeldolgozási útvonalat, így nincs hálózati sorosítási költség; a forgatókönyv várhatóan 300–600 ms válaszidővel teljesíthető. A mérés még nem történt meg.”

32.2.4 4. A döntési mátrix értelmezése

A döntési mátrix nem arról szól, hogy a legmagasabb pontszámú alternatíva automatikusan nyer. Sokkal fontosabb, hogy a mátrix megmutassa a trade-offokat. Ha egy alternatíva a teljesítményben és az egyszerűségben vezet, de a skálázhatóságban és a csapatszintű párhuzamos fejlesztésben lemarad, akkor a döntés attól függ, hogy a záróprojekt követelményei melyiket helyezik előtérbe.

32.2.4.1 4.1. Példa: három alternatíva összehasonlítása

Tegyük fel, hogy a záróprojektben a rendelésfeldolgozó szolgáltatás kapcsán három alternatívát vizsgálunk:

A1 – Moduláris monolit, szinkron REST belső interfészekkel

A rendelés, számlázás és készletkezelés modulok egy folyamatban futnak; a modulok közötti hívások szinkron REST hívások, de a folyamat egyetlen tranzakcióban kezelhető. A deploy egyetlen konténerként történik.

A2 – Mikroszolgáltatások, szinkron REST + néhány aszinkron esemény

A rendelés, számlázás és készletkezelés külön szolgáltatások. A rendelésfelvétel szinkron, a számlázási és készletműveletek aszinkron eseményekkel indulnak. A tranzakciók saga mintával konzisztensek.

A3 – Eseményvezérelt architektúra, CQRS és Event Sourcing

A rendelésállapot eseménysorként tárolódik. A szolgáltatások eseményeken keresztül kommunikálnak, az olvasási modell a lekérdezési igényekre optimalizált. A tranzakciók az eseménynaplóból származtathatók, de a konzisztencia végső soron biztosított.

A döntési mátrixot a következő dimenziókkal tölthetjük ki, ahol a súlyok a záróprojekt Követelmények, kényszerek és minőségi forgatókönyvek alfejezetben rögzített prioritásait tükrözik:

Dimenzió (súly)	A1	A2	A3
Fejlesztési sebesség (0,25)	4	3	2
Skálázhatóság (0,20)	2	4	5
Üzemeltethetőség (0,20)	5	2	3
Konzisztencia és tranzakciók (0,20)	5	3	2
Költség (0,15)	4	2	2
Súlyozott pontszám	3,95	2,85	2,85

A táblázat azt mutatja, hogy az A1 a legmagasabb súlyozott pontszámot kapja, de a skálázhatóság és a párhuzamos fejlesztés terén kompromisszumot kell kötni. Ha a záróprojekt követelményei között a csapat növekedése és a nagy kampányok idején fellépő kiugró terhelés a legfontosabb, akkor az A2 vagy A3 jobb lehet annak ellenére, hogy a mátrix pontszáma alacsonyabb. A mátrix tehát vitaanyag, nem döntési automata.

32.2.4.2 4.2. Bizonytalanság a pontozásban

A pontozás során gyakran olyan értékek szerepelnek, amelyek nem mérésen alapulnak. Ezt jelölni kell. A mátrixban használhatsz például tartományokat: „3–4 (spike hiányában)”, vagy külön oszlopban tüntetheted fel a bizonyíték szintjét. A dokumentációban a mért értéket soha ne keverd a várt értékkel. Ha egy dimenzióban még nem végeztél PoC-t, írd be, hogy „várt, nem mért”.

32.2.5 5. ADR-ek: a döntés nyomon követése

Az architektúradöntéseket ADR-ekben (Architecture Decision Record) rögzítjük. Az ADR nem jegyzőkönyv, hanem tömör, célzott dokumentum, amely megmagyarázza egyetlen architektúradöntés okát. A könyv szerződése szerint minden ADR tartalmazza a következőket:

- **Cím és azonosító:** például `ADR-014: Rendelésfeldolgozás moduláris monolitban`.
- **Állapot:** `javasolt`, `elfogadott`, `elutasított`, `érvényét vesztette`, `felváltotta: ADR-xyz`.
- **Kontextus:** a döntést szükségessé tevő körülmények, követelmények és kényszerek.
- **Döntés:** a választott megoldás és annak lényegi elemei.
- **Alternatívák:** a vizsgált lehetőségek és azok elutasításának rövid indoklása.

- **Következmények:** pozitív és negatív hatások, technikai adósság, üzemeltetési teher, szükséges utómunkálatok.
- **Kapcsolódó elemek:** minőségi forgatókönyvek, döntési mátrix, diagramok, PoC-k.

Az ADR-ek száma nem cél. Egy ADR akkor jó, ha a jövőbeli olvasó megérti belőle a döntés okát és a feladott alternatívákat. Az ADR-eket a kódbázisban, például a `docs/adr/` mappában tároljuk Markdown formátumban, így verziókövethetők.

32.2.5.1 5.1. Példa ADR a záróprojektből

```
# ADR-014: Rendelésfeldolgozás moduláris monolitban

**Állapot:** elfogadott

**Dátum:** 2025-11-04

**Kontextus:**
A záróprojekt rendelésfeldolgozó szolgáltatása kampányok alatt 500
egyidejű megrendelést fogad. A csapat 6 fejlesztőből áll, kevés
üzemeltetési tapasztalattal. A sa122-ben rögzített minőségi
forgatókönyv szerint a kérések 95%-át 2 másodpercen belül kell
feldolgozni. A jövőbeni terhelés növekedése várható, de a következő
12 hónapban a jelenlegi csapatméret nem változik.

**Döntés:**
A rendelésfeldolgozást moduláris monolitként valósítjuk meg. A rendelés,
számlázás és készletkezelés modulok egyetlen deploy egységben futnak, a
modulok közötti kommunikáció szinkron REST interfészeken történik, de a
publikus API kifelé aszinkron eseményeket is képes kibocsátani az outbox
mintával. A modulok közötti tranzakciókat helyi adatbázis-tranzakció
biztosítja.

**Alternatívák:**
- **A2 - Mikroszolgáltatások, saga:** nagyobb üzemeltetési terhet és
  hálózati késleltetést okozna; a 6 fős csapat jelenleg nem tudná
  biztonságosan üzemeltetni. Elutasítva, de 12 hónap múlva újra
  értékelendő.
- **A3 - Eseményvezérelt CQRS/Event Sourcing:** erős skálázhatóságot
  adna, de a konzisztencia és a fejlesztési sebesség romlana, és a
  csapatnak nincs tapasztalata eseménytárolós rendszerekkel. Elutasítva.

**Következmények:**
- Pozitív: egyszerűbb deploy, egyetlen tranzakciós határ, alacsonyabb
  üzemeltetési költség.
- Negatív: a modulok közötti szoros kapcsolat miatt a párhuzamos
  fejlesztés később nehezebb lehet; a skálázás csak horizontálisan, a
  teljes monolit replikálásával oldható meg.
- Szükséges utómunka: a modulok közötti interfészeket szigorúan
  verziózni kell, hogy később külön szolgáltatásokra választhatók
  legyenek.

**Kapcsolódó elemek:**
- Minőségi forgatókönyv: QA-003
- Döntési mátrix: `docs/architecture/decision-matrix.md`
- C4 container diagram: `docs/architecture/container.md`
```

32.2.6 6. C4 diagramok a döntési dokumentációban

A döntési dokumentációban a C4 modell segít abban, hogy az alternatívákat és a választott architektúrát egyértelműen ábrázoljuk. A C4 szintjei közül a context, container és component szint kötelező; a code szint csak indokolt esetben készül.

32.2.6.1 6.1. System context szint

A context diagram a rendszerhatárt és a külső szereplőket mutatja. A záróprojektben ez például:

```
flowchart LR
    U[Ügyfél] -->|HTTPS| R[Rendeléskezelő platform]
    P[Partner raktár] -->|REST/HTTPS| R
    R -->|SMTP| E[E-mail szolgáltató]
    R -->|HTTPS| B[Banki fizetési kapu]
```

Ezen a szinten nincs szó belső komponensről. A rendszerhatárt a diagramon a keret jelöli, a külső szereplők kívül helyezkednek el.

32.2.6.2 6.2. Container szint

A container diagram a választott architektúra fő deploy egységeit mutatja. A moduláris monolit esetén:

```
flowchart LR
    U[Ügyfél] -->|HTTPS| W[Webes API konténer]
    W -->|SQL| D[(Rendelés DB)]
    W -->|publish| M[Message Broker]
    M -->|subscribe| S[Számlázó worker]
    S -->|SQL| D
```

A konténer neve mellett jeleznünk kell a felelősségét és interfészét is. Például:

- **Webes API konténer:** befogadja a rendelési kéréseket, validál, elvégzi a rendelési tranzakciót, és az outbox táblán keresztül eseményeket tesz közzé.
- **Rendelés DB:** a rendelés, számlázás és készlet modul tranzakciós állapotát tárolja.

A kapcsolatokon fel kell tüntetni a protokollt és a szinkronitást: `HTTPS/szinkron`, `SQL/szinkron`, `AMQP/aszinkron`. A hibatűrést is jelezhetjük: például `retry exponential backoff` az aszinkron kapcsolaton.

32.2.6.3 6.3. Component szint

A component diagram a `Webes API konténer` belsejét mutatja:

```
flowchart LR
    C[API Controller] -->|metódushívás| S[RendelésSzolgáltatás]
    S -->|metódushívás| I[KészletModul]
    S -->|metódushívás| Z[SzámlázásModul]
    S -->|tranzakció| R[(Rendelés Repository)]
    S -->|publish| O[Outbox Publisher]
```

Itt minden komponenshez tartozik egy felelősség és egy interfész. Például:

- **RendelésSzolgáltatás:** feldolgozza a rendelési parancsot, koordinálja a készlet- és számlázási műveleteket, és betartja a `RendelésFeldolgozás` interfészt.
- **Outbox Publisher:** a tranzakcióval együtt rögzített eseményeket aszinkron módon közzéteszi a broker felé.

A C4 diagramok a döntési dokumentáció részét képezik. Ha az alternatívák közül többet is ábrázolni kell, mindegyiket ugyanolyan szinten és jelöléssel készítsd el, hogy az összehasonlítás vizuálisan is követhető legyen.

32.2.7 7. Trade-off és kockázat dokumentálása

A döntési mátrix és az ADR a döntés időpontjában rögzíti a választást, de a trade-offok és a kockázatok folyamatosan változhatnak. Ezért a döntési dokumentációban külön részt kell szentelni a trade-offoknak és a kockázati regiszternek.

32.2.7.1 7.1. Trade-off napló

A trade-off napló azt mutatja meg, mit nyerünk és mit veszítünk a választott megoldással. A napló egy egyszerű táblázat lehet:

Trade-off	Előny	Hátrány	Kezelési stratégia
Moduláris monolit szinkron REST	Egyszerű tranzakció, kevesebb hálózati hiba	Szoros kapcsolat, nehezebb párhuzamos fejlesztés	Szigorú modulhatárok, interfész-verziózás, ADR felülvizsgálat 6 havonta
Outbox minta aszinkron eseményekhez	Atomikus tranzakció és üzenetküldés	Az üzenetfeldolgozás késleltetve történik	A fogyasztói oldalon idempotencia és retry

32.2.7.2 7.2. Kockázati regiszter

A kockázati regiszter a döntéssel járó bizonytalanságokat gyűjti össze. Minden kockázathoz tartozik:

- azonosító,
- leírás,
- bekövetkezési valószínűség (kvalitatív vagy kvantitatív),
- hatás,
- mitigáció,
- felelős,
- felülvizsgálati időpont.

Például:

Kockázat	Valószínűség	Hatás	Mitigáció
A rendelésfeldolgozó terhelése meghaladja a monolit horizontális skálázási képességét	Közepes	Magas	Terhelési PoC a Proof of Concept és mérés alfejezetben; szükség esetén a rendelésmodul kiszervezése
Az outbox publisher üzenetduplikátumot okoz	Alacsony	Közepes	Idempotens fogyasztók, deduplikáció az üzenetazonosító alapján
A moduláris monolit határok fellazulnak, csökken a kohézió	Közepes	Közepes	Architektúra-teszt fitness function, kódreview-k a modulhatárokon

A kockázati regiszter nem statikus dokumentum; a projekt előrehaladtával frissíteni kell, és a kockázatok lezárását is rögzíteni kell.

32.2.8 8. Tipikus hibák az alternatívák összehasonlítása során

32.2.8.1 8.1. Egyetlen alternatíva vizsgálata

Ha a csapat csak egy megoldást vizsgál, nem tudja megindokolni, miért jobb, mint a többi. A döntési dokumentációban legalább két, de ideális esetben három értelmes alternatívának kell szerepelnie. Az alternatívák ne legyenek szalmabábok: az elutasított opciók erős oldalait is ismerd el.

32.2.8.2 8.2. Technológia és minta összemosása

Gyakori hiba, hogy a csapat a Kafka kontra RabbitMQ vitát folytatja le, miközben a valódi döntés az lenne, hogy a rendelésfeldolgozás szinkron vagy aszinkron legyen. Előbb a stílust és a mintákat rögzítsd, azután válassz technológiát.

32.2.8.3 8.3. Pontozás indoklás nélkül

A döntési mátrix csakis akkor használható, ha a cellák indoklást tartalmaznak. A puszta számok vitát generálnak, és nem segítenek a jövőbeli olvasónak megérteni a döntés okát.

32.2.8.4 8.4. A kényszerek figyelmen kívül hagyása

Ha egy alternatíva sérti a jogi vagy üzemeltetési kényszereket, azt a mátrixban nem szabad pontozni. A kényszereket a döntési dokumentáció elején, elkülönítve kell felsorolni.

32.2.8.5 8.5. A mért és a várt eredmény összemosása

A döntési mátrixban gyakran olyan számok szerepelnek, amelyek még nem estek át mérésen. Ilyenkor a „várt” jelzőt kötelező feltüntetni. A Proof of Concept és mérés PoC feladata lesz ezeket a várt értékeket méréssel igazolni vagy cáfolni.

32.2.9 9. Labor: alternatívák összehasonlítása és döntési dokumentáció

Ez a labor a záróprojekt egy kis szeletére koncentrál. A cél, hogy a tanuló saját maga készítsen döntési dokumentációt: C4 diagramot, ADR-eket és döntési mátrixot.

32.2.9.1 9.1. Kiinduló állapot

Adott a Követelmények, kényszerek és minőségi forgatókönyvek alfejezetből származó fiktív követelményhalmaz:

- A rendszer online rendelésfelvételt biztosít webes és mobil csatornán.
- A rendelésfeldolgozás három lépésből áll: rendelés rögzítése, számlázási tranzakció indítása, készletcsökkentés.
- A csapat 6 főből áll, Java és TypeScript ismeretekkel.
- Az első 12 hónapban legfeljebb 500 egyidejű megrendelés várható.
- A rendelkezésre állás havi 99,9% kell legyen.
- Az adatokat EU-n belül kell tárolni.
- A rendszernek integrálódnia kell egy partner készletkezelő API-val, amely szinkron REST hívásokat támogat.

A minőségi forgatókönyv:

> **QA-003:** Amikor egy kampány alatt 500 egyidejű rendelési kérés érkezik, a rendszernek a kérések 95%-át 2 másodpercen belül kell feldolgoznia, és a rendelésállapotnak konzisztensnek kell maradnia.

32.2.9.2 9.2. Feladat

1. **Alternatívák megfogalmazása:** Készíts három értelmes architektúra-alternatívát a rendelésfeldolgozó szolgáltatásra. Legalább egy legyen moduláris monolit, egy mikroszolgáltatás-alapú, és egy eseményvezérelt. Határozd meg mindegyik stílusát, főbb mintáit és technológiai vázát.
2. **Döntési mátrix:** Válassz legalább öt értékelési dimenziót, súlyozd őket a QA-003 és a kényszerek alapján. Pontozd az alternatívákat indoklással, és számold ki a súlyozott pontszámot.
3. **C4 diagramok:** Készíts context és container diagramot a legjobbnak ítélt alternatívához diagrams-as-code formában (Mermaid).
4. **ADR:** Írj egy ADR-t `ADR-015` azonosítóval a választott architektúrára, a fejezetben megadott sablon szerint.
5. **Kockázati regiszter:** Azonosíts legalább három kockázatot, és töltsd ki a regisztert.
6. **Dokumentum:** Rendezd az eredményeket egyetlen verziózott Markdown dokumentumba. Jelöld, mely értékek mért és melyek várt eredmények.

32.2.9.3 9.3. Elvárt eredmény

- A dokumentum tartalmazza a három alternatívát, a döntési mátrixot indoklással, a C4 diagramokat, az ADR-t és a kockázati regisztert.
- A döntési mátrixban a cellák indoklást tartalmaznak; a súlyok összege 1.
- A C4 context diagramon a rendszerhatár és a külső szereplők egyértelműek; a container diagramon a kapcsolatok protokollal és szinkronitással szerepelnek.
- Az ADR tartalmazza a kontextust, döntést, alternatívákat, következményeket és állapotot.
- A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

32.2.9.4 9.4. Ellenőrzési szempontok

- Minden alternatívát ugyanazokkal a minőségi forgatókönyvekkel értékeltél?
- A kényszereket elkülönítve tüntetted fel?
- A döntési mátrixban a pontszámokhoz indoklás tartozik?
- A mért és várt eredményeket külön jelölöd?
- A C4 diagramon a rendszerhatár és a külső szereplők szerepelnek?
- Az ADR alkalmas arra, hogy egy későbbi olvasó megértse a döntés okát?
- A kockázati regiszterben van mitigáció és felelős?

32.2.9.5 9.5. Visszaállítási út

Ez a labor nem hajt végre destruktív műveletet, ezért nincs szükség visszaállításra. A dokumentumot a labor végén a `docs/architecture/sa123/` mappába kell elmenteni, és a verziókövető rendszerbe felvenni.

32.2.10 10. Ellenőrzőlista a döntési dokumentációhoz

- ☐ A döntési dokumentum verziózott, és a döntésekhez ADR-azonosító kapcsolódik.
- ☐ Legalább két, de lehetőség szerint három alternatíva szerepel.
- ☐ Az alternatívákban a stílus, a minta és a technológia elkülönül.
- ☐ A minőségi attribútumokat quality attribute scenario formában idézed.
- ☐ A kényszereket a döntési mátrixtól elkülönítve rögzíted.
- ☐ A döntési mátrix súlyozott, és a cellák indoklást tartalmaznak.
- ☐ A mért és várt eredményeket külön jelölöd.
- ☐ A C4 context, container és component szint közül a context és container szint elkészült.
- ☐ Az ADR-ek tartalmazzák a kontextust, döntést, alternatívákat, következményeket és állapotot.
- ☐ A kockázati regiszterben szerepel a valószínűség, hatás, mitigáció és felelős.
- ☐ A dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

32.2.11 11. Önellenőrző kérdés

Miért nem elegendő a döntési mátrix legmagasabb pontszámú alternatíváját automatikusan kiválasztani, és hogyan egészíti ki az ADR és a kockázati regiszter a mátrix értelmezését?

Válasz elvárás: A mátrix pontszáma a súlyozott értékelés eredménye, de a pontozás gyakran bizonytalan, várt értékeket tartalmaz, és nem tükrözi a kényszerek teljes körét. Az ADR rögzíti a kontextust és az elutasított alternatívák indoklását, míg a kockázati regiszter a döntéssel járó bizonytalanságokat és mitigációkat mutatja. A kettő együtt teszi a döntést visszakereshetővé és felülvizsgálhatóvá.

Ezzel a fejezet célja teljesült: megismerted az architektúra-alternatívák összehasonlításának és dokumentálásának módszerét, a döntési mátrix, az ADR és a C4 diagramok együttes használatát, valamint a trade-offok és kockázatok rögzítésének gyakorlatát. A következő egységben (Proof of Concept és mérés) ezekre a döntésekre építve Proof of Conceptet és mérést készítesz, hogy a várt értékekből mért eredmények legyenek.

32.3 Proof of Concept és mérés

A záróprojekt eddigi szakaszában – a Követelmények, kényszerek és minőségi forgatókönyvek követelményfeltárás és az Architektúra-alternatívák és döntési dokumentáció architektúra-alternatívák és döntési dokumentáció után – rendelkezésre áll egy rögzített követelményhalmaz, egy sor minőségi forgatókönyv és egy ADR-ekkel alátámasztott architektúratervezet. Ez azonban még mindig hipotézisek halmaza: olyan állítások, amelyeket a tervező asztal mellett valószínűnek tart, de amelyek éles környezetben, valós terhelés és valós adatok mellett másként is viselkedhetnek. A jelen alfejezet célja, hogy ezek közül a legkockázatosabb feltevéseket kísérleti úton, ellenőrzött keretek között vizsgáljuk meg. A Proof of Concept (PoC) nem a teljes rendszer leszállítása, és nem is a kiválasztott architektúra igazolása mindenáron. A PoC célja a tanulás: egy feltételezés megerősítése vagy megcáfolása, a döntési dokumentáció frissítése, és végső soron a kockázatok csökkentése az Architektúra-review, kockázat és bemutató architektúra-review előtt.

Ebben a fejezetben a PoC-ot mint kísérleti eszközt mutatjuk be. A hangsúly a mérésen, a reprodukálhatóságon és a döntési küszöbök előzetes rögzítésén van. A fejezet nem ismétli meg a korábbi részek alapfogalmait, és nem tér ki a következő fejezet részletes review-módszertanára. Azt feltételezzük, hogy az olvasó már ismeri a minőségi attribútumokat, az ADR-formátumot és a C4-dokumentáció alapjait. A cél az, hogy a PoC-ot ne demonstrációnak, hanem reprodukálható, kritikus kísérletnek tekintsük.

32.3.1 1. A legkockázatosabb feltevések kísérleti ellenőrzése

32.3.2 1.1. Mi számít kockázatos feltevésnek?

Egy architektúratervezet számtalan feltételezésen nyugszik. Például: „a választott üzenetkezelő rendszer képes kezelni másodpercenként 10 000 üzenetet”, „a replikált adatbázis a régiók közötti hálózati késleltetés mellett is biztosítja a 200 ms-os olvasási válaszidőt”, „az adott API-kapuzat kompatibilis a jelenlegi OAuth-szolgáltatóval”, vagy „a konténerindítási idő nem haladja meg a 30 másodpercet”. Mindegyik állítás igaz lehet, de amíg nem ellenőriztük, addig kockázatot hordoz.

A kockázat mértéke két tényezőtől függ: a feltételezés hatásától, ha téves, és a bizonytalanságtól, vagyis attól, hogy mennyire lehetünk biztosak az állítás helyességében.

A legkockázatosabb feltevések azok, amelyek egyszerre nagy hatásúak és nagy bizonytalanságúak. Ha egy feltételezés tévesnek bizonyul, akkor az egész architektúrát érintő újragondolást igényelhet: például kiderül, hogy a szinkron híváslánc nem teljesíti a rendelkezésre állásra vonatkozó SLO-t, vagy hogy a választott adatbázis nem támogatja a tranzakciós izolációs szintet, amelyet a rendszer megkövetel. Ilyen esetben a hiba korai felismerése sokkal olcsóbb, mint a termelési bevezetés utáni javítás.

A kockázatos feltevések azonosítása nem véletlenszerű. Az architektúra-dokumentáció minőségi forgatókönyveiből, az ADR-ek „következmények” szakaszából, valamint a tervezés során felmerült vitatott pontokból gyűjthetők össze. Minden olyan döntés, amelyet a csapat „valószínűleg működik” megjegyzéssel hagyott jóvá, jelölt a PoC-ra.

32.3.3 1.2. Kockázati mátrix és rangsorolás

A gyakorlatban hasznos egy egyszerű kockázati mátrixot készíteni. Az egyik tengely a hatás: mi történik, ha a feltételezés hamis? A másik tengely a bizonytalanság: mennyire ismerjük a választ? A mátrix celláiba írjuk be a feltételezéseket, és a jobb felső sarokban lévőket válasszuk ki PoC-ra. Például:

Feltételezés	Hatás (ha hamis)	Bizonytalanság	Prioritás
A Kafka-topic képes 10 000 üzenet/s feldolgozására	Magas – a teljes eseményfolyam leáll	Közepes – a gyártó specifikációja alapján valószínű, de a pontos konfiguráció nem ismert	Magas
A REST API p95 válaszideje 200 ms alatt marad 500 req/s mellett	Magas – a felhasználói élmény romlik	Magas – a tényleges teljesítmény a kód minőségétől és az adatbázis lekérdezésektől függ	Kritikus
A választott OAuth-szolgáltató kompatibilis a meglévő felhasználói adatbázissal	Közepes – alternatív autentikáció bevethető	Alacsony – a dokumentáció egyértelmű	Alacsony

A rangsorolás után a PoC fókusza a kritikus és magas prioritású tételekre kerül. A záróprojektben gyakran elegendő egyetlen, jól megválasztott PoC, amely a legnagyobb kockázatot kezeli. A múltbéli tapasztalatok szerint a legtöbb architektúraprojektben a teljesítmény, a skálázhatóság, az integrációs kompatibilitás és az adatkonzisztencia a leggyakoribb kockázati terület.

32.3.4 1.3. A PoC és a spike különbsége

A szakirodalom megkülönbözteti a PoC-ot és a spike-ot. A spike egy rövid, időben korlátozott kísérlet, amelynek célja egy technológia vagy megközelítés megismerése, gyakran eldobható kóddal. A PoC ezzel szemben egy kicsit formálisabb: egy adott hipotézis ellenőrzésére szolgál, előre meghatározott metrikával és döntési küszöbbel, és az eredménye dokumentáltan visszacsatolódik

az architektúradöntésekbe. A kettő közötti határ nem éles, de a záróprojektben a PoC-ot részesítjük előnyben, mert az Architektúra-review, kockázat és bemutató review-hoz mérhető bizonyítékra van szükség. A PoC nem „gyors hack”, hanem egy kis, ellenőrzött kísérlet.

32.3.5 2. Hipotézis, metrika és döntési küszöb

A PoC akkor értékes, ha előre tudjuk, mit akarunk mérni, és milyen eredmény esetén fogadjuk el vagy utasítjuk el a feltételezést. Ezt a hármast – hipotézis, metrika, döntési küszöb – már a kísérlet megkezdése előtt rögzíteni kell. Ez megvéd minket attól, hogy utólag magyarázzuk bele a kívánt eredményt a mérési adatokba.

32.3.6 2.1. Hipotézis megfogalmazása

A hipotézis egy ellenőrizhető, konkrét állítás. Nem elég azt mondani: „az új üzenetkezelő gyors”. A hipotézisnek tartalmaznia kell a feltételt, a metrikát és az elfogadási tartományt. Egy jó sablon:

> Ha [a vizsgált komponenst / konfigurációt] [adott környezetben] [adott terhelés mellett] futtatjuk, akkor [a kiválasztott metrika] [adott értéktartományban] marad.

Például:

> Ha a rendeléskezelő szolgáltatást a teszt-környezetben, 500 párhuzamos felhasználói kérést szimulálva futtatjuk, akkor a kosárba helyezés művelet p95 válaszideje 200 ms alatt marad, miközben a hibaarány legfeljebb 0,1%.

Ez a hipotézis mérhető, konkrét környezetet és terhelést ad meg, és egyértelműen eldönthető, hogy teljesült-e.

A hipotézis felírásakor ügyeljünk arra, hogy ne keverjük a funkcionális és a nem funkcionális állításokat. A PoC elsősorban a minőségi attribútumokat és a technikai megvalósíthatóságot vizsgálja, nem pedig az üzleti logika helyességét. Azt a tesztpiramis alsóbb rétegei már ellenőrizték.

32.3.7 2.2. Metrika kiválasztása

A metrika a hipotézisben megnevezett, mérhető mennyiség. Legyen:

- **Releváns**: a kiválasztott metrika közvetlenül kapcsolódik a kockázatos feltételezéshez. Ha a kockázat a válaszidő, akkor ne a CPU-kihasználtságot mérjük, hanem a válaszidőt.
- **Mérhető**: a metrika megbízhatóan gyűjthető a választott eszközökkel, és a mérési hiba kicsi a várt hatáshoz képest.
- **Stabil**: a metrika ne ingadozzon véletlenszerűen a mérési ablakon belül; ha igen, akkor hosszabb ablak vagy több ismétlés szükséges.
- **Összehasonlítható**: a metrika lehetővé teszi a várt és a mért érték egyértelmű összevetését.

Gyakori metrikák PoC-hoz: p95 vagy p99 válaszidő, áteresztőképesség (kérés/másodperc), hibaarány, rendelkezésre állás adott ablakban, üzenetfeldolgozási késleltetés, erőforrás-felhasználás (CPU, memória, hálózat). A könyv korábbi fejezeteiben bevezetett minőségi attribútum-forgatókönyvek segítenek a metrika pontos meghatározásában. A rendelkezésre állás esetén például a képlet:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}$$

Az SLO-val és az error budgettel való kapcsolatot a SLO, SLI, error budget és mérés alfejezetben tárgyaltuk; itt elég annyi, hogy a PoC-nál a mért rendelkezésre állás csak akkor értelmezhető, ha az üzemidőt és az állásidőt egyértelműen definiáltuk, és a mérési ablakot rögzítettük.

32.3.8 2.3. Döntési küszöb és mérési terv

A döntési küszöb az az előre meghatározott érték, amely alapján a hipotézist elfogadjuk vagy elutasítjuk. A küszöböt a mérés előtt kell rögzíteni, és nem szabad a mérési eredmény ismeretében módosítani. Például a fenti hipotézis esetén a küszöb: p95 válaszidő < 200 ms ÉS hibaarány 0,1%. Ha a mért p95 205 ms, akkor a hipotézist elutasítjuk, még akkor is, ha a különbség kicsi. A szigorúság célja, hogy a döntés ne váljon szubjektív vitává.

A döntési küszöb mellett mérési tervet is kell készíteni. A mérési terv tartalmazza:

- **Környezet**: milyen hardveren, operációs rendszeren, konténer-runtime-on, hálózati topológián fut a kísérlet? A környezet leírásának elegendőnek kell lennie a reprodukáláshoz.
- **Konfiguráció**: a vizsgált komponens verziója, a függőségek verziói, a beállítási paraméterek.
- **Terhelés**: milyen terhelést generálunk? Hány párhuzamos felhasználó, milyen kérésmintázat, milyen adatok?
- **Mérési ablak**: mennyi ideig tart egy mérési ciklus, hány ismétlést végzünk?
- **Adatgyűjtés**: milyen eszközzel gyűjtjük a metrikákat, hol tároljuk a nyers adatokat?
- **Bizonytalanság**: hogyan becsüljük a mérési hibát, és milyen konfidenciaintervallumot fogadunk el?

A mérési tervet írásban rögzítjük, lehetőleg a PoC-dokumentum részeként. Ez biztosítja, hogy a kísérlet reprodukálható, és hogy az Architektúra-review, kockázat és bemutató review-ban a bizonyítékok ellenőrizhetők.

32.3.9 2.4. Mérési környezet és bizonytalanság

A PoC-környezet gyakran nem azonos a termelési környezettel. Ez elfogadható, ha a különbségeket dokumentáljuk, és a következtetéseket ennek megfelelően fogalmazzuk meg. Például ha a PoC egyetlen gépen fut, míg a termelési környezet elosztott, akkor a hálózati késleltetésre vonatkozó következtetések korlátozottak. A mérési bizonytalanságot mindig fel kell tüntetni: a mért értékek átlaga mellett a szórást vagy a konfidenciaintervallumot is meg kell adni. Egyetlen mérési pont nem elegendő; általában legalább 5–10 ismétlést végzünk, és a kiugró értékeket megvizsgáljuk.

A bizonytalanság forrásai lehetnek: a terhelésgenerátor és a vizsgált rendszer közötti versengés ugyanazon erőforrásokért, a hálózati zaj, a háttérfolyamatok, a melegítési idő hiánya, a szemétgyűjtés hatása. A mérési tervben ezeket a forrásokat lehetőség szerint kontrollálni kell: például a terhelésgenerátort külön gépen futtatjuk, a vizsgált rendszert előmelegítjük, a mérést többször megismételjük.

32.3.10 3. A PoC felépítése és végrehajtása

32.3.11 3.1. Minimális, de hiteles környezet

A PoC nem a teljes architektúra kicsinyített mása. A cél a kockázatos feltevés ellenőrzése, ezért a kísérleti összeállításnak a lehető legkisebbnek kell lennie, de elég hitelesnek ahhoz, hogy a mért eredményből következtetni lehessen a valós viselkedésre. Ha például a kockázat az adatbázis-lekérdezések válaszideje, akkor a PoC-ban egy valósághű adatmennyiséget kell betölteni, nem pedig egy üres táblát. Ha a kockázat a hálózati késleltetés, akkor a hálózati feltételeket szimulálni kell (például késleltetés és csomagvesztés bevezetésével), nem pedig localhoston mérni.

A hitelesség és a minimálisság egyensúlya a PoC tervezésének egyik legfontosabb művészete. Túl bonyolult környezet esetén a kísérlet drága és lassú lesz, túl egyszerű esetén az eredmény nem lesz releváns. Jó gyakorlat, ha a PoC-ot a C4 container szintjén írjuk le: mely konténereket indítjuk el, milyen kapcsolatokkal, és mely külső rendszereket szimulálunk.

32.3.12 3.2. Reprodukálhatóság és verziózás

A PoC reprodukálhatósága alapkövetelmény. Ehhez a kísérlet minden elemét verziózni kell: a kódot, a konfigurációt, a terhelésgeneráló szkriptet, az adathalmazt és a mérési eredményeket. A konténerek esetében a képek verziószámát rögzíteni kell, nem elég a `latest` címke. Az infrastruktúra leírását – például egy `docker-compose.yml` vagy egy Kubernetes manifest – a kódbázissal együtt verziózzuk. A mérési eredményeket nyers formában, például CSV- vagy JSON-fájlban tároljuk, hogy később újra elemezhetők legyenek.

A reprodukálhatóság azt is jelenti, hogy egy másik mérnök ugyanazt a kísérletet meg tudja ismételni, és statisztikailag hasonló eredményt kap. Ha a PoC eredménye nem reprodukálható, akkor nem tekinthető bizonyítéknak, csak anekdotának.

32.3.13 3.3. Mérőeszközök és terhelésgenerálás

A terhelésgenerálás és a metrikagyűjtés eszközeit a kísérlet előtt ki kell választani. Gyakori terhelésgeneráló eszközök: a `k6`, a `JMeter`, a `hey`, a `wrk`, vagy egy egyedi írt szkript. Az eszköz kiválasztásánál ügyeljünk arra, hogy a generált terhelés pontosan szabályozható legyen, és hogy a generátor maga ne váljon szűk keresztmetszetté. A metrikákat a vizsgált rendszerből gyűjtjük: az alkalmazás logjai, a Prometheus-metrikák, az OpenTelemetry-nyomkövetések. A terhelésgenerátor által mért válaszidőket és hibaarányt is rögzíteni kell.

A mérés során a rendszer erőforrás-felhasználását is figyelni érdemes, mert ez segít a szűk keresztmetszetek azonosításában. A CPU, memória, hálózati I/O és a lemez I/O metrikákat időbélyeggel együtt tároljuk.

32.3.14 3.4. Példa: gRPC vs REST API választás

Tegyük fel, hogy a záróprojektben egy belső szolgáltatások közötti kommunikációhoz választanunk kell a REST (JSON over HTTP) és a gRPC (protobuf over HTTP/2) között. Az Architektúra-alternatívák és döntési dokumentáció alfejezetben az ADR-ek azt rögzítették, hogy a gRPC-t választottuk a kisebb sorosítási overhead és a beépített streamingtámogatás miatt. A kockázatos feltevés azonban az, hogy a gRPC a mi konkrét terhelési profilunk mellett valóban jobb p95 válaszidőt ad, mint a REST, legalább 30%-kal. Ha ez nem igaz, akkor lehet, hogy a REST egyszerűbb választás lenne.

A PoC hipotézise:

> Ha a példa rendeléskezelő szolgáltatást 300 párhuzamos klienssel, másodpercenként összesen 1500 kérést generálva futtatjuk, akkor a gRPC implementáció p95 válaszideje legalább 30%-kal alacsonyabb, mint a REST implementációé, miközben a hibaarány mindkét esetben 0,1% alatt marad.

A metrika: p95 válaszidő (ms) és hibaarány (%).

A döntési küszöb:

- Elfogadjuk a gRPC-t, ha a gRPC p95 válaszideje 0{,}7 × REST p95 válaszideje, és a hibaarány 0{,}1%.
- Elutasítjuk a gRPC-t, ha a feltétel nem teljesül.

A mérési terv:

- Környezet: két konténer, egy-egy azonos funkcionalitású szolgáltatással (REST és gRPC), azonos üzleti logikával, azonos adatbázis-lekérdezésekkel. A konténerek egy külön hálózaton futnak, a terhelésgenerátor egy harmadik konténerben.
- Konfiguráció: a szolgáltatások verziója `v0.9.0`, a futásidő `openjdk:17-jre` (ellenőrzési igény: a konkrét verzió a projekt aktuális stabil verziója), a hálózati késleltetés 0 ms (a kísérlet a sorosítási és feldolgozási overheadre fókuszál).
- Terhelés: 300 virtuális felhasználó, egyenletes eloszlású kérések, 1500 req/s összesen, 5 perc felfutás, 10 perc mérési ablak.
- Ismétlések: 5 független mérési ciklus.
- Adatgyűjtés: a terhelésgenerátor rögzíti a válaszidőket és a státuszkódokat; a szolgáltatások Prometheus-metrikákat szolgáltatnak.

A PoC elvégzése után az eredményeket táblázatban rögzítjük, külön oszlopban a várt és a mért értékeket. Feltételezett eredmény (nem valós mérés, csak illusztráció):

Metrika	REST (mért)	gRPC (mért)	Várt (küszöb)	Döntés
p95 válaszidő	85 ms	52 ms	gRPC 0{,}7 × REST = 59{,}5 ms	Teljesül
Hibaarány	0{,}05%	0{,}03%	0{,}1%	Teljesül

A mérési bizonytalanságot is fel kell tüntetni, például: a p95 válaszidő 95%-os konfidenciaintervalluma REST esetén [80, 90] ms, gRPC esetén [49, 55] ms. A döntés egyértelmű: a PoC megerősíti a gRPC választását.

Ez a példa bemutatja a hipotézis, metrika, döntési küszöb és mérési terv együttes használatát. A laborban a résztvevőnek hasonló feladatot kell megoldania.

32.3.15 4. Az eredmények kiértékelése és tipikus hibák

32.3.16 4.1. Statisztikai értékelés és bizonytalanság

A mérési eredményeket nem elég egyetlen számmal jellemezni. Minden mért metrikához tartoznia kell:

- **Átlag és medián**: a középértékek a tipikus viselkedést mutatják, de a szélsőséges értékekre nem érzékenyek.
- **Percentilisek**: a p95, p99 a farok viselkedését mutatják, ami a felhasználói élmény szempontjából gyakran fontosabb, mint az átlag.
- **Szórás és konfidenciaintervallum**: a mérési bizonytalanság számszerűsítése.

A döntési küszöböt a megfelelő statisztikai mutatóval kell összevetni. Ha a hipotézis a p95 válaszidőre vonatkozik, akkor a mért p95-öt hasonlítjuk a küszöbhöz, nem az átlagot. A konfidenciaintervallum segít abban, hogy a döntés ne egyetlen mérési ponttól függjön. Ha a konfidenciaintervallum átfedi a küszöböt, akkor a döntés bizonytalan, és további mérésre van szükség.

32.3.17 4.2. Tipikus hibák a PoC-ban

A PoC-k során számos hiba csúszhat be. A leggyakoribbak:

1. **Utólagos küszöbmódosítás**: ha a mérési eredmény nem éri el a kívánt értéket, a csapat megváltoztatja a küszöböt. Ez aláássa a PoC hitelességét. A küszöböt a mérés előtt rögzíteni kell, és a döntést a rögzített küszöb alapján kell meghozni.
2. **Túl kevés minta**: egyetlen mérési ciklus nem elegendő. A véletlenszerű ingadozások miatt a következtetés téves lehet. Legalább 5–10 ismétlés szükséges, és a szórást is figyelembe kell venni.
3. **Melegítés hiánya**: a rendszer az első kérések során inicializál, a JIT-fordító még nem optimalizált, a cache-ek üresek. Ha a mérési ablak a melegítési szakaszt is tartalmazza, az eredmény torzít. A mérés előtt a rendszert elő kell melegíteni, majd a tényleges mérési ablakot külön kell kezelni.
4. **Zajos környezet**: ha a PoC ugyanazon a gépen fut, mint más folyamatok, a CPU- és hálózati versengés torzítja az eredményt. A lehető legnagyobb mértékben izolálni kell a kísérletet, vagy legalább a háttérterhelést dokumentálni.
5. **Nem reprezentatív adatok**: ha a PoC adatbázisa üres vagy szintetikus adatokat tartalmaz, amelyek nem tükrözik a valós eloszlást, a lekérdezési teljesítmény nem lesz hiteles. A lehető legvalósághűbb adatkészletet kell használni.
6. **A terhelésgenerátor szűk keresztmetszet**: ha a terhelésgenerátor maga telítődik, akkor a mért válaszidők nem a vizsgált rendszer, hanem a generátor korlátját tükrözik. A generátor erőforrás-felhasználását is monitorozni kell.
7. **A PoC-ot demonstrációként kezelni**: ha a cél a kiválasztott technológia „bemutatása” a stakeholdereknek, akkor a mérési szigor háttérbe szorul. A PoC ezzel szemben egy kritikus kísérlet, amelynek célja a feltételezés megerősítése vagy megcáfolása.

32.3.18 4.3. A mért és a várt eredmény elkülönítése

A PoC-dokumentumban a mért és a várt eredményt mindig külön kell feltüntetni. A várt eredmény a hipotézisben és a döntési küszöbben rögzített érték; a mért eredmény a kísérletből származó adat. A kettő összevetése adja a döntést: elfogadás, elutasítás vagy további vizsgálat. A dokumentációban ne keverjük a kettőt, és ne írjuk át a várt értéket a mért érték alapján. A laborban ezért külön táblázatot kérünk, amelyben a „várt” és a „mért” oszlopok egyértelműen elkülönülnek.

32.3.19 5. A PoC eredményének visszacsatolása

A PoC nem öncélú. Az eredménynek vissza kell csatolódnia az architektúradöntésekbe és a dokumentációba.

32.3.20 5.1. ADR frissítése

Ha a PoC egy ADR-ben rögzített döntést vizsgál, akkor az eredményt az ADR-hez kell kapcsolni. Az ADR „állapot” mezője frissül: például „Elfogadva”, „Elutasítva” vagy „Felülvizsgálva”. Az ADR „következmények” szakasza kiegészül a PoC eredményével: milyen feltételek teljesültek, milyen bizonytalanságok maradtak. Ha a PoC a döntést megcáfolta, akkor új ADR-t kell írni, amely a korábbi döntést visszavonja, és alternatívát javasol. Az ADR-ek száma nem cél, de a döntések nyomon követése igen; ezért minden PoC-hoz tartoznia kell egy ADR-bejegyzésnek vagy hivatkozásnak.

32.3.21 5.2. Architektúra-dokumentum és kockázati nyilvántartás

A PoC eredménye az architektúra-dokumentumot is érinti. A C4 diagramokon a komponensek felelőssége és interfészei nem változnak, ha a PoC sikeres; de ha a PoC egy adott kapcsolatról kideríti, hogy nem teljesíti a minőségi követelményt, akkor a diagramot és a kapcsolatok jellemzőit frissíteni kell. A kockázati nyilvántartásban a PoC által vizsgált kockázat állapota megváltozik: „nyitott” → „csökkentett” vagy „elfogadott”. A megmaradt kockázatokat az Architektúra-review, kockázat és bemutató review-ban tárgyaljuk.

32.3.22 5.3. Döntési pont: elfogadás, elutasítás, további vizsgálat

A PoC eredménye háromféle lehet:

- **Elfogadás**: a mért eredmény teljesíti a döntési küszöböt. A feltételezés megerősítve, a döntés fenntartható. A kockázat csökken, a projekt továbbléphet a review felé.
- **Elutasítás**: a mért eredmény nem teljesíti a küszöböt. A feltételezés hamis, a döntést felül kell vizsgálni. Ez nem kudarc, hanem értékes tanulás: elkerültük a rossz döntés termelési bevezetését.
- **További vizsgálat**: a mérési bizonytalanság túl nagy, vagy a konfidenciaintervallum átfedi a küszöböt. Ilyenkor bővíteni kell a mérési tervet: több ismétlés, hosszabb ablak, pontosabb eszközök.

A döntést dokumentálni kell, és a következő lépéseket meg kell határozni. Ha a PoC elutasította a feltételezést, akkor az Architektúra-alternatívák és döntési dokumentáció alfejezetben felállított alternatívák közül kell választani, és adott esetben új PoC-ot indítani.

32.3.23 6. Labor: PoC végrehajtása és mérése

A labor célja, hogy a résztvevő a fejezetben leírt módszertant egy konkrét, kis példán alkalmazza. A labor nem igényel éles rendszert; a kísérletet helyi, izolált környezetben kell elvégezni. A labor eredménye egy reprodukálható dokumentum vagy jegyzetfüzet, amely tartalmazza a hipotézist, a mérési tervet, a nyers adatokat, a kiértékelést és a döntést.

32.3.24 6.1. Kiinduló állapot

Adott egy egyszerű, konténerizált HTTP-szolgáltatás, amely egy listát ad vissza egy adatbázisból. A szolgáltatás két változatban érhető el: `rest-service` és `grpc-service` (a példa a fenti gRPC vs REST esetet egyszerűsíti). A kód és a konfiguráció a laborhoz mellékelt git-repozitóriumban található. A szolgáltatások azonos üzleti logikát valósítanak meg, és az adatbázis egy előre feltöltött, szintetikus adatkészletet tartalmaz.

A laborhoz szükséges eszközök: Docker, egy terhelésgeneráló eszköz (például `k6`), egy metrikakövető (például Prometheus) és egy jegyzetfüzet-környezet (például Jupyter vagy egyszerű Markdown + CSV). Minden eszköz verzióját a labor elején rögzíteni kell.

32.3.25 6.2. Feladat

A résztvevő feladata:

1. Válasszon ki egy kockázatos feltevést a megadott szolgáltatás kapcsán. Például: „A gRPC szolgáltatás p95 válaszideje legalább 20%-kal alacsonyabb, mint a REST szolgáltatásé 200 req/s terhelés mellett.”
2. Fogalmazzon meg egy hipotézist a sablon szerint.
3. Válasszon metrikát és határozzon meg döntési küszöböt.
4. Készítsen mérési tervet: környezet, konfiguráció, terhelés, mérési ablak, ismétlésszám, adatgyűjtés.
5. Hajtsa végre a mérést a megadott eszközökkel. Ügyeljen a melegítésre és a zaj csökkentésére.
6. Rögzítse a nyers adatokat CSV- vagy JSON-fájlban.
7. Értékelje az eredményt: számítsa ki a percentiliseket, a szórást és a konfidenciaintervallumot.
8. Hasonlítsa össze a mért eredményt a döntési küszöbbel, és hozzon döntést: elfogadás, elutasítás vagy további vizsgálat.
9. Készítsen egy rövid ADR-t a PoC eredményéről, és frissítse a kockázati nyilvántartást.

32.3.26 6.3. Elvárt eredmény

A labor végén a résztvevőnek be kell mutatnia egy reprodukálható dokumentumot, amely a következőket tartalmazza:

- A hipotézis pontos megfogalmazása.
- A metrika és a döntési küszöb.
- A mérési terv leírása, beleértve a környezetet, a konfigurációt, a terhelést és a mérési ablakot.
- A nyers mérési adatok (CSV vagy JSON), legalább 5 ismétléssel.
- A statisztikai kiértékelés: átlag, medián, p95, szórás, konfidenciaintervallum.
- A várt és a mért eredmény külön táblázatban.
- A döntés és annak indoklása.
- Az ADR (vagy annak kivonata) és a kockázati nyilvántartás frissített állapota.
- A kísérlet visszaállítási útja: hogyan lehet a környezetet az eredeti állapotba visszaállítani.

A dokumentum ne tartalmazzon valós hitelesítő adatot, belső URL-t vagy ügyfélnevet. A szolgáltatások nevei legyenek általánosak: `példa-szolgáltatás`, `teszt-adatbázis`.

32.3.27 6.4. Ellenőrzési szempont

A labor értékelésekor a következőket ellenőrizzük:

- A hipotézis konkrét, mérhető és ellenőrizhető.
- A döntési küszöb a mérés előtt rögzítve van.
- A mérési terv tartalmazza a környezetet, a konfigurációt, a terhelést és a mérési ablakot.
- A mérés legalább 5 ismétlést tartalmaz, és a melegítést külön kezeli.
- A nyers adatok rendelkezésre állnak, és azokból a statisztikák reprodukálhatók.
- A mért és a várt eredmény egyértelműen elkülönül.
- A döntés a küszöb alapján, nem utólagos módosítással született.
- A bizonytalanság szerepel a kiértékelésben.
- Az ADR és a kockázati nyilvántartás frissítése megtörtént.
- A visszaállítási út le van írva.

32.3.28 6.5. Visszaállítási út

A labor során használt konténereket le kell állítani, és a létrehozott adatbázis-állapotot törölni kell. Ha a kísérlet bármilyen fájlt hozott létre a gazdagépen, azokat a labor végén el kell távolítani. A Docker-hálózatot és a volume-okat is törölni kell, hogy a következő labor tiszta állapotból induljon. A visszaállítási lépéseket a dokumentum végén kell felsorolni, például:

```
docker compose down --volumes --remove-orphans
rm -f results/*.csv
```

A destruktív műveletek csak izolált környezetben végezhetők, ahogy a könyv labor-szerződése is előírja.

32.3.29 6.6. Dokumentációs minta

A labor dokumentumának egy lehetséges szerkezete:

```
# PoC: gRPC vs REST válaszidő

## Hipotézis
Ha a példa-szolgáltatást 200 req/s terhelés mellett futtatjuk,
akkor a gRPC implementáció p95 válaszideje legalább 20%-kal
alacsonyabb, mint a REST implementációé.

## Metrika
p95 válaszidő (ms), hibaarány (%)

## Döntési küszöb
Elfogadás: gRPC p95 <= 0,8 * REST p95 ÉS hibaarány <= 0,1%

## Környezet
- Operációs rendszer: Linux, kernel 6.8.0
- Docker: 27.0.3
- Konténerek: rest-service:v0.9.0, grpc-service:v0.9.0,
  k6:0.49.0, prometheus:2.45.0
- Hálózat: docker network, latency 0 ms

## Mérési terv
- Terhelés: 200 req/s, 300 virtuális felhasználó
- Felfutás: 3 perc
- Mérési ablak: 10 perc
```

```
- Ismétlések: 5
- Melegítés: 2 perc, nem számít bele a mérésbe

## Nyers adatok
Lásd: results/raw_*.csv

## Kiértékelés
| Metrika | REST (átlag ± CI) | gRPC (átlag ± CI) |
|---|---|---|
| p95 válaszidő | 85 ± 5 ms | 52 ± 4 ms |
| Hibaarány | 0,05% | 0,03% |

## Várt vs mért
| Metrika | Várt | Mért | Döntés |
|---|---|---|---|
| gRPC p95 <= 0,8 * REST p95 | <= 68 ms | 52 ms | Teljesül |
| Hibaarány | <= 0,1% | 0,03% | Teljesül |

## Döntés
A hipotézist elfogadjuk. A gRPC választása megerősítve.

## ADR
Lásd: adr/0015-grpc-poc.md

## Visszaállítás
docker compose down --volumes --remove-orphans
rm -f results/raw_*.csv
```

A dokumentum jól mutatja a mért és a várt eredmény elkülönítését.

32.3.30 7. Ellenőrzőlista az alfejezethez

A fejezet végén az alábbi ellenőrzőlistát használhatjuk a PoC minőségének felmérésére:

- ☐ A kockázatos feltevéseket azonosítottuk és rangsoroltuk (hatás × bizonytalanság).
- ☐ A PoC egyértelműen a legkockázatosabb feltevésre fókuszál.
- ☐ A hipotézis konkrét, mérhető és ellenőrizhető.
- ☐ A metrika releváns, mérhető, stabil és összehasonlítható.
- ☐ A döntési küszöböt a mérés előtt rögzítettük.
- ☐ A mérési terv tartalmazza a környezetet, a konfigurációt, a terhelést, a mérési ablakot és az ismétlésszámot.
- ☐ A mérés során a melegítést külön kezeltük, és a zajt kontrolláltuk.
- ☐ A nyers adatokat verziózott, reprodukálható formában tároljuk.
- ☐ A statisztikai kiértékelés tartalmazza a percentiliseket, a szórást és a konfidenciaintervallumot.
- ☐ A mért és a várt eredményt külön táblázatban tüntettük fel.
- ☐ A döntést a rögzített küszöb alapján hoztuk meg, utólagos módosítás nélkül.
- ☐ A bizonytalanságot és a környezeti eltéréseket dokumentáltuk.
- ☐ Az eredményt visszacsatoltuk az ADR-be és a kockázati nyilvántartásba.
- ☐ A visszaállítási lépéseket rögzítettük, és a destruktív műveletek izolált környezetben történtek.

32.3.31 8. Önellenőrző kérdés

Az alfejezet tanulási céljainak ellenőrzésére válaszoljon a következő kérdésre:

> Egy csapat PoC-ot végez, hogy eldöntse, egy új cache-réteg képes-e a p95 válaszidőt 150 ms alá csökkenteni 800 req/s terhelés mellett. A mérés első ciklusában a p95 160 ms, a másodikban 145 ms, a harmadikban 155 ms. A csapat a döntési küszöböt 150 ms-ról 160 ms-ra módosítja, hogy a cache-t elfogadhassák. Milyen hibát követnek el, és mi lenne a helyes eljárás?

A helyes válasz: a csapat megsérti a döntési küszöb előzetes rögzítésének elvét. A küszöböt a mérés előtt kellett volna 150 ms-ban rögzíteni, és a három mérési eredmény alapján a hipotézist elutasítani (vagy további mérést végezni, ha a konfidenciaintervallum átfedi a küszöböt). Az utólagos módosítás a PoC hitelességét ássa alá, és a kívánt eredménybe való belemagyarázást jelent. A helyes eljárás: a mért értékeket statisztikailag elemezni, a bizonytalanságot számszerűsíteni, és ha a konfidenciaintervallum nem fedi a 150 ms-ot, akkor a cache-réteget elutasítani, és alternatívát vizsgálni.

32.3.32 Záró gondolatok

A PoC és a mérés a záróprojekt azon pontja, ahol a papíron szép architektúra találkozik a valósággal. A legkockázatosabb feltevések kísérleti ellenőrzése nem a hibák elkerülését szolgálja – a hibák elkerülhetetlenek –, hanem azt, hogy a hibákat akkor fedezzük fel, amikor a javítási költség még alacsony. A jól megtervezett PoC egyben a csapat tanulási eszköze is: a mérési eredmények, a döntési küszöbök és a dokumentált bizonytalanságok olyan tudást adnak, amelyre az Architektúra-review, kockázat és bemutató architektúra-review során építeni lehet. A következő alfejezetben ezt a tudást használjuk fel, amikor a teljes architektúrát formálisan értékeljük, a kockázatokat áttekintjük, és a bemutatót előkészítjük.

32.4 Architektúra-review, kockázat és bemutató

A záróprojekt eddigi lépései során követelményeket tártunk fel, architektúra-alternatívákat elemeztünk, döntéseket dokumentáltunk, majd egy Proof of Concept (PoC) keretében mértük a kritikus minőségi jellemzőket. Ebben az alfejezetben a hangsúly az értékelésen és a kommunikáción van: hogyan vizsgáljuk felül a teljes architektúrát, hogyan azonosítjuk és kezeljük a kockázatokat, és hogyan mutatjuk be mindezt a döntéshozóknak úgy, hogy a következő lépésről felelősen dönthessenek. Az architektúra-review, a kockázatkezelés és a bemutató nem adminisztratív teher, hanem a tanulási ciklus utolsó, döntést érlelő fázisa: a Proof of Concept és mérés mérési eredményeit itt fordítjuk le döntési javaslattá.

32.4.1 Az architektúra-review célja és illeszkedése

Az architektúra-review olyan strukturált értékelés, amely azt vizsgálja, hogy a javasolt architektúra teljesíti-e a rögzített minőségi követelményeket, a döntések konzisztensek-e egymással, és milyen kockázatok maradtak a rendszerben. A kód-review-tól eltérően itt nem az implementáció részletei, hanem a komponensek felelőssége, a kapcsolatok protokolljai, a minőségi forgatókönyvekre adott válaszok és a döntési indoklások állnak a középpontban.

A korábbi fejezetben (ATAM, CBAM és architektúra-review) bemutatott ATAM ehhez jó keretet ad, de egy záróprojektben gyakran elegendő egy könnyebb, szcenárió-alapú review is. Fontos, hogy a review ne egyetlen személy magánvéleménye legyen, hanem az érintett szerepkörök – fejlesztő, üzemeltető, biztonsági felelős, termékmenedzser – közös elemzése. A review bemenetei:

- a záróprojekt architektúra-dokumentációja (C4 context, container és component szintű diagramok),
- az ADR-ek (Architecture Decision Records),
- a minőségi forgatókönyvek (quality attribute scenarios),
- a Proof of Concept és mérés PoC mérési eredményei, a mérési környezet és a bizonytalanság feltüntetésével,
- a korábban azonosított kockázatok és nyitott kérdések.

A review folyamata három szakaszból áll. Az előkészítés során rögzítjük a hatókört, a meghívottakat és a rendelkezésre álló anyagokat. A lefolyatás során a forgatókönyveket egyenként végigjárjuk, megvizsgáljuk a komponensek együttműködését, az esetleges szűk keresztmetszeteket és a döntések konzisztenciáját. A dokumentálás során megállapításokat, kockázatokat és döntési javaslatot rögzítünk, lehetőleg a döntési naplóban.

32.4.2 Minőségi attribútumok ellenőrzése szcenáriókkal

A review gerincét a minőségi forgatókönyvek jelentik. Minden kritikus minőségi követelményhez legalább egy forgatókönyv tartozik, amely az ingert, a környezetet, a várt választ és a mérőszámot tartalmazza. A review során minden forgatókönyvhöz hozzárendeljük a rendelkezésre álló bizonyítékot: lehet mért érték a PoC-ből, modellalapú becslés, vagy szakértői ítélet. A nem mért állításokat explicit módon meg kell jelölni, például „becslés, további mérés szükséges” megjegyzéssel.

Példaként vegyünk egy e-kereskedelmi záróprojektet, ahol a rendszer egy termékkatalógus-szolgáltatásból, egy rendeléskezelő szolgáltatásból és egy fizetési integrációból áll. A review egyik forgatókönyve:

Rendelkezésre állás forgatókönyv (S1)
- *Inger*: a fizetési szolgáltató külső API-ja 5 másodpercnél hosszabb késleltetéssel válaszol.
- *Környezet*: csúcsidő, a rendeléskezelő szolgáltatás terhelése 200 kérés/másodperc.
- *Válasz*: a rendeléskezelő szolgáltatás 500 ms-on belül visszajelzést ad a felhasználónak, a fizetés feldolgozása aszinkron folytatódik, a rendszer nem veszít tranzakciót.
- *Mérőszám*: a felhasználói válaszidő 95. percentilise < 500 ms, a tranzakció-veszteség 0%, az adott mérési ablakban (10 perc, csúcsidő).

A review során a csapat megvizsgálja, hogy a jelenlegi architektúra – például egy outbox minta és egy üzenetsor – hogyan teljesíti ezt a forgatókönyvet. A PoC mérési eredményeiből kiderülhet, hogy a válaszidő 95. percentilise 480 ms volt, a tranzakció-veszteség 0%, tehát a forgatókönyv teljesült. Ha viszont a mérés hiányos, vagy a válaszidő 620 ms volt, akkor a forgatókönyv részben teljesült, és kockázatként kell rögzíteni.

A szcenárió-értékelés kimenete egy táblázat, amely minden forgatókönyvhöz tartalmazza az állapotot és a bizonyítékot:

Forgatókönyv	Elvárt válasz	Mért/becsült érték	Értékelés	Bizonyíték
S1 – Fizetési késleltetés	95. p $<$ 500 ms, 0% veszteség	95. p $=$ 480 ms, veszteség 0%	Teljesítve	PoC mérés, 2025-03-10, 10 perc, terhelés 200 req/s

Forgatókönyv	Elvárt válasz	Mért/becsült érték	Értékelés	Bizonyíték
S2 – Katalógus olvasási csúcs	99. p $<$ 200 ms	99. p = 180 ms	Teljesítve	PoC mérés, 2025-03-11, 15 perc, terhelés 1000 req/s
S3 – Adatbázis-failover	RPO 5 s, RTO 60 s	RPO becsült 10 s, RTO mért 45 s	Részben	Failover teszt, izolált környezet, 2025-03-12; RPO becslés replikációs késleltetésből

Fontos, hogy a mért és a várt értékeket külön oszlopban tüntessük fel, és a becsléseket megjelöljük. Ha egy forgatókönyv nem teljesül, az nem feltétlenül jelenti az architektúra elutasítását, hanem egy kockázatot és egy lehetséges intézkedést, például a replikációs stratégia módosítását vagy további mérést.

32.4.3 A döntési napló és az ADR-ek felülvizsgálata

Az architektúra-döntések minősége közvetlenül befolyásolja a rendszer evolválhatóságát és a csapat döntési kultúráját. A review során minden ADR-t ellenőrizni kell:

- Létezik-e ADR minden jelentős döntéshez?
- A döntés indoklása kapcsolódik-e a minőségi követelményekhez és a trade-offokhoz?
- Az alternatívák és a következmények dokumentáltak-e?
- A döntés állapota aktuális-e (javasolt, elfogadott, elavult)?
- Vannak-e új információk a PoC óta, amelyek megváltoztathatják a döntést?

Gyakori hiba, hogy a csapat a szinkron REST integráció mellett döntött, mert egyszerűnek tűnt, de a PoC terhelési tesztje megmutatta, hogy a függőségi lánc miatt a válaszidő nem tartható. Ilyenkor a review feladata, hogy új ADR-t kezdeményezzen az aszinkron üzenetkezelésre, és a régit elavulttá nyilvánítsa.

Egy ADR felülvizsgálatának példája:

```
# ADR-014: A fizetési integráció aszinkronná alakítása

## Kontextus
A PoC terhelési tesztje (2025-03-10) azt mutatta, hogy a szinkron fizetési hívás
a rendeléskezelő szolgáltatásból 620 ms-os 95. percentilist eredményezett, ami
túllépi az SLO-ban rögzített 500 ms-t. A fizetési szolgáltató 5 másodpercnél
hosszabb késleltetése esetén a felhasználói válaszidő tovább romlik.

## Döntés
A rendeléskezelő szolgáltatás a fizetési kérést egy outbox táblába írja, majd
egy message brokeren keresztül aszinkron feldolgozásra küldi. A felhasználó
azonnal „rendelés rögzítve, fizetés folyamatban" visszajelzést kap.
```

```
## Alternatívák
1. Szinkron hívás megtartása, de timeout csökkentése 1 másodpercre.
   Elvetve: a timeout alatt a felhasználó nem kap visszajelzést, és a tranzakció
   állapota bizonytalan.
2. Retry logika a szinkron hívás körül.
   Elvetve: nem oldja meg a hosszú késleltetést, csak növeli a terhelést.

## Következmények
- Pozitív: a felhasználói válaszidő függetlenedik a fizetési szolgáltatótól,
  a tranzakció-veszteség csökken.
- Negatív: a rendszernek kezelnie kell a duplikált fizetési kéréseket, az
  idempotenciát és a visszaigazolás késleltetését.
- Új komponens: message broker (pl. Kafka vagy RabbitMQ), outbox tábla.

## Állapot
Javasolt - a review jóváhagyásra vár.
```

A döntési napló ilyen frissítése biztosítja, hogy a review ne csak a jelen állapotot értékelje, hanem a döntések nyomon követhetőségét is fenntartsa.

32.4.4 Kockázatkezelés a review-ban

A kockázat olyan bizonytalan esemény vagy körülmény, amely bekövetkezése esetén hatással van a projekt céljaira, például a határidőre, a minőségre vagy a költségre. A review során a kockázatokat szisztematikusan azonosítani, értékelni és csökkentő intézkedésekkel ellátni kell. A kockázat nem azonos a nyitott kérdéssel: a kockázatnak van bekövetkezési valószínűsége és hatása, míg a nyitott kérdés olyan információhiány, amely további vizsgálatot igényel, de önmagában nem feltétlenül negatív.

32.4.5 Kockázatok azonosítása

Az azonosításhoz több forrást használhatunk:

- a minőségi forgatókönyvek értékelése során feltárt eltérések,
- a PoC mérési bizonytalanságai,
- a csapat korábbi tapasztalatai hasonló rendszerekkel,
- általános architektúrális ellenőrzőlisták (pl. adatbiztonság, skálázhatóság, megfigyelhetőség),
- a külső függőségek (fizetési szolgáltató, felhőszolgáltató, adatbázis) elemzése.

A kockázatokat kategóriákba soroljuk: technikai, szervezeti, biztonsági, ütemezési és üzemeltetési. Például:

- **Technikai**: az adatbázis replikációs késleltetése miatt az RPO nem tartható.
- **Szervezeti**: a csapatnak nincs tapasztalata a választott message broker üzemeltetésében.
- **Biztonsági**: a fizetési adatok naplózása során érzékeny adat kerülhet a logokba.
- **Ütemezési**: a rendelkezésre álló idő nem elegendő a teljes terhelési teszt lefuttatására.
- **Üzemeltetési**: a megfigyelhetőségi stack még nem tartalmaz elosztott trace-t, ami megnehezíti a hibadiagnózist.

32.4.6 Kockázati mátrix és regiszter

Minden kockázathoz két dimenziót rendelünk: a bekövetkezés valószínűségét (ritka, lehetséges, valószínű) és a hatást (alacsony, közepes, magas). A két dimenzió alapján a kockázati szintet egy mátrixból olvassuk ki:

	Alacsony hatás	Közepes hatás	Magas hatás
Valószínű	Közepes	Magas	Kritikus
Lehetséges	Alacsony	Közepes	Magas
Ritka	Alacsony	Alacsony	Közepes

A kockázati regiszter táblázatban rögzíti az összes azonosított kockázatot, a becslést és a csökkentő intézkedést. Példa:

ID	Kockázat leírása	Kategória	Valószínűség	Hatás	Szint	Csökkentő intézkedés	Felelős	Határidő
R1	A fizetési szolgáltató API késleltetése miatt a felhasználói válaszidő meghaladja az SLO-t	Technikai	Valószínű	Magas	Kritikus	Aszinkron feldolgozás bevezetése (ADR-014)	Tech lead	2025-04-01
R2	A message broker üzemeltetési tudás hiánya a csapatban	Szervezeti	Lehetséges	Közepes	Közepes	Képzés, dedikált üzemeltetési runbook, managed broker használata	Platform lead	2025-04-15
R3	Érzékeny fizetési adat kerül a naplóba	Biztonsági	Lehetséges	Magas	Magas	Naplózási szűrő, PII redakció, biztonsági review	Security champion	2025-03-30

ID	Kockázat leírása	Kategória	Valószínűség	Hatás	Szint	Csökkentő intézkedés	Felelős	Határidő
R4	A replikációs késleltetés miatt az RPO becslés 10 s, ami nem teljesíti az 5 s-os követelményt	Technikai	Valószínű	Közepes	Magas	Szinkron replikáció vagy más adatbázismotor értékelése, további mérés	Adatbázis felelős	2025-04-10

A kockázati regiszterben a határidő és a felelős hozzárendelése biztosítja, hogy a kockázatkezelés ne maradjon szándéknyilatkozat. A kockázatcsökkentő intézkedések lehetnek:

- **Elkerülés**: az architektúra megváltoztatása, hogy a kockázat ne következzen be (pl. más integrációs minta).
- **Csökkentés**: a valószínűség vagy a hatás mérséklése (pl. timeoutok, retry, circuit breaker).
- **Áthárítás**: a kockázat átadása egy külső szolgáltatónak (pl. managed message broker).
- **Elfogadás**: tudatos döntés a kockázat vállalásáról, ha a csökkentés költsége nagyobb, mint a lehetséges hatás.

A review végén a kockázati regiszter és a nyitott kérdések listája adja a döntési javaslat alapját.

32.4.7 A döntés és a következő lépések

A review nem ér véget a megállapítások listájával. A döntéshozóknak – jellemzően a termékmenedzser, a tech lead és az architekt – formális döntést kell hozniuk a projekt továbbhaladásáról. Három tipikus kimenet létezik:

1. **Jóváhagyás**: az architektúra a rögzített követelményeknek megfelel, a kockázatok elfogadható szinten vannak, a következő lépés a bevezetési terv kidolgozása.
2. **Feltételes jóváhagyás**: az architektúra alapvetően megfelelő, de bizonyos kockázatokat csökkenteni kell, vagy további méréseket kell elvégezni a jóváhagyás véglegesítése előtt.
3. **Elutasítás vagy újraértékelés**: az architektúra nem teljesíti a kritikus követelményeket, vagy a kockázatok túl magasak; alternatívát kell kidolgozni.

A döntést érdemes ADR formában rögzíteni, még akkor is, ha az nem egy technológiai választás, hanem a projekt állapotára vonatkozó döntés. Példa:

```
# ADR-015: A záróprojekt architektúrájának feltételes jóváhagyása

## Kontextus
A 2025-03-15-i architektúra-review a rendelkezésre álló PoC mérések és a
kockázati regiszter alapján megállapította, hogy a kritikus S1 és S2
```

```
forgatókönyv teljesült, de az S3 forgatókönyv RPO értéke csak becslés, és
meghaladja az 5 s-os elvárást.

## Döntés
A review a záróprojekt architektúráját feltételesen jóváhagyja. A jóváhagyás
feltétele, hogy az R4 kockázatra vonatkozó csökkentő intézkedés (szinkron
replikáció vagy más adatbázis-motor értékelése) 2025-04-10-ig megtörténjen,
és az eredményt egy újabb review erősítse meg.

## Alternatívák
1. Azonnali jóváhagyás a becslés elfogadásával.
   Elvetve: a rendelkezésre állásra vonatkozó SLO sérülhet, ha a becslés téves.
2. Elutasítás és új adatbázis-architektúra kidolgozása.
   Elvetve: a jelenlegi megoldás a többi forgatókönyvben jól teljesít, a
   kockázat célzottan csökkenthető.

## Következmények
- A csapatnak 2025-04-10-ig további mérést kell végeznie.
- A bevezetési terv csak a feltétel teljesítése után véglegesíthető.
- A döntés kommunikációja a stakeholderek felé szükséges.

## Állapot
Elfogadott
```

A következő lépések a döntésből fakadnak: a kockázatcsökkentő intézkedések ütemezése, a hiányzó mérések elvégzése, valamint a bevezetési terv előkészítése. Ez utóbbi már átvezet az Architektúra-review checklist témájába, de itt elegendő annyit rögzíteni, hogy a review kimenetei bemenetként szolgálnak a kiadási checklisthez.

32.4.8 A bemutató: a döntést támogató kommunikáció

Az architektúra-review eredménye önmagában nem elegendő, ha a döntéshozók nem értik meg a trade-offokat. A bemutató – gyakran „architecture review meeting” vagy „design review” – célja, hogy a stakeholderek számára emészthető formában összefoglalja az értékelést, a kockázatokat és a döntési javaslatot. A sikeres bemutató nem a technikai részletek mélységéről szól, hanem arról, hogy a hallgatóság a saját felelősségi körében döntést tudjon hozni.

32.4.9 Célközönség és üzenet

A stakeholderek csoportjai eltérő nézőpontból vizsgálják ugyanazt az architektúrát:

- **Üzleti stakeholder**: milyen üzleti kockázatot jelent ez? Mikor lesz kész? Mennyibe kerül?
- **Termékmenedzser**: a funkcionális követelmények teljesülnek-e? Milyen kompromisszumok vannak a felhasználói élményben?
- **Fejlesztői csapat**: a döntések technikai következményei, a kódolási feladatok, az adósság.
- **Üzemeltetés**: a rendszer megfigyelhetősége, skálázhatósága, incidenskezelése.

A bemutató szerkezetét úgy kell felépíteni, hogy minden csoport megtalálja a számára releváns információt, de a döntési javaslat egyértelmű legyen.

32.4.10 Javasolt felépítés

A bemutató tipikus hossza 20–30 perc, 10–14 diával. A diák tartalmi váza:

1. **Cím és kontextus**: a záróprojekt neve, az értékelés időpontja, a résztvevők.

2. **Célok és minőségi követelmények összefoglalása**: 3–5 kulcsfontosságú minőségi attribútum, SLO-kkal.
3. **Architektúra áttekintés**: C4 context diagram, a rendszerhatárokkal és a külső szereplőkkel.
4. **Kritikus komponensek és döntések**: container szintű diagram, 2–3 meghatározó ADR kiemelése.
5. **Mérési eredmények**: a PoC legfontosabb mérései grafikonon, a mért és a várt értékek összehasonlítása.
6. **Kockázati összefoglaló**: a kockázati mátrix hőtérképként, a kritikus kockázatok kiemelése.
7. **Trade-off elemzés**: a legfontosabb kompromisszumok (pl. konzisztencia vs. rendelkezésre állás, költség vs. teljesítmény).
8. **Döntési javaslat**: jóváhagyás / feltételes jóváhagyás / elutasítás, indoklással.
9. **Következő lépések**: a kockázatcsökkentő intézkedések, határidők, felelősök.
10. **Nyitott kérdések**: amit még nem tudunk, és hogyan fogjuk eldönteni.

A bemutatóban használt diagramok diagrams-as-code formában készüljenek, például Mermaid segítségével, hogy a kódbázissal együtt verziózhatók legyenek. Egy egyszerű C4 context diagram:

```
graph TD
    U[Felhasználó] -->|HTTPS| W[Webes kliens]
    W -->|REST| A[Rendeléskezelő szolgáltatás]
    A -->|SQL| D[(Rendelés adatbázis)]
    A -->|Outbox + AMQP| B[Message broker]
    B --> F[Fizetési feldolgozó]
    F -->|HTTPS| P[Külső fizetési szolgáltató]
```

A diagram címe: „C4 context: a záróprojekt fő komponensei”. A jelmagyarázat: a szaggatott vonal rendszerhatárt jelöl, a nyílhegyek a protokollt és az irányt mutatják. Feltételjegyzet: a diagram a review időpontjában érvényes, a komponensek felelőssége az ADR-ekben részletezve.

32.4.11 Kommunikációs szabályok

A bemutató során több kommunikációs csapdát kell elkerülni:

- **Túl technikai**: a stakeholderek egy része nem ismeri a Kubernetes részleteit, ezért a konténer-orchestrációt csak akkor említsük, ha a döntés szempontjából releváns.
- **Nincs döntési javaslat**: a review összefoglaló nem önmagáért való; mindig fogalmazzunk meg egyértelmű javaslatot, még akkor is, ha az „további mérés szükséges”.
- **A kockázatok elhallgatása**: ha egy kockázatot nem mondunk ki, az később váratlanul fog megjelenni. A kockázat bemutatása nem gyengeség, hanem a felelős döntéshozatal része.
- **A bizonytalanság elrejtése**: a mérési eredményeknél mindig tüntessük fel a mérési ablakot, a konfidenciaintervallumot vagy a becslés jellegét. A „nem tudjuk pontosan” valid válasz, ha mellé tesszük, hogyan fogjuk megtudni.
- **A trade-offok leegyszerűsítése**: ne mutassunk be egyoldalúan egy megoldást; a döntési naplóból hozzuk az elvetett alternatívákat és azok következményeit.

32.4.12 Példa egy bemutató vázlatra

Az alábbi vázlat egy 12 diás bemutatót mutat be, a fenti szerkezet alapján:

1. **Cím**: „Záróprojekt architektúra-review – 2025. március 15.”
2. **Minőségi célok**: rendelkezésre állás 99,9%, válaszidő 95. p $<$ 500 ms, RPO 5 s, RTO 60 s.
3. **Architektúra áttekintés (C4 context)**: a fenti Mermaid diagram.
4. **Kulcsdöntések**: ADR-014 (aszinkron fizetés), ADR-010 (Kafka választás), ADR-008 (moduláris monolit bontás).
5. **PoC eredmények**: válaszidő grafikon, rendelkezésre állás mérés.
6. **Mért vs. várt táblázat**: S1, S2, S3 forgatókönyvek.
7. **Kockázati hőtérkép**: a mátrix színezett változata.
8. **Kritikus kockázatok**: R1, R4 kiemelése, csökkentő intézkedésekkel.
9. **Trade-off bemutatás**: konzisztencia vs. rendelkezésre állás – miért fogadtuk el az esetleges duplikátumokat.
10. **Döntési javaslat**: feltételes jóváhagyás, a feltétel megjelölésével.
11. **Következő lépések**: ütemterv a kockázatcsökkentésre, felelősökkel.
12. **Nyitott kérdések**: RPO pontos mérése, a message broker üzemeltetési modellje.

32.4.13 Labor: review és bemutató tartása

A labor célja, hogy a hallgató a Proof of Concept és mérés során létrehozott PoC és mérési eredményekre építve ténylegesen elvégezzen egy architektúra-review-t, kockázati regisztert készítsen, és bemutató vázlatot állítson össze. Az eredmény egy reprodukálható dokumentum, amely a döntési napló frissítését is tartalmazza.

32.4.14 Kiinduló állapot

Rendelkezésre áll:

- a záróprojekt architektúra-dokumentációja (C4 diagramok, komponensleírások),
- a döntési napló (ADR-ek),
- a Proof of Concept és mérés PoC mérési jegyzőkönyve, amely tartalmazza a környezetet, a konfigurációt, az időpontot és a bizonytalanságot,
- a minőségi forgatókönyvek listája.

32.4.15 Feladat lépései

1. **Válassz ki 3–5 kulcsfontosságú minőségi forgatókönyvet** a záróprojektből. Olyanokat válassz, amelyek a rendszer sikeressége szempontjából kritikusak (pl. rendelkezésre állás, válaszidő, adatbiztonság, skálázhatóság).

2. **Értékeld mindegyik forgatókönyvet** a PoC mérési eredmények alapján. Készíts táblázatot, amelyben szerepel az elvárt válasz, a mért vagy becsült érték, az értékelés (teljesítve / részben / nem teljesítve / ismeretlen) és a bizonyíték. A becsléseket explicit módon jelöld „becslés” címkével.
3. **Azonosíts 5–10 kockázatot**, és tölts ki kockázati regisztert. Minden kockázathoz adj kategóriát, valószínűséget, hatást, kockázati szintet, csökkentő intézkedést, felelőst és határidőt.
4. **Készíts bemutató vázlatot** 10–12 diára a fent bemutatott felépítés szerint. A vázlat minden diájához írj 2–4 mondatos tartalmi összefoglalót, és jelöld, hogy milyen vizuális anyag (diagram, grafikon, táblázat) kerülne rá. A döntési javaslatnak egyértelműnek kell lennie.
5. **Frissítsd a döntési naplót** egy új ADR-rel, amely rögzíti a review eredményét és a döntést (jóváhagyás / feltételes jóváhagyás / elutasítás). Az ADR struktúrája kövesse a könyv konvencióját: kontextus, döntés, alternatívák, következmények, állapot.
6. **Válaszd el a mért és a várt eredményeket** a dokumentumban. Például a mérési táblázatban külön oszlop legyen a mért érték (a PoC-ből, forrással és időponttal) és a várt érték (a minőségi forgatókönyvből). A becsléseket szintén jelöld.

32.4.16 Elvárt eredmény

Egy reprodukálható dokumentum (például Markdown vagy Jupyter notebook), amely tartalmazza:

- a review összefoglalóját,
- a szcenárió-értékelési táblázatot,
- a kockázati regisztert,
- a bemutató vázlatát,
- az új ADR-t,
- a mért és várt eredmények elkülönítését.

32.4.17 Ellenőrzési szempontok

- Minden kiválasztott forgatókönyvhöz tartozik értékelés és bizonyíték.
- A kockázati regiszter minden sora tartalmaz intézkedést és felelőst.
- A bemutató vázlatában szerepel döntési javaslat.
- A mért és várt adatok egyértelműen meg vannak különböztetve.
- Az ADR struktúrája és állapota megfelelő.
- A dokumentum reprodukálható: a mérési adatok forrása, időpontja és környezete visszakereshető.

32.4.18 Visszaállítás

A labor nem igényel éles környezetet, minden művelet dokumentum-szintű. Ha valamelyik lépést újra kell futtatni, elegendő a dokumentum korábbi verziójához visszatérni a verziókezelőben.

32.4.19 Ellenőrzőlista és önellenőrző kérdés

Az alábbi ellenőrzőlista segít a review és a bemutató minőségének gyors felmérésében:

- ☐ Minden kritikus minőségi forgatókönyv le van fedve a review-ban?
- ☐ A szcenárió-értékelések tartalmazzák a mért/becsült értéket és a bizonyítékot?
- ☐ Az ADR-ek naprakészek, és minden jelentős döntéshez tartozik ADR?
- ☐ A kockázati regiszter teljes, minden kockázathoz van csökkentő intézkedés?
- ☐ A bemutató vázlata tartalmaz döntési javaslatot és következő lépéseket?
- ☐ A bizonytalanság és a becslések explicit módon jelölve vannak?
- ☐ A bemutató nyelvezete a célközönséghez igazodik?

Önellenőrző kérdés: Mi a különbség a kockázat és a nyitott kérdés között, és hogyan kommunikálod mindkettőt úgy a bemutatóban, hogy az a döntéshozatalt támogassa? Válaszodban térj ki arra, hogy melyikhez rendelünk valószínűséget és hatást, és melyik az, amelyik további vizsgálatot igényel.

A review, a kockázatkezelés és a bemutató nem a projekt végét jelenti, hanem a következő iteráció kezdetét. A feltárt kockázatok és nyitott kérdések újabb tanulási ciklust indíthatnak – és ez így helyes. Az architektúra akkor válik a csapat közös tudásává, ha az értékelés eredményeit nemcsak dokumentáljuk, hanem a döntésekbe is visszavezetjük.

33 Kiadási checklist és továbbtanulás

33.1 Architektúra-review checklist

A kiadás előtti architektúra-review nem a fejlesztés utolsó adminisztratív lépése, hanem a csapat utolsó tanulási lehetősége, mielőtt a rendszer éles környezetbe kerül. A cél nem az, hogy valaki „kipipáljon” egy listát, hanem az, hogy a go döntés bizonyítékokon alapuljon, ne feltételezéseken. A záróprojektben a hallgató vagy a tervező csapat a Követelmények, kényszerek és minőségi forgatókönyvek–Architektúra-review, kockázat és bemutató lépések során előállította a követelményeket, a döntéseket, a kockázatokat és a méréseket. A review ezeket a szempontokat rendszerezi, és egyetlen, reprodukálható döntési pontba sűríti: mehet-e a kiadás, vagy sem.

A review fókusza a **követelmény, döntés, kockázat és dokumentáció** négyes. E négy pillér mindegyike más hibát képes felfedni. Ha a követelmény ellenőrzése hiányos, a rendszer lehet, hogy gyors, de nem azt csinálja, amit a felhasználó vár. Ha a döntési napló hiányos, a csapat nem tudja, miért választottak egy adott megoldást, és később rossz irányba refaktorál. Ha a kockázatokat nem rangsorolták, a release egy ismert, de figyelmen kívül hagyott hibával megy ki. Ha a dokumentáció elavult, a következő incidensnél senki sem tudja, hogyan kell beavatkozni. A checklist e négy területet járja körbe, és minden sorához bizonyítékot rendel.

33.1.1 Miért van szükség kiadás előtti architektúra-review-ra?

A kiadás előtti pillanatban a rendszer már működik, a tesztek lefutottak, a felhasználói történetek elkészültek. A fejlesztői csapat joggal érzi úgy, hogy minden rendben van. A review mégis más nézőpontot ad. A fejlesztő a kód szintjén látja a rendszert, a reviewer a rendszerhatárokat, a kapcsolatokat, a minőségi követelményeket és a döntések következményeit vizsgálja. Ez a különbség segít abban, hogy olyan problémákat is észrevegyünk, amelyek a napi munka során rejtve maradnak.

A review egyben kockázatcsökkentő tevékenység. Ha a release éles környezetbe kerül, a hibák költsége ugrásszerűen megnő. A review során felfedezett hiányosságokat még viszonylag olcsón lehet javítani. Ha viszont a hibás architektúra élesbe kerül, a javítás gyakran leállással, adatvesztéssel vagy ügyfél-elégedetlenséggel jár. A review tehát nem bürokratikus akadály, hanem gazdasági racionalitás.

A checklist emellett a csapat közös nyelvét is erősíti. Amikor a reviewer rákérdez egy minőségi követelményre, a csapat kénytelen pontosan megfogalmazni, mit jelent a „gyors”, „megbízható” vagy „biztonságos”. Ez a pontosítás már önmagában értékes, akkor is, ha a release végül go döntést kap.

33.1.2 A review tárgya és határai

A kiadás előtti architektúra-review nem ismétli meg a korábbi fejezetekben bemutatott értékelési módszereket. Nem kell újra elvégezni az ATAM-et vagy a threat modellezést, ha azok már megtörténtek. A review a meglévő dokumentumokra, mérésekre és kódállapotra támaszkodik,

és azt ellenőrzi, hogy azok konzisztensek, teljesek és naprakészek. A cél a release feltételeinek ellenőrzése, nem a teljes architektúra újraértékelése.

A review határait érdemes előre rögzíteni. Egy nagyobb rendszer esetén nem minden komponenst kell azonos mélységben vizsgálni. A fókuszt a release által érintett rendszerhatárokra, a kritikus minőségi követelményekre és a korábban azonosított magas kockázatú elemekre kell helyezni. Ha például a release egy új fizetési integrációt vezet be, a review a fizetési folyamat architektúráját vizsgálja részletesen, a többi részt pedig csak annyiban, amennyiben érinti őket a változás.

A review időpontja tipikusan a release előtti utolsó szakasz, de nagyobb változtatásoknál korábban is lehet. A cél, hogy a go/no-go döntés ne az utolsó pillanatban szülessen, hanem legyen idő a feltárt hibák javítására. Ezért a review ütemezése maga is architekturális döntés: a release tervbe kell építeni a review időablakát és a javítási tartalékot.

33.1.3 A checklist négy pillére

A checklist szerkezete négy fő területet fed le: **követelmény, döntés, kockázat és dokumentáció**. Mindegyikhez konkrét ellenőrzési szempontok tartoznak, amelyeket a záróprojektben elkészített anyagokkal kell összevetni.

33.1.4 1. Követelmény

A követelménypillér azt vizsgálja, hogy a rendszer a vállalt minőségi és funkcionális követelményeket teljesíti-e, és hogy ezek a követelmények mérhető formában léteznek. A korábbi fejezetekben bemutatott quality attribute scenario (Quality attribute scenarios) itt válik éles ellenőrzési eszközzé. Minden kritikus minőségi követelményt inger, környezet, válasz és mérőszám formában kell megfogalmazni. A review ezeket a forgatókönyveket veszi elő, és ellenőrzi, hogy van-e hozzájuk mérés.

Példa egy quality attribute scenario-ra:

> **Inger:** 500 egyidejű felhasználó rendelést ad le.
> **Környezet:** normál üzemi környezet, staging klaszter, 2 vCPU / 4 GB node.
> **Válasz:** a rendelésleadási kérés sikeresen feldolgozódik.
> **Mérőszám:** a válaszidő 95%-os percentilise legfeljebb 300 ms.

A review során ellenőrizni kell, hogy ehhez a forgatókönyvhöz tartozik-e valós, reprodukálható mérés. A mérési jegyzőkönyvnek tartalmaznia kell a környezetet, a konfigurációt, az időpontot és a mérési ablakot. A mért értéket külön kell választani a várt küszöbtől: a várt a követelményben rögzített cél, a mért a teszt vagy monitoring során kapott érték. Ha a mért érték hiányzik, a review nem „jóindulatból” elfogadja, hanem hiányosságként rögzíti.

A követelménypillér további szempontjai:

- Minden funkcionális követelmény lefedett-e a release scope-ban?
- A nem funkcionális követelmények mindegyikéhez tartozik-e SLI és SLO? (SLO, SLI, error budget és mérés)
- A rendelkezésre állási SLO konzisztens-e a mérési ablakkal? Például ha az SLO 99,9%, akkor a 30 napos ablakban az üzemidő aránya eleget tesz-e az $A = \frac{\text{üzemidő}}{\text{üzemidő+állásidő}}$ képletnek.
- A kapacitástervezés során a Little-törvényt ($L = \lambda W$) alkalmazták-e a várt érkezési rátára és tartózkodási időre?
- A jogi, szervezeti és technológiai kényszerek rögzítettek-e, és teljesülnek-e?
- A követelmények változása esetén van-e nyomon követhető folyamat?

A gyakori hiba itt az, hogy a csapat a követelményeket általánosan fogalmazza meg – „a rendszer gyors” –, és nincs mérhető küszöb. A review ilyenkor nem tudja eldönteni, hogy a rendszer megfelel-e. A checklist első feladata tehát a mérhetőség számonkérése.

33.1.5 2. Döntés

A döntéspillér az architektúradöntések nyomon követhetőségét és konzisztenciáját vizsgálja. Minden jelentős architektúradöntéshez ADR-nek kell tartoznia (Architecture Decision Records (ADR) és döntési napló). Az ADR-nek tartalmaznia kell a kontextust, a döntést, az alternatívákat, a következményeket és az állapotot. A review során ellenőrizni kell, hogy az ADR-ek naprakészek, és a kódban a döntésnek megfelelő megvalósítás szerepel.

Példa egy ADR ellenőrzésére:

> **ADR-014:** Az API gateway a BFF mintát követi.
> **Kontextus:** a mobil kliens más adatokat igényel, mint a webes kliens.
> **Döntés:** minden kliens saját BFF-en keresztül hívja a belső szolgáltatásokat.
> **Alternatívák:** egyetlen általános API gateway, közvetlen szolgáltatáshívás.
> **Következmények:** a kliensek száma növeli a BFF-ek számát, de csökkenti a hálózati forgalmat.

A review feladata, hogy ellenőrizze, a kódban valóban a BFF-en keresztül mennek-e a hívások, vagy vannak-e közvetlen hívások a kliensből a belső szolgáltatásokhoz. Ha ilyet talál, az nem feltétlenül release-blokkoló, de rögzíteni kell, és dönteni kell a javításról vagy az ADR módosításáról.

A döntéspillér további szempontjai:

- Minden jelentős architektúradöntés szerepel-e az ADR-naplóban?
- Az ADR-ek tartalmazzák-e az alternatívákat és a következményeket?
- A döntések állapota egyértelmű-e (javasolt, elfogadva, elavult, visszavont)?
- Van-e olyan döntés, amelyet a kód nem követ?
- A döntéseket a megfelelő stakeholder jóváhagyta-e?
- A technikai adósság rögzítve van-e, és a release scope tud róla?

A gyakori hiba, hogy a csapat a döntéseket a fejlesztői chatben vagy szóban hozza meg, és az ADR csak utólag, hiányosan készül el. A review ilyenkor felfedi a döntés és a dokumentáció közötti szakadékot. A helyes megoldás, ha a review visszaküldi a döntést dokumentálásra, vagy a release-hez kapcsolódó kockázatként kezeli.

33.1.6 3. Kockázat

A kockázatpillér azt vizsgálja, hogy az ismert kockázatokat azonosították, rangsorolták, és a magas kockázatokhoz ellenőrző intézkedéseket rendeltek. A záróprojektben a kockázati regiszternek már léteznie kell. A review során a regiszterből kiindulva ellenőrizni kell, hogy a kritikus kockázatokhoz tartozik-e mérés, teszt vagy fallback.

A kockázatokat valószínűség és hatás szerint kell rangsorolni. A magas valószínűségű és nagy hatású kockázatok a release előtt nem maradhatnak kezelés nélkül. Ha például a rendszer egy külső fizetési szolgáltatótól függ, a kockázat az, hogy a szolgáltató leállása elveszítheti a rendeléseket. A kezelés lehet az outbox minta, a retry mechanizmus vagy egy fallback szolgáltató. A review ellenőrzi, hogy ezek az intézkedések a kódban és a konfigurációban valóban jelen vannak, és tesztelve vannak.

A kockázatpillér további szempontjai:

- Van-e kockázati regiszter, és naprakész-e?
- A kockázatokat rangsorolták-e valószínűség és hatás alapján?
- A magas kockázatú elemekhez van-e mérés vagy teszt?
- A biztonsági fenyegetéseket modellezték-e (threat model, Threat modeling és támadási felület)?
- A reziliencia intézkedések – timeout, retry, circuit breaker, bulkhead – konfigurációja ellenőrzött-e?
- A külső függőségek (adatbázis, felhő, külső API) kockázatai ismertek-e?
- A chaos engineering vagy játéknapok (Chaos engineering és játéknapok) eredményei alapján vannak-e nyitott hibák?

A gyakori hiba, hogy a kockázati regiszter létezik, de senki sem nézi meg a release előtt. A review ezt a regisztert használja eszközként: a magas kockázatú sorokat sorra veszi, és mindegyikhez bizonyítékot kér. Ha a bizonyíték hiányzik, a release nem kaphat feltétel nélküli go döntést.

33.1.7 4. Dokumentáció

A dokumentációpillér a diagramok, a runbookok és a release-hez kapcsolódó anyagok ellenőrzését foglalja magában. A C4 modell szerint a context, container és component szint kötelező, a code szint csak indokolt esetben készül. A review során ellenőrizni kell, hogy a diagramok diagrams-as-code formában készültek-e, verziózottak-e, és a generált ábra megfelel-e a forrásnak.

A diagramokon a rendszerhatárt minden szinten jelölni kell, és a külső rendszereket, felhasználói szerepköröket fel kell tüntetni. A kapcsolatokon szerepelnie kell a protokollnak, a szinkronitásnak és a hibatűrésnek. Ha például a komponensdiagramon egy kapcsolat „HTTP, szinkron, nincs retry”, akkor ez az információnak szerepelnie kell a diagramon vagy a kapcsolódó leírásban.

A dokumentációpillér további szempontjai:

- A diagramok naprakészek-e, és a kódbázisból generálhatók-e?
- A C4 context, container és component szintek elkészültek-e?
- A dokumentáció nem tartalmaz-e valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot?
- A runbookok, riasztási szabályok és incidenskezelési eljárások elérhetők-e?
- A release-hez tartozó döntési napló és a review jegyzőkönyv verziózott-e?
- A mérési jegyzőkönyvek tartalmazzák-e a környezetet, a konfigurációt, az időpontot és a bizonytalanságot?

A gyakori hiba, hogy a dokumentáció a release előtt pár nappal készül, és nem követi a kódot. A review ezt az eltérést feltárja. A helyes megoldás, ha a dokumentáció a fejlesztéssel együtt, folyamatosan frissül, és a review csak a végső ellenőrzést végzi.

33.1.8 A review folyamata

A kiadás előtti architektúra-review nem egyetlen találkozó, hanem egy rövid, de strukturált folyamat. A lépések a következők:

1. **Kontextus rögzítése.** A reviewer és a csapat közösen rögzíti a release hatókörét, a kritikus minőségi követelményeket és azokat a kockázatokat, amelyeket a review során részletesen vizsgálni kell. Ez a lépés megakadályozza, hogy a review parttalanná váljon.

2. **Bizonyítékgyűjtés.** A csapat összegyűjti a checklist kitöltéséhez szükséges anyagokat: mérési jegyzőkönyveket, ADR-eket, diagramokat, kockázati regisztert, teszt eredményeket. A bizonyítékokat a review előtt megosztják, hogy a reviewer felkészülhessen.

3. **Checklist kitöltése.** A reviewer a checklist minden sorát megvizsgálja, és rögzíti az állapotot: megfelel, nem felel, feltételes, vagy nincs bizonyíték. A mért és várt értékeket külön oszlopban tünteti fel.

4. **Értékelés és rangsorolás.** A feltárt hiányosságokat és kockázatokat rangsorolják. Nem minden hiányosság blokkolja a release-t. A kritikus követelményt sértő hiányosságok no-go-hoz vezetnek, a kisebb dokumentációs eltérések feltételes go-val is elfogadhatók, ha a javítási határidő rögzített.

5. **Döntés.** A reviewer vagy a döntési jogkörrel rendelkező személy go/no-go döntést hoz. A döntést indokolni kell, és a kivételeket dokumentálni kell. A review eredménye maga is ADR-be vagy jegyzőkönyvbe kerül, hogy a döntés visszakereshető legyen.

A folyamat során a reviewer nem a csapat ellensége, hanem a minőség utolsó ellenőrzője. A cél a közös tanulás, nem a bűnbakkeresés. A pszichológiai biztonság elengedhetetlen ahhoz, hogy a csapat őszintén feltárja a hiányosságokat. Ha a csapat attól fél, hogy a review hibái büntetést vonnak maguk után, elrejti a problémákat, és a release hamis go döntéssel megy ki.

33.1.9 Go/no-go döntés és kivételek

A go/no-go döntés a review kimenete. A **go** azt jelenti, hogy a release a vállalt követelményekkel és kockázati szinttel élesbe engedhető. A **no-go** azt jelenti, hogy a release nem mehet ki, amíg a kritikus hiányosságokat nem javítják. A **feltételes go** a kettő közötti állapot: a release mehet, de bizonyos feltételekkel, például a következő release előtt javítani kell egy dokumentációs eltérést, vagy a magas kockázatú komponenst fokozott monitorozás alá kell helyezni.

A go feltételei jellemzően a következők:

- A kritikus minőségi követelményekhez tartozó mérések teljesítik a várt küszöböt.
- A magas kockázatú elemekhez van elfogadott kezelés, és a kezelés tesztelt.
- Minden jelentős architektúradöntés ADR-rel rendelkezik, és a kód a döntésnek megfelelően működik.
- A dokumentáció naprakész, és nem tartalmaz tiltott adatokat.
- A release-hez kapcsolódó incidenskezelési és visszaállítási eljárások elérhetők.

A no-go feltételei:

- A kritikus minőségi követelmény nem teljesül, és nincs elfogadott kockázatvállalás.
- A bizonyíték hiányos, és nincs idő vagy mód a mérés elvégzésére.
- A magas kockázatú elemhez nincs kezelés, vagy a kezelés nem működik.
- A dokumentációban valós hitelesítő adat vagy éles URL szerepel.
- A release hatóköre nincs egyértelműen rögzítve.

A **kivételek** a go/no-go döntés alóli tudatos eltérések. Ilyen lehet például egy sürgős biztonsági javítás, amelynek a késlekedése nagyobb kárt okoz, mint a review hiányossága. Ilyenkor a release mehet, de a kivétel okát, a döntéshozót és a kockázatvállalást dokumentálni kell. A kivétel nem a checklist megkerülése, hanem egy másik kockázati döntés: a csapat azt vállalja, hogy a review hiányosságát a release után pótolja, vagy a kockázatot elfogadja. A kivételt mindig a megfelelő döntési jogkörrel rendelkező személy hagyja jóvá, és a döntési naplóban rögzítik.

Példa kivételre:

> **Kivétel-003:** A fizetési szolgáltató új API-verziójának integrációja sürgős, mert a régi API-t a szolgáltató 24 órán belül leállítja. A release a fizetési outbox hiányos tesztelése ellenére megy, mert a régi API leállása azonnali üzleti kárt okoz. A hiányzó outbox-tesztet a release után 5 munkanapon belül pótolják. Felelős: platform csapat vezető.

A kivételeknek nyoma kell legyen, különben a csapat idővel hozzászokik ahhoz, hogy a checklistet meg lehet kerülni. A kivételek nyilvántartása segít abban is, hogy a rendszeres kivételekből mintázatok rajzolódjanak ki, és a csapat a folyamatot javítani tudja.

33.1.10 Tipikus hibák és felismerésük

A review során a leggyakoribb hibák nem technikai jellegűek, hanem a folyamattal vagy a gondolkodásmóddal kapcsolatosak. Ezeket érdemes tudatosan keresni.

33.1.11 1. A checklist mechanikus kipipálása

A csapat minden sor mellé „megfelel” jelölést tesz, de a bizonyíték hiányzik. A review így hamis biztonságérzetet ad. A felismerés módja: ha a checklist egy sorához nincs hivatkozott mérés, ADR vagy diagram, akkor az állapot nem lehet „megfelel”. A helyes megoldás: minden „megfelel” mögött konkrét bizonyítéknak kell állnia.

33.1.12 2. A mért és várt érték összemosása

A csapat azt írja, hogy a válaszidő 267 ms, de kiderül, hogy ez egy korábbi becslés, nem mért adat. A felismerés módja: a review során minden mért értéknél ellenőrizni kell a környezetet, az időpontot és a mérési ablakot. Ha ezek hiányoznak, az érték nem fogadható el. A helyes megoldás: a checklistben külön oszlopok szerepelnek a várt küszöbre és a mért értékre, és a mért értékhez forrás tartozik.

33.1.13 3. A diagram és a kód eltérése

A C4 diagramon a komponens egy módon jelenik meg, a kódban viszont más a kapcsolat. A review ezt akkor észleli, ha a diagramot a kóddal együtt vizsgálja. A felismerés módja: a reviewernek a diagram forrását és a kódot is meg kell nyitnia. A helyes megoldás: a diagramok diagrams-as-code formában készülnek, és a CI/CD pipeline a forrásból generálja az ábrát, így az eltérés felfedezhető.

33.1.14 4. A kockázatok nem rangsoroltak

A kockázati regiszterben minden kockázat egyenlő súllyal szerepel, és a magas kockázatok elvesznek. A felismerés módja: ha a regiszterben nincs valószínűség és hatás oszlop, vagy az értékek nem indokoltak. A helyes megoldás: a regiszterben minden kockázathoz tartozik valószínűség, hatás és egy rangsorolt prioritás, és a review a magas prioritásúakat vizsgálja először.

33.1.15 5. A go döntés nyomásra

A határidő vagy a stakeholder nyomása miatt a review go döntéssel zárul, pedig a bizonyíték hiányos. A felismerés módja: ha a döntési jegyzőkönyvben a go indoklása nem a checklist eredményeire hivatkozik, hanem a határidőre. A helyes megoldás: a döntési jogkört el kell választani a szállítási nyomástól. A reviewernek vagy a döntéshozónak joga van no-go-t mondani, és ezt a jogot a szervezeti folyamat garantálja.

33.1.16 6. A dokumentációban valós adatok szerepelnek

A release dokumentációjában valós hitelesítő adat, belső URL vagy ügyfélnév szerepel. Ez adatvédelmi és biztonsági hiba. A felismerés módja: a review során a dokumentációt át kell nézni ilyen elemek keresésére. A helyes megoldás: a dokumentáció-szerződés szerint minden ilyen adat helyett helyőrzőt kell használni, és a review ezt ellenőrzi.

33.1.17 Labor: review checklist kitöltése a záróprojekthez

Ennek a fejezetnek a gyakorlati célja, hogy a hallgató a saját záróprojektjére alkalmazza a review checklistet. A labor nem éles rendszeren fut, hanem a záróprojekt dokumentációjának és mérési anyagainak ellenőrzését jelenti. A cél a checklist módszerének elsajátítása, nem a release tényleges kiadása.

33.1.18 Kiinduló állapot

A hallgató a Követelmények, kényszerek és minőségi forgatókönyvek–Architektúra-review, kockázat és bemutató lépések során elkészítette a záróprojekt architektúráját. Rendelkezésére áll:

- a követelmények és a quality attribute scenario-k dokumentuma,
- az ADR-napló,
- a kockázati regiszter,
- a C4 diagramok diagrams-as-code forrásai,
- a PoC mérések jegyzőkönyvei,
- a kód vagy a komponensek leírása.

33.1.19 Feladat

Készítsen egy reprodukálható review dokumentumot, amely a checklist négy pillérén alapul. A dokumentum tartalmazzon legalább tíz ellenőrzési szempontot, és mindegyikhez bizonyítékot, várt küszöböt, mért értéket (vagy annak hiányát) és állapotot. A mért és várt eredményeket külön oszlopban kell feltüntetni. Ha egy mért érték hiányzik, a hiányt rögzíteni kell, és javaslatot kell tenni a mérésre.

A dokumentum végén rögzíteni kell a review döntését: go, no-go vagy feltételes go. A döntést indokolni kell, és a kivételeket, ha vannak, dokumentálni kell.

33.1.20 Ellenőrzési szempontok

A review dokumentum akkor megfelelő, ha:

- a checklist legalább tíz szempontot tartalmaz a négy pillérből,
- minden szempont mellé bizonyíték van rendelve,
- a várt és mért értékek külön oszlopban szerepelnek,
- minden mért értékhez tartozik környezet, konfiguráció, időpont és mérési ablak,
- a döntési naplóban a go/no-go döntés indokolt,
- a dokumentum nem tartalmaz valós hitelesítő adatot, belső URL-t vagy személyes adatot.

33.1.21 Visszaállítási út

A labor nem érint éles rendszert, ezért visszaállításra nincs szükség. A dokumentum verzióját azonban meg kell őrizni, hogy a review eredménye később visszakereshető legyen.

33.1.22 Mintatáblázat a checklisthez

Az alábbi táblázat egy illusztráció, nem végrehajtandó mérés. A mért értékek elképzeltek, és a laborban a hallgató saját adataival helyettesíti őket.

Azonosító	Ellenőrzési szempont	Bizonyíték	Várt küszöb	Mért érték	Állapot
RQ-01	Rendelésleadás válaszideje 500 egyidejű felhasználó alatt	Terhelési teszt jegyzőkönyv, staging, 2 vCPU/4 GB, 2025-06-01	p95 ≤ 300 ms	p95 = 267 ms	Megfelel
RQ-02	Rendelkezésre állás 30 napos ablakban	Monitoring adatok, 30 nap, production-like	SLO ≥ 99,9%	99,94%	Megfelel
ADR-01	API gateway BFF minta követése	ADR-014, kódvizsgálat	Minden kliens BFF-en keresztül hív	2 közvetlen hívás	Nem felel
RSK-01	Fizetési szolgáltató leállása esetén a rendelés ne vesszen el	Outbox teszt jegyzőkönyv	Outbox üzenetek újrapróbálva	Outbox sikeresen újrapróbál	Megfelel
DOC-01	C4 diagramok naprakészek	diagrams-as-code repo	Generált diagram a forrásból előállítható	2 diagram elavult	Feltételes

A táblázat egy sorában az állapot „Nem felel”. Ez a release szempontjából nem feltétlenül no-go, de a döntési naplóban rögzíteni kell, hogy a hibát a release előtt vagy után javítják. A labor célja

éppen ez: a hallgató megtanulja, hogy a checklist nem csak fekete-fehér, hanem a döntéseket indokolni kell.

33.1.23 Önellenőrző kérdés

Soroljon fel három tipikus hibát a kiadás előtti architektúra-review során, és magyarázza el, miért vezethetnek téves go döntéshez. A válaszban térjen ki arra, hogyan lehet ezeket a hibákat a checklist segítségével felismerni.

33.1.24 Gyakorlat

Válasszon ki egy architektúradöntést a saját záróprojektjéből. Írjon egy review bejegyzést, amely tartalmazza az ellenőrzési szempontot, a bizonyítékot, a várt küszöböt, a mért értéket és az állapotot. Ha nincs mért érték, jelölje a hiányt, és javasoljon mérést. A bejegyzést a döntési naplóba kell helyezni.

A kiadás előtti architektúra-review a záróprojekt utolsó minőségi kapuja. A checklist négy pillére – követelmény, döntés, kockázat, dokumentáció – együttesen biztosítja, hogy a go döntés ne reményen, hanem bizonyítékon alapuljon. A review nem a fejlesztők munkájának minősítése, hanem a rendszer és a dokumentáció konzisztenciájának ellenőrzése. Aki a checklistet nem mechanikus pipálásként, hanem tanulási eszközként használja, az a release előtt még egyszer rálát a rendszer egészére, és olyan összefüggéseket is észrevesz, amelyek a napi fejlesztés során rejtve maradtak. A következő alfejezetben a dokumentációs és megfelelési csomag összeállítása épít erre a review-ra: a go döntés után a csapatnak össze kell állítania azt a csomagot, amely a release-t jogi és üzemeltetési szempontból is alátámasztja.

33.2 Dokumentációs és megfelelési csomag

Amikor az architektúra-review ellenőrzőlistája lezárult, a következő lépés a kiadási csomag dokumentációs részének összeállítása. A dokumentáció nem egyszerűen a rendszer leírása, hanem olyan eszköz, amely lehetővé teszi, hogy egy új csapattag, egy auditor vagy a jövőbeli üzemeltető reprodukálhassa a rendszer állapotát, megértse a döntések okát, és ellenőrizhesse a megfelelést. Ez a fejezet ezt a reprodukálható és auditálható dokumentációs csomagot építi fel: bemutatja az alapfogalmakat, a gyakorlati eszközöket – C4, ADR, runbook, modellkártya –, a verziókezelés, jóváhagyás és megőrzés követelményeit, valamint egy kis, ellenőrizhető példán és laboron keresztül gyakoroltatja az összeállítást.

33.2.1 Reprodukálható és auditálható dokumentáció

A hagyományos architektúradokumentumok gyakran statikus leírások: egy vastag PDF, amely a tervezés pillanatában igaz volt, de a megvalósítás során elavulttá válik. Az ilyen dokumentum nem reprodukálható, mert nem kötődik a kód valós állapotához, és nem auditálható, mert nem lehet visszakeresni, ki, mikor és miért hozott egy döntést. Ezzel szemben a reprodukálható és auditálható dokumentáció a kódbázissal együtt él, verziózott, és minden állításához visszakereshető bizonyítékot rendel.

A reprodukálhatóság azt jelenti, hogy a dokumentáció alapján bárki – megfelelő hozzáféréssel – újra létre tudja hozni a rendszer egy adott állapotát, vagy legalábbis megértse annak felépítését, konfigurációját és üzemeltetési eljárásait. Ehhez a dokumentációnak szorosan kapcsolódnia kell a kódhoz, a konfigurációhoz és az infrastruktúra definíciójához. A diagramoknak a tényleges komponenseket kell tükrözniük, a runbookok lépéseinek valós parancsokat és visszaállítási utakat kell tartalmazniuk, a méréseknek pedig rögzíteniük kell a környezetet, a konfigurációt, az időpontot és a bizonytalanságot.

Az auditálhatóság ennél szigorúbb követelmény: a dokumentációnak lehetővé kell tennie, hogy egy külső fél – például egy belső auditor, egy szabályozó hatóság vagy egy ügyfél – ellenőrizhesse, hogy a rendszer megfelel az előírásoknak, a döntések nyomon követhetők, és a változások jóváhagyott folyamaton mentek keresztül. Ehhez minden dokumentumnak verzióval, szerzővel, jóváhagyóval és időbélyeggel kell rendelkeznie. Az architektúradöntéseknek hivatkozniuk kell az érintett komponensekre és követelményekre, a változtatásoknak pedig audit trail-t kell hagyniuk.

A dokumentációs csomag tehát nem egyetlen dokumentum, hanem egy összefüggő gyűjtemény, amely a következő elemekből áll: architektúradiagramok (elsősorban C4), döntési napló (ADR), üzemeltetési eljárások (runbook), modellkártyák (ha van gépi tanulási komponens), valamint a verzió-, jóváhagyási és megőrzési metaadatok. A csomagot a kóddal együtt, verziózott repository-ban kell tárolni, és a kiadási folyamat részévé kell tenni.

33.2.2 C4 diagramok mint kód

A C4 modell négy absztrakciós szintet különböztet meg: context, container, component és code. A könyv korábbi fejezeteiben már bemutattuk a C4 alapelveit, itt a dokumentációs csomagban betöltött szerepét hangsúlyozzuk. A kiadási csomagban a context, container és component szint kötelező; a code szint csak indokolt esetben készül, mert gyorsan elavul és ritkán ad hozzá auditálható értéket.

A kulcskövetelmény: a diagramok diagrams-as-code formában készüljenek – például Mermaid, PlantUML vagy Structurizr DSL segítségével –, hogy a kódbázissal együtt verziózhatók legyenek, és a változások nyomon követhetők legyenek. A statikus képfájlok (PNG, PDF) nem elégségesek, mert nem diffelhetők, és nehéz őket naprakészen tartani. A diagrams-as-code lehetővé teszi a pull request alapú review-t, az automatikus renderelést és a hivatkozások beágyazását.

Minden diagramon jelölni kell a rendszerhatárt, a külső rendszereket és a felhasználói szerepköröket a context szinten. A komponenseket nem elég névvel megadni: mindegyikhez felelősséget és interfészt kell rendelni. A kapcsolatokon szerepelnie kell a protokollnak, a szinkronitásnak és a hibatűrésnek. Például egy context diagramon ne csak annyi álljon, hogy „A komponens hívja B komponenst”, hanem azt is, hogy „HTTPS felett szinkron REST hívás, timeout 5 másodperc, újrapróbálkozás nélkül”.

Az alábbi Mermaid kódrészlet egy egyszerű rendelés-feldolgozó rendszer context szintű diagramját mutatja. A rendszerhatárt kommentár jelöli, és a külső rendszerek, szereplők egyértelműen el vannak különítve.

```
flowchart LR
    %% Rendszerhatár: Rendelés-feldolgozó rendszer
    subgraph "Rendelés-feldolgozó rendszer"
        API[Rendelés API]
        Worker[Rendelés-feldolgozó worker]
        DB[(Rendelés adatbázis)]
    end

    User[Vásárló] -->|HTTPS, szinkron REST| API
```

```
Admin[Adminisztrátor] -->|HTTPS, szinkron REST| API
API -->|AMQP, aszinkron üzenet| Worker
Worker -->|JDBC, szinkron SQL| DB
Worker -->|HTTPS, szinkron REST| PayExt[Külső fizetési szolgáltató]
API -->|HTTPS, szinkron REST| NotifyExt[Értesítési szolgáltató]
```

Ebben a diagramban a rendszerhatáron belül három konténer található: az API, a worker és az adatbázis. A külső rendszerek: a külső fizetési szolgáltató és az értesítési szolgáltató, valamint a felhasználói szerepkörök: vásárló és adminisztrátor. A kapcsolatokon feltüntettük a protokollt (HTTPS, AMQP, JDBC) és a szinkronitást (szinkron vagy aszinkron). A hibatűrési jellemzőket – például retry, timeout – a komponens szintű diagramon kell részletezni.

A container szint a fenti konténerek belső felépítését és a köztük lévő adatfolyamot mutatja, a component szint pedig például az API-n belüli szolgáltatásokat: `RendelésController`, `RendelésService`, `PaymentGatewayClient`, `RendelésRepository`. Minden komponenshez a forráskódban hivatkozás tartozik, így a diagram és a kód közötti kapcsolat ellenőrizhető. A component szintű diagramnál az interfészeket is le kell írni: például a `PaymentGatewayClient` interfész a `processPayment(orderId, amount)` metódust definiálja, és a kapcsolat a külső fizetési szolgáltató felé szinkron HTTPS hívás, timeout 5 másodperc, 1 újrapróbálkozással.

A dokumentációs csomagban minden C4 diagramhoz tartozik egy rövid leírás, amely megadja a diagram célját, a feltüntetett elemek forrását (melyik kód vagy konfiguráció alapján készült), valamint az esetleges egyszerűsítéseket. Ez segít az auditoroknak megérteni, hogy a diagram mennyire tekinthető aktuálisnak.

33.2.3 ADR-ek: a döntési napló

Az Architecture Decision Record (ADR) az architektúradöntések nyomon követésének alapvető eszköze. Egy ADR nem csupán azt rögzíti, hogy mit döntöttünk, hanem azt is, hogy miért, milyen alternatívák közül választottunk, és milyen következményekkel jár a döntés. A dokumentációs csomagban minden jelentős architektúradöntéshez ADR tartozik, és az ADR-ek hivatkoznak az érintett C4 diagramokra, kódkomponensekre és minőségi követelményekre.

Az ADR-ek száma nem cél, hanem eszköz. Egy ADR akkor jó, ha a jövőbeli olvasó – aki nem vett részt a döntésben – megérti belőle a döntés okát és a feladott alternatívákat. A túl sok, lényegtelen ADR zajt kelt; a túl kevés pedig elveszíti a nyomon követhetőséget. Általában azok a döntések igényelnek ADR-t, amelyek nehezen visszafordíthatók, széles körű hatással bírnak, vagy vitát váltottak ki.

Egy ADR minimális szerkezete a következő mezőket tartalmazza:

- **Cím:** rövid, egyértelmű azonosító (pl. „ADR-0007: Aszinkron rendelés-visszaigazolás”).
- **Státusz:** javasolt, elfogadott, elutasított, elavult, felváltotta másik ADR.
- **Kontextus:** a döntési helyzet leírása, beleértve az érintett követelményeket és kényszereket.
- **Döntés:** a választott megoldás pontos leírása.
- **Alternatívák:** legalább két másik lehetőség, és miért vetettük el őket.
- **Következmények:** a döntés pozitív és negatív hatásai, beleértve a kompromisszumokat.
- **Hivatkozások:** kapcsolódó ADR-ek, C4 diagramok, kódkomponensek, minőségi forgatókönyvek.
- **Dátum, szerző, jóváhagyó.**
- **Verzió:** az ADR maga is verziózott, ha módosul, új verziót kap, és a régi megőrződik.

Példa egy ADR-re a fenti rendelés-feldolgozó rendszerből:

```
# ADR-0007: Aszinkron rendelés-visszaigazolás

Státusz: Elfogadva
Dátum: 2025-02-10
Szerző: Kovács Anna
Jóváhagyta: Architektúra Tanács
Verzió: 1.2

Kontextus:
A rendelés-feladás során a külső fizetési szolgáltató válasza a terheléstől függően 2-30
 ↪ másodpercet vehet igénybe. A szinkron hívás blokkolja a felhasználói kérést, és megnöveli az
 ↪ API p95 válaszidejét. A QAS-003 minőségi forgatókönyv szerint a rendelés-feladás p95
 ↪ válaszideje nem haladhatja meg az 500 ms-ot.

Döntés:
A fizetési visszaigazolást aszinkron módon, RabbitMQ üzenetsoron keresztül dolgozzuk fel. A
 ↪ rendelés API azonnal létrehozza a rendelést „feldolgozás alatt" státusszal, majd egy
 ↪ üzenetet küld a workernek. A worker szinkron hívással lekérdezi a fizetési állapotot, és
 ↪ frissíti a rendelést.

Alternatívák:
1. Szinkron hívás 10 másodperces timeout-tal és 3 újrapróbálkozással.
   - Elutasítva: nem teljesíti a QAS-003 válaszidő-követelményt, és a fizetési szolgáltató
    ↪ lelassulása esetén az API szálkészlete kimerül.
2. Polling: a rendelés API rendszeresen lekérdezi a fizetési szolgáltatót.
   - Elutasítva: felesleges terhelést generál, és nehéz a terhelést jósolni.
3. Webhook a fizetési szolgáltatótól.
   - Elutasítva: a külső szolgáltató nem támogat megbízható webhookot, és a tűzfal konfiguráció
    ↪ bonyolult.

Következmények:
- Előny: az API válaszideje 200 ms alá csökken, a rendszer jobban skálázódik.
- Hátrány: növekszik a komplexitás - üzenetsor, worker, idempotencia kezelése, kompenzáló
 ↪ tranzakció szükséges sikertelen fizetés esetén.
- A rendelés státuszának konzisztenciája végső soron konzisztens lesz, amit az ügyfélnek is
 ↪ kommunikálni kell.

Hivatkozások:
- C4 container diagram: docs/architecture/containers.md
- C4 component diagram: docs/architecture/components-api.md
- ADR-0002 (RabbitMQ bevezetése)
- QAS-003 (rendelés-feladás válaszidő)
```

Az ilyen ADR-ek a repository `docs/adr/` mappájában, egyenként külön fájlban tárolódnak, és a fájl neve tartalmazza a sorszámot és a rövid címet, például `0007-async-payment-confirmation.md`. Az ADR-ek összefoglaló listája egy `README.md`-ben található, amely lehetővé teszi a gyors keresést. Az ADR-eket ugyanúgy code review alá kell vonni, mint a forráskódot: a státusz „javasolt” értékről „elfogadott” értékre csak a döntéshozó testület jóváhagyása után vált.

33.2.4 Runbook: az üzemeltetési eljárások

A runbook a dokumentációs csomag azon része, amely a reprodukálhatóságot közvetlenül szolgálja: konkrét, lépésről lépésre követhető eljárásokat ad az üzemeltetési feladatokhoz, hibaelhárításhoz és vészhelyzetekhez. A runbook nem elméleti leírás, hanem olyan parancssorozat, amelyet egy ügyeletes mérnök stresszhelyzetben is követni tud.

A runbook minden lépése mellett meg kell adni az elvárt eredményt és az ellenőrzés módját. A destruktív műveleteknél – például adatbázis-törlés, klaszter-átkonfigurálás – a runbooknak

tartalmaznia kell a visszaállítási utat is, és csak izolált környezetben szabad bemutatni. A runbook verziója a szolgáltatás release-éhez kötődik, így ha a kód változik, a runbookot is frissíteni kell.

Egy jól strukturált runbook a következő szakaszokból áll:

- **Cím és azonosító:** például „Runbook: Rendelés-feldolgozó worker újraindítása”.
- **Cél:** mikor kell használni, milyen hibát kezel.
- **Előfeltételek:** szükséges hozzáférések, eszközök, környezet.
- **Lépések:** számozott lista, minden lépéshez parancs vagy művelet, és az elvárt kimenet.
- **Visszaállítás:** hogyan lehet az eljárást visszavonni, ha ront a helyzeten.
- **Ellenőrzés:** milyen metrikát vagy állapotot kell megfigyelni a sikeresség megállapításához.
- **Kapcsolódó dokumentumok:** ADR-ek, C4 diagramok, monitoring dashboardok.

Példa runbook kivonat:

```
# Runbook: Rendelés-feldolgozó worker újraindítása

Cél: A worker szolgáltatás leállása vagy hibás működése esetén a várakozó üzenetek
↪  feldolgozásának helyreállítása.

Előfeltételek:
- Hozzáférés a Kubernetes klaszterhez (read-only jog)
- A worker deployment neve: order-worker
- Namespace: order-system

Lépések:
1. Ellenőrizd a worker podok állapotát:
   kubectl get pods -n order-system
   Elvárt: minden pod Running vagy legalábbis nem CrashLoopBackOff.
2. Ha a pod CrashLoopBackOff állapotban van, nézd meg a logokat:
   kubectl logs -n order-system <pod-név> --previous
3. Ha a hiba adatbázis-kapcsolatra utal, ellenőrizd az adatbázis elérhetőségét:
   kubectl get svc -n order-system order-db
4. Ha a hiba átmeneti, indítsd újra a deploymentet:
   kubectl rollout restart deployment/order-worker -n order-system
5. Várd meg, amíg az új podok Ready állapotba kerülnek:
   kubectl rollout status deployment/order-worker -n order-system
6. Ellenőrizd az üzenetsor mélységét, hogy csökken-e:
   kubectl exec -n order-system <rabbitmq-pod> -- rabbitmqctl list_queues

Visszaállítás:
Ha az újraindítás után a hibák gyakoribbá válnak, vonjuk vissza az utolsó rollout-ot:
kubectl rollout undo deployment/order-worker -n order-system

Ellenőrzés:
A worker metrikák (feldolgozott üzenetek száma, hibák aránya) a normál tartományba térnek
↪  vissza, és az üzenetsor mélysége csökken.
```

A runbookok tárolása a repository `runbooks/` mappájában történik, és mindegyikhez tartozik egy rövid metaadat-fejléc, amely tartalmazza az utolsó sikeres tesztelés dátumát és a tesztelő nevét. Ez biztosítja, hogy a runbook ne csak elméletben létezzen, hanem időszakosan – például game day keretében – ellenőrizzük a reprodukálhatóságát.

33.2.5 Modellkártya: ML komponensek dokumentálása

Ha a rendszer tartalmaz gépi tanulási modellt – például csalásdetektáló, ajánló vagy előrejelző komponenst –, a dokumentációs csomag részét képezi a modellkártya (model card). A modellkártya célja, hogy a modell célját, bemeneteit, kimeneteit, teljesítménymutatóit, korlátait és etikai

megfontolásait egy helyen, auditálható formában rögzítse. Ez különösen fontos a szabályozott környezetekben (például pénzügy, egészségügy), ahol a modell döntéseinek magyarázhatósága és a diszkrimináció elkerülése követelmény.

A modellkártya nem a modell kódjának dokumentációja, hanem a modell üzleti és etikai kontextusát írja le. A minimális tartalom:

- **Modell neve és verziója.**
- **Cél:** milyen üzleti problémát old meg.
- **Bemenetek:** a felhasznált jellemzők listája, típusa, forrása.
- **Kimenet:** a modell által szolgáltatott érték, értelmezése.
- **Teljesítménymutatók:** pontosság, F1, AUC, stb., valamint a mérési környezet, adathalmaz, időpont. A mérés bizonytalanságát is fel kell tüntetni.
- **Korlátok:** milyen esetekben megbízhatatlan a modell, milyen adateltolódásra érzékeny.
- **Etikai megfontolások:** milyen védett tulajdonságokat nem használ, milyen torzításokat vizsgáltak.
- **Felelős csapat és jóváhagyás.**

Példa egy rövid modellkártyára:

```
# Modellkártya: Rendelési csalásdetektáló modell

Modell neve: fraud-detector-v2.1
Cél: A rendelések csalásgyanús pontszámának kiszámítása 0 és 1 között.
Bemenetek: rendelés összege, szállítási cím, eszköz ujjlenyomat, vásárlói előzmények (utolsó 10
↪  rendelés).
Kimenet: csalásvalószínűség (0-1), 0.5 felett manuális felülvizsgálat szükséges.
Teljesítménymutatók (értékelési környezet: 2025-01-15, teszt adathalmaz, 100 000 minta, F1=0.89,
↪  AUC=0.93, konfidencia intervallum ±0.02).
Korlátok: A modell nem ismeri az új típusú csalási mintákat, havi újratanítás szükséges. A
↪  csalási arány erős szezonalitást mutat, ezért a küszöböt időszakosan kalibrálni kell.
Etikai megfontolások: A modell nem használ védett tulajdonságokat (nem, életkor, etnikum)
↪  közvetlenül. A cím alapú jellemzők esetén fennáll a földrajzi torzítás kockázata, ezért a
↪  címeket anonimizáljuk.
Felelős csapat: ML platform csapat
Jóváhagyta: Adatvédelmi tisztviselő, 2025-02-05
```

A modellkártya a repository `model-cards/` mappájában tárolódik, és ugyanaz a verzió- és jóváhagyási folyamat vonatkozik rá, mint a többi dokumentumra.

33.2.6 Verzió, jóváhagyás és megőrzés

A dokumentációs csomag akkor válik auditálhatóvá, ha a verziókezelés, a jóváhagyás és a megőrzés folyamata szabályozott. Ez a három elem biztosítja, hogy a dokumentumok időbeli változásai visszakereshetők, a jóváhagyások nyomon követhetők, és a korábbi állapotok hosszú távon megőrződnek.

Verzió: A dokumentációs fájlokat a kóddal együtt, ugyanabban a Git repository-ban tároljuk. Minden dokumentum rendelkezik verziószámmal, dátummezővel és szerzővel. A teljes dokumentációs csomaghoz egy release címkét (tag) rendelünk, például `docs-v1.2.0`, amely megegyezik a szolgáltatás release verziójával. Ez lehetővé teszi, hogy egy adott rendszerállapothoz pontosan meg tudjuk mondani, melyik dokumentációs csomag tartozik. A szemantikus verziózás itt is ajánlott: főverzió, alverzió, javítás.

Fontos, hogy a dokumentumok ne csak a fájlrendszerben, hanem a Git history-ban is verziózva legyenek. A változtatások commit üzenetei utaljanak a tartalmi módosításra és az érintett ADR-ekre.

Jóváhagyás: A dokumentációs változtatásokat ugyanolyan szigorú code review alá kell vetni, mint a forráskódot. A pull request (PR) workflow a következőket foglalja magában:

- A PR leírása összefoglalja a változtatás célját és hatását.
- Legalább egy architektúra-reviewer jóváhagyása szükséges.
- Az ADR-ek esetén a státuszváltás (például „javasolt” → „elfogadott”) csak a döntéshozó testület (például Architektúra Tanács) jóváhagyásával történhet, és ezt a PR kommentje vagy az ADR mezője rögzíti.
- A merge commit hash-e az audit trail része.

A jóváhagyási folyamat nem célja a bürokrácia, hanem annak biztosítása, hogy a dokumentáció ne váljon el a valóságtól, és minden döntés mögött legyen felelős.

Megőrzés: Az auditálhatóság hosszú távú követelmény. A dokumentációs csomagot a szervezeti adatmegőrzési szabályok szerint archiválni kell. A Git repository rendszeres mentése és a release tagek megőrzése biztosítja, hogy évekkel később is vissza lehessen állítani egy adott dokumentációs állapotot. Emellett érdemes a kiadott dokumentációs csomagot nem szerkeszthető formátumba (például PDF/A) is exportálni, de a mérvadó mindig a verziózott forrás. A megőrzésnél figyelni kell a hozzáférések kezelésére is: csak az arra jogosultak férjenek hozzá a belső dokumentumokhoz, és a hozzáférési napló is auditálható legyen.

33.2.7 Gyakorlati példa: dokumentációs csomag összeállítása

Vegyünk egy fiktív, de valósághű példát: a „Rendelés-feldolgozó rendszer” nevű szolgáltatást, amely REST API-t, RabbitMQ-t és PostgreSQL-t használ, valamint egy csalásdetektáló ML modellt. A kiadási review után a dokumentációs csomag hiányos: van egy `README.md`, de nincsenek C4 diagramok, ADR-ek, runbookok, modellkártya, és a verzió sem követett.

A feladat a csomag összeállítása a fent bemutatott elemekből. A repository szerkezete a következőképpen alakul:

```
.
  README.md                  # belépési pont, összefoglalja a csomagot
  docs/
      architecture/
          context.md         # C4 context diagram (Mermaid)
          containers.md      # C4 container diagram
          components-api.md  # C4 component diagram az API-ra
      adr/
          README.md          # ADR index
          0001-use-postgresql.md
          0002-introduce-rabbitmq.md
          0007-async-payment-confirmation.md
      compliance/
           measurements.md   # mérési jegyzőkönyvek
  runbooks/
      restart-worker.md
      check-db-connectivity.md
  model-cards/
      fraud-detector-v2.1.md
  VERSIONING.md              # verzió- és jóváhagyási folyamat leírása
```

A `README.md` a csomag bejárata: röviden bemutatja a rendszert, hivatkozik a diagramokra, az ADR indexre, a runbookokra és a modellkártyára, valamint feltünteti a dokumentációs csomag verzióját, dátumát és jóváhagyóját. Például:

```
# Rendelés-feldolgozó rendszer - Dokumentációs csomag

Verzió: 1.2.0
Dátum: 2025-02-12
Jóváhagyta: Architektúra Tanács
Státusz: Kiadásra jóváhagyva

Ez a dokumentum a Rendelés-feldolgozó rendszer architektúrájának és üzemeltetésének hivatalos
↪  dokumentációja.

## Tartalom
- [Architektúradiagramok](docs/architecture/)
- [Döntési napló (ADR)](docs/adr/)
- [Runbookok](runbooks/)
- [Modellkártya: csalásdetektáló](model-cards/fraud-detector-v2.1.md)
- [Mérési jegyzőkönyvek](docs/compliance/measurements.md)
```

A mérési jegyzőkönyv (`measurements.md`) kulcsfontosságú az auditálhatósághoz. Minden mérésnél rögzíteni kell a környezetet, konfigurációt, időpontot és bizonytalanságot. Például:

```
# Mérés: Rendelés-feladás API válaszidő

Cél: A QAS-003 teljesítésének ellenőrzése (p95 <= 500 ms).
Környezet: tesztkörnyezet, Kubernetes klaszter v1.28, 3 API pod, erőforrás limit 500m CPU /
↪  512Mi memória.
Konfiguráció: a rendelés API `application.properties` fájlja a `release-1.2.0` címke szerint.
Időpont: 2025-02-11 14:00-14:30 UTC.
Terhelés: 100 virtuális felhasználó, folyamatos terhelés, 30 perc.
Mért eredmény: p50 = 180 ms, p95 = 320 ms, p99 = 450 ms. Bizonytalanság: a mérési hiba ±10 ms (a
↪  terhelésgenerátor mérési pontossága miatt).
Várt eredmény: p95 <= 500 ms. A mérés szerint teljesült.
```

Ez a jegyzőkönyv megfelel a könyv labor- és kódszerződésének: nem futtatott mérést nem mutat be ellenőrzöttként, és a mért és várt eredményt elkülöníti.

A dokumentációs csomag így egyetlen commit-ban elkészíthető, és a PR review után a `docs-v1.2.0` release taggel megjelölhető.

33.2.8 Tipikus hibák és helyes megoldások

A dokumentációs csomag összeállítása során a következő hibák fordulnak elő leggyakrabban. Mindegyikhez megadjuk a felismerési módot és a helyes megoldást.

1. **A diagram nem diagrams-as-code, hanem statikus kép.**
   Hiba: A C4 diagramokat Visio vagy draw.io formátumban készítik, és PNG-ként commitolják. A változások nem diffelhetők, a review nehézkes, és a diagram hamar elavul.
   Megoldás: Használjunk Mermaid, PlantUML vagy Structurizr DSL-t. A diagram forrása szöveges fájl a repository-ban, amely a CI során automatikusan renderelhető.

2. **Hiányzó verzió vagy nem egyértelmű státusz.**
   Hiba: A dokumentációban nincs dátum, verziószám, szerző vagy jóváhagyó mező. Nem lehet megmondani, hogy a dokumentum aktuális-e vagy elavult.
   Megoldás: Minden dokumentum fejlécében legyen kötelező mező: `Verzió:`, `Dátum:`, `Szerző:`, `Jóváhagyta:`, `Státusz:`. A teljes csomaghoz tartozzon release tag.

3. **A mérések nem reprodukálhatók.**
   Hiba: A teljesítményteszt eredményét csak annyiban közlik, hogy „a p95 300 ms volt”, de nem adják meg a környezetet, terhelést, konfigurációt, időpontot és bizonytalanságot.

Megoldás: Használjuk a fenti mérési jegyzőkönyv sablont, és ragaszkodjunk hozzá, hogy minden mérésnél kitöltsék az összes mezőt.

4. **Valós adatok szivárognak a dokumentációba.**
   Hiba: A runbookokban éles IP-címek, belső hosztnevek, API kulcsok vagy ügyfélnevek szerepelnek. Ez biztonsági rést és adatvédelmi incidenst okoz.
   Megoldás: Helyőrzőket használjunk: `<internal-host>`, `<api-key-placeholder>`, `acme-internal.example.com`. A dokumentációs review során automatizált szűrést is futtathatunk titkos kulcsokra.

5. **Az ADR-ek szigetek.**
   Hiba: Az ADR-ek léteznek, de nem hivatkoznak a C4 diagramokra, kódra vagy más ADR-ekre. Az auditor nem tudja összekötni a döntést a megvalósítással.
   Megoldás: Minden ADR-ben legyen `Hivatkozások` szakasz, és a diagramokban kommentként utaljunk az ADR sorszámára.

6. **Nincs jóváhagyási nyom.**
   Hiba: A dokumentáció változtatásai közvetlenül a `main` branchre kerülnek, review nélkül. Nincs bizonyíték arra, hogy a változtatást bárki ellenőrizte volna.
   Megoldás: Kötelező pull request workflow, legalább egy reviewerrel. A merge commit megőrzi az audit trail-t.

7. **A runbook nem reprodukálható.**
   Hiba: A runbook lépései hiányosak, vagy olyan parancsokat tartalmaznak, amelyek nem egyeznek a kód aktuális állapotával. Nincs visszaállítási út a destruktív lépésekhez.
   Megoldás: A runbookokat időnként tesztelni kell (game day), és a runbook verzióját a szolgáltatás release-éhez kötni. Minden destruktív lépéshez visszaállítási utat kell adni.

Ezek a hibák mind azt jelzik, hogy a dokumentáció nem a rendszer élő része, hanem egy utólagos, kötelezően letudandó feladat. A helyes megoldás mindig az, hogy a dokumentációt a kódhoz hasonlóan kezeljük: verziózott, review-zott, tesztelt és naprakész.

33.2.9 Labor: dokumentációs csomag összeállítása

Cél: Egy meglévő, kiválasztott szolgáltatás (például a záróprojekt egy részének) dokumentációs csomagjának összeállítása a fenti elvek szerint.

Kiinduló állapot: Adott egy Git repository a szolgáltatás forráskódjával és konfigurációjával. Létezik egy architektúra-review jegyzőkönyv (az előző fejezetből, Architektúra-review checklist), amely felsorolja a hiányosságokat. A dokumentáció jelenleg csak egy informális `README.md`-ből áll; nincsenek C4 diagramok, ADR-ek, runbookok, modellkártya, és a verzió sem követett.

Feladat: Állítsd össze a dokumentációs csomagot egy reprodukálható dokumentumban vagy jegyzetfüzetben (például Markdown fájlokban a repository `docs/` mappájában). A csomagnak a következő elemeket kell tartalmaznia:

1. **Belépési pont:** `docs/README.md` a csomag összefoglalásával, verzióval, dátummal, jóváhagyóval, és hivatkozásokkal az elemekre.
2. **C4 diagramok:** context, container és component szint, diagrams-as-code formában (Mermaid vagy PlantUML). Minden diagramhoz rövid leírás, amely megadja a célját és az egyszerűsítéseket.
3. **ADR-ek:** legalább három fontos architektúradöntéshez írj ADR-t a fenti sablon szerint. Az ADR-ek tartalmazzák a hivatkozásokat a diagramokra és a követelményekre.

4. **Runbook:** legalább egy runbook a leggyakoribb hibaelhárítási forgatókönyvhöz (például szolgáltatás újraindítása, adatbázis-kapcsolat ellenőrzése). Tartalmazzon visszaállítási utat és ellenőrzési lépést.
5. **Modellkártya:** ha a rendszer tartalmaz ML modellt, tölts ki egy modellkártyát. Ha nincs, a laborban elég egy rövid indoklás, hogy miért nem releváns.
6. **Mérési jegyzőkönyv:** legalább egy mérés dokumentálása a környezettel, konfigurációval, időponttal, terheléssel, mért és várt eredménnyel. (Ha nincs valós mérés, akkor a jegyzőkönyvben egyértelműen jelöld, hogy a mérés nem futott le, és a várt eredmény becslés.)
7. **Verzió- és jóváhagyási metaadatok:** minden fájl fejlécében, valamint a `VERSIONING.md`-ben írd le a folyamatot.

Elvárt eredmény: Egy commit a repository-ban, amely tartalmazza a fenti dokumentumokat. A `docs/README.md` összefoglalja a csomagot, és hivatkozik az egyes elemekre. A dokumentumok helyőrzőket használnak valós adatok helyett, és a mérési jegyzőkönyv elkülöníti a mért és várt eredményt.

Ellenőrzési szempont: Egy oktató vagy társ átnézi a csomagot az alábbi ellenőrzőlista alapján. Különösen figyel a C4 szintek helyességére, az ADR-ek kötelező mezőire, a runbook reprodukálhatóságára, és arra, hogy nincs valós adat.

Visszaállítási út: Mivel a feladat csak dokumentációt ad hozzá a repository-hoz, nincs szükség éles környezeti visszaállításra. Ha hibát vétünk, a commit `git revert`-tel visszavonható, vagy a fájlok javíthatók.

33.2.10 Ellenőrzőlista

A dokumentációs csomag kiadás előtti ellenőrzéséhez az alábbi listát használhatjuk:

- ☐ Minden architektúradöntéshez ADR tartozik, és az ADR hivatkozik az érintett C4 diagramokra és követelményekre.
- ☐ A C4 context, container és component diagramok diagrams-as-code formában léteznek, és a repository `docs/` mappájában verziózottak.
- ☐ A diagramokon jelölve van a rendszerhatár, a komponensekhez felelősség és interfész van rendelve, a kapcsolatokon protokoll, szinkronitás és hibatűrés szerepel.
- ☐ Minden méréshez tartozik környezet, konfiguráció, időpont, terhelés, mért és várt eredmény, valamint bizonytalanság. Nem futtatott mérés esetén ez egyértelműen jelölve van.
- ☐ A dokumentáció nem tartalmaz valós hitelesítő adatot, belső URL-t, ügyfélnevet vagy személyes adatot; helyőrzőket használ.
- ☐ Minden dokumentumnak van verziója, dátuma, szerzője és jóváhagyója.
- ☐ A runbook lépései reprodukálhatók, tartalmazzák az elvárt kimeneteket, a visszaállítási utat destruktív műveleteknél, és hivatkoznak a releváns release verzióra.
- ☐ Ha van ML modell, a modellkártya kitöltött, és tartalmazza a teljesítménymutatókat, korlátokat és etikai megfontolásokat.
- ☐ A dokumentációs csomaghoz release tag tartozik, és a változtatások PR review-n mentek keresztül.
- ☐ A `docs/README.md` összefoglalja a csomag tartalmát, és minden elemre hivatkozik.

33.2.11 Önellenőrző kérdés

Mi a különbség egy leíró dokumentáció és egy reprodukálható, auditálható dokumentációs csomag között? Sorolj fel legalább három olyan elemet, amely a reprodukálhatóságot és auditálhatóságot biztosítja, és indokold meg, miért nélkülözhetetlenek.

A dokumentációs csomag nem öncélú műalkotás, hanem a rendszer élő része. Amikor a következő fejezet az ismert korlátokkal és nyitott kérdésekkel foglalkozik, ez a csomag adja azt az alapot, amelyre a korlátok azonosítása és a jövőbeli döntések épülnek. A reprodukálható és auditálható dokumentáció nem garantálja a rendszer hibamentességét, de biztosítja, hogy a hibákat fel lehessen tárni, a döntéseket vissza lehessen keresni, és a megfelelést igazolni lehessen.

33.3 Ismert korlátok és nyitott kérdések

A szoftverarchitektúra akkor válik mérnöki diszciplínává, amikor a döntések mellett a döntések határait is képes megfogalmazni. A legtöbb architektúra-dokumentáció erősségeket, célállapotokat és elvárt minőségi attribútumokat sorol fel, de ritkán mondja ki: „ezt a rendszer nem tudja megoldani”, „eddig a pontig teszteltük”, vagy „ezt a kérdést a csapat még nem tudta megválaszolni”. Az ilyen kijelentések hiánya nem a magabiztosság jele, hanem az adósságé. Ez az alfejezet a rendszer korlátainak őszinte dokumentálását tanítja meg: hogyan vedd számba a technikai, szervezeti és jogi határokat, hogyan kapcsold össze őket a monitoringgal és a továbbfejlesztési tervvel, és hogyan kerüld el azt a csapdát, amikor a „működik” állítás mögött kimondatlan feltételek és kimért adatok hiánya rejtőzik.

A fejezet nem ismétli meg a megfelelési csomag elemeit (lásd Dokumentációs és megfelelési csomag), és nem nyitja meg a következő alfejezet tanulási útvonalát sem. A fókusz egyetlen dolog: a korlátozás mint elsőrendű architektúra-lelet kezelése.

33.3.1 Miért kell a korlátokat dokumentálni?

Minden rendszernek vannak határai. A határok egy része szándékos tervezési döntés, más része fel nem ismert kockázat, megint más része külső kényszer. A korlátok dokumentálása három célt szolgál:

1. **Megelőzi a téves üzemeltetési döntéseket.** Ha egy üzemeltető nem tudja, hogy egy szolgáltatás csak 500 kérés/másodpercig tesztelt, akkor a 600 kéréses terhelésnél bekövetkező hiba számára váratlan incidens, nem ismert határeset.
2. **Megőrzi a döntési kontextust.** Egy év múlva a csapat új tagja nem fogja tudni, miért nem használtatok erős konzisztenciát egy fizetési rendszerben. Ha a korlátok között szerepel, hogy „a tranzakciós szolgáltatás 15 másodpercig végül is konzisztens a külső partner lassú válaszai miatt”, az új mérnök nem fog feleslegesen újra feltalálni egy szinkron megoldást.
3. **Kijelöli a továbbfejlesztés valódi terét.** Amíg egy korlát nincs dokumentálva, addig a roadmap-vita a vélemények csatája. Amikor a korlát mértékkel, hatással és kockázattal szerepel a nyilvántartásban, akkor a jövőbeli munka priorizálása mérnöki feladattá válik.

A korlátok dokumentálása kulturális kérdés is. Ahol a hiba a karriert veszélyezteti, ott a csapat hajlamos a korlátokat elhallgatni vagy a „majd megoldjuk” szőnyeg alá söpörni. A pszichológiai biztonság hiánya közvetlen architektúra-minőségi probléma: a rendszer papíron többet tud, mint élesben. Ezért a korlátdokumentum formátumának egyszerűnek, visszakereshetőnek és nem személyre szólónak kell lennie.

33.3.2 A korlátdokumentum helye a dokumentációs csomagban

A kiadási checklist (Architektúra-review checklist) és a megfelelési csomag (Dokumentációs és megfelelési csomag) mellett a korlát- és kockázatlista a harmadik olyan dokumentum, amelyet minden jelentős verzió előtt frissíteni kell. A formátum nem kötött: lehet ADR-ekhez kapcsolt nyilvántartás, egy verziózott Markdown fájl a repóban, vagy egy dedikált táblázat a dokumentációs rendszerben. A lényeg, hogy a döntési naplóhoz hasonlóan minden bejegyzésnek legyen azonosítója, kontextusa, állapota és felelőse.

Egy korlátdokumentum bejegyzése nem ugyanaz, mint egy hibajegy. A hibajegy egy konkrét, reprodukálható eltérés a specifikációtól. A korlát ezzel szemben gyakran egy tervezési döntés következménye, egy külső kényszer leképezése, vagy egy olyan terület, ahol a tudásunk hiányos. A hibajegyeket a hibakövetőben zárjuk le; a korlátokat a korlátdokumentumban frissítjük, és akár évekig is élnek.

A dokumentumban három szekciót érdemes elkülöníteni:

- **Ismert korlátok:** olyan technikai, szervezeti vagy jogi határok, amelyek ma fennállnak.
- **Nyitott kérdések:** olyan döntések vagy mérések, amelyekre a csapat még nem találta meg a választ.
- **Kockázatok:** olyan lehetséges jövőbeli események, amelyek a korlátokból vagy a nyitott kérdésekből fakadnak, és amelyekre a monitoring vagy a folyamatos értékelés figyel.

Ezeket a kategóriákat nem szabad összemosni. Ha egy bejegyzésben egyszerre akarod a korlátot, a kockázatot és a nyitott kérdést leírni, akkor a jövőbeli olvasó nem fogja tudni, mi a tény, mi a hipotézis és mi a teendő.

33.3.3 A technikai korlátok

A technikai korlátok a rendszer belső szerkezetéből, az alkalmazott technológiákból és a választott architektúrális stílusból fakadnak. Nem hibák, hanem a tervezési tér szükségszerű beszűkülései.

Teljesítmény- és skálázódási korlátok. A teljesítményígéretek mindig mérési ablakhoz és környezethez kötöttek. Ha a rendszer egy adott terhelési modell mellett p95-ös válaszidőre vonatkozó SLO-t teljesít, az SLO-n kívül eső tartomány korlát. A dokumentumban szerepeljen a mért felső határ, a mérés környezete és a bizonytalanság. Például:

> A katalógus-lekérdező szolgáltatás p95-ös válaszideje 180 ms, 250 kérés/másodperc terhelés mellett, a tesztkörnyezetben mérve (3 csomópont, 4 vCPU/csomópont). 300 kérés/másodperc felett a p95 válaszidő 420 ms fölé nő; ezt a tartományt nem tekintjük garantáltnak. A mérési bizonytalanság ±15 ms.

Az ilyen bejegyzés nem azt mondja, hogy a rendszer rossz; azt mondja, hogy a csapat pontosan ismeri a határt. A hiányzó mérés nem korlát, hanem nyitott kérdés. Ha a terhelést nem mérted meg, ne írd le azt, hogy „a rendszer jól skálázódik”. Írd le azt, hogy „a skálázódás felső határa jelenleg ismeretlen; mérési terv szükséges”.

Konzisztencia- és rendelkezésreállási korlátok. Az elosztott rendszerekben a CAP-tétel és a PACELC-modell határozza meg a lehetséges trade-off-okat (lásd Elosztott rendszerek alapjai: CAP, konzisztencia és hiba). A korlátdokumentumban pontosan le kell írni, hogy a rendszer partíció esetén mit áldoz fel. Ha a rendszer egy külső partnertől függ, a korlátokat a partner SLA-ja is kijelöli. Egy tipikus bejegyzés:

> A rendeléskezelő szolgáltatás az XYZ partner fizetési API-ját hívja szinkron módon. A partner SLA-ja 99,9% havi rendelkezésre állást garantál; ha a partner nem elérhető, a rendeléskezelő a tranzakciót elutasítja, nem helyezi sorba. Ez a döntés az ADR-023-ban rögzített elvet követi: a fizetési tranzakciónál a hamis siker kockázata nagyobb, mint a rendelkezésre állás csökkenése. A korlát: a saját rendelkezésre állásunk nem lehet magasabb, mint a partner SLA-ja.

Ez a bejegyzés azért jó, mert összeköti a korlátot a döntési naplóval, és kimondja az összefüggést a saját metrika és a külső függőség között.

Adatmodell- és migrációs korlátok. A perzisztencia rétegben a séma evolúciója mindig korlátokat hoz létre. Ha egy mező típusát nem lehet bővítés nélkül megváltoztatni, az korlát. Ha az adatbázisunk nem támogatja az online indexépítést, az is korlát. A dokumentumban a korlát mellé érdemes leírni az áthidaló megoldást is, ha van:

> A számlázási adatbázis PostgreSQL 15-öt használ. A rendszer egy korábbi döntése miatt a számla entitás JSONB mezőben tárolja a tételeket. Ez lehetővé teszi a rugalmas sémaevolúciót, de a tételek aggregálása (SUM, GROUP BY) nem tud indexelt lekérdezést használni. 10 milliós tételszám felett az aggregáló lekérdezések teljes táblapásztázást igényelnek. Ez a technikai korlát a jelenlegi séma- és adatmodell-választás következménye.

Biztonsági korlátok. A biztonsági architektúra (Biztonsági architektúra) mindig kompromisszum. Ha a rendszer nem támogatja a hardveres kulcstárolást, ha a jogosultsági modell nem tudja leképezni a több bérlős szervezeti hierarchiát, ha a naplózás nem tartalmaz személyes adatot, de ezért az elemzés nehezebb – ezek mind korlátok, és mind dokumentálást igényelnek. A biztonsági korlátoknál különösen fontos, hogy a bejegyzés ne adjon támadási útmutatót, csak a hiányzó képességet és annak hatását írja le.

33.3.4 A szervezeti korlátok

A szervezeti korlátok legalább annyira meghatározzák a rendszer architektúráját, mint a technikai döntések. Conway törvénye (Conway törvénye és szervezeti illeszkedés) szerint a szervezet kommunikációs szerkezete leképeződik a rendszer szerkezetére. Ha a szervezet nem tud egy adott architektúrát fenntartani, akkor az a legjobb szándék ellenére is szét fog esni.

Csapatméret és szakértelem. Ha egy mikroszolgáltatás-alapú rendszert háromfős csapat üzemeltet, a működtetési feladatok el fogják vonni az erőforrást a fejlesztéstől. Ez nem technikai, hanem szervezeti korlát. A dokumentum bejegyzése nem a csapat hibájaként, hanem a rendszer kényszereként fogalmaz:

> A jelenlegi szervezetben 2 csapat összesen 7 mérnökkel foglalkozik a fizetési platformmal. A 14 különálló deployolható szolgáltatás üzemeltetési terhe (on-call, frissítés, megfigyelés) hetente átlagosan 12 mérnöki órát igényel. Ez a szervezeti korlát azt jelenti, hogy a szolgáltatásonkénti on-call rotáció jelenleg nem tartható fenn minden szolgáltatásra; 3 szolgáltatásnak nincs dedikált ügyeletese. A korlát a 2026 Q2-re tervezett csapatbővítésig érvényes.

A jó szervezeti korlátbejegyzés tehát a tényt, a hatást és az érvényesség feltételét együtt adja meg.

Döntési jogkörök hiánya. Ha egy csapat nem jogosult önállóan dönteni az infrastruktúra skálázásáról, az adatmegőrzési időről vagy a külső beszállító kiválasztásáról, akkor a rendszer

architektúrája kívülről vezérelt. A korlátdokumentum ezt is rögzíti. A bejegyzés ne panaszkodás legyen, hanem a döntési jogkör pontos leírása:

> A GDPR-adatmegőrzési időt a vállalati adatvédelmi tisztviselő hagyja jóvá. A csapat nem módosíthatja a naplóadatok törlési ütemezését; minden ilyen változtatás a vállalati governance bizottság elé kerül. Ez azt jelenti, hogy a hibakeresési naplókat legfeljebb 30 napig őrizzük meg, ami korlátozza a régi incidensek elemzését.

Függőség más csapatoktól. A szervezeti korlátok gyakran a csapatok közötti interfészen jelentkeznek. Ha egy rendszer egy másik csapat által üzemeltetett belső platformra épül, a saját rendszerünk kiadási ciklusát a platform kiadási ciklusa korlátozhatja. Az ilyen korlát a belső platform termékként való kezelésének (Platform mint termék és golden path) természetes velejárója, de attól még korlát.

33.3.5 A jogi és megfelelési korlátok

A jogi korlátok nem képezik vita tárgyát; külső kényszerként, hivatkozással együtt kell őket dokumentálni. A könyv nem jogi tanácsadás, ezért a bejegyzéseknek a szabályozás hivatalos szövegére kell utalniuk, nem a csapat értelmezésére.

Adatkezelési korlátok. A GDPR előírja, hogy a személyes adatok kezelésének célhoz kötöttnek kell lennie. Ebből fakadó korlát például, hogy az anonimizálás után is visszavezethető adatokat nem lehet korlátlanul elemezni. A korlátdokumentumnak ezt a szabályozási hivatkozás szintjén kell rögzítenie:

> A rendszer a felhasználói eseményeket pszeudoanonimizált formában tárolja az analitikai adattárházban. Az álnevesítés ellenére az adatvédelmi szabályzat szerint a nyers eseményeket 13 hónap után törölni kell. Ennek következménye, hogy az éves szintű felhasználói viselkedés-elemzés nem végezhető el a nyers eseményeken; csak előre aggregált, nem személyes adatokon.

Ágazati szabályozás. A pénzügyi, egészségügyi vagy kritikus infrastruktúrákra vonatkozó szabályozások (NIS2, DORA, AI Act) szigorú követelményeket támasztanak a rendelkezésre állásra, az auditálhatóságra és a harmadik fél kockázatára. Ezek a követelmények gyakran korlátozzák a felhőszolgáltató választását, a deployolási gyakoriságot vagy a kódváltoztatások folyamatát. A korlátdokumentumnak a konkrét szabályozási szakaszra kell hivatkoznia – ha a csapat nem ismeri a pontos szakaszt, az nyitott kérdés, nem korlát.

Licenc- és szerződéses korlátok. A harmadik féltől származó komponensek licencei korlátozhatják a terjesztést, a módosítást vagy a felhőben való futtatást. A licencproblémák nemcsak jogi, hanem architekturális korlátot is jelentenek: ha egy könyvtárat nem lehet felhőben futtatni, akkor az adott komponenst nem lehet a kiválasztott célkörnyezetbe telepíteni. Az ilyen korlátokat a szoftverösszetétel-elemzés (SBOM, Ellátási lánc biztonsága: SBOM, SLSA és aláírás) eredménye alapján kell dokumentálni.

33.3.6 Nyitott kérdések: amit nem tudunk

A nyitott kérdések listája a korlátdokumentum legértékesebb része. Ide azok a bizonytalanságok kerülnek, amelyek ma befolyásolják a döntéseinket, de amelyekre még nincs megbízható válaszunk. A nyitott kérdés nem hiba, hanem a mérnöki tudás állapotának leírása.

A jó nyitott kérdés:

- konkrét,
- megfogalmazza, hogy miért fontos a válasz,
- megadja, hogy milyen módon lehetne megválaszolni,
- és rögzíti, hogy ki felel a megválaszolásáért.

Példa egy jól formált nyitott kérdésre:

> NYIT-014: Hogyan viselkedik a keresőszolgáltatás a 10 milliós termékszám felett? A jelenlegi indexelési stratégia a termékek teljes újraindexelését végzi éjszakánként. Nem tudjuk, hogy a jelenlegi indexelő hardveren ez 10 millió terméknél hány óráig tart, és hogy közben a keresési rendelkezésre állás csökken-e. A válasz fontos, mert a következő félévben a termékszám várhatóan eléri ezt a szintet. A megválaszolás módja: terhelési PoC a tesztkörnyezetben, szintetikus termékadatokkal. Felelős: a keresőcsapat tech leadje, határidő: 2026 Q1.

Ez a bejegyzés minden elemet tartalmaz: a bizonytalanságot, a fontosságot, a mérési módot és a felelőst. Ettől válik a nyitott kérdés a roadmap valódi bemenetévé.

A nyitott kérdések fontos forrása a monitoring hiánya. Ha a rendszer nem gyűjt egy adott metrikát, akkor nem tudjuk, hogy az SLO teljesül-e. Ezt nem szabad elhallgatni; a „nem mérünk, de valószínűleg jó” állítás nem mérnöki kijelentés. A helyes megfogalmazás:

> NYIT-015: A rendelésfeldolgozó szolgáltatás p99-es válaszidejét jelenleg nem mérjük percenkénti bontásban; a jelenlegi metrika csak 5 perces átlagot ad. Emiatt nem ismerjük a rövid, éles kiugrásokat, és nem tudjuk megmondani, hogy a p99-es SLO a csúcsterhelés alatt teljesül-e. A mérési hiányosság orvoslása az OpenTelemetry hisztogram-exportjának bevezetése a következő iterációban.

33.3.7 Hogyan készítsd el a korlát- és kockázatlistát?

A laborfeladat egy korlát- és kockázatlista készítése egy kis, ellenőrizhető példán. A példa lehet egy fiktív rendszer, de az elvárás ugyanaz: a dokumentum legyen reprodukálható, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal.

A kiinduló állapot. Vegyünk egy „Rendeléskezelő API” nevű egyszerű rendszert, amely három komponensből áll: egy REST API réteg, egy tranzakciós szolgáltatás és egy PostgreSQL adatbázis. Az API a felhasználói rendeléseket fogadja, a tranzakciós szolgáltatás a külső fizetési partnert hívja, az adatbázis a rendelések állapotát tárolja. A kiinduló állapotban van egy mérési jegyzőkönyv: a csapat a tesztkörnyezetben 100 kérés/másodperc terhelést generált, a p95-ös válaszidő 240 ms volt, a p99 pedig 380 ms. A rendszer a teszt során 2 órán át futott, incidens nélkül. Ennél több adat nincs.

Az elvárt eredmény. A korlátdokumentum Malomban vagy Markdownban készüljön, és három szekciót tartalmazzon: ismert korlátok, nyitott kérdések, kockázatok. Minden bejegyzés kapjon azonosítót (KORL-xxx, NYIT-xxx, KOCK-xxx). A döntésekre ADR-azonosítóval hivatkozzon. A mért és a várt eredményt különítse el: ami mérésből származik, az „mért”; ami hipotézis vagy cél, az „várt”. A dokumentum tegye lehetővé, hogy egy jövőbeli olvasó megértse, honnan jönnek az adatok.

Ellenőrzési szempontok. A labor akkor sikeres, ha:

- a dokumentumban legalább 5 ismert korlát, 3 nyitott kérdés és 3 kockázat szerepel;
- minden technikai korláthoz tartozik mértékegység és mérési ablak vagy konkrét mérési hivatkozás;

- a szervezeti korlátokhoz tartozik érvényességi feltétel;
- a jogi korlátokhoz tartozik szabályozási hivatkozás (akár helyőrzővel);
- a mért és a várt adatok vizuálisan is elkülönülnek;
- a dokumentum verziózott, és a döntési naplóhoz hivatkozás vezet.

Visszaállítási út. Mivel a labor dokumentumot állít elő, nincs szükség rendszer-visszaállításra. A visszaállítási út itt a dokumentum visszavonása: a verziókezelőben az előző verzióra való visszatérés. A laborleírás ezt explicitté teszi: a korlátdokumentum a `docs/limitations/` mappában, a főághoz kapcsolt merge requestben készül, és a review előtt draft állapotú.

33.3.8 A korlátlista mintái

Az alábbiakban egy rövid, tanulságos résszel mutatjuk meg, hogyan néz ki egy jól formált korlátdokumentum a fenti példán.

```
# Korlát- és kockázatlista - Rendeléskezelő API

Verzió: 2025-Q2-03
Állapot: Review
Döntések: ADR-001..ADR-004
Kontextus: Tesztkörnyezet, 3 csomópont, 4 vCPU/csomópont, 100 req/s mérés

## Ismert korlátok

KORL-001 [technikai, teljesítmény]
A p95 válaszidő 240 ms, a p99 380 ms, 100 req/s terhelés mellett,
2 órás mérési ablakban (mért). A 200 req/s feletti tartomány nem
garantált; a rendszer viselkedése ismeretlen. A mérési bizonytalanság
±20 ms.

KORL-002 [technikai, rendelkezésre állás]
A fizetési partner SLA-ja 99,9% havi szinten. A saját rendelkezésre
állásunk felső határa ezért 99,9% (várt). A partner hibája a rendelést
elutasítja, nem sorolja be újra (ADR-002).

KORL-003 [szervezeti]
A rendszert 2 mérnök fejleszti. A szolgáltatás on-call rotációját
nem tudják minden nap ellátni; 2 ünnepnapon nincs dedikált ügyeletes.
A korlát 2026 Q2-ig érvényes, amikor a csapatbővítés várható.

## Nyitott kérdések

NYIT-001 [mérési hiány]
A 200 req/s feletti viselkedés nem mért. A megválaszolás módja:
további terhelési PoC. Felelős: API csapat tech lead.

NYIT-002 [jogi]
A megrendeléshez tartozó IP-címek naplózása a GDPR szerinti személyes
adat-e? Az adatvédelmi tisztviselő még nem nyilatkozott. A válasz
befolyásolja a napló megőrzési idejét.

## Kockázatok

KOCK-001 [magas, technikai]
Ha a tranzakciós forgalom meghaladja a 200 req/s-et, a latencia
gyorsan nőhet, és az API hibaüzeneteket kezdhet visszaadni. A kockázat
valószínűsége közepes, hatása magas. Figyelés: a 150 req/s feletti
riasztás már aktív.
```

Ez a kivonat minden szükséges elemet bemutat: a mért adatoknál ott a mérési ablak és a bizonytalanság, a várt adatoknál a forrás, a szervezeti korlátnál az érvényességi feltétel, a jogi kérdésnél a hivatalos válasz hiánya.

33.3.9 A korlátok és a monitoring összekapcsolása

A korlátdokumentum nem önálló sziget; a monitoringrendszerhez kell kapcsolódnia. Minden technikai korláthoz, amely mérhető metrikára vonatkozik, olyan riasztást vagy műszerfal-elemet kell párosítani, amely a határ közeledtét jelzi. A korlátdokumentum bejegyzése és a monitoring közötti kapcsolatnak kódban vagy konfigurációban visszakereshetőnek kell lennie. A példában a KOCK-001 bejegyzés kimondja, hogy a 150 req/s feletti riasztás aktív. Ez a kapcsolat az, ami a korlátdokumentumot a napi üzemeltetés eszközévé teszi.

Ahol a korlát mérhető, de a mérés hiányzik, ott a nyitott kérdések szekcióba kell bejegyzést tenni, és a monitoring hiányosságot a backlognak kell tartalmaznia. A korlátdokumentum így nem egy szomorú lista a lehetetlenségekről, hanem a rendszer valódi állapotának aktív, élő leltára.

33.3.10 Tipikus hibák

A korlátdokumentumok leggyakoribb hibája az általánosítás. Az „a rendszer lassú lehet” bejegyzésnek nincs információértéke. A jó bejegyzés pontos: melyik komponens, milyen feltétel mellett, milyen mértékkel, milyen mért érték alapján, és mi a hatás. A második gyakori hiba az, hogy a csapat a korlátokat a hibajegyek közé rejti. A hibajegy egy konkrét javítandó dolog; a korlát egy állapot, amelyet lehet, hogy soha nem kell megszüntetni, csak ismerni és kezelni. A harmadik hiba a mért és várt adatok összemosása. A dokumentumban a kettőt el kell különíteni, mert a jövőbeli olvasó nem tudja, hogy egy szám mérésből, becslésből vagy célból származik. A negyedik gyakori hiba a jogi korlátok pontatlan hivatkozása: a „GDPR szerint tilos” mondat helyett a konkrét cikkelyt vagy a vállalati adatvédelmi szabályzat konkrét pontját kell megadni. Ha a csapat nem ismeri a pontos szakaszt, az nyitott kérdés.

33.3.11 A korlátok és a továbbfejlesztés

A korlátdokumentum nem a projekt végét jelöli; éppen ellenkezőleg, a következő iterációk fő bemenete. A roadmap-tervezésnél a nyitott kérdések megválaszolása és a kritikus korlátok megszüntetése a legfontosabb mérnöki feladatok közé tartozik. A korlátok megszüntetése gyakran nem új funkció, hanem architekturális refaktorálás, és a döntéshozóknak meg kell érteniük, hogy ez miért fontos. A dokumentált, mért korlát az egyetlen olyan érv, amely a funkcionális fejlesztésekkel szemben is képes priorizálni a technikai munkát.

A korlátok érvényességét a fitness function-ök (Fitness functions és folyamatos értékelés, Evolúciós architektúra és fitness functions) segítségével lehet folyamatosan ellenőrizni. Ha egy korlát megszűnt, mert a rendszer új verziója már más viselkedést mutat, a bejegyzést frissíteni kell, és az állapotát „megoldva” vagy „elavult” értékre kell állítani. A korlátdokumentum ezzel válik a rendszer evolúciójának tükrévé.

33.3.12 Fejezet-ellenőrzőlista

- ☐ A korlátdokumentum verziózott, és a döntési naplóhoz kapcsolódik.
- ☐ Minden technikai korláthoz tartozik mértékegység, mérési ablak vagy mérési hivatkozás.
- ☐ A szervezeti korlátoknál szerepel az érvényességi feltétel.
- ☐ A jogi korlátoknál szerepel a konkrét szabályozási hivatkozás.
- ☐ A mért és várt adatok vizuálisan elkülönülnek.
- ☐ Minden kritikus korláthoz tartozik monitoring- vagy riasztási kapcsolat.
- ☐ A nyitott kérdésekhez tartozik felelős, határidő és megválaszolási mód.
- ☐ A korlátdokumentum nem hibákra panaszkodik, hanem állapotot ír le.

33.3.13 Önellenőrző kérdések

1. Mi a különbség a hibajegy és a korlát között? Mondj egy példát arra, amikor ugyanaz a jelenség hibajegy is lehet és korlát is.
2. Miért fontos a mért és a várt adatok elkülönítése a korlátdokumentumban? Milyen döntési hibát okozhat a kettő összemosása?
3. hogyan változik a korlátdokumentum szerepe, ha egy rendszert egyetlen csapat üzemeltet, szemben azzal, amikor több csapat és külső partner is részt vesz a futtatásban?

Ezek a kérdések nem egyszeri válaszra várnak; a gyakorló architekt munkája során újra és újra visszatér hozzájuk, ahogyan a rendszer határai is folyamatosan mozognak.

33.4 Tanulási útvonal, könyvek és közösségek

A könyv törzsanyagának lezárásakor nem egy újabb architektúrális mintát vezetünk be, hanem azt a kérdést tesszük fel: mi következik azután, hogy az olvasó megismerte a rendszertervezés alapjait, a minőségi követelményeket, a stílusokat, a platformokat, a rezilienciát és a szervezeti tényezőket? A válasz egy személyes, folyamatosan felülvizsgált tanulási útvonal, amely ugyanolyan tudatosságot igényel, mint egy szoftverrendszer architektúrájának kialakítása. A szakmai fejlődés nem passzív fogyasztás, hanem tervezett, mérhető és iteratív tevékenység. Ez az alfejezet a következő lépések feltérképezését, a lehetséges specializációkat, valamint a tanulást támogató közösségeket és forrásokat mutatja be, majd egy laborral zárul, amelyben az olvasó saját tanulási tervet készít.

33.4.1 A szakmai fejlődés következő lépései

A tanulási útvonal első lépése nem egy új technológia elsajátítása, hanem a meglévő tudás rendszerezése és hiányosságainak feltárása. A könyv hét nagy részre tagolódott: alapok, követelmények, stílusok, adat és platform, megbízhatóság és biztonság, delivery és evolúció, valamint esettanulmányok. Mindegyik rész más-más készségcsoportot igényelt, és más-más döntési helyzetek elé állította az olvasót. A törzsanyag lezárása után érdemes visszatérni a fejezetekhez, és önértékelést végezni: mely témákban tudna önállóan döntést hozni, melyekben csak a fogalmakat ismeri, és melyekben van szüksége mélyebb gyakorlatra. Ez az önértékelés adja a személyes tanulási terv alapját.

A következő lépés a mélyítés irányának kiválasztása. A szoftverarchitektúra területén ritka az a szakember, aki minden részterületen azonos mélységű tudással rendelkezik. A tipikus karrierút a széles alapokból indul, majd egy vagy több specializáció felé halad. A mélyítés

történhet könyvek, hivatalos dokumentációk, nyílt forráskódú projektek tanulmányozásával, de a leghatékonyabb forma a gyakorlati alkalmazás: architektúra-katák megoldása, PoC-k készítése, meglévő rendszerek elemzése vagy open source hozzájárulás. A gyakorlat során a tanult minták és elvek kontextusba kerülnek, és kiderül, hogy egy adott megoldás milyen trade-offokkal jár.

A szakmai fejlődés része a tanúsítványok és formális képzések mérlegelése is. A tanúsítványok nem helyettesítik a tapasztalatot, de hasznosak lehetnek a tudás strukturálására, a szervezeti elvárások teljesítésére és a piaci láthatóságra. Ugyanakkor a tanulási tervben a tanúsítványt nem szabad célként kezelni; az csak egy mérföldkő, amely mögött valódi kompetenciának kell állnia. Hasonló a helyzet a konferencia-részvétellel: annak elsődleges értéke a kapcsolatépítés és a valós esettanulmányok megismerése, nem pedig a részvétel ténye.

A mentorálás és a tanítás szintén fontos lépés. Amikor az olvasó elkezdi magyarázni a tanult fogalmakat másoknak – akár belső előadás, akár blogbejegyzés, akár egy junior kolléga mentorálása formájában –, kénytelen rendszerezni a tudását, és szembesül a saját bizonytalanságaival. Ez a folyamat gyakran többet ér, mint néhány hét passzív olvasás. A tanulási tervbe ezért érdemes beépíteni legalább egy olyan elemet, amely a tudás átadását szolgálja.

A szakmai fejlődés következő lépéseit a következőképpen foglalhatjuk össze:

1. **Alapfelmérés** – a könyv fejezeteinek áttekintése, erősségek és hiányosságok azonosítása.
2. **Mélyítési irány kiválasztása** – egy vagy két specializáció, amely illeszkedik az érdeklődéshez, a szervezeti igényekhez és a piaci kereslethez.
3. **Gyakorlati alkalmazás** – katák, PoC-k, elemzések, open source munka.
4. **Visszajelzés és tanítás** – mentorálás, előadás, írás.
5. **Folyamatos felülvizsgálat** – a tanulási terv iterálása, mérése és igazítása.

33.4.2 Specializációk és szerepek

A szoftverarchitektúra nem egyetlen szerep, hanem szerepek családja. Az elnevezések szervezetenként változnak, de a felelősségi körök alapján érdemes néhány fő irányt megkülönböztetni. Az alábbi lista nem kimerítő, és a határok gyakran elmosódnak; a cél az, hogy az olvasó megértse, milyen irányokba fejlődhet tovább.

Alkalmazásarchitekt – egy adott alkalmazás vagy szolgáltatáscsoport belső szerkezetéért felel. Döntései jellemzően a modulokra, a rétegekre, a komponensek közötti kapcsolatokra, a keretrendszerekre és a kódalapú kényszerekre vonatkoznak. Ez az irány közel marad a fejlesztői mindennapokhoz, és jó választás azoknak, akik szeretnek mélyen egy-egy rendszerben dolgozni.

Megoldásarchitekt – egy üzleti probléma teljes megoldását tervezi meg, amely több alkalmazást, adatforrást és külső rendszert foglalhat magában. Ez a szerep erősebb érintettséget igényel a követelményanalízisben, a stakeholder-kezelésben és a költség-haszon elemzésben. A megoldásarchitekt gyakran hidat képez az üzlet és a technológia között.

Vállalati architekt – a szervezet egészére kiterjedő technológiai stratégia, szabványok és képességek összehangolásáért felel. Ez a szerep kevésbé érinti a napi kódolást, viszont erős rendszerszemléletet, governance-tudást és kommunikációs készséget igényel. A vállalati architektúra fókusza a portfólió, a standardizáció és a hosszú távú technológiai irányok.

Platformarchitekt – a belső fejlesztői platform, a konténerplatform, a CI/CD infrastruktúra és az üzemeltetési alapok tervezéséért felel. Ez az irány a 17. fejezet platform engineering témáira épít, és erős üzemeltetési, hálózati és automatizálási ismereteket igényel. A platformarchitekt terméke nem közvetlenül az üzleti funkcionalitás, hanem az a képesség, amellyel a fejlesztőcsapatok önkiszolgáló módon szállíthatnak.

Adatarchitekt – az adatmodellezés, az adattárházak, az adatfolyamok, az adatgovernance és az analitikai rendszerek tervezésére specializálódik. A 15. fejezetben tárgyalt polyglot persistence, lakehouse és adatkatalógus témák képezik az alapját. Ez a szerep egyre fontosabbá válik a data mesh és az AI/ML rendszerek terjedésével.

Biztonsági architekt – a fenyegetésmodellezésre, az identitás- és hozzáférés-kezelésre, a titkok védelmére, az ellátási lánc biztonságára és a megfelelésre összpontosít. A 20–21. fejezet anyaga adja a tudásalapot. A biztonsági architekt nem csupán technikai szakértő, hanem a szervezet kockázatkezelési folyamatának kulcsszereplője.

Cloud architect – a felhőszolgáltatások, a költségoptimalizálás, a multi-cloud és hybrid megoldások, valamint a managed szolgáltatások összehangolásáért felel. Ez a szerep erős kapcsolatban áll a FinOps-szal és a well-architected keretrendszerekkel. A felhőarchitekt gyakran a szervezet felhőstratégiáját is alakítja.

Integrációs architekt – az alkalmazások közötti kommunikáció, az API-k, az üzenetküldés és az eseményvezérelt integráció tervezésére szakosodik. A 13–14. fejezetben bemutatott EIP, API gateway, service mesh és DDD-alapú határok jelentik a mindennapi eszköztárát.

Ezeken túl léteznek olyan szerepek, amelyek nem kifejezetten "architekt" címet viselnek, de architekturális felelősséget hordoznak: **tech lead**, **staff engineer**, **principal engineer**. A modern szervezetekben az architekturális döntések egyre inkább ezekhez a szerepekhez kötődnek, és a formális architekt pozíció helyett a döntési jogkörök és a felelősségi körök számítanak. A tanulási terv szempontjából ez azt jelenti, hogy a karriercél ne egy cím legyen, hanem egy képesség- és felelősségkészlet.

A specializáció kiválasztásánál érdemes figyelembe venni három dimenziót: az egyéni érdeklődést, a szervezeti igényt és a piaci keresletet. Az érdeklődés hosszú távon fenntartható motivációt ad; a szervezeti igény lehetőséget teremt a gyakorlati alkalmazásra; a piaci kereslet pedig a mobilitást és a javadalmazást befolyásolja. A három dimenzió közül egyik sem abszolút; ha például az olvasó egy olyan terület iránt érdeklődik, amely a jelenlegi szervezetben nem létezik, akkor a tanulási tervben hangsúlyt kell fektetni az open source munkára vagy a saját PoC-kra, hogy a tudás ne maradjon elméleti.

A T-alakú készségmodell hasznos keret a specializáció megértéséhez. A "T" függőleges szára egy kiválasztott terület mély szakértelmét jelöli, a vízszintes szár pedig a széles, általános architektúraismeretet. A könyv törzsanyaga elsősorban a vízszintes szárat erősítette; a következő lépés a függőleges mélység kialakítása. Ez nem jelenti azt, hogy a széles alapokat el kell hanyagolni: a mélység csak akkor működik, ha a döntések kontextusát a széles látókör adja.

33.4.3 Közösségek, konferenciák és források

A szakmai fejlődés nem magányos tevékenység. A közösségek, konferenciák és források három különböző, de egymást kiegészítő tanulási csatornát jelentenek.

Közösségek. A helyi meetupok, szakmai műhelyek és online fórumok lehetőséget adnak a tapasztalatcserére, a kérdések feltevésére és a visszajelzésre. A nyílt forráskódú projektek – például a CNCF, a DDD közösség vagy az OpenTelemetry projekt – különösen értékesek, mert a kódbázis és a döntési folyamatok nyilvánosak, így a tanuló közvetlenül láthatja, hogyan születnek a valódi architektúradöntések. A közösségi részvételnek több szintje van: az olvasástól a kérdésfeltevésen át a hozzájárulásig. A tanulási tervben érdemes fokozatosan növelni az aktivitást, például először egy hírlevél követése, majd egy meetup látogatása, végül egy rövid előadás megtartása.

Konferenciák. A konferenciák elsődleges értéke nem a slide-ok letöltése, hanem a személyes kapcsolatok kiépítése, a gyakorlati esettanulmányok megismerése és a szakmai vitákban való részvétel. A nagy nemzetközi konferenciák – mint például a QCon, a GOTO, a KubeCon vagy a DDD eXchange – mellett számos regionális és ingyenes esemény létezik. A tanulási tervben a konferencia-részvételt nem szabad öncélúan szerepeltetni; érdemes előre meghatározni, hogy milyen kérdésekkel érkezik az olvasó, milyen előadásokat hallgat meg, és utána hogyan dolgozza fel a tanultakat. Egy konferencia akkor éri meg, ha a résztvevő legalább egy konkrét, a munkájában alkalmazható ötlettel tér haza.

Források. A szakirodalom és a hivatalos dokumentációk képezik a mélyítés gerincét. A klasszikus alapművek – például a szoftverarchitektúra, az elosztott rendszerek, a domain-vezérelt tervezés, a megbízhatóság vagy az evolúciós architektúra témájában – évek óta bizonyítják értéküket, de a kiadás és a verzió mindig ellenőrzést igényel, mert a gyorsan változó technológiáknál a pár évvel ezelőtti ajánlások már félrevezetők lehetnek. A hivatalos dokumentációk – Kubernetes, OpenTelemetry, CNCF, felhőszolgáltatók well-architected keretei – a legfrissebb, de gyakran kevésbé didaktikus formában tartalmazzák a tudást. A blogok, podcastok és hírlevelek jól kiegészítik ezeket, de kritikus fogyasztást igényelnek: mindig ellenőrizni kell, hogy egy állítás mögött milyen kontextus, mérés és dátum áll. A könyv forráskezelési konvenciója itt is érvényes: a "state of the art" állítás időhöz kötött, a nem reprodukált eredményt jelölni kell.

A források feldolgozásánál javasolt módszer a **befogadás → értelmezés → alkalmazás → visszacsatolás** ciklus. A passzív olvasás önmagában keveset ér; a tanulási tervben minden forráshoz hozzá kell rendelni egy kimenetet – például egy összefoglalót, egy ábrát, egy döntési naplót vagy egy rövid bemutatót. Ez a kimenet a mérés alapja is: ha az olvasó nem tudja megfogalmazni a forrás lényegét, akkor a feldolgozás valószínűleg nem volt elég mély.

33.4.4 Labor: személyes tanulási terv készítése

Ez a labor a fejezet központi gyakorlata. A cél egy olyan személyes tanulási terv elkészítése, amely konkrét, mérhető, elérhető, releváns és időhöz kötött (SMART) célokat tartalmaz, rögzített kontextussal, döntési naplóval és ellenőrzési szempontokkal. A terv terjedelme legalább 12 hét legyen, mert ennyi idő alatt érdemben lehet haladni egy kiválasztott témában, de még nem olyan hosszú, hogy a körülmények változása ellehetetlenítse az iterációt.

1. lépés: Kontextus rögzítése. Az olvasó írja le a jelenlegi szerepét, tapasztalatát, erősségeit és hiányosságait. Például: "Fejlesztőként dolgozom, 5 év tapasztalattal, erős Java-tudással. Az architektúra alapfogalmait ismerem, de a konténerplatformok és a reziliencia témájában nincs gyakorlati tapasztalatom." A kontextus tartalmazza az elérhető tanulási időt is: "heti 6 óra, ebből 3 óra munkaidőben, 3 óra szabadidőben."

2. lépés: Cél kiválasztása. A cél legyen specifikus, és kötődjön a fejezetben bemutatott specializációk valamelyikéhez. Például: "A következő 12 hétben a platform engineering irányába mélyülök, hogy képes legyek egy belső fejlesztői platform alapjait megtervezni és egy PoC-ot bemutatni." A célhoz tartozzon egy mérőszám is, például: "A 12. hét végére egy 30 perces belső előadást tartok, és a résztvevők legalább 70%-a a visszajelzésben azt jelzi, hogy a bemutatott koncepció érthető volt."

3. lépés: Döntési napló. A terv készítése közben az olvasó több alternatívát is mérlegel. Például választhatna a platform engineering, a biztonsági architektúra vagy a data mesh között. A döntési naplóban rögzíteni kell, hogy miért esett a választás a platform engineeringre, mely alternatívákat vetette el, és milyen következményekkel jár ez a döntés. A napló formátuma követheti az ADR-ek szerkezetét, de rövidebb lehet. Egy példa:

Kontextus: A jelenlegi szervezetben a fejlesztőcsapatok kézi környezetkezeléssel dolgoznak, és a release-ek lassúak. Szeretnék hozzájárulni a belső platform kialakításához, de hiányzik a Kubernetes és GitOps gyakorlati tudásom.
Döntés: A tanulási terv középpontjába a Kubernetes-alapú platform engineeringet helyezem, konkrétan egy lokális klaszter felépítését és egy GitOps folyamat PoC-ját.
Alternatívák: (1) Biztonsági architektúra – a szervezetben most nincs erre erős igény, így nehezebb lenne gyakorolni. (2) Data mesh – a szervezetben nincs adattermékgazda, a koncepció nem lenne közvetlenül alkalmazható. (3) A Kubernetes-téma helyett csak elméleti tanulás – ez nem adna gyakorlati visszajelzést.
Következmények: A következő 12 hétben a platform engineeringre fókuszálok; a biztonsági és adat témákat szinten tartom, de nem mélyítem. A tanulási terv lezárásakor újraértékelem a prioritásokat.
Állapot: Elfogadva, 2025. november 10.

4. lépés: Ütemezés és mérföldkövek. A 12 hetes tervet bontsuk 2–3 napos blokkokra, és minden blokkhoz rendeljünk konkrét tevékenységet és ellenőrzési pontot. Például:

Hét	Téma	Tevékenység	Ellenőrzés
1–2	Kubernetes alapok	Hivatalos dokumentáció alapján egy lokális klaszter (kind vagy minikube) felépítése, podok, deploymentek létrehozása	A klaszter működik, egy deployment 3 replikával fut, a `kubectl get pods` kimenete rögzítve
3–4	GitOps	Argo CD vagy Flux telepítése, egy példa alkalmazás deploymentjének verziózott kezelése	A Git repository-ban lévő változtatás automatikusan megjelenik a klaszterben; a folyamat lekövethető
5–6	Platform mint termék	A 17. fejezet golden path koncepciójának alkalmazása: egy belső dokumentum elkészítése arról, hogyan kellene a csapatnak szolgáltatást telepíteni	A dokumentumot egy kolléga átnézi, visszajelzést ad; a visszajelzés alapján egy iteráció
7–8	PoC tervezés	A PoC céljának, feltételeinek és mérési tervének megírása	A PoC tervben szerepel legalább egy quality attribute scenario, például: “Ha egy fejlesztő új szolgáltatást hoz létre, akkor a golden path segítségével 30 percen belül futó deploymentet kapjon; sikerességi küszöb: 3 próbából 2 sikeres.”
9–10	PoC implementáció	A PoC megvalósítása izolált környezetben	A PoC eredményét jegyzőkönyv rögzíti: mi működött, mi nem, milyen akadályok voltak
11–12	Bemutató és reflexió	30 perces belső előadás, a tanulási terv értékelése	A résztvevők visszajelzése alapján a sikerességi kritérium teljesül-e; a tanulási napló frissítése

5. lépés: Mérés és a várt/ mért eredmények szétválasztása. A tanulási tervben külön

kell választani, hogy mit várunk a tanulástól, és mit mértünk ténylegesen. A várt eredmény lehet például: “Képes leszek egy Kubernetes deploymentet hibakeresni.” A mért eredmény pedig: “A 8. hét végén egy hibás deploymentet 45 perc alatt megjavítottam; a folyamat lépéseit dokumentáltam.” A mérési adatokat – idő, konfiguráció, környezet – rögzíteni kell, ahogyan a könyv labor-szerződése előírja.

6. lépés: Iteráció. A tanulási terv nem kőbe vésett szerződés. A 6. hét környékén érdemes egy köztes felülvizsgálatot tartani: mi működik, mi nem, mi változott a kontextusban? Ha például a munkahelyi terhelés megnőtt, akkor a tervet lehet módosítani, de a módosítást a döntési naplóban dokumentálni kell. Az iteráció célja nem a kudarc elkerülése, hanem a tanulási folyamat fenntarthatósága.

Egy jól elkészített tanulási terv példája bemutatható a fenti táblázatban szereplő platform engineering irány mentén. A terv tartalmazza a kontextust, a döntési naplót, az ütemezést és a mérési pontokat. Az olvasó saját terve ettől eltérhet – más irány, más időkeret, más tevékenységek –, de a szerkezetnek hasonlónak kell lennie.

33.4.5 Tipikus hibák és elkerülésük

A személyes tanulási terv készítése során több gyakori hiba is felbukkan. Az első a **túl általános célok** megfogalmazása: “Jobban akarom érteni a Kubernetes-t” vagy “Szeretnék fejlődni a biztonságban.” Az ilyen célok nem mérhetők, és nem adnak irányt a napi tevékenységekhez. Helyette specifikus, mérhető kimenetet kell meghatározni, például: “Képes leszek egy Ingress erőforrást manuálisan létrehozni és hibakeresni, és ezt egy rövid videóban bemutatni.”

A második hiba a **túl sok párhuzamos szál** indítása. Ha az olvasó egyszerre akar Kubernetes-t, biztonságot, adatarchitektúrát és DDD-t tanulni, akkor a figyelem szétforgácsolódik, és egyik területen sem alakul ki mélység. A T-alakú modell alapján érdemes egy függőleges mélységet választani, és a többi területet szinten tartani.

A harmadik hiba a **passzív fogyasztás** túlsúlya: csak könyveket olvasni, videókat nézni, de nem alkalmazni. A tanulási tervben minden elméleti blokkhoz rendelni kell egy gyakorlati kimenetet – egy PoC-ot, egy ábrát, egy összefoglalót vagy egy bemutatót. A tudás csak akkor válik készséggé, ha használjuk.

A negyedik hiba az **ütemezés és a valós időkeret figyelmen kívül hagyása**. Ha az olvasó heti 2 órát tud tanulni, de a terv heti 10 órát igénylő tevékenységeket tartalmaz, akkor a terv már az első héten összeomlik. Az elérhető időt reálisan kell megbecsülni, és a tervet ennek megfelelően méretezni.

Az ötödik hiba a **visszajelzés hiánya**. A tanulási tervben szerepelnie kell legalább egy olyan ellenőrzési pontnak, ahol az olvasó visszajelzést kap egy mentortól, egy kollégától vagy egy közösségtől. A visszajelzés nélkül a tanuló könnyen a saját vakfoltjaiban ragad.

Végül a hatodik hiba a **merevség**: ha a tervet nem lehet módosítani, akkor a változó körülmények – munkahelyi projekt, családi esemény, technológiai váltás – kudarcra ítélik. A tanulási tervet ugyanúgy kell kezelni, mint egy architektúrát: folyamatosan értékelni, és szükség esetén evolúciósan fejleszteni.

33.4.6 Ellenőrzőlista

- ☐ A tanulási terv kontextusa rögzítve van: szerep, tapasztalat, erősségek, hiányosságok, elérhető idő.
- ☐ A cél specifikus, mérhető, elérhető, releváns és időhöz kötött (SMART).

- ☐ A döntési naplóban szerepel a választott irány, az elvetett alternatívák és a következmények.
- ☐ Az ütemezés legalább 12 hétre bontott, és minden héthez tartozik tevékenység és ellenőrzési pont.
- ☐ Minden elméleti blokkhoz gyakorlati kimenet tartozik.
- ☐ A várt és a mért eredmények külön vannak választva.
- ☐ A mérési adatokhoz tartozik környezet, időpont és bizonytalanság.
- ☐ A terv tartalmaz legalább egy visszajelzési pontot.
- ☐ A terv iterálható, és a köztes felülvizsgálat időpontja be van ütemezve.
- ☐ A források feldolgozásánál a kritikus értékelés szempontjai érvényesülnek.

33.4.7 Önellenőrző kérdés

Az olvasó készített egy 12 hetes tanulási tervet a biztonsági architektúra irányába. A célja: “Megtanulom a threat modelinget.” A tervben heti 1 óra olvasást tervez, de nincs gyakorlati feladat és nincs visszajelzési pont. Melyik három hibát követi el, és hogyan lehetne a célt SMART-tá alakítani? A válaszban azonosítsa a specifikusság, a mérhetőség és a gyakorlati alkalmazás hiányát, majd fogalmazzon meg egy javított célt és egy hozzá tartozó ellenőrzési pontot.

Függelékek

Fogalomtár (magyar–angol)

A fogalomtár a könyvben használt magyar és angol szakkifejezéseket, valamint a leggyakoribb félreértéseket rögzíti. A tételek ábécésorrendben szerepelnek.

Magyar	English	Jelentés / megkülönböztetés
6R migráció	6R migration	A migrációs stratégiák hat kategóriája: rehost, replatform, refactor, repurchase, retire, retain.
Adatmesh	data mesh	Domainorientált adatarchitektúra, amelyben az adat mint termék, a domaincsapatok tulajdonában van, federált governance mellett.
ADR	architecture decision record	Az architektúradöntés kontextusát, döntését, alternatíváit és következményeit rögzítő, verziózott dokumentum.
API gateway	API gateway	A kliens és a belső szolgáltatások közötti belépési pont, amely hitelesítést, útvonalválasztást, rate limitet és megfigyelést végez.
Architektúra	software architecture	A rendszer alapvető struktúrája, a komponensek és kapcsolataik, valamint az ezeket kialakító döntések és elvek összessége.
Architektúra-kata	architecture kata	Rövid, gyakorlati tervezési feladat, amelyben egy csapat követelményekből kiindulva architektúrát és döntési dokumentációt készít.
ATAM	Architecture Tradeoff Analysis Method	Forgatókönyv-alapú architektúraértékelési módszer a kockázatok, érzékenységi pontok és trade-offok feltárására.

Magyar	English	Jelentés / megkülönböztetés
Availability	availability	A rendszer azon képessége, hogy adott időablakban elérhető és használható; $A = \frac{\text{üzemidő}}{\text{üzemidő+állásidő}}$.
Backpressure	backpressure	A túlterhelt rendszer visszajelzése a termelő felé, amely lassítja vagy visszatartja az új kéréseket a minőség megőrzése érdekében.
BFF	backend for frontend	Egy adott klienshez vagy felhasználói élményhez illesztett backend réteg, amely a szolgáltatásokat összefogja.
Blue-green deployment	blue-green deployment	Két párhuzamos környezet segítségével történő kiadás; a forgalom a régi és az új verzió között vált.
Bulkhead	bulkhead	A hibaizoláció mintája, amelyben a rendszer erőforrásait rekeszekre osztják, hogy egy rész meghibásodása ne foglalja el az összes kapacitást.
C4 modell	C4 model	Négy szintű (context, container, component, code) architektúradokumentációs modell, amely a különböző közönségek számára külön nézeteket ad.
Canary release	canary release	Fokozatos kiadás, amelyben az új verzió először a forgalom kis részét kapja, majd siker esetén kiterjesztik.
CAP-tétel	CAP theorem	Elosztott rendszerben hálózati partíció alatt a konzisztencia és a rendelkezésre állás egyszerre nem garantálható.
Circuit breaker	circuit breaker	A hibás függőség hívását ideiglenesen leállító minta, amely megakadályozza a hibaterjedést és a túlterhelést.
Conway törvénye	Conway's law	A rendszer architektúrája tükrözi a azt tervező szervezet kommunikációs struktúráját.
CQRS	command query responsibility segregation	Az írási (parancs) és olvasási (lekérdezés) modellek szétválasztása, gyakran külön adatmodellel és tárolóval.

Magyar	English	Jelentés / megkülönböztetés
DDD	domain-driven design	A domainmodellre és a bounded contextekre épülő tervezési megközelítés, amely az üzleti nyelvet a kódba viszi.
Deployment	deployment	A buildelt és tesztelt szoftver egy környezetbe történő telepítése és aktiválása.
Distributed monolith	distributed monolith	Fizikailag szétválasztott, de erősen összekapcsolt, együtt telepítendő szolgáltatásokból álló anti-minta.
DORA-metrika	DORA metric	A szoftverszállítási teljesítmény és stabilitás négy kulcsmetrikája: deployment frequency, lead time, change failure rate, time to restore.
EDA	event-driven architecture	Események termelésére, továbbítására és feldolgozására épülő architektúra, amely lazán kapcsolja a résztvevőket.
EIP	enterprise integration patterns	Üzenet-alapú integrációra szolgáló, bevált minták gyűjteménye (például routing, transformation, aggregation).
Error budget	error budget	Az SLO-ból levezetett megengedett hiba mennyisége egy időablakban; a megbízhatóság és a változás sebességének egyensúlyát segíti.
Event sourcing	event sourcing	Az állapot helyett az események sorozatának tárolása; az állapot az események visszajátszásával áll elő.
Evolúciós architektúra	evolutionary architecture	Olyan architektúra, amelyet mérhető fitness function-ök és folyamatos visszacsatolás mellett szándékosan változtatnak.
Fitness function	fitness function	Az architektúra egy kívánt tulajdonságát mérő, automatizálható ellenőrzés vagy metrika.
Fog computing	fog computing	A felhő és az eszközök közötti köztes feldolgozási réteg, amely csökkenti a késleltetést és a hálózati terhelést.

Magyar	English	Jelentés / megkülönböztetés
gRPC	gRPC	HTTP/2-re és protobuf-szerződésre épülő, nagy teljesítményű RPC-keretrendszer.
Hexagonális architektúra	hexagonal architecture	Portok és adapterek segítségével a domainlogikát a külső technológiáktól elválasztó architektúra.
IaC	infrastructure as code	Az infrastruktúra deklaratív, verziózott kódként történő kezelése és telepítése.
Idempotencia	idempotency	Egy művelet többszöri végrehajtása ugyanazt az eredményt adja; kulcsfontosságú a retry és az üzenetfeldolgozás szempontjából.
Kapcsolódás	coupling	A komponensek közötti függőség mértéke; alacsony kapcsolódás csökkenti a változás terjedését.
Késleltetés	latency	Egy kérés vagy művelet végrehajtásának időtartama; gyakran percentilisben (p50, p95, p99) mérjük.
Kohézió	cohesion	Egy komponens elemeinek funkcionális összetartozása; magas kohézió egyértelmű felelősséget jelez.
Konténer	container	Izolált, hordozható futtatási egység, amely az alkalmazást és függőségeit csomagolja.
Kubernetes	Kubernetes	Konténerizált alkalmazások telepítését, skálázását és önhelyreállítását végző orchestrációs platform.
Layered architecture	layered architecture	A rendszert felelősségi rétegekre bontó stílus, amelyben a függőségek jellemzően fentről lefelé mutatnak.
Legacy rendszer	legacy system	Nehezen módosítható, elavult technológiájú, de üzletileg még fontos rendszer.
Little-törvény	Little's law	Sorban álló rendszerre $L = \lambda W$: a rendszerben lévő elemek átlagos száma az érkezési ráta és a tartózkodási idő szorzata.

Magyar	English	Jelentés / megkülönböztetés
Load balancing	load balancing	A bejövő forgalom elosztása több példány vagy erőforrás között.
Mikro-kernell	microkernel architecture	Kis magra és plug-in bővítményekre épülő architektúra, amely a változékony funkciókat izolálja.
Mikroszolgáltatás	microservice	Önállóan telepíthető, üzleti képesség köré szervezett, jól körülhatárolt szolgáltatás.
Moduláris monolit	modular monolith	Egy telepítési egység, amely belső modulokra bomlik, világos határokkal és függőségi szabályokkal.
Monolit	monolith	Egyetlen telepítési egységbe szervezett alkalmazás; nem feltétlenül jelent rossz szerkezetet.
Multi-cloud	multi-cloud	Több felhőszolgáltató együttes használata; növeli a rugalmasságot, de bonyolítja az üzemeltetést.
NFR	non-functional requirement	Nem funkcionális követelmény: teljesítmény, rendelkezésre állás, biztonság, karbantarthatóság és hasonlók.
Observability	observability	A rendszer belső állapotának következtetése a telemetriai adatokból: naplókból, metrikákból és nyomvonalakból.
Outbox	transactional outbox	Az adatbázis-tranzakció részeként eltárolt esemény, amelyet egy külön folyamat továbbít, így az üzenetküldés megbízható.
Platform engineering	platform engineering	A belső fejlesztői platform termékként való tervezése és üzemeltetése, golden pathokkal és önkiszolgálással.
PoC	proof of concept	Egy technikai vagy architektúrális feltevés megvalósíthatóságát igazoló, korlátozott kísérlet.
Quality attribute	quality attribute	A rendszer egy mérhető minőségi tulajdonsága, például teljesítmény, biztonság vagy módosíthatóság.

Magyar	English	Jelentés / megkülönböztetés
Quality attribute scenario	quality attribute scenario	Minőségi követelmény forgatókönyv formában: inger, környezet, elvárt válasz és mérőszám.
RAG	retrieval-augmented generation	Olyan LLM-architektúra, amely külső tudásbázisból keresett kontextust ad a generáló modellnek.
Rate limit	rate limiting	Az időegység alatt engedélyezett kérések számának korlátozása a rendszer védelme és a fairness érdekében.
REST	representational state transfer	Erőforrásokra és egységes interfészre épülő architektúrális stílus, amelyet HTTP-n gyakran használnak.
Reziliencia	resilience	A rendszer azon képessége, hogy hibák és zavarok esetén is elfogadható szolgáltatást nyújtson és helyreálljon.
Saga	saga	Elosztott tranzakció kompenzáló lépések sorozatával, amelynek minden résztvevője helyi tranzakciót hajt végre.
Service mesh	service mesh	A szolgáltatásközi forgalom kezelésére szolgáló infrastruktúraréteg, amely oldalkocsikat és központi vezérlést használ.
SLI	service level indicator	A szolgáltatás szintjét mérő indikátor, például sikerarány vagy késleltetés.
SLO	service level objective	A szolgáltatás megbízhatóságára vonatkozó mérhető cél egy időablakban.
SOA	service-oriented architecture	Szolgáltatásokra épülő enterprise architektúra, amely a szerződésalapú újrafelhasználást hangsúlyozza.
Strangler fig	strangler fig pattern	Legacy rendszer fokozatos lecserélése úgy, hogy az új rendszer fokozatosan átveszi a régi funkcióit.

Magyar	English	Jelentés / megkülönböztetés
Team Topologies	Team Topologies	A csapattípusok és interakciós minták szervezeti modellje a gyors áramlás és a platformgondolkodás érdekében.
Technical debt	technical debt	A gyors megoldásért vállalt későbbi javítási költség; lehet szándékos vagy nem szándékos.
Threat modeling	threat modeling	A rendszer fenyegetéseinek rendszerezett feltárása az adatfolyam, a bizalmi határok és a támadási felület alapján.
Twelve-Factor App	twelve-factor app	A felhőalapú alkalmazások tervezésének tizenkét alapelve, például konfiguráció, állapotmentesség és naplózás.
Zero trust	zero trust	Biztonsági modell, amely nem ad implicit bizalmat a hálózati helyzet alapján; minden hozzáférést ellenőriz.

Képlet- és mérőszámgyűjtemény

A gyűjtemény a könyvben használt legfontosabb mérőszámokat, képleteket és döntési számításokat rögzíti. Minden képlet mellett szerepel a feltétel, a mértékegység és a gyakorlati értelmezés. A mérési eredményt mindig a környezettel, az időablakkal és a bizonytalansággal együtt kell közölni.

Rendelkezésre állás

Az availability (rendelkezésre állás) egy időablakban:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{állásidő}}.$$

Ugyanez MTBF és MTTR segítségével:

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}},$$

ahol MTBF a hibák közötti átlagos idő, MTTR a helyreállítás átlagos ideje. A 99,9% éves szinten körülbelül 8,76 óra állásidőt enged meg; a százalék önmagában félrevezető, mert a mérési ablak és a hiba definíciója is számít.

Soros és párhuzamos megbízhatóság

Független komponensek soros kapcsolása esetén:

$$A_{\text{soros}} = \prod_{i=1}^{n} A_i.$$

Párhuzamos redundancia esetén:

$$A_{\text{párhuzamos}} = 1 - \prod_{i=1}^{n} (1 - A_i).$$

A képlet csak akkor érvényes, ha a komponensek hibái függetlenek. A közös függőségek (például egyetlen adatbázis vagy hálózat) a valós rendelkezésre állást a számítottnál rosszabbra csökkenthetik.

SLO és error budget

Ha az SLO egy időablakra S (például 0,999), akkor az error budget:

$$B = (1 - S) \cdot T,$$

ahol T az ablak hossza. A hibák arányából számított burn rate:

$$\text{burn rate} = \frac{\text{tényleges hibaarány}}{1 - S}.$$

A burn rate azt mutatja, hogy a hibabüdzsé hányszor gyorsabban fogy a tervezettnél. A gyors kiégés riasztást indokol; a lassú, tartós kiégés trendfigyelést igényel.

Késleltetés és percentilisek

Az átlagos késleltetés:

$$\bar{t} = \frac{1}{N} \sum_{i=1}^{N} t_i.$$

Az átlag önmagában nem elegendő, mert a felhasználói élményt a farok (késleltetési eloszlás felső része) is meghatározza. Ezért kell p50, p95, p99 értéket is mérni. A p95 például azt jelenti, hogy a kérések 95%-a ennél az értéknél gyorsabb volt.

Little-törvény és kapacitás

Little-törvénye szerint:

$$L = \lambda W,$$

ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta (kérés/másodperc), W pedig az átlagos tartózkodási idő. A képlet stabil, egyensúlyi rendszerre érvényes. Segít megbecsülni, hogy adott késleltetés mellett mekkora párhuzamosságra vagy kapacitásra van szükség.

Kihasználtság és várakozás

Egy kiszolgáló kihasználtsága:

$$\rho = \frac{\lambda}{\mu},$$

ahol λ az érkezési ráta, μ a kiszolgálási kapacitás. Egyszerű M/M/1 sor esetén az átlagos válaszidő:

$$W = \frac{1}{\mu - \lambda}.$$

A képlet szemlélteti, hogy a kihasználtság 1-hez közeledve a várakozási idő nem lineárisan, hanem rohamosan nő. A gyakorlatban ezért kell biztonsági tartalékot hagyni a kapacitástervezésben.

Amdahl-törvény

Ha a feladat egy p hányada párhuzamosítható, és a párhuzamos rész sebességnövelése s, akkor az elérhető gyorsulás:

$$S(s) = \frac{1}{(1-p) + \frac{p}{s}}.$$

A képlet felső korlátot ad: a nem párhuzamosítható rész határozza meg a maximális nyereséget. Architektúrális döntésnél ezért először a soros szakaszokat érdemes csökkenteni.

Univerzális skálázhatósági törvény

A versengés és a koordinációs költség miatt a skálázás nem lineáris:

$$X(N) = \frac{N}{1 + \alpha(N-1) + \beta N(N-1)},$$

ahol N a párhuzamos egységek száma, α a versengés, β a koherencia-költség paramétere. A képlet segít megérteni, miért nem duplázódik a teljesítmény a példányszám duplázásával, és miért van optimális pont.

Döntési mátrix

Több szempontú döntésnél a súlyozott pontszám:

$$S_j = \sum_{i=1}^{m} w_i \cdot s_{ij},$$

ahol w_i az i-edik szempont súlya, s_{ij} a j-edik alternatíva pontszáma. A súlyok összege 1. A pontszám önmagában nem bizonyíték: a bizonytalanságot, a nem számszerűsíthető szempontokat és a döntés visszafordíthatóságát is dokumentálni kell.

Kockázati kitettség

A kockázati kitettség egyszerű becslése:

$$R = P \cdot I,$$

ahol P a kockázat valószínűsége, I a hatás nagysága. A kockázat kezelésénél a valószínűség és a hatás bizonytalanságát külön kell jelölni. A súlyos, de kis valószínűségű kockázat kezelése más stratégiát igényel, mint a gyakori, kis hatású hiba.

RTO és RPO

A helyreállítási időcél (RTO) és a helyreállítási pontcél (RPO) a üzleti folytonosság két kulcsparamétere:

- **RTO:** mennyi idő alatt kell a szolgáltatást helyreállítani.
- **RPO:** mekkora adatvesztés fogadható el, azaz milyen régi lehet a legutolsó helyreállítható adat.

Az RTO és RPO közvetlenül meghatározza a replikáció, a backup és a failover architektúráját. A két értéket nem technikai, hanem üzleti alapon kell megállapítani.

Költség és egységköltség

Egy szolgáltatás teljes költsége:

$$C_{\text{total}} = C_{\text{infra}} + C_{\text{licenc}} + C_{\text{üzemeltetés}} + C_{\text{fejlesztés}} + C_{\text{hiba}}.$$

Az egységköltség:

$$C_{\text{egység}} = \frac{C_{\text{total}}}{\text{hasznos kimenet}}.$$

A hasznos kimenet lehet kérés, tranzakció, aktív felhasználó vagy feldolgozott dokumentum. A felhős számla önmagában nem mutatja meg a hatékonyságot; a kihasználtságot és a hulladékot is mérni kell.

DORA-metrikák

A négy kulcsmetrika:

- **Deployment frequency:** kiadások gyakorisága.
- **Lead time for changes:** a commit és a termelési kiadás közötti idő.
- **Change failure rate:** a hibát okozó kiadások aránya.
- **Time to restore:** a hiba helyreállításának ideje.

A metrikák együttesen mutatják a szállítási teljesítményt és a stabilitást. Egyik sem értelmezhető önmagában; a sebesség a stabilitás rovására is növelhető, ami hosszú távon romlást okoz.

Kapcsolódási metrikák

Egy komponens afferent (bejövő) és efferent (kimenő) kapcsolódása alapján az instabilitás:

$$I = \frac{C_e}{C_a + C_e},$$

ahol C_e a kimenő, C_a a bejövő függőségek száma. Az alacsony instabilitású komponens sok más komponenstől függ, ezért nehéz változtatni; a magas instabilitású komponens könnyebben módosítható. A metrika önmagában nem írja le a modularitás minőségét, de segít a függőségi irányok ellenőrzésében.

Szénlábnyom-becslés

Egy számítási feladat energiaigényének becslése:

$$E = P \cdot t,$$

ahol P a teljesítményfelvétel (watt), t a futásidő (óra). A szén-dioxid-kibocsátás:

$$CO_2 = E \cdot I_{\text{grid}},$$

ahol I_{grid} a hálózat szénintenzitása (gCO /kWh). A képlet csak becslés; a valós értéket a hardver, a hűtés, a kihasználtság és a helyi energiamix együtt határozza meg. A mérés bizonytalanságát mindig fel kell tüntetni.

Kompetenciatérkép és tanulási útvonal

A térkép a könyv fejezeteit nyolc kompetenciaterülethez rendeli. A fejezetszámok a PDF és a HTML azonosítóival egyeznek; a záróprojekt (sach31) minden területet integrál.

Kompetenciaterületek

Kompetencia	Fejezetek	Bizonyíték	Szint
C1 – Architektúra-alapok és elvek	sach01–sach05	definícióelemzés, elvkonfliktus, ADR, C4/arc42 dokumentáció	alapszint
C2 – Követelmények és minőség	sach06–sach09	követelménykatalógus, quality attribute scenario, SLO-terv, döntési mátrix	haladó
C3 – Stílusok és minták	sach10–sach14	stílus-összehasonlítás, szolgáltatáshatár-terv, eseményarchitektúra, API-szerződés	haladó
C4 – Adat, cloud és platform	sach15–sach18	adatarchitektúra, Kubernetes/cloud vázlat, platformterv, kapacitásmodell	haladó
C5 – Reziliencia, biztonság és megfelelés	sach19–sach22	failure mode analysis, threat model, compliance térkép, observability-terv	haladó
C6 – Delivery és minőségbiztosítás	sach23–sach24	CI/CD pipeline, deployment-stratégia, tesztstratégia, statikus elemzési szabályok	haladó
C7 – Evolúció és szervezet	sach25–sach27	modernizációs terv, fitness function, roadmap, stakeholder-terv	mesterszint
C8 – Esettanulmány és end-to-end tervezés	sach28–sach31	esettanulmány-elemzés, iparági minta, záróprojekt, review	mesterszint

Tanulási útvonalak

Alkalmazás- és rendszerarchitekt

Cél: egy üzleti alkalmazás architektúrájának megtervezése és megvédése.

1. Alapok: sach01–sach05.
2. Követelmények és minőség: sach06–sach09.
3. Stílusok és minták: sach10–sach14.
4. Adat és cloud: sach15–sach16.
5. Biztonság és üzemeltetés: sach19–sach22.
6. Delivery: sach23–sach24.
7. Záróprojekt: sach31.

Enterprise- és domain-architekt

Cél: nagy szervezet rendszerportfóliójának és domainjeinek összehangolása.

1. Alapok és dokumentáció: sach01–sach05.
2. Követelmények és minőség: sach06–sach09.
3. Domain- és enterprise minták: sach12–sach14.
4. Adat és platform: sach15–sach17.
5. Evolúció és governance: sach25–sach27.
6. Iparág és esettanulmányok: sach28–sach29.
7. Záróprojekt: sach31.

Platform- és cloud-architekt

Cél: skálázható, biztonságos és költséghatékony platform építése.

1. Alapok és minőség: sach01–sach09.
2. Cloud-native és platform: sach16–sach18.
3. Reziliencia és biztonság: sach19–sach22.
4. DevOps és tesztelés: sach23–sach24.
5. Evolúciós architektúra: sach26.
6. Modern trendek: sach30.
7. Záróprojekt: sach31.

Biztonsági, megfelelési és kockázati szakember

Cél: az architektúra kockázatainak felmérése és dokumentálása.

1. Alapok és dokumentáció: sach01–sach05.
2. Követelmények és minőség: sach06–sach09.
3. Integráció és adat: sach13–sach15.
4. Biztonság és megfelelés: sach19–sach22.
5. Üzemeltetés és incidenskezelés: sach22.
6. Esettanulmányok és iparág: sach28–sach29.
7. Review checklist: sach32.

Tech lead és engineering manager

Cél: a technikai döntések és a szervezeti működés összehangolása.

1. Alapok, elvek, döntések: sach01–sach05.
2. Minőség és tervezés: sach06–sach09.
3. Delivery és tesztelés: sach23–sach24.
4. Evolúció és szervezet: sach25–sach27.
5. Esettanulmányok: sach28.
6. Záróprojekt és review: sach31–sach32.

Adat- és analitikai architekt

Cél: adattermékek, adatplatform és analitikai rendszerek tervezése.

1. Alapok és követelmények: sach01–sach09.
2. Integráció és események: sach12–sach13.
3. Adatarchitektúra: sach15.
4. Cloud és platform: sach16–sach17.
5. Biztonság és megfelelés: sach19–sach21.
6. Iparági adatminták: sach29.
7. Záróprojekt: sach31.

Értékelési rubrika

Minden kompetenciaterületnél három szintet különböztetünk meg:

- **Ismeri:** meg tudja magyarázni a fogalmakat, a mintákat és a tipikus hibákat.
- **Alkalmazza:** önállóan el tudja készíteni a laborban kért architektúra-dokumentumot, döntési naplót vagy mérést.
- **Megtervezi:** adott üzleti, szervezeti és jogi feltétel mellett meg tudja indokolni az architektúra-, technológia- és értékelési döntést, ismeri a döntés korlátait, és képes a döntést felülvizsgálni.

A záróprojekt értékelése négy dimenzióban történik: követelménylefedettség és működés, döntések és trade-offok dokumentáltsága, biztonsági és megfelelési szempontok, valamint a bemutató és a reprodukálhatóság. A projekt csak akkor fogadható el, ha a mért és a várt eredményt elkülöníti, és a nem futtatott mérést, költséget vagy benchmarkot egyértelműen jelöli.

Bibliográfia és forráskulcsok

Ezek a tervezéshez használt kiindulópontok, nem ebben a könyvben tételesen ellenőrzött bizonyítékok. A fejezetek kutatási lépése a konkrét szakaszt, verziót és ellenőrzési dátumot rögzíti. A szabványoknál és jogszabályoknál a mindenkori hivatalos szöveg az elsődleges.

Szabványok, keretrendszerek és szabályozás

- https://www.iso.org/standard/74393.html (ISO/IEC/IEEE 42010:2022 – architektúraleírás)
- https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (ISO/IEC 25010 – minőségi modell)
- https://www.iso.org/standard/27001 (ISO/IEC 27001 – információbiztonsági irányítás)
- https://www.opengroup.org/togaf (TOGAF)
- https://www.zachman.com/about-the-zachman-framework (Zachman-keretrendszer)
- https://www.nist.gov/itl/ai-risk-management-framework (NIST AI RMF)
- https://csrc.nist.gov/pubs/sp/800/207/final (NIST Zero Trust)
- https://eur-lex.europa.eu/eli/reg/2016/679/oj (GDPR)
- https://eur-lex.europa.eu/eli/dir/2022/2555/oj (NIS2)
- https://eur-lex.europa.eu/eli/reg/2022/2554/oj (DORA)
- https://eur-lex.europa.eu/eli/reg/2024/1689/oj (EU AI Act)

Architektúraelmélet és könyvek

- https://www.sei.cmu.edu/library/software-architecture-in-practice-4th-edition/ (Software Architecture in Practice)
- https://www.sei.cmu.edu/library/documenting-software-architectures-views-and-beyond-2nd-edition/ (Documenting Software Architectures)
- https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/ (Fundamentals of Software Architecture)
- https://www.oreilly.com/library/view/software-architecture-the/9781492086888/ (Software Architecture: The Hard Parts)
- https://www.oreilly.com/library/view/building-evolutionary-architectures/9781492097532/ (Building Evolutionary Architectures)
- https://www.oreilly.com/library/view/the-software-architect/9781492077534/ (The Software Architect Elevator)
- https://www.oreilly.com/library/view/97-things-every/9780596800611/ (97 Things Every Software Architect Should Know)
- https://martinfowler.com/architecture/ (Martin Fowler: Architecture)
- https://martinfowler.com/eaaCatalog/ (Patterns of Enterprise Application Architecture)
- https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (Clean Architecture)
- https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ (Software Architecture Patterns)

Dokumentáció, nézetek és döntések

- https://c4model.com/ (C4 model)
- https://arc42.org/ (arc42)
- https://adr.github.io/ (Architecture Decision Records)
- https://structurizr.com/ (Structurizr)
- https://mermaid.js.org/ (Mermaid)
- https://plantuml.com/ (PlantUML)
- https://www.omg.org/spec/UML/ (UML specifikáció)
- https://www.sei.cmu.edu/library/quality-attribute-scenarios/ (Quality attribute scenarios)
- https://www.sei.cmu.edu/library/architecture-tradeoff-analysis-method-collection/ (ATAM)
- https://www.sei.cmu.edu/library/architecture-evaluation/ (Architecture evaluation)

Stílusok, minták és domain-architektúrák

- https://microservices.io/patterns/ (microservices.io mintakatalógus)
- https://martinfowler.com/articles/microservices.html (Microservices)
- https://martinfowler.com/bliki/PresentationDomainDataLayering.html (Layered architecture)
- https://www.enterpriseintegrationpatterns.com/ (Enterprise Integration Patterns)
- https://domainlanguage.com/ddd/ (Domain-Driven Design)
- https://www.oreilly.com/library/view/implementing-domain-driven-design/9780133039900/ (Implementing Domain-Driven Design)
- https://martinfowler.com/articles/data-mesh-principles.html (Data mesh)
- https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/ (CAP tétel)

- https://fundamentalsofsoftwarearchitecture.com/katas/ (Architecture Katas)

Integráció, események és API-k

- https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (Fielding: REST)
- https://spec.openapis.org/oas/latest.html (OpenAPI)
- https://www.asyncapi.com/docs (AsyncAPI)
- https://graphql.org/learn/ (GraphQL)
- https://grpc.io/docs/ (gRPC)
- https://martinfowler.com/bliki/CQRS.html (CQRS)
- https://martinfowler.com/eaaDev/EventSourcing.html (Event Sourcing)
- https://microservices.io/patterns/data/saga.html (Saga)
- https://microservices.io/patterns/data/transactional-outbox.html (Transactional outbox)
- https://kafka.apache.org/documentation/ (Apache Kafka)

Adat, cloud és platform

- https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/ (Designing Data-Intensive Applications)
- https://www.databricks.com/product/data-lakehouse (Lakehouse)
- https://12factor.net/ (The Twelve-Factor App)
- https://kubernetes.io/docs/ (Kubernetes)
- https://www.cncf.io/ (CNCF)
- https://istio.io/latest/docs/ (Istio)
- https://www.envoyproxy.io/docs (Envoy)
- https://developer.hashicorp.com/terraform/docs (Terraform)
- https://argo-cd.readthedocs.io/ (Argo CD)
- https://fluxcd.io/flux/ (Flux)
- https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html (AWS Well-Architected Framework)
- https://learn.microsoft.com/en-us/azure/architecture/ (Azure Architecture Center)
- https://cloud.google.com/architecture/framework (Google Cloud Architecture Framework)

Platform engineering, DevOps és tesztelés

- https://teamtopologies.com/ (Team Topologies)
- https://martinfowler.com/bliki/ConwaysLaw.html (Conway törvénye)
- https://dora.dev/ (DORA)
- https://itrevolution.com/product/accelerate/ (Accelerate)
- https://continuousdelivery.com/ (Continuous Delivery)
- https://martinfowler.com/articles/practical-test-pyramid.html (Practical Test Pyramid)
- https://docs.pact.io/ (Pact)
- https://stryker-mutator.io/ (Stryker mutation testing)
- https://docs.sonarsource.com/sonarqube/ (SonarQube)
- https://semgrep.dev/docs/ (Semgrep)
- https://k6.io/docs/ (k6)
- https://www.zaproxy.org/docs/ (OWASP ZAP)

Reziliencia, biztonság és üzemeltetés

- https://pragprog.com/titles/mnee2/release-it-second-edition/ (Release It!)
- https://principlesofchaos.org/ (Principles of Chaos Engineering)
- https://sre.google/sre-book/table-of-contents/ (Google SRE Book)
- https://sre.google/workbook/table-of-contents/ (The Site Reliability Workbook)
- https://aws.amazon.com/builders-library/ (Amazon Builders' Library)
- https://owasp.org/www-project-application-security-verification-standard/ (OWASP ASVS)
- https://owasp.org/www-project-threat-dragon/ (OWASP Threat Dragon)
- https://slsa.dev/ (SLSA)
- https://www.cisa.gov/sbom (SBOM)
- https://www.sigstore.dev/ (Sigstore)
- https://opentelemetry.io/docs/ (OpenTelemetry)
- https://prometheus.io/docs/ (Prometheus)
- https://grafana.com/docs/ (Grafana)
- https://www.jaegertracing.io/docs/ (Jaeger)
- https://www.finops.org/framework/ (FinOps Framework)
- https://greensoftware.foundation/ (Green Software Foundation)

Iparág, esettanulmányok és jövő

- https://www.pcisecuritystandards.org/ (PCI DSS)
- https://www.hhs.gov/hipaa/index.html (HIPAA)
- https://www.gs1.org/standards (GS1 szabványok)
- https://martinfowler.com/bliki/TechnicalDebt.html (Technical Debt)
- https://martinfowler.com/bliki/StranglerFigApplication.html (Strangler Fig)
- https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html (Migration strategies)
- https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/ (Building Microservices)
- https://www.oreilly.com/library/view/monolith-to-microservices/9781492047834/ (Monolith to Microservices)
- https://www.manning.com/books/cloud-native-patterns (Cloud Native Patterns)
- https://www.oreilly.com/library/view/kubernetes-patterns-2nd/9781098131678/ (Kubernetes Patterns)
- https://quantum.country/ (Quantum Country)

Magyar és oktatási források

- https://www.vik.bme.hu/ (BME Villamosmérnöki és Informatikai Kar)
- https://www.inf.elte.hu/ (ELTE Informatikai Kar)
- https://www.iit.bme.hu/ (BME Automatizálási és Alkalmazott Informatikai Tanszék)
- https://www.mit.bme.hu/ (BME Méréstechnika és Információs Rendszerek Tanszék)